If you got here from *Step 5. First Steps on ESP-IDF* when installing s/w for ESP32-S3 development, then you can continue with *Step 5. First Steps on ESP-IDF*.

Flashing Troubleshooting

Failed to Connect If you run the given command and see errors such as "Failed to connect", there might be several reasons for this. One of the reasons might be issues encountered by `esptool.py`, the utility that is called by the build system to reset the chip, interact with the ROM bootloader, and flash firmware. One simple solution to try is to manually reset as described below. If it does not help, you can find more details about possible issues in the esptool troubleshooting page.

`esptool.py` resets ESP32-S3 automatically by asserting DTR and RTS control lines of the USB-to-UART bridge, i.e., FTDI or CP210x (for more information, see *Establish Serial Connection with ESP32-S3*). The DTR and RTS control lines are in turn connected to `GPIO0` and `CHIP_PU` (EN) pins of ESP32-S3, thus changes in the voltage levels of DTR and RTS will boot ESP32-S3 into Firmware Download mode. As an example, check the schematic for the ESP32 DevKitC development board.

In general, you should have no problems with the official esp-idf development boards. However, `esptool.py` is not able to reset your hardware automatically in the following cases:

- Your hardware does not have the DTR and RTS lines connected to `GPIO0` and `CHIP_PU`.
- The DTR and RTS lines are configured differently.
- There are no such serial control lines at all.

Depending on the kind of hardware you have, it may also be possible to manually put your ESP32-S3 board into Firmware Download mode (reset).

- For development boards produced by Espressif, this information can be found in the respective getting started guides or user guides. For example, to manually reset an ESP-IDF development board, hold down the `Boot` button (`GPIO0`) and press the `EN` button (`CHIP_PU`).
- For other types of hardware, try pulling `GPIO0` down.

Standard Toolchain Setup for Linux and macOS

Installation Step by Step This is a detailed roadmap to walk you through the installation process.

Setting up Development Environment These are the steps for setting up the ESP-IDF for your ESP32-S3.

- *Step 1. Install Prerequisites*
- *Step 2. Get ESP-IDF*
- *Step 3. Set up the Tools*
- *Step 4. Set up the Environment Variables*
- *Step 5. First Steps on ESP-IDF*

Step 1. Install Prerequisites In order to use ESP-IDF with the ESP32-S3, you need to install some software packages based on your Operating System. This setup guide helps you on getting everything installed on Linux and macOS based systems.

For Linux Users To compile using ESP-IDF, you need to get the following packages. The command to run depends on which distribution of Linux you are using:

- Ubuntu and Debian:

```
sudo apt-get install git wget flex bison gperf python3 python3-pip python3-
↪venv cmake ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

- CentOS 7 & 8:

```
sudo yum -y update && sudo yum install git wget flex bison gperf python3 cmake␣
↪ninja-build ccache dfu-util libusbx
```

CentOS 7 is still supported but CentOS version 8 is recommended for a better user experience.

- Arch:

```
sudo pacman -S --needed gcc git make flex bison gperf python cmake ninja␣
↪ccache dfu-util libusb
```

Note:

- CMake version 3.16 or newer is required for use with ESP-IDF. Run "tools/idf_tools.py install cmake" to install a suitable version if your OS versions does not have one.
- If you do not see your Linux distribution in the above list then please check its documentation to find out which command to use for package installation.

For macOS Users ESP-IDF uses the version of Python installed by default on macOS.

- Install CMake & Ninja build:
  - If you have HomeBrew, you can run:

```
brew install cmake ninja dfu-util
```

  - If you have MacPorts, you can run:

```
sudo port install cmake ninja dfu-util
```

  - Otherwise, consult the CMake and Ninja home pages for macOS installation downloads.
- It is strongly recommended to also install ccache for faster builds. If you have HomeBrew, this can be done via `brew install ccache` or `sudo port install ccache` on MacPorts.

Note: If an error like this is shown during any step:

```
xcrun: error: invalid active developer path (/Library/Developer/CommandLineTools),␣
↪missing xcrun at: /Library/Developer/CommandLineTools/usr/bin/xcrun
```

Then you need to install the XCode command line tools to continue. You can install these by running `xcode-select --install`.

Apple M1 Users If you use Apple M1 platform and see an error like this:

```
WARNING: directory for tool xtensa-esp32-elf version esp-2021r2-patch3-8.4.0 is␣
↪present, but tool was not found
ERROR: tool xtensa-esp32-elf has no installed versions. Please run 'install.sh' to␣
↪install it.
```

or:

```
zsh: bad CPU type in executable: ~/.espressif/tools/xtensa-esp32-elf/esp-2021r2-
↪patch3-8.4.0/xtensa-esp32-elf/bin/xtensa-esp32-elf-gcc
```

Then you need to install Apple Rosetta 2 by running

```
/usr/sbin/softwareupdate --install-rosetta --agree-to-license
```

Installing Python 3 Based on macOS Catalina 10.15 release notes, use of Python 2.7 is not recommended and Python 2.7 is not included by default in future versions of macOS. Check what Python you currently have:

```
python --version
```

If the output is like `Python 2.7.17`, your default interpreter is Python 2.7. If so, also check if Python 3 is not already installed on your computer:

```
python3 --version
```

If the above command returns an error, it means Python 3 is not installed.

Below is an overview of the steps to install Python 3.

- Installing with HomeBrew can be done as follows:

  ```
  brew install python3
  ```

- If you have MacPorts, you can run:

  ```
  sudo port install python38
  ```

Step 2. Get ESP-IDF To build applications for the ESP32-S3, you need the software libraries provided by Espressif in ESP-IDF repository.

To get ESP-IDF, navigate to your installation directory and clone the repository with `git clone`, following instructions below specific to your operating system.

Open Terminal, and run the following commands:

```
mkdir -p ~/esp
cd ~/esp
git clone -b v5.3.1 --recursive https://github.com/espressif/esp-idf.git
```

ESP-IDF is downloaded into `~/esp/esp-idf`.

Consult *ESP-IDF Versions* for information about which ESP-IDF version to use in a given situation.

Step 3. Set up the Tools Aside from the ESP-IDF, you also need to install the tools used by ESP-IDF, such as the compiler, debugger, Python packages, etc, for projects supporting ESP32-S3.

```
cd ~/esp/esp-idf
./install.sh esp32s3
```

or with Fish shell

```
cd ~/esp/esp-idf
./install.fish esp32s3
```

The above commands install tools for ESP32-S3 only. If you intend to develop projects for more chip targets then you should list all of them and run for example:

```
cd ~/esp/esp-idf
./install.sh esp32,esp32s2
```

or with Fish shell

```
cd ~/esp/esp-idf
./install.fish esp32,esp32s2
```

In order to install tools for all supported targets please run the following command:

```
cd ~/esp/esp-idf
./install.sh all
```

or with Fish shell

```
cd ~/esp/esp-idf
./install.fish all
```

Note: For macOS users, if an error like this is shown during any step:

```
<urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: unable␣
↪to get local issuer certificate (_ssl.c:xxx)
```

You may run `Install Certificates.command` in the Python folder of your computer to install certificates. For details, see Download Error While Installing ESP-IDF Tools.

Alternative File Downloads The tools installer downloads a number of files attached to GitHub Releases. If accessing GitHub is slow then it is possible to set an environment variable to prefer Espressif's download server for GitHub asset downloads.

Note: This setting only controls individual tools downloaded from GitHub releases, it does not change the URLs used to access any Git repositories.

To prefer the Espressif download server when installing tools, use the following sequence of commands when running `install.sh`:

```
cd ~/esp/esp-idf
export IDF_GITHUB_ASSETS="dl.espressif.com/github_assets"
./install.sh
```

Note: For users in China, we recommend using our download server located in China for faster download speed.

```
cd ~/esp/esp-idf
export IDF_GITHUB_ASSETS="dl.espressif.cn/github_assets"
./install.sh
```

Customizing the Tools Installation Path The scripts introduced in this step install compilation tools required by ESP-IDF inside the user home directory: `$HOME/.espressif` on Linux. If you wish to install the tools into a different directory, **export the environment variable IDF_TOOLS_PATH before running the installation scripts**. Make sure that your user account has sufficient permissions to read and write this path.

```
export IDF_TOOLS_PATH="$HOME/required_idf_tools_path"
./install.sh

. ./export.sh
```

If changing the `IDF_TOOLS_PATH`, make sure it is exported in the environment before running any ESP-IDF tools or scripts.

Note: Using `IDF_TOOLS_PATH` in variable assignment, e.g., `IDF_TOOLS_PATH="$HOME/required_idf_tools_path" ./install.sh`, without prior exporting, will not work in most shells

because the variable assignment will not affect the current execution environment, even if it's exported/changed in the sourced script.

Step 4. Set up the Environment Variables The installed tools are not yet added to the PATH environment variable. To make the tools usable from the command line, some environment variables must be set. ESP-IDF provides another script which does that.

In the terminal where you are going to use ESP-IDF, run:

```
. $HOME/esp/esp-idf/export.sh
```

or for fish (supported only since fish version 3.0.0):

```
. $HOME/esp/esp-idf/export.fish
```

Note the space between the leading dot and the path!

If you plan to use esp-idf frequently, you can create an alias for executing `export.sh`:

1. Copy and paste the following command to your shell's profile (`.profile`, `.bashrc`, `.zprofile`, etc.)

   ```
   alias get_idf='. $HOME/esp/esp-idf/export.sh'
   ```

2. Refresh the configuration by restarting the terminal session or by running `source [path to profile]`, for example, `source ~/.bashrc`.

Now you can run `get_idf` to set up or refresh the esp-idf environment in any terminal session.

Technically, you can add `export.sh` to your shell's profile directly; however, it is not recommended. Doing so activates IDF virtual environment in every terminal session (including those where IDF is not needed), defeating the purpose of the virtual environment and likely affecting other software.

Step 5. First Steps on ESP-IDF Now since all requirements are met, the next topic will guide you on how to start your first project.

This guide helps you on the first steps using ESP-IDF. Follow this guide to start a new project on the ESP32-S3 and build, flash, and monitor the device output.

Note: If you have not yet installed ESP-IDF, please go to *Installation* and follow the instruction in order to get all the software needed to use this guide.

Start a Project Now you are ready to prepare your application for ESP32-S3. You can start with get-started/hello_world project from examples directory in ESP-IDF.

Important: The ESP-IDF build system does not support spaces in the paths to either ESP-IDF or to projects.

Copy the project get-started/hello_world to `~/esp` directory:

```
cd ~/esp
cp -r $IDF_PATH/examples/get-started/hello_world .
```

Note: There is a range of example projects in the examples directory in ESP-IDF. You can copy any project in the same way as presented above and run it. It is also possible to build examples in-place without copying them first.

Connect Your Device Now connect your ESP32-S3 board to the computer and check under which serial port the board is visible.

Serial ports have the following naming patterns:

- **Linux**: starting with `/dev/tty`
- **macOS**: starting with `/dev/cu.`

If you are not sure how to check the serial port name, please refer to *Establish Serial Connection with ESP32-S3* for full details.

Note: Keep the port name handy as it is needed in the next steps.

Configure Your Project Navigate to your `hello_world` directory, set ESP32-S3 as the target, and run the project configuration utility `menuconfig`.

```
cd ~/esp/hello_world
idf.py set-target esp32s3
idf.py menuconfig
```

After opening a new project, you should first set the target with `idf.py set-target esp32s3`. Note that existing builds and configurations in the project, if any, are cleared and initialized in this process. The target may be saved in the environment variable to skip this step at all. See *Select the Target Chip: set-target* for additional information.

If the previous steps have been done correctly, the following menu appears:

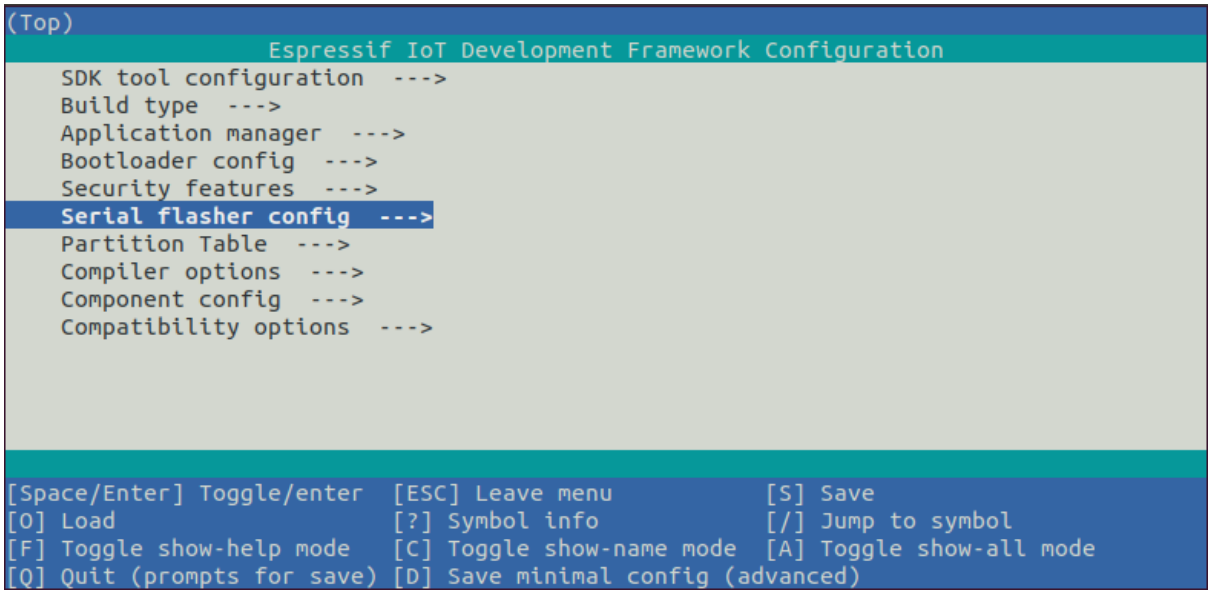


Fig. 25: Project configuration - Home window

You are using this menu to set up project specific variables, e.g., Wi-Fi network name and password, the processor speed, etc. Setting up the project with menuconfig may be skipped for "hello_world", since this example runs with default configuration.

Note: The colors of the menu could be different in your terminal. You can change the appearance with the option `--style`. Please run `idf.py menuconfig --help` for further information.

If you are using one of the supported development boards, you can speed up your development by using Board Support Package. See *Additional Tips* for more information.

Build the Project Build the project by running:

```
idf.py build
```

This command compiles the application and all ESP-IDF components, then it generates the bootloader, partition table, and application binaries.

```
$ idf.py build
Running cmake in directory /path/to/hello_world/build
Executing "cmake -G Ninja --warn-uninitialized /path/to/hello_world"...
Warn about uninitialized values.
-- Found Git: /usr/bin/git (found version "2.17.0")
-- Building empty aws_iot component due to configuration
-- Component names: ...
-- Component paths: ...

... (more lines of build system output)

[527/527] Generating hello_world.bin
esptool.py v2.3.1

Project build complete. To flash, run this command:
../../../components/esptool_py/esptool/esptool.py -p (PORT) -b 921600 write_flash -
↪-flash_mode dio --flash_size detect --flash_freq 40m 0x10000 build/hello_world.
↪bin  build 0x1000 build/bootloader/bootloader.bin 0x8000 build/partition_table/
↪partition-table.bin
or run 'idf.py -p PORT flash'
```

If there are no errors, the build finishes by generating the firmware binary .bin files.

Flash onto the Device To flash the binaries that you just built for the ESP32-S3 in the previous step, you need to run the following command:

```
idf.py -p PORT flash
```

Replace `PORT` with your ESP32-S3 board's USB port name. If the `PORT` is not defined, the *idf.py* will try to connect automatically using the available USB ports.

For more information on `idf.py` arguments, see *idf.py*.

Note: The option `flash` automatically builds and flashes the project, so running `idf.py build` is not necessary.

Encountered Issues While Flashing? See the "Additional Tips" below. You can also refer to *Flashing Troubleshooting* page or *Establish Serial Connection with ESP32-S3* for more detailed information.

Normal Operation When flashing, you will see the output log similar to the following:

```
...
esptool.py esp32s3 -p /dev/ttyUSB0 -b 460800 --before=default_reset --after=hard_
↪reset write_flash --flash_mode dio --flash_freq 80m --flash_size 2MB 0x0␣
↪bootloader/bootloader.bin 0x10000 hello_world.bin 0x8000 partition_table/
↪partition-table.bin
esptool.py v3.2-dev
Serial port /dev/ttyUSB0
Connecting....
Chip is ESP32-S3
Features: WiFi, BLE
Crystal is 40MHz
MAC: 7c:df:a1:e0:00:64
```

(continues on next page)

(continued from previous page)

```
Uploading stub...
Running stub...
Stub running...
Changing baud rate to 460800
Changed.
Configuring flash size...
Flash will be erased from 0x00000000 to 0x00004fff...
Flash will be erased from 0x00010000 to 0x00039fff...
Flash will be erased from 0x00008000 to 0x00008fff...
Compressed 18896 bytes to 11758...
Writing at 0x00000000... (100 %)
Wrote 18896 bytes (11758 compressed) at 0x00000000 in 0.5 seconds (effective 279.9␣
↪kbit/s)...
Hash of data verified.
Compressed 168208 bytes to 88178...
Writing at 0x00010000... (16 %)
Writing at 0x0001a80f... (33 %)
Writing at 0x000201f1... (50 %)
Writing at 0x00025dcf... (66 %)
Writing at 0x0002d0be... (83 %)
Writing at 0x00036c07... (100 %)
Wrote 168208 bytes (88178 compressed) at 0x00010000 in 2.4 seconds (effective 569.
↪2 kbit/s)...
Hash of data verified.
Compressed 3072 bytes to 103...
Writing at 0x00008000... (100 %)
Wrote 3072 bytes (103 compressed) at 0x00008000 in 0.1 seconds (effective 478.9␣
↪kbit/s)...
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
Done
```

If there are no issues by the end of the flash process, the board will reboot and start up the "hello_world" application.

If you would like to use the Eclipse or VS Code IDE instead of running `idf.py`, check out Eclipse Plugin, VSCode Extension.

Monitor the Output To check if "hello_world" is indeed running, type `idf.py -p PORT monitor` (Do not forget to replace PORT with your serial port name).

This command launches the *IDF Monitor* application.

```
$ idf.py -p <PORT> monitor
Running idf_monitor in directory [...]/esp/hello_world/build
Executing "python [...]/esp-idf/tools/idf_monitor.py -b 115200 [...]/esp/hello_
↪world/build/hello_world.elf"...
--- idf_monitor on <PORT> 115200 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
ets Jun  8 2016 00:22:57

rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
ets Jun  8 2016 00:22:57
...
```

After startup and diagnostic logs scroll up, you should see "Hello world!" printed out by the application.

```
...
Hello world!
Restarting in 10 seconds...
```

(continues on next page)

(continued from previous page)

```
    This is esp32s3 chip with 2 CPU core(s), WiFi/BLE, silicon revision 0, 2 MB␣
↪embedded flash
Minimum free heap size: 390684 bytes
    Restarting in 9 seconds...
    Restarting in 8 seconds...
    Restarting in 7 seconds...
```

To exit IDF monitor use the shortcut `Ctrl+]`.

Note: You can combine building, flashing and monitoring into one step by running:

```
idf.py -p PORT flash monitor
```

See also:

- *IDF Monitor* for handy shortcuts and more details on using IDF monitor.
- *idf.py* for a full reference of `idf.py` commands and options.

That is all that you need to get started with ESP32-S3!

Now you are ready to try some other examples, or go straight to developing your own applications.

Important: Some of examples do not support ESP32-S3 because required hardware is not included in ESP32-S3 so it cannot be supported.

If building an example, please check the README file for the `Supported Targets` table. If this is present including ESP32-S3 target, or the table does not exist at all, the example will work on ESP32-S3.

Additional Tips

Permission Denied Issue With some Linux distributions, you may get the error message similar to `Could not open port <PORT>: Permission denied: '<PORT>'` when flashing the ESP32-S3. *This can be solved by adding the current user to the specific group*, such as `dialout` or `uucp` group.

Python Compatibility ESP-IDF supports Python 3.8 or newer. It is recommended to upgrade your operating system to a recent version satisfying this requirement. Other options include the installation of Python from sources or the use of a Python version management system such as pyenv.

Start with Board Support Package To speed up prototyping on some development boards, you can use Board Support Packages (BSPs), which makes initialization of a particular board as easy as few function calls.

A BSP typically supports all of the hardware components provided on development board. Apart from the pinout definition and initialization functions, a BSP ships with drivers for the external components such as sensors, displays, audio codecs etc.

The BSPs are distributed via *IDF Component Manager*, so they can be found in IDF Component Registry.

Here is an example of how to add ESP-BOX BSP to your project:

```
idf.py add-dependency esp-box
```

More examples of BSP usage can be found in BSP examples folder.

Flash Erase Erasing the flash is also possible. To erase the entire flash memory you can run the following command:

```
idf.py -p PORT erase-flash
```

For erasing the OTA data, if present, you can run this command:

```
idf.py -p PORT erase-otadata
```

The flash erase command can take a while to be done. Do not disconnect your device while the flash erasing is in progress.

Tip: Updating ESP-IDF It is recommended to update ESP-IDF from time to time, as newer versions fix bugs and/or provide new features. Please note that each ESP-IDF major and minor release version has an associated support period, and when one release branch is approaching end of life (EOL), all users are encouraged to upgrade their projects to more recent ESP-IDF releases, to find out more about support periods, see *ESP-IDF Versions*.

The simplest way to do the update is to delete the existing `esp-idf` folder and clone it again, as if performing the initial installation described in *Step 2. Get ESP-IDF*.

Another solution is to update only what has changed. *The update procedure depends on the version of ESP-IDF you are using*.

After updating ESP-IDF, execute the Install script again, in case the new ESP-IDF version requires different versions of tools. See instructions at *Step 3. Set up the Tools*.

Once the new tools are installed, update the environment using the Export script. See instructions at *Step 4. Set up the Environment Variables*.

Related Documents

- *Establish Serial Connection with ESP32-S3*
- Eclipse Plugin
- VSCode Extension
- *IDF Monitor*

1.4 Build Your First Project

If you already have the ESP-IDF installed and are not using an IDE, you can build your first project from the command line following the *Start a Project on Windows* or *Start a Project on Linux and macOS*.

1.5 Uninstall ESP-IDF

If you want to remove ESP-IDF, please follow *Uninstall ESP-IDF*.

Chapter 2

API Reference

2.1 API Conventions

This document describes conventions and assumptions common to ESP-IDF Application Programming Interfaces (APIs).

ESP-IDF provides several kinds of programming interfaces:

- C functions, structures, enums, type definitions, and preprocessor macros declared in public header files of ESP-IDF components. Various pages in the API Reference section of the programming guide contain descriptions of these functions, structures, and types.
- Build system functions, predefined variables, and options. These are documented in the *ESP-IDF CMake Build System API*.
- *Kconfig* options can be used in code and in the build system (`CMakeLists.txt`) files.
- *Host tools* and their command line parameters are also part of the ESP-IDF interfaces.

ESP-IDF is made up of multiple components where these components either contain code specifically written for ESP chips, or contain a third-party library (i.e., a third-party component). In some cases, third-party components contain an "ESP-IDF specific" wrapper in order to provide an interface that is either simpler or better integrated with the rest of ESP-IDF's features. In other cases, third-party components present the original API of the underlying library directly.

The following sections explain some of the aspects of ESP-IDF APIs and their usage.

2.1.1 Error Handling

Most ESP-IDF APIs return error codes defined with the `esp_err_t` type. See *Error Handling* section for more information about error handling approaches. *Error Codes Reference* contains the list of error codes returned by ESP-IDF components.

2.1.2 Configuration Structures

Important: Correct initialization of configuration structures is an important part of making the application compatible with future versions of ESP-IDF.

Most initialization, configuration, and installation functions in ESP-IDF (typically named `...init()`, `...config()`, and `...install()`) take a configuration structure pointer as an argument. For example:

```
const esp_timer_create_args_t my_timer_args = {
    .callback = &my_timer_callback,
    .arg = callback_arg,
    .name = "my_timer"
};
esp_timer_handle_t my_timer;
esp_err_t err = esp_timer_create(&my_timer_args, &my_timer);
```

These functions never store the pointer to the configuration structure, so it is safe to allocate the structure on the stack.

The application must initialize all fields of the structure. The following is incorrect:

```
esp_timer_create_args_t my_timer_args;
my_timer_args.callback = &my_timer_callback;
/* Incorrect! Fields .arg and .name are not initialized */
esp_timer_create(&my_timer_args, &my_timer);
```

Most ESP-IDF examples use C99 designated initializers for structure initialization since they provide a concise way of setting a subset of fields, and zero-initializing the remaining fields:

```
const esp_timer_create_args_t my_timer_args = {
    .callback = &my_timer_callback,
    /* Correct, fields .arg and .name are zero-initialized */
};
```

The C++ language supports designated initializer syntax, too, but the initializers must be in the order of declaration. When using ESP-IDF APIs in C++ code, you may consider using the following pattern:

```
/* Correct, fields .dispatch_method, .name and .skip_unhandled_events are zero-
↪initialized */
const esp_timer_create_args_t my_timer_args = {
    .callback = &my_timer_callback,
    .arg = &my_arg,
};

///* Incorrect, .arg is declared after .callback in esp_timer_create_args_t */
//const esp_timer_create_args_t my_timer_args = {
//    .arg = &my_arg,
//    .callback = &my_timer_callback,
//};
```

For more information on designated initializers, see *Designated Initializers*. Note that C++ language versions older than C++20, which are not the default in the current version of ESP-IDF, do not support designated initializers. If you have to compile code with an older C++ standard than C++20, you may use GCC extensions to produce the following pattern:

```
esp_timer_create_args_t my_timer_args = {};
/* All the fields are zero-initialized */
my_timer_args.callback = &my_timer_callback;
```

Default Initializers

For some configuration structures, ESP-IDF provides macros for setting default values of fields:

```
httpd_config_t config = HTTPD_DEFAULT_CONFIG();
/* HTTPD_DEFAULT_CONFIG expands to a designated initializer. Now all fields are␣
↪set to the default values, and any field can still be modified: */
config.server_port = 8081;
```

(continues on next page)

(continued from previous page)

```
httpd_handle_t server;
esp_err_t err = httpd_start(&server, &config);
```

It is recommended to use default initializer macros whenever they are provided for a particular configuration structure.

2.1.3 Private APIs

Certain header files in ESP-IDF contain APIs intended to be used only in ESP-IDF source code rather than by the applications. Such header files often contain `private` or `esp_private` in their name or path. Certain components, such as *hal* only contain private APIs.

Private APIs may be removed or changed in an incompatible way between minor or patch releases.

2.1.4 Components in Example Projects

ESP-IDF examples contain a variety of projects demonstrating the usage of ESP-IDF APIs. In order to reduce code duplication in the examples, a few common helpers are defined inside components that are used by multiple examples. This includes components located in common_components directory, as well as some of the components located in the examples themselves. These components are not considered to be part of the ESP-IDF API.

It is not recommended to reference these components directly in custom projects (via `EXTRA_COMPONENT_DIRS` build system variable), as they may change significantly between ESP-IDF versions. When starting a new project based on an ESP-IDF example, copy both the project and the common components it depends on out of ESP-IDF, and treat the common components as part of the project. Note that the common components are written with examples in mind, and might not include all the error handling required for production applications. Before using, take time to read the code and understand if it is applicable to your use case.

2.1.5 API Stability

ESP-IDF uses Semantic Versioning as explained in the *Versioning Scheme*.

Minor and bugfix releases of ESP-IDF guarantee compatibility with previous releases. The sections below explain different aspects and limitations to compatibility.

Source-level Compatibility

ESP-IDF guarantees source-level compatibility of C functions, structures, enums, type definitions, and preprocessor macros declared in public header files of ESP-IDF components. Source-level compatibility implies that the application source code can be recompiled with the newer version of ESP-IDF without changes.

The following changes are allowed between minor versions and do not break source-level compatibility:

- Deprecating functions (using the `deprecated` attribute) and header files (using a preprocessor `#warning`). Deprecations are listed in ESP-IDF release notes. It is recommended to update the source code to use the newer functions or files that replace the deprecated ones, however, this is not mandatory. Deprecated functions and files can be removed from major versions of ESP-IDF.
- Renaming components, moving source and header files between components —provided that the build system ensures that correct files are still found.
- Renaming Kconfig options. Kconfig system's *backward compatibility* ensures that the original Kconfig option names can still be used by the application in `sdkconfig` file, CMake files, and source code.

Lack of Binary Compatibility

ESP-IDF does not guarantee binary compatibility between releases. This means that if a precompiled library is built with one ESP-IDF version, it is not guaranteed to work the same way with the next minor or bugfix release. The following are the possible changes that keep source-level compatibility but not binary compatibility:

- Changing numerical values for C enum members.
- Adding new structure members or changing the order of members. See *Configuration Structures* for tips that help ensure compatibility.
- Replacing an `extern` function with a `static inline` one with the same signature, or vice versa.
- Replacing a function-like macro with a compatible C function.

Other Exceptions from Compatibility

While we try to make upgrading to a new ESP-IDF version easy, there are parts of ESP-IDF that may change between minor versions in an incompatible way. We appreciate issuing reports about any unintended breaking changes that do not fall into the categories below.

- *Private APIs*.
- *Components in Example Projects*.
- Features clearly marked as "beta", "preview", or "experimental".
- Changes made to mitigate security issues or to replace insecure default behaviors with secure ones.
- Features that were never functional. For example, if it was never possible to use a certain function or an enumeration value, it may get renamed (as part of fixing it) or removed. This includes software features that depend on non-functional chip hardware features.
- Unexpected or undefined behavior that is not documented explicitly may be fixed/changed, such as due to missing validation of argument ranges.
- Location of *Kconfig* options in menuconfig.
- Location and names of example projects.

2.2 Application Protocols

2.2.1 ASIO Port

ASIO is a cross-platform C++ library, see https://think-async.com/Asio/. It provides a consistent asynchronous model using a modern C++ approach.

The ESP-IDF component `ASIO` has been moved from ESP-IDF since version v5.0 to a separate repository:

- ASIO component on GitHub

To add ASIO component in your project, please run `idf.py add-dependency espressif/asio`.

Hosted Documentation

The documentation can be found on the link below:

- ASIO documentation (English)

2.2.2 ESP-Modbus

The Espressif ESP-Modbus Library (esp-modbus) supports Modbus communication in the networks based on RS485, Wi-Fi, and Ethernet interfaces. Since ESP-IDF version v5.0, the component `freemodbus` has been moved from ESP-IDF to a separate repository:

- ESP-Modbus component on GitHub

Hosted Documentation

The documentation can be found through the link below:

- ESP-Modbus documentation (English)

Application Example

The examples below demonstrate the ESP-Modbus library of serial and TCP ports for both slave and master implementations respectively.

- protocols/modbus/serial/mb_slave
- protocols/modbus/serial/mb_master
- protocols/modbus/tcp/mb_tcp_slave
- protocols/modbus/tcp/mb_tcp_master

Please refer to the `README.md` documents of each specific example for details.

Protocol References

- For the detailed protocol specifications, see The Modbus Organization.

2.2.3 ESP-MQTT

Overview

ESP-MQTT is an implementation of MQTT protocol client, which is a lightweight publish/subscribe messaging protocol. Now ESP-MQTT supports MQTT v5.0.

Features

- Support MQTT over TCP, SSL with Mbed TLS, MQTT over WebSocket, and MQTT over WebSocket Secure
- Easy to setup with URI
- Multiple instances (multiple clients in one application)
- Support subscribing, publishing, authentication, last will messages, keep alive pings, and all 3 Quality of Service (QoS) levels (it should be a fully functional client)

Application Examples

- protocols/mqtt/tcp: MQTT over TCP, default port 1883
- protocols/mqtt/ssl: MQTT over TLS, default port 8883
- protocols/mqtt/ssl_ds: MQTT over TLS using digital signature peripheral for authentication, default port 8883
- protocols/mqtt/ssl_mutual_auth: MQTT over TLS using certificates for authentication, default port 8883

- protocols/mqtt/ssl_psk: MQTT over TLS using pre-shared keys for authentication, default port 8883
- protocols/mqtt/ws: MQTT over WebSocket, default port 80
- protocols/mqtt/wss: MQTT over WebSocket Secure, default port 443
- protocols/mqtt5: Uses ESP-MQTT library to connect to broker with MQTT v5.0

MQTT Message Retransmission

A new MQTT message is created by calling *esp_mqtt_client_publish* or its non blocking counterpart *esp_mqtt_client_enqueue*.

Messages with QoS 0 is sent only once. QoS 1 and 2 have different behaviors since the protocol requires extra steps to complete the process.

The ESP-MQTT library opts to always retransmit unacknowledged QoS 1 and 2 publish messages to avoid losses in faulty connections, even though the MQTT specification requires the re-transmission only on reconnect with Clean Session flag been set to 0 (set *disable_clean_session* to true for this behavior).

QoS 1 and 2 messages that may need retransmission are always enqueued, but first transmission try occurs immediately if *esp_mqtt_client_publish* is used. A transmission retry for unacknowledged messages will occur after *message_retransmit_timeout*. After *CONFIG_MQTT_OUTBOX_EXPIRED_TIMEOUT_MS* messages will expire and be deleted. If *CONFIG_MQTT_REPORT_DELETED_MESSAGES* is set, an event will be sent to notify the user.

Configuration

The configuration is made by setting fields in *esp_mqtt_client_config_t* struct. The configuration struct has the following sub structs to configure different aspects of the client operation.

- *esp_mqtt_client_config_t::broker_t* - Allow to set address and security verification.
- *esp_mqtt_client_config_t::credentials_t* - Client credentials for authentication.
- *esp_mqtt_client_config_t::session_t* - Configuration for MQTT session aspects.
- *esp_mqtt_client_config_t::network_t* - Networking related configuration.
- *esp_mqtt_client_config_t::task_t* - Allow to configure FreeRTOS task.
- *esp_mqtt_client_config_t::buffer_t* - Buffer size for input and output.

In the following sections, the most common aspects are detailed.

Broker

Address Broker address can be set by usage of *address* struct. The configuration can be made by usage of *uri* field or the combination of *hostname*, *transport* and *port*. Optionally, *path* could be set, this field is useful in WebSocket connections.

The *uri* field is used in the format `scheme://hostname:port/path`.

- Currently support `mqtt`, `mqtts`, `ws`, `wss` schemes
- MQTT over TCP samples:
  - `mqtt://mqtt.eclipseprojects.io`: MQTT over TCP, default port 1883
  - `mqtt://mqtt.eclipseprojects.io:1884`: MQTT over TCP, port 1884
  - `mqtt://username:password@mqtt.eclipseprojects.io:1884`: MQTT over TCP, port 1884, with username and password
- MQTT over SSL samples:
  - `mqtts://mqtt.eclipseprojects.io`: MQTT over SSL, port 8883
  - `mqtts://mqtt.eclipseprojects.io:8884`: MQTT over SSL, port 8884
- MQTT over WebSocket samples:
  - `ws://mqtt.eclipseprojects.io:80/mqtt`
- MQTT over WebSocket Secure samples:
  - `wss://mqtt.eclipseprojects.io:443/mqtt`
- Minimal configurations:

```
const esp_mqtt_client_config_t mqtt_cfg = {
    .broker.address.uri = "mqtt://mqtt.eclipseprojects.io",
};
esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler,␣
↪client);
esp_mqtt_client_start(client);
```

Note: By default MQTT client uses event loop library to post related MQTT events (connected, subscribed, published, etc.).

Verification For secure connections with TLS used, and to guarantee Broker's identity, the *verification* struct must be set. The broker certificate may be set in PEM or DER format. To select DER, the equivalent *certificate_len* field must be set. Otherwise, a null-terminated string in PEM format should be provided to *certificate* field.

- **Get certificate from server, example: `mqtt.eclipseprojects.io`**

```
openssl s_client -showcerts -connect mqtt.eclipseprojects.io:8883 < /dev/
↪null \
2> /dev/null | openssl x509 -outform PEM > mqtt_eclipse_org.pem
```

- Check the sample application: protocols/mqtt/ssl
- Configuration:

```
const esp_mqtt_client_config_t mqtt_cfg = {
    .broker = {
      .address.uri = "mqtts://mqtt.eclipseprojects.io:8883",
      .verification.certificate = (const char *)mqtt_eclipse_org_pem_start,
    },
};
```

For details about other fields, please check the *API Reference* and *TLS Server Verification*.

Client Credentials All client related credentials are under the *credentials* field.

- *username*: pointer to the username used for connecting to the broker, can also be set by URI
- *client_id*: pointer to the client ID, defaults to `ESP32_%CHIPID%` where `%CHIPID%` are the last 3 bytes of MAC address in hex format

Authentication It is possible to set authentication parameters through the *authentication* field. The client supports the following authentication methods:

- *password*: use a password by setting
- *certificate* and *key*: mutual authentication with TLS, and both can be provided in PEM or DER format
- *use_secure_element*: use secure element (ATECC608A) interfaced to ESP32
- *ds_data*: use Digital Signature Peripheral available in some Espressif devices

Session For MQTT session-related configurations, *session* fields should be used.

Last Will and Testament MQTT allows for a last will and testament (LWT) message to notify other clients when a client ungracefully disconnects. This is configured by the following fields in the *last_will* struct.

- *topic*: pointer to the LWT message topic
- *msg*: pointer to the LWT message
- *msg_len*: length of the LWT message, required if *msg* is not null-terminated

- *qos*: quality of service for the LWT message
- *retain*: specifies the retain flag of the LWT message

Change Settings in Project Configuration Menu The settings for MQTT can be found using `idf.py menuconfig`, under `Component config` > `ESP-MQTT Configuration`.

The following settings are available:

- *CONFIG_MQTT_PROTOCOL_311*: enable 3.1.1 version of MQTT protocol
- *CONFIG_MQTT_TRANSPORT_SSL* and *CONFIG_MQTT_TRANSPORT_WEBSOCKET*: enable specific MQTT transport layer, such as SSL, WEBSOCKET, and WEBSOCKET_SECURE
- *CONFIG_MQTT_CUSTOM_OUTBOX*: disable default implementation of mqtt_outbox, so a specific implementation can be supplied

Events

The following events may be posted by the MQTT client:

- `MQTT_EVENT_BEFORE_CONNECT`: The client is initialized and about to start connecting to the broker.
- `MQTT_EVENT_CONNECTED`: The client has successfully established a connection to the broker. The client is now ready to send and receive data.
- `MQTT_EVENT_DISCONNECTED`: The client has aborted the connection due to being unable to read or write data, e.g., because the server is unavailable.
- `MQTT_EVENT_SUBSCRIBED`: The broker has acknowledged the client's subscribe request. The event data contains the message ID of the subscribe message.
- `MQTT_EVENT_UNSUBSCRIBED`: The broker has acknowledged the client's unsubscribe request. The event data contains the message ID of the unsubscribe message.
- `MQTT_EVENT_PUBLISHED`: The broker has acknowledged the client's publish message. This is only posted for QoS level 1 and 2, as level 0 does not use acknowledgements. The event data contains the message ID of the publish message.
- `MQTT_EVENT_DATA`: The client has received a publish message. The event data contains: message ID, name of the topic it was published to, received data and its length. For data that exceeds the internal buffer, multiple `MQTT_EVENT_DATA` events are posted and *current_data_offset* and *total_data_len* from event data updated to keep track of the fragmented message.
- `MQTT_EVENT_ERROR`: The client has encountered an error. The field *error_handle* in the event data contains *error_type* that can be used to identify the error. The type of error determines which parts of the *error_handle* struct is filled.

API Reference

Header File

- components/mqtt/esp-mqtt/include/mqtt_client.h
- This header file can be included with:

```
#include "mqtt_client.h"
```

- This header file is a part of the API provided by the `mqtt` component. To declare that your component depends on `mqtt`, add the following to your CMakeLists.txt:

```
REQUIRES mqtt
```

or

```
PRIV_REQUIRES mqtt
```

Functions

esp_mqtt_client_handle_t **esp_mqtt_client_init** (const *esp_mqtt_client_config_t* *config)

Creates *MQTT* client handle based on the configuration.

Parameters **config** -- *MQTT* configuration structure
Returns mqtt_client_handle if successfully created, NULL on error

esp_err_t **esp_mqtt_client_set_uri** (*esp_mqtt_client_handle_t* client, const char *uri)

Sets *MQTT* connection URI. This API is usually used to overrides the URI configured in esp_mqtt_client_init.

Parameters
- **client** -- *MQTT* client handle
- **uri** --

Returns ESP_FAIL if URI parse error, ESP_OK on success

esp_err_t **esp_mqtt_client_start** (*esp_mqtt_client_handle_t* client)

Starts *MQTT* client with already created client handle.

Parameters **client** -- *MQTT* client handle
Returns ESP_OK on success ESP_ERR_INVALID_ARG on wrong initialization ESP_FAIL on other error

esp_err_t **esp_mqtt_client_reconnect** (*esp_mqtt_client_handle_t* client)

This api is typically used to force reconnection upon a specific event.

Parameters **client** -- *MQTT* client handle
Returns ESP_OK on success ESP_ERR_INVALID_ARG on wrong initialization ESP_FAIL if client is in invalid state

esp_err_t **esp_mqtt_client_disconnect** (*esp_mqtt_client_handle_t* client)

This api is typically used to force disconnection from the broker.

Parameters **client** -- *MQTT* client handle
Returns ESP_OK on success ESP_ERR_INVALID_ARG on wrong initialization

esp_err_t **esp_mqtt_client_stop** (*esp_mqtt_client_handle_t* client)

Stops *MQTT* client tasks.

- Notes:
- Cannot be called from the *MQTT* event handler

Parameters **client** -- *MQTT* client handle
Returns ESP_OK on success ESP_ERR_INVALID_ARG on wrong initialization ESP_FAIL if client is in invalid state

int **esp_mqtt_client_subscribe_single** (*esp_mqtt_client_handle_t* client, const char *topic, int qos)

Subscribe the client to defined topic with defined qos.

Notes:

- Client must be connected to send subscribe message
- This API is could be executed from a user task or from a *MQTT* event callback i.e. internal *MQTT* task (API is protected by internal mutex, so it might block if a longer data receive operation is in progress.
- `esp_mqtt_client_subscribe` could be used to call this function.

Parameters
- **client** -- *MQTT* client handle
- **topic** -- topic filter to subscribe
- **qos** -- Max qos level of the subscription

Returns message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

int **esp_mqtt_client_subscribe_multiple**(*esp_mqtt_client_handle_t* client, const *esp_mqtt_topic_t* *topic_list, int size)

Subscribe the client to a list of defined topics with defined qos.

Notes:

- Client must be connected to send subscribe message
- This API is could be executed from a user task or from a *MQTT* event callback i.e. internal *MQTT* task (API is protected by internal mutex, so it might block if a longer data receive operation is in progress.
- `esp_mqtt_client_subscribe` could be used to call this function.

Parameters
- **client** -- *MQTT* client handle
- **topic_list** -- List of topics to subscribe
- **size** -- size of topic_list

Returns message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

int **esp_mqtt_client_unsubscribe**(*esp_mqtt_client_handle_t* client, const char *topic)

Unsubscribe the client from defined topic.

Notes:

- Client must be connected to send unsubscribe message
- It is thread safe, please refer to `esp_mqtt_client_subscribe_single` for details

Parameters
- **client** -- *MQTT* client handle
- **topic** --

Returns message_id of the subscribe message on success -1 on failure

int **esp_mqtt_client_publish**(*esp_mqtt_client_handle_t* client, const char *topic, const char *data, int len, int qos, int retain)

Client to send a publish message to the broker.

Notes:

- This API might block for several seconds, either due to network timeout (10s) or if publishing payloads longer than internal buffer (due to message fragmentation)
- Client doesn't have to be connected for this API to work, enqueueing the messages with qos>1 (returning -1 for all the qos=0 messages if disconnected). If MQTT_SKIP_PUBLISH_IF_DISCONNECTED is enabled, this API will not attempt to publish when the client is not connected and will always return -1.
- It is thread safe, please refer to `esp_mqtt_client_subscribe` for details

Parameters
- **client** -- *MQTT* client handle
- **topic** -- topic string
- **data** -- payload string (set to NULL, sending empty payload message)
- **len** -- data length, if set to 0, length is calculated from payload string
- **qos** -- QoS of publish message
- **retain** -- retain flag

Returns message_id of the publish message (for QoS 0 message_id will always be zero) on success. -1 on failure, -2 in case of full outbox.

int **esp_mqtt_client_enqueue**(*esp_mqtt_client_handle_t* client, const char *topic, const char *data, int len, int qos, int retain, bool store)

Enqueue a message to the outbox, to be sent later. Typically used for messages with qos>0, but could be also used for qos=0 messages if store=true.

This API generates and stores the publish message into the internal outbox and the actual sending to the network is performed in the mqtt-task context (in contrast to the esp_mqtt_client_publish() which sends the publish message immediately in the user task's context). Thus, it could be used as a non blocking version of esp_mqtt_client_publish().

Parameters
- **client** -- *MQTT* client handle
- **topic** -- topic string
- **data** -- payload string (set to NULL, sending empty payload message)
- **len** -- data length, if set to 0, length is calculated from payload string
- **qos** -- QoS of publish message
- **retain** -- retain flag
- **store** -- if true, all messages are enqueued; otherwise only QoS 1 and QoS 2 are enqueued

Returns message_id if queued successfully, -1 on failure, -2 in case of full outbox.

esp_err_t **esp_mqtt_client_destroy**(*esp_mqtt_client_handle_t* client)

Destroys the client handle.

Notes:

- Cannot be called from the *MQTT* event handler

Parameters **client** -- *MQTT* client handle
Returns ESP_OK ESP_ERR_INVALID_ARG on wrong initialization

esp_err_t **esp_mqtt_set_config**(*esp_mqtt_client_handle_t* client, const *esp_mqtt_client_config_t* *config)

Set configuration structure, typically used when updating the config (i.e. on "before_connect" event.

Notes:

- When calling this function make sure to have all the intendend configurations set, otherwise default values are set.

Parameters
- **client** -- *MQTT* client handle
- **config** -- *MQTT* configuration structure

Returns ESP_ERR_NO_MEM if failed to allocate ESP_ERR_INVALID_ARG if conflicts on transport configuration. ESP_OK on success

esp_err_t **esp_mqtt_client_register_event**(*esp_mqtt_client_handle_t* client, *esp_mqtt_event_id_t* event, *esp_event_handler_t* event_handler, void *event_handler_arg)

Registers *MQTT* event.

Parameters
- **client** -- *MQTT* client handle
- **event** -- event type
- **event_handler** -- handler callback
- **event_handler_arg** -- handlers context

Returns ESP_ERR_NO_MEM if failed to allocate ESP_ERR_INVALID_ARG on wrong initialization ESP_OK on success

esp_err_t **esp_mqtt_client_unregister_event**(*esp_mqtt_client_handle_t* client, *esp_mqtt_event_id_t* event, *esp_event_handler_t* event_handler)

Unregisters mqtt event.

Parameters
- **client** -- mqtt client handle
- **event** -- event ID
- **event_handler** -- handler to unregister

Returns ESP_ERR_NO_MEM if failed to allocate ESP_ERR_INVALID_ARG on invalid event ID ESP_OK on success

int **esp_mqtt_client_get_outbox_size**(*esp_mqtt_client_handle_t* client)

Get outbox size.

Parameters **client** -- *MQTT* client handle
Returns outbox size 0 on wrong initialization

esp_err_t **esp_mqtt_dispatch_custom_event**(*esp_mqtt_client_handle_t* client, *esp_mqtt_event_t* *event)

Dispatch user event to the mqtt internal event loop.

Parameters
- **client** -- *MQTT* client handle
- **event** -- *MQTT* event handle structure

Returns ESP_OK on success ESP_ERR_TIMEOUT if the event couldn't be queued (ref also CONFIG_MQTT_EVENT_QUEUE_SIZE)

Structures

struct **esp_mqtt_error_codes**

MQTT error code structure to be passed as a contextual information into ERROR event

Important: This structure extends `esp_tls_last_error` error structure and is backward compatible with it (so might be down-casted and treated as `esp_tls_last_error` error, but recommended to update applications if used this way previously)

Use this structure directly checking error_type first and then appropriate error code depending on the source of the error:

| error_type | related member variables | note | | MQTT_ERROR_TYPE_TCP_TRANSPORT | esp_tls_last_esp_err, esp_tls_stack_err, esp_tls_cert_verify_flags, sock_errno | Error reported from tcp_transport/esp-tls | | MQTT_ERROR_TYPE_CONNECTION_REFUSED | connect_return_code | Internal error reported from *MQTT* broker on connection |

Public Members

esp_err_t **esp_tls_last_esp_err**

last esp_err code reported from esp-tls component

int **esp_tls_stack_err**

tls specific error code reported from underlying tls stack

int **esp_tls_cert_verify_flags**

tls flags reported from underlying tls stack during certificate verification

esp_mqtt_error_type_t **error_type**

error type referring to the source of the error

esp_mqtt_connect_return_code_t **connect_return_code**

connection refused error code reported from MQTT* broker on connection

int **esp_transport_sock_errno**

errno from the underlying socket

struct **esp_mqtt_event_t**

MQTT event configuration structure

Public Members

esp_mqtt_event_id_t **event_id**

MQTT event type

esp_mqtt_client_handle_t **client**

MQTT client handle for this event

char ***data**

Data associated with this event

int **data_len**

Length of the data for this event

int **total_data_len**

Total length of the data (longer data are supplied with multiple events)

int **current_data_offset**

Actual offset for the data associated with this event

char ***topic**

Topic associated with this event

int **topic_len**

Length of the topic for this event associated with this event

int **msg_id**

MQTT messaged id of message

int **session_present**

MQTT session_present flag for connection event

esp_mqtt_error_codes_t ***error_handle**

esp-mqtt error handle including esp-tls errors as well as internal *MQTT* errors

bool **retain**

Retained flag of the message associated with this event

int **qos**

QoS of the messages associated with this event

bool **dup**

dup flag of the message associated with this event

esp_mqtt_protocol_ver_t **protocol_ver**

MQTT protocol version used for connection, defaults to value from menuconfig

struct **esp_mqtt_client_config_t**

MQTT client configuration structure

- Default values can be set via menuconfig
- All certificates and key data could be passed in PEM or DER format. PEM format must have a terminating NULL character and the related len field set to 0. DER format requires a related len field set to the correct length.

Public Members

struct *esp_mqtt_client_config_t*::*broker_t* **broker**

Broker address and security verification

struct *esp_mqtt_client_config_t*::*credentials_t* **credentials**

User credentials for broker

struct *esp_mqtt_client_config_t*::*session_t* **session**

MQTT session configuration.

struct *esp_mqtt_client_config_t*::*network_t* **network**

Network configuration

struct *esp_mqtt_client_config_t*::*task_t* **task**

FreeRTOS task configuration.

struct *esp_mqtt_client_config_t*::*buffer_t* **buffer**

Buffer size configuration.

struct *esp_mqtt_client_config_t*::*outbox_config_t* **outbox**

Outbox configuration.

struct **broker_t**

Broker related configuration

Public Members

struct *esp_mqtt_client_config_t*::*broker_t*::*address_t* **address**

Broker address configuration

struct *esp_mqtt_client_config_t*::*broker_t*::*verification_t* **verification**

Security verification of the broker

struct **address_t**

Broker address

- uri have precedence over other fields
- If uri isn't set at least hostname, transport and port should.

Public Members

const char ***uri**

Complete *MQTT* broker URI

const char ***hostname**

Hostname, to set ipv4 pass it as string)

esp_mqtt_transport_t **transport**

Selects transport

const char ***path**

Path in the URI

uint32_t **port**

MQTT server port

struct **verification_t**

Broker identity verification

If fields are not set broker's identity isn't verified. it's recommended to set the options in this struct for security reasons.

Public Members

bool **use_global_ca_store**

Use a global ca_store, look esp-tls documentation for details.

esp_err_t (***crt_bundle_attach**)(void *conf)

Pointer to ESP x509 Certificate Bundle attach function for the usage of certificate bundles. Client only attach the bundle, the clean up must be done by the user.

const char ***certificate**

Certificate data, default is NULL. It's not copied nor freed by the client, user needs to clean up.

size_t **certificate_len**

Length of the buffer pointed to by certificate.

const struct *psk_key_hint* ***psk_hint_key**

Pointer to PSK struct defined in esp_tls.h to enable PSK authentication (as alternative to certificate verification). PSK is enabled only if there are no other ways to verify broker. It's not copied nor freed by the client, user needs to clean up.

bool **skip_cert_common_name_check**

Skip any validation of server certificate CN field, this reduces the security of TLS and makes the *MQTT* client susceptible to MITM attacks

const char ****alpn_protos**

NULL-terminated list of supported application protocols to be used for ALPN.

const char ***common_name**

Pointer to the string containing server certificate common name. If non-NULL, server certificate CN must match this name, If NULL, server certificate CN must match hostname. This is ignored if skip_cert_common_name_check=true. It's not copied nor freed by the client, user needs to clean up.

struct **buffer_t**

Client buffer size configuration

Client have two buffers for input and output respectivelly.

Public Members

int **size**

size of *MQTT* send/receive buffer

int **out_size**

size of *MQTT* output buffer. If not defined, defaults to the size defined by `buffer_size`

struct **credentials_t**

Client related credentials for authentication.

Public Members

const char ***username**

MQTT username

const char ***client_id**

Set *MQTT* client identifier. Ignored if set_null_client_id == true If NULL set the default client id. Default client id is `ESP32_CHIPID%` where `CHIPID%` are last 3 bytes of MAC address in hex format

bool **set_null_client_id**

Selects a NULL client id

struct *esp_mqtt_client_config_t*::*credentials_t*::*authentication_t* **authentication**

Client authentication

struct **authentication_t**

Client authentication

Fields related to client authentication by broker

For mutual authentication using TLS, user could select certificate and key, secure element or digital signature peripheral if available.

Public Members

const char ***password**

MQTT password

const char ***certificate**

Certificate for ssl mutual authentication, not required if mutual authentication is not needed. Must be provided with `key`. It's not copied nor freed by the client, user needs to clean up.

size_t **certificate_len**

Length of the buffer pointed to by certificate.

const char ***key**

Private key for SSL mutual authentication, not required if mutual authentication is not needed. If it is not NULL, also `certificate` has to be provided. It's not copied nor freed by the client, user needs to clean up.

size_t **key_len**

Length of the buffer pointed to by key.

const char ***key_password**

Client key decryption password, not PEM nor DER, if provided `key_password_len` must be correctly set.

int **key_password_len**

Length of the password pointed to by `key_password`

bool **use_secure_element**

Enable secure element, available in ESP32-ROOM-32SE, for SSL connection

void ***ds_data**

Carrier of handle for digital signature parameters, digital signature peripheral is available in some Espressif devices. It's not copied nor freed by the client, user needs to clean up.

struct **network_t**

Network related configuration

Public Members

int **reconnect_timeout_ms**

Reconnect to the broker after this value in miliseconds if auto reconnect is not disabled (defaults to 10s)

int **timeout_ms**

Abort network operation if it is not completed after this value, in milliseconds (defaults to 10s).

int **refresh_connection_after_ms**

Refresh connection after this value (in milliseconds)

bool **disable_auto_reconnect**

Client will reconnect to server (when errors/disconnect). Set `disable_auto_reconnect=true` to disable

esp_transport_handle_t **transport**

Custom transport handle to use. Warning: The transport should be valid during the client lifetime and is destroyed when esp_mqtt_client_destroy is called.

struct ifreq ***if_name**

The name of interface for data to go through. Use the default interface without setting

struct **outbox_config_t**

Client outbox configuration options.

Public Members

uint64_t **limit**

Size limit for the outbox in bytes.

struct **session_t**

MQTT Session related configuration

Public Members

struct *esp_mqtt_client_config_t*::*session_t*::*last_will_t* **last_will**

Last will configuration

bool **disable_clean_session**

MQTT clean session, default clean_session is true

int **keepalive**

MQTT keepalive, default is 120 seconds When configuring this value, keep in mind that the client attempts to communicate with the broker at half the interval that is actually set. This conservative approach allows for more attempts before the broker's timeout occurs

bool **disable_keepalive**

Set `disable_keepalive=true` to turn off keep-alive mechanism, keepalive is active by default. Note: setting the config value `keepalive` to `0` doesn't disable keepalive feature, but uses a default keepalive period

esp_mqtt_protocol_ver_t **protocol_ver**

MQTT protocol version used for connection.

int **message_retransmit_timeout**

timeout for retransmitting of failed packet

struct **last_will_t**

Last Will and Testament message configuration.

Public Members

const char *`topic`
LWT (Last Will and Testament) message topic

const char *`msg`
LWT message, may be NULL terminated

int `msg_len`
LWT message length, if msg isn't NULL terminated must have the correct length

int `qos`
LWT message QoS

int `retain`
LWT retained message flag

struct `task_t`
Client task configuration

Public Members

int `priority`
MQTT task priority

int `stack_size`
MQTT task stack size

struct `topic_t`
Topic definition struct

Public Members

const char *`filter`
Topic filter to subscribe

int `qos`
Max QoS level of the subscription

Macros

`MQTT_ERROR_TYPE_ESP_TLS`
MQTT_ERROR_TYPE_TCP_TRANSPORT error type hold all sorts of transport layer errors, including ESP-TLS error, but in the past only the errors from MQTT_ERROR_TYPE_ESP_TLS layer were reported, so the ESP-TLS error type is re-defined here for backward compatibility

esp_mqtt_client_subscribe(client_handle, topic_type, qos_or_size)

Convenience macro to select subscribe function to use.

Notes:

- Usage of `esp_mqtt_client_subscribe_single` is the same as previous esp_mqtt_client_subscribe, refer to it for details.

Parameters

- **client_handle** -- *MQTT* client handle
- **topic_type** -- Needs to be char* for single subscription or `esp_mqtt_topic_t` for multiple topics
- **qos_or_size** -- It's either a qos when subscribing to a single topic or the size of the subscription array when subscribing to multiple topics.

Returns message_id of the subscribe message on success -1 on failure -2 in case of full outbox.

Type Definitions

typedef struct esp_mqtt_client ***esp_mqtt_client_handle_t**

typedef enum *esp_mqtt_event_id_t* **esp_mqtt_event_id_t**

MQTT event types.

User event handler receives context data in *esp_mqtt_event_t* structure with

- client - *MQTT* client handle
- various other data depending on event type

typedef enum *esp_mqtt_connect_return_code_t* **esp_mqtt_connect_return_code_t**

MQTT connection error codes propagated via ERROR event

typedef enum *esp_mqtt_error_type_t* **esp_mqtt_error_type_t**

MQTT connection error codes propagated via ERROR event

typedef enum *esp_mqtt_transport_t* **esp_mqtt_transport_t**

typedef enum *esp_mqtt_protocol_ver_t* **esp_mqtt_protocol_ver_t**

MQTT protocol version used for connection

typedef struct *esp_mqtt_error_codes* **esp_mqtt_error_codes_t**

MQTT error code structure to be passed as a contextual information into ERROR event

Important: This structure extends *esp_tls_last_error* error structure and is backward compatible with it (so might be down-casted and treated as *esp_tls_last_error* error, but recommended to update applications if used this way previously)

Use this structure directly checking error_type first and then appropriate error code depending on the source of the error:

| error_type | related member variables | note | | MQTT_ERROR_TYPE_TCP_TRANSPORT | esp_tls_last_esp_err, esp_tls_stack_err, esp_tls_cert_verify_flags, sock_errno | Error reported from tcp_transport/esp-tls | | MQTT_ERROR_TYPE_CONNECTION_REFUSED | connect_return_code | Internal error reported from *MQTT* broker on connection |

typedef struct *esp_mqtt_event_t* **esp_mqtt_event_t**

MQTT event configuration structure

typedef *esp_mqtt_event_t* ***esp_mqtt_event_handle_t**

typedef struct *esp_mqtt_client_config_t* **esp_mqtt_client_config_t**

MQTT client configuration structure

- Default values can be set via menuconfig
- All certificates and key data could be passed in PEM or DER format. PEM format must have a terminating NULL character and the related len field set to 0. DER format requires a related len field set to the correct length.

typedef struct *topic_t* **esp_mqtt_topic_t**

Topic definition struct

Enumerations

enum **esp_mqtt_event_id_t**

MQTT event types.

User event handler receives context data in `esp_mqtt_event_t` structure with

- client - *MQTT* client handle
- various other data depending on event type

Values:

enumerator **MQTT_EVENT_ANY**

enumerator **MQTT_EVENT_ERROR**

on error event, additional context: connection return code, error handle from esp_tls (if supported)

enumerator **MQTT_EVENT_CONNECTED**

connected event, additional context: session_present flag

enumerator **MQTT_EVENT_DISCONNECTED**

disconnected event

enumerator **MQTT_EVENT_SUBSCRIBED**

subscribed event, additional context:

- msg_id message id
- error_handle `error_type` in case subscribing failed
- data pointer to broker response, check for errors.
- data_len length of the data for this event

enumerator **MQTT_EVENT_UNSUBSCRIBED**

unsubscribed event, additional context: msg_id

enumerator **MQTT_EVENT_PUBLISHED**

published event, additional context: msg_id

enumerator **MQTT_EVENT_DATA**

data event, additional context:

- msg_id message id
- topic pointer to the received topic
- topic_len length of the topic
- data pointer to the received data
- data_len length of the data for this event
- current_data_offset offset of the current data for this event
- total_data_len total length of the data received
- retain retain flag of the message
- qos QoS level of the message
- dup dup flag of the message Note: Multiple MQTT_EVENT_DATA could be fired for one message, if it is longer than internal buffer. In that case only first event contains topic pointer and length, other contain data only with current data length and current data offset updating.

enumerator **MQTT_EVENT_BEFORE_CONNECT**

The event occurs before connecting

enumerator **MQTT_EVENT_DELETED**

Notification on delete of one message from the internal outbox, if the message couldn't have been sent and acknowledged before expiring defined in OUTBOX_EXPIRED_TIMEOUT_MS. (events are not posted upon deletion of successfully acknowledged messages)

- This event id is posted only if MQTT_REPORT_DELETED_MESSAGES==1
- Additional context: msg_id (id of the deleted message).

enumerator **MQTT_USER_EVENT**

Custom event used to queue tasks into mqtt event handler All fields from the *esp_mqtt_event_t* type could be used to pass an additional context data to the handler.

enum **esp_mqtt_connect_return_code_t**

MQTT connection error codes propagated via ERROR event

Values:

enumerator **MQTT_CONNECTION_ACCEPTED**

Connection accepted

enumerator **MQTT_CONNECTION_REFUSE_PROTOCOL**

MQTT connection refused reason: Wrong protocol

enumerator **MQTT_CONNECTION_REFUSE_ID_REJECTED**

MQTT connection refused reason: ID rejected

enumerator **MQTT_CONNECTION_REFUSE_SERVER_UNAVAILABLE**

MQTT connection refused reason: Server unavailable

enumerator **MQTT_CONNECTION_REFUSE_BAD_USERNAME**

MQTT connection refused reason: Wrong user

enumerator **MQTT_CONNECTION_REFUSE_NOT_AUTHORIZED**

MQTT connection refused reason: Wrong username or password

enum **esp_mqtt_error_type_t**

MQTT connection error codes propagated via ERROR event

Values:

enumerator **MQTT_ERROR_TYPE_NONE**

enumerator **MQTT_ERROR_TYPE_TCP_TRANSPORT**

enumerator **MQTT_ERROR_TYPE_CONNECTION_REFUSED**

enumerator **MQTT_ERROR_TYPE_SUBSCRIBE_FAILED**

enum **esp_mqtt_transport_t**

Values:

enumerator **MQTT_TRANSPORT_UNKNOWN**

enumerator **MQTT_TRANSPORT_OVER_TCP**

MQTT over TCP, using scheme: `MQTT`

enumerator **MQTT_TRANSPORT_OVER_SSL**

MQTT over SSL, using scheme: `MQTTS`

enumerator **MQTT_TRANSPORT_OVER_WS**

MQTT over Websocket, using scheme:: `ws`

enumerator **MQTT_TRANSPORT_OVER_WSS**

MQTT over Websocket Secure, using scheme: `wss`

enum **esp_mqtt_protocol_ver_t**

MQTT protocol version used for connection

Values:

enumerator **MQTT_PROTOCOL_UNDEFINED**

enumerator **MQTT_PROTOCOL_V_3_1**

enumerator **MQTT_PROTOCOL_V_3_1_1**

enumerator **MQTT_PROTOCOL_V_5**

2.2.4 ESP-TLS

Overview

The ESP-TLS component provides a simplified API interface for accessing the commonly used TLS functions. It supports common scenarios like CA certification validation, SNI, ALPN negotiation, and non-blocking connection among others. All the configurations can be specified in the `esp_tls_cfg_t` data structure. Once done, TLS communication can be conducted using the following APIs:

- *esp_tls_init()*: for initializing the TLS connection handle.
- *esp_tls_conn_new_sync()*: for opening a new blocking TLS connection.
- *esp_tls_conn_new_async()*: for opening a new non-blocking TLS connection.
- *esp_tls_conn_read()*: for reading from the connection.
- *esp_tls_conn_write()*: for writing into the connection.
- *esp_tls_conn_destroy()*: for freeing up the connection.

Any application layer protocol like HTTP1, HTTP2, etc can be executed on top of this layer.

Application Example

Simple HTTPS example that uses ESP-TLS to establish a secure socket connection: protocols/https_request.

Tree Structure for ESP-TLS Component

```
├── esp_tls.c
├── esp_tls.h
├── esp_tls_mbedtls.c
├── esp_tls_wolfssl.c
└── private_include
    ├── esp_tls_mbedtls.h
    └── esp_tls_wolfssl.h
```

The ESP-TLS component has a file esp-tls/esp_tls.h which contains the public API headers for the component. Internally, the ESP-TLS component operates using either MbedTLS or WolfSSL, which are SSL/TLS libraries. APIs specific to MbedTLS are present in esp-tls/private_include/esp_tls_mbedtls.h and APIs specific to WolfSSL are present in esp-tls/private_include/esp_tls_wolfssl.h.

TLS Server Verification

ESP-TLS provides multiple options for TLS server verification on the client side. The ESP-TLS client can verify the server by validating the peer's server certificate or with the help of pre-shared keys. The user should select only one of the following options in the *esp_tls_cfg_t* structure for TLS server verification. If no option is selected, the client will return a fatal error by default during the TLS connection setup.

- **cacert_buf** and **cacert_bytes**: The CA certificate can be provided in a buffer to the *esp_tls_cfg_t* structure. The ESP-TLS uses the CA certificate present in the buffer to verify the server. The following variables in the *esp_tls_cfg_t* structure must be set.
  - `cacert_buf` - pointer to the buffer which contains the CA certification.
  - `cacert_bytes` - the size of the CA certificate in bytes.
- **use_global_ca_store**: The `global_ca_store` can be initialized and set at once. Then it can be used to verify the server for all the ESP-TLS connections which have set `use_global_ca_store = true` in their respective *esp_tls_cfg_t* structure. See the API Reference section below for information regarding different APIs used for initializing and setting up the `global_ca_store`.
- **crt_bundle_attach**: The ESP x509 Certificate Bundle API provides an easy way to include a bundle of custom x509 root certificates for TLS server verification. More details can be found at *ESP x509 Certificate Bundle*.
- **psk_hint_key**: To use pre-shared keys for server verification, *CONFIG_ESP_TLS_PSK_VERIFICATION* should be enabled in the ESP-TLS menuconfig. Then the pointer to the PSK hint and key should be provided to the *esp_tls_cfg_t* structure. The ESP-TLS will use the PSK for server verification only when no other option regarding server verification is selected.
- **skip server verification**: This is an insecure option provided in the ESP-TLS for testing purposes. The option can be set by enabling *CONFIG_ESP_TLS_INSECURE* and *CONFIG_ESP_TLS_SKIP_SERVER_CERT_VERIFY* in the ESP-TLS menuconfig. When this option

is enabled the ESP-TLS will skip server verification by default when no other options for server verification are selected in the *esp_tls_cfg_t* structure.

Warning: Enabling this option comes with a potential risk of establishing a TLS connection with a server that has a fake identity, provided that the server certificate is not provided either through API or other mechanisms like ca_store etc.

ESP-TLS Server Cert Selection Hook

The ESP-TLS component provides an option to set the server certification selection hook when using the MbedTLS stack. This provides an ability to configure and use a certificate selection callback during server handshake. The callback helps to select a certificate to present to the client based on the TLS extensions supplied in the client hello message, such as ALPN and SNI. To enable this feature, please enable *CONFIG_ESP_TLS_SERVER_CERT_SELECT_HOOK* in the ESP-TLS menuconfig.

The certificate selection callback can be configured in the *esp_tls_cfg_t* structure as follows:

```
int cert_selection_callback(mbedtls_ssl_context *ssl)
{
    /* Code that the callback should execute */
    return 0;
}

esp_tls_cfg_t cfg = {
    cert_select_cb = cert_section_callback,
};
```

Underlying SSL/TLS Library Options

The ESP-TLS component offers the option to use MbedTLS or WolfSSL as its underlying SSL/TLS library. By default, only MbedTLS is available and used, WolfSSL SSL/TLS library is also available publicly at https://github.com/espressif/esp-wolfssl. The repository provides the WolfSSL component in binary format, and it also provides a few examples that are useful for understanding the API. Please refer to the repository `README.md` for information on licensing and other options. Please see the below section for instructions on how to use WolfSSL in your project.

Note: As the library options are internal to ESP-TLS, switching the libraries will not change ESP-TLS specific code for a project.

How to Use WolfSSL with ESP-IDF

There are two ways to use WolfSSL in your project:

1) Directly add WolfSSL as a component in your project with the following three commands:

```
(First, change the directory (cd) to your project directory)
mkdir components
cd components
git clone --recursive https://github.com/espressif/esp-wolfssl.git
```

2) Add WolfSSL as an extra component in your project.

- Download WolfSSL with:

```
git clone --recursive https://github.com/espressif/esp-wolfssl.git
```

- Include ESP-WolfSSL in ESP-IDF with setting `EXTRA_COMPONENT_DIRS` in `CMakeLists.txt` of your project as done in wolfssl/examples. For reference see *Optional Project Variables* in *build-system.*.

After the above steps, you will have the option to choose WolfSSL as the underlying SSL/TLS library in the configuration menu of your project as follows:

```
idf.py menuconfig > ESP-TLS > SSL/TLS Library > Mbedtls/Wolfssl
```

Comparison Between MbedTLS and WolfSSL

The following table shows a typical comparison between WolfSSL and MbedTLS when the protocols/https_request example (which includes server authentication) is running with both SSL/TLS libraries and with all respective configurations set to default. For MbedTLS, the IN_CONTENT length and OUT_CONTENT length are set to 16384 bytes and 4096 bytes respectively.

Property	WolfSSL	MbedTLS
Total Heap Consumed	~ 19 KB	~ 37 KB
Task Stack Used	~ 2.2 KB	~ 3.6 KB
Bin size	~ 858 KB	~ 736 KB

Note: These values can vary based on configuration options and version of respective libraries.

Digital Signature with ESP-TLS

ESP-TLS provides support for using the Digital Signature (DS) with ESP32-S3. Use of the DS for TLS is supported only when ESP-TLS is used with MbedTLS (default stack) as its underlying SSL/TLS stack. For more details on Digital Signature, please refer to the *Digital Signature (DS)*. The technical details of Digital Signature such as how to calculate private key parameters can be found in **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF]. The DS peripheral must be configured before it can be used to perform Digital Signature, see *Configure the DS Peripheral for a TLS Connection*.

The DS peripheral must be initialized with the required encrypted private key parameters, which are obtained when the DS peripheral is configured. ESP-TLS internally initializes the DS peripheral when provided with the required DS context, i.e., DS parameters. Please see the below code snippet for passing the DS context to the ESP-TLS context. The DS context passed to the ESP-TLS context should not be freed till the TLS connection is deleted.

```
#include "esp_tls.h"
esp_ds_data_ctx_t *ds_ctx;
/* initialize ds_ctx with encrypted private key parameters, which can be read from␣
↪the nvs or provided through the application code */
esp_tls_cfg_t cfg = {
    .clientcert_buf = /* the client certification */,
    .clientcert_bytes = /* length of the client certification */,
    /* other configurations options */
    .ds_data = (void *)ds_ctx,
};
```

Note: When using Digital Signature for the TLS connection, along with the other required params, only the client certification (*clientcert_buf*) and the DS params (*ds_data*) are required and the client key (*clientkey_buf*) can be set to NULL.

- An example of mutual authentication with the DS peripheral can be found at ssl mutual auth which internally uses (ESP-TLS) for the TLS connection.

TLS Ciphersuites

ESP-TLS provides the ability to set a ciphersuites list in client mode. The TLS ciphersuites list informs the server about the supported ciphersuites for the specific TLS connection regardless of the TLS stack configuration. If the server supports any ciphersuite from this list, then the TLS connection will succeed; otherwise, it will fail.

You can set `ciphersuites_list` in the *esp_tls_cfg_t* structure during client connection as follows:

```
/* ciphersuites_list must end with 0 and must be available in the memory scope␣
↪active during the entire TLS connection */
static const int ciphersuites_list[] = {MBEDTLS_TLS_ECDHE_ECDSA_WITH_AES_256_GCM_
↪SHA384, MBEDTLS_TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384, 0};
esp_tls_cfg_t cfg = {
    .ciphersuites_list = ciphersuites_list,
};
```

ESP-TLS will not check the validity of `ciphersuites_list` that was set, you should call *esp_tls_get_ciphersuites_list()* to get ciphersuites list supported in the TLS stack and cross-check it against the supplied list.

Note: This feature is supported only in the MbedTLS stack.

TLS Protocol Version

ESP-TLS provides the ability to set the TLS protocol version for the respective TLS connection. Once the version is specified, it should be exclusively used to establish the TLS connection. This provides an ability to route different TLS connections to different protocol versions like TLS 1.2 and TLS 1.3 at runtime.

Note: At the moment, the feature is supported only when ESP-TLS is used with MbedTLS as its underlying SSL/TLS stack.

To set TLS protocol version with ESP-TLS, set `esp_tls_cfg_t::tls_version` to the required protocol version from *esp_tls_proto_ver_t*. If the protocol version field is not set, then the default policy is to allow TLS connection based on the server requirement.

The ESP-TLS connection can be configured to use the specified protocol version as follows:

```
#include "esp_tls.h"
esp_tls_cfg_t cfg = {
    .tls_version = ESP_TLS_VER_TLS_1_2,
};
```

API Reference

Header File

- components/esp-tls/esp_tls.h
- This header file can be included with:

  ```
  #include "esp_tls.h"
  ```

- This header file is a part of the API provided by the `esp-tls` component. To declare that your component depends on `esp-tls`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp-tls
  ```

  or

```
PRIV_REQUIRES esp-tls
```

Functions

esp_err_t **esp_tls_cfg_server_session_tickets_init** (*esp_tls_cfg_server_t* *cfg)

Initialize the server side TLS session ticket context.

This function initializes the server side tls session ticket context which holds all necessary data structures to enable tls session tickets according to RFC5077. Use esp_tls_cfg_server_session_tickets_free to free the data.

Parameters **cfg** -- **[in]** server configuration as esp_tls_cfg_server_t

Returns ESP_OK if setup succeeded ESP_ERR_INVALID_ARG if context is already initialized ESP_ERR_NO_MEM if memory allocation failed ESP_ERR_NOT_SUPPORTED if session tickets are not available due to build configuration ESP_FAIL if setup failed

void **esp_tls_cfg_server_session_tickets_free** (*esp_tls_cfg_server_t* *cfg)

Free the server side TLS session ticket context.

Parameters **cfg** -- server configuration as esp_tls_cfg_server_t

esp_tls_t ***esp_tls_init** (void)

Create TLS connection.

This function allocates and initializes esp-tls structure handle.

Returns tls Pointer to esp-tls as esp-tls handle if successfully initialized, NULL if allocation error

esp_tls_t ***esp_tls_conn_http_new** (const char *url, const *esp_tls_cfg_t* *cfg)

Create a new blocking TLS/SSL connection with a given "HTTP" url.

Note: This API is present for backward compatibility reasons. Alternative function with the same functionality is `esp_tls_conn_http_new_sync` (and its asynchronous version `esp_tls_conn_http_new_async`)

Parameters

- **url** -- **[in]** url of host.
- **cfg** -- **[in]** TLS configuration as esp_tls_cfg_t. If you wish to open non-TLS connection, keep this NULL. For TLS connection, a pass pointer to 'esp_tls_cfg_t'. At a minimum, this structure should be zero-initialized.

Returns pointer to esp_tls_t, or NULL if connection couldn't be opened.

int **esp_tls_conn_new_sync** (const char *hostname, int hostlen, int port, const *esp_tls_cfg_t* *cfg, *esp_tls_t* *tls)

Create a new blocking TLS/SSL connection.

This function establishes a TLS/SSL connection with the specified host in blocking manner.

Parameters

- **hostname** -- **[in]** Hostname of the host.
- **hostlen** -- **[in]** Length of hostname.
- **port** -- **[in]** Port number of the host.
- **cfg** -- **[in]** TLS configuration as esp_tls_cfg_t. If you wish to open non-TLS connection, keep this NULL. For TLS connection, a pass pointer to esp_tls_cfg_t. At a minimum, this structure should be zero-initialized.
- **tls** -- **[in]** Pointer to esp-tls as esp-tls handle.

Returns

- -1 If connection establishment fails.
- 1 If connection establishment is successful.
- 0 If connection state is in progress.

int **esp_tls_conn_http_new_sync** (const char *url, const *esp_tls_cfg_t* *cfg, *esp_tls_t* *tls)

Create a new blocking TLS/SSL connection with a given "HTTP" url.

The behaviour is same as esp_tls_conn_new_sync() API. However this API accepts host's url.

Parameters

- **url** -- **[in]** url of host.
- **cfg** -- **[in]** TLS configuration as esp_tls_cfg_t. If you wish to open non-TLS connection, keep this NULL. For TLS connection, a pass pointer to 'esp_tls_cfg_t'. At a minimum, this structure should be zero-initialized.
- **tls** -- **[in]** Pointer to esp-tls as esp-tls handle.

Returns

- -1 If connection establishment fails.
- 1 If connection establishment is successful.
- 0 If connection state is in progress.

int **esp_tls_conn_new_async**(const char *hostname, int hostlen, int port, const *esp_tls_cfg_t* *cfg, *esp_tls_t* *tls)

Create a new non-blocking TLS/SSL connection.

This function initiates a non-blocking TLS/SSL connection with the specified host, but due to its non-blocking nature, it doesn't wait for the connection to get established.

Parameters

- **hostname** -- **[in]** Hostname of the host.
- **hostlen** -- **[in]** Length of hostname.
- **port** -- **[in]** Port number of the host.
- **cfg** -- **[in]** TLS configuration as esp_tls_cfg_t. `non_block` member of this structure should be set to be true.
- **tls** -- **[in]** pointer to esp-tls as esp-tls handle.

Returns

- -1 If connection establishment fails.
- 0 If connection establishment is in progress.
- 1 If connection establishment is successful.

int **esp_tls_conn_http_new_async**(const char *url, const *esp_tls_cfg_t* *cfg, *esp_tls_t* *tls)

Create a new non-blocking TLS/SSL connection with a given "HTTP" url.

The behaviour is same as esp_tls_conn_new_async() API. However this API accepts host's url.

Parameters

- **url** -- **[in]** url of host.
- **cfg** -- **[in]** TLS configuration as esp_tls_cfg_t.
- **tls** -- **[in]** pointer to esp-tls as esp-tls handle.

Returns

- -1 If connection establishment fails.
- 0 If connection establishment is in progress.
- 1 If connection establishment is successful.

ssize_t **esp_tls_conn_write**(*esp_tls_t* *tls, const void *data, size_t datalen)

Write from buffer 'data' into specified tls connection.

Parameters

- **tls** -- **[in]** pointer to esp-tls as esp-tls handle.
- **data** -- **[in]** Buffer from which data will be written.
- **datalen** -- **[in]** Length of data buffer.

Returns

- >=0 if write operation was successful, the return value is the number of bytes actually written to the TLS/SSL connection.
- <0 if write operation was not successful, because either an error occured or an action must be taken by the calling process.
- ESP_TLS_ERR_SSL_WANT_READ/ ESP_TLS_ERR_SSL_WANT_WRITE. if the handshake is incomplete and waiting for data to be available for reading. In this case this functions needs to be called again when the underlying transport is ready for operation.

ssize_t **esp_tls_conn_read**(*esp_tls_t* *tls, void *data, size_t datalen)

Read from specified tls connection into the buffer 'data'.

Parameters
- **tls** -- **[in]** pointer to esp-tls as esp-tls handle.
- **data** -- **[in]** Buffer to hold read data.
- **datalen** -- **[in]** Length of data buffer.

Returns
- >0 if read operation was successful, the return value is the number of bytes actually read from the TLS/SSL connection.
- 0 if read operation was not successful. The underlying connection was closed.
- <0 if read operation was not successful, because either an error occured or an action must be taken by the calling process.

int **esp_tls_conn_destroy**(*esp_tls_t* *tls)

Close the TLS/SSL connection and free any allocated resources.

This function should be called to close each tls connection opened with esp_tls_conn_new_sync() (or esp_tls_conn_http_new_sync()) and esp_tls_conn_new_async() (or esp_tls_conn_http_new_async()) APIs.

Parameters **tls** -- **[in]** pointer to esp-tls as esp-tls handle.
Returns - 0 on success
- -1 if socket error or an invalid argument

ssize_t **esp_tls_get_bytes_avail**(*esp_tls_t* *tls)

Return the number of application data bytes remaining to be read from the current record.

This API is a wrapper over mbedtls's mbedtls_ssl_get_bytes_avail() API.

Parameters **tls** -- **[in]** pointer to esp-tls as esp-tls handle.
Returns
- -1 in case of invalid arg
- bytes available in the application data record read buffer

esp_err_t **esp_tls_get_conn_sockfd**(*esp_tls_t* *tls, int *sockfd)

Returns the connection socket file descriptor from esp_tls session.

Parameters
- **tls** -- **[in]** handle to esp_tls context
- **sockfd** -- **[out]** int pointer to sockfd value.

Returns - ESP_OK on success and value of sockfd will be updated with socket file descriptor for connection
- ESP_ERR_INVALID_ARG if (tls == NULL || sockfd == NULL)

esp_err_t **esp_tls_set_conn_sockfd**(*esp_tls_t* *tls, int sockfd)

Sets the connection socket file descriptor for the esp_tls session.

Parameters
- **tls** -- **[in]** handle to esp_tls context
- **sockfd** -- **[in]** sockfd value to set.

Returns - ESP_OK on success and value of sockfd for the tls connection shall updated withthe provided value
- ESP_ERR_INVALID_ARG if (tls == NULL || sockfd < 0)

esp_err_t **esp_tls_get_conn_state**(*esp_tls_t* *tls, *esp_tls_conn_state_t* *conn_state)

Gets the connection state for the esp_tls session.

Parameters
- **tls** -- **[in]** handle to esp_tls context
- **conn_state** -- **[out]** pointer to the connection state value.

Returns - ESP_OK on success and value of sockfd for the tls connection shall updated withthe provided value
- ESP_ERR_INVALID_ARG (Invalid arguments)

esp_err_t **esp_tls_set_conn_state**(*esp_tls_t* *tls, *esp_tls_conn_state_t* conn_state)

Sets the connection state for the esp_tls session.

Parameters
- **tls** -- **[in]** handle to esp_tls context
- **conn_state** -- **[in]** connection state value to set.

Returns - ESP_OK on success and value of sockfd for the tls connection shall updated withthe provided value
- ESP_ERR_INVALID_ARG (Invalid arguments)

void ***esp_tls_get_ssl_context**(*esp_tls_t* *tls)

Returns the ssl context.

Parameters **tls** -- **[in]** handle to esp_tls context

Returns - ssl_ctx pointer to ssl context of underlying TLS layer on success
- NULL in case of error

esp_err_t **esp_tls_init_global_ca_store**(void)

Create a global CA store, initially empty.

This function should be called if the application wants to use the same CA store for multiple connections. This function initialises the global CA store which can be then set by calling esp_tls_set_global_ca_store(). To be effective, this function must be called before any call to esp_tls_set_global_ca_store().

Returns
- ESP_OK if creating global CA store was successful.
- ESP_ERR_NO_MEM if an error occured when allocating the mbedTLS resources.

esp_err_t **esp_tls_set_global_ca_store**(const unsigned char *cacert_pem_buf, const unsigned int cacert_pem_bytes)

Set the global CA store with the buffer provided in pem format.

This function should be called if the application wants to set the global CA store for multiple connections i.e. to add the certificates in the provided buffer to the certificate chain. This function implicitly calls esp_tls_init_global_ca_store() if it has not already been called. The application must call this function before calling esp_tls_conn_new().

Parameters
- **cacert_pem_buf** -- **[in]** Buffer which has certificates in pem format. This buffer is used for creating a global CA store, which can be used by other tls connections.
- **cacert_pem_bytes** -- **[in]** Length of the buffer.

Returns
- ESP_OK if adding certificates was successful.
- Other if an error occured or an action must be taken by the calling process.

void **esp_tls_free_global_ca_store**(void)

Free the global CA store currently being used.

The memory being used by the global CA store to store all the parsed certificates is freed up. The application can call this API if it no longer needs the global CA store.

esp_err_t **esp_tls_get_and_clear_last_error**(*esp_tls_error_handle_t* h, int *esp_tls_code, int *esp_tls_flags)

Returns last error in esp_tls with detailed mbedtls related error codes. The error information is cleared internally upon return.

Parameters
- **h** -- **[in]** esp-tls error handle.
- **esp_tls_code** -- **[out]** last error code returned from mbedtls api (set to zero if none) This pointer could be NULL if caller does not care about esp_tls_code
- **esp_tls_flags** -- **[out]** last certification verification flags (set to zero if none) This pointer could be NULL if caller does not care about esp_tls_code

Returns
- ESP_ERR_INVALID_STATE if invalid parameters
- ESP_OK (0) if no error occurred
- specific error code (based on ESP_ERR_ESP_TLS_BASE) otherwise

esp_err_t **esp_tls_get_and_clear_error_type**(*esp_tls_error_handle_t* h, *esp_tls_error_type_t* err_type, int *error_code)

Returns the last error captured in esp_tls of a specific type The error information is cleared internally upon return.

Parameters
- **h** -- **[in]** esp-tls error handle.
- **err_type** -- **[in]** specific error type
- **error_code** -- **[out]** last error code returned from mbedtls api (set to zero if none) This pointer could be NULL if caller does not care about esp_tls_code

Returns
- ESP_ERR_INVALID_STATE if invalid parameters
- ESP_OK if a valid error returned and was cleared

esp_err_t **esp_tls_get_error_handle**(*esp_tls_t* *tls, *esp_tls_error_handle_t* *error_handle)

Returns the ESP-TLS error_handle.

Parameters
- **tls** -- **[in]** handle to esp_tls context
- **error_handle** -- **[out]** pointer to the error handle.

Returns
- ESP_OK on success and error_handle will be updated with the ESP-TLS error handle.
- ESP_ERR_INVALID_ARG if (tls == NULL || error_handle == NULL)

mbedtls_x509_crt ***esp_tls_get_global_ca_store**(void)

Get the pointer to the global CA store currently being used.

The application must first call esp_tls_set_global_ca_store(). Then the same CA store could be used by the application for APIs other than esp_tls.

Note: Modifying the pointer might cause a failure in verifying the certificates.

Returns
- Pointer to the global CA store currently being used if successful.
- NULL if there is no global CA store set.

const int ***esp_tls_get_ciphersuites_list**(void)

Get supported TLS ciphersuites list.

See https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml#tls-parameters-4 for the list of ciphersuites

Returns Pointer to a zero-terminated array of IANA identifiers of TLS ciphersuites.

int **esp_tls_server_session_create**(*esp_tls_cfg_server_t* *cfg, int sockfd, *esp_tls_t* *tls)

Create TLS/SSL server session.

This function creates a TLS/SSL server context for already accepted client connection and performs TLS/SSL handshake with the client

Parameters
- **cfg** -- **[in]** Pointer to esp_tls_cfg_server_t
- **sockfd** -- **[in]** FD of accepted connection
- **tls** -- **[out]** Pointer to allocated esp_tls_t

Returns
- 0 if successful
- <0 in case of error

void **esp_tls_server_session_delete**(*esp_tls_t* *tls)

Close the server side TLS/SSL connection and free any allocated resources.

This function should be called to close each tls connection opened with esp_tls_server_session_create()

Parameters **`tls`** -- **[in]** pointer to esp_tls_t

esp_err_t **`esp_tls_plain_tcp_connect`**(const char *host, int hostlen, int port, const *esp_tls_cfg_t* *cfg, *esp_tls_error_handle_t* error_handle, int *sockfd)

Creates a plain TCP connection, returning a valid socket fd on success or an error handle.

Parameters

- **`host`** -- **[in]** Hostname of the host.
- **`hostlen`** -- **[in]** Length of hostname.
- **`port`** -- **[in]** Port number of the host.
- **`cfg`** -- **[in]** ESP-TLS configuration as esp_tls_cfg_t.
- **`error_handle`** -- **[out]** ESP-TLS error handle holding potential errors occurred during connection
- **`sockfd`** -- **[out]** Socket descriptor if successfully connected on TCP layer

Returns ESP_OK on success ESP_ERR_INVALID_ARG if invalid output parameters ESP-TLS based error codes on failure

Structures

struct **`psk_key_hint`**

ESP-TLS preshared key and hint structure.

Public Members

const uint8_t ***`key`**

key in PSK authentication mode in binary format

const size_t **`key_size`**

length of the key

const char ***`hint`**

hint in PSK authentication mode in string format

struct **`tls_keep_alive_cfg`**

esp-tls client session ticket ctx

Keep alive parameters structure

Public Members

bool **`keep_alive_enable`**

Enable keep-alive timeout

int **`keep_alive_idle`**

Keep-alive idle time (second)

int **`keep_alive_interval`**

Keep-alive interval time (second)

int **`keep_alive_count`**

Keep-alive packet retry send count

struct **esp_tls_cfg**

ESP-TLS configuration parameters.

Note: Note about format of certificates:

- This structure includes certificates of a Certificate Authority, of client or server as well as private keys, which may be of PEM or DER format. In case of PEM format, the buffer must be NULL terminated (with NULL character included in certificate size).
- Certificate Authority's certificate may be a chain of certificates in case of PEM format, but could be only one certificate in case of DER format
- Variables names of certificates and private key buffers and sizes are defined as unions providing backward compatibility for legacy *_pem_buf and *_pem_bytes names which suggested only PEM format was supported. It is encouraged to use generic names such as cacert_buf and cacert_bytes.

Public Members

const char ****alpn_protos**

Application protocols required for HTTP2. If HTTP2/ALPN support is required, a list of protocols that should be negotiated. The format is length followed by protocol name. For the most common cases the following is ok: const char **alpn_protos = { "h2", NULL };

- where 'h2' is the protocol name

const unsigned char ***cacert_buf**

Certificate Authority's certificate in a buffer. Format may be PEM or DER, depending on mbedtls-support This buffer should be NULL terminated in case of PEM

const unsigned char ***cacert_pem_buf**

CA certificate buffer legacy name

unsigned int **cacert_bytes**

Size of Certificate Authority certificate pointed to by cacert_buf (including NULL-terminator in case of PEM format)

unsigned int **cacert_pem_bytes**

Size of Certificate Authority certificate legacy name

const unsigned char ***clientcert_buf**

Client certificate in a buffer Format may be PEM or DER, depending on mbedtls-support This buffer should be NULL terminated in case of PEM

const unsigned char ***clientcert_pem_buf**

Client certificate legacy name

unsigned int **clientcert_bytes**

Size of client certificate pointed to by clientcert_pem_buf (including NULL-terminator in case of PEM format)

unsigned int **clientcert_pem_bytes**

Size of client certificate legacy name

const unsigned char ***clientkey_buf**

Client key in a buffer Format may be PEM or DER, depending on mbedtls-support This buffer should be NULL terminated in case of PEM

const unsigned char ***clientkey_pem_buf**

Client key legacy name

unsigned int **clientkey_bytes**

Size of client key pointed to by clientkey_pem_buf (including NULL-terminator in case of PEM format)

unsigned int **clientkey_pem_bytes**

Size of client key legacy name

const unsigned char ***clientkey_password**

Client key decryption password string

unsigned int **clientkey_password_len**

String length of the password pointed to by clientkey_password

bool **use_ecdsa_peripheral**

Use the ECDSA peripheral for the private key operations

uint8_t **ecdsa_key_efuse_blk**

The efuse block where the ECDSA key is stored

bool **non_block**

Configure non-blocking mode. If set to true the underneath socket will be configured in non blocking mode after tls session is established

bool **use_secure_element**

Enable this option to use secure element or atecc608a chip

int **timeout_ms**

Network timeout in milliseconds. Note: If this value is not set, by default the timeout is set to 10 seconds. If you wish that the session should wait indefinitely then please use a larger value e.g., INT32_MAX

bool **use_global_ca_store**

Use a global ca_store for all the connections in which this bool is set.

const char ***common_name**

If non-NULL, server certificate CN must match this name. If NULL, server certificate CN must match hostname.

bool **skip_common_name**

Skip any validation of server certificate CN field

tls_keep_alive_cfg_t ***keep_alive_cfg**

Enable TCP keep-alive timeout for SSL connection

const *psk_hint_key_t* ***psk_hint_key**

Pointer to PSK hint and key. if not NULL (and certificates are NULL) then PSK authentication is enabled with configured setup. Important note: the pointer must be valid for connection

esp_err_t (***crt_bundle_attach**)(void *conf)

Function pointer to esp_crt_bundle_attach. Enables the use of certification bundle for server verification, must be enabled in menuconfig

void ***ds_data**

Pointer for digital signature peripheral context

bool **is_plain_tcp**

Use non-TLS connection: When set to true, the esp-tls uses plain TCP transport rather then TLS/SSL connection. Note, that it is possible to connect using a plain tcp transport directly with esp_tls_plain_tcp_connect() API

struct ifreq ***if_name**

The name of interface for data to go through. Use the default interface without setting

esp_tls_addr_family_t **addr_family**

The address family to use when connecting to a host.

const int ***ciphersuites_list**

Pointer to a zero-terminated array of IANA identifiers of TLS ciphersuites. Please check the list validity by esp_tls_get_ciphersuites_list() API

esp_tls_proto_ver_t **tls_version**

TLS protocol version of the connection, e.g., TLS 1.2, TLS 1.3 (default - no preference)

struct **esp_tls_cfg_server**

ESP-TLS Server configuration parameters.

Public Members

const char ****alpn_protos**

Application protocols required for HTTP2. If HTTP2/ALPN support is required, a list of protocols that should be negotiated. The format is length followed by protocol name. For the most common cases the following is ok: const char **alpn_protos = { "h2", NULL };

- where 'h2' is the protocol name

const unsigned char ***cacert_buf**

Client CA certificate in a buffer. This buffer should be NULL terminated

const unsigned char ***cacert_pem_buf**

Client CA certificate legacy name

unsigned int **cacert_bytes**

Size of client CA certificate pointed to by cacert_pem_buf

unsigned int **cacert_pem_bytes**

Size of client CA certificate legacy name

const unsigned char ***servercert_buf**

Server certificate in a buffer This buffer should be NULL terminated

const unsigned char ***servercert_pem_buf**

Server certificate legacy name

unsigned int **servercert_bytes**

Size of server certificate pointed to by servercert_pem_buf

unsigned int **servercert_pem_bytes**

Size of server certificate legacy name

const unsigned char ***serverkey_buf**

Server key in a buffer This buffer should be NULL terminated

const unsigned char ***serverkey_pem_buf**

Server key legacy name

unsigned int **serverkey_bytes**

Size of server key pointed to by serverkey_pem_buf

unsigned int **serverkey_pem_bytes**

Size of server key legacy name

const unsigned char ***serverkey_password**

Server key decryption password string

unsigned int **serverkey_password_len**

String length of the password pointed to by serverkey_password

bool **use_ecdsa_peripheral**

Use ECDSA peripheral to use private key

uint8_t **ecdsa_key_efuse_blk**

The efuse block where ECDSA key is stored

bool **use_secure_element**

Enable this option to use secure element or atecc608a chip

void ***userdata**

User data to be added to the ssl context. Can be retrieved by callbacks

Type Definitions

typedef enum *esp_tls_conn_state* **esp_tls_conn_state_t**

ESP-TLS Connection State.

typedef enum *esp_tls_role* **esp_tls_role_t**

typedef struct *psk_key_hint* **psk_hint_key_t**

ESP-TLS preshared key and hint structure.

typedef struct *tls_keep_alive_cfg* **tls_keep_alive_cfg_t**

esp-tls client session ticket ctx

Keep alive parameters structure

typedef enum *esp_tls_addr_family* **esp_tls_addr_family_t**

typedef struct *esp_tls_cfg* **esp_tls_cfg_t**

ESP-TLS configuration parameters.

Note: Note about format of certificates:

- This structure includes certificates of a Certificate Authority, of client or server as well as private keys, which may be of PEM or DER format. In case of PEM format, the buffer must be NULL terminated (with NULL character included in certificate size).
- Certificate Authority's certificate may be a chain of certificates in case of PEM format, but could be only one certificate in case of DER format
- Variables names of certificates and private key buffers and sizes are defined as unions providing backward compatibility for legacy *_pem_buf and *_pem_bytes names which suggested only PEM format was supported. It is encouraged to use generic names such as cacert_buf and cacert_bytes.

---

typedef void ***esp_tls_handshake_callback**

typedef struct *esp_tls_cfg_server* **esp_tls_cfg_server_t**

ESP-TLS Server configuration parameters.

typedef struct esp_tls **esp_tls_t**

Enumerations

enum **esp_tls_conn_state**

ESP-TLS Connection State.

Values:

enumerator **ESP_TLS_INIT**

enumerator **ESP_TLS_CONNECTING**

enumerator **ESP_TLS_HANDSHAKE**

enumerator **ESP_TLS_FAIL**

enumerator **ESP_TLS_DONE**

enum **esp_tls_role**

Values:

enumerator **ESP_TLS_CLIENT**

enumerator **ESP_TLS_SERVER**

enum **esp_tls_addr_family**

Values:

enumerator **ESP_TLS_AF_UNSPEC**

Unspecified address family.

enumerator **ESP_TLS_AF_INET**

IPv4 address family.

enumerator **ESP_TLS_AF_INET6**

IPv6 address family.

enum **esp_tls_proto_ver_t**

Values:

enumerator **ESP_TLS_VER_ANY**

enumerator **ESP_TLS_VER_TLS_1_2**

enumerator **ESP_TLS_VER_TLS_1_3**

enumerator **ESP_TLS_VER_TLS_MAX**

Header File

- components/esp-tls/esp_tls_errors.h
- This header file can be included with:

```
#include "esp_tls_errors.h"
```

- This header file is a part of the API provided by the `esp-tls` component. To declare that your component depends on `esp-tls`, add the following to your CMakeLists.txt:

```
REQUIRES esp-tls
```

or

```
PRIV_REQUIRES esp-tls
```

Structures

struct **esp_tls_last_error**

Error structure containing relevant errors in case tls error occurred.

Public Members

esp_err_t **last_error**

error code (based on ESP_ERR_ESP_TLS_BASE) of the last occurred error

int **esp_tls_error_code**

esp_tls error code from last esp_tls failed api

int **esp_tls_flags**

last certification verification flags

Macros

ESP_ERR_ESP_TLS_BASE

Starting number of ESP-TLS error codes

ESP_ERR_ESP_TLS_CANNOT_RESOLVE_HOSTNAME

Error if hostname couldn't be resolved upon tls connection

ESP_ERR_ESP_TLS_CANNOT_CREATE_SOCKET

Failed to create socket

ESP_ERR_ESP_TLS_UNSUPPORTED_PROTOCOL_FAMILY

Unsupported protocol family

ESP_ERR_ESP_TLS_FAILED_CONNECT_TO_HOST

Failed to connect to host

ESP_ERR_ESP_TLS_SOCKET_SETOPT_FAILED

failed to set/get socket option

ESP_ERR_ESP_TLS_CONNECTION_TIMEOUT

new connection in esp_tls_low_level_conn connection timeouted

ESP_ERR_ESP_TLS_SE_FAILED

ESP_ERR_ESP_TLS_TCP_CLOSED_FIN

ESP_ERR_MBEDTLS_CERT_PARTLY_OK

mbedtls parse certificates was partly successful

ESP_ERR_MBEDTLS_CTR_DRBG_SEED_FAILED

mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_SET_HOSTNAME_FAILED

mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONFIG_DEFAULTS_FAILED

mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONF_ALPN_PROTOCOLS_FAILED
mbedtls api returned error

ESP_ERR_MBEDTLS_X509_CRT_PARSE_FAILED
mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONF_OWN_CERT_FAILED
mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_SETUP_FAILED
mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_WRITE_FAILED
mbedtls api returned error

ESP_ERR_MBEDTLS_PK_PARSE_KEY_FAILED
mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_HANDSHAKE_FAILED
mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_CONF_PSK_FAILED
mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_TICKET_SETUP_FAILED
mbedtls api returned failed

ESP_ERR_WOLFSSL_SSL_SET_HOSTNAME_FAILED
wolfSSL api returned error

ESP_ERR_WOLFSSL_SSL_CONF_ALPN_PROTOCOLS_FAILED
wolfSSL api returned error

ESP_ERR_WOLFSSL_CERT_VERIFY_SETUP_FAILED
wolfSSL api returned error

ESP_ERR_WOLFSSL_KEY_VERIFY_SETUP_FAILED
wolfSSL api returned error

ESP_ERR_WOLFSSL_SSL_HANDSHAKE_FAILED
wolfSSL api returned failed

ESP_ERR_WOLFSSL_CTX_SETUP_FAILED
wolfSSL api returned failed

ESP_ERR_WOLFSSL_SSL_SETUP_FAILED
wolfSSL api returned failed

ESP_ERR_WOLFSSL_SSL_WRITE_FAILED

wolfSSL api returned failed

ESP_TLS_ERR_SSL_WANT_READ

Definition of errors reported from IO API (potentially non-blocking) in case of error:

- esp_tls_conn_read()
- esp_tls_conn_write()

ESP_TLS_ERR_SSL_WANT_WRITE

ESP_TLS_ERR_SSL_TIMEOUT

Type Definitions

typedef struct *esp_tls_last_error* ***esp_tls_error_handle_t**

typedef struct *esp_tls_last_error* **esp_tls_last_error_t**

Error structure containing relevant errors in case tls error occurred.

Enumerations

enum **esp_tls_error_type_t**

Definition of different types/sources of error codes reported from different components

Values:

enumerator **ESP_TLS_ERR_TYPE_UNKNOWN**

enumerator **ESP_TLS_ERR_TYPE_SYSTEM**

System error –errno

enumerator **ESP_TLS_ERR_TYPE_MBEDTLS**

Error code from mbedTLS library

enumerator **ESP_TLS_ERR_TYPE_MBEDTLS_CERT_FLAGS**

Certificate flags defined in mbedTLS

enumerator **ESP_TLS_ERR_TYPE_ESP**

ESP-IDF error type –esp_err_t

enumerator **ESP_TLS_ERR_TYPE_WOLFSSL**

Error code from wolfSSL library

enumerator **ESP_TLS_ERR_TYPE_WOLFSSL_CERT_FLAGS**

Certificate flags defined in wolfSSL

enumerator **ESP_TLS_ERR_TYPE_MAX**

Last err type –invalid entry

2.2.5 ESP HTTP Client

Overview

esp_http_client component provides a set of APIs for making HTTP/S requests from ESP-IDF applications. The steps to use these APIs are as follows:

- *esp_http_client_init()*: Creates an *esp_http_client_handle_t* instance, i.e., an HTTP client handle based on the given *esp_http_client_config_t* configuration. This function must be the first to be called; default values are assumed for the configuration values that are not explicitly defined by the user.
- *esp_http_client_perform()*: Performs all operations of the esp_http_client - opening the connection, exchanging data, and closing the connection (as required), while blocking the current task before its completion. All related events are invoked through the event handler (as specified in *esp_http_client_config_t*).
- *esp_http_client_cleanup()*: Closes the connection (if any) and frees up all the memory allocated to the HTTP client instance. This must be the last function to be called after the completion of operations.

Application Example

Simple example that uses ESP HTTP Client to make HTTP/S requests can be found at protocols/esp_http_client.

Basic HTTP Request

Check out the example functions http_rest_with_url and http_rest_with_hostname_path in the application example for implementation details.

Persistent Connections

Persistent connection means that the HTTP client can reuse the same connection for several exchanges. If the server does not request to close the connection with the Connection: close header, the connection is not dropped but is instead kept open and used for further requests.

To allow ESP HTTP client to take full advantage of persistent connections, one should make as many requests as possible using the same handle instance. Check out the example functions http_rest_with_url and http_rest_with_hostname_path in the application example. Here, once the connection is created, multiple requests (GET, POST, PUT, etc.) are made before the connection is closed.

HTTPS Request

ESP HTTP client supports SSL connections using **mbedTLS**, with the url configuration starting with https scheme or transport_type set to HTTP_TRANSPORT_OVER_SSL. HTTPS support can be configured via *CONFIG_ESP_HTTP_CLIENT_ENABLE_HTTPS* (enabled by default).

Note: While making HTTPS requests, if server verification is needed, an additional root certificate (in PEM format) needs to be provided to the cert_pem member in the esp_http_client_config_t configuration. Users can also use the ESP x509 Certificate Bundle for server verification using the crt_bundle_attach member of the esp_http_client_config_t configuration.

Check out the example functions https_with_url and https_with_hostname_path in the application example for implementation details of the above note.

HTTP Stream

Some applications need to open the connection and control the exchange of data actively (data streaming). In such cases, the application flow is different from regular requests. Example flow is given below:

- *esp_http_client_init()*: Create a HTTP client handle.
- esp_http_client_set_* or esp_http_client_delete_*: Modify the HTTP connection parameters (optional).
- *esp_http_client_open()*: Open the HTTP connection with write_len parameter (content length that needs to be written to server), set write_len=0 for read-only connection.
- *esp_http_client_write()*: Write data to server with a maximum length equal to write_len of *esp_http_client_open()* function; no need to call this function for write_len=0.
- *esp_http_client_fetch_headers()*: Read the HTTP Server response headers, after sending the request headers and server data (if any). Returns the content-length from the server and can be succeeded by *esp_http_client_get_status_code()* for getting the HTTP status of the connection.
- *esp_http_client_read()*: Read the HTTP stream.
- *esp_http_client_close()*: Close the connection.
- *esp_http_client_cleanup()*: Release allocated resources.

Check out the example function http_perform_as_stream_reader in the application example for implementation details.

HTTP Authentication

ESP HTTP client supports both Basic and Digest Authentication.

- Users can provide the username and password in the url or the username and password members of the esp_http_client_config_t configuration. For auth_type = HTTP_AUTH_TYPE_BASIC, the HTTP client takes only one perform operation to pass the authentication process.
- If auth_type = HTTP_AUTH_TYPE_NONE, but the username and password fields are present in the configuration, the HTTP client takes two perform operations. The client will receive the 401 Unauthorized header in its first attempt to connect to the server. Based on this information, it decides which authentication method to choose and performs it in the second operation.
- Check out the example functions http_auth_basic, http_auth_basic_redirect (for Basic authentication) and http_auth_digest (for Digest authentication) in the application example for implementation details.
- Currently, Digest authentication supports only MD5 and SHA-256 algorithms.

Examples of Authentication Configuration

- Authentication with URI

```
esp_http_client_config_t config = {
    .url = "http://user:passwd@httpbin.org/basic-auth/user/passwd",
    .auth_type = HTTP_AUTH_TYPE_BASIC,
};
```

- Authentication with username and password entry

```
esp_http_client_config_t config = {
    .url = "http://httpbin.org/basic-auth/user/passwd",
    .username = "user",
    .password = "passwd",
    .auth_type = HTTP_AUTH_TYPE_BASIC,
};
```

Event Handling

ESP HTTP Client supports event handling by triggering an event handler corresponding to the event which takes place. *esp_http_client_event_id_t* contains all the events which could occur while performing an HTTP request using the ESP HTTP Client.

To enable event handling, you just need to set a callback function using the *esp_http_client_config_t::event_handler* member.

ESP HTTP Client Diagnostic Information

Diagnostic information could be helpful to gain insights into a problem. In the case of ESP HTTP Client, the diagnostic information can be collected by registering an event handler with *the Event Loop library*. This feature has been added by keeping in mind the ESP Insights framework which collects the diagnostic information. However, this feature can also be used without any dependency on the ESP Insights framework for the diagnostic purpose. Event handler can be registered to the event loop using the *esp_event_handler_register()* function.

Expected data types for different HTTP Client events in the event loop are as follows:

- HTTP_EVENT_ERROR : `esp_http_client_handle_t`
- HTTP_EVENT_ON_CONNECTED : `esp_http_client_handle_t`
- HTTP_EVENT_HEADERS_SENT : `esp_http_client_handle_t`
- HTTP_EVENT_ON_HEADER : `esp_http_client_handle_t`
- HTTP_EVENT_ON_DATA : `esp_http_client_on_data_t`
- HTTP_EVENT_ON_FINISH : `esp_http_client_handle_t`
- HTTP_EVENT_DISCONNECTED : `esp_http_client_handle_t`
- HTTP_EVENT_REDIRECT : `esp_http_client_redirect_event_data_t`

The *esp_http_client_handle_t* received along with the event data will be valid until *HTTP_EVENT_DISCONNECTED* is not received. This handle has been sent primarily to differentiate between different client connections and must not be used for any other purpose, as it may change based on client connection state.

TLS Protocol Version

TLS protocol version to be used for the underlying TLS connection can be set in *esp_http_client_config_t*. Please refer to the **TLS Protocol Version** section in the *ESP-TLS* for more details.

The TLS protocol version for the HTTP client can be configured as follows:

```
#include "esp_http_client.h"
esp_http_client_config_t config = {
    .tls_version = ESP_HTTP_CLIENT_TLS_VER_TLS_1_2,
};
```

API Reference

Header File

- components/esp_http_client/include/esp_http_client.h
- This header file can be included with:

  ```
  #include "esp_http_client.h"
  ```

- This header file is a part of the API provided by the `esp_http_client` component. To declare that your component depends on `esp_http_client`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_http_client
  ```

  or

```
PRIV_REQUIRES esp_http_client
```

Functions

esp_http_client_handle_t **esp_http_client_init** (const *esp_http_client_config_t* *config)

Start a HTTP session This function must be the first function to call, and it returns a esp_http_client_handle_t that you must use as input to other functions in the interface. This call MUST have a corresponding call to esp_http_client_cleanup when the operation is complete.

Parameters **config** -- **[in]** The configurations, see `http_client_config_t`

Returns

- `esp_http_client_handle_t`
- NULL if any errors

esp_err_t **esp_http_client_perform** (*esp_http_client_handle_t* client)

Invoke this function after `esp_http_client_init` and all the options calls are made, and will perform the transfer as described in the options. It must be called with the same esp_http_client_handle_t as input as the esp_http_client_init call returned. esp_http_client_perform performs the entire request in either blocking or non-blocking manner. By default, the API performs request in a blocking manner and returns when done, or if it failed, and in non-blocking manner, it returns if EAGAIN/EWOULDBLOCK or EINPROGRESS is encountered, or if it failed. And in case of non-blocking request, the user may call this API multiple times unless request & response is complete or there is a failure. To enable non-blocking esp_http_client_perform(), `is_async` member of *esp_http_client_config_t* must be set while making a call to esp_http_client_init() API. You can do any amount of calls to esp_http_client_perform while using the same esp_http_client_handle_t. The underlying connection may be kept open if the server allows it. If you intend to transfer more than one file, you are even encouraged to do so. esp_http_client will then attempt to reuse the same connection for the following transfers, thus making the operations faster, less CPU intense and using less network resources. Just note that you will have to use `esp_http_client_set_**` between the invokes to set options for the following esp_http_client_perform.

Note: You must never call this function simultaneously from two places using the same client handle. Let the function return first before invoking it another time. If you want parallel transfers, you must use several esp_http_client_handle_t. This function include `esp_http_client_open` -> `esp_http_client_write` -> `esp_http_client_fetch_headers` -> `esp_http_client_read` (and option) `esp_http_client_close`.

Parameters **client** -- The esp_http_client handle

Returns

- ESP_OK on successful
- ESP_FAIL on error

esp_err_t **esp_http_client_cancel_request** (*esp_http_client_handle_t* client)

Cancel an ongoing HTTP request. This API closes the current socket and opens a new socket with the same esp_http_client context.

Parameters **client** -- The esp_http_client handle

Returns

- ESP_OK on successful
- ESP_FAIL on error
- ESP_ERR_INVALID_ARG
- ESP_ERR_INVALID_STATE

esp_err_t **esp_http_client_set_url** (*esp_http_client_handle_t* client, const char *url)

Set URL for client, when performing this behavior, the options in the URL will replace the old ones.

Parameters

- **client** -- **[in]** The esp_http_client handle
- **url** -- **[in]** The url

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_set_post_field**(*esp_http_client_handle_t* client, const char *data, int len)

Set post data, this function must be called before `esp_http_client_perform`. Note: The data parameter passed to this function is a pointer and this function will not copy the data.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **data** -- **[in]** post data pointer
- **len** -- **[in]** post length

Returns
- ESP_OK
- ESP_FAIL

int **esp_http_client_get_post_field**(*esp_http_client_handle_t* client, char **data)

Get current post field information.

Parameters
- **client** -- **[in]** The client
- **data** -- **[out]** Point to post data pointer

Returns Size of post data

esp_err_t **esp_http_client_set_header**(*esp_http_client_handle_t* client, const char *key, const char *value)

Set http request header, this function must be called after esp_http_client_init and before any perform function.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **key** -- **[in]** The header key
- **value** -- **[in]** The header value

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_get_header**(*esp_http_client_handle_t* client, const char *key, char **value)

Get http request header. The value parameter will be set to NULL if there is no header which is same as the key specified, otherwise the address of header value will be assigned to value parameter. This function must be called after `esp_http_client_init`.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **key** -- **[in]** The header key
- **value** -- **[out]** The header value

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_get_username**(*esp_http_client_handle_t* client, char **value)

Get http request username. The address of username buffer will be assigned to value parameter. This function must be called after `esp_http_client_init`.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **value** -- **[out]** The username value

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_username**(*esp_http_client_handle_t* client, const char *username)

Set http request username. The value of username parameter will be assigned to username buffer. If the username parameter is NULL then username buffer will be freed.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **username** -- **[in]** The username value

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_get_password**(*esp_http_client_handle_t* client, char **value)

Get http request password. The address of password buffer will be assigned to value parameter. This function must be called after `esp_http_client_init`.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **value** -- **[out]** The password value

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_password**(*esp_http_client_handle_t* client, const char *password)

Set http request password. The value of password parameter will be assigned to password buffer. If the password parameter is NULL then password buffer will be freed.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **password** -- **[in]** The password value

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_authtype**(*esp_http_client_handle_t* client, *esp_http_client_auth_type_t* auth_type)

Set http request auth_type.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **auth_type** -- **[in]** The esp_http_client auth type

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_get_user_data**(*esp_http_client_handle_t* client, void **data)

Get http request user_data. The value stored from the *esp_http_client_config_t* will be written to the address passed into data.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **data** -- **[out]** A pointer to the pointer that will be set to user_data.

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_user_data**(*esp_http_client_handle_t* client, void *data)

Set http request user_data. The value passed in +data+ will be available during event callbacks. No memory management will be performed on the user's behalf.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **data** -- **[in]** The pointer to the user data

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

int **esp_http_client_get_errno**(*esp_http_client_handle_t* client)

Get HTTP client session errno.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- (-1) if invalid argument
- errno

esp_err_t **esp_http_client_set_method**(*esp_http_client_handle_t* client, *esp_http_client_method_t* method)

Set http request method.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **method** -- **[in]** The method

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_timeout_ms**(*esp_http_client_handle_t* client, int timeout_ms)

Set http request timeout.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **timeout_ms** -- **[in]** The timeout value

Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_delete_header**(*esp_http_client_handle_t* client, const char *key)

Delete http request header.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **key** -- **[in]** The key

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_open**(*esp_http_client_handle_t* client, int write_len)

This function will be open the connection, write all header strings and return.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **write_len** -- **[in]** HTTP Content length need to write to the server

Returns
- ESP_OK
- ESP_FAIL

int **esp_http_client_write**(*esp_http_client_handle_t* client, const char *buffer, int len)

This function will write data to the HTTP connection previously opened by esp_http_client_open()

Parameters
- **client** -- **[in]** The esp_http_client handle
- **buffer** -- The buffer
- **len** -- **[in]** This value must not be larger than the write_len parameter provided to esp_http_client_open()

Returns
- (-1) if any errors

- Length of data written

int64_t **esp_http_client_fetch_headers** (*esp_http_client_handle_t* client)

This function need to call after esp_http_client_open, it will read from http stream, process all receive headers.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- (0) if stream doesn't contain content-length header, or chunked encoding (checked by `esp_http_client_is_chunked` response)
- (-1: ESP_FAIL) if any errors
- (-ESP_ERR_HTTP_EAGAIN = -0x7007) if call is timed-out before any data was ready
- Download data length defined by content-length header

bool **esp_http_client_is_chunked_response** (*esp_http_client_handle_t* client)

Check response data is chunked.

Parameters **client** -- **[in]** The esp_http_client handle
Returns true or false

int **esp_http_client_read** (*esp_http_client_handle_t* client, char *buffer, int len)

Read data from http stream.

Note: (-ESP_ERR_HTTP_EAGAIN = -0x7007) is returned when call is timed-out before any data was ready

---

Parameters
- **client** -- **[in]** The esp_http_client handle
- **buffer** -- The buffer
- **len** -- **[in]** The length

Returns
- (-1) if any errors
- Length of data was read

int **esp_http_client_get_status_code** (*esp_http_client_handle_t* client)

Get http response status code, the valid value if this function invoke after `esp_http_client_perform`

Parameters **client** -- **[in]** The esp_http_client handle
Returns Status code

int64_t **esp_http_client_get_content_length** (*esp_http_client_handle_t* client)

Get http response content length (from header Content-Length) the valid value if this function invoke after `esp_http_client_perform`

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- (-1) Chunked transfer
- Content-Length value as bytes

esp_err_t **esp_http_client_close** (*esp_http_client_handle_t* client)

Close http connection, still kept all http request resources.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_cleanup** (*esp_http_client_handle_t* client)

This function must be the last function to call for an session. It is the opposite of the esp_http_client_init function and must be called with the same handle as input that a esp_http_client_init call returned. This might close all connections this handle has used and possibly has kept open until now. Don't call this function if you intend to transfer more files, re-using handles is a key to good performance with esp_http_client.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- ESP_OK
- ESP_FAIL

esp_http_client_transport_t **esp_http_client_get_transport_type**(*esp_http_client_handle_t* client)

Get transport type.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- HTTP_TRANSPORT_UNKNOWN
- HTTP_TRANSPORT_OVER_TCP
- HTTP_TRANSPORT_OVER_SSL

esp_err_t **esp_http_client_set_redirection**(*esp_http_client_handle_t* client)

Set redirection URL. When received the 30x code from the server, the client stores the redirect URL provided by the server. This function will set the current URL to redirect to enable client to execute the redirection request. When `disable_auto_redirect` is set, the client will not call this function but the event `HTTP_EVENT_REDIRECT` will be dispatched giving the user control over the redirection event.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_reset_redirect_counter**(*esp_http_client_handle_t* client)

Reset the redirection counter. This is useful to reset redirect counter in cases where the same handle is used for multiple requests.

Parameters **client** -- **[in]** The esp_http_client handle
Returns
- ESP_OK
- ESP_ERR_INVALID_ARG

esp_err_t **esp_http_client_set_auth_data**(*esp_http_client_handle_t* client, const char *auth_data, int len)

On receiving a custom authentication header, this API can be invoked to set the authentication information from the header. This API can be called from the event handler.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **auth_data** -- **[in]** The authentication data received in the header
- **len** -- **[in]** length of auth_data.

Returns
- ESP_ERR_INVALID_ARG
- ESP_OK

void **esp_http_client_add_auth**(*esp_http_client_handle_t* client)

On receiving HTTP Status code 401, this API can be invoked to add authorization information.

Note: There is a possibility of receiving body message with redirection status codes, thus make sure to flush off body data after calling this API.

Parameters **client** -- **[in]** The esp_http_client handle

bool **esp_http_client_is_complete_data_received**(*esp_http_client_handle_t* client)

Checks if entire data in the response has been read without any error.

Parameters **client** -- **[in]** The esp_http_client handle
Returns

- true
- false

int **esp_http_client_read_response**(*esp_http_client_handle_t* client, char *buffer, int len)

Helper API to read larger data chunks This is a helper API which internally calls `esp_http_client_read` multiple times till the end of data is reached or till the buffer gets full.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **buffer** -- The buffer
- **len** -- **[in]** The buffer length

Returns
- Length of data was read

esp_err_t **esp_http_client_flush_response**(*esp_http_client_handle_t* client, int *len)

Process all remaining response data This uses an internal buffer to repeatedly receive, parse, and discard response data until complete data is processed. As no additional user-supplied buffer is required, this may be preferable to `esp_http_client_read_response` in situations where the content of the response may be ignored.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **len** -- Length of data discarded

Returns
- ESP_OK If successful, len will have discarded length
- ESP_FAIL If failed to read response
- ESP_ERR_INVALID_ARG If the client is NULL

esp_err_t **esp_http_client_get_url**(*esp_http_client_handle_t* client, char *url, const int len)

Get URL from client.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **url** -- **[inout]** The buffer to store URL
- **len** -- **[in]** The buffer length

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_http_client_get_chunk_length**(*esp_http_client_handle_t* client, int *len)

Get Chunk-Length from client.

Parameters
- **client** -- **[in]** The esp_http_client handle
- **len** -- **[out]** Variable to store length

Returns
- ESP_OK If successful, len will have length of current chunk
- ESP_FAIL If the server is not a chunked server
- ESP_ERR_INVALID_ARG If the client or len are NULL

Structures

struct **esp_http_client_event**

HTTP Client events data.

Public Members

esp_http_client_event_id_t **event_id**

event_id, to know the cause of the event

esp_http_client_handle_t **client**

esp_http_client_handle_t context

void ***data**

data of the event

int **data_len**

data length of data

void ***user_data**

user_data context, from *esp_http_client_config_t* user_data

char ***header_key**

For HTTP_EVENT_ON_HEADER event_id, it's store current http header key

char ***header_value**

For HTTP_EVENT_ON_HEADER event_id, it's store current http header value

struct **esp_http_client_on_data**

Argument structure for HTTP_EVENT_ON_DATA event.

Public Members

esp_http_client_handle_t **client**

Client handle

int64_t **data_process**

Total data processed

struct **esp_http_client_redirect_event_data**

Argument structure for HTTP_EVENT_REDIRECT event.

Public Members

esp_http_client_handle_t **client**

Client handle

int **status_code**

Status Code

struct **esp_http_client_config_t**

HTTP configuration.

Public Members

const char ***url**

HTTP URL, the information on the URL is most important, it overrides the other fields below, if any

const char ***host**

Domain or IP as string

int **port**

Port to connect, default depend on esp_http_client_transport_t (80 or 443)

const char ***username**

Using for Http authentication

const char ***password**

Using for Http authentication

esp_http_client_auth_type_t **auth_type**

Http authentication type, see `esp_http_client_auth_type_t`

const char ***path**

HTTP Path, if not set, default is `/`

const char ***query**

HTTP query

const char ***cert_pem**

SSL server certification, PEM format as string, if the client requires to verify server

size_t **cert_len**

Length of the buffer pointed to by cert_pem. May be 0 for null-terminated pem

const char ***client_cert_pem**

SSL client certification, PEM format as string, if the server requires to verify client

size_t **client_cert_len**

Length of the buffer pointed to by client_cert_pem. May be 0 for null-terminated pem

const char ***client_key_pem**

SSL client key, PEM format as string, if the server requires to verify client

size_t **client_key_len**

Length of the buffer pointed to by client_key_pem. May be 0 for null-terminated pem

const char ***client_key_password**

Client key decryption password string

size_t **client_key_password_len**

String length of the password pointed to by client_key_password

esp_http_client_proto_ver_t **tls_version**

TLS protocol version of the connection, e.g., TLS 1.2, TLS 1.3 (default - no preference)

const char ***user_agent**

The User Agent string to send with HTTP requests

esp_http_client_method_t **method**

HTTP Method

int **timeout_ms**

Network timeout in milliseconds

bool **disable_auto_redirect**

Disable HTTP automatic redirects

int **max_redirection_count**

Max number of redirections on receiving HTTP redirect status code, using default value if zero

int **max_authorization_retries**

Max connection retries on receiving HTTP unauthorized status code, using default value if zero. Disables authorization retry if -1

http_event_handle_cb **event_handler**

HTTP Event Handle

esp_http_client_transport_t **transport_type**

HTTP transport type, see `esp_http_client_transport_t`

int **buffer_size**

HTTP receive buffer size

int **buffer_size_tx**

HTTP transmit buffer size

void ***user_data**

HTTP user_data context

bool **is_async**

Set asynchronous mode, only supported with HTTPS for now

bool **use_global_ca_store**

Use a global ca_store for all the connections in which this bool is set.

bool **skip_cert_common_name_check**

Skip any validation of server certificate CN field

const char ***common_name**

Pointer to the string containing server certificate common name. If non-NULL, server certificate CN must match this name, If NULL, server certificate CN must match hostname.

esp_err_t (***crt_bundle_attach**)(void *conf)

Function pointer to esp_crt_bundle_attach. Enables the use of certification bundle for server verification, must be enabled in menuconfig

bool **keep_alive_enable**

Enable keep-alive timeout

int **keep_alive_idle**

Keep-alive idle time. Default is 5 (second)

int **keep_alive_interval**

Keep-alive interval time. Default is 5 (second)

int **keep_alive_count**

Keep-alive packet retry send count. Default is 3 counts

struct ifreq ***if_name**

The name of interface for data to go through. Use the default interface without setting

void ***ds_data**

Pointer for digital signature peripheral context, see ESP-TLS Documentation for more details

Macros

DEFAULT_HTTP_BUF_SIZE

ESP_ERR_HTTP_BASE

Starting number of HTTP error codes

ESP_ERR_HTTP_MAX_REDIRECT

The error exceeds the number of HTTP redirects

ESP_ERR_HTTP_CONNECT

Error open the HTTP connection

ESP_ERR_HTTP_WRITE_DATA

Error write HTTP data

ESP_ERR_HTTP_FETCH_HEADER

Error read HTTP header from server

ESP_ERR_HTTP_INVALID_TRANSPORT

There are no transport support for the input scheme

ESP_ERR_HTTP_CONNECTING

HTTP connection hasn't been established yet

ESP_ERR_HTTP_EAGAIN

Mapping of errno EAGAIN to esp_err_t

ESP_ERR_HTTP_CONNECTION_CLOSED

Read FIN from peer and the connection closed

Type Definitions

typedef struct esp_http_client ***esp_http_client_handle_t**

typedef struct *esp_http_client_event* ***esp_http_client_event_handle_t**

typedef struct *esp_http_client_event* **esp_http_client_event_t**

HTTP Client events data.

typedef struct *esp_http_client_on_data* **esp_http_client_on_data_t**

Argument structure for HTTP_EVENT_ON_DATA event.

typedef struct *esp_http_client_redirect_event_data* **esp_http_client_redirect_event_data_t**

Argument structure for HTTP_EVENT_REDIRECT event.

typedef *esp_err_t* (***http_event_handle_cb**)(*esp_http_client_event_t* *evt)

Enumerations

enum **esp_http_client_event_id_t**

HTTP Client events id.

Values:

enumerator **HTTP_EVENT_ERROR**

This event occurs when there are any errors during execution

enumerator **HTTP_EVENT_ON_CONNECTED**

Once the HTTP has been connected to the server, no data exchange has been performed

enumerator **HTTP_EVENT_HEADERS_SENT**

After sending all the headers to the server

enumerator **HTTP_EVENT_HEADER_SENT**

This header has been kept for backward compatibility and will be deprecated in future versions esp-idf

enumerator **HTTP_EVENT_ON_HEADER**

Occurs when receiving each header sent from the server

enumerator **HTTP_EVENT_ON_DATA**

Occurs when receiving data from the server, possibly multiple portions of the packet

enumerator **HTTP_EVENT_ON_FINISH**

Occurs when finish a HTTP session

enumerator **HTTP_EVENT_DISCONNECTED**

The connection has been disconnected

enumerator **HTTP_EVENT_REDIRECT**

Intercepting HTTP redirects to handle them manually

enum **esp_http_client_transport_t**

HTTP Client transport.

Values:

enumerator **HTTP_TRANSPORT_UNKNOWN**

Unknown

enumerator **HTTP_TRANSPORT_OVER_TCP**

Transport over tcp

enumerator **HTTP_TRANSPORT_OVER_SSL**

Transport over ssl

enum **esp_http_client_proto_ver_t**

Values:

enumerator **ESP_HTTP_CLIENT_TLS_VER_ANY**

enumerator **ESP_HTTP_CLIENT_TLS_VER_TLS_1_2**

enumerator **ESP_HTTP_CLIENT_TLS_VER_TLS_1_3**

enumerator **ESP_HTTP_CLIENT_TLS_VER_MAX**

enum **esp_http_client_method_t**

HTTP method.

Values:

enumerator **HTTP_METHOD_GET**

HTTP GET Method

enumerator **HTTP_METHOD_POST**

HTTP POST Method

enumerator **HTTP_METHOD_PUT**

HTTP PUT Method

enumerator **HTTP_METHOD_PATCH**

HTTP PATCH Method

enumerator **HTTP_METHOD_DELETE**

HTTP DELETE Method

enumerator **HTTP_METHOD_HEAD**

HTTP HEAD Method

enumerator **HTTP_METHOD_NOTIFY**

HTTP NOTIFY Method

enumerator **HTTP_METHOD_SUBSCRIBE**
HTTP SUBSCRIBE Method

enumerator **HTTP_METHOD_UNSUBSCRIBE**
HTTP UNSUBSCRIBE Method

enumerator **HTTP_METHOD_OPTIONS**
HTTP OPTIONS Method

enumerator **HTTP_METHOD_COPY**
HTTP COPY Method

enumerator **HTTP_METHOD_MOVE**
HTTP MOVE Method

enumerator **HTTP_METHOD_LOCK**
HTTP LOCK Method

enumerator **HTTP_METHOD_UNLOCK**
HTTP UNLOCK Method

enumerator **HTTP_METHOD_PROPFIND**
HTTP PROPFIND Method

enumerator **HTTP_METHOD_PROPPATCH**
HTTP PROPPATCH Method

enumerator **HTTP_METHOD_MKCOL**
HTTP MKCOL Method

enumerator **HTTP_METHOD_MAX**

enum **esp_http_client_auth_type_t**
HTTP Authentication type.

Values:

enumerator **HTTP_AUTH_TYPE_NONE**
No authention

enumerator **HTTP_AUTH_TYPE_BASIC**
HTTP Basic authentication

enumerator **HTTP_AUTH_TYPE_DIGEST**
HTTP Digest authentication

enum **HttpStatus_Code**
Enum for the HTTP status codes.

Values:

enumerator **HttpStatus_Ok**

enumerator **HttpStatus_MultipleChoices**

enumerator **HttpStatus_MovedPermanently**

enumerator **HttpStatus_Found**

enumerator **HttpStatus_SeeOther**

enumerator **HttpStatus_TemporaryRedirect**

enumerator **HttpStatus_PermanentRedirect**

enumerator **HttpStatus_BadRequest**

enumerator **HttpStatus_Unauthorized**

enumerator **HttpStatus_Forbidden**

enumerator **HttpStatus_NotFound**

enumerator **HttpStatus_InternalError**

2.2.6 ESP Local Control

Overview

ESP Local Control (**esp_local_ctrl**) component in ESP-IDF provides capability to control an ESP device over HTTPS or Bluetooth® Low Energy. It provides access to application defined **properties** that are available for reading/writing via a set of configurable handlers.

Initialization of the **esp_local_ctrl** service over Bluetooth Low Energy transport is performed as follows:

```
esp_local_ctrl_config_t config = {
    .transport = ESP_LOCAL_CTRL_TRANSPORT_BLE,
    .transport_config = {
        .ble = & (protocomm_ble_config_t) {
            .device_name  = SERVICE_NAME,
            .service_uuid = {
                /* LSB <---------------------------------------
                * ---------------------------------------> MSB */
                0x21, 0xd5, 0x3b, 0x8d, 0xbd, 0x75, 0x68, 0x8a,
                0xb4, 0x42, 0xeb, 0x31, 0x4a, 0x1e, 0x98, 0x3d
            }
        }
    },
    .proto_sec = {
        .version = PROTOCOM_SEC0,
```

(continues on next page)

(continued from previous page)

```
        .custom_handle = NULL,
        .sec_params = NULL,
    },
    .handlers = {
        /* User defined handler functions */
        .get_prop_values = get_property_values,
        .set_prop_values = set_property_values,
        .usr_ctx         = NULL,
        .usr_ctx_free_fn = NULL
    },
    /* Maximum number of properties that may be set */
    .max_properties = 10
};

/* Start esp_local_ctrl service */
ESP_ERROR_CHECK(esp_local_ctrl_start(&config));
```

Initialization of the **esp_local_ctrl** service over HTTPS transport is performed as follows:

```
/* Set the configuration */
httpd_ssl_config_t https_conf = HTTPD_SSL_CONFIG_DEFAULT();

/* Load server certificate */
extern const unsigned char servercert_start[] asm("_binary_servercert_pem_
↪start");
extern const unsigned char servercert_end[]   asm("_binary_servercert_pem_
↪end");
https_conf.servercert = servercert_start;
https_conf.servercert_len = servercert_end - servercert_start;

/* Load server private key */
extern const unsigned char prvtkey_pem_start[] asm("_binary_prvtkey_pem_
↪start");
extern const unsigned char prvtkey_pem_end[]   asm("_binary_prvtkey_pem_
↪end");
https_conf.prvtkey_pem = prvtkey_pem_start;
https_conf.prvtkey_len = prvtkey_pem_end - prvtkey_pem_start;

esp_local_ctrl_config_t config = {
    .transport = ESP_LOCAL_CTRL_TRANSPORT_HTTPD,
    .transport_config = {
        .httpd = &https_conf
    },
    .proto_sec = {
        .version = PROTOCOM_SEC0,
        .custom_handle = NULL,
        .sec_params = NULL,
    },
    .handlers = {
        /* User defined handler functions */
        .get_prop_values = get_property_values,
        .set_prop_values = set_property_values,
        .usr_ctx         = NULL,
        .usr_ctx_free_fn = NULL
    },
    /* Maximum number of properties that may be set */
    .max_properties = 10
};

/* Start esp_local_ctrl service */
ESP_ERROR_CHECK(esp_local_ctrl_start(&config));
```

You may set security for transport in ESP local control using following options:

1. `PROTOCOM_SEC2`: specifies that SRP6a-based key exchange and end-to-end encryption based on AES-GCM are used. This is the most preferred option as it adds a robust security with Augmented PAKE protocol, i.e., SRP6a.
2. `PROTOCOM_SEC1`: specifies that Curve25519-based key exchange and end-to-end encryption based on AES-CTR are used.
3. `PROTOCOM_SEC0`: specifies that data will be exchanged as a plain text (no security).
4. `PROTOCOM_SEC_CUSTOM`: you can define your own security requirement. Please note that you will also have to provide `custom_handle` of type `protocomm_security_t *` in this context.

Note: The respective security schemes need to be enabled through the project configuration menu. Please refer to the Enabling protocom security version section in *Protocol Communication* for more details.

Creating a Property

Now that we know how to start the **esp_local_ctrl** service, let's add a property to it. Each property must have a unique `name` (string), a `type` (e.g., enum), `flags` (bit fields) and `size`.

The `size` is to be kept 0, if we want our property value to be of variable length (e.g., if it is a string or bytestream). For data types with fixed-length property value, like int, float, etc., setting the `size` field to the right value helps **esp_local_ctrl** to perform internal checks on arguments received with write requests.

The interpretation of `type` and `flags` fields is totally upto the application, hence they may be used as enumerations, bitfields, or even simple integers. One way is to use `type` values to classify properties, while `flags` to specify characteristics of a property.

Here is an example property which is to function as a timestamp. It is assumed that the application defines `TYPE_TIMESTAMP` and `READONLY`, which are used for setting the `type` and `flags` fields here.

```
/* Create a timestamp property */
esp_local_ctrl_prop_t timestamp = {
    .name         = "timestamp",
    .type         = TYPE_TIMESTAMP,
    .size         = sizeof(int32_t),
    .flags        = READONLY,
    .ctx          = func_get_time,
    .ctx_free_fn = NULL
};

/* Now register the property */
esp_local_ctrl_add_property(&timestamp);
```

Also notice that there is a ctx field, which is set to point to some custom *func_get_time()*. This can be used inside the property get/set handlers to retrieve timestamp.

Here is an example of `get_prop_values()` handler, which is used for retrieving the timestamp.

```
static esp_err_t get_property_values(size_t props_count,
                                     const esp_local_ctrl_prop_t *props,
                                     esp_local_ctrl_prop_val_t *prop_
↪values,
                                     void *usr_ctx)
{
    for (uint32_t i = 0; i < props_count; i++) {
        ESP_LOGI(TAG, "Reading %s", props[i].name);
        if (props[i].type == TYPE_TIMESTAMP) {
            /* Obtain the timer function from ctx */
            int32_t (*func_get_time)(void) = props[i].ctx;
```

(continues on next page)

(continued from previous page)

```
            /* Use static variable for saving the value. This is␣
↪essential because the value has to be valid even after this function␣
↪returns. Alternative is to use dynamic allocation and set the free_fn␣
↪field */
            static int32_t ts = func_get_time();
            prop_values[i].data = &ts;
        }
    }
    return ESP_OK;
}
```

Here is an example of `set_prop_values()` handler. Notice how we restrict from writing to read-only properties.

```
static esp_err_t set_property_values(size_t props_count,
                                     const esp_local_ctrl_prop_t *props,
                                     const esp_local_ctrl_prop_val_t␣
↪*prop_values,
                                     void *usr_ctx)
{
    for (uint32_t i = 0; i < props_count; i++) {
        if (props[i].flags & READONLY) {
            ESP_LOGE(TAG, "Cannot write to read-only property %s",␣
↪props[i].name);
            return ESP_ERR_INVALID_ARG;
        } else {
            ESP_LOGI(TAG, "Setting %s", props[i].name);

            /* For keeping it simple, lets only log the incoming data */
            ESP_LOG_BUFFER_HEX_LEVEL(TAG, prop_values[i].data,
                                     prop_values[i].size, ESP_LOG_INFO);
        }
    }
    return ESP_OK;
}
```

For complete example see protocols/esp_local_ctrl.

Client Side Implementation

The client side implementation establishes a protocomm session with the device first, over the supported mode of transport, and then send and receive protobuf messages understood by the **esp_local_ctrl** service. The service translates these messages into requests and then call the appropriate handlers (set/get). Then, the generated response for each handler is again packed into a protobuf message and transmitted back to the client.

See below the various protobuf messages understood by the **esp_local_ctrl** service:

1. `get_prop_count` : This should simply return the total number of properties supported by the service.
2. `get_prop_values` : This accepts an array of indices and should return the information (name, type, flags) and values of the properties corresponding to those indices.
3. `set_prop_values` : This accepts an array of indices and an array of new values, which are used for setting the values of the properties corresponding to the indices.

Note that indices may or may not be the same for a property, across multiple sessions. Therefore, the client must only use the names of the properties to uniquely identify them. So, every time a new session is established, the client should first call `get_prop_count` and then `get_prop_values`, hence form an index-to-name mapping for all properties. Now when calling `set_prop_values` for a set of properties, it must first convert the names to indexes, using the created mapping. As emphasized earlier, the client must refresh the index-to-name mapping every time a new session is established with the same device.

The various protocomm endpoints provided by **esp_local_ctrl** are listed below:

Table 1: Endpoints provided by ESP Local Control

Endpoint Name (Bluetooth Low Energy + GATT Server)	URI (HTTPS Server + mDNS)	Description
esp_local_ctrl/version	https://<mdns-hostname>.local/esp_local_ctrl/version	Endpoint used for retrieving version string
esp_local_ctrl/control	https://<mdns-hostname>.local/esp_local_ctrl/control	Endpoint used for sending or receiving control messages

API Reference

Header File

- components/esp_local_ctrl/include/esp_local_ctrl.h
- This header file can be included with:

```
#include "esp_local_ctrl.h"
```

- This header file is a part of the API provided by the `esp_local_ctrl` component. To declare that your component depends on `esp_local_ctrl`, add the following to your CMakeLists.txt:

```
REQUIRES esp_local_ctrl
```

or

```
PRIV_REQUIRES esp_local_ctrl
```

Functions

const *esp_local_ctrl_transport_t* ***esp_local_ctrl_get_transport_ble**(void)

Function for obtaining BLE transport mode.

const *esp_local_ctrl_transport_t* ***esp_local_ctrl_get_transport_httpd**(void)

Function for obtaining HTTPD transport mode.

esp_err_t **esp_local_ctrl_start**(const *esp_local_ctrl_config_t* *config)

Start local control service.

Parameters **config** -- **[in]** Pointer to configuration structure

Returns

- ESP_OK : Success
- ESP_FAIL : Failure

esp_err_t **esp_local_ctrl_stop**(void)

Stop local control service.

esp_err_t **esp_local_ctrl_add_property**(const *esp_local_ctrl_prop_t* *prop)

Add a new property.

This adds a new property and allocates internal resources for it. The total number of properties that could be added is limited by configuration option `max_properties`

Parameters **prop** -- **[in]** Property description structure

Returns

- ESP_OK : Success
- ESP_FAIL : Failure

esp_err_t **esp_local_ctrl_remove_property**(const char *name)

Remove a property.

This finds a property by name, and releases the internal resources which are associated with it.

Parameters **name** -- **[in]** Name of the property to remove
Returns
- ESP_OK : Success
- ESP_ERR_NOT_FOUND : Failure

const *esp_local_ctrl_prop_t* ***esp_local_ctrl_get_property**(const char *name)

Get property description structure by name.

This API may be used to get a property's context structure `esp_local_ctrl_prop_t` when its name is known

Parameters **name** -- **[in]** Name of the property to find
Returns
- Pointer to property
- NULL if not found

esp_err_t **esp_local_ctrl_set_handler**(const char *ep_name, *protocomm_req_handler_t* handler, void *user_ctx)

Register protocomm handler for a custom endpoint.

This API can be called by the application to register a protocomm handler for an endpoint after the local control service has started.

Note: In case of BLE transport the names and uuids of all custom endpoints must be provided beforehand as a part of the `protocomm_ble_config_t` structure set in `esp_local_ctrl_config_t`, and passed to `esp_local_ctrl_start()`.

Parameters
- **ep_name** -- **[in]** Name of the endpoint
- **handler** -- **[in]** Endpoint handler function
- **user_ctx** -- **[in]** User data

Returns
- ESP_OK : Success
- ESP_FAIL : Failure

Unions

union **esp_local_ctrl_transport_config_t**

#include <esp_local_ctrl.h> Transport mode (BLE / HTTPD) configuration.

Public Members

esp_local_ctrl_transport_config_ble_t ***ble**

This is same as `protocomm_ble_config_t`. See `protocomm_ble.h` for available configuration parameters.

esp_local_ctrl_transport_config_httpd_t ***httpd**

This is same as `httpd_ssl_config_t`. See `esp_https_server.h` for available configuration parameters.

Structures

struct **esp_local_ctrl_prop**

Property description data structure, which is to be populated and passed to the `esp_local_ctrl_add_property()` function.

Once a property is added, its structure is available for read-only access inside `get_prop_values()` and `set_prop_values()` handlers.

Public Members

char ***name**

Unique name of property

uint32_t **type**

Type of property. This may be set to application defined enums

size_t **size**

Size of the property value, which:

- if zero, the property can have values of variable size
- if non-zero, the property can have values of fixed size only, therefore, checks are performed internally by esp_local_ctrl when setting the value of such a property

uint32_t **flags**

Flags set for this property. This could be a bit field. A flag may indicate property behavior, e.g. read-only / constant

void ***ctx**

Pointer to some context data relevant for this property. This will be available for use inside the `get_prop_values` and `set_prop_values` handlers as a part of this property structure. When set, this is valid throughout the lifetime of a property, till either the property is removed or the esp_local_ctrl service is stopped.

void (***ctx_free_fn**)(void *ctx)

Function used by esp_local_ctrl to internally free the property context when `esp_local_ctrl_remove_property()` or `esp_local_ctrl_stop()` is called.

struct **esp_local_ctrl_prop_val**

Property value data structure. This gets passed to the `get_prop_values()` and `set_prop_values()` handlers for the purpose of retrieving or setting the present value of a property.

Public Members

void ***data**

Pointer to memory holding property value

size_t **size**

Size of property value

void (*`free_fn`)(void *data)

This may be set by the application in `get_prop_values()` handler to tell `esp_local_ctrl` to call this function on the data pointer above, for freeing its resources after sending the `get_prop_values` response.

struct `esp_local_ctrl_handlers`

Handlers for receiving and responding to local control commands for getting and setting properties.

Public Members

esp_err_t (*`get_prop_values`)(size_t props_count, const *esp_local_ctrl_prop_t* props[], *esp_local_ctrl_prop_val_t* prop_values[], void *usr_ctx)

Handler function to be implemented for retrieving current values of properties.

Note: If any of the properties have fixed sizes, the size field of corresponding element in `prop_values` need to be set

---

Param props_count **[in]** Total elements in the props array
Param props **[in]** Array of properties, the current values for which have been requested by the client
Param prop_values **[out]** Array of empty property values, the elements of which need to be populated with the current values of those properties specified by props argument
Param usr_ctx **[in]** This provides value of the `usr_ctx` field of `esp_local_ctrl_handlers_t` structure
Return Returning different error codes will convey the corresponding protocol level errors to the client :
- ESP_OK : Success
- ESP_ERR_INVALID_ARG : InvalidArgument
- ESP_ERR_INVALID_STATE : InvalidProto
- All other error codes : InternalError

esp_err_t (*`set_prop_values`)(size_t props_count, const *esp_local_ctrl_prop_t* props[], const *esp_local_ctrl_prop_val_t* prop_values[], void *usr_ctx)

Handler function to be implemented for changing values of properties.

Note: If any of the properties have variable sizes, the size field of the corresponding element in `prop_values` must be checked explicitly before making any assumptions on the size.

Param props_count **[in]** Total elements in the props array
Param props **[in]** Array of properties, the values for which the client requests to change
Param prop_values **[in]** Array of property values, the elements of which need to be used for updating those properties specified by props argument
Param usr_ctx **[in]** This provides value of the `usr_ctx` field of `esp_local_ctrl_handlers_t` structure
Return Returning different error codes will convey the corresponding protocol level errors to the client :
- ESP_OK : Success
- ESP_ERR_INVALID_ARG : InvalidArgument
- ESP_ERR_INVALID_STATE : InvalidProto
- All other error codes : InternalError

void ***usr_ctx**

Context pointer to be passed to above handler functions upon invocation. This is different from the property level context, as this is valid throughout the lifetime of the `esp_local_ctrl` service, and freed only when the service is stopped.

void (***usr_ctx_free_fn**)(void *usr_ctx)

Pointer to function which will be internally invoked on `usr_ctx` for freeing the context resources when `esp_local_ctrl_stop()` is called.

struct **esp_local_ctrl_proto_sec_cfg**

Protocom security configs

Public Members

esp_local_ctrl_proto_sec_t **version**

This sets protocom security version, sec0/sec1 or custom If custom, user must provide handle via `proto_sec_custom_handle` below

void ***custom_handle**

Custom security handle if security is set custom via `proto_sec` above This handle must follow `protocomm_security_t` signature

const void ***pop**

Proof of possession to be used for local control. Could be NULL.

const void ***sec_params**

Pointer to security params (NULL if not needed). This is not needed for protocomm security 0 This pointer should hold the struct of type esp_local_ctrl_security1_params_t for protocomm security 1 and esp_local_ctrl_security2_params_t for protocomm security 2 respectively. Could be NULL.

struct **esp_local_ctrl_config**

Configuration structure to pass to `esp_local_ctrl_start()`

Public Members

const *esp_local_ctrl_transport_t* ***transport**

Transport layer over which service will be provided

esp_local_ctrl_transport_config_t **transport_config**

Transport layer over which service will be provided

esp_local_ctrl_proto_sec_cfg_t **proto_sec**

Security version and POP

esp_local_ctrl_handlers_t **handlers**

Register handlers for responding to get/set requests on properties

size_t **max_properties**

This limits the number of properties that are available at a time

Macros

ESP_LOCAL_CTRL_TRANSPORT_BLE

ESP_LOCAL_CTRL_TRANSPORT_HTTPD

Type Definitions

typedef struct *esp_local_ctrl_prop* **esp_local_ctrl_prop_t**

Property description data structure, which is to be populated and passed to the `esp_local_ctrl_add_property()` function.

Once a property is added, its structure is available for read-only access inside `get_prop_values()` and `set_prop_values()` handlers.

typedef struct *esp_local_ctrl_prop_val* **esp_local_ctrl_prop_val_t**

Property value data structure. This gets passed to the `get_prop_values()` and `set_prop_values()` handlers for the purpose of retrieving or setting the present value of a property.

typedef struct *esp_local_ctrl_handlers* **esp_local_ctrl_handlers_t**

Handlers for receiving and responding to local control commands for getting and setting properties.

typedef struct esp_local_ctrl_transport **esp_local_ctrl_transport_t**

Transport mode (BLE / HTTPD) over which the service will be provided.

This is forward declaration of a private structure, implemented internally by `esp_local_ctrl`.

typedef struct *protocomm_ble_config* **esp_local_ctrl_transport_config_ble_t**

Configuration for transport mode BLE.

This is a forward declaration for `protocomm_ble_config_t`. To use this, application must set CONFIG_BT_ENABLED and include `protocomm_ble.h`.

typedef struct *httpd_config* **esp_local_ctrl_transport_config_httpd_t**

Configuration for transport mode HTTPD.

This is a forward declaration for `httpd_ssl_config_t` (for HTTPS) or `httpd_config_t` (for HTTP)

typedef enum *esp_local_ctrl_proto_sec* **esp_local_ctrl_proto_sec_t**

Security types for esp_local_control.

typedef *protocomm_security1_params_t* **esp_local_ctrl_security1_params_t**

typedef *protocomm_security2_params_t* **esp_local_ctrl_security2_params_t**

typedef struct *esp_local_ctrl_proto_sec_cfg* **esp_local_ctrl_proto_sec_cfg_t**

Protocom security configs

typedef struct *esp_local_ctrl_config* **esp_local_ctrl_config_t**

Configuration structure to pass to `esp_local_ctrl_start()`

Enumerations

enum **esp_local_ctrl_proto_sec**

Security types for esp_local_control.

Values:

enumerator **PROTOCOM_SEC0**

enumerator **PROTOCOM_SEC1**

enumerator **PROTOCOM_SEC2**

enumerator **PROTOCOM_SEC_CUSTOM**

2.2.7 ESP Serial Slave Link

Overview

Espressif provides several chips that can work as slaves. These slave devices rely on some common buses, and have their own communication protocols over those buses. The `esp_serial_slave_link` component is designed for the master to communicate with ESP slave devices through those protocols over the bus drivers.

After an `esp_serial_slave_link` device is initialized properly, the application can use it to communicate with the ESP slave devices conveniently.

Note: The ESP-IDF component `esp_serial_slave_link` has been moved from ESP-IDF since version v5.0 to a separate repository:

- ESSL component on GitHub

To add ESSL component in your project, please run `idf.py add-dependency espressif/esp_serial_slave_link`.

Espressif Device Protocols

For more details about Espressif device protocols, see the following documents.

ESP SPI Slave HD (Half Duplex) Mode Protocol

SPI Slave Capabilities of Espressif Chips

	ESP32	ESP32-S2	ESP32-C3	ESP32-S3	ESP32-C2	ESP32-C6	ESP32-H2	ESP32-P4
SPI Slave HD	N	Y (v2)	Y (v2)	Y (v2)	Y (v2)	Y (v2)	Y (v2)	Y (v2)
Tohost intr		N	N	N	N	N	N	N
Frhost intr		2 *	2 *	2 *	2 *	2 *	2 *	2 *
TX DMA		Y	Y	Y	Y	Y	Y	Y
RX DMA		Y	Y	Y	Y	Y	Y	Y
Shared registers		72	64	64	64	64	64	64

Introduction In the half duplex mode, the master has to use the protocol defined by the slave to communicate with the slave. Each transaction may consist of the following phases (listed by the order they should exist):

- Command: 8-bit, master to slave
  This phase determines the rest phases of the transactions. See *Supported Commands*.
- Address: 8-bit, master to slave, optional
  For some commands (WRBUF, RDBUF), this phase specifies the address of the shared register to write to/read from. For other commands with this phase, they are meaningless but still have to exist in the transaction.
- Dummy: 8-bit, floating, optional
  This phase is the turnaround time between the master and the slave on the bus, and also provides enough time for the slave to prepare the data to send to the master.
- Data: variable length, the direction is also determined by the command.
  This may be a data OUT phase, in which the direction is slave to master, or a data IN phase, in which the direction is master to slave.

The **direction** means which side (master or slave) controls the MOSI, MISO, WP, and HD pins.

Data IO Modes In some IO modes, more data wires can be used to send the data. As a result, the SPI clock cycles required for the same amount of data will be less than in the 1-bit mode. For example, in QIO mode, address and data (IN and OUT) should be sent on all 4 data wires (MOSI, MISO, WP, and HD). Here are the modes supported by the ESP32-S2 SPI slave and the wire number (WN) used in corresponding modes.

Mode	Command WN	Address WN	Dummy cycles	Data WN
1-bit	1	1	1	1
DOUT	1	1	4	2
DIO	1	2	4	2
QOUT	1	1	4	4
QIO	1	4	4	4
QPI	4	4	4	4

Normally, which mode is used is determined by the command sent by the master (See *Supported Commands*), except the QPI mode.

QPI Mode The QPI mode is a special state of the SPI Slave. The master can send the ENQPI command to put the slave into the QPI mode state. In the QPI mode, the command is also sent in 4-bit, thus it is not compatible with the normal modes. The master should only send QPI commands when the slave is in QPI mode. To exit from the QPI mode, master can send the EXQPI command.

Supported Commands

Note: The command name is in a master-oriented direction. For example, WRBUF means master writes the buffer of slave.

Name	Description	Command	Address	Data
WRBUF	Write buffer	0x01	Buf addr	master to slave, no longer than buffer size
RDBUF	Read buffer	0x02	Buf addr	slave to master, no longer than buffer size
WRDMA	Write DMA	0x03	8 bits	master to slave, no longer than length provided by slave
RDDMA	Read DMA	0x04	8 bits	slave to master, no longer than length provided by slave
SEG_DONE	Segments done	0x05	•	•
ENQPI	Enter QPI mode	0x06	•	•
WR_DONE	Write segments done	0x07	•	•
CMD8	Interrupt	0x08	•	•
CMD9	Interrupt	0x09	•	•
CMDA	Interrupt	0x0A	•	•
EXQPI	Exit QPI mode	0xDD	•	•

Moreover, WRBUF, RDBUF, WRDMA, and RDDMA commands have their 2-bit and 4-bit version. To do transactions in 2-bit or 4-bit mode, send the original command ORed by the corresponding command mask below. For example, command 0xA1 means WRBUF in QIO mode.

Mode	Mask
1-bit	0x00
DOUT	0x10
DIO	0x50
QOUT	0x20
QIO	0xA0
QPI	0xA0

Segment Transaction Mode Segment transaction mode is the only mode supported by the SPI Slave HD driver for now. In this mode, for a transaction the slave loads onto the DMA, the master is allowed to read or write in segments. In this way, the master does not have to prepare a large buffer as the size of data provided by the slave. After the master finishes reading/writing a buffer, it has to send the corresponding termination command to the slave as a synchronization signal. The slave driver will update new data (if exist) onto the DMA upon seeing the termination command.

The termination command is WR_DONE (0x07) for WRDMA and CMD8 (0x08) for RDDMA.

Here is an example for the flow the master read data from the slave DMA:

1. The slave loads 4092 bytes of data onto the RDDMA.
2. The master do seven RDDMA transactions, each of them is 512 bytes long, and reads the first 3584 bytes from the slave.

3. The master do the last RDDMA transaction of 512 bytes (equal, longer, or shorter than the total length loaded by the slave are all allowed). The first 508 bytes are valid data from the slave, while the last 4 bytes are meaningless bytes.
4. The master sends CMD8 to the slave.
5. The slave loads another 4092 bytes of data onto the RDDMA.
6. The master can start new reading transactions after it sends the CMD8.

Terminology

- ESSL: Abbreviation for ESP Serial Slave Link, the component described by this document.
- Master: The device running the `esp_serial_slave_link` component.
- ESSL device: a virtual device on the master associated with an ESP slave device. The device context has the knowledge of the slave protocol above the bus, relying on some bus drivers to communicate with the slave.
- ESSL device handle: a handle to ESSL device context containing the configuration, status and data required by the ESSL component. The context stores the driver configurations, communication state, data shared by master and slave, etc.
  - The context should be initialized before it is used, and get deinitialized if not used any more. The master application operates on the ESSL device through this handle.
- ESP slave: the slave device connected to the bus, which ESSL component is designed to communicate with.
- Bus: The bus over which the master and the slave communicate with each other.
- Slave protocol: The special communication protocol specified by Espressif HW/SW over the bus.
- TX buffer num: a counter, which is on the slave and can be read by the master, indicates the accumulated buffer numbers that the slave has loaded to the hardware to receive data from the master.
- RX data size: a counter, which is on the slave and can be read by the master, indicates the accumulated data size that the slave has loaded to the hardware to send to the master.

Services Provided by ESP Slave

There are some common services provided by the Espressif slaves:

1. Tohost Interrupts: The slave can inform the master about certain events by the interrupt line. (optional)
2. Frhost Interrupts: The master can inform the slave about certain events.
3. TX FIFO (master to slave): The slave can receive data from the master in units of receiving buffers.
   The slave updates the TX buffer num to inform the master how much data it can receive, and the master then read the TX buffer num, and take off the used buffer number to know how many buffers are remaining.
4. RX FIFO (slave to master): The slave can send data in stream to the master. The SDIO slave can also indicate it has new data to send to master by the interrupt line.
   The slave updates the RX data size to inform the master how much data it has prepared to send, and then the master read the data size, and take off the data length it has already received to know how many data is remaining.
5. Shared registers: The master can read some part of the registers on the slave, and also write these registers to let the slave read.

The services provided by the slave depends on the slave's model. See *SPI Slave Capabilities of Espressif Chips* for more details.

Initialization of ESP Serial Slave Link

ESP SDIO Slave The ESP SDIO slave link (ESSL SDIO) devices relies on the SDMMC component. It includes the usage of communicating with ESP SDIO Slave device via the SDMMC Host or SDSPI Host feature. The ESSL device should be initialized as below:

1. Initialize a SDMMC card (see :doc:` Document of SDMMC driver </api-reference/storage/sdmmc>`) structure.
2. Call `sdmmc_card_init()` to initialize the card.

3. Initialize the ESSL device with *essl_sdio_config_t*. The `card` member should be the `sdmmc_card_t` got in step 2, and the `recv_buffer_size` member should be filled correctly according to pre-negotiated value.
4. Call *essl_init()* to do initialization of the SDIO part.
5. Call *essl_wait_for_ready()* to wait for the slave to be ready.

ESP SPI Slave

Note: If you are communicating with the ESP SDIO Slave device through SPI interface, you should use the *SDIO interface* instead.

Has not been supported yet.

APIs

After the initialization process above is performed, you can call the APIs below to make use of the services provided by the slave:

Tohost Interrupts (Optional)

1. Call *essl_get_intr_ena()* to know which events trigger the interrupts to the master.
2. Call *essl_set_intr_ena()* to set the events that trigger interrupts to the master.
3. Call *essl_wait_int()* to wait until interrupt from the slave, or timeout.
4. When interrupt is triggered, call *essl_get_intr()* to know which events are active, and call *essl_clear_intr()* to clear them.

Frhost Interrupts

1. Call *essl_send_slave_intr()* to trigger general purpose interrupt of the slave.

TX FIFO

1. Call *essl_get_tx_buffer_num()* to know how many buffers the slave has prepared to receive data from the master. This is optional. The master will poll `tx_buffer_num` when it tries to send packets to the slave, until the slave has enough buffer or timeout.
2. Call *essl_send_packet()* to send data to the slave.

RX FIFO

1. Call *essl_get_rx_data_size()* to know how many data the slave has prepared to send to the master. This is optional. When the master tries to receive data from the slave, it updates the `rx_data_size` for once, if the current `rx_data_size` is shorter than the buffer size the master prepared to receive. And it may poll the `rx_data_size` if the `rx_data_size` keeps 0, until timeout.
2. Call *essl_get_packet()* to receive data from the slave.

Reset Counters (Optional) Call *essl_reset_cnt()* to reset the internal counter if you find the slave has reset its counter.

Application Example

The example below shows how ESP32-S3 SDIO host and slave communicate with each other. The host uses the ESSL SDIO:

peripherals/sdio

Please refer to the specific example README.md for details.

API Reference

Header File

- components/driver/test_apps/components/esp_serial_slave_link/include/esp_serial_slave_link/essl.h

Functions

esp_err_t **essl_init** (*essl_handle_t* handle, uint32_t wait_ms)

Initialize the slave.

Parameters
- **handle** -- Handle of an ESSL device.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: If success
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- Other value returned from lower layer `init`.

esp_err_t **essl_wait_for_ready** (*essl_handle_t* handle, uint32_t wait_ms)

Wait for interrupt of an ESSL slave device.

Parameters
- **handle** -- Handle of an ESSL device.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: If success
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- One of the error codes from SDMMC host controller

esp_err_t **essl_get_tx_buffer_num** (*essl_handle_t* handle, uint32_t *out_tx_num, uint32_t wait_ms)

Get buffer num for the host to send data to the slave. The buffers are size of `buffer_size`.

Parameters
- **handle** -- Handle of a ESSL device.
- **out_tx_num** -- Output of buffer num that host can send data to ESSL slave.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: This API is not supported in this mode
- One of the error codes from SDMMC/SPI host controller

esp_err_t **essl_get_rx_data_size** (*essl_handle_t* handle, uint32_t *out_rx_size, uint32_t wait_ms)

Get the size, in bytes, of the data that the ESSL slave is ready to send

Parameters
- **handle** -- Handle of an ESSL device.
- **out_rx_size** -- Output of data size to read from slave, in bytes
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: This API is not supported in this mode
- One of the error codes from SDMMC/SPI host controller

esp_err_t **essl_reset_cnt** (*essl_handle_t* handle)

Reset the counters of this component. Usually you don't need to do this unless you know the slave is reset.

Parameters **handle** -- Handle of an ESSL device.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: This API is not supported in this mode
- ESP_ERR_INVALID_ARG: Invalid argument, handle is not init.

esp_err_t **essl_send_packet** (*essl_handle_t* handle, const void *start, size_t length, uint32_t wait_ms)

Send a packet to the ESSL Slave. The Slave receives the packet into buffers whose size is `buffer_size` (configured during initialization).

Parameters

- **handle** -- Handle of an ESSL device.
- **start** -- Start address of the packet to send
- **length** -- Length of data to send, if the packet is over-size, the it will be divided into blocks and hold into different buffers automatically.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG: Invalid argument, handle is not init or other argument is not valid.
- ESP_ERR_TIMEOUT: No buffer to use, or error ftrom SDMMC host controller.
- ESP_ERR_NOT_FOUND: Slave is not ready for receiving.
- ESP_ERR_NOT_SUPPORTED: This API is not supported in this mode
- One of the error codes from SDMMC/SPI host controller.

esp_err_t **essl_get_packet** (*essl_handle_t* handle, void *out_data, size_t size, size_t *out_length, uint32_t wait_ms)

Get a packet from ESSL slave.

Parameters

- **handle** -- Handle of an ESSL device.
- **out_data** -- **[out]** Data output address
- **size** -- The size of the output buffer, if the buffer is smaller than the size of data to receive from slave, the driver returns `ESP_ERR_NOT_FINISHED`
- **out_length** -- **[out]** Output of length the data actually received from slave.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns

- ESP_OK Success: All the data has been read from the slave.
- ESP_ERR_INVALID_ARG: Invalid argument, The handle is not initialized or the other arguments are invalid.
- ESP_ERR_NOT_FINISHED: Read was successful, but there is still data remaining.
- ESP_ERR_NOT_FOUND: Slave is not ready to send data.
- ESP_ERR_NOT_SUPPORTED: This API is not supported in this mode
- One of the error codes from SDMMC/SPI host controller.

esp_err_t **essl_write_reg** (*essl_handle_t* handle, uint8_t addr, uint8_t value, uint8_t *value_o, uint32_t wait_ms)

Write general purpose R/W registers (8-bit) of ESSL slave.

Note: sdio 28-31 are reserved, the lower API helps to skip.

Parameters

- **handle** -- Handle of an ESSL device.
- **addr** -- Address of register to write. For SDIO, valid address: 0-59. For SPI, see `essl_spi.h`
- **value** -- Value to write to the register.
- **value_o** -- Output of the returned written value.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns

- ESP_OK Success
- One of the error codes from SDMMC/SPI host controller

esp_err_t **essl_read_reg** (*essl_handle_t* handle, uint8_t add, uint8_t *value_o, uint32_t wait_ms)

Read general purpose R/W registers (8-bit) of ESSL slave.

Parameters
- **handle** -- Handle of a essl device.
- **add** -- Address of register to read. For SDIO, Valid address: 0-27, 32-63 (28-31 reserved, return interrupt bits on read). For SPI, see essl_spi.h
- **value_o** -- Output value read from the register.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK Success
- One of the error codes from SDMMC/SPI host controller

esp_err_t **essl_wait_int** (*essl_handle_t* handle, uint32_t wait_ms)

wait for an interrupt of the slave

Parameters
- **handle** -- Handle of an ESSL device.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: If interrupt is triggered.
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- ESP_ERR_TIMEOUT: No interrupts before timeout.

esp_err_t **essl_clear_intr** (*essl_handle_t* handle, uint32_t intr_mask, uint32_t wait_ms)

Clear interrupt bits of ESSL slave. All the bits set in the mask will be cleared, while other bits will stay the same.

Parameters
- **handle** -- Handle of an ESSL device.
- **intr_mask** -- Mask of interrupt bits to clear.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- One of the error codes from SDMMC host controller

esp_err_t **essl_get_intr** (*essl_handle_t* handle, uint32_t *intr_raw, uint32_t *intr_st, uint32_t wait_ms)

Get interrupt bits of ESSL slave.

Parameters
- **handle** -- Handle of an ESSL device.
- **intr_raw** -- Output of the raw interrupt bits. Set to NULL if only masked bits are read.
- **intr_st** -- Output of the masked interrupt bits. set to NULL if only raw bits are read.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_INVALID_ARG: If both intr_raw and intr_st are NULL.
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- One of the error codes from SDMMC host controller

esp_err_t **essl_set_intr_ena** (*essl_handle_t* handle, uint32_t ena_mask, uint32_t wait_ms)

Set interrupt enable bits of ESSL slave. The slave only sends interrupt on the line when there is a bit both the raw status and the enable are set.

Parameters
- **handle** -- Handle of an ESSL device.
- **ena_mask** -- Mask of the interrupt bits to enable.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- One of the error codes from SDMMC host controller

esp_err_t **essl_get_intr_ena**(*essl_handle_t* handle, uint32_t *ena_mask_o, uint32_t wait_ms)

Get interrupt enable bits of ESSL slave.

Parameters
- **handle** -- Handle of an ESSL device.
- **ena_mask_o** -- Output of interrupt bit enable mask.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK Success
- One of the error codes from SDMMC host controller

esp_err_t **essl_send_slave_intr**(*essl_handle_t* handle, uint32_t intr_mask, uint32_t wait_ms)

Send interrupts to slave. Each bit of the interrupt will be triggered.

Parameters
- **handle** -- Handle of an ESSL device.
- **intr_mask** -- Mask of interrupt bits to send to slave.
- **wait_ms** -- Millisecond to wait before timeout, will not wait at all if set to 0-9.

Returns
- ESP_OK: Success
- ESP_ERR_NOT_SUPPORTED: Current device does not support this function.
- One of the error codes from SDMMC host controller

Type Definitions

typedef struct essl_dev_t ***essl_handle_t**

Handle of an ESSL device.

Header File

- components/driver/test_apps/components/esp_serial_slave_link/include/esp_serial_slave_link/essl_sdio.h

Functions

esp_err_t **essl_sdio_init_dev**(*essl_handle_t* *out_handle, const *essl_sdio_config_t* *config)

Initialize the ESSL SDIO device and get its handle.

Parameters
- **out_handle** -- Output of the handle.
- **config** -- Configuration for the ESSL SDIO device.

Returns
- ESP_OK: on success
- ESP_ERR_NO_MEM: memory exhausted.

esp_err_t **essl_sdio_deinit_dev**(*essl_handle_t* handle)

Deinitialize and free the space used by the ESSL SDIO device.

Parameters **handle** -- Handle of the ESSL SDIO device to deinit.

Returns
- ESP_OK: on success
- ESP_ERR_INVALID_ARG: wrong handle passed

Structures

struct **essl_sdio_config_t**

Configuration for the ESSL SDIO device.

Public Members

sdmmc_card_t ***card**

The initialized sdmmc card pointer of the slave.

int **recv_buffer_size**

The pre-negotiated recv buffer size used by both the host and the slave.

Header File

- components/driver/test_apps/components/esp_serial_slave_link/include/esp_serial_slave_link/essl_spi.h

Functions

esp_err_t **essl_spi_init_dev**(*essl_handle_t* *out_handle, const *essl_spi_config_t* *init_config)

Initialize the ESSL SPI device function list and get its handle.

Parameters
- **out_handle** -- **[out]** Output of the handle
- **init_config** -- Configuration for the ESSL SPI device

Returns
- ESP_OK: On success
- ESP_ERR_NO_MEM: Memory exhausted
- ESP_ERR_INVALID_STATE: SPI driver is not initialized
- ESP_ERR_INVALID_ARG: Wrong register ID

esp_err_t **essl_spi_deinit_dev**(*essl_handle_t* handle)

Deinitialize the ESSL SPI device and free the memory used by the device.

Parameters **handle** -- Handle of the ESSL SPI device

Returns
- ESP_OK: On success
- ESP_ERR_INVALID_STATE: ESSL SPI is not in use

esp_err_t **essl_spi_read_reg**(void *arg, uint8_t addr, uint8_t *out_value, uint32_t wait_ms)

Read from the shared registers.

Note: The registers for Master/Slave synchronization are reserved. Do not use them. (see `rx_sync_reg` in *`essl_spi_config_t`*)

Parameters
- **arg** -- Context of the component. (Member `arg` from `essl_handle_t`)
- **addr** -- Address of the shared registers. (Valid: 0 ~ SOC_SPI_MAXIMUM_BUFFER_SIZE, registers for M/S sync are reserved, see note1).
- **out_value** -- **[out]** Read buffer for the shared registers.
- **wait_ms** -- Time to wait before timeout (reserved for future use, user should set this to 0).

Returns
- ESP_OK: success
- ESP_ERR_INVALID_STATE: ESSL SPI has not been initialized.
- ESP_ERR_INVALID_ARG: The address argument is not valid. See note 1.
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_get_packet**(void *arg, void *out_data, size_t size, uint32_t wait_ms)

Get a packet from Slave.

Parameters

- **arg** -- Context of the component. (Member `arg` from `essl_handle_t`)
- **out_data** -- **[out]** Output data address
- **size** -- The size of the output data.
- **wait_ms** -- Time to wait before timeout (reserved for future use, user should set this to 0).

Returns

- ESP_OK: On Success
- ESP_ERR_INVALID_STATE: ESSL SPI has not been initialized.
- ESP_ERR_INVALID_ARG: The output data address is neither DMA capable nor 4 byte-aligned
- ESP_ERR_INVALID_SIZE: Master requires `size` bytes of data but Slave did not load enough bytes.

esp_err_t **essl_spi_write_reg**(void *arg, uint8_t addr, uint8_t value, uint8_t *out_value, uint32_t wait_ms)

Write to the shared registers.

Note: The registers for Master/Slave synchronization are reserved. Do not use them. (see `tx_sync_reg` in *essl_spi_config_t*)

Note: Feature of checking the actual written value (`out_value`) is not supported.

Parameters

- **arg** -- Context of the component. (Member `arg` from `essl_handle_t`)
- **addr** -- Address of the shared registers. (Valid: 0 ~ SOC_SPI_MAXIMUM_BUFFER_SIZE, registers for M/S sync are reserved, see note1)
- **value** -- Buffer for data to send, should be align to 4.
- **out_value** -- **[out]** Not supported, should be set to NULL.
- **wait_ms** -- Time to wait before timeout (reserved for future use, user should set this to 0).

Returns

- ESP_OK: success
- ESP_ERR_INVALID_STATE: ESSL SPI has not been initialized.
- ESP_ERR_INVALID_ARG: The address argument is not valid. See note 1.
- ESP_ERR_NOT_SUPPORTED: Should set `out_value` to NULL. See note 2.
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_send_packet**(void *arg, const void *data, size_t size, uint32_t wait_ms)

Send a packet to Slave.

Parameters

- **arg** -- Context of the component. (Member `arg` from `essl_handle_t`)
- **data** -- Address of the data to send
- **size** -- Size of the data to send.
- **wait_ms** -- Time to wait before timeout (reserved for future use, user should set this to 0).

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_STATE: ESSL SPI has not been initialized.
- ESP_ERR_INVALID_ARG: The data address is not DMA capable
- ESP_ERR_INVALID_SIZE: Master will send `size` bytes of data but Slave did not load enough RX buffer

void **essl_spi_reset_cnt**(void *arg)

Reset the counter in Master context.

Note: Shall only be called if the slave has reset its counter. Else, Slave and Master would be desynchronized

---

Parameters **arg** -- Context of the component. (Member `arg` from `essl_handle_t`)

esp_err_t **essl_spi_rdbuf**(*spi_device_handle_t* spi, uint8_t *out_data, int addr, int len, uint32_t flags)

Read the shared buffer from the slave in ISR way.

Note: The slave's HW doesn't guarantee the data in one SPI transaction is consistent. It sends data in unit of byte. In other words, if the slave SW attempts to update the shared register when a rdbuf SPI transaction is in-flight, the data got by the master will be the combination of bytes of different writes of slave SW.

Note: `out_data` should be prepared in words and in the DRAM. The buffer may be written in words by the DMA. When a byte is written, the remaining bytes in the same word will also be overwritten, even the `len` is shorter than a word.

Parameters

- **spi** -- SPI device handle representing the slave
- **out_data** -- **[out]** Buffer for read data, strongly suggested to be in the DRAM and aligned to 4
- **addr** -- Address of the slave shared buffer
- **len** -- Length to read
- **flags** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: on success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_rdbuf_polling**(*spi_device_handle_t* spi, uint8_t *out_data, int addr, int len, uint32_t flags)

Read the shared buffer from the slave in polling way.

Note: `out_data` should be prepared in words and in the DRAM. The buffer may be written in words by the DMA. When a byte is written, the remaining bytes in the same word will also be overwritten, even the `len` is shorter than a word.

Parameters

- **spi** -- SPI device handle representing the slave
- **out_data** -- **[out]** Buffer for read data, strongly suggested to be in the DRAM and aligned to 4
- **addr** -- Address of the slave shared buffer
- **len** -- Length to read
- **flags** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: on success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_wrbuf**(*spi_device_handle_t* spi, const uint8_t *data, int addr, int len, uint32_t flags)

Write the shared buffer of the slave in ISR way.

Note: `out_data` should be prepared in words and in the DRAM. The buffer may be written in words by the DMA. When a byte is written, the remaining bytes in the same word will also be overwritten, even the `len` is shorter than a word.

Parameters

- **`spi`** -- SPI device handle representing the slave
- **`data`** -- Buffer for data to send, strongly suggested to be in the DRAM
- **`addr`** -- Address of the slave shared buffer,
- **`len`** -- Length to write
- **`flags`** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **`essl_spi_wrbuf_polling`**(*spi_device_handle_t* spi, const uint8_t *data, int addr, int len, uint32_t flags)

Write the shared buffer of the slave in polling way.

Note: `out_data` should be prepared in words and in the DRAM. The buffer may be written in words by the DMA. When a byte is written, the remaining bytes in the same word will also be overwritten, even the `len` is shorter than a word.

Parameters

- **`spi`** -- SPI device handle representing the slave
- **`data`** -- Buffer for data to send, strongly suggested to be in the DRAM
- **`addr`** -- Address of the slave shared buffer,
- **`len`** -- Length to write
- **`flags`** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_polling_transmit`.

esp_err_t **`essl_spi_rddma`**(*spi_device_handle_t* spi, uint8_t *out_data, int len, int seg_len, uint32_t flags)

Receive long buffer in segments from the slave through its DMA.

Note: This function combines several :cpp:func:`essl_spi_rddma_seg` and one :cpp:func:`essl_spi_rddma_done` at the end. Used when the slave is working in segment mode.

Parameters

- **`spi`** -- SPI device handle representing the slave
- **`out_data`** -- **[out]** Buffer to hold the received data, strongly suggested to be in the DRAM and aligned to 4
- **`len`** -- Total length of data to receive.
- **`seg_len`** -- Length of each segment, which is not larger than the maximum transaction length allowed for the spi device. Suggested to be multiples of 4. When set < 0, means send all data in one segment (the `rddma_done` will still be sent.)
- **`flags`** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_rddma_seg** (*spi_device_handle_t* spi, uint8_t *out_data, int seg_len, uint32_t flags)

Read one data segment from the slave through its DMA.

Note: To read long buffer, call :cpp:func:`essl_spi_rddma` instead.

Parameters

- **spi** -- SPI device handle representing the slave
- **out_data** -- **[out]** Buffer to hold the received data. strongly suggested to be in the DRAM and aligned to 4
- **seg_len** -- Length of this segment
- **flags** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_rddma_done** (*spi_device_handle_t* spi, uint32_t flags)

Send the `rddma_done` command to the slave. Upon receiving this command, the slave will stop sending the current buffer even there are data unsent, and maybe prepare the next buffer to send.

Note: This is required only when the slave is working in segment mode.

Parameters

- **spi** -- SPI device handle representing the slave
- **flags** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_wrdma** (*spi_device_handle_t* spi, const uint8_t *data, int len, int seg_len, uint32_t flags)

Send long buffer in segments to the slave through its DMA.

Note: This function combines several :cpp:func:`essl_spi_wrdma_seg` and one :cpp:func:`essl_spi_wrdma_done` at the end. Used when the slave is working in segment mode.

Parameters

- **spi** -- SPI device handle representing the slave
- **data** -- Buffer for data to send, strongly suggested to be in the DRAM
- **len** -- Total length of data to send.
- **seg_len** -- Length of each segment, which is not larger than the maximum transaction length allowed for the spi device. Suggested to be multiples of 4. When set < 0, means send all data in one segment (the `wrdma_done` will still be sent.)
- **flags** -- `SPI_TRANS_*` flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:`spi_device_transmit`.

esp_err_t **essl_spi_wrdma_seg** (*spi_device_handle_t* spi, const uint8_t *data, int seg_len, uint32_t flags)

Send one data segment to the slave through its DMA.

Note: To send long buffer, call :cpp:func:`essl_spi_wrdma` instead.

Parameters

- **spi** -- SPI device handle representing the slave
- **data** -- Buffer for data to send, strongly suggested to be in the DRAM
- **seg_len** -- Length of this segment
- **flags** -- SPI_TRANS_* flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:spi_device_transmit.

esp_err_t **essl_spi_wrdma_done**(*spi_device_handle_t* spi, uint32_t flags)

Send the wrdma_done command to the slave. Upon receiving this command, the slave will stop receiving, process the received data, and maybe prepare the next buffer to receive.

Note: This is required only when the slave is working in segment mode.

Parameters

- **spi** -- SPI device handle representing the slave
- **flags** -- SPI_TRANS_* flags to control the transaction mode of the transaction to send.

Returns

- ESP_OK: success
- or other return value from :cpp:func:spi_device_transmit.

Structures

struct **essl_spi_config_t**

Configuration of ESSL SPI device.

Public Members

spi_device_handle_t ***spi**

Pointer to SPI device handle.

uint32_t **tx_buf_size**

The pre-negotiated Master TX buffer size used by both the host and the slave.

uint8_t **tx_sync_reg**

The pre-negotiated register ID for Master-TX-SLAVE-RX synchronization. 1 word (4 Bytes) will be reserved for the synchronization.

uint8_t **rx_sync_reg**

The pre-negotiated register ID for Master-RX-Slave-TX synchronization. 1 word (4 Bytes) will be reserved for the synchronization.

2.2.8 ESP x509 Certificate Bundle

Overview

The ESP x509 Certificate Bundle API provides an easy way to include a bundle of custom x509 root certificates for TLS server verification.

Note: The bundle is currently not available when using WolfSSL.

The bundle comes with the complete list of root certificates from Mozilla's NSS root certificate store. Using the gen_crt_bundle.py python utility, the certificates' subject name and public key are stored in a file and embedded in the ESP32-S3 binary.

When generating the bundle you may choose between:

- The full root certificate bundle from Mozilla, containing more than 130 certificates. The current bundle was updated Mon Mar 11 15:25:27 2024 GMT.
- A pre-selected filter list of the name of the most commonly used root certificates, reducing the amount of certificates to around 41 while still having around 90% absolute usage coverage and 99% market share coverage according to SSL certificate authorities statistics.

In addition, it is possible to specify a path to a certificate file or a directory containing certificates which then will be added to the generated bundle.

Note: Trusting all root certificates means the list will have to be updated if any of the certificates are retracted. This includes removing them from `cacrt_all.pem`.

Configuration

Most configuration is done through menuconfig. CMake generates the bundle according to the configuration and embed it.

- *CONFIG_MBEDTLS_CERTIFICATE_BUNDLE*: automatically build and attach the bundle.
- *CONFIG_MBEDTLS_DEFAULT_CERTIFICATE_BUNDLE*: decide which certificates to include from the complete root certificate list.
- *CONFIG_MBEDTLS_CUSTOM_CERTIFICATE_BUNDLE_PATH*: specify the path of any additional certificates to embed in the bundle.

To enable the bundle when using ESP-TLS simply pass the function pointer to the bundle attach function:

```
esp_tls_cfg_t cfg = {
     .crt_bundle_attach = esp_crt_bundle_attach,
};
```

This is done to avoid embedding the certificate bundle unless activated by the user.

If using mbedTLS directly then the bundle may be activated by directly calling the attach function during the setup process:

```
mbedtls_ssl_config conf;
mbedtls_ssl_config_init(&conf);

esp_crt_bundle_attach(&conf);
```

Generating the List of Root Certificates

The list of root certificates comes from Mozilla's NSS root certificate store, which can be found here

The list can be downloaded and created by running the script `mk-ca-bundle.pl` that is distributed as a part of curl.

Another alternative would be to download the finished list directly from the curl website: CA certificates extracted from Mozilla

The common certificates bundle were made by selecting the authorities with a market share of more than 1% from w3tech's SSL Survey.

These authorities were then used to pick the names of the certificates for the filter list, `cmn_crt_authorities.csv`, from this list provided by Mozilla.

Updating the Certificate Bundle

The bundle is embedded into the app and can be updated along with the app by an OTA update. If you want to include a more up-to-date bundle than the bundle currently included in ESP-IDF, then the certificate list can be downloaded from Mozilla as described in *Generating the List of Root Certificates*.

Periodic Sync

The bundle is kept updated by periodic sync with the Mozilla's NSS root certificate store. The deprecated certs from the upstream bundle are added to deprecated list (for compatibility reasons) in ESP-IDF minor or patch release. If required, the deprecated certs can be added to the default bundle by enabling *CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEPRECATED_LIST*. The deprecated certs shall be removed (reset) on the next major ESP-IDF release.

Application Examples

Simple HTTPS example that uses ESP-TLS to establish a secure socket connection using the certificate bundle with two custom certificates added for verification: protocols/https_x509_bundle.

HTTPS example that uses ESP-TLS and the default bundle: protocols/https_request.

HTTPS example that uses mbedTLS and the default bundle: protocols/https_mbedtls.

API Reference

Header File

- components/mbedtls/esp_crt_bundle/include/esp_crt_bundle.h
- This header file can be included with:

```
#include "esp_crt_bundle.h"
```

- This header file is a part of the API provided by the `mbedtls` component. To declare that your component depends on `mbedtls`, add the following to your CMakeLists.txt:

```
REQUIRES mbedtls
```

or

```
PRIV_REQUIRES mbedtls
```

Functions

esp_err_t **esp_crt_bundle_attach**(void *conf)

Attach and enable use of a bundle for certificate verification.

Attach and enable use of a bundle for certificate verification through a verification callback. If no specific bundle has been set through esp_crt_bundle_set() it will default to the bundle defined in menuconfig and embedded in the binary.

Parameters **conf** -- **[in]** The config struct for the SSL connection.

Returns

- ESP_OK if adding certificates was successful.
- Other if an error occured or an action must be taken by the calling process.

void **esp_crt_bundle_detach**(mbedtls_ssl_config *conf)

Disable and dealloc the certification bundle.

Removes the certificate verification callback and deallocates used resources

Parameters **conf** -- **[in]** The config struct for the SSL connection.

esp_err_t **esp_crt_bundle_set**(const uint8_t *x509_bundle, size_t bundle_size)

Set the default certificate bundle used for verification.

Overrides the default certificate bundle only in case of successful initialization. In most use cases the bundle should be set through menuconfig. The bundle needs to be sorted by subject name since binary search is used to find certificates.

Parameters

- **x509_bundle** -- **[in]** A pointer to the certificate bundle.
- **bundle_size** -- **[in]** Size of the certificate bundle in bytes.

Returns

- ESP_OK if adding certificates was successful.
- Other if an error occured or an action must be taken by the calling process.

2.2.9 HTTP Server

Overview

The HTTP Server component provides an ability for running a lightweight web server on ESP32-S3. Following are detailed steps to use the API exposed by HTTP Server:

- *httpd_start()*: Creates an instance of HTTP server, allocate memory/resources for it depending upon the specified configuration and outputs a handle to the server instance. The server has both, a listening socket (TCP) for HTTP traffic, and a control socket (UDP) for control signals, which are selected in a round robin fashion in the server task loop. The task priority and stack size are configurable during server instance creation by passing `httpd_config_t` structure to `httpd_start()`. TCP traffic is parsed as HTTP requests and, depending on the requested URI, user registered handlers are invoked which are supposed to send back HTTP response packets.
- *httpd_stop()*: This stops the server with the provided handle and frees up any associated memory/resources. This is a blocking function that first signals a halt to the server task and then waits for the task to terminate. While stopping, the task closes all open connections, removes registered URI handlers and resets all session context data to empty.
- *httpd_register_uri_handler()*: A URI handler is registered by passing object of type `httpd_uri_t` structure which has members including `uri` name, `method` type (eg. `HTTPD_GET/HTTPD_POST/HTTPD_PUT` etc.), function pointer of type `esp_err_t *handler (httpd_req_t *req)` and `user_ctx` pointer to user context data.

Application Example

```
/* Our URI handler function to be called during GET /uri request */
esp_err_t get_handler(httpd_req_t *req)
{
    /* Send a simple response */
    const char resp[] = "URI GET Response";
    httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
    return ESP_OK;
```

(continues on next page)

(continued from previous page)

```
}

/* Our URI handler function to be called during POST /uri request */
esp_err_t post_handler(httpd_req_t *req)
{
    /* Destination buffer for content of HTTP POST request.
     * httpd_req_recv() accepts char* only, but content could
     * as well be any binary data (needs type casting).
     * In case of string data, null termination will be absent, and
     * content length would give length of string */
    char content[100];

    /* Truncate if content length larger than the buffer */
    size_t recv_size = MIN(req->content_len, sizeof(content));

    int ret = httpd_req_recv(req, content, recv_size);
    if (ret <= 0) {  /* 0 return value indicates connection closed */
        /* Check if timeout occurred */
        if (ret == HTTPD_SOCK_ERR_TIMEOUT) {
            /* In case of timeout one can choose to retry calling
             * httpd_req_recv(), but to keep it simple, here we
             * respond with an HTTP 408 (Request Timeout) error */
            httpd_resp_send_408(req);
        }
        /* In case of error, returning ESP_FAIL will
         * ensure that the underlying socket is closed */
        return ESP_FAIL;
    }

    /* Send a simple response */
    const char resp[] = "URI POST Response";
    httpd_resp_send(req, resp, HTTPD_RESP_USE_STRLEN);
    return ESP_OK;
}

/* URI handler structure for GET /uri */
httpd_uri_t uri_get = {
    .uri      = "/uri",
    .method   = HTTP_GET,
    .handler  = get_handler,
    .user_ctx = NULL
};

/* URI handler structure for POST /uri */
httpd_uri_t uri_post = {
    .uri      = "/uri",
    .method   = HTTP_POST,
    .handler  = post_handler,
    .user_ctx = NULL
};

/* Function for starting the webserver */
httpd_handle_t start_webserver(void)
{
    /* Generate default configuration */
    httpd_config_t config = HTTPD_DEFAULT_CONFIG();

    /* Empty handle to esp_http_server */
    httpd_handle_t server = NULL;

    /* Start the httpd server */
```

(continues on next page)

(continued from previous page)

```
    if (httpd_start(&server, &config) == ESP_OK) {
        /* Register URI handlers */
        httpd_register_uri_handler(server, &uri_get);
        httpd_register_uri_handler(server, &uri_post);
    }
    /* If server failed to start, handle will be NULL */
    return server;
}

/* Function for stopping the webserver */
void stop_webserver(httpd_handle_t server)
{
    if (server) {
        /* Stop the httpd server */
        httpd_stop(server);
    }
}
```

Simple HTTP Server Example Check HTTP server example under protocols/http_server/simple where handling of arbitrary content lengths, reading request headers and URL query parameters, and setting response headers is demonstrated.

Persistent Connections

HTTP server features persistent connections, allowing for the reuse of the same connection (session) for several transfers, all the while maintaining context specific data for the session. Context data may be allocated dynamically by the handler in which case a custom function may need to be specified for freeing this data when the connection/session is closed.

Persistent Connections Example

```
/* Custom function to free context */
void free_ctx_func(void *ctx)
{
    /* Could be something other than free */
    free(ctx);
}

esp_err_t adder_post_handler(httpd_req_t *req)
{
    /* Create session's context if not already available */
    if (! req->sess_ctx) {
        req->sess_ctx = malloc(sizeof(ANY_DATA_TYPE));  /*!< Pointer to context␣
↪data */
        req->free_ctx = free_ctx_func;                  /*!< Function to free␣
↪context data */
    }

    /* Access context data */
    ANY_DATA_TYPE *ctx_data = (ANY_DATA_TYPE *)req->sess_ctx;

    /* Respond */
    ..............
    ..............
    ..............

    return ESP_OK;
}
```

Check the example under protocols/http_server/persistent_sockets.

Websocket Server

The HTTP server component provides websocket support. The websocket feature can be enabled in menuconfig using the *CONFIG_HTTPD_WS_SUPPORT* option. Please refer to the protocols/http_server/ws_echo_server example which demonstrates usage of the websocket feature.

Event Handling

ESP HTTP server has various events for which a handler can be triggered by *the Event Loop library* when the particular event occurs. The handler has to be registered using `esp_event_handler_register()`. This helps in event handling for ESP HTTP server.

`esp_http_server_event_id_t` has all the events which can happen for ESP HTTP server.

Expected data type for different ESP HTTP server events in event loop:

- HTTP_SERVER_EVENT_ERROR : `httpd_err_code_t`
- HTTP_SERVER_EVENT_START : `NULL`
- HTTP_SERVER_EVENT_ON_CONNECTED : `int`
- HTTP_SERVER_EVENT_ON_HEADER : `int`
- HTTP_SERVER_EVENT_HEADERS_SENT : `int`
- HTTP_SERVER_EVENT_ON_DATA : `esp_http_server_event_data`
- HTTP_SERVER_EVENT_SENT_DATA : `esp_http_server_event_data`
- HTTP_SERVER_EVENT_DISCONNECTED : `int`
- HTTP_SERVER_EVENT_STOP : `NULL`

API Reference

Header File

- components/esp_http_server/include/esp_http_server.h
- This header file can be included with:

```
#include "esp_http_server.h"
```

- This header file is a part of the API provided by the `esp_http_server` component. To declare that your component depends on `esp_http_server`, add the following to your CMakeLists.txt:

```
REQUIRES esp_http_server
```

or

```
PRIV_REQUIRES esp_http_server
```

Functions

esp_err_t **httpd_start** (*httpd_handle_t* *handle, const *httpd_config_t* *config)

Starts the web server.

Create an instance of HTTP server and allocate memory/resources for it depending upon the specified configuration.

Example usage:

```
//Function for starting the webserver
httpd_handle_t start_webserver(void)
{
// Generate default configuration
```

(continues on next page)

(continued from previous page)

```
httpd_config_t config = HTTPD_DEFAULT_CONFIG();

// Empty handle to http_server
     httpd_handle_t server = NULL;

// Start the httpd server
if (httpd_start(&server, &config) == ESP_OK) {
// Register URI handlers
          httpd_register_uri_handler(server, &uri_get);
          httpd_register_uri_handler(server, &uri_post);
     }
// If server failed to start, handle will be NULL
return server;
}
```

Parameters

- **config** -- **[in]** Configuration for new instance of the server
- **handle** -- **[out]** Handle to newly created instance of the server. NULL on error

Returns

- ESP_OK : Instance created successfully
- ESP_ERR_INVALID_ARG : Null argument(s)
- ESP_ERR_HTTPD_ALLOC_MEM : Failed to allocate memory for instance
- ESP_ERR_HTTPD_TASK : Failed to launch server task

esp_err_t **httpd_stop**(*httpd_handle_t* handle)

Stops the web server.

Deallocates memory/resources used by an HTTP server instance and deletes it. Once deleted the handle can no longer be used for accessing the instance.

Example usage:

```
// Function for stopping the webserver
void stop_webserver(httpd_handle_t server)
{
// Ensure handle is non NULL
if (server != NULL) {
// Stop the httpd server
          httpd_stop(server);
     }
}
```

Parameters **handle** -- **[in]** Handle to server returned by httpd_start

Returns

- ESP_OK : Server stopped successfully
- ESP_ERR_INVALID_ARG : Handle argument is Null

esp_err_t **httpd_register_uri_handler**(*httpd_handle_t* handle, const *httpd_uri_t* *uri_handler)

Registers a URI handler.

Example usage:

```
esp_err_t my_uri_handler(httpd_req_t* req)
{
// Recv , Process and Send
    ....
    ....
    ....
```

(continues on next page)

(continued from previous page)

```
// Fail condition
if (....) {
// Return fail to close session //
return ESP_FAIL;
    }

// On success
return ESP_OK;
}

// URI handler structure
httpd_uri_t my_uri {
    .uri      = "/my_uri/path/xyz",
    .method   = HTTPD_GET,
    .handler  = my_uri_handler,
    .user_ctx = NULL
};

// Register handler
if (httpd_register_uri_handler(server_handle, &my_uri) != ESP_OK) {
// If failed to register handler
   ....
}
```

Note: URI handlers can be registered in real time as long as the server handle is valid.

Parameters
- **handle** -- **[in]** handle to HTTPD server instance
- **uri_handler** -- **[in]** pointer to handler that needs to be registered

Returns
- ESP_OK : On successfully registering the handler
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_HANDLERS_FULL : If no slots left for new handler
- ESP_ERR_HTTPD_HANDLER_EXISTS : If handler with same URI and method is already registered

esp_err_t **httpd_unregister_uri_handler**(*httpd_handle_t* handle, const char *uri, *httpd_method_t* method)

Unregister a URI handler.

Parameters
- **handle** -- **[in]** handle to HTTPD server instance
- **uri** -- **[in]** URI string
- **method** -- **[in]** HTTP method

Returns
- ESP_OK : On successfully deregistering the handler
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_NOT_FOUND : Handler with specified URI and method not found

esp_err_t **httpd_unregister_uri**(*httpd_handle_t* handle, const char *uri)

Unregister all URI handlers with the specified uri string.

Parameters
- **handle** -- **[in]** handle to HTTPD server instance
- **uri** -- **[in]** uri string specifying all handlers that need to be deregisterd

Returns
- ESP_OK : On successfully deregistering all such handlers

- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_NOT_FOUND : No handler registered with specified uri string

esp_err_t **httpd_sess_set_recv_override** (*httpd_handle_t* hd, int sockfd, *httpd_recv_func_t* recv_func)

Override web server's receive function (by session FD)

This function overrides the web server's receive function. This same function is used to read HTTP request packets.

Note: This API is supposed to be called either from the context of

- an http session APIs where sockfd is a valid parameter
- a URI handler where sockfd is obtained using httpd_req_to_sockfd()

Parameters

- **hd** -- **[in]** HTTPD instance handle
- **sockfd** -- **[in]** Session socket FD
- **recv_func** -- **[in]** The receive function to be set for this session

Returns

- ESP_OK : On successfully registering override
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **httpd_sess_set_send_override** (*httpd_handle_t* hd, int sockfd, *httpd_send_func_t* send_func)

Override web server's send function (by session FD)

This function overrides the web server's send function. This same function is used to send out any response to any HTTP request.

Note: This API is supposed to be called either from the context of

- an http session APIs where sockfd is a valid parameter
- a URI handler where sockfd is obtained using httpd_req_to_sockfd()

Parameters

- **hd** -- **[in]** HTTPD instance handle
- **sockfd** -- **[in]** Session socket FD
- **send_func** -- **[in]** The send function to be set for this session

Returns

- ESP_OK : On successfully registering override
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **httpd_sess_set_pending_override** (*httpd_handle_t* hd, int sockfd, *httpd_pending_func_t* pending_func)

Override web server's pending function (by session FD)

This function overrides the web server's pending function. This function is used to test for pending bytes in a socket.

Note: This API is supposed to be called either from the context of

- an http session APIs where sockfd is a valid parameter
- a URI handler where sockfd is obtained using httpd_req_to_sockfd()

Parameters

- **hd** -- **[in]** HTTPD instance handle
- **sockfd** -- **[in]** Session socket FD

- **pending_func** -- **[in]** The receive function to be set for this session

Returns

- ESP_OK : On successfully registering override
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **httpd_req_async_handler_begin** (*httpd_req_t* *r, *httpd_req_t* **out)

Start an asynchronous request. This function can be called in a request handler to get a request copy that can be used on a async thread.

Note:

- This function is necessary in order to handle multiple requests simultaneously. See examples/async_requests for example usage.
- You must call httpd_req_async_handler_complete() when you are done with the request.

Parameters

- **r** -- **[in]** The request to create an async copy of
- **out** -- **[out]** A newly allocated request which can be used on an async thread

Returns

- ESP_OK : async request object created

esp_err_t **httpd_req_async_handler_complete** (*httpd_req_t* *r)

Mark an asynchronous request as completed. This will.

- free the request memory
- relinquish ownership of the underlying socket, so it can be reused.
- allow the http server to close our socket if needed (lru_purge_enable)

Note: If async requests are not marked completed, eventually the server will no longer accept incoming connections. The server will log a "httpd_accept_conn: error in accept (23)" message if this happens.

Parameters **r** -- **[in]** The request to mark async work as completed

Returns

- ESP_OK : async request was marked completed

int **httpd_req_to_sockfd** (*httpd_req_t* *r)

Get the Socket Descriptor from the HTTP request.

This API will return the socket descriptor of the session for which URI handler was executed on reception of HTTP request. This is useful when user wants to call functions that require session socket fd, from within a URI handler, ie. : httpd_sess_get_ctx(), httpd_sess_trigger_close(), httpd_sess_update_lru_counter().

Note: This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.

Parameters **r** -- **[in]** The request whose socket descriptor should be found

Returns

- Socket descriptor : The socket descriptor for this request
- -1 : Invalid/NULL request pointer

int **httpd_req_recv** (*httpd_req_t* *r, char *buf, size_t buf_len)

API to read content data from the HTTP request.

This API will read HTTP content data from the HTTP request into provided buffer. Use content_len provided in httpd_req_t structure to know the length of data to be fetched. If content_len is too large for the buffer then user may have to make multiple calls to this function, each time fetching 'buf_len' number of bytes, while the pointer to content data is incremented internally by the same number.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- If an error is returned, the URI handler must further return an error. This will ensure that the erroneous socket is closed and cleaned up by the web server.
- Presently Chunked Encoding is not supported

Parameters

- **r** -- **[in]** The request being responded to
- **buf** -- **[in]** Pointer to a buffer that the data will be read into
- **buf_len** -- **[in]** Length of the buffer

Returns

- Bytes : Number of bytes read into the buffer successfully
- 0 : Buffer length parameter is zero / connection closed by peer
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket recv()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket recv()

size_t **httpd_req_get_hdr_value_len** (*httpd_req_t* *r, const char *field)

Search for a field in request headers and return the string length of it's value.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once httpd_resp_send() API is called all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters

- **r** -- **[in]** The request being responded to
- **field** -- **[in]** The header field to be searched in the request

Returns

- Length : If field is found in the request URL
- Zero : Field not found / Invalid request / Null arguments

esp_err_t **httpd_req_get_hdr_value_str** (*httpd_req_t* *r, const char *field, char *val, size_t val_size)

Get the value string of a field from the request headers.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once httpd_resp_send() API is called all request headers are purged, so request headers need be copied into separate buffers if they are required later.
- If output size is greater than input, then the value is truncated, accompanied by truncation error as return value.
- Use httpd_req_get_hdr_value_len() to know the right buffer length

---

Parameters

- **r** -- **[in]** The request being responded to
- **field** -- **[in]** The field to be searched in the header
- **val** -- **[out]** Pointer to the buffer into which the value will be copied if the field is found
- **val_size** -- **[in]** Size of the user buffer "val"

Returns

- ESP_OK : Field found in the request header and value string copied
- ESP_ERR_NOT_FOUND : Key not found
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_INVALID_REQ : Invalid HTTP request pointer
- ESP_ERR_HTTPD_RESULT_TRUNC : Value string truncated

size_t **httpd_req_get_url_query_len**(*httpd_req_t* *r)

Get Query string length from the request URL.

Note: This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid

---

Parameters **r** -- **[in]** The request being responded to

Returns

- Length : Query is found in the request URL
- Zero : Query not found / Null arguments / Invalid request

esp_err_t **httpd_req_get_url_query_str**(*httpd_req_t* *r, char *buf, size_t buf_len)

Get Query string from the request URL.

Note:

- Presently, the user can fetch the full URL query string, but decoding will have to be performed by the user. Request headers can be read using httpd_req_get_hdr_value_str() to know the 'Content-Type' (eg. Content-Type: application/x-www-form-urlencoded) and then the appropriate decoding algorithm needs to be applied.
- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid
- If output size is greater than input, then the value is truncated, accompanied by truncation error as return value
- Prior to calling this function, one can use httpd_req_get_url_query_len() to know the query string length beforehand and hence allocate the buffer of right size (usually query string length + 1 for null termination) for storing the query string

---

Parameters

- **r** -- **[in]** The request being responded to
- **buf** -- **[out]** Pointer to the buffer into which the query string will be copied (if found)
- **buf_len** -- **[in]** Length of output buffer

Returns

- ESP_OK : Query is found in the request URL and copied to buffer
- ESP_ERR_NOT_FOUND : Query not found
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_INVALID_REQ : Invalid HTTP request pointer
- ESP_ERR_HTTPD_RESULT_TRUNC : Query string truncated

esp_err_t **httpd_query_key_value**(const char *qry, const char *key, char *val, size_t val_size)

Helper function to get a URL query tag from a query string of the type param1=val1&param2=val2.

Note:

- The components of URL query string (keys and values) are not URLdecoded. The user must check for 'Content-Type' field in the request headers and then depending upon the specified encoding (URLencoded or otherwise) apply the appropriate decoding algorithm.
- If actual value size is greater than val_size, then the value is truncated, accompanied by truncation error as return value.

Parameters

- **qry** -- **[in]** Pointer to query string
- **key** -- **[in]** The key to be searched in the query string
- **val** -- **[out]** Pointer to the buffer into which the value will be copied if the key is found
- **val_size** -- **[in]** Size of the user buffer "val"

Returns

- ESP_OK : Key is found in the URL query string and copied to buffer
- ESP_ERR_NOT_FOUND : Key not found
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESULT_TRUNC : Value string truncated

esp_err_t **httpd_req_get_cookie_val**(*httpd_req_t* *req, const char *cookie_name, char *val, size_t *val_size)

Get the value string of a cookie value from the "Cookie" request headers by cookie name.

Parameters

- **req** -- **[in]** Pointer to the HTTP request
- **cookie_name** -- **[in]** The cookie name to be searched in the request
- **val** -- **[out]** Pointer to the buffer into which the value of cookie will be copied if the cookie is found
- **val_size** -- **[inout]** Pointer to size of the user buffer "val". This variable will contain cookie length if ESP_OK is returned and required buffer length incase ESP_ERR_HTTPD_RESULT_TRUNC is returned.

Returns

- ESP_OK : Key is found in the cookie string and copied to buffer
- ESP_ERR_NOT_FOUND : Key not found
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESULT_TRUNC : Value string truncated
- ESP_ERR_NO_MEM : Memory allocation failure

bool **httpd_uri_match_wildcard**(const char *uri_template, const char *uri_to_match, size_t match_upto)

Test if a URI matches the given wildcard template.

Template may end with "?" to make the previous character optional (typically a slash), "*" for a wildcard match, and "?*" to make the previous character optional, and if present, allow anything to follow.

Example:

- * matches everything
- /foo/? matches /foo and /foo/
- /foo/* (sans the backslash) matches /foo/ and /foo/bar, but not /foo or /fo
- /foo/?* or /foo/*? (sans the backslash) matches /foo/, /foo/bar, and also /foo, but not /foox or /fo

The special characters "?" and "*" anywhere else in the template will be taken literally.

Parameters

- **uri_template** -- **[in]** URI template (pattern)
- **uri_to_match** -- **[in]** URI to be matched

- **match_upto** -- **[in]** how many characters of the URI buffer to test (there may be trailing query string etc.)

Returns true if a match was found

esp_err_t **httpd_resp_send** (*httpd_req_t* *r, const char *buf, ssize_t buf_len)

API to send a complete HTTP response.

This API will send the data as an HTTP response to the request. This assumes that you have the entire response ready in a single buffer. If you wish to send response in incremental chunks use httpd_resp_send_chunk() instead.

If no status code and content-type were set, by default this will send 200 OK status code and content type as text/html. You may call the following functions before this API to configure the response headers : httpd_resp_set_status() - for setting the HTTP status string, httpd_resp_set_type() - for setting the Content Type, httpd_resp_set_hdr() - for appending any additional field value entries in the response header

---

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, the request has been responded to.
- No additional data can then be sent for the request.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters

- **r** -- **[in]** The request being responded to
- **buf** -- **[in]** Buffer from where the content is to be fetched
- **buf_len** -- **[in]** Length of the buffer, HTTPD_RESP_USE_STRLEN to use strlen()

Returns

- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null request pointer
- ESP_ERR_HTTPD_RESP_HDR : Essential headers are too large for internal buffer
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request

esp_err_t **httpd_resp_send_chunk** (*httpd_req_t* *r, const char *buf, ssize_t buf_len)

API to send one HTTP chunk.

This API will send the data as an HTTP response to the request. This API will use chunked-encoding and send the response in the form of chunks. If you have the entire response contained in a single buffer, please use httpd_resp_send() instead.

If no status code and content-type were set, by default this will send 200 OK status code and content type as text/html. You may call the following functions before this API to configure the response headers httpd_resp_set_status() - for setting the HTTP status string, httpd_resp_set_type() - for setting the Content Type, httpd_resp_set_hdr() - for appending any additional field value entries in the response header

---

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- When you are finished sending all your chunks, you must call this function with buf_len as 0.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters

- **r** -- **[in]** The request being responded to

- **buf** -- **[in]** Pointer to a buffer that stores the data
- **buf_len** -- **[in]** Length of the buffer, HTTPD_RESP_USE_STRLEN to use strlen()

Returns

- ESP_OK : On successfully sending the response packet chunk
- ESP_ERR_INVALID_ARG : Null request pointer
- ESP_ERR_HTTPD_RESP_HDR : Essential headers are too large for internal buffer
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

static inline *esp_err_t* **httpd_resp_sendstr**(*httpd_req_t* *r, const char *str)

API to send a complete string as HTTP response.

This API simply calls http_resp_send with buffer length set to string length assuming the buffer contains a null terminated string

Parameters

- **r** -- **[in]** The request being responded to
- **str** -- **[in]** String to be sent as response body

Returns

- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null request pointer
- ESP_ERR_HTTPD_RESP_HDR : Essential headers are too large for internal buffer
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request

static inline *esp_err_t* **httpd_resp_sendstr_chunk**(*httpd_req_t* *r, const char *str)

API to send a string as an HTTP response chunk.

This API simply calls http_resp_send_chunk with buffer length set to string length assuming the buffer contains a null terminated string

Parameters

- **r** -- **[in]** The request being responded to
- **str** -- **[in]** String to be sent as response body (NULL to finish response packet)

Returns

- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null request pointer
- ESP_ERR_HTTPD_RESP_HDR : Essential headers are too large for internal buffer
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request

esp_err_t **httpd_resp_set_status**(*httpd_req_t* *r, const char *status)

API to set the HTTP status code.

This API sets the status of the HTTP response to the value specified. By default, the '200 OK' response is sent as the response.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- This API only sets the status to this value. The status isn't sent out until any of the send APIs is executed.
- Make sure that the lifetime of the status string is valid till send function is called.

Parameters

- **r** -- **[in]** The request being responded to
- **status** -- **[in]** The HTTP status code of this response

Returns

- ESP_OK : On success
- ESP_ERR_INVALID_ARG : Null arguments

- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

esp_err_t **httpd_resp_set_type** (*httpd_req_t* *r, const char *type)

API to set the HTTP content type.

This API sets the 'Content Type' field of the response. The default content type is 'text/html'.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- This API only sets the content type to this value. The type isn't sent out until any of the send APIs is executed.
- Make sure that the lifetime of the type string is valid till send function is called.

Parameters

- **r** -- **[in]** The request being responded to
- **type** -- **[in]** The Content Type of the response

Returns

- ESP_OK : On success
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

esp_err_t **httpd_resp_set_hdr** (*httpd_req_t* *r, const char *field, const char *value)

API to append any additional headers.

This API sets any additional header fields that need to be sent in the response.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- The header isn't sent out until any of the send APIs is executed.
- The maximum allowed number of additional headers is limited to value of max_resp_headers in config structure.
- Make sure that the lifetime of the field value strings are valid till send function is called.

Parameters

- **r** -- **[in]** The request being responded to
- **field** -- **[in]** The field name of the HTTP header
- **value** -- **[in]** The value of this HTTP header

Returns

- ESP_OK : On successfully appending new header
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_HDR : Total additional headers exceed max allowed
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

esp_err_t **httpd_resp_send_err** (*httpd_req_t* *req, *httpd_err_code_t* error, const char *msg)

For sending out error code in response to HTTP request.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

- If you wish to send additional data in the body of the response, please use the lower-level functions directly.

Parameters
- **req** -- **[in]** Pointer to the HTTP request for which the response needs to be sent
- **error** -- **[in]** Error type to send
- **msg** -- **[in]** Error message string (pass NULL for default message)

Returns
- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

esp_err_t **httpd_resp_send_custom_err**(*httpd_req_t* *req, const char *status, const char *msg)

For sending out custom error code in response to HTTP request.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.
- If you wish to send additional data in the body of the response, please use the lower-level functions directly.

Parameters
- **req** -- **[in]** Pointer to the HTTP request for which the response needs to be sent
- **status** -- **[in]** Error status to send
- **msg** -- **[in]** Error message string

Returns
- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

static inline *esp_err_t* **httpd_resp_send_404**(*httpd_req_t* *r)

Helper function for HTTP 404.

Send HTTP 404 message. If you wish to send additional data in the body of the response, please use the lower-level functions directly.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters **r** -- **[in]** The request being responded to

Returns
- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

static inline *esp_err_t* **httpd_resp_send_408** (*httpd_req_t* *r)

Helper function for HTTP 408.

Send HTTP 408 message. If you wish to send additional data in the body of the response, please use the lower-level functions directly.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters **r** -- **[in]** The request being responded to
Returns
- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

static inline *esp_err_t* **httpd_resp_send_500** (*httpd_req_t* *r)

Helper function for HTTP 500.

Send HTTP 500 message. If you wish to send additional data in the body of the response, please use the lower-level functions directly.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Once this API is called, all request headers are purged, so request headers need be copied into separate buffers if they are required later.

Parameters **r** -- **[in]** The request being responded to
Returns
- ESP_OK : On successfully sending the response packet
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_HTTPD_RESP_SEND : Error in raw send
- ESP_ERR_HTTPD_INVALID_REQ : Invalid request pointer

int **httpd_send** (*httpd_req_t* *r, const char *buf, size_t buf_len)

Raw HTTP send.

Call this API if you wish to construct your custom response packet. When using this, all essential header, eg. HTTP version, Status Code, Content Type and Length, Encoding, etc. will have to be constructed manually, and HTTP delimeters (CRLF) will need to be placed correctly for separating sub-sections of the HTTP response packet.

If the send override function is set, this API will end up calling that function eventually to send data out.

Note:

- This API is supposed to be called only from the context of a URI handler where httpd_req_t* request pointer is valid.
- Unless the response has the correct HTTP structure (which the user must now ensure) it is not guaranteed that it will be recognized by the client. For most cases, you wouldn't have to call this API, but you would rather use either of : httpd_resp_send(), httpd_resp_send_chunk()

Parameters

- **r** -- **[in]** The request being responded to
- **buf** -- **[in]** Buffer from where the fully constructed packet is to be read
- **buf_len** -- **[in]** Length of the buffer

Returns

- Bytes : Number of bytes that were sent successfully
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket send()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket send()

int **httpd_socket_send**(*httpd_handle_t* hd, int sockfd, const char *buf, size_t buf_len, int flags)

A low level API to send data on a given socket

This internally calls the default send function, or the function registered by httpd_sess_set_send_override().

Note: This API is not recommended to be used in any request handler. Use this only for advanced use cases, wherein some asynchronous data is to be sent over a socket.

Parameters

- **hd** -- **[in]** server instance
- **sockfd** -- **[in]** session socket file descriptor
- **buf** -- **[in]** buffer with bytes to send
- **buf_len** -- **[in]** data size
- **flags** -- **[in]** flags for the send() function

Returns

- Bytes : The number of bytes sent successfully
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket send()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket send()

int **httpd_socket_recv**(*httpd_handle_t* hd, int sockfd, char *buf, size_t buf_len, int flags)

A low level API to receive data from a given socket

This internally calls the default recv function, or the function registered by httpd_sess_set_recv_override().

Note: This API is not recommended to be used in any request handler. Use this only for advanced use cases, wherein some asynchronous communication is required.

Parameters

- **hd** -- **[in]** server instance
- **sockfd** -- **[in]** session socket file descriptor
- **buf** -- **[in]** buffer with bytes to send
- **buf_len** -- **[in]** data size
- **flags** -- **[in]** flags for the send() function

Returns

- Bytes : The number of bytes received successfully
- 0 : Buffer length parameter is zero / connection closed by peer
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket recv()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket recv()

esp_err_t **httpd_register_err_handler**(*httpd_handle_t* handle, *httpd_err_code_t* error, *httpd_err_handler_func_t* handler_fn)

Function for registering HTTP error handlers.

This function maps a handler function to any supported error code given by `httpd_err_code_t`. See prototype `httpd_err_handler_func_t` above for details.

Parameters
- **handle** -- **[in]** HTTP server handle
- **error** -- **[in]** Error type
- **handler_fn** -- **[in]** User implemented handler function (Pass NULL to unset any previously set handler)

Returns
- ESP_OK : handler registered successfully
- ESP_ERR_INVALID_ARG : invalid error code or server handle

esp_err_t **httpd_queue_work**(*httpd_handle_t* handle, *httpd_work_fn_t* work, void *arg)

Queue execution of a function in HTTPD's context.

This API queues a work function for asynchronous execution

Note: Some protocols require that the web server generate some asynchronous data and send it to the persistently opened connection. This facility is for use by such protocols.

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **work** -- **[in]** Pointer to the function to be executed in the HTTPD's context
- **arg** -- **[in]** Pointer to the arguments that should be passed to this function

Returns
- ESP_OK : On successfully queueing the work
- ESP_FAIL : Failure in ctrl socket
- ESP_ERR_INVALID_ARG : Null arguments

void ***httpd_sess_get_ctx**(*httpd_handle_t* handle, int sockfd)

Get session context from socket descriptor.

Typically if a session context is created, it is available to URI handlers through the httpd_req_t structure. But, there are cases where the web server's send/receive functions may require the context (for example, for accessing keying information etc). Since the send/receive function only have the socket descriptor at their disposal, this API provides them with a way to retrieve the session context.

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor for which the context should be extracted.

Returns
- void* : Pointer to the context associated with this session
- NULL : Empty context / Invalid handle / Invalid socket fd

void **httpd_sess_set_ctx**(*httpd_handle_t* handle, int sockfd, void *ctx, *httpd_free_ctx_fn_t* free_fn)

Set session context by socket descriptor.

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor for which the context should be extracted.
- **ctx** -- **[in]** Context object to assign to the session
- **free_fn** -- **[in]** Function that should be called to free the context

void ***httpd_sess_get_transport_ctx**(*httpd_handle_t* handle, int sockfd)

Get session 'transport' context by socket descriptor.

This context is used by the send/receive functions, for example to manage SSL context.

See also:

httpd_sess_get_ctx()

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor for which the context should be extracted.

Returns
- void* : Pointer to the transport context associated with this session
- NULL : Empty context / Invalid handle / Invalid socket fd

void **httpd_sess_set_transport_ctx**(*httpd_handle_t* handle, int sockfd, void *ctx, *httpd_free_ctx_fn_t* free_fn)

Set session 'transport' context by socket descriptor.

See also:

httpd_sess_set_ctx()

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor for which the context should be extracted.
- **ctx** -- **[in]** Transport context object to assign to the session
- **free_fn** -- **[in]** Function that should be called to free the transport context

void ***httpd_get_global_user_ctx**(*httpd_handle_t* handle)

Get HTTPD global user context (it was set in the server config struct)

Parameters **handle** -- **[in]** Handle to server returned by httpd_start
Returns global user context

void ***httpd_get_global_transport_ctx**(*httpd_handle_t* handle)

Get HTTPD global transport context (it was set in the server config struct)

Parameters **handle** -- **[in]** Handle to server returned by httpd_start
Returns global transport context

esp_err_t **httpd_sess_trigger_close**(*httpd_handle_t* handle, int sockfd)

Trigger an httpd session close externally.

Note: Calling this API is only required in special circumstances wherein some application requires to close an httpd client session asynchronously.

Parameters
- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor of the session to be closed

Returns
- ESP_OK : On successfully initiating closure
- ESP_FAIL : Failure to queue work
- ESP_ERR_NOT_FOUND : Socket fd not found
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **httpd_sess_update_lru_counter** (*httpd_handle_t* handle, int sockfd)

Update LRU counter for a given socket.

LRU Counters are internally associated with each session to monitor how recently a session exchanged traffic. When LRU purge is enabled, if a client is requesting for connection but maximum number of sockets/sessions is reached, then the session having the earliest LRU counter is closed automatically.

Updating the LRU counter manually prevents the socket from being purged due to the Least Recently Used (LRU) logic, even though it might not have received traffic for some time. This is useful when all open sockets/session are frequently exchanging traffic but the user specifically wants one of the sessions to be kept open, irrespective of when it last exchanged a packet.

Note: Calling this API is only necessary if the LRU Purge Enable option is enabled.

Parameters

- **handle** -- **[in]** Handle to server returned by httpd_start
- **sockfd** -- **[in]** The socket descriptor of the session for which LRU counter is to be updated

Returns

- ESP_OK : Socket found and LRU counter updated
- ESP_ERR_NOT_FOUND : Socket not found
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **httpd_get_client_list** (*httpd_handle_t* handle, size_t *fds, int *client_fds)

Returns list of current socket descriptors of active sessions.

Note: Size of provided array has to be equal or greater then maximum number of opened sockets, configured upon initialization with max_open_sockets field in httpd_config_t structure.

Parameters

- **handle** -- **[in]** Handle to server returned by httpd_start
- **fds** -- **[inout]** In: Size of provided client_fds array Out: Number of valid client fds returned in client_fds,
- **client_fds** -- **[out]** Array of client fds

Returns

- ESP_OK : Successfully retrieved session list
- ESP_ERR_INVALID_ARG : Wrong arguments or list is longer than provided array

Structures

struct **esp_http_server_event_data**

Argument structure for HTTP_SERVER_EVENT_ON_DATA and HTTP_SERVER_EVENT_SENT_DATA event

Public Members

int **fd**

Session socket file descriptor

int **data_len**

Data length

struct **httpd_config**

HTTP Server Configuration Structure.

Note: Use HTTPD_DEFAULT_CONFIG() to initialize the configuration to a default value and then modify only those fields that are specifically determined by the use case.

Public Members

unsigned **task_priority**

Priority of FreeRTOS task which runs the server

size_t **stack_size**

The maximum stack size allowed for the server task

BaseType_t **core_id**

The core the HTTP server task will run on

uint32_t **task_caps**

The memory capabilities to use when allocating the HTTP server task's stack

uint16_t **server_port**

TCP Port number for receiving and transmitting HTTP traffic

uint16_t **ctrl_port**

UDP Port number for asynchronously exchanging control signals between various components of the server

uint16_t **max_open_sockets**

Max number of sockets/clients connected at any time (3 sockets are reserved for internal working of the HTTP server)

uint16_t **max_uri_handlers**

Maximum allowed uri handlers

uint16_t **max_resp_headers**

Maximum allowed additional headers in HTTP response

uint16_t **backlog_conn**

Number of backlog connections

bool **lru_purge_enable**

Purge "Least Recently Used" connection

uint16_t **recv_wait_timeout**

Timeout for recv function (in seconds)

uint16_t **send_wait_timeout**

Timeout for send function (in seconds)

void ***global_user_ctx**

Global user context.

This field can be used to store arbitrary user data within the server context. The value can be retrieved using the server handle, available e.g. in the httpd_req_t struct.

When shutting down, the server frees up the user context by calling free() on the global_user_ctx field. If you wish to use a custom function for freeing the global user context, please specify that here.

httpd_free_ctx_fn_t **global_user_ctx_free_fn**

Free function for global user context

void ***global_transport_ctx**

Global transport context.

Similar to global_user_ctx, but used for session encoding or encryption (e.g. to hold the SSL context). It will be freed using free(), unless global_transport_ctx_free_fn is specified.

httpd_free_ctx_fn_t **global_transport_ctx_free_fn**

Free function for global transport context

bool **enable_so_linger**

bool to enable/disable linger

int **linger_timeout**

linger timeout (in seconds)

bool **keep_alive_enable**

Enable keep-alive timeout

int **keep_alive_idle**

Keep-alive idle time. Default is 5 (second)

int **keep_alive_interval**

Keep-alive interval time. Default is 5 (second)

int **keep_alive_count**

Keep-alive packet retry send count. Default is 3 counts

httpd_open_func_t **open_fn**

Custom session opening callback.

Called on a new session socket just after accept(), but before reading any data.

This is an opportunity to set up e.g. SSL encryption using global_transport_ctx and the send/recv/pending session overrides.

If a context needs to be maintained between these functions, store it in the session using httpd_sess_set_transport_ctx() and retrieve it later with httpd_sess_get_transport_ctx()

Returning a value other than ESP_OK will immediately close the new socket.

httpd_close_func_t **close_fn**

Custom session closing callback.

Called when a session is deleted, before freeing user and transport contexts and before closing the socket. This is a place for custom de-init code common to all sockets.

The server will only close the socket if no custom session closing callback is set. If a custom callback is used, `close(sockfd)` should be called in here for most cases.

Set the user or transport context to NULL if it was freed here, so the server does not try to free it again.

This function is run for all terminated sessions, including sessions where the socket was closed by the network stack - that is, the file descriptor may not be valid anymore.

httpd_uri_match_func_t **uri_match_fn**

URI matcher function.

Called when searching for a matching URI: 1) whose request handler is to be executed right after an HTTP request is successfully parsed 2) in order to prevent duplication while registering a new URI handler using `httpd_register_uri_handler()`

Available options are: 1) NULL : Internally do basic matching using `strncmp()` 2) `httpd_uri_match_wildcard()` : URI wildcard matcher

Users can implement their own matching functions (See description of the `httpd_uri_match_func_t` function prototype)

struct **httpd_req**

HTTP Request Data Structure.

Public Members

httpd_handle_t **handle**

Handle to server instance

int **method**

The type of HTTP request, -1 if unsupported method, HTTP_ANY for wildcard method to support every method

const char **uri**[HTTPD_MAX_URI_LEN + 1]

The URI of this request (1 byte extra for null termination)

size_t **content_len**

Length of the request body

void ***aux**

Internally used members

void ***user_ctx**

User context pointer passed during URI registration.

void ***sess_ctx**

Session Context Pointer

A session context. Contexts are maintained across 'sessions' for a given open TCP connection. One session could have multiple request responses. The web server will ensure that the context persists across all these request and responses.

By default, this is NULL. URI Handlers can set this to any meaningful value.

If the underlying socket gets closed, and this pointer is non-NULL, the web server will free up the context by calling free(), unless free_ctx function is set.

httpd_free_ctx_fn_t **free_ctx**

Pointer to free context hook

Function to free session context

If the web server's socket closes, it frees up the session context by calling free() on the sess_ctx member. If you wish to use a custom function for freeing the session context, please specify that here.

bool **ignore_sess_ctx_changes**

Flag indicating if Session Context changes should be ignored

By default, if you change the sess_ctx in some URI handler, the http server will internally free the earlier context (if non NULL), after the URI handler returns. If you want to manage the allocation/reallocation/freeing of sess_ctx yourself, set this flag to true, so that the server will not perform any checks on it. The context will be cleared by the server (by calling free_ctx or free()) only if the socket gets closed.

struct **httpd_uri**

Structure for URI handler.

Public Members

const char ***uri**

The URI to handle

httpd_method_t **method**

Method supported by the URI, HTTP_ANY for wildcard method to support all methods

esp_err_t (***handler**)(*httpd_req_t* *r)

Handler to call for supported request method. This must return ESP_OK, or else the underlying socket will be closed.

void ***user_ctx**

Pointer to user context data which will be available to handler

Macros

HTTP_ANY

HTTPD_MAX_REQ_HDR_LEN

HTTPD_MAX_URI_LEN

HTTPD_SOCK_ERR_FAIL

HTTPD_SOCK_ERR_INVALID

HTTPD_SOCK_ERR_TIMEOUT

HTTPD_200
HTTP Response 200

HTTPD_204
HTTP Response 204

HTTPD_207
HTTP Response 207

HTTPD_400
HTTP Response 400

HTTPD_404
HTTP Response 404

HTTPD_408
HTTP Response 408

HTTPD_500
HTTP Response 500

HTTPD_TYPE_JSON
HTTP Content type JSON

HTTPD_TYPE_TEXT
HTTP Content type text/HTML

HTTPD_TYPE_OCTET
HTTP Content type octext-stream

ESP_HTTPD_DEF_CTRL_PORT
HTTP Server control socket port

HTTPD_DEFAULT_CONFIG()

ESP_ERR_HTTPD_BASE
Starting number of HTTPD error codes

ESP_ERR_HTTPD_HANDLERS_FULL
All slots for registering URI handlers have been consumed

ESP_ERR_HTTPD_HANDLER_EXISTS
URI handler with same method and target URI already registered

ESP_ERR_HTTPD_INVALID_REQ

Invalid request pointer

ESP_ERR_HTTPD_RESULT_TRUNC

Result string truncated

ESP_ERR_HTTPD_RESP_HDR

Response header field larger than supported

ESP_ERR_HTTPD_RESP_SEND

Error occured while sending response packet

ESP_ERR_HTTPD_ALLOC_MEM

Failed to dynamically allocate memory for resource

ESP_ERR_HTTPD_TASK

Failed to launch server task/thread

HTTPD_RESP_USE_STRLEN

Type Definitions

typedef void ***httpd_handle_t**

HTTP Server Instance Handle.

Every instance of the server will have a unique handle.

typedef enum http_method **httpd_method_t**

HTTP Method Type wrapper over "enum http_method" available in "http_parser" library.

typedef void (***httpd_free_ctx_fn_t**)(void *ctx)

Prototype for freeing context data (if any)

Param ctx **[in]** object to free

typedef *esp_err_t* (***httpd_open_func_t**)(*httpd_handle_t* hd, int sockfd)

Function prototype for opening a session.

Called immediately after the socket was opened to set up the send/recv functions and other parameters of the socket.

Param hd **[in]** server instance
Param sockfd **[in]** session socket file descriptor
Return

- ESP_OK : On success
- Any value other than ESP_OK will signal the server to close the socket immediately

typedef void (***httpd_close_func_t**)(*httpd_handle_t* hd, int sockfd)

Function prototype for closing a session.

Note: It's possible that the socket descriptor is invalid at this point, the function is called for all terminated sessions. Ensure proper handling of return codes.

Param hd **[in]** server instance
Param sockfd **[in]** session socket file descriptor

typedef bool (***httpd_uri_match_func_t***)(const char *reference_uri, const char *uri_to_match, size_t match_upto)

Function prototype for URI matching.

Param reference_uri **[in]** URI/template with respect to which the other URI is matched
Param uri_to_match **[in]** URI/template being matched to the reference URI/template
Param match_upto **[in]** For specifying the actual length of `uri_to_match` up to which the matching algorithm is to be applied (The maximum value is `strlen(uri_to_match)`, independent of the length of `reference_uri`)
Return true on match

typedef struct *httpd_config* **httpd_config_t**

HTTP Server Configuration Structure.

Note: Use HTTPD_DEFAULT_CONFIG() to initialize the configuration to a default value and then modify only those fields that are specifically determined by the use case.

---

typedef struct *httpd_req* **httpd_req_t**

HTTP Request Data Structure.

typedef struct *httpd_uri* **httpd_uri_t**

Structure for URI handler.

typedef int (***httpd_send_func_t***)(*httpd_handle_t* hd, int sockfd, const char *buf, size_t buf_len, int flags)

Prototype for HTTPDs low-level send function.

Note: User specified send function must handle errors internally, depending upon the set value of errno, and return specific HTTPD_SOCK_ERR_ codes, which will eventually be conveyed as return value of httpd_send() function

---

Param hd **[in]** server instance
Param sockfd **[in]** session socket file descriptor
Param buf **[in]** buffer with bytes to send
Param buf_len **[in]** data size
Param flags **[in]** flags for the send() function
Return

- Bytes : The number of bytes sent successfully
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket send()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket send()

typedef int (***httpd_recv_func_t***)(*httpd_handle_t* hd, int sockfd, char *buf, size_t buf_len, int flags)

Prototype for HTTPDs low-level recv function.

Note: User specified recv function must handle errors internally, depending upon the set value of errno, and return specific HTTPD_SOCK_ERR_ codes, which will eventually be conveyed as return value of httpd_req_recv() function

---

Param hd [in] server instance
Param sockfd [in] session socket file descriptor
Param buf [in] buffer with bytes to send
Param buf_len [in] data size
Param flags [in] flags for the send() function
Return

- Bytes : The number of bytes received successfully
- 0 : Buffer length parameter is zero / connection closed by peer
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket recv()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket recv()

typedef int (***httpd_pending_func_t**)(*httpd_handle_t* hd, int sockfd)

Prototype for HTTPDs low-level "get pending bytes" function.

Note: User specified pending function must handle errors internally, depending upon the set value of errno, and return specific HTTPD_SOCK_ERR_ codes, which will be handled accordingly in the server task.

Param hd [in] server instance
Param sockfd [in] session socket file descriptor
Return

- Bytes : The number of bytes waiting to be received
- HTTPD_SOCK_ERR_INVALID : Invalid arguments
- HTTPD_SOCK_ERR_TIMEOUT : Timeout/interrupted while calling socket pending()
- HTTPD_SOCK_ERR_FAIL : Unrecoverable error while calling socket pending()

typedef *esp_err_t* (***httpd_err_handler_func_t**)(*httpd_req_t* *req, *httpd_err_code_t* error)

Function prototype for HTTP error handling.

This function is executed upon HTTP errors generated during internal processing of an HTTP request. This is used to override the default behavior on error, which is to send HTTP error response and close the underlying socket.

Note:

- If implemented, the server will not automatically send out HTTP error response codes, therefore, httpd_resp_send_err() must be invoked inside this function if user wishes to generate HTTP error responses.
- When invoked, the validity of `uri`, `method`, `content_len` and `user_ctx` fields of the httpd_req_t parameter is not guaranteed as the HTTP request may be partially received/parsed.
- The function must return ESP_OK if underlying socket needs to be kept open. Any other value will ensure that the socket is closed. The return value is ignored when error is of type `HTTPD_500_INTERNAL_SERVER_ERROR` and the socket closed anyway.

Param req [in] HTTP request for which the error needs to be handled
Param error [in] Error type
Return

- ESP_OK : error handled successful
- ESP_FAIL : failure indicates that the underlying socket needs to be closed

typedef void (***httpd_work_fn_t**)(void *arg)

Prototype of the HTTPD work function Please refer to httpd_queue_work() for more details.

Param arg [in] The arguments for this work function

Enumerations

enum **httpd_err_code_t**

Error codes sent as HTTP response in case of errors encountered during processing of an HTTP request.

Values:

enumerator **HTTPD_500_INTERNAL_SERVER_ERROR**

enumerator **HTTPD_501_METHOD_NOT_IMPLEMENTED**

enumerator **HTTPD_505_VERSION_NOT_SUPPORTED**

enumerator **HTTPD_400_BAD_REQUEST**

enumerator **HTTPD_401_UNAUTHORIZED**

enumerator **HTTPD_403_FORBIDDEN**

enumerator **HTTPD_404_NOT_FOUND**

enumerator **HTTPD_405_METHOD_NOT_ALLOWED**

enumerator **HTTPD_408_REQ_TIMEOUT**

enumerator **HTTPD_411_LENGTH_REQUIRED**

enumerator **HTTPD_414_URI_TOO_LONG**

enumerator **HTTPD_431_REQ_HDR_FIELDS_TOO_LARGE**

enumerator **HTTPD_ERR_CODE_MAX**

enum **esp_http_server_event_id_t**

HTTP Server events id.

Values:

enumerator **HTTP_SERVER_EVENT_ERROR**

This event occurs when there are any errors during execution

enumerator **HTTP_SERVER_EVENT_START**

This event occurs when HTTP Server is started

enumerator **HTTP_SERVER_EVENT_ON_CONNECTED**

Once the HTTP Server has been connected to the client, no data exchange has been performed

enumerator **HTTP_SERVER_EVENT_ON_HEADER**

Occurs when receiving each header sent from the client

enumerator **HTTP_SERVER_EVENT_HEADERS_SENT**

After sending all the headers to the client

enumerator **HTTP_SERVER_EVENT_ON_DATA**

Occurs when receiving data from the client

enumerator **HTTP_SERVER_EVENT_SENT_DATA**

Occurs when an ESP HTTP server session is finished

enumerator **HTTP_SERVER_EVENT_DISCONNECTED**

The connection has been disconnected

enumerator **HTTP_SERVER_EVENT_STOP**

This event occurs when HTTP Server is stopped

2.2.10 HTTPS Server

Overview

This component is built on top of *HTTP Server*. The HTTPS server takes advantage of hook registration functions in the regular HTTP server to provide callback function for SSL session.

All documentation for *HTTP Server* applies also to a server you create this way.

Used APIs

The following APIs of *HTTP Server* should not be used with *HTTPS Server*, as they are used internally to handle secure sessions and to maintain internal state:

- "send", "receive" and "pending" callback registration functions - secure socket handling
  - *httpd_sess_set_send_override()*
  - *httpd_sess_set_recv_override()*
  - *httpd_sess_set_pending_override()*
- "transport context" - both global and session
  - *httpd_sess_get_transport_ctx()* - returns SSL used for the session
  - *httpd_sess_set_transport_ctx()*
  - *httpd_get_global_transport_ctx()* - returns the shared SSL context
  - *httpd_config::global_transport_ctx*
  - *httpd_config::global_transport_ctx_free_fn*
  - *httpd_config::open_fn* - used to set up secure sockets

Everything else can be used without limitations.

Usage

Please see the example protocols/https_server to learn how to set up a secure server.

Basically, all you need is to generate a certificate, embed it into the firmware, and pass the init struct into the start function after the certificate address and lengths are correctly configured in the init struct.

The server can be started with or without SSL by changing a flag in the init struct - *httpd_ssl_config::transport_mode*. This could be used, e.g., for testing or in trusted environments where you prefer speed over security.

Performance

The initial session setup can take about two seconds, or more with slower clock speed or more verbose logging. Subsequent requests through the open secure socket are much faster (down to under 100 ms).

Event Handling

ESP HTTPS Server has various events for which a handler can be triggered by the *Event Loop Library* when the particular event occurs. The handler has to be registered using `esp_event_handler_register()`. This helps in event handling for ESP HTTPS Server.

`esp_https_server_event_id_t` has all the events which can happen for ESP HTTPS server.

Expected data type for different ESP HTTPS server events in event loop:

- HTTPS_SERVER_EVENT_ERROR : `esp_https_server_last_error_t`
- HTTPS_SERVER_EVENT_START : `NULL`
- HTTPS_SERVER_EVENT_ON_CONNECTED : `NULL`
- HTTPS_SERVER_EVENT_ON_DATA : `int`
- HTTPS_SERVER_EVENT_SENT_DATA : `NULL`
- HTTPS_SERVER_EVENT_DISCONNECTED : `NULL`
- HTTPS_SERVER_EVENT_STOP : `NULL`

API Reference

Header File

- components/esp_https_server/include/esp_https_server.h
- This header file can be included with:

```
#include "esp_https_server.h"
```

- This header file is a part of the API provided by the `esp_https_server` component. To declare that your component depends on `esp_https_server`, add the following to your CMakeLists.txt:

```
REQUIRES esp_https_server
```

or

```
PRIV_REQUIRES esp_https_server
```

Functions

esp_err_t **httpd_ssl_start** (*httpd_handle_t* *handle, *httpd_ssl_config_t* *config)

Create a SSL capable HTTP server (secure mode may be disabled in config)

Parameters

- **config** -- **[inout]** - server config, must not be const. Does not have to stay valid after calling this function.
- **handle** -- **[out]** - storage for the server handle, must be a valid pointer

Returns success

esp_err_t **httpd_ssl_stop** (*httpd_handle_t* handle)

Stop the server. Blocks until the server is shut down.

Parameters **handle** -- **[in]**

Returns

- ESP_OK: Server stopped successfully
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_FAIL: Failure to shut down server

Structures

struct **esp_https_server_user_cb_arg**

Callback data struct, contains the ESP-TLS connection handle and the connection state at which the callback is executed.

Public Members

httpd_ssl_user_cb_state_t **user_cb_state**

State of user callback

esp_tls_t ***tls**

ESP-TLS connection handle

struct **httpd_ssl_config**

HTTPS server config struct

Please use HTTPD_SSL_CONFIG_DEFAULT() to initialize it.

Public Members

httpd_config_t **httpd**

Underlying HTTPD server config

Parameters like task stack size and priority can be adjusted here.

const uint8_t ***servercert**

Server certificate

size_t **servercert_len**

Server certificate byte length

const uint8_t ***cacert_pem**

CA certificate ((CA used to sign clients, or client cert itself)

size_t **cacert_len**

CA certificate byte length

const uint8_t ***prvtkey_pem**

Private key

size_t **prvtkey_len**

Private key byte length

bool **use_ecdsa_peripheral**

Use ECDSA peripheral to use private key

uint8_t **ecdsa_key_efuse_blk**

The efuse block where ECDSA key is stored

httpd_ssl_transport_mode_t **transport_mode**

Transport Mode (default secure)

uint16_t **port_secure**

Port used when transport mode is secure (default 443)

uint16_t **port_insecure**

Port used when transport mode is insecure (default 80)

bool **session_tickets**

Enable tls session tickets

bool **use_secure_element**

Enable secure element for server session

esp_https_server_user_cb ***user_cb**

User callback for esp_https_server

void ***ssl_userdata**

User data to add to the ssl context

esp_tls_handshake_callback **cert_select_cb**

Certificate selection callback to use. The callback is only applicable when CONFIG_ESP_TLS_SERVER_CERT_SELECT_HOOK is enabled in menuconfig

const char ****alpn_protos**

Application protocols the server supports in order of prefernece. Used for negotiating during the TLS handshake, first one the client supports is selected. The data structure must live as long as the https server itself

Macros

HTTPD_SSL_CONFIG_DEFAULT()

Default config struct init Notes:

- port is set when starting the server, according to 'transport_mode'
- one socket uses ~ 40kB RAM with SSL, we reduce the default socket count to 4
- SSL sockets are usually long-lived, closing LRU prevents pool exhaustion DOS
- Stack size may need adjustments depending on the user application

Type Definitions

typedef struct *esp_https_server_user_cb_arg* **esp_https_server_user_cb_arg_t**

Callback data struct, contains the ESP-TLS connection handle and the connection state at which the callback is executed.

typedef *esp_tls_last_error_t* **esp_https_server_last_error_t**

typedef void **esp_https_server_user_cb**(*esp_https_server_user_cb_arg_t* *user_cb)

Callback function prototype Can be used to get connection or client information (SSL context) E.g. Client certificate, Socket FD, Connection state, etc.

Param user_cb Callback data struct

typedef struct *httpd_ssl_config* **httpd_ssl_config_t**

Enumerations

enum **esp_https_server_event_id_t**

Values:

enumerator **HTTPS_SERVER_EVENT_ERROR**

This event occurs when there are any errors during execution

enumerator **HTTPS_SERVER_EVENT_START**

This event occurs when HTTPS Server is started

enumerator **HTTPS_SERVER_EVENT_ON_CONNECTED**

Once the HTTPS Server has been connected to the client

enumerator **HTTPS_SERVER_EVENT_ON_DATA**

Occurs when receiving data from the client

enumerator **HTTPS_SERVER_EVENT_SENT_DATA**

Occurs when an ESP HTTPS server sends data to the client

enumerator **HTTPS_SERVER_EVENT_DISCONNECTED**

The connection has been disconnected

enumerator **HTTPS_SERVER_EVENT_STOP**

This event occurs when HTTPS Server is stopped

enum **httpd_ssl_transport_mode_t**

Values:

enumerator **HTTPD_SSL_TRANSPORT_SECURE**

enumerator **HTTPD_SSL_TRANSPORT_INSECURE**

enum **httpd_ssl_user_cb_state_t**

Indicates the state at which the user callback is executed, i.e at session creation or session close.

Values:

enumerator **HTTPD_SSL_USER_CB_SESS_CREATE**

enumerator **HTTPD_SSL_USER_CB_SESS_CLOSE**

2.2.11 ICMP Echo

Overview

ICMP (Internet Control Message Protocol) is used for diagnostic or control purposes or generated in response to errors in IP operations. The common network util `ping` is implemented based on the ICMP packets with the type field value of 0, also called `Echo Reply`.

During a ping session, the source host firstly sends out an ICMP echo request packet and wait for an ICMP echo reply with specific times. In this way, it also measures the round-trip time for the messages. After receiving a valid ICMP echo reply, the source host will generate statistics about the IP link layer (e.g., packet loss, elapsed time, etc).

It is common that IoT device needs to check whether a remote server is alive or not. The device should show the warnings to users when it got offline. It can be achieved by creating a ping session and sending or parsing ICMP echo packets periodically.

To make this internal procedure much easier for users, ESP-IDF provides some out-of-box APIs.

Create a New Ping Session To create a ping session, you need to fill in the `esp_ping_config_t` configuration structure firstly, specifying target IP address, interval times, and etc. Optionally, you can also register some callback functions with the `esp_ping_callbacks_t` structure.

Example method to create a new ping session and register callbacks:

```
static void test_on_ping_success(esp_ping_handle_t hdl, void *args)
{
    // optionally, get callback arguments
    // const char* str = (const char*) args;
    // printf("%s\r\n", str); // "foo"
    uint8_t ttl;
    uint16_t seqno;
    uint32_t elapsed_time, recv_len;
    ip_addr_t target_addr;
    esp_ping_get_profile(hdl, ESP_PING_PROF_SEQNO, &seqno, sizeof(seqno));
    esp_ping_get_profile(hdl, ESP_PING_PROF_TTL, &ttl, sizeof(ttl));
    esp_ping_get_profile(hdl, ESP_PING_PROF_IPADDR, &target_addr, sizeof(target_
↪addr));
    esp_ping_get_profile(hdl, ESP_PING_PROF_SIZE, &recv_len, sizeof(recv_len));
    esp_ping_get_profile(hdl, ESP_PING_PROF_TIMEGAP, &elapsed_time, sizeof(elapsed_
↪time));
    printf("%d bytes from %s icmp_seq=%d ttl=%d time=%d ms\n",
           recv_len, inet_ntoa(target_addr.u_addr.ip4), seqno, ttl, elapsed_time);
}

static void test_on_ping_timeout(esp_ping_handle_t hdl, void *args)
{
    uint16_t seqno;
    ip_addr_t target_addr;
    esp_ping_get_profile(hdl, ESP_PING_PROF_SEQNO, &seqno, sizeof(seqno));
    esp_ping_get_profile(hdl, ESP_PING_PROF_IPADDR, &target_addr, sizeof(target_
↪addr));
    printf("From %s icmp_seq=%d timeout\n", inet_ntoa(target_addr.u_addr.ip4),␣
↪seqno);
}

static void test_on_ping_end(esp_ping_handle_t hdl, void *args)
{
    uint32_t transmitted;
    uint32_t received;
    uint32_t total_time_ms;

    esp_ping_get_profile(hdl, ESP_PING_PROF_REQUEST, &transmitted,␣
↪sizeof(transmitted));
    esp_ping_get_profile(hdl, ESP_PING_PROF_REPLY, &received, sizeof(received));
```

(continues on next page)

(continued from previous page)

```
    esp_ping_get_profile(hdl, ESP_PING_PROF_DURATION, &total_time_ms, sizeof(total_
↪time_ms));
    printf("%d packets transmitted, %d received, time %dms\n", transmitted,␣
↪received, total_time_ms);
}

void initialize_ping()
{
    /* convert URL to IP address */
    ip_addr_t target_addr;
    struct addrinfo hint;
    struct addrinfo *res = NULL;
    memset(&hint, 0, sizeof(hint));
    memset(&target_addr, 0, sizeof(target_addr));
    getaddrinfo("www.espressif.com", NULL, &hint, &res);
    struct in_addr addr4 = ((struct sockaddr_in *) (res->ai_addr))->sin_addr;
    inet_addr_to_ip4addr(ip_2_ip4(&target_addr), &addr4);
    freeaddrinfo(res);

    esp_ping_config_t ping_config = ESP_PING_DEFAULT_CONFIG();
    ping_config.target_addr = target_addr;          // target IP address
    ping_config.count = ESP_PING_COUNT_INFINITE;    // ping in infinite mode, esp_
↪ping_stop can stop it

    /* set callback functions */
    esp_ping_callbacks_t cbs;
    cbs.on_ping_success = test_on_ping_success;
    cbs.on_ping_timeout = test_on_ping_timeout;
    cbs.on_ping_end = test_on_ping_end;
    cbs.cb_args = "foo";  // arguments that feeds to all callback functions, can␣
↪be NULL
    cbs.cb_args = eth_event_group;

    esp_ping_handle_t ping;
    esp_ping_new_session(&ping_config, &cbs, &ping);
}
```

Start and Stop Ping Session You can start and stop ping session with the handle returned by `esp_ping_new_session`. Note that, the ping session does not start automatically after creation. If the ping session is stopped, and restart again, the sequence number in ICMP packets will recount from zero again.

Delete a Ping Session If a ping session will not be used any more, you can delete it with `esp_ping_delete_session`. Please make sure the ping session is in stop state (i.e., you have called `esp_ping_stop` before or the ping session has finished all the procedures) when you call this function.

Get Runtime Statistics As the example code above, you can call `esp_ping_get_profile` to get different runtime statistics of ping session in the callback function.

Application Example

ICMP echo example: protocols/icmp_echo

API Reference

Header File

- components/lwip/include/apps/ping/ping_sock.h
- This header file can be included with:

```
#include "ping/ping_sock.h"
```

- This header file is a part of the API provided by the `lwip` component. To declare that your component depends on `lwip`, add the following to your CMakeLists.txt:

```
REQUIRES lwip
```

or

```
PRIV_REQUIRES lwip
```

Functions

esp_err_t **esp_ping_new_session**(const *esp_ping_config_t* *config, const *esp_ping_callbacks_t* *cbs, *esp_ping_handle_t* *hdl_out)

Create a ping session.

Parameters
- **config** -- ping configuration
- **cbs** -- a bunch of callback functions invoked by internal ping task
- **hdl_out** -- handle of ping session

Returns
- ESP_ERR_INVALID_ARG: invalid parameters (e.g. configuration is null, etc)
- ESP_ERR_NO_MEM: out of memory
- ESP_FAIL: other internal error (e.g. socket error)
- ESP_OK: create ping session successfully, user can take the ping handle to do follow-on jobs

esp_err_t **esp_ping_delete_session**(*esp_ping_handle_t* hdl)

Delete a ping session.

Parameters **hdl** -- handle of ping session

Returns
- ESP_ERR_INVALID_ARG: invalid parameters (e.g. ping handle is null, etc)
- ESP_OK: delete ping session successfully

esp_err_t **esp_ping_start**(*esp_ping_handle_t* hdl)

Start the ping session.

Parameters **hdl** -- handle of ping session

Returns
- ESP_ERR_INVALID_ARG: invalid parameters (e.g. ping handle is null, etc)
- ESP_OK: start ping session successfully

esp_err_t **esp_ping_stop**(*esp_ping_handle_t* hdl)

Stop the ping session.

Parameters **hdl** -- handle of ping session

Returns
- ESP_ERR_INVALID_ARG: invalid parameters (e.g. ping handle is null, etc)
- ESP_OK: stop ping session successfully

esp_err_t **esp_ping_get_profile**(*esp_ping_handle_t* hdl, *esp_ping_profile_t* profile, void *data, uint32_t size)

Get runtime profile of ping session.

Parameters
- **hdl** -- handle of ping session
- **profile** -- type of profile
- **data** -- profile data

- **size** -- profile data size

Returns

- ESP_ERR_INVALID_ARG: invalid parameters (e.g. ping handle is null, etc)
- ESP_ERR_INVALID_SIZE: the actual profile data size doesn't match the "size" parameter
- ESP_OK: get profile successfully

Structures

struct **esp_ping_callbacks_t**

Type of "ping" callback functions.

Public Members

void ***cb_args**

arguments for callback functions

void (***on_ping_success**)(*esp_ping_handle_t* hdl, void *args)

Invoked by internal ping thread when received ICMP echo reply packet.

void (***on_ping_timeout**)(*esp_ping_handle_t* hdl, void *args)

Invoked by internal ping thread when receive ICMP echo reply packet timeout.

void (***on_ping_end**)(*esp_ping_handle_t* hdl, void *args)

Invoked by internal ping thread when a ping session is finished.

struct **esp_ping_config_t**

Type of "ping" configuration.

Public Members

uint32_t **count**

A "ping" session contains count procedures

uint32_t **interval_ms**

Milliseconds between each ping procedure

uint32_t **timeout_ms**

Timeout value (in milliseconds) of each ping procedure

uint32_t **data_size**

Size of the data next to ICMP packet header

int **tos**

Type of Service, a field specified in the IP header

int **ttl**

Time to Live,a field specified in the IP header

ip_addr_t **target_addr**

Target IP address, either IPv4 or IPv6

uint32_t **task_stack_size**

Stack size of internal ping task

uint32_t **task_prio**

Priority of internal ping task

uint32_t **interface**

Netif index, interface=0 means NETIF_NO_INDEX

Macros

ESP_PING_DEFAULT_CONFIG()

Default ping configuration.

ESP_PING_COUNT_INFINITE

Set ping count to zero will ping target infinitely

Type Definitions

typedef void ***esp_ping_handle_t**

Type of "ping" session handle.

Enumerations

enum **esp_ping_profile_t**

Profile of ping session.

Values:

enumerator **ESP_PING_PROF_SEQNO**

Sequence number of a ping procedure

enumerator **ESP_PING_PROF_TOS**

Type of service of a ping procedure

enumerator **ESP_PING_PROF_TTL**

Time to live of a ping procedure

enumerator **ESP_PING_PROF_REQUEST**

Number of request packets sent out

enumerator **ESP_PING_PROF_REPLY**

Number of reply packets received

enumerator **ESP_PING_PROF_IPADDR**

IP address of replied target

enumerator **ESP_PING_PROF_SIZE**

Size of received packet

enumerator **ESP_PING_PROF_TIMEGAP**

Elapsed time between request and reply packet

enumerator **ESP_PING_PROF_DURATION**

Elapsed time of the whole ping session

2.2.12 mDNS Service

mDNS is a multicast UDP service that is used to provide local network service and host discovery.

The ESP-IDF component `mDNS` has been moved from ESP-IDF since version v5.0 to a separate repository:

- mDNS component on GitHub

To add mDNS component in your project, please run `idf.py add-dependency espressif/mdns`.

Hosted Documentation

The documentation can be found on the link below:

- mDNS documentation

2.2.13 Mbed TLS

Mbed TLS is a C library that implements cryptographic primitives, X.509 certificate manipulation and the SSL/TLS and DTLS protocols. Its small code footprint makes it suitable for embedded systems.

Note: ESP-IDF uses a fork of Mbed TLS which includes a few patches (related to hardware routines of certain modules like `bignum (MPI)` and `ECC`) over vanilla Mbed TLS.

Mbed TLS supports SSL 3.0 up to TLS 1.3 and DTLS 1.0 to 1.2 communication by providing the following:

- TCP/IP communication functions: listen, connect, accept, read/write.
- SSL/TLS communication functions: init, handshake, read/write.
- X.509 functions: CRT, CRL and key handling
- Random number generation
- Hashing
- Encryption/decryption

Supported TLS versions include SSL 3.0, TLS 1.0, TLS 1.1, TLS 1.2, and TLS 1.3, but on the latest ESP-IDF, SSL 3.0, TLS 1.0, and TLS 1.1 have been removed from Mbed TLS. Supported DTLS versions include DTLS 1.0, DTLS 1.1, and DTLS 1.2, but on the latest ESP-IDF, DTLS 1.0 has been removed from Mbed TLS.

Mbed TLS Documentation

For Mbed TLS documentation please refer to the following (upstream) pointers:

- API Reference
- Knowledge Base

Mbed TLS Support in ESP-IDF

Please find the information about the Mbed TLS versions presented in different branches of ESP-IDF here.

Note: Please refer the *Mbed TLS* to migrate from Mbed TLS version 2.x to version 3.0 or greater.

Application Examples

Examples in ESP-IDF use *ESP-TLS* which provides a simplified API interface for accessing the commonly used TLS functionality.

Refer to the examples protocols/https_server/simple (Simple HTTPS server) and protocols/https_request (Make HTTPS requests) for more information.

If the Mbed TLS API is to be used directly, refer to the example protocols/https_mbedtls.

Alternatives

ESP-TLS acts as an abstraction layer over the underlying SSL/TLS library and thus has an option to use Mbed TLS or wolfSSL as the underlying library. By default, only Mbed TLS is available and used in ESP-IDF whereas wolfSSL is available publicly at *<https://github.com/espressif/esp-wolfSSL>* with the upstream submodule pointer.

Please refer to *ESP-TLS: Underlying SSL/TLS Library Options* docs for more information on this and comparison of Mbed TLS and wolfSSL.

Important Config Options

Following is a brief list of important config options accessible at `Component Config -> mbedTLS`. The full list of config options can be found *here*.

- *CONFIG_MBEDTLS_SSL_PROTO_TLS1_2*: Support for TLS 1.2
- *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*: Support for TLS 1.3
- *CONFIG_MBEDTLS_CERTIFICATE_BUNDLE*: Support for trusted root certificate bundle (more about this: *ESP x509 Certificate Bundle*)
- *CONFIG_MBEDTLS_CLIENT_SSL_SESSION_TICKETS*: Support for TLS Session Resumption: Client session tickets
- *CONFIG_MBEDTLS_SERVER_SSL_SESSION_TICKETS*: Support for TLS Session Resumption: Server session tickets
- *CONFIG_MBEDTLS_HARDWARE_SHA*: Support for hardware SHA acceleration
- *CONFIG_MBEDTLS_HARDWARE_AES*: Support for hardware AES acceleration
- *CONFIG_MBEDTLS_HARDWARE_MPI*: Support for hardware MPI (bignum) acceleration

Note: Mbed TLS v3.0.0 and later support only TLS 1.2 and TLS 1.3 (SSL 3.0, TLS 1.0, TLS 1.1, and DTLS 1.0 are not supported). The support for TLS 1.3 is experimental and only supports the client-side. More information about this can be found out here.

Performance and Memory Tweaks

Reducing Heap Usage The following table shows typical memory usage with different configs when the protocols/https_request example (with Server Validation enabled) was run with Mbed TLS as the SSL/TLS library.

Mbed TLS Test	Related Configs	Heap Usage (approx.)
Default	NA	42196 B
Enable SSL Variable Length	*CONFIG_MBEDTLS_SSL_VARIABLE_BUFFER_LENGTH*	42120 B
Disable Keep Peer Certificate	*CONFIG_MBEDTLS_SSL_KEEP_PEER_CERTIFICATE*	38533 B
Enable Dynamic TX/RX Buffer	*CONFIG_MBEDTLS_DYNAMIC_BUFFER CONFIG_MBEDTLS_DYNAMIC_FREE_CONFIG_DATA CONFIG_MBEDTLS_DYNAMIC_FREE_CA_CERT*	22013 B

Note: These values are subject to change with change in configuration options and versions of Mbed TLS.

Reducing Binary Size Under `Component Config -> mbedTLS`, there are multiple Mbed TLS features which are enabled by default but can be disabled if not needed to save code size. More information can be about this can be found in *Minimizing Binary Size* docs.

Code examples for this API section are provided in the protocols directory of ESP-IDF examples.

2.2.14 IP Network Layer

Documentation for IP Network Layer protocols (below the Application Protocol layer) are provided in *Networking APIs*.

2.3 Bluetooth® API

2.3.1 Bluetooth® Common

Bluetooth® Generic Defines

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_bt_defs.h
- This header file can be included with:

```
#include "esp_bt_defs.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Structures

struct **esp_bt_uuid_t**

UUID type.

Public Members

uint16_t **len**

UUID length, 16bit, 32bit or 128bit

uint16_t **uuid16**

16bit UUID

uint32_t **uuid32**

32bit UUID

uint8_t **uuid128**[ESP_UUID_LEN_128]

128bit UUID

union *esp_bt_uuid_t*::[anonymous] **uuid**

UUID

Macros

ESP_BLUEDROID_STATUS_CHECK(status)

ESP_BT_STATUS_BASE_FOR_HCI_ERR

ESP_BT_OCTET16_LEN

ESP_BT_OCTET8_LEN

ESP_DEFAULT_GATT_IF

Default GATT interface id.

ESP_BLE_PRIM_ADV_INT_MIN

Minimum advertising interval for undirected and low duty cycle directed advertising

ESP_BLE_PRIM_ADV_INT_MAX

Maximum advertising interval for undirected and low duty cycle directed advertising

ESP_BLE_CONN_INT_MIN

relate to BTM_BLE_CONN_INT_MIN in stack/btm_ble_api.h

ESP_BLE_CONN_INT_MAX

relate to BTM_BLE_CONN_INT_MAX in stack/btm_ble_api.h

ESP_BLE_CONN_LATENCY_MAX

relate to ESP_BLE_CONN_LATENCY_MAX in stack/btm_ble_api.h

ESP_BLE_CONN_SUP_TOUT_MIN

relate to BTM_BLE_CONN_SUP_TOUT_MIN in stack/btm_ble_api.h

ESP_BLE_CONN_SUP_TOUT_MAX

relate to ESP_BLE_CONN_SUP_TOUT_MAX in stack/btm_ble_api.h

ESP_BLE_IS_VALID_PARAM(x, min, max)

Check the param is valid or not.

ESP_UUID_LEN_16

ESP_UUID_LEN_32

ESP_UUID_LEN_128

ESP_BD_ADDR_LEN

Bluetooth address length.

ESP_PEER_IRK_LEN

Bluetooth peer irk.

ESP_BLE_ENC_KEY_MASK

Used to exchange the encryption key in the init key & response key.

ESP_BLE_ID_KEY_MASK

Used to exchange the IRK key in the init key & response key.

ESP_BLE_CSR_KEY_MASK

Used to exchange the CSRK key in the init key & response key.

ESP_BLE_LINK_KEY_MASK

Used to exchange the link key(this key just used in the BLE & BR/EDR coexist mode) in the init key & response key.

ESP_APP_ID_MIN

Minimum of the application id.

ESP_APP_ID_MAX

Maximum of the application id.

ESP_BD_ADDR_STR

ESP_BD_ADDR_HEX(addr)

Type Definitions

typedef uint8_t **esp_bt_octet16_t**[ESP_BT_OCTET16_LEN]

typedef uint8_t **esp_bt_octet8_t**[ESP_BT_OCTET8_LEN]

typedef uint8_t **esp_link_key**[ESP_BT_OCTET16_LEN]

typedef uint8_t **esp_bd_addr_t**[ESP_BD_ADDR_LEN]

Bluetooth device address.

typedef uint8_t **esp_ble_key_mask_t**

Enumerations

enum **esp_bt_status_t**

Status Return Value.

Values:

enumerator **ESP_BT_STATUS_SUCCESS**

enumerator **ESP_BT_STATUS_FAIL**

enumerator **ESP_BT_STATUS_NOT_READY**

enumerator **ESP_BT_STATUS_NOMEM**

enumerator **ESP_BT_STATUS_BUSY**

enumerator **ESP_BT_STATUS_DONE**

enumerator **ESP_BT_STATUS_UNSUPPORTED**

enumerator **ESP_BT_STATUS_PARM_INVALID**

enumerator **ESP_BT_STATUS_UNHANDLED**

enumerator **ESP_BT_STATUS_AUTH_FAILURE**

enumerator **ESP_BT_STATUS_RMT_DEV_DOWN**

enumerator **ESP_BT_STATUS_AUTH_REJECTED**

enumerator **ESP_BT_STATUS_INVALID_STATIC_RAND_ADDR**

enumerator **ESP_BT_STATUS_PENDING**

enumerator **ESP_BT_STATUS_UNACCEPT_CONN_INTERVAL**

enumerator **ESP_BT_STATUS_PARAM_OUT_OF_RANGE**

enumerator **ESP_BT_STATUS_TIMEOUT**

enumerator **ESP_BT_STATUS_PEER_LE_DATA_LEN_UNSUPPORTED**

enumerator **ESP_BT_STATUS_CONTROL_LE_DATA_LEN_UNSUPPORTED**

enumerator **ESP_BT_STATUS_ERR_ILLEGAL_PARAMETER_FMT**

enumerator **ESP_BT_STATUS_MEMORY_FULL**

enumerator **ESP_BT_STATUS_EIR_TOO_LARGE**

enumerator **ESP_BT_STATUS_HCI_SUCCESS**

enumerator **ESP_BT_STATUS_HCI_ILLEGAL_COMMAND**

enumerator **ESP_BT_STATUS_HCI_NO_CONNECTION**

enumerator **ESP_BT_STATUS_HCI_HW_FAILURE**

enumerator **ESP_BT_STATUS_HCI_PAGE_TIMEOUT**

enumerator **ESP_BT_STATUS_HCI_AUTH_FAILURE**

enumerator **ESP_BT_STATUS_HCI_KEY_MISSING**

enumerator **ESP_BT_STATUS_HCI_MEMORY_FULL**

enumerator **ESP_BT_STATUS_HCI_CONNECTION_TOUT**

enumerator **ESP_BT_STATUS_HCI_MAX_NUM_OF_CONNECTIONS**

enumerator **ESP_BT_STATUS_HCI_MAX_NUM_OF_SCOS**

enumerator **ESP_BT_STATUS_HCI_CONNECTION_EXISTS**

enumerator **ESP_BT_STATUS_HCI_COMMAND_DISALLOWED**

enumerator **ESP_BT_STATUS_HCI_HOST_REJECT_RESOURCES**

enumerator **ESP_BT_STATUS_HCI_HOST_REJECT_SECURITY**

enumerator **ESP_BT_STATUS_HCI_HOST_REJECT_DEVICE**

enumerator **ESP_BT_STATUS_HCI_HOST_TIMEOUT**

enumerator **ESP_BT_STATUS_HCI_UNSUPPORTED_VALUE**

enumerator **ESP_BT_STATUS_HCI_ILLEGAL_PARAMETER_FMT**

enumerator **ESP_BT_STATUS_HCI_PEER_USER**

enumerator **ESP_BT_STATUS_HCI_PEER_LOW_RESOURCES**

enumerator **ESP_BT_STATUS_HCI_PEER_POWER_OFF**

enumerator **ESP_BT_STATUS_HCI_CONN_CAUSE_LOCAL_HOST**

enumerator **ESP_BT_STATUS_HCI_REPEATED_ATTEMPTS**

enumerator **ESP_BT_STATUS_HCI_PAIRING_NOT_ALLOWED**

enumerator **ESP_BT_STATUS_HCI_UNKNOWN_LMP_PDU**

enumerator **ESP_BT_STATUS_HCI_UNSUPPORTED_REM_FEATURE**

enumerator **ESP_BT_STATUS_HCI_SCO_OFFSET_REJECTED**

enumerator **ESP_BT_STATUS_HCI_SCO_INTERVAL_REJECTED**

enumerator **ESP_BT_STATUS_HCI_SCO_AIR_MODE**

enumerator **ESP_BT_STATUS_HCI_INVALID_LMP_PARAM**

enumerator **ESP_BT_STATUS_HCI_UNSPECIFIED**

enumerator **ESP_BT_STATUS_HCI_UNSUPPORTED_LMP_PARAMETERS**

enumerator **ESP_BT_STATUS_HCI_ROLE_CHANGE_NOT_ALLOWED**

enumerator **ESP_BT_STATUS_HCI_LMP_RESPONSE_TIMEOUT**

enumerator **ESP_BT_STATUS_HCI_LMP_ERR_TRANS_COLLISION**

enumerator **ESP_BT_STATUS_HCI_LMP_PDU_NOT_ALLOWED**

enumerator **ESP_BT_STATUS_HCI_ENCRY_MODE_NOT_ACCEPTABLE**

enumerator **ESP_BT_STATUS_HCI_UNIT_KEY_USED**

enumerator **ESP_BT_STATUS_HCI_QOS_NOT_SUPPORTED**

enumerator **ESP_BT_STATUS_HCI_INSTANT_PASSED**

enumerator **ESP_BT_STATUS_HCI_PAIRING_WITH_UNIT_KEY_NOT_SUPPORTED**

enumerator **ESP_BT_STATUS_HCI_DIFF_TRANSACTION_COLLISION**

enumerator **ESP_BT_STATUS_HCI_UNDEFINED_0x2B**

enumerator **ESP_BT_STATUS_HCI_QOS_UNACCEPTABLE_PARAM**

enumerator **ESP_BT_STATUS_HCI_QOS_REJECTED**

enumerator **ESP_BT_STATUS_HCI_CHAN_CLASSIF_NOT_SUPPORTED**

enumerator **ESP_BT_STATUS_HCI_INSUFFCIENT_SECURITY**

enumerator **ESP_BT_STATUS_HCI_PARAM_OUT_OF_RANGE**

enumerator **ESP_BT_STATUS_HCI_UNDEFINED_0x31**

enumerator **ESP_BT_STATUS_HCI_ROLE_SWITCH_PENDING**

enumerator **ESP_BT_STATUS_HCI_UNDEFINED_0x33**

enumerator **ESP_BT_STATUS_HCI_RESERVED_SLOT_VIOLATION**

enumerator **ESP_BT_STATUS_HCI_ROLE_SWITCH_FAILED**

enumerator **ESP_BT_STATUS_HCI_INQ_RSP_DATA_TOO_LARGE**

enumerator **ESP_BT_STATUS_HCI_SIMPLE_PAIRING_NOT_SUPPORTED**

enumerator **ESP_BT_STATUS_HCI_HOST_BUSY_PAIRING**

enumerator **ESP_BT_STATUS_HCI_REJ_NO_SUITABLE_CHANNEL**

enumerator **ESP_BT_STATUS_HCI_CONTROLLER_BUSY**

enumerator **ESP_BT_STATUS_HCI_UNACCEPT_CONN_INTERVAL**

enumerator **ESP_BT_STATUS_HCI_DIRECTED_ADVERTISING_TIMEOUT**

enumerator **ESP_BT_STATUS_HCI_CONN_TOUT_DUE_TO_MIC_FAILURE**

enumerator **ESP_BT_STATUS_HCI_CONN_FAILED_ESTABLISHMENT**

enumerator **ESP_BT_STATUS_HCI_MAC_CONNECTION_FAILED**

enum **esp_bt_dev_type_t**

Bluetooth device type.

Values:

enumerator **ESP_BT_DEVICE_TYPE_BREDR**

enumerator **ESP_BT_DEVICE_TYPE_BLE**

enumerator **ESP_BT_DEVICE_TYPE_DUMO**

enum **esp_ble_addr_type_t**

BLE device address type.

Values:

enumerator **BLE_ADDR_TYPE_PUBLIC**

Public Device Address

enumerator **BLE_ADDR_TYPE_RANDOM**

Random Device Address. To set this address, use the function esp_ble_gap_set_rand_addr(esp_bd_addr_t rand_addr)

enumerator **BLE_ADDR_TYPE_RPA_PUBLIC**

Resolvable Private Address (RPA) with public identity address

enumerator **BLE_ADDR_TYPE_RPA_RANDOM**

Resolvable Private Address (RPA) with random identity address. To set this address, use the function esp_ble_gap_set_rand_addr(esp_bd_addr_t rand_addr)

enum **esp_ble_wl_addr_type_t**

white list address type

Values:

enumerator **BLE_WL_ADDR_TYPE_PUBLIC**

enumerator **BLE_WL_ADDR_TYPE_RANDOM**

Bluetooth® Main API

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_bt_main.h
- This header file can be included with:

```
#include "esp_bt_main.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_bluedroid_status_t **esp_bluedroid_get_status** (void)

Get bluetooth stack status.

Returns Bluetooth stack status

esp_err_t **esp_bluedroid_enable** (void)

Enable bluetooth, must after esp_bluedroid_init()/esp_bluedroid_init_with_cfg().

Returns

- ESP_OK : Succeed
- Other : Failed

esp_err_t **esp_bluedroid_disable** (void)

Disable Bluetooth, must be called prior to esp_bluedroid_deinit().

Note: Before calling this API, ensure that all activities related to the application, such as connections, scans, etc., are properly closed.

Returns

- ESP_OK : Succeed
- Other : Failed

esp_err_t **esp_bluedroid_init** (void)

Init and alloc the resource for bluetooth, must be prior to every bluetooth stuff.

Returns

- ESP_OK : Succeed
- Other : Failed

esp_err_t **esp_bluedroid_init_with_cfg** (*esp_bluedroid_config_t* *cfg)

Init and alloc the resource for bluetooth, must be prior to every bluetooth stuff.

Parameters **cfg** -- Initial configuration of ESP Bluedroid stack.

Returns

- ESP_OK : Succeed
- Other : Failed

esp_err_t **esp_bluedroid_deinit** (void)

Deinit and free the resource for bluetooth, must be after every bluetooth stuff.

Returns

- ESP_OK : Succeed
- Other : Failed

Structures

struct **esp_bluedroid_config_t**

Bluetooth stack configuration.

Public Members

bool **ssp_en**

Whether SSP(secure simple pairing) or legacy pairing is used for Classic Bluetooth

Macros

BT_BLUEDROID_INIT_CONFIG_DEFAULT()

Enumerations

enum **esp_bluedroid_status_t**

Bluetooth stack status type, to indicate whether the bluetooth stack is ready.

Values:

enumerator **ESP_BLUEDROID_STATUS_UNINITIALIZED**

Bluetooth not initialized

enumerator **ESP_BLUEDROID_STATUS_INITIALIZED**

Bluetooth initialized but not enabled

enumerator **ESP_BLUEDROID_STATUS_ENABLED**

Bluetooth initialized and enabled

Bluetooth® Device APIs

Overview Bluetooth device reference APIs.

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_bt_device.h
- This header file can be included with:

```
#include "esp_bt_device.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_bt_dev_register_callback**(*esp_bt_dev_cb_t* callback)

register callback function. This function should be called after esp_bluedroid_enable() completes successfully

Returns

- ESP_OK : Succeed
- ESP_FAIL: others

const uint8_t ***esp_bt_dev_get_address**(void)

Get bluetooth device address. Must use after "esp_bluedroid_enable".

Returns bluetooth device address (six bytes), or NULL if bluetooth stack is not enabled

esp_err_t **esp_bt_dev_set_device_name**(const char *name)

Set bluetooth device name. This function should be called after esp_bluedroid_enable() completes successfully.

A BR/EDR/LE device type shall have a single Bluetooth device name which shall be identical irrespective of the physical channel used to perform the name discovery procedure.

Parameters **name** -- **[in]** : device name to be set

Returns

- ESP_OK : Succeed
- ESP_ERR_INVALID_ARG : if name is NULL pointer or empty, or string length out of limit
- ESP_ERR_INVALID_STATE : if bluetooth stack is not yet enabled
- ESP_FAIL : others

esp_err_t **esp_bt_dev_get_device_name**(void)

Get bluetooth device name. This function should be called after esp_bluedroid_enable() completes successfully.

A BR/EDR/LE device type shall have a single Bluetooth device name which shall be identical irrespective of the physical channel used to perform the name discovery procedure.

Returns

- ESP_OK : Succeed
- ESP_ERR_INVALID_STATE : if bluetooth stack is not yet enabled
- ESP_FAIL : others

esp_err_t **esp_bt_dev_coex_status_config**(*esp_bt_dev_coex_type_t* type, *esp_bt_dev_coex_op_t* op, uint8_t status)

Config bluetooth device coexis status. This function should be called after esp_bluedroid_enable() completes successfully.

Parameters

- **type** -- **[in]** : coexist type to operate on
- **op** -- **[in]** : clear or set coexist status
- **status** -- **[in]** : coexist status to be configured

Returns

- ESP_OK : Succeed
- ESP_ERR_INVALID_ARG : if name is NULL pointer or empty, or string length out of limit
- ESP_ERR_INVALID_STATE : if bluetooth stack is not yet enabled
- ESP_FAIL : others

esp_err_t **esp_bt_config_file_path_update**(const char *file_path)

This function is used to update the path name of bluetooth bond keys saved in the NVS module and need to be called before esp_bluedroid_init().

Parameters **file_path** -- **[in]** the name of config file path, the length of file_path should be less than NVS_NS_NAME_MAX_SIZE

Returns

- ESP_OK: success
- other: failed

Unions

union **esp_bt_dev_cb_param_t**

#include <esp_bt_device.h> BT device callback parameters.

Public Members

struct *esp_bt_dev_cb_param_t*::*name_res_param* **name_res**

discovery result parameter struct

struct **name_res_param**

#include <esp_bt_device.h> ESP_BT_DEV_NAME_RES_EVT.

Public Members

esp_bt_status_t **status**

Status of getting device name

char ***name**

Name of Bluetooth device

Macros

ESP_BT_DEV_COEX_BLE_ST_MESH_CONFIG

ESP_BT_DEV_COEX_BLE_ST_MESH_TRAFFIC

ESP_BT_DEV_COEX_BLE_ST_MESH_STANDBY

ESP_BT_DEV_COEX_BT_ST_A2DP_STREAMING

ESP_BT_DEV_COEX_BT_ST_A2DP_PAUSED

ESP_BT_DEV_COEX_OP_CLEAR

ESP_BT_DEV_COEX_OP_SET

Type Definitions

typedef uint8_t **esp_bt_dev_coex_op_t**

typedef void (***esp_bt_dev_cb_t**)(*esp_bt_dev_cb_event_t* event, *esp_bt_dev_cb_param_t* *param)

bluetooth device callback function type

Param event : Event type
Param param : Pointer to callback parameter

Enumerations

enum **esp_bt_dev_coex_type_t**

Bluetooth device coex type.

Values:

enumerator **ESP_BT_DEV_COEX_TYPE_BLE**

enumerator **ESP_BT_DEV_COEX_TYPE_BT**

enum **esp_bt_dev_cb_event_t**

BT device callback events.

Values:

enumerator **ESP_BT_DEV_NAME_RES_EVT**

Device name result event

enumerator **ESP_BT_DEV_EVT_MAX**

2.3.2 Bluetooth® Low Energy (Bluetooth LE)

GAP API

Application Example Check the bluetooth/bluedroid/ble folder in ESP-IDF examples, which contains the following demos and their tutorials:

- The following shows an SMP security client demo with its tutorial. This demo initiates its security parameters and acts as a GATT client, which can send a security request to the peer device and then complete the encryption procedure.
  - bluetooth/bluedroid/ble/gatt_security_client
  - GATT Security Client Example Walkthrough
- The following shows an SMP security server demo with its tutorial. This demo initiates its security parameters and acts as a GATT server, which can send a pair request to the peer device and then complete the encryption procedure.
  - bluetooth/bluedroid/ble/gatt_security_server
  - GATT Security Server Example Walkthrough

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_gap_ble_api.h
- This header file can be included with:

```
#include "esp_gap_ble_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_gap_register_callback**(*esp_gap_ble_cb_t* callback)

This function is called to occur gap event, such as scan result.

Parameters **callback** -- **[in]** callback function

Returns

- ESP_OK : success
- other : failed

esp_gap_ble_cb_t **esp_ble_gap_get_callback**(void)

This function is called to get the current gap callback.

Returns

- esp_gap_ble_cb_t : callback function

esp_err_t **esp_ble_gap_config_adv_data**(*esp_ble_adv_data_t* *adv_data)

This function is called to override the BTA default ADV parameters.

Parameters **adv_data** -- **[in]** Pointer to User defined ADV data structure. This memory space can not be freed until callback of config_adv_data is received.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_scan_params**(*esp_ble_scan_params_t* *scan_params)

This function is called to set scan parameters.

Parameters **scan_params** -- **[in]** Pointer to User defined scan_params data structure. This memory space can not be freed until callback of set_scan_params

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_start_scanning**(uint32_t duration)

This procedure keep the device scanning the peer device which advertising on the air.

Parameters **duration** -- **[in]** Keeping the scanning time, the unit is second.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_stop_scanning**(void)

This function call to stop the device scanning the peer device which advertising on the air.

Returns

- ESP_OK : success
  - other : failed

esp_err_t **esp_ble_gap_start_advertising**(*esp_ble_adv_params_t* *adv_params)

This function is called to start advertising.

Parameters **adv_params** -- **[in]** pointer to User defined adv_params data structure.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_stop_advertising**(void)

This function is called to stop advertising.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_update_conn_params** (*esp_ble_conn_update_params_t* *params)

Update connection parameters, can only be used when connection is up.

Parameters **params** -- **[in]** - connection update parameters

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_pkt_data_len** (*esp_bd_addr_t* remote_device, uint16_t tx_data_length)

This function is to set maximum LE data packet size.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_rand_addr** (*esp_bd_addr_t* rand_addr)

This function allows configuring either a Non-Resolvable Private Address or a Static Random Address.

Parameters **rand_addr** -- **[in]** The address to be configured. Refer to the table below for possible address subtypes:

```
          | address [47:46] | Address Type                 |␣
↪Corresponding API                      |
          |-----------------|------------------------------|--------
↪-------------------------------|
          |      0b00       | Non-Resolvable Private       | esp_
↪ble_gap_addr_create_nrpa           |
          |                 | Address (NRPA)               |       ␣
↪                                    |
          |-----------------|------------------------------|--------
↪-------------------------------|
          |      0b11       | Static Random Address        | esp_
↪ble_gap_addr_create_static         |
          |-----------------|------------------------------|--------
↪-------------------------------|
```

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_addr_create_static** (*esp_bd_addr_t* rand_addr)

Create a static device address.

Parameters **rand_addr** -- **[out]** Pointer to the buffer where the static device address will be stored.

Returns - ESP_OK : Success

- Other : Failed

esp_err_t **esp_ble_gap_addr_create_nrpa** (*esp_bd_addr_t* rand_addr)

Create a non-resolvable private address (NRPA)

Parameters **rand_addr** -- **[out]** Pointer to the buffer where the NRPA will be stored.

Returns - ESP_OK : Success

- Other : Failed

esp_err_t **esp_ble_gap_set_resolvable_private_address_timeout** (uint16_t rpa_timeout)

This function sets the length of time the Controller uses a Resolvable Private Address before generating and starting to use a new resolvable private address.

Note: Note: This function is currently not supported on the ESP32 but will be enabled in a future update.

Parameters **rpa_timeout** -- **[in]** The timeout duration in seconds for how long a Resolvable Private Address is used before a new one is generated. The value must be within the range specified by the Bluetooth specification (0x0001 to 0x0E10), which corresponds to a time range of 1 second to 1 hour. The default value is 0x0384 (900 seconds or 15 minutes).

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_add_device_to_resolving_list**(*esp_bd_addr_t* peer_addr, uint8_t addr_type, uint8_t *peer_irk)

This function adds a device to the resolving list used to generate and resolve Resolvable Private Addresses in the Controller.

Note: Note: This function shall not be used when address resolution is enabled in the Controller and:

- Advertising (other than periodic advertising) is enabled,
- Scanning is enabled, or
- an HCI_LE_Create_Connection, HCI_LE_Extended_Create_Connection, or HCI_LE_Periodic_Advertising_Create_Sync command is pending. This command may be used at any time when address resolution is disabled in the Controller. The added device shall be set to Network Privacy mode.

Parameters

- **peer_addr** -- **[in]** The peer identity address of the device to be added to the resolving list.
- **addr_type** -- **[in]** The address type of the peer identity address (BLE_ADDR_TYPE_PUBLIC or BLE_ADDR_TYPE_RANDOM).
- **peer_irk** -- **[in]** The Identity Resolving Key (IRK) of the device.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_clear_rand_addr**(void)

This function clears the random address for the application.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_config_local_privacy**(bool privacy_enable)

Enable/disable privacy (including address resolution) on the local device.

Parameters **privacy_enable** -- **[in]** - enable/disable privacy on remote device.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_config_local_icon**(uint16_t icon)

set local gap appearance icon

Parameters **icon** -- **[in]** - External appearance value, these values are defined by the Bluetooth SIG, please refer to https://www.bluetooth.com/specifications/assigned-numbers/

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_update_whitelist**(bool add_remove, *esp_bd_addr_t* remote_bda, *esp_ble_wl_addr_type_t* wl_addr_type)

Add or remove device from white list.

Parameters
- **add_remove** -- **[in]** the value is true if added the ble device to the white list, and false remove to the white list.
- **remote_bda** -- **[in]** the remote device address add/remove from the white list.
- **wl_addr_type** -- **[in]** whitelist address type

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_clear_whitelist**(void)

Clear all white list.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_get_whitelist_size**(uint16_t *length)

Get the whitelist size in the controller.

Parameters **length** -- **[out]** the white list length.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_prefer_conn_params**(*esp_bd_addr_t* bd_addr, uint16_t min_conn_int, uint16_t max_conn_int, uint16_t slave_latency, uint16_t supervision_tout)

This function is called to set the preferred connection parameters when default connection parameter is not desired before connecting. This API can only be used in the master role.

Parameters
- **bd_addr** -- **[in]** BD address of the peripheral
- **min_conn_int** -- **[in]** minimum preferred connection interval
- **max_conn_int** -- **[in]** maximum preferred connection interval
- **slave_latency** -- **[in]** preferred slave latency
- **supervision_tout** -- **[in]** preferred supervision timeout

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_device_name**(const char *name)

Set device name to the local device Note: This API don't affect the advertising data.

Parameters **name** -- **[in]** - device name.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_get_device_name**(void)

Get device name of the local device.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_get_local_used_addr**(*esp_bd_addr_t* local_used_addr, uint8_t *addr_type)

This function is called to get local used address and address type. uint8_t *esp_bt_dev_get_address(void) get the public address.

Parameters
- **local_used_addr** -- **[in]** - current local used ble address (six bytes)
- **addr_type** -- **[in]** - ble address type

Returns - ESP_OK : success

- other : failed

uint8_t *`esp_ble_resolve_adv_data`(uint8_t *adv_data, uint8_t type, uint8_t *length)

This function is called to get ADV data for a specific type.

Parameters
- **`adv_data`** -- **[in]** - pointer of ADV data which to be resolved
- **`type`** -- **[in]** - finding ADV data type
- **`length`** -- **[out]** - return the length of ADV data not including type

Returns pointer of ADV data

esp_err_t `esp_ble_gap_config_adv_data_raw`(uint8_t *raw_data, uint32_t raw_data_len)

This function is called to set raw advertising data. User need to fill ADV data by self.

Parameters
- **`raw_data`** -- **[in]** : raw advertising data with the format: [Length 1][Data Type 1][Data 1][Length 2][Data Type 2][Data 2] ...
- **`raw_data_len`** -- **[in]** : raw advertising data length , less than 31 bytes

Returns
- ESP_OK : success
- other : failed

esp_err_t `esp_ble_gap_config_scan_rsp_data_raw`(uint8_t *raw_data, uint32_t raw_data_len)

This function is called to set raw scan response data. User need to fill scan response data by self.

Parameters
- **`raw_data`** -- **[in]** : raw scan response data
- **`raw_data_len`** -- **[in]** : raw scan response data length , less than 31 bytes

Returns
- ESP_OK : success
- other : failed

esp_err_t `esp_ble_gap_read_rssi`(*esp_bd_addr_t* remote_addr)

This function is called to read the RSSI of remote device. The address of link policy results are returned in the gap callback function with ESP_GAP_BLE_READ_RSSI_COMPLETE_EVT event.

Parameters **`remote_addr`** -- **[in]** : The remote connection device address.

Returns
- ESP_OK : success
- other : failed

esp_err_t `esp_ble_gap_add_duplicate_scan_exceptional_device`(*esp_ble_duplicate_exceptional_info_type_t* type, *esp_duplicate_info_t* device_info)

This function is called to add a device info into the duplicate scan exceptional list.

Parameters
- **`type`** -- **[in]** device info type, it is defined in esp_ble_duplicate_exceptional_info_type_t when type is MESH_BEACON_TYPE, MESH_PROV_SRV_ADV or MESH_PROXY_SRV_ADV , device_info is invalid.
- **`device_info`** -- **[in]** the device information.

Returns
- ESP_OK : success
- other : failed

esp_err_t `esp_ble_gap_remove_duplicate_scan_exceptional_device`(*esp_ble_duplicate_exceptional_info_type_t* type, *esp_duplicate_info_t* device_info)

This function is called to remove a device info from the duplicate scan exceptional list.

Parameters

- **type** -- **[in]** device info type, it is defined in esp_ble_duplicate_exceptional_info_type_t when type is MESH_BEACON_TYPE, MESH_PROV_SRV_ADV or MESH_PROXY_SRV_ADV , device_info is invalid.
- **device_info** -- **[in]** the device information.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_clean_duplicate_scan_exceptional_list**(*esp_duplicate_scan_exceptional_list_type_t* list_type)

This function is called to clean the duplicate scan exceptional list. This API will delete all device information in the duplicate scan exceptional list.

Parameters **list_type** -- **[in]** duplicate scan exceptional list type, the value can be one or more of esp_duplicate_scan_exceptional_list_type_t.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_security_param**(*esp_ble_sm_param_t* param_type, void *value, uint8_t len)

Set a GAP security parameter value. Overrides the default value.

```
                Secure connection is highly recommended to avoid some major
                vulnerabilities like 'Impersonation in the Pin Pairing Protocol'
                (CVE-2020-26555) and 'Authentication of the LE Legacy Pairing
                Protocol'.

                To accept only `secure connection mode`, it is necessary do as␣
↪following:

                1. Set bit `ESP_LE_AUTH_REQ_SC_ONLY` (`param_type` is
                `ESP_BLE_SM_AUTHEN_REQ_MODE`), bit `ESP_LE_AUTH_BOND` and bit
                `ESP_LE_AUTH_REQ_MITM` is optional as required.

                2. Set to `ESP_BLE_ONLY_ACCEPT_SPECIFIED_AUTH_ENABLE` (`param_
↪type` is
                `ESP_BLE_SM_ONLY_ACCEPT_SPECIFIED_SEC_AUTH`).
```

Parameters

- **param_type** -- **[in]** : the type of the param which to be set
- **value** -- **[in]** : the param value
- **len** -- **[in]** : the length of the param value

Returns - ESP_OK : success

- other : failed

esp_err_t **esp_ble_gap_security_rsp**(*esp_bd_addr_t* bd_addr, bool accept)

Grant security request access.

Parameters

- **bd_addr** -- **[in]** : BD address of the peer
- **accept** -- **[in]** : accept the security request or not

Returns - ESP_OK : success

- other : failed

esp_err_t **esp_ble_set_encryption**(*esp_bd_addr_t* bd_addr, *esp_ble_sec_act_t* sec_act)

Set a gap parameter value. Use this function to change the default GAP parameter values.

Parameters

- **bd_addr** -- **[in]** : the address of the peer device need to encryption

- **sec_act** -- **[in]** : This is the security action to indicate what kind of BLE security level is required for the BLE link if the BLE is supported

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_passkey_reply**(*esp_bd_addr_t* bd_addr, bool accept, uint32_t passkey)

Reply the key value to the peer device in the legacy connection stage.

Parameters
- **bd_addr** -- **[in]** : BD address of the peer
- **accept** -- **[in]** : passkey entry successful or declined.
- **passkey** -- **[in]** : passkey value, must be a 6 digit number, can be lead by 0.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_confirm_reply**(*esp_bd_addr_t* bd_addr, bool accept)

Reply the confirm value to the peer device in the secure connection stage.

Parameters
- **bd_addr** -- **[in]** : BD address of the peer device
- **accept** -- **[in]** : numbers to compare are the same or different.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_remove_bond_device**(*esp_bd_addr_t* bd_addr)

Removes a device from the security database list of peer device. It manages unpairing event while connected.

Parameters **bd_addr** -- **[in]** : BD address of the peer device

Returns - ESP_OK : success
- other : failed

int **esp_ble_get_bond_device_num**(void)

Get the device number from the security database list of peer device. It will return the device bonded number immediately.

Returns - >= 0 : bonded devices number.
- ESP_FAIL : failed

esp_err_t **esp_ble_get_bond_device_list**(int *dev_num, *esp_ble_bond_dev_t* *dev_list)

Get the device from the security database list of peer device. It will return the device bonded information immediately.

Parameters
- **dev_num** -- **[inout]** Indicate the dev_list array(buffer) size as input. If dev_num is large enough, it means the actual number as output. Suggest that dev_num value equal to esp_ble_get_bond_device_num().
- **dev_list** -- **[out]** an array(buffer) of `esp_ble_bond_dev_t` type. Use for storing the bonded devices address. The dev_list should be allocated by who call this API.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_oob_req_reply**(*esp_bd_addr_t* bd_addr, uint8_t *TK, uint8_t len)

This function is called to provide the OOB data for SMP in response to ESP_GAP_BLE_OOB_REQ_EVT.

Parameters
- **bd_addr** -- **[in]** BD address of the peer device.
- **TK** -- **[in]** Temporary Key value, the TK value shall be a 128-bit random number
- **len** -- **[in]** length of temporary key, should always be 128-bit

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_sc_oob_req_reply**(*esp_bd_addr_t* bd_addr, uint8_t p_c[16], uint8_t p_r[16])

This function is called to provide the OOB data for SMP in response to ESP_GAP_BLE_SC_OOB_REQ_EVT.

Parameters
- **bd_addr** -- **[in]** BD address of the peer device.
- **p_c** -- **[in]** Confirmation value, it shall be a 128-bit random number
- **p_r** -- **[in]** Randomizer value, it should be a 128-bit random number

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_create_sc_oob_data**(void)

This function is called to create the OOB data for SMP when secure connection.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_disconnect**(*esp_bd_addr_t* remote_device)

This function is to disconnect the physical connection of the peer device gattc may have multiple virtual GATT server connections when multiple app_id registered. esp_ble_gattc_close (esp_gatt_if_t gattc_if, uint16_t conn_id) only close one virtual GATT server connection. if there exist other virtual GATT server connections, it does not disconnect the physical connection. esp_ble_gap_disconnect(esp_bd_addr_t remote_device) disconnect the physical connection directly.

Parameters **remote_device** -- **[in]** : BD address of the peer device
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_get_current_conn_params**(*esp_bd_addr_t* bd_addr, *esp_gap_conn_params_t* *conn_params)

This function is called to read the connection parameters information of the device.

Parameters
- **bd_addr** -- **[in]** BD address of the peer device.
- **conn_params** -- **[out]** the connection parameters information

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_gap_ble_set_channels**(*esp_gap_ble_channels* channels)

BLE set channels.

Parameters **channels** -- **[in]** : The n th such field (in the range 0 to 36) contains the value for the link layer channel index n. 0 means channel n is bad. 1 means channel n is unknown. The most significant bits are reserved and shall be set to 0. At least one channel shall be marked as unknown.

Returns - ESP_OK : success
- ESP_ERR_INVALID_STATE: if bluetooth stack is not yet enabled
- other : failed

esp_err_t **esp_gap_ble_set_authorization**(*esp_bd_addr_t* bd_addr, bool authorize)

This function is called to authorized a link after Authentication(MITM protection)

Parameters
- **bd_addr** -- **[in]** BD address of the peer device.
- **authorize** -- **[out]** Authorized the link or not.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_read_phy**(*esp_bd_addr_t* bd_addr)

This function is used to read the current transmitter PHY and receiver PHY on the connection identified by remote address.

Parameters **bd_addr** -- **[in]** : BD address of the peer device

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_preferred_default_phy**(*esp_ble_gap_phy_mask_t* tx_phy_mask, *esp_ble_gap_phy_mask_t* rx_phy_mask)

This function is used to allows the Host to specify its preferred values for the transmitter PHY and receiver PHY to be used for all subsequent connections over the LE transport.

Parameters
- **tx_phy_mask** -- **[in]** : indicates the transmitter PHYs that the Host prefers the Controller to use
- **rx_phy_mask** -- **[in]** : indicates the receiver PHYs that the Host prefers the Controller to use

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_preferred_phy**(*esp_bd_addr_t* bd_addr, *esp_ble_gap_all_phys_t* all_phys_mask, *esp_ble_gap_phy_mask_t* tx_phy_mask, *esp_ble_gap_phy_mask_t* rx_phy_mask, *esp_ble_gap_prefer_phy_options_t* phy_options)

This function is used to set the PHY preferences for the connection identified by the remote address. The Controller might not be able to make the change (e.g. because the peer does not support the requested PHY) or may decide that the current PHY is preferable.

Parameters
- **bd_addr** -- **[in]** : remote address
- **all_phys_mask** -- **[in]** : a bit field that allows the Host to specify
- **tx_phy_mask** -- **[in]** : a bit field that indicates the transmitter PHYs that the Host prefers the Controller to use
- **rx_phy_mask** -- **[in]** : a bit field that indicates the receiver PHYs that the Host prefers the Controller to use
- **phy_options** -- **[in]** : a bit field that allows the Host to specify options for PHYs

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_set_rand_addr**(uint8_t instance, *esp_bd_addr_t* rand_addr)

This function is used by the Host to set the random device address specified by the Random_Address parameter.

Parameters
- **instance** -- **[in]** : Used to identify an advertising set
- **rand_addr** -- **[in]** : Random Device Address

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_set_params**(uint8_t instance, const *esp_ble_gap_ext_adv_params_t* *params)

This function is used by the Host to set the advertising parameters.

Parameters
- **instance** -- **[in]** : identifies the advertising set whose parameters are being configured.
- **params** -- **[in]** : advertising parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_config_ext_adv_data_raw**(uint8_t instance, uint16_t length, const uint8_t *data)

This function is used to set the data used in advertising PDUs that have a data field.

Parameters
- **instance** -- **[in]** : identifies the advertising set whose data are being configured
- **length** -- **[in]** : data length
- **data** -- **[in]** : data information

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_config_ext_scan_rsp_data_raw**(uint8_t instance, uint16_t length, const uint8_t *scan_rsp_data)

This function is used to provide scan response data used in scanning response PDUs.

Parameters
- **instance** -- **[in]** : identifies the advertising set whose response data are being configured.
- **length** -- **[in]** : responsedata length
- **scan_rsp_data** -- **[in]** : response data information

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_start**(uint8_t num_adv, const *esp_ble_gap_ext_adv_t* *ext_adv)

This function is used to request the Controller to enable one or more advertising sets using the advertising sets identified by the instance parameter.

Parameters
- **num_adv** -- **[in]** : Number of advertising sets to enable or disable
- **ext_adv** -- **[in]** : adv parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_stop**(uint8_t num_adv, const uint8_t *ext_adv_inst)

This function is used to request the Controller to disable one or more advertising sets using the advertising sets identified by the instance parameter.

Parameters
- **num_adv** -- **[in]** : Number of advertising sets to enable or disable
- **ext_adv_inst** -- **[in]** : ext adv instance

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_set_remove**(uint8_t instance)

This function is used to remove an advertising set from the Controller.

Parameters **instance** -- **[in]** : Used to identify an advertising set

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_ext_adv_set_clear**(void)

This function is used to remove all existing advertising sets from the Controller.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_set_params**(uint8_t instance, const *esp_ble_gap_periodic_adv_params_t* *params)

This function is used by the Host to set the parameters for periodic advertising.

Parameters
- **instance** -- **[in]** : identifies the advertising set whose periodic advertising parameters are being configured.
- **params** -- **[in]** : periodic adv parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_config_periodic_adv_data_raw**(uint8_t instance, uint16_t length, const uint8_t *data)

This function is used to set the data used in periodic advertising PDUs.

Parameters
- **instance** -- **[in]** : identifies the advertising set whose periodic advertising parameters are being configured.
- **length** -- **[in]** : the length of periodic data
- **data** -- **[in]** : periodic data information

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_start** (uint8_t instance)

This function is used to request the Controller to enable the periodic advertising for the advertising set specified.

Parameters **instance** -- **[in]** : Used to identify an advertising set
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_stop** (uint8_t instance)

This function is used to request the Controller to disable the periodic advertising for the advertising set specified.

Parameters **instance** -- **[in]** : Used to identify an advertising set
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_ext_scan_params** (const *esp_ble_ext_scan_params_t* *params)

This function is used to set the extended scan parameters to be used on the advertising channels.

Parameters **params** -- **[in]** : scan parameters
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_start_ext_scan** (uint32_t duration, uint16_t period)

This function is used to enable scanning.

Parameters
- **duration** -- **[in]** Scan duration time, where Time = N * 10 ms. Range: 0x0001 to 0xFFFF.
- **period** -- **[in]** Time interval from when the Controller started its last Scan Duration until it begins the subsequent Scan Duration. Time = N * 1.28 sec. Range: 0x0001 to 0xFFFF.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_stop_ext_scan** (void)

This function is used to disable scanning.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_create_sync** (const *esp_ble_gap_periodic_adv_sync_params_t* *params)

This function is used to synchronize with periodic advertising from an advertiser and begin receiving periodic advertising packets.

Parameters **params** -- **[in]** : sync parameters
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_sync_cancel** (void)

This function is used to cancel the LE_Periodic_Advertising_Create_Sync command while it is pending.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_sync_terminate** (uint16_t sync_handle)

This function is used to stop reception of the periodic advertising identified by the Sync Handle parameter.

Parameters **sync_handle** -- **[in]** : identify the periodic advertiser
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_add_dev_to_list** (*esp_ble_addr_type_t* addr_type, *esp_bd_addr_t* addr, uint8_t sid)

This function is used to add a single device to the Periodic Advertiser list stored in the Controller.

Parameters
- **addr_type** -- **[in]** : address type
- **addr** -- **[in]** : Device Address
- **sid** -- **[in]** : Advertising SID subfield in the ADI field used to identify the Periodic Advertising

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_remove_dev_from_list** (*esp_ble_addr_type_t* addr_type, *esp_bd_addr_t* addr, uint8_t sid)

This function is used to remove one device from the list of Periodic Advertisers stored in the Controller. Removals from the Periodic Advertisers List take effect immediately.

Parameters
- **addr_type** -- **[in]** : address type
- **addr** -- **[in]** : Device Address
- **sid** -- **[in]** : Advertising SID subfield in the ADI field used to identify the Periodic Advertising

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_clear_dev** (void)

This function is used to remove all devices from the list of Periodic Advertisers in the Controller.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_prefer_ext_connect_params_set** (*esp_bd_addr_t* addr, *esp_ble_gap_phy_mask_t* phy_mask, const *esp_ble_gap_conn_params_t* *phy_1m_conn_params, const *esp_ble_gap_conn_params_t* *phy_2m_conn_params, const *esp_ble_gap_conn_params_t* *phy_coded_conn_params)

This function is used to set aux connection parameters.

Parameters
- **addr** -- **[in]** : device address
- **phy_mask** -- **[in]** : indicates the PHY(s) on which the advertising packets should be received on the primary advertising channel and the PHYs for which connection parameters have been specified.
- **phy_1m_conn_params** -- **[in]** : Scan connectable advertisements on the LE 1M PHY. Connection parameters for the LE 1M PHY are provided.
- **phy_2m_conn_params** -- **[in]** : Connection parameters for the LE 2M PHY are provided.
- **phy_coded_conn_params** -- **[in]** : Scan connectable advertisements on the LE Coded PHY. Connection parameters for the LE Coded PHY are provided.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_recv_enable** (uint16_t sync_handle, uint8_t enable)

This function is used to set periodic advertising receive enable.

Parameters
- **sync_handle** -- **[in]** : Handle of periodic advertising sync
- **enable** -- **[in]** : Determines whether reporting and duplicate filtering are enabled or disabled

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_sync_trans**(*esp_bd_addr_t* addr, uint16_t service_data, uint16_t sync_handle)

This function is used to transfer periodic advertising sync.

Parameters
- **addr** -- **[in]** : Peer device address
- **service_data** -- **[in]** : Service data used by Host
- **sync_handle** -- **[in]** : Handle of periodic advertising sync

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_periodic_adv_set_info_trans**(*esp_bd_addr_t* addr, uint16_t service_data, uint8_t adv_handle)

This function is used to transfer periodic advertising set info.

Parameters
- **addr** -- **[in]** : Peer device address
- **service_data** -- **[in]** : Service data used by Host
- **adv_handle** -- **[in]** : Handle of advertising set

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_periodic_adv_sync_trans_params**(*esp_bd_addr_t* addr, const *esp_ble_gap_past_params_t* *params)

This function is used to set periodic advertising sync transfer params.

Parameters
- **addr** -- **[in]** : Peer device address
- **params** -- **[in]** : Params of periodic advertising sync transfer

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_dtm_tx_start**(const *esp_ble_dtm_tx_t* *tx_params)

This function is used to start a test where the DUT generates reference packets at a fixed interval.

Parameters **tx_params** -- **[in]** : DTM Transmitter parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_dtm_rx_start**(const *esp_ble_dtm_rx_t* *rx_params)

This function is used to start a test where the DUT receives test reference packets at a fixed interval.

Parameters **rx_params** -- **[in]** : DTM Receiver parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_dtm_enh_tx_start**(const *esp_ble_dtm_enh_tx_t* *tx_params)

This function is used to start a test where the DUT generates reference packets at a fixed interval.

Parameters **tx_params** -- **[in]** : DTM Transmitter parameters

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_dtm_enh_rx_start**(const *esp_ble_dtm_enh_rx_t* *rx_params)

This function is used to start a test where the DUT receives test reference packets at a fixed interval.

Parameters **rx_params** -- **[in]** : DTM Receiver parameters
Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_dtm_stop**(void)

This function is used to stop any test which is in progress.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_clear_advertising**(void)

This function is used to clear legacy advertising.

Returns - ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_vendor_command_send**(*esp_ble_vendor_cmd_params_t* *vendor_cmd_param)

This function is called to send vendor hci command.

Parameters **vendor_cmd_param** -- **[in]** vendor hci command parameters
Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gap_set_privacy_mode**(*esp_ble_addr_type_t* addr_type, *esp_bd_addr_t* addr, *esp_ble_privacy_mode_t* mode)

This function set the privacy mode of the device in resolving list.

Note: This feature is not supported on ESP32.

Parameters
- **addr_type** -- **[in]** The address type of the peer identity address (BLE_ADDR_TYPE_PUBLIC or BLE_ADDR_TYPE_RANDOM).
- **addr** -- **[in]** The peer identity address of the device.
- **mode** -- **[in]** The privacy mode of the device.

Returns
- ESP_OK : success
- other : failed

Unions

union **esp_ble_key_value_t**

#include <esp_gap_ble_api.h> union type of the security key value

Public Members

esp_ble_penc_keys_t **penc_key**

received peer encryption key

esp_ble_pcsrk_keys_t **pcsrk_key**

received peer device SRK

esp_ble_pid_keys_t **pid_key**

peer device ID key

esp_ble_lenc_keys_t **lenc_key**

local encryption reproduction keys LTK = = d1(ER,DIV,0)

esp_ble_lcsrk_keys **lcsrk_key**

local device CSRK = d1(ER,DIV,1)

union **esp_ble_sec_t**

#include <esp_gap_ble_api.h> union associated with ble security

Public Members

esp_ble_sec_key_notif_t **key_notif**

passkey notification

esp_ble_sec_req_t **ble_req**

BLE SMP related request

esp_ble_key_t **ble_key**

BLE SMP keys used when pairing

esp_ble_local_id_keys_t **ble_id_keys**

BLE IR event

esp_ble_local_oob_data_t **oob_data**

BLE SMP secure connection OOB data

esp_ble_auth_cmpl_t **auth_cmpl**

Authentication complete indication.

union **esp_ble_gap_cb_param_t**

#include <esp_gap_ble_api.h> Gap callback parameters union.

Public Members

struct *esp_ble_gap_cb_param_t*::*ble_get_dev_name_cmpl_evt_param* **get_dev_name_cmpl**

Event parameter of ESP_GAP_BLE_GET_DEV_NAME_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_data_cmpl_evt_param* **adv_data_cmpl**

Event parameter of ESP_GAP_BLE_ADV_DATA_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_rsp_data_cmpl_evt_param* **scan_rsp_data_cmpl**

Event parameter of ESP_GAP_BLE_SCAN_RSP_DATA_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_param_cmpl_evt_param* **scan_param_cmpl**

Event parameter of ESP_GAP_BLE_SCAN_PARAM_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_result_evt_param* **scan_rst**

Event parameter of ESP_GAP_BLE_SCAN_RESULT_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_data_raw_cmpl_evt_param* **adv_data_raw_cmpl**

Event parameter of ESP_GAP_BLE_ADV_DATA_RAW_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_rsp_data_raw_cmpl_evt_param* **scan_rsp_data_raw_cmpl**

Event parameter of ESP_GAP_BLE_SCAN_RSP_DATA_RAW_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_start_cmpl_evt_param* **adv_start_cmpl**

Event parameter of ESP_GAP_BLE_ADV_START_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_start_cmpl_evt_param* **scan_start_cmpl**

Event parameter of ESP_GAP_BLE_SCAN_START_COMPLETE_EVT

esp_ble_sec_t **ble_security**

ble gap security union type

struct *esp_ble_gap_cb_param_t*::*ble_scan_stop_cmpl_evt_param* **scan_stop_cmpl**

Event parameter of ESP_GAP_BLE_SCAN_STOP_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_stop_cmpl_evt_param* **adv_stop_cmpl**

Event parameter of ESP_GAP_BLE_ADV_STOP_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_clear_cmpl_evt_param* **adv_clear_cmpl**

Event parameter of ESP_GAP_BLE_ADV_CLEAR_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_rand_cmpl_evt_param* **set_rand_addr_cmpl**

Event parameter of ESP_GAP_BLE_SET_STATIC_RAND_ADDR_EVT

struct *esp_ble_gap_cb_param_t*::*ble_update_conn_params_evt_param* **update_conn_params**

Event parameter of ESP_GAP_BLE_UPDATE_CONN_PARAMS_EVT

struct *esp_ble_gap_cb_param_t*::*ble_pkt_data_length_cmpl_evt_param* **pkt_data_length_cmpl**

Event parameter of ESP_GAP_BLE_SET_PKT_LENGTH_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_local_privacy_cmpl_evt_param* **local_privacy_cmpl**

Event parameter of ESP_GAP_BLE_SET_LOCAL_PRIVACY_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_rpa_timeout_cmpl_evt_param* **set_rpa_timeout_cmpl**

Event parameter of ESP_GAP_BLE_SET_RPA_TIMEOUT_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_add_dev_to_resolving_list_cmpl_evt_param* **add_dev_to_resolving_list_cmpl**

Event parameter of ESP_GAP_BLE_ADD_DEV_TO_RESOLVING_LIST_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_remove_bond_dev_cmpl_evt_param* **remove_bond_dev_cmpl**

Event parameter of ESP_GAP_BLE_REMOVE_BOND_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_clear_bond_dev_cmpl_evt_param* **clear_bond_dev_cmpl**

Event parameter of ESP_GAP_BLE_CLEAR_BOND_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_get_bond_dev_cmpl_evt_param* **get_bond_dev_cmpl**

Event parameter of ESP_GAP_BLE_GET_BOND_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_read_rssi_cmpl_evt_param* **read_rssi_cmpl**

Event parameter of ESP_GAP_BLE_READ_RSSI_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_update_whitelist_cmpl_evt_param* **update_whitelist_cmpl**

Event parameter of ESP_GAP_BLE_UPDATE_WHITELIST_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_update_duplicate_exceptional_list_cmpl_evt_param* **update_duplicate_exceptional_list_cmpl**

Event parameter of ESP_GAP_BLE_UPDATE_DUPLICATE_EXCEPTIONAL_LIST_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_channels_evt_param* **ble_set_channels**

Event parameter of ESP_GAP_BLE_SET_CHANNELS_EVT

struct *esp_ble_gap_cb_param_t*::*ble_read_phy_cmpl_evt_param* **read_phy**

Event parameter of ESP_GAP_BLE_READ_PHY_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_perf_def_phy_cmpl_evt_param* **set_perf_def_phy**

Event parameter of ESP_GAP_BLE_SET_PREFERRED_DEFAULT_PHY_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_perf_phy_cmpl_evt_param* **set_perf_phy**

Event parameter of ESP_GAP_BLE_SET_PREFERRED_PHY_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_set_rand_addr_cmpl_evt_param* **ext_adv_set_rand_addr**

Event parameter of ESP_GAP_BLE_EXT_ADV_SET_RAND_ADDR_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_set_params_cmpl_evt_param* **ext_adv_set_params**

Event parameter of ESP_GAP_BLE_EXT_ADV_SET_PARAMS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_data_set_cmpl_evt_param* **ext_adv_data_set**

Event parameter of ESP_GAP_BLE_EXT_ADV_DATA_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_scan_rsp_set_cmpl_evt_param* **scan_rsp_set**

Event parameter of ESP_GAP_BLE_EXT_SCAN_RSP_DATA_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_start_cmpl_evt_param* **ext_adv_start**

Event parameter of ESP_GAP_BLE_EXT_ADV_START_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_stop_cmpl_evt_param* **ext_adv_stop**

Event parameter of ESP_GAP_BLE_EXT_ADV_STOP_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_set_remove_cmpl_evt_param* **ext_adv_remove**

Event parameter of ESP_GAP_BLE_EXT_ADV_SET_REMOVE_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_set_clear_cmpl_evt_param* **ext_adv_clear**

Event parameter of ESP_GAP_BLE_EXT_ADV_SET_CLEAR_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_set_params_cmpl_param* **peroid_adv_set_params**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SET_PARAMS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_data_set_cmpl_param* **period_adv_data_set**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_DATA_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_start_cmpl_param* **period_adv_start**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_START_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_stop_cmpl_param* **period_adv_stop**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_STOP_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_create_sync_cmpl_param* **period_adv_create_sync**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_CREATE_SYNC_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_sync_cancel_cmpl_param* **period_adv_sync_cancel**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_CANCEL_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_sync_terminate_cmpl_param* **period_adv_sync_term**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_TERMINATE_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_add_dev_cmpl_param* **period_adv_add_dev**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_ADD_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_remove_dev_cmpl_param* **period_adv_remove_dev**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_REMOVE_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_period_adv_clear_dev_cmpl_param* **period_adv_clear_dev**
Event parameter of ESP_GAP_BLE_PERIODIC_ADV_CLEAR_DEV_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_ext_scan_params_cmpl_param* **set_ext_scan_params**
Event parameter of ESP_GAP_BLE_SET_EXT_SCAN_PARAMS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_scan_start_cmpl_param* **ext_scan_start**
Event parameter of ESP_GAP_BLE_EXT_SCAN_START_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_scan_stop_cmpl_param* **ext_scan_stop**
Event parameter of ESP_GAP_BLE_EXT_SCAN_STOP_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_conn_params_set_cmpl_param* **ext_conn_params_set**
Event parameter of ESP_GAP_BLE_PREFER_EXT_CONN_PARAMS_SET_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_adv_terminate_param* **adv_terminate**
Event parameter of ESP_GAP_BLE_ADV_TERMINATED_EVT

struct *esp_ble_gap_cb_param_t*::*ble_scan_req_received_param* **scan_req_received**
Event parameter of ESP_GAP_BLE_SCAN_REQ_RECEIVED_EVT

struct *esp_ble_gap_cb_param_t*::*ble_channel_sel_alg_param* **channel_sel_alg**

Event parameter of ESP_GAP_BLE_CHANNEL_SELECT_ALGORITHM_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_sync_lost_param* **periodic_adv_sync_lost**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_LOST_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_sync_estab_param* **periodic_adv_sync_estab**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_ESTAB_EVT

struct *esp_ble_gap_cb_param_t*::*ble_phy_update_cmpl_param* **phy_update**

Event parameter of ESP_GAP_BLE_PHY_UPDATE_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_ext_adv_report_param* **ext_adv_report**

Event parameter of ESP_GAP_BLE_EXT_ADV_REPORT_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_report_param* **period_adv_report**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_REPORT_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_recv_enable_cmpl_param* **period_adv_recv_enable**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_RECV_ENABLE_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_sync_trans_cmpl_param* **period_adv_sync_trans**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_set_info_trans_cmpl_param* **period_adv_set_info_trans**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SET_INFO_TRANS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_past_params_cmpl_param* **set_past_params**

Event parameter of ESP_GAP_BLE_SET_PAST_PARAMS_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_periodic_adv_sync_trans_recv_param* **past_received**

Event parameter of ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_RECV_EVT

struct *esp_ble_gap_cb_param_t*::*ble_dtm_state_update_evt_param* **dtm_state_update**

Event parameter of ESP_GAP_BLE_DTM_TEST_UPDATE_EVT

struct *esp_ble_gap_cb_param_t*::*vendor_cmd_cmpl_evt_param* **vendor_cmd_cmpl**

Event parameter of ESP_GAP_BLE_VENDOR_CMD_COMPLETE_EVT

struct *esp_ble_gap_cb_param_t*::*ble_set_privacy_mode_cmpl_evt_param* **set_privacy_mode_cmpl**

Event parameter of ESP_GAP_BLE_SET_PRIVACY_MODE_COMPLETE_EVT

struct **ble_add_dev_to_resolving_list_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADD_DEV_TO_RESOLVING_LIST_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicates the success status of adding a device to the resolving list

struct **ble_adv_clear_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_CLEAR_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate adv clear operation success status

struct **ble_adv_data_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_DATA_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set advertising data operation success status

struct **ble_adv_data_raw_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_DATA_RAW_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set raw advertising data operation success status

struct **ble_adv_start_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_START_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate advertising start operation success status

struct **ble_adv_stop_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_STOP_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate adv stop operation success status

struct **ble_adv_terminate_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_ADV_TERMINATED_EVT.

Public Members

uint8_t **status**

Indicate adv terminate status

uint8_t **adv_instance**

extend advertising handle

uint16_t **conn_idx**

connection index

uint8_t **completed_event**

the number of completed extend advertising events

struct **ble_channel_sel_alg_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_CHANNEL_SELECT_ALGORITHM_EVT.

Public Members

uint16_t **conn_handle**

connection handle

uint8_t **channel_sel_alg**

channel selection algorithm

struct **ble_clear_bond_dev_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_CLEAR_BOND_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the clear bond device operation success status

struct **ble_dtm_state_update_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_DTM_TEST_UPDATE_EVT.

Public Members

esp_bt_status_t **status**

Indicate DTM operation success status

esp_ble_dtm_update_evt_t **update_evt**

DTM state change event, 0x00: DTM TX start, 0x01: DTM RX start, 0x02:DTM end

uint16_t **num_of_pkt**

number of packets received, only valid if update_evt is DTM_TEST_STOP_EVT and shall be reported as 0 for a transmitter

struct **ble_ext_adv_data_set_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_DATA_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising data set status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_report_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_REPORT_EVT.

Public Members

esp_ble_gap_ext_adv_report_t **params**

extend advertising report parameters

struct **ble_ext_adv_scan_rsp_set_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_SCAN_RSP_DATA_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising scan response data set status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_set_clear_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_SET_CLEAR_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate advertising stop operation success status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_set_params_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_SET_PARAMS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising parameters set status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_set_rand_addr_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_SET_RAND_ADDR_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising random address set status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_set_remove_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_SET_REMOVE_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate advertising stop operation success status

uint8_t **instance**

extend advertising handle

struct **ble_ext_adv_start_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_START_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate advertising start operation success status

uint8_t **instance_num**

extend advertising handle numble

uint8_t **instance**[EXT_ADV_NUM_SETS_MAX]

extend advertising handle list

struct **ble_ext_adv_stop_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_ADV_STOP_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate advertising stop operation success status

uint8_t **instance_num**

extend advertising handle numble

uint8_t **instance**[EXT_ADV_NUM_SETS_MAX]

extend advertising handle list

struct **ble_ext_conn_params_set_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PREFER_EXT_CONN_PARAMS_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend connection parameters set status

struct **ble_ext_scan_start_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_SCAN_START_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising start status

struct **ble_ext_scan_stop_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_EXT_SCAN_STOP_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising stop status

struct **ble_get_bond_dev_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_GET_BOND_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the get bond device operation success status

uint8_t **dev_num**

Indicate the get number device in the bond list

esp_ble_bond_dev_t ***bond_dev**

the pointer to the bond device Structure

struct **ble_get_dev_name_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_GET_DEV_NAME_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the get device name success status

char ***name**

Name of bluetooth device

struct **ble_local_privacy_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_LOCAL_PRIVACY_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set local privacy operation success status

struct **ble_period_adv_add_dev_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_ADD_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising device list add status

struct **ble_period_adv_clear_dev_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_CLEAR_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising device list clean status

struct **ble_period_adv_create_sync_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_CREATE_SYNC_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising create sync status

struct **ble_period_adv_remove_dev_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_REMOVE_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising device list remove status

struct **ble_period_adv_sync_cancel_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_CANCEL_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising sync cancel status

struct **ble_period_adv_sync_terminate_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_TERMINATE_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising sync terminate status

struct **ble_periodic_adv_data_set_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_DATA_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising data set status

uint8_t **instance**

extend advertising handle

struct **ble_periodic_adv_recv_enable_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_RECV_ENABLE_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Set periodic advertising receive enable status

struct **ble_periodic_adv_report_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_REPORT_EVT.

Public Members

esp_ble_gap_periodic_adv_report_t **params**

periodic advertising report parameters

struct **ble_periodic_adv_set_info_trans_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SET_INFO_TRANS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Periodic advertising set info transfer status

esp_bd_addr_t **bda**

The remote device address

struct **ble_periodic_adv_set_params_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SET_PARAMS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertisingparameters set status

uint8_t **instance**

extend advertising handle

struct **ble_periodic_adv_start_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_START_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising start status

uint8_t **instance**

extend advertising handle

struct **ble_periodic_adv_stop_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_STOP_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate periodic advertising stop status

uint8_t **instance**

extend advertising handle

struct **ble_periodic_adv_sync_estab_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_ESTAB_EVT.

Public Members

uint8_t **status**

periodic advertising sync status

uint16_t **sync_handle**

periodic advertising sync handle

uint8_t **sid**

periodic advertising sid

esp_ble_addr_type_t **adv_addr_type**

periodic advertising address type

esp_bd_addr_t **adv_addr**

periodic advertising address

esp_ble_gap_phy_t **adv_phy**

periodic advertising phy type

uint16_t **period_adv_interval**

periodic advertising interval

uint8_t **adv_clk_accuracy**

periodic advertising clock accuracy

struct **ble_periodic_adv_sync_lost_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_LOST_EVT.

Public Members

uint16_t **sync_handle**

sync handle

struct **ble_periodic_adv_sync_trans_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Periodic advertising sync transfer status

esp_bd_addr_t **bda**

The remote device address

struct **ble_periodic_adv_sync_trans_recv_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_RECV_EVT.

Public Members

esp_bt_status_t **status**

Periodic advertising sync transfer received status

esp_bd_addr_t **bda**

The remote device address

uint16_t **service_data**

The value provided by the peer device

uint16_t **sync_handle**

Periodic advertising sync handle

uint8_t **adv_sid**

Periodic advertising set id

uint8_t **adv_addr_type**

Periodic advertiser address type

esp_bd_addr_t **adv_addr**

Periodic advertiser address

esp_ble_gap_phy_t **adv_phy**

Periodic advertising PHY

uint16_t **adv_interval**

Periodic advertising interval

uint8_t **adv_clk_accuracy**

Periodic advertising clock accuracy

struct **ble_phy_update_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_PHY_UPDATE_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

phy update status

esp_bd_addr_t **bda**

address

esp_ble_gap_phy_t **tx_phy**

tx phy type

esp_ble_gap_phy_t **rx_phy**

rx phy type

struct **ble_pkt_data_length_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_PKT_LENGTH_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set pkt data length operation success status

esp_ble_pkt_data_length_params_t **params**

pkt data length value

struct **ble_read_phy_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_READ_PHY_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

read phy complete status

esp_bd_addr_t **bda**

read phy address

esp_ble_gap_phy_t **tx_phy**

tx phy type

esp_ble_gap_phy_t **rx_phy**

rx phy type

struct **ble_read_rssi_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_READ_RSSI_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the read adv tx power operation success status

int8_t **rssi**

The ble remote device rssi value, the range is from -127 to 20, the unit is dbm, if the RSSI cannot be read, the RSSI metric shall be set to 127.

esp_bd_addr_t **remote_addr**

The remote device address

struct **ble_remove_bond_dev_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_REMOVE_BOND_DEV_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the remove bond device operation success status

esp_bd_addr_t **bd_addr**

The device address which has been remove from the bond list

struct **ble_rpa_timeout_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_RPA_TIMEOUT_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set RPA timeout operation success status

struct **ble_scan_param_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_PARAM_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set scan param operation success status

struct **ble_scan_req_received_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_REQ_RECEIVED_EVT.

Public Members

uint8_t **adv_instance**

extend advertising handle

esp_ble_addr_type_t **scan_addr_type**

scanner address type

esp_bd_addr_t **scan_addr**

scanner address

struct **ble_scan_result_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_RESULT_EVT.

Public Members

esp_gap_search_evt_t **search_evt**

Search event type

esp_bd_addr_t **bda**

Bluetooth device address which has been searched

esp_bt_dev_type_t **dev_type**

Device type

esp_ble_addr_type_t **ble_addr_type**

Ble device address type

esp_ble_evt_type_t **ble_evt_type**

Ble scan result event type

int **rssi**

Searched device's RSSI

uint8_t **ble_adv**[ESP_BLE_ADV_DATA_LEN_MAX + ESP_BLE_SCAN_RSP_DATA_LEN_MAX]

Received EIR

int **flag**

Advertising data flag bit

int **num_resps**

Scan result number

uint8_t **adv_data_len**

Adv data length

uint8_t **scan_rsp_len**

Scan response length

uint32_t **num_dis**

The number of discard packets

struct **ble_scan_rsp_data_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_RSP_DATA_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set scan response data operation success status

struct **ble_scan_rsp_data_raw_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_RSP_DATA_RAW_SET_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the set raw advertising data operation success status

struct **ble_scan_start_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_START_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate scan start operation success status

struct **ble_scan_stop_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SCAN_STOP_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate scan stop operation success status

struct **ble_set_channels_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_CHANNELS_EVT.

Public Members

esp_bt_status_t **stat**

BLE set channel status

struct **ble_set_ext_scan_params_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_EXT_SCAN_PARAMS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate extend advertising parameters set status

struct **ble_set_past_params_cmpl_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_PAST_PARAMS_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Set periodic advertising sync transfer params status

esp_bd_addr_t **bda**

The remote device address

struct **ble_set_perf_def_phy_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_PREFERRED_DEFAULT_PHY_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate perf default phy set status

struct **ble_set_perf_phy_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_PREFERRED_PHY_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate perf phy set status

struct **ble_set_privacy_mode_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_PRIVACY_MODE_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate privacy mode set operation success status

struct **ble_set_rand_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_SET_STATIC_RAND_ADDR_EVT.

Public Members

esp_bt_status_t **status**

Indicate set static rand address operation success status

struct **ble_update_conn_params_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_UPDATE_CONN_PARAMS_EVT.

Public Members

esp_bt_status_t **status**

Indicate update connection parameters success status

esp_bd_addr_t **bda**

Bluetooth device address

uint16_t **min_int**

Min connection interval

uint16_t **max_int**

Max connection interval

uint16_t **latency**

Slave latency for the connection in number of connection events. Range: 0x0000 to 0x01F3

uint16_t **conn_int**

Current connection interval

uint16_t **timeout**

Supervision timeout for the LE Link. Range: 0x000A to 0x0C80. Mandatory Range: 0x000A to 0x0C80 Time = N * 10 msec

struct **ble_update_duplicate_exceptional_list_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_UPDATE_DUPLICATE_EXCEPTIONAL_LIST_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate update duplicate scan exceptional list operation success status

uint8_t **subcode**

Define in esp_bt_duplicate_exceptional_subcode_type_t

uint16_t **length**

The length of device_info

esp_duplicate_info_t **device_info**

device information, when subcode is ESP_BLE_DUPLICATE_EXCEPTIONAL_LIST_CLEAN, the value is invalid

struct **ble_update_whitelist_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_UPDATE_WHITELIST_COMPLETE_EVT.

Public Members

esp_bt_status_t **status**

Indicate the add or remove whitelist operation success status

esp_ble_wl_operation_t **wl_operation**

The value is ESP_BLE_WHITELIST_ADD if add address to whitelist operation success, ESP_BLE_WHITELIST_REMOVE if remove address from the whitelist operation success

struct **vendor_cmd_cmpl_evt_param**

#include <esp_gap_ble_api.h> ESP_GAP_BLE_VENDOR_CMD_COMPLETE_EVT.

Public Members

uint16_t **opcode**

vendor hci command opcode

uint16_t **param_len**

The length of parameter buffer

uint8_t ***p_param_buf**

The point of parameter buffer

Structures

struct **esp_ble_vendor_cmd_params_t**

Vendor HCI command parameters.

Public Members

uint16_t **opcode**

vendor hci command opcode

uint8_t **param_len**

the length of parameter

uint8_t ***p_param_buf**

the point of parameter buffer

struct **esp_ble_dtm_tx_t**

DTM TX parameters.

Public Members

uint8_t **tx_channel**

channel for sending test data, tx_channel = (Frequency -2402)/2, tx_channel range:0x00-0x27, Frequency range: 2402 MHz to 2480 MHz

uint8_t **len_of_data**

length in bytes of payload data in each packet

esp_ble_dtm_pkt_payload_t **pkt_payload**

packet payload type. value range: 0x00-0x07

struct **esp_ble_dtm_rx_t**

DTM RX parameters.

Public Members

uint8_t **rx_channel**

channel for test data reception, rx_channel = (Frequency -2402)/2, tx_channel range:0x00-0x27, Frequency range: 2402 MHz to 2480 MHz

struct **esp_ble_adv_params_t**

Advertising parameters.

Public Members

uint16_t **adv_int_min**

Minimum advertising interval for undirected and low duty cycle directed advertising. Range: 0x0020 to 0x4000 Default: N = 0x0800 (1.28 second) Time = N * 0.625 msec Time Range: 20 ms to 10.24 sec

uint16_t **adv_int_max**

Maximum advertising interval for undirected and low duty cycle directed advertising. Range: 0x0020 to 0x4000 Default: N = 0x0800 (1.28 second) Time = N * 0.625 msec Time Range: 20 ms to 10.24 sec Advertising max interval

esp_ble_adv_type_t **adv_type**

Advertising type

esp_ble_addr_type_t **own_addr_type**

Owner bluetooth device address type

esp_bd_addr_t **peer_addr**

Peer device bluetooth device address

esp_ble_addr_type_t **peer_addr_type**

Peer device bluetooth device address type, only support public address type and random address type

esp_ble_adv_channel_t **channel_map**

Advertising channel map

esp_ble_adv_filter_t **adv_filter_policy**

Advertising filter policy

struct **esp_ble_adv_data_t**

Advertising data content, according to "Supplement to the Bluetooth Core Specification".

Public Members

bool **set_scan_rsp**

Set this advertising data as scan response or not

bool **include_name**

Advertising data include device name or not

bool **include_txpower**

Advertising data include TX power

int **min_interval**

Advertising data show slave preferred connection min interval. The connection interval in the following manner: connIntervalmin = Conn_Interval_Min * 1.25 ms Conn_Interval_Min range: 0x0006 to 0x0C80 Value of 0xFFFF indicates no specific minimum. Values not defined above are reserved for future use.

int **max_interval**

Advertising data show slave preferred connection max interval. The connection interval in the following manner: connIntervalmax = Conn_Interval_Max * 1.25 ms Conn_Interval_Max range: 0x0006 to 0x0C80 Conn_Interval_Max shall be equal to or greater than the Conn_Interval_Min. Value of 0xFFFF indicates no specific maximum. Values not defined above are reserved for future use.

int **appearance**

External appearance of device

uint16_t **manufacturer_len**

Manufacturer data length

uint8_t ***p_manufacturer_data**

Manufacturer data point

uint16_t **service_data_len**

Service data length

uint8_t ***p_service_data**

Service data point

uint16_t **service_uuid_len**

Service uuid length

uint8_t *`p_service_uuid`

Service uuid array point

uint8_t `flag`

Advertising flag of discovery mode, see BLE_ADV_DATA_FLAG detail

struct `esp_ble_scan_params_t`

Ble scan parameters.

Public Members

esp_ble_scan_type_t `scan_type`

Scan type

esp_ble_addr_type_t `own_addr_type`

Owner address type

esp_ble_scan_filter_t `scan_filter_policy`

Scan filter policy

uint16_t `scan_interval`

Scan interval. This is defined as the time interval from when the Controller started its last LE scan until it begins the subsequent LE scan. Range: 0x0004 to 0x4000 Default: 0x0010 (10 ms) Time = N * 0.625 msec Time Range: 2.5 msec to 10.24 seconds

uint16_t `scan_window`

Scan window. The duration of the LE scan. LE_Scan_Window shall be less than or equal to LE_Scan_Interval Range: 0x0004 to 0x4000 Default: 0x0010 (10 ms) Time = N * 0.625 msec Time Range: 2.5 msec to 10240 msec

esp_ble_scan_duplicate_t `scan_duplicate`

The Scan_Duplicates parameter controls whether the Link Layer should filter out duplicate advertising reports (BLE_SCAN_DUPLICATE_ENABLE) to the Host, or if the Link Layer should generate advertising reports for each packet received

struct `esp_gap_conn_params_t`

connection parameters information

Public Members

uint16_t `interval`

connection interval

uint16_t `latency`

Slave latency for the connection in number of connection events. Range: 0x0000 to 0x01F3

uint16_t **timeout**

Supervision timeout for the LE Link. Range: 0x000A to 0x0C80. Mandatory Range: 0x000A to 0x0C80 Time = N * 10 msec Time Range: 100 msec to 32 seconds

struct **esp_ble_conn_update_params_t**

Connection update parameters.

Public Members

esp_bd_addr_t **bda**

Bluetooth device address

uint16_t **min_int**

Min connection interval

uint16_t **max_int**

Max connection interval

uint16_t **latency**

Slave latency for the connection in number of connection events. Range: 0x0000 to 0x01F3

uint16_t **timeout**

Supervision timeout for the LE Link. Range: 0x000A to 0x0C80. Mandatory Range: 0x000A to 0x0C80 Time = N * 10 msec Time Range: 100 msec to 32 seconds

struct **esp_ble_pkt_data_length_params_t**

BLE pkt date length keys.

Public Members

uint16_t **rx_len**

pkt rx data length value

uint16_t **tx_len**

pkt tx data length value

struct **esp_ble_penc_keys_t**

BLE encryption keys.

Public Members

esp_bt_octet16_t **ltk**

The long term key

esp_bt_octet8_t **rand**

The random number

uint16_t **ediv**

The ediv value

uint8_t **sec_level**

The security level of the security link

uint8_t **key_size**

The key size(7~16) of the security link

struct **esp_ble_pcsrk_keys_t**

BLE CSRK keys.

Public Members

uint32_t **counter**

The counter

esp_bt_octet16_t **csrk**

The csrk key

uint8_t **sec_level**

The security level

struct **esp_ble_pid_keys_t**

BLE pid keys.

Public Members

esp_bt_octet16_t **irk**

The irk value

esp_ble_addr_type_t **addr_type**

The address type

esp_bd_addr_t **static_addr**

The static address

struct **esp_ble_lenc_keys_t**

BLE Encryption reproduction keys.

Public Members

esp_bt_octet16_t **ltk**

The long term key

uint16_t **div**

The div value

uint8_t **key_size**

The key size of the security link

uint8_t **sec_level**

The security level of the security link

struct **esp_ble_lcsrk_keys**

BLE SRK keys.

Public Members

uint32_t **counter**

The counter value

uint16_t **div**

The div value

uint8_t **sec_level**

The security level of the security link

esp_bt_octet16_t **csrk**

The csrk key value

struct **esp_ble_sec_key_notif_t**

Structure associated with ESP_KEY_NOTIF_EVT.

Public Members

esp_bd_addr_t **bd_addr**

peer address

uint32_t **passkey**

the numeric value for comparison. If just_works, do not show this number to UI

struct **esp_ble_sec_req_t**

Structure of the security request.

Public Members

esp_bd_addr_t **bd_addr**

peer address

struct **esp_ble_bond_key_info_t**

struct type of the bond key information value

Public Members

esp_ble_key_mask_t **key_mask**
the key mask to indicate witch key is present

esp_ble_penc_keys_t **penc_key**
received peer encryption key

esp_ble_pcsrk_keys_t **pcsrk_key**
received peer device SRK

esp_ble_pid_keys_t **pid_key**
peer device ID key

struct **esp_ble_bond_dev_t**
struct type of the bond device value

Public Members

esp_bd_addr_t **bd_addr**
peer address

esp_ble_bond_key_info_t **bond_key**
the bond key information

struct **esp_ble_key_t**
union type of the security key value

Public Members

esp_bd_addr_t **bd_addr**
peer address

esp_ble_key_type_t **key_type**
key type of the security link

esp_ble_key_value_t **p_key_value**
the pointer to the key value

struct **esp_ble_local_id_keys_t**
structure type of the ble local id keys value

Public Members

esp_bt_octet16_t **ir**
the 16 bits of the ir value

esp_bt_octet16_t **irk**

the 16 bits of the ir key value

esp_bt_octet16_t **dhk**

the 16 bits of the dh key value

struct **esp_ble_local_oob_data_t**

structure type of the ble local oob data value

Public Members

esp_bt_octet16_t **oob_c**

the 128 bits of confirmation value

esp_bt_octet16_t **oob_r**

the 128 bits of randomizer value

struct **esp_ble_auth_cmpl_t**

Structure associated with ESP_AUTH_CMPL_EVT.

Public Members

esp_bd_addr_t **bd_addr**

BD address of peer device

bool **key_present**

True if the link key value is valid; false otherwise

esp_link_key **key**

Link key associated with peer device

uint8_t **key_type**

The type of link key

bool **success**

True if authentication succeeded; false otherwise

esp_ble_auth_fail_rsn_t **fail_reason**

The HCI reason/error code for failure when success is false

esp_ble_addr_type_t **addr_type**

Peer device address type

esp_bt_dev_type_t **dev_type**

Device type

esp_ble_auth_req_t **auth_mode**

Authentication mode

struct **esp_ble_gap_ext_adv_params_t**

ext adv parameters

Public Members

esp_ble_ext_adv_type_mask_t **type**

ext adv type

uint32_t **interval_min**

ext adv minimum interval

uint32_t **interval_max**

ext adv maximum interval

esp_ble_adv_channel_t **channel_map**

ext adv channel map

esp_ble_addr_type_t **own_addr_type**

ext adv own address type

esp_ble_addr_type_t **peer_addr_type**

ext adv peer address type

esp_bd_addr_t **peer_addr**

ext adv peer address

esp_ble_adv_filter_t **filter_policy**

ext adv filter policy

int8_t **tx_power**

ext adv tx power

esp_ble_gap_pri_phy_t **primary_phy**

ext adv primary phy

uint8_t **max_skip**

ext adv maximum skip

esp_ble_gap_phy_t **secondary_phy**

ext adv secondary phy

uint8_t **sid**

ext adv sid

bool **scan_req_notif**

ext adv scan request event notify

struct **esp_ble_ext_scan_cfg_t**

ext scan config

Public Members

esp_ble_scan_type_t **scan_type**

ext scan type

uint16_t **scan_interval**

ext scan interval

uint16_t **scan_window**

ext scan window

struct **esp_ble_ext_scan_params_t**

ext scan parameters

Public Members

esp_ble_addr_type_t **own_addr_type**

ext scan own address type

esp_ble_scan_filter_t **filter_policy**

ext scan filter policy

esp_ble_scan_duplicate_t **scan_duplicate**

ext scan duplicate scan

esp_ble_ext_scan_cfg_mask_t **cfg_mask**

ext scan config mask

esp_ble_ext_scan_cfg_t **uncoded_cfg**

ext scan uncoded config parameters

esp_ble_ext_scan_cfg_t **coded_cfg**

ext scan coded config parameters

struct **esp_ble_gap_conn_params_t**

create extend connection parameters

Public Members

uint16_t **scan_interval**
: init scan interval

uint16_t **scan_window**
: init scan window

uint16_t **interval_min**
: minimum interval

uint16_t **interval_max**
: maximum interval

uint16_t **latency**
: ext scan type

uint16_t **supervision_timeout**
: connection supervision timeout

uint16_t **min_ce_len**
: minimum ce length

uint16_t **max_ce_len**
: maximum ce length

struct **esp_ble_gap_ext_adv_t**
: extend adv enable parameters

Public Members

uint8_t **instance**
: advertising handle

int **duration**
: advertising duration

int **max_events**
: maximum number of extended advertising events

struct **esp_ble_gap_periodic_adv_params_t**
: periodic adv parameters

Public Members

uint16_t **interval_min**
: periodic advertising minimum interval

uint16_t **interval_max**

periodic advertising maximum interval

uint8_t **properties**

periodic advertising properties

struct **esp_ble_gap_periodic_adv_sync_params_t**

periodic adv sync parameters

Public Members

esp_ble_gap_sync_t **filter_policy**

Configures the filter policy for periodic advertising sync: 0: Use Advertising SID, Advertiser Address Type, and Advertiser Address parameters to determine the advertiser to listen to. 1: Use the Periodic Advertiser List to determine the advertiser to listen to.

uint8_t **sid**

SID of the periodic advertising

esp_ble_addr_type_t **addr_type**

Address type of the periodic advertising

esp_bd_addr_t **addr**

Address of the periodic advertising

uint16_t **skip**

Maximum number of periodic advertising events that can be skipped

uint16_t **sync_timeout**

Synchronization timeout

struct **esp_ble_gap_ext_adv_report_t**

extend adv report parameters

Public Members

esp_ble_gap_adv_type_t **event_type**

extend advertising type

uint8_t **addr_type**

extend advertising address type

esp_bd_addr_t **addr**

extend advertising address

esp_ble_gap_pri_phy_t **primary_phy**

extend advertising primary phy

esp_ble_gap_phy_t **secondly_phy**
extend advertising secondary phy

uint8_t **sid**
extend advertising sid

uint8_t **tx_power**
extend advertising tx power

int8_t **rssi**
extend advertising rssi

uint16_t **per_adv_interval**
periodic advertising interval

uint8_t **dir_addr_type**
direct address type

esp_bd_addr_t **dir_addr**
direct address

esp_ble_gap_ext_adv_data_status_t **data_status**
data type

uint8_t **adv_data_len**
extend advertising data length

uint8_t **adv_data**[251]
extend advertising data

struct **esp_ble_gap_periodic_adv_report_t**
periodic adv report parameters

Public Members

uint16_t **sync_handle**
periodic advertising train handle

uint8_t **tx_power**
periodic advertising tx power

int8_t **rssi**
periodic advertising rssi

esp_ble_gap_ext_adv_data_status_t **data_status**
periodic advertising data type

uint8_t **data_length**

periodic advertising data length

uint8_t **data**[251]

periodic advertising data

struct **esp_ble_gap_periodic_adv_sync_estab_t**

perodic adv sync establish parameters

Public Members

uint8_t **status**

periodic advertising sync status

uint16_t **sync_handle**

periodic advertising train handle

uint8_t **sid**

periodic advertising sid

esp_ble_addr_type_t **addr_type**

periodic advertising address type

esp_bd_addr_t **adv_addr**

periodic advertising address

esp_ble_gap_phy_t **adv_phy**

periodic advertising adv phy type

uint16_t **period_adv_interval**

periodic advertising interval

uint8_t **adv_clk_accuracy**

periodic advertising clock accuracy

struct **esp_ble_dtm_enh_tx_t**

DTM TX parameters.

Public Members

uint8_t **tx_channel**

channel for sending test data, tx_channel = (Frequency -2402)/2, tx_channel range:0x00-0x27, Frequency range: 2402 MHz to 2480 MHz

uint8_t **len_of_data**

length in bytes of payload data in each packet

esp_ble_dtm_pkt_payload_t **pkt_payload**

packet payload type. value range: 0x00-0x07

esp_ble_gap_phy_t **phy**

the phy type used by the transmitter, coded phy with S=2:0x04

struct **esp_ble_dtm_enh_rx_t**

DTM RX parameters.

Public Members

uint8_t **rx_channel**

channel for test data reception, rx_channel = (Frequency -2402)/2, tx_channel range:0x00-0x27, Frequency range: 2402 MHz to 2480 MHz

esp_ble_gap_phy_t **phy**

the phy type used by the receiver, 1M phy: 0x01, 2M phy:0x02, coded phy:0x03

uint8_t **modulation_idx**

modulation index, 0x00:standard modulation index, 0x01:stable modulation index

struct **esp_ble_gap_past_params_t**

periodic adv sync transfer parameters

Public Members

esp_ble_gap_past_mode_t **mode**

periodic advertising sync transfer mode

uint16_t **skip**

the number of periodic advertising packets that can be skipped

uint16_t **sync_timeout**

synchronization timeout for the periodic advertising train

uint8_t **cte_type**

periodic advertising sync transfer CET type

Macros

ESP_BLE_ADV_FLAG_LIMIT_DISC

BLE_ADV_DATA_FLAG data flag bit definition used for advertising data flag.

ESP_BLE_ADV_FLAG_GEN_DISC

ESP_BLE_ADV_FLAG_BREDR_NOT_SPT

ESP_BLE_ADV_FLAG_DMT_CONTROLLER_SPT

ESP_BLE_ADV_FLAG_DMT_HOST_SPT

ESP_BLE_ADV_FLAG_NON_LIMIT_DISC

ESP_LE_KEY_NONE

relate to BTM_LE_KEY_xxx in stack/btm_api.h

No encryption key

ESP_LE_KEY_PENC

encryption key, encryption information of peer device

ESP_LE_KEY_PID

identity key of the peer device

ESP_LE_KEY_PCSRK

peer SRK

ESP_LE_KEY_PLK

Link key

ESP_LE_KEY_LLK

peer link key

ESP_LE_KEY_LENC

master role security information:div

ESP_LE_KEY_LID

master device ID key

ESP_LE_KEY_LCSRK

local CSRK has been deliver to peer

ESP_LE_AUTH_NO_BOND

relate to BTM_LE_AUTH_xxx in stack/btm_api.h

0 no bondingv

ESP_LE_AUTH_BOND

1 << 0 device in the bonding with peer

ESP_LE_AUTH_REQ_MITM

1 << 2 man in the middle attack

ESP_LE_AUTH_REQ_BOND_MITM

0101 banding with man in the middle attack

ESP_LE_AUTH_REQ_SC_ONLY

1 << 3 secure connection

ESP_LE_AUTH_REQ_SC_BOND

1001 secure connection with band

ESP_LE_AUTH_REQ_SC_MITM

1100 secure conn with MITM

ESP_LE_AUTH_REQ_SC_MITM_BOND

1101 SC with MITM and Bonding

ESP_BLE_ONLY_ACCEPT_SPECIFIED_AUTH_DISABLE

authentication disable

ESP_BLE_ONLY_ACCEPT_SPECIFIED_AUTH_ENABLE

authentication enable

ESP_BLE_OOB_DISABLE

disable the out of bond

ESP_BLE_OOB_ENABLE

enable the out of bond

ESP_IO_CAP_OUT

relate to BTM_IO_CAP_xxx in stack/btm_api.h

DisplayOnly

ESP_IO_CAP_IO

DisplayYesNo

ESP_IO_CAP_IN

KeyboardOnly

ESP_IO_CAP_NONE

NoInputNoOutput

ESP_IO_CAP_KBDISP

Keyboard display

ESP_BLE_APPEARANCE_UNKNOWN

relate to BTM_BLE_APPEARANCE_UNKNOWN in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_PHONE

relate to BTM_BLE_APPEARANCE_GENERIC_PHONE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_COMPUTER

relate to BTM_BLE_APPEARANCE_GENERIC_COMPUTER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_WATCH

relate to BTM_BLE_APPEARANCE_GENERIC_WATCH in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_SPORTS_WATCH

relate to BTM_BLE_APPEARANCE_SPORTS_WATCH in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_CLOCK

relate to BTM_BLE_APPEARANCE_GENERIC_CLOCK in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_DISPLAY

relate to BTM_BLE_APPEARANCE_GENERIC_DISPLAY in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_REMOTE

relate to BTM_BLE_APPEARANCE_GENERIC_REMOTE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_EYEGLASSES

relate to BTM_BLE_APPEARANCE_GENERIC_EYEGLASSES in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_TAG

relate to BTM_BLE_APPEARANCE_GENERIC_TAG in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_KEYRING

relate to BTM_BLE_APPEARANCE_GENERIC_KEYRING in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_MEDIA_PLAYER

relate to BTM_BLE_APPEARANCE_GENERIC_MEDIA_PLAYER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_BARCODE_SCANNER

relate to BTM_BLE_APPEARANCE_GENERIC_BARCODE_SCANNER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_THERMOMETER

relate to BTM_BLE_APPEARANCE_GENERIC_THERMOMETER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_THERMOMETER_EAR

relate to BTM_BLE_APPEARANCE_THERMOMETER_EAR in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_HEART_RATE

relate to BTM_BLE_APPEARANCE_GENERIC_HEART_RATE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HEART_RATE_BELT

relate to BTM_BLE_APPEARANCE_HEART_RATE_BELT in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_BLOOD_PRESSURE

relate to BTM_BLE_APPEARANCE_GENERIC_BLOOD_PRESSURE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_BLOOD_PRESSURE_ARM

relate to BTM_BLE_APPEARANCE_BLOOD_PRESSURE_ARM in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_BLOOD_PRESSURE_WRIST

relate to BTM_BLE_APPEARANCE_BLOOD_PRESSURE_WRIST in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_HID

relate to BTM_BLE_APPEARANCE_GENERIC_HID in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_KEYBOARD

relate to BTM_BLE_APPEARANCE_HID_KEYBOARD in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_MOUSE

relate to BTM_BLE_APPEARANCE_HID_MOUSE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_JOYSTICK

relate to BTM_BLE_APPEARANCE_HID_JOYSTICK in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_GAMEPAD

relate to BTM_BLE_APPEARANCE_HID_GAMEPAD in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_DIGITIZER_TABLET

relate to BTM_BLE_APPEARANCE_HID_DIGITIZER_TABLET in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_CARD_READER

relate to BTM_BLE_APPEARANCE_HID_CARD_READER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_DIGITAL_PEN

relate to BTM_BLE_APPEARANCE_HID_DIGITAL_PEN in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_HID_BARCODE_SCANNER

relate to BTM_BLE_APPEARANCE_HID_BARCODE_SCANNER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_GLUCOSE

relate to BTM_BLE_APPEARANCE_GENERIC_GLUCOSE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_WALKING

relate to BTM_BLE_APPEARANCE_GENERIC_WALKING in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_WALKING_IN_SHOE

relate to BTM_BLE_APPEARANCE_WALKING_IN_SHOE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_WALKING_ON_SHOE

relate to BTM_BLE_APPEARANCE_WALKING_ON_SHOE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_WALKING_ON_HIP

relate to BTM_BLE_APPEARANCE_WALKING_ON_HIP in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_CYCLING

relate to BTM_BLE_APPEARANCE_GENERIC_CYCLING in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_CYCLING_COMPUTER

relate to BTM_BLE_APPEARANCE_CYCLING_COMPUTER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_CYCLING_SPEED

relate to BTM_BLE_APPEARANCE_CYCLING_SPEED in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_CYCLING_CADENCE

relate to BTM_BLE_APPEARANCE_CYCLING_CADENCE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_CYCLING_POWER

relate to BTM_BLE_APPEARANCE_CYCLING_POWER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_CYCLING_SPEED_CADENCE

relate to BTM_BLE_APPEARANCE_CYCLING_SPEED_CADENCE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_STANDALONE_SPEAKER

relate to BTM_BLE_APPEARANCE_STANDALONE_SPEAKER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_PULSE_OXIMETER

relate to BTM_BLE_APPEARANCE_GENERIC_PULSE_OXIMETER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_PULSE_OXIMETER_FINGERTIP

relate to BTM_BLE_APPEARANCE_PULSE_OXIMETER_FINGERTIP in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_PULSE_OXIMETER_WRIST

relate to BTM_BLE_APPEARANCE_PULSE_OXIMETER_WRIST in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_WEIGHT

relate to BTM_BLE_APPEARANCE_GENERIC_WEIGHT in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_PERSONAL_MOBILITY_DEVICE

relate to BTM_BLE_APPEARANCE_GENERIC_PERSONAL_MOBILITY_DEVICE in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_POWERED_WHEELCHAIR

relate to BTM_BLE_APPEARANCE_POWERED_WHEELCHAIR in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_MOBILITY_SCOOTER

relate to BTM_BLE_APPEARANCE_MOBILITY_SCOOTER in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_CONTINUOUS_GLUCOSE_MONITOR

relate to BTM_BLE_APPEARANCE_GENERIC_CONTINUOUS_GLUCOSE_MONITOR in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_INSULIN_PUMP

relate to BTM_BLE_APPEARANCE_GENERIC_INSULIN_PUMP in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_INSULIN_PUMP_DURABLE_PUMP

relate to BTM_BLE_APPEARANCE_INSULIN_PUMP_DURABLE_PUMP in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_INSULIN_PUMP_PATCH_PUMP

relate to BTM_BLE_APPEARANCE_INSULIN_PUMP_PATCH_PUMP in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_INSULIN_PEN

relate to BTM_BLE_APPEARANCE_INSULIN_PEN in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_MEDICATION_DELIVERY

relate to BTM_BLE_APPEARANCE_GENERIC_MEDICATION_DELIVERY in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_GENERIC_OUTDOOR_SPORTS

relate to BTM_BLE_APPEARANCE_GENERIC_OUTDOOR_SPORTS in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION

relate to BTM_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_AND_NAV

relate to BTM_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_AND_NAV in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_POD

relate to BTM_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_POD in stack/btm_ble_api.h

ESP_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_POD_AND_NAV

relate to BTM_BLE_APPEARANCE_OUTDOOR_SPORTS_LOCATION_POD_AND_NAV in stack/btm_ble_api.h

BLE_DTM_PKT_PAYLOAD_0x00

PRBS9 sequence '11111111100000111101...' (in transmission order) as described in [Vol 6] Part F, Section 4.1.5

BLE_DTM_PKT_PAYLOAD_0x01

Repeated '11110000' (in transmission order) sequence as described in [Vol 6] Part F, Section 4.1.5

BLE_DTM_PKT_PAYLOAD_0x02

Repeated '10101010' (in transmission order) sequence as described in [Vol 6] Part F, Section 4.1.5

BLE_DTM_PKT_PAYLOAD_0x03

PRBS15 sequence as described in [Vol 6] Part F, Section 4.1.5

BLE_DTM_PKT_PAYLOAD_0x04

Repeated '11111111' (in transmission order) sequence

BLE_DTM_PKT_PAYLOAD_0x05

Repeated '00000000' (in transmission order) sequence

BLE_DTM_PKT_PAYLOAD_0x06

Repeated '00001111' (in transmission order) sequence

BLE_DTM_PKT_PAYLOAD_0x07

Repeated '01010101' (in transmission order) sequence

BLE_DTM_PKT_PAYLOAD_MAX

0x08 ~ 0xFF, Reserved for future use

ESP_GAP_BLE_CHANNELS_LEN

channel length

ESP_GAP_BLE_ADD_WHITELIST_COMPLETE_EVT

This is the old name, just for backwards compatibility.

ESP_BLE_ADV_DATA_LEN_MAX

Advertising data maximum length.

ESP_BLE_SCAN_RSP_DATA_LEN_MAX

Scan response data maximum length.

VENDOR_HCI_CMD_MASK

BLE_BIT(n)

ESP_BLE_GAP_SET_EXT_ADV_PROP_NONCONN_NONSCANNABLE_UNDIRECTED

Non-Connectable and Non-Scannable Undirected advertising

ESP_BLE_GAP_SET_EXT_ADV_PROP_CONNECTABLE

Connectable advertising

ESP_BLE_GAP_SET_EXT_ADV_PROP_SCANNABLE

Scannable advertising

ESP_BLE_GAP_SET_EXT_ADV_PROP_DIRECTED

Directed advertising

ESP_BLE_GAP_SET_EXT_ADV_PROP_HD_DIRECTED

High Duty Cycle Directed Connectable advertising (<= 3.75 ms Advertising Interval)

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY

Use legacy advertising PDUs

ESP_BLE_GAP_SET_EXT_ADV_PROP_ANON_ADV

Omit advertiser's address from all PDUs ("anonymous advertising")

ESP_BLE_GAP_SET_EXT_ADV_PROP_INCLUDE_TX_PWR

Include TxPower in the extended header of the advertising PDU

ESP_BLE_GAP_SET_EXT_ADV_PROP_MASK

Reserved for future use If extended advertising PDU types are being used (bit 4 = 0) then: The advertisement shall not be both connectable and scannable. High duty cycle directed connectable advertising (<= 3.75 ms advertising interval) shall not be used (bit 3 = 0) ADV_IND

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY_IND

ADV_DIRECT_IND (low duty cycle)

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY_LD_DIR

ADV_DIRECT_IND (high duty cycle)

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY_HD_DIR

ADV_SCAN_IND

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY_SCAN

ADV_NONCONN_IND

ESP_BLE_GAP_SET_EXT_ADV_PROP_LEGACY_NONCONN

ESP_BLE_GAP_PHY_1M

Secondery Advertisement PHY is LE1M

ESP_BLE_GAP_PHY_2M

Secondery Advertisement PHY is LE2M

ESP_BLE_GAP_PHY_CODED

Secondery Advertisement PHY is LE Coded

ESP_BLE_GAP_NO_PREFER_TRANSMIT_PHY

No Prefer TX PHY supported by controller

ESP_BLE_GAP_NO_PREFER_RECEIVE_PHY

No Prefer RX PHY supported by controller

ESP_BLE_GAP_PRI_PHY_1M

Primary phy only support 1M and LE coded phy.

Primary Phy is LE1M

ESP_BLE_GAP_PRI_PHY_CODED

Primary Phy is LE CODED

ESP_BLE_GAP_PHY_1M_PREF_MASK

The Host prefers use the LE1M transmitter or receiver PHY

ESP_BLE_GAP_PHY_2M_PREF_MASK

The Host prefers use the LE2M transmitter or receiver PHY

ESP_BLE_GAP_PHY_CODED_PREF_MASK

The Host prefers use the LE CODED transmitter or receiver PHY

ESP_BLE_GAP_PHY_OPTIONS_NO_PREF

The Host has no preferred coding when transmitting on the LE Coded PHY

ESP_BLE_GAP_PHY_OPTIONS_PREF_S2_CODING

The Host prefers that S=2 coding be used when transmitting on the LE Coded PHY

ESP_BLE_GAP_PHY_OPTIONS_PREF_S8_CODING

The Host prefers that S=8 coding be used when transmitting on the LE Coded PHY

ESP_BLE_GAP_EXT_SCAN_CFG_UNCODE_MASK

Scan Advertisements on the LE1M PHY

ESP_BLE_GAP_EXT_SCAN_CFG_CODE_MASK

Scan advertisements on the LE coded PHY

ESP_BLE_GAP_EXT_ADV_DATA_COMPLETE

Advertising data.

extended advertising data compete

ESP_BLE_GAP_EXT_ADV_DATA_INCOMPLETE

extended advertising data incomplete

ESP_BLE_GAP_EXT_ADV_DATA_TRUNCATED

extended advertising data truncated mode

ESP_BLE_GAP_SYNC_POLICY_BY_ADV_INFO

Advertising SYNC policy.

sync policy by advertising info

ESP_BLE_GAP_SYNC_POLICY_BY_PERIODIC_LIST

periodic advertising sync policy

ESP_BLE_ADV_REPORT_EXT_ADV_IND

Advertising report.

advertising report with extended advertising indication type

ESP_BLE_ADV_REPORT_EXT_SCAN_IND

advertising report with extended scan indication type

ESP_BLE_ADV_REPORT_EXT_DIRECT_ADV

advertising report with extended direct advertising indication type

ESP_BLE_ADV_REPORT_EXT_SCAN_RSP

advertising report with extended scan response indication type Bluetooth 5.0, Vol 2, Part E, 7.7.65.13

ESP_BLE_LEGACY_ADV_TYPE_IND

advertising report with legacy advertising indication type

ESP_BLE_LEGACY_ADV_TYPE_DIRECT_IND

advertising report with legacy direct indication type

ESP_BLE_LEGACY_ADV_TYPE_SCAN_IND

advertising report with legacy scan indication type

ESP_BLE_LEGACY_ADV_TYPE_NONCON_IND

advertising report with legacy non connectable indication type

ESP_BLE_LEGACY_ADV_TYPE_SCAN_RSP_TO_ADV_IND

advertising report with legacy scan response indication type

ESP_BLE_LEGACY_ADV_TYPE_SCAN_RSP_TO_ADV_SCAN_IND

advertising report with legacy advertising with scan response indication type

EXT_ADV_TX_PWR_NO_PREFERENCE

Extend advertising tx power, range: [-127, +126] dBm.

host has no preference for tx power

EXT_ADV_NUM_SETS_MAX

max number of advertising sets to enable or disable

max evt instance num

ESP_BLE_GAP_PAST_MODE_NO_SYNC_EVT

Periodic advertising sync trans mode.

No attempt is made to sync and no periodic adv sync transfer received event

ESP_BLE_GAP_PAST_MODE_NO_REPORT_EVT

An periodic adv sync transfer received event and no periodic adv report events

ESP_BLE_GAP_PAST_MODE_DUP_FILTER_DISABLED

Periodic adv report events will be enabled with duplicate filtering disabled

ESP_BLE_GAP_PAST_MODE_DUP_FILTER_ENABLED

Periodic adv report events will be enabled with duplicate filtering enabled

Type Definitions

typedef uint8_t **esp_ble_key_type_t**

typedef uint8_t **esp_ble_auth_req_t**

combination of the above bit pattern

typedef uint8_t **esp_ble_io_cap_t**

combination of the io capability

typedef uint8_t **esp_ble_dtm_pkt_payload_t**

typedef uint8_t **esp_gap_ble_channels**[ESP_GAP_BLE_CHANNELS_LEN]

typedef uint8_t **esp_duplicate_info_t**[ESP_BD_ADDR_LEN]

typedef uint16_t **esp_ble_ext_adv_type_mask_t**

typedef uint8_t **esp_ble_gap_phy_t**

typedef uint8_t **esp_ble_gap_all_phys_t**

typedef uint8_t **esp_ble_gap_pri_phy_t**

typedef uint8_t **esp_ble_gap_phy_mask_t**

typedef uint16_t **esp_ble_gap_prefer_phy_options_t**

typedef uint8_t **esp_ble_ext_scan_cfg_mask_t**

typedef uint8_t **esp_ble_gap_ext_adv_data_status_t**

typedef uint8_t **esp_ble_gap_sync_t**

typedef uint8_t **esp_ble_gap_adv_type_t**

typedef uint8_t **esp_ble_gap_past_mode_t**

typedef void (***esp_gap_ble_cb_t**)(*esp_gap_ble_cb_event_t* event, *esp_ble_gap_cb_param_t* *param)

GAP callback function type.

Param event : Event type
Param param : Point to callback parameter, currently is union type

Enumerations

enum **esp_gap_ble_cb_event_t**

GAP BLE callback event type.

Values:

enumerator **ESP_GAP_BLE_ADV_DATA_SET_COMPLETE_EVT**

When advertising data set complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_RSP_DATA_SET_COMPLETE_EVT**

When scan response data set complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_PARAM_SET_COMPLETE_EVT**

When scan parameters set complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_RESULT_EVT**

When one scan result ready, the event comes each time

enumerator **ESP_GAP_BLE_ADV_DATA_RAW_SET_COMPLETE_EVT**

When raw advertising data set complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_RSP_DATA_RAW_SET_COMPLETE_EVT**

When raw scan response data set complete, the event comes

enumerator **ESP_GAP_BLE_ADV_START_COMPLETE_EVT**

When start advertising complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_START_COMPLETE_EVT**

When start scan complete, the event comes

enumerator **ESP_GAP_BLE_AUTH_CMPL_EVT**

Authentication complete indication.

enumerator **ESP_GAP_BLE_KEY_EVT**

BLE key event for peer device keys

enumerator **ESP_GAP_BLE_SEC_REQ_EVT**

BLE security request

enumerator **ESP_GAP_BLE_PASSKEY_NOTIF_EVT**

passkey notification event

enumerator **ESP_GAP_BLE_PASSKEY_REQ_EVT**

passkey request event

enumerator **ESP_GAP_BLE_OOB_REQ_EVT**

OOB request event

enumerator **ESP_GAP_BLE_LOCAL_IR_EVT**

BLE local IR (identity Root 128-bit random static value used to generate Long Term Key) event

enumerator **ESP_GAP_BLE_LOCAL_ER_EVT**

BLE local ER (Encryption Root value used to generate identity resolving key) event

enumerator **ESP_GAP_BLE_NC_REQ_EVT**

Numeric Comparison request event

enumerator **ESP_GAP_BLE_ADV_STOP_COMPLETE_EVT**

When stop adv complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_STOP_COMPLETE_EVT**

When stop scan complete, the event comes

enumerator **ESP_GAP_BLE_SET_STATIC_RAND_ADDR_EVT**

When set the static rand address complete, the event comes

enumerator **ESP_GAP_BLE_UPDATE_CONN_PARAMS_EVT**

When update connection parameters complete, the event comes

enumerator **ESP_GAP_BLE_SET_PKT_LENGTH_COMPLETE_EVT**

When set pkt length complete, the event comes

enumerator **ESP_GAP_BLE_SET_LOCAL_PRIVACY_COMPLETE_EVT**

When Enable/disable privacy on the local device complete, the event comes

enumerator **ESP_GAP_BLE_REMOVE_BOND_DEV_COMPLETE_EVT**

When remove the bond device complete, the event comes

enumerator **ESP_GAP_BLE_CLEAR_BOND_DEV_COMPLETE_EVT**

When clear the bond device clear complete, the event comes

enumerator **ESP_GAP_BLE_GET_BOND_DEV_COMPLETE_EVT**

When get the bond device list complete, the event comes

enumerator **ESP_GAP_BLE_READ_RSSI_COMPLETE_EVT**

When read the rssi complete, the event comes

enumerator **ESP_GAP_BLE_UPDATE_WHITELIST_COMPLETE_EVT**

When add or remove whitelist complete, the event comes

enumerator **ESP_GAP_BLE_UPDATE_DUPLICATE_EXCEPTIONAL_LIST_COMPLETE_EVT**

When update duplicate exceptional list complete, the event comes

enumerator **ESP_GAP_BLE_SET_CHANNELS_EVT**

When setting BLE channels complete, the event comes

enumerator **ESP_GAP_BLE_READ_PHY_COMPLETE_EVT**

when reading phy complete, this event comes

enumerator **ESP_GAP_BLE_SET_PREFERRED_DEFAULT_PHY_COMPLETE_EVT**

when preferred default phy complete, this event comes

enumerator **ESP_GAP_BLE_SET_PREFERRED_PHY_COMPLETE_EVT**

when preferred phy complete , this event comes

enumerator **ESP_GAP_BLE_EXT_ADV_SET_RAND_ADDR_COMPLETE_EVT**

when extended set random address complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_SET_PARAMS_COMPLETE_EVT**

when extended advertising parameter complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_DATA_SET_COMPLETE_EVT**

when extended advertising data complete, the event comes

enumerator **ESP_GAP_BLE_EXT_SCAN_RSP_DATA_SET_COMPLETE_EVT**

when extended scan response data complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_START_COMPLETE_EVT**
when extended advertising start complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_STOP_COMPLETE_EVT**
when extended advertising stop complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_SET_REMOVE_COMPLETE_EVT**
when extended advertising set remove complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_SET_CLEAR_COMPLETE_EVT**
when extended advertising set clear complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SET_PARAMS_COMPLETE_EVT**
when periodic advertising parameter complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_DATA_SET_COMPLETE_EVT**
when periodic advertising data complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_START_COMPLETE_EVT**
when periodic advertising start complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_STOP_COMPLETE_EVT**
when periodic advertising stop complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_CREATE_SYNC_COMPLETE_EVT**
when periodic advertising create sync complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_CANCEL_COMPLETE_EVT**
when extended advertising sync cancel complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_TERMINATE_COMPLETE_EVT**
when extended advertising sync terminate complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_ADD_DEV_COMPLETE_EVT**
when extended advertising add device complete , the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_REMOVE_DEV_COMPLETE_EVT**
when extended advertising remove device complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_CLEAR_DEV_COMPLETE_EVT**
when extended advertising clear device, the event comes

enumerator **ESP_GAP_BLE_SET_EXT_SCAN_PARAMS_COMPLETE_EVT**
when extended scan parameter complete, the event comes

enumerator **ESP_GAP_BLE_EXT_SCAN_START_COMPLETE_EVT**
when extended scan start complete, the event comes

enumerator **ESP_GAP_BLE_EXT_SCAN_STOP_COMPLETE_EVT**

when extended scan stop complete, the event comes

enumerator **ESP_GAP_BLE_PREFER_EXT_CONN_PARAMS_SET_COMPLETE_EVT**

when extended prefer connection parameter set complete, the event comes

enumerator **ESP_GAP_BLE_PHY_UPDATE_COMPLETE_EVT**

when ble phy update complete, the event comes

enumerator **ESP_GAP_BLE_EXT_ADV_REPORT_EVT**

when extended advertising report complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_TIMEOUT_EVT**

when scan timeout complete, the event comes

enumerator **ESP_GAP_BLE_ADV_TERMINATED_EVT**

when advertising terminate data complete, the event comes

enumerator **ESP_GAP_BLE_SCAN_REQ_RECEIVED_EVT**

when scan req received complete, the event comes

enumerator **ESP_GAP_BLE_CHANNEL_SELECT_ALGORITHM_EVT**

when channel select algorithm complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_REPORT_EVT**

when periodic report advertising complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_LOST_EVT**

when periodic advertising sync lost complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_ESTAB_EVT**

when periodic advertising sync establish complete, the event comes

enumerator **ESP_GAP_BLE_SC_OOB_REQ_EVT**

Secure Connection OOB request event

enumerator **ESP_GAP_BLE_SC_CR_LOC_OOB_EVT**

Secure Connection create OOB data complete event

enumerator **ESP_GAP_BLE_GET_DEV_NAME_COMPLETE_EVT**

When getting BT device name complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_RECV_ENABLE_COMPLETE_EVT**

when set periodic advertising receive enable complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_COMPLETE_EVT**

when periodic advertising sync transfer complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SET_INFO_TRANS_COMPLETE_EVT**

when periodic advertising set info transfer complete, the event comes

enumerator **ESP_GAP_BLE_SET_PAST_PARAMS_COMPLETE_EVT**

when set periodic advertising sync transfer params complete, the event comes

enumerator **ESP_GAP_BLE_PERIODIC_ADV_SYNC_TRANS_RECV_EVT**

when periodic advertising sync transfer received, the event comes

enumerator **ESP_GAP_BLE_DTM_TEST_UPDATE_EVT**

when direct test mode state changes, the event comes

enumerator **ESP_GAP_BLE_ADV_CLEAR_COMPLETE_EVT**

When clear advertising complete, the event comes

enumerator **ESP_GAP_BLE_SET_RPA_TIMEOUT_COMPLETE_EVT**

When set the Resolvable Private Address (RPA) timeout completes, the event comes

enumerator **ESP_GAP_BLE_ADD_DEV_TO_RESOLVING_LIST_COMPLETE_EVT**

when add a device to the resolving list completes, the event comes

enumerator **ESP_GAP_BLE_VENDOR_CMD_COMPLETE_EVT**

When vendor hci command complete, the event comes

enumerator **ESP_GAP_BLE_SET_PRIVACY_MODE_COMPLETE_EVT**

When set privacy mode complete, the event comes

enumerator **ESP_GAP_BLE_EVT_MAX**

when maximum advertising event complete, the event comes

enum **esp_ble_adv_data_type**

The type of advertising data(not adv_type)

Values:

enumerator **ESP_BLE_AD_TYPE_FLAG**

enumerator **ESP_BLE_AD_TYPE_16SRV_PART**

enumerator **ESP_BLE_AD_TYPE_16SRV_CMPL**

enumerator **ESP_BLE_AD_TYPE_32SRV_PART**

enumerator **ESP_BLE_AD_TYPE_32SRV_CMPL**

enumerator **ESP_BLE_AD_TYPE_128SRV_PART**

enumerator **ESP_BLE_AD_TYPE_128SRV_CMPL**

enumerator **ESP_BLE_AD_TYPE_NAME_SHORT**

enumerator **ESP_BLE_AD_TYPE_NAME_CMPL**

enumerator **ESP_BLE_AD_TYPE_TX_PWR**

enumerator **ESP_BLE_AD_TYPE_DEV_CLASS**

enumerator **ESP_BLE_AD_TYPE_SM_TK**

enumerator **ESP_BLE_AD_TYPE_SM_OOB_FLAG**

enumerator **ESP_BLE_AD_TYPE_INT_RANGE**

enumerator **ESP_BLE_AD_TYPE_SOL_SRV_UUID**

enumerator **ESP_BLE_AD_TYPE_128SOL_SRV_UUID**

enumerator **ESP_BLE_AD_TYPE_SERVICE_DATA**

enumerator **ESP_BLE_AD_TYPE_PUBLIC_TARGET**

enumerator **ESP_BLE_AD_TYPE_RANDOM_TARGET**

enumerator **ESP_BLE_AD_TYPE_APPEARANCE**

enumerator **ESP_BLE_AD_TYPE_ADV_INT**

enumerator **ESP_BLE_AD_TYPE_LE_DEV_ADDR**

enumerator **ESP_BLE_AD_TYPE_LE_ROLE**

enumerator **ESP_BLE_AD_TYPE_SPAIR_C256**

enumerator **ESP_BLE_AD_TYPE_SPAIR_R256**

enumerator **ESP_BLE_AD_TYPE_32SOL_SRV_UUID**

enumerator **ESP_BLE_AD_TYPE_32SERVICE_DATA**

enumerator **ESP_BLE_AD_TYPE_128SERVICE_DATA**

enumerator **ESP_BLE_AD_TYPE_LE_SECURE_CONFIRM**

enumerator **ESP_BLE_AD_TYPE_LE_SECURE_RANDOM**

enumerator **ESP_BLE_AD_TYPE_URI**

enumerator **ESP_BLE_AD_TYPE_INDOOR_POSITION**

enumerator **ESP_BLE_AD_TYPE_TRANS_DISC_DATA**

enumerator **ESP_BLE_AD_TYPE_LE_SUPPORT_FEATURE**

enumerator **ESP_BLE_AD_TYPE_CHAN_MAP_UPDATE**

enumerator **ESP_BLE_AD_MANUFACTURER_SPECIFIC_TYPE**

enum **esp_ble_adv_type_t**

Advertising mode.

Values:

enumerator **ADV_TYPE_IND**

enumerator **ADV_TYPE_DIRECT_IND_HIGH**

enumerator **ADV_TYPE_SCAN_IND**

enumerator **ADV_TYPE_NONCONN_IND**

enumerator **ADV_TYPE_DIRECT_IND_LOW**

enum **esp_ble_adv_channel_t**

Advertising channel mask.

Values:

enumerator **ADV_CHNL_37**

enumerator **ADV_CHNL_38**

enumerator **ADV_CHNL_39**

enumerator **ADV_CHNL_ALL**

enum **esp_ble_adv_filter_t**

Values:

enumerator **ADV_FILTER_ALLOW_SCAN_ANY_CON_ANY**

Allow both scan and connection requests from anyone.

enumerator **ADV_FILTER_ALLOW_SCAN_WLST_CON_ANY**

Allow both scan req from White List devices only and connection req from anyone.

enumerator **ADV_FILTER_ALLOW_SCAN_ANY_CON_WLST**

Allow both scan req from anyone and connection req from White List devices only.

enumerator **ADV_FILTER_ALLOW_SCAN_WLST_CON_WLST**

Allow scan and connection requests from White List devices only.

enum **esp_ble_sec_act_t**

Values:

enumerator **ESP_BLE_SEC_ENCRYPT**

relate to BTA_DM_BLE_SEC_ENCRYPT in bta/bta_api.h. If the device has already bonded, the stack will used Long Term Key (LTK) to encrypt with the remote device directly. Else if the device hasn't bonded, the stack will used the default authentication request used the esp_ble_gap_set_security_param function set by the user.

enumerator **ESP_BLE_SEC_ENCRYPT_NO_MITM**

relate to BTA_DM_BLE_SEC_ENCRYPT_NO_MITM in bta/bta_api.h. If the device has been already bonded, the stack will check the LTK (Long Term Key) Whether the authentication request has been met, and if met, use the LTK to encrypt with the remote device directly, else re-pair with the remote device. Else if the device hasn't been bonded, the stack will use NO MITM authentication request in the current link instead of using the authreq in the esp_ble_gap_set_security_param function set by the user.

enumerator **ESP_BLE_SEC_ENCRYPT_MITM**

relate to BTA_DM_BLE_SEC_ENCRYPT_MITM in bta/bta_api.h. If the device has been already bonded, the stack will check the LTK (Long Term Key) whether the authentication request has been met, and if met, use the LTK to encrypt with the remote device directly, else re-pair with the remote device. Else if the device hasn't been bonded, the stack will use MITM authentication request in the current link instead of using the authreq in the esp_ble_gap_set_security_param function set by the user.

enum **esp_ble_sm_param_t**

Values:

enumerator **ESP_BLE_SM_PASSKEY**

Authentication requirements of local device

enumerator **ESP_BLE_SM_AUTHEN_REQ_MODE**

The IO capability of local device

enumerator **ESP_BLE_SM_IOCAP_MODE**

Initiator Key Distribution/Generation

enumerator **ESP_BLE_SM_SET_INIT_KEY**

Responder Key Distribution/Generation

enumerator **ESP_BLE_SM_SET_RSP_KEY**

Maximum Encryption key size to support

enumerator **ESP_BLE_SM_MAX_KEY_SIZE**

Minimum Encryption key size requirement from Peer

enumerator **ESP_BLE_SM_MIN_KEY_SIZE**

Set static Passkey

enumerator **ESP_BLE_SM_SET_STATIC_PASSKEY**

Reset static Passkey

enumerator **ESP_BLE_SM_CLEAR_STATIC_PASSKEY**

Accept only specified SMP Authentication requirement

enumerator **ESP_BLE_SM_ONLY_ACCEPT_SPECIFIED_SEC_AUTH**

Enable/Disable OOB support

enumerator **ESP_BLE_SM_OOB_SUPPORT**

Appl encryption key size

enumerator **ESP_BLE_APP_ENC_KEY_SIZE**

authentication max param

enumerator **ESP_BLE_SM_MAX_PARAM**

enum **esp_ble_dtm_update_evt_t**

Values:

enumerator **DTM_TX_START_EVT**

DTM TX start event.

enumerator **DTM_RX_START_EVT**

DTM RX start event.

enumerator **DTM_TEST_STOP_EVT**

DTM test end event.

enum **esp_ble_scan_type_t**

Ble scan type.

Values:

enumerator **BLE_SCAN_TYPE_PASSIVE**

Passive scan

enumerator **BLE_SCAN_TYPE_ACTIVE**

Active scan

enum **esp_ble_scan_filter_t**

Ble scan filter type.

Values:

enumerator **BLE_SCAN_FILTER_ALLOW_ALL**

Accept all :

i. advertisement packets except directed advertising packets not addressed to this device (default).

enumerator **BLE_SCAN_FILTER_ALLOW_ONLY_WLST**

Accept only :

i. advertisement packets from devices where the advertiser's address is in the White list.
ii. Directed advertising packets which are not addressed for this device shall be ignored.

enumerator **BLE_SCAN_FILTER_ALLOW_UND_RPA_DIR**

Accept all :

i. undirected advertisement packets, and
ii. directed advertising packets where the initiator address is a resolvable private address, and
iii. directed advertising packets addressed to this device.

enumerator **BLE_SCAN_FILTER_ALLOW_WLIST_RPA_DIR**

Accept all :

i. advertisement packets from devices where the advertiser's address is in the White list, and
ii. directed advertising packets where the initiator address is a resolvable private address, and
iii. directed advertising packets addressed to this device.

enum **esp_ble_scan_duplicate_t**

Ble scan duplicate type.

Values:

enumerator **BLE_SCAN_DUPLICATE_DISABLE**

the Link Layer should generate advertising reports to the host for each packet received

enumerator **BLE_SCAN_DUPLICATE_ENABLE**

the Link Layer should filter out duplicate advertising reports to the Host

enumerator **BLE_SCAN_DUPLICATE_ENABLE_RESET**

Duplicate filtering enabled, reset for each scan period, only supported in BLE 5.0.

enumerator **BLE_SCAN_DUPLICATE_MAX**

Reserved for future use.

enum **esp_ble_auth_fail_rsn_t**

Definition of the authentication failed reason.

Values:

enumerator **ESP_AUTH_SMP_PASSKEY_FAIL**

The user input of passkey failed

enumerator **ESP_AUTH_SMP_OOB_FAIL**

The OOB data is not available

enumerator **ESP_AUTH_SMP_PAIR_AUTH_FAIL**

The authentication requirements cannot be met

enumerator **ESP_AUTH_SMP_CONFIRM_VALUE_FAIL**

The confirm value does not match the calculated comparison value

enumerator **ESP_AUTH_SMP_PAIR_NOT_SUPPORT**

Pairing is not supported by the device

enumerator **ESP_AUTH_SMP_ENC_KEY_SIZE**

The resultant encryption key size is not long enough

enumerator **ESP_AUTH_SMP_INVALID_CMD**

The SMP command received is not supported by this device

enumerator **ESP_AUTH_SMP_UNKNOWN_ERR**

Pairing failed due to an unspecified reason

enumerator **ESP_AUTH_SMP_REPEATED_ATTEMPT**

Pairing or authentication procedure is disallowed

enumerator **ESP_AUTH_SMP_INVALID_PARAMETERS**

The command length is invalid or that a parameter is outside the specified range

enumerator **ESP_AUTH_SMP_DHKEY_CHK_FAIL**

The DHKey Check value received doesn’ t match the one calculated by the local device

enumerator **ESP_AUTH_SMP_NUM_COMP_FAIL**

The confirm values in the numeric comparison protocol do not match

enumerator **ESP_AUTH_SMP_BR_PARING_IN_PROGR**

Pairing Request sent over the BR/EDR transport is in progress

enumerator **ESP_AUTH_SMP_XTRANS_DERIVE_NOT_ALLOW**

The BR/EDR Link Key or BLE LTK cannot be used to derive

enumerator **ESP_AUTH_SMP_INTERNAL_ERR**

Internal error in pairing procedure

enumerator **ESP_AUTH_SMP_UNKNOWN_IO**

Unknown IO capability, unable to decide association model

enumerator **ESP_AUTH_SMP_INIT_FAIL**

SMP pairing initiation failed

enumerator **ESP_AUTH_SMP_CONFIRM_FAIL**

The confirm value does not match

enumerator **ESP_AUTH_SMP_BUSY**

Pending security request on going

enumerator **ESP_AUTH_SMP_ENC_FAIL**
The Controller failed to start encryption

enumerator **ESP_AUTH_SMP_STARTED**
SMP pairing process started

enumerator **ESP_AUTH_SMP_RSP_TIMEOUT**
Security Manager timeout due to no SMP command being received

enumerator **ESP_AUTH_SMP_DIV_NOT_AVAIL**
Encrypted Diversifier value not available

enumerator **ESP_AUTH_SMP_UNSPEC_ERR**
Unspecified failed reason

enumerator **ESP_AUTH_SMP_CONN_TOUT**
Pairing process failed due to connection timeout

enum **esp_gap_search_evt_t**
Sub Event of ESP_GAP_BLE_SCAN_RESULT_EVT.

Values:

enumerator **ESP_GAP_SEARCH_INQ_RES_EVT**
Inquiry result for a peer device.

enumerator **ESP_GAP_SEARCH_INQ_CMPL_EVT**
Inquiry complete.

enumerator **ESP_GAP_SEARCH_DISC_RES_EVT**
Discovery result for a peer device.

enumerator **ESP_GAP_SEARCH_DISC_BLE_RES_EVT**
Discovery result for BLE GATT based service on a peer device.

enumerator **ESP_GAP_SEARCH_DISC_CMPL_EVT**
Discovery complete.

enumerator **ESP_GAP_SEARCH_DI_DISC_CMPL_EVT**
Discovery complete.

enumerator **ESP_GAP_SEARCH_SEARCH_CANCEL_CMPL_EVT**
Search cancelled

enumerator **ESP_GAP_SEARCH_INQ_DISCARD_NUM_EVT**
The number of pkt discarded by flow control

enum **esp_ble_evt_type_t**
Ble scan result event type, to indicate the result is scan response or advertising data or other.

Values:

enumerator **ESP_BLE_EVT_CONN_ADV**

Connectable undirected advertising (ADV_IND)

enumerator **ESP_BLE_EVT_CONN_DIR_ADV**

Connectable directed advertising (ADV_DIRECT_IND)

enumerator **ESP_BLE_EVT_DISC_ADV**

Scannable undirected advertising (ADV_SCAN_IND)

enumerator **ESP_BLE_EVT_NON_CONN_ADV**

Non connectable undirected advertising (ADV_NONCONN_IND)

enumerator **ESP_BLE_EVT_SCAN_RSP**

Scan Response (SCAN_RSP)

enum **esp_ble_wl_operation_t**

Values:

enumerator **ESP_BLE_WHITELIST_REMOVE**

remove mac from whitelist

enumerator **ESP_BLE_WHITELIST_ADD**

add address to whitelist

enumerator **ESP_BLE_WHITELIST_CLEAR**

clear all device in whitelist

enum **esp_bt_duplicate_exceptional_subcode_type_t**

Values:

enumerator **ESP_BLE_DUPLICATE_EXCEPTIONAL_LIST_ADD**

Add device info into duplicate scan exceptional list

enumerator **ESP_BLE_DUPLICATE_EXCEPTIONAL_LIST_REMOVE**

Remove device info from duplicate scan exceptional list

enumerator **ESP_BLE_DUPLICATE_EXCEPTIONAL_LIST_CLEAN**

Clean duplicate scan exceptional list

enum **esp_ble_duplicate_exceptional_info_type_t**

Values:

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_ADV_ADDR**

BLE advertising address , device info will be added into ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_ADDR_LIST

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_LINK_ID**

BLE mesh link ID, it is for BLE mesh, device info will be added into ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_LINK_ID_LIST

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_BEACON_TYPE**
BLE mesh beacon AD type, the format is | Len | 0x2B | Beacon Type | Beacon Data |

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_PROV_SRV_ADV**
BLE mesh provisioning service uuid, the format is | 0x02 | 0x01 | flags | 0x03 | 0x03 | 0x1827 | .... |`

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_PROXY_SRV_ADV**
BLE mesh adv with proxy service uuid, the format is | 0x02 | 0x01 | flags | 0x03 | 0x03 | 0x1828 | .... |`

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_PROXY_SOLIC_ADV**
BLE mesh adv with proxy service uuid, the format is | 0x02 | 0x01 | flags | 0x03 | 0x03 | 0x1859 | .... |`

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_INFO_MESH_URI_ADV**
BLE mesh URI adv, the format is ...| Len | 0x24 | data |...

enum **esp_duplicate_scan_exceptional_list_type_t**
Values:

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_ADDR_LIST**
duplicate scan exceptional addr list

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_LINK_ID_LIST**
duplicate scan exceptional mesh link ID list

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_BEACON_TYPE_LIST**
duplicate scan exceptional mesh beacon type list

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_PROV_SRV_ADV_LIST**
duplicate scan exceptional mesh adv with provisioning service uuid

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_PROXY_SRV_ADV_LIST**
duplicate scan exceptional mesh adv with proxy service uuid

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_PROXY_SOLIC_ADV_LIST**
duplicate scan exceptional mesh adv with proxy solicitation PDU uuid

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_MESH_URI_ADV_LIST**
duplicate scan exceptional URI list

enumerator **ESP_BLE_DUPLICATE_SCAN_EXCEPTIONAL_ALL_LIST**
duplicate scan exceptional all list

enum **esp_ble_privacy_mode_t**
Values:

enumerator **ESP_BLE_NETWORK_PRIVACY_MODE**
Network Privacy Mode for peer device (default)

enumerator **ESP_BLE_DEVICE_PRIVACY_MODE**
Device Privacy Mode for peer device

GATT Defines

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_gatt_defs.h
- This header file can be included with:

```
#include "esp_gatt_defs.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Unions

union **esp_gatt_rsp_t**

#include <esp_gatt_defs.h> Represents the response type for a GATT remote read request.

Public Members

esp_gatt_value_t **attr_value**

The GATT attribute value, including its data, handle, and metadata.

uint16_t **handle**

Only the handle of the GATT attribute, when that's the only required information.

Structures

struct **esp_gatt_id_t**

Represents a GATT identifier.

Public Members

esp_bt_uuid_t **uuid**

The UUID component of the GATT ID.

uint8_t **inst_id**

The instance ID component of the GATT ID, providing further differentiation of the GATT ID.

struct **esp_gatt_srvc_id_t**

Represents a GATT service identifier.

Public Members

esp_gatt_id_t **id**

Encapsulates the UUID and instance ID of the GATT service.

bool **is_primary**

Indicates if the service is primary. A value of true means it is a primary service, false indicates a secondary service.

struct **esp_attr_desc_t**

Defines an attribute's description.

This structure is used to describe an attribute in the GATT database. It includes details such as the UUID of the attribute, its permissions, and its value.

Public Members

uint16_t **uuid_length**

Length of the UUID in bytes.

uint8_t ***uuid_p**

Pointer to the UUID value.

uint16_t **perm**

Attribute permissions, defined by esp_gatt_perm_t.

uint16_t **max_length**

Maximum length of the attribute's value.

uint16_t **length**

Current length of the attribute's value.

uint8_t ***value**

Pointer to the attribute's value array.

struct **esp_attr_control_t**

Defines the auto response setting for attribute operations.

This structure is used to control whether the GATT stack or the application will handle responses to Read/Write operations.

Public Members

uint8_t **auto_rsp**

Controls who handles the response to Read/Write operations.

- If set to `ESP_GATT_RSP_BY_APP`, the application is responsible for generating the response.
- If set to `ESP_GATT_AUTO_RSP`, the GATT stack will automatically generate the response.

struct **esp_gatts_attr_db_t**

attribute type added to the GATT server database

Public Members

esp_attr_control_t **attr_control**

The attribute control type

esp_attr_desc_t **att_desc**

The attribute type

struct **esp_attr_value_t**

set the attribute value type

Public Members

uint16_t **attr_max_len**

attribute max value length

uint16_t **attr_len**

attribute current value length

uint8_t ***attr_value**

the pointer to attribute value

struct **esp_gatts_incl_svc_desc_t**

Gatt include service entry element.

Public Members

uint16_t **start_hdl**

Gatt start handle value of included service

uint16_t **end_hdl**

Gatt end handle value of included service

uint16_t **uuid**

Gatt attribute value UUID of included service

struct **esp_gatts_incl128_svc_desc_t**

Gatt include 128 bit service entry element.

Public Members

uint16_t **start_hdl**

Gatt start handle value of included 128 bit service

uint16_t **end_hdl**

Gatt end handle value of included 128 bit service

struct **esp_gatt_value_t**

Represents a GATT attribute's value.

Public Members

uint8_t **value**[ESP_GATT_MAX_ATTR_LEN]

Array holding the value of the GATT attribute.

uint16_t **handle**

Unique identifier (handle) of the GATT attribute.

uint16_t **offset**

Offset within the attribute's value, for partial updates.

uint16_t **len**

Current length of the data in the value array.

uint8_t **auth_req**

Authentication requirements for accessing this attribute.

struct **esp_gatt_conn_params_t**

Connection parameters for GATT.

Public Members

uint16_t **interval**

Connection interval.

uint16_t **latency**

Slave latency for the connection in number of connection events.

uint16_t **timeout**

Supervision timeout for the LE Link.

struct **esp_gattc_multi_t**

Represents multiple attributes for reading.

Public Members

uint8_t **num_attr**

Number of attributes.

uint16_t **handles**[ESP_GATT_MAX_READ_MULTI_HANDLES]

List of attribute handles.

struct **esp_gattc_db_elem_t**

GATT database attribute element.

Public Members

esp_gatt_db_attr_type_t **type**

Attribute type.

uint16_t **attribute_handle**

Attribute handle.

uint16_t **start_handle**

Service start handle.

uint16_t **end_handle**

Service end handle.

esp_gatt_char_prop_t **properties**

Characteristic properties.

esp_bt_uuid_t **uuid**

Attribute UUID.

struct **esp_gattc_service_elem_t**

Represents a GATT service element.

Public Members

bool **is_primary**

Indicates if the service is primary.

uint16_t **start_handle**

Service start handle.

uint16_t **end_handle**

Service end handle.

esp_bt_uuid_t **uuid**

Service UUID.

struct **esp_gattc_char_elem_t**

Represents a GATT characteristic element.

Public Members

uint16_t **char_handle**

Characteristic handle.

esp_gatt_char_prop_t **properties**

Characteristic properties.

esp_bt_uuid_t **uuid**

Characteristic UUID.

struct **esp_gattc_descr_elem_t**

Represents a GATT descriptor element.

Public Members

uint16_t **handle**

Descriptor handle.

esp_bt_uuid_t **uuid**

Descriptor UUID.

struct **esp_gattc_incl_svc_elem_t**

Represents an included GATT service element.

Public Members

uint16_t **handle**

Current attribute handle of the included service.

uint16_t **incl_srvc_s_handle**

Start handle of the included service.

uint16_t **incl_srvc_e_handle**

End handle of the included service.

esp_bt_uuid_t **uuid**

Included service UUID.

Macros

ESP_GATT_ILLEGAL_UUID

GATT INVALID UUID.

ESP_GATT_ILLEGAL_HANDLE

GATT INVALID HANDLE.

ESP_GATT_ATTR_HANDLE_MAX

GATT attribute max handle.

ESP_GATT_MAX_READ_MULTI_HANDLES

Maximum number of attributes to read in one request.

ESP_GATT_UUID_IMMEDIATE_ALERT_SVC

Immediate Alert Service UUID.

ESP_GATT_UUID_LINK_LOSS_SVC

Link Loss Service UUID.

ESP_GATT_UUID_TX_POWER_SVC

TX Power Service UUID.

ESP_GATT_UUID_CURRENT_TIME_SVC

Current Time Service UUID.

ESP_GATT_UUID_REF_TIME_UPDATE_SVC

Reference Time Update Service UUID.

ESP_GATT_UUID_NEXT_DST_CHANGE_SVC

Next DST Change Service UUID.

ESP_GATT_UUID_GLUCOSE_SVC

Glucose Service UUID.

ESP_GATT_UUID_HEALTH_THERMOM_SVC

Health Thermometer Service UUID.

ESP_GATT_UUID_DEVICE_INFO_SVC

Device Information Service UUID.

ESP_GATT_UUID_HEART_RATE_SVC

Heart Rate Service UUID.

ESP_GATT_UUID_PHONE_ALERT_STATUS_SVC

Phone Alert Status Service UUID.

ESP_GATT_UUID_BATTERY_SERVICE_SVC

Battery Service UUID.

ESP_GATT_UUID_BLOOD_PRESSURE_SVC

Blood Pressure Service UUID.

ESP_GATT_UUID_ALERT_NTF_SVC

Alert Notification Service UUID.

ESP_GATT_UUID_HID_SVC
HID Service UUID.

ESP_GATT_UUID_SCAN_PARAMETERS_SVC
Scan Parameters Service UUID.

ESP_GATT_UUID_RUNNING_SPEED_CADENCE_SVC
Running Speed and Cadence Service UUID.

ESP_GATT_UUID_Automation_IO_SVC
Automation IO Service UUID.

ESP_GATT_UUID_CYCLING_SPEED_CADENCE_SVC
Cycling Speed and Cadence Service UUID.

ESP_GATT_UUID_CYCLING_POWER_SVC
Cycling Power Service UUID.

ESP_GATT_UUID_LOCATION_AND_NAVIGATION_SVC
Location and Navigation Service UUID.

ESP_GATT_UUID_ENVIRONMENTAL_SENSING_SVC
Environmental Sensing Service UUID.

ESP_GATT_UUID_BODY_COMPOSITION
Body Composition Service UUID.

ESP_GATT_UUID_USER_DATA_SVC
User Data Service UUID.

ESP_GATT_UUID_WEIGHT_SCALE_SVC
Weight Scale Service UUID.

ESP_GATT_UUID_BOND_MANAGEMENT_SVC
Bond Management Service UUID.

ESP_GATT_UUID_CONT_GLUCOSE_MONITOR_SVC
Continuous Glucose Monitoring Service UUID.

ESP_GATT_UUID_PRI_SERVICE
Primary Service UUID.

ESP_GATT_UUID_SEC_SERVICE
Secondary Service UUID.

ESP_GATT_UUID_INCLUDE_SERVICE
Include Service UUID.

ESP_GATT_UUID_CHAR_DECLARE
Characteristic Declaration UUID.

ESP_GATT_UUID_CHAR_EXT_PROP
Characteristic Extended Properties UUID.

ESP_GATT_UUID_CHAR_DESCRIPTION
Characteristic User Description UUID.

ESP_GATT_UUID_CHAR_CLIENT_CONFIG
Client Characteristic Configuration UUID.

ESP_GATT_UUID_CHAR_SRVR_CONFIG
Server Characteristic Configuration UUID.

ESP_GATT_UUID_CHAR_PRESENT_FORMAT
Characteristic Presentation Format UUID.

ESP_GATT_UUID_CHAR_AGG_FORMAT
Characteristic Aggregate Format UUID.

ESP_GATT_UUID_CHAR_VALID_RANGE
Characteristic Valid Range UUID.

ESP_GATT_UUID_EXT_RPT_REF_DESCR
External Report Reference Descriptor UUID.

ESP_GATT_UUID_RPT_REF_DESCR
Report Reference Descriptor UUID.

ESP_GATT_UUID_NUM_DIGITALS_DESCR
Number of Digitals Descriptor UUID.

ESP_GATT_UUID_VALUE_TRIGGER_DESCR
Value Trigger Setting Descriptor UUID.

ESP_GATT_UUID_ENV_SENSING_CONFIG_DESCR
Environmental Sensing Configuration Descriptor UUID.

ESP_GATT_UUID_ENV_SENSING_MEASUREMENT_DESCR
Environmental Sensing Measurement Descriptor UUID.

ESP_GATT_UUID_ENV_SENSING_TRIGGER_DESCR
Environmental Sensing Trigger Setting Descriptor UUID.

ESP_GATT_UUID_TIME_TRIGGER_DESCR
Time Trigger Setting Descriptor UUID.

ESP_GATT_UUID_GAP_DEVICE_NAME
GAP Device Name UUID.

ESP_GATT_UUID_GAP_ICON
GAP Icon UUID.

ESP_GATT_UUID_GAP_PREF_CONN_PARAM
GAP Preferred Connection Parameters UUID.

ESP_GATT_UUID_GAP_CENTRAL_ADDR_RESOL
GAP Central Address Resolution UUID.

ESP_GATT_UUID_GATT_SRV_CHGD
GATT Service Changed UUID.

ESP_GATT_UUID_ALERT_LEVEL
Alert Level UUID.

ESP_GATT_UUID_TX_POWER_LEVEL
TX Power Level UUID.

ESP_GATT_UUID_CURRENT_TIME
Current Time UUID.

ESP_GATT_UUID_LOCAL_TIME_INFO
Local Time Info UUID.

ESP_GATT_UUID_REF_TIME_INFO
Reference Time Information UUID.

ESP_GATT_UUID_NW_STATUS
Network Availability Status UUID.

ESP_GATT_UUID_NW_TRIGGER
Network Availability Trigger UUID.

ESP_GATT_UUID_ALERT_STATUS
Alert Status UUID.

ESP_GATT_UUID_RINGER_CP
Ringer Control Point UUID.

ESP_GATT_UUID_RINGER_SETTING
Ringer Setting UUID.

ESP_GATT_UUID_GM_MEASUREMENT
Glucose Measurement Characteristic UUID.

ESP_GATT_UUID_GM_CONTEXT

Glucose Measurement Context Characteristic UUID.

ESP_GATT_UUID_GM_CONTROL_POINT

Glucose Control Point Characteristic UUID.

ESP_GATT_UUID_GM_FEATURE

Glucose Feature Characteristic UUID.

ESP_GATT_UUID_SYSTEM_ID

System ID Characteristic UUID.

ESP_GATT_UUID_MODEL_NUMBER_STR

Model Number String Characteristic UUID.

ESP_GATT_UUID_SERIAL_NUMBER_STR

Serial Number String Characteristic UUID.

ESP_GATT_UUID_FW_VERSION_STR

Firmware Revision String Characteristic UUID.

ESP_GATT_UUID_HW_VERSION_STR

Hardware Revision String Characteristic UUID.

ESP_GATT_UUID_SW_VERSION_STR

Software Revision String Characteristic UUID.

ESP_GATT_UUID_MANU_NAME

Manufacturer Name String Characteristic UUID.

ESP_GATT_UUID_IEEE_DATA

IEEE 11073-20601 Regulatory Certification Data List Characteristic UUID.

ESP_GATT_UUID_PNP_ID

PnP ID Characteristic UUID.

ESP_GATT_UUID_HID_INFORMATION

HID Information Characteristic UUID.

ESP_GATT_UUID_HID_REPORT_MAP

HID Report Map Characteristic UUID.

ESP_GATT_UUID_HID_CONTROL_POINT

HID Control Point Characteristic UUID.

ESP_GATT_UUID_HID_REPORT

HID Report Characteristic UUID.

ESP_GATT_UUID_HID_PROTO_MODE

HID Protocol Mode Characteristic UUID.

ESP_GATT_UUID_HID_BT_KB_INPUT

HID Bluetooth Keyboard Input Characteristic UUID.

ESP_GATT_UUID_HID_BT_KB_OUTPUT

HID Bluetooth Keyboard Output Characteristic UUID.

ESP_GATT_UUID_HID_BT_MOUSE_INPUT

HID Bluetooth Mouse Input Characteristic UUID.

ESP_GATT_HEART_RATE_MEAS

Heart Rate Measurement Characteristic UUID.

ESP_GATT_BODY_SENSOR_LOCATION

Body Sensor Location Characteristic UUID.

ESP_GATT_HEART_RATE_CNTL_POINT

Heart Rate Control Point Characteristic UUID.

ESP_GATT_UUID_BATTERY_LEVEL

Battery Level Characteristic UUID.

ESP_GATT_UUID_SC_CONTROL_POINT

Sensor Control Point Characteristic UUID.

ESP_GATT_UUID_SENSOR_LOCATION

Sensor Location Characteristic UUID.

ESP_GATT_UUID_RSC_MEASUREMENT

RSC Measurement Characteristic UUID.

ESP_GATT_UUID_RSC_FEATURE

RSC Feature Characteristic UUID.

ESP_GATT_UUID_CSC_MEASUREMENT

CSC Measurement Characteristic UUID.

ESP_GATT_UUID_CSC_FEATURE

CSC Feature Characteristic UUID.

ESP_GATT_UUID_SCAN_INT_WINDOW

Scan Interval Window Characteristic UUID.

ESP_GATT_UUID_SCAN_REFRESH

Scan Refresh UUID.

ESP_GATT_PERM_READ

Permission to read the attribute. Corresponds to BTA_GATT_PERM_READ.

ESP_GATT_PERM_READ_ENCRYPTED

Permission to read the attribute with encryption. Corresponds to BTA_GATT_PERM_READ_ENCRYPTED.

ESP_GATT_PERM_READ_ENC_MITM

Permission to read the attribute with encrypted MITM (Man In The Middle) protection. Corresponds to BTA_GATT_PERM_READ_ENC_MITM.

ESP_GATT_PERM_WRITE

Permission to write to the attribute. Corresponds to BTA_GATT_PERM_WRITE.

ESP_GATT_PERM_WRITE_ENCRYPTED

Permission to write to the attribute with encryption. Corresponds to BTA_GATT_PERM_WRITE_ENCRYPTED.

ESP_GATT_PERM_WRITE_ENC_MITM

Permission to write to the attribute with encrypted MITM protection. Corresponds to BTA_GATT_PERM_WRITE_ENC_MITM.

ESP_GATT_PERM_WRITE_SIGNED

Permission for signed writes to the attribute. Corresponds to BTA_GATT_PERM_WRITE_SIGNED.

ESP_GATT_PERM_WRITE_SIGNED_MITM

Permission for signed writes to the attribute with MITM protection. Corresponds to BTA_GATT_PERM_WRITE_SIGNED_MITM.

ESP_GATT_PERM_READ_AUTHORIZATION

Permission to read the attribute with authorization.

ESP_GATT_PERM_WRITE_AUTHORIZATION

Permission to write to the attribute with authorization.

ESP_GATT_PERM_ENCRYPT_KEY_SIZE(keysize)

Macro to specify minimum encryption key size.

Parameters

- **keysize** -- The minimum size of the encryption key, in bytes.

ESP_GATT_CHAR_PROP_BIT_BROADCAST

Ability to broadcast.Corresponds to BTA_GATT_CHAR_PROP_BIT_BROADCAST.

ESP_GATT_CHAR_PROP_BIT_READ

Ability to read.Corresponds to BTA_GATT_CHAR_PROP_BIT_READ.

ESP_GATT_CHAR_PROP_BIT_WRITE_NR

Ability to write without response.Corresponds to BTA_GATT_CHAR_PROP_BIT_WRITE_NR.

ESP_GATT_CHAR_PROP_BIT_WRITE

Ability to write.Corresponds to BTA_GATT_CHAR_PROP_BIT_WRITE.

ESP_GATT_CHAR_PROP_BIT_NOTIFY

Ability to notify.Corresponds to BTA_GATT_CHAR_PROP_BIT_NOTIFY.

ESP_GATT_CHAR_PROP_BIT_INDICATE

Ability to indicate.Corresponds to BTA_GATT_CHAR_PROP_BIT_INDICATE.

ESP_GATT_CHAR_PROP_BIT_AUTH

Ability to authenticate.Corresponds to BTA_GATT_CHAR_PROP_BIT_AUTH.

ESP_GATT_CHAR_PROP_BIT_EXT_PROP

Has extended properties.Corresponds to BTA_GATT_CHAR_PROP_BIT_EXT_PROP.

ESP_GATT_MAX_ATTR_LEN

Defines the maximum length of a GATT attribute.

This definition specifies the maximum number of bytes that a GATT attribute can hold. As same as GATT_MAX_ATTR_LEN.

ESP_GATT_RSP_BY_APP

Defines attribute control for GATT operations.

This module provides definitions for controlling attribute auto responses in GATT operations.

Response to Write/Read operations should be handled by the application.

ESP_GATT_AUTO_RSP

Response to Write/Read operations should be automatically handled by the GATT stack.

ESP_GATT_IF_NONE

Macro indicating no specific GATT interface.

No specific application GATT interface.

Type Definitions

typedef uint16_t **esp_gatt_perm_t**

Type to represent GATT attribute permissions.

typedef uint8_t **esp_gatt_char_prop_t**

Type for characteristic properties bitmask.

typedef uint8_t **esp_gatt_if_t**

GATT interface type for client applications.

Enumerations

enum **esp_gatt_prep_write_type**

Defines the attribute write operation types from the client.

These values are used to specify the type of write operation in a prepare write sequence. relate to BTA_GATT_PREP_WRITE_xxx in bta/bta_gatt_api.h.

Values:

enumerator **ESP_GATT_PREP_WRITE_CANCEL**

Prepare write cancel. Corresponds to BTA_GATT_PREP_WRITE_CANCEL.

enumerator **ESP_GATT_PREP_WRITE_EXEC**

Prepare write execute. Corresponds to BTA_GATT_PREP_WRITE_EXEC.

enum **esp_gatt_status_t**

GATT operation status codes.

These status codes are used to indicate the result of various GATT operations. relate to BTA_GATT_xxx in bta/bta_gatt_api.h .

Values:

enumerator **ESP_GATT_OK**

0x0, Operation successful. Corresponds to BTA_GATT_OK.

enumerator **ESP_GATT_INVALID_HANDLE**

0x01, Invalid handle. Corresponds to BTA_GATT_INVALID_HANDLE.

enumerator **ESP_GATT_READ_NOT_PERMIT**

0x02, Read operation not permitted. Corresponds to BTA_GATT_READ_NOT_PERMIT.

enumerator **ESP_GATT_WRITE_NOT_PERMIT**

0x03, Write operation not permitted. Corresponds to BTA_GATT_WRITE_NOT_PERMIT.

enumerator **ESP_GATT_INVALID_PDU**

0x04, Invalid PDU. Corresponds to BTA_GATT_INVALID_PDU.

enumerator **ESP_GATT_INSUF_AUTHENTICATION**

0x05, Insufficient authentication. Corresponds to BTA_GATT_INSUF_AUTHENTICATION.

enumerator **ESP_GATT_REQ_NOT_SUPPORTED**

0x06, Request not supported. Corresponds to BTA_GATT_REQ_NOT_SUPPORTED.

enumerator **ESP_GATT_INVALID_OFFSET**

0x07, Invalid offset. Corresponds to BTA_GATT_INVALID_OFFSET.

enumerator **ESP_GATT_INSUF_AUTHORIZATION**

0x08, Insufficient authorization. Corresponds to BTA_GATT_INSUF_AUTHORIZATION.

enumerator **ESP_GATT_PREPARE_Q_FULL**

0x09, Prepare queue full. Corresponds to BTA_GATT_PREPARE_Q_FULL.

enumerator **ESP_GATT_NOT_FOUND**

0x0a, Not found. Corresponds to BTA_GATT_NOT_FOUND.

enumerator **ESP_GATT_NOT_LONG**

0x0b, Not long. Corresponds to BTA_GATT_NOT_LONG.

enumerator **ESP_GATT_INSUF_KEY_SIZE**

0x0c, Insufficient key size. Corresponds to BTA_GATT_INSUF_KEY_SIZE.

enumerator **ESP_GATT_INVALID_ATTR_LEN**

0x0d, Invalid attribute length. Corresponds to BTA_GATT_INVALID_ATTR_LEN.

enumerator **ESP_GATT_ERR_UNLIKELY**

0x0e, Unlikely error. Corresponds to BTA_GATT_ERR_UNLIKELY.

enumerator **ESP_GATT_INSUF_ENCRYPTION**

0x0f, Insufficient encryption. Corresponds to BTA_GATT_INSUF_ENCRYPTION.

enumerator **ESP_GATT_UNSUPPORT_GRP_TYPE**

0x10, Unsupported group type. Corresponds to BTA_GATT_UNSUPPORT_GRP_TYPE.

enumerator **ESP_GATT_INSUF_RESOURCE**

0x11, Insufficient resource. Corresponds to BTA_GATT_INSUF_RESOURCE.

enumerator **ESP_GATT_NO_RESOURCES**

0x80, No resources. Corresponds to BTA_GATT_NO_RESOURCES.

enumerator **ESP_GATT_INTERNAL_ERROR**

0x81, Internal error. Corresponds to BTA_GATT_INTERNAL_ERROR.

enumerator **ESP_GATT_WRONG_STATE**

0x82, Wrong state. Corresponds to BTA_GATT_WRONG_STATE.

enumerator **ESP_GATT_DB_FULL**

0x83, Database full. Corresponds to BTA_GATT_DB_FULL.

enumerator **ESP_GATT_BUSY**

0x84, Busy. Corresponds to BTA_GATT_BUSY.

enumerator **ESP_GATT_ERROR**

0x85, Generic error. Corresponds to BTA_GATT_ERROR.

enumerator **ESP_GATT_CMD_STARTED**

0x86, Command started. Corresponds to BTA_GATT_CMD_STARTED.

enumerator **ESP_GATT_ILLEGAL_PARAMETER**

0x87, Illegal parameter. Corresponds to BTA_GATT_ILLEGAL_PARAMETER.

enumerator **ESP_GATT_PENDING**

0x88, Operation pending. Corresponds to BTA_GATT_PENDING.

enumerator **ESP_GATT_AUTH_FAIL**

0x89, Authentication failed. Corresponds to BTA_GATT_AUTH_FAIL.

enumerator **ESP_GATT_MORE**

0x8a, More data available. Corresponds to BTA_GATT_MORE.

enumerator **ESP_GATT_INVALID_CFG**

0x8b, Invalid configuration. Corresponds to BTA_GATT_INVALID_CFG.

enumerator **ESP_GATT_SERVICE_STARTED**

0x8c, Service started. Corresponds to BTA_GATT_SERVICE_STARTED.

enumerator **ESP_GATT_ENCRYPTED_MITM**

0x0, Encrypted, with MITM protection. Corresponds to BTA_GATT_ENCRYPTED_MITM.

enumerator **ESP_GATT_ENCRYPTED_NO_MITM**

0x8d, Encrypted, without MITM protection. Corresponds to BTA_GATT_ENCRYPTED_NO_MITM.

enumerator **ESP_GATT_NOT_ENCRYPTED**

0x8e, Not encrypted. Corresponds to BTA_GATT_NOT_ENCRYPTED.

enumerator **ESP_GATT_CONGESTED**

0x8f, Congested. Corresponds to BTA_GATT_CONGESTED.

enumerator **ESP_GATT_DUP_REG**

0x90, Duplicate registration. Corresponds to BTA_GATT_DUP_REG.

enumerator **ESP_GATT_ALREADY_OPEN**

0x91, Already open. Corresponds to BTA_GATT_ALREADY_OPEN.

enumerator **ESP_GATT_CANCEL**

0x92, Operation cancelled. Corresponds to BTA_GATT_CANCEL.

enumerator **ESP_GATT_STACK_RSP**

0xe0, Stack response. Corresponds to BTA_GATT_STACK_RSP.

enumerator **ESP_GATT_APP_RSP**

0xe1, Application response. Corresponds to BTA_GATT_APP_RSP.

enumerator **ESP_GATT_UNKNOWN_ERROR**

0xef, Unknown error. Corresponds to BTA_GATT_UNKNOWN_ERROR.

enumerator **ESP_GATT_CCC_CFG_ERR**

0xfd, Client Characteristic Configuration Descriptor improperly configured. Corresponds to BTA_GATT_CCC_CFG_ERR.

enumerator **ESP_GATT_PRC_IN_PROGRESS**

0xfe, Procedure already in progress. Corresponds to BTA_GATT_PRC_IN_PROGRESS.

enumerator **ESP_GATT_OUT_OF_RANGE**

0xff, Attribute value out of range. Corresponds to BTA_GATT_OUT_OF_RANGE.

enum **esp_gatt_conn_reason_t**

Enumerates reasons for GATT connection.

Values:

enumerator **ESP_GATT_CONN_UNKNOWN**

Unknown connection reason. Corresponds to BTA_GATT_CONN_UNKNOWN in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_L2C_FAILURE**

General L2CAP failure. Corresponds to BTA_GATT_CONN_L2C_FAILURE in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_TIMEOUT**

Connection timeout. Corresponds to BTA_GATT_CONN_TIMEOUT in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_TERMINATE_PEER_USER**

Connection terminated by peer user. Corresponds to BTA_GATT_CONN_TERMINATE_PEER_USER in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_TERMINATE_LOCAL_HOST**

Connection terminated by local host. Corresponds to BTA_GATT_CONN_TERMINATE_LOCAL_HOST in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_FAIL_ESTABLISH**

Failure to establish connection. Corresponds to BTA_GATT_CONN_FAIL_ESTABLISH in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_LMP_TIMEOUT**

Connection failed due to LMP response timeout. Corresponds to BTA_GATT_CONN_LMP_TIMEOUT in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_CONN_CANCEL**

L2CAP connection cancelled. Corresponds to BTA_GATT_CONN_CONN_CANCEL in bta/bta_gatt_api.h

enumerator **ESP_GATT_CONN_NONE**

No connection to cancel. Corresponds to BTA_GATT_CONN_NONE in bta/bta_gatt_api.h

enum **esp_gatt_auth_req_t**

Defines the GATT authentication request types.

This enumeration lists the types of authentication requests that can be made. It corresponds to the `BTA_GATT_AUTH_REQ_xxx` values defined in `bta/bta_gatt_api.h`. The types include options for no authentication, unauthenticated encryption, authenticated encryption, and both signed versions with and without MITM (Man-In-The-Middle) protection.

Values:

enumerator **ESP_GATT_AUTH_REQ_NONE**

No authentication required. Corresponds to BTA_GATT_AUTH_REQ_NONE.

enumerator **ESP_GATT_AUTH_REQ_NO_MITM**

Unauthenticated encryption. Corresponds to BTA_GATT_AUTH_REQ_NO_MITM.

enumerator **ESP_GATT_AUTH_REQ_MITM**

Authenticated encryption (MITM protection). Corresponds to BTA_GATT_AUTH_REQ_MITM.

enumerator **ESP_GATT_AUTH_REQ_SIGNED_NO_MITM**

Signed data, no MITM protection. Corresponds to BTA_GATT_AUTH_REQ_SIGNED_NO_MITM.

enumerator **ESP_GATT_AUTH_REQ_SIGNED_MITM**

Signed data with MITM protection. Corresponds to BTA_GATT_AUTH_REQ_SIGNED_MITM.

enum **esp_service_source_t**

Enumerates the possible sources of a GATT service discovery.

This enumeration identifies the source of a GATT service discovery process, indicating whether the service information was obtained from a remote device, from NVS (Non-Volatile Storage) flash, or the source is unknown.

Values:

enumerator **ESP_GATT_SERVICE_FROM_REMOTE_DEVICE**

Service information from a remote device. Relates to BTA_GATTC_SERVICE_INFO_FROM_REMOTE_DEVICE.

enumerator **ESP_GATT_SERVICE_FROM_NVS_FLASH**

Service information from NVS flash. Relates to BTA_GATTC_SERVICE_INFO_FROM_NVS_FLASH.

enumerator **ESP_GATT_SERVICE_FROM_UNKNOWN**

Service source is unknown. Relates to BTA_GATTC_SERVICE_INFO_FROM_UNKNOWN.

enum **esp_gatt_write_type_t**

Defines the types of GATT write operations.

Values:

enumerator **ESP_GATT_WRITE_TYPE_NO_RSP**

Write operation where no response is needed.

enumerator **ESP_GATT_WRITE_TYPE_RSP**

Write operation that requires a remote response.

enum **esp_gatt_db_attr_type_t**

Enumerates types of GATT database attributes.

Values:

enumerator **ESP_GATT_DB_PRIMARY_SERVICE**

Primary service attribute.

enumerator **ESP_GATT_DB_SECONDARY_SERVICE**

Secondary service attribute.

enumerator **ESP_GATT_DB_CHARACTERISTIC**

Characteristic attribute.

enumerator **ESP_GATT_DB_DESCRIPTOR**

Descriptor attribute.

enumerator **ESP_GATT_DB_INCLUDED_SERVICE**

Included service attribute.

enumerator **ESP_GATT_DB_ALL**

All attribute types.

GATT Server API

Application Examples Check bluetooth/bluedroid/ble folder in ESP-IDF examples, which contains the following demos and their tutorials:

- This is a GATT server demo and its tutorial. This demo creates a GATT service with an attribute table, which releases the user from the operation of adding attributes one by one. This is the recommended method of adding attributes (officially recommended).
  - bluetooth/bluedroid/ble/gatt_server_service_table
  - GATT Server Service Table Example Walkthrough
- This is a GATT server demo and its tutorial. This demo creates a GATT service by adding attributes one by one as defined by Bluedroid. The recommended method of adding attributes is presented in the example below.
  - bluetooth/bluedroid/ble/gatt_server
  - GATT Server Example Walkthrough
- This is a demo similar to Bluetooth® Low Energy (Bluetooth LE) SPP. In this demo, GATT server can receive data from UART and then send the data to the peer device automatically.
  - bluetooth/bluedroid/ble/ble_spp_server

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_gatts_api.h
- This header file can be included with:

```
#include "esp_gatts_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_gatts_register_callback**(*esp_gatts_cb_t* callback)

This function is called to register application callbacks with BTA GATTS module.

Returns

- ESP_OK : success
- other : failed

esp_gatts_cb_t **esp_ble_gatts_get_callback**(void)

This function is called to get the current application callbacks with BTA GATTS module.

Returns
- esp_gatts_cb_t : current callback

esp_err_t **esp_ble_gatts_app_register**(uint16_t app_id)

This function is called to register application identifier.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_app_unregister**(*esp_gatt_if_t* gatts_if)

unregister with GATT Server.

Parameters **gatts_if** -- **[in]** GATT server access interface
Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_create_service**(*esp_gatt_if_t* gatts_if, *esp_gatt_srvc_id_t* *service_id, uint16_t num_handle)

Create a service. When service creation is done, a callback event ESP_GATTS_CREATE_EVT is called to report status and service ID to the profile. The service ID obtained in the callback function needs to be used when adding included service and characteristics/descriptors into the service.

Parameters
- **gatts_if** -- **[in]** GATT server access interface
- **service_id** -- **[in]** service ID.
- **num_handle** -- **[in]** number of handle requested for this service.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_create_attr_tab**(const *esp_gatts_attr_db_t* *gatts_attr_db, *esp_gatt_if_t* gatts_if, uint16_t max_nb_attr, uint8_t srvc_inst_id)

Create a service attribute tab.

Parameters
- **gatts_attr_db** -- **[in]** the pointer to the service attr tab
- **gatts_if** -- **[in]** GATT server access interface
- **max_nb_attr** -- **[in]** the number of attribute to be added to the service database.
- **srvc_inst_id** -- **[in]** the instance id of the service

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_add_included_service**(uint16_t service_handle, uint16_t included_service_handle)

This function is called to add an included service. This function have to be called between 'esp_ble_gatts_create_service' and 'esp_ble_gatts_add_char'. After included service is included, a callback event ESP_GATTS_ADD_INCL_SRVC_EVT is reported the included service ID.

Parameters
- **service_handle** -- **[in]** service handle to which this included service is to be added.
- **included_service_handle** -- **[in]** the service ID to be included.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_add_char**(uint16_t service_handle, *esp_bt_uuid_t* *char_uuid, *esp_gatt_perm_t* perm, *esp_gatt_char_prop_t* property, *esp_attr_value_t* *char_val, *esp_attr_control_t* *control)

This function is called to add a characteristic into a service.

Parameters
- **service_handle** -- **[in]** service handle to which this included service is to be added.
- **char_uuid** -- **[in]** : Characteristic UUID.
- **perm** -- **[in]** : Characteristic value declaration attribute permission.
- **property** -- **[in]** : Characteristic Properties
- **char_val** -- **[in]** : Characteristic value
- **control** -- **[in]** : attribute response control byte

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_add_char_descr**(uint16_t service_handle, *esp_bt_uuid_t* *descr_uuid, *esp_gatt_perm_t* perm, *esp_attr_value_t* *char_descr_val, *esp_attr_control_t* *control)

This function is called to add characteristic descriptor. When it's done, a callback event ESP_GATTS_ADD_DESCR_EVT is called to report the status and an ID number for this descriptor.

Parameters
- **service_handle** -- **[in]** service handle to which this characteristic descriptor is to be added.
- **perm** -- **[in]** descriptor access permission.
- **descr_uuid** -- **[in]** descriptor UUID.
- **char_descr_val** -- **[in]** : Characteristic descriptor value
- **control** -- **[in]** : attribute response control byte

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_delete_service**(uint16_t service_handle)

This function is called to delete a service. When this is done, a callback event ESP_GATTS_DELETE_EVT is report with the status.

Parameters **service_handle** -- **[in]** service_handle to be deleted.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_start_service**(uint16_t service_handle)

This function is called to start a service.

Parameters **service_handle** -- **[in]** the service handle to be started.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_stop_service**(uint16_t service_handle)

This function is called to stop a service.

Parameters **service_handle** -- **[in]** - service to be topped.

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_send_indicate**(*esp_gatt_if_t* gatts_if, uint16_t conn_id, uint16_t attr_handle, uint16_t value_len, uint8_t *value, bool need_confirm)

Send indicate or notify to GATT client. Set param need_confirm as false will send notification, otherwise indication. Note: the size of indicate or notify data need less than MTU size,see "esp_ble_gattc_send_mtu_req".

Parameters

- **gatts_if** -- **[in]** GATT server access interface
- **conn_id** -- **[in]** - connection id to indicate.
- **attr_handle** -- **[in]** - attribute handle to indicate.
- **value_len** -- **[in]** - indicate value length.
- **value** -- **[in]** value to indicate.
- **need_confirm** -- **[in]** - Whether a confirmation is required. false sends a GATT notification, true sends a GATT indication.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_send_response**(*esp_gatt_if_t* gatts_if, uint16_t conn_id, uint32_t trans_id, *esp_gatt_status_t* status, *esp_gatt_rsp_t* *rsp)

This function is called to send a response to a request.

Parameters

- **gatts_if** -- **[in]** GATT server access interface
- **conn_id** -- **[in]** - connection identifier.
- **trans_id** -- **[in]** - transfer id
- **status** -- **[in]** - response status
- **rsp** -- **[in]** - response data.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_set_attr_value**(uint16_t attr_handle, uint16_t length, const uint8_t *value)

This function is called to set the attribute value by the application.

Parameters

- **attr_handle** -- **[in]** the attribute handle which to be set
- **length** -- **[in]** the value length
- **value** -- **[in]** the pointer to the attribute value

Returns

- ESP_OK : success
- other : failed

esp_gatt_status_t **esp_ble_gatts_get_attr_value**(uint16_t attr_handle, uint16_t *length, const uint8_t **value)

Retrieve attribute value.

Parameters

- **attr_handle** -- **[in]** Attribute handle.
- **length** -- **[out]** pointer to the attribute value length
- **value** -- **[out]** Pointer to attribute value payload, the value cannot be modified by user

Returns

- ESP_GATT_OK : success
- other : failed

esp_err_t **esp_ble_gatts_open**(*esp_gatt_if_t* gatts_if, *esp_bd_addr_t* remote_bda, bool is_direct)

Open a direct open connection or add a background auto connection.

Parameters

- **gatts_if** -- **[in]** GATT server access interface
- **remote_bda** -- **[in]** remote device bluetooth device address.
- **is_direct** -- **[in]** direct connection or background auto connection

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_close**(*esp_gatt_if_t* gatts_if, uint16_t conn_id)

Close a connection a remote device.

Parameters

- **gatts_if** -- **[in]** GATT server access interface
- **conn_id** -- **[in]** connection ID to be closed.

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_send_service_change_indication**(*esp_gatt_if_t* gatts_if, *esp_bd_addr_t* remote_bda)

Send service change indication.

Parameters

- **gatts_if** -- **[in]** GATT server access interface
- **remote_bda** -- **[in]** remote device bluetooth device address. If remote_bda is NULL then it will send service change indication to all the connected devices and if not then to a specific device

Returns

- ESP_OK : success
- other : failed

esp_err_t **esp_ble_gatts_show_local_database**(void)

Print local database (GATT service table)

Returns

- ESP_OK : success
- other : failed

Unions

union **esp_ble_gatts_cb_param_t**

#include <esp_gatts_api.h> Gatt server callback parameters union.

Public Members

struct *esp_ble_gatts_cb_param_t*::*gatts_reg_evt_param* **reg**

Gatt server callback param of ESP_GATTS_REG_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_read_evt_param* **read**

Gatt server callback param of ESP_GATTS_READ_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_write_evt_param* **write**

Gatt server callback param of ESP_GATTS_WRITE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_exec_write_evt_param* **exec_write**

Gatt server callback param of ESP_GATTS_EXEC_WRITE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_mtu_evt_param* **mtu**

Gatt server callback param of ESP_GATTS_MTU_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_conf_evt_param* **conf**

Gatt server callback param of ESP_GATTS_CONF_EVT (confirm)

struct *esp_ble_gatts_cb_param_t*::*gatts_create_evt_param* **create**

Gatt server callback param of ESP_GATTS_CREATE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_add_incl_srvc_evt_param* **add_incl_srvc**

Gatt server callback param of ESP_GATTS_ADD_INCL_SRVC_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_add_char_evt_param* **add_char**

Gatt server callback param of ESP_GATTS_ADD_CHAR_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_add_char_descr_evt_param* **add_char_descr**

Gatt server callback param of ESP_GATTS_ADD_CHAR_DESCR_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_delete_evt_param* **del**

Gatt server callback param of ESP_GATTS_DELETE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_start_evt_param* **start**

Gatt server callback param of ESP_GATTS_START_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_stop_evt_param* **stop**

Gatt server callback param of ESP_GATTS_STOP_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_connect_evt_param* **connect**

Gatt server callback param of ESP_GATTS_CONNECT_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_disconnect_evt_param* **disconnect**

Gatt server callback param of ESP_GATTS_DISCONNECT_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_open_evt_param* **open**

Gatt server callback param of ESP_GATTS_OPEN_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_cancel_open_evt_param* **cancel_open**

Gatt server callback param of ESP_GATTS_CANCEL_OPEN_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_close_evt_param* **close**

Gatt server callback param of ESP_GATTS_CLOSE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_congest_evt_param* **congest**

Gatt server callback param of ESP_GATTS_CONGEST_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_rsp_evt_param* **rsp**

Gatt server callback param of ESP_GATTS_RESPONSE_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_add_attr_tab_evt_param* **add_attr_tab**

Gatt server callback param of ESP_GATTS_CREAT_ATTR_TAB_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_set_attr_val_evt_param* **set_attr_val**

Gatt server callback param of ESP_GATTS_SET_ATTR_VAL_EVT

struct *esp_ble_gatts_cb_param_t*::*gatts_send_service_change_evt_param* **service_change**

Gatt server callback param of ESP_GATTS_SEND_SERVICE_CHANGE_EVT

struct **gatts_add_attr_tab_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_CREAT_ATTR_TAB_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

esp_bt_uuid_t **svc_uuid**

Service uuid type

uint8_t **svc_inst_id**

Service id

uint16_t **num_handle**

The number of the attribute handle to be added to the gatts database

uint16_t ***handles**

The number to the handles

struct **gatts_add_char_descr_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_ADD_CHAR_DESCR_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **attr_handle**

Descriptor attribute handle

uint16_t **service_handle**

Service attribute handle

esp_bt_uuid_t **descr_uuid**

Characteristic descriptor uuid

struct **gatts_add_char_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_ADD_CHAR_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **attr_handle**

Characteristic attribute handle

uint16_t **service_handle**

Service attribute handle

esp_bt_uuid_t **char_uuid**

Characteristic uuid

struct **gatts_add_incl_srvc_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_ADD_INCL_SRVC_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **attr_handle**

Included service attribute handle

uint16_t **service_handle**

Service attribute handle

struct **gatts_cancel_open_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_CANCEL_OPEN_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

struct **gatts_close_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_CLOSE_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

struct **gatts_conf_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_CONF_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

uint16_t **handle**

attribute handle

uint16_t **len**

The indication or notification value length, len is valid when send notification or indication failed

uint8_t ***value**

The indication or notification value , value is valid when send notification or indication failed

struct **gatts_congest_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_LISTEN_EVT.

ESP_GATTS_CONGEST_EVT

Public Members

uint16_t **conn_id**

Connection id

bool **congested**

Congested or not

struct **gatts_connect_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_CONNECT_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint8_t **link_role**

Link role : master role = 0 ; slave role = 1

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

esp_gatt_conn_params_t **conn_params**

current Connection parameters

esp_ble_addr_type_t **ble_addr_type**

Remote BLE device address type

uint16_t **conn_handle**

HCI connection handle

struct **gatts_create_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_UNREG_EVT.

ESP_GATTS_CREATE_EVT

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **service_handle**

Service attribute handle

esp_gatt_srvc_id_t **service_id**

Service id, include service uuid and other information

struct **gatts_delete_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_DELETE_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **service_handle**

Service attribute handle

struct **gatts_disconnect_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_DISCONNECT_EVT.

Public Members

uint16_t **conn_id**

Connection id

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

esp_gatt_conn_reason_t **reason**

Indicate the reason of disconnection

struct **gatts_exec_write_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_EXEC_WRITE_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint32_t **trans_id**

Transfer id

esp_bd_addr_t **bda**

The bluetooth device address which been written

uint8_t **exec_write_flag**

Execute write flag

struct **gatts_mtu_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_MTU_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint16_t **mtu**

MTU size

struct **gatts_open_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_OPEN_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

struct **gatts_read_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_READ_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint32_t **trans_id**

Transfer id

esp_bd_addr_t **bda**

The bluetooth device address which been read

uint16_t **handle**

The attribute handle

uint16_t **offset**

Offset of the value, if the value is too long

bool **is_long**

The value is too long or not

bool **need_rsp**

The read operation need to do response

struct **gatts_reg_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_REG_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **app_id**

Application id which input in register API

struct **gatts_rsp_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_RESPONSE_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **handle**

Attribute handle which send response

struct **gatts_send_service_change_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_SEND_SERVICE_CHANGE_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

struct **gatts_set_attr_val_evt_param**

#include <esp_gatts_api.h> ESP_GATTS_SET_ATTR_VAL_EVT.

Public Members

uint16_t **srvc_handle**
The service handle

uint16_t **attr_handle**
The attribute handle

esp_gatt_status_t **status**
Operation status

struct **gatts_start_evt_param**
#include <esp_gatts_api.h> ESP_GATTS_START_EVT.

Public Members

esp_gatt_status_t **status**
Operation status

uint16_t **service_handle**
Service attribute handle

struct **gatts_stop_evt_param**
#include <esp_gatts_api.h> ESP_GATTS_STOP_EVT.

Public Members

esp_gatt_status_t **status**
Operation status

uint16_t **service_handle**
Service attribute handle

struct **gatts_write_evt_param**
#include <esp_gatts_api.h> ESP_GATTS_WRITE_EVT.

Public Members

uint16_t **conn_id**
Connection id

uint32_t **trans_id**
Transfer id

esp_bd_addr_t **bda**
The bluetooth device address which been written

uint16_t **handle**

The attribute handle

uint16_t **offset**

Offset of the value, if the value is too long

bool **need_rsp**

The write operation need to do response

bool **is_prep**

This write operation is prepare write

uint16_t **len**

The write attribute value length

uint8_t ***value**

The write attribute value

Macros

ESP_GATT_PREP_WRITE_CANCEL

Prepare write flag to indicate cancel prepare write

ESP_GATT_PREP_WRITE_EXEC

Prepare write flag to indicate execute prepare write

Type Definitions

typedef void (***esp_gatts_cb_t**)(*esp_gatts_cb_event_t* event, *esp_gatt_if_t* gatts_if, *esp_ble_gatts_cb_param_t* *param)

GATT Server callback function type.

Param event : Event type
Param gatts_if : GATT server access interface, normally different gatts_if correspond to different profile
Param param : Point to callback parameter, currently is union type

Enumerations

enum **esp_gatts_cb_event_t**

GATT Server callback function events.

Values:

enumerator **ESP_GATTS_REG_EVT**

When register application id, the event comes

enumerator **ESP_GATTS_READ_EVT**

When gatt client request read operation, the event comes

enumerator **ESP_GATTS_WRITE_EVT**

When gatt client request write operation, the event comes

enumerator **ESP_GATTS_EXEC_WRITE_EVT**

When gatt client request execute write, the event comes

enumerator **ESP_GATTS_MTU_EVT**

When set mtu complete, the event comes

enumerator **ESP_GATTS_CONF_EVT**

When receive confirm, the event comes

enumerator **ESP_GATTS_UNREG_EVT**

When unregister application id, the event comes

enumerator **ESP_GATTS_CREATE_EVT**

When create service complete, the event comes

enumerator **ESP_GATTS_ADD_INCL_SRVC_EVT**

When add included service complete, the event comes

enumerator **ESP_GATTS_ADD_CHAR_EVT**

When add characteristic complete, the event comes

enumerator **ESP_GATTS_ADD_CHAR_DESCR_EVT**

When add descriptor complete, the event comes

enumerator **ESP_GATTS_DELETE_EVT**

When delete service complete, the event comes

enumerator **ESP_GATTS_START_EVT**

When start service complete, the event comes

enumerator **ESP_GATTS_STOP_EVT**

When stop service complete, the event comes

enumerator **ESP_GATTS_CONNECT_EVT**

When gatt client connect, the event comes

enumerator **ESP_GATTS_DISCONNECT_EVT**

When gatt client disconnect, the event comes

enumerator **ESP_GATTS_OPEN_EVT**

When connect to peer, the event comes

enumerator **ESP_GATTS_CANCEL_OPEN_EVT**

When disconnect from peer, the event comes

enumerator **ESP_GATTS_CLOSE_EVT**

When gatt server close, the event comes

enumerator **ESP_GATTS_LISTEN_EVT**

When gatt listen to be connected the event comes

enumerator **ESP_GATTS_CONGEST_EVT**

When congest happen, the event comes

enumerator **ESP_GATTS_RESPONSE_EVT**

When gatt send response complete, the event comes

enumerator **ESP_GATTS_CREAT_ATTR_TAB_EVT**

When gatt create table complete, the event comes

enumerator **ESP_GATTS_SET_ATTR_VAL_EVT**

When gatt set attr value complete, the event comes

enumerator **ESP_GATTS_SEND_SERVICE_CHANGE_EVT**

When gatt send service change indication complete, the event comes

GATT Client API

Application Example Check bluetooth/bluedroid/ble folder in ESP-IDF examples, which contains the following demos and their tutorials:

- This is a GATT client demo and its tutorial. This demo can scan for devices, connect to the GATT server and discover its services.
  - bluetooth/bluedroid/ble/gatt_client
  - GATT Client Example Walkthrough
- This is a multiple connection demo and its tutorial. This demo can connect to multiple GATT server devices and discover their services.
  - bluetooth/bluedroid/ble/gattc_multi_connect
  - GATT Client Multi-connection Example Walkthrough
- This is a demo similar to Bluetooth® Low Energy (Bluetooth LE) SPP. This demo, which acts as a GATT client, can receive data from UART and then send the data to the peer device automatically.
  - bluetooth/bluedroid/ble/ble_spp_client

API Reference

Header File

- components/bt/host/bluedroid/api/include/api/esp_gattc_api.h
- This header file can be included with:

```
#include "esp_gattc_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_gattc_register_callback** (*esp_gattc_cb_t* callback)

This function is called to register application callbacks with GATTC module.

Parameters **callback** -- **[in]** : pointer to the application callback function.
Returns
- ESP_OK: success
- other: failed

esp_gattc_cb_t **esp_ble_gattc_get_callback** (void)

This function is called to get the current application callbacks with BTA GATTC module.

Returns
- esp_gattC_cb_t : current callback

esp_err_t **esp_ble_gattc_app_register** (uint16_t app_id)

This function is called to register application callbacks with GATTC module.

Parameters **app_id** -- **[in]** : Application Identify (UUID), for different application
Returns
- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_app_unregister** (*esp_gatt_if_t* gattc_if)

This function is called to unregister an application from the GATTC module.

Note: Before calling this API, ensure that all activities related to the application, such as connections, scans, ADV, are properly closed.

Parameters **gattc_if** -- **[in]** Gatt client access interface.
Returns
- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_open** (*esp_gatt_if_t* gattc_if, *esp_bd_addr_t* remote_bda, *esp_ble_addr_type_t* remote_addr_type, bool is_direct)

Open a direct connection or add a background auto connection.

Parameters
- **gattc_if** -- **[in]** Gatt client access interface.
- **remote_bda** -- **[in]** remote device bluetooth device address.
- **remote_addr_type** -- **[in]** remote device bluetooth device the address type.
- **is_direct** -- **[in]** direct connection or background auto connection(by now, background auto connection is not supported).

Returns
- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_aux_open** (*esp_gatt_if_t* gattc_if, *esp_bd_addr_t* remote_bda, *esp_ble_addr_type_t* remote_addr_type, bool is_direct)

esp_err_t **esp_ble_gattc_close** (*esp_gatt_if_t* gattc_if, uint16_t conn_id)

Close the virtual connection to the GATT server. gattc may have multiple virtual GATT server connections when multiple app_id registered, this API only close one virtual GATT server connection. if there exist other virtual GATT server connections, it does not disconnect the physical connection. if you want to disconnect the physical connection directly, you can use esp_ble_gap_disconnect(esp_bd_addr_t remote_device).

Parameters
- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID to be closed.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_send_mtu_req**(*esp_gatt_if_t* gattc_if, uint16_t conn_id)

Configure the MTU size in the GATT channel. This can be done only once per connection. Before using, use esp_ble_gatt_set_local_mtu() to configure the local MTU size.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_search_service**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, *esp_bt_uuid_t* *filter_uuid)

This function is called to get service from local cache. This function report service search result by a callback event, and followed by a service search complete event. Note: 128-bit base UUID will automatically be converted to a 16-bit UUID in the search results. Other types of UUID remain unchanged.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID.
- **filter_uuid** -- **[in]** a UUID of the service application is interested in. If Null, discover for all services.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_service**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, *esp_bt_uuid_t* *svc_uuid, *esp_gattc_service_elem_t* *result, uint16_t *count, uint16_t offset)

Find all the service with the given service uuid in the gattc cache, if the svc_uuid is NULL, find all the service. Note: It just get service from local cache, won't get from remote devices. If want to get it from remote device, need to used the esp_ble_gattc_cache_refresh, then call esp_ble_gattc_get_service again.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **svc_uuid** -- **[in]** the pointer to the service uuid.
- **result** -- **[out]** The pointer to the service which has been found in the gattc cache.
- **count** -- **[inout]** input the number of service want to find, it will output the number of service has been found in the gattc cache with the given service uuid.
- **offset** -- **[in]** Offset of the service position to get.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_all_char**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_gattc_char_elem_t* *result, uint16_t *count, uint16_t offset)

Find all the characteristic with the given service in the gattc cache Note: It just get characteristic from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **start_handle** -- **[in]** the attribute start handle.
- **end_handle** -- **[in]** the attribute end handle

- **result** -- **[out]** The pointer to the characteristic in the service.
- **count** -- **[inout]** input the number of characteristic want to find, it will output the number of characteristic has been found in the gattc cache with the given service.
- **offset** -- **[in]** Offset of the characteristic position to get.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_all_descr**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t char_handle, *esp_gattc_descr_elem_t* *result, uint16_t *count, uint16_t offset)

Find all the descriptor with the given characteristic in the gattc cache Note: It just get descriptor from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **char_handle** -- **[in]** the given characteristic handle
- **result** -- **[out]** The pointer to the descriptor in the characteristic.
- **count** -- **[inout]** input the number of descriptor want to find, it will output the number of descriptor has been found in the gattc cache with the given characteristic.
- **offset** -- **[in]** Offset of the descriptor position to get.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_char_by_uuid**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_bt_uuid_t* char_uuid, *esp_gattc_char_elem_t* *result, uint16_t *count)

Find the characteristic with the given characteristic uuid in the gattc cache Note: It just get characteristic from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **start_handle** -- **[in]** the attribute start handle
- **end_handle** -- **[in]** the attribute end handle
- **char_uuid** -- **[in]** the characteristic uuid
- **result** -- **[out]** The pointer to the characteristic in the service.
- **count** -- **[inout]** input the number of characteristic want to find, it will output the number of characteristic has been found in the gattc cache with the given service.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_descr_by_uuid**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_bt_uuid_t* char_uuid, *esp_bt_uuid_t* descr_uuid, *esp_gattc_descr_elem_t* *result, uint16_t *count)

Find the descriptor with the given characteristic uuid in the gattc cache Note: It just get descriptor from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **start_handle** -- **[in]** the attribute start handle
- **end_handle** -- **[in]** the attribute end handle
- **char_uuid** -- **[in]** the characteristic uuid.
- **descr_uuid** -- **[in]** the descriptor uuid.

- **result** -- **[out]** The pointer to the descriptor in the given characteristic.
- **count** -- **[inout]** input the number of descriptor want to find, it will output the number of descriptor has been found in the gattc cache with the given characteristic.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_descr_by_char_handle**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t char_handle, *esp_bt_uuid_t* descr_uuid, *esp_gattc_descr_elem_t* *result, uint16_t *count)

Find the descriptor with the given characteristic handle in the gattc cache Note: It just get descriptor from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **char_handle** -- **[in]** the characteristic handle.
- **descr_uuid** -- **[in]** the descriptor uuid.
- **result** -- **[out]** The pointer to the descriptor in the given characteristic.
- **count** -- **[inout]** input the number of descriptor want to find, it will output the number of descriptor has been found in the gattc cache with the given characteristic.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_include_service**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_bt_uuid_t* *incl_uuid, *esp_gattc_incl_svc_elem_t* *result, uint16_t *count)

Find the include service with the given service handle in the gattc cache Note: It just get include service from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **start_handle** -- **[in]** the attribute start handle
- **end_handle** -- **[in]** the attribute end handle
- **incl_uuid** -- **[in]** the include service uuid
- **result** -- **[out]** The pointer to the include service in the given service.
- **count** -- **[inout]** input the number of include service want to find, it will output the number of include service has been found in the gattc cache with the given service.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_attr_count**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, *esp_gatt_db_attr_type_t* type, uint16_t start_handle, uint16_t end_handle, uint16_t char_handle, uint16_t *count)

Find the attribute count with the given service or characteristic in the gattc cache.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** connection ID which identify the server.
- **type** -- **[in]** the attribute type.
- **start_handle** -- **[in]** the attribute start handle, if the type is ESP_GATT_DB_DESCRIPTOR, this parameter should be ignore

- **end_handle** -- **[in]** the attribute end handle, if the type is ESP_GATT_DB_DESCRIPTOR, this parameter should be ignore
- **char_handle** -- **[in]** the characteristic handle, this parameter valid when the type is ESP_GATT_DB_DESCRIPTOR. If the type isn't ESP_GATT_DB_DESCRIPTOR, this parameter should be ignore.
- **count** -- **[out]** output the number of attribute has been found in the gattc cache with the given attribute type.

Returns

- ESP_OK: success
- other: failed

esp_gatt_status_t **esp_ble_gattc_get_db**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_gattc_db_elem_t* *db, uint16_t *count)

This function is called to get the GATT database. Note: It just get attribute data base from local cache, won't get from remote devices.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **start_handle** -- **[in]** the attribute start handle
- **end_handle** -- **[in]** the attribute end handle
- **conn_id** -- **[in]** connection ID which identify the server.
- **db** -- **[in]** output parameter which will contain the GATT database copy. Caller is responsible for freeing it.
- **count** -- **[in]** number of elements in database.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_read_char**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, *esp_gatt_auth_req_t* auth_req)

This function is called to read a service's characteristics of the given characteristic handle.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **handle** -- **[in]** : characteristic handle to read.
- **auth_req** -- **[in]** : authenticate request type

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_read_by_type**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t start_handle, uint16_t end_handle, *esp_bt_uuid_t* *uuid, *esp_gatt_auth_req_t* auth_req)

This function is called to read a service's characteristics of the given characteristic UUID.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **start_handle** -- **[in]** : the attribute start handle.
- **end_handle** -- **[in]** : the attribute end handle
- **uuid** -- **[in]** : The UUID of attribute which will be read.
- **auth_req** -- **[in]** : authenticate request type

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_read_multiple**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, *esp_gattc_multi_t* *read_multi, *esp_gatt_auth_req_t* auth_req)

This function is called to read multiple characteristic or characteristic descriptors.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **read_multi** -- **[in]** : pointer to the read multiple parameter.
- **auth_req** -- **[in]** : authenticate request type

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_read_multiple_variable**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, *esp_gattc_multi_t* *read_multi, *esp_gatt_auth_req_t* auth_req)

This function is called to read multiple variable length characteristic or characteristic descriptors.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **read_multi** -- **[in]** : pointer to the read multiple parameter.
- **auth_req** -- **[in]** : authenticate request type

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_read_char_descr**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, *esp_gatt_auth_req_t* auth_req)

This function is called to read a characteristics descriptor.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **handle** -- **[in]** : descriptor handle to read.
- **auth_req** -- **[in]** : authenticate request type

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_write_char**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, uint16_t value_len, uint8_t *value, *esp_gatt_write_type_t* write_type, *esp_gatt_auth_req_t* auth_req)

This function is called to write characteristic value.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **handle** -- **[in]** : characteristic handle to write.
- **value_len** -- **[in]** length of the value to be written.
- **value** -- **[in]** : the value to be written.
- **write_type** -- **[in]** : the type of attribute write operation.
- **auth_req** -- **[in]** : authentication request.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_write_char_descr**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, uint16_t value_len, uint8_t *value, *esp_gatt_write_type_t* write_type, *esp_gatt_auth_req_t* auth_req)

This function is called to write characteristic descriptor value.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID

- **handle** -- **[in]** : descriptor handle to write.
- **value_len** -- **[in]** length of the value to be written.
- **value** -- **[in]** : the value to be written.
- **write_type** -- **[in]** : the type of attribute write operation.
- **auth_req** -- **[in]** : authentication request.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_prepare_write**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, uint16_t offset, uint16_t value_len, uint8_t *value, *esp_gatt_auth_req_t* auth_req)

This function is called to prepare write a characteristic value.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **handle** -- **[in]** : characteristic handle to prepare write.
- **offset** -- **[in]** : offset of the write value.
- **value_len** -- **[in]** length of the value to be written.
- **value** -- **[in]** : the value to be written.
- **auth_req** -- **[in]** : authentication request.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_prepare_write_char_descr**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, uint16_t handle, uint16_t offset, uint16_t value_len, uint8_t *value, *esp_gatt_auth_req_t* auth_req)

This function is called to prepare write a characteristic descriptor value.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **handle** -- **[in]** : characteristic descriptor handle to prepare write.
- **offset** -- **[in]** : offset of the write value.
- **value_len** -- **[in]** length of the value to be written.
- **value** -- **[in]** : the value to be written.
- **auth_req** -- **[in]** : authentication request.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_execute_write**(*esp_gatt_if_t* gattc_if, uint16_t conn_id, bool is_execute)

This function is called to execute write a prepare write sequence.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **conn_id** -- **[in]** : connection ID.
- **is_execute** -- **[in]** : execute or cancel.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_register_for_notify**(*esp_gatt_if_t* gattc_if, *esp_bd_addr_t* server_bda, uint16_t handle)

This function is called to register for notification of a service.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.

- **server_bda** -- **[in]** : target GATT server.
- **handle** -- **[in]** : GATT characteristic handle.

Returns

- ESP_OK: registration succeeds
- other: failed

esp_err_t **esp_ble_gattc_unregister_for_notify**(*esp_gatt_if_t* gattc_if, *esp_bd_addr_t* server_bda, uint16_t handle)

This function is called to de-register for notification of a service.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **server_bda** -- **[in]** : target GATT server.
- **handle** -- **[in]** : GATT characteristic handle.

Returns

- ESP_OK: unregister succeeds
- other: failed

esp_err_t **esp_ble_gattc_cache_refresh**(*esp_bd_addr_t* remote_bda)

Refresh the server cache store in the gattc stack of the remote device. If the device is connected, this API will restart the discovery of service information of the remote device.

Parameters **remote_bda** -- **[in]** remote device BD address.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_cache_assoc**(*esp_gatt_if_t* gattc_if, *esp_bd_addr_t* src_addr, *esp_bd_addr_t* assoc_addr, bool is_assoc)

Add or delete the associated address with the source address. Note: The role of this API is mainly when the client side has stored a server-side database, when it needs to connect another device, but the device's attribute database is the same as the server database stored on the client-side, calling this API can use the database that the device has stored used as the peer server database to reduce the attribute database search and discovery process and speed up the connection time. The associated address mains that device want to used the database has stored in the local cache. The source address mains that device want to share the database to the associated address device.

Parameters

- **gattc_if** -- **[in]** Gatt client access interface.
- **src_addr** -- **[in]** the source address which provide the attribute table.
- **assoc_addr** -- **[in]** the associated device address which went to share the attribute table with the source address.
- **is_assoc** -- **[in]** true add the associated device address, false remove the associated device address.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_cache_get_addr_list**(*esp_gatt_if_t* gattc_if)

Get the address list which has store the attribute table in the gattc cache. There will callback ESP_GATTC_GET_ADDR_LIST_EVT event when get address list complete.

Parameters **gattc_if** -- **[in]** Gatt client access interface.

Returns

- ESP_OK: success
- other: failed

esp_err_t **esp_ble_gattc_cache_clean**(*esp_bd_addr_t* remote_bda)

Clean the service cache of this device in the gattc stack,.

Parameters **remote_bda** -- **[in]** remote device BD address.

Returns

- ESP_OK: success
- other: failed

Unions

union **esp_ble_gattc_cb_param_t**

#include <esp_gattc_api.h> Gatt client callback parameters union.

Public Members

struct *esp_ble_gattc_cb_param_t*::*gattc_reg_evt_param* **reg**

Gatt client callback param of ESP_GATTC_REG_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_open_evt_param* **open**

Gatt client callback param of ESP_GATTC_OPEN_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_close_evt_param* **close**

Gatt client callback param of ESP_GATTC_CLOSE_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_cfg_mtu_evt_param* **cfg_mtu**

Gatt client callback param of ESP_GATTC_CFG_MTU_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_search_cmpl_evt_param* **search_cmpl**

Gatt client callback param of ESP_GATTC_SEARCH_CMPL_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_search_res_evt_param* **search_res**

Gatt client callback param of ESP_GATTC_SEARCH_RES_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_read_char_evt_param* **read**

Gatt client callback param of ESP_GATTC_READ_CHAR_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_write_evt_param* **write**

Gatt client callback param of ESP_GATTC_WRITE_DESCR_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_exec_cmpl_evt_param* **exec_cmpl**

Gatt client callback param of ESP_GATTC_EXEC_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_notify_evt_param* **notify**

Gatt client callback param of ESP_GATTC_NOTIFY_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_srvc_chg_evt_param* **srvc_chg**

Gatt client callback param of ESP_GATTC_SRVC_CHG_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_congest_evt_param* **congest**

Gatt client callback param of ESP_GATTC_CONGEST_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_reg_for_notify_evt_param* **reg_for_notify**

Gatt client callback param of ESP_GATTC_REG_FOR_NOTIFY_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_unreg_for_notify_evt_param* **unreg_for_notify**

Gatt client callback param of ESP_GATTC_UNREG_FOR_NOTIFY_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_connect_evt_param* **connect**

Gatt client callback param of ESP_GATTC_CONNECT_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_disconnect_evt_param* **disconnect**

Gatt client callback param of ESP_GATTC_DISCONNECT_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_set_assoc_addr_cmp_evt_param* **set_assoc_cmp**

Gatt client callback param of ESP_GATTC_SET_ASSOC_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_get_addr_list_evt_param* **get_addr_list**

Gatt client callback param of ESP_GATTC_GET_ADDR_LIST_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_queue_full_evt_param* **queue_full**

Gatt client callback param of ESP_GATTC_QUEUE_FULL_EVT

struct *esp_ble_gattc_cb_param_t*::*gattc_dis_srvc_cmpl_evt_param* **dis_srvc_cmpl**

Gatt client callback param of ESP_GATTC_DIS_SRVC_CMPL_EVT

struct **gattc_cfg_mtu_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_CFG_MTU_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

uint16_t **mtu**

MTU size

struct **gattc_close_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_CLOSE_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

esp_gatt_conn_reason_t **reason**

The reason of gatt connection close

struct **gattc_congest_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_CONGEST_EVT.

Public Members

uint16_t **conn_id**

Connection id

bool **congested**

Congested or not

struct **gattc_connect_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_CONNECT_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint8_t **link_role**

Link role : master role = 0 ; slave role = 1

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

esp_gatt_conn_params_t **conn_params**

current connection parameters

esp_ble_addr_type_t **ble_addr_type**

Remote BLE device address type

uint16_t **conn_handle**

HCI connection handle

struct **gattc_dis_srvc_cmpl_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_DIS_SRVC_CMPL_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

struct **gattc_disconnect_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_DISCONNECT_EVT.

Public Members

esp_gatt_conn_reason_t **reason**

disconnection reason

uint16_t **conn_id**

Connection id

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

struct **gattc_exec_cmpl_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_EXEC_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

struct **gattc_get_addr_list_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_GET_ADDR_LIST_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint8_t **num_addr**

The number of address in the gattc cache address list

esp_bd_addr_t ***addr_list**

The pointer to the address list which has been get from the gattc cache

struct **gattc_notify_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_NOTIFY_EVT.

Public Members

uint16_t **conn_id**
Connection id

esp_bd_addr_t **remote_bda**
Remote bluetooth device address

uint16_t **handle**
The Characteristic or descriptor handle

uint16_t **value_len**
Notify attribute value

uint8_t ***value**
Notify attribute value

bool **is_notify**
True means notify, false means indicate

struct **gattc_open_evt_param**
#include <esp_gattc_api.h> ESP_GATTC_OPEN_EVT.

Public Members

esp_gatt_status_t **status**
Operation status

uint16_t **conn_id**
Connection id

esp_bd_addr_t **remote_bda**
Remote bluetooth device address

uint16_t **mtu**
MTU size

struct **gattc_queue_full_evt_param**
#include <esp_gattc_api.h> ESP_GATTC_QUEUE_FULL_EVT.

Public Members

esp_gatt_status_t **status**
Operation status

uint16_t **conn_id**
Connection id

bool **is_full**

The gattc command queue is full or not

struct **gattc_read_char_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_READ_CHAR_EVT, ESP_GATTC_READ_DESCR_EVT, ESP_GATTC_READ_MULTIPLE_EVT, ESP_GATTC_READ_MULTI_VAR_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

uint16_t **handle**

Characteristic handle

uint8_t ***value**

Characteristic value

uint16_t **value_len**

Characteristic value length

struct **gattc_reg_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_REG_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **app_id**

Application id which input in register API

struct **gattc_reg_for_notify_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_REG_FOR_NOTIFY_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **handle**

The characteristic or descriptor handle

struct **gattc_search_cmpl_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_SEARCH_CMPL_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

esp_service_source_t **searched_service_source**

The source of the service information

struct **gattc_search_res_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_SEARCH_RES_EVT.

Public Members

uint16_t **conn_id**

Connection id

uint16_t **start_handle**

Service start handle

uint16_t **end_handle**

Service end handle

esp_gatt_id_t **srvc_id**

Service id, include service uuid and other information

bool **is_primary**

True if this is the primary service

struct **gattc_set_assoc_addr_cmp_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_SET_ASSOC_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

struct **gattc_srvc_chg_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_SRVC_CHG_EVT.

Public Members

esp_bd_addr_t **remote_bda**

Remote bluetooth device address

struct **gattc_unreg_for_notify_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_UNREG_FOR_NOTIFY_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **handle**

The characteristic or descriptor handle

struct **gattc_write_evt_param**

#include <esp_gattc_api.h> ESP_GATTC_WRITE_CHAR_EVT, ESP_GATTC_PREP_WRITE_EVT, ESP_GATTC_WRITE_DESCR_EVT.

Public Members

esp_gatt_status_t **status**

Operation status

uint16_t **conn_id**

Connection id

uint16_t **handle**

The Characteristic or descriptor handle

uint16_t **offset**

The prepare write offset, this value is valid only when prepare write

Type Definitions

typedef void (***esp_gattc_cb_t**)(*esp_gattc_cb_event_t* event, *esp_gatt_if_t* gattc_if, *esp_ble_gattc_cb_param_t* *param)

GATT Client callback function type.

Param event : Event type
Param gattc_if : GATT client access interface, normally different gattc_if correspond to different profile
Param param : Point to callback parameter, currently is union type

Enumerations

enum **esp_gattc_cb_event_t**

GATT Client callback function events.

Values:

enumerator **ESP_GATTC_REG_EVT**

When GATT client is registered, the event comes

enumerator **ESP_GATTC_UNREG_EVT**

When GATT client is unregistered, the event comes

enumerator **ESP_GATTC_OPEN_EVT**

When GATT virtual connection is set up, the event comes

enumerator **ESP_GATTC_READ_CHAR_EVT**

When GATT characteristic is read, the event comes

enumerator **ESP_GATTC_WRITE_CHAR_EVT**

When GATT characteristic write operation completes, the event comes

enumerator **ESP_GATTC_CLOSE_EVT**

When GATT virtual connection is closed, the event comes

enumerator **ESP_GATTC_SEARCH_CMPL_EVT**

When GATT service discovery is completed, the event comes

enumerator **ESP_GATTC_SEARCH_RES_EVT**

When GATT service discovery result is got, the event comes

enumerator **ESP_GATTC_READ_DESCR_EVT**

When GATT characteristic descriptor read completes, the event comes

enumerator **ESP_GATTC_WRITE_DESCR_EVT**

When GATT characteristic descriptor write completes, the event comes

enumerator **ESP_GATTC_NOTIFY_EVT**

When GATT notification or indication arrives, the event comes

enumerator **ESP_GATTC_PREP_WRITE_EVT**

When GATT prepare-write operation completes, the event comes

enumerator **ESP_GATTC_EXEC_EVT**

When write execution completes, the event comes

enumerator **ESP_GATTC_ACL_EVT**

When ACL connection is up, the event comes

enumerator **ESP_GATTC_CANCEL_OPEN_EVT**

When GATT client ongoing connection is cancelled, the event comes

enumerator **ESP_GATTC_SRVC_CHG_EVT**

When "service changed" occurs, the event comes

enumerator **ESP_GATTC_ENC_CMPL_CB_EVT**

When encryption procedure completes, the event comes

enumerator **ESP_GATTC_CFG_MTU_EVT**

When configuration of MTU completes, the event comes

enumerator **ESP_GATTC_ADV_DATA_EVT**

When advertising of data, the event comes

enumerator **ESP_GATTC_MULT_ADV_ENB_EVT**

When multi-advertising is enabled, the event comes

enumerator **ESP_GATTC_MULT_ADV_UPD_EVT**

When multi-advertising parameters are updated, the event comes

enumerator **ESP_GATTC_MULT_ADV_DATA_EVT**

When multi-advertising data arrives, the event comes

enumerator **ESP_GATTC_MULT_ADV_DIS_EVT**

When multi-advertising is disabled, the event comes

enumerator **ESP_GATTC_CONGEST_EVT**

When GATT connection congestion comes, the event comes

enumerator **ESP_GATTC_BTH_SCAN_ENB_EVT**

When batch scan is enabled, the event comes

enumerator **ESP_GATTC_BTH_SCAN_CFG_EVT**

When batch scan storage is configured, the event comes

enumerator **ESP_GATTC_BTH_SCAN_RD_EVT**

When Batch scan read event is reported, the event comes

enumerator **ESP_GATTC_BTH_SCAN_THR_EVT**

When Batch scan threshold is set, the event comes

enumerator **ESP_GATTC_BTH_SCAN_PARAM_EVT**

When Batch scan parameters are set, the event comes

enumerator **ESP_GATTC_BTH_SCAN_DIS_EVT**

When Batch scan is disabled, the event comes

enumerator **ESP_GATTC_SCAN_FLT_CFG_EVT**

When Scan filter configuration completes, the event comes

enumerator **ESP_GATTC_SCAN_FLT_PARAM_EVT**

When Scan filter parameters are set, the event comes

enumerator **ESP_GATTC_SCAN_FLT_STATUS_EVT**

When Scan filter status is reported, the event comes

enumerator **ESP_GATTC_ADV_VSC_EVT**

When advertising vendor spec content event is reported, the event comes

enumerator **ESP_GATTC_REG_FOR_NOTIFY_EVT**

When register for notification of a service completes, the event comes

enumerator **ESP_GATTC_UNREG_FOR_NOTIFY_EVT**

When unregister for notification of a service completes, the event comes

enumerator **ESP_GATTC_CONNECT_EVT**

When the ble physical connection is set up, the event comes

enumerator **ESP_GATTC_DISCONNECT_EVT**

When the ble physical connection disconnected, the event comes

enumerator **ESP_GATTC_READ_MULTIPLE_EVT**

When the ble characteristic or descriptor multiple complete, the event comes

enumerator **ESP_GATTC_QUEUE_FULL_EVT**

When the gattc command queue full, the event comes

enumerator **ESP_GATTC_SET_ASSOC_EVT**

When the ble gattc set the associated address complete, the event comes

enumerator **ESP_GATTC_GET_ADDR_LIST_EVT**

When the ble get gattc address list in cache finish, the event comes

enumerator **ESP_GATTC_DIS_SRVC_CMPL_EVT**

When the ble discover service complete, the event comes

enumerator **ESP_GATTC_READ_MULTI_VAR_EVT**

When read multiple variable characteristic complete, the event comes

BluFi API

Overview BluFi is a profile based GATT to config ESP32 Wi-Fi to connect/disconnect AP or setup a softap and etc.

Use should concern these things:

1. The event sent from profile. Then you need to do something as the event indicate.
2. Security reference. You can write your own Security functions such as symmetrical encryption/decryption and checksum functions. Even you can define the "Key Exchange/Negotiation" procedure.

Application Example Check bluetooth folder in ESP-IDF examples, which contains the following application:

- This is the BluFi demo. This demo can set ESP32's Wi-Fi to softap/station/softap&station mode and config Wi-Fi connections - bluetooth/blufi

API Reference

Header File

- components/bt/common/api/include/api/esp_blufi_api.h
- This header file can be included with:

```
#include "esp_blufi_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_blufi_register_callbacks**(*esp_blufi_callbacks_t* *callbacks)

This function is called to receive blufi callback event.

Parameters **callbacks** -- **[in]** callback functions
Returns ESP_OK - success, other - failed

esp_err_t **esp_blufi_profile_init**(void)

This function is called to initialize blufi_profile.

Returns ESP_OK - success, other - failed

esp_err_t **esp_blufi_profile_deinit**(void)

This function is called to de-initialize blufi_profile.

Returns ESP_OK - success, other - failed

esp_err_t **esp_blufi_send_wifi_conn_report**(wifi_mode_t opmode, *esp_blufi_sta_conn_state_t* sta_conn_state, uint8_t softap_conn_num, *esp_blufi_extra_info_t* *extra_info)

This function is called to send wifi connection report.

Parameters
- **opmode** -- : wifi opmode
- **sta_conn_state** -- : station is already in connection or not
- **softap_conn_num** -- : softap connection number
- **extra_info** -- : extra information, such as sta_ssid, softap_ssid and etc.

Returns ESP_OK - success, other - failed

esp_err_t **esp_blufi_send_wifi_list**(uint16_t apCount, *esp_blufi_ap_record_t* *list)

This function is called to send wifi list.

Parameters
- **apCount** -- : wifi list count
- **list** -- : wifi list

Returns ESP_OK - success, other - failed

uint16_t **esp_blufi_get_version**(void)

Get BLUFI profile version.

Returns Most 8bit significant is Great version, Least 8bit is Sub version

esp_err_t **esp_blufi_send_error_info**(*esp_blufi_error_state_t* state)

This function is called to send blufi error information.

Parameters **state** -- : error state
Returns ESP_OK - success, other - failed

esp_err_t **esp_blufi_send_custom_data** (uint8_t *data, uint32_t data_len)

This function is called to custom data.

Parameters

- **data** -- : custom data value
- **data_len** -- : the length of custom data

Returns ESP_OK - success, other - failed

Unions

union **esp_blufi_cb_param_t**

#include <esp_blufi_api.h> BLUFI callback parameters union.

Public Members

struct *esp_blufi_cb_param_t*::*blufi_init_finish_evt_param* **init_finish**

Blufi callback param of ESP_BLUFI_EVENT_INIT_FINISH

struct *esp_blufi_cb_param_t*::*blufi_deinit_finish_evt_param* **deinit_finish**

Blufi callback param of ESP_BLUFI_EVENT_DEINIT_FINISH

struct *esp_blufi_cb_param_t*::*blufi_set_wifi_mode_evt_param* **wifi_mode**

Blufi callback param of ESP_BLUFI_EVENT_INIT_FINISH

struct *esp_blufi_cb_param_t*::*blufi_connect_evt_param* **connect**

Blufi callback param of ESP_BLUFI_EVENT_CONNECT

struct *esp_blufi_cb_param_t*::*blufi_disconnect_evt_param* **disconnect**

Blufi callback param of ESP_BLUFI_EVENT_DISCONNECT

struct *esp_blufi_cb_param_t*::*blufi_recv_sta_bssid_evt_param* **sta_bssid**

Blufi callback param of ESP_BLUFI_EVENT_RECV_STA_BSSID

struct *esp_blufi_cb_param_t*::*blufi_recv_sta_ssid_evt_param* **sta_ssid**

Blufi callback param of ESP_BLUFI_EVENT_RECV_STA_SSID

struct *esp_blufi_cb_param_t*::*blufi_recv_sta_passwd_evt_param* **sta_passwd**

Blufi callback param of ESP_BLUFI_EVENT_RECV_STA_PASSWD

struct *esp_blufi_cb_param_t*::*blufi_recv_softap_ssid_evt_param* **softap_ssid**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SOFTAP_SSID

struct *esp_blufi_cb_param_t*::*blufi_recv_softap_passwd_evt_param* **softap_passwd**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SOFTAP_PASSWD

struct *esp_blufi_cb_param_t*::*blufi_recv_softap_max_conn_num_evt_param* **softap_max_conn_num**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SOFTAP_MAX_CONN_NUM

struct *esp_blufi_cb_param_t*::*blufi_recv_softap_auth_mode_evt_param* **softap_auth_mode**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SOFTAP_AUTH_MODE

struct *esp_blufi_cb_param_t*::*blufi_recv_softap_channel_evt_param* **softap_channel**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SOFTAP_CHANNEL

struct *esp_blufi_cb_param_t*::*blufi_recv_username_evt_param* **username**

Blufi callback param of ESP_BLUFI_EVENT_RECV_USERNAME

struct *esp_blufi_cb_param_t*::*blufi_recv_ca_evt_param* **ca**

Blufi callback param of ESP_BLUFI_EVENT_RECV_CA_CERT

struct *esp_blufi_cb_param_t*::*blufi_recv_client_cert_evt_param* **client_cert**

Blufi callback param of ESP_BLUFI_EVENT_RECV_CLIENT_CERT

struct *esp_blufi_cb_param_t*::*blufi_recv_server_cert_evt_param* **server_cert**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SERVER_CERT

struct *esp_blufi_cb_param_t*::*blufi_recv_client_pkey_evt_param* **client_pkey**

Blufi callback param of ESP_BLUFI_EVENT_RECV_CLIENT_PRIV_KEY

struct *esp_blufi_cb_param_t*::*blufi_recv_server_pkey_evt_param* **server_pkey**

Blufi callback param of ESP_BLUFI_EVENT_RECV_SERVER_PRIV_KEY

struct *esp_blufi_cb_param_t*::*blufi_get_error_evt_param* **report_error**

Blufi callback param of ESP_BLUFI_EVENT_REPORT_ERROR

struct *esp_blufi_cb_param_t*::*blufi_recv_custom_data_evt_param* **custom_data**

Blufi callback param of ESP_BLUFI_EVENT_RECV_CUSTOM_DATA

struct **blufi_connect_evt_param**

#include <*esp_blufi_api.h*> ESP_BLUFI_EVENT_CONNECT.

Public Members

esp_blufi_bd_addr_t **remote_bda**

Blufi Remote bluetooth device address

uint8_t **server_if**

server interface

uint16_t **conn_id**

Connection id

struct **blufi_deinit_finish_evt_param**

#include <*esp_blufi_api.h*> ESP_BLUFI_EVENT_DEINIT_FINISH.

Public Members

esp_blufi_deinit_state_t **state**

De-initial status

struct **blufi_disconnect_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_DISCONNECT.

Public Members

esp_blufi_bd_addr_t **remote_bda**

Blufi Remote bluetooth device address

struct **blufi_get_error_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_REPORT_ERROR.

Public Members

esp_blufi_error_state_t **state**

Blufi error state

struct **blufi_init_finish_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_INIT_FINISH.

Public Members

esp_blufi_init_state_t **state**

Initial status

struct **blufi_recv_ca_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_CA_CERT.

Public Members

uint8_t ***cert**

CA certificate point

int **cert_len**

CA certificate length

struct **blufi_recv_client_cert_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_CLIENT_CERT

Public Members

uint8_t ***cert**

Client certificate point

int **cert_len**

Client certificate length

struct **blufi_recv_client_pkey_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_CLIENT_PRIV_KEY

Public Members

uint8_t ***pkey**

Client Private Key point, if Client certificate not contain Key

int **pkey_len**

Client Private key length

struct **blufi_recv_custom_data_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_CUSTOM_DATA.

Public Members

uint8_t ***data**

Custom data

uint32_t **data_len**

Custom data Length

struct **blufi_recv_server_cert_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SERVER_CERT

Public Members

uint8_t ***cert**

Client certificate point

int **cert_len**

Client certificate length

struct **blufi_recv_server_pkey_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SERVER_PRIV_KEY

Public Members

uint8_t ***pkey**

Client Private Key point, if Client certificate not contain Key

int **pkey_len**

Client Private key length

struct **blufi_recv_softap_auth_mode_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SOFTAP_AUTH_MODE.

Public Members

wifi_auth_mode_t **auth_mode**

Authentication mode

struct **blufi_recv_softap_channel_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SOFTAP_CHANNEL.

Public Members

uint8_t **channel**

Authentication mode

struct **blufi_recv_softap_max_conn_num_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SOFTAP_MAX_CONN_NUM.

Public Members

int **max_conn_num**

SSID

struct **blufi_recv_softap_passwd_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SOFTAP_PASSWD.

Public Members

uint8_t ***passwd**

Password

int **passwd_len**

Password Length

struct **blufi_recv_softap_ssid_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_SOFTAP_SSID.

Public Members

uint8_t ***ssid**

SSID

int **ssid_len**

SSID length

struct **blufi_recv_sta_bssid_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_STA_BSSID.

Public Members

uint8_t **bssid**[6]

BSSID

struct **blufi_recv_sta_passwd_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_STA_PASSWD.

Public Members

uint8_t ***passwd**

Password

int **passwd_len**

Password Length

struct **blufi_recv_sta_ssid_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_STA_SSID.

Public Members

uint8_t ***ssid**

SSID

int **ssid_len**

SSID length

struct **blufi_recv_username_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_RECV_USERNAME.

Public Members

uint8_t ***name**

Username point

int **name_len**

Username length

struct **blufi_set_wifi_mode_evt_param**

#include <esp_blufi_api.h> ESP_BLUFI_EVENT_SET_WIFI_MODE.

Public Members

wifi_mode_t **op_mode**

Wifi operation mode

Structures

struct **esp_blufi_extra_info_t**

BLUFI extra information structure.

Public Members

uint8_t **sta_bssid**[6]

BSSID of station interface

bool **sta_bssid_set**

is BSSID of station interface set

uint8_t ***sta_ssid**

SSID of station interface

int **sta_ssid_len**

length of SSID of station interface

uint8_t ***sta_passwd**

password of station interface

int **sta_passwd_len**

length of password of station interface

uint8_t ***softap_ssid**

SSID of softap interface

int **softap_ssid_len**

length of SSID of softap interface

uint8_t ***softap_passwd**

password of station interface

int **softap_passwd_len**

length of password of station interface

uint8_t **softap_authmode**

authentication mode of softap interface

bool **softap_authmode_set**

is authentication mode of softap interface set

uint8_t **softap_max_conn_num**
: max connection number of softap interface

bool **softap_max_conn_num_set**
: is max connection number of softap interface set

uint8_t **softap_channel**
: channel of softap interface

bool **softap_channel_set**
: is channel of softap interface set

uint8_t **sta_max_conn_retry**
: max retry of sta establish connection

bool **sta_max_conn_retry_set**
: is max retry of sta establish connection set

uint8_t **sta_conn_end_reason**
: reason of sta connection end

bool **sta_conn_end_reason_set**
: is reason of sta connection end set

int8_t **sta_conn_rssi**
: rssi of sta connection

bool **sta_conn_rssi_set**
: is rssi of sta connection set

struct **esp_blufi_ap_record_t**
: Description of an WiFi AP.

Public Members

uint8_t **ssid**[33]
: SSID of AP

int8_t **rssi**
: signal strength of AP

struct **esp_blufi_callbacks_t**
: BLUFI callback functions type.

Public Members

esp_blufi_event_cb_t **event_cb**
: BLUFI event callback

esp_blufi_negotiate_data_handler_t **negotiate_data_handler**

BLUFI negotiate data function for negotiate share key

esp_blufi_encrypt_func_t **encrypt_func**

BLUFI encrypt data function with share key generated by negotiate_data_handler

esp_blufi_decrypt_func_t **decrypt_func**

BLUFI decrypt data function with share key generated by negotiate_data_handler

esp_blufi_checksum_func_t **checksum_func**

BLUFI check sum function (FCS)

Macros

ESP_BLUFI_BD_ADDR_LEN

Bluetooth address length.

Type Definitions

typedef uint8_t **esp_blufi_bd_addr_t**[ESP_BLUFI_BD_ADDR_LEN]

Bluetooth device address.

typedef void (***esp_blufi_event_cb_t**)(*esp_blufi_cb_event_t* event, *esp_blufi_cb_param_t* *param)

BLUFI event callback function type.

Param event : Event type
Param param : Point to callback parameter, currently is union type

typedef void (***esp_blufi_negotiate_data_handler_t**)(uint8_t *data, int len, uint8_t **output_data, int *output_len, bool *need_free)

BLUFI negotiate data handler.

Param data : data from phone
Param len : length of data from phone
Param output_data : data want to send to phone
Param output_len : length of data want to send to phone
Param need_free : output reporting if memory needs to be freed or not *

typedef int (***esp_blufi_encrypt_func_t**)(uint8_t iv8, uint8_t *crypt_data, int crypt_len)

BLUFI encrypt the data after negotiate a share key.

Param iv8 : initial vector(8bit), normally, blufi core will input packet sequence number
Param crypt_data : plain text and encrypted data, the encrypt function must support autochthonous encrypt
Param crypt_len : length of plain text
Return Nonnegative number is encrypted length, if error, return negative number;

typedef int (***esp_blufi_decrypt_func_t**)(uint8_t iv8, uint8_t *crypt_data, int crypt_len)

BLUFI decrypt the data after negotiate a share key.

Param iv8 : initial vector(8bit), normally, blufi core will input packet sequence number
Param crypt_data : encrypted data and plain text, the encrypt function must support autochthonous decrypt
Param crypt_len : length of encrypted text
Return Nonnegative number is decrypted length, if error, return negative number;

typedef uint16_t (*`esp_blufi_checksum_func_t`)(uint8_t iv8, uint8_t *data, int len)

BLUFI checksum.

Param iv8 : initial vector(8bit), normally, blufi core will input packet sequence number
Param data : data need to checksum
Param len : length of data

Enumerations

enum `esp_blufi_cb_event_t`

Values:

enumerator `ESP_BLUFI_EVENT_INIT_FINISH`

enumerator `ESP_BLUFI_EVENT_DEINIT_FINISH`

enumerator `ESP_BLUFI_EVENT_SET_WIFI_OPMODE`

enumerator `ESP_BLUFI_EVENT_BLE_CONNECT`

enumerator `ESP_BLUFI_EVENT_BLE_DISCONNECT`

enumerator `ESP_BLUFI_EVENT_REQ_CONNECT_TO_AP`

enumerator `ESP_BLUFI_EVENT_REQ_DISCONNECT_FROM_AP`

enumerator `ESP_BLUFI_EVENT_GET_WIFI_STATUS`

enumerator `ESP_BLUFI_EVENT_DEAUTHENTICATE_STA`

enumerator `ESP_BLUFI_EVENT_RECV_STA_BSSID`

enumerator `ESP_BLUFI_EVENT_RECV_STA_SSID`

enumerator `ESP_BLUFI_EVENT_RECV_STA_PASSWD`

enumerator `ESP_BLUFI_EVENT_RECV_SOFTAP_SSID`

enumerator `ESP_BLUFI_EVENT_RECV_SOFTAP_PASSWD`

enumerator `ESP_BLUFI_EVENT_RECV_SOFTAP_MAX_CONN_NUM`

enumerator `ESP_BLUFI_EVENT_RECV_SOFTAP_AUTH_MODE`

enumerator `ESP_BLUFI_EVENT_RECV_SOFTAP_CHANNEL`

enumerator `ESP_BLUFI_EVENT_RECV_USERNAME`

enumerator **ESP_BLUFI_EVENT_RECV_CA_CERT**

enumerator **ESP_BLUFI_EVENT_RECV_CLIENT_CERT**

enumerator **ESP_BLUFI_EVENT_RECV_SERVER_CERT**

enumerator **ESP_BLUFI_EVENT_RECV_CLIENT_PRIV_KEY**

enumerator **ESP_BLUFI_EVENT_RECV_SERVER_PRIV_KEY**

enumerator **ESP_BLUFI_EVENT_RECV_SLAVE_DISCONNECT_BLE**

enumerator **ESP_BLUFI_EVENT_GET_WIFI_LIST**

enumerator **ESP_BLUFI_EVENT_REPORT_ERROR**

enumerator **ESP_BLUFI_EVENT_RECV_CUSTOM_DATA**

enum **esp_blufi_sta_conn_state_t**

BLUFI config status.

Values:

enumerator **ESP_BLUFI_STA_CONN_SUCCESS**

enumerator **ESP_BLUFI_STA_CONN_FAIL**

enumerator **ESP_BLUFI_STA_CONNECTING**

enumerator **ESP_BLUFI_STA_NO_IP**

enum **esp_blufi_init_state_t**

BLUFI init status.

Values:

enumerator **ESP_BLUFI_INIT_OK**

enumerator **ESP_BLUFI_INIT_FAILED**

enum **esp_blufi_deinit_state_t**

BLUFI deinit status.

Values:

enumerator **ESP_BLUFI_DEINIT_OK**

enumerator **ESP_BLUFI_DEINIT_FAILED**

enum **esp_blufi_error_state_t**

Values:

enumerator **ESP_BLUFI_SEQUENCE_ERROR**

enumerator **ESP_BLUFI_CHECKSUM_ERROR**

enumerator **ESP_BLUFI_DECRYPT_ERROR**

enumerator **ESP_BLUFI_ENCRYPT_ERROR**

enumerator **ESP_BLUFI_INIT_SECURITY_ERROR**

enumerator **ESP_BLUFI_DH_MALLOC_ERROR**

enumerator **ESP_BLUFI_DH_PARAM_ERROR**

enumerator **ESP_BLUFI_READ_PARAM_ERROR**

enumerator **ESP_BLUFI_MAKE_PUBLIC_ERROR**

enumerator **ESP_BLUFI_DATA_FORMAT_ERROR**

enumerator **ESP_BLUFI_CALC_MD5_ERROR**

enumerator **ESP_BLUFI_WIFI_SCAN_FAIL**

enumerator **ESP_BLUFI_MSG_STATE_ERROR**

2.3.3 Controller && VHCI

Application Example

Check bluetooth/hci folder in ESP-IDF examples, which contains the following application:

- This is a Bluetooth® Low Energy (Bluetooth LE) advertising demo with virtual HCI interface. Send Reset/ADV_PARAM/ADV_DATA/ADV_ENABLE HCI command for Bluetooth Low Energy advertising - bluetooth/hci/controller_vhci_ble_adv.

Please note that ESP32-S3 shares the same bt/include/esp32c3/include/esp_bt.h and bt/controller/esp32c3/bt.c files with ESP32-C3.

API Reference

Header File

- components/bt/include/esp32c3/include/esp_bt.h
- This header file can be included with:

```
#include "esp_bt.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_tx_power_set** (*esp_ble_power_type_t* power_type, *esp_power_level_t* power_level)

Set BLE TX power Connection Tx power should only be set after connection created.

Parameters
- **power_type** -- : The type of which tx power, could set Advertising/Connection/Default and etc
- **power_level** -- Power level(index) corresponding to absolute value(dbm)

Returns ESP_OK - success, other - failed

esp_power_level_t **esp_ble_tx_power_get** (*esp_ble_power_type_t* power_type)

Get BLE TX power Connection Tx power should only be get after connection created.

Parameters **power_type** -- : The type of which tx power, could set Advertising/Connection/Default and etc

Returns >= 0 - Power level, < 0 - Invalid

esp_err_t **esp_bt_controller_init** (*esp_bt_controller_config_t* *cfg)

Initialize BT controller to allocate task and other resource. This function should be called only once, before any other BT functions are called.

Parameters **cfg** -- Initial configuration of BT controller. Different from previous version, there's a mode and some connection configuration in "cfg" to configure controller work mode and allocate the resource which is needed.

Returns ESP_OK - success, other - failed

esp_err_t **esp_bt_controller_deinit** (void)

De-initialize BT controller to free resource and delete task. You should stop advertising and scanning, as well as disconnect all existing connections before de-initializing BT controller.

This function should be called only once, after any other BT functions are called. This function is not whole completed, esp_bt_controller_init cannot called after this function.

Returns ESP_OK - success, other - failed

esp_err_t **esp_bt_controller_enable** (*esp_bt_mode_t* mode)

Enable BT controller. Due to a known issue, you cannot call esp_bt_controller_enable() a second time to change the controller mode dynamically. To change controller mode, call esp_bt_controller_disable() and then call esp_bt_controller_enable() with the new mode.

Parameters **mode** -- : the mode(BLE/BT/BTDM) to enable. For compatible of API, retain this argument. This mode must be equal as the mode in "cfg" of esp_bt_controller_init().

Returns ESP_OK - success, other - failed

esp_err_t **esp_bt_controller_disable** (void)

Disable BT controller.

Returns ESP_OK - success, other - failed

esp_bt_controller_status_t **esp_bt_controller_get_status** (void)

Get BT controller is initialised/de-initialised/enabled/disabled.

Returns status value

uint16_t **esp_bt_get_tx_buf_num**(void)

bool **esp_vhci_host_check_send_available**(void)

esp_vhci_host_check_send_available used for check actively if the host can send packet to controller or not.

Returns true for ready to send, false means cannot send packet

void **esp_vhci_host_send_packet**(uint8_t *data, uint16_t len)

esp_vhci_host_send_packet host send packet to controller

Should not call this function from within a critical section or when the scheduler is suspended.

Parameters
- **data** -- the packet point
- **len** -- the packet length

esp_err_t **esp_vhci_host_register_callback**(const *esp_vhci_host_callback_t* *callback)

esp_vhci_host_register_callback register the vhci reference callback struct defined by vhci_host_callback structure.

Parameters **callback** -- *esp_vhci_host_callback* type variable
Returns ESP_OK - success, ESP_FAIL - failed

esp_err_t **esp_bt_controller_mem_release**(*esp_bt_mode_t* mode)

esp_bt_controller_mem_release release the controller memory as per the mode

This function releases the BSS, data and other sections of the controller to heap. The total size is about 70k bytes.

esp_bt_controller_mem_release(mode) should be called only before esp_bt_controller_init() or after esp_bt_controller_deinit().

Note that once BT controller memory is released, the process cannot be reversed. It means you cannot use the bluetooth mode which you have released by this function.

If your firmware will later upgrade the Bluetooth controller mode (BLE -> BT Classic or disabled -> enabled) then do not call this function.

If the app calls esp_bt_controller_enable(ESP_BT_MODE_BLE) to use BLE only then it is safe to call esp_bt_controller_mem_release(ESP_BT_MODE_CLASSIC_BT) at initialization time to free unused BT Classic memory.

If the mode is ESP_BT_MODE_BTDM, then it may be useful to call API esp_bt_mem_release(ESP_BT_MODE_BTDM) instead, which internally calls esp_bt_controller_mem_release(ESP_BT_MODE_BTDM) and additionally releases the BSS and data consumed by the BT/BLE host stack to heap. For more details about usage please refer to the documentation of esp_bt_mem_release() function

Parameters **mode** -- : the mode want to release memory
Returns ESP_OK - success, other - failed

esp_err_t **esp_bt_mem_release**(*esp_bt_mode_t* mode)

esp_bt_mem_release release controller memory and BSS and data section of the BT/BLE host stack as per the mode

This function first releases controller memory by internally calling esp_bt_controller_mem_release(). Additionally, if the mode is set to ESP_BT_MODE_BTDM, it also releases the BSS and data consumed by the BT/BLE host stack to heap

Note that once BT memory is released, the process cannot be reversed. It means you cannot use the bluetooth mode which you have released by this function.

If your firmware will later upgrade the Bluetooth controller mode (BLE -> BT Classic or disabled -> enabled) then do not call this function.

If you never intend to use bluetooth in a current boot-up cycle, you can call esp_bt_mem_release(ESP_BT_MODE_BTDM) before esp_bt_controller_init or after esp_bt_controller_deinit.

For example, if a user only uses bluetooth for setting the WiFi configuration, and does not use bluetooth in the rest of the product operation". In such cases, after receiving the WiFi configuration, you can disable/deinit bluetooth and release its memory. Below is the sequence of APIs to be called for such scenarios:

```
esp_bluedroid_disable();
esp_bluedroid_deinit();
esp_bt_controller_disable();
esp_bt_controller_deinit();
esp_bt_mem_release(ESP_BT_MODE_BTDM);
```

Parameters **mode** -- : the mode whose memory is to be released
Returns ESP_OK - success, other - failed

esp_err_t **esp_bt_sleep_enable**(void)

enable bluetooth to enter modem sleep

Note that this function shall not be invoked before esp_bt_controller_enable()

There are currently two options for bluetooth modem sleep, one is ORIG mode, and another is EVED Mode. EVED Mode is intended for BLE only.

For ORIG mode: Bluetooth modem sleep is enabled in controller start up by default if CONFIG_BTDM_CONTROLLER_MODEM_SLEEP is set and "ORIG mode" is selected. In ORIG modem sleep mode, bluetooth controller will switch off some components and pause to work every now and then, if there is no event to process; and wakeup according to the scheduled interval and resume the work. It can also wakeup earlier upon external request using function "esp_bt_controller_wakeup_request".

Returns
- ESP_OK : success
- other : failed

esp_err_t **esp_bt_sleep_disable**(void)

disable bluetooth modem sleep

Note that this function shall not be invoked before esp_bt_controller_enable()

If esp_bt_sleep_disable() is called, bluetooth controller will not be allowed to enter modem sleep;

If ORIG modem sleep mode is in use, if this function is called, bluetooth controller may not immediately wake up if it is dormant then. In this case, esp_bt_controller_wakeup_request() can be used to shorten the time for wakeup.

Returns
- ESP_OK : success
- other : failed

bool **esp_bt_controller_is_sleeping**(void)

to check whether bluetooth controller is sleeping at the instant, if modem sleep is enabled

Note that this function shall not be invoked before esp_bt_controller_enable() This function is supposed to be used ORIG mode of modem sleep

Returns true if in modem sleep state, false otherwise

void **esp_bt_controller_wakeup_request**(void)

request controller to wakeup from sleeping state during sleep mode

Note that this function shall not be invoked before esp_bt_controller_enable() Note that this function is supposed to be used ORIG mode of modem sleep Note that after this request, bluetooth controller may again enter sleep as long as the modem sleep is enabled

Profiling shows that it takes several milliseconds to wakeup from modem sleep after this request. Generally it takes longer if 32kHz XTAL is used than the main XTAL, due to the lower frequency of the former as the bluetooth low power clock source.

int **esp_bt_h4tl_eif_io_event_notify**(int event)

notify bluetooth controller task to process the event upon Tx or Rx done

Note that this function shall not be invoked before esp_bt_controller_enable() This function can be called in both ISR and non-ISR context

void **esp_wifi_bt_power_domain_on**(void)

bt Wi-Fi power domain power on

void **esp_wifi_bt_power_domain_off**(void)

bt Wi-Fi power domain power off

esp_bt_sleep_clock_t **esp_bt_get_lpclk_src**(void)

Get the Bluetooth module sleep clock source.

Note that this function shall not be invoked before esp_bt_controller_init()

Returns clock source used in Bluetooth low power mode

Structures

struct **esp_bt_hci_tl_t**

Controller HCI transport layer function structure This structure shall be registered when HCI transport layer is UART.

Public Members

uint32_t **_magic**

Magic number

uint32_t **_version**

Version number of the defined structure

uint32_t **_reserved**

Reserved for future use

int (***_open**)(void)

HCI transport layer open function

void (***_close**)(void)

HCI transport layer close function

void (***_finish_transfers**)(void)

HCI transport layer finish transfers function

void (***_recv**)(uint8_t *buf, uint32_t len, *esp_bt_hci_tl_callback_t* callback, void *arg)

HCI transport layer receive function

void (***_send**)(uint8_t *buf, uint32_t len, *esp_bt_hci_tl_callback_t* callback, void *arg)

HCI transport layer send function

bool (*__flow_off__)(void)

HCI transport layer flow off function

void (*__flow_on__)(void)

HCI transport layer flow on function

struct **esp_bt_controller_config_t**

Controller config options, depend on config mask. Config mask indicate which functions enabled, this means some options or parameters of some functions enabled by config mask.

Public Members

uint32_t **magic**

Magic number

uint32_t **version**

version number of the defined structure

uint16_t **controller_task_stack_size**

Bluetooth controller task stack size

uint8_t **controller_task_prio**

Bluetooth controller task priority

uint8_t **controller_task_run_cpu**

CPU num that Bluetooth controller task runs on

uint8_t **bluetooth_mode**

Controller mode: BR/EDR, BLE or Dual Mode

uint8_t **ble_max_act**

BLE maximum number of air activities

uint8_t **sleep_mode**

controller sleep mode

uint8_t **sleep_clock**

controller sleep clock

uint8_t **ble_st_acl_tx_buf_nb**

controller static ACL TX BUFFER number

uint8_t **ble_hw_cca_check**

controller hardware triggered CCA check

uint16_t **ble_adv_dup_filt_max**

maximum number of duplicate scan filter

bool **coex_param_en**

deprecated

uint8_t **ce_len_type**

connection event length computation method

bool **coex_use_hooks**

deprecated

uint8_t **hci_tl_type**

HCI transport layer, UART, VHCI, etc

esp_bt_hci_tl_t ***hci_tl_funcs**

hci transport functions used, must be set when hci_tl_type is UART

uint8_t **txant_dft**

default Tx antenna

uint8_t **rxant_dft**

default Rx antenna

uint8_t **txpwr_dft**

default Tx power

uint32_t **cfg_mask**

Configuration mask to set specific options

uint8_t **scan_duplicate_mode**

scan duplicate mode

uint8_t **scan_duplicate_type**

scan duplicate type

uint16_t **normal_adv_size**

Normal adv size for scan duplicate

uint16_t **mesh_adv_size**

Mesh adv size for scan duplicate

uint8_t **coex_phy_coded_tx_rx_time_limit**

limit on max tx/rx time in case of connection using CODED-PHY with Wi-Fi coexistence

uint32_t **hw_target_code**

hardware target

uint8_t **slave_ce_len_min**

slave minimum ce length

uint8_t **hw_recorrect_en**

Hardware re-correction enabled

uint8_t **cca_thresh**

cca threshold

uint16_t **scan_backoff_upperlimitmax**

scan backoff upperlimitmax value

uint16_t **dup_list_refresh_period**

duplicate scan list refresh time

bool **ble_50_feat_supp**

BLE 5.0 feature support

uint8_t **ble_cca_mode**

BLE CCA mode

uint8_t **ble_data_lenth_zero_aux**

Config ext adv aux option

uint8_t **ble_chan_ass_en**

BLE channel assessment enable

uint8_t **ble_ping_en**

BLE ping procedure enable

struct **esp_vhci_host_callback**

esp_vhci_host_callback used for vhci call host function to notify what host need to do

Public Members

void (*__notify_host_send_available__)(void)

callback used to notify that the host can send packet to controller

int (*__notify_host_recv__)(uint8_t *data, uint16_t len)

callback used to notify that the controller has a packet to send to the host

Macros

ESP_BT_CTRL_CONFIG_MAGIC_VAL

ESP_BT_CTRL_CONFIG_VERSION

ESP_BT_HCI_TL_MAGIC_VALUE

ESP_BT_HCI_TL_VERSION

ESP_BT_HCI_TL_STATUS_OK

HCI_TL Tx/Rx operation status OK

BT_CONTROLLER_INIT_CONFIG_DEFAULT()

Type Definitions

typedef void (***esp_bt_hci_tl_callback_t**)(void *arg, uint8_t status)

callback function for HCI Transport Layer send/receive operations

typedef struct *esp_vhci_host_callback* **esp_vhci_host_callback_t**

esp_vhci_host_callback used for vhci call host function to notify what host need to do

Enumerations

enum **esp_bt_mode_t**

Bluetooth mode for controller enable/disable.

Values:

enumerator **ESP_BT_MODE_IDLE**

Bluetooth is not running

enumerator **ESP_BT_MODE_BLE**

Run BLE mode

enumerator **ESP_BT_MODE_CLASSIC_BT**

Run Classic BT mode

enumerator **ESP_BT_MODE_BTDM**

Run dual mode

enum **esp_bt_ctrl_hci_tl_t**

Type of controller HCI transport layer.

Values:

enumerator **ESP_BT_CTRL_HCI_TL_UART**

HCI UART h4 transport layer

enumerator **ESP_BT_CTRL_HCI_TL_VHCI**

VHCI interface

enum **esp_ble_ce_len_t**

type of BLE connection event length computation

Values:

enumerator **ESP_BLE_CE_LEN_TYPE_ORIG**

original

enumerator **ESP_BLE_CE_LEN_TYPE_CE**

use CE_LEN parameter from HCI commands

enumerator **ESP_BLE_CE_LEN_TYPE_SD**

Espressif vendor defined

enum **esp_bt_sleep_mode_t**

Bluetooth sleep mode.

Values:

enumerator **ESP_BT_SLEEP_MODE_NONE**

Bluetooth sleep mode disabled

enumerator **ESP_BT_SLEEP_MODE_1**

Bluetooth sleep mode 1

enum **esp_bt_sleep_clock_t**

Bluetooth sleep clock.

Values:

enumerator **ESP_BT_SLEEP_CLOCK_NONE**

Sleep clock not configured

enumerator **ESP_BT_SLEEP_CLOCK_MAIN_XTAL**

SoC main crystal

enumerator **ESP_BT_SLEEP_CLOCK_EXT_32K_XTAL**

External 32.768kHz crystal

enumerator **ESP_BT_SLEEP_CLOCK_RTC_SLOW**

Internal 136kHz RC oscillator

enumerator **ESP_BT_SLEEP_CLOCK_FPGA_32K**

Hardwired 32KHz clock temporarily used for FPGA

enum **[anonymous]**

antenna index used for bluetooth

Values:

enumerator **ESP_BT_ANT_IDX_0**

anntena NO 0

enumerator **ESP_BT_ANT_IDX_1**

anntena NO 1

enum **[anonymous]**

Maximum Tx/Rx time limit on Coded-PHY connection.

Values:

enumerator **ESP_BT_COEX_PHY_CODED_TX_RX_TIME_LIMIT_FORCE_DISABLE**

Disable the limit

enumerator **ESP_BT_COEX_PHY_CODED_TX_RX_TIME_LIMIT_FORCE_ENABLE**

Always Enable the limit

enum **esp_bt_controller_status_t**

Bluetooth controller enable/disable/initialised/de-initialised status.

Values:

enumerator **ESP_BT_CONTROLLER_STATUS_IDLE**

enumerator **ESP_BT_CONTROLLER_STATUS_INITED**

enumerator **ESP_BT_CONTROLLER_STATUS_ENABLED**

enumerator **ESP_BT_CONTROLLER_STATUS_NUM**

enum **esp_ble_power_type_t**

BLE tx power type ESP_BLE_PWR_TYPE_CONN_HDL0-8: for each connection, and only be set after connection completed. when disconnect, the correspond TX power is not effected. ESP_BLE_PWR_TYPE_ADV : for advertising/scan response. ESP_BLE_PWR_TYPE_SCAN : for scan. ESP_BLE_PWR_TYPE_DEFAULT : if each connection's TX power is not set, it will use this default value. if neither in scan mode nor in adv mode, it will use this default value. If none of power type is set, system will use ESP_PWR_LVL_P3 as default for ADV/SCAN/CONN0-9.

Values:

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL0**

For connection handle 0

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL1**

For connection handle 1

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL2**

For connection handle 2

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL3**

For connection handle 3

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL4**

For connection handle 4

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL5**

For connection handle 5

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL6**

For connection handle 6

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL7**

For connection handle 7

enumerator **ESP_BLE_PWR_TYPE_CONN_HDL8**

For connection handle 8

enumerator **ESP_BLE_PWR_TYPE_ADV**

For advertising

enumerator **ESP_BLE_PWR_TYPE_SCAN**

For scan

enumerator **ESP_BLE_PWR_TYPE_DEFAULT**

For default, if not set other, it will use default value

enumerator **ESP_BLE_PWR_TYPE_NUM**

TYPE numbers

enum **esp_power_level_t**

Bluetooth TX power level(index), it's just a index corresponding to power(dbm).

Values:

enumerator **ESP_PWR_LVL_N24**

Corresponding to -24dbm

enumerator **ESP_PWR_LVL_N21**

Corresponding to -21dbm

enumerator **ESP_PWR_LVL_N18**

Corresponding to -18dbm

enumerator **ESP_PWR_LVL_N15**

Corresponding to -15dbm

enumerator **ESP_PWR_LVL_N12**

Corresponding to -12dbm

enumerator **ESP_PWR_LVL_N9**

Corresponding to -9dbm

enumerator **ESP_PWR_LVL_N6**

Corresponding to -6dbm

enumerator **ESP_PWR_LVL_N3**

Corresponding to -3dbm

enumerator **ESP_PWR_LVL_N0**

Corresponding to 0dbm

enumerator **ESP_PWR_LVL_P3**

Corresponding to +3dbm

enumerator **ESP_PWR_LVL_P6**

Corresponding to +6dbm

enumerator **ESP_PWR_LVL_P9**

Corresponding to +9dbm

enumerator **ESP_PWR_LVL_P12**

Corresponding to +12dbm

enumerator **ESP_PWR_LVL_P15**

Corresponding to +15dbm

enumerator **ESP_PWR_LVL_P18**

Corresponding to +18dbm

enumerator **ESP_PWR_LVL_P21**

Corresponding to +21dbm

enumerator **ESP_PWR_LVL_INVALID**

Indicates an invalid value

2.3.4 ESP-BLE-MESH

Note: The current ESP-BLE-MESH v1.1 related code is a preview version, so the Mesh Protocol v1.1 related Structures, MACROs, and APIs involved in the code may be changed.

With various features of ESP-BLE-MESH, users can create a managed flooding mesh network for several scenarios, such as lighting, sensor and etc.

For an ESP32 to join and work on a ESP-BLE-MESH network, it must be provisioned firstly. By provisioning, the ESP32, as an unprovisioned device, will join the ESP-BLE-MESH network and become a ESP-BLE-MESH node, communicating with other nodes within or beyond the radio range.

Apart from ESP-BLE-MESH nodes, inside ESP-BLE-MESH network, there is also ESP32 that works as ESP-BLE-MESH provisioner, which could provision unprovisioned devices into ESP-BLE-MESH nodes and configure the nodes with various features.

For information how to start using ESP32 and ESP-BLE-MESH, please see the Section *Getting Started with ESP-BLE-MESH*. If you are interested in information on ESP-BLE-MESH architecture, including some details of software implementation, please see Section *ESP-BLE-MESH Architecture*.

Application Examples and Demos

Please refer to Sections *ESP-BLE-MESH Examples* and *ESP-BLE-MESH Demo Videos*.

API Reference

ESP-BLE-MESH APIs are divided into the following parts:

- *ESP-BLE-MESH Definitions*
- *ESP-BLE-MESH Core API Reference*
- *ESP-BLE-MESH Models API Reference*

- *ESP-BLE-MESH (v1.1) Core API Reference*

ESP-BLE-MESH Definitions

This section contains only one header file, which lists the following items of ESP-BLE-MESH.

- ID of all the models and related message opcodes
- Structs of model, element and Composition Data
- Structs of used by ESP-BLE-MESH Node/Provisioner for provisioning
- Structs used to transmit/receive messages
- Event types and related event parameters

Header File

- components/bt/esp_ble_mesh/api/esp_ble_mesh_defs.h
- This header file can be included with:

```
#include "esp_ble_mesh_defs.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Unions

union **esp_ble_mesh_prov_cb_param_t**

#include <esp_ble_mesh_defs.h> BLE Mesh Node/Provisioner callback parameters union.

Public Members

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_prov_register_comp_param* **prov_register_comp**

Event parameter of ESP_BLE_MESH_PROV_REGISTER_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_set_unprov_dev_name_comp_param* **node_set_unprov_dev_name_comp**

Event parameter of ESP_BLE_MESH_NODE_SET_UNPROV_DEV_NAME_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_prov_enable_comp_param* **node_prov_enable_comp**

Event parameter of ESP_BLE_MESH_NODE_PROV_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_prov_disable_comp_param* **node_prov_disable_comp**

Event parameter of ESP_BLE_MESH_NODE_PROV_DISABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_link_open_evt_param* **node_prov_link_open**

Event parameter of ESP_BLE_MESH_NODE_PROV_LINK_OPEN_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_link_close_evt_param* **node_prov_link_close**

Event parameter of ESP_BLE_MESH_NODE_PROV_LINK_CLOSE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_output_num_evt_param* **node_prov_output_num**

Event parameter of ESP_BLE_MESH_NODE_PROV_OUTPUT_NUMBER_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_output_str_evt_param* **node_prov_output_str**

Event parameter of ESP_BLE_MESH_NODE_PROV_OUTPUT_STRING_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_input_evt_param* **node_prov_input**

Event parameter of ESP_BLE_MESH_NODE_PROV_INPUT_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provision_complete_evt_param* **node_prov_complete**

Event parameter of ESP_BLE_MESH_NODE_PROV_COMPLETE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provision_reset_param* **node_prov_reset**

Event parameter of ESP_BLE_MESH_NODE_PROV_RESET_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_set_oob_pub_key_comp_param* **node_prov_set_oob_pub_key_comp**

Event parameter of ESP_BLE_MESH_NODE_PROV_SET_OOB_PUB_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_input_number_comp_param* **node_prov_input_num_comp**

Event parameter of ESP_BLE_MESH_NODE_PROV_INPUT_NUM_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_input_string_comp_param* **node_prov_input_str_comp**

Event parameter of ESP_BLE_MESH_NODE_PROV_INPUT_STR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_identity_enable_comp_param* **node_proxy_identity_enable_comp**

Event parameter of ESP_BLE_MESH_NODE_PROXY_IDENTITY_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_gatt_enable_comp_param* **node_proxy_gatt_enable_comp**

Event parameter of ESP_BLE_MESH_NODE_PROXY_GATT_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_gatt_disable_comp_param* **node_proxy_gatt_disable_comp**

Event parameter of ESP_BLE_MESH_NODE_PROXY_GATT_DISABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_private_identity_enable_comp_param* **node_private_proxy_identity_enable_comp**

Event parameter of ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_private_identity_disable_comp_param* **node_private_proxy_identity_disable_comp**

Event parameter of ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_DISABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_node_add_local_net_key_comp_param*
node_add_net_key_comp

Event parameter of ESP_BLE_MESH_NODE_ADD_LOCAL_NET_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_node_add_local_app_key_comp_param*
node_add_app_key_comp

Event parameter of ESP_BLE_MESH_NODE_ADD_LOCAL_APP_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_node_bind_local_mod_app_comp_param*
node_bind_app_key_to_model_comp

Event parameter of ESP_BLE_MESH_NODE_BIND_APP_KEY_TO_MODEL_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_recv_unprov_adv_pkt_param*
provisioner_recv_unprov_adv_pkt

Event parameter of ESP_BLE_MESH_PROVISIONER_RECV_UNPROV_ADV_PKT_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_enable_comp_param*
provisioner_prov_enable_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_disable_comp_param*
provisioner_prov_disable_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_DISABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_link_open_evt_param*
provisioner_prov_link_open

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_LINK_OPEN_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_read_oob_pub_key_evt_param*
provisioner_prov_read_oob_pub_key

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_input_evt_param*
provisioner_prov_input

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_INPUT_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_output_evt_param*
provisioner_prov_output

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_OUTPUT_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_link_close_evt_param*
provisioner_prov_link_close

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_LINK_CLOSE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_comp_param*
provisioner_prov_complete

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_cert_based_prov_start_evt_param*
provisioner_cert_based_prov_start

Event parameter of ESP_BLE_MESH_PROVISIONER_CERT_BASED_PROV_START_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_recv_prov_records_list_evt_param*
recv_provisioner_records_list

Event parameter of ESP_BLE_MESH_PROVISIONER_RECV_PROV_RECORDS_LIST_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_record_recv_comp_evt_param*
provisioner_prov_record_recv_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_RECORD_RECV_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_send_prov_records_get_evt_param*
provisioner_send_records_get

Event parameter of ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORDS_GET_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_send_prov_record_req_evt_param*
provisioner_send_record_req

Event parameter of ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORD_REQUEST_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_send_prov_invite_evt_param*
provisioner_send_prov_invite

Event parameter of ESP_BLE_MESH_PROVISIONER_SEND_PROV_INVITE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_send_link_close_evt_param*
provisioner_send_link_close

Event parameter of ESP_BLE_MESH_PROVISIONER_SEND_LINK_CLOSE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_add_unprov_dev_comp_param*
provisioner_add_unprov_dev_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_ADD_UNPROV_DEV_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_dev_with_addr_comp_param*
provisioner_prov_dev_with_addr_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_DEV_WITH_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_delete_dev_comp_param*
provisioner_delete_dev_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_DELETE_DEV_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_set_dev_uuid_match_comp_param*
provisioner_set_dev_uuid_match_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_SET_DEV_UUID_MATCH_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_set_prov_data_info_comp_param*
provisioner_set_prov_data_info_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_SET_PROV_DATA_INFO_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_set_static_oob_val_comp_param*
provisioner_set_static_oob_val_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_SET_STATIC_OOB_VALUE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_set_primary_elem_addr_comp_param*
provisioner_set_primary_elem_addr_comp

Event parameter of ESP_BLE_MESH_PROVISIONER_SET_PRIMARY_ELEM_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_read_oob_pub_key_comp_param* **provisioner_prov_read_oob_pub_key_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_input_num_comp_param* **provisioner_prov_input_num_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_INPUT_NUMBER_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_input_str_comp_param* **provisioner_prov_input_str_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_PROV_INPUT_STRING_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_set_node_name_comp_param* **provisioner_set_node_name_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_SET_NODE_NAME_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_add_local_app_key_comp_param* **provisioner_add_app_key_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_APP_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_update_local_app_key_comp_param* **provisioner_update_app_key_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_APP_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_bind_local_mod_app_comp_param* **provisioner_bind_app_key_to_model_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_BIND_APP_KEY_TO_MODEL_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_add_local_net_key_comp_param* **provisioner_add_net_key_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_NET_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_update_local_net_key_comp_param* **provisioner_update_net_key_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_NET_KEY_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_store_node_comp_data_comp_param* **provisioner_store_node_comp_data_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_STORE_NODE_COMP_DATA_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_delete_node_with_uuid_comp_param* **provisioner_delete_node_with_uuid_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_UUID_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_delete_node_with_addr_comp_param* **provisioner_delete_node_with_addr_comp**

Event parameter of ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_ADDR_COMP_EVT

int **err_code**

Indicate the result of enabling/disabling to receive heartbeat messages by the Provisioner

Indicate the result of setting the heartbeat filter type by the Provisioner

Indicate the result of setting the heartbeat filter address by the Provisioner

Indicate the result of directly erasing settings by the Provisioner

Indicate the result of opening settings with index by the Provisioner

Indicate the result of opening settings with user id by the Provisioner

Indicate the result of closing settings with index by the Provisioner

Indicate the result of closing settings with user id by the Provisioner

Indicate the result of deleting settings with index by the Provisioner

Indicate the result of deleting settings with user id by the Provisioner

Indicate the result of Proxy Client send Solicitation PDU

bool **enable**

Indicate enabling or disabling receiving heartbeat messages

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_enable_heartbeat_recv_comp

ESP_BLE_MESH_PROVISIONER_ENABLE_HEARTBEAT_RECV_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_ENABLE_HEARTBEAT_RECV_COMP_EVT

uint8_t **type**

Type of the filter used for receiving heartbeat messages

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_set_heartbeat_filter_type_comp

ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_TYPE_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_TYPE_COMP_EVT

uint8_t **op**

Operation (add, remove, clean)

uint16_t **hb_src**

Heartbeat source address

uint16_t **hb_dst**

Heartbeat destination address

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_set_heartbeat_filter_info_comp

ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_INFO_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_INFO_COMP_EVT

uint8_t **init_ttl**

Heartbeat InitTTL

uint8_t **rx_ttl**

Heartbeat RxTTL

uint8_t **hops**

Heartbeat hops (InitTTL - RxTTL + 1)

uint16_t **feature**

Bit field of currently active features of the node

int8_t **rssi**

RSSI of the heartbeat message

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous] **provisioner_recv_heartbeat**

ESP_BLE_MESH_PROVISIONER_RECV_HEARTBEAT_MESSAGE_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_RECV_HEARTBEAT_MESSAGE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_direct_erase_settings_comp

ESP_BLE_MESH_PROVISIONER_DIRECT_ERASE_SETTINGS_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_DIRECT_ERASE_SETTINGS_COMP_EVT

uint8_t **index**

Index of Provisioner settings

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_open_settings_with_index_comp

ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_INDEX_COMP_EVT.

Event parameter of ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_INDEX_COMP_EVT

char **uid**[ESP_BLE_MESH_SETTINGS_UID_SIZE + 1]

Provisioner settings user id

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_open_settings_with_uid_comp

ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_UID_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_UID_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_close_settings_with_index_comp

ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_INDEX_COMP_EVT.

Event parameter of ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_INDEX_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_close_settings_with_uid_comp

ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_UID_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_UID_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_delete_settings_with_index_comp

ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_INDEX_COMP_EVT.

Event parameter of ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_INDEX_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous]
provisioner_delete_settings_with_uid_comp

ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_UID_COMP_EVT.

Event parameters of ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_UID_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_set_fast_prov_info_comp_param* **set_fast_prov_info_comp**

Event parameter of ESP_BLE_MESH_SET_FAST_PROV_INFO_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_set_fast_prov_action_comp_param* **set_fast_prov_action_comp**

Event parameter of ESP_BLE_MESH_SET_FAST_PROV_ACTION_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_heartbeat_msg_recv_param* **heartbeat_msg_recv**

Event parameter of ESP_BLE_MESH_HEARTBEAT_MESSAGE_RECV_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_lpn_enable_comp_param* **lpn_enable_comp**

Event parameter of ESP_BLE_MESH_LPN_ENABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_lpn_disable_comp_param* **lpn_disable_comp**

Event parameter of ESP_BLE_MESH_LPN_DISABLE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_lpn_poll_comp_param* **lpn_poll_comp**

Event parameter of ESP_BLE_MESH_LPN_POLL_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_lpn_friendship_establish_param* **lpn_friendship_establish**

Event parameter of ESP_BLE_MESH_LPN_FRIENDSHIP_ESTABLISH_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_lpn_friendship_terminate_param* **lpn_friendship_terminate**

Event parameter of ESP_BLE_MESH_LPN_FRIENDSHIP_TERMINATE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_friend_friendship_establish_param* **frnd_friendship_establish**

Event parameter of ESP_BLE_MESH_FRIEND_FRIENDSHIP_ESTABLISH_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_friend_friendship_terminate_param* **frnd_friendship_terminate**

Event parameter of ESP_BLE_MESH_FRIEND_FRIENDSHIP_TERMINATE_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_recv_adv_pkt_param* **proxy_client_recv_adv_pkt**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_RECV_ADV_PKT_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_connected_param* **proxy_client_connected**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_CONNECTED_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_disconnected_param* **proxy_client_disconnected**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_DISCONNECTED_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_recv_filter_status_param* **proxy_client_recv_filter_status**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_RECV_FILTER_STATUS_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_connect_comp_param* **proxy_client_connect_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_CONNECT_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_disconnect_comp_param* **proxy_client_disconnect_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_DISCONNECT_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_set_filter_type_comp_param* **proxy_client_set_filter_type_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_SET_FILTER_TYPE_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_add_filter_addr_comp_param* **proxy_client_add_filter_addr_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_ADD_FILTER_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_remove_filter_addr_comp_param* **proxy_client_remove_filter_addr_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_REMOVE_FILTER_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_client_directed_proxy_set_param* **proxy_client_directed_proxy_set_comp**

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_DIRECTED_PROXY_SET_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_server_connected_param* **proxy_server_connected**

Event parameter of ESP_BLE_MESH_PROXY_SERVER_CONNECTED_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_proxy_server_disconnected_param* **proxy_server_disconnected**

Event parameter of ESP_BLE_MESH_PROXY_SERVER_DISCONNECTED_EVT

uint16_t **net_idx**

Corresponding NetKey Index

uint16_t **ssrc**

Solicitation SRC

uint16_t **dst**

Solicitation DST

struct *esp_ble_mesh_prov_cb_param_t*::[anonymous] **proxy_client_send_solic_pdu_comp**

ESP_BLE_MESH_PROXY_CLIENT_SEND_SOLIC_PDU_COMP_EVT.

Event parameter of ESP_BLE_MESH_PROXY_CLIENT_SEND_SOLIC_PDU_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_model_sub_group_addr_comp_param* **model_sub_group_addr_comp**

Event parameters of ESP_BLE_MESH_MODEL_SUBSCRIBE_GROUP_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_model_unsub_group_addr_comp_param* **model_unsub_group_addr_comp**

Event parameters of ESP_BLE_MESH_MODEL_UNSUBSCRIBE_GROUP_ADDR_COMP_EVT

struct *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_deinit_mesh_comp_param* **deinit_mesh_comp**

Event parameter of ESP_BLE_MESH_DEINIT_MESH_COMP_EVT

struct **ble_mesh_deinit_mesh_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_DEINIT_MESH_COMP_EVT.

Public Members

int **err_code**

Indicate the result of BLE Mesh deinitialization

struct **ble_mesh_friend_friendship_establish_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_FRIEND_FRIENDSHIP_ESTABLISH_EVT.

Public Members

uint16_t **lpn_addr**

Low Power Node unicast address

struct **ble_mesh_friend_friendship_terminate_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_FRIEND_FRIENDSHIP_TERMINATE_EVT.

Public Types

enum **[anonymous]**

This enum value is the reason of friendship termination on the friend node side

Values:

enumerator **ESP_BLE_MESH_FRND_FRIENDSHIP_TERMINATE_ESTABLISH_FAIL**

Friend Offer has been sent, but Friend Offer is not received within 1 second, friendship fails to be established

enumerator **ESP_BLE_MESH_FRND_FRIENDSHIP_TERMINATE_POLL_TIMEOUT**

Friendship is established, PollTimeout timer expires and no Friend Poll/Sub Add/Sub Remove is received

enumerator **ESP_BLE_MESH_FRND_FRIENDSHIP_TERMINATE_RECV_FRND_REQ**

Receive Friend Request from existing Low Power Node

enumerator **ESP_BLE_MESH_FRND_FRIENDSHIP_TERMINATE_RECV_FRND_CLEAR**

Receive Friend Clear from other friend node

enumerator **ESP_BLE_MESH_FRND_FRIENDSHIP_TERMINATE_DISABLE**

Friend feature disabled or corresponding NetKey is deleted

Public Members

uint16_t **lpn_addr**

Low Power Node unicast address

enum *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_friend_friendship_terminate_param*::*[anonymous]* **reason**

This enum value is the reason of friendship termination on the friend node side Friendship terminated reason

struct **ble_mesh_heartbeat_msg_recv_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_HEARTBEAT_MESSAGE_RECV_EVT.

Public Members

uint8_t **hops**

Heartbeat hops (InitTTL - RxTTL + 1)

uint16_t **feature**

Bit field of currently active features of the node

struct **ble_mesh_input_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_INPUT_EVT.

Public Members

esp_ble_mesh_input_action_t **action**

Action of Input OOB Authentication

uint8_t **size**

Size of Input OOB Authentication

struct **ble_mesh_input_number_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_INPUT_NUM_COMP_EVT.

Public Members

int **err_code**

Indicate the result of inputting number

struct **ble_mesh_input_string_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_INPUT_STR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of inputting string

struct **ble_mesh_link_close_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_LINK_CLOSE_EVT.

Public Members

esp_ble_mesh_prov_bearer_t **bearer**

Type of the bearer used when device link is closed

uint8_t **reason**

Reason of the closed provisioning link

struct **ble_mesh_link_open_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_LINK_OPEN_EVT.

Public Members

esp_ble_mesh_prov_bearer_t **bearer**

Type of the bearer used when device link is open

struct **ble_mesh_lpn_disable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_LPN_DISABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of disabling LPN functionality

struct **ble_mesh_lpn_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_LPN_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling LPN functionality

struct **ble_mesh_lpn_friendship_establish_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_LPN_FRIENDSHIP_ESTABLISH_EVT.

Public Members

uint16_t **friend_addr**

Friend Node unicast address

struct **ble_mesh_lpn_friendship_terminate_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_LPN_FRIENDSHIP_TERMINATE_EVT.

Public Members

uint16_t **friend_addr**

Friend Node unicast address

struct **ble_mesh_lpn_poll_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_LPN_POLL_COMP_EVT.

Public Members

int **err_code**

Indicate the result of sending Friend Poll

struct **ble_mesh_model_sub_group_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_SUBSCRIBE_GROUP_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of local model subscribing group address

uint16_t **element_addr**

Element address

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

uint16_t **group_addr**

Group Address

struct **ble_mesh_model_unsub_group_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_UNSUBSCRIBE_GROUP_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of local model unsubscribing group address

uint16_t **element_addr**

Element address

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

uint16_t **group_addr**

Group Address

struct **ble_mesh_node_add_local_app_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_ADD_LOCAL_APP_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of adding local AppKey by the node

uint16_t **net_idx**

NetKey Index

uint16_t **app_idx**

AppKey Index

struct **ble_mesh_node_add_local_net_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_ADD_LOCAL_NET_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of adding local NetKey by the node

uint16_t **net_idx**

NetKey Index

struct **ble_mesh_node_bind_local_mod_app_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_BIND_APP_KEY_TO_MODEL_COMP_EVT.

Public Members

int **err_code**

Indicate the result of binding AppKey with model by the node

uint16_t **element_addr**

Element address

uint16_t **app_idx**

AppKey Index

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **ble_mesh_output_num_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_OUTPUT_NUMBER_EVT.

Public Members

esp_ble_mesh_output_action_t **action**

Action of Output OOB Authentication

uint32_t **number**

Number of Output OOB Authentication

struct **ble_mesh_output_str_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_OUTPUT_STRING_EVT.

Public Members

char **string**[8]

String of Output OOB Authentication

struct **ble_mesh_prov_disable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_DISABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of disabling BLE Mesh device

struct **ble_mesh_prov_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling BLE Mesh device

struct **ble_mesh_prov_register_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROV_REGISTER_COMP_EVT.

Public Members

int **err_code**

Indicate the result of BLE Mesh initialization

struct **ble_mesh_provision_complete_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_COMPLETE_EVT.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **net_key**[16]

NetKey

uint16_t **addr**

Primary address

uint8_t **flags**

Flags

uint32_t **iv_index**

IV Index

struct **ble_mesh_provision_reset_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_RESET_EVT.

struct **ble_mesh_provisioner_add_local_app_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_APP_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of adding local AppKey by the Provisioner

uint16_t **net_idx**

NetKey Index

uint16_t **app_idx**

AppKey Index

struct **ble_mesh_provisioner_add_local_net_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_NET_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of adding local NetKey by the Provisioner

uint16_t **net_idx**

NetKey Index

struct **ble_mesh_provisioner_add_unprov_dev_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_ADD_UNPROV_DEV_COMP_EVT.

Public Members

int **err_code**

Indicate the result of adding device into queue by the Provisioner

struct **ble_mesh_provisioner_bind_local_mod_app_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_BIND_APP_KEY_TO_MODEL_COMP_EVT.

Public Members

int **err_code**

Indicate the result of binding AppKey with model by the Provisioner

uint16_t **element_addr**

Element address

uint16_t **app_idx**

AppKey Index

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **ble_mesh_provisioner_cert_based_prov_start_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_CERT_BASED_PROV_START_EVT.

Public Members

uint16_t **link_idx**

Index of the provisioning link

struct **ble_mesh_provisioner_delete_dev_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_DELETE_DEV_COMP_EVT.

Public Members

int **err_code**

Indicate the result of deleting device by the Provisioner

struct **ble_mesh_provisioner_delete_node_with_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of deleting node with unicast address by the Provisioner

uint16_t **unicast_addr**

Node unicast address

struct **ble_mesh_provisioner_delete_node_with_uuid_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_UUID_COMP_EVT.

Public Members

int **err_code**

Indicate the result of deleting node with uuid by the Provisioner

uint8_t **uuid**[16]

Node device uuid

struct **ble_mesh_provisioner_link_close_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_LINK_CLOSE_EVT.

Public Members

esp_ble_mesh_prov_bearer_t **bearer**

Type of the bearer used when Provisioner link is closed

uint8_t **reason**

Reason of the closed provisioning link

struct **ble_mesh_provisioner_link_open_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_LINK_OPEN_EVT.

Public Members

esp_ble_mesh_prov_bearer_t **bearer**

Type of the bearer used when Provisioner link is opened

struct **ble_mesh_provisioner_prov_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT.

Public Members

uint16_t **node_idx**

Index of the provisioned device

esp_ble_mesh_octet16_t **device_uuid**

Device UUID of the provisioned device

uint16_t **unicast_addr**

Primary address of the provisioned device

uint8_t **element_num**

Element count of the provisioned device

uint16_t **netkey_idx**

NetKey Index of the provisioned device

struct **ble_mesh_provisioner_prov_dev_with_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_DEV_WITH_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Provisioner starting to provision a device

struct **ble_mesh_provisioner_prov_disable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_DISABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of disabling BLE Mesh Provisioner

struct **ble_mesh_provisioner_prov_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling BLE Mesh Provisioner

struct **ble_mesh_provisioner_prov_input_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_INPUT_EVT.

Public Members

esp_ble_mesh_oob_method_t **method**

Method of device Output OOB Authentication

esp_ble_mesh_output_action_t **action**

Action of device Output OOB Authentication

uint8_t **size**

Size of device Output OOB Authentication

uint8_t **link_idx**

Index of the provisioning link

struct **ble_mesh_provisioner_prov_input_num_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_INPUT_NUMBER_COMP_EVT.

Public Members

int **err_code**

Indicate the result of inputting number by the Provisioner

struct **ble_mesh_provisioner_prov_input_str_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_INPUT_STRING_COMP_EVT.

Public Members

int **err_code**

Indicate the result of inputting string by the Provisioner

struct **ble_mesh_provisioner_prov_output_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_OUTPUT_EVT.

Public Members

esp_ble_mesh_oob_method_t **method**

Method of device Input OOB Authentication

esp_ble_mesh_input_action_t **action**

Action of device Input OOB Authentication

uint8_t **size**

Size of device Input OOB Authentication

uint8_t **link_idx**

Index of the provisioning link

char **string**[8]

String output by the Provisioner

uint32_t **number**

Number output by the Provisioner

union *esp_ble_mesh_prov_cb_param_t*::*ble_mesh_provisioner_prov_output_evt_param*::[anonymous] **[anonymous]**

Union of output OOB

struct **ble_mesh_provisioner_prov_read_oob_pub_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting OOB Public Key by the Provisioner

struct **ble_mesh_provisioner_prov_read_oob_pub_key_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_EVT.

Public Members

uint8_t **link_idx**

Index of the provisioning link

struct **ble_mesh_provisioner_prov_record_recv_comp_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_PROV_RECORD_RECV_COMP_EVT.

Public Members

uint8_t **status**

Indicates whether or not the request was handled successfully

uint16_t **link_idx**

Index of the provisioning link

uint16_t **record_id**

Identifies the provisioning record for which the request is made

uint16_t **frag_offset**

The starting offset of the requested fragment in the provisioning record data

uint16_t **total_len**

Total length of the provisioning record data stored on the Provisionee

uint8_t ***record**

Provisioning record data fragment

struct **ble_mesh_provisioner_recv_prov_records_list_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_RECV_PROV_RECORDS_LIST_EVT.

Public Members

uint16_t **link_idx**

Index of the provisioning link

uint16_t **len**

Length of message

uint8_t ***msg**

Lists the Record IDs of the provisioning records stored on the Provisionee

struct **ble_mesh_provisioner_recv_unprov_adv_pkt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_RECV_UNPROV_ADV_PKT_EVT.

Public Members

uint8_t **dev_uuid**[16]

Device UUID of the unprovisioned device

esp_ble_mesh_bd_addr_t **addr**

Device address of the unprovisioned device

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint16_t **oob_info**

OOB Info of the unprovisioned device

uint8_t **adv_type**

Advertising type of the unprovisioned device

esp_ble_mesh_prov_bearer_t **bearer**

Bearer of the unprovisioned device

int8_t **rssi**

RSSI of the received advertising packet

struct **ble_mesh_provisioner_send_link_close_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SEND_LINK_CLOSE_EVT.

Public Members

uint16_t **link_idx**

Index of the provisioning link

int **err_code**

Indicate the result of send Link Close message

struct **ble_mesh_provisioner_send_prov_invite_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SEND_PROV_INVITE_EVT.

Public Members

uint16_t **link_idx**

Index of the provisioning link

int **err_code**

Indicate the result of send Provisioning Invite message

struct **ble_mesh_provisioner_send_prov_record_req_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORD_REQUEST_EVT.

Public Members

int **err_code**

Indicate the result of send Provisioning Record Request message

uint16_t **link_idx**

Index of the provisioning link

uint16_t **record_id**

Identifies the provisioning record for which the request is made

uint16_t **frag_offset**

The starting offset of the requested fragment in the provisioning record data

uint16_t **max_size**

The maximum size of the provisioning record fragment that the Provisioner can receive

struct **ble_mesh_provisioner_send_prov_records_get_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORDS_GET_EVT.

Public Members

int **err_code**

Indicate the result of send Provisioning Records List Get message

uint16_t **link_idx**

Index of the provisioning link

struct **ble_mesh_provisioner_set_dev_uuid_match_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SET_DEV_UUID_MATCH_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting Device UUID match value by the Provisioner

struct **ble_mesh_provisioner_set_node_name_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SET_NODE_NAME_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting provisioned device name by the Provisioner

uint16_t **node_index**

Index of the provisioned device

struct **ble_mesh_provisioner_set_primary_elem_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SET_PRIMARY_ELEM_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting unicast address of primary element by the Provisioner

struct **ble_mesh_provisioner_set_prov_data_info_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SET_PROV_DATA_INFO_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting provisioning info by the Provisioner

struct **ble_mesh_provisioner_set_static_oob_val_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_SET_STATIC_OOB_VALUE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting static oob value by the Provisioner

struct **ble_mesh_provisioner_store_node_comp_data_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_STORE_NODE_COMP_DATA_COMP_EVT.

Public Members

int **err_code**

Indicate the result of storing node composition data by the Provisioner

uint16_t **addr**

Node element address

struct **ble_mesh_provisioner_update_local_app_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_APP_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of updating local AppKey by the Provisioner

uint16_t **net_idx**

NetKey Index

uint16_t **app_idx**

AppKey Index

struct **ble_mesh_provisioner_update_local_net_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_NET_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of updating local NetKey by the Provisioner

uint16_t **net_idx**

NetKey Index

struct **ble_mesh_proxy_client_add_filter_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_ADD_FILTER_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Proxy Client add filter address

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_client_connect_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_CONNECT_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Proxy Client connect

esp_ble_mesh_bd_addr_t **addr**

Device address of the Proxy Server

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_client_connected_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_CONNECTED_EVT.

Public Members

esp_ble_mesh_bd_addr_t **addr**

Device address of the Proxy Server

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_client_directed_proxy_set_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_DIRECTED_PROXY_CONTROL_COMP_EV

Public Members

int **err_code**

Indicate the result of Proxy Client directed proxy control address

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_client_disconnect_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_DISCONNECT_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Proxy Client disconnect

uint8_t **conn_handle**

Proxy connection handle

struct **ble_mesh_proxy_client_disconnected_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_DISCONNECTED_EVT.

Public Members

esp_ble_mesh_bd_addr_t **addr**

Device address of the Proxy Server

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

uint8_t **reason**

Proxy disconnect reason

struct **ble_mesh_proxy_client_recv_adv_pkt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_RECV_ADV_PKT_EVT.

Public Members

esp_ble_mesh_bd_addr_t **addr**

Device address

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint16_t **net_idx**

Network ID related NetKey Index

uint8_t **net_id**[8]

Network ID contained in the advertising packet

int8_t **rssi**

RSSI of the received advertising packet

struct **ble_mesh_proxy_client_recv_filter_status_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_RECV_FILTER_STATUS_EVT.

Public Members

uint8_t **conn_handle**

Proxy connection handle

uint16_t **server_addr**

Proxy Server primary element address

uint16_t **net_idx**

Corresponding NetKey Index

uint8_t **filter_type**

Proxy Server filter type(whitelist or blacklist)

uint16_t **list_size**

Number of addresses in the Proxy Server filter list

struct **ble_mesh_proxy_client_remove_filter_addr_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_REMOVE_FILTER_ADDR_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Proxy Client remove filter address

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_client_set_filter_type_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_CLIENT_SET_FILTER_TYPE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of Proxy Client set filter type

uint8_t **conn_handle**

Proxy connection handle

uint16_t **net_idx**

Corresponding NetKey Index

struct **ble_mesh_proxy_gatt_disable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROXY_GATT_DISABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of disabling Mesh Proxy Service

struct **ble_mesh_proxy_gatt_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROXY_GATT_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling Mesh Proxy Service

struct **ble_mesh_proxy_identity_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROXY_IDENTITY_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling Mesh Proxy advertising

struct **ble_mesh_proxy_private_identity_disable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_DISABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of disabling Mesh Proxy private advertising

struct **ble_mesh_proxy_private_identity_enable_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_ENABLE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of enabling Mesh Proxy private advertising

struct **ble_mesh_proxy_server_connected_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_SERVER_CONNECTED_EVT.

Public Members

uint8_t **conn_handle**

Proxy connection handle

struct **ble_mesh_proxy_server_disconnected_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_PROXY_SERVER_DISCONNECTED_EVT.

Public Members

uint8_t **conn_handle**

Proxy connection handle

uint8_t **reason**

Proxy disconnect reason

struct **ble_mesh_set_fast_prov_action_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_SET_FAST_PROV_ACTION_COMP_EVT.

Public Members

uint8_t **status_action**

Indicate the result of setting action of fast provisioning

struct **ble_mesh_set_fast_prov_info_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_SET_FAST_PROV_INFO_COMP_EVT.

Public Members

uint8_t **status_unicast**

Indicate the result of setting unicast address range of fast provisioning

uint8_t **status_net_idx**

Indicate the result of setting NetKey Index of fast provisioning

uint8_t **status_match**

Indicate the result of setting matching Device UUID of fast provisioning

struct **ble_mesh_set_oob_pub_key_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_PROV_SET_OOB_PUB_KEY_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting OOB Public Key

struct **ble_mesh_set_unprov_dev_name_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_NODE_SET_UNPROV_DEV_NAME_COMP_EVT.

Public Members

int **err_code**

Indicate the result of setting BLE Mesh device name

union **esp_ble_mesh_server_state_value_t**

#include <esp_ble_mesh_defs.h> Server model state value union.

Public Members

uint8_t **onoff**

The value of the Generic OnOff state

The value of the Light LC Light OnOff state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **gen_onoff**

The Generic OnOff state

int16_t **level**

The value of the Generic Level state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **gen_level**

The Generic Level state

uint8_t **onpowerup**

The value of the Generic OnPowerUp state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **gen_onpowerup**

The Generic OnPowerUp state

uint16_t **power**

The value of the Generic Power Actual state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **gen_power_actual**

The Generic Power Actual state

uint16_t **lightness**

The value of the Light Lightness Actual state

The value of the Light Lightness Linear state

The value of the Light CTL Lightness state

The value of the Light HSL Lightness state

The value of the Light xyL Lightness state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_lightness_actual**

The Light Lightness Actual state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_lightness_linear**

The Light Lightness Linear state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_ctl_lightness**

The Light CTL Lightness state

uint16_t **temperature**

The value of the Light CTL Temperature state

int16_t **delta_uv**

The value of the Light CTL Delta UV state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_ctl_temp_delta_uv**

The Light CTL Temperature & Delta UV states

uint16_t **hue**

The value of the Light HSL Hue state

uint16_t **saturation**

The value of the Light HSL Saturation state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_hsl**

The Light HSL composite state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_hsl_lightness**

The Light HSL Lightness state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_hsl_hue**

The Light HSL Hue state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_hsl_saturation**

The Light HSL Saturation state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_xyl_lightness**

The Light xyL Lightness state

struct *esp_ble_mesh_server_state_value_t*::[anonymous] **light_lc_light_onoff**

The Light LC Light OnOff state

union **esp_ble_mesh_model_cb_param_t**

#include <esp_ble_mesh_defs.h> BLE Mesh model callback parameters union.

Public Members

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_model_operation_evt_param* **model_operation**

Event parameter of ESP_BLE_MESH_MODEL_OPERATION_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_model_send_comp_param* **model_send_comp**

Event parameter of ESP_BLE_MESH_MODEL_SEND_COMP_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_model_publish_comp_param* **model_publish_comp**

Event parameter of ESP_BLE_MESH_MODEL_PUBLISH_COMP_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_mod_recv_publish_msg_param* **client_recv_publish_msg**

Event parameter of ESP_BLE_MESH_CLIENT_MODEL_RECV_PUBLISH_MSG_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_client_model_send_timeout_param* **client_send_timeout**

Event parameter of ESP_BLE_MESH_CLIENT_MODEL_SEND_TIMEOUT_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_model_publish_update_evt_param* **model_publish_update**

Event parameter of ESP_BLE_MESH_MODEL_PUBLISH_UPDATE_EVT

struct *esp_ble_mesh_model_cb_param_t*::*ble_mesh_server_model_update_state_comp_param* **server_model_update_state**

Event parameter of ESP_BLE_MESH_SERVER_MODEL_UPDATE_STATE_COMP_EVT

struct **ble_mesh_client_model_send_timeout_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_CLIENT_MODEL_SEND_TIMEOUT_EVT.

Public Members

uint32_t **opcode**

Opcode of the previously sent message

esp_ble_mesh_model_t ***model**

Pointer to the model which sends the previous message

esp_ble_mesh_msg_ctx_t ***ctx**

Pointer to the context of the previous message

struct **ble_mesh_mod_recv_publish_msg_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_CLIENT_MODEL_RECV_PUBLISH_MSG_EVT.

Public Members

uint32_t **opcode**

Opcode of the unsolicited received message

esp_ble_mesh_model_t ***model**

Pointer to the model which receives the message

esp_ble_mesh_msg_ctx_t ***ctx**

Pointer to the context of the message

uint16_t **length**

Length of the received message

uint8_t ***msg**

Value of the received message

struct **ble_mesh_model_operation_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_OPERATION_EVT.

Public Members

uint32_t **opcode**

Opcode of the received message

esp_ble_mesh_model_t ***model**

Pointer to the model which receives the message

esp_ble_mesh_msg_ctx_t ***ctx**

Pointer to the context of the received message

uint16_t **length**

Length of the received message

uint8_t ***msg**

Value of the received message

struct **ble_mesh_model_publish_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_PUBLISH_COMP_EVT.

Public Members

int **err_code**

Indicate the result of publishing a message

esp_ble_mesh_model_t ***model**

Pointer to the model which publishes the message

struct **ble_mesh_model_publish_update_evt_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_PUBLISH_UPDATE_EVT.

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the model which is going to update its publish message

struct **ble_mesh_model_send_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_MODEL_SEND_COMP_EVT.

Public Members

int **err_code**

Indicate the result of sending a message

uint32_t **opcode**

Opcode of the message

esp_ble_mesh_model_t ***model**

Pointer to the model which sends the message

esp_ble_mesh_msg_ctx_t ***ctx**

Context of the message

struct **ble_mesh_server_model_update_state_comp_param**

#include <esp_ble_mesh_defs.h> ESP_BLE_MESH_SERVER_MODEL_UPDATE_STATE_COMP_EVT.

Public Members

int **err_code**

Indicate the result of updating server model state

esp_ble_mesh_model_t ***model**

Pointer to the server model which state value is updated

esp_ble_mesh_server_state_type_t **type**

Type of the updated server state

Structures

struct **esp_ble_mesh_deinit_param_t**

BLE Mesh deinit parameters

Public Members

bool **erase_flash**

Indicate if erasing flash when deinit mesh stack

struct **esp_ble_mesh_uar_t**

Format of Unicast Address Range

Public Members

uint16_t **len_present**

Indicate the presence or absence of the RangeLength field

uint16_t **range_start**

15 least significant bits of the starting unicast address

uint8_t **range_length**

Number of addresses in the range (0x02 - 0xFF)

struct **esp_ble_mesh_elem_t**

Abstraction that describes a BLE Mesh Element. This structure is associated with struct bt_mesh_elem in mesh_access.h

Public Members

uint16_t **element_addr**

Element Address, assigned during provisioning.

const uint16_t **location**

Location Descriptor (GATT Bluetooth Namespace Descriptors)

const uint8_t **sig_model_count**

SIG Model count

const uint8_t **vnd_model_count**

Vendor Model count

esp_ble_mesh_model_t ***sig_models**

SIG Models

esp_ble_mesh_model_t ***vnd_models**

Vendor Models

struct **esp_ble_mesh_model_pub_t**

Abstraction that describes a model publication context. This structure is associated with struct bt_mesh_model_pub in mesh_access.h

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the model to which the context belongs. Initialized by the stack.

uint16_t **publish_addr**

Publish Address.

uint16_t **app_idx**

Publish AppKey Index.

uint16_t **cred**

Friendship Credentials Flag.

uint16_t **send_rel**

Force reliable sending (segment acks)

uint16_t **send_szmic**

Size of TransMIC when publishing a Segmented Access message

uint8_t **ttl**

Publish Time to Live.

uint8_t **retransmit**

Retransmit Count & Interval Steps.

uint8_t **period**

Publish Period.

uint8_t **period_div**

Divisor for the Period.

uint8_t **fast_period**

Use FastPeriodDivisor

uint8_t **count**

Retransmissions left.

uint32_t **period_start**

Start of the current period.

struct net_buf_simple ***msg**

Publication buffer, containing the publication message.

This will get correctly created when the publication context has been defined using the ESP_BLE_MESH_MODEL_PUB_DEFINE macro.

ESP_BLE_MESH_MODEL_PUB_DEFINE(name, size);

esp_ble_mesh_cb_t **update**

Callback used to update publish message. Initialized by the stack.

struct k_delayed_work **timer**

Publish Period Timer. Initialized by the stack.

uint8_t **dev_role**

Role of the device that is going to publish messages

struct **esp_ble_mesh_model_op_t**

Abstraction that describes a model operation context. This structure is associated with struct bt_mesh_model_op in mesh_access.h

Public Members

const uint32_t **opcode**

Message opcode

const size_t **min_len**

Message minimum length

esp_ble_mesh_cb_t **param_cb**

Callback used to handle message. Initialized by the stack.

struct **esp_ble_mesh_model_cbs_t**

Abstraction that describes a model callback structure. This structure is associated with struct bt_mesh_model_cb in mesh_access.h.

Public Members

esp_ble_mesh_cb_t **init_cb**

Callback used during model initialization. Initialized by the stack.

struct **esp_ble_mesh_model**

Abstraction that describes a Mesh Model instance. This structure is associated with struct bt_mesh_model in mesh_access.h

Public Members

const uint16_t **model_id**

16-bit model identifier

uint16_t **company_id**

16-bit company identifier

uint16_t **model_id**

16-bit model identifier

struct *esp_ble_mesh_model*::[anonymous]::[anonymous] **vnd**

Structure encapsulating a model ID with a company ID

union *esp_ble_mesh_model*::[anonymous] **[anonymous]**

Model ID

uint8_t **element_idx**

Internal information, mainly for persistent storage Belongs to Nth element

uint8_t **model_idx**

Is the Nth model in the element

uint16_t **flags**

Information about what has changed

esp_ble_mesh_elem_t ***element**

The Element to which this Model belongs

esp_ble_mesh_model_pub_t *const **pub**

Model Publication

uint16_t **keys**[CONFIG_BLE_MESH_MODEL_KEY_COUNT]

AppKey List

uint16_t **groups**[CONFIG_BLE_MESH_MODEL_GROUP_COUNT]

Subscription List (group or virtual addresses)

esp_ble_mesh_model_op_t ***op**

Model operation context

esp_ble_mesh_model_cbs_t ***cb**

Model callback structure

void ***user_data**

Model-specific user data

struct **esp_ble_mesh_msg_ctx_t**

Message sending context. This structure is associated with struct bt_mesh_msg_ctx in mesh_access.h

Public Members

uint16_t **net_idx**

NetKey Index of the subnet through which to send the message.

uint16_t **app_idx**

AppKey Index for message encryption.

uint16_t **addr**

Remote address.

uint16_t **recv_dst**

Destination address of a received message. Not used for sending.

int8_t **recv_rssi**

RSSI of a received message. Not used for sending.

uint32_t **recv_op**

Opcode of a received message. Not used for sending.

uint8_t **recv_ttl**

Received TTL value. Not used for sending.

uint8_t **recv_cred**

Security credentials of a received message. Not used for sending.

uint8_t **recv_tag**

Tag of a received message. Not used for sending.

uint8_t **send_rel**

Force sending reliably by using segment acknowledgement.

uint8_t **send_szmic**

Size of TransMIC when sending a Segmented Access message.

uint8_t **send_ttl**

TTL, or ESP_BLE_MESH_TTL_DEFAULT for default TTL.

uint8_t **send_cred**

Security credentials used for sending the message

uint8_t **send_tag**

Tag used for sending the message.

esp_ble_mesh_model_t ***model**

Model corresponding to the message, no need to be initialized before sending message

bool **srv_send**

Indicate if the message is sent by a node server model, no need to be initialized before sending message

struct **esp_ble_mesh_prov_t**

Provisioning properties & capabilities. This structure is associated with struct bt_mesh_prov in mesh_access.h

struct **esp_ble_mesh_comp_t**

Node Composition data context. This structure is associated with struct bt_mesh_comp in mesh_access.h

Public Members

uint16_t **cid**

16-bit SIG-assigned company identifier

uint16_t **pid**

16-bit vendor-assigned product identifier

uint16_t **vid**

16-bit vendor-assigned product version identifier

size_t **element_count**

Element count

esp_ble_mesh_elem_t ***elements**

A sequence of elements

struct **esp_ble_mesh_unprov_dev_add_t**

Information of the device which is going to be added for provisioning.

Public Members

esp_ble_mesh_bd_addr_t **addr**

Device address

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint8_t **uuid**[16]

Device UUID

uint16_t **oob_info**

Device OOB Info ADD_DEV_START_PROV_NOW_FLAG shall not be set if the bearer has both PB-ADV and PB-GATT enabled

esp_ble_mesh_prov_bearer_t **bearer**

Provisioning Bearer

struct **esp_ble_mesh_device_delete_t**

Information of the device which is going to be deleted.

Public Members

esp_ble_mesh_bd_addr_t **addr**

Device address

esp_ble_mesh_addr_type_t **addr_type**

Device address type

uint8_t **uuid**[16]

Device UUID

union *esp_ble_mesh_device_delete_t*::[anonymous] **[anonymous]**

Union of Device information

uint8_t **flag**

BIT0: device address; BIT1: device UUID

struct **esp_ble_mesh_prov_data_info_t**

Information of the provisioner which is going to be updated.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **flags**

Flags

uint32_t **iv_index**

IV Index

union *esp_ble_mesh_prov_data_info_t*::[anonymous] **[anonymous]**

Provisioning data

uint8_t **flag**

BIT0: net_idx; BIT1: flags; BIT2: iv_index

struct **esp_ble_mesh_node_t**

Information of the provisioned node

Public Members

esp_ble_mesh_bd_addr_t **addr**

Node device address

esp_ble_mesh_addr_type_t **addr_type**

Node device address type

uint8_t **dev_uuid**[16]
Device UUID

uint16_t **oob_info**
Node OOB information

uint16_t **unicast_addr**
Node unicast address

uint8_t **element_num**
Node element number

uint16_t **net_idx**
Node NetKey Index

uint8_t **flags**
Node key refresh flag and iv update flag

uint32_t **iv_index**
Node IV Index

uint8_t **dev_key**[16]
Node device key

char **name**[ESP_BLE_MESH_NODE_NAME_MAX_LEN + 1]
Node name

uint16_t **comp_length**
Length of Composition Data

uint8_t ***comp_data**
Value of Composition Data

struct **esp_ble_mesh_fast_prov_info_t**
Context of fast provisioning which need to be set.

Public Members

uint16_t **unicast_min**
Minimum unicast address used for fast provisioning

uint16_t **unicast_max**
Maximum unicast address used for fast provisioning

uint16_t **net_idx**
Netkey index used for fast provisioning

uint8_t **flags**

Flags used for fast provisioning

uint32_t **iv_index**

IV Index used for fast provisioning

uint8_t **offset**

Offset of the UUID to be compared

uint8_t **match_len**

Length of the UUID to be compared

uint8_t **match_val**[16]

Value of UUID to be compared

struct **esp_ble_mesh_heartbeat_filter_info_t**

Context of Provisioner heartbeat filter information to be set

Public Members

uint16_t **hb_src**

Heartbeat source address (unicast address)

uint16_t **hb_dst**

Heartbeat destination address (unicast address or group address)

struct **esp_ble_mesh_client_op_pair_t**

BLE Mesh client models related definitions.

Client model Get/Set message opcode and corresponding Status message opcode

Public Members

uint32_t **cli_op**

The client message opcode

uint32_t **status_op**

The server status opcode corresponding to the client message opcode

struct **esp_ble_mesh_client_t**

Client Model user data context.

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the client model. Initialized by the stack.

uint32_t **op_pair_size**

Size of the op_pair

const *esp_ble_mesh_client_op_pair_t* ***op_pair**

Table containing get/set message opcode and corresponding status message opcode

uint32_t **publish_status**

Callback used to handle the received unsolicited message. Initialized by the stack.

void ***internal_data**

Pointer to the internal data of client model

void ***vendor_data**

Pointer to the vendor data of client model

uint8_t **msg_role**

Role of the device (Node/Provisioner) that is going to send messages

struct **esp_ble_mesh_client_common_param_t**

Common parameters of the messages sent by Client Model.

Public Members

esp_ble_mesh_opcode_t **opcode**

Message opcode

esp_ble_mesh_model_t ***model**

Pointer to the client model structure

esp_ble_mesh_msg_ctx_t **ctx**

The context used to send message

int32_t **msg_timeout**

Timeout value (ms) to get response to the sent message Note: if using default timeout value in menuconfig, make sure to set this value to 0

uint8_t **msg_role**

Role of the device - Node/Provisioner

struct **esp_ble_mesh_state_transition_t**

Parameters of the server model state transition

Public Functions

BLE_MESH_ATOMIC_DEFINE(flag, *ESP_BLE_MESH_SERVER_FLAG_MAX*)

Flag used to indicate if the transition timer has been started internally.

If the model which contains *esp_ble_mesh_state_transition_t* sets "set_auto_rsp" to ESP_BLE_MESH_SERVER_RSP_BY_APP, the handler of the timer shall be initialized by the users.

And users can use this flag to indicate whether the timer is started or not.

Public Members

bool **just_started**

Indicate if the state transition has just started

uint8_t **trans_time**

State transition time

uint8_t **remain_time**

Remaining time of state transition

uint8_t **delay**

Delay before starting state transition

uint32_t **quo_tt**

Duration of each divided transition step

uint32_t **counter**

Number of steps which the transition duration is divided

uint32_t **total_duration**

State transition total duration

int64_t **start_timestamp**

Time when the state transition is started

struct k_delayed_work **timer**

Timer used for state transition

struct **esp_ble_mesh_last_msg_info_t**

Parameters of the server model received last same set message.

Public Members

uint8_t **tid**

Transaction number of the last message

uint16_t **src**

Source address of the last message

uint16_t **dst**

Destination address of the last message

int64_t **timestamp**

Time when the last message is received

struct **esp_ble_mesh_server_rsp_ctrl_t**

Parameters of the Server Model response control

Public Members

uint8_t **get_auto_rsp**

BLE Mesh Server Response Option.

i. If get_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, then the response of Client Get messages need to be replied by the application;
ii. If get_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, then the response of Client Get messages will be replied by the server models;
iii. If set_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, then the response of Client Set messages need to be replied by the application;
iv. If set_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, then the response of Client Set messages will be replied by the server models;
v. If status_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, then the response of Server Status messages need to be replied by the application;
vi. If status_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, then the response of Server Status messages will be replied by the server models; Response control for Client Get messages

uint8_t **set_auto_rsp**

Response control for Client Set messages

uint8_t **status_auto_rsp**

Response control for Server Status messages

Macros

ESP_BLE_MESH_SDU_MAX_LEN

< The maximum length of a BLE Mesh message, including Opcode, Payload and TransMIC Length of a short Mesh MIC.

ESP_BLE_MESH_MIC_SHORT

Length of a long Mesh MIC.

ESP_BLE_MESH_MIC_LONG

The maximum length of a BLE Mesh provisioned node name

ESP_BLE_MESH_NODE_NAME_MAX_LEN

The maximum length of a BLE Mesh unprovisioned device name

ESP_BLE_MESH_DEVICE_NAME_MAX_LEN

The maximum length of settings user id

ESP_BLE_MESH_SETTINGS_UID_SIZE

The default value of Random Update Interval Steps

ESP_BLE_MESH_RAND_UPDATE_INTERVAL_DEFAULT

Invalid settings index

ESP_BLE_MESH_INVALID_SETTINGS_IDX

Define the BLE Mesh octet 16 bytes size

ESP_BLE_MESH_OCTET16_LEN

ESP_BLE_MESH_OCTET8_LEN

ESP_BLE_MESH_CID_NVAL

Special TTL value to request using configured default TTL

ESP_BLE_MESH_TTL_DEFAULT

Maximum allowed TTL value

ESP_BLE_MESH_TTL_MAX

ESP_BLE_MESH_ADDR_UNASSIGNED

ESP_BLE_MESH_ADDR_ALL_NODES

ESP_BLE_MESH_ADDR_PROXIES

ESP_BLE_MESH_ADDR_FRIENDS

ESP_BLE_MESH_ADDR_RELAYS

ESP_BLE_MESH_KEY_UNUSED

ESP_BLE_MESH_KEY_DEV

ESP_BLE_MESH_KEY_PRIMARY

ESP_BLE_MESH_KEY_ANY

Internal macros used to initialize array members

ESP_BLE_MESH_KEY_UNUSED_ELT_(IDX, _)

ESP_BLE_MESH_ADDR_UNASSIGNED_ELT_(IDX, _)

ESP_BLE_MESH_MODEL_KEYS_UNUSED

ESP_BLE_MESH_MODEL_GROUPS_UNASSIGNED

Primary Network Key index

ESP_BLE_MESH_NET_PRIMARY

Relay state value

ESP_BLE_MESH_RELAY_DISABLED

ESP_BLE_MESH_RELAY_ENABLED

ESP_BLE_MESH_RELAY_NOT_SUPPORTED

Beacon state value

ESP_BLE_MESH_BEACON_DISABLED

ESP_BLE_MESH_BEACON_ENABLED

ESP_BLE_MESH_PRIVATE_BEACON_DISABLE

ESP_BLE_MESH_PRIVATE_BEACON_ENABLE

GATT Proxy state value

ESP_BLE_MESH_GATT_PROXY_DISABLED

ESP_BLE_MESH_GATT_PROXY_ENABLED

ESP_BLE_MESH_GATT_PROXY_NOT_SUPPORTED

ESP_BLE_MESH_PRIVATE_GATT_PROXY_DISABLED

ESP_BLE_MESH_PRIVATE_GATT_PROXY_ENABLED

ESP_BLE_MESH_PRIVATE_GATT_PROXY_NOT_SUPPORTED

ESP_BLE_MESH_PRIVATE_NODE_IDENTITY_DISABLED

ESP_BLE_MESH_PRIVATE_NODE_IDENTITY_ENABLED

ESP_BLE_MESH_PRIVATE_NODE_IDENTITY_NOT_SUPPORTED

Friend state value

ESP_BLE_MESH_FRIEND_DISABLED

ESP_BLE_MESH_FRIEND_ENABLED

ESP_BLE_MESH_FRIEND_NOT_SUPPORTED

Node identity state value

ESP_BLE_MESH_NODE_IDENTITY_STOPPED

ESP_BLE_MESH_NODE_IDENTITY_RUNNING

ESP_BLE_MESH_NODE_IDENTITY_NOT_SUPPORTED

Subnet Bridge state value

ESP_BLE_MESH_SUBNET_BRIDGE_DISABLED

ESP_BLE_MESH_SUBNET_BRIDGE_ENABLED

Supported features

ESP_BLE_MESH_FEATURE_RELAY

ESP_BLE_MESH_FEATURE_PROXY

ESP_BLE_MESH_FEATURE_FRIEND

ESP_BLE_MESH_FEATURE_LOW_POWER

ESP_BLE_MESH_FEATURE_ALL_SUPPORTED

ESP_BLE_MESH_ADDR_IS_UNICAST(addr)

ESP_BLE_MESH_ADDR_IS_GROUP(addr)

ESP_BLE_MESH_ADDR_IS_VIRTUAL(addr)

ESP_BLE_MESH_ADDR_IS_RFU(addr)

ESP_BLE_MESH_INVALID_NODE_INDEX

ESP_BLE_MESH_PROV_RECORD_MAX_ID

ESP_BLE_MESH_TRANSMIT(count, int_ms)

Encode transmission count & interval steps.

Note: For example, ESP_BLE_MESH_TRANSMIT(2, 20) means that the message will be sent about 90ms(count is 3, step is 1, interval is 30 ms which includes 10ms of advertising interval random delay).

Parameters

- **count** -- Number of retransmissions (first transmission is excluded).
- **int_ms** -- Interval steps in milliseconds. Must be greater than 0 and a multiple of 10.

Returns BLE Mesh transmit value that can be used e.g. for the default values of the Configuration Model data.

ESP_BLE_MESH_GET_TRANSMIT_COUNT(transmit)

Decode transmit count from a transmit value.

Parameters

- **transmit** -- Encoded transmit count & interval value.

Returns Transmission count (actual transmissions equal to N + 1).

ESP_BLE_MESH_GET_TRANSMIT_INTERVAL(transmit)

Decode transmit interval from a transmit value.

Parameters

- **transmit** -- Encoded transmit count & interval value.

Returns Transmission interval in milliseconds.

ESP_BLE_MESH_PUBLISH_TRANSMIT(count, int_ms)

Encode Publish Retransmit count & interval steps.

Parameters
- **count** -- Number of retransmissions (first transmission is excluded).
- **int_ms** -- Interval steps in milliseconds. Must be greater than 0 and a multiple of 50.

Returns BLE Mesh transmit value that can be used e.g. for the default values of the Configuration Model data.

ESP_BLE_MESH_GET_PUBLISH_TRANSMIT_COUNT(transmit)

Decode Publish Retransmit count from a given value.

Parameters
- **transmit** -- Encoded Publish Retransmit count & interval value.

Returns Retransmission count (actual transmissions equal to N + 1).

ESP_BLE_MESH_GET_PUBLISH_TRANSMIT_INTERVAL(transmit)

Decode Publish Retransmit interval from a given value.

Callbacks which are not needed to be initialized by users (set with 0 and will be initialized internally)

Parameters
- **transmit** -- Encoded Publish Retransmit count & interval value.

Returns Transmission interval in milliseconds.

ESP_BLE_MESH_PROV_STATIC_OOB_MAX_LEN

Maximum length of string used by Output OOB authentication

ESP_BLE_MESH_PROV_OUTPUT_OOB_MAX_LEN

Maximum length of string used by Output OOB authentication

ESP_BLE_MESH_PROV_INPUT_OOB_MAX_LEN

Macros used to define message opcode

ESP_BLE_MESH_MODEL_OP_1(b0)

ESP_BLE_MESH_MODEL_OP_2(b0, b1)

ESP_BLE_MESH_MODEL_OP_3(b0, cid)

This macro is associated with BLE_MESH_MODEL_CB in mesh_access.h

ESP_BLE_MESH_SIG_MODEL(_id, _op, _pub, _user_data)

This macro is associated with BLE_MESH_MODEL_VND_CB in mesh_access.h

ESP_BLE_MESH_VENDOR_MODEL(_company, _id, _op, _pub, _user_data)

ESP_BLE_MESH_ELEMENT(_loc, _mods, _vnd_mods)

Helper to define a BLE Mesh element within an array.

In case the element has no SIG or Vendor models, the helper macro ESP_BLE_MESH_MODEL_NONE can be given instead.

Note: This macro is associated with BLE_MESH_ELEM in mesh_access.h

---

Parameters
- **_loc** -- Location Descriptor.
- **_mods** -- Array of SIG models.
- **_vnd_mods** -- Array of vendor models.

ESP_BLE_MESH_PROV (uuid, sta_val, sta_val_len, out_size, out_act, in_size, in_act)

BT_OCTET32_LEN

BD_ADDR_LEN

ESP_BLE_MESH_ADDR_TYPE_PUBLIC

ESP_BLE_MESH_ADDR_TYPE_RANDOM

ESP_BLE_MESH_ADDR_TYPE_RPA_PUBLIC

ESP_BLE_MESH_ADDR_TYPE_RPA_RANDOM

ESP_BLE_MESH_DIRECTED_FORWARDING_DISABLED

ESP_BLE_MESH_DIRECTED_FORWARDING_ENABLED

ESP_BLE_MESH_DIRECTED_RELAY_DISABLED

ESP_BLE_MESH_DIRECTED_RELAY_ENABLED

ESP_BLE_MESH_DIRECTED_PROXY_IGNORE

ESP_BLE_MESH_DIRECTED_PROXY_USE_DEFAULT_IGNORE

ESP_BLE_MESH_DIRECTED_FRIEND_IGNORE

ESP_BLE_MESH_DIRECTED_PROXY_DISABLED

ESP_BLE_MESH_DIRECTED_PROXY_ENABLED

ESP_BLE_MESH_DIRECTED_PROXY_NOT_SUPPORTED

ESP_BLE_MESH_DIRECTED_PROXY_USE_DEF_DISABLED

ESP_BLE_MESH_DIRECTED_PROXY_USE_DEF_ENABLED

ESP_BLE_MESH_DIRECTED_PROXY_USE_DEF_NOT_SUPPORTED

ESP_BLE_MESH_DIRECTED_FRIEND_DISABLED

ESP_BLE_MESH_DIRECTED_FRIEND_ENABLED

ESP_BLE_MESH_DIRECTED_FRIEND_NOT_SUPPORTED

ESP_BLE_MESH_DIRECTED_PUB_POLICY_FLOODING

ESP_BLE_MESH_DIRECTED_PUB_POLICY_FORWARD

ESP_BLE_MESH_PROXY_USE_DIRECTED_DISABLED

ESP_BLE_MESH_PROXY_USE_DIRECTED_ENABLED

ESP_BLE_MESH_FLOODING_CRED

ESP_BLE_MESH_FRIENDSHIP_CRED

ESP_BLE_MESH_DIRECTED_CRED

ESP_BLE_MESH_TAG_SEND_SEGMENTED

ESP_BLE_MESH_TAG_IMMUTABLE_CRED

ESP_BLE_MESH_TAG_USE_DIRECTED

ESP_BLE_MESH_TAG_RELAY

ESP_BLE_MESH_TAG_FRIENDSHIP

ESP_BLE_MESH_SEG_SZMIC_SHORT

ESP_BLE_MESH_SEG_SZMIC_LONG

ESP_BLE_MESH_MODEL_PUB_DEFINE(_name, _msg_len, _role)

Define a model publication context.

Parameters

- **_name** -- Variable name given to the context.
- **_msg_len** -- Length of the publication message.
- **_role** -- Role of the device which contains the model.

ESP_BLE_MESH_MODEL_OP(_opcode, _min_len)

Define a model operation context.

Parameters

- **_opcode** -- Message opcode.
- **_min_len** -- Message minimum length.

ESP_BLE_MESH_MODEL_OP_END

Define the terminator for the model operation table. Each model operation struct array must use this terminator as the end tag of the operation unit.

ESP_BLE_MESH_MODEL_NONE

Helper to define an empty model array. This structure is associated with BLE_MESH_MODEL_NONE in mesh_access.h

ADD_DEV_RM_AFTER_PROV_FLAG

Device will be removed from queue after provisioned successfully

ADD_DEV_START_PROV_NOW_FLAG

Start provisioning device immediately

ADD_DEV_FLUSHABLE_DEV_FLAG

Device can be remove when queue is full and new device is going to added

DEL_DEV_ADDR_FLAG

DEL_DEV_UUID_FLAG

PROV_DATA_NET_IDX_FLAG

PROV_DATA_FLAGS_FLAG

PROV_DATA_IV_INDEX_FLAG

ESP_BLE_MESH_HEARTBEAT_FILTER_ACCEPTLIST

ESP_BLE_MESH_HEARTBEAT_FILTER_REJECTLIST

Provisioner heartbeat filter operation

ESP_BLE_MESH_HEARTBEAT_FILTER_ADD

ESP_BLE_MESH_HEARTBEAT_FILTER_REMOVE

ESP_BLE_MESH_MODEL_ID_CONFIG_SRV

BLE Mesh models related Model ID and Opcode definitions.

< Foundation Models

ESP_BLE_MESH_MODEL_ID_CONFIG_CLI

ESP_BLE_MESH_MODEL_ID_HEALTH_SRV

ESP_BLE_MESH_MODEL_ID_HEALTH_CLI

ESP_BLE_MESH_MODEL_ID_RPR_SRV

ESP_BLE_MESH_MODEL_ID_RPR_CLI

ESP_BLE_MESH_MODEL_ID_DF_SRV

ESP_BLE_MESH_MODEL_ID_DF_CLI

ESP_BLE_MESH_MODEL_ID_BRC_SRV

ESP_BLE_MESH_MODEL_ID_BRC_CLI

ESP_BLE_MESH_MODEL_ID_PRB_SRV

ESP_BLE_MESH_MODEL_ID_PRB_CLI

ESP_BLE_MESH_MODEL_ID_ODP_SRV

ESP_BLE_MESH_MODEL_ID_ODP_CLI

ESP_BLE_MESH_MODEL_ID_SAR_SRV

ESP_BLE_MESH_MODEL_ID_SAR_CLI

ESP_BLE_MESH_MODEL_ID_AGG_SRV

ESP_BLE_MESH_MODEL_ID_AGG_CLI

ESP_BLE_MESH_MODEL_ID_LCD_SRV

ESP_BLE_MESH_MODEL_ID_LCD_CLI

ESP_BLE_MESH_MODEL_ID_SRPL_SRV

ESP_BLE_MESH_MODEL_ID_SRPL_CLI

Models from the Mesh Model Specification

ESP_BLE_MESH_MODEL_ID_GEN_ONOFF_SRV

ESP_BLE_MESH_MODEL_ID_GEN_ONOFF_CLI

ESP_BLE_MESH_MODEL_ID_GEN_LEVEL_SRV

ESP_BLE_MESH_MODEL_ID_GEN_LEVEL_CLI

ESP_BLE_MESH_MODEL_ID_GEN_DEF_TRANS_TIME_SRV

ESP_BLE_MESH_MODEL_ID_GEN_DEF_TRANS_TIME_CLI

ESP_BLE_MESH_MODEL_ID_GEN_POWER_ONOFF_SRV

ESP_BLE_MESH_MODEL_ID_GEN_POWER_ONOFF_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_POWER_ONOFF_CLI

ESP_BLE_MESH_MODEL_ID_GEN_POWER_LEVEL_SRV

ESP_BLE_MESH_MODEL_ID_GEN_POWER_LEVEL_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_POWER_LEVEL_CLI

ESP_BLE_MESH_MODEL_ID_GEN_BATTERY_SRV

ESP_BLE_MESH_MODEL_ID_GEN_BATTERY_CLI

ESP_BLE_MESH_MODEL_ID_GEN_LOCATION_SRV

ESP_BLE_MESH_MODEL_ID_GEN_LOCATION_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_LOCATION_CLI

ESP_BLE_MESH_MODEL_ID_GEN_ADMIN_PROP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_MANUFACTURER_PROP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_USER_PROP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_CLIENT_PROP_SRV

ESP_BLE_MESH_MODEL_ID_GEN_PROP_CLI

ESP_BLE_MESH_MODEL_ID_SENSOR_SRV

ESP_BLE_MESH_MODEL_ID_SENSOR_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_SENSOR_CLI

ESP_BLE_MESH_MODEL_ID_TIME_SRV

ESP_BLE_MESH_MODEL_ID_TIME_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_TIME_CLI

ESP_BLE_MESH_MODEL_ID_SCENE_SRV

ESP_BLE_MESH_MODEL_ID_SCENE_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_SCENE_CLI

ESP_BLE_MESH_MODEL_ID_SCHEDULER_SRV

ESP_BLE_MESH_MODEL_ID_SCHEDULER_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_SCHEDULER_CLI

ESP_BLE_MESH_MODEL_ID_LIGHT_LIGHTNESS_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_LIGHTNESS_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_LIGHTNESS_CLI

ESP_BLE_MESH_MODEL_ID_LIGHT_CTL_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_CTL_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_CTL_CLI

ESP_BLE_MESH_MODEL_ID_LIGHT_CTL_TEMP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_HSL_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_HSL_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_HSL_CLI

ESP_BLE_MESH_MODEL_ID_LIGHT_HSL_HUE_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_HSL_SAT_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_XYL_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_XYL_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_XYL_CLI

ESP_BLE_MESH_MODEL_ID_LIGHT_LC_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_LC_SETUP_SRV

ESP_BLE_MESH_MODEL_ID_LIGHT_LC_CLI

ESP_BLE_MESH_MODEL_ID_MBT_SRV

ESP_BLE_MESH_MODEL_ID_MBT_CLI

ESP_BLE_MESH_MODEL_OP_BEACON_GET

Config Beacon Get

ESP_BLE_MESH_MODEL_OP_COMPOSITION_DATA_GET

Config Composition Data Get

ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_GET

Config Default TTL Get

ESP_BLE_MESH_MODEL_OP_GATT_PROXY_GET

Config GATT Proxy Get

ESP_BLE_MESH_MODEL_OP_RELAY_GET

Config Relay Get

ESP_BLE_MESH_MODEL_OP_MODEL_PUB_GET

Config Model Publication Get

ESP_BLE_MESH_MODEL_OP_FRIEND_GET

Config Friend Get

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_GET

Config Heartbeat Publication Get

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_GET

Config Heartbeat Subscription Get

ESP_BLE_MESH_MODEL_OP_NET_KEY_GET

Config NetKey Get

ESP_BLE_MESH_MODEL_OP_APP_KEY_GET

Config AppKey Get

ESP_BLE_MESH_MODEL_OP_NODE_IDENTITY_GET

Config Node Identity Get

ESP_BLE_MESH_MODEL_OP_SIG_MODEL_SUB_GET

Config SIG Model Subscription Get

ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_SUB_GET

Config Vendor Model Subscription Get

ESP_BLE_MESH_MODEL_OP_SIG_MODEL_APP_GET

Config SIG Model App Get

ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_APP_GET

Config Vendor Model App Get

ESP_BLE_MESH_MODEL_OP_KEY_REFRESH_PHASE_GET

Config Key Refresh Phase Get

ESP_BLE_MESH_MODEL_OP_LPN_POLLTIMEOUT_GET

Config Low Power Node PollTimeout Get

ESP_BLE_MESH_MODEL_OP_NETWORK_TRANSMIT_GET

Config Network Transmit Get

ESP_BLE_MESH_MODEL_OP_BEACON_SET

Config Beacon Set

ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_SET

Config Default TTL Set

ESP_BLE_MESH_MODEL_OP_GATT_PROXY_SET

Config GATT Proxy Set

ESP_BLE_MESH_MODEL_OP_RELAY_SET

Config Relay Set

ESP_BLE_MESH_MODEL_OP_MODEL_PUB_SET

Config Model Publication Set

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_ADD

Config Model Subscription Add

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_ADD

Config Model Subscription Virtual Address Add

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_DELETE

Config Model Subscription Delete

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_DELETE

Config Model Subscription Virtual Address Delete

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_OVERWRITE

Config Model Subscription Overwrite

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_OVERWRITE

Config Model Subscription Virtual Address Overwrite

ESP_BLE_MESH_MODEL_OP_NET_KEY_ADD

Config NetKey Add

ESP_BLE_MESH_MODEL_OP_APP_KEY_ADD

Config AppKey Add

ESP_BLE_MESH_MODEL_OP_MODEL_APP_BIND

Config Model App Bind

ESP_BLE_MESH_MODEL_OP_NODE_RESET

Config Node Reset

ESP_BLE_MESH_MODEL_OP_FRIEND_SET

Config Friend Set

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_SET

Config Heartbeat Publication Set

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_SET

Config Heartbeat Subscription Set

ESP_BLE_MESH_MODEL_OP_NET_KEY_UPDATE

Config NetKey Update

ESP_BLE_MESH_MODEL_OP_NET_KEY_DELETE

Config NetKey Delete

ESP_BLE_MESH_MODEL_OP_APP_KEY_UPDATE

Config AppKey Update

ESP_BLE_MESH_MODEL_OP_APP_KEY_DELETE

Config AppKey Delete

ESP_BLE_MESH_MODEL_OP_NODE_IDENTITY_SET

Config Node Identity Set

ESP_BLE_MESH_MODEL_OP_KEY_REFRESH_PHASE_SET

Config Key Refresh Phase Set

ESP_BLE_MESH_MODEL_OP_MODEL_PUB_VIRTUAL_ADDR_SET

Config Model Publication Virtual Address Set

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_DELETE_ALL

Config Model Subscription Delete All

ESP_BLE_MESH_MODEL_OP_MODEL_APP_UNBIND

Config Model App Unbind

ESP_BLE_MESH_MODEL_OP_NETWORK_TRANSMIT_SET

Config Network Transmit Set

ESP_BLE_MESH_MODEL_OP_BEACON_STATUS

ESP_BLE_MESH_MODEL_OP_COMPOSITION_DATA_STATUS

ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_STATUS

ESP_BLE_MESH_MODEL_OP_GATT_PROXY_STATUS

ESP_BLE_MESH_MODEL_OP_RELAY_STATUS

ESP_BLE_MESH_MODEL_OP_MODEL_PUB_STATUS

ESP_BLE_MESH_MODEL_OP_MODEL_SUB_STATUS

ESP_BLE_MESH_MODEL_OP_SIG_MODEL_SUB_LIST

ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_SUB_LIST

ESP_BLE_MESH_MODEL_OP_NET_KEY_STATUS

ESP_BLE_MESH_MODEL_OP_NET_KEY_LIST

ESP_BLE_MESH_MODEL_OP_APP_KEY_STATUS

ESP_BLE_MESH_MODEL_OP_APP_KEY_LIST

ESP_BLE_MESH_MODEL_OP_NODE_IDENTITY_STATUS

ESP_BLE_MESH_MODEL_OP_MODEL_APP_STATUS

ESP_BLE_MESH_MODEL_OP_SIG_MODEL_APP_LIST

ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_APP_LIST

ESP_BLE_MESH_MODEL_OP_NODE_RESET_STATUS

ESP_BLE_MESH_MODEL_OP_FRIEND_STATUS

ESP_BLE_MESH_MODEL_OP_KEY_REFRESH_PHASE_STATUS

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_STATUS

ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_STATUS

ESP_BLE_MESH_MODEL_OP_LPN_POLLTIMEOUT_STATUS

ESP_BLE_MESH_MODEL_OP_NETWORK_TRANSMIT_STATUS

ESP_BLE_MESH_CFG_STATUS_SUCCESS

ESP_BLE_MESH_CFG_STATUS_INVALID_ADDRESS

ESP_BLE_MESH_CFG_STATUS_INVALID_MODEL

ESP_BLE_MESH_CFG_STATUS_INVALID_APPKEY

ESP_BLE_MESH_CFG_STATUS_INVALID_NETKEY

ESP_BLE_MESH_CFG_STATUS_INSUFFICIENT_RESOURCES

ESP_BLE_MESH_CFG_STATUS_KEY_INDEX_ALREADY_STORED

ESP_BLE_MESH_CFG_STATUS_INVALID_PUBLISH_PARAMETERS

ESP_BLE_MESH_CFG_STATUS_NOT_A_SUBSCRIBE_MODEL

ESP_BLE_MESH_CFG_STATUS_STORAGE_FAILURE

ESP_BLE_MESH_CFG_STATUS_FEATURE_NOT_SUPPORTED

ESP_BLE_MESH_CFG_STATUS_CANNOT_UPDATE

ESP_BLE_MESH_CFG_STATUS_CANNOT_REMOVE

ESP_BLE_MESH_CFG_STATUS_CANNOT_BIND

ESP_BLE_MESH_CFG_STATUS_TEMP_UNABLE_TO_CHANGE_STATE

ESP_BLE_MESH_CFG_STATUS_CANNOT_SET

ESP_BLE_MESH_CFG_STATUS_UNSPECIFIED_ERROR

ESP_BLE_MESH_CFG_STATUS_INVALID_BINDING

ESP_BLE_MESH_CFG_STATUS_INVALID_PATH_ENTRY

ESP_BLE_MESH_CFG_STATUS_CANNOT_GET

ESP_BLE_MESH_CFG_STATUS_OBSOLETE_INFO

ESP_BLE_MESH_CFG_STATUS_INVALID_BEARER

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_GET

Health Fault Get

ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_GET
Health Period Get

ESP_BLE_MESH_MODEL_OP_ATTENTION_GET
Health Attention Get

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR
Health Fault Clear

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR_UNACK
Health Fault Clear Unacknowledged

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_TEST
Health Fault Test

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_TEST_UNACK
Health Fault Test Unacknowledged

ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_SET
Health Period Set

ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_SET_UNACK
Health Period Set Unacknowledged

ESP_BLE_MESH_MODEL_OP_ATTENTION_SET
Health Attention Set

ESP_BLE_MESH_MODEL_OP_ATTENTION_SET_UNACK
Health Attention Set Unacknowledged

ESP_BLE_MESH_MODEL_OP_HEALTH_CURRENT_STATUS

ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_STATUS

ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_STATUS

ESP_BLE_MESH_MODEL_OP_ATTENTION_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_GET

ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET

ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_STATUS
Generic Level Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_GET

ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET

ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_DELTA_SET

ESP_BLE_MESH_MODEL_OP_GEN_DELTA_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_MOVE_SET

ESP_BLE_MESH_MODEL_OP_GEN_MOVE_SET_UNACK
Generic Default Transition Time Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_GET

ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_SET

ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_STATUS
Generic Power OnOff Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_GET

ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_STATUS
Generic Power OnOff Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_SET

ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_SET_UNACK
Generic Power Level Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_GET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_SET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LAST_GET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_LAST_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_GET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_GET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_STATUS

Generic Power Level Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_SET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_SET

ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_SET_UNACK

Generic Battery Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_BATTERY_GET

ESP_BLE_MESH_MODEL_OP_GEN_BATTERY_STATUS

Generic Location Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_GET

ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_GET

ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_STATUS

Generic Location Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_SET

ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_SET

ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_SET_UNACK

Generic Manufacturer Property Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTIES_GET

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTIES_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_GET

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_SET

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_STATUS

Generic Admin Property Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTIES_GET

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTIES_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_GET

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_SET

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_STATUS

Generic User Property Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTIES_GET

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTIES_STATUS

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_GET

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_SET

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_SET_UNACK

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_STATUS

Generic Client Property Message Opcode

ESP_BLE_MESH_MODEL_OP_GEN_CLIENT_PROPERTIES_GET

ESP_BLE_MESH_MODEL_OP_GEN_CLIENT_PROPERTIES_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_DESCRIPTOR_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_DESCRIPTOR_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_COLUMN_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_COLUMN_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_SERIES_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_SERIES_STATUS

Sensor Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_SET

ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_SET_UNACK

ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTINGS_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTINGS_STATUS

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_GET

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_SET

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_SET_UNACK

ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_STATUS

ESP_BLE_MESH_MODEL_OP_TIME_GET

ESP_BLE_MESH_MODEL_OP_TIME_SET

ESP_BLE_MESH_MODEL_OP_TIME_STATUS

ESP_BLE_MESH_MODEL_OP_TIME_ROLE_GET

ESP_BLE_MESH_MODEL_OP_TIME_ROLE_SET

ESP_BLE_MESH_MODEL_OP_TIME_ROLE_STATUS

ESP_BLE_MESH_MODEL_OP_TIME_ZONE_GET

ESP_BLE_MESH_MODEL_OP_TIME_ZONE_SET

ESP_BLE_MESH_MODEL_OP_TIME_ZONE_STATUS

ESP_BLE_MESH_MODEL_OP_TAI_UTC_DELTA_GET

ESP_BLE_MESH_MODEL_OP_TAI_UTC_DELTA_SET

ESP_BLE_MESH_MODEL_OP_TAI_UTC_DELTA_STATUS

Scene Message Opcode

ESP_BLE_MESH_MODEL_OP_SCENE_GET

ESP_BLE_MESH_MODEL_OP_SCENE_RECALL

ESP_BLE_MESH_MODEL_OP_SCENE_RECALL_UNACK

ESP_BLE_MESH_MODEL_OP_SCENE_STATUS

ESP_BLE_MESH_MODEL_OP_SCENE_REGISTER_GET

ESP_BLE_MESH_MODEL_OP_SCENE_REGISTER_STATUS

Scene Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_SCENE_STORE

ESP_BLE_MESH_MODEL_OP_SCENE_STORE_UNACK

ESP_BLE_MESH_MODEL_OP_SCENE_DELETE

ESP_BLE_MESH_MODEL_OP_SCENE_DELETE_UNACK

Scheduler Message Opcode

ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_GET

ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_STATUS

ESP_BLE_MESH_MODEL_OP_SCHEDULER_GET

ESP_BLE_MESH_MODEL_OP_SCHEDULER_STATUS

Scheduler Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_SET

ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LAST_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LAST_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_STATUS
Light Lightness Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_SET_UNACK
Light CTL Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_STATUS
Light CTL Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_SET_UNACK
Light HSL Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_TARGET_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_TARGET_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_STATUS
Light HSL Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_SET_UNACK
Light xyL Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_TARGET_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_TARGET_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_STATUS
Light xyL Setup Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_SET_UNACK

Light Control Message Opcode

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_STATUS

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_GET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_SET

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_SET_UNACK

ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_STATUS

ESP_BLE_MESH_MODEL_STATUS_SUCCESS

ESP_BLE_MESH_MODEL_STATUS_CANNOT_SET_RANGE_MIN

ESP_BLE_MESH_MODEL_STATUS_CANNOT_SET_RANGE_MAX

ESP_BLE_MESH_SERVER_RSP_BY_APP

Response need to be sent in the application

ESP_BLE_MESH_SERVER_AUTO_RSP

Response will be sent internally

Type Definitions

typedef uint8_t **esp_ble_mesh_octet16_t**[ESP_BLE_MESH_OCTET16_LEN]

Define the BLE Mesh octet 8 bytes size

typedef uint8_t **esp_ble_mesh_octet8_t**[ESP_BLE_MESH_OCTET8_LEN]

Invalid Company ID

typedef uint32_t **esp_ble_mesh_cb_t**

typedef uint8_t **UINT8**

typedef uint16_t **UINT16**

typedef uint32_t **UINT32**

typedef uint64_t **UINT64**

typedef *UINT8* **BT_OCTET32**[BT_OCTET32_LEN]

typedef uint8_t **BD_ADDR**[BD_ADDR_LEN]

typedef uint8_t **esp_ble_mesh_bd_addr_t**[BD_ADDR_LEN]

typedef uint8_t **esp_ble_mesh_addr_type_t**

BLE device address type.

typedef struct *esp_ble_mesh_model* **esp_ble_mesh_model_t**

typedef uint8_t **esp_ble_mesh_dev_add_flag_t**

typedef uint32_t **esp_ble_mesh_opcode_config_client_get_t**

esp_ble_mesh_opcode_config_client_get_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by esp_ble_mesh_config_client_get_state. The following opcodes will only be used in the esp_ble_mesh_config_client_get_state function.

typedef uint32_t **esp_ble_mesh_opcode_config_client_set_t**

esp_ble_mesh_opcode_config_client_set_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by esp_ble_mesh_config_client_set_state. The following opcodes will only be used in the esp_ble_mesh_config_client_set_state function.

typedef uint32_t **esp_ble_mesh_opcode_config_status_t**

esp_ble_mesh_opcode_config_status_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by the Config Model messages The following opcodes are used by the BLE Mesh Config Server Model internally to respond to the Config Client Model's request messages.

typedef uint8_t **esp_ble_mesh_cfg_status_t**

This typedef is only used to indicate the status code contained in some of the Configuration Server Model status message.

typedef uint32_t **esp_ble_mesh_opcode_health_client_get_t**

esp_ble_mesh_opcode_health_client_get_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by esp_ble_mesh_health_client_get_state. The following opcodes will only be used in the esp_ble_mesh_health_client_get_state function.

typedef uint32_t **esp_ble_mesh_opcode_health_client_set_t**

esp_ble_mesh_opcode_health_client_set_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by esp_ble_mesh_health_client_set_state. The following opcodes will only be used in the esp_ble_mesh_health_client_set_state function.

typedef uint32_t **esp_ble_mesh_health_model_status_t**

esp_ble_mesh_health_model_status_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by the Health Model messages. The following opcodes are used by the BLE Mesh Health Server Model internally to respond to the Health Client Model's request messages.

typedef uint32_t **esp_ble_mesh_generic_message_opcode_t**

esp_ble_mesh_generic_message_opcode_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by functions esp_ble_mesh_generic_client_get_state & esp_ble_mesh_generic_client_set_state. Generic OnOff Message Opcode

typedef uint32_t **esp_ble_mesh_sensor_message_opcode_t**

esp_ble_mesh_sensor_message_opcode_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by functions esp_ble_mesh_sensor_client_get_state & esp_ble_mesh_sensor_client_set_state. Sensor Message Opcode

typedef uint32_t **esp_ble_mesh_time_scene_message_opcode_t**

esp_ble_mesh_time_scene_message_opcode_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by functions esp_ble_mesh_time_scene_client_get_state & esp_ble_mesh_time_scene_client_set_state. Time Message Opcode

typedef uint32_t **esp_ble_mesh_light_message_opcode_t**

esp_ble_mesh_light_message_opcode_t belongs to esp_ble_mesh_opcode_t, this typedef is only used to locate the opcodes used by functions esp_ble_mesh_light_client_get_state & esp_ble_mesh_light_client_set_state. Light Lightness Message Opcode

typedef uint32_t **esp_ble_mesh_opcode_t**

End of defines of esp_ble_mesh_opcode_t

typedef uint8_t **esp_ble_mesh_model_status_t**

This typedef is only used to indicate the status code contained in some of the server models (e.g. Generic Server Model) status message.

Enumerations

enum **esp_ble_mesh_cb_type_t**

Values:

enumerator **ESP_BLE_MESH_TYPE_PROV_CB**

enumerator **ESP_BLE_MESH_TYPE_OUTPUT_NUM_CB**

enumerator **ESP_BLE_MESH_TYPE_OUTPUT_STR_CB**

enumerator **ESP_BLE_MESH_TYPE_INTPUT_CB**

enumerator **ESP_BLE_MESH_TYPE_LINK_OPEN_CB**

enumerator **ESP_BLE_MESH_TYPE_LINK_CLOSE_CB**

enumerator **ESP_BLE_MESH_TYPE_COMPLETE_CB**

enumerator **ESP_BLE_MESH_TYPE_RESET_CB**

enum **esp_ble_mesh_oob_method_t**

Values:

enumerator **ESP_BLE_MESH_NO_OOB**

enumerator **ESP_BLE_MESH_STATIC_OOB**

enumerator **ESP_BLE_MESH_OUTPUT_OOB**

enumerator **ESP_BLE_MESH_INPUT_OOB**

enum **esp_ble_mesh_output_action_t**

Values:

enumerator **ESP_BLE_MESH_NO_OUTPUT**

enumerator **ESP_BLE_MESH_BLINK**

enumerator **ESP_BLE_MESH_BEEP**

enumerator **ESP_BLE_MESH_VIBRATE**

enumerator **ESP_BLE_MESH_DISPLAY_NUMBER**

enumerator **ESP_BLE_MESH_DISPLAY_STRING**

enum **esp_ble_mesh_input_action_t**

Values:

enumerator **ESP_BLE_MESH_NO_INPUT**

enumerator **ESP_BLE_MESH_PUSH**

enumerator **ESP_BLE_MESH_TWIST**

enumerator **ESP_BLE_MESH_ENTER_NUMBER**

enumerator **ESP_BLE_MESH_ENTER_STRING**

enum **esp_ble_mesh_prov_bearer_t**

Values:

enumerator **ESP_BLE_MESH_PROV_ADV**

enumerator **ESP_BLE_MESH_PROV_GATT**

enum **esp_ble_mesh_prov_oob_info_t**

Values:

enumerator **ESP_BLE_MESH_PROV_OOB_OTHER**

enumerator **ESP_BLE_MESH_PROV_OOB_URI**

enumerator **ESP_BLE_MESH_PROV_OOB_2D_CODE**

enumerator **ESP_BLE_MESH_PROV_OOB_BAR_CODE**

enumerator **ESP_BLE_MESH_PROV_OOB_NFC**

enumerator **ESP_BLE_MESH_PROV_OOB_NUMBER**

enumerator **ESP_BLE_MESH_PROV_OOB_STRING**

enumerator **ESP_BLE_MESH_PROV_CERT_BASED**

enumerator **ESP_BLE_MESH_PROV_RECORDS**

enumerator **ESP_BLE_MESH_PROV_OOB_ON_BOX**

enumerator **ESP_BLE_MESH_PROV_OOB_IN_BOX**

enumerator **ESP_BLE_MESH_PROV_OOB_ON_PAPER**

enumerator **ESP_BLE_MESH_PROV_OOB_IN_MANUAL**

enumerator **ESP_BLE_MESH_PROV_OOB_ON_DEV**

enum **esp_ble_mesh_dev_role_t**

Values:

enumerator **ROLE_NODE**

enumerator **ROLE_PROVISIONER**

enumerator **ROLE_FAST_PROV**

enum **esp_ble_mesh_fast_prov_action_t**

Values:

enumerator **FAST_PROV_ACT_NONE**

enumerator **FAST_PROV_ACT_ENTER**

enumerator **FAST_PROV_ACT_SUSPEND**

enumerator **FAST_PROV_ACT_EXIT**

enumerator **FAST_PROV_ACT_MAX**

enum **esp_ble_mesh_proxy_filter_type_t**

Values:

enumerator **PROXY_FILTER_WHITELIST**

enumerator **PROXY_FILTER_BLACKLIST**

enum **esp_ble_mesh_prov_cb_event_t**

Values:

enumerator **ESP_BLE_MESH_PROV_REGISTER_COMP_EVT**

Initialize BLE Mesh provisioning capabilities and internal data information completion event

enumerator **ESP_BLE_MESH_NODE_SET_UNPROV_DEV_NAME_COMP_EVT**

Set the unprovisioned device name completion event

enumerator **ESP_BLE_MESH_NODE_PROV_ENABLE_COMP_EVT**

Enable node provisioning functionality completion event

enumerator **ESP_BLE_MESH_NODE_PROV_DISABLE_COMP_EVT**

Disable node provisioning functionality completion event

enumerator **ESP_BLE_MESH_NODE_PROV_LINK_OPEN_EVT**

Establish a BLE Mesh link event

enumerator **ESP_BLE_MESH_NODE_PROV_LINK_CLOSE_EVT**

Close a BLE Mesh link event

enumerator **ESP_BLE_MESH_NODE_PROV_OOB_PUB_KEY_EVT**

Generate Node input OOB public key event

enumerator **ESP_BLE_MESH_NODE_PROV_OUTPUT_NUMBER_EVT**

Generate Node Output Number event

enumerator **ESP_BLE_MESH_NODE_PROV_OUTPUT_STRING_EVT**

Generate Node Output String event

enumerator **ESP_BLE_MESH_NODE_PROV_INPUT_EVT**

Event requiring the user to input a number or string

enumerator **ESP_BLE_MESH_NODE_PROV_COMPLETE_EVT**

Provisioning done event

enumerator **ESP_BLE_MESH_NODE_PROV_RESET_EVT**

Provisioning reset event

enumerator **ESP_BLE_MESH_NODE_PROV_SET_OOB_PUB_KEY_COMP_EVT**

Node set oob public key completion event

enumerator **ESP_BLE_MESH_NODE_PROV_INPUT_NUMBER_COMP_EVT**

Node input number completion event

enumerator **ESP_BLE_MESH_NODE_PROV_INPUT_STRING_COMP_EVT**

Node input string completion event

enumerator **ESP_BLE_MESH_NODE_PROXY_IDENTITY_ENABLE_COMP_EVT**

Enable BLE Mesh Proxy Identity advertising completion event

enumerator **ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_ENABLE_COMP_EVT**

Enable BLE Mesh Private Proxy Identity advertising completion event

enumerator **ESP_BLE_MESH_NODE_PRIVATE_PROXY_IDENTITY_DISABLE_COMP_EVT**

Disable BLE Mesh Private Proxy Identity advertising completion event

enumerator **ESP_BLE_MESH_NODE_PROXY_GATT_ENABLE_COMP_EVT**

Enable BLE Mesh GATT Proxy Service completion event

enumerator **ESP_BLE_MESH_NODE_PROXY_GATT_DISABLE_COMP_EVT**

Disable BLE Mesh GATT Proxy Service completion event

enumerator **ESP_BLE_MESH_NODE_ADD_LOCAL_NET_KEY_COMP_EVT**
Node add NetKey locally completion event

enumerator **ESP_BLE_MESH_NODE_ADD_LOCAL_APP_KEY_COMP_EVT**
Node add AppKey locally completion event

enumerator **ESP_BLE_MESH_NODE_BIND_APP_KEY_TO_MODEL_COMP_EVT**
Node bind AppKey to model locally completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_ENABLE_COMP_EVT**
Provisioner enable provisioning functionality completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_DISABLE_COMP_EVT**
Provisioner disable provisioning functionality completion event

enumerator **ESP_BLE_MESH_PROVISIONER_RECV_UNPROV_ADV_PKT_EVT**
Provisioner receives unprovisioned device beacon event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_EVT**
Provisioner read unprovisioned device OOB public key event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_INPUT_EVT**
Provisioner input value for provisioning procedure event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_OUTPUT_EVT**
Provisioner output value for provisioning procedure event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_LINK_OPEN_EVT**
Provisioner establish a BLE Mesh link event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_LINK_CLOSE_EVT**
Provisioner close a BLE Mesh link event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT**
Provisioner provisioning done event

enumerator **ESP_BLE_MESH_PROVISIONER_CERT_BASED_PROV_START_EVT**
Provisioner initiate a certificate based provisioning

enumerator **ESP_BLE_MESH_PROVISIONER_RECV_PROV_RECORDS_LIST_EVT**
Provisioner receive provisioning records list event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_RECORD_RECV_COMP_EVT**
Provisioner receive provisioning record complete event

enumerator **ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORDS_GET_EVT**
Provisioner send provisioning records get to device event

enumerator **ESP_BLE_MESH_PROVISIONER_SEND_PROV_RECORD_REQUEST_EVT**

Provisioner send provisioning record request to device event

enumerator **ESP_BLE_MESH_PROVISIONER_SEND_PROV_INVITE_EVT**

Provisioner send provisioning invite to device event

enumerator **ESP_BLE_MESH_PROVISIONER_SEND_LINK_CLOSE_EVT**

Provisioner send link close to device event

enumerator **ESP_BLE_MESH_PROVISIONER_ADD_UNPROV_DEV_COMP_EVT**

Provisioner add a device to the list which contains devices that are waiting/going to be provisioned completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_DEV_WITH_ADDR_COMP_EVT**

Provisioner start to provision an unprovisioned device completion event

enumerator **ESP_BLE_MESH_PROVISIONER_DELETE_DEV_COMP_EVT**

Provisioner delete a device from the list, close provisioning link with the device completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_DEV_UUID_MATCH_COMP_EVT**

Provisioner set the value to be compared with part of the unprovisioned device UUID completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_PROV_DATA_INFO_COMP_EVT**

Provisioner set net_idx/flags/iv_index used for provisioning completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_STATIC_OOB_VALUE_COMP_EVT**

Provisioner set static oob value used for provisioning completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_PRIMARY_ELEM_ADDR_COMP_EVT**

Provisioner set unicast address of primary element completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_READ_OOB_PUB_KEY_COMP_EVT**

Provisioner read unprovisioned device OOB public key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_INPUT_NUMBER_COMP_EVT**

Provisioner input number completion event

enumerator **ESP_BLE_MESH_PROVISIONER_PROV_INPUT_STRING_COMP_EVT**

Provisioner input string completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_NODE_NAME_COMP_EVT**

Provisioner set node name completion event

enumerator **ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_APP_KEY_COMP_EVT**

Provisioner add local app key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_APP_KEY_COMP_EVT**

Provisioner update local app key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_BIND_APP_KEY_TO_MODEL_COMP_EVT**
Provisioner bind local model with local app key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_ADD_LOCAL_NET_KEY_COMP_EVT**
Provisioner add local network key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_UPDATE_LOCAL_NET_KEY_COMP_EVT**
Provisioner update local network key completion event

enumerator **ESP_BLE_MESH_PROVISIONER_STORE_NODE_COMP_DATA_COMP_EVT**
Provisioner store node composition data completion event

enumerator **ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_UUID_COMP_EVT**
Provisioner delete node with uuid completion event

enumerator **ESP_BLE_MESH_PROVISIONER_DELETE_NODE_WITH_ADDR_COMP_EVT**
Provisioner delete node with unicast address completion event

enumerator **ESP_BLE_MESH_PROVISIONER_ENABLE_HEARTBEAT_RECV_COMP_EVT**
Provisioner start to receive heartbeat message completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_TYPE_COMP_EVT**
Provisioner set the heartbeat filter type completion event

enumerator **ESP_BLE_MESH_PROVISIONER_SET_HEARTBEAT_FILTER_INFO_COMP_EVT**
Provisioner set the heartbeat filter information completion event

enumerator **ESP_BLE_MESH_PROVISIONER_RECV_HEARTBEAT_MESSAGE_EVT**
Provisioner receive heartbeat message event

enumerator **ESP_BLE_MESH_PROVISIONER_DIRECT_ERASE_SETTINGS_COMP_EVT**
Provisioner directly erase settings completion event

enumerator **ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_INDEX_COMP_EVT**
Provisioner open settings with index completion event

enumerator **ESP_BLE_MESH_PROVISIONER_OPEN_SETTINGS_WITH_UID_COMP_EVT**
Provisioner open settings with user id completion event

enumerator **ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_INDEX_COMP_EVT**
Provisioner close settings with index completion event

enumerator **ESP_BLE_MESH_PROVISIONER_CLOSE_SETTINGS_WITH_UID_COMP_EVT**
Provisioner close settings with user id completion event

enumerator **ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_INDEX_COMP_EVT**
Provisioner delete settings with index completion event

enumerator **ESP_BLE_MESH_PROVISIONER_DELETE_SETTINGS_WITH_UID_COMP_EVT**

Provisioner delete settings with user id completion event

enumerator **ESP_BLE_MESH_SET_FAST_PROV_INFO_COMP_EVT**

Set fast provisioning information (e.g. unicast address range, net_idx, etc.) completion event

enumerator **ESP_BLE_MESH_SET_FAST_PROV_ACTION_COMP_EVT**

Set fast provisioning action completion event

enumerator **ESP_BLE_MESH_HEARTBEAT_MESSAGE_RECV_EVT**

Receive Heartbeat message event

enumerator **ESP_BLE_MESH_LPN_ENABLE_COMP_EVT**

Enable Low Power Node completion event

enumerator **ESP_BLE_MESH_LPN_DISABLE_COMP_EVT**

Disable Low Power Node completion event

enumerator **ESP_BLE_MESH_LPN_POLL_COMP_EVT**

Low Power Node send Friend Poll completion event

enumerator **ESP_BLE_MESH_LPN_FRIENDSHIP_ESTABLISH_EVT**

Low Power Node establishes friendship event

enumerator **ESP_BLE_MESH_LPN_FRIENDSHIP_TERMINATE_EVT**

Low Power Node terminates friendship event

enumerator **ESP_BLE_MESH_FRIEND_FRIENDSHIP_ESTABLISH_EVT**

Friend Node establishes friendship event

enumerator **ESP_BLE_MESH_FRIEND_FRIENDSHIP_TERMINATE_EVT**

Friend Node terminates friendship event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_RECV_ADV_PKT_EVT**

Proxy Client receives Network ID advertising packet event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_CONNECTED_EVT**

Proxy Client establishes connection successfully event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_DISCONNECTED_EVT**

Proxy Client terminates connection successfully event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_RECV_FILTER_STATUS_EVT**

Proxy Client receives Proxy Filter Status event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_CONNECT_COMP_EVT**

Proxy Client connect completion event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_DISCONNECT_COMP_EVT**

Proxy Client disconnect completion event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_SET_FILTER_TYPE_COMP_EVT**

Proxy Client set filter type completion event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_ADD_FILTER_ADDR_COMP_EVT**

Proxy Client add filter address completion event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_REMOVE_FILTER_ADDR_COMP_EVT**

Proxy Client remove filter address completion event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_DIRECTED_PROXY_SET_COMP_EVT**

Proxy Client directed proxy set completion event

enumerator **ESP_BLE_MESH_PROXY_SERVER_CONNECTED_EVT**

Proxy Server establishes connection successfully event

enumerator **ESP_BLE_MESH_PROXY_SERVER_DISCONNECTED_EVT**

Proxy Server terminates connection successfully event

enumerator **ESP_BLE_MESH_PROXY_CLIENT_SEND_SOLIC_PDU_COMP_EVT**

Proxy Client send Solicitation PDU completion event

enumerator **ESP_BLE_MESH_MODEL_SUBSCRIBE_GROUP_ADDR_COMP_EVT**

Local model subscribes group address completion event

enumerator **ESP_BLE_MESH_MODEL_UNSUBSCRIBE_GROUP_ADDR_COMP_EVT**

Local model unsubscribes group address completion event

enumerator **ESP_BLE_MESH_DEINIT_MESH_COMP_EVT**

De-initialize BLE Mesh stack completion event

enumerator **ESP_BLE_MESH_PROV_EVT_MAX**

enum **[anonymous]**

BLE Mesh server models related definitions.

This enum value is the flag of transition timer operation

Values:

enumerator **ESP_BLE_MESH_SERVER_TRANS_TIMER_START**

enumerator **ESP_BLE_MESH_SERVER_FLAG_MAX**

enum **esp_ble_mesh_server_state_type_t**

This enum value is the type of server model states

Values:

enumerator **ESP_BLE_MESH_GENERIC_ONOFF_STATE**

enumerator **ESP_BLE_MESH_GENERIC_LEVEL_STATE**

enumerator **ESP_BLE_MESH_GENERIC_ONPOWERUP_STATE**

enumerator **ESP_BLE_MESH_GENERIC_POWER_ACTUAL_STATE**

enumerator **ESP_BLE_MESH_LIGHT_LIGHTNESS_ACTUAL_STATE**

enumerator **ESP_BLE_MESH_LIGHT_LIGHTNESS_LINEAR_STATE**

enumerator **ESP_BLE_MESH_LIGHT_CTL_LIGHTNESS_STATE**

enumerator **ESP_BLE_MESH_LIGHT_CTL_TEMP_DELTA_UV_STATE**

enumerator **ESP_BLE_MESH_LIGHT_HSL_STATE**

enumerator **ESP_BLE_MESH_LIGHT_HSL_LIGHTNESS_STATE**

enumerator **ESP_BLE_MESH_LIGHT_HSL_HUE_STATE**

enumerator **ESP_BLE_MESH_LIGHT_HSL_SATURATION_STATE**

enumerator **ESP_BLE_MESH_LIGHT_XYL_LIGHTNESS_STATE**

enumerator **ESP_BLE_MESH_LIGHT_LC_LIGHT_ONOFF_STATE**

enumerator **ESP_BLE_MESH_SERVER_MODEL_STATE_MAX**

enum **esp_ble_mesh_model_cb_event_t**

Values:

enumerator **ESP_BLE_MESH_MODEL_OPERATION_EVT**

User-defined models receive messages from peer devices (e.g. get, set, status, etc) event

enumerator **ESP_BLE_MESH_MODEL_SEND_COMP_EVT**

User-defined models send messages completion event

enumerator **ESP_BLE_MESH_MODEL_PUBLISH_COMP_EVT**

User-defined models publish messages completion event

enumerator **ESP_BLE_MESH_CLIENT_MODEL_RECV_PUBLISH_MSG_EVT**

User-defined client models receive publish messages event

enumerator **ESP_BLE_MESH_CLIENT_MODEL_SEND_TIMEOUT_EVT**

Timeout event for the user-defined client models that failed to receive response from peer server models

enumerator **ESP_BLE_MESH_MODEL_PUBLISH_UPDATE_EVT**

When a model is configured to publish messages periodically, this event will occur during every publish period

enumerator **ESP_BLE_MESH_SERVER_MODEL_UPDATE_STATE_COMP_EVT**

Server models update state value completion event

enumerator **ESP_BLE_MESH_MODEL_EVT_MAX**

ESP-BLE-MESH Core API Reference

This section contains ESP-BLE-MESH Core related APIs, which can be used to initialize ESP-BLE-MESH stack, provision, send/publish messages, etc.

This API reference covers six components:

- *ESP-BLE-MESH Stack Initialization*
- *Reading of Local Data Information*
- *Low Power Operation (Updating)*
- *Send/Publish Messages, add Local AppKey, etc.*
- *ESP-BLE-MESH Node/Provisioner Provisioning*
- *ESP-BLE-MESH GATT Proxy Server*

ESP-BLE-MESH Stack Initialization

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_common_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_common_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_init**(*esp_ble_mesh_prov_t* *prov, *esp_ble_mesh_comp_t* *comp)

Initialize BLE Mesh module. This API initializes provisioning capabilities and composition data information.

Note: After calling this API, the device needs to call esp_ble_mesh_prov_enable() to enable provisioning functionality again.

Parameters

- **prov** -- **[in]** Pointer to the device provisioning capabilities. This pointer must remain valid during the lifetime of the BLE Mesh device.
- **comp** -- **[in]** Pointer to the device composition data information. This pointer must remain valid during the lifetime of the BLE Mesh device.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_deinit** (*esp_ble_mesh_deinit_param_t* *param)

De-initialize BLE Mesh module.

Note:

a. This function shall be invoked after esp_ble_mesh_client_model_deinit().
b. This function is strictly forbidden to run in any BTC Task Context (e.g. registered Mesh Event Callback).

Parameters **param** -- **[in]** Pointer to the structure of BLE Mesh deinit parameters.
Returns ESP_OK on success or error code otherwise.

Reading of Local Data Information

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_local_data_operation_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_local_data_operation_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

int32_t **esp_ble_mesh_get_model_publish_period** (*esp_ble_mesh_model_t* *model)

Get the model publish period, the unit is ms.

Parameters **model** -- **[in]** Model instance pointer.
Returns Publish period value on success, 0 or (negative) error code from errno.h on failure.

uint16_t **esp_ble_mesh_get_primary_element_address** (void)

Get the address of the primary element.

Returns Address of the primary element on success, or ESP_BLE_MESH_ADDR_UNASSIGNED on failure which means the device has not been provisioned.

uint16_t ***esp_ble_mesh_is_model_subscribed_to_group** (*esp_ble_mesh_model_t* *model, uint16_t group_addr)

Check if the model has subscribed to the given group address. Note: E.g., once a status message is received and the destination address is a group address, the model uses this API to check if it is successfully subscribed to the given group address.

Parameters
- **model** -- **[in]** Pointer to the model.
- **group_addr** -- **[in]** Group address.

Returns Pointer to the group address within the Subscription List of the model on success, or NULL on failure which means the model has not subscribed to the given group address. Note: With the pointer to the group address returned, you can reset the group address to 0x0000 in order to unsubscribe the model from the group.

esp_ble_mesh_elem_t ***esp_ble_mesh_find_element**(uint16_t element_addr)

Find the BLE Mesh element pointer via the element address.

Parameters **element_addr** -- **[in]** Element address.
Returns Pointer to the element on success, or NULL on failure.

uint8_t **esp_ble_mesh_get_element_count**(void)

Get the number of elements that have been registered.

Returns Number of elements.

esp_ble_mesh_model_t ***esp_ble_mesh_find_vendor_model**(const *esp_ble_mesh_elem_t* *element, uint16_t company_id, uint16_t model_id)

Find the Vendor specific model with the given element, the company ID and the Vendor Model ID.

Parameters
- **element** -- **[in]** Element to which the model belongs.
- **company_id** -- **[in]** A 16-bit company identifier assigned by the Bluetooth SIG.
- **model_id** -- **[in]** A 16-bit vendor-assigned model identifier.

Returns Pointer to the Vendor Model on success, or NULL on failure which means the Vendor Model is not found.

esp_ble_mesh_model_t ***esp_ble_mesh_find_sig_model**(const *esp_ble_mesh_elem_t* *element, uint16_t model_id)

Find the SIG model with the given element and Model id.

Parameters
- **element** -- **[in]** Element to which the model belongs.
- **model_id** -- **[in]** SIG model identifier.

Returns Pointer to the SIG Model on success, or NULL on failure which means the SIG Model is not found.

const *esp_ble_mesh_comp_t* ***esp_ble_mesh_get_composition_data**(void)

Get the Composition data which has been registered.

Returns Pointer to the Composition data on success, or NULL on failure which means the Composition data is not initialized.

esp_err_t **esp_ble_mesh_model_subscribe_group_addr**(uint16_t element_addr, uint16_t company_id, uint16_t model_id, uint16_t group_addr)

A local model of node or Provisioner subscribes a group address.

Note: This function shall not be invoked before node is provisioned or Provisioner is enabled.

Parameters
- **element_addr** -- **[in]** Unicast address of the element to which the model belongs.
- **company_id** -- **[in]** A 16-bit company identifier.
- **model_id** -- **[in]** A 16-bit model identifier.
- **group_addr** -- **[in]** The group address to be subscribed.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_model_unsubscribe_group_addr**(uint16_t element_addr, uint16_t company_id, uint16_t model_id, uint16_t group_addr)

A local model of node or Provisioner unsubscribes a group address.

Note: This function shall not be invoked before node is provisioned or Provisioner is enabled.

Parameters
- **element_addr** -- **[in]** Unicast address of the element to which the model belongs.
- **company_id** -- **[in]** A 16-bit company identifier.
- **model_id** -- **[in]** A 16-bit model identifier.
- **group_addr** -- **[in]** The subscribed group address.

Returns ESP_OK on success or error code otherwise.

const uint8_t ***esp_ble_mesh_node_get_local_net_key**(uint16_t net_idx)

This function is called by Node to get the local NetKey.

Parameters **net_idx** -- **[in]** NetKey index.
Returns NetKey on success, or NULL on failure.

const uint8_t ***esp_ble_mesh_node_get_local_app_key**(uint16_t app_idx)

This function is called by Node to get the local AppKey.

Parameters **app_idx** -- **[in]** AppKey index.
Returns AppKey on success, or NULL on failure.

esp_err_t **esp_ble_mesh_node_add_local_net_key**(const uint8_t net_key[16], uint16_t net_idx)

This function is called by Node to add a local NetKey.

Note: This function can only be called after the device is provisioned.

Parameters
- **net_key** -- **[in]** NetKey to be added.
- **net_idx** -- **[in]** NetKey Index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_add_local_app_key**(const uint8_t app_key[16], uint16_t net_idx, uint16_t app_idx)

This function is called by Node to add a local AppKey.

Note: The net_idx must be an existing one. This function can only be called after the device is provisioned.

Parameters
- **app_key** -- **[in]** AppKey to be added.
- **net_idx** -- **[in]** NetKey Index.
- **app_idx** -- **[in]** AppKey Index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_bind_app_key_to_local_model**(uint16_t element_addr, uint16_t company_id, uint16_t model_id, uint16_t app_idx)

This function is called by Node to bind AppKey to model locally.

Note: If going to bind app_key with local vendor model, the company_id shall be set to 0xFFFF. This function can only be called after the device is provisioned.

Parameters
- **element_addr** -- **[in]** Node local element address
- **company_id** -- **[in]** Node local company id
- **model_id** -- **[in]** Node local model id
- **app_idx** -- **[in]** Node local appkey index

Returns ESP_OK on success or error code otherwise.

Low Power Operation (Updating)

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_low_power_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_low_power_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_lpn_enable**(void)

Enable BLE Mesh device LPN functionality.

Note: This API enables LPN functionality. Once called, the proper Friend Request will be sent.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_lpn_disable**(bool force)

Disable BLE Mesh device LPN functionality.

Parameters **force** -- **[in]** when disabling LPN functionality, use this flag to indicate whether directly clear corresponding information or just send friend clear to disable it if friendship has already been established.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_lpn_poll**(void)

LPN tries to poll messages from the Friend Node.

Note: The Friend Poll message is sent by a Low Power node to ask the Friend node to send a message that it has stored for the Low Power node. Users can call this API to send Friend Poll message manually. If this API is not invoked, the bottom layer of the Low Power node will send Friend Poll before the PollTimeout timer expires. If the corresponding Friend Update is received and MD is set to 0, which means there are no messages for the Low Power node, then the Low Power node will stop scanning.

Returns ESP_OK on success or error code otherwise.

Send/Publish Messages, Add Local AppKey, Etc.

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_networking_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_networking_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_custom_model_callback**(*esp_ble_mesh_model_cb_t* callback)

Register BLE Mesh callback for user-defined models' operations. This callback can report the following events generated for the user-defined models:

- Call back the messages received by user-defined client and server models to the application layer;
- If users call esp_ble_mesh_server/client_model_send, this callback notifies the application layer of the send_complete event;
- If user-defined client model sends a message that requires response, and the response message is received after the timer expires, the response message will be reported to the application layer as published by a peer device;
- If the user-defined client model fails to receive the response message during a specified period of time, a timeout event will be reported to the application layer.

Note: The client models (i.e. Config Client model, Health Client model, Generic Client models, Sensor Client model, Scene Client model and Lighting Client models) that have been realized internally have their specific register functions. For example, esp_ble_mesh_register_config_client_callback is the register function for Config Client Model.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_model_msg_opcode_init**(uint8_t *data, uint32_t opcode)

Add the message opcode to the beginning of the model message before sending or publishing the model message.

Note: This API is only used to set the opcode of the message.

Parameters

- **data** -- **[in]** Pointer to the message data.
- **opcode** -- **[in]** The message opcode.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_client_model_init**(*esp_ble_mesh_model_t* *model)

Initialize the user-defined client model. All user-defined client models shall call this function to initialize the client model internal data. Node: Before calling this API, the op_pair_size and op_pair variables within the user_data(defined using esp_ble_mesh_client_t_) of the client model need to be initialized.

Parameters **model** -- **[in]** BLE Mesh Client model to which the message belongs.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_client_model_deinit**(*esp_ble_mesh_model_t* *model)

De-initialize the user-defined client model.

Note: This function shall be invoked before esp_ble_mesh_deinit() is called.

Parameters **model** -- **[in]** Pointer of the Client model.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_server_model_send_msg**(*esp_ble_mesh_model_t* *model, *esp_ble_mesh_msg_ctx_t* *ctx, uint32_t opcode, uint16_t length, uint8_t *data)

Send server model messages(such as server model status messages).

Parameters
- **model** -- **[in]** BLE Mesh Server Model to which the message belongs.
- **ctx** -- **[in]** Message context, includes keys, TTL, etc.
- **opcode** -- **[in]** Message opcode.
- **length** -- **[in]** Message length (exclude the message opcode).
- **data** -- **[in]** Parameters of Access Payload (exclude the message opcode) to be sent.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_client_model_send_msg**(*esp_ble_mesh_model_t* *model, *esp_ble_mesh_msg_ctx_t* *ctx, uint32_t opcode, uint16_t length, uint8_t *data, int32_t msg_timeout, bool need_rsp, *esp_ble_mesh_dev_role_t* device_role)

Send client model message (such as model get, set, etc).

Parameters
- **model** -- **[in]** BLE Mesh Client Model to which the message belongs.
- **ctx** -- **[in]** Message context, includes keys, TTL, etc.
- **opcode** -- **[in]** Message opcode.
- **length** -- **[in]** Message length (exclude the message opcode).
- **data** -- **[in]** Parameters of the Access Payload (exclude the message opcode) to be sent.
- **msg_timeout** -- **[in]** Time to get response to the message (in milliseconds).
- **need_rsp** -- **[in]** TRUE if the opcode requires the peer device to reply, FALSE otherwise.
- **device_role** -- **[in]** Role of the device (Node/Provisioner) that sends the message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_model_publish**(*esp_ble_mesh_model_t* *model, uint32_t opcode, uint16_t length, uint8_t *data, *esp_ble_mesh_dev_role_t* device_role)

Send a model publication message.

Note: Before calling this function, the user needs to ensure that the model publication message (*esp_ble_mesh_model_pub_t::msg*) contains a valid message to be sent. And if users want to update the publishing message, this API should be called in ESP_BLE_MESH_MODEL_PUBLISH_UPDATE_EVT with the message updated.

Parameters
- **model** -- **[in]** Mesh (client) Model publishing the message.
- **opcode** -- **[in]** Message opcode.
- **length** -- **[in]** Message length (exclude the message opcode).
- **data** -- **[in]** Parameters of the Access Payload (exclude the message opcode) to be sent.
- **device_role** -- **[in]** Role of the device (node/provisioner) publishing the message of the type esp_ble_mesh_dev_role_t.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_server_model_update_state**(*esp_ble_mesh_model_t* *model, *esp_ble_mesh_server_state_type_t* type, *esp_ble_mesh_server_state_value_t* *value)

Update a server model state value. If the model publication state is set properly (e.g. publish address is set to a valid address), it will publish corresponding status message.

Note: Currently this API is used to update bound state value, not for all server model states.

Parameters
- **model** -- **[in]** Server model which is going to update the state.
- **type** -- **[in]** Server model state type.
- **value** -- **[in]** Server model state value.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_local_reset**(void)

Reset the provisioning procedure of the local BLE Mesh node.

Note: All provisioning information in this node will be deleted and the node needs to be re-provisioned. The API function esp_ble_mesh_node_prov_enable() needs to be called to start a new provisioning procedure.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_node_name**(uint16_t index, const char *name)

This function is called to set the node (provisioned device) name.

Note: index is obtained from the parameters of ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT.

Parameters
- **index** -- **[in]** Index of the node in the node queue.
- **name** -- **[in]** Name (end by '\0') to be set for the node.

Returns ESP_OK on success or error code otherwise.

const char ***esp_ble_mesh_provisioner_get_node_name**(uint16_t index)

This function is called to get the node (provisioned device) name.

Note: index is obtained from the parameters of ESP_BLE_MESH_PROVISIONER_PROV_COMPLETE_EVT.

Parameters **index** -- **[in]** Index of the node in the node queue.
Returns Node name on success, or NULL on failure.

uint16_t **esp_ble_mesh_provisioner_get_node_index**(const char *name)

This function is called to get the node (provisioned device) index.

Parameters **name** -- **[in]** Name of the node (end by '\0').
Returns Node index on success, or an invalid value (0xFFFF) on failure.

esp_err_t **esp_ble_mesh_provisioner_store_node_comp_data**(uint16_t unicast_addr, uint8_t *data, uint16_t length)

This function is called to store the Composition Data of the node.

Parameters
- **unicast_addr** -- **[in]** Element address of the node
- **data** -- **[in]** Pointer of Composition Data
- **length** -- **[in]** Length of Composition Data

Returns ESP_OK on success or error code otherwise.

esp_ble_mesh_node_t ***esp_ble_mesh_provisioner_get_node_with_uuid*** (const uint8_t uuid[16])

This function is called to get the provisioned node information with the node device uuid.

Parameters **uuid** -- **[in]** Device UUID of the node
Returns Pointer of the node info struct or NULL on failure.

esp_ble_mesh_node_t ***esp_ble_mesh_provisioner_get_node_with_addr*** (uint16_t unicast_addr)

This function is called to get the provisioned node information with the node unicast address.

Parameters **unicast_addr** -- **[in]** Unicast address of the node
Returns Pointer of the node info struct or NULL on failure.

esp_ble_mesh_node_t ***esp_ble_mesh_provisioner_get_node_with_name*** (const char *name)

This function is called to get the provisioned node information with the node name.

Parameters **name** -- **[in]** Name of the node (end by '\0').
Returns Pointer of the node info struct or NULL on failure.

uint16_t ***esp_ble_mesh_provisioner_get_prov_node_count*** (void)

This function is called by Provisioner to get provisioned node count.

Returns Number of the provisioned nodes.

const *esp_ble_mesh_node_t* ****esp_ble_mesh_provisioner_get_node_table_entry*** (void)

This function is called by Provisioner to get the entry of the node table.

Note: After invoking the function to get the entry of nodes, users can use the "for" loop combined with the macro CONFIG_BLE_MESH_MAX_PROV_NODES to get each node's information. Before trying to read the node's information, users need to check if the node exists, i.e. if the *(*esp_ble_mesh_node_t* **node) is NULL. For example: ``` const *esp_ble_mesh_node_t* **entry = esp_ble_mesh_provisioner_get_node_table_entry(); for (int i = 0; i < CONFIG_BLE_MESH_MAX_PROV_NODES; i++) { const *esp_ble_mesh_node_t* *node = entry[i]; if (node) { ..... } } ```

Returns Pointer to the start of the node table.

esp_err_t ***esp_ble_mesh_provisioner_delete_node_with_uuid*** (const uint8_t uuid[16])

This function is called to delete the provisioned node information with the node device uuid.

Parameters **uuid** -- **[in]** Device UUID of the node
Returns ESP_OK on success or error code otherwise.

esp_err_t ***esp_ble_mesh_provisioner_delete_node_with_addr*** (uint16_t unicast_addr)

This function is called to delete the provisioned node information with the node unicast address.

Parameters **unicast_addr** -- **[in]** Unicast address of the node
Returns ESP_OK on success or error code otherwise.

esp_err_t ***esp_ble_mesh_provisioner_add_local_app_key*** (const uint8_t app_key[16], uint16_t net_idx, uint16_t app_idx)

This function is called to add a local AppKey for Provisioner.

Note: app_key: If set to NULL, app_key will be generated internally. net_idx: Should be an existing one. app_idx: If it is going to be generated internally, it should be set to 0xFFFF, and the new app_idx will be reported via an event.

Parameters

- **app_key** -- **[in]** The app key to be set for the local BLE Mesh stack.
- **net_idx** -- **[in]** The network key index.

- **app_idx** -- **[in]** The app key index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_update_local_app_key**(const uint8_t app_key[16], uint16_t net_idx, uint16_t app_idx)

This function is used to update a local AppKey for Provisioner.

Parameters

- **app_key** -- **[in]** Value of the AppKey.
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **app_idx** -- **[in]** The AppKey Index

Returns ESP_OK on success or error code otherwise.

const uint8_t ***esp_ble_mesh_provisioner_get_local_app_key**(uint16_t net_idx, uint16_t app_idx)

This function is called by Provisioner to get the local app key value.

Parameters

- **net_idx** -- **[in]** Network key index.
- **app_idx** -- **[in]** Application key index.

Returns App key on success, or NULL on failure.

esp_err_t **esp_ble_mesh_provisioner_bind_app_key_to_local_model**(uint16_t element_addr, uint16_t app_idx, uint16_t model_id, uint16_t company_id)

This function is called by Provisioner to bind own model with proper app key.

Note: company_id: If going to bind app_key with local vendor model, company_id should be set to 0xFFFF.

Parameters

- **element_addr** -- **[in]** Provisioner local element address
- **app_idx** -- **[in]** Provisioner local appkey index
- **model_id** -- **[in]** Provisioner local model id
- **company_id** -- **[in]** Provisioner local company id

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_add_local_net_key**(const uint8_t net_key[16], uint16_t net_idx)

This function is called by Provisioner to add local network key.

Note: net_key: If set to NULL, net_key will be generated internally. net_idx: If it is going to be generated internally, it should be set to 0xFFFF, and the new net_idx will be reported via an event.

Parameters

- **net_key** -- **[in]** The network key to be added to the Provisioner local BLE Mesh stack.
- **net_idx** -- **[in]** The network key index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_update_local_net_key**(const uint8_t net_key[16], uint16_t net_idx)

This function is called by Provisioner to update a local network key.

Parameters

- **net_key** -- **[in]** Value of the NetKey.

- **net_idx** -- **[in]** The NetKey Index.

Returns ESP_OK on success or error code otherwise.

const uint8_t ***esp_ble_mesh_provisioner_get_local_net_key**(uint16_t net_idx)

This function is called by Provisioner to get the local network key value.

Parameters **net_idx** -- **[in]** Network key index.
Returns Network key on success, or NULL on failure.

esp_err_t **esp_ble_mesh_provisioner_recv_heartbeat**(bool enable)

This function is called by Provisioner to enable or disable receiving heartbeat messages.

Note: If enabling receiving heartbeat message successfully, the filter will be an empty rejectlist by default, which means all heartbeat messages received by the Provisioner will be reported to the application layer.

Parameters **enable** -- **[in]** Enable or disable receiving heartbeat messages.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_heartbeat_filter_type**(uint8_t type)

This function is called by Provisioner to set the heartbeat filter type.

Note: 1. If the filter type is not the same with the current value, then all the filter entries will be cleaned.

a. If the previous type is rejectlist, and changed to acceptlist, then the filter will be an empty acceptlist, which means no heartbeat messages will be reported. Users need to add SRC or DST into the filter entry, then heartbeat messages from the SRC or to the DST will be reported.

Parameters **type** -- **[in]** Heartbeat filter type (acceptlist or rejectlist).
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_heartbeat_filter_info**(uint8_t op, *esp_ble_mesh_heartbeat_filter_info_t* *info)

This function is called by Provisioner to add or remove a heartbeat filter entry.

a. If the operation is "REMOVE", the "hb_src" can be set to the SRC (can only be a unicast address) of heartbeat messages, and the "hb_dst" can be set to the DST (unicast address or group address), at least one of them needs to be set.
   - The filter entry with the same SRC or DST will be removed.

Note: 1. If the operation is "ADD", the "hb_src" can be set to the SRC (can only be a unicast address) of heartbeat messages, and the "hb_dst" can be set to the DST (unicast address or group address), at least one of them needs to be set.

- If only one of them is set, the filter entry will only use the configured SRC or DST to filter heartbeat messages.
- If both of them are set, the SRC and DST will both be used to decide if a heartbeat message will be handled.
- If SRC or DST already exists in some filter entry, then the corresponding entry will be cleaned firstly, then a new entry will be allocated to store the information.

Parameters

- **op** -- **[in]** Add or REMOVE

- **info** -- **[in]** Heartbeat filter entry information, including: hb_src - Heartbeat source address; hb_dst - Heartbeat destination address;

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_direct_erase_settings**(void)

This function is called by Provisioner to directly erase the mesh information from nvs namespace.

Note: This function can be invoked when the mesh stack is not initialized or has been de-initialized.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_open_settings_with_index**(uint8_t index)

This function is called by Provisioner to open a nvs namespace for storing mesh information.

Note: Before open another nvs namespace, the previously opened nvs namespace must be closed firstly.

Parameters **index** -- **[in]** Settings index.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_open_settings_with_uid**(const char *uid)

This function is called by Provisioner to open a nvs namespace for storing mesh information.

Note: Before open another nvs namespace, the previously opened nvs namespace must be closed firstly.

Parameters **uid** -- **[in]** Settings user id.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_close_settings_with_index**(uint8_t index, bool erase)

This function is called by Provisioner to close a nvs namespace which is opened previously for storing mesh information.

Note: 1. Before closing the nvs namespace, it must be open.

a. When the function is invoked, the Provisioner functionality will be disabled firstly, and: a) If the "erase" flag is set to false, the mesh information will be cleaned (e.g. removing NetKey, AppKey, nodes, etc) from the mesh stack. b) If the "erase" flag is set to true, the mesh information stored in the nvs namespace will also be erased besides been cleaned from the mesh stack.
b. If Provisioner tries to work properly again, we can invoke the open function to open a new nvs namespace or a previously added one, and restore the mesh information from it if not erased.
c. The working process shall be as following: a) Open settings A b) Start to provision and control nodes c) Close settings A d) Open settings B e) Start to provision and control other nodes f) Close settings B g) .....

Parameters

- **index** -- **[in]** Settings index.
- **erase** -- **[in]** Indicate if erasing mesh information.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_close_settings_with_uid**(const char *uid, bool erase)

This function is called by Provisioner to close a nvs namespace which is opened previously for storing mesh information.

Note: 1. Before closing the nvs namespace, it must be open.

a. When the function is invoked, the Provisioner functionality will be disabled firstly, and: a) If the "erase" flag is set to false, the mesh information will be cleaned (e.g. removing NetKey, AppKey, nodes, etc) from the mesh stack. b) If the "erase" flag is set to true, the mesh information stored in the nvs namespace will also be erased besides been cleaned from the mesh stack.
b. If Provisioner tries to work properly again, we can invoke the open function to open a new nvs namespace or a previously added one, and restore the mesh information from it if not erased.
c. The working process shall be as following: a) Open settings A b) Start to provision and control nodes c) Close settings A d) Open settings B e) Start to provision and control other nodes f) Close settings B g) .....

Parameters
- **uid** -- **[in]** Settings user id.
- **erase** -- **[in]** Indicate if erasing mesh information.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_delete_settings_with_index**(uint8_t index)

This function is called by Provisioner to erase the mesh information and settings user id from a nvs namespace.

Note: When this function is called, the nvs namespace must not be open. This function is used to erase the mesh information and settings user id which are not used currently.

Parameters **index** -- **[in]** Settings index.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_delete_settings_with_uid**(const char *uid)

This function is called by Provisioner to erase the mesh information and settings user id from a nvs namespace.

Note: When this function is called, the nvs namespace must not be open. This function is used to erase the mesh information and settings user id which are not used currently.

Parameters **uid** -- **[in]** Settings user id.
Returns ESP_OK on success or error code otherwise.

const char ***esp_ble_mesh_provisioner_get_settings_uid**(uint8_t index)

This function is called by Provisioner to get settings user id.

Parameters **index** -- **[in]** Settings index.
Returns Setting user id on success or NULL on failure.

uint8_t **esp_ble_mesh_provisioner_get_settings_index**(const char *uid)

This function is called by Provisioner to get settings index.

Parameters **uid** -- **[in]** Settings user id.
Returns Settings index.

uint8_t **esp_ble_mesh_provisioner_get_free_settings_count**(void)

This function is called by Provisioner to get the number of free settings user id.

Returns Number of free settings user id.

const uint8_t *__esp_ble_mesh_get_fast_prov_app_key__(uint16_t net_idx, uint16_t app_idx)

This function is called to get fast provisioning application key.

Parameters
- **net_idx** -- **[in]** Network key index.
- **app_idx** -- **[in]** Application key index.

Returns Application key on success, or NULL on failure.

Type Definitions

typedef void (*__esp_ble_mesh_model_cb_t__)(*esp_ble_mesh_model_cb_event_t* event, *esp_ble_mesh_model_cb_param_t* *param)

: event, event code of user-defined model events; param, parameters of user-defined model events

ESP-BLE-MESH Node/Provisioner Provisioning

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_provisioning_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_provisioning_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t __esp_ble_mesh_register_prov_callback__(*esp_ble_mesh_prov_cb_t* callback)

Register BLE Mesh provisioning callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

bool __esp_ble_mesh_node_is_provisioned__(void)

Check if a device has been provisioned.

Returns TRUE if the device is provisioned, FALSE if the device is unprovisioned.

esp_err_t __esp_ble_mesh_node_prov_enable__(*esp_ble_mesh_prov_bearer_t* bearers)

Enable specific provisioning bearers to get the device ready for provisioning.

Note: PB-ADV: send unprovisioned device beacon. PB-GATT: send connectable advertising packets.

Parameters **bearers** -- Bit-wise OR of provisioning bearers.
Returns ESP_OK on success or error code otherwise.

esp_err_t __esp_ble_mesh_node_prov_disable__(*esp_ble_mesh_prov_bearer_t* bearers)

Disable specific provisioning bearers to make a device inaccessible for provisioning.

Parameters **bearers** -- Bit-wise OR of provisioning bearers.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_set_oob_pub_key**(uint8_t pub_key_x[32], uint8_t pub_key_y[32], uint8_t private_key[32])

Unprovisioned device set own oob public key & private key pair.

Note: In order to avoid suffering brute-forcing attack (CVE-2020-26559). The Bluetooth SIG recommends that potentially vulnerable mesh provisioners use an out-of-band mechanism to exchange the public keys. So as an unprovisioned device, it should use this function to input the Public Key exchanged through the out-of-band mechanism.

Parameters
- **pub_key_x** -- **[in]** Unprovisioned device's Public Key X
- **pub_key_y** -- **[in]** Unprovisioned device's Public Key Y
- **private_key** -- **[in]** Unprovisioned device's Private Key

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_input_number**(uint32_t number)

Provide provisioning input OOB number.

Note: This is intended to be called if the user has received ESP_BLE_MESH_NODE_PROV_INPUT_EVT with ESP_BLE_MESH_ENTER_NUMBER as the action.

Parameters **number** -- **[in]** Number input by device.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_node_input_string**(const char *string)

Provide provisioning input OOB string.

Note: This is intended to be called if the user has received ESP_BLE_MESH_NODE_PROV_INPUT_EVT with ESP_BLE_MESH_ENTER_STRING as the action.

Parameters **string** -- **[in]** String input by device.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_set_unprovisioned_device_name**(const char *name)

Using this function, an unprovisioned device can set its own device name, which will be broadcasted in its advertising data.

Note: This API applicable to PB-GATT mode only by setting the name to the scan response data, it doesn't apply to PB-ADV mode.

Parameters **name** -- **[in]** Unprovisioned device name

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_read_oob_pub_key**(uint8_t link_idx, uint8_t pub_key_x[32], uint8_t pub_key_y[32])

Provisioner inputs unprovisioned device's oob public key.

Note: In order to avoid suffering brute-forcing attack (CVE-2020-26559). The Bluetooth SIG recommends that potentially vulnerable mesh provisioners use an out-of-band mechanism to exchange the public keys.

Parameters
- **link_idx** -- **[in]** The provisioning link index
- **pub_key_x** -- **[in]** Unprovisioned device's Public Key X
- **pub_key_y** -- **[in]** Unprovisioned device's Public Key Y

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_input_string**(const char *string, uint8_t link_idx)

Provide provisioning input OOB string.

```
        This is intended to be called after the esp_ble_mesh_prov_t prov_
↪input_num
        callback has been called with ESP_BLE_MESH_ENTER_STRING as the␣
↪action.
```

Parameters
- **string** -- **[in]** String input by Provisioner.
- **link_idx** -- **[in]** The provisioning link index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_input_number**(uint32_t number, uint8_t link_idx)

Provide provisioning input OOB number.

```
        This is intended to be called after the esp_ble_mesh_prov_t prov_
↪input_num
        callback has been called with ESP_BLE_MESH_ENTER_NUMBER as the␣
↪action.
```

Parameters
- **number** -- **[in]** Number input by Provisioner.
- **link_idx** -- **[in]** The provisioning link index.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_prov_enable**(*esp_ble_mesh_prov_bearer_t* bearers)

Enable one or more provisioning bearers.

Note: PB-ADV: Enable BLE scan. PB-GATT: Initialize corresponding BLE Mesh Proxy info.

Parameters **bearers** -- **[in]** Bit-wise OR of provisioning bearers.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_prov_disable**(*esp_ble_mesh_prov_bearer_t* bearers)

Disable one or more provisioning bearers.

Note: PB-ADV: Disable BLE scan. PB-GATT: Break any existing BLE Mesh Provisioning connections.

Parameters **bearers** -- **[in]** Bit-wise OR of provisioning bearers.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_add_unprov_dev**(*esp_ble_mesh_unprov_dev_add_t* *add_dev, *esp_ble_mesh_dev_add_flag_t* flags)

Add unprovisioned device info to the unprov_dev queue.

Note: : 1. Currently address type only supports public address and static random address.

a. If device UUID and/or device address as well as address type already exist in the device queue, but the bearer is different from the existing one, add operation will also be successful and it will update the provision bearer supported by the device.
b. For example, if the Provisioner wants to add an unprovisioned device info before receiving its unprovisioned device beacon or Mesh Provisioning advertising packets, the Provisioner can use this API to add the device info with each one or both of device UUID and device address added. When the Provisioner gets the device's advertising packets, it will start provisioning the device internally.
   - In this situation, the Provisioner can set bearers with each one or both of ESP_BLE_MESH_PROV_ADV and ESP_BLE_MESH_PROV_GATT enabled, and cannot set flags with ADD_DEV_START_PROV_NOW_FLAG enabled.
c. Another example is when the Provisioner receives the unprovisioned device's beacon or Mesh Provisioning advertising packets, the advertising packets will be reported on to the application layer using the callback registered by the function esp_ble_mesh_register_prov_callback. And in the callback, the Provisioner can call this API to start provisioning the device.
   - If the Provisioner uses PB-ADV to provision, either one or both of device UUID and device address can be added, bearers shall be set with ESP_BLE_MESH_PROV_ADV enabled and the flags shall be set with ADD_DEV_START_PROV_NOW_FLAG enabled.
   - If the Provisioner uses PB-GATT to provision, both the device UUID and device address need to be added, bearers shall be set with ESP_BLE_MESH_PROV_GATT enabled, and the flags shall be set with ADD_DEV_START_PROV_NOW_FLAG enabled.
   - If the Provisioner just wants to store the unprovisioned device info when receiving its advertising packets and start to provision it the next time (e.g. after receiving its advertising packets again), then it can add the device info with either one or both of device UUID and device address included. Bearers can be set with either one or both of ESP_BLE_MESH_PROV_ADV and ESP_BLE_MESH_PROV_GATT enabled (recommend to enable the bearer which will receive its advertising packets, because if the other bearer is enabled, the Provisioner is not aware if the device supports the bearer), and flags cannot be set with ADD_DEV_START_PROV_NOW_FLAG enabled.
   - Note: ESP_BLE_MESH_PROV_ADV, ESP_BLE_MESH_PROV_GATT and ADD_DEV_START_PROV_NOW_FLAG can not be enabled at the same time.

Parameters
- **`add_dev`** -- **[in]** Pointer to a struct containing the device information
- **`flags`** -- **[in]** Flags indicate several operations on the device information
  - Remove device information from queue after device has been provisioned (BIT0)
  - Start provisioning immediately after device is added to queue (BIT1)
  - Device can be removed if device queue is full (BIT2)

Returns ESP_OK on success or error code otherwise.

esp_err_t **`esp_ble_mesh_provisioner_prov_device_with_addr`**(const uint8_t uuid[16], *esp_ble_mesh_bd_addr_t* addr, *esp_ble_mesh_addr_type_t* addr_type, *esp_ble_mesh_prov_bearer_t* bearer, uint16_t oob_info, uint16_t unicast_addr)

Provision an unprovisioned device and assign a fixed unicast address for it in advance.

Note: : 1. Currently address type only supports public address and static random address.

a. Bearer must be equal to ESP_BLE_MESH_PROV_ADV or ESP_BLE_MESH_PROV_GATT, since Provisioner will start to provision a device immediately once this function is invoked. And the input bearer must be identical with the one within the parameters of the ESP_BLE_MESH_PROVISIONER_RECV_UNPROV_ADV_PKT_EVT event.
b. If this function is used by a Provisioner to provision devices, the application should take care of the assigned unicast address and avoid overlap of the unicast addresses of different nodes.

c. Recommend to use only one of the functions "esp_ble_mesh_provisioner_add_unprov_dev" and "esp_ble_mesh_provisioner_prov_device_with_addr" by a Provisioner.

Parameters
- **uuid** -- **[in]** Device UUID of the unprovisioned device
- **addr** -- **[in]** Device address of the unprovisioned device
- **addr_type** -- **[in]** Device address type of the unprovisioned device
- **bearer** -- **[in]** Provisioning bearer going to be used by Provisioner
- **oob_info** -- **[in]** OOB info of the unprovisioned device
- **unicast_addr** -- **[in]** Unicast address going to be allocated for the unprovisioned device

Returns Zero on success or (negative) error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_delete_dev**(*esp_ble_mesh_device_delete_t* *del_dev)

Delete device from queue, and reset current provisioning link with the device.

Note: If the device is in the queue, remove it from the queue; if the device is being provisioned, terminate the provisioning procedure. Either one of the device address or device UUID can be used as input.

Parameters **del_dev** -- **[in]** Pointer to a struct containing the device information.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_dev_uuid_match**(const uint8_t *match_val, uint8_t match_len, uint8_t offset, bool prov_after_match)

This function is called by Provisioner to set the part of the device UUID to be compared before starting to provision.

Parameters
- **match_val** -- **[in]** Value to be compared with the part of the device UUID.
- **match_len** -- **[in]** Length of the compared match value.
- **offset** -- **[in]** Offset of the device UUID to be compared (based on zero).
- **prov_after_match** -- **[in]** Flag used to indicate whether provisioner should start to provision the device immediately if the part of the UUID matches.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_prov_data_info**(*esp_ble_mesh_prov_data_info_t* *prov_data_info)

This function is called by Provisioner to set provisioning data information before starting to provision.

Parameters **prov_data_info** -- **[in]** Pointer to a struct containing net_idx or flags or iv_index.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_static_oob_value**(const uint8_t *value, uint8_t length)

This function is called by Provisioner to set static oob value used for provisioning.

AuthValues selected using a cryptographically secure random or pseudorandom number generator and having the maximum permitted entropy (128-bits) will be most difficult to brute-force. AuthValues with reduced entropy or generated in a predictable manner will not grant the same level of protection against this vulnerability. Selecting a new AuthValue with each provisioning attempt can also make it more difficult to launch a brute-force attack by requiring the attacker to restart the search with each provisioning attempt (CVE-2020-26556).

Note: The Bluetooth SIG recommends that mesh implementations enforce a randomly selected AuthValue using all of the available bits, where permitted by the implementation. A large entropy helps ensure that a

brute-force of the AuthValue, even a static AuthValue, cannot normally be completed in a reasonable time (CVE-2020-26557).

Parameters
- **value** -- **[in]** Pointer to the static oob value.
- **length** -- **[in]** Length of the static oob value.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_provisioner_set_primary_elem_addr**(uint16_t addr)

This function is called by Provisioner to set own Primary element address.

Note: This API must be invoked when BLE Mesh initialization is completed successfully, and can be invoked before Provisioner functionality is enabled. Once this API is invoked successfully, the prov_unicast_addr value in the struct *esp_ble_mesh_prov_t* will be ignored, and Provisioner will use this address as its own primary element address. And if the unicast address going to assigned for the next unprovisioned device is smaller than the input address + element number of Provisioner, then the address for the next unprovisioned device will be recalculated internally.

Parameters **addr** -- **[in]** Unicast address of the Primary element of Provisioner.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_set_fast_prov_info**(*esp_ble_mesh_fast_prov_info_t* *fast_prov_info)

This function is called to set provisioning data information before starting fast provisioning.

Parameters **fast_prov_info** -- **[in]** Pointer to a struct containing unicast address range, net_idx, etc.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_set_fast_prov_action**(*esp_ble_mesh_fast_prov_action_t* action)

This function is called to start/suspend/exit fast provisioning.

Parameters **action** -- **[in]** fast provisioning action (i.e. enter, suspend, exit).
Returns ESP_OK on success or error code otherwise.

Type Definitions

typedef void (***esp_ble_mesh_prov_cb_t**)(*esp_ble_mesh_prov_cb_event_t* event, *esp_ble_mesh_prov_cb_param_t* *param)

: event, event code of provisioning events; param, parameters of provisioning events

typedef void (***esp_ble_mesh_prov_adv_cb_t**)(const *esp_ble_mesh_bd_addr_t* addr, const *esp_ble_mesh_addr_type_t* addr_type, const uint8_t adv_type, const uint8_t *dev_uuid, uint16_t oob_info, *esp_ble_mesh_prov_bearer_t* bearer)

Callback for Provisioner that received advertising packets from unprovisioned devices which are not in the unprovisioned device queue.

Report on the unprovisioned device beacon and mesh provisioning service adv data to application.

Param addr **[in]** Pointer to the unprovisioned device address.
Param addr_type **[in]** Unprovisioned device address type.
Param adv_type **[in]** Adv packet type(ADV_IND or ADV_NONCONN_IND).
Param dev_uuid **[in]** Unprovisioned device UUID pointer.
Param oob_info **[in]** OOB information of the unprovisioned device.
Param bearer **[in]** Adv packet received from PB-GATT or PB-ADV bearer.

ESP-BLE-MESH GATT Proxy Server

Header File

- components/bt/esp_ble_mesh/api/core/include/esp_ble_mesh_proxy_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_proxy_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_proxy_identity_enable**(void)

Enable advertising with Node Identity.

Note: This API requires that GATT Proxy support be enabled. Once called, each subnet starts advertising using Node Identity for the next 60 seconds, and after 60s Network ID will be advertised. Under normal conditions, the BLE Mesh Proxy Node Identity and Network ID advertising will be enabled automatically by BLE Mesh stack after the device is provisioned.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_gatt_enable**(void)

Enable BLE Mesh GATT Proxy Service.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_gatt_disable**(void)

Disconnect the BLE Mesh GATT Proxy connection if there is any, and disable the BLE Mesh GATT Proxy Service.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_private_proxy_identity_enable**(void)

Enable advertising with Private Node Identity.

Note: This API requires that GATT Proxy support be enabled. Once called, each subnet starts advertising using Private Node Identity for the next 60 seconds, and after 60s Private Network ID will be advertised. Under normal conditions, the BLE Mesh Proxy Node Identity, Network ID advertising, Proxy Private Node Identity and Private Network ID advertising will be enabled automatically by BLE Mesh stack after the device is provisioned.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_private_proxy_identity_disable**(void)

Disable advertising with Private Node Identity.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_connect**(*esp_ble_mesh_bd_addr_t* addr, *esp_ble_mesh_addr_type_t* addr_type, uint16_t net_idx)

Proxy Client creates a connection with the Proxy Server.

Parameters
- **addr** -- **[in]** Device address of the Proxy Server.
- **addr_type** -- **[in]** Device address type(public or static random).
- **net_idx** -- **[in]** NetKey Index related with Network ID in the Mesh Proxy advertising packet.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_disconnect**(uint8_t conn_handle)

Proxy Client terminates a connection with the Proxy Server.

Parameters **conn_handle** -- **[in]** Proxy connection handle.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_set_filter_type**(uint8_t conn_handle, uint16_t net_idx, *esp_ble_mesh_proxy_filter_type_t* filter_type)

Proxy Client sets the filter type of the Proxy Server.

Parameters
- **conn_handle** -- **[in]** Proxy connection handle.
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **filter_type** -- **[in]** whitelist or blacklist.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_add_filter_addr**(uint8_t conn_handle, uint16_t net_idx, uint16_t *addr, uint16_t addr_num)

Proxy Client adds address to the Proxy Server filter list.

Parameters
- **conn_handle** -- **[in]** Proxy connection handle.
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **addr** -- **[in]** Pointer to the filter address.
- **addr_num** -- **[in]** Number of the filter address.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_remove_filter_addr**(uint8_t conn_handle, uint16_t net_idx, uint16_t *addr, uint16_t addr_num)

Proxy Client removes address from the Proxy Server filter list.

Parameters
- **conn_handle** -- **[in]** Proxy connection handle.
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **addr** -- **[in]** Pointer to the filter address.
- **addr_num** -- **[in]** Number of the filter address.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_directed_proxy_set**(uint8_t conn_handle, uint16_t net_idx, uint8_t use_directed)

Proxy Client sets whether or not the Directed Proxy Server uses directed forwarding for Directed Proxy Client messages.

Parameters
- **conn_handle** -- **[in]** Proxy connection handle.
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **use_directed** -- **[in]** Whether or not to send message by directed forwarding.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_proxy_client_send_solic_pdu**(uint8_t net_idx, uint16_t ssrc, uint16_t dst)

Proxy Client sends Solicitation PDU.

Parameters
- **net_idx** -- **[in]** Corresponding NetKey Index.
- **ssrc** -- **[in]** Solicitation SRC, shall be one of its element address.
- **dst** -- **[in]** Solicitation DST (TBD).

Returns ESP_OK on success or error code otherwise.

Macros

ESP_BLE_MESH_PROXY_CLI_DIRECTED_FORWARDING_ENABLE

ESP_BLE_MESH_PROXY_CLI_DIRECTED_FORWARDING_DISABLE

ESP-BLE-MESH Models API Reference

This section contains ESP-BLE-MESH Model related APIs, event types, event parameters, etc.

There are six categories of models:

- *Configuration Client/Server Models*
- *Health Client/Server Models*
- *Generic Client/Server Models*
- *Sensor Client/Server Models*
- *Time and Scenes Client/Server Models*
- *Lighting Client/Server Models*

Note: Definitions related to Server Models are being updated, and will be released soon.

Configuration Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_config_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_config_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_config_client_callback**(*esp_ble_mesh_cfg_client_cb_t* callback)

Register BLE Mesh Config Client Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_config_server_callback**(*esp_ble_mesh_cfg_server_cb_t* callback)

Register BLE Mesh Config Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_config_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_cfg_client_get_state_t* *get_state)

Get the value of Config Server Model states using the Config Client Model get messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_opcode_config_client_get_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_config_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_cfg_client_set_state_t* *set_state)

Set the value of the Configuration Server Model states using the Config Client Model set messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_opcode_config_client_set_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_cfg_client_get_state_t**

#include <esp_ble_mesh_config_model_api.h> For ESP_BLE_MESH_MODEL_OP_BEACON_GET ESP_BLE_MESH_MODEL_OP_COMPOSITION_DATA_GET ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_GET ESP_BLE_MESH_MODEL_OP_GATT_PROXY_GET ESP_BLE_MESH_MODEL_OP_RELAY_GET ESP_BLE_MESH_MODEL_OP_MODEL_PUB_GET ESP_BLE_MESH_MODEL_OP_FRIEND_GET ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_GET ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_GET the get_state parameter in the esp_ble_mesh_config_client_get_state function should not be set to NULL.

Public Members

esp_ble_mesh_cfg_model_pub_get_t **model_pub_get**

For ESP_BLE_MESH_MODEL_OP_MODEL_PUB_GET.

esp_ble_mesh_cfg_composition_data_get_t **comp_data_get**

For ESP_BLE_MESH_MODEL_OP_COMPOSITION_DATA_GET.

esp_ble_mesh_cfg_sig_model_sub_get_t **sig_model_sub_get**

For ESP_BLE_MESH_MODEL_OP_SIG_MODEL_SUB_GET

esp_ble_mesh_cfg_vnd_model_sub_get_t **vnd_model_sub_get**

For ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_SUB_GET

esp_ble_mesh_cfg_app_key_get_t **app_key_get**

For ESP_BLE_MESH_MODEL_OP_APP_KEY_GET.

esp_ble_mesh_cfg_node_identity_get_t **node_identity_get**

For ESP_BLE_MESH_MODEL_OP_NODE_IDENTITY_GET.

esp_ble_mesh_cfg_sig_model_app_get_t **sig_model_app_get**

For ESP_BLE_MESH_MODEL_OP_SIG_MODEL_APP_GET

esp_ble_mesh_cfg_vnd_model_app_get_t **vnd_model_app_get**

For ESP_BLE_MESH_MODEL_OP_VENDOR_MODEL_APP_GET

esp_ble_mesh_cfg_kr_phase_get_t **kr_phase_get**

For ESP_BLE_MESH_MODEL_OP_KEY_REFRESH_PHASE_GET

esp_ble_mesh_cfg_lpn_polltimeout_get_t **lpn_pollto_get**

For ESP_BLE_MESH_MODEL_OP_LPN_POLLTIMEOUT_GET

union **esp_ble_mesh_cfg_client_set_state_t**

#include <esp_ble_mesh_config_model_api.h> For ESP_BLE_MESH_MODEL_OP_BEACON_SET ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_SET ESP_BLE_MESH_MODEL_OP_GATT_PROXY_SET ESP_BLE_MESH_MODEL_OP_RELAY_SET ESP_BLE_MESH_MODEL_OP_MODEL_PUB_SET ESP_BLE_MESH_MODEL_OP_MODEL_SUB_ADD ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR ESP_BLE_MESH_MODEL_OP_MODEL_SUB_DELETE ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_AD ESP_BLE_MESH_MODEL_OP_MODEL_SUB_OVERWRITE ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUA ESP_BLE_MESH_MODEL_OP_NET_KEY_ADD ESP_BLE_MESH_MODEL_OP_APP_KEY_ADD ESP_BLE_MESH_MODEL_OP_MODEL_APP_BIND ESP_BLE_MESH_MODEL_OP_NODE_RESET ESP_BLE_MESH_MODEL_OP_FRIEND_SET ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_SET ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_SET the set_state parameter in the esp_ble_mesh_config_client_set_state function should not be set to NULL.

Public Members

esp_ble_mesh_cfg_beacon_set_t **beacon_set**

For ESP_BLE_MESH_MODEL_OP_BEACON_SET

esp_ble_mesh_cfg_default_ttl_set_t **default_ttl_set**

For ESP_BLE_MESH_MODEL_OP_DEFAULT_TTL_SET

esp_ble_mesh_cfg_friend_set_t **friend_set**

For ESP_BLE_MESH_MODEL_OP_FRIEND_SET

esp_ble_mesh_cfg_gatt_proxy_set_t **gatt_proxy_set**

For ESP_BLE_MESH_MODEL_OP_GATT_PROXY_SET

esp_ble_mesh_cfg_relay_set_t **relay_set**

For ESP_BLE_MESH_MODEL_OP_RELAY_SET

esp_ble_mesh_cfg_net_key_add_t **net_key_add**

For ESP_BLE_MESH_MODEL_OP_NET_KEY_ADD

esp_ble_mesh_cfg_app_key_add_t **app_key_add**

For ESP_BLE_MESH_MODEL_OP_APP_KEY_ADD

esp_ble_mesh_cfg_model_app_bind_t **model_app_bind**

For ESP_BLE_MESH_MODEL_OP_MODEL_APP_BIND

esp_ble_mesh_cfg_model_pub_set_t **model_pub_set**

For ESP_BLE_MESH_MODEL_OP_MODEL_PUB_SET

esp_ble_mesh_cfg_model_sub_add_t **model_sub_add**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_ADD

esp_ble_mesh_cfg_model_sub_delete_t **model_sub_delete**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_DELETE

esp_ble_mesh_cfg_model_sub_overwrite_t **model_sub_overwrite**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_OVERWRITE

esp_ble_mesh_cfg_model_sub_va_add_t **model_sub_va_add**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_ADD

esp_ble_mesh_cfg_model_sub_va_delete_t **model_sub_va_delete**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_DELETE

esp_ble_mesh_cfg_model_sub_va_overwrite_t **model_sub_va_overwrite**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_VIRTUAL_ADDR_OVERWRITE

esp_ble_mesh_cfg_heartbeat_pub_set_t **heartbeat_pub_set**

For ESP_BLE_MESH_MODEL_OP_HEARTBEAT_PUB_SET

esp_ble_mesh_cfg_heartbeat_sub_set_t **heartbeat_sub_set**

For ESP_BLE_MESH_MODEL_OP_HEARTBEAT_SUB_SET

esp_ble_mesh_cfg_model_pub_va_set_t **model_pub_va_set**

For ESP_BLE_MESH_MODEL_OP_MODEL_PUB_VIRTUAL_ADDR_SET

esp_ble_mesh_cfg_model_sub_delete_all_t **model_sub_delete_all**

For ESP_BLE_MESH_MODEL_OP_MODEL_SUB_DELETE_ALL

esp_ble_mesh_cfg_net_key_update_t **net_key_update**

For ESP_BLE_MESH_MODEL_OP_NET_KEY_UPDATE

esp_ble_mesh_cfg_net_key_delete_t **net_key_delete**

For ESP_BLE_MESH_MODEL_OP_NET_KEY_DELETE

esp_ble_mesh_cfg_app_key_update_t **app_key_update**

For ESP_BLE_MESH_MODEL_OP_APP_KEY_UPDATE

esp_ble_mesh_cfg_app_key_delete_t **app_key_delete**

For ESP_BLE_MESH_MODEL_OP_APP_KEY_DELETE

esp_ble_mesh_cfg_node_identity_set_t **node_identity_set**

For ESP_BLE_MESH_MODEL_OP_NODE_IDENTITY_SET

esp_ble_mesh_cfg_model_app_unbind_t **model_app_unbind**

For ESP_BLE_MESH_MODEL_OP_MODEL_APP_UNBIND

esp_ble_mesh_cfg_kr_phase_set_t **kr_phase_set**

For ESP_BLE_MESH_MODEL_OP_KEY_REFRESH_PHASE_SET

esp_ble_mesh_cfg_net_transmit_set_t **net_transmit_set**

For ESP_BLE_MESH_MODEL_OP_NETWORK_TRANSMIT_SET

union **esp_ble_mesh_cfg_client_common_cb_param_t**

#include <esp_ble_mesh_config_model_api.h> Configuration Client Model received message union.

Public Members

esp_ble_mesh_cfg_beacon_status_cb_t **beacon_status**

The beacon status value

esp_ble_mesh_cfg_comp_data_status_cb_t **comp_data_status**

The composition data status value

esp_ble_mesh_cfg_default_ttl_status_cb_t **default_ttl_status**

The default_ttl status value

esp_ble_mesh_cfg_gatt_proxy_status_cb_t **gatt_proxy_status**

The gatt_proxy status value

esp_ble_mesh_cfg_relay_status_cb_t **relay_status**

The relay status value

esp_ble_mesh_cfg_model_pub_status_cb_t **model_pub_status**

The model publication status value

esp_ble_mesh_cfg_model_sub_status_cb_t **model_sub_status**

The model subscription status value

esp_ble_mesh_cfg_net_key_status_cb_t **netkey_status**

The netkey status value

esp_ble_mesh_cfg_app_key_status_cb_t **appkey_status**

The appkey status value

esp_ble_mesh_cfg_mod_app_status_cb_t **model_app_status**

The model app status value

esp_ble_mesh_cfg_friend_status_cb_t **friend_status**

The friend status value

esp_ble_mesh_cfg_hb_pub_status_cb_t **heartbeat_pub_status**

The heartbeat publication status value

esp_ble_mesh_cfg_hb_sub_status_cb_t **heartbeat_sub_status**

The heartbeat subscription status value

esp_ble_mesh_cfg_net_trans_status_cb_t **net_transmit_status**

The network transmit status value

esp_ble_mesh_cfg_model_sub_list_cb_t **model_sub_list**

The model subscription list value

esp_ble_mesh_cfg_net_key_list_cb_t **netkey_list**

The network key index list value

esp_ble_mesh_cfg_app_key_list_cb_t **appkey_list**

The application key index list value

esp_ble_mesh_cfg_node_id_status_cb_t **node_identity_status**

The node identity status value

esp_ble_mesh_cfg_model_app_list_cb_t **model_app_list**

The model application key index list value

esp_ble_mesh_cfg_kr_phase_status_cb_t **kr_phase_status**

The key refresh phase status value

esp_ble_mesh_cfg_lpn_pollto_status_cb_t **lpn_timeout_status**

The low power node poll timeout status value

union **esp_ble_mesh_cfg_server_state_change_t**

#include <esp_ble_mesh_config_model_api.h> Configuration Server model state change value union.

Public Members

esp_ble_mesh_state_change_cfg_mod_pub_set_t **mod_pub_set**

The recv_op in ctx can be used to decide which state is changed. Config Model Publication Set

esp_ble_mesh_state_change_cfg_mod_pub_va_set_t **mod_pub_va_set**

Config Model Publication Virtual Address Set

esp_ble_mesh_state_change_cfg_model_sub_add_t **mod_sub_add**

Config Model Subscription Add

esp_ble_mesh_state_change_cfg_model_sub_delete_t **mod_sub_delete**

Config Model Subscription Delete

esp_ble_mesh_state_change_cfg_netkey_add_t **netkey_add**

Config NetKey Add

esp_ble_mesh_state_change_cfg_netkey_update_t **netkey_update**

Config NetKey Update

esp_ble_mesh_state_change_cfg_netkey_delete_t **netkey_delete**

Config NetKey Delete

esp_ble_mesh_state_change_cfg_appkey_add_t **appkey_add**

Config AppKey Add

esp_ble_mesh_state_change_cfg_appkey_update_t **appkey_update**

Config AppKey Update

esp_ble_mesh_state_change_cfg_appkey_delete_t **appkey_delete**

Config AppKey Delete

esp_ble_mesh_state_change_cfg_model_app_bind_t **mod_app_bind**

Config Model App Bind

esp_ble_mesh_state_change_cfg_model_app_unbind_t **mod_app_unbind**

Config Model App Unbind

esp_ble_mesh_state_change_cfg_kr_phase_set_t **kr_phase_set**

Config Key Refresh Phase Set

union **esp_ble_mesh_cfg_server_cb_value_t**

#include <esp_ble_mesh_config_model_api.h> Configuration Server model callback value union.

Public Members

esp_ble_mesh_cfg_server_state_change_t **state_change**

ESP_BLE_MESH_CFG_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_cfg_srv_t**

Configuration Server Model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Configuration Server Model

uint8_t **net_transmit**

Network Transmit state

uint8_t **relay**

Relay Mode state

uint8_t **relay_retransmit**

Relay Retransmit state

uint8_t **beacon**

Secure Network Beacon state

uint8_t **gatt_proxy**

GATT Proxy state

uint8_t **friend_state**

Friend state

uint8_t **default_ttl**

Default TTL

struct k_delayed_work **timer**

Heartbeat Publication timer

uint16_t **dst**

Destination address for Heartbeat messages

uint16_t **count**

Number of Heartbeat messages to be sent

Number of Heartbeat messages received

uint8_t **period**

Period for sending Heartbeat messages

uint8_t **ttl**

TTL to be used when sending Heartbeat messages

uint16_t **feature**

Bit field indicating features that trigger Heartbeat messages when changed

uint16_t **net_idx**

NetKey Index used by Heartbeat Publication

struct *esp_ble_mesh_cfg_srv_t*::[anonymous] **heartbeat_pub**

Heartbeat Publication

int64_t **expiry**

Timestamp when Heartbeat subscription period is expired

uint16_t **src**

Source address for Heartbeat messages

uint8_t **min_hops**

Minimum hops when receiving Heartbeat messages

uint8_t **max_hops**

Maximum hops when receiving Heartbeat messages

esp_ble_mesh_cb_t **heartbeat_recv_cb**

Optional heartbeat subscription tracking function

struct *esp_ble_mesh_cfg_srv_t*::[anonymous] **heartbeat_sub**

Heartbeat Subscription

struct **esp_ble_mesh_cfg_composition_data_get_t**

Parameters of Config Composition Data Get.

Public Members

uint8_t **page**

Page number of the Composition Data.

struct **esp_ble_mesh_cfg_model_pub_get_t**

Parameters of Config Model Publication Get.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_sig_model_sub_get_t**

Parameters of Config SIG Model Subscription Get.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

struct **esp_ble_mesh_cfg_vnd_model_sub_get_t**

Parameters of Config Vendor Model Subscription Get.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_app_key_get_t**

Parameters of Config AppKey Get.

Public Members

uint16_t **net_idx**

The network key index

struct **esp_ble_mesh_cfg_node_identity_get_t**

Parameters of Config Node Identity Get.

Public Members

uint16_t **net_idx**

The network key index

struct **esp_ble_mesh_cfg_sig_model_app_get_t**

Parameters of Config SIG Model App Get.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

struct **esp_ble_mesh_cfg_vnd_model_app_get_t**

Parameters of Config Vendor Model App Get.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_kr_phase_get_t**

Parameters of Config Key Refresh Phase Get.

Public Members

uint16_t **net_idx**

The network key index

struct **esp_ble_mesh_cfg_lpn_polltimeout_get_t**

Parameters of Config Low Power Node PollTimeout Get.

Public Members

uint16_t **lpn_addr**

The unicast address of the Low Power node

struct **esp_ble_mesh_cfg_beacon_set_t**

Parameters of Config Beacon Set.

Public Members

uint8_t **beacon**

New Secure Network Beacon state

struct **esp_ble_mesh_cfg_default_ttl_set_t**

Parameters of Config Default TTL Set.

Public Members

uint8_t **ttl**

The default TTL state value

struct **esp_ble_mesh_cfg_friend_set_t**

Parameters of Config Friend Set.

Public Members

uint8_t **friend_state**

The friend state value

struct **esp_ble_mesh_cfg_gatt_proxy_set_t**

Parameters of Config GATT Proxy Set.

Public Members

uint8_t **gatt_proxy**

The GATT Proxy state value

struct **esp_ble_mesh_cfg_relay_set_t**

Parameters of Config Relay Set.

Public Members

uint8_t **relay**

The relay value

uint8_t **relay_retransmit**

The relay retransmit value

struct **esp_ble_mesh_cfg_net_key_add_t**

Parameters of Config NetKey Add.

Public Members

uint16_t **net_idx**

The network key index

uint8_t **net_key**[16]

The network key value

struct **esp_ble_mesh_cfg_app_key_add_t**

Parameters of Config AppKey Add.

Public Members

uint16_t **net_idx**

The network key index

uint16_t **app_idx**

The app key index

uint8_t **app_key**[16]

The app key value

struct **esp_ble_mesh_cfg_model_app_bind_t**

Parameters of Config Model App Bind.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_app_idx**

Index of the app key to bind with the model

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_pub_set_t**

Parameters of Config Model Publication Set.

Public Members

uint16_t **element_addr**

The element address

uint16_t **publish_addr**

Value of the publish address

uint16_t **publish_app_idx**

Index of the application key

bool **cred_flag**

Value of the Friendship Credential Flag

uint8_t **publish_ttl**

Default TTL value for the publishing messages

uint8_t **publish_period**

Period for periodic status publishing

uint8_t **publish_retransmit**

Number of retransmissions and number of 50-millisecond steps between retransmissions

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_add_t**

Parameters of Config Model Subscription Add.

Public Members

uint16_t **element_addr**

The element address

uint16_t **sub_addr**

The address to be added to the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_delete_t**

Parameters of Config Model Subscription Delete.

Public Members

uint16_t **element_addr**

The element address

uint16_t **sub_addr**

The address to be removed from the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_overwrite_t**

Parameters of Config Model Subscription Overwrite.

Public Members

uint16_t **element_addr**

The element address

uint16_t **sub_addr**

The address to be added to the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_va_add_t**

Parameters of Config Model Subscription Virtual Address Add.

Public Members

uint16_t **element_addr**

The element address

uint8_t **label_uuid**[16]

The Label UUID of the virtual address to be added to the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_va_delete_t**

Parameters of Config Model Subscription Virtual Address Delete.

Public Members

uint16_t **element_addr**

The element address

uint8_t **label_uuid**[16]

The Label UUID of the virtual address to be removed from the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_va_overwrite_t**

Parameters of Config Model Subscription Virtual Address Overwrite.

Public Members

uint16_t **element_addr**

The element address

uint8_t **label_uuid**[16]

The Label UUID of the virtual address to be added to the Subscription List

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_pub_va_set_t**

Parameters of Config Model Publication Virtual Address Set.

Public Members

uint16_t **element_addr**

The element address

uint8_t **label_uuid**[16]

Value of the Label UUID publish address

uint16_t **publish_app_idx**

Index of the application key

bool **cred_flag**

Value of the Friendship Credential Flag

uint8_t **publish_ttl**

Default TTL value for the publishing messages

uint8_t **publish_period**

Period for periodic status publishing

uint8_t **publish_retransmit**

Number of retransmissions and number of 50-millisecond steps between retransmissions

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_model_sub_delete_all_t**

Parameters of Config Model Subscription Delete All.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_net_key_update_t**

Parameters of Config NetKey Update.

Public Members

uint16_t **net_idx**

The network key index

uint8_t **net_key**[16]

The network key value

struct **esp_ble_mesh_cfg_net_key_delete_t**

Parameters of Config NetKey Delete.

Public Members

uint16_t **net_idx**

The network key index

struct **esp_ble_mesh_cfg_app_key_update_t**

Parameters of Config AppKey Update.

Public Members

uint16_t **net_idx**

The network key index

uint16_t **app_idx**

The app key index

uint8_t **app_key**[16]

The app key value

struct **esp_ble_mesh_cfg_app_key_delete_t**

Parameters of Config AppKey Delete.

Public Members

uint16_t **net_idx**

The network key index

uint16_t **app_idx**

The app key index

struct **esp_ble_mesh_cfg_node_identity_set_t**

Parameters of Config Node Identity Set.

Public Members

uint16_t **net_idx**

The network key index

uint8_t **identity**

New Node Identity state

struct **esp_ble_mesh_cfg_model_app_unbind_t**

Parameters of Config Model App Unbind.

Public Members

uint16_t **element_addr**

The element address

uint16_t **model_app_idx**

Index of the app key to bind with the model

uint16_t **model_id**

The model id

uint16_t **company_id**

The company id, if not a vendor model, shall set to 0xFFFF

struct **esp_ble_mesh_cfg_kr_phase_set_t**

Parameters of Config Key Refresh Phase Set.

Public Members

uint16_t **net_idx**

The network key index

uint8_t **transition**

New Key Refresh Phase Transition

struct **esp_ble_mesh_cfg_net_transmit_set_t**

Parameters of Config Network Transmit Set.

Public Members

uint8_t **net_transmit**

Network Transmit State

struct **esp_ble_mesh_cfg_heartbeat_pub_set_t**

Parameters of Config Model Heartbeat Publication Set.

Public Members

uint16_t **dst**

Destination address for Heartbeat messages

uint8_t **count**

Number of Heartbeat messages to be sent

uint8_t **period**

Period for sending Heartbeat messages

uint8_t **ttl**

TTL to be used when sending Heartbeat messages

uint16_t **feature**

Bit field indicating features that trigger Heartbeat messages when changed

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_cfg_heartbeat_sub_set_t**

Parameters of Config Model Heartbeat Subscription Set.

Public Members

uint16_t **src**

Source address for Heartbeat messages

uint16_t **dst**

Destination address for Heartbeat messages

uint8_t **period**

Period for receiving Heartbeat messages

struct **esp_ble_mesh_cfg_beacon_status_cb_t**

Parameter of Config Beacon Status

Public Members

uint8_t **beacon**

Secure Network Beacon state value

struct **esp_ble_mesh_cfg_comp_data_status_cb_t**

Parameters of Config Composition Data Status

Public Members

uint8_t **page**

Page number of the Composition Data

struct net_buf_simple ***composition_data**

Pointer to Composition Data for the identified page

struct **esp_ble_mesh_cfg_default_ttl_status_cb_t**

Parameter of Config Default TTL Status

Public Members

uint8_t **default_ttl**

Default TTL state value

struct **esp_ble_mesh_cfg_gatt_proxy_status_cb_t**

Parameter of Config GATT Proxy Status

Public Members

uint8_t **gatt_proxy**

GATT Proxy state value

struct **esp_ble_mesh_cfg_relay_status_cb_t**

Parameters of Config Relay Status

Public Members

uint8_t **relay**

Relay state value

uint8_t **retransmit**

Relay retransmit value(number of retransmissions and number of 10-millisecond steps between retransmissions)

struct **esp_ble_mesh_cfg_model_pub_status_cb_t**

Parameters of Config Model Publication Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **element_addr**

Address of the element

uint16_t **publish_addr**

Value of the publish address

uint16_t **app_idx**

Index of the application key

bool **cred_flag**

Value of the Friendship Credential Flag

uint8_t **ttl**

Default TTL value for the outgoing messages

uint8_t **period**

Period for periodic status publishing

uint8_t **transmit**

Number of retransmissions and number of 50-millisecond steps between retransmissions

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_cfg_model_sub_status_cb_t**

Parameters of Config Model Subscription Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **element_addr**

Address of the element

uint16_t **sub_addr**

Value of the address

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_cfg_net_key_status_cb_t**

Parameters of Config NetKey Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **net_idx**

Index of the NetKey

struct **esp_ble_mesh_cfg_app_key_status_cb_t**

Parameters of Config AppKey Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **net_idx**

Index of the NetKey

uint16_t **app_idx**

Index of the application key

struct **esp_ble_mesh_cfg_mod_app_status_cb_t**

Parameters of Config Model App Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **element_addr**

Address of the element

uint16_t **app_idx**

Index of the application key

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_cfg_friend_status_cb_t**

Parameter of Config Friend Status

Public Members

uint8_t **friend_state**

Friend state value

struct **esp_ble_mesh_cfg_hb_pub_status_cb_t**

Parameters of Config Heartbeat Publication Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **dst**

Destination address for Heartbeat messages

uint8_t **count**

Number of Heartbeat messages remaining to be sent

uint8_t **period**

Period for sending Heartbeat messages

uint8_t **ttl**

TTL to be used when sending Heartbeat messages

uint16_t **features**

Features that trigger Heartbeat messages when changed

uint16_t **net_idx**

Index of the NetKey

struct **esp_ble_mesh_cfg_hb_sub_status_cb_t**

Parameters of Config Heartbeat Subscription Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **src**

Source address for Heartbeat messages

uint16_t **dst**

Destination address for Heartbeat messages

uint8_t **period**

Remaining Period for processing Heartbeat messages

uint8_t **count**

Number of Heartbeat messages received

uint8_t **min_hops**

Minimum hops when receiving Heartbeat messages

uint8_t **max_hops**

Maximum hops when receiving Heartbeat messages

struct **esp_ble_mesh_cfg_net_trans_status_cb_t**

Parameters of Config Network Transmit Status

Public Members

uint8_t **net_trans_count**

Number of transmissions for each Network PDU originating from the node

uint8_t **net_trans_step**

Maximum hops when receiving Heartbeat messages

struct **esp_ble_mesh_cfg_model_sub_list_cb_t**

Parameters of Config SIG/Vendor Subscription List

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **element_addr**

Address of the element

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct net_buf_simple ***sub_addr**

A block of all addresses from the Subscription List

struct **esp_ble_mesh_cfg_net_key_list_cb_t**

Parameter of Config NetKey List

Public Members

struct net_buf_simple ***net_idx**

A list of NetKey Indexes known to the node

struct **esp_ble_mesh_cfg_app_key_list_cb_t**

Parameters of Config AppKey List

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **net_idx**

NetKey Index of the NetKey that the AppKeys are bound to

struct net_buf_simple ***app_idx**

A list of AppKey indexes that are bound to the NetKey identified by NetKeyIndex

struct **esp_ble_mesh_cfg_node_id_status_cb_t**

Parameters of Config Node Identity Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **net_idx**

Index of the NetKey

uint8_t **identity**

Node Identity state

struct **esp_ble_mesh_cfg_model_app_list_cb_t**

Parameters of Config SIG/Vendor Model App List

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **element_addr**

Address of the element

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct net_buf_simple ***app_idx**

All AppKey indexes bound to the Model

struct **esp_ble_mesh_cfg_kr_phase_status_cb_t**

Parameters of Config Key Refresh Phase Status

Public Members

uint8_t **status**

Status Code for the request message

uint16_t **net_idx**

Index of the NetKey

uint8_t **phase**

Key Refresh Phase state

struct **esp_ble_mesh_cfg_lpn_pollto_status_cb_t**

Parameters of Config Low Power Node PollTimeout Status

Public Members

uint16_t **lpn_addr**

The unicast address of the Low Power node

int32_t **poll_timeout**

The current value of the PollTimeout timer of the Low Power node

struct **esp_ble_mesh_cfg_client_cb_param_t**

Configuration Client Model callback parameters

Public Members

int **error_code**

Appropriate error code

esp_ble_mesh_client_common_param_t ***params**

The client common parameters

esp_ble_mesh_cfg_client_common_cb_param_t **status_cb**

The config status message callback values

struct **esp_ble_mesh_state_change_cfg_mod_pub_set_t**

Configuration Server model related context.

Parameters of Config Model Publication Set

Public Members

uint16_t **element_addr**

Element Address

uint16_t **pub_addr**

Publish Address

uint16_t **app_idx**

AppKey Index

bool **cred_flag**

Friendship Credential Flag

uint8_t **pub_ttl**

Publish TTL

uint8_t **pub_period**

Publish Period

uint8_t **pub_retransmit**

Publish Retransmit

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_mod_pub_va_set_t**

Parameters of Config Model Publication Virtual Address Set

Public Members

uint16_t **element_addr**

Element Address

uint8_t **label_uuid**[16]

Label UUID

uint16_t **app_idx**

AppKey Index

bool **cred_flag**

Friendship Credential Flag

uint8_t **pub_ttl**

Publish TTL

uint8_t **pub_period**

Publish Period

uint8_t **pub_retransmit**

Publish Retransmit

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_model_sub_add_t**

Parameters of Config Model Subscription Add

Public Members

uint16_t **element_addr**

Element Address

uint16_t **sub_addr**

Subscription Address

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_model_sub_delete_t**

Parameters of Config Model Subscription Delete

Public Members

uint16_t **element_addr**

Element Address

uint16_t **sub_addr**

Subscription Address

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_netkey_add_t**

Parameters of Config NetKey Add

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **net_key**[16]

NetKey

struct **esp_ble_mesh_state_change_cfg_netkey_update_t**

Parameters of Config NetKey Update

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **net_key**[16]
NetKey

struct **esp_ble_mesh_state_change_cfg_netkey_delete_t**
Parameter of Config NetKey Delete

Public Members

uint16_t **net_idx**
NetKey Index

struct **esp_ble_mesh_state_change_cfg_appkey_add_t**
Parameters of Config AppKey Add

Public Members

uint16_t **net_idx**
NetKey Index

uint16_t **app_idx**
AppKey Index

uint8_t **app_key**[16]
AppKey

struct **esp_ble_mesh_state_change_cfg_appkey_update_t**
Parameters of Config AppKey Update

Public Members

uint16_t **net_idx**
NetKey Index

uint16_t **app_idx**
AppKey Index

uint8_t **app_key**[16]
AppKey

struct **esp_ble_mesh_state_change_cfg_appkey_delete_t**
Parameters of Config AppKey Delete

Public Members

uint16_t **net_idx**
NetKey Index

uint16_t **app_idx**

AppKey Index

struct **esp_ble_mesh_state_change_cfg_model_app_bind_t**

Parameters of Config Model App Bind

Public Members

uint16_t **element_addr**

Element Address

uint16_t **app_idx**

AppKey Index

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_model_app_unbind_t**

Parameters of Config Model App Unbind

Public Members

uint16_t **element_addr**

Element Address

uint16_t **app_idx**

AppKey Index

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_state_change_cfg_kr_phase_set_t**

Parameters of Config Key Refresh Phase Set

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **kr_phase**

New Key Refresh Phase Transition

struct **esp_ble_mesh_cfg_server_cb_param_t**

Configuration Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_cfg_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_CFG_SRV(srv_data)

Define a new Config Server Model.

Note: The Config Server Model can only be included by a Primary Element.

Parameters

- **srv_data** -- Pointer to a unique Config Server Model user_data.

Returns New Config Server Model instance.

ESP_BLE_MESH_MODEL_CFG_CLI(cli_data)

Define a new Config Client Model.

Note: The Config Client Model can only be included by a Primary Element.

Parameters

- **cli_data** -- Pointer to a unique struct *esp_ble_mesh_client_t*.

Returns New Config Client Model instance.

Type Definitions

typedef void (***esp_ble_mesh_cfg_client_cb_t**)(*esp_ble_mesh_cfg_client_cb_event_t* event, *esp_ble_mesh_cfg_client_cb_param_t* *param)

Bluetooth Mesh Config Client and Server Model functions.

Configuration Client Model callback function type

Param event Event type

Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_cfg_server_cb_t**)(*esp_ble_mesh_cfg_server_cb_event_t* event, *esp_ble_mesh_cfg_server_cb_param_t* *param)

Configuration Server Model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_cfg_client_cb_event_t**

This enum value is the event of Configuration Client Model

Values:

enumerator **ESP_BLE_MESH_CFG_CLIENT_GET_STATE_EVT**

enumerator **ESP_BLE_MESH_CFG_CLIENT_SET_STATE_EVT**

enumerator **ESP_BLE_MESH_CFG_CLIENT_PUBLISH_EVT**

enumerator **ESP_BLE_MESH_CFG_CLIENT_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_CFG_CLIENT_EVT_MAX**

enum **esp_ble_mesh_cfg_server_cb_event_t**

This enum value is the event of Configuration Server model

Values:

enumerator **ESP_BLE_MESH_CFG_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_CFG_SERVER_EVT_MAX**

Health Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_health_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_health_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_health_client_callback**(*esp_ble_mesh_health_client_cb_t* callback)

Register BLE Mesh Health Model callback, the callback will report Health Client & Server Model events.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_health_server_callback**(*esp_ble_mesh_health_server_cb_t* callback)

Register BLE Mesh Health Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_health_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_health_client_get_state_t* *get_state)

This function is called to get the Health Server states using the Health Client Model get messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_opcode_health_client_get_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_health_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_health_client_set_state_t* *set_state)

This function is called to set the Health Server states using the Health Client Model set messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_opcode_health_client_set_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_health_server_fault_update**(*esp_ble_mesh_elem_t* *element)

This function is called by the Health Server Model to update the context of its Health Current status.

Parameters **element** -- **[in]** The element to which the Health Server Model belongs.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_health_client_get_state_t**

#include <esp_ble_mesh_health_model_api.h> For ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_GET ESP_BLE_MESH_MODEL_OP_ATTENTION_GET ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_GET the get_state parameter in the esp_ble_mesh_health_client_get_state function should not be set to NULL.

Public Members

esp_ble_mesh_health_fault_get_t **fault_get**

For ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_GET.

union **esp_ble_mesh_health_client_set_state_t**

#include <esp_ble_mesh_health_model_api.h> For ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR_UNACK ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_TEST_UNACK ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_SET_UNACK ESP_BLE_MESH_MODEL_OP_ATTENTION_SET ESP_BLE_MESH_MODEL_OP_ATTENTION_SET_UNACK the set_state parameter in the esp_ble_mesh_health_client_set_state function should not be set to NULL.

Public Members

esp_ble_mesh_health_attention_set_t **attention_set**

For ESP_BLE_MESH_MODEL_OP_ATTENTION_SET or ESP_BLE_MESH_MODEL_OP_ATTENTION_SET_UNA

esp_ble_mesh_health_period_set_t **period_set**

For ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_SET or ESP_BLE_MESH_MODEL_OP_HEALTH_PERIOD_SET_UNACK.

esp_ble_mesh_health_fault_test_t **fault_test**

For ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_TEST or ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_TEST_UNACK.

esp_ble_mesh_health_fault_clear_t **fault_clear**

For ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR or ESP_BLE_MESH_MODEL_OP_HEALTH_FAULT_CLEAR_UNACK.

union **esp_ble_mesh_health_client_common_cb_param_t**

#include <esp_ble_mesh_health_model_api.h> Health Client Model received message union.

Public Members

esp_ble_mesh_health_current_status_cb_t **current_status**

The health current status value

esp_ble_mesh_health_fault_status_cb_t **fault_status**

The health fault status value

esp_ble_mesh_health_period_status_cb_t **period_status**

The health period status value

esp_ble_mesh_health_attention_status_cb_t **attention_status**

The health attention status value

union **esp_ble_mesh_health_server_cb_param_t**

#include <esp_ble_mesh_health_model_api.h> Health Server Model callback parameters union.

Public Members

esp_ble_mesh_health_fault_update_comp_cb_t **fault_update_comp**

ESP_BLE_MESH_HEALTH_SERVER_FAULT_UPDATE_COMP_EVT

esp_ble_mesh_health_fault_clear_cb_t **fault_clear**

ESP_BLE_MESH_HEALTH_SERVER_FAULT_CLEAR_EVT

esp_ble_mesh_health_fault_test_cb_t **fault_test**

ESP_BLE_MESH_HEALTH_SERVER_FAULT_TEST_EVT

esp_ble_mesh_health_attention_on_cb_t **attention_on**

ESP_BLE_MESH_HEALTH_SERVER_ATTENTION_ON_EVT

esp_ble_mesh_health_attention_off_cb_t **attention_off**

ESP_BLE_MESH_HEALTH_SERVER_ATTENTION_OFF_EVT

Structures

struct **esp_ble_mesh_health_srv_cb_t**

ESP BLE Mesh Health Server callback

Public Members

esp_ble_mesh_cb_t **fault_clear**

Clear health registered faults. Initialized by the stack.

esp_ble_mesh_cb_t **fault_test**

Run a specific health test. Initialized by the stack.

esp_ble_mesh_cb_t **attention_on**

Health attention on callback. Initialized by the stack.

esp_ble_mesh_cb_t **attention_off**

Health attention off callback. Initialized by the stack.

struct **esp_ble_mesh_health_test_t**

ESP BLE Mesh Health Server test Context

Public Members

uint8_t **id_count**

Number of Health self-test ID

const uint8_t ***test_ids**

Array of Health self-test IDs

uint16_t **company_id**

Company ID used to identify the Health Fault state

uint8_t **prev_test_id**

Current test ID of the health fault test

uint8_t **current_faults**[ESP_BLE_MESH_HEALTH_FAULT_ARRAY_SIZE]

Array of current faults

uint8_t **registered_faults**[ESP_BLE_MESH_HEALTH_FAULT_ARRAY_SIZE]

Array of registered faults

struct **esp_ble_mesh_health_srv_t**

ESP BLE Mesh Health Server Model Context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Health Server Model

esp_ble_mesh_health_srv_cb_t **health_cb**

Health callback struct

struct k_delayed_work **attention_timer**

Attention Timer state

bool **attention_timer_start**

Attention Timer start flag

esp_ble_mesh_health_test_t **health_test**

Health Server fault test

struct **esp_ble_mesh_health_fault_get_t**

Parameter of Health Fault Get

Public Members

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct **esp_ble_mesh_health_attention_set_t**

Parameter of Health Attention Set

Public Members

uint8_t **attention**

Value of the Attention Timer state

struct **esp_ble_mesh_health_period_set_t**

Parameter of Health Period Set

Public Members

uint8_t **fast_period_divisor**

Divider for the Publish Period

struct **esp_ble_mesh_health_fault_test_t**

Parameter of Health Fault Test

Public Members

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

uint8_t **test_id**

ID of a specific test to be performed

struct **esp_ble_mesh_health_fault_clear_t**

Parameter of Health Fault Clear

Public Members

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct **esp_ble_mesh_health_current_status_cb_t**

Parameters of Health Current Status

Public Members

uint8_t **test_id**

ID of a most recently performed test

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct net_buf_simple ***fault_array**

FaultArray field contains a sequence of 1-octet fault values

struct **esp_ble_mesh_health_fault_status_cb_t**

Parameters of Health Fault Status

Public Members

uint8_t **test_id**

ID of a most recently performed test

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct net_buf_simple ***fault_array**

FaultArray field contains a sequence of 1-octet fault values

struct **esp_ble_mesh_health_period_status_cb_t**

Parameter of Health Period Status

Public Members

uint8_t **fast_period_divisor**

Divider for the Publish Period

struct **esp_ble_mesh_health_attention_status_cb_t**

Parameter of Health Attention Status

Public Members

uint8_t **attention**

Value of the Attention Timer state

struct **esp_ble_mesh_health_client_cb_param_t**

Health Client Model callback parameters

Public Members

int **error_code**

Appropriate error code

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_health_client_common_cb_param_t **status_cb**

The health message status callback values

struct **esp_ble_mesh_health_fault_update_comp_cb_t**

Parameter of publishing Health Current Status completion event

Public Members

int **error_code**

The result of publishing Health Current Status

esp_ble_mesh_elem_t ***element**

Pointer to the element which contains the Health Server Model

struct **esp_ble_mesh_health_fault_clear_cb_t**

Parameters of Health Fault Clear event

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Health Server Model

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct **esp_ble_mesh_health_fault_test_cb_t**

Parameters of Health Fault Test event

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Health Server Model

uint8_t **test_id**

ID of a specific test to be performed

uint16_t **company_id**

Bluetooth assigned 16-bit Company ID

struct **esp_ble_mesh_health_attention_on_cb_t**

Parameter of Health Attention On event

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Health Server Model

uint8_t **time**

Duration of attention timer on (in seconds)

struct **esp_ble_mesh_health_attention_off_cb_t**

Parameter of Health Attention Off event

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Health Server Model

Macros

ESP_BLE_MESH_MODEL_HEALTH_SRV(srv, pub)

Define a new Health Server Model.

Note: The Health Server Model can only be included by a Primary Element.

Parameters
- **srv** -- Pointer to the unique struct *esp_ble_mesh_health_srv_t*.
- **pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.

Returns New Health Server Model instance.

ESP_BLE_MESH_MODEL_HEALTH_CLI(cli_data)

Define a new Health Client Model.

Note: This API needs to be called for each element on which the application needs to have a Health Client Model.

Parameters
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Health Client Model instance.

ESP_BLE_MESH_HEALTH_PUB_DEFINE(_name, _max, _role)

A helper to define a health publication context

Parameters
- **_name** -- Name given to the publication context variable.
- **_max** -- Maximum number of faults the element can have.
- **_role** -- Role of the device which contains the model.

ESP_BLE_MESH_HEALTH_STANDARD_TEST

SIG identifier of Health Fault Test. 0x01 ~ 0xFF: Vendor Specific Test.

ESP_BLE_MESH_NO_FAULT

Fault values of Health Fault Test. 0x33 ~ 0x7F: Reserved for Future Use. 0x80 ~ 0xFF: Vendor Specific Warning/Error.

ESP_BLE_MESH_BATTERY_LOW_WARNING

ESP_BLE_MESH_BATTERY_LOW_ERROR

ESP_BLE_MESH_SUPPLY_VOLTAGE_TOO_LOW_WARNING

ESP_BLE_MESH_SUPPLY_VOLTAGE_TOO_LOW_ERROR

ESP_BLE_MESH_SUPPLY_VOLTAGE_TOO_HIGH_WARNING

ESP_BLE_MESH_SUPPLY_VOLTAGE_TOO_HIGH_ERROR

ESP_BLE_MESH_POWER_SUPPLY_INTERRUPTED_WARNING

ESP_BLE_MESH_POWER_SUPPLY_INTERRUPTED_ERROR

ESP_BLE_MESH_NO_LOAD_WARNING

ESP_BLE_MESH_NO_LOAD_ERROR

ESP_BLE_MESH_OVERLOAD_WARNING

ESP_BLE_MESH_OVERLOAD_ERROR

ESP_BLE_MESH_OVERHEAT_WARNING

ESP_BLE_MESH_OVERHEAT_ERROR

ESP_BLE_MESH_CONDENSATION_WARNING

ESP_BLE_MESH_CONDENSATION_ERROR

ESP_BLE_MESH_VIBRATION_WARNING

ESP_BLE_MESH_VIBRATION_ERROR

ESP_BLE_MESH_CONFIGURATION_WARNING

ESP_BLE_MESH_CONFIGURATION_ERROR

ESP_BLE_MESH_ELEMENT_NOT_CALIBRATED_WARNING

ESP_BLE_MESH_ELEMENT_NOT_CALIBRATED_ERROR

ESP_BLE_MESH_MEMORY_WARNING

ESP_BLE_MESH_MEMORY_ERROR

ESP_BLE_MESH_SELF_TEST_WARNING

ESP_BLE_MESH_SELF_TEST_ERROR

ESP_BLE_MESH_INPUT_TOO_LOW_WARNING

ESP_BLE_MESH_INPUT_TOO_LOW_ERROR

ESP_BLE_MESH_INPUT_TOO_HIGH_WARNING

ESP_BLE_MESH_INPUT_TOO_HIGH_ERROR

ESP_BLE_MESH_INPUT_NO_CHANGE_WARNING

ESP_BLE_MESH_INPUT_NO_CHANGE_ERROR

ESP_BLE_MESH_ACTUATOR_BLOCKED_WARNING

ESP_BLE_MESH_ACTUATOR_BLOCKED_ERROR

ESP_BLE_MESH_HOUSING_OPENED_WARNING

ESP_BLE_MESH_HOUSING_OPENED_ERROR

ESP_BLE_MESH_TAMPER_WARNING

ESP_BLE_MESH_TAMPER_ERROR

ESP_BLE_MESH_DEVICE_MOVED_WARNING

ESP_BLE_MESH_DEVICE_MOVED_ERROR

ESP_BLE_MESH_DEVICE_DROPPED_WARNING

ESP_BLE_MESH_DEVICE_DROPPED_ERROR

ESP_BLE_MESH_OVERFLOW_WARNING

ESP_BLE_MESH_OVERFLOW_ERROR

ESP_BLE_MESH_EMPTY_WARNING

ESP_BLE_MESH_EMPTY_ERROR

ESP_BLE_MESH_INTERNAL_BUS_WARNING

ESP_BLE_MESH_INTERNAL_BUS_ERROR

ESP_BLE_MESH_MECHANISM_JAMMED_WARNING

ESP_BLE_MESH_MECHANISM_JAMMED_ERROR

ESP_BLE_MESH_HEALTH_FAULT_ARRAY_SIZE

Type Definitions

typedef void (***esp_ble_mesh_health_client_cb_t**)(*esp_ble_mesh_health_client_cb_event_t* event, *esp_ble_mesh_health_client_cb_param_t* *param)

Bluetooth Mesh Health Client and Server Model function.

Health Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_health_server_cb_t**)(*esp_ble_mesh_health_server_cb_event_t* event, *esp_ble_mesh_health_server_cb_param_t* *param)

Health Server Model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_health_client_cb_event_t**

This enum value is the event of Health Client Model

Values:

enumerator **ESP_BLE_MESH_HEALTH_CLIENT_GET_STATE_EVT**

enumerator **ESP_BLE_MESH_HEALTH_CLIENT_SET_STATE_EVT**

enumerator **ESP_BLE_MESH_HEALTH_CLIENT_PUBLISH_EVT**

enumerator **ESP_BLE_MESH_HEALTH_CLIENT_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_HEALTH_CLIENT_EVT_MAX**

enum **esp_ble_mesh_health_server_cb_event_t**

This enum value is the event of Health Server Model

Values:

enumerator **ESP_BLE_MESH_HEALTH_SERVER_FAULT_UPDATE_COMP_EVT**

enumerator **ESP_BLE_MESH_HEALTH_SERVER_FAULT_CLEAR_EVT**

enumerator **ESP_BLE_MESH_HEALTH_SERVER_FAULT_TEST_EVT**

enumerator **ESP_BLE_MESH_HEALTH_SERVER_ATTENTION_ON_EVT**

enumerator **ESP_BLE_MESH_HEALTH_SERVER_ATTENTION_OFF_EVT**

enumerator **ESP_BLE_MESH_HEALTH_SERVER_EVT_MAX**

Generic Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_generic_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_generic_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_generic_client_callback**(*esp_ble_mesh_generic_client_cb_t* callback)

Register BLE Mesh Generic Client Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_generic_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_generic_client_get_state_t* *get_state)

Get the value of Generic Server Model states using the Generic Client Model get messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_generic_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer to generic get message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_generic_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_generic_client_set_state_t* *set_state)

Set the value of Generic Server Model states using the Generic Client Model set messages.

Note: If you want to find the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_generic_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer to generic set message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_generic_server_callback**(*esp_ble_mesh_generic_server_cb_t* callback)

Register BLE Mesh Generic Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_generic_client_get_state_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Client Model get message union.

Public Members

esp_ble_mesh_gen_user_property_get_t **user_property_get**

For ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_GET

esp_ble_mesh_gen_admin_property_get_t **admin_property_get**

For ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_GET

esp_ble_mesh_gen_manufacturer_property_get_t **manufacturer_property_get**

For ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_SET

esp_ble_mesh_gen_client_properties_get_t **client_properties_get**

For ESP_BLE_MESH_MODEL_OP_GEN_CLIENT_PROPERTIES_GET

union **esp_ble_mesh_generic_client_set_state_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Client Model set message union.

Public Members

esp_ble_mesh_gen_onoff_set_t **onoff_set**

For ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET & ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_SET_UN

esp_ble_mesh_gen_level_set_t **level_set**

For ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET & ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_SET_UN

esp_ble_mesh_gen_delta_set_t **delta_set**

For ESP_BLE_MESH_MODEL_OP_GEN_DELTA_SET & ESP_BLE_MESH_MODEL_OP_GEN_DELTA_SET_UN

esp_ble_mesh_gen_move_set_t **move_set**

For ESP_BLE_MESH_MODEL_OP_GEN_MOVE_SET & ESP_BLE_MESH_MODEL_OP_GEN_MOVE_SET_UNA

esp_ble_mesh_gen_def_trans_time_set_t **def_trans_time_set**

For ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_SET & ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_SET_UNACK

esp_ble_mesh_gen_onpowerup_set_t **power_set**

For ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_SET & ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_SET_UNACK

esp_ble_mesh_gen_power_level_set_t **power_level_set**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_SET & ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_SET_UNACK

esp_ble_mesh_gen_power_default_set_t **power_default_set**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_SET & ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_SET_UNACK

esp_ble_mesh_gen_power_range_set_t **power_range_set**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_SET & ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_SET_UNACK

esp_ble_mesh_gen_loc_global_set_t **loc_global_set**

For ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_SET & ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_SET_UNACK

esp_ble_mesh_gen_loc_local_set_t **loc_local_set**

For ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_SET & ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_SET_UNACK

esp_ble_mesh_gen_user_property_set_t **user_property_set**

For ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_SET & ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_SET_UNACK

esp_ble_mesh_gen_admin_property_set_t **admin_property_set**

For ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_SET & ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_SET_UNACK

esp_ble_mesh_gen_manufacturer_property_set_t **manufacturer_property_set**

For ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_SET & ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_SET_UNACK

union **esp_ble_mesh_gen_client_status_cb_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Client Model received message union.

Public Members

esp_ble_mesh_gen_onoff_status_cb_t **onoff_status**

For ESP_BLE_MESH_MODEL_OP_GEN_ONOFF_STATUS

esp_ble_mesh_gen_level_status_cb_t **level_status**

For ESP_BLE_MESH_MODEL_OP_GEN_LEVEL_STATUS

esp_ble_mesh_gen_def_trans_time_status_cb_t **def_trans_time_status**

For ESP_BLE_MESH_MODEL_OP_GEN_DEF_TRANS_TIME_STATUS

esp_ble_mesh_gen_onpowerup_status_cb_t **onpowerup_status**

For ESP_BLE_MESH_MODEL_OP_GEN_ONPOWERUP_STATUS

esp_ble_mesh_gen_power_level_status_cb_t **power_level_status**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_LEVEL_STATUS

esp_ble_mesh_gen_power_last_status_cb_t **power_last_status**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_LAST_STATUS

esp_ble_mesh_gen_power_default_status_cb_t **power_default_status**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_DEFAULT_STATUS

esp_ble_mesh_gen_power_range_status_cb_t **power_range_status**

For ESP_BLE_MESH_MODEL_OP_GEN_POWER_RANGE_STATUS

esp_ble_mesh_gen_battery_status_cb_t **battery_status**

For ESP_BLE_MESH_MODEL_OP_GEN_BATTERY_STATUS

esp_ble_mesh_gen_loc_global_status_cb_t **location_global_status**

For ESP_BLE_MESH_MODEL_OP_GEN_LOC_GLOBAL_STATUS

esp_ble_mesh_gen_loc_local_status_cb_t **location_local_status**

ESP_BLE_MESH_MODEL_OP_GEN_LOC_LOCAL_STATUS

esp_ble_mesh_gen_user_properties_status_cb_t **user_properties_status**

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTIES_STATUS

esp_ble_mesh_gen_user_property_status_cb_t **user_property_status**

ESP_BLE_MESH_MODEL_OP_GEN_USER_PROPERTY_STATUS

esp_ble_mesh_gen_admin_properties_status_cb_t **admin_properties_status**

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTIES_STATUS

esp_ble_mesh_gen_admin_property_status_cb_t **admin_property_status**

ESP_BLE_MESH_MODEL_OP_GEN_ADMIN_PROPERTY_STATUS

esp_ble_mesh_gen_manufacturer_properties_status_cb_t **manufacturer_properties_status**

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTIES_STATUS

esp_ble_mesh_gen_manufacturer_property_status_cb_t **manufacturer_property_status**

ESP_BLE_MESH_MODEL_OP_GEN_MANUFACTURER_PROPERTY_STATUS

esp_ble_mesh_gen_client_properties_status_cb_t **client_properties_status**

ESP_BLE_MESH_MODEL_OP_GEN_CLIENT_PROPERTIES_STATUS

union **esp_ble_mesh_generic_server_state_change_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Server Model state change value union.

Public Members

esp_ble_mesh_state_change_gen_onoff_set_t **onoff_set**

The recv_op in ctx can be used to decide which state is changed. Generic OnOff Set

esp_ble_mesh_state_change_gen_level_set_t **level_set**

Generic Level Set

esp_ble_mesh_state_change_gen_delta_set_t **delta_set**

Generic Delta Set

esp_ble_mesh_state_change_gen_move_set_t **move_set**

Generic Move Set

esp_ble_mesh_state_change_gen_def_trans_time_set_t **def_trans_time_set**

Generic Default Transition Time Set

esp_ble_mesh_state_change_gen_onpowerup_set_t **onpowerup_set**

Generic OnPowerUp Set

esp_ble_mesh_state_change_gen_power_level_set_t **power_level_set**

Generic Power Level Set

esp_ble_mesh_state_change_gen_power_default_set_t **power_default_set**

Generic Power Default Set

esp_ble_mesh_state_change_gen_power_range_set_t **power_range_set**

Generic Power Range Set

esp_ble_mesh_state_change_gen_loc_global_set_t **loc_global_set**

Generic Location Global Set

esp_ble_mesh_state_change_gen_loc_local_set_t **loc_local_set**

Generic Location Local Set

esp_ble_mesh_state_change_gen_user_property_set_t **user_property_set**

Generic User Property Set

esp_ble_mesh_state_change_gen_admin_property_set_t **admin_property_set**

Generic Admin Property Set

esp_ble_mesh_state_change_gen_manu_property_set_t **manu_property_set**

Generic Manufacturer Property Set

union **esp_ble_mesh_generic_server_recv_get_msg_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Server Model received get message union.

Public Members

esp_ble_mesh_server_recv_gen_user_property_get_t **user_property**

Generic User Property Get

esp_ble_mesh_server_recv_gen_admin_property_get_t **admin_property**

Generic Admin Property Get

esp_ble_mesh_server_recv_gen_manufacturer_property_get_t **manu_property**

Generic Manufacturer Property Get

esp_ble_mesh_server_recv_gen_client_properties_get_t **client_properties**

Generic Client Properties Get

union **esp_ble_mesh_generic_server_recv_set_msg_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Server Model received set message union.

Public Members

esp_ble_mesh_server_recv_gen_onoff_set_t **onoff**

Generic OnOff Set/Generic OnOff Set Unack

esp_ble_mesh_server_recv_gen_level_set_t **level**

Generic Level Set/Generic Level Set Unack

esp_ble_mesh_server_recv_gen_delta_set_t **delta**

Generic Delta Set/Generic Delta Set Unack

esp_ble_mesh_server_recv_gen_move_set_t **move**

Generic Move Set/Generic Move Set Unack

esp_ble_mesh_server_recv_gen_def_trans_time_set_t **def_trans_time**

Generic Default Transition Time Set/Generic Default Transition Time Set Unack

esp_ble_mesh_server_recv_gen_onpowerup_set_t **onpowerup**

Generic OnPowerUp Set/Generic OnPowerUp Set Unack

esp_ble_mesh_server_recv_gen_power_level_set_t **power_level**

Generic Power Level Set/Generic Power Level Set Unack

esp_ble_mesh_server_recv_gen_power_default_set_t **power_default**

Generic Power Default Set/Generic Power Default Set Unack

esp_ble_mesh_server_recv_gen_power_range_set_t **power_range**

Generic Power Range Set/Generic Power Range Set Unack

esp_ble_mesh_server_recv_gen_loc_global_set_t **location_global**

Generic Location Global Set/Generic Location Global Set Unack

esp_ble_mesh_server_recv_gen_loc_local_set_t **location_local**

Generic Location Local Set/Generic Location Local Set Unack

esp_ble_mesh_server_recv_gen_user_property_set_t **user_property**

Generic User Property Set/Generic User Property Set Unack

esp_ble_mesh_server_recv_gen_admin_property_set_t **admin_property**

Generic Admin Property Set/Generic Admin Property Set Unack

esp_ble_mesh_server_recv_gen_manufacturer_property_set_t **manu_property**

Generic Manufacturer Property Set/Generic Manufacturer Property Set Unack

union **esp_ble_mesh_generic_server_cb_value_t**

#include <esp_ble_mesh_generic_model_api.h> Generic Server Model callback value union.

Public Members

esp_ble_mesh_generic_server_state_change_t **state_change**

ESP_BLE_MESH_GENERIC_SERVER_STATE_CHANGE_EVT

esp_ble_mesh_generic_server_recv_get_msg_t **get**

ESP_BLE_MESH_GENERIC_SERVER_RECV_GET_MSG_EVT

esp_ble_mesh_generic_server_recv_set_msg_t **set**

ESP_BLE_MESH_GENERIC_SERVER_RECV_SET_MSG_EVT

Structures

struct **esp_ble_mesh_gen_onoff_set_t**

Bluetooth Mesh Generic Client Model Get and Set parameters structure.

Parameters of Generic OnOff Set.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint8_t **onoff**

Target value of Generic OnOff state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_gen_level_set_t**

Parameters of Generic Level Set.

Public Members

bool **op_en**

Indicate if optional parameters are included

int16_t **level**

Target value of Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_gen_delta_set_t**

Parameters of Generic Delta Set.

Public Members

bool **op_en**

Indicate if optional parameters are included

int32_t **level**

Delta change of Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_gen_move_set_t**

Parameters of Generic Move Set.

Public Members

bool **op_en**

Indicate if optional parameters are included

int16_t **delta_level**

Delta Level step to calculate Move speed for Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_gen_def_trans_time_set_t**

Parameter of Generic Default Transition Time Set.

Public Members

uint8_t **trans_time**

The value of the Generic Default Transition Time state

struct **esp_ble_mesh_gen_onpowerup_set_t**

Parameter of Generic OnPowerUp Set.

Public Members

uint8_t **onpowerup**

The value of the Generic OnPowerUp state

struct **esp_ble_mesh_gen_power_level_set_t**

Parameters of Generic Power Level Set.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **power**

Target value of Generic Power Actual state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_gen_power_default_set_t**

Parameter of Generic Power Default Set.

Public Members

uint16_t **power**

The value of the Generic Power Default state

struct **esp_ble_mesh_gen_power_range_set_t**

Parameters of Generic Power Range Set.

Public Members

uint16_t **range_min**

Value of Range Min field of Generic Power Range state

uint16_t **range_max**

Value of Range Max field of Generic Power Range state

struct **esp_ble_mesh_gen_loc_global_set_t**

Parameters of Generic Location Global Set.

Public Members

int32_t **global_latitude**

Global Coordinates (Latitude)

int32_t **global_longitude**

Global Coordinates (Longitude)

int16_t **global_altitude**

Global Altitude

struct **esp_ble_mesh_gen_loc_local_set_t**

Parameters of Generic Location Local Set.

Public Members

int16_t **local_north**

Local Coordinates (North)

int16_t **local_east**

Local Coordinates (East)

int16_t **local_altitude**

Local Altitude

uint8_t **floor_number**

Floor Number

uint16_t **uncertainty**

Uncertainty

struct **esp_ble_mesh_gen_user_property_get_t**

Parameter of Generic User Property Get.

Public Members

uint16_t **property_id**

Property ID identifying a Generic User Property

struct **esp_ble_mesh_gen_user_property_set_t**

Parameters of Generic User Property Set.

Public Members

uint16_t **property_id**

Property ID identifying a Generic User Property

struct net_buf_simple ***property_value**

Raw value for the User Property

struct **esp_ble_mesh_gen_admin_property_get_t**

Parameter of Generic Admin Property Get.

Public Members

uint16_t **property_id**

Property ID identifying a Generic Admin Property

struct **esp_ble_mesh_gen_admin_property_set_t**

Parameters of Generic Admin Property Set.

Public Members

uint16_t **property_id**

Property ID identifying a Generic Admin Property

uint8_t **user_access**

Enumeration indicating user access

struct net_buf_simple ***property_value**

Raw value for the Admin Property

struct **esp_ble_mesh_gen_manufacturer_property_get_t**

Parameter of Generic Manufacturer Property Get.

Public Members

uint16_t **property_id**

Property ID identifying a Generic Manufacturer Property

struct **esp_ble_mesh_gen_manufacturer_property_set_t**

Parameters of Generic Manufacturer Property Set.

Public Members

uint16_t **property_id**

Property ID identifying a Generic Manufacturer Property

uint8_t **user_access**

Enumeration indicating user access

struct **esp_ble_mesh_gen_client_properties_get_t**

Parameter of Generic Client Properties Get.

Public Members

uint16_t **property_id**

A starting Client Property ID present within an element

struct **esp_ble_mesh_gen_onoff_status_cb_t**

Bluetooth Mesh Generic Client Model Get and Set callback parameters structure.

Parameters of Generic OnOff Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint8_t **present_onoff**

Current value of Generic OnOff state

uint8_t **target_onoff**

Target value of Generic OnOff state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_gen_level_status_cb_t**

Parameters of Generic Level Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

int16_t **present_level**

Current value of Generic Level state

int16_t **target_level**

Target value of the Generic Level state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_gen_def_trans_time_status_cb_t**

Parameter of Generic Default Transition Time Status.

Public Members

uint8_t **trans_time**

The value of the Generic Default Transition Time state

struct **esp_ble_mesh_gen_onpowerup_status_cb_t**

Parameter of Generic OnPowerUp Status.

Public Members

uint8_t **onpowerup**

The value of the Generic OnPowerUp state

struct **esp_ble_mesh_gen_power_level_status_cb_t**

Parameters of Generic Power Level Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_power**

Current value of Generic Power Actual state

uint16_t **target_power**

Target value of Generic Power Actual state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_gen_power_last_status_cb_t**

Parameter of Generic Power Last Status.

Public Members

uint16_t **power**

The value of the Generic Power Last state

struct **esp_ble_mesh_gen_power_default_status_cb_t**

Parameter of Generic Power Default Status.

Public Members

uint16_t **power**

The value of the Generic Default Last state

struct **esp_ble_mesh_gen_power_range_status_cb_t**

Parameters of Generic Power Range Status.

Public Members

uint8_t **status_code**

Status Code for the request message

uint16_t **range_min**

Value of Range Min field of Generic Power Range state

uint16_t **range_max**

Value of Range Max field of Generic Power Range state

struct **esp_ble_mesh_gen_battery_status_cb_t**

Parameters of Generic Battery Status.

Public Members

uint32_t **battery_level**

Value of Generic Battery Level state

uint32_t **time_to_discharge**

Value of Generic Battery Time to Discharge state

uint32_t **time_to_charge**

Value of Generic Battery Time to Charge state

uint32_t **flags**

Value of Generic Battery Flags state

struct **esp_ble_mesh_gen_loc_global_status_cb_t**

Parameters of Generic Location Global Status.

Public Members

int32_t **global_latitude**

Global Coordinates (Latitude)

int32_t **global_longitude**

Global Coordinates (Longitude)

int16_t **global_altitude**

Global Altitude

struct **esp_ble_mesh_gen_loc_local_status_cb_t**

Parameters of Generic Location Local Status.

Public Members

int16_t **local_north**

Local Coordinates (North)

int16_t **local_east**

Local Coordinates (East)

int16_t **local_altitude**

Local Altitude

uint8_t **floor_number**

Floor Number

uint16_t **uncertainty**

Uncertainty

struct **esp_ble_mesh_gen_user_properties_status_cb_t**

Parameter of Generic User Properties Status.

Public Members

struct net_buf_simple ***property_ids**

Buffer contains a sequence of N User Property IDs

struct **esp_ble_mesh_gen_user_property_status_cb_t**

Parameters of Generic User Property Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID identifying a Generic User Property

uint8_t **user_access**

Enumeration indicating user access (optional)

struct net_buf_simple ***property_value**

Raw value for the User Property (C.1)

struct **esp_ble_mesh_gen_admin_properties_status_cb_t**

Parameter of Generic Admin Properties Status.

Public Members

struct net_buf_simple ***property_ids**

Buffer contains a sequence of N Admin Property IDs

struct **esp_ble_mesh_gen_admin_property_status_cb_t**

Parameters of Generic Admin Property Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID identifying a Generic Admin Property

uint8_t **user_access**

Enumeration indicating user access (optional)

struct net_buf_simple ***property_value**

Raw value for the Admin Property (C.1)

struct **esp_ble_mesh_gen_manufacturer_properties_status_cb_t**

Parameter of Generic Manufacturer Properties Status.

Public Members

struct net_buf_simple ***property_ids**

Buffer contains a sequence of N Manufacturer Property IDs

struct **esp_ble_mesh_gen_manufacturer_property_status_cb_t**

Parameters of Generic Manufacturer Property Status.

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID identifying a Generic Manufacturer Property

uint8_t **user_access**

Enumeration indicating user access (optional)

struct net_buf_simple ***property_value**

Raw value for the Manufacturer Property (C.1)

struct **esp_ble_mesh_gen_client_properties_status_cb_t**

Parameter of Generic Client Properties Status.

Public Members

struct net_buf_simple ***property_ids**

Buffer contains a sequence of N Client Property IDs

struct **esp_ble_mesh_generic_client_cb_param_t**

Generic Client Model callback parameters

Public Members

int **error_code**

Appropriate error code

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_gen_client_status_cb_t **status_cb**

The generic status message callback values

struct **esp_ble_mesh_gen_onoff_state_t**

Parameters of Generic OnOff state

Public Members

uint8_t **onoff**

The present value of the Generic OnOff state

uint8_t **target_onoff**

The target value of the Generic OnOff state

struct **esp_ble_mesh_gen_onoff_srv_t**

User data of Generic OnOff Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic OnOff Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_onoff_state_t **state**

Parameters of the Generic OnOff state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

struct **esp_ble_mesh_gen_level_state_t**

Parameters of Generic Level state

Public Members

int16_t **level**

The present value of the Generic Level state

int16_t **target_level**

The target value of the Generic Level state

int16_t **last_level**

When a new transaction starts, level should be set to last_last, and use "level + incoming delta" to calculate the target level. In another word, "last_level" is used to record "level" of the last transaction, and "last_delta" is used to record the previously received delta_level value. The last value of the Generic Level state

int32_t **last_delta**

The last delta change of the Generic Level state

bool **move_start**

Indicate if the transition of the Generic Level state has been started

bool **positive**

Indicate if the transition is positive or negative

struct **esp_ble_mesh_gen_level_srv_t**

User data of Generic Level Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Level Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_level_state_t **state**

Parameters of the Generic Level state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_level**

Delta change value of level state transition

struct **esp_ble_mesh_gen_def_trans_time_state_t**

Parameter of Generic Default Transition Time state

Public Members

uint8_t **trans_time**

The value of the Generic Default Transition Time state

struct **esp_ble_mesh_gen_def_trans_time_srv_t**

User data of Generic Default Transition Time Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Default Transition Time Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_def_trans_time_state_t **state**

Parameters of the Generic Default Transition Time state

struct **esp_ble_mesh_gen_onpowerup_state_t**

Parameter of Generic OnPowerUp state

Public Members

uint8_t **onpowerup**

The value of the Generic OnPowerUp state

struct **esp_ble_mesh_gen_power_onoff_srv_t**

User data of Generic Power OnOff Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Power OnOff Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_onpowerup_state_t ***state**

Parameters of the Generic OnPowerUp state

struct **esp_ble_mesh_gen_power_onoff_setup_srv_t**

User data of Generic Power OnOff Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Power OnOff Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_onpowerup_state_t ***state**

Parameters of the Generic OnPowerUp state

struct **esp_ble_mesh_gen_power_level_state_t**

Parameters of Generic Power Level state

Public Members

uint16_t **power_actual**

The present value of the Generic Power Actual state

uint16_t **target_power_actual**

The target value of the Generic Power Actual state

uint16_t **power_last**

The value of the Generic Power Last state

uint16_t **power_default**

The value of the Generic Power Default state

uint8_t **status_code**

The status code of setting Generic Power Range state

uint16_t **power_range_min**

The minimum value of the Generic Power Range state

uint16_t **power_range_max**

The maximum value of the Generic Power Range state

struct **esp_ble_mesh_gen_power_level_srv_t**

User data of Generic Power Level Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Power Level Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_power_level_state_t ***state**

Parameters of the Generic Power Level state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_level**

Delta change value of level state transition

struct **esp_ble_mesh_gen_power_level_setup_srv_t**

User data of Generic Power Level Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Power Level Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_power_level_state_t ***state**

Parameters of the Generic Power Level state

struct **esp_ble_mesh_gen_battery_state_t**

Parameters of Generic Battery state

Public Members

uint32_t **battery_level**

The value of the Generic Battery Level state

uint32_t **time_to_discharge**

The value of the Generic Battery Time to Discharge state

uint32_t **time_to_charge**

The value of the Generic Battery Time to Charge state

uint32_t **battery_flags**

The value of the Generic Battery Flags state

struct **esp_ble_mesh_gen_battery_srv_t**

User data of Generic Battery Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Battery Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_battery_state_t **state**

Parameters of the Generic Battery state

struct **esp_ble_mesh_gen_location_state_t**

Parameters of Generic Location state

Public Members

int32_t **global_latitude**

The value of the Global Latitude field

int32_t **global_longitude**

The value of the Global Longitude field

int16_t **global_altitude**

The value of the Global Altitude field

int16_t **local_north**

The value of the Local North field

int16_t **local_east**

The value of the Local East field

int16_t **local_altitude**

The value of the Local Altitude field

uint8_t **floor_number**

The value of the Floor Number field

uint16_t **uncertainty**

The value of the Uncertainty field

struct **esp_ble_mesh_gen_location_srv_t**

User data of Generic Location Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Location Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_location_state_t ***state**

Parameters of the Generic Location state

struct **esp_ble_mesh_gen_location_setup_srv_t**

User data of Generic Location Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Location Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_gen_location_state_t ***state**

Parameters of the Generic Location state

struct **esp_ble_mesh_generic_property_t**

Parameters of Generic Property states

Public Members

uint16_t **id**

The value of User/Admin/Manufacturer Property ID

uint8_t **user_access**

The value of User Access field

uint8_t **admin_access**

The value of Admin Access field

uint8_t **manu_access**

The value of Manufacturer Access field

struct net_buf_simple ***val**

The value of User/Admin/Manufacturer Property

struct **esp_ble_mesh_gen_user_prop_srv_t**

User data of Generic User Property Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic User Property Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

uint8_t **property_count**

Generic User Property count

esp_ble_mesh_generic_property_t ***properties**

Parameters of the Generic User Property state

struct **esp_ble_mesh_gen_admin_prop_srv_t**

User data of Generic Admin Property Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Admin Property Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

uint8_t **property_count**

Generic Admin Property count

esp_ble_mesh_generic_property_t ***properties**

Parameters of the Generic Admin Property state

struct **esp_ble_mesh_gen_manu_prop_srv_t**

User data of Generic Manufacturer Property Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Manufacturer Property Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

uint8_t **property_count**

Generic Manufacturer Property count

esp_ble_mesh_generic_property_t ***properties**

Parameters of the Generic Manufacturer Property state

struct **esp_ble_mesh_gen_client_prop_srv_t**

User data of Generic Client Property Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Generic Client Property Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

uint8_t **id_count**

Generic Client Property ID count

uint16_t ***property_ids**

Parameters of the Generic Client Property state

struct **esp_ble_mesh_state_change_gen_onoff_set_t**

Parameter of Generic OnOff Set state change event

Public Members

uint8_t **onoff**

The value of Generic OnOff state

struct **esp_ble_mesh_state_change_gen_level_set_t**

Parameter of Generic Level Set state change event

Public Members

int16_t **level**

The value of Generic Level state

struct **esp_ble_mesh_state_change_gen_delta_set_t**

Parameter of Generic Delta Set state change event

Public Members

int16_t **level**

The value of Generic Level state

struct **esp_ble_mesh_state_change_gen_move_set_t**

Parameter of Generic Move Set state change event

Public Members

int16_t **level**

The value of Generic Level state

struct **esp_ble_mesh_state_change_gen_def_trans_time_set_t**

Parameter of Generic Default Transition Time Set state change event

Public Members

uint8_t **trans_time**

The value of Generic Default Transition Time state

struct **esp_ble_mesh_state_change_gen_onpowerup_set_t**

Parameter of Generic OnPowerUp Set state change event

Public Members

uint8_t **onpowerup**

The value of Generic OnPowerUp state

struct **esp_ble_mesh_state_change_gen_power_level_set_t**

Parameter of Generic Power Level Set state change event

Public Members

uint16_t **power**

The value of Generic Power Actual state

struct **esp_ble_mesh_state_change_gen_power_default_set_t**

Parameter of Generic Power Default Set state change event

Public Members

uint16_t **power**

The value of Generic Power Default state

struct **esp_ble_mesh_state_change_gen_power_range_set_t**

Parameters of Generic Power Range Set state change event

Public Members

uint16_t **range_min**

The minimum value of Generic Power Range state

uint16_t **range_max**

The maximum value of Generic Power Range state

struct **esp_ble_mesh_state_change_gen_loc_global_set_t**

Parameters of Generic Location Global Set state change event

Public Members

int32_t **latitude**

The Global Latitude value of Generic Location state

int32_t **longitude**

The Global Longitude value of Generic Location state

int16_t **altitude**

The Global Altitude value of Generic Location state

struct **esp_ble_mesh_state_change_gen_loc_local_set_t**

Parameters of Generic Location Local Set state change event

Public Members

int16_t **north**

The Local North value of Generic Location state

int16_t **east**

The Local East value of Generic Location state

int16_t **altitude**

The Local Altitude value of Generic Location state

uint8_t **floor_number**

The Floor Number value of Generic Location state

uint16_t **uncertainty**

The Uncertainty value of Generic Location state

struct **esp_ble_mesh_state_change_gen_user_property_set_t**

Parameters of Generic User Property Set state change event

Public Members

uint16_t **id**

The property id of Generic User Property state

struct net_buf_simple ***value**

The property value of Generic User Property state

struct **esp_ble_mesh_state_change_gen_admin_property_set_t**

Parameters of Generic Admin Property Set state change event

Public Members

uint16_t **id**

The property id of Generic Admin Property state

uint8_t **access**

The property access of Generic Admin Property state

struct net_buf_simple ***value**

The property value of Generic Admin Property state

struct **esp_ble_mesh_state_change_gen_manu_property_set_t**

Parameters of Generic Manufacturer Property Set state change event

Public Members

uint16_t **id**

The property id of Generic Manufacturer Property state

uint8_t **access**

The property value of Generic Manufacturer Property state

struct **esp_ble_mesh_server_recv_gen_user_property_get_t**

Context of the received Generic User Property Get message

Public Members

uint16_t **property_id**

Property ID identifying a Generic User Property

struct **esp_ble_mesh_server_recv_gen_admin_property_get_t**

Context of the received Generic Admin Property Get message

Public Members

uint16_t **property_id**

Property ID identifying a Generic Admin Property

struct **esp_ble_mesh_server_recv_gen_manufacturer_property_get_t**

Context of the received Generic Manufacturer Property message

Public Members

uint16_t **property_id**

Property ID identifying a Generic Manufacturer Property

struct **esp_ble_mesh_server_recv_gen_client_properties_get_t**

Context of the received Generic Client Properties Get message

Public Members

uint16_t **property_id**

A starting Client Property ID present within an element

struct **esp_ble_mesh_server_recv_gen_onoff_set_t**

Context of the received Generic OnOff Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint8_t **onoff**

Target value of Generic OnOff state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_gen_level_set_t**

Context of the received Generic Level Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

int16_t **level**

Target value of Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_gen_delta_set_t**

Context of the received Generic Delta Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

int32_t **delta_level**

Delta change of Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_gen_move_set_t**

Context of the received Generic Move Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

int16_t **delta_level**

Delta Level step to calculate Move speed for Generic Level state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_gen_def_trans_time_set_t**

Context of the received Generic Default Transition Time Set message

Public Members

uint8_t **trans_time**

The value of the Generic Default Transition Time state

struct **esp_ble_mesh_server_recv_gen_onpowerup_set_t**

Context of the received Generic OnPowerUp Set message

Public Members

uint8_t **onpowerup**

The value of the Generic OnPowerUp state

struct **esp_ble_mesh_server_recv_gen_power_level_set_t**

Context of the received Generic Power Level Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **power**

Target value of Generic Power Actual state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_gen_power_default_set_t**

Context of the received Generic Power Default Set message

Public Members

uint16_t **power**

The value of the Generic Power Default state

struct **esp_ble_mesh_server_recv_gen_power_range_set_t**

Context of the received Generic Power Range Set message

Public Members

uint16_t **range_min**

Value of Range Min field of Generic Power Range state

uint16_t **range_max**

Value of Range Max field of Generic Power Range state

struct **esp_ble_mesh_server_recv_gen_loc_global_set_t**

Context of the received Generic Location Global Set message

Public Members

int32_t **global_latitude**

Global Coordinates (Latitude)

int32_t **global_longitude**

Global Coordinates (Longitude)

int16_t **global_altitude**

Global Altitude

struct **esp_ble_mesh_server_recv_gen_loc_local_set_t**

Context of the received Generic Location Local Set message

Public Members

int16_t **local_north**

Local Coordinates (North)

int16_t **local_east**

Local Coordinates (East)

int16_t **local_altitude**

Local Altitude

uint8_t **floor_number**

Floor Number

uint16_t **uncertainty**

Uncertainty

struct **esp_ble_mesh_server_recv_gen_user_property_set_t**

Context of the received Generic User Property Set message

Public Members

uint16_t **property_id**

Property ID identifying a Generic User Property

struct net_buf_simple ***property_value**

Raw value for the User Property

struct **esp_ble_mesh_server_recv_gen_admin_property_set_t**

Context of the received Generic Admin Property Set message

Public Members

uint16_t **property_id**

Property ID identifying a Generic Admin Property

uint8_t **user_access**

Enumeration indicating user access

struct net_buf_simple ***property_value**

Raw value for the Admin Property

struct **esp_ble_mesh_server_recv_gen_manufacturer_property_set_t**

Context of the received Generic Manufacturer Property Set message

Public Members

uint16_t **property_id**

Property ID identifying a Generic Manufacturer Property

uint8_t **user_access**

Enumeration indicating user access

struct **esp_ble_mesh_generic_server_cb_param_t**

Generic Server Model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Generic Server Models

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received messages

esp_ble_mesh_generic_server_cb_value_t **value**

Value of the received Generic Messages

Macros

ESP_BLE_MESH_MODEL_GEN_ONOFF_CLI(cli_pub, cli_data)

Define a new Generic OnOff Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic OnOff Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic OnOff Client Model instance.

ESP_BLE_MESH_MODEL_GEN_LEVEL_CLI(cli_pub, cli_data)

Define a new Generic Level Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Level Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Level Client Model instance.

ESP_BLE_MESH_MODEL_GEN_DEF_TRANS_TIME_CLI(cli_pub, cli_data)

Define a new Generic Default Transition Time Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Default Transition Time Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.

- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Default Transition Time Client Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_ONOFF_CLI(cli_pub, cli_data)

Define a new Generic Power OnOff Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Power OnOff Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Power OnOff Client Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_LEVEL_CLI(cli_pub, cli_data)

Define a new Generic Power Level Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Power Level Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Power Level Client Model instance.

ESP_BLE_MESH_MODEL_GEN_BATTERY_CLI(cli_pub, cli_data)

Define a new Generic Battery Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Battery Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Battery Client Model instance.

ESP_BLE_MESH_MODEL_GEN_LOCATION_CLI(cli_pub, cli_data)

Define a new Generic Location Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Location Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Location Client Model instance.

ESP_BLE_MESH_MODEL_GEN_PROPERTY_CLI(cli_pub, cli_data)

Define a new Generic Property Client Model.

Note: This API needs to be called for each element on which the application needs to have a Generic Property Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Generic Location Client Model instance.

ESP_BLE_MESH_MODEL_GEN_ONOFF_SRV(srv_pub, srv_data)

Generic Server Models related context.

Define a new Generic OnOff Server Model.

Note: 1. The Generic OnOff Server Model is a root model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_onoff_srv_t*.

Returns New Generic OnOff Server Model instance.

ESP_BLE_MESH_MODEL_GEN_LEVEL_SRV(srv_pub, srv_data)

Define a new Generic Level Server Model.

Note: 1. The Generic Level Server Model is a root model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_level_srv_t*.

Returns New Generic Level Server Model instance.

ESP_BLE_MESH_MODEL_GEN_DEF_TRANS_TIME_SRV(srv_pub, srv_data)

Define a new Generic Default Transition Time Server Model.

Note: 1. The Generic Default Transition Time Server Model is a root model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_def_trans_time_srv_t*.

Returns New Generic Default Transition Time Server Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_ONOFF_SRV(srv_pub, srv_data)

Define a new Generic Power OnOff Server Model.

Note: 1. The Generic Power OnOff Server model extends the Generic OnOff Server model. When this model is present on an element, the corresponding Generic Power OnOff Setup Server model shall also be present.

a. This model may be used to represent a variety of devices that do not fit any of the model descriptions that have been defined but support the generic properties of On/Off.
b. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_power_onoff_srv_t*.

Returns New Generic Power OnOff Server Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_ONOFF_SETUP_SRV(srv_pub, srv_data)

Define a new Generic Power OnOff Setup Server Model.

Note: 1. The Generic Power OnOff Setup Server model extends the Generic Power OnOff Server model and the Generic Default Transition Time Server model.

a. This model shall support model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_power_onoff_setup_srv_t*.

Returns New Generic Power OnOff Setup Server Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_LEVEL_SRV(srv_pub, srv_data)

Define a new Generic Power Level Server Model.

Note: 1. The Generic Power Level Server model extends the Generic Power OnOff Server model and the Generic Level Server model. When this model is present on an Element, the corresponding Generic Power Level Setup Server model shall also be present.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_power_level_srv_t*.

Returns New Generic Power Level Server Model instance.

ESP_BLE_MESH_MODEL_GEN_POWER_LEVEL_SETUP_SRV(srv_pub, srv_data)

Define a new Generic Power Level Setup Server Model.

Note: 1. The Generic Power Level Setup Server model extends the Generic Power Level Server model and the Generic Power OnOff Setup Server model.

a. This model shall support model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_power_level_setup_srv_t*.

Returns New Generic Power Level Setup Server Model instance.

ESP_BLE_MESH_MODEL_GEN_BATTERY_SRV(srv_pub, srv_data)

Define a new Generic Battery Server Model.

Note: 1. The Generic Battery Server Model is a root model.

a. This model shall support model publication and model subscription.
b. The model may be used to represent an element that is powered by a battery.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_battery_srv_t*.

Returns New Generic Battery Server Model instance.

ESP_BLE_MESH_MODEL_GEN_LOCATION_SRV(srv_pub, srv_data)

Define a new Generic Location Server Model.

Note: 1. The Generic Location Server model is a root model. When this model is present on an Element, the corresponding Generic Location Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. The model may be used to represent an element that knows its location (global or local).

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_location_srv_t*.

Returns New Generic Location Server Model instance.

ESP_BLE_MESH_MODEL_GEN_LOCATION_SETUP_SRV(srv_pub, srv_data)

Define a new Generic Location Setup Server Model.

Note: 1. The Generic Location Setup Server model extends the Generic Location Server model.

a. This model shall support model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_location_setup_srv_t*.

Returns New Generic Location Setup Server Model instance.

ESP_BLE_MESH_MODEL_GEN_USER_PROP_SRV(srv_pub, srv_data)

Define a new Generic User Property Server Model.

Note: 1. The Generic User Property Server model is a root model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_user_prop_srv_t*.

Returns New Generic User Property Server Model instance.

ESP_BLE_MESH_MODEL_GEN_ADMIN_PROP_SRV(srv_pub, srv_data)

Define a new Generic Admin Property Server Model.

Note: 1. The Generic Admin Property Server model extends the Generic User Property Server model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_admin_prop_srv_t*.

Returns New Generic Admin Property Server Model instance.

ESP_BLE_MESH_MODEL_GEN_MANUFACTURER_PROP_SRV(srv_pub, srv_data)

Define a new Generic Manufacturer Property Server Model.

Note: 1. The Generic Manufacturer Property Server model extends the Generic User Property Server model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_manu_prop_srv_t*.

Returns New Generic Manufacturer Property Server Model instance.

ESP_BLE_MESH_MODEL_GEN_CLIENT_PROP_SRV(srv_pub, srv_data)

Define a new Generic User Property Server Model.

Note: 1. The Generic Client Property Server model is a root model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_gen_client_prop_srv_t*.

Returns New Generic Client Property Server Model instance.

Type Definitions

typedef void (***esp_ble_mesh_generic_client_cb_t**)(*esp_ble_mesh_generic_client_cb_event_t* event, *esp_ble_mesh_generic_client_cb_param_t* *param)

Bluetooth Mesh Generic Client Model function.

Generic Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_generic_server_cb_t**)(*esp_ble_mesh_generic_server_cb_event_t* event, *esp_ble_mesh_generic_server_cb_param_t* *param)

Bluetooth Mesh Generic Server Model function.

Generic Server Model callback function type

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_generic_client_cb_event_t**

This enum value is the event of Generic Client Model

Values:

enumerator **ESP_BLE_MESH_GENERIC_CLIENT_GET_STATE_EVT**

enumerator **ESP_BLE_MESH_GENERIC_CLIENT_SET_STATE_EVT**

enumerator **ESP_BLE_MESH_GENERIC_CLIENT_PUBLISH_EVT**

enumerator **ESP_BLE_MESH_GENERIC_CLIENT_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_GENERIC_CLIENT_EVT_MAX**

enum **esp_ble_mesh_gen_user_prop_access_t**

This enum value is the access value of Generic User Property

Values:

enumerator **ESP_BLE_MESH_GEN_USER_ACCESS_PROHIBIT**

enumerator **ESP_BLE_MESH_GEN_USER_ACCESS_READ**

enumerator **ESP_BLE_MESH_GEN_USER_ACCESS_WRITE**

enumerator **ESP_BLE_MESH_GEN_USER_ACCESS_READ_WRITE**

enum **esp_ble_mesh_gen_admin_prop_access_t**

This enum value is the access value of Generic Admin Property

Values:

enumerator **ESP_BLE_MESH_GEN_ADMIN_NOT_USER_PROP**

enumerator **ESP_BLE_MESH_GEN_ADMIN_ACCESS_READ**

enumerator **ESP_BLE_MESH_GEN_ADMIN_ACCESS_WRITE**

enumerator **ESP_BLE_MESH_GEN_ADMIN_ACCESS_READ_WRITE**

enum **esp_ble_mesh_gen_manu_prop_access_t**

This enum value is the access value of Generic Manufacturer Property

Values:

enumerator **ESP_BLE_MESH_GEN_MANU_NOT_USER_PROP**

enumerator **ESP_BLE_MESH_GEN_MANU_ACCESS_READ**

enum **esp_ble_mesh_generic_server_cb_event_t**

This enum value is the event of Generic Server Model

Values:

enumerator **ESP_BLE_MESH_GENERIC_SERVER_STATE_CHANGE_EVT**

i. When get_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, no event will be callback to the application layer when Generic Get messages are received.
ii. When set_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, this event will be callback to the application layer when Generic Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_GENERIC_SERVER_RECV_GET_MSG_EVT**

When get_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Generic Get messages are received.

enumerator **ESP_BLE_MESH_GENERIC_SERVER_RECV_SET_MSG_EVT**

When set_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Generic Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_GENERIC_SERVER_EVT_MAX**

Sensor Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_sensor_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_sensor_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_sensor_client_callback**(*esp_ble_mesh_sensor_client_cb_t* callback)

Register BLE Mesh Sensor Client Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_sensor_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_sensor_client_get_state_t* *get_state)

Get the value of Sensor Server Model states using the Sensor Client Model get messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_sensor_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer to sensor get message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_sensor_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_sensor_client_set_state_t* *set_state)

Set the value of Sensor Server Model states using the Sensor Client Model set messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_sensor_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer to sensor set message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_sensor_server_callback**(*esp_ble_mesh_sensor_server_cb_t* callback)

Register BLE Mesh Sensor Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_sensor_client_get_state_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Client Model get message union.

Public Members

esp_ble_mesh_sensor_descriptor_get_t **descriptor_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_DESCRIPTOR_GET

esp_ble_mesh_sensor_cadence_get_t **cadence_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_GET

esp_ble_mesh_sensor_settings_get_t **settings_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SETTINGS_GET

esp_ble_mesh_sensor_setting_get_t **setting_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_GET

esp_ble_mesh_sensor_get_t **sensor_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_GET

esp_ble_mesh_sensor_column_get_t **column_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_COLUMN_GET

esp_ble_mesh_sensor_series_get_t **series_get**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SERIES_GET

union **esp_ble_mesh_sensor_client_set_state_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Client Model set message union.

Public Members

esp_ble_mesh_sensor_cadence_set_t **cadence_set**

For ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_SET & ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_SET_UNACK

esp_ble_mesh_sensor_setting_set_t **setting_set**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_SET & ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_SET_UNACK

union **esp_ble_mesh_sensor_client_status_cb_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Client Model received message union.

Public Members

esp_ble_mesh_sensor_descriptor_status_cb_t **descriptor_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_DESCRIPTOR_STATUS

esp_ble_mesh_sensor_cadence_status_cb_t **cadence_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_CADENCE_STATUS

esp_ble_mesh_sensor_settings_status_cb_t **settings_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SETTINGS_STATUS

esp_ble_mesh_sensor_setting_status_cb_t **setting_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SETTING_STATUS

esp_ble_mesh_sensor_status_cb_t **sensor_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_STATUS

esp_ble_mesh_sensor_column_status_cb_t **column_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_COLUMN_STATUS

esp_ble_mesh_sensor_series_status_cb_t **series_status**

For ESP_BLE_MESH_MODEL_OP_SENSOR_SERIES_STATUS

union **esp_ble_mesh_sensor_server_state_change_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Server Model state change value union.

Public Members

esp_ble_mesh_state_change_sensor_cadence_set_t **sensor_cadence_set**

The recv_op in ctx can be used to decide which state is changed. Sensor Cadence Set

esp_ble_mesh_state_change_sensor_setting_set_t **sensor_setting_set**

Sensor Setting Set

union **esp_ble_mesh_sensor_server_recv_get_msg_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Server Model received get message union.

Public Members

esp_ble_mesh_server_recv_sensor_descriptor_get_t **sensor_descriptor**

Sensor Descriptor Get

esp_ble_mesh_server_recv_sensor_cadence_get_t **sensor_cadence**

Sensor Cadence Get

esp_ble_mesh_server_recv_sensor_settings_get_t **sensor_settings**

Sensor Settings Get

esp_ble_mesh_server_recv_sensor_setting_get_t **sensor_setting**

Sensor Setting Get

esp_ble_mesh_server_recv_sensor_get_t **sensor_data**

Sensor Get

esp_ble_mesh_server_recv_sensor_column_get_t **sensor_column**

Sensor Column Get

esp_ble_mesh_server_recv_sensor_series_get_t **sensor_series**

Sensor Series Get

union **esp_ble_mesh_sensor_server_recv_set_msg_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Server Model received set message union.

Public Members

esp_ble_mesh_server_recv_sensor_cadence_set_t **sensor_cadence**

Sensor Cadence Set

esp_ble_mesh_server_recv_sensor_setting_set_t **sensor_setting**

Sensor Setting Set

union **esp_ble_mesh_sensor_server_cb_value_t**

#include <esp_ble_mesh_sensor_model_api.h> Sensor Server Model callback value union.

Public Members

esp_ble_mesh_sensor_server_state_change_t **state_change**

ESP_BLE_MESH_SENSOR_SERVER_STATE_CHANGE_EVT

esp_ble_mesh_sensor_server_recv_get_msg_t **get**

ESP_BLE_MESH_SENSOR_SERVER_RECV_GET_MSG_EVT

esp_ble_mesh_sensor_server_recv_set_msg_t **set**

ESP_BLE_MESH_SENSOR_SERVER_RECV_SET_MSG_EVT

Structures

struct **esp_ble_mesh_sensor_descriptor_get_t**

Bluetooth Mesh Sensor Client Model Get and Set parameters structure.

Parameters of Sensor Descriptor Get

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID of a sensor (optional)

struct **esp_ble_mesh_sensor_cadence_get_t**

Parameter of Sensor Cadence Get

Public Members

uint16_t **property_id**

Property ID of a sensor

struct **esp_ble_mesh_sensor_cadence_set_t**

Parameters of Sensor Cadence Set

Public Members

uint16_t **property_id**

Property ID for the sensor

uint8_t **fast_cadence_period_divisor**

Divisor for the publish period

uint8_t **status_trigger_type**

The unit and format of the Status Trigger Delta fields

struct net_buf_simple ***status_trigger_delta_down**

Delta down value that triggers a status message

struct net_buf_simple ***status_trigger_delta_up**

Delta up value that triggers a status message

uint8_t **status_min_interval**

Minimum interval between two consecutive Status messages

struct net_buf_simple ***fast_cadence_low**

Low value for the fast cadence range

struct net_buf_simple *fast_cadence_high
Fast value for the fast cadence range

struct esp_ble_mesh_sensor_settings_get_t
Parameter of Sensor Settings Get

Public Members

uint16_t sensor_property_id
Property ID of a sensor

struct esp_ble_mesh_sensor_setting_get_t
Parameters of Sensor Setting Get

Public Members

uint16_t sensor_property_id
Property ID of a sensor

uint16_t sensor_setting_property_id
Setting ID identifying a setting within a sensor

struct esp_ble_mesh_sensor_setting_set_t
Parameters of Sensor Setting Set

Public Members

uint16_t sensor_property_id
Property ID identifying a sensor

uint16_t sensor_setting_property_id
Setting ID identifying a setting within a sensor

struct net_buf_simple *sensor_setting_raw
Raw value for the setting

struct esp_ble_mesh_sensor_get_t
Parameters of Sensor Get

Public Members

bool op_en
Indicate if optional parameters are included

uint16_t property_id
Property ID for the sensor (optional)

struct **esp_ble_mesh_sensor_column_get_t**

Parameters of Sensor Column Get

Public Members

uint16_t **property_id**

Property identifying a sensor

struct net_buf_simple ***raw_value_x**

Raw value identifying a column

struct **esp_ble_mesh_sensor_series_get_t**

Parameters of Sensor Series Get

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property identifying a sensor

struct net_buf_simple ***raw_value_x1**

Raw value identifying a starting column (optional)

struct net_buf_simple ***raw_value_x2**

Raw value identifying an ending column (C.1)

struct **esp_ble_mesh_sensor_descriptor_status_cb_t**

Bluetooth Mesh Sensor Client Model Get and Set callback parameters structure.

Parameter of Sensor Descriptor Status

Public Members

struct net_buf_simple ***descriptor**

Sequence of 8-octet sensor descriptors (optional)

struct **esp_ble_mesh_sensor_cadence_status_cb_t**

Parameters of Sensor Cadence Status

Public Members

uint16_t **property_id**

Property for the sensor

struct net_buf_simple ***sensor_cadence_value**

Value of sensor cadence state

struct **esp_ble_mesh_sensor_settings_status_cb_t**

Parameters of Sensor Settings Status

Public Members

uint16_t **sensor_property_id**

Property ID identifying a sensor

struct net_buf_simple ***sensor_setting_property_ids**

A sequence of N sensor setting property IDs (optional)

struct **esp_ble_mesh_sensor_setting_status_cb_t**

Parameters of Sensor Setting Status

Public Members

bool **op_en**

Indicate id optional parameters are included

uint16_t **sensor_property_id**

Property ID identifying a sensor

uint16_t **sensor_setting_property_id**

Setting ID identifying a setting within a sensor

uint8_t **sensor_setting_access**

Read/Write access rights for the setting (optional)

struct net_buf_simple ***sensor_setting_raw**

Raw value for the setting

struct **esp_ble_mesh_sensor_status_cb_t**

Parameter of Sensor Status

Public Members

struct net_buf_simple ***marshalled_sensor_data**

Value of sensor data state (optional)

struct **esp_ble_mesh_sensor_column_status_cb_t**

Parameters of Sensor Column Status

Public Members

uint16_t **property_id**

Property identifying a sensor and the Y axis

struct net_buf_simple ***sensor_column_value**

Left values of sensor column status

struct **esp_ble_mesh_sensor_series_status_cb_t**

Parameters of Sensor Series Status

Public Members

uint16_t **property_id**

Property identifying a sensor and the Y axis

struct net_buf_simple ***sensor_series_value**

Left values of sensor series status

struct **esp_ble_mesh_sensor_client_cb_param_t**

Sensor Client Model callback parameters

Public Members

int **error_code**

0: success, otherwise failure. For the error code values please refer to errno.h file. A negative sign is added to the standard error codes in errno.h.

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_sensor_client_status_cb_t **status_cb**

The sensor status message callback values

struct **esp_ble_mesh_sensor_descriptor_t**

Parameters of Sensor Descriptor state

Public Members

uint32_t **positive_tolerance**

The value of Sensor Positive Tolerance field

uint32_t **negative_tolerance**

The value of Sensor Negative Tolerance field

uint32_t **sampling_function**

The value of Sensor Sampling Function field

uint8_t **measure_period**

The value of Sensor Measurement Period field

uint8_t **update_interval**

The value of Sensor Update Interval field

struct **esp_ble_mesh_sensor_setting_t**

Parameters of Sensor Setting state

Public Members

uint16_t **property_id**

The value of Sensor Setting Property ID field

uint8_t **access**

The value of Sensor Setting Access field

struct net_buf_simple ***raw**

The value of Sensor Setting Raw field

struct **esp_ble_mesh_sensor_cadence_t**

Parameters of Sensor Cadence state

Public Members

uint8_t **period_divisor**

The value of Fast Cadence Period Divisor field

uint8_t **trigger_type**

The value of Status Trigger Type field

struct net_buf_simple ***trigger_delta_down**

Note: The parameter "size" in trigger_delta_down, trigger_delta_up, fast_cadence_low & fast_cadence_high indicates the exact length of these four parameters, and they are associated with the Sensor Property ID. Users need to initialize the "size" precisely. The value of Status Trigger Delta Down field

struct net_buf_simple ***trigger_delta_up**

The value of Status Trigger Delta Up field

uint8_t **min_interval**

The value of Status Min Interval field

struct net_buf_simple ***fast_cadence_low**

The value of Fast Cadence Low field

struct net_buf_simple ***fast_cadence_high**

The value of Fast Cadence High field

struct **esp_ble_mesh_sensor_data_t**

Parameters of Sensor Data state

Public Members

uint8_t **format**

Format A: The Length field is a 1-based uint4 value (valid range 0x0–0xF, representing range of 1 –16). Format B: The Length field is a 1-based uint7 value (valid range 0x0–0x7F, representing range of 1 – 127). The value 0x7F represents a length of zero. The value of the Sensor Data format

uint8_t **length**

The value of the Sensor Data length

struct net_buf_simple ***raw_value**

The value of Sensor Data raw value

struct **esp_ble_mesh_sensor_series_column_t**

Parameters of Sensor Series Column state

Public Members

struct net_buf_simple ***raw_value_x**

The value of Sensor Raw Value X field

struct net_buf_simple ***column_width**

The value of Sensor Column Width field

struct net_buf_simple ***raw_value_y**

The value of Sensor Raw Value Y field

struct **esp_ble_mesh_sensor_state_t**

Parameters of Sensor states

Public Members

uint16_t **sensor_property_id**

The value of Sensor Property ID field

esp_ble_mesh_sensor_descriptor_t **descriptor**

Parameters of the Sensor Descriptor state

const uint8_t **setting_count**

Multiple Sensor Setting states may be present for each sensor. The Sensor Setting Property ID values shall be unique for each Sensor Property ID that identifies a sensor within an element.

esp_ble_mesh_sensor_setting_t ***settings**

Parameters of the Sensor Setting state

esp_ble_mesh_sensor_cadence_t ***cadence**

The Sensor Cadence state may be not supported by sensors based on device properties referencing "non-scalar characteristics" such as "histograms" or "composite characteristics". Parameters of the Sensor Cadence state

esp_ble_mesh_sensor_data_t **sensor_data**

Parameters of the Sensor Data state

esp_ble_mesh_sensor_series_column_t **series_column**

Parameters of the Sensor Series Column state

struct **esp_ble_mesh_sensor_srv_t**

User data of Sensor Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Sensor Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

const uint8_t **state_count**

Sensor state count

esp_ble_mesh_sensor_state_t ***states**

Parameters of the Sensor states

struct **esp_ble_mesh_sensor_setup_srv_t**

User data of Sensor Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Sensor Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

const uint8_t **state_count**

Sensor state count

esp_ble_mesh_sensor_state_t ***states**

Parameters of the Sensor states

struct **esp_ble_mesh_state_change_sensor_cadence_set_t**

Parameters of Sensor Cadence Set state change event

Public Members

uint16_t **property_id**

The value of Sensor Property ID state

uint8_t **period_divisor**

The value of Fast Cadence Period Divisor state

uint8_t **trigger_type**

The value of Status Trigger Type state

struct net_buf_simple ***trigger_delta_down**

The value of Status Trigger Delta Down state

struct net_buf_simple ***trigger_delta_up**

The value of Status Trigger Delta Up state

uint8_t **min_interval**

The value of Status Min Interval state

struct net_buf_simple ***fast_cadence_low**

The value of Fast Cadence Low state

struct net_buf_simple ***fast_cadence_high**

The value of Fast Cadence High state

struct **esp_ble_mesh_state_change_sensor_setting_set_t**

Parameters of Sensor Setting Set state change event

Public Members

uint16_t **property_id**

The value of Sensor Property ID state

uint16_t **setting_property_id**

The value of Sensor Setting Property ID state

struct net_buf_simple ***setting_value**

The value of Sensor Property Value state

struct **esp_ble_mesh_server_recv_sensor_descriptor_get_t**

Context of the received Sensor Descriptor Get message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID of a sensor (optional)

struct **esp_ble_mesh_server_recv_sensor_cadence_get_t**

Context of the received Sensor Cadence Get message

Public Members

uint16_t **property_id**

Property ID of a sensor

struct **esp_ble_mesh_server_recv_sensor_settings_get_t**

Context of the received Sensor Settings Get message

Public Members

uint16_t **property_id**

Property ID of a sensor

struct **esp_ble_mesh_server_recv_sensor_setting_get_t**

Context of the received Sensor Setting Get message

Public Members

uint16_t **property_id**

Property ID of a sensor

uint16_t **setting_property_id**

Setting ID identifying a setting within a sensor

struct **esp_ble_mesh_server_recv_sensor_get_t**

Context of the received Sensor Get message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property ID for the sensor (optional)

struct **esp_ble_mesh_server_recv_sensor_column_get_t**

Context of the received Sensor Column Get message

Public Members

uint16_t **property_id**

Property identifying a sensor

struct net_buf_simple ***raw_value_x**

Raw value identifying a column

struct **esp_ble_mesh_server_recv_sensor_series_get_t**

Context of the received Sensor Series Get message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **property_id**

Property identifying a sensor

struct net_buf_simple ***raw_value**

Raw value containing X1 and X2 (optional)

struct **esp_ble_mesh_server_recv_sensor_cadence_set_t**

Context of the received Sensor Cadence Set message

Public Members

uint16_t **property_id**

Property ID for the sensor

struct net_buf_simple ***cadence**

Value of Sensor Cadence state

struct **esp_ble_mesh_server_recv_sensor_setting_set_t**

Context of the received Sensor Setting Set message

Public Members

uint16_t **property_id**

Property ID identifying a sensor

uint16_t **setting_property_id**

Setting ID identifying a setting within a sensor

struct net_buf_simple ***setting_raw**

Raw value for the setting

struct **esp_ble_mesh_sensor_server_cb_param_t**

Sensor Server Model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Sensor Server Models

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received messages

esp_ble_mesh_sensor_server_cb_value_t **value**

Value of the received Sensor Messages

Macros

ESP_BLE_MESH_MODEL_SENSOR_CLI(cli_pub, cli_data)

Define a new Sensor Client Model.

Note: This API needs to be called for each element on which the application needs to have a Sensor Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Sensor Client Model instance.

ESP_BLE_MESH_MODEL_SENSOR_SRV(srv_pub, srv_data)

Sensor Server Models related context.

Define a new Sensor Server Model.

Note: 1. The Sensor Server model is a root model. When this model is present on an element, the corresponding Sensor Setup Server model shall also be present.

a. This model shall support model publication and model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_sensor_srv_t*.

Returns New Sensor Server Model instance.

ESP_BLE_MESH_MODEL_SENSOR_SETUP_SRV(srv_pub, srv_data)

Define a new Sensor Setup Server Model.

Note: 1. The Sensor Setup Server model extends the Sensor Server model.

a. This model shall support model publication and model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.

- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_sensor_setup_srv_t*.

Returns New Sensor Setup Server Model instance.

ESP_BLE_MESH_INVALID_SENSOR_PROPERTY_ID

Invalid Sensor Property ID

ESP_BLE_MESH_SENSOR_PROPERTY_ID_LEN

Length of Sensor Property ID

ESP_BLE_MESH_SENSOR_DESCRIPTOR_LEN

Length of Sensor Descriptor state

ESP_BLE_MESH_SENSOR_UNSPECIFIED_POS_TOLERANCE

Unspecified Sensor Positive Tolerance

ESP_BLE_MESH_SENSOR_UNSPECIFIED_NEG_TOLERANCE

Unspecified Sensor Negative Tolerance

ESP_BLE_MESH_SENSOR_NOT_APPL_MEASURE_PERIOD

Not applicable Sensor Measurement Period

ESP_BLE_MESH_SENSOR_NOT_APPL_UPDATE_INTERVAL

Not applicable Sensor Update Interval

ESP_BLE_MESH_INVALID_SENSOR_SETTING_PROPERTY_ID

Invalid Sensor Setting Property ID

ESP_BLE_MESH_SENSOR_SETTING_PROPERTY_ID_LEN

Length of Sensor Setting Property ID

ESP_BLE_MESH_SENSOR_SETTING_ACCESS_LEN

Length of Sensor Setting Access

ESP_BLE_MESH_SENSOR_SETTING_ACCESS_READ

Sensor Setting Access - Read

ESP_BLE_MESH_SENSOR_SETTING_ACCESS_READ_WRITE

Sensor Setting Access - Read & Write

ESP_BLE_MESH_SENSOR_DIVISOR_TRIGGER_TYPE_LEN

Length of Sensor Divisor Trigger Type

ESP_BLE_MESH_SENSOR_STATUS_MIN_INTERVAL_LEN

Length of Sensor Status Min Interval

ESP_BLE_MESH_SENSOR_PERIOD_DIVISOR_MAX_VALUE

Maximum value of Sensor Period Divisor

ESP_BLE_MESH_SENSOR_STATUS_MIN_INTERVAL_MAX

Maximum value of Sensor Status Min Interval

ESP_BLE_MESH_SENSOR_STATUS_TRIGGER_TYPE_CHAR

Sensor Status Trigger Type - Format Type of the characteristic that the Sensor Property ID state references

ESP_BLE_MESH_SENSOR_STATUS_TRIGGER_TYPE_UINT16

Sensor Status Trigger Type - Format Type "uint16"

ESP_BLE_MESH_SENSOR_DATA_FORMAT_A

Sensor Data Format A

ESP_BLE_MESH_SENSOR_DATA_FORMAT_B

Sensor Data Format B

ESP_BLE_MESH_SENSOR_DATA_FORMAT_A_MPID_LEN

MPID length of Sensor Data Format A

ESP_BLE_MESH_SENSOR_DATA_FORMAT_B_MPID_LEN

MPID length of Sensor Data Format B

ESP_BLE_MESH_SENSOR_DATA_ZERO_LEN

Zero length of Sensor Data.

Note: The Length field is a 1-based uint7 value (valid range 0x0–0x7F, representing range of 1–127). The value 0x7F represents a length of zero.

ESP_BLE_MESH_GET_SENSOR_DATA_FORMAT(_data)

Get format of the sensor data.

Note: Multiple sensor data may be concatenated. Make sure the _data pointer is updated before getting the format of the corresponding sensor data.

Parameters

- **_data** -- Pointer to the start of the sensor data.

Returns Format of the sensor data.

ESP_BLE_MESH_GET_SENSOR_DATA_LENGTH(_data, _fmt)

Get length of the sensor data.

Note: Multiple sensor data may be concatenated. Make sure the _data pointer is updated before getting the length of the corresponding sensor data.

Parameters

- **_data** -- Pointer to the start of the sensor data.
- **_fmt** -- Format of the sensor data.

Returns Length (zero-based) of the sensor data.

ESP_BLE_MESH_GET_SENSOR_DATA_PROPERTY_ID(_data, _fmt)

Get Sensor Property ID of the sensor data.

Note: Multiple sensor data may be concatenated. Make sure the _data pointer is updated before getting Sensor Property ID of the corresponding sensor data.

Parameters
- **_data** -- Pointer to the start of the sensor data.
- **_fmt** -- Format of the sensor data.

Returns Sensor Property ID of the sensor data.

ESP_BLE_MESH_SENSOR_DATA_FORMAT_A_MPID(_len, _id)

Generate a MPID value for sensor data with Format A.

Note: 1. The Format field is 0b0 and indicates that Format A is used.

a. The Length field is a 1-based uint4 value (valid range 0x0–0xF, representing range of 1–16).
b. The Property ID is an 11-bit bit field representing 11 LSb of a Property ID.
c. This format may be used for Property Values that are not longer than 16 octets and for Property IDs less than 0x0800.

Parameters
- **_len** -- Length of Sensor Raw value.
- **_id** -- Sensor Property ID.

Returns 2-octet MPID value for sensor data with Format A.

ESP_BLE_MESH_SENSOR_DATA_FORMAT_B_MPID(_len, _id)

Generate a MPID value for sensor data with Format B.

Note: 1. The Format field is 0b1 and indicates Format B is used.

a. The Length field is a 1-based uint7 value (valid range 0x0–0x7F, representing range of 1–127). The value 0x7F represents a length of zero.
b. The Property ID is a 16-bit bit field representing a Property ID.
c. This format may be used for Property Values not longer than 128 octets and for any Property IDs. Property values longer than 128 octets are not supported by the Sensor Status message.
d. Exclude the generated 1-octet value, the 2-octet Sensor Property ID

Parameters
- **_len** -- Length of Sensor Raw value.
- **_id** -- Sensor Property ID.

Returns 3-octet MPID value for sensor data with Format B.

Type Definitions

typedef void (***esp_ble_mesh_sensor_client_cb_t**)(*esp_ble_mesh_sensor_client_cb_event_t* event, *esp_ble_mesh_sensor_client_cb_param_t* *param)

Bluetooth Mesh Sensor Client Model function.

Sensor Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (*`esp_ble_mesh_sensor_server_cb_t`)(*esp_ble_mesh_sensor_server_cb_event_t* event, *esp_ble_mesh_sensor_server_cb_param_t* *param)

Bluetooth Mesh Sensor Server Model function.

Sensor Server Model callback function type

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum `esp_ble_mesh_sensor_client_cb_event_t`

This enum value is the event of Sensor Client Model

Values:

enumerator `ESP_BLE_MESH_SENSOR_CLIENT_GET_STATE_EVT`

enumerator `ESP_BLE_MESH_SENSOR_CLIENT_SET_STATE_EVT`

enumerator `ESP_BLE_MESH_SENSOR_CLIENT_PUBLISH_EVT`

enumerator `ESP_BLE_MESH_SENSOR_CLIENT_TIMEOUT_EVT`

enumerator `ESP_BLE_MESH_SENSOR_CLIENT_EVT_MAX`

enum `esp_ble_mesh_sensor_sample_func`

This enum value is value of Sensor Sampling Function

Values:

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_UNSPECIFIED`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_INSTANTANEOUS`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_ARITHMETIC_MEAN`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_RMS`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_MAXIMUM`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_MINIMUM`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_ACCUMULATED`

enumerator `ESP_BLE_MESH_SAMPLE_FUNC_COUNT`

enum `esp_ble_mesh_sensor_server_cb_event_t`

This enum value is the event of Sensor Server Model

Values:

enumerator **ESP_BLE_MESH_SENSOR_SERVER_STATE_CHANGE_EVT**

i. When get_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, no event will be callback to the application layer when Sensor Get messages are received.
ii. When set_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, this event will be callback to the application layer when Sensor Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_SENSOR_SERVER_RECV_GET_MSG_EVT**

When get_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Sensor Get messages are received.

enumerator **ESP_BLE_MESH_SENSOR_SERVER_RECV_SET_MSG_EVT**

When set_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Sensor Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_SENSOR_SERVER_EVT_MAX**

Time and Scenes Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_time_scene_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_time_scene_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_time_scene_client_callback**(*esp_ble_mesh_time_scene_client_cb_t* callback)

Register BLE Mesh Time Scene Client Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_time_scene_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_time_scene_client_get_state_t* *get_state)

Get the value of Time Scene Server Model states using the Time Scene Client Model get messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_time_scene_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer to time scene get message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_time_scene_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_time_scene_client_set_state_t* *set_state)

Set the value of Time Scene Server Model states using the Time Scene Client Model set messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_time_scene_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer to time scene set message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_time_scene_server_callback**(*esp_ble_mesh_time_scene_server_cb_t* callback)

Register BLE Mesh Time and Scenes Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_time_scene_client_get_state_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Client Model get message union.

Public Members

esp_ble_mesh_scheduler_act_get_t **scheduler_act_get**

For ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_GET

union **esp_ble_mesh_time_scene_client_set_state_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Client Model set message union.

Public Members

esp_ble_mesh_time_set_t **time_set**

For ESP_BLE_MESH_MODEL_OP_TIME_SET

esp_ble_mesh_time_zone_set_t **time_zone_set**

For ESP_BLE_MESH_MODEL_OP_TIME_ZONE_SET

esp_ble_mesh_tai_utc_delta_set_t **tai_utc_delta_set**

For ESP_BLE_MESH_MODEL_OP_TAI_UTC_DELTA_SET

esp_ble_mesh_time_role_set_t **time_role_set**

For ESP_BLE_MESH_MODEL_OP_TIME_ROLE_SET

esp_ble_mesh_scene_store_t **scene_store**

For ESP_BLE_MESH_MODEL_OP_SCENE_STORE & ESP_BLE_MESH_MODEL_OP_SCENE_STORE_UNACK

esp_ble_mesh_scene_recall_t **scene_recall**

For ESP_BLE_MESH_MODEL_OP_SCENE_RECALL & ESP_BLE_MESH_MODEL_OP_SCENE_RECALL_UNA

esp_ble_mesh_scene_delete_t **scene_delete**

For ESP_BLE_MESH_MODEL_OP_SCENE_DELETE & ESP_BLE_MESH_MODEL_OP_SCENE_DELETE_UNAC

esp_ble_mesh_scheduler_act_set_t **scheduler_act_set**

For ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_SET & ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_SET_UNACK

union **esp_ble_mesh_time_scene_client_status_cb_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Client Model received message union.

Public Members

esp_ble_mesh_time_status_cb_t **time_status**

For ESP_BLE_MESH_MODEL_OP_TIME_STATUS

esp_ble_mesh_time_zone_status_cb_t **time_zone_status**

For ESP_BLE_MESH_MODEL_OP_TIME_ZONE_STATUS

esp_ble_mesh_tai_utc_delta_status_cb_t **tai_utc_delta_status**

For ESP_BLE_MESH_MODEL_OP_TAI_UTC_DELTA_STATUS

esp_ble_mesh_time_role_status_cb_t **time_role_status**

For ESP_BLE_MESH_MODEL_OP_TIME_ROLE_STATUS

esp_ble_mesh_scene_status_cb_t **scene_status**

For ESP_BLE_MESH_MODEL_OP_SCENE_STATUS

esp_ble_mesh_scene_register_status_cb_t **scene_register_status**

For ESP_BLE_MESH_MODEL_OP_SCENE_REGISTER_STATUS

esp_ble_mesh_scheduler_status_cb_t **scheduler_status**

For ESP_BLE_MESH_MODEL_OP_SCHEDULER_STATUS

esp_ble_mesh_scheduler_act_status_cb_t **scheduler_act_status**

For ESP_BLE_MESH_MODEL_OP_SCHEDULER_ACT_STATUS

union **esp_ble_mesh_time_scene_server_state_change_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Server Model state change value union.

Public Members

esp_ble_mesh_state_change_time_set_t **time_set**

The recv_op in ctx can be used to decide which state is changed. Time Set

esp_ble_mesh_state_change_time_status_t **time_status**

Time Status

esp_ble_mesh_state_change_time_zone_set_t **time_zone_set**

Time Zone Set

esp_ble_mesh_state_change_tai_utc_delta_set_t **tai_utc_delta_set**

TAI UTC Delta Set

esp_ble_mesh_state_change_time_role_set_t **time_role_set**

Time Role Set

esp_ble_mesh_state_change_scene_store_t **scene_store**

Scene Store

esp_ble_mesh_state_change_scene_recall_t **scene_recall**

Scene Recall

esp_ble_mesh_state_change_scene_delete_t **scene_delete**

Scene Delete

esp_ble_mesh_state_change_scheduler_act_set_t **scheduler_act_set**

Scheduler Action Set

union **esp_ble_mesh_time_scene_server_recv_get_msg_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Server Model received get message union.

Public Members

esp_ble_mesh_server_recv_scheduler_act_get_t **scheduler_act**

Scheduler Action Get

union **esp_ble_mesh_time_scene_server_recv_set_msg_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Server Model received set message union.

Public Members

esp_ble_mesh_server_recv_time_set_t **time**

Time Set

esp_ble_mesh_server_recv_time_zone_set_t **time_zone**

Time Zone Set

esp_ble_mesh_server_recv_tai_utc_delta_set_t **tai_utc_delta**

TAI-UTC Delta Set

esp_ble_mesh_server_recv_time_role_set_t **time_role**

Time Role Set

esp_ble_mesh_server_recv_scene_store_t **scene_store**

Scene Store/Scene Store Unack

esp_ble_mesh_server_recv_scene_recall_t **scene_recall**

Scene Recall/Scene Recall Unack

esp_ble_mesh_server_recv_scene_delete_t **scene_delete**

Scene Delete/Scene Delete Unack

esp_ble_mesh_server_recv_scheduler_act_set_t **scheduler_act**

Scheduler Action Set/Scheduler Action Set Unack

union **esp_ble_mesh_time_scene_server_recv_status_msg_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Server Model received status message union.

Public Members

esp_ble_mesh_server_recv_time_status_t **time_status**

Time Status

union **esp_ble_mesh_time_scene_server_cb_value_t**

#include <esp_ble_mesh_time_scene_model_api.h> Time Scene Server Model callback value union.

Public Members

esp_ble_mesh_time_scene_server_state_change_t **state_change**

ESP_BLE_MESH_TIME_SCENE_SERVER_STATE_CHANGE_EVT

esp_ble_mesh_time_scene_server_recv_get_msg_t **get**

ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_GET_MSG_EVT

esp_ble_mesh_time_scene_server_recv_set_msg_t **set**

ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_SET_MSG_EVT

esp_ble_mesh_time_scene_server_recv_status_msg_t **status**

ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_STATUS_MSG_EVT

Structures

struct **esp_ble_mesh_time_set_t**

Bluetooth Mesh Time Scene Client Model Get and Set parameters structure.

Parameters of Time Set

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **sub_second**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_time_zone_set_t**

Parameters of Time Zone Set

Public Members

uint8_t **time_zone_offset_new**

Upcoming local time zone offset

uint8_t **tai_zone_change**[5]

TAI Seconds time of the upcoming Time Zone Offset change

struct **esp_ble_mesh_tai_utc_delta_set_t**

Parameters of TAI-UTC Delta Set

Public Members

uint16_t **tai_utc_delta_new**

Upcoming difference between TAI and UTC in seconds

uint16_t **padding**

Always 0b0. Other values are Prohibited.

uint8_t **tai_delta_change**[5]

TAI Seconds time of the upcoming TAI-UTC Delta change

struct **esp_ble_mesh_time_role_set_t**

Parameter of Time Role Set

Public Members

uint8_t **time_role**

The Time Role for the element

struct **esp_ble_mesh_scene_store_t**

Parameter of Scene Store

Public Members

uint16_t **scene_number**

The number of scenes to be stored

struct **esp_ble_mesh_scene_recall_t**

Parameters of Scene Recall

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **scene_number**

The number of scenes to be recalled

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_scene_delete_t**

Parameter of Scene Delete

Public Members

uint16_t **scene_number**

The number of scenes to be deleted

struct **esp_ble_mesh_scheduler_act_get_t**

Parameter of Scheduler Action Get

Public Members

uint8_t **index**

Index of the Schedule Register entry to get

struct **esp_ble_mesh_scheduler_act_set_t**

Parameters of Scheduler Action Set

Public Members

uint64_t **index**

Index of the Schedule Register entry to set

uint64_t **year**

Scheduled year for the action

uint64_t **month**

Scheduled month for the action

uint64_t **day**

Scheduled day of the month for the action

uint64_t **hour**

Scheduled hour for the action

uint64_t **minute**

Scheduled minute for the action

uint64_t **second**

Scheduled second for the action

uint64_t **day_of_week**

Schedule days of the week for the action

uint64_t **action**

Action to be performed at the scheduled time

uint64_t **trans_time**

Transition time for this action

uint16_t **scene_number**

Transition time for this action

struct **esp_ble_mesh_time_status_cb_t**

Bluetooth Mesh Time Scene Client Model Get and Set callback parameters structure.

Parameters of Time Status

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **sub_second**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_time_zone_status_cb_t**

Parameters of Time Zone Status

Public Members

uint8_t **time_zone_offset_curr**

Current local time zone offset

uint8_t **time_zone_offset_new**

Upcoming local time zone offset

uint8_t **tai_zone_change**[5]

TAI Seconds time of the upcoming Time Zone Offset change

struct **esp_ble_mesh_tai_utc_delta_status_cb_t**

Parameters of TAI-UTC Delta Status

Public Members

uint16_t **tai_utc_delta_curr**

Current difference between TAI and UTC in seconds

uint16_t **padding_1**

Always 0b0. Other values are Prohibited.

uint16_t **tai_utc_delta_new**

Upcoming difference between TAI and UTC in seconds

uint16_t **padding_2**

Always 0b0. Other values are Prohibited.

uint8_t **tai_delta_change**[5]

TAI Seconds time of the upcoming TAI-UTC Delta change

struct **esp_ble_mesh_time_role_status_cb_t**

Parameter of Time Role Status

Public Members

uint8_t **time_role**

The Time Role for the element

struct **esp_ble_mesh_scene_status_cb_t**

Parameters of Scene Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint8_t **status_code**

Status code of the last operation

uint16_t **current_scene**

Scene Number of the current scene

uint16_t **target_scene**

Scene Number of the target scene (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_scene_register_status_cb_t**

Parameters of Scene Register Status

Public Members

uint8_t **status_code**

Status code for the previous operation

uint16_t **current_scene**

Scene Number of the current scene

struct net_buf_simple ***scenes**

A list of scenes stored within an element

struct **esp_ble_mesh_scheduler_status_cb_t**

Parameter of Scheduler Status

Public Members

uint16_t **schedules**

Bit field indicating defined Actions in the Schedule Register

struct **esp_ble_mesh_scheduler_act_status_cb_t**

Parameters of Scheduler Action Status

Public Members

uint64_t **index**

Enumerates (selects) a Schedule Register entry

uint64_t **year**

Scheduled year for the action

uint64_t **month**

Scheduled month for the action

uint64_t **day**

Scheduled day of the month for the action

uint64_t **hour**

Scheduled hour for the action

uint64_t **minute**

Scheduled minute for the action

uint64_t **second**

Scheduled second for the action

uint64_t **day_of_week**

Schedule days of the week for the action

uint64_t **action**

Action to be performed at the scheduled time

uint64_t **trans_time**

Transition time for this action

uint16_t **scene_number**

Transition time for this action

struct **esp_ble_mesh_time_scene_client_cb_param_t**

Time Scene Client Model callback parameters

Public Members

int **error_code**

Appropriate error code

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_time_scene_client_status_cb_t **status_cb**

The scene status message callback values

struct **esp_ble_mesh_time_state_t**

Parameters of Time state

Public Members

uint8_t **tai_seconds**[5]

The value of the TAI Seconds state

uint8_t **subsecond**

The value of the Subsecond field

uint8_t **uncertainty**

The value of the Uncertainty field

uint8_t **time_zone_offset_curr**

The value of the Time Zone Offset Current field

uint8_t **time_zone_offset_new**

The value of the Time Zone Offset New state

uint8_t **tai_zone_change**[5]

The value of the TAI of Zone Change field

uint16_t **time_authority**

The value of the Time Authority bit

uint16_t **tai_utc_delta_curr**

The value of the TAI-UTC Delta Current state

uint16_t **tai_utc_delta_new**

The value of the TAI-UTC Delta New state

uint8_t **tai_delta_change**[5]

The value of the TAI of Delta Change field

struct *esp_ble_mesh_time_state_t*::[anonymous] **time**

Parameters of the Time state

uint8_t **time_role**

The value of the Time Role state

struct **esp_ble_mesh_time_srv_t**

User data of Time Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Time Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_time_state_t ***state**

Parameters of the Time state

struct **esp_ble_mesh_time_setup_srv_t**

User data of Time Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Time Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_time_state_t ***state**

Parameters of the Time state

struct **esp_ble_mesh_scene_register_t**

a. Scene Store is an operation of storing values of a present state of an element.
b. The structure and meaning of the stored state is determined by a model. States to be stored are specified by each model.
c. The Scene Store operation shall persistently store all values of all states marked as Stored with Scene for all models present on all elements of a node.
d. If a model is extending another model, the extending model shall determine the Stored with Scene behavior of that model. Parameters of Scene Register state

Public Members

uint16_t **scene_number**

The value of the Scene Number

uint8_t **scene_type**

The value of the Scene Type

struct net_buf_simple ***scene_value**

Scene value may use a union to represent later, the union contains structures of all the model states which can be stored in a scene. The value of the Scene Value

struct **esp_ble_mesh_scenes_state_t**

Parameters of Scenes state.

Scenes serve as memory banks for storage of states (e.g., a power level or a light level/color). Values of states of an element can be stored as a scene and can be recalled later from the scene memory.

A scene is represented by a Scene Number, which is a 16-bit non-zero, mesh-wide value. (There can be a maximum of 65535 scenes in a mesh network.) The meaning of a scene, as well as the state storage container associated with it, are determined by a model.

The Scenes state change may start numerous parallel model transitions. In that case, each individual model handles the transition internally.

The scene transition is defined as a group of individual model transitions started by a Scene Recall operation. The scene transition is in progress when at least one transition from the group of individual model transitions is in progress.

Public Members

const uint16_t **scene_count**

The Scenes state's scene count

esp_ble_mesh_scene_register_t ***scenes**

Parameters of the Scenes state

uint16_t **current_scene**

The Current Scene state is a 16-bit value that contains either the Scene Number of the currently active scene or a value of 0x0000 when no scene is active.

When a Scene Store operation or a Scene Recall operation completes with success, the Current Scene state value shall be to the Scene Number used during that operation.

When the Current Scene Number is deleted from a Scene Register state as a result of Scene Delete operation, the Current Scene state shall be set to 0x0000.

When any of the element's state that is marked as "Stored with Scene" has changed not as a result of a Scene Recall operation, the value of the Current Scene state shall be set to 0x0000.

When a scene transition is in progress, the value of the Current Scene state shall be set to 0x0000. The value of the Current Scene state

uint16_t **target_scene**

The Target Scene state is a 16-bit value that contains the target Scene Number when a scene transition is in progress.

When the scene transition is in progress and the target Scene Number is deleted from a Scene Register state as a result of Scene Delete operation, the Target Scene state shall be set to 0x0000.

When the scene transition is in progress and a new Scene Number is stored in the Scene Register as a result of Scene Store operation, the Target Scene state shall be set to the new Scene Number.

When the scene transition is not in progress, the value of the Target Scene state shall be set to 0x0000. The value of the Target Scene state

uint8_t **status_code**

The status code of the last scene operation

bool **in_progress**

Indicate if the scene transition is in progress

struct **esp_ble_mesh_scene_srv_t**

User data of Scene Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Scene Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_scenes_state_t ***state**

Parameters of the Scenes state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

struct **esp_ble_mesh_scene_setup_srv_t**

User data of Scene Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Scene Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_scenes_state_t ***state**

Parameters of the Scenes state

struct **esp_ble_mesh_schedule_register_t**

Parameters of Scheduler Register state

Public Members

bool **in_use**

Indicate if the registered schedule is in use

uint64_t **year**

The value of Scheduled year for the action

uint64_t **month**

The value of Scheduled month for the action

uint64_t **day**

The value of Scheduled day of the month for the action

uint64_t **hour**

The value of Scheduled hour for the action

uint64_t **minute**

The value of Scheduled minute for the action

uint64_t **second**

The value of Scheduled second for the action

uint64_t **day_of_week**

The value of Schedule days of the week for the action

uint64_t **action**

The value of Action to be performed at the scheduled time

uint64_t **trans_time**

The value of Transition time for this action

uint16_t **scene_number**

The value of Scene Number to be used for some actions

struct **esp_ble_mesh_scheduler_state_t**

Parameters of Scheduler state

Public Members

const uint8_t **schedule_count**

Scheduler count

esp_ble_mesh_schedule_register_t ***schedules**

Up to 16 scheduled entries

struct **esp_ble_mesh_scheduler_srv_t**

User data of Scheduler Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Scheduler Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_scheduler_state_t ***state**

Parameters of the Scheduler state

struct **esp_ble_mesh_scheduler_setup_srv_t**

User data of Scheduler Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Scheduler Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_scheduler_state_t ***state**

Parameters of the Scheduler state

struct **esp_ble_mesh_state_change_time_set_t**

Parameters of Time Set state change event

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **subsecond**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta_curr**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset_curr**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_state_change_time_status_t**

Parameters of Time Status state change event

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **subsecond**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta_curr**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset_curr**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_state_change_time_zone_set_t**

Parameters of Time Zone Set state change event

Public Members

uint8_t **time_zone_offset_new**

Upcoming local time zone offset

uint8_t **tai_zone_change**[5]

TAI Seconds time of the upcoming Time Zone Offset change

struct **esp_ble_mesh_state_change_tai_utc_delta_set_t**

Parameters of TAI UTC Delta Set state change event

Public Members

uint16_t **tai_utc_delta_new**

Upcoming difference between TAI and UTC in seconds

uint8_t **tai_delta_change**[5]

TAI Seconds time of the upcoming TAI-UTC Delta change

struct **esp_ble_mesh_state_change_time_role_set_t**

Parameter of Time Role Set state change event

Public Members

uint8_t **time_role**

The Time Role for the element

struct **esp_ble_mesh_state_change_scene_store_t**

Parameter of Scene Store state change event

Public Members

uint16_t **scene_number**

The number of scenes to be stored

struct **esp_ble_mesh_state_change_scene_recall_t**

Parameter of Scene Recall state change event

Public Members

uint16_t **scene_number**

The number of scenes to be recalled

struct **esp_ble_mesh_state_change_scene_delete_t**

Parameter of Scene Delete state change event

Public Members

uint16_t **scene_number**

The number of scenes to be deleted

struct **esp_ble_mesh_state_change_scheduler_act_set_t**

Parameter of Scheduler Action Set state change event

Public Members

uint64_t **index**

Index of the Schedule Register entry to set

uint64_t **year**

Scheduled year for the action

uint64_t **month**

Scheduled month for the action

uint64_t **day**

Scheduled day of the month for the action

uint64_t **hour**

Scheduled hour for the action

uint64_t **minute**

Scheduled minute for the action

uint64_t **second**

Scheduled second for the action

uint64_t **day_of_week**

Schedule days of the week for the action

uint64_t **action**

Action to be performed at the scheduled time

uint64_t **trans_time**

Transition time for this action

uint16_t **scene_number**

Scene number to be used for some actions

struct **esp_ble_mesh_server_recv_scheduler_act_get_t**

Context of the received Scheduler Action Get message

Public Members

uint8_t **index**

Index of the Schedule Register entry to get

struct **esp_ble_mesh_server_recv_time_set_t**

Context of the received Time Set message

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **subsecond**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_server_recv_time_zone_set_t**

Context of the received Time Zone Set message

Public Members

uint8_t **time_zone_offset_new**

Upcoming local time zone offset

uint8_t **tai_zone_change**[5]

TAI Seconds time of the upcoming Time Zone Offset change

struct **esp_ble_mesh_server_recv_tai_utc_delta_set_t**

Context of the received TAI UTC Delta Set message

Public Members

uint16_t **tai_utc_delta_new**

Upcoming difference between TAI and UTC in seconds

uint16_t **padding**

Always 0b0. Other values are Prohibited.

uint8_t **tai_delta_change**[5]

TAI Seconds time of the upcoming TAI-UTC Delta change

struct **esp_ble_mesh_server_recv_time_role_set_t**

Context of the received Time Role Set message

Public Members

uint8_t **time_role**

The Time Role for the element

struct **esp_ble_mesh_server_recv_scene_store_t**

Context of the received Scene Store message

Public Members

uint16_t **scene_number**

The number of scenes to be stored

struct **esp_ble_mesh_server_recv_scene_recall_t**

Context of the received Scene Recall message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **scene_number**

The number of scenes to be recalled

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_scene_delete_t**

Context of the received Scene Delete message

Public Members

uint16_t **scene_number**

The number of scenes to be deleted

struct **esp_ble_mesh_server_recv_scheduler_act_set_t**

Context of the received Scheduler Action Set message

Public Members

uint64_t **index**

Index of the Schedule Register entry to set

uint64_t **year**

Scheduled year for the action

uint64_t **month**

Scheduled month for the action

uint64_t **day**

Scheduled day of the month for the action

uint64_t **hour**

Scheduled hour for the action

uint64_t **minute**

Scheduled minute for the action

uint64_t **second**

Scheduled second for the action

uint64_t **day_of_week**

Schedule days of the week for the action

uint64_t **action**

Action to be performed at the scheduled time

uint64_t **trans_time**

Transition time for this action

uint16_t **scene_number**

Scene number to be used for some actions

struct **esp_ble_mesh_server_recv_time_status_t**

Context of the received Time Status message

Public Members

uint8_t **tai_seconds**[5]

The current TAI time in seconds

uint8_t **subsecond**

The sub-second time in units of 1/256 second

uint8_t **uncertainty**

The estimated uncertainty in 10-millisecond steps

uint16_t **time_authority**

0 = No Time Authority, 1 = Time Authority

uint16_t **tai_utc_delta**

Current difference between TAI and UTC in seconds

uint8_t **time_zone_offset**

The local time zone offset in 15-minute increments

struct **esp_ble_mesh_time_scene_server_cb_param_t**

Time Scene Server Model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Time and Scenes Server Models

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received messages

esp_ble_mesh_time_scene_server_cb_value_t **value**

Value of the received Time and Scenes Messages

Macros

ESP_BLE_MESH_MODEL_TIME_CLI(cli_pub, cli_data)

Define a new Time Client Model.

Note: This API needs to be called for each element on which the application needs to have a Time Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Time Client Model instance.

ESP_BLE_MESH_MODEL_SCENE_CLI(cli_pub, cli_data)

Define a new Scene Client Model.

Note: This API needs to be called for each element on which the application needs to have a Scene Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Scene Client Model instance.

ESP_BLE_MESH_MODEL_SCHEDULER_CLI(cli_pub, cli_data)

Define a new Scheduler Client Model.

Note: This API needs to be called for each element on which the application needs to have a Scheduler Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Scheduler Client Model instance.

ESP_BLE_MESH_MODEL_TIME_SRV(srv_pub, srv_data)

Time Scene Server Models related context.

Define a new Time Server Model.

Note: 1. The Time Server model is a root model. When this model is present on an Element, the corresponding Time Setup Server model shall also be present.

a. This model shall support model publication and model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_time_srv_t*.

Returns New Time Server Model instance.

ESP_BLE_MESH_MODEL_TIME_SETUP_SRV(srv_data)

Define a new Time Setup Server Model.

Note: 1. The Time Setup Server model extends the Time Server model. Time is sensitive information that is propagated across a mesh network.

a. Only an authorized Time Client should be allowed to change the Time and Time Role states. A dedicated application key Bluetooth SIG Proprietary should be used on the Time Setup Server to restrict access to the server to only authorized Time Clients.
b. This model does not support subscribing nor publishing.

Parameters

- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_time_setup_srv_t*.

Returns New Time Setup Server Model instance.

ESP_BLE_MESH_MODEL_SCENE_SRV(srv_pub, srv_data)

Define a new Scene Server Model.

Note: 1. The Scene Server model is a root model. When this model is present on an Element, the corresponding Scene Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. The model may be present only on the Primary element of a node.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_scene_srv_t*.

Returns New Scene Server Model instance.

ESP_BLE_MESH_MODEL_SCENE_SETUP_SRV(srv_pub, srv_data)

Define a new Scene Setup Server Model.

Note: 1. The Scene Setup Server model extends the Scene Server model and the Generic Default Transition Time Server model.

a. This model shall support model subscription.
b. The model may be present only on the Primary element of a node.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_scene_setup_srv_t*.

Returns New Scene Setup Server Model instance.

ESP_BLE_MESH_MODEL_SCHEDULER_SRV(srv_pub, srv_data)

Define a new Scheduler Server Model.

Note: 1. The Scheduler Server model extends the Scene Server model. When this model is present on an Element, the corresponding Scheduler Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. The model may be present only on the Primary element of a node.
c. The model requires the Time Server model shall be present on the element.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_scheduler_srv_t*.

Returns New Scheduler Server Model instance.

ESP_BLE_MESH_MODEL_SCHEDULER_SETUP_SRV(srv_pub, srv_data)

Define a new Scheduler Setup Server Model.

Note: 1. The Scheduler Setup Server model extends the Scheduler Server and the Scene Setup Server models.

a. This model shall support model subscription.
b. The model may be present only on the Primary element of a node.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_scheduler_setup_srv_t*.

Returns New Scheduler Setup Server Model instance.

ESP_BLE_MESH_UNKNOWN_TAI_SECONDS

Unknown TAI Seconds

ESP_BLE_MESH_UNKNOWN_TAI_ZONE_CHANGE

Unknown TAI of Zone Change

ESP_BLE_MESH_UNKNOWN_TAI_DELTA_CHANGE

Unknown TAI of Delta Change

ESP_BLE_MESH_TAI_UTC_DELTA_MAX_VALUE

Maximum TAI-UTC Delta value

ESP_BLE_MESH_TAI_SECONDS_LEN

Length of TAI Seconds

ESP_BLE_MESH_TAI_OF_ZONE_CHANGE_LEN

Length of TAI of Zone Change

ESP_BLE_MESH_TAI_OF_DELTA_CHANGE_LEN

Length of TAI of Delta Change

ESP_BLE_MESH_INVALID_SCENE_NUMBER

Invalid Scene Number

ESP_BLE_MESH_SCENE_NUMBER_LEN

Length of the Scene Number

ESP_BLE_MESH_SCHEDULE_YEAR_ANY_YEAR

Any year of the Scheduled year

ESP_BLE_MESH_SCHEDULE_DAY_ANY_DAY

Any day of the Scheduled day

ESP_BLE_MESH_SCHEDULE_HOUR_ANY_HOUR

Any hour of the Scheduled hour

ESP_BLE_MESH_SCHEDULE_HOUR_ONCE_A_DAY

Any hour of the Scheduled Day

ESP_BLE_MESH_SCHEDULE_SEC_ANY_OF_HOUR

Any minute of the Scheduled hour

ESP_BLE_MESH_SCHEDULE_SEC_EVERY_15_MIN

Every 15 minutes of the Scheduled hour

ESP_BLE_MESH_SCHEDULE_SEC_EVERY_20_MIN

Every 20 minutes of the Scheduled hour

ESP_BLE_MESH_SCHEDULE_SEC_ONCE_AN_HOUR

Once of the Scheduled hour

ESP_BLE_MESH_SCHEDULE_SEC_ANY_OF_MIN

Any second of the Scheduled minute

ESP_BLE_MESH_SCHEDULE_SEC_EVERY_15_SEC

Every 15 seconds of the Scheduled minute

ESP_BLE_MESH_SCHEDULE_SEC_EVERY_20_SEC

Every 20 seconds of the Scheduled minute

ESP_BLE_MESH_SCHEDULE_SEC_ONCE_AN_MIN

Once of the Scheduled minute

ESP_BLE_MESH_SCHEDULE_ACT_TURN_OFF

Scheduled Action - Turn Off

ESP_BLE_MESH_SCHEDULE_ACT_TURN_ON

Scheduled Action - Turn On

ESP_BLE_MESH_SCHEDULE_ACT_SCENE_RECALL

Scheduled Action - Scene Recall

ESP_BLE_MESH_SCHEDULE_ACT_NO_ACTION

Scheduled Action - No Action

ESP_BLE_MESH_SCHEDULE_SCENE_NO_SCENE

Scheduled Scene - No Scene

ESP_BLE_MESH_SCHEDULE_ENTRY_MAX_INDEX

Maximum number of Scheduled entries

ESP_BLE_MESH_TIME_NONE

Time Role - None

ESP_BLE_MESH_TIME_AUTHORITY

Time Role - Mesh Time Authority

ESP_BLE_MESH_TIME_RELAY

Time Role - Mesh Time Relay

ESP_BLE_MESH_TIME_CLIENT

Time Role - Mesh Time Client

ESP_BLE_MESH_SCENE_SUCCESS

Scene operation - Success

ESP_BLE_MESH_SCENE_REG_FULL

Scene operation - Scene Register Full

ESP_BLE_MESH_SCENE_NOT_FOUND

Scene operation - Scene Not Found

Type Definitions

typedef void (***esp_ble_mesh_time_scene_client_cb_t**)(*esp_ble_mesh_time_scene_client_cb_event_t* event, *esp_ble_mesh_time_scene_client_cb_param_t* *param)

Bluetooth Mesh Time Scene Client Model function.

Time Scene Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_time_scene_server_cb_t**)(*esp_ble_mesh_time_scene_server_cb_event_t* event, *esp_ble_mesh_time_scene_server_cb_param_t* *param)

Bluetooth Mesh Time and Scenes Server Model function.

Time Scene Server Model callback function type

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_time_scene_client_cb_event_t**

This enum value is the event of Time Scene Client Model

Values:

enumerator **ESP_BLE_MESH_TIME_SCENE_CLIENT_GET_STATE_EVT**

enumerator **ESP_BLE_MESH_TIME_SCENE_CLIENT_SET_STATE_EVT**

enumerator **ESP_BLE_MESH_TIME_SCENE_CLIENT_PUBLISH_EVT**

enumerator **ESP_BLE_MESH_TIME_SCENE_CLIENT_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_TIME_SCENE_CLIENT_EVT_MAX**

enum **esp_ble_mesh_time_scene_server_cb_event_t**

This enum value is the event of Time Scene Server Model

Values:

enumerator **ESP_BLE_MESH_TIME_SCENE_SERVER_STATE_CHANGE_EVT**

i. When get_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, no event will be callback to the application layer when Time Scene Get messages are received.
ii. When set_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, this event will be callback to the application layer when Time Scene Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_GET_MSG_EVT**

When get_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Time Scene Get messages are received.

enumerator **ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_SET_MSG_EVT**

When set_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Time Scene Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_TIME_SCENE_SERVER_RECV_STATUS_MSG_EVT**

When status_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when TIme Status message is received.

enumerator **ESP_BLE_MESH_TIME_SCENE_SERVER_EVT_MAX**

Lighting Client/Server Models

Header File

- components/bt/esp_ble_mesh/api/models/include/esp_ble_mesh_lighting_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_lighting_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_light_client_callback**(*esp_ble_mesh_light_client_cb_t* callback)

Register BLE Mesh Light Client Model callback.

Parameters **callback** -- **[in]** pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_light_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_light_client_get_state_t* *get_state)

Get the value of Light Server Model states using the Light Client Model get messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_light_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get_state** -- **[in]** Pointer of light get message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_light_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_light_client_set_state_t* *set_state)

Set the value of Light Server Model states using the Light Client Model set messages.

Note: If you want to know the opcodes and corresponding meanings accepted by this API, please refer to esp_ble_mesh_light_message_opcode_t in esp_ble_mesh_defs.h

---

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set_state** -- **[in]** Pointer of light set message value. Shall not be set to NULL.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_lighting_server_callback**(*esp_ble_mesh_lighting_server_cb_t* callback)

Register BLE Mesh Lighting Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_light_client_get_state_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Client Model get message union.

Public Members

esp_ble_mesh_light_lc_property_get_t **lc_property_get**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_GET

union **esp_ble_mesh_light_client_set_state_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Client Model set message union.

Public Members

esp_ble_mesh_light_lightness_set_t **lightness_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_SET_UNACK

esp_ble_mesh_light_lightness_linear_set_t **lightness_linear_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_SET_UNACK

esp_ble_mesh_light_lightness_default_set_t **lightness_default_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_SET_UNACK

esp_ble_mesh_light_lightness_range_set_t **lightness_range_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_SET_UNACK

esp_ble_mesh_light_ctl_set_t **ctl_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_SET_UNA

esp_ble_mesh_light_ctl_temperature_set_t **ctl_temperature_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_SET_UNACK

esp_ble_mesh_light_ctl_temperature_range_set_t **ctl_temperature_range_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_SET_UNACK

esp_ble_mesh_light_ctl_default_set_t **ctl_default_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_SET_UNACK

esp_ble_mesh_light_hsl_set_t **hsl_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SET_UNA

esp_ble_mesh_light_hsl_hue_set_t **hsl_hue_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE

esp_ble_mesh_light_hsl_saturation_set_t **hsl_saturation_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_SET_UNACK

esp_ble_mesh_light_hsl_default_set_t **hsl_default_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_SET_UNACK

esp_ble_mesh_light_hsl_range_set_t **hsl_range_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_SET_UNACK

esp_ble_mesh_light_xyl_set_t **xyl_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_SET_UNA

esp_ble_mesh_light_xyl_default_set_t **xyl_default_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_SET_UNACK

esp_ble_mesh_light_xyl_range_set_t **xyl_range_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_SET_UNACK

esp_ble_mesh_light_lc_mode_set_t **lc_mode_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_SET_UNACK

esp_ble_mesh_light_lc_om_set_t **lc_om_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_SET

esp_ble_mesh_light_lc_light_onoff_set_t **lc_light_onoff_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_SET_UNACK

esp_ble_mesh_light_lc_property_set_t **lc_property_set**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_SET & ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_SET_UNACK

union **esp_ble_mesh_light_client_status_cb_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Client Model received message union.

Public Members

esp_ble_mesh_light_lightness_status_cb_t **lightness_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_STATUS

esp_ble_mesh_light_lightness_linear_status_cb_t **lightness_linear_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LINEAR_STATUS

esp_ble_mesh_light_lightness_last_status_cb_t **lightness_last_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_LAST_STATUS

esp_ble_mesh_light_lightness_default_status_cb_t **lightness_default_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_DEFAULT_STATUS

esp_ble_mesh_light_lightness_range_status_cb_t **lightness_range_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LIGHTNESS_RANGE_STATUS

esp_ble_mesh_light_ctl_status_cb_t **ctl_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_STATUS

esp_ble_mesh_light_ctl_temperature_status_cb_t **ctl_temperature_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_STATUS

esp_ble_mesh_light_ctl_temperature_range_status_cb_t **ctl_temperature_range_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_TEMPERATURE_RANGE_STATUS

esp_ble_mesh_light_ctl_default_status_cb_t **ctl_default_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_CTL_DEFAULT_STATUS

esp_ble_mesh_light_hsl_status_cb_t **hsl_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_STATUS

esp_ble_mesh_light_hsl_target_status_cb_t **hsl_target_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_TARGET_STATUS

esp_ble_mesh_light_hsl_hue_status_cb_t **hsl_hue_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_HUE_STATUS

esp_ble_mesh_light_hsl_saturation_status_cb_t **hsl_saturation_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_SATURATION_STATUS

esp_ble_mesh_light_hsl_default_status_cb_t **hsl_default_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_DEFAULT_STATUS

esp_ble_mesh_light_hsl_range_status_cb_t **hsl_range_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_HSL_RANGE_STATUS

esp_ble_mesh_light_xyl_status_cb_t **xyl_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_STATUS

esp_ble_mesh_light_xyl_target_status_cb_t **xyl_target_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_TARGET_STATUS

esp_ble_mesh_light_xyl_default_status_cb_t **xyl_default_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_DEFAULT_STATUS

esp_ble_mesh_light_xyl_range_status_cb_t **xyl_range_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_XYL_RANGE_STATUS

esp_ble_mesh_light_lc_mode_status_cb_t **lc_mode_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_MODE_STATUS

esp_ble_mesh_light_lc_om_status_cb_t **lc_om_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_OM_STATUS

esp_ble_mesh_light_lc_light_onoff_status_cb_t **lc_light_onoff_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_LIGHT_ONOFF_STATUS

esp_ble_mesh_light_lc_property_status_cb_t **lc_property_status**

For ESP_BLE_MESH_MODEL_OP_LIGHT_LC_PROPERTY_STATUS

union **esp_ble_mesh_lighting_server_state_change_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Server Model state change value union.

Public Members

esp_ble_mesh_state_change_light_lightness_set_t **lightness_set**

The recv_op in ctx can be used to decide which state is changed. Light Lightness Set

esp_ble_mesh_state_change_light_lightness_linear_set_t **lightness_linear_set**

Light Lightness Linear Set

esp_ble_mesh_state_change_light_lightness_default_set_t **lightness_default_set**

Light Lightness Default Set

esp_ble_mesh_state_change_light_lightness_range_set_t **lightness_range_set**

Light Lightness Range Set

esp_ble_mesh_state_change_light_ctl_set_t **ctl_set**

Light CTL Set

esp_ble_mesh_state_change_light_ctl_temperature_set_t **ctl_temp_set**

Light CTL Temperature Set

esp_ble_mesh_state_change_light_ctl_temperature_range_set_t **ctl_temp_range_set**

Light CTL Temperature Range Set

esp_ble_mesh_state_change_light_ctl_default_set_t **ctl_default_set**

Light CTL Default Set

esp_ble_mesh_state_change_light_hsl_set_t **hsl_set**

Light HSL Set

esp_ble_mesh_state_change_light_hsl_hue_set_t **hsl_hue_set**

Light HSL Hue Set

esp_ble_mesh_state_change_light_hsl_saturation_set_t **hsl_saturation_set**

Light HSL Saturation Set

esp_ble_mesh_state_change_light_hsl_default_set_t **hsl_default_set**

Light HSL Default Set

esp_ble_mesh_state_change_light_hsl_range_set_t **hsl_range_set**

Light HSL Range Set

esp_ble_mesh_state_change_light_xyl_set_t **xyl_set**

Light xyL Set

esp_ble_mesh_state_change_light_xyl_default_set_t **xyl_default_set**

Light xyL Default Set

esp_ble_mesh_state_change_light_xyl_range_set_t **xyl_range_set**

Light xyL Range Set

esp_ble_mesh_state_change_light_lc_mode_set_t **lc_mode_set**

Light LC Mode Set

esp_ble_mesh_state_change_light_lc_om_set_t **lc_om_set**

Light LC Occupancy Mode Set

esp_ble_mesh_state_change_light_lc_light_onoff_set_t **lc_light_onoff_set**

Light LC Light OnOff Set

esp_ble_mesh_state_change_light_lc_property_set_t **lc_property_set**

Light LC Property Set

esp_ble_mesh_state_change_sensor_status_t **sensor_status**

Sensor Status

union **esp_ble_mesh_lighting_server_recv_get_msg_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Server Model received get message union.

Public Members

esp_ble_mesh_server_recv_light_lc_property_get_t **lc_property**

Light LC Property Get

union **esp_ble_mesh_lighting_server_recv_set_msg_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Server Model received set message union.

Public Members

esp_ble_mesh_server_recv_light_lightness_set_t **lightness**

Light Lightness Set/Light Lightness Set Unack

esp_ble_mesh_server_recv_light_lightness_linear_set_t **lightness_linear**

Light Lightness Linear Set/Light Lightness Linear Set Unack

esp_ble_mesh_server_recv_light_lightness_default_set_t **lightness_default**

Light Lightness Default Set/Light Lightness Default Set Unack

esp_ble_mesh_server_recv_light_lightness_range_set_t **lightness_range**

Light Lightness Range Set/Light Lightness Range Set Unack

esp_ble_mesh_server_recv_light_ctl_set_t **ctl**

Light CTL Set/Light CTL Set Unack

esp_ble_mesh_server_recv_light_ctl_temperature_set_t **ctl_temp**

Light CTL Temperature Set/Light CTL Temperature Set Unack

esp_ble_mesh_server_recv_light_ctl_temperature_range_set_t **ctl_temp_range**

Light CTL Temperature Range Set/Light CTL Temperature Range Set Unack

esp_ble_mesh_server_recv_light_ctl_default_set_t **ctl_default**

Light CTL Default Set/Light CTL Default Set Unack

esp_ble_mesh_server_recv_light_hsl_set_t **hsl**

Light HSL Set/Light HSL Set Unack

esp_ble_mesh_server_recv_light_hsl_hue_set_t **hsl_hue**

Light HSL Hue Set/Light HSL Hue Set Unack

esp_ble_mesh_server_recv_light_hsl_saturation_set_t **hsl_saturation**

Light HSL Saturation Set/Light HSL Saturation Set Unack

esp_ble_mesh_server_recv_light_hsl_default_set_t **hsl_default**

Light HSL Default Set/Light HSL Default Set Unack

esp_ble_mesh_server_recv_light_hsl_range_set_t **hsl_range**

Light HSL Range Set/Light HSL Range Set Unack

esp_ble_mesh_server_recv_light_xyl_set_t **xyl**

Light xyL Set/Light xyL Set Unack

esp_ble_mesh_server_recv_light_xyl_default_set_t **xyl_default**

Light xyL Default Set/Light xyL Default Set Unack

esp_ble_mesh_server_recv_light_xyl_range_set_t **xyl_range**

Light xyL Range Set/Light xyL Range Set Unack

esp_ble_mesh_server_recv_light_lc_mode_set_t **lc_mode**

Light LC Mode Set/Light LC Mode Set Unack

esp_ble_mesh_server_recv_light_lc_om_set_t **lc_om**

Light LC OM Set/Light LC OM Set Unack

esp_ble_mesh_server_recv_light_lc_light_onoff_set_t **lc_light_onoff**

Light LC Light OnOff Set/Light LC Light OnOff Set Unack

esp_ble_mesh_server_recv_light_lc_property_set_t **lc_property**

Light LC Property Set/Light LC Property Set Unack

union **esp_ble_mesh_lighting_server_recv_status_msg_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Server Model received status message union.

Public Members

esp_ble_mesh_server_recv_sensor_status_t **sensor_status**

Sensor Status

union **esp_ble_mesh_lighting_server_cb_value_t**

#include <esp_ble_mesh_lighting_model_api.h> Lighting Server Model callback value union.

Public Members

esp_ble_mesh_lighting_server_state_change_t **state_change**

ESP_BLE_MESH_LIGHTING_SERVER_STATE_CHANGE_EVT

esp_ble_mesh_lighting_server_recv_get_msg_t **get**

ESP_BLE_MESH_LIGHTING_SERVER_RECV_GET_MSG_EVT

esp_ble_mesh_lighting_server_recv_set_msg_t **set**

ESP_BLE_MESH_LIGHTING_SERVER_RECV_SET_MSG_EVT

esp_ble_mesh_lighting_server_recv_status_msg_t **status**

ESP_BLE_MESH_LIGHTING_SERVER_RECV_STATUS_MSG_EVT

Structures

struct **esp_ble_mesh_light_lightness_set_t**

Bluetooth Mesh Light Lightness Client Model Get and Set parameters structure.

Parameters of Light Lightness Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light lightness actual state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_lightness_linear_set_t**

Parameters of Light Lightness Linear Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light lightness linear state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_lightness_default_set_t**

Parameter of Light Lightness Default Set

Public Members

uint16_t **lightness**

The value of the Light Lightness Default state

struct **esp_ble_mesh_light_lightness_range_set_t**

Parameters of Light Lightness Range Set

Public Members

uint16_t **range_min**

Value of range min field of light lightness range state

uint16_t **range_max**

Value of range max field of light lightness range state

struct **esp_ble_mesh_light_ctl_set_t**

Parameters of Light CTL Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **ctl_lightness**

Target value of light ctl lightness state

uint16_t **ctl_temperature**

Target value of light ctl temperature state

int16_t **ctl_delta_uv**

Target value of light ctl delta UV state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_ctl_temperature_set_t**

Parameters of Light CTL Temperature Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **ctl_temperature**

Target value of light ctl temperature state

int16_t **ctl_delta_uv**

Target value of light ctl delta UV state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_ctl_temperature_range_set_t**

Parameters of Light CTL Temperature Range Set

Public Members

uint16_t **range_min**

Value of temperature range min field of light ctl temperature range state

uint16_t **range_max**

Value of temperature range max field of light ctl temperature range state

struct **esp_ble_mesh_light_ctl_default_set_t**

Parameters of Light CTL Default Set

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **temperature**

Value of light temperature default state

int16_t **delta_uv**

Value of light delta UV default state

struct **esp_ble_mesh_light_hsl_set_t**

Parameters of Light HSL Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **hsl_lightness**

Target value of light hsl lightness state

uint16_t **hsl_hue**

Target value of light hsl hue state

uint16_t **hsl_saturation**

Target value of light hsl saturation state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_hsl_hue_set_t**

Parameters of Light HSL Hue Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **hue**

Target value of light hsl hue state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_hsl_saturation_set_t**

Parameters of Light HSL Saturation Set

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **saturation**

Target value of light hsl hue state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_hsl_default_set_t**

Parameters of Light HSL Default Set

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **hue**

Value of light hue default state

uint16_t **saturation**

Value of light saturation default state

struct **esp_ble_mesh_light_hsl_range_set_t**

Parameters of Light HSL Range Set

Public Members

uint16_t **hue_range_min**

Value of hue range min field of light hsl hue range state

uint16_t **hue_range_max**

Value of hue range max field of light hsl hue range state

uint16_t **saturation_range_min**

Value of saturation range min field of light hsl saturation range state

uint16_t **saturation_range_max**

Value of saturation range max field of light hsl saturation range state

struct **esp_ble_mesh_light_xyl_set_t**

Parameters of Light xyL Set

Public Members

bool **op_en**

Indicate whether optional parameters included

uint16_t **xyl_lightness**

The target value of the Light xyL Lightness state

uint16_t **xyl_x**

The target value of the Light xyL x state

uint16_t **xyl_y**

The target value of the Light xyL y state

uint8_t **tid**

Transaction Identifier

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_xyl_default_set_t**

Parameters of Light xyL Default Set

Public Members

uint16_t **lightness**

The value of the Light Lightness Default state

uint16_t **xyl_x**

The value of the Light xyL x Default state

uint16_t **xyl_y**

The value of the Light xyL y Default state

struct **esp_ble_mesh_light_xyl_range_set_t**

Parameters of Light xyL Range Set

Public Members

uint16_t **xyl_x_range_min**

The value of the xyL x Range Min field of the Light xyL x Range state

uint16_t **xyl_x_range_max**

The value of the xyL x Range Max field of the Light xyL x Range state

uint16_t **xyl_y_range_min**

The value of the xyL y Range Min field of the Light xyL y Range state

uint16_t **xyl_y_range_max**

The value of the xyL y Range Max field of the Light xyL y Range state

struct **esp_ble_mesh_light_lc_mode_set_t**

Parameter of Light LC Mode Set

Public Members

uint8_t **mode**

The target value of the Light LC Mode state

struct **esp_ble_mesh_light_lc_om_set_t**

Parameter of Light LC OM Set

Public Members

uint8_t **mode**

The target value of the Light LC Occupancy Mode state

struct **esp_ble_mesh_light_lc_light_onoff_set_t**

Parameters of Light LC Light OnOff Set

Public Members

bool **op_en**

Indicate whether optional parameters included

uint8_t **light_onoff**

The target value of the Light LC Light OnOff state

uint8_t **tid**

Transaction Identifier

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_light_lc_property_get_t**

Parameter of Light LC Property Get

Public Members

uint16_t **property_id**

Property ID identifying a Light LC Property

struct **esp_ble_mesh_light_lc_property_set_t**

Parameters of Light LC Property Set

Public Members

uint16_t **property_id**

Property ID identifying a Light LC Property

struct net_buf_simple ***property_value**

Raw value for the Light LC Property

struct **esp_ble_mesh_light_lightness_status_cb_t**

Bluetooth Mesh Light Lightness Client Model Get and Set callback parameters structure.

Parameters of Light Lightness Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_lightness**

Current value of light lightness actual state

uint16_t **target_lightness**

Target value of light lightness actual state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_lightness_linear_status_cb_t**

Parameters of Light Lightness Linear Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_lightness**

Current value of light lightness linear state

uint16_t **target_lightness**

Target value of light lightness linear state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_lightness_last_status_cb_t**

Parameter of Light Lightness Last Status

Public Members

uint16_t **lightness**

The value of the Light Lightness Last state

struct **esp_ble_mesh_light_lightness_default_status_cb_t**

Parameter of Light Lightness Default Status

Public Members

uint16_t **lightness**

The value of the Light Lightness default State

struct **esp_ble_mesh_light_lightness_range_status_cb_t**

Parameters of Light Lightness Range Status

Public Members

uint8_t **status_code**

Status Code for the request message

uint16_t **range_min**

Value of range min field of light lightness range state

uint16_t **range_max**

Value of range max field of light lightness range state

struct **esp_ble_mesh_light_ctl_status_cb_t**

Parameters of Light CTL Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_ctl_lightness**

Current value of light ctl lightness state

uint16_t **present_ctl_temperature**

Current value of light ctl temperature state

uint16_t **target_ctl_lightness**

Target value of light ctl lightness state (optional)

uint16_t **target_ctl_temperature**

Target value of light ctl temperature state (C.1)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_ctl_temperature_status_cb_t**

Parameters of Light CTL Temperature Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_ctl_temperature**

Current value of light ctl temperature state

uint16_t **present_ctl_delta_uv**

Current value of light ctl delta UV state

uint16_t **target_ctl_temperature**

Target value of light ctl temperature state (optional)

uint16_t **target_ctl_delta_uv**

Target value of light ctl delta UV state (C.1)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_ctl_temperature_range_status_cb_t**

Parameters of Light CTL Temperature Range Status

Public Members

uint8_t **status_code**

Status code for the request message

uint16_t **range_min**

Value of temperature range min field of light ctl temperature range state

uint16_t **range_max**

Value of temperature range max field of light ctl temperature range state

struct **esp_ble_mesh_light_ctl_default_status_cb_t**

Parameters of Light CTL Default Status

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **temperature**

Value of light temperature default state

int16_t **delta_uv**

Value of light delta UV default state

struct **esp_ble_mesh_light_hsl_status_cb_t**

Parameters of Light HSL Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **hsl_lightness**

Current value of light hsl lightness state

uint16_t **hsl_hue**

Current value of light hsl hue state

uint16_t **hsl_saturation**

Current value of light hsl saturation state

uint8_t **remain_time**

Time to complete state transition (optional)

struct **esp_ble_mesh_light_hsl_target_status_cb_t**

Parameters of Light HSL Target Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **hsl_lightness_target**

Target value of light hsl lightness state

uint16_t **hsl_hue_target**

Target value of light hsl hue state

uint16_t **hsl_saturation_target**

Target value of light hsl saturation state

uint8_t **remain_time**

Time to complete state transition (optional)

struct **esp_ble_mesh_light_hsl_hue_status_cb_t**

Parameters of Light HSL Hue Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_hue**

Current value of light hsl hue state

uint16_t **target_hue**

Target value of light hsl hue state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_hsl_saturation_status_cb_t**

Parameters of Light HSL Saturation Status

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **present_saturation**

Current value of light hsl saturation state

uint16_t **target_saturation**

Target value of light hsl saturation state (optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_hsl_default_status_cb_t**

Parameters of Light HSL Default Status

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **hue**

Value of light hue default state

uint16_t **saturation**

Value of light saturation default state

struct **esp_ble_mesh_light_hsl_range_status_cb_t**

Parameters of Light HSL Range Status

Public Members

uint8_t **status_code**

Status code for the request message

uint16_t **hue_range_min**

Value of hue range min field of light hsl hue range state

uint16_t **hue_range_max**

Value of hue range max field of light hsl hue range state

uint16_t **saturation_range_min**

Value of saturation range min field of light hsl saturation range state

uint16_t **saturation_range_max**

Value of saturation range max field of light hsl saturation range state

struct **esp_ble_mesh_light_xyl_status_cb_t**

Parameters of Light xyL Status

Public Members

bool **op_en**

Indicate whether optional parameters included

uint16_t **xyl_lightness**

The present value of the Light xyL Lightness state

uint16_t **xyl_x**

The present value of the Light xyL x state

uint16_t **xyl_y**

The present value of the Light xyL y state

uint8_t **remain_time**

Time to complete state transition (optional)

struct **esp_ble_mesh_light_xyl_target_status_cb_t**

Parameters of Light xyL Target Status

Public Members

bool **op_en**

Indicate whether optional parameters included

uint16_t **target_xyl_lightness**

The target value of the Light xyL Lightness state

uint16_t **target_xyl_x**

The target value of the Light xyL x state

uint16_t **target_xyl_y**

The target value of the Light xyL y state

uint8_t **remain_time**

Time to complete state transition (optional)

struct **esp_ble_mesh_light_xyl_default_status_cb_t**

Parameters of Light xyL Default Status

Public Members

uint16_t **lightness**

The value of the Light Lightness Default state

uint16_t **xyl_x**

The value of the Light xyL x Default state

uint16_t **xyl_y**

The value of the Light xyL y Default state

struct **esp_ble_mesh_light_xyl_range_status_cb_t**

Parameters of Light xyL Range Status

Public Members

uint8_t **status_code**

Status Code for the requesting message

uint16_t **xyl_x_range_min**

The value of the xyL x Range Min field of the Light xyL x Range state

uint16_t **xyl_x_range_max**

The value of the xyL x Range Max field of the Light xyL x Range state

uint16_t **xyl_y_range_min**

The value of the xyL y Range Min field of the Light xyL y Range state

uint16_t **xyl_y_range_max**

The value of the xyL y Range Max field of the Light xyL y Range state

struct **esp_ble_mesh_light_lc_mode_status_cb_t**

Parameter of Light LC Mode Status

Public Members

uint8_t **mode**

The present value of the Light LC Mode state

struct **esp_ble_mesh_light_lc_om_status_cb_t**

Parameter of Light LC OM Status

Public Members

uint8_t **mode**

The present value of the Light LC Occupancy Mode state

struct **esp_ble_mesh_light_lc_light_onoff_status_cb_t**

Parameters of Light LC Light OnOff Status

Public Members

bool **op_en**

Indicate whether optional parameters included

uint8_t **present_light_onoff**

The present value of the Light LC Light OnOff state

uint8_t **target_light_onoff**

The target value of the Light LC Light OnOff state (Optional)

uint8_t **remain_time**

Time to complete state transition (C.1)

struct **esp_ble_mesh_light_lc_property_status_cb_t**

Parameters of Light LC Property Status

Public Members

uint16_t **property_id**

Property ID identifying a Light LC Property

struct net_buf_simple ***property_value**

Raw value for the Light LC Property

struct **esp_ble_mesh_light_client_cb_param_t**

Lighting Client Model callback parameters

Public Members

int **error_code**

Appropriate error code

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_light_client_status_cb_t **status_cb**

The light status message callback values

struct **esp_ble_mesh_light_lightness_state_t**

Parameters of Light Lightness state

Public Members

uint16_t **lightness_linear**

The present value of Light Lightness Linear state

uint16_t **target_lightness_linear**

The target value of Light Lightness Linear state

uint16_t **lightness_actual**

The present value of Light Lightness Actual state

uint16_t **target_lightness_actual**

The target value of Light Lightness Actual state

uint16_t **lightness_last**

The value of Light Lightness Last state

uint16_t **lightness_default**

The value of Light Lightness Default state

uint8_t **status_code**

The status code of setting Light Lightness Range state

uint16_t **lightness_range_min**

The minimum value of Light Lightness Range state

uint16_t **lightness_range_max**

The maximum value of Light Lightness Range state

struct **esp_ble_mesh_light_lightness_srv_t**

User data of Light Lightness Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting Lightness Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_lightness_state_t ***state**

Parameters of the Light Lightness state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **actual_transition**

Parameters of state transition

esp_ble_mesh_state_transition_t **linear_transition**

Parameters of state transition

int32_t **tt_delta_lightness_actual**

Delta change value of lightness actual state transition

int32_t **tt_delta_lightness_linear**

Delta change value of lightness linear state transition

struct **esp_ble_mesh_light_lightness_setup_srv_t**

User data of Light Lightness Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting Lightness Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_lightness_state_t ***state**

Parameters of the Light Lightness state

struct **esp_ble_mesh_light_ctl_state_t**

Parameters of Light CTL state

Public Members

uint16_t **lightness**

The present value of Light CTL Lightness state

uint16_t **target_lightness**

The target value of Light CTL Lightness state

uint16_t **temperature**

The present value of Light CTL Temperature state

uint16_t **target_temperature**

The target value of Light CTL Temperature state

int16_t **delta_uv**

The present value of Light CTL Delta UV state

int16_t **target_delta_uv**

The target value of Light CTL Delta UV state

uint8_t **status_code**

The statue code of setting Light CTL Temperature Range state

uint16_t **temperature_range_min**

The minimum value of Light CTL Temperature Range state

uint16_t **temperature_range_max**

The maximum value of Light CTL Temperature Range state

uint16_t **lightness_default**

The value of Light Lightness Default state

uint16_t **temperature_default**

The value of Light CTL Temperature Default state

int16_t **delta_uv_default**

The value of Light CTL Delta UV Default state

struct **esp_ble_mesh_light_ctl_srv_t**

User data of Light CTL Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting CTL Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_ctl_state_t ***state**

Parameters of the Light CTL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_lightness**

Delta change value of lightness state transition

int32_t **tt_delta_temperature**

Delta change value of temperature state transition

int32_t **tt_delta_delta_uv**

Delta change value of delta uv state transition

struct **esp_ble_mesh_light_ctl_setup_srv_t**

User data of Light CTL Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting CTL Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_ctl_state_t ***state**

Parameters of the Light CTL state

struct **esp_ble_mesh_light_ctl_temp_srv_t**

User data of Light CTL Temperature Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting CTL Temperature Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_ctl_state_t ***state**

Parameters of the Light CTL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_temperature**

Delta change value of temperature state transition

int32_t **tt_delta_delta_uv**

Delta change value of delta uv state transition

struct **esp_ble_mesh_light_hsl_state_t**

Parameters of Light HSL state

Public Members

uint16_t **lightness**

The present value of Light HSL Lightness state

uint16_t **target_lightness**

The target value of Light HSL Lightness state

uint16_t **hue**

The present value of Light HSL Hue state

uint16_t **target_hue**

The target value of Light HSL Hue state

uint16_t **saturation**

The present value of Light HSL Saturation state

uint16_t **target_saturation**

The target value of Light HSL Saturation state

uint16_t **lightness_default**

The value of Light Lightness Default state

uint16_t **hue_default**

The value of Light HSL Hue Default state

uint16_t **saturation_default**

The value of Light HSL Saturation Default state

uint8_t **status_code**

The status code of setting Light HSL Hue & Saturation Range state

uint16_t **hue_range_min**

The minimum value of Light HSL Hue Range state

uint16_t **hue_range_max**

The maximum value of Light HSL Hue Range state

uint16_t **saturation_range_min**

The minimum value of Light HSL Saturation state

uint16_t **saturation_range_max**

The maximum value of Light HSL Saturation state

struct **esp_ble_mesh_light_hsl_srv_t**

User data of Light HSL Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting HSL Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_hsl_state_t ***state**

Parameters of the Light HSL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_lightness**

Delta change value of lightness state transition

int32_t **tt_delta_hue**

Delta change value of hue state transition

int32_t **tt_delta_saturation**

Delta change value of saturation state transition

struct **esp_ble_mesh_light_hsl_setup_srv_t**

User data of Light HSL Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting HSL Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_hsl_state_t ***state**

Parameters of the Light HSL state

struct **esp_ble_mesh_light_hsl_hue_srv_t**

User data of Light HSL Hue Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting HSL Hue Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_hsl_state_t ***state**

Parameters of the Light HSL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_hue**

Delta change value of hue state transition

struct **esp_ble_mesh_light_hsl_sat_srv_t**

User data of Light HSL Saturation Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting HSL Saturation Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_hsl_state_t ***state**

Parameters of the Light HSL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_saturation**

Delta change value of saturation state transition

struct **esp_ble_mesh_light_xyl_state_t**

Parameters of Light xyL state

Public Members

uint16_t **lightness**

The present value of Light xyL Lightness state

uint16_t **target_lightness**

The target value of Light xyL Lightness state

uint16_t **x**

The present value of Light xyL x state

uint16_t **target_x**

The target value of Light xyL x state

uint16_t **y**

The present value of Light xyL y state

uint16_t **target_y**

The target value of Light xyL y state

uint16_t **lightness_default**

The value of Light Lightness Default state

uint16_t **x_default**

The value of Light xyL x Default state

uint16_t **y_default**

The value of Light xyL y Default state

uint8_t **status_code**

The status code of setting Light xyL x & y Range state

uint16_t **x_range_min**

The minimum value of Light xyL x Range state

uint16_t **x_range_max**

The maximum value of Light xyL x Range state

uint16_t **y_range_min**

The minimum value of Light xyL y Range state

uint16_t **y_range_max**

The maximum value of Light xyL y Range state

struct **esp_ble_mesh_light_xyl_srv_t**

User data of Light xyL Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting xyL Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_xyl_state_t ***state**

Parameters of the Light xyL state

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

int32_t **tt_delta_lightness**

Delta change value of lightness state transition

int32_t **tt_delta_x**

Delta change value of x state transition

int32_t **tt_delta_y**

Delta change value of y state transition

struct **esp_ble_mesh_light_xyl_setup_srv_t**

User data of Light xyL Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting xyL Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_xyl_state_t ***state**

Parameters of the Light xyL state

struct **esp_ble_mesh_light_lc_state_t**

Parameters of Light LC states

Public Members

uint32_t **mode**

0b0 The controller is turned off.

- The binding with the Light Lightness state is disabled. 0b1 The controller is turned on.
- The binding with the Light Lightness state is enabled. The value of Light LC Mode state

uint32_t **occupancy_mode**

The value of Light LC Occupancy Mode state

uint32_t **light_onoff**

The present value of Light LC Light OnOff state

uint32_t **target_light_onoff**

The target value of Light LC Light OnOff state

uint32_t **occupancy**

The value of Light LC Occupancy state

uint32_t **ambient_luxlevel**

The value of Light LC Ambient LuxLevel state

uint16_t **linear_output**

i. Light LC Linear Output = max((Lightness Out)^2/65535, Regulator Output)
ii. If the Light LC Mode state is set to 0b1, the binding is enabled and upon a change of the Light LC Linear Output state, the following operation shall be performed: Light Lightness Linear = Light LC Linear Output
iii. If the Light LC Mode state is set to 0b0, the binding is disabled (i.e., upon a change of the Light LC Linear Output state, no operation on the Light Lightness Linear state is performed). The value of Light LC Linear Output state

struct **esp_ble_mesh_light_lc_property_state_t**

Parameters of Light Property states. The Light LC Property states are read / write states that determine the configuration of a Light Lightness Controller. Each state is represented by a device property and is controlled by Light LC Property messages.

Public Members

uint32_t **time_occupancy_delay**

A timing state that determines the delay for changing the Light LC Occupancy state upon receiving a Sensor Status message from an occupancy sensor. The value of Light LC Time Occupancy Delay state

uint32_t **time_fade_on**

A timing state that determines the time the controlled lights fade to the level determined by the Light LC Lightness On state. The value of Light LC Time Fade On state

uint32_t **time_run_on**

A timing state that determines the time the controlled lights stay at the level determined by the Light LC Lightness On state. The value of Light LC Time Run On state

uint32_t **time_fade**

A timing state that determines the time the controlled lights fade from the level determined by the Light LC Lightness On state to the level determined by the Light Lightness Prolong state. The value of Light LC Time Fade state

uint32_t **time_prolong**

A timing state that determines the time the controlled lights stay at the level determined by the Light LC Lightness Prolong state. The value of Light LC Time Prolong state

uint32_t **time_fade_standby_auto**

A timing state that determines the time the controlled lights fade from the level determined by the Light LC Lightness Prolong state to the level determined by the Light LC Lightness Standby state when the transition is automatic. The value of Light LC Time Fade Standby Auto state

uint32_t **time_fade_standby_manual**

A timing state that determines the time the controlled lights fade from the level determined by the Light LC Lightness Prolong state to the level determined by the Light LC Lightness Standby state when the transition is triggered by a change in the Light LC Light OnOff state. The value of Light LC Time Fade Standby Manual state

uint16_t **lightness_on**

A lightness state that determines the perceptive light lightness at the Occupancy and Run internal controller states. The value of Light LC Lightness On state

uint16_t **lightness_prolong**

A lightness state that determines the light lightness at the Prolong internal controller state. The value of Light LC Lightness Prolong state

uint16_t **lightness_standby**

A lightness state that determines the light lightness at the Standby internal controller state. The value of Light LC Lightness Standby state

uint16_t **ambient_luxlevel_on**

A uint16 state representing the Ambient LuxLevel level that determines if the controller transitions from the Light Control Standby state. The value of Light LC Ambient LuxLevel On state

uint16_t **ambient_luxlevel_prolong**

A uint16 state representing the required Ambient LuxLevel level in the Prolong state. The value of Light LC Ambient LuxLevel Prolong state

uint16_t **ambient_luxlevel_standby**

A uint16 state representing the required Ambient LuxLevel level in the Standby state. The value of Light LC Ambient LuxLevel Standby state

float **regulator_kiu**

A float32 state representing the integral coefficient that determines the integral part of the equation defining the output of the Light LC PI Feedback Regulator, when Light LC Ambient LuxLevel is less than LuxLevel Out. Valid range: 0.0 ~ 1000.0. The default value is 250.0. The value of Light LC Regulator Kiu state

float **regulator_kid**

A float32 state representing the integral coefficient that determines the integral part of the equation defining the output of the Light LC PI Feedback Regulator, when Light LC Ambient LuxLevel is greater than or equal to the value of the LuxLevel Out state. Valid range: 0.0 ~ 1000.0. The default value is 25.0. The value of Light LC Regulator Kid state

float **regulator_kpu**

A float32 state representing the proportional coefficient that determines the proportional part of the equation defining the output of the Light LC PI Feedback Regulator, when Light LC Ambient LuxLevel is less than the value of the LuxLevel Out state. Valid range: 0.0 ~ 1000.0. The default value is 80.0. The value of Light LC Regulator Kpu state

float **regulator_kpd**

A float32 state representing the proportional coefficient that determines the proportional part of the equation defining the output of the Light LC PI Feedback Regulator, when Light LC Ambient LuxLevel is greater than or equal to the value of the LuxLevel Out state. Valid range: 0.0 ~ 1000.0. The default value is 80.0. The value of Light LC Regulator Kpd state

int8_t **regulator_accuracy**

A int8 state representing the percentage accuracy of the Light LC PI Feedback Regulator. Valid range: 0.0 ~ 100.0. The default value is 2.0. The value of Light LC Regulator Accuracy state

uint32_t **set_occupancy_to_1_delay**

If the message Raw field contains a Raw Value for the Time Since Motion Sensed device property, which represents a value less than or equal to the value of the Light LC Occupancy Delay state, it shall delay setting the Light LC Occupancy state to 0b1 by the difference between the value of the Light LC Occupancy Delay state and the received Time Since Motion value. The value of the difference between value of the Light LC Occupancy Delay state and the received Time Since Motion value

struct **esp_ble_mesh_light_lc_state_machine_t**

Parameters of Light LC state machine

Public Members

uint8_t **fade_on**

The value of transition time of Light LC Time Fade On

uint8_t **fade**

The value of transition time of Light LC Time Fade

uint8_t **fade_standby_auto**

The value of transition time of Light LC Time Fade Standby Auto

uint8_t **fade_standby_manual**

The value of transition time of Light LC Time Fade Standby Manual

struct *esp_ble_mesh_light_lc_state_machine_t*::[anonymous] **trans_time**

The Fade On, Fade, Fade Standby Auto, and Fade Standby Manual states are transition states that define the transition of the Lightness Out and LuxLevel Out states. This transition can be started as a result of the Light LC State Machine change or as a result of receiving the Light LC Light OnOff Set or Light LC Light Set Unacknowledged message. The value of transition time

esp_ble_mesh_lc_state_t **state**

The value of Light LC state machine state

struct k_delayed_work **timer**

Timer of Light LC state machine

struct **esp_ble_mesh_light_control_t**

Parameters of Light Lightness controller

Public Members

esp_ble_mesh_light_lc_state_t **state**

Parameters of Light LC state

esp_ble_mesh_light_lc_property_state_t **prop_state**

Parameters of Light LC Property state

esp_ble_mesh_light_lc_state_machine_t **state_machine**

Parameters of Light LC state machine

struct **esp_ble_mesh_light_lc_srv_t**

User data of Light LC Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting LC Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_control_t ***lc**

Parameters of the Light controller

esp_ble_mesh_last_msg_info_t **last**

Parameters of the last received set message

esp_ble_mesh_state_transition_t **transition**

Parameters of state transition

struct **esp_ble_mesh_light_lc_setup_srv_t**

User data of Light LC Setup Server Model

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the Lighting LC Setup Server Model. Initialized internally.

esp_ble_mesh_server_rsp_ctrl_t **rsp_ctrl**

Response control of the server model received messages

esp_ble_mesh_light_control_t ***lc**

Parameters of the Light controller

struct **esp_ble_mesh_state_change_light_lightness_set_t**

Parameter of Light Lightness Actual state change event

Public Members

uint16_t **lightness**

The value of Light Lightness Actual state

struct **esp_ble_mesh_state_change_light_lightness_linear_set_t**

Parameter of Light Lightness Linear state change event

Public Members

uint16_t **lightness**

The value of Light Lightness Linear state

struct **esp_ble_mesh_state_change_light_lightness_default_set_t**

Parameter of Light Lightness Default state change event

Public Members

uint16_t **lightness**

The value of Light Lightness Default state

struct **esp_ble_mesh_state_change_light_lightness_range_set_t**

Parameters of Light Lightness Range state change event

Public Members

uint16_t **range_min**

The minimum value of Light Lightness Range state

uint16_t **range_max**

The maximum value of Light Lightness Range state

struct **esp_ble_mesh_state_change_light_ctl_set_t**

Parameters of Light CTL state change event

Public Members

uint16_t **lightness**

The value of Light CTL Lightness state

uint16_t **temperature**

The value of Light CTL Temperature state

int16_t **delta_uv**

The value of Light CTL Delta UV state

struct **esp_ble_mesh_state_change_light_ctl_temperature_set_t**

Parameters of Light CTL Temperature state change event

Public Members

uint16_t **temperature**

The value of Light CTL Temperature state

int16_t **delta_uv**

The value of Light CTL Delta UV state

struct **esp_ble_mesh_state_change_light_ctl_temperature_range_set_t**

Parameters of Light CTL Temperature Range state change event

Public Members

uint16_t **range_min**

The minimum value of Light CTL Temperature Range state

uint16_t **range_max**

The maximum value of Light CTL Temperature Range state

struct **esp_ble_mesh_state_change_light_ctl_default_set_t**

Parameters of Light CTL Default state change event

Public Members

uint16_t **lightness**

The value of Light Lightness Default state

uint16_t **temperature**

The value of Light CTL Temperature Default state

int16_t **delta_uv**

The value of Light CTL Delta UV Default state

struct **esp_ble_mesh_state_change_light_hsl_set_t**

Parameters of Light HSL state change event

Public Members

uint16_t **lightness**

The value of Light HSL Lightness state

uint16_t **hue**

The value of Light HSL Hue state

uint16_t **saturation**

The value of Light HSL Saturation state

struct **esp_ble_mesh_state_change_light_hsl_hue_set_t**

Parameter of Light HSL Hue state change event

Public Members

uint16_t **hue**

The value of Light HSL Hue state

struct **esp_ble_mesh_state_change_light_hsl_saturation_set_t**

Parameter of Light HSL Saturation state change event

Public Members

uint16_t **saturation**

The value of Light HSL Saturation state

struct **esp_ble_mesh_state_change_light_hsl_default_set_t**

Parameters of Light HSL Default state change event

Public Members

uint16_t **lightness**

The value of Light HSL Lightness Default state

uint16_t **hue**

The value of Light HSL Hue Default state

uint16_t **saturation**

The value of Light HSL Saturation Default state

struct **esp_ble_mesh_state_change_light_hsl_range_set_t**

Parameters of Light HSL Range state change event

Public Members

uint16_t **hue_range_min**

The minimum hue value of Light HSL Range state

uint16_t **hue_range_max**

The maximum hue value of Light HSL Range state

uint16_t **saturation_range_min**

The minimum saturation value of Light HSL Range state

uint16_t **saturation_range_max**

The maximum saturation value of Light HSL Range state

struct **esp_ble_mesh_state_change_light_xyl_set_t**

Parameters of Light xyL state change event

Public Members

uint16_t **lightness**

The value of Light xyL Lightness state

uint16_t **x**

The value of Light xyL x state

uint16_t **y**

The value of Light xyL y state

struct **esp_ble_mesh_state_change_light_xyl_default_set_t**

Parameters of Light xyL Default state change event

Public Members

uint16_t **lightness**

The value of Light Lightness Default state

uint16_t **x**

The value of Light xyL x Default state

uint16_t **y**

The value of Light xyL y Default state

struct **esp_ble_mesh_state_change_light_xyl_range_set_t**

Parameters of Light xyL Range state change event

Public Members

uint16_t **x_range_min**

The minimum value of Light xyL x Range state

uint16_t **x_range_max**

The maximum value of Light xyL x Range state

uint16_t **y_range_min**

The minimum value of Light xyL y Range state

uint16_t **y_range_max**

The maximum value of Light xyL y Range state

struct **esp_ble_mesh_state_change_light_lc_mode_set_t**

Parameter of Light LC Mode state change event

Public Members

uint8_t **mode**

The value of Light LC Mode state

struct **esp_ble_mesh_state_change_light_lc_om_set_t**

Parameter of Light LC Occupancy Mode state change event

Public Members

uint8_t **mode**

The value of Light LC Occupancy Mode state

struct **esp_ble_mesh_state_change_light_lc_light_onoff_set_t**

Parameter of Light LC Light OnOff state change event

Public Members

uint8_t **onoff**

The value of Light LC Light OnOff state

struct **esp_ble_mesh_state_change_light_lc_property_set_t**

Parameters of Light LC Property state change event

Public Members

uint16_t **property_id**

The property id of Light LC Property state

struct net_buf_simple ***property_value**

The property value of Light LC Property state

struct **esp_ble_mesh_state_change_sensor_status_t**

Parameters of Sensor Status state change event

Public Members

uint16_t **property_id**

The value of Sensor Property ID

uint8_t **occupancy**

The value of Light LC Occupancy state

uint32_t **set_occupancy_to_1_delay**

The value of Light LC Set Occupancy to 1 Delay state

uint32_t **ambient_luxlevel**

The value of Light LC Ambient Luxlevel state

union *esp_ble_mesh_state_change_sensor_status_t*::[anonymous] **state**

Parameters of Sensor Status related state

struct **esp_ble_mesh_server_recv_light_lc_property_get_t**

Context of the received Light LC Property Get message

Public Members

uint16_t **property_id**

Property ID identifying a Light LC Property

struct **esp_ble_mesh_server_recv_light_lightness_set_t**

Context of the received Light Lightness Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light lightness actual state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_lightness_linear_set_t**

Context of the received Light Lightness Linear Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light lightness linear state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_lightness_default_set_t**

Context of the received Light Lightness Default Set message

Public Members

uint16_t **lightness**

The value of the Light Lightness Default state

struct **esp_ble_mesh_server_recv_light_lightness_range_set_t**

Context of the received Light Lightness Range Set message

Public Members

uint16_t **range_min**

Value of range min field of light lightness range state

uint16_t **range_max**

Value of range max field of light lightness range state

struct **esp_ble_mesh_server_recv_light_ctl_set_t**

Context of the received Light CTL Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light ctl lightness state

uint16_t **temperature**

Target value of light ctl temperature state

int16_t **delta_uv**

Target value of light ctl delta UV state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_ctl_temperature_set_t**

Context of the received Light CTL Temperature Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **temperature**

Target value of light ctl temperature state

int16_t **delta_uv**

Target value of light ctl delta UV state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_ctl_temperature_range_set_t**

Context of the received Light CTL Temperature Range Set message

Public Members

uint16_t **range_min**

Value of temperature range min field of light ctl temperature range state

uint16_t **range_max**

Value of temperature range max field of light ctl temperature range state

struct **esp_ble_mesh_server_recv_light_ctl_default_set_t**

Context of the received Light CTL Default Set message

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **temperature**

Value of light temperature default state

int16_t **delta_uv**

Value of light delta UV default state

struct **esp_ble_mesh_server_recv_light_hsl_set_t**

Context of the received Light HSL Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **lightness**

Target value of light hsl lightness state

uint16_t **hue**

Target value of light hsl hue state

uint16_t **saturation**

Target value of light hsl saturation state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_hsl_hue_set_t**

Context of the received Light HSL Hue Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **hue**

Target value of light hsl hue state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_hsl_saturation_set_t**

Context of the received Light HSL Saturation Set message

Public Members

bool **op_en**

Indicate if optional parameters are included

uint16_t **saturation**

Target value of light hsl hue state

uint8_t **tid**

Transaction ID

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_hsl_default_set_t**

Context of the received Light HSL Default Set message

Public Members

uint16_t **lightness**

Value of light lightness default state

uint16_t **hue**

Value of light hue default state

uint16_t **saturation**

Value of light saturation default state

struct **esp_ble_mesh_server_recv_light_hsl_range_set_t**

Context of the received Light HSL Range Set message

Public Members

uint16_t **hue_range_min**

Value of hue range min field of light hsl hue range state

uint16_t **hue_range_max**

Value of hue range max field of light hsl hue range state

uint16_t **saturation_range_min**

Value of saturation range min field of light hsl saturation range state

uint16_t **saturation_range_max**

Value of saturation range max field of light hsl saturation range state

struct **esp_ble_mesh_server_recv_light_xyl_set_t**

Context of the received Light xyL Set message

Public Members

bool **op_en**

Indicate whether optional parameters included

uint16_t **lightness**

The target value of the Light xyL Lightness state

uint16_t **x**

The target value of the Light xyL x state

uint16_t **y**

The target value of the Light xyL y state

uint8_t **tid**

Transaction Identifier

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_xyl_default_set_t**

Context of the received Light xyL Default Set message

Public Members

uint16_t **lightness**

The value of the Light Lightness Default state

uint16_t **x**

The value of the Light xyL x Default state

uint16_t **y**

The value of the Light xyL y Default state

struct **esp_ble_mesh_server_recv_light_xyl_range_set_t**

Context of the received Light xyl Range Set message

Public Members

uint16_t **x_range_min**

The value of the xyL x Range Min field of the Light xyL x Range state

uint16_t **x_range_max**

The value of the xyL x Range Max field of the Light xyL x Range state

uint16_t **y_range_min**

The value of the xyL y Range Min field of the Light xyL y Range state

uint16_t **y_range_max**

The value of the xyL y Range Max field of the Light xyL y Range state

struct **esp_ble_mesh_server_recv_light_lc_mode_set_t**

Context of the received Light LC Mode Set message

Public Members

uint8_t **mode**

The target value of the Light LC Mode state

struct **esp_ble_mesh_server_recv_light_lc_om_set_t**

Context of the received Light OM Set message

Public Members

uint8_t **mode**

The target value of the Light LC Occupancy Mode state

struct **esp_ble_mesh_server_recv_light_lc_light_onoff_set_t**

Context of the received Light LC Light OnOff Set message

Public Members

bool **op_en**

Indicate whether optional parameters included

uint8_t **light_onoff**

The target value of the Light LC Light OnOff state

uint8_t **tid**

Transaction Identifier

uint8_t **trans_time**

Time to complete state transition (optional)

uint8_t **delay**

Indicate message execution delay (C.1)

struct **esp_ble_mesh_server_recv_light_lc_property_set_t**

Context of the received Light LC Property Set message

Public Members

uint16_t **property_id**

Property ID identifying a Light LC Property

struct net_buf_simple ***property_value**

Raw value for the Light LC Property

struct **esp_ble_mesh_server_recv_sensor_status_t**

Context of the received Sensor Status message

Public Members

struct net_buf_simple ***data**

Value of sensor data state (optional)

struct **esp_ble_mesh_lighting_server_cb_param_t**

Lighting Server Model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Lighting Server Models

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received messages

esp_ble_mesh_lighting_server_cb_value_t **value**

Value of the received Lighting Messages

Macros

ESP_BLE_MESH_MODEL_LIGHT_LIGHTNESS_CLI(cli_pub, cli_data)

Define a new Light Lightness Client Model.

Note: This API needs to be called for each element on which the application needs to have a Light Lightness Client Model.

Parameters

- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Light Lightness Client Model instance.

ESP_BLE_MESH_MODEL_LIGHT_CTL_CLI(cli_pub, cli_data)

Define a new Light CTL Client Model.

Note: This API needs to be called for each element on which the application needs to have a Light CTL Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Light CTL Client Model instance.

ESP_BLE_MESH_MODEL_LIGHT_HSL_CLI(cli_pub, cli_data)

Define a new Light HSL Client Model.

Note: This API needs to be called for each element on which the application needs to have a Light HSL Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Light HSL Client Model instance.

ESP_BLE_MESH_MODEL_LIGHT_XYL_CLI(cli_pub, cli_data)

Define a new Light xyL Client Model.

Note: This API needs to be called for each element on which the application needs to have a Light xyL Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Light xyL Client Model instance.

ESP_BLE_MESH_MODEL_LIGHT_LC_CLI(cli_pub, cli_data)

Define a new Light LC Client Model.

Note: This API needs to be called for each element on which the application needs to have a Light LC Client Model.

Parameters
- **cli_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **cli_data** -- Pointer to the unique struct *esp_ble_mesh_client_t*.

Returns New Light LC Client Model instance.

ESP_BLE_MESH_MODEL_LIGHT_LIGHTNESS_SRV(srv_pub, srv_data)

Lighting Server Models related context.

Define a new Light Lightness Server Model.

Note: 1. The Light Lightness Server model extends the Generic Power OnOff Server model and the Generic Level Server model. When this model is present on an Element, the corresponding Light Lightness Setup Server model shall also be present.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_lightness_srv_t*.

Returns New Light Lightness Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_LIGHTNESS_SETUP_SRV(srv_pub, srv_data)

Define a new Light Lightness Setup Server Model.

Note: 1. The Light Lightness Setup Server model extends the Light Lightness Server model and the Generic Power OnOff Setup Server model.

a. This model shall support model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_lightness_setup_srv_t*.

Returns New Light Lightness Setup Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_CTL_SRV(srv_pub, srv_data)

Define a new Light CTL Server Model.

Note: 1. The Light CTL Server model extends the Light Lightness Server model. When this model is present on an Element, the corresponding Light CTL Temperature Server model and the corresponding Light CTL Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. The model requires two elements: the main element and the Temperature element. The Temperature element contains the corresponding Light CTL Temperature Server model and an instance of a Generic Level state bound to the Light CTL Temperature state on the Temperature element. The Light CTL Temperature state on the Temperature element is bound to the Light CTL state on the main element.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_ctl_srv_t*.

Returns New Light CTL Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_CTL_SETUP_SRV(srv_pub, srv_data)

Define a new Light CTL Setup Server Model.

Note: 1. The Light CTL Setup Server model extends the Light CTL Server and the Light Lightness Setup Server.

a. This model shall support model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_ctl_setup_srv_t*.

Returns New Light CTL Setup Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_CTL_TEMP_SRV(srv_pub, srv_data)

Define a new Light CTL Temperature Server Model.

Note: 1. The Light CTL Temperature Server model extends the Generic Level Server model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_ctl_temp_srv_t*.

Returns New Light CTL Temperature Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_HSL_SRV(srv_pub, srv_data)

Define a new Light HSL Server Model.

Note: 1. The Light HSL Server model extends the Light Lightness Server model. When this model is present on an Element, the corresponding Light HSL Hue Server model and the corresponding Light HSL Saturation Server model and the corresponding Light HSL Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. The model requires three elements: the main element and the Hue element and the Saturation element. The Hue element contains the corresponding Light HSL Hue Server model and an instance of a Generic Level state bound to the Light HSL Hue state on the Hue element. The Saturation element contains the corresponding Light HSL Saturation Server model and an instance of a Generic Level state bound to the Light HSL Saturation state on the Saturation element. The Light HSL Hue state on the Hue element is bound to the Light HSL state on the main element and the Light HSL Saturation state on the Saturation element is bound to the Light HSL state on the main element.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_hsl_srv_t*.

Returns New Light HSL Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_HSL_SETUP_SRV(srv_pub, srv_data)

Define a new Light HSL Setup Server Model.

Note: 1. The Light HSL Setup Server model extends the Light HSL Server and the Light Lightness Setup Server.

a. This model shall support model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_hsl_setup_srv_t*.

Returns New Light HSL Setup Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_HSL_HUE_SRV(srv_pub, srv_data)

Define a new Light HSL Hue Server Model.

Note: 1. The Light HSL Hue Server model extends the Generic Level Server model. This model is associated with the Light HSL Server model.

a. This model shall support model publication and model subscription.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_hsl_hue_srv_t*.

Returns New Light HSL Hue Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_HSL_SAT_SRV(srv_pub, srv_data)

Define a new Light HSL Saturation Server Model.

Note: 1. The Light HSL Saturation Server model extends the Generic Level Server model. This model is associated with the Light HSL Server model.

a. This model shall support model publication and model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_hsl_sat_srv_t*.

Returns New Light HSL Saturation Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_XYL_SRV(srv_pub, srv_data)

Define a new Light xyL Server Model.

Note: 1. The Light xyL Server model extends the Light Lightness Server model. When this model is present on an Element, the corresponding Light xyL Setup Server model shall also be present.

a. This model shall support model publication and model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_xyl_srv_t*.

Returns New Light xyL Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_XYL_SETUP_SRV(srv_pub, srv_data)

Define a new Light xyL Setup Server Model.

Note: 1. The Light xyL Setup Server model extends the Light xyL Server and the Light Lightness Setup Server.

a. This model shall support model subscription.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_xyl_setup_srv_t*.

Returns New Light xyL Setup Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_LC_SRV(srv_pub, srv_data)

Define a new Light LC Server Model.

Note: 1. The Light LC (Lightness Control) Server model extends the Light Lightness Server model and the Generic OnOff Server model. When this model is present on an Element, the corresponding Light LC Setup Server model shall also be present.

a. This model shall support model publication and model subscription.
b. This model may be used to represent an element that is a client to a Sensor Server model and controls the Light Lightness Actual state via defined state bindings.

Parameters

- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_lc_srv_t*.

Returns New Light LC Server Model instance.

ESP_BLE_MESH_MODEL_LIGHT_LC_SETUP_SRV(srv_pub, srv_data)

Define a new Light LC Setup Server Model.

Note: 1. The Light LC (Lightness Control) Setup model extends the Light LC Server model.

a. This model shall support model publication and model subscription.
b. This model may be used to configure setup parameters for the Light LC Server model.

Parameters
- **srv_pub** -- Pointer to the unique struct *esp_ble_mesh_model_pub_t*.
- **srv_data** -- Pointer to the unique struct *esp_ble_mesh_light_lc_setup_srv_t*.

Returns New Light LC Setup Server Model instance.

Type Definitions

typedef void (***esp_ble_mesh_light_client_cb_t**)(*esp_ble_mesh_light_client_cb_event_t* event, *esp_ble_mesh_light_client_cb_param_t* *param)

Bluetooth Mesh Light Client Model function.

Lighting Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_lighting_server_cb_t**)(*esp_ble_mesh_lighting_server_cb_event_t* event, *esp_ble_mesh_lighting_server_cb_param_t* *param)

Bluetooth Mesh Lighting Server Model function.

Lighting Server Model callback function type

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_light_client_cb_event_t**

This enum value is the event of Lighting Client Model

Values:

enumerator **ESP_BLE_MESH_LIGHT_CLIENT_GET_STATE_EVT**

enumerator **ESP_BLE_MESH_LIGHT_CLIENT_SET_STATE_EVT**

enumerator **ESP_BLE_MESH_LIGHT_CLIENT_PUBLISH_EVT**

enumerator **ESP_BLE_MESH_LIGHT_CLIENT_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_LIGHT_CLIENT_EVT_MAX**

enum **esp_ble_mesh_lc_state_t**

This enum value is the Light LC State Machine states

Values:

enumerator **ESP_BLE_MESH_LC_OFF**

enumerator **ESP_BLE_MESH_LC_STANDBY**

enumerator **ESP_BLE_MESH_LC_FADE_ON**

enumerator **ESP_BLE_MESH_LC_RUN**

enumerator **ESP_BLE_MESH_LC_FADE**

enumerator **ESP_BLE_MESH_LC_PROLONG**

enumerator **ESP_BLE_MESH_LC_FADE_STANDBY_AUTO**

enumerator **ESP_BLE_MESH_LC_FADE_STANDBY_MANUAL**

enum **esp_ble_mesh_lighting_server_cb_event_t**

This enum value is the event of Lighting Server Model

Values:

enumerator **ESP_BLE_MESH_LIGHTING_SERVER_STATE_CHANGE_EVT**

i. When get_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, no event will be callback to the application layer when Lighting Get messages are received.
ii. When set_auto_rsp is set to ESP_BLE_MESH_SERVER_AUTO_RSP, this event will be callback to the application layer when Lighting Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_LIGHTING_SERVER_RECV_GET_MSG_EVT**

When get_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Lighting Get messages are received.

enumerator **ESP_BLE_MESH_LIGHTING_SERVER_RECV_SET_MSG_EVT**

When set_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Lighting Set/Set Unack messages are received.

enumerator **ESP_BLE_MESH_LIGHTING_SERVER_RECV_STATUS_MSG_EVT**

When status_auto_rsp is set to ESP_BLE_MESH_SERVER_RSP_BY_APP, this event will be callback to the application layer when Sensor Status message is received.

enumerator **ESP_BLE_MESH_LIGHTING_SERVER_EVT_MAX**

ESP-BLE-MESH (v1.1) Core API Reference

Note: This section is a preview version, so the related structures, macros, and APIs may be changed.

This section contains ESP-BLE-MESH v1.1 Core related APIs, event types, event parameters, etc.

This API reference covers 10 components:

- *Remote Provisioning*
- *Directed Forwarding*
- *Subnet Bridge Configuration*
- *Mesh Private Beacon*
- *On-Demand Private Proxy*
- *SAR Configuration*
- *Solicitation PDU RPL Configuration*
- *Opcodes Aggregator*
- *Large Composition Data*
- *Composition and Metadata*

Remote Provisioning

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_rpr_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_rpr_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_rpr_client_callback**(*esp_ble_mesh_rpr_client_cb_t* callback)

Register BLE Mesh Remote Provisioning Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_rpr_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_rpr_client_msg_t* *msg)

Get the value of Remote Provisioning Server model state with the corresponding get message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Remote Provisioning Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_rpr_client_action**(*esp_ble_mesh_rpr_client_act_type_t* type, *esp_ble_mesh_rpr_client_act_param_t* *param)

Remote Provisioning Client model perform related actions, e.g. start remote provisioning.

Parameters
- **type** -- **[in]** Type of the action to be performed.
- **param** -- **[in]** Parameters of the action to be performed.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_rpr_server_callback**(*esp_ble_mesh_rpr_server_cb_t* callback)

Register BLE Mesh Remote Provisioning Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_rpr_client_msg_t**

#include <esp_ble_mesh_rpr_model_api.h> Remote Provisioning Client model message union.

Public Members

esp_ble_mesh_rpr_scan_start_t **scan_start**

For ESP_BLE_MESH_MODEL_OP_RPR_SCAN_START

esp_ble_mesh_rpr_ext_scan_start_t **ext_scan_start**

For ESP_BLE_MESH_MODEL_OP_RPR_EXT_SCAN_START

esp_ble_mesh_rpr_link_open_t **link_open**

For ESP_BLE_MESH_MODEL_OP_RPR_LINK_OPEN

esp_ble_mesh_rpr_link_close_t **link_close**

For ESP_BLE_MESH_MODEL_OP_RPR_LINK_CLOSE

union **esp_ble_mesh_rpr_client_act_param_t**

#include <esp_ble_mesh_rpr_model_api.h> Remote Provisioning Client model action union.

Public Members

esp_ble_mesh_rpr_client_start_rpr_t **start_rpr**

Start remote provisioning

union **esp_ble_mesh_rpr_client_recv_cb_t**

#include <esp_ble_mesh_rpr_model_api.h> Remote Provisioning Client model received message union.

Public Members

esp_ble_mesh_rpr_scan_caps_status_t **scan_caps_status**

For ESP_BLE_MESH_MODEL_OP_RPR_SCAN_CAPS_STATUS

esp_ble_mesh_rpr_scan_status_t **scan_status**

For ESP_BLE_MESH_MODEL_OP_RPR_SCAN_STATUS

esp_ble_mesh_rpr_scan_report_t **scan_report**

For ESP_BLE_MESH_MODEL_OP_RPR_SCAN_REPORT

esp_ble_mesh_rpr_ext_scan_report_t **ext_scan_report**

For ESP_BLE_MESH_MODEL_OP_RPR_EXT_SCAN_REPORT

esp_ble_mesh_rpr_link_status_t **link_status**

For ESP_BLE_MESH_MODEL_OP_RPR_LINK_STATUS

esp_ble_mesh_rpr_link_report_t **link_report**

For ESP_BLE_MESH_MODEL_OP_RPR_LINK_REPORT

union **esp_ble_mesh_rpr_client_cb_param_t**

#include <esp_ble_mesh_rpr_model_api.h> Remote Provisioning Client model callback parameters

Public Members

int **err_code**

Result of sending a message

Result of starting remote provisioning

esp_ble_mesh_client_common_param_t ***params**

Client common parameters

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **send**

Event parameters of sending messages Event parameters of sending messages

esp_ble_mesh_rpr_client_recv_cb_t **val**

Parameters of received status message

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **recv**

Event parameters of receiving messages Event parameters of receiving messages

esp_ble_mesh_rpr_client_act_evt_t **sub_evt**

Event type of the performed action

esp_ble_mesh_model_t ***model**

Pointer of Remote Provisioning Client

uint16_t **rpr_srv_addr**

Unicast address of Remote Provisioning Server

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous]::[anonymous] **start_rpr_comp**

ESP_BLE_MESH_START_RPR_COMP_SUB_EVT.

Event parameter of ESP_BLE_MESH_START_RPR_COMP_SUB_EVT

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **act**

Event parameters of performed actions Event parameters of performed actions

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **link_open**

ESP_BLE_MESH_RPR_CLIENT_LINK_OPEN_EVT.

Event parameters of ESP_BLE_MESH_RPR_CLIENT_LINK_OPEN_EVT

uint8_t **reason**

Reason of closing provisioning link

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **link_close**

ESP_BLE_MESH_RPR_CLIENT_LINK_CLOSE_EVT.

Event parameters of ESP_BLE_MESH_RPR_CLIENT_LINK_CLOSE_EVT

uint8_t **nppi**

NPPI Procedure

uint16_t **index**

Index of the provisioned node

uint8_t **uuid**[16]

Device UUID

uint16_t **unicast_addr**

Primary element address

uint8_t **element_num**

Element number

uint16_t **net_idx**

NetKey Index

struct *esp_ble_mesh_rpr_client_cb_param_t*::[anonymous] **prov**

ESP_BLE_MESH_RPR_CLIENT_PROV_COMP_EVT.

Event parameters of ESP_BLE_MESH_RPR_CLIENT_PROV_COMP_EVT

union **esp_ble_mesh_rpr_server_cb_param_t**

#include <esp_ble_mesh_rpr_model_api.h> Remote Provisioning Server model related context.

Remote Provisioning Server model callback value union

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

uint8_t **scan_items_limit**

Maximum number of scanned items to be reported

uint8_t **timeout**

Time limit for a scan (in seconds)

Time limit for extended scan (in seconds)

Time limit for opening a link (in seconds)

uint8_t **uuid**[16]

Device UUID (All ZERO if not present)

Device UUID (ZERO if not present)

uint16_t **net_idx**

NetKey Index used by Remote Provisioning Client

uint16_t **rpr_cli_addr**

Unicast address of Remote Provisioning Client

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **scan_start**

ESP_BLE_MESH_RPR_SERVER_SCAN_START_EVT.

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **scan_stop**

ESP_BLE_MESH_RPR_SERVER_SCAN_STOP_EVT.

uint8_t **ad_type_filter_count**

Number of AD Types in the ADTypeFilter field

uint8_t ***ad_type_filter**

List of AD Types to be reported

uint8_t **index**

Index of the extended scan instance

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **ext_scan_start**

ESP_BLE_MESH_RPR_SERVER_EXT_SCAN_START_EVT.

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **ext_scan_stop**

ESP_BLE_MESH_RPR_SERVER_EXT_SCAN_STOP_EVT.

uint8_t **status**

Status of Link Open procedure

uint8_t **nppi**

Node Provisioning Protocol Interface

Provisioning bearer link close reason code

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **link_open**

ESP_BLE_MESH_RPR_SERVER_LINK_OPEN_EVT.

bool **close_by_device**

Indicate if the link is closed by the Unprovisioned Device

uint8_t **reason**

Provisioning bearer link close reason code

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **link_close**

ESP_BLE_MESH_RPR_SERVER_LINK_CLOSE_EVT.

struct *esp_ble_mesh_rpr_server_cb_param_t*::[anonymous] **prov_comp**

ESP_BLE_MESH_RPR_SERVER_PROV_COMP_EVT. TODO: Duplicate with Link Close event?

Structures

struct **esp_ble_mesh_rpr_scan_start_t**

Remote Provisioning Server model context Parameters of Remote Provisioning Scan Start

Public Members

uint8_t **scan_items_limit**

Maximum number of scanned items to be reported

uint8_t **timeout**

Time limit for a scan (in seconds)

bool **uuid_en**

Indicate if Device UUID is present

uint8_t **uuid**[16]

Device UUID (Optional)

struct **esp_ble_mesh_rpr_ext_scan_start_t**

Parameters of Remote Provisioning Extended Scan Start

Public Members

uint8_t **ad_type_filter_count**

Number of AD Types in the ADTypeFilter field

uint8_t **ad_type_filter**[16]

List of AD Types to be reported. Minimum is 1, maximum is 16

bool **uuid_en**

Indicate if Device UUID is present

uint8_t **uuid**[16]

Device UUID (Optional)

uint8_t **timeout**

Time limit for a scan (in seconds) (C.1)

struct **esp_ble_mesh_rpr_link_open_t**

Parameters of Remote Provisioning Link Open

Public Members

bool **uuid_en**

Indicate if Device UUID is present

uint8_t **uuid**[16]

Device UUID (Optional)

bool **timeout_en**

Indicate if Link open timeout is present

uint8_t **timeout**

Link open timeout in seconds (C.1)

uint8_t **nppi**

Node Provisioning Protocol Interface (C.2)

struct **esp_ble_mesh_rpr_link_close_t**

Parameters of Remote Provisioning Link Close

Public Members

uint8_t **reason**

Provisioning bearer link close reason code

struct **esp_ble_mesh_rpr_client_start_rpr_t**

Parameters of starting remote provisioning

Public Members

esp_ble_mesh_model_t ***model**

Pointer of Remote Provisioning Client

uint16_t **rpr_srv_addr**

Unicast address of Remote Provisioning Server

struct **esp_ble_mesh_rpr_scan_caps_status_t**

Parameters of Remote Provisioning Scan Capabilities Status

Public Members

uint8_t **max_scan_items**

The maximum number of UUIDs that can be reported during scanning

uint8_t **active_scan**

Indication if active scan is supported

struct **esp_ble_mesh_rpr_scan_status_t**

Parameters of Remote Provisioning Scan Status

Public Members

uint8_t **status**

Status for the requesting message

uint8_t **rpr_scanning**

The Remote Provisioning Scan state value

uint8_t **scan_items_limit**

Maximum number of scanned items to be reported

uint8_t **timeout**

Time limit for a scan (in seconds)

struct **esp_ble_mesh_rpr_scan_report_t**

Parameters of Remote Provisioning Scan Report

Public Members

int8_t **rssi**

An indication of received signal strength measured in dBm

uint8_t **uuid**[16]

Device UUID

uint16_t **oob_info**

OOB information

uint32_t **uri_hash**

URI Hash (Optional)

struct **esp_ble_mesh_rpr_ext_scan_report_t**

Parameters of Remote Provisioning Extended Scan Report

Public Members

uint8_t **status**

Status for the requesting message

uint8_t **uuid**[16]

Device UUID

bool **oob_info_en**

Indicate if OOB Information is present

uint16_t **oob_info**

OOB Information (Optional)

struct net_buf_simple ***adv_structures**

Concatenated list of AD Structures (C.1)

struct **esp_ble_mesh_rpr_link_status_t**

Parameters of Remote Provisioning Link Status

Public Members

uint8_t **status**

Status for the requesting message

uint8_t **rpr_state**

Remote Provisioning Link state

struct **esp_ble_mesh_rpr_link_report_t**

Parameters of Remote Provisioning Link Report

Public Members

uint8_t **status**

Status of the provisioning bearer or the NPPI

uint8_t **rpr_state**

Remote Provisioning Link state

bool **reason_en**

Indicate if Link close Reason code is present

uint8_t **reason**

Link close Reason code (Optional)

Macros

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_CAPS_GET

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_CAPS_STATUS

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_GET

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_START

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_STOP

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_STATUS

ESP_BLE_MESH_MODEL_OP_RPR_SCAN_REPORT

ESP_BLE_MESH_MODEL_OP_RPR_EXT_SCAN_START

ESP_BLE_MESH_MODEL_OP_RPR_EXT_SCAN_REPORT

ESP_BLE_MESH_MODEL_OP_RPR_LINK_GET

ESP_BLE_MESH_MODEL_OP_RPR_LINK_OPEN

ESP_BLE_MESH_MODEL_OP_RPR_LINK_CLOSE

ESP_BLE_MESH_MODEL_OP_RPR_LINK_STATUS

ESP_BLE_MESH_MODEL_OP_RPR_LINK_REPORT

ESP_BLE_MESH_MODEL_OP_RPR_PDU_SEND

ESP_BLE_MESH_MODEL_OP_RPR_PDU_OUTBOUND_REPORT

ESP_BLE_MESH_MODEL_OP_RPR_PDU_REPORT

ESP_BLE_MESH_RPR_SRV_MAX_SCANNED_ITEMS_MIN

ESP_BLE_MESH_RPR_NOT_SUPPORT_ACTIVE_SCAN

ESP_BLE_MESH_RPR_SUPPORT_ACTIVE_SCAN

ESP_BLE_MESH_RPR_SCAN_IDLE

ESP_BLE_MESH_RPR_SCAN_MULTIPLE_DEVICE

ESP_BLE_MESH_RPR_SCAN_SINGLE_DEVICE

ESP_BLE_MESH_RPR_SCAN_NOT_IN_PROGRESS

ESP_BLE_MESH_RPR_PROHIBIT_SCAN_TIMEOUT

ESP_BLE_MESH_RPR_EXT_SCAN_TIMEOUT_MIN

ESP_BLE_MESH_RPR_EXT_SCAN_TIMEOUT_MAX

ESP_BLE_MESH_RPR_AD_TYPE_FILTER_CNT_MIN

ESP_BLE_MESH_RPR_AD_TYPE_FILTER_CNT_MAX

ESP_BLE_MESH_RPR_LINK_OPEN_TIMEOUT_MIN

ESP_BLE_MESH_RPR_LINK_OPEN_TIMEOUT_MAX

ESP_BLE_MESH_RPR_LINK_TIMEOUT_DEFAULT

ESP_BLE_MESH_RPR_REASON_SUCCESS

ESP_BLE_MESH_RPR_REASON_FAIL

ESP_BLE_MESH_RPR_LINK_IDLE

ESP_BLE_MESH_RPR_LINK_OPENING

ESP_BLE_MESH_RPR_LINK_ACTIVE

ESP_BLE_MESH_RPR_OUTBOUND_PACKET_TRANSFER

ESP_BLE_MESH_RPR_LINK_CLOSING

ESP_BLE_MESH_RPR_STATUS_SUCCESS

ESP_BLE_MESH_RPR_STATUS_SCANNING_CANNOT_START

ESP_BLE_MESH_RPR_STATUS_INVALID_STATE

ESP_BLE_MESH_RPR_STATUS_LIMITED_RESOURCES

ESP_BLE_MESH_RPR_STATUS_LINK_CANNOT_OPEN

ESP_BLE_MESH_RPR_STATUS_LINK_OPEN_FAILED

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_BY_DEVICE

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_BY_SERVER

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_BY_CLIENT

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_AS_CANNOT_RECEIVE_PDU

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_AS_CANNOT_SEND_PDU

ESP_BLE_MESH_RPR_STATUS_LINK_CLOSED_AS_CANNOT_DELIVER_PDU_REPORT

ESP_BLE_MESH_MODEL_RPR_SRV(srv_data)

Define a new Remote Provisioning Server model.

Note: If supported, the model shall be supported by a primary element and may be supported by any secondary element.

Parameters
- **srv_data** -- Pointer to a unique Remote Provisioning Server model user_data.

Returns New Remote Provisioning Server model instance.

ESP_BLE_MESH_MODEL_RPR_CLI(cli_data)

Define a new Remote Provisioning Client model.

Note: If supported, the model shall be supported by a primary element and may be supported by any secondary element.

Parameters
- **cli_data** -- Pointer to a unique Remote Provisioning Client model user_data.

Returns New Remote Provisioning Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_rpr_client_cb_t**)(*esp_ble_mesh_rpr_client_cb_event_t* event, *esp_ble_mesh_rpr_client_cb_param_t* *param)

Remote Provisioning client and server model functions.

Remote Provisioning Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_rpr_server_cb_t**)(*esp_ble_mesh_rpr_server_cb_event_t* event, *esp_ble_mesh_rpr_server_cb_param_t* *param)

Remote Provisioning Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_rpr_client_act_type_t**

This enum value is the action of Remote Provisioning Client model

Values:

enumerator **ESP_BLE_MESH_RPR_CLIENT_ACT_START_RPR**

enumerator **ESP_BLE_MESH_RPR_CLIENT_ACT_MAX**

enum **esp_ble_mesh_rpr_client_act_evt_t**

This enum value is the event type of the performed action

Values:

enumerator **ESP_BLE_MESH_START_RPR_COMP_SUB_EVT**

enum **esp_ble_mesh_rpr_client_cb_event_t**

This enum value is the event of Remote Provisioning Client model

Values:

enumerator **ESP_BLE_MESH_RPR_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_ACT_COMP_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_LINK_OPEN_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_LINK_CLOSE_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_PROV_COMP_EVT**

enumerator **ESP_BLE_MESH_RPR_CLIENT_EVT_MAX**

enum **esp_ble_mesh_rpr_server_cb_event_t**

This enum value is the event of Remote Provisioning Server model

Values:

enumerator **ESP_BLE_MESH_RPR_SERVER_SCAN_START_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_SCAN_STOP_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_EXT_SCAN_START_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_EXT_SCAN_STOP_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_LINK_OPEN_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_LINK_CLOSE_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_PROV_COMP_EVT**

enumerator **ESP_BLE_MESH_RPR_SERVER_EVT_MAX**

Directed Forwarding

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_df_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_df_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_df_client_callback**(*esp_ble_mesh_df_client_cb_t* callback)

Register BLE Mesh Directed Forwarding Configuration Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_df_server_callback**(*esp_ble_mesh_df_server_cb_t* callback)

Register BLE Mesh Directed Forwarding Configuration Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_df_client_get_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_df_client_get_t* *get)

Get the value of Directed Forwarding Configuration Server model state with the corresponding get message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **get** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_df_client_set_state**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_df_client_set_t* *set)

Set the value of Directed Forwarding Configuration Server model state with the corresponding set message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **set** -- **[in]** Pointer to a union, each kind of opcode corresponds to one structure inside.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_df_client_get_t**

#include <esp_ble_mesh_df_model_api.h> Directed Forwarding Configuration Client model get message union.

Public Members

esp_ble_mesh_directed_control_get_t **directed_control_get**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_GET

esp_ble_mesh_path_metric_get_t **path_metric_get**

For ESP_BLE_MESH_MODEL_OP_PATH_METRIC_GET

esp_ble_mesh_discovery_table_caps_get_t **disc_table_caps_get**

For ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_GET

esp_ble_mesh_forwarding_table_entries_cnt_get_t **forwarding_table_entries_cnt_get**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_CNT_GET

esp_ble_mesh_forwarding_table_entries_get_t **forwarding_table_entries_get**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_GET

esp_ble_mesh_forwarding_table_deps_get_t **forwarding_table_deps_get**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_GET

esp_ble_mesh_wanted_lanes_get_t **wanted_lanes_get**

For ESP_BLE_MESH_MODEL_OP_WANTED_LANES_GET

esp_ble_mesh_two_way_path_get_t **two_way_path_get**

For ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_GET

esp_ble_mesh_path_echo_interval_get_t **path_echo_interval_get**

For ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_GET

esp_ble_mesh_directed_publish_policy_get_t **directed_pub_policy_get**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_GET

union **esp_ble_mesh_df_client_set_t**

#include <esp_ble_mesh_df_model_api.h> Directed Forwarding Configuration Client model set message union.

Public Members

esp_ble_mesh_directed_control_set_t **directed_control_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_SET

esp_ble_mesh_path_metric_set_t **path_metric_set**

For ESP_BLE_MESH_MODEL_OP_PATH_METRIC_SET

esp_ble_mesh_discovery_table_caps_set_t **disc_table_caps_set**

For ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_SET

esp_ble_mesh_forwarding_table_add_t **forwarding_table_add**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ADD

esp_ble_mesh_forwarding_table_delete_t **forwarding_table_del**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEL

esp_ble_mesh_forwarding_table_deps_add_t **forwarding_table_deps_add**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_ADD

esp_ble_mesh_forwarding_table_deps_delete_t **forwarding_table_deps_del**

For ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_DEL

esp_ble_mesh_wanted_lanes_set_t **wanted_lanes_set**

For ESP_BLE_MESH_MODEL_OP_WANTED_LANES_SET

esp_ble_mesh_two_way_path_set_t **two_way_path_set**

For ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_SET

esp_ble_mesh_path_echo_interval_set_t **path_echo_interval_set**

For ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_SET

esp_ble_mesh_directed_net_transmit_set_t **directed_net_transmit_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_NET_TRANSMIT_SET

esp_ble_mesh_directed_relay_retransmit_set_t **directed_relay_retransmit_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_RELAY_RETRANSMIT_SET

esp_ble_mesh_rssi_threshold_set_t **rssi_threshold_set**

For ESP_BLE_MESH_MODEL_OP_RSSI_THRESHOLD_SET

esp_ble_mesh_directed_publish_policy_set_t **directed_pub_policy_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_SET

esp_ble_mesh_path_discovery_timing_ctl_set_t **path_disc_timing_ctl_set**

For ESP_BLE_MESH_MODEL_OP_PATH_DISCOVERY_TIMING_CTL_SET

esp_ble_mesh_directed_ctl_net_transmit_set_t **directed_ctl_net_transmit_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_NET_TRANSMIT_SET

esp_ble_mesh_directed_ctl_relay_retransmit_set_t **directed_ctl_relay_retransmit_set**

For ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_RELAY_RETRANSMIT_SET

union **esp_ble_mesh_df_client_recv_cb_t**

#include <esp_ble_mesh_df_model_api.h> Directed Forwarding Configuration Client model received message union.

Public Members

esp_ble_mesh_directed_control_status_t **directed_control_status**

ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_STATUS

esp_ble_mesh_path_metric_status_t **path_metric_status**

ESP_BLE_MESH_MODEL_OP_PATH_METRIC_STATUS

esp_ble_mesh_discovery_table_caps_status_t **disc_table_caps_status**
ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_STATUS

esp_ble_mesh_forwarding_table_status_t **forwarding_table_status**
ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_STATUS

esp_ble_mesh_forwarding_table_deps_status_t **forwarding_table_deps_status**
ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_STATUS

esp_ble_mesh_forwarding_table_entries_cnt_status_t **forwarding_table_entries_cnt_status**
ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_CNT_STATUS

esp_ble_mesh_forwarding_table_entries_status_t **forwarding_table_entries_status**
ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_STATUS

esp_ble_mesh_forwarding_table_deps_get_status_t **forwarding_table_deps_get_status**
ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_GET_STATUS

esp_ble_mesh_wanted_lanes_status_t **wanted_lanes_status**
ESP_BLE_MESH_MODEL_OP_WANTED_LANES_STATUS

esp_ble_mesh_two_way_path_status_t **two_way_path_status**
ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_STATUS

esp_ble_mesh_path_echo_interval_status_t **path_echo_interval_status**
ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_STATUS

esp_ble_mesh_directed_net_transmit_status_t **directed_net_transmit_status**
ESP_BLE_MESH_MODEL_OP_DIRECTED_NET_TRANSMIT_STATUS

esp_ble_mesh_directed_relay_retransmit_status_t **directed_relay_retransmit_status**
ESP_BLE_MESH_MODEL_OP_DIRECTED_RELAY_RETRANSMIT_STATUS

esp_ble_mesh_rssi_threshold_status_t **rssi_threshold_status**
ESP_BLE_MESH_MODEL_OP_RSSI_THRESHOLD_STATUS

esp_ble_mesh_directed_paths_status_t **directed_paths_status**
ESP_BLE_MESH_MODEL_OP_DIRECTED_PATHS_STATUS

esp_ble_mesh_directed_pub_policy_status_t **directed_pub_policy_status**
ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_STATUS

esp_ble_mesh_path_disc_timing_ctl_status_cb_t **path_disc_timing_ctl_status**
ESP_BLE_MESH_MODEL_OP_PATH_DISCOVERY_TIMING_CTL_STATUS

esp_ble_mesh_directed_ctl_net_transmit_status_t **directed_ctl_net_transmit_status**
ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_NET_TRANSMIT_STATUS

esp_ble_mesh_directed_ctl_relay_retransmit_status_t **directed_ctl_relay_retransmit_status**

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_RELAY_RETRANSMIT_STATUS

union **esp_ble_mesh_df_server_state_change_t**

#include <esp_ble_mesh_df_model_api.h> Directed Forwarding Configuration Server model related context.

Directed Forwarding Configuration Server model state change value union

Public Members

uint8_t **dummy**

Event not used currently

union **esp_ble_mesh_df_server_cb_value_t**

#include <esp_ble_mesh_df_model_api.h> Directed Forwarding Configuration Server model callback value union.

Public Members

esp_ble_mesh_df_server_state_change_t **state_change**

For ESP_BLE_MESH_DF_SERVER_STATE_CHANGE_EVT

esp_ble_mesh_df_server_table_change_t **table_change**

For ESP_BLE_MESH_DF_SERVER_TABLE_CHANGE_EVT

Structures

struct **esp_ble_mesh_df_srv_t**

Directed Forwarding Configuration Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Directed Forwarding Configuration Server model

uint8_t **directed_net_transmit**

Directed Network Transmit state

uint8_t **directed_relay_retransmit**

Directed Relay Retransmit state

int8_t **default_rssi_threshold**

Default RSSI Threshold state

uint8_t **rssi_margin**

RSSI Margin state

uint16_t **directed_node_paths**
Directed Node Paths state

uint16_t **directed_relay_paths**
Directed Relay Paths state

uint16_t **directed_proxy_paths**
Directed Proxy Paths state

uint16_t **directed_friend_paths**
Directed Friend Paths state

uint16_t **path_monitor_interval**
Path Monitoring Interval state

uint16_t **path_disc_retry_interval**
Path Discovery Retry Interval state

uint8_t **path_disc_interval**
Path Discovery Interval state

uint8_t **lane_disc_guard_interval**
Lane Discovery Guard Interval state

uint8_t **directed_ctl_net_transmit**
Directed Control Network Transmit state

uint8_t **directed_ctl_relay_retransmit**
Directed Control Relay Retransmit state

struct **esp_ble_mesh_directed_control_get_t**
Parameters of Directed Control Get.

Public Members

uint16_t **net_idx**
NetKey Index

struct **esp_ble_mesh_directed_control_set_t**
Parameters of Directed Control Set.

Public Members

uint16_t **net_idx**
NetKey Index

uint8_t **directed_forwarding**
New Directed Forwarding state

uint8_t **directed_relay**

New Directed Relay state

uint8_t **directed_proxy**

New Directed Proxy state

uint8_t **directed_proxy_use_default**

New Directed Proxy Use Directed Default state or value to ignore

uint8_t **directed_friend**

New Directed Friend state or value to ignore

struct **esp_ble_mesh_path_metric_get_t**

Parameters of Path Metric Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_path_metric_set_t**

Parameters of Path Metric Set.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **path_metric_type**

New Path Metric Type state

uint8_t **path_lifetime**

New Path Lifetime state

struct **esp_ble_mesh_discovery_table_caps_get_t**

Parameters of Discovery Table Capabilities Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_discovery_table_caps_set_t**

Parameters of Discovery Table Capabilities Set.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **max_concurr_init**

New Max Concurrent Init state

struct **esp_ble_mesh_forwarding_table_add_t**

Parameters of Forwarding Table Add.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **unicast_dst**

Indicates whether or not the destination of the path is a unicast address

uint16_t **bw_path_validated**

Indicates whether or not the backward path has been validated

esp_ble_mesh_uar_t **path_origin**

Unicast address range of the Path Origin

esp_ble_mesh_uar_t **path_target**

Unicast address range of the Path Target

uint16_t **multicast_dst**

Multicast destination address

union *esp_ble_mesh_forwarding_table_add_t*::[anonymous] **[anonymous]**

Path target address

uint16_t **bearer_twd_path_origin**

Index of the bearer toward the Path Origin

uint16_t **bearer_twd_path_target**

Index of the bearer toward the Path Target

struct **esp_ble_mesh_forwarding_table_delete_t**

Parameters of Forwarding Table Delete.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

struct **esp_ble_mesh_forwarding_table_deps_add_t**

Parameters of Forwarding Table Dependents Add.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

uint8_t **dep_origin_uar_list_size**

Number of entries in the Dependent_Origin_Unicast_Addr_Range_List field

uint8_t **dep_target_uar_list_size**

Number of entries in the Dependent_Target_Unicast_Addr_Range_List field

esp_ble_mesh_uar_t ***dep_origin_uar_list**

List of the unicast address ranges of the dependent nodes of the Path Origin

esp_ble_mesh_uar_t ***dep_target_uar_list**

List of the unicast address ranges of the dependent nodes of the Path Target

struct **esp_ble_mesh_forwarding_table_deps_delete_t**

Parameters of Forwarding Table Dependents Delete.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

uint8_t **dep_origin_list_size**

Number of entries in the Dependent_Origin_List field

uint8_t **dep_target_list_size**

Number of entries in the Dependent_Target_List field

uint16_t ***dep_origin_list**

List of the primary element addresses of the dependent nodes of the Path Origin

uint16_t ***dep_target_list**

List of the primary element addresses of the dependent nodes of the Path Target

struct **esp_ble_mesh_forwarding_table_entries_cnt_get_t**

Parameters of Forwarding Table Entries Count Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_forwarding_table_entries_get_t**

Parameters of Forwarding Table Entries Get.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **filter_mask**

Filter to be applied to the Forwarding Table entries

uint16_t **start_index**

Start offset to read in units of Forwarding Table entries

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

bool **include_id**

Indicate whether or not the Forwarding Table Update Identifier is present

uint16_t **update_id**

Last saved Forwarding Table Update Identifier (Optional)

struct **esp_ble_mesh_forwarding_table_deps_get_t**

Parameters of Forwarding Table Dependents Get.

Public Members

uint16_t **net_idx**

NetKey Index

uint16_t **dep_list_mask**

Filter applied to the lists of unicast address ranges for dependent nodes

uint16_t **fixed_path_flag**

Indicate whether or not to return the unicast address ranges of dependent nodes in a fixed path entry

uint16_t **start_index**

Start offset in units of unicast address ranges

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

bool **include_id**

Indicate whether or not the Forwarding Table Update Identifier is present

uint16_t **update_id**

Last saved Forwarding Table Update Identifier (Optional)

struct **esp_ble_mesh_wanted_lanes_get_t**

Parameters of Wanted Lanes Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_wanted_lanes_set_t**

Parameters of Wanted Lanes Set.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **wanted_lanes**

New Wanted Lanes state

struct **esp_ble_mesh_two_way_path_get_t**

Parameters of Two Way Path Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_two_way_path_set_t**

Parameters of Two Way Path Set.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **two_way_path**

Two way path flag

struct **esp_ble_mesh_path_echo_interval_get_t**

Parameters of Path Echo Interval Get.

Public Members

uint16_t **net_idx**

NetKey Index

struct **esp_ble_mesh_path_echo_interval_set_t**

Parameters of Path Echo Interval Set.

Public Members

uint16_t **net_idx**

NetKey Index

uint8_t **unicast_echo_interval**

New Unicast Echo Interval state or indication of no state change

uint8_t **multicast_echo_interval**

New Multicast Echo Interval state or indication of no state change

struct **esp_ble_mesh_directed_net_transmit_set_t**

Parameters of Directed Network Transmit Set.

Public Members

uint8_t **net_transmit**

New Directed Network Transmit state

struct **esp_ble_mesh_directed_relay_retransmit_set_t**

Parameters of Directed Relay Retransmit Set.

Public Members

uint8_t **relay_retransmit**

New Directed Relay Retransmit state

struct **esp_ble_mesh_rssi_threshold_set_t**

Parameters of RSSI Threshold Set.

Public Members

uint8_t **rssi_margin**

New RSSI Margin state

struct **esp_ble_mesh_directed_publish_policy_get_t**

Parameters of Directed Publish Policy Get.

Public Members

uint16_t **element_addr**

Address of the element

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_directed_publish_policy_set_t**

Parameters of Directed Publish Policy Set.

Public Members

uint8_t **direct_pub_policy**

New Directed Publish Policy state

uint16_t **element_addr**

Address of the element

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_path_discovery_timing_ctl_set_t**

Parameters of Path Discovery Timing Control Set.

Public Members

uint16_t **path_monitor_interval**

New Path Monitoring Interval state

uint16_t **path_disc_retry_interval**

New Path Discovery Retry Interval state

uint8_t **path_disc_interval**

New Path Discovery Interval state

uint8_t **lane_disc_guard_interval**

New Lane Discovery Guard Interval state

struct **esp_ble_mesh_directed_ctl_net_transmit_set_t**

Parameters of Directed Control Network Transmit Set.

Public Members

uint8_t **net_transmit**

New Directed Control Network Transmit Count state

struct **esp_ble_mesh_directed_ctl_relay_retransmit_set_t**

Parameters of Directed Control Relay Retransmit Set.

Public Members

uint8_t **relay_retransmit**

New Directed Control Relay Retransmit Count state

struct **esp_ble_mesh_directed_control_status_t**

Parameters of Directed Control Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **directed_forwarding**

Current Directed Forwarding state

uint8_t **directed_relay**

Current Directed Relay state

uint8_t **directed_proxy**

Current Directed Proxy state

uint8_t **directed_proxy_use_default**

Current Directed Proxy Use Directed Default state or 0xFF

uint8_t **directed_friend**

Current Directed Friend state

struct **esp_ble_mesh_path_metric_status_t**

Parameters of Path Metric Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **path_metric_type**

Current Path Metric Type state

uint8_t **path_lifetime**

Current Path Lifetime state

struct **esp_ble_mesh_discovery_table_caps_status_t**

Parameters of Discovery Table Capabilities Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **max_concurr_init**

Current Max Concurrent Init state

uint8_t **max_disc_entries**

Max Discovery Table Entries Count state

struct **esp_ble_mesh_forwarding_table_status_t**

Parameters of Forwarding Table Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

struct **esp_ble_mesh_forwarding_table_deps_status_t**

Parameters of Forwarding Table Dependent Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

struct **esp_ble_mesh_forwarding_table_entries_cnt_status_t**

Parameters of Forwarding Table Entries Count Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint16_t **update_id**

Current Forwarding Table Update Identifier state

uint16_t **fixed_entry_cnt**

Number of fixed path entries in the Forwarding Table

uint16_t **non_fixed_entry_cnt**

Number of non-fixed path entries in the Forwarding Table

struct **esp_ble_mesh_forwarding_table_entry_t**

Parameters of Forwarding Table Entry.

Public Members

uint16_t **fixed_path_flag**

Indicates whether the table entry is a fixed path entry or a non-fixed path entry

uint16_t **unicast_dst_flag**

Indicates whether or not the destination of the path is a unicast address

uint16_t **bw_path_validated_flag**

Indicates whether or not the backward path has been validated

uint16_t **bearer_twd_path_origin_ind**

Indicates the presence or absence of the Bearer_Toward_Path_Origin field

uint16_t **bearer_twd_path_target_ind**

Indicates the presence or absence of the Bearer_Toward_Path_Target field

uint16_t **dep_origin_list_size_ind**

Indicates the size of the Dependent_Origin_List field

uint16_t **dep_target_list_size_ind**

Indicates the size of the Dependent_Target_List field

uint8_t **lane_counter**

Number of lanes in the path

uint16_t **path_remaining_time**

Path lifetime remaining

uint8_t **path_origin_forward_number**

Forwarding number of the Path Origin

esp_ble_mesh_uar_t **path_origin**

Path Origin unicast address range

uint16_t **dep_origin_list_size**

Current number of entries in the list of dependent nodes of the Path Origin

uint16_t **bearer_twd_path_origin**

Index of the bearer toward the Path Origin

esp_ble_mesh_uar_t **path_target**

Path Target unicast address range

uint16_t **multicast_dst**

Multicast destination address

uint16_t **dep_target_list_size**

Current number of entries in the list of dependent nodes of the Path Target

uint16_t **bearer_twd_path_target**

Index of the bearer toward the Path Target

struct **esp_ble_mesh_forwarding_table_entries_status_t**

Parameters of Forwarding Table Entries Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint16_t **filter_mask**

Filter applied to the Forwarding Table entries

uint16_t **start_index**

Start offset in units of Forwarding Table entries

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

bool **include_id**

Indicate whether or not the Forwarding Table Update Identifier is present

uint16_t **update_id**

Current Forwarding Table Update Identifier state

uint8_t **entry_list_size**

Current number of entries in the list of Forwarding Table entries

esp_ble_mesh_forwarding_table_entry_t ***entry_list**

List of Forwarding Table entries

struct **esp_ble_mesh_forwarding_table_deps_get_status_t**

Parameters of Forwarding Table Dependents Get Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint16_t **dep_list_mask**

Filter applied to the lists of unicast address ranges for dependent nodes

uint16_t **fixed_path_flag**

Flag indicating whether or not to return the unicast address ranges of dependent nodes in a fixed path entry

uint16_t **start_index**

Start offset in units of unicast address ranges

uint16_t **path_origin**

Primary element address of the Path Origin

uint16_t **dst**

Destination address

bool **include_id**

Indicate whether or not the Forwarding Table Update Identifier is present

uint16_t **update_id**

Current Forwarding Table Update Identifier state

uint8_t **dep_origin_uar_list_size**

Number of unicast address ranges in the Dependent_Origin_Unicast_Addr_Range_List field

uint8_t **dep_target_uar_list_size**

Number of unicast address ranges in the Dependent_Target_Unicast_Addr_Range_List field

esp_ble_mesh_uar_t ***dep_origin_uar_list**

List of unicast address ranges of dependent nodes of the Path Origin

esp_ble_mesh_uar_t ***dep_target_uar_list**

List of unicast address ranges of dependent nodes of the Path Target

struct **esp_ble_mesh_wanted_lanes_status_t**

Parameters of Wanted Lanes Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **wanted_lanes**

Current Wanted Lanes state

struct **esp_ble_mesh_two_way_path_status_t**

Parameters of Two Way Path Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **two_way_path**

Current Two Way Path state

struct **esp_ble_mesh_path_echo_interval_status_t**

Parameters of Path Echo Interval Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint16_t **net_idx**

NetKey Index

uint8_t **unicast_echo_interval**

Current Unicast Echo Interval state

uint8_t **multicast_echo_interval**

Current Multicast Echo Interval state

struct **esp_ble_mesh_directed_net_transmit_status_t**

Parameters of Directed Network Transmit Status.

Public Members

uint8_t **net_transmit**

Current Directed Network Transmit state

struct **esp_ble_mesh_directed_relay_retransmit_status_t**

Parameters of Directed Relay Retransmit Status.

Public Members

uint8_t **relay_retransmit**

Current Directed Relay Retransmit state

struct **esp_ble_mesh_rssi_threshold_status_t**

Parameters of RSSI Threshold Status.

Public Members

uint8_t **default_rssi_threshold**

Default RSSI Threshold state

uint8_t **rssi_margin**

Current RSSI Margin state

struct **esp_ble_mesh_directed_paths_status_t**

Parameters of Directed Paths Status.

Public Members

uint16_t **directed_node_paths**

Directed Node Paths state

uint16_t **directed_relay_paths**

Directed Relay Paths state

uint16_t **directed_proxy_paths**

Directed Proxy Paths state

uint16_t **directed_friend_paths**

Directed Friend Paths state

struct **esp_ble_mesh_directed_pub_policy_status_t**

Parameters of Directed Publish Policy Status.

Public Members

uint8_t **status**

Status code for the requesting message

uint8_t **directed_pub_policy**

Current Directed Publish Policy state

uint16_t **element_addr**

Address of the element

uint16_t **company_id**

Company ID

uint16_t **model_id**

Model ID

struct **esp_ble_mesh_path_disc_timing_ctl_status_cb_t**

Parameters of Path Discovery Timing Control Status.

Public Members

uint16_t **path_monitor_interval**

Current Path Monitoring Interval state

uint16_t **path_disc_retry_interval**

Current Path Discovery Retry Interval state

uint8_t **path_disc_interval**

Current Path Discovery Interval state

uint8_t **lane_disc_guard_interval**

Current Lane Discovery Guard Interval state

struct **esp_ble_mesh_directed_ctl_net_transmit_status_t**

Parameters of Directed Control Network Transmit Status.

Public Members

uint8_t **net_transmit**

Current Directed Control Network Transmit state

struct **esp_ble_mesh_directed_ctl_relay_retransmit_status_t**

Parameters of Directed Control Relay Retransmit Status.

Public Members

uint8_t **relay_retransmit**

Current Directed Control Relay Retransmit state

struct **esp_ble_mesh_df_client_send_cb_t**

Result of sending Directed Forwarding Configuration Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_df_client_cb_param_t**

Directed Forwarding Configuration Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events.

esp_ble_mesh_df_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_df_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_df_client_cb_param_t*::[anonymous] **[anonymous]**

Union of DF Client callback

struct **esp_ble_mesh_df_server_table_change_t**

Parameters of directed forwarding table entry change

Public Members

esp_ble_mesh_df_table_action_t **action**

Action of directed forwarding table

esp_ble_mesh_uar_t **path_origin**

Primary element address of the Path Origin

esp_ble_mesh_uar_t **path_target**

Primary element address of the Path Target

esp_ble_mesh_uar_t ***dep_origin_data**

List of the primary element addresses of the dependent nodes of the Path Origin

uint32_t **dep_origin_num**

Number of entries in the Dependent_Origin_List field of the message

esp_ble_mesh_uar_t ***dep_target_data**

List of the primary element addresses of the dependent nodes of the Path Target

uint32_t **dep_target_num**

Number of entries in the Dependent_Target_List field of the message

uint8_t **fixed_path**

Indicates whether the table entry is a fixed path entry or a non-fixed path entry

uint8_t **bw_path_validate**

Indicates whether or not the backward path has been validated

uint8_t **path_not_ready**

Flag indicating whether or not the path is ready for use

uint8_t **forward_number**

Forwarding number of the Path Origin; If the entry is associated with a fixed path, the value is 0

uint8_t **lane_counter**

Number of lanes discovered; if the entry is associated with a fixed path, the value is 1.

struct *esp_ble_mesh_df_server_table_change_t*::[anonymous]::[anonymous] **df_table_entry_add_remove**

Structure of directed forwarding table add and remove Structure of directed forwarding table add and remove

uint8_t **dummy**

Event not used currently

struct *esp_ble_mesh_df_server_table_change_t*::[anonymous]::[anonymous] **df_table_entry_change**

Structure of directed forwarding table entry change Directed forwarding table entry change

union *esp_ble_mesh_df_server_table_change_t*::[anonymous] **df_table_info**

Union of directed forwarding table information Directed forwarding table information

struct **esp_ble_mesh_df_server_cb_param_t**

Directed Forwarding Configuration Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_df_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CONTROL_STATUS

ESP_BLE_MESH_MODEL_OP_PATH_METRIC_GET

ESP_BLE_MESH_MODEL_OP_PATH_METRIC_SET

ESP_BLE_MESH_MODEL_OP_PATH_METRIC_STATUS

ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_GET

ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_SET

ESP_BLE_MESH_MODEL_OP_DISCOVERY_TABLE_CAPS_STATUS

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ADD

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEL

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_STATUS

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_ADD

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_DEL

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_STATUS

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_GET

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_DEPS_GET_STATUS

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_CNT_GET

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_CNT_STATUS

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_GET

ESP_BLE_MESH_MODEL_OP_FORWARDING_TABLE_ENTRIES_STATUS

ESP_BLE_MESH_MODEL_OP_WANTED_LANES_GET

ESP_BLE_MESH_MODEL_OP_WANTED_LANES_SET

ESP_BLE_MESH_MODEL_OP_WANTED_LANES_STATUS

ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_GET

ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_SET

ESP_BLE_MESH_MODEL_OP_TWO_WAY_PATH_STATUS

ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_GET

ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_SET

ESP_BLE_MESH_MODEL_OP_PATH_ECHO_INTERVAL_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_NET_TRANSMIT_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_NET_TRANSMIT_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_NET_TRANSMIT_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_RELAY_RETRANSMIT_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_RELAY_RETRANSMIT_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_RELAY_RETRANSMIT_STATUS

ESP_BLE_MESH_MODEL_OP_RSSI_THRESHOLD_GET

ESP_BLE_MESH_MODEL_OP_RSSI_THRESHOLD_SET

ESP_BLE_MESH_MODEL_OP_RSSI_THRESHOLD_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_PATHS_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_PATHS_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_PUB_POLICY_STATUS

ESP_BLE_MESH_MODEL_OP_PATH_DISCOVERY_TIMING_CTL_GET

ESP_BLE_MESH_MODEL_OP_PATH_DISCOVERY_TIMING_CTL_SET

ESP_BLE_MESH_MODEL_OP_PATH_DISCOVERY_TIMING_CTL_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_NET_TRANSMIT_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_NET_TRANSMIT_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_NET_TRANSMIT_STATUS

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_RELAY_RETRANSMIT_GET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_RELAY_RETRANSMIT_SET

ESP_BLE_MESH_MODEL_OP_DIRECTED_CTL_RELAY_RETRANSMIT_STATUS

ESP_BLE_MESH_PATH_DISC_INTERVAL_5_SEC

ESP_BLE_MESH_PATH_DISC_INTERVAL_30_SEC

ESP_BLE_MESH_LANE_DISC_GUARD_INTERVAL_2_SEC

ESP_BLE_MESH_LANE_DISC_GUARD_INTERVAL_10_SEC

ESP_BLE_MESH_DIRECTED_PUB_POLICY_MANAGED_FLOODING

ESP_BLE_MESH_DIRECTED_PUB_POLICY_DIRECTED_FORWARDING

ESP_BLE_MESH_GET_FILTER_MASK(fp, nfp, pom, dm)

ESP_BLE_MESH_MODEL_DF_SRV(srv_data)

Define a new Directed Forwarding Configuration Server model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **srv_data** -- Pointer to a unique Directed Forwarding Configuration Server model user_data.

Returns New Directed Forwarding Configuration Server model instance.

ESP_BLE_MESH_MODEL_DF_CLI(cli_data)

Define a new Directed Forwarding Configuration Client model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique Directed Forwarding Configuration Client model user_data.

Returns New Directed Forwarding Configuration Client model instance.

Type Definitions

typedef void (*`esp_ble_mesh_df_client_cb_t`)(*esp_ble_mesh_df_client_cb_event_t* event, *esp_ble_mesh_df_client_cb_param_t* *param)

Bluetooth Mesh Directed Forwarding Configuration client and server model functions.

Directed Forwarding Configuration Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (*`esp_ble_mesh_df_server_cb_t`)(*esp_ble_mesh_df_server_cb_event_t* event, *esp_ble_mesh_df_server_cb_param_t* *param)

Directed Forwarding Configuration Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum `esp_ble_mesh_df_client_cb_event_t`

This enum value is the event of Directed Forwarding Configuration Client model

Values:

enumerator `ESP_BLE_MESH_DF_CLIENT_SEND_COMP_EVT`

enumerator `ESP_BLE_MESH_DF_CLIENT_SEND_TIMEOUT_EVT`

enumerator `ESP_BLE_MESH_DF_CLIENT_RECV_GET_RSP_EVT`

enumerator `ESP_BLE_MESH_DF_CLIENT_RECV_SET_RSP_EVT`

enumerator `ESP_BLE_MESH_DF_CLIENT_RECV_PUB_EVT`

enumerator `ESP_BLE_MESH_DF_CLIENT_EVT_MAX`

enum `esp_ble_mesh_df_table_action_t`

Values:

enumerator `ESP_BLE_MESH_DF_TABLE_ACT_EMPTY`

enumerator `ESP_BLE_MESH_DF_TABLE_ADD`

enumerator `ESP_BLE_MESH_DF_TABLE_REMOVE`

enumerator `ESP_BLE_MESH_DF_TABLE_ENTRY_CHANGE`

enumerator `ESP_BLE_MESH_DF_TABLE_ACT_MAX_LIMIT`

enum `esp_ble_mesh_df_server_cb_event_t`

This enum value is the event of Directed Forwarding Configuration Server model

Values:

enumerator **ESP_BLE_MESH_DF_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_DF_SERVER_TABLE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_DF_SERVER_EVT_MAX**

Subnet Bridge Configuration

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_brc_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_brc_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_brc_client_callback**(*esp_ble_mesh_brc_client_cb_t* callback)

Register BLE Mesh Bridge Configuration Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_brc_server_callback**(*esp_ble_mesh_brc_server_cb_t* callback)

Register BLE Mesh Bridge Configuration Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_brc_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_brc_client_msg_t* *msg)

Get/Set the value of Bridge Configuration Server model state with the corresponding message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Bridge Configuration Client message.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_brc_client_msg_t**

#include <esp_ble_mesh_brc_model_api.h> Bridge Configuration Client model message union.

Public Members

esp_ble_mesh_bridged_subnets_get_t **bridged_subnets_get**

For ESP_BLE_MESH_MODEL_OP_BRIDGED_SUBNETS_GET

esp_ble_mesh_bridging_table_get_t **bridging_table_get**

For ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_GET

esp_ble_mesh_subnet_bridge_set_t **subnet_bridge_set**

For ESP_BLE_MESH_MODEL_OP_SUBNET_BRIDGE_SET

esp_ble_mesh_bridging_table_add_t **bridging_table_add**

For ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_ADD

esp_ble_mesh_bridging_table_remove_t **bridging_table_remove**

For ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_REMOVE

union **esp_ble_mesh_brc_client_recv_cb_t**

#include <esp_ble_mesh_brc_model_api.h> Bridge Configuration Client model received message union.

Public Members

esp_ble_mesh_subnet_bridge_status_t **subnet_bridge_status**

ESP_BLE_MESH_MODEL_OP_SUBNET_BRIDGE_STATUS

esp_ble_mesh_bridging_table_status_t **bridging_table_status**

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_STATUS

esp_ble_mesh_bridged_subnets_list_t **bridged_subnets_list**

ESP_BLE_MESH_MODEL_OP_BRIDGED_SUBNETS_LIST

esp_ble_mesh_bridging_table_list_t **bridging_table_list**

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_LIST

esp_ble_mesh_bridging_table_size_status_t **bridging_table_size_status**

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_SIZE_STATUS

union **esp_ble_mesh_brc_server_state_change_t**

#include <esp_ble_mesh_brc_model_api.h> Bridge Configuration Server model state change value union.

Public Members

esp_ble_mesh_state_change_bridging_table_add_t **bridging_table_add**

The recv_op in ctx can be used to decide which state is changed. Bridging Table Add

esp_ble_mesh_state_change_bridging_table_remove_t **bridging_table_remove**

Bridging Table Remove

union **esp_ble_mesh_brc_server_cb_value_t**

#include <esp_ble_mesh_brc_model_api.h> Bridge Configuration Server model callback value union.

Public Members

esp_ble_mesh_brc_server_state_change_t **state_change**

For ESP_BLE_MESH_BRC_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_subnet_bridge_table_t**

Parameters of subnet bridge table

Public Members

uint8_t **bridge_direction**

Allowed directions for the bridged traffic

uint8_t **bridge_net_idx**[3]

Two NetKey Indexes are packed into three octets

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_subnet_bridge_set_t**

Parameters of Subnet Bridge Set

Public Members

uint8_t **subnet_bridge**

New Subnet Bridge state

struct **esp_ble_mesh_bridging_table_add_t**

Parameters of Bridging Table Add

Public Members

uint8_t **bridge_direction**

Allowed directions for the bridged traffic

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_bridging_table_remove_t**

Parameters of Bridging Table Remove

Public Members

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_bridged_subnets_get_t**

Parameters of Bridged Subnets Get

Public Members

uint16_t **bridge_filter**

Filter to be applied when reporting the set of pairs of NetKey Indexes

uint16_t **bridge_net_idx**

NetKey Index of any of the subnets

uint8_t **bridge_start_idx**

Start offset in units of Bridging Table state entries

struct **esp_ble_mesh_bridging_table_get_t**

Parameters of Bridging Table Get

Public Members

uint16_t **bridge_net_idx_1**

NetKey Index of first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_start_idx**

Start offset in units of Bridging Table state entries

struct **esp_ble_mesh_subnet_bridge_status_t**

Parameters of Subnet Bridge Status

Public Members

uint8_t **subnet_bridge**

Current Subnet Bridge state

struct **esp_ble_mesh_bridging_table_status_t**

Parameters of Bridging Table Status

Public Members

uint8_t **status**

Status Code for the requesting message

uint8_t **bridge_direction**

Allowed directions for the bridged traffic

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_bridge_net_idx_pair_entry_t**

Bridged_Subnets_List entry format

Public Members

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

struct **esp_ble_mesh_bridged_subnets_list_t**

Parameters of Bridged Subnets List

Public Members

uint16_t **bridge_filter**

Filter applied to the set of pairs of NetKey Indexes

uint16_t **bridge_net_idx**

NetKey Index used for filtering or ignored

uint8_t **bridge_start_idx**

Start offset in units of bridges

uint8_t **bridged_entry_list_size**

Num of pairs of NetKey Indexes

esp_ble_mesh_bridge_net_idx_pair_entry_t ***net_idx_pair**

Filtered set of N pairs of NetKey Indexes

struct **esp_ble_mesh_bridged_addr_list_entry_t**

Bridged_Addresses_List entry format

Public Members

uint8_t **bridge_direction**

Allowed directions for bridged traffic

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_bridging_table_list_t**

Parameters of Bridging Table List

Public Members

uint8_t **status**

Status Code for the requesting message

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_start_idx**

Start offset in units of Bridging Table state entries

uint16_t **bridged_addr_list_size**

Num of pairs of entry

esp_ble_mesh_bridged_addr_list_entry_t ***bridged_addr_list**

List of bridged addresses and allowed traffic directions

struct **esp_ble_mesh_bridging_table_size_status_t**

Parameters of Bridging Table Size Status

Public Members

uint16_t **bridging_table_size**

Bridging Table Size state

struct **esp_ble_mesh_brc_client_send_cb_t**

Result of sending Bridge Configuration Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_brc_client_cb_param_t**

Bridge Configuration Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events.

esp_ble_mesh_brc_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_brc_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_brc_client_cb_param_t*::[anonymous] **[anonymous]**

Union of Bridge Configuration Client callback

struct **esp_ble_mesh_state_change_bridging_table_add_t**

Bridge Configuration Server model related context.

Parameters of Bridging Table Add

Public Members

uint8_t **bridge_direction**

Allowed directions for the bridged traffic

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_state_change_bridging_table_remove_t**

Parameters of Bridging Table Remove

Public Members

uint16_t **bridge_net_idx_1**

NetKey Index of the first subnet

uint16_t **bridge_net_idx_2**

NetKey Index of the second subnet

uint16_t **bridge_addr_1**

Address of the node in the first subnet

uint16_t **bridge_addr_2**

Address of the node in the second subnet

struct **esp_ble_mesh_brc_server_cb_param_t**

Bridge Configuration Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_brc_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_SUBNET_BRIDGE_GET

ESP_BLE_MESH_MODEL_OP_SUBNET_BRIDGE_SET

ESP_BLE_MESH_MODEL_OP_SUBNET_BRIDGE_STATUS

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_ADD

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_REMOVE

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_STATUS

ESP_BLE_MESH_MODEL_OP_BRIDGED_SUBNETS_GET

ESP_BLE_MESH_MODEL_OP_BRIDGED_SUBNETS_LIST

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_GET

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_LIST

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_SIZE_GET

ESP_BLE_MESH_MODEL_OP_BRIDGING_TABLE_SIZE_STATUS

ESP_BLE_MESH_MODEL_BRC_SRV(srv_data)

Define a new Bridge Configuration Server model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **srv_data** -- Pointer to a unique Bridge Configuration Server model user_data.

Returns New Bridge Configuration Server model instance.

ESP_BLE_MESH_MODEL_BRC_CLI(cli_data)

Define a new Bridge Configuration Client model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique Bridge Configuration Client model user_data.

Returns New Bridge Configuration Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_brc_client_cb_t**)(*esp_ble_mesh_brc_client_cb_event_t* event, *esp_ble_mesh_brc_client_cb_param_t* *param)

Bluetooth Mesh Bridge Configuration client and server model functions.

Bridge Configuration Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_brc_server_cb_t**)(*esp_ble_mesh_brc_server_cb_event_t* event, *esp_ble_mesh_brc_server_cb_param_t* *param)

Bridge Configuration Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_brc_client_cb_event_t**

This enum value is the event of Bridge Configuration Client model

Values:

enumerator **ESP_BLE_MESH_BRC_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_BRC_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_BRC_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_BRC_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_BRC_CLIENT_EVT_MAX**

enum **esp_ble_mesh_brc_server_cb_event_t**

This enum value is the event of Bridge Configuration Server model

Values:

enumerator **ESP_BLE_MESH_BRC_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_BRC_SERVER_EVT_MAX**

Mesh Private Beacon

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_prb_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_prb_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_prb_client_callback**(*esp_ble_mesh_prb_client_cb_t* callback)

Register BLE Mesh Private Beacon Client Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_prb_server_callback**(*esp_ble_mesh_prb_server_cb_t* callback)

Register BLE Mesh Private Beacon Server Model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_prb_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_prb_client_msg_t* *msg)

Get/Set the value of Private Beacon Server Model states using the corresponding messages of Private Beacon Client Model.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Mesh Private Beacon Client message.

Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_prb_client_msg_t**

#include <esp_ble_mesh_prb_model_api.h> Mesh Private Beacon Client model message union.

Public Members

esp_ble_mesh_priv_beacon_set_t **priv_beacon_set**

ESP_BLE_MESH_MODEL_OP_PRIV_BEACON_SET.

esp_ble_mesh_priv_gatt_proxy_set_t **priv_gatt_proxy_set**

ESP_BLE_MESH_MODEL_OP_PRIV_GATT_PROXY_SET.

esp_ble_mesh_priv_node_id_get_t **priv_node_id_get**

ESP_BLE_MESH_MODEL_OP_PRIV_NODE_IDENTITY_GET.

esp_ble_mesh_priv_node_id_set_t **priv_node_id_set**

ESP_BLE_MESH_MODEL_OP_PRIV_NODE_IDENTITY_SET.

union **esp_ble_mesh_prb_client_recv_cb_t**

#include <esp_ble_mesh_prb_model_api.h> Private Beacon Client Model received message union.

Public Members

esp_ble_mesh_priv_beacon_status_cb_t **priv_beacon_status**

The private beacon status value

esp_ble_mesh_priv_gatt_proxy_status_cb_t **priv_gatt_proxy_status**

The private gatt proxy status value

esp_ble_mesh_priv_node_identity_status_cb_t **priv_node_id_status**

The private node identity status value

union **esp_ble_mesh_prb_server_state_change_t**

#include <esp_ble_mesh_prb_model_api.h> Mesh Private Beacon Server model state change value union.

Public Members

esp_ble_mesh_state_change_priv_beacon_set_t **priv_beacon_set**

The recv_op in ctx can be used to decide which state is changed. Private Beacon Set

esp_ble_mesh_state_change_priv_gatt_proxy_set_t **priv_gatt_proxy_set**

Private GATT Proxy Set

esp_ble_mesh_state_change_priv_node_id_set_t **priv_node_id_set**

Private Node Identity Set

union **esp_ble_mesh_prb_server_cb_value_t**

#include <esp_ble_mesh_prb_model_api.h> Private Beacon Server model callback value union.

Public Members

esp_ble_mesh_prb_server_state_change_t **state_change**

ESP_BLE_MESH_PRB_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_prb_srv_t**

Private Beacon Server Model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Private Beacon Server Model

uint8_t **private_beacon**

Value of Private Beacon state

uint8_t **random_update_interval**

Value of Random Update Interval Steps state

uint8_t **private_gatt_proxy**

Value of Private GATT Proxy state

struct k_delayed_work **update_timer**

Timer for update the random field of private beacon

struct **esp_ble_mesh_priv_beacon_set_t**

Parameter of private Beacon Set

Public Members

uint8_t **private_beacon**

New Private Beacon state

bool **is_effect**

Decide if update_interval exists

uint8_t **update_interval**

New Random Update Interval Steps state

struct **esp_ble_mesh_priv_gatt_proxy_set_t**

Parameter of Private GATT Proxy Set

Public Members

uint8_t **private_gatt_proxy**

New Private GATT Proxy state

struct **esp_ble_mesh_priv_node_id_get_t**

Parameter of Private node identity Get

Public Members

uint16_t **net_idx**

Index of the NetKey

struct **esp_ble_mesh_priv_node_id_set_t**

Parameter of Private node identity Set

Public Members

uint16_t **net_idx**

Index of the NetKey

uint8_t **private_node_id**

New Private Node Identity state

struct **esp_ble_mesh_priv_beacon_status_cb_t**

Parameter of Private Beacon Status

Public Members

uint8_t **private_beacon**

Current value of the Private Beacon state

uint8_t **update_interval**

Current value of the Random Update Interval Steps state

struct **esp_ble_mesh_priv_gatt_proxy_status_cb_t**

Parameter of Private GATT Proxy Status

Public Members

uint8_t **private_gatt_proxy**

Private GATT Proxy state

struct **esp_ble_mesh_priv_node_identity_status_cb_t**

Parameters of Private Node Identity Status

Public Members

uint8_t **status**

Status Code for the requesting message

uint16_t **net_idx**

Index of the NetKey

uint8_t **private_node_id**

Private Node Identity state

struct **esp_ble_mesh_prb_client_send_cb_t**

Result of sending Bridge Configuration Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_prb_client_cb_param_t**

Mesh Private Beacon Client Model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

The client common parameters.

esp_ble_mesh_prb_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_prb_client_recv_cb_t **recv**

The private beacon message status callback values

union *esp_ble_mesh_prb_client_cb_param_t*::[anonymous] **[anonymous]**

Union of Private Beacon Client callback

struct **esp_ble_mesh_state_change_priv_beacon_set_t**

Mesh Private Beacon Server model related context.

Parameters of Private Beacon Set.

Public Members

uint8_t **private_beacon**

Private Beacon state

bool **is_effect**

Decide whether update_interval effect

uint8_t **update_interval**

Random Update Interval Steps state

struct **esp_ble_mesh_state_change_priv_gatt_proxy_set_t**

Parameters of Private GATT Proxy Set.

Public Members

uint8_t **private_gatt_proxy**

Private GATT Proxy state

struct **esp_ble_mesh_state_change_priv_node_id_set_t**

Parameters of Private Node Identity Set.

Public Members

uint16_t **net_idx**

Index of the NetKey

uint8_t **private_node_id**

Private Node Identity state

struct **esp_ble_mesh_prb_server_cb_param_t**

Private Beacon Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_prb_server_cb_value_t **value**

Value of the received private beacon messages

Macros

ESP_BLE_MESH_MODEL_OP_PRIV_BEACON_GET

ESP_BLE_MESH_MODEL_OP_PRIV_BEACON_SET

ESP_BLE_MESH_MODEL_OP_PRIV_BEACON_STATUS

ESP_BLE_MESH_MODEL_OP_PRIV_GATT_PROXY_GET

ESP_BLE_MESH_MODEL_OP_PRIV_GATT_PROXY_SET

ESP_BLE_MESH_MODEL_OP_PRIV_GATT_PROXY_STATUS

ESP_BLE_MESH_MODEL_OP_PRIV_NODE_IDENTITY_GET

ESP_BLE_MESH_MODEL_OP_PRIV_NODE_IDENTITY_SET

ESP_BLE_MESH_MODEL_OP_PRIV_NODE_IDENTITY_STATUS

ESP_BLE_MESH_MODEL_PRB_SRV(srv_data)

Define a new Private Beacon Server Model.

Note: The Private Beacon Server Model can only be included by a Primary Element.

Parameters

- **srv_data** -- Pointer to a unique Private Beacon Server Model user_data.

Returns New Private Beacon Server Model instance.

ESP_BLE_MESH_MODEL_PRB_CLI(cli_data)

Define a new Private Beacon Client Model.

Note: The Private Beacon Client Model can only be included by a Primary Element.

Parameters

- **cli_data** -- Pointer to a unique struct *esp_ble_mesh_client_t*.

Returns New Private Beacon Client Model instance.

Type Definitions

typedef void (***esp_ble_mesh_prb_client_cb_t**)(*esp_ble_mesh_prb_client_cb_event_t* event, *esp_ble_mesh_prb_client_cb_param_t* *param)

Bluetooth Mesh Private Beacon Client and Server Model functions.

Private Beacon Client Model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_prb_server_cb_t**)(*esp_ble_mesh_prb_server_cb_event_t* event, *esp_ble_mesh_prb_server_cb_param_t* *param)

Private Beacon Server Model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_prb_client_cb_event_t**

This enum value is the event of Private Beacon Client Model

Values:

enumerator **ESP_BLE_MESH_PRB_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_PRB_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_PRB_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_PRB_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_PRB_CLIENT_EVT_MAX**

enum **esp_ble_mesh_prb_server_cb_event_t**

This enum value is the event of Private Beacon Server model

Values:

enumerator **ESP_BLE_MESH_PRB_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_PRB_SERVER_EVT_MAX**

On-Demand Private Proxy

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_odp_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_odp_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_odp_client_callback**(*esp_ble_mesh_odp_client_cb_t* callback)

Register BLE Mesh On-Demand Private Proxy Config Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_odp_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_odp_client_msg_t* *msg)

Get the value of On-Demand Private Proxy Config Server model state with the corresponding get message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to On-Demand Private Proxy Config Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_odp_server_callback**(*esp_ble_mesh_odp_server_cb_t* callback)

Register BLE Mesh On-Demand Private Proxy Config Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_odp_client_msg_t**

#include <esp_ble_mesh_odp_model_api.h> On-Demand Private Proxy Client model message union.

Public Members

esp_ble_mesh_od_priv_proxy_set_t **od_priv_proxy_set**

For ESP_BLE_MESH_MODEL_OP_OD_PRIV_PROXY_SET

union **esp_ble_mesh_odp_client_recv_cb_t**

#include <esp_ble_mesh_odp_model_api.h> On-Demand Private Proxy Client model received message union.

Public Members

esp_ble_mesh_od_priv_proxy_status_t **od_priv_proxy_status**

For ESP_BLE_MESH_MODEL_OP_OD_PRIV_PROXY_STATUS

union **esp_ble_mesh_odp_server_state_change_t**

#include <esp_ble_mesh_odp_model_api.h> On-Demand Private Proxy Config Server model related context.

On-Demand Private Proxy Config Server model state change value union

Public Members

uint8_t **dummy**

Event not used currently

union **esp_ble_mesh_odp_server_cb_value_t**

#include <esp_ble_mesh_odp_model_api.h> On-Demand Private Proxy Config Server model callback value union.

Public Members

esp_ble_mesh_odp_server_state_change_t **state_change**

For ESP_BLE_MESH_ODP_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_odp_srv_t**

On-Demand Private Proxy Config Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to On-Demand Private Proxy Config Server model

uint8_t **on_demand_private_gatt_proxy**

Duration in seconds of the interval during which advertising with Private Network Identity type is enabled after receiving a Solicitation PDU or after a client disconnect. Note: Binding with the Private GATT Proxy state.

struct **esp_ble_mesh_od_priv_proxy_set_t**

Parameter of On-Demand Private Proxy Set

Public Members

uint8_t **gatt_proxy**

On-Demand Private GATT Proxy

struct **esp_ble_mesh_od_priv_proxy_status_t**

Parameter of On-Demand Private Proxy Status

Public Members

uint8_t **gatt_proxy**

On-Demand Private GATT Proxy

struct **esp_ble_mesh_odp_client_send_cb_t**

Result of sending On-Demand Private Proxy Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_odp_client_cb_param_t**

On-Demand Private Proxy Config Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events

esp_ble_mesh_odp_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_odp_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_odp_client_cb_param_t*::[anonymous] **[anonymous]**

Union of ODP Client callback

struct **esp_ble_mesh_odp_server_cb_param_t**

On-Demand Private Proxy Config Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_odp_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_OD_PRIV_PROXY_GET

ESP_BLE_MESH_MODEL_OP_OD_PRIV_PROXY_SET

ESP_BLE_MESH_MODEL_OP_OD_PRIV_PROXY_STATUS

ESP_BLE_MESH_MODEL_ODP_SRV(srv_data)

Define a new On-Demand Private Proxy Config Server model.

Note: The On-Demand Private Proxy Server model is used to represent the ability to enable advertising with Private Network Identity type of a node. This model extends the Mesh Private Beacon Server model. When this model is present on an element, the corresponding Solicitation PDU RPL Configuration Server model shall also be present. The model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **srv_data** -- Pointer to a unique On-Demand Private Proxy Config Server model user_data.

Returns New On-Demand Private Proxy Config Server model instance.

ESP_BLE_MESH_MODEL_ODP_CLI(cli_data)

Define a new On-Demand Private Proxy Config Client model.

Note: The model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique On-Demand Private Proxy Config Client model user_data.

Returns New On-Demand Private Proxy Config Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_odp_client_cb_t**)(*esp_ble_mesh_odp_client_cb_event_t* event, *esp_ble_mesh_odp_client_cb_param_t* *param)

Bluetooth Mesh On-Demand Private Proxy Config client and server model functions.

On-Demand Private Proxy Config Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_odp_server_cb_t**)(*esp_ble_mesh_odp_server_cb_event_t* event, *esp_ble_mesh_odp_server_cb_param_t* *param)

On-Demand Private Proxy Config Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_odp_client_cb_event_t**

This enum value is the event of On-Demand Private Proxy Config Client model

Values:

enumerator **ESP_BLE_MESH_ODP_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_ODP_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_ODP_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_ODP_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_ODP_CLIENT_EVT_MAX**

enum **esp_ble_mesh_odp_server_cb_event_t**

This enum value is the event of On-Demand Private Proxy Config Server model

Values:

enumerator **ESP_BLE_MESH_ODP_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_ODP_SERVER_EVT_MAX**

SAR Configuration

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_sar_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_sar_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_sar_client_callback**(*esp_ble_mesh_sar_client_cb_t* callback)

Register BLE Mesh SAR Configuration Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_sar_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_sar_client_msg_t* *msg)

Get the value of SAR Configuration Server model state with the corresponding get message.

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to SAR Configuration Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_sar_server_callback**(*esp_ble_mesh_sar_server_cb_t* callback)

Register BLE Mesh SAR Configuration Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_sar_client_msg_t**

#include <esp_ble_mesh_sar_model_api.h> SAR Configuration Client model message union.

Public Members

esp_ble_mesh_sar_transmitter_set_t **sar_transmitter_set**

For ESP_BLE_MESH_MODEL_OP_SAR_TRANSMITTER_SET

esp_ble_mesh_sar_receiver_set_t **sar_receiver_set**

For ESP_BLE_MESH_MODEL_OP_SAR_RECEIVER_SET

union **esp_ble_mesh_sar_client_recv_cb_t**

#include <esp_ble_mesh_sar_model_api.h> SAR Configuration Client model received message union.

Public Members

esp_ble_mesh_sar_transmitter_status_t **sar_transmitter_status**

For ESP_BLE_MESH_MODEL_OP_SAR_TRANSMITTER_STATUS

esp_ble_mesh_sar_receiver_status_t **sar_receiver_status**

For ESP_BLE_MESH_MODEL_OP_SAR_RECEIVE_STATUS

union **esp_ble_mesh_sar_server_state_change_t**

#include <esp_ble_mesh_sar_model_api.h> SAR Configuration Server model related context.

SAR Configuration Server model state change value union

Public Members

uint8_t **dummy**

Event not used currently

union **esp_ble_mesh_sar_server_cb_value_t**

#include <esp_ble_mesh_sar_model_api.h> SAR Configuration Server model callback value union.

Public Members

esp_ble_mesh_sar_server_state_change_t **state_change**

For ESP_BLE_MESH_SAR_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_sar_srv_t**

SAR Configuration Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to SAR Configuration Server model

struct **esp_ble_mesh_sar_transmitter_set_t**

Parameters of SAR Transmitter Set

Public Members

uint8_t **sar_segment_interval_step**

SAR Segment Interval Step state value

uint8_t **sar_unicast_retrans_count**

SAR Unicast Retransmissions Count state

uint8_t **sar_unicast_retrans_without_progress_count**

SAR Unicast Retransmissions Without Progress Count state

uint8_t **sar_unicast_retrans_interval_step**

SAR Unicast Retransmissions Interval Step state

uint8_t **sar_unicast_retrans_interval_increment**

SAR Unicast Retransmissions Interval Increment state

uint8_t **sar_multicast_retrans_count**

SAR Multicast Retransmissions Count state

uint8_t **sar_multicast_retrans_interval_step**

SAR Multicast Retransmissions Interval state

struct **esp_ble_mesh_sar_receiver_set_t**

Parameters of SAR Receiver Set

Public Members

uint8_t **sar_segments_threshold**

SAR Segments Threshold state

uint8_t **sar_ack_delay_increment**

SAR Acknowledgment Delay Increment state

uint8_t **sar_discard_timeout**

SAR Discard Timeout state

uint8_t **sar_receiver_segment_interval_step**

SAR Receiver Segment Interval Step state

uint8_t **sar_ack_retrans_count**

SAR Acknowledgment Retransmissions Count state

struct **esp_ble_mesh_sar_transmitter_status_t**

Parameters of SAR Transmitter Status

Public Members

uint8_t **sar_segment_interval_step**

SAR Segment Interval Step state value

uint8_t **sar_unicast_retrans_count**

SAR Unicast Retransmissions Count state

uint8_t **sar_unicast_retrans_without_progress_count**

SAR Unicast Retransmissions Without Progress Count state

uint8_t **sar_unicast_retrans_interval_step**

SAR Unicast Retransmissions Interval Step state

uint8_t **sar_unicast_retrans_interval_increment**

SAR Unicast Retransmissions Interval Increment state

uint8_t **sar_multicast_retrans_count**

SAR Multicast Retransmissions Count state

uint8_t **sar_multicast_retrans_interval_step**

SAR Multicast Retransmissions Interval state

struct **esp_ble_mesh_sar_receiver_status_t**

Parameters of SAR Receiver Status

Public Members

uint8_t **sar_segments_threshold**

SAR Segments Threshold state

uint8_t **sar_ack_delay_increment**

SAR Acknowledgment Delay Increment state

uint8_t **sar_discard_timeout**

SAR Discard Timeout state

uint8_t **sar_receiver_segment_interval_step**

SAR Receiver Segment Interval Step state

uint8_t **sar_ack_retrans_count**

SAR Acknowledgment Retransmissions Count state

struct **esp_ble_mesh_sar_client_send_cb_t**

Result of sending SAR Configuration Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_sar_client_cb_param_t**

SAR Configuration Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events.

esp_ble_mesh_sar_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_sar_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_sar_client_cb_param_t*::[anonymous] **[anonymous]**

Union of SAR Client callback

struct **esp_ble_mesh_sar_server_cb_param_t**

SAR Configuration Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_sar_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_SAR_TRANSMITTER_GET

ESP_BLE_MESH_MODEL_OP_SAR_TRANSMITTER_SET

ESP_BLE_MESH_MODEL_OP_SAR_TRANSMITTER_STATUS

ESP_BLE_MESH_MODEL_OP_SAR_RECEIVER_GET

ESP_BLE_MESH_MODEL_OP_SAR_RECEIVER_SET

ESP_BLE_MESH_MODEL_OP_SAR_RECEIVER_STATUS

ESP_BLE_MESH_MODEL_SAR_SRV(srv_data)

Define a new SAR Configuration Server model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters
- **srv_data** -- Pointer to a unique SAR Configuration Server model user_data.

Returns New SAR Configuration Server model instance.

ESP_BLE_MESH_MODEL_SAR_CLI(cli_data)

Define a new SAR Configuration Client model.

Note: If supported, the model shall be supported by the primary element and shall not be supported by any secondary elements.

Parameters
- **cli_data** -- Pointer to a unique SAR Configuration Client model user_data.

Returns New SAR Configuration Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_sar_client_cb_t**)(*esp_ble_mesh_sar_client_cb_event_t* event, *esp_ble_mesh_sar_client_cb_param_t* *param)

Bluetooth Mesh SAR Configuration client and server model functions.

SAR Configuration Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_sar_server_cb_t**)(*esp_ble_mesh_sar_server_cb_event_t* event, *esp_ble_mesh_sar_server_cb_param_t* *param)

SAR Configuration Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_sar_client_cb_event_t**

This enum value is the event of SAR Configuration Client model

Values:

enumerator **ESP_BLE_MESH_SAR_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_SAR_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_SAR_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_SAR_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_SAR_CLIENT_EVT_MAX**

enum **esp_ble_mesh_sar_server_cb_event_t**

This enum value is the event of SAR Configuration Server model

Values:

enumerator **ESP_BLE_MESH_SAR_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_SAR_SERVER_EVT_MAX**

Solicitation PDU RPL Configuration

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_srpl_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_srpl_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_srpl_client_callback**(*esp_ble_mesh_srpl_client_cb_t* callback)

Register BLE Mesh Solicitation PDU RPL Configuration Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_srpl_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_srpl_client_msg_t* *msg)

Set the value of Solicitation PDU RPL Configuration Server model state with the corresponding set message.

Parameters

- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Solicitation PDU RPL Configuration Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_srpl_server_callback**(*esp_ble_mesh_srpl_server_cb_t* callback)

Register BLE Mesh Solicitation PDU RPL Configuration Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_srpl_client_msg_t**

#include <esp_ble_mesh_srpl_model_api.h> Solicitation PDU RPL Configuration Client model message union.

Public Members

esp_ble_mesh_srpl_items_clear_t **srpl_items_clear**

For ESP_BLE_MESH_MODEL_OP_SRPL_ITEMS_CLEAR

union **esp_ble_mesh_srpl_client_recv_cb_t**

#include <esp_ble_mesh_srpl_model_api.h> Solicitation PDU RPL Configuration Client model received message union.

Public Members

esp_ble_mesh_srpl_items_status_t **srpl_items_status**

For ESP_BLE_MESH_MODEL_OP_SRPL_ITEMS_STATUS

union **esp_ble_mesh_srpl_server_state_change_t**

#include <esp_ble_mesh_srpl_model_api.h> Solicitation PDU RPL Configuration Server model related context.

Solicitation PDU RPL Configuration Server model state change value union

Public Members

uint8_t **dummy**

Currently this event is not used.

union **esp_ble_mesh_srpl_server_cb_value_t**

#include <esp_ble_mesh_srpl_model_api.h> Solicitation PDU RPL Configuration Server model callback value union.

Public Members

esp_ble_mesh_srpl_server_state_change_t **state_change**

ESP_BLE_MESH_SRPL_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_srpl_srv_t**

Solicitation PDU RPL Configuration Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Solicitation PDU RPL Configuration Server model

struct **esp_ble_mesh_srpl_items_clear_t**

Parameter of Solicitation PDU RPL Items Clear

Public Members

esp_ble_mesh_uar_t **addr_range**

Unicast address range

struct **esp_ble_mesh_srpl_items_status_t**

Parameter of Solicitation PDU RPL Items Clear Status

Public Members

esp_ble_mesh_uar_t **addr_range**

Unicast address range

struct **esp_ble_mesh_srpl_client_send_cb_t**

Result of sending Solicitation PDU RPL Configuration Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_srpl_client_cb_param_t**

Solicitation PDU RPL Configuration Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events.

esp_ble_mesh_srpl_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_srpl_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_srpl_client_cb_param_t*::[anonymous] **[anonymous]**

Union of SRPL Client callback

struct **esp_ble_mesh_srpl_server_cb_param_t**

Solicitation PDU RPL Configuration Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_srpl_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_SRPL_ITEMS_CLEAR

ESP_BLE_MESH_MODEL_OP_SRPL_ITEMS_CLEAR_UNACK

ESP_BLE_MESH_MODEL_OP_SRPL_ITEMS_STATUS

ESP_BLE_MESH_MODEL_SRPL_SRV(srv_data)

Define a new Solicitation PDU RPL Configuration Server model.

Note: The Solicitation PDU RPL Configuration Server model extends the On-Demand Private Proxy Server model. If the model is supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **srv_data** -- Pointer to a unique Solicitation PDU RPL Configuration Server model user_data.

Returns New Solicitation PDU RPL Configuration Server model instance.

ESP_BLE_MESH_MODEL_SRPL_CLI(cli_data)

Define a new Solicitation PDU RPL Configuration Client model.

Note: If supported, the model shall be supported by the primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique Solicitation PDU RPL Configuration Client model user_data.

Returns New Solicitation PDU RPL Configuration Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_srpl_client_cb_t**)(*esp_ble_mesh_srpl_client_cb_event_t* event, *esp_ble_mesh_srpl_client_cb_param_t* *param)

Bluetooth Mesh Solicitation PDU RPL Configuration client and server model functions.

Solicitation PDU RPL Configuration Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_srpl_server_cb_t**)(*esp_ble_mesh_srpl_server_cb_event_t* event, *esp_ble_mesh_srpl_server_cb_param_t* *param)

Solicitation PDU RPL Configuration Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_srpl_client_cb_event_t**

This enum value is the event of Solicitation PDU RPL Configuration Client model

Values:

enumerator **ESP_BLE_MESH_SRPL_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_SRPL_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_SRPL_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_SRPL_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_SRPL_CLIENT_EVT_MAX**

enum **esp_ble_mesh_srpl_server_cb_event_t**

This enum value is the event of Solicitation PDU RPL Configuration Server model

Values:

enumerator **ESP_BLE_MESH_SRPL_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_SRPL_SERVER_EVT_MAX**

Opcodes Aggregator

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_agg_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_agg_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_agg_client_callback**(*esp_ble_mesh_agg_client_cb_t* callback)

Register BLE Mesh Opcodes Aggregator Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_agg_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_agg_client_msg_t* *msg)

Set the value of Opcodes Aggregator Server model state with the corresponding set message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Opcodes Aggregator Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_agg_server_callback**(*esp_ble_mesh_agg_server_cb_t* callback)

Register BLE Mesh Opcodes Aggregator Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_agg_client_msg_t**

#include <esp_ble_mesh_agg_model_api.h> Opcodes Aggregator Client model message union.

Public Members

esp_ble_mesh_agg_sequence_t **agg_sequence**

For ESP_BLE_MESH_MODEL_OP_AGG_SEQUENCE

union **esp_ble_mesh_agg_client_recv_cb_t**

#include <esp_ble_mesh_agg_model_api.h> Opcodes Aggregator Client model received message union.

Public Members

esp_ble_mesh_agg_status_t **agg_status**

For ESP_BLE_MESH_MODEL_OP_AGG_STATUS

union **esp_ble_mesh_agg_server_recv_msg_t**

#include <esp_ble_mesh_agg_model_api.h> Opcodes Aggregator Server model related context.

Opcodes Aggregator Server model received message union

Public Members

esp_ble_mesh_agg_sequence_t **agg_sequence**

For ESP_BLE_MESH_MODEL_OP_AGG_SEQUENCE

Structures

struct **esp_ble_mesh_agg_srv_t**

Opcodes Aggregator Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Opcodes Aggregator Server model

struct **esp_ble_mesh_agg_item_t**

Parameters of Aggregator Item

Public Members

uint16_t **length_format**

0: Length_Short; 1: Length_Long

uint16_t **length**

Size of Opcode_And_Parameters field

const uint8_t ***data**

Opcode and parameters

struct **esp_ble_mesh_agg_sequence_t**

Parameters of Opcodes Aggregator Sequence

Public Members

uint16_t **element_addr**

Element address

struct net_buf_simple ***items**

List of items with each item represented as an Aggregator Item

struct **esp_ble_mesh_agg_status_t**

Parameters of Opcodes Aggregator Status

Public Members

uint8_t **status**

Status of the most recent operation

uint16_t **element_addr**

Element Address

struct net_buf_simple ***items**

List of status items with each status item containing an unacknowledged access layer message or empty item (Optional)

struct **esp_ble_mesh_agg_client_send_cb_t**

Result of sending Opcodes Aggregator Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_agg_client_cb_param_t**

Opcodes Aggregator Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events

esp_ble_mesh_agg_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_agg_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_agg_client_cb_param_t*::[anonymous] **[anonymous]**

Union of AGG Client callback

struct **esp_ble_mesh_agg_server_cb_param_t**

Opcodes Aggregator Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_agg_server_recv_msg_t **recv**

Received message callback values

union *esp_ble_mesh_agg_server_cb_param_t*::[anonymous] **[anonymous]**

Union of AGG Server callback

Macros

ESP_BLE_MESH_MODEL_OP_AGG_SEQUENCE

Defines the Opcodes Aggregator message opcode.

ESP_BLE_MESH_MODEL_OP_AGG_STATUS

ESP_BLE_MESH_AGG_STATUS_SUCCESS

Defines the status codes for Opcodes Aggregator messages.

ESP_BLE_MESH_AGG_STATUS_INVALID_ADDRESS

ESP_BLE_MESH_AGG_STATUS_INVALID_MODEL

ESP_BLE_MESH_AGG_STATUS_WRONG_ACCESS_KEY

ESP_BLE_MESH_AGG_STATUS_WRONG_OPCODE

ESP_BLE_MESH_AGG_STATUS_MSG_NOT_UNDERSTOOD

ESP_BLE_MESH_AGG_ITEM_LENGTH_FORMAT_SHORT

Values of the Length_Format

ESP_BLE_MESH_AGG_ITEM_LENGTH_FORMAT_LONG

ESP_BLE_MESH_MODEL_AGG_SRV(srv_data)

Define a new Opcodes Aggregator Server model.

Note: If supported, the Opcodes Aggregator Server model shall be supported by a primary element.

Parameters

- **srv_data** -- Pointer to a unique Opcodes Aggregator Server model user_data.

Returns New Opcodes Aggregator Server model instance.

ESP_BLE_MESH_MODEL_AGG_CLI(cli_data)

Define a new Opcodes Aggregator Client model.

Note: If supported, the model shall be supported by the primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique Opcodes Aggregator Client model user_data.

Returns New Opcodes Aggregator Client model instance.

Type Definitions

typedef void (*`esp_ble_mesh_agg_client_cb_t`)(*esp_ble_mesh_agg_client_cb_event_t* event, *esp_ble_mesh_agg_client_cb_param_t* *param)

Bluetooth Mesh Opcodes Aggregator client and server model functions.

Opcodes Aggregator Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (*`esp_ble_mesh_agg_server_cb_t`)(*esp_ble_mesh_agg_server_cb_event_t* event, *esp_ble_mesh_agg_server_cb_param_t* *param)

Opcodes Aggregator Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum `esp_ble_mesh_agg_client_cb_event_t`

This enum value is the event of Opcodes Aggregator Client model

Values:

enumerator `ESP_BLE_MESH_AGG_CLIENT_SEND_COMP_EVT`

enumerator `ESP_BLE_MESH_AGG_CLIENT_SEND_TIMEOUT_EVT`

enumerator `ESP_BLE_MESH_AGG_CLIENT_RECV_RSP_EVT`

enumerator `ESP_BLE_MESH_AGG_CLIENT_RECV_PUB_EVT`

enumerator `ESP_BLE_MESH_AGG_CLIENT_EVT_MAX`

enum `esp_ble_mesh_agg_server_cb_event_t`

This enum value is the event of Opcodes Aggregator Server model

Values:

enumerator `ESP_BLE_MESH_AGG_SERVER_RECV_MSG_EVT`

enumerator `ESP_BLE_MESH_AGG_SERVER_EVT_MAX`

Large Composition Data

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_lcd_model_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_lcd_model_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_register_lcd_client_callback**(*esp_ble_mesh_lcd_client_cb_t* callback)

Register BLE Mesh Large Composition Data Client model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_lcd_client_send**(*esp_ble_mesh_client_common_param_t* *params, *esp_ble_mesh_lcd_client_msg_t* *msg)

Get the value of Large Composition Data Server model state with the corresponding get message.

Parameters
- **params** -- **[in]** Pointer to BLE Mesh common client parameters.
- **msg** -- **[in]** Pointer to Large Composition Data Client message.

Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_register_lcd_server_callback**(*esp_ble_mesh_lcd_server_cb_t* callback)

Register BLE Mesh Large Composition Data Server model callback.

Parameters **callback** -- **[in]** Pointer to the callback function.
Returns ESP_OK on success or error code otherwise.

Unions

union **esp_ble_mesh_lcd_client_msg_t**

#include <esp_ble_mesh_lcd_model_api.h> Large Composition Data Client model message union.

Public Members

esp_ble_mesh_large_comp_data_get_t **large_comp_data_get**

For ESP_BLE_MESH_MODEL_OP_LARGE_COMP_DATA_GET

esp_ble_mesh_models_metadata_get_t **models_metadata_get**

For ESP_BLE_MESH_MODEL_OP_MODELS_METADATA_GET

union **esp_ble_mesh_lcd_client_recv_cb_t**

#include <esp_ble_mesh_lcd_model_api.h> Large Composition Data Client model received message union.

Public Members

esp_ble_mesh_large_comp_data_status_t **large_comp_data_status**

For ESP_BLE_MESH_MODEL_OP_LARGE_COMP_DATA_STATUS

esp_ble_mesh_models_metadata_status_t **models_metadata_status**

For ESP_BLE_MESH_MODEL_OP_MODELS_METADATA_STATUS

union **esp_ble_mesh_lcd_server_state_change_t**

#include <esp_ble_mesh_lcd_model_api.h> Large Composition Data Server model related context.

Large Composition Data Server model state change value union

Public Members

uint8_t **dummy**

Event not used currently

union **esp_ble_mesh_lcd_server_cb_value_t**

#include <esp_ble_mesh_lcd_model_api.h> Large Composition Data Server model callback value union.

Public Members

esp_ble_mesh_lcd_server_state_change_t **state_change**

For ESP_BLE_MESH_LCD_SERVER_STATE_CHANGE_EVT

Structures

struct **esp_ble_mesh_lcd_srv_t**

Large Composition Data Server model context

Public Members

esp_ble_mesh_model_t ***model**

Pointer to Large Composition Data Server model

struct **esp_ble_mesh_large_comp_data_get_t**

Parameters of Large Composition Data Get

Public Members

uint8_t **page**

Page number of the Composition Data

uint16_t **offset**

Offset within the page

struct **esp_ble_mesh_models_metadata_get_t**

Parameters of Models Metadata Get

Public Members

uint8_t **metadata_page**

Page number of the Models Metadata

uint16_t **offset**

Offset within the page

struct **esp_ble_mesh_large_comp_data_status_t**

Parameters of Large Composition Data Status

Public Members

uint8_t **page**

Page number of the Composition Data

uint16_t **offset**

Offset within the page

uint16_t **total_size**

Total size of the page

struct net_buf_simple ***data**

Composition Data for the identified portion of the page

struct **esp_ble_mesh_models_metadata_status_t**

Parameters of Models Metadata Data Status

Public Members

uint8_t **metadata_page**

Page number of the Models Metadata

uint16_t **offset**

Offset within the page

uint16_t **total_size**

Total size of the page

struct net_buf_simple ***data**

Models Metadata for the identified portion of the page

struct **esp_ble_mesh_lcd_client_send_cb_t**

Result of sending Large Composition Data Client messages

Public Members

int **err_code**

Result of sending a message

struct **esp_ble_mesh_lcd_client_cb_param_t**

Large Composition Data Client model callback parameters

Public Members

esp_ble_mesh_client_common_param_t ***params**

Client common parameters, used by all events.

esp_ble_mesh_lcd_client_send_cb_t **send**

Result of sending a message

esp_ble_mesh_lcd_client_recv_cb_t **recv**

Parameters of received status message

union *esp_ble_mesh_lcd_client_cb_param_t*::[anonymous] **[anonymous]**

Union of LCD Client callback

struct **esp_ble_mesh_lcd_server_cb_param_t**

Large Composition Data Server model callback parameters

Public Members

esp_ble_mesh_model_t ***model**

Pointer to the server model structure

esp_ble_mesh_msg_ctx_t **ctx**

Context of the received message

esp_ble_mesh_lcd_server_cb_value_t **value**

Value of the received configuration messages

Macros

ESP_BLE_MESH_MODEL_OP_LARGE_COMP_DATA_GET

ESP_BLE_MESH_MODEL_OP_LARGE_COMP_DATA_STATUS

ESP_BLE_MESH_MODEL_OP_MODELS_METADATA_GET

ESP_BLE_MESH_MODEL_OP_MODELS_METADATA_STATUS

ESP_BLE_MESH_MODEL_LCD_SRV(srv_data)

Define a new Large Composition Data Server model.

Note: If supported, the model shall be supported by a primary element and shall not be supported by any secondary elements.

Parameters

- **srv_data** -- Pointer to a unique Large Composition Data Server model user_data.

Returns New Large Composition Data Server model instance.

ESP_BLE_MESH_MODEL_LCD_CLI(cli_data)

Define a new Large Composition Data Client model.

Note: If supported, the model shall be supported by the primary element and shall not be supported by any secondary elements.

Parameters

- **cli_data** -- Pointer to a unique Large Composition Data Client model user_data.

Returns New Large Composition Data Client model instance.

Type Definitions

typedef void (***esp_ble_mesh_lcd_client_cb_t**)(*esp_ble_mesh_lcd_client_cb_event_t* event, *esp_ble_mesh_lcd_client_cb_param_t* *param)

Large Composition Data client and server model functions.

Large Composition Data Client model callback function type

Param event Event type
Param param Pointer to callback parameter

typedef void (***esp_ble_mesh_lcd_server_cb_t**)(*esp_ble_mesh_lcd_server_cb_event_t* event, *esp_ble_mesh_lcd_server_cb_param_t* *param)

Large Composition Data Server model callback function type.

Param event Event type
Param param Pointer to callback parameter

Enumerations

enum **esp_ble_mesh_lcd_client_cb_event_t**

This enum value is the event of Large Composition Data Client model

Values:

enumerator **ESP_BLE_MESH_LCD_CLIENT_SEND_COMP_EVT**

enumerator **ESP_BLE_MESH_LCD_CLIENT_SEND_TIMEOUT_EVT**

enumerator **ESP_BLE_MESH_LCD_CLIENT_RECV_RSP_EVT**

enumerator **ESP_BLE_MESH_LCD_CLIENT_RECV_PUB_EVT**

enumerator **ESP_BLE_MESH_LCD_CLIENT_EVT_MAX**

enum **esp_ble_mesh_lcd_server_cb_event_t**

This enum value is the event of Large Composition Data Server model

Values:

enumerator **ESP_BLE_MESH_LCD_SERVER_STATE_CHANGE_EVT**

enumerator **ESP_BLE_MESH_LCD_SERVER_EVT_MAX**

Composition and Metadata

Header File

- components/bt/esp_ble_mesh/v1.1/api/core/include/esp_ble_mesh_cm_data_api.h
- This header file can be included with:

```
#include "esp_ble_mesh_cm_data_api.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_ble_mesh_comp_1_register**(const *esp_ble_mesh_comp_1_t* *comp)

Register Composition Data Page 1.

Parameters **comp** -- **[in]** Pointer to Composition Data Page 1.
Returns ESP_OK on success or error code otherwise.

esp_err_t **esp_ble_mesh_models_metadata_register**(const *esp_ble_mesh_models_metadata_t* *metadata, uint8_t metadata_page)

Register Models Metadata Page 0 or 128.

Parameters
- **metadata** -- **[in]** Pointer to Models Metadata Page 0 or 128.
- **metadata_page** -- **[in]** Models Metadata Page number, i.e. 0 or 128.

Returns ESP_OK on success or error code otherwise.

Structures

struct **esp_ble_mesh_extended_model_item_t**

Format of Extended Model Item

Public Members

uint8_t **element_offset**

Element address modifier, in the range -4 to 3. See above.

uint8_t **model_item_idx**

Model Index, in the range 0 to 31

Model index, in the range 0 to 255

int8_t **element_offset**

Element address modifier, in the range -128 to 127

struct *esp_ble_mesh_extended_model_item_t*::[anonymous]::[anonymous] **long_fmt**

Extended Model Item long format Extended Model Item long format

union *esp_ble_mesh_extended_model_item_t*::[anonymous] **[anonymous]**

Union of Extended Model Item

struct **esp_ble_mesh_model_item_t**

Format of Model Item

Public Members

uint8_t **corresponding_present**

Corresponding_Group_ID field indicator

uint8_t **format**

Format of Extended_Model_Items indicator

uint8_t **extended_items_count**

Number of Extended Model Items in the Extended_Model_Items field

uint8_t **corresponding_group_id**

Corresponding group identifier

esp_ble_mesh_extended_model_item_t *const **extended_model_items**

List of Extended Model Items

struct **esp_ble_mesh_comp_1_elem_t**

Format of element of Composition Data Page 1

Public Members

const uint8_t **num_s**

A count of SIG Models Items in this element

const uint8_t **num_v**

A count of Vendor Models Items in this element

esp_ble_mesh_model_item_t *const **model_items_s**

A sequence of "num_s" SIG Model Items

esp_ble_mesh_model_item_t *const **model_items_v**

A sequence of "num_v" Vendor Model Items

struct **esp_ble_mesh_comp_1_t**

Format of Composition Data Page 1

Public Members

size_t **element_count**

Element count

esp_ble_mesh_comp_1_elem_t ***elements**

A sequence of element descriptions

struct **esp_ble_mesh_metadata_entry_t**

Format of Metadata entry

Public Members

uint16_t **metadata_len**

Size of the Metadata field

uint16_t **metadata_id**

Bluetooth assigned number for the Metadata Identifier

const uint8_t ***metadata**

Model’ s metadata

struct **esp_ble_mesh_metadata_item_t**

Format of Metadata item

Public Members

uint16_t **model_id**

Model ID

uint16_t **company_id**

Company ID

struct *esp_ble_mesh_metadata_item_t*::[anonymous]::[anonymous] **vnd**

Vendor model identifier Vendor model identifier

union *esp_ble_mesh_metadata_item_t*::[anonymous] **[anonymous]**

Union of model ID

uint8_t **metadata_entries_num**

Number of metadata entries

esp_ble_mesh_metadata_entry_t *const **metadata_entries**

List of model’ s metadata

struct **esp_ble_mesh_metadata_elem_t**

Format of Metadata element of Models Metadata Page 0/128

Public Members

const uint8_t **items_num_s**

Number of metadata items for SIG models in the element

const uint8_t **items_num_v**

Number of metadata items for Vendor models in the element

esp_ble_mesh_metadata_item_t *const **metadata_items_s**

List of metadata items for SIG models in the element

esp_ble_mesh_metadata_item_t *const **metadata_items_v**

List of metadata items for Vendor models in the element

struct **esp_ble_mesh_models_metadata_t**

Format of the Models Metadata Page 0/128

Public Members

size_t **element_count**

Element count

esp_ble_mesh_metadata_elem_t ***elements**

List of metadata for models for each element

Macros

ESP_BLE_MESH_MODEL_ITEM_SHORT

< Definitions of the format of Extended_Model_Items indicator

ESP_BLE_MESH_MODEL_ITEM_LONG

2.3.5 NimBLE-based Host APIs

Overview

Apache MyNewt NimBLE is a highly configurable and Bluetooth® SIG qualifiable Bluetooth Low Energy (Bluetooth LE) stack providing both host and controller functionalities. ESP-IDF supports NimBLE host stack which is specifically ported for ESP32 platform and FreeRTOS. The underlying controller is still the same (as in case of Bluedroid) providing VHCI interface. Refer to NimBLE user guide for a complete list of features and additional information on NimBLE stack. Most features of NimBLE including Bluetooth Low Energy Mesh are supported by ESP-IDF. The porting layer is kept cleaner by maintaining all the existing APIs of NimBLE along with a single ESP-NimBLE API for initialization, making it simpler for the application developers.

Architecture

Currently, NimBLE host and controller support different transports such as UART and RAM between them. However, RAM transport cannot be used as is in case of ESP as ESP controller supports VHCI interface and buffering schemes used by NimBLE host is incompatible with that used by ESP controller. Therefore, a new transport between NimBLE host and ESP controller has been added. This is depicted in the figure below. This layer is responsible for maintaining pool of transport buffers and formatting buffers exchanges between host and controller as per the requirements.

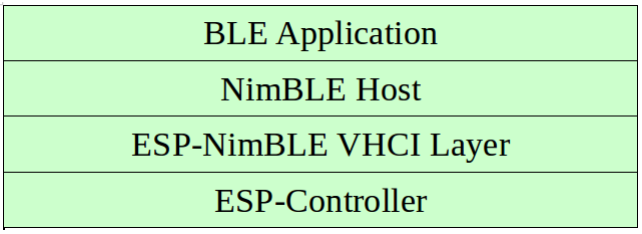


Fig. 1: ESP NimBLE Stack

Threading Model

The NimBLE host can run inside the application thread or can have its own independent thread. This flexibility is inherently provided by NimBLE design. By default, a thread is spawned by the porting function `nimble_port_freertos_init`. This behavior can be changed by overriding the same function. For Bluetooth Low Energy Mesh, additional thread (advertising thread) is used which keeps on feeding advertisement events to the main thread.

Programming Sequence

To begin with, make sure that the NimBLE stack is enabled from menuconfig *choose NimBLE for the Bluetooth host*.

Typical programming sequence with NimBLE stack consists of the following steps:

- Initialize NVS flash using *`nvs_flash_init()`* API. This is because ESP controller uses NVS during initialization.
- Initialize the host and controller stack using `nimble_port_init`.
- Initialize the required NimBLE host configuration parameters and callbacks
- Perform application specific tasks/initialization
- Run the thread for host stack using `nimble_port_freertos_init`

This documentation does not cover NimBLE APIs. Refer to NimBLE tutorial for more details on the programming sequence/NimBLE APIs for different scenarios.

API Reference

Header File

- components/bt/host/nimble/esp-hci/include/esp_nimble_hci.h
- This header file can be included with:

```
#include "esp_nimble_hci.h"
```

- This header file is a part of the API provided by the `bt` component. To declare that your component depends on `bt`, add the following to your CMakeLists.txt:

```
REQUIRES bt
```

or

```
PRIV_REQUIRES bt
```

Functions

esp_err_t **esp_nimble_hci_init** (void)

Initialize VHCI transport layer between NimBLE Host and ESP Bluetooth controller.

This function initializes the transport buffers to be exchanged between NimBLE host and ESP controller. It also registers required host callbacks with the controller.

Returns

- ESP_OK if the initialization is successful
- Appropriate error code from esp_err_t in case of an error

esp_err_t **esp_nimble_hci_deinit**(void)

Deinitialize VHCI transport layer between NimBLE Host and ESP Bluetooth controller.

Note: This function should be called after the NimBLE host is deinitialized.

Returns

- ESP_OK if the deinitialization is successful
- Appropriate error codes from esp_err_t in case of an error

Macros

BLE_HCI_UART_H4_NONE

BLE_HCI_UART_H4_CMD

BLE_HCI_UART_H4_ACL

BLE_HCI_UART_H4_SCO

BLE_HCI_UART_H4_EVT

ESP-IDF currently supports two host stacks. The Bluedroid based stack (default) supports classic Bluetooth as well as Bluetooth Low Energy (Bluetooth LE). On the other hand, Apache NimBLE based stack is Bluetooth Low Energy only. For users to make a choice:

- For usecases involving classic Bluetooth as well as Bluetooth Low Energy, Bluedroid should be used.
- For Bluetooth Low Energy-only usecases, using NimBLE is recommended. It is less demanding in terms of code footprint and runtime memory, making it suitable for such scenarios.

Code examples for this API section are provided in the bluetooth/bluedroid directory of ESP-IDF examples.

The following examples contain detailed walkthroughs:

- GATT Client Example Walkthrough
- GATT Server Service Table Example Walkthrough
- GATT Server Example Walkthrough
- GATT Security Client Example Walkthrough
- GATT Security Server Example Walkthrough
- GATT Client Multi-connection Example Walkthrough

2.4 Error Codes Reference

This section lists various error code constants defined in ESP-IDF.

For general information about error codes in ESP-IDF, see *Error Handling*.

ESP_FAIL (-1): Generic esp_err_t code indicating failure

ESP_OK (0): esp_err_t value indicating success (no error)

ESP_ERR_NO_MEM **(0x101)**: Out of memory

ESP_ERR_INVALID_ARG **(0x102)**: Invalid argument

ESP_ERR_INVALID_STATE **(0x103)**: Invalid state

ESP_ERR_INVALID_SIZE **(0x104)**: Invalid size

ESP_ERR_NOT_FOUND **(0x105)**: Requested resource not found

ESP_ERR_NOT_SUPPORTED **(0x106)**: Operation or feature not supported

ESP_ERR_TIMEOUT **(0x107)**: Operation timed out

ESP_ERR_INVALID_RESPONSE **(0x108)**: Received response was invalid

ESP_ERR_INVALID_CRC **(0x109)**: CRC or checksum was invalid

ESP_ERR_INVALID_VERSION **(0x10a)**: Version was invalid

ESP_ERR_INVALID_MAC **(0x10b)**: MAC address was invalid

ESP_ERR_NOT_FINISHED **(0x10c)**: Operation has not fully completed

ESP_ERR_NOT_ALLOWED **(0x10d)**: Operation is not allowed

ESP_ERR_NVS_BASE **(0x1100)**: Starting number of error codes

ESP_ERR_NVS_NOT_INITIALIZED **(0x1101)**: The storage driver is not initialized

ESP_ERR_NVS_NOT_FOUND **(0x1102)**: A requested entry couldn't be found or namespace doesn't exist yet and mode is NVS_READONLY

ESP_ERR_NVS_TYPE_MISMATCH **(0x1103)**: The type of set or get operation doesn't match the type of value stored in NVS

ESP_ERR_NVS_READ_ONLY **(0x1104)**: Storage handle was opened as read only

ESP_ERR_NVS_NOT_ENOUGH_SPACE **(0x1105)**: There is not enough space in the underlying storage to save the value

ESP_ERR_NVS_INVALID_NAME **(0x1106)**: Namespace name doesn't satisfy constraints

ESP_ERR_NVS_INVALID_HANDLE **(0x1107)**: Handle has been closed or is NULL

ESP_ERR_NVS_REMOVE_FAILED **(0x1108)**: The value wasn't updated because flash write operation has failed. The value was written however, and update will be finished after re-initialization of nvs, provided that flash operation doesn't fail again.

ESP_ERR_NVS_KEY_TOO_LONG **(0x1109)**: Key name is too long

ESP_ERR_NVS_PAGE_FULL **(0x110a)**: Internal error; never returned by nvs API functions

ESP_ERR_NVS_INVALID_STATE **(0x110b)**: NVS is in an inconsistent state due to a previous error. Call nvs_flash_init and nvs_open again, then retry.

ESP_ERR_NVS_INVALID_LENGTH **(0x110c)**: String or blob length is not sufficient to store data

ESP_ERR_NVS_NO_FREE_PAGES **(0x110d)**: NVS partition doesn't contain any empty pages. This may happen if NVS partition was truncated. Erase the whole partition and call nvs_flash_init again.

ESP_ERR_NVS_VALUE_TOO_LONG **(0x110e)**: Value doesn't fit into the entry or string or blob length is longer than supported by the implementation

ESP_ERR_NVS_PART_NOT_FOUND **(0x110f)**: Partition with specified name is not found in the partition table

ESP_ERR_NVS_NEW_VERSION_FOUND **(0x1110)**: NVS partition contains data in new format and cannot be recognized by this version of code

ESP_ERR_NVS_XTS_ENCR_FAILED **(0x1111)**: XTS encryption failed while writing NVS entry

ESP_ERR_NVS_XTS_DECR_FAILED **(0x1112)**: XTS decryption failed while reading NVS entry

ESP_ERR_NVS_XTS_CFG_FAILED **(0x1113)**: XTS configuration setting failed

ESP_ERR_NVS_XTS_CFG_NOT_FOUND **(0x1114)**: XTS configuration not found

ESP_ERR_NVS_ENCR_NOT_SUPPORTED **(0x1115)**: NVS encryption is not supported in this version

ESP_ERR_NVS_KEYS_NOT_INITIALIZED **(0x1116)**: NVS key partition is uninitialized

ESP_ERR_NVS_CORRUPT_KEY_PART **(0x1117)**: NVS key partition is corrupt

ESP_ERR_NVS_CONTENT_DIFFERS **(0x1118)**: Internal error; never returned by nvs API functions. NVS key is different in comparison

ESP_ERR_NVS_WRONG_ENCRYPTION **(0x1119)**: NVS partition is marked as encrypted with generic flash encryption. This is forbidden since the NVS encryption works differently.

ESP_ERR_ULP_BASE **(0x1200)**: Offset for ULP-related error codes

ESP_ERR_ULP_SIZE_TOO_BIG **(0x1201)**: Program doesn't fit into RTC memory reserved for the ULP

ESP_ERR_ULP_INVALID_LOAD_ADDR **(0x1202)**: Load address is outside of RTC memory reserved for the ULP

ESP_ERR_ULP_DUPLICATE_LABEL **(0x1203)**: More than one label with the same number was defined

ESP_ERR_ULP_UNDEFINED_LABEL **(0x1204)**: Branch instructions references an undefined label

ESP_ERR_ULP_BRANCH_OUT_OF_RANGE **(0x1205)**: Branch target is out of range of B instruction (try replacing with BX)

ESP_ERR_OTA_BASE **(0x1500)**: Base error code for ota_ops api

ESP_ERR_OTA_PARTITION_CONFLICT **(0x1501)**: Error if request was to write or erase the current running partition

ESP_ERR_OTA_SELECT_INFO_INVALID **(0x1502)**: Error if OTA data partition contains invalid content

ESP_ERR_OTA_VALIDATE_FAILED **(0x1503)**: Error if OTA app image is invalid

ESP_ERR_OTA_SMALL_SEC_VER **(0x1504)**: Error if the firmware has a secure version less than the running firmware.

ESP_ERR_OTA_ROLLBACK_FAILED **(0x1505)**: Error if flash does not have valid firmware in passive partition and hence rollback is not possible

ESP_ERR_OTA_ROLLBACK_INVALID_STATE **(0x1506)**: Error if current active firmware is still marked in pending validation state (ESP_OTA_IMG_PENDING_VERIFY), essentially first boot of firmware image post upgrade and hence firmware upgrade is not possible

ESP_ERR_EFUSE **(0x1600)**: Base error code for efuse api.

ESP_OK_EFUSE_CNT **(0x1601)**: OK the required number of bits is set.

ESP_ERR_EFUSE_CNT_IS_FULL **(0x1602)**: Error field is full.

ESP_ERR_EFUSE_REPEATED_PROG **(0x1603)**: Error repeated programming of programmed bits is strictly forbidden.

ESP_ERR_CODING **(0x1604)**: Error while a encoding operation.

ESP_ERR_NOT_ENOUGH_UNUSED_KEY_BLOCKS **(0x1605)**: Error not enough unused key blocks available

ESP_ERR_DAMAGED_READING **(0x1606)**: Error. Burn or reset was done during a reading operation leads to damage read data. This error is internal to the efuse component and not returned by any public API.

ESP_ERR_IMAGE_BASE **(0x2000)**

ESP_ERR_IMAGE_FLASH_FAIL **(0x2001)**

ESP_ERR_IMAGE_INVALID **(0x2002)**

ESP_ERR_WIFI_BASE **(0x3000)**: Starting number of WiFi error codes

ESP_ERR_WIFI_NOT_INIT **(0x3001)**: WiFi driver was not installed by esp_wifi_init

ESP_ERR_WIFI_NOT_STARTED **(0x3002)**: WiFi driver was not started by esp_wifi_start

ESP_ERR_WIFI_NOT_STOPPED **(0x3003)**: WiFi driver was not stopped by esp_wifi_stop

ESP_ERR_WIFI_IF **(0x3004)**: WiFi interface error

ESP_ERR_WIFI_MODE **(0x3005)**: WiFi mode error

ESP_ERR_WIFI_STATE **(0x3006)**: WiFi internal state error

ESP_ERR_WIFI_CONN **(0x3007)**: WiFi internal control block of station or soft-AP error

ESP_ERR_WIFI_NVS **(0x3008)**: WiFi internal NVS module error

ESP_ERR_WIFI_MAC **(0x3009)**: MAC address is invalid

ESP_ERR_WIFI_SSID **(0x300a)**: SSID is invalid

ESP_ERR_WIFI_PASSWORD **(0x300b)**: Password is invalid

ESP_ERR_WIFI_TIMEOUT **(0x300c)**: Timeout error

ESP_ERR_WIFI_WAKE_FAIL **(0x300d)**: WiFi is in sleep state(RF closed) and wakeup fail

ESP_ERR_WIFI_WOULD_BLOCK **(0x300e)**: The caller would block

ESP_ERR_WIFI_NOT_CONNECT **(0x300f)**: Station still in disconnect status

ESP_ERR_WIFI_POST **(0x3012)**: Failed to post the event to WiFi task

ESP_ERR_WIFI_INIT_STATE **(0x3013)**: Invalid WiFi state when init/deinit is called

ESP_ERR_WIFI_STOP_STATE **(0x3014)**: Returned when WiFi is stopping

ESP_ERR_WIFI_NOT_ASSOC **(0x3015)**: The WiFi connection is not associated

ESP_ERR_WIFI_TX_DISALLOW **(0x3016)**: The WiFi TX is disallowed

ESP_ERR_WIFI_TWT_FULL **(0x3017)**: no available flow id

ESP_ERR_WIFI_TWT_SETUP_TIMEOUT **(0x3018)**: Timeout of receiving twt setup response frame, timeout times can be set during twt setup

ESP_ERR_WIFI_TWT_SETUP_TXFAIL **(0x3019)**: TWT setup frame tx failed

ESP_ERR_WIFI_TWT_SETUP_REJECT **(0x301a)**: The twt setup request was rejected by the AP

ESP_ERR_WIFI_DISCARD **(0x301b)**: Discard frame

ESP_ERR_WIFI_ROC_IN_PROGRESS **(0x301c)**: ROC op is in progress

ESP_ERR_WIFI_REGISTRAR **(0x3033)**: WPS registrar is not supported

ESP_ERR_WIFI_WPS_TYPE **(0x3034)**: WPS type error

ESP_ERR_WIFI_WPS_SM **(0x3035)**: WPS state machine is not initialized

ESP_ERR_ESPNOW_BASE **(0x3064)**: ESPNOW error number base.

ESP_ERR_ESPNOW_NOT_INIT **(0x3065)**: ESPNOW is not initialized.

ESP_ERR_ESPNOW_ARG **(0x3066)**: Invalid argument

ESP_ERR_ESPNOW_NO_MEM **(0x3067)**: Out of memory

ESP_ERR_ESPNOW_FULL **(0x3068)**: ESPNOW peer list is full

ESP_ERR_ESPNOW_NOT_FOUND **(0x3069)**: ESPNOW peer is not found

ESP_ERR_ESPNOW_INTERNAL **(0x306a)**: Internal error

ESP_ERR_ESPNOW_EXIST **(0x306b)**: ESPNOW peer has existed

ESP_ERR_ESPNOW_IF **(0x306c)**: Interface error

ESP_ERR_ESPNOW_CHAN **(0x306d)**: Channel error

ESP_ERR_DPP_FAILURE **(0x3097)**: Generic failure during DPP Operation

ESP_ERR_DPP_TX_FAILURE **(0x3098)**: DPP Frame Tx failed OR not Acked

ESP_ERR_DPP_INVALID_ATTR **(0x3099)**: Encountered invalid DPP Attribute

ESP_ERR_DPP_AUTH_TIMEOUT **(0x309a)**: DPP Auth response was not recieved in time

ESP_ERR_MESH_BASE **(0x4000)**: Starting number of MESH error codes

ESP_ERR_MESH_WIFI_NOT_START **(0x4001)**

ESP_ERR_MESH_NOT_INIT **(0x4002)**

ESP_ERR_MESH_NOT_CONFIG **(0x4003)**

ESP_ERR_MESH_NOT_START **(0x4004)**

ESP_ERR_MESH_NOT_SUPPORT **(0x4005)**

ESP_ERR_MESH_NOT_ALLOWED **(0x4006)**

ESP_ERR_MESH_NO_MEMORY **(0x4007)**

ESP_ERR_MESH_ARGUMENT **(0x4008)**

ESP_ERR_MESH_EXCEED_MTU **(0x4009)**

ESP_ERR_MESH_TIMEOUT **(0x400a)**

ESP_ERR_MESH_DISCONNECTED **(0x400b)**

ESP_ERR_MESH_QUEUE_FAIL **(0x400c)**

ESP_ERR_MESH_QUEUE_FULL **(0x400d)**

ESP_ERR_MESH_NO_PARENT_FOUND **(0x400e)**

ESP_ERR_MESH_NO_ROUTE_FOUND **(0x400f)**

ESP_ERR_MESH_OPTION_NULL **(0x4010)**

ESP_ERR_MESH_OPTION_UNKNOWN **(0x4011)**

ESP_ERR_MESH_XON_NO_WINDOW **(0x4012)**

ESP_ERR_MESH_INTERFACE **(0x4013)**

ESP_ERR_MESH_DISCARD_DUPLICATE **(0x4014)**

ESP_ERR_MESH_DISCARD **(0x4015)**

ESP_ERR_MESH_VOTING **(0x4016)**

ESP_ERR_MESH_XMIT **(0x4017)**

ESP_ERR_MESH_QUEUE_READ **(0x4018)**

ESP_ERR_MESH_PS **(0x4019)**

ESP_ERR_MESH_RECV_RELEASE **(0x401a)**

ESP_ERR_ESP_NETIF_BASE **(0x5000)**

ESP_ERR_ESP_NETIF_INVALID_PARAMS **(0x5001)**

ESP_ERR_ESP_NETIF_IF_NOT_READY **(0x5002)**

ESP_ERR_ESP_NETIF_DHCPC_START_FAILED **(0x5003)**

ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED **(0x5004)**

ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED **(0x5005)**

ESP_ERR_ESP_NETIF_NO_MEM **(0x5006)**

ESP_ERR_ESP_NETIF_DHCP_NOT_STOPPED **(0x5007)**

ESP_ERR_ESP_NETIF_DRIVER_ATTACH_FAILED **(0x5008)**

ESP_ERR_ESP_NETIF_INIT_FAILED **(0x5009)**

ESP_ERR_ESP_NETIF_DNS_NOT_CONFIGURED **(0x500a)**

ESP_ERR_ESP_NETIF_MLD6_FAILED **(0x500b)**

ESP_ERR_ESP_NETIF_IP6_ADDR_FAILED **(0x500c)**

ESP_ERR_ESP_NETIF_DHCPS_START_FAILED **(0x500d)**

ESP_ERR_FLASH_BASE **(0x6000)**: Starting number of flash error codes

ESP_ERR_FLASH_OP_FAIL **(0x6001)**

ESP_ERR_FLASH_OP_TIMEOUT **(0x6002)**

ESP_ERR_FLASH_NOT_INITIALISED **(0x6003)**

ESP_ERR_FLASH_UNSUPPORTED_HOST **(0x6004)**

ESP_ERR_FLASH_UNSUPPORTED_CHIP **(0x6005)**

ESP_ERR_FLASH_PROTECTED **(0x6006)**

ESP_ERR_HTTP_BASE **(0x7000)**: Starting number of HTTP error codes

ESP_ERR_HTTP_MAX_REDIRECT **(0x7001)**: The error exceeds the number of HTTP redirects

ESP_ERR_HTTP_CONNECT **(0x7002)**: Error open the HTTP connection

ESP_ERR_HTTP_WRITE_DATA **(0x7003)**: Error write HTTP data

ESP_ERR_HTTP_FETCH_HEADER **(0x7004)**: Error read HTTP header from server

ESP_ERR_HTTP_INVALID_TRANSPORT **(0x7005)**: There are no transport support for the input scheme

ESP_ERR_HTTP_CONNECTING **(0x7006)**: HTTP connection hasn't been established yet

ESP_ERR_HTTP_EAGAIN **(0x7007)**: Mapping of errno EAGAIN to esp_err_t

ESP_ERR_HTTP_CONNECTION_CLOSED **(0x7008)**: Read FIN from peer and the connection closed

ESP_ERR_ESP_TLS_BASE **(0x8000)**: Starting number of ESP-TLS error codes

ESP_ERR_ESP_TLS_CANNOT_RESOLVE_HOSTNAME **(0x8001)**: Error if hostname couldn't be resolved upon tls connection

ESP_ERR_ESP_TLS_CANNOT_CREATE_SOCKET **(0x8002)**: Failed to create socket

ESP_ERR_ESP_TLS_UNSUPPORTED_PROTOCOL_FAMILY **(0x8003)**: Unsupported protocol family

ESP_ERR_ESP_TLS_FAILED_CONNECT_TO_HOST **(0x8004)**: Failed to connect to host

ESP_ERR_ESP_TLS_SOCKET_SETOPT_FAILED **(0x8005)**: failed to set/get socket option

ESP_ERR_ESP_TLS_CONNECTION_TIMEOUT **(0x8006)**: new connection in esp_tls_low_level_conn connection timeouted

ESP_ERR_ESP_TLS_SE_FAILED **(0x8007)**

ESP_ERR_ESP_TLS_TCP_CLOSED_FIN **(0x8008)**

ESP_ERR_MBEDTLS_CERT_PARTLY_OK **(0x8010)**: mbedtls parse certificates was partly successful

ESP_ERR_MBEDTLS_CTR_DRBG_SEED_FAILED **(0x8011)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_SET_HOSTNAME_FAILED **(0x8012)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONFIG_DEFAULTS_FAILED **(0x8013)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONF_ALPN_PROTOCOLS_FAILED **(0x8014)**: mbedtls api returned error

ESP_ERR_MBEDTLS_X509_CRT_PARSE_FAILED **(0x8015)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_CONF_OWN_CERT_FAILED **(0x8016)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_SETUP_FAILED **(0x8017)**: mbedtls api returned error

ESP_ERR_MBEDTLS_SSL_WRITE_FAILED **(0x8018)**: mbedtls api returned error

ESP_ERR_MBEDTLS_PK_PARSE_KEY_FAILED **(0x8019)**: mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_HANDSHAKE_FAILED **(0x801a)**: mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_CONF_PSK_FAILED **(0x801b)**: mbedtls api returned failed

ESP_ERR_MBEDTLS_SSL_TICKET_SETUP_FAILED **(0x801c)**: mbedtls api returned failed

ESP_ERR_WOLFSSL_SSL_SET_HOSTNAME_FAILED **(0x8031)**: wolfSSL api returned error

ESP_ERR_WOLFSSL_SSL_CONF_ALPN_PROTOCOLS_FAILED **(0x8032)**: wolfSSL api returned error

ESP_ERR_WOLFSSL_CERT_VERIFY_SETUP_FAILED **(0x8033)**: wolfSSL api returned error

ESP_ERR_WOLFSSL_KEY_VERIFY_SETUP_FAILED **(0x8034)**: wolfSSL api returned error

ESP_ERR_WOLFSSL_SSL_HANDSHAKE_FAILED **(0x8035)**: wolfSSL api returned failed

ESP_ERR_WOLFSSL_CTX_SETUP_FAILED **(0x8036)**: wolfSSL api returned failed

ESP_ERR_WOLFSSL_SSL_SETUP_FAILED **(0x8037)**: wolfSSL api returned failed

ESP_ERR_WOLFSSL_SSL_WRITE_FAILED **(0x8038)**: wolfSSL api returned failed

ESP_ERR_HTTPS_OTA_BASE **(0x9000)**

ESP_ERR_HTTPS_OTA_IN_PROGRESS **(0x9001)**

ESP_ERR_PING_BASE **(0xa000)**

ESP_ERR_PING_INVALID_PARAMS **(0xa001)**

ESP_ERR_PING_NO_MEM **(0xa002)**

ESP_ERR_HTTPD_BASE **(0xb000)**: Starting number of HTTPD error codes

ESP_ERR_HTTPD_HANDLERS_FULL **(0xb001)**: All slots for registering URI handlers have been consumed

ESP_ERR_HTTPD_HANDLER_EXISTS **(0xb002)**: URI handler with same method and target URI already registered

ESP_ERR_HTTPD_INVALID_REQ **(0xb003)**: Invalid request pointer

ESP_ERR_HTTPD_RESULT_TRUNC **(0xb004)**: Result string truncated

ESP_ERR_HTTPD_RESP_HDR **(0xb005)**: Response header field larger than supported

ESP_ERR_HTTPD_RESP_SEND **(0xb006)**: Error occured while sending response packet

ESP_ERR_HTTPD_ALLOC_MEM **(0xb007)**: Failed to dynamically allocate memory for resource

ESP_ERR_HTTPD_TASK **(0xb008)**: Failed to launch server task/thread

ESP_ERR_HW_CRYPTO_BASE **(0xc000)**: Starting number of HW cryptography module error codes

ESP_ERR_HW_CRYPTO_DS_HMAC_FAIL **(0xc001)**: HMAC peripheral problem

ESP_ERR_HW_CRYPTO_DS_INVALID_KEY **(0xc002)**

ESP_ERR_HW_CRYPTO_DS_INVALID_DIGEST **(0xc004)**

ESP_ERR_HW_CRYPTO_DS_INVALID_PADDING **(0xc005)**

ESP_ERR_MEMPROT_BASE **(0xd000)**: Starting number of Memory Protection API error codes

ESP_ERR_MEMPROT_MEMORY_TYPE_INVALID **(0xd001)**

ESP_ERR_MEMPROT_SPLIT_ADDR_INVALID **(0xd002)**

ESP_ERR_MEMPROT_SPLIT_ADDR_OUT_OF_RANGE **(0xd003)**

ESP_ERR_MEMPROT_SPLIT_ADDR_UNALIGNED **(0xd004)**

ESP_ERR_MEMPROT_UNIMGMT_BLOCK_INVALID **(0xd005)**

ESP_ERR_MEMPROT_WORLD_INVALID **(0xd006)**

ESP_ERR_MEMPROT_AREA_INVALID **(0xd007)**

ESP_ERR_MEMPROT_CPUID_INVALID **(0xd008)**

ESP_ERR_TCP_TRANSPORT_BASE **(0xe000)**: Starting number of TCP Transport error codes

ESP_ERR_TCP_TRANSPORT_CONNECTION_TIMEOUT **(0xe001)**: Connection has timed out

ESP_ERR_TCP_TRANSPORT_CONNECTION_CLOSED_BY_FIN **(0xe002)**: Read FIN from peer and the connection has closed (in a clean way)

ESP_ERR_TCP_TRANSPORT_CONNECTION_FAILED **(0xe003)**: Failed to connect to the peer

ESP_ERR_TCP_TRANSPORT_NO_MEM **(0xe004)**: Memory allocation failed

ESP_ERR_NVS_SEC_BASE **(0xf000)**: Starting number of error codes

ESP_ERR_NVS_SEC_HMAC_KEY_NOT_FOUND **(0xf001)**: HMAC Key required to generate the NVS encryption keys not found

ESP_ERR_NVS_SEC_HMAC_KEY_BLK_ALREADY_USED **(0xf002)**: Provided eFuse block for HMAC key generation is already in use

ESP_ERR_NVS_SEC_HMAC_KEY_GENERATION_FAILED **(0xf003)**: Failed to generate/write the HMAC key to eFuse

ESP_ERR_NVS_SEC_HMAC_XTS_KEYS_DERIV_FAILED **(0xf004)**: Failed to derive the NVS encryption keys based on the HMAC-based scheme

2.5 Networking APIs

2.5.1 Wi-Fi

ESP-NOW

Overview ESP-NOW is a kind of connectionless Wi-Fi communication protocol that is defined by Espressif. In ESP-NOW, application data is encapsulated in a vendor-specific action frame and then transmitted from one Wi-Fi device to another without connection.

CTR with CBC-MAC Protocol (CCMP) is used to protect the action frame for security. ESP-NOW is widely used in smart light, remote controlling, sensor, etc.

Frame Format ESP-NOW uses a vendor-specific action frame to transmit ESP-NOW data. The default ESP-NOW bit rate is 1 Mbps. The format of the vendor-specific action frame is as follows:

```
--------------------------------------------------------------------------------
↪-------------------------
| MAC Header | Category Code | Organization Identifier | Random Values | Vendor␣
↪Specific Content |   FCS   |
--------------------------------------------------------------------------------
↪-------------------------
  24 bytes         1 byte              3 bytes               4 bytes             7-
↪257 bytes        4 bytes
```

- Category Code: The Category Code field is set to the value (127) indicating the vendor-specific category.
- Organization Identifier: The Organization Identifier contains a unique identifier (0x18fe34), which is the first three bytes of MAC address applied by Espressif.
- Random Value: The Random Value filed is used to prevents relay attacks.

- Vendor Specific Content: The Vendor Specific Content contains vendor-specific fields as follows:

Element ID	Length	Organization Identifier	Type	Version	Body
1 byte	1 byte	3 bytes	1 byte	1 byte	0-250 bytes

- Element ID: The Element ID field is set to the value (221), indicating the vendor-specific element.
- Length: The length is the total length of Organization Identifier, Type, Version and Body.
- Organization Identifier: The Organization Identifier contains a unique identifier (0x18fe34), which is the first three bytes of MAC address applied by Espressif.
- Type: The Type field is set to the value (4) indicating ESP-NOW.
- Version: The Version field is set to the version of ESP-NOW.
- Body: The Body contains the ESP-NOW data.

As ESP-NOW is connectionless, the MAC header is a little different from that of standard frames. The FromDS and ToDS bits of FrameControl field are both 0. The first address field is set to the destination address. The second address field is set to the source address. The third address field is set to broadcast address (0xff:0xff:0xff:0xff:0xff:0xff).

Security ESP-NOW uses the CCMP method, which is described in IEEE Std. 802.11-2012, to protect the vendor-specific action frame. The Wi-Fi device maintains a Primary Master Key (PMK) and several Local Master Keys (LMKs, each paired device has one LMK). The lengths of both PMK and LMK are 16 bytes.

- PMK is used to encrypt LMK with the AES-128 algorithm. Call `esp_now_set_pmk()` to set PMK. If PMK is not set, a default PMK will be used.
- LMK of the paired device is used to encrypt the vendor-specific action frame with the CCMP method. If the LMK of the paired device is not set, the vendor-specific action frame will not be encrypted.

Encrypting multicast vendor-specific action frame is not supported.

Initialization and Deinitialization Call `esp_now_init()` to initialize ESP-NOW and `esp_now_deinit()` to de-initialize ESP-NOW. ESP-NOW data must be transmitted after Wi-Fi is started, so it is recommended to start Wi-Fi before initializing ESP-NOW and stop Wi-Fi after de-initializing ESP-NOW.

When `esp_now_deinit()` is called, all of the information of paired devices are deleted.

Add Paired Device Call `esp_now_add_peer()` to add the device to the paired device list before you send data to this device. If security is enabled, the LMK must be set. You can send ESP-NOW data via both the Station and the SoftAP interface. Make sure that the interface is enabled before sending ESP-NOW data.

The maximum number of paired devices is 20, and the paired encryption devices are no more than 17, the default is 7. If you want to change the number of paired encryption devices, set *CONFIG_ESP_WIFI_ESPNOW_MAX_ENCRYPT_NUM* in the Wi-Fi component configuration menu.

A device with a broadcast MAC address must be added before sending broadcast data. The range of the channel of paired devices is from 0 to 14. If the channel is set to 0, data will be sent on the current channel. Otherwise, the channel must be set as the channel that the local device is on.

Send ESP-NOW Data Call `esp_now_send()` to send ESP-NOW data and `esp_now_register_send_cb()` to register sending callback function. It will return *ESP_NOW_SEND_SUCCESS* in sending callback function if the data is received successfully on the MAC layer. Otherwise, it will return *ESP_NOW_SEND_FAIL*. Several reasons can lead to ESP-NOW fails to send data. For example, the destination device does not exist; the channels of the devices are not the same; the action frame is lost when transmitting on the air, etc. It is not guaranteed that application layer can receive the data. If necessary, send back ack data when receiving ESP-NOW data. If receiving ack data timeouts, retransmit the ESP-NOW data. A sequence number can also be assigned to ESP-NOW data to drop the duplicate data.

If there is a lot of ESP-NOW data to send, call `esp_now_send()` to send less than or equal to 250 bytes of data once a time. Note that too short interval between sending two ESP-NOW data may lead to disorder of sending

callback function. So, it is recommended that sending the next ESP-NOW data after the sending callback function of the previous sending has returned. The sending callback function runs from a high-priority Wi-Fi task. So, do not do lengthy operations in the callback function. Instead, post the necessary data to a queue and handle it from a lower priority task.

Receiving ESP-NOW Data Call *esp_now_register_recv_cb()* to register receiving callback function. Call the receiving callback function when receiving ESP-NOW. The receiving callback function also runs from the Wi-Fi task. So, do not do lengthy operations in the callback function. Instead, post the necessary data to a queue and handle it from a lower priority task.

Config ESP-NOW Rate Call *esp_wifi_config_espnow_rate()* to config ESP-NOW rate of specified interface. Make sure that the interface is enabled before config rate. This API should be called after *esp_wifi_start()*.

Config ESP-NOW Power-saving Parameter Sleep is supported only when ESP32-S3 is configured as station.

Call *esp_now_set_wake_window()* to configure Window for ESP-NOW RX at sleep. The default value is the maximum, which allowing RX all the time.

If Power-saving is needed for ESP-NOW, call *esp_wifi_connectionless_module_set_wake_interval()* to configure Interval as well.

Please refer to *connectionless module power save* to get more detail.

Application Examples

- Example of sending and receiving ESP-NOW data between two devices: wifi/espnow.
- For more application examples of how to use ESP-NOW, please visit ESP-NOW repository.

API Reference

Header File

- components/esp_wifi/include/esp_now.h
- This header file can be included with:

```
#include "esp_now.h"
```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_wifi
```

or

```
PRIV_REQUIRES esp_wifi
```

Functions

esp_err_t **esp_now_init** (void)

Initialize ESPNOW function.

Returns

- ESP_OK : succeed
- ESP_ERR_ESPNOW_INTERNAL : Internal error

esp_err_t **esp_now_deinit**(void)

De-initialize ESPNOW function.

Returns
- ESP_OK : succeed

esp_err_t **esp_now_get_version**(uint32_t *version)

Get the version of ESPNOW.

Parameters **version** -- ESPNOW version
Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_ARG : invalid argument

esp_err_t **esp_now_register_recv_cb**(*esp_now_recv_cb_t* cb)

Register callback function of receiving ESPNOW data.

Parameters **cb** -- callback function of receiving ESPNOW data
Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_INTERNAL : internal error

esp_err_t **esp_now_unregister_recv_cb**(void)

Unregister callback function of receiving ESPNOW data.

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized

esp_err_t **esp_now_register_send_cb**(*esp_now_send_cb_t* cb)

Register callback function of sending ESPNOW data.

Parameters **cb** -- callback function of sending ESPNOW data
Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_INTERNAL : internal error

esp_err_t **esp_now_unregister_send_cb**(void)

Unregister callback function of sending ESPNOW data.

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized

esp_err_t **esp_now_send**(const uint8_t *peer_addr, const uint8_t *data, size_t len)

Send ESPNOW data.

Attention 1. If peer_addr is not NULL, send data to the peer whose MAC address matches peer_addr
Attention 2. If peer_addr is NULL, send data to all of the peers that are added to the peer list
Attention 3. The maximum length of data must be less than ESP_NOW_MAX_DATA_LEN
Attention 4. The buffer pointed to by data argument does not need to be valid after esp_now_send returns

Parameters
- **peer_addr** -- peer MAC address
- **data** -- data to send
- **len** -- length of data

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized

- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_INTERNAL : internal error
- ESP_ERR_ESPNOW_NO_MEM : out of memory, when this happens, you can delay a while before sending the next data
- ESP_ERR_ESPNOW_NOT_FOUND : peer is not found
- ESP_ERR_ESPNOW_IF : current Wi-Fi interface doesn't match that of peer
- ESP_ERR_ESPNOW_CHAN: current Wi-Fi channel doesn't match that of peer

esp_err_t **esp_now_add_peer**(const *esp_now_peer_info_t* *peer)

Add a peer to peer list.

Parameters **peer** -- peer information

Returns

- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_FULL : peer list is full
- ESP_ERR_ESPNOW_NO_MEM : out of memory
- ESP_ERR_ESPNOW_EXIST : peer has existed

esp_err_t **esp_now_del_peer**(const uint8_t *peer_addr)

Delete a peer from peer list.

Parameters **peer_addr** -- peer MAC address

Returns

- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_NOT_FOUND : peer is not found

esp_err_t **esp_now_mod_peer**(const *esp_now_peer_info_t* *peer)

Modify a peer.

Parameters **peer** -- peer information

Returns

- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_FULL : peer list is full

esp_err_t **esp_wifi_config_espnow_rate**(wifi_interface_t ifx, wifi_phy_rate_t rate)

Config ESPNOW rate of specified interface.

Deprecated:

please use *esp_now_set_peer_rate_config()* instead.

Attention 1. This API should be called after esp_wifi_start().

Attention 2. This API only work when not use Wi-Fi 6 and *esp_now_set_peer_rate_config()* not called.

Parameters

- **ifx** -- Interface to be configured.
- **rate** -- Phy rate to be configured.

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_now_set_peer_rate_config** (const uint8_t *peer_addr, *esp_now_rate_config_t* *config)

Set ESPNOW rate config for each peer.

Attention 1. This API should be called after esp_wifi_start() and *esp_now_init()*.

Parameters
- **peer_addr** -- peer MAC address
- **config** -- rate config to be configured.

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_INTERNAL : internal error

esp_err_t **esp_now_get_peer** (const uint8_t *peer_addr, *esp_now_peer_info_t* *peer)

Get a peer whose MAC address matches peer_addr from peer list.

Parameters
- **peer_addr** -- peer MAC address
- **peer** -- peer information

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_NOT_FOUND : peer is not found

esp_err_t **esp_now_fetch_peer** (bool from_head, *esp_now_peer_info_t* *peer)

Fetch a peer from peer list. Only return the peer which address is unicast, for the multicast/broadcast address, the function will ignore and try to find the next in the peer list.

Parameters
- **from_head** -- fetch from head of list or not
- **peer** -- peer information

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument
- ESP_ERR_ESPNOW_NOT_FOUND : peer is not found

bool **esp_now_is_peer_exist** (const uint8_t *peer_addr)

Peer exists or not.

Parameters **peer_addr** -- peer MAC address

Returns
- true : peer exists
- false : peer not exists

esp_err_t **esp_now_get_peer_num** (*esp_now_peer_num_t* *num)

Get the number of peers.

Parameters **num** -- number of peers

Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument

esp_err_t **esp_now_set_pmk** (const uint8_t *pmk)

Set the primary master key.

Attention 1. primary master key is used to encrypt local master key

Parameters **pmk** -- primary master key
Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized
- ESP_ERR_ESPNOW_ARG : invalid argument

esp_err_t **esp_now_set_wake_window**(uint16_t window)

Set wake window for esp_now to wake up in interval unit.

Attention 1. This configuration could work at connected status. When ESP_WIFI_STA_DISCONNECTED_PM_ENABLE is enabled, this configuration could work at disconnected status.
Attention 2. Default value is the maximum.

Parameters **window** -- Milliseconds would the chip keep waked each interval, from 0 to 65535.
Returns
- ESP_OK : succeed
- ESP_ERR_ESPNOW_NOT_INIT : ESPNOW is not initialized

Structures

struct **esp_now_peer_info**

ESPNOW peer information parameters.

Public Members

uint8_t **peer_addr**[ESP_NOW_ETH_ALEN]

ESPNOW peer MAC address that is also the MAC address of station or softap

uint8_t **lmk**[ESP_NOW_KEY_LEN]

ESPNOW peer local master key that is used to encrypt data

uint8_t **channel**

Wi-Fi channel that peer uses to send/receive ESPNOW data. If the value is 0, use the current channel which station or softap is on. Otherwise, it must be set as the channel that station or softap is on.

wifi_interface_t **ifidx**

Wi-Fi interface that peer uses to send/receive ESPNOW data

bool **encrypt**

ESPNOW data that this peer sends/receives is encrypted or not

void ***priv**

ESPNOW peer private data

struct **esp_now_peer_num**

Number of ESPNOW peers which exist currently.

Public Members

int **total_num**

Total number of ESPNOW peers, maximum value is ESP_NOW_MAX_TOTAL_PEER_NUM

int **encrypt_num**

Number of encrypted ESPNOW peers, maximum value is ESP_NOW_MAX_ENCRYPT_PEER_NUM

struct **esp_now_recv_info**

ESPNOW packet information.

Public Members

uint8_t ***src_addr**

Source address of ESPNOW packet

uint8_t ***des_addr**

Destination address of ESPNOW packet

wifi_pkt_rx_ctrl_t ***rx_ctrl**

Rx control info of ESPNOW packet

struct **esp_now_rate_config**

ESPNOW rate config.

Public Members

wifi_phy_mode_t **phymode**

ESPNOW phymode of specified interface

wifi_phy_rate_t **rate**

ESPNOW rate of specified interface

bool **ersu**

ESPNOW using ersu send frame

bool **dcm**

ESPNOW using dcm rate to send frame

Macros

ESP_ERR_ESPNOW_BASE

ESPNOW error number base.

ESP_ERR_ESPNOW_NOT_INIT

ESPNOW is not initialized.

ESP_ERR_ESPNOW_ARG

Invalid argument

ESP_ERR_ESPNOW_NO_MEM

Out of memory

ESP_ERR_ESPNOW_FULL

ESPNOW peer list is full

ESP_ERR_ESPNOW_NOT_FOUND

ESPNOW peer is not found

ESP_ERR_ESPNOW_INTERNAL

Internal error

ESP_ERR_ESPNOW_EXIST

ESPNOW peer has existed

ESP_ERR_ESPNOW_IF

Interface error

ESP_ERR_ESPNOW_CHAN

Channel error

ESP_NOW_ETH_ALEN

Length of ESPNOW peer MAC address

ESP_NOW_KEY_LEN

Length of ESPNOW peer local master key

ESP_NOW_MAX_TOTAL_PEER_NUM

Maximum number of ESPNOW total peers

ESP_NOW_MAX_ENCRYPT_PEER_NUM

Maximum number of ESPNOW encrypted peers

ESP_NOW_MAX_DATA_LEN

Maximum length of ESPNOW data which is sent very time

Type Definitions

typedef struct *esp_now_peer_info* **esp_now_peer_info_t**

ESPNOW peer information parameters.

typedef struct *esp_now_peer_num* **esp_now_peer_num_t**

Number of ESPNOW peers which exist currently.

typedef struct *esp_now_recv_info* **esp_now_recv_info_t**

ESPNOW packet information.

typedef struct *esp_now_rate_config* **esp_now_rate_config_t**

ESPNOW rate config.

typedef void (***esp_now_recv_cb_t**)(const *esp_now_recv_info_t* *esp_now_info, const uint8_t *data, int data_len)

Callback function of receiving ESPNOW data.

Attention esp_now_info is a local variable, it can only be used in the callback.

Param esp_now_info received ESPNOW packet information
Param data received data
Param data_len length of received data

typedef void (***esp_now_send_cb_t**)(const uint8_t *mac_addr, *esp_now_send_status_t* status)

Callback function of sending ESPNOW data.

Param mac_addr peer MAC address
Param status status of sending ESPNOW data (succeed or fail)

Enumerations

enum **esp_now_send_status_t**

Status of sending ESPNOW data .

Values:

enumerator **ESP_NOW_SEND_SUCCESS**

Send ESPNOW data successfully

enumerator **ESP_NOW_SEND_FAIL**

Send ESPNOW data fail

ESP-WIFI-MESH Programming Guide

This is a programming guide for ESP-WIFI-MESH, including the API reference and coding examples. This guide is split into the following parts:

1. *ESP-WIFI-MESH Programming Model*
2. *Writing an ESP-WIFI-MESH Application*
3. *Self-Organized Networking*
4. *Application Examples*
5. *API Reference*

For documentation regarding the ESP-WIFI-MESH protocol, please see the *ESP-WIFI-MESH API Guide*. For more information about ESP-WIFI-MESH Development Framework, please see ESP-WIFI-MESH Development Framework.

ESP-WIFI-MESH Programming Model

Software Stack The ESP-WIFI-MESH software stack is built atop the Wi-Fi Driver/FreeRTOS and may use the LwIP Stack in some instances (i.e., the root node). The following diagram illustrates the ESP-WIFI-MESH software stack.

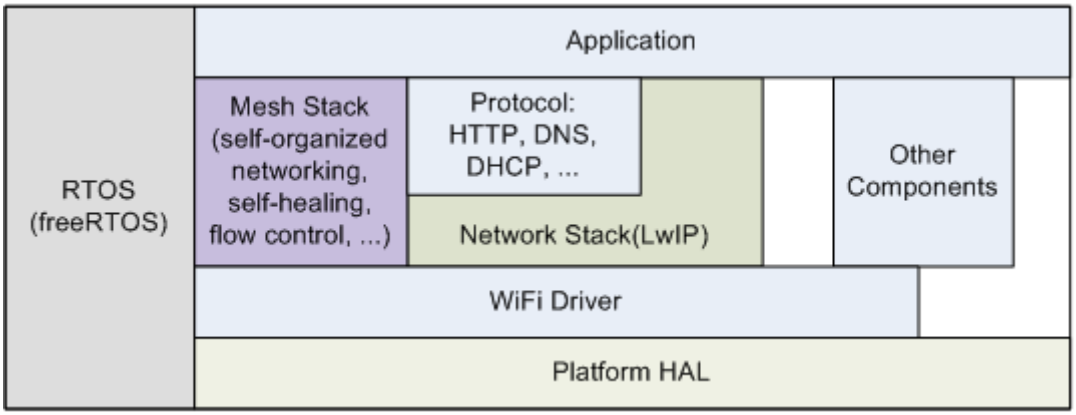


Fig. 2: ESP-WIFI-MESH Software Stack

System Events An application interfaces with ESP-WIFI-MESH via **ESP-WIFI-MESH Events**. Since ESP-WIFI-MESH is built atop the Wi-Fi stack, it is also possible for the application to interface with the Wi-Fi driver via the **Wi-Fi Event Task**. The following diagram illustrates the interfaces for the various System Events in an ESP-WIFI-MESH application.

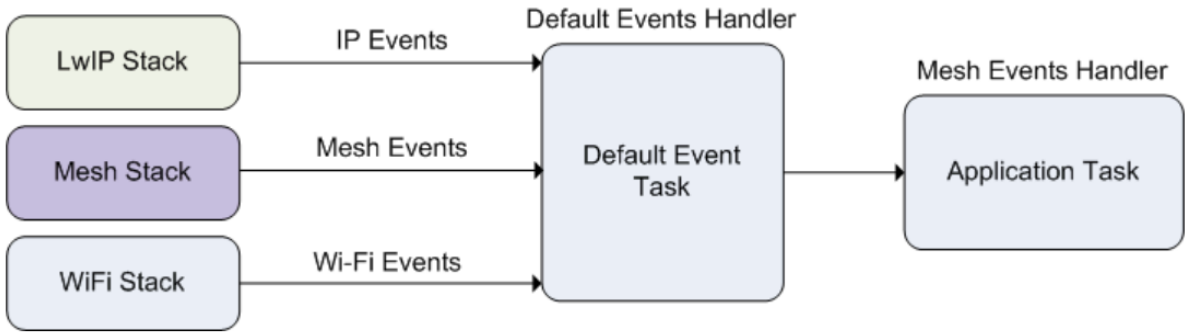


Fig. 3: ESP-WIFI-MESH System Events Delivery

The `mesh_event_id_t` defines all possible ESP-WIFI-MESH events and can indicate events such as the connection/disconnection of parent/child. Before ESP-WIFI-MESH events can be used, the application must register a **Mesh Events handler** via `esp_event_handler_register()` to the default event task. The Mesh Events handler that is registered contain handlers for each ESP-WIFI-MESH event relevant to the application.

Typical use cases of mesh events include using events such as `MESH_EVENT_PARENT_CONNECTED` and `MESH_EVENT_CHILD_CONNECTED` to indicate when a node can begin transmitting data upstream and downstream respectively. Likewise, `IP_EVENT_STA_GOT_IP` and `IP_EVENT_STA_LOST_IP` can be used to indicate when the root node can and cannot transmit data to the external IP network.

Warning: When using ESP-WIFI-MESH under self-organized mode, users must ensure that no calls to Wi-Fi API are made. This is due to the fact that the self-organizing mode will internally make Wi-Fi API calls to connect/disconnect/scan etc. **Any Wi-Fi calls from the application (including calls from callbacks and handlers of Wi-Fi events) may interfere with ESP-WIFI-MESH's self-organizing behavior**. Therefore, users should not call Wi-Fi APIs after `esp_mesh_start()` is called, and before `esp_mesh_stop()` is called.

LwIP & ESP-WIFI-MESH The application can access the ESP-WIFI-MESH stack directly without having to go through the LwIP stack. The LwIP stack is only required by the root node to transmit/receive data to/from an external IP network. However, since every node can potentially become the root node (due to automatic root node selection), each node must still initialize the LwIP stack.

Each node that could become root is required to initialize LwIP by calling *esp_netif_init()*. In order to prevent non-root node access to LwIP, the application should not create or register any network interfaces using esp_netif APIs.

> ESP-WIFI-MESH requires a root node to be connected with a router. Therefore, in the event that a node becomes the root, **the corresponding handler must start the DHCP client service and immediately obtain an IP address**. Doing so will allow other nodes to begin transmitting/receiving packets to/from the external IP network. However, this step is unnecessary if static IP settings are used.

Writing an ESP-WIFI-MESH Application The prerequisites for starting ESP-WIFI-MESH is to initialize LwIP and Wi-Fi, The following code snippet demonstrates the necessary prerequisite steps before ESP-WIFI-MESH itself can be initialized.

```
ESP_ERROR_CHECK(esp_netif_init());

/*  event initialization */
ESP_ERROR_CHECK(esp_event_loop_create_default());

/*  Wi-Fi initialization */
wifi_init_config_t config = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&config));
/*  register IP events handler */
ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &ip_
↪event_handler, NULL));
ESP_ERROR_CHECK(esp_wifi_set_storage(WIFI_STORAGE_FLASH));
ESP_ERROR_CHECK(esp_wifi_start());
```

After initializing LwIP and Wi-Fi, the process of getting an ESP-WIFI-MESH network up and running can be summarized into the following three steps:

1. *Initialize Mesh*
2. *Configuring an ESP-WIFI-MESH Network*
3. *Start Mesh*

Initialize Mesh The following code snippet demonstrates how to initialize ESP-WIFI-MESH

```
/*  mesh initialization */
ESP_ERROR_CHECK(esp_mesh_init());
/*  register mesh events handler */
ESP_ERROR_CHECK(esp_event_handler_register(MESH_EVENT, ESP_EVENT_ANY_ID, &mesh_
↪event_handler, NULL));
```

Configuring an ESP-WIFI-MESH Network ESP-WIFI-MESH is configured via *esp_mesh_set_config()* which receives its arguments using the *mesh_cfg_t* structure. The structure contains the following parameters used to configure ESP-WIFI-MESH:

Parameter	Description
Channel	Range from 1 to 14
Mesh ID	ID of ESP-WIFI-MESH Network, see *mesh_addr_t*
Router	Router Configuration, see *mesh_router_t*
Mesh AP	Mesh AP Configuration, see *mesh_ap_cfg_t*
Crypto Functions	Crypto Functions for Mesh IE, see mesh_crypto_funcs_t

The following code snippet demonstrates how to configure ESP-WIFI-MESH.

```
/* Enable the Mesh IE encryption by default */
mesh_cfg_t cfg = MESH_INIT_CONFIG_DEFAULT();
```

(continues on next page)

(continued from previous page)

```
/* mesh ID */
memcpy((uint8_t *) &cfg.mesh_id, MESH_ID, 6);
/* channel (must match the router's channel) */
cfg.channel = CONFIG_MESH_CHANNEL;
/* router */
cfg.router.ssid_len = strlen(CONFIG_MESH_ROUTER_SSID);
memcpy((uint8_t *) &cfg.router.ssid, CONFIG_MESH_ROUTER_SSID, cfg.router.ssid_len);
memcpy((uint8_t *) &cfg.router.password, CONFIG_MESH_ROUTER_PASSWD,
       strlen(CONFIG_MESH_ROUTER_PASSWD));
/* mesh softAP */
cfg.mesh_ap.max_connection = CONFIG_MESH_AP_CONNECTIONS;
memcpy((uint8_t *) &cfg.mesh_ap.password, CONFIG_MESH_AP_PASSWD,
       strlen(CONFIG_MESH_AP_PASSWD));
ESP_ERROR_CHECK(esp_mesh_set_config(&cfg));
```

Start Mesh The following code snippet demonstrates how to start ESP-WIFI-MESH.

```
/* mesh start */
ESP_ERROR_CHECK(esp_mesh_start());
```

After starting ESP-WIFI-MESH, the application should check for ESP-WIFI-MESH events to determine when it has connected to the network. After connecting, the application can start transmitting and receiving packets over the ESP-WIFI-MESH network using *esp_mesh_send()* and *esp_mesh_recv()*.

Self-Organized Networking Self-organized networking is a feature of ESP-WIFI-MESH where nodes can autonomously scan/select/connect/reconnect to other nodes and routers. This feature allows an ESP-WIFI-MESH network to operate with high degree of autonomy by making the network robust to dynamic network topologies and conditions. With self-organized networking enabled, nodes in an ESP-WIFI-MESH network are able to carry out the following actions without autonomously:

- Selection or election of the root node (see **Automatic Root Node Selection** in *Building a Network*)
- Selection of a preferred parent node (see **Parent Node Selection** in *Building a Network*)
- Automatic reconnection upon detecting a disconnection (see **Intermediate Parent Node Failure** in *Managing a Network*)

When self-organized networking is enabled, the ESP-WIFI-MESH stack will internally make calls to Wi-Fi APIs. Therefore, **the application layer should not make any calls to Wi-Fi APIs whilst self-organized networking is enabled as doing so would risk interfering with ESP-WIFI-MESH**.

Toggling Self-Organized Networking Self-organized networking can be enabled or disabled by the application at runtime by calling the *esp_mesh_set_self_organized()* function. The function has the two following parameters:

- `bool enable` specifies whether to enable or disable self-organized networking.
- `bool select_parent` specifies whether a new parent node should be selected when enabling self-organized networking. Selecting a new parent has different effects depending the node type and the node's current state. This parameter is unused when disabling self-organized networking.

Disabling Self-Organized Networking The following code snippet demonstrates how to disable self-organized networking.

```
//Disable self-organized networking
esp_mesh_set_self_organized(false, false);
```

ESP-WIFI-MESH will attempt to maintain the node's current Wi-Fi state when disabling self-organized networking.

- If the node was previously connected to other nodes, it will remain connected.

- If the node was previously disconnected and was scanning for a parent node or router, it will stop scanning.
- If the node was previously attempting to reconnect to a parent node or router, it will stop reconnecting.

Enabling Self-Organized Networking ESP-WIFI-MESH will attempt to maintain the node's current Wi-Fi state when enabling self-organized networking. However, depending on the node type and whether a new parent is selected, the Wi-Fi state of the node can change. The following table shows effects of enabling self-organized networking.

Select Parent	Is Root Node	Effects
N	N	• Nodes already connected to a parent node will remain connected. • Nodes previously scanning for a parent nodes will stop scanning. Call *esp_mesh_connect()* to restart.
	Y	• A root node already connected to router will stay connected. • A root node disconnected from router will need to call *esp_mesh_connect()* to reconnect.
Y	N	• Nodes without a parent node will automatically select a preferred parent and connect. • Nodes already connected to a parent node will disconnect, reselect a preferred parent node, and connect.
	Y	• For a root node to connect to a parent node, it must give up it's role as root. Therefore, a root node will disconnect from the router and all child nodes, select a preferred parent node, and connect.

The following code snipping demonstrates how to enable self-organized networking.

```
//Enable self-organized networking and select a new parent
esp_mesh_set_self_organized(true, true);

...

//Enable self-organized networking and manually reconnect
esp_mesh_set_self_organized(true, false);
esp_mesh_connect();
```

Calling Wi-Fi API There can be instances in which an application may want to directly call Wi-Fi API whilst using ESP-WIFI-MESH. For example, an application may want to manually scan for neighboring APs. However, **self-organized networking must be disabled before the application calls any Wi-Fi APIs**. This will prevent the ESP-WIFI-MESH stack from attempting to call any Wi-Fi APIs and potentially interfering with the application's calls.

Therefore, application calls to Wi-Fi APIs should be placed in between calls of *esp_mesh_set_self_organized()* which disable and enable self-organized networking. The following code snippet demonstrates how an application can safely call *esp_wifi_scan_start()* whilst using ESP-WIFI-MESH.

```
//Disable self-organized networking
esp_mesh_set_self_organized(0, 0);

//Stop any scans already in progress
esp_wifi_scan_stop();
//Manually start scan. Will automatically stop when run to completion
esp_wifi_scan_start();

//Process scan results

...

//Re-enable self-organized networking if still connected
esp_mesh_set_self_organized(1, 0);

...

//Re-enable self-organized networking if non-root and disconnected
esp_mesh_set_self_organized(1, 1);

...

//Re-enable self-organized networking if root and disconnected
esp_mesh_set_self_organized(1, 0);  //Do not select new parent
esp_mesh_connect();                 //Manually reconnect to router
```

Application Examples ESP-IDF contains these ESP-WIFI-MESH example projects:

The Internal Communication Example demonstrates how to set up a ESP-WIFI-MESH network and have the root node send a data packet to every node within the network.

The Manual Networking Example demonstrates how to use ESP-WIFI-MESH without the self-organizing features. This example shows how to program a node to manually scan for a list of potential parent nodes and select a parent node based on custom criteria.

API Reference

Header File

- components/esp_wifi/include/esp_mesh.h
- This header file can be included with:

  ```
  #include "esp_mesh.h"
  ```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_wifi
  ```

  or

```
PRIV_REQUIRES esp_wifi
```

Functions

esp_err_t **esp_mesh_init**(void)

Mesh initialization.

- Check whether Wi-Fi is started.
- Initialize mesh global variables with default values.

Attention This API shall be called after Wi-Fi is started.

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_deinit**(void)

Mesh de-initialization.

```
        - Release resources and stop the mesh
```

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_start**(void)

Start mesh.

- Initialize mesh IE.
- Start mesh network management service.
- Create TX and RX queues according to the configuration.
- Register mesh packets receive callback.

Attention This API shall be called after mesh initialization and configuration.

Returns
- ESP_OK
- ESP_FAIL
- ESP_ERR_MESH_NOT_INIT
- ESP_ERR_MESH_NOT_CONFIG
- ESP_ERR_MESH_NO_MEMORY

esp_err_t **esp_mesh_stop**(void)

Stop mesh.

- Deinitialize mesh IE.
- Disconnect with current parent.
- Disassociate all currently associated children.
- Stop mesh network management service.
- Unregister mesh packets receive callback.
- Delete TX and RX queues.
- Release resources.

- Restore Wi-Fi softAP to default settings if Wi-Fi dual mode is enabled.
- Set Wi-Fi Power Save type to WIFI_PS_NONE.

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_send**(const *mesh_addr_t* *to, const *mesh_data_t* *data, int flag, const *mesh_opt_t* opt[], int opt_count)

Send a packet over the mesh network.

- Send a packet to any device in the mesh network.
- Send a packet to external IP network.

Attention This API is not reentrant.

Parameters
- **to** -- **[in]** the address of the final destination of the packet
  - If the packet is to the root, set this parameter to NULL.
  - If the packet is to an external IP network, set this parameter to the IPv4:PORT combination. This packet will be delivered to the root firstly, then the root will forward this packet to the final IP server address.
- **data** -- **[in]** pointer to a sending mesh packet
  - Field size should not exceed MESH_MPS. Note that the size of one mesh packet should not exceed MESH_MTU.
  - Field proto should be set to data protocol in use (default is MESH_PROTO_BIN for binary).
  - Field tos should be set to transmission tos (type of service) in use (default is MESH_TOS_P2P for point-to-point reliable).
    - If the packet is to the root, MESH_TOS_P2P must be set to ensure reliable transmission.
    - As long as the MESH_TOS_P2P is set, the API is blocking, even if the flag is set with MESH_DATA_NONBLOCK.
    - As long as the MESH_TOS_DEF is set, the API is non-blocking.
- **flag** -- **[in]** bitmap for data sent
  - Flag is at least one of the three MESH_DATA_P2P/MESH_DATA_FROMDS/MESH_DATA_TODS, which represents the direction of packet sending.
  - Speed up the route search
    - If the packet is to an internal device, MESH_DATA_P2P should be set.
    - If the packet is to the root ("to" parameter isn't NULL) or to external IP network, MESH_DATA_TODS should be set.
    - If the packet is from the root to an internal device, MESH_DATA_FROMDS should be set.
  - Specify whether this API is blocking or non-blocking, blocking by default.
  - In the situation of the root change, MESH_DATA_DROP identifies this packet can be dropped by the new root for upstream data to external IP network, we try our best to avoid data loss caused by the root change, but there is a risk that the new root is running out of memory because most of memory is occupied by the pending data which isn't read out in time by esp_mesh_recv_toDS().
    Generally, we suggest esp_mesh_recv_toDS() is called after a connection with IP network is created. Thus data outgoing to external IP network via socket is just from reading esp_mesh_recv_toDS() which avoids unnecessary memory copy.
- **opt** -- **[in]** options
  - In case of sending a packet to a certain group, MESH_OPT_SEND_GROUP is a good choice. In this option, the value field should be set to the target receiver addresses in this group.

  - Root sends a packet to an internal device, this packet is from external IP network in case the receiver device responds this packet, MESH_OPT_RECV_DS_ADDR is required to attach the target DS address.
- **opt_count** -- **[in]** option count
  - Currently, this API only takes one option, so opt_count is only supported to be 1.

Returns

- ESP_OK
- ESP_FAIL
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_START
- ESP_ERR_MESH_DISCONNECTED
- ESP_ERR_MESH_OPT_UNKNOWN
- ESP_ERR_MESH_EXCEED_MTU
- ESP_ERR_MESH_NO_MEMORY
- ESP_ERR_MESH_TIMEOUT
- ESP_ERR_MESH_QUEUE_FULL
- ESP_ERR_MESH_NO_ROUTE_FOUND
- ESP_ERR_MESH_DISCARD

esp_err_t **esp_mesh_send_block_time**(uint32_t time_ms)

Set blocking time of esp_mesh_send()

- Suggest to set the blocking time to at least 5s when the environment is poor. Otherwise, esp_mesh_send() may timeout frequently.

Attention This API shall be called before mesh is started.

Parameters **time_ms** -- **[in]** blocking time of esp_mesh_send(), unit:ms

Returns

- ESP_OK

esp_err_t **esp_mesh_recv**(*mesh_addr_t* *from, *mesh_data_t* *data, int timeout_ms, int *flag, *mesh_opt_t* opt[], int opt_count)

Receive a packet targeted to self over the mesh network.

flag could be MESH_DATA_FROMDS or MESH_DATA_TODS.

Attention Mesh RX queue should be checked regularly to avoid running out of memory.

- Use esp_mesh_get_rx_pending() to check the number of packets available in the queue waiting to be received by applications.

Parameters

- **from** -- **[out]** the address of the original source of the packet
- **data** -- **[out]** pointer to the received mesh packet
  - Field proto is the data protocol in use. Should follow it to parse the received data.
  - Field tos is the transmission tos (type of service) in use.
- **timeout_ms** -- **[in]** wait time if a packet isn't immediately available (0:no wait, portMAX_DELAY:wait forever)
- **flag** -- **[out]** bitmap for data received
  - MESH_DATA_FROMDS represents data from external IP network
  - MESH_DATA_TODS represents data directed upward within the mesh network
- **opt** -- **[out]** options desired to receive
  - MESH_OPT_RECV_DS_ADDR attaches the DS address
- **opt_count** -- **[in]** option count desired to receive
  - Currently, this API only takes one option, so opt_count is only supported to be 1.

Returns

- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_START
- ESP_ERR_MESH_TIMEOUT
- ESP_ERR_MESH_DISCARD

esp_err_t **esp_mesh_recv_toDS**(*mesh_addr_t* *from, *mesh_addr_t* *to, *mesh_data_t* *data, int timeout_ms, int *flag, *mesh_opt_t* opt[], int opt_count)

Receive a packet targeted to external IP network.

- Root uses this API to receive packets destined to external IP network
- Root forwards the received packets to the final destination via socket.
- If no socket connection is ready to send out the received packets and this esp_mesh_recv_toDS() hasn't been called by applications, packets from the whole mesh network will be pending in toDS queue.

Use esp_mesh_get_rx_pending() to check the number of packets available in the queue waiting to be received by applications in case of running out of memory in the root.

Using esp_mesh_set_xon_qsize() users may configure the RX queue size, default:32. If this size is too large, and esp_mesh_recv_toDS() isn't called in time, there is a risk that a great deal of memory is occupied by the pending packets. If this size is too small, it will impact the efficiency on upstream. How to decide this value depends on the specific application scenarios.

flag could be MESH_DATA_TODS.

Attention This API is only called by the root.

Parameters

- **from** -- **[out]** the address of the original source of the packet
- **to** -- **[out]** the address contains remote IP address and port (IPv4:PORT)
- **data** -- **[out]** pointer to the received packet
  - Contain the protocol and applications should follow it to parse the data.
- **timeout_ms** -- **[in]** wait time if a packet isn't immediately available (0:no wait, port-MAX_DELAY:wait forever)
- **flag** -- **[out]** bitmap for data received
  - MESH_DATA_TODS represents the received data target to external IP network. Root shall forward this data to external IP network via the association with router.
- **opt** -- **[out]** options desired to receive
- **opt_count** -- **[in]** option count desired to receive

Returns

- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_START
- ESP_ERR_MESH_TIMEOUT
- ESP_ERR_MESH_DISCARD
- ESP_ERR_MESH_RECV_RELEASE

esp_err_t **esp_mesh_set_config**(const *mesh_cfg_t* *config)

Set mesh stack configuration.

- Use MESH_INIT_CONFIG_DEFAULT() to initialize the default values, mesh IE is encrypted by default.
- Mesh network is established on a fixed channel (1-14).
- Mesh event callback is mandatory.
- Mesh ID is an identifier of an MBSS. Nodes with the same mesh ID can communicate with each other.

- Regarding to the router configuration, if the router is hidden, BSSID field is mandatory.

If BSSID field isn't set and there exists more than one router with same SSID, there is a risk that more roots than one connected with different BSSID will appear. It means more than one mesh network is established with the same mesh ID.

Root conflict function could eliminate redundant roots connected with the same BSSID, but couldn't handle roots connected with different BSSID. Because users might have such requirements of setting up routers with same SSID for the future replacement. But in that case, if the above situations happen, please make sure applications implement forward functions on the root to guarantee devices in different mesh networks can communicate with each other. max_connection of mesh softAP is limited by the max number of Wi-Fi softAP supported (max:10).

Attention This API shall be called before mesh is started after mesh is initialized.

Parameters **config** -- **[in]** pointer to mesh stack configuration
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_ALLOWED

esp_err_t **esp_mesh_get_config**(*mesh_cfg_t* *config)

Get mesh stack configuration.

Parameters **config** -- **[out]** pointer to mesh stack configuration
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_set_router**(const *mesh_router_t* *router)

Get router configuration.

Attention This API is used to dynamically modify the router configuration after mesh is configured.

Parameters **router** -- **[in]** pointer to router configuration
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_get_router**(*mesh_router_t* *router)

Get router configuration.

Parameters **router** -- **[out]** pointer to router configuration
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_set_id**(const *mesh_addr_t* *id)

Set mesh network ID.

Attention This API is used to dynamically modify the mesh network ID.

Parameters **id** -- **[in]** pointer to mesh network ID
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT: invalid argument

esp_err_t **esp_mesh_get_id**(*mesh_addr_t* *id)

Get mesh network ID.

Parameters **id** -- **[out]** pointer to mesh network ID
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_set_type**(*mesh_type_t* type)

Designate device type over the mesh network.

- MESH_IDLE: designates a device as a self-organized node for a mesh network
- MESH_ROOT: designates the root node for a mesh network
- MESH_LEAF: designates a device as a standalone Wi-Fi station that connects to a parent
- MESH_STA: designates a device as a standalone Wi-Fi station that connects to a router

Parameters **type** -- **[in]** device type
Returns
- ESP_OK
- ESP_ERR_MESH_NOT_ALLOWED

mesh_type_t **esp_mesh_get_type**(void)

Get device type over mesh network.

Attention This API shall be called after having received the event MESH_EVENT_PARENT_CONNECTED.

Returns mesh type

esp_err_t **esp_mesh_set_max_layer**(int max_layer)

Set network max layer value.

- for tree topology, the max is 25.
- for chain topology, the max is 1000.
- Network max layer limits the max hop count.

Attention This API shall be called before mesh is started.

Parameters **max_layer** -- **[in]** max layer value
Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_ALLOWED

int **esp_mesh_get_max_layer**(void)

Get max layer value.

Returns max layer value

esp_err_t **esp_mesh_set_ap_password**(const uint8_t *pwd, int len)

Set mesh softAP password.

Attention This API shall be called before mesh is started.

Parameters
- **pwd** -- **[in]** pointer to the password
- **len** -- **[in]** password length

Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_ALLOWED

esp_err_t **esp_mesh_set_ap_authmode**(wifi_auth_mode_t authmode)

Set mesh softAP authentication mode.

Attention This API shall be called before mesh is started.

Parameters **authmode** -- **[in]** authentication mode

Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT
- ESP_ERR_MESH_NOT_ALLOWED

wifi_auth_mode_t **esp_mesh_get_ap_authmode**(void)

Get mesh softAP authentication mode.

Returns authentication mode

esp_err_t **esp_mesh_set_ap_connections**(int connections)

Set mesh max connection value.

- Set mesh softAP max connection = mesh max connection + non-mesh max connection

Attention This API shall be called before mesh is started.

Parameters **connections** -- **[in]** the number of max connections

Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

int **esp_mesh_get_ap_connections**(void)

Get mesh max connection configuration.

Returns the number of mesh max connections

int **esp_mesh_get_non_mesh_connections**(void)

Get non-mesh max connection configuration.

Returns the number of non-mesh max connections

int **esp_mesh_get_layer**(void)

Get current layer value over the mesh network.

Attention This API shall be called after having received the event MESH_EVENT_PARENT_CONNECTED.

Returns layer value

esp_err_t **esp_mesh_get_parent_bssid**(*mesh_addr_t* *bssid)

Get the parent BSSID.

Attention This API shall be called after having received the event MESH_EVENT_PARENT_CONNECTED.

Parameters **bssid** -- **[out]** pointer to parent BSSID

Returns

- ESP_OK
- ESP_FAIL

bool **esp_mesh_is_root**(void)

Return whether the device is the root node of the network.

Returns true/false

esp_err_t **esp_mesh_set_self_organized**(bool enable, bool select_parent)

Enable/disable self-organized networking.

- Self-organized networking has three main functions: select the root node; find a preferred parent; initiate reconnection if a disconnection is detected.
- Self-organized networking is enabled by default.
- If self-organized is disabled, users should set a parent for the device via esp_mesh_set_parent().

Attention This API is used to dynamically modify whether to enable the self organizing.

Parameters

- **enable** -- **[in]** enable or disable self-organized networking
- **select_parent** -- **[in]** Only valid when self-organized networking is enabled.
  - if select_parent is set to true, the root will give up its mesh root status and search for a new parent like other non-root devices.

Returns

- ESP_OK
- ESP_FAIL

bool **esp_mesh_get_self_organized**(void)

Return whether enable self-organized networking or not.

Returns true/false

esp_err_t **esp_mesh_waive_root**(const *mesh_vote_t* *vote, int reason)

Cause the root device to give up (waive) its mesh root status.

- A device is elected root primarily based on RSSI from the external router.
- If external router conditions change, users can call this API to perform a root switch.
- In this API, users could specify a desired root address to replace itself or specify an attempts value to ask current root to initiate a new round of voting. During the voting, a better root candidate would be expected to find to replace the current one.
- If no desired root candidate, the vote will try a specified number of attempts (at least 15). If no better root candidate is found, keep the current one. If a better candidate is found, the new better one will send a root switch request to the current root, current root will respond with a root switch acknowledgment.
- After that, the new candidate will connect to the router to be a new root, the previous root will disconnect with the router and choose another parent instead.

Root switch is completed with minimal disruption to the whole mesh network.

Attention This API is only called by the root.

Parameters
- **vote** -- **[in]** vote configuration
  - If this parameter is set NULL, the vote will perform the default 15 times.
  - Field percentage threshold is 0.9 by default.
  - Field is_rc_specified shall be false.
  - Field attempts shall be at least 15 times.
- **reason** -- **[in]** only accept MESH_VOTE_REASON_ROOT_INITIATED for now

Returns
- ESP_OK
- ESP_ERR_MESH_QUEUE_FULL
- ESP_ERR_MESH_DISCARD
- ESP_FAIL

esp_err_t **esp_mesh_set_vote_percentage**(float percentage)

Set vote percentage threshold for approval of being a root (default:0.9)

- During the networking, only obtaining vote percentage reaches this threshold, the device could be a root.

Attention This API shall be called before mesh is started.

Parameters **percentage** -- **[in]** vote percentage threshold

Returns
- ESP_OK
- ESP_FAIL

float **esp_mesh_get_vote_percentage**(void)

Get vote percentage threshold for approval of being a root.

Returns percentage threshold

esp_err_t **esp_mesh_set_ap_assoc_expire**(int seconds)

Set mesh softAP associate expired time (default:10 seconds)

- If mesh softAP hasn't received any data from an associated child within this time, mesh softAP will take this child inactive and disassociate it.
- If mesh softAP is encrypted, this value should be set a greater value, such as 30 seconds.

Parameters **seconds** -- **[in]** the expired time

Returns
- ESP_OK
- ESP_FAIL

int **esp_mesh_get_ap_assoc_expire**(void)

Get mesh softAP associate expired time.

Returns seconds

int **esp_mesh_get_total_node_num**(void)

Get total number of devices in current network (including the root)

Attention The returned value might be incorrect when the network is changing.

Returns total number of devices (including the root)

int **esp_mesh_get_routing_table_size**(void)

Get the number of devices in this device's sub-network (including self)

Returns the number of devices over this device's sub-network (including self)

esp_err_t **esp_mesh_get_routing_table**(*mesh_addr_t* *mac, int len, int *size)

Get routing table of this device's sub-network (including itself)

Parameters
- **mac** -- **[out]** pointer to routing table
- **len** -- **[in]** routing table size(in bytes)
- **size** -- **[out]** pointer to the number of devices in routing table (including itself)

Returns
- ESP_OK
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_post_toDS_state**(bool reachable)

Post the toDS state to the mesh stack.

Attention This API is only for the root.

Parameters **reachable** -- **[in]** this state represents whether the root is able to access external IP network

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_get_tx_pending**(*mesh_tx_pending_t* *pending)

Return the number of packets pending in the queue waiting to be sent by the mesh stack.

Parameters **pending** -- **[out]** pointer to the TX pending

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_get_rx_pending**(*mesh_rx_pending_t* *pending)

Return the number of packets available in the queue waiting to be received by applications.

Parameters **pending** -- **[out]** pointer to the RX pending

Returns
- ESP_OK
- ESP_FAIL

int **esp_mesh_available_txupQ_num**(const *mesh_addr_t* *addr, uint32_t *xseqno_in)

Return the number of packets could be accepted from the specified address.

Parameters
- **addr** -- **[in]** self address or an associate children address
- **xseqno_in** -- **[out]** sequence number of the last received packet from the specified address

Returns the number of upQ for a certain address

esp_err_t **esp_mesh_set_xon_qsize**(int qsize)

Set the number of RX queue for the node, the average number of window allocated to one of its child node is: wnd = xon_qsize / (2 * max_connection + 1). However, the window of each child node is not strictly equal to the average value, it is affected by the traffic also.

Attention This API shall be called before mesh is started.

Parameters **qsize** -- **[in]** default:32 (min:16)
Returns
- ESP_OK
- ESP_FAIL

int **esp_mesh_get_xon_qsize**(void)

Get queue size.

Returns the number of queue

esp_err_t **esp_mesh_allow_root_conflicts**(bool allowed)

Set whether allow more than one root existing in one network.

- The default value is true, that is, multiple roots are allowed.

Parameters **allowed** -- **[in]** allow or not
Returns
- ESP_OK
- ESP_WIFI_ERR_NOT_INIT
- ESP_WIFI_ERR_NOT_START

bool **esp_mesh_is_root_conflicts_allowed**(void)

Check whether allow more than one root to exist in one network.

Returns true/false

esp_err_t **esp_mesh_set_group_id**(const *mesh_addr_t* *addr, int num)

Set group ID addresses.

Parameters
- **addr** -- **[in]** pointer to new group ID addresses
- **num** -- **[in]** the number of group ID addresses

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT

esp_err_t **esp_mesh_delete_group_id**(const *mesh_addr_t* *addr, int num)

Delete group ID addresses.

Parameters
- **addr** -- **[in]** pointer to deleted group ID address
- **num** -- **[in]** the number of group ID addresses

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT

int **esp_mesh_get_group_num**(void)

Get the number of group ID addresses.

Returns the number of group ID addresses

esp_err_t **esp_mesh_get_group_list**(*mesh_addr_t* *addr, int num)

Get group ID addresses.

Parameters
- **addr** -- **[out]** pointer to group ID addresses
- **num** -- **[in]** the number of group ID addresses

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT

bool **esp_mesh_is_my_group**(const *mesh_addr_t* *addr)

Check whether the specified group address is my group.

Returns true/false

esp_err_t **esp_mesh_set_capacity_num**(int num)

Set mesh network capacity (max:1000, default:300)

Attention This API shall be called before mesh is started.

Parameters **num** -- **[in]** mesh network capacity

Returns
- ESP_OK
- ESP_ERR_MESH_NOT_ALLOWED
- ESP_MESH_ERR_ARGUMENT

int **esp_mesh_get_capacity_num**(void)

Get mesh network capacity.

Returns mesh network capacity

esp_err_t **esp_mesh_set_ie_crypto_funcs**(const mesh_crypto_funcs_t *crypto_funcs)

Set mesh IE crypto functions.

Attention This API can be called at any time after mesh is configured.

Parameters **crypto_funcs** -- **[in]** crypto functions for mesh IE
- If crypto_funcs is set to NULL, mesh IE is no longer encrypted.

Returns
- ESP_OK

esp_err_t **esp_mesh_set_ie_crypto_key**(const char *key, int len)

Set mesh IE crypto key.

Attention This API can be called at any time after mesh is configured.

Parameters
- **key** -- **[in]** ASCII crypto key
- **len** -- **[in]** length in bytes, range:8~64

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT

esp_err_t **esp_mesh_get_ie_crypto_key**(char *key, int len)

Get mesh IE crypto key.

Parameters
- **key** -- **[out]** ASCII crypto key
- **len** -- **[in]** length in bytes, range:8~64

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT

esp_err_t **esp_mesh_set_root_healing_delay**(int delay_ms)

Set delay time before starting root healing.

Parameters **delay_ms** -- **[in]** delay time in milliseconds
Returns
- ESP_OK

int **esp_mesh_get_root_healing_delay**(void)

Get delay time before network starts root healing.

Returns delay time in milliseconds

esp_err_t **esp_mesh_fix_root**(bool enable)

Enable network Fixed Root Setting.

- Enabling fixed root disables automatic election of the root node via voting.
- All devices in the network shall use the same Fixed Root Setting (enabled or disabled).
- If Fixed Root is enabled, users should make sure a root node is designated for the network.

Parameters **enable** -- **[in]** enable or not
Returns
- ESP_OK

bool **esp_mesh_is_root_fixed**(void)

Check whether network Fixed Root Setting is enabled.

- Enable/disable network Fixed Root Setting by API esp_mesh_fix_root().
- Network Fixed Root Setting also changes with the "flag" value in parent networking IE.

Returns true/false

esp_err_t **esp_mesh_set_parent**(const wifi_config_t *parent, const *mesh_addr_t* *parent_mesh_id, *mesh_type_t* my_type, int my_layer)

Set a specified parent for the device.

Attention This API can be called at any time after mesh is configured.

Parameters
- **parent** -- **[in]** parent configuration, the SSID and the channel of the parent are mandatory.
  - If the BSSID is set, make sure that the SSID and BSSID represent the same parent, otherwise the device will never find this specified parent.
- **parent_mesh_id** -- **[in]** parent mesh ID,
  - If this value is not set, the original mesh ID is used.
- **my_type** -- **[in]** mesh type
  - MESH_STA is not supported.
  - If the parent set for the device is the same as the router in the network configuration, then my_type shall set MESH_ROOT and my_layer shall set MESH_ROOT_LAYER.
- **my_layer** -- **[in]** mesh layer
  - my_layer of the device may change after joining the network.
  - If my_type is set MESH_NODE, my_layer shall be greater than MESH_ROOT_LAYER.
  - If my_type is set MESH_LEAF, the device becomes a standalone Wi-Fi station and no longer has the ability to extend the network.

Returns
- ESP_OK

- ESP_ERR_ARGUMENT
- ESP_ERR_MESH_NOT_CONFIG

esp_err_t **esp_mesh_scan_get_ap_ie_len**(int *len)

Get mesh networking IE length of one AP.

Parameters **len** -- **[out]** mesh networking IE length

Returns

- ESP_OK
- ESP_ERR_WIFI_NOT_INIT
- ESP_ERR_INVALID_ARG
- ESP_ERR_WIFI_FAIL

esp_err_t **esp_mesh_scan_get_ap_record**(wifi_ap_record_t *ap_record, void *buffer)

Get AP record.

Attention Different from esp_wifi_scan_get_ap_records(), this API only gets one of APs scanned each time. See "manual_networking" example.

Parameters

- **ap_record** -- **[out]** pointer to one AP record
- **buffer** -- **[out]** pointer to the mesh networking IE of this AP

Returns

- ESP_OK
- ESP_ERR_WIFI_NOT_INIT
- ESP_ERR_INVALID_ARG
- ESP_ERR_WIFI_FAIL

esp_err_t **esp_mesh_flush_upstream_packets**(void)

Flush upstream packets pending in to_parent queue and to_parent_p2p queue.

Returns

- ESP_OK

esp_err_t **esp_mesh_get_subnet_nodes_num**(const *mesh_addr_t* *child_mac, int *nodes_num)

Get the number of nodes in the subnet of a specific child.

Parameters

- **child_mac** -- **[in]** an associated child address of this device
- **nodes_num** -- **[out]** pointer to the number of nodes in the subnet of a specific child

Returns

- ESP_OK
- ESP_ERR_MESH_NOT_START
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_get_subnet_nodes_list**(const *mesh_addr_t* *child_mac, *mesh_addr_t* *nodes, int nodes_num)

Get nodes in the subnet of a specific child.

Parameters

- **child_mac** -- **[in]** an associated child address of this device
- **nodes** -- **[out]** pointer to nodes in the subnet of a specific child
- **nodes_num** -- **[in]** the number of nodes in the subnet of a specific child

Returns

- ESP_OK
- ESP_ERR_MESH_NOT_START
- ESP_ERR_MESH_ARGUMENT

esp_err_t **esp_mesh_disconnect**(void)

Disconnect from current parent.

Returns
- ESP_OK

esp_err_t **esp_mesh_connect**(void)

Connect to current parent.

Returns
- ESP_OK

esp_err_t **esp_mesh_flush_scan_result**(void)

Flush scan result.

Returns
- ESP_OK

esp_err_t **esp_mesh_switch_channel**(const uint8_t *new_bssid, int csa_newchan, int csa_count)

Cause the root device to add Channel Switch Announcement Element (CSA IE) to beacon.

- Set the new channel
- Set how many beacons with CSA IE will be sent before changing a new channel
- Enable the channel switch function

Attention This API is only called by the root.

Parameters
- **new_bssid** -- **[in]** the new router BSSID if the router changes
- **csa_newchan** -- **[in]** the new channel number to which the whole network is moving
- **csa_count** -- **[in]** channel switch period(beacon count), unit is based on beacon interval of its softAP, the default value is 15.

Returns
- ESP_OK

esp_err_t **esp_mesh_get_router_bssid**(uint8_t *router_bssid)

Get the router BSSID.

Parameters **router_bssid** -- **[out]** pointer to the router BSSID

Returns
- ESP_OK
- ESP_ERR_WIFI_NOT_INIT
- ESP_ERR_INVALID_ARG

int64_t **esp_mesh_get_tsf_time**(void)

Get the TSF time.

Returns the TSF time

esp_err_t **esp_mesh_set_topology**(*esp_mesh_topology_t* topo)

Set mesh topology. The default value is MESH_TOPO_TREE.

- MESH_TOPO_CHAIN supports up to 1000 layers

Attention This API shall be called before mesh is started.

Parameters **topo** -- **[in]** MESH_TOPO_TREE or MESH_TOPO_CHAIN

Returns
- ESP_OK
- ESP_MESH_ERR_ARGUMENT
- ESP_ERR_MESH_NOT_ALLOWED

esp_mesh_topology_t **esp_mesh_get_topology**(void)

Get mesh topology.

Returns MESH_TOPO_TREE or MESH_TOPO_CHAIN

esp_err_t **esp_mesh_enable_ps**(void)

Enable mesh Power Save function.

Attention This API shall be called before mesh is started.

Returns
- ESP_OK
- ESP_ERR_WIFI_NOT_INIT
- ESP_ERR_MESH_NOT_ALLOWED

esp_err_t **esp_mesh_disable_ps**(void)

Disable mesh Power Save function.

Attention This API shall be called before mesh is started.

Returns
- ESP_OK
- ESP_ERR_WIFI_NOT_INIT
- ESP_ERR_MESH_NOT_ALLOWED

bool **esp_mesh_is_ps_enabled**(void)

Check whether the mesh Power Save function is enabled.

Returns true/false

bool **esp_mesh_is_device_active**(void)

Check whether the device is in active state.

- If the device is not in active state, it will neither transmit nor receive frames.

Returns true/false

esp_err_t **esp_mesh_set_active_duty_cycle**(int dev_duty, int dev_duty_type)

Set the device duty cycle and type.

- The range of dev_duty values is 1 to 100. The default value is 10.
- dev_duty = 100, the PS will be stopped.
- dev_duty is better to not less than 5.
- dev_duty_type could be MESH_PS_DEVICE_DUTY_REQUEST or MESH_PS_DEVICE_DUTY_DEMAND.
- If dev_duty_type is set to MESH_PS_DEVICE_DUTY_REQUEST, the device will use a nwk_duty provided by the network.
- If dev_duty_type is set to MESH_PS_DEVICE_DUTY_DEMAND, the device will use the specified dev_duty.

Attention This API can be called at any time after mesh is started.

Parameters
- **dev_duty** -- **[in]** device duty cycle
- **dev_duty_type** -- **[in]** device PS duty cycle type, not accept MESH_PS_NETWORK_DUTY_MASTER

Returns
- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_get_active_duty_cycle**(int *dev_duty, int *dev_duty_type)

Get device duty cycle and type.

Parameters
- **dev_duty** -- **[out]** device duty cycle
- **dev_duty_type** -- **[out]** device PS duty cycle type

Returns
- ESP_OK

esp_err_t **esp_mesh_set_network_duty_cycle**(int nwk_duty, int duration_mins, int applied_rule)

Set the network duty cycle, duration and rule.

- The range of nwk_duty values is 1 to 100. The default value is 10.
- nwk_duty is the network duty cycle the entire network or the up-link path will use. A device that successfully sets the nwk_duty is known as a NWK-DUTY-MASTER.
- duration_mins specifies how long the specified nwk_duty will be used. Once duration_mins expires, the root will take over as the NWK-DUTY-MASTER. If an existing NWK-DUTY-MASTER leaves the network, the root will take over as the NWK-DUTY-MASTER again.
- duration_mins = (-1) represents nwk_duty will be used until a new NWK-DUTY-MASTER with a different nwk_duty appears.
- Only the root can set duration_mins to (-1).
- If applied_rule is set to MESH_PS_NETWORK_DUTY_APPLIED_ENTIRE, the nwk_duty will be used by the entire network.
- If applied_rule is set to MESH_PS_NETWORK_DUTY_APPLIED_UPLINK, the nwk_duty will only be used by the up-link path nodes.
- The root does not accept MESH_PS_NETWORK_DUTY_APPLIED_UPLINK.
- A nwk_duty with duration_mins(-1) set by the root is the default network duty cycle used by the entire network.

Attention This API can be called at any time after mesh is started.
- In self-organized network, if this API is called before mesh is started in all devices, (1)nwk_duty shall be set to the same value for all devices; (2)duration_mins shall be set to (-1); (3)applied_rule shall be set to MESH_PS_NETWORK_DUTY_APPLIED_ENTIRE; after the voted root appears, the root will become the NWK-DUTY-MASTER and broadcast the nwk_duty and its identity of NWK-DUTY-MASTER.
- If the root is specified (FIXED-ROOT), call this API in the root to provide a default nwk_duty for the entire network.
- After joins the network, any device can call this API to change the nwk_duty, duration_mins or applied_rule.

Parameters
- **nwk_duty** -- **[in]** network duty cycle
- **duration_mins** -- **[in]** duration (unit: minutes)
- **applied_rule** -- **[in]** only support MESH_PS_NETWORK_DUTY_APPLIED_ENTIRE

Returns

- ESP_OK
- ESP_FAIL

esp_err_t **esp_mesh_get_network_duty_cycle**(int *nwk_duty, int *duration_mins, int *dev_duty_type, int *applied_rule)

Get the network duty cycle, duration, type and rule.

Parameters
- **nwk_duty** -- **[out]** current network duty cycle
- **duration_mins** -- **[out]** the duration of current nwk_duty
- **dev_duty_type** -- **[out]** if it includes MESH_PS_DEVICE_DUTY_MASTER, this device is the current NWK-DUTY-MASTER.
- **applied_rule** -- **[out]** MESH_PS_NETWORK_DUTY_APPLIED_ENTIRE

Returns
- ESP_OK

int **esp_mesh_get_running_active_duty_cycle**(void)

Get the running active duty cycle.

- The running active duty cycle of the root is 100.
- If duty type is set to MESH_PS_DEVICE_DUTY_REQUEST, the running active duty cycle is nwk_duty provided by the network.
- If duty type is set to MESH_PS_DEVICE_DUTY_DEMAND, the running active duty cycle is dev_duty specified by the users.
- In a mesh network, devices are typically working with a certain duty-cycle (transmitting, receiving and sleep) to reduce the power consumption. The running active duty cycle decides the amount of awake time within a beacon interval. At each start of beacon interval, all devices wake up, broadcast beacons, and transmit packets if they do have pending packets for their parents or for their children. Note that Low-duty-cycle means devices may not be active in most of the time, the latency of data transmission might be greater.

Returns the running active duty cycle

esp_err_t **esp_mesh_ps_duty_signaling**(int fwd_times)

Duty signaling.

Parameters **fwd_times** -- **[in]** the times of forwarding duty signaling packets
Returns
- ESP_OK

Unions

union **mesh_addr_t**

#include <esp_mesh.h> Mesh address.

Public Members

uint8_t **addr**[6]

mac address

mip_t **mip**

mip address

union **mesh_event_info_t**

#include <esp_mesh.h> Mesh event information.

Public Members

mesh_event_channel_switch_t **channel_switch**

channel switch

mesh_event_child_connected_t **child_connected**

child connected

mesh_event_child_disconnected_t **child_disconnected**

child disconnected

mesh_event_routing_table_change_t **routing_table**

routing table change

mesh_event_connected_t **connected**

parent connected

mesh_event_disconnected_t **disconnected**

parent disconnected

mesh_event_no_parent_found_t **no_parent**

no parent found

mesh_event_layer_change_t **layer_change**

layer change

mesh_event_toDS_state_t **toDS_state**

toDS state, devices shall check this state firstly before trying to send packets to external IP network. This state indicates right now whether the root is capable of sending packets out. If not, devices had better to wait until this state changes to be MESH_TODS_REACHABLE.

mesh_event_vote_started_t **vote_started**

vote started

mesh_event_root_address_t **root_addr**

root address

mesh_event_root_switch_req_t **switch_req**

root switch request

mesh_event_root_conflict_t **root_conflict**

other powerful root

mesh_event_root_fixed_t **root_fixed**

fixed root

mesh_event_scan_done_t **scan_done**

scan done

mesh_event_network_state_t **network_state**

network state, such as whether current mesh network has a root.

mesh_event_find_network_t **find_network**

network found that can join

mesh_event_router_switch_t **router_switch**

new router information

mesh_event_ps_duty_t **ps_duty**

PS duty information

union **mesh_rc_config_t**

#include <esp_mesh.h> Vote address configuration.

Public Members

int **attempts**

max vote attempts before a new root is elected automatically by mesh network. (min:15, 15 by default)

mesh_addr_t **rc_addr**

a new root address specified by users for API esp_mesh_waive_root()

Structures

struct **mip_t**

IP address and port.

Public Members

esp_ip4_addr_t **ip4**

IP address

uint16_t **port**

port

struct **mesh_event_channel_switch_t**

Channel switch information.

Public Members

uint8_t **channel**

new channel

struct **mesh_event_connected_t**

Parent connected information.

Public Members

wifi_event_sta_connected_t **connected**

parent information, same as Wi-Fi event SYSTEM_EVENT_STA_CONNECTED does

uint16_t **self_layer**

layer

uint8_t **duty**

parent duty

struct **mesh_event_no_parent_found_t**

No parent found information.

Public Members

int **scan_times**

scan times being through

struct **mesh_event_layer_change_t**

Layer change information.

Public Members

uint16_t **new_layer**

new layer

struct **mesh_event_vote_started_t**

vote started information

Public Members

int **reason**

vote reason, vote could be initiated by children or by the root itself

int **attempts**

max vote attempts before stopped

mesh_addr_t **rc_addr**

root address specified by users via API esp_mesh_waive_root()

struct **mesh_event_find_network_t**

find a mesh network that this device can join

Public Members

uint8_t **channel**

channel number of the new found network

uint8_t **router_bssid**[6]

router BSSID

struct **mesh_event_root_switch_req_t**

Root switch request information.

Public Members

int **reason**

root switch reason, generally root switch is initialized by users via API esp_mesh_waive_root()

mesh_addr_t **rc_addr**

the address of root switch requester

struct **mesh_event_root_conflict_t**

Other powerful root address.

Public Members

int8_t **rssi**

rssi with router

uint16_t **capacity**

the number of devices in current network

uint8_t **addr**[6]

other powerful root address

struct **mesh_event_routing_table_change_t**

Routing table change.

Public Members

uint16_t **rt_size_new**

the new value

uint16_t **rt_size_change**

the changed value

struct **mesh_event_root_fixed_t**

Root fixed.

Public Members

bool **is_fixed**
: status

struct **mesh_event_scan_done_t**
: Scan done event information.

Public Members

uint8_t **number**
: the number of APs scanned

struct **mesh_event_network_state_t**
: Network state information.

Public Members

bool **is_rootless**
: whether current mesh network has a root

struct **mesh_event_ps_duty_t**
: PS duty information.

Public Members

uint8_t **duty**
: parent or child duty

mesh_event_child_connected_t **child_connected**
: child info

struct **mesh_opt_t**
: Mesh option.

Public Members

uint8_t **type**
: option type

uint16_t **len**
: option length

uint8_t ***val**
: option value

struct **mesh_data_t**

Mesh data for esp_mesh_send() and esp_mesh_recv()

Public Members

uint8_t ***data**

data

uint16_t **size**

data size

mesh_proto_t **proto**

data protocol

mesh_tos_t **tos**

data type of service

struct **mesh_router_t**

Router configuration.

Public Members

uint8_t **ssid**[32]

SSID

uint8_t **ssid_len**

length of SSID

uint8_t **bssid**[6]

BSSID, if this value is specified, users should also specify "allow_router_switch".

uint8_t **password**[64]

password

bool **allow_router_switch**

if the BSSID is specified and this value is also set, when the router of this specified BSSID fails to be found after "fail" (mesh_attempts_t) times, the whole network is allowed to switch to another router with the same SSID. The new router might also be on a different channel. The default value is false. There is a risk that if the password is different between the new switched router and the previous one, the mesh network could be established but the root will never connect to the new switched router.

struct **mesh_ap_cfg_t**

Mesh softAP configuration.

Public Members

uint8_t **password**[64]

mesh softAP password

uint8_t **max_connection**

max number of stations allowed to connect in, default 6, max 10 = max_connection + nonmesh_max_connection max mesh connections

uint8_t **nonmesh_max_connection**

max non-mesh connections

struct **mesh_cfg_t**

Mesh initialization configuration.

Public Members

uint8_t **channel**

channel, the mesh network on

bool **allow_channel_switch**

if this value is set, when "fail" (mesh_attempts_t) times is reached, device will change to a full channel scan for a network that could join. The default value is false.

mesh_addr_t **mesh_id**

mesh network identification

mesh_router_t **router**

router configuration

mesh_ap_cfg_t **mesh_ap**

mesh softAP configuration

const mesh_crypto_funcs_t ***crypto_funcs**

crypto functions

struct **mesh_vote_t**

Vote.

Public Members

float **percentage**

vote percentage threshold for approval of being a root

bool **is_rc_specified**

if true, rc_addr shall be specified (Unimplemented). if false, attempts value shall be specified to make network start root election.

mesh_rc_config_t **config**

vote address configuration

struct **mesh_tx_pending_t**

The number of packets pending in the queue waiting to be sent by the mesh stack.

Public Members

int **to_parent**

to parent queue

int **to_parent_p2p**

to parent (P2P) queue

int **to_child**

to child queue

int **to_child_p2p**

to child (P2P) queue

int **mgmt**

management queue

int **broadcast**

broadcast and multicast queue

struct **mesh_rx_pending_t**

The number of packets available in the queue waiting to be received by applications.

Public Members

int **toDS**

to external DS

int **toSelf**

to self

Macros

MESH_ROOT_LAYER

root layer value

MESH_MTU

max transmit unit(in bytes)

MESH_MPS

max payload size(in bytes)

ESP_ERR_MESH_WIFI_NOT_START

Mesh error code definition.

Wi-Fi isn't started

ESP_ERR_MESH_NOT_INIT
mesh isn't initialized

ESP_ERR_MESH_NOT_CONFIG
mesh isn't configured

ESP_ERR_MESH_NOT_START
mesh isn't started

ESP_ERR_MESH_NOT_SUPPORT
not supported yet

ESP_ERR_MESH_NOT_ALLOWED
operation is not allowed

ESP_ERR_MESH_NO_MEMORY
out of memory

ESP_ERR_MESH_ARGUMENT
illegal argument

ESP_ERR_MESH_EXCEED_MTU
packet size exceeds MTU

ESP_ERR_MESH_TIMEOUT
timeout

ESP_ERR_MESH_DISCONNECTED
disconnected with parent on station interface

ESP_ERR_MESH_QUEUE_FAIL
queue fail

ESP_ERR_MESH_QUEUE_FULL
queue full

ESP_ERR_MESH_NO_PARENT_FOUND
no parent found to join the mesh network

ESP_ERR_MESH_NO_ROUTE_FOUND
no route found to forward the packet

ESP_ERR_MESH_OPTION_NULL
no option found

ESP_ERR_MESH_OPTION_UNKNOWN
unknown option

ESP_ERR_MESH_XON_NO_WINDOW

no window for software flow control on upstream

ESP_ERR_MESH_INTERFACE

low-level Wi-Fi interface error

ESP_ERR_MESH_DISCARD_DUPLICATE

discard the packet due to the duplicate sequence number

ESP_ERR_MESH_DISCARD

discard the packet

ESP_ERR_MESH_VOTING

vote in progress

ESP_ERR_MESH_XMIT

XMIT

ESP_ERR_MESH_QUEUE_READ

error in reading queue

ESP_ERR_MESH_PS

mesh PS is not specified as enable or disable

ESP_ERR_MESH_RECV_RELEASE

release esp_mesh_recv_toDS

MESH_DATA_ENC

Flags bitmap for esp_mesh_send() and esp_mesh_recv()

data encrypted (Unimplemented)

MESH_DATA_P2P

point-to-point delivery over the mesh network

MESH_DATA_FROMDS

receive from external IP network

MESH_DATA_TODS

identify this packet is target to external IP network

MESH_DATA_NONBLOCK

esp_mesh_send() non-block

MESH_DATA_DROP

in the situation of the root having been changed, identify this packet can be dropped by new root

MESH_DATA_GROUP

identify this packet is target to a group address

MESH_OPT_SEND_GROUP

Option definitions for esp_mesh_send() and esp_mesh_recv()

data transmission by group; used with esp_mesh_send() and shall have payload

MESH_OPT_RECV_DS_ADDR

return a remote IP address; used with esp_mesh_send() and esp_mesh_recv()

MESH_ASSOC_FLAG_MAP_ASSOC

Flag of mesh networking IE.

Mesh AP doesn't detect children leave yet

MESH_ASSOC_FLAG_VOTE_IN_PROGRESS

station in vote, set when root vote start, clear when connect to router or when root switch

MESH_ASSOC_FLAG_STA_VOTED

station vote done, set when connect to router

MESH_ASSOC_FLAG_NETWORK_FREE

no root in current network

MESH_ASSOC_FLAG_STA_VOTE_EXPIRE

the voted address is expired, means the voted device lose the chance to be root

MESH_ASSOC_FLAG_ROOTS_FOUND

roots conflict is found, means that there are at least two roots in the mesh network

MESH_ASSOC_FLAG_ROOT_FIXED

the root is fixed in the mesh network

MESH_PS_DEVICE_DUTY_REQUEST

Mesh PS (Power Save) duty cycle type.

requests to join a network PS without specifying a device duty cycle. After the device joins the network, a network duty cycle will be provided by the network

MESH_PS_DEVICE_DUTY_DEMAND

requests to join a network PS and specifies a demanded device duty cycle

MESH_PS_NETWORK_DUTY_MASTER

indicates the device is the NWK-DUTY-MASTER (network duty cycle master)

MESH_PS_NETWORK_DUTY_APPLIED_ENTIRE

Mesh PS (Power Save) duty cycle applied rule.

MESH_PS_NETWORK_DUTY_APPLIED_UPLINK

MESH_INIT_CONFIG_DEFAULT()

Type Definitions

typedef *mesh_addr_t* **mesh_event_root_address_t**

Root address.

typedef wifi_event_sta_disconnected_t **mesh_event_disconnected_t**

Parent disconnected information.

typedef wifi_event_ap_staconnected_t **mesh_event_child_connected_t**

Child connected information.

typedef wifi_event_ap_stadisconnected_t **mesh_event_child_disconnected_t**

Child disconnected information.

typedef wifi_event_sta_connected_t **mesh_event_router_switch_t**

New router information.

Enumerations

enum **mesh_event_id_t**

Enumerated list of mesh event id.

Values:

enumerator **MESH_EVENT_STARTED**

mesh is started

enumerator **MESH_EVENT_STOPPED**

mesh is stopped

enumerator **MESH_EVENT_CHANNEL_SWITCH**

channel switch

enumerator **MESH_EVENT_CHILD_CONNECTED**

a child is connected on softAP interface

enumerator **MESH_EVENT_CHILD_DISCONNECTED**

a child is disconnected on softAP interface

enumerator **MESH_EVENT_ROUTING_TABLE_ADD**

routing table is changed by adding newly joined children

enumerator **MESH_EVENT_ROUTING_TABLE_REMOVE**

routing table is changed by removing leave children

enumerator **MESH_EVENT_PARENT_CONNECTED**

parent is connected on station interface

enumerator **MESH_EVENT_PARENT_DISCONNECTED**

parent is disconnected on station interface

enumerator **MESH_EVENT_NO_PARENT_FOUND**

no parent found

enumerator **MESH_EVENT_LAYER_CHANGE**

layer changes over the mesh network

enumerator **MESH_EVENT_TODS_STATE**

state represents whether the root is able to access external IP network. This state is a manual event that needs to be triggered with esp_mesh_post_toDS_state().

enumerator **MESH_EVENT_VOTE_STARTED**

the process of voting a new root is started either by children or by the root

enumerator **MESH_EVENT_VOTE_STOPPED**

the process of voting a new root is stopped

enumerator **MESH_EVENT_ROOT_ADDRESS**

the root address is obtained. It is posted by mesh stack automatically.

enumerator **MESH_EVENT_ROOT_SWITCH_REQ**

root switch request sent from a new voted root candidate

enumerator **MESH_EVENT_ROOT_SWITCH_ACK**

root switch acknowledgment responds the above request sent from current root

enumerator **MESH_EVENT_ROOT_ASKED_YIELD**

the root is asked yield by a more powerful existing root. If self organized is disabled and this device is specified to be a root by users, users should set a new parent for this device. if self organized is enabled, this device will find a new parent by itself, users could ignore this event.

enumerator **MESH_EVENT_ROOT_FIXED**

when devices join a network, if the setting of Fixed Root for one device is different from that of its parent, the device will update the setting the same as its parent's. Fixed Root Setting of each device is variable as that setting changes of the root.

enumerator **MESH_EVENT_SCAN_DONE**

if self-organized networking is disabled, user can call esp_wifi_scan_start() to trigger this event, and add the corresponding scan done handler in this event.

enumerator **MESH_EVENT_NETWORK_STATE**

network state, such as whether current mesh network has a root.

enumerator **MESH_EVENT_STOP_RECONNECTION**

the root stops reconnecting to the router and non-root devices stop reconnecting to their parents.

enumerator **MESH_EVENT_FIND_NETWORK**

when the channel field in mesh configuration is set to zero, mesh stack will perform a full channel scan to find a mesh network that can join, and return the channel value after finding it.

enumerator **MESH_EVENT_ROUTER_SWITCH**

if users specify BSSID of the router in mesh configuration, when the root connects to another router with the same SSID, this event will be posted and the new router information is attached.

enumerator **MESH_EVENT_PS_PARENT_DUTY**

parent duty

enumerator **MESH_EVENT_PS_CHILD_DUTY**

child duty

enumerator **MESH_EVENT_PS_DEVICE_DUTY**

device duty

enumerator **MESH_EVENT_MAX**

enum **mesh_type_t**

Device type.

Values:

enumerator **MESH_IDLE**

hasn't joined the mesh network yet

enumerator **MESH_ROOT**

the only sink of the mesh network. Has the ability to access external IP network

enumerator **MESH_NODE**

intermediate device. Has the ability to forward packets over the mesh network

enumerator **MESH_LEAF**

has no forwarding ability

enumerator **MESH_STA**

connect to router with a standalone Wi-Fi station mode, no network expansion capability

enum **mesh_proto_t**

Protocol of transmitted application data.

Values:

enumerator **MESH_PROTO_BIN**

binary

enumerator **MESH_PROTO_HTTP**

HTTP protocol

enumerator **MESH_PROTO_JSON**

JSON format

enumerator **MESH_PROTO_MQTT**

MQTT protocol

enumerator **MESH_PROTO_AP**

IP network mesh communication of node's AP interface

enumerator **MESH_PROTO_STA**

IP network mesh communication of node's STA interface

enum **mesh_tos_t**

For reliable transmission, mesh stack provides three type of services.

Values:

enumerator **MESH_TOS_P2P**

provide P2P (point-to-point) retransmission on mesh stack by default

enumerator **MESH_TOS_E2E**

provide E2E (end-to-end) retransmission on mesh stack (Unimplemented)

enumerator **MESH_TOS_DEF**

no retransmission on mesh stack

enum **mesh_vote_reason_t**

Vote reason.

Values:

enumerator **MESH_VOTE_REASON_ROOT_INITIATED**

vote is initiated by the root

enumerator **MESH_VOTE_REASON_CHILD_INITIATED**

vote is initiated by children

enum **mesh_disconnect_reason_t**

Mesh disconnect reason code.

Values:

enumerator **MESH_REASON_CYCLIC**

cyclic is detected

enumerator **MESH_REASON_PARENT_IDLE**

parent is idle

enumerator **MESH_REASON_LEAF**

the connected device is changed to a leaf

enumerator **MESH_REASON_DIFF_ID**

in different mesh ID

enumerator **MESH_REASON_ROOTS**

root conflict is detected

enumerator **MESH_REASON_PARENT_STOPPED**

parent has stopped the mesh

enumerator **MESH_REASON_SCAN_FAIL**

scan fail

enumerator **MESH_REASON_IE_UNKNOWN**

unknown IE

enumerator **MESH_REASON_WAIVE_ROOT**

waive root

enumerator **MESH_REASON_PARENT_WORSE**

parent with very poor RSSI

enumerator **MESH_REASON_EMPTY_PASSWORD**

use an empty password to connect to an encrypted parent

enumerator **MESH_REASON_PARENT_UNENCRYPTED**

connect to an unencrypted parent/router

enum **esp_mesh_topology_t**

Mesh topology.

Values:

enumerator **MESH_TOPO_TREE**

tree topology

enumerator **MESH_TOPO_CHAIN**

chain topology

enum **mesh_event_toDS_state_t**

The reachability of the root to a DS (distribute system)

Values:

enumerator **MESH_TODS_UNREACHABLE**

the root isn't able to access external IP network

enumerator **MESH_TODS_REACHABLE**

the root is able to access external IP network

SmartConfig

The SmartConfig™ is a provisioning technology developed by TI to connect a new Wi-Fi device to a Wi-Fi network. It uses a mobile application to broadcast the network credentials from a smartphone, or a tablet, to an un-provisioned Wi-Fi device.

The advantage of this technology is that the device does not need to directly know SSID or password of an Access Point (AP). This information is provided using the smartphone. This is particularly important to headless device and systems, due to their lack of a user interface.

If you are looking for other options to provision your ESP32-S3 devices, check *Provisioning API*.

Application Example Connect ESP32-S3 to the target AP using SmartConfig: wifi/smart_config.

API Reference

Header File

- components/esp_wifi/include/esp_smartconfig.h
- This header file can be included with:

```
#include "esp_smartconfig.h"
```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_wifi
```

or

```
PRIV_REQUIRES esp_wifi
```

Functions

const char ***esp_smartconfig_get_version**(void)

Get the version of SmartConfig.

Returns
- SmartConfig version const char.

esp_err_t **esp_smartconfig_start**(const *smartconfig_start_config_t* *config)

Start SmartConfig, config ESP device to connect AP. You need to broadcast information by phone APP. Device sniffer special packets from the air that containing SSID and password of target AP.

Attention 1. This API can be called in station or softAP-station mode.
Attention 2. Can not call esp_smartconfig_start twice before it finish, please call esp_smartconfig_stop first.

Parameters **config** -- pointer to smartconfig start configure structure
Returns
- ESP_OK: succeed
- others: fail

esp_err_t **esp_smartconfig_stop**(void)

Stop SmartConfig, free the buffer taken by esp_smartconfig_start.

Attention Whether connect to AP succeed or not, this API should be called to free memory taken by smartconfig_start.

Returns
- ESP_OK: succeed
- others: fail

esp_err_t **esp_esptouch_set_timeout** (uint8_t time_s)

Set timeout of SmartConfig process.

Attention Timing starts from SC_STATUS_FIND_CHANNEL status. SmartConfig will restart if timeout.

Parameters **time_s** -- range 15s~255s, offset:45s.

Returns

- ESP_OK: succeed
- others: fail

esp_err_t **esp_smartconfig_set_type** (*smartconfig_type_t* type)

Set protocol type of SmartConfig.

Attention If users need to set the SmartConfig type, please set it before calling esp_smartconfig_start.

Parameters **type** -- Choose from the smartconfig_type_t.

Returns

- ESP_OK: succeed
- others: fail

esp_err_t **esp_smartconfig_fast_mode** (bool enable)

Set mode of SmartConfig. default normal mode.

Attention 1. Please call it before API esp_smartconfig_start.
Attention 2. Fast mode have corresponding APP(phone).
Attention 3. Two mode is compatible.

Parameters **enable** -- false-disable(default); true-enable;

Returns

- ESP_OK: succeed
- others: fail

esp_err_t **esp_smartconfig_get_rvd_data** (uint8_t *rvd_data, uint8_t len)

Get reserved data of ESPTouch v2.

Parameters

- **rvd_data** -- reserved data
- **len** -- length of reserved data

Returns

- ESP_OK: succeed
- others: fail

Structures

struct **smartconfig_event_got_ssid_pswd_t**

Argument structure for SC_EVENT_GOT_SSID_PSWD event

Public Members

uint8_t **ssid**[32]

SSID of the AP. Null terminated string.

uint8_t **password**[64]

Password of the AP. Null terminated string.

bool **bssid_set**

whether set MAC address of target AP or not.

uint8_t **bssid**[6]

MAC address of target AP.

smartconfig_type_t **type**

Type of smartconfig(ESPTouch or AirKiss).

uint8_t **token**

Token from cellphone which is used to send ACK to cellphone.

uint8_t **cellphone_ip**[4]

IP address of cellphone.

struct **smartconfig_start_config_t**

Configure structure for esp_smartconfig_start

Public Members

bool **enable_log**

Enable smartconfig logs.

bool **esp_touch_v2_enable_crypt**

Enable ESPTouch v2 crypt.

char ***esp_touch_v2_key**

ESPTouch v2 crypt key, len should be 16.

Macros

SMARTCONFIG_START_CONFIG_DEFAULT()

Enumerations

enum **smartconfig_type_t**

Values:

enumerator **SC_TYPE_ESPTOUCH**

protocol: ESPTouch

enumerator **SC_TYPE_AIRKISS**

protocol: AirKiss

enumerator **SC_TYPE_ESPTOUCH_AIRKISS**

protocol: ESPTouch and AirKiss

enumerator **SC_TYPE_ESPTOUCH_V2**

protocol: ESPTouch v2

enum **smartconfig_event_t**

Smartconfig event declarations

Values:

enumerator **SC_EVENT_SCAN_DONE**

Station smartconfig has finished to scan for APs

enumerator **SC_EVENT_FOUND_CHANNEL**

Station smartconfig has found the channel of the target AP

enumerator **SC_EVENT_GOT_SSID_PSWD**

Station smartconfig got the SSID and password

enumerator **SC_EVENT_SEND_ACK_DONE**

Station smartconfig has sent ACK to cellphone

Wi-Fi

Introduction The Wi-Fi libraries provide support for configuring and monitoring the ESP32-S3 Wi-Fi networking functionality. This includes configuration for:

- Station mode (aka STA mode or Wi-Fi client mode). ESP32-S3 connects to an access point.
- AP mode (aka Soft-AP mode or Access Point mode). Stations connect to the ESP32-S3.
- Station/AP-coexistence mode (ESP32-S3 is concurrently an access point and a station connected to another access point).
- Various security modes for the above (WPA, WPA2, WPA3, etc.)
- Scanning for access points (active & passive scanning).
- Promiscuous mode for monitoring of IEEE802.11 Wi-Fi packets.

Application Examples Several application examples demonstrating the functionality of Wi-Fi library are provided in wifi directory of ESP-IDF repository. Please check the README for more details.

API Reference

Header File

- components/esp_wifi/include/esp_wifi.h
- This header file can be included with:

```
#include "esp_wifi.h"
```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_wifi
```

or

```
PRIV_REQUIRES esp_wifi
```

Functions

esp_err_t **esp_wifi_init** (const *wifi_init_config_t* *config)

Initialize WiFi Allocate resource for WiFi driver, such as WiFi control structure, RX/TX buffer, WiFi NVS structure etc. This WiFi also starts WiFi task.

Attention 1. This API must be called before all other WiFi API can be called

Attention 2. Always use WIFI_INIT_CONFIG_DEFAULT macro to initialize the configuration to default values, this can guarantee all the fields get correct value when more fields are added into *wifi_init_config_t* in future release. If you want to set your own initial values, overwrite the default values which are set by WIFI_INIT_CONFIG_DEFAULT. Please be notified that the field 'magic' of *wifi_init_config_t* should always be WIFI_INIT_CONFIG_MAGIC!

Parameters **config** -- pointer to WiFi initialized configuration structure; can point to a temporary variable.

Returns

- ESP_OK: succeed
- ESP_ERR_NO_MEM: out of memory
- others: refer to error code esp_err.h

esp_err_t **esp_wifi_deinit** (void)

Deinit WiFi Free all resource allocated in esp_wifi_init and stop WiFi task.

Attention 1. This API should be called if you want to remove WiFi driver from the system

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_set_mode** (wifi_mode_t mode)

Set the WiFi operating mode.

```
Set the WiFi operating mode as station, soft-AP, station+soft-AP or NAN.
The default mode is station mode.
```

Parameters **mode** -- WiFi operating mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- others: refer to error code in esp_err.h

esp_err_t **esp_wifi_get_mode** (wifi_mode_t *mode)

Get current operating mode of WiFi.

Parameters **mode** -- **[out]** store current WiFi mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_start**(void)

Start WiFi according to current configuration If mode is WIFI_MODE_STA, it creates station control block and starts station If mode is WIFI_MODE_AP, it creates soft-AP control block and starts soft-AP If mode is WIFI_MODE_APSTA, it creates soft-AP and station control block and starts soft-AP and station If mode is WIFI_MODE_NAN, it creates NAN control block and starts NAN.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: It doesn't normally happen, the function called inside the API was passed invalid argument, user should check if the wifi related config is correct
- ESP_ERR_NO_MEM: out of memory
- ESP_ERR_WIFI_CONN: WiFi internal error, station or soft-AP control block wrong
- ESP_FAIL: other WiFi internal errors

esp_err_t **esp_wifi_stop**(void)

Stop WiFi If mode is WIFI_MODE_STA, it stops station and frees station control block If mode is WIFI_MODE_AP, it stops soft-AP and frees soft-AP control block If mode is WIFI_MODE_APSTA, it stops station/soft-AP and frees station/soft-AP control block If mode is WIFI_MODE_NAN, it stops NAN and frees NAN control block.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_restore**(void)

Restore WiFi stack persistent settings to default values.

This function will reset settings made using the following APIs:

- esp_wifi_set_bandwidth,
- esp_wifi_set_protocol,
- esp_wifi_set_config related
- esp_wifi_set_mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_connect**(void)

Connect WiFi station to the AP.

Attention 1. This API only impact WIFI_MODE_STA or WIFI_MODE_APSTA mode

Attention 2. If station interface is connected to an AP, call esp_wifi_disconnect to disconnect.

Attention 3. The scanning triggered by esp_wifi_scan_start() will not be effective until connection between device and the AP is established. If device is scanning and connecting at the same time, it will abort scanning and return a warning message and error number ESP_ERR_WIFI_STATE.

Attention 4. This API attempts to connect to an Access Point (AP) only once. To enable reconnection in case of a connection failure, please use the 'failure_retry_cnt' feature in the 'wifi_sta_config_t'. Users are suggested to implement reconnection logic in their application for scenarios where the specified AP does not exist, or reconnection is desired after the device has received a disconnect event.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_WIFI_MODE: WiFi mode error
- ESP_ERR_WIFI_CONN: WiFi internal error, station or soft-AP control block wrong
- ESP_ERR_WIFI_SSID: SSID of AP which station connects is invalid

esp_err_t **esp_wifi_disconnect**(void)

Disconnect WiFi station from the AP.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi was not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi was not started by esp_wifi_start
- ESP_FAIL: other WiFi internal errors

esp_err_t **esp_wifi_clear_fast_connect**(void)

Currently this API is just an stub API.

Returns

- ESP_OK: succeed
- others: fail

esp_err_t **esp_wifi_deauth_sta**(uint16_t aid)

deauthenticate all stations or associated id equals to aid

Parameters **aid** -- when aid is 0, deauthenticate all stations, otherwise deauthenticate station whose associated id is aid

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi was not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_MODE: WiFi mode is wrong

esp_err_t **esp_wifi_scan_start**(const wifi_scan_config_t *config, bool block)

Scan all available APs.

Attention If this API is called, the found APs are stored in WiFi driver dynamic allocated memory. And then can be freed in esp_wifi_scan_get_ap_records(), esp_wifi_scan_get_ap_record() or esp_wifi_clear_ap_list(), so call any one to free the memory once the scan is done.

Attention The values of maximum active scan time and passive scan time per channel are limited to 1500 milliseconds. Values above 1500ms may cause station to disconnect from AP and are not recommended.

Parameters

- **config** -- configuration settings for scanning, if set to NULL default settings will be used of which default values are show_hidden:false, scan_type:active, scan_time.active.min:0, scan_time.active.max:120 milliseconds, scan_time.passive:360 milliseconds home_chan_dwell_time:30ms
- **block** -- if block is true, this API will block the caller until the scan is done, otherwise it will return immediately

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi was not started by esp_wifi_start
- ESP_ERR_WIFI_TIMEOUT: blocking scan is timeout
- ESP_ERR_WIFI_STATE: wifi still connecting when invoke esp_wifi_scan_start
- others: refer to error code in esp_err.h

esp_err_t **esp_wifi_set_scan_parameters**(const wifi_scan_default_params_t *config)

Set default parameters used for scanning by station.

Attention The values set using this API are also used for scans used while connecting.

Attention The values of maximum active scan time and passive scan time per channel are limited to 1500 milliseconds.

Attention The home_chan_dwell_time needs to be a minimum of 30ms and a maximum of 150ms.
Attention Set any of the parameters to 0 to indicate using the default parameters - scan_time.active.min : 0ms, scan_time.active.max : 120ms home_chan_dwell_time : 30ms scan_time.passive : 360ms
Attention Default values can be retrieved using the macro WIFI_SCAN_PARAMS_DEFAULT_CONFIG()
Attention Set the config parameter to NULL to reset previously set scan parameters to their default values.

Parameters **`config`** -- default configuration settings for all scans by stations
Returns
- ESP_OK: succeed
- ESP_FAIL: failed as station mode has not been started yet
- ESP_ERR_INVALID_ARG: values provided do not satisfy the requirements
- ESP_ERR_NOT_SUPPORTED: This API is not supported in AP mode yet
- ESP_ERR_INVALID_STATE: a scan/connect is in progress right now, cannot change scan parameters
- others: refer to error code in esp_err.h

esp_err_t **`esp_wifi_get_scan_parameters`**(wifi_scan_default_params_t *config)

Get default parameters used for scanning by station.

Parameters **`config`** -- structure variable within which scan default params will be stored
Returns
- ESP_OK: succeed
- ESP_ERR_INVALID_ARG: passed parameter does not point to a valid memory
- others: refer to error code in esp_err.h

esp_err_t **`esp_wifi_scan_stop`**(void)

Stop the scan in process.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start

esp_err_t **`esp_wifi_scan_get_ap_num`**(uint16_t *number)

Get number of APs found in last scan.

Attention This API can only be called when the scan is completed, otherwise it may get wrong value.

Parameters **`number`** -- **[out]** store number of APs found in last scan
Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **`esp_wifi_scan_get_ap_records`**(uint16_t *number, wifi_ap_record_t *ap_records)

Get AP list found in last scan.

Attention This API will free all memory occupied by scanned AP list.

Parameters
- **`number`** -- **[inout]** As input param, it stores max AP number ap_records can hold. As output param, it receives the actual AP number this API returns.
- **`ap_records`** -- wifi_ap_record_t array to hold the found APs

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_NO_MEM: out of memory

esp_err_t **esp_wifi_scan_get_ap_record**(wifi_ap_record_t *ap_record)

Get one AP record from the scanned AP list.

Attention Different from esp_wifi_scan_get_ap_records(), this API only gets one AP record from the scanned AP list each time. This API will free the memory of one AP record, if the user doesn't get all records in the scannned AP list, then needs to call esp_wifi_clear_ap_list() to free the remaining memory.

Parameters **ap_record** -- **[out]** pointer to one AP record

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_FAIL: scan APs is NULL, means all AP records fetched or no AP found

esp_err_t **esp_wifi_clear_ap_list**(void)

Clear AP list found in last scan.

Attention This API will free all memory occupied by scanned AP list. When the obtained AP list fails, AP records must be cleared,otherwise it may cause memory leakage.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_WIFI_MODE: WiFi mode is wrong
- ESP_ERR_INVALID_ARG: It doesn't normally happen, the function called inside the API was passed invalid argument, user should check if the wifi related config is correct

esp_err_t **esp_wifi_sta_get_ap_info**(wifi_ap_record_t *ap_info)

Get information of AP to which the device is associated with.

Attention When the obtained country information is empty, it means that the AP does not carry country information

Parameters **ap_info** -- the wifi_ap_record_t to hold AP information sta can get the connected ap's phy mode info through the struct member phy_11b, phy_11g, phy_11n, phy_lr in the wifi_ap_record_t struct. For example, phy_11b = 1 imply that ap support 802.11b mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_CONN: The station interface don't initialized
- ESP_ERR_WIFI_NOT_CONNECT: The station is in disconnect status

esp_err_t **esp_wifi_set_ps**(wifi_ps_type_t type)

Set current WiFi power save type.

Attention Default power save type is WIFI_PS_MIN_MODEM.

Parameters **type** -- power save type

Returns ESP_OK: succeed

esp_err_t **esp_wifi_get_ps**(wifi_ps_type_t *type)

Get current WiFi power save type.

Attention Default power save type is WIFI_PS_MIN_MODEM.

Parameters **type** -- **[out]** store current power save type
Returns ESP_OK: succeed

esp_err_t **esp_wifi_set_protocol**(wifi_interface_t ifx, uint8_t protocol_bitmap)

Set protocol type of specified interface The default protocol is (WIFI_PROTOCOL_11B|WIFI_PROTOCOL_11G|WIFI_PROT
if CONFIG_SOC_WIFI_HE_SUPPORT and band is 2.4G, the default protocol is (WIFI_PROTOCOL_11B|WIFI_PROTOCOL_11G|WIFI_PROTOCOL_11N|WIFI_PROTOCOL_11AX).
if CONFIG_SOC_WIFI_HE_SUPPORT and band is 5G, the default protocol is (WIFI_PROTOCOL_11A|WIFI_PROTOCOL_11N|WIFI_PROTOCOL_11AC|WIFI_PROTOCOL_11AX).

Attention 2.4G: Support 802.11b or 802.11bg or 802.11bgn or 802.11bgnax or LR mode 5G: Support 802.11a or 802.11an or 802.11anac or 802.11anacax

Parameters
- **ifx** -- interfaces
- **protocol_bitmap** -- WiFi protocol bitmap

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_IF: invalid interface
- others: refer to error codes in esp_err.h

esp_err_t **esp_wifi_get_protocol**(wifi_interface_t ifx, uint8_t *protocol_bitmap)

Get the current protocol bitmap of the specified interface.

Parameters
- **ifx** -- interface
- **protocol_bitmap** -- **[out]** store current WiFi protocol bitmap of interface ifx

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_IF: invalid interface
- ESP_ERR_INVALID_ARG: invalid argument
- others: refer to error codes in esp_err.h

esp_err_t **esp_wifi_set_bandwidth**(wifi_interface_t ifx, wifi_bandwidth_t bw)

Set the bandwidth of specified interface.

Attention 1. API return false if try to configure an interface that is not enabled
Attention 2. WIFI_BW_HT40 is supported only when the interface support 11N

Parameters
- **ifx** -- interface to be configured
- **bw** -- bandwidth

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_IF: invalid interface

- ESP_ERR_INVALID_ARG: invalid argument
- others: refer to error codes in esp_err.h

esp_err_t **esp_wifi_get_bandwidth**(wifi_interface_t ifx, wifi_bandwidth_t *bw)

Get the bandwidth of specified interface.

Attention 1. API return false if try to get a interface that is not enable

Parameters
- **ifx** -- interface to be configured
- **bw** -- **[out]** store bandwidth of interface ifx

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_IF: invalid interface
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_channel**(uint8_t primary, wifi_second_chan_t second)

Set primary/secondary channel of device.

Attention 1. This API should be called after esp_wifi_start() and before esp_wifi_stop()

Attention 2. When device is in STA mode, this API should not be called when STA is scanning or connecting to an external AP

Attention 3. When device is in softAP mode, this API should not be called when softAP has connected to external STAs

Attention 4. When device is in STA+softAP mode, this API should not be called when in the scenarios described above

Attention 5. The channel info set by this API will not be stored in NVS. So If you want to remember the channel used before wifi stop, you need to call this API again after wifi start, or you can call `esp_wifi_set_config()` to store the channel info in NVS.

Parameters
- **primary** -- for HT20, primary is the channel number, for HT40, primary is the primary channel
- **second** -- for HT20, second is ignored, for HT40, second is the second channel

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_IF: invalid interface
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start

esp_err_t **esp_wifi_get_channel**(uint8_t *primary, wifi_second_chan_t *second)

Get the primary/secondary channel of device.

Attention 1. API return false if try to get a interface that is not enable

Parameters
- **primary** -- store current primary channel
- **second** -- **[out]** store current second channel

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_country**(const wifi_country_t *country)

configure country info

Attention 1. It is discouraged to call this API since this doesn't validate the per-country rules, it's up to the user to fill in all fields according to local regulations. Please use esp_wifi_set_country_code instead.

Attention 2. The default country is "01" (world safe mode) {.cc="01", .schan=1, .nchan=11, .policy=WIFI_COUNTRY_POLICY_AUTO}.

Attention 3. The third octet of country code string is one of the following: ' ', 'O', 'I', 'X', otherwise it is considered as ' '.

Attention 4. When the country policy is WIFI_COUNTRY_POLICY_AUTO, the country info of the AP to which the station is connected is used. E.g. if the configured country info is {.cc="US", .schan=1, .nchan=11} and the country info of the AP to which the station is connected is {.cc="JP", .schan=1, .nchan=14} then the country info that will be used is {.cc="JP", .schan=1, .nchan=14}. If the station disconnected from the AP the country info is set back to the country info of the station automatically, {.cc="US", .schan=1, .nchan=11} in the example.

Attention 5. When the country policy is WIFI_COUNTRY_POLICY_MANUAL, then the configured country info is used always.

Attention 6. When the country info is changed because of configuration or because the station connects to a different external AP, the country IE in probe response/beacon of the soft-AP is also changed.

Attention 7. The country configuration is stored into flash.

Attention 8. When this API is called, the PHY init data will switch to the PHY init data type corresponding to the country info.

Parameters **country** -- the configured country info

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_get_country**(wifi_country_t *country)

get the current country info

Parameters **country** -- country info

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_mac**(wifi_interface_t ifx, const uint8_t mac[6])

Set MAC address of WiFi station, soft-AP or NAN interface.

Attention 1. This API can only be called when the interface is disabled

Attention 2. Above mentioned interfaces have different MAC addresses, do not set them to be the same.

Attention 3. The bit 0 of the first byte of MAC address can not be 1. For example, the MAC address can set to be "1a:XX:XX:XX:XX:XX", but can not be "15:XX:XX:XX:XX:XX".

Parameters

- **ifx** -- interface
- **mac** -- the MAC address

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_IF: invalid interface
- ESP_ERR_WIFI_MAC: invalid mac address
- ESP_ERR_WIFI_MODE: WiFi mode is wrong

- others: refer to error codes in esp_err.h

esp_err_t **esp_wifi_get_mac**(wifi_interface_t ifx, uint8_t mac[6])

Get mac of specified interface.

Parameters
- **ifx** -- interface
- **mac** -- **[out]** store mac of the interface ifx

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_IF: invalid interface

esp_err_t **esp_wifi_set_promiscuous_rx_cb**(*wifi_promiscuous_cb_t* cb)

Register the RX callback function in the promiscuous mode.

Each time a packet is received, the registered callback function will be called.

Parameters **cb** -- callback

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_set_promiscuous**(bool en)

Enable the promiscuous mode.

Parameters **en** -- false - disable, true - enable

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_get_promiscuous**(bool *en)

Get the promiscuous mode.

Parameters **en** -- **[out]** store the current status of promiscuous mode

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_promiscuous_filter**(const wifi_promiscuous_filter_t *filter)

Enable the promiscuous mode packet type filter.

Note: The default filter is to filter all packets except WIFI_PKT_MISC

Parameters **filter** -- the packet type filtered in promiscuous mode.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_get_promiscuous_filter**(wifi_promiscuous_filter_t *filter)

Get the promiscuous filter.

Parameters **filter** -- **[out]** store the current status of promiscuous filter

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_promiscuous_ctrl_filter**(const wifi_promiscuous_filter_t *filter)

Enable subtype filter of the control packet in promiscuous mode.

Note: The default filter is to filter none control packet.

Parameters **filter** -- the subtype of the control packet filtered in promiscuous mode.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_get_promiscuous_ctrl_filter**(wifi_promiscuous_filter_t *filter)

Get the subtype filter of the control packet in promiscuous mode.

Parameters **filter** -- **[out]** store the current status of subtype filter of the control packet in promiscuous mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_config**(wifi_interface_t interface, wifi_config_t *conf)

Set the configuration of the STA, AP or NAN.

Attention 1. This API can be called only when specified interface is enabled, otherwise, API fail

Attention 2. For station configuration, bssid_set needs to be 0; and it needs to be 1 only when users need to check the MAC address of the AP.

Attention 3. ESP devices are limited to only one channel, so when in the soft-AP+station mode, the soft-AP will adjust its channel automatically to be the same as the channel of the station.

Attention 4. The configuration will be stored in NVS for station and soft-AP

Parameters

- **interface** -- interface
- **conf** -- station, soft-AP or NAN configuration

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_IF: invalid interface
- ESP_ERR_WIFI_MODE: invalid mode
- ESP_ERR_WIFI_PASSWORD: invalid password
- ESP_ERR_WIFI_NVS: WiFi internal NVS error
- others: refer to the error code in esp_err.h

esp_err_t **esp_wifi_get_config**(wifi_interface_t interface, wifi_config_t *conf)

Get configuration of specified interface.

Parameters

- **interface** -- interface
- **conf** -- **[out]** station or soft-AP configuration

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_IF: invalid interface

esp_err_t **esp_wifi_ap_get_sta_list**(*wifi_sta_list_t* *sta)

Get STAs associated with soft-AP.

Attention SSC only API

Parameters **sta** -- **[out]** station list ap can get the connected sta's phy mode info through the struct member phy_11b, phy_11g, phy_11n, phy_lr in the wifi_sta_info_t struct. For example, phy_11b = 1 imply that sta support 802.11b mode

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_WIFI_MODE: WiFi mode is wrong
- ESP_ERR_WIFI_CONN: WiFi internal error, the station/soft-AP control block is invalid

esp_err_t **esp_wifi_ap_get_sta_aid**(const uint8_t mac[6], uint16_t *aid)

Get AID of STA connected with soft-AP.

Parameters

- **mac** -- STA's mac address
- **aid** -- **[out]** Store the AID corresponding to STA mac

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_NOT_FOUND: Requested resource not found
- ESP_ERR_WIFI_MODE: WiFi mode is wrong
- ESP_ERR_WIFI_CONN: WiFi internal error, the station/soft-AP control block is invalid

esp_err_t **esp_wifi_set_storage**(wifi_storage_t storage)

Set the WiFi API configuration storage type.

Attention 1. The default value is WIFI_STORAGE_FLASH

Parameters **storage** -- : storage type

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_vendor_ie**(bool enable, wifi_vendor_ie_type_t type, wifi_vendor_ie_id_t idx, const void *vnd_ie)

Set 802.11 Vendor-Specific Information Element.

Parameters

- **enable** -- If true, specified IE is enabled. If false, specified IE is removed.
- **type** -- Information Element type. Determines the frame type to associate with the IE.
- **idx** -- Index to set or clear. Each IE type can be associated with up to two elements (indices 0 & 1).
- **vnd_ie** -- Pointer to vendor specific element data. First 6 bytes should be a header with fields matching vendor_ie_data_t. If enable is false, this argument is ignored and can be NULL. Data does not need to remain valid after the function returns.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init()
- ESP_ERR_INVALID_ARG: Invalid argument, including if first byte of vnd_ie is not WIFI_VENDOR_IE_ELEMENT_ID (0xDD) or second byte is an invalid length.

- ESP_ERR_NO_MEM: Out of memory

esp_err_t **esp_wifi_set_vendor_ie_cb**(*esp_vendor_ie_cb_t* cb, void *ctx)

Register Vendor-Specific Information Element monitoring callback.

Parameters
- **cb** -- Callback function
- **ctx** -- Context argument, passed to callback function.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_set_max_tx_power**(int8_t power)

Set maximum transmitting power after WiFi start.

Attention 1. Maximum power before wifi startup is limited by PHY init data bin.

Attention 2. The value set by this API will be mapped to the max_tx_power of the structure wifi_country_t variable.

Attention 3. Mapping Table {Power, max_tx_power} = {{8, 2}, {20, 5}, {28, 7}, {34, 8}, {44, 11}, {52, 13}, {56, 14}, {60, 15}, {66, 16}, {72, 18}, {80, 20}}.

Attention 4. Param power unit is 0.25dBm, range is [8, 84] corresponding to 2dBm - 20dBm.

Attention 5. Relationship between set value and actual value. As follows: {set value range, actual value} = {{[8, 19],8}, {[20, 27],20}, {[28, 33],28}, {[34, 43],34}, {[44, 51],44}, {[52, 55],52}, {[56, 59],56}, {[60, 65],60}, {[66, 71],66}, {[72, 79],72}, {[80, 84],80}}.

Parameters **power** -- Maximum WiFi transmitting power.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument, e.g. parameter is out of range

esp_err_t **esp_wifi_get_max_tx_power**(int8_t *power)

Get maximum transmitting power after WiFi start.

Parameters **power** -- Maximum WiFi transmitting power, unit is 0.25dBm.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_set_event_mask**(uint32_t mask)

Set mask to enable or disable some WiFi events.

Attention 1. Mask can be created by logical OR of various WIFI_EVENT_MASK_ constants. Events which have corresponding bit set in the mask will not be delivered to the system event handler.

Attention 2. Default WiFi event mask is WIFI_EVENT_MASK_AP_PROBEREQRECVED.

Attention 3. There may be lots of stations sending probe request data around. Don't unmask this event unless you need to receive probe request data.

Parameters **mask** -- WiFi event mask.

Returns
- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_get_event_mask**(uint32_t *mask)

Get mask of WiFi events.

Parameters **mask** -- WiFi event mask.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_80211_tx**(wifi_interface_t ifx, const void *buffer, int len, bool en_sys_seq)

Send raw ieee80211 data.

Attention Currently only support for sending beacon/probe request/probe response/action and non-QoS data frame

Parameters

- **ifx** -- interface if the Wi-Fi mode is Station, the ifx should be WIFI_IF_STA. If the Wi-Fi mode is SoftAP, the ifx should be WIFI_IF_AP. If the Wi-Fi mode is Station+SoftAP, the ifx should be WIFI_IF_STA or WIFI_IF_AP. If the ifx is wrong, the API returns ESP_ERR_WIFI_IF.
- **buffer** -- raw ieee80211 buffer
- **len** -- the length of raw buffer, the len must be <= 1500 Bytes and >= 24 Bytes
- **en_sys_seq** -- indicate whether use the internal sequence number. If en_sys_seq is false, the sequence in raw buffer is unchanged, otherwise it will be overwritten by WiFi driver with the system sequence number. Generally, if esp_wifi_80211_tx is called before the Wi-Fi connection has been set up, both en_sys_seq==true and en_sys_seq==false are fine. However, if the API is called after the Wi-Fi connection has been set up, en_sys_seq must be true, otherwise ESP_ERR_INVALID_ARG is returned.

Returns

- ESP_OK: success
- ESP_ERR_WIFI_IF: Invalid interface
- ESP_ERR_INVALID_ARG: Invalid parameter
- ESP_ERR_WIFI_NO_MEM: out of memory

esp_err_t **esp_wifi_set_csi_rx_cb**(*wifi_csi_cb_t* cb, void *ctx)

Register the RX callback function of CSI data.

```
Each time a CSI data is received, the callback function will be called.
```

Parameters

- **cb** -- callback
- **ctx** -- context argument, passed to callback function

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init

esp_err_t **esp_wifi_set_csi_config**(const *wifi_csi_config_t* *config)

Set CSI data configuration.

return

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start or promiscuous mode is not enabled
- ESP_ERR_INVALID_ARG: invalid argument

Parameters **config** -- configuration

esp_err_t **esp_wifi_get_csi_config**(*wifi_csi_config_t* *config)

Get CSI data configuration.

return

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start or promiscuous mode is not enabled
- ESP_ERR_INVALID_ARG: invalid argument

Parameters **config** -- configuration

esp_err_t **esp_wifi_set_csi**(bool en)

Enable or disable CSI.

return

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start or promiscuous mode is not enabled
- ESP_ERR_INVALID_ARG: invalid argument

Parameters **en** -- true - enable, false - disable

esp_err_t **esp_wifi_set_ant_gpio**(const wifi_ant_gpio_config_t *config)

Set antenna GPIO configuration.

Parameters **config** -- Antenna GPIO configuration.
Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: Invalid argument, e.g. parameter is NULL, invalid GPIO number etc

esp_err_t **esp_wifi_get_ant_gpio**(wifi_ant_gpio_config_t *config)

Get current antenna GPIO configuration.

Parameters **config** -- Antenna GPIO configuration.
Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument, e.g. parameter is NULL

esp_err_t **esp_wifi_set_ant**(const wifi_ant_config_t *config)

Set antenna configuration.

Parameters **config** -- Antenna configuration.
Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: Invalid argument, e.g. parameter is NULL, invalid antenna mode or invalid GPIO number

esp_err_t **esp_wifi_get_ant**(wifi_ant_config_t *config)

Get current antenna configuration.

Parameters **config** -- Antenna configuration.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument, e.g. parameter is NULL

int64_t **esp_wifi_get_tsf_time**(wifi_interface_t interface)

Get the TSF time In Station mode or SoftAP+Station mode if station is not connected or station doesn't receive at least one beacon after connected, will return 0.

Attention Enabling power save may cause the return value inaccurate, except WiFi modem sleep

Parameters **interface** -- The interface whose tsf_time is to be retrieved.

Returns 0 or the TSF time

esp_err_t **esp_wifi_set_inactive_time**(wifi_interface_t ifx, uint16_t sec)

Set the inactive time of the STA or AP.

Attention 1. For Station, If the station does not receive a beacon frame from the connected SoftAP during the inactive time, disconnect from SoftAP. Default 6s.

Attention 2. For SoftAP, If the softAP doesn't receive any data from the connected STA during inactive time, the softAP will force deauth the STA. Default is 300s.

Attention 3. The inactive time configuration is not stored into flash

Parameters

- **ifx** -- interface to be configured.
- **sec** -- Inactive time. Unit seconds.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument, For Station, if sec is less than 3. For SoftAP, if sec is less than 10.

esp_err_t **esp_wifi_get_inactive_time**(wifi_interface_t ifx, uint16_t *sec)

Get inactive time of specified interface.

Parameters

- **ifx** -- Interface to be configured.
- **sec** -- Inactive time. Unit seconds.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_statis_dump**(uint32_t modules)

Dump WiFi statistics.

Parameters **modules** -- statistic modules to be dumped

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_set_rssi_threshold**(int32_t rssi)

Set RSSI threshold, if average rssi gets lower than threshold, WiFi task will post event WIFI_EVENT_STA_BSS_RSSI_LOW.

Attention If the user wants to receive another WIFI_EVENT_STA_BSS_RSSI_LOW event after receiving one, this API needs to be called again with an updated/same RSSI threshold.

Parameters **rssi** -- threshold value in dbm between -100 to 10 Note that in some rare cases where signal strength is very strong, rssi values can be slightly positive.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_ftm_initiate_session**(wifi_ftm_initiator_cfg_t *cfg)

Start an FTM Initiator session by sending FTM request If successful, event WIFI_EVENT_FTM_REPORT is generated with the result of the FTM procedure.

Attention 1. Use this API only in Station mode.

Attention 2. If FTM is initiated on a different channel than Station is connected in or internal SoftAP is started in, FTM defaults to a single burst in ASAP mode.

Parameters **cfg** -- FTM Initiator session configuration

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_ftm_end_session**(void)

End the ongoing FTM Initiator session.

Attention This API works only on FTM Initiator

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_ftm_resp_set_offset**(int16_t offset_cm)

Set offset in cm for FTM Responder. An equivalent offset is calculated in picoseconds and added in TOD of FTM Measurement frame (T1).

Attention Use this API only in AP mode before performing FTM as responder

Parameters **offset_cm** -- T1 Offset to be added in centimeters

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_ftm_get_report**(wifi_ftm_report_entry_t *report, uint8_t num_entries)

Get FTM measurements report copied into a user provided buffer.

Attention 1. To get the FTM report, user first needs to allocate a buffer of size (sizeof(wifi_ftm_report_entry_t) * num_entries) where the API will fill up to num_entries valid FTM measurements in the buffer. Total number of entries can be found in the event WIFI_EVENT_FTM_REPORT as ftm_report_num_entries

Attention 2. The internal FTM report is freed upon use of this API which means the API can only be used once after every FTM session initiated

Attention 3. Passing the buffer as NULL merely frees the FTM report

Parameters

- **report** -- Pointer to the buffer for receiving the FTM report
- **num_entries** -- Number of FTM report entries to be filled in the report

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_config_11b_rate**(wifi_interface_t ifx, bool disable)

Enable or disable 11b rate of specified interface.

Attention 1. This API should be called after esp_wifi_init() and before esp_wifi_start().

Attention 2. Only when really need to disable 11b rate call this API otherwise don't call this.

Parameters

- **ifx** -- Interface to be configured.
- **disable** -- true means disable 11b rate while false means enable 11b rate.

Returns

- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_connectionless_module_set_wake_interval**(uint16_t wake_interval)

Set wake interval for connectionless modules to wake up periodically.

Attention 1. Only one wake interval for all connectionless modules.

Attention 2. This configuration could work at connected status. When ESP_WIFI_STA_DISCONNECTED_PM_ENABLE is enabled, this configuration could work at disconnected status.

Attention 3. Event WIFI_EVENT_CONNECTIONLESS_MODULE_WAKE_INTERVAL_START would be posted each time wake interval starts.

Attention 4. Recommend to configure interval in multiples of hundred. (e.g. 100ms)

Attention 5. Recommend to configure interval to ESP_WIFI_CONNECTIONLESS_INTERVAL_DEFAULT_MODE to get stable performance at coexistence mode.

Parameters **wake_interval** -- Milliseconds after would the chip wake up, from 1 to 65535.

esp_err_t **esp_wifi_force_wakeup_acquire**(void)

Request extra reference of Wi-Fi radio. Wi-Fi keep active state(RF opened) to be able to receive packets.

Attention Please pair the use of `esp_wifi_force_wakeup_acquire` with `esp_wifi_force_wakeup_release`.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start

esp_err_t **esp_wifi_force_wakeup_release**(void)

Release extra reference of Wi-Fi radio. Wi-Fi go to sleep state(RF closed) if no more use of radio.

Attention Please pair the use of `esp_wifi_force_wakeup_acquire` with `esp_wifi_force_wakeup_release`.

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_WIFI_NOT_STARTED: WiFi is not started by esp_wifi_start

esp_err_t **esp_wifi_set_country_code**(const char *country, bool ieee80211d_enabled)

configure country

Attention 1. When ieee80211d_enabled, the country info of the AP to which the station is connected is used. E.g. if the configured country is US and the country info of the AP to which the station is connected is JP then the country info that will be used is JP. If the station disconnected from the AP the country info is set back to the country info of the station automatically, US in the example.

Attention 2. When ieee80211d_enabled is disabled, then the configured country info is used always.

Attention 3. When the country info is changed because of configuration or because the station connects to a different external AP, the country IE in probe response/beacon of the soft-AP is also changed.

Attention 4. The country configuration is stored into flash.

Attention 5. When this API is called, the PHY init data will switch to the PHY init data type corresponding to the country info.

Attention 6. Supported country codes are "01"(world safe mode) "AT","AU","BE","BG","BR", "CA","CH","CN","CY","CZ","DE","DK","EE","ES","FI","FR","GB","GR","HK","HR","HU", "IE","IN","IS","IT","JP","KR","LI","LT","LU","LV","MT","MX","NL","NO","NZ","PL","PT", "RO","SE","SI","SK","TW","US"

Attention 7. When country code "01" (world safe mode) is set, SoftAP mode won't contain country IE.

Attention 8. The default country is "01" (world safe mode) and ieee80211d_enabled is TRUE.

Attention 9. The third octet of country code string is one of the following: ' ', 'O', 'I', 'X', otherwise it is considered as ' '.

Parameters

- **country** -- the configured country ISO code
- **ieee80211d_enabled** -- 802.11d is enabled or not

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_get_country_code**(char *country)

get the current country code

Parameters **country** -- country code

Returns

- ESP_OK: succeed
- ESP_ERR_WIFI_NOT_INIT: WiFi is not initialized by esp_wifi_init
- ESP_ERR_INVALID_ARG: invalid argument

esp_err_t **esp_wifi_config_80211_tx_rate**(wifi_interface_t ifx, wifi_phy_rate_t rate)

Config 80211 tx rate of specified interface.

Attention 1. This API should be called after esp_wifi_init() and before esp_wifi_start().

Parameters
- **ifx** -- Interface to be configured.
- **rate** -- Phy rate to be configured.

Returns
- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_disable_pmf_config**(wifi_interface_t ifx)

Disable PMF configuration for specified interface.

Attention This API should be called after esp_wifi_set_config() and before esp_wifi_start().

Parameters **ifx** -- Interface to be configured.

Returns
- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_sta_get_aid**(uint16_t *aid)

Get the Association id assigned to STA by AP.

Attention aid = 0 if station is not connected to AP.

Parameters **aid** -- **[out]** store the aid

Returns
- ESP_OK: succeed

esp_err_t **esp_wifi_sta_get_negotiated_phymode**(wifi_phy_mode_t *phymode)

Get the negotiated phymode after connection.

Parameters **phymode** -- **[out]** store the negotiated phymode.

Returns
- ESP_OK: succeed

esp_err_t **esp_wifi_set_dynamic_cs**(bool enabled)

Config dynamic carrier sense.

Attention This API should be called after esp_wifi_start().

Parameters **enabled** -- Dynamic carrier sense is enabled or not.

Returns
- ESP_OK: succeed
- others: failed

esp_err_t **esp_wifi_sta_get_rssi**(int *rssi)

Get the rssi information of AP to which the device is associated with.

Attention 1. This API should be called after station connected to AP.
Attention 2. Use this API only in WIFI_MODE_STA or WIFI_MODE_APSTA mode.

Parameters **rssi** -- store the rssi info received from last beacon.

Returns
- ESP_OK: succeed
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_FAIL: failed

Structures

struct **wifi_init_config_t**

WiFi stack configuration parameters passed to esp_wifi_init call.

Public Members

wifi_osi_funcs_t ***osi_funcs**

WiFi OS functions

wpa_crypto_funcs_t **wpa_crypto_funcs**

WiFi station crypto functions when connect

int **static_rx_buf_num**

WiFi static RX buffer number

int **dynamic_rx_buf_num**

WiFi dynamic RX buffer number

int **tx_buf_type**

WiFi TX buffer type

int **static_tx_buf_num**

WiFi static TX buffer number

int **dynamic_tx_buf_num**

WiFi dynamic TX buffer number

int **rx_mgmt_buf_type**

WiFi RX MGMT buffer type

int **rx_mgmt_buf_num**

WiFi RX MGMT buffer number

int **cache_tx_buf_num**

WiFi TX cache buffer number

int **csi_enable**

WiFi channel state information enable flag

int **ampdu_rx_enable**

WiFi AMPDU RX feature enable flag

int **ampdu_tx_enable**

WiFi AMPDU TX feature enable flag

int **amsdu_tx_enable**

WiFi AMSDU TX feature enable flag

int **nvs_enable**

WiFi NVS flash enable flag

int **nano_enable**

Nano option for printf/scan family enable flag

int **rx_ba_win**

WiFi Block Ack RX window size

int **wifi_task_core_id**

WiFi Task Core ID

int **beacon_max_len**

WiFi softAP maximum length of the beacon

int **mgmt_sbuf_num**

WiFi management short buffer number, the minimum value is 6, the maximum value is 32

uint64_t **feature_caps**

Enables additional WiFi features and capabilities

bool **sta_disconnected_pm**

WiFi Power Management for station at disconnected status

int **espnow_max_encrypt_num**

Maximum encrypt number of peers supported by espnow

int **tx_hetb_queue_num**

WiFi TX HE TB QUEUE number for STA HE TB PPDU transmission

bool **dump_hesigb_enable**

enable dump sigb field

int **magic**

WiFi init magic number, it should be the last field

Macros

ESP_ERR_WIFI_NOT_INIT

WiFi driver was not installed by esp_wifi_init

ESP_ERR_WIFI_NOT_STARTED

WiFi driver was not started by esp_wifi_start

ESP_ERR_WIFI_NOT_STOPPED

WiFi driver was not stopped by esp_wifi_stop

ESP_ERR_WIFI_IF

WiFi interface error

ESP_ERR_WIFI_MODE
WiFi mode error

ESP_ERR_WIFI_STATE
WiFi internal state error

ESP_ERR_WIFI_CONN
WiFi internal control block of station or soft-AP error

ESP_ERR_WIFI_NVS
WiFi internal NVS module error

ESP_ERR_WIFI_MAC
MAC address is invalid

ESP_ERR_WIFI_SSID
SSID is invalid

ESP_ERR_WIFI_PASSWORD
Password is invalid

ESP_ERR_WIFI_TIMEOUT
Timeout error

ESP_ERR_WIFI_WAKE_FAIL
WiFi is in sleep state(RF closed) and wakeup fail

ESP_ERR_WIFI_WOULD_BLOCK
The caller would block

ESP_ERR_WIFI_NOT_CONNECT
Station still in disconnect status

ESP_ERR_WIFI_POST
Failed to post the event to WiFi task

ESP_ERR_WIFI_INIT_STATE
Invalid WiFi state when init/deinit is called

ESP_ERR_WIFI_STOP_STATE
Returned when WiFi is stopping

ESP_ERR_WIFI_NOT_ASSOC
The WiFi connection is not associated

ESP_ERR_WIFI_TX_DISALLOW
The WiFi TX is disallowed

ESP_ERR_WIFI_TWT_FULL

no available flow id

ESP_ERR_WIFI_TWT_SETUP_TIMEOUT

Timeout of receiving twt setup response frame, timeout times can be set during twt setup

ESP_ERR_WIFI_TWT_SETUP_TXFAIL

TWT setup frame tx failed

ESP_ERR_WIFI_TWT_SETUP_REJECT

The twt setup request was rejected by the AP

ESP_ERR_WIFI_DISCARD

Discard frame

ESP_ERR_WIFI_ROC_IN_PROGRESS

ROC op is in progress

WIFI_STATIC_TX_BUFFER_NUM

WIFI_CACHE_TX_BUFFER_NUM

WIFI_DYNAMIC_TX_BUFFER_NUM

WIFI_RX_MGMT_BUF_NUM_DEF

WIFI_CSI_ENABLED

WIFI_AMPDU_RX_ENABLED

WIFI_AMPDU_TX_ENABLED

WIFI_AMSDU_TX_ENABLED

WIFI_NVS_ENABLED

WIFI_NANO_FORMAT_ENABLED

WIFI_INIT_CONFIG_MAGIC

WIFI_DEFAULT_RX_BA_WIN

WIFI_TASK_CORE_ID

WIFI_SOFTAP_BEACON_MAX_LEN

WIFI_MGMT_SBUF_NUM

WIFI_STA_DISCONNECTED_PM_ENABLED

WIFI_ENABLE_WPA3_SAE

WIFI_ENABLE_SPIRAM

WIFI_FTM_INITIATOR

WIFI_FTM_RESPONDER

WIFI_ENABLE_GCMP

WIFI_ENABLE_GMAC

WIFI_ENABLE_11R

WIFI_ENABLE_ENTERPRISE

WIFI_DUMP_HESIGB_ENABLED

WIFI_TX_HETB_QUEUE_NUM

CONFIG_FEATURE_WPA3_SAE_BIT

CONFIG_FEATURE_CACHE_TX_BUF_BIT

CONFIG_FEATURE_FTM_INITIATOR_BIT

CONFIG_FEATURE_FTM_RESPONDER_BIT

CONFIG_FEATURE_GCMP_BIT

CONFIG_FEATURE_GMAC_BIT

CONFIG_FEATURE_11R_BIT

CONFIG_FEATURE_WIFI_ENT_BIT

WIFI_FEATURE_CAPS

WIFI_INIT_CONFIG_DEFAULT()

ESP_WIFI_CONNECTIONLESS_INTERVAL_DEFAULT_MODE

Type Definitions

typedef struct *wifi_osi_funcs_t* **wifi_osi_funcs_t**

typedef void (***wifi_promiscuous_cb_t**)(void *buf, wifi_promiscuous_pkt_type_t type)

The RX callback function in the promiscuous mode. Each time a packet is received, the callback function will be called.

Param buf Data received. Type of data in buffer (wifi_promiscuous_pkt_t or wifi_pkt_rx_ctrl_t) indicated by 'type' parameter.
Param type promiscuous packet type.

typedef struct *wifi_sta_list_t* **wifi_sta_list_t**

Forward declare wifi_sta_list_t. The definition depends on the target device that implements esp_wifi.

typedef void (***esp_vendor_ie_cb_t**)(void *ctx, wifi_vendor_ie_type_t type, const uint8_t sa[6], const vendor_ie_data_t *vnd_ie, int rssi)

Function signature for received Vendor-Specific Information Element callback.

Param ctx Context argument, as passed to esp_wifi_set_vendor_ie_cb() when registering callback.
Param type Information element type, based on frame type received.
Param sa Source 802.11 address.
Param vnd_ie Pointer to the vendor specific element data received.
Param rssi Received signal strength indication.

typedef void (***wifi_csi_cb_t**)(void *ctx, wifi_csi_info_t *data)

The RX callback function of Channel State Information(CSI) data.

```
Each time a CSI data is received, the callback function will be called.
```

Param ctx context argument, passed to esp_wifi_set_csi_rx_cb() when registering callback function.
Param data CSI data received. The memory that it points to will be deallocated after callback function returns.

Header File

- components/esp_wifi/include/esp_wifi_types.h
- This header file can be included with:

```
#include "esp_wifi_types.h"
```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_wifi
```

or

```
PRIV_REQUIRES esp_wifi
```

Type Definitions

typedef struct *wifi_csi_config_t* **wifi_csi_config_t**

typedef struct *wifi_pkt_rx_ctrl_t* **wifi_pkt_rx_ctrl_t**

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_eap_client.h
- This header file can be included with:

```
#include "esp_eap_client.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

esp_err_t **esp_wifi_sta_enterprise_enable**(void)

Enable EAP authentication(WiFi Enterprise) for the station mode.

This function enables Extensible Authentication Protocol (EAP) authentication for the Wi-Fi station mode. When EAP authentication is enabled, the ESP device will attempt to authenticate with the configured EAP credentials when connecting to a secure Wi-Fi network.

Note: Before calling this function, ensure that the Wi-Fi configuration and EAP credentials (such as username and password) have been properly set using the appropriate configuration APIs.

Returns

- ESP_OK: EAP authentication enabled successfully.
- ESP_ERR_NO_MEM: Failed to enable EAP authentication due to memory allocation failure.

esp_err_t **esp_wifi_sta_enterprise_disable**(void)

Disable EAP authentication(WiFi Enterprise) for the station mode.

This function disables Extensible Authentication Protocol (EAP) authentication for the Wi-Fi station mode. When EAP authentication is disabled, the ESP device will not attempt to authenticate using EAP credentials when connecting to a secure Wi-Fi network.

Note: Disabling EAP authentication may cause the device to connect to the Wi-Fi network using other available authentication methods, if configured using esp_wifi_set_config().

Returns

- ESP_OK: EAP authentication disabled successfully.
- ESP_ERR_INVALID_STATE: EAP client is in an invalid state for disabling.

esp_err_t **esp_eap_client_set_identity**(const unsigned char *identity, int len)

Set identity for PEAP/TTLS authentication method.

This function sets the identity to be used during PEAP/TTLS authentication.

Parameters

- **identity** -- **[in]** Pointer to the identity data.
- **len** -- **[in]** Length of the identity data (limited to 1~127 bytes).

Returns

- ESP_OK: The identity was set successfully.
- ESP_ERR_INVALID_ARG: Invalid argument (len <= 0 or len >= 128).
- ESP_ERR_NO_MEM: Memory allocation failure.

void **esp_eap_client_clear_identity**(void)

Clear the previously set identity for PEAP/TTLS authentication.

This function clears the identity that was previously set for the EAP client. After calling this function, the EAP client will no longer use the previously configured identity during the authentication process.

esp_err_t **esp_eap_client_set_username**(const unsigned char *username, int len)

Set username for PEAP/TTLS authentication method.

This function sets the username to be used during PEAP/TTLS authentication.

Parameters

- **username** -- **[in]** Pointer to the username data.
- **len** -- **[in]** Length of the username data (limited to 1~127 bytes).

Returns

- ESP_OK: The username was set successfully.
- ESP_ERR_INVALID_ARG: Failed due to an invalid argument (len <= 0 or len >= 128).
- ESP_ERR_NO_MEM: Failed due to memory allocation failure.

void **esp_eap_client_clear_username**(void)

Clear username for PEAP/TTLS method.

This function clears the previously set username for the EAP client.

esp_err_t **esp_eap_client_set_password**(const unsigned char *password, int len)

Set password for PEAP/TTLS authentication method.

This function sets the password to be used during PEAP/TTLS authentication.

Parameters

- **password** -- **[in]** Pointer to the password data.
- **len** -- **[in]** Length of the password data (len > 0).

Returns

- ESP_OK: The password was set successfully.
- ESP_ERR_INVALID_ARG: Failed due to an invalid argument (len <= 0).
- ESP_ERR_NO_MEM: Failed due to memory allocation failure.

void **esp_eap_client_clear_password**(void)

Clear password for PEAP/TTLS method.

This function clears the previously set password for the EAP client.

esp_err_t **esp_eap_client_set_new_password**(const unsigned char *new_password, int len)

Set a new password for MSCHAPv2 authentication method.

This function sets the new password to be used during MSCHAPv2 authentication. The new password is used to substitute the old password when an eap-mschapv2 failure request message with error code ERROR_PASSWD_EXPIRED is received.

Parameters

- **new_password** -- **[in]** Pointer to the new password data.
- **len** -- **[in]** Length of the new password data.

Returns

- ESP_OK: The new password was set successfully.
- ESP_ERR_INVALID_ARG: Failed due to an invalid argument (len <= 0).
- ESP_ERR_NO_MEM: Failed due to memory allocation failure.

void **esp_eap_client_clear_new_password**(void)

Clear new password for MSCHAPv2 method.

This function clears the previously set new password for the EAP client.

esp_err_t **esp_eap_client_set_ca_cert** (const unsigned char *ca_cert, int ca_cert_len)

Set CA certificate for EAP authentication.

This function sets the Certificate Authority (CA) certificate to be used during EAP authentication. The CA certificate is passed to the EAP client module through a global pointer.

Parameters
- **ca_cert** -- **[in]** Pointer to the CA certificate data.
- **ca_cert_len** -- **[in]** Length of the CA certificate data.

Returns
- ESP_OK: The CA certificate was set successfully.

void **esp_eap_client_clear_ca_cert** (void)

Clear the previously set Certificate Authority (CA) certificate for EAP authentication.

This function clears the CA certificate that was previously set for the EAP client. After calling this function, the EAP client will no longer use the previously configured CA certificate during the authentication process.

esp_err_t **esp_eap_client_set_certificate_and_key** (const unsigned char *client_cert, int client_cert_len, const unsigned char *private_key, int private_key_len, const unsigned char *private_key_password, int private_key_passwd_len)

Set client certificate and private key for EAP authentication.

This function sets the client certificate and private key to be used during authentication. Optionally, a private key password can be provided for encrypted private keys.

Attention 1. The client certificate, private key, and private key password are provided as pointers to the respective data arrays.

Attention 2. The client_cert, private_key, and private_key_password should be zero-terminated.

Parameters
- **client_cert** -- **[in]** Pointer to the client certificate data.
- **client_cert_len** -- **[in]** Length of the client certificate data.
- **private_key** -- **[in]** Pointer to the private key data.
- **private_key_len** -- **[in]** Length of the private key data (limited to 1~4096 bytes).
- **private_key_password** -- **[in]** Pointer to the private key password data (optional).
- **private_key_passwd_len** -- **[in]** Length of the private key password data (can be 0 for no password).

Returns
- ESP_OK: The certificate, private key, and password (if provided) were set successfully.

void **esp_eap_client_clear_certificate_and_key** (void)

Clear the previously set client certificate and private key for EAP authentication.

This function clears the client certificate and private key that were previously set for the EAP client. After calling this function, the EAP client will no longer use the previously configured certificate and private key during the authentication process.

esp_err_t **esp_eap_client_set_disable_time_check** (bool disable)

Set EAP client certificates time check (disable or not).

This function enables or disables the time check for EAP client certificates. When disabled, the certificates' expiration time will not be checked during the authentication process.

Parameters **disable** -- **[in]** True to disable EAP client certificates time check, false to enable it.

Returns
- ESP_OK: The EAP client certificates time check setting was updated successfully.

esp_err_t **esp_eap_client_get_disable_time_check**(bool *disable)

Get EAP client certificates time check status.

This function retrieves the current status of the EAP client certificates time check.

Parameters **disable** -- **[out]** Pointer to a boolean variable to store the disable status.
Returns
- ESP_OK: The status of EAP client certificates time check was retrieved successfully.

esp_err_t **esp_eap_client_set_ttls_phase2_method**(*esp_eap_ttls_phase2_types* type)

Set EAP-TTLS phase 2 method.

This function sets the phase 2 method to be used during EAP-TTLS authentication.

Parameters **type** -- **[in]** The type of phase 2 method to be used (e.g., EAP, MSCHAPv2, MSCHAP, PAP, CHAP).
Returns
- ESP_OK: The EAP-TTLS phase 2 method was set successfully.

esp_err_t **esp_eap_client_set_suiteb_192bit_certification**(bool enable)

Enable or disable Suite-B 192-bit certification checks.

This function enables or disables the 192-bit Suite-B certification checks during EAP-TLS authentication. Suite-B is a set of cryptographic algorithms which generally are considered more secure.

Parameters **enable** -- **[in]** True to enable 192-bit Suite-B certification checks, false to disable it.
Returns
- ESP_OK: The 192-bit Suite-B certification checks were set successfully.

esp_err_t **esp_eap_client_set_pac_file**(const unsigned char *pac_file, int pac_file_len)

Set the PAC (Protected Access Credential) file for EAP-FAST authentication.

EAP-FAST requires a PAC file that contains the client's credentials.

Attention 1. For files read from the file system, length has to be decremented by 1 byte.
Attention 2. Disabling the ESP_WIFI_MBEDTLS_TLS_CLIENT config is required to use EAP-FAST.

Parameters
- **pac_file** -- **[in]** Pointer to the PAC file buffer.
- **pac_file_len** -- **[in]** Length of the PAC file buffer.

Returns
- ESP_OK: The PAC file for EAP-FAST authentication was set successfully.

esp_err_t **esp_eap_client_set_fast_params**(*esp_eap_fast_config* config)

Set the parameters for EAP-FAST Phase 1 authentication.

EAP-FAST supports Fast Provisioning, where clients can be authenticated faster using precomputed keys (PAC). This function allows configuring parameters for Fast Provisioning.

Attention 1. Disabling the ESP_WIFI_MBEDTLS_TLS_CLIENT config is required to use EAP-FAST.

Parameters **config** -- **[in]** Configuration structure with Fast Provisioning parameters.
Returns
- ESP_OK: The parameters for EAP-FAST Phase 1 authentication were set successfully.

esp_err_t **esp_eap_client_use_default_cert_bundle**(bool use_default_bundle)

Use the default certificate bundle for EAP authentication.

By default, the EAP client uses a built-in certificate bundle for server verification. Enabling this option allows the use of the default certificate bundle.

Parameters **use_default_bundle** -- **[in]** True to use the default certificate bundle, false to use a custom bundle.

Returns

- ESP_OK: The option to use the default certificate bundle was set successfully.

Structures

struct **esp_eap_fast_config**

Configuration settings for EAP-FAST (Extensible Authentication Protocol - Flexible Authentication via Secure Tunneling).

This structure defines the configuration options that can be used to customize the behavior of the EAP-FAST authentication protocol, specifically for Fast Provisioning and PAC (Protected Access Credential) handling.

Public Members

int **fast_provisioning**

Enable or disable Fast Provisioning in EAP-FAST (0 = disabled, 1 = enabled)

int **fast_max_pac_list_len**

Maximum length of the PAC (Protected Access Credential) list

bool **fast_pac_format_binary**

Set to true for binary format PAC, false for ASCII format PAC

Enumerations

enum **esp_eap_ttls_phase2_types**

Enumeration of phase 2 authentication types for EAP-TTLS.

This enumeration defines the supported phase 2 authentication methods that can be used in the EAP-TTLS (Extensible Authentication Protocol - Tunneled Transport Layer Security) protocol for the second authentication phase.

Values:

enumerator **ESP_EAP_TTLS_PHASE2_EAP**

EAP (Extensible Authentication Protocol)

enumerator **ESP_EAP_TTLS_PHASE2_MSCHAPV2**

MS-CHAPv2 (Microsoft Challenge Handshake Authentication Protocol - Version 2)

enumerator **ESP_EAP_TTLS_PHASE2_MSCHAP**

MS-CHAP (Microsoft Challenge Handshake Authentication Protocol)

enumerator **ESP_EAP_TTLS_PHASE2_PAP**

PAP (Password Authentication Protocol)

enumerator **ESP_EAP_TTLS_PHASE2_CHAP**

CHAP (Challenge Handshake Authentication Protocol)

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_wps.h
- This header file can be included with:

```
#include "esp_wps.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

esp_err_t **esp_wifi_wps_enable**(const *esp_wps_config_t* *config)

Enable Wi-Fi WPS function.

Parameters **config** -- : WPS config to be used in connection

Returns

- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_TYPE : wps type is invalid
- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on
- ESP_FAIL : wps initialization fails

esp_err_t **esp_wifi_wps_disable**(void)

Disable Wi-Fi WPS function and release resource it taken.

Returns

- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on

esp_err_t **esp_wifi_wps_start**(int timeout_ms)

Start WPS session.

Attention WPS can only be used when station is enabled. WPS needs to be enabled first for using this API.

Parameters **timeout_ms** -- : deprecated: This argument's value will have not effect in functionality of API. The argument will be removed in future. The app should start WPS and register for WIFI events to get the status. WPS status is updated through WPS events. See wifi_event_t enum for more info.

Returns

- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_TYPE : wps type is invalid
- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on
- ESP_ERR_WIFI_WPS_SM : wps state machine is not initialized
- ESP_FAIL : wps initialization fails

esp_err_t **esp_wifi_ap_wps_enable**(const *esp_wps_config_t* *config)

Enable Wi-Fi AP WPS function.

Attention WPS can only be used when softAP is enabled.

Parameters **config** -- wps configuration to be used.

Returns

- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_TYPE : wps type is invalid

- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on
- ESP_FAIL : wps initialization fails

esp_err_t **esp_wifi_ap_wps_disable**(void)

Disable Wi-Fi SoftAP WPS function and release resource it taken.

Returns
- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on

esp_err_t **esp_wifi_ap_wps_start**(const unsigned char *pin)

WPS starts to work.

Attention WPS can only be used when softAP is enabled.

Parameters **pin** -- : Pin to be used in case of WPS mode is pin. If Pin is not provided, device will use the pin generated/provided during *esp_wifi_ap_wps_enable()* and reported in WIFI_EVENT_AP_WPS_RG_PIN

Returns
- ESP_OK : succeed
- ESP_ERR_WIFI_WPS_TYPE : wps type is invalid
- ESP_ERR_WIFI_WPS_MODE : wifi is not in station mode or sniffer mode is on
- ESP_ERR_WIFI_WPS_SM : wps state machine is not initialized
- ESP_FAIL : wps initialization fails

Structures

struct **wps_factory_information_t**

Structure representing WPS factory information for ESP device.

This structure holds various strings representing factory information for a device, such as the manufacturer, model number, model name, and device name. Each string is a null-terminated character array. If any of the strings are empty, the default values are used.

Public Members

char **manufacturer**[WPS_MAX_MANUFACTURER_LEN]

Manufacturer of the device. If empty, the default manufacturer is used.

char **model_number**[WPS_MAX_MODEL_NUMBER_LEN]

Model number of the device. If empty, the default model number is used.

char **model_name**[WPS_MAX_MODEL_NAME_LEN]

Model name of the device. If empty, the default model name is used.

char **device_name**[WPS_MAX_DEVICE_NAME_LEN]

Device name. If empty, the default device name is used.

struct **esp_wps_config_t**

Structure representing configuration settings for WPS (Wi-Fi Protected Setup).

This structure encapsulates various configuration settings for WPS, including the WPS type (PBC or PIN), factory information that will be shown in the WPS Information Element (IE), and a PIN if the WPS type is set to PIN.

Public Members

wps_type_t **wps_type**

The type of WPS to be used (PBC or PIN).

wps_factory_information_t **factory_info**

Factory information to be shown in the WPS Information Element (IE). Vendor can choose to display their own information.

char **pin**[PIN_LEN]

WPS PIN (Personal Identification Number) used when wps_type is set to WPS_TYPE_PIN.

Macros

ESP_ERR_WIFI_REGISTRAR

WPS registrar is not supported

ESP_ERR_WIFI_WPS_TYPE

WPS type error

ESP_ERR_WIFI_WPS_SM

WPS state machine is not initialized

WPS_MAX_MANUFACTURER_LEN

Maximum length of the manufacturer name in WPS information

WPS_MAX_MODEL_NUMBER_LEN

Maximum length of the model number in WPS information

WPS_MAX_MODEL_NAME_LEN

Maximum length of the model name in WPS information

WPS_MAX_DEVICE_NAME_LEN

Maximum length of the device name in WPS information

PIN_LEN

The length of the WPS PIN (Personal Identification Number).

WPS_CONFIG_INIT_DEFAULT(type)

Initialize a default WPS configuration structure with specified WPS type.

This macro initializes a *esp_wps_config_t* structure with default values for the specified WPS type. It sets the WPS type, factory information (including default manufacturer, model number, model name, and device name), and a default PIN value if applicable.

Parameters

- **type** -- The WPS type to be used (PBC or PIN).

Returns An initialized *esp_wps_config_t* structure with the specified WPS type and default values.

Type Definitions

typedef enum *wps_type* **wps_type_t**

Enumeration of WPS (Wi-Fi Protected Setup) types.

Enumerations

enum **wps_type**

Enumeration of WPS (Wi-Fi Protected Setup) types.

Values:

enumerator **WPS_TYPE_DISABLE**

WPS is disabled

enumerator **WPS_TYPE_PBC**

WPS Push Button Configuration method

enumerator **WPS_TYPE_PIN**

WPS PIN (Personal Identification Number) method

enumerator **WPS_TYPE_MAX**

Maximum value for WPS type enumeration

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_rrm.h
- This header file can be included with:

```
#include "esp_rrm.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

int **esp_rrm_send_neighbor_rep_request**(*neighbor_rep_request_cb* cb, void *cb_ctx)

Send Radio measurement neighbor report request to connected AP.

Deprecated:

This function is deprecated and will be removed in the future. Please use 'esp_rrm_send_neighbor_report_request'

Parameters

- **cb** -- callback function for neighbor report
- **cb_ctx** -- callback context

Returns

- 0: success
- -1: AP does not support RRM
- -2: station not connected to AP

int **esp_rrm_send_neighbor_report_request**(void)

Send Radio measurement neighbor report request to connected AP.

Returns

- 0: success

- -1: AP does not support RRM
- -2: station not connected to AP

bool **esp_rrm_is_rrm_supported_connection**(void)

Check RRM capability of connected AP.

Returns

- true: AP supports RRM
- false: AP does not support RRM or station not connected to AP

Type Definitions

typedef void (***neighbor_rep_request_cb**)(void *ctx, const uint8_t *report, size_t report_len)

Callback function type to get neighbor report.

Param ctx neighbor report context
Param report neighbor report
Param report_len neighbor report length
Return

- void

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_wnm.h
- This header file can be included with:

```
#include "esp_wnm.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

int **esp_wnm_send_bss_transition_mgmt_query**(enum *btm_query_reason* query_reason, const char *btm_candidates, int cand_list)

Send bss transition query to connected AP.

Parameters

- **query_reason** -- reason for sending query
- **btm_candidates** -- btm candidates list if available
- **cand_list** -- whether candidate list to be included from scan results available in supplicant's cache.

Returns

- 0: success
- -1: AP does not support BTM
- -2: station not connected to AP

bool **esp_wnm_is_btm_supported_connection**(void)

Check bss trasition capability of connected AP.

Returns

- true: AP supports BTM
- false: AP does not support BTM or station not connected to AP

Enumerations

enum **btm_query_reason**

enum btm_query_reason: Reason code for sending btm query

Values:

enumerator **REASON_UNSPECIFIED**

enumerator **REASON_FRAME_LOSS**

enumerator **REASON_DELAY**

enumerator **REASON_BANDWIDTH**

enumerator **REASON_LOAD_BALANCE**

enumerator **REASON_RSSI**

enumerator **REASON_RETRANSMISSIONS**

enumerator **REASON_INTERFERENCE**

enumerator **REASON_GRAY_ZONE**

enumerator **REASON_PREMIUM_AP**

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_mbo.h
- This header file can be included with:

```
#include "esp_mbo.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

int **esp_mbo_update_non_pref_chan**(struct *non_pref_chan_s* *non_pref_chan)

Update channel preference for MBO IE.

Parameters **non_pref_chan** -- Non preference channel list

Returns

- 0: success else failure

Structures

struct **non_pref_chan**

Structure representing a non-preferred channel in a wireless network.

This structure encapsulates information about a non-preferred channel including the reason for its non-preference, the operating class, channel number, and preference level.

Public Members

enum *non_pref_chan_reason* **reason**

Reason for the channel being non-preferred

uint8_t **oper_class**

Operating class of the channel

uint8_t **chan**

Channel number

uint8_t **preference**

Preference level of the channel

struct **non_pref_chan_s**

Structure representing a list of non-preferred channels in a wireless network.

This structure encapsulates information about a list of non-preferred channels including the number of non-preferred channels and an array of structures representing individual non-preferred channels.

Public Members

size_t **non_pref_chan_num**

Number of non-preferred channels in the list

struct *non_pref_chan* **chan**[]

Array of structures representing individual non-preferred channels

Enumerations

enum **non_pref_chan_reason**

Enumeration of reasons for a channel being non-preferred in a wireless network.

This enumeration defines various reasons why a specific channel might be considered non-preferred in a wireless network configuration.

Values:

enumerator **NON_PREF_CHAN_REASON_UNSPECIFIED**

Unspecified reason for non-preference

enumerator **NON_PREF_CHAN_REASON_RSSI**

Non-preferred due to low RSSI (Received Signal Strength Indication)

enumerator **NON_PREF_CHAN_REASON_EXT_INTERFERENCE**

Non-preferred due to external interference

enumerator **NON_PREF_CHAN_REASON_INT_INTERFERENCE**

Non-preferred due to internal interference

Wi-Fi Easy Connect™ (DPP)

Wi-Fi Easy Connect™, also known as Device Provisioning Protocol (DPP) or Easy Connect, is a provisioning protocol certified by Wi-Fi Alliance. It is a secure and standardized provisioning protocol for configuration of Wi-Fi Devices. With Easy Connect, adding a new device to a network is as simple as scanning a QR Code. This reduces complexity and enhances user experience while onboarding devices without UI like Smart Home and IoT products. Unlike old protocols like Wi-Fi Protected Setup (WPS), Wi-Fi Easy Connect in corporates strong encryption through public key cryptography to ensure networks remain secure as new devices are added.

Easy Connect brings many benefits in the user experience:

- Simple and intuitive to use; no lengthy instructions to follow for new device setup
- No need to remember and enter passwords into the device being provisioned
- Works with electronic or printed QR codes, or human-readable strings
- Supports both WPA2 and WPA3 networks

Please refer to Wi-Fi Alliance's official page on Easy Connect for more information.

ESP32-S3 supports Enrollee mode of Easy Connect with QR Code as the provisioning method. A display is required to display this QR Code. Users can scan this QR Code using their capable device and provision the ESP32-S3 to their Wi-Fi network. The provisioning device needs to be connected to the AP which need not support Wi-Fi Easy Connect™.

Easy Connect is still an evolving protocol. Of known platforms that support the QR Code method are some Android smartphones with Android 10 or higher. To use Easy Connect, no additional App needs to be installed on the supported smartphone.

Application Example Example on how to provision ESP32-S3 using a supported smartphone: wifi/wifi_easy_connect/dpp-enrollee.

API Reference

Header File

- components/wpa_supplicant/esp_supplicant/include/esp_dpp.h
- This header file can be included with:

```
#include "esp_dpp.h"
```

- This header file is a part of the API provided by the `wpa_supplicant` component. To declare that your component depends on `wpa_supplicant`, add the following to your CMakeLists.txt:

```
REQUIRES wpa_supplicant
```

or

```
PRIV_REQUIRES wpa_supplicant
```

Functions

esp_err_t **esp_supp_dpp_init**(*esp_supp_dpp_event_cb_t* evt_cb)

Initialize DPP Supplicant.

```
Starts DPP Supplicant and initializes related Data Structures.
```

return

- ESP_OK: Success
- ESP_FAIL: Failure

Parameters **evt_cb** -- Callback function to receive DPP related events

esp_err_t **esp_supp_dpp_deinit**(void)

De-initalize DPP Supplicant.

```
Frees memory from DPP Supplicant Data Structures.
```

Returns

- ESP_OK: Success

esp_err_t **esp_supp_dpp_bootstrap_gen**(const char *chan_list, *esp_supp_dpp_bootstrap_t* type, const char *key, const char *info)

Generates Bootstrap Information as an Enrollee.

```
Generates Out Of Band Bootstrap information as an Enrollee which can be
used by a DPP Configurator to provision the Enrollee.
```

Parameters

- **chan_list** -- List of channels device will be available on for listening
- **type** -- Bootstrap method type, only QR Code method is supported for now.
- **key** -- (Optional) 32 byte Raw Private Key for generating a Bootstrapping Public Key
- **info** -- (Optional) Ancilliary Device Information like Serial Number

Returns

- ESP_OK: Success
- ESP_FAIL: Failure

esp_err_t **esp_supp_dpp_start_listen**(void)

Start listening on Channels provided during esp_supp_dpp_bootstrap_gen.

```
Listens on every Channel from Channel List for a pre-defined wait time.
```

Returns

- ESP_OK: Success
- ESP_FAIL: Generic Failure
- ESP_ERR_INVALID_STATE: ROC attempted before WiFi is started
- ESP_ERR_NO_MEM: Memory allocation failed while posting ROC request

esp_err_t **esp_supp_dpp_stop_listen**(void)

Stop listening on Channels.

```
Stops listening on Channels and cancels ongoing listen operation.
```

Returns

- ESP_OK: Success
- ESP_FAIL: Failure

Macros

ESP_DPP_AUTH_TIMEOUT_SECS

ESP_ERR_DPP_FAILURE

Generic failure during DPP Operation

ESP_ERR_DPP_TX_FAILURE

DPP Frame Tx failed OR not Acked

ESP_ERR_DPP_INVALID_ATTR

Encountered invalid DPP Attribute

ESP_ERR_DPP_AUTH_TIMEOUT

DPP Auth response was not recieved in time

Type Definitions

typedef enum *dpp_bootstrap_type* **esp_supp_dpp_bootstrap_t**

Types of Bootstrap Methods for DPP.

typedef void (***esp_supp_dpp_event_cb_t**)(*esp_supp_dpp_event_t* evt, void *data)

Callback function for receiving DPP Events from Supplicant.

```
Callback function will be called with DPP related information.
```

Param evt DPP event ID
Param data Event data payload

Enumerations

enum **dpp_bootstrap_type**

Types of Bootstrap Methods for DPP.

Values:

enumerator **DPP_BOOTSTRAP_QR_CODE**

QR Code Method

enumerator **DPP_BOOTSTRAP_PKEX**

Proof of Knowledge Method

enumerator **DPP_BOOTSTRAP_NFC_URI**

NFC URI record Method

enum **esp_supp_dpp_event_t**

Types of Callback Events received from DPP Supplicant.

Values:

enumerator **ESP_SUPP_DPP_URI_READY**

URI is ready through Bootstrapping

enumerator **ESP_SUPP_DPP_CFG_RECVD**

Config received via DPP Authentication

enumerator **ESP_SUPP_DPP_PDR_RECVD**

Peer Discovery Response is received

enumerator **ESP_SUPP_DPP_FAIL**

DPP Authentication failure

Code examples for the Wi-Fi API are provided in the wifi directory of ESP-IDF examples.

Code examples for ESP-WIFI-MESH are provided in the mesh directory of ESP-IDF examples.

2.5.2 Ethernet

Ethernet

Overview ESP-IDF provides a set of consistent and flexible APIs to support external SPI-Ethernet modules.

This programming guide is split into the following sections:

1. *Basic Ethernet Concepts*
2. *Configure MAC and PHY*
3. *Connect Driver to TCP/IP Stack*
4. *Misc Control of Ethernet Driver*

Basic Ethernet Concepts Ethernet is an asynchronous Carrier Sense Multiple Access with Collision Detect (CSMA/CD) protocol/interface. It is generally not well suited for low-power applications. However, with ubiquitous deployment, internet connectivity, high data rates, and limitless-range expandability, Ethernet can accommodate nearly all wired communications.

Normal IEEE 802.3 compliant Ethernet frames are between 64 and 1518 bytes in length. They are made up of five or six different fields: a destination MAC address (DA), a source MAC address (SA), a type/length field, a data payload, an optional padding field and a Cyclic Redundancy Check (CRC). Additionally, when transmitted on the Ethernet medium, a 7-byte preamble field and Start-of-Frame (SOF) delimiter byte are appended to the beginning of the Ethernet packet.

Thus the traffic on the twist-pair cabling appears as shown below:

Preamble and Start-of-Frame Delimiter The preamble contains seven bytes of `55H`. It allows the receiver to lock onto the stream of data before the actual frame arrives.

The Start-of-Frame Delimiter (SFD) is a binary sequence `10101011` (as seen on the physical medium). It is sometimes considered to be part of the preamble.

When transmitting and receiving data, the preamble and SFD bytes will be automatically generated or stripped from the packets.

Destination Address The destination address field contains a 6-byte length MAC address of the device that the packet is directed to. If the Least Significant bit in the first byte of the MAC address is set, the address is a multicast destination. For example, 01-00-00-00-F0-00 and 33-45-67-89-AB-CD are multi-cast addresses, while 00-00-00-00-F0-00 and 32-45-67-89-AB-CD are not.

Packets with multi-cast destination addresses are designed to arrive and be important to a selected group of Ethernet nodes. If the destination address field is the reserved multicast address, i.e., FF-FF-FF-FF-FF-FF, the packet is a

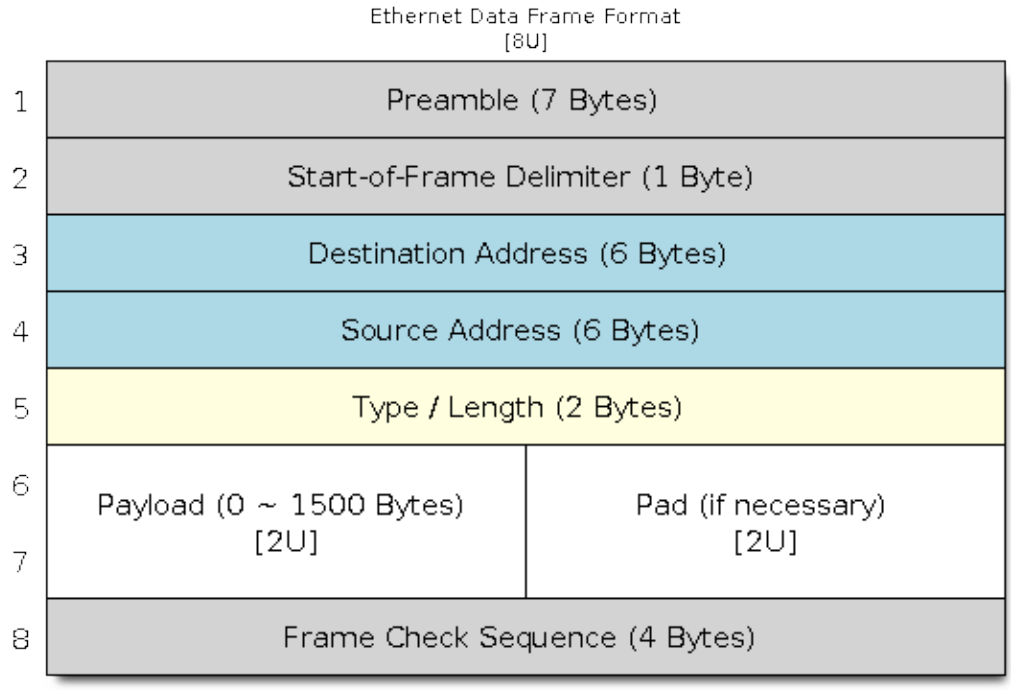


Fig. 4: Ethernet Data Frame Format

broadcast packet and it will be directed to everyone sharing the network. If the Least Significant bit in the first byte of the MAC address is clear, the address is a unicast address and will be designed for usage by only the addressed node.

Normally the EMAC controller incorporates receive filters which can be used to discard or accept packets with multicast, broadcast and/or unicast destination addresses. When transmitting packets, the host controller is responsible for writing the desired destination address into the transmit buffer.

Source Address The source address field contains a 6-byte length MAC address of the node which created the Ethernet packet. Users of Ethernet must generate a unique MAC address for each controller used. MAC addresses consist of two portions. The first three bytes are known as the Organizationally Unique Identifier (OUI). OUIs are distributed by the IEEE. The last three bytes are address bytes at the discretion of the company that purchased the OUI. For more information about MAC Address used in ESP-IDF, please see *MAC Address Allocation*.

When transmitting packets, the assigned source MAC address must be written into the transmit buffer by the host controller.

Type/Length The type/length field is a 2-byte field. If the value in this field is <= 1500 (decimal), it is considered a length field and it specifies the amount of non-padding data which follows in the data field. If the value is >= 1536, it represents the protocol the following packet data belongs to. The following are the most common type values:

- IPv4 = 0800H
- IPv6 = 86DDH
- ARP = 0806H

Users implementing proprietary networks may choose to treat this field as a length field, while applications implementing protocols such as the Internet Protocol (IP) or Address Resolution Protocol (ARP), should program this field with the appropriate type defined by the protocol's specification when transmitting packets.

Payload The payload field is a variable length field, anywhere from 0 to 1500 bytes. Larger data packets violates Ethernet standards and will be dropped by most Ethernet nodes.

This field contains the client data, such as an IP datagram.

Padding and FCS The padding field is a variable length field added to meet the IEEE 802.3 specification requirements when small data payloads are used.

The DA, SA, type, payload, and padding of an Ethernet packet must be no smaller than 60 bytes in total. If the required 4-byte FCS field is added, packets must be no smaller than 64 bytes. If the payload field is less than 46-byte long, a padding field is required.

The FCS field is a 4-byte field that contains an industry-standard 32-bit CRC calculated with the data from the DA, SA, type, payload, and padding fields. Given the complexity of calculating a CRC, the hardware normally automatically generates a valid CRC and transmit it. Otherwise, the host controller must generate the CRC and place it in the transmit buffer.

Normally, the host controller does not need to concern itself with padding and the CRC which the hardware EMAC will also be able to automatically generate when transmitting and verify when receiving. However, the padding and CRC fields will be written into the receive buffer when packets arrive, so they may be evaluated by the host controller if needed.

Note: Besides the basic data frame described above, there are two other common frame types in 10/100 Mbps Ethernet: control frames and VLAN-tagged frames. They are not supported in ESP-IDF.

Configure MAC and PHY The Ethernet driver is composed of two parts: MAC and PHY.

You need to set up the necessary parameters for MAC and PHY respectively based on your Ethernet board design, and then combine the two together to complete the driver installation.

Basic common configuration for MAC layer is described in *eth_mac_config_t*, including:

- *eth_mac_config_t::sw_reset_timeout_ms*: software reset timeout value, in milliseconds. Typically, MAC reset should be finished within 100 ms.
- *eth_mac_config_t::rx_task_stack_size* and *eth_mac_config_t::rx_task_prio*: the MAC driver creates a dedicated task to process incoming packets. These two parameters are used to set the stack size and priority of the task.
- *eth_mac_config_t::flags*: specifying extra features that the MAC driver should have, it could be useful in some special situations. The value of this field can be OR'd with macros prefixed with `ETH_MAC_FLAG_`. For example, if the MAC driver should work when the cache is disabled, then you should configure this field with *ETH_MAC_FLAG_WORK_WITH_CACHE_DISABLE*.

Configuration for PHY is described in *eth_phy_config_t*, including:

- *eth_phy_config_t::phy_addr*: multiple PHY devices can share the same SMI bus, so each PHY needs a unique address. Usually, this address is configured during hardware design by pulling up/down some PHY strapping pins. You can set the value from `0` to `15` based on your Ethernet board. Especially, if the SMI bus is shared by only one PHY device, setting this value to `-1` can enable the driver to detect the PHY address automatically.
- *eth_phy_config_t::reset_timeout_ms*: reset timeout value, in milliseconds. Typically, PHY reset should be finished within 100 ms.
- *eth_phy_config_t::autonego_timeout_ms*: auto-negotiation timeout value, in milliseconds. The Ethernet driver starts negotiation with the peer Ethernet node automatically, to determine to duplex and speed mode. This value usually depends on the ability of the PHY device on your board.
- *eth_phy_config_t::reset_gpio_num*: if your board also connects the PHY reset pin to one of the GPIO, then set it here. Otherwise, set this field to `-1`.

ESP-IDF provides a default configuration for MAC and PHY in macro *ETH_MAC_DEFAULT_CONFIG* and *ETH_PHY_DEFAULT_CONFIG*.

Create MAC and PHY Instance The Ethernet driver is implemented in an Object-Oriented style. Any operation on MAC and PHY should be based on the instance of the two.

SPI-Ethernet Module

```
eth_mac_config_t mac_config = ETH_MAC_DEFAULT_CONFIG();       // apply default␣
↪common MAC configuration
eth_phy_config_t phy_config = ETH_PHY_DEFAULT_CONFIG();       // apply default PHY␣
↪configuration
phy_config.phy_addr = CONFIG_EXAMPLE_ETH_PHY_ADDR;            // alter the PHY␣
↪address according to your board design
phy_config.reset_gpio_num = CONFIG_EXAMPLE_ETH_PHY_RST_GPIO; // alter the GPIO␣
↪used for PHY reset
// Install GPIO interrupt service (as the SPI-Ethernet module is interrupt-driven)
gpio_install_isr_service(0);
// SPI bus configuration
spi_device_handle_t spi_handle = NULL;
spi_bus_config_t buscfg = {
    .miso_io_num = CONFIG_EXAMPLE_ETH_SPI_MISO_GPIO,
    .mosi_io_num = CONFIG_EXAMPLE_ETH_SPI_MOSI_GPIO,
    .sclk_io_num = CONFIG_EXAMPLE_ETH_SPI_SCLK_GPIO,
    .quadwp_io_num = -1,
    .quadhd_io_num = -1,
};
ESP_ERROR_CHECK(spi_bus_initialize(CONFIG_EXAMPLE_ETH_SPI_HOST, &buscfg, 1));
// Configure SPI device
spi_device_interface_config_t spi_devcfg = {
    .mode = 0,
    .clock_speed_hz = CONFIG_EXAMPLE_ETH_SPI_CLOCK_MHZ * 1000 * 1000,
    .spics_io_num = CONFIG_EXAMPLE_ETH_SPI_CS_GPIO,
    .queue_size = 20
};
/* dm9051 ethernet driver is based on spi driver */
eth_dm9051_config_t dm9051_config = ETH_DM9051_DEFAULT_CONFIG(CONFIG_EXAMPLE_ETH_␣
↪SPI_HOST, &spi_devcfg);
dm9051_config.int_gpio_num = CONFIG_EXAMPLE_ETH_SPI_INT_GPIO;
esp_eth_mac_t *mac = esp_eth_mac_new_dm9051(&dm9051_config, &mac_config);
esp_eth_phy_t *phy = esp_eth_phy_new_dm9051(&phy_config);
```

Note:

- When creating MAC and PHY instances for SPI-Ethernet modules (e.g., DM9051), the constructor function must have the same suffix (e.g., *esp_eth_mac_new_dm9051* and *esp_eth_phy_new_dm9051*). This is because we don not have other choices but the integrated PHY.
- The SPI device configuration (i.e., *spi_device_interface_config_t*) may slightly differ for other Ethernet modules or to meet SPI timing on specific PCB. Please check out your module's specs and the examples in ESP-IDF.

Install Driver To install the Ethernet driver, we need to combine the instance of MAC and PHY and set some additional high-level configurations (i.e., not specific to either MAC or PHY) in *esp_eth_config_t*:

- *esp_eth_config_t::mac*: instance that created from MAC generator (e.g., `esp_eth_mac_new_esp32()`).
- *esp_eth_config_t::phy*: instance that created from PHY generator (e.g., *esp_eth_phy_new_ip101()*).
- *esp_eth_config_t::check_link_period_ms*: Ethernet driver starts an OS timer to check the link status periodically, this field is used to set the interval, in milliseconds.

- *esp_eth_config_t::stack_input*: In most Ethernet IoT applications, any Ethernet frame received by a driver should be passed to the upper layer (e.g., TCP/IP stack). This field is set to a function that is responsible to deal with the incoming frames. You can even update this field at runtime via function *esp_eth_update_input_path()* after driver installation.
- *esp_eth_config_t::on_lowlevel_init_done* and *esp_eth_config_t::on_lowlevel_deinit_done*: These two fields are used to specify the hooks which get invoked when low-level hardware has been initialized or de-initialized.

ESP-IDF provides a default configuration for driver installation in macro *ETH_DEFAULT_CONFIG*.

```
esp_eth_config_t config = ETH_DEFAULT_CONFIG(mac, phy); // apply default driver␣
↪configuration
esp_eth_handle_t eth_handle = NULL; // after the driver is installed, we will get␣
↪the handle of the driver
esp_eth_driver_install(&config, &eth_handle); // install driver
```

The Ethernet driver also includes an event-driven model, which sends useful and important events to user space. We need to initialize the event loop before installing the Ethernet driver. For more information about event-driven programming, please refer to *ESP Event*.

```
/** Event handler for Ethernet events */
static void eth_event_handler(void *arg, esp_event_base_t event_base,
                              int32_t event_id, void *event_data)
{
    uint8_t mac_addr[6] = {0};
    /* we can get the ethernet driver handle from event data */
    esp_eth_handle_t eth_handle = *(esp_eth_handle_t *)event_data;

    switch (event_id) {
    case ETHERNET_EVENT_CONNECTED:
        esp_eth_ioctl(eth_handle, ETH_CMD_G_MAC_ADDR, mac_addr);
        ESP_LOGI(TAG, "Ethernet Link Up");
        ESP_LOGI(TAG, "Ethernet HW Addr %02x:%02x:%02x:%02x:%02x:%02x",
                 mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_
↪addr[4], mac_addr[5]);
        break;
    case ETHERNET_EVENT_DISCONNECTED:
        ESP_LOGI(TAG, "Ethernet Link Down");
        break;
    case ETHERNET_EVENT_START:
        ESP_LOGI(TAG, "Ethernet Started");
        break;
    case ETHERNET_EVENT_STOP:
        ESP_LOGI(TAG, "Ethernet Stopped");
        break;
    default:
        break;
    }
}

esp_event_loop_create_default(); // create a default event loop that runs in the␣
↪background
esp_event_handler_register(ETH_EVENT, ESP_EVENT_ANY_ID, &eth_event_handler, NULL);␣
↪// register Ethernet event handler (to deal with user-specific stuff when events␣
↪like link up/down happened)
```

Start Ethernet Driver After driver installation, we can start Ethernet immediately.

```
esp_eth_start(eth_handle); // start Ethernet driver state machine
```

Connect Driver to TCP/IP Stack Up until now, we have installed the Ethernet driver. From the view of OSI (Open System Interconnection), we are still on level 2 (i.e., Data Link Layer). While we can detect link up and down events and gain MAC address in user space, it is infeasible to obtain the IP address, let alone send an HTTP request. The TCP/IP stack used in ESP-IDF is called LwIP. For more information about it, please refer to *LwIP*.

To connect the Ethernet driver to TCP/IP stack, follow these three steps:

1. Create a network interface for the Ethernet driver
2. Attach the network interface to the Ethernet driver
3. Register IP event handlers

For more information about the network interface, please refer to *Network Interface*.

```
/** Event handler for IP_EVENT_ETH_GOT_IP */
static void got_ip_event_handler(void *arg, esp_event_base_t event_base,
                                 int32_t event_id, void *event_data)
{
    ip_event_got_ip_t *event = (ip_event_got_ip_t *) event_data;
    const esp_netif_ip_info_t *ip_info = &event->ip_info;

    ESP_LOGI(TAG, "Ethernet Got IP Address");
    ESP_LOGI(TAG, "~~~~~~~~~~~");
    ESP_LOGI(TAG, "ETHIP:" IPSTR, IP2STR(&ip_info->ip));
    ESP_LOGI(TAG, "ETHMASK:" IPSTR, IP2STR(&ip_info->netmask));
    ESP_LOGI(TAG, "ETHGW:" IPSTR, IP2STR(&ip_info->gw));
    ESP_LOGI(TAG, "~~~~~~~~~~~");
}

esp_netif_init()); // Initialize TCP/IP network interface (should be called only␣
↪once in application)
esp_netif_config_t cfg = ESP_NETIF_DEFAULT_ETH(); // apply default network␣
↪interface configuration for Ethernet
esp_netif_t *eth_netif = esp_netif_new(&cfg); // create network interface for␣
↪Ethernet driver

esp_netif_attach(eth_netif, esp_eth_new_netif_glue(eth_handle)); // attach␣
↪Ethernet driver to TCP/IP stack
esp_event_handler_register(IP_EVENT, IP_EVENT_ETH_GOT_IP, &got_ip_event_handler,␣
↪NULL); // register user defined IP event handlers
esp_eth_start(eth_handle); // start Ethernet driver state machine
```

Warning: It is recommended to fully initialize the Ethernet driver and network interface before registering the user's Ethernet/IP event handlers, i.e., register the event handlers as the last thing prior to starting the Ethernet driver. Such an approach ensures that Ethernet/IP events get executed first by the Ethernet driver or network interface so the system is in the expected state when executing the user's handlers.

Misc Control of Ethernet Driver The following functions should only be invoked after the Ethernet driver has been installed.

- Stop Ethernet driver: *esp_eth_stop()*
- Update Ethernet data input path: *esp_eth_update_input_path()*
- Misc get/set of Ethernet driver attributes: *esp_eth_ioctl()*

```
/* get MAC address */
uint8_t mac_addr[6];
memset(mac_addr, 0, sizeof(mac_addr));
esp_eth_ioctl(eth_handle, ETH_CMD_G_MAC_ADDR, mac_addr);
ESP_LOGI(TAG, "Ethernet MAC Address: %02x:%02x:%02x:%02x:%02x:%02x",
         mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_
↪addr[5]);
```

(continues on next page)

(continued from previous page)

```
/* get PHY address */
int phy_addr = -1;
esp_eth_ioctl(eth_handle, ETH_CMD_G_PHY_ADDR, &phy_addr);
ESP_LOGI(TAG, "Ethernet PHY Address: %d", phy_addr);
```

Flow Control Ethernet on MCU usually has a limitation in the number of frames it can handle during network congestion, because of the limitation in RAM size. A sending station might be transmitting data faster than the peer end can accept it. The ethernet flow control mechanism allows the receiving node to signal the sender requesting the suspension of transmissions until the receiver catches up. The magic behind that is the pause frame, which was defined in IEEE 802.3x.

Pause frame is a special Ethernet frame used to carry the pause command, whose EtherType field is `0x8808`, with the Control opcode set to `0x0001`. Only stations configured for full-duplex operation may send pause frames. When a station wishes to pause the other end of a link, it sends a pause frame to the 48-bit reserved multicast address of `01-80-C2-00-00-01`. The pause frame also includes the period of pause time being requested, in the form of a two-byte integer, ranging from `0` to `65535`.

After the Ethernet driver installation, the flow control feature is disabled by default. You can enable it by:

```
bool flow_ctrl_enable = true;
esp_eth_ioctl(eth_handle, ETH_CMD_S_FLOW_CTRL, &flow_ctrl_enable);
```

One thing that should be kept in mind is that the pause frame ability is advertised to the peer end by PHY during auto-negotiation. The Ethernet driver sends a pause frame only when both sides of the link support it.

Application Examples

- Ethernet basic example: ethernet/basic
- Ethernet iperf example: ethernet/iperf
- Ethernet to Wi-Fi AP "router": network/eth2ap
- Wi-Fi station to Ethernet "bridge": network/sta2eth
- Most protocol examples should also work for Ethernet: protocols

Advanced Topics

Custom PHY Driver There are multiple PHY manufacturers with wide portfolios of chips available. The ESP-IDF already supports several PHY chips however one can easily get to a point where none of them satisfies the user's actual needs due to price, features, stock availability, etc.

Luckily, a management interface between EMAC and PHY is standardized by IEEE 802.3 in Section 22.2.4 Management Functions. It defines provisions of the so-called "MII Management Interface" to control the PHY and gather status from the PHY. A set of management registers is defined to control chip behavior, link properties, auto-negotiation configuration, etc. This basic management functionality is addressed by esp_eth/src/phy/esp_eth_phy_802_3.c in ESP-IDF and so it makes the creation of a new custom PHY chip driver quite a simple task.

Note: Always consult with PHY datasheet since some PHY chips may not comply with IEEE 802.3, Section 22.2.4. It does not mean you are not able to create a custom PHY driver, but it just requires more effort. You will have to define all PHY management functions.

The majority of PHY management functionality required by the ESP-IDF Ethernet driver is covered by the esp_eth/src/phy/esp_eth_phy_802_3.c. However, the following may require developing chip-specific management functions:

- Link status which is almost always chip-specific

- Chip initialization, even though not strictly required, should be customized to at least ensure that the expected chip is used
- Chip-specific features configuration

Steps to create a custom PHY driver:

1. Define vendor-specific registry layout based on the PHY datasheet. See esp_eth/src/phy/esp_eth_phy_ip101.c as an example.
2. Prepare derived PHY management object info structure which:
   - must contain at least parent IEEE 802.3 *phy_802_3_t* object
   - optionally contain additional variables needed to support non-IEEE 802.3 or customized functionality. See esp_eth/src/phy/esp_eth_phy_ksz80xx.c as an example.
3. Define chip-specific management call-back functions.
4. Initialize parent IEEE 802.3 object and re-assign chip-specific management call-back functions.

Once you finish the new custom PHY driver implementation, consider sharing it among other users via IDF Component Registry.

API Reference

Header File

- components/hal/include/hal/eth_types.h
- This header file can be included with:

```
#include "hal/eth_types.h"
```

Enumerations

enum **eth_data_interface_t**

Ethernet interface.

Values:

enumerator **EMAC_DATA_INTERFACE_RMII**

Reduced Media Independent Interface

enumerator **EMAC_DATA_INTERFACE_MII**

Media Independent Interface

enum **eth_link_t**

Ethernet link status.

Values:

enumerator **ETH_LINK_UP**

Ethernet link is up

enumerator **ETH_LINK_DOWN**

Ethernet link is down

enum **eth_speed_t**

Ethernet speed.

Values:

enumerator **ETH_SPEED_10M**

Ethernet speed is 10Mbps

enumerator **ETH_SPEED_100M**

Ethernet speed is 100Mbps

enumerator **ETH_SPEED_MAX**

Max speed mode (for checking purpose)

enum **eth_duplex_t**

Ethernet duplex mode.

Values:

enumerator **ETH_DUPLEX_HALF**

Ethernet is in half duplex

enumerator **ETH_DUPLEX_FULL**

Ethernet is in full duplex

enum **eth_checksum_t**

Ethernet Checksum.

Values:

enumerator **ETH_CHECKSUM_SW**

Ethernet checksum calculate by software

enumerator **ETH_CHECKSUM_HW**

Ethernet checksum calculate by hardware

enum **eth_mac_dma_burst_len_t**

Internal ethernet EMAC's DMA available burst sizes.

Values:

enumerator **ETH_DMA_BURST_LEN_32**

enumerator **ETH_DMA_BURST_LEN_16**

enumerator **ETH_DMA_BURST_LEN_8**

enumerator **ETH_DMA_BURST_LEN_4**

enumerator **ETH_DMA_BURST_LEN_2**

enumerator **ETH_DMA_BURST_LEN_1**

Header File

- components/esp_eth/include/esp_eth.h
- This header file can be included with:

```
#include "esp_eth.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Header File

- components/esp_eth/include/esp_eth_driver.h
- This header file can be included with:

```
#include "esp_eth_driver.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Functions

esp_err_t **esp_eth_driver_install** (const *esp_eth_config_t* *config, *esp_eth_handle_t* *out_hdl)

Install Ethernet driver.

Parameters
- **config** -- **[in]** configuration of the Ethernet driver
- **out_hdl** -- **[out]** handle of Ethernet driver

Returns
- ESP_OK: install esp_eth driver successfully
- ESP_ERR_INVALID_ARG: install esp_eth driver failed because of some invalid argument
- ESP_ERR_NO_MEM: install esp_eth driver failed because there's no memory for driver
- ESP_FAIL: install esp_eth driver failed because some other error occurred

esp_err_t **esp_eth_driver_uninstall** (*esp_eth_handle_t* hdl)

Uninstall Ethernet driver.

Note: It's not recommended to uninstall Ethernet driver unless it won't get used any more in application code. To uninstall Ethernet driver, you have to make sure, all references to the driver are released. Ethernet driver can only be uninstalled successfully when reference counter equals to one.

Parameters **hdl** -- **[in]** handle of Ethernet driver

Returns
- ESP_OK: uninstall esp_eth driver successfully
- ESP_ERR_INVALID_ARG: uninstall esp_eth driver failed because of some invalid argument
- ESP_ERR_INVALID_STATE: uninstall esp_eth driver failed because it has more than one reference

- ESP_FAIL: uninstall esp_eth driver failed because some other error occurred

esp_err_t **esp_eth_start** (*esp_eth_handle_t* hdl)

Start Ethernet driver **ONLY** in standalone mode (i.e. without TCP/IP stack)

Note: This API will start driver state machine and internal software timer (for checking link status).

Parameters **hdl** -- **[in]** handle of Ethernet driver

Returns

- ESP_OK: start esp_eth driver successfully
- ESP_ERR_INVALID_ARG: start esp_eth driver failed because of some invalid argument
- ESP_ERR_INVALID_STATE: start esp_eth driver failed because driver has started already
- ESP_FAIL: start esp_eth driver failed because some other error occurred

esp_err_t **esp_eth_stop** (*esp_eth_handle_t* hdl)

Stop Ethernet driver.

Note: This function does the oppsite operation of `esp_eth_start`.

Parameters **hdl** -- **[in]** handle of Ethernet driver

Returns

- ESP_OK: stop esp_eth driver successfully
- ESP_ERR_INVALID_ARG: stop esp_eth driver failed because of some invalid argument
- ESP_ERR_INVALID_STATE: stop esp_eth driver failed because driver has not started yet
- ESP_FAIL: stop esp_eth driver failed because some other error occurred

esp_err_t **esp_eth_update_input_path** (*esp_eth_handle_t* hdl, *esp_err_t* (*stack_input)(*esp_eth_handle_t* hdl, uint8_t *buffer, uint32_t length, void *priv), void *priv)

Update Ethernet data input path (i.e. specify where to pass the input buffer)

Note: After install driver, Ethernet still don't know where to deliver the input buffer. In fact, this API registers a callback function which get invoked when Ethernet received new packets.

Parameters

- **hdl** -- **[in]** handle of Ethernet driver
- **stack_input** -- **[in]** function pointer, which does the actual process on incoming packets
- **priv** -- **[in]** private resource, which gets passed to `stack_input` callback without any modification

Returns

- ESP_OK: update input path successfully
- ESP_ERR_INVALID_ARG: update input path failed because of some invalid argument
- ESP_FAIL: update input path failed because some other error occurred

esp_err_t **esp_eth_transmit** (*esp_eth_handle_t* hdl, void *buf, size_t length)

General Transmit.

Parameters

- **hdl** -- **[in]** handle of Ethernet driver
- **buf** -- **[in]** buffer of the packet to transfer
- **length** -- **[in]** length of the buffer to transfer

Returns

- ESP_OK: transmit frame buffer successfully
- ESP_ERR_INVALID_ARG: transmit frame buffer failed because of some invalid argument
- ESP_ERR_INVALID_STATE: invalid driver state (e.i. driver is not started)
- ESP_ERR_TIMEOUT: transmit frame buffer failed because HW was not get available in predefined period
- ESP_FAIL: transmit frame buffer failed because some other error occurred

esp_err_t **esp_eth_transmit_vargs** (*esp_eth_handle_t* hdl, uint32_t argc, ...)

Special Transmit with variable number of arguments.

Parameters

- **hdl** -- **[in]** handle of Ethernet driver
- **argc** -- **[in]** number variable arguments
- ... -- variable arguments

Returns

- ESP_OK: transmit successful
- ESP_ERR_INVALID_STATE: invalid driver state (e.i. driver is not started)
- ESP_ERR_TIMEOUT: transmit frame buffer failed because HW was not get available in predefined period
- ESP_FAIL: transmit frame buffer failed because some other error occurred

esp_err_t **esp_eth_ioctl** (*esp_eth_handle_t* hdl, *esp_eth_io_cmd_t* cmd, void *data)

Misc IO function of Ethernet driver.

The following common IO control commands are supported:

- ETH_CMD_S_MAC_ADDR sets Ethernet interface MAC address. data argument is pointer to MAC address buffer with expected size of 6 bytes.
- ETH_CMD_G_MAC_ADDR gets Ethernet interface MAC address. data argument is pointer to a buffer to which MAC address is to be copied. The buffer size must be at least 6 bytes.
- ETH_CMD_S_PHY_ADDR sets PHY address in range of <0-31>. data argument is pointer to memory of uint32_t datatype from where the configuration option is read.
- ETH_CMD_G_PHY_ADDR gets PHY address. data argument is pointer to memory of uint32_t datatype to which the PHY address is to be stored.
- ETH_CMD_S_AUTONEGO enables or disables Ethernet link speed and duplex mode autonegotiation. data argument is pointer to memory of bool datatype from which the configuration option is read. Preconditions: Ethernet driver needs to be stopped.
- ETH_CMD_G_AUTONEGO gets current configuration of the Ethernet link speed and duplex mode autonegotiation. data argument is pointer to memory of bool datatype to which the current configuration is to be stored.
- ETH_CMD_S_SPEED sets the Ethernet link speed. data argument is pointer to memory of eth_speed_t datatype from which the configuration option is read. Preconditions: Ethernet driver needs to be stopped and auto-negotiation disabled.
- ETH_CMD_G_SPEED gets current Ethernet link speed. data argument is pointer to memory of eth_speed_t datatype to which the speed is to be stored.
- ETH_CMD_S_PROMISCUOUS sets/resets Ethernet interface promiscuous mode. data argument is pointer to memory of bool datatype from which the configuration option is read.
- ETH_CMD_S_FLOW_CTRL sets/resets Ethernet interface flow control. data argument is pointer to memory of bool datatype from which the configuration option is read.
- ETH_CMD_S_DUPLEX_MODE sets the Ethernet duplex mode. data argument is pointer to memory of eth_duplex_t datatype from which the configuration option is read. Preconditions: Ethernet driver needs to be stopped and auto-negotiation disabled.
- ETH_CMD_G_DUPLEX_MODE gets current Ethernet link duplex mode. data argument is pointer to memory of eth_duplex_t datatype to which the duplex mode is to be stored.
- ETH_CMD_S_PHY_LOOPBACK sets/resets PHY to/from loopback mode. data argument is pointer to memory of bool datatype from which the configuration option is read.

- Note that additional control commands may be available for specific MAC or PHY chips. Please consult specific MAC or PHY documentation or driver code.

Parameters
- **hdl** -- **[in]** handle of Ethernet driver
- **cmd** -- **[in]** IO control command
- **data** -- **[inout]** address of data for `set` command or address where to store the data when used with `get` command

Returns
- ESP_OK: process io command successfully
- ESP_ERR_INVALID_ARG: process io command failed because of some invalid argument
- ESP_FAIL: process io command failed because some other error occurred
- ESP_ERR_NOT_SUPPORTED: requested feature is not supported

esp_err_t **esp_eth_get_phy_instance**(*esp_eth_handle_t* hdl, *esp_eth_phy_t* **phy)

Get PHY instance memory address.

Parameters
- **hdl** -- **[in]** handle of Ethernet driver
- **phy** -- **[out]** pointer to memory to store the instance

Returns esp_err_t
- ESP_OK: success
- ESP_ERR_INVALID_ARG: failed because of some invalid argument

esp_err_t **esp_eth_get_mac_instance**(*esp_eth_handle_t* hdl, *esp_eth_mac_t* **mac)

Get MAC instance memory address.

Parameters
- **hdl** -- **[in]** handle of Ethernet driver
- **mac** -- **[out]** pointer to memory to store the instance

Returns esp_err_t
- ESP_OK: success
- ESP_ERR_INVALID_ARG: failed because of some invalid argument

esp_err_t **esp_eth_increase_reference**(*esp_eth_handle_t* hdl)

Increase Ethernet driver reference.

Note: Ethernet driver handle can be obtained by os timer, netif, etc. It's dangerous when thread A is using Ethernet but thread B uninstall the driver. Using reference counter can prevent such risk, but care should be taken, when you obtain Ethernet driver, this API must be invoked so that the driver won't be uninstalled during your using time.

Parameters **hdl** -- **[in]** handle of Ethernet driver

Returns
- ESP_OK: increase reference successfully
- ESP_ERR_INVALID_ARG: increase reference failed because of some invalid argument

esp_err_t **esp_eth_decrease_reference**(*esp_eth_handle_t* hdl)

Decrease Ethernet driver reference.

Parameters **hdl** -- **[in]** handle of Ethernet driver

Returns
- ESP_OK: increase reference successfully
- ESP_ERR_INVALID_ARG: increase reference failed because of some invalid argument

Structures

struct **esp_eth_config_t**

Configuration of Ethernet driver.

Public Members

esp_eth_mac_t ***mac**

Ethernet MAC object.

esp_eth_phy_t ***phy**

Ethernet PHY object.

uint32_t **check_link_period_ms**

Period time of checking Ethernet link status.

esp_err_t (***stack_input**)(*esp_eth_handle_t* eth_handle, uint8_t *buffer, uint32_t length, void *priv)

Input frame buffer to user's stack.

Param eth_handle **[in]** handle of Ethernet driver
Param buffer **[in]** frame buffer that will get input to upper stack
Param length **[in]** length of the frame buffer
Return

- ESP_OK: input frame buffer to upper stack successfully
- ESP_FAIL: error occurred when inputting buffer to upper stack

esp_err_t (***on_lowlevel_init_done**)(*esp_eth_handle_t* eth_handle)

Callback function invoked when lowlevel initialization is finished.

Param eth_handle **[in]** handle of Ethernet driver
Return

- ESP_OK: process extra lowlevel initialization successfully
- ESP_FAIL: error occurred when processing extra lowlevel initialization

esp_err_t (***on_lowlevel_deinit_done**)(*esp_eth_handle_t* eth_handle)

Callback function invoked when lowlevel deinitialization is finished.

Param eth_handle **[in]** handle of Ethernet driver
Return

- ESP_OK: process extra lowlevel deinitialization successfully
- ESP_FAIL: error occurred when processing extra lowlevel deinitialization

esp_err_t (***read_phy_reg**)(*esp_eth_handle_t* eth_handle, uint32_t phy_addr, uint32_t phy_reg, uint32_t *reg_value)

Read PHY register.

Note: Usually the PHY register read/write function is provided by MAC (SMI interface), but if the PHY device is managed by other interface (e.g. I2C), then user needs to implement the corresponding read/write. Setting this to NULL means your PHY device is managed by MAC's SMI interface.

Param eth_handle **[in]** handle of Ethernet driver
Param phy_addr **[in]** PHY chip address (0~31)
Param phy_reg **[in]** PHY register index code

Param reg_value **[out]** PHY register value
Return
- ESP_OK: read PHY register successfully
- ESP_ERR_INVALID_ARG: read PHY register failed because of invalid argument
- ESP_ERR_TIMEOUT: read PHY register failed because of timeout
- ESP_FAIL: read PHY register failed because some other error occurred

esp_err_t (***write_phy_reg**)(*esp_eth_handle_t* eth_handle, uint32_t phy_addr, uint32_t phy_reg, uint32_t reg_value)

Write PHY register.

Note: Usually the PHY register read/write function is provided by MAC (SMI interface), but if the PHY device is managed by other interface (e.g. I2C), then user needs to implement the corresponding read/write. Setting this to NULL means your PHY device is managed by MAC's SMI interface.

Param eth_handle **[in]** handle of Ethernet driver
Param phy_addr **[in]** PHY chip address (0~31)
Param phy_reg **[in]** PHY register index code
Param reg_value **[in]** PHY register value
Return
- ESP_OK: write PHY register successfully
- ESP_ERR_INVALID_ARG: read PHY register failed because of invalid argument
- ESP_ERR_TIMEOUT: write PHY register failed because of timeout
- ESP_FAIL: write PHY register failed because some other error occurred

struct **esp_eth_phy_reg_rw_data_t**

Data structure to Read/Write PHY register via ioctl API.

Public Members

uint32_t **reg_addr**

PHY register address

uint32_t ***reg_value_p**

Pointer to a memory where the register value is read/written

Macros

ETH_DEFAULT_CONFIG(emac, ephy)

Default configuration for Ethernet driver.

Type Definitions

typedef void ***esp_eth_handle_t**

Handle of Ethernet driver.

Enumerations

enum **esp_eth_io_cmd_t**

Command list for ioctl API.

Values:

enumerator **ETH_CMD_G_MAC_ADDR**

Get MAC address

enumerator **ETH_CMD_S_MAC_ADDR**

Set MAC address

enumerator **ETH_CMD_G_PHY_ADDR**

Get PHY address

enumerator **ETH_CMD_S_PHY_ADDR**

Set PHY address

enumerator **ETH_CMD_G_AUTONEGO**

Get PHY Auto Negotiation

enumerator **ETH_CMD_S_AUTONEGO**

Set PHY Auto Negotiation

enumerator **ETH_CMD_G_SPEED**

Get Speed

enumerator **ETH_CMD_S_SPEED**

Set Speed

enumerator **ETH_CMD_S_PROMISCUOUS**

Set promiscuous mode

enumerator **ETH_CMD_S_FLOW_CTRL**

Set flow control

enumerator **ETH_CMD_G_DUPLEX_MODE**

Get Duplex mode

enumerator **ETH_CMD_S_DUPLEX_MODE**

Set Duplex mode

enumerator **ETH_CMD_S_PHY_LOOPBACK**

Set PHY loopback

enumerator **ETH_CMD_READ_PHY_REG**

Read PHY register

enumerator **ETH_CMD_WRITE_PHY_REG**

Write PHY register

enumerator **ETH_CMD_CUSTOM_MAC_CMDS**

enumerator **ETH_CMD_CUSTOM_PHY_CMDS**

Header File

- components/esp_eth/include/esp_eth_com.h
- This header file can be included with:

```
#include "esp_eth_com.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Structures

struct **esp_eth_mediator_s**

Ethernet mediator.

Public Members

esp_err_t (***phy_reg_read**)(*esp_eth_mediator_t* *eth, uint32_t phy_addr, uint32_t phy_reg, uint32_t *reg_value)

Read PHY register.

Param eth **[in]** mediator of Ethernet driver
Param phy_addr **[in]** PHY Chip address (0~31)
Param phy_reg **[in]** PHY register index code
Param reg_value **[out]** PHY register value
Return
- ESP_OK: read PHY register successfully
- ESP_FAIL: read PHY register failed because some error occurred

esp_err_t (***phy_reg_write**)(*esp_eth_mediator_t* *eth, uint32_t phy_addr, uint32_t phy_reg, uint32_t reg_value)

Write PHY register.

Param eth **[in]** mediator of Ethernet driver
Param phy_addr **[in]** PHY Chip address (0~31)
Param phy_reg **[in]** PHY register index code
Param reg_value **[in]** PHY register value
Return
- ESP_OK: write PHY register successfully
- ESP_FAIL: write PHY register failed because some error occurred

esp_err_t (***stack_input**)(*esp_eth_mediator_t* *eth, uint8_t *buffer, uint32_t length)

Deliver packet to upper stack.

Param eth **[in]** mediator of Ethernet driver
Param buffer **[in]** packet buffer
Param length **[in]** length of the packet
Return
- ESP_OK: deliver packet to upper stack successfully
- ESP_FAIL: deliver packet failed because some error occurred

esp_err_t (***on_state_changed**)(*esp_eth_mediator_t* *eth, *esp_eth_state_t* state, void *args)

Callback on Ethernet state changed.

Param eth **[in]** mediator of Ethernet driver
Param state **[in]** new state
Param args **[in]** optional argument for the new state
Return

- ESP_OK: process the new state successfully
- ESP_FAIL: process the new state failed because some error occurred

Macros

ETH_CMD_CUSTOM_MAC_CMDS_OFFSET

Offset for start of MAC custom ioctl commands.

ETH_CMD_CUSTOM_PHY_CMDS_OFFSET

Offset for start of PHY custom ioctl commands.

Type Definitions

typedef struct *esp_eth_mediator_s* **esp_eth_mediator_t**

Ethernet mediator.

Enumerations

enum **esp_eth_state_t**

Ethernet driver state.

Values:

enumerator **ETH_STATE_LLINIT**

Lowlevel init done

enumerator **ETH_STATE_DEINIT**

Deinit done

enumerator **ETH_STATE_LINK**

Link status changed

enumerator **ETH_STATE_SPEED**

Speed updated

enumerator **ETH_STATE_DUPLEX**

Duplex updated

enumerator **ETH_STATE_PAUSE**

Pause ability updated

enum **eth_event_t**

Ethernet event declarations.

Values:

enumerator **ETHERNET_EVENT_START**

Ethernet driver start

enumerator **ETHERNET_EVENT_STOP**

Ethernet driver stop

enumerator **ETHERNET_EVENT_CONNECTED**

Ethernet got a valid link

enumerator **ETHERNET_EVENT_DISCONNECTED**

Ethernet lost a valid link

Header File

- components/esp_eth/include/esp_eth_mac.h
- This header file can be included with:

```
#include "esp_eth_mac.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Structures

struct **esp_eth_mac_s**

Ethernet MAC.

Public Members

esp_err_t (*__set_mediator__)(*esp_eth_mac_t* *mac, *esp_eth_mediator_t* *eth)

Set mediator for Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Param eth **[in]** Ethernet mediator
Return

- ESP_OK: set mediator for Ethernet MAC successfully
- ESP_ERR_INVALID_ARG: set mediator for Ethernet MAC failed because of invalid argument

esp_err_t (*__init__)(*esp_eth_mac_t* *mac)

Initialize Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Return

- ESP_OK: initialize Ethernet MAC successfully
- ESP_ERR_TIMEOUT: initialize Ethernet MAC failed because of timeout
- ESP_FAIL: initialize Ethernet MAC failed because some other error occurred

esp_err_t (*`deinit`)(*esp_eth_mac_t* *mac)

Deinitialize Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Return
- ESP_OK: deinitialize Ethernet MAC successfully
- ESP_FAIL: deinitialize Ethernet MAC failed because some error occurred

esp_err_t (*`start`)(*esp_eth_mac_t* *mac)

Start Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Return
- ESP_OK: start Ethernet MAC successfully
- ESP_FAIL: start Ethernet MAC failed because some other error occurred

esp_err_t (*`stop`)(*esp_eth_mac_t* *mac)

Stop Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Return
- ESP_OK: stop Ethernet MAC successfully
- ESP_FAIL: stop Ethernet MAC failed because some error occurred

esp_err_t (*`transmit`)(*esp_eth_mac_t* *mac, uint8_t *buf, uint32_t length)

Transmit packet from Ethernet MAC.

Note: Returned error codes may differ for each specific MAC chip.

Param mac **[in]** Ethernet MAC instance
Param buf **[in]** packet buffer to transmit
Param length **[in]** length of packet
Return
- ESP_OK: transmit packet successfully
- ESP_ERR_INVALID_SIZE: number of actually sent bytes differs to expected
- ESP_FAIL: transmit packet failed because some other error occurred

esp_err_t (*`transmit_vargs`)(*esp_eth_mac_t* *mac, uint32_t argc, va_list args)

Transmit packet from Ethernet MAC constructed with special parameters at Layer2.

Note: Typical intended use case is to make possible to construct a frame from multiple higher layer buffers without a need of buffer reallocations. However, other use cases are not limited.

Note: Returned error codes may differ for each specific MAC chip.

Param mac **[in]** Ethernet MAC instance
Param argc **[in]** number variable arguments
Param args **[in]** variable arguments
Return
- ESP_OK: transmit packet successfully
- ESP_ERR_INVALID_SIZE: number of actually sent bytes differs to expected
- ESP_FAIL: transmit packet failed because some other error occurred

esp_err_t (*`receive`)(*esp_eth_mac_t* *mac, uint8_t *buf, uint32_t *length)

Receive packet from Ethernet MAC.

Note: Memory of buf is allocated in the Layer2, make sure it get free after process.

Note: Before this function got invoked, the value of "length" should set by user, equals the size of buffer. After the function returned, the value of "length" means the real length of received data.

Param mac **[in]** Ethernet MAC instance
Param buf **[out]** packet buffer which will preserve the received frame
Param length **[out]** length of the received packet
Return
- ESP_OK: receive packet successfully
- ESP_ERR_INVALID_ARG: receive packet failed because of invalid argument
- ESP_ERR_INVALID_SIZE: input buffer size is not enough to hold the incoming data. in this case, value of returned "length" indicates the real size of incoming data.
- ESP_FAIL: receive packet failed because some other error occurred

esp_err_t (*`read_phy_reg`)(*esp_eth_mac_t* *mac, uint32_t phy_addr, uint32_t phy_reg, uint32_t *reg_value)

Read PHY register.

Param mac **[in]** Ethernet MAC instance
Param phy_addr **[in]** PHY chip address (0~31)
Param phy_reg **[in]** PHY register index code
Param reg_value **[out]** PHY register value
Return
- ESP_OK: read PHY register successfully
- ESP_ERR_INVALID_ARG: read PHY register failed because of invalid argument
- ESP_ERR_INVALID_STATE: read PHY register failed because of wrong state of MAC
- ESP_ERR_TIMEOUT: read PHY register failed because of timeout
- ESP_FAIL: read PHY register failed because some other error occurred

esp_err_t (*`write_phy_reg`)(*esp_eth_mac_t* *mac, uint32_t phy_addr, uint32_t phy_reg, uint32_t reg_value)

Write PHY register.

Param mac **[in]** Ethernet MAC instance
Param phy_addr **[in]** PHY chip address (0~31)
Param phy_reg **[in]** PHY register index code
Param reg_value **[in]** PHY register value
Return
- ESP_OK: write PHY register successfully
- ESP_ERR_INVALID_STATE: write PHY register failed because of wrong state of MAC
- ESP_ERR_TIMEOUT: write PHY register failed because of timeout
- ESP_FAIL: write PHY register failed because some other error occurred

esp_err_t (*`set_addr`)(*esp_eth_mac_t* *mac, uint8_t *addr)

Set MAC address.

Param mac **[in]** Ethernet MAC instance
Param addr **[in]** MAC address

Return
- ESP_OK: set MAC address successfully
- ESP_ERR_INVALID_ARG: set MAC address failed because of invalid argument
- ESP_FAIL: set MAC address failed because some other error occurred

esp_err_t (*`get_addr`)(*esp_eth_mac_t* *mac, uint8_t *addr)

Get MAC address.

Param mac **[in]** Ethernet MAC instance
Param addr **[out]** MAC address
Return
- ESP_OK: get MAC address successfully
- ESP_ERR_INVALID_ARG: get MAC address failed because of invalid argument
- ESP_FAIL: get MAC address failed because some other error occurred

esp_err_t (*`set_speed`)(*esp_eth_mac_t* *mac, *eth_speed_t* speed)

Set speed of MAC.

Param ma:c **[in]** Ethernet MAC instance
Param speed **[in]** MAC speed
Return
- ESP_OK: set MAC speed successfully
- ESP_ERR_INVALID_ARG: set MAC speed failed because of invalid argument
- ESP_FAIL: set MAC speed failed because some other error occurred

esp_err_t (*`set_duplex`)(*esp_eth_mac_t* *mac, *eth_duplex_t* duplex)

Set duplex mode of MAC.

Param mac **[in]** Ethernet MAC instance
Param duplex **[in]** MAC duplex
Return
- ESP_OK: set MAC duplex mode successfully
- ESP_ERR_INVALID_ARG: set MAC duplex failed because of invalid argument
- ESP_FAIL: set MAC duplex failed because some other error occurred

esp_err_t (*`set_link`)(*esp_eth_mac_t* *mac, *eth_link_t* link)

Set link status of MAC.

Param mac **[in]** Ethernet MAC instance
Param link **[in]** Link status
Return
- ESP_OK: set link status successfully
- ESP_ERR_INVALID_ARG: set link status failed because of invalid argument
- ESP_FAIL: set link status failed because some other error occurred

esp_err_t (*`set_promiscuous`)(*esp_eth_mac_t* *mac, bool enable)

Set promiscuous of MAC.

Param mac **[in]** Ethernet MAC instance
Param enable **[in]** set true to enable promiscuous mode; set false to disable promiscuous mode
Return
- ESP_OK: set promiscuous mode successfully
- ESP_FAIL: set promiscuous mode failed because some error occurred

esp_err_t (*`enable_flow_ctrl`)(*esp_eth_mac_t* *mac, bool enable)

Enable flow control on MAC layer or not.

Param mac **[in]** Ethernet MAC instance

Param enable **[in]** set true to enable flow control; set false to disable flow control
Return
- ESP_OK: set flow control successfully
- ESP_FAIL: set flow control failed because some error occurred

esp_err_t (***set_peer_pause_ability**)(*esp_eth_mac_t* *mac, uint32_t ability)

Set the PAUSE ability of peer node.

Param mac **[in]** Ethernet MAC instance
Param ability **[in]** zero indicates that pause function is supported by link partner; non-zero indicates that pause function is not supported by link partner
Return
- ESP_OK: set peer pause ability successfully
- ESP_FAIL: set peer pause ability failed because some error occurred

esp_err_t (***custom_ioctl**)(*esp_eth_mac_t* *mac, int cmd, void *data)

Custom IO function of MAC driver. This function is intended to extend common options of esp_eth_ioctl to cover specifics of MAC chip.

Note: This function may not be assigned when the MAC chip supports only most common set of configuration options.

Param mac **[in]** Ethernet MAC instance
Param cmd **[in]** IO control command
Param data **[inout]** address of data for `set` command or address where to store the data when used with `get` command
Return
- ESP_OK: process io command successfully
- ESP_ERR_INVALID_ARG: process io command failed because of some invalid argument
- ESP_FAIL: process io command failed because some other error occurred
- ESP_ERR_NOT_SUPPORTED: requested feature is not supported

esp_err_t (***del**)(*esp_eth_mac_t* *mac)

Free memory of Ethernet MAC.

Param mac **[in]** Ethernet MAC instance
Return
- ESP_OK: free Ethernet MAC instance successfully
- ESP_FAIL: free Ethernet MAC instance failed because some error occurred

struct **eth_mac_config_t**

Configuration of Ethernet MAC object.

Public Members

uint32_t **sw_reset_timeout_ms**

Software reset timeout value (Unit: ms)

uint32_t **rx_task_stack_size**

Stack size of the receive task

uint32_t **rx_task_prio**

Priority of the receive task

uint32_t **flags**

Flags that specify extra capability for mac driver

Macros

ETH_MAC_FLAG_WORK_WITH_CACHE_DISABLE

MAC driver can work when cache is disabled

ETH_MAC_FLAG_PIN_TO_CORE

Pin MAC task to the CPU core where driver installation happened

ETH_MAC_DEFAULT_CONFIG()

Default configuration for Ethernet MAC object.

Type Definitions

typedef struct *esp_eth_mac_s* **esp_eth_mac_t**

Ethernet MAC.

Header File

- components/esp_eth/include/esp_eth_mac_esp.h
- This header file can be included with:

```
#include "esp_eth_mac_esp.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Header File

- components/esp_eth/include/esp_eth_mac_spi.h
- This header file can be included with:

```
#include "esp_eth_mac_spi.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Structures

struct **eth_spi_custom_driver_config_t**

Custom SPI Driver Configuration. This structure declares configuration and callback functions to access Ethernet SPI module via user's custom SPI driver.

Public Members

void ***config**

Custom driver specific configuration data used by *init()* function.

Note: Type and its content is fully under user's control

void *(***init**)(const void *spi_config)

Custom driver SPI Initialization.

Note: return type and its content is fully under user's control

Param spi_config **[in]** Custom driver specific configuration
Return
- spi_ctx: when initialization is successful, a pointer to context structure holding all variables needed for subsequent SPI access operations (e.g. SPI bus identification, mutexes, etc.)
- NULL: driver initialization failed

esp_err_t (***deinit**)(void *spi_ctx)

Custom driver De-initialization.

Param spi_ctx **[in]** a pointer to driver specific context structure
Return
- ESP_OK: driver de-initialization was successful
- ESP_FAIL: driver de-initialization failed
- any other failure codes are allowed to be used to provide failure isolation

esp_err_t (***read**)(void *spi_ctx, uint32_t cmd, uint32_t addr, void *data, uint32_t data_len)

Custom driver SPI read.

Note: The read function is responsible to construct command, address and data fields of the SPI frame in format expected by particular SPI Ethernet module

Param spi_ctx **[in]** a pointer to driver specific context structure
Param cmd **[in]** command
Param addr **[in]** register address
Param data **[out]** read data
Param data_len **[in]** read data length in bytes
Return
- ESP_OK: read was successful
- ESP_FAIL: read failed
- any other failure codes are allowed to be used to provide failure isolation

esp_err_t (***write**)(void *spi_ctx, uint32_t cmd, uint32_t addr, const void *data, uint32_t data_len)

Custom driver SPI write.

Note: The write function is responsible to construct command, address and data fields of the SPI frame in format expected by particular SPI Ethernet module

Param spi_ctx **[in]** a pointer to driver specific context structure
Param cmd **[in]** command
Param addr **[in]** register address
Param data **[in]** data to write
Param data_len **[in]** length of data to write in bytes
Return

- ESP_OK: write was successful
- ESP_FAIL: write failed
- any other failure codes are allowed to be used to provide failure isolation

Macros

ETH_DEFAULT_SPI

Default configuration of the custom SPI driver. Internal ESP-IDF SPI Master driver is used by default.

Header File

- components/esp_eth/include/esp_eth_phy.h
- This header file can be included with:

```
#include "esp_eth_phy.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Functions

esp_eth_phy_t ***esp_eth_phy_new_ip101** (const *eth_phy_config_t* *config)

Create a PHY instance of IP101.

Parameters **config** -- **[in]** configuration of PHY
Returns

- instance: create PHY instance successfully
- NULL: create PHY instance failed because some error occurred

esp_eth_phy_t ***esp_eth_phy_new_rtl8201** (const *eth_phy_config_t* *config)

Create a PHY instance of RTL8201.

Parameters **config** -- **[in]** configuration of PHY
Returns

- instance: create PHY instance successfully
- NULL: create PHY instance failed because some error occurred

esp_eth_phy_t ***esp_eth_phy_new_lan87xx** (const *eth_phy_config_t* *config)

Create a PHY instance of LAN87xx.

Parameters **config** -- **[in]** configuration of PHY

Returns
- instance: create PHY instance successfully
- NULL: create PHY instance failed because some error occurred

esp_eth_phy_t ***esp_eth_phy_new_dp83848** (const *eth_phy_config_t* *config)

Create a PHY instance of DP83848.

Parameters **config** -- **[in]** configuration of PHY
Returns
- instance: create PHY instance successfully
- NULL: create PHY instance failed because some error occurred

esp_eth_phy_t ***esp_eth_phy_new_ksz80xx** (const *eth_phy_config_t* *config)

Create a PHY instance of KSZ80xx.

The phy model from the KSZ80xx series is detected automatically. If the driver is unable to detect a supported model, `NULL` is returned.

Currently, the following models are supported: KSZ8001, KSZ8021, KSZ8031, KSZ8041, KSZ8051, KSZ8061, KSZ8081, KSZ8091

Parameters **config** -- **[in]** configuration of PHY
Returns
- instance: create PHY instance successfully
- NULL: create PHY instance failed because some error occurred

Structures

struct **esp_eth_phy_s**

Ethernet PHY.

Public Members

esp_err_t (***set_mediator**)(*esp_eth_phy_t* *phy, *esp_eth_mediator_t* *mediator)

Set mediator for PHY.

Param phy **[in]** Ethernet PHY instance
Param mediator **[in]** mediator of Ethernet driver
Return
- ESP_OK: set mediator for Ethernet PHY instance successfully
- ESP_ERR_INVALID_ARG: set mediator for Ethernet PHY instance failed because of some invalid arguments

esp_err_t (***reset**)(*esp_eth_phy_t* *phy)

Software Reset Ethernet PHY.

Param phy **[in]** Ethernet PHY instance
Return
- ESP_OK: reset Ethernet PHY successfully
- ESP_FAIL: reset Ethernet PHY failed because some error occurred

esp_err_t (***reset_hw**)(*esp_eth_phy_t* *phy)

Hardware Reset Ethernet PHY.

Note: Hardware reset is mostly done by pull down and up PHY's nRST pin

Param phy **[in]** Ethernet PHY instance

Return
- ESP_OK: reset Ethernet PHY successfully
- ESP_FAIL: reset Ethernet PHY failed because some error occurred

esp_err_t (*`init`)(*esp_eth_phy_t* *phy)

Initialize Ethernet PHY.

Param phy **[in]** Ethernet PHY instance
Return
- ESP_OK: initialize Ethernet PHY successfully
- ESP_FAIL: initialize Ethernet PHY failed because some error occurred

esp_err_t (*`deinit`)(*esp_eth_phy_t* *phy)

Deinitialize Ethernet PHY.

Param phy **[in]** Ethernet PHY instance
Return
- ESP_OK: deinitialize Ethernet PHY successfully
- ESP_FAIL: deinitialize Ethernet PHY failed because some error occurred

esp_err_t (*`autonego_ctrl`)(*esp_eth_phy_t* *phy, *eth_phy_autoneg_cmd_t* cmd, bool *autonego_en_stat)

Configure auto negotiation.

Param phy **[in]** Ethernet PHY instance
Param cmd **[in]** Configuration command, it is possible to Enable (restart), Disable or get current status of PHY auto negotiation
Param autonego_en_stat **[out]** Address where to store current status of auto negotiation configuration
Return
- ESP_OK: restart auto negotiation successfully
- ESP_FAIL: restart auto negotiation failed because some error occurred
- ESP_ERR_INVALID_ARG: invalid command

esp_err_t (*`get_link`)(*esp_eth_phy_t* *phy)

Get Ethernet PHY link status.

Param phy **[in]** Ethernet PHY instance
Return
- ESP_OK: get Ethernet PHY link status successfully
- ESP_FAIL: get Ethernet PHY link status failed because some error occurred

esp_err_t (*`set_link`)(*esp_eth_phy_t* *phy, *eth_link_t* link)

Set Ethernet PHY link status.

Param phy **[in]** Ethernet PHY instance
Param link **[in]** new link status
Return
- ESP_OK: set Ethernet PHY link status successfully
- ESP_FAIL: set Ethernet PHY link status failed because some error occurred

esp_err_t (*`pwrctl`)(*esp_eth_phy_t* *phy, bool enable)

Power control of Ethernet PHY.

Param phy **[in]** Ethernet PHY instance
Param enable **[in]** set true to power on Ethernet PHY; ser false to power off Ethernet PHY
Return
- ESP_OK: control Ethernet PHY power successfully
- ESP_FAIL: control Ethernet PHY power failed because some error occurred

esp_err_t (*`set_addr`)(*esp_eth_phy_t* *phy, uint32_t addr)

Set PHY chip address.

Param phy **[in]** Ethernet PHY instance
Param addr **[in]** PHY chip address
Return
- ESP_OK: set Ethernet PHY address successfully
- ESP_FAIL: set Ethernet PHY address failed because some error occurred

esp_err_t (*`get_addr`)(*esp_eth_phy_t* *phy, uint32_t *addr)

Get PHY chip address.

Param phy **[in]** Ethernet PHY instance
Param addr **[out]** PHY chip address
Return
- ESP_OK: get Ethernet PHY address successfully
- ESP_ERR_INVALID_ARG: get Ethernet PHY address failed because of invalid argument

esp_err_t (*`advertise_pause_ability`)(*esp_eth_phy_t* *phy, uint32_t ability)

Advertise pause function supported by MAC layer.

Param phy **[in]** Ethernet PHY instance
Param addr **[out]** Pause ability
Return
- ESP_OK: Advertise pause ability successfully
- ESP_ERR_INVALID_ARG: Advertise pause ability failed because of invalid argument

esp_err_t (*`loopback`)(*esp_eth_phy_t* *phy, bool enable)

Sets the PHY to loopback mode.

Param phy **[in]** Ethernet PHY instance
Param enable **[in]** enables or disables PHY loopback
Return
- ESP_OK: PHY instance loopback mode has been configured successfully
- ESP_FAIL: PHY instance loopback configuration failed because some error occurred

esp_err_t (*`set_speed`)(*esp_eth_phy_t* *phy, *eth_speed_t* speed)

Sets PHY speed mode.

Note: Autonegotiation feature needs to be disabled prior to calling this function for the new setting to be applied

---

Param phy **[in]** Ethernet PHY instance
Param speed **[in]** Speed mode to be set
Return
- ESP_OK: PHY instance speed mode has been configured successfully
- ESP_FAIL: PHY instance speed mode configuration failed because some error occurred

esp_err_t (*`set_duplex`)(*esp_eth_phy_t* *phy, *eth_duplex_t* duplex)

Sets PHY duplex mode.

Note: Autonegotiation feature needs to be disabled prior to calling this function for the new setting to be applied

Param phy **[in]** Ethernet PHY instance
Param duplex **[in]** Duplex mode to be set
Return
- ESP_OK: PHY instance duplex mode has been configured successfully
- ESP_FAIL: PHY instance duplex mode configuration failed because some error occurred

esp_err_t (*`custom_ioctl`)(*esp_eth_phy_t* *phy, int cmd, void *data)

Custom IO function of PHY driver. This function is intended to extend common options of esp_eth_ioctl to cover specifics of PHY chip.

Note: This function may not be assigned when the PHY chip supports only most common set of configuration options.

Param phy **[in]** Ethernet PHY instance
Param cmd **[in]** IO control command
Param data **[inout]** address of data for `set` command or address where to store the data when used with `get` command
Return
- ESP_OK: process io command successfully
- ESP_ERR_INVALID_ARG: process io command failed because of some invalid argument
- ESP_FAIL: process io command failed because some other error occurred
- ESP_ERR_NOT_SUPPORTED: requested feature is not supported

esp_err_t (*`del`)(*esp_eth_phy_t* *phy)

Free memory of Ethernet PHY instance.

Param phy **[in]** Ethernet PHY instance
Return
- ESP_OK: free PHY instance successfully
- ESP_FAIL: free PHY instance failed because some error occurred

struct **`eth_phy_config_t`**

Ethernet PHY configuration.

Public Members

int32_t **`phy_addr`**

PHY address, set -1 to enable PHY address detection at initialization stage

uint32_t **`reset_timeout_ms`**

Reset timeout value (Unit: ms)

uint32_t **`autonego_timeout_ms`**

Auto-negotiation timeout value (Unit: ms)

int **reset_gpio_num**

Reset GPIO number, -1 means no hardware reset

Macros

ESP_ETH_PHY_ADDR_AUTO

ETH_PHY_DEFAULT_CONFIG()

Default configuration for Ethernet PHY object.

Type Definitions

typedef struct *esp_eth_phy_s* **esp_eth_phy_t**

Ethernet PHY.

Enumerations

enum **eth_phy_autoneg_cmd_t**

Auto-negotiation control commands.

Values:

enumerator **ESP_ETH_PHY_AUTONEGO_RESTART**

enumerator **ESP_ETH_PHY_AUTONEGO_EN**

enumerator **ESP_ETH_PHY_AUTONEGO_DIS**

enumerator **ESP_ETH_PHY_AUTONEGO_G_STAT**

Header File

- components/esp_eth/include/esp_eth_phy_802_3.h
- This header file can be included with:

```
#include "esp_eth_phy_802_3.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Functions

esp_err_t **esp_eth_phy_802_3_set_mediator**(*phy_802_3_t* *phy_802_3, *esp_eth_mediator_t* *eth)

Set Ethernet mediator.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **eth** -- Ethernet mediator pointer

Returns

- ESP_OK: Ethermet mediator set successfully
- ESP_ERR_INVALID_ARG: if `eth` is `NULL`

esp_err_t **esp_eth_phy_802_3_reset**(*phy_802_3_t* *phy_802_3)

Reset PHY.

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure

Returns

- ESP_OK: Ethernet PHY reset successfully
- ESP_FAIL: reset Ethernet PHY failed because some error occurred

esp_err_t **esp_eth_phy_802_3_autonego_ctrl**(*phy_802_3_t* *phy_802_3, *eth_phy_autoneg_cmd_t* cmd, bool *autonego_en_stat)

Control autonegotiation mode of Ethernet PHY.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **cmd** -- autonegotiation command enumeration
- **autonego_en_stat** -- **[out]** autonegotiation enabled flag

Returns

- ESP_OK: Ethernet PHY autonegotiation configured successfully
- ESP_FAIL: Ethernet PHY autonegotiation configuration fail because some error occurred
- ESP_ERR_INVALID_ARG: invalid value of `cmd`

esp_err_t **esp_eth_phy_802_3_pwrctl**(*phy_802_3_t* *phy_802_3, bool enable)

Power control of Ethernet PHY.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **enable** -- set true to power ON Ethernet PHY; set false to power OFF Ethernet PHY

Returns

- ESP_OK: Ethernet PHY power down mode set successfully
- ESP_FAIL: Ethernet PHY power up or power down failed because some error occurred

esp_err_t **esp_eth_phy_802_3_set_addr**(*phy_802_3_t* *phy_802_3, uint32_t addr)

Set Ethernet PHY address.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **addr** -- new PHY address

Returns

- ESP_OK: Ethernet PHY address set

esp_err_t **esp_eth_phy_802_3_get_addr**(*phy_802_3_t* *phy_802_3, uint32_t *addr)

Get Ethernet PHY address.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **addr** -- **[out]** Ethernet PHY address

Returns

- ESP_OK: Ethernet PHY address read successfully
- ESP_ERR_INVALID_ARG: `addr` pointer is `NULL`

esp_err_t **esp_eth_phy_802_3_advertise_pause_ability**(*phy_802_3_t* *phy_802_3, uint32_t ability)

Advertise pause function ability.

Parameters

- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **ability** -- enable or disable pause ability

Returns

- ESP_OK: pause ability set successfully
- ESP_FAIL: Advertise pause function ability failed because some error occurred

esp_err_t **esp_eth_phy_802_3_loopback**(*phy_802_3_t* *phy_802_3, bool enable)

Set Ethernet PHY loopback mode.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **enable** -- set true to enable loopback; set false to disable loopback

Returns
- ESP_OK: Ethernet PHY loopback mode set successfully
- ESP_FAIL: Ethernet PHY loopback configuration failed because some error occurred

esp_err_t **esp_eth_phy_802_3_set_speed**(*phy_802_3_t* *phy_802_3, *eth_speed_t* speed)

Set Ethernet PHY speed.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **speed** -- new speed of Ethernet PHY link

Returns
- ESP_OK: Ethernet PHY speed set successfully
- ESP_FAIL: Set Ethernet PHY speed failed because some error occurred

esp_err_t **esp_eth_phy_802_3_set_duplex**(*phy_802_3_t* *phy_802_3, *eth_duplex_t* duplex)

Set Ethernet PHY duplex mode.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **duplex** -- new duplex mode for Ethernet PHY link

Returns
- ESP_OK: Ethernet PHY duplex mode set successfully
- ESP_ERR_INVALID_STATE: unable to set duplex mode to Half if loopback is enabled
- ESP_FAIL: Set Ethernet PHY duplex mode failed because some error occurred

esp_err_t **esp_eth_phy_802_3_set_link**(*phy_802_3_t* *phy_802_3, *eth_link_t* link)

Set Ethernet PHY link status.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **link** -- new link status

Returns
- ESP_OK: Ethernet PHY link set successfully

esp_err_t **esp_eth_phy_802_3_init**(*phy_802_3_t* *phy_802_3)

Initialize Ethernet PHY.

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure
Returns
- ESP_OK: Ethernet PHY initialized successfully

esp_err_t **esp_eth_phy_802_3_deinit**(*phy_802_3_t* *phy_802_3)

Power off Eternet PHY.

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure
Returns
- ESP_OK: Ethernet PHY powered off successfully

esp_err_t **esp_eth_phy_802_3_del**(*phy_802_3_t* *phy_802_3)

Delete Ethernet PHY infostructure.

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure
Returns
- ESP_OK: Ethrnet PHY infostructure deleted

esp_err_t **esp_eth_phy_802_3_reset_hw**(*phy_802_3_t* *phy_802_3, uint32_t reset_assert_us)

Performs hardware reset with specific reset pin assertion time.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **reset_assert_us** -- Hardware reset pin assertion time

Returns
- ESP_OK: reset Ethernet PHY successfully

esp_err_t **esp_eth_phy_802_3_detect_phy_addr**(*esp_eth_mediator_t* *eth, int *detected_addr)

Detect PHY address.

Parameters
- **eth** -- Mediator of Ethernet driver
- **detected_addr** -- **[out]** a valid address after detection

Returns
- ESP_OK: detect phy address successfully
- ESP_ERR_INVALID_ARG: invalid parameter
- ESP_ERR_NOT_FOUND: can't detect any PHY device
- ESP_FAIL: detect phy address failed because some error occurred

esp_err_t **esp_eth_phy_802_3_basic_phy_init**(*phy_802_3_t* *phy_802_3)

Performs basic PHY chip initialization.

Note: It should be called as the first function in PHY specific driver instance

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure

Returns
- ESP_OK: initialized Ethernet PHY successfully
- ESP_FAIL: initialization of Ethernet PHY failed because some error occurred
- ESP_ERR_INVALID_ARG: invalid argument
- ESP_ERR_NOT_FOUND: PHY device not detected
- ESP_ERR_TIMEOUT: MII Management read/write operation timeout
- ESP_ERR_INVALID_STATE: PHY is in invalid state to perform requested operation

esp_err_t **esp_eth_phy_802_3_basic_phy_deinit**(*phy_802_3_t* *phy_802_3)

Performs basic PHY chip de-initialization.

Note: It should be called as the last function in PHY specific driver instance

Parameters **phy_802_3** -- IEEE 802.3 PHY object infostructure

Returns
- ESP_OK: de-initialized Ethernet PHY successfully
- ESP_FAIL: de-initialization of Ethernet PHY failed because some error occurred
- ESP_ERR_TIMEOUT: MII Management read/write operation timeout
- ESP_ERR_INVALID_STATE: PHY is in invalid state to perform requested operation

esp_err_t **esp_eth_phy_802_3_read_oui**(*phy_802_3_t* *phy_802_3, uint32_t *oui)

Reads raw content of OUI field.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **oui** -- **[out]** OUI value

Returns
- ESP_OK: OUI field read successfully
- ESP_FAIL: OUI field read failed because some error occurred
- ESP_ERR_INVALID_ARG: invalid `oui` argument
- ESP_ERR_TIMEOUT: MII Management read/write operation timeout
- ESP_ERR_INVALID_STATE: PHY is in invalid state to perform requested operation

esp_err_t **esp_eth_phy_802_3_read_manufac_info**(*phy_802_3_t* *phy_802_3, uint8_t *model, uint8_t *rev)

Reads manufacturer's model and revision number.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **model** -- **[out]** Manufacturer's model number (can be NULL when not required)
- **rev** -- **[out]** Manufacturer's revision number (can be NULL when not required)

Returns
- ESP_OK: Manufacturer's info read successfully
- ESP_FAIL: Manufacturer's info read failed because some error occurred
- ESP_ERR_TIMEOUT: MII Management read/write operation timeout
- ESP_ERR_INVALID_STATE: PHY is in invalid state to perform requested operation

esp_err_t **esp_eth_phy_802_3_get_mmd_addr**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, uint16_t *mmd_addr)

Reads MDIO device's internal address register.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **mmd_addr** -- **[out]** Current address stored in device's register

Returns
- ESP_OK: Address register read successfully
- ESP_FAIL: Address register read failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits)

esp_err_t **esp_eth_phy_802_3_set_mmd_addr**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, uint16_t mmd_addr)

Write to DIO device's internal address register.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **mmd_addr** -- **[out]** New value of MDIO device's address register value

Returns
- ESP_OK: Address register written to successfully
- ESP_FAIL: Address register write failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits)

esp_err_t **esp_eth_phy_802_3_read_mmd_data**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, *esp_eth_phy_802_3_mmd_func_t* function, uint32_t *data)

Read data of MDIO device's memory at address register.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **function** -- MMD function
- **data** -- **[out]** Data read from the device's memory

Returns
- ESP_OK: Memory read successfully
- ESP_FAIL: Memory read failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits) or MMD access function is invalid

esp_err_t **esp_eth_phy_802_3_write_mmd_data**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, *esp_eth_phy_802_3_mmd_func_t* function, uint32_t data)

Write data to MDIO device's memory at address register.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **function** -- MMD function
- **data** -- **[out]** Data to write to the device's memory

Returns
- ESP_OK: Memory written successfully
- ESP_FAIL: Memory write failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits) or MMD access function is invalid

esp_err_t **esp_eth_phy_802_3_read_mmd_register**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, uint16_t mmd_addr, uint32_t *data)

Set MMD address to mmd_addr with function MMD_FUNC_NOINCR and read contents to *data.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **mmd_addr** -- Address of MDIO device register
- **data** -- **[out]** Data read from the device's memory

Returns
- ESP_OK: Memory read successfully
- ESP_FAIL: Memory read failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits)

esp_err_t **esp_eth_phy_802_3_write_mmd_register**(*phy_802_3_t* *phy_802_3, uint8_t devaddr, uint16_t mmd_addr, uint32_t data)

Set MMD address to mmd_addr with function MMD_FUNC_NOINCR and write data.

Parameters
- **phy_802_3** -- IEEE 802.3 PHY object infostructure
- **devaddr** -- Address of MDIO device
- **mmd_addr** -- Address of MDIO device register
- **data** -- **[out]** Data to write to the device's memory

Returns
- ESP_OK: Memory written to successfully
- ESP_FAIL: Memory write failed because of some error occurred
- ESP_ERR_INVALID_ARG: Device address provided is out of range (hardware limits device address to 5 bits)

inline *phy_802_3_t* ***esp_eth_phy_into_phy_802_3**(*esp_eth_phy_t* *phy)

Returns address to parent IEEE 802.3 PHY object infostructure.

Parameters **phy** -- Ethernet PHY instance

Returns phy_802_3_t*
- address to parent IEEE 802.3 PHY object infostructure

esp_err_t **esp_eth_phy_802_3_obj_config_init**(*phy_802_3_t* *phy_802_3, const *eth_phy_config_t* *config)

Initializes configuration of parent IEEE 802.3 PHY object infostructure.

Parameters
- **phy_802_3** -- Address to IEEE 802.3 PHY object infostructure
- **config** -- Configuration of the IEEE 802.3 PHY object

Returns
- ESP_OK: configuration initialized successfully
- ESP_ERR_INVALID_ARG: invalid `config` argument

Structures

struct **phy_802_3_t**

IEEE 802.3 PHY object infostructure.

Public Members

esp_eth_phy_t **parent**

Parent Ethernet PHY instance

esp_eth_mediator_t ***eth**

Mediator of Ethernet driver

int **addr**

PHY address

uint32_t **reset_timeout_ms**

Reset timeout value (Unit: ms)

uint32_t **autonego_timeout_ms**

Auto-negotiation timeout value (Unit: ms)

eth_link_t **link_status**

Current Link status

int **reset_gpio_num**

Reset GPIO number, -1 means no hardware reset

Enumerations

enum **esp_eth_phy_802_3_mmd_func_t**

IEEE 802.3 MMD modes enumeration.

Values:

enumerator **MMD_FUNC_ADDRESS**

enumerator **MMD_FUNC_DATA_NOINCR**

enumerator **MMD_FUNC_DATA_RWINCR**

enumerator **MMD_FUNC_DATA_WINCR**

Header File

- components/esp_eth/include/esp_eth_netif_glue.h
- This header file can be included with:

```
#include "esp_eth_netif_glue.h"
```

- This header file is a part of the API provided by the `esp_eth` component. To declare that your component depends on `esp_eth`, add the following to your CMakeLists.txt:

```
REQUIRES esp_eth
```

or

```
PRIV_REQUIRES esp_eth
```

Functions

esp_eth_netif_glue_handle_t **esp_eth_new_netif_glue**(*esp_eth_handle_t* eth_hdl)

Create a netif glue for Ethernet driver.

Note: netif glue is used to attach io driver to TCP/IP netif

Parameters **eth_hdl** -- Ethernet driver handle
Returns glue object, which inherits esp_netif_driver_base_t

esp_err_t **esp_eth_del_netif_glue**(*esp_eth_netif_glue_handle_t* eth_netif_glue)

Delete netif glue of Ethernet driver.

Parameters **eth_netif_glue** -- netif glue
Returns -ESP_OK: delete netif glue successfully

Type Definitions

typedef struct esp_eth_netif_glue_t ***esp_eth_netif_glue_handle_t**

Handle of netif glue - an intermediate layer between netif and Ethernet driver.

Code examples for the Ethernet API are provided in the ethernet directory of ESP-IDF examples.

2.5.3 Thread

Thread

Introduction Thread is an IP-based mesh networking protocol. It is based on the 802.15.4 physical and MAC layer.

Application Examples The openthread directory of ESP-IDF examples contains the following applications:

- The OpenThread interactive shell openthread/ot_cli
- The Thread Border Router openthread/ot_br
- The Thread Radio Co-Processor openthread/ot_rcp

API Reference For manipulating the Thread network, the OpenThread API shall be used. The OpenThread API docs can be found at the OpenThread API docs.

ESP-IDF provides extra APIs for launching and managing the OpenThread stack, binding to network interfaces and border routing features.

Header File

- components/openthread/include/esp_openthread.h
- This header file can be included with:

```
#include "esp_openthread.h"
```

- This header file is a part of the API provided by the `openthread` component. To declare that your component depends on `openthread`, add the following to your CMakeLists.txt:

```
REQUIRES openthread
```

or

```
PRIV_REQUIRES openthread
```

Functions

esp_err_t **esp_openthread_init**(const *esp_openthread_platform_config_t* *init_config)

Initializes the full OpenThread stack.

Note: The OpenThread instance will also be initialized in this function.

Parameters **init_config** -- **[in]** The initialization configuration.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if allocation has failed
- ESP_ERR_INVALID_ARG if radio or host connection mode not supported
- ESP_ERR_INVALID_STATE if already initialized

esp_err_t **esp_openthread_auto_start**(otOperationalDatasetTlvs *datasetTlvs)

Starts the Thread protocol operation and attaches to a Thread network.

Parameters **datasetTlvs** -- **[in]** The operational dataset (TLV encoded), if it's NULL, the function will generate the dataset based on the configurations from kconfig.

Returns

- ESP_OK on success
- ESP_FAIL on failures

esp_err_t **esp_openthread_launch_mainloop**(void)

Launches the OpenThread main loop.

Note: This function will not return unless error happens when running the OpenThread stack.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if allocation has failed
- ESP_FAIL on other failures

esp_err_t **esp_openthread_deinit**(void)

This function performs OpenThread stack and platform driver deinitialization.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if not initialized

otInstance *`esp_openthread_get_instance`(void)

This function acquires the underlying OpenThread instance.

Note: This function can be called on other tasks without lock.

Returns The OpenThread instance pointer

Header File

- components/openthread/include/esp_openthread_types.h
- This header file can be included with:

```
#include "esp_openthread_types.h"
```

- This header file is a part of the API provided by the `openthread` component. To declare that your component depends on `openthread`, add the following to your CMakeLists.txt:

```
REQUIRES openthread
```

or

```
PRIV_REQUIRES openthread
```

Structures

struct **`esp_openthread_role_changed_event_t`**

OpenThread role changed event data.

Public Members

otDeviceRole **`previous_role`**

Previous Thread role

otDeviceRole **`current_role`**

Current Thread role

struct **`esp_openthread_mainloop_context_t`**

This structure represents a context for a select() based mainloop.

Public Members

fd_set **`read_fds`**

The read file descriptors

fd_set **`write_fds`**

The write file descriptors

fd_set **`error_fds`**

The error file descriptors

int **max_fd**

The max file descriptor

struct timeval **timeout**

The timeout

struct **esp_openthread_uart_config_t**

The uart port config for OpenThread.

Public Members

uart_port_t **port**

UART port number

uart_config_t **uart_config**

UART configuration, see *uart_config_t* docs

gpio_num_t **rx_pin**

UART RX pin

gpio_num_t **tx_pin**

UART TX pin

struct **esp_openthread_spi_host_config_t**

The spi port config for OpenThread.

Public Members

spi_host_device_t **host_device**

SPI host device

spi_dma_chan_t **dma_channel**

DMA channel

spi_bus_config_t **spi_interface**

SPI bus

spi_device_interface_config_t **spi_device**

SPI peripheral device

gpio_num_t **intr_pin**

SPI interrupt pin

struct **esp_openthread_spi_slave_config_t**

The spi slave config for OpenThread.

Public Members

spi_host_device_t **host_device**

SPI host device

spi_bus_config_t **bus_config**

SPI bus config

spi_slave_interface_config_t **slave_config**

SPI slave config

gpio_num_t **intr_pin**

SPI interrupt pin

struct **esp_openthread_radio_config_t**

The OpenThread radio configuration.

Public Members

esp_openthread_radio_mode_t **radio_mode**

The radio mode

esp_openthread_uart_config_t **radio_uart_config**

The uart configuration to RCP

esp_openthread_spi_host_config_t **radio_spi_config**

The spi configuration to RCP

struct **esp_openthread_host_connection_config_t**

The OpenThread host connection configuration.

Public Members

esp_openthread_host_connection_mode_t **host_connection_mode**

The host connection mode

esp_openthread_uart_config_t **host_uart_config**

The uart configuration to host

usb_serial_jtag_driver_config_t **host_usb_config**

The usb configuration to host

esp_openthread_spi_slave_config_t **spi_slave_config**

The spi configuration to host

struct **esp_openthread_port_config_t**

The OpenThread port specific configuration.

Public Members

const char *`storage_partition_name`

The partition for storing OpenThread dataset

uint8_t `netif_queue_size`

The packet queue size for the network interface

uint8_t `task_queue_size`

The task queue size

struct `esp_openthread_platform_config_t`

The OpenThread platform configuration.

Public Members

esp_openthread_radio_config_t `radio_config`

The radio configuration

esp_openthread_host_connection_config_t `host_config`

The host connection configuration

esp_openthread_port_config_t `port_config`

The port configuration

Type Definitions

typedef void (*`esp_openthread_rcp_failure_handler`)(void)

Enumerations

enum `esp_openthread_event_t`

OpenThread event declarations.

Values:

enumerator `OPENTHREAD_EVENT_START`

OpenThread stack start

enumerator `OPENTHREAD_EVENT_STOP`

OpenThread stack stop

enumerator `OPENTHREAD_EVENT_DETACHED`

OpenThread detached

enumerator `OPENTHREAD_EVENT_ATTACHED`

OpenThread attached

enumerator **OPENTHREAD_EVENT_ROLE_CHANGED**

OpenThread role changed

enumerator **OPENTHREAD_EVENT_IF_UP**

OpenThread network interface up

enumerator **OPENTHREAD_EVENT_IF_DOWN**

OpenThread network interface down

enumerator **OPENTHREAD_EVENT_GOT_IP6**

OpenThread stack added IPv6 address

enumerator **OPENTHREAD_EVENT_LOST_IP6**

OpenThread stack removed IPv6 address

enumerator **OPENTHREAD_EVENT_MULTICAST_GROUP_JOIN**

OpenThread stack joined IPv6 multicast group

enumerator **OPENTHREAD_EVENT_MULTICAST_GROUP_LEAVE**

OpenThread stack left IPv6 multicast group

enumerator **OPENTHREAD_EVENT_TREL_ADD_IP6**

OpenThread stack added TREL IPv6 address

enumerator **OPENTHREAD_EVENT_TREL_REMOVE_IP6**

OpenThread stack removed TREL IPv6 address

enumerator **OPENTHREAD_EVENT_TREL_MULTICAST_GROUP_JOIN**

OpenThread stack joined TREL IPv6 multicast group

enumerator **OPENTHREAD_EVENT_SET_DNS_SERVER**

OpenThread stack set DNS server >

enumerator **OPENTHREAD_EVENT_PUBLISH_MESHCOP_E**

OpenThread stack start to publish meshcop-e service >

enumerator **OPENTHREAD_EVENT_REMOVE_MESHCOP_E**

OpenThread stack start to remove meshcop-e service >

enum **esp_openthread_radio_mode_t**

The radio mode of OpenThread.

Values:

enumerator **RADIO_MODE_NATIVE**

Use the native 15.4 radio

enumerator **RADIO_MODE_UART_RCP**

UART connection to a 15.4 capable radio co-processor (RCP)

enumerator **RADIO_MODE_SPI_RCP**

SPI connection to a 15.4 capable radio co-processor (RCP)

enumerator **RADIO_MODE_MAX**

Using for parameter check

enum **esp_openthread_host_connection_mode_t**

How OpenThread connects to the host.

Values:

enumerator **HOST_CONNECTION_MODE_NONE**

Disable host connection

enumerator **HOST_CONNECTION_MODE_CLI_UART**

CLI UART connection to the host

enumerator **HOST_CONNECTION_MODE_CLI_USB**

CLI USB connection to the host

enumerator **HOST_CONNECTION_MODE_RCP_UART**

RCP UART connection to the host

enumerator **HOST_CONNECTION_MODE_RCP_SPI**

RCP SPI connection to the host

enumerator **HOST_CONNECTION_MODE_MAX**

Using for parameter check

Header File

- components/openthread/include/esp_openthread_lock.h
- This header file can be included with:

```
#include "esp_openthread_lock.h"
```

- This header file is a part of the API provided by the `openthread` component. To declare that your component depends on `openthread`, add the following to your CMakeLists.txt:

```
REQUIRES openthread
```

or

```
PRIV_REQUIRES openthread
```

Functions

esp_err_t **esp_openthread_lock_init**(void)

This function initializes the OpenThread API lock.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if allocation has failed
- ESP_ERR_INVALID_STATE if already initialized

void **esp_openthread_lock_deinit**(void)

This function deinitializes the OpenThread API lock.

bool **esp_openthread_lock_acquire**(TickType_t block_ticks)

This function acquires the OpenThread API lock.

Note: Every Openthread APIs that takes an otInstance argument MUST be protected with this API lock except that the call site is in Openthread callbacks.

Parameters **block_ticks** -- **[in]** The maximum number of RTOS ticks to wait for the lock.
Returns

- True on lock acquired
- False on failing to acquire the lock with the timeout.

void **esp_openthread_lock_release**(void)

This function releases the OpenThread API lock.

bool **esp_openthread_task_switching_lock_acquire**(TickType_t block_ticks)

This function acquires the OpenThread API task switching lock.

Note: In OpenThread API context, it waits for some actions to be done in other tasks (like lwip), after task switching, it needs to call OpenThread API again. Normally it's not allowed, since the previous OpenThread API lock is not released yet. This task_switching lock allows the OpenThread API can be called in this case.

Note: Please use esp_openthread_lock_acquire() for normal cases.

Parameters **block_ticks** -- **[in]** The maximum number of RTOS ticks to wait for the lock.
Returns

- True on lock acquired
- False on failing to acquire the lock with the timeout.

void **esp_openthread_task_switching_lock_release**(void)

This function releases the OpenThread API task switching lock.

Note: This API must be called after `esp_openthread_task_switching_lock_acquire` or `esp_openthread_lock_acquire` and will cause a crash if the current task is not the task switching lock holder. This error could be caused by calling OpenThread APIs without locking OpenThread stack.

Header File

- components/openthread/include/esp_openthread_netif_glue.h
- This header file can be included with:

```
#include "esp_openthread_netif_glue.h"
```

- This header file is a part of the API provided by the `openthread` component. To declare that your component depends on `openthread`, add the following to your CMakeLists.txt:

```
REQUIRES openthread
```

or

```
PRIV_REQUIRES openthread
```

Functions

void ***esp_openthread_netif_glue_init**(const *esp_openthread_platform_config_t* *config)

This function initializes the OpenThread network interface glue.

Parameters **config** -- **[in]** The platform configuration.

Returns

- glue pointer on success
- NULL on failure

void **esp_openthread_netif_glue_deinit**(void)

This function deinitializes the OpenThread network interface glue.

esp_netif_t ***esp_openthread_get_netif**(void)

This function acquires the OpenThread netif.

Returns The OpenThread netif or NULL if not initialzied.

void **esp_openthread_register_meshcop_e_handler**(*esp_event_handler_t* handler, bool for_publish)

This function register a handler for meshcop-e service publish event and remove event.

Parameters

- **handler** -- **[in]** The handler.
- **for_publish** -- **[in]** The usage of handler, true for publish event and false for remove event.

Macros

ESP_NETIF_INHERENT_DEFAULT_OPENTHREAD()

Default configuration reference of OpenThread esp-netif.

ESP_NETIF_DEFAULT_OPENTHREAD()

Header File

- components/openthread/include/esp_openthread_border_router.h
- This header file can be included with:

```
#include "esp_openthread_border_router.h"
```

- This header file is a part of the API provided by the `openthread` component. To declare that your component depends on `openthread`, add the following to your CMakeLists.txt:

```
REQUIRES openthread
```

or

```
PRIV_REQUIRES openthread
```

Functions

void **esp_openthread_set_backbone_netif**(*esp_netif_t* *backbone_netif)

Sets the backbone interface used for border routing.

Note: This function must be called before esp_openthread_init

Parameters **backbone_netif** -- **[in]** The backbone network interface (WiFi or ethernet)

esp_err_t **esp_openthread_border_router_init**(void)

Initializes the border router features of OpenThread.

Note: Calling this function will make the device behave as an OpenThread border router. Kconfig option CONFIG_OPENTHREAD_BORDER_ROUTER is required.

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if feature not supported
- ESP_ERR_INVALID_STATE if already initialized
- ESP_FIAL on other failures

esp_err_t **esp_openthread_border_router_deinit**(void)

Deinitializes the border router features of OpenThread.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if not initialized
- ESP_FIAL on other failures

esp_netif_t ***esp_openthread_get_backbone_netif**(void)

Gets the backbone interface of OpenThread border router.

Returns The backbone interface or NULL if border router not initialized.

void **esp_openthread_register_rcp_failure_handler**(*esp_openthread_rcp_failure_handler* handler)

Registers the callback for RCP failure.

esp_err_t **esp_openthread_rcp_deinit**(void)

Deinitializes the connection to RCP.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if fail to deinitialize RCP

esp_err_t **esp_openthread_rcp_init**(void)

Initializes the connection to RCP.

Returns

- ESP_OK on success
- ESP_FAIL if fail to initialize RCP

esp_err_t **esp_openthread_set_meshcop_instance_name**(const char *instance_name)

Sets the meshcop(e) instance name.

Note: This function can only be called before `esp_openthread_border_router_init`. If `instance_name` is NULL, then the service will use the hostname as instance name.

Parameters **instance_name** -- **[in]** The instance name, can be `NULL`.

Returns

- ESP_OK on success
- ESP_FAIL if fail to initialize RCP

Thread is an IPv6-based mesh networking technology for IoT.

Code examples for the Thread API are provided in the openthread directory of ESP-IDF examples.

2.5.4 ESP-NETIF

ESP-NETIF

The purpose of the ESP-NETIF library is twofold:

- It provides an abstraction layer for the application on top of the TCP/IP stack. This allows applications to choose between IP stacks in the future.
- The APIs it provides are thread-safe, even if the underlying TCP/IP stack APIs are not.

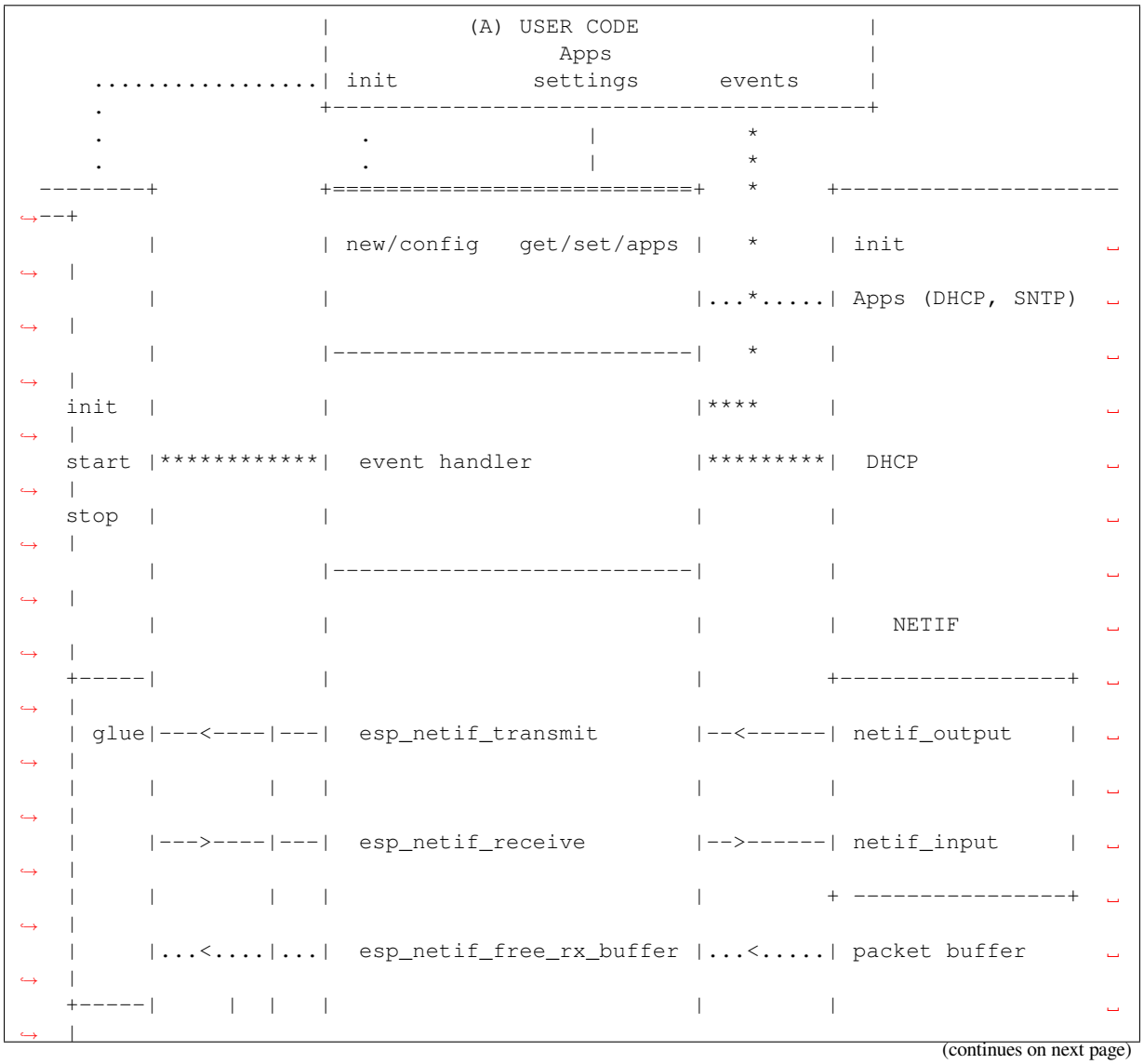
ESP-IDF currently implements ESP-NETIF for the lwIP TCP/IP stack only. However, the adapter itself is TCP/IP implementation-agnostic and allows different implementations.

It is also possible to use a custom TCP/IP stack with ESP-IDF, provided it implements BSD API. For more information on building ESP-IDF without lwIP, please refer to components/esp_netif_stack/README.md.

Some ESP-NETIF API functions are intended to be called by application code, for example, to get or set interface IP addresses, and configure DHCP. Other functions are intended for internal ESP-IDF use by the network driver layer.

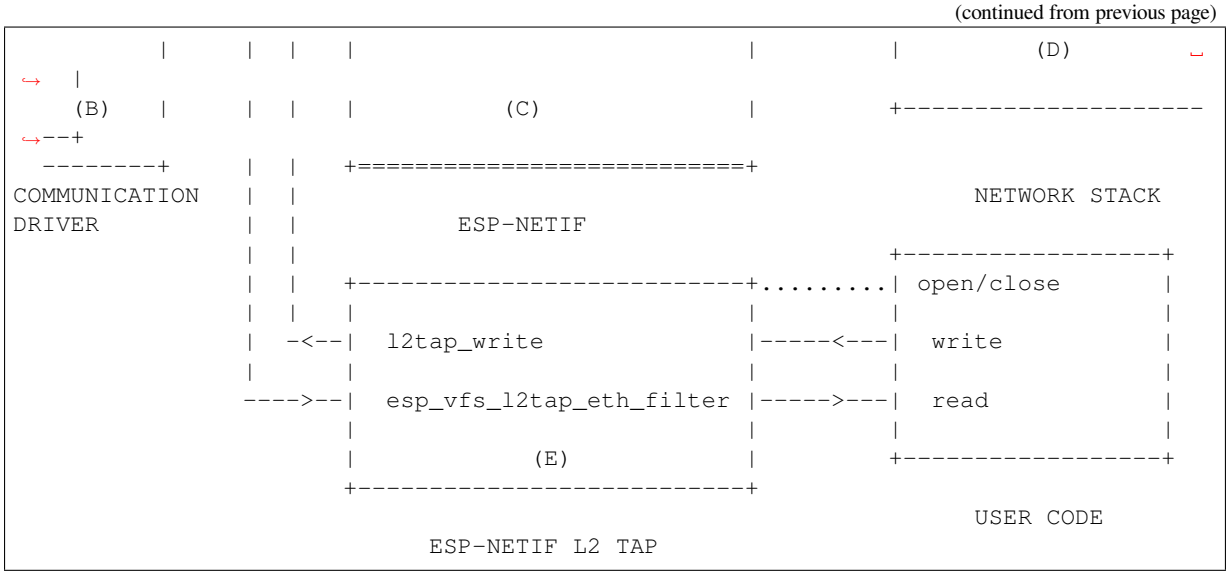
In many cases, applications do not need to call ESP-NETIF APIs directly as they are called by the default network event handlers.

ESP-NETIF Architecture



(continues on next page)

(continued from previous page)



Data and Event Flow in the Diagram

- `. . . . . .` Initialization line from user code to ESP-NETIF and communication driver
- `--<---->--` Data packets going from communication media to TCP/IP stack and back
- `********` Events aggregated in ESP-NETIF propagate to the driver, user code, and network stack
- `|` User settings and runtime configuration

ESP-NETIF Interaction

A) User Code, Boilerplate Overall application interaction with a specific IO driver for communication media and configured TCP/IP network stack is abstracted using ESP-NETIF APIs and is outlined as below:

A) Initialization code

1) Initializes IO driver
2) Creates a new instance of ESP-NETIF and configure it with
   - ESP-NETIF specific options (flags, behavior, name)
   - Network stack options (netif init and input functions, not publicly available)
   - IO driver specific options (transmit, free rx buffer functions, IO driver handle)
3) Attaches the IO driver handle to the ESP-NETIF instance created in the above steps
4) Configures event handlers
   - Use default handlers for common interfaces defined in IO drivers; or define a specific handler for customized behavior or new interfaces
   - Register handlers for app-related events (such as IP lost or acquired)

B) Interaction with network interfaces using ESP-NETIF API

1) Gets and sets TCP/IP-related parameters (DHCP, IP, etc)
2) Receives IP events (connect or disconnect)
3) Controls application lifecycle (set interface up or down)

B) Communication Driver, IO Driver, and Media Driver Communication driver plays these two important roles in relation to ESP-NETIF:

1) Event handlers: Defines behavior patterns of interaction with ESP-NETIF (e.g., ethernet link-up -> turn netif on)
2) Glue IO layer: Adapts the input or output functions to use ESP-NETIF transmit, receive, and free receive buffer

- Installs driver_transmit to the appropriate ESP-NETIF object so that outgoing packets from the network stack are passed to the IO driver
- Calls `esp_netif_receive()` to pass incoming data to the network stack

C) ESP-NETIF ESP-NETIF serves as an intermediary between an IO driver and a network stack, connecting the packet data path between the two. It provides a set of interfaces for attaching a driver to an ESP-NETIF object at runtime and configures a network stack during compiling. Additionally, a set of APIs is provided to control the network interface lifecycle and its TCP/IP properties. As an overview, the ESP-NETIF public interface can be divided into six groups:

1) Initialization APIs (to create and configure ESP-NETIF instance)
2) Input or Output API (for passing data between IO driver and network stack)
3) Event or Action API

- Used for network interface lifecycle management
- ESP-NETIF provides building blocks for designing event handlers

4) Setters and Getters API for basic network interface properties
5) Network stack abstraction API: enabling user interaction with TCP/IP stack

- Set interface up or down
- DHCP server and client API
- DNS API
- *SNTP API*

6) Driver conversion utilities API

D) Network Stack The network stack has no public interaction with application code with regard to public interfaces and shall be fully abstracted by ESP-NETIF API.

E) ESP-NETIF L2 TAP Interface The ESP-NETIF L2 TAP interface is a mechanism in ESP-IDF used to access Data Link Layer (L2 per OSI/ISO) for frame reception and transmission from the user application. Its typical usage in the embedded world might be the implementation of non-IP-related protocols, e.g., PTP, Wake on LAN. Note that only Ethernet (IEEE 802.3) is currently supported.

From a user perspective, the ESP-NETIF L2 TAP interface is accessed using file descriptors of VFS, which provides file-like interfacing (using functions like `open()`, `read()`, `write()`, etc). To learn more, refer to *Virtual Filesystem Component*.

There is only one ESP-NETIF L2 TAP interface device (path name) available. However multiple file descriptors with different configurations can be opened at a time since the ESP-NETIF L2 TAP interface can be understood as a generic entry point to the Layer 2 infrastructure. What is important is then the specific configuration of the particular file descriptor. It can be configured to give access to a specific Network Interface identified by `if_key` (e.g., *ETH_DEF*) and to filter only specific frames based on their type (e.g., Ethernet type in the case of IEEE 802.3). Filtering only specific frames is crucial since the ESP-NETIF L2 TAP needs to exist along with the IP stack and so the IP-related traffic (IP, ARP, etc.) should not be passed directly to the user application. Even though this option is still configurable, it is not recommended in standard use cases. Filtering is also advantageous from the perspective of the user's application, as it only gets access to the frame types it is interested in, and the remaining traffic is either passed to other L2 TAP file descriptors or to the IP stack.

ESP-NETIF L2 TAP Interface Usage Manual

Initialization To be able to use the ESP-NETIF L2 TAP interface, it needs to be enabled in Kconfig by *CONFIG_ESP_NETIF_L2_TAP* first and then registered by `esp_vfs_l2tap_intf_register()` prior usage of any VFS function.

open() Once the ESP-NETIF L2 TAP is registered, it can be opened at path name "/dev/net/tap". The same path name can be opened multiple times up to *CONFIG_ESP_NETIF_L2_TAP_MAX_FDS* and multiple file descriptors with a different configuration may access the Data Link Layer frames.

The ESP-NETIF L2 TAP can be opened with the `O_NONBLOCK` file status flag to make sure the `read()` does not block. Note that the `write()` may block in the current implementation when accessing a Network interface since it is a shared resource among multiple ESP-NETIF L2 TAP file descriptors and IP stack, and there is currently no queuing mechanism deployed. The file status flag can be retrieved and modified using `fcntl()`.

On success, `open()` returns the new file descriptor (a nonnegative integer). On error, -1 is returned, and `errno` is set to indicate the error.

ioctl() The newly opened ESP-NETIF L2 TAP file descriptor needs to be configured prior to its usage since it is not bounded to any specific Network Interface and no frame type filter is configured. The following configuration options are available to do so:

- `L2TAP_S_INTF_DEVICE` - bounds the file descriptor to a specific Network Interface that is identified by its `if_key`. ESP-NETIF Network Interface `if_key` is passed to `ioctl()` as the third parameter. Note that default Network Interfaces `if_key`'s used in ESP-IDF can be found in esp_netif/include/esp_netif_defaults.h.
- `L2TAP_S_DEVICE_DRV_HNDL` - is another way to bound the file descriptor to a specific Network Interface. In this case, the Network interface is identified directly by IO Driver handle (e.g., *esp_eth_handle_t* in case of Ethernet). The IO Driver handle is passed to `ioctl()` as the third parameter.
- `L2TAP_S_RCV_FILTER` - sets the filter to frames with the type to be passed to the file descriptor. In the case of Ethernet frames, the frames are to be filtered based on the Length and Ethernet type field. In case the filter value is set less than or equal to 0x05DC, the Ethernet type field is considered to represent IEEE802.3 Length Field, and all frames with values in interval <0, 0x05DC> at that field are passed to the file descriptor. The IEEE802.2 logical link control (LLC) resolution is then expected to be performed by the user's application. In case the filter value is set greater than 0x05DC, the Ethernet type field is considered to represent protocol identification and only frames that are equal to the set value are to be passed to the file descriptor.

All above-set configuration options have a getter counterpart option to read the current settings.

Warning: The file descriptor needs to be firstly bounded to a specific Network Interface by `L2TAP_S_INTF_DEVICE` or `L2TAP_S_DEVICE_DRV_HNDL` to make `L2TAP_S_RCV_FILTER` option available.

Note: VLAN-tagged frames are currently not recognized. If the user needs to process VLAN-tagged frames, they need a set filter to be equal to the VLAN tag (i.e., 0x8100 or 0x88A8) and process the VLAN-tagged frames in the user application.

Note: `L2TAP_S_DEVICE_DRV_HNDL` is particularly useful when the user's application does not require the usage of an IP stack and so ESP-NETIF is not required to be initialized too. As a result, Network Interface cannot be identified by its `if_key` and hence it needs to be identified directly by its IO Driver handle.

On success, `ioctl()` returns 0. On error, -1 is returned, and `errno` is set to indicate the error.

* EBADF - not a valid file descriptor.

* EACCES - options change is denied in this state (e.g., file descriptor has not been bounded to Network interface yet).

* EINVAL - invalid configuration argument. Ethernet type filter is already used by other file descriptors on that same Network interface.

* ENODEV - no such Network Interface which is tried to be assigned to the file descriptor exists.

* ENOSYS - unsupported operation, passed configuration option does not exist.

fcntl() `fcntl()` is used to manipulate with properties of opened ESP-NETIF L2 TAP file descriptor.

The following commands manipulate the status flags associated with the file descriptor:

- `F_GETFD` - the function returns the file descriptor flags, and the third argument is ignored.
- `F_SETFD` - sets the file descriptor flags to the value specified by the third argument. Zero is returned.

On success, `ioctl()` returns 0. On error, -1 is returned, and `errno` is set to indicate the error.
* EBADF - not a valid file descriptor.
* ENOSYS - unsupported command.

read() Opened and configured ESP-NETIF L2 TAP file descriptor can be accessed by `read()` to get inbound frames. The read operation can be either blocking or non-blocking based on the actual state of the `O_NONBLOCK` file status flag. When the file status flag is set to blocking, the read operation waits until a frame is received and the context is switched to other tasks. When the file status flag is set to non-blocking, the read operation returns immediately. In such case, either a frame is returned if it was already queued or the function indicates the queue is empty. The number of queued frames associated with one file descriptor is limited by *CONFIG_ESP_NETIF_L2_TAP_RX_QUEUE_SIZE* Kconfig option. Once the number of queued frames reached a configured threshold, the newly arrived frames are dropped until the queue has enough room to accept incoming traffic (Tail Drop queue management).

On success, `read()` returns the number of bytes read. Zero is returned when the size of the destination buffer is 0. On error, -1 is returned, and `errno` is set to indicate the error.
* EBADF - not a valid file descriptor.
* EAGAIN - the file descriptor has been marked non-blocking (`O_NONBLOCK`), and the read would block.

write() A raw Data Link Layer frame can be sent to Network Interface via opened and configured ESP-NETIF L2 TAP file descriptor. The user's application is responsible to construct the whole frame except for fields which are added automatically by the physical interface device. The following fields need to be constructed by the user's application in case of an Ethernet link: source/destination MAC addresses, Ethernet type, actual protocol header, and user data. The length of these fields is as follows:

Destination MAC	Source MAC	Type/Length	Payload (protocol header/data)
6 B	6 B	2 B	0-1486 B

In other words, there is no additional frame processing performed by the ESP-NETIF L2 TAP interface. It only checks the Ethernet type of the frame is the same as the filter configured in the file descriptor. If the Ethernet type is different, an error is returned and the frame is not sent. Note that the `write()` may block in the current implementation when accessing a Network interface since it is a shared resource among multiple ESP-NETIF L2 TAP file descriptors and IP stack, and there is currently no queuing mechanism deployed.

On success, `write()` returns the number of bytes written. Zero is returned when the size of the input buffer is 0. On error, -1 is returned, and `errno` is set to indicate the error.
* EBADF - not a valid file descriptor.
* EBADMSG - The Ethernet type of the frame is different from the file descriptor configured filter.
* EIO - Network interface not available or busy.

close() Opened ESP-NETIF L2 TAP file descriptor can be closed by the `close()` to free its allocated resources. The ESP-NETIF L2 TAP implementation of `close()` may block. On the other hand, it is thread-safe and can be called from a different task than the file descriptor is actually used. If such a situation occurs and one task is blocked in the I/O operation and another task tries to close the file descriptor, the first task is unblocked. The first's task read operation then ends with an error.

On success, `close()` returns zero. On error, -1 is returned, and `errno` is set to indicate the error.
* EBADF - not a valid file descriptor.

`select()` Select is used in a standard way, just *CONFIG_VFS_SUPPORT_SELECT* needs to be enabled to make the `select()` function available.

SNTP API You can find a brief introduction to SNTP in general, its initialization code, and basic modes in Section *SNTP Time Synchronization* in *System Time*.

This section provides more details about specific use cases of the SNTP service, with statically configured servers, or use the DHCP-provided servers, or both. The workflow is usually very simple:

1) Initialize and configure the service using *`esp_netif_sntp_init()`*. This operations can only be called once (unless the SNTP service has been destroyed by *`esp_netif_sntp_deinit()`*)
2) Start the service via *`esp_netif_sntp_start()`*. This step is not needed if we auto-started the service in the previous step (default). It is useful to start the service explicitly after connecting if we want to use the DHCP-obtained NTP servers. Please note, this option needs to be enabled before connecting, but the SNTP service should be started after.
3) Wait for the system time to synchronize using *`esp_netif_sntp_sync_wait()`* (only if needed).
4) Stop and destroy the service using *`esp_netif_sntp_deinit()`*.

Basic Mode with Statically Defined Server(s) Initialize the module with the default configuration after connecting to the network. Note that it is possible to provide multiple NTP servers in the configuration struct:

```
esp_sntp_config_t config = ESP_NETIF_SNTP_DEFAULT_CONFIG_MULTIPLE(2,
                           ESP_SNTP_SERVER_LIST("time.windows.com", "pool.ntp.org"␣
↪) );
esp_netif_sntp_init(&config);
```

Note: If we want to configure multiple SNTP servers, we have to update the lwIP configuration *CONFIG_LWIP_SNTP_MAX_SERVERS*.

Use DHCP-Obtained SNTP Server(s) First of all, we have to enable the lwIP configuration option *CONFIG_LWIP_DHCP_GET_NTP_SRV*. Then we have to initialize the SNTP module with the DHCP option and without the NTP server:

```
esp_sntp_config_t config = ESP_NETIF_SNTP_DEFAULT_CONFIG_MULTIPLE(0, {} );
config.start = false;                       // start the SNTP service explicitly
config.server_from_dhcp = true;             // accept the NTP offer from the DHCP␣
↪server
esp_netif_sntp_init(&config);
```

Then, once we are connected, we could start the service using:

```
esp_netif_sntp_start();
```

Note: It is also possible to start the service during initialization (default `config.start=true`). This would likely to cause the initial SNTP request to fail (since we are not connected yet) and lead to some back-off time for subsequent requests.

Use Both Static and Dynamic Servers Very similar to the scenario above (DHCP provided SNTP server), but in this configuration, we need to make sure that the static server configuration is refreshed when obtaining NTP servers by DHCP. The underlying lwIP code cleans up the rest of the list of NTP servers when the DHCP-provided information

gets accepted. Thus the ESP-NETIF SNTP module saves the statically configured server(s) and reconfigures them after obtaining a DHCP lease.

The typical configuration now looks as per below, providing the specific `IP_EVENT` to update the config and index of the first server to reconfigure (for example setting `config.index_of_first_server=1` would keep the DHCP provided server at index 0, and the statically configured server at index 1).

```
esp_sntp_config_t config = ESP_NETIF_SNTP_DEFAULT_CONFIG("pool.ntp.org");
config.start = false;                        // start the SNTP service explicitly␣
↪(after connecting)
config.server_from_dhcp = true;              // accept the NTP offers from DHCP␣
↪server
config.renew_servers_after_new_IP = true;    // let esp-netif update the configured␣
↪SNTP server(s) after receiving the DHCP lease
config.index_of_first_server = 1;            // updates from server num 1, leaving␣
↪server 0 (from DHCP) intact
config.ip_event_to_renew = IP_EVENT_STA_GOT_IP;  // IP event on which we refresh␣
↪the configuration
```

Then we start the service normally with *esp_netif_sntp_start()*.

ESP-NETIF Programmer's Manual Please refer to the following example to understand the initialization process of the default interface:

- Wi-Fi Station: wifi/getting_started/station/main/station_example_main.c
- Ethernet: ethernet/basic/main/ethernet_example_main.c
- L2 TAP: protocols/l2tap/main/l2tap_main.c
- Wi-Fi Access Point: wifi/getting_started/softAP/main/softap_example_main.c

For more specific cases, please consult this guide: *ESP-NETIF Custom I/O Driver*.

Wi-Fi Default Initialization The initialization code as well as registering event handlers for default interfaces, such as softAP and station, are provided in separate APIs to facilitate simple startup code for most applications:

- *esp_netif_create_default_wifi_sta()*
- *esp_netif_create_default_wifi_ap()*

Please note that these functions return the `esp_netif` handle, i.e., a pointer to a network interface object allocated and configured with default settings, as a consequence, which means that:

- The created object has to be destroyed if a network de-initialization is provided by an application using *esp_netif_destroy_default_wifi()*.
- These *default* interfaces must not be created multiple times unless the created handle is deleted using *esp_netif_destroy()*.
- When using Wi-Fi in `AP+STA` mode, both these interfaces have to be created.

API Reference

Header File

- components/esp_netif/include/esp_netif.h
- This header file can be included with:

```
#include "esp_netif.h"
```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

```
REQUIRES esp_netif
```

or

```
PRIV_REQUIRES esp_netif
```

Functions

esp_err_t **esp_netif_init**(void)

Initialize the underlying TCP/IP stack.

Note: This function should be called exactly once from application code, when the application starts up.

Returns

- ESP_OK on success
- ESP_FAIL if initializing failed

esp_err_t **esp_netif_deinit**(void)

Deinitialize the esp-netif component (and the underlying TCP/IP stack)

```
Note: Deinitialization is not supported yet
```

Returns

- ESP_ERR_INVALID_STATE if esp_netif not initialized
- ESP_ERR_NOT_SUPPORTED otherwise

esp_netif_t ***esp_netif_new**(const *esp_netif_config_t* *esp_netif_config)

Creates an instance of new esp-netif object based on provided config.

Parameters **esp_netif_config** -- **[in]** pointer esp-netif configuration

Returns

- pointer to esp-netif object on success
- NULL otherwise

void **esp_netif_destroy**(*esp_netif_t* *esp_netif)

Destroys the esp_netif object.

Parameters **esp_netif** -- **[in]** pointer to the object to be deleted

esp_err_t **esp_netif_set_driver_config**(*esp_netif_t* *esp_netif, const *esp_netif_driver_ifconfig_t* *driver_config)

Configures driver related options of esp_netif object.

Parameters

- **esp_netif** -- **[inout]** pointer to the object to be configured
- **driver_config** -- **[in]** pointer esp-netif io driver related configuration

Returns

- ESP_OK on success
- ESP_ERR_ESP_NETIF_INVALID_PARAMS if invalid parameters provided

esp_err_t **esp_netif_attach**(*esp_netif_t* *esp_netif, *esp_netif_iodriver_handle* driver_handle)

Attaches esp_netif instance to the io driver handle.

Calling this function enables connecting specific esp_netif object with already initialized io driver to update esp_netif object with driver specific configuration (i.e. calls post_attach callback, which typically sets io driver callbacks to esp_netif instance and starts the driver)

Parameters

- **esp_netif** -- **[inout]** pointer to esp_netif object to be attached

- **driver_handle** -- **[in]** pointer to the driver handle

Returns

- ESP_OK on success
- ESP_ERR_ESP_NETIF_DRIVER_ATTACH_FAILED if driver's pot_attach callback failed

esp_err_t **esp_netif_receive**(*esp_netif_t* *esp_netif, void *buffer, size_t len, void *eb)

Passes the raw packets from communication media to the appropriate TCP/IP stack.

This function is called from the configured (peripheral) driver layer. The data are then forwarded as frames to the TCP/IP stack.

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **buffer** -- **[in]** Received data
- **len** -- **[in]** Length of the data frame
- **eb** -- **[in]** Pointer to internal buffer (used in Wi-Fi driver)

Returns

- ESP_OK

void **esp_netif_action_start**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IO driver start event Creates network interface, if AUTOUP enabled turns the interface on, if DHCPS enabled starts dhcp server.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_stop**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IO driver stop event.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_connected**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IO driver connected event.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_disconnected**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IO driver disconnected event.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_got_ip**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon network got IP event.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_join_ip6_multicast_group**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IPv6 multicast group join.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_leave_ip6_multicast_group**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IPv6 multicast group leave.

Note: This API can be directly used as event handler

Parameters

- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_add_ip6_address**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IPv6 address added by the underlying stack.

Note: This API can be directly used as event handler

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

void **esp_netif_action_remove_ip6_address**(void *esp_netif, esp_event_base_t base, int32_t event_id, void *data)

Default building block for network interface action upon IPv6 address removed by the underlying stack.

Note: This API can be directly used as event handler

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **base** -- The base type of the event
- **event_id** -- The specific ID of the event
- **data** -- Optional data associated with the event

esp_err_t **esp_netif_set_default_netif**(*esp_netif_t* *esp_netif)

Manual configuration of the default netif.

This API overrides the automatic configuration of the default interface based on the route_prio If the selected netif is set default using this API, no other interface could be set-default disregarding its route_prio number (unless the selected netif gets destroyed)

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns ESP_OK on success

esp_netif_t ***esp_netif_get_default_netif**(void)

Getter function of the default netif.

This API returns the selected default netif.

Returns Handle to esp-netif instance of the default netif.

esp_err_t **esp_netif_join_ip6_multicast_group**(*esp_netif_t* *esp_netif, const *esp_ip6_addr_t* *addr)

Cause the TCP/IP stack to join a IPv6 multicast group.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **addr** -- **[in]** The multicast group to join

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_MLD6_FAILED
- ESP_ERR_NO_MEM

esp_err_t **esp_netif_leave_ip6_multicast_group**(*esp_netif_t* *esp_netif, const *esp_ip6_addr_t* *addr)

Cause the TCP/IP stack to leave a IPv6 multicast group.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **addr** -- **[in]** The multicast group to leave

Returns

- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_MLD6_FAILED
- ESP_ERR_NO_MEM

esp_err_t **esp_netif_set_mac**(*esp_netif_t* *esp_netif, uint8_t mac[])

Set the mac address for the interface instance.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **mac** -- **[in]** Desired mac address for the related network interface

Returns
- ESP_OK - success
- ESP_ERR_ESP_NETIF_IF_NOT_READY - interface status error
- ESP_ERR_NOT_SUPPORTED - mac not supported on this interface

esp_err_t **esp_netif_get_mac**(*esp_netif_t* *esp_netif, uint8_t mac[])

Get the mac address for the interface instance.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **mac** -- **[out]** Resultant mac address for the related network interface

Returns
- ESP_OK - success
- ESP_ERR_ESP_NETIF_IF_NOT_READY - interface status error
- ESP_ERR_NOT_SUPPORTED - mac not supported on this interface

esp_err_t **esp_netif_set_hostname**(*esp_netif_t* *esp_netif, const char *hostname)

Set the hostname of an interface.

The configured hostname overrides the default configuration value CONFIG_LWIP_LOCAL_HOSTNAME. Please note that when the hostname is altered after interface started/connected the changes would only be reflected once the interface restarts/reconnects

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **hostname** -- **[in]** New hostname for the interface. Maximum length 32 bytes.

Returns
- ESP_OK - success
- ESP_ERR_ESP_NETIF_IF_NOT_READY - interface status error
- ESP_ERR_ESP_NETIF_INVALID_PARAMS - parameter error

esp_err_t **esp_netif_get_hostname**(*esp_netif_t* *esp_netif, const char **hostname)

Get interface hostname.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **hostname** -- **[out]** Returns a pointer to the hostname. May be NULL if no hostname is set. If set non-NULL, pointer remains valid (and string may change if the hostname changes).

Returns
- ESP_OK - success
- ESP_ERR_ESP_NETIF_IF_NOT_READY - interface status error
- ESP_ERR_ESP_NETIF_INVALID_PARAMS - parameter error

bool **esp_netif_is_netif_up**(*esp_netif_t* *esp_netif)

Test if supplied interface is up or down.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance

Returns
- true - Interface is up
- false - Interface is down

esp_err_t **esp_netif_get_ip_info**(*esp_netif_t* *esp_netif, *esp_netif_ip_info_t* *ip_info)

Get interface's IP address information.

If the interface is up, IP information is read directly from the TCP/IP stack. If the interface is down, IP information is read from a copy kept in the ESP-NETIF instance

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **ip_info** -- **[out]** If successful, IP information will be returned in this argument.

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

esp_err_t **esp_netif_get_old_ip_info**(*esp_netif_t* *esp_netif, *esp_netif_ip_info_t* *ip_info)

Get interface's old IP information.

Returns an "old" IP address previously stored for the interface when the valid IP changed.

If the IP lost timer has expired (meaning the interface was down for longer than the configured interval) then the old IP information will be zero.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **ip_info** -- **[out]** If successful, IP information will be returned in this argument.

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

esp_err_t **esp_netif_set_ip_info**(*esp_netif_t* *esp_netif, const *esp_netif_ip_info_t* *ip_info)

Set interface's IP address information.

This function is mainly used to set a static IP on an interface.

If the interface is up, the new IP information is set directly in the TCP/IP stack.

The copy of IP information kept in the ESP-NETIF instance is also updated (this copy is returned if the IP is queried while the interface is still down.)

Note: DHCP client/server must be stopped (if enabled for this interface) before setting new IP information.

Note: Calling this interface for may generate a SYSTEM_EVENT_STA_GOT_IP or SYSTEM_EVENT_ETH_GOT_IP event.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **ip_info** -- **[in]** IP information to set on the specified interface

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_NOT_STOPPED If DHCP server or client is still running

esp_err_t **esp_netif_set_old_ip_info**(*esp_netif_t* *esp_netif, const *esp_netif_ip_info_t* *ip_info)

Set interface old IP information.

This function is called from the DHCP client (if enabled), before a new IP is set. It is also called from the default handlers for the SYSTEM_EVENT_STA_CONNECTED and SYSTEM_EVENT_ETH_CONNECTED events.

Calling this function stores the previously configured IP, which can be used to determine if the IP changes in the future.

If the interface is disconnected or down for too long, the "IP lost timer" will expire (after the configured interval) and set the old IP information to zero.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **ip_info** -- **[in]** Store the old IP information for the specified interface

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

int **esp_netif_get_netif_impl_index**(*esp_netif_t* *esp_netif)

Get net interface index from network stack implementation.

Note: This index could be used in `setsockopt()` to bind socket with multicast interface

---

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns implementation specific index of interface represented with supplied esp_netif

esp_err_t **esp_netif_get_netif_impl_name**(*esp_netif_t* *esp_netif, char *name)

Get net interface name from network stack implementation.

Note: This name could be used in `setsockopt()` to bind socket with appropriate interface

---

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **name** -- **[out]** Interface name as specified in underlying TCP/IP stack. Note that the actual name will be copied to the specified buffer, which must be allocated to hold maximum interface name size (6 characters for lwIP)

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

esp_err_t **esp_netif_napt_enable**(*esp_netif_t* *esp_netif)

Enable NAPT on an interface.

Note: Enable operation can be performed only on one interface at a time. NAPT cannot be enabled on multiple interfaces according to this implementation.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns
- ESP_OK
- ESP_FAIL
- ESP_ERR_NOT_SUPPORTED

esp_err_t **esp_netif_napt_disable**(*esp_netif_t* *esp_netif)

Disable NAPT on an interface.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns
- ESP_OK
- ESP_FAIL
- ESP_ERR_NOT_SUPPORTED

esp_err_t **esp_netif_dhcps_option**(*esp_netif_t* *esp_netif, *esp_netif_dhcp_option_mode_t* opt_op, *esp_netif_dhcp_option_id_t* opt_id, void *opt_val, uint32_t opt_len)

Set or Get DHCP server option.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **opt_op** -- **[in]** ESP_NETIF_OP_SET to set an option, ESP_NETIF_OP_GET to get an option.
- **opt_id** -- **[in]** Option index to get or set, must be one of the supported enum values.
- **opt_val** -- **[inout]** Pointer to the option parameter.
- **opt_len** -- **[in]** Length of the option parameter.

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED

esp_err_t **esp_netif_dhcpc_option**(*esp_netif_t* *esp_netif, *esp_netif_dhcp_option_mode_t* opt_op, *esp_netif_dhcp_option_id_t* opt_id, void *opt_val, uint32_t opt_len)

Set or Get DHCP client option.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **opt_op** -- **[in]** ESP_NETIF_OP_SET to set an option, ESP_NETIF_OP_GET to get an option.
- **opt_id** -- **[in]** Option index to get or set, must be one of the supported enum values.
- **opt_val** -- **[inout]** Pointer to the option parameter.
- **opt_len** -- **[in]** Length of the option parameter.

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED

esp_err_t **esp_netif_dhcpc_start**(*esp_netif_t* *esp_netif)

Start DHCP client (only if enabled in interface object)

Note: The default event handlers for the SYSTEM_EVENT_STA_CONNECTED and SYSTEM_EVENT_ETH_CONNECTED events call this function.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED
- ESP_ERR_ESP_NETIF_DHCPC_START_FAILED

esp_err_t **esp_netif_dhcpc_stop**(*esp_netif_t* *esp_netif)

Stop DHCP client (only if enabled in interface object)

Note: Calling action_netif_stop() will also stop the DHCP Client if it is running.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED
- ESP_ERR_ESP_NETIF_IF_NOT_READY

esp_err_t **esp_netif_dhcpc_get_status**(*esp_netif_t* *esp_netif, *esp_netif_dhcp_status_t* *status)

Get DHCP client status.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **status** -- **[out]** If successful, the status of DHCP client will be returned in this argument.

Returns
- ESP_OK

esp_err_t **esp_netif_dhcps_get_status**(*esp_netif_t* *esp_netif, *esp_netif_dhcp_status_t* *status)

Get DHCP Server status.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **status** -- **[out]** If successful, the status of the DHCP server will be returned in this argument.

Returns
- ESP_OK

esp_err_t **esp_netif_dhcps_start**(*esp_netif_t* *esp_netif)

Start DHCP server (only if enabled in interface object)

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED

esp_err_t **esp_netif_dhcps_stop**(*esp_netif_t* *esp_netif)

Stop DHCP server (only if enabled in interface object)

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED
- ESP_ERR_ESP_NETIF_IF_NOT_READY

esp_err_t **esp_netif_dhcps_get_clients_by_mac**(*esp_netif_t* *esp_netif, int num, *esp_netif_pair_mac_ip_t* *mac_ip_pair)

Populate IP addresses of clients connected to DHCP server listed by their MAC addresses.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **num** -- **[in]** Number of clients with specified MAC addresses in the array of pairs
- **mac_ip_pair** -- **[inout]** Array of pairs of MAC and IP addresses (MAC are inputs, IP outputs)

Returns
- ESP_OK on success
- ESP_ERR_ESP_NETIF_INVALID_PARAMS on invalid params
- ESP_ERR_NOT_SUPPORTED if DHCP server not enabled

esp_err_t **esp_netif_set_dns_info**(*esp_netif_t* *esp_netif, *esp_netif_dns_type_t* type, *esp_netif_dns_info_t* *dns)

Set DNS Server information.

This function behaves differently if DHCP server or client is enabled

If DHCP client is enabled, main and backup DNS servers will be updated automatically from the DHCP lease if the relevant DHCP options are set. Fallback DNS Server is never updated from the DHCP lease and is designed to be set via this API. If DHCP client is disabled, all DNS server types can be set via this API only.

If DHCP server is enabled, the Main DNS Server setting is used by the DHCP server to provide a DNS Server option to DHCP clients (Wi-Fi stations).

- The default Main DNS server is typically the IP of the DHCP server itself.
- This function can override it by setting server type ESP_NETIF_DNS_MAIN.
- Other DNS Server types are not supported for the DHCP server.
- To propagate the DNS info to client, please stop the DHCP server before using this API.

Parameters
- **`esp_netif`** -- **[in]** Handle to esp-netif instance
- **`type`** -- **[in]** Type of DNS Server to set: ESP_NETIF_DNS_MAIN, ESP_NETIF_DNS_BACKUP, ESP_NETIF_DNS_FALLBACK
- **`dns`** -- **[in]** DNS Server address to set

Returns
- ESP_OK on success
- ESP_ERR_ESP_NETIF_INVALID_PARAMS invalid params

esp_err_t **`esp_netif_get_dns_info`**(*esp_netif_t* *esp_netif, *esp_netif_dns_type_t* type, *esp_netif_dns_info_t* *dns)

Get DNS Server information.

Return the currently configured DNS Server address for the specified interface and Server type.

This may be result of a previous call to *esp_netif_set_dns_info()*. If the interface's DHCP client is enabled, the Main or Backup DNS Server may be set by the current DHCP lease.

Parameters
- **`esp_netif`** -- **[in]** Handle to esp-netif instance
- **`type`** -- **[in]** Type of DNS Server to get: ESP_NETIF_DNS_MAIN, ESP_NETIF_DNS_BACKUP, ESP_NETIF_DNS_FALLBACK
- **`dns`** -- **[out]** DNS Server result is written here on success

Returns
- ESP_OK on success
- ESP_ERR_ESP_NETIF_INVALID_PARAMS invalid params

esp_err_t **`esp_netif_create_ip6_linklocal`**(*esp_netif_t* *esp_netif)

Create interface link-local IPv6 address.

Cause the TCP/IP stack to create a link-local IPv6 address for the specified interface.

This function also registers a callback for the specified interface, so that if the link-local address becomes verified as the preferred address then a SYSTEM_EVENT_GOT_IP6 event will be sent.

Parameters **`esp_netif`** -- **[in]** Handle to esp-netif instance

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS

esp_err_t **`esp_netif_get_ip6_linklocal`**(*esp_netif_t* *esp_netif, *esp_ip6_addr_t* *if_ip6)

Get interface link-local IPv6 address.

If the specified interface is up and a preferred link-local IPv6 address has been created for the interface, return a copy of it.

Parameters
- **`esp_netif`** -- **[in]** Handle to esp-netif instance
- **`if_ip6`** -- **[out]** IPv6 information will be returned in this argument if successful.

Returns
- ESP_OK

- ESP_FAIL If interface is down, does not have a link-local IPv6 address, or the link-local IPv6 address is not a preferred address.

esp_err_t **esp_netif_get_ip6_global** (*esp_netif_t* *esp_netif, *esp_ip6_addr_t* *if_ip6)

Get interface global IPv6 address.

If the specified interface is up and a preferred global IPv6 address has been created for the interface, return a copy of it.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **if_ip6** -- **[out]** IPv6 information will be returned in this argument if successful.

Returns
- ESP_OK
- ESP_FAIL If interface is down, does not have a global IPv6 address, or the global IPv6 address is not a preferred address.

int **esp_netif_get_all_ip6** (*esp_netif_t* *esp_netif, *esp_ip6_addr_t* if_ip6[])

Get all IPv6 addresses of the specified interface.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **if_ip6** -- **[out]** Array of IPv6 addresses will be copied to the argument

Returns number of returned IPv6 addresses

esp_err_t **esp_netif_add_ip6_address** (*esp_netif_t* *esp_netif, const *esp_ip6_addr_t* addr, bool preferred)

Cause the TCP/IP stack to add an IPv6 address to the interface.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **addr** -- **[in]** The address to be added
- **preferred** -- **[in]** The preferred status of the address

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_IP6_ADDR_FAILED
- ESP_ERR_NO_MEM

esp_err_t **esp_netif_remove_ip6_address** (*esp_netif_t* *esp_netif, const *esp_ip6_addr_t* *addr)

Cause the TCP/IP stack to remove an IPv6 address from the interface.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **addr** -- **[in]** The address to be removed

Returns
- ESP_OK
- ESP_ERR_ESP_NETIF_INVALID_PARAMS
- ESP_ERR_ESP_NETIF_IP6_ADDR_FAILED
- ESP_ERR_NO_MEM

void **esp_netif_set_ip4_addr** (*esp_ip4_addr_t* *addr, uint8_t a, uint8_t b, uint8_t c, uint8_t d)

Sets IPv4 address to the specified octets.

Parameters
- **addr** -- **[out]** IP address to be set
- **a** -- the first octet (127 for IP 127.0.0.1)
- **b** --
- **c** --
- **d** --

char ***esp_ip4addr_ntoa** (const *esp_ip4_addr_t* *addr, char *buf, int buflen)

Converts numeric IP address into decimal dotted ASCII representation.

Parameters
- **addr** -- ip address in network order to convert
- **buf** -- target buffer where the string is stored
- **buflen** -- length of buf

Returns either pointer to buf which now holds the ASCII representation of addr or NULL if buf was too small

uint32_t **esp_ip4addr_aton**(const char *addr)

Ascii internet address interpretation routine The value returned is in network order.

Parameters **addr** -- IP address in ascii representation (e.g. "127.0.0.1")
Returns ip address in network order

esp_err_t **esp_netif_str_to_ip4**(const char *src, *esp_ip4_addr_t* *dst)

Converts Ascii internet IPv4 address into esp_ip4_addr_t.

Parameters
- **src** -- **[in]** IPv4 address in ascii representation (e.g. "127.0.0.1")
- **dst** -- **[out]** Address of the target esp_ip4_addr_t structure to receive converted address

Returns
- ESP_OK on success
- ESP_FAIL if conversion failed
- ESP_ERR_INVALID_ARG if invalid parameter is passed into

esp_err_t **esp_netif_str_to_ip6**(const char *src, *esp_ip6_addr_t* *dst)

Converts Ascii internet IPv6 address into esp_ip4_addr_t Zeros in the IP address can be stripped or completely ommited: "2001:db8:85a3:0:0:0:2:1" or "2001:db8::2:1")

Parameters
- **src** -- **[in]** IPv6 address in ascii representation (e.g. ""2001:0db8:85a3:0000:0000:0000:0002:0001")
- **dst** -- **[out]** Address of the target esp_ip6_addr_t structure to receive converted address

Returns
- ESP_OK on success
- ESP_FAIL if conversion failed
- ESP_ERR_INVALID_ARG if invalid parameter is passed into

esp_netif_iodriver_handle **esp_netif_get_io_driver**(*esp_netif_t* *esp_netif)

Gets media driver handle for this esp-netif instance.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns opaque pointer of related IO driver

esp_netif_t ***esp_netif_get_handle_from_ifkey**(const char *if_key)

Searches over a list of created objects to find an instance with supplied if key.

Parameters **if_key** -- Textual description of network interface
Returns Handle to esp-netif instance

esp_netif_flags_t **esp_netif_get_flags**(*esp_netif_t* *esp_netif)

Returns configured flags for this interface.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns Configuration flags

const char ***esp_netif_get_ifkey**(*esp_netif_t* *esp_netif)

Returns configured interface key for this esp-netif instance.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns Textual description of related interface

const char ***esp_netif_get_desc**(*esp_netif_t* *esp_netif)

Returns configured interface type for this esp-netif instance.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns Enumerated type of this interface, such as station, AP, ethernet

int **esp_netif_get_route_prio**(*esp_netif_t* *esp_netif)

Returns configured routing priority number.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns Integer representing the instance's route-prio, or -1 if invalid paramters

int32_t **esp_netif_get_event_id**(*esp_netif_t* *esp_netif, *esp_netif_ip_event_type_t* event_type)

Returns configured event for this esp-netif instance and supplied event type.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **event_type** -- (either get or lost IP)

Returns specific event id which is configured to be raised if the interface lost or acquired IP address
-1 if supplied event_type is not known

esp_netif_t ***esp_netif_next**(*esp_netif_t* *esp_netif)

Iterates over list of interfaces. Returns first netif if NULL given as parameter.

Note: This API doesn't lock the list, nor the TCPIP context, as this it's usually required to get atomic access between iteration steps rather that within a single iteration. Therefore it is recommended to iterate over the interfaces inside *esp_netif_tcpip_exec()*

Note: This API is deprecated. Please use *esp_netif_next_unsafe()* directly if all the system interfaces are under your control and you can safely iterate over them. Otherwise, iterate over interfaces using *esp_netif_tcpip_exec()*, or use *esp_netif_find_if()* to search in the list of netifs with defined predicate.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns First netif from the list if supplied parameter is NULL, next one otherwise

esp_netif_t ***esp_netif_next_unsafe**(*esp_netif_t* *esp_netif)

Iterates over list of interfaces without list locking. Returns first netif if NULL given as parameter.

Used for bulk search loops within TCPIP context, e.g. using *esp_netif_tcpip_exec()*, or if we're sure that the iteration is safe from our application perspective (e.g. no interface is removed between iterations)

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns First netif from the list if supplied parameter is NULL, next one otherwise

esp_netif_t ***esp_netif_find_if**(*esp_netif_find_predicate_t* fn, void *ctx)

Return a netif pointer for the first interface that meets criteria defined by the callback.

Parameters
- **fn** -- Predicate function returning true for the desired interface
- **ctx** -- Context pointer passed to the predicate, typically a descriptor to compare with

Returns valid netif pointer if found, NULL if not

size_t **esp_netif_get_nr_of_ifs**(void)

Returns number of registered esp_netif objects.

Returns Number of esp_netifs

void **esp_netif_netstack_buf_ref**(void *netstack_buf)

increase the reference counter of net stack buffer

Parameters **netstack_buf** -- **[in]** the net stack buffer

void **esp_netif_netstack_buf_free**(void *netstack_buf)

free the netstack buffer

Parameters **netstack_buf** -- **[in]** the net stack buffer

esp_err_t **esp_netif_tcpip_exec**(*esp_netif_callback_fn* fn, void *ctx)

Utility to execute the supplied callback in TCP/IP context.

Parameters
- **fn** -- Pointer to the callback
- **ctx** -- Parameter to the callback

Returns The error code (esp_err_t) returned by the callback

Type Definitions

typedef bool (***esp_netif_find_predicate_t**)(*esp_netif_t* *netif, void *ctx)

Predicate callback for *esp_netif_find_if()* used to find interface which meets defined criteria.

typedef *esp_err_t* (***esp_netif_callback_fn**)(void *ctx)

TCPIP thread safe callback used with *esp_netif_tcpip_exec()*

Header File

- components/esp_netif/include/esp_netif_sntp.h
- This header file can be included with:

```
#include "esp_netif_sntp.h"
```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

```
REQUIRES esp_netif
```

or

```
PRIV_REQUIRES esp_netif
```

Functions

esp_err_t **esp_netif_sntp_init**(const *esp_sntp_config_t* *config)

Initialize SNTP with supplied config struct.

Parameters **config** -- Config struct
Returns ESP_OK on success

esp_err_t **esp_netif_sntp_start**(void)

Start SNTP service if it wasn't started during init (config.start = false) or restart it if already started.

Returns ESP_OK on success

void **esp_netif_sntp_deinit**(void)

Deinitialize esp_netif SNTP module.

esp_err_t **esp_netif_sntp_sync_wait**(TickType_t tout)

Wait for time sync event.

Parameters **tout** -- Specified timeout in RTOS ticks
Returns ESP_TIMEOUT if sync event didn't came withing the timeout ESP_ERR_NOT_FINISHED if the sync event came, but we're in smooth update mode and still in progress (SNTP_SYNC_STATUS_IN_PROGRESS) ESP_OK if time sync'ed

esp_err_t **esp_netif_sntp_reachability**(unsigned int index, unsigned int *reachability)

Returns SNTP server's reachability shift register as described in RFC 5905.

Parameters

- **index** -- Index of the SERVER
- **reachability** -- reachability shift register

Returns ESP_OK on success, ESP_ERR_INVALID_STATE if SNTP not initialized ESP_ERR_INVALID_ARG if invalid arguments

Structures

struct **esp_sntp_config**

SNTP configuration struct.

Public Members

bool **smooth_sync**

set to true if smooth sync required

bool **server_from_dhcp**

set to true to request NTP server config from DHCP

bool **wait_for_sync**

if true, we create a semaphore to signal time sync event

bool **start**

set to true to automatically start the SNTP service

esp_sntp_time_cb_t **sync_cb**

optionally sets callback function on time sync event

bool **renew_servers_after_new_IP**

this is used to refresh server list if NTP provided by DHCP (which cleans other pre-configured servers)

ip_event_t **ip_event_to_renew**

set the IP event id on which we refresh server list (if renew_servers_after_new_IP=true)

size_t **index_of_first_server**

refresh server list after this server (if renew_servers_after_new_IP=true)

size_t **num_of_servers**

number of preconfigured NTP servers

const char ***servers**[1]

list of servers

Macros

ESP_SNTP_SERVER_LIST(...)

Utility macro for providing multiple servers in parentheses.

ESP_NETIF_SNTP_DEFAULT_CONFIG_MULTIPLE(servers_in_list, list_of_servers)

Default configuration to init SNTP with multiple servers.

Parameters

- **servers_in_list** -- Number of servers in the list
- **list_of_servers** -- List of servers (use *ESP_SNTP_SERVER_LIST(...)*)

ESP_NETIF_SNTP_DEFAULT_CONFIG(server)

Default configuration with a single server.

Type Definitions

typedef void (***esp_sntp_time_cb_t**)(struct timeval *tv)

Time sync notification function.

typedef struct *esp_sntp_config* **esp_sntp_config_t**

SNTP configuration struct.

Header File

- components/esp_netif/include/esp_netif_types.h
- This header file can be included with:

```
#include "esp_netif_types.h"
```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

```
REQUIRES esp_netif
```

or

```
PRIV_REQUIRES esp_netif
```

Structures

struct **esp_netif_dns_info_t**

DNS server info.

Public Members

esp_ip_addr_t **ip**

IPV4 address of DNS server

struct **esp_netif_ip_info_t**

Event structure for IP_EVENT_STA_GOT_IP, IP_EVENT_ETH_GOT_IP events

Public Members

esp_ip4_addr_t **ip**

Interface IPV4 address

esp_ip4_addr_t **netmask**

Interface IPV4 netmask

esp_ip4_addr_t **gw**

Interface IPV4 gateway address

struct **esp_netif_ip6_info_t**

IPV6 IP address information.

Public Members

esp_ip6_addr_t **ip**

Interface IPV6 address

struct **ip_event_got_ip_t**

Event structure for IP_EVENT_GOT_IP event.

Public Members

esp_netif_t ***esp_netif**

Pointer to corresponding esp-netif object

esp_netif_ip_info_t **ip_info**

IP address, netmask, gatway IP address

bool **ip_changed**

Whether the assigned IP has changed or not

struct **ip_event_got_ip6_t**

Event structure for IP_EVENT_GOT_IP6 event

Public Members

esp_netif_t ***esp_netif**

Pointer to corresponding esp-netif object

esp_netif_ip6_info_t **ip6_info**

IPv6 address of the interface

int **ip_index**

IPv6 address index

struct **ip_event_add_ip6_t**

Event structure for ADD_IP6 event

Public Members

esp_ip6_addr_t **addr**

The address to be added to the interface

bool **preferred**

The default preference of the address

struct **ip_event_ap_staipassigned_t**

Event structure for IP_EVENT_AP_STAIPASSIGNED event

Public Members

esp_netif_t ***esp_netif**

Pointer to the associated netif handle

esp_ip4_addr_t **ip**

IP address which was assigned to the station

uint8_t **mac**[6]

MAC address of the connected client

struct **bridgeif_config**

LwIP bridge configuration

Public Members

uint16_t **max_fdb_dyn_entries**

maximum number of entries in dynamic forwarding database

uint16_t **max_fdb_sta_entries**

maximum number of entries in static forwarding database

uint8_t **max_ports**

maximum number of ports the bridge can consist of

struct **esp_netif_inherent_config**

ESP-netif inherent config parameters.

Public Members

esp_netif_flags_t **flags**

flags that define esp-netif behavior

uint8_t **mac**[6]

initial mac address for this interface

const *esp_netif_ip_info_t* ***ip_info**

initial ip address for this interface

uint32_t **get_ip_event**

event id to be raised when interface gets an IP

uint32_t **lost_ip_event**

event id to be raised when interface losts its IP

const char ***if_key**

string identifier of the interface

const char ***if_desc**

textual description of the interface

int **route_prio**

numeric priority of this interface to become a default routing if (if other netifs are up). A higher value of route_prio indicates a higher priority

bridgeif_config_t ***bridge_info**

LwIP bridge configuration

struct **esp_netif_driver_base_s**

ESP-netif driver base handle.

Public Members

esp_err_t (***post_attach**)(*esp_netif_t* *netif, *esp_netif_iodriver_handle* h)

post attach function pointer

esp_netif_t ***netif**

netif handle

struct **esp_netif_driver_ifconfig**

Specific IO driver configuration.

Public Members

esp_netif_iodriver_handle **handle**

io-driver handle

esp_err_t (***transmit**)(void *h, void *buffer, size_t len)

transmit function pointer

esp_err_t (***transmit_wrap**)(void *h, void *buffer, size_t len, void *netstack_buffer)

transmit wrap function pointer

void (***driver_free_rx_buffer**)(void *h, void *buffer)

free rx buffer function pointer

struct **esp_netif_config**

Generic esp_netif configuration.

Public Members

const *esp_netif_inherent_config_t* ***base**

base config

const *esp_netif_driver_ifconfig_t* ***driver**

driver config

const *esp_netif_netstack_config_t* ***stack**

stack config

struct **esp_netif_pair_mac_ip_t**

DHCP client's addr info (pair of MAC and IP address)

Public Members

uint8_t **mac**[6]

Clients MAC address

esp_ip4_addr_t **ip**

Clients IP address

Macros

ESP_ERR_ESP_NETIF_BASE

Definition of ESP-NETIF based errors.

ESP_ERR_ESP_NETIF_INVALID_PARAMS

ESP_ERR_ESP_NETIF_IF_NOT_READY

ESP_ERR_ESP_NETIF_DHCPC_START_FAILED

ESP_ERR_ESP_NETIF_DHCP_ALREADY_STARTED

ESP_ERR_ESP_NETIF_DHCP_ALREADY_STOPPED

ESP_ERR_ESP_NETIF_NO_MEM

ESP_ERR_ESP_NETIF_DHCP_NOT_STOPPED

ESP_ERR_ESP_NETIF_DRIVER_ATTACH_FAILED

ESP_ERR_ESP_NETIF_INIT_FAILED

ESP_ERR_ESP_NETIF_DNS_NOT_CONFIGURED

ESP_ERR_ESP_NETIF_MLD6_FAILED

ESP_ERR_ESP_NETIF_IP6_ADDR_FAILED

ESP_ERR_ESP_NETIF_DHCPS_START_FAILED

ESP_NETIF_BR_FLOOD
Definition of ESP-NETIF bridge controll.

ESP_NETIF_BR_DROP

ESP_NETIF_BR_FDW_CPU

Type Definitions

typedef struct esp_netif_obj **esp_netif_t**

typedef enum *esp_netif_flags* **esp_netif_flags_t**

typedef enum *esp_netif_ip_event_type* **esp_netif_ip_event_type_t**

typedef struct *bridgeif_config* **bridgeif_config_t**
LwIP bridge configuration

typedef struct *esp_netif_inherent_config* **esp_netif_inherent_config_t**
ESP-netif inherent config parameters.

typedef struct *esp_netif_config* **esp_netif_config_t**

typedef void ***esp_netif_iodriver_handle**
IO driver handle type.

typedef struct *esp_netif_driver_base_s* **esp_netif_driver_base_t**
ESP-netif driver base handle.

typedef struct *esp_netif_driver_ifconfig* **esp_netif_driver_ifconfig_t**

typedef struct esp_netif_netstack_config **esp_netif_netstack_config_t**
Specific L3 network stack configuration.

typedef *esp_err_t* (***esp_netif_receive_t**)(*esp_netif_t* *esp_netif, void *buffer, size_t len, void *eb)
ESP-NETIF Receive function type.

Enumerations

enum **esp_netif_dns_type_t**

Type of DNS server.

Values:

enumerator **ESP_NETIF_DNS_MAIN**

DNS main server address

enumerator **ESP_NETIF_DNS_BACKUP**

DNS backup server address (Wi-Fi STA and Ethernet only)

enumerator **ESP_NETIF_DNS_FALLBACK**

DNS fallback server address (Wi-Fi STA and Ethernet only)

enumerator **ESP_NETIF_DNS_MAX**

enum **esp_netif_dhcp_status_t**

Status of DHCP client or DHCP server.

Values:

enumerator **ESP_NETIF_DHCP_INIT**

DHCP client/server is in initial state (not yet started)

enumerator **ESP_NETIF_DHCP_STARTED**

DHCP client/server has been started

enumerator **ESP_NETIF_DHCP_STOPPED**

DHCP client/server has been stopped

enumerator **ESP_NETIF_DHCP_STATUS_MAX**

enum **esp_netif_dhcp_option_mode_t**

Mode for DHCP client or DHCP server option functions.

Values:

enumerator **ESP_NETIF_OP_START**

enumerator **ESP_NETIF_OP_SET**

Set option

enumerator **ESP_NETIF_OP_GET**

Get option

enumerator **ESP_NETIF_OP_MAX**

enum **esp_netif_dhcp_option_id_t**

Supported options for DHCP client or DHCP server.

Values:

enumerator **ESP_NETIF_SUBNET_MASK**

Network mask

enumerator **ESP_NETIF_DOMAIN_NAME_SERVER**

Domain name server

enumerator **ESP_NETIF_ROUTER_SOLICITATION_ADDRESS**

Solicitation router address

enumerator **ESP_NETIF_REQUESTED_IP_ADDRESS**

Request specific IP address

enumerator **ESP_NETIF_IP_ADDRESS_LEASE_TIME**

Request IP address lease time

enumerator **ESP_NETIF_IP_REQUEST_RETRY_TIME**

Request IP address retry counter

enumerator **ESP_NETIF_VENDOR_CLASS_IDENTIFIER**

Vendor Class Identifier of a DHCP client

enumerator **ESP_NETIF_VENDOR_SPECIFIC_INFO**

Vendor Specific Information of a DHCP server

enum **ip_event_t**

IP event declarations

Values:

enumerator **IP_EVENT_STA_GOT_IP**

station got IP from connected AP

enumerator **IP_EVENT_STA_LOST_IP**

station lost IP and the IP is reset to 0

enumerator **IP_EVENT_AP_STAIPASSIGNED**

soft-AP assign an IP to a connected station

enumerator **IP_EVENT_GOT_IP6**

station or ap or ethernet interface v6IP addr is preferred

enumerator **IP_EVENT_ETH_GOT_IP**

ethernet got IP from connected AP

enumerator **IP_EVENT_ETH_LOST_IP**

ethernet lost IP and the IP is reset to 0

enumerator **IP_EVENT_PPP_GOT_IP**

PPP interface got IP

enumerator **IP_EVENT_PPP_LOST_IP**

PPP interface lost IP

enum **esp_netif_flags**

Values:

enumerator **ESP_NETIF_DHCP_CLIENT**

enumerator **ESP_NETIF_DHCP_SERVER**

enumerator **ESP_NETIF_FLAG_AUTOUP**

enumerator **ESP_NETIF_FLAG_GARP**

enumerator **ESP_NETIF_FLAG_EVENT_IP_MODIFIED**

enumerator **ESP_NETIF_FLAG_IS_PPP**

enumerator **ESP_NETIF_FLAG_IS_BRIDGE**

enumerator **ESP_NETIF_FLAG_MLDV6_REPORT**

enum **esp_netif_ip_event_type**

Values:

enumerator **ESP_NETIF_IP_EVENT_GOT_IP**

enumerator **ESP_NETIF_IP_EVENT_LOST_IP**

Header File

- components/esp_netif/include/esp_netif_ip_addr.h
- This header file can be included with:

```
#include "esp_netif_ip_addr.h"
```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

```
REQUIRES esp_netif
```

or

```
PRIV_REQUIRES esp_netif
```

Functions

esp_ip6_addr_type_t **esp_netif_ip6_get_addr_type**(*esp_ip6_addr_t* *ip6_addr)

Get the IPv6 address type.

Parameters **ip6_addr** -- **[in]** IPv6 type
Returns IPv6 type in form of enum esp_ip6_addr_type_t

static inline void **esp_netif_ip_addr_copy**(*esp_ip_addr_t* *dest, const *esp_ip_addr_t* *src)

Copy IP addresses.

Parameters

- **dest** -- **[out]** destination IP
- **src** -- **[in]** source IP

Structures

struct **esp_ip6_addr**

IPv6 address.

Public Members

uint32_t **addr**[4]

IPv6 address

uint8_t **zone**

zone ID

struct **esp_ip4_addr**

IPv4 address.

Public Members

uint32_t **addr**

IPv4 address

struct **_ip_addr**

IP address.

Public Members

esp_ip6_addr_t **ip6**

IPv6 address type

esp_ip4_addr_t **ip4**

IPv4 address type

union *_ip_addr*::[anonymous] **u_addr**

IP address union

uint8_t **type**

ipaddress type

Macros

esp_netif_htonl (x)

esp_netif_ip4_makeu32 (a, b, c, d)

ESP_IP6_ADDR_BLOCK1 (ip6addr)

ESP_IP6_ADDR_BLOCK2 (ip6addr)

ESP_IP6_ADDR_BLOCK3 (ip6addr)

ESP_IP6_ADDR_BLOCK4 (ip6addr)

ESP_IP6_ADDR_BLOCK5 (ip6addr)

ESP_IP6_ADDR_BLOCK6 (ip6addr)

ESP_IP6_ADDR_BLOCK7 (ip6addr)

ESP_IP6_ADDR_BLOCK8 (ip6addr)

IPSTR

esp_ip4_addr_get_byte (ipaddr, idx)

esp_ip4_addr1 (ipaddr)

esp_ip4_addr2 (ipaddr)

esp_ip4_addr3 (ipaddr)

esp_ip4_addr4 (ipaddr)

esp_ip4_addr1_16 (ipaddr)

esp_ip4_addr2_16 (ipaddr)

esp_ip4_addr3_16 (ipaddr)

esp_ip4_addr4_16 (ipaddr)

IP2STR (ipaddr)

IPV6STR

IPV62STR (ipaddr)

ESP_IPADDR_TYPE_V4

ESP_IPADDR_TYPE_V6

ESP_IPADDR_TYPE_ANY

ESP_IP4TOUINT32 (a, b, c, d)

ESP_IP4TOADDR (a, b, c, d)

ESP_IP4ADDR_INIT (a, b, c, d)

ESP_IP6ADDR_INIT (a, b, c, d)

IP4ADDR_STRLEN_MAX

ESP_IP_IS_ANY (addr)

Type Definitions

typedef struct *esp_ip4_addr* **esp_ip4_addr_t**

typedef struct *esp_ip6_addr* **esp_ip6_addr_t**

typedef struct *_ip_addr* **esp_ip_addr_t**

IP address.

Enumerations

enum **esp_ip6_addr_type_t**

Values:

enumerator **ESP_IP6_ADDR_IS_UNKNOWN**

enumerator **ESP_IP6_ADDR_IS_GLOBAL**

enumerator **ESP_IP6_ADDR_IS_LINK_LOCAL**

enumerator **ESP_IP6_ADDR_IS_SITE_LOCAL**

enumerator **ESP_IP6_ADDR_IS_UNIQUE_LOCAL**

enumerator **ESP_IP6_ADDR_IS_IPV4_MAPPED_IPV6**

Header File

- components/esp_netif/include/esp_vfs_l2tap.h
- This header file can be included with:

```
#include "esp_vfs_l2tap.h"
```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

```
REQUIRES esp_netif
```

or

```
PRIV_REQUIRES esp_netif
```

Functions

esp_err_t **esp_vfs_l2tap_intf_register**(*l2tap_vfs_config_t* *config)

Add L2 TAP virtual filesystem driver.

This function must be called prior usage of ESP-NETIF L2 TAP Interface

Parameters **config** -- L2 TAP virtual filesystem driver configuration. Default base path /dev/net/tap is used when this paramenter is NULL.

Returns esp_err_t

- ESP_OK on success

esp_err_t **esp_vfs_l2tap_intf_unregister**(const char *base_path)

Removes L2 TAP virtual filesystem driver.

Parameters **base_path** -- Base path to the L2 TAP virtual filesystem driver. Default path /dev/net/tap is used when this paramenter is NULL.

Returns esp_err_t

- ESP_OK on success

esp_err_t **esp_vfs_l2tap_eth_filter**(*l2tap_iodriver_handle* driver_handle, void *buff, size_t *size)

Filters received Ethernet L2 frames into L2 TAP infrastructure.

Parameters

- **driver_handle** -- handle of driver at which the frame was received
- **buff** -- received L2 frame
- **size** -- input length of the L2 frame which is set to 0 when frame is filtered into L2 TAP

Returns esp_err_t

- ESP_OK is always returned

Structures

struct **l2tap_vfs_config_t**

L2Tap VFS config parameters.

Public Members

const char ***base_path**

vfs base path

Macros

L2TAP_VFS_DEFAULT_PATH

L2TAP_VFS_CONFIG_DEFAULT()

Type Definitions

typedef void ***l2tap_iodriver_handle**

Enumerations

enum **l2tap_ioctl_opt_t**

Values:

enumerator **L2TAP_S_RCV_FILTER**

enumerator **L2TAP_G_RCV_FILTER**

enumerator **L2TAP_S_INTF_DEVICE**

enumerator **L2TAP_G_INTF_DEVICE**

enumerator **L2TAP_S_DEVICE_DRV_HNDL**

enumerator **L2TAP_G_DEVICE_DRV_HNDL**

Wi-Fi Default API Reference

Header File

- components/esp_wifi/include/esp_wifi_default.h
- This header file can be included with:

```
#include "esp_wifi_default.h"
```

- This header file is a part of the API provided by the `esp_wifi` component. To declare that your component depends on `esp_wifi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_wifi
```

or

```
PRIV_REQUIRES esp_wifi
```

Functions

esp_err_t **esp_netif_attach_wifi_station**(*esp_netif_t* *esp_netif)

Attaches wifi station interface to supplied netif.

Parameters **esp_netif** -- instance to attach the wifi station to

Returns

- ESP_OK on success
- ESP_FAIL if attach failed

esp_err_t **esp_netif_attach_wifi_ap**(*esp_netif_t* *esp_netif)

Attaches wifi soft AP interface to supplied netif.

Parameters **esp_netif** -- instance to attach the wifi AP to

Returns

- ESP_OK on success
- ESP_FAIL if attach failed

esp_err_t **esp_wifi_set_default_wifi_sta_handlers**(void)

Sets default wifi event handlers for STA interface.

Returns

- ESP_OK on success, error returned from esp_event_handler_register if failed

esp_err_t **esp_wifi_set_default_wifi_ap_handlers**(void)

Sets default wifi event handlers for AP interface.

Returns

- ESP_OK on success, error returned from esp_event_handler_register if failed

esp_err_t **esp_wifi_set_default_wifi_nan_handlers**(void)

Sets default wifi event handlers for NAN interface.

Returns

- ESP_OK on success, error returned from esp_event_handler_register if failed

esp_err_t **esp_wifi_clear_default_wifi_driver_and_handlers**(void *esp_netif)

Clears default wifi event handlers for supplied network interface.

Parameters **esp_netif** -- instance of corresponding if object

Returns

- ESP_OK on success, error returned from esp_event_handler_register if failed

esp_netif_t ***esp_netif_create_default_wifi_ap**(void)

Creates default WIFI AP. In case of any init error this API aborts.

Note: The API creates esp_netif object with default WiFi access point config, attaches the netif to wifi and registers wifi handlers to the default event loop. This API uses assert() to check for potential errors, so it could abort the program. (Note that the default event loop needs to be created prior to calling this API)

Returns pointer to esp-netif instance

esp_netif_t ***esp_netif_create_default_wifi_sta**(void)

Creates default WIFI STA. In case of any init error this API aborts.

Note: The API creates esp_netif object with default WiFi station config, attaches the netif to wifi and registers wifi handlers to the default event loop. This API uses assert() to check for potential errors, so it could abort the program. (Note that the default event loop needs to be created prior to calling this API)

Returns pointer to esp-netif instance

esp_netif_t ***esp_netif_create_default_wifi_nan**(void)

Creates default WIFI NAN. In case of any init error this API aborts.

Note: The API creates esp_netif object with default WiFi station config, attaches the netif to wifi and registers wifi handlers to the default event loop. (Note that the default event loop needs to be created prior to calling this API)

Returns pointer to esp-netif instance

void **esp_netif_destroy_default_wifi**(void *esp_netif)

Destroys default WIFI netif created with esp_netif_create_default_wifi_...() API.

Note: This API unregisters wifi handlers and detaches the created object from the wifi. (this function is a no-operation if esp_netif is NULL)

Parameters **esp_netif** -- **[in]** object to detach from WiFi and destroy

esp_netif_t ***esp_netif_create_wifi**(wifi_interface_t wifi_if, const *esp_netif_inherent_config_t* *esp_netif_config)

Creates esp_netif WiFi object based on the custom configuration.

Attention This API DOES NOT register default handlers!

Parameters

- **wifi_if** -- **[in]** type of wifi interface
- **esp_netif_config** -- **[in]** inherent esp-netif configuration pointer

Returns pointer to esp-netif instance

esp_err_t **esp_netif_create_default_wifi_mesh_netifs**(*esp_netif_t* **p_netif_sta, *esp_netif_t* **p_netif_ap)

Creates default STA and AP network interfaces for esp-mesh.

Both netifs are almost identical to the default station and softAP, but with DHCP client and server disabled. Please note that the DHCP client is typically enabled only if the device is promoted to a root node.

Returns created interfaces which could be ignored setting parameters to NULL if an application code does not need to save the interface instances for further processing.

Parameters

- **p_netif_sta** -- **[out]** pointer where the resultant STA interface is saved (if non NULL)
- **p_netif_ap** -- **[out]** pointer where the resultant AP interface is saved (if non NULL)

Returns ESP_OK on success

2.5.5 IP Network Layer

ESP-NETIF Custom I/O Driver

This section outlines implementing a new I/O driver with ESP-NETIF connection capabilities.

By convention, the I/O driver has to register itself as an ESP-NETIF driver, and thus holds a dependency on ESP-NETIF component and is responsible for providing data path functions, post-attach callback and in most cases, also default event handlers to define network interface actions based on driver's lifecycle transitions.

Packet Input/Output According to the diagram shown in the *ESP-NETIF Architecture* part, the following three API functions for the packet data path must be defined for connecting with ESP-NETIF:

- *esp_netif_transmit()*
- *esp_netif_free_rx_buffer()*
- *esp_netif_receive()*

The first two functions for transmitting and freeing the rx buffer are provided as callbacks, i.e., they get called from ESP-NETIF (and its underlying TCP/IP stack) and I/O driver provides their implementation.

The receiving function on the other hand gets called from the I/O driver, so that the driver's code simply calls *esp_netif_receive()* on a new data received event.

Post Attach Callback A final part of the network interface initialization consists of attaching the ESP-NETIF instance to the I/O driver, by means of calling the following API:

```
esp_err_t esp_netif_attach(esp_netif_t *esp_netif, esp_netif_iodriver_handle␣
↪driver_handle);
```

It is assumed that the `esp_netif_iodriver_handle` is a pointer to driver's object, a struct derived from `struct esp_netif_driver_base_s`, so that the first member of I/O driver structure must be this base structure with pointers to:

- post-attach function callback
- related ESP-NETIF instance

As a result, the I/O driver has to create an instance of the struct per below:

```
typedef struct my_netif_driver_s {
        esp_netif_driver_base_t base;           /*!< base structure reserved as␣
↪esp-netif driver */
        driver_impl             *h;             /*!< handle of driver␣
↪implementation */
    } my_netif_driver_t;
```

with actual values of `my_netif_driver_t::base.post_attach` and the actual drivers handle `my_netif_driver_t::h`.

So when the *esp_netif_attach()* gets called from the initialization code, the post-attach callback from I/O driver's code gets executed to mutually register callbacks between ESP-NETIF and I/O driver instances. Typically the driver is started as well in the post-attach callback. An example of a simple post-attach callback is outlined below:

```
static esp_err_t my_post_attach_start(esp_netif_t * esp_netif, void * args)
{
    my_netif_driver_t *driver = args;
    const esp_netif_driver_ifconfig_t driver_ifconfig = {
            .driver_free_rx_buffer = my_free_rx_buf,
            .transmit = my_transmit,
            .handle = driver->driver_impl
    };
    driver->base.netif = esp_netif;
    ESP_ERROR_CHECK(esp_netif_set_driver_config(esp_netif, &driver_ifconfig));
    my_driver_start(driver->driver_impl);
    return ESP_OK;
}
```

Default Handlers I/O drivers also typically provide default definitions of lifecycle behavior of related network interfaces based on state transitions of I/O drivers. For example *driver start -> network start*, etc.

An example of such a default handler is provided below:

```
esp_err_t my_driver_netif_set_default_handlers(my_netif_driver_t *driver, esp_
↪netif_t * esp_netif)
{
    driver_set_event_handler(driver->driver_impl, esp_netif_action_start, MY_DRV_
↪EVENT_START, esp_netif);
    driver_set_event_handler(driver->driver_impl, esp_netif_action_stop, MY_DRV_
↪EVENT_STOP, esp_netif);
    return ESP_OK;
}
```

Network Stack Connection The packet data path functions for transmitting and freeing the rx buffer (defined in the I/O driver) are called from the ESP-NETIF, specifically from its TCP/IP stack connecting layer.

Note that ESP-IDF provides several network stack configurations for the most common network interfaces, such as for the Wi-Fi station or Ethernet. These configurations are defined in esp_netif/include/esp_netif_defaults.h and should be sufficient for most network drivers. In rare cases, expert users might want to define custom lwIP based interface layers; it is possible, but an explicit dependency to lwIP needs to be set.

The following API reference outlines these network stack interaction with the ESP-NETIF:

Header File

- components/esp_netif/include/esp_netif_net_stack.h
- This header file can be included with:

  ```
  #include "esp_netif_net_stack.h"
  ```

- This header file is a part of the API provided by the `esp_netif` component. To declare that your component depends on `esp_netif`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_netif
  ```

  or

```
PRIV_REQUIRES esp_netif
```

Functions

esp_netif_t ***esp_netif_get_handle_from_netif_impl*** (void *dev)

Returns esp-netif handle.

Parameters **dev** -- **[in]** opaque ptr to network interface of specific TCP/IP stack
Returns handle to related esp-netif instance

void ***esp_netif_get_netif_impl*** (*esp_netif_t* *esp_netif)

Returns network stack specific implementation handle.

Parameters **esp_netif** -- **[in]** Handle to esp-netif instance
Returns handle to related network stack netif handle

esp_err_t **esp_netif_set_link_speed** (*esp_netif_t* *esp_netif, uint32_t speed)

Set link-speed for the specified network interface.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **speed** -- **[in]** Link speed in bit/s

Returns ESP_OK on success

esp_err_t **esp_netif_transmit** (*esp_netif_t* *esp_netif, void *data, size_t len)

Outputs packets from the TCP/IP stack to the media to be transmitted.

This function gets called from network stack to output packets to IO driver.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **data** -- **[in]** Data to be transmitted
- **len** -- **[in]** Length of the data frame

Returns ESP_OK on success, an error passed from the I/O driver otherwise

esp_err_t **esp_netif_transmit_wrap** (*esp_netif_t* *esp_netif, void *data, size_t len, void *netstack_buf)

Outputs packets from the TCP/IP stack to the media to be transmitted.

This function gets called from network stack to output packets to IO driver.

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **data** -- **[in]** Data to be transmitted
- **len** -- **[in]** Length of the data frame
- **netstack_buf** -- **[in]** net stack buffer

Returns ESP_OK on success, an error passed from the I/O driver otherwise

void **esp_netif_free_rx_buffer** (void *esp_netif, void *buffer)

Free the rx buffer allocated by the media driver.

This function gets called from network stack when the rx buffer to be freed in IO driver context, i.e. to deallocate a buffer owned by io driver (when data packets were passed to higher levels to avoid copying)

Parameters
- **esp_netif** -- **[in]** Handle to esp-netif instance
- **buffer** -- **[in]** Rx buffer pointer

Code examples for TCP/IP socket APIs are provided in the protocols/sockets directory of ESP-IDF examples.

2.5.6 Application Layer

Documentation for Application layer network protocols (above the IP Network layer) are provided in *Application Protocols*.

2.6 Peripherals API

2.6.1 Analog to Digital Converter (ADC) Oneshot Mode Driver

Introduction

The Analog to Digital Converter is integrated on the chip and is capable of measuring analog signals from specific analog IO pins.

ESP32-S3 has two ADC unit(s), which can be used in scenario(s) like:

- Generate one-shot ADC conversion result
- Generate continuous ADC conversion results

This guide introduces ADC oneshot mode conversion.

Functional Overview

The following sections of this document cover the typical steps to install and operate an ADC:

- *Resource Allocation* - covers which parameters should be set up to get an ADC handle and how to recycle the resources when ADC finishes working.
- *Unit Configuration* - covers the parameters that should be set up to configure the ADC unit, so as to get ADC conversion raw result.
- *Read Conversion Result* - covers how to get ADC conversion raw result.
- *Hardware Limitations* - describes the ADC-related hardware limitations.
- *Power Management* - covers power management-related information.
- *IRAM Safe* - describes tips on how to read ADC conversion raw results when the cache is disabled.
- *Thread Safety* - lists which APIs are guaranteed to be thread-safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can be used to make a different effect on driver behavior.

Resource Allocation The ADC oneshot mode driver is implemented based on ESP32-S3 SAR ADC module. Different ESP chips might have different numbers of independent ADCs. From the oneshot mode driver's point of view, an ADC instance is represented by `adc_oneshot_unit_handle_t`.

To install an ADC instance, set up the required initial configuration structure `adc_oneshot_unit_init_cfg_t`:

- `adc_oneshot_unit_init_cfg_t::unit_id` selects the ADC. Please refer to the datasheet to know dedicated analog IOs for this ADC.
- `adc_oneshot_unit_init_cfg_t::clk_src` selects the source clock of the ADC. If set to 0, the driver will fall back to using a default clock source, see `adc_oneshot_clk_src_t` to know the details.
- `adc_oneshot_unit_init_cfg_t::ulp_mode` sets if the ADC will be working under ULP mode.

After setting up the initial configurations for the ADC, call `adc_oneshot_new_unit()` with the prepared `adc_oneshot_unit_init_cfg_t`. This function will return an ADC unit handle if the allocation is successful.

This function may fail due to various errors such as invalid arguments, insufficient memory, etc. Specifically, when the to-be-allocated ADC instance is registered already, this function will return `ESP_ERR_NOT_FOUND` error. Number of available ADC(s) is recorded by `SOC_ADC_PERIPH_NUM`.

If a previously created ADC instance is no longer required, you should recycle the ADC instance by calling `adc_oneshot_del_unit()`, related hardware and software resources will be recycled as well.

Create an ADC Unit Handle Under Normal Oneshot Mode

```
adc_oneshot_unit_handle_t adc1_handle;
adc_oneshot_unit_init_cfg_t init_config1 = {
    .unit_id = ADC_UNIT_1,
    .ulp_mode = ADC_ULP_MODE_DISABLE,
};
ESP_ERROR_CHECK(adc_oneshot_new_unit(&init_config1, &adc1_handle));
```

Recycle the ADC Unit

```
ESP_ERROR_CHECK(adc_oneshot_del_unit(adc1_handle));
```

Unit Configuration After an ADC instance is created, set up the *adc_oneshot_chan_cfg_t* to configure ADC IOs to measure analog signal:

- *adc_oneshot_chan_cfg_t::atten*, ADC attenuation. Refer to TRM > On-Chip Sensor and Analog Signal Processing.
- *adc_oneshot_chan_cfg_t::bitwidth*, the bitwidth of the raw conversion result.

Note: For the IO corresponding ADC channel number, check datasheet to know the ADC IOs.

Additionally, *adc_continuous_io_to_channel()* and *adc_continuous_channel_to_io()* can be used to know the ADC channels and ADC IOs.

To make these settings take effect, call *adc_oneshot_config_channel()* with the above configuration structure. You should specify an ADC channel to be configured as well. Function *adc_oneshot_config_channel()* can be called multiple times to configure different ADC channels. The Driver will save each of these channel configurations internally.

Configure Two ADC Channels

```
adc_oneshot_chan_cfg_t config = {
    .bitwidth = ADC_BITWIDTH_DEFAULT,
    .atten = ADC_ATTEN_DB_12,
};
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, EXAMPLE_ADC1_CHAN0, &
↪config));
ESP_ERROR_CHECK(adc_oneshot_config_channel(adc1_handle, EXAMPLE_ADC1_CHAN1, &
↪config));
```

Read Conversion Result After above configurations, the ADC is ready to measure the analog signal(s) from the configured ADC channel(s). Call *adc_oneshot_read()* to get the conversion raw result of an ADC channel.

- *adc_oneshot_read()* is safe to use. ADC(s) are shared by some other drivers/peripherals, see *Hardware Limitations*. This function uses mutexes to avoid concurrent hardware usage. Therefore, this function should not be used in an ISR context. This function may fail when the ADC is in use by other drivers/peripherals, and return *ESP_ERR_TIMEOUT*. Under this condition, the ADC raw result is invalid.

This function will fail due to invalid arguments.

The ADC conversion results read from this function are raw data. To calculate the voltage based on the ADC raw results, this formula can be used:

```
Vout = Dout * Vmax / Dmax        (1)
```

where:

Vout	Digital output result, standing for the voltage.
Dout	ADC raw digital reading result.
Vmax	Maximum measurable input analog voltage, this is related to the ADC attenuation, please refer to TRM > `On-Chip Sensor and Analog Signal Processing`.
Dmax	Maximum of the output ADC raw digital reading result, which is 2^bitwidth, where bitwidth is the :cpp:member::*adc_oneshot_chan_cfg_t:bitwidth* configured before.

To do further calibration to convert the ADC raw result to voltage in mV, please refer to calibration doc *Analog to Digital Converter (ADC) Calibration Driver*.

Read Raw Result

```
ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, EXAMPLE_ADC1_CHAN0, &adc_raw[0][0]));
ESP_LOGI(TAG, "ADC%d Channel[%d] Raw Data: %d", ADC_UNIT_1 + 1, EXAMPLE_ADC1_CHAN0,
↪ adc_raw[0][0]);

ESP_ERROR_CHECK(adc_oneshot_read(adc1_handle, EXAMPLE_ADC1_CHAN1, &adc_raw[0][1]));
ESP_LOGI(TAG, "ADC%d Channel[%d] Raw Data: %d", ADC_UNIT_1 + 1, EXAMPLE_ADC1_CHAN1,
↪ adc_raw[0][1]);
```

Hardware Limitations

- Random Number Generator (RNG) uses ADC as an input source. When ADC *`adc_oneshot_read()`* works, the random number generated from RNG will be less random.
- A specific ADC unit can only work under one operating mode at any one time, either continuous mode or oneshot mode. *`adc_oneshot_read()`* has provided the protection.
- ADC2 is also used by Wi-Fi. *`adc_oneshot_read()`* has provided protection between the Wi-Fi driver and ADC oneshot mode driver.

Power Management When power management is enabled, i.e., *CONFIG_PM_ENABLE* is on, the system clock frequency may be adjusted when the system is in an idle state. However, the ADC oneshot mode driver works in a polling routine, the *`adc_oneshot_read()`* will poll the CPU until the function returns. During this period of time, the task in which ADC oneshot mode driver resides will not be blocked. Therefore the clock frequency is stable when reading.

IRAM Safe By default, all the ADC oneshot mode driver APIs are not supposed to be run when the Cache is disabled. Cache may be disabled due to many reasons, such as Flash writing/erasing, OTA, etc. If these APIs execute when the Cache is disabled, you will probably see errors like `Illegal Instruction` or `Load/Store Prohibited`.

Thread Safety

- *`adc_oneshot_new_unit()`*
- *`adc_oneshot_config_channel()`*
- *`adc_oneshot_read()`*

Above functions are guaranteed to be thread-safe. Therefore, you can call them from different RTOS tasks without protection by extra locks.

- *`adc_oneshot_del_unit()`* is not thread-safe. Besides, concurrently calling this function may result in failures of the above thread-safe APIs.

Kconfig Options

- *CONFIG_ADC_ONESHOT_CTRL_FUNC_IN_IRAM* controls where to place the ADC fast read function (IRAM or Flash), see *IRAM Safe* for more details.

Application Examples

- ADC oneshot mode example: peripherals/adc/oneshot_read.

API Reference

Header File

- components/hal/include/hal/adc_types.h
- This header file can be included with:

```
#include "hal/adc_types.h"
```

Structures

struct **adc_digi_pattern_config_t**

ADC digital controller pattern configuration.

Public Members

uint8_t **atten**

Attenuation of this ADC channel.

uint8_t **channel**

ADC channel.

uint8_t **unit**

ADC unit.

uint8_t **bit_width**

ADC output bit width.

struct **adc_digi_output_data_t**

ADC digital controller (DMA mode) output data format. Used to analyze the acquired ADC (DMA) data.

Public Members

uint32_t **data**

ADC real output data info. Resolution: 12 bit.

uint32_t **reserved12**

Reserved12.

uint32_t **channel**

ADC channel index info. If (channel < ADC_CHANNEL_MAX), The data is valid. If (channel > ADC_CHANNEL_MAX), The data is invalid.

uint32_t **unit**

ADC unit index info. 0: ADC1; 1: ADC2.

uint32_t **reserved17_31**

Reserved17.

struct *adc_digi_output_data_t*::[anonymous]::[anonymous] **type2**

When the configured output format is 12bit.

uint32_t **val**

Raw data value

Type Definitions

typedef *soc_periph_adc_digi_clk_src_t* **adc_oneshot_clk_src_t**

Clock source type of oneshot mode which uses digital controller.

typedef *soc_periph_adc_digi_clk_src_t* **adc_continuous_clk_src_t**

Clock source type of continuous mode which uses digital controller.

Enumerations

enum **adc_unit_t**

ADC unit.

Values:

enumerator **ADC_UNIT_1**

SAR ADC 1.

enumerator **ADC_UNIT_2**

SAR ADC 2.

enum **adc_channel_t**

ADC channels.

Values:

enumerator **ADC_CHANNEL_0**

ADC channel.

enumerator **ADC_CHANNEL_1**

ADC channel.

enumerator **ADC_CHANNEL_2**

ADC channel.

enumerator **ADC_CHANNEL_3**

ADC channel.

enumerator **ADC_CHANNEL_4**

ADC channel.

enumerator **ADC_CHANNEL_5**

ADC channel.

enumerator **ADC_CHANNEL_6**

ADC channel.

enumerator **ADC_CHANNEL_7**

ADC channel.

enumerator **ADC_CHANNEL_8**

ADC channel.

enumerator **ADC_CHANNEL_9**

ADC channel.

enum **adc_atten_t**

ADC attenuation parameter. Different parameters determine the range of the ADC.

Values:

enumerator **ADC_ATTEN_DB_0**

No input attenuation, ADC can measure up to approx.

enumerator **ADC_ATTEN_DB_2_5**

The input voltage of ADC will be attenuated extending the range of measurement by about 2.5 dB.

enumerator **ADC_ATTEN_DB_6**

The input voltage of ADC will be attenuated extending the range of measurement by about 6 dB.

enumerator **ADC_ATTEN_DB_12**

The input voltage of ADC will be attenuated extending the range of measurement by about 12 dB.

enumerator **ADC_ATTEN_DB_11**

This is deprecated, it behaves the same as `ADC_ATTEN_DB_12`

enum **adc_bitwidth_t**

Values:

enumerator **ADC_BITWIDTH_DEFAULT**

Default ADC output bits, max supported width will be selected.

enumerator **ADC_BITWIDTH_9**

ADC output width is 9Bit.

enumerator **ADC_BITWIDTH_10**

ADC output width is 10Bit.

enumerator **ADC_BITWIDTH_11**

ADC output width is 11Bit.

enumerator **ADC_BITWIDTH_12**

ADC output width is 12Bit.

enumerator **ADC_BITWIDTH_13**

ADC output width is 13Bit.

enum **adc_ulp_mode_t**

Values:

enumerator **ADC_ULP_MODE_DISABLE**

ADC ULP mode is disabled.

enumerator **ADC_ULP_MODE_FSM**

ADC is controlled by ULP FSM.

enumerator **ADC_ULP_MODE_RISCV**

ADC is controlled by ULP RISCV.

enum **adc_digi_convert_mode_t**

ADC digital controller (DMA mode) work mode.

Values:

enumerator **ADC_CONV_SINGLE_UNIT_1**

Only use ADC1 for conversion.

enumerator **ADC_CONV_SINGLE_UNIT_2**

Only use ADC2 for conversion.

enumerator **ADC_CONV_BOTH_UNIT**

Use Both ADC1 and ADC2 for conversion simultaneously.

enumerator **ADC_CONV_ALTER_UNIT**

Use both ADC1 and ADC2 for conversion by turn. e.g. ADC1 -> ADC2 -> ADC1 -> ADC2 .....

enum **adc_digi_output_format_t**

ADC digital controller (DMA mode) output data format option.

Values:

enumerator **ADC_DIGI_OUTPUT_FORMAT_TYPE1**

See `adc_digi_output_data_t.type1`

enumerator **ADC_DIGI_OUTPUT_FORMAT_TYPE2**

See *adc_digi_output_data_t.type2*

enum **adc_digi_iir_filter_t**

ADC IIR Filter ID.

Values:

enumerator **ADC_DIGI_IIR_FILTER_0**

Filter 0.

enumerator **ADC_DIGI_IIR_FILTER_1**

Filter 1.

enum **adc_digi_iir_filter_coeff_t**

IIR Filter Coefficient.

Values:

enumerator **ADC_DIGI_IIR_FILTER_COEFF_2**

The filter coefficient is 2.

enumerator **ADC_DIGI_IIR_FILTER_COEFF_4**

The filter coefficient is 4.

enumerator **ADC_DIGI_IIR_FILTER_COEFF_8**

The filter coefficient is 8.

enumerator **ADC_DIGI_IIR_FILTER_COEFF_16**

The filter coefficient is 16.

enumerator **ADC_DIGI_IIR_FILTER_COEFF_64**

The filter coefficient is 64.

enum **adc_monitor_id_t**

ADC monitor (continuous mode) ID.

Values:

enumerator **ADC_MONITOR_0**

The monitor index 0.

enumerator **ADC_MONITOR_1**

The monitor index 1.

enum **adc_monitor_mode_t**

Monitor config/event mode type.

Values:

enumerator **ADC_MONITOR_MODE_HIGH**

ADC raw_result > threshold value, monitor interrupt will be generated.

enumerator **ADC_MONITOR_MODE_LOW**

ADC raw_result < threshold value, monitor interrupt will be generated.

Header File

- components/esp_adc/include/esp_adc/adc_oneshot.h
- This header file can be included with:

```
#include "esp_adc/adc_oneshot.h"
```

- This header file is a part of the API provided by the `esp_adc` component. To declare that your component depends on `esp_adc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_adc
```

or

```
PRIV_REQUIRES esp_adc
```

Functions

esp_err_t **adc_oneshot_new_unit**(const *adc_oneshot_unit_init_cfg_t* *init_config, *adc_oneshot_unit_handle_t* *ret_unit)

Create a handle to a specific ADC unit.

Note: This API is thread-safe. For more details, see ADC programming guide

---

Parameters

- **init_config** -- **[in]** Driver initial configurations
- **ret_unit** -- **[out]** ADC unit handle

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_ERR_NO_MEM: No memory
- ESP_ERR_NOT_FOUND: The ADC peripheral to be claimed is already in use
- ESP_FAIL: Clock source isn't initialised correctly

esp_err_t **adc_oneshot_config_channel**(*adc_oneshot_unit_handle_t* handle, *adc_channel_t* channel, const *adc_oneshot_chan_cfg_t* *config)

Set ADC oneshot mode required configurations.

Note: This API is thread-safe. For more details, see ADC programming guide

---

Parameters

- **handle** -- **[in]** ADC handle
- **channel** -- **[in]** ADC channel to be configured
- **config** -- **[in]** ADC configurations

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid arguments

esp_err_t **adc_oneshot_read**(*adc_oneshot_unit_handle_t* handle, *adc_channel_t* chan, int *out_raw)

Get one ADC conversion raw result.

Note: This API is thread-safe. For more details, see ADC programming guide

Note: This API should NOT be called in an ISR context

Parameters
- **handle** -- **[in]** ADC handle
- **chan** -- **[in]** ADC channel
- **out_raw** -- **[out]** ADC conversion raw result

Returns
- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_ERR_TIMEOUT: Timeout, the ADC result is invalid

esp_err_t **adc_oneshot_del_unit**(*adc_oneshot_unit_handle_t* handle)

Delete the ADC unit handle.

Note: This API is thread-safe. For more details, see ADC programming guide

Parameters **handle** -- **[in]** ADC handle

Returns
- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_ERR_NOT_FOUND: The ADC peripheral to be disclaimed isn't in use

esp_err_t **adc_oneshot_io_to_channel**(int io_num, *adc_unit_t* *const unit_id, *adc_channel_t* *const channel)

Get ADC channel from the given GPIO number.

Parameters
- **io_num** -- **[in]** GPIO number
- **unit_id** -- **[out]** ADC unit
- **channel** -- **[out]** ADC channel

Returns
- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_NOT_FOUND: The IO is not a valid ADC pad

esp_err_t **adc_oneshot_channel_to_io**(*adc_unit_t* unit_id, *adc_channel_t* channel, int *const io_num)

Get GPIO number from the given ADC channel.

Parameters
- **unit_id** -- **[in]** ADC unit
- **channel** -- **[in]** ADC channel
- **io_num** -- **[out]** GPIO number
- **-** -- ESP_OK: On success
  - ESP_ERR_INVALID_ARG: Invalid argument

esp_err_t **adc_oneshot_get_calibrated_result**(*adc_oneshot_unit_handle_t* handle, *adc_cali_handle_t* cali_handle, *adc_channel_t* chan, int *cali_result)

Convenience function to get ADC calibrated result.

This is an all-in-one function which does:

- oneshot read ADC raw result

- calibrate the raw result and convert it into calibrated result (in mV)

  Parameters
  - **handle** -- **[in]** ADC oneshot handle, you should call adc_oneshot_new_unit() to get this handle
  - **cali_handle** -- **[in]** ADC calibration handle, you should call adc_cali_create_scheme_x() in adc_cali_scheme.h to create a handle
  - **chan** -- **[in]** ADC channel
  - **cali_result** -- **[out]** Calibrated ADC result (in mV)

  Returns
  - ESP_OK Other return errors from adc_oneshot_read() and adc_cali_raw_to_voltage()

Structures

struct **adc_oneshot_unit_init_cfg_t**

ADC oneshot driver initial configurations.

Public Members

adc_unit_t **unit_id**

ADC unit.

adc_oneshot_clk_src_t **clk_src**

Clock source.

adc_ulp_mode_t **ulp_mode**

ADC controlled by ULP, see `adc_ulp_mode_t`

struct **adc_oneshot_chan_cfg_t**

ADC channel configurations.

Public Members

adc_atten_t **atten**

ADC attenuation.

adc_bitwidth_t **bitwidth**

ADC conversion result bits.

Type Definitions

typedef struct adc_oneshot_unit_ctx_t ***adc_oneshot_unit_handle_t**

Type of ADC unit handle for oneshot mode.

2.6.2 Analog to Digital Converter (ADC) Continuous Mode Driver

Introduction

The Analog to Digital Converter is integrated on the chip and is capable of measuring analog signals from specific analog IO pads. Additionally, the Direct Memory Access (DMA) functionality is utilized to efficiently retrieve ADC conversion results.

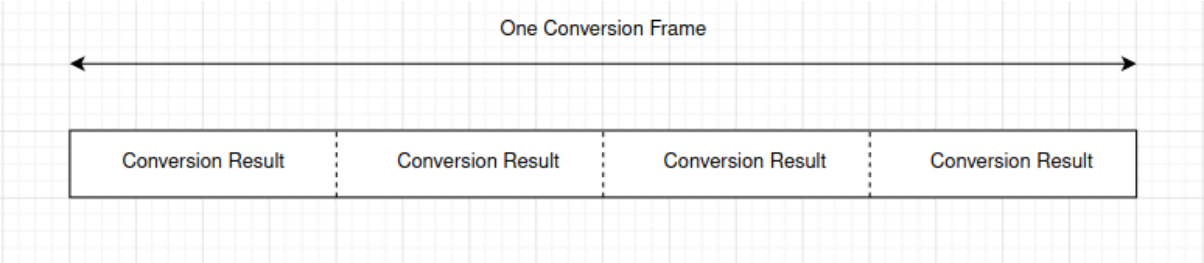
ESP32-S3 has two ADC unit(s), which can be used in scenarios like:

- Generate one-shot ADC conversion result
- Generate continuous ADC conversion results

This guide introduces ADC continuous mode conversion.

Driver Concepts ADC continuous mode conversion is made up of multiple conversion frames.

- Conversion Frame: One conversion frame contains multiple conversion results. Conversion frame size is configured in `adc_continuous_new_handle()` in bytes.
- Conversion Result: One conversion result contains multiple bytes, see `SOC_ADC_DIGI_RESULT_BYTES`. Its structure is `adc_digi_output_data_t`, including ADC unit, ADC channel, and raw data.



Functional Overview

The following sections of this document cover the typical steps to install the ADC continuous mode driver, and read ADC conversion results from a group of ADC channels continuously:

- *Resource Allocation*: covers which parameters should be set up to initialize the ADC continuous mode driver and how to deinitialize it.
- *ADC Configurations*: describes how to configure the ADC(s) to make it work under continuous mode.
- *ADC Control*: describes ADC control functions.
- *Register Event Callbacks*: describes how to hook user-specific code to an ADC continuous mode event callback function.
- *Read Conversion Result*: covers how to get ADC conversion result.
- *Hardware Limitations*: describes the ADC-related hardware limitations.
- *Power Management*: covers power management-related information.
- *IRAM Safe*: covers the IRAM safe functions.
- *Thread Safety*: lists which APIs are guaranteed to be thread-safe by the driver.

Resource Allocation The ADC continuous mode driver is implemented based on ESP32-S3 SAR ADC module. Different ESP targets might have different numbers of independent ADCs.

To create an ADC continuous mode driver handle, set up the required configuration structure `adc_continuous_handle_cfg_t`:

- `adc_continuous_handle_cfg_t::max_store_buf_size`: set the maximum size of the pool in bytes, and the driver saves ADC conversion result into the pool. If this pool is full, new conversion results will be lost.
- `adc_continuous_handle_cfg_t::conv_frame_size`: set the size of the ADC conversion frame, in bytes.
- `adc_continuous_handle_cfg_t::flags`: set the flags that can change the driver's behavior.
  - `flush_pool`: auto flush the pool when it's full.

After setting up the above configurations for the ADC, call *adc_continuous_new_handle()* with the prepared *adc_continuous_handle_cfg_t*. This function may fail due to various errors such as invalid arguments, insufficient memory, etc.

Especially, when this function returns *ESP_ERR_NOT_FOUND*, this means there is no free GDMA channel.

If the ADC continuous mode driver is no longer used, you should deinitialize the driver by calling *adc_continuous_deinit()*.

IIR filter Two IIR filters are available when ADC is working in continuous mode. To create an ADC IIR filter, you should set up `adc_continuous_iir_filter_config_t` and call `adc_new_continuous_iir_filter()`.

- `adc_digi_filter_config_t::unit`: ADC unit.
- `adc_digi_filter_config_t::channel`: ADC channel to be filtered.
- `adc_digi_filter_config_t::coeff`: Filter coefficient.

To recycle a filter, you should call `adc_del_continuous_iir_filter()`.

Note: If you use both filters on the same ADC channel, then only the first one will take effect.

Monitor 2 monitors are available when ADC is working under continuous mode, you can set one or two threshold(s) of a monitor on a working ADC channel, then the monitor will invoke interrupts every sample loop if conversion result outranges of the threshold. To create an ADC monitor, you need to set up the `adc_monitor_config_t` and call `adc_new_continuous_monitor()`.

- `adc_monitor_config_t::adc_unit`: Configures which ADC unit the channel you want to monitor belongs to.
- `adc_monitor_config_t::channel`: The channel you want to monitor.
- `adc_monitor_config_t::h_threshold`: The high threshold, conversion result larger than this value invokes interrupt, set to -1 if do not use.
- `adc_monitor_config_t::l_threshold`: The low threshold, conversion result less than this value invokes interrupt, set to -1 if do not use.

Once a monitor is created, you can operate it by following APIs to construct your apps.

- `adc_continuous_monitor_enable()`: Enable a monitor.
- `adc_continuous_monitor_disable()`: Disable a monitor.
- `adc_monitor_register_callbacks()`: register user callbacks to take action when the ADC value exceeds of the threshold.
- `adc_del_continuous_monitor()`: Delete a created monitor and free resources.

Initialize the ADC Continuous Mode Driver

```
adc_continuous_handle_cfg_t adc_config = {
    .max_store_buf_size = 1024,
    .conv_frame_size = 100,
};
ESP_ERROR_CHECK(adc_continuous_new_handle(&adc_config));
```

Recycle the ADC Unit

```
ESP_ERROR_CHECK(adc_continuous_deinit());
```

ADC Configurations After the ADC continuous mode driver is initialized, set up the *adc_continuous_config_t* to configure ADC IOs to measure analog signal:

- *adc_continuous_config_t::pattern_num*: number of ADC channels that will be used.

- *adc_continuous_config_t::adc_pattern*: list of configs for each ADC channel that will be used, see the description below.
- *adc_continuous_config_t::sample_freq_hz*: expected ADC sampling frequency in Hz.
- *adc_continuous_config_t::conv_mode*: continuous conversion mode.
- *adc_continuous_config_t::format*: conversion output format.

Set *adc_digi_pattern_config_t* with the following process:

- *adc_digi_pattern_config_t::atten*: ADC attenuation. Refer to the On-Chip Sensor and Analog Signal Processing chapter in TRM.
- *adc_digi_pattern_config_t::channel*: the IO corresponding ADC channel number. See the note below.
- *adc_digi_pattern_config_t::unit*: the ADC that the IO is subordinate to.
- *adc_digi_pattern_config_t::bit_width*: the bitwidth of the raw conversion result.

Note: For the IO corresponding ADC channel number, check TRM to acquire the ADC IOs. Besides, *adc_continuous_io_to_channel()* and *adc_continuous_channel_to_io()* can be used to acquire the ADC channels and ADC IOs.

To make these settings take effect, call *adc_continuous_config()* with the configuration structure above. This API may fail due to reasons like *ESP_ERR_INVALID_ARG*. When it returns *ESP_ERR_INVALID_STATE*, this means the ADC continuous mode driver is started, you should not call this API at this moment.

See ADC continuous mode example peripherals/adc/continuous_read to see configuration codes.

To enable/disable the ADC IIR filter, you should call `adc_continuous_iir_filter_enable()` / `adc_continuous_iir_filter_disable()`.

To enable/disable the ADC monitor, you should call `adc_continuous_monitor_enable()` / `adc_continuous_monitor_disable()`.

ADC Control

Start and Stop Calling *adc_continuous_start()* makes the ADC start to measure analog signals from the configured ADC channels, and generate the conversion results.

On the contrary, calling *adc_continuous_stop()* stops the ADC conversion.

```
ESP_ERROR_CHECK(adc_continuous_stop());
```

Register Event Callbacks By calling *adc_continuous_register_event_callbacks()*, you can hook your own function to the driver ISR. Supported event callbacks are listed in *adc_continuous_evt_cbs_t*.

- *adc_continuous_evt_cbs_t::on_conv_done*: this is invoked when one conversion frame finishes.
- *adc_continuous_evt_cbs_t::on_pool_ovf*: this is invoked when the internal pool is full. Newer conversion results will be discarded.

As the above callbacks are called in an ISR context, you should always ensure the callback function is suitable for an ISR context. Blocking logic should not appear in these callbacks. The callback function prototype is declared in *adc_continuous_callback_t*.

You can also register your own context when calling *adc_continuous_register_event_callbacks()* by the parameter `user_data`. This user data will be passed to the callback functions directly.

This function may fail due to reasons like *ESP_ERR_INVALID_ARG*. Especially, when *CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE* is enabled, this error may indicate that the callback functions are not in the internal RAM. Check the error log for more details. Besides, when it fails due to *ESP_ERR_INVALID_STATE*, it indicates that the ADC continuous mode driver is started, and you should not add a callback at this moment.

Conversion Done Event When the driver completes a conversion, it triggers the *adc_continuous_evt_cbs_t::on_conv_done* event and fills the event data. Event data contains a buffer pointer to a conversion frame buffer, together with the size. Refer to *adc_continuous_evt_data_t* to know the event data structure.

Note: It is worth noting that, the data buffer *adc_continuous_evt_data_t::conv_frame_buffer* is maintained by the driver itself. Therefore, never free this piece of memory.

Note: When the Kconfig option *CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE* is enabled, the registered callbacks and the functions called by the callbacks should be placed in IRAM. The involved variables should be placed in internal RAM as well.

Pool Overflow Event The ADC continuous mode driver has an internal pool to save the conversion results. When the pool is full, a pool overflow event will emerge. Under this condition, the driver will not fill in the event data. This usually happens because the speed to read data from the pool by calling *adc_continuous_read()* is much slower than the ADC conversion speed.

Read Conversion Result After calling *adc_continuous_start()*, the ADC continuous conversion starts. Call *adc_continuous_read()* to get the conversion results of the ADC channels. You need to provide a buffer to get the raw results.

Function *adc_continuous_read()* tries to read the expected length of conversion results each time.

- When calling *adc_continuous_read()*, you can request to read a conversion result of the specified length. Sometimes, however, the actual available conversion results may be less than the requested length, in which case the function still moves the data from the internal pool into the buffer you provided. Therefore, to learn the number of conversion results actually moved into the buffer, please check the value of `out_length`.
- If there is no conversion result generated in the internal pool, the function will block for `timeout_ms` until the conversion results are generated. If there are still no generated results, the function will return *ESP_ERR_TIMEOUT*.
- If the generated results fill up the internal pool, newly generated results will be lost. Next time when *adc_continuous_read()* is called, this function will return *ESP_ERR_INVALID_STATE* to indicate this situation.

This API aims to give you a chance to read all the ADC continuous conversion results.

The ADC conversion results read from the above function are raw data. To calculate the voltage based on the ADC raw results, this formula can be used:

```
Vout = Dout * Vmax / Dmax        (1)
```

where:

Vout	Digital output result, standing for the voltage.
Dout	ADC raw digital reading result.
Vmax	Maximum measurable input analog voltage, this is related to the ADC attenuation, please refer to the On-Chip Sensor and Analog Signal Processing chapter in TRM.
Dmax	Maximum of the output ADC raw digital reading result, which is 2^bitwidth, where the bitwidth is the *adc_digi_pattern_config_t::bit_width* configured before.

To do further calibration to convert the ADC raw result to voltage in mV, please refer to *Analog to Digital Converter (ADC) Calibration Driver*.

Hardware Limitations

- A specific ADC unit can only work under one operating mode at any one time, either continuous mode or one-shot mode. *adc_continuous_start()* has provided the protection.
- Random Number Generator (RNG) uses ADC as an input source. When ADC continuous mode driver works, the random number generated from RNG will be less random.
- ADC2 DMA functionality is no longer supported to retrieve ADC conversion results due to hardware limitations, as unstable results have been observed. This issue can be found in ESP32S3 Errata. For compatibility, you can enable *CONFIG_ADC_CONTINUOUS_FORCE_USE_ADC2_ON_C3_S3* to force use ADC2.

Power Management When power management is enabled, i.e., *CONFIG_PM_ENABLE* is on, the APB clock frequency may be adjusted when the system is in an idle state, thus potentially changing the behavior of ADC continuous conversion.

However, the continuous mode driver can prevent this change by acquiring a power management lock of type *ESP_PM_APB_FREQ_MAX*. The lock is acquired after the continuous conversion is started by *adc_continuous_start()*. Similarly, the lock will be released after *adc_continuous_stop()*. Therefore, *adc_continuous_start()* and *adc_continuous_stop()* should appear in pairs, otherwise, the power management will be out of action.

IRAM Safe All the ADC continuous mode driver APIs are not IRAM-safe. They are not supposed to be run when the Cache is disabled. By enabling the Kconfig option *CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE*, the driver's internal ISR handler is IRAM-safe, which means even when the Cache is disabled, the driver will still save the conversion results into its internal pool.

Thread Safety ADC continuous mode driver APIs are not guaranteed to be thread-safe. However, the share hardware mutual exclusion is provided by the driver. See *Hardware Limitations* for more details.

Application Examples

- ADC continuous mode example: peripherals/adc/continuous_read.

API Reference

Header File

- components/esp_adc/include/esp_adc/adc_continuous.h
- This header file can be included with:

```
#include "esp_adc/adc_continuous.h"
```

- This header file is a part of the API provided by the `esp_adc` component. To declare that your component depends on `esp_adc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_adc
```

or

```
PRIV_REQUIRES esp_adc
```

Functions

esp_err_t **adc_continuous_new_handle**(const *adc_continuous_handle_cfg_t* *hdl_config, *adc_continuous_handle_t* *ret_handle)

Initialize ADC continuous driver and get a handle to it.

Parameters

- **hdl_config** -- **[in]** Pointer to ADC initialization config. Refer to *adc_continuous_handle_cfg_t*.
- **ret_handle** -- **[out]** ADC continuous mode driver handle

Returns

- ESP_ERR_INVALID_ARG If the combination of arguments is invalid.
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags
- ESP_ERR_NO_MEM If out of memory
- ESP_OK On success

esp_err_t **adc_continuous_config**(*adc_continuous_handle_t* handle, const *adc_continuous_config_t* *config)

Set ADC continuous mode required configurations.

Parameters

- **handle** -- **[in]** ADC continuous mode driver handle
- **config** -- **[in]** Refer to `adc_digi_config_t`.

Returns

- ESP_ERR_INVALID_STATE: Driver state is invalid, you shouldn't call this API at this moment
- ESP_ERR_INVALID_ARG: If the combination of arguments is invalid.
- ESP_OK: On success

esp_err_t **adc_continuous_register_event_callbacks**(*adc_continuous_handle_t* handle, const *adc_continuous_evt_cbs_t* *cbs, void *user_data)

Register callbacks.

Note: User can deregister a previously registered callback by calling this function and setting the to-be-deregistered callback member in the `cbs` structure to NULL.

Note: When CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. Involved variables (including `user_data`) should be in internal RAM as well.

Note: You should only call this API when the ADC continuous mode driver isn't started. Check return value to know this.

Parameters

- **handle** -- **[in]** ADC continuous mode driver handle
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be delivered to the callback functions directly

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_ERR_INVALID_STATE: Driver state is invalid, you shouldn't call this API at this moment

esp_err_t **adc_continuous_start**(*adc_continuous_handle_t* handle)

Start the ADC under continuous mode. After this, the hardware starts working.

Parameters **handle** -- **[in]** ADC continuous mode driver handle

Returns

- ESP_ERR_INVALID_STATE Driver state is invalid.
- ESP_OK On success

esp_err_t **adc_continuous_read**(*adc_continuous_handle_t* handle, uint8_t *buf, uint32_t length_max, uint32_t *out_length, uint32_t timeout_ms)

Read bytes from ADC under continuous mode.

Parameters

- **handle** -- **[in]** ADC continuous mode driver handle
- **buf** -- **[out]** Conversion result buffer to read from ADC. Suggest convert to *adc_digi_output_data_t* for `ADC Conversion Results`. See the subsection `Driver Backgrounds` in this header file to learn about this concept.
- **length_max** -- **[in]** Expected length of the Conversion Results read from the ADC, in bytes.
- **out_length** -- **[out]** Real length of the Conversion Results read from the ADC via this API, in bytes.
- **timeout_ms** -- **[in]** Time to wait for data via this API, in millisecond.

Returns

- ESP_ERR_INVALID_STATE Driver state is invalid. Usually it means the ADC sampling rate is faster than the task processing rate.
- ESP_ERR_TIMEOUT Operation timed out
- ESP_OK On success

esp_err_t **adc_continuous_stop**(*adc_continuous_handle_t* handle)

Stop the ADC. After this, the hardware stops working.

Parameters **handle** -- **[in]** ADC continuous mode driver handle

Returns

- ESP_ERR_INVALID_STATE Driver state is invalid.
- ESP_OK On success

esp_err_t **adc_continuous_deinit**(*adc_continuous_handle_t* handle)

Deinitialize the ADC continuous driver.

Parameters **handle** -- **[in]** ADC continuous mode driver handle

Returns

- ESP_ERR_INVALID_STATE Driver state is invalid.
- ESP_OK On success

esp_err_t **adc_continuous_flush_pool**(*adc_continuous_handle_t* handle)

Flush the driver internal pool.

Note: This API is not supposed to be called in an ISR context

---

Parameters **handle** -- **[in]** ADC continuous mode driver handle

Returns

- ESP_ERR_INVALID_STATE Driver state is invalid, you should call this API when it's in init state
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_OK On success

esp_err_t **adc_continuous_io_to_channel**(int io_num, *adc_unit_t* *const unit_id, *adc_channel_t* *const channel)

Get ADC channel from the given GPIO number.

Parameters

- **io_num** -- **[in]** GPIO number
- **unit_id** -- **[out]** ADC unit
- **channel** -- **[out]** ADC channel

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument

- ESP_ERR_NOT_FOUND: The IO is not a valid ADC pad

esp_err_t **adc_continuous_channel_to_io**(*adc_unit_t* unit_id, *adc_channel_t* channel, int *const io_num)

Get GPIO number from the given ADC channel.

Parameters

- **unit_id** -- **[in]** ADC unit
- **channel** -- **[in]** ADC channel
- **io_num** -- **[out]** GPIO number
- **–** -- ESP_OK: On success
  - ESP_ERR_INVALID_ARG: Invalid argument

Structures

struct **adc_continuous_handle_cfg_t**

ADC continuous mode driver initial configurations.

Public Members

uint32_t **max_store_buf_size**

Max length of the conversion results that driver can store, in bytes.

uint32_t **conv_frame_size**

Conversion frame size, in bytes. This should be in multiples of `SOC_ADC_DIGI_DATA_BYTES_PER_CONV`.

uint32_t **flush_pool**

Flush the internal pool when the pool is full.

struct *adc_continuous_handle_cfg_t*::[anonymous] **flags**

Driver flags.

struct **adc_continuous_config_t**

ADC continuous mode driver configurations.

Public Members

uint32_t **pattern_num**

Number of ADC channels that will be used.

adc_digi_pattern_config_t ***adc_pattern**

List of configs for each ADC channel that will be used.

uint32_t **sample_freq_hz**

The expected ADC sampling frequency in Hz. Please refer to `soc/soc_caps.h` to know available sampling frequency range

adc_digi_convert_mode_t **conv_mode**

ADC DMA conversion mode, see `adc_digi_convert_mode_t`.

adc_digi_output_format_t **format**

ADC DMA conversion output format, see `adc_digi_output_format_t`.

struct **adc_continuous_evt_data_t**

Event data structure.

Note: The `conv_frame_buffer` is maintained by the driver itself, so never free this piece of memory.

Public Members

uint8_t ***conv_frame_buffer**

Pointer to conversion result buffer for one conversion frame.

uint32_t **size**

Conversion frame size.

struct **adc_continuous_evt_cbs_t**

Group of ADC continuous mode callbacks.

Note: These callbacks are all running in an ISR environment.

Note: When CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. Involved variables should be in internal RAM as well.

Public Members

adc_continuous_callback_t **on_conv_done**

Event callback, invoked when one conversion frame is done. See the subsection `Driver Backgrounds` in this header file to learn about the `conversion frame` concept.

adc_continuous_callback_t **on_pool_ovf**

Event callback, invoked when the internal pool is full.

Macros

ADC_MAX_DELAY

ADC read max timeout value, it may make the `adc_continuous_read` block forever if the OS supports.

Type Definitions

typedef struct adc_continuous_ctx_t ***adc_continuous_handle_t**

Type of adc continuous mode driver handle.

typedef bool (***adc_continuous_callback_t**)(*adc_continuous_handle_t* handle, const *adc_continuous_evt_data_t* *edata, void *user_data)

Prototype of ADC continuous mode event callback.

Param handle [in] ADC continuous mode driver handle
Param edata [in] Pointer to ADC continuous mode event data
Param user_data [in] User registered context, registered when in `adc_continuous_register_event_callbacks()`
Return Whether a high priority task is woken up by this function

2.6.3 Analog to Digital Converter (ADC) Calibration Driver

Introduction

In ESP32-S3, the digital-to-analog converter (ADC) compares the input analog voltage to the reference, and determines each bit of the output digital result. By design, the ADC reference voltage for ESP32-S3 is 1100 mV. However, the true reference voltage can range from 1000 mV to 1200 mV among different chips. This guide introduces the ADC calibration driver to minimize the effect of different reference voltages, and get more accurate output results.

Functional Overview

The following sections of this document cover the typical steps to install and use the ADC calibration driver:

- *Calibration Scheme Creation* - covers how to create a calibration scheme handle and delete the calibration scheme handle.
- *Result Conversion* - covers how to convert ADC raw result to calibrated result.
- *Thread Safety* - lists which APIs are guaranteed to be thread-safe by the driver.
- *Minimize Noise* - describes a general way to minimize the noise.

Calibration Scheme Creation The ADC calibration driver provides ADC calibration scheme(s). From the calibration driver's point of view, an ADC calibration scheme is created for an ADC calibration handle `adc_cali_handle_t`.

`adc_cali_check_scheme()` can be used to know which calibration scheme is supported on the chip. If you already know the supported schemes, this step can be skipped. Just call the corresponding function to create the scheme handle.

If you use your custom ADC calibration schemes, you could either modify this function `adc_cali_check_scheme()`, or just skip this step and call your custom creation function.

ADC Calibration Curve Fitting Scheme ESP32-S3 supports `ADC_CALI_SCHEME_VER_CURVE_FITTING` scheme. To create this scheme, set up `adc_cali_curve_fitting_config_t` first.

- `adc_cali_curve_fitting_config_t::unit_id`, the ADC that your ADC raw results are from.
- `adc_cali_curve_fitting_config_t::chan`, this member is kept here for extensibility. The calibration scheme only differs by attenuation, there is no difference among different channels.
- `adc_cali_curve_fitting_config_t::atten`, ADC attenuation that your ADC raw results use.
- `adc_cali_curve_fitting_config_t::bitwidth`, bit width of ADC raw result.

After setting up the configuration structure, call `adc_cali_create_scheme_curve_fitting()` to create a Curve Fitting calibration scheme handle. This function may fail due to reasons such as `ESP_ERR_INVALID_ARG` or `ESP_ERR_NO_MEM`.

ADC Calibration eFuse Related Failures When the function `adc_cali_create_scheme_curve_fitting()` returns *ESP_ERR_NOT_SUPPORTED*, this means the calibration scheme required eFuse bits are not correct on your board.

The ADC calibration scheme provided by ESP-IDF is based on the values in certain ADC calibration related on-chip eFuse bits. Espressif guarantees that these bits are burned during module manufacturing, so you don't have to burn these eFuses bits yourself.

If you see such an error, please contact us at Technical Inquiries website.

Create Curve Fitting Scheme

```
ESP_LOGI(TAG, "calibration scheme version is %s", "Curve Fitting");
adc_cali_curve_fitting_config_t cali_config = {
    .unit_id = unit,
    .atten = atten,
    .bitwidth = ADC_BITWIDTH_DEFAULT,
};
ESP_ERROR_CHECK(adc_cali_create_scheme_curve_fitting(&cali_config, &handle));
```

When the ADC calibration is no longer used, please delete the calibration scheme driver from the calibration handle by calling `adc_cali_delete_scheme_curve_fitting()`.

Delete Curve Fitting Scheme

```
ESP_LOGI(TAG, "delete %s calibration scheme", "Curve Fitting");
ESP_ERROR_CHECK(adc_cali_delete_scheme_curve_fitting(handle));
```

Note: If you want to use your custom calibration schemes, you could provide a creation function to create your calibration scheme handle. Check the function table `adc_cali_scheme_t` in `components/esp_adc/interface/adc_cali_interface.h` to know the ESP ADC calibration interface.

Result Conversion After setting up the calibration characteristics, you can call *adc_cali_raw_to_voltage()* to convert the ADC raw result into calibrated result. The calibrated result is in the unit of mV. This function may fail due to an invalid argument. Especially, if this function returns *ESP_ERR_INVALID_STATE*, this means the calibration scheme is not created. You need to create a calibration scheme handle, use *adc_cali_check_scheme()* to know the supported calibration scheme. On the other hand, you could also provide a custom calibration scheme and create the handle.

Get Voltage

```
ESP_ERROR_CHECK(adc_cali_raw_to_voltage(adc_cali_handle, adc_raw[0][0], &
↪voltage[0][0]));
ESP_LOGI(TAG, "ADC%d Channel[%d] Cali Voltage: %d mV", ADC_UNIT_1 + 1, EXAMPLE_
↪ADC1_CHAN0, voltage[0][0]);
```

Thread Safety The factory function `esp_adc_cali_new_scheme()` is guaranteed to be thread-safe by the driver. Therefore, you can call them from different RTOS tasks without protection by extra locks.

Other functions that take the *adc_cali_handle_t* as the first positional parameter are not thread-safe, you should avoid calling them from multiple tasks.

Minimize Noise The ESP32-S3 ADC is sensitive to noise, leading to large discrepancies in ADC readings. Depending on the usage scenario, you may need to connect a bypass capacitor (e.g., a 100 nF ceramic capacitor) to the ADC input pad in use, to minimize noise. Besides, multisampling may also be used to further mitigate the effects of noise.

API Reference

Header File

- components/esp_adc/include/esp_adc/adc_cali.h
- This header file can be included with:

```
#include "esp_adc/adc_cali.h"
```

- This header file is a part of the API provided by the `esp_adc` component. To declare that your component depends on `esp_adc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_adc
```

or

```
PRIV_REQUIRES esp_adc
```

Functions

esp_err_t **adc_cali_check_scheme**(*adc_cali_scheme_ver_t* *scheme_mask)

Check the supported ADC calibration scheme.

Parameters **scheme_mask** -- **[out]** Supported ADC calibration scheme(s)

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_NOT_SUPPORTED: No supported calibration scheme

esp_err_t **adc_cali_raw_to_voltage**(*adc_cali_handle_t* handle, int raw, int *voltage)

Convert ADC raw data to calibrated voltage.

Parameters

- **handle** -- **[in]** ADC calibration handle
- **raw** -- **[in]** ADC raw data
- **voltage** -- **[out]** Calibrated ADC voltage (in mV)

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_INVALID_STATE: Invalid state, scheme didn't registered

Type Definitions

typedef struct adc_cali_scheme_t ***adc_cali_handle_t**

ADC calibration handle.

Enumerations

enum **adc_cali_scheme_ver_t**

ADC calibration scheme.

Values:

enumerator **ADC_CALI_SCHEME_VER_LINE_FITTING**

Line fitting scheme.

enumerator **ADC_CALI_SCHEME_VER_CURVE_FITTING**

Curve fitting scheme.

Header File

- components/esp_adc/include/esp_adc/adc_cali_scheme.h
- This header file can be included with:

```
#include "esp_adc/adc_cali_scheme.h"
```

- This header file is a part of the API provided by the `esp_adc` component. To declare that your component depends on `esp_adc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_adc
```

or

```
PRIV_REQUIRES esp_adc
```

2.6.4 Clock Tree

The clock subsystem of ESP32-S3 is used to source and distribute system/module clocks from a range of root clocks. The clock tree driver maintains the basic functionality of the system clock and the intricate relationship among module clocks.

This document starts with the introduction to root and module clocks. Then it covers the clock tree APIs that can be called to monitor the status of the module clocks at runtime.

Introduction

This section lists definitions of ESP32-S3's supported root clocks and module clocks. These definitions are commonly used in the driver configuration, to help select a proper source clock for the peripheral.

Root Clocks Root clocks generate reliable clock signals. These clock signals then pass through various gates, muxes, dividers, or multipliers to become the clock sources for every functional module: the CPU core(s), Wi-Fi, Bluetooth, the RTC, and the peripherals.

ESP32-S3's root clocks are listed in *soc_root_clk_t*:

- Internal 17.5 MHz RC Oscillator (RC_FAST)
  This RC oscillator generates a about 17.5 MHz clock signal output as the `RC_FAST_CLK`.
  The about 17.5 MHz signal output is also passed into a configurable divider, which by default divides the input clock frequency by 256, to generate a `RC_FAST_D256_CLK`.
  The exact frequency of `RC_FAST_CLK` can be computed in runtime through calibration on the `RC_FAST_D256_CLK`.
- External 40 MHz Crystal (XTAL)
- Internal 136 kHz RC Oscillator (RC_SLOW)
  This RC oscillator generates a about 136kHz clock signal output as the `RC_SLOW_CLK`.
  The exact frequency of this clock can be computed in runtime through calibration.
- External 32 kHz Crystal - optional (XTAL32K)
  The clock source for this `XTAL32K_CLK` can be either a 32 kHz crystal connecting to the `XTAL_32K_P` and `XTAL_32K_N` pins or a 32 kHz clock signal generated by an external circuit. The external signal must be connected to the `XTAL_32K_P` pin.
  `XTAL32K_CLK` can also be calibrated to get its exact frequency.

Typically, the frequency of the signal generated from an RC oscillator circuit is less accurate and more sensitive to the environment compared to the signal generated from a crystal. ESP32-S3 provides several clock source options

for the RTC_SLOW_CLK, and it is possible to make the choice based on the requirements for system time accuracy and power consumption. For more details, please refer to *RTC Timer Clock Sources*.

Module Clocks ESP32-S3's available module clocks are listed in *soc_module_clk_t*. Each module clock has a unique ID. You can get more information on each clock by checking the documented enum value.

API Usage

The clock tree driver provides an all-in-one API to get the frequency of the module clocks, *esp_clk_tree_src_get_freq_hz()*. This function allows you to obtain the clock frequency at any time by providing the clock name *soc_module_clk_t* and specifying the desired precision level for the returned frequency value *esp_clk_tree_src_freq_precision_t*.

API Reference

Header File

- components/soc/esp32s3/include/soc/clk_tree_defs.h
- This header file can be included with:

```
#include "soc/clk_tree_defs.h"
```

Macros

SOC_CLK_RC_FAST_FREQ_APPROX

Approximate RC_FAST_CLK frequency in Hz

SOC_CLK_RC_SLOW_FREQ_APPROX

Approximate RC_SLOW_CLK frequency in Hz

SOC_CLK_RC_FAST_D256_FREQ_APPROX

Approximate RC_FAST_D256_CLK frequency in Hz

SOC_CLK_XTAL32K_FREQ_APPROX

Approximate XTAL32K_CLK frequency in Hz

SOC_GPTIMER_CLKS

Array initializer for all supported clock sources of GPTimer.

The following code can be used to iterate all possible clocks:

```
soc_periph_gptimer_clk_src_t gptimer_clks[] = (soc_periph_gptimer_clk_src_
↪t)SOC_GPTIMER_CLKS;
for (size_t i = 0; i< sizeof(gptimer_clks) / sizeof(gptimer_clks[0]); i++) {
    soc_periph_gptimer_clk_src_t clk = gptimer_clks[i];
// Test GPTimer with the clock `clk`
}
```

SOC_LCD_CLKS

Array initializer for all supported clock sources of LCD.

SOC_RMT_CLKS

Array initializer for all supported clock sources of RMT.

SOC_TEMP_SENSOR_CLKS

Array initializer for all supported clock sources of Temperature Sensor.

SOC_UART_CLKS

Array initializer for all supported clock sources of UART.

SOC_MCPWM_TIMER_CLKS

Array initializer for all supported clock sources of MCPWM Timer.

SOC_MCPWM_CAPTURE_CLKS

Array initializer for all supported clock sources of MCPWM Capture Timer.

SOC_MCPWM_CARRIER_CLKS

Array initializer for all supported clock sources of MCPWM Carrier.

SOC_I2S_CLKS

Array initializer for all supported clock sources of I2S.

SOC_I2C_CLKS

Array initializer for all supported clock sources of I2C.

SOC_SPI_CLKS

Array initializer for all supported clock sources of SPI.

SOC_SDM_CLKS

Array initializer for all supported clock sources of SDM.

SOC_GLITCH_FILTER_CLKS

Array initializer for all supported clock sources of Glitch Filter.

SOC_TWAI_CLKS

Array initializer for all supported clock sources of TWAI.

SOC_ADC_DIGI_CLKS

Array initializer for all supported clock sources of ADC digital controller.

SOC_ADC_RTC_CLKS

Array initializer for all supported clock sources of ADC RTC controller.

SOC_MWDT_CLKS

Array initializer for all supported clock sources of MWDT.

SOC_LEDC_CLKS

Array initializer for all supported clock sources of LEDC.

SOC_SDMMC_CLKS

Array initializer for all supported clock sources of SDMMC.

Enumerations

enum **soc_root_clk_t**

Root clock.

Values:

enumerator **SOC_ROOT_CLK_INT_RC_FAST**

Internal 17.5MHz RC oscillator

enumerator **SOC_ROOT_CLK_INT_RC_SLOW**

Internal 136kHz RC oscillator

enumerator **SOC_ROOT_CLK_EXT_XTAL**

External 40MHz crystal

enumerator **SOC_ROOT_CLK_EXT_XTAL32K**

External 32kHz crystal/clock signal

enum **soc_cpu_clk_src_t**

CPU_CLK mux inputs, which are the supported clock sources for the CPU_CLK.

Note: Enum values are matched with the register field values on purpose

Values:

enumerator **SOC_CPU_CLK_SRC_XTAL**

Select XTAL_CLK as CPU_CLK source

enumerator **SOC_CPU_CLK_SRC_PLL**

Select PLL_CLK as CPU_CLK source (PLL_CLK is the output of 40MHz crystal oscillator frequency multiplier, can be 480MHz or 320MHz)

enumerator **SOC_CPU_CLK_SRC_RC_FAST**

Select RC_FAST_CLK as CPU_CLK source

enumerator **SOC_CPU_CLK_SRC_INVALID**

Invalid CPU_CLK source

enum **soc_rtc_slow_clk_src_t**

RTC_SLOW_CLK mux inputs, which are the supported clock sources for the RTC_SLOW_CLK.

Note: Enum values are matched with the register field values on purpose

Values:

enumerator **SOC_RTC_SLOW_CLK_SRC_RC_SLOW**

Select RC_SLOW_CLK as RTC_SLOW_CLK source

enumerator **SOC_RTC_SLOW_CLK_SRC_XTAL32K**

Select XTAL32K_CLK as RTC_SLOW_CLK source

enumerator **SOC_RTC_SLOW_CLK_SRC_RC_FAST_D256**

Select RC_FAST_D256_CLK (referred as FOSC_DIV or 8m_d256/8md256 in TRM and reg. description) as RTC_SLOW_CLK source

enumerator **SOC_RTC_SLOW_CLK_SRC_INVALID**

Invalid RTC_SLOW_CLK source

enum **soc_rtc_fast_clk_src_t**

RTC_FAST_CLK mux inputs, which are the supported clock sources for the RTC_FAST_CLK.

Note: Enum values are matched with the register field values on purpose

Values:

enumerator **SOC_RTC_FAST_CLK_SRC_XTAL_D2**

Select XTAL_D2_CLK (may referred as XTAL_CLK_DIV_2) as RTC_FAST_CLK source

enumerator **SOC_RTC_FAST_CLK_SRC_XTAL_DIV**

Alias name for `SOC_RTC_FAST_CLK_SRC_XTAL_D2`

enumerator **SOC_RTC_FAST_CLK_SRC_RC_FAST**

Select RC_FAST_CLK as RTC_FAST_CLK source

enumerator **SOC_RTC_FAST_CLK_SRC_INVALID**

Invalid RTC_FAST_CLK source

enum **soc_xtal_freq_t**

Possible main XTAL frequency options on the target.

Note: Enum values equal to the frequency value in MHz

Note: Not all frequency values listed here are supported in IDF. Please check SOC_XTAL_SUPPORT_XXX in soc_caps.h for the supported ones.

Values:

enumerator **SOC_XTAL_FREQ_32M**

32MHz XTAL

enumerator **SOC_XTAL_FREQ_40M**

40MHz XTAL

enum **soc_module_clk_t**

Supported clock sources for modules (CPU, peripherals, RTC, etc.)

Note: enum starts from 1, to save 0 for special purpose

Values:

enumerator **SOC_MOD_CLK_CPU**

CPU_CLK can be sourced from XTAL, PLL, or RC_FAST by configuring soc_cpu_clk_src_t

enumerator **SOC_MOD_CLK_RTC_FAST**

RTC_FAST_CLK can be sourced from XTAL_D2 or RC_FAST by configuring soc_rtc_fast_clk_src_t

enumerator **SOC_MOD_CLK_RTC_SLOW**

RTC_SLOW_CLK can be sourced from RC_SLOW, XTAL32K, or RC_FAST_D256 by configuring soc_rtc_slow_clk_src_t

enumerator **SOC_MOD_CLK_APB**

APB_CLK is highly dependent on the CPU_CLK source

enumerator **SOC_MOD_CLK_PLL_F80M**

PLL_F80M_CLK is derived from PLL, and has a fixed frequency of 80MHz

enumerator **SOC_MOD_CLK_PLL_F160M**

PLL_F160M_CLK is derived from PLL, and has a fixed frequency of 160MHz

enumerator **SOC_MOD_CLK_PLL_D2**

PLL_D2_CLK is derived from PLL, it has a fixed divider of 2

enumerator **SOC_MOD_CLK_XTAL32K**

XTAL32K_CLK comes from the external 32kHz crystal, passing a clock gating to the peripherals

enumerator **SOC_MOD_CLK_RC_FAST**

RC_FAST_CLK comes from the internal 20MHz rc oscillator, passing a clock gating to the peripherals

enumerator **SOC_MOD_CLK_RC_FAST_D256**

RC_FAST_D256_CLK comes from the internal 20MHz rc oscillator, divided by 256, and passing a clock gating to the peripherals

enumerator **SOC_MOD_CLK_XTAL**

XTAL_CLK comes from the external 40MHz crystal

enumerator **SOC_MOD_CLK_TEMP_SENSOR**

TEMP_SENSOR_CLK comes directly from the internal 20MHz rc oscillator

enumerator **SOC_MOD_CLK_INVALID**

Indication of the end of the available module clock sources

enum **soc_periph_systimer_clk_src_t**

Type of SYSTIMER clock source.

Values:

enumerator **SYSTIMER_CLK_SRC_XTAL**

SYSTIMER source clock is XTAL

enumerator **SYSTIMER_CLK_SRC_DEFAULT**

SYSTIMER source clock default choice is XTAL

enum **soc_periph_gptimer_clk_src_t**

Type of GPTimer clock source.

Values:

enumerator **GPTIMER_CLK_SRC_APB**

Select APB as the source clock

enumerator **GPTIMER_CLK_SRC_XTAL**

Select XTAL as the source clock

enumerator **GPTIMER_CLK_SRC_DEFAULT**

Select APB as the default choice

enum **soc_periph_tg_clk_src_legacy_t**

Type of Timer Group clock source, reserved for the legacy timer group driver.

Values:

enumerator **TIMER_SRC_CLK_APB**

Timer group source clock is APB

enumerator **TIMER_SRC_CLK_XTAL**

Timer group source clock is XTAL

enumerator **TIMER_SRC_CLK_DEFAULT**

Timer group source clock default choice is APB

enum **soc_periph_lcd_clk_src_t**

Type of LCD clock source.

Values:

enumerator **LCD_CLK_SRC_PLL160M**

Select PLL_F160M as the source clock

enumerator **LCD_CLK_SRC_PLL240M**

Select PLL_D2 as the source clock

enumerator **LCD_CLK_SRC_XTAL**

Select XTAL as the source clock

enumerator **LCD_CLK_SRC_DEFAULT**

Select PLL_F160M as the default choice

enum **soc_periph_rmt_clk_src_t**

Type of RMT clock source.

Values:

enumerator **RMT_CLK_SRC_APB**

Select APB as the source clock

enumerator **RMT_CLK_SRC_RC_FAST**

Select RC_FAST as the source clock

enumerator **RMT_CLK_SRC_XTAL**

Select XTAL as the source clock

enumerator **RMT_CLK_SRC_DEFAULT**

Select APB as the default choice

enum **soc_periph_rmt_clk_src_legacy_t**

Type of RMT clock source, reserved for the legacy RMT driver.

Values:

enumerator **RMT_BASECLK_APB**

RMT source clock is APB

enumerator **RMT_BASECLK_XTAL**

RMT source clock is XTAL

enumerator **RMT_BASECLK_DEFAULT**

RMT source clock default choice is APB

enum **soc_periph_temperature_sensor_clk_src_t**

Type of Temp Sensor clock source.

Values:

enumerator **TEMPERATURE_SENSOR_CLK_SRC_RC_FAST**

Select RC_FAST as the source clock

enumerator **TEMPERATURE_SENSOR_CLK_SRC_DEFAULT**

Select RC_FAST as the default choice

enum **soc_periph_uart_clk_src_legacy_t**

Type of UART clock source, reserved for the legacy UART driver.

Values:

enumerator **UART_SCLK_APB**

UART source clock is APB CLK

enumerator **UART_SCLK_RTC**

UART source clock is RC_FAST

enumerator **UART_SCLK_XTAL**

UART source clock is XTAL

enumerator **UART_SCLK_DEFAULT**

UART source clock default choice is APB

enum **soc_periph_mcpwm_timer_clk_src_t**

Type of MCPWM timer clock source.

Values:

enumerator **MCPWM_TIMER_CLK_SRC_PLL160M**

Select PLL_F160M as the source clock

enumerator **MCPWM_TIMER_CLK_SRC_DEFAULT**

Select PLL_F160M as the default clock choice

enum **soc_periph_mcpwm_capture_clk_src_t**

Type of MCPWM capture clock source.

Values:

enumerator **MCPWM_CAPTURE_CLK_SRC_APB**

Select APB as the source clock

enumerator **MCPWM_CAPTURE_CLK_SRC_DEFAULT**

Select APB as the default clock choice

enum **soc_periph_mcpwm_carrier_clk_src_t**

Type of MCPWM carrier clock source.

Values:

enumerator **MCPWM_CARRIER_CLK_SRC_PLL160M**

Select PLL_F160M as the source clock

enumerator **MCPWM_CARRIER_CLK_SRC_DEFAULT**

Select PLL_F160M as the default clock choice

enum **soc_periph_i2s_clk_src_t**

I2S clock source enum.

Values:

enumerator **I2S_CLK_SRC_DEFAULT**

Select PLL_F160M as the default source clock

enumerator **I2S_CLK_SRC_PLL_160M**

Select PLL_F160M as the source clock

enumerator **I2S_CLK_SRC_XTAL**

Select XTAL as the source clock

enumerator **I2S_CLK_SRC_EXTERNAL**

Select external clock as source clock

enum **soc_periph_i2c_clk_src_t**

Type of I2C clock source.

Values:

enumerator **I2C_CLK_SRC_XTAL**

enumerator **I2C_CLK_SRC_RC_FAST**

enumerator **I2C_CLK_SRC_DEFAULT**

enum **soc_periph_spi_clk_src_t**

Type of SPI clock source.

Values:

enumerator **SPI_CLK_SRC_DEFAULT**

Select APB as SPI source clock

enumerator **SPI_CLK_SRC_APB**

Select APB as SPI source clock

enumerator **SPI_CLK_SRC_XTAL**

Select XTAL as SPI source clock

enum **soc_periph_sdm_clk_src_t**

Sigma Delta Modulator clock source.

Values:

enumerator **SDM_CLK_SRC_APB**

Select APB as the source clock

enumerator **SDM_CLK_SRC_DEFAULT**

Select APB as the default clock choice

enum **soc_periph_glitch_filter_clk_src_t**

Glitch filter clock source.

Values:

enumerator **GLITCH_FILTER_CLK_SRC_APB**

Select APB clock as the source clock

enumerator **GLITCH_FILTER_CLK_SRC_DEFAULT**

Select APB clock as the default clock choice

enum **soc_periph_twai_clk_src_t**

TWAI clock source.

Values:

enumerator **TWAI_CLK_SRC_APB**

Select APB as the source clock

enumerator **TWAI_CLK_SRC_DEFAULT**

Select APB as the default clock choice

enum **soc_periph_adc_digi_clk_src_t**

ADC digital controller clock source.

Values:

enumerator **ADC_DIGI_CLK_SRC_APB**

Select APB as the source clock

enumerator **ADC_DIGI_CLK_SRC_PLL_F240M**

Select PLL_D2 (default value PLL_F240M) as the source clock

enumerator **ADC_DIGI_CLK_SRC_DEFAULT**

Select APB as the default clock choice

enum **soc_periph_adc_rtc_clk_src_t**

ADC RTC controller clock source.

Values:

enumerator **ADC_RTC_CLK_SRC_RC_FAST**

Select RC_FAST as the source clock

enumerator **ADC_RTC_CLK_SRC_DEFAULT**

Select RC_FAST as the default clock choice

enum **soc_periph_mwdt_clk_src_t**

MWDT clock source.

Values:

enumerator **MWDT_CLK_SRC_APB**

Select APB as the source clock

enumerator **MWDT_CLK_SRC_DEFAULT**

Select APB as the default clock choice

enum **soc_periph_ledc_clk_src_legacy_t**

Type of LEDC clock source, reserved for the legacy LEDC driver.

Values:

enumerator **LEDC_AUTO_CLK**

LEDC source clock will be automatically selected based on the giving resolution and duty parameter when init the timer

enumerator **LEDC_USE_APB_CLK**

Select APB as the source clock

enumerator **LEDC_USE_RC_FAST_CLK**

Select RC_FAST as the source clock

enumerator **LEDC_USE_XTAL_CLK**

Select XTAL as the source clock

enumerator **LEDC_USE_RTC8M_CLK**

Alias of 'LEDC_USE_RC_FAST_CLK'

enum **soc_periph_sdmmc_clk_src_t**

Type of SDMMC clock source.

Values:

enumerator **SDMMC_CLK_SRC_DEFAULT**

Select PLL_160M as the default choice

enumerator **SDMMC_CLK_SRC_PLL160M**

Select PLL_160M as the source clock

enumerator **SDMMC_CLK_SRC_XTAL**

Select XTAL as the source clock

enum **soc_clkout_sig_id_t**

Values:

enumerator **CLKOUT_SIG_PLL**

PLL_CLK is the output of crystal oscillator frequency multiplier

enumerator **CLKOUT_SIG_RC_SLOW**

RC slow clock, depends on the RTC_CLK_SRC configuration

enumerator **CLKOUT_SIG_XTAL**

Main crystal oscillator clock

enumerator **CLKOUT_SIG_PLL_F80M**

From PLL, usually be 80MHz

enumerator **CLKOUT_SIG_RC_FAST**

RC fast clock, about 17.5MHz

enumerator **CLKOUT_SIG_INVALID**

Header File

- components/esp_hw_support/include/esp_clk_tree.h
- This header file can be included with:

```
#include "esp_clk_tree.h"
```

Functions

esp_err_t **esp_clk_tree_src_get_freq_hz**(*soc_module_clk_t* clk_src, *esp_clk_tree_src_freq_precision_t* precision, uint32_t *freq_value)

Get frequency of module clock source.

Parameters

- **clk_src** -- **[in]** Clock source available to modules, in soc_module_clk_t
- **precision** -- **[in]** Degree of precision, one of esp_clk_tree_src_freq_precision_t values This arg only applies to the clock sources that their frequencies can vary: SOC_MOD_CLK_RTC_FAST, SOC_MOD_CLK_RTC_SLOW, SOC_MOD_CLK_RC_FAST, SOC_MOD_CLK_RC_FAST_D256, SOC_MOD_CLK_XTAL32K For other clock sources, this field is ignored.
- **freq_value** -- **[out]** Frequency of the clock source, in Hz

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Calibration failed

Enumerations

enum **esp_clk_tree_src_freq_precision_t**

Degree of precision of frequency value to be returned by esp_clk_tree_src_get_freq_hz()

Values:

enumerator **ESP_CLK_TREE_SRC_FREQ_PRECISION_CACHED**

enumerator **ESP_CLK_TREE_SRC_FREQ_PRECISION_APPROX**

enumerator **ESP_CLK_TREE_SRC_FREQ_PRECISION_EXACT**

enumerator **ESP_CLK_TREE_SRC_FREQ_PRECISION_INVALID**

2.6.5 GPIO & RTC GPIO

GPIO Summary

The ESP32-S3 chip features 45 physical GPIO pins (GPIO0 ~ GPIO21 and GPIO26 ~ GPIO48). Each pin can be used as a general-purpose I/O, or be connected to an internal peripheral signal. Through GPIO matrix, IO MUX, and RTC IO MUX, peripheral input signals can be from any GPIO pin, and peripheral output signals can be routed to any GPIO pin. Together these modules provide highly configurable I/O. For more details, see *ESP32-S3 Technical Reference Manual* > *IO MUX and GPIO Matrix (GPIO, IO_MUX)* [PDF].

The table below provides more information on pin usage, and please note the comments in the table for GPIOs with restrictions.

GPIO	Analog Function	RTC GPIO	Comment
GPIO0		RTC_GPIO0	Strapping pin
GPIO1	ADC1_CH0	RTC_GPIO1	
GPIO2	ADC1_CH1	RTC_GPIO2	
GPIO3	ADC1_CH2	RTC_GPIO3	Strapping pin
GPIO4	ADC1_CH3	RTC_GPIO4	
GPIO5	ADC1_CH4	RTC_GPIO5	
GPIO6	ADC1_CH5	RTC_GPIO6	
GPIO7	ADC1_CH6	RTC_GPIO7	
GPIO8	ADC1_CH7	RTC_GPIO8	
GPIO9	ADC1_CH8	RTC_GPIO9	
GPIO10	ADC1_CH9	RTC_GPIO10	
GPIO11	ADC2_CH0	RTC_GPIO11	
GPIO12	ADC2_CH1	RTC_GPIO12	
GPIO13	ADC2_CH2	RTC_GPIO13	
GPIO14	ADC2_CH3	RTC_GPIO14	
GPIO15	ADC2_CH4	RTC_GPIO15	
GPIO16	ADC2_CH5	RTC_GPIO16	
GPIO17	ADC2_CH6	RTC_GPIO17	
GPIO18	ADC2_CH7	RTC_GPIO18	
GPIO19	ADC2_CH8	RTC_GPIO19	USB-JTAG
GPIO20	ADC2_CH9	RTC_GPIO20	USB-JTAG
GPIO21		RTC_GPIO21	
GPIO26			SPI0/1
GPIO27			SPI0/1
GPIO28			SPI0/1
GPIO29			SPI0/1
GPIO30			SPI0/1
GPIO31			SPI0/1
GPIO32			SPI0/1
GPIO33			SPI0/1
GPIO34			SPI0/1
GPIO35			SPI0/1
GPIO36			SPI0/1
GPIO37			SPI0/1
GPIO38			
GPIO39			
GPIO40			
GPIO41			
GPIO42			
GPIO43			
GPIO44			
GPIO45			Strapping pin
GPIO46			Strapping pin
GPIO47			
GPIO48			

Note:

- Strapping pin: GPIO0, GPIO3, GPIO45 and GPIO46 are strapping pins. For more information, please refer to ESP32-S3 datasheet.
- SPI0/1: GPIO26 ~ GPIO32 are usually used for SPI flash and PSRAM and not recommended for other uses. When using Octal flash or Octal PSRAM or both, GPIO33 ~ GPIO37 are connected to SPIIO4 ~ SPIIO7 and SPIDQS. Therefore, on boards embedded with ESP32-S3R8 / ESP32-S3R8V chip, GPIO33 ~ GPIO37 are also not recommended for other uses.
- USB-JTAG: GPIO19 and GPIO20 are used by USB-JTAG by default. If they are reconfigured to operate as

normal GPIOs, USB-JTAG functionality will be disabled.

There is also separate "RTC GPIO" support, which functions when GPIOs are routed to the "RTC" low-power and analog subsystem. These pin functions can be used when:

- In Deep-sleep mode
- The *Ultra Low Power FSM co-processor* is running
- The *Ultra Low Power RISC-V co-processor* is running
- Analog functions such as ADC/DAC/etc are in use

Check Current Configuration of IOs

GPIO driver offers a dump function *gpio_dump_io_configuration()* to show the current configurations of IOs, such as pull-up/pull-down, input/output enable, pin mapping, etc. Below is an example of how to dump the configuration of GPIO4, GPIO18, and GPIO26:

```
gpio_dump_io_configuration(stdout, (1ULL << 4) | (1ULL << 18) | (1ULL << 26));
```

The dump will be like this:

```
================IO DUMP Start================
IO[4] -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigIn ID: (simple GPIO input)
  SleepSelEn: 1

IO[18] -
  Pullup: 0, Pulldown: 0, DriveCap: 2
  InputEn: 0, OutputEn: 1, OpenDrain: 0
  FuncSel: 1 (GPIO)
  GPIO Matrix SigOut ID: 256 (simple GPIO output)
  SleepSelEn: 1

IO[26] **RESERVED** -
  Pullup: 1, Pulldown: 0, DriveCap: 2
  InputEn: 1, OutputEn: 0, OpenDrain: 0
  FuncSel: 0 (IOMUX)
  SleepSelEn: 1

================IO DUMP End================
```

In addition, if you would like to dump the configurations of all IOs, you can use:

```
gpio_dump_all_io_configuration(stdout, SOC_GPIO_VALID_GPIO_MASK);
```

If an IO pin is routed to a peripheral signal through the GPIO matrix, the signal ID printed in the dump information is defined in the soc/esp32s3/include/soc/gpio_sig_map.h header file. The word `**RESERVED**` indicates the IO is occupied by either SPI flash or PSRAM. It is strongly not recommended to reconfigure them for other application purposes.

Do not rely on the default configurations values in the Technical Reference Manual, because it may be changed in the bootloader or application startup code before app_main.

Configure USB PHY Pins to GPIO

To configure the USB PHY pins to GPIO, you can use the function *gpio_config()*. Below is an example of how to configure the USB PHY pins to GPIO:

```
gpio_config_t usb_phy_conf = {
    .pin_bit_mask = (1ULL << USB_PHY_DP_PIN) | (1ULL << USB_PHY_DM_PIN),
    .mode = GPIO_MODE_INPUT_OUTPUT,
    .pull_up_en = 0,
    .pull_down_en = 0,
    .intr_type = GPIO_INTR_DISABLE,
};
gpio_config(&usb_phy_conf);
```

GPIO Glitch Filter

The ESP32-S3 chip features hardware filters to remove unwanted glitch pulses from the input GPIO, which can help reduce false triggering of the interrupt and prevent a noise being routed to the peripheral side.

Each GPIO can be configured with a glitch filter, which can be used to filter out pulses shorter than **two** sample clock cycles. The duration of the filter is not configurable. The sample clock is the clock source of the IO_MUX. In the driver, we call this kind of filter as `pin glitch filter`. You can create the filter handle by calling *gpio_new_pin_glitch_filter()*. All the configurations for a pin glitch filter are listed in the *gpio_pin_glitch_filter_config_t* structure.

- *gpio_pin_glitch_filter_config_t::gpio_num* sets the GPIO number to enable the glitch filter.

The glitch filter is disabled by default, and can be enabled by calling *gpio_glitch_filter_enable()*. To recycle the filter, you can call *gpio_del_glitch_filter()*. Please note, before deleting the filter, you should disable it first by calling *gpio_glitch_filter_disable()*.

Application Example

- GPIO output and input interrupt example: peripherals/gpio/generic_gpio.

API Reference - Normal GPIO

Header File

- components/esp_driver_gpio/include/driver/gpio.h
- This header file can be included with:

  ```
  #include "driver/gpio.h"
  ```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_driver_gpio
  ```

  or

  ```
  PRIV_REQUIRES esp_driver_gpio
  ```

Functions

esp_err_t **gpio_config** (const *gpio_config_t* *pGPIOConfig)

GPIO common configuration.

```
Configure GPIO's Mode,pull-up,PullDown,IntrType
```

Parameters **pGPIOConfig** -- Pointer to GPIO configure struct
Returns
- ESP_OK success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_reset_pin**(gpio_num_t gpio_num)

Reset an gpio to default state (select gpio function, enable pullup and disable input and output).

Note: This function also configures the IOMUX for this pin to the GPIO function, and disconnects any other peripheral output configured via GPIO Matrix.

Parameters **gpio_num** -- GPIO number.
Returns Always return ESP_OK.

esp_err_t **gpio_set_intr_type**(gpio_num_t gpio_num, *gpio_int_type_t* intr_type)

GPIO set interrupt trigger type.

Parameters
- **gpio_num** -- GPIO number. If you want to set the trigger type of e.g. of GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **intr_type** -- Interrupt type, select from gpio_int_type_t

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_intr_enable**(gpio_num_t gpio_num)

Enable GPIO module interrupt signal.

Note: ESP32: Please do not use the interrupt of GPIO36 and GPIO39 when using ADC or Wi-Fi and Bluetooth with sleep mode enabled. Please refer to the comments of `adc1_get_raw`. Please refer to Section 3.11 of ESP32 ECO and Workarounds for Bugs for the description of this issue.

Parameters **gpio_num** -- GPIO number. If you want to enable an interrupt on e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_intr_disable**(gpio_num_t gpio_num)

Disable GPIO module interrupt signal.

Note: This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters **gpio_num** -- GPIO number. If you want to disable the interrupt of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
Returns
- ESP_OK success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_set_level**(gpio_num_t gpio_num, uint32_t level)

GPIO set output level.

Note: This function is allowed to be executed when Cache is disabled within ISR context, by enabling `CONFIG_GPIO_CTRL_FUNC_IN_IRAM`

Parameters
- **gpio_num** -- GPIO number. If you want to set the output level of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **level** -- Output level. 0: low ; 1: high

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO number error

int **gpio_get_level**(gpio_num_t gpio_num)

GPIO get input level.

Warning: If the pad is not configured for input (or input and output) the returned value is always 0.

Parameters **gpio_num** -- GPIO number. If you want to get the logic level of e.g. pin GPIO16, gpio_num should be GPIO_NUM_16 (16);

Returns
- 0 the GPIO input level is 0
- 1 the GPIO input level is 1

esp_err_t **gpio_set_direction**(gpio_num_t gpio_num, *gpio_mode_t* mode)

GPIO set direction.

Configure GPIO direction,such as output_only,input_only,output_and_input

Parameters
- **gpio_num** -- Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** -- GPIO direction

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

esp_err_t **gpio_set_pull_mode**(gpio_num_t gpio_num, *gpio_pull_mode_t* pull)

Configure GPIO pull-up/pull-down resistors.

Note: ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

Parameters
- **gpio_num** -- GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **pull** -- GPIO pull up/down mode.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG : Parameter error

esp_err_t **gpio_wakeup_enable**(gpio_num_t gpio_num, *gpio_int_type_t* intr_type)

Enable GPIO wake-up function.

Parameters
- **gpio_num** -- GPIO number.
- **intr_type** -- GPIO wake-up type. Only GPIO_INTR_LOW_LEVEL or GPIO_INTR_HIGH_LEVEL can be used.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_wakeup_disable**(gpio_num_t gpio_num)

Disable GPIO wake-up function.

Parameters **gpio_num** -- GPIO number

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_isr_register**(void (*fn)(void*), void *arg, int intr_alloc_flags, *gpio_isr_handle_t* *handle)

Register GPIO interrupt handler, the handler is an ISR. The handler will be attached to the same CPU core that this function is running on.

This ISR function is called whenever any GPIO interrupt occurs. See the alternative gpio_install_isr_service() and gpio_isr_handler_add() API in order to have the driver support per-GPIO ISRs.

To disable or remove the ISR, pass the returned handle to the *interrupt allocation functions*.

Parameters
- **fn** -- Interrupt handler function.
- **arg** -- Parameter for handler function
- **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.
- **handle** -- Pointer to return handle. If non-NULL, a handle for the interrupt will be returned here.

Returns
- ESP_OK Success ;
- ESP_ERR_INVALID_ARG GPIO error
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags

esp_err_t **gpio_pullup_en**(gpio_num_t gpio_num)

Enable pull-up on GPIO.

Parameters **gpio_num** -- GPIO number

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_pullup_dis**(gpio_num_t gpio_num)

Disable pull-up on GPIO.

Parameters **gpio_num** -- GPIO number

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_pulldown_en**(gpio_num_t gpio_num)

Enable pull-down on GPIO.

Parameters **gpio_num** -- GPIO number

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_pulldown_dis**(gpio_num_t gpio_num)

Disable pull-down on GPIO.

Parameters **gpio_num** -- GPIO number

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_install_isr_service**(int intr_alloc_flags)

Install the GPIO driver's ETS_GPIO_INTR_SOURCE ISR handler service, which allows per-pin GPIO interrupt handlers.

This function is incompatible with gpio_isr_register() - if that function is used, a single global ISR is registered for all GPIO interrupts. If this function is used, the ISR service provides a global GPIO ISR and individual pin handlers are registered via the gpio_isr_handler_add() function.

Parameters **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.

Returns

- ESP_OK Success
- ESP_ERR_NO_MEM No memory to install this service
- ESP_ERR_INVALID_STATE ISR service already installed.
- ESP_ERR_NOT_FOUND No free interrupt found with the specified flags
- ESP_ERR_INVALID_ARG GPIO error

void **gpio_uninstall_isr_service**(void)

Uninstall the driver's GPIO ISR service, freeing related resources.

esp_err_t **gpio_isr_handler_add**(gpio_num_t gpio_num, *gpio_isr_t* isr_handler, void *args)

Add ISR handler for the corresponding GPIO pin.

Call this function after using gpio_install_isr_service() to install the driver's GPIO ISR handler service.

The pin ISR handlers no longer need to be declared with IRAM_ATTR, unless you pass the ESP_INTR_FLAG_IRAM flag when allocating the ISR in gpio_install_isr_service().

This ISR handler will be called from an ISR. So there is a stack size limit (configurable as "ISR stack size" in menuconfig). This limit is smaller compared to a global GPIO interrupt handler due to the additional level of indirection.

Parameters

- **gpio_num** -- GPIO number
- **isr_handler** -- ISR handler function for the corresponding GPIO number.
- **args** -- parameter for ISR handler.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_isr_handler_remove**(gpio_num_t gpio_num)

Remove ISR handler for the corresponding GPIO pin.

Parameters **gpio_num** -- GPIO number

Returns

- ESP_OK Success
- ESP_ERR_INVALID_STATE Wrong state, the ISR service has not been initialized.
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_set_drive_capability**(gpio_num_t gpio_num, *gpio_drive_cap_t* strength)

Set GPIO pad drive capability.

Parameters

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Drive capability of the pad

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_get_drive_capability**(gpio_num_t gpio_num, *gpio_drive_cap_t* *strength)

Get GPIO pad drive capability.

Parameters
- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Pointer to accept drive capability of the pad

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **gpio_hold_en**(gpio_num_t gpio_num)

Enable gpio pad hold function.

When a GPIO is set to hold, its state is latched at that moment and will not change when the internal signal or the IO MUX/GPIO configuration is modified (including input enable, output enable, output value, function, and drive strength values). This function can be used to retain the state of GPIOs when the power domain of where GPIO/IOMUX belongs to becomes off. For example, chip or system is reset (e.g. watchdog time-out, deep-sleep events are triggered), or peripheral power-down in light-sleep.

This function works in both input and output modes, and only applicable to output-capable GPIOs. If this function is enabled: in output mode: the output level of the GPIO will be locked and can not be changed. in input mode: the input read value can still reflect the changes of the input signal.

However, on ESP32/S2/C3/S3/C2, this function cannot be used to hold the state of a digital GPIO during Deep-sleep. Even if this function is enabled, the digital GPIO will be reset to its default state when the chip wakes up from Deep-sleep. If you want to hold the state of a digital GPIO during Deep-sleep, please call `gpio_deep_sleep_hold_en`.

Power down or call `gpio_hold_dis` will disable this function.

Parameters **gpio_num** -- GPIO number, only support output-capable GPIOs
Returns
- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

esp_err_t **gpio_hold_dis**(gpio_num_t gpio_num)

Disable gpio pad hold function.

When the chip is woken up from peripheral power-down sleep, the gpio will be set to the default mode, so, the gpio will output the default level if this function is called. If you don't want the level changes, the gpio should be configured to a known state before this function is called. e.g. If you hold gpio18 high during Deep-sleep, after the chip is woken up and `gpio_hold_dis` is called, gpio18 will output low level(because gpio18 is input mode by default). If you don't want this behavior, you should configure gpio18 as output mode and set it to high level before calling `gpio_hold_dis`.

Parameters **gpio_num** -- GPIO number, only support output-capable GPIOs
Returns
- ESP_OK Success
- ESP_ERR_NOT_SUPPORTED Not support pad hold function

void **gpio_deep_sleep_hold_en**(void)

Enable all digital gpio pads hold function during Deep-sleep.

Enabling this feature makes all digital gpio pads be at the holding state during Deep-sleep. The state of each pad holds is its active configuration (not pad's sleep configuration!).

Note that this pad hold feature only works when the chip is in Deep-sleep mode. When the chip is in active mode, the digital gpio state can be changed freely even you have called this function.

After this API is being called, the digital gpio Deep-sleep hold feature will work during every sleep process. You should call `gpio_deep_sleep_hold_dis` to disable this feature.

void **gpio_deep_sleep_hold_dis**(void)

Disable all digital gpio pads hold function during Deep-sleep.

void **gpio_iomux_in**(uint32_t gpio_num, uint32_t signal_idx)

SOC_GPIO_SUPPORT_HOLD_SINGLE_IO_IN_DSLP.

Set pad input to a peripheral signal through the IOMUX.

Parameters

- **gpio_num** -- GPIO number of the pad.
- **signal_idx** -- Peripheral signal id to input. One of the `*_IN_IDX` signals in `soc/gpio_sig_map.h`.

void **gpio_iomux_out**(uint8_t gpio_num, int func, bool out_en_inv)

Set peripheral output to an GPIO pad through the IOMUX.

Parameters

- **gpio_num** -- gpio_num GPIO number of the pad.
- **func** -- The function number of the peripheral pin to output pin. One of the `FUNC_X_*` of specified pin (X) in `soc/io_mux_reg.h`.
- **out_en_inv** -- True if the output enable needs to be inverted, otherwise False.

esp_err_t **gpio_force_hold_all**(void)

Force hold all digital and rtc gpio pads.

GPIO force hold, no matter the chip in active mode or sleep modes.

This function will immediately cause all pads to latch the current values of input enable, output enable, output value, function, and drive strength values.

Warning:

a. This function will hold flash and UART pins as well. Therefore, this function, and all code run afterwards (till calling `gpio_force_unhold_all` to disable this feature), MUST be placed in internal RAM as holding the flash pins will halt SPI flash operation, and holding the UART pins will halt any UART logging.
b. The hold state of all pads will be cancelled during ROM boot, so it is not recommended to use this API to hold the pads state during deepsleep and reset.

esp_err_t **gpio_force_unhold_all**(void)

Unhold all digital and rtc gpio pads.

Note: The global hold signal and the hold signal of each IO act on the PAD through 'or' logic, so if a pad has already been configured to hold by `gpio_hold_en`, this API can't release its hold state.

---

esp_err_t **gpio_sleep_sel_en**(gpio_num_t gpio_num)

Enable SLP_SEL to change GPIO status automantically in lightsleep.

Parameters **gpio_num** -- GPIO number of the pad.
Returns

- ESP_OK Success

esp_err_t **gpio_sleep_sel_dis**(gpio_num_t gpio_num)

Disable SLP_SEL to change GPIO status automantically in lightsleep.

Parameters **gpio_num** -- GPIO number of the pad.
Returns

- ESP_OK Success

esp_err_t **gpio_sleep_set_direction** (gpio_num_t gpio_num, *gpio_mode_t* mode)

GPIO set direction at sleep.

Configure GPIO direction,such as output_only,input_only,output_and_input

Parameters
- **gpio_num** -- Configure GPIO pins number, it should be GPIO number. If you want to set direction of e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **mode** -- GPIO direction

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO error

esp_err_t **gpio_sleep_set_pull_mode** (gpio_num_t gpio_num, *gpio_pull_mode_t* pull)

Configure GPIO pull-up/pull-down resistors at sleep.

Note: ESP32: Only pins that support both input & output have integrated pull-up and pull-down resistors. Input-only GPIOs 34-39 do not.

Parameters
- **gpio_num** -- GPIO number. If you want to set pull up or down mode for e.g. GPIO16, gpio_num should be GPIO_NUM_16 (16);
- **pull** -- GPIO pull up/down mode.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG : Parameter error

esp_err_t **gpio_dump_io_configuration** (FILE *out_stream, uint64_t io_bit_mask)

Dump IO configuration information to console.

Parameters
- **out_stream** -- IO stream (e.g. stdout)
- **io_bit_mask** -- IO pin bit mask, each bit maps to an IO

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

Structures

struct **gpio_config_t**

Configuration parameters of GPIO pad for gpio_config function.

Public Members

uint64_t **pin_bit_mask**

GPIO pin: set with bit mask, each bit maps to a GPIO

gpio_mode_t **mode**

GPIO mode: set input/output mode

gpio_pullup_t **pull_up_en**

GPIO pull-up

gpio_pulldown_t **pull_down_en**

GPIO pull-down

gpio_int_type_t **intr_type**

GPIO interrupt type

Macros

GPIO_PIN_COUNT

GPIO_IS_VALID_GPIO(gpio_num)

Check whether it is a valid GPIO number.

GPIO_IS_VALID_OUTPUT_GPIO(gpio_num)

Check whether it can be a valid GPIO number of output mode.

GPIO_IS_VALID_DIGITAL_IO_PAD(gpio_num)

Check whether it can be a valid digital I/O pad.

Type Definitions

typedef *intr_handle_t* **gpio_isr_handle_t**

typedef void (***gpio_isr_t**)(void *arg)

GPIO interrupt handler.

Param arg User registered data

Header File

- components/hal/include/hal/gpio_types.h
- This header file can be included with:

```
#include "hal/gpio_types.h"
```

Macros

GPIO_PIN_REG_0

GPIO_PIN_REG_1

GPIO_PIN_REG_2

GPIO_PIN_REG_3

GPIO_PIN_REG_4

GPIO_PIN_REG_5

GPIO_PIN_REG_6

GPIO_PIN_REG_7

GPIO_PIN_REG_8

GPIO_PIN_REG_9

GPIO_PIN_REG_10

GPIO_PIN_REG_11

GPIO_PIN_REG_12

GPIO_PIN_REG_13

GPIO_PIN_REG_14

GPIO_PIN_REG_15

GPIO_PIN_REG_16

GPIO_PIN_REG_17

GPIO_PIN_REG_18

GPIO_PIN_REG_19

GPIO_PIN_REG_20

GPIO_PIN_REG_21

GPIO_PIN_REG_22

GPIO_PIN_REG_23

GPIO_PIN_REG_24

GPIO_PIN_REG_25

GPIO_PIN_REG_26

GPIO_PIN_REG_27

GPIO_PIN_REG_28

GPIO_PIN_REG_29

GPIO_PIN_REG_30

GPIO_PIN_REG_31

GPIO_PIN_REG_32

GPIO_PIN_REG_33

GPIO_PIN_REG_34

GPIO_PIN_REG_35

GPIO_PIN_REG_36

GPIO_PIN_REG_37

GPIO_PIN_REG_38

GPIO_PIN_REG_39

GPIO_PIN_REG_40

GPIO_PIN_REG_41

GPIO_PIN_REG_42

GPIO_PIN_REG_43

GPIO_PIN_REG_44

GPIO_PIN_REG_45

GPIO_PIN_REG_46

GPIO_PIN_REG_47

GPIO_PIN_REG_48

GPIO_PIN_REG_49

GPIO_PIN_REG_50

GPIO_PIN_REG_51

GPIO_PIN_REG_52

GPIO_PIN_REG_53

GPIO_PIN_REG_54

Enumerations

enum **gpio_port_t**

Values:

enumerator **GPIO_PORT_0**

enumerator **GPIO_PORT_MAX**

enum **gpio_int_type_t**

Values:

enumerator **GPIO_INTR_DISABLE**

Disable GPIO interrupt

enumerator **GPIO_INTR_POSEDGE**

GPIO interrupt type : rising edge

enumerator **GPIO_INTR_NEGEDGE**

GPIO interrupt type : falling edge

enumerator **GPIO_INTR_ANYEDGE**

GPIO interrupt type : both rising and falling edge

enumerator **GPIO_INTR_LOW_LEVEL**

GPIO interrupt type : input low level trigger

enumerator **GPIO_INTR_HIGH_LEVEL**

GPIO interrupt type : input high level trigger

enumerator **GPIO_INTR_MAX**

enum **gpio_mode_t**

Values:

enumerator **GPIO_MODE_DISABLE**

GPIO mode : disable input and output

enumerator **GPIO_MODE_INPUT**

GPIO mode : input only

enumerator **GPIO_MODE_OUTPUT**

GPIO mode : output only mode

enumerator **GPIO_MODE_OUTPUT_OD**

GPIO mode : output only with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT_OD**

GPIO mode : output and input with open-drain mode

enumerator **GPIO_MODE_INPUT_OUTPUT**

GPIO mode : output and input mode

enum **gpio_pullup_t**

Values:

enumerator **GPIO_PULLUP_DISABLE**

Disable GPIO pull-up resistor

enumerator **GPIO_PULLUP_ENABLE**

Enable GPIO pull-up resistor

enum **gpio_pulldown_t**

Values:

enumerator **GPIO_PULLDOWN_DISABLE**

Disable GPIO pull-down resistor

enumerator **GPIO_PULLDOWN_ENABLE**

Enable GPIO pull-down resistor

enum **gpio_pull_mode_t**

Values:

enumerator **GPIO_PULLUP_ONLY**

Pad pull up

enumerator **GPIO_PULLDOWN_ONLY**

Pad pull down

enumerator **GPIO_PULLUP_PULLDOWN**

Pad pull up + pull down

enumerator **GPIO_FLOATING**

Pad floating

enum **gpio_drive_cap_t**

Values:

enumerator **GPIO_DRIVE_CAP_0**

Pad drive capability: weak

enumerator **GPIO_DRIVE_CAP_1**

Pad drive capability: stronger

enumerator **GPIO_DRIVE_CAP_2**

Pad drive capability: medium

enumerator **GPIO_DRIVE_CAP_DEFAULT**

Pad drive capability: medium

enumerator **GPIO_DRIVE_CAP_3**

Pad drive capability: strongest

enumerator **GPIO_DRIVE_CAP_MAX**

API Reference - RTC GPIO

Header File

- components/esp_driver_gpio/include/driver/rtc_io.h
- This header file can be included with:

```
#include "driver/rtc_io.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

bool **rtc_gpio_is_valid_gpio**(gpio_num_t gpio_num)

Determine if the specified GPIO is a valid RTC GPIO.

Parameters **gpio_num** -- GPIO number
Returns true if GPIO is valid for RTC GPIO use. false otherwise.

int **rtc_io_number_get**(gpio_num_t gpio_num)

Get RTC IO index number by gpio number.

Parameters **gpio_num** -- GPIO number
Returns >=0: Index of rtcio. -1 : The gpio is not rtcio.

esp_err_t **rtc_gpio_init**(gpio_num_t gpio_num)

Init a GPIO as RTC GPIO.

This function must be called when initializing a pad for an analog function.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
Returns

- ESP_OK success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_deinit**(gpio_num_t gpio_num)

Init a GPIO as digital GPIO.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
Returns

- ESP_OK success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

uint32_t **rtc_gpio_get_level**(gpio_num_t gpio_num)

Get the RTC IO input level.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
Returns
- 1 High level
- 0 Low level
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_set_level**(gpio_num_t gpio_num, uint32_t level)

Set the RTC IO output level.

Parameters
- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **level** -- output level

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_set_direction**(gpio_num_t gpio_num, *rtc_gpio_mode_t* mode)

RTC GPIO set direction.

Configure RTC GPIO direction, such as output only, input only, output and input.

Parameters
- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **mode** -- GPIO direction

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_set_direction_in_sleep**(gpio_num_t gpio_num, *rtc_gpio_mode_t* mode)

RTC GPIO set direction in deep sleep mode or disable sleep status (default). In some application scenarios, IO needs to have another states during deep sleep.

NOTE: ESP32 supports INPUT_ONLY mode. The rest targets support INPUT_ONLY, OUTPUT_ONLY, INPUT_OUTPUT mode.

Parameters
- **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
- **mode** -- GPIO direction

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_pullup_en**(gpio_num_t gpio_num)

RTC GPIO pullup enable.

This function only works for RTC IOs. In general, call gpio_pullup_en, which will work both for normal GPIOs and RTC IOs.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_pulldown_en**(gpio_num_t gpio_num)

RTC GPIO pulldown enable.

This function only works for RTC IOs. In general, call gpio_pulldown_en, which will work both for normal GPIOs and RTC IOs.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)
Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_pullup_dis**(gpio_num_t gpio_num)

RTC GPIO pullup disable.

This function only works for RTC IOs. In general, call gpio_pullup_dis, which will work both for normal GPIOs and RTC IOs.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_pulldown_dis**(gpio_num_t gpio_num)

RTC GPIO pulldown disable.

This function only works for RTC IOs. In general, call gpio_pulldown_dis, which will work both for normal GPIOs and RTC IOs.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_set_drive_capability**(gpio_num_t gpio_num, *gpio_drive_cap_t* strength)

Set RTC GPIO pad drive capability.

Parameters

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Drive capability of the pad

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **rtc_gpio_get_drive_capability**(gpio_num_t gpio_num, *gpio_drive_cap_t* *strength)

Get RTC GPIO pad drive capability.

Parameters

- **gpio_num** -- GPIO number, only support output GPIOs
- **strength** -- Pointer to accept drive capability of the pad

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **rtc_gpio_iomux_func_sel**(gpio_num_t gpio_num, int func)

Select a RTC IOMUX function for the RTC IO.

Parameters

- **gpio_num** -- GPIO number
- **func** -- Function to assign to the pin

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **rtc_gpio_hold_en**(gpio_num_t gpio_num)

Enable hold function on an RTC IO pad.

Enabling HOLD function will cause the pad to latch current values of input enable, output enable, output value, function, drive strength values. This function is useful when going into light or deep sleep mode to prevent the pin configuration from changing.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_hold_dis**(gpio_num_t gpio_num)

Disable hold function on an RTC IO pad.

Disabling hold function will allow the pad receive the values of input enable, output enable, output value, function, drive strength from RTC_IO peripheral.

Parameters **gpio_num** -- GPIO number (e.g. GPIO_NUM_12)

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG GPIO is not an RTC IO

esp_err_t **rtc_gpio_force_hold_en_all**(void)

Enable force hold signal for all RTC IOs.

Each RTC pad has a "force hold" input signal from the RTC controller. If this signal is set, pad latches current values of input enable, function, output enable, and other signals which come from the RTC mux. Force hold signal is enabled before going into deep sleep for pins which are used for EXT1 wakeup.

esp_err_t **rtc_gpio_force_hold_dis_all**(void)

Disable force hold signal for all RTC IOs.

esp_err_t **rtc_gpio_wakeup_enable**(gpio_num_t gpio_num, *gpio_int_type_t* intr_type)

Enable wakeup from sleep mode using specific GPIO.

Parameters

- **gpio_num** -- GPIO number
- **intr_type** -- Wakeup on high level (GPIO_INTR_HIGH_LEVEL) or low level (GPIO_INTR_LOW_LEVEL)

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if gpio_num is not an RTC IO, or intr_type is not one of GPIO_INTR_HIGH_LEVEL, GPIO_INTR_LOW_LEVEL.

esp_err_t **rtc_gpio_wakeup_disable**(gpio_num_t gpio_num)

Disable wakeup from sleep mode using specific GPIO.

Parameters **gpio_num** -- GPIO number

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if gpio_num is not an RTC IO

Macros

RTC_GPIO_IS_VALID_GPIO(gpio_num)

Header File

- components/esp_driver_gpio/include/driver/lp_io.h
- This header file can be included with:

```
#include "driver/lp_io.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Header File

- components/hal/include/hal/rtc_io_types.h
- This header file can be included with:

```
#include "hal/rtc_io_types.h"
```

Enumerations

enum **rtc_gpio_mode_t**

RTCIO output/input mode type.

Values:

enumerator **RTC_GPIO_MODE_INPUT_ONLY**

Pad input

enumerator **RTC_GPIO_MODE_OUTPUT_ONLY**

Pad output

enumerator **RTC_GPIO_MODE_INPUT_OUTPUT**

Pad input + output

enumerator **RTC_GPIO_MODE_DISABLED**

Pad (output + input) disable

enumerator **RTC_GPIO_MODE_OUTPUT_OD**

Pad open-drain output

enumerator **RTC_GPIO_MODE_INPUT_OUTPUT_OD**

Pad input + open-drain output

API Reference - GPIO Glitch Filter

Header File

- components/esp_driver_gpio/include/driver/gpio_filter.h
- This header file can be included with:

```
#include "driver/gpio_filter.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

esp_err_t **gpio_new_pin_glitch_filter**(const *gpio_pin_glitch_filter_config_t* *config, *gpio_glitch_filter_handle_t* *ret_filter)

Create a pin glitch filter.

Note: Pin glitch filter parameters are fixed, pulses shorter than two sample clocks (IO-MUX's source clock) will be filtered out. It's independent with "flex" glitch filter. See also `gpio_new_flex_glitch_filter`.

Note: The created filter handle can later be deleted by `gpio_del_glitch_filter`.

Parameters
- **config** -- **[in]** Glitch filter configuration
- **ret_filter** -- **[out]** Returned glitch filter handle

Returns
- ESP_OK: Create a pin glitch filter successfully
- ESP_ERR_INVALID_ARG: Create a pin glitch filter failed because of invalid arguments (e.g. wrong GPIO number)
- ESP_ERR_NO_MEM: Create a pin glitch filter failed because of out of memory
- ESP_FAIL: Create a pin glitch filter failed because of other error

esp_err_t **gpio_new_flex_glitch_filter**(const *gpio_flex_glitch_filter_config_t* *config, *gpio_glitch_filter_handle_t* *ret_filter)

Allocate a flex glitch filter.

Note: "flex" means the filter parameters (window, threshold) are adjustable. It's independent with pin glitch filter. See also `gpio_new_pin_glitch_filter`.

Note: The created filter handle can later be deleted by `gpio_del_glitch_filter`.

Parameters
- **config** -- **[in]** Glitch filter configuration
- **ret_filter** -- **[out]** Returned glitch filter handle

Returns
- ESP_OK: Allocate a flex glitch filter successfully
- ESP_ERR_INVALID_ARG: Allocate a flex glitch filter failed because of invalid arguments (e.g. wrong GPIO number, filter parameters out of range)
- ESP_ERR_NO_MEM: Allocate a flex glitch filter failed because of out of memory
- ESP_ERR_NOT_FOUND: Allocate a flex glitch filter failed because the underlying hardware resources are used up
- ESP_FAIL: Allocate a flex glitch filter failed because of other error

esp_err_t **gpio_del_glitch_filter**(*gpio_glitch_filter_handle_t* filter)

Delete a glitch filter.

Parameters **filter** -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns
- ESP_OK: Delete glitch filter successfully
- ESP_ERR_INVALID_ARG: Delete glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Delete glitch filter failed because the glitch filter is still in working
- ESP_FAIL: Delete glitch filter failed because of other error

esp_err_t **gpio_glitch_filter_enable** (*gpio_glitch_filter_handle_t* filter)

Enable a glitch filter.

Parameters `filter` -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns

- ESP_OK: Enable glitch filter successfully
- ESP_ERR_INVALID_ARG: Enable glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Enable glitch filter failed because the glitch filter is already enabled
- ESP_FAIL: Enable glitch filter failed because of other error

esp_err_t **gpio_glitch_filter_disable** (*gpio_glitch_filter_handle_t* filter)

Disable a glitch filter.

Parameters `filter` -- **[in]** Glitch filter handle returned from `gpio_new_flex_glitch_filter` or `gpio_new_pin_glitch_filter`

Returns

- ESP_OK: Disable glitch filter successfully
- ESP_ERR_INVALID_ARG: Disable glitch filter failed because of invalid arguments
- ESP_ERR_INVALID_STATE: Disable glitch filter failed because the glitch filter is not enabled yet
- ESP_FAIL: Disable glitch filter failed because of other error

Structures

struct **gpio_pin_glitch_filter_config_t**

Configuration of GPIO pin glitch filter.

Public Members

glitch_filter_clock_source_t **clk_src**

Clock source for the glitch filter

gpio_num_t **gpio_num**

GPIO number

struct **gpio_flex_glitch_filter_config_t**

Configuration of GPIO flex glitch filter.

Public Members

glitch_filter_clock_source_t **clk_src**

Clock source for the glitch filter

gpio_num_t **gpio_num**

GPIO number

uint32_t **window_width_ns**

Sample window width (in ns)

uint32_t **window_thres_ns**

Sample window threshold (in ns), during the `window_width_ns` sample window, any pulse whose width < window_thres_ns will be discarded.

Type Definitions

typedef struct gpio_glitch_filter_t ***gpio_glitch_filter_handle_t**

Typedef of GPIO glitch filter handle.

2.6.6 General Purpose Timer (GPTimer)

Introduction

GPTimer (General Purpose Timer) is the driver of ESP32-S3 Timer Group peripheral. The hardware timer features high resolution and flexible alarm action. The behavior when the internal counter of a timer reaches a specific target value is called a timer alarm. When a timer alarms, a user registered per-timer callback would be called.

Typically, a general purpose timer can be used in scenarios like:

- Free running as a wall clock, fetching a high-resolution timestamp at any time and any places
- Generate period alarms, trigger events periodically
- Generate one-shot alarm, respond in target time

Functional Overview

The following sections of this document cover the typical steps to install and operate a timer:

- *Resource Allocation* - covers which parameters should be set up to get a timer handle and how to recycle the resources when GPTimer finishes working.
- *Set and Get Count Value* - covers how to force the timer counting from a start point and how to get the count value at anytime.
- *Set up Alarm Action* - covers the parameters that should be set up to enable the alarm event.
- *Register Event Callbacks* - covers how to hook user specific code to the alarm event callback function.
- *Enable and Disable Timer* - covers how to enable and disable the timer.
- *Start and Stop Timer* - shows some typical use cases that start the timer with different alarm behavior.
- *Power Management* - describes how different source clock selections can affect power consumption.
- *IRAM Safe* - describes tips on how to make the timer interrupt and IO control functions work better along with a disabled cache.
- *Thread Safety* - lists which APIs are guaranteed to be thread safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can be used to make a different effect on driver behavior.

Resource Allocation Different ESP chips might have different numbers of independent timer groups, and within each group, there could also be several independent timers.[1]

A GPTimer instance is represented by `gptimer_handle_t`. The driver behind manages all available hardware resources in a pool, so that you do not need to care about which timer and which group it belongs to.

To install a timer instance, there is a configuration structure that needs to be given in advance: `gptimer_config_t`:

[1] Different ESP chip series might have different numbers of GPTimer instances. For more details, please refer to **ESP32-S3 Technical Reference Manual** > Chapter **Timer Group (TIMG)** [PDF]. The driver does forbid you from applying for more timers, but it returns error when all available hardware resources are used up. Please always check the return value when doing resource allocation (e.g., `gptimer_new_timer()`).

- *gptimer_config_t::clk_src* selects the source clock for the timer. The available clocks are listed in *gptimer_clock_source_t*, you can only pick one of them. For the effect on power consumption of different clock source, please refer to Section *Power Management*.
- *gptimer_config_t::direction* sets the counting direction of the timer, supported directions are listed in *gptimer_count_direction_t*, you can only pick one of them.
- *gptimer_config_t::resolution_hz* sets the resolution of the internal counter. Each count step is equivalent to **1 / resolution_hz** seconds.
- gptimer_config::intr_priority sets the priority of the timer interrupt. If it is set to 0, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- Optional *gptimer_config_t::intr_shared* sets whether or not mark the timer interrupt source as a shared one. For the pros/cons of a shared interrupt, you can refer to *Interrupt Handling*.

With all the above configurations set in the structure, the structure can be passed to *gptimer_new_timer()* which will instantiate the timer instance and return a handle of the timer.

The function can fail due to various errors such as insufficient memory, invalid arguments, etc. Specifically, when there are no more free timers (i.e., all hardware resources have been used up), then *ESP_ERR_NOT_FOUND* will be returned. The total number of available timers is represented by the *SOC_TIMER_GROUP_TOTAL_TIMERS* and its value depends on the ESP chip.

If a previously created GPTimer instance is no longer required, you should recycle the timer by calling *gptimer_del_timer()*. This allows the underlying HW timer to be used for other purposes. Before deleting a GPTimer handle, please disable it by *gptimer_disable()* in advance or make sure it has not enabled yet by *gptimer_enable()*.

Creating a GPTimer Handle with Resolution of 1 MHz

```
gptimer_handle_t gptimer = NULL;
gptimer_config_t timer_config = {
    .clk_src = GPTIMER_CLK_SRC_DEFAULT,
    .direction = GPTIMER_COUNT_UP,
    .resolution_hz = 1 * 1000 * 1000, // 1MHz, 1 tick = 1us
};
ESP_ERROR_CHECK(gptimer_new_timer(&timer_config, &gptimer));
```

Set and Get Count Value When the GPTimer is created, the internal counter will be reset to zero by default. The counter value can be updated asynchronously by *gptimer_set_raw_count()*. The maximum count value is dependent on the bit width of the hardware timer, which is also reflected by the SOC macro *SOC_TIMER_GROUP_COUNTER_BIT_WIDTH*. When updating the raw count of an active timer, the timer will immediately start counting from the new value.

Count value can be retrieved by *gptimer_get_raw_count()*, at any time.

Set up Alarm Action For most of the use cases of GPTimer, you should set up the alarm action before starting the timer, except for the simple wall-clock scenario, where a free running timer is enough. To set up the alarm action, you should configure several members of *gptimer_alarm_config_t* based on how you make use of the alarm event:

- *gptimer_alarm_config_t::alarm_count* sets the target count value that triggers the alarm event. You should also take the counting direction into consideration when setting the alarm value. Specially, *gptimer_alarm_config_t::alarm_count* and *gptimer_alarm_config_t::reload_count* cannot be set to the same value when *gptimer_alarm_config_t::auto_reload_on_alarm* is true, as keeping reload with a target alarm count is meaningless.
- *gptimer_alarm_config_t::reload_count* sets the count value to be reloaded when the alarm event happens. This configuration only takes effect when *gptimer_alarm_config_t::auto_reload_on_alarm* is set to true.
- *gptimer_alarm_config_t::auto_reload_on_alarm* flag sets whether to enable the auto-reload feature. If enabled, the hardware timer will reload the value of *gptimer_alarm_config_t::reload_count* into counter immediately when an alarm event happens.

To make the alarm configurations take effect, you should call *gptimer_set_alarm_action()*. Especially, if *gptimer_alarm_config_t* is set to NULL, the alarm function will be disabled.

Note: If an alarm value is set and the timer has already exceeded this value, the alarm will be triggered immediately.

Register Event Callbacks After the timer starts up, it can generate a specific event (e.g., the "Alarm Event") dynamically. If you have some functions that should be called when the event happens, please hook your function to the interrupt service routine by calling *gptimer_register_event_callbacks()*. All supported event callbacks are listed in *gptimer_event_callbacks_t*:

- *gptimer_event_callbacks_t::on_alarm* sets a callback function for alarm events. As this function is called within the ISR context, you must ensure that the function does not attempt to block (e.g., by making sure that only FreeRTOS APIs with ISR suffix are called from within the function). The function prototype is declared in *gptimer_alarm_cb_t*.

You can save your own context to *gptimer_register_event_callbacks()* as well, via the parameter user_data. The user data will be directly passed to the callback function.

This function lazy installs the interrupt service for the timer but not enable it. So please call this function before *gptimer_enable()*, otherwise the *ESP_ERR_INVALID_STATE* error will be returned. See Section *Enable and Disable Timer* for more information.

Enable and Disable Timer Before doing IO control to the timer, you needs to enable the timer first, by calling *gptimer_enable()*. This function:

- Switches the timer driver state from **init** to **enable**.
- Enables the interrupt service if it has been lazy installed by *gptimer_register_event_callbacks()*.
- Acquires a proper power management lock if a specific clock source (e.g., APB clock) is selected. See Section *Power Management* for more information.

Calling *gptimer_disable()* does the opposite, that is, put the timer driver back to the **init** state, disable the interrupts service and release the power management lock.

Start and Stop Timer The basic IO operation of a timer is to start and stop. Calling *gptimer_start()* can make the internal counter work, while calling *gptimer_stop()* can make the counter stop working. The following illustrates how to start a timer with or without an alarm event.

Calling *gptimer_start()* transits the driver state from **enable** to **run**, and vice versa. You need to make sure the start and stop functions are used in pairs, otherwise, the functions may return *ESP_ERR_INVALID_STATE*. Most of the time, this error means that the timer is already stopped or in the "start protection" state (i.e., *gptimer_start()* is called but not finished).

Start Timer as a Wall Clock

```
ESP_ERROR_CHECK(gptimer_enable(gptimer));
ESP_ERROR_CHECK(gptimer_start(gptimer));
// Retrieve the timestamp at any time
uint64_t count;
ESP_ERROR_CHECK(gptimer_get_raw_count(gptimer, &count));
```

Trigger Period Events

```
typedef struct {
    uint64_t event_count;
} example_queue_element_t;
```

(continues on next page)

(continued from previous page)

```
static bool example_timer_on_alarm_cb(gptimer_handle_t timer, const gptimer_alarm_
↪event_data_t *edata, void *user_ctx)
{
    BaseType_t high_task_awoken = pdFALSE;
    QueueHandle_t queue = (QueueHandle_t)user_ctx;
    // Retrieve the count value from event data
    example_queue_element_t ele = {
        .event_count = edata->count_value
    };
    // Optional: send the event data to other task by OS queue
    // Do not introduce complex logics in callbacks
    // Suggest dealing with event data in the main loop, instead of in this␣
↪callback
    xQueueSendFromISR(queue, &ele, &high_task_awoken);
    // return whether we need to yield at the end of ISR
    return high_task_awoken == pdTRUE;
}

gptimer_alarm_config_t alarm_config = {
    .reload_count = 0, // counter will reload with 0 on alarm event
    .alarm_count = 1000000, // period = 1s @resolution 1MHz
    .flags.auto_reload_on_alarm = true, // enable auto-reload
};
ESP_ERROR_CHECK(gptimer_set_alarm_action(gptimer, &alarm_config));

gptimer_event_callbacks_t cbs = {
    .on_alarm = example_timer_on_alarm_cb, // register user callback
};
ESP_ERROR_CHECK(gptimer_register_event_callbacks(gptimer, &cbs, queue));
ESP_ERROR_CHECK(gptimer_enable(gptimer));
ESP_ERROR_CHECK(gptimer_start(gptimer));
```

Trigger One-Shot Event

```
typedef struct {
    uint64_t event_count;
} example_queue_element_t;

static bool example_timer_on_alarm_cb(gptimer_handle_t timer, const gptimer_alarm_
↪event_data_t *edata, void *user_ctx)
{
    BaseType_t high_task_awoken = pdFALSE;
    QueueHandle_t queue = (QueueHandle_t)user_ctx;
    // Stop timer the sooner the better
    gptimer_stop(timer);
    // Retrieve the count value from event data
    example_queue_element_t ele = {
        .event_count = edata->count_value
    };
    // Optional: send the event data to other task by OS queue
    xQueueSendFromISR(queue, &ele, &high_task_awoken);
    // return whether we need to yield at the end of ISR
    return high_task_awoken == pdTRUE;
}

gptimer_alarm_config_t alarm_config = {
    .alarm_count = 1 * 1000 * 1000, // alarm target = 1s @resolution 1MHz
};
ESP_ERROR_CHECK(gptimer_set_alarm_action(gptimer, &alarm_config));
```

(continues on next page)

(continued from previous page)

```
gptimer_event_callbacks_t cbs = {
    .on_alarm = example_timer_on_alarm_cb, // register user callback
};
ESP_ERROR_CHECK(gptimer_register_event_callbacks(gptimer, &cbs, queue));
ESP_ERROR_CHECK(gptimer_enable(gptimer));
ESP_ERROR_CHECK(gptimer_start(gptimer));
```

Dynamic Alarm Update Alarm value can be updated dynamically inside the ISR handler callback, by changing *gptimer_alarm_event_data_t::alarm_value*. Then the alarm value will be updated after the callback function returns.

```
typedef struct {
    uint64_t event_count;
} example_queue_element_t;

static bool example_timer_on_alarm_cb(gptimer_handle_t timer, const gptimer_alarm_
↪event_data_t *edata, void *user_ctx)
{
    BaseType_t high_task_awoken = pdFALSE;
    QueueHandle_t queue = (QueueHandle_t)user_data;
    // Retrieve the count value from event data
    example_queue_element_t ele = {
        .event_count = edata->count_value
    };
    // Optional: send the event data to other task by OS queue
    xQueueSendFromISR(queue, &ele, &high_task_awoken);
    // reconfigure alarm value
    gptimer_alarm_config_t alarm_config = {
        .alarm_count = edata->alarm_value + 1000000, // alarm in next 1s
    };
    gptimer_set_alarm_action(timer, &alarm_config);
    // return whether we need to yield at the end of ISR
    return high_task_awoken == pdTRUE;
}

gptimer_alarm_config_t alarm_config = {
    .alarm_count = 1000000, // initial alarm target = 1s @resolution 1MHz
};
ESP_ERROR_CHECK(gptimer_set_alarm_action(gptimer, &alarm_config));

gptimer_event_callbacks_t cbs = {
    .on_alarm = example_timer_on_alarm_cb, // register user callback
};
ESP_ERROR_CHECK(gptimer_register_event_callbacks(gptimer, &cbs, queue));
ESP_ERROR_CHECK(gptimer_enable(gptimer));
ESP_ERROR_CHECK(gptimer_start(gptimer, &alarm_config));
```

Power Management There are some power management strategies, which might turn off or change the frequency of GPTimer's source clock to save power consumption. For example, during DFS, APB clock will be scaled down. If light-sleep is also enabled, PLL and XTAL clocks will be powered off. Both of them can result in an inaccurate time keeping.

The driver can prevent the above situation from happening by creating different power management lock according to different clock source. The driver increases the reference count of that power management lock in the *gptimer_enable()* and decrease it in the *gptimer_disable()*. So we can ensure the clock source is stable between *gptimer_enable()* and *gptimer_disable()*.

IRAM Safe By default, the GPTimer interrupt will be deferred when the cache is disabled because of writing or erasing the flash. Thus the alarm interrupt will not get executed in time, which is not expected in a real-time application.

There is a Kconfig option *CONFIG_GPTIMER_ISR_IRAM_SAFE* that:

- Enables the interrupt being serviced even when the cache is disabled
- Places all functions that used by the ISR into IRAM[2]
- Places driver object into DRAM (in case it is mapped to PSRAM by accident)

This allows the interrupt to run while the cache is disabled, but comes at the cost of increased IRAM consumption.

There is another Kconfig option *CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM* that can put commonly used IO control functions into IRAM as well. So, these functions can also be executable when the cache is disabled. These IO control functions are as follows:

- `gptimer_start()`
- `gptimer_stop()`
- `gptimer_get_raw_count()`
- `gptimer_set_raw_count()`
- `gptimer_set_alarm_action()`

Thread Safety All the APIs provided by the driver are guaranteed to be thread safe, which means you can call them from different RTOS tasks without protection by extra locks. The following functions are allowed to run under ISR context.

- `gptimer_start()`
- `gptimer_stop()`
- `gptimer_get_raw_count()`
- `gptimer_set_raw_count()`
- `gptimer_get_captured_count()`
- `gptimer_set_alarm_action()`

Kconfig Options

- *CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM* controls where to place the GPTimer control functions (IRAM or flash).
- *CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM* controls where to place the GPTimer ISR handler (IRAM or flash).
- *CONFIG_GPTIMER_ISR_IRAM_SAFE* controls whether the default ISR handler should be masked when the cache is disabled, see Section *IRAM Safe* for more information.
- *CONFIG_GPTIMER_ENABLE_DEBUG_LOG* is used to enabled the debug log output. Enable this option will increase the firmware binary size.

Application Examples

- Typical use cases of GPTimer are listed in the example peripherals/timer_group/gptimer.

API Reference

Header File

- components/esp_driver_gptimer/include/driver/gptimer.h
- This header file can be included with:

[2] `gptimer_event_callbacks_t::on_alarm` callback and the functions invoked by the callback should also be placed in IRAM, please take care of them by yourself.

```
#include "driver/gptimer.h"
```

- This header file is a part of the API provided by the `esp_driver_gptimer` component. To declare that your component depends on `esp_driver_gptimer`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gptimer
```

or

```
PRIV_REQUIRES esp_driver_gptimer
```

Functions

esp_err_t **gptimer_new_timer**(const *gptimer_config_t* *config, *gptimer_handle_t* *ret_timer)

Create a new General Purpose Timer, and return the handle.

Note: The newly created timer is put in the "init" state.

Parameters
- **config** -- **[in]** GPTimer configuration
- **ret_timer** -- **[out]** Returned timer handle

Returns
- ESP_OK: Create GPTimer successfully
- ESP_ERR_INVALID_ARG: Create GPTimer failed because of invalid argument
- ESP_ERR_NO_MEM: Create GPTimer failed because out of memory
- ESP_ERR_NOT_FOUND: Create GPTimer failed because all hardware timers are used up and no more free one
- ESP_FAIL: Create GPTimer failed because of other error

esp_err_t **gptimer_del_timer**(*gptimer_handle_t* timer)

Delete the GPTimer handle.

Note: A timer must be in the "init" state before it can be deleted.

Parameters **timer** -- **[in]** Timer handle created by `gptimer_new_timer`

Returns
- ESP_OK: Delete GPTimer successfully
- ESP_ERR_INVALID_ARG: Delete GPTimer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Delete GPTimer failed because the timer is not in init state
- ESP_FAIL: Delete GPTimer failed because of other error

esp_err_t **gptimer_set_raw_count**(*gptimer_handle_t* timer, uint64_t value)

Set GPTimer raw count value.

Note: When updating the raw count of an active timer, the timer will immediately start counting from the new value.

Note: This function is allowed to run within ISR context

Note: If `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Flash Cache is disabled.

Parameters
- **timer** -- **[in]** Timer handle created by gptimer_new_timer
- **value** -- **[in]** Count value to be set

Returns
- ESP_OK: Set GPTimer raw count value successfully
- ESP_ERR_INVALID_ARG: Set GPTimer raw count value failed because of invalid argument
- ESP_FAIL: Set GPTimer raw count value failed because of other error

esp_err_t **gptimer_get_raw_count**(*gptimer_handle_t* timer, uint64_t *value)

Get GPTimer raw count value.

Note: This function will trigger a software capture event and then return the captured count value.

Note: With the raw count value and the resolution returned from gptimer_get_resolution, you can convert the count value into seconds.

Note: This function is allowed to run within ISR context

Note: If CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Flash Cache is disabled.

Parameters
- **timer** -- **[in]** Timer handle created by gptimer_new_timer
- **value** -- **[out]** Returned GPTimer count value

Returns
- ESP_OK: Get GPTimer raw count value successfully
- ESP_ERR_INVALID_ARG: Get GPTimer raw count value failed because of invalid argument
- ESP_FAIL: Get GPTimer raw count value failed because of other error

esp_err_t **gptimer_get_resolution**(*gptimer_handle_t* timer, uint32_t *out_resolution)

Return the real resolution of the timer.

Note: usually the timer resolution is same as what you configured in the *gptimer_config_t::resolution_hz*, but some unstable clock source (e.g. RC_FAST) will do a calibration, the real resolution can be different from the configured one.

Parameters
- **timer** -- **[in]** Timer handle created by gptimer_new_timer
- **out_resolution** -- **[out]** Returned timer resolution, in Hz

Returns
- ESP_OK: Get GPTimer resolution successfully
- ESP_ERR_INVALID_ARG: Get GPTimer resolution failed because of invalid argument
- ESP_FAIL: Get GPTimer resolution failed because of other error

esp_err_t **gptimer_get_captured_count**(*gptimer_handle_t* timer, uint64_t *value)

Get GPTimer captured count value.

Note: The capture action can be issued either by ETM event or by software (see also `gptimer_get_raw_count`).

Note: This function is allowed to run within ISR context

Note: If `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Flash Cache is disabled.

Parameters

- **`timer`** -- **[in]** Timer handle created by `gptimer_new_timer`
- **`value`** -- **[out]** Returned captured count value

Returns

- ESP_OK: Get GPTimer captured count value successfully
- ESP_ERR_INVALID_ARG: Get GPTimer captured count value failed because of invalid argument
- ESP_FAIL: Get GPTimer captured count value failed because of other error

esp_err_t **`gptimer_register_event_callbacks`**(*gptimer_handle_t* timer, const *gptimer_event_callbacks_t* *cbs, void *user_data)

Set callbacks for GPTimer.

Note: User registered callbacks are expected to be runnable within ISR context

Note: The first call to this function needs to be before the call to `gptimer_enable`

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **`timer`** -- **[in]** Timer handle created by `gptimer_new_timer`
- **`cbs`** -- **[in]** Group of callback functions
- **`user_data`** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set event callbacks failed because the timer is not in init state
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **`gptimer_set_alarm_action`**(*gptimer_handle_t* timer, const *gptimer_alarm_config_t* *config)

Set alarm event actions for GPTimer.

Note: This function is allowed to run within ISR context, so you can update new alarm action immediately in any ISR callback.

Note: If `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM

by linker, makes it possible to execute even when the Flash Cache is disabled. In this case, please also ensure the *gptimer_alarm_config_t* instance is placed in the static data section instead of in the read-only data section. e.g.: `static gptimer_alarm_config_t alarm_config = { ... };`

Parameters

- **timer** -- **[in]** Timer handle created by `gptimer_new_timer`
- **config** -- **[in]** Alarm configuration, especially, set config to NULL means disabling the alarm function

Returns

- ESP_OK: Set alarm action for GPTimer successfully
- ESP_ERR_INVALID_ARG: Set alarm action for GPTimer failed because of invalid argument
- ESP_FAIL: Set alarm action for GPTimer failed because of other error

esp_err_t **gptimer_enable**(*gptimer_handle_t* timer)

Enable GPTimer.

Note: This function will transit the timer state from "init" to "enable".

Note: This function will enable the interrupt service, if it's lazy installed in `gptimer_register_event_callbacks`.

Note: This function will acquire a PM lock, if a specific source clock (e.g. APB) is selected in the *gptimer_config_t*, while `CONFIG_PM_ENABLE` is enabled.

Note: Enable a timer doesn't mean to start it. See also `gptimer_start` for how to make the timer start counting.

Parameters **timer** -- **[in]** Timer handle created by `gptimer_new_timer`

Returns

- ESP_OK: Enable GPTimer successfully
- ESP_ERR_INVALID_ARG: Enable GPTimer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Enable GPTimer failed because the timer is already enabled
- ESP_FAIL: Enable GPTimer failed because of other error

esp_err_t **gptimer_disable**(*gptimer_handle_t* timer)

Disable GPTimer.

Note: This function will transit the timer state from "enable" to "init".

Note: This function will disable the interrupt service if it's installed.

Note: This function will release the PM lock if it's acquired in the `gptimer_enable`.

Note: Disable a timer doesn't mean to stop it. See also `gptimer_stop` for how to make the timer stop counting.

Parameters **`timer`** -- **[in]** Timer handle created by `gptimer_new_timer`
Returns
- ESP_OK: Disable GPTimer successfully
- ESP_ERR_INVALID_ARG: Disable GPTimer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable GPTimer failed because the timer is not enabled yet
- ESP_FAIL: Disable GPTimer failed because of other error

esp_err_t **`gptimer_start`** (*gptimer_handle_t* timer)

Start GPTimer (internal counter starts counting)

Note: This function will transit the timer state from "enable" to "run".

Note: This function is allowed to run within ISR context

Note: If `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Flash Cache is disabled.

Parameters **`timer`** -- **[in]** Timer handle created by `gptimer_new_timer`
Returns
- ESP_OK: Start GPTimer successfully
- ESP_ERR_INVALID_ARG: Start GPTimer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Start GPTimer failed because the timer is not enabled or is already in running
- ESP_FAIL: Start GPTimer failed because of other error

esp_err_t **`gptimer_stop`** (*gptimer_handle_t* timer)

Stop GPTimer (internal counter stops counting)

Note: This function will transit the timer state from "run" to "enable".

Note: This function is allowed to run within ISR context

Note: If `CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Flash Cache is disabled.

Parameters **`timer`** -- **[in]** Timer handle created by `gptimer_new_timer`
Returns
- ESP_OK: Stop GPTimer successfully
- ESP_ERR_INVALID_ARG: Stop GPTimer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Stop GPTimer failed because the timer is not in running.
- ESP_FAIL: Stop GPTimer failed because of other error

Structures

struct **gptimer_config_t**

General Purpose Timer configuration.

Public Members

gptimer_clock_source_t **clk_src**

GPTimer clock source

gptimer_count_direction_t **direction**

Count direction

uint32_t **resolution_hz**

Counter resolution (working frequency) in Hz, hence, the step size of each count tick equals to (1 / resolution_hz) seconds

int **intr_priority**

GPTimer interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **intr_shared**

Set true, the timer interrupt number can be shared with other peripherals

uint32_t **backup_before_sleep**

If set, the driver will backup/restore the GPTimer registers before/after entering/exist sleep mode. By this approach, the system can power off GPTimer's power domain. This can save power, but at the expense of more RAM being consumed

struct *gptimer_config_t*::[anonymous] **flags**

GPTimer config flags

struct **gptimer_event_callbacks_t**

Group of supported GPTimer callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_GPTIMER_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM.

Public Members

gptimer_alarm_cb_t **on_alarm**

Timer alarm callback

struct **gptimer_alarm_config_t**

General Purpose Timer alarm configuration.

Public Members

uint64_t **alarm_count**

Alarm target count value

uint64_t **reload_count**

Alarm reload count value, effect only when `auto_reload_on_alarm` is set to true

uint32_t **auto_reload_on_alarm**

Reload the count value by hardware, immediately at the alarm event

struct *gptimer_alarm_config_t*::[anonymous] **flags**

Alarm config flags

Header File

- components/esp_driver_gptimer/include/driver/gptimer_etm.h
- This header file can be included with:

```
#include "driver/gptimer_etm.h"
```

- This header file is a part of the API provided by the `esp_driver_gptimer` component. To declare that your component depends on `esp_driver_gptimer`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gptimer
```

or

```
PRIV_REQUIRES esp_driver_gptimer
```

Functions

esp_err_t **gptimer_new_etm_event**(*gptimer_handle_t* timer, const *gptimer_etm_event_config_t* *config, esp_etm_event_handle_t *out_event)

Get the ETM event for GPTimer.

Note: The created ETM event object can be deleted later by calling `esp_etm_del_event`

Parameters

- **timer** -- **[in]** Timer handle created by `gptimer_new_timer`
- **config** -- **[in]** GPTimer ETM event configuration
- **out_event** -- **[out]** Returned ETM event handle

Returns

- ESP_OK: Get ETM event successfully
- ESP_ERR_INVALID_ARG: Get ETM event failed because of invalid argument
- ESP_FAIL: Get ETM event failed because of other error

esp_err_t **gptimer_new_etm_task**(*gptimer_handle_t* timer, const *gptimer_etm_task_config_t* *config, esp_etm_task_handle_t *out_task)

Get the ETM task for GPTimer.

Note: The created ETM task object can be deleted later by calling `esp_etm_del_task`

Parameters

- **timer** -- **[in]** Timer handle created by `gptimer_new_timer`
- **config** -- **[in]** GPTimer ETM task configuration
- **out_task** -- **[out]** Returned ETM task handle

Returns

- ESP_OK: Get ETM task successfully
- ESP_ERR_INVALID_ARG: Get ETM task failed because of invalid argument
- ESP_FAIL: Get ETM task failed because of other error

Structures

struct **gptimer_etm_event_config_t**

GPTimer ETM event configuration.

Public Members

gptimer_etm_event_type_t **event_type**

GPTimer ETM event type

struct **gptimer_etm_task_config_t**

GPTimer ETM task configuration.

Public Members

gptimer_etm_task_type_t **task_type**

GPTimer ETM task type

Header File

- components/esp_driver_gptimer/include/driver/gptimer_types.h
- This header file can be included with:

```
#include "driver/gptimer_types.h"
```

- This header file is a part of the API provided by the `esp_driver_gptimer` component. To declare that your component depends on `esp_driver_gptimer`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gptimer
```

or

```
PRIV_REQUIRES esp_driver_gptimer
```

Structures

struct **gptimer_alarm_event_data_t**

GPTimer alarm event data.

Public Members

uint64_t **count_value**

Current count value

uint64_t **alarm_value**

Current alarm value

Type Definitions

typedef struct gptimer_t ***gptimer_handle_t**

Type of General Purpose Timer handle.

typedef bool (***gptimer_alarm_cb_t**)(*gptimer_handle_t* timer, const *gptimer_alarm_event_data_t* *edata, void *user_ctx)

Timer alarm callback prototype.

Param timer **[in]** Timer handle created by `gptimer_new_timer`
Param edata **[in]** Alarm event data, fed by driver
Param user_ctx **[in]** User data, passed from `gptimer_register_event_callbacks`
Return Whether a high priority task has been waken up by this function

Header File

- components/hal/include/hal/timer_types.h
- This header file can be included with:

```
#include "hal/timer_types.h"
```

Type Definitions

typedef *soc_periph_gptimer_clk_src_t* **gptimer_clock_source_t**

GPTimer clock source.

Note: User should select the clock source based on the power and resolution requirement

Enumerations

enum **gptimer_count_direction_t**

GPTimer count direction.

Values:

enumerator **GPTIMER_COUNT_DOWN**

Decrease count value

enumerator **GPTIMER_COUNT_UP**

Increase count value

enum **gptimer_etm_task_type_t**

GPTimer specific tasks that supported by the ETM module.

Values:

enumerator **GPTIMER_ETM_TASK_START_COUNT**

Start the counter

enumerator **GPTIMER_ETM_TASK_STOP_COUNT**

Stop the counter

enumerator **GPTIMER_ETM_TASK_EN_ALARM**

Enable the alarm

enumerator **GPTIMER_ETM_TASK_RELOAD**

Reload preset value into counter

enumerator **GPTIMER_ETM_TASK_CAPTURE**

Capture current count value into specific register

enumerator **GPTIMER_ETM_TASK_MAX**

Maximum number of tasks

enum **gptimer_etm_event_type_t**

GPTimer specific events that supported by the ETM module.

Values:

enumerator **GPTIMER_ETM_EVENT_ALARM_MATCH**

Count value matches the alarm target value

enumerator **GPTIMER_ETM_EVENT_MAX**

Maximum number of events

2.6.7 Dedicated GPIO

Overview

The dedicated GPIO is designed for CPU interaction with GPIO matrix and IO MUX. Any GPIO that is configured as "dedicated" can be access by CPU instructions directly, which makes it easy to achieve a high GPIO flip speed, and simulate serial/parallel interface in a bit-banging way. As toggling a GPIO in this "CPU Dedicated" way costs few overhead, it would be great for cases like performance measurement using an oscilloscope.

Create/Destroy GPIO Bundle

A GPIO bundle is a group of GPIOs, which can be manipulated at the same time in one CPU cycle. The maximal number of GPIOs that a bundle can contain is limited by each CPU. What's more, the GPIO bundle has a strong relevance to the CPU which it derives from. **Any operations on the GPIO bundle should be put inside a task which is running on the same CPU core to the GPIO bundle belongs to.** Likewise, only those ISRs who are installed on the same CPU core are allowed to do operations on that GPIO bundle.

Note: Dedicated GPIO is more of a CPU peripheral, so it has a strong relationship with CPU core. It's highly recommended to install and operate GPIO bundle in a pin-to-core task. For example, if GPIOA is connected to CPU0, and the dedicated GPIO instruction is issued from CPU1, then it's impossible to control GPIOA.

To install a GPIO bundle, one needs to call *dedic_gpio_new_bundle()* to allocate the software resources and connect the dedicated channels to user selected GPIOs. Configurations for a GPIO bundle are covered in *dedic_gpio_bundle_config_t* structure:

- gpio_array: An array that contains GPIO number.
- array_size: Element number of gpio_array.
- flags: Extra flags to control the behavior of GPIO Bundle.
  - in_en and out_en are used to select whether to enable the input and output function (note, they can be enabled together).
  - in_invert and out_invert are used to select whether to invert the GPIO signal.

The following code shows how to install a output only GPIO bundle:

```
// configure GPIO
const int bundleA_gpios[] = {0, 1};
gpio_config_t io_conf = {
    .mode = GPIO_MODE_OUTPUT,
};
for (int i = 0; i < sizeof(bundleA_gpios) / sizeof(bundleA_gpios[0]); i++) {
    io_conf.pin_bit_mask = 1ULL << bundleA_gpios[i];
    gpio_config(&io_conf);
}
// Create bundleA, output only
dedic_gpio_bundle_handle_t bundleA = NULL;
dedic_gpio_bundle_config_t bundleA_config = {
    .gpio_array = bundleA_gpios,
    .array_size = sizeof(bundleA_gpios) / sizeof(bundleA_gpios[0]),
    .flags = {
        .out_en = 1,
    },
};
ESP_ERROR_CHECK(dedic_gpio_new_bundle(&bundleA_config, &bundleA));
```

To uninstall the GPIO bundle, one needs to call *dedic_gpio_del_bundle()*.

Note: *dedic_gpio_new_bundle()* doesn't cover any GPIO pad configuration (e.g., pull up/down, drive ability, output/input enable), so before installing a dedicated GPIO bundle, you have to configure the GPIO separately using GPIO driver API (e.g., *gpio_config()*). For more information about GPIO driver, please refer to *GPIO API Reference*.

GPIO Bundle Operations

Operations	Functions
Write to GPIOs in the bundle by mask	*dedic_gpio_bundle_write()*
Read the value that output from the given GPIO bundle	*dedic_gpio_bundle_read_out()*
Read the value that input to the given GPIO bundle	*dedic_gpio_bundle_read_in()*

Note: Using the above functions might not get a high GPIO flip speed because of the overhead of function calls and the bit operations involved inside. Users can try *Manipulate GPIOs by Writing Assembly Code* instead to reduce the overhead but should take care of the thread safety by themselves.

Manipulate GPIOs by Writing Assembly Code

For advanced users, they can always manipulate the GPIOs by writing assembly code or invoking CPU Low Level APIs. The usual procedure could be:

1. Allocate a GPIO bundle: *dedic_gpio_new_bundle()*
2. Query the mask occupied by that bundle: *dedic_gpio_get_out_mask()* or/and *dedic_gpio_get_in_mask()*
3. Call CPU LL apis (e.g., *dedic_gpio_cpu_ll_write_mask*) or write assembly code with that mask
4. The fastest way of toggling IO is to use the dedicated "set/clear" instructions:
   - Set bits of GPIO: `set_bit_gpio_out imm[7:0]`
   - Clear bits of GPIO: `clr_bit_gpio_out imm[7:0]`
   - Note: Immediate value width depends on the number of dedicated GPIO channels

For details of supported dedicated GPIO instructions, please refer to **ESP32-S3 Technical Reference Manual > Processor Instruction Extensions (PIE) (to be added later)** [PDF].

Some of the dedicated CPU instructions are also wrapped inside `hal/dedic_gpio_cpu_ll.h` as helper inline functions.

Note: Writing assembly code in application could make your code hard to port between targets, because those customized instructions are not guaranteed to remain the same format on different targets.

Application Example

- Software emulation (bit banging) of the UART/I2C/SPI protocols in assembly using the dedicated GPIOs and their associated CPU instructions: peripherals/dedicated_gpio.

API Reference

Header File

- components/esp_driver_gpio/include/driver/dedic_gpio.h
- This header file can be included with:

```
#include "driver/dedic_gpio.h"
```

- This header file is a part of the API provided by the `esp_driver_gpio` component. To declare that your component depends on `esp_driver_gpio`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_gpio
```

or

```
PRIV_REQUIRES esp_driver_gpio
```

Functions

esp_err_t **dedic_gpio_get_out_mask**(*dedic_gpio_bundle_handle_t* bundle, uint32_t *mask)

Get allocated channel mask.

Note: Each bundle should have at least one mask (in or/and out), based on bundle configuration.

Note: With the returned mask, user can directly invoke LL function like "dedic_gpio_cpu_ll_write_mask" or write assembly code with dedicated GPIO instructions, to get better performance on GPIO manipulation.

Parameters

- **bundle** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"

- **mask** -- **[out]** Returned mask value for on specific direction (in or out)

Returns

- ESP_OK: Get channel mask successfully
- ESP_ERR_INVALID_ARG: Get channel mask failed because of invalid argument
- ESP_FAIL: Get channel mask failed because of other error

esp_err_t **dedic_gpio_get_in_mask**(*dedic_gpio_bundle_handle_t* bundle, uint32_t *mask)

esp_err_t **dedic_gpio_get_out_offset**(*dedic_gpio_bundle_handle_t* bundle, uint32_t *offset)

Get the channel offset of the GPIO bundle.

A GPIO bundle maps the GPIOS of a particular direction to a consecutive set of channels within a particular GPIO bank of a particular CPU. This function returns the offset to the bundle's first channel of a particular direction within the bank.

Parameters

- **bundle** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"
- **offset** -- **[out]** Offset value to the first channel of a specific direction (in or out)

Returns

- ESP_OK: Get channel offset successfully
- ESP_ERR_INVALID_ARG: Get channel offset failed because of invalid argument
- ESP_FAIL: Get channel offset failed because of other error

esp_err_t **dedic_gpio_get_in_offset**(*dedic_gpio_bundle_handle_t* bundle, uint32_t *offset)

esp_err_t **dedic_gpio_new_bundle**(const *dedic_gpio_bundle_config_t* *config, *dedic_gpio_bundle_handle_t* *ret_bundle)

Create GPIO bundle and return the handle.

Note: One has to enable at least input or output mode in "config" parameter.

Parameters

- **config** -- **[in]** Configuration of GPIO bundle
- **ret_bundle** -- **[out]** Returned handle of the new created GPIO bundle

Returns

- ESP_OK: Create GPIO bundle successfully
- ESP_ERR_INVALID_ARG: Create GPIO bundle failed because of invalid argument
- ESP_ERR_NO_MEM: Create GPIO bundle failed because of no capable memory
- ESP_ERR_NOT_FOUND: Create GPIO bundle failed because of no enough continuous dedicated channels
- ESP_FAIL: Create GPIO bundle failed because of other error

esp_err_t **dedic_gpio_del_bundle**(*dedic_gpio_bundle_handle_t* bundle)

Destroy GPIO bundle.

Parameters **bundle** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"

Returns

- ESP_OK: Destroy GPIO bundle successfully
- ESP_ERR_INVALID_ARG: Destroy GPIO bundle failed because of invalid argument
- ESP_FAIL: Destroy GPIO bundle failed because of other error

void **dedic_gpio_bundle_write**(*dedic_gpio_bundle_handle_t* bundle, uint32_t mask, uint32_t value)

Write value to GPIO bundle.

Note: The mask is seen from the view of GPIO bundle. For example, bundleA contains [GPIO10, GPIO12, GPIO17], to set GPIO17 individually, the mask should be 0x04.

Note: For performance reasons, this function doesn't check the validity of any parameters, and is placed in IRAM.

Parameters

- **`bundle`** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"
- **`mask`** -- **[in]** Mask of the GPIOs to be written in the given bundle
- **`value`** -- **[in]** Value to write to given GPIO bundle, low bit represents low member in the bundle

uint32_t **`dedic_gpio_bundle_read_out`**(*dedic_gpio_bundle_handle_t* bundle)

Read the value that output from the given GPIO bundle.

Note: For performance reasons, this function doesn't check the validity of any parameters, and is placed in IRAM.

Parameters **`bundle`** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"

Returns Value that output from the GPIO bundle, low bit represents low member in the bundle

uint32_t **`dedic_gpio_bundle_read_in`**(*dedic_gpio_bundle_handle_t* bundle)

Read the value that input to the given GPIO bundle.

Note: For performance reasons, this function doesn't check the validity of any parameters, and is placed in IRAM.

Parameters **`bundle`** -- **[in]** Handle of GPIO bundle that returned from "dedic_gpio_new_bundle"

Returns Value that input to the GPIO bundle, low bit represents low member in the bundle

Structures

struct **`dedic_gpio_bundle_config_t`**

Type of Dedicated GPIO bundle configuration.

Public Members

const int ***`gpio_array`**

Array of GPIO numbers, gpio_array[0] ~ gpio_array[size-1] <=> low_dedic_channel_num ~ high_dedic_channel_num

size_t **`array_size`**

Number of GPIOs in gpio_array

unsigned int **`in_en`**

Enable input

unsigned int **`in_invert`**

Invert input signal

unsigned int **out_en**

Enable output

unsigned int **out_invert**

Invert output signal

struct *dedic_gpio_bundle_config_t*::[anonymous] **flags**

Flags to control specific behaviour of GPIO bundle

Type Definitions

typedef struct dedic_gpio_bundle_t ***dedic_gpio_bundle_handle_t**

Type of Dedicated GPIO bundle.

2.6.8 Hash-Based Message Authentication Code (HMAC)

Hash-based Message Authentication Code (HMAC) is a secure authentication technique that verifies the authenticity and integrity of a message with a pre-shared key. This module provides hardware acceleration for SHA256-HMAC generation using a key burned into an eFuse block.

For more detailed information on the application workflow and the HMAC calculation process, see **ESP32-S3 Technical Reference Manual > HMAC Accelerator (HMAC)** [PDF].

Generalized Application Scheme

Let there be two parties, A and B. They want to verify the authenticity and integrity of messages sent between each other. Before they can start sending messages, they need to exchange the secret key via a secure channel.

To verify A's messages, B can do the following:

- A calculates the HMAC of the message it wants to send.
- A sends the message and the HMAC to B.
- B calculates the HMAC of the received message itself.
- B checks whether the received and calculated HMACs match.

If they do match, the message is authentic.

However, the HMAC itself is not bound to this use case. It can also be used for challenge-response protocols supporting HMAC or as a key input for further security modules (see below), etc.

HMAC on ESP32-S3

On ESP32-S3, the HMAC module works with a secret key burnt into the eFuses. This eFuse key can be made completely inaccessible for any resources outside the cryptographic modules, thus avoiding key leakage.

Furthermore, ESP32-S3 has three different application scenarios for its HMAC module:

1. HMAC is generated for software use
2. HMAC is used as a key for the Digital Signature (DS) module
3. HMAC is used for enabling the soft-disabled JTAG interface

The first mode is called **Upstream** mode, while the last two modes are called **Downstream** modes.

eFuse Keys for HMAC Six physical eFuse blocks can be used as keys for the HMAC module: block 4 ~ block 9. The enum *hmac_key_id_t* in the API maps them to HMAC_KEY0 ~ HMAC_KEY5.

Each key has a corresponding eFuse parameter **key purpose** determining for which of the three HMAC application scenarios (see below) the key may be used:

Key Purpose	Application Scenario
8	HMAC generated for software use
7	HMAC used as a key for the Digital Signature (DS) module
6	HMAC used for enabling the soft-disabled JTAG interface
5	HMAC both as a key for the DS module and for enabling JTAG

This is to prevent the usage of a key for a different function than originally intended.

To calculate an HMAC, the software has to provide the ID of the key block containing the secret key as well as the **key purpose** (see **ESP32-S3 Technical Reference Manual > eFuse Controller (eFuse)** [PDF]).

Before the HMAC key calculation, the HMAC module looks up the purpose of the provided key block. The calculation only proceeds if the purpose of the provided key block matches the purpose stored in the eFuses of the key block provided by the ID.

HMAC Generation for Software Key purpose value: 8

In this case, the HMAC is given out to the software, e.g., to authenticate a message.

The API to calculate the HMAC is *esp_hmac_calculate()*. The input arguments for the function are the message, message length, and the eFuse key block ID which contains the secret and has the efuse key purpose set to Upstream mode.

HMAC for Digital Signature Key purpose values: 7, 5

The HMAC can be used as a key derivation function to decrypt private key parameters which are used by the Digital Signature module. A standard message is used by the hardware in that case. You only need to provide the eFuse key block and purpose on the HMAC side, additional parameters are required for the Digital Signature component in that case.

Neither the key nor the actual HMAC is ever exposed outside the HMAC module and DS component. The calculation of the HMAC and its handover to the DS component happen internally.

For more details, see **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF].

HMAC for Enabling JTAG Key purpose values: 6, 5

The third application is using the HMAC as a key to enable JTAG if it was soft-disabled before.

Following is the procedure to re-enable the JTAG:

Stage 1: Setup

1. Generate a 256-bit HMAC secret key to use for JTAG re-enable.
2. Write the key to an eFuse block with key purpose HMAC_DOWN_ALL (5) or HMAC_DOWN_JTAG (6). This can be done using the `esp_efuse_write_key()` function in the firmware or using `espefuse.py` from the host.
3. Configure the eFuse key block to be read-protected using the `esp_efuse_set_read_protect()`, so that software cannot read back the value.
4. Burn the `soft JTAG disable` bit/bits on ESP32-S3. This will permanently disable JTAG unless the correct key value is provided by the software.

Note: The API **esp_efuse_write_field_cnt(ESP_EFUSE_SOFT_DIS_JTAG, ESP_EFUSE_SOFT_DIS_JTAG[0]->bit_count)** can be used to burn `soft JTAG disable` bits on ESP32-S3.

Note: If `HARD_DIS_JTAG` eFuse is set, then `SOFT_DIS_JTAG` functionality does not work because JTAG is permanently disabled.

JTAG enables

1. The key to re-enable JTAG is the output of the HMAC-SHA256 function using the secret key in eFuse and 32 `0x00` bytes as the message.
2. Pass this key value when calling the *`esp_hmac_jtag_enable()`* function from the firmware.
3. To re-disable JTAG in the firmware, reset the system or call *`esp_hmac_jtag_disable()`*.

End-to-end example of soft disable and re-enable JTAG workflow: security/hmac_soft_jtag.

For more details, see **ESP32-S3 Technical Reference Manual > HMAC Accelerator (HMAC)** [PDF].

Application Outline

The following code is an outline of how to set an eFuse key and then use it to calculate an HMAC for software usage.

We use `esp_efuse_write_key` to set physical key block 4 in the eFuse for the HMAC module together with its purpose. `ESP_EFUSE_KEY_PURPOSE_HMAC_UP` (8) means that this key can only be used for HMAC generation for software usage:

```
#include "esp_efuse.h"

const uint8_t key_data[32] = { ... };

esp_err_t status = esp_efuse_write_key(EFUSE_BLK_KEY4,
                    ESP_EFUSE_KEY_PURPOSE_HMAC_UP,
                    key_data, sizeof(key_data));

if (status == ESP_OK) {
    // written key
} else {
    // writing key failed, maybe written already
}
```

Now we can use the saved key to calculate an HMAC for software usage.

```
#include "esp_hmac.h"

uint8_t hmac[32];

const char *message = "Hello, HMAC!";
const size_t msg_len = 12;

esp_err_t result = esp_hmac_calculate(HMAC_KEY4, message, msg_len, hmac);

if (result == ESP_OK) {
    // HMAC written to hmac now
} else {
    // failure calculating HMAC
}
```

API Reference

Header File

- components/esp_hw_support/include/esp_hmac.h
- This header file can be included with:

```
#include "esp_hmac.h"
```

Functions

esp_err_t **esp_hmac_calculate**(*hmac_key_id_t* key_id, const void *message, size_t message_len, uint8_t *hmac)

Calculate the HMAC of a given message.

Calculate the HMAC `hmac` of a given message `message` with length `message_len`. SHA256 is used for the calculation.

Note: Uses the HMAC peripheral in "upstream" mode.

Parameters

- **key_id** -- Determines which of the 6 key blocks in the efuses should be used for the HMAC calcuation. The corresponding purpose field of the key block in the efuse must be set to the HMAC upstream purpose value.
- **message** -- the message for which to calculate the HMAC
- **message_len** -- message length return ESP_ERR_INVALID_STATE if unsuccessful
- **hmac** -- **[out]** the hmac result; the buffer behind the provided pointer must be a writeable buffer of 32 bytes

Returns

- ESP_OK, if the calculation was successful,
- ESP_ERR_INVALID_ARG if message or hmac is a nullptr or if key_id out of range
- ESP_FAIL, if the hmac calculation failed

esp_err_t **esp_hmac_jtag_enable**(*hmac_key_id_t* key_id, const uint8_t *token)

Use HMAC peripheral in Downstream mode to re-enable the JTAG, if it is not permanently disabled by HW. In downstream mode, HMAC calculations performed by peripheral are used internally and not provided back to user.

Note: Return value of the API does not indicate the JTAG status.

Parameters

- **key_id** -- Determines which of the 6 key blocks in the efuses should be used for the HMAC calculation. The corresponding purpose field of the key block in the efuse must be set to HMAC downstream purpose.
- **token** -- Pre calculated HMAC value of the 32-byte 0x00 using SHA-256 and the known private HMAC key. The key is already programmed to a eFuse key block. The key block number is provided as the first parameter to this function.

Returns

- ESP_OK, if the key_purpose of the key_id matches to HMAC downstread mode, The API returns success even if calculated HMAC does not match with the provided token. However, The JTAG will be re-enabled only if the calculated HMAC value matches with provided token, otherwise JTAG will remain disabled.
- ESP_FAIL, if the key_purpose of the key_id is not set to HMAC downstream purpose or JTAG is permanently disabled by EFUSE_HARD_DIS_JTAG eFuse parameter.
- ESP_ERR_INVALID_ARG, invalid input arguments

esp_err_t **esp_hmac_jtag_disable**(void)

Disable the JTAG which might be enabled using the HMAC downstream mode. This function just clears the result generated by calling esp_hmac_jtag_enable() API.

Returns

- ESP_OK return ESP_OK after writing the HMAC_SET_INVALIDATE_JTAG_REG with value 1.

Enumerations

enum **hmac_key_id_t**

The possible efuse keys for the HMAC peripheral

Values:

enumerator **HMAC_KEY0**

enumerator **HMAC_KEY1**

enumerator **HMAC_KEY2**

enumerator **HMAC_KEY3**

enumerator **HMAC_KEY4**

enumerator **HMAC_KEY5**

enumerator **HMAC_KEY_MAX**

2.6.9 Digital Signature (DS)

The Digital Signature (DS) module provides hardware acceleration of signing messages based on RSA. It uses pre-encrypted parameters to calculate a signature. The parameters are encrypted using HMAC as a key-derivation function. In turn, the HMAC uses eFuses as the input key. The whole process happens in hardware so that neither the decryption key for the RSA parameters nor the input key for the HMAC key derivation function can be seen by the software while calculating the signature.

For more detailed information on the hardware involved in the signature calculation and the registers used, see **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF].

Private Key Parameters

The private key parameters for the RSA signature are stored in flash. To prevent unauthorized access, they are AES-encrypted. The HMAC module is used as a key-derivation function to calculate the AES encryption key for the private key parameters. In turn, the HMAC module uses a key from the eFuses key block which can be read-protected to prevent unauthorized access as well.

Upon signature calculation invocation, the software only specifies which eFuse key to use, the corresponding eFuse key purpose, the location of the encrypted RSA parameters, and the message.

Key Generation

Both the HMAC key and the RSA private key have to be created and stored before the DS peripheral can be used. This needs to be done in software on the ESP32-S3 or alternatively on a host. For this context, ESP-IDF provides *esp_efuse_write_block()* to set the HMAC key and *esp_hmac_calculate()* to encrypt the private RSA key parameters.

You can find instructions on how to calculate and assemble the private key parameters in **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF].

Signature Calculation with ESP-IDF

For more detailed information on the workflow and the registers used, see **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF].

Three parameters need to be prepared to calculate the digital signature:

1. The eFuse key block ID which is used as the key for the HMAC
2. The location of the encrypted private key parameters
3. The message to be signed

Since the signature calculation takes some time, there are two possible API versions to use in ESP-IDF. The first one is *esp_ds_sign()* and simply blocks until the calculation is finished. If software needs to do something else during the calculation, *esp_ds_start_sign()* can be called, followed by periodic calls to *esp_ds_is_busy()* to check when the calculation has finished. Once the calculation has finished, *esp_ds_finish_sign()* can be called to get the resulting signature.

The APIs *esp_ds_sign()* and *esp_ds_start_sign()* calculate a plain RSA signature with the help of the DS peripheral. This signature needs to be converted to an appropriate format for further use. For example, the MbedTLS SSL stack supports PKCS#1 format. The API `esp_ds_rsa_sign()` can be used to obtain the signature directly in the PKCS#1 v1.5 format. It internally uses *esp_ds_start_sign()* and converts the signature into PKCS#1 v1.5 format.

Note: This is only the basic DS building block, the message length is fixed. To create signatures of arbitrary messages, the input is normally a hash of the actual message, padded up to the required length. An API to do this is planned in the future.

Configure the DS Peripheral for a TLS Connection

The DS peripheral on ESP32-S3 chip must be configured before it can be used for a TLS connection. The configuration involves the following steps:

1) Randomly generate a 256-bit value called the `Initialization Vector` (IV).
2) Randomly generate a 256-bit value called the `HMAC_KEY`.
3) Calculate the encrypted private key parameters from the client private key (RSA) and the parameters generated in the above steps.
4) Then burn the 256-bit `HMAC_KEY` on the eFuse, which can only be read by the DS peripheral.

For more details, see **ESP32-S3 Technical Reference Manual > Digital Signature (DS)** [PDF].

To configure the DS peripheral for development purposes, you can use the esp-secure-cert-tool.

The encrypted private key parameters obtained after the DS peripheral configuration are then to be kept in flash. Furthermore, they are to be passed to the DS peripheral which makes use of those parameters for the Digital Signature operation. The application then needs to read the DS data from the flash, which has been done through the APIs provided by the esp_secure_cert_mgr component. Please refer to the component/README for more details.

The process of initializing the DS peripheral and then performing the Digital Signature operation is done internally with the help of *ESP-TLS*. Please refer to *Digital Signature with ESP-TLS* for more details.

As mentioned in the *ESP-TLS* documentation, the application only needs to provide the encrypted private key parameters to the esp_tls context (as *ds_data*), which internally performs all necessary operations for initializing the DS peripheral and then performing the DS operation.

Example for SSL Mutual Authentication Using DS

The example protocols/mqtt/ssl_ds shows how to use the DS peripheral for mutual authentication. The example uses *mqtt_client* (Implemented through *ESP-MQTT*) to connect to broker `test.mosquitto.org` using SSL transport with mutual authentication. The SSL part is internally performed with *ESP-TLS*. See protocols/mqtt/ssl_ds/README.md for more details.

API Reference

Header File

- components/esp_hw_support/include/esp_ds.h
- This header file can be included with:

```
#include "esp_ds.h"
```

Functions

esp_err_t **esp_ds_sign**(const void *message, const *esp_ds_data_t* *data, *hmac_key_id_t* key_id, void *signature)

Sign the message with a hardware key from specific key slot. The function calculates a plain RSA signature with help of the DS peripheral. The RSA encryption operation is as follows: Z = XY mod M where, Z is the signature, X is the input message, Y and M are the RSA private key parameters.

This function is a wrapper around `esp_ds_finish_sign()` and `esp_ds_start_sign()`, so do not use them in parallel. It blocks until the signing is finished and then returns the signature.

Note: Please see note section of `esp_ds_start_sign()` for more details about the input parameters.

Parameters

- **`message`** -- the message to be signed; its length should be (data->rsa_length + 1)*4 bytes, and those bytes must be in little endian format. It is your responsibility to apply your hash function and padding before calling this function, if required. (e.g. message = padding(hash(inputMsg)))
- **`data`** -- the encrypted signing key data (AES encrypted RSA key + IV)
- **`key_id`** -- the HMAC key ID determining the HMAC key of the HMAC which will be used to decrypt the signing key data
- **`signature`** -- the destination of the signature, should be (data->rsa_length + 1)*4 bytes long

Returns

- ESP_OK if successful, the signature was written to the parameter `signature`.
- ESP_ERR_INVALID_ARG if one of the parameters is NULL or data->rsa_length is too long or 0
- ESP_ERR_HW_CRYPTO_DS_HMAC_FAIL if there was an HMAC failure during retrieval of the decryption key
- ESP_ERR_NO_MEM if there hasn't been enough memory to allocate the context object
- ESP_ERR_HW_CRYPTO_DS_INVALID_KEY if there's a problem with passing the HMAC key to the DS component
- ESP_ERR_HW_CRYPTO_DS_INVALID_DIGEST if the message digest didn't match; the signature is invalid.
- ESP_ERR_HW_CRYPTO_DS_INVALID_PADDING if the message padding is incorrect, the signature can be read though since the message digest matches.

esp_err_t **esp_ds_start_sign**(const void *message, const *esp_ds_data_t* *data, *hmac_key_id_t* key_id, *esp_ds_context_t* **esp_ds_ctx)

Start the signing process.

This function yields a context object which needs to be passed to `esp_ds_finish_sign()` to finish the signing process. The function calculates a plain RSA signature with help of the DS peripheral. The RSA encryption operation is as follows: Z = XY mod M where, Z is the signature, X is the input message, Y and M are the RSA private key parameters.

Note: This function locks the HMAC, SHA, AES and RSA components, so the user has to ensure to call `esp_ds_finish_sign()` in a timely manner. The numbers Y, M, Rb which are a part of esp_ds_data_t should be provided in little endian format and should be of length equal to the RSA private key bit length The message length in bits should also be equal to the RSA private key bit length. No padding is applied to the message automatically, Please ensure the message is appropriate padded before calling the API.

Parameters

- **message** -- the message to be signed; its length should be (data->rsa_length + 1)*4 bytes, and those bytes must be in little endian format. It is your responsibility to apply your hash function and padding before calling this function, if required. (e.g. message = padding(hash(inputMsg)))
- **data** -- the encrypted signing key data (AES encrypted RSA key + IV)
- **key_id** -- the HMAC key ID determining the HMAC key of the HMAC which will be used to decrypt the signing key data
- **esp_ds_ctx** -- the context object which is needed for finishing the signing process later

Returns

- ESP_OK if successful, the ds operation was started now and has to be finished with `esp_ds_finish_sign()`
- ESP_ERR_INVALID_ARG if one of the parameters is NULL or data->rsa_length is too long or 0
- ESP_ERR_HW_CRYPTO_DS_HMAC_FAIL if there was an HMAC failure during retrieval of the decryption key
- ESP_ERR_NO_MEM if there hasn't been enough memory to allocate the context object
- ESP_ERR_HW_CRYPTO_DS_INVALID_KEY if there's a problem with passing the HMAC key to the DS component

bool **esp_ds_is_busy**(void)

Return true if the DS peripheral is busy, otherwise false.

Note: Only valid if `esp_ds_start_sign()` was called before.

---

esp_err_t **esp_ds_finish_sign**(void *signature, *esp_ds_context_t* *esp_ds_ctx)

Finish the signing process.

Parameters

- **signature** -- the destination of the signature, should be (data->rsa_length + 1)*4 bytes long, the resultant signature bytes shall be written in little endian format.
- **esp_ds_ctx** -- the context object retreived by `esp_ds_start_sign()`

Returns

- ESP_OK if successful, the ds operation has been finished and the result is written to signature.
- ESP_ERR_INVALID_ARG if one of the parameters is NULL
- ESP_ERR_HW_CRYPTO_DS_INVALID_DIGEST if the message digest didn't match; the signature is invalid. This means that the encrypted RSA key parameters are invalid, indicating that they may have been tampered with or indicating a flash error, etc.

- ESP_ERR_HW_CRYPTO_DS_INVALID_PADDING if the message padding is incorrect, the signature can be read though since the message digest matches (see TRM for more details).

esp_err_t **esp_ds_encrypt_params**(*esp_ds_data_t* *data, const void *iv, const *esp_ds_p_data_t* *p_data, const void *key)

Encrypt the private key parameters.

The encryption is a prerequisite step before any signature operation can be done. It is not strictly necessary to use this encryption function, the encryption could also happen on an external device.

Note: The numbers Y, M, Rb which are a part of esp_ds_data_t should be provided in little endian format and should be of length equal to the RSA private key bit length The message length in bits should also be equal to the RSA private key bit length. No padding is applied to the message automatically, Please ensure the message is appropriate padded before calling the API.

Parameters

- **data** -- Output buffer to store encrypted data, suitable for later use generating signatures.
- **iv** -- Pointer to 16 byte IV buffer, will be copied into 'data'. Should be randomly generated bytes each time.
- **p_data** -- Pointer to input plaintext key data. The expectation is this data will be deleted after this process is done and 'data' is stored.
- **key** -- Pointer to 32 bytes of key data. Type determined by key_type parameter. The expectation is the corresponding HMAC key will be stored to efuse and then permanently erased.

Returns

- ESP_OK if successful, the ds operation has been finished and the result is written to signature.
- ESP_ERR_INVALID_ARG if one of the parameters is NULL or p_data->rsa_length is too long

Structures

struct **esp_digital_signature_data**

Encrypted private key data. Recommended to store in flash in this format.

Note: This struct has to match to one from the ROM code! This documentation is mostly taken from there.

Public Members

esp_digital_signature_length_t **rsa_length**

RSA LENGTH register parameters (number of words in RSA key & operands, minus one).

This value must match the length field encrypted and stored in 'c', or invalid results will be returned. (The DS peripheral will always use the value in 'c', not this value, so an attacker can't alter the DS peripheral results this way, it will just truncate or extend the message and the resulting signature in software.)

Note: In IDF, the enum type length is the same as of type unsigned, so they can be used interchangably. See the ROM code for the original declaration of struct `ets_ds_data_t`.

---

uint32_t **iv**[ESP_DS_IV_BIT_LEN / 32]

IV value used to encrypt 'c'

uint8_t **c**[ESP_DS_C_LEN]

Encrypted Digital Signature parameters. Result of AES-CBC encryption of plaintext values. Includes an encrypted message digest.

struct **esp_ds_p_data_t**

Plaintext parameters used by Digital Signature.

This is only used for encrypting the RSA parameters by calling esp_ds_encrypt_params(). Afterwards, the result can be stored in flash or in other persistent memory. The encryption is a prerequisite step before any signature operation can be done.

Note: Y, M, Rb, & M_Prime must all be in little endian format.

Public Members

uint32_t **Y**[ESP_DS_SIGNATURE_MAX_BIT_LEN / 32]

RSA exponent.

uint32_t **M**[ESP_DS_SIGNATURE_MAX_BIT_LEN / 32]

RSA modulus.

uint32_t **Rb**[ESP_DS_SIGNATURE_MAX_BIT_LEN / 32]

RSA r inverse operand.

uint32_t **M_prime**

RSA M prime operand.

uint32_t **length**

RSA length in words (32 bit)

Macros

ESP_DS_IV_BIT_LEN

ESP_DS_IV_LEN

ESP_DS_SIGNATURE_MAX_BIT_LEN

ESP_DS_SIGNATURE_MD_BIT_LEN

ESP_DS_SIGNATURE_M_PRIME_BIT_LEN

ESP_DS_SIGNATURE_L_BIT_LEN

ESP_DS_SIGNATURE_PADDING_BIT_LEN

ESP_DS_C_LEN

Type Definitions

typedef struct esp_ds_context **esp_ds_context_t**

typedef struct *esp_digital_signature_data* **esp_ds_data_t**

Encrypted private key data. Recommended to store in flash in this format.

Note: This struct has to match to one from the ROM code! This documentation is mostly taken from there.

Enumerations

enum **esp_digital_signature_length_t**

Values:

enumerator **ESP_DS_RSA_1024**

enumerator **ESP_DS_RSA_2048**

enumerator **ESP_DS_RSA_3072**

enumerator **ESP_DS_RSA_4096**

2.6.10 Inter-Integrated Circuit (I2C)

Introduction

I2C is a serial, synchronous, multi-device, half-duplex communication protocol that allows co-existence of multiple masters and slaves on the same bus. I2C uses two bidirectional open-drain lines: serial data line (SDA) and serial clock line (SCL), pulled up by resistors.

ESP32-S3 has 2 I2C controller (also called port), responsible for handling communication on the I2C bus. A single I2C controller can be a master or a slave.

Typically, an I2C slave device has a 7-bit address or 10-bit address. ESP32-S3 supports both I2C Standard-mode (Sm) and Fast-mode (Fm) which can go up to 100KHz and 400KHz respectively.

Warning: The clock frequency of SCL in master mode should not be larger than 400 KHz

---

Note: The frequency of SCL is influenced by both the pull-up resistor and the wire capacitance. Therefore, users are strongly recommended to choose appropriate pull-up resistors to make the frequency accurate. The recommended value for pull-up resistors usually ranges from 1K Ohms to 10K Ohms.

Keep in mind that the higher the frequency, the smaller the pull-up resistor should be (but not less than 1 KOhms). Indeed, large resistors will decline the current, which will increase the clock switching time and reduce the frequency. We usually recommend a range of 2 KOhms to 5 KOhms, but users may also need to make some adjustments depending on their current draw requirements.

I2C Clock Configuration

- `i2c_clock_source_t::I2C_CLK_SRC_DEFAULT`: Default I2C source clock.
- `i2c_clock_source_t::I2C_CLK_SRC_XTAL`: External crystal for I2C clock source.
- `i2c_clock_source_t::I2C_CLK_RC_FAST`: Internal 20MHz rc oscillator for I2C clock source.

I2C File Structure

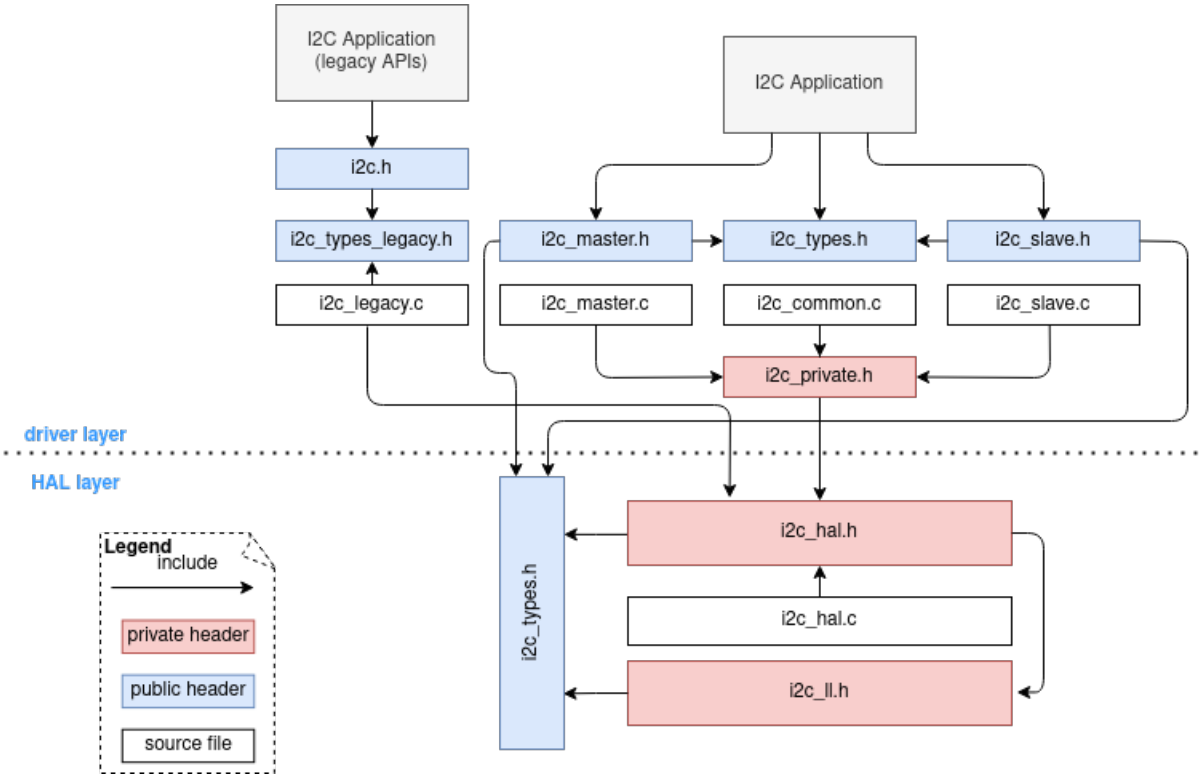


Fig. 5: I2C file structure

Public headers that need to be included in the I2C application

- `i2c.h`: The header file of legacy I2C APIs (for apps using legacy driver).
- `i2c_master.h`: The header file that provides standard communication mode specific APIs (for apps using new driver with master mode).
- `i2c_slave.h`: The header file that provides standard communication mode specific APIs (for apps using new driver with slave mode).

Note: The legacy driver can't coexist with the new driver. Include `i2c.h` to use the legacy driver or the other two headers to use the new driver. Please keep in mind that the legacy driver is now deprecated and will be removed in future.

Public headers that have been included in the headers above

- `i2c_types_legacy.h`: The legacy public types that only used in the legacy driver.
- `i2c_types.h`: The header file that provides public types.

Functional Overview

The I2C driver offers following services:

- *Resource Allocation* - covers how to allocate I2C bus with properly set of configurations. It also covers how to recycle the resources when they finished working.

- *I2C Master Controller* - covers behavior of I2C master controller. Introduce data transmit, data receive, and data transmit and receive.
- *I2C Slave Controller* - covers behavior of I2C slave controller. Involve data transmit and data receive.
- *Power Management* - describes how different source clock will affect power consumption.
- *IRAM Safe* - describes tips on how to make the I2C interrupt work better along with a disabled cache.
- *Thread Safety* - lists which APIs are guaranteed to be thread safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can bring different effects to the driver.

Resource Allocation Both I2C master bus and I2C slave bus, when supported, are represented by `i2c_bus_handle_t` in the driver. The available ports are managed in a resource pool that allocates a free port on request.

Install I2C master bus and device The I2C master is designed based on bus-device model. So `i2c_master_bus_config_t` and `i2c_device_config_t` are required separately to allocate the I2C master bus instance and I2C device instance.

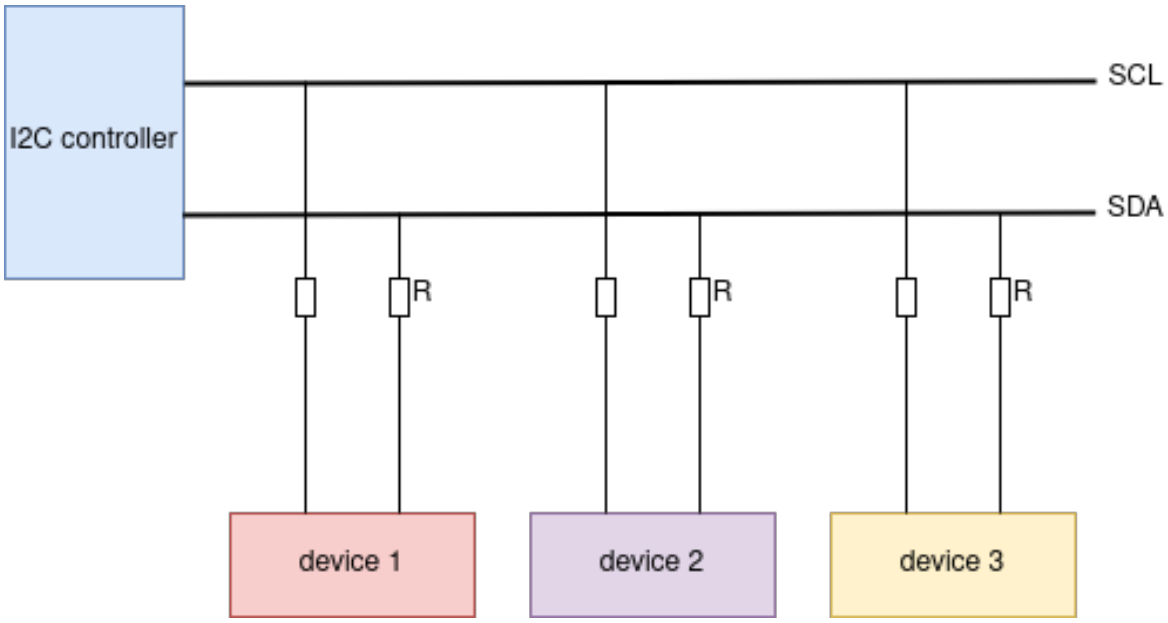


Fig. 6: I2C master bus-device module

I2C master bus requires the configuration that specified by `i2c_master_bus_config_t`:

- `i2c_master_bus_config_t::i2c_port` sets the I2C port used by the controller.
- `i2c_master_bus_config_t::sda_io_num` sets the GPIO number for the serial data bus (SDA).
- `i2c_master_bus_config_t::scl_io_num` sets the GPIO number for the serial clock bus (SCL).
- `i2c_master_bus_config_t::clk_source` selects the source clock for I2C bus. The available clocks are listed in `i2c_clock_source_t`. For the effect on power consumption of different clock source, please refer to *Power Management* section.
- `i2c_master_bus_config_t::glitch_ignore_cnt` sets the glitch period of master bus, if the glitch period on the line is less than this value, it can be filtered out, typically value is 7.
- `i2c_master_bus_config_t::intr_priority` Set the priority of the interrupt. If set to `0` , then the driver will use a interrupt with low or medium priority (priority level may be one of 1,2 or 3), otherwise use the priority indicated by `i2c_master_bus_config_t::intr_priority` Please use the number form (1,2,3) , not the bitmask form ((1<<1),(1<<2),(1<<3)).
- `i2c_master_bus_config_t::trans_queue_depth` Depth of internal transfer queue. Only valid in asynchronous transaction.
- `i2c_master_bus_config_t::enable_internal_pullup` Enable internal pullups. Note: This is not strong enough to pullup buses under high-speed frequency. A suitable external pullup is recommended.

If the configurations in *i2c_master_bus_config_t* is specified, users can call *i2c_new_master_bus()* to allocate and initialize an I2C master bus. This function will return an I2C bus handle if it runs correctly. Specifically, when there are no more I2C port available, this function will return *ESP_ERR_NOT_FOUND* error.

I2C master device requires the configuration that specified by *i2c_device_config_t*:

- *i2c_device_config_t::dev_addr_length* configure the address bit length of the slave device. User can choose from enumerator *I2C_ADDR_BIT_LEN_7* or *I2C_ADDR_BIT_LEN_10* (if supported).
- *i2c_device_config_t::device_address* I2C device raw address. Please parse the device address to this member directly. For example, the device address is 0x28, then parse 0x28 to *i2c_device_config_t::device_address*, don't carry a write/read bit.
- *i2c_device_config_t::scl_speed_hz* set the scl line frequency of this device.
- *i2c_device_config_t::scl_wait_us*. SCL await time (in us). Usually this value should not be very small because slave stretch will happen in pretty long time. (It's possible even stretch for 12ms). Set 0 means use default reg value.

Once the *i2c_device_config_t* structure is populated with mandatory parameters, users can call *i2c_master_bus_add_device()* to allocate an I2C device instance and mounted to the master bus then. This function will return an I2C device handle if it runs correctly. Specifically, when the I2C bus is not initialized properly, calling this function will result in a *ESP_ERR_INVALID_ARG* error.

```
#include "driver/i2c_master.h"

i2c_master_bus_config_t i2c_mst_config = {
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .scl_io_num = I2C_MASTER_SCL_IO,
    .sda_io_num = I2C_MASTER_SDA_IO,
    .glitch_ignore_cnt = 7,
    .flags.enable_internal_pullup = true,
};

i2c_master_bus_handle_t bus_handle;
ESP_ERROR_CHECK(i2c_new_master_bus(&i2c_mst_config, &bus_handle));

i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address = 0x58,
    .scl_speed_hz = 100000,
};

i2c_master_dev_handle_t dev_handle;
ESP_ERROR_CHECK(i2c_master_bus_add_device(bus_handle, &dev_cfg, &dev_handle));
```

Uninstall I2C master bus and device If a previously installed I2C bus or device is no longer needed, it's recommended to recycle the resource by calling *i2c_master_bus_rm_device()* or *i2c_del_master_bus()*, so that to release the underlying hardware.

Install I2C slave device I2C slave requires the configuration that specified by *i2c_slave_config_t*:

- *i2c_slave_config_t::i2c_port* sets the I2C port used by the controller.
- *i2c_slave_config_t::sda_io_num* sets the GPIO number for serial data bus (SDA).
- *i2c_slave_config_t::scl_io_num* sets the GPIO number for serial clock bus (SCL).
- *i2c_slave_config_t::clk_source* selects the source clock for I2C bus. The available clocks are listed in *i2c_clock_source_t*. For the effect on power consumption of different clock source, please refer to *Power Management* section.
- *i2c_slave_config_t::send_buf_depth* sets the sending buffer length.
- *i2c_slave_config_t::slave_addr* sets the slave address

- *i2c_master_bus_config_t::intr_priority* Set the priority of the interrupt. If set to 0 , then the driver will use a interrupt with low or medium priority (priority level may be one of 1,2 or 3), otherwise use the priority indicated by *i2c_master_bus_config_t::intr_priority* Please use the number form (1,2,3) , not the bitmask form ((1<<1),(1<<2),(1<<3)). Please pay attention that once the interrupt priority is set, it cannot be changed until *i2c_del_master_bus()* is called.
- *i2c_slave_config_t::addr_bit_len* sets true if you need the slave to have a 10-bit address.
- *i2c_slave_config_t::access_ram_en* Set true to enable the non-fifo mode. Thus the I2C data fifo can be used as RAM, and double addressing will be synchronised opened.

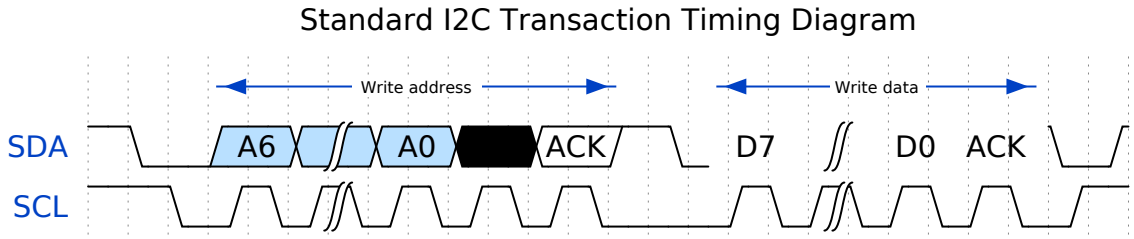
Once the *i2c_slave_config_t* structure is populated with mandatory parameters, users can call *i2c_new_slave_device()* to allocate and initialize an I2C master bus. This function will return an I2C bus handle if it runs correctly. Specifically, when there are no more I2C port available, this function will return *ESP_ERR_NOT_FOUND* error.

```
i2c_slave_config_t i2c_slv_config = {
    .addr_bit_len = I2C_ADDR_BIT_LEN_7,
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .send_buf_depth = 256,
    .scl_io_num = I2C_SLAVE_SCL_IO,
    .sda_io_num = I2C_SLAVE_SDA_IO,
    .slave_addr = 0x58,
};

i2c_slave_dev_handle_t slave_handle;
ESP_ERROR_CHECK(i2c_new_slave_device(&i2c_slv_config, &slave_handle));
```

Uninstall I2C slave device If a previously installed I2C bus is no longer needed, it's recommended to recycle the resource by calling *i2c_del_slave_device()*, so that to release the underlying hardware.

I2C Master Controller After installing the i2c master driver by *i2c_new_master_bus()*, ESP32-S3 is ready to communicate with other I2C devices. I2C APIs allow the standard transactions. Like the wave as follows:



I2C Master Write After installing I2C master bus successfully, you can simply call *i2c_master_transmit()* to write data to the slave device. The principle of this function can be explained by following chart.

In order to organize the process, the driver uses a command link, that should be populated with a sequence of commands and then passed to I2C controller for execution.

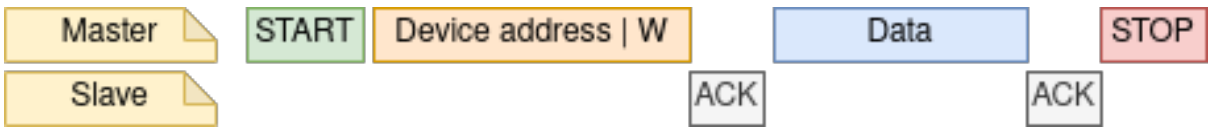


Fig. 7: I2C master write to slave

Simple example for writing data to slave:

```
#define DATA_LENGTH 100
i2c_master_bus_config_t i2c_mst_config = {
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = I2C_PORT_NUM_0,
    .scl_io_num = I2C_MASTER_SCL_IO,
    .sda_io_num = I2C_MASTER_SDA_IO,
    .glitch_ignore_cnt = 7,
};
i2c_master_bus_handle_t bus_handle;

ESP_ERROR_CHECK(i2c_new_master_bus(&i2c_mst_config, &bus_handle));

i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address = 0x58,
    .scl_speed_hz = 100000,
};

i2c_master_dev_handle_t dev_handle;
ESP_ERROR_CHECK(i2c_master_bus_add_device(bus_handle, &dev_cfg, &dev_handle));

ESP_ERROR_CHECK(i2c_master_transmit(dev_handle, data_wr, DATA_LENGTH, -1));
```

I2C Master Read After installing I2C master bus successfully, you can simply call *i2c_master_receive()* to read data from the slave device. The principle of this function can be explained by following chart.

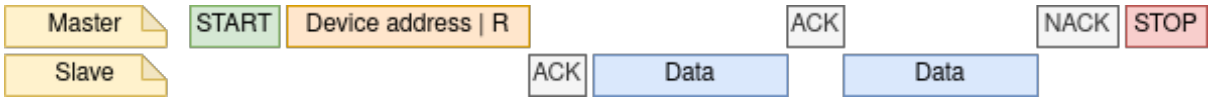


Fig. 8: I2C master read from slave

Simple example for reading data from slave:

```
#define DATA_LENGTH 100
i2c_master_bus_config_t i2c_mst_config = {
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = I2C_PORT_NUM_0,
    .scl_io_num = I2C_MASTER_SCL_IO,
    .sda_io_num = I2C_MASTER_SDA_IO,
    .glitch_ignore_cnt = 7,
};
i2c_master_bus_handle_t bus_handle;

ESP_ERROR_CHECK(i2c_new_master_bus(&i2c_mst_config, &bus_handle));

i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address = 0x58,
    .scl_speed_hz = 100000,
};

i2c_master_dev_handle_t dev_handle;
ESP_ERROR_CHECK(i2c_master_bus_add_device(bus_handle, &dev_cfg, &dev_handle));

i2c_master_receive(dev_handle, data_rd, DATA_LENGTH, -1);
```

I2C Master Write and Read Some I2C device needs write configurations before reading data from it, therefore, an interface called *i2c_master_transmit_receive()* can help. The principle of this function can be explained by following chart.

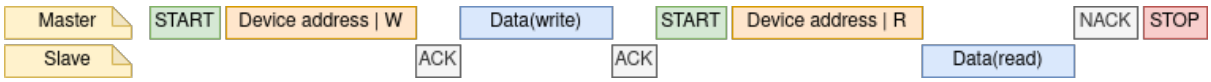


Fig. 9: I2C master write to slave and read from slave

Simple example for writing and reading from slave:

```
i2c_device_config_t dev_cfg = {
    .dev_addr_length = I2C_ADDR_BIT_LEN_7,
    .device_address = 0x58,
    .scl_speed_hz = 100000,
};

i2c_master_dev_handle_t dev_handle;
ESP_ERROR_CHECK(i2c_master_bus_add_device(I2C_PORT_NUM_0, &dev_cfg, &dev_handle));
uint8_t buf[20] = {0x20};
uint8_t buffer[2];
ESP_ERROR_CHECK(i2c_master_transmit_receive(dev_handle, buf, sizeof(buf), buffer,␣
↪2, -1));
```

I2C Master Probe I2C driver can use *i2c_master_probe()* to detect whether the specific device has been connected on I2C bus. If this function return `ESP_OK`, that means the device has been detected.

Important: Pull-ups must be connected to the SCL and SDA pins when this function is called. If you get *ESP_ERR_TIMEOUT* while *xfer_timeout_ms* was parsed correctly, you should check the pull-up resistors. If you do not have proper resistors nearby, setting *flags.enable_internal_pullup* as true is also acceptable.

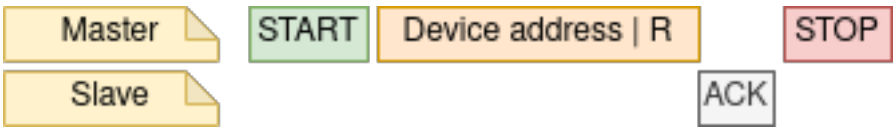


Fig. 10: I2C master probe

Simple example for probing an I2C device:

```
i2c_master_bus_config_t i2c_mst_config_1 = {
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .scl_io_num = I2C_MASTER_SCL_IO,
    .sda_io_num = I2C_MASTER_SDA_IO,
    .glitch_ignore_cnt = 7,
    .flags.enable_internal_pullup = true,
};
i2c_master_bus_handle_t bus_handle;

ESP_ERROR_CHECK(i2c_new_master_bus(&i2c_mst_config_1, &bus_handle));
ESP_ERROR_CHECK(i2c_master_probe(bus_handle, 0x22, -1));
ESP_ERROR_CHECK(i2c_del_master_bus(bus_handle));
```

I2C Slave Controller After installing the i2c slave driver by *i2c_new_slave_device()*, ESP32-S3 is ready to communicate with other I2C master as a slave.

I2C Slave Write The send buffer of the I2C slave is used as a FIFO to store the data to be sent. The data will queue up until the master requests them. You can call *i2c_slave_transmit()* to transfer data.

Simple example for writing data to FIFO:

```
uint8_t *data_wr = (uint8_t *) malloc(DATA_LENGTH);

i2c_slave_config_t i2c_slv_config = {
    .addr_bit_len = I2C_ADDR_BIT_LEN_7,    // 7-bit address
    .clk_source = I2C_CLK_SRC_DEFAULT,     // set the clock source
    .i2c_port = 0,                         // set I2C port number
    .send_buf_depth = 256,                 // set tx buffer length
    .scl_io_num = 2,                       // SCL gpio number
    .sda_io_num = 1,                       // SDA gpio number
    .slave_addr = 0x58,                    // slave address
};

i2c_bus_handle_t i2c_bus_handle;
ESP_ERROR_CHECK(i2c_new_slave_device(&i2c_slv_config, &i2c_bus_handle));
for (int i = 0; i < DATA_LENGTH; i++) {
    data_wr[i] = i;
}

ESP_ERROR_CHECK(i2c_slave_transmit(i2c_bus_handle, data_wr, DATA_LENGTH, 10000));
```

I2C Slave Read Whenever the master writes data to the slave, the slave will automatically store data in the receive buffer. This allows the slave application to call the function *i2c_slave_receive()* as its own discretion. As *i2c_slave_receive()* is designed as a non-blocking interface. So the user needs to register callback *i2c_slave_register_event_callbacks()* to know when the receive has finished.

```
static IRAM_ATTR bool i2c_slave_rx_done_callback(i2c_slave_dev_handle_t channel,␣
↪const i2c_slave_rx_done_event_data_t *edata, void *user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t receive_queue = (QueueHandle_t)user_data;
    xQueueSendFromISR(receive_queue, edata, &high_task_wakeup);
    return high_task_wakeup == pdTRUE;
}

uint8_t *data_rd = (uint8_t *) malloc(DATA_LENGTH);
uint32_t size_rd = 0;

i2c_slave_config_t i2c_slv_config = {
    .addr_bit_len = I2C_ADDR_BIT_LEN_7,
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .send_buf_depth = 256,
    .scl_io_num = I2C_SLAVE_SCL_IO,
    .sda_io_num = I2C_SLAVE_SDA_IO,
    .slave_addr = 0x58,
};

i2c_slave_dev_handle_t slave_handle;
ESP_ERROR_CHECK(i2c_new_slave_device(&i2c_slv_config, &slave_handle));

s_receive_queue = xQueueCreate(1, sizeof(i2c_slave_rx_done_event_data_t));
i2c_slave_event_callbacks_t cbs = {
    .on_recv_done = i2c_slave_rx_done_callback,
};
ESP_ERROR_CHECK(i2c_slave_register_event_callbacks(slave_handle, &cbs, s_receive_
↪queue));

i2c_slave_rx_done_event_data_t rx_data;
ESP_ERROR_CHECK(i2c_slave_receive(slave_handle, data_rd, DATA_LENGTH));
```

(continues on next page)

(continued from previous page)

```
xQueueReceive(s_receive_queue, &rx_data, pdMS_TO_TICKS(10000));
// Receive done.
```

Put Data In I2C Slave RAM I2C slave fifo mentioned above can be used as RAM, which means user can access the RAM directly via address fields. For example, writing data to the 3rd ram block with following graph. Before using this, please note that *i2c_slave_config_t::access_ram_en* needs to be set to true.

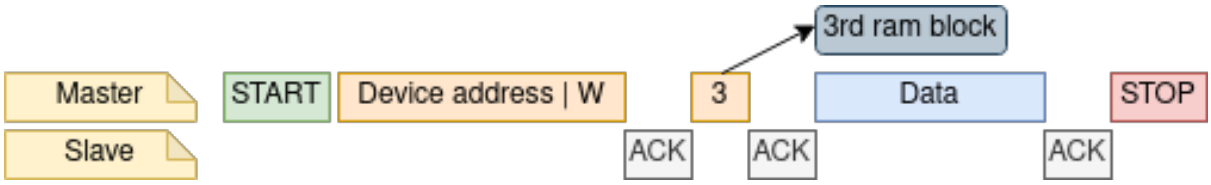


Fig. 11: Put data in I2C slave RAM

```
uint8_t data_rd[DATA_LENGTH_RAM] = {0};

i2c_slave_config_t i2c_slv_config = {
    .addr_bit_len = I2C_ADDR_BIT_LEN_7,
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .send_buf_depth = 256,
    .scl_io_num = I2C_SLAVE_SCL_IO,
    .sda_io_num = I2C_SLAVE_SDA_IO,
    .slave_addr = 0x58,
    .flags.access_ram_en = true,
};

// Master write to slave.

i2c_slave_dev_handle_t slave_handle;
ESP_ERROR_CHECK(i2c_new_slave_device(&i2c_slv_config, &slave_handle));
ESP_ERROR_CHECK(i2c_slave_read_ram(slave_handle, 0x5, data_rd, DATA_LENGTH_RAM));
ESP_ERROR_CHECK(i2c_del_slave_device(slave_handle));
```

Get Data From I2C Slave RAM Data can be stored in the RAM with a specific offset by the slave controller, and the master can read this data directly via the RAM address. For example, if the data is stored in 3rd ram block, master can read this data by following graph. Before using this, please note that *i2c_slave_config_t::access_ram_en* needs to be set to true.

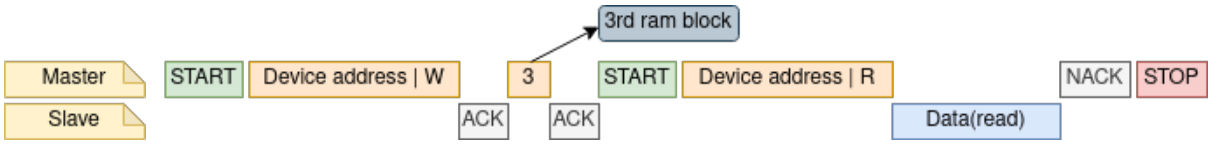


Fig. 12: Get data from I2C slave RAM

```
uint8_t data_wr[DATA_LENGTH_RAM] = {0};

i2c_slave_config_t i2c_slv_config = {
    .addr_bit_len = I2C_ADDR_BIT_LEN_7,
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .i2c_port = TEST_I2C_PORT,
    .send_buf_depth = 256,
    .scl_io_num = I2C_SLAVE_SCL_IO,
```

(continues on next page)

(continued from previous page)

```
    .sda_io_num = I2C_SLAVE_SDA_IO,
    .slave_addr = 0x58,
    .flags.access_ram_en = true,
};

i2c_slave_dev_handle_t slave_handle;
ESP_ERROR_CHECK(i2c_new_slave_device(&i2c_slv_config, &slave_handle));
ESP_ERROR_CHECK(i2c_slave_write_ram(slave_handle, 0x2, data_wr, DATA_LENGTH_RAM));
ESP_ERROR_CHECK(i2c_del_slave_device(slave_handle));
```

Register Event Callbacks

I2C master callbacks When an I2C master bus triggers an interrupt, a specific event will be generated and notify the CPU. If you have some functions that need to be called when those events occurred, you can hook your functions to the ISR (Interrupt Service Routine) by calling *i2c_master_register_event_callbacks()*. Since the registered callback functions are called in the interrupt context, user should ensure the callback function doesn't attempt to block (e.g. by making sure that only FreeRTOS APIs with `ISR` suffix are called from within the function). The callback functions are required to return a boolean value, to tell the ISR whether a high priority task is woke up by it.

I2C master event callbacks are listed in the *i2c_master_event_callbacks_t*.

Although I2C is a synchronous communication protocol, we also support asynchronous behavior by registering above callback. In this way, I2C APIs will be non-blocking interface. But note that on the same bus, only one device can adopt asynchronous operation.

Important: I2C master asynchronous transaction is still an experimental feature. (The issue is when asynchronous transaction is very large, it will cause memory problem.)

- `i2c_master_event_callbacks_t::on_recv_done` sets a callback function for master "transaction-done" event. The function prototype is declared in *i2c_master_callback_t*.

I2C slave callbacks When an I2C slave bus triggers an interrupt, a specific event will be generated and notify the CPU. If you have some function that needs to be called when those events occurred, you can hook your function to the ISR (Interrupt Service Routine) by calling *i2c_slave_register_event_callbacks()*. Since the registered callback functions are called in the interrupt context, user should ensure the callback function doesn't attempt to block (e.g. by making sure that only FreeRTOS APIs with `ISR` suffix are called from within the function). The callback function has a boolean return value, to tell the caller whether a high priority task is woke up by it.

I2C slave event callbacks are listed in the *i2c_slave_event_callbacks_t*.

- *i2c_slave_event_callbacks_t::on_recv_done* sets a callback function for "receive-done" event. The function prototype is declared in *i2c_slave_received_callback_t*.

Power Management If the controller clock source is selected to *I2C_CLK_SRC_XTAL*, then the driver won't install power management lock for it, which is more suitable for a low power application as long as the source clock can still provide sufficient resolution.

IRAM Safe By default, the I2C interrupt will be deferred when the Cache is disabled for reasons like writing/erasing Flash. Thus the event callback functions will not get executed in time, which is not expected in a real-time application.

There's a Kconfig option *CONFIG_I2C_ISR_IRAM_SAFE* that will:

1. Enable the interrupt being serviced even when cache is disabled

2. Place all functions that used by the ISR into IRAM
3. Place driver object into DRAM (in case it's mapped to PSRAM by accident)

This will allow the interrupt to run while the cache is disabled but will come at the cost of increased IRAM consumption.

Thread Safety The factory function `i2c_new_master_bus()` and `i2c_new_slave_device()` are guaranteed to be thread safe by the driver, which means, user can call them from different RTOS tasks without protection by extra locks. Other public I2C APIs are not thread safe. which means the user should avoid calling them from multiple tasks, if user strongly needs to call them in multiple tasks, please add extra lock.

Kconfig Options

- *CONFIG_I2C_ISR_IRAM_SAFE* controls whether the default ISR handler can work when cache is disabled, see also *IRAM Safe* for more information.
- *CONFIG_I2C_ENABLE_DEBUG_LOG* is used to enable the debug log at the cost of increased firmware binary size.

API Reference

Header File

- components/esp_driver_i2c/include/driver/i2c_master.h
- This header file can be included with:

```
#include "driver/i2c_master.h"
```

- This header file is a part of the API provided by the `esp_driver_i2c` component. To declare that your component depends on `esp_driver_i2c`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2c
```

or

```
PRIV_REQUIRES esp_driver_i2c
```

Functions

esp_err_t **i2c_new_master_bus**(const *i2c_master_bus_config_t* *bus_config, *i2c_master_bus_handle_t* *ret_bus_handle)

Allocate an I2C master bus.

Parameters

- **bus_config** -- **[in]** I2C master bus configuration.
- **ret_bus_handle** -- **[out]** I2C bus handle

Returns

- ESP_OK: I2C master bus initialized successfully.
- ESP_ERR_INVALID_ARG: I2C bus initialization failed because of invalid argument.
- ESP_ERR_NO_MEM: Create I2C bus failed because of out of memory.
- ESP_ERR_NOT_FOUND: No more free bus.

esp_err_t **i2c_master_bus_add_device**(*i2c_master_bus_handle_t* bus_handle, const *i2c_device_config_t* *dev_config, *i2c_master_dev_handle_t* *ret_handle)

Add I2C master BUS device.

Parameters

- **bus_handle** -- **[in]** I2C bus handle.
- **dev_config** -- **[in]** device config.
- **ret_handle** -- **[out]** device handle.

Returns

- ESP_OK: Create I2C master device successfully.
- ESP_ERR_INVALID_ARG: I2C bus initialization failed because of invalid argument.
- ESP_ERR_NO_MEM: Create I2C bus failed because of out of memory.

esp_err_t **i2c_del_master_bus**(*i2c_master_bus_handle_t* bus_handle)

Deinitialize the I2C master bus and delete the handle.

Parameters **bus_handle** -- **[in]** I2C bus handle.

Returns

- ESP_OK: Delete I2C bus success, otherwise, failed.
- Otherwise: Some module delete failed.

esp_err_t **i2c_master_bus_rm_device**(*i2c_master_dev_handle_t* handle)

I2C master bus delete device.

Parameters **handle** -- i2c device handle

Returns

- ESP_OK: If device is successfully deleted.

esp_err_t **i2c_master_transmit**(*i2c_master_dev_handle_t* i2c_dev, const uint8_t *write_buffer, size_t write_size, int xfer_timeout_ms)

Perform a write transaction on the I2C bus. The transaction will be undergoing until it finishes or it reaches the timeout provided.

Note: If a callback was registered with `i2c_master_register_event_callbacks`, the transaction will be asynchronous, and thus, this function will return directly, without blocking. You will get finish information from callback. Besides, data buffer should always be completely prepared when callback is registered, otherwise, the data will get corrupt.

Parameters

- **i2c_dev** -- **[in]** I2C master device handle that created by `i2c_master_bus_add_device`.
- **write_buffer** -- **[in]** Data bytes to send on the I2C bus.
- **write_size** -- **[in]** Size, in bytes, of the write buffer.
- **xfer_timeout_ms** -- **[in]** Wait timeout, in ms. Note: -1 means wait forever.

Returns

- ESP_OK: I2C master transmit success
- ESP_ERR_INVALID_ARG: I2C master transmit parameter invalid.
- ESP_ERR_TIMEOUT: Operation timeout(larger than xfer_timeout_ms) because the bus is busy or hardware crash.

esp_err_t **i2c_master_transmit_receive**(*i2c_master_dev_handle_t* i2c_dev, const uint8_t *write_buffer, size_t write_size, uint8_t *read_buffer, size_t read_size, int xfer_timeout_ms)

Perform a write-read transaction on the I2C bus. The transaction will be undergoing until it finishes or it reaches the timeout provided.

Note: If a callback was registered with `i2c_master_register_event_callbacks`, the transaction will be asynchronous, and thus, this function will return directly, without blocking. You will get finish information from callback. Besides, data buffer should always be completely prepared when callback is registered, otherwise, the data will get corrupt.

Parameters

- **i2c_dev** -- **[in]** I2C master device handle that created by `i2c_master_bus_add_device`.
- **write_buffer** -- **[in]** Data bytes to send on the I2C bus.

- **write_size** -- **[in]** Size, in bytes, of the write buffer.
- **read_buffer** -- **[out]** Data bytes received from i2c bus.
- **read_size** -- **[in]** Size, in bytes, of the read buffer.
- **xfer_timeout_ms** -- **[in]** Wait timeout, in ms. Note: -1 means wait forever.

Returns

- ESP_OK: I2C master transmit-receive success
- ESP_ERR_INVALID_ARG: I2C master transmit parameter invalid.
- ESP_ERR_TIMEOUT: Operation timeout(larger than xfer_timeout_ms) because the bus is busy or hardware crash.

esp_err_t **i2c_master_receive**(*i2c_master_dev_handle_t* i2c_dev, uint8_t *read_buffer, size_t read_size, int xfer_timeout_ms)

Perform a read transaction on the I2C bus. The transaction will be undergoing until it finishes or it reaches the timeout provided.

Note: If a callback was registered with `i2c_master_register_event_callbacks`, the transaction will be asynchronous, and thus, this function will return directly, without blocking. You will get finish information from callback. Besides, data buffer should always be completely prepared when callback is registered, otherwise, the data will get corrupt.

Parameters

- **i2c_dev** -- **[in]** I2C master device handle that created by `i2c_master_bus_add_device`.
- **read_buffer** -- **[out]** Data bytes received from i2c bus.
- **read_size** -- **[in]** Size, in bytes, of the read buffer.
- **xfer_timeout_ms** -- **[in]** Wait timeout, in ms. Note: -1 means wait forever.

Returns

- ESP_OK: I2C master receive success
- ESP_ERR_INVALID_ARG: I2C master receive parameter invalid.
- ESP_ERR_TIMEOUT: Operation timeout(larger than xfer_timeout_ms) because the bus is busy or hardware crash.

esp_err_t **i2c_master_probe**(*i2c_master_bus_handle_t* bus_handle, uint16_t address, int xfer_timeout_ms)

Probe I2C address, if address is correct and ACK is received, this function will return ESP_OK.

Attention Pull-ups must be connected to the SCL and SDA pins when this function is called. If you get `ESP_ERR_TIMEOUT whilexfer_timeout_mswas parsed correctly, you should check the pull-up resistors. If you do not have proper resistors nearby.` flags.enable_internal_pullup` is also acceptable.

Note: The principle of this function is to sent device address with a write command. If the device on your I2C bus, there would be an ACK signal and function returns `ESP_OK`. If the device is not on your I2C bus, there would be a NACK signal and function returns `ESP_ERR_NOT_FOUND`. `ESP_ERR_TIMEOUT` is not an expected failure, which indicated that the i2c probe not works properly, usually caused by pull-up resistors not be connected properly. Suggestion check data on SDA/SCL line to see whether there is ACK/NACK signal is on line when i2c probe function fails.

Note: There are lots of I2C devices all over the world, we assume that not all I2C device support the behavior like `device_address+nack/ack`. So, if the on line data is strange and no ack/nack got respond. Please check the device datasheet.

Parameters

- **bus_handle** -- **[in]** I2C master device handle that created by `i2c_master_bus_add_device`.
- **address** -- **[in]** I2C device address that you want to probe.
- **xfer_timeout_ms** -- **[in]** Wait timeout, in ms. Note: -1 means wait forever (Not recommended in this function).

Returns

- ESP_OK: I2C device probe successfully
- ESP_ERR_NOT_FOUND: I2C probe failed, doesn't find the device with specific address you gave.
- ESP_ERR_TIMEOUT: Operation timeout(larger than xfer_timeout_ms) because the bus is busy or hardware crash.

esp_err_t **i2c_master_register_event_callbacks**(*i2c_master_dev_handle_t* i2c_dev, const *i2c_master_event_callbacks_t* *cbs, void *user_data)

Register I2C transaction callbacks for a master device.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Note: When CONFIG_I2C_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well. The `user_data` should also reside in SRAM.

Note: If the callback is used for helping asynchronous transaction. On the same bus, only one device can be used for performing asynchronous operation.

Parameters

- **i2c_dev** -- **[in]** I2C master device handle that created by `i2c_master_bus_add_device`.
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set I2C transaction callbacks successfully
- ESP_ERR_INVALID_ARG: Set I2C transaction callbacks failed because of invalid argument
- ESP_FAIL: Set I2C transaction callbacks failed because of other error

esp_err_t **i2c_master_bus_reset**(*i2c_master_bus_handle_t* bus_handle)

Reset the I2C master bus.

Parameters **bus_handle** -- I2C bus handle.

Returns

- ESP_OK: Reset succeed.
- ESP_ERR_INVALID_ARG: I2C master bus handle is not initialized.
- Otherwise: Reset failed.

esp_err_t **i2c_master_bus_wait_all_done**(*i2c_master_bus_handle_t* bus_handle, int timeout_ms)

Wait for all pending I2C transactions done.

Parameters

- **bus_handle** -- **[in]** I2C bus handle
- **timeout_ms** -- **[in]** Wait timeout, in ms. Specially, -1 means to wait forever.

Returns

- ESP_OK: Flush transactions successfully
- ESP_ERR_INVALID_ARG: Flush transactions failed because of invalid argument
- ESP_ERR_TIMEOUT: Flush transactions failed because of timeout
- ESP_FAIL: Flush transactions failed because of other error

Structures

struct **i2c_master_bus_config_t**

I2C master bus specific configurations.

Public Members

i2c_port_num_t **i2c_port**

I2C port number, `-1` for auto selecting, (not include LP I2C instance)

gpio_num_t **sda_io_num**

GPIO number of I2C SDA signal, pulled-up internally

gpio_num_t **scl_io_num**

GPIO number of I2C SCL signal, pulled-up internally

i2c_clock_source_t **clk_source**

Clock source of I2C master bus

uint8_t **glitch_ignore_cnt**

If the glitch period on the line is less than this value, it can be filtered out, typically value is 7 (unit: I2C module clock cycle)

int **intr_priority**

I2C interrupt priority, if set to 0, driver will select the default priority (1,2,3).

size_t **trans_queue_depth**

Depth of internal transfer queue, increase this value can support more transfers pending in the background, only valid in asynchronous transaction. (Typically max_device_num * per_transaction)

uint32_t **enable_internal_pullup**

Enable internal pullups. Note: This is not strong enough to pullup buses under high-speed frequency. Recommend proper external pull-up if possible

struct *i2c_master_bus_config_t*::[anonymous] **flags**

I2C master config flags

struct **i2c_device_config_t**

I2C device configuration.

Public Members

i2c_addr_bit_len_t **dev_addr_length**

Select the address length of the slave device.

uint16_t **device_address**

I2C device raw address. (The 7/10 bit address without read/write bit)

uint32_t **scl_speed_hz**

I2C SCL line frequency.

uint32_t **scl_wait_us**

Timeout value. (unit: us). Please note this value should not be so small that it can handle stretch/disturbance properly. If 0 is set, that means use the default reg value

uint32_t **disable_ack_check**

Disable ACK check. If this is set false, that means ack check is enabled, the transaction will be stopped and API returns error when nack is detected.

struct *i2c_device_config_t*::[anonymous] **flags**

I2C device config flags

struct **i2c_master_event_callbacks_t**

Group of I2C master callbacks, can be used to get status during transaction or doing other small things. But take care potential concurrency issues.

Note: The callbacks are all running under ISR context

Note: When CONFIG_I2C_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well.

Public Members

i2c_master_callback_t **on_trans_done**

I2C master transaction finish callback

Header File

- components/esp_driver_i2c/include/driver/i2c_slave.h
- This header file can be included with:

```
#include "driver/i2c_slave.h"
```

- This header file is a part of the API provided by the `esp_driver_i2c` component. To declare that your component depends on `esp_driver_i2c`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2c
```

or

```
PRIV_REQUIRES esp_driver_i2c
```

Functions

esp_err_t **i2c_new_slave_device**(const *i2c_slave_config_t* *slave_config, *i2c_slave_dev_handle_t* *ret_handle)

Initialize an I2C slave device.

Parameters
- **slave_config** -- **[in]** I2C slave device configurations
- **ret_handle** -- **[out]** Return a generic I2C device handle

Returns
- ESP_OK: I2C slave device initialized successfully
- ESP_ERR_INVALID_ARG: I2C device initialization failed because of invalid argument.
- ESP_ERR_NO_MEM: Create I2C device failed because of out of memory.

esp_err_t **i2c_del_slave_device**(*i2c_slave_dev_handle_t* i2c_slave)

Deinitialize the I2C slave device.

Parameters **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.

Returns
- ESP_OK: Delete I2C device successfully.
- ESP_ERR_INVALID_ARG: I2C device initialization failed because of invalid argument.

esp_err_t **i2c_slave_receive**(*i2c_slave_dev_handle_t* i2c_slave, uint8_t *data, size_t buffer_size)

Read bytes from I2C internal buffer. Start a job to receive I2C data.

Note: This function is non-blocking, it initiates a new receive job and then returns. User should check the received data from the `on_recv_done` callback that registered by `i2c_slave_register_event_callbacks()`.

Parameters
- **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.
- **data** -- **[out]** Buffer to store data from I2C fifo. Should be valid until `on_recv_done` is triggered.
- **buffer_size** -- **[in]** Buffer size of data that provided by users.

Returns
- ESP_OK: I2C slave receive success.
- ESP_ERR_INVALID_ARG: I2C slave receive parameter invalid.
- ESP_ERR_NOT_SUPPORTED: This function should be work in fifo mode, but I2C_SLAVE_NONFIFO mode is configured

esp_err_t **i2c_slave_transmit**(*i2c_slave_dev_handle_t* i2c_slave, const uint8_t *data, int size, int xfer_timeout_ms)

Write bytes to internal ringbuffer of the I2C slave data. When the TX fifo empty, the ISR will fill the hardware FIFO with the internal ringbuffer's data.

Note: If you connect this slave device to some master device, the data transaction direction is from slave device to master device.

Parameters
- **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.
- **data** -- **[in]** Buffer to write to slave fifo, can pickup by master. Can be freed after this function returns. Equal or larger than `size`.
- **size** -- **[in]** In bytes, of `data` buffer.
- **xfer_timeout_ms** -- **[in]** Wait timeout, in ms. Note: -1 means wait forever.

Returns
- ESP_OK: I2C slave transmit success.

- ESP_ERR_INVALID_ARG: I2C slave transmit parameter invalid.
- ESP_ERR_TIMEOUT: Operation timeout(larger than xfer_timeout_ms) because the device is busy or hardware crash.
- ESP_ERR_NOT_SUPPORTED: This function should be work in fifo mode, but I2C_SLAVE_NONFIFO mode is configured

esp_err_t **i2c_slave_register_event_callbacks**(*i2c_slave_dev_handle_t* i2c_slave, const *i2c_slave_event_callbacks_t* *cbs, void *user_data)

Set I2C slave event callbacks for I2C slave channel.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Note: When CONFIG_I2C_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well. The `user_data` should also reside in SRAM.

Parameters

- **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set I2C transaction callbacks successfully
- ESP_ERR_INVALID_ARG: Set I2C transaction callbacks failed because of invalid argument
- ESP_FAIL: Set I2C transaction callbacks failed because of other error

esp_err_t **i2c_slave_read_ram**(*i2c_slave_dev_handle_t* i2c_slave, uint8_t ram_address, uint8_t *data, size_t receive_size)

Read bytes from I2C internal ram. This can be only used when `access_ram_en` in configuration structure set to true.

Parameters

- **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.
- **ram_address** -- **[in]** The offset of RAM (Cannot larger than I2C RAM memory)
- **data** -- **[out]** Buffer to store data read from I2C ram.
- **receive_size** -- **[in]** Received size from RAM.

Returns

- ESP_OK: I2C slave transmit success.
- ESP_ERR_INVALID_ARG: I2C slave transmit parameter invalid.
- ESP_ERR_NOT_SUPPORTED: This function should be work in non-fifo mode, but I2C_SLAVE_FIFO mode is configured

esp_err_t **i2c_slave_write_ram**(*i2c_slave_dev_handle_t* i2c_slave, uint8_t ram_address, const uint8_t *data, size_t size)

Write bytes to I2C internal ram. This can be only used when `access_ram_en` in configuration structure set to true.

Parameters

- **i2c_slave** -- **[in]** I2C slave device handle that created by `i2c_new_slave_device`.
- **ram_address** -- **[in]** The offset of RAM (Cannot larger than I2C RAM memory)
- **data** -- **[in]** Buffer to fill.
- **size** -- **[in]** Received size from RAM.

Returns

- ESP_OK: I2C slave transmit success.
- ESP_ERR_INVALID_ARG: I2C slave transmit parameter invalid.
- ESP_ERR_INVALID_SIZE: Write size is larger than
- ESP_ERR_NOT_SUPPORTED: This function should be work in non-fifo mode, but I2C_SLAVE_FIFO mode is configured

Structures

struct **i2c_slave_config_t**

I2C slave specific configurations.

Public Members

i2c_port_num_t **i2c_port**

I2C port number, `-1` for auto selecting

gpio_num_t **sda_io_num**

SDA IO number used by I2C bus

gpio_num_t **scl_io_num**

SCL IO number used by I2C bus

i2c_clock_source_t **clk_source**

Clock source of I2C bus.

uint32_t **send_buf_depth**

Depth of internal transfer ringbuffer, increase this value can support more transfers pending in the background

uint16_t **slave_addr**

I2C slave address

i2c_addr_bit_len_t **addr_bit_len**

I2C slave address in bit length

int **intr_priority**

I2C interrupt priority, if set to 0, driver will select the default priority (1,2,3).

uint32_t **broadcast_en**

I2C slave enable broadcast

uint32_t **access_ram_en**

Can get access to I2C RAM directly

struct *i2c_slave_config_t*::[anonymous] **flags**

I2C slave config flags

struct **i2c_slave_event_callbacks_t**

Group of I2C slave callbacks (e.g. get i2c slave stretch cause). But take care of potential concurrency issues.

Note: The callbacks are all running under ISR context

Note: When CONFIG_I2C_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well.

Public Members

i2c_slave_received_callback_t **on_recv_done**

I2C slave receive done callback

Header File

- components/esp_driver_i2c/include/driver/i2c_types.h
- This header file can be included with:

```
#include "driver/i2c_types.h"
```

- This header file is a part of the API provided by the `esp_driver_i2c` component. To declare that your component depends on `esp_driver_i2c`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2c
```

or

```
PRIV_REQUIRES esp_driver_i2c
```

Structures

struct **i2c_master_event_data_t**

Data type used in I2C event callback.

Public Members

i2c_master_event_t **event**

The I2C hardware event that I2C callback is called.

struct **i2c_slave_rx_done_event_data_t**

Event structure used in I2C slave.

Public Members

uint8_t ***buffer**

Pointer for buffer received in callback.

Type Definitions

typedef int **i2c_port_num_t**

I2C port number.

typedef struct i2c_master_bus_t ***i2c_master_bus_handle_t**

Type of I2C master bus handle.

typedef struct i2c_master_dev_t ***i2c_master_dev_handle_t**

Type of I2C master bus device handle.

typedef struct i2c_slave_dev_t ***i2c_slave_dev_handle_t**

Type of I2C slave device handle.

typedef bool (***i2c_master_callback_t**)(*i2c_master_dev_handle_t* i2c_dev, const *i2c_master_event_data_t* *evt_data, void *arg)

An callback for I2C transaction.

Param i2c_dev [in] Handle for I2C device.
Param evt_data [out] I2C capture event data, fed by driver
Param arg [in] User data, set in `i2c_master_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***i2c_slave_received_callback_t**)(*i2c_slave_dev_handle_t* i2c_slave, const *i2c_slave_rx_done_event_data_t* *evt_data, void *arg)

Callback signature for I2C slave.

Param i2c_slave [in] Handle for I2C slave.
Param evt_data [out] I2C capture event data, fed by driver
Param arg [in] User data, set in `i2c_slave_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

Enumerations

enum **i2c_master_status_t**

Enumeration for I2C fsm status.

Values:

enumerator **I2C_STATUS_READ**

read status for current master command

enumerator **I2C_STATUS_WRITE**

write status for current master command

enumerator **I2C_STATUS_START**

Start status for current master command

enumerator **I2C_STATUS_STOP**

stop status for current master command

enumerator **I2C_STATUS_IDLE**

idle status for current master command

enumerator **I2C_STATUS_ACK_ERROR**

ack error status for current master command

enumerator **I2C_STATUS_DONE**

I2C command done

enumerator **I2C_STATUS_TIMEOUT**

I2C bus status error, and operation timeout

enum **i2c_master_event_t**

Enumeration for I2C event.

Values:

enumerator **I2C_EVENT_ALIVE**

i2c bus in alive status.

enumerator **I2C_EVENT_DONE**

i2c bus transaction done

enumerator **I2C_EVENT_NACK**

i2c bus nack

enumerator **I2C_EVENT_TIMEOUT**

i2c bus timeout

Header File

- components/hal/include/hal/i2c_types.h
- This header file can be included with:

```
#include "hal/i2c_types.h"
```

Structures

struct **i2c_hal_clk_config_t**

Data structure for calculating I2C bus timing.

Public Members

uint16_t **clkm_div**

I2C core clock divider

uint16_t **scl_low**

I2C scl low period

uint16_t **scl_high**

I2C scl high period

uint16_t **scl_wait_high**

I2C scl wait_high period

uint16_t **sda_hold**

I2C scl low period

uint16_t **sda_sample**

I2C sda sample time

uint16_t **setup**

I2C start and stop condition setup period

uint16_t **hold**

I2C start and stop condition hold period

uint16_t **tout**

I2C bus timeout period

Type Definitions

typedef *soc_periph_i2c_clk_src_t* **i2c_clock_source_t**

I2C group clock source.

Enumerations

enum **i2c_port_t**

I2C port number, can be I2C_NUM_0 ~ (I2C_NUM_MAX-1).

Values:

enumerator **I2C_NUM_0**

I2C port 0

enumerator **I2C_NUM_1**

I2C port 1

enumerator **I2C_NUM_MAX**

I2C port max

enum **i2c_addr_bit_len_t**

Enumeration for I2C device address bit length.

Values:

enumerator **I2C_ADDR_BIT_LEN_7**

i2c address bit length 7

enumerator **I2C_ADDR_BIT_LEN_10**

i2c address bit length 10

enum **i2c_mode_t**

Values:

enumerator **I2C_MODE_SLAVE**

I2C slave mode

enumerator **I2C_MODE_MASTER**

I2C master mode

enumerator **I2C_MODE_MAX**

enum **i2c_rw_t**

Values:

enumerator **I2C_MASTER_WRITE**

I2C write data

enumerator **I2C_MASTER_READ**

I2C read data

enum **i2c_trans_mode_t**

Values:

enumerator **I2C_DATA_MODE_MSB_FIRST**

I2C data msb first

enumerator **I2C_DATA_MODE_LSB_FIRST**

I2C data lsb first

enumerator **I2C_DATA_MODE_MAX**

enum **i2c_addr_mode_t**

Values:

enumerator **I2C_ADDR_BIT_7**

I2C 7bit address for slave mode

enumerator **I2C_ADDR_BIT_10**

I2C 10bit address for slave mode

enumerator **I2C_ADDR_BIT_MAX**

enum **i2c_ack_type_t**

Values:

enumerator **I2C_MASTER_ACK**

I2C ack for each byte read

enumerator **I2C_MASTER_NACK**

I2C nack for each byte read

enumerator **I2C_MASTER_LAST_NACK**

I2C nack for the last byte

enumerator **I2C_MASTER_ACK_MAX**

enum **i2c_slave_stretch_cause_t**

Enum for I2C slave stretch causes.

Values:

enumerator **I2C_SLAVE_STRETCH_CAUSE_ADDRESS_MATCH**

Stretching SCL low when the slave is read by the master and the address just matched

enumerator **I2C_SLAVE_STRETCH_CAUSE_TX_EMPTY**

Stretching SCL low when TX FIFO is empty in slave mode

enumerator **I2C_SLAVE_STRETCH_CAUSE_RX_FULL**

Stretching SCL low when RX FIFO is full in slave mode

enumerator **I2C_SLAVE_STRETCH_CAUSE_SENDING_ACK**

Stretching SCL low when slave sending ACK

2.6.11 Inter-IC Sound (I2S)

Introduction

I2S (Inter-IC Sound) is a synchronous serial communication protocol usually used for transmitting audio data between two digital audio devices.

ESP32-S3 contains two I2S peripheral(s). These peripherals can be configured to input and output sample data via the I2S driver.

An I2S bus that communicates in standard or TDM mode consists of the following lines:

- **MCLK:** Master clock line. It is an optional signal depending on the slave side, mainly used for offering a reference clock to the I2S slave device.
- **BCLK:** Bit clock line. The bit clock for data line.
- **WS:** Word (Slot) select line. It is usually used to identify the vocal tract except PDM mode.
- **DIN/DOUT:** Serial data input/output line. Data will loopback internally if DIN and DOUT are set to a same GPIO.

An I2S bus that communicates in PDM mode consists of the following lines:

- **CLK:** PDM clock line.
- **DIN/DOUT:** Serial data input/output line.

Each I2S controller has the following features that can be configured by the I2S driver:

- Operation as system master or slave
- Capable of acting as transmitter or receiver
- DMA controller that allows stream sampling of data without requiring the CPU to copy each data sample

Each controller has separate RX and TX channels. That means they are able to work under different clocks and slot configurations with separate GPIO pins. Note that although the internal MCLKs of TX channel and RX channel are separate on a controller, the output MCLK signal can only be attached to one channel. If independent MCLK output is required for each channel, they must be allocated on different I2S controllers.

I2S File Structure

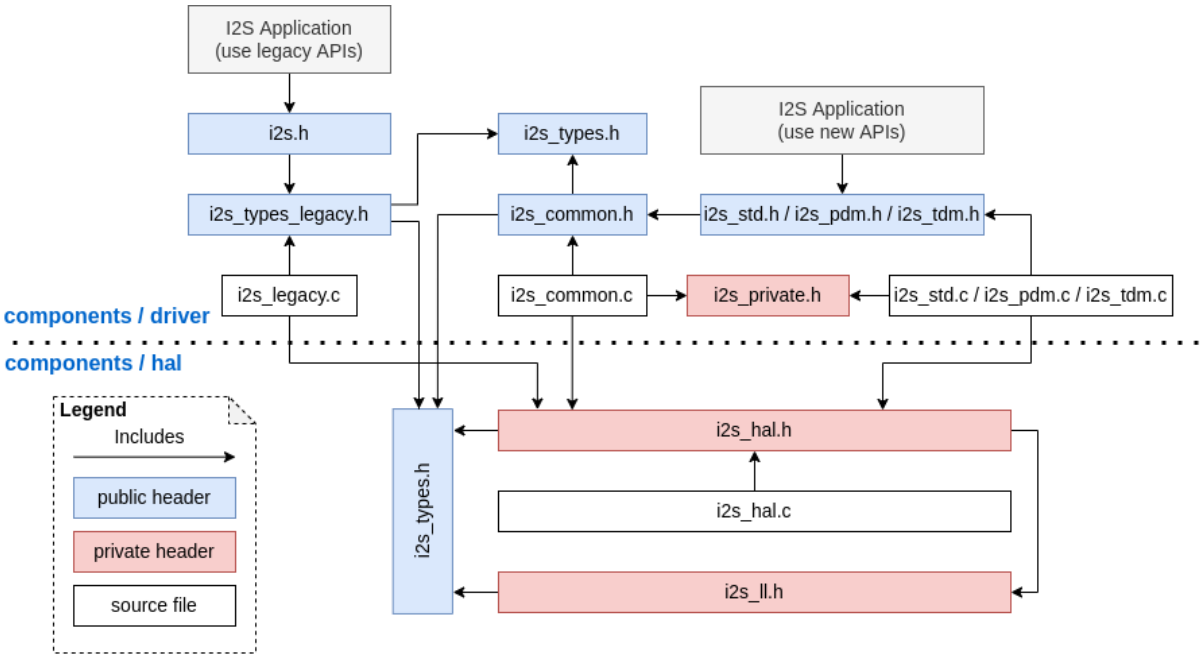


Fig. 13: I2S File Structure

Public headers that need to be included in the I2S application are as follows:

- `i2s.h`: The header file that provides legacy I2S APIs (for apps using legacy driver).
- `i2s_std.h`: The header file that provides standard communication mode specific APIs (for apps using new driver with standard mode).
- `i2s_pdm.h`: The header file that provides PDM communication mode specific APIs (for apps using new driver with PDM mode).
- `i2s_tdm.h`: The header file that provides TDM communication mode specific APIs (for apps using new driver with TDM mode).

Note: The legacy driver cannot coexist with the new driver. Include `i2s.h` to use the legacy driver, or include the other three headers to use the new driver. The legacy driver might be removed in future.

Public headers that have been included in the headers above are as follows:

- `i2s_types_legacy.h`: The header file that provides legacy public types that are only used in the legacy driver.
- `i2s_types.h`: The header file that provides public types.
- `i2s_common.h`: The header file that provides common APIs for all communication modes.

I2S Clock

Clock Source

- `i2s_clock_src_t::I2S_CLK_SRC_DEFAULT`: Default PLL clock.
- `i2s_clock_src_t::I2S_CLK_SRC_PLL_160M`: 160 MHz PLL clock.

Clock Terminology

- **Sample rate**: The number of sampled data in one second per slot.
- **SCLK**: Source clock frequency. It is the frequency of the clock source.
- **MCLK**: Master clock frequency. BCLK is generated from this clock. The MCLK signal usually serves as a reference clock and is mostly needed to synchronize BCLK and WS between I2S master and slave roles.
- **BCLK**: Bit clock frequency. Every tick of this clock stands for one data bit on data pin. The slot bit width configured in *i2s_std_slot_config_t::slot_bit_width* is equal to the number of BCLK ticks, which means there will be 8/16/24/32 BCLK ticks in one slot.
- **LRCK / WS**: Left/right clock or word select clock. For non-PDM mode, its frequency is equal to the sample rate.

Note: Normally, MCLK should be the multiple of `sample rate` and BCLK at the same time. The field *i2s_std_clk_config_t::mclk_multiple* indicates the multiple of MCLK to the `sample rate`. In most cases, `I2S_MCLK_MULTIPLE_256` should be enough. However, if `slot_bit_width` is set to `I2S_SLOT_BIT_WIDTH_24BIT`, to keep MCLK a multiple to the BCLK, *i2s_std_clk_config_t::mclk_multiple* should be set to multiples that are divisible by 3 such as `I2S_MCLK_MULTIPLE_384`. Otherwise, WS will be inaccurate.

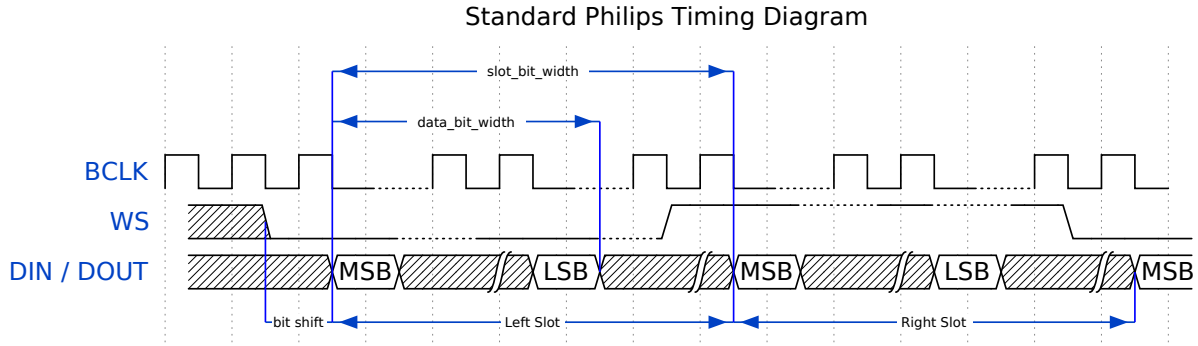
I2S Communication Mode

Overview of All Modes

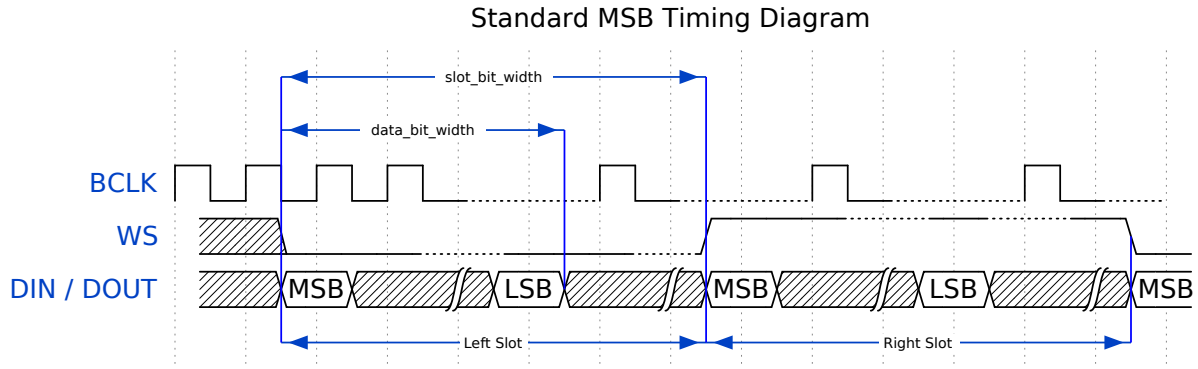
Target	Standard	PDM TX	PDM RX	TDM	ADC/DAC	LCD/Camera
ESP32	I2S 0/1	I2S 0	I2S 0	none	I2S 0	I2S 0
ESP32-S2	I2S 0	none	none	none	none	I2S 0
ESP32-C3	I2S 0	I2S 0	none	I2S 0	none	none
ESP32-C6	I2S 0	I2S 0	none	I2S 0	none	none
ESP32-S3	I2S 0/1	I2S 0	I2S 0	I2S 0/1	none	none
ESP32-H2	I2S 0	I2S 0	none	I2S 0	none	none
ESP32-P4	I2S 0~2	I2S 0	I2S 0	I2S 0~2	none	none

Standard Mode In standard mode, there are always two sound channels, i.e., the left and right channels, which are called "slots". These slots support 8/16/24/32-bit width sample data. The communication format for the slots mainly includes the following:

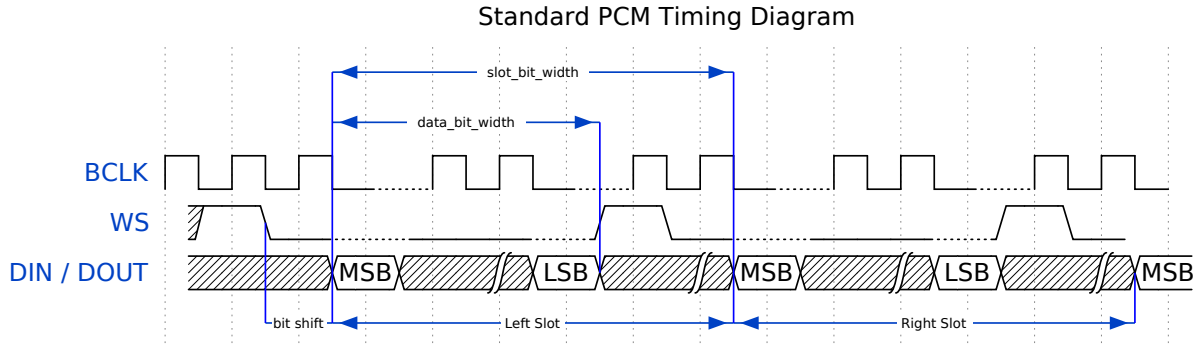
- **Philips Format**: Data signal has one-bit shift comparing to the WS signal, and the duty of WS signal is 50%.



- **MSB Format**: Basically the same as Philips format, but without data shift.

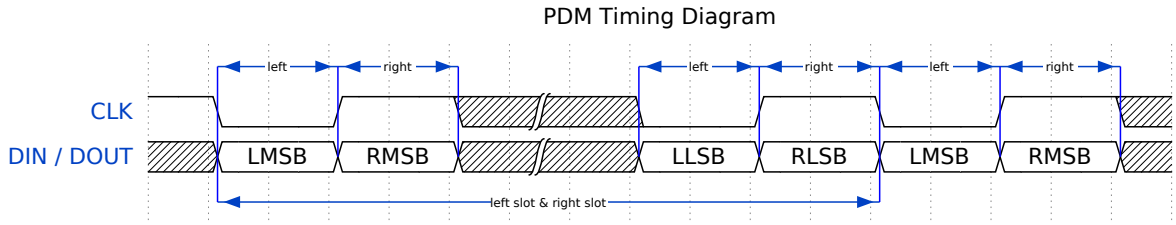


- **PCM Short Format**: Data has one-bit shift and meanwhile the WS signal becomes a pulse lasting for one BCLK cycle.



PDM Mode (TX) PDM (Pulse-density Modulation) mode for the TX channel can convert PCM data into PDM format which always has left and right slots. PDM TX is only supported on I2S0 and it only supports 16-bit width sample data. It needs at least a CLK pin for clock signal and a DOUT pin for data signal (i.e., the WS and SD signal in the following figure; the BCK signal is an internal bit sampling clock, which is not needed between PDM devices). This mode allows users to configure the up-sampling parameters *i2s_pdm_tx_clk_config_t::up_sample_fp* and *i2s_pdm_tx_clk_config_t::up_sample_fs*. The up-sampling rate can be calculated by `up_sample_rate = i2s_pdm_tx_clk_config_t::up_sample_fp / i2s_pdm_tx_clk_config_t::up_sample_fs`. There are two up-sampling modes in PDM TX:

- **Fixed Clock Frequency**: In this mode, the up-sampling rate changes according to the sample rate. Setting `fp = 960` and `fs = sample_rate / 100`, then the clock frequency (Fpdm) on CLK pin will be fixed to `128 * 48 KHz = 6.144 MHz`. Note that this frequency is not equal to the sample rate (Fpcm).
- **Fixed Up-sampling Rate**: In this mode, the up-sampling rate is fixed to 2. Setting `fp = 960` and `fs = 480`, then the clock frequency (Fpdm) on CLK pin will be `128 * sample_rate`.



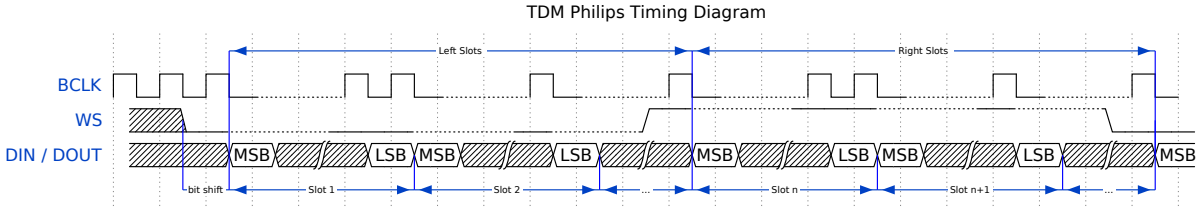
PDM Mode (RX) PDM (Pulse-density Modulation) mode for RX channel can receive PDM-format data and convert the data into PCM format. PDM RX is only supported on I2S0, and it only supports 16-bit width sample data. PDM RX needs at least a CLK pin for clock signal and a DIN pin for data signal. This mode allows users to configure the down-sampling parameter *i2s_pdm_rx_clk_config_t::dn_sample_mode*. There are two down-sampling modes in PDM RX:

- *i2s_pdm_dsr_t::I2S_PDM_DSR_8S*: In this mode, the clock frequency (Fpdm) on the WS pin is `sample_rate (Fpcm) * 64`.
- *i2s_pdm_dsr_t::I2S_PDM_DSR_16S*: In this mode, the clock frequency (Fpdm) on the WS pin is `sample_rate (Fpcm) * 128`.

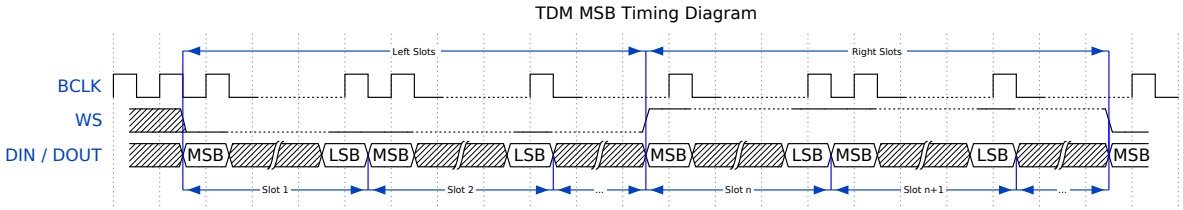
TDM Mode TDM (Time Division Multiplexing) mode supports up to 16 slots. These slots can be enabled by *i2s_tdm_slot_config_t::slot_mask*.

But due to the hardware limitation, only up to 4 slots are supported while the slot is set to 32 bit-width, and 8 slots for 16 bit-width, 16 slots for 8 bit-width. The slot communication format of TDM is almost the same as the standard mode, yet with some small differences.

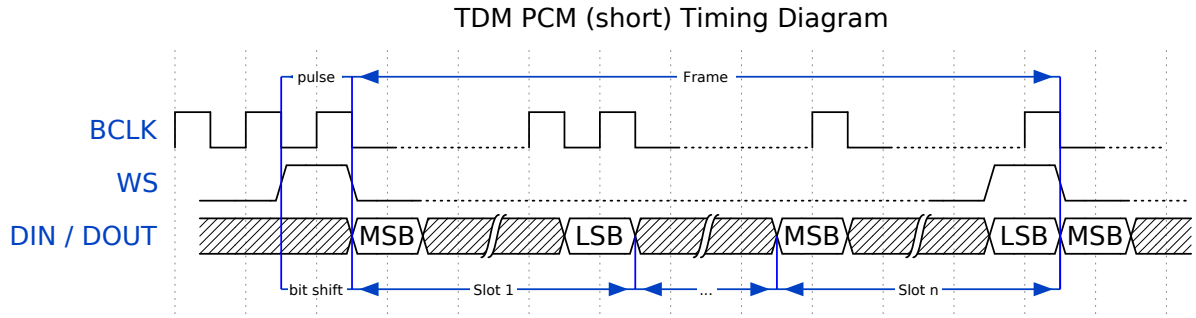
- **Philips Format**: Data signal has one-bit shift comparing to the WS signal. And no matter how many slots are contained in one frame, the duty of WS signal always keeps 50%.



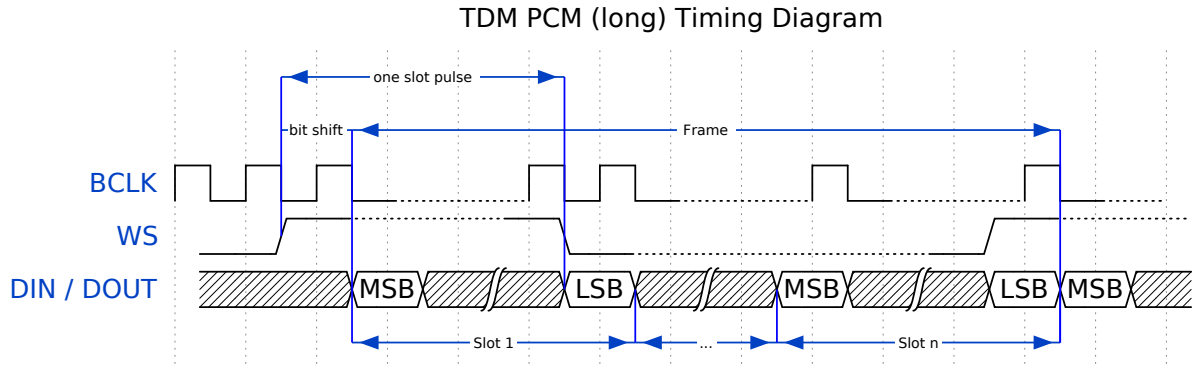
- **MSB Format**: Basically the same as the Philips format, but without data shift.



- **PCM Short Format**: Data has one-bit shift and the WS signal becomes a pulse lasting one BCLK cycle for every frame.



- **PCM Long Format**: Data has one-bit shift and the WS signal lasts one-slot bit width for every frame. For example, the duty of WS will be 25% if there are four slots enabled, and 20% if there are five slots.



Functional Overview

The I2S driver offers the following services:

Resource Management There are three levels of resources in the I2S driver:

- `platform level`: Resources of all I2S controllers in the current target.
- `controller level`: Resources in one I2S controller.
- `channel level`: Resources of TX or RX channel in one I2S controller.

The public APIs are all channel-level APIs. The channel handle *`i2s_chan_handle_t`* can help users to manage the resources under a specific channel without considering the other two levels. The other two upper levels' resources are private and are managed by the driver automatically. Users can call *`i2s_new_channel()`* to allocate a channel handle and call *`i2s_del_channel()`* to delete it.

Power Management When the power management is enabled (i.e., *CONFIG_PM_ENABLE* is on), the system will adjust or stop the source clock of I2S before entering Light-sleep, thus potentially changing the I2S signals and leading to transmitting or receiving invalid data.

The I2S driver can prevent the system from changing or stopping the source clock by acquiring a power management lock. When the source clock is generated from APB, the lock type will be set to *`esp_pm_lock_type_t::ESP_PM_APB_FREQ_MAX`* and when the source clock is APLL (if supported), it will be set to *`esp_pm_lock_type_t::ESP_PM_NO_LIGHT_SLEEP`*. Whenever the user is reading or writing via I2S (i.e., calling *`i2s_channel_read()`* or *`i2s_channel_write()`*), the driver guarantees that the power management lock is acquired. Likewise, the driver releases the lock after the reading or writing finishes.

Finite State Machine There are three states for an I2S channel, namely, `registered`, `ready`, and `running`. Their relationship is shown in the following diagram:

The `<mode>` in the diagram can be replaced by corresponding I2S communication modes, e.g., `std` for standard two-slot mode. For more information about communication modes, please refer to the *I2S Communication Mode* section.

Data Transport The data transport of the I2S peripheral, including sending and receiving, is realized by DMA. Before transporting data, please call *`i2s_channel_enable()`* to enable the specific channel. When the sent or received data reaches the size of one DMA buffer, the `I2S_OUT_EOF` or `I2S_IN_SUC_EOF` interrupt will be triggered. Note that the DMA buffer size is not equal to *`i2s_chan_config_t::dma_frame_num`*. One frame here refers to all the sampled data in one WS circle. Therefore, `dma_buffer_size = dma_frame_num * slot_num * slot_bit_width / 8`. For the data transmitting, users can input the data by calling *`i2s_channel_write()`*. This function helps users to copy the data from the source buffer to the DMA TX buffer and wait for the transmission to finish. Then it will repeat until the sent bytes reach the given size. For the data receiving, the function *`i2s_channel_read()`* waits to receive the message queue which contains the DMA buffer address. It helps users copy the data from the DMA RX buffer to the destination buffer.

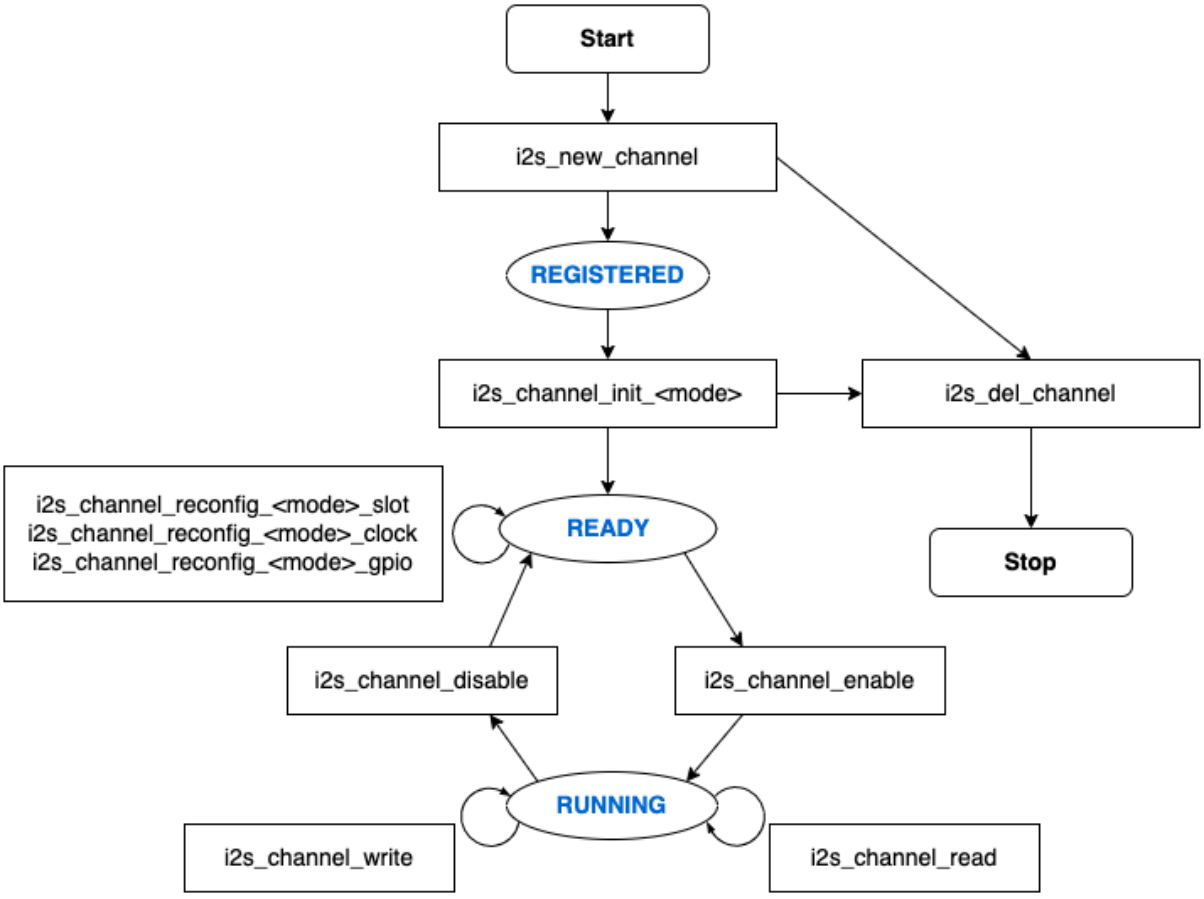


Fig. 14: I2S Finite State Machine

Both *i2s_channel_write()* and *i2s_channel_read()* are blocking functions. They keeps waiting until the whole source buffer is sent or the whole destination buffer is loaded, unless they exceed the max blocking time, where the error code ESP_ERR_TIMEOUT returns. To send or receive data asynchronously, callbacks can be registered by *i2s_channel_register_event_callback()*. Users are able to access the DMA buffer directly in the callback function instead of transmitting or receiving by the two blocking functions. However, please be aware that it is an interrupt callback, so do not add complex logic, run floating operation, or call non-reentrant functions in the callback.

Configuration Users can initialize a channel by calling corresponding functions (i.e., i2s_channel_init_std_mode(), i2s_channel_init_pdm_rx_mode(), i2s_channel_init_pdm_tx_mode(), or i2s_channel_init_tdm_mode()) to a specific mode. If the configurations need to be updated after initialization, users have to first call *i2s_channel_disable()* to ensure that the channel has stopped, and then call corresponding reconfig functions, like *i2s_channel_reconfig_std_slot()*, *i2s_channel_reconfig_std_clock()*, and *i2s_channel_reconfig_std_gpio()*.

IRAM Safe By default, the I2S interrupt will be deferred when the cache is disabled for reasons like writing/erasing flash. Thus the EOF interrupt will not get executed in time.

To avoid such case in real-time applications, you can enable the Kconfig option *CONFIG_I2S_ISR_IRAM_SAFE* that:

1. Keeps the interrupt being serviced even when the cache is disabled.
2. Places driver object into DRAM (in case it is linked to PSRAM by accident).

This allows the interrupt to run while the cache is disabled, but comes at the cost of increased IRAM consumption.

Thread Safety All the public I2S APIs are guaranteed to be thread safe by the driver, which means users can call them from different RTOS tasks without protection by extra locks. Notice that the I2S driver uses mutex lock to ensure the thread safety, thus these APIs are not allowed to be used in ISR.

Kconfig Options

- *CONFIG_I2S_ISR_IRAM_SAFE* controls whether the default ISR handler can work when the cache is disabled. See *IRAM Safe* for more information.
- *CONFIG_I2S_SUPPRESS_DEPRECATE_WARN* controls whether to suppress the compiling warning message while using the legacy I2S driver.
- *CONFIG_I2S_ENABLE_DEBUG_LOG* is used to enable the debug log output. Enable this option increases the firmware binary size.

Application Example

The examples of the I2S driver can be found in the directory peripherals/i2s. Here are some simple usages of each mode:

Standard TX/RX Usage Different slot communication formats can be generated by the following helper macros for standard mode. As described above, there are three formats in standard mode, and their helper macros are:

- *I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG*
- *I2S_STD_PCM_SLOT_DEFAULT_CONFIG*
- *I2S_STD_MSB_SLOT_DEFAULT_CONFIG*

The clock config helper macro is:

- *I2S_STD_CLK_DEFAULT_CONFIG*

Please refer to *Standard Mode* for information about STD API. And for more details, please refer to esp_driver_i2s/include/driver/i2s_std.h.

STD TX Mode Take 16-bit data width for example. When the data in a `uint16_t` writing buffer are:

data 0	data 1	data 2	data 3	data 4	data 5	data 6	data 7	...
0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	...

Here is the table of the real data on the line with different *i2s_std_slot_config_t::slot_mode* and *i2s_std_slot_config_t::slot_mask*.

data bit width	slot mode	slot mask	WS low	WS high	WS low	WS high	WS low	WS high	WS low	WS high
16 bit	mono	left	0x0001	0x0000	0x0002	0x0000	0x0003	0x0000	0x0004	0x0000
		right	0x0000	0x0001	0x0000	0x0002	0x0000	0x0003	0x0000	0x0004
		both	0x0001	0x0001	0x0002	0x0002	0x0003	0x0003	0x0004	0x0004
	stereo	left	0x0001	0x0000	0x0003	0x0000	0x0005	0x0000	0x0007	0x0000
		right	0x0000	0x0002	0x0000	0x0004	0x0000	0x0006	0x0000	0x0008
		both	0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008

Note: Similar for 8-bit and 32-bit data widths, the type of the buffer is better to be `uint8_t` and `uint32_t`. But specially, when the data width is 24-bit, the data buffer should be aligned with 3-byte (i.e., every 3 bytes stands for a 24-bit data in one slot). Additionally, *i2s_chan_config_t::dma_frame_num*, *i2s_std_clk_config_t::mclk_multiple*, and the writing buffer size should be the multiple of `3`, otherwise the data on the line or the sample rate will be incorrect.

```
#include "driver/i2s_std.h"
#include "driver/gpio.h"

i2s_chan_handle_t tx_handle;
/* Get the default channel configuration by the helper macro.
 * This helper macro is defined in `i2s_common.h` and shared by all the I2S␣
↪communication modes.
 * It can help to specify the I2S role and port ID */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_AUTO, I2S_ROLE_
↪MASTER);
/* Allocate a new TX channel and get the handle of this channel */
i2s_new_channel(&chan_cfg, &tx_handle, NULL);

/* Setting the configurations, the slot configuration and clock configuration can␣
↪be generated by the macros
 * These two helper macros are defined in `i2s_std.h` which can only be used in␣
↪STD mode.
 * They can help to specify the slot and clock configurations for initialization␣
↪or updating */
i2s_std_config_t std_cfg = {
    .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(48000),
    .slot_cfg = I2S_STD_MSB_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_32BIT, I2S_SLOT_
↪MODE_STEREO),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = GPIO_NUM_18,
        .din = I2S_GPIO_UNUSED,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
```

(continues on next page)

(continued from previous page)

```
    },
};
/* Initialize the channel */
i2s_channel_init_std_mode(tx_handle, &std_cfg);

/* Before writing data, start the TX channel first */
i2s_channel_enable(tx_handle);
i2s_channel_write(tx_handle, src_buf, bytes_to_write, bytes_written, ticks_to_
↪wait);

/* If the configurations of slot or clock need to be updated,
 * stop the channel first and then update it */
// i2s_channel_disable(tx_handle);
// std_cfg.slot_cfg.slot_mode = I2S_SLOT_MODE_MONO; // Default is stereo
// i2s_channel_reconfig_std_slot(tx_handle, &std_cfg.slot_cfg);
// std_cfg.clk_cfg.sample_rate_hz = 96000;
// i2s_channel_reconfig_std_clock(tx_handle, &std_cfg.clk_cfg);

/* Have to stop the channel before deleting it */
i2s_channel_disable(tx_handle);
/* If the handle is not needed any more, delete it to release the channel␣
↪resources */
i2s_del_channel(tx_handle);
```

STD RX Mode Taking 16-bit data width for example, when the data on the line are:

WS low	WS high	WS low	WS high	WS low	WS high	WS low	WS high	...
0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	...

Here is the table of the data received in the buffer with different *i2s_std_slot_config_t::slot_mode* and *i2s_std_slot_config_t::slot_mask*.

data bit width	slot mode	slot mask	data 0	data 1	data 2	data 3	data 4	data 5	data 6	data 7
16 bit	mono	left	0x0001	0x0003	0x0005	0x0007	0x0009	0x000b	0x000d	0x000f
		right	0x0002	0x0004	0x0006	0x0008	0x000a	0x000c	0x000e	0x0010
	stereo	any	0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008

Note: 8-bit, 24-bit, and 32-bit are similar as 16-bit, the data bit-width in the receiving buffer is equal to the data bit-width on the line. Additionally, when using 24-bit data width, *i2s_chan_config_t::dma_frame_num*, *i2s_std_clk_config_t::mclk_multiple*, and the receiving buffer size should be the multiple of 3, otherwise the data on the line or the sample rate will be incorrect.

```
#include "driver/i2s_std.h"
#include "driver/gpio.h"

i2s_chan_handle_t rx_handle;
/* Get the default channel configuration by helper macro.
 * This helper macro is defined in `i2s_common.h` and shared by all the I2S␣
↪communication modes.
 * It can help to specify the I2S role and port ID */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_AUTO, I2S_ROLE_
↪MASTER);
/* Allocate a new RX channel and get the handle of this channel */
i2s_new_channel(&chan_cfg, NULL, &rx_handle);
```

(continues on next page)

(continued from previous page)

```
/* Setting the configurations, the slot configuration and clock configuration can␣
↪be generated by the macros
 * These two helper macros are defined in `i2s_std.h` which can only be used in␣
↪STD mode.
 * They can help to specify the slot and clock configurations for initialization␣
↪or updating */
i2s_std_config_t std_cfg = {
    .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(48000),
    .slot_cfg = I2S_STD_MSB_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_32BIT, I2S_SLOT_
↪MODE_STEREO),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = I2S_GPIO_UNUSED,
        .din = GPIO_NUM_19,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
    },
};
/* Initialize the channel */
i2s_channel_init_std_mode(rx_handle, &std_cfg);

/* Before reading data, start the RX channel first */
i2s_channel_enable(rx_handle);
i2s_channel_read(rx_handle, desc_buf, bytes_to_read, bytes_read, ticks_to_wait);

/* Have to stop the channel before deleting it */
i2s_channel_disable(rx_handle);
/* If the handle is not needed any more, delete it to release the channel␣
↪resources */
i2s_del_channel(rx_handle);
```

PDM TX Usage For PDM mode in TX channel, the slot configuration helper macro is:

- *I2S_PDM_TX_SLOT_DEFAULT_CONFIG*

The clock configuration helper macro is:

- *I2S_PDM_TX_CLK_DEFAULT_CONFIG*

Please refer to *PDM Mode* for information about PDM TX API. And for more details, please refer to esp_driver_i2s/include/driver/i2s_pdm.h.

The PDM data width is fixed to 16-bit. When the data in an `int16_t` writing buffer is:

data 0	data 1	data 2	data 3	data 4	data 5	data 6	data 7	...
0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	...

Here is the table of the real data on the line with different *i2s_pdm_tx_slot_config_t::slot_mode* and *i2s_pdm_tx_slot_config_t::line_mode* (The PDM format on the line is transferred to PCM format for easier comprehension).

line mode	slot mode	line	left	right	left	right	left	right	left	right
one-line Codec	mono	dout	0x0001	0x0000	0x0002	0x0000	0x0003	0x0000	0x0004	0x0000
	stereo	dout	0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008
one-line DAC	mono	dout	0x0001	0x0001	0x0002	0x0002	0x0003	0x0003	0x0004	0x0004
two-line DAC	mono	dout	0x0002	0x0002	0x0004	0x0004	0x0006	0x0006	0x0008	0x0008
		dout2	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000	0x0000
	stereo	dout	0x0002	0x0002	0x0004	0x0004	0x0006	0x0006	0x0008	0x0008
		dout2	0x0001	0x0001	0x0003	0x0003	0x0005	0x0005	0x0007	0x0007

Note: There are three line modes for PDM TX mode, i.e., `I2S_PDM_TX_ONE_LINE_CODEC`, `I2S_PDM_TX_ONE_LINE_DAC`, and `I2S_PDM_TX_TWO_LINE_DAC`. One-line codec is for the PDM codecs that require clock signal. The PDM codec can differentiate the left and right slots by the clock level. The other two modes are used to drive power amplifiers directly with a low-pass filter. They do not need the clock signal, so there are two lines to differentiate the left and right slots. Additionally, for the mono mode of one-line codec, users can force change the slot to the right by setting the clock invert flag in GPIO configuration.

```
#include "driver/i2s_pdm.h"
#include "driver/gpio.h"

/* Allocate an I2S TX channel */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0, I2S_ROLE_
↪MASTER);
i2s_new_channel(&chan_cfg, &tx_handle, NULL);

/* Init the channel into PDM TX mode */
i2s_pdm_tx_config_t pdm_tx_cfg = {
    .clk_cfg = I2S_PDM_TX_CLK_DEFAULT_CONFIG(36000),
    .slot_cfg = I2S_PDM_TX_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_
↪MODE_MONO),
    .gpio_cfg = {
        .clk = GPIO_NUM_5,
        .dout = GPIO_NUM_18,
        .invert_flags = {
            .clk_inv = false,
        },
    },
};
i2s_channel_init_pdm_tx_mode(tx_handle, &pdm_tx_cfg);

...
```

PDM RX Usage For PDM mode in RX channel, the slot configuration helper macro is:

- *I2S_PDM_RX_SLOT_DEFAULT_CONFIG*

The clock configuration helper macro is:

- *I2S_PDM_RX_CLK_DEFAULT_CONFIG*

Please refer to *PDM Mode* for information about PDM RX API. And for more details, please refer to esp_driver_i2s/include/driver/i2s_pdm.h.

The PDM data width is fixed to 16-bit. When the data on the line (The PDM format on the line is transferred to PCM format for easier comprehension) is:

left	right	left	right	left	right	left	right	...
0x0001	0x0002	0x0003	0x0004	0x0005	0x0006	0x0007	0x0008	...

Here is the table of the data received in a `int16_t` buffer with different *i2s_pdm_rx_slot_config_t::slot_mode* and *i2s_pdm_rx_slot_config_t::slot_mask*.

slot mode	slot mask	data 0	data 1	data 2	data 3	data 4	data 5	data 6	data 7
mono	left	0x0001	0x0003	0x0005	0x0007	0x0009	0x000b	0x000d	0x000f
	right	0x0002	0x0004	0x0006	0x0008	0x000a	0x000c	0x000e	0x0010
stereo	both	0x0002	0x0001	0x0004	0x0003	0x0006	0x0005	0x0008	0x0007

Note: The right slot is received first in stereo mode. To switch the left and right slots in the buffer, please set the `i2s_pdm_rx_gpio_config_t::invert_flags::clk_inv` to force invert the clock signal.

Specially, ESP32-S3 supports up to 4 data lines in PDM RX mode, where each data line can be connected to two PDM MICs (left and right slots). This means that the PDM RX on ESP32-S3 can support up to 8 PDM MICs. To enable multiple data lines, set the bits in `i2s_pdm_rx_gpio_config_t::slot_mask` to enable corresponding slots first, and then set the data GPIOs in *i2s_pdm_rx_gpio_config_t*.

```
#include "driver/i2s_pdm.h"
#include "driver/gpio.h"

i2s_chan_handle_t rx_handle;

/* Allocate an I2S RX channel */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0, I2S_ROLE_
↪MASTER);
i2s_new_channel(&chan_cfg, NULL, &rx_handle);

/* Init the channel into PDM RX mode */
i2s_pdm_rx_config_t pdm_rx_cfg = {
    .clk_cfg = I2S_PDM_RX_CLK_DEFAULT_CONFIG(36000),
    .slot_cfg = I2S_PDM_RX_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_
↪MODE_MONO),
    .gpio_cfg = {
        .clk = GPIO_NUM_5,
        .din = GPIO_NUM_19,
        .invert_flags = {
            .clk_inv = false,
        },
    },
};
i2s_channel_init_pdm_rx_mode(rx_handle, &pdm_rx_cfg);

...
```

TDM TX/RX Usage Different slot communication formats can be generated by the following helper macros for TDM mode. As described above, there are four formats in TDM mode, and their helper macros are:

- *I2S_TDM_PHILIPS_SLOT_DEFAULT_CONFIG*
- *I2S_TDM_MSB_SLOT_DEFAULT_CONFIG*
- *I2S_TDM_PCM_SHORT_SLOT_DEFAULT_CONFIG*
- *I2S_TDM_PCM_LONG_SLOT_DEFAULT_CONFIG*

The clock config helper macro is:

- *I2S_TDM_CLK_DEFAULT_CONFIG*

Please refer to *TDM Mode* for information about TDM API. And for more details, please refer to esp_driver_i2s/include/driver/i2s_tdm.h.

Note: Due to hardware limitation, when setting the clock configuration for a slave role, please be aware that

i2s_tdm_clk_config_t::bclk_div should not be smaller than 8. Increasing this field can reduce the lagging of the data sent from the slave. In the high sample rate case, the data might lag behind for more than one BCLK which leads to data malposition. Users may gradually increase *i2s_tdm_clk_config_t::bclk_div* to correct it.

As *i2s_tdm_clk_config_t::bclk_div* is the division of MCLK to BCLK, increasing it also increases the MCLK frequency. Therefore, the clock calculation may fail if MCLK is too high to divide from the source clock. This means that a larger value for *i2s_tdm_clk_config_t::bclk_div* is not necessarily better.

TDM TX Mode

```
#include "driver/i2s_tdm.h"
#include "driver/gpio.h"

/* Allocate an I2S TX channel */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_AUTO, I2S_ROLE_
↪MASTER);
i2s_new_channel(&chan_cfg, &tx_handle, NULL);

/* Init the channel into TDM mode */
i2s_tdm_config_t tdm_cfg = {
    .clk_cfg = I2S_TDM_CLK_DEFAULT_CONFIG(44100),
    .slot_cfg = I2S_TDM_MSB_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_
↪MODE_STEREO,
                I2S_TDM_SLOT0 | I2S_TDM_SLOT1 | I2S_TDM_SLOT2 | I2S_TDM_SLOT3),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = GPIO_NUM_18,
        .din = I2S_GPIO_UNUSED,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
    },
};
i2s_channel_init_tdm_mode(tx_handle, &tdm_cfg);

...
```

TDM RX Mode

```
#include "driver/i2s_tdm.h"
#include "driver/gpio.h"

/* Set the channel mode to TDM */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_CONFIG(I2S_ROLE_MASTER, I2S_COMM_MODE_TDM,
↪ &i2s_pin);
i2s_new_channel(&chan_cfg, NULL, &rx_handle);

/* Init the channel into TDM mode */
i2s_tdm_config_t tdm_cfg = {
    .clk_cfg = I2S_TDM_CLK_DEFAULT_CONFIG(44100),
    .slot_cfg = I2S_TDM_MSB_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_
↪MODE_STEREO,
                I2S_TDM_SLOT0 | I2S_TDM_SLOT1 | I2S_TDM_SLOT2 | I2S_TDM_SLOT3),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
```

(continues on next page)

(continued from previous page)

```
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = I2S_GPIO_UNUSED,
        .din = GPIO_NUM_18,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
    },
};
i2s_channel_init_tdm_mode(rx_handle, &tdm_cfg);
...
```

Full-duplex Full-duplex mode registers TX and RX channel in an I2S port at the same time, and the channels share the BCLK and WS signals. Currently, STD and TDM communication modes supports full-duplex mode in the following way, but PDM full-duplex is not supported because due to different PDM TX and RX clocks.

Note that one handle can only stand for one channel. Therefore, it is still necessary to configure the slot and clock for both TX and RX channels one by one.

Here is an example of how to allocate a pair of full-duplex channels:

```
#include "driver/i2s_std.h"
#include "driver/gpio.h"

i2s_chan_handle_t tx_handle;
i2s_chan_handle_t rx_handle;

/* Allocate a pair of I2S channel */
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_AUTO, I2S_ROLE_
↪MASTER);
/* Allocate for TX and RX channel at the same time, then they will work in full-
↪duplex mode */
i2s_new_channel(&chan_cfg, &tx_handle, &rx_handle);

/* Set the configurations for BOTH TWO channels, since TX and RX channel have to␣
↪be same in full-duplex mode */
i2s_std_config_t std_cfg = {
    .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(32000),
    .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_
↪SLOT_MODE_STEREO),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = GPIO_NUM_18,
        .din = GPIO_NUM_19,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
    },
};
i2s_channel_init_std_mode(tx_handle, &std_cfg);
i2s_channel_init_std_mode(rx_handle, &std_cfg);

i2s_channel_enable(tx_handle);
i2s_channel_enable(rx_handle);
```

(continues on next page)

(continued from previous page)

```
...
```

Simplex Mode To allocate a channel in simplex mode, *i2s_new_channel()* should be called for each channel. The clock and GPIO pins of TX/RX channel on ESP32-S3 are independent, so they can be configured with different modes and clocks, and are able to coexist on the same I2S port in simplex mode. PDM duplex can be realized by registering PDM TX simplex and PDM RX simplex on the same I2S port. But in this way, PDM TX/RX might work with different clocks, so take care when configuring the GPIO pins and clocks.

The following example offers a use case for the simplex mode, but note that although the internal MCLK signals for TX and RX channel are separate, the output MCLK can only be bound to one of them if they are from the same controller. If MCLK has been initialized by both channels, it will be bound to the channel that initializes later.

```
#include "driver/i2s_std.h"
#include "driver/gpio.h"

i2s_chan_handle_t tx_handle;
i2s_chan_handle_t rx_handle;
i2s_chan_config_t chan_cfg = I2S_CHANNEL_DEFAULT_CONFIG(I2S_NUM_0, I2S_ROLE_
↪MASTER);
i2s_new_channel(&chan_cfg, &tx_handle, NULL);
i2s_std_config_t std_tx_cfg = {
    .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(48000),
    .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_16BIT, I2S_
↪SLOT_MODE_STEREO),
    .gpio_cfg = {
        .mclk = GPIO_NUM_0,
        .bclk = GPIO_NUM_4,
        .ws = GPIO_NUM_5,
        .dout = GPIO_NUM_18,
        .din = I2S_GPIO_UNUSED,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
            .ws_inv = false,
        },
    },
};
/* Initialize the channel */
i2s_channel_init_std_mode(tx_handle, &std_tx_cfg);
i2s_channel_enable(tx_handle);

/* RX channel will be registered on another I2S, if no other available I2S unit␣
↪found
 * it will return ESP_ERR_NOT_FOUND */
i2s_new_channel(&chan_cfg, NULL, &rx_handle); // Both RX and TX channel will be␣
↪registered on I2S0, but they can work with different configurations.
i2s_std_config_t std_rx_cfg = {
    .clk_cfg = I2S_STD_CLK_DEFAULT_CONFIG(16000),
    .slot_cfg = I2S_STD_MSB_SLOT_DEFAULT_CONFIG(I2S_DATA_BIT_WIDTH_32BIT, I2S_SLOT_
↪MODE_STEREO),
    .gpio_cfg = {
        .mclk = I2S_GPIO_UNUSED,
        .bclk = GPIO_NUM_6,
        .ws = GPIO_NUM_7,
        .dout = I2S_GPIO_UNUSED,
        .din = GPIO_NUM_19,
        .invert_flags = {
            .mclk_inv = false,
            .bclk_inv = false,
```

(continues on next page)

(continued from previous page)

```
            .ws_inv = false,
        },
    },
};
i2s_channel_init_std_mode(rx_handle, &std_rx_cfg);
i2s_channel_enable(rx_handle);
```

Application Notes

How to Prevent Data Lost For applications that need a high frequency sample rate, the massive data throughput may cause data lost. Users can receive data lost event by registering the ISR callback function to receive the event queue:

```
static IRAM_ATTR bool i2s_rx_queue_overflow_callback(i2s_chan_handle_t␣
↪handle, i2s_event_data_t *event, void *user_ctx)
{
    // handle RX queue overflow event ...
    return false;
}

i2s_event_callbacks_t cbs = {
    .on_recv = NULL,
    .on_recv_q_ovf = i2s_rx_queue_overflow_callback,
    .on_sent = NULL,
    .on_send_q_ovf = NULL,
};
TEST_ESP_OK(i2s_channel_register_event_callback(rx_handle, &cbs, NULL));
```

Please follow these steps to prevent data lost:

1. Determine the interrupt interval. Generally, when data lost happens, the bigger the interval, the better, which helps to reduce the interrupt times. This means `dma_frame_num` should be as big as possible while the DMA buffer size is below the maximum value of 4092. The relationships are:

   ```
   interrupt_interval(unit: sec) = dma_frame_num / sample_rate
   dma_buffer_size = dma_frame_num * slot_num * data_bit_width / 8 <= 4092
   ```

2. Determine `dma_desc_num`. `dma_desc_num` is decided by the maximum time of `i2s_channel_read` polling cycle. All the received data is supposed to be stored between two `i2s_channel_read`. This cycle can be measured by a timer or an outputting GPIO signal. The relationship is:

   ```
   dma_desc_num > polling_cycle / interrupt_interval
   ```

3. Determine the receiving buffer size. The receiving buffer offered by users in `i2s_channel_read` should be able to take all the data in all DMA buffers, which means that it should be larger than the total size of all the DMA buffers:

   ```
   recv_buffer_size > dma_desc_num * dma_buffer_size
   ```

For example, if there is an I2S application, and the known values are:

```
sample_rate = 144000 Hz
data_bit_width = 32 bits
slot_num = 2
polling_cycle = 10 ms
```

Then the parameters `dma_frame_num`, `dma_desc_num`, and `recv_buf_size` can be calculated as follows:

```
dma_frame_num * slot_num * data_bit_width / 8 = dma_buffer_size <= 4092
dma_frame_num <= 511
```

(continues on next page)

(continued from previous page)

```
interrupt_interval = dma_frame_num / sample_rate = 511 / 144000 = 0.003549 s = 3.
↪549 ms
dma_desc_num > polling_cycle / interrupt_interval = cell(10 / 3.549) = cell(2.818)␣
↪= 3
recv_buffer_size > dma_desc_num * dma_buffer_size = 3 * 4092 = 12276 bytes
```

API Reference

Standard Mode

Header File

- components/esp_driver_i2s/include/driver/i2s_std.h
- This header file can be included with:

```
#include "driver/i2s_std.h"
```

- This header file is a part of the API provided by the `esp_driver_i2s` component. To declare that your component depends on `esp_driver_i2s`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2s
```

or

```
PRIV_REQUIRES esp_driver_i2s
```

Functions

esp_err_t **i2s_channel_init_std_mode**(*i2s_chan_handle_t* handle, const *i2s_std_config_t* *std_cfg)

Initialize I2S channel to standard mode.

Note: Only allowed to be called when the channel state is REGISTERED, (i.e., channel has been allocated, but not initialized) and the state will be updated to READY if initialization success, otherwise the state will return to REGISTERED.

Parameters

- **handle** -- **[in]** I2S channel handler
- **std_cfg** -- **[in]** Configurations for standard mode, including clock, slot and GPIO The clock configuration can be generated by the helper macro `I2S_STD_CLK_DEFAULT_CONFIG` The slot configuration can be generated by the helper macro `I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG`, `I2S_STD_PCM_SLOT_DEFAULT_CONFIG` or `I2S_STD_MSB_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Initialize successfully
- ESP_ERR_NO_MEM No memory for storing the channel information
- ESP_ERR_INVALID_ARG NULL pointer or invalid configuration
- ESP_ERR_INVALID_STATE This channel is not registered

esp_err_t **i2s_channel_reconfig_std_clock**(*i2s_chan_handle_t* handle, const *i2s_std_clk_config_t* *clk_cfg)

Reconfigure the I2S clock for standard mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to standard mode, i.e., `i2s_channel_init_std_mode` has been called before reconfiguring

Parameters

- **`handle`** -- **[in]** I2S channel handler
- **`clk_cfg`** -- **[in]** Standard mode clock configuration, can be generated by `I2S_STD_CLK_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not standard mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **`i2s_channel_reconfig_std_slot`**(*i2s_chan_handle_t* handle, const *i2s_std_slot_config_t* *slot_cfg)

Reconfigure the I2S slot for standard mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to standard mode, i.e., `i2s_channel_init_std_mode` has been called before reconfiguring

Parameters

- **`handle`** -- **[in]** I2S channel handler
- **`slot_cfg`** -- **[in]** Standard mode slot configuration, can be generated by `I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG`, `I2S_STD_PCM_SLOT_DEFAULT_CONFIG` and `I2S_STD_MSB_SLOT_DEFAULT_CONFIG`.

Returns

- ESP_OK Set clock successfully
- ESP_ERR_NO_MEM No memory for DMA buffer
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not standard mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **`i2s_channel_reconfig_std_gpio`**(*i2s_chan_handle_t* handle, const *i2s_std_gpio_config_t* *gpio_cfg)

Reconfigure the I2S GPIO for standard mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to standard mode, i.e., `i2s_channel_init_std_mode` has been called before reconfiguring

Parameters

- **`handle`** -- **[in]** I2S channel handler
- **`gpio_cfg`** -- **[in]** Standard mode GPIO configuration, specified by user

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not standard mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

Structures

struct **i2s_std_slot_config_t**

I2S slot configuration for standard mode.

Public Members

i2s_data_bit_width_t **data_bit_width**

I2S sample data bit width (valid data bits per sample)

i2s_slot_bit_width_t **slot_bit_width**

I2S slot bit width (total bits per slot)

i2s_slot_mode_t **slot_mode**

Set mono or stereo mode with I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO In TX direction, mono means the written buffer contains only one slot data and stereo means the written buffer contains both left and right data

i2s_std_slot_mask_t **slot_mask**

Select the left, right or both slot

uint32_t **ws_width**

WS signal width (i.e. the number of BCLK ticks that WS signal is high)

bool **ws_pol**

WS signal polarity, set true to enable high lever first

bool **bit_shift**

Set to enable bit shift in Philips mode

bool **left_align**

Set to enable left alignment

bool **big_endian**

Set to enable big endian

bool **bit_order_lsb**

Set to enable lsb first

struct **i2s_std_clk_config_t**

I2S clock configuration for standard mode.

Public Members

uint32_t **sample_rate_hz**

I2S sample rate

i2s_clock_src_t **clk_src**

Choose clock source, see `soc_periph_i2s_clk_src_t` for the supported clock sources. selected `I2S_CLK_SRC_EXTERNAL` (if supports) to enable the external source clock input via MCLK pin,

uint32_t **ext_clk_freq_hz**

External clock source frequency in Hz, only take effect when `clk_src = I2S_CLK_SRC_EXTERNAL`, otherwise this field will be ignored, Please make sure the frequency input is equal or greater than BCLK, i.e. `sample_rate_hz * slot_bits * 2`

i2s_mclk_multiple_t **mclk_multiple**

The multiple of MCLK to the sample rate Default is 256 in the helper macro, it can satisfy most of cases, but please set this field a multiple of `3` (like 384) when using 24-bit data width, otherwise the sample rate might be inaccurate

struct **i2s_std_gpio_config_t**

I2S standard mode GPIO pins configuration.

Public Members

gpio_num_t **mclk**

MCK pin, output by default, input if the clock source is selected to `I2S_CLK_SRC_EXTERNAL`

gpio_num_t **bclk**

BCK pin, input in slave role, output in master role

gpio_num_t **ws**

WS pin, input in slave role, output in master role

gpio_num_t **dout**

DATA pin, output

gpio_num_t **din**

DATA pin, input

uint32_t **mclk_inv**

Set 1 to invert the MCLK input/output

uint32_t **bclk_inv**

Set 1 to invert the BCLK input/output

uint32_t **ws_inv**

Set 1 to invert the WS input/output

struct *i2s_std_gpio_config_t*::[anonymous] **invert_flags**

GPIO pin invert flags

struct **i2s_std_config_t**

I2S standard mode major configuration that including clock/slot/GPIO configuration.

Public Members

i2s_std_clk_config_t **clk_cfg**

Standard mode clock configuration, can be generated by macro I2S_STD_CLK_DEFAULT_CONFIG

i2s_std_slot_config_t **slot_cfg**

Standard mode slot configuration, can be generated by macros I2S_STD_[mode]_SLOT_DEFAULT_CONFIG, [mode] can be replaced with PHILIPS/MSB/PCM

i2s_std_gpio_config_t **gpio_cfg**

Standard mode GPIO configuration, specified by user

Macros

I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

Philips format in 2 slots.

This file is specified for I2S standard communication mode Features:

- Philips/MSB/PCM are supported in standard mode
- Fixed to 2 slots

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_STD_PCM_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

PCM(short) format in 2 slots.

Note: PCM(long) is same as Philips in 2 slots

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_STD_MSB_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

MSB format in 2 slots.

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_STD_CLK_DEFAULT_CONFIG(rate)

I2S default standard clock configuration.

Note: Please set the mclk_multiple to I2S_MCLK_MULTIPLE_384 while using 24 bits data width Otherwise the sample rate might be imprecise since the BCLK division is not a integer

Parameters

- **rate** -- sample rate

PDM Mode

Header File

- components/esp_driver_i2s/include/driver/i2s_pdm.h
- This header file can be included with:

```
#include "driver/i2s_pdm.h"
```

- This header file is a part of the API provided by the `esp_driver_i2s` component. To declare that your component depends on `esp_driver_i2s`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2s
```

or

```
PRIV_REQUIRES esp_driver_i2s
```

Functions

esp_err_t **i2s_channel_init_pdm_rx_mode**(*i2s_chan_handle_t* handle, const *i2s_pdm_rx_config_t* *pdm_rx_cfg)

Initialize I2S channel to PDM RX mode.

Note: Only allowed to be called when the channel state is REGISTERED, (i.e., channel has been allocated, but not initialized) and the state will be updated to READY if initialization success, otherwise the state will return to REGISTERED.

Parameters

- **handle** -- **[in]** I2S RX channel handler
- **pdm_rx_cfg** -- **[in]** Configurations for PDM RX mode, including clock, slot and GPIO The clock configuration can be generated by the helper macro `I2S_PDM_RX_CLK_DEFAULT_CONFIG` The slot configuration can be generated by the helper macro `I2S_PDM_RX_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Initialize successfully
- ESP_ERR_NO_MEM No memory for storing the channel information
- ESP_ERR_INVALID_ARG NULL pointer or invalid configuration
- ESP_ERR_INVALID_STATE This channel is not registered

esp_err_t **i2s_channel_reconfig_pdm_rx_clock**(*i2s_chan_handle_t* handle, const *i2s_pdm_rx_clk_config_t* *clk_cfg)

Reconfigure the I2S clock for PDM RX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM RX mode, i.e., `i2s_channel_init_pdm_rx_mode` has been called before reconfiguring

---

Parameters

- **handle** -- **[in]** I2S RX channel handler
- **clk_cfg** -- **[in]** PDM RX mode clock configuration, can be generated by `I2S_PDM_RX_CLK_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_pdm_rx_slot**(*i2s_chan_handle_t* handle, const *i2s_pdm_rx_slot_config_t* *slot_cfg)

Reconfigure the I2S slot for PDM RX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM RX mode, i.e., `i2s_channel_init_pdm_rx_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S RX channel handler
- **slot_cfg** -- **[in]** PDM RX mode slot configuration, can be generated by `I2S_PDM_RX_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_NO_MEM No memory for DMA buffer
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_pdm_rx_gpio**(*i2s_chan_handle_t* handle, const *i2s_pdm_rx_gpio_config_t* *gpio_cfg)

Reconfigure the I2S GPIO for PDM RX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM RX mode, i.e., `i2s_channel_init_pdm_rx_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S RX channel handler
- **gpio_cfg** -- **[in]** PDM RX mode GPIO configuration, specified by user

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_init_pdm_tx_mode**(*i2s_chan_handle_t* handle, const *i2s_pdm_tx_config_t* *pdm_tx_cfg)

Initialize I2S channel to PDM TX mode.

Note: Only allowed to be called when the channel state is REGISTERED, (i.e., channel has been allocated, but not initialized) and the state will be updated to READY if initialization success, otherwise the state will

return to REGISTERED.

Parameters

- **handle** -- **[in]** I2S TX channel handler
- **pdm_tx_cfg** -- **[in]** Configurations for PDM TX mode, including clock, slot and GPIO The clock configuration can be generated by the helper macro `I2S_PDM_TX_CLK_DEFAULT_CONFIG` The slot configuration can be generated by the helper macro `I2S_PDM_TX_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Initialize successfully
- ESP_ERR_NO_MEM No memory for storing the channel information
- ESP_ERR_INVALID_ARG NULL pointer or invalid configuration
- ESP_ERR_INVALID_STATE This channel is not registered

esp_err_t **i2s_channel_reconfig_pdm_tx_clock**(*i2s_chan_handle_t* handle, const *i2s_pdm_tx_clk_config_t* *clk_cfg)

Reconfigure the I2S clock for PDM TX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM TX mode, i.e., `i2s_channel_init_pdm_tx_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S TX channel handler
- **clk_cfg** -- **[in]** PDM TX mode clock configuration, can be generated by `I2S_PDM_TX_CLK_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_pdm_tx_slot**(*i2s_chan_handle_t* handle, const *i2s_pdm_tx_slot_config_t* *slot_cfg)

Reconfigure the I2S slot for PDM TX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM TX mode, i.e., `i2s_channel_init_pdm_tx_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S TX channel handler
- **slot_cfg** -- **[in]** PDM TX mode slot configuration, can be generated by `I2S_PDM_TX_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_NO_MEM No memory for DMA buffer
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_pdm_tx_gpio**(*i2s_chan_handle_t* handle, const *i2s_pdm_tx_gpio_config_t* *gpio_cfg)

Reconfigure the I2S GPIO for PDM TX mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to PDM TX mode, i.e., `i2s_channel_init_pdm_tx_mode` has been called before reconfiguring

---

Parameters

- **handle** -- **[in]** I2S TX channel handler
- **gpio_cfg** -- **[in]** PDM TX mode GPIO configuration, specified by user

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not PDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

Structures

struct **i2s_pdm_rx_slot_config_t**

I2S slot configuration for PDM RX mode.

Public Members

i2s_data_bit_width_t **data_bit_width**

I2S sample data bit width (valid data bits per sample), only support 16 bits for PDM mode

i2s_slot_bit_width_t **slot_bit_width**

I2S slot bit width (total bits per slot) , only support 16 bits for PDM mode

i2s_slot_mode_t **slot_mode**

Set mono or stereo mode with I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

i2s_pdm_slot_mask_t **slot_mask**

Choose the slots to activate

struct **i2s_pdm_rx_clk_config_t**

I2S clock configuration for PDM RX mode.

Public Members

uint32_t **sample_rate_hz**

I2S sample rate

i2s_clock_src_t **clk_src**

Choose clock source

i2s_mclk_multiple_t **mclk_multiple**

The multiple of MCLK to the sample rate

i2s_pdm_dsr_t **dn_sample_mode**

Down-sampling rate mode

uint32_t **bclk_div**

The division from MCLK to BCLK. The typical and minimum value is I2S_PDM_RX_BCLK_DIV_MIN. It will be set to I2S_PDM_RX_BCLK_DIV_MIN by default if it is smaller than I2S_PDM_RX_BCLK_DIV_MIN

struct **i2s_pdm_rx_gpio_config_t**

I2S PDM TX mode GPIO pins configuration.

Public Members

gpio_num_t **clk**

PDM clk pin, output

gpio_num_t **din**

DATA pin 0, input

gpio_num_t **dins**[(4)]

DATA pins, input, only take effect when corresponding I2S_PDM_RX_LINEx_SLOT_xxx is enabled in *i2s_pdm_rx_slot_config_t::slot_mask*

uint32_t **clk_inv**

Set 1 to invert the clk output

struct *i2s_pdm_rx_gpio_config_t*::[anonymous] **invert_flags**

GPIO pin invert flags

struct **i2s_pdm_rx_config_t**

I2S PDM RX mode major configuration that including clock/slot/GPIO configuration.

Public Members

i2s_pdm_rx_clk_config_t **clk_cfg**

PDM RX clock configurations, can be generated by macro I2S_PDM_RX_CLK_DEFAULT_CONFIG

i2s_pdm_rx_slot_config_t **slot_cfg**

PDM RX slot configurations, can be generated by macro I2S_PDM_RX_SLOT_DEFAULT_CONFIG

i2s_pdm_rx_gpio_config_t **gpio_cfg**

PDM RX slot configurations, specified by user

struct **i2s_pdm_tx_slot_config_t**

I2S slot configuration for PDM TX mode.

Public Members

i2s_data_bit_width_t **data_bit_width**

I2S sample data bit width (valid data bits per sample), only support 16 bits for PDM mode

i2s_slot_bit_width_t **slot_bit_width**

I2S slot bit width (total bits per slot), only support 16 bits for PDM mode

i2s_slot_mode_t **slot_mode**

Set mono or stereo mode with I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO For PDM TX mode, mono means the data buffer only contains one slot data, Stereo means the data buffer contains two slots data

uint32_t **sd_prescale**

Sigma-delta filter prescale

i2s_pdm_sig_scale_t **sd_scale**

Sigma-delta filter scaling value

i2s_pdm_sig_scale_t **hp_scale**

High pass filter scaling value

i2s_pdm_sig_scale_t **lp_scale**

Low pass filter scaling value

i2s_pdm_sig_scale_t **sinc_scale**

Sinc filter scaling value

i2s_pdm_tx_line_mode_t **line_mode**

PDM TX line mode, one-line codec, one-line dac, two-line dac mode can be selected

bool **hp_en**

High pass filter enable

float **hp_cut_off_freq_hz**

High pass filter cut-off frequency, range 23.3Hz ~ 185Hz, see cut-off frequency sheet above

uint32_t **sd_dither**

Sigma-delta filter dither

uint32_t **sd_dither2**

Sigma-delta filter dither2

struct **i2s_pdm_tx_clk_config_t**

I2S clock configuration for PDM TX mode.

Public Members

uint32_t **sample_rate_hz**

I2S sample rate, not suggest to exceed 48000 Hz, otherwise more glitches and noise may appear

i2s_clock_src_t **clk_src**

Choose clock source

i2s_mclk_multiple_t **mclk_multiple**

The multiple of MCLK to the sample rate

uint32_t **up_sample_fp**

Up-sampling param fp

uint32_t **up_sample_fs**

Up-sampling param fs, not allowed to be greater than 480

uint32_t **bclk_div**

The division from MCLK to BCLK. The minimum value is I2S_PDM_TX_BCLK_DIV_MIN. It will be set to I2S_PDM_TX_BCLK_DIV_MIN by default if it is smaller than I2S_PDM_TX_BCLK_DIV_MIN

struct **i2s_pdm_tx_gpio_config_t**

I2S PDM TX mode GPIO pins configuration.

Public Members

gpio_num_t **clk**

PDM clk pin, output

gpio_num_t **dout**

DATA pin, output

gpio_num_t **dout2**

The second data pin for the DAC dual-line mode, only take effect when the line mode is `I2S_PDM_TX_TWO_LINE_DAC`

uint32_t **clk_inv**

Set 1 to invert the clk output

struct *i2s_pdm_tx_gpio_config_t*::[anonymous] **invert_flags**

GPIO pin invert flags

struct **i2s_pdm_tx_config_t**

I2S PDM TX mode major configuration that including clock/slot/GPIO configuration.

Public Members

i2s_pdm_tx_clk_config_t **clk_cfg**

PDM TX clock configurations, can be generated by macro I2S_PDM_TX_CLK_DEFAULT_CONFIG

i2s_pdm_tx_slot_config_t **slot_cfg**

PDM TX slot configurations, can be generated by macro I2S_PDM_TX_SLOT_DEFAULT_CONFIG

i2s_pdm_tx_gpio_config_t **gpio_cfg**

PDM TX GPIO configurations, specified by user

Macros

I2S_PDM_RX_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

PDM format in 2 slots(RX)

This file is specified for I2S PDM communication mode Features:

- Only support PDM TX/RX mode
- Fixed to 2 slots
- Data bit width only support 16 bits

Parameters

- **bits_per_sample** -- I2S data bit width, only support 16 bits for PDM mode
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_PDM_RX_CLK_DEFAULT_CONFIG(rate)

I2S default PDM RX clock configuration.

Parameters

- **rate** -- sample rate

I2S_PDM_TX_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

PDM style in 2 slots(TX) for codec line mode.

Parameters

- **bits_per_sample** -- I2S data bit width, only support 16 bits for PDM mode
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_PDM_TX_SLOT_DAC_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo)

PDM style in 1 slots(TX) for DAC line mode.

Note: The noise might be different with different configurations, this macro provides a set of configurations that have relatively high SNR (Signal Noise Ratio), you can also adjust them to fit your case.

Parameters

- **bits_per_sample** -- I2S data bit width, only support 16 bits for PDM mode
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

I2S_PDM_TX_CLK_DEFAULT_CONFIG(rate)

I2S default PDM TX clock configuration for codec line mode.

Note: TX PDM can only be set to the following two up-sampling rate configurations: 1: fp = 960, fs = sample_rate_hz / 100, in this case, Fpdm = 128*48000 2: fp = 960, fs = 480, in this case, Fpdm = 128*Fpcm = 128*sample_rate_hz If the PDM receiver do not care the PDM serial clock, it's recommended set Fpdm = 128*48000. Otherwise, the second configuration should be adopted.

Parameters

- **rate** -- sample rate (not suggest to exceed 48000 Hz, otherwise more glitches and noise may appear)

I2S_PDM_TX_CLK_DAC_DEFAULT_CONFIG(rate)

I2S default PDM TX clock configuration for DAC line mode.

Note: TX PDM can only be set to the following two up-sampling rate configurations: 1: fp = 960, fs = sample_rate_hz / 100, in this case, Fpdm = 128*48000 2: fp = 960, fs = 480, in this case, Fpdm = 128*Fpcm = 128*sample_rate_hz If the PDM receiver do not care the PDM serial clock, it's recommended set Fpdm = 128*48000. Otherwise, the second configuration should be adopted.

Note: The noise might be different with different configurations, this macro provides a set of configurations that have relatively high SNR (Signal Noise Ratio), you can also adjust them to fit your case.

Parameters

- **rate** -- sample rate (not suggest to exceed 48000 Hz, otherwise more glitches and noise may appear)

TDM Mode

Header File

- components/esp_driver_i2s/include/driver/i2s_tdm.h
- This header file can be included with:

```
#include "driver/i2s_tdm.h"
```

- This header file is a part of the API provided by the `esp_driver_i2s` component. To declare that your component depends on `esp_driver_i2s`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2s
```

or

```
PRIV_REQUIRES esp_driver_i2s
```

Functions

esp_err_t **i2s_channel_init_tdm_mode**(*i2s_chan_handle_t* handle, const *i2s_tdm_config_t* *tdm_cfg)

Initialize I2S channel to TDM mode.

Note: Only allowed to be called when the channel state is REGISTERED, (i.e., channel has been allocated, but not initialized) and the state will be updated to READY if initialization success, otherwise the state will return to REGISTERED.

Parameters

- **handle** -- **[in]** I2S channel handler
- **tdm_cfg** -- **[in]** Configurations for TDM mode, including clock, slot and GPIO The clock configuration can be generated by the helper macro `I2S_TDM_CLK_DEFAULT_CONFIG` The slot configuration can be generated by the helper macro `I2S_TDM_PHILIPS_SLOT_DEFAULT_CONFIG`, `I2S_TDM_PCM_SHORT_SLOT_DEFAULT_CONFIG`,

`I2S_TDM_PCM_LONG_SLOT_DEFAULT_CONFIG` or `I2S_TDM_MSB_SLOT_DEFAULT_CONFIG`

Returns

- ESP_OK Initialize successfully
- ESP_ERR_NO_MEM No memory for storing the channel information
- ESP_ERR_INVALID_ARG NULL pointer or invalid configuration
- ESP_ERR_INVALID_STATE This channel is not registered

esp_err_t **i2s_channel_reconfig_tdm_clock**(*i2s_chan_handle_t* handle, const *i2s_tdm_clk_config_t* *clk_cfg)

Reconfigure the I2S clock for TDM mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to TDM mode, i.e., `i2s_channel_init_tdm_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S channel handler
- **clk_cfg** -- **[in]** Standard mode clock configuration, can be generated by `I2S_TDM_CLK_DEFAULT_CONFIG`

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not TDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_tdm_slot**(*i2s_chan_handle_t* handle, const *i2s_tdm_slot_config_t* *slot_cfg)

Reconfigure the I2S slot for TDM mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to TDM mode, i.e., `i2s_channel_init_tdm_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S channel handler
- **slot_cfg** -- **[in]** Standard mode slot configuration, can be generated by `I2S_TDM_PHILIPS_SLOT_DEFAULT_CONFIG`, `I2S_TDM_PCM_SHORT_SLOT_DEFAULT_CONFIG`, `I2S_TDM_PCM_LONG_SLOT_DEFAULT_CONFIG` or `I2S_TDM_MSB_SLOT_DEFAULT_CONFIG`.

Returns

- ESP_OK Set clock successfully
- ESP_ERR_NO_MEM No memory for DMA buffer
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not TDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

esp_err_t **i2s_channel_reconfig_tdm_gpio**(*i2s_chan_handle_t* handle, const *i2s_tdm_gpio_config_t* *gpio_cfg)

Reconfigure the I2S GPIO for TDM mode.

Note: Only allowed to be called when the channel state is READY, i.e., channel has been initialized, but not started this function won't change the state. `i2s_channel_disable` should be called before calling this function if I2S has started.

Note: The input channel handle has to be initialized to TDM mode, i.e., `i2s_channel_init_tdm_mode` has been called before reconfiguring

Parameters

- **handle** -- **[in]** I2S channel handler
- **gpio_cfg** -- **[in]** Standard mode GPIO configuration, specified by user

Returns

- ESP_OK Set clock successfully
- ESP_ERR_INVALID_ARG NULL pointer, invalid configuration or not TDM mode
- ESP_ERR_INVALID_STATE This channel is not initialized or not stopped

Structures

struct **i2s_tdm_slot_config_t**

I2S slot configuration for TDM mode.

Public Members

i2s_data_bit_width_t **data_bit_width**

I2S sample data bit width (valid data bits per sample)

i2s_slot_bit_width_t **slot_bit_width**

I2S slot bit width (total bits per slot)

i2s_slot_mode_t **slot_mode**

Set mono or stereo mode with I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO

i2s_tdm_slot_mask_t **slot_mask**

Slot mask. Activating slots by setting 1 to corresponding bits. When the activated slots is not consecutive, those data in inactivated slots will be ignored

uint32_t **ws_width**

WS signal width (i.e. the number of BCLK ticks that WS signal is high)

bool **ws_pol**

WS signal polarity, set true to enable high lever first

bool **bit_shift**

Set true to enable bit shift in Philips mode

bool **left_align**

Set true to enable left alignment

bool **big_endian**

Set true to enable big endian

bool **bit_order_lsb**

Set true to enable lsb first

bool **skip_mask**

Set true to enable skip mask. If it is enabled, only the data of the enabled channels will be sent, otherwise all data stored in DMA TX buffer will be sent

uint32_t **total_slot**

I2S total number of slots. If it is smaller than the biggest activated channel number, it will be set to this number automatically.

struct **i2s_tdm_clk_config_t**

I2S clock configuration for TDM mode.

Public Members

uint32_t **sample_rate_hz**

I2S sample rate

i2s_clock_src_t **clk_src**

Choose clock source, see `soc_periph_i2s_clk_src_t` for the supported clock sources. selected `I2S_CLK_SRC_EXTERNAL` (if supports) to enable the external source clock inputted via MCLK pin, please make sure the frequency inputted is equal or greater than `sample_rate_hz * mclk_multiple`

uint32_t **ext_clk_freq_hz**

External clock source frequency in Hz, only take effect when `clk_src = I2S_CLK_SRC_EXTERNAL`, otherwise this field will be ignored Please make sure the frequency inputted is equal or greater than BCLK, i.e. `sample_rate_hz * slot_bits * slot_num`

i2s_mclk_multiple_t **mclk_multiple**

The multiple of MCLK to the sample rate, only take effect for master role

uint32_t **bclk_div**

The division from MCLK to BCLK, only take effect for slave role, it shouldn't be smaller than 8. Increase this field when data sent by slave lag behind

struct **i2s_tdm_gpio_config_t**

I2S TDM mode GPIO pins configuration.

Public Members

gpio_num_t **mclk**

MCK pin, output by default, input if the clock source is selected to `I2S_CLK_SRC_EXTERNAL`

gpio_num_t **bclk**

BCK pin, input in slave role, output in master role

gpio_num_t **ws**

WS pin, input in slave role, output in master role

gpio_num_t **dout**

DATA pin, output

gpio_num_t **din**

DATA pin, input

uint32_t **mclk_inv**

Set 1 to invert the MCLK input/output

uint32_t **bclk_inv**

Set 1 to invert the BCLK input/output

uint32_t **ws_inv**

Set 1 to invert the WS input/output

struct *i2s_tdm_gpio_config_t*::[anonymous] **invert_flags**

GPIO pin invert flags

struct **i2s_tdm_config_t**

I2S TDM mode major configuration that including clock/slot/GPIO configuration.

Public Members

i2s_tdm_clk_config_t **clk_cfg**

TDM mode clock configuration, can be generated by macro I2S_TDM_CLK_DEFAULT_CONFIG

i2s_tdm_slot_config_t **slot_cfg**

TDM mode slot configuration, can be generated by macros I2S_TDM_[mode]_SLOT_DEFAULT_CONFIG, [mode] can be replaced with PHILIPS/MSB/PCM_SHORT/PCM_LONG

i2s_tdm_gpio_config_t **gpio_cfg**

TDM mode GPIO configuration, specified by user

Macros

I2S_TDM_AUTO_SLOT_NUM

This file is specified for I2S TDM communication mode Features:

- More than 2 slots

I2S_TDM_AUTO_WS_WIDTH

I2S_TDM_PHILIPS_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo, mask)

Philips format in active slot that enabled by mask.

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO
- **mask** -- active slot mask

I2S_TDM_MSB_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo, mask)

MSB format in active slot enabled that by mask.

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO
- **mask** -- active slot mask

I2S_TDM_PCM_SHORT_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo, mask)

PCM(short) format in active slot that enabled by mask.

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO
- **mask** -- active slot mask

I2S_TDM_PCM_LONG_SLOT_DEFAULT_CONFIG(bits_per_sample, mono_or_stereo, mask)

PCM(long) format in active slot that enabled by mask.

Parameters

- **bits_per_sample** -- I2S data bit width
- **mono_or_stereo** -- I2S_SLOT_MODE_MONO or I2S_SLOT_MODE_STEREO
- **mask** -- active slot mask

I2S_TDM_CLK_DEFAULT_CONFIG(rate)

I2S default TDM clock configuration.

Note: Please set the mclk_multiple to I2S_MCLK_MULTIPLE_384 while the data width in slot configuration is set to 24 bits Otherwise the sample rate might be imprecise since the BCLK division is not a integer

---

Parameters

- **rate** -- sample rate

I2S Driver

Header File

- components/esp_driver_i2s/include/driver/i2s_common.h
- This header file can be included with:

```
#include "driver/i2s_common.h"
```

- This header file is a part of the API provided by the `esp_driver_i2s` component. To declare that your component depends on `esp_driver_i2s`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2s
```

or

```
PRIV_REQUIRES esp_driver_i2s
```

Functions

esp_err_t **i2s_new_channel**(const *i2s_chan_config_t* *chan_cfg, *i2s_chan_handle_t* *ret_tx_handle, *i2s_chan_handle_t* *ret_rx_handle)

Allocate new I2S channel(s)

Note: The new created I2S channel handle will be REGISTERED state after it is allocated successfully.

Note: When the port id in channel configuration is I2S_NUM_AUTO, driver will allocate I2S port automatically on one of the I2S controller, otherwise driver will try to allocate the new channel on the selected port.

Note: If both tx_handle and rx_handle are not NULL, it means this I2S controller will work at full-duplex mode, the RX and TX channels will be allocated on a same I2S port in this case. Note that some configurations of TX/RX channel are shared on ESP32 and ESP32S2, so please make sure they are working at same condition and under same status(start/stop). Currently, full-duplex mode can't guarantee TX/RX channels write/read synchronously, they can only share the clock signals for now.

Note: If tx_handle OR rx_handle is NULL, it means this I2S controller will work at simplex mode. For ESP32 and ESP32S2, the whole I2S controller (i.e. both RX and TX channel) will be occupied, even if only one of RX or TX channel is registered. For the other targets, another channel on this controller will still available.

Parameters

- **chan_cfg** -- **[in]** I2S controller channel configurations
- **ret_tx_handle** -- **[out]** I2S channel handler used for managing the sending channel(optional)
- **ret_rx_handle** -- **[out]** I2S channel handler used for managing the receiving channel(optional)

Returns

- ESP_OK Allocate new channel(s) success
- ESP_ERR_NOT_SUPPORTED The communication mode is not supported on the current chip
- ESP_ERR_INVALID_ARG NULL pointer or illegal parameter in *i2s_chan_config_t*
- ESP_ERR_NOT_FOUND No available I2S channel found

esp_err_t **i2s_del_channel**(*i2s_chan_handle_t* handle)

Delete the I2S channel.

Note: Only allowed to be called when the I2S channel is at REGISTERED or READY state (i.e., it should stop before deleting it).

Note: Resource will be free automatically if all channels in one port are deleted

---

Parameters **handle** -- **[in]** I2S channel handler

- ESP_OK Delete successfully
- ESP_ERR_INVALID_ARG NULL pointer

esp_err_t **i2s_channel_get_info**(*i2s_chan_handle_t* handle, *i2s_chan_info_t* *chan_info)

Get I2S channel information.

Parameters
- **handle** -- **[in]** I2S channel handler
- **chan_info** -- **[out]** I2S channel basic information

Returns
- ESP_OK Get I2S channel information success
- ESP_ERR_NOT_FOUND The input handle doesn't match any registered I2S channels, it may not an I2S channel handle or not available any more
- ESP_ERR_INVALID_ARG The input handle or chan_info pointer is NULL

esp_err_t **i2s_channel_enable**(*i2s_chan_handle_t* handle)

Enable the I2S channel.

Note: Only allowed to be called when the channel state is READY, (i.e., channel has been initialized, but not started) the channel will enter RUNNING state once it is enabled successfully.

Note: Enable the channel can start the I2S communication on hardware. It will start outputting BCLK and WS signal. For MCLK signal, it will start to output when initialization is finished

---

Parameters **handle** -- **[in]** I2S channel handler
- ESP_OK Start successfully
- ESP_ERR_INVALID_ARG NULL pointer
- ESP_ERR_INVALID_STATE This channel has not initialized or already started

esp_err_t **i2s_channel_disable**(*i2s_chan_handle_t* handle)

Disable the I2S channel.

Note: Only allowed to be called when the channel state is RUNNING, (i.e., channel has been started) the channel will enter READY state once it is disabled successfully.

Note: Disable the channel can stop the I2S communication on hardware. It will stop BCLK and WS signal but not MCLK signal

---

Parameters **handle** -- **[in]** I2S channel handler

Returns
- ESP_OK Stop successfully
- ESP_ERR_INVALID_ARG NULL pointer
- ESP_ERR_INVALID_STATE This channel has not stated

esp_err_t **i2s_channel_preload_data**(*i2s_chan_handle_t* tx_handle, const void *src, size_t size, size_t *bytes_loaded)

Preload the data into TX DMA buffer.

Note: Only allowed to be called when the channel state is READY, (i.e., channel has been initialized, but not started)

Note: As the initial DMA buffer has no data inside, it will transmit the empty buffer after enabled the channel, this function is used to preload the data into the DMA buffer, so that the valid data can be transmitted

immediately after the channel is enabled.

Note: This function can be called multiple times before enabling the channel, the buffer that loaded later will be concatenated behind the former loaded buffer. But when all the DMA buffers have been loaded, no more data can be preload then, please check the `bytes_loaded` parameter to see how many bytes are loaded successfully, when the `bytes_loaded` is smaller than the `size`, it means the DMA buffers are full.

Parameters

- **`tx_handle`** -- **[in]** I2S TX channel handler
- **`src`** -- **[in]** The pointer of the source buffer to be loaded
- **`size`** -- **[in]** The source buffer size
- **`bytes_loaded`** -- **[out]** The bytes that successfully been loaded into the TX DMA buffer

Returns

- ESP_OK Load data successful
- ESP_ERR_INVALID_ARG NULL pointer or not TX direction
- ESP_ERR_INVALID_STATE This channel has not stated

esp_err_t **`i2s_channel_write`**(*i2s_chan_handle_t* handle, const void *src, size_t size, size_t *bytes_written, uint32_t timeout_ms)

I2S write data.

Note: Only allowed to be called when the channel state is RUNNING, (i.e., TX channel has been started and is not writing now) but the RUNNING only stands for the software state, it doesn't mean there is no the signal transporting on line.

Parameters

- **`handle`** -- **[in]** I2S channel handler
- **`src`** -- **[in]** The pointer of sent data buffer
- **`size`** -- **[in]** Max data buffer length
- **`bytes_written`** -- **[out]** Byte number that actually be sent, can be NULL if not needed
- **`timeout_ms`** -- **[in]** Max block time

Returns

- ESP_OK Write successfully
- ESP_ERR_INVALID_ARG NULL pointer or this handle is not TX handle
- ESP_ERR_TIMEOUT Writing timeout, no writing event received from ISR within ticks_to_wait
- ESP_ERR_INVALID_STATE I2S is not ready to write

esp_err_t **`i2s_channel_read`**(*i2s_chan_handle_t* handle, void *dest, size_t size, size_t *bytes_read, uint32_t timeout_ms)

I2S read data.

Note: Only allowed to be called when the channel state is RUNNING but the RUNNING only stands for the software state, it doesn't mean there is no the signal transporting on line.

Parameters

- **`handle`** -- **[in]** I2S channel handler
- **`dest`** -- **[in]** The pointer of receiving data buffer
- **`size`** -- **[in]** Max data buffer length
- **`bytes_read`** -- **[out]** Byte number that actually be read, can be NULL if not needed
- **`timeout_ms`** -- **[in]** Max block time

Returns

- ESP_OK Read successfully
- ESP_ERR_INVALID_ARG NULL pointer or this handle is not RX handle
- ESP_ERR_TIMEOUT Reading timeout, no reading event received from ISR within ticks_to_wait
- ESP_ERR_INVALID_STATE I2S is not ready to read

esp_err_t **i2s_channel_register_event_callback**(*i2s_chan_handle_t* handle, const *i2s_event_callbacks_t* *callbacks, void *user_data)

Set event callbacks for I2S channel.

Note: Only allowed to be called when the channel state is REGISTERED / READY, (i.e., before channel starts)

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `callbacks` structure to NULL.

Note: When CONFIG_I2S_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well. The `user_data` should also reside in SRAM or internal RAM as well.

Parameters

- **handle** -- **[in]** I2S channel handler
- **callbacks** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK Set event callbacks successfully
- ESP_ERR_INVALID_ARG Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE Set event callbacks failed because the current channel state is not REGISTERED or READY

Structures

struct **i2s_event_callbacks_t**

Group of I2S callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_I2S_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well.

Public Members

i2s_isr_callback_t **on_recv**

Callback of data received event, only for RX channel The event data includes DMA buffer address and size that just finished receiving data

i2s_isr_callback_t **on_recv_q_ovf**

Callback of receiving queue overflowed event, only for RX channel The event data includes buffer size that has been overwritten

i2s_isr_callback_t **on_sent**

Callback of data sent event, only for TX channel The event data includes DMA buffer address and size that just finished sending data

i2s_isr_callback_t **on_send_q_ovf**

Callback of sending queue overflowed event, only for TX channel The event data includes buffer size that has been overwritten

struct **i2s_chan_config_t**

I2S controller channel configuration.

Public Members

i2s_port_t **id**

I2S port id

i2s_role_t **role**

I2S role, I2S_ROLE_MASTER or I2S_ROLE_SLAVE

uint32_t **dma_desc_num**

I2S DMA buffer number, it is also the number of DMA descriptor

uint32_t **dma_frame_num**

I2S frame number in one DMA buffer. One frame means one-time sample data in all slots, it should be the multiple of `3` when the data bit width is 24.

bool **auto_clear**

Alias of `auto_clear_after_cb`

bool **auto_clear_after_cb**

Set to auto clear DMA TX buffer after `on_sent` callback, I2S will always send zero automatically if no data to send. So that user can assign the data to the DMA buffers directly in the callback, and the data won't be cleared after quit the callback.

bool **auto_clear_before_cb**

Set to auto clear DMA TX buffer before `on_sent` callback, I2S will always send zero automatically if no data to send So that user can access data in the callback that just finished to send.

int **intr_priority**

I2S interrupt priority, range [0, 7], if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

struct **i2s_chan_info_t**

I2S channel information.

Public Members

i2s_port_t **id**

I2S port id

i2s_role_t **role**

I2S role, I2S_ROLE_MASTER or I2S_ROLE_SLAVE

i2s_dir_t **dir**

I2S channel direction

i2s_comm_mode_t **mode**

I2S channel communication mode

i2s_chan_handle_t **pair_chan**

I2S pair channel handle in duplex mode, always NULL in simplex mode

uint32_t **total_dma_buf_size**

Total size of all the allocated DMA buffers

- 0 if the channel has not been initialized
- non-zero if the channel has been initialized

Macros

I2S_CHANNEL_DEFAULT_CONFIG(i2s_num, i2s_role)

get default I2S property

I2S_GPIO_UNUSED

Used in i2s_gpio_config_t for signals which are not used

I2S Types

Header File

- components/esp_driver_i2s/include/driver/i2s_types.h
- This header file can be included with:

```
#include "driver/i2s_types.h"
```

- This header file is a part of the API provided by the `esp_driver_i2s` component. To declare that your component depends on `esp_driver_i2s`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_i2s
```

or

```
PRIV_REQUIRES esp_driver_i2s
```

Structures

struct **i2s_event_data_t**

Event structure used in I2S event queue.

Public Members

void ***data***

(Deprecated) The secondary pointer of DMA buffer that just finished sending or receiving for `on_recv` and `on_sent` callback NULL for `on_recv_q_ovf` and `on_send_q_ovf` callback

void ***dma_buf***

The first level pointer of DMA buffer that just finished sending or receiving for `on_recv` and `on_sent` callback NULL for `on_recv_q_ovf` and `on_send_q_ovf` callback

size_t **size**

The buffer size of DMA buffer when success to send or receive, also the buffer size that dropped when queue overflow. It is related to the dma_frame_num and data_bit_width, typically it is fixed when data_bit_width is not changed.

Type Definitions

typedef struct i2s_channel_obj_t ***i2s_chan_handle_t***

I2S channel object handle, the control unit of the I2S driver

typedef bool (***i2s_isr_callback_t***)(*i2s_chan_handle_t* handle, *i2s_event_data_t* *event, void *user_ctx)

I2S event callback.

Param handle **[in]** I2S channel handle, created from `i2s_new_channel()`
Param event **[in]** I2S event data
Param user_ctx **[in]** User registered context, passed from `i2s_channel_register_event_callback()`
Return Whether a high priority task has been waken up by this callback function

Enumerations

enum **i2s_port_t**

I2S controller port number, the max port number is (SOC_I2S_NUM -1).

Values:

enumerator **I2S_NUM_0**

I2S controller port 0

enumerator **I2S_NUM_1**

I2S controller port 1

enumerator **I2S_NUM_AUTO**

Select whichever port is available

enum **i2s_comm_mode_t**

I2S controller communication mode.

Values:

enumerator **I2S_COMM_MODE_STD**

I2S controller using standard communication mode, support Philips/MSB/PCM format

enumerator **I2S_COMM_MODE_PDM**

I2S controller using PDM communication mode, support PDM output or input

enumerator **I2S_COMM_MODE_TDM**

I2S controller using TDM communication mode, support up to 16 slots per frame

enumerator **I2S_COMM_MODE_NONE**

Unspecified I2S controller mode

enum **i2s_mclk_multiple_t**

The multiple of MCLK to sample rate.

Note: MCLK is the minimum resolution of the I2S clock. Increasing mclk multiple can reduce the clock jitter of BCLK and WS, which is also useful for the codec that don't require MCLK but have strict requirement to BCLK. For the 24-bit slot width, please choose a multiple that can be divided by 3 (i.e. 24-bit compatible).

Values:

enumerator **I2S_MCLK_MULTIPLE_128**

MCLK = sample_rate * 128

enumerator **I2S_MCLK_MULTIPLE_192**

MCLK = sample_rate * 192 (24-bit compatible)

enumerator **I2S_MCLK_MULTIPLE_256**

MCLK = sample_rate * 256

enumerator **I2S_MCLK_MULTIPLE_384**

MCLK = sample_rate * 384 (24-bit compatible)

enumerator **I2S_MCLK_MULTIPLE_512**

MCLK = sample_rate * 512

enumerator **I2S_MCLK_MULTIPLE_576**

MCLK = sample_rate * 576 (24-bit compatible)

enumerator **I2S_MCLK_MULTIPLE_768**

MCLK = sample_rate * 768 (24-bit compatible)

enumerator **I2S_MCLK_MULTIPLE_1024**

MCLK = sample_rate * 1024

enumerator **I2S_MCLK_MULTIPLE_1152**

MCLK = sample_rate * 1152 (24-bit compatible)

Header File

- components/hal/include/hal/i2s_types.h
- This header file can be included with:

```
#include "hal/i2s_types.h"
```

Type Definitions

typedef *soc_periph_i2s_clk_src_t* **i2s_clock_src_t**

I2S clock source

Enumerations

enum **i2s_slot_mode_t**

I2S channel slot mode.

Values:

enumerator **I2S_SLOT_MODE_MONO**

I2S channel slot format mono, transmit same data in all slots for tx mode, only receive the data in the first slots for rx mode.

enumerator **I2S_SLOT_MODE_STEREO**

I2S channel slot format stereo, transmit different data in different slots for tx mode, receive the data in all slots for rx mode.

enum **i2s_dir_t**

I2S channel direction.

Values:

enumerator **I2S_DIR_RX**

I2S channel direction RX

enumerator **I2S_DIR_TX**

I2S channel direction TX

enum **i2s_role_t**

I2S controller role.

Values:

enumerator **I2S_ROLE_MASTER**

I2S controller master role, bclk and ws signal will be set to output

enumerator **I2S_ROLE_SLAVE**

I2S controller slave role, bclk and ws signal will be set to input

enum **i2s_data_bit_width_t**

Available data bit width in one slot.

Values:

enumerator **I2S_DATA_BIT_WIDTH_8BIT**

I2S channel data bit-width: 8

enumerator **I2S_DATA_BIT_WIDTH_16BIT**

I2S channel data bit-width: 16

enumerator **I2S_DATA_BIT_WIDTH_24BIT**

I2S channel data bit-width: 24

enumerator **I2S_DATA_BIT_WIDTH_32BIT**

I2S channel data bit-width: 32

enum **i2s_slot_bit_width_t**

Total slot bit width in one slot.

Values:

enumerator **I2S_SLOT_BIT_WIDTH_AUTO**

I2S channel slot bit-width equals to data bit-width

enumerator **I2S_SLOT_BIT_WIDTH_8BIT**

I2S channel slot bit-width: 8

enumerator **I2S_SLOT_BIT_WIDTH_16BIT**

I2S channel slot bit-width: 16

enumerator **I2S_SLOT_BIT_WIDTH_24BIT**

I2S channel slot bit-width: 24

enumerator **I2S_SLOT_BIT_WIDTH_32BIT**

I2S channel slot bit-width: 32

enum **i2s_pcm_compress_t**

A/U-law decompress or compress configuration.

Values:

enumerator **I2S_PCM_DISABLE**

Disable A/U law decompress or compress

enumerator **I2S_PCM_A_DECOMPRESS**

A-law decompress

enumerator **I2S_PCM_A_COMPRESS**

A-law compress

enumerator **I2S_PCM_U_DECOMPRESS**

U-law decompress

enumerator **I2S_PCM_U_COMPRESS**

U-law compress

enum **i2s_pdm_dsr_t**

I2S PDM RX down-sampling mode.

Values:

enumerator **I2S_PDM_DSR_8S**

downsampling number is 8 for PDM RX mode

enumerator **I2S_PDM_DSR_16S**

downsampling number is 16 for PDM RX mode

enumerator **I2S_PDM_DSR_MAX**

enum **i2s_pdm_sig_scale_t**

pdm tx singnal scaling mode

Values:

enumerator **I2S_PDM_SIG_SCALING_DIV_2**

I2S TX PDM signal scaling: /2

enumerator **I2S_PDM_SIG_SCALING_MUL_1**

I2S TX PDM signal scaling: x1

enumerator **I2S_PDM_SIG_SCALING_MUL_2**

I2S TX PDM signal scaling: x2

enumerator **I2S_PDM_SIG_SCALING_MUL_4**

I2S TX PDM signal scaling: x4

enum **i2s_pdm_tx_line_mode_t**

PDM TX line mode.

Note: For the standard codec mode, PDM pins are connect to a codec which requires both clock signal and data signal For the DAC output mode, PDM data signal can be connected to a power amplifier directly with a low-pass filter, normally, DAC output mode doesn't need the clock signal.

Values:

enumerator **I2S_PDM_TX_ONE_LINE_CODEC**

Standard PDM format output, left and right slot data on a single line

enumerator **I2S_PDM_TX_ONE_LINE_DAC**

PDM DAC format output, left or right slot data on a single line

enumerator **I2S_PDM_TX_TWO_LINE_DAC**

PDM DAC format output, left and right slot data on separated lines

enum **i2s_std_slot_mask_t**

I2S slot select in standard mode.

Note: It has different meanings in tx/rx/mono/stereo mode, and it may have differen behaviors on different targets For the details, please refer to the I2S API reference

---

Values:

enumerator **I2S_STD_SLOT_LEFT**

I2S transmits or receives left slot

enumerator **I2S_STD_SLOT_RIGHT**

I2S transmits or receives right slot

enumerator **I2S_STD_SLOT_BOTH**

I2S transmits or receives both left and right slot

enum **i2s_pdm_slot_mask_t**

I2S slot select in PDM mode.

Values:

enumerator **I2S_PDM_SLOT_RIGHT**

I2S PDM only transmits or receives the PDM device whose 'select' pin is pulled up

enumerator **I2S_PDM_SLOT_LEFT**

I2S PDM only transmits or receives the PDM device whose 'select' pin is pulled down

enumerator **I2S_PDM_SLOT_BOTH**

I2S PDM transmits or receives both two slots

enumerator **I2S_PDM_RX_LINE0_SLOT_RIGHT**

I2S PDM receives the right slot on line 0

enumerator **I2S_PDM_RX_LINE0_SLOT_LEFT**

I2S PDM receives the left slot on line 0

enumerator **I2S_PDM_RX_LINE1_SLOT_RIGHT**

I2S PDM receives the right slot on line 1

enumerator **I2S_PDM_RX_LINE1_SLOT_LEFT**

I2S PDM receives the left slot on line 1

enumerator **I2S_PDM_RX_LINE2_SLOT_RIGHT**

I2S PDM receives the right slot on line 2

enumerator **I2S_PDM_RX_LINE2_SLOT_LEFT**

I2S PDM receives the left slot on line 2

enumerator **I2S_PDM_RX_LINE3_SLOT_RIGHT**

I2S PDM receives the right slot on line 3

enumerator **I2S_PDM_RX_LINE3_SLOT_LEFT**

I2S PDM receives the left slot on line 3

enumerator **I2S_PDM_LINE_SLOT_ALL**

I2S PDM receives all slots

enum **i2s_tdm_slot_mask_t**

tdm slot number

Note: Multiple slots in TDM mode. For TX module, only the active slot send the audio data, the inactive slot send a constant or will be skipped if 'skip_msk' is set. For RX module, only receive the audio data in active slots, the data in inactive slots will be ignored. the bit map of active slot can not exceed (0x1<<total_slot_num). e.g: slot_mask = (I2S_TDM_SLOT0 | I2S_TDM_SLOT3), here the active slot number is 2 and total_slot is not supposed to be smaller than 4.

Values:

enumerator **I2S_TDM_SLOT0**

I2S slot 0 enabled

enumerator **I2S_TDM_SLOT1**

I2S slot 1 enabled

enumerator **I2S_TDM_SLOT2**

I2S slot 2 enabled

enumerator **I2S_TDM_SLOT3**

I2S slot 3 enabled

enumerator **I2S_TDM_SLOT4**

I2S slot 4 enabled

enumerator **I2S_TDM_SLOT5**

I2S slot 5 enabled

enumerator **I2S_TDM_SLOT6**

I2S slot 6 enabled

enumerator **I2S_TDM_SLOT7**

I2S slot 7 enabled

enumerator **I2S_TDM_SLOT8**

I2S slot 8 enabled

enumerator **I2S_TDM_SLOT9**

I2S slot 9 enabled

enumerator **I2S_TDM_SLOT10**

I2S slot 10 enabled

enumerator **I2S_TDM_SLOT11**

I2S slot 11 enabled

enumerator **I2S_TDM_SLOT12**

I2S slot 12 enabled

enumerator **I2S_TDM_SLOT13**

I2S slot 13 enabled

enumerator **I2S_TDM_SLOT14**

I2S slot 14 enabled

enumerator **I2S_TDM_SLOT15**

I2S slot 15 enabled

2.6.12 LCD

Introduction

ESP chips can generate various kinds of timings needed by common LCDs on the market, like SPI LCD, I2C LCD, Parallel LCD (Intel 8080), RGB/SRGB LCD, MIPI DSI LCD and etc. The `esp_lcd` component offers an abstracted driver framework to support them in a unified way.

An LCD typically consists of two main planes:

- **Control Plane**: This plane allows us to read and write to the internal registers of the LCD device controller. Host typically uses this plane for tasks such as initializing the LCD power supply and performing gamma calibration.
- **Data Plane**: The data plane is responsible for transmitting pixel data to the LCD device.

Functional Overview

In the context of `esp_lcd`, both the data plane and the control plane are represented by the *`esp_lcd_panel_handle_t`* type.

On some LCDs, these two planes may be combined into a single plane. In this configuration, pixel data is transmitted through the control plane, achieving functionality similar to that of the data plane. This merging is common in SPI LCDs and I2C LCDs. Additionally, there are LCDs that do not require a separate control plane. For instance, certain RGB LCDs automatically execute necessary initialization procedures after power-up. Host devices only need to continuously refresh pixel data through the data plane. However, it's essential to note that not all RGB LCDs eliminate the control plane entirely. Some LCD devices can simultaneously support multiple interfaces, requiring the host to send specific commands via the control plane (such as those based on the SPI interface) to enable the RGB mode.

This document will discuss how to create the control plane and data plane, as mentioned earlier, based on different types of LCDs.

SPI Interfaced LCD

1. Create an SPI bus. Please refer to *SPI Master API doc* for more details.

```
spi_bus_config_t buscfg = {
    .sclk_io_num = EXAMPLE_PIN_NUM_SCLK,
    .mosi_io_num = EXAMPLE_PIN_NUM_MOSI,
    .miso_io_num = EXAMPLE_PIN_NUM_MISO,
    .quadwp_io_num = -1, // Quad SPI LCD driver is not yet supported
    .quadhd_io_num = -1, // Quad SPI LCD driver is not yet supported
    .max_transfer_sz = EXAMPLE_LCD_H_RES * 80 * sizeof(uint16_t), //␣
↪transfer 80 lines of pixels (assume pixel is RGB565) at most in one␣
↪SPI transaction
};
ESP_ERROR_CHECK(spi_bus_initialize(LCD_HOST, &buscfg, SPI_DMA_CH_
↪AUTO)); // Enable the DMA feature
```

2. Allocate an LCD IO device handle from the SPI bus. In this step, you need to provide the following information:
    - *esp_lcd_panel_io_spi_config_t::dc_gpio_num*: Sets the gpio number for the DC signal line (some LCD calls this `RS` line). The LCD driver uses this GPIO to switch between sending command and sending data.
    - *esp_lcd_panel_io_spi_config_t::cs_gpio_num*: Sets the gpio number for the CS signal line. The LCD driver uses this GPIO to select the LCD chip. If the SPI bus only has one device attached (i.e., this LCD), you can set the gpio number to `-1` to occupy the bus exclusively.
    - *esp_lcd_panel_io_spi_config_t::pclk_hz* sets the frequency of the pixel clock, in Hz. The value should not exceed the range recommended in the LCD spec.
    - *esp_lcd_panel_io_spi_config_t::spi_mode* sets the SPI mode. The LCD driver uses this mode to communicate with the LCD. For the meaning of the SPI mode, please refer to the *SPI Master API doc*.
    - *esp_lcd_panel_io_spi_config_t::lcd_cmd_bits* and *esp_lcd_panel_io_spi_config_t::lcd_param_bits* set the bit width of the command and parameter that recognized by the LCD controller chip. This is chip specific, you should refer to your LCD spec in advance.
    - *esp_lcd_panel_io_spi_config_t::trans_queue_depth* sets the depth of the SPI transaction queue. A bigger value means more transactions can be queued up, but it also consumes more memory.

```
esp_lcd_panel_io_handle_t io_handle = NULL;
esp_lcd_panel_io_spi_config_t io_config = {
    .dc_gpio_num = EXAMPLE_PIN_NUM_LCD_DC,
    .cs_gpio_num = EXAMPLE_PIN_NUM_LCD_CS,
    .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
    .lcd_cmd_bits = EXAMPLE_LCD_CMD_BITS,
    .lcd_param_bits = EXAMPLE_LCD_PARAM_BITS,
    .spi_mode = 0,
    .trans_queue_depth = 10,
};
// Attach the LCD to the SPI bus
ESP_ERROR_CHECK(esp_lcd_new_panel_io_spi((esp_lcd_spi_bus_handle_t)LCD_
↪HOST, &io_config, &io_handle));
```

3. Install the LCD controller driver. The LCD controller driver is responsible for sending the commands and parameters to the LCD controller chip. In this step, you need to specify the SPI IO device handle that allocated in the last step, and some panel specific configurations:
    - `esp_lcd_panel_dev_config_t::reset_gpio_num` sets the LCD's hardware reset GPIO number. If the LCD does not have a hardware reset pin, set this to `-1`.
    - `esp_lcd_panel_dev_config_t::rgb_ele_order` sets the R-G-B element order of each color data.
    - `esp_lcd_panel_dev_config_t::bits_per_pixel` sets the bit width of the pixel color data. The LCD driver uses this value to calculate the number of bytes to send to the LCD controller chip.
    - `esp_lcd_panel_dev_config_t::data_endian` specifies the data endian to be transmitted to the screen. No need to specify for color data within 1 byte, like RGB232. For drivers that do not support specifying data endian, this field would be ignored.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_panel_dev_config_t panel_config = {
    .reset_gpio_num = EXAMPLE_PIN_NUM_RST,
    .rgb_ele_order = LCD_RGB_ELEMENT_ORDER_BGR,
    .bits_per_pixel = 16,
};
// Create LCD panel handle for ST7789, with the SPI IO device handle
ESP_ERROR_CHECK(esp_lcd_new_panel_st7789(io_handle, &panel_config, &
↪panel_handle));
```

API Reference

Header File

- components/esp_lcd/include/esp_lcd_io_spi.h
- This header file can be included with:

```
#include "esp_lcd_io_spi.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t **esp_lcd_new_panel_io_spi**(*esp_lcd_spi_bus_handle_t* bus, const *esp_lcd_panel_io_spi_config_t* *io_config, *esp_lcd_panel_io_handle_t* *ret_io)

Create LCD panel IO handle, for SPI interface.

Parameters

- **bus** -- **[in]** SPI bus handle
- **io_config** -- **[in]** IO configuration, for SPI interface
- **ret_io** -- **[out]** Returned IO handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

Structures

struct **esp_lcd_panel_io_spi_config_t**

Panel IO configuration structure, for SPI interface.

Public Members

int **cs_gpio_num**

GPIO used for CS line

int **dc_gpio_num**

GPIO used to select the D/C line, set this to -1 if the D/C line is not used

int **spi_mode**

Traditional SPI mode (0~3)

unsigned int **pclk_hz**

Frequency of pixel clock

size_t **trans_queue_depth**

Size of internal transaction queue

esp_lcd_panel_io_color_trans_done_cb_t **on_color_trans_done**

Callback invoked when color data transfer has finished

void ***user_ctx**

User private data, passed directly to on_color_trans_done's user_ctx

int **lcd_cmd_bits**

Bit-width of LCD command

int **lcd_param_bits**

Bit-width of LCD parameter

unsigned int **dc_high_on_cmd**

If enabled, DC level = 1 indicates command transfer

unsigned int **dc_low_on_data**

If enabled, DC level = 0 indicates color data transfer

unsigned int **dc_low_on_param**

If enabled, DC level = 0 indicates parameter transfer

unsigned int **octal_mode**

transmit with octal mode (8 data lines), this mode is used to simulate Intel 8080 timing

unsigned int **quad_mode**

transmit with quad mode (4 data lines), this mode is useful when transmitting LCD parameters (Only use one line for command)

unsigned int **sio_mode**

Read and write through a single data line (MOSI)

unsigned int **lsb_first**

transmit LSB bit first

unsigned int **cs_high_active**

CS line is high active

struct *esp_lcd_panel_io_spi_config_t*::[anonymous] **flags**

Extra flags to fine-tune the SPI device

Type Definitions

typedef int **esp_lcd_spi_bus_handle_t**

Type of LCD SPI bus handle

I2C Interfaced LCD

1. Create I2C bus. Please refer to *I2C API doc* for more details.

```
i2c_master_bus_handle_t i2c_bus = NULL;
i2c_master_bus_config_t bus_config = {
    .clk_source = I2C_CLK_SRC_DEFAULT,
    .glitch_ignore_cnt = 7,
    .i2c_port = I2C_BUS_PORT,
    .sda_io_num = EXAMPLE_PIN_NUM_SDA,
    .scl_io_num = EXAMPLE_PIN_NUM_SCL,
    .flags.enable_internal_pullup = true,
};
ESP_ERROR_CHECK(i2c_new_master_bus(&bus_config, &i2c_bus));
```

2. Allocate an LCD IO device handle from the I2C bus. In this step, you need to provide the following information:
   - *esp_lcd_panel_io_i2c_config_t::dev_addr* sets the I2C device address of the LCD controller chip. The LCD driver uses this address to communicate with the LCD controller chip.
   - *esp_lcd_panel_io_i2c_config_t::scl_speed_hz* sets the I2C clock frequency in Hz. The value should not exceed the range recommended in the LCD spec.
   - *esp_lcd_panel_io_i2c_config_t::lcd_cmd_bits* and *esp_lcd_panel_io_i2c_config_t::lcd_param_bits* set the bit width of the command and parameter that recognized by the LCD controller chip. This is chip specific, you should refer to your LCD spec in advance.

```
esp_lcd_panel_io_handle_t io_handle = NULL;
esp_lcd_panel_io_i2c_config_t io_config = {
    .dev_addr = EXAMPLE_I2C_HW_ADDR,
    .scl_speed_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
    .control_phase_bytes = 1, // refer to LCD spec
    .dc_bit_offset = 6,       // refer to LCD spec
    .lcd_cmd_bits = EXAMPLE_LCD_CMD_BITS,
    .lcd_param_bits = EXAMPLE_LCD_CMD_BITS,
};
ESP_ERROR_CHECK(esp_lcd_new_panel_io_i2c(i2c_bus, &io_config, &io_
↪handle));
```

3. Install the LCD controller driver. The LCD controller driver is responsible for sending the commands and parameters to the LCD controller chip. In this step, you need to specify the I2C IO device handle that allocated in the last step, and some panel specific configurations:
   - `esp_lcd_panel_dev_config_t::reset_gpio_num` sets the LCD's hardware reset GPIO number. If the LCD does not have a hardware reset pin, set this to `-1`.
   - `esp_lcd_panel_dev_config_t::bits_per_pixel` sets the bit width of the pixel color data. The LCD driver uses this value to calculate the number of bytes to send to the LCD controller chip.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_panel_dev_config_t panel_config = {
    .bits_per_pixel = 1,
    .reset_gpio_num = EXAMPLE_PIN_NUM_RST,
};
ESP_ERROR_CHECK(esp_lcd_new_panel_ssd1306(io_handle, &panel_config, &
↪panel_handle));
```

API Reference

Header File

- components/esp_lcd/include/esp_lcd_io_i2c.h
- This header file can be included with:

```
#include "esp_lcd_io_i2c.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t **esp_lcd_new_panel_io_i2c_v1**(uint32_t bus, const *esp_lcd_panel_io_i2c_config_t* *io_config, *esp_lcd_panel_io_handle_t* *ret_io)

Create LCD panel IO handle, for I2C interface in legacy implementation.

Note: Please don't call this function in your project directly. Please call `esp_lcd_new_panel_to_i2c` instead.

Parameters

- **bus** -- **[in]** I2C bus handle, (in uint32_t)
- **io_config** -- **[in]** IO configuration, for I2C interface
- **ret_io** -- **[out]** Returned IO handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

esp_err_t **esp_lcd_new_panel_io_i2c_v2**(*i2c_master_bus_handle_t* bus, const *esp_lcd_panel_io_i2c_config_t* *io_config, *esp_lcd_panel_io_handle_t* *ret_io)

Create LCD panel IO handle, for I2C interface in new implementation.

Note: Please don't call this function in your project directly. Please call `esp_lcd_new_panel_to_i2c` instead.

Parameters

- **bus** -- **[in]** I2C bus handle, (in i2c_master_dev_handle_t)
- **io_config** -- **[in]** IO configuration, for I2C interface
- **ret_io** -- **[out]** Returned IO handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

Structures

struct **esp_lcd_panel_io_i2c_config_t**

Panel IO configuration structure, for I2C interface.

Public Members

uint32_t **dev_addr**

I2C device address

esp_lcd_panel_io_color_trans_done_cb_t **on_color_trans_done**

Callback invoked when color data transfer has finished

void ***user_ctx**

User private data, passed directly to on_color_trans_done's user_ctx

size_t **control_phase_bytes**

I2C LCD panel will encode control information (e.g. D/C selection) into control phase, in several bytes

unsigned int **dc_bit_offset**

Offset of the D/C selection bit in control phase

int **lcd_cmd_bits**

Bit-width of LCD command

int **lcd_param_bits**

Bit-width of LCD parameter

unsigned int **dc_low_on_data**

If this flag is enabled, DC line = 0 means transfer data, DC line = 1 means transfer command; vice versa

unsigned int **disable_control_phase**

If this flag is enabled, the control phase isn't used

struct *esp_lcd_panel_io_i2c_config_t*::[anonymous] **flags**

Extra flags to fine-tune the I2C device

uint32_t **scl_speed_hz**

I2C LCD SCL frequency (hz)

Macros

esp_lcd_new_panel_io_i2c(bus, io_config, ret_io)

Create LCD panel IO handle.

Parameters

- **bus** -- **[in]** I2C bus handle
- **io_config** -- **[in]** IO configuration, for I2C interface
- **ret_io** -- **[out]** Returned IO handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

Type Definitions

typedef uint32_t **esp_lcd_i2c_bus_handle_t**

Type of LCD I2C bus handle

I80 Interfaced LCD

1. Create I80 bus by *esp_lcd_new_i80_bus()*. You need to set up the following parameters for an Intel 8080 parallel bus:
    - *esp_lcd_i80_bus_config_t::clk_src* sets the clock source of the I80 bus. Note, the default clock source may be different between ESP targets.
    - *esp_lcd_i80_bus_config_t::wr_gpio_num* sets the GPIO number of the pixel clock (also referred as `WR` in some LCD spec)
    - *esp_lcd_i80_bus_config_t::dc_gpio_num* sets the GPIO number of the data/command select pin (also referred as `RS` in some LCD spec)
    - *esp_lcd_i80_bus_config_t::bus_width* sets the bit width of the data bus (only support `8` or `16`)
    - *esp_lcd_i80_bus_config_t::data_gpio_nums* is the array of the GPIO number of the data bus. The number of GPIOs should be equal to the *esp_lcd_i80_bus_config_t::bus_width* value.
    - *esp_lcd_i80_bus_config_t::max_transfer_bytes* sets the maximum number of bytes that can be transferred in one transaction.

```
esp_lcd_i80_bus_handle_t i80_bus = NULL;
esp_lcd_i80_bus_config_t bus_config = {
    .clk_src = LCD_CLK_SRC_DEFAULT,
    .dc_gpio_num = EXAMPLE_PIN_NUM_DC,
    .wr_gpio_num = EXAMPLE_PIN_NUM_PCLK,
    .data_gpio_nums = {
        EXAMPLE_PIN_NUM_DATA0,
        EXAMPLE_PIN_NUM_DATA1,
        EXAMPLE_PIN_NUM_DATA2,
        EXAMPLE_PIN_NUM_DATA3,
        EXAMPLE_PIN_NUM_DATA4,
        EXAMPLE_PIN_NUM_DATA5,
        EXAMPLE_PIN_NUM_DATA6,
        EXAMPLE_PIN_NUM_DATA7,
    },
    .bus_width = 8,
    .max_transfer_bytes = EXAMPLE_LCD_H_RES * 100 * sizeof(uint16_t), /
↪/ transfer 100 lines of pixels (assume pixel is RGB565) at most in␣
↪one transaction
    .dma_burst_size = EXAMPLE_DMA_BURST_SIZE,
};
ESP_ERROR_CHECK(esp_lcd_new_i80_bus(&bus_config, &i80_bus));
```

2. Allocate an LCD IO device handle from the I80 bus. In this step, you need to provide the following information:
    - *esp_lcd_panel_io_i80_config_t::cs_gpio_num* sets the GPIO number of the chip select pin.
    - *esp_lcd_panel_io_i80_config_t::pclk_hz* sets the pixel clock frequency in Hz. Higher pixel clock frequency results in higher refresh rate, but may cause flickering if the DMA bandwidth is not sufficient or the LCD controller chip does not support high pixel clock frequency.
    - *esp_lcd_panel_io_i80_config_t::lcd_cmd_bits* and *esp_lcd_panel_io_i80_config_t::lcd_param_bits* set the bit width of the command and parameter that recognized by the LCD controller chip. This is chip specific, you should refer to your LCD spec in advance.
    - *esp_lcd_panel_io_i80_config_t::trans_queue_depth* sets the maximum number of transactions that can be queued in the LCD IO device. A bigger value means more transactions can be queued up, but it also consumes more memory.

```
esp_lcd_panel_io_handle_t io_handle = NULL;
esp_lcd_panel_io_i80_config_t io_config = {
    .cs_gpio_num = EXAMPLE_PIN_NUM_CS,
    .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
    .trans_queue_depth = 10,
```

(continues on next page)

(continued from previous page)

```
    .dc_levels = {
        .dc_idle_level = 0,
        .dc_cmd_level = 0,
        .dc_dummy_level = 0,
        .dc_data_level = 1,
    },
    .lcd_cmd_bits = EXAMPLE_LCD_CMD_BITS,
    .lcd_param_bits = EXAMPLE_LCD_PARAM_BITS,
};
ESP_ERROR_CHECK(esp_lcd_new_panel_io_i80(i80_bus, &io_config, &io_
↪handle));
```

3. Install the LCD controller driver. The LCD controller driver is responsible for sending the commands and parameters to the LCD controller chip. In this step, you need to specify the I80 IO device handle that allocated in the last step, and some panel specific configurations:
    - `esp_lcd_panel_dev_config_t::bits_per_pixel` sets the bit width of the pixel color data. The LCD driver uses this value to calculate the number of bytes to send to the LCD controller chip.
    - `esp_lcd_panel_dev_config_t::reset_gpio_num` sets the GPIO number of the reset pin. If the LCD controller chip does not have a reset pin, you can set this value to `-1`.
    - `esp_lcd_panel_dev_config_t::rgb_ele_order` sets the color order the pixel color data.

```
esp_lcd_panel_dev_config_t panel_config = {
    .reset_gpio_num = EXAMPLE_PIN_NUM_RST,
    .rgb_ele_order = LCD_RGB_ELEMENT_ORDER_RGB,
    .bits_per_pixel = 16,
};
ESP_ERROR_CHECK(esp_lcd_new_panel_st7789(io_handle, &panel_config, &
↪panel_handle));
```

API Reference

Header File

- components/esp_lcd/include/esp_lcd_io_i80.h
- This header file can be included with:

```
#include "esp_lcd_io_i80.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t **esp_lcd_new_i80_bus**(const *esp_lcd_i80_bus_config_t* *bus_config, *esp_lcd_i80_bus_handle_t* *ret_bus)

Create Intel 8080 bus handle.

Parameters

- **bus_config** -- **[in]** Bus configuration
- **ret_bus** -- **[out]** Returned bus handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid

- ESP_ERR_NO_MEM if out of memory
- ESP_ERR_NOT_FOUND if no free bus is available
- ESP_OK on success

esp_err_t **esp_lcd_del_i80_bus**(*esp_lcd_i80_bus_handle_t* bus)

Destroy Intel 8080 bus handle.

Parameters **bus** -- **[in]** Intel 8080 bus handle, created by `esp_lcd_new_i80_bus()`

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_INVALID_STATE if there still be some device attached to the bus
- ESP_OK on success

esp_err_t **esp_lcd_new_panel_io_i80**(*esp_lcd_i80_bus_handle_t* bus, const *esp_lcd_panel_io_i80_config_t* *io_config, *esp_lcd_panel_io_handle_t* *ret_io)

Create LCD panel IO, for Intel 8080 interface.

Parameters

- **bus** -- **[in]** Intel 8080 bus handle, created by `esp_lcd_new_i80_bus()`
- **io_config** -- **[in]** IO configuration, for i80 interface
- **ret_io** -- **[out]** Returned panel IO handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NOT_SUPPORTED if some configuration can't be satisfied, e.g. pixel clock out of the range
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

void ***esp_lcd_i80_alloc_draw_buffer**(*esp_lcd_panel_io_handle_t* io, size_t size, uint32_t caps)

Allocate a draw buffer that can be used by I80 interfaced LCD panel.

Note: This function differs from the normal 'heap_caps_*' functions in that it can also automatically handle the alignment required by DMA burst, cache line size, etc.

Parameters

- **io** -- **[in]** Panel IO handle, created by `esp_lcd_new_panel_io_i80()`
- **size** -- **[in]** Size of memory to be allocated
- **caps** -- **[in]** Bitwise OR of MALLOC_CAP_* flags indicating the type of memory desired for the allocation

Returns Pointer to a new buffer of size 'size' with capabilities 'caps', or NULL if allocation failed

Structures

struct **esp_lcd_i80_bus_config_t**

LCD Intel 8080 bus configuration structure.

Public Members

int **dc_gpio_num**

GPIO used for D/C line

int **wr_gpio_num**

GPIO used for WR line

lcd_clock_source_t **clk_src**

Clock source for the I80 LCD peripheral

int **data_gpio_nums**[ESP_LCD_I80_BUS_WIDTH_MAX]

GPIOs used for data lines

size_t **bus_width**

Number of data lines, 8 or 16

size_t **max_transfer_bytes**

Maximum transfer size, this determines the length of internal DMA link

size_t **psram_trans_align**

DMA transfer alignment for data allocated from PSRAM

size_t **dma_burst_size**

DMA burst size, in bytes

size_t **sram_trans_align**

DMA transfer alignment for data allocated from SRAM

struct **esp_lcd_panel_io_i80_config_t**

Panel IO configuration structure, for intel 8080 interface.

Public Members

int **cs_gpio_num**

GPIO used for CS line, set to -1 will declaim exclusively use of I80 bus

uint32_t **pclk_hz**

Frequency of pixel clock

size_t **trans_queue_depth**

Transaction queue size, larger queue, higher throughput

esp_lcd_panel_io_color_trans_done_cb_t **on_color_trans_done**

Callback invoked when color data was transferred done

void ***user_ctx**

User private data, passed directly to on_color_trans_done's user_ctx

int **lcd_cmd_bits**

Bit-width of LCD command

int **lcd_param_bits**

Bit-width of LCD parameter

unsigned int **dc_idle_level**

Level of DC line in IDLE phase

unsigned int **dc_cmd_level**

Level of DC line in CMD phase

unsigned int **dc_dummy_level**

Level of DC line in DUMMY phase

unsigned int **dc_data_level**

Level of DC line in DATA phase

struct *esp_lcd_panel_io_i80_config_t*::[anonymous] **dc_levels**

Each i80 device might have its own D/C control logic

unsigned int **cs_active_high**

If set, a high level of CS line will select the device, otherwise, CS line is low level active

unsigned int **reverse_color_bits**

Reverse the data bits, D[N:0] -> D[0:N]

unsigned int **swap_color_bytes**

Swap adjacent two color bytes

unsigned int **pclk_active_neg**

The display will write data lines when there's a falling edge on WR signal (a.k.a the PCLK)

unsigned int **pclk_idle_low**

The WR signal (a.k.a the PCLK) stays at low level in IDLE phase

struct *esp_lcd_panel_io_i80_config_t*::[anonymous] **flags**

Panel IO config flags

Macros

ESP_LCD_I80_BUS_WIDTH_MAX

Maximum width of I80 bus

Type Definitions

typedef struct esp_lcd_i80_bus_t ***esp_lcd_i80_bus_handle_t**

Type of LCD intel 8080 bus handle

RGB Interfaced LCD RGB LCD panel is allocated in one step: *esp_lcd_new_rgb_panel()*, with various configurations specified by *esp_lcd_rgb_panel_config_t*.

- *esp_lcd_rgb_panel_config_t::clk_src* selects the clock source for the RGB LCD controller. The available clock sources are listed in *lcd_clock_source_t*.
- *esp_lcd_rgb_panel_config_t::data_width* set number of data lines used by the RGB interface. Currently, the supported value can be 8 or 16.

- *esp_lcd_rgb_panel_config_t::bits_per_pixel* set the number of bits per pixel. This is different from *esp_lcd_rgb_panel_config_t::data_width*. By default, if you set this field to 0, the driver will automatically adjust the bpp to the *esp_lcd_rgb_panel_config_t::data_width*. But in some cases, these two value must be different. For example, a Serial RGB interface LCD only needs `8` data lines, but the color width can reach to `RGB888`, i.e., the *esp_lcd_rgb_panel_config_t::bits_per_pixel* should be set to `24`.
- *esp_lcd_rgb_panel_config_t::hsync_gpio_num*, *esp_lcd_rgb_panel_config_t::vsync_gpio_nu*
  esp_lcd_rgb_panel_config_t::de_gpio_num, *esp_lcd_rgb_panel_config_t::pclk_gpio_num*,
  esp_lcd_rgb_panel_config_t::disp_gpio_num and *esp_lcd_rgb_panel_config_t::data_gpio_nu*
  are the GPIO pins used by the RGB LCD controller. If some of them are not used, please set it to *-1*.
- *esp_lcd_rgb_panel_config_t::dma_burst_size* set the DMA transfer burst size, the value must be a power of 2.
- *esp_lcd_rgb_panel_config_t::bounce_buffer_size_px* set the size of bounce buffer. This is only necessary for a so-called "bounce buffer" mode. Please refer to *Bounce Buffer with Single PSRAM Frame Buffer* for more information.
- *esp_lcd_rgb_panel_config_t::timings* sets the LCD panel specific timing parameters. All required parameters are listed in the *esp_lcd_rgb_timing_t*, including the LCD resolution and blanking porches. Please fill them according to the datasheet of your LCD.
- *esp_lcd_rgb_panel_config_t::fb_in_psram* sets whether to allocate the frame buffer from PSRAM or not. Please refer to *Single Frame Buffer in PSRAM* for more information.
- *esp_lcd_rgb_panel_config_t::num_fbs* sets the number of frame buffers allocated by the driver. For backward compatibility, `0` means to allocate `one` frame buffer. Please use *esp_lcd_rgb_panel_config_t::no_fb* if you do not want to allocate any frame buffer.
- *esp_lcd_rgb_panel_config_t::no_fb* if sets, no frame buffer will be allocated. This is also called the *Bounce Buffer Only* mode.

RGB LCD Frame Buffer Operation Modes Most of the time, the RGB LCD driver should maintain at least one screen sized frame buffer. According to the number and location of the frame buffer, the driver provides several different buffer modes.

Single Frame Buffer in Internal Memory This is the default and simplest and you do not have to specify flags or bounce buffer options. A frame buffer is allocated from the internal memory. The frame data is read out by DMA to the LCD verbatim. It needs no CPU intervention to function, but it has the downside that it uses up a fair bit of the limited amount of internal memory.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_rgb_panel_config_t panel_config = {
    .data_width = 16, // RGB565 in parallel mode, thus 16bit in width
    .clk_src = LCD_CLK_SRC_DEFAULT,
    .disp_gpio_num = EXAMPLE_PIN_NUM_DISP_EN,
    .pclk_gpio_num = EXAMPLE_PIN_NUM_PCLK,
    .vsync_gpio_num = EXAMPLE_PIN_NUM_VSYNC,
    .hsync_gpio_num = EXAMPLE_PIN_NUM_HSYNC,
    .de_gpio_num = EXAMPLE_PIN_NUM_DE,
    .data_gpio_nums = {
        EXAMPLE_PIN_NUM_DATA0,
        EXAMPLE_PIN_NUM_DATA1,
        EXAMPLE_PIN_NUM_DATA2,
        // other GPIOs
        // The number of GPIOs here should be the same to the value of␣
↪`data_width` above
        ...
    },
    // The timing parameters should refer to your LCD spec
    .timings = {
        .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
        .h_res = EXAMPLE_LCD_H_RES,
        .v_res = EXAMPLE_LCD_V_RES,
```

(continues on next page)

(continued from previous page)

```
        .hsync_back_porch = 40,
        .hsync_front_porch = 20,
        .hsync_pulse_width = 1,
        .vsync_back_porch = 8,
        .vsync_front_porch = 4,
        .vsync_pulse_width = 1,
    },
};
ESP_ERROR_CHECK(esp_lcd_new_rgb_panel(&panel_config, &panel_handle));
```

Single Frame Buffer in PSRAM If you have PSRAM and want to store the frame buffer there rather than in the limited internal memory, the LCD peripheral will use EDMA to fetch frame data directly from the PSRAM, bypassing the internal cache. You can enable this feature by setting the *esp_lcd_rgb_panel_config_t::fb_in_psram* to `true`. The downside of this is that when both the CPU as well as EDMA need access to the PSRAM, the bandwidth will be **shared** between them, that is, EDMA gets half and the CPUs get the other half. If there are other peripherals using EDMA as well, with a high enough pixel clock this can lead to starvation of the LCD peripheral, leading to display corruption. However, if the pixel clock is low enough for this not to be an issue, this is a solution that uses almost no CPU intervention.

The PSRAM shares the same SPI bus with the main Flash (the one stores your firmware binary). At one time, there only be one consumer of the SPI bus. When you also use the main flash to serve your file system (e.g., *SPIFFS*), the bandwidth of the underlying SPI bus will also be shared, leading to display corruption. You can use *esp_lcd_rgb_panel_set_pclk()* to update the pixel clock frequency to a lower value.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_rgb_panel_config_t panel_config = {
    .data_width = 16, // RGB565 in parallel mode, thus 16bit in width
    .clk_src = LCD_CLK_SRC_DEFAULT,
    .disp_gpio_num = EXAMPLE_PIN_NUM_DISP_EN,
    .pclk_gpio_num = EXAMPLE_PIN_NUM_PCLK,
    .vsync_gpio_num = EXAMPLE_PIN_NUM_VSYNC,
    .hsync_gpio_num = EXAMPLE_PIN_NUM_HSYNC,
    .de_gpio_num = EXAMPLE_PIN_NUM_DE,
    .data_gpio_nums = {
        EXAMPLE_PIN_NUM_DATA0,
        EXAMPLE_PIN_NUM_DATA1,
        EXAMPLE_PIN_NUM_DATA2,
        // other GPIOs
        // The number of GPIOs here should be the same to the value of␣
↪`data_width` above
        ...
    },
    // The timing parameters should refer to your LCD spec
    .timings = {
        .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
        .h_res = EXAMPLE_LCD_H_RES,
        .v_res = EXAMPLE_LCD_V_RES,
        .hsync_back_porch = 40,
        .hsync_front_porch = 20,
        .hsync_pulse_width = 1,
        .vsync_back_porch = 8,
        .vsync_front_porch = 4,
        .vsync_pulse_width = 1,
    },
    .flags.fb_in_psram = true, // allocate frame buffer from PSRAM
};
ESP_ERROR_CHECK(esp_lcd_new_rgb_panel(&panel_config, &panel_handle));
```

Double Frame Buffer in PSRAM To avoid tearing effect, using two screen sized frame buffers is the easiest approach. In this mode, the frame buffer can only be allocated from PSRAM, because of the limited internal memory. The frame buffer that the CPU write to and the frame buffer that the EDMA read from are guaranteed to be different and independent. The EDMA will only switch between the two frame buffers when the previous write operation is finished and the current frame has been sent to the LCD. The downside of this mode is that, you have to maintain the synchronization between the two frame buffers.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_rgb_panel_config_t panel_config = {
    .data_width = 16, // RGB565 in parallel mode, thus 16bit in width
    .num_fbs = 2,     // allocate double frame buffer
    .clk_src = LCD_CLK_SRC_DEFAULT,
    .disp_gpio_num = EXAMPLE_PIN_NUM_DISP_EN,
    .pclk_gpio_num = EXAMPLE_PIN_NUM_PCLK,
    .vsync_gpio_num = EXAMPLE_PIN_NUM_VSYNC,
    .hsync_gpio_num = EXAMPLE_PIN_NUM_HSYNC,
    .de_gpio_num = EXAMPLE_PIN_NUM_DE,
    .data_gpio_nums = {
        EXAMPLE_PIN_NUM_DATA0,
        EXAMPLE_PIN_NUM_DATA1,
        EXAMPLE_PIN_NUM_DATA2,
        // other GPIOs
        // The number of GPIOs here should be the same to the value of␣
↪`data_width` above
        ...
    },
    // The timing parameters should refer to your LCD spec
    .timings = {
        .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
        .h_res = EXAMPLE_LCD_H_RES,
        .v_res = EXAMPLE_LCD_V_RES,
        .hsync_back_porch = 40,
        .hsync_front_porch = 20,
        .hsync_pulse_width = 1,
        .vsync_back_porch = 8,
        .vsync_front_porch = 4,
        .vsync_pulse_width = 1,
    },
    .flags.fb_in_psram = true, // allocate frame buffer from PSRAM
};
ESP_ERROR_CHECK(esp_lcd_new_rgb_panel(&panel_config, &panel_handle));
```

Bounce Buffer with Single PSRAM Frame Buffer This mode allocates two so-called `bounce buffers` from the internal memory, and a main frame buffer that is still in PSRAM. This mode is selected by setting the *esp_lcd_rgb_panel_config_t::fb_in_psram* flag and additionally specifying a non-zero *esp_lcd_rgb_panel_config_t::bounce_buffer_size_px* value. The bounce buffers only need to be large enough to hold a few lines of display data, which is significantly less than the main frame buffer. The LCD peripheral uses DMA to read data from one of the bounce buffers, and meanwhile an interrupt routine uses the CPU DCache to copy data from the main PSRAM frame buffer into the other bounce buffer. Once the LCD peripheral has finished reading the bounce buffer, the two buffers change place and the CPU can fill the others. The advantage of this mode is that, you can achieve higher pixel clock frequency. As the bounce buffers are larger than the FIFOs in the EDMA path, this method is also more robust against short bandwidth spikes. The downside is a major increase in CPU use and the LCD **CAN NOT** work if we disable the cache of the external memory, via e.g., OTA or NVS write to the main flash.

Note: It is highly recommended to turn on the "PSRAM XIP (Execute In Place)" feature in this mode by enabling the Kconfig options: *CONFIG_SPIRAM_FETCH_INSTRUCTIONS* and *CONFIG_SPIRAM_RODATA*, which allows the CPU to fetch instructions and readonly data from the PSRAM instead of the main flash. What is more, the external memory cache will not be disabled even if you

attempt to write to the main flash through SPI1. This makes it possible to display an OTA progress bar for your application.

Note: This mode still has another problem which is also caused by insufficient PSRAM bandwidth. e.g., when your draw buffers are allocated from PSRAM, and their contents are copied into the internal frame buffer on CPU core 1. On CPU core 0, there is another memory copy happening in the DMA EOF ISR. In this situation, both CPUs are accessing the PSRAM by cache and sharing the bandwidth of the PSRAM. This increases the memory copy time that spent in the DMA EOF ISR significantly. The driver can not switch the bounce buffer in time, thus leading to a shift on the LCD screen. Although the driver can detect such a condition and perform a restart in the LCD's VSYNC interrupt handler, you still can see a flickering on the screen.

```
esp_lcd_panel_handle_t panel_handle = NULL;
esp_lcd_rgb_panel_config_t panel_config = {
    .data_width = 16, // RGB565 in parallel mode, thus 16bit in width
    .clk_src = LCD_CLK_SRC_DEFAULT,
    .bounce_buffer_size_px = 10 * EXAMPLE_LCD_H_RES, // allocate 10 lines␣
↪data as bounce buffer from internal memory
    .disp_gpio_num = EXAMPLE_PIN_NUM_DISP_EN,
    .pclk_gpio_num = EXAMPLE_PIN_NUM_PCLK,
    .vsync_gpio_num = EXAMPLE_PIN_NUM_VSYNC,
    .hsync_gpio_num = EXAMPLE_PIN_NUM_HSYNC,
    .de_gpio_num = EXAMPLE_PIN_NUM_DE,
    .data_gpio_nums = {
        EXAMPLE_PIN_NUM_DATA0,
        EXAMPLE_PIN_NUM_DATA1,
        EXAMPLE_PIN_NUM_DATA2,
        // other GPIOs
        // The number of GPIOs here should be the same to the value of␣
↪`data_width` above
        ...
    },
    // The timing parameters should refer to your LCD spec
    .timings = {
        .pclk_hz = EXAMPLE_LCD_PIXEL_CLOCK_HZ,
        .h_res = EXAMPLE_LCD_H_RES,
        .v_res = EXAMPLE_LCD_V_RES,
        .hsync_back_porch = 40,
        .hsync_front_porch = 20,
        .hsync_pulse_width = 1,
        .vsync_back_porch = 8,
        .vsync_front_porch = 4,
        .vsync_pulse_width = 1,
    },
    .flags.fb_in_psram = true, // allocate frame buffer from PSRAM
};
ESP_ERROR_CHECK(esp_lcd_new_rgb_panel(&panel_config, &panel_handle));
```

Note that this mode also allows for a *esp_lcd_rgb_panel_config_t::bb_invalidate_cache* flag to be set. Enabling this frees up the cache lines after they are used to read out the frame buffer data from PSRAM, but it may lead to slight corruption if the other core writes data to the frame buffer at the exact time the cache lines are freed up. (Technically, a write to the frame buffer can be ignored if it falls between the cache writeback and the cache invalidate calls.)

Bounce Buffer Only This mode is similar to the *Bounce Buffer with Single PSRAM Frame Buffer*, but there is no PSRAM frame buffer initialized by the LCD driver. Instead, the user supplies a callback function that is responsible for filling the bounce buffers. As this driver does not care where the written pixels come from, this allows for the callback doing e.g., on-the-fly conversion from a

smaller, 8-bit-per-pixel PSRAM frame buffer to an 16-bit LCD, or even procedurally-generated frame-buffer-less graphics. This option is selected by setting the *esp_lcd_rgb_panel_config_t::no_fb* flag and supplying a *esp_lcd_rgb_panel_config_t::bounce_buffer_size_px* value. And then register the *esp_lcd_rgb_panel_event_callbacks_t::on_bounce_empty* callback by calling *esp_lcd_rgb_panel_register_event_callbacks()*.

Note: It should never happen in a well-designed embedded application, but it can in theory be possible that the DMA cannot deliver data as fast as the LCD consumes it. In the ESP32-S3 hardware, this leads to the LCD simply outputting dummy bytes while DMA waits for data. If we were to run DMA in a stream fashion, this would mean a de-sync between the LCD address the DMA reads the data for and the LCD address the LCD peripheral thinks it outputs data for, leading to a **permanently** shifted image. In order to stop this from happening, you can either enable the *CONFIG_LCD_RGB_RESTART_IN_VSYNC* option, so the driver can restart the DMA in the VBlank interrupt automatically or call *esp_lcd_rgb_panel_restart()* to restart the DMA manually. Note *esp_lcd_rgb_panel_restart()* does not restart the DMA immediately, the DMA is still restarted in the next VSYNC event.

API Reference

Header File

- components/esp_lcd/rgb/include/esp_lcd_panel_rgb.h
- This header file can be included with:

```
#include "esp_lcd_panel_rgb.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t **esp_lcd_new_rgb_panel**(const *esp_lcd_rgb_panel_config_t* *rgb_panel_config, *esp_lcd_panel_handle_t* *ret_panel)

Create RGB LCD panel.

Parameters

- **rgb_panel_config** -- **[in]** RGB panel configuration
- **ret_panel** -- **[out]** Returned LCD panel handle

Returns

- ESP_ERR_INVALID_ARG: Create RGB LCD panel failed because of invalid argument
- ESP_ERR_NO_MEM: Create RGB LCD panel failed because of out of memory
- ESP_ERR_NOT_FOUND: Create RGB LCD panel failed because some mandatory hardware resources are not found
- ESP_OK: Create RGB LCD panel successfully

esp_err_t **esp_lcd_rgb_panel_register_event_callbacks**(*esp_lcd_panel_handle_t* panel, const *esp_lcd_rgb_panel_event_callbacks_t* *callbacks, void *user_ctx)

Register LCD RGB panel event callbacks.

Parameters

- **panel** -- **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
- **callbacks** -- **[in]** Group of callback functions

- **user_ctx** -- **[in]** User data, which will be passed to the callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **esp_lcd_rgb_panel_set_pclk**(*esp_lcd_panel_handle_t* panel, uint32_t freq_hz)

Set frequency of PCLK for RGB LCD panel.

Note: The PCLK frequency is set in the *esp_lcd_rgb_timing_t* and gets configured during LCD panel initialization. Usually you don't need to call this function to set the PCLK again, but in some cases, you might want to change the PCLK frequency. e.g. slow down the PCLK frequency to reduce power consumption or to reduce the memory throughput during OTA.

Note: This function doesn't cause the hardware to update the PCLK immediately but to record the new frequency and set a flag internally. Only in the next VSYNC event handler, will the driver attempt to update the PCLK frequency.

Parameters

- **panel** -- **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
- **freq_hz** -- **[in]** Frequency of pixel clock, in Hz

Returns

- ESP_ERR_INVALID_ARG: Set PCLK frequency failed because of invalid argument
- ESP_OK: Set PCLK frequency successfully

esp_err_t **esp_lcd_rgb_panel_restart**(*esp_lcd_panel_handle_t* panel)

Restart the LCD transmission.

Note: This function can be useful when the LCD controller is out of sync with the DMA because of insufficient bandwidth. To save the screen from a permanent shift, you can call this function to restart the LCD DMA.

Note: This function doesn't restart the DMA immediately but to set a flag internally. Only in the next VSYNC event handler, will the driver attempt to do the restart job.

Note: If CONFIG_LCD_RGB_RESTART_IN_VSYNC is enabled, you don't need to call this function manually, because the restart job will be done automatically in the VSYNC event handler.

Parameters **panel** -- **[in]** panel LCD panel handle, returned from `esp_lcd_new_rgb_panel`

Returns

- ESP_ERR_INVALID_ARG: Restart the LCD failed because of invalid argument
- ESP_ERR_INVALID_STATE: Restart the LCD failed because the LCD diver is working in refresh-on-demand mode
- ESP_OK: Restart the LCD successfully

esp_err_t **esp_lcd_rgb_panel_get_frame_buffer**(*esp_lcd_panel_handle_t* panel, uint32_t fb_num, void **fb0, ...)

Get the address of the frame buffer(s) that allocated by the driver.

Parameters

- **panel** -- **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`

- **fb_num** -- **[in]** Number of frame buffer(s) to get. This value must be the same as the number of the following parameters.
- **fb0** -- **[out]** Returned address of the frame buffer 0
- **...** -- **[out]** List of other frame buffer addresses

Returns

- ESP_ERR_INVALID_ARG: Get frame buffer address failed because of invalid argument
- ESP_OK: Get frame buffer address successfully

esp_err_t **esp_lcd_rgb_panel_refresh**(*esp_lcd_panel_handle_t* panel)

Manually trigger once transmission of the frame buffer to the LCD panel.

Note: This function should only be called when the RGB panel is working under the `refresh_on_demand` mode.

Parameters **panel** -- **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`

Returns

- ESP_ERR_INVALID_ARG: Start a refresh failed because of invalid argument
- ESP_ERR_INVALID_STATE: Start a refresh failed because the LCD panel is not created with the `refresh_on_demand` flag enabled.
- ESP_OK: Start a refresh successfully

esp_err_t **esp_lcd_rgb_panel_set_yuv_conversion**(*esp_lcd_panel_handle_t* panel, const *esp_lcd_yuv_conv_config_t* *config)

Configure how to convert the color format between RGB and YUV.

Note: Pass in `config` as NULL will disable the RGB-YUV converter.

Note: The hardware converter can only parse a "packed" storage format, while "planar" and "semi-planar" format is not supported.

Parameters

- **panel** -- **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
- **config** -- **[in]** Configuration of RGB-YUV conversion

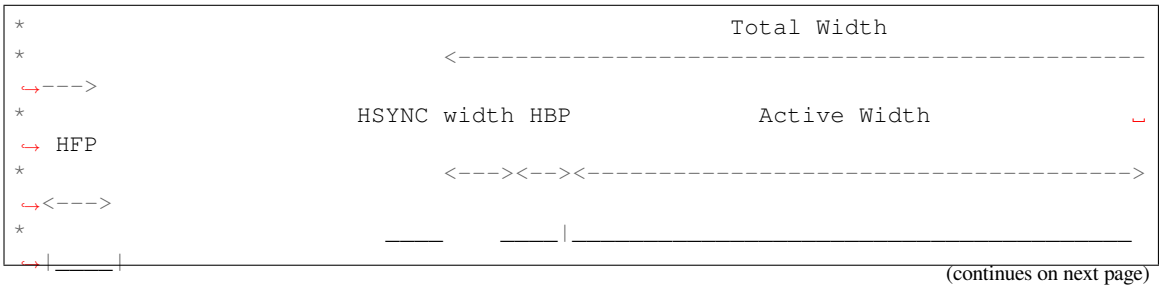
Returns

- ESP_ERR_INVALID_ARG: Configure RGB-YUV conversion failed because of invalid argument
- ESP_ERR_NOT_SUPPORTED: Configure RGB-YUV conversion failed because the conversion mode is not supported by the hardware
- ESP_OK: Configure RGB-YUV conversion successfully

Structures

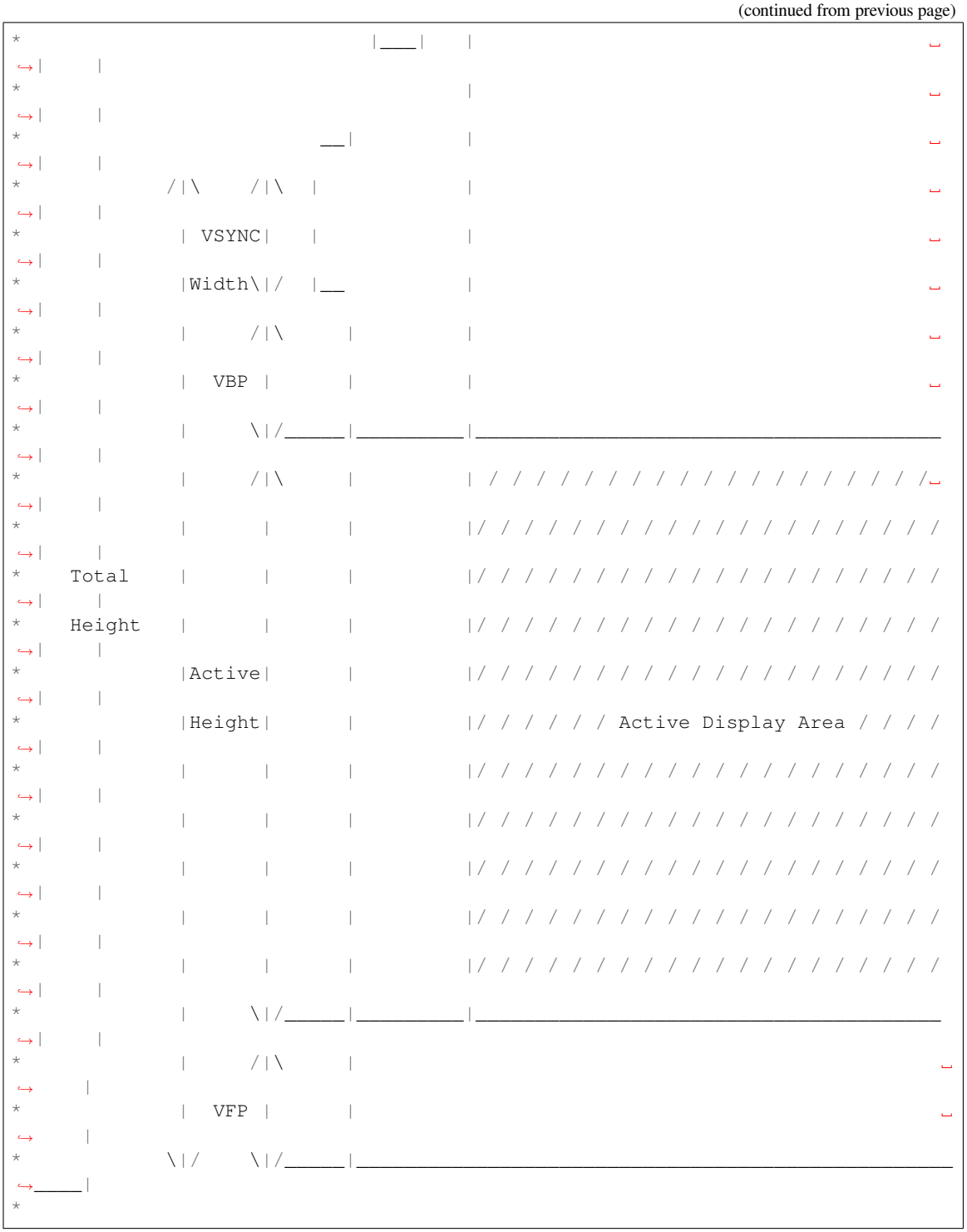
struct **esp_lcd_rgb_timing_t**

LCD RGB timing structure.



(continues on next page)

(continued from previous page)



Public Members

uint32_t **pclk_hz**

Frequency of pixel clock

uint32_t **h_res**

Horizontal resolution, i.e. the number of pixels in a line

uint32_t **v_res**

Vertical resolution, i.e. the number of lines in the frame

uint32_t **hsync_pulse_width**

Horizontal sync width, unit: PCLK period

uint32_t **hsync_back_porch**

Horizontal back porch, number of PCLK between hsync and start of line active data

uint32_t **hsync_front_porch**

Horizontal front porch, number of PCLK between the end of active data and the next hsync

uint32_t **vsync_pulse_width**

Vertical sync width, unit: number of lines

uint32_t **vsync_back_porch**

Vertical back porch, number of invalid lines between vsync and start of frame

uint32_t **vsync_front_porch**

Vertical front porch, number of invalid lines between the end of frame and the next vsync

uint32_t **hsync_idle_low**

The hsync signal is low in IDLE state

uint32_t **vsync_idle_low**

The vsync signal is low in IDLE state

uint32_t **de_idle_high**

The de signal is high in IDLE state

uint32_t **pclk_active_neg**

Whether the display data is clocked out on the falling edge of PCLK

uint32_t **pclk_idle_high**

The PCLK stays at high level in IDLE phase

struct *esp_lcd_rgb_timing_t*::[anonymous] **flags**

LCD RGB timing flags

struct **esp_lcd_rgb_panel_event_data_t**

Type of RGB LCD panel event data.

struct **esp_lcd_rgb_panel_event_callbacks_t**

Group of supported RGB LCD panel callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_LCD_RGB_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM.

Public Members

esp_lcd_rgb_panel_vsync_cb_t **on_vsync**

VSYNC event callback

esp_lcd_rgb_panel_bounce_buf_fill_cb_t **on_bounce_empty**

Bounce buffer empty callback.

esp_lcd_rgb_panel_bounce_buf_finish_cb_t **on_bounce_frame_finish**

Bounce buffer finish callback.

struct **esp_lcd_rgb_panel_config_t**

LCD RGB panel configuration structure.

Public Members

lcd_clock_source_t **clk_src**

Clock source for the RGB LCD peripheral

esp_lcd_rgb_timing_t **timings**

RGB timing parameters, including the screen resolution

size_t **data_width**

Number of data lines

size_t **bits_per_pixel**

Frame buffer color depth, in bpp, specially, if set to zero, it will default to `data_width`. When using a Serial RGB interface, this value could be different from `data_width`

size_t **num_fbs**

Number of screen-sized frame buffers that allocated by the driver. By default (set to either 0 or 1) only one frame buffer will be used. Maximum number of buffers are 3

size_t **bounce_buffer_size_px**

If it's non-zero, the driver allocates two DRAM bounce buffers for DMA use. DMA fetching from DRAM bounce buffer is much faster than PSRAM frame buffer.

size_t **sram_trans_align**

Alignment of buffers (frame buffer or bounce buffer) that allocated in SRAM

size_t **psram_trans_align**

Alignment of buffers (frame buffer) that allocated in PSRAM

size_t **dma_burst_size**

DMA burst size, in bytes

int **hsync_gpio_num**

GPIO used for HSYNC signal

int **vsync_gpio_num**

GPIO used for VSYNC signal

int **de_gpio_num**

GPIO used for DE signal, set to -1 if it's not used

int **pclk_gpio_num**

GPIO used for PCLK signal, set to -1 if it's not used

int **disp_gpio_num**

GPIO used for display control signal, set to -1 if it's not used

int **data_gpio_nums**[(16)]

GPIOs used for data lines

uint32_t **disp_active_low**

If this flag is enabled, a low level of display control signal can turn the screen on; vice versa

uint32_t **refresh_on_demand**

If this flag is enabled, the host only refresh the frame buffer when `esp_lcd_panel_draw_bitmap` is called. This is useful when the LCD screen has a GRAM and can refresh the LCD by itself.

uint32_t **fb_in_psram**

If this flag is enabled, the frame buffer will be allocated from PSRAM, preferentially

uint32_t **double_fb**

If this flag is enabled, the driver will allocate two screen sized frame buffer, same as num_fbs=2

uint32_t **no_fb**

If this flag is enabled, the driver won't allocate frame buffer. Instead, user should fill in the bounce buffer manually in the `on_bounce_empty` callback

uint32_t **bb_invalidate_cache**

If this flag is enabled, in bounce back mode we'll do a cache invalidate on the read data, freeing the cache. Can be dangerous if data is written from other core(s).

struct *esp_lcd_rgb_panel_config_t*::[anonymous] **flags**

LCD RGB panel configuration flags

struct **esp_lcd_color_conv_profile_t**

LCD color conversion profile.

Public Members

lcd_color_space_t **color_space**

Color space of the image

lcd_color_range_t **color_range**

Color range of the image

lcd_yuv_sample_t **yuv_sample**

YUV sample format of the image

struct **esp_lcd_yuv_conv_config_t**

Configuration of YUG-RGB conversion.

Public Members

lcd_yuv_conv_std_t **std**

YUV conversion standard: BT601, BT709

esp_lcd_color_conv_profile_t **src**

Color conversion profile of the input image

esp_lcd_color_conv_profile_t **dst**

Color conversion profile of the output image

Type Definitions

typedef bool (***esp_lcd_rgb_panel_vsync_cb_t**)(*esp_lcd_panel_handle_t* panel, const *esp_lcd_rgb_panel_event_data_t* *edata, void *user_ctx)

RGB LCD VSYNC event callback prototype.

Param panel **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
Param edata **[in]** Panel event data, fed by driver
Param user_ctx **[in]** User data, passed from `esp_lcd_rgb_panel_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***esp_lcd_rgb_panel_bounce_buf_fill_cb_t**)(*esp_lcd_panel_handle_t* panel, void *bounce_buf, int pos_px, int len_bytes, void *user_ctx)

Prototype for function to re-fill a bounce buffer, rather than copying from the frame buffer.

Param panel **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
Param bounce_buf **[in]** Bounce buffer to write data into
Param pos_px **[in]** How many pixels already were sent to the display in this frame, in other words, at what pixel the routine should start putting data into bounce_buf
Param len_bytes **[in]** Length, in bytes, of the bounce buffer. Routine should fill this length fully.
Param user_ctx **[in]** Opaque pointer that was passed from `esp_lcd_rgb_panel_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***esp_lcd_rgb_panel_bounce_buf_finish_cb_t**)(*esp_lcd_panel_handle_t* panel, const *esp_lcd_rgb_panel_event_data_t* *edata, void *user_ctx)

Prototype for the function to be called when the bounce buffer finish copying the entire frame.

Param panel **[in]** LCD panel handle, returned from `esp_lcd_new_rgb_panel`
Param edata **[in]** Panel event data, fed by driver
Param user_ctx **[in]** User data, passed from `esp_lcd_rgb_panel_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

---

Note: ESP-IDF provides only a limited number of LCD device controller drivers out of the box (e.g., ST7789), more drivers are available in the Espressif Component Registry.

LCD Control Panel Operations

- *`esp_lcd_panel_reset()`* can reset the LCD control panel.
- *`esp_lcd_panel_init()`* performs a basic initialization of the control panel. To perform more manufacture specific initialization, please refer to *Steps to Add Manufacture Specific Initialization*.
- By combining using *`esp_lcd_panel_swap_xy()`* and *`esp_lcd_panel_mirror()`*, you can achieve the functionality of rotating or mirroring the LCD screen.
- *`esp_lcd_panel_disp_on_off()`* can turn on or off the LCD screen by cutting down the output path from the frame buffer to the LCD screen. Please note, this is not controlling the LCD backlight. Backlight control is not covered by the `esp_lcd` driver.
- *`esp_lcd_panel_disp_sleep()`* can reduce the power consumption of the LCD screen by entering the sleep mode. The internal frame buffer is still retained.

LCD Data Panel Operations

- *`esp_lcd_panel_reset()`* can reset the LCD data panel.
- *`esp_lcd_panel_init()`* performs a basic initialization of the data panel.
- *`esp_lcd_panel_draw_bitmap()`* is the function which does the magic to flush the user draw buffer to the LCD screen, where the target draw window is configurable. Please note, this function expects the draw buffer is a 1-D array and there's no stride in between each lines.

Steps to Add Manufacture Specific Initialization

The LCD controller drivers (e.g., st7789) in ESP-IDF only provide basic initialization in the *`esp_lcd_panel_init()`*, leaving the vast majority of settings to the default values. Some LCD modules needs to set a bunch of manufacture specific configurations before it can display normally. These configurations usually include gamma, power voltage and so on. If you want to add manufacture specific initialization, please follow the steps below:

```
esp_lcd_panel_reset(panel_handle);
esp_lcd_panel_init(panel_handle);
// set extra configurations e.g., gamma control
// with the underlying IO handle
// please consult your manufacture for special commands and corresponding values
esp_lcd_panel_io_tx_param(io_handle, GAMMA_CMD, (uint8_t[]) {
        GAMMA_ARRAY
    }, N);
// turn on the display
esp_lcd_panel_disp_on_off(panel_handle, true);
```

Application Example

- Software JPEG decoding and display - peripherals/lcd/tjpgd
- Universal SPI LCD example with SPI touch - peripherals/lcd/spi_lcd_touch
- i80 controller based LCD and LVGL animation UI - peripherals/lcd/i80_controller

- RGB panel example with scatter chart UI - peripherals/lcd/rgb_panel
- I2C interfaced OLED display scrolling text - peripherals/lcd/i2c_oled

API Reference

Header File

- components/hal/include/hal/lcd_types.h
- This header file can be included with:

```
#include "hal/lcd_types.h"
```

Type Definitions

typedef *soc_periph_lcd_clk_src_t* **lcd_clock_source_t**

LCD clock source.

Enumerations

enum **lcd_rgb_data_endian_t**

RGB data endian.

Values:

enumerator **LCD_RGB_DATA_ENDIAN_BIG**

RGB data endian: MSB first

enumerator **LCD_RGB_DATA_ENDIAN_LITTLE**

RGB data endian: LSB first

enum **lcd_color_space_t**

LCD color space.

Values:

enumerator **LCD_COLOR_SPACE_RGB**

Color space: RGB

enumerator **LCD_COLOR_SPACE_YUV**

Color space: YUV

enum **lcd_color_rgb_pixel_format_t**

LCD color pixel format in RGB color space.

Values:

enumerator **LCD_COLOR_PIXEL_FORMAT_RGB565**

16 bits, 5 bits per R/B value, 6 bits for G value

enumerator **LCD_COLOR_PIXEL_FORMAT_RGB666**

18 bits, 6 bits per R/G/B value

enumerator **LCD_COLOR_PIXEL_FORMAT_RGB888**

24 bits, 8 bits per R/G/B value

enum **lcd_color_range_t**

LCD color range.

Values:

enumerator **LCD_COLOR_RANGE_LIMIT**

Limited color range

enumerator **LCD_COLOR_RANGE_FULL**

Full color range

enum **lcd_yuv_sample_t**

YUV sampling method.

Values:

enumerator **LCD_YUV_SAMPLE_422**

YUV 4:2:2 sampling

enumerator **LCD_YUV_SAMPLE_420**

YUV 4:2:0 sampling

enumerator **LCD_YUV_SAMPLE_411**

YUV 4:1:1 sampling

enum **lcd_yuv_conv_std_t**

The standard used for conversion between RGB and YUV.

Values:

enumerator **LCD_YUV_CONV_STD_BT601**

YUV<->RGB conversion standard: BT.601

enumerator **LCD_YUV_CONV_STD_BT709**

YUV<->RGB conversion standard: BT.709

Header File

- components/esp_lcd/include/esp_lcd_types.h
- This header file can be included with:

```
#include "esp_lcd_types.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Structures

struct **esp_lcd_video_timing_t**

Timing parameters for the video data transmission.

Public Members

uint32_t **h_size**

Horizontal resolution, i.e. the number of pixels in a line

uint32_t **v_size**

Vertical resolution, i.e. the number of lines in the frame

uint32_t **hsync_pulse_width**

Horizontal sync width, in pixel clock

uint32_t **hsync_back_porch**

Horizontal back porch, number of pixel clock between hsync and start of line active data

uint32_t **hsync_front_porch**

Horizontal front porch, number of pixel clock between the end of active data and the next hsync

uint32_t **vsync_pulse_width**

Vertical sync width, in number of lines

uint32_t **vsync_back_porch**

Vertical back porch, number of invalid lines between vsync and start of frame

uint32_t **vsync_front_porch**

Vertical front porch, number of invalid lines between the end of frame and the next vsync

struct **esp_lcd_panel_io_event_data_t**

Type of LCD panel IO event data.

struct **esp_lcd_panel_io_callbacks_t**

Type of LCD panel IO callbacks.

Public Members

esp_lcd_panel_io_color_trans_done_cb_t **on_color_trans_done**

Callback invoked when color data transfer has finished

Type Definitions

typedef struct esp_lcd_panel_io_t ***esp_lcd_panel_io_handle_t**

Type of LCD panel IO handle

typedef struct esp_lcd_panel_t ***esp_lcd_panel_handle_t**

Type of LCD panel handle

typedef bool (*`esp_lcd_panel_io_color_trans_done_cb_t`)(*esp_lcd_panel_io_handle_t* panel_io, *esp_lcd_panel_io_event_data_t* *edata, void *user_ctx)

Declare the prototype of the function that will be invoked when panel IO finishes transferring color data.

Param panel_io [in] LCD panel IO handle, which is created by factory API like `esp_lcd_new_panel_io_spi()`
Param edata [in] Panel IO event data, fed by driver
Param user_ctx [in] User data, passed from `esp_lcd_panel_io_xxx_config_t`
Return Whether a high priority task has been waken up by this function

Enumerations

enum `lcd_rgb_element_order_t`

RGB element order.

Values:

enumerator `LCD_RGB_ELEMENT_ORDER_RGB`

RGB element order: RGB

enumerator `LCD_RGB_ELEMENT_ORDER_BGR`

RGB element order: BGR

Header File

- components/esp_lcd/include/esp_lcd_panel_io.h
- This header file can be included with:

```
#include "esp_lcd_panel_io.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t `esp_lcd_panel_io_rx_param`(*esp_lcd_panel_io_handle_t* io, int lcd_cmd, void *param, size_t param_size)

Transmit LCD command and receive corresponding parameters.

Note: Commands sent by this function are short, so they are sent using polling transactions. The function does not return before the command transfer is completed. If any queued transactions sent by `esp_lcd_panel_io_tx_color()` are still pending when this function is called, this function will wait until they are finished and the queue is empty before sending the command(s).

Parameters

- **`io`** -- **[in]** LCD panel IO handle, which is created by other factory API like `esp_lcd_new_panel_io_spi()`
- **`lcd_cmd`** -- **[in]** The specific LCD command, set to -1 if no command needed
- **`param`** -- **[out]** Buffer for the command data
- **`param_size`** -- **[in]** Size of `param` buffer

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NOT_SUPPORTED if read is not supported by transport
- ESP_OK on success

esp_err_t **esp_lcd_panel_io_tx_param**(*esp_lcd_panel_io_handle_t* io, int lcd_cmd, const void *param, size_t param_size)

Transmit LCD command and corresponding parameters.

Note: Commands sent by this function are short, so they are sent using polling transactions. The function does not return before the command transfer is completed. If any queued transactions sent by `esp_lcd_panel_io_tx_color()` are still pending when this function is called, this function will wait until they are finished and the queue is empty before sending the command(s).

Parameters

- **io** -- **[in]** LCD panel IO handle, which is created by other factory API like `esp_lcd_new_panel_io_spi()`
- **lcd_cmd** -- **[in]** The specific LCD command, set to -1 if no command needed
- **param** -- **[in]** Buffer that holds the command specific parameters, set to NULL if no parameter is needed for the command
- **param_size** -- **[in]** Size of `param` in memory, in bytes, set to zero if no parameter is needed for the command

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

esp_err_t **esp_lcd_panel_io_tx_color**(*esp_lcd_panel_io_handle_t* io, int lcd_cmd, const void *color, size_t color_size)

Transmit LCD RGB data.

Note: This function will package the command and RGB data into a transaction, and push into a queue. The real transmission is performed in the background (DMA+interrupt). The caller should take care of the lifecycle of the `color` buffer. Recycling of color buffer should be done in the callback `on_color_trans_done()`.

Parameters

- **io** -- **[in]** LCD panel IO handle, which is created by factory API like `esp_lcd_new_panel_io_spi()`
- **lcd_cmd** -- **[in]** The specific LCD command, set to -1 if no command needed
- **color** -- **[in]** Buffer that holds the RGB color data
- **color_size** -- **[in]** Size of `color` in memory, in bytes

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

esp_err_t **esp_lcd_panel_io_del**(*esp_lcd_panel_io_handle_t* io)

Destroy LCD panel IO handle (deinitialize panel and free all corresponding resource)

Parameters **io** -- **[in]** LCD panel IO handle, which is created by factory API like `esp_lcd_new_panel_io_spi()`

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

esp_err_t **esp_lcd_panel_io_register_event_callbacks**(*esp_lcd_panel_io_handle_t* io, const *esp_lcd_panel_io_callbacks_t* *cbs, void *user_ctx)

Register LCD panel IO callbacks.

Parameters

- **io** -- **[in]** LCD panel IO handle, which is created by factory API like `esp_lcd_new_panel_io_spi()`
- **cbs** -- **[in]** structure with all LCD panel IO callbacks
- **user_ctx** -- **[in]** User private data, passed directly to callback's user_ctx

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

Header File

- components/esp_lcd/include/esp_lcd_panel_ops.h
- This header file can be included with:

```
#include "esp_lcd_panel_ops.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

Functions

esp_err_t **esp_lcd_panel_reset**(*esp_lcd_panel_handle_t* panel)

Reset LCD panel.

Note: Panel reset must be called before attempting to initialize the panel using `esp_lcd_panel_init()`.

Parameters **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_init**(*esp_lcd_panel_handle_t* panel)

Initialize LCD panel.

Note: Before calling this function, make sure the LCD panel has finished the `reset` stage by `esp_lcd_panel_reset()`.

Parameters **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_del**(*esp_lcd_panel_handle_t* panel)

Deinitialize the LCD panel.

Parameters **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_draw_bitmap**(*esp_lcd_panel_handle_t* panel, int x_start, int y_start, int x_end, int y_end, const void *color_data)

Draw bitmap on LCD panel.

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **x_start** -- **[in]** Start pixel index in the target frame buffer, on x-axis (x_start is included)
- **y_start** -- **[in]** Start pixel index in the target frame buffer, on y-axis (y_start is included)
- **x_end** -- **[in]** End pixel index in the target frame buffer, on x-axis (x_end is not included)
- **y_end** -- **[in]** End pixel index in the target frame buffer, on y-axis (y_end is not included)
- **color_data** -- **[in]** RGB color data that will be dumped to the specific window range

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_mirror**(*esp_lcd_panel_handle_t* panel, bool mirror_x, bool mirror_y)

Mirror the LCD panel on specific axis.

Note: Combined with `esp_lcd_panel_swap_xy()`, one can realize screen rotation

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **mirror_x** -- **[in]** Whether the panel will be mirrored about the x axis
- **mirror_y** -- **[in]** Whether the panel will be mirrored about the y axis

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if this function is not supported by the panel

esp_err_t **esp_lcd_panel_swap_xy**(*esp_lcd_panel_handle_t* panel, bool swap_axes)

Swap/Exchange x and y axis.

Note: Combined with `esp_lcd_panel_mirror()`, one can realize screen rotation

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **swap_axes** -- **[in]** Whether to swap the x and y axis

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if this function is not supported by the panel

esp_err_t **esp_lcd_panel_set_gap**(*esp_lcd_panel_handle_t* panel, int x_gap, int y_gap)

Set extra gap in x and y axis.

The gap is the space (in pixels) between the left/top sides of the LCD panel and the first row/column respectively of the actual contents displayed.

Note: Setting a gap is useful when positioning or centering a frame that is smaller than the LCD.

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`

- **x_gap** -- **[in]** Extra gap on x axis, in pixels
- **y_gap** -- **[in]** Extra gap on y axis, in pixels

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_invert_color**(*esp_lcd_panel_handle_t* panel, bool invert_color_data)

Invert the color (bit-wise invert the color data line)

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **invert_color_data** -- **[in]** Whether to invert the color data

Returns

- ESP_OK on success

esp_err_t **esp_lcd_panel_disp_on_off**(*esp_lcd_panel_handle_t* panel, bool on_off)

Turn on or off the display.

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **on_off** -- **[in]** True to turns on display, False to turns off display

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if this function is not supported by the panel

esp_err_t **esp_lcd_panel_disp_off**(*esp_lcd_panel_handle_t* panel, bool off)

Turn off the display.

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **off** -- **[in]** Whether to turn off the screen

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if this function is not supported by the panel

esp_err_t **esp_lcd_panel_disp_sleep**(*esp_lcd_panel_handle_t* panel, bool sleep)

Enter or exit sleep mode.

Parameters

- **panel** -- **[in]** LCD panel handle, which is created by other factory API like `esp_lcd_new_panel_st7789()`
- **sleep** -- **[in]** True to enter sleep mode, False to wake up

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if this function is not supported by the panel

Header File

- components/esp_lcd/include/esp_lcd_panel_vendor.h
- This header file can be included with:

```
#include "esp_lcd_panel_vendor.h"
```

- This header file is a part of the API provided by the `esp_lcd` component. To declare that your component depends on `esp_lcd`, add the following to your CMakeLists.txt:

```
REQUIRES esp_lcd
```

or

```
PRIV_REQUIRES esp_lcd
```

2.6.13 LED Control (LEDC)

Introduction

The LED control (LEDC) peripheral is primarily designed to control the intensity of LEDs, although it can also be used to generate PWM signals for other purposes. It has 8 channels which can generate independent waveforms that can be used, for example, to drive RGB LED devices.

The PWM controller can automatically increase or decrease the duty cycle gradually, allowing for fades without any processor interference.

Functionality Overview

Setting up a channel of the LEDC is done in three steps. Note that unlike ESP32, ESP32-S3 only supports configuring channels in "low speed" mode.

1. *Timer Configuration* by specifying the PWM signal's frequency and duty cycle resolution.
2. *Channel Configuration* by associating it with the timer and GPIO to output the PWM signal.
3. *Change PWM Signal* that drives the output in order to change LED's intensity. This can be done under the full control of software or with hardware fading functions.

As an optional step, it is also possible to set up an interrupt on fade end.

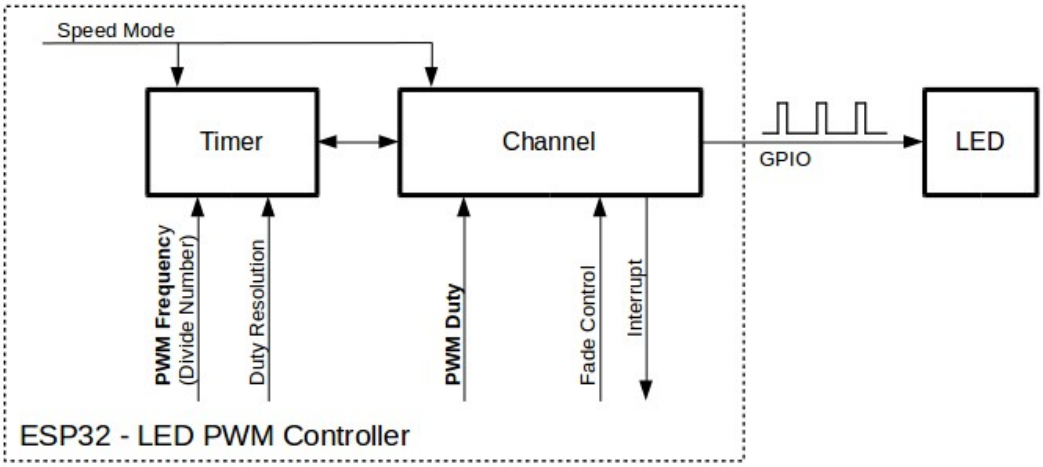


Fig. 15: Key Settings of LED PWM Controller's API

Note: For an initial setup, it is recommended to configure for the timers first (by calling `ledc_timer_config()`), and then for the channels (by calling `ledc_channel_config()`). This ensures the PWM frequency is at the desired value since the appearance of the PWM signal from the IO pad.

Timer Configuration Setting the timer is done by calling the function *ledc_timer_config()* and passing the data structure *ledc_timer_config_t* that contains the following configuration settings:

- Speed mode (value must be LEDC_LOW_SPEED_MODE)
- Timer number *ledc_timer_t*
- PWM signal frequency in Hz
- Resolution of PWM duty
- Source clock *ledc_clk_cfg_t*

The frequency and the duty resolution are interdependent. The higher the PWM frequency, the lower the duty resolution which is available, and vice versa. This relationship might be important if you are planning to use this API for purposes other than changing the intensity of LEDs. For more details, see Section *Supported Range of Frequency and Duty Resolutions*.

The source clock can also limit the PWM frequency. The higher the source clock frequency, the higher the maximum PWM frequency can be configured.

Table 3: Characteristics of ESP32-S3 LEDC source clocks

Clock name	Clock freq	Clock capabilities
APB_CLK	80 MHz	/
RC_FAST_CLK	~ 20 MHz	Dynamic Frequency Scaling compatible, Light sleep compatible
XTAL_CLK	40 MHz	Dynamic Frequency Scaling compatible

Note:

1. On ESP32-S3, if RC_FAST_CLK is chosen as the LEDC clock source, an internal calibration will be performed to get the exact frequency of the clock. This ensures the accuracy of output PWM signal frequency.
2. For ESP32-S3, all timers share one clock source. In other words, it is impossible to use different clock sources for different timers.

The LEDC driver offers a helper function *ledc_find_suitable_duty_resolution()* to find the maximum possible resolution for the timer, given the source clock frequency and the desired PWM signal frequency.

When a timer is no longer needed by any channel, it can be deconfigured by calling the same function *ledc_timer_config()*. The configuration structure *ledc_timer_config_t* passes in should be:

- *ledc_timer_config_t::speed_mode* The speed mode of the timer which wants to be deconfigured belongs to (*ledc_mode_t*)
- *ledc_timer_config_t::timer_num* The ID of the timers which wants to be deconfigured (*ledc_timer_t*)
- *ledc_timer_config_t::deconfigure* Set this to true so that the timer specified can be deconfigured

Channel Configuration When the timer is set up, configure the desired channel (one out of *ledc_channel_t*). This is done by calling the function *ledc_channel_config()*.

Similar to the timer configuration, the channel setup function should be passed a structure *ledc_channel_config_t* that contains the channel's configuration parameters.

At this point, the channel should start operating and generating the PWM signal on the selected GPIO, as configured in *ledc_channel_config_t*, with the frequency specified in the timer settings and the given duty cycle. The channel operation (signal generation) can be suspended at any time by calling the function *ledc_stop()*.

Change PWM Signal Once the channel starts operating and generating the PWM signal with the constant duty cycle and frequency, there are a couple of ways to change this signal. When driving LEDs, primarily the duty cycle is changed to vary the light intensity.

The following two sections describe how to change the duty cycle using software and hardware fading. If required, the signal's frequency can also be changed; it is covered in Section *Change PWM Frequency*.

Note: All the timers and channels in the ESP32-S3's LED PWM Controller only support low speed mode. Any change of PWM settings must be explicitly triggered by software (see below).

Change PWM Duty Cycle Using Software To set the duty cycle, use the dedicated function `ledc_set_duty()`. After that, call `ledc_update_duty()` to activate the changes. To check the currently set value, use the corresponding `_get_` function `ledc_get_duty()`.

Another way to set the duty cycle, as well as some other channel parameters, is by calling `ledc_channel_config()` covered in Section *Channel Configuration*.

The range of the duty cycle values passed to functions depends on selected `duty_resolution` and should be from `0` to `(2 ** duty_resolution)`. For example, if the selected duty resolution is 10, then the duty cycle values can range from 0 to 1024. This provides the resolution of ~ 0.1%.

Warning: On ESP32-S3, when channel's binded timer selects its maximum duty resolution, the duty cycle value cannot be set to `(2 ** duty_resolution)`. Otherwise, the internal duty counter in the hardware will overflow and be messed up.

Change PWM Duty Cycle Using Hardware The LEDC hardware provides the means to gradually transition from one duty cycle value to another. To use this functionality, enable fading with `ledc_fade_func_install()` and then configure it by calling one of the available fading functions:

- `ledc_set_fade_with_time()`
- `ledc_set_fade_with_step()`
- `ledc_set_fade()`

Start fading with `ledc_fade_start()`. A fade can be operated in blocking or non-blocking mode, please check `ledc_fade_mode_t` for the difference between the two available fade modes. Note that with either fade mode, the next fade or fixed-duty update will not take effect until the last fade finishes or is stopped. `ledc_fade_stop()` has to be called to stop a fade that is in progress.

To get a notification about the completion of a fade operation, a fade end callback function can be registered for each channel by calling `ledc_cb_register()` after the fade service being installed. The fade end callback prototype is defined in `ledc_cb_t`, where you should return a boolean value from the callback function, indicating whether a high priority task is woken up by this callback function. It is worth mentioning, the callback and the function invoked by itself should be placed in IRAM, as the interrupt service routine is in IRAM. `ledc_cb_register()` will print a warning message if it finds the addresses of callback and user context are incorrect.

If not required anymore, fading and an associated interrupt can be disabled with `ledc_fade_func_uninstall()`.

Change PWM Frequency The LEDC API provides several ways to change the PWM frequency "on the fly":

- Set the frequency by calling `ledc_set_freq()`. There is a corresponding function `ledc_get_freq()` to check the current frequency.
- Change the frequency and the duty resolution by calling `ledc_bind_channel_timer()` to bind some other timer to the channel.
- Change the channel's timer by calling `ledc_channel_config()`.

More Control Over PWM There are several lower level timer-specific functions that can be used to change PWM settings:

- `ledc_timer_set()`

- *ledc_timer_rst()*
- *ledc_timer_pause()*
- *ledc_timer_resume()*

The first two functions are called "behind the scenes" by *ledc_channel_config()* to provide a startup of a timer after it is configured.

Use Interrupts When configuring an LEDC channel, one of the parameters selected within *ledc_channel_config_t* is *ledc_intr_type_t* which triggers an interrupt on fade completion.

For registration of a handler to address this interrupt, call *ledc_isr_register()*.

Supported Range of Frequency and Duty Resolutions

The LED PWM Controller is designed primarily to drive LEDs. It provides a large flexibility of PWM duty cycle settings. For instance, the PWM frequency of 5 kHz can have the maximum duty resolution of 13 bits. This means that the duty can be set anywhere from 0 to 100% with a resolution of ~ 0.012% (2 ** 13 = 8192 discrete levels of the LED intensity). Note, however, that these parameters depend on the clock signal clocking the LED PWM Controller timer which in turn clocks the channel (see *timer configuration* and the **ESP32-S3 Technical Reference Manual > LED PWM Controller (LEDC)** [PDF]).

The LEDC can be used for generating signals at much higher frequencies that are sufficient enough to clock other devices, e.g., a digital camera module. In this case, the maximum available frequency is 40 MHz with duty resolution of 1 bit. This means that the duty cycle is fixed at 50% and cannot be adjusted.

The LEDC API is designed to report an error when trying to set a frequency and a duty resolution that exceed the range of LEDC's hardware. For example, an attempt to set the frequency to 20 MHz and the duty resolution to 3 bits results in the following error reported on a serial monitor:

```
E (196) ledc: requested frequency and duty resolution cannot be achieved, try␣
↪reducing freq_hz or duty_resolution. div_param=128
```

In such a situation, either the duty resolution or the frequency must be reduced. For example, setting the duty resolution to 2 resolves this issue and makes it possible to set the duty cycle at 25% steps, i.e., at 25%, 50% or 75%.

The LEDC driver also captures and reports attempts to configure frequency/duty resolution combinations that are below the supported minimum, e.g.,:

```
E (196) ledc: requested frequency and duty resolution cannot be achieved, try␣
↪increasing freq_hz or duty_resolution. div_param=128000000
```

The duty resolution is normally set using *ledc_timer_bit_t*. This enumeration covers the range from 10 to 15 bits. If a smaller duty resolution is required (from 10 down to 1), enter the equivalent numeric values directly.

Application Example

The LEDC basic example: peripherals/ledc/ledc_basic.

The LEDC change duty cycle and fading control example: peripherals/ledc/ledc_fade.

API Reference

Header File

- components/esp_driver_ledc/include/driver/ledc.h
- This header file can be included with:

```
#include "driver/ledc.h"
```

- This header file is a part of the API provided by the `esp_driver_ledc` component. To declare that your component depends on `esp_driver_ledc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_ledc
```

or

```
PRIV_REQUIRES esp_driver_ledc
```

Functions

esp_err_t **ledc_channel_config**(const *ledc_channel_config_t* *ledc_conf)

LEDC channel configuration Configure LEDC channel with the given channel/output gpio_num/interrupt/source timer/frequency(Hz)/LEDC duty.

Parameters **ledc_conf** -- Pointer of LEDC channel configure struct

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

uint32_t **ledc_find_suitable_duty_resolution**(uint32_t src_clk_freq, uint32_t timer_freq)

Helper function to find the maximum possible duty resolution in bits for ledc_timer_config()

Parameters

- **src_clk_freq** -- LEDC timer source clock frequency (Hz) (See doxygen comments of `ledc_clk_cfg_t` or get from `esp_clk_tree_src_get_freq_hz`)
- **timer_freq** -- Desired LEDC timer frequency (Hz)

Returns

- 0 The timer frequency cannot be achieved
- Others The largest duty resolution value to be set

esp_err_t **ledc_timer_config**(const *ledc_timer_config_t* *timer_conf)

LEDC timer configuration Configure LEDC timer with the given source timer/frequency(Hz)/duty_resolution.

Parameters **timer_conf** -- Pointer of LEDC timer configure struct

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Can not find a proper pre-divider number base on the given frequency and the current duty_resolution.
- ESP_ERR_INVALID_STATE Timer cannot be de-configured because timer is not configured or is not paused

esp_err_t **ledc_update_duty**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel)

LEDC update channel parameters.

Note: Call this function to activate the LEDC updated parameters. After ledc_set_duty, we need to call this function to update the settings. And the new LEDC parameters don't take effect until the next PWM cycle.

Note: ledc_set_duty, ledc_set_duty_with_hpoint and ledc_update_duty are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_duty_and_update

---

Note: If `CONFIG_LEDC_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Cache is disabled.

Note: This function is allowed to run within ISR context.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **ledc_set_pin**(int gpio_num, *ledc_mode_t* speed_mode, *ledc_channel_t* ledc_channel)

Set LEDC output gpio.

Note: This function only routes the LEDC signal to GPIO through matrix, other LEDC resources initialization are not involved. Please use `ledc_channel_config()` instead to fully configure a LEDC channel.

Parameters

- **gpio_num** -- The LEDC output gpio
- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **ledc_channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **ledc_stop**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t idle_level)

LEDC stop. Disable LEDC output, and set idle level.

Note: If `CONFIG_LEDC_CTRL_FUNC_IN_IRAM` is enabled, this function will be placed in the IRAM by linker, makes it possible to execute even when the Cache is disabled.

Note: This function is allowed to run within ISR context.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **idle_level** -- Set output idle level after LEDC stops.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **ledc_set_freq**(*ledc_mode_t* speed_mode, *ledc_timer_t* timer_num, uint32_t freq_hz)

LEDC set channel frequency (Hz)

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.

- **timer_num** -- LEDC timer index (0-3), select from ledc_timer_t
- **freq_hz** -- Set the LEDC frequency

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Can not find a proper pre-divider number base on the given frequency and the current duty_resolution.

uint32_t **ledc_get_freq**(*ledc_mode_t* speed_mode, *ledc_timer_t* timer_num)

LEDC get channel frequency (Hz)

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_num** -- LEDC timer index (0-3), select from ledc_timer_t

Returns

- 0 error
- Others Current LEDC frequency

esp_err_t **ledc_set_duty_with_hpoint**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t duty, uint32_t hpoint)

LEDC set duty and hpoint value Only after calling ledc_update_duty will the duty update.

Note: ledc_set_duty, ledc_set_duty_with_hpoint and ledc_update_duty are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_duty_and_update

---

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]
- **hpoint** -- Set the LEDC hpoint value, the range is [0, (2**duty_resolution)-1]

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

int **ledc_get_hpoint**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel)

LEDC get hpoint value, the counter value when the output is set high level.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t

Returns

- LEDC_ERR_VAL if parameter error
- Others Current hpoint value of LEDC channel

esp_err_t **ledc_set_duty**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t duty)

LEDC set duty This function do not change the hpoint value of this channel. if needed, please call ledc_set_duty_with_hpoint. only after calling ledc_update_duty will the duty update.

Note: ledc_set_duty, ledc_set_duty_with_hpoint and ledc_update_duty are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_duty_and_update.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

uint32_t **ledc_get_duty**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel)

LEDC get duty This function returns the duty at the present PWM cycle. You shouldn't expect the function to return the new duty in the same cycle of calling ledc_update_duty, because duty update doesn't take effect until the next cycle.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t

Returns

- LEDC_ERR_DUTY if parameter error
- Others Current LEDC duty

esp_err_t **ledc_set_fade**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t duty, *ledc_duty_direction_t* fade_direction, uint32_t step_num, uint32_t duty_cycle_num, uint32_t duty_scale)

LEDC set gradient Set LEDC gradient, After the function calls the ledc_update_duty function, the function can take effect.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **duty** -- Set the start of the gradient duty, the range of duty setting is [0, (2**duty_resolution)]
- **fade_direction** -- Set the direction of the gradient
- **step_num** -- Set the number of the gradient
- **duty_cycle_num** -- Set how many LEDC tick each time the gradient lasts
- **duty_scale** -- Set gradient change amplitude

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **ledc_isr_register** (void (*fn)(void*), void *arg, int intr_alloc_flags, *ledc_isr_handle_t* *handle)

Register LEDC interrupt handler, the handler is an ISR. The handler will be attached to the same CPU core that this function is running on.

Parameters

- **fn** -- Interrupt handler function.
- **arg** -- User-supplied argument passed to the handler function.
- **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.
- **handle** -- Pointer to return handle. If non-NULL, a handle for the interrupt will be returned here.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_NOT_FOUND Failed to find available interrupt source

esp_err_t **ledc_timer_set** (*ledc_mode_t* speed_mode, *ledc_timer_t* timer_sel, uint32_t clock_divider, uint32_t duty_resolution, *ledc_clk_src_t* clk_src)

Configure LEDC settings.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_sel** -- Timer index (0-3), there are 4 timers in LEDC module
- **clock_divider** -- Timer clock divide value, the timer clock is divided from the selected clock source
- **duty_resolution** -- Resolution of duty setting in number of bits. The range is [1, SOC_LEDC_TIMER_BIT_WIDTH]
- **clk_src** -- Select LEDC source clock.

Returns

- (-1) Parameter error
- Other Current LEDC duty

esp_err_t **ledc_timer_rst** (*ledc_mode_t* speed_mode, *ledc_timer_t* timer_sel)

Reset LEDC timer.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_sel** -- LEDC timer index (0-3), select from ledc_timer_t

Returns

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

esp_err_t **ledc_timer_pause** (*ledc_mode_t* speed_mode, *ledc_timer_t* timer_sel)

Pause LEDC timer counter.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **timer_sel** -- LEDC timer index (0-3), select from ledc_timer_t

Returns

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

esp_err_t **ledc_timer_resume** (*ledc_mode_t* speed_mode, *ledc_timer_t* timer_sel)

Resume LEDC timer.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.

- **timer_sel** -- LEDC timer index (0-3), select from ledc_timer_t

Returns

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

esp_err_t **ledc_bind_channel_timer**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, *ledc_timer_t* timer_sel)

Bind LEDC channel with the selected timer.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **timer_sel** -- LEDC timer index (0-3), select from ledc_timer_t

Returns

- ESP_ERR_INVALID_ARG Parameter error
- ESP_OK Success

esp_err_t **ledc_set_fade_with_step**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t target_duty, uint32_t scale, uint32_t cycle_num)

Set LEDC fade function.

Note: Call ledc_fade_func_install() once before calling this function. Call ledc_fade_start() after this to start fading.

Note: ledc_set_fade_with_step, ledc_set_fade_with_time and ledc_fade_start are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_fade_step_and_start

---

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
- **scale** -- Controls the increase or decrease step scale.
- **cycle_num** -- increase or decrease the duty every cycle_num cycles

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

esp_err_t **ledc_set_fade_with_time**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t target_duty, int max_fade_time_ms)

Set LEDC fade function, with a limited time.

Note: Call ledc_fade_func_install() once before calling this function. Call ledc_fade_start() after this to start fading.

Note: ledc_set_fade_with_step, ledc_set_fade_with_time and ledc_fade_start are not thread-safe, do not call these functions to control one LEDC channel in different tasks at the same time. A thread-safe version of API is ledc_set_fade_step_and_start

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **`speed_mode`** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **`channel`** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **`target_duty`** -- Target duty of fading [0, (2**duty_resolution)]
- **`max_fade_time_ms`** -- The maximum time of the fading (ms).

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

esp_err_t **`ledc_fade_func_install`**(int intr_alloc_flags)

Install LEDC fade function. This function will occupy interrupt of LEDC module.

Parameters **`intr_alloc_flags`** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Intr flag error
- ESP_ERR_NOT_FOUND Failed to find available interrupt source
- ESP_ERR_INVALID_STATE Fade function already installed

void **`ledc_fade_func_uninstall`**(void)

Uninstall LEDC fade function.

esp_err_t **`ledc_fade_start`**(*ledc_mode_t* speed_mode, *ledc_channel_t* channel, *ledc_fade_mode_t* fade_mode)

Start LEDC fading.

Note: Call ledc_fade_func_install() once before calling this function. Call this API right after ledc_set_fade_with_time or ledc_set_fade_with_step before to start fading.

Note: Starting fade operation with this API is not thread-safe, use with care.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **`speed_mode`** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **`channel`** -- LEDC channel number

- **fade_mode** -- Whether to block until fading done. See ledc_types.h ledc_fade_mode_t for more info. Note that this function will not return until fading to the target duty if LEDC_FADE_WAIT_DONE mode is selected.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_STATE Channel not initialized or fade function not installed.
- ESP_ERR_INVALID_ARG Parameter error.

esp_err_t **ledc_fade_stop** (*ledc_mode_t* speed_mode, *ledc_channel_t* channel)

Stop LEDC fading. The duty of the channel is guaranteed to be fixed at most one PWM cycle after the function returns.

Note: This API can be called if a new fixed duty or a new fade want to be set while the last fade operation is still running in progress.

Note: Call this API will abort the fading operation only if it was started by calling ledc_fade_start with LEDC_FADE_NO_WAIT mode.

Note: If a fade was started with LEDC_FADE_WAIT_DONE mode, calling this API afterwards has no use in stopping the fade. Fade will continue until it reaches the target duty.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel number

Returns

- ESP_OK Success
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Fade function init error

esp_err_t **ledc_set_duty_and_update** (*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t duty, uint32_t hpoint)

A thread-safe API to set duty for LEDC channel and return when duty updated.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **duty** -- Set the LEDC duty, the range of duty setting is [0, (2**duty_resolution)]
- **hpoint** -- Set the LEDC hpoint value, the range is [0, (2**duty_resolution)-1]

Returns

- ESP_OK Success
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_ERR_INVALID_ARG Parameter error
- ESP_FAIL Fade function init error

esp_err_t **ledc_set_fade_time_and_start** (*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t target_duty, uint32_t max_fade_time_ms, *ledc_fade_mode_t* fade_mode)

A thread-safe API to set and start LEDC fade function, with a limited time.

Note: Call ledc_fade_func_install() once, before calling this function.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
- **max_fade_time_ms** -- The maximum time of the fading (ms).
- **fade_mode** -- choose blocking or non-blocking mode

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

esp_err_t **ledc_set_fade_step_and_start** (*ledc_mode_t* speed_mode, *ledc_channel_t* channel, uint32_t target_duty, uint32_t scale, uint32_t cycle_num, *ledc_fade_mode_t* fade_mode)

A thread-safe API to set and start LEDC fade function.

Note: Call ledc_fade_func_install() once before calling this function.

Note: For ESP32, hardware does not support any duty change while a fade operation is running in progress on that channel. Other duty operations will have to wait until the fade operation has finished.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **target_duty** -- Target duty of fading [0, (2**duty_resolution)]
- **scale** -- Controls the increase or decrease step scale.
- **cycle_num** -- increase or decrease the duty every cycle_num cycles
- **fade_mode** -- choose blocking or non-blocking mode

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

esp_err_t **ledc_cb_register** (*ledc_mode_t* speed_mode, *ledc_channel_t* channel, *ledc_cbs_t* *cbs, void *user_arg)

LEDC callback registration function.

Note: The callback is called from an ISR, it must never attempt to block, and any FreeRTOS API called must be ISR capable.

Parameters

- **speed_mode** -- Select the LEDC channel group with specified speed mode. Note that not all targets support high speed mode.
- **channel** -- LEDC channel index (0 - LEDC_CHANNEL_MAX-1), select from ledc_channel_t
- **cbs** -- Group of LEDC callback functions
- **user_arg** -- user registered data for the callback function

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Channel not initialized
- ESP_FAIL Fade function init error

Structures

struct **ledc_channel_config_t**

Configuration parameters of LEDC channel for ledc_channel_config function.

Public Members

int **gpio_num**

the LEDC output gpio_num, if you want to use gpio16, gpio_num = 16

ledc_mode_t **speed_mode**

LEDC speed speed_mode, high-speed mode (only exists on esp32) or low-speed mode

ledc_channel_t **channel**

LEDC channel (0 - LEDC_CHANNEL_MAX-1)

ledc_intr_type_t **intr_type**

configure interrupt, Fade interrupt enable or Fade interrupt disable

ledc_timer_t **timer_sel**

Select the timer source of channel (0 - LEDC_TIMER_MAX-1)

uint32_t **duty**

LEDC channel duty, the range of duty setting is [0, (2**duty_resolution)]

int **hpoint**

LEDC channel hpoint value, the range is [0, (2**duty_resolution)-1]

unsigned int **output_invert**

Enable (1) or disable (0) gpio output invert

struct *ledc_channel_config_t*::[anonymous] **flags**

LEDC flags

struct **ledc_timer_config_t**

Configuration parameters of LEDC timer for ledc_timer_config function.

Public Members

ledc_mode_t **speed_mode**

LEDC speed speed_mode, high-speed mode (only exists on esp32) or low-speed mode

ledc_timer_bit_t **duty_resolution**

LEDC channel duty resolution

ledc_timer_t **timer_num**

The timer source of channel (0 - LEDC_TIMER_MAX-1)

uint32_t **freq_hz**

LEDC timer frequency (Hz)

ledc_clk_cfg_t **clk_cfg**

Configure LEDC source clock from ledc_clk_cfg_t. Note that LEDC_USE_RC_FAST_CLK and LEDC_USE_XTAL_CLK are non-timer-specific clock sources. You can not have one LEDC timer uses RC_FAST_CLK as the clock source and have another LEDC timer uses XTAL_CLK as its clock source. All chips except esp32 and esp32s2 do not have timer-specific clock sources, which means clock source for all timers must be the same one.

bool **deconfigure**

Set this field to de-configure a LEDC timer which has been configured before Note that it will not check whether the timer wants to be de-configured is binded to any channel. Also, the timer has to be paused first before it can be de-configured. When this field is set, duty_resolution, freq_hz, clk_cfg fields are ignored.

struct **ledc_cb_param_t**

LEDC callback parameter.

Public Members

ledc_cb_event_t **event**

Event name

uint32_t **speed_mode**

Speed mode of the LEDC channel group

uint32_t **channel**

LEDC channel (0 - LEDC_CHANNEL_MAX-1)

uint32_t **duty**

LEDC current duty of the channel, the range of duty is [0, (2**duty_resolution)]

struct **ledc_cbs_t**

Group of supported LEDC callbacks.

Note: The callbacks are all running under ISR environment

Public Members

ledc_cb_t **fade_cb**

LEDC fade_end callback function

Macros

LEDC_APB_CLK_HZ

Frequency of one of the LEDC peripheral clock sources, APB_CLK.

Note: This macro should have no use in your application, we keep it here only for backward compatible

LEDC_ERR_DUTY

LEDC_ERR_VAL

Type Definitions

typedef *intr_handle_t* **ledc_isr_handle_t**

typedef bool (***ledc_cb_t**)(const *ledc_cb_param_t* *param, void *user_arg)

Type of LEDC event callback.

Param param LEDC callback parameter
Param user_arg User registered data
Return Whether a high priority task has been waken up by this function

Enumerations

enum **ledc_cb_event_t**

LEDC callback event type.

Values:

enumerator **LEDC_FADE_END_EVT**

LEDC fade end event

Header File

- components/hal/include/hal/ledc_types.h
- This header file can be included with:

```
#include "hal/ledc_types.h"
```

Type Definitions

typedef *soc_periph_ledc_clk_src_legacy_t* **ledc_clk_cfg_t**

LEDC clock source configuration struct.

In theory, the following enumeration shall be placed in LEDC driver's header. However, as the next enumeration, `ledc_clk_src_t`, makes the use of some of these values and to avoid mutual inclusion of the headers, we must define it here.

Enumerations

enum **ledc_mode_t**

Values:

enumerator **LEDC_LOW_SPEED_MODE**

LEDC low speed speed_mode

enumerator **LEDC_SPEED_MODE_MAX**

LEDC speed limit

enum **ledc_intr_type_t**

Values:

enumerator **LEDC_INTR_DISABLE**

Disable LEDC interrupt

enumerator **LEDC_INTR_FADE_END**

Enable LEDC interrupt

enumerator **LEDC_INTR_MAX**

enum **ledc_duty_direction_t**

Values:

enumerator **LEDC_DUTY_DIR_DECREASE**

LEDC duty decrease direction

enumerator **LEDC_DUTY_DIR_INCREASE**

LEDC duty increase direction

enumerator **LEDC_DUTY_DIR_MAX**

enum **ledc_slow_clk_sel_t**

LEDC global clock sources.

Values:

enumerator **LEDC_SLOW_CLK_RC_FAST**

LEDC low speed timer clock source is RC_FAST clock

enumerator **LEDC_SLOW_CLK_APB**

LEDC low speed timer clock source is 80MHz APB clock

enumerator **LEDC_SLOW_CLK_XTAL**

LEDC low speed timer clock source XTAL clock

enumerator **LEDC_SLOW_CLK_RTC8M**

Alias of 'LEDC_SLOW_CLK_RC_FAST'

enum **ledc_clk_src_t**

LEDC timer-specific clock sources.

Note: Setting numeric values to match ledc_clk_cfg_t values are a hack to avoid collision with LEDC_AUTO_CLK in the driver, as these enums have very similar names and user may pass one of these by mistake.

Values:

enumerator **LEDC_APB_CLK**

LEDC timer clock divided from APB clock (80Mhz)

enumerator **LEDC_SCLK**

Selecting this value for LEDC_TICK_SEL_TIMER let the hardware take its source clock from LEDC_APB_CLK_SEL

enum **ledc_timer_t**

Values:

enumerator **LEDC_TIMER_0**

LEDC timer 0

enumerator **LEDC_TIMER_1**

LEDC timer 1

enumerator **LEDC_TIMER_2**

LEDC timer 2

enumerator **LEDC_TIMER_3**

LEDC timer 3

enumerator **LEDC_TIMER_MAX**

enum **ledc_channel_t**

Values:

enumerator **LEDC_CHANNEL_0**

LEDC channel 0

enumerator **LEDC_CHANNEL_1**

LEDC channel 1

enumerator **LEDC_CHANNEL_2**

LEDC channel 2

enumerator **LEDC_CHANNEL_3**
LEDC channel 3

enumerator **LEDC_CHANNEL_4**
LEDC channel 4

enumerator **LEDC_CHANNEL_5**
LEDC channel 5

enumerator **LEDC_CHANNEL_6**
LEDC channel 6

enumerator **LEDC_CHANNEL_7**
LEDC channel 7

enumerator **LEDC_CHANNEL_MAX**

enum **ledc_timer_bit_t**
Values:

enumerator **LEDC_TIMER_1_BIT**
LEDC PWM duty resolution of 1 bits

enumerator **LEDC_TIMER_2_BIT**
LEDC PWM duty resolution of 2 bits

enumerator **LEDC_TIMER_3_BIT**
LEDC PWM duty resolution of 3 bits

enumerator **LEDC_TIMER_4_BIT**
LEDC PWM duty resolution of 4 bits

enumerator **LEDC_TIMER_5_BIT**
LEDC PWM duty resolution of 5 bits

enumerator **LEDC_TIMER_6_BIT**
LEDC PWM duty resolution of 6 bits

enumerator **LEDC_TIMER_7_BIT**
LEDC PWM duty resolution of 7 bits

enumerator **LEDC_TIMER_8_BIT**
LEDC PWM duty resolution of 8 bits

enumerator **LEDC_TIMER_9_BIT**
LEDC PWM duty resolution of 9 bits

enumerator **LEDC_TIMER_10_BIT**
LEDC PWM duty resolution of 10 bits

enumerator **LEDC_TIMER_11_BIT**

LEDC PWM duty resolution of 11 bits

enumerator **LEDC_TIMER_12_BIT**

LEDC PWM duty resolution of 12 bits

enumerator **LEDC_TIMER_13_BIT**

LEDC PWM duty resolution of 13 bits

enumerator **LEDC_TIMER_14_BIT**

LEDC PWM duty resolution of 14 bits

enumerator **LEDC_TIMER_BIT_MAX**

enum **ledc_fade_mode_t**

Values:

enumerator **LEDC_FADE_NO_WAIT**

LEDC fade function will return immediately

enumerator **LEDC_FADE_WAIT_DONE**

LEDC fade function will block until fading to the target duty

enumerator **LEDC_FADE_MAX**

2.6.14 Motor Control Pulse Width Modulator (MCPWM)

The MCPWM peripheral is a versatile PWM generator, which contains various submodules to make it a key element in power electronic applications like motor control, digital power, and so on. Typically, the MCPWM peripheral can be used in the following scenarios:

- Digital motor control, e.g., brushed/brushless DC motor, RC servo motor
- Switch mode-based digital power conversion
- Power DAC, where the duty cycle is equivalent to a DAC analog value
- Calculate external pulse width, and convert it into other analog values like speed, distance
- Generate Space Vector PWM (SVPWM) signals for Field Oriented Control (FOC)

The main submodules are listed in the following diagram:

- **MCPWM Timer**: The time base of the final PWM signal. It also determines the event timing of other submodules.
- **MCPWM Operator**: The key module that is responsible for generating the PWM waveforms. It consists of other submodules, like comparator, PWM generator, dead time, and carrier modulator.
- **MCPWM Comparator**: The compare module takes the time-base count value as input, and continuously compares it to the threshold value configured. When the timer is equal to any of the threshold values, a compare event will be generated and the MCPWM generator can update its level accordingly.
- **MCPWM Generator**: One MCPWM generator can generate a pair of PWM waves, complementarily or independently, based on various events triggered by other submodules like MCPWM Timer and MCPWM Comparator.
- **MCPWM Fault**: The fault module is used to detect the fault condition from outside, mainly via the GPIO matrix. Once the fault signal is active, MCPWM Operator will force all the generators into a predefined state to protect the system from damage.

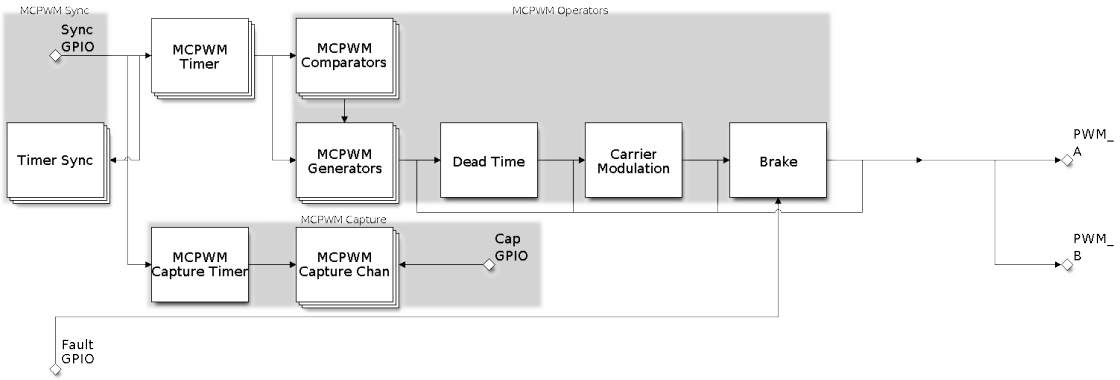


Fig. 16: MCPWM Overview

- **MCPWM Sync**: The sync module is used to synchronize the MCPWM timers, so that the final PWM signals generated by different MCPWM generators can have a fixed phase difference. The sync signal can be routed from the GPIO matrix or from an MCPWM Timer event.
- **Dead Time**: This submodule is used to insert extra delay to the existing PWM edges generated in the previous steps.
- **Carrier Modulation**: The carrier submodule can modulate a high-frequency carrier signal into PWM waveforms by the generator and dead time submodules. This capability is mandatory for controlling the power-switching elements.
- **Brake**: MCPWM operator can set how to brake the generators when a particular fault is detected. You can shut down the PWM output immediately or regulate the PWM output cycle by cycle, depending on how critical the fault is.
- **MCPWM Capture**: This is a standalone submodule that can work even without the above MCPWM operators. The capture consists one dedicated timer and several independent channels, with each channel connected to the GPIO. A pulse on the GPIO triggers the capture timer to store the time-base count value and then notify you by an interrupt. Using this feature, you can measure a pulse width precisely. What is more, the capture timer can also be synchronized by the MCPWM Sync submodule.

Functional Overview

Description of the MCPWM functionality is divided into the following sections:

- *Resource Allocation and Initialization* - covers how to allocate various MCPWM objects, like timers, operators, comparators, generators and so on. These objects are the basis of the following IO setting and control functions.
- *Timer Operations and Events* - describes control functions and event callbacks supported by the MCPWM timer.
- *Comparator Operations and Events* - describes control functions and event callbacks supported by the MCPWM comparator.
- *Generator Actions on Events* - describes how to set actions for MCPWM generators on particular events that are generated by the MCPWM timer and comparators.
- *Generator Configurations for Classical PWM Waveforms* - demonstrates some classical PWM waveforms that can be achieved by configuring generator actions.
- *Dead Time* - describes how to set dead time for MCPWM generators.
- *Dead Time Configurations for Classical PWM Waveforms* - demonstrates some classical PWM waveforms that can be achieved by configuring dead time.
- *Carrier Modulation* - describes how to set and modulate a high frequency onto the final PWM waveforms.
- *Faults and Brake Actions* - describes how to set brake actions for MCPWM operators on particular fault events.
- *Generator Force Actions* - describes how to control the generator output level asynchronously in a forceful way.
- *Synchronization* - describes how to synchronize the MCPWM timers and get a fixed phase difference between the generated PWM signals.

- *Capture* - describes how to use the MCPWM capture module to measure the pulse width of a signal.
- *Power Management* - describes how different source clocks affects power consumption.
- *IRAM Safe* - describes tips on how to make the RMT interrupt work better along with a disabled cache.
- *Thread Safety* - lists which APIs are guaranteed to be thread-safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can bring different effects to the driver.

Resource Allocation and Initialization As displayed in the diagram above, the MCPWM peripheral consists of several submodules. Each submodule has its own resource allocation, which is described in the following sections.

MCPWM Timers You can allocate a MCPWM timer object by calling `mcpwm_new_timer()` function, with a configuration structure `mcpwm_timer_config_t` as the parameter. The configuration structure is defined as:

- `mcpwm_timer_config_t::group_id` specifies the MCPWM group ID. The ID should belong to [0, `SOC_MCPWM_GROUPS` - 1] range. Please note, timers located in different groups are totally independent.
- `mcpwm_timer_config_t::intr_priority` sets the priority of the interrupt. If it is set to `0`, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- `mcpwm_timer_config_t::clk_src` sets the clock source of the timer.
- `mcpwm_timer_config_t::resolution_hz` sets the expected resolution of the timer. The driver internally sets a proper divider based on the clock source and the resolution.
- `mcpwm_timer_config_t::count_mode` sets the count mode of the timer.
- `mcpwm_timer_config_t::period_ticks` sets the period of the timer, in ticks (the tick resolution is set in the `mcpwm_timer_config_t::resolution_hz`).
- `mcpwm_timer_config_t::update_period_on_empty` sets whether to update the period value when the timer counts to zero.
- `mcpwm_timer_config_t::update_period_on_sync` sets whether to update the period value when the timer takes a sync signal.

The `mcpwm_new_timer()` will return a pointer to the allocated timer object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free timers in the MCPWM group, this function will return the `ESP_ERR_NOT_FOUND` error.[1]

On the contrary, calling the `mcpwm_del_timer()` function will free the allocated timer object.

MCPWM Operators You can allocate a MCPWM operator object by calling `mcpwm_new_operator()()` function, with a configuration structure `mcpwm_operator_config_t` as the parameter. The configuration structure is defined as:

- `mcpwm_operator_config_t::group_id` specifies the MCPWM group ID. The ID should belong to [0, `SOC_MCPWM_GROUPS` - 1] range. Please note, operators located in different groups are totally independent.
- `mcpwm_operator_config_t::intr_priority` sets the priority of the interrupt. If it is set to `0`, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- `mcpwm_operator_config_t::update_gen_action_on_tez` sets whether to update the generator action when the timer counts to zero. Here and below, the timer refers to the one that is connected to the operator by `mcpwm_operator_connect_timer()`.
- `mcpwm_operator_config_t::update_gen_action_on_tep` sets whether to update the generator action when the timer counts to peak.
- `mcpwm_operator_config_t::update_gen_action_on_sync` sets whether to update the generator action when the timer takes a sync signal.
- `mcpwm_operator_config_t::update_dead_time_on_tez` sets whether to update the dead time when the timer counts to zero.
- `mcpwm_operator_config_t::update_dead_time_on_tep` sets whether to update the dead time when the timer counts to the peak.
- `mcpwm_operator_config_t::update_dead_time_on_sync` sets whether to update the dead time when the timer takes a sync signal.

[1] Different ESP chip series might have a different number of MCPWM resources (e.g., groups, timers, comparators, operators, generators, triggers and so on). Please refer to the [TRM] for details. The driver does not forbid you from applying for more MCPWM resources, but it returns an error when there are no hardware resources available. Please always check the return value when doing *Resource Allocation and Initialization*.

The *mcpwm_new_operator()* () will return a pointer to the allocated operator object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free operators in the MCPWM group, this function will return the *ESP_ERR_NOT_FOUND* error.[1]

On the contrary, calling *mcpwm_del_operator()* () function will free the allocated operator object.

MCPWM Comparators You can allocate a MCPWM comparator object by calling the *mcpwm_new_comparator()* function, with a MCPWM operator handle and configuration structure *mcpwm_comparator_config_t* as the parameter. The operator handle is created by *mcpwm_new_operator()* (). The configuration structure is defined as:

- *mcpwm_comparator_config_t::intr_priority* sets the priority of the interrupt. If it is set to 0, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- *mcpwm_comparator_config_t::update_cmp_on_tez* sets whether to update the compare threshold when the timer counts to zero.
- *mcpwm_comparator_config_t::update_cmp_on_tep* sets whether to update the compare threshold when the timer counts to the peak.
- *mcpwm_comparator_config_t::update_cmp_on_sync* sets whether to update the compare threshold when the timer takes a sync signal.

The *mcpwm_new_comparator()* will return a pointer to the allocated comparator object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free comparators in the MCPWM operator, this function will return the *ESP_ERR_NOT_FOUND* error.Page 1131, 1

On the contrary, calling the *mcpwm_del_comparator()* function will free the allocated comparator object.

MCPWM Generators You can allocate a MCPWM generator object by calling the *mcpwm_new_generator()* function, with a MCPWM operator handle and configuration structure *mcpwm_generator_config_t* as the parameter. The operator handle is created by *mcpwm_new_operator()* (). The configuration structure is defined as:

- *mcpwm_generator_config_t::gen_gpio_num* sets the GPIO number used by the generator.
- *mcpwm_generator_config_t::invert_pwm* sets whether to invert the PWM signal.
- *mcpwm_generator_config_t::io_loop_back* sets whether to enable the Loop-back mode. It is for debugging purposes only. It enables both the GPIO's input and output ability through the GPIO matrix peripheral.
- *mcpwm_generator_config_t::io_od_mode* configures the PWM GPIO as open-drain output.
- *mcpwm_generator_config_t::pull_up* and *mcpwm_generator_config_t::pull_down* controls whether to enable the internal pull-up and pull-down resistors accordingly.

The *mcpwm_new_generator()* will return a pointer to the allocated generator object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free generators in the MCPWM operator, this function will return the *ESP_ERR_NOT_FOUND* error.Page 1131, 1

On the contrary, calling the *mcpwm_del_generator()* function will free the allocated generator object.

MCPWM Faults There are two types of faults: A fault signal reflected from the GPIO and a fault generated by software.

To allocate a GPIO fault object, you can call the *mcpwm_new_gpio_fault()* function, with the configuration structure *mcpwm_gpio_fault_config_t* as the parameter. The configuration structure is defined as:

- *mcpwm_gpio_fault_config_t::group_id* sets the MCPWM group ID. The ID should belong to [0, *SOC_MCPWM_GROUPS* - 1] range. Please note, GPIO faults located in different groups are totally independent, i.e., GPIO faults in group 0 can not be detected by the operator in group 1.
- *mcpwm_gpio_fault_config_t::intr_priority* sets the priority of the interrupt. If it is set to 0, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- *mcpwm_gpio_fault_config_t::gpio_num* sets the GPIO number used by the fault.
- *mcpwm_gpio_fault_config_t::active_level* sets the active level of the fault signal.
- *mcpwm_gpio_fault_config_t::pull_up* and *mcpwm_gpio_fault_config_t::pull_down* set whether to pull up and/or pull down the GPIO internally.

- *mcpwm_gpio_fault_config_t::io_loop_back* sets whether to enable the loopback mode. It is for debugging purposes only. It enables both the GPIO's input and output ability through the GPIO matrix peripheral.

The *mcpwm_new_gpio_fault()* will return a pointer to the allocated fault object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free GPIO faults in the MCPWM group, this function will return the *ESP_ERR_NOT_FOUND* error.[Page 1131, 1]

Software fault object can be used to trigger a fault by calling the function *mcpwm_soft_fault_activate()* instead of waiting for a real fault signal on the GPIO. A software fault object can be allocated by calling the *mcpwm_new_soft_fault()* function, with configuration structure *mcpwm_soft_fault_config_t* as the parameter. Currently, this configuration structure is left for future purposes.

The *mcpwm_new_soft_fault()* function will return a pointer to the allocated fault object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there is no memory left for the fault object, this function will return the *ESP_ERR_NO_MEM* error. Although the software fault and GPIO fault are of different types, the returned fault handle is of the same type.

On the contrary, calling the *mcpwm_del_fault()* function will free the allocated fault object, this function works for both software and GPIO fault.

MCPWM Sync Sources The sync source is what can be used to synchronize the MCPWM timer and MCPWM capture timer. There are three types of sync sources: a sync source reflected from the GPIO, a sync source generated by software, and a sync source generated by an MCPWM timer event.

To allocate a GPIO sync source, you can call the *mcpwm_new_gpio_sync_src()* function, with configuration structure *mcpwm_gpio_sync_src_config_t* as the parameter. The configuration structure is defined as:

- *mcpwm_gpio_sync_src_config_t::group_id* sets the MCPWM group ID. The ID should belong to [0, *SOC_MCPWM_GROUPS* - 1] range. Please note, the GPIO sync sources located in different groups are totally independent, i.e., GPIO sync source in group 0 can not be detected by the timers in group 1.
- *mcpwm_gpio_sync_src_config_t::gpio_num* sets the GPIO number used by the sync source.
- *mcpwm_gpio_sync_src_config_t::active_neg* sets whether the sync signal is active on falling edges.
- *mcpwm_gpio_sync_src_config_t::pull_up* and *mcpwm_gpio_sync_src_config_t::pull_down* set whether to pull up and/or pull down the GPIO internally.
- *mcpwm_gpio_sync_src_config_t::io_loop_back* sets whether to enable the Loop-back mode. It is for debugging purposes only. It enables both the GPIO's input and output ability through the GPIO matrix peripheral.

The *mcpwm_new_gpio_sync_src()* will return a pointer to the allocated sync source object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there are no more free GPIO sync sources in the MCPWM group, this function will return the *ESP_ERR_NOT_FOUND* error.[Page 1131, 1]

To allocate a timer event sync source, you can call the *mcpwm_new_timer_sync_src()* function, with configuration structure *mcpwm_timer_sync_src_config_t* as the parameter. The configuration structure is defined as:

- *mcpwm_timer_sync_src_config_t::timer_event* specifies on what timer event to generate the sync signal.
- *mcpwm_timer_sync_src_config_t::propagate_input_sync* sets whether to propagate the input sync signal (i.e., the input sync signal will be routed to its sync output).

The *mcpwm_new_timer_sync_src()* will return a pointer to the allocated sync source object if the allocation succeeds. Otherwise, it will return an error code. Specifically, if a sync source has been allocated from the same timer before, this function will return the *ESP_ERR_INVALID_STATE* error.

Last but not least, to allocate a software sync source, you can call the *mcpwm_new_soft_sync_src()* function, with configuration structure *mcpwm_soft_sync_config_t* as the parameter. Currently, this configuration structure is left for future purposes.

mcpwm_new_soft_sync_src() will return a pointer to the allocated sync source object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there is no memory left for the sync source object,

this function will return the *ESP_ERR_NO_MEM* error. Please note, to make a software sync source take effect, do not forget to call *mcpwm_soft_sync_activate()*.

On the contrary, calling the *mcpwm_del_sync_src()* function will free the allocated sync source object. This function works for all types of sync sources.

MCPWM Capture Timer and Channels The MCPWM group has a dedicated timer which is used to capture the timestamp when a specific event occurred. The capture timer is connected to several independent channels, each channel is assigned a GPIO.

To allocate a capture timer, you can call the *mcpwm_new_capture_timer()* function, with configuration structure *mcpwm_capture_timer_config_t* as the parameter. The configuration structure is defined as:

- *mcpwm_capture_timer_config_t::group_id* sets the MCPWM group ID. The ID should belong to [0, *SOC_MCPWM_GROUPS* - 1] range.
- *mcpwm_capture_timer_config_t::clk_src* sets the clock source of the capture timer.
- *mcpwm_capture_timer_config_t::resolution_hz* The driver internally will set a proper divider based on the clock source and the resolution. If it is set to `0`, the driver will pick an appropriate resolution on its own, and you can subsequently view the current timer resolution via *mcpwm_capture_timer_get_resolution()*.

Note: In ESP32-S3, *mcpwm_capture_timer_config_t::resolution_hz* parameter is invalid, the capture timer resolution is always equal to the *MCPWM_CAPTURE_CLK_SRC_APB*.

The *mcpwm_new_capture_timer()* will return a pointer to the allocated capture timer object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there is no free capture timer left in the MCPWM group, this function will return the *ESP_ERR_NOT_FOUND* error.[Page 1131, 1]

Next, to allocate a capture channel, you can call the *mcpwm_new_capture_channel()* function, with a capture timer handle and configuration structure *mcpwm_capture_channel_config_t* as the parameter. The configuration structure is defined as:

- *mcpwm_capture_channel_config_t::intr_priority* sets the priority of the interrupt. If it is set to `0`, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.
- *mcpwm_capture_channel_config_t::gpio_num* sets the GPIO number used by the capture channel.
- *mcpwm_capture_channel_config_t::prescale* sets the prescaler of the input signal.
- *mcpwm_capture_channel_config_t::extra_flags::pos_edge* and *mcpwm_capture_channel_config_t::extra_flags::neg_edge* set whether to capture on the positive and/or falling edge of the input signal.
- *mcpwm_capture_channel_config_t::extra_flags::pull_up* and *mcpwm_capture_channel_config_t::extra_flags::pull_down* set whether to pull up and/or pull down the GPIO internally.
- *mcpwm_capture_channel_config_t::extra_flags::invert_cap_signal* sets whether to invert the capture signal.
- *mcpwm_capture_channel_config_t::extra_flags::io_loop_back* sets whether to enable the Loop-back mode. It is for debugging purposes only. It enables both the GPIO's input and output ability through the GPIO matrix peripheral.
- *mcpwm_capture_channel_config_t::extra_flags::keep_io_conf_at_exit* sets whether to keep the GPIO configuration when the capture channel is deleted.

The *mcpwm_new_capture_channel()* will return a pointer to the allocated capture channel object if the allocation succeeds. Otherwise, it will return an error code. Specifically, when there is no free capture channel left in the capture timer, this function will return the *ESP_ERR_NOT_FOUND* error.

On the contrary, calling *mcpwm_del_capture_channel()* and *mcpwm_del_capture_timer()* will free the allocated capture channel and timer object accordingly.

MCPWM Interrupt Priority MCPWM allows configuring interrupts separately for timer, operator, comparator, fault, and capture events. The interrupt priority is determined by the respective `config_t::intr_priority`. Additionally, events within the same MCPWM group share a common interrupt source. When registering multiple interrupt events, the interrupt priorities need to remain consistent.

Note: When registering multiple interrupt events within an MCPWM group, the driver will use the interrupt priority of the first registered event as the MCPWM group's interrupt priority.

Timer Operations and Events

Update Period The timer period is initialized by the *mcpwm_timer_config_t::period_ticks* parameter in *mcpwm_timer_config_t*. You can update the period at runtime by calling *mcpwm_timer_set_period()* function. The new period will take effect based on how you set the *mcpwm_timer_config_t::update_period_on_empty* and *mcpwm_timer_config_t::update_period_on_sync* parameters in *mcpwm_timer_config_t*. If none of them are set, the timer period will take effect immediately.

Register Timer Event Callbacks The MCPWM timer can generate different events at runtime. If you have some function that should be called when a particular event happens, you should hook your function to the interrupt service routine by calling *mcpwm_timer_register_event_callbacks()*. The callback function prototype is declared in *mcpwm_timer_event_cb_t*. All supported event callbacks are listed in the *mcpwm_timer_event_callbacks_t*:

- *mcpwm_timer_event_callbacks_t::on_full* sets the callback function for the timer when it counts to peak value.
- *mcpwm_timer_event_callbacks_t::on_empty* sets the callback function for the timer when it counts to zero.
- *mcpwm_timer_event_callbacks_t::on_stop* sets the callback function for the timer when it is stopped.

The callback functions above are called within the ISR context, so they should **not** attempt to block. For example, you may make sure that only FreeRTOS APIs with the `ISR` suffix are called within the function.

The parameter `user_data` of the *mcpwm_timer_register_event_callbacks()* function is used to save your own context. It is passed to each callback function directly.

This function will lazy the install interrupt service for the MCPWM timer without enabling it. It is only allowed to be called before *mcpwm_timer_enable()*, otherwise the *ESP_ERR_INVALID_STATE* error will be returned. See also *Enable and Disable timer* for more information.

Enable and Disable Timer Before doing IO control to the timer, you need to enable the timer first, by calling *mcpwm_timer_enable()*. This function:

- switches the timer state from **init** to **enable**.
- enables the interrupt service if it has been lazy installed by *mcpwm_timer_register_event_callbacks()*.
- acquire a proper power management lock if a specific clock source (e.g., PLL_160M clock) is selected. See also *Power management* for more information.

On the contrary, calling *mcpwm_timer_disable()* will put the timer driver back to the **init** state, disable the interrupt service and release the power management lock.

Start and Stop Timer The basic IO operation of a timer is to start and stop. Calling *mcpwm_timer_start_stop()* with different *mcpwm_timer_start_stop_cmd_t* commands can start the timer immediately or stop the timer at a specific event. What is more, you can even start the timer for only one round, which means, the timer will count to peak value or zero, and then stop itself.

Connect Timer with Operator The allocated MCPWM timer should be connected with an MCPWM operator by calling *mcpwm_operator_connect_timer()*, so that the operator can take that timer as its time base, and generate the required PWM waves. Please make sure the MCPWM timer and operator are in the same group. Otherwise, this function will return the *ESP_ERR_INVALID_ARG* error.

Comparator Operations and Events

Register Comparator Event Callbacks The MCPWM comparator can inform you when the timer counter equals the compare value. If you have some function that should be called when this event happens, you should hook your function to the interrupt service routine by calling *mcpwm_comparator_register_event_callbacks()*. The callback function prototype is declared in *mcpwm_compare_event_cb_t*. All supported event callbacks are listed in the *mcpwm_comparator_event_callbacks_t*:

- *mcpwm_comparator_event_callbacks_t::on_reach* sets the callback function for the comparator when the timer counter equals the compare value.

The callback function provides event-specific data of type *mcpwm_compare_event_data_t* to you. The callback function is called within the ISR context, so it should **not** attempt to block. For example, you may make sure that only FreeRTOS APIs with the `ISR` suffix are called within the function.

The parameter `user_data` of *mcpwm_comparator_register_event_callbacks()* function is used to save your own context. It is passed to the callback function directly.

This function will lazy the installation of interrupt service for the MCPWM comparator, whereas the service can only be removed in *mcpwm_del_comparator*.

Set Compare Value You can set the compare value for the MCPWM comparator at runtime by calling *mcpwm_comparator_set_compare_value()*. There are a few points to note:

- A new compare value might not take effect immediately. The update time for the compare value is set by *mcpwm_comparator_config_t::update_cmp_on_tez* or *mcpwm_comparator_config_t::update_cmp_on_tep* or *mcpwm_comparator_config_t::update_cmp*
- Make sure the operator has connected to one MCPWM timer already by *mcpwm_operator_connect_timer()*. Otherwise, it will return the error code *ESP_ERR_INVALID_STATE*.
- The compare value should not exceed the timer's count peak, otherwise, the compare event will never get triggered.

Generator Actions on Events

Set Generator Action on Timer Event One generator can set multiple actions on different timer events, by calling *mcpwm_generator_set_actions_on_timer_event()* with a variable number of action configurations. The action configuration is defined in *mcpwm_gen_timer_event_action_t*:

- *mcpwm_gen_timer_event_action_t::direction* specifies the timer direction. The supported directions are listed in *mcpwm_timer_direction_t*.
- *mcpwm_gen_timer_event_action_t::event* specifies the timer event. The supported timer events are listed in *mcpwm_timer_event_t*.
- *mcpwm_gen_timer_event_action_t::action* specifies the generator action to be taken. The supported actions are listed in *mcpwm_generator_action_t*.

There is a helper macro *MCPWM_GEN_TIMER_EVENT_ACTION* to simplify the construction of a timer event action entry.

Please note, the argument list of *mcpwm_generator_set_actions_on_timer_event()* **must** be terminated by *MCPWM_GEN_TIMER_EVENT_ACTION_END*.

You can also set the timer action one by one by calling *mcpwm_generator_set_action_on_timer_event()* without varargs.

Set Generator Action on Compare Event One generator can set multiple actions on different compare events, by calling *mcpwm_generator_set_actions_on_compare_event()* with a variable number of action configurations. The action configuration is defined in *mcpwm_gen_compare_event_action_t*:

- *mcpwm_gen_compare_event_action_t::direction* specifies the timer direction. The supported directions are listed in *mcpwm_timer_direction_t*.
- *mcpwm_gen_compare_event_action_t::comparator* specifies the comparator handle. See *MCPWM Comparators* for how to allocate a comparator.
- *mcpwm_gen_compare_event_action_t::action* specifies the generator action to be taken. The supported actions are listed in *mcpwm_generator_action_t*.

There is a helper macro *MCPWM_GEN_COMPARE_EVENT_ACTION* to simplify the construction of a compare event action entry.

Please note, the argument list of *mcpwm_generator_set_actions_on_compare_event()* **must** be terminated by *MCPWM_GEN_COMPARE_EVENT_ACTION_END*.

You can also set the compare action one by one by calling *mcpwm_generator_set_action_on_compare_event()* without varargs.

Set Generator Action on Fault Event One generator can set action on fault based trigger events, by calling *mcpwm_generator_set_action_on_fault_event()* with an action configurations. The action configuration is defined in *mcpwm_gen_fault_event_action_t*:

- *mcpwm_gen_fault_event_action_t::direction* specifies the timer direction. The supported directions are listed in *mcpwm_timer_direction_t*.
- *mcpwm_gen_fault_event_action_t::fault* specifies the fault used for the trigger. See *MCPWM Faults* for how to allocate a fault.
- *mcpwm_gen_fault_event_action_t::action* specifies the generator action to be taken. The supported actions are listed in *mcpwm_generator_action_t*.

When no free trigger slot is left in the operator to which the generator belongs, this function will return the *ESP_ERR_NOT_FOUND* error.[Page 1131, 1]

The trigger only support GPIO fault. when the input is not a GPIO fault, this function will return the *ESP_ERR_NOT_SUPPORTED* error.

There is a helper macro *MCPWM_GEN_FAULT_EVENT_ACTION* to simplify the construction of a trigger event action entry.

Please note, fault event does not have variadic function like `mcpwm_generator_set_actions_on_fault_event()`.

Set Generator Action on Sync Event One generator can set action on sync based trigger events, by calling *mcpwm_generator_set_action_on_sync_event()* with an action configurations. The action configuration is defined in *mcpwm_gen_sync_event_action_t*:

- *mcpwm_gen_sync_event_action_t::direction* specifies the timer direction. The supported directions are listed in *mcpwm_timer_direction_t*.
- *mcpwm_gen_sync_event_action_t::sync* specifies the sync source used for the trigger. See *MCPWM Sync Sources* for how to allocate a sync source.
- *mcpwm_gen_sync_event_action_t::action* specifies the generator action to be taken. The supported actions are listed in *mcpwm_generator_action_t*.

When no free trigger slot is left in the operator to which the generator belongs, this function will return the *ESP_ERR_NOT_FOUND* error.[Page 1131, 1]

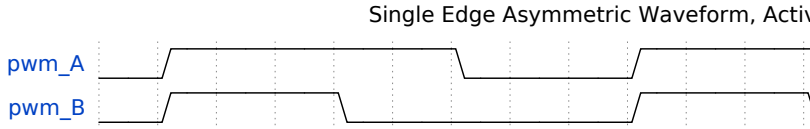
The trigger only support one sync action, regardless of the kinds. When set sync actions more than once, this function will return the *ESP_ERR_INVALID_STATE* error.

There is a helper macro *MCPWM_GEN_SYNC_EVENT_ACTION* to simplify the construction of a trigger event action entry.

Please note, sync event does not have variadic function like `mcpwm_generator_set_actions_on_sync_event()`.

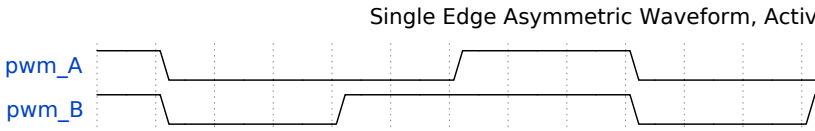
Generator Configurations for Classical PWM Waveforms This section will demonstrate the classical PWM waveforms that can be generated by the pair of generators. The code snippet that is used to generate the waveforms is also provided below the diagram. Some general summary:

- The **Symmetric** or **Asymmetric** of the waveforms is determined by the count mode of the MCPWM timer.
- The **active level** of the waveform pair is determined by the level of the PWM with a smaller duty cycle.
- The period of the PWM waveform is determined by the timer's period and count mode.
- The duty cycle of the PWM waveform is determined by the generator's various action combinations.



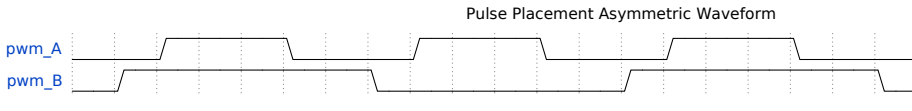
Single Edge Asymmetric Waveform - Active High

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW)));
}
```



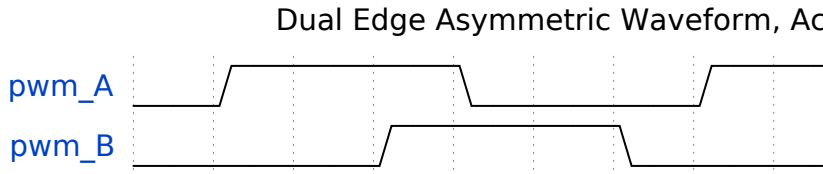
Single Edge Asymmetric Waveform - Active Low

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_FULL, MCPWM_GEN_ACTION_LOW)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_FULL, MCPWM_GEN_ACTION_LOW)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_HIGH)));
}
```



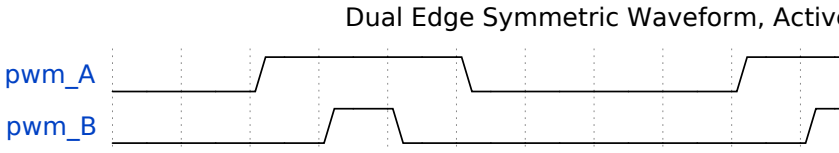
Pulse Placement Asymmetric Waveform

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_TOGGLE),
                    MCPWM_GEN_TIMER_EVENT_ACTION_END()));
}
```



Dual Edge Asymmetric Waveform - Active Low

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN,␣
↪cmpb, MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN, MCPWM_
↪TIMER_EVENT_FULL, MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_TIMER_EVENT_ACTION_END()));
}
```



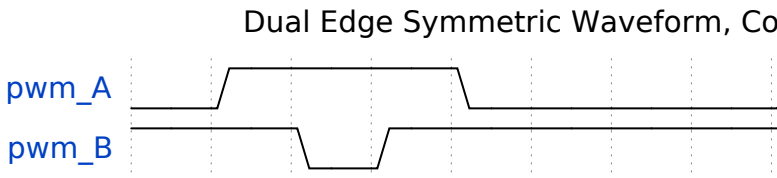
Dual Edge Symmetric Waveform - Active Low

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN,␣
↪cmpa, MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_HIGH),
```

(continues on next page)

(continued from previous page)

```
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN,␣
↪cmpb, MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
}
```



Dual Edge Symmetric Waveform - Complementary

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN,␣
↪cmpa, MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
    ESP_ERROR_CHECK(mcpwm_generator_set_actions_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW),
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_DOWN,␣
↪cmpb, MCPWM_GEN_ACTION_HIGH),
                    MCPWM_GEN_COMPARE_EVENT_ACTION_END()));
}
```

Dead Time In power electronics, the rectifier and inverter are commonly used. This requires the use of a rectifier bridge and an inverter bridge. Each bridge arm has two power electronic devices, such as MOSFET, IGBT, etc. The two MOSFETs on the same arm can not conduct at the same time, otherwise there will be a short circuit. The fact is that, although the PWM wave shows it is turning off the switch, the MOSFET still needs a small time window to make that happen. This requires an extra delay to be added to the existing PWM wave generated by setting *Generator Actions on Events*.

The dead time driver works like a **decorator**. This is also reflected in the function parameters of *mcpwm_generator_set_dead_time()*, where it takes the primary generator handle (`in_generator`), and returns a new generator (`out_generator`) after applying the dead time. Please note, if the `out_generator` and `in_generator` are the same, it means you are adding the time delay to the PWM waveform in an "in-place" fashion. In turn, if the `out_generator` and `in_generator` are different, it means you are deriving a new PWM waveform from the existing `in_generator`.

Dead time specific configuration is listed in the *mcpwm_dead_time_config_t* structure:

- *mcpwm_dead_time_config_t::posedge_delay_ticks* and *mcpwm_dead_time_config_t::negedge_de* set the number of ticks to delay the PWM waveform on the rising and falling edge. Specifically, setting both of them to zero means bypassing the dead time module. The resolution of the dead time tick is the same as the timer that is connected with the operator by *mcpwm_operator_connect_timer()*.
- *mcpwm_dead_time_config_t::invert_output* sets whether to invert the signal after applying the dead time, which can be used to control the delay edge polarity.

Warning: Due to the hardware limitation, one delay module (either `posedge delay` or `negedge delay`) can not be applied to multiple MCPWM generators at the same time. e.g., the following configuration is **invalid**:

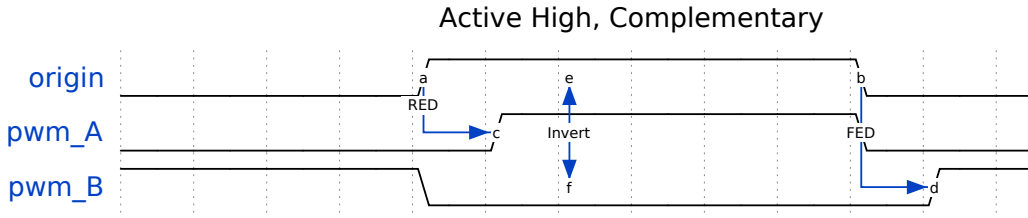
```
mcpwm_dead_time_config_t dt_config = {
    .posedge_delay_ticks = 10,
```

```
};
// Set posedge delay to generator A
mcpwm_generator_set_dead_time(mcpwm_gen_a, mcpwm_gen_a, &dt_config);
// NOTE: This is invalid, you can not apply the posedge delay to another␣
↪generator
mcpwm_generator_set_dead_time(mcpwm_gen_b, mcpwm_gen_b, &dt_config);
```

However, you can apply `posedge delay` to generator A and `negedge delay` to generator B. You can also set both `posedge delay` and `negedge delay` for generator A, while letting generator B bypass the dead time module.

Note: It is also possible to generate the required dead time by setting *Generator Actions on Events*, especially by controlling edge placement using different comparators. However, if the more classical edge delay-based dead time with polarity control is required, then the dead time submodule should be used.

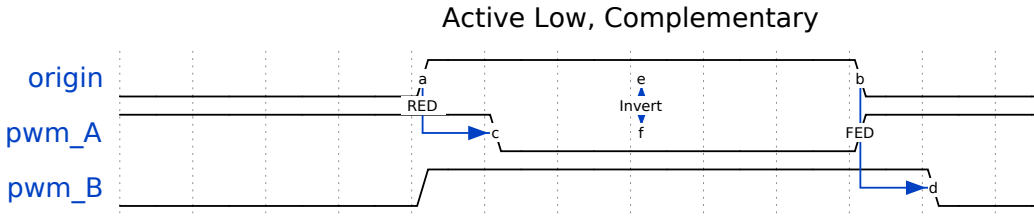
Dead Time Configurations for Classical PWM Waveforms This section demonstrates the classical PWM waveforms that can be generated by the dead time submodule. The code snippet that is used to generate the waveforms is also provided below the diagram.



Active High Complementary

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
}

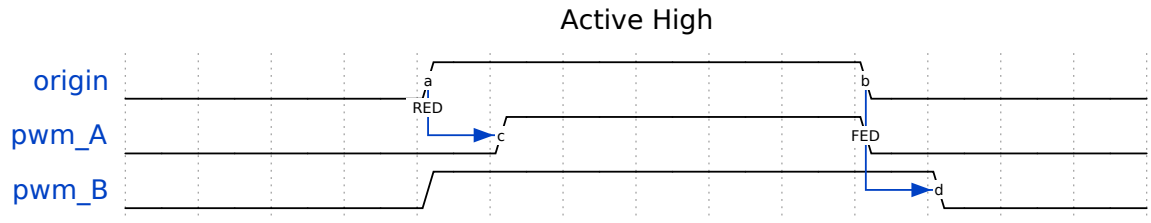
static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 50,
        .negedge_delay_ticks = 0
    };
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    dead_time_config.posedge_delay_ticks = 0;
    dead_time_config.negedge_delay_ticks = 100;
    dead_time_config.flags.invert_output = true;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, genb, &dead_time_config));
}
```



Active Low Complementary

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 50,
        .negedge_delay_ticks = 0,
        .flags.invert_output = true
    };
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    dead_time_config.posedge_delay_ticks = 0;
    dead_time_config.negedge_delay_ticks = 100;
    dead_time_config.flags.invert_output = false;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, genb, &dead_time_config));
}
```



Active High

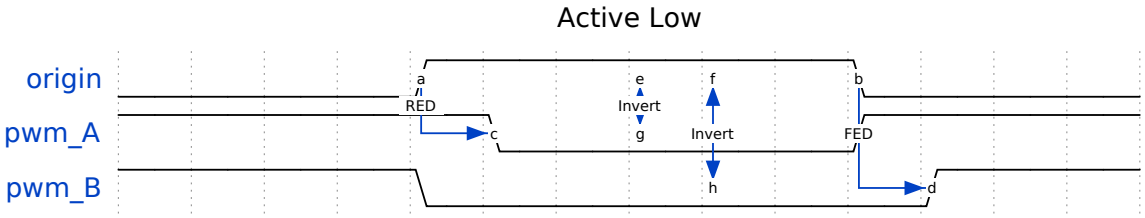
```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 50,
        .negedge_delay_ticks = 0,
    };
```

(continues on next page)

(continued from previous page)

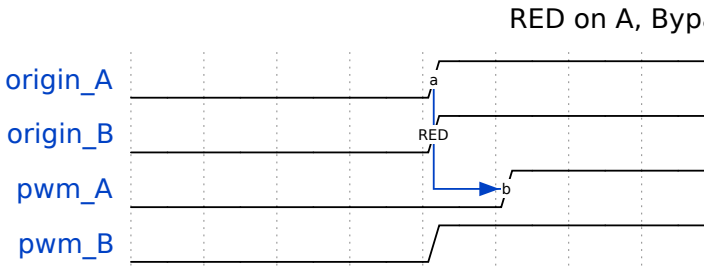
```
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    dead_time_config.posedge_delay_ticks = 0;
    dead_time_config.negedge_delay_ticks = 100;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, genb, &dead_time_config));
}
```



Active Low

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 50,
        .negedge_delay_ticks = 0,
        .flags.invert_output = true
    };
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    dead_time_config.posedge_delay_ticks = 0;
    dead_time_config.negedge_delay_ticks = 100;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, genb, &dead_time_config));
}
```



Rising Delay on PWMA and Bypass Dead Time for PWMB

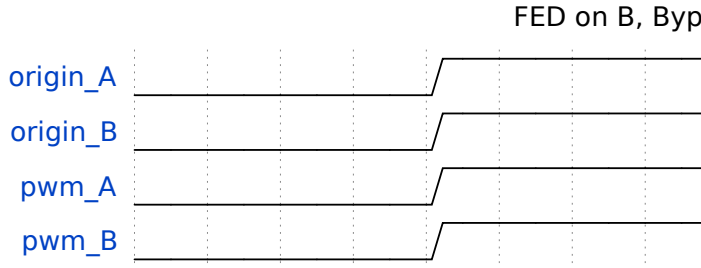
```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
```

(continues on next page)

(continued from previous page)

```
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 50,
        .negedge_delay_ticks = 0,
    };
    // apply deadtime to generator_a
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    // bypass deadtime module for generator_b
    dead_time_config.posedge_delay_ticks = 0;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(genb, genb, &dead_time_config));
}
```



Falling Delay on PWMB and Bypass Dead Time for PWMA

```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 0,
        .negedge_delay_ticks = 0,
    };
    // generator_a bypass the deadtime module (no delay)
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    // apply dead time to generator_b
    dead_time_config.negedge_delay_ticks = 50;
```

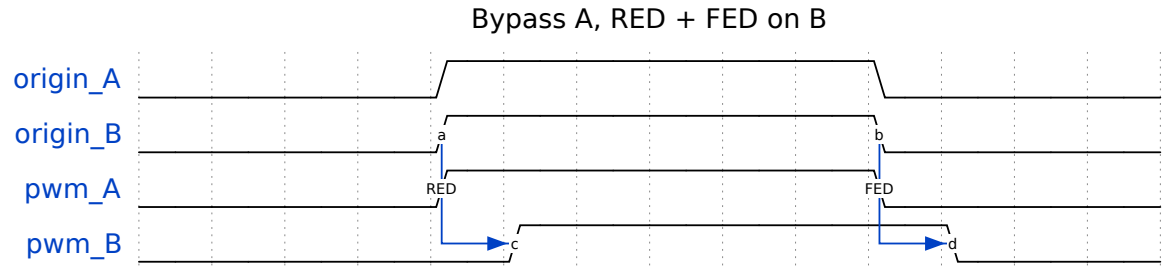
(continues on next page)

(continued from previous page)

```
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(genb, genb, &dead_time_config));

}
```

Rising and Falling Delay on PWMB and Bypass Dead Time for PWMA




```
static void gen_action_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb,␣
↪mcpwm_cmpr_handle_t cmpa, mcpwm_cmpr_handle_t cmpb)
{
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(gena,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(gena,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpa,␣
↪MCPWM_GEN_ACTION_LOW)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_timer_event(genb,
                    MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, MCPWM_
↪TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH)));
    ESP_ERROR_CHECK(mcpwm_generator_set_action_on_compare_event(genb,
                    MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, cmpb,␣
↪MCPWM_GEN_ACTION_LOW)));
}

static void dead_time_config(mcpwm_gen_handle_t gena, mcpwm_gen_handle_t genb)
{
    mcpwm_dead_time_config_t dead_time_config = {
        .posedge_delay_ticks = 0,
        .negedge_delay_ticks = 0,
    };
    // generator_a bypass the deadtime module (no delay)
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(gena, gena, &dead_time_config));
    // apply dead time on both edge for generator_b
    dead_time_config.negedge_delay_ticks = 50;
    dead_time_config.posedge_delay_ticks = 50;
    ESP_ERROR_CHECK(mcpwm_generator_set_dead_time(genb, genb, &dead_time_config));
}
```

Carrier Modulation The MCPWM operator has a carrier submodule that can be used if galvanic isolation from the motor driver is required (e.g., isolated digital power application) by passing the PWM output signals through transformers. Any of the PWM output signals may be at 100% duty and not changing whenever a motor is required to run steadily at the full load. Coupling with non-alternating signals with a transformer is problematic, so the signals are modulated by the carrier submodule to create an AC waveform, to make the coupling possible.

To configure the carrier submodule, you can call *mcpwm_operator_apply_carrier()*, and provide configuration structure *mcpwm_carrier_config_t*:

- *mcpwm_carrier_config_t::clk_src* sets the clock source of the carrier.
- *mcpwm_carrier_config_t::frequency_hz* indicates carrier frequency in Hz.

- *mcpwm_carrier_config_t::duty_cycle* indicates the duty cycle of the carrier. Note that, the supported choices of the duty cycle are discrete, the driver searches for the nearest one based on your configuration.
- *mcpwm_carrier_config_t::first_pulse_duration_us* indicates the duration of the first pulse in microseconds. The resolution of the first pulse duration is determined by the carrier frequency you set in the *mcpwm_carrier_config_t::frequency_hz*. The first pulse duration can not be zero, and it has to be at least one period of the carrier. A longer pulse width can help conduct the inductance quicker.
- *mcpwm_carrier_config_t::invert_before_modulate* and *mcpwm_carrier_config_t::invert_after_modulate* set whether to invert the carrier output before and after modulation.

Specifically, the carrier submodule can be disabled by calling *mcpwm_operator_apply_carrier()* with a `NULL` configuration.

Faults and Brake Actions The MCPWM operator is able to sense external signals with information about the failure of the motor, the power driver or any other device connected. These failure signals are encapsulated into MCPWM fault objects.

You should determine possible failure modes of the motor and what action should be performed on detection of a particular fault, e.g., drive all outputs low for a brushed motor, lock current state for a stepper motor, etc. Because of this action, the motor should be put into a safe state to reduce the likelihood of damage caused by the fault.

Set Operator Brake Mode on Fault The way that MCPWM operator reacts to the fault is called **Brake**. The MCPWM operator can be configured to perform different brake modes for each fault object by calling *mcpwm_operator_set_brake_on_fault()*. Specific brake configuration is passed as a structure *mcpwm_brake_config_t*:

- *mcpwm_brake_config_t::fault* sets which fault the operator should react to.
- *mcpwm_brake_config_t::brake_mode* sets the brake mode that should be used for the fault. The supported brake modes are listed in the *mcpwm_operator_brake_mode_t*. For *MCPWM_OPER_BRAKE_MODE_CBC* mode, the operator recovers itself automatically as long as the fault disappears. You can specify the recovery time in *mcpwm_brake_config_t::cbc_recover_on_tez* and *mcpwm_brake_config_t::cbc_recover_on_tep*. For *MCPWM_OPER_BRAKE_MODE_OST* mode, the operator can not recover even though the fault disappears. You have to call *mcpwm_operator_recover_from_fault()* to manually recover it.

Set Generator Action on Brake Event One generator can set multiple actions on different brake events, by calling *mcpwm_generator_set_actions_on_brake_event()* with a variable number of action configurations. The action configuration is defined in *mcpwm_gen_brake_event_action_t*:

- *mcpwm_gen_brake_event_action_t::direction* specifies the timer direction. The supported directions are listed in *mcpwm_timer_direction_t*.
- *mcpwm_gen_brake_event_action_t::brake_mode* specifies the brake mode. The supported brake modes are listed in the *mcpwm_operator_brake_mode_t*.
- *mcpwm_gen_brake_event_action_t::action* specifies the generator action to be taken. The supported actions are listed in *mcpwm_generator_action_t*.

There is a helper macro *MCPWM_GEN_BRAKE_EVENT_ACTION* to simplify the construction of a brake event action entry.

Please note, the argument list of *mcpwm_generator_set_actions_on_brake_event()* **must** be terminated by *MCPWM_GEN_BRAKE_EVENT_ACTION_END*.

You can also set the brake action one by one by calling *mcpwm_generator_set_action_on_brake_event()* without varargs.

Register Fault Event Callbacks The MCPWM fault detector can inform you when it detects a valid fault or a fault signal disappears. If you have some function that should be called when such an event happens, you should hook your function to the interrupt service routine by calling *mcpwm_fault_register_event_callbacks()*.

The callback function prototype is declared in *mcpwm_fault_event_cb_t*. All supported event callbacks are listed in the *mcpwm_fault_event_callbacks_t*:

- *mcpwm_fault_event_callbacks_t::on_fault_enter* sets the callback function that will be called when a fault is detected.
- *mcpwm_fault_event_callbacks_t::on_fault_exit* sets the callback function that will be called when a fault is cleared.

The callback function is called within the ISR context, so it should **not** attempt to block. For example, you may make sure that only FreeRTOS APIs with the `ISR` suffix are called within the function.

The parameter `user_data` of *mcpwm_fault_register_event_callbacks()* function is used to save your own context. It is passed to the callback function directly.

This function will lazy the install interrupt service for the MCPWM fault, whereas the service can only be removed in *mcpwm_del_fault*.

Register Brake Event Callbacks The MCPWM operator can inform you when it is going to take a brake action. If you have some function that should be called when this event happens, you should hook your function to the interrupt service routine by calling *mcpwm_operator_register_event_callbacks()*. The callback function prototype is declared in *mcpwm_brake_event_cb_t*. All supported event callbacks are listed in the *mcpwm_operator_event_callbacks_t*:

- *mcpwm_operator_event_callbacks_t::on_brake_cbc* sets the callback function that will be called when the operator is going to take a **CBC** action.
- *mcpwm_operator_event_callbacks_t::on_brake_ost* sets the callback function that will be called when the operator is going to take an **OST** action.

The callback function is called within the ISR context, so it should **not** attempt to block. For example, you may make sure that only FreeRTOS APIs with the `ISR` suffix are called within the function.

The parameter `user_data` of the *mcpwm_operator_register_event_callbacks()* function is used to save your own context. It will be passed to the callback function directly.

This function will lazy the install interrupt service for the MCPWM operator, whereas the service can only be removed in *mcpwm_del_operator*.

Generator Force Actions Software can override generator output level at runtime, by calling *mcpwm_generator_set_force_level()*. The software force level always has a higher priority than other event actions set in e.g., *mcpwm_generator_set_actions_on_timer_event()*.

- Set the `level` to -1 means to disable the force action, and the generator's output level will be controlled by the event actions again.
- Set the `hold_on` to true, and the force output level will keep alive until it is removed by assigning `level` to -1.
- Set the `hole_on` to false, the force output level will only be active for a short time, and any upcoming event can override it.

Synchronization When a sync signal is taken by the MCPWM timer, the timer will be forced into a predefined **phase**, where the phase is determined by count value and count direction. You can set the sync phase by calling *mcpwm_timer_set_phase_on_sync()*. The sync phase configuration is defined in *mcpwm_timer_sync_phase_config_t* structure:

- *mcpwm_timer_sync_phase_config_t::sync_src* sets the sync signal source. See *MCPWM Sync Sources* for how to create a sync source object. Specifically, if this is set to `NULL`, the driver will disable the sync feature for the MCPWM timer.
- *mcpwm_timer_sync_phase_config_t::count_value* sets the count value to load when the sync signal is taken.
- *mcpwm_timer_sync_phase_config_t::direction* sets the count direction when the sync signal is taken.

Note: When the MCPWM timer is working in *MCPWM_TIMER_COUNT_MODE_UP_DOWN* mode, special attention needs to be taken. In this mode, counter range `[0 -> peak-1]` belongs to the **increment** phase, and counter range `[peak -> 1]` belongs to the **decrement** phase. Thus if you set the *mcpwm_timer_sync_phase_config_t::count_value* to zero, you may also want to set the *mcpwm_timer_sync_phase_config_t::direction* to *MCPWM_TIMER_DIRECTION_UP*. Otherwise, the timer will be continue with the decrement phase, and the count value underflows to `peak`.

Likewise, the *MCPWM Capture Timer* can be synced as well. You can set the sync phase for the capture timer by calling *mcpwm_capture_timer_set_phase_on_sync()*. The sync phase configuration is defined in *mcpwm_capture_timer_sync_phase_config_t* structure:

- *mcpwm_capture_timer_sync_phase_config_t::sync_src* sets the sync signal source. See *MCPWM Sync Sources* for how to create a sync source object. Specifically, if this is set to `NULL`, the driver will disable the sync feature for the MCPWM capture timer.
- *mcpwm_capture_timer_sync_phase_config_t::count_value* sets the count value to load when the sync signal is taken.
- *mcpwm_capture_timer_sync_phase_config_t::direction* sets the count direction when the sync signal is taken. Note that, different from MCPWM Timer, the capture timer can only support one count direction: *MCPWM_TIMER_DIRECTION_UP*.

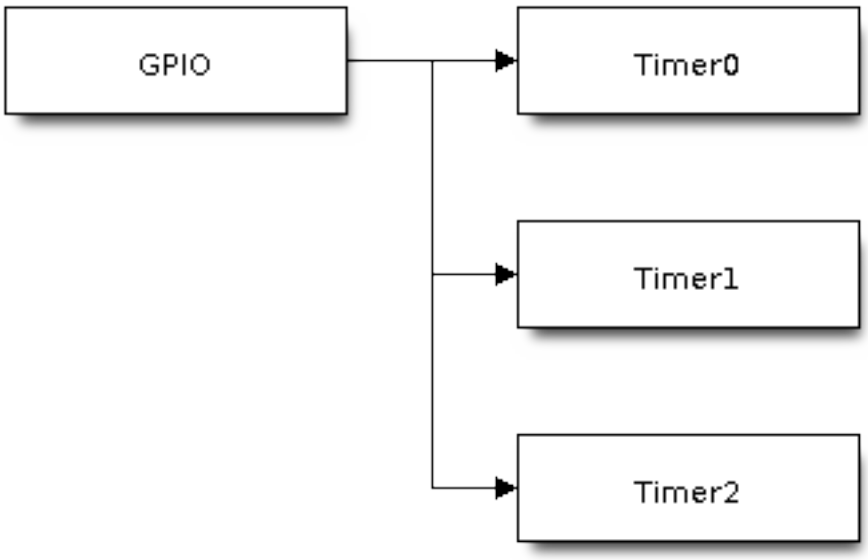


Fig. 17: GPIO Sync All MCPWM Timers

Sync Timers by GPIO

```
static void example_setup_sync_strategy(mcpwm_timer_handle_t timers[])
{
    mcpwm_sync_handle_t gpio_sync_source = NULL;
    mcpwm_gpio_sync_src_config_t gpio_sync_config = {
        .group_id = 0,              // GPIO fault should be in the same group of␣
↪the above timers
        .gpio_num = EXAMPLE_SYNC_GPIO,
        .flags.pull_down = true,
```

(continues on next page)

(continued from previous page)

```
        .flags.active_neg = false,  // By default, a posedge pulse can trigger a␣
↪sync event
    };
    ESP_ERROR_CHECK(mcpwm_new_gpio_sync_src(&gpio_sync_config, &gpio_sync_source));

    mcpwm_timer_sync_phase_config_t sync_phase_config = {
        .count_value = 0,                       // sync phase: target count value
        .direction = MCPWM_TIMER_DIRECTION_UP, // sync phase: count direction
        .sync_src = gpio_sync_source,           // sync source
    };
    for (int i = 0; i < 3; i++) {
        ESP_ERROR_CHECK(mcpwm_timer_set_phase_on_sync(timers[i], &sync_phase_
↪config));
    }
}
```

Capture The basic functionality of MCPWM capture is to record the time when any pulse edge of the capture signal turns active. Then you can get the pulse width and convert it into other physical quantities like distance or speed in the capture callback function. For example, in the BLDC (Brushless DC, see figure below) scenario, you can use the capture submodule to sense the rotor position from the Hall sensor.

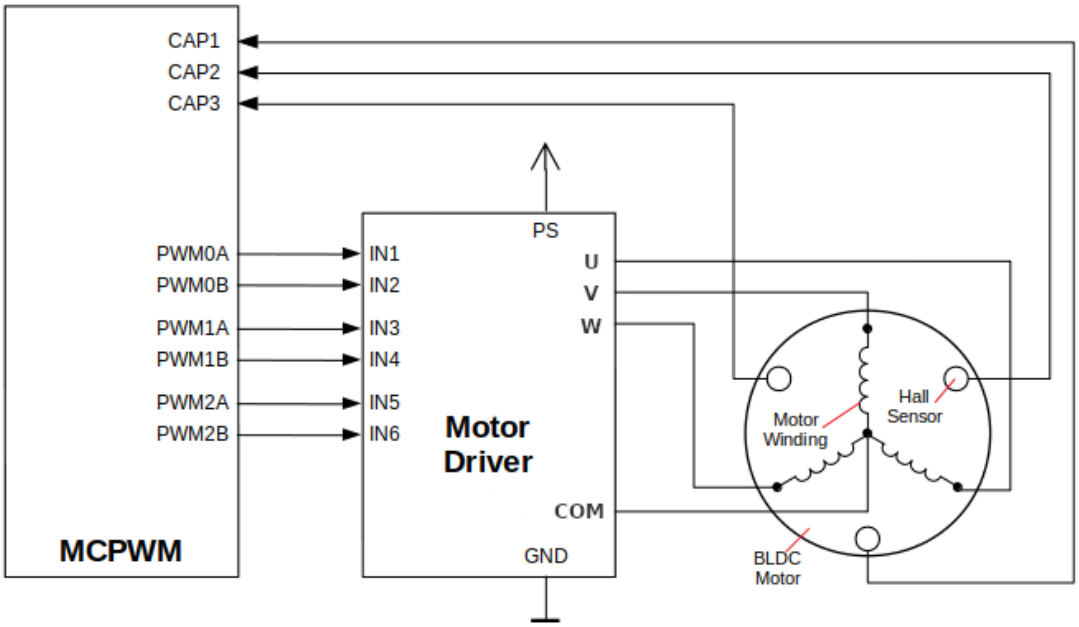


Fig. 18: MCPWM BLDC with Hall Sensor

The capture timer is usually connected to several capture channels. Please refer to *MCPWM Capture Timer and Channels* for more information about resource allocation.

Register Capture Event Callbacks The MCPWM capture channel can inform you when there is a valid edge detected on the signal. You have to register a callback function to get the timer count value of the captured moment, by calling `mcpwm_capture_channel_register_event_callbacks()`. The callback function prototype is declared in `mcpwm_capture_event_cb_t`. All supported capture callbacks are listed in the `mcpwm_capture_event_callbacks_t`:

- *mcpwm_capture_event_callbacks_t::on_cap* sets the callback function for the capture channel when a valid edge is detected.

The callback function provides event-specific data of type *mcpwm_capture_event_data_t*, so that you can get the edge of the capture signal in *mcpwm_capture_event_data_t::cap_edge* and the count value of that moment in *mcpwm_capture_event_data_t::cap_value*. To convert the capture count into a timestamp, you need to know the resolution of the capture timer by calling *mcpwm_capture_timer_get_resolution()*.

The callback function is called within the ISR context, so it should **not** attempt to block. For example, you may make sure that only FreeRTOS APIs with the `ISR` suffix are called within the function.

The parameter `user_data` of *mcpwm_capture_channel_register_event_callbacks()* function is used to save your context. It is passed to the callback function directly.

This function will lazy install interrupt service for the MCPWM capture channel, whereas the service can only be removed in *mcpwm_del_capture_channel*.

Enable and Disable Capture Channel The capture channel is not enabled after allocation by *mcpwm_new_capture_channel()*. You should call *mcpwm_capture_channel_enable()* and *mcpwm_capture_channel_disable()* accordingly to enable or disable the channel. If the interrupt service is lazy installed during registering event callbacks for the channel in *mcpwm_capture_channel_register_event_callbacks()*, *mcpwm_capture_channel_enable()* will enable the interrupt service as well.

Enable and Disable Capture Timer Before doing IO control to the capture timer, you need to enable the timer first, by calling *mcpwm_capture_timer_enable()*. Internally, this function:

- switches the capture timer state from **init** to **enable**.
- acquires a proper power management lock if a specific clock source (e.g., APB clock) is selected. See also *Power management* for more information.

On the contrary, calling *mcpwm_capture_timer_disable()* will put the timer driver back to **init** state, and release the power management lock.

Start and Stop Capture Timer The basic IO operation of a capture timer is to start and stop. Calling *mcpwm_capture_timer_start()* can start the timer and calling *mcpwm_capture_timer_stop()* can stop the timer immediately.

Trigger a Software Capture Event Sometimes, the software also wants to trigger a "fake" capture event. The *mcpwm_capture_channel_trigger_soft_catch()* is provided for that purpose. Please note that, even though it is a "fake" capture event, it can still cause an interrupt, thus your capture event callback function gets invoked as well.

Power Management When power management is enabled (i.e., *CONFIG_PM_ENABLE* is on), the system will adjust the PLL and APB frequency before going into Light-sleep, thus potentially changing the period of an MCPWM timers' counting step and leading to inaccurate time-keeping.

However, the driver can prevent the system from changing APB frequency by acquiring a power management lock of type *ESP_PM_APB_FREQ_MAX*. Whenever the driver creates an MCPWM timer instance that has selected *MCPWM_TIMER_CLK_SRC_PLL160M* as its clock source, the driver guarantees that the power management lock is acquired when enabling the timer by *mcpwm_timer_enable()*. On the contrary, the driver releases the lock when *mcpwm_timer_disable()* is called for that timer.

Likewise, whenever the driver creates an MCPWM capture timer instance that has selected *MCPWM_CAPTURE_CLK_SRC_APB* as its clock source, the driver guarantees that the power management lock is acquired when enabling the timer by *mcpwm_capture_timer_enable()*. And releases the lock in *mcpwm_capture_timer_disable()*.

IRAM Safe By default, the MCPWM interrupt will be deferred when the Cache is disabled for reasons like writing/erasing Flash. Thus the event callback functions will not get executed in time, which is not expected in a real-time application.

There is a Kconfig option *CONFIG_MCPWM_ISR_IRAM_SAFE* that:

- enables the interrupt to be serviced even when the cache is disabled
- places all functions used by the ISR into IRAM[2]
- places the driver object into DRAM (in case it is mapped to PSRAM by accident)

This allows the interrupt to run while the cache is disabled but comes at the cost of increased IRAM consumption.

There is another Kconfig option *CONFIG_MCPWM_CTRL_FUNC_IN_IRAM* that can put commonly used IO control functions into IRAM as well. So, these functions can also be executable when the cache is disabled. The IO control function is as follows:

- `mcpwm_comparator_set_compare_value()`
- `mcpwm_timer_set_period()`

Thread Safety The factory functions like `mcpwm_new_timer()` are guaranteed to be thread-safe by the driver, which means, you can call it from different RTOS tasks without protection by extra locks.

The following function is allowed to run under the ISR context, as the driver uses a critical section to prevent them from being called concurrently in the task and ISR.

- `mcpwm_comparator_set_compare_value()`
- `mcpwm_timer_set_period()`

Other functions that are not related to *Resource Allocation and Initialization*, are not thread-safe. Thus, you should avoid calling them in different tasks without mutex protection.

Kconfig Options

- *CONFIG_MCPWM_ISR_IRAM_SAFE* controls whether the default ISR handler can work when the cache is disabled, see *IRAM Safe* for more information.
- *CONFIG_MCPWM_CTRL_FUNC_IN_IRAM* controls where to place the MCPWM control functions (IRAM or flash), see *IRAM Safe* for more information.
- *CONFIG_MCPWM_ENABLE_DEBUG_LOG* is used to enable the debug log output. Enabling this option will increase the firmware binary size.

Application Examples

- Brushed DC motor speed control by PID algorithm: peripherals/mcpwm/mcpwm_bdc_speed_control
- BLDC motor control with hall sensor feedback: peripherals/mcpwm/mcpwm_bldc_hall_control
- Ultrasonic sensor (HC-SR04) distance measurement: peripherals/mcpwm/mcpwm_capture_hc_sr04
- Servo motor angle control: peripherals/mcpwm/mcpwm_servo_control
- MCPWM synchronization between timers: peripherals/mcpwm/mcpwm_sync

API Reference

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_timer.h
- This header file can be included with:

```
#include "driver/mcpwm_timer.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

[2] The callback function and the sub-functions invoked by itself should also be placed in IRAM. You need to take care of this by yourself.

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_timer**(const *mcpwm_timer_config_t* *config, *mcpwm_timer_handle_t* *ret_timer)

Create MCPWM timer.

Parameters
- **config** -- **[in]** MCPWM timer configuration
- **ret_timer** -- **[out]** Returned MCPWM timer handle

Returns
- ESP_OK: Create MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Create MCPWM timer failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM timer failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM timer failed because all hardware timers are used up and no more free one
- ESP_FAIL: Create MCPWM timer failed because of other error

esp_err_t **mcpwm_del_timer**(*mcpwm_timer_handle_t* timer)

Delete MCPWM timer.

Parameters **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`

Returns
- ESP_OK: Delete MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM timer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Delete MCPWM timer failed because timer is not in init state
- ESP_FAIL: Delete MCPWM timer failed because of other error

esp_err_t **mcpwm_timer_set_period**(*mcpwm_timer_handle_t* timer, uint32_t period_ticks)

Set a new period for MCPWM timer.

Note: If *mcpwm_timer_config_t::update_period_on_empty* and *mcpwm_timer_config_t::update_period_on_sync* are not set, the new period will take effect immediately. Otherwise, the new period will take effect when timer counts to zero or on sync event.

Note: You may need to use `mcpwm_comparator_set_compare_value` to set a new compare value for MCPWM comparator in order to keep the same PWM duty cycle.

Parameters
- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer`
- **period_ticks** -- **[in]** New period in count ticks

Returns
- ESP_OK: Set new period for MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Set new period for MCPWM timer failed because of invalid argument
- ESP_FAIL: Set new period for MCPWM timer failed because of other error

esp_err_t **mcpwm_timer_enable**(*mcpwm_timer_handle_t* timer)

Enable MCPWM timer.

Parameters **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`

Returns

- ESP_OK: Enable MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Enable MCPWM timer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Enable MCPWM timer failed because timer is enabled already
- ESP_FAIL: Enable MCPWM timer failed because of other error

esp_err_t **mcpwm_timer_disable**(*mcpwm_timer_handle_t* timer)

Disable MCPWM timer.

Parameters **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`

Returns

- ESP_OK: Disable MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Disable MCPWM timer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable MCPWM timer failed because timer is disabled already
- ESP_FAIL: Disable MCPWM timer failed because of other error

esp_err_t **mcpwm_timer_start_stop**(*mcpwm_timer_handle_t* timer, *mcpwm_timer_start_stop_cmd_t* command)

Send specific start/stop commands to MCPWM timer.

Parameters

- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`
- **command** -- **[in]** Supported command list for MCPWM timer

Returns

- ESP_OK: Start or stop MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Start or stop MCPWM timer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Start or stop MCPWM timer failed because timer is not enabled
- ESP_FAIL: Start or stop MCPWM timer failed because of other error

esp_err_t **mcpwm_timer_register_event_callbacks**(*mcpwm_timer_handle_t* timer, const *mcpwm_timer_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for MCPWM timer.

Note: The first call to this function needs to be before the call to `mcpwm_timer_enable`

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set event callbacks failed because timer is not in init state
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **mcpwm_timer_set_phase_on_sync**(*mcpwm_timer_handle_t* timer, const *mcpwm_timer_sync_phase_config_t* *config)

Set sync phase for MCPWM timer.

Parameters

- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`
- **config** -- **[in]** MCPWM timer sync phase configuration

Returns

- ESP_OK: Set sync phase for MCPWM timer successfully
- ESP_ERR_INVALID_ARG: Set sync phase for MCPWM timer failed because of invalid argument
- ESP_FAIL: Set sync phase for MCPWM timer failed because of other error

Structures

struct **mcpwm_timer_event_callbacks_t**

Group of supported MCPWM timer event callbacks.

Note: The callbacks are all running under ISR environment

Public Members

mcpwm_timer_event_cb_t **on_full**

callback function when MCPWM timer counts to peak value

mcpwm_timer_event_cb_t **on_empty**

callback function when MCPWM timer counts to zero

mcpwm_timer_event_cb_t **on_stop**

callback function when MCPWM timer stops

struct **mcpwm_timer_config_t**

MCPWM timer configuration.

Public Members

int **group_id**

Specify from which group to allocate the MCPWM timer

mcpwm_timer_clock_source_t **clk_src**

MCPWM timer clock source

uint32_t **resolution_hz**

Counter resolution in Hz The step size of each count tick equals to (1 / resolution_hz) seconds

mcpwm_timer_count_mode_t **count_mode**

Count mode

uint32_t **period_ticks**

Number of count ticks within a period

int **intr_priority**

MCPWM timer interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **update_period_on_empty**

Whether to update period when timer counts to zero

uint32_t **update_period_on_sync**

Whether to update period on sync event

struct *mcpwm_timer_config_t*::[anonymous] **flags**

Extra configuration flags for timer

struct **mcpwm_timer_sync_phase_config_t**

MCPWM Timer sync phase configuration.

Public Members

mcpwm_sync_handle_t **sync_src**

The sync event source. Set to NULL will disable the timer being synced by others

uint32_t **count_value**

The count value that should lock to upon sync event

mcpwm_timer_direction_t **direction**

The count direction that should lock to upon sync event

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_oper.h
- This header file can be included with:

```
#include "driver/mcpwm_oper.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_operator**(const *mcpwm_operator_config_t* *config, *mcpwm_oper_handle_t* *ret_oper)

Create MCPWM operator.

Parameters

- **config** -- **[in]** MCPWM operator configuration
- **ret_oper** -- **[out]** Returned MCPWM operator handle

Returns

- ESP_OK: Create MCPWM operator successfully
- ESP_ERR_INVALID_ARG: Create MCPWM operator failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM operator failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM operator failed because can't find free resource
- ESP_FAIL: Create MCPWM operator failed because of other error

esp_err_t **mcpwm_del_operator**(*mcpwm_oper_handle_t* oper)

Delete MCPWM operator.

Parameters **oper** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`

Returns

- ESP_OK: Delete MCPWM operator successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM operator failed because of invalid argument
- ESP_FAIL: Delete MCPWM operator failed because of other error

esp_err_t **mcpwm_operator_connect_timer**(*mcpwm_oper_handle_t* oper, *mcpwm_timer_handle_t* timer)

Connect MCPWM operator and timer, so that the operator can be driven by the timer.

Parameters

- **oper** -- **[in]** MCPWM operator handle, allocated by `mcpwm_new_operator()`
- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`

Returns

- ESP_OK: Connect MCPWM operator and timer successfully
- ESP_ERR_INVALID_ARG: Connect MCPWM operator and timer failed because of invalid argument
- ESP_FAIL: Connect MCPWM operator and timer failed because of other error

esp_err_t **mcpwm_operator_set_brake_on_fault**(*mcpwm_oper_handle_t* oper, const *mcpwm_brake_config_t* *config)

Set brake method for MCPWM operator.

Parameters

- **oper** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`
- **config** -- **[in]** MCPWM brake configuration

Returns

- ESP_OK: Set trip for operator successfully
- ESP_ERR_INVALID_ARG: Set trip for operator failed because of invalid argument
- ESP_FAIL: Set trip for operator failed because of other error

esp_err_t **mcpwm_operator_recover_from_fault**(*mcpwm_oper_handle_t* oper, *mcpwm_fault_handle_t* fault)

Try to make the operator recover from fault.

Note: To recover from fault or escape from trip, you make sure the fault signal has disappeared already. Otherwise the recovery can't succeed.

Parameters

- **oper** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`
- **fault** -- **[in]** MCPWM fault handle

Returns

- ESP_OK: Recover from fault successfully
- ESP_ERR_INVALID_ARG: Recover from fault failed because of invalid argument
- ESP_ERR_INVALID_STATE: Recover from fault failed because the fault source is still active
- ESP_FAIL: Recover from fault failed because of other error

esp_err_t **mcpwm_operator_register_event_callbacks**(*mcpwm_oper_handle_t* oper, const *mcpwm_operator_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for MCPWM operator.

Note: User can deregister a previously registered callback by calling this function and setting the callback

member in the `cbs` structure to NULL.

Parameters

- **`oper`** -- **[in]** MCPWM operator handle, allocated by `mcpwm_new_operator()`
- **`cbs`** -- **[in]** Group of callback functions
- **`user_data`** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **`mcpwm_operator_apply_carrier`**(*mcpwm_oper_handle_t* oper, const *mcpwm_carrier_config_t* *config)

Apply carrier feature for MCPWM operator.

Parameters

- **`oper`** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`
- **`config`** -- **[in]** MCPWM carrier specific configuration

Returns

- ESP_OK: Set carrier for operator successfully
- ESP_ERR_INVALID_ARG: Set carrier for operator failed because of invalid argument
- ESP_FAIL: Set carrier for operator failed because of other error

Structures

struct **`mcpwm_operator_config_t`**

MCPWM operator configuration.

Public Members

int **`group_id`**

Specify from which group to allocate the MCPWM operator

int **`intr_priority`**

MCPWM operator interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **`update_gen_action_on_tez`**

Whether to update generator action when timer counts to zero

uint32_t **`update_gen_action_on_tep`**

Whether to update generator action when timer counts to peak

uint32_t **`update_gen_action_on_sync`**

Whether to update generator action on sync event

uint32_t **`update_dead_time_on_tez`**

Whether to update dead time when timer counts to zero

uint32_t **`update_dead_time_on_tep`**

Whether to update dead time when timer counts to peak

uint32_t **update_dead_time_on_sync**

Whether to update dead time on sync event

struct *mcpwm_operator_config_t*::[anonymous] **flags**

Extra configuration flags for operator

struct **mcpwm_brake_config_t**

MCPWM brake configuration structure.

Public Members

mcpwm_fault_handle_t **fault**

Which fault causes the operator to brake

mcpwm_operator_brake_mode_t **brake_mode**

Brake mode

uint32_t **cbc_recover_on_tez**

Recovery CBC brake state on tez event

uint32_t **cbc_recover_on_tep**

Recovery CBC brake state on tep event

struct *mcpwm_brake_config_t*::[anonymous] **flags**

Extra flags for brake configuration

struct **mcpwm_operator_event_callbacks_t**

Group of supported MCPWM operator event callbacks.

Note: The callbacks are all running under ISR environment

Public Members

mcpwm_brake_event_cb_t **on_brake_cbc**

callback function when mcpwm operator brakes in CBC

mcpwm_brake_event_cb_t **on_brake_ost**

callback function when mcpwm operator brakes in OST

struct **mcpwm_carrier_config_t**

MCPWM carrier configuration structure.

Public Members

mcpwm_carrier_clock_source_t **clk_src**

MCPWM carrier clock source

uint32_t **frequency_hz**

Carrier frequency in Hz

uint32_t **first_pulse_duration_us**

The duration of the first PWM pulse, in us

float **duty_cycle**

Carrier duty cycle

uint32_t **invert_before_modulate**

Invert the raw signal

uint32_t **invert_after_modulate**

Invert the modulated signal

struct *mcpwm_carrier_config_t*::[anonymous] **flags**

Extra flags for carrier configuration

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_cmpr.h
- This header file can be included with:

```
#include "driver/mcpwm_cmpr.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_comparator**(*mcpwm_oper_handle_t* oper, const *mcpwm_comparator_config_t* *config, *mcpwm_cmpr_handle_t* *ret_cmpr)

Create MCPWM comparator.

Parameters

- **oper** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`, the new comparator will be allocated from this operator
- **config** -- **[in]** MCPWM comparator configuration
- **ret_cmpr** -- **[out]** Returned MCPWM comparator

Returns

- ESP_OK: Create MCPWM comparator successfully
- ESP_ERR_INVALID_ARG: Create MCPWM comparator failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM comparator failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM comparator failed because can't find free resource
- ESP_FAIL: Create MCPWM comparator failed because of other error

esp_err_t **mcpwm_del_comparator**(*mcpwm_cmpr_handle_t* cmpr)

Delete MCPWM comparator.

Parameters **cmpr** -- **[in]** MCPWM comparator handle, allocated by `mcpwm_new_comparator()`

Returns

- ESP_OK: Delete MCPWM comparator successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM comparator failed because of invalid argument
- ESP_FAIL: Delete MCPWM comparator failed because of other error

esp_err_t **mcpwm_comparator_register_event_callbacks**(*mcpwm_cmpr_handle_t* cmpr, const *mcpwm_comparator_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for MCPWM comparator.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **cmpr** -- **[in]** MCPWM comparator handle, allocated by `mcpwm_new_comparator()`
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **mcpwm_comparator_set_compare_value**(*mcpwm_cmpr_handle_t* cmpr, uint32_t cmp_ticks)

Set MCPWM comparator's compare value.

Parameters

- **cmpr** -- **[in]** MCPWM comparator handle, allocated by `mcpwm_new_comparator()`
- **cmp_ticks** -- **[in]** The new compare value

Returns

- ESP_OK: Set MCPWM compare value successfully
- ESP_ERR_INVALID_ARG: Set MCPWM compare value failed because of invalid argument (e.g. the cmp_ticks is out of range)
- ESP_ERR_INVALID_STATE: Set MCPWM compare value failed because the operator doesn't have a timer connected
- ESP_FAIL: Set MCPWM compare value failed because of other error

Structures

struct **mcpwm_comparator_config_t**

MCPWM comparator configuration.

Public Members

int **intr_priority**

MCPWM comparator interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **update_cmp_on_tez**

Whether to update compare value when timer count equals to zero (tez)

uint32_t **update_cmp_on_tep**

Whether to update compare value when timer count equals to peak (tep)

uint32_t **update_cmp_on_sync**

Whether to update compare value on sync event

struct *mcpwm_comparator_config_t*::[anonymous] **flags**

Extra configuration flags for comparator

struct **mcpwm_comparator_event_callbacks_t**

Group of supported MCPWM compare event callbacks.

Note: The callbacks are all running under ISR environment

Public Members

mcpwm_compare_event_cb_t **on_reach**

ISR callback function which would be invoked when counter reaches compare value

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_gen.h
- This header file can be included with:

```
#include "driver/mcpwm_gen.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_generator**(*mcpwm_oper_handle_t* oper, const *mcpwm_generator_config_t* *config, *mcpwm_gen_handle_t* *ret_gen)

Allocate MCPWM generator from given operator.

Parameters

- **oper** -- **[in]** MCPWM operator, allocated by `mcpwm_new_operator()`
- **config** -- **[in]** MCPWM generator configuration
- **ret_gen** -- **[out]** Returned MCPWM generator

Returns

- ESP_OK: Create MCPWM generator successfully
- ESP_ERR_INVALID_ARG: Create MCPWM generator failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM generator failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM generator failed because can't find free resource
- ESP_FAIL: Create MCPWM generator failed because of other error

esp_err_t **mcpwm_del_generator**(*mcpwm_gen_handle_t* gen)

Delete MCPWM generator.

Parameters **gen** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`

Returns

- ESP_OK: Delete MCPWM generator successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM generator failed because of invalid argument
- ESP_FAIL: Delete MCPWM generator failed because of other error

esp_err_t **mcpwm_generator_set_force_level**(*mcpwm_gen_handle_t* gen, int level, bool hold_on)

Set force level for MCPWM generator.

Note: The force level will be applied to the generator immediately, regardless any other events that would change the generator's behaviour.

Note: If the `hold_on` is true, the force level will retain forever, until user removes the force level by setting the force level to `-1`.

Note: If the `hold_on` is false, the force level can be overridden by the next event action.

Note: The force level set by this function can be inverted by GPIO matrix or dead-time module. So the level set here doesn't equal to the final output level.

Parameters

- **gen** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **level** -- **[in]** GPIO level to be applied to MCPWM generator, specially, -1 means to remove the force level
- **hold_on** -- **[in]** Whether the forced PWM level should retain (i.e. will remain unchanged until manually remove the force level)

Returns

- ESP_OK: Set force level for MCPWM generator successfully
- ESP_ERR_INVALID_ARG: Set force level for MCPWM generator failed because of invalid argument
- ESP_FAIL: Set force level for MCPWM generator failed because of other error

esp_err_t **mcpwm_generator_set_action_on_timer_event**(*mcpwm_gen_handle_t* gen, *mcpwm_gen_timer_event_action_t* ev_act)

Set generator action on MCPWM timer event.

Parameters

- **gen** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM timer event action, can be constructed by `MCPWM_GEN_TIMER_EVENT_ACTION` helper macro

Returns

- ESP_OK: Set generator action successfully
- ESP_ERR_INVALID_ARG: Set generator action failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set generator action failed because of timer is not connected to operator
- ESP_FAIL: Set generator action failed because of other error

esp_err_t **mcpwm_generator_set_actions_on_timer_event**(*mcpwm_gen_handle_t* gen, *mcpwm_gen_timer_event_action_t* ev_act, ...)

Set generator actions on multiple MCPWM timer events.

Note: This is an aggregation version of `mcpwm_generator_set_action_on_timer_event`, which allows user to set multiple actions in one call.

Parameters

- **gen** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM timer event action list, must be terminated by `MCPWM_GEN_TIMER_EVENT_ACTION_END()`

Returns

- ESP_OK: Set generator actions successfully
- ESP_ERR_INVALID_ARG: Set generator actions failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set generator actions failed because of timer is not connected to operator
- ESP_FAIL: Set generator actions failed because of other error

esp_err_t **mcpwm_generator_set_action_on_compare_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_compare_event_action_t* ev_act)

Set generator action on MCPWM compare event.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM compare event action, can be constructed by `MCPWM_GEN_COMPARE_EVENT_ACTION` helper macro

Returns

- ESP_OK: Set generator action successfully
- ESP_ERR_INVALID_ARG: Set generator action failed because of invalid argument
- ESP_FAIL: Set generator action failed because of other error

esp_err_t **mcpwm_generator_set_actions_on_compare_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_compare_event_action_t* ev_act, ...)

Set generator actions on multiple MCPWM compare events.

Note: This is an aggregation version of `mcpwm_generator_set_action_on_compare_event`, which allows user to set multiple actions in one call.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM compare event action list, must be terminated by `MCPWM_GEN_COMPARE_EVENT_ACTION_END()`

Returns

- ESP_OK: Set generator actions successfully
- ESP_ERR_INVALID_ARG: Set generator actions failed because of invalid argument
- ESP_FAIL: Set generator actions failed because of other error

esp_err_t **mcpwm_generator_set_action_on_brake_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_brake_event_action_t* ev_act)

Set generator action on MCPWM brake event.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM brake event action, can be constructed by `MCPWM_GEN_BRAKE_EVENT_ACTION` helper macro

Returns

- ESP_OK: Set generator action successfully
- ESP_ERR_INVALID_ARG: Set generator action failed because of invalid argument
- ESP_FAIL: Set generator action failed because of other error

esp_err_t **mcpwm_generator_set_actions_on_brake_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_brake_event_action_t* ev_act, ...)

Set generator actions on multiple MCPWM brake events.

Note: This is an aggregation version of `mcpwm_generator_set_action_on_brake_event`, which allows user to set multiple actions in one call.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM brake event action list, must be terminated by `MCPWM_GEN_BRAKE_EVENT_ACTION_END()`

Returns

- ESP_OK: Set generator actions successfully
- ESP_ERR_INVALID_ARG: Set generator actions failed because of invalid argument
- ESP_FAIL: Set generator actions failed because of other error

esp_err_t **mcpwm_generator_set_action_on_fault_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_fault_event_action_t* ev_act)

Set generator action on MCPWM Fault event.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`
- **ev_act** -- **[in]** MCPWM trigger event action, can be constructed by `MCPWM_GEN_FAULT_EVENT_ACTION` helper macro

Returns

- ESP_OK: Set generator action successfully
- ESP_ERR_INVALID_ARG: Set generator action failed because of invalid argument
- ESP_FAIL: Set generator action failed because of other error

esp_err_t **mcpwm_generator_set_action_on_sync_event**(*mcpwm_gen_handle_t* generator, *mcpwm_gen_sync_event_action_t* ev_act)

Set generator action on MCPWM Sync event.

Note: The trigger only support one sync action, regardless of the kinds. Should not call this function more than once.

Parameters

- **generator** -- **[in]** MCPWM generator handle, allocated by `mcpwm_new_generator()`

- **ev_act** -- **[in]** MCPWM trigger event action, can be constructed by `MCPWM_GEN_SYNC_EVENT_ACTION` helper macro

Returns

- ESP_OK: Set generator action successfully
- ESP_ERR_INVALID_ARG: Set generator action failed because of invalid argument
- ESP_FAIL: Set generator action failed because of other error

esp_err_t **mcpwm_generator_set_dead_time**(*mcpwm_gen_handle_t* in_generator, *mcpwm_gen_handle_t* out_generator, const *mcpwm_dead_time_config_t* *config)

Set dead time for MCPWM generator.

Note: Due to a hardware limitation, you can't set rising edge delay for both MCPWM generator 0 and 1 at the same time, otherwise, there will be a conflict inside the dead time module. The same goes for the falling edge setting. But you can set both the rising edge and falling edge delay for the same MCPWM generator.

Parameters

- **in_generator** -- **[in]** MCPWM generator, before adding the dead time
- **out_generator** -- **[in]** MCPWM generator, after adding the dead time
- **config** -- **[in]** MCPWM dead time configuration

Returns

- ESP_OK: Set dead time for MCPWM generator successfully
- ESP_ERR_INVALID_ARG: Set dead time for MCPWM generator failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set dead time for MCPWM generator failed because of invalid state (e.g. delay module is already in use by other generator)
- ESP_FAIL: Set dead time for MCPWM generator failed because of other error

Structures

struct **mcpwm_generator_config_t**

MCPWM generator configuration.

Public Members

int **gen_gpio_num**

The GPIO number used to output the PWM signal

uint32_t **invert_pwm**

Whether to invert the PWM signal (done by GPIO matrix)

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

uint32_t **io_od_mode**

Configure the GPIO as open-drain mode

uint32_t **pull_up**

Whether to pull up internally

uint32_t **pull_down**

Whether to pull down internally

struct *mcpwm_generator_config_t*::[anonymous] **flags**

Extra configuration flags for generator

struct **mcpwm_gen_timer_event_action_t**

Generator action on specific timer event.

Public Members

mcpwm_timer_direction_t **direction**

Timer direction

mcpwm_timer_event_t **event**

Timer event

mcpwm_generator_action_t **action**

Generator action should perform

struct **mcpwm_gen_compare_event_action_t**

Generator action on specific comparator event.

Public Members

mcpwm_timer_direction_t **direction**

Timer direction

mcpwm_cmpr_handle_t **comparator**

Comparator handle

mcpwm_generator_action_t **action**

Generator action should perform

struct **mcpwm_gen_brake_event_action_t**

Generator action on specific brake event.

Public Members

mcpwm_timer_direction_t **direction**

Timer direction

mcpwm_operator_brake_mode_t **brake_mode**

Brake mode

mcpwm_generator_action_t **action**

Generator action should perform

struct **mcpwm_gen_fault_event_action_t**

Generator action on specific fault event.

Public Members

mcpwm_timer_direction_t **direction**

Timer direction

mcpwm_fault_handle_t **fault**

Which fault as the trigger. Only support GPIO fault

mcpwm_generator_action_t **action**

Generator action should perform

struct **mcpwm_gen_sync_event_action_t**

Generator action on specific sync event.

Public Members

mcpwm_timer_direction_t **direction**

Timer direction

mcpwm_sync_handle_t **sync**

Which sync as the trigger

mcpwm_generator_action_t **action**

Generator action should perform

struct **mcpwm_dead_time_config_t**

MCPWM dead time configuration structure.

Public Members

uint32_t **posedge_delay_ticks**

delay time applied to rising edge, 0 means no rising delay time

uint32_t **negedge_delay_ticks**

delay time applied to falling edge, 0 means no falling delay time

uint32_t **invert_output**

Invert the signal after applied the dead time

struct *mcpwm_dead_time_config_t*::[anonymous] **flags**

Extra flags for dead time configuration

Macros

MCPWM_GEN_TIMER_EVENT_ACTION(dir, ev, act)

Help macros to construct a *mcpwm_gen_timer_event_action_t* entry.

MCPWM_GEN_TIMER_EVENT_ACTION_END()

MCPWM_GEN_COMPARE_EVENT_ACTION(dir, cmp, act)

Help macros to construct a *mcpwm_gen_compare_event_action_t* entry.

MCPWM_GEN_COMPARE_EVENT_ACTION_END()

MCPWM_GEN_BRAKE_EVENT_ACTION(dir, mode, act)

Help macros to construct a *mcpwm_gen_brake_event_action_t* entry.

MCPWM_GEN_BRAKE_EVENT_ACTION_END()

MCPWM_GEN_FAULT_EVENT_ACTION(dir, flt, act)

Help macros to construct a *mcpwm_gen_fault_event_action_t* entry.

MCPWM_GEN_SYNC_EVENT_ACTION(dir, syn, act)

Help macros to construct a *mcpwm_gen_sync_event_action_t* entry.

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_fault.h
- This header file can be included with:

```
#include "driver/mcpwm_fault.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_gpio_fault**(const *mcpwm_gpio_fault_config_t* *config, *mcpwm_fault_handle_t* *ret_fault)

Create MCPWM GPIO fault.

Parameters

- **config** -- **[in]** MCPWM GPIO fault configuration
- **ret_fault** -- **[out]** Returned GPIO fault handle

Returns

- ESP_OK: Create MCPWM GPIO fault successfully
- ESP_ERR_INVALID_ARG: Create MCPWM GPIO fault failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM GPIO fault failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM GPIO fault failed because can't find free resource
- ESP_FAIL: Create MCPWM GPIO fault failed because of other error

esp_err_t **mcpwm_new_soft_fault**(const *mcpwm_soft_fault_config_t* *config, *mcpwm_fault_handle_t* *ret_fault)

Create MCPWM software fault.

Parameters

- **config** -- **[in]** MCPWM software fault configuration
- **ret_fault** -- **[out]** Returned software fault handle

Returns

- ESP_OK: Create MCPWM software fault successfully
- ESP_ERR_INVALID_ARG: Create MCPWM software fault failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM software fault failed because out of memory

- ESP_FAIL: Create MCPWM software fault failed because of other error

esp_err_t **mcpwm_del_fault**(*mcpwm_fault_handle_t* fault)

Delete MCPWM fault.

Parameters **fault** -- **[in]** MCPWM fault handle allocated by `mcpwm_new_gpio_fault()` or `mcpwm_new_soft_fault()`

Returns

- ESP_OK: Delete MCPWM fault successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM fault failed because of invalid argument
- ESP_FAIL: Delete MCPWM fault failed because of other error

esp_err_t **mcpwm_soft_fault_activate**(*mcpwm_fault_handle_t* fault)

Activate the software fault, trigger the fault event for once.

Parameters **fault** -- **[in]** MCPWM soft fault, allocated by `mcpwm_new_soft_fault()`

Returns

- ESP_OK: Trigger MCPWM software fault event successfully
- ESP_ERR_INVALID_ARG: Trigger MCPWM software fault event failed because of invalid argument
- ESP_FAIL: Trigger MCPWM software fault event failed because of other error

esp_err_t **mcpwm_fault_register_event_callbacks**(*mcpwm_fault_handle_t* fault, const *mcpwm_fault_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for MCPWM fault.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **fault** -- **[in]** MCPWM GPIO fault handle, allocated by `mcpwm_new_gpio_fault()`
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

Structures

struct **mcpwm_gpio_fault_config_t**

MCPWM GPIO fault configuration structure.

Public Members

int **group_id**

In which MCPWM group that the GPIO fault belongs to

int **intr_priority**

MCPWM GPIO fault interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

int **gpio_num**

GPIO used by the fault signal

uint32_t **active_level**

On which level the fault signal is treated as active

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

uint32_t **pull_up**

Whether to pull up internally

uint32_t **pull_down**

Whether to pull down internally

struct *mcpwm_gpio_fault_config_t*::[anonymous] **flags**

Extra configuration flags for GPIO fault

struct **mcpwm_soft_fault_config_t**

MCPWM software fault configuration structure.

struct **mcpwm_fault_event_callbacks_t**

Group of supported MCPWM fault event callbacks.

Note: The callbacks are all running under ISR environment

Public Members

mcpwm_fault_event_cb_t **on_fault_enter**

ISR callback function that would be invoked when fault signal becomes active

mcpwm_fault_event_cb_t **on_fault_exit**

ISR callback function that would be invoked when fault signal becomes inactive

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_sync.h
- This header file can be included with:

```
#include "driver/mcpwm_sync.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_timer_sync_src**(*mcpwm_timer_handle_t* timer, const *mcpwm_timer_sync_src_config_t* *config, *mcpwm_sync_handle_t* *ret_sync)

Create MCPWM timer sync source.

Parameters
- **timer** -- **[in]** MCPWM timer handle, allocated by `mcpwm_new_timer()`
- **config** -- **[in]** MCPWM timer sync source configuration
- **ret_sync** -- **[out]** Returned MCPWM sync handle

Returns
- ESP_OK: Create MCPWM timer sync source successfully
- ESP_ERR_INVALID_ARG: Create MCPWM timer sync source failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM timer sync source failed because out of memory
- ESP_ERR_INVALID_STATE: Create MCPWM timer sync source failed because the timer has created a sync source before
- ESP_FAIL: Create MCPWM timer sync source failed because of other error

esp_err_t **mcpwm_new_gpio_sync_src**(const *mcpwm_gpio_sync_src_config_t* *config, *mcpwm_sync_handle_t* *ret_sync)

Create MCPWM GPIO sync source.

Parameters
- **config** -- **[in]** MCPWM GPIO sync source configuration
- **ret_sync** -- **[out]** Returned MCPWM GPIO sync handle

Returns
- ESP_OK: Create MCPWM GPIO sync source successfully
- ESP_ERR_INVALID_ARG: Create MCPWM GPIO sync source failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM GPIO sync source failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM GPIO sync source failed because can't find free resource
- ESP_FAIL: Create MCPWM GPIO sync source failed because of other error

esp_err_t **mcpwm_new_soft_sync_src**(const *mcpwm_soft_sync_config_t* *config, *mcpwm_sync_handle_t* *ret_sync)

Create MCPWM software sync source.

Parameters
- **config** -- **[in]** MCPWM software sync source configuration
- **ret_sync** -- **[out]** Returned software sync handle

Returns
- ESP_OK: Create MCPWM software sync successfully
- ESP_ERR_INVALID_ARG: Create MCPWM software sync failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM software sync failed because out of memory
- ESP_FAIL: Create MCPWM software sync failed because of other error

esp_err_t **mcpwm_del_sync_src**(*mcpwm_sync_handle_t* sync)

Delete MCPWM sync source.

Parameters **sync** -- **[in]** MCPWM sync handle, allocated by `mcpwm_new_timer_sync_src()` or `mcpwm_new_gpio_sync_src()` or `mcpwm_new_soft_sync_src()`

Returns
- ESP_OK: Delete MCPWM sync source successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM sync source failed because of invalid argument
- ESP_FAIL: Delete MCPWM sync source failed because of other error

esp_err_t **mcpwm_soft_sync_activate**(*mcpwm_sync_handle_t* sync)

Activate the software sync, trigger the sync event for once.

Parameters **sync** -- **[in]** MCPWM soft sync handle, allocated by `mcpwm_new_soft_sync_src()`

Returns

- ESP_OK: Trigger MCPWM software sync event successfully
- ESP_ERR_INVALID_ARG: Trigger MCPWM software sync event failed because of invalid argument
- ESP_FAIL: Trigger MCPWM software sync event failed because of other error

Structures

struct **mcpwm_timer_sync_src_config_t**

MCPWM timer sync source configuration.

Public Members

mcpwm_timer_event_t **timer_event**

Timer event, upon which MCPWM timer will generate the sync signal

uint32_t **propagate_input_sync**

The input sync signal would be routed to its sync output

struct *mcpwm_timer_sync_src_config_t*::[anonymous] **flags**

Extra configuration flags for timer sync source

struct **mcpwm_gpio_sync_src_config_t**

MCPWM GPIO sync source configuration.

Public Members

int **group_id**

MCPWM group ID

int **gpio_num**

GPIO used by sync source

uint32_t **active_neg**

Whether the sync signal is active on negedge, by default, the sync signal's posedge is treated as active

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

uint32_t **pull_up**

Whether to pull up internally

uint32_t **pull_down**

Whether to pull down internally

struct *mcpwm_gpio_sync_src_config_t*::[anonymous] **flags**

Extra configuration flags for GPIO sync source

struct **mcpwm_soft_sync_config_t**

MCPWM software sync configuration structure.

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_cap.h
- This header file can be included with:

```
#include "driver/mcpwm_cap.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_new_capture_timer**(const *mcpwm_capture_timer_config_t* *config, *mcpwm_cap_timer_handle_t* *ret_cap_timer)

Create MCPWM capture timer.

Parameters
- **config** -- **[in]** MCPWM capture timer configuration
- **ret_cap_timer** -- **[out]** Returned MCPWM capture timer handle

Returns
- ESP_OK: Create MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Create MCPWM capture timer failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM capture timer failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM capture timer failed because can't find free resource
- ESP_FAIL: Create MCPWM capture timer failed because of other error

esp_err_t **mcpwm_del_capture_timer**(*mcpwm_cap_timer_handle_t* cap_timer)

Delete MCPWM capture timer.

Parameters **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`

Returns
- ESP_OK: Delete MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM capture timer failed because of invalid argument
- ESP_FAIL: Delete MCPWM capture timer failed because of other error

esp_err_t **mcpwm_capture_timer_enable**(*mcpwm_cap_timer_handle_t* cap_timer)

Enable MCPWM capture timer.

Parameters **cap_timer** -- **[in]** MCPWM capture timer handle, allocated by `mcpwm_new_capture_timer()`

Returns
- ESP_OK: Enable MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Enable MCPWM capture timer failed because of invalid argument

- ESP_ERR_INVALID_STATE: Enable MCPWM capture timer failed because timer is enabled already
- ESP_FAIL: Enable MCPWM capture timer failed because of other error

esp_err_t **mcpwm_capture_timer_disable**(*mcpwm_cap_timer_handle_t* cap_timer)

Disable MCPWM capture timer.

Parameters **cap_timer** -- **[in]** MCPWM capture timer handle, allocated by `mcpwm_new_capture_timer()`

Returns

- ESP_OK: Disable MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Disable MCPWM capture timer failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable MCPWM capture timer failed because timer is disabled already
- ESP_FAIL: Disable MCPWM capture timer failed because of other error

esp_err_t **mcpwm_capture_timer_start**(*mcpwm_cap_timer_handle_t* cap_timer)

Start MCPWM capture timer.

Parameters **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`

Returns

- ESP_OK: Start MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Start MCPWM capture timer failed because of invalid argument
- ESP_FAIL: Start MCPWM capture timer failed because of other error

esp_err_t **mcpwm_capture_timer_stop**(*mcpwm_cap_timer_handle_t* cap_timer)

Stop MCPWM capture timer.

Parameters **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`

Returns

- ESP_OK: Stop MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Stop MCPWM capture timer failed because of invalid argument
- ESP_FAIL: Stop MCPWM capture timer failed because of other error

esp_err_t **mcpwm_capture_timer_get_resolution**(*mcpwm_cap_timer_handle_t* cap_timer, uint32_t *out_resolution)

Get MCPWM capture timer resolution, in Hz.

Parameters

- **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`
- **out_resolution** -- **[out]** Returned capture timer resolution, in Hz

Returns

- ESP_OK: Get capture timer resolution successfully
- ESP_ERR_INVALID_ARG: Get capture timer resolution failed because of invalid argument
- ESP_FAIL: Get capture timer resolution failed because of other error

esp_err_t **mcpwm_capture_timer_set_phase_on_sync**(*mcpwm_cap_timer_handle_t* cap_timer, const *mcpwm_capture_timer_sync_phase_config_t* *config)

Set sync phase for MCPWM capture timer.

Parameters

- **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`
- **config** -- **[in]** MCPWM capture timer sync phase configuration

Returns

- ESP_OK: Set sync phase for MCPWM capture timer successfully
- ESP_ERR_INVALID_ARG: Set sync phase for MCPWM capture timer failed because of invalid argument
- ESP_FAIL: Set sync phase for MCPWM capture timer failed because of other error

esp_err_t **mcpwm_new_capture_channel**(*mcpwm_cap_timer_handle_t* cap_timer, const *mcpwm_capture_channel_config_t* *config, *mcpwm_cap_channel_handle_t* *ret_cap_channel)

Create MCPWM capture channel.

Note: The created capture channel won't be enabled until calling `mcpwm_capture_channel_enable`

Parameters

- **cap_timer** -- **[in]** MCPWM capture timer, allocated by `mcpwm_new_capture_timer()`, will be connected to the new capture channel
- **config** -- **[in]** MCPWM capture channel configuration
- **ret_cap_channel** -- **[out]** Returned MCPWM capture channel

Returns

- ESP_OK: Create MCPWM capture channel successfully
- ESP_ERR_INVALID_ARG: Create MCPWM capture channel failed because of invalid argument
- ESP_ERR_NO_MEM: Create MCPWM capture channel failed because out of memory
- ESP_ERR_NOT_FOUND: Create MCPWM capture channel failed because can't find free resource
- ESP_FAIL: Create MCPWM capture channel failed because of other error

esp_err_t **mcpwm_del_capture_channel**(*mcpwm_cap_channel_handle_t* cap_channel)

Delete MCPWM capture channel.

Parameters **cap_channel** -- **[in]** MCPWM capture channel handle, allocated by `mcpwm_new_capture_channel()`

Returns

- ESP_OK: Delete MCPWM capture channel successfully
- ESP_ERR_INVALID_ARG: Delete MCPWM capture channel failed because of invalid argument
- ESP_FAIL: Delete MCPWM capture channel failed because of other error

esp_err_t **mcpwm_capture_channel_enable**(*mcpwm_cap_channel_handle_t* cap_channel)

Enable MCPWM capture channel.

Note: This function will transit the channel state from init to enable.

Note: This function will enable the interrupt service, if it's lazy installed in `mcpwm_capture_channel_register_event_callbacks()`.

Parameters **cap_channel** -- **[in]** MCPWM capture channel handle, allocated by `mcpwm_new_capture_channel()`

Returns

- ESP_OK: Enable MCPWM capture channel successfully
- ESP_ERR_INVALID_ARG: Enable MCPWM capture channel failed because of invalid argument

- ESP_ERR_INVALID_STATE: Enable MCPWM capture channel failed because the channel is already enabled
- ESP_FAIL: Enable MCPWM capture channel failed because of other error

esp_err_t **mcpwm_capture_channel_disable**(*mcpwm_cap_channel_handle_t* cap_channel)

Disable MCPWM capture channel.

Parameters **cap_channel** -- **[in]** MCPWM capture channel handle, allocated by `mcpwm_new_capture_channel()`

Returns

- ESP_OK: Disable MCPWM capture channel successfully
- ESP_ERR_INVALID_ARG: Disable MCPWM capture channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable MCPWM capture channel failed because the channel is not enabled yet
- ESP_FAIL: Disable MCPWM capture channel failed because of other error

esp_err_t **mcpwm_capture_channel_register_event_callbacks**(*mcpwm_cap_channel_handle_t* cap_channel, const *mcpwm_capture_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for MCPWM capture channel.

Note: The first call to this function needs to be before the call to `mcpwm_capture_channel_enable`

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **cap_channel** -- **[in]** MCPWM capture channel handle, allocated by `mcpwm_new_capture_channel()`
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set event callbacks failed because the channel is not in init state
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **mcpwm_capture_channel_trigger_soft_catch**(*mcpwm_cap_channel_handle_t* cap_channel)

Trigger a catch by software.

Parameters **cap_channel** -- **[in]** MCPWM capture channel handle, allocated by `mcpwm_new_capture_channel()`

Returns

- ESP_OK: Trigger software catch successfully
- ESP_ERR_INVALID_ARG: Trigger software catch failed because of invalid argument
- ESP_ERR_INVALID_STATE: Trigger software catch failed because the channel is not enabled yet
- ESP_FAIL: Trigger software catch failed because of other error

Structures

struct **mcpwm_capture_timer_config_t**

MCPWM capture timer configuration structure.

Public Members

int **group_id**

Specify from which group to allocate the capture timer

mcpwm_capture_clock_source_t **clk_src**

MCPWM capture timer clock source

uint32_t **resolution_hz**

Resolution of capture timer

struct **mcpwm_capture_timer_sync_phase_config_t**

MCPWM Capture timer sync phase configuration.

Public Members

mcpwm_sync_handle_t **sync_src**

The sync event source

uint32_t **count_value**

The count value that should lock to upon sync event

mcpwm_timer_direction_t **direction**

The count direction that should lock to upon sync event

struct **mcpwm_capture_channel_config_t**

MCPWM capture channel configuration structure.

Public Members

int **gpio_num**

GPIO used capturing input signal

int **intr_priority**

MCPWM capture interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **prescale**

Prescale of input signal, effective frequency = cap_input_clk/prescale

struct *mcpwm_capture_channel_config_t*::*extra_flags* **flags**

Extra configuration flags for capture channel

struct **extra_flags**

Extra configuration flags for capture channel.

Public Members

uint32_t **pos_edge**

Whether to capture on positive edge

uint32_t **neg_edge**

Whether to capture on negative edge

uint32_t **pull_up**

Whether to pull up internally

uint32_t **pull_down**

Whether to pull down internally

uint32_t **invert_cap_signal**

Invert the input capture signal

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

uint32_t **keep_io_conf_at_exit**

For debug/test, whether to keep the GPIO configuration when capture channel is deleted. By default, driver will reset the GPIO pin at exit.

struct **mcpwm_capture_event_callbacks_t**

Group of supported MCPWM capture event callbacks.

Note: The callbacks are all running under ISR environment

Public Members

mcpwm_capture_event_cb_t **on_cap**

Callback function that would be invoked when capture event occurred

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_etm.h
- This header file can be included with:

```
#include "driver/mcpwm_etm.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Functions

esp_err_t **mcpwm_comparator_new_etm_event**(*mcpwm_cmpr_handle_t* cmpr, const *mcpwm_cmpr_etm_event_config_t* *config, esp_etm_event_handle_t *out_event)

Get the ETM event for MCPWM comparator.

Note: The created ETM event object can be deleted later by calling `esp_etm_del_event`

Parameters

- **cmpr** -- **[in]** MCPWM comparator, allocated by `mcpwm_new_comparator()` or `mcpwm_new_event_comparator()`
- **config** -- **[in]** MCPWM ETM comparator event configuration
- **out_event** -- **[out]** Returned ETM event handle

Returns

- ESP_OK: Get ETM event successfully
- ESP_ERR_INVALID_ARG: Get ETM event failed because of invalid argument
- ESP_FAIL: Get ETM event failed because of other error

Structures

struct **mcpwm_cmpr_etm_event_config_t**

MCPWM event comparator ETM event configuration.

Public Members

mcpwm_comparator_etm_event_type_t **event_type**

MCPWM comparator ETM event type

Header File

- components/esp_driver_mcpwm/include/driver/mcpwm_types.h
- This header file can be included with:

```
#include "driver/mcpwm_types.h"
```

- This header file is a part of the API provided by the `esp_driver_mcpwm` component. To declare that your component depends on `esp_driver_mcpwm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_mcpwm
```

or

```
PRIV_REQUIRES esp_driver_mcpwm
```

Structures

struct **mcpwm_timer_event_data_t**

MCPWM timer event data.

Public Members

uint32_t **count_value**

MCPWM timer count value

mcpwm_timer_direction_t **direction**

MCPWM timer count direction

struct **mcpwm_brake_event_data_t**

MCPWM brake event data.

struct **mcpwm_fault_event_data_t**

MCPWM fault event data.

struct **mcpwm_compare_event_data_t**

MCPWM compare event data.

Public Members

uint32_t **compare_ticks**

Compare value

mcpwm_timer_direction_t **direction**

Count direction

struct **mcpwm_capture_event_data_t**

MCPWM capture event data.

Public Members

uint32_t **cap_value**

Captured value

mcpwm_capture_edge_t **cap_edge**

Capture edge

Type Definitions

typedef struct mcpwm_timer_t ***mcpwm_timer_handle_t**

Type of MCPWM timer handle.

typedef struct mcpwm_oper_t ***mcpwm_oper_handle_t**

Type of MCPWM operator handle.

typedef struct mcpwm_cmpr_t ***mcpwm_cmpr_handle_t**

Type of MCPWM comparator handle.

typedef struct mcpwm_gen_t ***mcpwm_gen_handle_t**

Type of MCPWM generator handle.

typedef struct mcpwm_fault_t ***mcpwm_fault_handle_t**

Type of MCPWM fault handle.

typedef struct mcpwm_sync_t ***mcpwm_sync_handle_t**

Type of MCPWM sync handle.

typedef struct mcpwm_cap_timer_t ***mcpwm_cap_timer_handle_t**

Type of MCPWM capture timer handle.

typedef struct mcpwm_cap_channel_t ***mcpwm_cap_channel_handle_t**

Type of MCPWM capture channel handle.

typedef bool (***mcpwm_timer_event_cb_t**)(*mcpwm_timer_handle_t* timer, const *mcpwm_timer_event_data_t* *edata, void *user_ctx)

MCPWM timer event callback function.

Param timer **[in]** MCPWM timer handle
Param edata **[in]** MCPWM timer event data, fed by driver
Param user_ctx **[in]** User data, set in `mcpwm_timer_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***mcpwm_brake_event_cb_t**)(*mcpwm_oper_handle_t* oper, const *mcpwm_brake_event_data_t* *edata, void *user_ctx)

MCPWM operator brake event callback function.

Param oper **[in]** MCPWM operator handle
Param edata **[in]** MCPWM brake event data, fed by driver
Param user_ctx **[in]** User data, set in `mcpwm_operator_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***mcpwm_fault_event_cb_t**)(*mcpwm_fault_handle_t* fault, const *mcpwm_fault_event_data_t* *edata, void *user_ctx)

MCPWM fault event callback function.

Param fault MCPWM fault handle
Param edata MCPWM fault event data, fed by driver
Param user_ctx User data, set in `mcpwm_fault_register_event_callbacks()`
Return whether a task switch is needed after the callback returns

typedef bool (***mcpwm_compare_event_cb_t**)(*mcpwm_cmpr_handle_t* comparator, const *mcpwm_compare_event_data_t* *edata, void *user_ctx)

MCPWM comparator event callback function.

Param comparator MCPWM comparator handle
Param edata MCPWM comparator event data, fed by driver
Param user_ctx User data, set in `mcpwm_comparator_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

typedef bool (***mcpwm_capture_event_cb_t**)(*mcpwm_cap_channel_handle_t* cap_channel, const *mcpwm_capture_event_data_t* *edata, void *user_ctx)

MCPWM capture event callback function.

Param cap_channel MCPWM capture channel handle
Param edata MCPWM capture event data, fed by driver
Param user_ctx User data, set in `mcpwm_capture_channel_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

Header File

- components/hal/include/hal/mcpwm_types.h
- This header file can be included with:

```
#include "hal/mcpwm_types.h"
```

Type Definitions

typedef *soc_periph_mcpwm_timer_clk_src_t* **mcpwm_timer_clock_source_t**

MCPWM timer clock source.

typedef *soc_periph_mcpwm_capture_clk_src_t* **mcpwm_capture_clock_source_t**

MCPWM capture clock source.

typedef *soc_periph_mcpwm_carrier_clk_src_t* **mcpwm_carrier_clock_source_t**

MCPWM carrier clock source.

Enumerations

enum **mcpwm_timer_direction_t**

MCPWM timer count direction.

Values:

enumerator **MCPWM_TIMER_DIRECTION_UP**

Counting direction: Increase

enumerator **MCPWM_TIMER_DIRECTION_DOWN**

Counting direction: Decrease

enum **mcpwm_timer_event_t**

MCPWM timer events.

Values:

enumerator **MCPWM_TIMER_EVENT_EMPTY**

MCPWM timer counts to zero (i.e. counter is empty)

enumerator **MCPWM_TIMER_EVENT_FULL**

MCPWM timer counts to peak (i.e. counter is full)

enumerator **MCPWM_TIMER_EVENT_INVALID**

MCPWM timer invalid event

enum **mcpwm_timer_count_mode_t**

MCPWM timer count modes.

Values:

enumerator **MCPWM_TIMER_COUNT_MODE_PAUSE**

MCPWM timer paused

enumerator **MCPWM_TIMER_COUNT_MODE_UP**

MCPWM timer counting up

enumerator **MCPWM_TIMER_COUNT_MODE_DOWN**

MCPWM timer counting down

enumerator **MCPWM_TIMER_COUNT_MODE_UP_DOWN**

MCPWM timer counting up and down

enum **mcpwm_timer_start_stop_cmd_t**

MCPWM timer commands, specify the way to start or stop the timer.

Values:

enumerator **MCPWM_TIMER_STOP_EMPTY**

MCPWM timer stops when next count reaches zero

enumerator **MCPWM_TIMER_STOP_FULL**

MCPWM timer stops when next count reaches peak

enumerator **MCPWM_TIMER_START_NO_STOP**

MCPWM timer starts couting, and don't stop until received stop command

enumerator **MCPWM_TIMER_START_STOP_EMPTY**

MCPWM timer starts counting and stops when next count reaches zero

enumerator **MCPWM_TIMER_START_STOP_FULL**

MCPWM timer starts counting and stops when next count reaches peak

enum **mcpwm_generator_action_t**

MCPWM generator actions.

Values:

enumerator **MCPWM_GEN_ACTION_KEEP**

Generator action: Keep the same level

enumerator **MCPWM_GEN_ACTION_LOW**

Generator action: Force to low level

enumerator **MCPWM_GEN_ACTION_HIGH**

Generator action: Force to high level

enumerator **MCPWM_GEN_ACTION_TOGGLE**

Generator action: Toggle level

enum **mcpwm_operator_brake_mode_t**

MCPWM operator brake mode.

Values:

enumerator **MCPWM_OPER_BRAKE_MODE_CBC**

Brake mode: CBC (cycle by cycle)

enumerator **MCPWM_OPER_BRAKE_MODE_OST**

Brake mode: OST (one shot)

enumerator **MCPWM_OPER_BRAKE_MODE_INVALID**

MCPWM operator invalid brake mode

enum **mcpwm_capture_edge_t**

MCPWM capture edge.

Values:

enumerator **MCPWM_CAP_EDGE_POS**

Capture on the positive edge

enumerator **MCPWM_CAP_EDGE_NEG**

Capture on the negative edge

enum **mcpwm_comparator_etm_event_type_t**

MCPWM comparator specific events that supported by the ETM module.

Values:

enumerator **MCPWM_CMPR_ETM_EVENT_EQUAL**

The count value equals the value of comparator

enumerator **MCPWM_CMPR_ETM_EVENT_MAX**

Maximum number of comparator events

2.6.15 Pulse Counter (PCNT)

Introduction

The PCNT (Pulse Counter) module is designed to count the number of rising and/or falling edges of input signals. The ESP32-S3 contains multiple pulse counter units in the module.[1] Each unit is in effect an independent counter with multiple channels, where each channel can increment/decrement the counter on a rising/falling edge. Furthermore, each channel can be configured separately.

PCNT channels can react to signals of **edge** type and **level** type, however for simple applications, detecting the edge signal is usually sufficient. PCNT channels can be configured react to both pulse edges (i.e., rising and falling edge), and can be configured to increase, decrease or do nothing to the unit's counter on each edge. The level signal is the so-called **control signal**, which is used to control the counting mode of the edge signals that are attached to the same channel. By combining the usage of both edge and level signals, a PCNT unit can act as a **quadrature decoder**.

Besides that, PCNT unit is equipped with a separate glitch filter, which is helpful to remove noise from the signal.

Typically, a PCNT module can be used in scenarios like:

- Calculate periodic signal's frequency by counting the pulse numbers within a time slice
- Decode quadrature signals into speed and direction

[1] Different ESP chip series might have different number of PCNT units and channels. Please refer to the [TRM] for details. The driver does not forbid you from applying for more PCNT units and channels, but it returns error when all available hardware resources are used up. Please always check the return value when doing resource allocation (e.g., `pcnt_new_unit()`).

Functional Overview

Description of the PCNT functionality is divided into the following sections:

- *Resource Allocation* - covers how to allocate PCNT units and channels with properly set of configurations. It also covers how to recycle the resources when they finished working.
- *Set Up Channel Actions* - covers how to configure the PCNT channel to behave on different signal edges and levels.
- *Watch Points* - describes how to configure PCNT watch points (i.e., tell PCNT unit to trigger an event when the count reaches a certain value).
- *Register Event Callbacks* - describes how to hook your specific code to the watch point event callback function.
- *Set Glitch Filter* - describes how to enable and set the timing parameters for the internal glitch filter.
- *Enable and Disable Unit* - describes how to enable and disable the PCNT unit.
- *Unit IO Control* - describes IO control functions of PCNT unit, like enable glitch filter, start and stop unit, get and clear count value.
- *Power Management* - describes what functionality will prevent the chip from going into low power mode.
- *IRAM Safe* - describes tips on how to make the PCNT interrupt and IO control functions work better along with a disabled cache.
- *Thread Safety* - lists which APIs are guaranteed to be thread safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can be used to make a different effect on driver behavior.

Resource Allocation The PCNT unit and channel are represented by `pcnt_unit_handle_t` and `pcnt_channel_handle_t` respectively. All available units and channels are maintained by the driver in a resource pool, so you do not need to know the exact underlying instance ID.

Install PCNT Unit To install a PCNT unit, there is a configuration structure that needs to be given in advance: `pcnt_unit_config_t`:

- `pcnt_unit_config_t::low_limit` and `pcnt_unit_config_t::high_limit` specify the range for the internal hardware counter. The counter will reset to zero automatically when it crosses either the high or low limit.
- `pcnt_unit_config_t::accum_count` sets whether to create an internal accumulator for the counter. This is helpful when you want to extend the counter's width, which by default is 16 bit at most, defined in the hardware. See also *Compensate Overflow Loss* for how to use this feature to compensate the overflow loss.
- `pcnt_unit_config_t::intr_priority` sets the priority of the interrupt. If it is set to `0`, the driver will allocate an interrupt with a default priority. Otherwise, the driver will use the given priority.

Note: Since all PCNT units share the same interrupt source, when installing multiple PCNT units make sure that the interrupt priority `pcnt_unit_config_t::intr_priority` is the same for each unit.

Unit allocation and initialization is done by calling a function `pcnt_new_unit()` with `pcnt_unit_config_t` as an input parameter. The function will return a PCNT unit handle only when it runs correctly. Specifically, when there are no more free PCNT units in the pool (i.e., unit resources have been used up), then this function will return `ESP_ERR_NOT_FOUND` error. The total number of available PCNT units is recorded by `SOC_PCNT_UNITS_PER_GROUP` for reference.

If a previously created PCNT unit is no longer needed, it is recommended to recycle the resource by calling `pcnt_del_unit()`. Which in return allows the underlying unit hardware to be used for other purposes. Before deleting a PCNT unit, one should ensure the following prerequisites:

- The unit is in the init state, in other words, the unit is either disabled by `pcnt_unit_disable()` or not enabled yet.
- The attached PCNT channels are all removed by `pcnt_del_channel()`.

```
#define EXAMPLE_PCNT_HIGH_LIMIT 100
#define EXAMPLE_PCNT_LOW_LIMIT  -100

pcnt_unit_config_t unit_config = {
    .high_limit = EXAMPLE_PCNT_HIGH_LIMIT,
    .low_limit = EXAMPLE_PCNT_LOW_LIMIT,
};
pcnt_unit_handle_t pcnt_unit = NULL;
ESP_ERROR_CHECK(pcnt_new_unit(&unit_config, &pcnt_unit));
```

Install PCNT Channel To install a PCNT channel, you must initialize a *pcnt_chan_config_t* structure in advance, and then call *pcnt_new_channel()*. The configuration fields of the *pcnt_chan_config_t* structure are described below:

- *pcnt_chan_config_t::edge_gpio_num* and *pcnt_chan_config_t::level_gpio_num* specify the GPIO numbers used by **edge** type signal and **level** type signal. Please note, either of them can be assigned to `-1` if it is not actually used, and thus it will become a **virtual IO**. For some simple pulse counting applications where one of the level/edge signals is fixed (i.e., never changes), you can reclaim a GPIO by setting the signal as a virtual IO on channel allocation. Setting the level/edge signal as a virtual IO causes that signal to be internally routed to a fixed High/Low logic level, thus allowing you to save a GPIO for other purposes.
- *pcnt_chan_config_t::virt_edge_io_level* and *pcnt_chan_config_t::virt_level_io_level* specify the virtual IO level for **edge** and **level** input signal, to ensure a deterministic state for such control signal. Please note, they are only valid when either *pcnt_chan_config_t::edge_gpio_num* or *pcnt_chan_config_t::level_gpio_num* is assigned to `-1`.
- *pcnt_chan_config_t::invert_edge_input* and *pcnt_chan_config_t::invert_level_input* are used to decide whether to invert the input signals before they going into PCNT hardware. The invert is done by GPIO matrix instead of PCNT hardware.
- *pcnt_chan_config_t::io_loop_back* is for debug only, which enables both the GPIO's input and output paths. This can help to simulate the pulse signals by function *gpio_set_level()* on the same GPIO.

Channel allocating and initialization is done by calling a function *pcnt_new_channel()* with the above *pcnt_chan_config_t* as an input parameter plus a PCNT unit handle returned from *pcnt_new_unit()*. This function will return a PCNT channel handle if it runs correctly. Specifically, when there are no more free PCNT channel within the unit (i.e., channel resources have been used up), then this function will return *ESP_ERR_NOT_FOUND* error. The total number of available PCNT channels within the unit is recorded by *SOC_PCNT_CHANNELS_PER_UNIT* for reference. Note that, when install a PCNT channel for a specific unit, one should ensure the unit is in the init state, otherwise this function will return *ESP_ERR_INVALID_STATE* error.

If a previously created PCNT channel is no longer needed, it is recommended to recycle the resources by calling *pcnt_del_channel()*. Which in return allows the underlying channel hardware to be used for other purposes.

```
#define EXAMPLE_CHAN_GPIO_A 0
#define EXAMPLE_CHAN_GPIO_B 2

pcnt_chan_config_t chan_config = {
    .edge_gpio_num = EXAMPLE_CHAN_GPIO_A,
    .level_gpio_num = EXAMPLE_CHAN_GPIO_B,
};
pcnt_channel_handle_t pcnt_chan = NULL;
ESP_ERROR_CHECK(pcnt_new_channel(pcnt_unit, &chan_config, &pcnt_chan));
```

Set Up Channel Actions The PCNT will increase/decrease/hold its internal count value when the input pulse signal toggles. You can set different actions for edge signal and/or level signal.

- *pcnt_channel_set_edge_action()* function is to set specific actions for rising and falling edge of the signal attached to the *pcnt_chan_config_t::edge_gpio_num*. Supported actions are listed in

pcnt_channel_edge_action_t.

- *pcnt_channel_set_level_action()* function is to set specific actions for high and low level of the signal attached to the *pcnt_chan_config_t::level_gpio_num*. Supported actions are listed in *pcnt_channel_level_action_t*. This function is not mandatory if the *pcnt_chan_config_t::level_gpio_num* is set to `-1` when allocating PCNT channel by *pcnt_new_channel()*.

```
// decrease the counter on rising edge, increase the counter on falling edge
ESP_ERROR_CHECK(pcnt_channel_set_edge_action(pcnt_chan, PCNT_CHANNEL_EDGE_ACTION_
↪DECREASE, PCNT_CHANNEL_EDGE_ACTION_INCREASE));
// keep the counting mode when the control signal is high level, and reverse the␣
↪counting mode when the control signal is low level
ESP_ERROR_CHECK(pcnt_channel_set_level_action(pcnt_chan, PCNT_CHANNEL_LEVEL_ACTION_
↪KEEP, PCNT_CHANNEL_LEVEL_ACTION_INVERSE));
```

Watch Points Each PCNT unit can be configured to watch several different values that you are interested in. The value to be watched is also called **Watch Point**. The watch point itself can not exceed the range set in *pcnt_unit_config_t* by *pcnt_unit_config_t::low_limit* and *pcnt_unit_config_t::high_limit*. When the counter reaches either watch point, a watch event will be triggered and notify you by interrupt if any watch event callback has ever registered in *pcnt_unit_register_event_callbacks()*. See *Register Event Callbacks* for how to register event callbacks.

The watch point can be added and removed by *pcnt_unit_add_watch_point()* and *pcnt_unit_remove_watch_point()*. The commonly-used watch points are: **zero cross**, **maximum/minimum count** and other threshold values. The number of available watch point is limited, *pcnt_unit_add_watch_point()* will return error *ESP_ERR_NOT_FOUND* if it can not find any free hardware resource to save the watch point. You can not add the same watch point for multiple times, otherwise it will return error *ESP_ERR_INVALID_STATE*.

It is recommended to remove the unused watch point by *pcnt_unit_remove_watch_point()* to recycle the watch point resources.

```
// add zero across watch point
ESP_ERROR_CHECK(pcnt_unit_add_watch_point(pcnt_unit, 0));
// add high limit watch point
ESP_ERROR_CHECK(pcnt_unit_add_watch_point(pcnt_unit, EXAMPLE_PCNT_HIGH_LIMIT));
```

Note: Due to the hardware limitation, after adding a watch point, you should call *pcnt_unit_clear_count()* to make it take effect.

Register Event Callbacks When PCNT unit reaches any enabled watch point, specific event will be generated and notify the CPU by interrupt. If you have some function that want to get executed when event happens, you should hook your function to the interrupt service routine by calling *pcnt_unit_register_event_callbacks()*. All supported event callbacks are listed in the *pcnt_event_callbacks_t*:

- *pcnt_event_callbacks_t::on_reach* sets a callback function for watch point event. As this function is called within the ISR context, you must ensure that the function does not attempt to block (e.g., by making sure that only FreeRTOS APIs with `ISR` suffix are called from within the function). The function prototype is declared in *pcnt_watch_cb_t*.

You can save their own context to *pcnt_unit_register_event_callbacks()* as well, via the parameter `user_ctx`. This user data will be directly passed to the callback functions.

In the callback function, the driver will fill in the event data of specific event. For example, the watch point event data is declared as *pcnt_watch_event_data_t*:

- *pcnt_watch_event_data_t::watch_point_value* saves the watch point value that triggers the event.

- *pcnt_watch_event_data_t::zero_cross_mode* saves how the PCNT unit crosses the zero point in the latest time. The possible zero cross modes are listed in the *pcnt_unit_zero_cross_mode_t*. Usually different zero cross mode means different **counting direction** and **counting step size**.

Registering callback function results in lazy installation of interrupt service, thus this function should only be called before the unit is enabled by *pcnt_unit_enable()*. Otherwise, it can return *ESP_ERR_INVALID_STATE* error.

```
static bool example_pcnt_on_reach(pcnt_unit_handle_t unit, const pcnt_watch_event_
↪data_t *edata, void *user_ctx)
{
    BaseType_t high_task_wakeup;
    QueueHandle_t queue = (QueueHandle_t)user_ctx;
    // send watch point to queue, from this interrupt callback
    xQueueSendFromISR(queue, &(edata->watch_point_value), &high_task_wakeup);
    // return whether a high priority task has been waken up by this function
    return (high_task_wakeup == pdTRUE);
}

pcnt_event_callbacks_t cbs = {
    .on_reach = example_pcnt_on_reach,
};
QueueHandle_t queue = xQueueCreate(10, sizeof(int));
ESP_ERROR_CHECK(pcnt_unit_register_event_callbacks(pcnt_unit, &cbs, queue));
```

Set Glitch Filter The PCNT unit features filters to ignore possible short glitches in the signals. The parameters that can be configured for the glitch filter are listed in *pcnt_glitch_filter_config_t*:

- *pcnt_glitch_filter_config_t::max_glitch_ns* sets the maximum glitch width, in nano seconds. If a signal pulse's width is smaller than this value, then it will be treated as noise and will not increase/decrease the internal counter.

You can enable the glitch filter for PCNT unit by calling *pcnt_unit_set_glitch_filter()* with the filter configuration provided above. Particularly, you can disable the glitch filter later by calling *pcnt_unit_set_glitch_filter()* with a `NULL` filter configuration.

This function should be called when the unit is in the init state. Otherwise, it will return *ESP_ERR_INVALID_STATE* error.

Note: The glitch filter is clocked from APB. For the counter not to miss any pulses, the maximum glitch width should be longer than one APB_CLK cycle (usually 12.5 ns if APB equals 80 MHz). As the APB frequency would be changed after DFS (Dynamic Frequency Scaling) enabled, which means the filter does not work as expect in that case. So the driver installs a PM lock for PCNT unit during the first time you enable the glitch filter. For more information related to power management strategy used in PCNT driver, please see *Power Management*.

```
pcnt_glitch_filter_config_t filter_config = {
    .max_glitch_ns = 1000,
};
ESP_ERROR_CHECK(pcnt_unit_set_glitch_filter(pcnt_unit, &filter_config));
```

Enable and Disable Unit Before doing IO control to the PCNT unit, you need to enable it first, by calling *pcnt_unit_enable()*. Internally, this function:

- switches the PCNT driver state from **init** to **enable**.
- enables the interrupt service if it has been lazy installed in *pcnt_unit_register_event_callbacks()*.
- acquires a proper power management lock if it has been lazy installed in *pcnt_unit_set_glitch_filter()*. See also *Power Management* for more information.

On the contrary, calling *pcnt_unit_disable()* will do the opposite, that is, put the PCNT driver back to the **init** state, disable the interrupts service and release the power management lock.

Unit IO Control

Start/Stop and Clear Calling *pcnt_unit_start()* makes the PCNT unit start to work, increase or decrease counter according to pulse signals. On the contrary, calling *pcnt_unit_stop()* will stop the PCNT unit but retain current count value. Instead, clearing counter can only be done by calling *pcnt_unit_clear_count()*.

Note, *pcnt_unit_start()* and *pcnt_unit_stop()* should be called when the unit has been enabled by *pcnt_unit_enable()*. Otherwise, it will return *ESP_ERR_INVALID_STATE* error.

Get Count Value You can read current count value at any time by calling *pcnt_unit_get_count()*. The returned count value is a **signed** integer, where the sign can be used to reflect the direction.

```
int pulse_count = 0;
ESP_ERROR_CHECK(pcnt_unit_get_count(pcnt_unit, &pulse_count));
```

Compensate Overflow Loss The internal hardware counter will be cleared to zero automatically when it reaches high or low limit. If you want to compensate for that count loss and extend the counter's bit-width, you can:

1. Enable *pcnt_unit_config_t::accum_count* when installing the PCNT unit.
2. Add the high/low limit as the *Watch Points*.
3. Now, the returned count value from the *pcnt_unit_get_count()* function not only reflects the hardware's count value, but also accumulates the high/low overflow loss to it.

Note: *pcnt_unit_clear_count()* resets the accumulated count value as well.

Power Management When power management is enabled (i.e., *CONFIG_PM_ENABLE* is on), the system will adjust the APB frequency before going into light sleep, thus potentially changing the behavior of PCNT glitch filter and leading to valid signal being treated as noise.

However, the driver can prevent the system from changing APB frequency by acquiring a power management lock of type *ESP_PM_APB_FREQ_MAX*. Whenever you enable the glitch filter by *pcnt_unit_set_glitch_filter()*, the driver guarantees that the power management lock is acquired after the PCNT unit is enabled by *pcnt_unit_enable()*. Likewise, the driver releases the lock after *pcnt_unit_disable()* is called.

IRAM Safe By default, the PCNT interrupt will be deferred when the Cache is disabled for reasons like writing/erasing Flash. Thus the alarm interrupt will not get executed in time, which is not expected in a real-time application.

There is a Kconfig option *CONFIG_PCNT_ISR_IRAM_SAFE* that:

1. Enables the interrupt being serviced even when cache is disabled
2. Places all functions that used by the ISR into IRAM[2]
3. Places driver object into DRAM (in case it is mapped to PSRAM by accident)

This allows the interrupt to run while the cache is disabled but comes at the cost of increased IRAM consumption.

There is another Kconfig option *CONFIG_PCNT_CTRL_FUNC_IN_IRAM* that can put commonly used IO control functions into IRAM as well. So that these functions can also be executable when the cache is disabled. These IO control functions are as follows:

[2] *pcnt_event_callbacks_t::on_reach* callback and the functions invoked by itself should also be placed in IRAM, you need to take care of them by themselves.

- *pcnt_unit_start()*
- *pcnt_unit_stop()*
- *pcnt_unit_clear_count()*
- *pcnt_unit_get_count()*

Thread Safety The factory functions *pcnt_new_unit()* and *pcnt_new_channel()* are guaranteed to be thread safe by the driver, which means, you can call them from different RTOS tasks without protection by extra locks.

The following functions are allowed to run under ISR context, the driver uses a critical section to prevent them being called concurrently in both task and ISR.

- *pcnt_unit_start()*
- *pcnt_unit_stop()*
- *pcnt_unit_clear_count()*
- *pcnt_unit_get_count()*

Other functions that take the *pcnt_unit_handle_t* and *pcnt_channel_handle_t* as the first positional parameter, are not treated as thread safe. This means you should avoid calling them from multiple tasks.

Kconfig Options

- *CONFIG_PCNT_CTRL_FUNC_IN_IRAM* controls where to place the PCNT control functions (IRAM or Flash), see *IRAM Safe* for more information.
- *CONFIG_PCNT_ISR_IRAM_SAFE* controls whether the default ISR handler can work when cache is disabled, see *IRAM Safe* for more information.
- *CONFIG_PCNT_ENABLE_DEBUG_LOG* is used to enabled the debug log output. Enabling this option increases the firmware binary size.

Application Examples

- Decode the quadrature signals from rotary encoder: peripherals/pcnt/rotary_encoder.

API Reference

Header File

- components/esp_driver_pcnt/include/driver/pulse_cnt.h
- This header file can be included with:

```
#include "driver/pulse_cnt.h"
```

- This header file is a part of the API provided by the `esp_driver_pcnt` component. To declare that your component depends on `esp_driver_pcnt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_pcnt
```

or

```
PRIV_REQUIRES esp_driver_pcnt
```

Functions

esp_err_t **pcnt_new_unit**(const *pcnt_unit_config_t* *config, *pcnt_unit_handle_t* *ret_unit)

Create a new PCNT unit, and return the handle.

Note: The newly created PCNT unit is put in the init state.

Parameters
- **config** -- **[in]** PCNT unit configuration
- **ret_unit** -- **[out]** Returned PCNT unit handle

Returns
- ESP_OK: Create PCNT unit successfully
- ESP_ERR_INVALID_ARG: Create PCNT unit failed because of invalid argument (e.g. high/low limit value out of the range)
- ESP_ERR_NO_MEM: Create PCNT unit failed because out of memory
- ESP_ERR_NOT_FOUND: Create PCNT unit failed because all PCNT units are used up and no more free one
- ESP_FAIL: Create PCNT unit failed because of other error

esp_err_t **pcnt_del_unit**(*pcnt_unit_handle_t* unit)

Delete the PCNT unit handle.

Note: A PCNT unit can't be in the enable state when this function is invoked. See also `pcnt_unit_disable()` for how to disable a unit.

Parameters **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`

Returns
- ESP_OK: Delete the PCNT unit successfully
- ESP_ERR_INVALID_ARG: Delete the PCNT unit failed because of invalid argument
- ESP_ERR_INVALID_STATE: Delete the PCNT unit failed because the unit is not in init state or some PCNT channel is still in working
- ESP_FAIL: Delete the PCNT unit failed because of other error

esp_err_t **pcnt_unit_set_glitch_filter**(*pcnt_unit_handle_t* unit, const *pcnt_glitch_filter_config_t* *config)

Set glitch filter for PCNT unit.

Note: The glitch filter module is clocked from APB, and APB frequency can be changed during DFS, which in return make the filter out of action. So this function will lazy-install a PM lock internally when the power management is enabled. With this lock, the APB frequency won't be changed. The PM lock can be uninstalled in `pcnt_del_unit()`.

Note: This function should be called when the PCNT unit is in the init state (i.e. before calling `pcnt_unit_enable()`)

Parameters
- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **config** -- **[in]** PCNT filter configuration, set config to NULL means disabling the filter function

Returns
- ESP_OK: Set glitch filter successfully
- ESP_ERR_INVALID_ARG: Set glitch filter failed because of invalid argument (e.g. glitch width is too big)
- ESP_ERR_INVALID_STATE: Set glitch filter failed because the unit is not in the init state
- ESP_FAIL: Set glitch filter failed because of other error

esp_err_t **pcnt_unit_enable**(*pcnt_unit_handle_t* unit)

Enable the PCNT unit.

Note: This function will transit the unit state from init to enable.

Note: This function will enable the interrupt service, if it's lazy installed in `pcnt_unit_register_event_callbacks()`.

Note: This function will acquire the PM lock if it's lazy installed in `pcnt_unit_set_glitch_filter()`.

Note: Enable a PCNT unit doesn't mean to start it. See also `pcnt_unit_start()` for how to start the PCNT counter.

Parameters **`unit`** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`

Returns

- ESP_OK: Enable PCNT unit successfully
- ESP_ERR_INVALID_ARG: Enable PCNT unit failed because of invalid argument
- ESP_ERR_INVALID_STATE: Enable PCNT unit failed because the unit is already enabled
- ESP_FAIL: Enable PCNT unit failed because of other error

esp_err_t **`pcnt_unit_disable`**(*pcnt_unit_handle_t* unit)

Disable the PCNT unit.

Note: This function will do the opposite work to the `pcnt_unit_enable()`

Note: Disable a PCNT unit doesn't mean to stop it. See also `pcnt_unit_stop()` for how to stop the PCNT counter.

Parameters **`unit`** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`

Returns

- ESP_OK: Disable PCNT unit successfully
- ESP_ERR_INVALID_ARG: Disable PCNT unit failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable PCNT unit failed because the unit is not enabled yet
- ESP_FAIL: Disable PCNT unit failed because of other error

esp_err_t **`pcnt_unit_start`**(*pcnt_unit_handle_t* unit)

Start the PCNT unit, the counter will start to count according to the edge and/or level input signals.

Note: This function should be called when the unit is in the enable state (i.e. after calling `pcnt_unit_enable()`)

Note: This function is allowed to run within ISR context

Note: This function will be placed into IRAM if `CONFIG_PCNT_CTRL_FUNC_IN_IRAM` is on, so that it's allowed to be executed when Cache is disabled

Parameters **`unit`** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
Returns
- ESP_OK: Start PCNT unit successfully
- ESP_ERR_INVALID_ARG: Start PCNT unit failed because of invalid argument
- ESP_ERR_INVALID_STATE: Start PCNT unit failed because the unit is not enabled yet
- ESP_FAIL: Start PCNT unit failed because of other error

esp_err_t **`pcnt_unit_stop`**(*pcnt_unit_handle_t* unit)

Stop PCNT from counting.

Note: This function should be called when the unit is in the enable state (i.e. after calling `pcnt_unit_enable()`)

Note: The stop operation won't clear the counter. Also see `pcnt_unit_clear_count()` for how to clear pulse count value.

Note: This function is allowed to run within ISR context

Note: This function will be placed into IRAM if `CONFIG_PCNT_CTRL_FUNC_IN_IRAM`, so that it is allowed to be executed when Cache is disabled

Parameters **`unit`** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
Returns
- ESP_OK: Stop PCNT unit successfully
- ESP_ERR_INVALID_ARG: Stop PCNT unit failed because of invalid argument
- ESP_ERR_INVALID_STATE: Stop PCNT unit failed because the unit is not enabled yet
- ESP_FAIL: Stop PCNT unit failed because of other error

esp_err_t **`pcnt_unit_clear_count`**(*pcnt_unit_handle_t* unit)

Clear PCNT pulse count value to zero.

Note: It's recommended to call this function after adding a watch point by `pcnt_unit_add_watch_point()`, so that the newly added watch point is effective immediately.

Note: This function is allowed to run within ISR context

Note: This function will be placed into IRAM if `CONFIG_PCNT_CTRL_FUNC_IN_IRAM`, so that it's allowed to be executed when Cache is disabled

Parameters **`unit`** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
Returns
- ESP_OK: Clear PCNT pulse count successfully

- ESP_ERR_INVALID_ARG: Clear PCNT pulse count failed because of invalid argument
- ESP_FAIL: Clear PCNT pulse count failed because of other error

esp_err_t **pcnt_unit_get_count** (*pcnt_unit_handle_t* unit, int *value)

Get PCNT count value.

Note: This function is allowed to run within ISR context

Note: This function will be placed into IRAM if `CONFIG_PCNT_CTRL_FUNC_IN_IRAM`, so that it's allowed to be executed when Cache is disabled

Parameters

- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **value** -- **[out]** Returned count value

Returns

- ESP_OK: Get PCNT pulse count successfully
- ESP_ERR_INVALID_ARG: Get PCNT pulse count failed because of invalid argument
- ESP_FAIL: Get PCNT pulse count failed because of other error

esp_err_t **pcnt_unit_register_event_callbacks** (*pcnt_unit_handle_t* unit, const *pcnt_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for PCNT unit.

Note: User registered callbacks are expected to be runnable within ISR context

Note: The first call to this function needs to be before the call to `pcnt_unit_enable`

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Parameters

- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **cbs** -- **[in]** Group of callback functions
- **user_data** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_ERR_INVALID_STATE: Set event callbacks failed because the unit is not in init state
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **pcnt_unit_add_watch_point** (*pcnt_unit_handle_t* unit, int watch_point)

Add a watch point for PCNT unit, PCNT will generate an event when the counter value reaches the watch point value.

Parameters

- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **watch_point** -- **[in]** Value to be watched

Returns

- ESP_OK: Add watch point successfully

- ESP_ERR_INVALID_ARG: Add watch point failed because of invalid argument (e.g. the value to be watched is out of the limitation set in *pcnt_unit_config_t*)
- ESP_ERR_INVALID_STATE: Add watch point failed because the same watch point has already been added
- ESP_ERR_NOT_FOUND: Add watch point failed because no more hardware watch point can be configured
- ESP_FAIL: Add watch point failed because of other error

esp_err_t **pcnt_unit_remove_watch_point** (*pcnt_unit_handle_t* unit, int watch_point)

Remove a watch point for PCNT unit.

Parameters
- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **watch_point** -- **[in]** Watch point value

Returns
- ESP_OK: Remove watch point successfully
- ESP_ERR_INVALID_ARG: Remove watch point failed because of invalid argument
- ESP_ERR_INVALID_STATE: Remove watch point failed because the watch point was not added by `pcnt_unit_add_watch_point()` yet
- ESP_FAIL: Remove watch point failed because of other error

esp_err_t **pcnt_new_channel** (*pcnt_unit_handle_t* unit, const *pcnt_chan_config_t* *config, *pcnt_channel_handle_t* *ret_chan)

Create PCNT channel for specific unit, each PCNT has several channels associated with it.

Note: This function should be called when the unit is in init state (i.e. before calling `pcnt_unit_enable()`)

Parameters
- **unit** -- **[in]** PCNT unit handle created by `pcnt_new_unit()`
- **config** -- **[in]** PCNT channel configuration
- **ret_chan** -- **[out]** Returned channel handle

Returns
- ESP_OK: Create PCNT channel successfully
- ESP_ERR_INVALID_ARG: Create PCNT channel failed because of invalid argument
- ESP_ERR_NO_MEM: Create PCNT channel failed because of insufficient memory
- ESP_ERR_NOT_FOUND: Create PCNT channel failed because all PCNT channels are used up and no more free one
- ESP_ERR_INVALID_STATE: Create PCNT channel failed because the unit is not in the init state
- ESP_FAIL: Create PCNT channel failed because of other error

esp_err_t **pcnt_del_channel** (*pcnt_channel_handle_t* chan)

Delete the PCNT channel.

Parameters **chan** -- **[in]** PCNT channel handle created by `pcnt_new_channel()`

Returns
- ESP_OK: Delete the PCNT channel successfully
- ESP_ERR_INVALID_ARG: Delete the PCNT channel failed because of invalid argument
- ESP_FAIL: Delete the PCNT channel failed because of other error

esp_err_t **pcnt_channel_set_edge_action** (*pcnt_channel_handle_t* chan, *pcnt_channel_edge_action_t* pos_act, *pcnt_channel_edge_action_t* neg_act)

Set channel actions when edge signal changes (e.g. falling or rising edge occurred). The edge signal is input from the `edge_gpio_num` configured in *pcnt_chan_config_t*. We use these actions to control when and how to change the counter value.

Parameters

- **chan** -- **[in]** PCNT channel handle created by `pcnt_new_channel()`
- **pos_act** -- **[in]** Action on posedge signal
- **neg_act** -- **[in]** Action on negedge signal

Returns

- ESP_OK: Set edge action for PCNT channel successfully
- ESP_ERR_INVALID_ARG: Set edge action for PCNT channel failed because of invalid argument
- ESP_FAIL: Set edge action for PCNT channel failed because of other error

esp_err_t **pcnt_channel_set_level_action** (*pcnt_channel_handle_t* chan, *pcnt_channel_level_action_t* high_act, *pcnt_channel_level_action_t* low_act)

Set channel actions when level signal changes (e.g. signal level goes from high to low). The level signal is input from the `level_gpio_num` configured in *pcnt_chan_config_t*. We use these actions to control when and how to change the counting mode.

Parameters

- **chan** -- **[in]** PCNT channel handle created by `pcnt_new_channel()`
- **high_act** -- **[in]** Action on high level signal
- **low_act** -- **[in]** Action on low level signal

Returns

- ESP_OK: Set level action for PCNT channel successfully
- ESP_ERR_INVALID_ARG: Set level action for PCNT channel failed because of invalid argument
- ESP_FAIL: Set level action for PCNT channel failed because of other error

Structures

struct **pcnt_watch_event_data_t**

PCNT watch event data.

Public Members

int **watch_point_value**

Watch point value that triggered the event

pcnt_unit_zero_cross_mode_t **zero_cross_mode**

Zero cross mode

struct **pcnt_event_callbacks_t**

Group of supported PCNT callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_PCNT_ISR_IRAM_SAFE is enabled, the callback itself and functions callbed by it should be placed in IRAM.

Public Members

pcnt_watch_cb_t **on_reach**

Called when PCNT unit counter reaches any watch point

struct **pcnt_unit_config_t**

PCNT unit configuration.

Public Members

int **low_limit**

Low limitation of the count unit, should be lower than 0

int **high_limit**

High limitation of the count unit, should be higher than 0

int **intr_priority**

PCNT interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **accum_count**

Whether to accumulate the count value when overflows at the high/low limit

struct *pcnt_unit_config_t*::[anonymous] **flags**

Extra flags

struct **pcnt_chan_config_t**

PCNT channel configuration.

Public Members

int **edge_gpio_num**

GPIO number used by the edge signal, input mode with pull up enabled. Set to -1 if unused

int **level_gpio_num**

GPIO number used by the level signal, input mode with pull up enabled. Set to -1 if unused

uint32_t **invert_edge_input**

Invert the input edge signal

uint32_t **invert_level_input**

Invert the input level signal

uint32_t **virt_edge_io_level**

Virtual edge IO level, 0: low, 1: high. Only valid when edge_gpio_num is set to -1

uint32_t **virt_level_io_level**

Virtual level IO level, 0: low, 1: high. Only valid when level_gpio_num is set to -1

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

struct *pcnt_chan_config_t*::[anonymous] **flags**

Channel config flags

struct **pcnt_glitch_filter_config_t**

PCNT glitch filter configuration.

Public Members

uint32_t **max_glitch_ns**

Pulse width smaller than this threshold will be treated as glitch and ignored, in the unit of ns

Type Definitions

typedef struct pcnt_unit_t ***pcnt_unit_handle_t**

Type of PCNT unit handle.

typedef struct pcnt_chan_t ***pcnt_channel_handle_t**

Type of PCNT channel handle.

typedef bool (***pcnt_watch_cb_t**)(*pcnt_unit_handle_t* unit, const *pcnt_watch_event_data_t* *edata, void *user_ctx)

PCNT watch event callback prototype.

Note: The callback function is invoked from an ISR context, so it should meet the restrictions of not calling any blocking APIs when implementing the callback. e.g. must use ISR version of FreeRTOS APIs.

Param unit **[in]** PCNT unit handle
Param edata **[in]** PCNT event data, fed by the driver
Param user_ctx **[in]** User data, passed from `pcnt_unit_register_event_callbacks()`
Return Whether a high priority task has been woken up by this function

Header File

- components/hal/include/hal/pcnt_types.h
- This header file can be included with:

```
#include "hal/pcnt_types.h"
```

Enumerations

enum **pcnt_channel_level_action_t**

PCNT channel action on control level.

Values:

enumerator **PCNT_CHANNEL_LEVEL_ACTION_KEEP**

Keep current count mode

enumerator **PCNT_CHANNEL_LEVEL_ACTION_INVERSE**

Invert current count mode (increase -> decrease, decrease -> increase)

enumerator **PCNT_CHANNEL_LEVEL_ACTION_HOLD**

Hold current count value

enum **pcnt_channel_edge_action_t**

PCNT channel action on signal edge.

Values:

enumerator **PCNT_CHANNEL_EDGE_ACTION_HOLD**

Hold current count value

enumerator **PCNT_CHANNEL_EDGE_ACTION_INCREASE**

Increase count value

enumerator **PCNT_CHANNEL_EDGE_ACTION_DECREASE**

Decrease count value

enum **pcnt_unit_zero_cross_mode_t**

PCNT unit zero cross mode.

Values:

enumerator **PCNT_UNIT_ZERO_CROSS_POS_ZERO**

start from positive value, end to zero, i.e. +N->0

enumerator **PCNT_UNIT_ZERO_CROSS_NEG_ZERO**

start from negative value, end to zero, i.e. -N->0

enumerator **PCNT_UNIT_ZERO_CROSS_NEG_POS**

start from negative value, end to positive value, i.e. -N->+M

enumerator **PCNT_UNIT_ZERO_CROSS_POS_NEG**

start from positive value, end to negative value, i.e. +N->-M

2.6.16 Remote Control Transceiver (RMT)

Introduction

The RMT (Remote Control Transceiver) peripheral was designed to act as an infrared transceiver. However, due to the flexibility of its data format, RMT can be extended to a versatile and general-purpose transceiver, transmitting or receiving many other types of signals. From the perspective of network layering, the RMT hardware contains both physical and data link layers. The physical layer defines the communication media and bit signal representation. The data link layer defines the format of an RMT frame. The minimal data unit in the frame is called the **RMT symbol**, which is represented by *rmt_symbol_word_t* in the driver.

ESP32-S3 contains multiple channels in the RMT peripheral[1]. Each channel can be independently configured as either transmitter or receiver.

Typically, the RMT peripheral can be used in the following scenarios:

[1] Different ESP chip series might have different numbers of RMT channels. Please refer to [TRM] for details. The driver does not forbid you from applying for more RMT channels, but it returns an error when there are no hardware resources available. Please always check the return value when doing *Resource Allocation*.

- Transmit or receive infrared signals, with any IR protocols, e.g., NEC
- General-purpose sequence generator
- Transmit signals in a hardware-controlled loop, with a finite or infinite number of times
- Multi-channel simultaneous transmission
- Modulate the carrier to the output signal or demodulate the carrier from the input signal

Layout of RMT Symbols The RMT hardware defines data in its own pattern -- the **RMT symbol**. The diagram below illustrates the bit fields of an RMT symbol. Each symbol consists of two pairs of two values. The first value in the pair is a 15-bit value representing the signal's duration in units of RMT ticks. The second in the pair is a 1-bit value representing the signal's logic level, i.e., high or low.

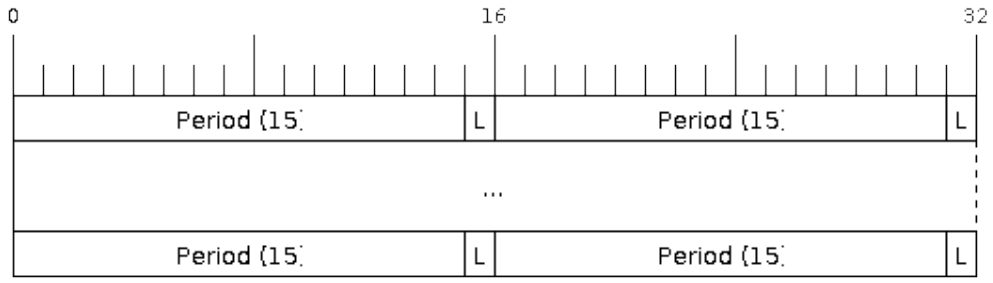


Fig. 19: Structure of RMT symbols (L - signal level)

RMT Transmitter Overview The data path and control path of an RMT TX channel is illustrated in the figure below:

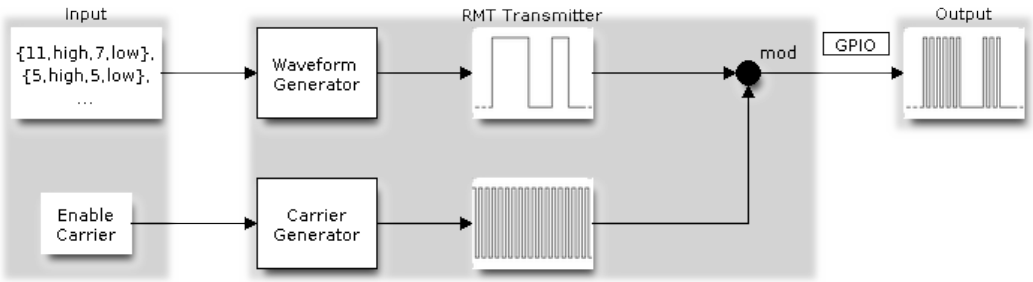


Fig. 20: RMT Transmitter Overview

The driver encodes the user's data into RMT data format, then the RMT transmitter can generate the waveforms according to the encoding artifacts. It is also possible to modulate a high-frequency carrier signal before being routed to a GPIO pad.

RMT Receiver Overview The data path and control path of an RMT RX channel is illustrated in the figure below:

The RMT receiver can sample incoming signals into RMT data format, and store the data in memory. It is also possible to tell the receiver the basic characteristics of the incoming signal, so that the signal's stop condition can be recognized, and signal glitches and noise can be filtered out. The RMT peripheral also supports demodulating the high-frequency carrier from the base signal.

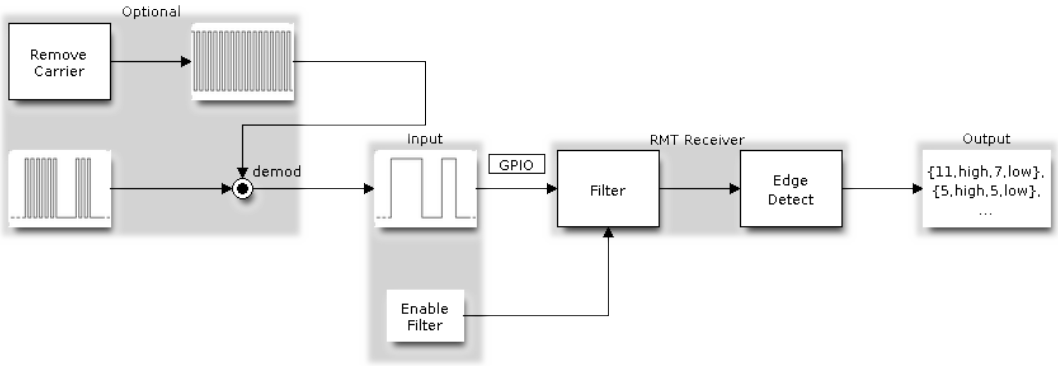


Fig. 21: RMT Receiver Overview

Functional Overview

The description of the RMT functionality is divided into the following sections:

- *Resource Allocation* - covers how to allocate and properly configure RMT channels. It also covers how to recycle channels and other resources when they are no longer used.
- *Carrier Modulation and Demodulation* - describes how to modulate and demodulate the carrier signals for TX and RX channels respectively.
- *Register Event Callbacks* - covers how to register user-provided event callbacks to receive RMT channel events.
- *Enable and Disable Channel* - shows how to enable and disable the RMT channel.
- *Initiate TX Transaction* - describes the steps to initiate a transaction for a TX channel.
- *Initiate RX Transaction* - describes the steps to initiate a transaction for an RX channel.
- *Multiple Channels Simultaneous Transmission* - describes how to collect multiple channels into a sync group so that their transmissions can be started simultaneously.
- *RMT Encoder* - focuses on how to write a customized encoder by combining multiple primitive encoders that are provided by the driver.
- *Power Management* - describes how different clock sources affects power consumption.
- *IRAM Safe* - describes how disabling the cache affects the RMT driver, and tips to mitigate it.
- *Thread Safety* - lists which APIs are guaranteed to be thread-safe by the driver.
- *Kconfig Options* - describes the various Kconfig options supported by the RMT driver.

Resource Allocation Both RMT TX and RX channels are represented by `rmt_channel_handle_t` in the driver. The driver internally manages which channels are available and hands out a free channel on request.

Install RMT TX Channel To install an RMT TX channel, there is a configuration structure that needs to be given in advance `rmt_tx_channel_config_t`. The following list describes each member of the configuration structure.

- `rmt_tx_channel_config_t::gpio_num` sets the GPIO number used by the transmitter.
- `rmt_tx_channel_config_t::clk_src` selects the source clock for the RMT channel. The available clocks are listed in `rmt_clock_source_t`. Note that, the selected clock is also used by other channels, which means the user should ensure this configuration is the same when allocating other channels, regardless of TX or RX. For the effect on the power consumption of different clock sources, please refer to the *Power Management* section.
- `rmt_tx_channel_config_t::resolution_hz` sets the resolution of the internal tick counter. The timing parameter of the RMT signal is calculated based on this **tick**.
- `rmt_tx_channel_config_t::mem_block_symbols` has a slightly different meaning based on if the DMA backend is enabled or not.
  - If the DMA is enabled via `rmt_tx_channel_config_t::with_dma`, then this field controls the size of the internal DMA buffer. To achieve a better throughput and smaller CPU overhead, you can set

a larger value, e.g., `1024`.
  - If DMA is not used, this field controls the size of the dedicated memory block owned by the channel, which should be at least 48.
- *rmt_tx_channel_config_t::trans_queue_depth* sets the depth of the internal transaction queue, the deeper the queue, the more transactions can be prepared in the backlog.
- *rmt_tx_channel_config_t::invert_out* is used to decide whether to invert the RMT signal before sending it to the GPIO pad.
- *rmt_tx_channel_config_t::with_dma* enables the DMA backend for the channel. Using the DMA allows a significant amount of the channel's workload to be offloaded from the CPU. However, the DMA backend is not available on all ESP chips, please refer to [TRM] before you enable this option. Or you might encounter a *ESP_ERR_NOT_SUPPORTED* error.
- *rmt_tx_channel_config_t::io_loop_back* enables both input and output capabilities on the channel's assigned GPIO. Thus, by binding a TX and RX channel to the same GPIO, loopback can be achieved.
- *rmt_tx_channel_config_t::io_od_mode* configures the channel's assigned GPIO as open-drain. When combined with *rmt_tx_channel_config_t::io_loop_back*, a bi-directional bus (e.g., 1-wire) can be achieved.
- *rmt_tx_channel_config_t::intr_priority* Set the priority of the interrupt. If set to `0` , then the driver will use a interrupt with low or medium priority (priority level may be one of 1,2 or 3), otherwise use the priority indicated by *rmt_tx_channel_config_t::intr_priority*. Please use the number form (1,2,3) , not the bitmask form ((1<<1),(1<<2),(1<<3)). Please pay attention that once the interrupt priority is set, it cannot be changed until *rmt_del_channel()* is called.

Once the *rmt_tx_channel_config_t* structure is populated with mandatory parameters, users can call *rmt_new_tx_channel()* to allocate and initialize a TX channel. This function returns an RMT channel handle if it runs correctly. Specifically, when there are no more free channels in the RMT resource pool, this function returns *ESP_ERR_NOT_FOUND* error. If some feature (e.g., DMA backend) is not supported by the hardware, it returns *ESP_ERR_NOT_SUPPORTED* error.

```
rmt_channel_handle_t tx_chan = NULL;
rmt_tx_channel_config_t tx_chan_config = {
    .clk_src = RMT_CLK_SRC_DEFAULT,   // select source clock
    .gpio_num = 0,                    // GPIO number
    .mem_block_symbols = 64,          // memory block size, 64 * 4 = 256 Bytes
    .resolution_hz = 1 * 1000 * 1000, // 1 MHz tick resolution, i.e., 1 tick = 1 µs
    .trans_queue_depth = 4,           // set the number of transactions that can␣
↪pend in the background
    .flags.invert_out = false,        // do not invert output signal
    .flags.with_dma = false,          // do not need DMA backend
};
ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_chan_config, &tx_chan));
```

Install RMT RX Channel To install an RMT RX channel, there is a configuration structure that needs to be given in advance *rmt_rx_channel_config_t*. The following list describes each member of the configuration structure.

- *rmt_rx_channel_config_t::gpio_num* sets the GPIO number used by the receiver.
- *rmt_rx_channel_config_t::clk_src* selects the source clock for the RMT channel. The available clocks are listed in *rmt_clock_source_t*. Note that, the selected clock is also used by other channels, which means the user should ensure this configuration is the same when allocating other channels, regardless of TX or RX. For the effect on the power consumption of different clock sources, please refer to the *Power Management* section.
- *rmt_rx_channel_config_t::resolution_hz* sets the resolution of the internal tick counter. The timing parameter of the RMT signal is calculated based on this **tick**.
- *rmt_rx_channel_config_t::mem_block_symbols* has a slightly different meaning based on whether the DMA backend is enabled.
  - If the DMA is enabled via *rmt_rx_channel_config_t::with_dma*, this field controls the maximum size of the DMA buffer.
  - If DMA is not used, this field controls the size of the dedicated memory block owned by the channel, which should be at least 48.

- *rmt_rx_channel_config_t::invert_in* is used to invert the input signals before it is passed to the RMT receiver. The inversion is done by the GPIO matrix instead of by the RMT peripheral.
- *rmt_rx_channel_config_t::with_dma* enables the DMA backend for the channel. Using the DMA allows a significant amount of the channel's workload to be offloaded from the CPU. However, the DMA backend is not available on all ESP chips, please refer to [TRM] before you enable this option. Or you might encounter a *ESP_ERR_NOT_SUPPORTED* error.
- *rmt_rx_channel_config_t::io_loop_back* enables both input and output capabilities on the channel's assigned GPIO. Thus, by binding a TX and RX channel to the same GPIO, loopback can be achieved.
- *rmt_rx_channel_config_t::intr_priority* Set the priority of the interrupt. If set to `0` , then the driver will use a interrupt with low or medium priority (priority level may be one of 1,2 or 3), otherwise use the priority indicated by *rmt_rx_channel_config_t::intr_priority*. Please use the number form (1,2,3) , not the bitmask form ((1<<1),(1<<2),(1<<3)). Please pay attention that once the interrupt priority is set, it cannot be changed until *rmt_del_channel()* is called.

Once the *rmt_rx_channel_config_t* structure is populated with mandatory parameters, users can call *rmt_new_rx_channel()* to allocate and initialize an RX channel. This function returns an RMT channel handle if it runs correctly. Specifically, when there are no more free channels in the RMT resource pool, this function returns *ESP_ERR_NOT_FOUND* error. If some feature (e.g., DMA backend) is not supported by the hardware, it returns *ESP_ERR_NOT_SUPPORTED* error.

```
rmt_channel_handle_t rx_chan = NULL;
rmt_rx_channel_config_t rx_chan_config = {
    .clk_src = RMT_CLK_SRC_DEFAULT,   // select source clock
    .resolution_hz = 1 * 1000 * 1000, // 1 MHz tick resolution, i.e., 1 tick = 1 µs
    .mem_block_symbols = 64,          // memory block size, 64 * 4 = 256 Bytes
    .gpio_num = 2,                    // GPIO number
    .flags.invert_in = false,         // do not invert input signal
    .flags.with_dma = false,          // do not need DMA backend
};
ESP_ERROR_CHECK(rmt_new_rx_channel(&rx_chan_config, &rx_chan));
```

Note: Due to a software limitation in the GPIO driver, when both TX and RX channels are bound to the same GPIO, ensure the RX Channel is initialized before the TX Channel. If the TX Channel was set up first, then during the RX Channel setup, the previous RMT TX Channel signal will be overridden by the GPIO control signal.

Uninstall RMT Channel If a previously installed RMT channel is no longer needed, it is recommended to recycle the resources by calling *rmt_del_channel()*, which in return allows the underlying software and hardware resources to be reused for other purposes.

Carrier Modulation and Demodulation The RMT transmitter can generate a carrier wave and modulate it onto the message signal. Compared to the message signal, the carrier signal's frequency is significantly higher. In addition, the user can only set the frequency and duty cycle for the carrier signal. The RMT receiver can demodulate the carrier signal from the incoming signal. Note that, carrier modulation and demodulation are not supported on all ESP chips, please refer to [TRM] before configuring the carrier, or you might encounter a *ESP_ERR_NOT_SUPPORTED* error.

Carrier-related configurations lie in *rmt_carrier_config_t*:

- *rmt_carrier_config_t::frequency_hz* sets the carrier frequency, in Hz.
- *rmt_carrier_config_t::duty_cycle* sets the carrier duty cycle.
- *rmt_carrier_config_t::polarity_active_low* sets the carrier polarity, i.e., on which level the carrier is applied.
- *rmt_carrier_config_t::always_on* sets whether to output the carrier even when the data transmission has finished. This configuration is only valid for the TX channel.

Note: For the RX channel, we should not set the carrier frequency exactly to the theoretical value. It is recommended to leave a tolerance for the carrier frequency. For example, in the snippet below, we set the frequency to 25 KHz,

instead of the 38 KHz configured on the TX side. The reason is that reflection and refraction occur when a signal travels through the air, leading to distortion on the receiver side.

```
rmt_carrier_config_t tx_carrier_cfg = {
    .duty_cycle = 0.33,                 // duty cycle 33%
    .frequency_hz = 38000,              // 38 KHz
    .flags.polarity_active_low = false, // carrier should be modulated to high␣
↪level
};
// modulate carrier to TX channel
ESP_ERROR_CHECK(rmt_apply_carrier(tx_chan, &tx_carrier_cfg));

rmt_carrier_config_t rx_carrier_cfg = {
    .duty_cycle = 0.33,                 // duty cycle 33%
    .frequency_hz = 25000,              // 25 KHz carrier, should be smaller than␣
↪the transmitter's carrier frequency
    .flags.polarity_active_low = false, // the carrier is modulated to high level
};
// demodulate carrier from RX channel
ESP_ERROR_CHECK(rmt_apply_carrier(rx_chan, &rx_carrier_cfg));
```

Register Event Callbacks When an event occurs on an RMT channel (e.g., transmission or receiving is completed), the CPU is notified of this event via an interrupt. If you have some function that needs to be called when a particular events occur, you can register a callback for that event to the RMT driver's ISR (Interrupt Service Routine) by calling *rmt_tx_register_event_callbacks()* and *rmt_rx_register_event_callbacks()* for TX and RX channel respectively. Since the registered callback functions are called in the interrupt context, the user should ensure the callback function does not block, e.g., by making sure that only FreeRTOS APIs with the `FromISR` suffix are called from within the function. The callback function has a boolean return value used to indicate whether a higher priority task has been unblocked by the callback.

The TX channel-supported event callbacks are listed in the *rmt_tx_event_callbacks_t*:

- *rmt_tx_event_callbacks_t::on_trans_done* sets a callback function for the "trans-done" event. The function prototype is declared in *rmt_tx_done_callback_t*.

The RX channel-supported event callbacks are listed in the *rmt_rx_event_callbacks_t*:

- *rmt_rx_event_callbacks_t::on_recv_done* sets a callback function for "receive-done" event. The function prototype is declared in *rmt_rx_done_callback_t*.

Note: The "receive-done" is not equivalent to "receive-finished". This callback can also be called at a "partial-receive-done" time, for many times during one receive transaction.

Users can save their own context in *rmt_tx_register_event_callbacks()* and *rmt_rx_register_event_callbacks()* as well, via the parameter `user_data`. The user data is directly passed to each callback function.

In the callback function, users can fetch the event-specific data that is filled by the driver in the `edata`. Note that the `edata` pointer is **only** valid during the callback, please do not try to save this pointer and use that outside of the callback function.

The TX-done event data is defined in *rmt_tx_done_event_data_t*:

- *rmt_tx_done_event_data_t::num_symbols* indicates the number of transmitted RMT symbols. This also reflects the size of the encoding artifacts. Please note, this value accounts for the `EOF` symbol as well, which is appended by the driver to mark the end of one transaction.

The RX-complete event data is defined in *rmt_rx_done_event_data_t*:

- *rmt_rx_done_event_data_t::received_symbols* points to the received RMT symbols. These symbols are saved in the `buffer` parameter of the *rmt_receive()* function. Users should not free this

receive buffer before the callback returns. If you also enabled the partial receive feature, then the user buffer will be used as a "second level buffer", where its content can be overwritten by data comes in afterwards. In this case, you should copy the received data to another place if you want to keep it or process it later.

- *rmt_rx_done_event_data_t::num_symbols* indicates the number of received RMT symbols. This value is not larger than the `buffer_size` parameter of *rmt_receive()* function. If the `buffer_size` is not sufficient to accommodate all the received RMT symbols, the driver only keeps the maximum number of symbols that the buffer can hold, and excess symbols are discarded or ignored.
- *rmt_rx_done_event_data_t::is_last* indicates whether the current received buffer is the last one in the transaction. This is useful when you enable the partial reception feature by *rmt_receive_config_t::extra_flags::en_partial_rx*.

Enable and Disable Channel *rmt_enable()* must be called in advance before transmitting or receiving RMT symbols. For TX channels, enabling a channel enables a specific interrupt and prepares the hardware to dispatch transactions. For RX channels, enabling a channel enables an interrupt, but the receiver is not started during this time, as the characteristics of the incoming signal have yet to be specified. The receiver is started in *rmt_receive()*.

rmt_disable() does the opposite by disabling the interrupt and clearing any pending interrupts. The transmitter and receiver are disabled as well.

```
ESP_ERROR_CHECK(rmt_enable(tx_chan));
ESP_ERROR_CHECK(rmt_enable(rx_chan));
```

Initiate TX Transaction RMT is a special communication peripheral, as it is unable to transmit raw byte streams like SPI and I2C. RMT can only send data in its own format *rmt_symbol_word_t*. However, the hardware does not help to convert the user data into RMT symbols, this can only be done in software by the so-called **RMT Encoder**. The encoder is responsible for encoding user data into RMT symbols and then writing to the RMT memory block or the DMA buffer. For how to create an RMT encoder, please refer to *RMT Encoder*.

Once you created an encoder, you can initiate a TX transaction by calling *rmt_transmit()*. This function takes several positional parameters like channel handle, encoder handle, and payload buffer. Besides, you also need to provide a transmission-specific configuration in *rmt_transmit_config_t*:

- *rmt_transmit_config_t::loop_count* sets the number of transmission loops. After the transmitter has finished one round of transmission, it can restart the same transmission again if this value is not set to zero. As the loop is controlled by hardware, the RMT channel can be used to generate many periodic sequences with minimal CPU intervention.
  - Setting *rmt_transmit_config_t::loop_count* to *-1* means an infinite loop transmission. In this case, the channel does not stop until *rmt_disable()* is called. The "trans-done" event is not generated as well.
  - Setting *rmt_transmit_config_t::loop_count* to a positive number means finite number of iterations. In this case, the "trans-done" event is when the specified number of iterations have completed.

  Note: The **loop transmit** feature is not supported on all ESP chips, please refer to [TRM] before you configure this option, or you might encounter *ESP_ERR_NOT_SUPPORTED* error.

- *rmt_transmit_config_t::eot_level* sets the output level when the transmitter finishes working or stops working by calling *rmt_disable()*.
- *rmt_transmit_config_t::queue_nonblocking* sets whether to wait for a free slot in the transaction queue when it is full. If this value is set to `true`, then the function will return with an error code *ESP_ERR_INVALID_STATE* when the queue is full. Otherwise, the function will block until a free slot is available in the queue.

Note: There is a limitation in the transmission size if the *rmt_transmit_config_t::loop_count* is set to non-zero, i.e., to enable the loop feature. The encoded RMT symbols should not exceed the capacity of the RMT hardware memory block size, or you might see an error message like `encoding artifacts can't exceed hw memory block for loop transmission`. If you have to start a large transaction by loop, you can try either of the following methods.

- Increase the *rmt_tx_channel_config_t::mem_block_symbols*. This approach does not work if the DMA backend is also enabled.
- Customize an encoder and construct an infinite loop in the encoding function. See also *RMT Encoder*.

Internally, *rmt_transmit()* constructs a transaction descriptor and sends it to a job queue, which is dispatched in the ISR. So it is possible that the transaction is not started yet when *rmt_transmit()* returns. You cannot recycle or modify the payload buffer until the transaction is finished. You can get the transaction completion event by registering a callback function via *rmt_tx_register_event_callbacks()*. To ensure all pending transactions to complete, you can also use *rmt_tx_wait_all_done()*.

Multiple Channels Simultaneous Transmission In some real-time control applications (e.g., to make two robotic arms move simultaneously), you do not want any time drift between different channels. The RMT driver can help to manage this by creating a so-called **Sync Manager**. The sync manager is represented by *rmt_sync_manager_handle_t* in the driver. The procedure of RMT sync transmission is shown as follows:

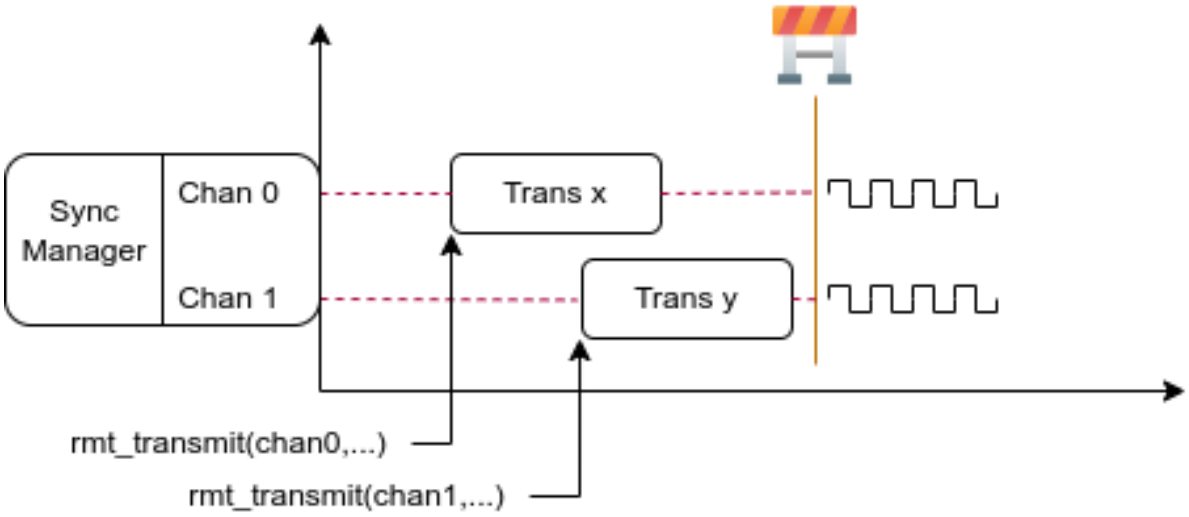


Fig. 22: RMT TX Sync

Install RMT Sync Manager To create a sync manager, the user needs to tell which channels are going to be managed in the *rmt_sync_manager_config_t*:

- *rmt_sync_manager_config_t::tx_channel_array* points to the array of TX channels to be managed.
- *rmt_sync_manager_config_t::array_size* sets the number of channels to be managed.

rmt_new_sync_manager() can return a manager handle on success. This function could also fail due to various errors such as invalid arguments, etc. Especially, when the sync manager has been installed before, and there are no hardware resources to create another manager, this function reports *ESP_ERR_NOT_FOUND* error. In addition, if the sync manager is not supported by the hardware, it reports a *ESP_ERR_NOT_SUPPORTED* error. Please refer to [TRM] before using the sync manager feature.

Start Transmission Simultaneously For any managed TX channel, it does not start the machine until *rmt_transmit()* has been called on all channels in *rmt_sync_manager_config_t::tx_channel_array*. Before that, the channel is just put in a waiting state. TX channels will usually complete their transactions at different times due to differing transactions, thus resulting in a loss of sync. So before restarting a simultaneous transmission, the user needs to call *rmt_sync_reset()* to synchronize all channels again.

Calling *rmt_del_sync_manager()* can recycle the sync manager and enable the channels to initiate transactions independently afterward.

```
rmt_channel_handle_t tx_channels[2] = {NULL}; // declare two channels
int tx_gpio_number[2] = {0, 2};
// install channels one by one
for (int i = 0; i < 2; i++) {
    rmt_tx_channel_config_t tx_chan_config = {
        .clk_src = RMT_CLK_SRC_DEFAULT,        // select source clock
        .gpio_num = tx_gpio_number[i],     // GPIO number
        .mem_block_symbols = 64,           // memory block size, 64 * 4 = 256 Bytes
        .resolution_hz = 1 * 1000 * 1000, // 1 MHz resolution
        .trans_queue_depth = 1,            // set the number of transactions that␣
↪can pend in the background
    };
    ESP_ERROR_CHECK(rmt_new_tx_channel(&tx_chan_config, &tx_channels[i]));
}
// install sync manager
rmt_sync_manager_handle_t synchro = NULL;
rmt_sync_manager_config_t synchro_config = {
    .tx_channel_array = tx_channels,
    .array_size = sizeof(tx_channels) / sizeof(tx_channels[0]),
};
ESP_ERROR_CHECK(rmt_new_sync_manager(&synchro_config, &synchro));

ESP_ERROR_CHECK(rmt_transmit(tx_channels[0], led_strip_encoders[0], led_data, led_
↪num * 3, &transmit_config));
// tx_channels[0] does not start transmission until call of `rmt_transmit()` for␣
↪tx_channels[1] returns
ESP_ERROR_CHECK(rmt_transmit(tx_channels[1], led_strip_encoders[1], led_data, led_
↪num * 3, &transmit_config));
```

Initiate RX Transaction As also discussed in the *Enable and Disable Channel*, calling `rmt_enable()` does not prepare an RX to receive RMT symbols. The user needs to specify the basic characteristics of the incoming signals in `rmt_receive_config_t`:

- `rmt_receive_config_t::signal_range_min_ns` specifies the minimal valid pulse duration in either high or low logic levels. A pulse width that is smaller than this value is treated as a glitch, and ignored by the hardware.
- `rmt_receive_config_t::signal_range_max_ns` specifies the maximum valid pulse duration in either high or low logic levels. A pulse width that is bigger than this value is treated as **Stop Signal**, and the receiver generates receive-complete event immediately.
- If the incoming packet is long, that they cannot be stored in the user buffer at once, you can enable the partial reception feature by setting `rmt_receive_config_t::extra_flags::en_partial_rx` to `true`. In this case, the driver invokes `rmt_rx_event_callbacks_t::on_recv_done` callback multiple times during one transaction, when the user buffer is **almost full**. You can check the value of :cpp:member::*rmt_rx_done_event_data_t::is_last* to know if the transaction is about to finish. Please note this features is not supported on all ESP series chips because it relies on hardware abilities like "ping-pong receive" or "DMA receive".

The RMT receiver starts the RX machine after the user calls `rmt_receive()` with the provided configuration above. Note that, this configuration is transaction specific, which means, to start a new round of reception, the user needs to set the `rmt_receive_config_t` again. The receiver saves the incoming signals into its internal memory block or DMA buffer, in the format of `rmt_symbol_word_t`.

Due to the limited size of the memory block, the RMT receiver notifies the driver to copy away the accumulated symbols in a ping-pong way.

The copy destination should be provided in the `buffer` parameter of `rmt_receive()` function. If this buffer overflows due to an insufficient buffer size, the receiver can continue to work, but overflowed symbols are dropped and the following error message is reported: `user buffer too small, received symbols truncated`. Please take care of the lifecycle of the `buffer` parameter, ensuring that the buffer is not recycled before the receiver is finished or stopped.

The receiver is stopped by the driver when it finishes working, i.e., receive a signal whose duration is bigger than *rmt_receive_config_t::signal_range_max_ns*. The user needs to call *rmt_receive()* again to restart the receiver, if necessary. The user can get the received data in the *rmt_rx_event_callbacks_t::on_recv_done* callback. See also *Register Event Callbacks* for more information.

```
static bool example_rmt_rx_done_callback(rmt_channel_handle_t channel, const rmt_
↪rx_done_event_data_t *edata, void *user_data)
{
    BaseType_t high_task_wakeup = pdFALSE;
    QueueHandle_t receive_queue = (QueueHandle_t)user_data;
    // send the received RMT symbols to the parser task
    xQueueSendFromISR(receive_queue, edata, &high_task_wakeup);
    // return whether any task is woken up
    return high_task_wakeup == pdTRUE;
}

QueueHandle_t receive_queue = xQueueCreate(1, sizeof(rmt_rx_done_event_data_t));
rmt_rx_event_callbacks_t cbs = {
    .on_recv_done = example_rmt_rx_done_callback,
};
ESP_ERROR_CHECK(rmt_rx_register_event_callbacks(rx_channel, &cbs, receive_queue));

// the following timing requirement is based on NEC protocol
rmt_receive_config_t receive_config = {
    .signal_range_min_ns = 1250,     // the shortest duration for NEC signal is␣
↪560 µs, 1250 ns < 560 µs, valid signal is not treated as noise
    .signal_range_max_ns = 12000000, // the longest duration for NEC signal is␣
↪9000 µs, 12000000 ns > 9000 µs, the receive does not stop early
};

rmt_symbol_word_t raw_symbols[64]; // 64 symbols should be sufficient for a␣
↪standard NEC frame
// ready to receive
ESP_ERROR_CHECK(rmt_receive(rx_channel, raw_symbols, sizeof(raw_symbols), &receive_
↪config));
// wait for the RX-done signal
rmt_rx_done_event_data_t rx_data;
xQueueReceive(receive_queue, &rx_data, portMAX_DELAY);
// parse the received symbols
example_parse_nec_frame(rx_data.received_symbols, rx_data.num_symbols);
```

RMT Encoder An RMT encoder is part of the RMT TX transaction, whose responsibility is to generate and write the correct RMT symbols into hardware memory or DMA buffer at a specific time. There are some special restrictions for an encoding function:

- During a single transaction, the encoding function may be called multiple times. This is necessary because the target RMT memory block cannot hold all the artifacts at once. To overcome this limitation, the driver utilizes a **ping-pong** approach, where the encoding session is divided into multiple parts. This means that the encoder needs to **keep track of its state** to continue encoding from where it left off in the previous part.
- The encoding function is running in the ISR context. To speed up the encoding session, it is highly recommended to put the encoding function into IRAM. This can also avoid the cache miss during encoding.

To help get started with the RMT driver faster, some commonly used encoders are provided out-of-the-box. They can either work alone or be chained together into a new encoder. See also Composite Pattern for the principle behind it. The driver has defined the encoder interface in *rmt_encoder_t*, it contains the following functions:

- *rmt_encoder_t::encode* is the fundamental function of an encoder. This is where the encoding session happens.
  - The function might be called multiple times within a single transaction. The encode function should return the state of the current encoding session.

  - The supported states are listed in the *rmt_encode_state_t*. If the result contains *RMT_ENCODING_COMPLETE*, it means the current encoder has finished work.
  - If the result contains *RMT_ENCODING_MEM_FULL*, the program needs to yield from the current session, as there is no space to save more encoding artifacts.
- *rmt_encoder_t::reset* should reset the encoder state back to the initial state (the RMT encoder is stateful).
  - If the RMT transmitter is manually stopped without resetting its corresponding encoder, subsequent encoding session can be erroneous.
  - This function is also called implicitly in *rmt_disable()*.
- *rmt_encoder_t::del* should free the resources allocated by the encoder.

Copy Encoder A copy encoder is created by calling *rmt_new_copy_encoder()*. A copy encoder's main functionality is to copy the RMT symbols from user space into the driver layer. It is usually used to encode `const` data, i.e., data does not change at runtime after initialization such as the leading code in the IR protocol.

A configuration structure *rmt_copy_encoder_config_t* should be provided in advance before calling *rmt_new_copy_encoder()*. Currently, this configuration is reserved for future expansion, and has no specific use or setting items for now.

Bytes Encoder A bytes encoder is created by calling *rmt_new_bytes_encoder()*. The bytes encoder's main functionality is to convert the user space byte stream into RMT symbols dynamically. It is usually used to encode dynamic data, e.g., the address and command fields in the IR protocol.

A configuration structure *rmt_bytes_encoder_config_t* should be provided in advance before calling *rmt_new_bytes_encoder()*:

- *rmt_bytes_encoder_config_t::bit0* and *rmt_bytes_encoder_config_t::bit1* are necessary to specify the encoder how to represent bit zero and bit one in the format of *rmt_symbol_word_t*.
- *rmt_bytes_encoder_config_t::msb_first* sets the bit endianness of each byte. If it is set to true, the encoder encodes the **Most Significant Bit** first. Otherwise, it encodes the **Least Significant Bit** first.

Besides the primitive encoders provided by the driver, the user can implement his own encoder by chaining the existing encoders together. A common encoder chain is shown as follows:

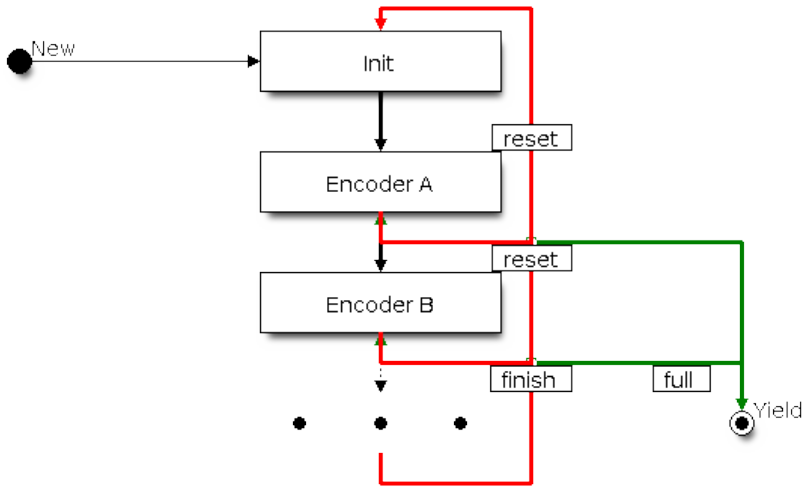


Fig. 23: RMT Encoder Chain

Simple Callback Encoder A simple callback encoder is created by calling *rmt_new_simple_encoder()*. The simple callback encoder allows you to provide a callback that reads data from userspace and writes symbols to the output stream without having to chain other encoders. The callback itself gets a pointer to the data passed to

rmt_transmit(), a counter indicating the amount of symbols already written by the callback in this transmission, and a pointer where to write the encoded RMT symbols as well as the free space there. If the space is not enough for the callback to encode something, it can return 0 and the RMT will wait for previous symbols to be transmitted and call the callback again, now with more space available. If the callback successfully writes RMT symbols, it should return the number of symbols written.

A configuration structure *rmt_simple_encoder_config_t* should be provided in advance before calling *rmt_new_simple_encoder()*:

- *rmt_simple_encoder_config_t::callback* and *rmt_simple_encoder_config_t::arg* provide the callback function and an opaque argument that will be passed to that function.
- *rmt_simple_encoder_config_t::min_chunk_size* specifies the minimum amount of free space, in symbols, the encoder will be always be able to write some data to. In other words, when this amount of free space is passed to the encoder, it should never return 0 (except when the encoder is done encoding symbols).

While the functionality of an encoding process using the simple callback encoder can usually also realized by chaining other encoders, the simple callback can be more easy to understand and maintain than an encoder chain.

Customize RMT Encoder for NEC Protocol This section demonstrates how to write an NEC encoder. The NEC IR protocol uses pulse distance encoding of the message bits. Each pulse burst is `562.5 µs` in length, logical bits are transmitted as follows. It is worth mentioning that the least significant bit of each byte is sent first.

- Logical `0`: a `562.5 µs` pulse burst followed by a `562.5 µs` space, with a total transmit time of `1.125 ms`
- Logical `1`: a `562.5 µs` pulse burst followed by a `1.6875 ms` space, with a total transmit time of `2.25 ms`

When a key is pressed on the remote controller, the transmitted message includes the following elements in the specified order:

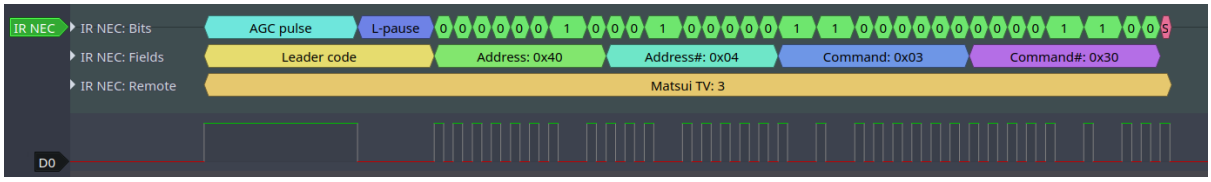


Fig. 24: IR NEC Frame

- `9 ms` leading pulse burst, also called the "AGC pulse"
- `4.5 ms` space
- 8-bit address for the receiving device
- 8-bit logical inverse of the address
- 8-bit command
- 8-bit logical inverse of the command
- a final `562.5 µs` pulse burst to signify the end of message transmission

Then you can construct the NEC *rmt_encoder_t::encode* function in the same order, for example:

```
// IR NEC scan code representation
typedef struct {
    uint16_t address;
    uint16_t command;
} ir_nec_scan_code_t;

// construct an encoder by combining primitive encoders
typedef struct {
    rmt_encoder_t base;           // the base "class" declares the standard␣
↪encoder interface
    rmt_encoder_t *copy_encoder;  // use the copy_encoder to encode the leading␣
↪and ending pulse
```

(continues on next page)

(continued from previous page)

```
    rmt_encoder_t *bytes_encoder; // use the bytes_encoder to encode the address␣
↪and command data
    rmt_symbol_word_t nec_leading_symbol; // NEC leading code with RMT␣
↪representation
    rmt_symbol_word_t nec_ending_symbol;  // NEC ending code with RMT␣
↪representation
    int state; // record the current encoding state, i.e., we are in which␣
↪encoding phase
} rmt_ir_nec_encoder_t;

static size_t rmt_encode_ir_nec(rmt_encoder_t *encoder, rmt_channel_handle_t␣
↪channel, const void *primary_data, size_t data_size, rmt_encode_state_t *ret_
↪state)
{
    rmt_ir_nec_encoder_t *nec_encoder = __containerof(encoder, rmt_ir_nec_encoder_
↪t, base);
    rmt_encode_state_t session_state = RMT_ENCODING_RESET;
    rmt_encode_state_t state = RMT_ENCODING_RESET;
    size_t encoded_symbols = 0;
    ir_nec_scan_code_t *scan_code = (ir_nec_scan_code_t *)primary_data;
    rmt_encoder_handle_t copy_encoder = nec_encoder->copy_encoder;
    rmt_encoder_handle_t bytes_encoder = nec_encoder->bytes_encoder;
    switch (nec_encoder->state) {
    case 0: // send leading code
        encoded_symbols += copy_encoder->encode(copy_encoder, channel, &nec_
↪encoder->nec_leading_symbol,
                                                sizeof(rmt_symbol_word_t), &
↪session_state);
        if (session_state & RMT_ENCODING_COMPLETE) {
            nec_encoder->state = 1; // we can only switch to the next state when␣
↪the current encoder finished
        }
        if (session_state & RMT_ENCODING_MEM_FULL) {
            state |= RMT_ENCODING_MEM_FULL;
            goto out; // yield if there is no free space to put other encoding␣
↪artifacts
        }
    // fall-through
    case 1: // send address
        encoded_symbols += bytes_encoder->encode(bytes_encoder, channel, &scan_
↪code->address, sizeof(uint16_t), &session_state);
        if (session_state & RMT_ENCODING_COMPLETE) {
            nec_encoder->state = 2; // we can only switch to the next state when␣
↪the current encoder finished
        }
        if (session_state & RMT_ENCODING_MEM_FULL) {
            state |= RMT_ENCODING_MEM_FULL;
            goto out; // yield if there is no free space to put other encoding␣
↪artifacts
        }
    // fall-through
    case 2: // send command
        encoded_symbols += bytes_encoder->encode(bytes_encoder, channel, &scan_
↪code->command, sizeof(uint16_t), &session_state);
        if (session_state & RMT_ENCODING_COMPLETE) {
            nec_encoder->state = 3; // we can only switch to the next state when␣
↪the current encoder finished
        }
        if (session_state & RMT_ENCODING_MEM_FULL) {
            state |= RMT_ENCODING_MEM_FULL;
            goto out; // yield if there is no free space to put other encoding␣
↪artifacts
```

(continues on next page)

(continued from previous page)

```
        }
    // fall-through
    case 3: // send ending code
        encoded_symbols += copy_encoder->encode(copy_encoder, channel, &nec_
↪encoder->nec_ending_symbol,
                                                sizeof(rmt_symbol_word_t), &
↪session_state);
        if (session_state & RMT_ENCODING_COMPLETE) {
            nec_encoder->state = RMT_ENCODING_RESET; // back to the initial␣
↪encoding session
            state |= RMT_ENCODING_COMPLETE; // telling the caller the NEC encoding␣
↪has finished
        }
        if (session_state & RMT_ENCODING_MEM_FULL) {
            state |= RMT_ENCODING_MEM_FULL;
            goto out; // yield if there is no free space to put other encoding␣
↪artifacts
        }
    }
out:
    *ret_state = state;
    return encoded_symbols;
}
```

A full sample code can be found in peripherals/rmt/ir_nec_transceiver. In the above snippet, we use a `switch-case` and several `goto` statements to implement a Finite-state machine . With this pattern, users can construct much more complex IR protocols.

Power Management When power management is enabled, i.e., *CONFIG_PM_ENABLE* is on, the system adjusts the APB frequency before going into Light-sleep, thus potentially changing the resolution of the RMT internal counter.

However, the driver can prevent the system from changing APB frequency by acquiring a power management lock of type *ESP_PM_APB_FREQ_MAX*. Whenever the user creates an RMT channel that has selected *RMT_CLK_SRC_APB* as the clock source, the driver guarantees that the power management lock is acquired after the channel enabled by *rmt_enable()*. Likewise, the driver releases the lock after *rmt_disable()* is called for the same channel. This also reveals that the *rmt_enable()* and *rmt_disable()* should appear in pairs.

If the channel clock source is selected to others like *RMT_CLK_SRC_XTAL*, then the driver does not install a power management lock for it, which is more suitable for a low-power application as long as the source clock can still provide sufficient resolution.

IRAM Safe By default, the RMT interrupt is deferred when the Cache is disabled for reasons like writing or erasing the main Flash. Thus the transaction-done interrupt does not get handled in time, which is not acceptable in a real-time application. What is worse, when the RMT transaction relies on **ping-pong** interrupt to successively encode or copy RMT symbols, a delayed interrupt can lead to an unpredictable result.

There is a Kconfig option *CONFIG_RMT_ISR_IRAM_SAFE* that has the following features:

1. Enable the interrupt being serviced even when the cache is disabled
2. Place all functions used by the ISR into IRAM[2]
3. Place the driver object into DRAM in case it is mapped to PSRAM by accident

This Kconfig option allows the interrupt handler to run while the cache is disabled but comes at the cost of increased IRAM consumption.

Another Kconfig option *CONFIG_RMT_RECV_FUNC_IN_IRAM* can place *rmt_receive()* into the IRAM as well. So that the receive function can be used even when the flash cache is disabled.

[2] The callback function, e.g., *rmt_tx_event_callbacks_t::on_trans_done*, and the functions invoked by itself should also reside in IRAM, users need to take care of this by themselves.

Thread Safety The factory function *rmt_new_tx_channel()*, *rmt_new_rx_channel()* and *rmt_new_sync_manager()* are guaranteed to be thread-safe by the driver, which means, user can call them from different RTOS tasks without protection by extra locks. Other functions that take the *rmt_channel_handle_t* and *rmt_sync_manager_handle_t* as the first positional parameter, are not thread-safe. which means the user should avoid calling them from multiple tasks.

The following functions are allowed to use under ISR context as well.

- *rmt_receive()*

Kconfig Options

- *CONFIG_RMT_ISR_IRAM_SAFE* controls whether the default ISR handler can work when cache is disabled, see also *IRAM Safe* for more information.
- *CONFIG_RMT_ENABLE_DEBUG_LOG* is used to enable the debug log at the cost of increased firmware binary size.
- *CONFIG_RMT_RECV_FUNC_IN_IRAM* controls where to place the RMT receive function (IRAM or Flash), see *IRAM Safe* for more information.

Application Examples

- RMT-based RGB LED strip customized encoder: peripherals/rmt/led_strip
- RMT IR NEC protocol encoding and decoding: peripherals/rmt/ir_nec_transceiver
- RMT transactions in queue: peripherals/rmt/musical_buzzer
- RMT-based stepper motor with S-curve algorithm: : peripherals/rmt/stepper_motor
- RMT infinite loop for driving DShot ESC: peripherals/rmt/dshot_esc
- RMT simulate 1-wire protocol (take DS18B20 as example): peripherals/rmt/onewire

FAQ

- Why the RMT transmits more data than expected?

  The encoding for the RMT transmission is carried out within the ISR context. Should the RMT encoding process be prolonged (for example, through logging or tracing the procedure) or if it is delayed due to interrupt latency and preemptive interrupts, the hardware transmitter might read from the memory before the encoder has written to it. Consequently, the transmitter would end up sending outdated data. Although it's not possible to instruct the transmitter to pause and wait, this issue can be mitigated by employing a combination of the following strategies:
  - Increase the *rmt_tx_channel_config_t::mem_block_symbols*, in steps of 48.
  - Place the encoding function in the IRAM with `IRAM_ATTR` attribute.
  - Enable the *rmt_tx_channel_config_t::with_dma* if DMA is available.
  - Install the RMT driver on a separate CPU core to avoid competing for the same CPU resources with other interrupt heavy peripherals (e.g. WiFi, Bluetooth).

API Reference

Header File

- components/esp_driver_rmt/include/driver/rmt_tx.h
- This header file can be included with:

```
#include "driver/rmt_tx.h"
```

- This header file is a part of the API provided by the `esp_driver_rmt` component. To declare that your component depends on `esp_driver_rmt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_rmt
```

or

```
PRIV_REQUIRES esp_driver_rmt
```

Functions

esp_err_t **rmt_new_tx_channel**(const *rmt_tx_channel_config_t* *config, *rmt_channel_handle_t* *ret_chan)

Create a RMT TX channel.

Parameters

- **config** -- **[in]** TX channel configurations
- **ret_chan** -- **[out]** Returned generic RMT channel handle

Returns

- ESP_OK: Create RMT TX channel successfully
- ESP_ERR_INVALID_ARG: Create RMT TX channel failed because of invalid argument
- ESP_ERR_NO_MEM: Create RMT TX channel failed because out of memory
- ESP_ERR_NOT_FOUND: Create RMT TX channel failed because all RMT channels are used up and no more free one
- ESP_ERR_NOT_SUPPORTED: Create RMT TX channel failed because some feature is not supported by hardware, e.g. DMA feature is not supported by hardware
- ESP_FAIL: Create RMT TX channel failed because of other error

esp_err_t **rmt_transmit**(*rmt_channel_handle_t* tx_channel, *rmt_encoder_handle_t* encoder, const void *payload, size_t payload_bytes, const *rmt_transmit_config_t* *config)

Transmit data by RMT TX channel.

Note: This function constructs a transaction descriptor then pushes to a queue. The transaction will not start immediately if there's another one under processing. Based on the setting of *rmt_transmit_config_t::queue_nonblocking*, if there're too many transactions pending in the queue, this function can block until it has free slot, otherwise just return quickly.

Note: The payload data to be transmitted will be encoded into RMT symbols by the specific `encoder`.

Note: You CAN'T modify the `payload` during the transmission, it should be kept valid until the transmission is finished.

Parameters

- **tx_channel** -- **[in]** RMT TX channel that created by `rmt_new_tx_channel()`
- **encoder** -- **[in]** RMT encoder that created by various factory APIs like `rmt_new_bytes_encoder()`
- **payload** -- **[in]** The raw data to be encoded into RMT symbols
- **payload_bytes** -- **[in]** Size of the `payload` in bytes
- **config** -- **[in]** Transmission specific configuration

Returns

- ESP_OK: Transmit data successfully
- ESP_ERR_INVALID_ARG: Transmit data failed because of invalid argument
- ESP_ERR_INVALID_STATE: Transmit data failed because channel is not enabled
- ESP_ERR_NOT_SUPPORTED: Transmit data failed because some feature is not supported by hardware, e.g. unsupported loop count
- ESP_FAIL: Transmit data failed because of other error

esp_err_t **rmt_tx_wait_all_done**(*rmt_channel_handle_t* tx_channel, int timeout_ms)

Wait for all pending TX transactions done.

Note: This function will block forever if the pending transaction can't be finished within a limited time (e.g. an infinite loop transaction). See also `rmt_disable()` for how to terminate a working channel.

Parameters

- **`tx_channel`** -- **[in]** RMT TX channel that created by `rmt_new_tx_channel()`
- **`timeout_ms`** -- **[in]** Wait timeout, in ms. Specially, -1 means to wait forever.

Returns

- ESP_OK: Flush transactions successfully
- ESP_ERR_INVALID_ARG: Flush transactions failed because of invalid argument
- ESP_ERR_TIMEOUT: Flush transactions failed because of timeout
- ESP_FAIL: Flush transactions failed because of other error

esp_err_t **`rmt_tx_register_event_callbacks`**(*rmt_channel_handle_t* tx_channel, const *rmt_tx_event_callbacks_t* *cbs, void *user_data)

Set event callbacks for RMT TX channel.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Note: When CONFIG_RMT_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well. The `user_data` should also reside in SRAM.

Parameters

- **`tx_channel`** -- **[in]** RMT generic channel that created by `rmt_new_tx_channel()`
- **`cbs`** -- **[in]** Group of callback functions
- **`user_data`** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

esp_err_t **`rmt_new_sync_manager`**(const *rmt_sync_manager_config_t* *config, *rmt_sync_manager_handle_t* *ret_synchro)

Create a synchronization manager for multiple TX channels, so that the managed channel can start transmitting at the same time.

Note: All the channels to be managed should be enabled by `rmt_enable()` before put them into sync manager.

Parameters

- **`config`** -- **[in]** Synchronization manager configuration
- **`ret_synchro`** -- **[out]** Returned synchronization manager handle

Returns

- ESP_OK: Create sync manager successfully
- ESP_ERR_INVALID_ARG: Create sync manager failed because of invalid argument
- ESP_ERR_NOT_SUPPORTED: Create sync manager failed because it is not supported by hardware
- ESP_ERR_INVALID_STATE: Create sync manager failed because not all channels are enabled

- ESP_ERR_NO_MEM: Create sync manager failed because out of memory
- ESP_ERR_NOT_FOUND: Create sync manager failed because all sync controllers are used up and no more free one
- ESP_FAIL: Create sync manager failed because of other error

esp_err_t **rmt_del_sync_manager**(*rmt_sync_manager_handle_t* synchro)

Delete synchronization manager.

Parameters **synchro** -- **[in]** Synchronization manager handle returned from `rmt_new_sync_manager()`

Returns

- ESP_OK: Delete the synchronization manager successfully
- ESP_ERR_INVALID_ARG: Delete the synchronization manager failed because of invalid argument
- ESP_FAIL: Delete the synchronization manager failed because of other error

esp_err_t **rmt_sync_reset**(*rmt_sync_manager_handle_t* synchro)

Reset synchronization manager.

Parameters **synchro** -- **[in]** Synchronization manager handle returned from `rmt_new_sync_manager()`

Returns

- ESP_OK: Reset the synchronization manager successfully
- ESP_ERR_INVALID_ARG: Reset the synchronization manager failed because of invalid argument
- ESP_FAIL: Reset the synchronization manager failed because of other error

Structures

struct **rmt_tx_event_callbacks_t**

Group of RMT TX callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_RMT_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well.

Public Members

rmt_tx_done_callback_t **on_trans_done**

Event callback, invoked when transmission is finished

struct **rmt_tx_channel_config_t**

RMT TX channel specific configuration.

Public Members

gpio_num_t **gpio_num**

GPIO number used by RMT TX channel. Set to -1 if unused

rmt_clock_source_t **clk_src**

Clock source of RMT TX channel, channels in the same group must use the same clock source

uint32_t **resolution_hz**

Channel clock resolution, in Hz

size_t **mem_block_symbols**

Size of memory block, in number of *rmt_symbol_word_t*, must be an even. In the DMA mode, this field controls the DMA buffer size, it can be set to a large value; In the normal mode, this field controls the number of RMT memory block that will be used by the channel.

size_t **trans_queue_depth**

Depth of internal transfer queue, increase this value can support more transfers pending in the background

int **intr_priority**

RMT interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **invert_out**

Whether to invert the RMT channel signal before output to GPIO pad

uint32_t **with_dma**

If set, the driver will allocate an RMT channel with DMA capability

uint32_t **io_loop_back**

The signal output from the GPIO will be fed to the input path as well

uint32_t **io_od_mode**

Configure the GPIO as open-drain mode

struct *rmt_tx_channel_config_t*::[anonymous] **flags**

TX channel config flags

struct **rmt_transmit_config_t**

RMT transmit specific configuration.

Public Members

int **loop_count**

Specify the times of transmission in a loop, -1 means transmitting in an infinite loop

uint32_t **eot_level**

Set the output level for the "End Of Transmission"

uint32_t **queue_nonblocking**

If set, when the transaction queue is full, driver will not block the thread but return directly

struct *rmt_transmit_config_t*::[anonymous] **flags**

Transmit specific config flags

struct **rmt_sync_manager_config_t**

Synchronous manager configuration.

Public Members

const *rmt_channel_handle_t* ***tx_channel_array**

Array of TX channels that are about to be managed by a synchronous controller

size_t **array_size**

Size of the `tx_channel_array`

Header File

- components/esp_driver_rmt/include/driver/rmt_rx.h
- This header file can be included with:

```
#include "driver/rmt_rx.h"
```

- This header file is a part of the API provided by the `esp_driver_rmt` component. To declare that your component depends on `esp_driver_rmt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_rmt
```

or

```
PRIV_REQUIRES esp_driver_rmt
```

Functions

esp_err_t **rmt_new_rx_channel**(const *rmt_rx_channel_config_t* *config, *rmt_channel_handle_t* *ret_chan)

Create a RMT RX channel.

Parameters

- **config** -- **[in]** RX channel configurations
- **ret_chan** -- **[out]** Returned generic RMT channel handle

Returns

- ESP_OK: Create RMT RX channel successfully
- ESP_ERR_INVALID_ARG: Create RMT RX channel failed because of invalid argument
- ESP_ERR_NO_MEM: Create RMT RX channel failed because out of memory
- ESP_ERR_NOT_FOUND: Create RMT RX channel failed because all RMT channels are used up and no more free one
- ESP_ERR_NOT_SUPPORTED: Create RMT RX channel failed because some feature is not supported by hardware, e.g. DMA feature is not supported by hardware
- ESP_FAIL: Create RMT RX channel failed because of other error

esp_err_t **rmt_receive**(*rmt_channel_handle_t* rx_channel, void *buffer, size_t buffer_size, const *rmt_receive_config_t* *config)

Initiate a receive job for RMT RX channel.

Note: This function is non-blocking, it initiates a new receive job and then returns. User should check the received data from the `on_recv_done` callback that registered by `rmt_rx_register_event_callbacks()`.

Note: This function can also be called in ISR context.

Note: If you want this function to work even when the flash cache is disabled, please enable the `CONFIG_RMT_RECV_FUNC_IN_IRAM` option.

Parameters

- **`rx_channel`** -- **[in]** RMT RX channel that created by `rmt_new_rx_channel()`
- **`buffer`** -- **[in]** The buffer to store the received RMT symbols
- **`buffer_size`** -- **[in]** size of the `buffer`, in bytes
- **`config`** -- **[in]** Receive specific configurations

Returns

- ESP_OK: Initiate receive job successfully
- ESP_ERR_INVALID_ARG: Initiate receive job failed because of invalid argument
- ESP_ERR_INVALID_STATE: Initiate receive job failed because channel is not enabled
- ESP_FAIL: Initiate receive job failed because of other error

esp_err_t **`rmt_rx_register_event_callbacks`**(*rmt_channel_handle_t* rx_channel, const *rmt_rx_event_callbacks_t* *cbs, void *user_data)

Set callbacks for RMT RX channel.

Note: User can deregister a previously registered callback by calling this function and setting the callback member in the `cbs` structure to NULL.

Note: When CONFIG_RMT_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well. The `user_data` should also reside in SRAM.

Parameters

- **`rx_channel`** -- **[in]** RMT generic channel that created by `rmt_new_rx_channel()`
- **`cbs`** -- **[in]** Group of callback functions
- **`user_data`** -- **[in]** User data, which will be passed to callback functions directly

Returns

- ESP_OK: Set event callbacks successfully
- ESP_ERR_INVALID_ARG: Set event callbacks failed because of invalid argument
- ESP_FAIL: Set event callbacks failed because of other error

Structures

struct **`rmt_rx_event_callbacks_t`**

Group of RMT RX callbacks.

Note: The callbacks are all running under ISR environment

Note: When CONFIG_RMT_ISR_IRAM_SAFE is enabled, the callback itself and functions called by it should be placed in IRAM. The variables used in the function should be in the SRAM as well.

Public Members

rmt_rx_done_callback_t **on_recv_done**

Event callback, invoked when the RMT channel reception is finished or partial data is received

struct **rmt_rx_channel_config_t**

RMT RX channel specific configuration.

Public Members

gpio_num_t **gpio_num**

GPIO number used by RMT RX channel. Set to -1 if unused

rmt_clock_source_t **clk_src**

Clock source of RMT RX channel, channels in the same group must use the same clock source

uint32_t **resolution_hz**

Channel clock resolution, in Hz

size_t **mem_block_symbols**

Size of memory block, in number of *rmt_symbol_word_t*, must be an even. In the DMA mode, this field controls the DMA buffer size, it can be set to a large value (e.g. 1024); In the normal mode, this field controls the number of RMT memory block that will be used by the channel.

int **intr_priority**

RMT interrupt priority, if set to 0, the driver will try to allocate an interrupt with a relative low priority (1,2,3)

uint32_t **invert_in**

Whether to invert the incoming RMT channel signal

uint32_t **with_dma**

If set, the driver will allocate an RMT channel with DMA capability

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

struct *rmt_rx_channel_config_t*::[anonymous] **flags**

RX channel config flags

struct **rmt_receive_config_t**

RMT receive specific configuration.

Public Members

uint32_t **signal_range_min_ns**

A pulse whose width is smaller than this threshold will be treated as glitch and ignored

uint32_t **signal_range_max_ns**

RMT will stop receiving if one symbol level has kept more than `signal_range_max_ns`

struct *rmt_receive_config_t*::*extra_flags* **flags**

Receive specific config flags

struct **extra_flags**

Receive specific flags.

Public Members

uint32_t **en_partial_rx**

Set this flag if the incoming data is very long, and the driver can only receive the data piece by piece, because the user buffer is not sufficient to save all the data.

Header File

- components/esp_driver_rmt/include/driver/rmt_common.h
- This header file can be included with:

```
#include "driver/rmt_common.h"
```

- This header file is a part of the API provided by the `esp_driver_rmt` component. To declare that your component depends on `esp_driver_rmt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_rmt
```

or

```
PRIV_REQUIRES esp_driver_rmt
```

Functions

esp_err_t **rmt_del_channel**(*rmt_channel_handle_t* channel)

Delete an RMT channel.

Parameters **channel** -- **[in]** RMT generic channel that created by `rmt_new_tx_channel()` or `rmt_new_rx_channel()`

Returns

- ESP_OK: Delete RMT channel successfully
- ESP_ERR_INVALID_ARG: Delete RMT channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Delete RMT channel failed because it is still in working
- ESP_FAIL: Delete RMT channel failed because of other error

esp_err_t **rmt_apply_carrier**(*rmt_channel_handle_t* channel, const *rmt_carrier_config_t* *config)

Apply modulation feature for TX channel or demodulation feature for RX channel.

Parameters

- **channel** -- **[in]** RMT generic channel that created by `rmt_new_tx_channel()` or `rmt_new_rx_channel()`
- **config** -- **[in]** Carrier configuration. Specially, a NULL config means to disable the carrier modulation or demodulation feature

Returns

- ESP_OK: Apply carrier configuration successfully
- ESP_ERR_INVALID_ARG: Apply carrier configuration failed because of invalid argument
- ESP_FAIL: Apply carrier configuration failed because of other error

esp_err_t **rmt_enable**(*rmt_channel_handle_t* channel)

Enable the RMT channel.

Note: This function will acquire a PM lock that might be installed during channel allocation

Parameters **channel** -- **[in]** RMT generic channel that created by `rmt_new_tx_channel()` or `rmt_new_rx_channel()`

Returns

- ESP_OK: Enable RMT channel successfully
- ESP_ERR_INVALID_ARG: Enable RMT channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Enable RMT channel failed because it's enabled already
- ESP_FAIL: Enable RMT channel failed because of other error

esp_err_t **rmt_disable**(*rmt_channel_handle_t* channel)

Disable the RMT channel.

Note: This function will release a PM lock that might be installed during channel allocation

Parameters **channel** -- **[in]** RMT generic channel that created by `rmt_new_tx_channel()` or `rmt_new_rx_channel()`

Returns

- ESP_OK: Disable RMT channel successfully
- ESP_ERR_INVALID_ARG: Disable RMT channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable RMT channel failed because it's not enabled yet
- ESP_FAIL: Disable RMT channel failed because of other error

Structures

struct **rmt_carrier_config_t**

RMT carrier wave configuration (for either modulation or demodulation)

Public Members

uint32_t **frequency_hz**

Carrier wave frequency, in Hz, 0 means disabling the carrier

float **duty_cycle**

Carrier wave duty cycle (0~100%)

uint32_t **polarity_active_low**

Specify the polarity of carrier, by default it's modulated to base signal's high level

uint32_t **always_on**

If set, the carrier can always exist even there's not transfer undergoing

struct *rmt_carrier_config_t*::[anonymous] **flags**

Carrier config flags

Header File

- components/esp_driver_rmt/include/driver/rmt_encoder.h
- This header file can be included with:

```
#include "driver/rmt_encoder.h"
```

- This header file is a part of the API provided by the `esp_driver_rmt` component. To declare that your component depends on `esp_driver_rmt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_rmt
```

or

```
PRIV_REQUIRES esp_driver_rmt
```

Functions

esp_err_t **rmt_new_bytes_encoder**(const *rmt_bytes_encoder_config_t* *config, *rmt_encoder_handle_t* *ret_encoder)

Create RMT bytes encoder, which can encode byte stream into RMT symbols.

Parameters
- **config** -- **[in]** Bytes encoder configuration
- **ret_encoder** -- **[out]** Returned encoder handle

Returns
- ESP_OK: Create RMT bytes encoder successfully
- ESP_ERR_INVALID_ARG: Create RMT bytes encoder failed because of invalid argument
- ESP_ERR_NO_MEM: Create RMT bytes encoder failed because out of memory
- ESP_FAIL: Create RMT bytes encoder failed because of other error

esp_err_t **rmt_bytes_encoder_update_config**(*rmt_encoder_handle_t* bytes_encoder, const *rmt_bytes_encoder_config_t* *config)

Update the configuration of the bytes encoder.

Note: The configurations of the bytes encoder is also set up by `rmt_new_bytes_encoder()`. This function is used to update the configuration of the bytes encoder at runtime.

Parameters
- **bytes_encoder** -- **[in]** Bytes encoder handle, created by e.g `rmt_new_bytes_encoder()`
- **config** -- **[in]** Bytes encoder configuration

Returns
- ESP_OK: Update RMT bytes encoder successfully
- ESP_ERR_INVALID_ARG: Update RMT bytes encoder failed because of invalid argument
- ESP_FAIL: Update RMT bytes encoder failed because of other error

esp_err_t **rmt_new_copy_encoder**(const *rmt_copy_encoder_config_t* *config, *rmt_encoder_handle_t* *ret_encoder)

Create RMT copy encoder, which copies the given RMT symbols into RMT memory.

Parameters
- **config** -- **[in]** Copy encoder configuration
- **ret_encoder** -- **[out]** Returned encoder handle

Returns
- ESP_OK: Create RMT copy encoder successfully
- ESP_ERR_INVALID_ARG: Create RMT copy encoder failed because of invalid argument

- ESP_ERR_NO_MEM: Create RMT copy encoder failed because out of memory
- ESP_FAIL: Create RMT copy encoder failed because of other error

esp_err_t **rmt_new_simple_encoder**(const *rmt_simple_encoder_config_t* *config, *rmt_encoder_handle_t* *ret_encoder)

Create RMT simple callback encoder, which uses a callback to convert incoming data into RMT symbols.

Parameters
- **config** -- **[in]** Simple callback encoder configuration
- **ret_encoder** -- **[out]** Returned encoder handle

Returns
- ESP_OK: Create RMT simple callback encoder successfully
- ESP_ERR_INVALID_ARG: Create RMT simple callback encoder failed because of invalid argument
- ESP_ERR_NO_MEM: Create RMT simple callback encoder failed because out of memory
- ESP_FAIL: Create RMT simple callback encoder failed because of other error

esp_err_t **rmt_del_encoder**(*rmt_encoder_handle_t* encoder)

Delete RMT encoder.

Parameters **encoder** -- **[in]** RMT encoder handle, created by e.g `rmt_new_bytes_encoder()`

Returns
- ESP_OK: Delete RMT encoder successfully
- ESP_ERR_INVALID_ARG: Delete RMT encoder failed because of invalid argument
- ESP_FAIL: Delete RMT encoder failed because of other error

esp_err_t **rmt_encoder_reset**(*rmt_encoder_handle_t* encoder)

Reset RMT encoder.

Parameters **encoder** -- **[in]** RMT encoder handle, created by e.g `rmt_new_bytes_encoder()`

Returns
- ESP_OK: Reset RMT encoder successfully
- ESP_ERR_INVALID_ARG: Reset RMT encoder failed because of invalid argument
- ESP_FAIL: Reset RMT encoder failed because of other error

void ***rmt_alloc_encoder_mem**(size_t size)

A helper function to allocate a proper memory for RMT encoder.

Parameters **size** -- Size of memory to be allocated

Returns Pointer to the allocated memory if the allocation is successful, NULL otherwise

Structures

struct **rmt_encoder_t**

Interface of RMT encoder.

Public Members

size_t (***encode**)(*rmt_encoder_t* *encoder, *rmt_channel_handle_t* tx_channel, const void *primary_data, size_t data_size, *rmt_encode_state_t* *ret_state)

Encode the user data into RMT symbols and write into RMT memory.

Note: The encoding function will also be called from an ISR context, thus the function must not call any blocking API.

Note: It's recommended to put this function implementation in the IRAM, to achieve a high performance and less interrupt latency.

Param encoder [in] Encoder handle
Param tx_channel [in] RMT TX channel handle, returned from `rmt_new_tx_channel()`
Param primary_data [in] App data to be encoded into RMT symbols
Param data_size [in] Size of primary_data, in bytes
Param ret_state [out] Returned current encoder's state
Return Number of RMT symbols that the primary data has been encoded into

esp_err_t (*`reset`)(*rmt_encoder_t* *encoder)

Reset encoding state.

Param encoder [in] Encoder handle
Return
- ESP_OK: reset encoder successfully
- ESP_FAIL: reset encoder failed

esp_err_t (*`del`)(*rmt_encoder_t* *encoder)

Delete encoder object.

Param encoder [in] Encoder handle
Return
- ESP_OK: delete encoder successfully
- ESP_FAIL: delete encoder failed

struct **`rmt_bytes_encoder_config_t`**

Bytes encoder configuration.

Public Members

rmt_symbol_word_t **`bit0`**

How to represent BIT0 in RMT symbol

rmt_symbol_word_t **`bit1`**

How to represent BIT1 in RMT symbol

uint32_t **`msb_first`**

Whether to encode MSB bit first

struct *rmt_bytes_encoder_config_t*::[anonymous] **`flags`**

Encoder config flag

struct **`rmt_copy_encoder_config_t`**

Copy encoder configuration.

struct **`rmt_simple_encoder_config_t`**

Simple callback encoder configuration.

Public Members

rmt_encode_simple_cb_t **callback**

Callback to call for encoding data into RMT items

void ***arg**

Opaque user-supplied argument for callback

size_t **min_chunk_size**

Minimum amount of free space, in RMT symbols, the encoder needs in order to guarantee it always returns non-zero. Defaults to 64 if zero / not given.

Type Definitions

typedef size_t (***rmt_encode_simple_cb_t**)(const void *data, size_t data_size, size_t symbols_written, size_t symbols_free, *rmt_symbol_word_t* *symbols, bool *done, void *arg)

Callback for simple callback encoder.

This will get called to encode the data stream of given length (as passed to rmt_transmit by the user) into symbols to be sent by the hardware.

The callback will be initially called with symbol_pos=0. If the callback encodes N symbols and finishes, the next callback will always be with symbols_written=N. If the callback then encodes M symbols, the next callback will always be with symbol_pos=N+M, etc. The only exception is when the encoder is reset (e.g. to begin a new transaction) in which case symbol_pos will always restart at 0.

If the amount of free space in the symbol buffer (as indicated by symbols_free) is too low, the function can return 0 as result and the RMT will call the function again once there is more space available. Note that the callback should eventually return non-0 if called with free space of *rmt_simple_encoder_config_t::min_chunk_size* or more. It is acceptable to return 0 for a given free space N, then on the next call (possibly with a larger free buffer space) return less or more than N symbols.

When the transaction is done (all data_size data is encoded), the callback can indicate this by setting *done to true. This can either happen on the last callback call that returns an amount of symbols encoded, or on a callback that returns zero. In either case, the callback will not be called again for this transaction.

Param data **[in]** Data pointer, as passed to rmt_transmit()
Param data_size **[in]** Size of the data, as passed to rmt_transmit()
Param symbols_written **[in]** Current position in encoded stream, in symbols
Param symbols_free **[in]** The maximum amount of symbols that can be written into the `symbols` buffer
Param symbols **[out]** Symbols to be sent to the RMT hardware
Param done **[out]** Setting this to true marks this transaction as finished
Param arg Opaque argument
Return Amount of symbols encoded in this callback round. 0 if more space is needed.

Enumerations

enum **rmt_encode_state_t**

RMT encoding state.

Values:

enumerator **RMT_ENCODING_RESET**

The encoding session is in reset state

enumerator **RMT_ENCODING_COMPLETE**

The encoding session is finished, the caller can continue with subsequent encoding

enumerator **RMT_ENCODING_MEM_FULL**

The encoding artifact memory is full, the caller should return from current encoding session

Header File

- components/esp_driver_rmt/include/driver/rmt_types.h
- This header file can be included with:

```
#include "driver/rmt_types.h"
```

- This header file is a part of the API provided by the `esp_driver_rmt` component. To declare that your component depends on `esp_driver_rmt`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_rmt
```

or

```
PRIV_REQUIRES esp_driver_rmt
```

Structures

struct **rmt_tx_done_event_data_t**

Type of RMT TX done event data.

Public Members

size_t **num_symbols**

The number of transmitted RMT symbols, including one EOF symbol, which is appended by the driver to mark the end of a transmission. For a loop transmission, this value only counts for one round.

struct **rmt_rx_done_event_data_t**

Type of RMT RX done event data.

Public Members

rmt_symbol_word_t ***received_symbols**

Point to the received RMT symbols

size_t **num_symbols**

The number of received RMT symbols

uint32_t **is_last**

Indicating if the current received data are the last part of the transaction

struct *rmt_rx_done_event_data_t*::[anonymous] **flags**

Extra flags

Type Definitions

typedef struct rmt_channel_t ***rmt_channel_handle_t**

Type of RMT channel handle.

typedef struct rmt_sync_manager_t ***rmt_sync_manager_handle_t**

Type of RMT synchronization manager handle.

typedef struct *rmt_encoder_t* ***rmt_encoder_handle_t**

Type of RMT encoder handle.

typedef bool (***rmt_tx_done_callback_t**)(*rmt_channel_handle_t* tx_chan, const *rmt_tx_done_event_data_t* *edata, void *user_ctx)

Prototype of RMT event callback.

Param tx_chan **[in]** RMT channel handle, created from `rmt_new_tx_channel()`
Param edata **[in]** Point to RMT event data. The lifecycle of this pointer memory is inside this function, user should copy it into static memory if used outside this function.
Param user_ctx **[in]** User registered context, passed from `rmt_tx_register_event_callbacks()`
Return Whether a high priority task has been waken up by this callback function

typedef bool (***rmt_rx_done_callback_t**)(*rmt_channel_handle_t* rx_chan, const *rmt_rx_done_event_data_t* *edata, void *user_ctx)

Prototype of RMT event callback.

Param rx_chan **[in]** RMT channel handle, created from `rmt_new_rx_channel()`
Param edata **[in]** Point to RMT event data. The lifecycle of this pointer memory is inside this function, user should copy it into static memory if used outside this function.
Param user_ctx **[in]** User registered context, passed from `rmt_rx_register_event_callbacks()`
Return Whether a high priority task has been waken up by this function

Header File

- components/hal/include/hal/rmt_types.h
- This header file can be included with:

```
#include "hal/rmt_types.h"
```

Unions

union **rmt_symbol_word_t**

#include <rmt_types.h> The layout of RMT symbol stored in memory, which is decided by the hardware design.

Public Members

uint16_t **duration0**

Duration of level0

uint16_t **level0**

Level of the first part

uint16_t **duration1**

Duration of level1

uint16_t **level1**

Level of the second part

struct *rmt_symbol_word_t*::[anonymous] **[anonymous]**

uint32_t **val**

Equivalent unsigned value for the RMT symbol

Type Definitions

typedef *soc_periph_rmt_clk_src_t* **rmt_clock_source_t**

RMT group clock source.

Note: User should select the clock source based on the power and resolution requirement

2.6.17 SD Pull-up Requirements

Espressif hardware products are designed for multiple use cases which may require different pull states on pins. For this reason, the pull state of particular pins on certain products needs to be adjusted to provide the pull-ups required in the SD bus.

SD pull-up requirements apply to cases where ESP32-S3 uses the SPI or SDMMC controller to communicate with SD cards. When an SD card is operating in SPI mode or 1-bit SD mode, the CMD and DATA (DAT0 - DAT3) lines of the SD bus must be pulled up by 10 kOhm resistors. SD cards and SDIO devices should also have pull-ups on all above-mentioned lines (regardless of whether these lines are connected to the host) in order to prevent them from entering a wrong state.

This document has the following structure:

- *Overview of compatibility* between the default pull states on pins of Espressif's products and the states required by the SD bus
- *Solutions* - ideas on how to resolve compatibility issues
- *Related information* - other relevant information

Overview of Compatibility

This section provides an overview of compatibility issues that might occur when using SDIO (secure digital input output). Since the SD bus needs to be connected to pull-ups, these issues should be resolved regardless of whether they are related to master (host) or slave (device). Each issue has links to its respective solution. A solution for a host and device may differ.

Systems on a Chip (SoCs) ESP32-S3 SDMMC host controller allows using any of GPIOs for any of SD interface signals. However, it is recommended to avoid using strapping GPIOs, GPIOs with internal weak pull-downs and GPIOs commonly used for other purposes to prevent conflicts:

- GPIO0 (strapping pin)
- GPIO45, GPIO46 (strapping pins, internal weak pulldown)
- GPIO26 - GPIO32 (commonly used for SPI Flash and PSRAM)
- GPIO33 - GPIO37 (when using chips and modules with Octal SPI Flash or Octal PSRAM)
- GPIO43, GPIO44 (GPIOs used for UART0 by default)

- GPIO19, GPIO20 (GPIOs used for USB by default)

Systems in Packages (SIP)

Modules

Development Boards

- ESP32-S3-DevKitC-1
  - *No Pull-ups*
- ESP32-S3-USB-OTG
  - The board may be used in 1-line and 4-line SD mode or SPI mode.
  - Required pull-ups are provided on GPIOs 33-38.
- ESP32-S3-EYE
  - The board is limited to 1-line SD mode.
  - Required pull-ups are provided on GPIOs 38-40.

Solutions

No Pull-ups If you use a development board without pull-ups, you can do the following:

- If your host and slave device are on separate boards, replace one of them with a board that has pull-ups. For the list of Espressif's development boards with pull-ups, go to *Development Boards*.
- Attach external pull-ups by connecting each pin which requires a pull-up to VDD via a 10 kOhm resistor.

Related Information

2.6.18 SDMMC Host Driver

Overview

ESP32-S3's SDMMC host peripheral has two slots. Each slot can be used independently to connect to an SD card, SDIO device, or eMMC chip.

Both slots `SDMMC_HOST_SLOT_0` and `SDMMC_HOST_SLOT_1` support 1-, 4- and 8-line SD interfaces. The slots are connected to ESP32-S3 GPIOs using the GPIO matrix. This means that any GPIO may be used for each of the SD card signals.

Supported Speed Modes

SDMMC Host driver supports the following speed modes:

- Default Speed (20 MHz): 1-line or 4-line with SD cards, and 1-line, 4-line, or 8-line with 3.3 V eMMC
- High Speed (40 MHz): 1-line or 4-line with SD cards, and 1-line, 4-line, or 8-line with 3.3 V eMMC
- High Speed DDR (40 MHz): 4-line with 3.3 V eMMC

Speed modes not supported at present:

- High Speed DDR mode: 8-line eMMC
- UHS-I 1.8 V modes: 4-line SD cards

Using the SDMMC Host Driver

Of all the functions listed below, only the following ones will be used directly by most applications:

- *sdmmc_host_init()*
- *sdmmc_host_init_slot()*
- *sdmmc_host_deinit()*

Other functions, such as the ones given below, will be called by the SD/MMC protocol layer via function pointers in the sdmmc_host_t structure:

- *sdmmc_host_set_bus_width()*
- *sdmmc_host_set_card_clk()*
- *sdmmc_host_do_transaction()*

Configuring Bus Width and Frequency

With the default initializers for sdmmc_host_t and *sdmmc_slot_config_t*, i.e., SDMMC_HOST_DEFAULT and SDMMC_SLOT_CONFIG_DEFAULT, SDMMC Host driver will attempt to use the widest bus supported by the card (4 lines for SD, 8 lines for eMMC) and the frequency of 20 MHz.

In the designs where communication at 40 MHz frequency can be achieved, it is possible to increase the bus frequency by changing the max_freq_khz field of sdmmc_host_t:

```
sdmmc_host_t host = SDMMC_HOST_DEFAULT();
host.max_freq_khz = SDMMC_FREQ_HIGHSPEED;
```

If you need a specific frequency other than standard speeds, you are free to use any value from within an appropriate range of the SD interface given (SDMMC or SDSPI). However, the real clock frequency shall be calculated by the underlying driver and the value can be different from the one required.

For the SDMMC, max_freq_khz works as the upper limit so the final frequency value shall be always lower or equal. For the SDSPI, the nearest fitting frequency is supplied and thus the value can be greater than/equal to/lower than max_freq_khz.

To configure the bus width, set the width field of *sdmmc_slot_config_t*. For example, to set 1-line mode:

```
sdmmc_slot_config_t slot = SDMMC_SLOT_CONFIG_DEFAULT();
slot.width = 1;
```

Configuring GPIOs

ESP32-S3 SDMMC Host can be configured to use arbitrary GPIOs for each of the signals. Configuration is performed by setting members of *sdmmc_slot_config_t* structure.

For example, to use GPIOs 1-6 for CLK, CMD, and D0-D3 signals respectively:

```
sdmmc_slot_config_t slot = SDMMC_SLOT_CONFIG_DEFAULT();
slot.clk = GPIO_NUM_1;
slot.cmd = GPIO_NUM_2;
slot.d0 = GPIO_NUM_3;
slot.d1 = GPIO_NUM_4;
slot.d2 = GPIO_NUM_5;
slot.d3 = GPIO_NUM_6;
```

It is also possible to configure Card Detect and Write Protect pins. Similar to other signals, set cd and wp members of the same structure:

```
slot.cd = GPIO_NUM_7;
slot.wp = GPIO_NUM_8;
```

SDMMC_SLOT_CONFIG_DEFAULT sets both to GPIO_NUM_NC, meaning that by default the signals are not used.

Once *sdmmc_slot_config_t* structure is initialized this way, you can use it when calling *sdmmc_host_init_slot()* or one of the higher level functions (such as *esp_vfs_fat_sdmmc_mount()*).

DDR Mode for eMMC Chips

By default, DDR mode will be used if:

- SDMMC host frequency is set to SDMMC_FREQ_HIGHSPEED in sdmmc_host_t structure, and
- eMMC chip reports DDR mode support in its CSD register

DDR mode places higher requirements for signal integrity. To disable DDR mode while keeping the SDMMC_FREQ_HIGHSPEED frequency, clear the SDMMC_HOST_FLAG_DDR bit in sdmmc_host_t::flags field of the sdmmc_host_t:

```
sdmmc_host_t host = SDMMC_HOST_DEFAULT();
host.max_freq_khz = SDMMC_FREQ_HIGHSPEED;
host.flags &= ~SDMMC_HOST_FLAG_DDR;
```

See also

- *SD/SDIO/MMC Driver*: introduces the higher-level driver which implements the protocol layer.
- *SD SPI Host Driver*: introduces a similar driver that uses the SPI controller and is limited to SD protocol's SPI mode.
- *SD Pull-up Requirements*: introduces pull-up support and compatibilities of modules and development kits.

API Reference

Header File

- components/esp_driver_sdmmc/include/driver/sdmmc_host.h
- This header file can be included with:

  ```
  #include "driver/sdmmc_host.h"
  ```

- This header file is a part of the API provided by the esp_driver_sdmmc component. To declare that your component depends on esp_driver_sdmmc, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_driver_sdmmc
  ```

  or

  ```
  PRIV_REQUIRES esp_driver_sdmmc
  ```

Functions

esp_err_t **sdmmc_host_init**(void)

Initialize SDMMC host peripheral.

Note: This function is not thread safe

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if sdmmc_host_init was already called
- ESP_ERR_NO_MEM if memory can not be allocated

esp_err_t **sdmmc_host_init_slot**(int slot, const *sdmmc_slot_config_t* *slot_config)

Initialize given slot of SDMMC peripheral.

On the ESP32, SDMMC peripheral has two slots:

- Slot 0: 8-bit wide, maps to HS1_* signals in PIN MUX
- Slot 1: 4-bit wide, maps to HS2_* signals in PIN MUX

Card detect and write protect signals can be routed to arbitrary GPIOs using GPIO matrix.

Note: This function is not thread safe

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **slot_config** -- additional configuration for the slot

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE if host has not been initialized using sdmmc_host_init

esp_err_t **sdmmc_host_set_bus_width**(int slot, size_t width)

Select bus width to be used for data transfer.

SD/MMC card must be initialized prior to this command, and a command to set bus width has to be sent to the card (e.g. SD_APP_SET_BUS_WIDTH)

Note: This function is not thread safe

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **width** -- bus width (1, 4, or 8 for slot 0; 1 or 4 for slot 1)

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if slot number or width is not valid

size_t **sdmmc_host_get_slot_width**(int slot)

Get bus width configured in `sdmmc_host_init_slot` to be used for data transfer.

Parameters **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
Returns configured bus width of the specified slot.

esp_err_t **sdmmc_host_set_card_clk**(int slot, uint32_t freq_khz)

Set card clock frequency.

Currently only integer fractions of 40MHz clock can be used. For High Speed cards, 40MHz can be used. For Default Speed cards, 20MHz can be used.

Note: This function is not thread safe

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **freq_khz** -- card clock frequency, in kHz

Returns
- ESP_OK on success
- other error codes may be returned in the future

esp_err_t **sdmmc_host_set_bus_ddr_mode**(int slot, bool ddr_enabled)

Enable or disable DDR mode of SD interface.

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **ddr_enabled** -- enable or disable DDR mode

Returns
- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if DDR mode is not supported on this slot

esp_err_t **sdmmc_host_set_cclk_always_on**(int slot, bool cclk_always_on)

Enable or disable always-on card clock When cclk_always_on is false, the host controller is allowed to shut down the card clock between the commands. When cclk_always_on is true, the clock is generated even if no command is in progress.

Parameters
- **slot** -- slot number
- **cclk_always_on** -- enable or disable always-on clock

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if the slot number is invalid

esp_err_t **sdmmc_host_do_transaction**(int slot, sdmmc_command_t *cmdinfo)

Send command to the card and get response.

This function returns when command is sent and response is received, or data is transferred, or timeout occurs.

Attention Data buffer passed in cmdinfo->data must be in DMA capable memory and aligned to 4 byte boundary. If it's behind the cache, both cmdinfo->data and cmdinfo->buflen need to be aligned to cache line boundary.

Note: This function is not thread safe w.r.t. init/deinit functions, and bus width/clock speed configuration functions. Multiple tasks can call sdmmc_host_do_transaction as long as other sdmmc_host_* functions are not called.

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **cmdinfo** -- pointer to structure describing command and data to transfer

Returns
- ESP_OK on success
- ESP_ERR_TIMEOUT if response or data transfer has timed out
- ESP_ERR_INVALID_CRC if response or data transfer CRC check has failed
- ESP_ERR_INVALID_RESPONSE if the card has sent an invalid response
- ESP_ERR_INVALID_SIZE if the size of data transfer is not valid in SD protocol
- ESP_ERR_INVALID_ARG if the data buffer is not in DMA capable memory

esp_err_t **sdmmc_host_io_int_enable**(int slot)

Enable IO interrupts.

This function configures the host to accept SDIO interrupts.

Parameters **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
Returns returns ESP_OK, other errors possible in the future

esp_err_t **sdmmc_host_io_int_wait**(int slot, TickType_t timeout_ticks)

Block until an SDIO interrupt is received, or timeout occurs.

Parameters
- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)

- **timeout_ticks** -- number of RTOS ticks to wait for the interrupt

Returns

- ESP_OK on success (interrupt received)
- ESP_ERR_TIMEOUT if the interrupt did not occur within timeout_ticks

esp_err_t **sdmmc_host_deinit**(void)

Disable SDMMC host and release allocated resources.

Note: This function is not thread safe

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if sdmmc_host_init function has not been called

esp_err_t **sdmmc_host_get_real_freq**(int slot, int *real_freq_khz)

Provides a real frequency used for an SD card installed on specific slot of SD/MMC host controller.

This function calculates real working frequency given by current SD/MMC host controller setup for required slot: it reads associated host and card dividers from corresponding SDMMC registers, calculates respective frequency and stores the value into the 'real_freq_khz' parameter

Parameters

- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **real_freq_khz** -- **[out]** output parameter for the result frequency (in kHz)

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG on real_freq_khz == NULL or invalid slot number used

esp_err_t **sdmmc_host_set_input_delay**(int slot, sdmmc_delay_phase_t delay_phase)

set input delay

- This API sets delay when the SDMMC Host samples the signal from the SD Slave.
- This API will check if the given `delay_phase` is valid or not.
- This API will print out the delay time, in picosecond (ps)

Note: ESP32 doesn't support this feature, you will get an `ESP_ERR_NOT_SUPPORTED`

Parameters

- **slot** -- slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **delay_phase** -- delay phase, this API will convert the phase into picoseconds and print it out

Returns

- ESP_OK: ON success.
- ESP_ERR_INVALID_ARG: Invalid argument.
- ESP_ERR_NOT_SUPPORTED: ESP32 doesn't support this feature.

esp_err_t **sdmmc_host_get_dma_info**(int slot, esp_dma_mem_info_t *dma_mem_info)

Get the DMA memory information for the host driver.

Parameters

- **slot** -- **[in]** slot number (SDMMC_HOST_SLOT_0 or SDMMC_HOST_SLOT_1)
- **dma_mem_info** -- **[out]** DMA memory information structure

Returns

- ESP_OK: ON success.
- ESP_ERR_INVALID_ARG: Invalid argument.

Structures

struct **sdmmc_slot_config_t**

Extra configuration for SDMMC peripheral slot

Public Members

gpio_num_t **clk**

GPIO number of CLK signal.

gpio_num_t **cmd**

GPIO number of CMD signal.

gpio_num_t **d0**

GPIO number of D0 signal.

gpio_num_t **d1**

GPIO number of D1 signal.

gpio_num_t **d2**

GPIO number of D2 signal.

gpio_num_t **d3**

GPIO number of D3 signal.

gpio_num_t **d4**

GPIO number of D4 signal. Ignored in 1- or 4- line mode.

gpio_num_t **d5**

GPIO number of D5 signal. Ignored in 1- or 4- line mode.

gpio_num_t **d6**

GPIO number of D6 signal. Ignored in 1- or 4- line mode.

gpio_num_t **d7**

GPIO number of D7 signal. Ignored in 1- or 4- line mode.

gpio_num_t **gpio_cd**

GPIO number of card detect signal.

gpio_num_t **cd**

GPIO number of card detect signal; shorter name.

gpio_num_t **gpio_wp**

GPIO number of write protect signal.

gpio_num_t **wp**

GPIO number of write protect signal; shorter name.

uint8_t **width**

Bus width used by the slot (might be less than the max width supported)

uint32_t **flags**

Features used by this slot.

Macros

SDMMC_SLOT_FLAG_INTERNAL_PULLUP

Enable internal pullups on enabled pins. The internal pullups are insufficient however, please make sure external pullups are connected on the bus. This is for debug / example purpose only.

SDMMC_SLOT_FLAG_WP_ACTIVE_HIGH

GPIO write protect polarity. 0 means "active low", i.e. card is protected when the GPIO is low; 1 means "active high", i.e. card is protected when GPIO is high.

2.6.19 SD SPI Host Driver

Overview

The SD SPI host driver allows communication with one or more SD cards using the SPI Master driver, which utilizes the SPI host. Each card is accessed through an SD SPI device, represented by an SD SPI handle *sdspi_dev_handle_t*, which returns when the device is attached to an SPI bus by calling *sdspi_host_init_device()*. It is important to note that the SPI bus should be initialized beforehand by *spi_bus_initialize()*.

With the help of *SPI Master Driver* the SD SPI host driver based on, the SPI bus can be shared among SD cards and other SPI devices. The SPI Master driver will handle exclusive access from different tasks.

The SD SPI driver uses software-controlled CS signal.

How to Use

Firstly, use the macro *SDSPI_DEVICE_CONFIG_DEFAULT* to initialize the structure *sdspi_device_config_t*, which is used to initialize an SD SPI device. This macro will also fill in the default pin mappings, which are the same as the pin mappings of the SDMMC host driver. Modify the host and pins of the structure to desired value. Then call `sdspi_host_init_device` to initialize the SD SPI device and attach to its bus.

Then use the *SDSPI_HOST_DEFAULT* macro to initialize the `sdmmc_host_t` structure, which is used to store the state and configurations of the upper layer (SD/SDIO/MMC driver). Modify the `slot` parameter of the structure to the SD SPI device SD SPI handle just returned from `sdspi_host_init_device`. Call `sdmmc_card_init` with the `sdmmc_host_t` to probe and initialize the SD card.

Now you can use SD/SDIO/MMC driver functions to access your card!

Other Details

Only the following driver's API functions are normally used by most applications:

- *sdspi_host_init()*
- *sdspi_host_init_device()*
- *sdspi_host_remove_device()*

- *sdspi_host_deinit()*

Other functions are mostly used by the protocol level SD/SDIO/MMC driver via function pointers in the sdmmc_host_t structure. For more details, see *SD/SDIO/MMC Driver*.

Note: SD over SPI does not support speeds above SDMMC_FREQ_DEFAULT due to the limitations of the SPI driver.

Warning: If you want to share the SPI bus among SD card and other SPI devices, there are some restrictions, see *Sharing the SPI Bus Among SD Cards and Other SPI Devices*.

Related Docs

Sharing the SPI Bus Among SD Cards and Other SPI Devices

The SD card has an SPI mode, enabling it to function as an SPI device, but there are some restrictions that we need to pay attention to.

Pin Loading of Other Devices When adding more devices onto the same bus, the overall pin loading increases. The loading consists of AC loading (pin capacitor) and DC loading (pull-ups).

AC Loading SD cards, designed for high-speed communications, have small pin capacitors (AC loading) to work until 50 MHz. However, the other attached devices will increase the pin's AC loading.

Heavy AC loading of a pin may prevent the pin from being toggled quickly. By using an oscilloscope, you will see the edges of the pin become smoother, i.e., the gradient of the edge is smaller. The setup timing requirements of an SD card may be violated when the card is connected to a bus with a high AC load. Even worse, high AC loads may cause the SD card and other SPI devices to fail to properly resolve clock signals from the host, affecting communication stability.

This issue may be more obvious if other attached devices are not designed to work at the same frequency as the SD card, because they may have larger pin capacitors. The larger the pin capacity, the greater the pin response time, the smaller the max frequency the SD bus can work.

To see if your pin AC loading is too heavy, you can try the following tests:

Terminology:

- **launch edge**: at which clock edge the data starts to toggle;
- **latch edge**: at which clock edge the data is supposed to be sampled by the receiver. For SD card, it is the rising edge.

1. Use an oscilloscope to see the clock and compare the data line to the clock.
   - If you see the clock is not fast enough, e.g., the rising/falling edge is longer than 1/4 of the clock cycle, it means the clock is skewed too much.
   - If you see the data line unstable before the latch edge of the clock, it means the load of the data line is too large.

   You may also observe the corresponding phenomenon that data delayed largely from the launching edge of the clock with logic analyzers. But it is not as obvious as with an oscilloscope.
2. Try to use a slower clock frequency.
   If the lower frequency can work while the higher frequency cannot, it is an indication that the AC loading on the pins is too large.

If the AC loading of the pins is too large, you can either use other faster devices with lower pin load or slow down the clock speed.

DC Loading The pull-ups required by SD cards are usually around 10 kOhm to 50 kOhm, which may be too strong for some other SPI devices.

Check the specification of your device about its DC output current, it should be larger than 700 µA, otherwise, the device output may not be read correctly.

Initialization Sequence

Note: If you see any problem in the following steps, please make sure the timing is correct first. You can try to slow down the clock speed, such as setting `SDMMC_FREQ_PROBING` to 400 kHz for SD card, to avoid the influence of pin AC loading, as discussed in the previous section.

When using an SD card with other SPI devices on the same SPI bus, due to the restrictions of the SD card startup flow, the following initialization sequence should be followed. Refer to storage/sd_card for further details.

1. Initialize the SPI bus properly by *`spi_bus_initialize()`*.
2. Tie the CS lines of all other devices than the SD card to idle state (by default it's high). This is to avoid conflicts with the SD card in the following step.
   You can do this by either:
   1. Attach devices to the SPI bus by calling *`spi_bus_add_device()`*. This function will by default initialize the GPIO that is used as CS to the idle level: high.
   2. Initialize GPIO on the CS pin that needs to be tied up before actually adding a new device.
   3. Rely on the internal/external pull-up (**not recommended**) to pull up all the CS pins when the GPIOs of ESP are not initialized yet. You need to check carefully the pull-up is strong enough and there are no other pull-downs that will influence the pull-up. For example, internal pull-down should be enabled.
3. Mount the card to the filesystem by calling *`esp_vfs_fat_sdspi_mount()`*.
   This step will put the SD card into the SPI mode, which **should** be done before all other SPI communications on the same bus. Otherwise, the card will stay in the SD mode, in which mode it may randomly respond to any SPI communications on the bus, even when its CS line is not addressed.
   If you want to test this behavior, please also note that, once the card is put into SPI mode, it will not return to SD mode before the next power cycle, i.e., powered down and powered up again.
4. Now you can talk to other SPI devices freely!

API Reference

Header File

- components/esp_driver_sdspi/include/driver/sdspi_host.h
- This header file can be included with:

```
#include "driver/sdspi_host.h"
```

- This header file is a part of the API provided by the `esp_driver_sdspi` component. To declare that your component depends on `esp_driver_sdspi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_sdspi
```

or

```
PRIV_REQUIRES esp_driver_sdspi
```

Functions

esp_err_t **sdspi_host_init**(void)

Initialize SD SPI driver.

Note: This function is not thread safe

Returns

- ESP_OK on success
- other error codes may be returned in future versions

esp_err_t **sdspi_host_init_device**(const *sdspi_device_config_t* *dev_config, *sdspi_dev_handle_t* *out_handle)

Attach and initialize an SD SPI device on the specific SPI bus.

Note: This function is not thread safe

Note: Initialize the SPI bus by `spi_bus_initialize()` before calling this function.

Note: The SDIO over sdspi needs an extra interrupt line. Call `gpio_install_isr_service()` before this function.

Parameters
- **dev_config** -- pointer to device configuration structure
- **out_handle** -- Output of the handle to the sdspi device.

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if sdspi_host_init_device has invalid arguments
- ESP_ERR_NO_MEM if memory can not be allocated
- other errors from the underlying spi_master and gpio drivers

esp_err_t **sdspi_host_remove_device**(*sdspi_dev_handle_t* handle)

Remove an SD SPI device.

Parameters **handle** -- Handle of the SD SPI device
Returns Always ESP_OK

esp_err_t **sdspi_host_do_transaction**(*sdspi_dev_handle_t* handle, sdmmc_command_t *cmdinfo)

Send command to the card and get response.

This function returns when command is sent and response is received, or data is transferred, or timeout occurs.

Note: This function is not thread safe w.r.t. init/deinit functions, and bus width/clock speed configuration functions. Multiple tasks can call sdspi_host_do_transaction as long as other sdspi_host_* functions are not called.

Parameters
- **handle** -- Handle of the sdspi device
- **cmdinfo** -- pointer to structure describing command and data to transfer

Returns
- ESP_OK on success
- ESP_ERR_TIMEOUT if response or data transfer has timed out
- ESP_ERR_INVALID_CRC if response or data transfer CRC check has failed
- ESP_ERR_INVALID_RESPONSE if the card has sent an invalid response

esp_err_t **sdspi_host_set_card_clk**(*sdspi_dev_handle_t* host, uint32_t freq_khz)

Set card clock frequency.

Currently only integer fractions of 40MHz clock can be used. For High Speed cards, 40MHz can be used. For Default Speed cards, 20MHz can be used.

Note: This function is not thread safe

Parameters
- **host** -- Handle of the sdspi device
- **freq_khz** -- card clock frequency, in kHz

Returns
- ESP_OK on success
- other error codes may be returned in the future

esp_err_t **sdspi_host_get_real_freq**(*sdspi_dev_handle_t* handle, int *real_freq_khz)

Calculate working frequency for specific device.

Parameters
- **handle** -- SDSPI device handle
- **real_freq_khz** -- **[out]** output parameter to hold the calculated frequency (in kHz)

Returns
- ESP_ERR_INVALID_ARG : `handle` is NULL or invalid or `real_freq_khz` parameter is NULL
- ESP_OK : Success

esp_err_t **sdspi_host_deinit**(void)

Release resources allocated using sdspi_host_init.

Note: This function is not thread safe

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE if sdspi_host_init function has not been called

esp_err_t **sdspi_host_io_int_enable**(*sdspi_dev_handle_t* handle)

Enable SDIO interrupt.

Parameters **handle** -- Handle of the sdspi device

Returns
- ESP_OK on success

esp_err_t **sdspi_host_io_int_wait**(*sdspi_dev_handle_t* handle, TickType_t timeout_ticks)

Wait for SDIO interrupt until timeout.

Parameters
- **handle** -- Handle of the sdspi device
- **timeout_ticks** -- Ticks to wait before timeout.

Returns
- ESP_OK on success

esp_err_t **sdspi_host_get_dma_info**(int slot, esp_dma_mem_info_t *dma_mem_info)

Get the DMA memory information for the host driver.

Parameters
- **slot** -- **[in]** Not used
- **dma_mem_info** -- **[out]** DMA memory information structure

Returns
- ESP_OK: ON success.
- ESP_ERR_INVALID_ARG: Invalid argument.

Structures

struct **sdspi_device_config_t**

Extra configuration for SD SPI device.

Public Members

spi_host_device_t **host_id**

SPI host to use, SPIx_HOST (see spi_types.h).

gpio_num_t **gpio_cs**

GPIO number of CS signal.

gpio_num_t **gpio_cd**

GPIO number of card detect signal.

gpio_num_t **gpio_wp**

GPIO number of write protect signal.

gpio_num_t **gpio_int**

GPIO number of interrupt line (input) for SDIO card.

bool **gpio_wp_polarity**

GPIO write protect polarity 0 means "active low", i.e. card is protected when the GPIO is low; 1 means "active high", i.e. card is protected when GPIO is high.

Macros

SDSPI_DEFAULT_HOST

SDSPI_DEFAULT_DMA

SDSPI_HOST_DEFAULT()

Default sdmmc_host_t structure initializer for SD over SPI driver.

Uses SPI mode and max frequency set to 20MHz

'slot' should be set to an sdspi device initialized by `sdspi_host_init_device()`.

SDSPI_SLOT_NO_CS

indicates that card select line is not used

SDSPI_SLOT_NO_CD

indicates that card detect line is not used

SDSPI_SLOT_NO_WP

indicates that write protect line is not used

SDSPI_SLOT_NO_INT

indicates that interrupt line is not used

SDSPI_IO_ACTIVE_LOW

SDSPI_DEVICE_CONFIG_DEFAULT()

Macro defining default configuration of SD SPI device.

Type Definitions

typedef int **sdspi_dev_handle_t**

Handle representing an SD SPI device.

2.6.20 Sigma-Delta Modulation (SDM)

Introduction

ESP32-S3 has a second-order sigma-delta modulator, which can generate independent PDM pulses to multiple channels. Please refer to the TRM to check how many hardware channels are available.[1]

Delta-sigma modulation converts an analog voltage signal into a pulse frequency, or pulse density, which can be understood as pulse-density modulation (PDM) (refer to Delta-sigma modulation on Wikipedia).

The main differences comparing to I2S PDM mode and DAC peripheral are:

1. SDM has no clock signal, it is just like the DAC mode of PDM;
2. SDM has no DMA, and it can not change its output density continuously. If you have to, you can update the density in a timer's callback;
3. Based on the former two points, unlike the DAC peripheral, an external active or passive low-pass filter is required additionally to restore the analog wave (See *Convert to an Analog Signal (Optional)*).

Typically, a Sigma-Delta modulated channel can be used in scenarios like:

- LED dimming
- Simple DAC (8-bit), with the help of an active RC low-pass filter
- Class D amplifier, with the help of a half-bridge or full-bridge circuit plus an LC low-pass filter

Functional Overview

The following sections of this document cover the typical steps to install and operate an SDM channel:

- *Resource Allocation* - covers how to initialize and configure an SDM channel and how to recycle the resources when it finishes working.
- *Enable and Disable Channel* - covers how to enable and disable the channel.
- *Set Pulse Density* - describes how to set the equivalent duty cycle of the PDM pulses.
- *Power Management* - describes how different source clock selections can affect power consumption.
- *IRAM Safe* - lists which functions are supposed to work even when the cache is disabled.
- *Thread Safety* - lists which APIs are guaranteed to be thread-safe by the driver.
- *Kconfig Options* - lists the supported Kconfig options that can be used to make a different effect on driver behavior.

[1] Different ESP chip series might have different numbers of SDM channels. Please refer to Chapter GPIO and IOMUX in ESP32-S3 Technical Reference Manual for more details. The driver does not forbid you from applying for more channels, but it will return an error when all available hardware resources are used up. Please always check the return value when doing resource allocation (e.g., `sdm_new_channel()`).

Resource Allocation In ESP-IDF, the information and attributes of SDM channels are managed and accessed through specific data structures, where the data structure is called *sdm_channel_handle_t*. Each channel is capable to output the binary, hardware-generated signal with the sigma-delta modulation. The driver manages all available channels in a pool so that there is no need to manually assign a fixed channel to a GPIO.

To install an SDM channel, you should call *sdm_new_channel()* to get a channel handle. Channel-specific configurations are passed in the *sdm_config_t* structure:

- *sdm_config_t::gpio_num* sets the GPIO that the PDM pulses output from.
- *sdm_config_t::clk_src* selects the source clock for the SDM module. Note that, all channels should select the same clock source.
- *sdm_config_t::sample_rate_hz* sets the sample rate of the SDM module.
- *sdm_config_t::invert_out* sets whether to invert the output signal.
- *sdm_config_t::io_loop_back* is for debugging purposes only. It enables both the GPIO's input and output ability through the GPIO matrix peripheral.

The function *sdm_new_channel()* can fail due to various errors such as insufficient memory, invalid arguments, etc. Specifically, when there are no more free channels (i.e., all hardware SDM channels have been used up), *ESP_ERR_NOT_FOUND* will be returned.

If a previously created SDM channel is no longer required, you should recycle it by calling *sdm_del_channel()*. It allows the underlying HW channel to be used for other purposes. Before deleting an SDM channel handle, you should disable it by *sdm_channel_disable()* in advance or make sure it has not been enabled yet by *sdm_channel_enable()*.

Creating an SDM Channel with a Sample Rate of 1 MHz

```
sdm_channel_handle_t chan = NULL;
sdm_config_t config = {
    .clk_src = SDM_CLK_SRC_DEFAULT,
    .sample_rate_hz = 1 * 1000 * 1000,
    .gpio_num = 0,
};
ESP_ERROR_CHECK(sdm_new_channel(&config, &chan));
```

Enable and Disable Channel Before doing further IO control to the SDM channel, you should enable it first, by calling *sdm_channel_enable()*. Internally, this function:

- switches the channel state from **init** to **enable**
- acquires a proper power management lock if a specific clock source (e.g., APB clock) is selected. See also *Power Management* for more information.

On the contrary, calling *sdm_channel_disable()* does the opposite, that is, put the channel back to the **init** state and releases the power management lock.

Set Pulse Density For the output PDM signals, the pulse density decides the output analog voltage that is restored by a low-pass filter. The restored analog voltage from the channel is calculated by `Vout = VDD_IO / 256 * duty + VDD_IO / 2`. The range of the quantized `density` input parameter of *sdm_channel_set_pulse_density()* is from -128 to 127 (8-bit signed integer). Depending on the value of the `density` parameter, the duty cycle of the output signal will be changed accordingly. For example, if a zero value is set, then the output signal's duty will be around 50%.

Power Management When power management is enabled (i.e., *CONFIG_PM_ENABLE* is on), the system will adjust the APB frequency before going into Light-sleep, thus potentially changing the sample rate of the sigma-delta modulator.

However, the driver can prevent the system from changing APB frequency by acquiring a power management lock of type *ESP_PM_APB_FREQ_MAX*. Whenever the driver creates an SDM channel instance that has selected *SDM_CLK_SRC_APB* as its clock source, the driver guarantees that the power management lock is ac-

quired when enabling the channel by *sdm_channel_enable()*. Likewise, the driver releases the lock when *sdm_channel_disable()* is called for that channel.

IRAM Safe There is a Kconfig option *CONFIG_SDM_CTRL_FUNC_IN_IRAM* that can put commonly-used IO control functions into IRAM as well. So that these functions can also be executable when the cache is disabled. These IO control functions are listed as follows:

- *sdm_channel_set_pulse_density()*

Thread Safety The factory function *sdm_new_channel()* is guaranteed to be thread-safe by the driver, which means, the user can call it from different RTOS tasks without protection by extra locks.

The following functions are allowed to run under ISR context, the driver uses a critical section to prevent them being called concurrently in both task and ISR.

- *sdm_channel_set_pulse_density()*

Other functions that take the *sdm_channel_handle_t* as the first positional parameter, are not treated as thread-safe. This means the user should avoid calling them from multiple tasks.

Kconfig Options

- *CONFIG_SDM_CTRL_FUNC_IN_IRAM* controls where to place the SDM channel control functions (IRAM or Flash), see *IRAM Safe* for more information.
- *CONFIG_SDM_ENABLE_DEBUG_LOG* is used to enable the debug log output. Enabling this option increases the firmware binary size.

Convert to an Analog Signal (Optional)

Typically, if a Sigma-Delta signal is connected to an LED to adjust the brightness, you do not have to add any filter between them, because our eyes have their own low-pass filters for changes in light intensity. However, if you want to check the real voltage or watch the analog waveform, you need to design an analog low-pass filter. Also, it is recommended to use an active filter instead of a passive filter to gain better isolation and not lose too much voltage.

For example, you can take the following Sallen-Key topology Low Pass Filter as a reference.

Application Example

- 100 Hz sine wave that is modulated with Sigma-Delta: peripherals/sigma_delta/sdm_dac.
- LED driven by a GPIO that is modulated with Sigma-Delta: peripherals/sigma_delta/sdm_led.

API Reference

Header File

- components/esp_driver_sdm/include/driver/sdm.h
- This header file can be included with:

```
#include "driver/sdm.h"
```

- This header file is a part of the API provided by the `esp_driver_sdm` component. To declare that your component depends on `esp_driver_sdm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_sdm
```

or

```
PRIV_REQUIRES esp_driver_sdm
```

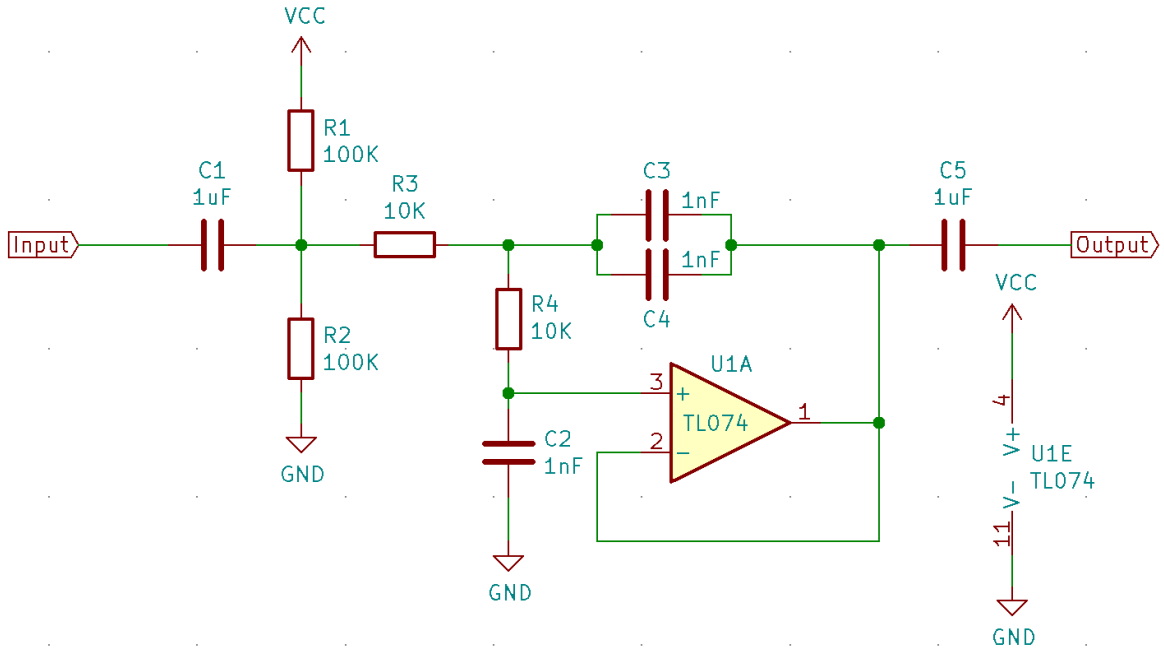


Fig. 25: Sallen-Key Low Pass Filter

Functions

esp_err_t **sdm_new_channel** (const *sdm_config_t* *config, *sdm_channel_handle_t* *ret_chan)

Create a new Sigma Delta channel.

Parameters

- **config** -- **[in]** SDM configuration
- **ret_chan** -- **[out]** Returned SDM channel handle

Returns

- ESP_OK: Create SDM channel successfully
- ESP_ERR_INVALID_ARG: Create SDM channel failed because of invalid argument
- ESP_ERR_NO_MEM: Create SDM channel failed because out of memory
- ESP_ERR_NOT_FOUND: Create SDM channel failed because all channels are used up and no more free one
- ESP_FAIL: Create SDM channel failed because of other error

esp_err_t **sdm_del_channel** (*sdm_channel_handle_t* chan)

Delete the Sigma Delta channel.

Parameters **chan** -- **[in]** SDM channel created by `sdm_new_channel`

Returns

- ESP_OK: Delete the SDM channel successfully
- ESP_ERR_INVALID_ARG: Delete the SDM channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Delete the SDM channel failed because the channel is not in init state
- ESP_FAIL: Delete the SDM channel failed because of other error

esp_err_t **sdm_channel_enable** (*sdm_channel_handle_t* chan)

Enable the Sigma Delta channel.

Note: This function will transit the channel state from init to enable.

Note: This function will acquire a PM lock, if a specific source clock (e.g. APB) is selected in the

sdm_config_t, while CONFIG_PM_ENABLE is enabled.

Parameters **chan** -- **[in]** SDM channel created by sdm_new_channel
Returns
- ESP_OK: Enable SDM channel successfully
- ESP_ERR_INVALID_ARG: Enable SDM channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Enable SDM channel failed because the channel is already enabled
- ESP_FAIL: Enable SDM channel failed because of other error

esp_err_t **sdm_channel_disable** (*sdm_channel_handle_t* chan)

Disable the Sigma Delta channel.

Note: This function will do the opposite work to the sdm_channel_enable()

Parameters **chan** -- **[in]** SDM channel created by sdm_new_channel
Returns
- ESP_OK: Disable SDM channel successfully
- ESP_ERR_INVALID_ARG: Disable SDM channel failed because of invalid argument
- ESP_ERR_INVALID_STATE: Disable SDM channel failed because the channel is not enabled yet
- ESP_FAIL: Disable SDM channel failed because of other error

esp_err_t **sdm_channel_set_pulse_density** (*sdm_channel_handle_t* chan, int8_t density)

Set the pulse density of the PDM output signal.

Note: The raw output signal requires a low-pass filter to restore it into analog voltage, the restored analog output voltage could be Vout = VDD_IO / 256 * density + VDD_IO / 2

Note: This function is allowed to run within ISR context

Note: This function will be placed into IRAM if CONFIG_SDM_CTRL_FUNC_IN_IRAM is on, so that it's allowed to be executed when Cache is disabled

---

Parameters
- **chan** -- **[in]** SDM channel created by sdm_new_channel
- **density** -- **[in]** Quantized pulse density of the PDM output signal, ranges from -128 to 127. But the range of [-90, 90] can provide a better randomness.

Returns
- ESP_OK: Set pulse density successfully
- ESP_ERR_INVALID_ARG: Set pulse density failed because of invalid argument
- ESP_FAIL: Set pulse density failed because of other error

esp_err_t **sdm_channel_set_duty** (*sdm_channel_handle_t* chan, int8_t duty)

The alias function of sdm_channel_set_pulse_density, it decides the pulse density of the output signal.

Note: sdm_channel_set_pulse_density has a more appropriate name compare this alias function, suggest to turn to sdm_channel_set_pulse_density instead

---

Parameters

- **chan** -- **[in]** SDM channel created by `sdm_new_channel`
- **duty** -- **[in]** Actually it's the quantized pulse density of the PDM output signal

Returns

- ESP_OK: Set duty cycle successfully
- ESP_ERR_INVALID_ARG: Set duty cycle failed because of invalid argument
- ESP_FAIL: Set duty cycle failed because of other error

Structures

struct **sdm_config_t**

Sigma Delta channel configuration.

Public Members

int **gpio_num**

GPIO number

sdm_clock_source_t **clk_src**

Clock source

uint32_t **sample_rate_hz**

Over sample rate in Hz, it determines the frequency of the carrier pulses

uint32_t **invert_out**

Whether to invert the output signal

uint32_t **io_loop_back**

For debug/test, the signal output from the GPIO will be fed to the input path as well

struct *sdm_config_t*::[anonymous] **flags**

Extra flags

Type Definitions

typedef struct sdm_channel_t ***sdm_channel_handle_t**

Type of Sigma Delta channel handle.

Header File

- components/hal/include/hal/sdm_types.h
- This header file can be included with:

```
#include "hal/sdm_types.h"
```

Type Definitions

typedef *soc_periph_sdm_clk_src_t* **sdm_clock_source_t**

2.6.21 SPI Flash API

Overview

The spi_flash component contains API functions related to reading, writing, erasing, and memory mapping for data in the external flash.

For higher-level API functions which work with partitions defined in the *partition table*, see *Partitions API*

Note: `esp_partition_*` APIs are recommended to be used instead of the lower level `esp_flash_*` API functions when accessing the main SPI flash chip, since they conduct bounds checking and are guaranteed to calculate correct offsets in flash based on the information in the partition table. `esp_flash_*` functions can still be used directly when accessing an external (secondary) SPI flash chip.

Different from the API before ESP-IDF v4.0, the functionality of `esp_flash_*` APIs is not limited to the "main" SPI flash chip (the same SPI flash chip from which program runs). With different chip pointers, you can access external flash chips connected to not only SPI0/1 but also other SPI buses like SPI2.

Note: Instead of going through the cache connected to the SPI0 peripheral, most `esp_flash_*` APIs go through other SPI peripherals like SPI1, SPI2, etc. This makes them able to access not only the main flash, but also external (secondary) flash.

However, due to the limitations of the cache, operations through the cache are limited to the main flash. The address range limitation for these operations is also on the cache side. The cache is not able to access external flash chips or address range above its capabilities. These cache operations include: mmap, encrypted read/write, executing code or access to variables in the flash.

Note: Flash APIs after ESP-IDF v4.0 are no longer **atomic**. If a write operation occurs during another on-going read operation, and the flash addresses of both operations overlap, the data returned from the read operation may contain both old data and new data (that was updated written by the write operation).

Note: Encrypted flash operations are only supported with the main flash chip (and not with other flash chips, that is on SPI1 with different CS, or on other SPI buses). Reading through cache is only supported on the main flash, which is determined by the HW.

Support for Features of Flash Chips

Quad/Dual Mode Chips Features of different flashes are implemented in different ways and thus need special support. The fast/slow read and Dual mode (DOUT/DIO) of almost all flashes with 24-bit address are supported, because they do not need any vendor-specific commands.

Quad mode (QIO/QOUT) is supported on the following chip types:

1. ISSI
2. GD
3. MXIC
4. FM
5. Winbond
6. XMC
7. BOYA

Note: Only when one flash series listed above is supported by ESP32-S3, this flash series is supported by the chip driver by default. You can use `Component config` > `SPI Flash driver` > `Auto-detect flash chips` in menuconfig to enable/disable a flash series.

Optional Features

Optional Features for Flash Some features are not supported on all ESP chips and Flash chips. You can check the list below for more information.

- *Auto Suspend & Resume*
- *Flash unique ID*
- *High Performance Mode of QSPI Flash Chips*
- *32-bit Address Support of QSPI Flash Chips*
- *OPI Flash Support*

Note: When Flash optional features listed in this page are used, aside from the capability of ESP chips, and ESP-IDF version you are using, you will also need to make sure these features are supported by flash chips used.

- If you are using an official Espressif modules/SiP. Some of the modules/SiPs always support the feature, in this case you can see these features listed in the datasheet. Otherwise please contact Espressif's business team to know if we can supply such products for you.
- If you are making your own modules with your own bought flash chips, and you need features listed above. Please contact your vendor if they support the those features, and make sure that the chips can be supplied continuously.

Attention: This document only shows that ESP-IDF code has supported the features of those flash chips. It is not a list of stable flash chips certified by Espressif. If you build your own hardware from flash chips with your own brought flash chips (even with flash listed in this page), you need to validate the reliability of flash chips yourself.

Auto Suspend & Resume This feature is only supported on ESP32-S3, ESP32-C2, ESP32-C3, ESP32-C6, ESP32-H2 for now.

The support for ESP32-P4 may be added in the future.

List of Flash chips that support this feature:

1. XM25QxxC series.
2. GD25QxxE series.
3. FM25Q32

Attention: There are multiple limitations about the auto-suspend feature, please do read *Flash Auto Suspend Feature* for more information before you enable this feature.

Flash Unique ID This feature is supported on all Espressif chips.

Unique ID is not flash id, which means flash has 64-Bit unique ID for each device. The instruction to read the unique ID (4Bh) accesses a factory-set read-only 64-bit number that is unique to each flash device. This ID number helps you to recognize each single device. Not all flash vendors support this feature. If you try to read the unique ID on a chip which does not have this feature, the behavior is not determined. The support list is as follows.

List of Flash chips that support this feature:

1. ISSI
2. GD
3. TH
4. FM
5. Winbond
6. XMC
7. BOYA

High Performance Mode of QSPI Flash Chips This featuer is only supported on ESP32-S3 for now.

The support for ESP32-S2, ESP32-C3, ESP32-C6, ESP32-H2, ESP32-P4 may be added in the future.

Note: This section is provided for QSPI flash chips. Octal flash used on ESP-chips support High performance mode by default so far, please refer to the *OPI Flash Support* for the list of supported octal flash chips.

High performance mode (HPM) means that the SPI1 and flash chip works under high frequency. Usually, when the operating frequency of the flash is greater than 80 MHz, it is considered that the flash works under HPM.

As far as we acknowledged, there are more than three strategies for High Performance Mode (HPM) in typical SPI flash parts. For some flash chips, HPM is controlled by dummy cycle bit in the registers, while for other chips, it can be controlled by other bits (like HPM bit) in the register, or some special command. The difference in strategies requires the driver to explicitly add support for each chip.

Attention: It is hard to create several strategies to cover all situations, so all flash chips using HPM need to be supported explicitly. Therefore, if you try to use a flash not listed in *Quad Flash HPM support list*, it might cause some error. So, when you try to use the flash chip beyond supported list, please test properly.

Moreover, when the *Dummy Cycle adjustment* strategy is adopted by the flash chip, the flash remains in a state in which DC is different from the default value after a software reset. The sub mode of HPM that adjusts the dummy cycle to run at higher frequency in the application is called *HPM-DC*. *HPM-DC* feature needs a feature *DC Aware* to be enabled in the bootloader. Otherwise different DC value will forbid the 2nd bootloader from being boot up after reset.

To enable High Performance Mode:

1. De-select *CONFIG_ESPTOOLPY_OCT_FLASH* and *CONFIG_ESPTOOLPY_FLASH_MODE_AUTO_DETECT*. HPM is not used for Octal flash, enabling related options may bypass HPM functions.
2. Enable `CONFIG_SPI_FLASH_HPM_ENA` option.
3. Switch Flash frequency to HPM ones. For example, `CONFIG_ESPTOOLPY_FLASHFREQ_120M`.
4. Make sure the config option for *HPM-DC* feature (under `CONFIG_SPI_FLASH_HPM_DC` choices) is selected correctly according to whether the bootloader supports *DC Aware*.
   - If bootloader supports *DC Aware*, select `CONFIG_SPI_FLASH_HPM_DC_AUTO`. This allows the usage of flash chips that adopted *Dummy Cycle adjustment* strategy.
   - If bootloader doesn't support *DC Aware*, select `CONFIG_SPI_FLASH_HPM_DC_DISABLE`. It avoid consequences caused by running HPM-DC with non-DC-aware bootloaders. But please avoid using flash chips that adopts *Dummy Cycle adjustment* strategy if `CONFIG_SPI_FLASH_HPM_DC_DISABLE` is selected. See list of flash models that adpot DC strategy below.

Check whether the bootloader supports *DC Aware* in the following way:

- If you are starting a new project, it's suggested to enable *DC Aware* by selecting *CONFIG_BOOTLOADER_FLASH_DC_AWARE* option in the bootloader menu. Please note that, you won't be able to modify this option via OTA, because the support is in the bootloader.
- If you are working on an existing project and want to update *HPM-DC* config option in the app via OTA, check the sdkconfig file used to build your bootloader: (Upgrading ESP-IDF version may make this file different from the one used by bootloader to build.)
  - For latest version (ESP-IDF v5.2 and above), if *CONFIG_BOOTLOADER_FLASH_DC_AWARE* is selected, the bootloader supports *DC Aware*.

  - For versions in this range: (v4.4.4+, v5.0+, and v5.1+), if `CONFIG_ESPTOOLPY_FLASHFREQ_120M` is selected, the bootloader supports *DC Aware*. In this case, enable *CONFIG_BOOTLOADER_FLASH_DC_AWARE* to confirm this (though it will not affect bootloader in devices in the field).
  - For versions below v4.4.4, the bootloader doesn't support *DC Aware*.

Quad Flash HPM support list Flash chips that don't need HPM-DC:

1. GD25Q64C (ID: 0xC84017)
2. GD25Q32C (ID: 0xC84016)
3. ZB25VQ32B (ID: 0x5E4016)
4. GD25LQ255E (ID: 0xC86019)

Following flash chips also have HPM feature, but requires the bootloader to support *DC Aware*:

1. GD25Q64E (ID: 0xC84017)
2. GD25Q128E (ID: 0xC84018)
3. XM25QH64C (ID: 0x204017)
4. XM25QH128C (ID: 0x204018)

32-bit Address Support of QSPI Flash Chips This feature is supported on all Espressif chips (see restrictions to application below).

Note: This section is provided for QSPI flash chips. The 32-bit address support of Octal Flash chips are considered as part of the Octal flash support. Please refer to the *OPI Flash Support* for the list of supported octal flash chips.

Most NOR flash chips used by Espressif chips use 24-bits address, which can cover 16 MBytes memory. However, for larger memory (usually equal to or larger than 32 MBytes), flash uses a 32-bits address to address memory region higher than 16 MBytes. Unfortunately, 32-bits address chips have vendor-specific commands, so we need to support the chips one by one.

List of Flash chips that support this feature:

1. W25Q256
2. GD25Q256

Restrictions By default, space over 16 MBytes on flash mentioned above can be used for `data saving`, like file system.

Furthermore, to map data/instructions to 32-bit physical address space (so as to be accessed by the CPU), please enable the config `IDF_EXPERIMENTAL_FEATURES` and `BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH`.

Please note that, this option is experimental, which means it can not be used on all flash chips stably. For more information, please contact Espressif Business Support.

OPI Flash Support This feature is only supporetd on ESP32-S3 for now.

OPI flash means that the flash chip supports octal peripheral interface, which has octal I/O pins. Different octal flash has different configurations and different commands. Hence, it is necessary to carefully check the support list.

Note: To know how to configure menuconfig for a board with different Flash and PSRAM, please refer to the *SPI Flash and External SPI RAM Configuration*

List of Flash chips that support this feature:

1. MX25UM25645G
2. MX25UM12345G

There are some features that are not supported by all flash chips, or not supported by all Espressif chips. These features include:

- OPI flash - means that flash supports octal mode.
- 32-bit address flash - usually means that the flash has higher capacity (equal to or larger than 16 MB) that needs longer addresses.
- High performance mode (HPM) - means that flash works under high frequency which is higher than 80 MHz.
- Flash unique ID - means that flash supports its unique 64-bit ID.
- Suspend & Resume - means that flash can accept suspend/resume command during its writing/erasing. The ESP32-S3 may keep the cache on when the flash is being written/erased and suspend it to read its contents randomly.

If you want to use these features, please ensure both ESP32-S3 and ALL flash chips in your product support these features. For more details, refer to *Optional Features for Flash*.

You may also customise your own flash chip driver. See *Overriding Default Chip Drivers* for more details.

Warning: Customizing SPI Flash Chip Drivers is considered an "expert" feature. Users should only do so at their own risk. (See the notes below)

Overriding Default Chip Drivers During the SPI Flash driver's initialization (i.e., `esp_flash_init()`), there is a chip detection step during which the driver iterates through a Default Chip Driver List and determine which chip driver can properly support the currently connected flash chip. The Default Chip Drivers are provided by the ESP-IDF, thus are updated in together with each ESP-IDF version. However ESP-IDF also allows users to customize their own chip drivers.

Users should note the following when customizing chip drivers:

1. You may need to rely on some non-public ESP-IDF functions, which have slight possibility to change between ESP-IDF versions. On the one hand, these changes may be useful bug fixes for your driver, on the other hand, they may also be breaking changes (i.e., breaks your code).
2. Some ESP-IDF bug fixes to other chip drivers are not automatically applied to your own custom chip drivers.
3. If the protection of flash is not handled properly, there may be some random reliability issues.
4. If you update to a newer ESP-IDF version that has support for more chips, you will have to manually add those new chip drivers into your custom chip driver list. Otherwise the driver will only search for the drivers in custom list you provided.

Steps For Creating Custom Chip Drivers and Overriding the ESP-IDF Default Driver List

1. Enable the *CONFIG_SPI_FLASH_OVERRIDE_CHIP_DRIVER_LIST* config option. This prevents compilation and linking of the Default Chip Driver List (`default_registered_chips`) provided by ESP-IDF. Instead, the linker searches for the structure of the same name (`default_registered_chips`) that must be provided by the user.
2. Add a new component in your project, e.g., `custom_chip_driver`.
3. Copy the necessary chip driver files from the `spi_flash` component in ESP-IDF. This may include:
   - `spi_flash_chip_drivers.c` (to provide the `default_registered_chips` structure)
   - Any of the `spi_flash_chip_*.c` files that matches your own flash model best
   - `CMakeLists.txt` and `linker.lf` files

   Modify the files above properly. Including:
   - Change the `default_registered_chips` variable to non-static and remove the #ifdef logic around it.
   - Update `linker.lf` file to rename the fragment header and the library name to match the new component.
   - If reusing other drivers, some header names need prefixing with `spi_flash/` when included from outside spi_flash component.

Note:

- When writing your own flash chip driver, you can set your flash chip capabilities through `spi_flash_chip_***(vendor)_get_caps` and points the function pointer `get_chip_caps` for protection to the `spi_flash_chip_***_get_caps` function. The steps are as follows.
    1. Please check whether your flash chip have the capabilities listed in `spi_flash_caps_t` by checking the flash datasheet.
    2. Write a function named `spi_flash_chip_***(vendor)_get_caps`. Take the example below as a reference. (if the flash support `suspend` and `read unique id`).
    3. Points the pointer `get_chip_caps` (in `spi_flash_chip_t`) to the function mentioned above.

```
spi_flash_caps_t spi_flash_chip_***(vendor)_get_caps(esp_flash_t *chip)
{
    spi_flash_caps_t caps_flags = 0;
    // 32-bit-address flash is not supported
    flash-suspend is supported
    caps_flags |= SPI_FLAHS_CHIP_CAP_SUSPEND;
    // flash read unique id.
    caps_flags |= SPI_FLASH_CHIP_CAP_UNIQUE_ID;
    return caps_flags;
}
```

```
const spi_flash_chip_t esp_flash_chip_eon = {
    // Other function pointers
    .get_chip_caps = spi_flash_chip_eon_get_caps,
};
```

- You also can see how to implement this in the example storage/custom_flash_driver.

4. Write a new `CMakeLists.txt` file for the `custom_chip_driver` component, including an additional line to add a linker dependency from `spi_flash` to `custom_chip_driver`:

```
idf_component_register(SRCS "spi_flash_chip_drivers.c"
                       "spi_flash_chip_mychip.c"  # modify as needed
                       REQUIRES hal
                       PRIV_REQUIRES spi_flash
                       LDFRAGMENTS linker.lf)
idf_component_add_link_dependency(FROM spi_flash)
```

    - An example of this component CMakeLists.txt can be found in storage/custom_flash_driver/components/custom_chip_driver/CMakeLists.txt
5. The `linker.lf` is used to put every chip driver that you are going to use whilst cache is disabled into internal RAM. See *Linker Script Generation* for more details. Make sure this file covers all the source files that you add.
6. Build your project, and you will see the new flash driver is used.

Example See also storage/custom_flash_driver.

Initializing a Flash Device

To use the `esp_flash_*` APIs, you need to initialise a flash chip on a certain SPI bus, as shown below:

1. Call *`spi_bus_initialize()`* to properly initialize an SPI bus. This function initializes the resources (I/O, DMA, interrupts) shared among devices attached to this bus.
2. Call *`spi_bus_add_flash_device()`* to attach the flash device to the bus. This function allocates memory and fills the members for the `esp_flash_t` structure. The CS I/O is also initialized here.
3. Call *`esp_flash_init()`* to actually communicate with the chip. This also detects the chip type, and influence the following operations.

Note: Multiple flash chips can be attached to the same bus now.

SPI Flash Access API

This is the set of API functions for working with data in flash:

- *esp_flash_read()* reads data from flash to RAM
- *esp_flash_write()* writes data from RAM to flash
- *esp_flash_erase_region()* erases specific region of flash
- *esp_flash_erase_chip()* erases the whole flash
- esp_flash_get_chip_size() returns flash chip size, in bytes, as configured in menuconfig

Generally, try to avoid using the raw SPI flash functions to the "main" SPI flash chip in favour of *partition-specific functions*.

SPI Flash Size

The SPI flash size is configured by writing a field in the software bootloader image header, flashed at offset 0x1000.

By default, the SPI flash size is detected by esptool.py when this bootloader is written to flash, and the header is updated with the correct size. Alternatively, it is possible to generate a fixed flash size by setting *CONFIG_ESPTOOLPY_FLASHSIZE* in the project configuration.

If it is necessary to override the configured flash size at runtime, it is possible to set the chip_size member of the g_rom_flashchip structure. This size is used by esp_flash_* functions (in both software & ROM) to check the bounds.

Concurrency Constraints for Flash on SPI1

Concurrency Constraints for Flash on SPI1

The SPI0/1 bus is shared between the instruction & data cache (for firmware execution) and the SPI1 peripheral (controlled by the drivers including this SPI Flash driver). Hence, operations to SPI1 will cause significant influence to the whole system. This kind of operations include calling SPI Flash API or other drivers on SPI1 bus, any operations like read/write/erase or other user defined SPI operations, regardless to the main flash or other SPI slave devices.

On ESP32-S3, the config option *CONFIG_SPI_FLASH_AUTO_SUSPEND* allows the cache to read flash concurrently with SPI1 operations. This is an optional feature that depends on special SPI Flash models, hence disabled by default. See *Flash Auto Suspend Feature* for more details.

If this option is disabled, the caches must be disabled while reading/writing/erasing operations. There are some constraints using driver on the SPI1 bus, see *When the Caches Are Disabled*. These constraints will cause more IRAM/DRAM usages.

On ESP32-S3, the config options *CONFIG_SPIRAM_XIP_FROM_PSRAM* (disabled by default) allows the cache to read/write PSRAM concurrently with SPI1 operations. See *XIP from PSRAM Feature* for more details.

If these options are disabled, the caches must be disabled while reading/writing/erasing operations. There are some constraints using driver on the SPI1 bus, see *When the Caches Are Disabled*. These constraints will cause more IRAM/DRAM usages.

When the Caches Are Disabled Under this condition, all CPUs should always execute code and access data from internal RAM. The APIs documented in this file will disable the caches automatically and transparently.

Note: When *CONFIG_SPIRAM_XIP_FROM_PSRAM* is enabled, these APIs will not disable the caches.

The way that these APIs disable the caches suspends all the other tasks. Besides, all non-IRAM-safe interrupts will be disabled. The other core will be polling in a busy loop. These will be restored until the Flash operation completes.

See also *OS Functions* and *SPI Bus Lock*.

There are no such constraints and impacts for flash chips on other SPI buses than SPI0/1.

For differences between internal RAM (e.g., IRAM, DRAM) and flash cache, please refer to the *application memory layout* documentation.

IRAM-Safe Interrupt Handlers For interrupt handlers which need to execute when the cache is disabled (e.g., for low latency operations), set the `ESP_INTR_FLAG_IRAM` flag when the *interrupt handler is registered*.

You must ensure that all data and functions accessed by these interrupt handlers, including the ones that handlers call, are located in IRAM or DRAM. See *How to Place Code in IRAM*.

If a function or symbol is not correctly put into IRAM/DRAM, and the interrupt handler reads from the flash cache during a flash operation, it will cause a crash due to Illegal Instruction exception (for code which should be in IRAM) or garbage data to be read (for constant data which should be in DRAM).

Note: When working with strings in ISRs, it is not advised to use `printf` and other output functions. For debugging purposes, use `ESP_DRAM_LOGE()` and similar macros when logging from ISRs. Make sure that both `TAG` and format string are placed into `DRAM` in that case.

Non-IRAM-Safe Interrupt Handlers If the `ESP_INTR_FLAG_IRAM` flag is not set when registering, the interrupt handler will not get executed when the caches are disabled. Once the caches are restored, the non-IRAM-safe interrupts will be re-enabled. After this moment, the interrupt handler will run normally again. This means that as long as caches are disabled, users will not see the corresponding hardware event happening.

Flash Auto Suspend Feature

Important:

1. The flash chip you are using should have a suspend/resume feature.
2. The MSPI hardware should support the auto-suspend feature, i.e., hardware can send suspend command automatically.

If you use suspend feature on an unsupported chip, it may cause a severe crash. Therefore, we strongly suggest you reading the flash chip datasheets first. Ensure the flash chip satisfies the following conditions at minimum.

1. With the current software implementation, SUS bit in status registers should in SR2 bit7 or SR bit15.
2. With the current software implementation, suspend command should be 75H, with resume command being 7AH.
3. When the flash is successfully suspended, all address of the flash, except from the section/block being erased, can be read correctly. At this state , resume can be sent immediately as well.
4. When the flash is successfully resumed, another suspend can be sent immediately at this state.

When *CONFIG_SPI_FLASH_AUTO_SUSPEND* is enabled, the caches will be kept enabled. They would be disabled if *CONFIG_SPI_FLASH_AUTO_SUSPEND* is disabled. The hardware handles the arbitration between SPI0 and SPI1. If the SPI1 operation is short, such as a reading operation, the CPU and the cache will wait until the SPI1 operation is completed. However, during processes like erasing, page programming, or status register writing (e.g., `SE`, `PP`, and `WRSR`), an auto suspend will happen, interrupting the ongoing flash operation. This allows the CPU to access data from the cache and flash within limited time.

This approach allows certain code/variables to be stored in flash/PSRAM instead of IRAM/DRAM, while still being executable during flash erasing. This reduces the usage of IRAM/DRAM.

Please note that this feature comes with the overhead of flash suspend/resume. Frequent interruptions during flash erasing can significantly prolong the erasing process. To avoid this, you may use FreeRTOS task priorities to ensure

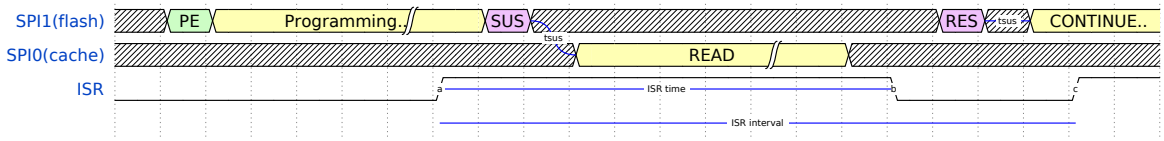
that only real-time critical tasks are executed at a higher priority than flash erasing, allowing the flash erasing to complete in reasonable time.

There are three kinds of code:

1. Critical code: inside IRAM/DRAM. This kind of code usually has high performance requirements, related to cache/flash/PSRAM, or is called very often.
2. Cached code: inside flash/PSRAM. This kind of code has lower performance requirements or is called less often. They will execute during erasing, with some overhead.
3. Low-priority code: inside flash/PSRAM and disabled during erasing. This kind of code should be forbidden from being executed to avoid affecting the flash erasing, by setting a lower task priority than the erasing task.

Regarding the flash suspend feature usage and corresponding response time delay, please also see the system/flash_suspend example.

Note: The flash auto suspend feature relies heavily on strict timing. You can see it as a kind of optimization plan, which means that you cannot use it in every situation, like high requirement of real-time system or triggering interrupt very frequently (e.g., LCD flush, bluetooth, Wi-Fi, etc.). You should take following steps to evaluate what kind of ISR can be used together with flash suspend.



As you can see from the diagram, two key values should be noted:

1. ISR time: The ISR time cannot be very long, at least not larger than the value you set in `IWDT`. But it will significantly lengthen the erasing/writing completion time.
2. ISR interval: ISR cannot be triggered very often. The most important time is the **ISR interval minus ISR time** (from point b to point c in the diagram). During this time, SPI1 will send resume command to restart the operation. However, it needs a time `tsus` for preparation, and the typical value of `tsus` is about **40 us**. If SPI1 cannot resume the operation but another suspend command comes, it will cause CPU starve and `TWDT` may be triggered.

The `tsus` time mentioned in point 2 can be found by looking through the flash datasheets, usually in the AC CHARACTERISTICS section. Users needs to make sure that the `tsus` value obtained from the datasheets is not greater than the *CONFIG_SPI_FLASH_SUSPEND_TSUS_VAL_US* value in Kconfig.

Furthermore, the flash suspend might be delayed. If both the CPU and the cache access the flash via SPI0 frequently and SPI1 sends the suspend command frequently as well, the efficiency of MSPI data transfer will be influenced. So, we have a **lock** inside to prevent this. When SPI1 sends the suspend command, SPI0 will take over memory SPI bus and take the lock. After SPI0 finishes sending data, it will retain control of the memory SPI bus until the lock delay period time finishes. During this lock delay period, if there is any other SPI0 transaction, then the SPI0 transaction will be proceeded and a new lock delay period will start. Otherwise, SPI0 will release the memory bus and start SPI0/1 arbitration.

XIP from PSRAM Feature If *CONFIG_SPIRAM_XIP_FROM_PSRAM* is enabled, the flash `.text` sections (used for instructions) and the flash `.rodata` sections (used for read only data) will be placed in PSRAM.

The corresponding virtual memory range will be mapped to PSRAM.

If both of the above options are enabled, the Cache won't be disabled during an SPI1 Flash operation. You don't need to make sure ISRs, ISR callbacks and involved data are placed in internal RAM.

Attention: The SPI0/1 bus is shared between the instruction & data cache (for firmware execution) and the SPI1 peripheral (controlled by the drivers including this SPI flash driver). Hence, calling SPI Flash API on SPI1 bus (including the main flash) causes significant influence to the whole system. See *Concurrency Constraints for Flash on SPI1* for more details.

SPI Flash Encryption

It is possible to encrypt the contents of SPI flash and have it transparently decrypted by hardware.

Refer to the *Flash Encryption documentation* for more details.

Memory Mapping API

ESP32-S3 features memory hardware which allows regions of flash memory to be mapped into instruction and data address spaces. This mapping works only for read operations. It is not possible to modify contents of flash memory by writing to a mapped memory region.

Mapping happens in 64 KB pages. Memory mapping hardware can map flash into the data address space and the instruction address space. See the technical reference manual for more details and limitations about memory mapping hardware.

Note that some pages are used to map the application itself into memory, so the actual number of available pages may be less than the capability of the hardware.

Reading data from flash using a memory mapped region is the only way to decrypt contents of flash when *flash encryption* is enabled. Decryption is performed at the hardware level.

Memory mapping API are declared in `spi_flash_mmap.h` and `esp_partition.h`:

- *`spi_flash_mmap()`* maps a region of physical flash addresses into instruction space or data space of the CPU.
- *`spi_flash_munmap()`* unmaps previously mapped region.
- *`esp_partition_mmap()`* maps part of a partition into the instruction space or data space of the CPU.

Differences between *`spi_flash_mmap()`* and *`esp_partition_mmap()`* are as follows:

- *`spi_flash_mmap()`* must be given a 64 KB aligned physical address.
- *`esp_partition_mmap()`* may be given any arbitrary offset within the partition. It adjusts the returned pointer to mapped memory as necessary.

Note that since memory mapping happens in pages, it may be possible to read data outside of the partition provided to `esp_partition_mmap`, regardless of the partition boundary.

Note: mmap is supported by cache, so it can only be used on main flash.

SPI Flash Implementation

The `esp_flash_t` structure holds chip data as well as three important parts of this API:

1. The host driver, which provides the hardware support to access the chip;
2. The chip driver, which provides compatibility service to different chips;
3. The OS functions, provide support of some OS functions (e.g., lock, delay) in different stages (1st/2nd boot, or the app).

Host Driver The host driver relies on an interface (`spi_flash_host_driver_t`) defined in the `spi_flash_types.h` (in the `hal/include/hal` folder). This interface provides some common functions to communicate with the chip.

In other files of the SPI HAL, some of these functions are implemented with existing ESP32-S3 memory-spi functionalities. However, due to the speed limitations of ESP32-S3, the HAL layer cannot provide high-speed implementations to some reading commands (so the support for it was dropped). The files (`memspi_host_driver.h` and `.c`) implement the high-speed version of these commands with the `common_command` function provided in the HAL, and wrap these functions as `spi_flash_host_driver_t` for upper layer to use.

You can also implement your own host driver, even with the GPIO. As long as all the functions in the `spi_flash_host_driver_t` are implemented, the esp_flash API can access the flash regardless of the low-level hardware.

Chip Driver The chip driver, defined in `spi_flash_chip_driver.h`, wraps basic functions provided by the host driver for the API layer to use.

Some operations need some commands to be sent first, or read some status afterwards. Some chips need different commands or values, or need special communication ways.

There is a type of chip called `generic chip` which stands for common chips. Other special chip drivers can be developed on the base of the generic chip.

The chip driver relies on the host driver.

OS Functions

SPI Features

SPI Master

SPI Bus Lock To realize the multiplexing of different devices from different drivers, including SPI Master, SPI Flash, etc., an SPI bus lock is applied on each SPI bus. Drivers can attach their devices to the bus with the arbitration of the lock.

Each bus lock is initialized with a BG (background) service registered. All devices that request transactions on the bus should wait until the BG is successfully disabled.

- For the SPI1 bus, the BG is the cache. The bus lock disables the cache before device operations start, and enables it again after the device releases the lock. No devices on SPI1 are allowed to use ISR, since it is meaningless for the task to yield to other tasks when the cache is disabled.
  The SPI Master driver has not supported SPI1 bus. Only the SPI Flash driver can attach to the bus.
- For other buses, the driver can register the ISR as a BG. If a device task requests exclusive bus access, the bus lock will block the task, disable the ISR, and then unblock the task. After the task releases the lock, the lock will try to re-enable the ISR if there are still pending transactions in the ISR.

Currently the OS function layer provides entries of a lock and delay.

The lock (see *SPI Bus Lock*) is used to resolve the conflicts among the access of devices on the same SPI bus, and the SPI Flash chip access. E.g.

1. On SPI1 bus, the cache (used to fetch the data (code) in the Flash and PSRAM) should be disabled when the flash chip on the SPI0/1 is being accessed.
2. On the other buses, the flash driver needs to disable the ISR registered by SPI Master driver, to avoid conflicts.
3. Some devices of SPI Master driver may require to use the bus monopolized during a period (especially when the device does not have a CS wire, or the wire is controlled by software like SDSPI driver).

The delay is used by some long operations which requires the master to wait or polling periodically.

The top API wraps these the chip driver and OS functions into an entire component, and also provides some argument checking.

OS functions can also help to avoid a watchdog timeout when erasing large flash areas. During this time, the CPU is occupied with the flash erasing task. This stops other tasks from being executed. Among these tasks is the idle task to feed the watchdog timer (WDT). If the configuration option *CONFIG_ESP_TASK_WDT_PANIC* is selected and the flash operation time is longer than the watchdog timeout period, the system will reboot.

It is pretty hard to totally eliminate this risk, because the erasing time varies with different flash chips, making it hard to be compatible in flash drivers. Therefore, users need to pay attention to it. Please use the following guidelines:

1. It is recommended to enable the *CONFIG_SPI_FLASH_YIELD_DURING_ERASE* option to allow the scheduler to re-schedule during erasing flash memory. Besides, following parameters can also be used.

- Increase *CONFIG_SPI_FLASH_ERASE_YIELD_TICKS* or decrease *CONFIG_SPI_FLASH_ERASE_YIELD_DURATION_MS* in menuconfig.
- You can also increase *CONFIG_ESP_TASK_WDT_TIMEOUT_S* in menuconfig for a larger watchdog timeout period. However, with larger watchdog timeout period, previously detected timeouts may no longer be detected.

2. Please be aware of the consequences of enabling the *CONFIG_ESP_TASK_WDT_PANIC* option when doing long-running SPI flash operations which triggers the panic handler when it times out. However, this option can also help dealing with unexpected exceptions in your application. Please decide whether this is needed to be enabled according to actual condition.
3. During your development, please carefully review the actual flash operation according to the specific requirements and time limits on erasing flash memory of your projects. Always allow reasonable redundancy based on your specific product requirements when configuring the flash erasing timeout threshold, thus improving the reliability of your product.

Implementation Details

In order to perform some flash operations, it is necessary to make sure that both CPUs are not running any code from flash for the duration of the flash operation:

- In a single-core setup, the SDK needs to disable interrupts or scheduler before performing the flash operation.
- In a dual-core setup, the SDK needs to make sure that both CPUs are not running any code from flash.

When SPI flash API is called on CPU A (can be PRO or APP), start the `spi_flash_op_block_func` function on CPU B using the `esp_ipc_call` API. This API wakes up a high priority task on CPU B and tells it to execute a given function, in this case, `spi_flash_op_block_func`. This function disables cache on CPU B and signals that the cache is disabled by setting the `s_flash_op_can_start` flag. Then the task on CPU A disables cache as well and proceeds to execute flash operation.

While a flash operation is running, interrupts can still run on CPUs A and B. It is assumed that all interrupt code is placed into RAM. Once the interrupt allocation API is added, a flag should be added to request the interrupt to be disabled for the duration of a flash operations.

Once the flash operation is complete, the function on CPU A sets another flag, `s_flash_op_complete`, to let the task on CPU B know that it can re-enable cache and release the CPU. Then the function on CPU A re-enables the cache on CPU A as well and returns control to the calling code.

Additionally, all API functions are protected with a mutex (`s_flash_op_mutex`).

In a single core environment (*CONFIG_FREERTOS_UNICORE* enabled), you need to disable both caches, so that no inter-CPU communication can take place.

Related Documents

- *Optional Features for Flash*
- *Concurrency Constraints for Flash on SPI1*
- *SPI Flash API ESP-IDF Version vs Chip-ROM Version*

SPI Flash API ESP-IDF Version vs Chip-ROM Version There is a set of SPI Flash drivers in Chip-ROM which you can use by enabling *CONFIG_SPI_FLASH_ROM_IMPL*. Most of the ESP-IDF SPI Flash driver code are in internal RAM, therefore enabling this option frees some internal RAM usage. Note if you enable this option, this means some SPI Flash driver features and bugfixes that are done in ESP-IDF might not be included in the Chip-ROM version.

Feature Supported by ESP-IDF but Not in Chip-ROM

- Octal Flash chip support. See *OPI Flash Support* for details.
- 32-bit-address support for GD25Q256. Note this feature is an optional feature, please do read *32-bit Address Support of QSPI Flash Chips* for details.
- TH Flash chip support.
- Kconfig option *CONFIG_SPI_FLASH_CHECK_ERASE_TIMEOUT_DISABLED*.
- *CONFIG_SPI_FLASH_VERIFY_WRITE*, enabling this option helps you detect bad writing.
- *CONFIG_SPI_FLASH_LOG_FAILED_WRITE*, enabling this option prints the bad writing.
- *CONFIG_SPI_FLASH_WARN_SETTING_ZERO_TO_ONE*, enabling this option checks if you are writing zero to one.
- *CONFIG_SPI_FLASH_DANGEROUS_WRITE*, enabling this option checks for flash programming to certain protected regions like bootloader, partition table or application itself.
- *CONFIG_SPI_FLASH_ENABLE_COUNTERS*, enabling this option to collect performance data for ESP-IDF SPI Flash driver APIs.
- *CONFIG_SPI_FLASH_AUTO_SUSPEND*, enabling this option to automatically suspend / resume a long Flash operation when short Flash operation happens. Note this feature is an optional feature, please do read *Auto Suspend & Resume* for more limitations.

Bugfixes Introduced in ESP-IDF but Not in Chip-ROM

- Detected Flash physical size correctly, for larger than 256MBit Flash chips. (Commit ID: b4964279d44f73cce7cfd5cf684567fbdfd6fd9e)
- Fixed issue that 4-line Flash encryption cannot work normally when 8-line PSRAM enabled. (Commit ID: 683d92bc884e0f2a7eebea40a551cf05f0c28256)
- Fixed issue that only 128KB virtual address ranges can be mapped to instructions on Flash.
- Fixed issue that only 16MB virtual address ranges can be mapped to read-only data on Flash.

API Reference - SPI Flash

Header File

- components/spi_flash/include/esp_flash_spi_init.h
- This header file can be included with:

```
#include "esp_flash_spi_init.h"
```

- This header file is a part of the API provided by the `spi_flash` component. To declare that your component depends on `spi_flash`, add the following to your CMakeLists.txt:

```
REQUIRES spi_flash
```

or

```
PRIV_REQUIRES spi_flash
```

Functions

esp_err_t **spi_bus_add_flash_device**(*esp_flash_t* **out_chip, const *esp_flash_spi_device_config_t* *config)

Add a SPI Flash device onto the SPI bus.

The bus should be already initialized by `spi_bus_initialization`.

Parameters

- **out_chip** -- Pointer to hold the initialized chip.
- **config** -- Configuration of the chips to initialize.

Returns

- ESP_ERR_INVALID_ARG: out_chip is NULL, or some field in the config is invalid.
- ESP_ERR_NO_MEM: failed to allocate memory for the chip structures.
- ESP_OK: success.

esp_err_t **spi_bus_remove_flash_device**(*esp_flash_t* *chip)

Remove a SPI Flash device from the SPI bus.

Parameters **chip** -- The flash device to remove.

Returns

- ESP_ERR_INVALID_ARG: The chip is invalid.
- ESP_OK: success.

Structures

struct **esp_flash_spi_device_config_t**

Configurations for the SPI Flash to init.

Public Members

spi_host_device_t **host_id**

Bus to use.

int **cs_io_num**

GPIO pin to output the CS signal.

esp_flash_io_mode_t **io_mode**

IO mode to read from the Flash.

enum *esp_flash_speed_s* **speed**

Speed of the Flash clock. Replaced by freq_mhz.

int **input_delay_ns**

Input delay of the data pins, in ns. Set to 0 if unknown.

int **cs_id**

CS line ID, ignored when not `host_id` is not SPI1_HOST, or `CONFIG_SPI_FLASH_SHARE_SPI1_BUS` is enabled. In this case, the CS line used is automatically assigned by the SPI bus lock.

int **freq_mhz**

The frequency of flash chip(MHZ)

Header File

- components/spi_flash/include/esp_flash.h
- This header file can be included with:

```
#include "esp_flash.h"
```

- This header file is a part of the API provided by the `spi_flash` component. To declare that your component depends on `spi_flash`, add the following to your CMakeLists.txt:

```
REQUIRES spi_flash
```

or

```
PRIV_REQUIRES spi_flash
```

Functions

esp_err_t **esp_flash_init**(*esp_flash_t* *chip)

Initialise SPI flash chip interface.

This function must be called before any other API functions are called for this chip.

Note: Only the `host` and `read_mode` fields of the chip structure must be initialised before this function is called. Other fields may be auto-detected if left set to zero or NULL.

Note: If the chip->drv pointer is NULL, chip chip_drv will be auto-detected based on its manufacturer & product IDs. See `esp_flash_registered_flash_drivers` pointer for details of this process.

Parameters **chip** -- Pointer to SPI flash chip to use. If NULL, esp_flash_default_chip is substituted.

Returns ESP_OK on success, or a flash error code if initialisation fails.

bool **esp_flash_chip_driver_initialized**(const *esp_flash_t* *chip)

Check if appropriate chip driver is set.

Parameters **chip** -- Pointer to SPI flash chip to use. If NULL, esp_flash_default_chip is substituted.

Returns true if set, otherwise false.

esp_err_t **esp_flash_read_id**(*esp_flash_t* *chip, uint32_t *out_id)

Read flash ID via the common "RDID" SPI flash command.

ID is a 24-bit value. Lower 16 bits of 'id' are the chip ID, upper 8 bits are the manufacturer ID.

Parameters

- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **out_id** -- **[out]** Pointer to receive ID value.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_get_size**(*esp_flash_t* *chip, uint32_t *out_size)

Detect flash size based on flash ID.

Note: 1. Most flash chips use a common format for flash ID, where the lower 4 bits specify the size as a power of 2. If the manufacturer doesn't follow this convention, the size may be incorrectly detected.

a. The out_size returned only stands for The out_size stands for the size in the binary image header. If you want to get the real size of the chip, please call `esp_flash_get_physical_size` instead.

Parameters

- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **out_size** -- **[out]** Detected size in bytes, standing for the size in the binary image header.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_get_physical_size**(*esp_flash_t* *chip, uint32_t *flash_size)

Detect flash size based on flash ID.

Note: Most flash chips use a common format for flash ID, where the lower 4 bits specify the size as a power of 2. If the manufacturer doesn't follow this convention, the size may be incorrectly detected.

Parameters
- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **flash_size** -- **[out]** Detected size in bytes.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_read_unique_chip_id**(*esp_flash_t* *chip, uint64_t *out_id)

Read flash unique ID via the common "RDUID" SPI flash command.

ID is a 64-bit value.

Note: This is an optional feature, which is not supported on all flash chips. READ PROGRAMMING GUIDE FIRST!

Parameters
- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init().
- **out_id** -- **[out]** Pointer to receive unique ID value.

Returns
- ESP_OK on success, or a flash error code if operation failed.
- ESP_ERR_NOT_SUPPORTED if the chip doesn't support read id.

esp_err_t **esp_flash_erase_chip**(*esp_flash_t* *chip)

Erase flash chip contents.

Parameters **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()

Returns
- ESP_OK on success,
- ESP_ERR_NOT_SUPPORTED if the chip is not able to perform the operation. This is indicated by WREN = 1 after the command is sent.
- ESP_ERR_NOT_ALLOWED if a read-only partition is present.
- Other flash error code if operation failed.

esp_err_t **esp_flash_erase_region**(*esp_flash_t* *chip, uint32_t start, uint32_t len)

Erase a region of the flash chip.

Sector size is specifyed in chip->drv->sector_size field (typically 4096 bytes.) ESP_ERR_INVALID_ARG will be returned if the start & length are not a multiple of this size.

Erase is performed using block (multi-sector) erases where possible (block size is specified in chip->drv->block_erase_size field, typically 65536 bytes). Remaining sectors are erased using individual sector erase commands.

Parameters
- **chip** -- Pointer to identify flash chip. If NULL, esp_flash_default_chip is substituted. Must have been successfully initialised via esp_flash_init()
- **start** -- Address to start erasing flash. Must be sector aligned.
- **len** -- Length of region to erase. Must also be sector aligned.

Returns
- ESP_OK on success,

- ESP_ERR_NOT_SUPPORTED if the chip is not able to perform the operation. This is indicated by WREN = 1 after the command is sent.
- ESP_ERR_NOT_ALLOWED if the address range (start –start + len) overlaps with a read-only partition address space
- Other flash error code if operation failed.

esp_err_t **esp_flash_get_chip_write_protect** (*esp_flash_t* *chip, bool *write_protected)

Read if the entire chip is write protected.

Note: A correct result for this flag depends on the SPI flash chip model and chip_drv in use (via the 'chip->drv' field).

Parameters

- **chip** -- Pointer to identify flash chip. If NULL, esp_flash_default_chip is substituted. Must have been successfully initialised via esp_flash_init()
- **write_protected** -- **[out]** Pointer to boolean, set to the value of the write protect flag.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_set_chip_write_protect** (*esp_flash_t* *chip, bool write_protect)

Set write protection for the SPI flash chip.

Some SPI flash chips may require a power cycle before write protect status can be cleared. Otherwise, write protection can be removed via a follow-up call to this function.

Note: Correct behaviour of this function depends on the SPI flash chip model and chip_drv in use (via the 'chip->drv' field).

Parameters

- **chip** -- Pointer to identify flash chip. If NULL, esp_flash_default_chip is substituted. Must have been successfully initialised via esp_flash_init()
- **write_protect** -- Boolean value for the write protect flag

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_get_protectable_regions** (const *esp_flash_t* *chip, const *esp_flash_region_t* **out_regions, uint32_t *out_num_regions)

Read the list of individually protectable regions of this SPI flash chip.

Note: Correct behaviour of this function depends on the SPI flash chip model and chip_drv in use (via the 'chip->drv' field).

Parameters

- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **out_regions** -- **[out]** Pointer to receive a pointer to the array of protectable regions of the chip.
- **out_num_regions** -- **[out]** Pointer to an integer receiving the count of protectable regions in the array returned in 'regions'.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_get_protected_region**(*esp_flash_t* *chip, const *esp_flash_region_t* *region, bool *out_protected)

Detect if a region of the SPI flash chip is protected.

Note: It is possible for this result to be false and write operations to still fail, if protection is enabled for the entire chip.

Note: Correct behaviour of this function depends on the SPI flash chip model and chip_drv in use (via the 'chip->drv' field).

Parameters

- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **region** -- Pointer to a struct describing a protected region. This must match one of the regions returned from esp_flash_get_protectable_regions(...).
- **out_protected** -- **[out]** Pointer to a flag which is set based on the protected status for this region.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_set_protected_region**(*esp_flash_t* *chip, const *esp_flash_region_t* *region, bool protect)

Update the protected status for a region of the SPI flash chip.

Note: It is possible for the region protection flag to be cleared and write operations to still fail, if protection is enabled for the entire chip.

Note: Correct behaviour of this function depends on the SPI flash chip model and chip_drv in use (via the 'chip->drv' field).

Parameters

- **chip** -- Pointer to identify flash chip. Must have been successfully initialised via esp_flash_init()
- **region** -- Pointer to a struct describing a protected region. This must match one of the regions returned from esp_flash_get_protectable_regions(...).
- **protect** -- Write protection flag to set.

Returns ESP_OK on success, or a flash error code if operation failed.

esp_err_t **esp_flash_read**(*esp_flash_t* *chip, void *buffer, uint32_t address, uint32_t length)

Read data from the SPI flash chip.

There are no alignment constraints on buffer, address or length.

Note: If on-chip flash encryption is used, this function returns raw (ie encrypted) data. Use the flash cache to transparently decrypt data.

Parameters

- **chip** -- Pointer to identify flash chip. If NULL, esp_flash_default_chip is substituted. Must have been successfully initialised via esp_flash_init()

- **buffer** -- Pointer to a buffer where the data will be read. To get better performance, this should be in the DRAM and word aligned.
- **address** -- Address on flash to read from. Must be less than chip->size field.
- **length** -- Length (in bytes) of data to read.

Returns

- ESP_OK: success
- ESP_ERR_NO_MEM: Buffer is in external PSRAM which cannot be concurrently accessed, and a temporary internal buffer could not be allocated.
- or a flash error code if operation failed.

esp_err_t **esp_flash_write**(*esp_flash_t* *chip, const void *buffer, uint32_t address, uint32_t length)

Write data to the SPI flash chip.

There are no alignment constraints on buffer, address or length.

Parameters

- **chip** -- Pointer to identify flash chip. If NULL, esp_flash_default_chip is substituted. Must have been successfully initialised via esp_flash_init()
- **address** -- Address on flash to write to. Must be previously erased (SPI NOR flash can only write bits 1->0).
- **buffer** -- Pointer to a buffer with the data to write. To get better performance, this should be in the DRAM and word aligned.
- **length** -- Length (in bytes) of data to write.

Returns

- ESP_OK on success
- ESP_FAIL, bad write, this will be detected only when CONFIG_SPI_FLASH_VERIFY_WRITE is enabled
- ESP_ERR_NOT_SUPPORTED if the chip is not able to perform the operation. This is indicated by WREN = 1 after the command is sent.
- ESP_ERR_NOT_ALLOWED if the address range (address –address + length) overlaps with a read-only partition address space
- Other flash error code if operation failed.

esp_err_t **esp_flash_write_encrypted**(*esp_flash_t* *chip, uint32_t address, const void *buffer, uint32_t length)

Encrypted and write data to the SPI flash chip using on-chip hardware flash encryption.

Note: Both address & length must be 16 byte aligned, as this is the encryption block size

---

Parameters

- **chip** -- Pointer to identify flash chip. Must be NULL (the main flash chip). For other chips, encrypted write is not supported.
- **address** -- Address on flash to write to. 16 byte aligned. Must be previously erased (SPI NOR flash can only write bits 1->0).
- **buffer** -- Pointer to a buffer with the data to write.
- **length** -- Length (in bytes) of data to write. 16 byte aligned.

Returns

- ESP_OK: on success
- ESP_FAIL: bad write, this will be detected only when CONFIG_SPI_FLASH_VERIFY_WRITE is enabled
- ESP_ERR_NOT_SUPPORTED: encrypted write not supported for this chip.
- ESP_ERR_INVALID_ARG: Either the address, buffer or length is invalid.
- ESP_ERR_NOT_ALLOWED if the address range (address –address + length) overlaps with a read-only partition address space

esp_err_t **esp_flash_read_encrypted**(*esp_flash_t* *chip, uint32_t address, void *out_buffer, uint32_t length)

Read and decrypt data from the SPI flash chip using on-chip hardware flash encryption.

Parameters
- **chip** -- Pointer to identify flash chip. Must be NULL (the main flash chip). For other chips, encrypted read is not supported.
- **address** -- Address on flash to read from.
- **out_buffer** -- Pointer to a buffer for the data to read to.
- **length** -- Length (in bytes) of data to read.

Returns
- ESP_OK: on success
- ESP_ERR_NOT_SUPPORTED: encrypted read not supported for this chip.

static inline bool **esp_flash_is_quad_mode**(const *esp_flash_t* *chip)

Returns true if chip is configured for Quad I/O or Quad Fast Read.

Parameters **chip** -- Pointer to SPI flash chip to use. If NULL, esp_flash_default_chip is substituted.

Returns true if flash works in quad mode, otherwise false

Structures

struct **esp_flash_region_t**

Structure for describing a region of flash.

Public Members

uint32_t **offset**

Start address of this region.

uint32_t **size**

Size of the region.

struct **esp_flash_os_functions_t**

OS-level integration hooks for accessing flash chips inside a running OS.

It's in the public header because some instances should be allocated statically in the startup code. May be updated according to hardware version and new flash chip feature requirements, shouldn't be treated as public API.

For advanced developers, you may replace some of them with your implementations at your own risk.

Public Members

esp_err_t (***start**)(void *arg)

Called before commencing any flash operation. Does not need to be recursive (ie is called at most once for each call to 'end').

esp_err_t (***end**)(void *arg)

Called after completing any flash operation.

esp_err_t (***region_protected**)(void *arg, size_t start_addr, size_t size)

Called before any erase/write operations to check whether the region is limited by the OS

esp_err_t (***delay_us***)(void *arg, uint32_t us)

Delay for at least 'us' microseconds. Called in between 'start' and 'end'.

void *(***get_temp_buffer***)(void *arg, size_t reqest_size, size_t *out_size)

Called for get temp buffer when buffer from application cannot be directly read into/write from.

void (***release_temp_buffer***)(void *arg, void *temp_buf)

Called for release temp buffer.

esp_err_t (***check_yield***)(void *arg, uint32_t chip_status, uint32_t *out_request)

Yield to other tasks. Called during erase operations.

Return ESP_OK means yield needs to be called (got an event to handle), while ESP_ERR_TIMEOUT means skip yield.

esp_err_t (***yield***)(void *arg, uint32_t *out_status)

Yield to other tasks. Called during erase operations.

int64_t (***get_system_time***)(void *arg)

Called for get system time.

void (***set_flash_op_status***)(uint32_t op_status)

Call to set flash operation status

struct **esp_flash_t**

Structure to describe a SPI flash chip connected to the system.

Structure must be initialized before use (passed to esp_flash_init()). It's in the public header because some instances should be allocated statically in the startup code. May be updated according to hardware version and new flash chip feature requirements, shouldn't be treated as public API.

For advanced developers, you may replace some of them with your implementations at your own risk.

Public Members

spi_flash_host_inst_t ***host**

Pointer to hardware-specific "host_driver" structure. Must be initialized before used.

const *spi_flash_chip_t* ***chip_drv**

Pointer to chip-model-specific "adapter" structure. If NULL, will be detected during initialisation.

const *esp_flash_os_functions_t* ***os_func**

Pointer to os-specific hook structure. Call `esp_flash_init_os_functions()` to setup this field, after the host is properly initialized.

void ***os_func_data**

Pointer to argument for os-specific hooks. Left NULL and will be initialized with `os_func`.

esp_flash_io_mode_t **read_mode**

Configured SPI flash read mode. Set before `esp_flash_init` is called.

uint32_t **size**

Size of SPI flash in bytes. If 0, size will be detected during initialisation. Note: this stands for the size in the binary image header. If you want to get the flash physical size, please call `esp_flash_get_physical_size`.

uint32_t **chip_id**

Detected chip id.

uint32_t **busy**

This flag is used to verify chip's status.

uint32_t **hpm_dummy_ena**

This flag is used to verify whether flash works under HPM status.

uint32_t **reserved_flags**

reserved.

Macros

SPI_FLASH_YIELD_REQ_YIELD

SPI_FLASH_YIELD_REQ_SUSPEND

SPI_FLASH_YIELD_STA_RESUME

SPI_FLASH_OS_IS_ERASING_STATUS_FLAG

Type Definitions

typedef struct *spi_flash_chip_t* **spi_flash_chip_t**

Header File

- components/spi_flash/include/spi_flash_mmap.h
- This header file can be included with:

```
#include "spi_flash_mmap.h"
```

- This header file is a part of the API provided by the `spi_flash` component. To declare that your component depends on `spi_flash`, add the following to your CMakeLists.txt:

```
REQUIRES spi_flash
```

or

```
PRIV_REQUIRES spi_flash
```

Functions

esp_err_t **spi_flash_mmap**(size_t src_addr, size_t size, *spi_flash_mmap_memory_t* memory, const void **out_ptr, *spi_flash_mmap_handle_t* *out_handle)

Map region of flash memory into data or instruction address space.

This function allocates sufficient number of 64kB MMU pages and configures them to map the requested region of flash memory into the address space. It may reuse MMU pages which already provide the required mapping.

As with any allocator, if mmap/munmap are heavily used then the address space may become fragmented. To troubleshoot issues with page allocation, use spi_flash_mmap_dump() function.

Parameters

- **`src_addr`** -- Physical address in flash where requested region starts. This address *must* be aligned to 64kB boundary (SPI_FLASH_MMU_PAGE_SIZE)
- **`size`** -- Size of region to be mapped. This size will be rounded up to a 64kB boundary
- **`memory`** -- Address space where the region should be mapped (data or instruction)
- **`out_ptr`** -- **[out]** Output, pointer to the mapped memory region
- **`out_handle`** -- **[out]** Output, handle which should be used for spi_flash_munmap call

Returns ESP_OK on success, ESP_ERR_NO_MEM if pages can not be allocated

esp_err_t **`spi_flash_mmap_pages`**(const int *pages, size_t page_count, *spi_flash_mmap_memory_t* memory, const void **out_ptr, *spi_flash_mmap_handle_t* *out_handle)

Map sequences of pages of flash memory into data or instruction address space.

This function allocates sufficient number of 64kB MMU pages and configures them to map the indicated pages of flash memory contiguously into address space. In this respect, it works in a similar way as spi_flash_mmap() but it allows mapping a (maybe non-contiguous) set of pages into a contiguous region of memory.

Parameters

- **`pages`** -- An array of numbers indicating the 64kB pages in flash to be mapped contiguously into memory. These indicate the indexes of the 64kB pages, not the byte-size addresses as used in other functions. Array must be located in internal memory.
- **`page_count`** -- Number of entries in the pages array
- **`memory`** -- Address space where the region should be mapped (instruction or data)
- **`out_ptr`** -- **[out]** Output, pointer to the mapped memory region
- **`out_handle`** -- **[out]** Output, handle which should be used for spi_flash_munmap call

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if pages can not be allocated
- ESP_ERR_INVALID_ARG if pagecount is zero or pages array is not in internal memory

void **`spi_flash_munmap`**(*spi_flash_mmap_handle_t* handle)

Release region previously obtained using spi_flash_mmap.

Note: Calling this function will not necessarily unmap memory region. Region will only be unmapped when there are no other handles which reference this region. In case of partially overlapping regions it is possible that memory will be unmapped partially.

Parameters **`handle`** -- Handle obtained from spi_flash_mmap

void **`spi_flash_mmap_dump`**(void)

Display information about mapped regions.

This function lists handles obtained using spi_flash_mmap, along with range of pages allocated to each handle. It also lists all non-zero entries of MMU table and corresponding reference counts.

uint32_t **`spi_flash_mmap_get_free_pages`**(*spi_flash_mmap_memory_t* memory)

get free pages number which can be mmap

This function will return number of free pages available in mmu table. This could be useful before calling actual spi_flash_mmap (maps flash range to DCache or ICache memory) to check if there is sufficient space available for mapping.

Parameters **`memory`** -- memory type of MMU table free page
Returns number of free pages which can be mmaped

size_t **spi_flash_cache2phys**(const void *cached)

Given a memory address where flash is mapped, return the corresponding physical flash offset.

Cache address does not have have been assigned via spi_flash_mmap(), any address in memory mapped flash space can be looked up.

Parameters **cached** -- Pointer to flashed cached memory.

Returns

- SPI_FLASH_CACHE2PHYS_FAIL If cache address is outside flash cache region, or the address is not mapped.
- Otherwise, returns physical offset in flash

const void *__spi_flash_phys2cache__(size_t phys_offs, *spi_flash_mmap_memory_t* memory)

Given a physical offset in flash, return the address where it is mapped in the memory space.

Physical address does not have to have been assigned via spi_flash_mmap(), any address in flash can be looked up.

Note: Only the first matching cache address is returned. If MMU flash cache table is configured so multiple entries point to the same physical address, there may be more than one cache address corresponding to that physical address. It is also possible for a single physical address to be mapped to both the IROM and DROM regions.

Note: This function doesn't impose any alignment constraints, but if memory argument is SPI_FLASH_MMAP_INST and phys_offs is not 4-byte aligned, then reading from the returned pointer will result in a crash.

Parameters

- **phys_offs** -- Physical offset in flash memory to look up.
- **memory** -- Address space type to look up a flash cache address mapping for (instruction or data)

Returns

- NULL if the physical address is invalid or not mapped to flash cache of the specified memory type.
- Cached memory address (in IROM or DROM space) corresponding to phys_offs.

Macros

ESP_ERR_FLASH_OP_FAIL

This file contains `spi_flash_mmap_xx` APIs, mainly for doing memory mapping to an SPI0-connected external Flash, as well as some helper functions to convert between virtual and physical address

ESP_ERR_FLASH_OP_TIMEOUT

SPI_FLASH_SEC_SIZE

SPI Flash sector size

SPI_FLASH_MMU_PAGE_SIZE

Flash cache MMU mapping page size

SPI_FLASH_CACHE2PHYS_FAIL

Type Definitions

typedef uint32_t **spi_flash_mmap_handle_t**

Opaque handle for memory region obtained from spi_flash_mmap.

Enumerations

enum **spi_flash_mmap_memory_t**

Enumeration which specifies memory space requested in an mmap call.

Values:

enumerator **SPI_FLASH_MMAP_DATA**

map to data memory, allows byte-aligned access

enumerator **SPI_FLASH_MMAP_INST**

map to instruction memory, allows only 4-byte-aligned access

Header File

- components/hal/include/hal/spi_flash_types.h
- This header file can be included with:

```
#include "hal/spi_flash_types.h"
```

Structures

struct **spi_flash_trans_t**

Definition of a common transaction. Also holds the return value.

Public Members

uint8_t **reserved**

Reserved, must be 0.

uint8_t **mosi_len**

Output data length, in bytes.

uint8_t **miso_len**

Input data length, in bytes.

uint8_t **address_bitlen**

Length of address in bits, set to 0 if command does not need an address.

uint32_t **address**

Address to perform operation on.

const uint8_t ***mosi_data**

Output data to salve.

uint8_t *__miso_data__

[out] Input data from slave, little endian

uint32_t __flags__

Flags for this transaction. Set to 0 for now.

uint16_t __command__

Command to send.

uint8_t __dummy_bitlen__

Basic dummy bits to use.

uint32_t __io_mode__

Flash working mode when `SPI_FLASH_IGNORE_BASEIO` is specified.

struct __spi_flash_sus_cmd_conf__

Configuration structure for the flash chip suspend feature.

Public Members

uint32_t __sus_mask__

SUS/SUS1/SUS2 bit in flash register.

uint32_t __cmd_rdsr__

Read flash status register(2) command.

uint32_t __sus_cmd__

Flash suspend command.

uint32_t __res_cmd__

Flash resume command.

uint32_t __reserved__

Reserved, set to 0.

struct __spi_flash_encryption_t__

Structure for flash encryption operations.

Public Members

void (*__flash_encryption_enable__)(void)

Enable the flash encryption.

void (*__flash_encryption_disable__)(void)

Disable the flash encryption.

void (*`flash_encryption_data_prepare`)(uint32_t address, const uint32_t *buffer, uint32_t size)

Prepare flash encryption before operation.

Note: address and buffer must be 8-word aligned.

Param address The destination address in flash for the write operation.
Param buffer Data for programming
Param size Size to program.

void (*`flash_encryption_done`)(void)

flash data encryption operation is done.

void (*`flash_encryption_destroy`)(void)

Destroy encrypted result

bool (*`flash_encryption_check`)(uint32_t address, uint32_t length)

Check if is qualified to encrypt the buffer

Param address the address of written flash partition.
Param length Buffer size.

struct `spi_flash_host_inst_t`

SPI Flash Host driver instance

Public Members

const struct *spi_flash_host_driver_s* *`driver`

Pointer to the implementation function table.

struct `spi_flash_host_driver_s`

Host driver configuration and context structure.

Public Members

esp_err_t (*`dev_config`)(*spi_flash_host_inst_t* *host)

Configure the device-related register before transactions. This saves some time to re-configure those registers when we send continuously

esp_err_t (*`common_command`)(*spi_flash_host_inst_t* *host, *spi_flash_trans_t* *t)

Send an user-defined spi transaction to the device.

esp_err_t (*`read_id`)(*spi_flash_host_inst_t* *host, uint32_t *id)

Read flash ID.

void (*`erase_chip`)(*spi_flash_host_inst_t* *host)

Erase whole flash chip.

void (***erase_sector**)(*spi_flash_host_inst_t* *host, uint32_t start_address)

Erase a specific sector by its start address.

void (***erase_block**)(*spi_flash_host_inst_t* *host, uint32_t start_address)

Erase a specific block by its start address.

esp_err_t (***read_status**)(*spi_flash_host_inst_t* *host, uint8_t *out_sr)

Read the status of the flash chip.

esp_err_t (***set_write_protect**)(*spi_flash_host_inst_t* *host, bool wp)

Disable write protection.

void (***program_page**)(*spi_flash_host_inst_t* *host, const void *buffer, uint32_t address, uint32_t length)

Program a page of the flash. Check `max_write_bytes` for the maximum allowed writing length.

bool (***supports_direct_write**)(*spi_flash_host_inst_t* *host, const void *p)

Check whether the SPI host supports direct write.

When cache is disabled, SPI1 doesn't support directly write when buffer isn't internal.

int (***write_data_slicer**)(*spi_flash_host_inst_t* *host, uint32_t address, uint32_t len, uint32_t *align_addr, uint32_t page_size)

Slicer for write data. The `program_page` should be called iteratively with the return value of this function.

Param address Beginning flash address to write
Param len Length request to write
Param align_addr Output of the aligned address to write to
Param page_size Physical page size of the flash chip
Return Length that can be actually written in one `program_page` call

esp_err_t (***read**)(*spi_flash_host_inst_t* *host, void *buffer, uint32_t address, uint32_t read_len)

Read data from the flash. Check `max_read_bytes` for the maximum allowed reading length.

bool (***supports_direct_read**)(*spi_flash_host_inst_t* *host, const void *p)

Check whether the SPI host supports direct read.

When cache is disabled, SPI1 doesn't support directly read when the given buffer isn't internal.

int (***read_data_slicer**)(*spi_flash_host_inst_t* *host, uint32_t address, uint32_t len, uint32_t *align_addr, uint32_t page_size)

Slicer for read data. The `read` should be called iteratively with the return value of this function.

Param address Beginning flash address to read
Param len Length request to read
Param align_addr Output of the aligned address to read
Param page_size Physical page size of the flash chip
Return Length that can be actually read in one `read` call

uint32_t (***host_status**)(*spi_flash_host_inst_t* *host)

Check the host status, 0:busy, 1:idle, 2:suspended.

esp_err_t (***configure_host_io_mode**)(*spi_flash_host_inst_t* *host, uint32_t command, uint32_t addr_bitlen, int dummy_bitlen_base, *esp_flash_io_mode_t* io_mode)

Configure the host to work at different read mode. Responsible to compensate the timing and set IO mode.

void (*`poll_cmd_done`)(*spi_flash_host_inst_t* *host)

Internal use, poll the HW until the last operation is done.

esp_err_t (*`flush_cache`)(*spi_flash_host_inst_t* *host, uint32_t addr, uint32_t size)

For some host (SPI1), they are shared with a cache. When the data is modified, the cache needs to be flushed. Left NULL if not supported.

void (*`check_suspend`)(*spi_flash_host_inst_t* *host)

Suspend check erase/program operation, reserved for ESP32-C3 and ESP32-S3 spi flash ROM IMPL.

void (*`resume`)(*spi_flash_host_inst_t* *host)

Resume flash from suspend manually

void (*`suspend`)(*spi_flash_host_inst_t* *host)

Set flash in suspend status manually

esp_err_t (*`sus_setup`)(*spi_flash_host_inst_t* *host, const *spi_flash_sus_cmd_conf* *sus_conf)

Suspend feature setup for setting cmd and status register mask.

Macros

`SPI_FLASH_TRANS_FLAG_CMD16`

Send command of 16 bits.

`SPI_FLASH_TRANS_FLAG_IGNORE_BASEIO`

Not applying the basic io mode configuration for this transaction.

`SPI_FLASH_TRANS_FLAG_BYTE_SWAP`

Used for DTR mode, to swap the bytes of a pair of rising/falling edge.

`SPI_FLASH_TRANS_FLAG_PE_CMD`

Indicates that this transaction is to erase/program flash chip.

`SPI_FLASH_CONFIG_CONF_BITS`

OR the io_mode with this mask, to enable the dummy output feature or replace the first several dummy bits into address to meet the requirements of conf bits. (Used in DIO/QIO/OIO mode)

`SPI_FLASH_OPI_FLAG`

A flag for flash work in opi mode, the io mode below are opi, above are SPI/QSPI mode. DO NOT use this value in any API.

`SPI_FLASH_READ_MODE_MIN`

Slowest io mode supported by ESP32, currently SlowRd.

Type Definitions

typedef enum *esp_flash_speed_s* **esp_flash_speed_t**

SPI flash clock speed values, always refer to them by the enum rather than the actual value (more speed may be appended into the list).

A strategy to select the maximum allowed speed is to enumerate from the `ESP_FLSH_SPEED_MAX-1` or highest frequency supported by your flash, and decrease the speed until the probing success.

typedef struct *spi_flash_host_driver_s* **spi_flash_host_driver_t**

Enumerations

enum **esp_flash_speed_s**

SPI flash clock speed values, always refer to them by the enum rather than the actual value (more speed may be appended into the list).

A strategy to select the maximum allowed speed is to enumerate from the `ESP_FLSH_SPEED_MAX-1` or highest frequency supported by your flash, and decrease the speed until the probing success.

Values:

enumerator **ESP_FLASH_5MHZ**

The flash runs under 5MHz.

enumerator **ESP_FLASH_10MHZ**

The flash runs under 10MHz.

enumerator **ESP_FLASH_20MHZ**

The flash runs under 20MHz.

enumerator **ESP_FLASH_26MHZ**

The flash runs under 26MHz.

enumerator **ESP_FLASH_40MHZ**

The flash runs under 40MHz.

enumerator **ESP_FLASH_80MHZ**

The flash runs under 80MHz.

enumerator **ESP_FLASH_120MHZ**

The flash runs under 120MHz, 120MHZ can only be used by main flash after timing tuning in system. Do not use this directely in any API.

enumerator **ESP_FLASH_SPEED_MAX**

The maximum frequency supported by the host is `ESP_FLASH_SPEED_MAX-1`.

enum **esp_flash_io_mode_t**

Mode used for reading from SPI flash.

Values:

enumerator **SPI_FLASH_SLOWRD**

Data read using single I/O, some limits on speed.

enumerator **SPI_FLASH_FASTRD**

Data read using single I/O, no limit on speed.

enumerator **SPI_FLASH_DOUT**

Data read using dual I/O.

enumerator **SPI_FLASH_DIO**

Both address & data transferred using dual I/O.

enumerator **SPI_FLASH_QOUT**

Data read using quad I/O.

enumerator **SPI_FLASH_QIO**

Both address & data transferred using quad I/O.

enumerator **SPI_FLASH_OPI_STR**

Only support on OPI flash, flash read and write under STR mode.

enumerator **SPI_FLASH_OPI_DTR**

Only support on OPI flash, flash read and write under DTR mode.

enumerator **SPI_FLASH_READ_MODE_MAX**

The fastest io mode supported by the host is `ESP_FLASH_READ_MODE_MAX-1`.

Header File

- components/hal/include/hal/esp_flash_err.h
- This header file can be included with:

```
#include "hal/esp_flash_err.h"
```

Macros

ESP_ERR_FLASH_NOT_INITIALISED

esp_flash_chip_t structure not correctly initialised by esp_flash_init().

ESP_ERR_FLASH_UNSUPPORTED_HOST

Requested operation isn't supported via this host SPI bus (chip->spi field).

ESP_ERR_FLASH_UNSUPPORTED_CHIP

Requested operation isn't supported by this model of SPI flash chip.

ESP_ERR_FLASH_PROTECTED

Write operation failed due to chip's write protection being enabled.

Enumerations

enum **[anonymous]**

Values:

enumerator **ESP_ERR_FLASH_SIZE_NOT_MATCH**

The chip doesn't have enough space for the current partition table.

enumerator **ESP_ERR_FLASH_NO_RESPONSE**

Chip did not respond to the command, or timed out.

Header File

- components/spi_flash/include/esp_spi_flash_counters.h
- This header file can be included with:

```
#include "esp_spi_flash_counters.h"
```

- This header file is a part of the API provided by the `spi_flash` component. To declare that your component depends on `spi_flash`, add the following to your CMakeLists.txt:

```
REQUIRES spi_flash
```

or

```
PRIV_REQUIRES spi_flash
```

Functions

void **esp_flash_reset_counters**(void)

Reset SPI flash operation counters.

void **spi_flash_reset_counters**(void)

void **esp_flash_dump_counters**(FILE *stream)

Print SPI flash operation counters.

void **spi_flash_dump_counters**(void)

const *esp_flash_counters_t* ***esp_flash_get_counters**(void)

Return current SPI flash operation counters.

Returns pointer to the *esp_flash_counters_t* structure holding values of the operation counters

const *spi_flash_counters_t* ***spi_flash_get_counters**(void)

Structures

struct **esp_flash_counter_t**

Structure holding statistics for one type of operation

Public Members

uint32_t **count**

number of times operation was executed

uint32_t **time**

total time taken, in microseconds

uint32_t **bytes**

total number of bytes

struct **esp_flash_counters_t**

Structure for counters of flash actions

Public Members

esp_flash_counter_t **read**

counters for read action, like `esp_flash_read`

esp_flash_counter_t **write**

counters for write action, like `esp_flash_write`

esp_flash_counter_t **erase**

counters for erase action, like `esp_flash_erase`

Type Definitions

typedef *esp_flash_counter_t* **spi_flash_counter_t**

typedef *esp_flash_counters_t* **spi_flash_counters_t**

API Reference - Flash Encrypt

Header File

- components/bootloader_support/include/esp_flash_encrypt.h
- This header file can be included with:

```
#include "esp_flash_encrypt.h"
```

- This header file is a part of the API provided by the `bootloader_support` component. To declare that your component depends on `bootloader_support`, add the following to your CMakeLists.txt:

```
REQUIRES bootloader_support
```

or

```
PRIV_REQUIRES bootloader_support
```

Functions

bool **esp_flash_encryption_enabled**(void)

Is flash encryption currently enabled in hardware?

Flash encryption is enabled if the FLASH_CRYPT_CNT efuse has an odd number of bits set.

Returns true if flash encryption is enabled.

esp_err_t **esp_flash_encrypt_check_and_update**(void)

bool **esp_flash_encrypt_state**(void)

Returns the Flash Encryption state and prints it.

Returns True - Flash Encryption is enabled False - Flash Encryption is not enabled

bool **esp_flash_encrypt_initialized_once**(void)

Checks if the first initialization was done.

If the first initialization was done then FLASH_CRYPT_CNT != 0

Returns true - the first initialization was done false - the first initialization was NOT done

esp_err_t **esp_flash_encrypt_init**(void)

The first initialization of Flash Encryption key and related eFuses.

Returns ESP_OK if all operations succeeded

esp_err_t **esp_flash_encrypt_contents**(void)

Encrypts flash content.

Returns ESP_OK if all operations succeeded

esp_err_t **esp_flash_encrypt_enable**(void)

Activates Flash encryption on the chip.

It burns FLASH_CRYPT_CNT eFuse based on the CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE option.

Returns ESP_OK if all operations succeeded

bool **esp_flash_encrypt_is_write_protected**(bool print_error)

Returns True if the write protection of FLASH_CRYPT_CNT is set.

Parameters **print_error** -- Print error if it is write protected
Returns true - if FLASH_CRYPT_CNT is write protected

esp_err_t **esp_flash_encrypt_region**(uint32_t src_addr, size_t data_length)

Encrypt-in-place a block of flash sectors.

Note: This function resets RTC_WDT between operations with sectors.

Parameters

- **src_addr** -- Source offset in flash. Should be multiple of 4096 bytes.
- **data_length** -- Length of data to encrypt in bytes. Will be rounded up to next multiple of 4096 bytes.

Returns ESP_OK if all operations succeeded, ESP_ERR_FLASH_OP_FAIL if SPI flash fails, ESP_ERR_FLASH_OP_TIMEOUT if flash times out.

void **esp_flash_write_protect_crypt_cnt**(void)

Write protect FLASH_CRYPT_CNT.

Intended to be called as a part of boot process if flash encryption is enabled but secure boot is not used. This should protect against serial re-flashing of an unauthorised code in absence of secure boot.

Note: On ESP32 V3 only, write protecting FLASH_CRYPT_CNT will also prevent disabling UART Download Mode. If both are wanted, call esp_efuse_disable_rom_download_mode() before calling this function.

esp_flash_enc_mode_t **esp_get_flash_encryption_mode**(void)

Return the flash encryption mode.

The API is called during boot process but can also be called by application to check the current flash encryption mode of ESP32

Returns

void **esp_flash_encryption_init_checks**(void)

Check the flash encryption mode during startup.

Verifies the flash encryption config during startup:

- Correct any insecure flash encryption settings if hardware Secure Boot is enabled.
- Log warnings if the efuse config doesn't match the project config in any way

Note: This function is called automatically during app startup, it doesn't need to be called from the app.

bool **esp_flash_encryption_cfg_verify_release_mode**(void)

Returns the verification status for all physical security features of flash encryption in release mode.

If the device has flash encryption feature configured in the release mode, then it is highly recommended to call this API in the application startup code. This API verifies the sanity of the eFuse configuration against the release (production) mode of the flash encryption feature.

Returns

- True - all eFuses are configured correctly
- False - not all eFuses are configured correctly.

void **esp_flash_encryption_set_release_mode**(void)

Switches Flash Encryption from "Development" to "Release".

If already in "Release" mode, the function will do nothing. If flash encryption efuse is not enabled yet then abort. It burns:

- "disable encrypt in dl mode"
- set FLASH_CRYPT_CNT efuse to max

Enumerations

enum **esp_flash_enc_mode_t**

Values:

enumerator **ESP_FLASH_ENC_MODE_DISABLED**

enumerator **ESP_FLASH_ENC_MODE_DEVELOPMENT**

enumerator **ESP_FLASH_ENC_MODE_RELEASE**

2.6.22 SPI Master Driver

SPI Master driver is a program that controls ESP32-S3's General Purpose SPI (GP-SPI) peripheral(s) when it functions as a master.

For more hardware information about the GP-SPI peripheral(s), see **ESP32-S3 Technical Reference Manual > SPI Controller** [PDF].

Terminology

The terms used in relation to the SPI Master driver are given in the table below.

Term	Definition
Host	The SPI controller peripheral inside ESP32-S3 initiates SPI transmissions over the bus and acts as an SPI Master.
Device	SPI slave Device. An SPI bus may be connected to one or more Devices. Each Device shares the MOSI, MISO, and SCLK signals but is only active on the bus when the Host asserts the Device's individual CS line.
Bus	A signal bus, common to all Devices connected to one Host. In general, a bus includes the following lines: MISO, MOSI, SCLK, one or more CS lines, and, optionally, QUADWP and QUADHD. So Devices are connected to the same lines, with the exception that each Device has its own CS line. Several Devices can also share one CS line if connected in a daisy-chain manner.
MOSI	Master Out, Slave In, a.k.a. D. Data transmission from a Host to Device. Also data0 signal in Octal/OPI mode.
MISO	Master In, Slave Out, a.k.a. Q. Data transmission from a Device to Host. Also data1 signal in Octal/OPI mode.
SCLK	Serial Clock. The oscillating signal generated by a Host keeps the transmission of data bits in sync.
CS	Chip Select. Allows a Host to select individual Device(s) connected to the bus in order to send or receive data.
QUADWP	Write Protect signal. Used for 4-bit (qio/qout) transactions. Also for the data2 signal in Octal/OPI mode.
QUADHD	Hold signal. Used for 4-bit (qio/qout) transactions. Also for the data3 signal in Octal/OPI mode.
DATA4	Data4 signal in Octal/OPI mode.
DATA5	Data5 signal in Octal/OPI mode.
DATA6	Data6 signal in Octal/OPI mode.
DATA7	Data7 signal in Octal/OPI mode.
Assertion	The action of activating a line.
De-assertion	The action of returning the line back to inactive (back to idle) status.
Transaction	One instance of a Host asserting a CS line, transferring data to and from a Device, and de-asserting the CS line. Transactions are atomic, which means they can never be interrupted by another transaction.
Launch Edge	Edge of the clock at which the source register **launches** the signal onto the line.
Latch Edge	Edge of the clock at which the destination register **latches in** the signal.

Driver Features

The SPI Master driver governs the communications between Hosts and Devices. The driver supports the following features:

- Multi-threaded environments
- Transparent handling of DMA transfers while reading and writing data
- Automatic time-division multiplexing of data coming from different Devices on the same signal bus, see *SPI Bus Lock*.

Warning: The SPI Master driver allows multiple Devices to be connected on a same SPI bus (sharing a single ESP32-S3 SPI peripheral). As long as each Device is accessed by only one task, the driver is thread-safe. However, if multiple tasks try to access the same SPI Device, the driver is **not thread-safe**. In this case, it is recommended to either:

- Refactor your application so that each SPI peripheral is only accessed by a single task at a time. You can

use *spi_bus_config_t::isr_cpu_id* to register the SPI ISR to the same core as SPI peripheral-related tasks to ensure thread safety.
- Add a mutex lock around the shared Device using *xSemaphoreCreateMutex*.

SPI Transactions

An SPI bus transaction consists of five phases which can be found in the table below. Any of these phases can be skipped.

Phase	Description
Command	In this phase, a command (0-16 bit) is written to the bus by the Host.
Address	In this phase, an address (0-32 bit) is transmitted over the bus by the Host.
Dummy	This phase is configurable and is used to meet the timing requirements.
Write	Host sends data to a Device. This data follows the optional command and address phases and is indistinguishable from them at the electrical level.
Read	Device sends data to its Host.

The attributes of a transaction are determined by the bus configuration structure *spi_bus_config_t*, Device configuration structure *spi_device_interface_config_t*, and transaction configuration structure *spi_transaction_t*.

An SPI Host can send full-duplex transactions, during which the Read and Write phases occur simultaneously. The total transaction length is determined by the sum of the following members:

- *spi_device_interface_config_t::command_bits*
- *spi_device_interface_config_t::address_bits*
- *spi_transaction_t::length*

While the member *spi_transaction_t::rxlength* only determines the length of data received into the buffer.

In half-duplex transactions, the Read and Write phases are not simultaneous (one direction at a time). The lengths of the Write and Read phases are determined by *spi_transaction_t::length* and *spi_transaction_t::rxlength* respectively.

The Command and Address phases are optional, as not every SPI Device requires a command and/or address. This is reflected in the Device's configuration: if *spi_device_interface_config_t::command_bits* and/or *spi_device_interface_config_t::address_bits* are set to zero, no Command or Address phase will occur.

The Read and Write phases can also be optional, as not every transaction requires both writing and reading data. If *spi_transaction_t::rx_buffer* is `NULL` and *SPI_TRANS_USE_RXDATA* is not set, the Read phase is skipped. If *spi_transaction_t::tx_buffer* is `NULL` and *SPI_TRANS_USE_TXDATA* is not set, the Write phase is skipped.

The driver supports two types of transactions: interrupt transactions and polling transactions. The programmer can choose to use a different transaction type per Device. If your Device requires both transaction types, see *Notes on Sending Mixed Transactions to the Same Device*.

Interrupt Transactions Interrupt transactions blocks the transaction routine until the transaction completes, thus allowing the CPU to run other tasks.

An application task can queue multiple transactions, and the driver automatically handles them one by one in the interrupt service routine (ISR). It allows the task to switch to other procedures until all the transactions are complete.

Polling Transactions Polling transactions do not use interrupts. The routine keeps polling the SPI Host's status bit until the transaction is finished.

All the tasks that use interrupt transactions can be blocked by the queue. At this point, they need to wait for the ISR to run twice before the transaction is finished. Polling transactions save time otherwise spent on queue handling and context switching, which results in smaller transaction duration. The disadvantage is that the CPU is busy while these transactions are in progress.

The *spi_device_polling_end()* routine needs an overhead of at least 1 µs to unblock other tasks when the transaction is finished. It is strongly recommended to wrap a series of polling transactions using the functions *spi_device_acquire_bus()* and *spi_device_release_bus()* to avoid the overhead. For more information, see *Bus Acquiring*.

Transaction Line Mode Supported line modes for ESP32-S3 are listed as follows, to make use of these modes, set the member `flags` in the struct *spi_transaction_t* as shown in the `Transaction Flag` column. If you want to check if corresponding IO pins are set or not, set the member `flags` in the *spi_bus_config_t* as shown in the `Bus IO setting Flag` column.

Mode name	Command Line Width	Address Line Width	Data Line Width	Transaction Flag	Bus IO Setting Flag
Normal SPI	1	1	1	0	0
Dual Output	1	1	2	SPI_TRANS_MODE_DIO	SPICOMMON_BUSFLAG_DUAL
Dual I/O	1	2	2	SPI_TRANS_MODE_DIO SPI_TRANS_MULTILINE_ADDR	SPICOMMON_BUSFLAG_DUAL
Quad Output	1	1	4	SPI_TRANS_MODE_QIO	SPICOMMON_BUSFLAG_QUAD
Quad I/O	1	4	4	SPI_TRANS_MODE_QIO SPI_TRANS_MULTILINE_ADDR	SPICOMMON_BUSFLAG_QUAD
Octal Output	1	1	8	SPI_TRANS_MODE_OCT	SPICOMMON_BUSFLAG_OCTAL
OPI	8	8	8	SPI_TRANS_MODE_OCT SPI_TRANS_MULTILINE_ADDR SPI_TRANS_MULTILINE_CMD	SPICOMMON_BUSFLAG_OCTAL

Command and Address Phases During the Command and Address phases, the members *spi_transaction_t::cmd* and *spi_transaction_t::addr* are sent to the bus, nothing is read at this time. The default lengths of the Command and Address phases are set in *spi_device_interface_config_t* by calling *spi_bus_add_device()*. If the flags *SPI_TRANS_VARIABLE_CMD* and *SPI_TRANS_VARIABLE_ADDR* in the member *spi_transaction_t::flags* are not set, the driver automatically sets the length of these phases to default values during Device initialization.

If the lengths of the Command and Address phases need to be variable, declare the struct *spi_transaction_ext_t*, set the flags *SPI_TRANS_VARIABLE_CMD* and/or *SPI_TRANS_VARIABLE_ADDR* in the member *spi_transaction_ext_t::base* and configure the rest of base as usual. Then the length of each phase will be equal to *spi_transaction_ext_t::command_bits* and *spi_transaction_ext_t::address_bits* set in the struct *spi_transaction_ext_t*.

If the Command and Address phase need to have the same number of lines as the data phase, you need to set `SPI_TRANS_MULTILINE_CMD` and/or `SPI_TRANS_MULTILINE_ADDR` to the `flags` member in the struct *spi_transaction_t*. Also see *Transaction Line Mode*.

Write and Read Phases Normally, the data that needs to be transferred to or from a Device is read from or written to a chunk of memory indicated by the members *spi_transaction_t::rx_buffer* and *spi_transaction_t::tx_buffer*. If DMA is enabled for transfers, the buffers are required to be:

1. Allocated in DMA-capable internal memory (MALLOC_CAP_DMA), see *DMA-Capable Memory*.
2. 32-bit aligned (starting from a 32-bit boundary and having a length of multiples of 4 bytes).

If these requirements are not satisfied, the transaction efficiency will be affected due to the allocation and copying of temporary buffers.

If using more than one data line to transmit, please set `SPI_DEVICE_HALFDUPLEX` flag for the member `flags` in the struct *`spi_device_interface_config_t`*. And the member `flags` in the struct *`spi_transaction_t`* should be set as described in *Transaction Line Mode*.

Note: Half-duplex transactions with both Read and Write phases are not supported. Please use full duplex mode.

Bus Acquiring Sometimes you might want to send SPI transactions exclusively and continuously so that it takes as little time as possible. For this, you can use bus acquiring, which helps to suspend transactions (both polling or interrupt) to other Devices until the bus is released. To acquire and release a bus, use the functions *`spi_device_acquire_bus()`* and *`spi_device_release_bus()`*.

Driver Usage

- Initialize an SPI bus by calling the function *`spi_bus_initialize()`*. Make sure to set the correct I/O pins in the struct *`spi_bus_config_t`*. Set the signals that are not needed to `-1`.
- Register a Device connected to the bus with the driver by calling the function *`spi_bus_add_device()`*. Make sure to configure any timing requirements the Device might need with the parameter `dev_config`. You should now have obtained the Device's handle which will be used when sending a transaction to it.
- To interact with the Device, fill one or more *`spi_transaction_t`* structs with any transaction parameters required. Then send the structs either using a polling transaction or an interrupt transaction:
  - ***Interrupt*** Either queue all transactions by calling the function *`spi_device_queue_trans()`* and, at a later time, query the result using the function *`spi_device_get_trans_result()`*, or handle all requests synchronously by feeding them into *`spi_device_transmit()`*.
  - ***Polling*** Call the function *`spi_device_polling_transmit()`* to send polling transactions. Alternatively, if you want to insert something in between, send the transactions by using *`spi_device_polling_start()`* and *`spi_device_polling_end()`*.
- (Optional) To perform back-to-back transactions with a Device, call the function *`spi_device_acquire_bus()`* before sending transactions and *`spi_device_release_bus()`* after the transactions have been sent.
- (Optional) To remove a certain Device from the bus, call *`spi_bus_remove_device()`* with the Device handle as an argument.
- (Optional) To remove the driver from the bus, make sure no more devices are attached and call *`spi_bus_free()`*.

The example code for the SPI Master driver can be found in the peripherals/spi_master directory of ESP-IDF examples.

Transactions with Data Not Exceeding 32 Bits When the transaction data size is equal to or less than 32 bits, it will be sub-optimal to allocate a buffer for the data. The data can be directly stored in the transaction struct instead. For transmitted data, it can be achieved by using the *`spi_transaction_t::tx_data`* member and setting the *`SPI_TRANS_USE_TXDATA`* flag on the transmission. For received data, use *`spi_transaction_t::rx_data`* and set *`SPI_TRANS_USE_RXDATA`*. In both cases, do not touch the *`spi_transaction_t::tx_buffer`* or *`spi_transaction_t::rx_buffer`* members, because they use the same memory locations as *`spi_transaction_t::tx_data`* and *`spi_transaction_t::rx_data`*.

Transactions with Integers Other than `uint8_t` An SPI Host reads and writes data into memory byte by byte. By default, data is sent with the most significant bit (MSB) first, as LSB is first used in rare cases. If a value of fewer than 8 bits needs to be sent, the bits should be written into memory in the MSB first manner.

For example, if `0b00010` needs to be sent, it should be written into a `uint8_t` variable, and the length for reading should be set to 5 bits. The Device will still receive 8 bits with 3 additional "random" bits, so the reading must be performed correctly.

On top of that, ESP32-S3 is a little-endian chip, which means that the least significant byte of `uint16_t` and `uint32_t` variables is stored at the smallest address. Hence, if `uint16_t` is stored in memory, bits [7:0] are sent first, followed by bits [15:8].

For cases when the data to be transmitted has a size differing from `uint8_t` arrays, the following macros can be used to transform data to the format that can be sent by the SPI driver directly:

- *SPI_SWAP_DATA_TX* for data to be transmitted
- *SPI_SWAP_DATA_RX* for data received

Notes on Sending Mixed Transactions to the Same Device To reduce coding complexity, send only one type of transaction (interrupt or polling) to one Device. However, you still can send both interrupt and polling transactions alternately. The notes below explain how to do this.

The polling transactions should be initiated only after all the polling and interrupt transactions are finished.

Since an unfinished polling transaction blocks other transactions, please do not forget to call the function *spi_device_polling_end()* after *spi_device_polling_start()* to allow other transactions or to allow other Devices to use the bus. Remember that if there is no need to switch to other tasks during your polling transaction, you can initiate a transaction with *spi_device_polling_transmit()* so that it will be ended automatically.

In-flight polling transactions are disturbed by the ISR operation to accommodate interrupt transactions. Always make sure that all the interrupt transactions sent to the ISR are finished before you call *spi_device_polling_start()*. To do that, you can keep calling *spi_device_get_trans_result()* until all the transactions are returned.

To have better control of the calling sequence of functions, send mixed transactions to the same Device only within a single task.

GPIO Matrix and IO_MUX Most of the chip's peripheral signals have a direct connection to their dedicated IO_MUX pins. However, the signals can also be routed to any other available pins using the less direct GPIO matrix. If at least one signal is routed through the GPIO matrix, then all signals will be routed through it.

When an SPI Host is set to 80 MHz or lower frequencies, routing SPI pins via the GPIO matrix will behave the same compared to routing them via IOMUX.

The IO_MUX pins for SPI buses are given below.

Pin Name	GPIO Number (SPI2)
CS0[1]	10
SCLK	12
MISO	13
MOSI	11
QUADWP	14
QUADHD	9

Transfer Speed Considerations

There are three factors limiting the transfer speed:

- Transaction interval
- SPI clock frequency
- Cache miss of SPI functions, including callbacks

[1] Only the first Device attached to the bus can use the CS0 pin.

The main parameter that determines the transfer speed for large transactions is clock frequency. For multiple small transactions, the transfer speed is mostly determined by the length of transaction intervals.

Transaction Duration Transaction duration includes setting up SPI peripheral registers, copying data to FIFOs or setting up DMA links, and the time for SPI transactions.

Interrupt transactions allow appending extra overhead to accommodate the cost of FreeRTOS queues and the time needed for switching between tasks and the ISR.

For **interrupt transactions**, the CPU can switch to other tasks when a transaction is in progress. This saves CPU time but increases the transaction duration. See *Interrupt Transactions*. For **polling transactions**, it does not block the task but allows to do polling when the transaction is in progress. For more information, see *Polling Transactions*.

If DMA is enabled, setting up the linked list requires about 2 µs per transaction. When a master is transferring data, it automatically reads the data from the linked list. If DMA is not enabled, the CPU has to write and read each byte from the FIFO by itself. Usually, this is faster than 2 µs, but the transaction length is limited to 64 bytes for both write and read.

The typical transaction duration for one byte of data is given below.

- Interrupt Transaction via DMA: 26 µs.
- Interrupt Transaction via CPU: 24 µs.
- Polling Transaction via DMA: 11 µs.
- Polling Transaction via CPU: 9 µs.

Note that these data are tested with *CONFIG_SPI_MASTER_ISR_IN_IRAM* enabled. SPI transaction related code are placed in the internal memory. If this option is turned off (for example, for internal memory optimization), the transaction duration may be affected.

SPI Clock Frequency The clock source of the GPSPI peripherals can be selected by setting `spi_device_handle_t::cfg::clock_source`. You can refer to *`spi_clock_source_t`* to know the supported clock sources.

By default driver sets `spi_device_handle_t::cfg::clock_source` to `SPI_CLK_SRC_DEFAULT`. This usually stands for the highest frequency among GPSPI clock sources. Its value is different among chips.

The actual clock frequency of a Device may not be exactly equal to the number you set, it is re-calculated by the driver to the nearest hardware-compatible number, and not larger than the clock frequency of the clock source. You can call *`spi_device_get_actual_freq()`* to know the actual frequency computed by the driver.

The theoretical maximum transfer speed of the Write or Read phase can be calculated according to the table below:

Line Width of Write/Read phase	Speed (Bps)
1-Line	*SPI Frequency / 8*
2-Line	*SPI Frequency / 4*
4-Line	*SPI Frequency / 2*
8-Line	*SPI Frequency*

The transfer speed calculation of other phases (Command, Address, Dummy) is similar.

Cache Missing The default config puts only the ISR into the IRAM. Other SPI-related functions, including the driver itself and the callback, might suffer from cache misses and need to wait until the code is read from flash. Select *CONFIG_SPI_MASTER_IN_IRAM* to put the whole SPI driver into IRAM and put the entire callback(s) and its callee functions into IRAM to prevent cache missing.

Note: SPI driver implementation is based on FreeRTOS APIs, to use *CONFIG_SPI_MASTER_IN_IRAM*, you should not enable *CONFIG_FREERTOS_PLACE_FUNCTIONS_INTO_FLASH*.

For an interrupt transaction, the overall cost is **20+8n/Fspi[MHz]** [µs] for n bytes transferred in one transaction. Hence, the transferring speed is: **n/(20+8n/Fspi)**. An example of transferring speed at 8 MHz clock speed is given in the following table.

Frequency (MHz)	Transaction Interval (µs)	Transaction Length (bytes)	Total Time (µs)	Total Speed (KBps)
8	25	1	26	38.5
8	25	8	33	242.4
8	25	16	41	490.2
8	25	64	89	719.1
8	25	128	153	836.6

When a transaction length is short, the cost of the transaction interval is high. If possible, try to squash several short transactions into one transaction to achieve a higher transfer speed.

Please note that the ISR is disabled during flash operation by default. To keep sending transactions during flash operations, enable *CONFIG_SPI_MASTER_ISR_IN_IRAM* and set *ESP_INTR_FLAG_IRAM* in the member *spi_bus_config_t::intr_flags*. In this case, all the transactions queued before starting flash operations are handled by the ISR in parallel. Also note that the callback of each Device and their `callee` functions should be in IRAM, or your callback will crash due to cache missing. For more details, see *IRAM-Safe Interrupt Handlers*.

Application Example

The code example for using the SPI master half duplex mode to read/write an AT93C46D EEPROM (8-bit mode) can be found in the peripherals/spi_master/hd_eeprom directory of ESP-IDF examples.

The code example for using the SPI master full duplex mode to drive a SPI_LCD (e.g. ST7789V or ILI9341) can be found in the peripherals/spi_master/lcd directory of ESP-IDF examples.

API Reference - SPI Common

Header File

- components/hal/include/hal/spi_types.h
- This header file can be included with:

```
#include "hal/spi_types.h"
```

Structures

struct **spi_line_mode_t**

Line mode of SPI transaction phases: CMD, ADDR, DOUT/DIN.

Public Members

uint8_t **cmd_lines**

The line width of command phase, e.g. 2-line-cmd-phase.

uint8_t **addr_lines**

The line width of address phase, e.g. 1-line-addr-phase.

uint8_t **data_lines**

The line width of data phase, e.g. 4-line-data-phase.

Type Definitions

typedef *soc_periph_spi_clk_src_t* **spi_clock_source_t**

Type of SPI clock source.

Enumerations

enum **spi_host_device_t**

Enum with the three SPI peripherals that are software-accessible in it.

Values:

enumerator **SPI1_HOST**

SPI1.

enumerator **SPI2_HOST**

SPI2.

enumerator **SPI3_HOST**

SPI3.

enumerator **SPI_HOST_MAX**

invalid host value

enum **spi_event_t**

SPI Events.

Values:

enumerator **SPI_EV_BUF_TX**

The buffer has sent data to master.

enumerator **SPI_EV_BUF_RX**

The buffer has received data from master.

enumerator **SPI_EV_SEND_DMA_READY**

Slave has loaded its TX data buffer to the hardware (DMA).

enumerator **SPI_EV_SEND**

Master has received certain number of the data, the number is determined by Master.

enumerator **SPI_EV_RECV_DMA_READY**

Slave has loaded its RX data buffer to the hardware (DMA).

enumerator **SPI_EV_RECV**

Slave has received certain number of data from master, the number is determined by Master.

enumerator **SPI_EV_CMD9**

Received CMD9 from master.

enumerator **SPI_EV_CMDA**

Received CMDA from master.

enumerator **SPI_EV_TRANS**

A transaction has done.

enum **spi_command_t**

SPI command.

Values:

enumerator **SPI_CMD_HD_WRBUF**

enumerator **SPI_CMD_HD_RDBUF**

enumerator **SPI_CMD_HD_WRDMA**

enumerator **SPI_CMD_HD_RDDMA**

enumerator **SPI_CMD_HD_SEG_END**

enumerator **SPI_CMD_HD_EN_QPI**

enumerator **SPI_CMD_HD_WR_END**

enumerator **SPI_CMD_HD_INT0**

enumerator **SPI_CMD_HD_INT1**

enumerator **SPI_CMD_HD_INT2**

Header File

- components/esp_driver_spi/include/driver/spi_common.h
- This header file can be included with:

```
#include "driver/spi_common.h"
```

- This header file is a part of the API provided by the `esp_driver_spi` component. To declare that your component depends on `esp_driver_spi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_spi
```

or

```
PRIV_REQUIRES esp_driver_spi
```

Functions

esp_err_t **spi_bus_initialize**(*spi_host_device_t* host_id, const *spi_bus_config_t* *bus_config, *spi_dma_chan_t* dma_chan)

Initialize a SPI bus.

Warning: SPI0/1 is not supported

Warning: If a DMA channel is selected, any transmit and receive buffer used should be allocated in DMA-capable memory.

Warning: The ISR of SPI is always executed on the core which calls this function. Never starve the ISR on this core or the SPI transactions will not be handled.

Parameters

- **host_id** -- SPI peripheral that controls this bus
- **bus_config** -- Pointer to a *spi_bus_config_t* struct specifying how the host should be initialized
- **dma_chan** -- - Selecting a DMA channel for an SPI bus allows transactions on the bus with size only limited by the amount of internal memory.
  - Selecting SPI_DMA_DISABLED limits the size of transactions.
  - Set to SPI_DMA_DISABLED if only the SPI flash uses this bus.
  - Set to SPI_DMA_CH_AUTO to let the driver to allocate the DMA channel.

Returns

- ESP_ERR_INVALID_ARG if configuration is invalid
- ESP_ERR_INVALID_STATE if host already is in use
- ESP_ERR_NOT_FOUND if there is no available DMA channel
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

esp_err_t **spi_bus_free**(*spi_host_device_t* host_id)

Free a SPI bus.

Warning: In order for this to succeed, all devices have to be removed first.

Parameters **host_id** -- SPI peripheral to free

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_INVALID_STATE if bus hasn't been initialized before, or not all devices on the bus are freed
- ESP_OK on success

Structures

struct **spi_bus_config_t**

This is a configuration structure for a SPI bus.

You can use this structure to specify the GPIO pins of the bus. Normally, the driver will use the GPIO matrix to route the signals. An exception is made when all signals either can be routed through the IO_MUX or are -1. In that case, the IO_MUX is used. On ESP32, using GPIO matrix will bring about 25ns of input delay, which may cause incorrect read for >40MHz speeds.

Note: Be advised that the slave driver does not use the quadwp/quadhd lines and fields in *spi_bus_config_t* referring to these lines will be ignored and can thus safely be left uninitialized.

Public Members

int **mosi_io_num**

GPIO pin for Master Out Slave In (=spi_d) signal, or -1 if not used.

int **data0_io_num**

GPIO pin for spi data0 signal in quad/octal mode, or -1 if not used.

int **miso_io_num**

GPIO pin for Master In Slave Out (=spi_q) signal, or -1 if not used.

int **data1_io_num**

GPIO pin for spi data1 signal in quad/octal mode, or -1 if not used.

int **sclk_io_num**

GPIO pin for SPI Clock signal, or -1 if not used.

int **quadwp_io_num**

GPIO pin for WP (Write Protect) signal, or -1 if not used.

int **data2_io_num**

GPIO pin for spi data2 signal in quad/octal mode, or -1 if not used.

int **quadhd_io_num**

GPIO pin for HD (Hold) signal, or -1 if not used.

int **data3_io_num**

GPIO pin for spi data3 signal in quad/octal mode, or -1 if not used.

int **data4_io_num**

GPIO pin for spi data4 signal in octal mode, or -1 if not used.

int **data5_io_num**

GPIO pin for spi data5 signal in octal mode, or -1 if not used.

int **data6_io_num**

GPIO pin for spi data6 signal in octal mode, or -1 if not used.

int **data7_io_num**

GPIO pin for spi data7 signal in octal mode, or -1 if not used.

int **max_transfer_sz**

Maximum transfer size, in bytes. Defaults to 4092 if 0 when DMA enabled, or to `SOC_SPI_MAXIMUM_BUFFER_SIZE` if DMA is disabled.

uint32_t **flags**

Abilities of bus to be checked by the driver. Or-ed value of `SPICOMMON_BUSFLAG_*` flags.

esp_intr_cpu_affinity_t **isr_cpu_id**

Select cpu core to register SPI ISR.

int **intr_flags**

Interrupt flag for the bus to set the priority, and IRAM attribute, see `esp_intr_alloc.h`. Note that the EDGE, INTRDISABLED attribute are ignored by the driver. Note that if ESP_INTR_FLAG_IRAM is set, ALL the callbacks of the driver, and their callee functions, should be put in the IRAM.

Macros

SPI_MAX_DMA_LEN

SPI_SWAP_DATA_TX(DATA, LEN)

Transform unsigned integer of length <= 32 bits to the format which can be sent by the SPI driver directly.

E.g. to send 9 bits of data, you can:

```
uint16_t data = SPI_SWAP_DATA_TX(0x145, 9);
```

Then points tx_buffer to `&data`.

Parameters

- **DATA** -- Data to be sent, can be uint8_t, uint16_t or uint32_t.
- **LEN** -- Length of data to be sent, since the SPI peripheral sends from the MSB, this helps to shift the data to the MSB.

SPI_SWAP_DATA_RX(DATA, LEN)

Transform received data of length <= 32 bits to the format of an unsigned integer.

E.g. to transform the data of 15 bits placed in a 4-byte array to integer:

```
uint16_t data = SPI_SWAP_DATA_RX(*(uint32_t*)t->rx_data, 15);
```

Parameters

- **DATA** -- Data to be rearranged, can be uint8_t, uint16_t or uint32_t.
- **LEN** -- Length of data received, since the SPI peripheral writes from the MSB, this helps to shift the data to the LSB.

SPICOMMON_BUSFLAG_SLAVE

Initialize I/O in slave mode.

SPICOMMON_BUSFLAG_MASTER

Initialize I/O in master mode.

SPICOMMON_BUSFLAG_IOMUX_PINS

Check using iomux pins. Or indicates the pins are configured through the IO mux rather than GPIO matrix.

SPICOMMON_BUSFLAG_GPIO_PINS

Force the signals to be routed through GPIO matrix. Or indicates the pins are routed through the GPIO matrix.

SPICOMMON_BUSFLAG_SCLK

Check existing of SCLK pin. Or indicates CLK line initialized.

SPICOMMON_BUSFLAG_MISO

Check existing of MISO pin. Or indicates MISO line initialized.

SPICOMMON_BUSFLAG_MOSI

Check existing of MOSI pin. Or indicates MOSI line initialized.

SPICOMMON_BUSFLAG_DUAL

Check MOSI and MISO pins can output. Or indicates bus able to work under DIO mode.

SPICOMMON_BUSFLAG_WPHD

Check existing of WP and HD pins. Or indicates WP & HD pins initialized.

SPICOMMON_BUSFLAG_QUAD

Check existing of MOSI/MISO/WP/HD pins as output. Or indicates bus able to work under QIO mode.

SPICOMMON_BUSFLAG_IO4_IO7

Check existing of IO4~IO7 pins. Or indicates IO4~IO7 pins initialized.

SPICOMMON_BUSFLAG_OCTAL

Check existing of MOSI/MISO/WP/HD/SPIIO4/SPIIO5/SPIIO6/SPIIO7 pins as output. Or indicates bus able to work under octal mode.

SPICOMMON_BUSFLAG_NATIVE_PINS

Type Definitions

typedef *spi_common_dma_t* **spi_dma_chan_t**

Enumerations

enum **spi_common_dma_t**

SPI DMA channels.

Values:

enumerator **SPI_DMA_DISABLED**

Do not enable DMA for SPI.

enumerator **SPI_DMA_CH_AUTO**

Enable DMA, channel is automatically selected by driver.

API Reference - SPI Master

Header File

- components/esp_driver_spi/include/driver/spi_master.h
- This header file can be included with:

```
#include "driver/spi_master.h"
```

- This header file is a part of the API provided by the `esp_driver_spi` component. To declare that your component depends on `esp_driver_spi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_spi
```

or

```
PRIV_REQUIRES esp_driver_spi
```

Functions

esp_err_t **spi_bus_add_device**(*spi_host_device_t* host_id, const *spi_device_interface_config_t* *dev_config, *spi_device_handle_t* *handle)

Allocate a device on a SPI bus.

This initializes the internal structures for a device, plus allocates a CS pin on the indicated SPI master peripheral and routes it to the indicated GPIO. All SPI master devices have three CS pins and can thus control up to three devices.

There's no notable delay on chips other than ESP32.

Note: On ESP32, due to the delay of GPIO matrix, the maximum frequency SPI Master can correctly samples the slave's output is lower than the case using IOMUX. Typical maximum frequency communicating with an ideal slave without data output delay: 80MHz (IOMUX pins) and 26MHz (GPIO matrix pins). With the help of extra dummy cycles in half-duplex mode, the delay can be compensated by setting `input_delay_ns` in `dev_config` structure correctly.

Parameters

- **host_id** -- SPI peripheral to allocate device on
- **dev_config** -- SPI interface protocol config for the device
- **handle** -- Pointer to variable to hold the device handle

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid or configuration combination is not supported (e.g. `dev_config->post_cb` isn't set while flag `SPI_DEVICE_NO_RETURN_RESULT` is enabled)
- ESP_ERR_INVALID_STATE if selected clock source is unavailable or spi bus not initialized
- ESP_ERR_NOT_FOUND if host doesn't have any free CS slots
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

esp_err_t **spi_bus_remove_device**(*spi_device_handle_t* handle)

Remove a device from the SPI bus.

Parameters **handle** -- Device handle to free

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_INVALID_STATE if device already is freed
- ESP_OK on success

esp_err_t **spi_device_queue_trans**(*spi_device_handle_t* handle, *spi_transaction_t* *trans_desc, TickType_t ticks_to_wait)

Queue a SPI transaction for interrupt transaction execution. Get the result by `spi_device_get_trans_result`.

Note: Normally a device cannot start (queue) polling and interrupt transactions simultaneously.

Parameters

- **handle** -- Device handle obtained using spi_host_add_dev
- **trans_desc** -- Description of transaction to execute
- **ticks_to_wait** -- Ticks to wait until there's room in the queue; use portMAX_DELAY to never time out.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid. This can happen if SPI_TRANS_CS_KEEP_ACTIVE flag is specified while the bus was not ac-

quired (`spi_device_acquire_bus()` should be called first) or set flag SPI_TRANS_DMA_BUFFER_ALIGN_MANUAL but tx or rx buffer not DMA-capable, or addr&len not align to cache line size
- ESP_ERR_TIMEOUT if there was no room in the queue before ticks_to_wait expired
- ESP_ERR_NO_MEM if allocating DMA-capable temporary buffer failed
- ESP_ERR_INVALID_STATE if previous transactions are not finished
- ESP_OK on success

esp_err_t **spi_device_get_trans_result**(*spi_device_handle_t* handle, *spi_transaction_t* **trans_desc, TickType_t ticks_to_wait)

Get the result of a SPI transaction queued earlier by `spi_device_queue_trans`.

This routine will wait until a transaction to the given device successfully completed. It will then return the description of the completed transaction so software can inspect the result and e.g. free the memory or reuse the buffers.

Parameters
- **handle** -- Device handle obtained using spi_host_add_dev
- **trans_desc** -- Pointer to variable able to contain a pointer to the description of the transaction that is executed. The descriptor should not be modified until the descriptor is returned by spi_device_get_trans_result.
- **ticks_to_wait** -- Ticks to wait until there's a returned item; use portMAX_DELAY to never time out.

Returns
- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NOT_SUPPORTED if flag `SPI_DEVICE_NO_RETURN_RESULT` is set
- ESP_ERR_TIMEOUT if there was no completed transaction before ticks_to_wait expired
- ESP_OK on success

esp_err_t **spi_device_transmit**(*spi_device_handle_t* handle, *spi_transaction_t* *trans_desc)

Send a SPI transaction, wait for it to complete, and return the result.

This function is the equivalent of calling spi_device_queue_trans() followed by spi_device_get_trans_result(). Do not use this when there is still a transaction separately queued (started) from spi_device_queue_trans() or polling_start/transmit that hasn't been finalized.

Note: This function is not thread safe when multiple tasks access the same SPI device. Normally a device cannot start (queue) polling and interrupt transactions simutanuously.

Parameters
- **handle** -- Device handle obtained using spi_host_add_dev
- **trans_desc** -- Description of transaction to execute

Returns
- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

esp_err_t **spi_device_polling_start**(*spi_device_handle_t* handle, *spi_transaction_t* *trans_desc, TickType_t ticks_to_wait)

Immediately start a polling transaction.

Note: Normally a device cannot start (queue) polling and interrupt transactions simutanuously. Moreover, a device cannot start a new polling transaction if another polling transaction is not finished.

Parameters
- **handle** -- Device handle obtained using spi_host_add_dev
- **trans_desc** -- Description of transaction to execute

- **ticks_to_wait** -- Ticks to wait until there's room in the queue; currently only portMAX_DELAY is supported.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid. This can happen if SPI_TRANS_CS_KEEP_ACTIVE flag is specified while the bus was not acquired (`spi_device_acquire_bus()` should be called first) or set flag SPI_TRANS_DMA_BUFFER_ALIGN_MANUAL but tx or rx buffer not DMA-capable, or addr&len not align to cache line size
- ESP_ERR_TIMEOUT if the device cannot get control of the bus before `ticks_to_wait` expired
- ESP_ERR_NO_MEM if allocating DMA-capable temporary buffer failed
- ESP_ERR_INVALID_STATE if previous transactions are not finished
- ESP_OK on success

esp_err_t **spi_device_polling_end**(*spi_device_handle_t* handle, TickType_t ticks_to_wait)

Poll until the polling transaction ends.

This routine will not return until the transaction to the given device has successfully completed. The task is not blocked, but actively busy-spins for the transaction to be completed.

Parameters

- **handle** -- Device handle obtained using spi_host_add_dev
- **ticks_to_wait** -- Ticks to wait until there's a returned item; use portMAX_DELAY to never time out.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_TIMEOUT if the transaction cannot finish before ticks_to_wait expired
- ESP_OK on success

esp_err_t **spi_device_polling_transmit**(*spi_device_handle_t* handle, *spi_transaction_t* *trans_desc)

Send a polling transaction, wait for it to complete, and return the result.

This function is the equivalent of calling spi_device_polling_start() followed by spi_device_polling_end(). Do not use this when there is still a transaction that hasn't been finalized.

Note: This function is not thread safe when multiple tasks access the same SPI device. Normally a device cannot start (queue) polling and interrupt transactions simutanuously.

Parameters

- **handle** -- Device handle obtained using spi_host_add_dev
- **trans_desc** -- Description of transaction to execute

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_TIMEOUT if the device cannot get control of the bus
- ESP_ERR_NO_MEM if allocating DMA-capable temporary buffer failed
- ESP_ERR_INVALID_STATE if previous transactions of same device are not finished
- ESP_OK on success

esp_err_t **spi_device_acquire_bus**(*spi_device_handle_t* device, TickType_t wait)

Occupy the SPI bus for a device to do continuous transactions.

Transactions to all other devices will be put off until `spi_device_release_bus` is called.

Note: The function will wait until all the existing transactions have been sent.

Parameters

- **device** -- The device to occupy the bus.

- **wait** -- Time to wait before the the bus is occupied by the device. Currently MUST set to portMAX_DELAY.

Returns

- ESP_ERR_INVALID_ARG : `wait` is not set to portMAX_DELAY.
- ESP_OK : Success.

void **spi_device_release_bus**(*spi_device_handle_t* dev)

Release the SPI bus occupied by the device. All other devices can start sending transactions.

Parameters **dev** -- The device to release the bus.

esp_err_t **spi_device_get_actual_freq**(*spi_device_handle_t* handle, int *freq_khz)

Calculate working frequency for specific device.

Parameters

- **handle** -- SPI device handle
- **freq_khz** -- **[out]** output parameter to hold calculated frequency in kHz

Returns

- ESP_ERR_INVALID_ARG : `handle` or `freq_khz` parameter is NULL
- ESP_OK : Success

int **spi_get_actual_clock**(int fapb, int hz, int duty_cycle)

Calculate the working frequency that is most close to desired frequency.

Parameters

- **fapb** -- The frequency of apb clock, should be `APB_CLK_FREQ`.
- **hz** -- Desired working frequency
- **duty_cycle** -- Duty cycle of the spi clock

Returns Actual working frequency that most fit.

void **spi_get_timing**(bool gpio_is_used, int input_delay_ns, int eff_clk, int *dummy_o, int *cycles_remain_o)

Calculate the timing settings of specified frequency and settings.

Note: If **dummy_o* is not zero, it means dummy bits should be applied in half duplex mode, and full duplex mode may not work.

Parameters

- **gpio_is_used** -- True if using GPIO matrix, or False if iomux pins are used.
- **input_delay_ns** -- Input delay from SCLK launch edge to MISO data valid.
- **eff_clk** -- Effective clock frequency (in Hz) from `spi_get_actual_clock()`.
- **dummy_o** -- Address of dummy bits used output. Set to NULL if not needed.
- **cycles_remain_o** -- Address of cycles remaining (after dummy bits are used) output.
  - -1 If too many cycles remaining, suggest to compensate half a clock.
  - 0 If no remaining cycles or dummy bits are not used.
  - positive value: cycles suggest to compensate.

int **spi_get_freq_limit**(bool gpio_is_used, int input_delay_ns)

Get the frequency limit of current configurations. SPI master working at this limit is OK, while above the limit, full duplex mode and DMA will not work, and dummy bits will be applied in the half duplex mode.

Parameters

- **gpio_is_used** -- True if using GPIO matrix, or False if native pins are used.
- **input_delay_ns** -- Input delay from SCLK launch edge to MISO data valid.

Returns Frequency limit of current configurations.

esp_err_t **spi_bus_get_max_transaction_len**(*spi_host_device_t* host_id, size_t *max_bytes)

Get max length (in bytes) of one transaction.

Parameters

- **host_id** -- SPI peripheral
- **max_bytes** -- **[out]** Max length of one transaction, in bytes

Returns

- ESP_OK: On success
- ESP_ERR_INVALID_ARG: Invalid argument

Structures

struct **spi_device_interface_config_t**

This is a configuration for a SPI slave device that is connected to one of the SPI buses.

Public Members

uint8_t **command_bits**

Default amount of bits in command phase (0-16), used when `SPI_TRANS_VARIABLE_CMD` is not used, otherwise ignored.

uint8_t **address_bits**

Default amount of bits in address phase (0-64), used when `SPI_TRANS_VARIABLE_ADDR` is not used, otherwise ignored.

uint8_t **dummy_bits**

Amount of dummy bits to insert between address and data phase.

uint8_t **mode**

SPI mode, representing a pair of (CPOL, CPHA) configuration:

- 0: (0, 0)
- 1: (0, 1)
- 2: (1, 0)
- 3: (1, 1)

spi_clock_source_t **clock_source**

Select SPI clock source, `SPI_CLK_SRC_DEFAULT` by default.

uint16_t **duty_cycle_pos**

Duty cycle of positive clock, in 1/256th increments (128 = 50%/50% duty). Setting this to 0 (=not setting it) is equivalent to setting this to 128.

uint16_t **cs_ena_pretrans**

Amount of SPI bit-cycles the cs should be activated before the transmission (0-16). This only works on half-duplex transactions.

uint8_t **cs_ena_posttrans**

Amount of SPI bit-cycles the cs should stay active after the transmission (0-16)

int **clock_speed_hz**

SPI clock speed in Hz. Derived from `clock_source`.

int **input_delay_ns**

Maximum data valid time of slave. The time required between SCLK and MISO valid, including the possible clock delay from slave to master. The driver uses this value to give an extra delay before the MISO is ready on the line. Leave at 0 unless you know you need a delay. For better timing performance at high frequency (over 8MHz), it's suggest to have the right value.

int **spics_io_num**

CS GPIO pin for this device, or -1 if not used.

uint32_t **flags**

Bitwise OR of SPI_DEVICE_* flags.

int **queue_size**

Transaction queue size. This sets how many transactions can be 'in the air' (queued using spi_device_queue_trans but not yet finished using spi_device_get_trans_result) at the same time.

transaction_cb_t **pre_cb**

Callback to be called before a transmission is started.

This callback is called within interrupt context should be in IRAM for best performance, see "Transferring Speed" section in the SPI Master documentation for full details. If not, the callback may crash during flash operation when the driver is initialized with ESP_INTR_FLAG_IRAM.

transaction_cb_t **post_cb**

Callback to be called after a transmission has completed.

This callback is called within interrupt context should be in IRAM for best performance, see "Transferring Speed" section in the SPI Master documentation for full details. If not, the callback may crash during flash operation when the driver is initialized with ESP_INTR_FLAG_IRAM.

struct **spi_transaction_t**

This structure describes one SPI transaction. The descriptor should not be modified until the transaction finishes.

Public Members

uint32_t **flags**

Bitwise OR of SPI_TRANS_* flags.

uint16_t **cmd**

Command data, of which the length is set in the `command_bits` of *spi_device_interface_config_t*.

NOTE: this field, used to be "command" in ESP-IDF 2.1 and before, is re-written to be used in a new way in ESP-IDF 3.0.

Example: write 0x0123 and command_bits=12 to send command 0x12, 0x3_ (in previous version, you may have to write 0x3_12).

uint64_t **addr**

Address data, of which the length is set in the `address_bits` of *spi_device_interface_config_t*.

NOTE: this field, used to be "address" in ESP-IDF 2.1 and before, is re-written to be used in a new way in ESP-IDF3.0.

Example: write 0x123400 and address_bits=24 to send address of 0x12, 0x34, 0x00 (in previous version, you may have to write 0x12340000).

size_t **length**

Total data length, in bits.

size_t **rxlength**

Total data length received, should be not greater than `length` in full-duplex mode (0 defaults this to the value of `length`).

void ***user**

User-defined variable. Can be used to store eg transaction ID.

const void ***tx_buffer**

Pointer to transmit buffer, or NULL for no MOSI phase.

uint8_t **tx_data**[4]

If SPI_TRANS_USE_TXDATA is set, data set here is sent directly from this variable.

void ***rx_buffer**

Pointer to receive buffer, or NULL for no MISO phase. Written by 4 bytes-unit if DMA is used.

uint8_t **rx_data**[4]

If SPI_TRANS_USE_RXDATA is set, data is received directly to this variable.

struct **spi_transaction_ext_t**

This struct is for SPI transactions which may change their address and command length. Please do set the flags in base to `SPI_TRANS_VARIABLE_CMD_ADR` to use the bit length here.

Public Members

struct *spi_transaction_t* **base**

Transaction data, so that pointer to *spi_transaction_t* can be converted into *spi_transaction_ext_t*.

uint8_t **command_bits**

The command length in this transaction, in bits.

uint8_t **address_bits**

The address length in this transaction, in bits.

uint8_t **dummy_bits**

The dummy length in this transaction, in bits.

Macros

SPI_MASTER_FREQ_8M

SPI common used frequency (in Hz)

Note: SPI peripheral only has an integer divider, and the default clock source can be different on other targets, so the actual frequency may be slightly different from the desired frequency. 8MHz

SPI_MASTER_FREQ_9M

8.89MHz

SPI_MASTER_FREQ_10M

10MHz

SPI_MASTER_FREQ_11M

11.43MHz

SPI_MASTER_FREQ_13M

13.33MHz

SPI_MASTER_FREQ_16M

16MHz

SPI_MASTER_FREQ_20M

20MHz

SPI_MASTER_FREQ_26M

26.67MHz

SPI_MASTER_FREQ_40M

40MHz

SPI_MASTER_FREQ_80M

80MHz

SPI_DEVICE_TXBIT_LSBFIRST

Transmit command/address/data LSB first instead of the default MSB first.

SPI_DEVICE_RXBIT_LSBFIRST

Receive data LSB first instead of the default MSB first.

SPI_DEVICE_BIT_LSBFIRST

Transmit and receive LSB first.

SPI_DEVICE_3WIRE

Use MOSI (=spid) for both sending and receiving data.

SPI_DEVICE_POSITIVE_CS

Make CS positive during a transaction instead of negative.

SPI_DEVICE_HALFDUPLEX

Transmit data before receiving it, instead of simultaneously.

SPI_DEVICE_CLK_AS_CS

Output clock on CS line if CS is active.

SPI_DEVICE_NO_DUMMY

There are timing issue when reading at high frequency (the frequency is related to whether iomux pins are used, valid time after slave sees the clock).

- In half-duplex mode, the driver automatically inserts dummy bits before reading phase to fix the timing issue. Set this flag to disable this feature.
- In full-duplex mode, however, the hardware cannot use dummy bits, so there is no way to prevent data being read from getting corrupted. Set this flag to confirm that you're going to work with output only, or read without dummy bits at your own risk.

SPI_DEVICE_DDRCLK

SPI_DEVICE_NO_RETURN_RESULT

Don't return the descriptor to the host on completion (use post_cb to notify instead)

SPI_TRANS_MODE_DIO

Transmit/receive data in 2-bit mode.

SPI_TRANS_MODE_QIO

Transmit/receive data in 4-bit mode.

SPI_TRANS_USE_RXDATA

Receive into rx_data member of *spi_transaction_t* instead into memory at rx_buffer.

SPI_TRANS_USE_TXDATA

Transmit tx_data member of *spi_transaction_t* instead of data at tx_buffer. Do not set tx_buffer when using this.

SPI_TRANS_MODE_DIOQIO_ADDR

Also transmit address in mode selected by SPI_MODE_DIO/SPI_MODE_QIO.

SPI_TRANS_VARIABLE_CMD

Use the `command_bits` in *spi_transaction_ext_t* rather than default value in *spi_device_interface_config_t*.

SPI_TRANS_VARIABLE_ADDR

Use the `address_bits` in *spi_transaction_ext_t* rather than default value in *spi_device_interface_config_t*.

SPI_TRANS_VARIABLE_DUMMY

Use the `dummy_bits` in *spi_transaction_ext_t* rather than default value in *spi_device_interface_config_t*.

SPI_TRANS_CS_KEEP_ACTIVE

Keep CS active after data transfer.

SPI_TRANS_MULTILINE_CMD

The data lines used at command phase is the same as data phase (otherwise, only one data line is used at command phase)

SPI_TRANS_MODE_OCT

Transmit/receive data in 8-bit mode.

SPI_TRANS_MULTILINE_ADDR

The data lines used at address phase is the same as data phase (otherwise, only one data line is used at address phase)

SPI_TRANS_DMA_BUFFER_ALIGN_MANUAL

By default driver will automatically re-alloc dma buffer if it doesn't meet hardware alignment or dma_capable requirements, this flag is for you to disable this feature, you will need to take care of the alignment otherwise driver will return you error ESP_ERR_INVALID_ARG.

Type Definitions

typedef void (***transaction_cb_t**)(*spi_transaction_t* *trans)

typedef struct spi_device_t ***spi_device_handle_t**

Handle for a device on a SPI bus.

2.6.23 SPI Slave Driver

SPI Slave driver is a program that controls ESP32-S3's General Purpose SPI (GP-SPI) peripheral(s) when it functions as a slave.

For more hardware information about the GP-SPI peripheral(s), see **ESP32-S3 Technical Reference Manual > SPI Controller** [PDF].

Terminology

The terms used in relation to the SPI slave driver are given in the table below.

Term	Definition
Host	The SPI controller peripheral external to ESP32-S3 that initiates SPI transmissions over the bus, and acts as an SPI Master.
Device	SPI slave device (general purpose SPI controller). Each Device shares the MOSI, MISO and SCLK signals but is only active on the bus when the Host asserts the Device's individual CS line.
Bus	A signal bus, common to all Devices connected to one Host. In general, a bus includes the following lines: MISO, MOSI, SCLK, one or more CS lines, and, optionally, QUADWP and QUADHD. So Devices are connected to the same lines, with the exception that each Device has its own CS line. Several Devices can also share one CS line if connected in the daisy-chain manner.
MISO	Master In, Slave Out, a.k.a. Q. Data transmission from a Device to Host.
MOSI	Master Out, Slave In, a.k.a. D. Data transmission from a Host to Device.
SCLK	Serial Clock. Oscillating signal generated by a Host that keeps the transmission of data bits in sync.
CS	Chip Select. Allows a Host to select individual Device(s) connected to the bus in order to send or receive data.
QUADWP	Write Protect signal. Only used for 4-bit (qio/qout) transactions.
QUADHD	Hold signal. Only used for 4-bit (qio/qout) transactions.
Assertion	The action of activating a line. The opposite action of returning the line back to inactive (back to idle) is called **de-assertion**.
Transaction	One instance of a Host asserting a CS line, transferring data to and from a Device, and de-asserting the CS line. Transactions are atomic, which means they can never be interrupted by another transaction.
Launch Edge	Edge of the clock at which the source register **launches** the signal onto the line.
Latch Edge	Edge of the clock at which the destination register **latches in** the signal.

Driver Features

The SPI slave driver allows using the SPI peripherals as full-duplex Devices. The driver can send/receive transactions up to 64 bytes in length, or utilize DMA to send/receive longer transactions. However, there are some *known issues* related to DMA.

The SPI slave driver supports registering the SPI ISR to a certain CPU core. If multiple tasks try to access the same SPI Device simultaneously, it is recommended that your application be refactored so that each SPI peripheral is only accessed by a single task at a time. Please also use `spi_bus_config_t::isr_cpu_id` to register the SPI ISR to the same core as SPI peripheral related tasks to ensure thread safety.

SPI Transactions

A full-duplex SPI transaction begins when the Host asserts the CS line and starts sending out clock pulses on the SCLK line. Every clock pulse, a data bit is shifted from the Host to the Device on the MOSI line and back on the MISO line at the same time. At the end of the transaction, the Host de-asserts the CS line.

The attributes of a transaction are determined by the configuration structure for an SPI peripheral acting as a slave device `spi_slave_interface_config_t`, and transaction configuration structure `spi_slave_transaction_t`.

As not every transaction requires both writing and reading data, you can choose to configure the `spi_transaction_t` structure for TX only, RX only, or TX and RX transactions. If `spi_slave_transaction_t::rx_buffer` is set to `NULL`, the read phase will be skipped. Similarly, if `spi_slave_transaction_t::tx_buffer` is set to `NULL`, the write phase will be skipped.

Note: A Host should not start a transaction before its Device is ready for receiving data. It is recommended to use another GPIO pin for a handshake signal to sync the Devices. For more details, see *Transaction Interval*.

Driver Usage

- Initialize an SPI peripheral as a Device by calling the function *spi_slave_initialize()*. Make sure to set the correct I/O pins in the struct *bus_config*. Set the unused signals to `-1`.
- Before initiating transactions, fill one or more *spi_slave_transaction_t* structs with the transaction parameters required. Either queue all transactions by calling the function *spi_slave_queue_trans()* and, at a later time, query the result by using the function *spi_slave_get_trans_result()*, or handle all requests individually by feeding them into *spi_slave_transmit()*. The latter two functions will be blocked until the Host has initiated and finished a transaction, causing the queued data to be sent and received.
- (Optional) To unload the SPI slave driver, call *spi_slave_free()*.

Transaction Data and Master/Slave Length Mismatches

Normally, the data that needs to be transferred to or from a Device is read or written to a chunk of memory indicated by the *spi_slave_transaction_t::rx_buffer* and *spi_slave_transaction_t::tx_buffer*. The SPI driver can be configured to use DMA for transfers, in which case these buffers must be allocated in DMA-capable memory using `pvPortMallocCaps(size, MALLOC_CAP_DMA)`.

The amount of data that the driver can read or write to the buffers is limited by *spi_slave_transaction_t::length*. However, this member does not define the actual length of an SPI transaction. A transaction's length is determined by the clock and CS lines driven by the Host. The actual length of the transmission can be read only after a transaction is finished from the member *spi_slave_transaction_t::trans_len*.

If the length of the transmission is greater than the buffer length, only the initial number of bits specified in the *spi_slave_transaction_t::length* member will be sent and received. In this case, *spi_slave_transaction_t::trans_len* is set to *spi_slave_transaction_t::length* instead of the actual transaction length. To meet the actual transaction length requirements, set *spi_slave_transaction_t::length* to a value greater than the maximum *spi_slave_transaction_t::trans_len* expected. If the transmission length is shorter than the buffer length, only the data equal to the length of the buffer will be transmitted.

GPIO Matrix and IO_MUX Most of chip's peripheral signals have direct connection to their dedicated IO_MUX pins. However, the signals can also be routed to any other available pins using the less direct GPIO matrix. If at least one signal is routed through the GPIO matrix, then all signals will be routed through it.

When an SPI Host is set to 80 MHz or lower frequencies, routing SPI pins via GPIO matrix will behave the same compared to routing them via IO_MUX.

The IO_MUX pins for SPI buses are given below.

Pin Name	GPIO Number (SPI2)
CS0	10
SCLK	12
MISO	13
MOSI	11
QUADWP	14
QUADHD	9

Speed and Timing Considerations

Transaction Interval The ESP32-S3 SPI slave peripherals are designed as general purpose Devices controlled by a CPU. As opposed to dedicated slaves, CPU-based SPI Devices have a limited number of pre-defined registers. All transactions must be handled by the CPU, which means that the transfers and responses are not real-time, and there might be noticeable latency.

As a solution, a Device's response rate can be doubled by using the functions *spi_slave_queue_trans()* and then *spi_slave_get_trans_result()* instead of using *spi_slave_transmit()*.

You can also configure a GPIO pin through which the Device will signal to the Host when it is ready for a new transaction. A code example of this can be found in peripherals/spi_slave.

SCLK Frequency Requirements The SPI slaves are designed to operate at up to 60 MHz. The data cannot be recognized or received correctly if the clock is too fast or does not have a 50% duty cycle.

Restrictions and Known Issues

1. If DMA is enabled, the rx buffer should be word-aligned (starting from a 32-bit boundary and having a length of multiples of 4 bytes). Otherwise, DMA may write incorrectly or not in a boundary aligned manner. The driver reports an error if this condition is not satisfied.
   Also, a Host should write lengths that are multiples of 4 bytes. The data with inappropriate lengths will be discarded.

Application Example

The code example for Device/Host communication can be found in the peripherals/spi_slave directory of ESP-IDF examples.

API Reference

Header File

- components/esp_driver_spi/include/driver/spi_slave.h
- This header file can be included with:

```
#include "driver/spi_slave.h"
```

- This header file is a part of the API provided by the `esp_driver_spi` component. To declare that your component depends on `esp_driver_spi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_spi
```

or

```
PRIV_REQUIRES esp_driver_spi
```

Functions

esp_err_t **spi_slave_initialize**(*spi_host_device_t* host, const *spi_bus_config_t* *bus_config, const *spi_slave_interface_config_t* *slave_config, *spi_dma_chan_t* dma_chan)

Initialize a SPI bus as a slave interface.

Warning: SPI0/1 is not supported

Warning: If a DMA channel is selected, any transmit and receive buffer used should be allocated in DMA-capable memory.

Warning: The ISR of SPI is always executed on the core which calls this function. Never starve the ISR on this core or the SPI transactions will not be handled.

Parameters

- **host** -- SPI peripheral to use as a SPI slave interface
- **bus_config** -- Pointer to a *spi_bus_config_t* struct specifying how the host should be initialized
- **slave_config** -- Pointer to a *spi_slave_interface_config_t* struct specifying the details for the slave interface
- **dma_chan** -- - Selecting a DMA channel for an SPI bus allows transactions on the bus with size only limited by the amount of internal memory.
  - Selecting SPI_DMA_DISABLED limits the size of transactions.
  - Set to SPI_DMA_DISABLED if only the SPI flash uses this bus.
  - Set to SPI_DMA_CH_AUTO to let the driver to allocate the DMA channel.

Returns

- ESP_ERR_INVALID_ARG if configuration is invalid
- ESP_ERR_INVALID_STATE if host already is in use
- ESP_ERR_NOT_FOUND if there is no available DMA channel
- ESP_ERR_NO_MEM if out of memory
- ESP_OK on success

esp_err_t **spi_slave_free** (*spi_host_device_t* host)

Free a SPI bus claimed as a SPI slave interface.

Parameters **host** -- SPI peripheral to free

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_INVALID_STATE if not all devices on the bus are freed
- ESP_OK on success

esp_err_t **spi_slave_queue_trans** (*spi_host_device_t* host, const *spi_slave_transaction_t* *trans_desc, TickType_t ticks_to_wait)

Queue a SPI transaction for execution.

Queues a SPI transaction to be executed by this slave device. (The transaction queue size was specified when the slave device was initialised via spi_slave_initialize.) This function may block if the queue is full (depending on the ticks_to_wait parameter). No SPI operation is directly initiated by this function, the next queued transaction will happen when the master initiates a SPI transaction by pulling down CS and sending out clock signals.

This function hands over ownership of the buffers in `trans_desc` to the SPI slave driver; the application is not to access this memory until `spi_slave_queue_trans` is called to hand ownership back to the application.

Note: On esp32, if trans length not WORD aligned, the rx buffer last word memory will still overwritten by DMA HW

Parameters

- **host** -- SPI peripheral that is acting as a slave
- **trans_desc** -- Description of transaction to execute. Not const because we may want to write status back into the transaction description.
- **ticks_to_wait** -- Ticks to wait until there's room in the queue; use port-MAX_DELAY to never time out.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NO_MEM if set flag `SPI_SLAVE_TRANS_DMA_BUFFER_ALIGN_AUTO` but there is no free memory
- ESP_ERR_INVALID_STATE if sync data between Cache and memory failed
- ESP_OK on success

esp_err_t **spi_slave_get_trans_result**(*spi_host_device_t* host, *spi_slave_transaction_t* **trans_desc, TickType_t ticks_to_wait)

Get the result of a SPI transaction queued earlier.

This routine will wait until a transaction to the given device (queued earlier with spi_slave_queue_trans) has successfully completed. It will then return the description of the completed transaction so software can inspect the result and e.g. free the memory or reuse the buffers.

It is mandatory to eventually use this function for any transaction queued by `spi_slave_queue_trans`.

Parameters

- **host** -- SPI peripheral to that is acting as a slave
- **trans_desc** -- **[out]** Pointer to variable able to contain a pointer to the description of the transaction that is executed
- **ticks_to_wait** -- Ticks to wait until there's a returned item; use portMAX_DELAY to never time out.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_ERR_NOT_SUPPORTED if flag `SPI_SLAVE_NO_RETURN_RESULT` is set
- ESP_OK on success

esp_err_t **spi_slave_transmit**(*spi_host_device_t* host, *spi_slave_transaction_t* *trans_desc, TickType_t ticks_to_wait)

Do a SPI transaction.

Essentially does the same as spi_slave_queue_trans followed by spi_slave_get_trans_result. Do not use this when there is still a transaction queued that hasn't been finalized using spi_slave_get_trans_result.

Parameters

- **host** -- SPI peripheral to that is acting as a slave
- **trans_desc** -- Pointer to variable able to contain a pointer to the description of the transaction that is executed. Not const because we may want to write status back into the transaction description.
- **ticks_to_wait** -- Ticks to wait until there's a returned item; use portMAX_DELAY to never time out.

Returns

- ESP_ERR_INVALID_ARG if parameter is invalid
- ESP_OK on success

Structures

struct **spi_slave_interface_config_t**

This is a configuration for a SPI host acting as a slave device.

Public Members

int **spics_io_num**

CS GPIO pin for this device.

uint32_t **flags**

Bitwise OR of SPI_SLAVE_* flags.

int **queue_size**

Transaction queue size. This sets how many transactions can be 'in the air' (queued using spi_slave_queue_trans but not yet finished using spi_slave_get_trans_result) at the same time.

uint8_t **mode**

SPI mode, representing a pair of (CPOL, CPHA) configuration:

- 0: (0, 0)
- 1: (0, 1)
- 2: (1, 0)
- 3: (1, 1)

slave_transaction_cb_t **post_setup_cb**

Callback called after the SPI registers are loaded with new data.

This callback is called within interrupt context should be in IRAM for best performance, see "Transferring Speed" section in the SPI Master documentation for full details. If not, the callback may crash during flash operation when the driver is initialized with ESP_INTR_FLAG_IRAM.

slave_transaction_cb_t **post_trans_cb**

Callback called after a transaction is done.

This callback is called within interrupt context should be in IRAM for best performance, see "Transferring Speed" section in the SPI Master documentation for full details. If not, the callback may crash during flash operation when the driver is initialized with ESP_INTR_FLAG_IRAM.

struct **spi_slave_transaction_t**

This structure describes one SPI transaction

Public Members

uint32_t **flags**

Bitwise OR of SPI_SLAVE_TRANS_* flags.

size_t **length**

Total data length, in bits.

size_t **trans_len**

Transaction data length, in bits.

const void ***tx_buffer**

Pointer to transmit buffer, or NULL for no MOSI phase.

void ***rx_buffer**

Pointer to receive buffer, or NULL for no MISO phase. When the DMA is enabled, must start at WORD boundary (`rx_buffer%4==0`), and has length of a multiple of 4 bytes.

void ***user**

User-defined variable. Can be used to store eg transaction ID.

Macros

SPI_SLAVE_TXBIT_LSBFIRST

Transmit command/address/data LSB first instead of the default MSB first.

SPI_SLAVE_RXBIT_LSBFIRST

Receive data LSB first instead of the default MSB first.

SPI_SLAVE_BIT_LSBFIRST

Transmit and receive LSB first.

SPI_SLAVE_NO_RETURN_RESULT

Don't return the descriptor to the host on completion (use `post_trans_cb` to notify instead)

SPI_SLAVE_TRANS_DMA_BUFFER_ALIGN_AUTO

Automatically re-malloc dma buffer if user buffer doesn't meet hardware alignment or dma_capable, this process may loss some memory and performance.

Type Definitions

typedef void (***slave_transaction_cb_t**)(*spi_slave_transaction_t* *trans)

2.6.24 SPI Slave Half Duplex

Introduction

The Half Duplex (HD) Mode is a special mode provided by ESP SPI Slave peripheral. Under this mode, the hardware provides more services than the Full Duplex (FD) Mode (the mode for general-purpose SPI transactions, see *SPI Slave Driver*). These services reduce the CPU load and the response time of SPI Slave. However, it is important to note that the communication format is determined by the hardware and is always in a half-duplex configuration, allowing only one-way data transfer at any given time. Hence, the mode is named Half Duplex Mode due to this characteristic.

When conducting an SPI transaction, transactions can be classified into several types based on the **command** phase of the transaction. Each transaction may consist of the following phases: command, address, dummy, and data. The command phase is mandatory, while the other phases may be determined by the command field. During the command, address, and dummy phases, the bus is always controlled by the master (usually the host), while the direction of the data phase depends on the command. The data phase can be either an input phase, where the master writes data to the slave (e.g., the host sends data to the slave), or an output phase, where the master reads data from the slave (e.g., the host receives data from the slave).

Protocol About the details of how master should communicate with the SPI Slave, see *ESP SPI Slave HD (Half Duplex) Mode Protocol*.

Through these different transactions, the slave provides these services to the master:

- A DMA channel for the master to write a great amount of data to the slave.
- A DMA channel for the master to read a great amount of data from the slave.
- Several general purpose registers, shared between the master and the slave.
- Several general purpose interrupts, for the master to interrupt the SW of the slave.

Terminology

- Transaction
- Channel
- Sending
- Receiving
- Data Descriptor

Driver Feature

- Transaction read/write by master in segments
- Queues for data to send and received

Driver Usage

Slave Initialization Call *spi_slave_hd_init()* to initialize the SPI bus as well as the peripheral and the driver. The SPI Slave exclusively uses the SPI peripheral, pins of the bus before it is deinitialized, which means other devices are unable to use the above resources during initialization. Thus, to ensure SPI resources are correctly occupied and the connections work properly, most configurations of the slave should be done as soon as the slave is initialized.

The *spi_bus_config_t* specifies how the bus should be initialized, while *spi_slave_hd_slot_config_t* specifies how the SPI Slave driver should work.

Deinitialization (Optional) Call *spi_slave_hd_deinit()* to uninstall the driver. The resources, including the pins, SPI peripheral, internal memory used by the driver, and interrupt sources, are released by the `deinit()` function.

Send/Receive Data by DMA Channels To send data to the master through the sending DMA channel, the application should properly wrap the data in an *spi_slave_hd_data_t* descriptor structure before calling *spi_slave_hd_queue_trans()* with the data descriptor and the channel argument of *SPI_SLAVE_CHAN_TX*. The pointers to descriptors are stored in the queue, and the data is sent to the master in the same order they are enqueued using *spi_slave_hd_queue_trans()*, upon receiving the master's `Rd_DMA` command.

The application should check the result of data sending by calling *spi_slave_hd_get_trans_res()* with the channel set as *SPI_SLAVE_CHAN_TX*. This function blocks until the transaction with the command `Rd_DMA` from the master successfully completes (or timeout). The `out_trans` argument of the function outputs the pointer of the data descriptor which is just finished, providing information about the sending.

Receiving data from the master through the receiving DMA channel is quite similar. The application calls *spi_slave_hd_queue_trans()* with proper data descriptor and the channel argument of *SPI_SLAVE_CHAN_RX*. And the application calls the *spi_slave_hd_get_trans_res()* later to get the descriptor to the receiving buffer before it handles the data in the receiving buffer.

Note: This driver itself does not have an internal buffer for the data to send or just received. The application should provide data buffer for driver via data descriptors to send to the master, or to receive data from the master.

The application has to properly keep the data descriptor as well as the buffer it points, after the descriptor is successfully sent into the driver internal queue by *spi_slave_hd_queue_trans()*, and before returned by *spi_slave_hd_get_trans_res()*. During this period, the hardware as well as the driver may read or write to the buffer and the descriptor when required at any time.

Please note that, when using this driver for data transfer, the buffer does not have to be fully sent or filled before it is terminated. For example, in the segment transaction mode, the master has to send `CMD7` to terminate a `Wr_DMA` transaction or send `CMD8` to terminate an `Rd_DMA` transaction (in segments), no matter whether the send (receive) buffer is used up (full) or not.

Using Data Descriptor with Customized User Arguments Sometimes you may have initiator (sending data descriptor) and closure (handling returned descriptors) functions in different places. When you get the returned data descriptor in the closure, you may need some extra information when handling the finished data descriptor. For example, you may want to know which round it is for the returned descriptor when you send the same piece of data several times.

Set the `arg` member in the data descriptor to a variable indicating the transaction by force casting, or point it to a structure that wraps all the information you may need when handling the sending/receiving data. Then you can get what you need in your closure.

Using Callbacks

Note: These callbacks are called in the ISR, so the required operations need to be processed quickly and returned as soon as possible to ensure that the system is functioning properly. You may need to be very careful to write the code in the ISR.

Since the interrupt handling is executed concurrently with the application, long delays or blocking may cause the system to respond slower or lead to unpredictable behavior. Therefore, when writing callback functions, avoid using operations that may cause delays or blocking, e.g., waiting, sleeping, resource locking, etc.

The *`spi_slave_hd_callback_config_t`* member in the *`spi_slave_hd_slot_config_t`* configuration structure passed when initializing the SPI Slave HD driver, allows you to have callbacks for each event you may concern.

The corresponding interrupt for each callback that is not **NULL** is enabled, so that the callbacks can be called immediately when the events happen. You do not need to provide callbacks for the unconcerned events.

The `arg` member in the configuration structure can help you pass some context to the callback or indicate the specific SPI Slave instance when using the same callbacks for multiple SPI Slave peripherals. You can set the arg member to a variable that indicates the SPI Slave instance by performing a forced type casting or point it to a context structure. All the callbacks are called with this `arg` argument you set when the callbacks are initialized.

There are two other arguments: the `event` and the `awoken`.

- The `event` passes the information of the current event to the callback. The *`spi_slave_hd_event_t`* type contains the information of the event, for example, event type, the data descriptor just finished (The *data argument* is very useful in this case!).
- The `awoken` argument serves as an output parameter. It informs the ISR that tasks have been awakened after the callback function, and the ISR should call *portYIELD_FROM_ISR()* to schedule these tasks. Simply pass the `awoken` argument to all FreeRTOS APIs that may unblock tasks, and the value of `awoken` will be returned to the ISR.

Writing/Reading Shared Registers

Call *`spi_slave_hd_write_buffer()`* to write the shared buffer, and *`spi_slave_hd_read_buffer()`* to read the shared buffer.

Note: On ESP32-S3, the shared registers are read/written in words by the application but read/written in bytes by the master. There is no guarantee four continuous bytes read from the master are from the same word written by the slave's application. It is also possible that if the slave reads a word while the master is writing bytes of the word, the slave may get one word with half of them just written by the master, and the other half has not been written into.

The master can confirm that the word is not in transition by reading the word twice and comparing the values.

For the slave, it is more difficult to ensure the word is not in transition because the process of master writing four bytes can be very long (32 SPI clocks). You can put some CRC in the last (largest address) byte of a word so that when the byte is written, the word is sure to be all written.

Due to the conflicts that may be among read/write from SW (worse if there are multi-cores) and master, it is suggested that a word is only used in one direction (only written by the master or only written by the slave).

Receiving General Purpose Interrupts from the Master

When the master sends `CMD8`, `CMD9` or `CMDA`, the slave corresponding is triggered. Currently the `CMD8` is permanently used to indicate the termination of `Rd_DMA` segments. To receive general-purpose interrupts, register callbacks for `CMD9` and `CMDA` when the slave is initialized, see *Using Callbacks*.

Application Example

The code example for Device/Host communication can be found in the peripherals/spi_slave_hd directory of ESP-IDF examples.

API Reference

Header File

- components/esp_driver_spi/include/driver/spi_slave_hd.h
- This header file can be included with:

```
#include "driver/spi_slave_hd.h"
```

- This header file is a part of the API provided by the `esp_driver_spi` component. To declare that your component depends on `esp_driver_spi`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_spi
```

or

```
PRIV_REQUIRES esp_driver_spi
```

Functions

esp_err_t **spi_slave_hd_init**(*spi_host_device_t* host_id, const *spi_bus_config_t* *bus_config, const *spi_slave_hd_slot_config_t* *config)

Initialize the SPI Slave HD driver.

Parameters
- **host_id** -- The host to use
- **bus_config** -- Bus configuration for the bus used
- **config** -- Configuration for the SPI Slave HD driver

Returns
- ESP_OK: on success
- ESP_ERR_INVALID_ARG: invalid argument given
- ESP_ERR_INVALID_STATE: function called in invalid state, may be some resources are already in use
- ESP_ERR_NOT_FOUND if there is no available DMA channel
- ESP_ERR_NO_MEM: memory allocation failed
- or other return value from `esp_intr_alloc`

esp_err_t **spi_slave_hd_deinit**(*spi_host_device_t* host_id)

Deinitialize the SPI Slave HD driver.

Parameters **host_id** -- The host to deinitialize the driver

Returns
- ESP_OK: on success
- ESP_ERR_INVALID_ARG: if the host_id is not correct

esp_err_t **spi_slave_hd_queue_trans**(*spi_host_device_t* host_id, *spi_slave_chan_t* chan, *spi_slave_hd_data_t* *trans, TickType_t timeout)

Queue transactions (segment mode)

Parameters
- **host_id** -- Host to queue the transaction
- **chan** -- SPI_SLAVE_CHAN_TX or SPI_SLAVE_CHAN_RX
- **trans** -- Transaction descriptors
- **timeout** -- Timeout before the data is queued

Returns
- ESP_OK: on success
- ESP_ERR_INVALID_ARG: The input argument is invalid. Can be the following reason:

  - The buffer given is not DMA capable
  - The length of data is invalid (not larger than 0, or exceed the max transfer length)
  - The transaction direction is invalid
- ESP_ERR_TIMEOUT: Cannot queue the data before timeout. Master is still processing previous transaction.
- ESP_ERR_INVALID_STATE: Function called in invalid state. This API should be called under segment mode.

esp_err_t **spi_slave_hd_get_trans_res**(*spi_host_device_t* host_id, *spi_slave_chan_t* chan, *spi_slave_hd_data_t* **out_trans, TickType_t timeout)

Get the result of a data transaction (segment mode)

Note: This API should be called successfully the same times as the `spi_slave_hd_queue_trans`.

Parameters

- **host_id** -- Host to queue the transaction
- **chan** -- Channel to get the result, SPI_SLAVE_CHAN_TX or SPI_SLAVE_CHAN_RX
- **out_trans** -- **[out]** Pointer to the transaction descriptor (*spi_slave_hd_data_t*) passed to the driver before. Hardware has finished this transaction. Member `trans_len` indicates the actual number of bytes of received data, it's meaningless for TX.
- **timeout** -- Timeout before the result is got

Returns

- ESP_OK: on success
- ESP_ERR_INVALID_ARG: Function is not valid
- ESP_ERR_TIMEOUT: There's no transaction done before timeout
- ESP_ERR_INVALID_STATE: Function called in invalid state. This API should be called under segment mode.

void **spi_slave_hd_read_buffer**(*spi_host_device_t* host_id, int addr, uint8_t *out_data, size_t len)

Read the shared registers.

Parameters

- **host_id** -- Host to read the shared registers
- **addr** -- Address of register to read, 0 to `SOC_SPI_MAXIMUM_BUFFER_SIZE-1`
- **out_data** -- **[out]** Output buffer to store the read data
- **len** -- Length to read, not larger than `SOC_SPI_MAXIMUM_BUFFER_SIZE-addr`

void **spi_slave_hd_write_buffer**(*spi_host_device_t* host_id, int addr, uint8_t *data, size_t len)

Write the shared registers.

Parameters

- **host_id** -- Host to write the shared registers
- **addr** -- Address of register to write, 0 to `SOC_SPI_MAXIMUM_BUFFER_SIZE-1`
- **data** -- Buffer holding the data to write
- **len** -- Length to write, `SOC_SPI_MAXIMUM_BUFFER_SIZE-addr`

esp_err_t **spi_slave_hd_append_trans**(*spi_host_device_t* host_id, *spi_slave_chan_t* chan, *spi_slave_hd_data_t* *trans, TickType_t timeout)

Load transactions (append mode)

Note: In this mode, user transaction descriptors will be appended to the DMA and the DMA will keep processing the data without stopping

---

Parameters

- **host_id** -- Host to load transactions
- **chan** -- SPI_SLAVE_CHAN_TX or SPI_SLAVE_CHAN_RX

- **trans** -- Transaction descriptor
- **timeout** -- Timeout before the transaction is loaded

Returns

- ESP_OK: on success
- ESP_ERR_INVALID_ARG: The input argument is invalid. Can be the following reason:
  - The buffer given is not DMA capable
  - The length of data is invalid (not larger than 0, or exceed the max transfer length)
  - The transaction direction is invalid
- ESP_ERR_TIMEOUT: Master is still processing previous transaction. There is no available transaction for slave to load
- ESP_ERR_INVALID_STATE: Function called in invalid state. This API should be called under append mode.

esp_err_t **spi_slave_hd_get_append_trans_res**(*spi_host_device_t* host_id, *spi_slave_chan_t* chan, *spi_slave_hd_data_t* **out_trans, TickType_t timeout)

Get the result of a data transaction (append mode)

Note: This API should be called the same times as the `spi_slave_hd_append_trans`

Parameters

- **host_id** -- Host to load the transaction
- **chan** -- SPI_SLAVE_CHAN_TX or SPI_SLAVE_CHAN_RX
- **out_trans** -- **[out]** Pointer to the transaction descriptor (*spi_slave_hd_data_t*) passed to the driver before. Hardware has finished this transaction. Member `trans_len` indicates the actual number of bytes of received data, it's meaningless for TX.
- **timeout** -- Timeout before the result is got

Returns

- ESP_OK: on success
- ESP_ERR_INVALID_ARG: Function is not valid
- ESP_ERR_TIMEOUT: There's no transaction done before timeout
- ESP_ERR_INVALID_STATE: Function called in invalid state. This API should be called under append mode.

Structures

struct **spi_slave_hd_data_t**

Descriptor of data to send/receive.

Public Members

uint8_t ***data**

Buffer to send, must be DMA capable.

size_t **len**

Len of data to send/receive. For receiving the buffer length should be multiples of 4 bytes, otherwise the extra part will be truncated.

size_t **trans_len**

For RX direction, it indicates the data actually received. For TX direction, it is meaningless.

uint32_t **flags**

Bitwise OR of SPI_SLAVE_HD_TRANS_* flags.

void ***arg**

Extra argument indicating this data.

struct **spi_slave_hd_event_t**

Information of SPI Slave HD event.

Public Members

spi_event_t **event**

Event type.

spi_slave_hd_data_t ***trans**

Corresponding transaction for SPI_EV_SEND and SPI_EV_RECV events.

struct **spi_slave_hd_callback_config_t**

Callback configuration structure for SPI Slave HD.

Public Members

slave_cb_t **cb_buffer_tx**

Callback when master reads from shared buffer.

slave_cb_t **cb_buffer_rx**

Callback when master writes to shared buffer.

slave_cb_t **cb_send_dma_ready**

Callback when TX data buffer is loaded to the hardware (DMA)

slave_cb_t **cb_sent**

Callback when data are sent.

slave_cb_t **cb_recv_dma_ready**

Callback when RX data buffer is loaded to the hardware (DMA)

slave_cb_t **cb_recv**

Callback when data are received.

slave_cb_t **cb_cmd9**

Callback when CMD9 received.

slave_cb_t **cb_cmdA**

Callback when CMDA received.

void ***arg**

Argument indicating this SPI Slave HD peripheral instance.

struct **spi_slave_hd_slot_config_t**

Configuration structure for the SPI Slave HD driver.

Public Members

uint8_t **mode**

SPI mode, representing a pair of (CPOL, CPHA) configuration:

- 0: (0, 0)
- 1: (0, 1)
- 2: (1, 0)
- 3: (1, 1)

uint32_t **spics_io_num**

CS GPIO pin for this device.

uint32_t **flags**

Bitwise OR of SPI_SLAVE_HD_* flags.

uint32_t **command_bits**

command field bits, multiples of 8 and at least 8.

uint32_t **address_bits**

address field bits, multiples of 8 and at least 8.

uint32_t **dummy_bits**

dummy field bits, multiples of 8 and at least 8.

uint32_t **queue_size**

Transaction queue size. This sets how many transactions can be 'in the air' (queued using spi_slave_hd_queue_trans but not yet finished using spi_slave_hd_get_trans_result) at the same time.

spi_dma_chan_t **dma_chan**

DMA channel to used.

spi_slave_hd_callback_config_t **cb_config**

Callback configuration.

Macros

SPI_SLAVE_HD_TRANS_DMA_BUFFER_ALIGN_AUTO

Automatically re-malloc dma buffer if user buffer doesn't meet hardware alignment or dma_capable, this process may lose some memory and performance.

SPI_SLAVE_HD_TXBIT_LSBFIRST

Transmit command/address/data LSB first instead of the default MSB first.

SPI_SLAVE_HD_RXBIT_LSBFIRST

Receive data LSB first instead of the default MSB first.

SPI_SLAVE_HD_BIT_LSBFIRST

Transmit and receive LSB first.

SPI_SLAVE_HD_APPEND_MODE

Adopt DMA append mode for transactions. In this mode, users can load(append) DMA descriptors without stopping the DMA.

Type Definitions

typedef bool (***slave_cb_t**)(void *arg, *spi_slave_hd_event_t* *event, BaseType_t *awoken)

Callback for SPI Slave HD.

Enumerations

enum **spi_slave_chan_t**

Channel of SPI Slave HD to do data transaction.

Values:

enumerator **SPI_SLAVE_CHAN_TX**

The output channel (RDDMA)

enumerator **SPI_SLAVE_CHAN_RX**

The input channel (WRDMA)

2.6.25 Temperature Sensor

Introduction

The ESP32-S3 has a built-in sensor used to measure the chip's internal temperature. The temperature sensor module contains an 8-bit Sigma-Delta analog-to-digital converter (ADC) and a digital-to-analog converter (DAC) to compensate for the temperature measurement.

Due to restrictions of hardware, the sensor has predefined measurement ranges with specific measurement errors. See the table below for details.

Predefined Range (°C)	Error (°C)
50 ~ 125	< 3
20 ~ 100	< 2
-10 ~ 80	< 1
-30 ~ 50	< 2
-40 ~ 20	< 3

Note: The temperature sensor is designed primarily to measure the temperature changes inside the chip. The internal temperature of a chip is usually higher than the ambient temperature, and is affected by factors such as the microcontroller's clock frequency or I/O load, and the external thermal environment.

Functional Overview

The description of the temperature sensor functionality is divided into the following sections:

- *Resource Allocation* - covers which parameters should be set up to get a temperature sensor handle and how to recycle the resources when the temperature sensor finishes working.
- *Enable and Disable Temperature Sensor* - covers how to enable and disable the temperature sensor.
- *Get Temperature Value* - covers how to get the real-time temperature value.
- *Power Management* - covers how the temperature sensor is affected when changing power mode (e.g., Light-sleep mode).
- *Thread Safety* - covers how to make the driver to be thread-safe.

Resource Allocation The ESP32-S3 has just one built-in temperature sensor hardware. The temperature sensor instance is represented by `temperature_sensor_handle_t`, which is also the bond of the context. By using `temperature_sensor_handle_t`, the temperature sensor properties can be accessed and modified in different function calls to control and manage the temperature sensor. The variable would always be the parameter of the temperature APIs with the information of hardware and configurations, so you can just create a pointer of type `temperature_sensor_handle_t` and passing to APIs as needed.

In order to install a built-in temperature sensor instance, the first thing is to evaluate the temperature range in your detection environment. For example, if the testing environment is in a room, the range you evaluate might be 10 °C ~ 30 °C; if the testing in a lamp bulb, the range you evaluate might be 60 °C ~ 110 °C. Based on that, configuration structure `temperature_sensor_config_t` should be defined in advance:

- `range_min`: The minimum value of the testing range you have evaluated.
- `range_max`: The maximum value of the testing range you have evaluated.

After the ranges are set, the structure could be passed to `temperature_sensor_install()`, which will instantiate the temperature sensor instance and return a handle.

As mentioned above, different measure ranges have different measurement errors. You do not need to care about the measurement error because we have an internal mechanism to choose the minimum error according to the given range.

If the temperature sensor is no longer needed, you need to call `temperature_sensor_uninstall()` to free the temperature sensor resource.

Creating a Temperature Sensor Handle

- Step 1: Evaluate the testing range. In this example, the range is 20 °C ~ 50 °C.
- Step 2: Configure the range and obtain a handle.

```
temperature_sensor_handle_t temp_handle = NULL;
temperature_sensor_config_t temp_sensor_config = TEMPERATURE_SENSOR_CONFIG_
↪DEFAULT(20, 50);
ESP_ERROR_CHECK(temperature_sensor_install(&temp_sensor_config, &temp_handle));
```

Enable and Disable Temperature Sensor

1. Enable the temperature sensor by calling `temperature_sensor_enable()`. The internal temperature sensor circuit will start to work. The driver state will transit from init to enable.
2. To Disable the temperature sensor, please call `temperature_sensor_disable()`.

Get Temperature Value After the temperature sensor is enabled by `temperature_sensor_enable()`, you can get the current temperature by calling `temperature_sensor_get_celsius()`.

```
// Enable temperature sensor
ESP_ERROR_CHECK(temperature_sensor_enable(temp_handle));
// Get converted sensor data
float tsens_out;
ESP_ERROR_CHECK(temperature_sensor_get_celsius(temp_handle, &tsens_out));
printf("Temperature in %f °C\n", tsens_out);
// Disable the temperature sensor if it is not needed and save the power
ESP_ERROR_CHECK(temperature_sensor_disable(temp_handle));
```

Power Management As the temperature sensor does not use the APB clock, it will keep working no matter if the power management is enabled with `CONFIG_PM_ENABLE`.

Thread Safety In the temperature sensor driver, we do not add any protection to ensure the thread safety, because typically this driver is only supposed to be used in one task. If you have to use this driver in different tasks, please add extra locks to protect it.

Unexpected Behaviors

1. The value you get from the chip is usually different from the ambient temperature. It is because the temperature sensor is built inside the chip. To some extent, it measures the temperature of the chip.
2. When installing the temperature sensor, the driver may print `the boundary you gave cannot meet the range of internal temperature sensor`. It is because the built-in temperature sensor has a testing limit. The error comes from the incorrect configuration of *`temperature_sensor_config_t`* as follow:
   (1) Totally out of range, like 200 °C ~ 300 °C.
   (2) Cross the boundary of each predefined measurement. like 40 °C ~ 110 °C.

Application Example

- Temperature sensor reading example: peripherals/temperature_sensor/temp_sensor.

API Reference

Header File

- components/esp_driver_tsens/include/driver/temperature_sensor.h
- This header file can be included with:

  ```
  #include "driver/temperature_sensor.h"
  ```

- This header file is a part of the API provided by the `esp_driver_tsens` component. To declare that your component depends on `esp_driver_tsens`, add the following to your CMakeLists.txt:

  ```
  REQUIRES esp_driver_tsens
  ```

  or

  ```
  PRIV_REQUIRES esp_driver_tsens
  ```

Functions

esp_err_t **temperature_sensor_install** (const *temperature_sensor_config_t* *tsens_config, *temperature_sensor_handle_t* *ret_tsens)

Install temperature sensor driver.

Parameters
- **tsens_config** -- Pointer to config structure.
- **ret_tsens** -- Return the pointer of temperature sensor handle.

Returns
- ESP_OK if succeed

esp_err_t **temperature_sensor_uninstall** (*temperature_sensor_handle_t* tsens)

Uninstall the temperature sensor driver.

Parameters **tsens** -- The handle created by `temperature_sensor_install()`.

Returns
- ESP_OK if succeed.

esp_err_t **temperature_sensor_enable** (*temperature_sensor_handle_t* tsens)

Enable the temperature sensor.

Parameters **tsens** -- The handle created by `temperature_sensor_install()`.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_STATE if temperature sensor is enabled already.

esp_err_t **temperature_sensor_disable** (*temperature_sensor_handle_t* tsens)

Disable temperature sensor.

Parameters **tsens** -- The handle created by `temperature_sensor_install()`.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_STATE if temperature sensor is not enabled yet.

esp_err_t **temperature_sensor_get_celsius** (*temperature_sensor_handle_t* tsens, float *out_celsius)

Read temperature sensor data that is converted to degrees Celsius.

Note: Should not be called from interrupt.

Parameters
- **tsens** -- The handle created by `temperature_sensor_install()`.
- **out_celsius** -- The measure output value.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG invalid arguments
- ESP_ERR_INVALID_STATE Temperature sensor is not enabled yet.
- ESP_FAIL Parse the sensor data into ambient temperature failed (e.g. out of the range).

Structures

struct **temperature_sensor_config_t**

Configuration of measurement range for the temperature sensor.

Note: If you see the log `the boundary you gave cannot meet the range of internal temperature sensor`. You may need to refer to predefined range listed doc `api-reference/peripherals/Temperature sensor`.

Public Members

int **range_min**

the minimum value of the temperature you want to test

int **range_max**

the maximum value of the temperature you want to test

temperature_sensor_clk_src_t **clk_src**

the clock source of the temperature sensor.

Macros

TEMPERATURE_SENSOR_CONFIG_DEFAULT(min, max)

temperature_sensor_config_t default constructor

Type Definitions

typedef struct temperature_sensor_obj_t ***temperature_sensor_handle_t**

Type of temperature sensor driver handle.

Header File

- components/hal/include/hal/temperature_sensor_types.h
- This header file can be included with:

```
#include "hal/temperature_sensor_types.h"
```

Type Definitions

typedef *soc_periph_temperature_sensor_clk_src_t* **temperature_sensor_clk_src_t**

temperature sensor clock source

Enumerations

enum **temperature_sensor_etm_event_type_t**

temperature sensor event types enum

Values:

enumerator **TEMPERATURE_SENSOR_EVENT_OVER_LIMIT**

Temperature sensor over limit event

enumerator **TEMPERATURE_SENSOR_EVENT_MAX**

Maximum number of temperature sensor events

enum **temperature_sensor_etm_task_type_t**

temperature sensor task types enum

Values:

enumerator **TEMPERATURE_SENSOR_TASK_START**

Temperature sensor start task

enumerator **TEMPERATURE_SENSOR_TASK_STOP**

Temperature sensor stop task

enumerator **TEMPERATURE_SENSOR_TASK_MAX**

Maximum number of temperature sensor tasks

2.6.26 Touch Sensor

Introduction

A touch sensor system is built on a substrate which carries electrodes and relevant connections under a protective flat surface. When the surface is touched, the capacitance variation is used to evaluate if the touch was valid.

The sensing pads can be arranged in different combinations (e.g., matrix, slider), so that a larger area or more points can be detected. The touch pad sensing process is under the control of a hardware-implemented finite-state machine (FSM) which is initiated by software or a dedicated hardware timer.

For design, operation, and control registers of a touch sensor, see **ESP32-S3 Technical Reference Manual > On-Chip Sensors and Analog Signal Processing** [PDF].

In-depth design details of touch sensors and firmware development guidelines for ESP32-S3 are available in Touch Sensor Application Note.

Functionality Overview

Description of API is broken down into groups of functions to provide a quick overview of the following features:

- Initialization of touch pad driver
- Configuration of touch pad GPIO pins
- Taking measurements
- Adjusting parameters of measurements
- Filtering measurements
- Touch detection methods
- Setting up interrupts to report touch detection
- Waking up from Sleep mode on interrupt

For detailed description of a particular function, please go to Section *API Reference*. Practical implementation of this API is covered in Section *Application Examples*.

Initialization Before using a touch pad, you need to initialize the touch pad driver by calling the function `touch_pad_init()`. This function sets several `..._DEFAULT` driver parameters listed in *API Reference* under **Macros**. It also removes the information about which pads have been touched before, if any, and disables interrupts.

If the driver is not required anymore, deinitialize it by calling `touch_pad_deinit()`.

Configuration Enabling the touch sensor functionality for a particular GPIO is done with `touch_pad_config()`. The following 14 capacitive touch pads are supported for ESP32-S3.

Touch Pad	GPIO Pin
T0	Internal channel, not connect to a GPIO
T1	GPIO1
T2	GPIO2
T3	GPIO3
T4	GPIO4
T5	GPIO5
T6	GPIO6
T7	GPIO7
T8	GPIO8
T9	GPIO9
T10	GPIO10
T11	GPIO11
T12	GPIO12
T13	GPIO13
T14	GPIO14

Use the function `touch_pad_set_fsm_mode()` to select if touch pad measurement (operated by FSM) should be started automatically by a hardware timer, or by software. If software mode is selected, use `touch_pad_sw_start()` to start the FSM.

Touch State Measurements The following function come in handy to read raw measurements from the sensor:

- `touch_pad_read_raw_data()`

It can also be used, for example, to evaluate a particular touch pad design by checking the range of sensor readings when a pad is touched or released. This information can be then used to establish a touch threshold.

For the demonstration of how to read the touch pad data, check the application example peripherals/touch_sensor/touch_sensor_v2/touch_pad_read.

Method of Measurements The touch sensor records the period of time (i.e., the number of clock cycles) over a fixed charge/discharge cycles (specified by `touch_pad_set_charge_discharge_times()`). The count result is the raw data that read from `touch_pad_read_raw_data()`. After finishing one measurement, the touch sensor sleeps until the next measurement start, this interval between two measurements can be set by `touch_pad_set_measurement_interval()`.

Note: If the specified charge and discharge cycles for measurement is too small, the result may be inaccurate, but increasing charge and discharge cycles will increase the power consumption as well. Additionally, the response of the touch sensor will slow down if the total time of the interval and measurement is too long.

Optimization of Measurements A touch sensor has several configurable parameters to match the characteristics of a particular touch pad design. For instance, to sense smaller capacity changes, it is possible to narrow down the reference voltage range within which the touch pads are charged/discharged. The high and low reference voltages are set using the function `touch_pad_set_voltage()`.

Besides the ability to discern smaller capacity changes, a positive side effect is reduction of power consumption for low power applications. A likely negative effect is an increase in measurement noise. If the dynamic range of obtained readings is still satisfactory, then further reduction of power consumption might be done by reducing the measurement time with `touch_pad_set_charge_discharge_times()`.

The following list summarizes available measurement parameters and corresponding 'set' functions:

- Touch pad charge / discharge parameters:
  - voltage range: `touch_pad_set_voltage()`
  - speed (slope): `touch_pad_set_cnt_mode()`

- Charge and discharge times of one measurement: `touch_pad_set_charge_discharge_times()`

Relationship between the voltage range (high/low reference voltages), speed (slope), and measurement time is shown in the figure below.

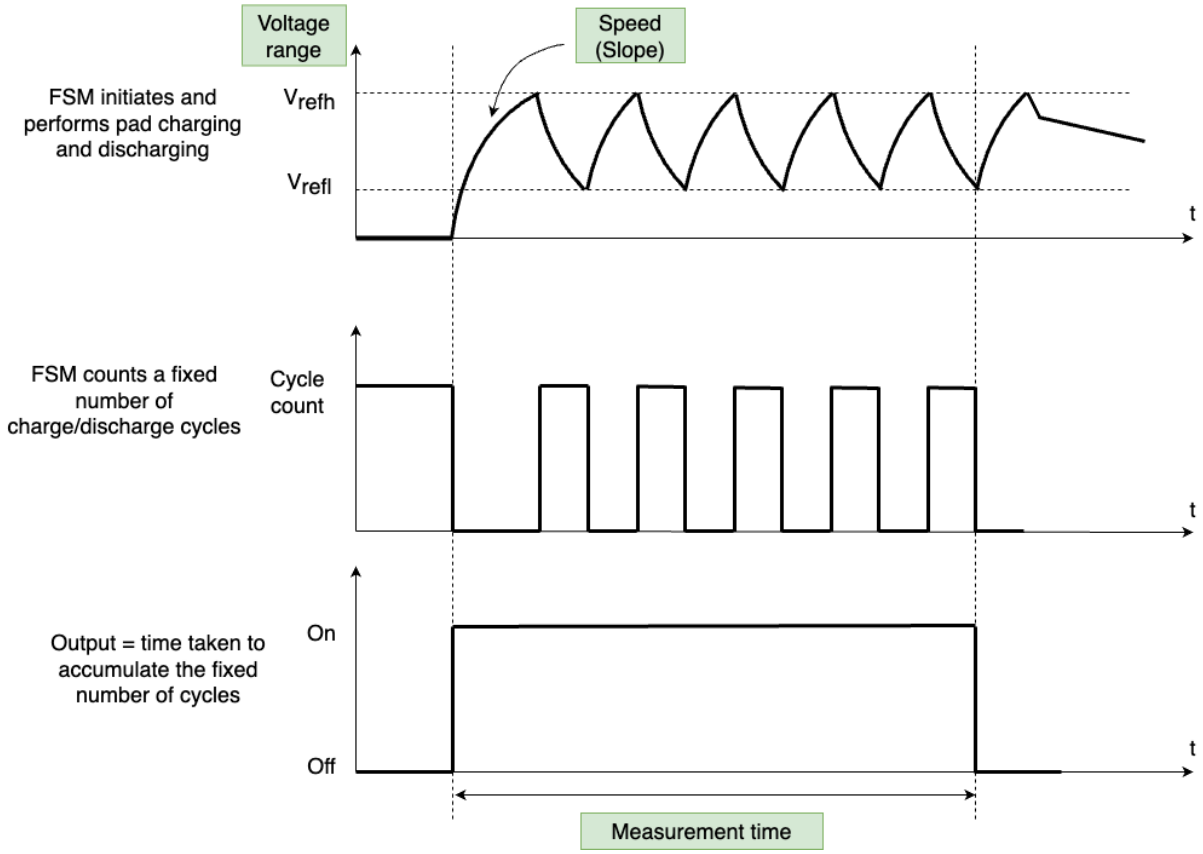


Fig. 26: Touch pad - relationship between measurement parameters

The last chart **Output** represents the touch sensor reading, i.e., the time taken to accumulate the fixed number of cycles.

All functions are provided in pairs to **set** a specific parameter and to **get** the current parameter's value, e.g., `touch_pad_set_voltage()` and `touch_pad_get_voltage()`.

Filtering of Measurements If measurements are noisy, you can filter them with provided API functions. The ESP32-S3's touch functionality provide two sets of APIs for doing this.

There is an internal touch channel that is not connected to any external GPIO. The measurements from this denoise pad can be used to filters out interference introduced on all channels, such as noise introduced by the power supply and external EMI.

The denoise parameters are set with the function `touch_pad_denoise_set_config()` and started by with `touch_pad_denoise_enable()`

There is also a configurable hardware implemented IIR-filter (infinite impulse response). This IIR-filter is configured with the function `touch_pad_filter_set_config()` and enabled by calling `touch_pad_filter_enable()`

Touch Detection Touch detection is implemented in ESP32's hardware based on the user-configured threshold and raw measurements executed by FSM. Use the functions `touch_pad_get_status()` to check which pads have been touched and `touch_pad_clear_status()` to clear the touch status information.

Hardware touch detection can also be wired to interrupts. This is described in the next section.

If measurements are noisy and capacity changes are small, hardware touch detection might be unreliable. To resolve this issue, instead of using hardware detection/provided interrupts, implement measurement filtering and perform touch detection in your own application. For sample implementation of both methods of touch detection, see peripherals/touch_sensor/touch_sensor_v2/touch_pad_interrupt.

Touch Triggered Interrupts Before enabling an interrupt on a touch detection, you should establish a touch detection threshold. Use the functions described in *Touch State Measurements* to read and display sensor measurements when a pad is touched and released. Apply a filter if measurements are noisy and relative capacity changes are small. Depending on your application and environment conditions, test the influence of temperature and power supply voltage changes on measured values.

Once a detection threshold is established, it can be set during initialization with `touch_pad_config()` or at the runtime with `touch_pad_set_thresh()`.

Finally, configure and manage interrupt calls using the following functions:

- `touch_pad_isr_register()` / `touch_pad_isr_deregister()`
- `touch_pad_intr_enable()` / `touch_pad_intr_disable()`

When interrupts are operational, you can obtain the information from which particular pad an interrupt came by invoking `touch_pad_get_status()` and clear the pad status with `touch_pad_clear_status()`.

Application Examples

- Touch sensor read example: peripherals/touch_sensor/touch_sensor_v2/touch_pad_read.
- Touch sensor interrupt example: peripherals/touch_sensor/touch_sensor_v2/touch_pad_interrupt.

API Reference

Header File

- components/driver/touch_sensor/esp32s3/include/driver/touch_sensor.h
- This header file can be included with:

```
#include "driver/touch_sensor.h"
```

- This header file is a part of the API provided by the `driver` component. To declare that your component depends on `driver`, add the following to your CMakeLists.txt:

```
REQUIRES driver
```

or

```
PRIV_REQUIRES driver
```

Functions

esp_err_t **touch_pad_fsm_start** (void)

Set touch sensor FSM start.

Note: Start FSM after the touch sensor FSM mode is set.

Note: Call this function will reset benchmark of all touch channels.

Returns

- ESP_OK on success

esp_err_t **touch_pad_fsm_stop**(void)

Stop touch sensor FSM.

Returns

- ESP_OK on success

esp_err_t **touch_pad_sw_start**(void)

Trigger a touch sensor measurement, only support in SW mode of FSM.

Returns

- ESP_OK on success

esp_err_t **touch_pad_set_charge_discharge_times**(uint16_t charge_discharge_times)

Set charge and discharge times of each measurement.

Note: This function will specify the charge and discharge times in each measurement period The clock is sourced from SOC_MOD_CLK_RTC_FAST, and its default frequency is SOC_CLK_RC_FAST_FREQ_APPROX The touch sensor will record the total clock cycles of all the charge and discharge cycles as the final result (raw value)

Note: If the charge and discharge times is too small, it may lead to inaccurate results.

Parameters **charge_discharge_times** -- Charge and discharge times, range: 0 ~ 0xffff. No exact typical value can be recommended because the capacity is influenced by the hardware design and how finger touches, but suggest adjusting this value to make the measurement time around 1 ms.

Returns

- ESP_OK Set charge and discharge times success

esp_err_t **touch_pad_get_charge_discharge_times**(uint16_t *charge_discharge_times)

Get charge and discharge times of each measurement.

Parameters **charge_discharge_times** -- Charge and discharge times

Returns

- ESP_OK Get charge_discharge_times success
- ESP_ERR_INVALID_ARG The input parameter is NULL

esp_err_t **touch_pad_set_measurement_interval**(uint16_t interval_cycle)

Set the interval between two measurements.

Note: The touch sensor will sleep between two measurements This function is to set the interval cycle And the interval is clocked from SOC_MOD_CLK_RTC_SLOW, its default frequency is SOC_CLK_RC_SLOW_FREQ_APPROX

Parameters **interval_cycle** -- The interval between two measurements sleep_time = interval_cycle / SOC_CLK_RC_SLOW_FREQ_APPROX. The approximate frequency value of RTC_SLOW_CLK can be obtained using rtc_clk_slow_freq_get_hz function.

Returns

- ESP_OK Set interval cycle success

esp_err_t **touch_pad_get_measurement_interval**(uint16_t *interval_cycle)

Get the interval between two measurements.

Parameters **interval_cycle** -- The interval between two measurements

Returns

- ESP_OK Get interval cycle success

- ESP_ERR_INVALID_ARG The input parameter is NULL

esp_err_t **touch_pad_set_meas_time**(uint16_t sleep_cycle, uint16_t meas_times)

Set touch sensor times of charge and discharge and sleep time. Excessive total time will slow down the touch response. Too small measurement time will not be sampled enough, resulting in inaccurate measurements.

Note: The touch sensor will measure time of a fixed number of charge/discharge cycles (specified as the second parameter). That means the time (raw value) will increase as the capacity of the touch pad is increasing. The time (raw value) here is the number of clock cycles which is sourced from SOC_MOD_CLK_RTC_FAST and at (SOC_CLK_RC_FAST_FREQ_APPROX) Hz as default

---

Note: The greater the duty cycle of the measurement time, the more system power is consumed.

Parameters

- **sleep_cycle** -- The touch sensor will sleep after each measurement. sleep_cycle decide the interval between each measurement. t_sleep = sleep_cycle / SOC_CLK_RC_SLOW_FREQ_APPROX. The approximate frequency value of RTC_SLOW_CLK can be obtained using rtc_clk_slow_freq_get_hz function.
- **meas_times** -- The times of charge and discharge in each measurement of touch channels. Range: 0 ~ 0xffff. Recommended typical value: Modify this value to make the measurement time around 1 ms.

Returns

- ESP_OK on success

esp_err_t **touch_pad_get_meas_time**(uint16_t *sleep_cycle, uint16_t *meas_times)

Get touch sensor times of charge and discharge and sleep time.

Parameters

- **sleep_cycle** -- Pointer to accept sleep cycle number
- **meas_times** -- Pointer to accept measurement times count.

Returns

- ESP_OK on success

esp_err_t **touch_pad_set_idle_channel_connect**(*touch_pad_conn_type_t* type)

Set the connection type of touch channels in idle status. When a channel is in measurement mode, other initialized channels are in idle mode. The touch channel is generally adjacent to the trace, so the connection state of the idle channel affects the stability and sensitivity of the test channel. The `CONN_HIGHZ`(high resistance) setting increases the sensitivity of touch channels. The `CONN_GND`(grounding) setting increases the stability of touch channels.

Parameters **type** -- Select idle channel connect to high resistance state or ground.

Returns

- ESP_OK on success

esp_err_t **touch_pad_get_idle_channel_connect**(*touch_pad_conn_type_t* *type)

Get the connection type of touch channels in idle status. When a channel is in measurement mode, other initialized channels are in idle mode. The touch channel is generally adjacent to the trace, so the connection state of the idle channel affects the stability and sensitivity of the test channel. The `CONN_HIGHZ`(high resistance) setting increases the sensitivity of touch channels. The `CONN_GND`(grounding) setting increases the stability of touch channels.

Parameters **type** -- Pointer to connection type.

Returns

- ESP_OK on success

esp_err_t **touch_pad_set_thresh**(*touch_pad_t* touch_num, uint32_t threshold)

Set the trigger threshold of touch sensor. The threshold determines the sensitivity of the touch sensor. The threshold is the original value of the trigger state minus the benchmark value.

Note: If set "TOUCH_PAD_THRESHOLD_MAX", the touch is never be triggered.

Parameters
- **touch_num** -- touch pad index
- **threshold** -- threshold of touch sensor. Should be less than the max change value of touch.

Returns
- ESP_OK on success

esp_err_t **touch_pad_get_thresh**(*touch_pad_t* touch_num, uint32_t *threshold)

Get touch sensor trigger threshold.

Parameters
- **touch_num** -- touch pad index
- **threshold** -- pointer to accept threshold

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_set_channel_mask**(uint16_t enable_mask)

Register touch channel into touch sensor scan group. The working mode of the touch sensor is cyclically scanned. This function will set the scan bits according to the given bitmask.

Note: If set this mask, the FSM timer should be stop firsty.

Note: The touch sensor that in scan map, should be deinit GPIO function firstly by `touch_pad_io_init`.

Parameters **enable_mask** -- bitmask of touch sensor scan group. e.g. TOUCH_PAD_NUM14 -> BIT(14)

Returns
- ESP_OK on success

esp_err_t **touch_pad_get_channel_mask**(uint16_t *enable_mask)

Get the touch sensor scan group bit mask.

Parameters **enable_mask** -- Pointer to bitmask of touch sensor scan group. e.g. TOUCH_PAD_NUM14 -> BIT(14)

Returns
- ESP_OK on success

esp_err_t **touch_pad_clear_channel_mask**(uint16_t enable_mask)

Clear touch channel from touch sensor scan group. The working mode of the touch sensor is cyclically scanned. This function will clear the scan bits according to the given bitmask.

Note: If clear all mask, the FSM timer should be stop firsty.

Parameters **enable_mask** -- bitmask of touch sensor scan group. e.g. TOUCH_PAD_NUM14 -> BIT(14)

Returns
- ESP_OK on success

esp_err_t **touch_pad_config**(*touch_pad_t* touch_num)

Configure parameter for each touch channel.

Note: Touch num 0 is denoise channel, please use `touch_pad_denoise_enable` to set denoise function

Parameters **touch_num** -- touch pad index

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG if argument wrong
- ESP_FAIL if touch pad not initialized

esp_err_t **touch_pad_reset**(void)

Reset the FSM of touch module.

Note: Call this function after `touch_pad_fsm_stop`.

Returns

- ESP_OK Success

touch_pad_t **touch_pad_get_current_meas_channel**(void)

Get the current measure channel.

Note: Should be called when touch sensor measurement is in cyclic scan mode.

Returns

- touch channel number

uint32_t **touch_pad_read_intr_status_mask**(void)

Get the touch sensor interrupt status mask.

Returns

- touch interrupt bit

esp_err_t **touch_pad_intr_enable**(*touch_pad_intr_mask_t* int_mask)

Enable touch sensor interrupt by bitmask.

Note: This API can be called in ISR handler.

Parameters **int_mask** -- Pad mask to enable interrupts

Returns

- ESP_OK on success

esp_err_t **touch_pad_intr_disable**(*touch_pad_intr_mask_t* int_mask)

Disable touch sensor interrupt by bitmask.

Note: This API can be called in ISR handler.

Parameters **int_mask** -- Pad mask to disable interrupts

Returns

- ESP_OK on success

esp_err_t **touch_pad_intr_clear** (*touch_pad_intr_mask_t* int_mask)

Clear touch sensor interrupt by bitmask.

Parameters **int_mask** -- Pad mask to clear interrupts

Returns

- ESP_OK on success

esp_err_t **touch_pad_isr_register** (*intr_handler_t* fn, void *arg, *touch_pad_intr_mask_t* intr_mask)

Register touch-pad ISR. The handler will be attached to the same CPU core that this function is running on.

Parameters

- **fn** -- Pointer to ISR handler
- **arg** -- Parameter for ISR
- **intr_mask** -- Enable touch sensor interrupt handler by bitmask.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Arguments error
- ESP_ERR_NO_MEM No memory

esp_err_t **touch_pad_timeout_set** (bool enable, uint32_t threshold)

Enable/disable the timeout check and set timeout threshold for all touch sensor channels measurements. If enable: When the touch reading of a touch channel exceeds the measurement threshold, a timeout interrupt will be generated. If disable: the FSM does not check if the channel under measurement times out.

Note: The threshold compared with touch readings.

Note: In order to avoid abnormal short circuit of some touch channels. This function should be turned on. Ensure the normal operation of other touch channels.

Parameters

- **enable** -- true(default): Enable the timeout check; false: Disable the timeout check.
- **threshold** -- For all channels, the maximum value that will not be exceeded during normal operation.

Returns

- ESP_OK Success

esp_err_t **touch_pad_timeout_resume** (void)

Call this interface after timeout to make the touch channel resume normal work. Point on the next channel to measure. If this API is not called, the touch FSM will stop the measurement after timeout interrupt.

Note: Call this API after finishes the exception handling by user.

Returns

- ESP_OK Success

esp_err_t **touch_pad_read_raw_data** (*touch_pad_t* touch_num, uint32_t *raw_data)

get raw data of touch sensor.

Note: After the initialization is complete, the "raw_data" is max value. You need to wait for a measurement cycle before you can read the correct touch value.

Parameters

- **touch_num** -- touch pad index

- **raw_data** -- pointer to accept touch sensor value

Returns

- ESP_OK Success
- ESP_FAIL Touch channel 0 haven't this parameter.

esp_err_t **touch_pad_read_benchmark**(*touch_pad_t* touch_num, uint32_t *benchmark)

get benchmark of touch sensor.

Note: After initialization, the benchmark value is the maximum during the first measurement period.

Parameters

- **touch_num** -- touch pad index
- **benchmark** -- pointer to accept touch sensor benchmark value

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Touch channel 0 haven't this parameter.

esp_err_t **touch_pad_filter_read_smooth**(*touch_pad_t* touch_num, uint32_t *smooth)

Get smoothed data that obtained by filtering the raw data.

Parameters

- **touch_num** -- touch pad index
- **smooth** -- pointer to smoothed data

esp_err_t **touch_pad_reset_benchmark**(*touch_pad_t* touch_num)

Force reset benchmark to raw data of touch sensor.

Parameters **touch_num** -- touch pad index

- TOUCH_PAD_MAX Reset basaline of all channels

Returns

- ESP_OK Success

esp_err_t **touch_pad_filter_set_config**(const *touch_filter_config_t* *filter_info)

set parameter of touch sensor filter and detection algorithm. For more details on the detection algorithm, please refer to the application documentation.

Parameters **filter_info** -- select filter type and threshold of detection algorithm

Returns

- ESP_OK Success

esp_err_t **touch_pad_filter_get_config**(*touch_filter_config_t* *filter_info)

get parameter of touch sensor filter and detection algorithm. For more details on the detection algorithm, please refer to the application documentation.

Parameters **filter_info** -- select filter type and threshold of detection algorithm

Returns

- ESP_OK Success

esp_err_t **touch_pad_filter_enable**(void)

enable touch sensor filter for detection algorithm. For more details on the detection algorithm, please refer to the application documentation.

Returns

- ESP_OK Success

esp_err_t **touch_pad_filter_disable**(void)

disable touch sensor filter for detection algorithm. For more details on the detection algorithm, please refer to the application documentation.

Returns

- ESP_OK Success

esp_err_t **touch_pad_denoise_set_config**(const *touch_pad_denoise_t* *denoise)

set parameter of denoise pad (TOUCH_PAD_NUM0). T0 is an internal channel that does not have a corresponding external GPIO. T0 will work simultaneously with the measured channel Tn. Finally, the actual measured value of Tn is the value after subtracting lower bits of T0. The noise reduction function filters out interference introduced simultaneously on all channels, such as noise introduced by power supplies and external EMI.

Parameters **denoise** -- parameter of denoise

Returns

- ESP_OK Success

esp_err_t **touch_pad_denoise_get_config**(*touch_pad_denoise_t* *denoise)

get parameter of denoise pad (TOUCH_PAD_NUM0).

Parameters **denoise** -- Pointer to parameter of denoise

Returns

- ESP_OK Success

esp_err_t **touch_pad_denoise_enable**(void)

enable denoise function. T0 is an internal channel that does not have a corresponding external GPIO. T0 will work simultaneously with the measured channel Tn. Finally, the actual measured value of Tn is the value after subtracting lower bits of T0. The noise reduction function filters out interference introduced simultaneously on all channels, such as noise introduced by power supplies and external EMI.

Returns

- ESP_OK Success

esp_err_t **touch_pad_denoise_disable**(void)

disable denoise function.

Returns

- ESP_OK Success

esp_err_t **touch_pad_denoise_read_data**(uint32_t *data)

Get denoise measure value (TOUCH_PAD_NUM0).

Parameters **data** -- Pointer to receive denoise value

Returns

- ESP_OK Success

esp_err_t **touch_pad_waterproof_set_config**(const *touch_pad_waterproof_t* *waterproof)

set parameter of waterproof function.

```
   The waterproof function includes a shielded channel (TOUCH_PAD_NUM14) and a␣
↪guard channel.
   Guard pad is used to detect the large area of water covering the touch␣
↪panel.
   Shield pad is used to shield the influence of water droplets covering the␣
↪touch panel.
   It is generally designed as a grid and is placed around the touch buttons.
```

Parameters **waterproof** -- parameter of waterproof

Returns

- ESP_OK Success

esp_err_t **touch_pad_waterproof_get_config**(*touch_pad_waterproof_t* *waterproof)

get parameter of waterproof function.

Parameters **waterproof** -- parameter of waterproof

Returns

- ESP_OK Success

esp_err_t **touch_pad_waterproof_enable** (void)

Enable parameter of waterproof function. Should be called after function `touch_pad_waterproof_set_config`.

Returns
- ESP_OK Success

esp_err_t **touch_pad_waterproof_disable** (void)

Disable parameter of waterproof function.

Returns
- ESP_OK Success

esp_err_t **touch_pad_proximity_enable** (*touch_pad_t* touch_num, bool enabled)

Enable/disable proximity function of touch channels. The proximity sensor measurement is the accumulation of touch channel measurements.

Note: Supports up to three touch channels configured as proximity sensors.

Parameters
- **touch_num** -- touch pad index
- **enabled** -- true: enable the proximity function; false: disable the proximity function

Returns
- ESP_OK: Configured correctly.
- ESP_ERR_INVALID_ARG: Touch channel number error.
- ESP_ERR_NOT_SUPPORTED: Don't support configured.

esp_err_t **touch_pad_proximity_set_count** (*touch_pad_t* touch_num, uint32_t count)

Set measure count of proximity channel. The proximity sensor measurement is the accumulation of touch channel measurements.

Note: All proximity channels use the same `count` value. So please pass the parameter `TOUCH_PAD_MAX`.

Parameters
- **touch_num** -- Touch pad index. In this version, pass the parameter `TOUCH_PAD_MAX`.
- **count** -- The cumulative times of measurements for proximity pad. Range: 0 ~ 255.

Returns
- ESP_OK: Configured correctly.
- ESP_ERR_INVALID_ARG: Touch channel number error.

esp_err_t **touch_pad_proximity_get_count** (*touch_pad_t* touch_num, uint32_t *count)

Get measure count of proximity channel. The proximity sensor measurement is the accumulation of touch channel measurements.

Note: All proximity channels use the same `count` value. So please pass the parameter `TOUCH_PAD_MAX`.

Parameters
- **touch_num** -- Touch pad index. In this version, pass the parameter `TOUCH_PAD_MAX`.
- **count** -- The cumulative times of measurements for proximity pad. Range: 0 ~ 255.

Returns
- ESP_OK: Configured correctly.
- ESP_ERR_INVALID_ARG: Touch channel number error.

esp_err_t **touch_pad_proximity_get_data** (*touch_pad_t* touch_num, uint32_t *measure_out)

Get the accumulated measurement of the proximity sensor. The proximity sensor measurement is the accumulation of touch channel measurements.

Parameters
- **touch_num** -- touch pad index
- **measure_out** -- If the accumulation process does not end, the `measure_out` is the process value.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Touch num is not proximity

esp_err_t **touch_pad_sleep_channel_get_info** (*touch_pad_sleep_channel_t* *slp_config)

Get parameter of touch sensor sleep channel. The touch sensor can works in sleep mode to wake up sleep.

Note: After the sleep channel is configured, Please use special functions for sleep channel. e.g. The user should uses `touch_pad_sleep_channel_read_data` instead of `touch_pad_read_raw_data` to obtain the sleep channel reading.

Parameters **slp_config** -- touch sleep pad config.

Returns
- ESP_OK Success

esp_err_t **touch_pad_sleep_channel_enable** (*touch_pad_t* pad_num, bool enable)

Enable/Disable sleep channel function for touch sensor. The touch sensor can works in sleep mode to wake up sleep.

Note: ESP32S2 only support one sleep channel.

Note: After the sleep channel is configured, Please use special functions for sleep channel. e.g. The user should uses `touch_pad_sleep_channel_read_data` instead of `touch_pad_read_raw_data` to obtain the sleep channel reading.

Parameters
- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **enable** -- true: enable sleep pad for touch sensor; false: disable sleep pad for touch sensor;

Returns
- ESP_OK Success

esp_err_t **touch_pad_sleep_channel_enable_proximity** (*touch_pad_t* pad_num, bool enable)

Enable/Disable proximity function for sleep channel. The touch sensor can works in sleep mode to wake up sleep.

Note: ESP32S2 only support one sleep channel.

Parameters
- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **enable** -- true: enable proximity for sleep channel; false: disable proximity for sleep channel;

Returns

- ESP_OK Success

esp_err_t **touch_pad_sleep_set_threshold**(*touch_pad_t* pad_num, uint32_t touch_thres)

Set the trigger threshold of touch sensor in deep sleep. The threshold determines the sensitivity of the touch sensor.

Note: In general, the touch threshold during sleep can use the threshold parameter parameters before sleep.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **touch_thres** -- touch sleep pad threshold

Returns

- ESP_OK Success

esp_err_t **touch_pad_sleep_get_threshold**(*touch_pad_t* pad_num, uint32_t *touch_thres)

Get the trigger threshold of touch sensor in deep sleep. The threshold determines the sensitivity of the touch sensor.

Note: In general, the touch threshold during sleep can use the threshold parameter parameters before sleep.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **touch_thres** -- touch sleep pad threshold

Returns

- ESP_OK Success

esp_err_t **touch_pad_sleep_channel_read_benchmark**(*touch_pad_t* pad_num, uint32_t *benchmark)

Read benchmark of touch sensor sleep channel.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **benchmark** -- pointer to accept touch sensor benchmark value

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG parameter is NULL

esp_err_t **touch_pad_sleep_channel_read_smooth**(*touch_pad_t* pad_num, uint32_t *smooth_data)

Read smoothed data of touch sensor sleep channel. Smoothed data is filtered from the raw data.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **smooth_data** -- pointer to accept touch sensor smoothed data

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG parameter is NULL

esp_err_t **touch_pad_sleep_channel_read_data**(*touch_pad_t* pad_num, uint32_t *raw_data)

Read raw data of touch sensor sleep channel.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **raw_data** -- pointer to accept touch sensor raw data

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG parameter is NULL

esp_err_t **touch_pad_sleep_channel_reset_benchmark**(void)

Reset benchmark of touch sensor sleep channel.

Returns

- ESP_OK Success

esp_err_t **touch_pad_sleep_channel_read_proximity_cnt**(*touch_pad_t* pad_num, uint32_t *proximity_cnt)

Read proximity count of touch sensor sleep channel.

Parameters

- **pad_num** -- Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode.
- **proximity_cnt** -- pointer to accept touch sensor proximity count value

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG parameter is NULL

esp_err_t **touch_pad_sleep_channel_set_work_time**(uint16_t sleep_cycle, uint16_t meas_times)

Change the operating frequency of touch pad in deep sleep state. Reducing the operating frequency can effectively reduce power consumption. If this function is not called, the working frequency of touch in the deep sleep state is the same as that in the wake-up state.

Parameters

- **sleep_cycle** -- The touch sensor will sleep after each measurement. sleep_cycle decide the interval between each measurement. t_sleep = sleep_cycle / (RTC_SLOW_CLK frequency). The approximate frequency value of RTC_SLOW_CLK can be obtained using rtc_clk_slow_freq_get_hz function.
- **meas_times** -- The times of charge and discharge in each measure process of touch channels. The timer frequency is 8Mhz. Range: 0 ~ 0xffff. Recommended typical value: Modify this value to make the measurement time around 1ms.

Returns

- ESP_OK Success

Header File

- components/driver/touch_sensor/include/driver/touch_sensor_common.h
- This header file can be included with:

```
#include "driver/touch_sensor_common.h"
```

- This header file is a part of the API provided by the `driver` component. To declare that your component depends on `driver`, add the following to your CMakeLists.txt:

```
REQUIRES driver
```

or

```
PRIV_REQUIRES driver
```

Functions

esp_err_t **touch_pad_init**(void)

Initialize touch module.

Note: If default parameter don't match the usage scenario, it can be changed after this function.

Returns
- ESP_OK Success
- ESP_ERR_NO_MEM Touch pad init error
- ESP_ERR_NOT_SUPPORTED Touch pad is providing current to external XTAL

esp_err_t **touch_pad_deinit** (void)

Un-install touch pad driver.

Note: After this function is called, other touch functions are prohibited from being called.

Returns
- ESP_OK Success
- ESP_FAIL Touch pad driver not initialized

esp_err_t **touch_pad_io_init** (*touch_pad_t* touch_num)

Initialize touch pad GPIO.

Parameters **touch_num** -- touch pad index

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_set_voltage** (*touch_high_volt_t* refh, *touch_low_volt_t* refl, *touch_volt_atten_t* atten)

Set touch sensor high voltage threshold of chanrge. The touch sensor measures the channel capacitance value by charging and discharging the channel. So the high threshold should be less than the supply voltage.

Parameters
- **refh** -- the value of DREFH
- **refl** -- the value of DREFL
- **atten** -- the attenuation on DREFH

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_get_voltage** (*touch_high_volt_t* *refh, *touch_low_volt_t* *refl, *touch_volt_atten_t* *atten)

Get touch sensor reference voltage,.

Parameters
- **refh** -- pointer to accept DREFH value
- **refl** -- pointer to accept DREFL value
- **atten** -- pointer to accept the attenuation on DREFH

Returns
- ESP_OK on success

esp_err_t **touch_pad_set_cnt_mode** (*touch_pad_t* touch_num, *touch_cnt_slope_t* slope, *touch_tie_opt_t* opt)

Set touch sensor charge/discharge speed for each pad. If the slope is 0, the counter would always be zero. If the slope is 1, the charging and discharging would be slow, accordingly. If the slope is set 7, which is the maximum value, the charging and discharging would be fast.

Note: The higher the charge and discharge current, the greater the immunity of the touch channel, but it will increase the system power consumption.

Parameters
- **touch_num** -- touch pad index
- **slope** -- touch pad charge/discharge speed
- **opt** -- the initial voltage

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_get_cnt_mode**(*touch_pad_t* touch_num, *touch_cnt_slope_t* *slope, *touch_tie_opt_t* *opt)

Get touch sensor charge/discharge speed for each pad.

Parameters
- **touch_num** -- touch pad index
- **slope** -- pointer to accept touch pad charge/discharge slope
- **opt** -- pointer to accept the initial voltage

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_isr_deregister**(void (*fn)(void*), void *arg)

Deregister the handler previously registered using touch_pad_isr_handler_register.

Parameters
- **fn** -- handler function to call (as passed to touch_pad_isr_handler_register)
- **arg** -- argument of the handler (as passed to touch_pad_isr_handler_register)

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE if a handler matching both fn and arg isn't registered

esp_err_t **touch_pad_get_wakeup_status**(*touch_pad_t* *pad_num)

Get the touch pad which caused wakeup from deep sleep.

Parameters **pad_num** -- pointer to touch pad which caused wakeup

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG parameter is NULL

esp_err_t **touch_pad_set_fsm_mode**(*touch_fsm_mode_t* mode)

Set touch sensor FSM mode, the test action can be triggered by the timer, as well as by the software.

Parameters **mode** -- FSM mode

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if argument is wrong

esp_err_t **touch_pad_get_fsm_mode**(*touch_fsm_mode_t* *mode)

Get touch sensor FSM mode.

Parameters **mode** -- pointer to accept FSM mode

Returns
- ESP_OK on success

esp_err_t **touch_pad_clear_status**(void)

To clear the touch sensor channel active status.

Note: The FSM automatically updates the touch sensor status. It is generally not necessary to call this API to clear the status.

Returns

- ESP_OK on success

uint32_t **touch_pad_get_status**(void)

Get the touch sensor channel active status mask. The bit position represents the channel number. The 0/1 status of the bit represents the trigger status.

Returns

- The touch sensor status. e.g. Touch1 trigger status is `status_mask & (BIT1)`.

bool **touch_pad_meas_is_done**(void)

Check touch sensor measurement status.

Returns

- True measurement is under way
- False measurement done

GPIO Lookup Macros Some useful macros can be used to specified the GPIO number of a touch pad channel, or vice versa. e.g.,

1. `TOUCH_PAD_NUM5_GPIO_NUM` is the GPIO number of channel 5 (12);
2. `TOUCH_PAD_GPIO4_CHANNEL` is the channel number of GPIO 4 (channel 0).

Header File

- components/soc/esp32s3/include/soc/touch_sensor_channel.h
- This header file can be included with:

```
#include "soc/touch_sensor_channel.h"
```

Macros

TOUCH_PAD_GPIO1_CHANNEL

TOUCH_PAD_NUM1_GPIO_NUM

TOUCH_PAD_GPIO2_CHANNEL

TOUCH_PAD_NUM2_GPIO_NUM

TOUCH_PAD_GPIO3_CHANNEL

TOUCH_PAD_NUM3_GPIO_NUM

TOUCH_PAD_GPIO4_CHANNEL

TOUCH_PAD_NUM4_GPIO_NUM

TOUCH_PAD_GPIO5_CHANNEL

TOUCH_PAD_NUM5_GPIO_NUM

TOUCH_PAD_GPIO6_CHANNEL

TOUCH_PAD_NUM6_GPIO_NUM

TOUCH_PAD_GPIO7_CHANNEL

TOUCH_PAD_NUM7_GPIO_NUM

TOUCH_PAD_GPIO8_CHANNEL

TOUCH_PAD_NUM8_GPIO_NUM

TOUCH_PAD_GPIO9_CHANNEL

TOUCH_PAD_NUM9_GPIO_NUM

TOUCH_PAD_GPIO10_CHANNEL

TOUCH_PAD_NUM10_GPIO_NUM

TOUCH_PAD_GPIO11_CHANNEL

TOUCH_PAD_NUM11_GPIO_NUM

TOUCH_PAD_GPIO12_CHANNEL

TOUCH_PAD_NUM12_GPIO_NUM

TOUCH_PAD_GPIO13_CHANNEL

TOUCH_PAD_NUM13_GPIO_NUM

TOUCH_PAD_GPIO14_CHANNEL

TOUCH_PAD_NUM14_GPIO_NUM

Header File

- components/hal/include/hal/touch_sensor_types.h
- This header file can be included with:

```
#include "hal/touch_sensor_types.h"
```

Structures

struct **touch_pad_denoise**

Touch sensor denoise configuration

Public Members

touch_pad_denoise_grade_t **grade**

Select denoise range of denoise channel. Determined by measuring the noise amplitude of the denoise channel.

touch_pad_denoise_cap_t **cap_level**

Select internal reference capacitance of denoise channel. Ensure that the denoise readings are closest to the readings of the channel being measured. Use `touch_pad_denoise_read_data` to get the reading of denoise channel. The equivalent capacitance of the shielded channel can be calculated from the reading of denoise channel.

struct **touch_pad_waterproof**

Touch sensor waterproof configuration

Public Members

touch_pad_t **guard_ring_pad**

Waterproof. Select touch channel use for guard pad. Guard pad is used to detect the large area of water covering the touch panel.

touch_pad_shield_driver_t **shield_driver**

Waterproof. Shield channel drive capability configuration. Shield pad is used to shield the influence of water droplets covering the touch panel. When the waterproof function is enabled, Touch14 is set as shield channel by default. The larger the parasitic capacitance on the shielding channel, the higher the drive capability needs to be set. The equivalent capacitance of the shield channel can be estimated through the reading value of the denoise channel(Touch0).

struct **touch_filter_config**

Touch sensor filter configuration

Public Members

touch_filter_mode_t **mode**

Set filter mode. The input of the filter is the raw value of touch reading, and the output of the filter is involved in the judgment of the touch state.

uint32_t **debounce_cnt**

Set debounce count, such as `n`. If the measured values continue to exceed the threshold for `n+1` times, the touch sensor state changes. Range: 0 ~ 7

uint32_t **noise_thr**

Noise threshold coefficient. Higher = More noise resistance. The actual noise should be less than (noise coefficient * touch threshold). Range: 0 ~ 3. The coefficient is 0: 4/8; 1: 3/8; 2: 2/8; 3: 1;

uint32_t **jitter_step**

Set jitter filter step size. Range: 0 ~ 15

touch_smooth_mode_t **smh_lvl**

Level of filter applied on the original data against large noise interference.

struct **touch_pad_sleep_channel_t**

Touch sensor channel sleep configuration

Public Members

touch_pad_t **touch_num**

Set touch channel number for sleep pad. Only one touch sensor channel is supported in deep sleep mode. If clear the sleep channel, point this pad to `TOUCH_PAD_NUM0`

bool **en_proximity**

enable proximity function for sleep pad

Macros

TOUCH_PAD_BIT_MASK_ALL

TOUCH_PAD_SLOPE_DEFAULT

TOUCH_PAD_TIE_OPT_DEFAULT

TOUCH_PAD_BIT_MASK_MAX

TOUCH_PAD_HIGH_VOLTAGE_THRESHOLD

TOUCH_PAD_LOW_VOLTAGE_THRESHOLD

TOUCH_PAD_ATTEN_VOLTAGE_THRESHOLD

TOUCH_PAD_IDLE_CH_CONNECT_DEFAULT

TOUCH_PAD_THRESHOLD_MAX

If set touch threshold max value, The touch sensor can't be in touched status

TOUCH_PAD_SLEEP_CYCLE_DEFAULT

Excessive total time will slow down the touch response. Too small measurement time will not be sampled enough, resulting in inaccurate measurements.

Note: The greater the duty cycle of the measurement time, the more system power is consumed. The number of sleep cycle in each measure process of touch channels. The timer frequency is RTC_SLOW_CLK (can be 150k or 32k depending on the options). Range: 0 ~ 0xffff

---

TOUCH_PAD_MEASURE_CYCLE_DEFAULT

The times of charge and discharge in each measure process of touch channels. The timer frequency is 8Mhz. Recommended typical value: Modify this value to make the measurement time around 1ms. Range: 0 ~ 0xffff

TOUCH_PAD_INTR_MASK_ALL

All touch interrupt type enable.

TOUCH_PROXIMITY_MEAS_NUM_MAX

Touch sensor proximity detection configuration

TOUCH_DEBOUNCE_CNT_MAX

TOUCH_NOISE_THR_MAX

TOUCH_JITTER_STEP_MAX

Type Definitions

typedef struct *touch_pad_denoise* **touch_pad_denoise_t**

Touch sensor denoise configuration

typedef struct *touch_pad_waterproof* **touch_pad_waterproof_t**

Touch sensor waterproof configuration

typedef struct *touch_filter_config* **touch_filter_config_t**

Touch sensor filter configuration

Enumerations

enum **touch_pad_t**

Touch pad channel

Values:

enumerator **TOUCH_PAD_NUM0**

Touch pad channel 0 is GPIO4(ESP32)

enumerator **TOUCH_PAD_NUM1**

Touch pad channel 1 is GPIO0(ESP32) / GPIO1(ESP32-S2)

enumerator **TOUCH_PAD_NUM2**

Touch pad channel 2 is GPIO2(ESP32) / GPIO2(ESP32-S2)

enumerator **TOUCH_PAD_NUM3**

Touch pad channel 3 is GPIO15(ESP32) / GPIO3(ESP32-S2)

enumerator **TOUCH_PAD_NUM4**

Touch pad channel 4 is GPIO13(ESP32) / GPIO4(ESP32-S2)

enumerator **TOUCH_PAD_NUM5**

Touch pad channel 5 is GPIO12(ESP32) / GPIO5(ESP32-S2)

enumerator **TOUCH_PAD_NUM6**

Touch pad channel 6 is GPIO14(ESP32) / GPIO6(ESP32-S2)

enumerator **TOUCH_PAD_NUM7**

Touch pad channel 7 is GPIO27(ESP32) / GPIO7(ESP32-S2)

enumerator **TOUCH_PAD_NUM8**

Touch pad channel 8 is GPIO33(ESP32) / GPIO8(ESP32-S2)

enumerator **TOUCH_PAD_NUM9**

Touch pad channel 9 is GPIO32(ESP32) / GPIO9(ESP32-S2)

enumerator **TOUCH_PAD_NUM10**

Touch channel 10 is GPIO10(ESP32-S2)

enumerator **TOUCH_PAD_NUM11**

Touch channel 11 is GPIO11(ESP32-S2)

enumerator **TOUCH_PAD_NUM12**

Touch channel 12 is GPIO12(ESP32-S2)

enumerator **TOUCH_PAD_NUM13**

Touch channel 13 is GPIO13(ESP32-S2)

enumerator **TOUCH_PAD_NUM14**

Touch channel 14 is GPIO14(ESP32-S2)

enumerator **TOUCH_PAD_MAX**

enum **touch_high_volt_t**

Touch sensor high reference voltage

Values:

enumerator **TOUCH_HVOLT_KEEP**

Touch sensor high reference voltage, no change

enumerator **TOUCH_HVOLT_2V4**

Touch sensor high reference voltage, 2.4V

enumerator **TOUCH_HVOLT_2V5**

Touch sensor high reference voltage, 2.5V

enumerator **TOUCH_HVOLT_2V6**

Touch sensor high reference voltage, 2.6V

enumerator **TOUCH_HVOLT_2V7**

Touch sensor high reference voltage, 2.7V

enumerator **TOUCH_HVOLT_MAX**

enum **touch_low_volt_t**

Touch sensor low reference voltage

Values:

enumerator **TOUCH_LVOLT_KEEP**

Touch sensor low reference voltage, no change

enumerator **TOUCH_LVOLT_0V5**

Touch sensor low reference voltage, 0.5V

enumerator **TOUCH_LVOLT_0V6**

Touch sensor low reference voltage, 0.6V

enumerator **TOUCH_LVOLT_0V7**

Touch sensor low reference voltage, 0.7V

enumerator **TOUCH_LVOLT_0V8**

Touch sensor low reference voltage, 0.8V

enumerator **TOUCH_LVOLT_MAX**

enum **touch_volt_atten_t**

Touch sensor high reference voltage attenuation

Values:

enumerator **TOUCH_HVOLT_ATTEN_KEEP**

Touch sensor high reference voltage attenuation, no change

enumerator **TOUCH_HVOLT_ATTEN_1V5**

Touch sensor high reference voltage attenuation, 1.5V attenuation

enumerator **TOUCH_HVOLT_ATTEN_1V**

Touch sensor high reference voltage attenuation, 1.0V attenuation

enumerator **TOUCH_HVOLT_ATTEN_0V5**

Touch sensor high reference voltage attenuation, 0.5V attenuation

enumerator **TOUCH_HVOLT_ATTEN_0V**

Touch sensor high reference voltage attenuation, 0V attenuation

enumerator **TOUCH_HVOLT_ATTEN_MAX**

enum **touch_cnt_slope_t**

Touch sensor charge/discharge speed

Values:

enumerator **TOUCH_PAD_SLOPE_0**

Touch sensor charge / discharge speed, always zero

enumerator **TOUCH_PAD_SLOPE_1**

Touch sensor charge / discharge speed, slowest

enumerator **TOUCH_PAD_SLOPE_2**

Touch sensor charge / discharge speed

enumerator **TOUCH_PAD_SLOPE_3**

Touch sensor charge / discharge speed

enumerator **TOUCH_PAD_SLOPE_4**

Touch sensor charge / discharge speed

enumerator **TOUCH_PAD_SLOPE_5**

Touch sensor charge / discharge speed

enumerator **TOUCH_PAD_SLOPE_6**

Touch sensor charge / discharge speed

enumerator **TOUCH_PAD_SLOPE_7**

Touch sensor charge / discharge speed, fast

enumerator **TOUCH_PAD_SLOPE_MAX**

enum **touch_tie_opt_t**

Touch sensor initial charge level

Values:

enumerator **TOUCH_PAD_TIE_OPT_LOW**

Initial level of charging voltage, low level

enumerator **TOUCH_PAD_TIE_OPT_HIGH**

Initial level of charging voltage, high level

enumerator **TOUCH_PAD_TIE_OPT_MAX**

enum **touch_fsm_mode_t**

Touch sensor FSM mode

Values:

enumerator **TOUCH_FSM_MODE_TIMER**

To start touch FSM by timer

enumerator **TOUCH_FSM_MODE_SW**

To start touch FSM by software trigger

enumerator **TOUCH_FSM_MODE_MAX**

enum **touch_trigger_mode_t**

Values:

enumerator **TOUCH_TRIGGER_BELOW**

Touch interrupt will happen if counter value is less than threshold.

enumerator **TOUCH_TRIGGER_ABOVE**

Touch interrupt will happen if counter value is larger than threshold.

enumerator **TOUCH_TRIGGER_MAX**

enum **touch_trigger_src_t**

Values:

enumerator **TOUCH_TRIGGER_SOURCE_BOTH**

wakeup interrupt is generated if both SET1 and SET2 are "touched"

enumerator **TOUCH_TRIGGER_SOURCE_SET1**

wakeup interrupt is generated if SET1 is "touched"

enumerator **TOUCH_TRIGGER_SOURCE_MAX**

enum **touch_pad_intr_mask_t**

Values:

enumerator **TOUCH_PAD_INTR_MASK_DONE**

Measurement done for one of the enabled channels.

enumerator **TOUCH_PAD_INTR_MASK_ACTIVE**

Active for one of the enabled channels.

enumerator **TOUCH_PAD_INTR_MASK_INACTIVE**

Inactive for one of the enabled channels.

enumerator **TOUCH_PAD_INTR_MASK_SCAN_DONE**

Measurement done for all the enabled channels.

enumerator **TOUCH_PAD_INTR_MASK_TIMEOUT**

Timeout for one of the enabled channels.

enumerator **TOUCH_PAD_INTR_MASK_PROXI_MEAS_DONE**

For proximity sensor, when the number of measurements reaches the set count of measurements, an interrupt will be generated.

enum **touch_pad_denoise_grade_t**

Values:

enumerator **TOUCH_PAD_DENOISE_BIT12**

Denoise range is 12bit

enumerator **TOUCH_PAD_DENOISE_BIT10**

Denoise range is 10bit

enumerator **TOUCH_PAD_DENOISE_BIT8**

Denoise range is 8bit

enumerator **TOUCH_PAD_DENOISE_BIT4**

Denoise range is 4bit

enumerator **TOUCH_PAD_DENOISE_MAX**

enum **touch_pad_denoise_cap_t**

Values:

enumerator **TOUCH_PAD_DENOISE_CAP_L0**

Denoise channel internal reference capacitance is 5pf

enumerator **TOUCH_PAD_DENOISE_CAP_L1**

Denoise channel internal reference capacitance is 6.4pf

enumerator **TOUCH_PAD_DENOISE_CAP_L2**

Denoise channel internal reference capacitance is 7.8pf

enumerator **TOUCH_PAD_DENOISE_CAP_L3**

Denoise channel internal reference capacitance is 9.2pf

enumerator **TOUCH_PAD_DENOISE_CAP_L4**

Denoise channel internal reference capacitance is 10.6pf

enumerator **TOUCH_PAD_DENOISE_CAP_L5**

Denoise channel internal reference capacitance is 12.0pf

enumerator **TOUCH_PAD_DENOISE_CAP_L6**

Denoise channel internal reference capacitance is 13.4pf

enumerator **TOUCH_PAD_DENOISE_CAP_L7**

Denoise channel internal reference capacitance is 14.8pf

enumerator **TOUCH_PAD_DENOISE_CAP_MAX**

enum **touch_pad_shield_driver_t**

Touch sensor shield channel drive capability level

Values:

enumerator **TOUCH_PAD_SHIELD_DRV_L0**

The max equivalent capacitance in shield channel is 40pf

enumerator **TOUCH_PAD_SHIELD_DRV_L1**

The max equivalent capacitance in shield channel is 80pf

enumerator **TOUCH_PAD_SHIELD_DRV_L2**

The max equivalent capacitance in shield channel is 120pf

enumerator **TOUCH_PAD_SHIELD_DRV_L3**

The max equivalent capacitance in shield channel is 160pf

enumerator **TOUCH_PAD_SHIELD_DRV_L4**

The max equivalent capacitance in shield channel is 200pf

enumerator **TOUCH_PAD_SHIELD_DRV_L5**

The max equivalent capacitance in shield channel is 240pf

enumerator **TOUCH_PAD_SHIELD_DRV_L6**

The max equivalent capacitance in shield channel is 280pf

enumerator **TOUCH_PAD_SHIELD_DRV_L7**

The max equivalent capacitance in shield channel is 320pf

enumerator **TOUCH_PAD_SHIELD_DRV_MAX**

enum **touch_pad_conn_type_t**

Touch channel idle state configuration

Values:

enumerator **TOUCH_PAD_CONN_HIGHZ**

Idle status of touch channel is high resistance state

enumerator **TOUCH_PAD_CONN_GND**

Idle status of touch channel is ground connection

enumerator **TOUCH_PAD_CONN_MAX**

enum **touch_filter_mode_t**

Touch channel IIR filter coefficient configuration.

Note: On ESP32S2. There is an error in the IIR calculation. The magnitude of the error is twice the filter coefficient. So please select a smaller filter coefficient on the basis of meeting the filtering requirements. Recommended filter coefficient selection `IIR_16`.

Values:

enumerator **TOUCH_PAD_FILTER_IIR_4**

The filter mode is first-order IIR filter. The coefficient is 4.

enumerator **TOUCH_PAD_FILTER_IIR_8**

The filter mode is first-order IIR filter. The coefficient is 8.

enumerator **TOUCH_PAD_FILTER_IIR_16**

The filter mode is first-order IIR filter. The coefficient is 16 (Typical value).

enumerator **TOUCH_PAD_FILTER_IIR_32**

The filter mode is first-order IIR filter. The coefficient is 32.

enumerator **TOUCH_PAD_FILTER_IIR_64**

The filter mode is first-order IIR filter. The coefficient is 64.

enumerator **TOUCH_PAD_FILTER_IIR_128**

The filter mode is first-order IIR filter. The coefficient is 128.

enumerator **TOUCH_PAD_FILTER_IIR_256**

The filter mode is first-order IIR filter. The coefficient is 256.

enumerator **TOUCH_PAD_FILTER_JITTER**

The filter mode is jitter filter

enumerator **TOUCH_PAD_FILTER_MAX**

enum **touch_smooth_mode_t**

Level of filter applied on the original data against large noise interference.

Note: On ESP32S2. There is an error in the IIR calculation. The magnitude of the error is twice the filter coefficient. So please select a smaller filter coefficient on the basis of meeting the filtering requirements. Recommended filter coefficient selection `IIR_2`.

Values:

enumerator **TOUCH_PAD_SMOOTH_OFF**

No filtering of raw data.

enumerator **TOUCH_PAD_SMOOTH_IIR_2**

Filter the raw data. The coefficient is 2 (Typical value).

enumerator **TOUCH_PAD_SMOOTH_IIR_4**

Filter the raw data. The coefficient is 4.

enumerator **TOUCH_PAD_SMOOTH_IIR_8**

Filter the raw data. The coefficient is 8.

enumerator **TOUCH_PAD_SMOOTH_MAX**

2.6.27 Two-Wire Automotive Interface (TWAI)

Overview

The Two-Wire Automotive Interface (TWAI) is a real-time serial communication protocol suited for automotive and industrial applications. It is compatible with ISO11898-1 Classical frames, thus can support Standard Frame

Format (11-bit ID) and Extended Frame Format (29-bit ID). The ESP32-S3 contains 1 TWAI controller(s) that can be configured to communicate on a TWAI bus via an external transceiver.

Warning: The TWAI controller is not compatible with ISO11898-1 FD Format frames, and will interpret such frames as errors.

This programming guide is split into the following sections:

Sections

- *Two-Wire Automotive Interface (TWAI)*
  - *Overview*
  - *TWAI Protocol Summary*
  - *Signals Lines and Transceiver*
  - *API Naming Conventions*
  - *Driver Configuration*
  - *Driver Operation*
  - *Examples*
  - *API Reference*

TWAI Protocol Summary

The TWAI is a multi-master, multi-cast, asynchronous, serial communication protocol. TWAI also supports error detection and signalling, and inbuilt message prioritization.

Multi-master: Any node on the bus can initiate the transfer of a message.

Multi-cast: When a node transmits a message, all nodes on the bus will receive the message (i.e., broadcast) thus ensuring data consistency across all nodes. However, some nodes can selectively choose which messages to accept via the use of acceptance filtering (multi-cast).

Asynchronous: The bus does not contain a clock signal. All nodes on the bus operate at the same bit rate and synchronize using the edges of the bits transmitted on the bus.

Error Detection and Signaling: Every node constantly monitors the bus. When any node detects an error, it signals the detection by transmitting an error frame. Other nodes will receive the error frame and transmit their own error frames in response. This results in an error detection being propagated to all nodes on the bus.

Message Priorities: Messages contain an ID field. If two or more nodes attempt to transmit simultaneously, the node transmitting the message with the lower ID value will win arbitration of the bus. All other nodes will become receivers ensuring that there is at most one transmitter at any time.

TWAI Messages TWAI Messages are split into Data Frames and Remote Frames. Data Frames are used to deliver a data payload to other nodes, whereas a Remote Frame is used to request a Data Frame from other nodes (other nodes can optionally respond with a Data Frame). Data and Remote Frames have two frame formats known as **Extended Frame** and **Standard Frame** which contain a 29-bit ID and an 11-bit ID respectively. A TWAI message consists of the following fields:

- 29-bit or 11-bit ID: Determines the priority of the message (lower value has higher priority).
- Data Length Code (DLC) between 0 to 8: Indicates the size (in bytes) of the data payload for a Data Frame, or the amount of data to request for a Remote Frame.
- Up to 8 bytes of data for a Data Frame (should match DLC).

Error States and Counters The TWAI protocol implements a feature known as "fault confinement" where a persistently erroneous node will eventually eliminate itself from the bus. This is implemented by requiring every node to maintain two internal error counters known as the **Transmit Error Counter (TEC)** and the **Receive Error**

Counter (REC). The two error counters are incremented and decremented according to a set of rules (where the counters increase on an error, and decrease on a successful message transmission/reception). The values of the counters are used to determine a node's **error state**, namely **Error Active**, **Error Passive**, and **Bus-Off**.

Error Active: A node is Error Active when **both TEC and REC are less than 128** and indicates that the node is operating normally. Error Active nodes are allowed to participate in bus communications, and will actively signal the detection of any errors by automatically transmitting an **Active Error Flag** over the bus.

Error Passive: A node is Error Passive when **either the TEC or REC becomes greater than or equal to 128**. Error Passive nodes are still able to take part in bus communications, but will instead transmit a **Passive Error Flag** upon detection of an error.

Bus-Off: A node becomes Bus-Off when the **TEC becomes greater than or equal to 256**. A Bus-Off node is unable influence the bus in any manner (essentially disconnected from the bus) thus eliminating itself from the bus. A node will remain in the Bus-Off state until it undergoes bus-off recovery.

Signals Lines and Transceiver

The TWAI controller does not contain a integrated transceiver. Therefore, to connect the TWAI controller to a TWAI bus, **an external transceiver is required**. The type of external transceiver used should depend on the application's physical layer specification (e.g., using SN65HVD23x transceivers for ISO 11898-2 compatibility).

The TWAI controller's interface consists of 4 signal lines known as **TX, RX, BUS-OFF, and CLKOUT**. These four signal lines can be routed through the GPIO Matrix to the ESP32-S3's GPIO pads.

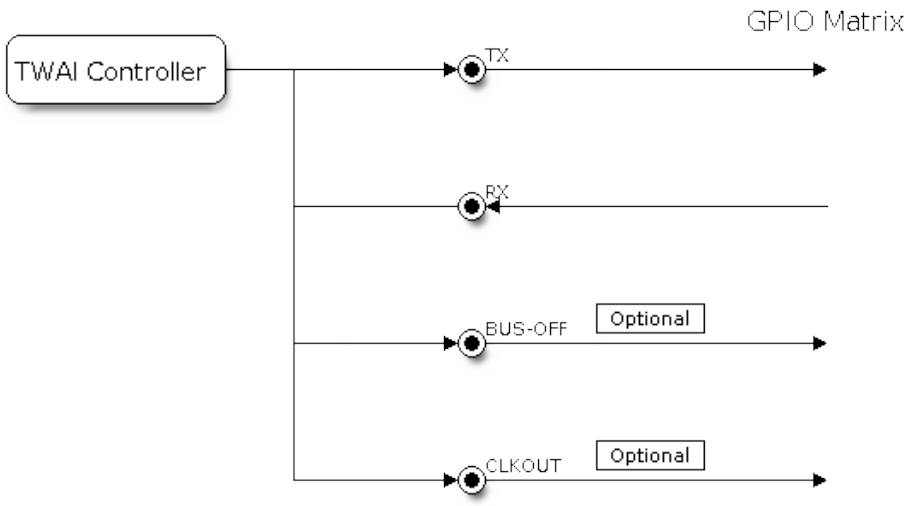


Fig. 27: Signal lines of the TWAI controller

TX and RX: The TX and RX signal lines are required to interface with an external transceiver. Both signal lines represent/interpret a dominant bit as a low logic level (0 V), and a recessive bit as a high logic level (3.3 V).

BUS-OFF: The BUS-OFF signal line is **optional** and is set to a low logic level (0 V) whenever the TWAI controller reaches a bus-off state. The BUS-OFF signal line is set to a high logic level (3.3 V) otherwise.

CLKOUT: The CLKOUT signal line is **optional** and outputs a prescaled version of the controller's source clock.

Note: An external transceiver **must internally loopback the TX to RX** such that a change in logic level to the TX signal line can be observed on the RX line. Failing to do so will cause the TWAI controller to interpret differences in logic levels between the two signal lines as a loss in arbitration or a bit error.

API Naming Conventions

Note: The TWAI driver provides two sets of API. One is handle-free and is widely used in IDF versions earlier than v5.2, but it can only support one TWAI hardware controller. The other set is with handles, and the function name is usually suffixed with "v2", which can support any number of TWAI controllers. These two sets of API can be used at the same time, but it is recommended to use the "v2" version in your new projects.

Driver Configuration

This section covers how to configure the TWAI driver.

Operating Modes The TWAI driver supports the following modes of operations:

Normal Mode: The normal operating mode allows the TWAI controller to take part in bus activities such as transmitting and receiving messages/error frames. Acknowledgement from another node is required when transmitting a message.

No Ack Mode: The No Acknowledgement mode is similar to normal mode, however acknowledgements are not required for a message transmission to be considered successful. This mode is useful when self testing the TWAI controller (loopback of transmissions).

Listen Only Mode: This mode prevents the TWAI controller from influencing the bus. Therefore, transmission of messages/acknowledgement/error frames will be disabled. However the TWAI controller is still able to receive messages but will not acknowledge the message. This mode is suited for bus monitor applications.

Alerts The TWAI driver contains an alert feature that is used to notify the application layer of certain TWAI controller or TWAI bus events. Alerts are selectively enabled when the TWAI driver is installed, but can be reconfigured during runtime by calling *twai_reconfigure_alerts()*. The application can then wait for any enabled alerts to occur by calling *twai_read_alerts()*. The TWAI driver supports the following alerts:

Table 4: TWAI Driver Alerts

Alert Flag	Description
TWAI_ALERT_TX_IDLE	No more messages queued for transmission
TWAI_ALERT_TX_SUCCESS	The previous transmission was successful
TWAI_ALERT_RX_DATA	A frame has been received and added to the RX queue
TWAI_ALERT_BELOW_ERR_WARN	Both error counters have dropped below error warning limit
TWAI_ALERT_ERR_ACTIVE	TWAI controller has become error active
TWAI_ALERT_RECOVERY_IN_PROGRESS	TWAI controller is undergoing bus recovery
TWAI_ALERT_BUS_RECOVERED	TWAI controller has successfully completed bus recovery
TWAI_ALERT_ARB_LOST	The previous transmission lost arbitration
TWAI_ALERT_ABOVE_ERR_WARN	One of the error counters have exceeded the error warning limit
TWAI_ALERT_BUS_ERROR	A (Bit, Stuff, CRC, Form, ACK) error has occurred on the bus
TWAI_ALERT_TX_FAILED	The previous transmission has failed
TWAI_ALERT_RX_QUEUE_FULL	The RX queue is full causing a received frame to be lost
TWAI_ALERT_ERR_PASS	TWAI controller has become error passive
TWAI_ALERT_BUS_OFF	Bus-off condition occurred. TWAI controller can no longer influence bus

Note: The TWAI controller's **error warning limit** is used to preemptively warn the application of bus errors before the error passive state is reached. By default, the TWAI driver sets the **error warning limit** to **96**. The TWAI_ALERT_ABOVE_ERR_WARN is raised when the TEC or REC becomes larger then or equal to the error

warning limit. The `TWAI_ALERT_BELOW_ERR_WARN` is raised when both TEC and REC return back to values below **96**.

Note: When enabling alerts, the `TWAI_ALERT_AND_LOG` flag can be used to cause the TWAI driver to log any raised alerts to UART. However, alert logging is disabled and `TWAI_ALERT_AND_LOG` if the *CONFIG_TWAI_ISR_IN_IRAM* option is enabled (see *Placing ISR into IRAM*).

Note: The `TWAI_ALERT_ALL` and `TWAI_ALERT_NONE` macros can also be used to enable/disable all alerts during configuration/reconfiguration.

Bit Timing The operating bit rate of the TWAI driver is configured using the *`twai_timing_config_t`* structure. The period of each bit is made up of multiple **time quanta**, and the period of a **time quantum** is determined by a pre-scaled version of the TWAI controller's source clock. A single bit contains the following segments in the following order:

1. The **Synchronization Segment** consists of a single time quantum
2. **Timing Segment 1** consists of 1 to 16 time quanta before sample point
3. **Timing Segment 2** consists of 1 to 8 time quanta after sample point

The **Baudrate Prescaler** is used to determine the period of each time quantum by dividing the TWAI controller's source clock. On the ESP32-S3, the `brp` can be **any even number from 2 to 16384**. Alternatively, you can decide the resolution of each quantum, by setting *`twai_timing_config_t::quanta_resolution_hz`* to a non-zero value. In this way, the driver can calculate the underlying `brp` value for you. It is useful when you set different clock sources but want the bitrate to keep the same.

Supported clock source for a TWAI controller is listed in the *`twai_clock_source_t`* and can be specified in *`twai_timing_config_t::clk_src`*.

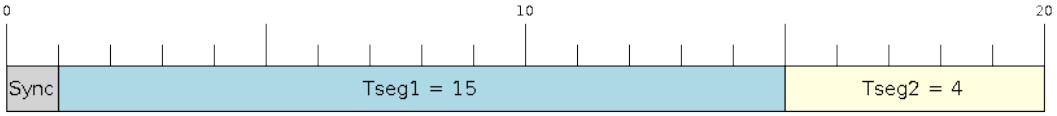


Fig. 28: Bit timing configuration for 500kbit/s given BRP = 8, clock source frequency is 80MHz

The sample point of a bit is located on the intersection of Timing Segment 1 and 2. Enabling **Triple Sampling** causes 3 time quanta to be sampled per bit instead of 1 (extra samples are located at the tail end of Timing Segment 1).

The **Synchronization Jump Width** is used to determine the maximum number of time quanta a single bit time can be lengthened/shortened for synchronization purposes. `sjw` can **range from 1 to 4**.

Note: Multiple combinations of `brp`, `tseg_1`, `tseg_2`, and `sjw` can achieve the same bit rate. Users should tune these values to the physical characteristics of their bus by taking into account factors such as **propagation delay, node information processing time, and phase errors**.

Bit timing **macro initializers** are also available for commonly used bit rates. The following macro initializers are provided by the TWAI driver.

- `TWAI_TIMING_CONFIG_1MBITS`
- `TWAI_TIMING_CONFIG_800KBITS`

- `TWAI_TIMING_CONFIG_500KBITS`
- `TWAI_TIMING_CONFIG_250KBITS`
- `TWAI_TIMING_CONFIG_125KBITS`
- `TWAI_TIMING_CONFIG_100KBITS`
- `TWAI_TIMING_CONFIG_50KBITS`
- `TWAI_TIMING_CONFIG_25KBITS`
- `TWAI_TIMING_CONFIG_20KBITS`
- `TWAI_TIMING_CONFIG_16KBITS`
- `TWAI_TIMING_CONFIG_12_5KBITS`
- `TWAI_TIMING_CONFIG_10KBITS`
- `TWAI_TIMING_CONFIG_5KBITS`
- `TWAI_TIMING_CONFIG_1KBITS`

Acceptance Filter The TWAI controller contains a hardware acceptance filter which can be used to filter messages of a particular ID. A node that filters out a message **does not receive the message, but will still acknowledge it**. Acceptance filters can make a node more efficient by filtering out messages sent over the bus that are irrelevant to the node. The acceptance filter is configured using two 32-bit values within *`twai_filter_config_t`* known as the **acceptance code** and the **acceptance mask**.

The **acceptance code** specifies the bit sequence which a message's ID, RTR, and data bytes must match in order for the message to be received by the TWAI controller. The **acceptance mask** is a bit sequence specifying which bits of the acceptance code can be ignored. This allows for a messages of different IDs to be accepted by a single acceptance code.

The acceptance filter can be used under **Single or Dual Filter Mode**. Single Filter Mode uses the acceptance code and mask to define a single filter. This allows for the first two data bytes of a standard frame to be filtered, or the entirety of an extended frame's 29-bit ID. The following diagram illustrates how the 32-bit acceptance code and mask are interpreted under Single Filter Mode (Note: The yellow and blue fields represent standard and extended frame formats respectively).

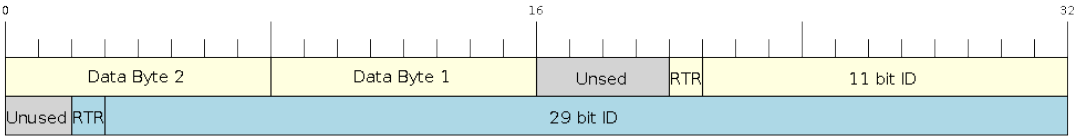


Fig. 29: Bit layout of single filter mode (Right side MSBit)

Dual Filter Mode uses the acceptance code and mask to define two separate filters allowing for increased flexibility of ID's to accept, but does not allow for all 29-bits of an extended ID to be filtered. The following diagram illustrates how the 32-bit acceptance code and mask are interpreted under **Dual Filter Mode** (Note: The yellow and blue fields represent standard and extended frame formats respectively).

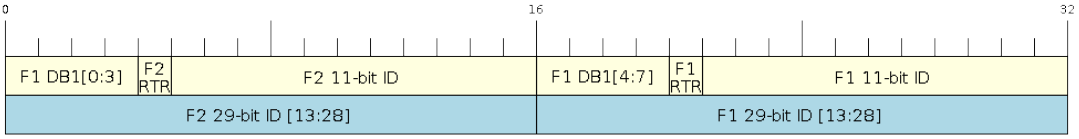


Fig. 30: Bit layout of dual filter mode (Right side MSBit)

Disabling TX Queue The TX queue can be disabled during configuration by setting the `tx_queue_len` member of *`twai_general_config_t`* to `0`. This allows applications that do not require message transmission to save a small amount of memory when using the TWAI driver.

Placing ISR into IRAM The TWAI driver's ISR (Interrupt Service Routine) can be placed into IRAM so that the ISR can still run whilst the cache is disabled. Placing the ISR into IRAM may be necessary to maintain the TWAI driver's functionality during lengthy cache disabling operations (such as SPI Flash writes, OTA updates etc). Whilst the cache is disabled, the ISR continues to:

- Read received messages from the RX buffer and place them into the driver's RX queue.
- Load messages pending transmission from the driver's TX queue and write them into the TX buffer.

To place the TWAI driver's ISR, users must do the following:

- Enable the *CONFIG_TWAI_ISR_IN_IRAM* option using `idf.py menuconfig`.
- When calling *`twai_driver_install()`*, the `intr_flags` member of *`twai_general_config_t`* should set the *`ESP_INTR_FLAG_IRAM`* set.

Note: When the *CONFIG_TWAI_ISR_IN_IRAM* option is enabled, the TWAI driver will no longer log any alerts (i.e., the `TWAI_ALERT_AND_LOG` flag will not have any effect).

Driver Operation

The TWAI driver is designed with distinct states and strict rules regarding the functions or conditions that trigger a state transition. The following diagram illustrates the various states and their transitions.

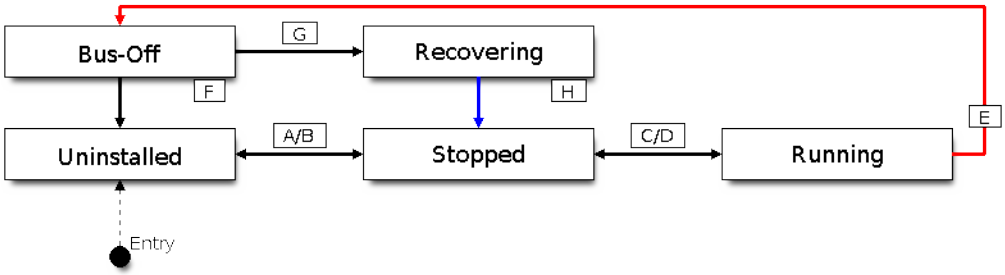


Fig. 31: State transition diagram of the TWAI driver (see table below)

Label	Transition	Action/Condition
A	Uninstalled > Stopped	*`twai_driver_install()`*
B	Stopped > Uninstalled	*`twai_driver_uninstall()`*
C	Stopped > Running	*`twai_start()`*
D	Running > Stopped	*`twai_stop()`*
E	Running > Bus-Off	Transmit Error Counter >= 256
F	Bus-Off > Uninstalled	*`twai_driver_uninstall()`*
G	Bus-Off > Recovering	*`twai_initiate_recovery()`*
H	Recovering > Stopped	128 occurrences of 11 consecutive recessive bits.

Driver States **Uninstalled**: In the uninstalled state, no memory is allocated for the driver and the TWAI controller is powered OFF.

Stopped: In this state, the TWAI controller is powered ON and the TWAI driver has been installed. However the TWAI controller is unable to take part in any bus activities such as transmitting, receiving, or acknowledging messages.

Running: In the running state, the TWAI controller is able to take part in bus activities. Therefore messages can be transmitted/received/acknowledged. Furthermore, the TWAI controller is able to transmit error frames upon detection of errors on the bus.

Bus-Off: The bus-off state is automatically entered when the TWAI controller's Transmit Error Counter becomes greater than or equal to 256. The bus-off state indicates the occurrence of severe errors on the bus or in the TWAI controller. Whilst in the bus-off state, the TWAI controller is unable to take part in any bus activities. To exit the bus-off state, the TWAI controller must undergo the bus recovery process.

Recovering: The recovering state is entered when the TWAI controller undergoes bus recovery. The TWAI controller/TWAI driver remains in the recovering state until the 128 occurrences of 11 consecutive recessive bits is observed on the bus.

Message Fields and Flags The TWAI driver distinguishes different types of messages by using the various bit field members of the *twai_message_t* structure. These bit field members determine whether a message is in standard or extended format, a remote frame, and the type of transmission to use when transmitting such a message.

These bit field members can also be toggled using the `flags` member of *twai_message_t* and the following message flags:

Message Flag	Description
TWAI_MSG_FLAG_EXTD	Message is in Extended Frame Format (29bit ID)
TWAI_MSG_FLAG_RTR	Message is a Remote Frame (Remote Transmission Request)
TWAI_MSG_FLAG_SS	Transmit message using Single Shot Transmission (Message will not be retransmitted upon error or loss of arbitration). Unused for received message.
TWAI_MSG_FLAG_SELF	Transmit message using Self Reception Request (Transmitted message will also received by the same node). Unused for received message.
TWAI_MSG_FLAG_DLC_NON_COMP	Message's Data length code is larger than 8. This will break compliance with TWAI
TWAI_MSG_FLAG_NONE	Clears all bit fields. Equivalent to a Standard Frame Format (11bit ID) Data Frame.

Examples

Configuration & Installation The following code snippet demonstrates how to configure, install, and start the TWAI driver via the use of the various configuration structures, macro initializers, the *twai_driver_install()* function, and the *twai_start()* function.

```
#include "driver/gpio.h"
#include "driver/twai.h"

void app_main()
{
    //Initialize configuration structures using macro initializers
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(GPIO_NUM_21, GPIO_
↪NUM_22, TWAI_MODE_NORMAL);
    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

    //Install TWAI driver
    if (twai_driver_install(&g_config, &t_config, &f_config) == ESP_OK) {
        printf("Driver installed\n");
    } else {
        printf("Failed to install driver\n");
        return;
    }

    //Start TWAI driver
    if (twai_start() == ESP_OK) {
        printf("Driver started\n");
    } else {
        printf("Failed to start driver\n");
```

(continues on next page)

(continued from previous page)

```
        return;
    }

    ...

}
```

The usage of macro initializers is not mandatory and each of the configuration structures can be manually.

Install Multiple TWAI Instances The following code snippet demonstrates how to install multiple TWAI instances via the use of the *twai_driver_install_v2()* function.

```
#include "driver/gpio.h"
#include "driver/twai.h"

void app_main()
{
    twai_handle_t twai_bus_0;
    twai_handle_t twai_bus_1;
    //Initialize configuration structures using macro initializers
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(GPIO_NUM_0, GPIO_
↪NUM_1, TWAI_MODE_NORMAL);
    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

    //Install driver for TWAI bus 0
    g_config.controller_id = 0;
    if (twai_driver_install_v2(&g_config, &t_config, &f_config, &twai_bus_0) ==␣
↪ESP_OK) {
        printf("Driver installed\n");
    } else {
        printf("Failed to install driver\n");
        return;
    }
    //Start TWAI driver
    if (twai_start_v2(twai_bus_0) == ESP_OK) {
        printf("Driver started\n");
    } else {
        printf("Failed to start driver\n");
        return;
    }

    //Install driver for TWAI bus 1
    g_config.controller_id = 1;
    g_config.tx_io = GPIO_NUM_2;
    g_config.rx_io = GPIO_NUM_3;
    if (twai_driver_install_v2(&g_config, &t_config, &f_config, &twai_bus_1) ==␣
↪ESP_OK) {
        printf("Driver installed\n");
    } else {
        printf("Failed to install driver\n");
        return;
    }
    //Start TWAI driver
    if (twai_start_v2(twai_bus_1) == ESP_OK) {
        printf("Driver started\n");
    } else {
        printf("Failed to start driver\n");
        return;
    }
```

(continues on next page)

(continued from previous page)

```
    //Other Driver operations must use version 2 API as well
    ...

}
```

Message Transmission The following code snippet demonstrates how to transmit a message via the usage of the *twai_message_t* type and *twai_transmit()* function.

```
#include "driver/twai.h"

...

// Configure message to transmit
twai_message_t message = {
    // Message type and format settings
    .extd = 1,              // Standard vs extended format
    .rtr = 0,               // Data vs RTR frame
    .ss = 0,                // Whether the message is single shot (i.e., does not␣
↪repeat on error)
    .self = 0,              // Whether the message is a self reception request␣
↪(loopback)
    .dlc_non_comp = 0,      // DLC is less than 8
    // Message ID and payload
    .identifier = 0xAAAA,
    .data_length_code = 4,
    .data = {0, 1, 2, 3},
};

//Queue message for transmission
if (twai_transmit(&message, pdMS_TO_TICKS(1000)) == ESP_OK) {
    printf("Message queued for transmission\n");
} else {
    printf("Failed to queue message for transmission\n");
}
```

Message Reception The following code snippet demonstrates how to receive a message via the usage of the *twai_message_t* type and *twai_receive()* function.

```
#include "driver/twai.h"

...

//Wait for message to be received
twai_message_t message;
if (twai_receive(&message, pdMS_TO_TICKS(10000)) == ESP_OK) {
    printf("Message received\n");
} else {
    printf("Failed to receive message\n");
    return;
}

//Process received message
if (message.extd) {
    printf("Message is in Extended Format\n");
} else {
    printf("Message is in Standard Format\n");
}
```

(continues on next page)

(continued from previous page)

```
printf("ID is %d\n", message.identifier);
if (!(message.rtr)) {
    for (int i = 0; i < message.data_length_code; i++) {
        printf("Data byte %d = %d\n", i, message.data[i]);
    }
}
```

Reconfiguring and Reading Alerts The following code snippet demonstrates how to reconfigure and read TWAI driver alerts via the use of the *twai_reconfigure_alerts()* and *twai_read_alerts()* functions.

```
#include "driver/twai.h"

...

//Reconfigure alerts to detect Error Passive and Bus-Off error states
uint32_t alerts_to_enable = TWAI_ALERT_ERR_PASS | TWAI_ALERT_BUS_OFF;
if (twai_reconfigure_alerts(alerts_to_enable, NULL) == ESP_OK) {
    printf("Alerts reconfigured\n");
} else {
    printf("Failed to reconfigure alerts");
}

//Block indefinitely until an alert occurs
uint32_t alerts_triggered;
twai_read_alerts(&alerts_triggered, portMAX_DELAY);
```

Stop and Uninstall The following code demonstrates how to stop and uninstall the TWAI driver via the use of the *twai_stop()* and *twai_driver_uninstall()* functions.

```
#include "driver/twai.h"

...

//Stop the TWAI driver
if (twai_stop() == ESP_OK) {
    printf("Driver stopped\n");
} else {
    printf("Failed to stop driver\n");
    return;
}

//Uninstall the TWAI driver
if (twai_driver_uninstall() == ESP_OK) {
    printf("Driver uninstalled\n");
} else {
    printf("Failed to uninstall driver\n");
    return;
}
```

Multiple ID Filter Configuration The acceptance mask in *twai_filter_config_t* can be configured such that two or more IDs are accepted for a single filter. For a particular filter to accept multiple IDs, the conflicting bit positions amongst the IDs must be set in the acceptance mask. The acceptance code can be set to any one of the IDs.

The following example shows how the calculate the acceptance mask given multiple IDs:

```
ID1 =  11'b101 1010 0000
ID2 =  11'b101 1010 0001
```

(continues on next page)

(continued from previous page)

```
ID3 =  11'b101 1010 0100
ID4 =  11'b101 1010 1000
//Acceptance Mask
MASK = 11'b000 0000 1101
```

Application Examples **Network Example:** The TWAI Network example demonstrates communication between two ESP32-S3s using the TWAI driver API. One TWAI node acts as a network master that initiates and ceases the transfer of a data from another node acting as a network slave. The example can be found via peripherals/twai/twai_network.

Alert and Recovery Example: This example demonstrates how to use the TWAI driver's alert and bus-off recovery API. The example purposely introduces errors on the bus to put the TWAI controller into the Bus-Off state. An alert is used to detect the Bus-Off state and trigger the bus recovery process. The example can be found via peripherals/twai/twai_alert_and_recovery.

Self Test Example: This example uses the No Acknowledge Mode and Self Reception Request to cause the TWAI controller to send and simultaneously receive a series of messages. This example can be used to verify if the connections between the TWAI controller and the external transceiver are working correctly. The example can be found via peripherals/twai/twai_self_test.

API Reference

Header File

- components/hal/include/hal/twai_types.h
- This header file can be included with:

```
#include "hal/twai_types.h"
```

Structures

struct **twai_message_t**

Structure to store a TWAI message.

Note: The flags member is deprecated

Public Members

uint32_t **extd**

Extended Frame Format (29bit ID)

uint32_t **rtr**

Message is a Remote Frame

uint32_t **ss**

Transmit as a Single Shot Transmission. Unused for received.

uint32_t **self**

Transmit as a Self Reception Request. Unused for received.

uint32_t **dlc_non_comp**

Message's Data length code is larger than 8. This will break compliance with ISO 11898-1

uint32_t **reserved**

Reserved bits

uint32_t **flags**

Deprecated: Alternate way to set bits using message flags

uint32_t **identifier**

11 or 29 bit identifier

uint8_t **data_length_code**

Data length code

uint8_t **data**[TWAI_FRAME_MAX_DLC]

Data bytes (not relevant in RTR frame)

struct **twai_timing_config_t**

Structure for bit timing configuration of the TWAI driver.

Note: Macro initializers are available for this structure

Public Members

twai_clock_source_t **clk_src**

Clock source, set to 0 or TWAI_CLK_SRC_DEFAULT if you want a default clock source

uint32_t **quanta_resolution_hz**

The resolution of one timing quanta, in Hz. Note: the value of `brp` will reflected by this field if it's non-zero, otherwise, `brp` needs to be set manually

uint32_t **brp**

Baudrate prescale (i.e., clock divider). Any even number from 2 to 128 for ESP32, 2 to 32768 for non-ESP32 chip. Note: For ESP32 ECO 2 or later, multiples of 4 from 132 to 256 are also supported

uint8_t **tseg_1**

Timing segment 1 (Number of time quanta, between 1 to 16)

uint8_t **tseg_2**

Timing segment 2 (Number of time quanta, 1 to 8)

uint8_t **sjw**

Synchronization Jump Width (Max time quanta jump for synchronize from 1 to 4)

bool **triple_sampling**

Enables triple sampling when the TWAI controller samples a bit

struct **twai_filter_config_t**

Structure for acceptance filter configuration of the TWAI driver (see documentation)

Note: Macro initializers are available for this structure

Public Members

uint32_t **acceptance_code**

32-bit acceptance code

uint32_t **acceptance_mask**

32-bit acceptance mask

bool **single_filter**

Use Single Filter Mode (see documentation)

Macros

TWAI_EXTD_ID_MASK

TWAI Constants.

Bit mask for 29 bit Extended Frame Format ID

TWAI_STD_ID_MASK

Bit mask for 11 bit Standard Frame Format ID

TWAI_FRAME_MAX_DLC

Max data bytes allowed in TWAI

TWAI_FRAME_EXTD_ID_LEN_BYTES

EFF ID requires 4 bytes (29bit)

TWAI_FRAME_STD_ID_LEN_BYTES

SFF ID requires 2 bytes (11bit)

TWAI_ERR_PASS_THRESH

Error counter threshold for error passive

Type Definitions

typedef *soc_periph_twai_clk_src_t* **twai_clock_source_t**

RMT group clock source.

Note: User should select the clock source based on the power and resolution requirement

Enumerations

enum **twai_mode_t**

TWAI Controller operating modes.

Values:

enumerator **TWAI_MODE_NORMAL**

Normal operating mode where TWAI controller can send/receive/acknowledge messages

enumerator **TWAI_MODE_NO_ACK**

Transmission does not require acknowledgment. Use this mode for self testing

enumerator **TWAI_MODE_LISTEN_ONLY**

The TWAI controller will not influence the bus (No transmissions or acknowledgments) but can receive messages

Header File

- components/driver/twai/include/driver/twai.h
- This header file can be included with:

```
#include "driver/twai.h"
```

- This header file is a part of the API provided by the `driver` component. To declare that your component depends on `driver`, add the following to your CMakeLists.txt:

```
REQUIRES driver
```

or

```
PRIV_REQUIRES driver
```

Functions

esp_err_t **twai_driver_install** (const *twai_general_config_t* *g_config, const *twai_timing_config_t* *t_config, const *twai_filter_config_t* *f_config)

Install TWAI driver.

This function installs the TWAI driver using three configuration structures. The required memory is allocated and the TWAI driver is placed in the stopped state after running this function.

Note: Macro initializers are available for the configuration structures (see documentation)

Note: To reinstall the TWAI driver, call twai_driver_uninstall() first

Parameters

- **g_config** -- **[in]** General configuration structure
- **t_config** -- **[in]** Timing configuration structure
- **f_config** -- **[in]** Filter configuration structure

Returns

- ESP_OK: Successfully installed TWAI driver
- ESP_ERR_INVALID_ARG: Arguments are invalid, e.g. invalid clock source, invalid quanta resolution
- ESP_ERR_NO_MEM: Insufficient memory
- ESP_ERR_INVALID_STATE: Driver is already installed

esp_err_t **twai_driver_install_v2** (const *twai_general_config_t* *g_config, const *twai_timing_config_t* *t_config, const *twai_filter_config_t* *f_config, *twai_handle_t* *ret_twai)

Install TWAI driver and return a handle.

Note: This is an advanced version of `twai_driver_install` that can return a driver handle, so that it allows you to install multiple TWAI drivers. Don't forget to set the proper controller_id in the *twai_general_config_t* Please refer to the documentation of `twai_driver_install` for more details.

Parameters

- **g_config** -- **[in]** General configuration structure
- **t_config** -- **[in]** Timing configuration structure
- **f_config** -- **[in]** Filter configuration structure
- **ret_twai** -- **[out]** Pointer to a new created TWAI handle

Returns

- ESP_OK: Successfully installed TWAI driver
- ESP_ERR_INVALID_ARG: Arguments are invalid, e.g. invalid clock source, invalid quanta resolution, invalid controller ID
- ESP_ERR_NO_MEM: Insufficient memory
- ESP_ERR_INVALID_STATE: Driver is already installed

esp_err_t **twai_driver_uninstall** (void)

Uninstall the TWAI driver.

This function uninstalls the TWAI driver, freeing the memory utilized by the driver. This function can only be called when the driver is in the stopped state or the bus-off state.

Warning: The application must ensure that no tasks are blocked on TX/RX queues or alerts when this function is called.

Returns

- ESP_OK: Successfully uninstalled TWAI driver
- ESP_ERR_INVALID_STATE: Driver is not in stopped/bus-off state, or is not installed

esp_err_t **twai_driver_uninstall_v2** (*twai_handle_t* handle)

Uninstall the TWAI driver with a given handle.

Note: This is an advanced version of `twai_driver_uninstall` that can uninstall a TWAI driver with a given handle. Please refer to the documentation of `twai_driver_uninstall` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`

Returns

- ESP_OK: Successfully uninstalled TWAI driver
- ESP_ERR_INVALID_STATE: Driver is not in stopped/bus-off state, or is not installed

esp_err_t **twai_start** (void)

Start the TWAI driver.

This function starts the TWAI driver, putting the TWAI driver into the running state. This allows the TWAI driver to participate in TWAI bus activities such as transmitting/receiving messages. The TX and RX queue are reset in this function, clearing any messages that are unread or pending transmission. This function can only be called when the TWAI driver is in the stopped state.

Returns

- ESP_OK: TWAI driver is now running
- ESP_ERR_INVALID_STATE: Driver is not in stopped state, or is not installed

esp_err_t **twai_start_v2** (*twai_handle_t* handle)

Start the TWAI driver with a given handle.

Note: This is an advanced version of `twai_start` that can start a TWAI driver with a given handle. Please refer to the documentation of `twai_start` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`

Returns

- ESP_OK: TWAI driver is now running
- ESP_ERR_INVALID_STATE: Driver is not in stopped state, or is not installed

esp_err_t **twai_stop** (void)

Stop the TWAI driver.

This function stops the TWAI driver, preventing any further message from being transmitted or received until twai_start() is called. Any messages in the TX queue are cleared. Any messages in the RX queue should be read by the application after this function is called. This function can only be called when the TWAI driver is in the running state.

Warning: A message currently being transmitted/received on the TWAI bus will be ceased immediately. This may lead to other TWAI nodes interpreting the unfinished message as an error.

Returns

- ESP_OK: TWAI driver is now Stopped
- ESP_ERR_INVALID_STATE: Driver is not in running state, or is not installed

esp_err_t **twai_stop_v2** (*twai_handle_t* handle)

Stop the TWAI driver with a given handle.

Note: This is an advanced version of `twai_stop` that can stop a TWAI driver with a given handle. Please refer to the documentation of `twai_stop` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`

Returns

- ESP_OK: TWAI driver is now Stopped
- ESP_ERR_INVALID_STATE: Driver is not in running state, or is not installed

esp_err_t **twai_transmit** (const *twai_message_t* *message, TickType_t ticks_to_wait)

Transmit a TWAI message.

This function queues a TWAI message for transmission. Transmission will start immediately if no other messages are queued for transmission. If the TX queue is full, this function will block until more space becomes available or until it times out. If the TX queue is disabled (TX queue length = 0 in configuration), this function will return immediately if another message is undergoing transmission. This function can only be called when the TWAI driver is in the running state and cannot be called under Listen Only Mode.

Note: This function does not guarantee that the transmission is successful. The TX_SUCCESS/TX_FAILED alert can be enabled to alert the application upon the success/failure of a transmission.

Note: The TX_IDLE alert can be used to alert the application when no other messages are awaiting transmission.

Parameters

- **message** -- **[in]** Message to transmit
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block on the TX queue

Returns

- ESP_OK: Transmission successfully queued/initiated
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_TIMEOUT: Timed out waiting for space on TX queue
- ESP_FAIL: TX queue is disabled and another message is currently transmitting
- ESP_ERR_INVALID_STATE: TWAI driver is not in running state, or is not installed
- ESP_ERR_NOT_SUPPORTED: Listen Only Mode does not support transmissions

esp_err_t **twai_transmit_v2** (*twai_handle_t* handle, const *twai_message_t* *message, TickType_t ticks_to_wait)

Transmit a TWAI message via a given handle.

Note: This is an advanced version of `twai_transmit` that can transmit a TWAI message with a given handle. Please refer to the documentation of `twai_transmit` for more details.

Parameters

- **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
- **message** -- **[in]** Message to transmit
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block on the TX queue

Returns

- ESP_OK: Transmission successfully queued/initiated
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_TIMEOUT: Timed out waiting for space on TX queue
- ESP_FAIL: TX queue is disabled and another message is currently transmitting
- ESP_ERR_INVALID_STATE: TWAI driver is not in running state, or is not installed
- ESP_ERR_NOT_SUPPORTED: Listen Only Mode does not support transmissions

esp_err_t **twai_receive** (*twai_message_t* *message, TickType_t ticks_to_wait)

Receive a TWAI message.

This function receives a message from the RX queue. The flags field of the message structure will indicate the type of message received. This function will block if there are no messages in the RX queue

Warning: The flags field of the received message should be checked to determine if the received message contains any data bytes.

Parameters

- **message** -- **[out]** Received message
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block on RX queue

Returns

- ESP_OK: Message successfully received from RX queue
- ESP_ERR_TIMEOUT: Timed out waiting for message
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_receive_v2** (*twai_handle_t* handle, *twai_message_t* *message, TickType_t ticks_to_wait)

Receive a TWAI message via a given handle.

Note: This is an advanced version of `twai_receive` that can receive a TWAI message with a given handle. Please refer to the documentation of `twai_receive` for more details.

Parameters

- **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
- **message** -- **[out]** Received message
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block on RX queue

Returns

- ESP_OK: Message successfully received from RX queue
- ESP_ERR_TIMEOUT: Timed out waiting for message
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_read_alerts** (uint32_t *alerts, TickType_t ticks_to_wait)

Read TWAI driver alerts.

This function will read the alerts raised by the TWAI driver. If no alert has been issued when this function is called, this function will block until an alert occurs or until it timeouts.

Note: Multiple alerts can be raised simultaneously. The application should check for all alerts that have been enabled.

Parameters

- **alerts** -- **[out]** Bit field of raised alerts (see documentation for alert flags)
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block for alert

Returns

- ESP_OK: Alerts read
- ESP_ERR_TIMEOUT: Timed out waiting for alerts
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_read_alerts_v2** (*twai_handle_t* handle, uint32_t *alerts, TickType_t ticks_to_wait)

Read TWAI driver alerts with a given handle.

Note: This is an advanced version of `twai_read_alerts` that can read TWAI driver alerts with a given handle. Please refer to the documentation of `twai_read_alerts` for more details.

Parameters

- **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
- **alerts** -- **[out]** Bit field of raised alerts (see documentation for alert flags)
- **ticks_to_wait** -- **[in]** Number of FreeRTOS ticks to block for alert

Returns

- ESP_OK: Alerts read
- ESP_ERR_TIMEOUT: Timed out waiting for alerts
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_reconfigure_alerts** (uint32_t alerts_enabled, uint32_t *current_alerts)

Reconfigure which alerts are enabled.

This function reconfigures which alerts are enabled. If there are alerts which have not been read whilst reconfiguring, this function can read those alerts.

Parameters
- **alerts_enabled** -- **[in]** Bit field of alerts to enable (see documentation for alert flags)
- **current_alerts** -- **[out]** Bit field of currently raised alerts. Set to NULL if unused

Returns
- ESP_OK: Alerts reconfigured
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_reconfigure_alerts_v2** (*twai_handle_t* handle, uint32_t alerts_enabled, uint32_t *current_alerts)

Reconfigure which alerts are enabled, with a given handle.

Note: This is an advanced version of `twai_reconfigure_alerts` that can reconfigure which alerts are enabled with a given handle. Please refer to the documentation of `twai_reconfigure_alerts` for more details.

Parameters
- **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
- **alerts_enabled** -- **[in]** Bit field of alerts to enable (see documentation for alert flags)
- **current_alerts** -- **[out]** Bit field of currently raised alerts. Set to NULL if unused

Returns
- ESP_OK: Alerts reconfigured
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_initiate_recovery** (void)

Start the bus recovery process.

This function initiates the bus recovery process when the TWAI driver is in the bus-off state. Once initiated, the TWAI driver will enter the recovering state and wait for 128 occurrences of the bus-free signal on the TWAI bus before returning to the stopped state. This function will reset the TX queue, clearing any messages pending transmission.

Note: The BUS_RECOVERED alert can be enabled to alert the application when the bus recovery process completes.

Returns
- ESP_OK: Bus recovery started
- ESP_ERR_INVALID_STATE: TWAI driver is not in the bus-off state, or is not installed

esp_err_t **twai_initiate_recovery_v2** (*twai_handle_t* handle)

Start the bus recovery process with a given handle.

Note: This is an advanced version of `twai_initiate_recovery` that can start the bus recovery process with a given handle. Please refer to the documentation of `twai_initiate_recovery` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`

Returns
- ESP_OK: Bus recovery started
- ESP_ERR_INVALID_STATE: TWAI driver is not in the bus-off state, or is not installed

esp_err_t **twai_get_status_info** (*twai_status_info_t* *status_info)

Get current status information of the TWAI driver.

Parameters **status_info** -- **[out]** Status information
Returns
- ESP_OK: Status information retrieved
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_get_status_info_v2** (*twai_handle_t* handle, *twai_status_info_t* *status_info)

Get current status information of a given TWAI driver handle.

Note: This is an advanced version of `twai_get_status_info` that can get current status information of a given TWAI driver handle. Please refer to the documentation of `twai_get_status_info` for more details.

Parameters
- **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
- **status_info** -- **[out]** Status information

Returns
- ESP_OK: Status information retrieved
- ESP_ERR_INVALID_ARG: Arguments are invalid
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_clear_transmit_queue** (void)

Clear the transmit queue.

This function will clear the transmit queue of all messages.

Note: The transmit queue is automatically cleared when twai_stop() or twai_initiate_recovery() is called.

Returns
- ESP_OK: Transmit queue cleared
- ESP_ERR_INVALID_STATE: TWAI driver is not installed or TX queue is disabled

esp_err_t **twai_clear_transmit_queue_v2** (*twai_handle_t* handle)

Clear the transmit queue of a given TWAI driver handle.

Note: This is an advanced version of `twai_clear_transmit_queue` that can clear the transmit queue of a given TWAI driver handle. Please refer to the documentation of `twai_clear_transmit_queue` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`
Returns
- ESP_OK: Transmit queue cleared
- ESP_ERR_INVALID_STATE: TWAI driver is not installed or TX queue is disabled

esp_err_t **twai_clear_receive_queue** (void)

Clear the receive queue.

This function will clear the receive queue of all messages.

Note: The receive queue is automatically cleared when twai_start() is called.

Returns
- ESP_OK: Transmit queue cleared

- ESP_ERR_INVALID_STATE: TWAI driver is not installed

esp_err_t **twai_clear_receive_queue_v2** (*twai_handle_t* handle)

Clear the receive queue of a given TWAI driver handle.

Note: This is an advanced version of `twai_clear_receive_queue` that can clear the receive queue of a given TWAI driver handle. Please refer to the documentation of `twai_clear_receive_queue` for more details.

Parameters **handle** -- **[in]** TWAI driver handle returned by `twai_driver_install_v2`

Returns

- ESP_OK: Transmit queue cleared
- ESP_ERR_INVALID_STATE: TWAI driver is not installed

Structures

struct **twai_general_config_t**

Structure for general configuration of the TWAI driver.

Note: Macro initializers are available for this structure

Public Members

int **controller_id**

TWAI controller ID, index from 0. If you want to install TWAI driver with a non-zero controller_id, please use `twai_driver_install_v2`

twai_mode_t **mode**

Mode of TWAI controller

gpio_num_t **tx_io**

Transmit GPIO number

gpio_num_t **rx_io**

Receive GPIO number

gpio_num_t **clkout_io**

CLKOUT GPIO number (optional, set to -1 if unused)

gpio_num_t **bus_off_io**

Bus off indicator GPIO number (optional, set to -1 if unused)

uint32_t **tx_queue_len**

Number of messages TX queue can hold (set to 0 to disable TX Queue)

uint32_t **rx_queue_len**

Number of messages RX queue can hold

uint32_t **alerts_enabled**

Bit field of alerts to enable (see documentation)

uint32_t **clkout_divider**

CLKOUT divider. Can be 1 or any even number from 2 to 14 (optional, set to 0 if unused)

int **intr_flags**

Interrupt flags to set the priority of the driver's ISR. Note that to use the ESP_INTR_FLAG_IRAM, the CONFIG_TWAI_ISR_IN_IRAM option should be enabled first.

struct **twai_status_info_t**

Structure to store status information of TWAI driver.

Public Members

twai_state_t **state**

Current state of TWAI controller (Stopped/Running/Bus-Off/Recovery)

uint32_t **msgs_to_tx**

Number of messages queued for transmission or awaiting transmission completion

uint32_t **msgs_to_rx**

Number of messages in RX queue waiting to be read

uint32_t **tx_error_counter**

Current value of Transmit Error Counter

uint32_t **rx_error_counter**

Current value of Receive Error Counter

uint32_t **tx_failed_count**

Number of messages that failed transmissions

uint32_t **rx_missed_count**

Number of messages that were lost due to a full RX queue (or errata workaround if enabled)

uint32_t **rx_overrun_count**

Number of messages that were lost due to a RX FIFO overrun

uint32_t **arb_lost_count**

Number of instances arbitration was lost

uint32_t **bus_error_count**

Number of instances a bus error has occurred

Macros

TWAI_IO_UNUSED

Marks GPIO as unused in TWAI configuration

Type Definitions

typedef struct twai_obj_t ***twai_handle_t**

TWAI controller handle.

Enumerations

enum **twai_state_t**

TWAI driver states.

Values:

enumerator **TWAI_STATE_STOPPED**

Stopped state. The TWAI controller will not participate in any TWAI bus activities

enumerator **TWAI_STATE_RUNNING**

Running state. The TWAI controller can transmit and receive messages

enumerator **TWAI_STATE_BUS_OFF**

Bus-off state. The TWAI controller cannot participate in bus activities until it has recovered

enumerator **TWAI_STATE_RECOVERING**

Recovering state. The TWAI controller is undergoing bus recovery

2.6.28 Universal Asynchronous Receiver/Transmitter (UART)

Introduction

A Universal Asynchronous Receiver/Transmitter (UART) is a hardware feature that handles communication (i.e., timing requirements and data framing) using widely-adopted asynchronous serial communication interfaces, such as RS232, RS422, and RS485. A UART provides a widely adopted and cheap method to realize full-duplex or half-duplex data exchange among different devices.

The ESP32-S3 chip has 3 UART controllers (also referred to as port), each featuring an identical set of registers to simplify programming and for more flexibility.

Each UART controller is independently configurable with parameters such as baud rate, data bit length, bit ordering, number of stop bits, parity bit, etc. All the regular UART controllers are compatible with UART-enabled devices from various manufacturers and can also support Infrared Data Association (IrDA) protocols.

Functional Overview

The overview describes how to establish communication between an ESP32-S3 and other UART devices using the functions and data types of the UART driver. A typical programming workflow is broken down into the sections provided below:

1. *Set Communication Parameters* - Setting baud rate, data bits, stop bits, etc.
2. *Set Communication Pins* - Assigning pins for connection to a device
3. *Install Drivers* - Allocating ESP32-S3's resources for the UART driver
4. *Run UART Communication* - Sending/receiving data
5. *Use Interrupts* - Triggering interrupts on specific communication events
6. *Deleting a Driver* - Freeing allocated resources if a UART communication is no longer required

Steps 1 to 3 comprise the configuration stage. Step 4 is where the UART starts operating. Steps 5 and 6 are optional.

The UART driver's functions identify each of the UART controllers using *uart_port_t*. This identification is needed for all the following function calls.

Set Communication Parameters UART communication parameters can be configured all in a single step or individually in multiple steps.

Single Step Call the function *uart_param_config()* and pass to it a *uart_config_t* structure. The *uart_config_t* structure should contain all the required parameters. See the example below.

```
const uart_port_t uart_num = UART_NUM_2;
uart_config_t uart_config = {
    .baud_rate = 115200,
    .data_bits = UART_DATA_8_BITS,
    .parity = UART_PARITY_DISABLE,
    .stop_bits = UART_STOP_BITS_1,
    .flow_ctrl = UART_HW_FLOWCTRL_CTS_RTS,
    .rx_flow_ctrl_thresh = 122,
};
// Configure UART parameters
ESP_ERROR_CHECK(uart_param_config(uart_num, &uart_config));
```

For more information on how to configure the hardware flow control options, please refer to peripherals/uart/uart_echo.

Multiple Steps Configure specific parameters individually by calling a dedicated function from the table given below. These functions are also useful if re-configuring a single parameter.

Table 5: Functions for Configuring specific parameters individually

Parameter to Configure	Function
Baud rate	*uart_set_baudrate()*
Number of transmitted bits	*uart_set_word_length()* selected out of *uart_word_length_t*
Parity control	*uart_set_parity()* selected out of *uart_parity_t*
Number of stop bits	*uart_set_stop_bits()* selected out of *uart_stop_bits_t*
Hardware flow control mode	*uart_set_hw_flow_ctrl()* selected out of *uart_hw_flowcontrol_t*
Communication mode	*uart_set_mode()* selected out of *uart_mode_t*

Each of the above functions has a _get_ counterpart to check the currently set value. For example, to check the current baud rate value, call *uart_get_baudrate()*.

Set Communication Pins After setting communication parameters, configure the physical GPIO pins to which the other UART device will be connected. For this, call the function *uart_set_pin()* and specify the GPIO pin numbers to which the driver should route the TX, RX, RTS, and CTS signals. If you want to keep a currently allocated pin number for a specific signal, pass the macro *UART_PIN_NO_CHANGE*.

The same macro *UART_PIN_NO_CHANGE* should be specified for pins that will not be used.

```
// Set UART pins(TX: IO4, RX: IO5, RTS: IO18, CTS: IO19)
ESP_ERROR_CHECK(uart_set_pin(UART_NUM_2, 4, 5, 18, 19));
```

Install Drivers Once the communication pins are set, install the driver by calling *uart_driver_install()* and specify the following parameters:

- UART port number

- Size of TX ring buffer
- Size of RX ring buffer
- Pointer to store the event queue handle
- Event queue size
- Flags to allocate an interrupt

The function allocates the required internal resources for the UART driver.

```
// Setup UART buffered IO with event queue
const int uart_buffer_size = (1024 * 2);
QueueHandle_t uart_queue;
// Install UART driver using an event queue here
ESP_ERROR_CHECK(uart_driver_install(UART_NUM_2, uart_buffer_size, \
                                        uart_buffer_size, 10, &uart_queue, 0));
```

Once this step is complete, you can connect the external UART device and check the communication.

Run UART Communication Serial communication is controlled by each UART controller's finite state machine (FSM).

The process of sending data involves the following steps:

1. Write data into TX FIFO buffer
2. FSM serializes the data
3. FSM sends the data out

The process of receiving data is similar, but the steps are reversed:

1. FSM processes an incoming serial stream and parallelizes it
2. FSM writes the data into RX FIFO buffer
3. Read the data from RX FIFO buffer

Therefore, an application only writes and reads data from a specific buffer using *uart_write_bytes()* and *uart_read_bytes()* respectively, and the FSM does the rest.

Transmit Data After preparing the data for transmission, call the function *uart_write_bytes()* and pass the data buffer's address and data length to it. The function copies the data to the TX ring buffer (either immediately or after enough space is available), and then exit. When there is free space in the TX FIFO buffer, an interrupt service routine (ISR) moves the data from the TX ring buffer to the TX FIFO buffer in the background. The code below demonstrates the use of this function.

```
// Write data to UART.
char* test_str = "This is a test string.\n";
uart_write_bytes(uart_num, (const char*)test_str, strlen(test_str));
```

The function *uart_write_bytes_with_break()* is similar to *uart_write_bytes()* but adds a serial break signal at the end of the transmission. A 'serial break signal' means holding the TX line low for a period longer than one data frame.

```
// Write data to UART, end with a break signal.
uart_write_bytes_with_break(uart_num, "test break\n",strlen("test break\n"), 100);
```

Another function for writing data to the TX FIFO buffer is *uart_tx_chars()*. Unlike *uart_write_bytes()*, this function does not block until space is available. Instead, it writes all data which can immediately fit into the hardware TX FIFO, and then return the number of bytes that were written.

There is a 'companion' function *uart_wait_tx_done()* that monitors the status of the TX FIFO buffer and returns once it is empty.

```
// Wait for packet to be sent
const uart_port_t uart_num = UART_NUM_2;
ESP_ERROR_CHECK(uart_wait_tx_done(uart_num, 100)); // wait timeout is 100 RTOS␣
↪ticks (TickType_t)
```

Receive Data Once the data is received by the UART and saved in the RX FIFO buffer, it needs to be retrieved using the function *uart_read_bytes()*. Before reading data, you can check the number of bytes available in the RX FIFO buffer by calling *uart_get_buffered_data_len()*. An example of using these functions is given below.

```
// Read data from UART.
const uart_port_t uart_num = UART_NUM_2;
uint8_t data[128];
int length = 0;
ESP_ERROR_CHECK(uart_get_buffered_data_len(uart_num, (size_t*)&length));
length = uart_read_bytes(uart_num, data, length, 100);
```

If the data in the RX FIFO buffer is no longer needed, you can clear the buffer by calling *uart_flush()*.

Software Flow Control If the hardware flow control is disabled, you can manually set the RTS and DTR signal levels by using the functions *uart_set_rts()* and *uart_set_dtr()* respectively.

Communication Mode Selection The UART controller supports a number of communication modes. A mode can be selected using the function *uart_set_mode()*. Once a specific mode is selected, the UART driver handles the behavior of a connected UART device accordingly. As an example, it can control the RS485 driver chip using the RTS line to allow half-duplex RS485 communication.

```
// Setup UART in rs485 half duplex mode
ESP_ERROR_CHECK(uart_set_mode(uart_num, UART_MODE_RS485_HALF_DUPLEX));
```

Use Interrupts There are many interrupts that can be generated depending on specific UART states or detected errors. The full list of available interrupts is provided in *ESP32-S3 Technical Reference Manual* > *UART Controller (UART)* > *UART Interrupts* and *UHCI Interrupts* [PDF]. You can enable or disable specific interrupts by calling *uart_enable_intr_mask()* or *uart_disable_intr_mask()* respectively.

The UART driver provides a convenient way to handle specific interrupts by wrapping them into corresponding events. Events defined in *uart_event_type_t* can be reported to a user application using the FreeRTOS queue functionality.

To receive the events that have happened, call *uart_driver_install()* and get the event queue handle returned from the function. Please see the above *code snippet* as an example.

The processed events include the following:

- **FIFO overflow** (*UART_FIFO_OVF*): The RX FIFO can trigger an interrupt when it receives more data than the FIFO can store.
  - (Optional) Configure the full threshold of the FIFO space by entering it in the structure *uart_intr_config_t* and call *uart_intr_config()* to set the configuration. This can help the data stored in the RX FIFO can be processed timely in the driver to avoid FIFO overflow.
  - Enable the interrupts using the functions *uart_enable_rx_intr()*.
  - Disable these interrupts using the corresponding functions *uart_disable_rx_intr()*.

```
const uart_port_t uart_num = UART_NUM_2;
// Configure a UART interrupt threshold and timeout
uart_intr_config_t uart_intr = {
    .intr_enable_mask = UART_INTR_RXFIFO_FULL | UART_INTR_RXFIFO_TOUT,
    .rxfifo_full_thresh = 100,
```

(continues on next page)

(continued from previous page)

```
    .rx_timeout_thresh = 10,
};
ESP_ERROR_CHECK(uart_intr_config(uart_num, &uart_intr));

// Enable UART RX FIFO full threshold and timeout interrupts
ESP_ERROR_CHECK(uart_enable_rx_intr(uart_num));
```

- **Pattern detection** (*UART_PATTERN_DET*): An interrupt triggered on detecting a 'pattern' of the same character being received/sent repeatedly. It can be used, e.g., to detect a command string with a specific number of identical characters (the 'pattern') at the end. The following functions are available:
  - Configure and enable this interrupt using *uart_enable_pattern_det_baud_intr()*
  - Disable the interrupt using *uart_disable_pattern_det_intr()*

```
//Set UART pattern detect function
uart_enable_pattern_det_baud_intr(EX_UART_NUM, '+', PATTERN_CHR_NUM, 9, 0, 0);
```

- **Other events**: The UART driver can report other events such as data receiving (*UART_DATA*), ring buffer full (*UART_BUFFER_FULL*), detecting NULL after the stop bit (*UART_BREAK*), parity check error (*UART_PARITY_ERR*), and frame error (*UART_FRAME_ERR*).

The strings inside of brackets indicate corresponding event names. An example of how to handle various UART events can be found in peripherals/uart/uart_events.

Deleting a Driver If the communication established with *uart_driver_install()* is no longer required, the driver can be removed to free allocated resources by calling *uart_driver_delete()*.

Macros The API also defines several macros. For example, *UART_HW_FIFO_LEN* defines the length of hardware FIFO buffers; *UART_BITRATE_MAX* gives the maximum baud rate supported by the UART controllers, etc.

Overview of RS485 Specific Communication 0ptions

Note: The following section uses `[UART_REGISTER_NAME].[UART_FIELD_BIT]` to refer to UART register fields/bits. For more information on a specific option bit, see **ESP32-S3 Technical Reference Manual > UART Controller (UART) > Register Summary** [PDF]. Use the register name to navigate to the register description and then find the field/bit.

- `UART_RS485_CONF_REG.UART_RS485_EN`: setting this bit enables RS485 communication mode support.
- `UART_RS485_CONF_REG.UART_RS485TX_RX_EN`: if this bit is set, the transmitter's output signal loops back to the receiver's input signal.
- `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN`: if this bit is set, the transmitter will still be sending data if the receiver is busy (remove collisions automatically by hardware).

The ESP32-S3's RS485 UART hardware can detect signal collisions during transmission of a datagram and generate the interrupt `UART_RS485_CLASH_INT` if this interrupt is enabled. The term collision means that a transmitted datagram is not equal to the one received on the other end. Data collisions are usually associated with the presence of other active devices on the bus or might occur due to bus errors.

The collision detection feature allows handling collisions when their interrupts are activated and triggered. The interrupts `UART_RS485_FRM_ERR_INT` and `UART_RS485_PARITY_ERR_INT` can be used with the collision detection feature to control frame errors and parity bit errors accordingly in RS485 mode. This functionality is supported in the UART driver and can be used by selecting the *UART_MODE_RS485_APP_CTRL* mode (see the function *uart_set_mode()*).

The collision detection feature can work with circuit A and circuit C (see Section *Interface Connection Options*). In the case of using circuit A or B, the RTS pin connected to the DE pin of the bus driver should be controlled by the

user application. Use the function *uart_get_collision_flag()* to check if the collision detection flag has been raised.

The ESP32-S3 UART controllers themselves do not support half-duplex communication as they cannot provide automatic control of the RTS pin connected to the RE/DE input of RS485 bus driver. However, half-duplex communication can be achieved via software control of the RTS pin by the UART driver. This can be enabled by selecting the *UART_MODE_RS485_HALF_DUPLEX* mode when calling *uart_set_mode()*.

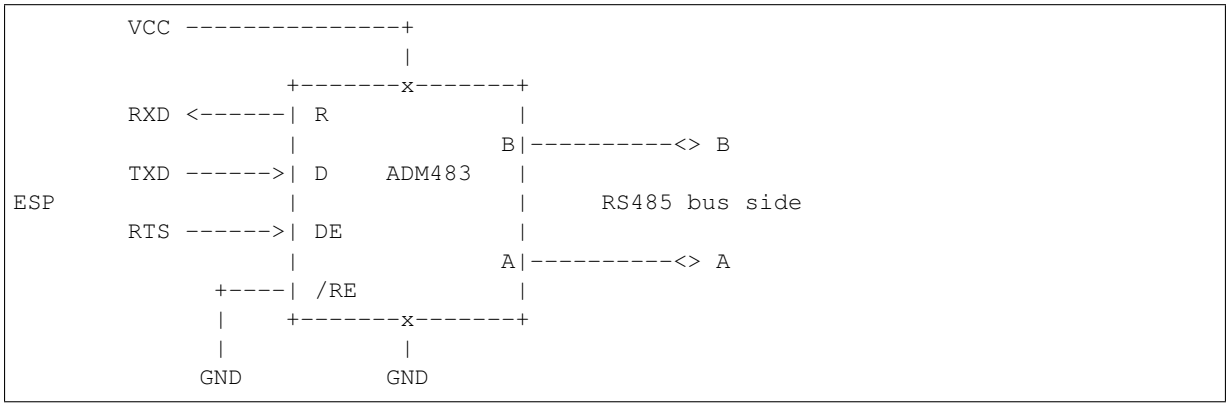
Once the host starts writing data to the TX FIFO buffer, the UART driver automatically asserts the RTS pin (logic 1); once the last bit of the data has been transmitted, the driver de-asserts the RTS pin (logic 0). To use this mode, the software would have to disable the hardware flow control function. This mode works with all the used circuits shown below.

Interface Connection Options This section provides example schematics to demonstrate the basic aspects of ESP32-S3's RS485 interface connection.

Note:

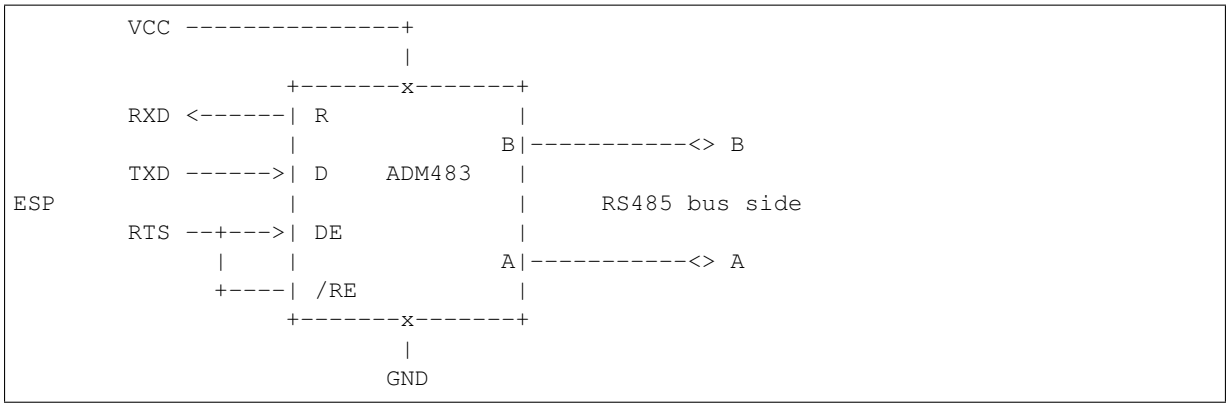
- The schematics below do **not** necessarily contain **all required elements**.
- The **analog devices** ADM483 & ADM2483 are examples of common RS485 transceivers and **can be replaced** with other similar transceivers.

Circuit A: Collision Detection Circuit



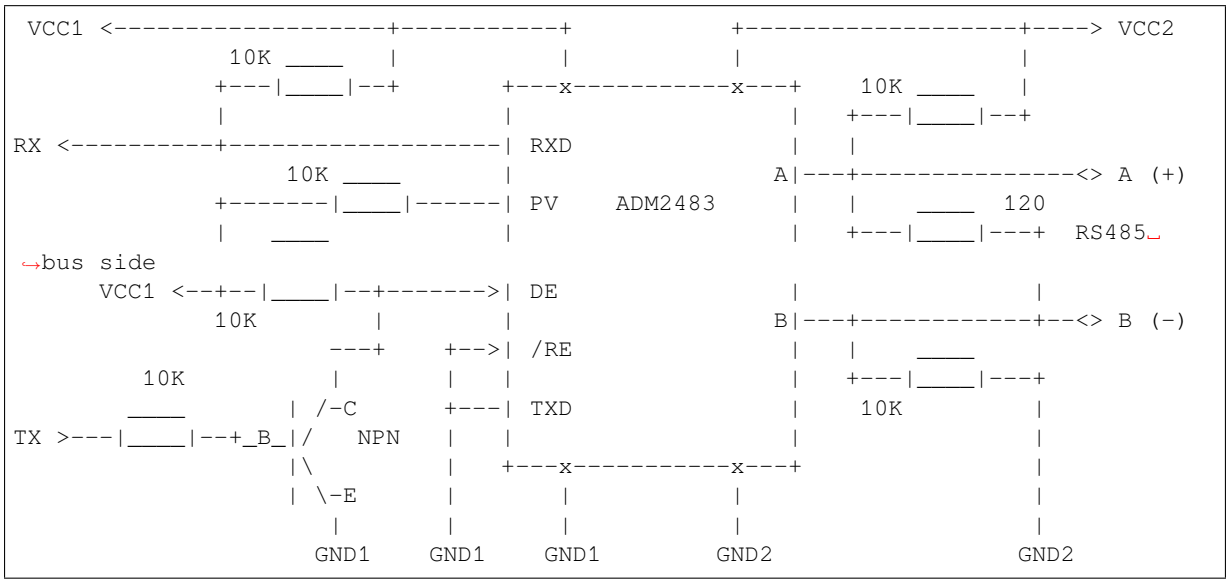
This circuit is preferable because it allows for collision detection and is quite simple at the same time. The receiver in the line driver is constantly enabled, which allows the UART to monitor the RS485 bus. Echo suppression is performed by the UART peripheral when the bit `UART_RS485_CONF_REG.UART_RS485TX_RX_EN` is enabled.

Circuit B: Manual Switching Transmitter/Receiver Without Collision Detection



This circuit does not allow for collision detection. It suppresses the null bytes that the hardware receives when the bit `UART_RS485_CONF_REG.UART_RS485TX_RX_EN` is set. The bit `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN` is not applicable in this case.

Circuit C: Auto Switching Transmitter/Receiver



This galvanically isolated circuit does not require RTS pin control by a software application or driver because it controls the transceiver direction automatically. However, it requires suppressing null bytes during transmission by setting `UART_RS485_CONF_REG.UART_RS485RXBY_TX_EN` to 1 and `UART_RS485_CONF_REG.UART_RS485TX_RX_EN` to 0. This setup can work in any RS485 UART mode or even in *`UART_MODE_UART`*.

Application Examples

The table below describes the code examples available in the directory peripherals/uart/.

Code Example	Description
peripherals/uart/uart_echo	Configuring UART settings, installing the UART driver, and reading/writing over the UART1 interface.
peripherals/uart/uart_events	Reporting various communication events, using pattern detection interrupts.
peripherals/uart/uart_async_rxtxtasks	Transmitting and receiving data in two separate FreeRTOS tasks over the same UART.
peripherals/uart/uart_select	Using synchronous I/O multiplexing for UART file descriptors.
peripherals/uart/uart_echo_rs485	Setting up UART driver to communicate over RS485 interface in half-duplex mode. This example is similar to peripherals/uart/uart_echo but allows communication through an RS485 interface chip connected to ESP32-S3 pins.
peripherals/uart/nmea0183_parser	Obtaining GPS information by parsing NMEA0183 statements received from GPS via the UART peripheral.

API Reference

Header File

- components/esp_driver_uart/include/driver/uart.h
- This header file can be included with:

```
#include "driver/uart.h"
```

- This header file is a part of the API provided by the `esp_driver_uart` component. To declare that your component depends on `esp_driver_uart`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_uart
```

or

```
PRIV_REQUIRES esp_driver_uart
```

Functions

esp_err_t **uart_driver_install**(*uart_port_t* uart_num, int rx_buffer_size, int tx_buffer_size, int queue_size, *QueueHandle_t* *uart_queue, int intr_alloc_flags)

Install UART driver and set the UART to the default configuration.

UART ISR handler will be attached to the same CPU core that this function is running on.

Note: Rx_buffer_size should be greater than UART_HW_FIFO_LEN(uart_num). Tx_buffer_size should be either zero or greater than UART_HW_FIFO_LEN(uart_num).

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **rx_buffer_size** -- UART RX ring buffer size.
- **tx_buffer_size** -- UART TX ring buffer size. If set to zero, driver will not use TX buffer, TX function will block task until all data have been sent out.
- **queue_size** -- UART event queue size/depth.
- **uart_queue** -- UART event queue handle (out param). On success, a new queue handle is written here to provide access to UART events. If set to NULL, driver will not use an event queue.
- **intr_alloc_flags** -- Flags used to allocate the interrupt. One or multiple (ORred) ESP_INTR_FLAG_* values. See esp_intr_alloc.h for more info. Do not set ESP_INTR_FLAG_IRAM here (the driver's ISR handler is not located in IRAM)

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_driver_delete**(*uart_port_t* uart_num)

Uninstall UART driver.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

bool **uart_is_driver_installed**(*uart_port_t* uart_num)

Checks whether the driver is installed or not.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- true driver is installed
- false driver is not installed

esp_err_t **uart_set_word_length**(*uart_port_t* uart_num, *uart_word_length_t* data_bit)

Set UART data bits.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **data_bit** -- UART data bits

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_get_word_length**(*uart_port_t* uart_num, *uart_word_length_t* *data_bit)

Get the UART data bit configuration.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **data_bit** -- Pointer to accept value of UART data bits.

Returns

- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*data_bit)

esp_err_t **uart_set_stop_bits**(*uart_port_t* uart_num, *uart_stop_bits_t* stop_bits)

Set UART stop bits.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **stop_bits** -- UART stop bits

Returns

- ESP_OK Success
- ESP_FAIL Fail

esp_err_t **uart_get_stop_bits**(*uart_port_t* uart_num, *uart_stop_bits_t* *stop_bits)

Get the UART stop bit configuration.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **stop_bits** -- Pointer to accept value of UART stop bits.

Returns

- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*stop_bit)

esp_err_t **uart_set_parity**(*uart_port_t* uart_num, *uart_parity_t* parity_mode)

Set UART parity mode.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **parity_mode** -- the enum of uart parity configuration

Returns

- ESP_FAIL Parameter error
- ESP_OK Success

esp_err_t **uart_get_parity**(*uart_port_t* uart_num, *uart_parity_t* *parity_mode)

Get the UART parity mode configuration.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **parity_mode** -- Pointer to accept value of UART parity mode.

Returns

- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*parity_mode)

esp_err_t **uart_get_sclk_freq**(*uart_sclk_t* sclk, uint32_t *out_freq_hz)

Get the frequency of a clock source for the HP UART port.

Parameters

- **sclk** -- Clock source
- **out_freq_hz** -- **[out]** Output of frequency, in Hz

Returns

- ESP_ERR_INVALID_ARG: if the clock source is not supported
- otherwise ESP_OK

esp_err_t **uart_set_baudrate**(*uart_port_t* uart_num, uint32_t baudrate)

Set UART baud rate.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **baudrate** -- UART baud rate.

Returns
- ESP_FAIL Parameter error
- ESP_OK Success

esp_err_t **uart_get_baudrate**(*uart_port_t* uart_num, uint32_t *baudrate)

Get the UART baud rate configuration.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **baudrate** -- Pointer to accept value of UART baud rate

Returns
- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*baudrate)

esp_err_t **uart_set_line_inverse**(*uart_port_t* uart_num, uint32_t inverse_mask)

Set UART line inverse mode.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **inverse_mask** -- Choose the wires that need to be inverted. Using the ORred mask of `uart_signal_inv_t`

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_hw_flow_ctrl**(*uart_port_t* uart_num, *uart_hw_flowcontrol_t* flow_ctrl, uint8_t rx_thresh)

Set hardware flow control.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **flow_ctrl** -- Hardware flow control mode
- **rx_thresh** -- Threshold of Hardware RX flow control (0 ~ UART_HW_FIFO_LEN(uart_num)). Only when UART_HW_FLOWCTRL_RTS is set, will the rx_thresh value be set.

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_sw_flow_ctrl**(*uart_port_t* uart_num, bool enable, uint8_t rx_thresh_xon, uint8_t rx_thresh_xoff)

Set software flow control.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1)
- **enable** -- switch on or off
- **rx_thresh_xon** -- low water mark
- **rx_thresh_xoff** -- high water mark

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_get_hw_flow_ctrl** (*uart_port_t* uart_num, *uart_hw_flowcontrol_t* *flow_ctrl)

Get the UART hardware flow control configuration.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **flow_ctrl** -- Option for different flow control mode.

Returns
- ESP_FAIL Parameter error
- ESP_OK Success, result will be put in (*flow_ctrl)

esp_err_t **uart_clear_intr_status** (*uart_port_t* uart_num, uint32_t clr_mask)

Clear UART interrupt status.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **clr_mask** -- Bit mask of the interrupt status to be cleared.

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_enable_intr_mask** (*uart_port_t* uart_num, uint32_t enable_mask)

Set UART interrupt enable.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **enable_mask** -- Bit mask of the enable bits.

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_disable_intr_mask** (*uart_port_t* uart_num, uint32_t disable_mask)

Clear UART interrupt enable bits.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **disable_mask** -- Bit mask of the disable bits.

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_enable_rx_intr** (*uart_port_t* uart_num)

Enable UART RX interrupt (RX_FULL & RX_TIMEOUT INTERRUPT)

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_disable_rx_intr** (*uart_port_t* uart_num)

Disable UART RX interrupt (RX_FULL & RX_TIMEOUT INTERRUPT)

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns
- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_disable_tx_intr** (*uart_port_t* uart_num)

Disable UART TX interrupt (TX_FULL & TX_TIMEOUT INTERRUPT)

Parameters **uart_num** -- UART port number

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_enable_tx_intr** (*uart_port_t* uart_num, int enable, int thresh)

Enable UART TX interrupt (TX_FULL & TX_TIMEOUT INTERRUPT)

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **enable** -- 1: enable; 0: disable
- **thresh** -- Threshold of TX interrupt, 0 ~ UART_HW_FIFO_LEN(uart_num)

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_pin** (*uart_port_t* uart_num, int tx_io_num, int rx_io_num, int rts_io_num, int cts_io_num)

Assign signals of a UART peripheral to GPIO pins.

Note: If the GPIO number configured for a UART signal matches one of the IOMUX signals for that GPIO, the signal will be connected directly via the IOMUX. Otherwise the GPIO and signal will be connected via the GPIO Matrix. For example, if on an ESP32 the call `uart_set_pin(0, 1, 3, -1, -1)` is performed, as GPIO1 is UART0's default TX pin and GPIO3 is UART0's default RX pin, both will be connected to respectively U0TXD and U0RXD through the IOMUX, totally bypassing the GPIO matrix. The check is performed on a per-pin basis. Thus, it is possible to have RX pin binded to a GPIO through the GPIO matrix, whereas TX is binded to its GPIO through the IOMUX.

Note: Internal signal can be output to multiple GPIO pads. Only one GPIO pad can connect with input signal.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **tx_io_num** -- UART TX pin GPIO number.
- **rx_io_num** -- UART RX pin GPIO number.
- **rts_io_num** -- UART RTS pin GPIO number.
- **cts_io_num** -- UART CTS pin GPIO number.

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_rts** (*uart_port_t* uart_num, int level)

Manually set the UART RTS pin level.

Note: UART must be configured with hardware flow control disabled.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **level** -- 1: RTS output low (active); 0: RTS output high (block)

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_dtr** (*uart_port_t* uart_num, int level)

Manually set the UART DTR pin level.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

- **level** -- 1: DTR output low; 0: DTR output high

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_set_tx_idle_num**(*uart_port_t* uart_num, uint16_t idle_num)

Set UART idle interval after tx FIFO is empty.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **idle_num** -- idle interval after tx FIFO is empty(unit: the time it takes to send one bit under current baudrate)

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_param_config**(*uart_port_t* uart_num, const *uart_config_t* *uart_config)

Set UART configuration parameters.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **uart_config** -- UART parameter settings

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_intr_config**(*uart_port_t* uart_num, const *uart_intr_config_t* *intr_conf)

Configure UART interrupts.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **intr_conf** -- UART interrupt settings

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_wait_tx_done**(*uart_port_t* uart_num, TickType_t ticks_to_wait)

Wait until UART TX FIFO is empty.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **ticks_to_wait** -- Timeout, count in RTOS ticks

Returns

- ESP_OK Success
- ESP_FAIL Parameter error
- ESP_ERR_TIMEOUT Timeout

int **uart_tx_chars**(*uart_port_t* uart_num, const char *buffer, uint32_t len)

Send data to the UART port from a given buffer and length.

This function will not wait for enough space in TX FIFO. It will just fill the available TX FIFO and return when the FIFO is full.

Note: This function should only be used when UART TX buffer is not enabled.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **buffer** -- data buffer address
- **len** -- data length to send

Returns

- (-1) Parameter error
- OTHERS (>=0) The number of bytes pushed to the TX FIFO

int **uart_write_bytes**(*uart_port_t* uart_num, const void *src, size_t size)

Send data to the UART port from a given buffer and length,.

If the UART driver's parameter 'tx_buffer_size' is set to zero: This function will not return until all the data have been sent out, or at least pushed into TX FIFO.

Otherwise, if the 'tx_buffer_size' > 0, this function will return after copying all the data to tx ring buffer, UART ISR will then move data from the ring buffer to TX FIFO gradually.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **src** -- data buffer address
- **size** -- data length to send

Returns
- (-1) Parameter error
- OTHERS (>=0) The number of bytes pushed to the TX FIFO

int **uart_write_bytes_with_break**(*uart_port_t* uart_num, const void *src, size_t size, int brk_len)

Send data to the UART port from a given buffer and length,.

If the UART driver's parameter 'tx_buffer_size' is set to zero: This function will not return until all the data and the break signal have been sent out. After all data is sent out, send a break signal.

Otherwise, if the 'tx_buffer_size' > 0, this function will return after copying all the data to tx ring buffer, UART ISR will then move data from the ring buffer to TX FIFO gradually. After all data sent out, send a break signal.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **src** -- data buffer address
- **size** -- data length to send
- **brk_len** -- break signal duration(unit: the time it takes to send one bit at current baudrate)

Returns
- (-1) Parameter error
- OTHERS (>=0) The number of bytes pushed to the TX FIFO

int **uart_read_bytes**(*uart_port_t* uart_num, void *buf, uint32_t length, TickType_t ticks_to_wait)

UART read bytes from UART buffer.

Parameters
- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **buf** -- pointer to the buffer.
- **length** -- data length
- **ticks_to_wait** -- sTimeout, count in RTOS ticks

Returns
- (-1) Error
- OTHERS (>=0) The number of bytes read from UART buffer

esp_err_t **uart_flush**(*uart_port_t* uart_num)

Alias of uart_flush_input. UART ring buffer flush. This will discard all data in the UART RX buffer.

Note: Instead of waiting the data sent out, this function will clear UART rx buffer. In order to send all the data in tx FIFO, we can use uart_wait_tx_done function.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns
- ESP_OK Success

- ESP_FAIL Parameter error

esp_err_t **uart_flush_input**(*uart_port_t* uart_num)

Clear input buffer, discard all the data is in the ring-buffer.

Note: In order to send all the data in tx FIFO, we can use uart_wait_tx_done function.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_get_buffered_data_len**(*uart_port_t* uart_num, size_t *size)

UART get RX ring buffer cached data length.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **size** -- Pointer of size_t to accept cached data length

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_get_tx_buffer_free_size**(*uart_port_t* uart_num, size_t *size)

UART get TX ring buffer free space size.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **size** -- Pointer of size_t to accept the free space size

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **uart_disable_pattern_det_intr**(*uart_port_t* uart_num)

UART disable pattern detect function. Designed for applications like 'AT commands'. When the hardware detects a series of one same character, the interrupt will be triggered.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

esp_err_t **uart_enable_pattern_det_baud_intr**(*uart_port_t* uart_num, char pattern_chr, uint8_t chr_num, int chr_tout, int post_idle, int pre_idle)

UART enable pattern detect function. Designed for applications like 'AT commands'. When the hardware detect a series of one same character, the interrupt will be triggered.

Parameters

- **uart_num** -- UART port number.
- **pattern_chr** -- character of the pattern.
- **chr_num** -- number of the character, 8bit value.
- **chr_tout** -- timeout of the interval between each pattern characters, 16bit value, unit is the baud-rate cycle you configured. When the duration is more than this value, it will not take this data as at_cmd char.
- **post_idle** -- idle time after the last pattern character, 16bit value, unit is the baud-rate cycle you configured. When the duration is less than this value, it will not take the previous data as the last at_cmd char

- **pre_idle** -- idle time before the first pattern character, 16bit value, unit is the baud-rate cycle you configured. When the duration is less than this value, it will not take this data as the first at_cmd char.

Returns

- ESP_OK Success
- ESP_FAIL Parameter error

int **uart_pattern_pop_pos**(*uart_port_t* uart_num)

Return the nearest detected pattern position in buffer. The positions of the detected pattern are saved in a queue, this function will dequeue the first pattern position and move the pointer to next pattern position.

The following APIs will modify the pattern position info: uart_flush_input, uart_read_bytes, uart_driver_delete, uart_pop_pattern_pos It is the application's responsibility to ensure atomic access to the pattern queue and the rx data buffer when using pattern detect feature.

Note: If the RX buffer is full and flow control is not enabled, the detected pattern may not be found in the rx buffer due to overflow.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- (-1) No pattern found for current index or parameter error
- others the pattern position in rx buffer.

int **uart_pattern_get_pos**(*uart_port_t* uart_num)

Return the nearest detected pattern position in buffer. The positions of the detected pattern are saved in a queue, This function do nothing to the queue.

The following APIs will modify the pattern position info: uart_flush_input, uart_read_bytes, uart_driver_delete, uart_pop_pattern_pos It is the application's responsibility to ensure atomic access to the pattern queue and the rx data buffer when using pattern detect feature.

Note: If the RX buffer is full and flow control is not enabled, the detected pattern may not be found in the rx buffer due to overflow.

Parameters **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).

Returns

- (-1) No pattern found for current index or parameter error
- others the pattern position in rx buffer.

esp_err_t **uart_pattern_queue_reset**(*uart_port_t* uart_num, int queue_length)

Allocate a new memory with the given length to save record the detected pattern position in rx buffer.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1).
- **queue_length** -- Max queue length for the detected pattern. If the queue length is not large enough, some pattern positions might be lost. Set this value to the maximum number of patterns that could be saved in data buffer at the same time.

Returns

- ESP_ERR_NO_MEM No enough memory
- ESP_ERR_INVALID_STATE Driver not installed
- ESP_FAIL Parameter error

- ESP_OK Success

esp_err_t **uart_set_mode**(*uart_port_t* uart_num, *uart_mode_t* mode)

UART set communication mode.

Note: This function must be executed after uart_driver_install(), when the driver object is initialized.

Parameters

- **uart_num** -- Uart number to configure, the max port number is (UART_NUM_MAX -1).
- **mode** -- UART UART mode to set

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **uart_set_rx_full_threshold**(*uart_port_t* uart_num, int threshold)

Set uart threshold value for RX fifo full.

Note: If application is using higher baudrate and it is observed that bytes in hardware RX fifo are overwritten then this threshold can be reduced

---

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1)
- **threshold** -- Threshold value above which RX fifo full interrupt is generated

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Driver is not installed

esp_err_t **uart_set_tx_empty_threshold**(*uart_port_t* uart_num, int threshold)

Set uart threshold values for TX fifo empty.

Parameters

- **uart_num** -- UART port number, the max port number is (UART_NUM_MAX -1)
- **threshold** -- Threshold value below which TX fifo empty interrupt is generated

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Driver is not installed

esp_err_t **uart_set_rx_timeout**(*uart_port_t* uart_num, const uint8_t tout_thresh)

UART set threshold timeout for TOUT feature.

Parameters

- **uart_num** -- Uart number to configure, the max port number is (UART_NUM_MAX -1).
- **tout_thresh** -- This parameter defines timeout threshold in uart symbol periods. The maximum value of threshold is 126. tout_thresh = 1, defines TOUT interrupt timeout equal to transmission time of one symbol (~11 bit) on current baudrate. If the time is expired the UART_RXFIFO_TOUT_INT interrupt is triggered. If tout_thresh == 0, the TOUT feature is disabled.

Returns

- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error
- ESP_ERR_INVALID_STATE Driver is not installed

esp_err_t **uart_get_collision_flag**(*uart_port_t* uart_num, bool *collision_flag)

Returns collision detection flag for RS485 mode Function returns the collision detection flag into variable pointed by collision_flag. *collision_flag = true, if collision detected else it is equal to false. This function should be executed when actual transmission is completed (after uart_write_bytes()).

Parameters
- **uart_num** -- Uart number to configure the max port number is (UART_NUM_MAX -1).
- **collision_flag** -- Pointer to variable of type bool to return collision flag.

Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

esp_err_t **uart_set_wakeup_threshold**(*uart_port_t* uart_num, int wakeup_threshold)

Set the number of RX pin signal edges for light sleep wakeup.

UART can be used to wake up the system from light sleep. This feature works by counting the number of positive edges on RX pin and comparing the count to the threshold. When the count exceeds the threshold, system is woken up from light sleep. This function allows setting the threshold value.

Stop bit and parity bits (if enabled) also contribute to the number of edges. For example, letter 'a' with ASCII code 97 is encoded as 0100001101 on the wire (with 8n1 configuration), start and stop bits included. This sequence has 3 positive edges (transitions from 0 to 1). Therefore, to wake up the system when 'a' is sent, set wakeup_threshold=3.

The character that triggers wakeup is not received by UART (i.e. it can not be obtained from UART FIFO). Depending on the baud rate, a few characters after that will also not be received. Note that when the chip enters and exits light sleep mode, APB frequency will be changing. To ensure that UART has correct Baud rate all the time, it is necessary to select a source clock which has a fixed frequency and remains active during sleep. For the supported clock sources of the chips, please refer to `uart_sclk_t` or `soc_periph_uart_clk_src_legacy_t`

Note: in ESP32, the wakeup signal can only be input via IO_MUX (i.e. GPIO3 should be configured as function_1 to wake up UART0, GPIO9 should be configured as function_5 to wake up UART1), UART2 does not support light sleep wakeup feature.

Parameters
- **uart_num** -- UART number, the max port number is (UART_NUM_MAX -1).
- **wakeup_threshold** -- number of RX edges for light sleep wakeup, value is 3 .. 0x3ff.

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if uart_num is incorrect or wakeup_threshold is outside of [3, 0x3ff] range.

esp_err_t **uart_get_wakeup_threshold**(*uart_port_t* uart_num, int *out_wakeup_threshold)

Get the number of RX pin signal edges for light sleep wakeup.

See description of uart_set_wakeup_threshold for the explanation of UART wakeup feature.

Parameters
- **uart_num** -- UART number, the max port number is (UART_NUM_MAX -1).
- **out_wakeup_threshold** -- **[out]** output, set to the current value of wakeup threshold for the given UART.

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if out_wakeup_threshold is NULL

esp_err_t **uart_wait_tx_idle_polling**(*uart_port_t* uart_num)

Wait until UART tx memory empty and the last char send ok (polling mode).

-

  Returns
  - ESP_OK on success
  - ESP_ERR_INVALID_ARG Parameter error
  - ESP_FAIL Driver not installed

  Parameters **uart_num** -- UART number

esp_err_t **uart_set_loop_back** (*uart_port_t* uart_num, bool loop_back_en)

Configure TX signal loop back to RX module, just for the test usage.

-

  Returns
  - ESP_OK on success
  - ESP_ERR_INVALID_ARG Parameter error
  - ESP_FAIL Driver not installed

  Parameters
  - **uart_num** -- UART number
  - **loop_back_en** -- Set true to enable the loop back function, else set it false.

void **uart_set_always_rx_timeout** (*uart_port_t* uart_num, bool always_rx_timeout_en)

Configure behavior of UART RX timeout interrupt.

When always_rx_timeout is true, timeout interrupt is triggered even if FIFO is full. This function can cause extra timeout interrupts triggered only to send the timeout event. Call this function only if you want to ensure timeout interrupt will always happen after a byte stream.

Parameters
- **uart_num** -- UART number
- **always_rx_timeout_en** -- Set to false enable the default behavior of timeout interrupt, set it to true to always trigger timeout interrupt.

Structures

struct **uart_config_t**

UART configuration parameters for uart_param_config function.

Public Members

int **baud_rate**

UART baud rate

uart_word_length_t **data_bits**

UART byte size

uart_parity_t **parity**

UART parity mode

uart_stop_bits_t **stop_bits**

UART stop bits

uart_hw_flowcontrol_t **flow_ctrl**

UART HW flow control mode (cts/rts)

uint8_t **rx_flow_ctrl_thresh**

UART HW RTS threshold

uart_sclk_t **source_clk**

UART source clock selection

uint32_t **backup_before_sleep**

If set, the driver will backup/restore the HP UART registers before entering/after exiting sleep mode. By this approach, the system can power off HP UART's power domain. This can save power, but at the expense of more RAM being consumed

struct *uart_config_t*::[anonymous] **flags**

Configuration flags

struct **uart_intr_config_t**

UART interrupt configuration parameters for uart_intr_config function.

Public Members

uint32_t **intr_enable_mask**

UART interrupt enable mask, choose from UART_XXXX_INT_ENA_M under UART_INT_ENA_REG(i), connect with bit-or operator

uint8_t **rx_timeout_thresh**

UART timeout interrupt threshold (unit: time of sending one byte)

uint8_t **txfifo_empty_intr_thresh**

UART TX empty interrupt threshold.

uint8_t **rxfifo_full_thresh**

UART RX full interrupt threshold.

struct **uart_event_t**

Event structure used in UART event queue.

Public Members

uart_event_type_t **type**

UART event type

size_t **size**

UART data size for UART_DATA event

bool **timeout_flag**

UART data read timeout flag for UART_DATA event (no new data received during configured RX TOUT) If the event is caused by FIFO-full interrupt, then there will be no event with the timeout flag before the next byte coming.

Macros

UART_PIN_NO_CHANGE

UART_FIFO_LEN

Length of the HP UART HW FIFO.

UART_HW_FIFO_LEN(uart_num)

Length of the UART HW FIFO.

UART_BITRATE_MAX

Maximum configurable bitrate.

Type Definitions

typedef *intr_handle_t* **uart_isr_handle_t**

Enumerations

enum **uart_event_type_t**

UART event types used in the ring buffer.

Values:

enumerator **UART_DATA**

UART data event

enumerator **UART_BREAK**

UART break event

enumerator **UART_BUFFER_FULL**

UART RX buffer full event

enumerator **UART_FIFO_OVF**

UART FIFO overflow event

enumerator **UART_FRAME_ERR**

UART RX frame error event

enumerator **UART_PARITY_ERR**

UART RX parity event

enumerator **UART_DATA_BREAK**

UART TX data and break event

enumerator **UART_PATTERN_DET**

UART pattern detected

enumerator **UART_WAKEUP**

UART wakeup event

enumerator **UART_EVENT_MAX**

UART event max index

Header File

- components/hal/include/hal/uart_types.h
- This header file can be included with:

```
#include "hal/uart_types.h"
```

Structures

struct **uart_at_cmd_t**

UART AT cmd char configuration parameters Note that this function may different on different chip. Please refer to the TRM at confirguration.

Public Members

uint8_t **cmd_char**

UART AT cmd char

uint8_t **char_num**

AT cmd char repeat number

uint32_t **gap_tout**

gap time(in baud-rate) between AT cmd char

uint32_t **pre_idle**

the idle time(in baud-rate) between the non AT char and first AT char

uint32_t **post_idle**

the idle time(in baud-rate) between the last AT char and the none AT char

struct **uart_sw_flowctrl_t**

UART software flow control configuration parameters.

Public Members

uint8_t **xon_char**

Xon flow control char

uint8_t **xoff_char**

Xoff flow control char

uint8_t **xon_thrd**

If the software flow control is enabled and the data amount in rxfifo is less than xon_thrd, an xon_char will be sent

uint8_t **xoff_thrd**

If the software flow control is enabled and the data amount in rxfifo is more than xoff_thrd, an xoff_char will be sent

Type Definitions

typedef *soc_periph_uart_clk_src_legacy_t* **uart_sclk_t**

UART source clock.

Enumerations

enum **uart_port_t**

UART port number, can be UART_NUM_0 ~ (UART_NUM_MAX -1).

Values:

enumerator **UART_NUM_0**

UART port 0

enumerator **UART_NUM_1**

UART port 1

enumerator **UART_NUM_2**

UART port 2

enumerator **UART_NUM_MAX**

UART port max

enum **uart_mode_t**

UART mode selection.

Values:

enumerator **UART_MODE_UART**

mode: regular UART mode

enumerator **UART_MODE_RS485_HALF_DUPLEX**

mode: half duplex RS485 UART mode control by RTS pin

enumerator **UART_MODE_IRDA**

mode: IRDA UART mode

enumerator **UART_MODE_RS485_COLLISION_DETECT**

mode: RS485 collision detection UART mode (used for test purposes)

enumerator **UART_MODE_RS485_APP_CTRL**

mode: application control RS485 UART mode (used for test purposes)

enum **uart_word_length_t**

UART word length constants.

Values:

enumerator **UART_DATA_5_BITS**

word length: 5bits

enumerator **UART_DATA_6_BITS**

word length: 6bits

enumerator **UART_DATA_7_BITS**

word length: 7bits

enumerator **UART_DATA_8_BITS**

word length: 8bits

enumerator **UART_DATA_BITS_MAX**

enum **uart_stop_bits_t**

UART stop bits number.

Values:

enumerator **UART_STOP_BITS_1**

stop bit: 1bit

enumerator **UART_STOP_BITS_1_5**

stop bit: 1.5bits

enumerator **UART_STOP_BITS_2**

stop bit: 2bits

enumerator **UART_STOP_BITS_MAX**

enum **uart_parity_t**

UART parity constants.

Values:

enumerator **UART_PARITY_DISABLE**

Disable UART parity

enumerator **UART_PARITY_EVEN**

Enable UART even parity

enumerator **UART_PARITY_ODD**

Enable UART odd parity

enum **uart_hw_flowcontrol_t**

UART hardware flow control modes.

Values:

enumerator **UART_HW_FLOWCTRL_DISABLE**

disable hardware flow control

enumerator **UART_HW_FLOWCTRL_RTS**

enable RX hardware flow control (rts)

enumerator **UART_HW_FLOWCTRL_CTS**

enable TX hardware flow control (cts)

enumerator **UART_HW_FLOWCTRL_CTS_RTS**

enable hardware flow control

enumerator **UART_HW_FLOWCTRL_MAX**

enum **uart_signal_inv_t**

UART signal bit map.

Values:

enumerator **UART_SIGNAL_INV_DISABLE**

Disable UART signal inverse

enumerator **UART_SIGNAL_IRDA_TX_INV**

inverse the UART irda_tx signal

enumerator **UART_SIGNAL_IRDA_RX_INV**

inverse the UART irda_rx signal

enumerator **UART_SIGNAL_RXD_INV**

inverse the UART rxd signal

enumerator **UART_SIGNAL_CTS_INV**

inverse the UART cts signal

enumerator **UART_SIGNAL_DSR_INV**

inverse the UART dsr signal

enumerator **UART_SIGNAL_TXD_INV**

inverse the UART txd signal

enumerator **UART_SIGNAL_RTS_INV**

inverse the UART rts signal

enumerator **UART_SIGNAL_DTR_INV**

inverse the UART dtr signal

GPIO Lookup Macros The UART peripherals have dedicated IO_MUX pins to which they are connected directly. However, signals can also be routed to other pins using the less direct GPIO matrix. To use direct routes, you need to know which pin is a dedicated IO_MUX pin for a UART channel. GPIO Lookup Macros simplify the process of finding and assigning IO_MUX pins. You choose a macro based on either the IO_MUX pin number, or a required UART channel name, and the macro returns the matching counterpart for you. See some examples below.

Note: These macros are useful if you need very high UART baud rates (over 40 MHz), which means you will have to use IO_MUX pins only. In other cases, these macros can be ignored, and you can use the GPIO Matrix as it allows you to configure any GPIO pin for any UART function.

---

1. `UART_NUM_2_TXD_DIRECT_GPIO_NUM` returns the IO_MUX pin number of UART channel 2 TXD pin (pin 17)
2. *`UART_GPIO19_DIRECT_CHANNEL`* returns the UART number of GPIO 19 when connected to the UART peripheral via IO_MUX (this is UART_NUM_0)
3. `UART_CTS_GPIO19_DIRECT_CHANNEL` returns the UART number of GPIO 19 when used as the UART CTS pin via IO_MUX (this is UART_NUM_0). It is similar to the above macro but specifies the pin function which is also part of the IO_MUX assignment.

Header File

- components/soc/esp32s3/include/soc/uart_channel.h
- This header file can be included with:

```
#include "soc/uart_channel.h"
```

Macros

UART_GPIO43_DIRECT_CHANNEL

UART_NUM_0_TXD_DIRECT_GPIO_NUM

UART_GPIO44_DIRECT_CHANNEL

UART_NUM_0_RXD_DIRECT_GPIO_NUM

UART_GPIO16_DIRECT_CHANNEL

UART_NUM_0_CTS_DIRECT_GPIO_NUM

UART_GPIO15_DIRECT_CHANNEL

UART_NUM_0_RTS_DIRECT_GPIO_NUM

UART_TXD_GPIO43_DIRECT_CHANNEL

UART_RXD_GPIO44_DIRECT_CHANNEL

UART_CTS_GPIO16_DIRECT_CHANNEL

UART_RTS_GPIO15_DIRECT_CHANNEL

UART_GPIO17_DIRECT_CHANNEL

UART_NUM_1_TXD_DIRECT_GPIO_NUM

UART_GPIO18_DIRECT_CHANNEL

UART_NUM_1_RXD_DIRECT_GPIO_NUM

UART_GPIO20_DIRECT_CHANNEL

UART_NUM_1_CTS_DIRECT_GPIO_NUM

UART_GPIO19_DIRECT_CHANNEL

UART_NUM_1_RTS_DIRECT_GPIO_NUM

UART_TXD_GPIO17_DIRECT_CHANNEL

UART_RXD_GPIO18_DIRECT_CHANNEL

UART_CTS_GPIO20_DIRECT_CHANNEL

UART_RTS_GPIO19_DIRECT_CHANNEL

2.6.29 USB Device Stack

Overview

The ESP-IDF USB Device Stack (hereinafter referred to as the Device Stack) enables USB Device support on ESP32-S3. By using the Device Stack, ESP32-S3 can be programmed with any well defined USB device functions (e.g., keyboard, mouse, camera), a custom function (aka vendor-specific class), or a combination of those functions (aka a composite device).

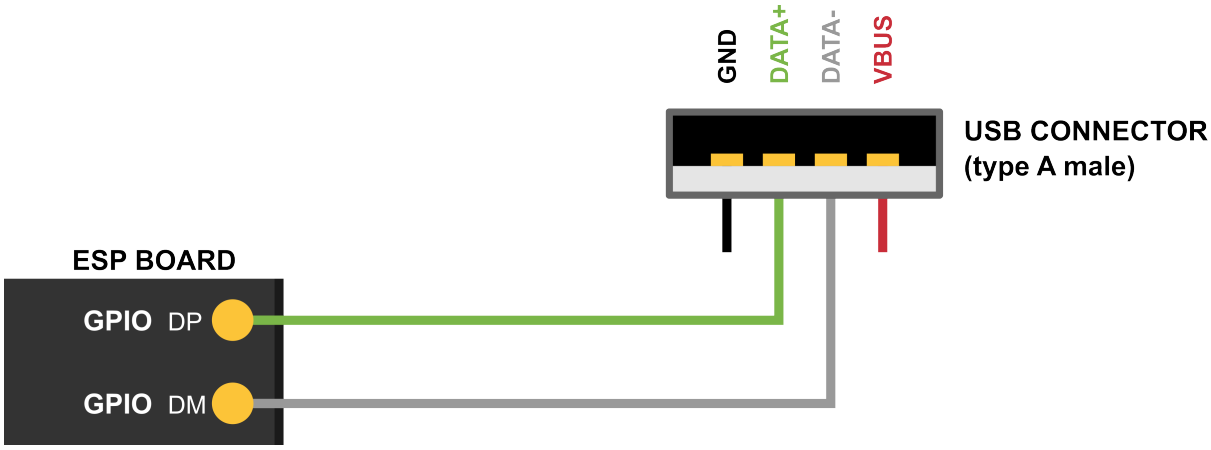
The Device Stack is built around the TinyUSB stack, but extends TinyUSB with some minor features and modifications for better integration with ESP-IDF. The Device stack is distributed as a managed component via the ESP-IDF Component Registry.

Features

- Multiple supported device classes (CDC, HID, MIDI, MSC)
- Composite devices
- Vendor specific classes
- Maximum of 6 endpoints
  - 5 IN/OUT endpoints
  - 1 IN endpoints
- VBUS monitoring for self-powered devices

Hardware Connection

The ESP32-S3 routes the USB D+ and D- signals to GPIOs 20 and 19 respectively. For USB device functionality, these GPIOs should be connected to the bus in some way (e.g., via a Micro-B port, USB-C port, or directly to standard-A plug).



Note: If you are using an ESP32-S3 development board with two USB ports, the port labeled "USB" will already be connected to the D+ and D- GPIOs.

Note: Self-powered devices must also connect VBUS through a voltage divider or comparator. For more details, please refer to *Self-Powered Device*.

Device Stack Structure

The basis of the Device Stack is TinyUSB, where the Device Stack implements the following features on top of TinyUSB:

- Customization of USB descriptors
- Serial device support
- Redirecting of standard streams through the Serial device
- Storage Media (SPI-Flash and SD-Card) for USB Device MSC Class.
- A task within the encapsulated device stack that handles TinyUSB servicing

Component Dependency The Device Stack is distributed via the ESP-IDF Component Registry. Thus, to use it, please add the Device Stack component as dependency using the following command:

```
idf.py add-dependency esp_tinyusb
```

Configuration Options Multiple aspects of the Device Stack can be configured using menuconfig. These include:

- The verbosity of the TinyUSB's log
- Device Stack task related options
- Default device/string descriptor options
- Class specific options

Descriptor Configuration The `tinyusb_config_t` structure provides USB descriptor related fields that should be initialized.

The following descriptors should be initialized for both full-speed and high-speed devices:

- device_descriptor
- string_descriptor

Full-speed devices should initialize the following field to provide their configuration descriptor:

- configuration_descriptor

High-speed devices should initialize the following fields to provide configuration descriptors at each speed:

- fs_configuration_descriptor
- hs_configuration_descriptor
- qualifier_descriptor

Note: When Device Stack supports high-speed, both fs_configuration_descriptor and hs_configuration_descriptor should be present to comply with usb2.0 specification.

The Device Stack will instantiate a USB device based on the descriptors provided in the fields described above when tinyusb_driver_install() is called.

The Device Stack also provides default descriptors that can be installed by setting the corresponding field in tinyusb_driver_install() to NULL. Default descriptors include:

- Default device descriptor: Enabled by setting device_descriptor to NULL. Default device descriptor will use the values set by the corresponding menuconfig options (e.g., PID, VID, bcdDevice etc).
- Default string descriptor: Enabled by setting string_descriptor to NULL. Default string descriptors will use the value set by corresponding menuconfig options (e.g., manufacturer, product, and serial string descriptor options).
- Default configuration descriptor. Some classes that rarely require custom configuration (such as CDC and MSC) will provide default configuration descriptors. These can be enabled by setting associated configuration descriptor field to NULL:
  - configuration_descriptor full-speed descriptor for full-speed devices only
  - fs_configuration_descriptor full-speed descriptor for high-speed devices
  - hs_configuration_descriptor high-speed descriptor for high-speed devices

Note: Backward compatibility: when Device Stack supports high-speed, field configuration_descriptor could be used instead of fs_configuration_descriptor for full-speed configuration descriptor.

Installation

To install the Device Stack, please call tinyusb_driver_install(). The Device Stack's configuration is specified in a tinyusb_config_t structure that is passed as an argument to tinyusb_driver_install().

Note: The tinyusb_config_t structure can be zero-initialized (e.g., const tinyusb_config_t tusb_cfg = { 0 };) or partially (as shown below). For any member that is initialized to 0 or NULL, the stack uses its default configuration values for that member, see example below.

```
const tinyusb_config_t partial_init = {
    .device_descriptor = NULL,  // Use the default device descriptor specified in␣
↪Menuconfig
    .string_descriptor = NULL,  // Use the default string descriptors specified in␣
↪Menuconfig
    .external_phy = false,      // Use internal USB PHY
#if (TUD_OPT_HIGH_SPEED)
    .fs_configuration_descriptor = NULL, // Use the default full-speed␣
↪configuration descriptor according to settings in Menuconfig
    .hs_configuration_descriptor = NULL, // Use the default high-ppeed␣
↪configuration descriptor according to settings in Menuconfig
```

(continues on next page)

(continued from previous page)

```
    .qualifier_descriptor = NULL,  // Use the default qualifier descriptor, with␣
↪values from default device descriptor
#else
    .configuration_descriptor = NULL,   // Use the default configuration␣
↪descriptor according to settings in Menuconfig
#endif // TUD_OPT_HIGH_SPEED

};
```

Self-Powered Device

USB specification mandates self-powered devices to monitor voltage levels on USB's VBUS signal. As opposed to bus-powered devices, a self-powered device can be fully functional even without a USB connection. The self-powered device detects connection and disconnection events by monitoring the VBUS voltage level. VBUS is considered valid if it rises above 4.75 V and invalid if it falls below 4.35 V.

On the ESP32-S3, this will require using a GPIO to act as a voltage sensing pin to detect when VBUS goes above/below the prescribed thresholds. However, ESP32-S3 pins are 3.3 V tolerant. Thus, even if VBUS rises/falls above/below the thresholds mentioned above, it would still appear as a logic HIGH to the ESP32-S3. Thus, in order to detect the VBUS valid condition, users can do one of the following:

- Connect VBUS to a voltage comparator chip/circuit that detects the thresholds described above (i.e., 4.35 V and 4.75 V), and outputs a 3.3 V logic level to the ESP32-S3 indicating whether VBUS is valid or not.
- Use a resistor voltage divider that outputs (0.75 x Vdd) if VBUS is 4.4 V (see figure below).

Note: In either case, the voltage on the sensing pin must be logic low within 3 ms after the device is unplugged from the USB host.

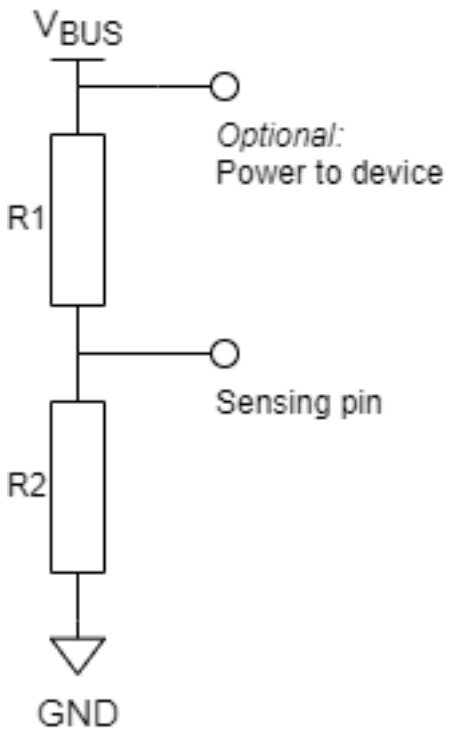


Fig. 32: Simple voltage divider for VBUS monitoring

To use this feature, in `tinyusb_config_t`, you must set `self_powered` to `true` and `vbus_monitor_io` to GPIO number that is used for VBUS monitoring.

USB Serial Device (CDC-ACM)

If the CDC option is enabled in Menuconfig, the USB Serial Device can be initialized with `tusb_cdc_acm_init()` according to the settings from `tinyusb_config_cdcacm_t`, see example below.

```
const tinyusb_config_cdcacm_t acm_cfg = {
    .usb_dev = TINYUSB_USBDEV_0,
    .cdc_port = TINYUSB_CDC_ACM_0,
    .rx_unread_buf_sz = 64,
    .callback_rx = NULL,
    .callback_rx_wanted_char = NULL,
    .callback_line_state_changed = NULL,
    .callback_line_coding_changed = NULL
};
tusb_cdc_acm_init(&acm_cfg);
```

To specify callbacks, you can either set the pointer to your `tusb_cdcacm_callback_t` function in the configuration structure or call `tinyusb_cdcacm_register_callback()` after initialization.

USB Serial Console The USB Serial Device allows the redirection of all standard input/output streams (stdin, stdout, stderr) to USB. Thus, calling standard library input/output functions such as `printf()` will result into the data being sent/received over USB instead of UART.

Users should call `esp_tusb_init_console()` to switch the standard input/output streams to USB, and `esp_tusb_deinit_console()` to switch them back to UART.

USB Mass Storage Device (MSC)

If the MSC `CONFIG_TINYUSB_MSC_ENABLED` option is enabled in Menuconfig, the ESP Chip can be used as USB MSC Device. The storage media (SPI-Flash or SD-Card) can be initialized as shown below.

- SPI-Flash

```
static esp_err_t storage_init_spiflash(wl_handle_t *wl_handle)
{
    ***
    esp_partition_t *data_partition = esp_partition_find_first(ESP_PARTITION_TYPE_
↪DATA, ESP_PARTITION_SUBTYPE_DATA_FAT, NULL);
    ***
    wl_mount(data_partition, wl_handle);
    ***
}
storage_init_spiflash(&wl_handle);

const tinyusb_msc_spiflash_config_t config_spi = {
    .wl_handle = wl_handle
};
tinyusb_msc_storage_init_spiflash(&config_spi);
```

- SD-Card

```
static esp_err_t storage_init_sdmmc(sdmmc_card_t **card)
{
    ***
    sdmmc_host_t host = SDMMC_HOST_DEFAULT();
    sdmmc_slot_config_t slot_config = SDMMC_SLOT_CONFIG_DEFAULT();
    // For SD Card, set bus width to use

    slot_config.width = 4;
```

(continues on next page)

(continued from previous page)

```
    slot_config.clk = CONFIG_EXAMPLE_PIN_CLK;
    slot_config.cmd = CONFIG_EXAMPLE_PIN_CMD;
    slot_config.d0 = CONFIG_EXAMPLE_PIN_D0;
    slot_config.d1 = CONFIG_EXAMPLE_PIN_D1;
    slot_config.d2 = CONFIG_EXAMPLE_PIN_D2;
    slot_config.d3 = CONFIG_EXAMPLE_PIN_D3;
    slot_config.flags |= SDMMC_SLOT_FLAG_INTERNAL_PULLUP;

    sd_card = (sdmmc_card_t *)malloc(sizeof(sdmmc_card_t));
    (*host.init)();
    sdmmc_host_init_slot(host.slot, (const sdmmc_slot_config_t *) &slot_config);
    sdmmc_card_init(&host, sd_card);
    ***
}
storage_init_sdmmc(&card);

const tinyusb_msc_sdmmc_config_t config_sdmmc = {
    .card = card
};
tinyusb_msc_storage_init_sdmmc(&config_sdmmc);
```

Application Examples

The table below describes the code examples available in the directory peripherals/usb/device:

Code Example	Description
peripherals/usb/device/tusb_console	How to set up ESP32-S3 chip to get log output via Serial Device connection
peripherals/usb/device/tusb_serial_device	How to set up ESP32-S3 chip to work as a USB Serial Device
peripherals/usb/device/tusb_midi	How to set up ESP32-S3 chip to work as a USB MIDI Device
peripherals/usb/device/tusb_hid	How to set up ESP32-S3 chip to work as a USB Human Interface Device
peripherals/usb/device/tusb_msc	How to set up ESP32-S3 chip to work as a USB Mass Storage Device
peripherals/usb/device/tusb_composite_msc_serial_device	How to set up ESP32-S3 chip to work as a Composite USB Device (MSC + CDC)

2.6.30 USB Host

The document provides information regarding the USB Host Library. This document is split into the following sections:

Sections

- *USB Host*
  - *Overview*
  - *Architecture*
  - *Usage*
  - *Examples*
  - *Host Stack Configuration*
  - *API Reference*
  - *Maintainers Notes*

Overview

The USB Host Library (hereinafter referred to as the Host Library) is the lowest layer of the USB Host stack that exposes a public facing API. In most cases, applications that require USB Host functionality do not need to interface with the Host Library directly. Instead, most applications use the API provided by a host class driver that is implemented on top of the Host Library.

However, you may want to use the Host Library directly for some of (but not limited to) the following reasons:

- Implementation of a custom host class driver
- Usage of lower level USB Host API

Features & Limitations

The Host Library has the following features:

- Supports Full Speed (FS) and Low Speed (LS) Devices.
- Supports all four transfer types, i.e., Control, Bulk, Interrupt, and Isochronous.
- Allows multiple class drivers to run simultaneously, i.e., multiple clients of the Host Library.
- A single device can be used by multiple clients simultaneously, e.g., composite devices.
- The Host Library itself and the underlying Host Stack does not internally instantiate any OS tasks. The number of tasks is entirely controlled by how the Host Library interface is used. However, a general rule of thumb regarding the number of tasks is `(the number of host class drivers running + 1)`.

Currently, the Host Library and the underlying Host Stack has the following limitations:

- Only supports a single device, but the Host Library's API is designed for multiple device support.
- Only supports Asynchronous transfers.
- Only supports using the first configuration found. Changing to other configurations is not supported yet.
- Transfer timeouts are not supported yet.

Architecture

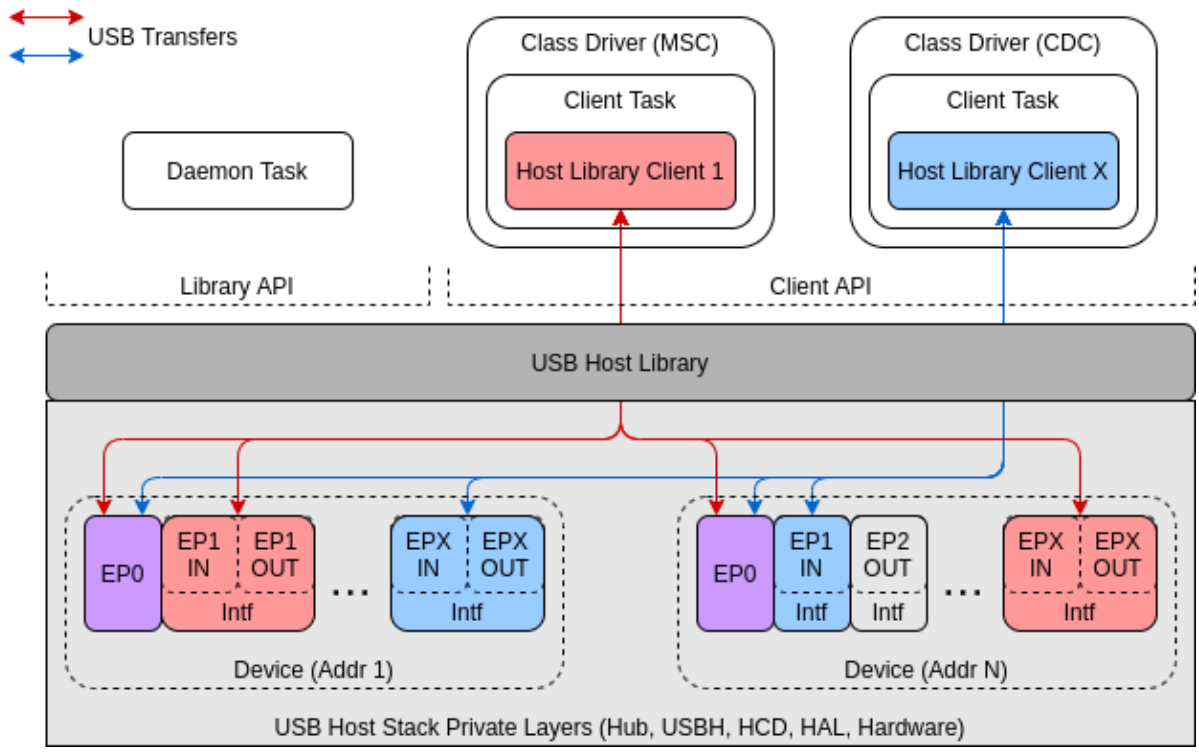


Fig. 33: Diagram of the key entities involved in USB Host functionality

The diagram above shows the key entities that are involved when implementing USB Host functionality. These entities are:

- The **Host Library**
- **Clients** of the Host Library
- **Devices**
- Host Library **Daemon Task**

Host Library The Host Library is the lowest public-facing API layer of the ESP-IDF USB Host Stack. Any other ESP-IDF component (such as a class driver or a user component) that needs to communicate with a connected USB device can only do so using the Host Library API either directly or indirectly.

The Host Library's API is split into two sub-sets, namely the **Library API** and **Client API**.

- The Client API handles the communication between a client of the Host Library and one or more USB devices. The Client API should only be called by registered clients of the Host Library.
- The Library API handles all of the Host Library processing that is not specific to a single client, such as device enumeration. Usually, the library API is called by a Host Library Daemon Task.

Clients A client of the Host Library is a software component, such as a host class driver or user component, which utilizes the Host Library to establish communication with a USB device. Generally, each client has a one-to-one relation with a task. This implies that all Client API calls pertaining to a specific client must originate from the context of the same task.

By organizing the software components that use the Host Library's into clients, the Host Library can delegate the handling of all events specific to that client to the client's task. In other words, each client task is responsible for all the required processing and event handling associated with the USB communication that the client initiates.

Daemon Task Although the Host Library delegates the handling of client events to the clients themselves, there are still Library events –events that are not specific to any particular client –that need to be handled. Library event handling can include things such as:

- Handling USB device connection, enumeration, and disconnection
- Rerouting control transfers to/from clients
- Forwarding events to clients

Therefore, in addition to the client tasks, the Host Library also requires a task, which is usually the Host Library Daemon Task, to handle all of the library events.

Devices The Host Library shields clients from the details of device handling, encompassing details such as connection, memory allocation, and enumeration. The clients are provided only with a list of already connected and enumerated devices to choose from. During enumeration, each device is automatically configured to use the first configuration found, namely, the first configuration descriptor returned on a Get Configuration Descriptor request. For most standard devices, the first configuration will have a `bConfigurationValue` of `1`.

It is possible for two or more clients to simultaneously communicate with the same device as long as they are not communicating to the same interface. However, multiple clients can simultaneously communicate with the same device's default endpoint (i.e., EP0), which will result in their control transfers being serialized.

For a client to communicate with a device, the client must:

1. Open the device using the device's address. This lets the Host Library know that the client is using that device.
2. Claim the interface(s) that will be used for communication. This prevents other clients from claiming the same interface(s).
3. Send transfers to the endpoints of claimed interfaces. The client's task is responsible for handling its own processing and events related to USB device communication.

Usage

The Host Library and the underlying Host Stack will not create any tasks. All tasks, namely the client tasks and the Daemon Task, need to be created by the class drivers or the user. Instead, the Host Library provides two event handler

functions that handle all of the required Host Library processing, thus these functions should be called repeatedly from the client tasks and the Daemon Task. Therefore, the implementation of client tasks and the Daemon Task will be largely centered around the invocation of these event handler functions.

Host Library & Daemon Task

Basic Usage The Host Library API provides *usb_host_lib_handle_events()* to handle library events. This function should be called repeatedly, typically from the Daemon Task. Some notable features regarding *usb_host_lib_handle_events()* are:

- The function can block until a library event needs handling.
- Event flags are returned on each invocation. These event flags are useful for knowing when the Host Library can be uninstalled.

A bare-bones Daemon Task would resemble something like the following code snippet:

```
#include "usb/usb_host.h"

void daemon_task(void *arg)
{
    ...
    bool exit = false;
    while (!exit) {
        uint32_t event_flags;
        usb_host_lib_handle_events(portMAX_DELAY, &event_flags);
        if (event_flags & USB_HOST_LIB_EVENT_FLAGS_NO_CLIENTS) {
            ...
        }
        if (event_flags & USB_HOST_LIB_EVENT_FLAGS_ALL_FREE) {
            ...
        }
        ...
    }
    ...
}
```

Note: See the peripherals/usb/host/usb_host_lib example for full implementation of the Daemon Task.

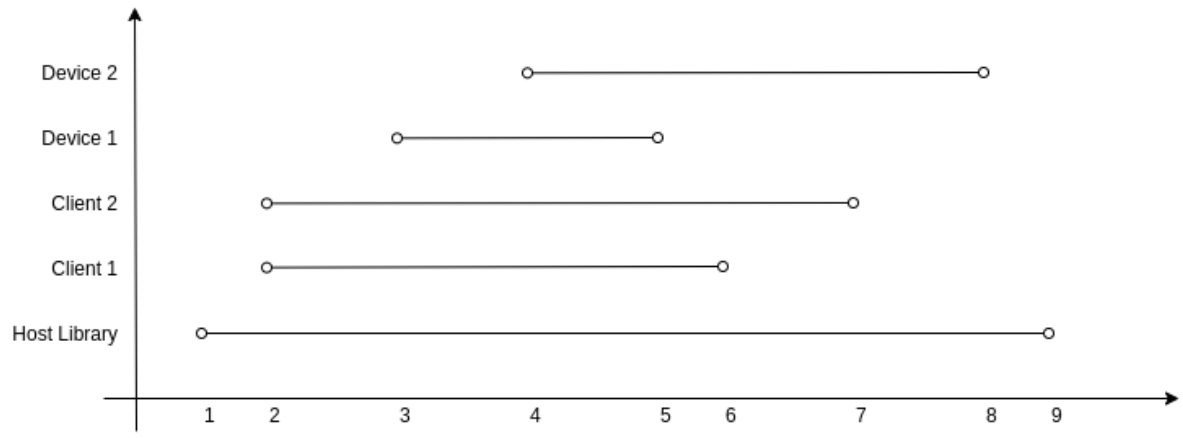


Fig. 34: Graph of Typical USB Host Library Lifecycle

Lifecycle The graph above illustrates the typical lifecycle of the Host Library with multiple clients and devices. Specifically, the example involves:

- two registered clients (Client 1 and Client 2).
- two connected devices (Device 1 and Device 2), where Client 1 communicates with Device 1 and Client 2 communicates with Device 2.

With reference to the graph above, the typical lifecycle involves the following key stages.

1. **The Host Library is installed by calling `usb_host_install()`.**
    - Installation must be done before any other Host Library API is called.
    - Where `usb_host_install()` is called (e.g., from the Daemon Task or another task) depends on the synchronization logic between the Daemon Task, client tasks, and the rest of the system.
2. **Once the Host Library is installed, the clients can be registered by calling `usb_host_client_register()`.**
    - This is typically called from the client task, where the client task waits for a signal from the Daemon Task.
    - This can be called elsewhere if necessary as long it is called after `usb_host_install()`.
3. **Device 1 connects and is then enumerated.**
    - Each registered client (in this case Client 1 and Client 2) is notified of the new device by way of the `USB_HOST_CLIENT_EVENT_NEW_DEV` event.
    - Client 1 opens Device 1 and begins communication with it.
4. **Similarly Device 2 connects and is enumerated.**
    - Client 1 and 2 are notified of a new device via a `USB_HOST_CLIENT_EVENT_NEW_DEV` event.
    - Client 2 opens Device 2 and begins communication with it.
5. **Device 1 suddenly disconnects.**
    - Client 1 is notified by way of `USB_HOST_CLIENT_EVENT_DEV_GONE` and begins its cleanup.
    - Client 2 is not notified as it has not opened Device 1.
6. **Client 1 completes its cleanup and deregisters by calling `usb_host_client_deregister()`.**
    - This is typically called from the client task before the task exits.
    - This can be called elsewhere if necessary as long as Client 1 has already completed its cleanup.
7. **Client 2 completes its communication with Device 2. Client 2 then closes Device 2 and deregisters itself.**
    - The Daemon Task is notified of the deregistration of all clients by way the `USB_HOST_LIB_EVENT_FLAGS_NO_CLIENTS` event flag as Client 2 is the last client to deregister.
    - Device 2 is still allocated (i.e., not freed), as it is still connected albeit not currently opened by any client.
8. **The Daemon Task decides to clean up as there are no more clients.**
    - The Daemon Task must free Device 2 first by calling `usb_host_device_free_all()`.
    - If `usb_host_device_free_all()` was able to free all devices, the function will return *ESP_OK* indicating that all devices have been freed.
    - If `usb_host_device_free_all()` was unable to free all devices for reasons like the device is still opened by a client, the function will return *ESP_ERR_NOT_FINISHED*.
    - The Daemon Task must wait for `usb_host_lib_handle_events()` to return the `USB_HOST_LIB_EVENT_FLAGS_ALL_FREE` event flag in order to know when all devices have been freed.
9. Once the Daemon Task has verified that all clients have deregistered and all devices have been freed, it can now uninstall the Host Library by calling `usb_host_uninstall()`.

Clients & Class Driver

Basic Usage The Host Library API provides `usb_host_client_handle_events()` to handle a particular client's events. This function should be called repeatedly, typically from the client's task. Some notable features regarding `usb_host_client_handle_events()` are:

- The function can block until a client event needs handling.
- The function's primary purpose is to call the various event handling callbacks when a client event occurs.

The following callbacks are called from within `usb_host_client_handle_events()` thus allowing the client task to be notified of events.

- The client event callback of type *usb_host_client_event_cb_t* delivers client event messages to the client. Client event messages indicate events such as the addition or removal of a device.
- The USB transfer completion callback of type *usb_transfer_cb_t* indicates that a particular USB transfer previously submitted by the client has been completed.

Note: Given that the callbacks are called from within *usb_host_client_handle_events()*, users should avoid blocking from within the callbacks as this will result in *usb_host_client_handle_events()* being blocked as well, thus preventing other pending client events from being handled.

The following code snippet demonstrates a bare-bones host class driver and its client task. The code snippet contains:

- A simple client task function `client_task` that calls *usb_host_client_handle_events()* in a loop.
- Implementations of a client event callback and transfer completion callbacks.
- Implementation of a simple state machine for the class driver. The class driver simply opens a device, sends an OUT transfer to EP1, then closes the device.

```
#include <string.h>
#include "usb/usb_host.h"

#define CLASS_DRIVER_ACTION_OPEN_DEV    0x01
#define CLASS_DRIVER_ACTION_TRANSFER    0x02
#define CLASS_DRIVER_ACTION_CLOSE_DEV   0x03

struct class_driver_control {
    uint32_t actions;
    uint8_t dev_addr;
    usb_host_client_handle_t client_hdl;
    usb_device_handle_t dev_hdl;
};

static void client_event_cb(const usb_host_client_event_msg_t *event_msg, void␣
↪*arg)
{
    // This function is called from within usb_host_client_handle_events(). Do not␣
↪block and try to keep it short
    struct class_driver_control *class_driver_obj = (struct class_driver_control␣
↪*)arg;
    switch (event_msg->event) {
        case USB_HOST_CLIENT_EVENT_NEW_DEV:
            class_driver_obj->actions |= CLASS_DRIVER_ACTION_OPEN_DEV;
            class_driver_obj->dev_addr = event_msg->new_dev.address; //Store the␣
↪address of the new device
            break;
        case USB_HOST_CLIENT_EVENT_DEV_GONE:
            class_driver_obj->actions |= CLASS_DRIVER_ACTION_CLOSE_DEV;
            break;
        default:
            break;
    }
}

static void transfer_cb(usb_transfer_t *transfer)
{
    // This function is called from within usb_host_client_handle_events(). Do not␣
↪block and try to keep it short
    struct class_driver_control *class_driver_obj = (struct class_driver_control␣
↪*)transfer->context;
    printf("Transfer status %d, actual number of bytes transferred %d\n", transfer-
↪>status, transfer->actual_num_bytes);
    class_driver_obj->actions |= CLASS_DRIVER_ACTION_CLOSE_DEV;
```

(continues on next page)

(continued from previous page)

```
}

void client_task(void *arg)
{
    ... // Wait until Host Library is installed
    // Initialize class driver objects
    struct class_driver_control class_driver_obj = {0};
    // Register the client
    usb_host_client_config_t client_config = {
        .is_synchronous = false,
        .max_num_event_msg = 5,
        .async = {
            .client_event_callback = client_event_cb,
            .callback_arg = &class_driver_obj,
        }
    };
    usb_host_client_register(&client_config, &class_driver_obj.client_hdl);
    //Allocate a USB transfer
    usb_transfer_t *transfer;
    usb_host_transfer_alloc(1024, 0, &transfer);

    //Event handling loop
    bool exit = false;
    while (!exit) {
        // Call the client event handler function
        usb_host_client_handle_events(class_driver_obj.client_hdl, portMAX_DELAY);
        // Execute pending class driver actions
        if (class_driver_obj.actions & CLASS_DRIVER_ACTION_OPEN_DEV) {
            // Open the device and claim interface 1
            usb_host_device_open(class_driver_obj.client_hdl, class_driver_obj.dev_
↪addr, &class_driver_obj.dev_hdl);
            usb_host_interface_claim(class_driver_obj.client_hdl, class_driver_obj.
↪dev_hdl, 1, 0);
        }
        if (class_driver_obj.actions & CLASS_DRIVER_ACTION_TRANSFER) {
            // Send an OUT transfer to EP1
            memset(transfer->data_buffer, 0xAA, 1024);
            transfer->num_bytes = 1024;
            transfer->device_handle = class_driver_obj.dev_hdl;
            transfer->bEndpointAddress = 0x01;
            transfer->callback = transfer_cb;
            transfer->context = (void *)&class_driver_obj;
            usb_host_transfer_submit(transfer);
        }
        if (class_driver_obj.actions & CLASS_DRIVER_ACTION_CLOSE_DEV) {
            // Release the interface and close the device
            usb_host_interface_release(class_driver_obj.client_hdl, class_driver_
↪obj.dev_hdl, 1);
            usb_host_device_close(class_driver_obj.client_hdl, class_driver_obj.
↪dev_hdl);
            exit = true;
        }
        ... // Handle any other actions required by the class driver
    }

    // Cleanup class driver
    usb_host_transfer_free(transfer);
    usb_host_client_deregister(class_driver_obj.client_hdl);
    ... // Delete the client task. Signal the Daemon Task if necessary.
}
```

Note: An actual host class driver is likely to support many more features, thus will have a much more complex state machine. A host class driver is also likely to need to:

- Be able to open multiple devices
- Parse an opened device's descriptors to identify if the device is of the target class
- Communicate with multiple endpoints of an interface in a particular order
- Claim multiple interfaces of a device
- Handle various errors

Lifecycle The typical life cycle of a client task and class driver will go through the following stages:

1. Wait for some signal regarding the Host Library being installed.
2. Register the client via `usb_host_client_register()` and allocate any other class driver resources, such as allocating transfers using `usb_host_transfer_alloc()`.
3. For each new device that the class driver needs to communicate with:
   a. Check if the device is already connected via `usb_host_device_addr_list_fill()`.
   b. If the device is not already connected, wait for a `USB_HOST_CLIENT_EVENT_NEW_DEV` event from the client event callback.
   c. Open the device via `usb_host_device_open()`.
   d. Parse the device and configuration descriptors via `usb_host_get_device_descriptor()` and `usb_host_get_active_config_descriptor()` respectively.
   e. Claim the necessary interfaces of the device via `usb_host_interface_claim()`.
4. Submit transfers to the device via `usb_host_transfer_submit()` or `usb_host_transfer_submit_control()`.
5. Once an opened device is no longer needed by the class driver, or has disconnected, as indicated by a `USB_HOST_CLIENT_EVENT_DEV_GONE` event:
   a. Stop any previously submitted transfers to the device's endpoints by calling `usb_host_endpoint_halt()` and `usb_host_endpoint_flush()` on those endpoints.
   b. Release all previously claimed interfaces via `usb_host_interface_release()`.
   c. Close the device via `usb_host_device_close()`.
6. Deregister the client via `usb_host_client_deregister()` and free any other class driver resources.
7. Delete the client task. Signal the Daemon Task if necessary.

Examples

Host Library Examples The peripherals/usb/host/usb_host_lib demonstrates basic usage of the USB Host Library's API to implement a pseudo-class driver.

Class Driver Examples The USB Host Stack provides a number of examples that implement host class drivers using the Host Library's API.

CDC-ACM

- A host class driver for the Communication Device Class (Abstract Control Model) is distributed as a managed component via the ESP-IDF Component Registry.
- The peripherals/usb/host/cdc/cdc_acm_host example uses the CDC-ACM host driver component to communicate with CDC-ACM devices.
- The peripherals/usb/host/cdc/cdc_acm_vcp example shows how can you extend the CDC-ACM host driver to interface Virtual COM Port devices.
- The CDC-ACM driver is also used in esp_modem examples, where it is used for communication with cellular modems.

MSC

- A host class driver for the Mass Storage Class (Bulk-Only Transport) is deployed to ESP-IDF Component Registry.
- The peripherals/usb/host/msc example demonstrates the usage of the MSC host driver to read and write to a USB flash drive.

HID

- A host class driver for the HID (Human interface device) is distributed as a managed component via the ESP-IDF Component Registry.
- The peripherals/usb/host/hid example demonstrates the possibility to receive reports from a USB HID device with several interfaces.

UVC

- A host class driver for the USB Video Device Class is distributed as a managed component via the ESP-IDF Component Registry.
- The peripherals/usb/host/uvc example demonstrates the usage of the UVC host driver to receive a video stream from a USB camera and optionally forward that stream over Wi-Fi.

Host Stack Configuration

Non-Compliant Device Support To support USB devices that are non-compliant in various scenarios or exhibit specific behaviors, it is possible to configure the USB Host stack.

As a USB device may be hot-plugged, it is essential to have configurable delays between power switching and device attachment, and when the device's internal power has stabilized.

Enumeration Configuration During the process of enumerating connected USB devices, several delay values ensure the proper functioning of the device.

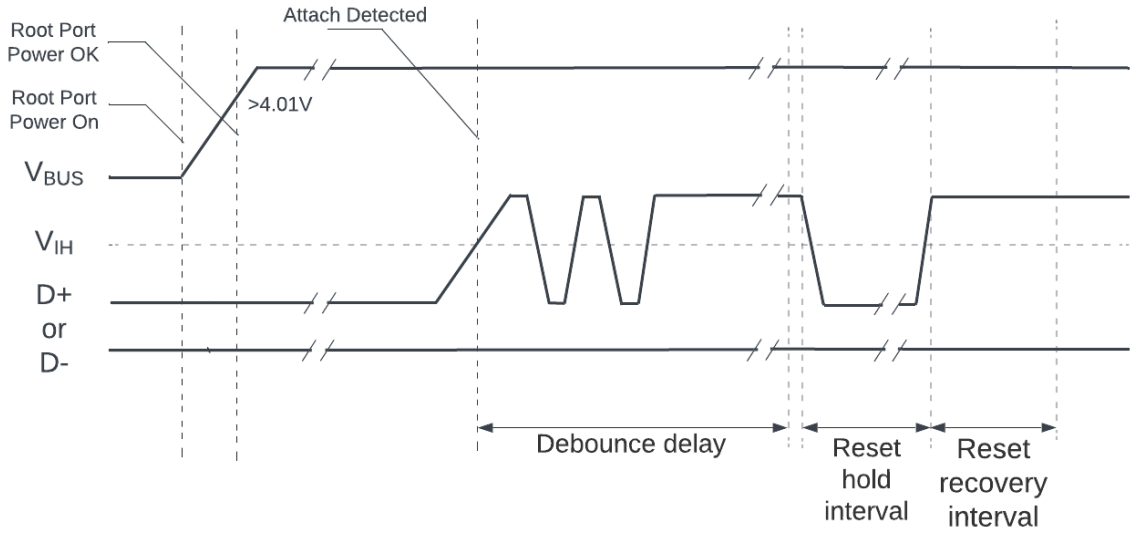


Fig. 35: USB Root Hub Power-on and Connection Events Timing

The figure above shows all the delay values associated with both turning on port power with a device connected and hot-plugging a device.

- After a port is reset or resumed, the USB system software is expected to provide a "recovery" interval of 10 ms before the device attached to the port is expected to respond to data transfers.

- After the reset/resume recovery interval, if a device receives a `SetAddress()` request, the device must be able to complete processing of the request and be able to successfully complete the Status stage of the request within 50 ms.
- After successful completion of the Status stage, the device is allowed a `SetAddress()` recovery interval of 2 ms.

Note: For more details regarding connection event timings, please refer to USB 2.0 Specification > Chapter 7.1.7.3 *Connect and Disconnect Signaling*.

Configurable parameters of the USB host stack can be configured with multiple options via Menuconfig.

- For debounce delay, refer to *CONFIG_USB_HOST_DEBOUNCE_DELAY_MS*.
- For reset hold interval, refer to *CONFIG_USB_HOST_RESET_HOLD_MS*.
- For reset recovery interval, refer to *CONFIG_USB_HOST_RESET_RECOVERY_MS*.
- For `SetAddress()` recovery interval, refer to *CONFIG_USB_HOST_SET_ADDR_RECOVERY_MS*.

Multiple Configuration Support To support USB devices that have more than one configuration, it is possible to specify the desired configuration number during a device's enumeration process.

Enumeration Filter The enumeration filter is a callback function of type *usb_host_enum_filter_cb_t* called at the beginning of the enumeration process once a device descriptor is read from a newly attached USB device. Consequently, the user is provided with the obtained device descriptor. Through this callback, the user can:

- Select the configuration of the USB device.
- Filter which USB devices should be enumerated.

To use the enumeration filter, users should enable the *CONFIG_USB_HOST_ENABLE_ENUM_FILTER_CALLBACK* option using menuconfig. Users can specify the callback by setting *usb_host_config_t::enum_filter_cb* which is then passed to the Host Library when calling *usb_host_install()*.

API Reference

The API of the USB Host Library is separated into the following header files. However, it is sufficient for applications to only `#include "usb/usb_host.h"` and all USB Host Library headers will also be included.

- usb/include/usb/usb_host.h contains the functions and types of the USB Host Library.
- usb/include/usb/usb_helpers.h contains various helper functions that are related to the USB protocol such as descriptor parsing.
- usb/include/usb/usb_types_stack.h contains types that are used across multiple layers of the USB Host stack.
- usb/include/usb/usb_types_ch9.h contains types and macros related to Chapter 9 of the USB2.0 specification, i.e., descriptors and standard requests.

Header File

- components/usb/include/usb/usb_host.h
- This header file can be included with:

```
#include "usb/usb_host.h"
```

- This header file is a part of the API provided by the `usb` component. To declare that your component depends on `usb`, add the following to your CMakeLists.txt:

```
REQUIRES usb
```

or

```
PRIV_REQUIRES usb
```

Functions

esp_err_t **usb_host_install** (const *usb_host_config_t* *config)

Install the USB Host Library.

- This function should only once to install the USB Host Library
- This function should be called before any other USB Host Library functions are called

---

Note: If skip_phy_setup is set in the install configuration, the user is responsible for ensuring that the underlying Host Controller is enabled and the USB PHY (internal or external) is already setup before this function is called.

Parameters **config** -- **[in]** USB Host Library configuration
Returns esp_err_t

esp_err_t **usb_host_uninstall** (void)

Uninstall the USB Host Library.

- This function should be called to uninstall the USB Host Library, thereby freeing its resources
- All clients must have been deregistered before calling this function
- All devices must have been freed by calling usb_host_device_free_all() and receiving the USB_HOST_LIB_EVENT_FLAGS_ALL_FREE event flag

---

Note: If skip_phy_setup was set when the Host Library was installed, the user is responsible for disabling the underlying Host Controller and USB PHY (internal or external).

Returns esp_err_t

esp_err_t **usb_host_lib_handle_events** (TickType_t timeout_ticks, uint32_t *event_flags_ret)

Handle USB Host Library events.

- This function handles all of the USB Host Library's processing and should be called repeatedly in a loop
- Check event_flags_ret to see if an flags are set indicating particular USB Host Library events
- This function should never be called by multiple threads simultaneously

---

Note: This function can block

Parameters
- **timeout_ticks** -- **[in]** Timeout in ticks to wait for an event to occur
- **event_flags_ret** -- **[out]** Event flags that indicate what USB Host Library event occurred.

Returns esp_err_t

esp_err_t **usb_host_lib_unblock** (void)

Unblock the USB Host Library handler.

- This function simply unblocks the USB Host Library event handling function (usb_host_lib_handle_events())

Returns esp_err_t

esp_err_t **usb_host_lib_info**(*usb_host_lib_info_t* *info_ret)

Get current information about the USB Host Library.

Parameters **info_ret** -- **[out]** USB Host Library Information
Returns esp_err_t

esp_err_t **usb_host_client_register**(const *usb_host_client_config_t* *client_config, *usb_host_client_handle_t* *client_hdl_ret)

Register a client of the USB Host Library.

- This function registers a client of the USB Host Library
- Once a client is registered, its processing function usb_host_client_handle_events() should be called repeatedly

Parameters
- **client_config** -- **[in]** Client configuration
- **client_hdl_ret** -- **[out]** Client handle

Returns esp_err_t

esp_err_t **usb_host_client_deregister**(*usb_host_client_handle_t* client_hdl)

Deregister a USB Host Library client.

- This function deregisters a client of the USB Host Library
- The client must have closed all previously opened devices before attempting to deregister

Parameters **client_hdl** -- **[in]** Client handle
Returns esp_err_t

esp_err_t **usb_host_client_handle_events**(*usb_host_client_handle_t* client_hdl, TickType_t timeout_ticks)

USB Host Library client processing function.

- This function handles all of a client's processing and should be called repeatedly in a loop
- For a particular client, this function should never be called by multiple threads simultaneously

---

Note: This function can block

Parameters
- **client_hdl** -- **[in]** Client handle
- **timeout_ticks** -- **[in]** Timeout in ticks to wait for an event to occur

Returns esp_err_t

esp_err_t **usb_host_client_unblock**(*usb_host_client_handle_t* client_hdl)

Unblock a client.

- This function simply unblocks a client if it is blocked on the usb_host_client_handle_events() function.
- This function is useful when need to unblock a client in order to deregister it.

Parameters **client_hdl** -- **[in]** Client handle
Returns esp_err_t

esp_err_t **usb_host_device_open**(*usb_host_client_handle_t* client_hdl, uint8_t dev_addr, *usb_device_handle_t* *dev_hdl_ret)

Open a device.

- This function allows a client to open a device
- A client must open a device first before attempting to use it (e.g., sending transfers, device requests etc.)

Parameters
- **client_hdl** -- **[in]** Client handle
- **dev_addr** -- **[in]** Device's address
- **dev_hdl_ret** -- **[out]** Device's handle

Returns esp_err_t

esp_err_t **usb_host_device_close**(*usb_host_client_handle_t* client_hdl, *usb_device_handle_t* dev_hdl)

Close a device.

- This function allows a client to close a device
- A client must close a device after it has finished using the device (claimed interfaces must also be released)
- A client must close all devices it has opened before deregistering

---

Note: This function can block

Parameters
- **client_hdl** -- **[in]** Client handle
- **dev_hdl** -- **[in]** Device handle

Returns esp_err_t

esp_err_t **usb_host_device_free_all**(void)

Indicate that all devices can be freed when possible.

- This function marks all devices as waiting to be freed
- If a device is not opened by any clients, it will be freed immediately
- If a device is opened by at least one client, the device will be free when the last client closes that device.
- Wait for the USB_HOST_LIB_EVENT_FLAGS_ALL_FREE flag to be set by usb_host_lib_handle_events() in order to know when all devices have been freed
- This function is useful when cleaning up devices before uninstalling the USB Host Library

Returns
- ESP_ERR_NOT_FINISHED: There are one or more devices that still need to be freed. Wait for USB_HOST_LIB_EVENT_FLAGS_ALL_FREE event
- ESP_OK: All devices already freed (i.e., there were no devices)
- Other: Error

esp_err_t **usb_host_device_addr_list_fill**(int list_len, uint8_t *dev_addr_list, int *num_dev_ret)

Fill a list of device address.

- This function fills an empty list with the address of connected devices
- The Device addresses can then used in usb_host_device_open()
- If there are more devices than the list_len, this function will only fill up to list_len number of devices.

Parameters
- **list_len** -- **[in]** Length of the empty list

- **dev_addr_list** -- **[inout]** Empty list to be filled
- **num_dev_ret** -- **[out]** Number of devices

Returns esp_err_t

esp_err_t **usb_host_device_info**(*usb_device_handle_t* dev_hdl, *usb_device_info_t* *dev_info)

Get device's information.

- This function gets some basic information of a device
- The device must be opened first before attempting to get its information

Note: This function can block

Parameters

- **dev_hdl** -- **[in]** Device handle
- **dev_info** -- **[out]** Device information

Returns esp_err_t

esp_err_t **usb_host_get_device_descriptor**(*usb_device_handle_t* dev_hdl, const *usb_device_desc_t* **device_desc)

Get device's device descriptor.

- A client must call usb_host_device_open() first
- No control transfer is sent. The device's descriptor is cached on enumeration
- This function simple returns a pointer to the cached descriptor

Note: No control transfer is sent. The device's descriptor is cached on enumeration

Parameters

- **dev_hdl** -- **[in]** Device handle
- **device_desc** -- **[out]** Device descriptor

Returns esp_err_t

esp_err_t **usb_host_get_active_config_descriptor**(*usb_device_handle_t* dev_hdl, const *usb_config_desc_t* **config_desc)

Get device's active configuration descriptor.

- A client must call usb_host_device_open() first
- No control transfer is sent. The device's active configuration descriptor is cached on enumeration
- This function simple returns a pointer to the cached descriptor

Note: This function can block

Note: No control transfer is sent. A device's active configuration descriptor is cached on enumeration

Parameters

- **dev_hdl** -- **[in]** Device handle
- **config_desc** -- **[out]** Configuration descriptor

Returns esp_err_t

esp_err_t **usb_host_get_config_desc**(*usb_host_client_handle_t* client_hdl, *usb_device_handle_t* dev_hdl, uint8_t bConfigurationValue, const *usb_config_desc_t* **config_desc_ret)

Get get device's configuration descriptor.

- The USB Host library only caches a device's active configuration descriptor.
- This function reads any configuration descriptor of a particular device (specified by bConfigurationValue).
- This function will read the specified configuration descriptor via control transfers, and allocate memory to store that descriptor.
- Users can call usb_host_free_config_desc() to free the descriptor's memory afterwards.

Note: This function can block

Note: A client must call usb_host_device_open() on the device first

Note: bConfigurationValue starts from index 1

Parameters
- **client_hdl** -- **[in]** Client handle - usb_host_client_handle_events() should be called repeatedly in a separate task to handle client events
- **dev_hdl** -- **[in]** Device handle
- **config_desc_ret** -- **[out]** Returned configuration descriptor
- **bConfigurationValue** -- **[in]** Index of device's configuration descriptor to be read

Returns esp_err_t

esp_err_t **usb_host_free_config_desc**(const *usb_config_desc_t* *config_desc)

Free a configuration descriptor.

This function frees a configuration descriptor that was returned by usb_host_get_config_desc()

Parameters **config_desc** -- **[out]** Configuration descriptor
Returns esp_err_t

esp_err_t **usb_host_interface_claim**(*usb_host_client_handle_t* client_hdl, *usb_device_handle_t* dev_hdl, uint8_t bInterfaceNumber, uint8_t bAlternateSetting)

Function for a client to claim a device's interface.

- A client must claim a device's interface before attempting to communicate with any of its endpoints
- Once an interface is claimed by a client, it cannot be claimed by any other client.

Note: This function can block

Parameters
- **client_hdl** -- **[in]** Client handle
- **dev_hdl** -- **[in]** Device handle
- **bInterfaceNumber** -- **[in]** Interface number
- **bAlternateSetting** -- **[in]** Interface alternate setting number

Returns esp_err_t

esp_err_t **usb_host_interface_release**(*usb_host_client_handle_t* client_hdl, *usb_device_handle_t* dev_hdl, uint8_t bInterfaceNumber)

Function for a client to release a previously claimed interface.

- A client should release a device's interface after it no longer needs to communicate with the interface
- A client must release all of its interfaces of a device it has claimed before being able to close the device

---

Note: This function can block

Parameters
- **client_hdl** -- **[in]** Client handle
- **dev_hdl** -- **[in]** Device handle
- **bInterfaceNumber** -- **[in]** Interface number

Returns esp_err_t

esp_err_t **usb_host_endpoint_halt**(*usb_device_handle_t* dev_hdl, uint8_t bEndpointAddress)

Halt a particular endpoint.

- The device must have been opened by a client
- The endpoint must be part of an interface claimed by a client
- Once halted, the endpoint must be cleared using usb_host_endpoint_clear() before it can communicate again

---

Note: This function can block

Parameters
- **dev_hdl** -- Device handle
- **bEndpointAddress** -- Endpoint address

Returns esp_err_t

esp_err_t **usb_host_endpoint_flush**(*usb_device_handle_t* dev_hdl, uint8_t bEndpointAddress)

Flush a particular endpoint.

- The device must have been opened by a client
- The endpoint must be part of an interface claimed by a client
- The endpoint must have been halted (either through a transfer error, or usb_host_endpoint_halt())
- Flushing an endpoint will caused an queued up transfers to be canceled

---

Note: This function can block

Parameters
- **dev_hdl** -- Device handle
- **bEndpointAddress** -- Endpoint address

Returns esp_err_t

esp_err_t **usb_host_endpoint_clear**(*usb_device_handle_t* dev_hdl, uint8_t bEndpointAddress)

Clear a halt on a particular endpoint.

- The device must have been opened by a client

- The endpoint must be part of an interface claimed by a client
- The endpoint must have been halted (either through a transfer error, or usb_host_endpoint_halt())
- If the endpoint has any queued up transfers, clearing a halt will resume their execution

---

Note: This function can block

Parameters
- **`dev_hdl`** -- Device handle
- **`bEndpointAddress`** -- Endpoint address

Returns esp_err_t

esp_err_t **`usb_host_transfer_alloc`**(size_t data_buffer_size, int num_isoc_packets, *usb_transfer_t* **transfer)

Allocate a transfer object.

- This function allocates a transfer object
- Each transfer object has a fixed sized buffer specified on allocation
- A transfer object can be re-used indefinitely
- A transfer can be submitted using usb_host_transfer_submit() or usb_host_transfer_submit_control()

Parameters
- **`data_buffer_size`** -- **[in]** Size of the transfer's data buffer
- **`num_isoc_packets`** -- **[in]** Number of isochronous packets in transfer (set to 0 for non-isochronous transfers)
- **`transfer`** -- **[out]** Transfer object

Returns esp_err_t

esp_err_t **`usb_host_transfer_free`**(*usb_transfer_t* *transfer)

Free a transfer object.

- Free a transfer object previously allocated using usb_host_transfer_alloc()
- The transfer must not be in-flight when attempting to free it
- If a NULL pointer is passed, this function will simply return ESP_OK

Parameters **`transfer`** -- **[in]** Transfer object
Returns esp_err_t

esp_err_t **`usb_host_transfer_submit`**(*usb_transfer_t* *transfer)

Submit a non-control transfer.

- Submit a transfer to a particular endpoint. The device and endpoint number is specified inside the transfer
- The transfer must be properly initialized before submitting
- On completion, the transfer's callback will be called from the client's usb_host_client_handle_events() function.

Parameters **`transfer`** -- **[in]** Initialized transfer object
Returns esp_err_t

esp_err_t **`usb_host_transfer_submit_control`**(*usb_host_client_handle_t* client_hdl, *usb_transfer_t* *transfer)

Submit a control transfer.

- Submit a control transfer to a particular device. The client must have opened the device first
- The transfer must be properly initialized before submitting. The first 8 bytes of the transfer's data buffer should contain the control transfer setup packet
- On completion, the transfer's callback will be called from the client's usb_host_client_handle_events() function.

Parameters
- **`client_hdl`** -- **[in]** Client handle
- **`transfer`** -- **[in]** Initialized transfer object

Returns esp_err_t

Structures

struct **`usb_host_client_event_msg_t`**

Client event message.

Client event messages are sent to each client of the USB Host Library in order to notify them of various USB Host Library events such as:

- Addition of new devices
- Removal of existing devices

Note: The event message structure has a union with members corresponding to each particular event. Based on the event type, only the relevant member field should be accessed.

Public Members

usb_host_client_event_t **`event`**

Type of event

uint8_t **`address`**

New device's address

struct *usb_host_client_event_msg_t*::[anonymous]::[anonymous] **`new_dev`**

New device info

usb_device_handle_t **`dev_hdl`**

The handle of the device that was gone

struct *usb_host_client_event_msg_t*::[anonymous]::[anonymous] **`dev_gone`**

Gone device info

struct **`usb_host_lib_info_t`**

Current information about the USB Host Library obtained via usb_host_lib_info()

Public Members

int **`num_devices`**

Current number of connected (and enumerated) devices

int **num_clients**

Current number of registered clients

struct **usb_host_config_t**

USB Host Library configuration.

Configuration structure of the USB Host Library. Provided in the usb_host_install() function

Public Members

bool **skip_phy_setup**

If set, the USB Host Library will not configure the USB PHY thus allowing the user to manually configure the USB PHY before calling usb_host_install(). Users should set this if they want to use an external USB PHY. Otherwise, the USB Host Library will automatically configure the internal USB PHY

int **intr_flags**

Interrupt flags for the underlying ISR used by the USB Host stack

usb_host_enum_filter_cb_t **enum_filter_cb**

Enumeration filter callback. Enable CONFIG_USB_HOST_ENABLE_ENUM_FILTER_CALLBACK to use this feature. Set to NULL otherwise.

struct **usb_host_client_config_t**

USB Host Library Client configuration.

Configuration structure for a USB Host Library client. Provided in usb_host_client_register()

Public Members

bool **is_synchronous**

Whether the client is asynchronous or synchronous or not. Set to false for now.

int **max_num_event_msg**

Maximum number of event messages that can be stored (e.g., 3)

usb_host_client_event_cb_t **client_event_callback**

Client's event callback function

void ***callback_arg**

Event callback function argument

struct *usb_host_client_config_t*::[anonymous]::[anonymous] **async**

Async callback config

Macros

USB_HOST_LIB_EVENT_FLAGS_NO_CLIENTS

All clients have been deregistered from the USB Host Library

USB_HOST_LIB_EVENT_FLAGS_ALL_FREE

The USB Host Library has freed all devices

Type Definitions

typedef struct usb_host_client_handle_s ***usb_host_client_handle_t**

Handle to a USB Host Library asynchronous client.

An asynchronous client can be registered using usb_host_client_register()

Note: Asynchronous API

---

typedef void (***usb_host_client_event_cb_t**)(const *usb_host_client_event_msg_t* *event_msg, void *arg)

Client event callback.

- Each client of the USB Host Library must register an event callback to receive event messages from the USB Host Library.
- The client event callback is run from the context of the clients usb_host_client_handle_events() function

Enumerations

enum **usb_host_client_event_t**

The type event in a client event message.

Values:

enumerator **USB_HOST_CLIENT_EVENT_NEW_DEV**

A new device has been enumerated and added to the USB Host Library

enumerator **USB_HOST_CLIENT_EVENT_DEV_GONE**

A device opened by the client is now gone

Header File

- components/usb/include/usb/usb_helpers.h
- This header file can be included with:

```
#include "usb/usb_helpers.h"
```

- This header file is a part of the API provided by the `usb` component. To declare that your component depends on `usb`, add the following to your CMakeLists.txt:

```
REQUIRES usb
```

or

```
PRIV_REQUIRES usb
```

Functions

const *usb_standard_desc_t* ***usb_parse_next_descriptor**(const *usb_standard_desc_t* *cur_desc, uint16_t wTotalLength, int *offset)

Get the next descriptor.

Given a particular descriptor within a full configuration descriptor, get the next descriptor within the configuration descriptor. This is a convenience function that can be used to walk each individual descriptor within a full configuration descriptor.

Parameters

- **cur_desc** -- **[in]** Current descriptor
- **wTotalLength** -- **[in]** Total length of the configuration descriptor
- **offset** -- **[inout]** Byte offset relative to the start of the configuration descriptor. On input, it is the offset of the current descriptor. On output, it is the offset of the returned descriptor.

Returns usb_standard_desc_t* Next descriptor, NULL if end of configuration descriptor reached

const *usb_standard_desc_t* ***usb_parse_next_descriptor_of_type**(const *usb_standard_desc_t* *cur_desc, uint16_t wTotalLength, uint8_t bDescriptorType, int *offset)

Get the next descriptor of a particular type.

Given a particular descriptor within a full configuration descriptor, get the next descriptor of a particular type (i.e., using the bDescriptorType value) within the configuration descriptor.

Parameters

- **cur_desc** -- **[in]** Current descriptor
- **wTotalLength** -- **[in]** Total length of the configuration descriptor
- **bDescriptorType** -- **[in]** Type of the next descriptor to get
- **offset** -- **[inout]** Byte offset relative to the start of the configuration descriptor. On input, it is the offset of the current descriptor. On output, it is the offset of the returned descriptor.

Returns usb_standard_desc_t* Next descriptor, NULL if end descriptor is not found or configuration descriptor reached

int **usb_parse_interface_number_of_alternate**(const *usb_config_desc_t* *config_desc, uint8_t bInterfaceNumber)

Get the number of alternate settings for a bInterfaceNumber.

Given a particular configuration descriptor, for a particular bInterfaceNumber, get the number of alternate settings available for that interface (i.e., the max possible value of bAlternateSetting for that bInterfaceNumber).

Parameters

- **config_desc** -- **[in]** Pointer to the start of a full configuration descriptor
- **bInterfaceNumber** -- **[in]** Interface number

Returns int The number of alternate settings that the interface has, -1 if bInterfaceNumber not found

const *usb_intf_desc_t* ***usb_parse_interface_descriptor**(const *usb_config_desc_t* *config_desc, uint8_t bInterfaceNumber, uint8_t bAlternateSetting, int *offset)

Get a particular interface descriptor (using bInterfaceNumber and bAlternateSetting)

Given a full configuration descriptor, get a particular interface descriptor.

Note: To get the number of alternate settings for a particular bInterfaceNumber, call usb_parse_interface_number_of_alternate()

Parameters

- **config_desc** -- **[in]** Pointer to the start of a full configuration descriptor
- **bInterfaceNumber** -- **[in]** Interface number
- **bAlternateSetting** -- **[in]** Alternate setting number

- **offset** -- **[out]** Byte offset of the interface descriptor relative to the start of the configuration descriptor. Can be NULL.

Returns const usb_intf_desc_t* Pointer to interface descriptor, NULL if not found.

const *usb_ep_desc_t* ***usb_parse_endpoint_descriptor_by_index**(const *usb_intf_desc_t* *intf_desc, int index, uint16_t wTotalLength, int *offset)

Get an endpoint descriptor within an interface descriptor.

Given an interface descriptor, get the Nth endpoint descriptor of the interface. The number of endpoints in an interface is indicated by the bNumEndpoints field of the interface descriptor.

Note: If bNumEndpoints is 0, it means the interface uses the default endpoint only

---

Parameters

- **intf_desc** -- **[in]** Pointer to thee start of an interface descriptor
- **index** -- **[in]** Endpoint index
- **wTotalLength** -- **[in]** Total length of the containing configuration descriptor
- **offset** -- **[inout]** Byte offset relative to the start of the configuration descriptor. On input, it is the offset of the interface descriptor. On output, it is the offset of the endpoint descriptor.

Returns const usb_ep_desc_t* Pointer to endpoint descriptor, NULL if not found.

const *usb_ep_desc_t* ***usb_parse_endpoint_descriptor_by_address**(const *usb_config_desc_t* *config_desc, uint8_t bInterfaceNumber, uint8_t bAlternateSetting, uint8_t bEndpointAddress, int *offset)

Get an endpoint descriptor based on an endpoint's address.

Given a configuration descriptor, get an endpoint descriptor based on it's bEndpointAddress, bAlternateSetting, and bInterfaceNumber.

Parameters

- **config_desc** -- **[in]** Pointer to the start of a full configuration descriptor
- **bInterfaceNumber** -- **[in]** Interface number
- **bAlternateSetting** -- **[in]** Alternate setting number
- **bEndpointAddress** -- **[in]** Endpoint address
- **offset** -- **[out]** Byte offset of the endpoint descriptor relative to the start of the configuration descriptor. Can be NULL

Returns const usb_ep_desc_t* Pointer to endpoint descriptor, NULL if not found.

void **usb_print_device_descriptor**(const *usb_device_desc_t* *devc_desc)

Print device descriptor.

Parameters **devc_desc** -- Device descriptor

void **usb_print_config_descriptor**(const *usb_config_desc_t* *cfg_desc, *print_class_descriptor_cb* class_specific_cb)

Print configuration descriptor.

- This function prints the full contents of a configuration descriptor (including interface and endpoint descriptors)
- When a non-standard descriptor is encountered, this function will call the class_specific_cb if it is provided

Parameters

- **cfg_desc** -- Configuration descriptor
- **class_specific_cb** -- Class specific descriptor callback. Can be NULL

void **usb_print_string_descriptor**(const *usb_str_desc_t* *str_desc)

Print a string descriptor.

This funciton will only print ASCII characters of the UTF-16 encoded string

Parameters **str_desc** -- String descriptor

static inline int **usb_round_up_to_mps**(int num_bytes, int mps)

Round up to an integer multiple of endpoint's MPS.

This is a convenience function to round up a size/length to an endpoint's MPS (Maximum packet size). This is useful when calculating transfer or buffer lengths of IN endpoints.

- If MPS <= 0, this function will return 0
- If num_bytes <= 0, this function will return 0

Parameters

- **num_bytes** -- **[in]** Number of bytes
- **mps** -- **[in]** MPS

Returns int Round up integer multiple of MPS

Type Definitions

typedef void (***print_class_descriptor_cb**)(const *usb_standard_desc_t**)

Print class specific descriptor callback.

Optional callback to be provided to usb_print_config_descriptor() function. The callback is called when when a non-standard descriptor is encountered. The callback should decode the descriptor as print it.

Header File

- components/usb/include/usb/usb_types_stack.h
- This header file can be included with:

```
#include "usb/usb_types_stack.h"
```

- This header file is a part of the API provided by the `usb` component. To declare that your component depends on `usb`, add the following to your CMakeLists.txt:

```
REQUIRES usb
```

or

```
PRIV_REQUIRES usb
```

Structures

struct **usb_device_info_t**

Basic information of an enumerated device.

Public Members

usb_speed_t **speed**

Device's speed

uint8_t **dev_addr**

Device's address

uint8_t **bMaxPacketSize0**

The maximum packet size of the device's default endpoint

uint8_t **bConfigurationValue**

Device's current configuration number

const *usb_str_desc_t* ***str_desc_manufacturer**

Pointer to Manufacturer string descriptor (can be NULL)

const *usb_str_desc_t* ***str_desc_product**

Pointer to Product string descriptor (can be NULL)

const *usb_str_desc_t* ***str_desc_serial_num**

Pointer to Serial Number string descriptor (can be NULL)

struct **usb_isoc_packet_desc_t**

Isochronous packet descriptor.

If the number of bytes in an Isochronous transfer is larger than the MPS of the endpoint, the transfer is split into multiple packets transmitted at the endpoint's specified interval. An array of Isochronous packet descriptors describes how an Isochronous transfer should be split into multiple packets.

Public Members

int **num_bytes**

Number of bytes to transmit/receive in the packet. IN packets should be integer multiple of MPS

int **actual_num_bytes**

Actual number of bytes transmitted/received in the packet

usb_transfer_status_t **status**

Status of the packet

struct **usb_transfer_s**

USB transfer structure.

Public Members

uint8_t *const **data_buffer**

Pointer to data buffer

const size_t **data_buffer_size**

Size of the data buffer in bytes

int **num_bytes**

Number of bytes to transfer. Control transfers should include the size of the setup packet. Isochronous transfer should be the total transfer size of all packets. For non-control IN transfers, num_bytes should be an integer multiple of MPS.

int **actual_num_bytes**

Actual number of bytes transferred

uint32_t **flags**

Transfer flags

usb_device_handle_t **device_handle**

Device handle

uint8_t **bEndpointAddress**

Endpoint Address

usb_transfer_status_t **status**

Status of the transfer

uint32_t **timeout_ms**

Timeout (in milliseconds) of the packet (currently not supported yet)

usb_transfer_cb_t **callback**

Transfer callback

void ***context**

Context variable for transfer to associate transfer with something

const int **num_isoc_packets**

Only relevant to Isochronous. Number of service periods (i.e., intervals) to transfer data buffer over.

usb_isoc_packet_desc_t **isoc_packet_desc**[]

Descriptors for each Isochronous packet

Macros

USB_TRANSFER_FLAG_ZERO_PACK

Terminate Bulk/Interrupt OUT transfer with a zero length packet.

OUT transfers normally terminate when the Host has transferred the exact amount of data it needs to the device. However, for bulk and interrupt OUT transfers, if the transfer size just happened to be a multiple of MPS, it will be impossible to know the boundary between two consecutive transfers to the same endpoint.

Therefore, this flag will cause the transfer to automatically add a zero length packet (ZLP) at the end of the transfer if the following conditions are met:

- The target endpoint is a Bulk/Interrupt OUT endpoint (Host to device)
- The transfer's length (i.e., transfer.num_bytes) is a multiple of the endpoint's MPS

Otherwise, this flag has no effect.

Users should check whether their target device's class requires a ZLP, as not all Bulk/Interrupt OUT endpoints require them. For example:

- For MSC Bulk Only Transport class, the Host MUST NEVER send a ZLP. Bulk transfer boundaries are determined by the CBW and CSW instead
- For CDC Ethernet, the Host MUST ALWAYS send a ZLP if a segment (i.e., a transfer) is a multiple of MPS (See 3.3.1 Segment Delineation)

Note: See USB2.0 specification 5.7.3 and 5.8.3 for more details

---

Note: IN transfers normally terminate when the Host as receive the exact amount of data it needs (must be multiple of MPS) or the endpoint sends a short packet to the Host (For bulk OUT only). Indicates that a bulk OUT transfers should always terminate with a short packet, even if it means adding an extra zero length packet

---

Type Definitions

typedef struct usb_device_handle_s ***usb_device_handle_t**

Handle of a USB Device connected to a USB Host.

typedef bool (***usb_host_enum_filter_cb_t**)(const *usb_device_desc_t* *dev_desc, uint8_t *bConfigurationValue)

Enumeration filter callback.

This callback is called at the beginning of the enumeration process for a newly attached device. Through this callback, users are able to:

- filter which devices should be enumerated
- select the configuration number to use when enumerating the device

The device descriptor is passed to this callback to allow users to filter devices based on Vendor ID, Product ID, and class code.

Attention This callback must be non-blocking
Attention This callback must not submit any USB transfers

Param dev_desc **[in]** Device descriptor of the device to enumerate
Param bConfigurationValue **[out]** Configuration number to use when enumerating the device (starts with 1)
Return bool
- true: USB device will be enumerated
- false: USB device will not be enumerated

typedef struct *usb_transfer_s* **usb_transfer_t**

USB transfer structure.

This structure is used to represent a transfer from a software client to an endpoint over the USB bus. Some of the fields are made const on purpose as they are fixed on allocation. Users should call the appropriate USB Host Library function to allocate a USB transfer structure instead of allocating this structure themselves.

The transfer type is inferred from the endpoint this transfer is sent to. Depending on the transfer type, users should note the following:

- Bulk: This structure represents a single bulk transfer. If the number of bytes exceeds the endpoint's MPS, the transfer will be split into multiple MPS sized packets followed by a short packet.

- Control: This structure represents a single control transfer. This first 8 bytes of the data_buffer must be filled with the setup packet (see *usb_setup_packet_t*). The num_bytes field should be the total size of the transfer (i.e., size of setup packet + wLength).
- Interrupt: Represents an interrupt transfer. If num_bytes exceeds the MPS of the endpoint, the transfer will be split into multiple packets, and each packet is transferred at the endpoint's specified interval.
- Isochronous: Represents a stream of bytes that should be transferred to an endpoint at a fixed rate. The transfer is split into packets according to the each isoc_packet_desc. A packet is transferred at each interval of the endpoint. If an entire ISOC URB was transferred without error (skipped packets do not count as errors), the URB's overall status and the status of each packet descriptor will be updated, and the actual_num_bytes reflects the total bytes transferred over all packets. If the ISOC URB encounters an error, the entire URB is considered erroneous so only the overall status will updated.

Note: For Bulk/Control/Interrupt IN transfers, the num_bytes must be a integer multiple of the endpoint's MPS

Note: This structure should be allocated via usb_host_transfer_alloc()

Note: Once the transfer has be submitted, users should not modify the structure until the transfer has completed

---

typedef void (***usb_transfer_cb_t**)(*usb_transfer_t* *transfer)

USB transfer completion callback.

Enumerations

enum **usb_speed_t**

USB Standard Speeds.

Values:

enumerator **USB_SPEED_LOW**

USB Low Speed (1.5 Mbit/s)

enumerator **USB_SPEED_FULL**

USB Full Speed (12 Mbit/s)

enumerator **USB_SPEED_HIGH**

USB High Speed (480 Mbit/s)

enum **usb_transfer_type_t**

The type of USB transfer.

Note: The enum values need to match the bmAttributes field of an EP descriptor

---

Values:

enumerator **USB_TRANSFER_TYPE_CTRL**

enumerator **USB_TRANSFER_TYPE_ISOCHRONOUS**

enumerator **USB_TRANSFER_TYPE_BULK**

enumerator **USB_TRANSFER_TYPE_INTR**

enum **usb_transfer_status_t**

The status of a particular transfer.

Values:

enumerator **USB_TRANSFER_STATUS_COMPLETED**

The transfer was successful (but may be short)

enumerator **USB_TRANSFER_STATUS_ERROR**

The transfer failed because due to excessive errors (e.g. no response or CRC error)

enumerator **USB_TRANSFER_STATUS_TIMED_OUT**

The transfer failed due to a time out

enumerator **USB_TRANSFER_STATUS_CANCELED**

The transfer was canceled

enumerator **USB_TRANSFER_STATUS_STALL**

The transfer was stalled

enumerator **USB_TRANSFER_STATUS_OVERFLOW**

The transfer as more data was sent than was requested

enumerator **USB_TRANSFER_STATUS_SKIPPED**

ISOC packets only. The packet was skipped due to system latency or bus overload

enumerator **USB_TRANSFER_STATUS_NO_DEVICE**

The transfer failed because the target device is gone

Header File

- components/usb/include/usb/usb_types_ch9.h
- This header file can be included with:

```
#include "usb/usb_types_ch9.h"
```

- This header file is a part of the API provided by the `usb` component. To declare that your component depends on `usb`, add the following to your CMakeLists.txt:

```
REQUIRES usb
```

or

```
PRIV_REQUIRES usb
```

Unions

union **usb_setup_packet_t**

#include <usb_types_ch9.h> Structure representing a USB control transfer setup packet.

See Table 9-2 of USB2.0 specification for more details

Public Members

uint8_t **bmRequestType**

Characteristics of request

uint8_t **bRequest**

Specific request

uint16_t **wValue**

Word-sized field that varies according to request

uint16_t **wIndex**

Word-sized field that varies according to request; typically used to pass an index or offset

uint16_t **wLength**

Number of bytes to transfer if there is a data stage

struct *usb_setup_packet_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_SETUP_PACKET_SIZE]

Descriptor value

union **usb_standard_desc_t**

#include <usb_types_ch9.h> USB standard descriptor.

All USB standard descriptors start with these two bytes. Use this type when traversing over configuration descriptors

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

Descriptor Type

struct *usb_standard_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_STANDARD_DESC_SIZE]

Descriptor value

union **usb_device_desc_t**

#include <usb_types_ch9.h> Structure representing a USB device descriptor.

See Table 9-8 of USB2.0 specification for more details

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

DEVICE Descriptor Type

uint16_t **bcdUSB**

USB Specification Release Number in Binary-Coded Decimal (i.e., 2.10 is 210H)

uint8_t **bDeviceClass**

Class code (assigned by the USB-IF)

uint8_t **bDeviceSubClass**

Subclass code (assigned by the USB-IF)

uint8_t **bDeviceProtocol**

Protocol code (assigned by the USB-IF)

uint8_t **bMaxPacketSize0**

Maximum packet size for endpoint zero (only 8, 16, 32, or 64 are valid)

uint16_t **idVendor**

Vendor ID (assigned by the USB-IF)

uint16_t **idProduct**

Product ID (assigned by the manufacturer)

uint16_t **bcdDevice**

Device release number in binary-coded decimal

uint8_t **iManufacturer**

Index of string descriptor describing manufacturer

uint8_t **iProduct**

Index of string descriptor describing product

uint8_t **iSerialNumber**

Index of string descriptor describing the device’ s serial number

uint8_t **bNumConfigurations**

Number of possible configurations

struct *usb_device_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_DEVICE_DESC_SIZE]

Descriptor value

union **usb_config_desc_t**

#include <usb_types_ch9.h> Structure representing a short USB configuration descriptor.

See Table 9-10 of USB2.0 specification for more details

Note: The full USB configuration includes all the interface and endpoint descriptors of that configuration.

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

CONFIGURATION Descriptor Type

uint16_t **wTotalLength**

Total length of data returned for this configuration

uint8_t **bNumInterfaces**

Number of interfaces supported by this configuration

uint8_t **bConfigurationValue**

Value to use as an argument to the SetConfiguration() request to select this configuration

uint8_t **iConfiguration**

Index of string descriptor describing this configuration

uint8_t **bmAttributes**

Configuration characteristics

uint8_t **bMaxPower**

Maximum power consumption of the USB device from the bus in this specific configuration when the device is fully operational.

struct *usb_config_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_CONFIG_DESC_SIZE]

Descriptor value

union **usb_iad_desc_t**

#include <usb_types_ch9.h> Structure representing a USB interface association descriptor.

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

INTERFACE ASSOCIATION Descriptor Type

uint8_t **bFirstInterface**

Interface number of the first interface that is associated with this function

uint8_t **bInterfaceCount**

Number of contiguous interfaces that are associated with this function

uint8_t **bFunctionClass**

Class code (assigned by USB-IF)

uint8_t **bFunctionSubClass**

Subclass code (assigned by USB-IF)

uint8_t **bFunctionProtocol**

Protocol code (assigned by USB-IF)

uint8_t **iFunction**

Index of string descriptor describing this function

struct *usb_iad_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_IAD_DESC_SIZE]

Descriptor value

union **usb_intf_desc_t**

#include <usb_types_ch9.h> Structure representing a USB interface descriptor.

See Table 9-12 of USB2.0 specification for more details

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

INTERFACE Descriptor Type

uint8_t **bInterfaceNumber**

Number of this interface.

uint8_t **bAlternateSetting**

Value used to select this alternate setting for the interface identified in the prior field

uint8_t **bNumEndpoints**

Number of endpoints used by this interface (excluding endpoint zero).

uint8_t **bInterfaceClass**

Class code (assigned by the USB-IF)

uint8_t **bInterfaceSubClass**

Subclass code (assigned by the USB-IF)

uint8_t **bInterfaceProtocol**

Protocol code (assigned by the USB)

uint8_t **iInterface**

Index of string descriptor describing this interface

struct *usb_intf_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_INTF_DESC_SIZE]

Descriptor value

union **usb_ep_desc_t**

#include <usb_types_ch9.h> Structure representing a USB endpoint descriptor.

See Table 9-13 of USB2.0 specification for more details

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

ENDPOINT Descriptor Type

uint8_t **bEndpointAddress**

The address of the endpoint on the USB device described by this descriptor

uint8_t **bmAttributes**

This field describes the endpoint’ s attributes when it is configured using the bConfigurationValue.

uint16_t **wMaxPacketSize**

Maximum packet size this endpoint is capable of sending or receiving when this configuration is selected.

uint8_t **bInterval**

Interval for polling Isochronous and Interrupt endpoints. Expressed in frames or microframes depending on the device operating speed (1 ms for Low-Speed and Full-Speed or 125 us for USB High-Speed and above).

struct *usb_ep_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_EP_DESC_SIZE]

Descriptor value

union **usb_str_desc_t**

#include <usb_types_ch9.h> Structure representing a USB string descriptor.

Public Members

uint8_t **bLength**

Size of the descriptor in bytes

uint8_t **bDescriptorType**

STRING Descriptor Type

uint16_t **wData**[]

UTF-16LE encoded

struct *usb_str_desc_t*::[anonymous] **USB_DESC_ATTR**

USB descriptor attributes

uint8_t **val**[USB_STR_DESC_SIZE]

Descriptor value

Macros

USB_DESC_ATTR

USB_B_DESCRIPTOR_TYPE_DEVICE

Descriptor types from USB2.0 specification table 9.5.

USB_B_DESCRIPTOR_TYPE_CONFIGURATION

USB_B_DESCRIPTOR_TYPE_STRING

USB_B_DESCRIPTOR_TYPE_INTERFACE

USB_B_DESCRIPTOR_TYPE_ENDPOINT

USB_B_DESCRIPTOR_TYPE_DEVICE_QUALIFIER

USB_B_DESCRIPTOR_TYPE_OTHER_SPEED_CONFIGURATION

USB_B_DESCRIPTOR_TYPE_INTERFACE_POWER

USB_B_DESCRIPTOR_TYPE_OTG

Descriptor types from USB 2.0 ECN.

USB_B_DESCRIPTOR_TYPE_DEBUG

USB_B_DESCRIPTOR_TYPE_INTERFACE_ASSOCIATION

USB_B_DESCRIPTOR_TYPE_SECURITY

Descriptor types from Wireless USB spec.

USB_B_DESCRIPTOR_TYPE_KEY

USB_B_DESCRIPTOR_TYPE_ENCRYPTION_TYPE

USB_B_DESCRIPTOR_TYPE_BOS

USB_B_DESCRIPTOR_TYPE_DEVICE_CAPABILITY

USB_B_DESCRIPTOR_TYPE_WIRELESS_ENDPOINT_COMP

USB_B_DESCRIPTOR_TYPE_WIRE_ADAPTER

USB_B_DESCRIPTOR_TYPE_RPIPE

USB_B_DESCRIPTOR_TYPE_CS_RADIO_CONTROL

USB_B_DESCRIPTOR_TYPE_PIPE_USAGE

Descriptor types from UAS specification.

USB_SETUP_PACKET_SIZE

Size of a USB control transfer setup packet in bytes.

USB_BM_REQUEST_TYPE_DIR_OUT

Bit masks belonging to the bmRequestType field of a setup packet.

USB_BM_REQUEST_TYPE_DIR_IN

USB_BM_REQUEST_TYPE_TYPE_STANDARD

USB_BM_REQUEST_TYPE_TYPE_CLASS

USB_BM_REQUEST_TYPE_TYPE_VENDOR

USB_BM_REQUEST_TYPE_TYPE_RESERVED

USB_BM_REQUEST_TYPE_TYPE_MASK

USB_BM_REQUEST_TYPE_RECIP_DEVICE

USB_BM_REQUEST_TYPE_RECIP_INTERFACE

USB_BM_REQUEST_TYPE_RECIP_ENDPOINT

USB_BM_REQUEST_TYPE_RECIP_OTHER

USB_BM_REQUEST_TYPE_RECIP_MASK

USB_B_REQUEST_GET_STATUS

Bit masks belonging to the bRequest field of a setup packet.

USB_B_REQUEST_CLEAR_FEATURE

USB_B_REQUEST_SET_FEATURE

USB_B_REQUEST_SET_ADDRESS

USB_B_REQUEST_GET_DESCRIPTOR

USB_B_REQUEST_SET_DESCRIPTOR

USB_B_REQUEST_GET_CONFIGURATION

USB_B_REQUEST_SET_CONFIGURATION

USB_B_REQUEST_GET_INTERFACE

USB_B_REQUEST_SET_INTERFACE

USB_B_REQUEST_SYNCH_FRAME

USB_W_VALUE_DT_DEVICE

Bit masks belonging to the wValue field of a setup packet.

USB_W_VALUE_DT_CONFIG

USB_W_VALUE_DT_STRING

USB_W_VALUE_DT_INTERFACE

USB_W_VALUE_DT_ENDPOINT

USB_W_VALUE_DT_DEVICE_QUALIFIER

USB_W_VALUE_DT_OTHER_SPEED_CONFIG

USB_W_VALUE_DT_INTERFACE_POWER

USB_SETUP_PACKET_INIT_SET_ADDR(setup_pkt_ptr, addr)

Initializer for a SET_ADDRESS request.

Sets the address of a connected device

USB_SETUP_PACKET_INIT_GET_DEVICE_DESC(setup_pkt_ptr)

Initializer for a request to get a device's device descriptor.

USB_SETUP_PACKET_INIT_GET_CONFIG(setup_pkt_ptr)

Initializer for a request to get a device's current configuration number.

USB_SETUP_PACKET_INIT_GET_CONFIG_DESC(setup_pkt_ptr, desc_index, desc_len)

Initializer for a request to get one of the device's current configuration descriptor.

- desc_index indicates the configuration's index number
- Number of bytes of the configuration descriptor to get

USB_SETUP_PACKET_INIT_SET_CONFIG(setup_pkt_ptr, config_num)

Initializer for a request to set a device's current configuration number.

USB_SETUP_PACKET_INIT_SET_INTERFACE(setup_pkt_ptr, intf_num, alt_setting_num)

Initializer for a request to set an interface's alternate setting.

USB_SETUP_PACKET_INIT_GET_STR_DESC(setup_pkt_ptr, string_index, lang_id, desc_len)

Initializer for a request to get an string descriptor.

USB_STANDARD_DESC_SIZE

Size of dummy USB standard descriptor.

USB_DEVICE_DESC_SIZE

Size of a USB device descriptor in bytes.

USB_CLASS_PER_INTERFACE

Possible base class values of the bDeviceClass field of a USB device descriptor.

USB_CLASS_AUDIO

USB_CLASS_COMM

USB_CLASS_HID

USB_CLASS_PHYSICAL

USB_CLASS_STILL_IMAGE

USB_CLASS_PRINTER

USB_CLASS_MASS_STORAGE

USB_CLASS_HUB

USB_CLASS_CDC_DATA

USB_CLASS_CSCID

USB_CLASS_CONTENT_SEC

USB_CLASS_VIDEO

USB_CLASS_WIRELESS_CONTROLLER

USB_CLASS_PERSONAL_HEALTHCARE

USB_CLASS_AUDIO_VIDEO

USB_CLASS_BILLBOARD

USB_CLASS_USB_TYPE_C_BRIDGE

USB_CLASS_MISC

USB_CLASS_APP_SPEC

USB_CLASS_VENDOR_SPEC

USB_SUBCLASS_VENDOR_SPEC

Vendor specific subclass code.

USB_CONFIG_DESC_SIZE

Size of a short USB configuration descriptor in bytes.

Note: The size of a full USB configuration includes all the interface and endpoint descriptors of that configuration.

USB_BM_ATTRIBUTES_ONE

Bit masks belonging to the bmAttributes field of a configuration descriptor.

Must be set

USB_BM_ATTRIBUTES_SELFPOWER

Self powered

USB_BM_ATTRIBUTES_WAKEUP

Can wake-up

USB_BM_ATTRIBUTES_BATTERY

Battery powered

USB_IAD_DESC_SIZE

Size of a USB interface association descriptor in bytes.

USB_INTF_DESC_SIZE

Size of a USB interface descriptor in bytes.

USB_EP_DESC_SIZE

Size of a USB endpoint descriptor in bytes.

USB_B_ENDPOINT_ADDRESS_EP_NUM_MASK

Bit masks belonging to the bEndpointAddress field of an endpoint descriptor.

USB_B_ENDPOINT_ADDRESS_EP_DIR_MASK

USB_W_MAX_PACKET_SIZE_MPS_MASK

Bit masks belonging to the wMaxPacketSize field of endpoint descriptor.

USB_W_MAX_PACKET_SIZE_MULT_MASK

USB_BM_ATTRIBUTES_XFERTYPE_MASK

Bit masks belonging to the bmAttributes field of an endpoint descriptor.

USB_BM_ATTRIBUTES_XFER_CONTROL

USB_BM_ATTRIBUTES_XFER_ISOC

USB_BM_ATTRIBUTES_XFER_BULK

USB_BM_ATTRIBUTES_XFER_INT

USB_BM_ATTRIBUTES_SYNCTYPE_MASK

USB_BM_ATTRIBUTES_SYNC_NONE

USB_BM_ATTRIBUTES_SYNC_ASYNC

USB_BM_ATTRIBUTES_SYNC_ADAPTIVE

USB_BM_ATTRIBUTES_SYNC_SYNC

USB_BM_ATTRIBUTES_USAGETYPE_MASK

USB_BM_ATTRIBUTES_USAGE_DATA

USB_BM_ATTRIBUTES_USAGE_FEEDBACK

USB_BM_ATTRIBUTES_USAGE_IMPLICIT_FB

USB_EP_DESC_GET_XFERTYPE(desc_ptr)

Macro helpers to get information about an endpoint from its descriptor.

USB_EP_DESC_GET_EP_NUM(desc_ptr)

USB_EP_DESC_GET_EP_DIR(desc_ptr)

USB_EP_DESC_GET_MPS(desc_ptr)

USB_EP_DESC_GET_MULT(desc_ptr)

USB_EP_DESC_GET_SYNCTYPE(desc_ptr)

USB_EP_DESC_GET_USAGETYPE(desc_ptr)

USB_STR_DESC_SIZE

Size of a short USB string descriptor in bytes.

Enumerations

enum **usb_device_state_t**

USB2.0 device states.

See Table 9-1 of USB2.0 specification for more details

Note: The USB_DEVICE_STATE_NOT_ATTACHED is not part of the USB2.0 specification, but is a catch all state for devices that need to be cleaned up after a sudden disconnection or port error.

Values:

enumerator **USB_DEVICE_STATE_NOT_ATTACHED**

The device was previously configured or suspended, but is no longer attached (either suddenly disconnected or a port error)

enumerator **USB_DEVICE_STATE_ATTACHED**

Device is attached to the USB, but is not powered.

enumerator **USB_DEVICE_STATE_POWERED**

Device is attached to the USB and powered, but has not been reset.

enumerator **USB_DEVICE_STATE_DEFAULT**

Device is attached to the USB and powered and has been reset, but has not been assigned a unique address. Device responds at the default address.

enumerator **USB_DEVICE_STATE_ADDRESS**

Device is attached to the USB, powered, has been reset, and a unique device address has been assigned. Device is not configured.

enumerator **USB_DEVICE_STATE_CONFIGURED**

Device is attached to the USB, powered, has been reset, has a unique address, is configured, and is not suspended. The host may now use the function provided by the device.

enumerator **USB_DEVICE_STATE_SUSPENDED**

Device is, at minimum, attached to the USB and is powered and has not seen bus activity for 3 ms. It may also have a unique address and be configured for use. However, because the device is suspended, the host may not use the device's function.

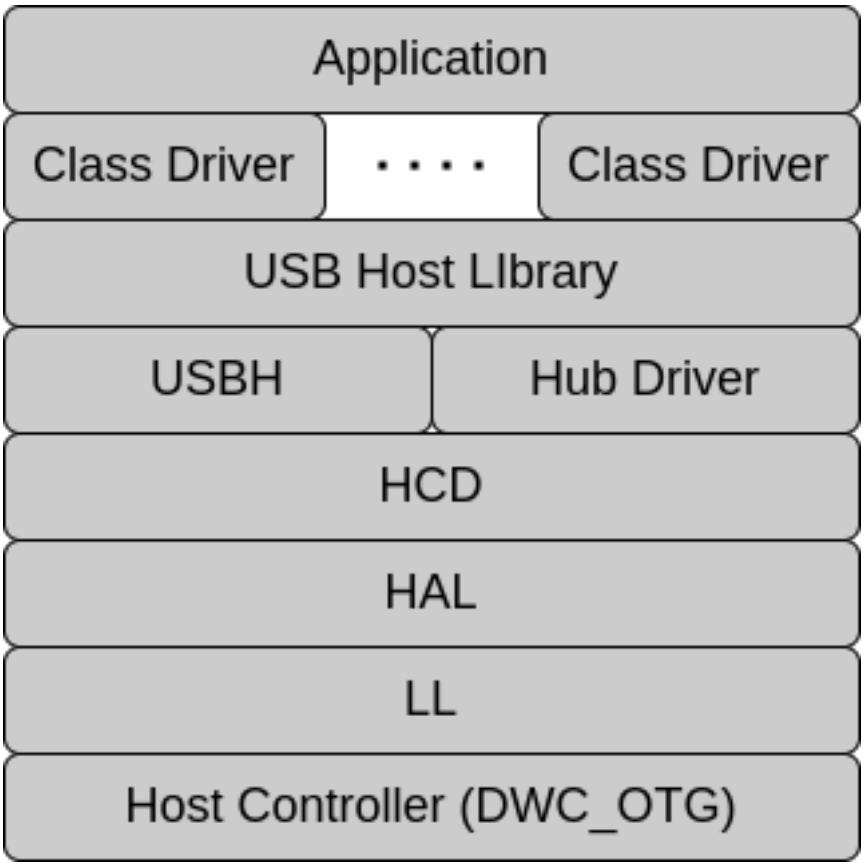
Maintainers Notes

Note: For more details regarding the internal implementation details of the USB Host stack, please refer to *USB Host Maintainers Notes (Introduction)*.

USB Host Maintainers Notes (Introduction)

This document contains information regarding the implementation details of the USB Host stack. This document is intended for the maintainers and third-party contributors of the USB Host stack. Users of the USB Host stack should refer to *USB Host* instead.

Warning: The implementations details of the USB Host stack is categorized as private API. Thus, all layers (other than the USB Host Library) do not adhere to *ESP-IDF's versioning scheme* (i.e., breaking changes are permitted).



This document is split into the following sections:

USB Host Maintainers Notes (Design Guidelines)

Design Considerations The design of the Host Stack takes into account the following design considerations:

Limited Hardware Resources:

The embedded nature of Host Stack means limited hardware resources (such as memory and processing power) when compared to larger host systems.

USB2.0 Chapter 10:

Chapter 10 of the USB 2.0 specification specifies certain requirements of USB Host systems, in particular the required layers of the USB Host's system software.

Diverse Use Case Complexities:

The embedded nature of the Host Stack also means a wide range of use cases with differing complexities. Some USB Host applications aim to only support a single vendor specific device, whereas other applications require support for a wide range of devices of different classes.

Requirements Given the design considerations above, the Host Stack was designed with the following set of requirements:

DMA Support **Requirement: The Host Stack must support DMA.**

The Host Stack must support DMA in order to reduce CPU's workload. DMA support allows the automatic copying of USB transfer data to/from the Host Controller without CPU intervention. This is especially critical given the embedded nature of the CPU (i.e., lower CPU frequencies) and large maximum data payloads of USB transfers (e.g., 1023 bytes for isochronous transfers).

Minimize Memory Copies **Requirement: The Host Stack should minimize the amount of memory copies when passing data between layers.**

Various data and objects (e.g., USB transfers) need to be passed between multiple layers of the Host Stack. The Host Stack should minimize the amount of memory copies that occur between layers by allocating the data's/object's memory once, and simply passing a pointer to that data/object between the layers. Therefore, the Host Stack requires some standardized data types shared across multiple layers (see USB2.0 Section 10.3.4).

Event Driven **Requirement: The Host Stack must allow for event driven operation (i.e., the Host Stack's API must not be polling).**

The Host Stack needs to support some CPU intensive use application scenarios such as video streaming (i.e., UVC class). Therefore, the Host Stack should minimize CPU usage by allowing for completely event driven operation, thus reserving the majority of CPU time for the application itself (i.e., video encoding/decoding in this case).

The Host Stack needs to communicate events across the layers using interrupts, callbacks, and FreeRTOS synchronization primitives (e.g., queues and semaphores).

No Task Creation **Requirement: All layers of the Host Stack below (and including) the Host Library layer must not create any tasks.**

Task stacks are generally one of the most memory intensive parts of an ESP-IDF applications. Given the wide range of applications scenarios, the number of tasks created (and their stack sizes) can vary greatly. For example...

- applications that require low latency or high throughput (such as isochronous transfers) may choose to create a dedicated task to handle those transfers in order to minimize latency.

- applications that do not have strict latency requirements (such as bulk transfers) may choose to handle those transfers from a shared task in order to save some memory.

Therefore, all layers of the Host Stack below (and including) the Host Library layer **must not** create any tasks. Instead, these layers should expose handlers functions to be called from tasks created by the upper layers. Task creation will be delegated to the class driver or application layer.

Operable at Different Layers Given the wide range of use case complexities, the Host Stack must be operable at different layers, allowing users to use the Host Stack at a lower layer (e.g., the HCD or HAL) or at a higher layer (e.g., a class driver).

Being operable at different layers allows the users to decide on the appropriate trade-off between convenience, control, and optimization for their application when using the Host Stack. For example...

- Host Stack applications that support a dedicated custom device may want to use a lower level of abstraction for better optimization, control, and simpler API.
- Host Stack applications that need to support a wide range of device classes requires the full Host Stack so that device enumeration is automatically handled.

Coding Conventions The Host Stack follows the following set of coding rules/guidelines for better code readability and maintainability:

Symbols Use Layer Name As Prefix For each layer of the Host Stack, the symbols exposed by that layer (i.e., functions, types, macros) must be prefixed with that layer's name. For example, the symbols exposed by the HCD layer will be prefixed `hcd_.../HCD_...`.

However, internal symbols (e.g., static functions) **should not** be prefixed with their layer's name. This makes it easier to differentiate between internal and external symbols when modifying that layer's source code.

Critical Section Functions Prefixed With _ In each layer of the Host Stack, there are various static functions that must be called inside a critical section. The names of these functions are prefixed with _ (e.g., `_func_called_from_crit()`) to make it easier for maintainers to differentiate which functions should be called from critical sections. For example...

```
some_func();  // Called outside critical section
taskENTER_CRITICAL(&some_lock);
_some_func_crit();  // Called inside critical section. _ prefix makes it easier to␣
↪differentiate
taskEXIT_CRITICAL(&some_lock);
```

Grouping Structure Members by Locking Mechanism Some layers of the Host Stack utilize multiple locking schemes (e.g., critical sections and task mutexes) to ensure thread safety, where each locking scheme offers a different level of protection. However, member variables of the same object can be protected by different locking scheme. Therefore, to clearly demarcate the different locking schemes and their associated variables, structure members are grouped by locking scheme as nested structures.

Table 6: Locking Scheme

Locking Scheme	Nested Structure	Description
Critical Sections	`dynamic`	Shared data accessed from both a task context and ISR context are protected by a critical section.
Task Mutexes	`mux_protected`	Shared data accessed from only a task context are protected by a FreeRTOS Mutex
Single Thread	`single_thread`	Data that is only ever accessed by the same task do not require the use of any locks.
Constant	`constant`	Constant data is set once during the object's instantiation and never changed again. Thus, any task or ISR can freely the constant data without the use of locks, so long as the variable is never written to.

Grouping structure members by locking scheme makes the code more maintainable as it makes clear which locking scheme is required when accessing a particular member variable, as demonstrated in the code snippet below:

```
typedef struct some_obj some_obj_t;

some_obj_t obj;

// Accessing dynamic members requires critical section
taskENTER_CRITICAL(&some_lock);
obj.dynamic.varA = 1;
taskEXIT_CRITICAL(&some_lock);

// Accessing mutex protected members requires taking the mutex
xSemaphoreTake(&some_mux, portMAX_DELAY);
obj.mux_protected.varB = 1;
xSemaphoreGive(&some_mux);

// Accessing single thread members does not require locking so long as this is the␣
↪only task to access it
obj.single_thread.varC = 1;

// Accessing constant members requires no locking. But only read access is allowed
int local_var = obj.constant.varD;
```

USB Host Maintainers Notes (Architecture) The Host Stack is roughly split into multiple layers of abstraction, with each layer representing different USB concepts and a different level of USB Host operation. For example, a higher layer may present an abstraction of devices and application data transfers, whereas a lower layer may present an abstraction of endpoints and USB transfers.

Layer Descriptions The layers of the Host Stack are described in the following table. The layers are ordered from lowest layer (i.e, furthest away from the user) to highest layer (i.e., closest to the user).

Table 7: Host Stack Layers

Layer	Files	Description
Host Controller (DWC_OTG)	N/A	This layer represents the USB Controller Hardware of the ESP32-S3. The API presented by this layer is the register interface of the controller.
LL	`usbh_ll.h`	The LL (Low Level) layer abstracts the basic register access of the USB controller according to ESP-IDF's *Hardware Abstraction API Guidelines*. In other words, this layer provides APIs to access the controller's registers and format/parse the controller's DMA descriptors.
HAL	`usbh_hal.h`, `usbh_hal.c`	The HAL (Hardware Abstraction Layer) abstracts the operating steps of the USB controller into functions according to ESP-IDF's *Hardware Abstraction API Guidelines*. This layer also abstracts the controller's host port and host channels, and provides APIs to operate the them.
HCD	`hcd.h`, `hcd.c`	The HCD (Host Controller Driver) acts as hardware agnostic API for all USB controllers (i.e., an API that can theoretically be used with any USB controller implementation). This layer also abstracts the root port (i.e., root hub) and USB pipes.
USBH and Hub Driver	`usbh.h`, `usbh.c`	The USBH (USB Host Driver) layer is equivalent to the USBD layer described in chapter 10 of the USB2.0 specification. The USBH presents an abstraction of USB devices, internally manages a list of connected devices (i.e., device pool), and also arbitrates device sharing between clients (i.e., tracks which endpoints are in use and also presents a shared endpoint 0).
Hub Driver	`hub.h`, `hub.c`	The Hub Driver layer acts as a special client of the USBH that is responsible for handling device attachment/detachment, and notifying the USBH of such events. For device attachment, the Hub Driver also handles the enumeration process as well.
USB Host Library	`usb_host.h`, `usb_host.c`	The USB Host Library layer is the lowest public API layer of the Host Stack and presents the concept of USB Host Clients. The abstraction of clients allows for multiple class drivers to coexist simultaneously (where each class roughly maps to a single client) and also acts as a mechanism for division of labor (where each client is responsible for its own processing and event handling).
Host Class Drivers	See the ESP-IDF Extra Components repository or the USB Host examples in ESP-IDF (via peripherals/usb/host).	The Host Class Drivers implement the host side of a particular device class (e.g., CDC, MSC, HID). The exposed API is specific to each class driver.

Layer Dependencies The Host Stack roughly follows a top to bottom hierarchy with inter-layer dependencies. Given layers A (highest), B, and C (lowest), the Host Stack has the following inter-layer dependency rules:

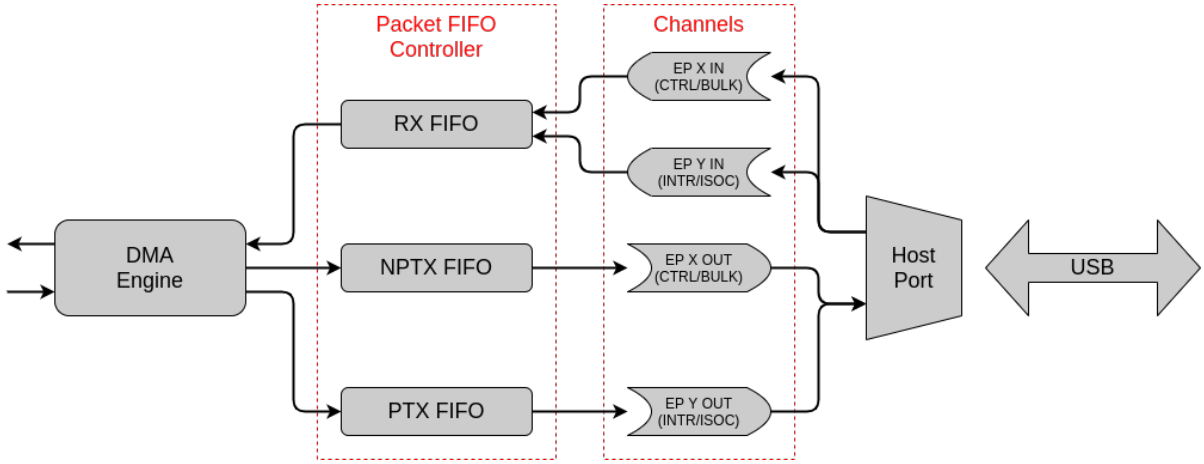
- a particular layer can use the API of any layer directly below (Layer A using layer B is allowed)
- a particular layer can use the API of any layer indirectly below (Layer A using layer C is allowed) i.e., skipping layers.
- a particular layer must not use the API of any layer above (Layer C using layer A/B is forbidden)

Note: Layer skipping is permitted in order to circumvent the need to repeat the same abstraction across multiple layers. For example, the abstraction of pipes are presented at the HCD layer but are used by multiple layers above.

USB Host Maintainers Notes (DWC_OTG Controller) The ESP32-S3 uses a DesignWare USB 2.0 On-the-Go Controller (henceforth referred to as DWC_OTG in this document) as its underlying hardware controller, where the DWC_OTG operates in Host Mode with Scatter/Gather DMA enabled.

Note: This section only summarizes the operation of the DWC_OTG operation in Host Mode at a high level. For full details of the DWC_OTG, refer to the DWC_OTG Databook and Programming Guide.

Host Mode Operating Model A simplified version of the operating model of the DWC_OTG in Host Mode is illustrated in the diagram below. The diagram contains some of the key concepts and terms regarding DWC_OTG Host Mode.



Note: Refer to Databook section 2.1.4 (Host Architecture) for more details

Host Port The Host Port represents the single USB port provided by the DWC_OTG (in USB terms, this can be thought a single USB port of the root hub of the bus). The Host Port can only connect to a single device, though more devices can be connected via hub devices.

The Host Port is responsible for:

- detecting direct device connections/disconnections
- detecting the speed of the directly connected device
- issuing various bus signals (such as suspend, resume, reset)

Host Channels In Host Mode, the DWC_OTG uses channels to communicate with device endpoints, where one channel maps to a particular endpoint (in USB terms, channels are the hardware representation of pipes). For example, a channel will map to EP 1 OUT. Each channel has its own set of CSRs (Control and Status Registers) so that they can independently configured and controlled independently. A channel's CSRs are used to...

- specify the details of the channel's target endpoint (e.g., device address, EP number, transfer type, direction)
- start a transfer on the channel (e.g., by setting up DMA descriptors)

When using Scatter/Gather DMA, transfers on Host Channels are completely event driven. Users simply fill out the appropriate DMA descriptors, fill in the channel's CSRs, then enable the channel. The channel will then generate an interrupt when the transfer completes.

Data FIFOs In Host Mode, the DWC_OTG uses multiple FIFOs as a staging area for the data payloads of USB transfers. When using DMA, the DMA engine will copy data between the TX/RX FIFOs and ESP32-S3's internal memory:

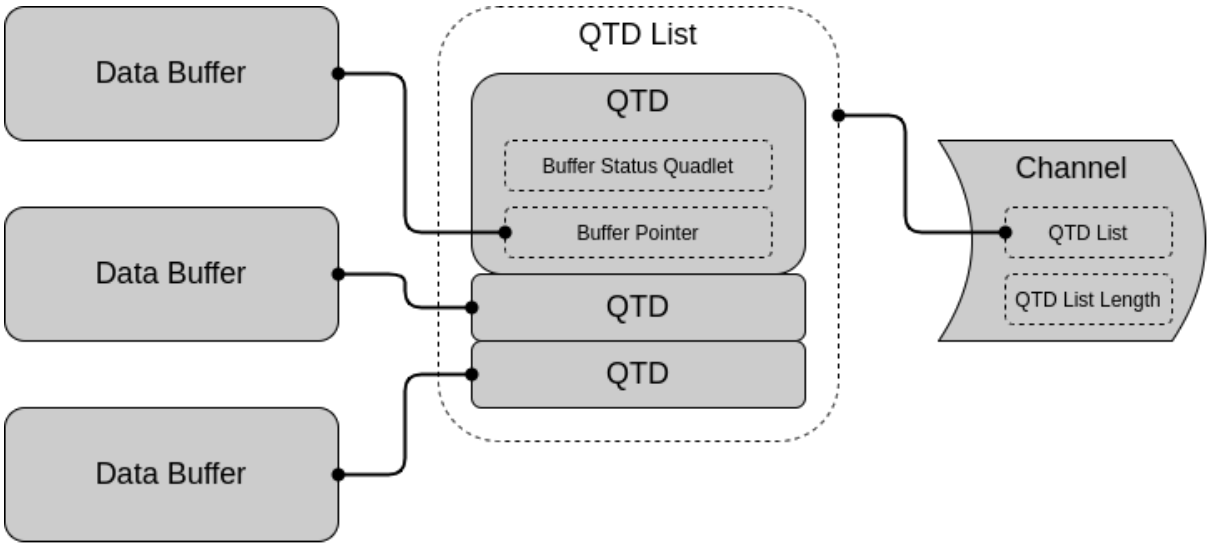
- For an OUT transfer, the transfer's data payload is copied from main memory to one of the TX FIFOs by DMA. The MAC Layer will then transmit that data payload in accordance to USB packet formatting.
- For an IN transfer, the MAC Layer will parse the received USB packet and store the received data payload in the RX FIFO. The data is then copied to main memory by DMA.

The destination FIFO depends on the direction and transfer type of the channel:

- All IN channel data goes to the RX FIFO
- All non-periodic (i.e., Control and Bulk) OUT channel data goes to the Non-periodic TX (NPTX) FIFO
- All periodic (i.e., Interrupt and Isochronous) OUT channel data goes to the Periodic TX (PTX) FIFO

Note: The separation of non-periodic and periodic OUT channels to the NPTX and PTX FIFOs is due to the periodic nature of Interrupt and Isochronous endpoints (specified by the `bInterval` value of the endpoint). The DWC_OTG automatically schedules these periodic transfers, thus a separate PTX FIFO allows these periodic transfers to be staged separately.

DMA Engine The DMA engine is responsible for copying data between the FIFOs and main memory. In Host Mode Scatter/Gather DMA, a particular channel can carry out multiple transfers automatically without software intervention. The following diagram illustrates the DWC_OTG Host Mode Scatter/Gather DMA Memory Structures.



- Each USB transfer is described by a Queue Transfer Descriptor (QTD). Each QTD consists of:
  - A 32-bit Buffer Status Quadlet specifying details of the transfer, and also reports the status of the transfer on completion. The Buffer Status Quadlet has bits to specify whether the QTD should generate an interrupt and/or halt the channel on completion.
  - A 32-bit pointer to the data buffer containing the data payload for OUT transfers, or an empty buffer used to store the data payload for IN transfers.
- The data payload of each QTD can be larger than the MPS (Maximum Packet Size) of its target endpoint. The DWC_OTG hardware automatically handles splitting of the transfer into multiple transactions.
- Multiple QTDs can be placed into a single QTD List. A channel will then execute each QTD in the list automatically, and optionally loop back around if configured to do so.
- Before a channel starts data transfer, it is configured with a QTD list (and QTD list length). Once the channel is enabled, USB transfers are executed automatically by the hardware.
- A channel can generate interrupts (configurable) on completion of a particular QTD, or an entire QTD list.

Note: Refer to Programming Guide section 6.2.1 (Descriptor Memory Structures) for more details

Hardware Configuration The DWC_OTG IP is configurable. The notable Host related configurations of the ESP32-S3's DWC_OTG are listed below:

Table 8: ESP32-S3's DWC_OTG Configuration

Description	Configuration
Host and Device Mode support with OTG	`OTG_MODE = 0`
Full Speed (FS) and Low Speed (LS) support	`OTG_FSPHY_INTERFACE = 1, OTG_HSPHY_INTERFACE = 0`
Internal DMA controller with Scatter/Gather DMA	`OTG_ARCHITECTURE = 2, OTG_EN_DESC_DMA = 1`
FS Hubs are supported but HS Hub are not (i.e., split transfers not supported)	`OTG_SINGLE_POINT = 0`
8 Host Mode channels	`OTG_NUM_HOST_CHAN = 8`
All transfer types supported, including ISOC and INTR OUT transfers	`OTG_EN_PERIO_HOST = 1`
Dynamically sized Data FIFO of 1024 bytes (256 lines)	`OTG_DFIFO_DYNAMIC = 1, OTG_DFIFO_DEPTH = 256`

Scatter/Gather DMA Transfer The basic operating procedure for Host channels transfers consists of the following steps:

1. Prepare data buffers, QTDs, and QTD list. In particular, which QTDs should halt the channel (and generate an interrupt) on completion.
2. Set channel/endpoint characteristics via CSRs (such as EP address, transfer type, EP MPS etc).
3. Set channel's QTD list related CSRs (such as QTD list pointer and QTD list length) and channel interrupt CSRs
4. Enable the channel. Transfers are now handled automatically by hardware using DMA.
5. The Channel generates an interrupt on a channel event (e.g., QTD completion or channel error).
6. Parse the channel interrupt to determine what event occurred.
7. Parse the QTDs to determine the result of each individual transfer.

However, there are some minor differences in channel operation and QTD list usage depending on the transfer type.

Bulk Bulk transfers are a simplest. Each QTD represents a bulk transfer of a particular direction, where the DWC_OTG automatically splits a particular QTD into multiple MPS sized transactions. Thus it is possible to fill a QTD list with multiple bulk transfers, and have the entire list executed automatically (i.e., only interrupt on completion of the last QTD).

Control Control transfers are more complicated as they are bi-directional (i.e., each control transfer stage can have a different direction). Thus, a separate QTD is required for each stage, and each QTD must halt the channel on completion. Halting the channel after each QTD allows changing the channel's direction to be changed by reconfiguring the channel's CSRs. Thus a typical control transfer consists of 3 QTDs (one for each stage).

Interrupt In accordance with the USB2.0 specification, interrupt transfers executes transactions at the endpoints specified service period (i.e., `bInterval`). A particular interrupt endpoint may not execute more than one interrupt transaction within a service period. The service period is specified in number of microframes/frames, thus a particular interrupt endpoint will generally execute one transaction every Nth microframe/frame until the transfer is complete. For interrupt channels, the service period of a particular channel (i.e., `bInterval`) is specified via the Host Frame List (see section 6.5 of programming guide for more details).

Note: HS USB allows an interrupt endpoint to have 3 interrupt transactions in a single microframe. See USB2.0 specification section 5.7.3 (Interrupt Transfer Packet Size Constraints) for more details.

Thus, interrupt transfers in Host Mode Scatter/Gather DMA have the following peculiarities:

- If a QTD payload is larger than the endpoint's MPS, the channel will automatically split the transfer into multiple MPS sized transactions (similar to bulk transfers). However, each transaction **is executed at endpoint's specified service period** (i.e., one transaction per `bInterval`) until the transfer completes.
- For Interrupt IN transfers, if a short packet is received (i.e., transaction's data payload is < MPS), this indicates that the endpoint has no more data to send. In this case:
  - the channel generates an extra channel interrupt even if the transfer's QTD did not set the IOC (interrupt on complete) bit.
  - however, the channel is not halted even if this extra channel interrupt is generated.
  - software must then use this extra interrupt to manually halt the interrupt channel (thus canceling any remaining QTDs in the QTD list).

Note: Due to the interrupt transfer peculiarities, it may be easier for software allocate a QTD for each transaction instead of an entire transfer.

Isochronous In accordance with the USB2.0 specification, isochronous transfers executes transactions at the endpoints specified service period (i.e., `bInterval`) in order to achieve a constant rate of data transfer. A particular isochronous endpoint may not execute more than one isochronous transaction within a service period. The service period is specified in number of microframes/frames, thus a particular isochronous endpoint will generally execute one transaction every Nth microframe/frame until the transfer is complete. For isochronous channels, the service period of a particular channel (i.e., `bInterval`) is specified via the Host Frame List (see section 6.5 of programming guide for more details).

However, unlike interrupt transactions, isochronous transactions are not retried on failure (or NAK), due to the need to maintain the constant data rate.

Note: HS USB allows an isochronous endpoint to have 3 interrupt transactions in a single microframe. See USB2.0 specification section 5.6.3 (Isochronous Transfer Packet Size Constraints) for more details.

Thus, isochronous transfers in Host Mode Scatter/Gather DMA have the following peculiarities:

- A QTD must be allocated for each microframe/frame. However, non-service service period QTDs should be left blank (i.e., only ever Nth QTD should be filled if the channel's service period is every Nth microframe/frame).
- **Each filled QTD must represent a single transaction instead of a transfer**.
- Because isochronous transactions are not retried on failure, the status each completed QTD must be checked.

Supplemental Notes Some of the DWC_OTG's behaviors are not mentioned in the Databook or Programming Guide. This section describes some of those behaviors that are relevant to the Host stack's implementation.

Port Errors Do Not Trigger a Channel Interrupt If a port error occurs (such as a sudden disconnection or port over-current) while there are one or more active channels...

- The active channels remains active (i.e., `HCCHAR.ChEna` remains set) and no channel interrupts are generated.
- Channels could in theory be disabled by setting `HCCHAR.ChDis`, but this does not work for Isochronous channels as the channel disabled interrupt is never generated.

Therefore, on port errors, a controller soft reset should be used to ensure all channels are disabled.

Port Reset Interrupts

- When the DWC_OTG issues a reset signal on its port, and during the reset signal the device disconnects, the disconnection interrupt (i.e., `HPRT.PrtConnDet`) is not generated until the reset is deasserted.
- When resetting an already enabled port (i.e., `HPRT.PrtEna`) such as a second reset during enumeration or a run-time reset, a Port Enable/Disable Change interrupt (i.e., `HPRT.PrtEnChng`) is generated both on the assertion and deassertion of the reset signal.

USB Host Driver (USBH)

Introduction The USB Host Driver (henceforth referred to as USBH) provides a USB Host software interface which abstracts USB devices. The USBH interface provides APIs to...

- manage the device pool (i.e., adding and removing devices)
- address and configure a device (i.e., setting device and configuration descriptors)
- submit transfers to a particular endpoint of a device

Requirements

USB Specification Requirements Chapter 10 of the USB 2.0 specification outlines some requirements of the USBH (referred to as USBD in the specification). The design of the USBH takes into consideration these requirements from the specification.

- Default pipe of a device is owned by the USBH
- All other pipes are owned and managed by clients of the USBH
- USBH interface must provide the following services
  - Configuration and command mechanism
  - Transfer services via both command and pipe mechanisms
  - Event notification
  - Status reporting and error recovery

Host Stack Requirements In addition to the USB 2.0 specification requirements, the USBH also takes into consideration the requirements set for the overall Host Stack (see *USB Host Maintainers Notes (Design Guidelines)*):

- USBH must not instantiate any tasks/threads
- USBH must be event driven, providing event callbacks and an event processing function

Implementation & Usage

Events & Processing The USBH is completely event driven and all event handling is done via then `usbh_process()` function. The `usbh_config_t.proc_req_cb` callback provided on USBH installation will be called when processing is required. Typically, `usbh_process()` will be called from a dedicated thread/task.

The USBH exposes the following event callbacks:

- `usbh_event_cb_t` used to indicate various events regarding a particular device and control transfers to EP0. This callback is called from the context of `usbh_process()`
- `usbh_ep_cb_t` used to indicate events for all other endpoints. This callback is not called from the `usbh_process()` context (currently called from an HCD interrupt context).

Device Pool The USBH keeps track of all currently connected devices by internally maintaining a device pool (simply a linked list) where each device is represented by a device object.

The USB 2.0 specification "assumes a specialized client of the USBD, called a hub driver, that acts as a clearinghouse for the addition and removal of devices from a particular hub". As a result, the USBH is completely reliant on an external client(s) (typically a Hub Driver) to inform the USBH of device addition and removal. The USBH provides the following APIs for device addition and removal:

- `usbh_devs_add()` which will allocate a new device object and add it to the device pool. The newly added device will be unenumerated, meaning the device object will...
  - be assigned to address 0
  - have no device and configuration descriptor
- `usbh_devs_remove()` which will indicate to the USBH that a device has been removed (such as due to a disconnection or a port error).
  - If the device is not currently opened (i.e., used by one or more clients), the USBH will free the underlying device object immediately.
  - If the device is currently opened, a `USBH_EVENT_DEV_GONE` event will be propagated and the device will be flagged for removal. The last client to close the device will free the device object.
  - When a device object is freed, a `USBH_EVENT_DEV_FREE` event will be propagated. This event is used to indicate that the device's upstream port can be recycled.

Device Enumeration Newly added devices will need to be enumerated. The USBH provides various `usbh_dev_set_...()` functions to enumerate the device, such as assigning the device's address and setting device/configuration/string descriptors. Given that USBH devices can be shared by multiple clients, attempting to enumerate a device while another client has opened the device can cause issues.

Thus, before calling any `usbh_dev_set_...()` enumeration function, a device must be locked for enumeration by calling `usbh_dev_enum_lock()`. This prevents the device from being opened by any other client but the enumerating client.

After enumeration is complete, the enumerating client can call `usbh_devs_trigger_new_dev_event()` to propagate a `USBH_EVENT_NEW_DEV` event.

Device Usage Clients that want to use a device must open the device by calling `usbh_devs_open()` and providing the device's address. The device's address can either be obtained from a `USBH_EVENT_NEW_DEV` event or by calling `usbh_devs_addr_list_fill()`.

Opening a device will do the following:

- Return a `usb_device_handle_t` device handle which can be used to refer to the device in various USBH functions
- Increment the device's internal `open_count` which indicates how many clients have opened the device. As long as `open_count > 0`, the underlying device object will not be freed, thus guaranteeing that the device handle refers to a valid device object.

Once a client no longer needs to use a device, the client should call `usbh_devs_close()` thus invalidating the device handle.

Note: Most device related APIs accept `usb_device_handle_t` as an argument, which means that the calling client must have previously opened the device to obtain the device handle beforehand. This design choice is intentional in order to enforce an "open before use" pattern.

However, a limited set of APIs (e.g., `usbh_devs_remove()`) refer to devices using a Unique Identifier (`uid`) which is assigned on device addition (see `usbh_devs_add()`). The use of `uid` in these functions allows their callers to refer to a device **without needing to open it** due to the lack of a `usb_device_handle_t`.

As a result, it is possible that a caller of a `uid` function may refer to a device that has already been freed. Thus, callers should account for a fact that these functions may return *`ESP_ERR_NOT_FOUND`*.

Endpoints & Transfers USBH supports transfer to default (i.e., EP0) and non-default endpoints.

For non-default endpoints:

- A client must first allocate the endpoint by calling `usbh_ep_alloc()` which assigns a `usbh_ep_cb_t` callback and returns a `usbh_ep_handle_t` endpoint handle so that the endpoint can be referred to.
- A client can then enqueue a `urb_t` transfer to the endpoint by calling `usbh_ep_enqueue_urb()`.
- The `usbh_ep_cb_t` callback is called to indicate transfer completion
- The client must then dequeue the transfer using `usbh_ep_dequeue_urb()`

Default endpoints are owned and managed by the USBH, thus API for control transfers are different:

- EP0 is always allocated for each device, thus clients do no need to allocate EP0 or provide an endpoint callback.
- Clients call should call `usbh_dev_submit_ctrl_urb()` to submit a control transfer to a device's EP0.
- A `USBH_EVENT_CTRL_XFER` event will be propagated when the transfer is complete
- Control transfers do not need to be dequeued

Todo:

- USB Host Maintainers Notes (HAL & LL)
- USB Host Maintainers Notes (HCD)
- USB Host Maintainers Notes (Hub)
- USB Host Maintainers Notes (USB Host Library)

Introduction The ESP-IDF USB Host Stack allows the ESP32-S3 to operate as a USB Host. Operating as a USB Host allows the ESP32-S3 to communicate with a wide range of USB devices. However, most USB Host Stack implementations do not run on embedded hardware (i.e., runs on PCs and smartphones), thus have comparatively more resources (i.e., memory and CPU speed).

The implementation of the ESP-IDF USB Host Stack (henceforth referred to as the Host Stack) takes into account the embedded nature of the ESP32-S3 which is reflected in various aspects of the Host Stack's design.

Features & Limitations **The Host Stack currently supports the following notable features:**

- Supports FS (Full Speed) and LS (Low Speed) devices
- Supports all transfer types (Control, Bulk, Isochronous, and Interrupt)
- Automatically enumerates connected devices
- Allows multiple class drivers (i.e., Clients of the USB Host Library) to run simultaneously and share the same device (i.e., composite devices)

The Host Stack currently has the following notable limitations:

- No HS (High Speed) support
- No Hub support (currently only supports a single device)

Code examples for this API section are provided in the peripherals directory of ESP-IDF examples.

2.7 Project Configuration

2.7.1 Introduction

The esp-idf-kconfig package that ESP-IDF uses is based on kconfiglib, which is a Python extension to the Kconfig system. Kconfig provides a compile-time project configuration mechanism and offers configuration options of several types (e.g., integers, strings, and Booleans). Kconfig files specify dependencies between options, default values of options, the way options are grouped together, etc.

For the full list of available features, please see Kconfig and kconfiglib extensions.

2.7.2 Project Configuration Menu

Application developers can open a terminal-based project configuration menu with the `idf.py menuconfig` build target.

After being updated, this configuration is saved in the `sdkconfig` file under the project root directory. Based on `sdkconfig`, application build targets will generate the `sdkconfig.h` file under the build directory, and will make the `sdkconfig` options available to the project build system and source files.

2.7.3 Using `sdkconfig.defaults`

In some cases, for example, when the `sdkconfig` file is under revision control, it may be inconvenient for the build system to change the `sdkconfig` file. The build system offers a solution to prevent it from happening, which is to create the `sdkconfig.defaults` file. This file is never touched by the build system, and can be created manually or automatically. It contains all the options which matter to the given application and are different from the default ones. The format is the same as that of the `sdkconfig` file. `sdkconfig.defaults` can be created manually when one remembers all the changed configuration, or it can be generated automatically by running the `idf.py save-defconfig` command.

Once `sdkconfig.defaults` is created, `sdkconfig` can be deleted or added to the ignore list of the revision control system (e.g., the `.gitignore` file for `git`). Project build targets will automatically create the `sdkconfig` file, populate it with the settings from the `sdkconfig.defaults` file, and configure the rest of the settings to their default values. Note that during the build process, settings from `sdkconfig.defaults` will not override those already in `sdkconfig`. For more information, see *Custom Sdkconfig Defaults*.

2.7.4 Kconfig Format Rules

Format rules for Kconfig files are as follows:

- Option names in any menus should have consistent prefixes. The prefix currently should have at least 3 characters.
- The unit of indentation should be 4 spaces. All sub-items belonging to a parent item are indented by one level deeper. For example, `menu` is indented by 0 spaces, `config` `menu` by 4 spaces, `help` in `config` by 8 spaces, and the text under `help` by 12 spaces.
- No trailing spaces are allowed at the end of the lines.
- The maximum length of options is 50 characters.
- The maximum length of lines is 120 characters.

Note: The `help` section of each config in the menu is treated as reStructuredText to generate the reference documentation for each option.

Format Checker

`kconfcheck` tool in esp-idf-kconfig package is provided for checking Kconfig files against the above format rules. The checker checks all Kconfig and `Kconfig.projbuild` files given as arguments, and generates a new file with suffix `.new` with some suggestions about how to fix issues (if there are any). Please note that the checker cannot correct all format issues and the responsibility of the developer is to final check and make corrections in order to pass the tests. For example, indentations will be corrected if there is not any misleading formatting, but it cannot come up with a common prefix for options inside a menu.

The `esp-idf-kconfig` package is available in ESP-IDF environments, where the checker tool can be invoked by running command `python -m kconfcheck <path_to_kconfig_file>`.

For more information, please refer to esp-idf-kconfig package documentation.

2.7.5 Backward Compatibility of Kconfig Options

The standard Kconfig tools ignore unknown options in `sdkconfig`. So if a developer has custom settings for options which are renamed in newer ESP-IDF releases, then the given setting for the option would be silently ignored. Therefore, several features have been adopted to avoid this:

1. `kconfgen` is used by the tool chain to pre-process `sdkconfig` files before anything else. For example, `menuconfig` would read them, so the settings for old options is kept and not ignored.
2. `kconfgen` recursively finds all `sdkconfig.rename` files in ESP-IDF directory which contain old and new `Kconfig` option names. Old options are replaced by new ones in the `sdkconfig` file. Renames that should only appear for a single target can be placed in a target-specific rename file `sdkconfig.rename.TARGET`, where `TARGET` is the target name, e.g., `sdkconfig.rename.esp32s2`.
3. `kconfgen` post-processes `sdkconfig` files and generates all build outputs (`sdkconfig.h`, `sdkconfig.cmake`, and `auto.conf`) by adding a list of compatibility statements, i.e., the values of old options are set for new options after modification. If users still use old options in their code, this will prevent it from breaking.
4. *Deprecated options and their replacements* are automatically generated by `kconfgen`.

2.7.6 Configuration Options Reference

Subsequent sections contain the list of available ESP-IDF options automatically generated from Kconfig files. Note that due to dependencies between options, some options listed here may not be visible by default in `menuconfig`.

By convention, all option names are upper-case letters with underscores. When Kconfig generates `sdkconfig` and `sdkconfig.h` files, option names are prefixed with `CONFIG_`. So if an option `ENABLE_FOO` is defined in a Kconfig file and selected in `menuconfig`, then the `sdkconfig` and `sdkconfig.h` files will have `CONFIG_ENABLE_FOO` defined. In the following sections, option names are also prefixed with `CONFIG_`, same as in the source code.

Build type

Contains:

- *CONFIG_APP_BUILD_TYPE*
- *CONFIG_APP_BUILD_TYPE_PURE_RAM_APP*
- *CONFIG_APP_REPRODUCIBLE_BUILD*
- *CONFIG_APP_NO_BLOBS*

CONFIG_APP_BUILD_TYPE

Application build type

Found in: Build type

Select the way the application is built.

By default, the application is built as a binary file in a format compatible with the ESP-IDF bootloader. In addition to this application, 2nd stage bootloader is also built. Application and bootloader binaries can be written into flash and loaded/executed from there.

Another option, useful for only very small and limited applications, is to only link the .elf file of the application, such that it can be loaded directly into RAM over JTAG or UART. Note that since IRAM and DRAM sizes are very limited, it is not possible to build any complex application this way. However for some kinds of testing and debugging, this option may provide faster iterations, since the application does not need to be written into flash.

Note: when APP_BUILD_TYPE_RAM is selected and loaded with JTAG, ESP-IDF does not contain all the startup code required to initialize the CPUs and ROM memory (data/bss). Therefore it is necessary to execute a bit of ROM code prior to executing the application. A gdbinit file may look as follows (for ESP32):

```
# Connect to a running instance of OpenOCD target remote :3333 # Reset and halt the target
mon reset halt # Run to a specific point in ROM code, # where most of initialization is
complete. thb *0x40007d54 c # Load the application into RAM load # Run till app_main tb
app_main c
```

Execute this gdbinit file as follows:

```
xtensa-esp32-elf-gdb build/app-name.elf -x gdbinit
```

Example gdbinit files for other targets can be found in tools/test_apps/system/gdb_loadable_elf/

When loading the BIN with UART, the ROM will jump to ram and run the app after finishing the ROM startup code, so there's no additional startup initialization required. You can use the *load_ram* in esptool.py to load the generated .bin file into ram and execute.

Example: esptool.py --chip {chip} -p {port} -b {baud} --no-stub load_ram {app.bin}

Recommended sdkconfig.defaults for building loadable ELF files is as follows. CONFIG_APP_BUILD_TYPE_RAM is required, other options help reduce application memory footprint.

```
CONFIG_APP_BUILD_TYPE_RAM=y CONFIG_VFS_SUPPORT_TERMIOS= CON-
FIG_NEWLIB_NANO_FORMAT=y CONFIG_ESP_SYSTEM_PANIC_PRINT_HALT=y
CONFIG_ESP_DEBUG_STUBS_ENABLE= CONFIG_ESP_ERR_TO_NAME_LOOKUP=
```

Available options:

- Default (binary application + 2nd stage bootloader) (CONFIG_APP_BUILD_TYPE_APP_2NDBOOT)
- Build app runs entirely in RAM (EXPERIMENTAL) (CONFIG_APP_BUILD_TYPE_RAM)

CONFIG_APP_BUILD_TYPE_PURE_RAM_APP

Build app without SPI_FLASH/PSRAM support (saves ram)

Found in: Build type

If this option is enabled, external memory and related peripherals, such as Cache, MMU, Flash and PSRAM, won't be initialized. Corresponding drivers won't be introduced either. Components that depend on the spi_flash component will also be unavailable, such as app_update, etc. When this option is enabled, about 26KB of RAM space can be saved.

CONFIG_APP_REPRODUCIBLE_BUILD

Enable reproducible build

Found in: Build type

If enabled, all date, time, and path information would be eliminated. A .gdbinit file would be create automatically. (or will be append if you have one already)

Default value:

- No (disabled)

CONFIG_APP_NO_BLOBS

No Binary Blobs

Found in: Build type

If enabled, this disables the linking of binary libraries in the application build. Note that after enabling this Wi-Fi/Bluetooth will not work.

Default value:

- No (disabled)

Bootloader config

Contains:

- *CONFIG_BOOTLOADER_LOG_LEVEL*
- *Bootloader manager*
- *CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION*
- *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE*
- *CONFIG_BOOTLOADER_REGION_PROTECTION_ENABLE*
- *CONFIG_BOOTLOADER_APP_TEST*
- *CONFIG_BOOTLOADER_FACTORY_RESET*
- *CONFIG_BOOTLOADER_HOLD_TIME_GPIO*
- *CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC*
- *Serial Flash Configurations*
- *CONFIG_BOOTLOADER_SKIP_VALIDATE_ALWAYS*
- *CONFIG_BOOTLOADER_SKIP_VALIDATE_ON_POWER_ON*
- *CONFIG_BOOTLOADER_SKIP_VALIDATE_IN_DEEP_SLEEP*
- *CONFIG_BOOTLOADER_WDT_ENABLE*
- *CONFIG_BOOTLOADER_VDDSDIO_BOOST*

Bootloader manager

Contains:

- *CONFIG_BOOTLOADER_PROJECT_VER*
- *CONFIG_BOOTLOADER_COMPILE_TIME_DATE*

CONFIG_BOOTLOADER_COMPILE_TIME_DATE

Use time/date stamp for bootloader

Found in: Bootloader config > Bootloader manager

If set, then the bootloader will be built with the current time/date stamp. It is stored in the bootloader description structure. If not set, time/date stamp will be excluded from bootloader image. This can be useful for getting the same binary image files made from the same source, but at different times.

CONFIG_BOOTLOADER_PROJECT_VER

Project version

Found in: Bootloader config > Bootloader manager

Project version. It is placed in "version" field of the esp_bootloader_desc structure. The type of this field is "uint32_t".

Range:
- from 0 to 4294967295

Default value:
- 1

CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION

Bootloader optimization Level

Found in: Bootloader config

This option sets compiler optimization level (gcc -O argument) for the bootloader.

- The default "Size" setting will add the -0s flag to CFLAGS.
- The "Debug" setting will add the -Og flag to CFLAGS.

- The "Performance" setting will add the -O2 flag to CFLAGS.

Note that custom optimization levels may be unsupported.

Available options:

- Size (-Os) (CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION_SIZE)
- Debug (-Og) (CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION_DEBUG)
- Optimize for performance (-O2) (CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION_PERF)
- Debug without optimization (-O0) (Deprecated, will be removed in IDF v6.0) (CONFIG_BOOTLOADER_COMPILER_OPTIMIZATION_NONE)

CONFIG_BOOTLOADER_LOG_LEVEL

Bootloader log verbosity

Found in: Bootloader config

Specify how much output to see in bootloader logs.

Available options:

- No output (CONFIG_BOOTLOADER_LOG_LEVEL_NONE)
- Error (CONFIG_BOOTLOADER_LOG_LEVEL_ERROR)
- Warning (CONFIG_BOOTLOADER_LOG_LEVEL_WARN)
- Info (CONFIG_BOOTLOADER_LOG_LEVEL_INFO)
- Debug (CONFIG_BOOTLOADER_LOG_LEVEL_DEBUG)
- Verbose (CONFIG_BOOTLOADER_LOG_LEVEL_VERBOSE)

Serial Flash Configurations

Contains:

- *CONFIG_BOOTLOADER_FLASH_DC_AWARE*
- *CONFIG_BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH*
- *CONFIG_BOOTLOADER_FLASH_XMC_SUPPORT*

CONFIG_BOOTLOADER_FLASH_DC_AWARE

Allow app adjust Dummy Cycle bits in SPI Flash for higher frequency (READ HELP FIRST)

Found in: Bootloader config > Serial Flash Configurations

This will force 2nd bootloader to be loaded by DOUT mode, and will restore Dummy Cycle setting by resetting the Flash

CONFIG_BOOTLOADER_FLASH_XMC_SUPPORT

Enable the support for flash chips of XMC (READ DOCS FIRST)

Found in: Bootloader config > Serial Flash Configurations

Perform the startup flow recommended by XMC. Please consult XMC for the details of this flow. XMC chips will be forbidden to be used, when this option is disabled.

DON'T DISABLE THIS UNLESS YOU KNOW WHAT YOU ARE DOING.

comment "Features below require specific hardware (READ DOCS FIRST!)"

Default value:

- Yes (enabled)

CONFIG_BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH

Enable cache access to 32-bit-address (over 16MB) range of SPI Flash (READ DOCS FIRST)

Found in: Bootloader config > Serial Flash Configurations

Enabling this option allows the CPU to access 32-bit-address flash beyond 16M range. 1. This option only valid for 4-line flash. Octal flash doesn't need this. 2. This option is experimental, which means it can’ t use on all flash chips stable, for more information, please contact Espressif Business support.

CONFIG_BOOTLOADER_VDDSDIO_BOOST

VDDSDIO LDO voltage

Found in: Bootloader config

If this option is enabled, and VDDSDIO LDO is set to 1.8V (using eFuse or MTDI bootstrapping pin), bootloader will change LDO settings to output 1.9V instead. This helps prevent flash chip from browning out during flash programming operations.

This option has no effect if VDDSDIO is set to 3.3V, or if the internal VDDSDIO regulator is disabled via eFuse.

Available options:

- 1.8V (CONFIG_BOOTLOADER_VDDSDIO_BOOST_1_8V)
- 1.9V (CONFIG_BOOTLOADER_VDDSDIO_BOOST_1_9V)

CONFIG_BOOTLOADER_FACTORY_RESET

GPIO triggers factory reset

Found in: Bootloader config

Allows to reset the device to factory settings: - clear one or more data partitions; - boot from "factory" partition. The factory reset will occur if there is a GPIO input held at the configured level while device starts up. See settings below.

Default value:

- No (disabled)

CONFIG_BOOTLOADER_NUM_PIN_FACTORY_RESET

Number of the GPIO input for factory reset

Found in: Bootloader config > CONFIG_BOOTLOADER_FACTORY_RESET

The selected GPIO will be configured as an input with internal pull-up enabled (note that on some SoCs. not all pins have an internal pull-up, consult the hardware datasheet for details.) To trigger a factory reset, this GPIO must be held high or low (as configured) on startup.

Default value:

- 4 if *CONFIG_BOOTLOADER_FACTORY_RESET*

CONFIG_BOOTLOADER_FACTORY_RESET_PIN_LEVEL

Factory reset GPIO level

Found in: Bootloader config > CONFIG_BOOTLOADER_FACTORY_RESET

Pin level for factory reset, can be triggered on low or high.

Available options:

- Reset on GPIO low (CONFIG_BOOTLOADER_FACTORY_RESET_PIN_LOW)
- Reset on GPIO high (CONFIG_BOOTLOADER_FACTORY_RESET_PIN_HIGH)

CONFIG_BOOTLOADER_OTA_DATA_ERASE

Clear OTA data on factory reset (select factory partition)

Found in: Bootloader config > CONFIG_BOOTLOADER_FACTORY_RESET

The device will boot from "factory" partition (or OTA slot 0 if no factory partition is present) after a factory reset.

CONFIG_BOOTLOADER_DATA_FACTORY_RESET

Comma-separated names of partitions to clear on factory reset

Found in: Bootloader config > CONFIG_BOOTLOADER_FACTORY_RESET

Allows customers to select which data partitions will be erased while factory reset.

Specify the names of partitions as a comma-delimited with optional spaces for readability. (Like this: "nvs, phy_init, ...") Make sure that the name specified in the partition table and here are the same. Partitions of type "app" cannot be specified here.

Default value:

- "nvs" if *CONFIG_BOOTLOADER_FACTORY_RESET*

CONFIG_BOOTLOADER_APP_TEST

GPIO triggers boot from test app partition

Found in: Bootloader config

Allows to run the test app from "TEST" partition. A boot from "test" partition will occur if there is a GPIO input pulled low while device starts up. See settings below.

CONFIG_BOOTLOADER_NUM_PIN_APP_TEST

Number of the GPIO input to boot TEST partition

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_TEST

The selected GPIO will be configured as an input with internal pull-up enabled. To trigger a test app, this GPIO must be pulled low on reset. After the GPIO input is deactivated and the device reboots, the old application will boot. (factory or OTA[x]). Note that GPIO34-39 do not have an internal pullup and an external one must be provided.

Range:

- from 0 to 39 if *CONFIG_BOOTLOADER_APP_TEST*

Default value:

- 18 if *CONFIG_BOOTLOADER_APP_TEST*

CONFIG_BOOTLOADER_APP_TEST_PIN_LEVEL

App test GPIO level

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_TEST

Pin level for app test, can be triggered on low or high.

Available options:

- Enter test app on GPIO low (CONFIG_BOOTLOADER_APP_TEST_PIN_LOW)

- Enter test app on GPIO high (CONFIG_BOOTLOADER_APP_TEST_PIN_HIGH)

CONFIG_BOOTLOADER_HOLD_TIME_GPIO

Hold time of GPIO for reset/test mode (seconds)

Found in: Bootloader config

The GPIO must be held low continuously for this period of time after reset before a factory reset or test partition boot (as applicable) is performed.

Default value:

- 5 if *CONFIG_BOOTLOADER_FACTORY_RESET* || *CONFIG_BOOTLOADER_APP_TEST*

CONFIG_BOOTLOADER_REGION_PROTECTION_ENABLE

Enable protection for unmapped memory regions

Found in: Bootloader config

Protects the unmapped memory regions of the entire address space from unintended accesses. This will ensure that an exception will be triggered whenever the CPU performs a memory operation on unmapped regions of the address space.

Default value:

- Yes (enabled)

CONFIG_BOOTLOADER_WDT_ENABLE

Use RTC watchdog in start code

Found in: Bootloader config

Tracks the execution time of startup code. If the execution time is exceeded, the RTC_WDT will restart system. It is also useful to prevent a lock up in start code caused by an unstable power source. NOTE: Tracks the execution time starts from the bootloader code - re-set timeout, while selecting the source for slow_clk - and ends calling app_main. Re-set timeout is needed due to WDT uses a SLOW_CLK clock source. After changing a frequency slow_clk a time of WDT needs to re-set for new frequency. slow_clk depends on RTC_CLK_SRC (INTERNAL_RC or EXTERNAL_CRYSTAL).

Default value:

- Yes (enabled)

CONFIG_BOOTLOADER_WDT_DISABLE_IN_USER_CODE

Allows RTC watchdog disable in user code

Found in: Bootloader config > CONFIG_BOOTLOADER_WDT_ENABLE

If this option is set, the ESP-IDF app must explicitly reset, feed, or disable the rtc_wdt in the app's own code. If this option is not set (default), then rtc_wdt will be disabled by ESP-IDF before calling the app_main() function.

Use function wdt_hal_feed() for resetting counter of RTC_WDT. For esp32/s2 you can also use rtc_wdt_feed().

Use function wdt_hal_disable() for disabling RTC_WDT. For esp32/s2 you can also use rtc_wdt_disable().

Default value:

- No (disabled)

CONFIG_BOOTLOADER_WDT_TIME_MS

Timeout for RTC watchdog (ms)

Found in: Bootloader config > CONFIG_BOOTLOADER_WDT_ENABLE

Verify that this parameter is correct and more then the execution time. Pay attention to options such as reset to factory, trigger test partition and encryption on boot - these options can increase the execution time. Note: RTC_WDT will reset while encryption operations will be performed.

Range:
- from 0 to 120000

Default value:
- 9000

CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE

Enable app rollback support

Found in: Bootloader config

After updating the app, the bootloader runs a new app with the "ESP_OTA_IMG_PENDING_VERIFY" state set. This state prevents the re-run of this app. After the first boot of the new app in the user code, the function should be called to confirm the operability of the app or vice versa about its non-operability. If the app is working, then it is marked as valid. Otherwise, it is marked as not valid and rolls back to the previous working app. A reboot is performed, and the app is booted before the software update. Note: If during the first boot a new app the power goes out or the WDT works, then roll back will happen. Rollback is possible only between the apps with the same security versions.

Default value:
- No (disabled)

CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK

Enable app anti-rollback support

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE

This option prevents rollback to previous firmware/application image with lower security version.

Default value:
- No (disabled) if *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE*

CONFIG_BOOTLOADER_APP_SECURE_VERSION

eFuse secure version of app

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE > CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK

The secure version is the sequence number stored in the header of each firmware. The security version is set in the bootloader, version is recorded in the eFuse field as the number of set ones. The allocated number of bits in the efuse field for storing the security version is limited (see BOOTLOADER_APP_SEC_VER_SIZE_EFUSE_FIELD option).

Bootloader: When bootloader selects an app to boot, an app is selected that has a security version greater or equal that recorded in eFuse field. The app is booted with a higher (or equal) secure version.

The security version is worth increasing if in previous versions there is a significant vulnerability and their use is not acceptable.

Your partition table should has a scheme with ota_0 + ota_1 (without factory).

Default value:
- 0 if *CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK*

CONFIG_BOOTLOADER_APP_SEC_VER_SIZE_EFUSE_FIELD

Size of the efuse secure version field

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE > CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK

The size of the efuse secure version field. Its length is limited to 32 bits for ESP32 and 16 bits for ESP32-S2. This determines how many times the security version can be increased.

Range:
- from 1 to 16 if *CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK*

Default value:
- 16 if *CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK*

CONFIG_BOOTLOADER_EFUSE_SECURE_VERSION_EMULATE

Emulate operations with efuse secure version(only test)

Found in: Bootloader config > CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE > CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK

This option allows to emulate read/write operations with all eFuses and efuse secure version. It allows to test anti-rollback implementation without permanent write eFuse bits. There should be an entry in partition table with following details: *emul_efuse, data, efuse, , 0x2000*.

This option enables: EFUSE_VIRTUAL and EFUSE_VIRTUAL_KEEP_IN_FLASH.

Default value:
- No (disabled) if *CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK*

CONFIG_BOOTLOADER_SKIP_VALIDATE_IN_DEEP_SLEEP

Skip image validation when exiting deep sleep

Found in: Bootloader config

This option disables the normal validation of an image coming out of deep sleep (checksums, SHA256, and signature). This is a trade-off between wakeup performance from deep sleep, and image integrity checks.

Only enable this if you know what you are doing. It should not be used in conjunction with using deep_sleep() entry and changing the active OTA partition as this would skip the validation upon first load of the new OTA partition.

It is possible to enable this option with Secure Boot if "allow insecure options" is enabled, however it's strongly recommended to NOT enable it as it may allow a Secure Boot bypass.

Default value:
- No (disabled) if *CONFIG_SECURE_BOOT* && *CONFIG_SECURE_BOOT_INSECURE*

CONFIG_BOOTLOADER_SKIP_VALIDATE_ON_POWER_ON

Skip image validation from power on reset (READ HELP FIRST)

Found in: Bootloader config

Some applications need to boot very quickly from power on. By default, the entire app binary is read from flash and verified which takes up a significant portion of the boot time.

Enabling this option will skip validation of the app when the SoC boots from power on. Note that in this case it's not possible for the bootloader to detect if an app image is corrupted in the flash, therefore it's not possible to safely fall back to a different app partition. Flash corruption of this kind is unlikely but can happen if there is a serious firmware bug or physical damage.

Following other reset types, the bootloader will still validate the app image. This increases the chances that flash corruption resulting in a crash can be detected following soft reset, and the bootloader will fall back to a valid app image. To increase the chances of successfully recovering from a flash corruption event, keep the option BOOTLOADER_WDT_ENABLE enabled and consider also enabling BOOTLOADER_WDT_DISABLE_IN_USER_CODE - then manually disable the RTC Watchdog once the app is running. In addition, enable both the Task and Interrupt watchdog timers with reset options set.

Default value:

- No (disabled)

CONFIG_BOOTLOADER_SKIP_VALIDATE_ALWAYS

Skip image validation always (READ HELP FIRST)

Found in: Bootloader config

Selecting this option prevents the bootloader from ever validating the app image before booting it. Any flash corruption of the selected app partition will make the entire SoC unbootable.

Although flash corruption is a very rare case, it is not recommended to select this option. Consider selecting "Skip image validation from power on reset" instead. However, if boot time is the only important factor then it can be enabled.

Default value:

- No (disabled)

CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC

Reserve RTC FAST memory for custom purposes

Found in: Bootloader config

This option allows the customer to place data in the RTC FAST memory, this area remains valid when rebooted, except for power loss. This memory is located at a fixed address and is available for both the bootloader and the application. (The application and bootloader must be compiled with the same option). The RTC FAST memory has access only through PRO_CPU.

Default value:

- No (disabled)

CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC_IN_CRC

Include custom memory in the CRC calculation

Found in: Bootloader config > CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC

This option allows the customer to use the legacy bootloader behavior when the RTC FAST memory CRC calculation takes place. When this option is enabled, the allocated user custom data will be taken into account in the CRC calculation. This means that any change to the custom data would need a CRC update to prevent the bootloader from marking this data as corrupted. If this option is disabled, the custom data will not be taken into account when calculating the RTC FAST memory CRC. The user custom data can be changed freely, without the need to update the CRC. THIS OPTION MUST BE THE SAME FOR BOTH THE BOOTLOADER AND THE APPLICATION BUILDS.

Default value:

- No (disabled) if *CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC*

CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC_SIZE

Size in bytes for custom purposes

Found in: Bootloader config > CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC

This option reserves in RTC FAST memory the area for custom purposes. If you want to create your own bootloader and save more information in this area of memory, you can increase it. It must be a multiple of 4 bytes. This area (rtc_retain_mem_t) is reserved and has access from the bootloader and an application.

Default value:

- 0 if *CONFIG_BOOTLOADER_CUSTOM_RESERVE_RTC*

Security features

Contains:

- *CONFIG_SECURE_BOOT_INSECURE*
- *CONFIG_SECURE_SIGNED_APPS_SCHEME*
- *CONFIG_SECURE_SIGNED_ON_BOOT_NO_SECURE_BOOT*
- *CONFIG_SECURE_FLASH_CHECK_ENC_EN_IN_APP*
- *CONFIG_SECURE_BOOT_ECDSA_KEY_LEN_SIZE*
- *CONFIG_SECURE_BOOT_ENABLE_AGGRESSIVE_KEY_REVOKE*
- *CONFIG_SECURE_FLASH_ENC_ENABLED*
- *CONFIG_SECURE_BOOT*
- *CONFIG_SECURE_FLASH_ENCRYPT_ONLY_IMAGE_LEN_IN_APP_PART*
- *CONFIG_SECURE_BOOT_FLASH_BOOTLOADER_DEFAULT*
- *CONFIG_SECURE_BOOTLOADER_KEY_ENCODING*
- *Potentially insecure options*
- *CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT*
- *CONFIG_SECURE_BOOT_VERIFICATION_KEY*
- *CONFIG_SECURE_BOOTLOADER_MODE*
- *CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES*
- *CONFIG_SECURE_UART_ROM_DL_MODE*
- *CONFIG_SECURE_SIGNED_ON_UPDATE_NO_SECURE_BOOT*

CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT

Require signed app images

Found in: Security features

Require apps to be signed to verify their integrity.

This option uses the same app signature scheme as hardware secure boot, but unlike hardware secure boot it does not prevent the bootloader from being physically updated. This means that the device can be secured against remote network access, but not physical access. Compared to using hardware Secure Boot this option is much simpler to implement.

CONFIG_SECURE_SIGNED_APPS_SCHEME

App Signing Scheme

Found in: Security features

Select the Secure App signing scheme. Depends on the Chip Revision. There are two secure boot versions:

1. **Secure boot V1**
   - Legacy custom secure boot scheme. Supported in ESP32 SoC.
2. **Secure boot V2**
   - RSA based secure boot scheme. Supported in ESP32-ECO3 (ESP32 Chip Revision 3 onwards), ESP32-S2, ESP32-C3, ESP32-S3 SoCs.
   - ECDSA based secure boot scheme. Supported in ESP32-C2 SoC.

Available options:

- ECDSA (CONFIG_SECURE_SIGNED_APPS_ECDSA_SCHEME)
  Embeds the ECDSA public key in the bootloader and signs the application with an ECDSA key. Refer to the documentation before enabling.
- RSA (CONFIG_SECURE_SIGNED_APPS_RSA_SCHEME)
  Appends the RSA-3072 based Signature block to the application. Refer to <Secure Boot Version 2 documentation link> before enabling.
- ECDSA (V2) (CONFIG_SECURE_SIGNED_APPS_ECDSA_V2_SCHEME)
  For Secure boot V2 (e.g., ESP32-C2 SoC), appends ECDSA based signature block to the application. Refer to documentation before enabling.

CONFIG_SECURE_BOOT_ECDSA_KEY_LEN_SIZE

ECDSA key size

Found in: Security features

Select the ECDSA key size. Two key sizes are supported

- 192 bit key using NISTP192 curve
- 256 bit key using NISTP256 curve (Recommended)

The advantage of using 256 bit key is the extra randomness which makes it difficult to be bruteforced compared to 192 bit key. At present, both key sizes are practically implausible to bruteforce.

Available options:

- Using ECC curve NISTP192 (CONFIG_SECURE_BOOT_ECDSA_KEY_LEN_192_BITS)
- Using ECC curve NISTP256 (Recommended) (CONFIG_SECURE_BOOT_ECDSA_KEY_LEN_256_BITS)

CONFIG_SECURE_SIGNED_ON_BOOT_NO_SECURE_BOOT

Bootloader verifies app signatures

Found in: Security features

If this option is set, the bootloader will be compiled with code to verify that an app is signed before booting it.

If hardware secure boot is enabled, this option is always enabled and cannot be disabled. If hardware secure boot is not enabled, this option doesn't add significant security by itself so most users will want to leave it disabled.

Default value:

- No (disabled) if *CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT* && *CONFIG_SECURE_SIGNED_APPS_ECDSA_SCHEME*

CONFIG_SECURE_SIGNED_ON_UPDATE_NO_SECURE_BOOT

Verify app signature on update

Found in: Security features

If this option is set, any OTA updated apps will have the signature verified before being considered valid.

When enabled, the signature is automatically checked whenever the esp_ota_ops.h APIs are used for OTA updates, or esp_image_format.h APIs are used to verify apps.

If hardware secure boot is enabled, this option is always enabled and cannot be disabled. If hardware secure boot is not enabled, this option still adds significant security against network-based attackers by preventing spoofing of OTA updates.

Default value:

- Yes (enabled) if *CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT*

CONFIG_SECURE_BOOT

Enable hardware Secure Boot in bootloader (READ DOCS FIRST)

Found in: Security features

Build a bootloader which enables Secure Boot on first boot.

Once enabled, Secure Boot will not boot a modified bootloader. The bootloader will only load a partition table or boot an app if the data has a verified digital signature. There are implications for reflashing updated apps once secure boot is enabled.

When enabling secure boot, JTAG and ROM BASIC Interpreter are permanently disabled by default.

Default value:

- No (disabled)

CONFIG_SECURE_BOOT_VERSION

Select secure boot version

Found in: Security features > CONFIG_SECURE_BOOT

Select the Secure Boot Version. Depends on the Chip Revision. Secure Boot V2 is the new RSA / ECDSA based secure boot scheme.

- RSA based scheme is supported in ESP32 (Revision 3 onwards), ESP32-S2, ESP32-C3 (ECO3), ESP32-S3.
- ECDSA based scheme is supported in ESP32-C2 SoC.

Please note that, RSA or ECDSA secure boot is property of specific SoC based on its HW design, supported crypto accelerators, die-size, cost and similar parameters. Please note that RSA scheme has requirement for bigger key sizes but at the same time it is comparatively faster than ECDSA verification.

Secure Boot V1 is the AES based (custom) secure boot scheme supported in ESP32 SoC.

Available options:

- Enable Secure Boot version 1 (CONFIG_SECURE_BOOT_V1_ENABLED)
  Build a bootloader which enables secure boot version 1 on first boot. Refer to the Secure Boot section of the ESP-IDF Programmer's Guide for this version before enabling.
- Enable Secure Boot version 2 (CONFIG_SECURE_BOOT_V2_ENABLED)
  Build a bootloader which enables Secure Boot version 2 on first boot. Refer to Secure Boot V2 section of the ESP-IDF Programmer's Guide for this version before enabling.

CONFIG_SECURE_BOOTLOADER_MODE

Secure bootloader mode

Found in: Security features

Available options:

- One-time flash (CONFIG_SECURE_BOOTLOADER_ONE_TIME_FLASH)
  On first boot, the bootloader will generate a key which is not readable externally or by software. A digest is generated from the bootloader image itself. This digest will be verified on each subsequent boot.
  Enabling this option means that the bootloader cannot be changed after the first time it is booted.

- Reflashable (CONFIG_SECURE_BOOTLOADER_REFLASHABLE)
  Generate a reusable secure bootloader key, derived (via SHA-256) from the secure boot signing key.
  This allows the secure bootloader to be re-flashed by anyone with access to the secure boot signing key.
  This option is less secure than one-time flash, because a leak of the digest key from one device allows reflashing of any device that uses it.

CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES

Sign binaries during build

Found in: Security features

Once secure boot or signed app requirement is enabled, app images are required to be signed.

If enabled (default), these binary files are signed as part of the build process. The file named in "Secure boot private signing key" will be used to sign the image.

If disabled, unsigned app/partition data will be built. They must be signed manually using espsecure.py. Version 1 to enable ECDSA Based Secure Boot and Version 2 to enable RSA based Secure Boot. (for example, on a remote signing server.)

CONFIG_SECURE_BOOT_SIGNING_KEY

Secure boot private signing key

Found in: Security features > CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES

Path to the key file used to sign app images.

Key file is an ECDSA private key (NIST256p curve) in PEM format for Secure Boot V1. Key file is an RSA private key in PEM format for Secure Boot V2.

Path is evaluated relative to the project directory.

You can generate a new signing key by running the following command: espsecure.py generate_signing_key secure_boot_signing_key.pem

See the Secure Boot section of the ESP-IDF Programmer's Guide for this version for details.

Default value:

- "secure_boot_signing_key.pem" if *CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES*

CONFIG_SECURE_BOOT_VERIFICATION_KEY

Secure boot public signature verification key

Found in: Security features

Path to a public key file used to verify signed images. Secure Boot V1: This ECDSA public key is compiled into the bootloader and/or app, to verify app images.

Key file is in raw binary format, and can be extracted from a PEM formatted private key using the espsecure.py extract_public_key command.

Refer to the Secure Boot section of the ESP-IDF Programmer's Guide for this version before enabling.

CONFIG_SECURE_BOOT_ENABLE_AGGRESSIVE_KEY_REVOKE

Enable Aggressive key revoke strategy

Found in: Security features

If this option is set, ROM bootloader will revoke the public key digest burned in efuse block if it fails to verify the signature of software bootloader with it. Revocation of keys does not happen when enabling

secure boot. Once secure boot is enabled, key revocation checks will be done on subsequent boot-up, while verifying the software bootloader

This feature provides a strong resistance against physical attacks on the device.

NOTE: Once a digest slot is revoked, it can never be used again to verify an image This can lead to permanent bricking of the device, in case all keys are revoked because of signature verification failure.

Default value:

- No (disabled) if *CONFIG_SECURE_BOOT*

CONFIG_SECURE_BOOT_FLASH_BOOTLOADER_DEFAULT

Flash bootloader along with other artifacts when using the default flash command

Found in: Security features

When Secure Boot V2 is enabled, by default the bootloader is not flashed along with other artifacts like the application and the partition table images, i.e. bootloader has to be separately flashed using the command *idf.py bootloader flash*, whereas, the application and partition table can be flashed using the command *idf.py flash* itself. Enabling this option allows flashing the bootloader along with the other artifacts by invocation of the command *idf.py flash*.

If this option is enabled make sure that even the bootloader is signed using the correct secure boot key, otherwise the bootloader signature verification would fail, as hash of the public key which is present in the bootloader signature would not match with the digest stored into the efuses and thus the device will not be able to boot up.

Default value:

- No (disabled) if *CONFIG_SECURE_BOOT_V2_ENABLED* && *CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES*

CONFIG_SECURE_BOOTLOADER_KEY_ENCODING

Hardware Key Encoding

Found in: Security features

In reflashable secure bootloader mode, a hardware key is derived from the signing key (with SHA-256) and can be written to eFuse with espefuse.py.

Normally this is a 256-bit key, but if 3/4 Coding Scheme is used on the device then the eFuse key is truncated to 192 bits.

This configuration item doesn't change any firmware code, it only changes the size of key binary which is generated at build time.

Available options:

- No encoding (256 bit key) (CONFIG_SECURE_BOOTLOADER_KEY_ENCODING_256BIT)
- 3/4 encoding (192 bit key) (CONFIG_SECURE_BOOTLOADER_KEY_ENCODING_192BIT)

CONFIG_SECURE_BOOT_INSECURE

Allow potentially insecure options

Found in: Security features

You can disable some of the default protections offered by secure boot, in order to enable testing or a custom combination of security features.

Only enable these options if you are very sure.

Refer to the Secure Boot section of the ESP-IDF Programmer's Guide for this version before enabling.

Default value:

- No (disabled) if *CONFIG_SECURE_BOOT*

CONFIG_SECURE_FLASH_ENC_ENABLED

Enable flash encryption on boot (READ DOCS FIRST)

Found in: Security features

If this option is set, flash contents will be encrypted by the bootloader on first boot.

Note: After first boot, the system will be permanently encrypted. Re-flashing an encrypted system is complicated and not always possible.

Read *Flash Encryption* before enabling.

Default value:

- No (disabled)

CONFIG_SECURE_FLASH_ENCRYPTION_KEYSIZE

Size of generated XTS-AES key

Found in: Security features > CONFIG_SECURE_FLASH_ENC_ENABLED

Size of generated XTS-AES key.

- AES-128 uses a 256-bit key (32 bytes) derived from 128 bits (16 bytes) burned in half Efuse key block. Internally, it calculates SHA256(128 bits)
- AES-128 uses a 256-bit key (32 bytes) which occupies one Efuse key block.
- AES-256 uses a 512-bit key (64 bytes) which occupies two Efuse key blocks.

This setting is ignored if either type of key is already burned to Efuse before the first boot. In this case, the pre-burned key is used and no new key is generated.

Available options:

- AES-128 key derived from 128 bits (SHA256(128 bits)) (CONFIG_SECURE_FLASH_ENCRYPTION_AES128_DERIVED)
- AES-128 (256-bit key) (CONFIG_SECURE_FLASH_ENCRYPTION_AES128)
- AES-256 (512-bit key) (CONFIG_SECURE_FLASH_ENCRYPTION_AES256)

CONFIG_SECURE_FLASH_ENCRYPTION_MODE

Enable usage mode

Found in: Security features > CONFIG_SECURE_FLASH_ENC_ENABLED

By default Development mode is enabled which allows ROM download mode to perform flash encryption operations (plaintext is sent to the device, and it encrypts it internally and writes ciphertext to flash.) This mode is not secure, it's possible for an attacker to write their own chosen plaintext to flash.

Release mode should always be selected for production or manufacturing. Once enabled it's no longer possible for the device in ROM Download Mode to use the flash encryption hardware.

When EFUSE_VIRTUAL is enabled, SECURE_FLASH_ENCRYPTION_MODE_RELEASE is not available. For CI tests we use IDF_CI_BUILD to bypass it ("export IDF_CI_BUILD=1"). We do not recommend bypassing it for other purposes.

Refer to the Flash Encryption section of the ESP-IDF Programmer's Guide for details.

Available options:

- Development (NOT SECURE) (CONFIG_SECURE_FLASH_ENCRYPTION_MODE_DEVELOPMENT)
- Release (CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE)

Potentially insecure options Contains:

- *CONFIG_SECURE_BOOT_V2_ALLOW_EFUSE_RD_DIS*
- *CONFIG_SECURE_BOOT_ALLOW_SHORT_APP_PARTITION*
- *CONFIG_SECURE_BOOT_ALLOW_JTAG*
- *CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_ENC*
- *CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_CACHE*
- *CONFIG_SECURE_BOOT_ALLOW_UNUSED_DIGEST_SLOTS*
- *CONFIG_SECURE_FLASH_REQUIRE_ALREADY_ENABLED*
- *CONFIG_SECURE_FLASH_SKIP_WRITE_PROTECTION_CACHE*

CONFIG_SECURE_BOOT_ALLOW_JTAG

Allow JTAG Debugging

Found in: Security features > Potentially insecure options

If not set (default), the bootloader will permanently disable JTAG (across entire chip) on first boot when either secure boot or flash encryption is enabled.

Setting this option leaves JTAG on for debugging, which negates all protections of flash encryption and some of the protections of secure boot.

Only set this option in testing environments.

Default value:

- No (disabled) if *CONFIG_SECURE_BOOT_INSECURE* || *CONFIG_SECURE_FLASH_ENCRYPTION_MODE_DEVELOPMENT*

CONFIG_SECURE_BOOT_ALLOW_SHORT_APP_PARTITION

Allow app partition length not 64KB aligned

Found in: Security features > Potentially insecure options

If not set (default), app partition size must be a multiple of 64KB. App images are padded to 64KB length, and the bootloader checks any trailing bytes after the signature (before the next 64KB boundary) have not been written. This is because flash cache maps entire 64KB pages into the address space. This prevents an attacker from appending unverified data after the app image in the flash, causing it to be mapped into the address space.

Setting this option allows the app partition length to be unaligned, and disables padding of the app image to this length. It is generally not recommended to set this option, unless you have a legacy partitioning scheme which doesn't support 64KB aligned partition lengths.

CONFIG_SECURE_BOOT_V2_ALLOW_EFUSE_RD_DIS

Allow additional read protecting of efuses

Found in: Security features > Potentially insecure options

If not set (default, recommended), on first boot the bootloader will burn the WR_DIS_RD_DIS efuse when Secure Boot is enabled. This prevents any more efuses from being read protected.

If this option is set, it will remain possible to write the EFUSE_RD_DIS efuse field after Secure Boot is enabled. This may allow an attacker to read-protect the BLK2 efuse (for ESP32) and BLOCK4-BLOCK10 (i.e. BLOCK_KEY0-BLOCK_KEY5)(for other chips) holding the public key digest, causing an immediate denial of service and possibly allowing an additional fault injection attack to bypass the signature protection.

NOTE: Once a BLOCK is read-protected, the application will read all zeros from that block

NOTE: If "UART ROM download mode (Permanently disabled (recommended))" or "UART ROM download mode (Permanently switch to Secure mode (recommended))" is set, then it is __NOT__ possible to read/write efuses using espefuse.py utility. However, efuse can be read/written from the application

CONFIG_SECURE_BOOT_ALLOW_UNUSED_DIGEST_SLOTS

Leave unused digest slots available (not revoke)

Found in: Security features > Potentially insecure options

If not set (default), during startup in the app all unused digest slots will be revoked. To revoke unused slot will be called esp_efuse_set_digest_revoke(num_digest) for each digest. Revoking unused digest slots makes ensures that no trusted keys can be added later by an attacker. If set, it means that you have a plan to use unused digests slots later.

Note that if you plan to enable secure boot during the first boot up, the bootloader will intentionally revoke the unused digest slots while enabling secure boot, even if the above config is enabled because keeping the unused key slots un-revoked would a security hazard. In case for any development workflow if you need to avoid this revocation, you should enable secure boot externally (host based mechanism) rather than enabling it during the boot up, so that the bootloader would not need to enable secure boot and thus you could avoid its revocation strategy.

Default value:

- No (disabled) if *CONFIG_SECURE_BOOT_INSECURE*

CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_ENC

Leave UART bootloader encryption enabled

Found in: Security features > Potentially insecure options

If not set (default), the bootloader will permanently disable UART bootloader encryption access on first boot. If set, the UART bootloader will still be able to access hardware encryption.

It is recommended to only set this option in testing environments.

Default value:

- No (disabled) if *CONFIG_SECURE_FLASH_ENCRYPTION_MODE_DEVELOPMENT*

CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_CACHE

Leave UART bootloader flash cache enabled

Found in: Security features > Potentially insecure options

If not set (default), the bootloader will permanently disable UART bootloader flash cache access on first boot. If set, the UART bootloader will still be able to access the flash cache.

Only set this option in testing environments.

Default value:

- No (disabled) if *CONFIG_SECURE_FLASH_ENCRYPTION_MODE_DEVELOPMENT*

CONFIG_SECURE_FLASH_REQUIRE_ALREADY_ENABLED

Require flash encryption to be already enabled

Found in: Security features > Potentially insecure options

If not set (default), and flash encryption is not yet enabled in eFuses, the 2nd stage bootloader will enable flash encryption: generate the flash encryption key and program eFuses. If this option is set, and flash

encryption is not yet enabled, the bootloader will error out and reboot. If flash encryption is enabled in eFuses, this option does not change the bootloader behavior.

Only use this option in testing environments, to avoid accidentally enabling flash encryption on the wrong device. The device needs to have flash encryption already enabled using espefuse.py.

Default value:

- No (disabled) if *CONFIG_SECURE_FLASH_ENCRYPTION_MODE_DEVELOPMENT*

CONFIG_SECURE_FLASH_SKIP_WRITE_PROTECTION_CACHE

Skip write-protection of DIS_CACHE (DIS_ICACHE, DIS_DCACHE)

Found in: Security features > Potentially insecure options

If not set (default, recommended), on the first boot the bootloader will burn the write-protection of DIS_CACHE(for ESP32) or DIS_ICACHE/DIS_DCACHE(for other chips) eFuse when Flash Encryption is enabled. Write protection for cache disable efuse prevents the chip from being blocked if it is set by accident. App and bootloader use cache so disabling it makes the chip useless for IDF. Due to other eFuses are linked with the same write protection bit (see the list below) then write-protection will not be done if these SECURE_FLASH_UART_BOOTLOADER_ALLOW_ENC, SECURE_BOOT_ALLOW_JTAG or SECURE_FLASH_UART_BOOTLOADER_ALLOW_CACHE options are selected to give a chance to turn on the chip into the release mode later.

List of eFuses with the same write protection bit: ESP32: MAC, MAC_CRC, DISABLE_APP_CPU, DISABLE_BT, DIS_CACHE, VOL_LEVEL_HP_INV.

ESP32-C3: DIS_ICACHE, DIS_USB_JTAG, DIS_DOWNLOAD_ICACHE, DIS_USB_SERIAL_JTAG, DIS_FORCE_DOWNLOAD, DIS_TWAI, JTAG_SEL_ENABLE, DIS_PAD_JTAG, DIS_DOWNLOAD_MANUAL_ENCRYPT.

ESP32-C6: SWAP_UART_SDIO_EN, DIS_ICACHE, DIS_USB_JTAG, DIS_DOWNLOAD_ICACHE, DIS_USB_SERIAL_JTAG, DIS_FORCE_DOWNLOAD, DIS_TWAI, JTAG_SEL_ENABLE, DIS_PAD_JTAG, DIS_DOWNLOAD_MANUAL_ENCRYPT.

ESP32-H2: DIS_ICACHE, DIS_USB_JTAG, POWERGLITCH_EN, DIS_FORCE_DOWNLOAD, SPI_DOWNLOAD_MSPI_DIS, DIS_TWAI, JTAG_SEL_ENABLE, DIS_PAD_JTAG, DIS_DOWNLOAD_MANUAL_ENCRYPT.

ESP32-S2: DIS_ICACHE, DIS_DCACHE, DIS_DOWNLOAD_ICACHE, DIS_DOWNLOAD_DCACHE, DIS_FORCE_DOWNLOAD, DIS_USB, DIS_TWAI, DIS_BOOT_REMAP, SOFT_DIS_JTAG, HARD_DIS_JTAG, DIS_DOWNLOAD_MANUAL_ENCRYPT.

ESP32-S3: DIS_ICACHE, DIS_DCACHE, DIS_DOWNLOAD_ICACHE, DIS_DOWNLOAD_DCACHE, DIS_FORCE_DOWNLOAD, DIS_USB_OTG, DIS_TWAI, DIS_APP_CPU, DIS_PAD_JTAG, DIS_DOWNLOAD_MANUAL_ENCRYPT, DIS_USB_JTAG, DIS_USB_SERIAL_JTAG, STRAP_JTAG_SEL, USB_PHY_SEL.

CONFIG_SECURE_FLASH_ENCRYPT_ONLY_IMAGE_LEN_IN_APP_PART

Encrypt only the app image that is present in the partition of type app

Found in: Security features

If set (default), optimise encryption time for the partition of type APP, by only encrypting the app image that is present in the partition, instead of the whole partition. The image length used for encryption is derived from the image metadata, which includes the size of the app image, checksum, hash and also the signature sector when secure boot is enabled.

If not set, the whole partition of type APP would be encrypted, which increases the encryption time but might be useful if there is any custom data appended to the firmware image.

CONFIG_SECURE_FLASH_CHECK_ENC_EN_IN_APP

Check Flash Encryption enabled on app startup

Found in: Security features

If set (default), in an app during startup code, there is a check of the flash encryption eFuse bit is on (as the bootloader should already have set it). The app requires this bit is on to continue work otherwise abort.

If not set, the app does not care if the flash encryption eFuse bit is set or not.

Default value:

- Yes (enabled) if *CONFIG_SECURE_FLASH_ENC_ENABLED*

CONFIG_SECURE_UART_ROM_DL_MODE

UART ROM download mode

Found in: Security features

Available options:

- UART ROM download mode (Permanently disabled (recommended)) (CONFIG_SECURE_DISABLE_ROM_DL_MODE)
  If set, during startup the app will burn an eFuse bit to permanently disable the UART ROM Download Mode. This prevents any future use of esptool.py, espefuse.py and similar tools.
  Once disabled, if the SoC is booted with strapping pins set for ROM Download Mode then an error is printed instead.
  It is recommended to enable this option in any production application where Flash Encryption and/or Secure Boot is enabled and access to Download Mode is not required.
  It is also possible to permanently disable Download Mode by calling esp_efuse_disable_rom_download_mode() at runtime.
- UART ROM download mode (Permanently switch to Secure mode (recommended)) (CONFIG_SECURE_ENABLE_SECURE_ROM_DL_MODE)
  If set, during startup the app will burn an eFuse bit to permanently switch the UART ROM Download Mode into a separate Secure Download mode. This option can only work if Download Mode is not already disabled by eFuse.
  Secure Download mode limits the use of Download Mode functions to update SPI config, changing baud rate, basic flash write and a command to return a summary of currently enabled security features (*get_security_info*).
  Secure Download mode is not compatible with the esptool.py flasher stub feature, espefuse.py, read/writing memory or registers, encrypted download, or any other features that interact with unsupported Download Mode commands.
  Secure Download mode should be enabled in any application where Flash Encryption and/or Secure Boot is enabled. Disabling this option does not immediately cancel the benefits of the security features, but it increases the potential "attack surface" for an attacker to try and bypass them with a successful physical attack.
  It is also possible to enable secure download mode at runtime by calling esp_efuse_enable_rom_secure_download_mode()
  Note: Secure Download mode is not available for ESP32 (includes revisions till ECO3).
- UART ROM download mode (Enabled (not recommended)) (CONFIG_SECURE_INSECURE_ALLOW_DL_MODE)
  This is a potentially insecure option. Enabling this option will allow the full UART download mode to stay enabled. This option SHOULD NOT BE ENABLED for production use cases.

Application manager

Contains:

- *CONFIG_APP_EXCLUDE_PROJECT_NAME_VAR*
- *CONFIG_APP_EXCLUDE_PROJECT_VER_VAR*
- *CONFIG_APP_PROJECT_VER_FROM_CONFIG*
- *CONFIG_APP_RETRIEVE_LEN_ELF_SHA*
- *CONFIG_APP_COMPILE_TIME_DATE*

CONFIG_APP_COMPILE_TIME_DATE

Use time/date stamp for app

Found in: Application manager

If set, then the app will be built with the current time/date stamp. It is stored in the app description structure. If not set, time/date stamp will be excluded from app image. This can be useful for getting the same binary image files made from the same source, but at different times.

CONFIG_APP_EXCLUDE_PROJECT_VER_VAR

Exclude PROJECT_VER from firmware image

Found in: Application manager

The PROJECT_VER variable from the build system will not affect the firmware image. This value will not be contained in the esp_app_desc structure.

Default value:

- No (disabled)

CONFIG_APP_EXCLUDE_PROJECT_NAME_VAR

Exclude PROJECT_NAME from firmware image

Found in: Application manager

The PROJECT_NAME variable from the build system will not affect the firmware image. This value will not be contained in the esp_app_desc structure.

Default value:

- No (disabled)

CONFIG_APP_PROJECT_VER_FROM_CONFIG

Get the project version from Kconfig

Found in: Application manager

If this is enabled, then config item APP_PROJECT_VER will be used for the variable PROJECT_VER. Other ways to set PROJECT_VER will be ignored.

Default value:

- No (disabled)

CONFIG_APP_PROJECT_VER

Project version

Found in: Application manager > CONFIG_APP_PROJECT_VER_FROM_CONFIG

Project version

Default value:

- 1 if *CONFIG_APP_PROJECT_VER_FROM_CONFIG*

CONFIG_APP_RETRIEVE_LEN_ELF_SHA

The length of APP ELF SHA is stored in RAM(chars)

Found in: Application manager

At startup, the app will read the embedded APP ELF SHA-256 hash value from flash and convert it into a string and store it in a RAM buffer. This ensures the panic handler and core dump will be able to print this string even when cache is disabled. The size of the buffer is APP_RETRIEVE_LEN_ELF_SHA plus the null terminator. Changing this value will change the size of this buffer, in bytes.

Range:

- from 8 to 64

Default value:

- 9

Boot ROM Behavior

Contains:

- *CONFIG_BOOT_ROM_LOG_SCHEME*

CONFIG_BOOT_ROM_LOG_SCHEME

Permanently change Boot ROM output

Found in: Boot ROM Behavior

Controls the Boot ROM log behavior. The rom log behavior can only be changed for once, specific eFuse bit(s) will be burned at app boot stage.

Available options:

- Always Log (CONFIG_BOOT_ROM_LOG_ALWAYS_ON)
  Always print ROM logs, this is the default behavior.
- Permanently disable logging (CONFIG_BOOT_ROM_LOG_ALWAYS_OFF)
  Don't print ROM logs.
- Log on GPIO High (CONFIG_BOOT_ROM_LOG_ON_GPIO_HIGH)
  Print ROM logs when GPIO level is high during start up. The GPIO number is chip dependent, e.g. on ESP32-S2, the control GPIO is GPIO46.
- Log on GPIO Low (CONFIG_BOOT_ROM_LOG_ON_GPIO_LOW)
  Print ROM logs when GPIO level is low during start up. The GPIO number is chip dependent, e.g. on ESP32-S2, the control GPIO is GPIO46.

Serial flasher config

Contains:

- *CONFIG_ESPTOOLPY_AFTER*
- *CONFIG_ESPTOOLPY_BEFORE*
- *CONFIG_ESPTOOLPY_FLASH_MODE_AUTO_DETECT*
- *CONFIG_ESPTOOLPY_HEADER_FLASHSIZE_UPDATE*
- *CONFIG_ESPTOOLPY_NO_STUB*
- *CONFIG_ESPTOOLPY_OCT_FLASH*
- *CONFIG_ESPTOOLPY_FLASH_SAMPLE_MODE*
- *CONFIG_ESPTOOLPY_FLASHSIZE*

- *CONFIG_ESPTOOLPY_FLASHMODE*
- *CONFIG_ESPTOOLPY_FLASHFREQ*

CONFIG_ESPTOOLPY_NO_STUB

Disable download stub

Found in: Serial flasher config

The flasher tool sends a precompiled download stub first by default. That stub allows things like compressed downloads and more. Usually you should not need to disable that feature

CONFIG_ESPTOOLPY_OCT_FLASH

Enable Octal Flash

Found in: Serial flasher config

CONFIG_ESPTOOLPY_FLASH_MODE_AUTO_DETECT

Choose flash mode automatically (please read help)

Found in: Serial flasher config

This config option helps decide whether flash is Quad or Octal, but please note some limitations:

1. If the flash chip is an Octal one, even if one of "QIO", "QOUT", "DIO", "DOUT" options is selected in *ESPTOOLPY_FLASHMODE*, our code will automatically change the mode to "OPI" and the sample mode will be STR.
2. If the flash chip is a Quad one, even if "OPI" is selected in *ESPTOOLPY_FLASHMODE*, our code will automatically change the mode to "DIO".
3. This option is mainly to improve the out-of-box experience of developers. It doesn't guarantee the feature-complete. Some code still rely on *ESPTOOLPY_OCT_FLASH*. Please do not rely on this option when you are pretty sure that you are using Octal flash. In this case, please enable *ESPTOOLPY_OCT_FLASH* option, then you can choose *DTR* sample mode in *ESPTOOLPY_FLASH_SAMPLE_MODE*. Otherwise, only *STR* mode is available.
4. Enabling this feature reduces available internal RAM size (around 900 bytes). If your IRAM space is insufficient and you're aware of your flash type, disable this option and select corresponding flash type options.

CONFIG_ESPTOOLPY_FLASHMODE

Flash SPI mode

Found in: Serial flasher config

Mode the flash chip is flashed in, as well as the default mode for the binary to run in.

Available options:

- QIO (CONFIG_ESPTOOLPY_FLASHMODE_QIO)
- QOUT (CONFIG_ESPTOOLPY_FLASHMODE_QOUT)
- DIO (CONFIG_ESPTOOLPY_FLASHMODE_DIO)
- DOUT (CONFIG_ESPTOOLPY_FLASHMODE_DOUT)
- OPI (CONFIG_ESPTOOLPY_FLASHMODE_OPI)

CONFIG_ESPTOOLPY_FLASH_SAMPLE_MODE

Flash Sampling Mode

Found in: Serial flasher config

Available options:

- STR Mode (CONFIG_ESPTOOLPY_FLASH_SAMPLE_MODE_STR)
- DTR Mode (CONFIG_ESPTOOLPY_FLASH_SAMPLE_MODE_DTR)

CONFIG_ESPTOOLPY_FLASHFREQ

Flash SPI speed

Found in: Serial flasher config

Available options:

- 120 MHz (READ DOCS FIRST) (CONFIG_ESPTOOLPY_FLASHFREQ_120M)
  - Optional feature for QSPI Flash. Read docs and enable *CONFIG_SPI_FLASH_HPM_ENA* first!
  - Flash 120 MHz SDR mode is stable.
  - Flash 120 MHz DDR mode is an experimental feature, it works when the temperature is stable.

    Risks: If your chip powers on at a certain temperature, then after the temperature increases or decreases by approximately 20 Celsius degrees (depending on the chip), the program will crash randomly.
- 80 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_80M)
- 64 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_64M)
- 60 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_60M)
- 48 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_48M)
- 40 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_40M)
- 32 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_32M)
- 30 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_30M)
- 26 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_26M)
- 24 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_24M)
- 20 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_20M)
- 16 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_16M)
- 15 MHz (CONFIG_ESPTOOLPY_FLASHFREQ_15M)

CONFIG_ESPTOOLPY_FLASHSIZE

Flash size

Found in: Serial flasher config

SPI flash size, in megabytes

Available options:

- 1 MB (CONFIG_ESPTOOLPY_FLASHSIZE_1MB)
- 2 MB (CONFIG_ESPTOOLPY_FLASHSIZE_2MB)
- 4 MB (CONFIG_ESPTOOLPY_FLASHSIZE_4MB)
- 8 MB (CONFIG_ESPTOOLPY_FLASHSIZE_8MB)
- 16 MB (CONFIG_ESPTOOLPY_FLASHSIZE_16MB)
- 32 MB (CONFIG_ESPTOOLPY_FLASHSIZE_32MB)
- 64 MB (CONFIG_ESPTOOLPY_FLASHSIZE_64MB)
- 128 MB (CONFIG_ESPTOOLPY_FLASHSIZE_128MB)

CONFIG_ESPTOOLPY_HEADER_FLASHSIZE_UPDATE

Detect flash size when flashing bootloader

Found in: Serial flasher config

If this option is set, flashing the project will automatically detect the flash size of the target chip and update the bootloader image before it is flashed.

Enabling this option turns off the image protection against corruption by a SHA256 digest. Updating the bootloader image before flashing would invalidate the digest.

CONFIG_ESPTOOLPY_BEFORE

Before flashing

Found in: Serial flasher config

Configure whether esptool.py should reset the ESP32 before flashing.

Automatic resetting depends on the RTS & DTR signals being wired from the serial port to the ESP32. Most USB development boards do this internally.

Available options:

- Reset to bootloader (CONFIG_ESPTOOLPY_BEFORE_RESET)
- No reset (CONFIG_ESPTOOLPY_BEFORE_NORESET)

CONFIG_ESPTOOLPY_AFTER

After flashing

Found in: Serial flasher config

Configure whether esptool.py should reset the ESP32 after flashing.

Automatic resetting depends on the RTS & DTR signals being wired from the serial port to the ESP32. Most USB development boards do this internally.

Available options:

- Reset after flashing (CONFIG_ESPTOOLPY_AFTER_RESET)
- Stay in bootloader (CONFIG_ESPTOOLPY_AFTER_NORESET)

Partition Table

Contains:

- *CONFIG_PARTITION_TABLE_CUSTOM_FILENAME*
- *CONFIG_PARTITION_TABLE_MD5*
- *CONFIG_PARTITION_TABLE_OFFSET*
- *CONFIG_PARTITION_TABLE_TYPE*

CONFIG_PARTITION_TABLE_TYPE

Partition Table

Found in: Partition Table

The partition table to flash to the ESP32. The partition table determines where apps, data and other resources are expected to be found.

The predefined partition table CSV descriptions can be found in the components/partition_table directory. These are mostly intended for example and development use, it's expect that for production use you will copy one of these CSV files and create a custom partition CSV for your application.

Available options:

- Single factory app, no OTA (CONFIG_PARTITION_TABLE_SINGLE_APP)
  This is the default partition table, designed to fit into a 2MB or larger flash with a single 1MB app partition.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_singleapp.csv
  This partition table is not suitable for an app that needs OTA (over the air update) capability.
- Single factory app (large), no OTA (CONFIG_PARTITION_TABLE_SINGLE_APP_LARGE)
  This is a variation of the default partition table, that expands the 1MB app partition size to 1.5MB to fit more code.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_singleapp_large.csv
  This partition table is not suitable for an app that needs OTA (over the air update) capability.
- Factory app, two OTA definitions (CONFIG_PARTITION_TABLE_TWO_OTA)
  This is a basic OTA-enabled partition table with a factory app partition plus two OTA app partitions. All are 1MB, so this partition table requires 4MB or larger flash size.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_two_ota.csv
- Custom partition table CSV (CONFIG_PARTITION_TABLE_CUSTOM)
  Specify the path to the partition table CSV to use for your project.
  Consult the Partition Table section in the ESP-IDF Programmers Guide for more information.
- Single factory app, no OTA, encrypted NVS (CONFIG_PARTITION_TABLE_SINGLE_APP_ENCRYPTED_NVS)
  This is a variation of the default "Single factory app, no OTA" partition table that supports encrypted NVS when using flash encryption. See the Flash Encryption section in the ESP-IDF Programmers Guide for more information.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_singleapp_encr_nvs.csv
- Single factory app (large), no OTA, encrypted NVS (CONFIG_PARTITION_TABLE_SINGLE_APP_LARGE_ENC_NVS)
  This is a variation of the "Single factory app (large), no OTA" partition table that supports encrypted NVS when using flash encryption. See the Flash Encryption section in the ESP-IDF Programmers Guide for more information.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_singleapp_large_encr_nvs.csv
- Factory app, two OTA definitions, encrypted NVS (CONFIG_PARTITION_TABLE_TWO_OTA_ENCRYPTED_NVS)
  This is a variation of the "Factory app, two OTA definitions" partition table that supports encrypted NVS when using flash encryption. See the Flash Encryption section in the ESP-IDF Programmers Guide for more information.
  The corresponding CSV file in the IDF directory is components/partition_table/partitions_two_ota_encr_nvs.csv

CONFIG_PARTITION_TABLE_CUSTOM_FILENAME

Custom partition CSV file

Found in: Partition Table

Name of the custom partition CSV filename. This path is evaluated relative to the project root directory

by default. However, if the absolute path for the CSV file is provided, then the absolute path is configured.

Default value:

- "partitions.csv"

CONFIG_PARTITION_TABLE_OFFSET

Offset of partition table

Found in: Partition Table

The address of partition table (by default 0x8000). Allows you to move the partition table, it gives more space for the bootloader. Note that the bootloader and app will both need to be compiled with the same PARTITION_TABLE_OFFSET value.

This number should be a multiple of 0x1000.

Note that partition offsets in the partition table CSV file may need to be changed if this value is set to a higher value. To have each partition offset adapt to the configured partition table offset, leave all partition offsets blank in the CSV file.

Default value:

- "0x8000"

CONFIG_PARTITION_TABLE_MD5

Generate an MD5 checksum for the partition table

Found in: Partition Table

Generate an MD5 checksum for the partition table for protecting the integrity of the table. The generation should be turned off for legacy bootloaders which cannot recognize the MD5 checksum in the partition table.

Default value:

- Yes (enabled)

Compiler options

Contains:

- *CONFIG_COMPILER_OPTIMIZATION_ASSERTION_LEVEL*
- *CONFIG_COMPILER_FLOAT_LIB_FROM*
- *CONFIG_COMPILER_RT_LIB*
- *CONFIG_COMPILER_OPTIMIZATION_CHECKS_SILENT*
- *CONFIG_COMPILER_DISABLE_GCC12_WARNINGS*
- *CONFIG_COMPILER_DISABLE_GCC13_WARNINGS*
- *CONFIG_COMPILER_DUMP_RTL_FILES*
- *CONFIG_COMPILER_WARN_WRITE_STRINGS*
- *CONFIG_COMPILER_CXX_EXCEPTIONS*
- *CONFIG_COMPILER_CXX_RTTI*
- *CONFIG_COMPILER_OPTIMIZATION*
- *CONFIG_COMPILER_ORPHAN_SECTIONS*
- *CONFIG_COMPILER_HIDE_PATHS_MACROS*
- *CONFIG_COMPILER_STACK_CHECK_MODE*

CONFIG_COMPILER_OPTIMIZATION

Optimization Level

Found in: Compiler options

This option sets compiler optimization level (gcc -O argument) for the app.

- The "Debug" setting will add the -0g flag to CFLAGS.
- The "Size" setting will add the -0s flag to CFLAGS.
- The "Performance" setting will add the -O2 flag to CFLAGS.
- The "None" setting will add the -O0 flag to CFLAGS.

The "Size" setting cause the compiled code to be smaller and faster, but may lead to difficulties of correlating code addresses to source file lines when debugging.

The "Performance" setting causes the compiled code to be larger and faster, but will be easier to correlated code addresses to source file lines.

"None" with -O0 produces compiled code without optimization.

Note that custom optimization levels may be unsupported.

Compiler optimization for the IDF bootloader is set separately, see the BOOTLOADER_COMPILER_OPTIMIZATION setting.

Available options:

- Debug (-Og) (CONFIG_COMPILER_OPTIMIZATION_DEBUG)
- Optimize for size (-Os) (CONFIG_COMPILER_OPTIMIZATION_SIZE)
- Optimize for performance (-O2) (CONFIG_COMPILER_OPTIMIZATION_PERF)
- Debug without optimization (-O0) (CONFIG_COMPILER_OPTIMIZATION_NONE)

CONFIG_COMPILER_OPTIMIZATION_ASSERTION_LEVEL

Assertion level

Found in: Compiler options

Assertions can be:

- Enabled. Failure will print verbose assertion details. This is the default.
- Set to "silent" to save code size (failed assertions will abort() but user needs to use the aborting address to find the line number with the failed assertion.)
- Disabled entirely (not recommended for most configurations.) -DNDEBUG is added to CPPFLAGS in this case.

Available options:

- Enabled (CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_ENABLE)
  Enable assertions. Assertion content and line number will be printed on failure.
- Silent (saves code size) (CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_SILENT)
  Enable silent assertions. Failed assertions will abort(), user needs to use the aborting address to find the line number with the failed assertion.
- Disabled (sets -DNDEBUG) (CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_DISABLE)
  If assertions are disabled, -DNDEBUG is added to CPPFLAGS.

CONFIG_COMPILER_FLOAT_LIB_FROM

Compiler float lib source

Found in: Compiler options

In the soft-fp part of libgcc, riscv version is written in C, and handles all edge cases in IEEE754, which makes it larger and performance is slow.

RVfplib is an optimized RISC-V library for FP arithmetic on 32-bit integer processors, for single and double-precision FP. RVfplib is "fast", but it has a few exceptions from IEEE 754 compliance.

Available options:

- libgcc (CONFIG_COMPILER_FLOAT_LIB_FROM_GCCLIB)
- librvfp (CONFIG_COMPILER_FLOAT_LIB_FROM_RVFPLIB)

CONFIG_COMPILER_OPTIMIZATION_CHECKS_SILENT

Disable messages in ESP_RETURN_ON_* and ESP_EXIT_ON_* macros

Found in: Compiler options

If enabled, the error messages will be discarded in following check macros: - ESP_RETURN_ON_ERROR - ESP_EXIT_ON_ERROR - ESP_RETURN_ON_FALSE - ESP_EXIT_ON_FALSE

Default value:

- No (disabled)

CONFIG_COMPILER_HIDE_PATHS_MACROS

Replace ESP-IDF and project paths in binaries

Found in: Compiler options

When expanding the __FILE__ and __BASE_FILE__ macros, replace paths inside ESP-IDF with paths relative to the placeholder string "IDF", and convert paths inside the project directory to relative paths.

This allows building the project with assertions or other code that embeds file paths, without the binary containing the exact path to the IDF or project directories.

This option passes -fmacro-prefix-map options to the GCC command line. To replace additional paths in your binaries, modify the project CMakeLists.txt file to pass custom -fmacro-prefix-map or -ffile-prefix-map arguments.

Default value:

- Yes (enabled)

CONFIG_COMPILER_CXX_EXCEPTIONS

Enable C++ exceptions

Found in: Compiler options

Enabling this option compiles all IDF C++ files with exception support enabled.

Disabling this option disables C++ exception support in all compiled files, and any libstdc++ code which throws an exception will abort instead.

Enabling this option currently adds an additional ~500 bytes of heap overhead when an exception is thrown in user code for the first time.

Default value:

- No (disabled)

Contains:

- *CONFIG_COMPILER_CXX_EXCEPTIONS_EMG_POOL_SIZE*

CONFIG_COMPILER_CXX_EXCEPTIONS_EMG_POOL_SIZE

Emergency Pool Size

Found in: Compiler options > CONFIG_COMPILER_CXX_EXCEPTIONS

Size (in bytes) of the emergency memory pool for C++ exceptions. This pool will be used to allocate memory for thrown exceptions when there is not enough memory on the heap.

Default value:

- 0 if *CONFIG_COMPILER_CXX_EXCEPTIONS*

CONFIG_COMPILER_CXX_RTTI

Enable C++ run-time type info (RTTI)

Found in: Compiler options

Enabling this option compiles all C++ files with RTTI support enabled. This increases binary size (typically by tens of kB) but allows using dynamic_cast conversion and typeid operator.

Default value:

- No (disabled)

CONFIG_COMPILER_STACK_CHECK_MODE

Stack smashing protection mode

Found in: Compiler options

Stack smashing protection mode. Emit extra code to check for buffer overflows, such as stack smashing attacks. This is done by adding a guard variable to functions with vulnerable objects. The guards are initialized when a function is entered and then checked when the function exits. If a guard check fails, program is halted. Protection has the following modes:

- In NORMAL mode (GCC flag: -fstack-protector) only functions that call alloca, and functions with buffers larger than 8 bytes are protected.
- STRONG mode (GCC flag: -fstack-protector-strong) is like NORMAL, but includes additional functions to be protected -- those that have local array definitions, or have references to local frame addresses.
- In OVERALL mode (GCC flag: -fstack-protector-all) all functions are protected.

Modes have the following impact on code performance and coverage:

- performance: NORMAL > STRONG > OVERALL
- coverage: NORMAL < STRONG < OVERALL

The performance impact includes increasing the amount of stack memory required for each task.

Available options:

- None (CONFIG_COMPILER_STACK_CHECK_MODE_NONE)
- Normal (CONFIG_COMPILER_STACK_CHECK_MODE_NORM)
- Strong (CONFIG_COMPILER_STACK_CHECK_MODE_STRONG)
- Overall (CONFIG_COMPILER_STACK_CHECK_MODE_ALL)

CONFIG_COMPILER_WARN_WRITE_STRINGS

Enable -Wwrite-strings warning flag

Found in: Compiler options

Adds -Wwrite-strings flag for the C/C++ compilers.

For C, this gives string constants the type `const char[]` so that copying the address of one into a non-const `char *` pointer produces a warning. This warning helps to find at compile time code that tries to write into a string constant.

For C++, this warns about the deprecated conversion from string literals to `char *`.

Default value:

- No (disabled)

CONFIG_COMPILER_DISABLE_GCC12_WARNINGS

Disable new warnings introduced in GCC 12

Found in: Compiler options

Enable this option if use GCC 12 or newer, and want to disable warnings which don't appear with GCC 11.

Default value:

- No (disabled)

CONFIG_COMPILER_DISABLE_GCC13_WARNINGS

Disable new warnings introduced in GCC 13

Found in: Compiler options

Enable this option if use GCC 13 or newer, and want to disable warnings which don't appear with GCC 12.

Default value:

- No (disabled)

CONFIG_COMPILER_DUMP_RTL_FILES

Dump RTL files during compilation

Found in: Compiler options

If enabled, RTL files will be produced during compilation. These files can be used by other tools, for example to calculate call graphs.

CONFIG_COMPILER_RT_LIB

Compiler runtime library

Found in: Compiler options

Select runtime library to be used by compiler. - GCC toolchain supports libgcc only. - Clang allows to choose between libgcc or libclang_rt. - For host builds ("linux" target), uses the default library.

Available options:

- libgcc (CONFIG_COMPILER_RT_LIB_GCCLIB)
- libclang_rt (CONFIG_COMPILER_RT_LIB_CLANGRT)
- Host (CONFIG_COMPILER_RT_LIB_HOST)

CONFIG_COMPILER_ORPHAN_SECTIONS

Orphan sections handling

Found in: Compiler options

If the linker finds orphan sections, it attempts to place orphan sections after sections of the same attribute such as code vs data, loadable vs non-loadable, etc. That means that orphan sections could placed between sections defined in IDF linker scripts. This could lead to corruption of the binary image. Configure the linker action here.

Available options:

- Place with warning (CONFIG_COMPILER_ORPHAN_SECTIONS_WARNING)
  Places orphan sections without a warning message.

- Place silently (CONFIG_COMPILER_ORPHAN_SECTIONS_PLACE)
  Places orphan sections without a warning/error message.

Component config

Contains:

- *ADC and ADC Calibration*
- *Application Level Tracing*
- *Bluetooth*
- *Common ESP-related*
- *Console Library*
- *Core dump*
- *Driver Configurations*
- *eFuse Bit Manager*
- *CONFIG_BLE_MESH*
- *ESP HTTP client*
- *ESP HTTPS OTA*
- *ESP HTTPS server*
- *ESP NETIF Adapter*
- *ESP PSRAM*
- *ESP Ringbuf*
- *ESP System Settings*
- *ESP Timer (High Resolution Timer)*
- *ESP-Driver:Analog Comparator Configurations*
- *ESP-Driver:Camera Controller Configurations*
- *ESP-Driver:DAC Configurations*
- *ESP-Driver:GPIO Configurations*
- *ESP-Driver:GPTimer Configurations*
- *ESP-Driver:I2C Configurations*
- *ESP-Driver:I2S Configurations*
- *ESP-Driver:ISP Configurations*
- *ESP-Driver:JPEG-Codec Configurations*
- *ESP-Driver:LEDC Configurations*
- *ESP-Driver:MCPWM Configurations*
- *ESP-Driver:Parallel IO Configurations*
- *ESP-Driver:PCNT Configurations*
- *ESP-Driver:RMT Configurations*
- *ESP-Driver:Sigma Delta Modulator Configurations*
- *ESP-Driver:SPI Configurations*
- *ESP-Driver:Temperature Sensor Configurations*
- *ESP-Driver:Touch Sensor Configurations*
- *ESP-Driver:UART Configurations*
- *ESP-Driver:USB Serial/JTAG Configuration*
- *ESP-MQTT Configurations*
- *ESP-TLS*
- *Ethernet*
- *Event Loop Library*
- *FAT Filesystem support*
- *FreeRTOS*
- *GDB Stub*
- *Hardware Abstraction Layer (HAL) and Low Level (LL)*
- *Hardware Settings*
- *Heap memory debugging*
- *HTTP Server*
- *IEEE 802.15.4*
- *IPC (Inter-Processor Call)*
- *LCD and Touch Panel*

- *Log output*
- *LWIP*
- *Main Flash configuration*
- *mbedTLS*
- *Newlib*
- *NVS*
- *NVS Security Provider*
- *OpenThread*
- *Partition API Configuration*
- *PHY*
- *Power Management*
- *Protocomm*
- *PThreads*
- *SoC Settings*
- *SPI Flash driver*
- *SPIFFS Configuration*
- *TCP Transport*
- *Ultra Low Power (ULP) Co-processor*
- *Unity unit testing library*
- *USB-OTG*
- *Virtual file system*
- *Wear Levelling*
- *Wi-Fi*
- *Wi-Fi Provisioning Manager*
- *Wireless Coexistence*

Application Level Tracing Contains:

- *CONFIG_APPTRACE_DESTINATION1*
- *CONFIG_APPTRACE_DESTINATION2*
- *FreeRTOS SystemView Tracing*
- *CONFIG_APPTRACE_GCOV_ENABLE*
- *CONFIG_APPTRACE_BUF_SIZE*
- *CONFIG_APPTRACE_PENDING_DATA_SIZE_MAX*
- *CONFIG_APPTRACE_POSTMORTEM_FLUSH_THRESH*
- *CONFIG_APPTRACE_ONPANIC_HOST_FLUSH_TMO*
- *CONFIG_APPTRACE_UART_BAUDRATE*
- *CONFIG_APPTRACE_UART_RX_GPIO*
- *CONFIG_APPTRACE_UART_RX_BUFF_SIZE*
- *CONFIG_APPTRACE_UART_TASK_PRIO*
- *CONFIG_APPTRACE_UART_TX_MSG_SIZE*
- *CONFIG_APPTRACE_UART_TX_GPIO*
- *CONFIG_APPTRACE_UART_TX_BUFF_SIZE*

CONFIG_APPTRACE_DESTINATION1

Data Destination 1

Found in: Component config > Application Level Tracing

Select destination for application trace: JTAG or none (to disable).

Available options:

- JTAG (CONFIG_APPTRACE_DEST_JTAG)
- None (CONFIG_APPTRACE_DEST_NONE)

CONFIG_APPTRACE_DESTINATION2

Data Destination 2

Found in: Component config > Application Level Tracing

Select destination for application trace: UART(XX) or none (to disable).

Available options:

- UART0 (CONFIG_APPTRACE_DEST_UART0)
- UART1 (CONFIG_APPTRACE_DEST_UART1)
- UART2 (CONFIG_APPTRACE_DEST_UART2)
- USB_CDC (CONFIG_APPTRACE_DEST_USB_CDC)
- None (CONFIG_APPTRACE_DEST_UART_NONE)

CONFIG_APPTRACE_UART_TX_GPIO

UART TX on GPIO#

Found in: Component config > Application Level Tracing

This GPIO is used for UART TX pin.

CONFIG_APPTRACE_UART_RX_GPIO

UART RX on GPIO#

Found in: Component config > Application Level Tracing

This GPIO is used for UART RX pin.

CONFIG_APPTRACE_UART_BAUDRATE

UART baud rate

Found in: Component config > Application Level Tracing

This baud rate is used for UART.

The app's maximum baud rate depends on the UART clock source. If Power Management is disabled, the UART clock source is the APB clock and all baud rates in the available range will be sufficiently accurate. If Power Management is enabled, REF_TICK clock source is used so the baud rate is divided from 1MHz. Baud rates above 1Mbps are not possible and values between 500Kbps and 1Mbps may not be accurate.

CONFIG_APPTRACE_UART_RX_BUFF_SIZE

UART RX ring buffer size

Found in: Component config > Application Level Tracing

Size of the UART input ring buffer. This size related to the baudrate, system tick frequency and amount of data to transfer. The data placed to this buffer before sent out to the interface.

CONFIG_APPTRACE_UART_TX_BUFF_SIZE

UART TX ring buffer size

Found in: Component config > Application Level Tracing

Size of the UART output ring buffer. This size related to the baudrate, system tick frequency and amount of data to transfer.

CONFIG_APPTRACE_UART_TX_MSG_SIZE

UART TX message size

Found in: Component config > Application Level Tracing

Maximum size of the single message to transfer.

CONFIG_APPTRACE_UART_TASK_PRIO

UART Task Priority

Found in: Component config > Application Level Tracing

UART task priority. In case of high events rate, this parameter could be changed up to (config-MAX_PRIORITIES-1).

Range:
- from 1 to 32

Default value:
- 1

CONFIG_APPTRACE_ONPANIC_HOST_FLUSH_TMO

Timeout for flushing last trace data to host on panic

Found in: Component config > Application Level Tracing

Timeout for flushing last trace data to host in case of panic. In ms. Use -1 to disable timeout and wait forever.

CONFIG_APPTRACE_POSTMORTEM_FLUSH_THRESH

Threshold for flushing last trace data to host on panic

Found in: Component config > Application Level Tracing

Threshold for flushing last trace data to host on panic in post-mortem mode. This is minimal amount of data needed to perform flush. In bytes.

CONFIG_APPTRACE_BUF_SIZE

Size of the apptrace buffer

Found in: Component config > Application Level Tracing

Size of the memory buffer for trace data in bytes.

CONFIG_APPTRACE_PENDING_DATA_SIZE_MAX

Size of the pending data buffer

Found in: Component config > Application Level Tracing

Size of the buffer for events in bytes. It is useful for buffering events from the time critical code (scheduler, ISRs etc). If this parameter is 0 then events will be discarded when main HW buffer is full.

FreeRTOS SystemView Tracing Contains:

- *CONFIG_APPTRACE_SV_CPU*
- *CONFIG_APPTRACE_SV_EVT_ISR_ENTER_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_ISR_EXIT_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_ISR_TO_SCHED_ENABLE*
- *CONFIG_APPTRACE_SV_MAX_TASKS*

- *CONFIG_APPTRACE_SV_EVT_IDLE_ENABLE*
- *CONFIG_APPTRACE_SV_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_CREATE_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_START_EXEC_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_START_READY_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_STOP_EXEC_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_STOP_READY_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TASK_TERMINATE_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TIMER_ENTER_ENABLE*
- *CONFIG_APPTRACE_SV_EVT_TIMER_EXIT_ENABLE*
- *CONFIG_APPTRACE_SV_TS_SOURCE*
- *CONFIG_APPTRACE_SV_EVT_OVERFLOW_ENABLE*
- *CONFIG_APPTRACE_SV_BUF_WAIT_TMO*

CONFIG_APPTRACE_SV_ENABLE

SystemView Tracing Enable

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables supporrt for SEGGER SystemView tracing functionality.

CONFIG_APPTRACE_SV_DEST

SystemView destination

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing > CONFIG_APPTRACE_SV_ENABLE

SystemView witt transfer data trough defined interface.

Available options:

- Data destination JTAG (CONFIG_APPTRACE_SV_DEST_JTAG)
  Send SEGGER SystemView events through JTAG interface.
- Data destination UART (CONFIG_APPTRACE_SV_DEST_UART)
  Send SEGGER SystemView events through UART interface.

CONFIG_APPTRACE_SV_CPU

CPU to trace

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Define the CPU to trace by SystemView.

Available options:

- CPU0 (CONFIG_APPTRACE_SV_DEST_CPU_0)
  Send SEGGER SystemView events for Pro CPU.
- CPU1 (CONFIG_APPTRACE_SV_DEST_CPU_1)
  Send SEGGER SystemView events for App CPU.

CONFIG_APPTRACE_SV_TS_SOURCE

Timer to use as timestamp source

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

SystemView needs to use a hardware timer as the source of timestamps when tracing. This option selects the timer for it.

Available options:

- CPU cycle counter (CCOUNT) (CONFIG_APPTRACE_SV_TS_SOURCE_CCOUNT)
- General Purpose Timer (Timer Group) (CONFIG_APPTRACE_SV_TS_SOURCE_GPTIMER)
- esp_timer high resolution timer (CONFIG_APPTRACE_SV_TS_SOURCE_ESP_TIMER)

CONFIG_APPTRACE_SV_MAX_TASKS

Maximum supported tasks

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Configures maximum supported tasks in sysview debug

CONFIG_APPTRACE_SV_BUF_WAIT_TMO

Trace buffer wait timeout

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Configures timeout (in us) to wait for free space in trace buffer. Set to -1 to wait forever and avoid lost events.

CONFIG_APPTRACE_SV_EVT_OVERFLOW_ENABLE

Trace Buffer Overflow Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Trace Buffer Overflow" event.

CONFIG_APPTRACE_SV_EVT_ISR_ENTER_ENABLE

ISR Enter Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "ISR Enter" event.

CONFIG_APPTRACE_SV_EVT_ISR_EXIT_ENABLE

ISR Exit Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "ISR Exit" event.

CONFIG_APPTRACE_SV_EVT_ISR_TO_SCHED_ENABLE

ISR Exit to Scheduler Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "ISR to Scheduler" event.

CONFIG_APPTRACE_SV_EVT_TASK_START_EXEC_ENABLE

Task Start Execution Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Start Execution" event.

CONFIG_APPTRACE_SV_EVT_TASK_STOP_EXEC_ENABLE

Task Stop Execution Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Stop Execution" event.

CONFIG_APPTRACE_SV_EVT_TASK_START_READY_ENABLE

Task Start Ready State Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Start Ready State" event.

CONFIG_APPTRACE_SV_EVT_TASK_STOP_READY_ENABLE

Task Stop Ready State Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Stop Ready State" event.

CONFIG_APPTRACE_SV_EVT_TASK_CREATE_ENABLE

Task Create Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Create" event.

CONFIG_APPTRACE_SV_EVT_TASK_TERMINATE_ENABLE

Task Terminate Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Task Terminate" event.

CONFIG_APPTRACE_SV_EVT_IDLE_ENABLE

System Idle Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "System Idle" event.

CONFIG_APPTRACE_SV_EVT_TIMER_ENTER_ENABLE

Timer Enter Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Timer Enter" event.

CONFIG_APPTRACE_SV_EVT_TIMER_EXIT_ENABLE

Timer Exit Event

Found in: Component config > Application Level Tracing > FreeRTOS SystemView Tracing

Enables "Timer Exit" event.

CONFIG_APPTRACE_GCOV_ENABLE

GCOV to Host Enable

Found in: Component config > Application Level Tracing

Enables support for GCOV data transfer to host.

CONFIG_APPTRACE_GCOV_DUMP_TASK_STACK_SIZE

Gcov dump task stack size

Found in: Component config > Application Level Tracing > CONFIG_APPTRACE_GCOV_ENABLE

Configures stack size of Gcov dump task

Default value:

- 2048 if *CONFIG_APPTRACE_GCOV_ENABLE*

Bluetooth Contains:

- *Bluedroid Options*
- *CONFIG_BT_ENABLED*
- *Common Options*
- *Controller Options*
- *CONFIG_BT_HCI_LOG_DEBUG_EN*
- *NimBLE Options*
- *CONFIG_BT_RELEASE_IRAM*

CONFIG_BT_ENABLED

Bluetooth

Found in: Component config > Bluetooth

Select this option to enable Bluetooth and show the submenu with Bluetooth configuration choices.

CONFIG_BT_HOST

Host

Found in: Component config > Bluetooth > CONFIG_BT_ENABLED

This helps to choose Bluetooth host stack

Available options:

- Bluedroid - Dual-mode (CONFIG_BT_BLUEDROID_ENABLED)
  This option is recommended for classic Bluetooth or for dual-mode usecases
- NimBLE - BLE only (CONFIG_BT_NIMBLE_ENABLED)
  This option is recommended for BLE only usecases to save on memory
- Disabled (CONFIG_BT_CONTROLLER_ONLY)
  This option is recommended when you want to communicate directly with the controller (without any host) or when you are using any other host stack not supported by Espressif (not mentioned here).

CONFIG_BT_CONTROLLER

Controller

Found in: Component config > Bluetooth > CONFIG_BT_ENABLED

This helps to choose Bluetooth controller stack

Available options:

- Enabled (CONFIG_BT_CONTROLLER_ENABLED)
  This option is recommended for Bluetooth controller usecases
- Disabled (CONFIG_BT_CONTROLLER_DISABLED)
  This option is recommended for Bluetooth Host only usecases

Bluedroid Options Contains:

- *CONFIG_BT_BLE_HOST_QUEUE_CONG_CHECK*
- *CONFIG_BT_BLUEDROID_MEM_DEBUG*
- *CONFIG_BT_BTU_TASK_STACK_SIZE*
- *CONFIG_BT_BTC_TASK_STACK_SIZE*
- *CONFIG_BT_BLE_ENABLED*
- *BT DEBUG LOG LEVEL*
- *CONFIG_BT_ACL_CONNECTIONS*
- *CONFIG_BT_SMP_MAX_BONDS*
- *CONFIG_BT_ALLOCATION_FROM_SPIRAM_FIRST*
- *CONFIG_BT_CLASSIC_ENABLED*
- *CONFIG_BT_HID_ENABLED*
- *CONFIG_BT_STACK_NO_LOG*
- *CONFIG_BT_BLE_42_FEATURES_SUPPORTED*
- *CONFIG_BT_BLE_50_FEATURES_SUPPORTED*
- *CONFIG_BT_BLE_HIGH_DUTY_ADV_INTERVAL*
- *CONFIG_BT_MULTI_CONNECTION_ENBALE*
- *CONFIG_BT_BLE_FEAT_PERIODIC_ADV_SYNC_TRANSFER*
- *CONFIG_BT_BLE_FEAT_CREATE_SYNC_ENH*
- *CONFIG_BT_BLUEDROID_ESP_COEX_VSC*
- *CONFIG_BT_BLE_FEAT_PERIODIC_ADV_ENH*
- *CONFIG_BT_MAX_DEVICE_NAME_LEN*
- *CONFIG_BT_BLE_ACT_SCAN_REP_ADV_SCAN*
- *CONFIG_BT_BLUEDROID_PINNED_TO_CORE_CHOICE*
- *CONFIG_BT_BLE_ESTAB_LINK_CONN_TOUT*
- *CONFIG_BT_BLE_RPA_TIMEOUT*
- *CONFIG_BT_BLE_RPA_SUPPORTED*
- *CONFIG_BT_BLE_DYNAMIC_ENV_MEMORY*
- *CONFIG_BT_HFP_WBS_ENABLE*

CONFIG_BT_BTC_TASK_STACK_SIZE

Bluetooth event (callback to application) task stack size

Found in: Component config > Bluetooth > Bluedroid Options

This select btc task stack size

Default value:

- 3072 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLUEDROID_PINNED_TO_CORE_CHOICE

The cpu core which Bluedroid run

Found in: Component config > Bluetooth > Bluedroid Options

Which the cpu core to run Bluedroid. Can choose core0 and core1. Can not specify no-affinity.

Available options:

- Core 0 (PRO CPU) (CONFIG_BT_BLUEDROID_PINNED_TO_CORE_0)
- Core 1 (APP CPU) (CONFIG_BT_BLUEDROID_PINNED_TO_CORE_1)

CONFIG_BT_BTU_TASK_STACK_SIZE

Bluetooth Bluedroid Host Stack task stack size

Found in: Component config > Bluetooth > Bluedroid Options

This select btu task stack size

Default value:

- 4352 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLUEDROID_MEM_DEBUG

Bluedroid memory debug

Found in: Component config > Bluetooth > Bluedroid Options

Bluedroid memory debug

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLUEDROID_ESP_COEX_VSC

Enable Espressif Vendor-specific HCI commands for coexist status configuration

Found in: Component config > Bluetooth > Bluedroid Options

Enable Espressif Vendor-specific HCI commands for coexist status configuration

Default value:

- Yes (enabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_CLASSIC_ENABLED

Classic Bluetooth

Found in: Component config > Bluetooth > Bluedroid Options

For now this option needs "SMP_ENABLE" to be set to yes

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && ((*CONFIG_BT_CONTROLLER_ENABLED* && SOC_BT_CLASSIC_SUPPORTED) || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_ENC_KEY_SIZE_CTRL_ENABLED

configure encryption key size

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

This chooses the support status of configuring encryption key size

Available options:

- Supported by standard HCI command (CONFIG_BT_ENC_KEY_SIZE_CTRL_STD)
- Supported by Vendor-specific HCI command (CONFIG_BT_ENC_KEY_SIZE_CTRL_VSC)
- Not supported (CONFIG_BT_ENC_KEY_SIZE_CTRL_NONE)

CONFIG_BT_CLASSIC_BQB_ENABLED

Host Qualitifcation support for Classic Bluetooth

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

This enables functionalities of Host qualification for Classic Bluetooth.

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_A2DP_ENABLE

A2DP

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

Advanced Audio Distribution Profile

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_SPP_ENABLED

SPP

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

This enables the Serial Port Profile

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_L2CAP_ENABLED

BT L2CAP

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

This enables the Logical Link Control and Adaptation Layer Protocol. Only supported classic bluetooth.

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_HFP_ENABLE

Hands Free/Handset Profile

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED

Hands Free Unit and Audio Gateway can be included simultaneously but they cannot run simultaneously due to internal limitations.

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Contains:

- *CONFIG_BT_HFP_AG_ENABLE*
- *CONFIG_BT_HFP_AUDIO_DATA_PATH*
- *CONFIG_BT_HFP_CLIENT_ENABLE*

CONFIG_BT_HFP_CLIENT_ENABLE

Hands Free Unit

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED > CONFIG_BT_HFP_ENABLE

Default value:

- Yes (enabled) if *CONFIG_BT_HFP_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_HFP_AG_ENABLE

Audio Gateway

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED > CONFIG_BT_HFP_ENABLE

Default value:

- Yes (enabled) if *CONFIG_BT_HFP_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_HFP_AUDIO_DATA_PATH

audio(SCO) data path

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_CLASSIC_ENABLED > CONFIG_BT_HFP_ENABLE

SCO data path, i.e. HCI or PCM. This option is set using API "esp_bredr_sco_datapath_set" in Bluetooth host. Default SCO data path can also be set in Bluetooth Controller.

Available options:

- PCM (CONFIG_BT_HFP_AUDIO_DATA_PATH_PCM)
- HCI (CONFIG_BT_HFP_AUDIO_DATA_PATH_HCI)

CONFIG_BT_HFP_WBS_ENABLE

Wide Band Speech

Found in: Component config > Bluetooth > Bluedroid Options

This enables Wide Band Speech. Should disable it when SCO data path is PCM. Otherwise there will be no data transmitted via GPIOs.

Default value:

- Yes (enabled) if *CONFIG_BT_HFP_AUDIO_DATA_PATH_HCI* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_HID_ENABLED

Classic BT HID

Found in: Component config > Bluetooth > Bluedroid Options

This enables the BT HID Host

Default value:

- No (disabled) if *CONFIG_BT_CLASSIC_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Contains:

- *CONFIG_BT_HID_DEVICE_ENABLED*
- *CONFIG_BT_HID_HOST_ENABLED*

CONFIG_BT_HID_HOST_ENABLED

Classic BT HID Host

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_HID_ENABLED

This enables the BT HID Host

Default value:

- No (disabled) if *CONFIG_BT_HID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_HID_DEVICE_ENABLED

Classic BT HID Device

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_HID_ENABLED

This enables the BT HID Device

CONFIG_BT_BLE_ENABLED

Bluetooth Low Energy

Found in: Component config > Bluetooth > Bluedroid Options

This enables Bluetooth Low Energy

Default value:

- Yes (enabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTS_ENABLE

Include GATT server module(GATTS)

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED

This option can be disabled when the app work only on gatt client mode

Default value:

- Yes (enabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTS_PPCP_CHAR_GAP

Enable Peripheral Preferred Connection Parameters characteristic in GAP service

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

This enables "Peripheral Preferred Connection Parameters" characteristic (UUID: 0x2A04) in GAP service that has connection parameters like min/max connection interval, slave latency and supervision timeout multiplier

Default value:

- No (disabled) if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_BLUFI_ENABLE

Include blufi function

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

This option can be close when the app does not require blufi function.

Default value:

- No (disabled) if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATT_MAX_SR_PROFILES

Max GATT Server Profiles

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

Maximum GATT Server Profiles Count

Range:

- from 1 to 32 if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 8 if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATT_MAX_SR_ATTRIBUTES

Max GATT Service Attributes

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

Maximum GATT Service Attributes Count

Range:

- from 1 to 500 if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 100 if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTS_SEND_SERVICE_CHANGE_MODE

GATTS Service Change Mode

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

Service change indication mode for GATT Server.

Available options:

- GATTS manually send service change indication (CONFIG_BT_GATTS_SEND_SERVICE_CHANGE_MANUAL)
  Manually send service change indication through API esp_ble_gatts_send_service_change_indication()
- GATTS automatically send service change indication (CONFIG_BT_GATTS_SEND_SERVICE_CHANGE_AUTO)
  Let Bluedroid handle the service change indication internally

CONFIG_BT_GATTS_ROBUST_CACHING_ENABLED

Enable Robust Caching on Server Side

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

This option enables the GATT robust caching feature on the server. if turned on, the Client Supported Features characteristic, Database Hash characteristic, and Server Supported Features characteristic will be included in the GAP SERVICE.

Default value:

- No (disabled) if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTS_DEVICE_NAME_WRITABLE

Allow to write device name by GATT clients

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

Enabling this option allows remote GATT clients to write device name

Default value:

- No (disabled) if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTS_APPEARANCE_WRITABLE

Allow to write appearance by GATT clients

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTS_ENABLE

Enabling this option allows remote GATT clients to write appearance

Default value:

- No (disabled) if *CONFIG_BT_GATTS_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTC_ENABLE

Include GATT client module(GATTC)

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED

This option can be close when the app work only on gatt server mode

Default value:

- Yes (enabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTC_MAX_CACHE_CHAR

Max gattc cache characteristic for discover

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTC_ENABLE

Maximum GATTC cache characteristic count

Range:
- from 1 to 500 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:
- 40 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTC_NOTIF_REG_MAX

Max gattc notify(indication) register number

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTC_ENABLE

Maximum GATTC notify(indication) register number

Range:
- from 1 to 64 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:
- 5 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTC_CACHE_NVS_FLASH

Save gattc cache data to nvs flash

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTC_ENABLE

This select can save gattc cache data to nvs flash

Default value:
- No (disabled) if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_GATTC_CONNECT_RETRY_COUNT

The number of attempts to reconnect if the connection establishment failed

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_GATTC_ENABLE

The number of attempts to reconnect if the connection establishment failed

Range:
- from 0 to 255 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:
- 3 if *CONFIG_BT_GATTC_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_SMP_ENABLE

Include BLE security module(SMP)

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED

This option can be close when the app not used the ble security connect.

Default value:
- Yes (enabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_SMP_SLAVE_CON_PARAMS_UPD_ENABLE

Slave enable connection parameters update during pairing

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_BLE_SMP_ENABLE

In order to reduce the pairing time, slave actively initiates connection parameters update during pairing.

Default value:

- No (disabled) if *CONFIG_BT_BLE_SMP_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_SMP_ID_RESET_ENABLE

Reset device identity when all bonding records are deleted

Found in: Component config > Bluetooth > Bluedroid Options > CONFIG_BT_BLE_ENABLED > CONFIG_BT_BLE_SMP_ENABLE

There are tracking risks associated with using a fixed or static IRK. If enabled this option, Bluedroid will assign a new randomly-generated IRK when all pairing and bonding records are deleted. This would decrease the ability of a previously paired peer to be used to determine whether a device with which it previously shared an IRK is within range.

Default value:

- No (disabled) if *CONFIG_BT_BLE_SMP_ENABLE* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_STACK_NO_LOG

Disable BT debug logs (minimize bin size)

Found in: Component config > Bluetooth > Bluedroid Options

This select can save the rodata code size

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

BT DEBUG LOG LEVEL

Contains:

- *CONFIG_BT_LOG_A2D_TRACE_LEVEL*
- *CONFIG_BT_LOG_APPL_TRACE_LEVEL*
- *CONFIG_BT_LOG_AVCT_TRACE_LEVEL*
- *CONFIG_BT_LOG_AVDT_TRACE_LEVEL*
- *CONFIG_BT_LOG_AVRC_TRACE_LEVEL*
- *CONFIG_BT_LOG_BLUFI_TRACE_LEVEL*
- *CONFIG_BT_LOG_BNEP_TRACE_LEVEL*
- *CONFIG_BT_LOG_BTC_TRACE_LEVEL*
- *CONFIG_BT_LOG_BTIF_TRACE_LEVEL*
- *CONFIG_BT_LOG_BTM_TRACE_LEVEL*
- *CONFIG_BT_LOG_GAP_TRACE_LEVEL*
- *CONFIG_BT_LOG_GATT_TRACE_LEVEL*
- *CONFIG_BT_LOG_HCI_TRACE_LEVEL*
- *CONFIG_BT_LOG_HID_TRACE_LEVEL*
- *CONFIG_BT_LOG_L2CAP_TRACE_LEVEL*
- *CONFIG_BT_LOG_MCA_TRACE_LEVEL*
- *CONFIG_BT_LOG_OSI_TRACE_LEVEL*
- *CONFIG_BT_LOG_PAN_TRACE_LEVEL*
- *CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL*

- *CONFIG_BT_LOG_SDP_TRACE_LEVEL*
- *CONFIG_BT_LOG_SMP_TRACE_LEVEL*

CONFIG_BT_LOG_HCI_TRACE_LEVEL

HCI layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for HCI layer

Available options:

- NONE (CONFIG_BT_LOG_HCI_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_HCI_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_HCI_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_HCI_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_HCI_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_HCI_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_HCI_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_BTM_TRACE_LEVEL

BTM layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for BTM layer

Available options:

- NONE (CONFIG_BT_LOG_BTM_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_BTM_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_BTM_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_BTM_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_BTM_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_BTM_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_BTM_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_L2CAP_TRACE_LEVEL

L2CAP layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for L2CAP layer

Available options:

- NONE (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_L2CAP_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL

RFCOMM layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for RFCOMM layer

Available options:

- NONE (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_SDP_TRACE_LEVEL

SDP layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for SDP layer

Available options:

- NONE (CONFIG_BT_LOG_SDP_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_SDP_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_SDP_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_SDP_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_SDP_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_SDP_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_SDP_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_GAP_TRACE_LEVEL

GAP layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for GAP layer

Available options:

- NONE (CONFIG_BT_LOG_GAP_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_GAP_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_GAP_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_GAP_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_GAP_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_GAP_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_GAP_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_BNEP_TRACE_LEVEL

BNEP layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for BNEP layer

Available options:

- NONE (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_BNEP_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_PAN_TRACE_LEVEL

PAN layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for PAN layer

Available options:

- NONE (CONFIG_BT_LOG_PAN_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_PAN_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_PAN_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_PAN_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_PAN_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_PAN_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_PAN_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_A2D_TRACE_LEVEL

A2D layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for A2D layer

Available options:

- NONE (CONFIG_BT_LOG_A2D_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_A2D_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_A2D_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_A2D_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_A2D_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_A2D_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_A2D_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_AVDT_TRACE_LEVEL

AVDT layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for AVDT layer

Available options:

- NONE (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_AVDT_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_AVCT_TRACE_LEVEL

AVCT layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for AVCT layer

Available options:

- NONE (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_AVCT_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_AVRC_TRACE_LEVEL

AVRC layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for AVRC layer

Available options:

- NONE (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_AVRC_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_MCA_TRACE_LEVEL

MCA layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for MCA layer

Available options:

- NONE (CONFIG_BT_LOG_MCA_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_MCA_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_MCA_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_MCA_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_MCA_TRACE_LEVEL_EVENT)

- DEBUG (CONFIG_BT_LOG_MCA_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_MCA_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_HID_TRACE_LEVEL

HID layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for HID layer

Available options:

- NONE (CONFIG_BT_LOG_HID_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_HID_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_HID_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_HID_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_HID_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_HID_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_HID_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_APPL_TRACE_LEVEL

APPL layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for APPL layer

Available options:

- NONE (CONFIG_BT_LOG_APPL_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_APPL_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_APPL_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_APPL_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_APPL_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_APPL_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_APPL_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_GATT_TRACE_LEVEL

GATT layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for GATT layer

Available options:

- NONE (CONFIG_BT_LOG_GATT_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_GATT_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_GATT_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_GATT_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_GATT_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_GATT_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_GATT_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_SMP_TRACE_LEVEL

SMP layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for SMP layer

Available options:

- NONE (CONFIG_BT_LOG_SMP_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_SMP_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_SMP_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_SMP_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_SMP_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_SMP_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_SMP_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_BTIF_TRACE_LEVEL

BTIF layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for BTIF layer

Available options:

- NONE (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_BTIF_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_BTC_TRACE_LEVEL

BTC layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for BTC layer

Available options:

- NONE (CONFIG_BT_LOG_BTC_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_BTC_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_BTC_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_BTC_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_BTC_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_BTC_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_BTC_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_OSI_TRACE_LEVEL

OSI layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for OSI layer

Available options:

- NONE (CONFIG_BT_LOG_OSI_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_OSI_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_OSI_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_OSI_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_OSI_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_OSI_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_OSI_TRACE_LEVEL_VERBOSE)

CONFIG_BT_LOG_BLUFI_TRACE_LEVEL

BLUFI layer

Found in: Component config > Bluetooth > Bluedroid Options > BT DEBUG LOG LEVEL

Define BT trace level for BLUFI layer

Available options:

- NONE (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_WARNING)
- API (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_API)
- EVENT (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_EVENT)
- DEBUG (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BT_LOG_BLUFI_TRACE_LEVEL_VERBOSE)

CONFIG_BT_ACL_CONNECTIONS

BT/BLE MAX ACL CONNECTIONS(1~9)

Found in: Component config > Bluetooth > Bluedroid Options

Maximum BT/BLE connection count. The ESP32-C3/S3 chip supports a maximum of 10 instances, including ADV, SCAN and connections. The ESP32-C3/S3 chip can connect up to 9 devices if ADV or SCAN uses only one. If ADV and SCAN are both used, The ESP32-C3/S3 chip is connected to a maximum of 8 devices. Because Bluetooth cannot reclaim used instances once ADV or SCAN is used.

Range:

- from 1 to 9 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 4 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_MULTI_CONNECTION_ENBALE

Enable BLE multi-conections

Found in: Component config > Bluetooth > Bluedroid Options

Enable this option if there are multiple connections

Default value:

- Yes (enabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_ALLOCATION_FROM_SPIRAM_FIRST

BT/BLE will first malloc the memory from the PSRAM

Found in: Component config > Bluetooth > Bluedroid Options

This select can save the internal RAM if there have the PSRAM

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_DYNAMIC_ENV_MEMORY

Use dynamic memory allocation in BT/BLE stack

Found in: Component config > Bluetooth > Bluedroid Options

This select can make the allocation of memory will become more flexible

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_HOST_QUEUE_CONG_CHECK

BLE queue congestion check

Found in: Component config > Bluetooth > Bluedroid Options

When scanning and scan duplicate is not enabled, if there are a lot of adv packets around or application layer handling adv packets is slow, it will cause the controller memory to run out. if enabled, adv packets will be lost when host queue is congested.

Default value:

- No (disabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_SMP_MAX_BONDS

BT/BLE maximum bond device count

Found in: Component config > Bluetooth > Bluedroid Options

The number of security records for peer devices.

CONFIG_BT_BLE_ACT_SCAN_REP_ADV_SCAN

Report adv data and scan response individually when BLE active scan

Found in: Component config > Bluetooth > Bluedroid Options

Originally, when doing BLE active scan, Bluedroid will not report adv to application layer until receive scan response. This option is used to disable the behavior. When enable this option, Bluedroid will report adv data or scan response to application layer immediately.

Memory reserved at start of DRAM for Bluetooth stack

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_ESTAB_LINK_CONN_TOUT

Timeout of BLE connection establishment

Found in: Component config > Bluetooth > Bluedroid Options

Bluetooth Connection establishment maximum time, if connection time exceeds this value, the connection establishment fails, ESP_GATTC_OPEN_EVT or ESP_GATTS_OPEN_EVT is triggered.

Range:

- from 1 to 60 if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 30 if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_MAX_DEVICE_NAME_LEN

length of bluetooth device name

Found in: Component config > Bluetooth > Bluedroid Options

Bluetooth Device name length shall be no larger than 248 octets, If the broadcast data cannot contain the complete device name, then only the shortname will be displayed, the rest parts that can't fit in will be truncated.

Range:

- from 32 to 248 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 32 if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_RPA_SUPPORTED

Update RPA to Controller

Found in: Component config > Bluetooth > Bluedroid Options

This enables controller RPA list function. For ESP32, ESP32 only support network privacy mode. If this option is enabled, ESP32 will only accept advertising packets from peer devices that contain private address, HW will not receive the advertising packets contain identity address after IRK changed. If this option is disabled, address resolution will be performed in the host, so the functions that require controller to resolve address in the white list cannot be used. This option is disabled by default on ESP32, please enable or disable this option according to your own needs.

For other BLE chips, devices support network privacy mode and device privacy mode, users can switch the two modes according to their own needs. So this option is enabled by default.

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_RPA_TIMEOUT

Timeout of resolvable private address

Found in: Component config > Bluetooth > Bluedroid Options

This set RPA timeout of Controller and Host. Default is 900 s (15 minutes). Range is 1 s to 1 hour (3600 s).

Range:

- from 1 to 3600 if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

Default value:

- 900 if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_50_FEATURES_SUPPORTED

Enable BLE 5.0 features

Found in: Component config > Bluetooth > Bluedroid Options

Enabling this option activates BLE 5.0 features. This option is universally supported in chips that support BLE, except for ESP32.

Default value:

- Yes (enabled) if *CONFIG_BT_BLE_ENABLED* && (*CONFIG_BT_CONTROLLER_ENABLED* || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_42_FEATURES_SUPPORTED

Enable BLE 4.2 features

Found in: Component config > Bluetooth > Bluedroid Options

This enables BLE 4.2 features.

Default value:

- No (disabled) if *CONFIG_BT_BLE_ENABLED* && (*CONFIG_BT_CONTROLLER_ENABLED* || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_FEAT_PERIODIC_ADV_SYNC_TRANSFER

Enable BLE periodic advertising sync transfer feature

Found in: Component config > Bluetooth > Bluedroid Options

This enables BLE periodic advertising sync transfer feature

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLE_50_FEATURES_SUPPORTED* && ((*CONFIG_BT_CONTROLLER_ENABLED* && SOC_ESP_NIMBLE_CONTROLLER) || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_FEAT_PERIODIC_ADV_ENH

Enable periodic adv enhancements(adi support)

Found in: Component config > Bluetooth > Bluedroid Options

Enable the periodic advertising enhancements

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLE_50_FEATURES_SUPPORTED* && ((*CONFIG_BT_CONTROLLER_ENABLED* && SOC_ESP_NIMBLE_CONTROLLER) || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_FEAT_CREATE_SYNC_ENH

Enable create sync enhancements(reporting disable and duplicate filtering enable support)

Found in: Component config > Bluetooth > Bluedroid Options

Enable the create sync enhancements

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* && *CONFIG_BT_BLE_50_FEATURES_SUPPORTED* && ((*CONFIG_BT_CONTROLLER_ENABLED* && SOC_ESP_NIMBLE_CONTROLLER) || *CONFIG_BT_CONTROLLER_DISABLED*) && *CONFIG_BT_BLUEDROID_ENABLED*

CONFIG_BT_BLE_HIGH_DUTY_ADV_INTERVAL

Enable BLE high duty advertising interval feature

Found in: Component config > Bluetooth > Bluedroid Options

This enable BLE high duty advertising interval feature

Default value:

- No (disabled) if *CONFIG_BT_BLE_ENABLED* && *CONFIG_BT_BLUEDROID_ENABLED*

NimBLE Options

Contains:

- *CONFIG_BT_NIMBLE_SVC_GAP_DEVICE_NAME*
- *CONFIG_BT_NIMBLE_HS_STOP_TIMEOUT_MS*
- *CONFIG_BT_NIMBLE_HOST_QUEUE_CONG_CHECK*
- *BLE Services*
- *CONFIG_BT_NIMBLE_WHITELIST_SIZE*
- *CONFIG_BT_NIMBLE_BLE_GATT_BLOB_TRANSFER*
- *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT*
- *CONFIG_BT_NIMBLE_ROLE_BROADCASTER*
- *CONFIG_BT_NIMBLE_ROLE_CENTRAL*
- *CONFIG_BT_NIMBLE_HIGH_DUTY_ADV_ITVL*
- *CONFIG_BT_NIMBLE_MESH*
- *CONFIG_BT_NIMBLE_ROLE_OBSERVER*
- *CONFIG_BT_NIMBLE_ROLE_PERIPHERAL*
- *CONFIG_BT_NIMBLE_SECURITY_ENABLE*
- *CONFIG_BT_NIMBLE_BLUFI_ENABLE*
- *CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT*
- *CONFIG_BT_NIMBLE_DYNAMIC_SERVICE*
- *CONFIG_BT_NIMBLE_USE_ESP_TIMER*
- *CONFIG_BT_NIMBLE_DEBUG*
- *CONFIG_BT_NIMBLE_HS_FLOW_CTRL*
- *CONFIG_BT_NIMBLE_VS_SUPPORT*
- *CONFIG_BT_NIMBLE_OPTIMIZE_MULTI_CONN*
- *CONFIG_BT_NIMBLE_ENC_ADV_DATA*
- *CONFIG_BT_NIMBLE_SVC_GAP_APPEARANCE*
- *GAP Service*
- *Host-controller Transport*
- *CONFIG_BT_NIMBLE_GAP_DEVICE_NAME_MAX_LEN*
- *CONFIG_BT_NIMBLE_MAX_BONDS*
- *CONFIG_BT_NIMBLE_MAX_CCCDS*
- *CONFIG_BT_NIMBLE_MAX_CONNECTIONS*
- *CONFIG_BT_NIMBLE_L2CAP_COC_MAX_NUM*
- *CONFIG_BT_NIMBLE_GATT_MAX_PROCS*
- *CONFIG_BT_NIMBLE_MEM_ALLOC_MODE*
- *Memory Settings*
- *CONFIG_BT_NIMBLE_LOG_LEVEL*
- *CONFIG_BT_NIMBLE_HOST_TASK_STACK_SIZE*
- *CONFIG_BT_NIMBLE_CRYPTO_STACK_MBEDTLS*
- *CONFIG_BT_NIMBLE_NVS_PERSIST*
- *CONFIG_BT_NIMBLE_ATT_PREFERRED_MTU*
- *CONFIG_BT_NIMBLE_SMP_ID_RESET*
- *CONFIG_BT_NIMBLE_RPA_TIMEOUT*

- *CONFIG_BT_NIMBLE_PINNED_TO_CORE_CHOICE*
- *CONFIG_BT_NIMBLE_TEST_THROUGHPUT_TEST*

CONFIG_BT_NIMBLE_MEM_ALLOC_MODE

Memory allocation strategy

Found in: Component config > Bluetooth > NimBLE Options

Allocation strategy for NimBLE host stack, essentially provides ability to allocate all required dynamic allocations from,

- Internal DRAM memory only
- External SPIRAM memory only
- Either internal or external memory based on default malloc() behavior in ESP-IDF
- Internal IRAM memory wherever applicable else internal DRAM

Available options:

- Internal memory (CONFIG_BT_NIMBLE_MEM_ALLOC_MODE_INTERNAL)
- External SPIRAM (CONFIG_BT_NIMBLE_MEM_ALLOC_MODE_EXTERNAL)
- Default alloc mode (CONFIG_BT_NIMBLE_MEM_ALLOC_MODE_DEFAULT)
- Internal IRAM (CONFIG_BT_NIMBLE_MEM_ALLOC_MODE_IRAM_8BIT)
  Allows to use IRAM memory region as 8bit accessible region.
  Every unaligned (8bit or 16bit) access will result in an exception and incur penalty of certain clock cycles per unaligned read/write.

CONFIG_BT_NIMBLE_LOG_LEVEL

NimBLE Host log verbosity

Found in: Component config > Bluetooth > NimBLE Options

Select NimBLE log level. Please make a note that the selected NimBLE log verbosity can not exceed the level set in "Component config --> Log output --> Default log verbosity".

Available options:

- No logs (CONFIG_BT_NIMBLE_LOG_LEVEL_NONE)
- Error logs (CONFIG_BT_NIMBLE_LOG_LEVEL_ERROR)
- Warning logs (CONFIG_BT_NIMBLE_LOG_LEVEL_WARNING)
- Info logs (CONFIG_BT_NIMBLE_LOG_LEVEL_INFO)
- Debug logs (CONFIG_BT_NIMBLE_LOG_LEVEL_DEBUG)

CONFIG_BT_NIMBLE_MAX_CONNECTIONS

Maximum number of concurrent connections

Found in: Component config > Bluetooth > NimBLE Options

Defines maximum number of concurrent BLE connections. For ESP32, user is expected to configure BTDM_CTRL_BLE_MAX_CONN from controller menu along with this option. Similarly for ESP32-C3 or ESP32-S3, user is expected to configure BT_CTRL_BLE_MAX_ACT from controller menu. For ESP32C2, ESP32C6 and ESP32H2, each connection will take about 1k DRAM.

Range:

- from 1 to 9 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 3 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_BONDS

Maximum number of bonds to save across reboots

Found in: Component config > Bluetooth > NimBLE Options

Defines maximum number of bonds to save for peer security and our security

Default value:

- 3 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_CCCDS

Maximum number of CCC descriptors to save across reboots

Found in: Component config > Bluetooth > NimBLE Options

Defines maximum number of CCC descriptors to save

Default value:

- 8 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_L2CAP_COC_MAX_NUM

Maximum number of connection oriented channels

Found in: Component config > Bluetooth > NimBLE Options

Defines maximum number of BLE Connection Oriented Channels. When set to (0), BLE COC is not compiled in

Range:

- from 0 to 9 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 0 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_PINNED_TO_CORE_CHOICE

The CPU core on which NimBLE host will run

Found in: Component config > Bluetooth > NimBLE Options

The CPU core on which NimBLE host will run. You can choose Core 0 or Core 1. Cannot specify no-affinity

Available options:

- Core 0 (PRO CPU) (CONFIG_BT_NIMBLE_PINNED_TO_CORE_0)
- Core 1 (APP CPU) (CONFIG_BT_NIMBLE_PINNED_TO_CORE_1)

CONFIG_BT_NIMBLE_HOST_TASK_STACK_SIZE

NimBLE Host task stack size

Found in: Component config > Bluetooth > NimBLE Options

This configures stack size of NimBLE host task

Default value:

- 5120 if *CONFIG_BLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*
- 4096 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ROLE_CENTRAL

Enable BLE Central role

Found in: Component config > Bluetooth > NimBLE Options

Enables central role

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ROLE_PERIPHERAL

Enable BLE Peripheral role

Found in: Component config > Bluetooth > NimBLE Options

Enable peripheral role

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ROLE_BROADCASTER

Enable BLE Broadcaster role

Found in: Component config > Bluetooth > NimBLE Options

Enables broadcaster role

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ROLE_OBSERVER

Enable BLE Observer role

Found in: Component config > Bluetooth > NimBLE Options

Enables observer role

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_NVS_PERSIST

Persist the BLE Bonding keys in NVS

Found in: Component config > Bluetooth > NimBLE Options

Enable this flag to make bonding persistent across device reboots

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SMP_ID_RESET

Reset device identity when all bonding records are deleted

Found in: Component config > Bluetooth > NimBLE Options

There are tracking risks associated with using a fixed or static IRK. If enabled this option, Bluedroid will assign a new randomly-generated IRK when all pairing and bonding records are deleted. This would decrease the ability of a previously paired peer to be used to determine whether a device with which it previously shared an IRK is within range.

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SECURITY_ENABLE

Enable BLE SM feature

Found in: Component config > Bluetooth > NimBLE Options

Enable BLE sm feature

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Contains:

- *CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_ENCRYPTION*
- *CONFIG_BT_NIMBLE_SM_LVL*
- *CONFIG_BT_NIMBLE_SM_LEGACY*
- *CONFIG_BT_NIMBLE_SM_SC*

CONFIG_BT_NIMBLE_SM_LEGACY

Security manager legacy pairing

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_SECURITY_ENABLE

Enable security manager legacy pairing

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_SECURITY_ENABLE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SM_SC

Security manager secure connections (4.2)

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_SECURITY_ENABLE

Enable security manager secure connections

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_SECURITY_ENABLE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SM_SC_DEBUG_KEYS

Use predefined public-private key pair

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_SECURITY_ENABLE > CONFIG_BT_NIMBLE_SM_SC

If this option is enabled, SM uses predefined DH key pair as described in Core Specification, Vol. 3, Part H, 2.3.5.6.1. This allows to decrypt air traffic easily and thus should only be used for debugging.

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SECURITY_ENABLE* && *CONFIG_BT_NIMBLE_SM_SC* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_ENCRYPTION

Enable LE encryption

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_SECURITY_ENABLE

Enable encryption connection

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_SECURITY_ENABLE* && *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SM_LVL

Security level

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_SECURITY_ENABLE

LE Security Mode 1 Levels: 1. No Security 2. Unauthenticated pairing with encryption 3. Authenticated pairing with encryption 4. Authenticated LE Secure Connections pairing with encryption using a 128-bit strength encryption key.

Default value:

- 0 if *CONFIG_BT_NIMBLE_SECURITY_ENABLE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_DEBUG

Enable extra runtime asserts and host debugging

Found in: Component config > Bluetooth > NimBLE Options

This enables extra runtime asserts and host debugging

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_DYNAMIC_SERVICE

Enable dynamic services

Found in: Component config > Bluetooth > NimBLE Options

This enables user to add/remove Gatt services at runtime

CONFIG_BT_NIMBLE_SVC_GAP_DEVICE_NAME

BLE GAP default device name

Found in: Component config > Bluetooth > NimBLE Options

The Device Name characteristic shall contain the name of the device as an UTF-8 string. This name can be changed by using API ble_svc_gap_device_name_set()

Default value:

- "nimble" if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_GAP_DEVICE_NAME_MAX_LEN

Maximum length of BLE device name in octets

Found in: Component config > Bluetooth > NimBLE Options

Device Name characteristic value shall be 0 to 248 octets in length

Default value:

- 31 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ATT_PREFERRED_MTU

Preferred MTU size in octets

Found in: Component config > Bluetooth > NimBLE Options

This is the default value of ATT MTU indicated by the device during an ATT MTU exchange. This value can be changed using API ble_att_set_preferred_mtu()

Default value:

- 256 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_APPEARANCE

External appearance of the device

Found in: Component config > Bluetooth > NimBLE Options

Standard BLE GAP Appearance value in HEX format e.g. 0x02C0

Default value:

- 0 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Memory Settings

Contains:

- *CONFIG_BT_NIMBLE_TRANSPORT_ACL_FROM_LL_COUNT*
- *CONFIG_BT_NIMBLE_TRANSPORT_EVT_DISCARD_COUNT*
- *CONFIG_BT_NIMBLE_MSYS_BUF_FROM_HEAP*
- *CONFIG_BT_NIMBLE_L2CAP_COC_SDU_BUFF_COUNT*
- *CONFIG_BT_NIMBLE_MSYS_1_BLOCK_COUNT*
- *CONFIG_BT_NIMBLE_MSYS_1_BLOCK_SIZE*
- *CONFIG_BT_NIMBLE_MSYS_2_BLOCK_COUNT*
- *CONFIG_BT_NIMBLE_MSYS_2_BLOCK_SIZE*
- *CONFIG_BT_NIMBLE_TRANSPORT_ACL_SIZE*
- *CONFIG_BT_NIMBLE_TRANSPORT_EVT_COUNT*
- *CONFIG_BT_NIMBLE_TRANSPORT_EVT_SIZE*

CONFIG_BT_NIMBLE_MSYS_1_BLOCK_COUNT

MSYS_1 Block Count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

MSYS is a system level mbuf registry. For prepare write & prepare responses MBUFs are allocated out of msys_1 pool. For NIMBLE_MESH enabled cases, this block count is increased by 8 than user defined count.

Default value:

- 24 if SOC_ESP_NIMBLE_CONTROLLER && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MSYS_1_BLOCK_SIZE

MSYS_1 Block Size

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

Dynamic memory size of block 1

Default value:

- 128 if SOC_ESP_NIMBLE_CONTROLLER && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MSYS_2_BLOCK_COUNT

MSYS_2 Block Count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

Dynamic memory count

Default value:

- 24 if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MSYS_2_BLOCK_SIZE

MSYS_2 Block Size

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

Dynamic memory size of block 2

Default value:

- 320 if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MSYS_BUF_FROM_HEAP

Get Msys Mbuf from heap

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This option sets the source of the shared msys mbuf memory between the Host and the Controller. Allocate the memory from the heap if this option is sets, from the mempool otherwise.

Default value:

- Yes (enabled) if BT_LE_MSYS_INIT_IN_CONTROLLER && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_ACL_FROM_LL_COUNT

ACL Buffer count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

The number of ACL data buffers allocated for host.

Default value:

- 24 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_ACL_SIZE

Transport ACL Buffer size

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This is the maximum size of the data portion of HCI ACL data packets. It does not include the HCI data header (of 4 bytes)

Default value:

- 255 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_EVT_SIZE

Transport Event Buffer size

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This is the size of each HCI event buffer in bytes. In case of extended advertising, packets can be fragmented. 257 bytes is the maximum size of a packet.

Default value:

- 257 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*
- 70 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_EVT_COUNT

Transport Event Buffer count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This is the high priority HCI events' buffer size. High-priority event buffers are for everything except advertising reports. If there are no free high-priority event buffers then host will try to allocate a low-priority buffer instead

Default value:

- 30 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_EVT_DISCARD_COUNT

Discardable Transport Event Buffer count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This is the low priority HCI events' buffer size. Low-priority event buffers are only used for advertising reports. If there are no free low-priority event buffers, then an incoming advertising report will get dropped

Default value:

- 8 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_L2CAP_COC_SDU_BUFF_COUNT

L2cap coc Service Data Unit Buffer count

Found in: Component config > Bluetooth > NimBLE Options > Memory Settings

This is the service data unit buffer count for l2cap coc.

Default value:

- 1 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_GATT_MAX_PROCS

Maximum number of GATT client procedures

Found in: Component config > Bluetooth > NimBLE Options

Maximum number of GATT client procedures that can be executed.

Default value:

- 4 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HS_FLOW_CTRL

Enable Host Flow control

Found in: Component config > Bluetooth > NimBLE Options

Enable Host Flow control

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HS_FLOW_CTRL_ITVL

Host Flow control interval

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_HS_FLOW_CTRL

Host flow control interval in msecs

Default value:

- 1000 if *CONFIG_BT_NIMBLE_HS_FLOW_CTRL* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HS_FLOW_CTRL_THRESH

Host Flow control threshold

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_HS_FLOW_CTRL

Host flow control threshold, if the number of free buffers are at or below this threshold, send an immediate number-of-completed-packets event

Default value:

- 2 if *CONFIG_BT_NIMBLE_HS_FLOW_CTRL* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HS_FLOW_CTRL_TX_ON_DISCONNECT

Host Flow control on disconnect

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_HS_FLOW_CTRL

Enable this option to send number-of-completed-packets event to controller after disconnection

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_HS_FLOW_CTRL* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_RPA_TIMEOUT

RPA timeout in seconds

Found in: Component config > Bluetooth > NimBLE Options

Time interval between RPA address change.

Range:

- from 1 to 41400 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 900 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH

Enable BLE mesh functionality

Found in: Component config > Bluetooth > NimBLE Options

Enable BLE Mesh example present in upstream mynewt-nimble and not maintained by Espressif.

IDF maintains ESP-BLE-MESH as the official Mesh solution. Please refer to ESP-BLE-MESH guide at: *:doc:../esp32/api-guides/esp-ble-mesh/ble-mesh-index``*

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Contains:

- *CONFIG_BT_NIMBLE_MESH_PROVISIONER*
- *CONFIG_BT_NIMBLE_MESH_PROV*

- *CONFIG_BT_NIMBLE_MESH_GATT_PROXY*
- *CONFIG_BT_NIMBLE_MESH_FRIEND*
- *CONFIG_BT_NIMBLE_MESH_LOW_POWER*
- *CONFIG_BT_NIMBLE_MESH_PROXY*
- *CONFIG_BT_NIMBLE_MESH_RELAY*
- *CONFIG_BT_NIMBLE_MESH_DEVICE_NAME*
- *CONFIG_BT_NIMBLE_MESH_NODE_COUNT*

CONFIG_BT_NIMBLE_MESH_PROXY

Enable mesh proxy functionality

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH

Enable proxy. This is automatically set whenever NIMBLE_MESH_PB_GATT or NIMBLE_MESH_GATT_PROXY is set

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_PROV

Enable BLE mesh provisioning

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH

Enable mesh provisioning

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_PB_ADV

Enable mesh provisioning over advertising bearer

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH > CONFIG_BT_NIMBLE_MESH_PROV

Enable this option to allow the device to be provisioned over the advertising bearer

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_MESH_PROV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_PB_GATT

Enable mesh provisioning over GATT bearer

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH > CONFIG_BT_NIMBLE_MESH_PROV

Enable this option to allow the device to be provisioned over the GATT bearer

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_MESH_PROV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_GATT_PROXY

Enable GATT Proxy functionality

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH

This option enables support for the Mesh GATT Proxy Service, i.e. the ability to act as a proxy between a Mesh GATT Client and a Mesh network

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_RELAY

Enable mesh relay functionality

Found in: Component config > *Bluetooth* > *NimBLE Options* > *CONFIG_BT_NIMBLE_MESH*

Support for acting as a Mesh Relay Node

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_LOW_POWER

Enable mesh low power mode

Found in: Component config > *Bluetooth* > *NimBLE Options* > *CONFIG_BT_NIMBLE_MESH*

Enable this option to be able to act as a Low Power Node

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_FRIEND

Enable mesh friend functionality

Found in: Component config > *Bluetooth* > *NimBLE Options* > *CONFIG_BT_NIMBLE_MESH*

Enable this option to be able to act as a Friend Node

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_DEVICE_NAME

Set mesh device name

Found in: Component config > *Bluetooth* > *NimBLE Options* > *CONFIG_BT_NIMBLE_MESH*

This value defines Bluetooth Mesh device/node name

Default value:

- "nimble-mesh-node" if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_NODE_COUNT

Set mesh node count

Found in: Component config > *Bluetooth* > *NimBLE Options* > *CONFIG_BT_NIMBLE_MESH*

Defines mesh node count.

Default value:

- 1 if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MESH_PROVISIONER

Enable BLE mesh provisioner

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_MESH

Enable mesh provisioner.

Default value:

- 0 if *CONFIG_BT_NIMBLE_MESH* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_CRYPTO_STACK_MBEDTLS

Override TinyCrypt with mbedTLS for crypto computations

Found in: Component config > Bluetooth > NimBLE Options

Enable this option to choose mbedTLS instead of TinyCrypt for crypto computations.

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HS_STOP_TIMEOUT_MS

BLE host stop timeout in msec

Found in: Component config > Bluetooth > NimBLE Options

BLE Host stop procedure timeout in milliseconds.

Default value:

- 2000 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT

Enable connection reattempts on connection establishment error

Found in: Component config > Bluetooth > NimBLE Options

Enable to make the NimBLE host to reattempt GAP connection on connection establishment failure.

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_CONN_REATTEMPT

Maximum number connection reattempts

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT

Defines maximum number of connection reattempts.

Range:

- from 1 to 255 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 3 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLE_CONN_REATTEMPT* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable BLE 5 feature

Found in: Component config > Bluetooth > NimBLE Options

Enable BLE 5 feature

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Contains:

- *CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_2M_PHY*
- *CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_CODED_PHY*
- *CONFIG_BT_NIMBLE_EXT_ADV*
- *CONFIG_BT_NIMBLE_GATT_CACHING*
- *CONFIG_BT_NIMBLE_BLE_POWER_CONTROL*
- *CONFIG_BT_NIMBLE_MAX_PERIODIC_ADVERTISER_LIST*
- *CONFIG_BT_NIMBLE_MAX_PERIODIC_SYNCS*
- *CONFIG_BT_NIMBLE_PERIODIC_ADV_ENH*

CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_2M_PHY

Enable 2M Phy

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable 2M-PHY

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_LL_CFG_FEAT_LE_CODED_PHY

Enable coded Phy

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable coded-PHY

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_EXT_ADV

Enable extended advertising

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable this option to do extended advertising. Extended advertising will be supported from BLE 5.0 onwards.

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_EXT_ADV_INSTANCES

Maximum number of extended advertising instances.

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_EXT_ADV

Change this option to set maximum number of extended advertising instances. Minimum there is always one instance of advertising. Enter how many more advertising instances you want. For ESP32C2, ESP32C6 and ESP32H2, each extended advertising instance will take about 0.5k DRAM.

Range:

- from 0 to 4 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 1 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*
- 0 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_EXT_ADV_MAX_SIZE

Maximum length of the advertising data.

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_EXT_ADV

Defines the length of the extended adv data. The value should not exceed 1650.

Range:

- from 0 to 1650 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 1650 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*
- 0 if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ENABLE_PERIODIC_ADV

Enable periodic advertisement.

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_EXT_ADV

Enable this option to start periodic advertisement.

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_PERIODIC_ADV_SYNC_TRANSFER

Enable Transfer Sync Events

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_EXT_ADV > CONFIG_BT_NIMBLE_ENABLE_PERIODIC_ADV

This enables controller transfer periodic sync events to host

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLE_PERIODIC_ADV* && *CONFIG_BT_NIMBLE_EXT_ADV* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_PERIODIC_SYNCS

Maximum number of periodic advertising syncs

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Set this option to set the upper limit for number of periodic sync connections. This should be less than maximum connections allowed by controller.

Range:

- from 0 to 8 if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 1 if *CONFIG_BT_NIMBLE_ENABLE_PERIODIC_ADV* && *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*
- 0 if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_MAX_PERIODIC_ADVERTISER_LIST

Maximum number of periodic advertiser list

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Set this option to set the upper limit for number of periodic advertiser list.

Range:

- from 1 to 5 if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && SOC_ESP_NIMBLE_CONTROLLER && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 5 if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && SOC_ESP_NIMBLE_CONTROLLER && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_BLE_POWER_CONTROL

Enable support for BLE Power Control

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Set this option to enable the Power Control feature

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT* && SOC_BLE_POWER_CONTROL_SUPPORTED && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_PERIODIC_ADV_ENH

Periodic adv enhancements(adi support)

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable the periodic advertising enhancements

CONFIG_BT_NIMBLE_GATT_CACHING

Enable GATT caching

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT

Enable GATT caching

Contains:

- *CONFIG_BT_NIMBLE_GATT_CACHING_MAX_CONNS*
- *CONFIG_BT_NIMBLE_GATT_CACHING_MAX_CHRS*
- *CONFIG_BT_NIMBLE_GATT_CACHING_MAX_DSCS*
- *CONFIG_BT_NIMBLE_GATT_CACHING_MAX_SVCS*

CONFIG_BT_NIMBLE_GATT_CACHING_MAX_CONNS

Maximum connections to be cached

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_GATT_CACHING

Set this option to set the upper limit on number of connections to be cached.

Default value:

- 1 if *CONFIG_BT_NIMBLE_GATT_CACHING* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_GATT_CACHING_MAX_SVCS

Maximum number of services per connection

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_GATT_CACHING

Set this option to set the upper limit on number of services per connection to be cached.

Default value:

- 64 if *CONFIG_BT_NIMBLE_GATT_CACHING* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_GATT_CACHING_MAX_CHRS

Maximum number of characteristics per connection

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_GATT_CACHING

Set this option to set the upper limit on number of characteristics per connection to be cached.

Default value:

- 64 if *CONFIG_BT_NIMBLE_GATT_CACHING* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_GATT_CACHING_MAX_DSCS

Maximum number of descriptors per connection

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_50_FEATURE_SUPPORT > CONFIG_BT_NIMBLE_GATT_CACHING

Set this option to set the upper limit on number of descriptors per connection to be cached.

Default value:

- 64 if *CONFIG_BT_NIMBLE_GATT_CACHING* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_WHITELIST_SIZE

BLE white list size

Found in: Component config > Bluetooth > NimBLE Options

BLE list size

Range:

- from 1 to 15 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Default value:

- 12 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TEST_THROUGHPUT_TEST

Throughput Test Mode enable

Found in: Component config > Bluetooth > NimBLE Options

Enable the throughput test mode

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_BLUFI_ENABLE

Enable blufi functionality

Found in: Component config > Bluetooth > NimBLE Options

Set this option to enable blufi functionality.

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_USE_ESP_TIMER

Enable Esp Timer for Nimble

Found in: Component config > Bluetooth > NimBLE Options

Set this option to use Esp Timer which has higher priority timer instead of FreeRTOS timer

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_BLE_GATT_BLOB_TRANSFER

Blob transfer

Found in: Component config > Bluetooth > NimBLE Options

This option is used when data to be sent is more than 512 bytes. For peripheral role, BT_NIMBLE_MSYS_1_BLOCK_COUNT needs to be increased according to the need.

GAP Service Contains:

- *GAP Appearance write permissions*
- *CONFIG_BT_NIMBLE_SVC_GAP_CENT_ADDR_RESOLUTION*
- *GAP device name write permissions*
- *CONFIG_BT_NIMBLE_SVC_GAP_PPCP_MAX_CONN_INTERVAL*
- *CONFIG_BT_NIMBLE_SVC_GAP_PPCP_MIN_CONN_INTERVAL*
- *CONFIG_BT_NIMBLE_SVC_GAP_PPCP_SLAVE_LATENCY*
- *CONFIG_BT_NIMBLE_SVC_GAP_PPCP_SUPERVISION_TMO*

GAP Appearance write permissions Contains:

- *CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE*

CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE

Write

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP Appearance write permissions

Enable write permission (BLE_GATT_CHR_F_WRITE)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE_ENC

Write with encryption

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP Appearance write permissions > CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE

Enable write with encryption permission (BLE_GATT_CHR_F_WRITE_ENC)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE_AUTHEN

Write with authentication

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP Appearance write permissions > CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE

Enable write with authentication permission (BLE_GATT_CHR_F_WRITE_AUTHEN)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE_AUTHOR

Write with authorisation

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP Appearance write permissions > CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE

Enable write with authorisation permission (BLE_GATT_CHR_F_WRITE_AUTHOR)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_APPEAR_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_CENT_ADDR_RESOLUTION

GAP Characteristic - Central Address Resolution

Found in: Component config > Bluetooth > NimBLE Options > GAP Service

Weather or not Central Address Resolution characteristic is supported on the device, and if supported, weather or not Central Address Resolution is supported.

- Central Address Resolution characteristic not supported
- Central Address Resolution not supported
- Central Address Resolution supported

Available options:

- Characteristic not supported (CONFIG_BT_NIMBLE_SVC_GAP_CAR_CHAR_NOT_SUPP)
- Central Address Resolution not supported (CONFIG_BT_NIMBLE_SVC_GAP_CAR_NOT_SUPP)
- Central Address Resolution supported (CONFIG_BT_NIMBLE_SVC_GAP_CAR_SUPP)

GAP device name write permissions Contains:

- *CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE*

CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE

Write

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP device name write permissions

Enable write permission (BLE_GATT_CHR_F_WRITE)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE_ENC

Write with encryption

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP device name write permissions > CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE

Enable write with encryption permission (BLE_GATT_CHR_F_WRITE_ENC)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE_AUTHEN

Write with authentication

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP device name write permissions > CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE

Enable write with authentication permission (BLE_GATT_CHR_F_WRITE_AUTHEN)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE_AUTHOR

Write with authorisation

Found in: Component config > Bluetooth > NimBLE Options > GAP Service > GAP device name write permissions > CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE

Enable write with authorisation permission (BLE_GATT_CHR_F_WRITE_AUTHOR)

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_SVC_GAP_NAME_WRITE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_PPCP_MAX_CONN_INTERVAL

PPCP Connection Interval Max (Unit: 1.25 ms)

Found in: Component config > Bluetooth > NimBLE Options > GAP Service

Peripheral Preferred Connection Parameter: Connection Interval maximum value Interval Max = value * 1.25 ms

Default value:

- 0 if *CONFIG_BT_NIMBLE_ROLE_PERIPHERAL* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_PPCP_MIN_CONN_INTERVAL

PPCP Connection Interval Min (Unit: 1.25 ms)

Found in: Component config > Bluetooth > NimBLE Options > GAP Service

Peripheral Preferred Connection Parameter: Connection Interval minimum value Interval Min = value * 1.25 ms

Default value:

- 0 if *CONFIG_BT_NIMBLE_ROLE_PERIPHERAL* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_PPCP_SLAVE_LATENCY

PPCP Slave Latency

Found in: Component config > Bluetooth > NimBLE Options > GAP Service

Peripheral Preferred Connection Parameter: Slave Latency

Default value:

- 0 if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_GAP_PPCP_SUPERVISION_TMO

PPCP Supervision Timeout (Uint: 10 ms)

Found in: Component config > Bluetooth > NimBLE Options > GAP Service

Peripheral Preferred Connection Parameter: Supervision Timeout Timeout = Value * 10 ms

Default value:

- 0 if *CONFIG_BT_NIMBLE_ENABLED*

BLE Services Contains:

- *CONFIG_BT_NIMBLE_HID_SERVICE*

CONFIG_BT_NIMBLE_HID_SERVICE

HID service

Found in: Component config > Bluetooth > NimBLE Options > BLE Services

Enable HID service support

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Contains:

- *CONFIG_BT_NIMBLE_SVC_HID_MAX_RPTS*
- *CONFIG_BT_NIMBLE_SVC_HID_MAX_INSTANCES*

CONFIG_BT_NIMBLE_SVC_HID_MAX_INSTANCES

Maximum HID service instances

Found in: Component config > Bluetooth > NimBLE Options > BLE Services > CONFIG_BT_NIMBLE_HID_SERVICE

Defines maximum number of HID service instances

Default value:

- 2 if *CONFIG_BT_NIMBLE_HID_SERVICE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_SVC_HID_MAX_RPTS

Maximum HID Report characteristics per service instance

Found in: Component config > Bluetooth > NimBLE Options > BLE Services > CONFIG_BT_NIMBLE_HID_SERVICE

Defines maximum number of report characteristics per service instance

Default value:

- 3 if *CONFIG_BT_NIMBLE_HID_SERVICE* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_VS_SUPPORT

Enable support for VSC and VSE

Found in: Component config > Bluetooth > NimBLE Options

This option is used to enable support for sending Vendor Specific HCI commands and handling Vendor Specific HCI Events.

CONFIG_BT_NIMBLE_OPTIMIZE_MULTI_CONN

Enable the optimization of multi-connection

Found in: Component config > Bluetooth > NimBLE Options

This option enables the use of vendor-specific APIs for multi-connections, which can greatly enhance the stability of coexistence between numerous central and peripheral devices. It will prohibit the usage of standard APIs.

Default value:

- No (disabled) if SOC_BLE_MULTI_CONN_OPTIMIZATION && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_ENC_ADV_DATA

Encrypted Advertising Data

Found in: Component config > Bluetooth > NimBLE Options

This option is used to enable encrypted advertising data.

CONFIG_BT_NIMBLE_MAX_EADS

Maximum number of EAD devices to save across reboots

Found in: Component config > Bluetooth > NimBLE Options > CONFIG_BT_NIMBLE_ENC_ADV_DATA

Defines maximum number of encrypted advertising data key material to save

Default value:

- 10 if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENC_ADV_DATA* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HIGH_DUTY_ADV_ITVL

Enable BLE high duty advertising interval feature

Found in: Component config > Bluetooth > NimBLE Options

This enable BLE high duty advertising interval feature

CONFIG_BT_NIMBLE_HOST_QUEUE_CONG_CHECK

BLE queue congestion check

Found in: Component config > Bluetooth > NimBLE Options

When scanning and scan duplicate is not enabled, if there are a lot of adv packets around or application layer handling adv packets is slow, it will cause the controller memory to run out. if enabled, adv packets will be lost when host queue is congested.

Default value:

- No (disabled) if *CONFIG_BT_NIMBLE_ENABLED* && *CONFIG_BT_NIMBLE_ENABLED*

Host-controller Transport Contains:

- *CONFIG_BT_NIMBLE_TRANSPORT_UART*
- *CONFIG_BT_NIMBLE_HCI_UART_CTS_PIN*
- *CONFIG_BT_NIMBLE_USE_HCI_UART_FLOW_CTRL*
- *CONFIG_BT_NIMBLE_HCI_UART_RTS_PIN*

CONFIG_BT_NIMBLE_TRANSPORT_UART

Enable Uart Transport

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport

Use UART transport

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_TRANSPORT_UART_PORT

Uart port

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport > CONFIG_BT_NIMBLE_TRANSPORT_UART

Uart port

Default value:

- 1 if *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_NIMBLE_TRANSPORT_UART* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HCI_USE_UART_BAUDRATE

Uart Hci Baud Rate

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport > CONFIG_BT_NIMBLE_TRANSPORT_UART

Uart Baud Rate

Available options:

- 115200 (CONFIG_UART_BAUDRATE_115200)
- 230400 (CONFIG_UART_BAUDRATE_230400)
- 460800 (CONFIG_UART_BAUDRATE_460800)
- 921600 (CONFIG_UART_BAUDRATE_921600)

CONFIG_BT_NIMBLE_USE_HCI_UART_PARITY

Uart PARITY

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport > CONFIG_BT_NIMBLE_TRANSPORT_UART

Uart Parity

Available options:

- None (CONFIG_UART_PARITY_NONE)
- Odd (CONFIG_UART_PARITY_ODD)
- Even (CONFIG_UART_PARITY_EVEN)

CONFIG_BT_NIMBLE_UART_RX_PIN

UART Rx pin

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport > CONFIG_BT_NIMBLE_TRANSPORT_UART

Rx pin for Nimble Transport

Default value:

- 5 if *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_NIMBLE_TRANSPORT_UART* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_UART_TX_PIN

UART Tx pin

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport > CONFIG_BT_NIMBLE_TRANSPORT_UART

Tx pin for Nimble Transport

Default value:

- 4 if *CONFIG_BT_CONTROLLER_DISABLED* && *CONFIG_BT_NIMBLE_TRANSPORT_UART* && *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_USE_HCI_UART_FLOW_CTRL

Uart Flow Control

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport

Uart Flow Control

Available options:

- Disable (CONFIG_UART_HW_FLOWCTRL_DISABLE)
- Enable hardware flow control (CONFIG_UART_HW_FLOWCTRL_CTS_RTS)

CONFIG_BT_NIMBLE_HCI_UART_RTS_PIN

UART Rts Pin

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport

UART HCI RTS pin

Default value:

- 19 if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_NIMBLE_HCI_UART_CTS_PIN

UART Cts Pin

Found in: Component config > Bluetooth > NimBLE Options > Host-controller Transport

UART HCI CTS pin

Default value:

- 23 if *CONFIG_BT_NIMBLE_ENABLED*

Controller Options Contains:

- *CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP*
- *CONFIG_BT_BLE_CCA_MODE*
- *CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL*
- *CONFIG_BT_CTRL_BLE_MAX_ACT*
- *CONFIG_BT_CTRL_BLE_SCAN_DUPL*
- *CONFIG_BT_CTRL_BLE_STATIC_ACL_TX_BUF_NB*
- *CONFIG_BT_CTRL_HW_CCA_VAL*
- *CONFIG_BT_CTRL_COEX_PHY_CODED_TX_RX_TLIM*
- *CONFIG_BT_CTRL_CE_LENGTH_TYPE*
- *CONFIG_BT_CTRL_RX_ANTENNA_INDEX*
- *CONFIG_BT_CTRL_TX_ANTENNA_INDEX*
- *CONFIG_BT_CTRL_SCAN_BACKOFF_UPPERLIMITMAX*
- *CONFIG_BT_BLE_ADV_DATA_LENGTH_ZERO_AUX*
- *CONFIG_BT_CTRL_CHAN_ASS_EN*
- *CONFIG_BT_CTRL_AGC_RECORRECT_EN*
- *CONFIG_BT_CTRL_LE_PING_EN*
- *CONFIG_BT_CTRL_HCI_MODE_CHOICE*
- *MODEM SLEEP Options*
- *CONFIG_BT_CTRL_PINNED_TO_CORE_CHOICE*
- *CONFIG_BT_CTRL_ADV_DUP_FILT_MAX*

CONFIG_BT_CTRL_BLE_MAX_ACT

BLE Max Instances

Found in: Component config > Bluetooth > Controller Options

BLE maximum activities of bluetooth controller , both of connections, scan , sync and adv(periodic adv, multi-adv). Each instance needs to consume 828 bytes, you can save RAM by modifying the instance value according to actual needs.

Range:

- from 1 to 10 if *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 6 if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_BLE_STATIC_ACL_TX_BUF_NB

BLE static ACL TX buffer numbers

Found in: Component config > Bluetooth > Controller Options

BLE ACL buffer have two methods to be allocated. One is persistent allocating (allocate when controller initialise, never free until controller de-initialise) another is dynamically allocating (allocate before TX and free after TX).

Range:
- from 0 to 12 if *CONFIG_BT_CONTROLLER_ENABLED*

Default value:
- 0 if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_PINNED_TO_CORE_CHOICE

The cpu core which bluetooth controller run

Found in: Component config > Bluetooth > Controller Options

Specify the cpu core to run bluetooth controller. Can not specify no-affinity.

Available options:

- Core 0 (PRO CPU) (CONFIG_BT_CTRL_PINNED_TO_CORE_0)
- Core 1 (APP CPU) (CONFIG_BT_CTRL_PINNED_TO_CORE_1)

CONFIG_BT_CTRL_HCI_MODE_CHOICE

HCI mode

Found in: Component config > Bluetooth > Controller Options

Specify HCI mode as VHCI or UART(H4)

Available options:

- VHCI (CONFIG_BT_CTRL_HCI_MODE_VHCI)
  Normal option. Mostly, choose this VHCI when bluetooth host run on ESP32S3 or ESP32C3.
- UART(H4) (CONFIG_BT_CTRL_HCI_MODE_UART_H4)
  If use external bluetooth host which run on other hardware and use UART as the HCI interface, choose this option.

CONFIG_BT_CTRL_ADV_DUP_FILT_MAX

The maximum number of 5.0 extend duplicate scan filter

Found in: Component config > Bluetooth > Controller Options

The maximum number of suplicate scan filter

Range:
- from 1 to 500 if *CONFIG_BT_CONTROLLER_ENABLED*

Default value:
- 30 if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_BLE_CCA_MODE

BLE CCA mode

Found in: Component config > Bluetooth > Controller Options

Define BT BLE CCA mode

Available options:

- NONE (CONFIG_BT_BLE_CCA_MODE_NONE)
- Hardware (CONFIG_BT_BLE_CCA_MODE_HW)
- Software (CONFIG_BT_BLE_CCA_MODE_SW)

CONFIG_BT_CTRL_HW_CCA_VAL

CCA threshold value

Found in: Component config > Bluetooth > Controller Options

It is the threshold value of HW CCA, if the value is 30, it means CCA threshold is -30 dBm.

Range:
- from 20 to 100 if *CONFIG_BT_CONTROLLER_ENABLED*

Default value:
- 20 if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_CE_LENGTH_TYPE

Connection event length determination method

Found in: Component config > Bluetooth > Controller Options

Specify connection event length determination

Available options:

- ORIGINAL (CONFIG_BT_CTRL_CE_LENGTH_TYPE_ORIG)
- Use CE parameter for HCI command (CONFIG_BT_CTRL_CE_LENGTH_TYPE_CE)
- Use Espressif self-defined method (CONFIG_BT_CTRL_CE_LENGTH_TYPE_SD)

CONFIG_BT_CTRL_TX_ANTENNA_INDEX

default Tx anntena used

Found in: Component config > Bluetooth > Controller Options

Specify default Tx antenna used for bluetooth

Available options:

- Antenna 0 (CONFIG_BT_CTRL_TX_ANTENNA_INDEX_0)
- Antenna 1 (CONFIG_BT_CTRL_TX_ANTENNA_INDEX_1)

CONFIG_BT_CTRL_RX_ANTENNA_INDEX

default Rx anntena used

Found in: Component config > Bluetooth > Controller Options

Specify default Rx antenna used for bluetooth

Available options:

- Antenna 0 (CONFIG_BT_CTRL_RX_ANTENNA_INDEX_0)
- Antenna 1 (CONFIG_BT_CTRL_RX_ANTENNA_INDEX_1)

CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL

BLE default Tx power level

Found in: Component config > Bluetooth > Controller Options

Specify default Tx power level

Available options:

- -24dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N24)
- -21dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N21)
- -18dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N18)
- -15dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N15)
- -12dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N12)
- **-9dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N9)
- **-6dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N6)
- **-3dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N3)
- 0dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_N0)
- **+3dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P3)
- **+6dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P6)
- **+9dBm** (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P9)
- +12dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P12)
- +15dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P15)
- +18dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P18)
- +21dBm (CONFIG_BT_CTRL_DFT_TX_POWER_LEVEL_P21)

CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP

BLE adv report flow control supported

Found in: Component config > Bluetooth > Controller Options

The function is mainly used to enable flow control for advertising reports. When it is enabled, advertising reports will be discarded by the controller if the number of unprocessed advertising reports exceeds the size of BLE adv report flow control.

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_NUM

BLE adv report flow control number

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP

The number of unprocessed advertising report that bluetooth host can save.If you set *BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_NUM* to a small value, this may cause adv packets lost. If you set *BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_NUM* to a large value, bluetooth host may cache a lot of adv packets and this may cause system memory run out. For example, if you set it to 50, the maximum memory consumed by host is 35 * 50 bytes. Please set *BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_NUM* according to your system free memory and handle adv packets as fast as possible, otherwise it will cause adv packets lost.

Range:

- from 50 to 1000 if *CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP* && *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 100 if *CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_BLE_ADV_REPORT_DISCARD_THRSHOLD

BLE adv lost event threshold value

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP

When adv report flow control is enabled, The ADV lost event will be generated when the number of ADV packets lost in the controller reaches this threshold. It is better to set a larger value. If you set *BT_CTRL_BLE_ADV_REPORT_DISCARD_THRSHOLD* to a small value or printf every adv lost event, it may cause adv packets lost more.

Range:

- from 1 to 1000 if *CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP* && *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 20 if *CONFIG_BT_CTRL_BLE_ADV_REPORT_FLOW_CTRL_SUPP* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_BLE_SCAN_DUPL

BLE Scan Duplicate Options

Found in: Component config > Bluetooth > Controller Options

This select enables parameters setting of BLE scan duplicate.

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_SCAN_DUPL_TYPE

Scan Duplicate Type

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_SCAN_DUPL

Scan duplicate have three ways. one is "Scan Duplicate By Device Address", This way is to use advertiser address filtering. The adv packet of the same address is only allowed to be reported once. Another way is "Scan Duplicate By Device Address And Advertising Data". This way is to use advertising data and device address filtering. All different adv packets with the same address are allowed to be reported. The last way is "Scan Duplicate By Advertising Data". This way is to use advertising data filtering. All same advertising data only allow to be reported once even though they are from different devices.

Available options:

- Scan Duplicate By Device Address (CONFIG_BT_CTRL_SCAN_DUPL_TYPE_DEVICE)
  Advertising packets with the same address, address type, and advertising type are reported once.
- Scan Duplicate By Advertising Data (CONFIG_BT_CTRL_SCAN_DUPL_TYPE_DATA)
  Advertising packets with identical advertising data, address type, and advertising type are reported only once, even if they originate from different devices.
- Scan Duplicate By Device Address And Advertising Data (CONFIG_BT_CTRL_SCAN_DUPL_TYPE_DATA_DEVICE)
  Advertising packets with the same address, advertising data, address type, and advertising type are reported only once.

CONFIG_BT_CTRL_SCAN_DUPL_CACHE_SIZE

Maximum number of devices in scan duplicate filter

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_SCAN_DUPL

Maximum number of devices which can be recorded in scan duplicate filter. When the maximum amount of device in the filter is reached, the cache will be refreshed.

Range:

- from 10 to 1000 if *CONFIG_BT_CTRL_BLE_SCAN_DUPL* && *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 100 if *CONFIG_BT_CTRL_BLE_SCAN_DUPL* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_DUPL_SCAN_CACHE_REFRESH_PERIOD

Duplicate scan list refresh period (seconds)

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_SCAN_DUPL

If the period value is non-zero, the controller will periodically clear the device information stored in the scan duuplicate filter. If it is 0, the scan duuplicate filter will not be cleared until the scanning is disabled. Duplicate advertisements for this period should not be sent to the Host in advertising report events. There are two scenarios where the ADV packet will be repeatedly reported: 1. The duplicate scan cache is full, the controller will delete the oldest device information and add new device information. 2. When the refresh period is up, the controller will clear all device information and start filtering again.

Range:

- from 0 to 1000 if *CONFIG_BT_CTRL_BLE_SCAN_DUPL* && *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 0 if *CONFIG_BT_CTRL_BLE_SCAN_DUPL* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_BLE_MESH_SCAN_DUPL_EN

Special duplicate scan mechanism for BLE Mesh scan

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_SCAN_DUPL

This enables the BLE scan duplicate for special BLE Mesh scan.

Default value:

- No (disabled) if *CONFIG_BT_CTRL_BLE_SCAN_DUPL* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_MESH_DUPL_SCAN_CACHE_SIZE

Maximum number of Mesh adv packets in scan duplicate filter

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_BLE_SCAN_DUPL > CONFIG_BT_CTRL_BLE_MESH_SCAN_DUPL_EN

Maximum number of adv packets which can be recorded in duplicate scan cache for BLE Mesh. When the maximum amount of device in the filter is reached, the cache will be refreshed.

Range:

- from 10 to 1000 if *CONFIG_BT_CTRL_BLE_MESH_SCAN_DUPL_EN* && *CONFIG_BT_CONTROLLER_ENABLED*

Default value:

- 100 if *CONFIG_BT_CTRL_BLE_MESH_SCAN_DUPL_EN* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_COEX_PHY_CODED_TX_RX_TLIM

Coexistence: limit on MAX Tx/Rx time for coded-PHY connection

Found in: Component config > Bluetooth > Controller Options

When using PHY-Coded in BLE connection, limitation on max tx/rx time can be applied to better avoid dramatic performance deterioration of Wi-Fi.

Available options:

- Force Enable (CONFIG_BT_CTRL_COEX_PHY_CODED_TX_RX_TLIM_EN)
  Always enable the limitation on max tx/rx time for Coded-PHY connection
- Force Disable (CONFIG_BT_CTRL_COEX_PHY_CODED_TX_RX_TLIM_DIS)
  Disable the limitation on max tx/rx time for Coded-PHY connection

MODEM SLEEP Options

Contains:

- *CONFIG_BT_CTRL_MODEM_SLEEP*
- *CONFIG_BT_CTRL_MAIN_XTAL_PU_DURING_LIGHT_SLEEP*

CONFIG_BT_CTRL_MODEM_SLEEP

Bluetooth modem sleep

Found in: Component config > Bluetooth > Controller Options > MODEM SLEEP Options

Enable/disable bluetooth controller low power mode. Modem sleep is not supported to be used with UART HCI.

CONFIG_BT_CTRL_MODEM_SLEEP_MODE_1

Bluetooth Modem sleep Mode 1

Found in: Component config > Bluetooth > Controller Options > MODEM SLEEP Options > CONFIG_BT_CTRL_MODEM_SLEEP

Mode 1 is the currently supported sleep mode. In this mode, bluetooth controller sleeps between and BLE events. A low power clock is used to maintain bluetooth reference clock.

Default value:

- Yes (enabled) if *CONFIG_BT_CTRL_MODEM_SLEEP* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_LOW_POWER_CLOCK

Bluetooth low power clock

Found in: Component config > Bluetooth > Controller Options > MODEM SLEEP Options > CONFIG_BT_CTRL_MODEM_SLEEP > CONFIG_BT_CTRL_MODEM_SLEEP_MODE_1

Select the low power clock source for bluetooth controller

Available options:

- Main crystal (CONFIG_BT_CTRL_LPCLK_SEL_MAIN_XTAL)
  Main crystal can be used as low power clock for bluetooth modem sleep. If this option is selected, bluetooth modem sleep can work under Dynamic Frequency Scaling(DFS) enabled, and bluetooth can work under light sleep enabled. Main crystal has a relatively better performance than other bluetooth low power clock sources.
- External 32kHz crystal (CONFIG_BT_CTRL_LPCLK_SEL_EXT_32K_XTAL)
  External 32kHz crystal has a nominal frequency of 32.768kHz and provides good frequency stability. If used as Bluetooth low power clock, External 32kHz can support Bluetooth modem sleep to be used with both DFS and light sleep.
- Internal 150kHz RC oscillator (CONFIG_BT_CTRL_LPCLK_SEL_RTC_SLOW)
  Internal 150kHz RC oscillator. The accuracy of this clock is a lot larger than 500ppm which is required in Bluetooth communication, so don't select this option in scenarios such as BLE connection state.

CONFIG_BT_CTRL_MAIN_XTAL_PU_DURING_LIGHT_SLEEP

power up main XTAL during light sleep

Found in: Component config > Bluetooth > Controller Options > MODEM SLEEP Options

If this option is selected, the main crystal will power up during light sleep when the low power clock selects an external 32kHz crystal but the external 32kHz crystal does not exist or the low power clock selects the main crystal.

Default value:

- No (disabled) if (*CONFIG_BT_CTRL_LPCLK_SEL_MAIN_XTAL* || *CONFIG_BT_CTRL_LPCLK_SEL_EXT_32K_XTAL*) && *CONFIG_FREERTOS_USE_TICKLESS_IDLE* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_AGC_RECORRECT_EN

Enable HW AGC recorrect

Found in: Component config > Bluetooth > Controller Options

Enable uncoded phy AGC recorrect

Default value:

- No (disabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_CODED_AGC_RECORRECT_EN

Enable coded phy AGC recorrect

Found in: Component config > Bluetooth > Controller Options > CONFIG_BT_CTRL_AGC_RECORRECT_EN

Enable coded phy AGC recorrect

Default value:

- No (disabled) if *CONFIG_BT_CTRL_AGC_RECORRECT_EN* && *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_SCAN_BACKOFF_UPPERLIMITMAX

Disable active scan backoff

Found in: Component config > Bluetooth > Controller Options

Disable active scan backoff. The bluetooth spec requires that scanners should run a backoff procedure to minimize collision of scan request PDUs from nultiple scanners. If scan backoff is disabled, in active scanning, scan request PDU will be sent every time when HW receives scannable ADV PDU.

Default value:

- No (disabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_BLE_ADV_DATA_LENGTH_ZERO_AUX

Enable aux packet when ext adv data length is zero

Found in: Component config > Bluetooth > Controller Options

When this option is enabled, auxiliary packets will be present in the events of 'Non-Connectable and Non-Scannable' regardless of whether the advertising length is 0. If this option is not enabled, auxiliary packets will only be present when the advertising length is not 0.

Default value:

- No (disabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_CHAN_ASS_EN

Enable channel assessment

Found in: Component config > Bluetooth > Controller Options

If this option is enabled, The Controller will records the communication quality for each channel and then start a timer to check and update the channel map every 4 seconds.

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_CTRL_LE_PING_EN

Enable LE Ping procedure

Found in: Component config > Bluetooth > Controller Options

If this option is disabled, The Controller will not start the LE authenticated payload timer. This option is used for some compatibility problems related to LE ping procedure.

Default value:

- Yes (enabled) if *CONFIG_BT_CONTROLLER_ENABLED*

CONFIG_BT_RELEASE_IRAM

Release Bluetooth text (READ DOCS FIRST)

Found in: Component config > Bluetooth

This option release Bluetooth text section and merge Bluetooth data, bss & text into a large free heap region when esp_bt_mem_release is called, total saving ~21kB or more of IRAM. ESP32-C2 only 3 configurable PMP entries available, rest of them are hard-coded. We cannot split the memory into 3 different regions (IRAM, BLE-IRAM, DRAM). So this option will disable the PMP (ESP_SYSTEM_PMP_IDRAM_SPLIT)

Default value:

- No (disabled) if *CONFIG_BT_ENABLED* && BT_LE_RELEASE_IRAM_SUPPORTED

Common Options Contains:

- *CONFIG_BT_ALARM_MAX_NUM*

CONFIG_BT_ALARM_MAX_NUM

Maximum number of Bluetooth alarms

Found in: Component config > Bluetooth > Common Options

This option decides the maximum number of alarms which could be used by Bluetooth host.

Default value:

- 50

CONFIG_BT_HCI_LOG_DEBUG_EN

Enable Bluetooth HCI debug mode

Found in: Component config > Bluetooth

This option is used to enable bluetooth debug mode, which saves the hci layer data stream.

Default value:

- No (disabled) if *CONFIG_BT_BLUEDROID_ENABLED* || *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_BT_HCI_LOG_DATA_BUFFER_SIZE

Size of the cache used for HCI data in Bluetooth HCI debug mode (N*1024 bytes)

Found in: Component config > Bluetooth > CONFIG_BT_HCI_LOG_DEBUG_EN

This option is to configure the buffer size of the hci data steam cache in hci debug mode. This is a ring buffer, the new data will overwrite the oldest data if the buffer is full.

Range:

- from 1 to 100 if *CONFIG_BT_HCI_LOG_DEBUG_EN*

Default value:

- 5 if *CONFIG_BT_HCI_LOG_DEBUG_EN*

CONFIG_BT_HCI_LOG_ADV_BUFFER_SIZE

Size of the cache used for adv report in Bluetooth HCI debug mode (N*1024 bytes)

Found in: Component config > Bluetooth > CONFIG_BT_HCI_LOG_DEBUG_EN

This option is to configure the buffer size of the hci adv report cache in hci debug mode. This is a ring buffer, the new data will overwrite the oldest data if the buffer is full.

Range:

- from 1 to 100 if *CONFIG_BT_HCI_LOG_DEBUG_EN*

Default value:

- 8 if *CONFIG_BT_HCI_LOG_DEBUG_EN*

CONFIG_BLE_MESH

ESP BLE Mesh Support

Found in: Component config

This option enables ESP BLE Mesh support. The specific features that are available may depend on other features that have been enabled in the stack, such as Bluetooth Support, Bluedroid Support & GATT support.

Contains:

- *BLE Mesh and BLE coexistence support*
- *CONFIG_BLE_MESH_GATT_PROXY_CLIENT*
- *CONFIG_BLE_MESH_GATT_PROXY_SERVER*
- *BLE Mesh NET BUF DEBUG LOG LEVEL*
- *CONFIG_BLE_MESH_PROV*
- *CONFIG_BLE_MESH_PROXY*
- *BLE Mesh specific test option*
- *BLE Mesh STACK DEBUG LOG LEVEL*
- *CONFIG_BLE_MESH_NO_LOG*
- *CONFIG_BLE_MESH_IVU_DIVIDER*
- *CONFIG_BLE_MESH_FAST_PROV*
- *CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC*
- *CONFIG_BLE_MESH_EXPERIMENTAL*
- *CONFIG_BLE_MESH_CRPL*
- *CONFIG_BLE_MESH_RX_SDU_MAX*
- *CONFIG_BLE_MESH_MODEL_KEY_COUNT*
- *CONFIG_BLE_MESH_APP_KEY_COUNT*
- *CONFIG_BLE_MESH_MODEL_GROUP_COUNT*
- *CONFIG_BLE_MESH_LABEL_COUNT*
- *CONFIG_BLE_MESH_SUBNET_COUNT*
- *CONFIG_BLE_MESH_TX_SEG_MAX*
- *CONFIG_BLE_MESH_RX_SEG_MSG_COUNT*
- *CONFIG_BLE_MESH_TX_SEG_MSG_COUNT*
- *CONFIG_BLE_MESH_MEM_ALLOC_MODE*
- *CONFIG_BLE_MESH_MSG_CACHE_SIZE*
- *CONFIG_BLE_MESH_NOT_RELAY_REPLAY_MSG*
- *CONFIG_BLE_MESH_ADV_BUF_COUNT*
- *CONFIG_BLE_MESH_PB_GATT*
- *CONFIG_BLE_MESH_PB_ADV*
- *CONFIG_BLE_MESH_IVU_RECOVERY_IVI*
- *CONFIG_BLE_MESH_RELAY*
- *CONFIG_BLE_MESH_SAR_ENHANCEMENT*
- *CONFIG_BLE_MESH_SETTINGS*
- *CONFIG_BLE_MESH_ACTIVE_SCAN*
- *CONFIG_BLE_MESH_DEINIT*
- *CONFIG_BLE_MESH_USE_DUPLICATE_SCAN*
- *Support for BLE Mesh Client/Server models*
- *Support for BLE Mesh Foundation models*
- *CONFIG_BLE_MESH_NODE*
- *CONFIG_BLE_MESH_PROVISIONER*
- *CONFIG_BLE_MESH_FRIEND*
- *CONFIG_BLE_MESH_LOW_POWER*
- *CONFIG_BLE_MESH_HCI_5_0*
- *CONFIG_BLE_MESH_RANDOM_ADV_INTERVAL*
- *CONFIG_BLE_MESH_IV_UPDATE_TEST*
- *CONFIG_BLE_MESH_CLIENT_MSG_TIMEOUT*

CONFIG_BLE_MESH_HCI_5_0

Support sending 20ms non-connectable adv packets

Found in: Component config > CONFIG_BLE_MESH

It is a temporary solution and needs further modifications.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RANDOM_ADV_INTERVAL

Support using random adv interval for mesh packets

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow using random advertising interval for mesh packets. And this could help avoid collision of advertising packets.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_USE_DUPLICATE_SCAN

Support Duplicate Scan in BLE Mesh

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow using specific duplicate scan filter in BLE Mesh, and Scan Duplicate Type must be set by choosing the option in the Bluetooth Controller section in menuconfig, which is "Scan Duplicate By Device Address and Advertising Data".

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_ACTIVE_SCAN

Support Active Scan in BLE Mesh

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow using BLE Active Scan for BLE Mesh.

CONFIG_BLE_MESH_MEM_ALLOC_MODE

Memory allocation strategy

Found in: Component config > CONFIG_BLE_MESH

Allocation strategy for BLE Mesh stack, essentially provides ability to allocate all required dynamic allocations from,

- Internal DRAM memory only
- External SPIRAM memory only
- Either internal or external memory based on default malloc() behavior in ESP-IDF
- Internal IRAM memory wherever applicable else internal DRAM

Recommended mode here is always internal (*), since that is most preferred from security perspective. But if application requirement does not allow sufficient free internal memory then alternate mode can be selected.

(*) In case of ESP32-S2/ESP32-S3, hardware allows encryption of external SPIRAM contents provided hardware flash encryption feature is enabled. In that case, using external SPIRAM allocation strategy is also safe choice from security perspective.

Available options:

- Internal DRAM (CONFIG_BLE_MESH_MEM_ALLOC_MODE_INTERNAL)
- External SPIRAM (CONFIG_BLE_MESH_MEM_ALLOC_MODE_EXTERNAL)
- Default alloc mode (CONFIG_BLE_MESH_MEM_ALLOC_MODE_DEFAULT)
  Enable this option to use the default memory allocation strategy when external SPIRAM is enabled. See the SPIRAM options for more details.
- Internal IRAM (CONFIG_BLE_MESH_MEM_ALLOC_MODE_IRAM_8BIT)
  Allows to use IRAM memory region as 8bit accessible region. Every unaligned (8bit or 16bit) access will result in an exception and incur penalty of certain clock cycles per unaligned read/write.

CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC

Enable FreeRTOS static allocation

Found in: Component config > CONFIG_BLE_MESH

Enable this option to use FreeRTOS static allocation APIs for BLE Mesh, which provides the ability to use different dynamic memory (i.e. SPIRAM or IRAM) for FreeRTOS objects. If this option is disabled, the FreeRTOS static allocation APIs will not be used, and internal DRAM will be allocated for FreeRTOS objects.

Default value:

- No (disabled) if ESP32_IRAM_AS_8BIT_ACCESSIBLE_MEMORY && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC_MODE

Memory allocation for FreeRTOS objects

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC

Choose the memory to be used for FreeRTOS objects.

Available options:

- External SPIRAM (CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC_EXTERNAL)
  If enabled, BLE Mesh allocates dynamic memory from external SPIRAM for FreeRTOS objects, i.e. mutex, queue, and task stack. External SPIRAM can only be used for task stack when SPIRAM_ALLOW_STACK_EXTERNAL_MEMORY is enabled. See the SPIRAM options for more details.
- Internal IRAM (CONFIG_BLE_MESH_FREERTOS_STATIC_ALLOC_IRAM_8BIT)
  If enabled, BLE Mesh allocates dynamic memory from internal IRAM for FreeRTOS objects, i.e. mutex, queue. Note: IRAM region cannot be used as task stack.

CONFIG_BLE_MESH_DEINIT

Support de-initialize BLE Mesh stack

Found in: Component config > CONFIG_BLE_MESH

If enabled, users can use the function esp_ble_mesh_deinit() to de-initialize the whole BLE Mesh stack.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

BLE Mesh and BLE coexistence support Contains:

- *CONFIG_BLE_MESH_SUPPORT_BLE_SCAN*
- *CONFIG_BLE_MESH_SUPPORT_BLE_ADV*

CONFIG_BLE_MESH_SUPPORT_BLE_ADV

Support sending normal BLE advertising packets

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh and BLE coexistence support

When selected, users can send normal BLE advertising packets with specific API.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_BLE_ADV_BUF_COUNT

Number of advertising buffers for BLE advertising packets

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh and BLE coexistence support > CONFIG_BLE_MESH_SUPPORT_BLE_ADV

Number of advertising buffers for BLE packets available.

Range:
- from 1 to 255 if *CONFIG_BLE_MESH_SUPPORT_BLE_ADV* && *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH_SUPPORT_BLE_ADV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SUPPORT_BLE_SCAN

Support scanning normal BLE advertising packets

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh and BLE coexistence support

When selected, users can register a callback and receive normal BLE advertising packets in the application layer.

Default value:
- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FAST_PROV

Enable BLE Mesh Fast Provisioning

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow BLE Mesh fast provisioning solution to be used. When there are multiple unprovisioned devices around, fast provisioning can greatly reduce the time consumption of the whole provisioning process. When this option is enabled, and after an unprovisioned device is provisioned into a node successfully, it can be changed to a temporary Provisioner.

Default value:
- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_NODE

Support for BLE Mesh Node

Found in: Component config > CONFIG_BLE_MESH

Enable the device to be provisioned into a node. This option should be enabled when an unprovisioned device is going to be provisioned into a node and communicate with other nodes in the BLE Mesh network.

CONFIG_BLE_MESH_PROVISIONER

Support for BLE Mesh Provisioner

Found in: Component config > CONFIG_BLE_MESH

Enable the device to be a Provisioner. The option should be enabled when a device is going to act as a Provisioner and provision unprovisioned devices into the BLE Mesh network.

CONFIG_BLE_MESH_WAIT_FOR_PROV_MAX_DEV_NUM

Maximum number of unprovisioned devices that can be added to device queue

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROVISIONER

This option specifies how many unprovisioned devices can be added to device queue for provisioning. Users can use this option to define the size of the queue in the bottom layer which is used to store unprovisioned device information (e.g. Device UUID, address).

Range:

- from 1 to 100 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:

- 10 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_PROV_NODES

Maximum number of devices that can be provisioned by Provisioner

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROVISIONER

This option specifies how many devices can be provisioned by a Provisioner. This value indicates the maximum number of unprovisioned devices which can be provisioned by a Provisioner. For instance, if the value is 6, it means the Provisioner can provision up to 6 unprovisioned devices. Theoretically a Provisioner without the limitation of its memory can provision up to 32766 unprovisioned devices, here we limit the maximum number to 100 just to limit the memory used by a Provisioner. The bigger the value is, the more memory it will cost by a Provisioner to store the information of nodes.

Range:

- from 1 to 1000 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:

- 10 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PBA_SAME_TIME

Maximum number of PB-ADV running at the same time by Provisioner

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROVISIONER

This option specifies how many devices can be provisioned at the same time using PB-ADV. For examples, if the value is 2, it means a Provisioner can provision two unprovisioned devices with PB-ADV at the same time.

Range:

- from 1 to 10 if *CONFIG_BLE_MESH_PB_ADV* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_PB_ADV* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PBG_SAME_TIME

Maximum number of PB-GATT running at the same time by Provisioner

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROVISIONER

This option specifies how many devices can be provisioned at the same time using PB-GATT. For example, if the value is 2, it means a Provisioner can provision two unprovisioned devices with PB-GATT at the same time.

Range:

- from 1 to 5 if *CONFIG_BLE_MESH_PB_GATT* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:

- 1 if *CONFIG_BLE_MESH_PB_GATT* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROVISIONER_SUBNET_COUNT

Maximum number of mesh subnets that can be created by Provisioner

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_PROVISIONER*

This option specifies how many subnets per network a Provisioner can create. Indeed, this value decides the number of network keys which can be added by a Provisioner.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROVISIONER_APP_KEY_COUNT

Maximum number of application keys that can be owned by Provisioner

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_PROVISIONER*

This option specifies how many application keys the Provisioner can have. Indeed, this value decides the number of the application keys which can be added by a Provisioner.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROVISIONER_RECV_HB

Support receiving Heartbeat messages

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_PROVISIONER*

When this option is enabled, Provisioner can call specific functions to enable or disable receiving Heartbeat messages and notify them to the application layer.

Default value:
- No (disabled) if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROVISIONER_RECV_HB_FILTER_SIZE

Maximum number of filter entries for receiving Heartbeat messages

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_PROVISIONER* > *CONFIG_BLE_MESH_PROVISIONER_RECV_HB*

This option specifies how many heartbeat filter entries Provisioner supports. The heartbeat filter (acceptlist or rejectlist) entries are used to store a list of SRC and DST which can be used to decide if a heartbeat message will be processed and notified to the application layer by Provisioner. Note: The filter is an empty rejectlist by default.

Range:
- from 1 to 1000 if *CONFIG_BLE_MESH_PROVISIONER_RECV_HB* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH_PROVISIONER_RECV_HB* && *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROV

BLE Mesh Provisioning support

Found in: Component config > CONFIG_BLE_MESH

Enable this option to support BLE Mesh Provisioning functionality. For BLE Mesh, this option should be always enabled.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROV_EPA

BLE Mesh enhanced provisioning authentication

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROV

Enable this option to support BLE Mesh enhanced provisioning authentication functionality. This option can increase the security level of provisioning. It is recommended to enable this option.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH_PROV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_CERT_BASED_PROV

Support Certificate-based provisioning

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROV

Enable this option to support BLE Mesh Certificate-Based Provisioning.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_PROV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RECORD_FRAG_MAX_SIZE

Maximum size of the provisioning record fragment that Provisioner can receive

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PROV > CONFIG_BLE_MESH_CERT_BASED_PROV

This option sets the maximum size of the provisioning record fragment that the Provisioner can receive. The range depends on provisioning bearer.

Range:

- from 1 to 57 if *CONFIG_BLE_MESH_CERT_BASED_PROV* && *CONFIG_BLE_MESH*

Default value:

- 56 if *CONFIG_BLE_MESH_CERT_BASED_PROV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PB_ADV

Provisioning support using the advertising bearer (PB-ADV)

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow the device to be provisioned over the advertising bearer. This option should be enabled if PB-ADV is going to be used during provisioning procedure.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_UNPROVISIONED_BEACON_INTERVAL

Interval between two consecutive Unprovisioned Device Beacon

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_PB_ADV

This option specifies the interval of sending two consecutive unprovisioned device beacon, users can use this option to change the frequency of sending unprovisioned device beacon. For example, if the value is 5, it means the unprovisioned device beacon will send every 5 seconds. When the option of BLE_MESH_FAST_PROV is selected, the value is better to be 3 seconds, or less.

Range:

- from 1 to 100 if *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH_PB_ADV* && *CONFIG_BLE_MESH*

Default value:

- 5 if *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH_PB_ADV* && *CONFIG_BLE_MESH*
- 3 if *CONFIG_BLE_MESH_FAST_PROV* && *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH_PB_ADV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PB_GATT

Provisioning support using GATT (PB-GATT)

Found in: Component config > CONFIG_BLE_MESH

Enable this option to allow the device to be provisioned over GATT. This option should be enabled if PB-GATT is going to be used during provisioning procedure.

Virtual option enabled whenever any Proxy protocol is needed

CONFIG_BLE_MESH_PROXY

BLE Mesh Proxy protocol support

Found in: Component config > CONFIG_BLE_MESH

Enable this option to support BLE Mesh Proxy protocol used by PB-GATT and other proxy pdu transmission.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_GATT_PROXY_SERVER

BLE Mesh GATT Proxy Server

Found in: Component config > CONFIG_BLE_MESH

This option enables support for Mesh GATT Proxy Service, i.e. the ability to act as a proxy between a Mesh GATT Client and a Mesh network. This option should be enabled if a node is going to be a Proxy Server.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_NODE_ID_TIMEOUT

Node Identity advertising timeout

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_SERVER

This option determines for how long the local node advertises using Node Identity. The given value is in seconds. The specification limits this to 60 seconds and lists it as the recommended value as well. So leaving the default value is the safest option. When an unprovisioned device is provisioned successfully

and becomes a node, it will start to advertise using Node Identity during the time set by this option. And after that, Network ID will be advertised.

Range:
- from 1 to 60 if *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH*

Default value:
- 60 if *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROXY_FILTER_SIZE

Maximum number of filter entries per Proxy Client

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_SERVER

This option specifies how many Proxy Filter entries the local node supports. The entries of Proxy filter (whitelist or blacklist) are used to store a list of addresses which can be used to decide which messages will be forwarded to the Proxy Client by the Proxy Server.

Range:
- from 1 to 32767 if *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH*

Default value:
- 4 if *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROXY_PRIVACY

Support Proxy Privacy

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_SERVER

The Proxy Privacy parameter controls the privacy of the Proxy Server over the connection. The value of the Proxy Privacy parameter is controlled by the type of proxy connection, which is dependent on the bearer used by the proxy connection.

Default value:
- Yes (enabled) if *CONFIG_BLE_MESH_PRB_SRV* && *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROXY_SOLIC_PDU_RX

Support receiving Proxy Solicitation PDU

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_SERVER

Enable this option to support receiving Proxy Solicitation PDU.

CONFIG_BLE_MESH_PROXY_SOLIC_RX_CRPL

Maximum capacity of solicitation replay protection list

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_SERVER > CONFIG_BLE_MESH_PROXY_SOLIC_PDU_RX

This option specifies the maximum capacity of the solicitation replay protection list. The solicitation replay protection list is used to reject Solicitation PDUs that were already processed by a node, which will store the solicitation src and solicitation sequence number of the received Solicitation PDU message.

Range:
- from 1 to 255 if *CONFIG_BLE_MESH_PROXY_SOLIC_PDU_RX* && *CONFIG_BLE_MESH*

Default value:
- 2 if *CONFIG_BLE_MESH_PROXY_SOLIC_PDU_RX* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_GATT_PROXY_CLIENT

BLE Mesh GATT Proxy Client

Found in: Component config > CONFIG_BLE_MESH

This option enables support for Mesh GATT Proxy Client. The Proxy Client can use the GATT bearer to send mesh messages to a node that supports the advertising bearer.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PROXY_SOLIC_PDU_TX

Support sending Proxy Solicitation PDU

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_CLIENT

Enable this option to support sending Proxy Solicitation PDU.

CONFIG_BLE_MESH_PROXY_SOLIC_TX_SRC_COUNT

Maximum number of SSRC that can be used by Proxy Client

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_GATT_PROXY_CLIENT > CONFIG_BLE_MESH_PROXY_SOLIC_PDU_TX

This option specifies the maximum number of Solicitation Source (SSRC) that can be used by Proxy Client for sending a Solicitation PDU. A Proxy Client may use the primary address or any of the secondary addresses as the SSRC for a Solicitation PDU. So for a Proxy Client, it's better to choose the value based on its own element count.

Range:

- from 1 to 16 if *CONFIG_BLE_MESH_PROXY_SOLIC_PDU_TX* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_PROXY_SOLIC_PDU_TX* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SETTINGS

Store BLE Mesh configuration persistently

Found in: Component config > CONFIG_BLE_MESH

When selected, the BLE Mesh stack will take care of storing/restoring the BLE Mesh configuration persistently in flash. If the device is a BLE Mesh node, when this option is enabled, the configuration of the device will be stored persistently, including unicast address, NetKey, AppKey, etc. And if the device is a BLE Mesh Provisioner, the information of the device will be stored persistently, including the information of provisioned nodes, NetKey, AppKey, etc.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_STORE_TIMEOUT

Delay (in seconds) before storing anything persistently

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_SETTINGS

This value defines in seconds how soon any pending changes are actually written into persistent storage (flash) after a change occurs. The option allows nodes to delay a certain period of time to save proper information to flash. The default value is 0, which means information will be stored immediately once there are updates.

Range:

- from 0 to 1000000 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

Default value:

- 0 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SEQ_STORE_RATE

How often the sequence number gets updated in storage

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_SETTINGS*

This value defines how often the local sequence number gets updated in persistent storage (i.e. flash). e.g. a value of 100 means that the sequence number will be stored to flash on every 100th increment. If the node sends messages very frequently a higher value makes more sense, whereas if the node sends infrequently a value as low as 0 (update storage for every increment) can make sense. When the stack gets initialized it will add sequence number to the last stored one, so that it starts off with a value that's guaranteed to be larger than the last one used before power off.

Range:

- from 0 to 1000000 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

Default value:

- 0 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RPL_STORE_TIMEOUT

Minimum frequency that the RPL gets updated in storage

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_SETTINGS*

This value defines in seconds how soon the RPL (Replay Protection List) gets written to persistent storage after a change occurs. If the node receives messages frequently, then a large value is recommended. If the node receives messages rarely, then the value can be as low as 0 (which means the RPL is written into the storage immediately). Note that if the node operates in a security-sensitive case, and there is a risk of sudden power-off, then a value of 0 is strongly recommended. Otherwise, a power loss before RPL being written into the storage may introduce message replay attacks and system security will be in a vulnerable state.

Range:

- from 0 to 1000000 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

Default value:

- 0 if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SETTINGS_BACKWARD_COMPATIBILITY

A specific option for settings backward compatibility

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_SETTINGS*

This option is created to solve the issue of failure in recovering node information after mesh stack updates. In the old version mesh stack, there is no key of "mesh/role" in nvs. In the new version mesh stack, key of "mesh/role" is added in nvs, recovering node information needs to check "mesh/role" key in nvs and implements selective recovery of mesh node information. Therefore, there may be failure in recovering node information during node restarting after OTA.

The new version mesh stack adds the option of "mesh/role" because we have added the support of storing Provisioner information, while the old version only supports storing node information.

If users are updating their nodes from old version to new version, we recommend enabling this option, so that system could set the flag in advance before recovering node information and make sure the node information recovering could work as expected.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SPECIFIC_PARTITION

Use a specific NVS partition for BLE Mesh

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_SETTINGS

When selected, the mesh stack will use a specified NVS partition instead of default NVS partition. Note that the specified partition must be registered with NVS using nvs_flash_init_partition() API, and the partition must exists in the csv file. When Provisioner needs to store a large amount of nodes' information in the flash (e.g. more than 20), this option is recommended to be enabled.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PARTITION_NAME

Name of the NVS partition for BLE Mesh

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_SETTINGS > CONFIG_BLE_MESH_SPECIFIC_PARTITION

This value defines the name of the specified NVS partition used by the mesh stack.

Default value:

- "ble_mesh" if *CONFIG_BLE_MESH_SPECIFIC_PARTITION* && *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_USE_MULTIPLE_NAMESPACE

Support using multiple NVS namespaces by Provisioner

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_SETTINGS

When selected, Provisioner can use different NVS namespaces to store different instances of mesh information. For example, if in the first room, Provisioner uses NetKey A, AppKey A and provisions three devices, these information will be treated as mesh information instance A. When the Provisioner moves to the second room, it uses NetKey B, AppKey B and provisions two devices, then the information will be treated as mesh information instance B. Here instance A and instance B will be stored in different namespaces. With this option enabled, Provisioner needs to use specific functions to open the corresponding NVS namespace, restore the mesh information, release the mesh information or erase the mesh information.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_PROVISIONER* && *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_NVS_NAMESPACE

Maximum number of NVS namespaces

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_SETTINGS > CONFIG_BLE_MESH_USE_MULTIPLE_NAMESPACE

This option specifies the maximum NVS namespaces supported by Provisioner.

Range:

- from 1 to 255 if *CONFIG_BLE_MESH_USE_MULTIPLE_NAMESPACE* && *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_USE_MULTIPLE_NAMESPACE* && *CONFIG_BLE_MESH_SETTINGS* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SUBNET_COUNT

Maximum number of mesh subnets per network

Found in: Component config > CONFIG_BLE_MESH

This option specifies how many subnets a Mesh network can have at the same time. Indeed, this value decides the number of the network keys which can be owned by a node.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_APP_KEY_COUNT

Maximum number of application keys per network

Found in: Component config > CONFIG_BLE_MESH

This option specifies how many application keys the device can store per network. Indeed, this value decides the number of the application keys which can be owned by a node.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MODEL_KEY_COUNT

Maximum number of application keys per model

Found in: Component config > CONFIG_BLE_MESH

This option specifies the maximum number of application keys to which each model can be bound.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MODEL_GROUP_COUNT

Maximum number of group address subscriptions per model

Found in: Component config > CONFIG_BLE_MESH

This option specifies the maximum number of addresses to which each model can be subscribed.

Range:
- from 1 to 4096 if *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LABEL_COUNT

Maximum number of Label UUIDs used for Virtual Addresses

Found in: Component config > CONFIG_BLE_MESH

This option specifies how many Label UUIDs can be stored. Indeed, this value decides the number of the Virtual Addresses can be supported by a node.

Range:
- from 0 to 4096 if *CONFIG_BLE_MESH*

Default value:

- 3 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_CRPL

Maximum capacity of the replay protection list

Found in: Component config > CONFIG_BLE_MESH

This option specifies the maximum capacity of the replay protection list. It is similar to Network message cache size, but has a different purpose. The replay protection list is used to prevent a node from replay attack, which will store the source address and sequence number of the received mesh messages. For Provisioner, the replay protection list size should not be smaller than the maximum number of nodes whose information can be stored. And the element number of each node should also be taken into consideration. For example, if Provisioner can provision up to 20 nodes and each node contains two elements, then the replay protection list size of Provisioner should be at least 40.

Range:

- from 2 to 65535 if *CONFIG_BLE_MESH*

Default value:

- 10 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_NOT_RELAY_REPLAY_MSG

Not relay replayed messages in a mesh network

Found in: Component config > CONFIG_BLE_MESH

There may be many expired messages in a complex mesh network that would be considered replayed messages. Enable this option will refuse to relay such messages, which could help to reduce invalid packets in the mesh network. However, it should be noted that enabling this option may result in packet loss in certain environments. Therefore, users need to decide whether to enable this option according to the actual usage situation.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_EXPERIMENTAL* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MSG_CACHE_SIZE

Network message cache size

Found in: Component config > CONFIG_BLE_MESH

Number of messages that are cached for the network. This helps prevent unnecessary decryption operations and unnecessary relays. This option is similar to Replay protection list, but has a different purpose. A node is not required to cache the entire Network PDU and may cache only part of it for tracking, such as values for SRC/SEQ or others.

Range:

- from 2 to 65535 if *CONFIG_BLE_MESH*

Default value:

- 10 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_ADV_BUF_COUNT

Number of advertising buffers

Found in: Component config > CONFIG_BLE_MESH

Number of advertising buffers available. The transport layer reserves ADV_BUF_COUNT - 3 buffers for outgoing segments. The maximum outgoing SDU size is 12 times this value (out of which 4 or 8 bytes are used for the Transport Layer MIC). For example, 5 segments means the maximum SDU size

is 60 bytes, which leaves 56 bytes for application layer data using a 4-byte MIC, or 52 bytes using an 8-byte MIC.

Range:
- from 6 to 256 if *CONFIG_BLE_MESH*

Default value:
- 60 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_IVU_DIVIDER

Divider for IV Update state refresh timer

Found in: Component config > CONFIG_BLE_MESH

When the IV Update state enters Normal operation or IV Update in Progress, we need to keep track of how many hours has passed in the state, since the specification requires us to remain in the state at least for 96 hours (Update in Progress has an additional upper limit of 144 hours).

In order to fulfill the above requirement, even if the node might be powered off once in a while, we need to store persistently how many hours the node has been in the state. This doesn't necessarily need to happen every hour (thanks to the flexible duration range). The exact cadence will depend a lot on the ways that the node will be used and what kind of power source it has.

Since there is no single optimal answer, this configuration option allows specifying a divider, i.e. how many intervals the 96 hour minimum gets split into. After each interval the duration that the node has been in the current state gets stored to flash. E.g. the default value of 4 means that the state is saved every 24 hours (96 / 4).

Range:
- from 2 to 96 if *CONFIG_BLE_MESH*

Default value:
- 4 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_IVU_RECOVERY_IVI

Recovery the IV index when the latest whole IV update procedure is missed

Found in: Component config > CONFIG_BLE_MESH

According to Section 3.10.5 of Mesh Specification v1.0.1. If a node in Normal Operation receives a Secure Network beacon with an IV index equal to the last known IV index+1 and the IV Update Flag set to 0, the node may update its IV without going to the IV Update in Progress state, or it may initiate an IV Index Recovery procedure (Section 3.10.6), or it may ignore the Secure Network beacon. The node makes the choice depending on the time since last IV update and the likelihood that the node has missed the Secure Network beacons with the IV update Flag. When the above situation is encountered, this option can be used to decide whether to perform the IV index recovery procedure.

Default value:
- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SAR_ENHANCEMENT

Segmentation and reassembly enhancement

Found in: Component config > CONFIG_BLE_MESH

Enable this option to use the enhanced segmentation and reassembly mechanism introduced in Bluetooth Mesh Protocol 1.1.

Default value:
- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_TX_SEG_MSG_COUNT

Maximum number of simultaneous outgoing segmented messages

Found in: Component config > CONFIG_BLE_MESH

Maximum number of simultaneous outgoing multi-segment and/or reliable messages. The default value is 1, which means the device can only send one segmented message at a time. And if another segmented message is going to be sent, it should wait for the completion of the previous one. If users are going to send multiple segmented messages at the same time, this value should be configured properly.

Range:
- from 1 to if *CONFIG_BLE_MESH*

Default value:
- 1 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RX_SEG_MSG_COUNT

Maximum number of simultaneous incoming segmented messages

Found in: Component config > CONFIG_BLE_MESH

Maximum number of simultaneous incoming multi-segment and/or reliable messages. The default value is 1, which means the device can only receive one segmented message at a time. And if another segmented message is going to be received, it should wait for the completion of the previous one. If users are going to receive multiple segmented messages at the same time, this value should be configured properly.

Range:
- from 1 to 255 if *CONFIG_BLE_MESH*

Default value:
- 1 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RX_SDU_MAX

Maximum incoming Upper Transport Access PDU length

Found in: Component config > CONFIG_BLE_MESH

Maximum incoming Upper Transport Access PDU length. Leave this to the default value, unless you really need to optimize memory usage.

Range:
- from 36 to 384 if *CONFIG_BLE_MESH*

Default value:
- 384 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_TX_SEG_MAX

Maximum number of segments in outgoing messages

Found in: Component config > CONFIG_BLE_MESH

Maximum number of segments supported for outgoing messages. This value should typically be fine-tuned based on what models the local node supports, i.e. what's the largest message payload that the node needs to be able to send. This value affects memory and call stack consumption, which is why the default is lower than the maximum that the specification would allow (32 segments).

The maximum outgoing SDU size is 12 times this number (out of which 4 or 8 bytes is used for the Transport Layer MIC). For example, 5 segments means the maximum SDU size is 60 bytes, which leaves 56 bytes for application layer data using a 4-byte MIC and 52 bytes using an 8-byte MIC.

Be sure to specify a sufficient number of advertising buffers when setting this option to a higher value. There must be at least three more advertising buffers (BLE_MESH_ADV_BUF_COUNT) as there are outgoing segments.

Range:
- from 2 to 32 if *CONFIG_BLE_MESH*

Default value:
- 32 if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RELAY

Relay support

Found in: Component config > CONFIG_BLE_MESH

Support for acting as a Mesh Relay Node. Enabling this option will allow a node to support the Relay feature, and the Relay feature can still be enabled or disabled by proper configuration messages. Disabling this option will let a node not support the Relay feature.

Default value:
- Yes (enabled) if *CONFIG_BLE_MESH_NODE* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RELAY_ADV_BUF

Use separate advertising buffers for relay packets

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_RELAY

When selected, self-send packets will be put in a high-priority queue and relay packets will be put in a low-priority queue.

Default value:
- No (disabled) if *CONFIG_BLE_MESH_RELAY* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RELAY_ADV_BUF_COUNT

Number of advertising buffers for relay packets

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_RELAY > CONFIG_BLE_MESH_RELAY_ADV_BUF

Number of advertising buffers for relay packets available.

Range:
- from 6 to 256 if *CONFIG_BLE_MESH_RELAY_ADV_BUF* && *CONFIG_BLE_MESH_RELAY* && *CONFIG_BLE_MESH*

Default value:
- 60 if *CONFIG_BLE_MESH_RELAY_ADV_BUF* && *CONFIG_BLE_MESH_RELAY* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LOW_POWER

Support for Low Power features

Found in: Component config > CONFIG_BLE_MESH

Enable this option to operate as a Low Power Node. If low power consumption is required by a node, this option should be enabled. And once the node enters the mesh network, it will try to find a Friend node and establish a friendship.

CONFIG_BLE_MESH_LPN_ESTABLISHMENT

Perform Friendship establishment using low power

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_LOW_POWER

Perform the Friendship establishment using low power with the help of a reduced scan duty cycle. The downside of this is that the node may miss out on messages intended for it until it has successfully set up

Friendship with a Friend node. When this option is enabled, the node will stop scanning for a period of time after a Friend Request or Friend Poll is sent, so as to reduce more power consumption.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_AUTO

Automatically start looking for Friend nodes once provisioned

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

Once provisioned, automatically enable LPN functionality and start looking for Friend nodes. If this option is disabled LPN mode needs to be manually enabled by calling bt_mesh_lpn_set(true). When an unprovisioned device is provisioned successfully and becomes a node, enabling this option will trigger the node starts to send Friend Request at a certain period until it finds a proper Friend node.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_AUTO_TIMEOUT

Time from last received message before going to LPN mode

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER* > *CONFIG_BLE_MESH_LPN_AUTO*

Time in seconds from the last received message, that the node waits out before starting to look for Friend nodes.

Range:

- from 0 to 3600 if *CONFIG_BLE_MESH_LPN_AUTO* && *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:

- 15 if *CONFIG_BLE_MESH_LPN_AUTO* && *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_RETRY_TIMEOUT

Retry timeout for Friend requests

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

Time in seconds between Friend Requests, if a previous Friend Request did not yield any acceptable Friend Offers.

Range:

- from 1 to 3600 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:

- 6 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_RSSI_FACTOR

RSSIFactor, used in Friend Offer Delay calculation

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

The contribution of the RSSI, measured by the Friend node, used in Friend Offer Delay calculations. 0 = 1, 1 = 1.5, 2 = 2, 3 = 2.5. RSSIFactor, one of the parameters carried by Friend Request sent by Low Power node, which is used to calculate the Friend Offer Delay.

Range:

- from 0 to 3 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:

- 0 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_RECV_WIN_FACTOR

ReceiveWindowFactor, used in Friend Offer Delay calculation

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

The contribution of the supported Receive Window used in Friend Offer Delay calculations. 0 = 1, 1 = 1.5, 2 = 2, 3 = 2.5. ReceiveWindowFactor, one of the parameters carried by Friend Request sent by Low Power node, which is used to calculate the Friend Offer Delay.

Range:
- from 0 to 3 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 0 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_MIN_QUEUE_SIZE

Minimum size of the acceptable friend queue (MinQueueSizeLog)

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

The MinQueueSizeLog field is defined as log_2(N), where N is the minimum number of maximum size Lower Transport PDUs that the Friend node can store in its Friend Queue. As an example, MinQueueSizeLog value 1 gives N = 2, and value 7 gives N = 128.

Range:
- from 1 to 7 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 1 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_RECV_DELAY

Receive delay requested by the local node

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

The ReceiveDelay is the time between the Low Power node sending a request and listening for a response. This delay allows the Friend node time to prepare the response. The value is in units of milliseconds.

Range:
- from 10 to 255 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 100 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_POLL_TIMEOUT

The value of the PollTimeout timer

Found in: Component config > *CONFIG_BLE_MESH* > *CONFIG_BLE_MESH_LOW_POWER*

PollTimeout timer is used to measure time between two consecutive requests sent by a Low Power node. If no requests are received the Friend node before the PollTimeout timer expires, then the friendship is considered terminated. The value is in units of 100 milliseconds, so e.g. a value of 300 means 30 seconds. The smaller the value, the faster the Low Power node tries to get messages from corresponding Friend node and vice versa.

Range:
- from 10 to 244735 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 300 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_INIT_POLL_TIMEOUT

The starting value of the PollTimeout timer

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_LOW_POWER

The initial value of the PollTimeout timer when Friendship is to be established for the first time. After this, the timeout gradually grows toward the actual PollTimeout, doubling in value for each iteration. The value is in units of 100 milliseconds, so e.g. a value of 300 means 30 seconds.

Range:
- from 10 to if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_SCAN_LATENCY

Latency for enabling scanning

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_LOW_POWER

Latency (in milliseconds) is the time it takes to enable scanning. In practice, it means how much time in advance of the Receive Window, the request to enable scanning is made.

Range:
- from 0 to 50 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 10 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_GROUPS

Number of groups the LPN can subscribe to

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_LOW_POWER

Maximum number of groups to which the LPN can subscribe.

Range:
- from 0 to 16384 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

Default value:
- 8 if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LPN_SUB_ALL_NODES_ADDR

Automatically subscribe all nodes address

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_LOW_POWER

Automatically subscribe all nodes address when friendship established.

Default value:
- No (disabled) if *CONFIG_BLE_MESH_LOW_POWER* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FRIEND

Support for Friend feature

Found in: Component config > CONFIG_BLE_MESH

Enable this option to be able to act as a Friend Node.

CONFIG_BLE_MESH_FRIEND_RECV_WIN

Friend Receive Window

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FRIEND

Receive Window in milliseconds supported by the Friend node.

Range:
- from 1 to 255 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

Default value:
- 255 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FRIEND_QUEUE_SIZE

Minimum number of buffers supported per Friend Queue

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FRIEND

Minimum number of buffers available to be stored for each local Friend Queue. This option decides the size of each buffer which can be used by a Friend node to store messages for each Low Power node.

Range:
- from 2 to 65536 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

Default value:
- 16 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FRIEND_SUB_LIST_SIZE

Friend Subscription List Size

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FRIEND

Size of the Subscription List that can be supported by a Friend node for a Low Power node. And Low Power node can send Friend Subscription List Add or Friend Subscription List Remove messages to the Friend node to add or remove subscription addresses.

Range:
- from 0 to 1023 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

Default value:
- 3 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FRIEND_LPN_COUNT

Number of supported LPN nodes

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FRIEND

Number of Low Power Nodes with which a Friend can have Friendship simultaneously. A Friend node can have friendship with multiple Low Power nodes at the same time, while a Low Power node can only establish friendship with only one Friend node at the same time.

Range:
- from 1 to 1000 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

Default value:
- 2 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_FRIEND_SEG_RX

Number of incomplete segment lists per LPN

Found in: Component config > CONFIG_BLE_MESH > CONFIG_BLE_MESH_FRIEND

Number of incomplete segment lists tracked for each Friends' LPN. In other words, this determines from how many elements can segmented messages destined for the Friend queue be received simultaneously.

Range:
- from 1 to 1000 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

Default value:
- 1 if *CONFIG_BLE_MESH_FRIEND* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_NO_LOG

Disable BLE Mesh debug logs (minimize bin size)

Found in: Component config > CONFIG_BLE_MESH

Select this to save the BLE Mesh related rodata code size. Enabling this option will disable the output of BLE Mesh debug log.

Default value:
- No (disabled) if *CONFIG_BLE_MESH* && *CONFIG_BLE_MESH*

BLE Mesh STACK DEBUG LOG LEVEL

Contains:

- *CONFIG_BLE_MESH_STACK_TRACE_LEVEL*

CONFIG_BLE_MESH_STACK_TRACE_LEVEL

BLE_MESH_STACK

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh STACK DEBUG LOG LEVEL

Define BLE Mesh trace level for BLE Mesh stack.

Available options:

- NONE (CONFIG_BLE_MESH_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BLE_MESH_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BLE_MESH_TRACE_LEVEL_WARNING)
- INFO (CONFIG_BLE_MESH_TRACE_LEVEL_INFO)
- DEBUG (CONFIG_BLE_MESH_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BLE_MESH_TRACE_LEVEL_VERBOSE)

BLE Mesh NET BUF DEBUG LOG LEVEL

Contains:

- *CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL*

CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL

BLE_MESH_NET_BUF

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh NET BUF DEBUG LOG LEVEL

Define BLE Mesh trace level for BLE Mesh net buffer.

Available options:

- NONE (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_NONE)
- ERROR (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_ERROR)
- WARNING (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_WARNING)
- INFO (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_INFO)
- DEBUG (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_DEBUG)
- VERBOSE (CONFIG_BLE_MESH_NET_BUF_TRACE_LEVEL_VERBOSE)

CONFIG_BLE_MESH_CLIENT_MSG_TIMEOUT

Timeout(ms) for client message response

Found in: Component config > CONFIG_BLE_MESH

Timeout value used by the node to get response of the acknowledged message which is sent by the client model. This value indicates the maximum time that a client model waits for the response of the sent acknowledged messages. If a client model uses 0 as the timeout value when sending acknowledged messages, then the default value will be used which is four seconds.

Range:

- from 100 to 1200000 if *CONFIG_BLE_MESH*

Default value:

- 4000 if *CONFIG_BLE_MESH*

Support for BLE Mesh Foundation models

Contains:

- *CONFIG_BLE_MESH_BRC_CLI*
- *CONFIG_BLE_MESH_BRC_SRV*
- *CONFIG_BLE_MESH_CFG_CLI*
- *CONFIG_BLE_MESH_DF_CLI*
- *CONFIG_BLE_MESH_DF_SRV*
- *CONFIG_BLE_MESH_HEALTH_CLI*
- *CONFIG_BLE_MESH_HEALTH_SRV*
- *CONFIG_BLE_MESH_LCD_CLI*
- *CONFIG_BLE_MESH_LCD_SRV*
- *CONFIG_BLE_MESH_PRB_CLI*
- *CONFIG_BLE_MESH_PRB_SRV*
- *CONFIG_BLE_MESH_ODP_CLI*
- *CONFIG_BLE_MESH_ODP_SRV*
- *CONFIG_BLE_MESH_AGG_CLI*
- *CONFIG_BLE_MESH_AGG_SRV*
- *CONFIG_BLE_MESH_RPR_CLI*
- *CONFIG_BLE_MESH_RPR_SRV*
- *CONFIG_BLE_MESH_SAR_CLI*
- *CONFIG_BLE_MESH_SAR_SRV*
- *CONFIG_BLE_MESH_SRPL_CLI*
- *CONFIG_BLE_MESH_SRPL_SRV*
- *CONFIG_BLE_MESH_COMP_DATA_1*
- *CONFIG_BLE_MESH_COMP_DATA_128*
- *CONFIG_BLE_MESH_MODELS_METADATA_0*

CONFIG_BLE_MESH_CFG_CLI

Configuration Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Configuration Client model.

CONFIG_BLE_MESH_HEALTH_CLI

Health Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Health Client model.

CONFIG_BLE_MESH_HEALTH_SRV

Health Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Health Server model.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_BRC_CLI

Bridge Configuration Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Bridge Configuration Client model.

CONFIG_BLE_MESH_BRC_SRV

Bridge Configuration Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Bridge Configuration Server model.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_BRIDGING_TABLE_ENTRY_COUNT

Maximum number of Bridging Table entries

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_BRC_SRV

Maximum number of Bridging Table entries that the Bridge Configuration Server can support.

Range:

- from 16 to 65535 if *CONFIG_BLE_MESH_BRC_SRV* && *CONFIG_BLE_MESH*

Default value:

- 16 if *CONFIG_BLE_MESH_BRC_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_BRIDGE_CRPL

Maximum capacity of bridge replay protection list

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_BRC_SRV

This option specifies the maximum capacity of the bridge replay protection list. The bridge replay protection list is used to prevent a bridged subnet from replay attack, which will store the source address and sequence number of the received bridge messages.

Range:

- from 1 to 255 if *CONFIG_BLE_MESH_BRC_SRV* && *CONFIG_BLE_MESH*

Default value:

- 5 if *CONFIG_BLE_MESH_BRC_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PRB_CLI

Mesh Private Beacon Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Mesh Private Beacon Client model.

CONFIG_BLE_MESH_PRB_SRV

Mesh Private Beacon Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Mesh Private Beacon Server model.

CONFIG_BLE_MESH_ODP_CLI

On-Demand Private Proxy Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for On-Demand Private Proxy Client model.

CONFIG_BLE_MESH_ODP_SRV

On-Demand Private Proxy Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for On-Demand Private Proxy Server model.

CONFIG_BLE_MESH_SRPL_CLI

Solicitation PDU RPL Configuration Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Solicitation PDU RPL Configuration Client model.

CONFIG_BLE_MESH_SRPL_SRV

Solicitation PDU RPL Configuration Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Solicitation PDU RPL Configuration Server model. Note: This option depends on the functionality of receiving Solicitation PDU. If the device doesn't support receiving Solicitation PDU, then there is no need to enable this server model.

CONFIG_BLE_MESH_AGG_CLI

Opcodes Aggregator Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Opcodes Aggregator Client model.

CONFIG_BLE_MESH_AGG_SRV

Opcodes Aggregator Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Opcodes Aggregator Server model.

CONFIG_BLE_MESH_SAR_CLI

SAR Configuration Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for SAR Configuration Client model.

CONFIG_BLE_MESH_SAR_SRV

SAR Configuration Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for SAR Configuration Server model.

CONFIG_BLE_MESH_COMP_DATA_1

Support Composition Data Page 1

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Composition Data Page 1 contains information about the relationships among models. Each model either can be a root model or can extend other models.

CONFIG_BLE_MESH_COMP_DATA_128

Support Composition Data Page 128

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Composition Data Page 128 is used to indicate the structure of elements, features, and models of a node after the successful execution of the Node Address Refresh procedure or the Node Composition Refresh procedure, or after the execution of the Node Removal procedure followed by the provisioning process. Composition Data Page 128 shall be present if the node supports the Remote Provisioning Server model; otherwise it is optional.

CONFIG_BLE_MESH_MODELS_METADATA_0

Support Models Metadata Page 0

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

The Models Metadata state contains metadata of a node' s models. The Models Metadata state is composed of a number of pages of information. Models Metadata Page 0 shall be present if the node supports the Large Composition Data Server model.

CONFIG_BLE_MESH_MODELS_METADATA_128

Support Models Metadata Page 128

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_MODELS_METADATA_0

The Models Metadata state contains metadata of a node' s models. The Models Metadata state is composed of a number of pages of information. Models Metadata Page 128 contains metadata for the node' s models after the successful execution of the Node Address Refresh procedure or the Node Composition Refresh procedure, or after the execution of the Node Removal procedure followed by the provisioning process. Models Metadata Page 128 shall be present if the node supports the Remote Provisioning Server model and the node supports the Large Composition Data Server model.

CONFIG_BLE_MESH_LCD_CLI

Large Composition Data Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Large Composition Data Client model.

CONFIG_BLE_MESH_LCD_SRV

Large Composition Data Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Large Composition Data Server model.

CONFIG_BLE_MESH_RPR_CLI

Remote Provisioning Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Remote Provisioning Client model

CONFIG_BLE_MESH_RPR_CLI_PROV_SAME_TIME

Maximum number of PB-Remote running at the same time by Provisioner

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_RPR_CLI

This option specifies how many devices can be provisioned at the same time using PB-REMOTE. For example, if the value is 2, it means a Provisioner can provision two unprovisioned devices with PB-REMOTE at the same time.

Range:

- from 1 to 5 if *CONFIG_BLE_MESH_RPR_CLI* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_RPR_CLI* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RPR_SRV

Remote Provisioning Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Remote Provisioning Server model

CONFIG_BLE_MESH_RPR_SRV_MAX_SCANNED_ITEMS

Maximum number of device information can be scanned

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_RPR_SRV

This option specifies how many device information can a Remote Provisioning Server store each time while scanning.

Range:

- from 4 to 255 if *CONFIG_BLE_MESH_RPR_SRV* && *CONFIG_BLE_MESH*

Default value:

- 10 if *CONFIG_BLE_MESH_RPR_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_RPR_SRV_ACTIVE_SCAN

Support Active Scan for remote provisioning

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_RPR_SRV

Enable this option to support Active Scan for remote provisioning.

CONFIG_BLE_MESH_RPR_SRV_MAX_EXT_SCAN

Maximum number of extended scan procedures

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_RPR_SRV

This option specifies how many extended scan procedures can be started by the Remote Provisioning Server.

Range:

- from 1 to 10 if *CONFIG_BLE_MESH_RPR_SRV* && *CONFIG_BLE_MESH*

Default value:

- 1 if *CONFIG_BLE_MESH_RPR_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_DF_CLI

Directed Forwarding Configuration Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Directed Forwarding Configuration Client model.

CONFIG_BLE_MESH_DF_SRV

Directed Forwarding Configuration Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models

Enable support for Directed Forwarding Configuration Server model.

CONFIG_BLE_MESH_MAX_DISC_TABLE_ENTRY_COUNT

Maximum number of discovery table entries in a given subnet

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_DF_SRV

Maximum number of Discovery Table entries supported by the node in a given subnet.

Range:

- from 2 to 255 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_FORWARD_TABLE_ENTRY_COUNT

Maximum number of forward table entries in a given subnet

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Foundation models > CONFIG_BLE_MESH_DF_SRV

Maximum number of Forward Table entries supported by the node in a given subnet.

Range:

- from 2 to 64 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_DEPS_NODES_PER_PATH

Maximum number of dependent nodes per path

Found in: Component config > *CONFIG_BLE_MESH* > *Support for BLE Mesh Foundation models* > *CONFIG_BLE_MESH_DF_SRV*

Maximum size of dependent nodes list supported by each forward table entry.

Range:

- from 2 to 64 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_PATH_MONITOR_TEST

Enable Path Monitoring test mode

Found in: Component config > *CONFIG_BLE_MESH* > *Support for BLE Mesh Foundation models* > *CONFIG_BLE_MESH_DF_SRV*

The option only removes the Path Use timer; all other behavior of the device is not changed. If Path Monitoring test mode is going to be used, this option should be enabled.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SUPPORT_DIRECTED_PROXY

Enable Directed Proxy functionality

Found in: Component config > *CONFIG_BLE_MESH* > *Support for BLE Mesh Foundation models* > *CONFIG_BLE_MESH_DF_SRV*

Support Directed Proxy functionality.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH_GATT_PROXY_SERVER* && *CONFIG_BLE_MESH_DF_SRV* && *CONFIG_BLE_MESH*

Support for BLE Mesh Client/Server models

Contains:

- *CONFIG_BLE_MESH_MBT_CLI*
- *CONFIG_BLE_MESH_MBT_SRV*
- *CONFIG_BLE_MESH_GENERIC_BATTERY_CLI*
- *CONFIG_BLE_MESH_GENERIC_DEF_TRANS_TIME_CLI*
- *CONFIG_BLE_MESH_GENERIC_LEVEL_CLI*
- *CONFIG_BLE_MESH_GENERIC_LOCATION_CLI*
- *CONFIG_BLE_MESH_GENERIC_ONOFF_CLI*
- *CONFIG_BLE_MESH_GENERIC_POWER_LEVEL_CLI*
- *CONFIG_BLE_MESH_GENERIC_POWER_ONOFF_CLI*
- *CONFIG_BLE_MESH_GENERIC_PROPERTY_CLI*
- *CONFIG_BLE_MESH_GENERIC_SERVER*
- *CONFIG_BLE_MESH_LIGHT_CTL_CLI*
- *CONFIG_BLE_MESH_LIGHT_HSL_CLI*
- *CONFIG_BLE_MESH_LIGHT_LC_CLI*
- *CONFIG_BLE_MESH_LIGHT_LIGHTNESS_CLI*
- *CONFIG_BLE_MESH_LIGHT_XYL_CLI*
- *CONFIG_BLE_MESH_LIGHTING_SERVER*

- *CONFIG_BLE_MESH_SCENE_CLI*
- *CONFIG_BLE_MESH_SCHEDULER_CLI*
- *CONFIG_BLE_MESH_SENSOR_CLI*
- *CONFIG_BLE_MESH_SENSOR_SERVER*
- *CONFIG_BLE_MESH_TIME_SCENE_SERVER*
- *CONFIG_BLE_MESH_TIME_CLI*

CONFIG_BLE_MESH_GENERIC_ONOFF_CLI

Generic OnOff Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic OnOff Client model.

CONFIG_BLE_MESH_GENERIC_LEVEL_CLI

Generic Level Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Level Client model.

CONFIG_BLE_MESH_GENERIC_DEF_TRANS_TIME_CLI

Generic Default Transition Time Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Default Transition Time Client model.

CONFIG_BLE_MESH_GENERIC_POWER_ONOFF_CLI

Generic Power OnOff Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Power OnOff Client model.

CONFIG_BLE_MESH_GENERIC_POWER_LEVEL_CLI

Generic Power Level Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Power Level Client model.

CONFIG_BLE_MESH_GENERIC_BATTERY_CLI

Generic Battery Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Battery Client model.

CONFIG_BLE_MESH_GENERIC_LOCATION_CLI

Generic Location Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Location Client model.

CONFIG_BLE_MESH_GENERIC_PROPERTY_CLI

Generic Property Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic Property Client model.

CONFIG_BLE_MESH_SENSOR_CLI

Sensor Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Sensor Client model.

CONFIG_BLE_MESH_TIME_CLI

Time Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Time Client model.

CONFIG_BLE_MESH_SCENE_CLI

Scene Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Scene Client model.

CONFIG_BLE_MESH_SCHEDULER_CLI

Scheduler Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Scheduler Client model.

CONFIG_BLE_MESH_LIGHT_LIGHTNESS_CLI

Light Lightness Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Light Lightness Client model.

CONFIG_BLE_MESH_LIGHT_CTL_CLI

Light CTL Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Light CTL Client model.

CONFIG_BLE_MESH_LIGHT_HSL_CLI

Light HSL Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Light HSL Client model.

CONFIG_BLE_MESH_LIGHT_XYL_CLI

Light XYL Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Light XYL Client model.

CONFIG_BLE_MESH_LIGHT_LC_CLI

Light LC Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Light LC Client model.

CONFIG_BLE_MESH_GENERIC_SERVER

Generic server models

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Generic server models.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SENSOR_SERVER

Sensor server models

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Sensor server models.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_TIME_SCENE_SERVER

Time and Scenes server models

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Time and Scenes server models.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_LIGHTING_SERVER

Lighting server models

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for Lighting server models.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MBT_CLI

BLOB Transfer Client model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for BLOB Transfer Client model.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MAX_BLOB_RECEIVERS

Maximum number of simultaneous blob receivers

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models > CONFIG_BLE_MESH_MBT_CLI

Maximum number of BLOB Transfer Server models that can participating in the BLOB transfer with a BLOB Transfer Client model.

Range:

- from 1 to 255 if *CONFIG_BLE_MESH_MBT_CLI* && *CONFIG_BLE_MESH*

Default value:

- 2 if *CONFIG_BLE_MESH_MBT_CLI* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_MBT_SRV

BLOB Transfer Server model

Found in: Component config > CONFIG_BLE_MESH > Support for BLE Mesh Client/Server models

Enable support for BLOB Transfer Server model.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_IV_UPDATE_TEST

Test the IV Update Procedure

Found in: Component config > CONFIG_BLE_MESH

This option removes the 96 hour limit of the IV Update Procedure and lets the state to be changed at any time. If IV Update test mode is going to be used, this option should be enabled.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

BLE Mesh specific test option

Contains:

- *CONFIG_BLE_MESH_DEBUG*
- *CONFIG_BLE_MESH_SHELL*
- *CONFIG_BLE_MESH_BQB_TEST*
- *CONFIG_BLE_MESH_SELF_TEST*
- *CONFIG_BLE_MESH_TEST_AUTO_ENTER_NETWORK*
- *CONFIG_BLE_MESH_TEST_USE_WHITE_LIST*

CONFIG_BLE_MESH_SELF_TEST

Perform BLE Mesh self-tests

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

This option adds extra self-tests which are run every time BLE Mesh networking is initialized.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_BQB_TEST

Enable BLE Mesh specific internal test

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

This option is used to enable some internal functions for auto-pts test.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_TEST_AUTO_ENTER_NETWORK

Unprovisioned device enters mesh network automatically

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

With this option enabled, an unprovisioned device can automatically enters mesh network using a specific test function without the pro- visioning procedure. And on the Provisioner side, a test function needs to be invoked to add the node information into the mesh stack.

Default value:

- Yes (enabled) if *CONFIG_BLE_MESH_SELF_TEST* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_TEST_USE_WHITE_LIST

Use white list to filter mesh advertising packets

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

With this option enabled, users can use white list to filter mesh advertising packets while scanning.

Default value:

- No (disabled) if *CONFIG_BLE_MESH_SELF_TEST* && *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_SHELL

Enable BLE Mesh shell

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

Activate shell module that provides BLE Mesh commands to the console.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_DEBUG

Enable BLE Mesh debug logs

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option

Enable debug logs for the BLE Mesh functionality.

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

CONFIG_BLE_MESH_DEBUG_NET

Network layer debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Network layer debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_TRANS

Transport layer debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Transport layer debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_BEACON

Beacon debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Beacon-related debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_CRYPTO

Crypto debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable cryptographic debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_PROV

Provisioning debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Provisioning debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_ACCESS

Access layer debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Access layer debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_MODEL

Foundation model debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Foundation Models debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_ADV

Advertising debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable advertising debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_LOW_POWER

Low Power debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Low Power debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_FRIEND

Friend debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Friend debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_DEBUG_PROXY

Proxy debug

Found in: Component config > CONFIG_BLE_MESH > BLE Mesh specific test option > CONFIG_BLE_MESH_DEBUG

Enable Proxy protocol debug logs for the BLE Mesh functionality.

CONFIG_BLE_MESH_EXPERIMENTAL

Make BLE Mesh experimental features visible

Found in: Component config > CONFIG_BLE_MESH

Make BLE Mesh Experimental features visible. Experimental features list: - CONFIG_BLE_MESH_NOT_RELAY_REPLAY_MSG

Default value:

- No (disabled) if *CONFIG_BLE_MESH*

Console Library Contains:

- *CONFIG_CONSOLE_SORTED_HELP*

CONFIG_CONSOLE_SORTED_HELP

Enable sorted help

Found in: Component config > Console Library

Instead of listing the commands in the order of registration, the help command lists the available commands in sorted order, if this option is enabled.

Default value:

- No (disabled)

Driver Configurations Contains:

- *Legacy ADC Driver Configuration*
- *Legacy DAC Driver Configurations*
- *Legacy I2S Driver Configurations*
- *Legacy MCPWM Driver Configurations*
- *Legacy PCNT Driver Configurations*
- *Legacy RMT Driver Configurations*
- *Legacy SDM Driver Configurations*
- *Legacy Temperature Sensor Driver Configurations*
- *Legacy Timer Group Driver Configurations*
- *TWAI Configuration*

TWAI Configuration Contains:

- *CONFIG_TWAI_ERRATA_FIX_LISTEN_ONLY_DOM*
- *CONFIG_TWAI_ISR_IN_IRAM*

CONFIG_TWAI_ISR_IN_IRAM

Place TWAI ISR function into IRAM

Found in: Component config > Driver Configurations > TWAI Configuration

Place the TWAI ISR in to IRAM. This will allow the ISR to avoid cache misses, and also be able to run whilst the cache is disabled (such as when writing to SPI Flash). Note that if this option is enabled: - Users should also set the ESP_INTR_FLAG_IRAM in the driver configuration structure when installing the driver (see docs for specifics). - Alert logging (i.e., setting of the TWAI_ALERT_AND_LOG flag) will have no effect.

Default value:

- No (disabled)

CONFIG_TWAI_ERRATA_FIX_LISTEN_ONLY_DOM

Add SW workaround for listen only transmits dominant bit errata

Found in: Component config > Driver Configurations > TWAI Configuration

When in the listen only mode, the TWAI controller must not influence the TWAI bus (i.e., must not send any dominant bits). However, while in listen only mode on the ESP32/ESP32-S2/ESP32-S3/ESP32-C3, the TWAI controller will still transmit dominant bits when it detects an error (i.e., as part of an active error frame). Enabling this option will add a workaround that forces the TWAI controller into an error passive state on initialization, thus preventing any dominant bits from being sent.

Default value:

- Yes (enabled)

Legacy ADC Driver Configuration Contains:

- *CONFIG_ADC_DISABLE_DAC*
- *Legacy ADC Calibration Configuration*
- *CONFIG_ADC_SUPPRESS_DEPRECATE_WARN*

CONFIG_ADC_DISABLE_DAC

Disable DAC when ADC2 is used on GPIO 25 and 26

Found in: Component config > Driver Configurations > Legacy ADC Driver Configuration

If this is set, the ADC2 driver will disable the output of the DAC corresponding to the specified channel. This is the default value.

For testing, disable this option so that we can measure the output of DAC by internal ADC.

Default value:

- Yes (enabled) if SOC_DAC_SUPPORTED

CONFIG_ADC_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy ADC Driver Configuration

Whether to suppress the deprecation warnings when using legacy adc driver (driver/adc.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy ADC Calibration Configuration Contains:

- *CONFIG_ADC_CALI_SUPPRESS_DEPRECATE_WARN*

CONFIG_ADC_CALI_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy ADC Driver Configuration > Legacy ADC Calibration Configuration

Whether to suppress the deprecation warnings when using legacy adc calibration driver (esp_adc_cal.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy DAC Driver Configurations Contains:

- *CONFIG_DAC_SUPPRESS_DEPRECATE_WARN*

CONFIG_DAC_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy DAC Driver Configurations

Whether to suppress the deprecation warnings when using legacy dac driver (driver/dac.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled) if SOC_DAC_SUPPORTED

Legacy MCPWM Driver Configurations Contains:

- *CONFIG_MCPWM_SUPPRESS_DEPRECATE_WARN*

CONFIG_MCPWM_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy MCPWM Driver Configurations

Whether to suppress the deprecation warnings when using legacy MCPWM driver (driver/mcpwm.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy Timer Group Driver Configurations Contains:

- *CONFIG_GPTIMER_SUPPRESS_DEPRECATE_WARN*

CONFIG_GPTIMER_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy Timer Group Driver Configurations

Whether to suppress the deprecation warnings when using legacy timer group driver (driver/timer.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy RMT Driver Configurations Contains:

- *CONFIG_RMT_SUPPRESS_DEPRECATE_WARN*

CONFIG_RMT_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy RMT Driver Configurations

Whether to suppress the deprecation warnings when using legacy rmt driver (driver/rmt.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy I2S Driver Configurations Contains:

- *CONFIG_I2S_SUPPRESS_DEPRECATE_WARN*

CONFIG_I2S_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy I2S Driver Configurations

Whether to suppress the deprecation warnings when using legacy i2s driver (driver/i2s.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy PCNT Driver Configurations Contains:

- *CONFIG_PCNT_SUPPRESS_DEPRECATE_WARN*

CONFIG_PCNT_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy PCNT Driver Configurations

whether to suppress the deprecation warnings when using legacy PCNT driver (driver/pcnt.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy SDM Driver Configurations Contains:

- *CONFIG_SDM_SUPPRESS_DEPRECATE_WARN*

CONFIG_SDM_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy SDM Driver Configurations

whether to suppress the deprecation warnings when using legacy SDM driver (driver/sigmadelta.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

Legacy Temperature Sensor Driver Configurations Contains:

- *CONFIG_TEMP_SENSOR_SUPPRESS_DEPRECATE_WARN*

CONFIG_TEMP_SENSOR_SUPPRESS_DEPRECATE_WARN

Suppress legacy driver deprecated warning

Found in: Component config > Driver Configurations > Legacy Temperature Sensor Driver Configurations

whether to suppress the deprecation warnings when using legacy temperature sensor driver (driver/temp_sensor.h). If you want to continue using the legacy driver, and don't want to see related deprecation warnings, you can enable this option.

Default value:

- No (disabled)

eFuse Bit Manager Contains:

- *CONFIG_EFUSE_VIRTUAL*
- *CONFIG_EFUSE_CUSTOM_TABLE*

CONFIG_EFUSE_CUSTOM_TABLE

Use custom eFuse table

Found in: Component config > eFuse Bit Manager

Allows to generate a structure for eFuse from the CSV file.

Default value:

- No (disabled)

CONFIG_EFUSE_CUSTOM_TABLE_FILENAME

Custom eFuse CSV file

Found in: Component config > eFuse Bit Manager > CONFIG_EFUSE_CUSTOM_TABLE

Name of the custom eFuse CSV filename. This path is evaluated relative to the project root directory.

Default value:

- "main/esp_efuse_custom_table.csv" if *CONFIG_EFUSE_CUSTOM_TABLE*

CONFIG_EFUSE_VIRTUAL

Simulate eFuse operations in RAM

Found in: Component config > eFuse Bit Manager

If "n" - No virtual mode. All eFuse operations are real and use eFuse registers. If "y" - The virtual mode is enabled and all eFuse operations (read and write) are redirected to RAM instead of eFuse registers, all permanent changes (via eFuse) are disabled. Log output will state changes that would be applied, but they will not be.

If it is "y", then SECURE_FLASH_ENCRYPTION_MODE_RELEASE cannot be used. Because the EFUSE VIRT mode is for testing only.

During startup, the eFuses are copied into RAM. This mode is useful for fast tests.

Default value:

- No (disabled)

CONFIG_EFUSE_VIRTUAL_KEEP_IN_FLASH

Keep eFuses in flash

Found in: Component config > eFuse Bit Manager > CONFIG_EFUSE_VIRTUAL

In addition to the "Simulate eFuse operations in RAM" option, this option just adds a feature to keep eFuses after reboots in flash memory. To use this mode the partition_table should have the *efuse* partition. partition.csv: "efuse_em, data, efuse, , 0x2000,"

During startup, the eFuses are copied from flash or, in case if flash is empty, from real eFuse to RAM and then update flash. This mode is useful when need to keep changes after reboot (testing secure_boot and flash_encryption).

CONFIG_EFUSE_VIRTUAL_LOG_ALL_WRITES

Log all virtual writes

Found in: Component config > eFuse Bit Manager > CONFIG_EFUSE_VIRTUAL

If enabled, log efuse burns. This shows changes that would be made.

ESP-TLS Contains:

- *CONFIG_ESP_TLS_INSECURE*
- *CONFIG_ESP_TLS_SERVER_CERT_SELECT_HOOK*
- *CONFIG_ESP_TLS_LIBRARY_CHOOSE*
- *CONFIG_ESP_TLS_CLIENT_SESSION_TICKETS*
- *CONFIG_ESP_DEBUG_WOLFSSL*
- *CONFIG_ESP_TLS_PSK_VERIFICATION*
- *CONFIG_ESP_TLS_SERVER_SESSION_TICKETS*
- *CONFIG_ESP_WOLFSSL_SMALL_CERT_VERIFY*
- *CONFIG_ESP_TLS_SERVER_MIN_AUTH_MODE_OPTIONAL*
- *CONFIG_ESP_TLS_USE_DS_PERIPHERAL*

CONFIG_ESP_TLS_LIBRARY_CHOOSE

Choose SSL/TLS library for ESP-TLS (See help for more Info)

Found in: Component config > ESP-TLS

The ESP-TLS APIs support multiple backend TLS libraries. Currently mbedTLS and WolfSSL are supported. Different TLS libraries may support different features and have different resource usage. Consult the ESP-TLS documentation in ESP-IDF Programming guide for more details.

Available options:

- mbedTLS (CONFIG_ESP_TLS_USING_MBEDTLS)
- wolfSSL (License info in wolfSSL directory README) (CONFIG_ESP_TLS_USING_WOLFSSL)

CONFIG_ESP_TLS_USE_DS_PERIPHERAL

Use Digital Signature (DS) Peripheral with ESP-TLS

Found in: Component config > ESP-TLS

Enable use of the Digital Signature Peripheral for ESP-TLS.The DS peripheral can only be used when it is appropriately configured for TLS. Consult the ESP-TLS documentation in ESP-IDF Programming Guide for more details.

Default value:

- Yes (enabled)

CONFIG_ESP_TLS_CLIENT_SESSION_TICKETS

Enable client session tickets

Found in: Component config > ESP-TLS

Enable session ticket support as specified in RFC5077.

CONFIG_ESP_TLS_SERVER_SESSION_TICKETS

Enable server session tickets

Found in: Component config > ESP-TLS

Enable session ticket support as specified in RFC5077

CONFIG_ESP_TLS_SERVER_SESSION_TICKET_TIMEOUT

Server session ticket timeout in seconds

Found in: Component config > ESP-TLS > CONFIG_ESP_TLS_SERVER_SESSION_TICKETS

Sets the session ticket timeout used in the tls server.

Default value:

- 86400 if *CONFIG_ESP_TLS_SERVER_SESSION_TICKETS*

CONFIG_ESP_TLS_SERVER_CERT_SELECT_HOOK

Certificate selection hook

Found in: Component config > ESP-TLS

Ability to configure and use a certificate selection callback during server handshake, to select a certificate to present to the client based on the TLS extensions supplied in the client hello (alpn, sni, etc).

CONFIG_ESP_TLS_SERVER_MIN_AUTH_MODE_OPTIONAL

ESP-TLS Server: Set minimum Certificate Verification mode to Optional

Found in: Component config > ESP-TLS

When this option is enabled, the peer (here, the client) certificate is checked by the server, however the handshake continues even if verification failed. By default, the peer certificate is not checked and ignored by the server.

mbedtls_ssl_get_verify_result() can be called after the handshake is complete to retrieve status of verification.

CONFIG_ESP_TLS_PSK_VERIFICATION

Enable PSK verification

Found in: Component config > ESP-TLS

Enable support for pre shared key ciphers, supported for both mbedTLS as well as wolfSSL TLS library.

CONFIG_ESP_TLS_INSECURE

Allow potentially insecure options

Found in: Component config > ESP-TLS

You can enable some potentially insecure options. These options should only be used for testing pusposes. Only enable these options if you are very sure.

CONFIG_ESP_TLS_SKIP_SERVER_CERT_VERIFY

Skip server certificate verification by default (WARNING: ONLY FOR TESTING PURPOSE, READ HELP)

Found in: Component config > ESP-TLS > CONFIG_ESP_TLS_INSECURE

After enabling this option the esp-tls client will skip the server certificate verification by default. Note that this option will only modify the default behaviour of esp-tls client regarding server cert verification. The default behaviour should only be applicable when no other option regarding the server cert verification is opted in the esp-tls config (e.g. crt_bundle_attach, use_global_ca_store etc.). WARNING : Enabling this option comes with a potential risk of establishing a TLS connection with a server which has a fake identity, provided that the server certificate is not provided either through API or other mechanism like ca_store etc.

CONFIG_ESP_WOLFSSL_SMALL_CERT_VERIFY

Enable SMALL_CERT_VERIFY

Found in: Component config > ESP-TLS

Enables server verification with Intermediate CA cert, does not authenticate full chain of trust upto the root CA cert (After Enabling this option client only needs to have Intermediate CA certificate of the server to authenticate server, root CA cert is not necessary).

Default value:

- Yes (enabled) if *CONFIG_ESP_TLS_USING_WOLFSSL*

CONFIG_ESP_DEBUG_WOLFSSL

Enable debug logs for wolfSSL

Found in: Component config > ESP-TLS

Enable detailed debug prints for wolfSSL SSL library.

ADC and ADC Calibration

Contains:

- *ADC Calibration Configurations*
- *CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE*
- *CONFIG_ADC_DISABLE_DAC_OUTPUT*
- *CONFIG_ADC_ENABLE_DEBUG_LOG*
- *CONFIG_ADC_CONTINUOUS_FORCE_USE_ADC2_ON_C3_S3*
- *CONFIG_ADC_ONESHOT_CTRL_FUNC_IN_IRAM*

CONFIG_ADC_ONESHOT_CTRL_FUNC_IN_IRAM

Place ISR version ADC oneshot mode read function into IRAM

Found in: Component config > ADC and ADC Calibration

Place ISR version ADC oneshot mode read function into IRAM.

Default value:

- No (disabled)

CONFIG_ADC_CONTINUOUS_ISR_IRAM_SAFE

ADC continuous mode driver ISR IRAM-Safe

Found in: Component config > ADC and ADC Calibration

Ensure the ADC continuous mode ISR is IRAM-Safe. When enabled, the ISR handler will be available when the cache is disabled.

Default value:

- No (disabled)

ADC Calibration Configurations

CONFIG_ADC_DISABLE_DAC_OUTPUT

Disable DAC when ADC2 is in use

Found in: Component config > ADC and ADC Calibration

By default, this is set. The ADC oneshot driver will disable the output of the corresponding DAC channels: ESP32: IO25 and IO26 ESP32S2: IO17 and IO18

Disable this option so as to measure the output of DAC by internal ADC, for test usage.

Default value:

- Yes (enabled) if SOC_DAC_SUPPORTED

CONFIG_ADC_CONTINUOUS_FORCE_USE_ADC2_ON_C3_S3

Force use ADC2 continumous mode on ESP32S3 or ESP32C3

Found in: Component config > ADC and ADC Calibration

On ESP32C3 and ESP32S3, ADC2 Digital Controller is not stable. Therefore, ADC2 continuous mode is not suggested on ESP32S3 and ESP32C3

If you stick to this, you can enable this option to force use ADC2 under above conditions. For more details, you can search for errata on espressif website.

Default value:

- No (disabled)

CONFIG_ADC_ENABLE_DEBUG_LOG

Enable ADC debug log

Found in: Component config > ADC and ADC Calibration

whether to enable the debug log message for ADC driver. Note that this option only controls the ADC driver log, will not affect other drivers.

note: This cannot be used in the ADC legacy driver.

Default value:

- No (disabled)

Wireless Coexistence

Contains:

- *CONFIG_ESP_COEX_EXTERNAL_COEXIST_ENABLE*
- *CONFIG_ESP_COEX_SW_COEXIST_ENABLE*
- *CONFIG_ESP_COEX_POWER_MANAGEMENT*

CONFIG_ESP_COEX_SW_COEXIST_ENABLE

Software controls WiFi/Bluetooth coexistence

Found in: Component config > Wireless Coexistence

If enabled, WiFi & Bluetooth coexistence is controlled by software rather than hardware. Recommended for heavy traffic scenarios. Both coexistence configuration options are automatically managed, no user intervention is required. If only Bluetooth is used, it is recommended to disable this option to reduce binary file size.

Default value:

- Yes (enabled) if *CONFIG_BT_ENABLED* || *CONFIG_IEEE802154_ENABLED* || (*CONFIG_IEEE802154_ENABLED* && *CONFIG_BT_ENABLED*)

CONFIG_ESP_COEX_EXTERNAL_COEXIST_ENABLE

External Coexistence

Found in: Component config > Wireless Coexistence

If enabled, HW External coexistence arbitration is managed by GPIO pins. It can support three types of wired combinations so far which are 1-wired/2-wired/3-wired. User can select GPIO pins in application code with configure interfaces.

This function depends on BT-off because currently we do not support external coex and internal coex simultaneously.

CONFIG_ESP_COEX_POWER_MANAGEMENT

Support power management under coexistence

Found in: Component config > Wireless Coexistence

If enabled, coexist power management will be enabled.

Default value:

- No (disabled) if *CONFIG_ESP_COEX_SW_COEXIST_ENABLE*

Common ESP-related Contains:

- *CONFIG_ESP_ERR_TO_NAME_LOOKUP*

CONFIG_ESP_ERR_TO_NAME_LOOKUP

Enable lookup of error code strings

Found in: Component config > Common ESP-related

Functions esp_err_to_name() and esp_err_to_name_r() return string representations of error codes from a pre-generated lookup table. This option can be used to turn off the use of the look-up table in order to save memory but this comes at the price of sacrificing distinguishable (meaningful) output string representations.

Default value:

- Yes (enabled)

ESP-Driver:Analog Comparator Configurations Contains:

- *CONFIG_ANA_CMPR_ISR_IRAM_SAFE*
- *CONFIG_ANA_CMPR_ENABLE_DEBUG_LOG*
- *CONFIG_ANA_CMPR_CTRL_FUNC_IN_IRAM*

CONFIG_ANA_CMPR_ISR_IRAM_SAFE

Analog comparator ISR IRAM-Safe

Found in: Component config > ESP-Driver:Analog Comparator Configurations

Ensure the Analog Comparator interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled) if SOC_ANA_CMPR_SUPPORTED

CONFIG_ANA_CMPR_CTRL_FUNC_IN_IRAM

Place Analog Comparator control functions into IRAM

Found in: Component config > ESP-Driver:Analog Comparator Configurations

Place Analog Comparator control functions (like ana_cmpr_set_internal_reference) into IRAM, so that these functions can be IRAM-safe and able to be called in an IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled) if SOC_ANA_CMPR_SUPPORTED

CONFIG_ANA_CMPR_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:Analog Comparator Configurations

whether to enable the debug log message for Analog Comparator driver. Note that, this option only controls the Analog Comparator driver log, won't affect other drivers.

Default value:

- No (disabled) if SOC_ANA_CMPR_SUPPORTED

ESP-Driver:Camera Controller Configurations Contains:

- *CONFIG_CAM_CTLR_MIPI_CSI_ISR_IRAM_SAFE*
- *CONFIG_CAM_CTLR_DVP_CAM_ISR_IRAM_SAFE*
- *CONFIG_CAM_CTLR_ISP_DVP_ISR_IRAM_SAFE*

CONFIG_CAM_CTLR_MIPI_CSI_ISR_IRAM_SAFE

CSI ISR IRAM-Safe

Found in: Component config > ESP-Driver:Camera Controller Configurations

Ensure the CSI driver ISR is IRAM-Safe. When enabled, the ISR handler will be available when the cache is disabled.

Default value:

- No (disabled) if SOC_MIPI_CSI_SUPPORTED && (SOC_MIPI_CSI_SUPPORTED || SOC_LCDCAM_CAM_SUPPORTED)

CONFIG_CAM_CTLR_ISP_DVP_ISR_IRAM_SAFE

ISP_DVP ISR IRAM-Safe

Found in: Component config > ESP-Driver:Camera Controller Configurations

Ensure the ISP_DVP driver ISR is IRAM-Safe. When enabled, the ISR handler will be available when the cache is disabled.

Default value:

- No (disabled) if SOC_MIPI_CSI_SUPPORTED || SOC_LCDCAM_CAM_SUPPORTED

CONFIG_CAM_CTLR_DVP_CAM_ISR_IRAM_SAFE

DVP ISR IRAM-Safe

Found in: Component config > ESP-Driver:Camera Controller Configurations

Ensure the DVP driver ISR is IRAM-Safe. When enabled, the ISR handler will be available when the cache is disabled.

Default value:

- No (disabled) if SOC_LCDCAM_CAM_SUPPORTED && (SOC_MIPI_CSI_SUPPORTED || SOC_LCDCAM_CAM_SUPPORTED)

ESP-Driver:DAC Configurations Contains:

- *CONFIG_DAC_DMA_AUTO_16BIT_ALIGN*
- *CONFIG_DAC_ISR_IRAM_SAFE*
- *CONFIG_DAC_ENABLE_DEBUG_LOG*
- *CONFIG_DAC_CTRL_FUNC_IN_IRAM*

CONFIG_DAC_CTRL_FUNC_IN_IRAM

Place DAC control functions into IRAM

Found in: Component config > ESP-Driver:DAC Configurations

Place DAC control functions (e.g. 'dac_oneshot_output_voltage') into IRAM, so that this function can be IRAM-safe and able to be called in the other IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled) if SOC_DAC_SUPPORTED

CONFIG_DAC_ISR_IRAM_SAFE

DAC ISR IRAM-Safe

Found in: Component config > ESP-Driver:DAC Configurations

Ensure the DAC interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled) if SOC_DAC_SUPPORTED

CONFIG_DAC_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:DAC Configurations

whether to enable the debug log message for DAC driver. Note that, this option only controls the DAC driver log, won't affect other drivers.

Default value:

- No (disabled) if SOC_DAC_SUPPORTED

CONFIG_DAC_DMA_AUTO_16BIT_ALIGN

Align the continuous data to 16 bit automatically

Found in: Component config > ESP-Driver:DAC Configurations

Whether to left shift the continuous data to align every bytes to 16 bits in the driver. On ESP32, although the DAC resolution is only 8 bits, the hardware requires 16 bits data in continuous mode. By enabling this option, the driver will left shift 8 bits for the input data automatically. Only disable this option when you decide to do this step by yourself. Note that the driver will allocate a new piece of memory to save the converted data.

Default value:

- Yes (enabled) if SOC_DAC_DMA_16BIT_ALIGN && SOC_DAC_SUPPORTED

ESP-Driver:GPIO Configurations

Contains:

- *CONFIG_GPIO_CTRL_FUNC_IN_IRAM*

CONFIG_GPIO_CTRL_FUNC_IN_IRAM

Place GPIO control functions into IRAM

Found in: Component config > ESP-Driver:GPIO Configurations

Place GPIO control functions (like intr_disable/set_level) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context.

Default value:

- No (disabled)

ESP-Driver:GPTimer Configurations Contains:

- *CONFIG_GPTIMER_ENABLE_DEBUG_LOG*
- *CONFIG_GPTIMER_ISR_IRAM_SAFE*
- *CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM*
- *CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM*

CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM

Place GPTimer ISR handler into IRAM

Found in: Component config > ESP-Driver:GPTimer Configurations

Place GPTimer ISR handler into IRAM for better performance and fewer cache misses.

Default value:

- Yes (enabled)

CONFIG_GPTIMER_CTRL_FUNC_IN_IRAM

Place GPTimer control functions into IRAM

Found in: Component config > ESP-Driver:GPTimer Configurations

Place GPTimer control functions (like start/stop) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_GPTIMER_ISR_IRAM_SAFE

GPTimer ISR IRAM-Safe

Found in: Component config > ESP-Driver:GPTimer Configurations

Ensure the GPTimer interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_GPTIMER_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:GPTimer Configurations

whether to enable the debug log message for GPTimer driver. Note that, this option only controls the GPTimer driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:I2C Configurations Contains:

- *CONFIG_I2C_ENABLE_DEBUG_LOG*
- *CONFIG_I2C_ISR_IRAM_SAFE*

CONFIG_I2C_ISR_IRAM_SAFE

I2C ISR IRAM-Safe

Found in: Component config > ESP-Driver:I2C Configurations

Ensure the I2C interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write). note: This cannot be used in the I2C legacy driver.

Default value:

- No (disabled)

CONFIG_I2C_ENABLE_DEBUG_LOG

Enable I2C debug log

Found in: Component config > ESP-Driver:I2C Configurations

whether to enable the debug log message for I2C driver. Note that this option only controls the I2C driver log, will not affect other drivers.

note: This cannot be used in the I2C legacy driver.

Default value:

- No (disabled)

ESP-Driver:I2S Configurations

Contains:

- *CONFIG_I2S_ENABLE_DEBUG_LOG*
- *CONFIG_I2S_ISR_IRAM_SAFE*

CONFIG_I2S_ISR_IRAM_SAFE

I2S ISR IRAM-Safe

Found in: Component config > ESP-Driver:I2S Configurations

Ensure the I2S interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_I2S_ENABLE_DEBUG_LOG

Enable I2S debug log

Found in: Component config > ESP-Driver:I2S Configurations

whether to enable the debug log message for I2S driver. Note that, this option only controls the I2S driver log, will not affect other drivers.

Default value:

- No (disabled)

ESP-Driver:ISP Configurations

Contains:

- *CONFIG_ISP_ISR_IRAM_SAFE*

CONFIG_ISP_ISR_IRAM_SAFE

ISP driver ISR IRAM-Safe

Found in: Component config > ESP-Driver:ISP Configurations

Ensure the ISP driver ISR is IRAM-Safe. When enabled, the ISR handler will be available when the cache is disabled.

Default value:

- No (disabled) if SOC_ISP_SUPPORTED

ESP-Driver:JPEG-Codec Configurations Contains:

- *CONFIG_JPEG_ENABLE_DEBUG_LOG*

CONFIG_JPEG_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:JPEG-Codec Configurations

whether to enable the debug log message for JPEG driver. Note that, this option only controls the JPEG driver log, won't affect other drivers. Please also note, enable this option will make jpeg codec process speed much slower.

Default value:

- No (disabled) if SOC_JPEG_CODEC_SUPPORTED

ESP-Driver:LEDC Configurations Contains:

- *CONFIG_LEDC_CTRL_FUNC_IN_IRAM*

CONFIG_LEDC_CTRL_FUNC_IN_IRAM

Place LEDC control functions into IRAM

Found in: Component config > ESP-Driver:LEDC Configurations

Place LEDC control functions (ledc_update_duty and ledc_stop) into IRAM, so that these functions can be IRAM-safe and able to be called in an IRAM context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

ESP-Driver:MCPWM Configurations Contains:

- *CONFIG_MCPWM_ENABLE_DEBUG_LOG*
- *CONFIG_MCPWM_CTRL_FUNC_IN_IRAM*
- *CONFIG_MCPWM_ISR_IRAM_SAFE*

CONFIG_MCPWM_ISR_IRAM_SAFE

Place MCPWM ISR function into IRAM

Found in: Component config > ESP-Driver:MCPWM Configurations

This will ensure the MCPWM interrupt handle is IRAM-Safe, allow to avoid flash cache misses, and also be able to run whilst the cache is disabled. (e.g. SPI Flash write)

Default value:

- No (disabled)

CONFIG_MCPWM_CTRL_FUNC_IN_IRAM

Place MCPWM control functions into IRAM

Found in: Component config > ESP-Driver:MCPWM Configurations

Place MCPWM control functions (like set_compare_value) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_MCPWM_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:MCPWM Configurations

whether to enable the debug log message for MCPWM driver. Note that, this option only controls the MCPWM driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:Parallel IO Configurations Contains:

- *CONFIG_PARLIO_ENABLE_DEBUG_LOG*
- *CONFIG_PARLIO_ISR_IRAM_SAFE*

CONFIG_PARLIO_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:Parallel IO Configurations

whether to enable the debug log message for parallel IO driver. Note that, this option only controls the parallel IO driver log, won't affect other drivers.

Default value:

- No (disabled) if SOC_PARLIO_SUPPORTED

CONFIG_PARLIO_ISR_IRAM_SAFE

Parallel IO ISR IRAM-Safe

Found in: Component config > ESP-Driver:Parallel IO Configurations

Ensure the Parallel IO interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled) if SOC_PARLIO_SUPPORTED

ESP-Driver:PCNT Configurations Contains:

- *CONFIG_PCNT_ENABLE_DEBUG_LOG*
- *CONFIG_PCNT_ISR_IRAM_SAFE*
- *CONFIG_PCNT_CTRL_FUNC_IN_IRAM*

CONFIG_PCNT_CTRL_FUNC_IN_IRAM

Place PCNT control functions into IRAM

Found in: Component config > ESP-Driver:PCNT Configurations

Place PCNT control functions (like start/stop) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_PCNT_ISR_IRAM_SAFE

PCNT ISR IRAM-Safe

Found in: Component config > ESP-Driver:PCNT Configurations

Ensure the PCNT interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_PCNT_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:PCNT Configurations

whether to enable the debug log message for PCNT driver. Note that, this option only controls the PCNT driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:RMT Configurations

Contains:

- *CONFIG_RMT_ENABLE_DEBUG_LOG*
- *CONFIG_RMT_RECV_FUNC_IN_IRAM*
- *CONFIG_RMT_ISR_IRAM_SAFE*

CONFIG_RMT_ISR_IRAM_SAFE

RMT ISR IRAM-Safe

Found in: Component config > ESP-Driver:RMT Configurations

Ensure the RMT interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_RMT_RECV_FUNC_IN_IRAM

Place RMT receive function into IRAM

Found in: Component config > ESP-Driver:RMT Configurations

Place RMT receive function into IRAM, so that the receive function can be IRAM-safe and able to be called when the flash cache is disabled. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_RMT_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:RMT Configurations

whether to enable the debug log message for RMT driver. Note that, this option only controls the RMT driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:Sigma Delta Modulator Configurations Contains:

- *CONFIG_SDM_ENABLE_DEBUG_LOG*
- *CONFIG_SDM_CTRL_FUNC_IN_IRAM*

CONFIG_SDM_CTRL_FUNC_IN_IRAM

Place SDM control functions into IRAM

Found in: Component config > ESP-Driver:Sigma Delta Modulator Configurations

Place SDM control functions (like set_duty) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_SDM_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:Sigma Delta Modulator Configurations

whether to enable the debug log message for SDM driver. Note that, this option only controls the SDM driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:SPI Configurations Contains:

- *CONFIG_SPI_MASTER_ISR_IN_IRAM*
- *CONFIG_SPI_SLAVE_ISR_IN_IRAM*
- *CONFIG_SPI_MASTER_IN_IRAM*
- *CONFIG_SPI_SLAVE_IN_IRAM*

CONFIG_SPI_MASTER_IN_IRAM

Place transmitting functions of SPI master into IRAM

Found in: Component config > ESP-Driver:SPI Configurations

Normally only the ISR of SPI master is placed in the IRAM, so that it can work without the flash when interrupt is triggered. For other functions, there's some possibility that the flash cache miss when running inside and out of SPI functions, which may increase the interval of SPI transactions. Enable this to put `queue\_trans`, `get\_trans\_result` and `transmit` functions into the IRAM to avoid possible cache miss.

This configuration won't be available if *CONFIG_FREERTOS_PLACE_FUNCTIONS_INTO_FLASH* is enabled.

During unit test, this is enabled to measure the ideal case of api.

CONFIG_SPI_MASTER_ISR_IN_IRAM

Place SPI master ISR function into IRAM

Found in: Component config > ESP-Driver:SPI Configurations

Place the SPI master ISR in to IRAM to avoid possible cache miss.

Enabling this configuration is possible only when HEAP_PLACE_FUNCTION_INTO_FLASH is disabled since the spi master uses can allocate transactions buffers into DMA memory section using the heap component API that ipso facto has to be placed in IRAM.

Also you can forbid the ISR being disabled during flash writing access, by add ESP_INTR_FLAG_IRAM when initializing the driver.

CONFIG_SPI_SLAVE_IN_IRAM

Place transmitting functions of SPI slave into IRAM

Found in: Component config > ESP-Driver:SPI Configurations

Normally only the ISR of SPI slave is placed in the IRAM, so that it can work without the flash when interrupt is triggered. For other functions, there's some possibility that the flash cache miss when running inside and out of SPI functions, which may increase the interval of SPI transactions. Enable this to put `queue\_trans`, `get\_trans\_result` and `transmit` functions into the IRAM to avoid possible cache miss.

Default value:

- No (disabled)

CONFIG_SPI_SLAVE_ISR_IN_IRAM

Place SPI slave ISR function into IRAM

Found in: Component config > ESP-Driver:SPI Configurations

Place the SPI slave ISR in to IRAM to avoid possible cache miss.

Also you can forbid the ISR being disabled during flash writing access, by add ESP_INTR_FLAG_IRAM when initializing the driver.

Default value:

- Yes (enabled)

ESP-Driver:Touch Sensor Configurations

Contains:

- *CONFIG_TOUCH_ENABLE_DEBUG_LOG*
- *CONFIG_TOUCH_CTRL_FUNC_IN_IRAM*
- *CONFIG_TOUCH_ISR_IRAM_SAFE*

CONFIG_TOUCH_CTRL_FUNC_IN_IRAM

Place touch sensor control functions into IRAM

Found in: Component config > ESP-Driver:Touch Sensor Configurations

Place touch sensor oneshot scanning and continuous scanning functions into IRAM, so that these function can be IRAM-safe and able to be called when the flash cache is disabled. Enabling this option can improve driver performance as well.

Default value:

- No (disabled)

CONFIG_TOUCH_ISR_IRAM_SAFE

Touch sensor ISR IRAM-Safe

Found in: Component config > ESP-Driver:Touch Sensor Configurations

Ensure the touch sensor interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_TOUCH_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:Touch Sensor Configurations

Whether to enable the debug log message for touch driver. Note that, this option only controls the touch driver log, won't affect other drivers.

Default value:

- No (disabled)

ESP-Driver:Temperature Sensor Configurations Contains:

- *CONFIG_TEMP_SENSOR_ENABLE_DEBUG_LOG*
- *CONFIG_TEMP_SENSOR_ISR_IRAM_SAFE*

CONFIG_TEMP_SENSOR_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > ESP-Driver:Temperature Sensor Configurations

whether to enable the debug log message for temperature sensor driver. Note that, this option only controls the temperature sensor driver log, won't affect other drivers.

Default value:

- No (disabled)

CONFIG_TEMP_SENSOR_ISR_IRAM_SAFE

Temperature sensor ISR IRAM-Safe

Found in: Component config > ESP-Driver:Temperature Sensor Configurations

Ensure the Temperature Sensor interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write).

Default value:

- No (disabled) if SOC_TEMPERATURE_SENSOR_INTR_SUPPORT

ESP-Driver:UART Configurations Contains:

- *CONFIG_UART_ISR_IN_IRAM*

CONFIG_UART_ISR_IN_IRAM

Place UART ISR function into IRAM

Found in: Component config > ESP-Driver:UART Configurations

If this option is not selected, UART interrupt will be disabled for a long time and may cause data lost when doing spi flash operation.

ESP-Driver:USB Serial/JTAG Configuration Contains:

- *CONFIG_USJ_ENABLE_USB_SERIAL_JTAG*

CONFIG_USJ_ENABLE_USB_SERIAL_JTAG

Enable USB-Serial-JTAG Module

Found in: Component config > ESP-Driver:USB Serial/JTAG Configuration

The USB-Serial-JTAG module on ESP chips is turned on by default after power-on. If your application does not need it and not rely on it to be used as system console or use the built-in JTAG for debugging, you can disable this option, then the clock of this module will be disabled at startup, which will save some power consumption.

Default value:

- Yes (enabled)

CONFIG_USJ_NO_AUTO_LS_ON_CONNECTION

Don't enter the automatic light sleep when USB Serial/JTAG port is connected

Found in: Component config > ESP-Driver:USB Serial/JTAG Configuration > CONFIG_USJ_ENABLE_USB_SERIAL_JTAG

If enabled, the chip will constantly monitor the connection status of the USB Serial/JTAG port. As long as the USB Serial/JTAG is connected, a ESP_PM_NO_LIGHT_SLEEP power management lock will be acquired to prevent the system from entering light sleep. This option can be useful if serial monitoring is needed via USB Serial/JTAG while power management is enabled, as the USB Serial/JTAG cannot work under light sleep and after waking up from light sleep. Note. This option can only control the automatic Light-Sleep behavior. If esp_light_sleep_start() is called manually from the program, enabling this option will not prevent light sleep entry even if the USB Serial/JTAG is in use.

Ethernet Contains:

- *CONFIG_ETH_TRANSMIT_MUTEX*
- *CONFIG_ETH_USE_ESP32_EMAC*
- *CONFIG_ETH_USE_OPENETH*
- *CONFIG_ETH_USE_SPI_ETHERNET*

CONFIG_ETH_USE_ESP32_EMAC

Support ESP32 internal EMAC controller

Found in: Component config > Ethernet

ESP32 integrates a 10/100M Ethernet MAC controller.

Default value:

- Yes (enabled) if SOC_EMAC_SUPPORTED

Contains:

- *CONFIG_ETH_DMA_RX_BUFFER_NUM*
- *CONFIG_ETH_DMA_TX_BUFFER_NUM*

- *CONFIG_ETH_IRAM_OPTIMIZATION*
- *CONFIG_ETH_SOFT_FLOW_CONTROL*
- *CONFIG_ETH_DMA_BUFFER_SIZE*
- *CONFIG_ETH_PHY_INTERFACE*

CONFIG_ETH_PHY_INTERFACE

PHY interface

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

Select the communication interface between MAC and PHY chip.

Available options:

- Reduced Media Independent Interface (RMII) (CONFIG_ETH_PHY_INTERFACE_RMII)

CONFIG_ETH_DMA_BUFFER_SIZE

Ethernet DMA buffer size (Byte)

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

Set the size of each buffer used by Ethernet MAC DMA. !! Important !! Make sure it is 64B aligned for ESP32P4!

Range:
- from 256 to 1600 if *CONFIG_ETH_USE_ESP32_EMAC*

Default value:
- 512 if *CONFIG_ETH_USE_ESP32_EMAC*

CONFIG_ETH_DMA_RX_BUFFER_NUM

Amount of Ethernet DMA Rx buffers

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

Number of DMA receive buffers. Each buffer's size is ETH_DMA_BUFFER_SIZE. Larger number of buffers could increase throughput somehow.

Range:
- from 3 to 30 if *CONFIG_ETH_USE_ESP32_EMAC*

CONFIG_ETH_DMA_TX_BUFFER_NUM

Amount of Ethernet DMA Tx buffers

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

Number of DMA transmit buffers. Each buffer's size is ETH_DMA_BUFFER_SIZE. Larger number of buffers could increase throughput somehow.

Range:
- from 3 to 30 if *CONFIG_ETH_USE_ESP32_EMAC*

Default value:
- 10 if *CONFIG_ETH_USE_ESP32_EMAC*

CONFIG_ETH_SOFT_FLOW_CONTROL

Enable software flow control

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

Ethernet MAC engine on ESP32 doesn't feature a flow control logic. The MAC driver can perform a software flow control if you enable this option. Note that, if the RX buffer number is small, enabling software flow control will cause obvious performance loss.

Default value:

- No (disabled) if *CONFIG_ETH_DMA_RX_BUFFER_NUM* > 15 && *CONFIG_ETH_USE_ESP32_EMAC*

CONFIG_ETH_IRAM_OPTIMIZATION

Enable IRAM optimization

Found in: Component config > Ethernet > CONFIG_ETH_USE_ESP32_EMAC

If enabled, functions related to RX/TX are placed into IRAM. It can improve Ethernet throughput. If disabled, all functions are placed into FLASH.

Default value:

- No (disabled) if *CONFIG_ETH_USE_ESP32_EMAC*

CONFIG_ETH_USE_SPI_ETHERNET

Support SPI to Ethernet Module

Found in: Component config > Ethernet

ESP-IDF can also support some SPI-Ethernet modules.

Default value:

- Yes (enabled)

Contains:

- *CONFIG_ETH_SPI_ETHERNET_DM9051*
- *CONFIG_ETH_SPI_ETHERNET_KSZ8851SNL*
- *CONFIG_ETH_SPI_ETHERNET_W5500*

CONFIG_ETH_SPI_ETHERNET_DM9051

Use DM9051

Found in: Component config > Ethernet > CONFIG_ETH_USE_SPI_ETHERNET

DM9051 is a fast Ethernet controller with an SPI interface. It's also integrated with a 10/100M PHY and MAC. Select this to enable DM9051 driver.

CONFIG_ETH_SPI_ETHERNET_W5500

Use W5500 (MAC RAW)

Found in: Component config > Ethernet > CONFIG_ETH_USE_SPI_ETHERNET

W5500 is a HW TCP/IP embedded Ethernet controller. TCP/IP stack, 10/100 Ethernet MAC and PHY are embedded in a single chip. However the driver in ESP-IDF only enables the RAW MAC mode, making it compatible with the software TCP/IP stack. Say yes to enable W5500 driver.

CONFIG_ETH_SPI_ETHERNET_KSZ8851SNL

Use KSZ8851SNL

Found in: Component config > Ethernet > CONFIG_ETH_USE_SPI_ETHERNET

The KSZ8851SNL is a single-chip Fast Ethernet controller consisting of a 10/100 physical layer transceiver (PHY), a MAC, and a Serial Peripheral Interface (SPI). Select this to enable KSZ8851SNL driver.

CONFIG_ETH_USE_OPENETH

Support OpenCores Ethernet MAC (for use with QEMU)

Found in: Component config > Ethernet

OpenCores Ethernet MAC driver can be used when an ESP-IDF application is executed in QEMU. This driver is not supported when running on a real chip.

Default value:

- No (disabled)

Contains:

- *CONFIG_ETH_OPENETH_DMA_RX_BUFFER_NUM*
- *CONFIG_ETH_OPENETH_DMA_TX_BUFFER_NUM*

CONFIG_ETH_OPENETH_DMA_RX_BUFFER_NUM

Number of Ethernet DMA Rx buffers

Found in: Component config > Ethernet > CONFIG_ETH_USE_OPENETH

Number of DMA receive buffers, each buffer is 1600 bytes.

Range:

- from 1 to 64 if *CONFIG_ETH_USE_OPENETH*

Default value:

- 4 if *CONFIG_ETH_USE_OPENETH*

CONFIG_ETH_OPENETH_DMA_TX_BUFFER_NUM

Number of Ethernet DMA Tx buffers

Found in: Component config > Ethernet > CONFIG_ETH_USE_OPENETH

Number of DMA transmit buffers, each buffer is 1600 bytes.

Range:

- from 1 to 64 if *CONFIG_ETH_USE_OPENETH*

Default value:

- 1 if *CONFIG_ETH_USE_OPENETH*

CONFIG_ETH_TRANSMIT_MUTEX

Enable Transmit Mutex

Found in: Component config > Ethernet

Prevents multiple accesses when Ethernet interface is used as shared resource and multiple functionalities might try to access it at a time.

Default value:

- No (disabled)

Event Loop Library Contains:

- *CONFIG_ESP_EVENT_LOOP_PROFILING*
- *CONFIG_ESP_EVENT_POST_FROM_ISR*

CONFIG_ESP_EVENT_LOOP_PROFILING

Enable event loop profiling

Found in: Component config > Event Loop Library

Enables collections of statistics in the event loop library such as the number of events posted to/recieved by an event loop, number of callbacks involved, number of events dropped to to a full event loop queue, run time of event handlers, and number of times/run time of each event handler.

Default value:

- No (disabled)

CONFIG_ESP_EVENT_POST_FROM_ISR

Support posting events from ISRs

Found in: Component config > Event Loop Library

Enable posting events from interrupt handlers.

Default value:

- Yes (enabled)

CONFIG_ESP_EVENT_POST_FROM_IRAM_ISR

Support posting events from ISRs placed in IRAM

Found in: Component config > Event Loop Library > CONFIG_ESP_EVENT_POST_FROM_ISR

Enable posting events from interrupt handlers placed in IRAM. Enabling this option places API functions esp_event_post and esp_event_post_to in IRAM.

Default value:

- Yes (enabled)

GDB Stub Contains:

- *CONFIG_ESP_GDBSTUB_SUPPORT_TASKS*
- *CONFIG_ESP_SYSTEM_GDBSTUB_RUNTIME*

CONFIG_ESP_SYSTEM_GDBSTUB_RUNTIME

GDBStub at runtime

Found in: Component config > GDB Stub

Enable builtin GDBStub. This allows to debug the target device using serial port: - Run 'idf.py monitor'. - Wait for the device to initialize. - Press Ctrl+C to interrupt the execution and enter GDB attached to your device for debugging. NOTE: all UART input will be handled by GDBStub.

CONFIG_ESP_GDBSTUB_SUPPORT_TASKS

Enable listing FreeRTOS tasks through GDB Stub

Found in: Component config > GDB Stub

If enabled, GDBStub can supply the list of FreeRTOS tasks to GDB. Thread list can be queried from GDB using 'info threads' command. Note that if GDB task lists were corrupted, this feature may not work. If GDBStub fails, try disabling this feature.

Default value:

- Yes (enabled)

CONFIG_ESP_GDBSTUB_MAX_TASKS

Maximum number of tasks supported by GDB Stub

Found in: Component config > GDB Stub > CONFIG_ESP_GDBSTUB_SUPPORT_TASKS

Set the number of tasks which GDB Stub will support.

Default value:

- 32

ESP HTTP client

Contains:

- *CONFIG_ESP_HTTP_CLIENT_ENABLE_CUSTOM_TRANSPORT*
- *CONFIG_ESP_HTTP_CLIENT_ENABLE_BASIC_AUTH*
- *CONFIG_ESP_HTTP_CLIENT_ENABLE_DIGEST_AUTH*
- *CONFIG_ESP_HTTP_CLIENT_ENABLE_HTTPS*

CONFIG_ESP_HTTP_CLIENT_ENABLE_HTTPS

Enable https

Found in: Component config > ESP HTTP client

This option will enable https protocol by linking esp-tls library and initializing SSL transport

Default value:

- Yes (enabled)

CONFIG_ESP_HTTP_CLIENT_ENABLE_BASIC_AUTH

Enable HTTP Basic Authentication

Found in: Component config > ESP HTTP client

This option will enable HTTP Basic Authentication. It is disabled by default as Basic auth uses unencrypted encoding, so it introduces a vulnerability when not using TLS

Default value:

- No (disabled)

CONFIG_ESP_HTTP_CLIENT_ENABLE_DIGEST_AUTH

Enable HTTP Digest Authentication

Found in: Component config > ESP HTTP client

This option will enable HTTP Digest Authentication. It is enabled by default, but use of this configuration is not recommended as the password can be derived from the exchange, so it introduces a vulnerability when not using TLS

Default value:

- No (disabled)

CONFIG_ESP_HTTP_CLIENT_ENABLE_CUSTOM_TRANSPORT

Enable custom transport

Found in: Component config > ESP HTTP client

This option will enable injection of a custom tcp_transport handle, so the http operation will be performed on top of the user defined transport abstraction (if configured)

Default value:

- No (disabled)

HTTP Server Contains:

- *CONFIG_HTTPD_QUEUE_WORK_BLOCKING*
- *CONFIG_HTTPD_PURGE_BUF_LEN*
- *CONFIG_HTTPD_LOG_PURGE_DATA*
- *CONFIG_HTTPD_MAX_REQ_HDR_LEN*
- *CONFIG_HTTPD_MAX_URI_LEN*
- *CONFIG_HTTPD_ERR_RESP_NO_DELAY*
- *CONFIG_HTTPD_WS_SUPPORT*

CONFIG_HTTPD_MAX_REQ_HDR_LEN

Max HTTP Request Header Length

Found in: Component config > HTTP Server

This sets the maximum supported size of headers section in HTTP request packet to be processed by the server

Default value:

- 512

CONFIG_HTTPD_MAX_URI_LEN

Max HTTP URI Length

Found in: Component config > HTTP Server

This sets the maximum supported size of HTTP request URI to be processed by the server

Default value:

- 512

CONFIG_HTTPD_ERR_RESP_NO_DELAY

Use TCP_NODELAY socket option when sending HTTP error responses

Found in: Component config > HTTP Server

Using TCP_NODEALY socket option ensures that HTTP error response reaches the client before the underlying socket is closed. Please note that turning this off may cause multiple test failures

Default value:

- Yes (enabled)

CONFIG_HTTPD_PURGE_BUF_LEN

Length of temporary buffer for purging data

Found in: Component config > HTTP Server

This sets the size of the temporary buffer used to receive and discard any remaining data that is received from the HTTP client in the request, but not processed as part of the server HTTP request handler.

If the remaining data is larger than the available buffer size, the buffer will be filled in multiple iterations. The buffer should be small enough to fit on the stack, but large enough to avoid excessive iterations.

Default value:

- 32

CONFIG_HTTPD_LOG_PURGE_DATA

Log purged content data at Debug level

Found in: Component config > HTTP Server

Enabling this will log discarded binary HTTP request data at Debug level. For large content data this may not be desirable as it will clutter the log.

Default value:

- No (disabled)

CONFIG_HTTPD_WS_SUPPORT

WebSocket server support

Found in: Component config > HTTP Server

This sets the WebSocket server support.

Default value:

- No (disabled)

CONFIG_HTTPD_QUEUE_WORK_BLOCKING

httpd_queue_work as blocking API

Found in: Component config > HTTP Server

This makes httpd_queue_work() API to wait until a message space is available on UDP control socket. It internally uses a counting semaphore with count set to *LWIP_UDP_RECVMBOX_SIZE* to achieve this. This config will slightly change API behavior to block until message gets delivered on control socket.

ESP HTTPS OTA

Contains:

- *CONFIG_ESP_HTTPS_OTA_ALLOW_HTTP*
- *CONFIG_ESP_HTTPS_OTA_DECRYPT_CB*

CONFIG_ESP_HTTPS_OTA_DECRYPT_CB

Provide decryption callback

Found in: Component config > ESP HTTPS OTA

Exposes an additional callback whereby firmware data could be decrypted before being processed by OTA update component. This can help to integrate external encryption related format and removal of such encapsulation layer from firmware image.

Default value:

- No (disabled)

CONFIG_ESP_HTTPS_OTA_ALLOW_HTTP

Allow HTTP for OTA (WARNING: ONLY FOR TESTING PURPOSE, READ HELP)

Found in: Component config > ESP HTTPS OTA

It is highly recommended to keep HTTPS (along with server certificate validation) enabled. Enabling this option comes with potential risk of: - Non-encrypted communication channel with server - Accepting firmware upgrade image from server with fake identity

Default value:

- No (disabled)

ESP HTTPS server Contains:

- *CONFIG_ESP_HTTPS_SERVER_ENABLE*

CONFIG_ESP_HTTPS_SERVER_ENABLE

Enable ESP_HTTPS_SERVER component

Found in: Component config > ESP HTTPS server

Enable ESP HTTPS server component

Hardware Settings Contains:

- *2D-DMA Configurations*
- *Chip revision*
- *Crypto DPA Protection*
- *DW_GDMA Configurations*
- *ETM Configuration*
- *GDMA Configurations*
- *MAC Config*
- *Main XTAL Config*
- *Peripheral Control*
- *RTC Clock Config*
- *Sleep Config*

Chip revision Contains:

- *CONFIG_ESP_REV_NEW_CHIP_TEST*
- *CONFIG_ESP32S3_REV_MIN*

CONFIG_ESP32S3_REV_MIN

Minimum Supported ESP32-S3 Revision

Found in: Component config > Hardware Settings > Chip revision

Required minimum chip revision. ESP-IDF will check for it and reject to boot if the chip revision fails the check. This ensures the chip used will have some modifications (features, or bugfixes).

The complied binary will only support chips above this revision, this will also help to reduce binary size.

Available options:

- Rev v0.0 (ECO0) (CONFIG_ESP32S3_REV_MIN_0)
- Rev v0.1 (ECO1) (CONFIG_ESP32S3_REV_MIN_1)
- Rev v0.2 (ECO2) (CONFIG_ESP32S3_REV_MIN_2)

CONFIG_ESP_REV_NEW_CHIP_TEST

Internal test mode

Found in: Component config > Hardware Settings > Chip revision

For internal chip testing, a small number of new versions chips didn't update the version field in eFuse, you can enable this option to force the software recognize the chip version based on the rev selected in menuconfig.

Default value:

- No (disabled)

MAC Config Contains:

- *CONFIG_ESP_MAC_USE_CUSTOM_MAC_AS_BASE_MAC*
- *CONFIG_ESP32S3_UNIVERSAL_MAC_ADDRESSES*

CONFIG_ESP32S3_UNIVERSAL_MAC_ADDRESSES

Number of universally administered (by IEEE) MAC address

Found in: Component config > Hardware Settings > MAC Config

Configure the number of universally administered (by IEEE) MAC addresses. During initialization, MAC addresses for each network interface are generated or derived from a single base MAC address. If the number of universal MAC addresses is four, all four interfaces (WiFi station, WiFi softap, Bluetooth and Ethernet) receive a universally administered MAC address. These are generated sequentially by adding 0, 1, 2 and 3 (respectively) to the final octet of the base MAC address. If the number of universal MAC addresses is two, only two interfaces (WiFi station and Bluetooth) receive a universally administered MAC address. These are generated sequentially by adding 0 and 1 (respectively) to the base MAC address. The remaining two interfaces (WiFi softap and Ethernet) receive local MAC addresses. These are derived from the universal WiFi station and Bluetooth MAC addresses, respectively. When using the default (Espressif-assigned) base MAC address, either setting can be used. When using a custom universal MAC address range, the correct setting will depend on the allocation of MAC addresses in this range (either 2 or 4 per device.)

Available options:

- Two (CONFIG_ESP32S3_UNIVERSAL_MAC_ADDRESSES_TWO)
- Four (CONFIG_ESP32S3_UNIVERSAL_MAC_ADDRESSES_FOUR)

CONFIG_ESP_MAC_USE_CUSTOM_MAC_AS_BASE_MAC

Enable using custom mac as base mac

Found in: Component config > Hardware Settings > MAC Config

When this configuration is enabled, the user can invoke *esp_read_mac* to obtain the desired type of MAC using a custom MAC as the base MAC.

Default value:

- No (disabled)

Sleep Config Contains:

- *CONFIG_ESP_SLEEP_GPIO_ENABLE_INTERNAL_RESISTORS*
- *CONFIG_ESP_SLEEP_CACHE_SAFE_ASSERTION*
- *CONFIG_ESP_SLEEP_EVENT_CALLBACKS*
- *CONFIG_ESP_SLEEP_DEBUG*
- *CONFIG_ESP_SLEEP_WAIT_FLASH_READY_EXTRA_DELAY*

- *CONFIG_ESP_SLEEP_GPIO_RESET_WORKAROUND*
- *CONFIG_ESP_SLEEP_POWER_DOWN_FLASH*
- *CONFIG_ESP_SLEEP_MSPI_NEED_ALL_IO_PU*
- *CONFIG_ESP_SLEEP_FLASH_LEAKAGE_WORKAROUND*
- *CONFIG_ESP_SLEEP_PSRAM_LEAKAGE_WORKAROUND*

CONFIG_ESP_SLEEP_POWER_DOWN_FLASH

Power down flash in light sleep when there is no SPIRAM

Found in: Component config > Hardware Settings > Sleep Config

If enabled, chip will try to power down flash as part of esp_light_sleep_start(), which costs more time when chip wakes up. Can only be enabled if there is no SPIRAM configured.

This option will power down flash under a strict but relatively safe condition. Also, it is possible to power down flash under a relaxed condition by using esp_sleep_pd_config() to set ESP_PD_DOMAIN_VDDSDIO to ESP_PD_OPTION_OFF. It should be noted that there is a risk in powering down flash, you can refer *ESP-IDF Programming Guide/API Reference/System API/Sleep Modes/Power-down of Flash* for more details.

CONFIG_ESP_SLEEP_FLASH_LEAKAGE_WORKAROUND

Pull-up Flash CS pin in light sleep

Found in: Component config > Hardware Settings > Sleep Config

All IOs will be set to isolate(floating) state by default during sleep. Since the power supply of SPI Flash is not lost during lightsleep, if its CS pin is recognized as low level(selected state) in the floating state, there will be a large current leakage, and the data in Flash may be corrupted by random signals on other SPI pins. Select this option will set the CS pin of Flash to PULL-UP state during sleep, but this will increase the sleep current about 10 uA. If you are developing with esp32xx modules, you must select this option, but if you are developing with chips, you can also pull up the CS pin of SPI Flash in the external circuit to save power consumption caused by internal pull-up during sleep. (!!! Don't deselect this option if you don't have external SPI Flash CS pin pullups.)

CONFIG_ESP_SLEEP_PSRAM_LEAKAGE_WORKAROUND

Pull-up PSRAM CS pin in light sleep

Found in: Component config > Hardware Settings > Sleep Config

All IOs will be set to isolate(floating) state by default during sleep. Since the power supply of PSRAM is not lost during lightsleep, if its CS pin is recognized as low level(selected state) in the floating state, there will be a large current leakage, and the data in PSRAM may be corrupted by random signals on other SPI pins. Select this option will set the CS pin of PSRAM to PULL-UP state during sleep, but this will increase the sleep current about 10 uA. If you are developing with esp32xx modules, you must select this option, but if you are developing with chips, you can also pull up the CS pin of PSRAM in the external circuit to save power consumption caused by internal pull-up during sleep. (!!! Don't deselect this option if you don't have external PSRAM CS pin pullups.)

Default value:

- Yes (enabled) if *CONFIG_SPIRAM*

CONFIG_ESP_SLEEP_MSPI_NEED_ALL_IO_PU

Pull-up all SPI pins in light sleep

Found in: Component config > Hardware Settings > Sleep Config

To reduce leakage current, some types of SPI Flash/RAM only need to pull up the CS pin during light sleep. But there are also some kinds of SPI Flash/RAM that need to pull up all pins. It depends on the SPI Flash/RAM chip used.

CONFIG_ESP_SLEEP_GPIO_RESET_WORKAROUND

light sleep GPIO reset workaround

Found in: Component config > Hardware Settings > Sleep Config

esp32c2, esp32c3, esp32s3, esp32c6 and esp32h2 will reset at wake-up if GPIO is received a small electrostatic pulse during light sleep, with specific condition

- GPIO needs to be configured as input-mode only
- The pin receives a small electrostatic pulse, and reset occurs when the pulse voltage is higher than 6 V

For GPIO set to input mode only, it is not a good practice to leave it open/floating, The hardware design needs to controlled it with determined supply or ground voltage is necessary.

This option provides a software workaround for this issue. Configure to isolate all GPIO pins in sleep state.

Default value:

- Yes (enabled)

CONFIG_ESP_SLEEP_WAIT_FLASH_READY_EXTRA_DELAY

Extra delay (in us) after flash powerdown sleep wakeup to wait flash ready

Found in: Component config > Hardware Settings > Sleep Config

When the chip exits sleep, the CPU and the flash chip are powered on at the same time. CPU will run rom code (deepsleep) or ram code (lightsleep) first, and then load or execute code from flash.

Some flash chips need sufficient time to pass between power on and first read operation. By default, without any extra delay, this time is approximately 900us, although some flash chip types need more than that.

(!!! Please adjust this value according to the Data Sheet of SPI Flash used in your project.) In Flash Data Sheet, the parameters that define the Flash ready timing after power-up (minimum time from Vcc(min) to CS activeare) usually named tVSL in ELECTRICAL CHARACTERISTICS chapter, and the configuration value here should be: ESP_SLEEP_WAIT_FLASH_READY_EXTRA_DELAY = tVSL - 900

For esp32 and esp32s3, the default extra delay is set to 2000us. When optimizing startup time for applications which require it, this value may be reduced.

If you are seeing "flash read err, 1000" message printed to the console after deep sleep reset on esp32, or triggered RTC_WDT/LP_WDT after lightsleep wakeup, try increasing this value. (For esp32, the delay will be executed in both deep sleep and light sleep wake up flow. For chips after esp32, the delay will be executed only in light sleep flow, the delay controlled by the EFUSE_FLASH_TPUW in ROM will be executed in deepsleep wake up flow.)

Range:

- from 0 to 5000

Default value:

- 2000
- 0

CONFIG_ESP_SLEEP_CACHE_SAFE_ASSERTION

Check the cache safety of the sleep wakeup code in sleep process

Found in: Component config > Hardware Settings > Sleep Config

Enabling it will check the cache safety of the code before the flash power is ready after light sleep wakeup, and check PM_SLP_IRAM_OPT related code cache safety. This option is only for code quality inspection. Enabling it will increase the time overhead of entering and exiting sleep. It is not recommended to enable it in the release version.

Default value:

- No (disabled)

CONFIG_ESP_SLEEP_DEBUG

esp sleep debug

Found in: Component config > Hardware Settings > Sleep Config

Enable esp sleep debug.

Default value:

- No (disabled)

CONFIG_ESP_SLEEP_GPIO_ENABLE_INTERNAL_RESISTORS

Allow to enable internal pull-up/downs for the Deep-Sleep wakeup IOs

Found in: Component config > Hardware Settings > Sleep Config

When using rtc gpio wakeup source during deepsleep without external pull-up/downs, you may want to make use of the internal ones.

Default value:

- Yes (enabled)

CONFIG_ESP_SLEEP_EVENT_CALLBACKS

Enable registration of sleep event callbacks

Found in: Component config > Hardware Settings > Sleep Config

If enabled, it allows user to register sleep event callbacks. It is primarily designed for internal developers and customers can use PM_LIGHT_SLEEP_CALLBACKS as an alternative.

NOTE: These callbacks are executed from the IDLE task context hence you cannot have any blocking calls in your callbacks.

NOTE: Enabling these callbacks may change sleep duration calculations based on time spent in callback and hence it is highly recommended to keep them as short as possible.

Default value:

- No (disabled) if *CONFIG_FREERTOS_USE_TICKLESS_IDLE*

RTC Clock Config

Contains:

- *CONFIG_RTC_CLK_CAL_CYCLES*
- *CONFIG_RTC_CLK_SRC*

CONFIG_RTC_CLK_SRC

RTC clock source

Found in: Component config > Hardware Settings > RTC Clock Config

Choose which clock is used as RTC clock source.

Available options:

- Internal 136kHz RC oscillator (CONFIG_RTC_CLK_SRC_INT_RC)
- External 32kHz crystal (CONFIG_RTC_CLK_SRC_EXT_CRYS)
- External 32kHz oscillator at 32K_XP pin (CONFIG_RTC_CLK_SRC_EXT_OSC)
- Internal 17.5MHz oscillator, divided by 256 (CONFIG_RTC_CLK_SRC_INT_8MD256)

CONFIG_RTC_CLK_CAL_CYCLES

Number of cycles for RTC_SLOW_CLK calibration

Found in: Component config > Hardware Settings > RTC Clock Config

When the startup code initializes RTC_SLOW_CLK, it can perform calibration by comparing the RTC_SLOW_CLK frequency with main XTAL frequency. This option sets the number of RTC_SLOW_CLK cycles measured by the calibration routine. Higher numbers increase calibration precision, which may be important for applications which spend a lot of time in deep sleep. Lower numbers reduce startup time.

When this option is set to 0, clock calibration will not be performed at startup, and approximate clock frequencies will be assumed:

- 150000 Hz if internal RC oscillator is used as clock source. For this use value 1024.
- **32768 Hz if the 32k crystal oscillator is used. For this use value 3000 or more.** In case more value will help improve the definition of the launch of the crystal. If the crystal could not start, it will be switched to internal RC.

Range:
- from 0 to 8190 if *CONFIG_RTC_CLK_SRC_EXT_CRYS* || *CONFIG_RTC_CLK_SRC_EXT_OSC* || *CONFIG_RTC_CLK_SRC_INT_8MD256*
- from 0 to 32766

Default value:
- 3000 if *CONFIG_RTC_CLK_SRC_EXT_CRYS* || *CONFIG_RTC_CLK_SRC_EXT_OSC* || *CONFIG_RTC_CLK_SRC_INT_8MD256*
- 1024

Peripheral Control

Contains:

- *CONFIG_PERIPH_CTRL_FUNC_IN_IRAM*

CONFIG_PERIPH_CTRL_FUNC_IN_IRAM

Place peripheral control functions into IRAM

Found in: Component config > Hardware Settings > Peripheral Control

Place peripheral control functions (e.g. periph_module_reset) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context.

Default value:
- No (disabled)

ETM Configuration

Contains:

- *CONFIG_ETM_ENABLE_DEBUG_LOG*

CONFIG_ETM_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > Hardware Settings > ETM Configuration

whether to enable the debug log message for ETM core driver. Note that, this option only controls the ETM related driver log, won't affect other drivers.

Default value:

- No (disabled) if SOC_ETM_SUPPORTED

GDMA Configurations Contains:

- *CONFIG_GDMA_ENABLE_DEBUG_LOG*
- *CONFIG_GDMA_ISR_IRAM_SAFE*
- *CONFIG_GDMA_CTRL_FUNC_IN_IRAM*

CONFIG_GDMA_CTRL_FUNC_IN_IRAM

Place GDMA control functions in IRAM

Found in: Component config > Hardware Settings > GDMA Configurations

Place GDMA control functions (like start/stop/append/reset) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context.

Default value:

- No (disabled)

CONFIG_GDMA_ISR_IRAM_SAFE

GDMA ISR IRAM-Safe

Found in: Component config > Hardware Settings > GDMA Configurations

This will ensure the GDMA interrupt handler is IRAM-Safe, allow to avoid flash cache misses, and also be able to run whilst the cache is disabled. (e.g. SPI Flash write).

Default value:

- No (disabled)

CONFIG_GDMA_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > Hardware Settings > GDMA Configurations

Whether to enable the debug log message for GDMA driver. Note that, this option only controls the GDMA driver log, won't affect other drivers.

Default value:

- No (disabled)

DW_GDMA Configurations Contains:

- *CONFIG_DW_GDMA_ENABLE_DEBUG_LOG*

CONFIG_DW_GDMA_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > Hardware Settings > DW_GDMA Configurations

Whether to enable the debug log message for DW_GDMA driver. Note that, this option only controls the DW_GDMA driver log, won't affect other drivers.

Default value:

- No (disabled) if SOC_DW_GDMA_SUPPORTED

2D-DMA Configurations

Contains:

- *CONFIG_DMA2D_ISR_IRAM_SAFE*
- *CONFIG_DMA2D_OPERATION_FUNC_IN_IRAM*

CONFIG_DMA2D_OPERATION_FUNC_IN_IRAM

Place 2D-DMA operation functions into IRAM

Found in: Component config > Hardware Settings > 2D-DMA Configurations

Place 2D-DMA all operation functions, including control functions (e.g. start/stop/append/reset) and setter functions (e.g. connect/strategy/callback registration) into IRAM, so that these functions can be IRAM-safe and able to be called in the other IRAM interrupt context. It also helps optimizing the performance.

Default value:

- No (disabled) if SOC_DMA2D_SUPPORTED

CONFIG_DMA2D_ISR_IRAM_SAFE

2D-DMA ISR IRAM-Safe

Found in: Component config > Hardware Settings > 2D-DMA Configurations

This will ensure the 2D-DMA interrupt handler is IRAM-Safe, allow to avoid flash cache misses, and also be able to run whilst the cache is disabled. (e.g. SPI Flash write).

Default value:

- No (disabled) if SOC_DMA2D_SUPPORTED

Main XTAL Config

Contains:

- *CONFIG_XTAL_FREQ_SEL*

CONFIG_XTAL_FREQ_SEL

Main XTAL frequency

Found in: Component config > Hardware Settings > Main XTAL Config

This option selects the operating frequency of the XTAL (crystal) clock used to drive the ESP target. The selected value MUST reflect the frequency of the given hardware.

Note: The XTAL_FREQ_AUTO option allows the ESP target to automatically estimating XTAL clock's operating frequency. However, this feature is only supported on the ESP32. The ESP32 uses the internal 8MHZ as a reference when estimating. Due to the internal oscillator's frequency being temperature dependent, usage of the XTAL_FREQ_AUTO is not recommended in applications that operate in high ambient temperatures or use high-temperature qualified chips and modules.

Available options:

- 24 MHz (CONFIG_XTAL_FREQ_24)
- 26 MHz (CONFIG_XTAL_FREQ_26)
- 32 MHz (CONFIG_XTAL_FREQ_32)
- 40 MHz (CONFIG_XTAL_FREQ_40)
- 48 MHz (CONFIG_XTAL_FREQ_48)
- Autodetect (CONFIG_XTAL_FREQ_AUTO)

Crypto DPA Protection Contains:

- *CONFIG_ESP_CRYPTO_DPA_PROTECTION_AT_STARTUP*

CONFIG_ESP_CRYPTO_DPA_PROTECTION_AT_STARTUP

Enable crypto DPA protection at startup

Found in: Component config > Hardware Settings > Crypto DPA Protection

This config controls the DPA (Differential Power Analysis) protection knob for the crypto peripherals. DPA protection dynamically adjusts the clock frequency of the crypto peripheral. DPA protection helps to make it difficult to perform SCA attacks on the crypto peripherals. However, there is also associated performance impact based on the security level set. Please refer to the TRM for more details.

Default value:

- Yes (enabled) if SOC_CRYPTO_DPA_PROTECTION_SUPPORTED

CONFIG_ESP_CRYPTO_DPA_PROTECTION_LEVEL

DPA protection level

Found in: Component config > Hardware Settings > Crypto DPA Protection > CONFIG_ESP_CRYPTO_DPA_PROTECTION_AT_STARTUP

Configure the DPA protection security level

Available options:

- Security level low (CONFIG_ESP_CRYPTO_DPA_PROTECTION_LEVEL_LOW)
- Security level medium (CONFIG_ESP_CRYPTO_DPA_PROTECTION_LEVEL_MEDIUM)
- Security level high (CONFIG_ESP_CRYPTO_DPA_PROTECTION_LEVEL_HIGH)

LCD and Touch Panel Contains:

- *LCD Peripheral Configuration*

LCD Peripheral Configuration Contains:

- *CONFIG_LCD_DSI_ISR_IRAM_SAFE*
- *CONFIG_LCD_ENABLE_DEBUG_LOG*
- *CONFIG_LCD_PANEL_IO_FORMAT_BUF_SIZE*
- *CONFIG_LCD_RGB_RESTART_IN_VSYNC*
- *CONFIG_LCD_RGB_ISR_IRAM_SAFE*

CONFIG_LCD_PANEL_IO_FORMAT_BUF_SIZE

LCD panel io format buffer size

Found in: Component config > LCD and Touch Panel > LCD Peripheral Configuration

LCD driver allocates an internal buffer to transform the data into a proper format, because of the endian order mismatch. This option is to set the size of the buffer, in bytes.

Default value:

- 32

CONFIG_LCD_ENABLE_DEBUG_LOG

Enable debug log

Found in: Component config > LCD and Touch Panel > LCD Peripheral Configuration

whether to enable the debug log message for LCD driver. Note that, this option only controls the LCD driver log, won't affect other drivers.

Default value:

- No (disabled)

CONFIG_LCD_RGB_ISR_IRAM_SAFE

RGB LCD ISR IRAM-Safe

Found in: Component config > LCD and Touch Panel > LCD Peripheral Configuration

Ensure the LCD interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write). If you want the LCD driver to keep flushing the screen even when cache ops disabled, you can enable this option. Note, this will also increase the IRAM usage.

Default value:

- No (disabled)

CONFIG_LCD_RGB_RESTART_IN_VSYNC

Restart transmission in VSYNC

Found in: Component config > LCD and Touch Panel > LCD Peripheral Configuration

Reset the GDMA channel every VBlank to stop permanent desyncs from happening. Only need to enable it when in your application, the DMA can't deliver data as fast as the LCD consumes it.

Default value:

- No (disabled)

CONFIG_LCD_DSI_ISR_IRAM_SAFE

DSI LCD ISR IRAM-Safe

Found in: Component config > LCD and Touch Panel > LCD Peripheral Configuration

Ensure the LCD interrupt is IRAM-Safe by allowing the interrupt handler to be executable when the cache is disabled (e.g. SPI Flash write). If you want the LCD driver to keep flushing the screen even when cache ops disabled, you can enable this option. Note, this will also increase the IRAM usage.

Default value:

- No (disabled) if SOC_MIPI_DSI_SUPPORTED

ESP NETIF Adapter

Contains:

- *CONFIG_ESP_NETIF_BRIDGE_EN*
- *CONFIG_ESP_NETIF_L2_TAP*
- *CONFIG_ESP_NETIF_IP_LOST_TIMER_INTERVAL*
- *CONFIG_ESP_NETIF_USE_TCPIP_STACK_LIB*
- *CONFIG_ESP_NETIF_RECEIVE_REPORT_ERRORS*

CONFIG_ESP_NETIF_IP_LOST_TIMER_INTERVAL

IP Address lost timer interval (seconds)

Found in: Component config > ESP NETIF Adapter

The value of 0 indicates the IP lost timer is disabled, otherwise the timer is enabled.

The IP address may be lost because of some reasons, e.g. when the station disconnects from soft-AP, or when DHCP IP renew fails etc. If the IP lost timer is enabled, it will be started everytime the IP is lost. Event SYSTEM_EVENT_STA_LOST_IP will be raised if the timer expires. The IP lost timer is stopped if the station get the IP again before the timer expires.

Range:
- from 0 to 65535

Default value:
- 120

CONFIG_ESP_NETIF_USE_TCPIP_STACK_LIB

TCP/IP Stack Library

Found in: Component config > ESP NETIF Adapter

Choose the TCP/IP Stack to work, for example, LwIP, uIP, etc.

Available options:

- LwIP (CONFIG_ESP_NETIF_TCPIP_LWIP)
  lwIP is a small independent implementation of the TCP/IP protocol suite.
- Loopback (CONFIG_ESP_NETIF_LOOPBACK)
  Dummy implementation of esp-netif functionality which connects driver transmit to receive function. This option is for testing purpose only

CONFIG_ESP_NETIF_RECEIVE_REPORT_ERRORS

Use esp_err_t to report errors from esp_netif_receive

Found in: Component config > ESP NETIF Adapter

Enable if esp_netif_receive() should return error code. This is useful to inform upper layers that packet input to TCP/IP stack failed, so the upper layers could implement flow control. This option is disabled by default due to backward compatibility and will be enabled in v6.0 (IDF-7194)

Default value:
- No (disabled)

CONFIG_ESP_NETIF_L2_TAP

Enable netif L2 TAP support

Found in: Component config > ESP NETIF Adapter

A user program can read/write link layer (L2) frames from/to ESP TAP device. The ESP TAP device can be currently associated only with Ethernet physical interfaces.

CONFIG_ESP_NETIF_L2_TAP_MAX_FDS

Maximum number of opened L2 TAP File descriptors

Found in: Component config > ESP NETIF Adapter > CONFIG_ESP_NETIF_L2_TAP

Maximum number of opened File descriptors (FD's) associated with ESP TAP device. ESP TAP FD's take up a certain amount of memory, and allowing fewer FD's to be opened at the same time conserves memory.

Range:
- from 1 to 10 if *CONFIG_ESP_NETIF_L2_TAP*

Default value:
- 5 if *CONFIG_ESP_NETIF_L2_TAP*

CONFIG_ESP_NETIF_L2_TAP_RX_QUEUE_SIZE

Size of L2 TAP Rx queue

Found in: Component config > ESP NETIF Adapter > CONFIG_ESP_NETIF_L2_TAP

Maximum number of frames queued in opened File descriptor. Once the queue is full, the newly arriving frames are dropped until the queue has enough room to accept incoming traffic (Tail Drop queue management).

Range:
- from 1 to 100 if *CONFIG_ESP_NETIF_L2_TAP*

Default value:
- 20 if *CONFIG_ESP_NETIF_L2_TAP*

CONFIG_ESP_NETIF_BRIDGE_EN

Enable LwIP IEEE 802.1D bridge

Found in: Component config > ESP NETIF Adapter

Enable LwIP IEEE 802.1D bridge support in ESP-NETIF. Note that "Number of clients store data in netif" (LWIP_NUM_NETIF_CLIENT_DATA) option needs to be properly configured to be LwIP bridge avaiable!

Default value:
- No (disabled)

Partition API Configuration

PHY

Contains:

- *CONFIG_ESP_PHY_CALIBRATION_MODE*
- *CONFIG_ESP_PHY_PLL_TRACK_DEBUG*
- *CONFIG_ESP_PHY_ENABLE_CERT_TEST*
- *CONFIG_ESP_PHY_IMPROVE_RX_11B*
- *CONFIG_ESP_PHY_ENABLE_USB*
- *CONFIG_ESP_PHY_MAX_WIFI_TX_POWER*
- *CONFIG_ESP_PHY_MAC_BB_PD*
- *CONFIG_ESP_PHY_REDUCE_TX_POWER*
- *CONFIG_ESP_PHY_CALIBRATION_AND_DATA_STORAGE*
- *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*

CONFIG_ESP_PHY_CALIBRATION_AND_DATA_STORAGE

Store phy calibration data in NVS

Found in: Component config > PHY

If this option is enabled, NVS will be initialized and calibration data will be loaded from there. PHY calibration will be skipped on deep sleep wakeup. If calibration data is not found, full calibration will

be performed and stored in NVS. Normally, only partial calibration will be performed. If this option is disabled, full calibration will be performed.

If it's easy that your board calibrate bad data, choose 'n'. Two cases for example, you should choose 'n': 1.If your board is easy to be booted up with antenna disconnected. 2.Because of your board design, each time when you do calibration, the result are too unstable. If unsure, choose 'y'.

Default value:

- Yes (enabled)

CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION

Use a partition to store PHY init data

Found in: Component config > PHY

If enabled, PHY init data will be loaded from a partition. When using a custom partition table, make sure that PHY data partition is included (type: 'data', subtype: 'phy'). With default partition tables, this is done automatically. If PHY init data is stored in a partition, it has to be flashed there, otherwise runtime error will occur.

If this option is not enabled, PHY init data will be embedded into the application binary.

If unsure, choose 'n'.

Default value:

- No (disabled)

Contains:

- *CONFIG_ESP_PHY_DEFAULT_INIT_IF_INVALID*
- *CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN*

CONFIG_ESP_PHY_DEFAULT_INIT_IF_INVALID

Reset default PHY init data if invalid

Found in: Component config > PHY > CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION

If enabled, PHY init data will be restored to default if it cannot be verified successfully to avoid endless bootloops.

If unsure, choose 'n'.

Default value:

- No (disabled) if *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*

CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN

Support multiple PHY init data bin

Found in: Component config > PHY > CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION

If enabled, the corresponding PHY init data type can be automatically switched according to the country code. China's PHY init data bin is used by default. Can be modified by country information in API esp_wifi_set_country(). The priority of switching the PHY init data type is: 1. Country configured by API esp_wifi_set_country() and the parameter policy is WIFI_COUNTRY_POLICY_MANUAL. 2. Country notified by the connected AP. 3. Country configured by API esp_wifi_set_country() and the parameter policy is WIFI_COUNTRY_POLICY_AUTO.

Default value:

- No (disabled) if *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION* && *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*

CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN_EMBED

Support embedded multiple phy init data bin to app bin

Found in: Component config > PHY > CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION > CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN

If enabled, multiple phy init data bin will embedded into app bin If not enabled, multiple phy init data bin will still leave alone, and need to be flashed by users.

Default value:

- No (disabled) if *CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN* && *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*

CONFIG_ESP_PHY_INIT_DATA_ERROR

Terminate operation when PHY init data error

Found in: Component config > PHY > CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION > CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN

If enabled, when an error occurs while the PHY init data is updated, the program will terminate and restart. If not enabled, the PHY init data will not be updated when an error occurs.

Default value:

- No (disabled) if *CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN* && *CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*

CONFIG_ESP_PHY_MAX_WIFI_TX_POWER

Max WiFi TX power (dBm)

Found in: Component config > PHY

Set maximum transmit power for WiFi radio. Actual transmit power for high data rates may be lower than this setting.

Range:

- from 10 to 20

Default value:

- 20

CONFIG_ESP_PHY_MAC_BB_PD

Power down MAC and baseband of Wi-Fi and Bluetooth when PHY is disabled

Found in: Component config > PHY

If enabled, the MAC and baseband of Wi-Fi and Bluetooth will be powered down when PHY is disabled. Enabling this setting reduces power consumption by a small amount but increases RAM use by approximately 4 KB(Wi-Fi only), 2 KB(Bluetooth only) or 5.3 KB(Wi-Fi + Bluetooth).

Default value:

- No (disabled) if *CONFIG_FREERTOS_USE_TICKLESS_IDLE*

CONFIG_ESP_PHY_REDUCE_TX_POWER

Reduce PHY TX power when brownout reset

Found in: Component config > PHY

When brownout reset occurs, reduce PHY TX power to keep the code running.

Default value:

- No (disabled)

CONFIG_ESP_PHY_ENABLE_USB

Keep the USB PHY enabled when initializing WiFi

Found in: Component config > PHY

On some ESP targets, the USB PHY can interfere with WiFi thus lowering WiFi performance. As a result, on those affected ESP targets, the ESP PHY library's initialization will automatically disable the USB PHY to get best WiFi performance. This option controls whether or not the ESP PHY library will keep the USB PHY enabled on initialization.

Note: This option can be disabled to increase WiFi performance. However, disabling this option will also mean that the USB PHY cannot be used while WiFi is enabled.

Default value:

- Yes (enabled)
- No (disabled)

CONFIG_ESP_PHY_ENABLE_CERT_TEST

Enable RF certification test functions

Found in: Component config > PHY

If enabled, you can use RF certification test APIs.

Default value:

- No (disabled)

CONFIG_ESP_PHY_CALIBRATION_MODE

Calibration mode

Found in: Component config > PHY

Select PHY calibration mode. During RF initialization, the partial calibration method is used by default for RF calibration. Full calibration takes about 100ms more than partial calibration. If boot duration is not critical, it is suggested to use the full calibration method. No calibration method is only used when the device wakes up from deep sleep.

Available options:

- Calibration partial (CONFIG_ESP_PHY_RF_CAL_PARTIAL)
- Calibration none (CONFIG_ESP_PHY_RF_CAL_NONE)
- Calibration full (CONFIG_ESP_PHY_RF_CAL_FULL)

CONFIG_ESP_PHY_IMPROVE_RX_11B

Improve Wi-Fi receive 11b pkts

Found in: Component config > PHY

This is a workaround to improve Wi-Fi receive 11b pkts for some modules using AC-DC power supply with high interference, enable this option will sacrifice Wi-Fi OFDM receive performance. But to guarantee 11b receive performance serves as a bottom line in this case.

Default value:

- No (disabled) if SOC_PHY_IMPROVE_RX_11B

CONFIG_ESP_PHY_PLL_TRACK_DEBUG

Enable pll track logging

Found in: Component config > PHY

If enabled, there will be some logs while pll tracking

Default value:

- No (disabled)

Power Management Contains:

- *CONFIG_PM_LIGHTSLEEP_RTC_OSC_CAL_INTERVAL*
- *CONFIG_PM_SLP_DISABLE_GPIO*
- *CONFIG_PM_LIGHT_SLEEP_CALLBACKS*
- *CONFIG_PM_POWER_DOWN_CPU_IN_LIGHT_SLEEP*
- *CONFIG_PM_POWER_DOWN_PERIPHERAL_IN_LIGHT_SLEEP*
- *CONFIG_PM_SLP_IRAM_OPT*
- *CONFIG_PM_RTOS_IDLE_OPT*
- *CONFIG_PM_ENABLE*

CONFIG_PM_ENABLE

Support for power management

Found in: Component config > Power Management

If enabled, application is compiled with support for power management. This option has run-time overhead (increased interrupt latency, longer time to enter idle state), and it also reduces accuracy of RTOS ticks and timers used for timekeeping. Enable this option if application uses power management APIs.

Default value:

- No (disabled) if __DOXYGEN__

CONFIG_PM_DFS_INIT_AUTO

Enable dynamic frequency scaling (DFS) at startup

Found in: Component config > Power Management > CONFIG_PM_ENABLE

If enabled, startup code configures dynamic frequency scaling. Max CPU frequency is set to DEFAULT_CPU_FREQ_MHZ setting, min frequency is set to XTAL frequency. If disabled, DFS will not be active until the application configures it using esp_pm_configure function.

Default value:

- No (disabled) if *CONFIG_PM_ENABLE*

CONFIG_PM_PROFILING

Enable profiling counters for PM locks

Found in: Component config > Power Management > CONFIG_PM_ENABLE

If enabled, esp_pm_* functions will keep track of the amount of time each of the power management locks has been held, and esp_pm_dump_locks function will print this information. This feature can be used to analyze which locks are preventing the chip from going into a lower power state, and see what time the chip spends in each power saving mode. This feature does incur some run-time overhead, so should typically be disabled in production builds.

Default value:

- No (disabled) if *CONFIG_PM_ENABLE*

CONFIG_PM_TRACE

Enable debug tracing of PM using GPIOs

Found in: Component config > Power Management > CONFIG_PM_ENABLE

If enabled, some GPIOs will be used to signal events such as RTOS ticks, frequency switching, entry/exit from idle state. Refer to pm_trace.c file for the list of GPIOs. This feature is intended to be used when analyzing/debugging behavior of power management implementation, and should be kept disabled in applications.

Default value:

- No (disabled) if *CONFIG_PM_ENABLE*

CONFIG_PM_SLP_IRAM_OPT

Put lightsleep related codes in internal RAM

Found in: Component config > Power Management

If enabled, about 2.1KB of lightsleep related source code would be in IRAM and chip would sleep longer for 310us at 160MHz CPU frequency most each time. This feature is intended to be used when lower power consumption is needed while there is enough place in IRAM to place source code.

CONFIG_PM_RTOS_IDLE_OPT

Put RTOS IDLE related codes in internal RAM

Found in: Component config > Power Management

If enabled, about 180Bytes of RTOS_IDLE related source code would be in IRAM and chip would sleep longer for 20us at 160MHz CPU frequency most each time. This feature is intended to be used when lower power consumption is needed while there is enough place in IRAM to place source code.

CONFIG_PM_SLP_DISABLE_GPIO

Disable all GPIO when chip at sleep

Found in: Component config > Power Management

This feature is intended to disable all GPIO pins at automantic sleep to get a lower power mode. If enabled, chips will disable all GPIO pins at automantic sleep to reduce about 200~300 uA current. If you want to specifically use some pins normally as chip wakes when chip sleeps, you can call 'gpio_sleep_sel_dis' to disable this feature on those pins. You can also keep this feature on and call 'gpio_sleep_set_direction' and 'gpio_sleep_set_pull_mode' to have a different GPIO configuration at sleep. Warning: If you want to enable this option on ESP32, you should enable *GPIO_ESP32_SUPPORT_SWITCH_SLP_PULL* at first, otherwise you will not be able to switch pullup/pulldown mode.

CONFIG_PM_LIGHTSLEEP_RTC_OSC_CAL_INTERVAL

Calibrate the RTC_FAST/SLOW clock every N times of light sleep

Found in: Component config > Power Management

The value of this option determines the calibration interval of the RTC_FAST/SLOW clock during sleep when power management is enabled. When it is configured as N, the RTC_FAST/SLOW clock will be calibrated every N times of lightsleep. Decreasing this value will increase the time the chip is in the active state, thereby increasing the average power consumption of the chip. Increasing this value can reduce the average power consumption, but when the external environment changes drastically and the chip RTC_FAST/SLOW oscillator frequency drifts, it may cause system instability.

Range:

- from 1 to 128 if *CONFIG_PM_ENABLE*

Default value:

- 1 if *CONFIG_PM_ENABLE*

CONFIG_PM_POWER_DOWN_CPU_IN_LIGHT_SLEEP

Power down CPU in light sleep

Found in: Component config > Power Management

If enabled, the CPU will be powered down in light sleep, ESP chips supports saving and restoring CPU's running context before and after light sleep, the feature provides applications with seamless CPU powerdowned lightsleep without user awareness. But this will takes up some internal memory. On esp32c3 soc, enabling this option will consume 1.68 KB of internal RAM and will reduce sleep current consumption by about 100 uA. On esp32s3 soc, enabling this option will consume 8.58 KB of internal RAM and will reduce sleep current consumption by about 650 uA.

Default value:

- Yes (enabled)

CONFIG_PM_RESTORE_CACHE_TAGMEM_AFTER_LIGHT_SLEEP

Restore I/D-cache tag memory after power down CPU light sleep

Found in: Component config > Power Management > CONFIG_PM_POWER_DOWN_CPU_IN_LIGHT_SLEEP

Cache tag memory and CPU both belong to the CPU power domain. ESP chips supports saving and restoring Cache tag memory before and after sleep, this feature supports accesses to the external memory that was cached before sleep still be cached when the CPU wakes up from a powerdowned CPU lightsleep. This option controls the restore method for Cache tag memory in lightsleep. If this option is enabled, the I/D-cache tag memory will be backuped to the internal RAM before sleep and restored upon wakeup. Depending on the the cache configuration, if this option is enabled, it will consume up to 9 KB of internal RAM. If this option is disabled, all cached data won't be kept after sleep, the DCache will be writeback before sleep and invalid all cached data after sleep, all accesses to external memory(Flash/PSRAM) will be cache missed after waking up, resulting in performance degradation due to increased memory accesses latency.

Default value:

- Yes (enabled)

CONFIG_PM_POWER_DOWN_PERIPHERAL_IN_LIGHT_SLEEP

Power down Digital Peripheral in light sleep (EXPERIMENTAL)

Found in: Component config > Power Management

If enabled, digital peripherals will be powered down in light sleep, it will reduce sleep current consumption by about 100 uA. Chip will save/restore register context at sleep/wake time to keep the system running. Enabling this option will increase static RAM and heap usage, the actual cost depends on the peripherals you have initialized. In order to save/restore the context of the necessary hardware for FreeRTOS to run, it will need at least 4.55 KB free heap at sleep time. Otherwise sleep will not power down the peripherals.

Note1: Please use this option with caution, the current IDF does not support the retention of all peripherals. When the digital peripherals are powered off and a sleep and wake-up is completed, the peripherals that have not saved the running context are equivalent to performing a reset. !!! Please confirm the peripherals used in your application and their sleep retention support status before enabling this option, peripherals sleep retention driver support status is tracked in power_management.rst

Note2: When this option is enabled simultaneously with FREERTOS_USE_TICKLESS_IDLE, since the UART will be powered down, the uart FIFO will be flushed before sleep to avoid data loss, however, this has the potential to block the sleep process and cause the wakeup time to be skipped, which will

cause the tick of freertos to not be compensated correctly when returning from sleep and cause the system to crash. To avoid this, you can increase FREERTOS_IDLE_TIME_BEFORE_SLEEP threshold in menuconfig.

Default value:

- No (disabled) if SOC_PM_SUPPORT_TOP_PD && SOC_PAU_SUPPORTED

CONFIG_PM_LIGHT_SLEEP_CALLBACKS

Enable registration of pm light sleep callbacks

Found in: Component config > Power Management

If enabled, it allows user to register entry and exit callbacks which are called before and after entering auto light sleep.

NOTE: These callbacks are executed from the IDLE task context hence you cannot have any blocking calls in your callbacks.

NOTE: Enabling these callbacks may change sleep duration calculations based on time spent in callback and hence it is highly recommended to keep them as short as possible

Default value:

- No (disabled) if *CONFIG_FREERTOS_USE_TICKLESS_IDLE*

ESP PSRAM Contains:

- *CONFIG_SPIRAM*

CONFIG_SPIRAM

Support for external, SPI-connected RAM

Found in: Component config > ESP PSRAM

This enables support for an external SPI RAM chip, connected in parallel with the main SPI flash chip.

SPI RAM config Contains:

- *CONFIG_SPIRAM_ALLOW_BSS_SEG_EXTERNAL_MEMORY*
- *CONFIG_SPIRAM_ALLOW_STACK_EXTERNAL_MEMORY*
- *CONFIG_SPIRAM_XIP_FROM_PSRAM*
- *CONFIG_SPIRAM_ECC_ENABLE*
- *CONFIG_SPIRAM_BOOT_INIT*
- *CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL*
- *CONFIG_SPIRAM_MODE*
- *CONFIG_SPIRAM_FETCH_INSTRUCTIONS*
- *CONFIG_SPIRAM_RODATA*
- *CONFIG_SPIRAM_MALLOC_RESERVE_INTERNAL*
- *CONFIG_SPIRAM_MEMTEST*
- *CONFIG_SPIRAM_SPEED*
- *CONFIG_SPIRAM_USE*
- *CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP*
- *CONFIG_SPIRAM_TYPE*

CONFIG_SPIRAM_MODE

Mode (QUAD/OCT) of SPI RAM chip in use

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Available options:

- Quad Mode PSRAM (CONFIG_SPIRAM_MODE_QUAD)
- Octal Mode PSRAM (CONFIG_SPIRAM_MODE_OCT)

CONFIG_SPIRAM_TYPE

Type of SPIRAM chip in use

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Available options:

- Auto-detect (CONFIG_SPIRAM_TYPE_AUTO)
- ESP-PSRAM16 or APS1604 (CONFIG_SPIRAM_TYPE_ESPPSRAM16)
- ESP-PSRAM32 (CONFIG_SPIRAM_TYPE_ESPPSRAM32)
- ESP-PSRAM64 , LY68L6400 or APS6408 (CONFIG_SPIRAM_TYPE_ESPPSRAM64)

CONFIG_SPIRAM_ALLOW_STACK_EXTERNAL_MEMORY

Allow external memory as an argument to xTaskCreateStatic

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Accessing memory in SPIRAM has certain restrictions, so task stacks allocated by xTaskCreate are by default allocated from internal RAM.

This option allows for passing memory allocated from SPIRAM to be passed to xTaskCreateStatic. This should only be used for tasks where the stack is never accessed while the cache is disabled.

CONFIG_SPIRAM_XIP_FROM_PSRAM

Enable Executable in place from (XiP) from PSRAM feature

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Helper for selecting both *SPIRAM_FETCH_INSTRUCTIONS* and *SPIRAM_RODATA*

CONFIG_SPIRAM_FETCH_INSTRUCTIONS

Move Instructions in Flash to PSRAM

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

If enabled, instructions in flash will be moved into PSRAM on startup. If SPIRAM_RODATA is also enabled, code that requires execution during an MSPI1 Flash operation can forgo being placed in IRAM. Therefore codes that need to be executing during Flash operation can continue working normally.

This feature is useful for high throughput peripheral involved applications to improve the performance during MSPI1 flash operations. PSRAM access speed is faster than Flash access. So the performance is better. (see External RAM documentation for more details).

CONFIG_SPIRAM_RODATA

Move Read-Only Data in Flash to PSRAM

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

If enabled, rodata in flash will be moved into PSRAM on startup. If SPIRAM_FETCH_INSTRUCTIONS is also enabled, code that requires execution during an MSPI1 Flash operation is not necessary to be placed in IRAM. Therefore codes that need to be executing during Flash operation can continue working normally.

This feature is useful for high throughput peripheral involved applications to improve the performance during MSPI1 flash operations. PSRAM access speed is faster than Flash access. So the performance is better. (see External RAM documentation for more details).

CONFIG_SPIRAM_SPEED

Set RAM clock speed

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Select the speed for the SPI RAM chip.

Available options:

- 120MHz clock speed (READ DOCS FIRST) (CONFIG_SPIRAM_SPEED_120M)
  - Quad PSRAM 120 MHz is stable.
  - Octal PSRAM 120 MHz is an experimental feature, it works when the temperature is stable.

    Risks: If your chip powers on at a certain temperature, then after the temperature increases or decreases by approximately 20 Celsius degrees (depending on the chip), the accesses to / from PSRAM will crash randomly.
- 80MHz clock speed (CONFIG_SPIRAM_SPEED_80M)
- 40Mhz clock speed (CONFIG_SPIRAM_SPEED_40M)

CONFIG_SPIRAM_ECC_ENABLE

Enable SPI RAM ECC

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Enable MSPI Error-Correcting Code function when accessing SPIRAM.

If enabled, 1/16 of the SPI RAM total size will be reserved for error-correcting code.

CONFIG_SPIRAM_BOOT_INIT

Initialize SPI RAM during startup

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

If this is enabled, the SPI RAM will be enabled during initial boot. Unless you have specific requirements, you'll want to leave this enabled so memory allocated during boot-up can also be placed in SPI RAM.

CONFIG_SPIRAM_IGNORE_NOTFOUND

Ignore PSRAM when not found

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config > CONFIG_SPIRAM_BOOT_INIT

Normally, if psram initialization is enabled during compile time but not found at runtime, it is seen as an error making the CPU panic. If this is enabled, booting will complete but no PSRAM will be available. If PSRAM failed to initialize, the following configs may be affected and may need to be corrected manually. SPIRAM_TRY_ALLOCATE_WIFI_LWIP will affect some LWIP and WiFi buffer default values and range values. Enable SPIRAM_TRY_ALLOCATE_WIFI_LWIP, ESP_WIFI_AMSDU_TX_ENABLED, ESP_WIFI_CACHE_TX_BUFFER_NUM and use static WiFi Tx buffer may cause potential memory exhaustion issues. Suggest disable SPIRAM_TRY_ALLOCATE_WIFI_LWIP. Suggest disable ESP_WIFI_AMSDU_TX_ENABLED. Suggest disable ESP_WIFI_CACHE_TX_BUFFER_NUM, need clear CONFIG_FEATURE_CACHE_TX_BUF_BIT of config->feature_caps. Suggest change ESP_WIFI_TX_BUFFER from static to dynamic. Also suggest to adjust some buffer numbers

to the values used without PSRAM case. Such as, ESP_WIFI_STATIC_TX_BUFFER_NUM, ESP_WIFI_DYNAMIC_TX_BUFFER_NUM.

CONFIG_SPIRAM_USE

SPI RAM access method

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

The SPI RAM can be accessed in multiple methods: by just having it available as an unmanaged memory region in the CPU's memory map, by integrating it in the heap as 'special' memory needing heap_caps_malloc to allocate, or by fully integrating it making malloc() also able to return SPI RAM pointers.

Available options:

- Integrate RAM into memory map (CONFIG_SPIRAM_USE_MEMMAP)
- Make RAM allocatable using heap_caps_malloc(..., MALLOC_CAP_SPIRAM) (CONFIG_SPIRAM_USE_CAPS_ALLOC)
- Make RAM allocatable using malloc() as well (CONFIG_SPIRAM_USE_MALLOC)

CONFIG_SPIRAM_MEMTEST

Run memory test on SPI RAM initialization

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Runs a rudimentary memory test on initialization. Aborts when memory test fails. Disable this for slightly faster startup.

CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL

Maximum malloc() size, in bytes, to always put in internal memory

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

If malloc() is capable of also allocating SPI-connected ram, its allocation strategy will prefer to allocate chunks less than this size in internal memory, while allocations larger than this will be done from external RAM. If allocation from the preferred region fails, an attempt is made to allocate from the non-preferred region instead, so malloc() will not suddenly fail when either internal or external memory is full.

CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP

Try to allocate memories of WiFi and LWIP in SPIRAM firstly. If failed, allocate internal memory

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Try to allocate memories of WiFi and LWIP in SPIRAM firstly. If failed, try to allocate internal memory then.

CONFIG_SPIRAM_MALLOC_RESERVE_INTERNAL

Reserve this amount of bytes for data that specifically needs to be in DMA or internal memory

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

Because the external/internal RAM allocation strategy is not always perfect, it sometimes may happen that the internal memory is entirely filled up. This causes allocations that are specifically done in internal memory, for example the stack for new tasks or memory to service DMA or have memory that's also available when SPI cache is down, to fail. This option reserves a pool specifically for requests like that; the memory in this pool is not given out when a normal malloc() is called.

Set this to 0 to disable this feature.

Note that because FreeRTOS stacks are forced to internal memory, they will also use this memory pool; be sure to keep this in mind when adjusting this value.

Note also that the DMA reserved pool may not be one single contiguous memory region, depending on the configured size and the static memory usage of the app.

CONFIG_SPIRAM_ALLOW_BSS_SEG_EXTERNAL_MEMORY

Allow .bss segment placed in external memory

Found in: Component config > ESP PSRAM > CONFIG_SPIRAM > SPI RAM config

If enabled, variables with EXT_RAM_BSS_ATTR attribute will be placed in SPIRAM instead of internal DRAM. BSS section of *lwip*, *net80211*, *pp*, *bt* libraries will be automatically placed in SPIRAM. BSS sections from other object files and libraries can also be placed in SPIRAM through linker fragment scheme *extram_bss*.

Note that the variables placed in SPIRAM using EXT_RAM_BSS_ATTR will be zero initialized.

ESP Ringbuf

Contains:

- *CONFIG_RINGBUF_PLACE_FUNCTIONS_INTO_FLASH*

CONFIG_RINGBUF_PLACE_FUNCTIONS_INTO_FLASH

Place non-ISR ringbuf functions into flash

Found in: Component config > ESP Ringbuf

Place non-ISR ringbuf functions (like xRingbufferCreate/xRingbufferSend) into flash. This frees up IRAM, but the functions can no longer be called when the cache is disabled.

Default value:

- No (disabled)

CONFIG_RINGBUF_PLACE_ISR_FUNCTIONS_INTO_FLASH

Place ISR ringbuf functions into flash

Found in: Component config > ESP Ringbuf > CONFIG_RINGBUF_PLACE_FUNCTIONS_INTO_FLASH

Place ISR ringbuf functions (like xRingbufferSendFromISR/xRingbufferReceiveFromISR) into flash. This frees up IRAM, but the functions can no longer be called when the cache is disabled or from an IRAM interrupt context.

This option is not compatible with ESP-IDF drivers which are configured to run the ISR from an IRAM context, e.g. CONFIG_UART_ISR_IN_IRAM.

Default value:

- No (disabled) if *CONFIG_RINGBUF_PLACE_FUNCTIONS_INTO_FLASH*

ESP System Settings

Contains:

- *CONFIG_ESP_SYSTEM_RTC_EXT_XTAL_BOOTSTRAP_CYCLES*
- *Brownout Detector*
- *Cache config*
- *CONFIG_ESP_CONSOLE_UART*
- *CONFIG_ESP_CONSOLE_SECONDARY*
- *CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ*
- *CONFIG_ESP_CONSOLE_USB_CDC_SUPPORT_ETS_PRINTF*
- *CONFIG_ESP_SYSTEM_ALLOW_RTC_FAST_MEM_AS_HEAP*

- *CONFIG_ESP_TASK_WDT_EN*
- *CONFIG_ESP_SYSTEM_EVENT_TASK_STACK_SIZE*
- *CONFIG_ESP_SYSTEM_HW_PC_RECORD*
- *CONFIG_ESP_SYSTEM_HW_STACK_GUARD*
- *CONFIG_ESP_XT_WDT*
- *CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL*
- *CONFIG_ESP_INT_WDT*
- *CONFIG_ESP_MAIN_TASK_AFFINITY*
- *CONFIG_ESP_MAIN_TASK_STACK_SIZE*
- *CONFIG_ESP_DEBUG_OCDAWARE*
- *Memory*
- *Memory protection*
- *CONFIG_ESP_MINIMAL_SHARED_STACK_SIZE*
- *CONFIG_ESP_DEBUG_STUBS_ENABLE*
- *CONFIG_ESP_SYSTEM_PANIC*
- *CONFIG_ESP_SYSTEM_PANIC_REBOOT_DELAY_SECONDS*
- *CONFIG_ESP_PANIC_HANDLER_IRAM*
- *CONFIG_ESP_SYSTEM_BBPLL_RECALIB*
- *CONFIG_ESP_CONSOLE_USB_CDC_RX_BUF_SIZE*
- *CONFIG_ESP_SYSTEM_EVENT_QUEUE_SIZE*
- *Trace memory*
- *CONFIG_ESP_CONSOLE_UART_BAUDRATE*
- *CONFIG_ESP_CONSOLE_UART_NUM*
- *CONFIG_ESP_CONSOLE_UART_RX_GPIO*
- *CONFIG_ESP_CONSOLE_UART_TX_GPIO*

CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ

CPU frequency

Found in: Component config > ESP System Settings

CPU frequency to be set on application startup.

Available options:

- 40 MHz (CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_40)
- 80 MHz (CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_80)
- 160 MHz (CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_160)
- 240 MHz (CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_240)

Cache config Contains:

- *CONFIG_ESP32S3_DCACHE_ASSOCIATED_WAYS*
- *CONFIG_ESP32S3_DATA_CACHE_LINE_SIZE*
- *CONFIG_ESP32S3_DATA_CACHE_SIZE*
- *CONFIG_ESP32S3_ICACHE_ASSOCIATED_WAYS*
- *CONFIG_ESP32S3_INSTRUCTION_CACHE_LINE_SIZE*
- *CONFIG_ESP32S3_INSTRUCTION_CACHE_SIZE*

CONFIG_ESP32S3_INSTRUCTION_CACHE_SIZE

Instruction cache size

Found in: Component config > ESP System Settings > Cache config

Instruction cache size to be set on application startup. If you use 16KB instruction cache rather than 32KB instruction cache, then the other 16KB will be managed by heap allocator.

Available options:

- 16KB (CONFIG_ESP32S3_INSTRUCTION_CACHE_16KB)
- 32KB (CONFIG_ESP32S3_INSTRUCTION_CACHE_32KB)

CONFIG_ESP32S3_ICACHE_ASSOCIATED_WAYS

Instruction cache associated ways

Found in: Component config > ESP System Settings > Cache config

Instruction cache associated ways to be set on application startup.

Available options:

- 4 ways (CONFIG_ESP32S3_INSTRUCTION_CACHE_4WAYS)
- 8 ways (CONFIG_ESP32S3_INSTRUCTION_CACHE_8WAYS)

CONFIG_ESP32S3_INSTRUCTION_CACHE_LINE_SIZE

Instruction cache line size

Found in: Component config > ESP System Settings > Cache config

Instruction cache line size to be set on application startup.

Available options:

- 16 Bytes (CONFIG_ESP32S3_INSTRUCTION_CACHE_LINE_16B)
- 32 Bytes (CONFIG_ESP32S3_INSTRUCTION_CACHE_LINE_32B)

CONFIG_ESP32S3_DATA_CACHE_SIZE

Data cache size

Found in: Component config > ESP System Settings > Cache config

Data cache size to be set on application startup. If you use 32KB data cache rather than 64KB data cache, the other 32KB will be added to the heap.

Available options:

- 16KB (CONFIG_ESP32S3_DATA_CACHE_16KB)
- 32KB (CONFIG_ESP32S3_DATA_CACHE_32KB)
- 64KB (CONFIG_ESP32S3_DATA_CACHE_64KB)

CONFIG_ESP32S3_DCACHE_ASSOCIATED_WAYS

Data cache associated ways

Found in: Component config > ESP System Settings > Cache config

Data cache associated ways to be set on application startup.

Available options:

- 4 ways (CONFIG_ESP32S3_DATA_CACHE_4WAYS)
- 8 ways (CONFIG_ESP32S3_DATA_CACHE_8WAYS)

CONFIG_ESP32S3_DATA_CACHE_LINE_SIZE

Data cache line size

Found in: Component config > ESP System Settings > Cache config

Data cache line size to be set on application startup.

Available options:

- 16 Bytes (CONFIG_ESP32S3_DATA_CACHE_LINE_16B)
- 32 Bytes (CONFIG_ESP32S3_DATA_CACHE_LINE_32B)
- 64 Bytes (CONFIG_ESP32S3_DATA_CACHE_LINE_64B)

Memory Contains:

- *CONFIG_ESP32S3_RTCDATA_IN_FAST_MEM*
- *CONFIG_ESP32S3_USE_FIXED_STATIC_RAM_SIZE*

CONFIG_ESP32S3_RTCDATA_IN_FAST_MEM

Place RTC_DATA_ATTR and RTC_RODATA_ATTR variables into RTC fast memory segment

Found in: Component config > ESP System Settings > Memory

This option allows to place .rtc_data and .rtc_rodata sections into RTC fast memory segment to free the slow memory region for ULP programs.

Default value:

- No (disabled)

CONFIG_ESP32S3_USE_FIXED_STATIC_RAM_SIZE

Use fixed static RAM size

Found in: Component config > ESP System Settings > Memory

If this option is disabled, the DRAM part of the heap starts right after the .bss section, within the dram0_0 region. As a result, adding or removing some static variables will change the available heap size.

If this option is enabled, the DRAM part of the heap starts right after the dram0_0 region, where its length is set with ESP32S3_FIXED_STATIC_RAM_SIZE

Default value:

- No (disabled)

CONFIG_ESP32S3_FIXED_STATIC_RAM_SIZE

Fixed Static RAM size

Found in: Component config > ESP System Settings > Memory > CONFIG_ESP32S3_USE_FIXED_STATIC_RAM_SIZE

RAM size dedicated for static variables (.data & .bss sections). This value is less than the chips total memory, as not all of it can be used for this purpose. E.g. parts are used by the software bootloader, and will only be available as heap memory after app startup.

Range:

- from 0 to 0x54700 if *CONFIG_ESP32S3_USE_FIXED_STATIC_RAM_SIZE*

Default value:

- "0x10000" if *CONFIG_ESP32S3_USE_FIXED_STATIC_RAM_SIZE*

Trace memory Contains:

- *CONFIG_ESP32S3_TRAX*

CONFIG_ESP32S3_TRAX

Use TRAX tracing feature

Found in: Component config > ESP System Settings > Trace memory

The esp32-s3 contains a feature which allows you to trace the execution path the processor has taken through the program. This is stored in a chunk of 32K (16K for single-processor) of memory that can't be used for general purposes anymore. Disable this if you do not know what this is.

Default value:

- No (disabled)

CONFIG_ESP32S3_TRAX_TWOBANKS

Reserve memory for tracing both pro as well as app cpu execution

Found in: Component config > ESP System Settings > Trace memory > CONFIG_ESP32S3_TRAX

The esp32-s3 contains a feature which allows you to trace the execution path the processor has taken through the program. This is stored in a chunk of 32K (16K for single-processor) of memory that can't be used for general purposes anymore. Disable this if you do not know what this is.

Default value:

- No (disabled) if *CONFIG_ESP32S3_TRAX*

CONFIG_ESP_SYSTEM_PANIC

Panic handler behaviour

Found in: Component config > ESP System Settings

If FreeRTOS detects unexpected behaviour or an unhandled exception, the panic handler is invoked. Configure the panic handler's action here.

Available options:

- Print registers and halt (CONFIG_ESP_SYSTEM_PANIC_PRINT_HALT)
  Outputs the relevant registers over the serial port and halt the processor. Needs a manual reset to restart.
- Print registers and reboot (CONFIG_ESP_SYSTEM_PANIC_PRINT_REBOOT)
  Outputs the relevant registers over the serial port and immediately reset the processor.
- Silent reboot (CONFIG_ESP_SYSTEM_PANIC_SILENT_REBOOT)
  Just resets the processor without outputting anything
- GDBStub on panic (CONFIG_ESP_SYSTEM_PANIC_GDBSTUB)
  Invoke gdbstub on the serial port, allowing for gdb to attach to it to do a postmortem of the crash.

CONFIG_ESP_SYSTEM_PANIC_REBOOT_DELAY_SECONDS

Panic reboot delay (Seconds)

Found in: Component config > ESP System Settings

After the panic handler executes, you can specify a number of seconds to wait before the device reboots.

Range:

- from 0 to 99

Default value:

- 0

CONFIG_ESP_SYSTEM_RTC_EXT_XTAL_BOOTSTRAP_CYCLES

Bootstrap cycles for external 32kHz crystal

Found in: Component config > ESP System Settings

To reduce the startup time of an external RTC crystal, we bootstrap it with a 32kHz square wave for a fixed number of cycles. Setting 0 will disable bootstrapping (if disabled, the crystal may take longer to start up or fail to oscillate under some conditions).

If this value is too high, a faulty crystal may initially start and then fail. If this value is too low, an otherwise good crystal may not start.

To accurately determine if the crystal has started, set a larger "Number of cycles for RTC_SLOW_CLK calibration" (about 3000).

CONFIG_ESP_SYSTEM_ALLOW_RTC_FAST_MEM_AS_HEAP

Enable RTC fast memory for dynamic allocations

Found in: Component config > ESP System Settings

This config option allows to add RTC fast memory region to system heap with capability similar to that of DRAM region but without DMA. This memory will be consumed first per heap initialization order by early startup services and scheduler related code. Speed wise RTC fast memory operates on APB clock and hence does not have much performance impact.

Default value:

- Yes (enabled)

Memory protection Contains:

- *CONFIG_ESP_SYSTEM_PMP_IDRAM_SPLIT*
- *CONFIG_ESP_SYSTEM_MEMPROT_FEATURE*

CONFIG_ESP_SYSTEM_PMP_IDRAM_SPLIT

Enable IRAM/DRAM split protection

Found in: Component config > ESP System Settings > Memory protection

If enabled, the CPU watches all the memory access and raises an exception in case of any memory violation. This feature automatically splits the SRAM memory, using PMP, into data and instruction segments and sets Read/Execute permissions for the instruction part (below given splitting address) and Read/Write permissions for the data part (above the splitting address). The memory protection is effective on all access through the IRAM0 and DRAM0 buses.

Default value:

- Yes (enabled) if SOC_CPU_IDRAM_SPLIT_USING_PMP

CONFIG_ESP_SYSTEM_MEMPROT_FEATURE

Enable memory protection

Found in: Component config > ESP System Settings > Memory protection

If enabled, the permission control module watches all the memory access and fires the panic handler if a permission violation is detected. This feature automatically splits the SRAM memory into data and instruction segments and sets Read/Execute permissions for the instruction part (below given splitting address) and Read/Write permissions for the data part (above the splitting address). The memory protection is effective on all access through the IRAM0 and DRAM0 buses.

Default value:

- Yes (enabled)

CONFIG_ESP_SYSTEM_MEMPROT_FEATURE_LOCK

Lock memory protection settings

Found in: Component config > ESP System Settings > Memory protection > CONFIG_ESP_SYSTEM_MEMPROT_FEATURE

Once locked, memory protection settings cannot be changed anymore. The lock is reset only on the chip startup.

Default value:

- Yes (enabled)

CONFIG_ESP_SYSTEM_EVENT_QUEUE_SIZE

System event queue size

Found in: Component config > ESP System Settings

Config system event queue size in different application.

Default value:

- 32

CONFIG_ESP_SYSTEM_EVENT_TASK_STACK_SIZE

Event loop task stack size

Found in: Component config > ESP System Settings

Config system event task stack size in different application.

Default value:

- 2304

CONFIG_ESP_MAIN_TASK_STACK_SIZE

Main task stack size

Found in: Component config > ESP System Settings

Configure the "main task" stack size. This is the stack of the task which calls app_main(). If app_main() returns then this task is deleted and its stack memory is freed.

Default value:

- 3584

CONFIG_ESP_MAIN_TASK_AFFINITY

Main task core affinity

Found in: Component config > ESP System Settings

Configure the "main task" core affinity. This is the used core of the task which calls app_main(). If app_main() returns then this task is deleted.

Available options:

- CPU0 (CONFIG_ESP_MAIN_TASK_AFFINITY_CPU0)
- CPU1 (CONFIG_ESP_MAIN_TASK_AFFINITY_CPU1)
- No affinity (CONFIG_ESP_MAIN_TASK_AFFINITY_NO_AFFINITY)

CONFIG_ESP_MINIMAL_SHARED_STACK_SIZE

Minimal allowed size for shared stack

Found in: Component config > ESP System Settings

Minimal value of size, in bytes, accepted to execute a expression with shared stack.

Default value:

- 2048

CONFIG_ESP_CONSOLE_UART

Channel for console output

Found in: Component config > ESP System Settings

Select where to send console output (through stdout and stderr).

- Default is to use UART0 on pre-defined GPIOs.
- If "Custom" is selected, UART0 or UART1 can be chosen, and any pins can be selected.
- If "None" is selected, there will be no console output on any UART, except for initial output from ROM bootloader. This ROM output can be suppressed by GPIO strapping or EFUSE, refer to chip datasheet for details.
- On chips with USB OTG peripheral, "USB CDC" option redirects output to the CDC port. This option uses the CDC driver in the chip ROM. This option is incompatible with TinyUSB stack.
- On chips with an USB serial/JTAG debug controller, selecting the option for that redirects output to the CDC/ACM (serial port emulation) component of that device.

Available options:

- Default: UART0 (CONFIG_ESP_CONSOLE_UART_DEFAULT)
- USB CDC (CONFIG_ESP_CONSOLE_USB_CDC)
- USB Serial/JTAG Controller (CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG)
- Custom UART (CONFIG_ESP_CONSOLE_UART_CUSTOM)
- None (CONFIG_ESP_CONSOLE_NONE)

CONFIG_ESP_CONSOLE_SECONDARY

Channel for console secondary output

Found in: Component config > ESP System Settings

This secondary option supports output through other specific port like USB_SERIAL_JTAG when UART0 port as a primary is selected but not connected. This secondary output currently only supports non-blocking mode without using REPL. If you want to output in blocking mode with REPL or input through this secondary port, please change the primary config to this port in *Channel for console output* menu.

Available options:

- No secondary console (CONFIG_ESP_CONSOLE_SECONDARY_NONE)
- USB_SERIAL_JTAG PORT (CONFIG_ESP_CONSOLE_SECONDARY_USB_SERIAL_JTAG)
  This option supports output through USB_SERIAL_JTAG port when the UART0 port is not connected. The output currently only supports non-blocking mode without using the console. If you want to output in blocking mode with REPL or input through USB_SERIAL_JTAG port, please change the primary config to ESP_CONSOLE_USB_SERIAL_JTAG above.

CONFIG_ESP_CONSOLE_UART_NUM

UART peripheral to use for console output (0-1)

Found in: Component config > ESP System Settings

This UART peripheral is used for console output from the ESP-IDF Bootloader and the app.

If the configuration is different in the Bootloader binary compared to the app binary, UART is reconfigured after the bootloader exits and the app starts.

Due to an ESP32 ROM bug, UART2 is not supported for console output via esp_rom_printf.

Available options:

- UART0 (CONFIG_ESP_CONSOLE_UART_CUSTOM_NUM_0)
- UART1 (CONFIG_ESP_CONSOLE_UART_CUSTOM_NUM_1)

CONFIG_ESP_CONSOLE_UART_TX_GPIO

UART TX on GPIO#

Found in: Component config > ESP System Settings

This GPIO is used for console UART TX output in the ESP-IDF Bootloader and the app (including boot log output and default standard output and standard error of the app).

If the configuration is different in the Bootloader binary compared to the app binary, UART is reconfigured after the bootloader exits and the app starts.

Range:
- from 0 to 48 if *CONFIG_ESP_CONSOLE_UART_CUSTOM*

Default value:
- 43 if *CONFIG_ESP_CONSOLE_UART_CUSTOM*

CONFIG_ESP_CONSOLE_UART_RX_GPIO

UART RX on GPIO#

Found in: Component config > ESP System Settings

This GPIO is used for UART RX input in the ESP-IDF Bootloader and the app (including default default standard input of the app).

Note: The default ESP-IDF Bootloader configures this pin but doesn't read anything from the UART.

If the configuration is different in the Bootloader binary compared to the app binary, UART is reconfigured after the bootloader exits and the app starts.

Range:
- from 0 to 48 if *CONFIG_ESP_CONSOLE_UART_CUSTOM*

Default value:
- 44 if *CONFIG_ESP_CONSOLE_UART_CUSTOM*

CONFIG_ESP_CONSOLE_UART_BAUDRATE

UART console baud rate

Found in: Component config > ESP System Settings

This baud rate is used by both the ESP-IDF Bootloader and the app (including boot log output and default standard input/output/error of the app).

The app's maximum baud rate depends on the UART clock source. If Power Management is disabled, the UART clock source is the APB clock and all baud rates in the available range will be sufficiently

accurate. If Power Management is enabled, REF_TICK clock source is used so the baud rate is divided from 1MHz. Baud rates above 1Mbps are not possible and values between 500Kbps and 1Mbps may not be accurate.

If the configuration is different in the Bootloader binary compared to the app binary, UART is reconfigured after the bootloader exits and the app starts.

Range:
- from 1200 to 1000000 if *CONFIG_PM_ENABLE*

Default value:
- 115200

CONFIG_ESP_CONSOLE_USB_CDC_RX_BUF_SIZE

Size of USB CDC RX buffer

Found in: Component config > ESP System Settings

Set the size of USB CDC RX buffer. Increase the buffer size if your application is often receiving data over USB CDC.

Range:
- from 4 to 16384 if *CONFIG_ESP_CONSOLE_USB_CDC*

Default value:
- 64 if *CONFIG_ESP_CONSOLE_USB_CDC*

CONFIG_ESP_CONSOLE_USB_CDC_SUPPORT_ETS_PRINTF

Enable esp_rom_printf / ESP_EARLY_LOG via USB CDC

Found in: Component config > ESP System Settings

If enabled, esp_rom_printf and ESP_EARLY_LOG output will also be sent over USB CDC. Disabling this option saves about 1kB or RAM.

Default value:
- No (disabled) if *CONFIG_ESP_CONSOLE_USB_CDC*

CONFIG_ESP_INT_WDT

Interrupt watchdog

Found in: Component config > ESP System Settings

This watchdog timer can detect if the FreeRTOS tick interrupt has not been called for a certain time, either because a task turned off interrupts and did not turn them on for a long time, or because an interrupt handler did not return. It will try to invoke the panic handler first and failing that reset the SoC.

Default value:
- Yes (enabled)

CONFIG_ESP_INT_WDT_TIMEOUT_MS

Interrupt watchdog timeout (ms)

Found in: Component config > ESP System Settings > CONFIG_ESP_INT_WDT

The timeout of the watchdog, in milliseconds. Make this higher than the FreeRTOS tick rate.

Range:
- from 10 to 10000

Default value:
- 300

CONFIG_ESP_INT_WDT_CHECK_CPU1

Also watch CPU1 tick interrupt

Found in: Component config > ESP System Settings > CONFIG_ESP_INT_WDT

Also detect if interrupts on CPU 1 are disabled for too long.

CONFIG_ESP_TASK_WDT_EN

Enable Task Watchdog Timer

Found in: Component config > ESP System Settings

The Task Watchdog Timer can be used to make sure individual tasks are still running. Enabling this option will enable the Task Watchdog Timer. It can be either initialized automatically at startup or initialized after startup (see Task Watchdog Timer API Reference)

Default value:

- Yes (enabled)

CONFIG_ESP_TASK_WDT_INIT

Initialize Task Watchdog Timer on startup

Found in: Component config > ESP System Settings > CONFIG_ESP_TASK_WDT_EN

Enabling this option will cause the Task Watchdog Timer to be initialized automatically at startup.

Default value:

- Yes (enabled)

CONFIG_ESP_TASK_WDT_PANIC

Invoke panic handler on Task Watchdog timeout

Found in: Component config > ESP System Settings > CONFIG_ESP_TASK_WDT_EN > CONFIG_ESP_TASK_WDT_INIT

If this option is enabled, the Task Watchdog Timer will be configured to trigger the panic handler when it times out. This can also be configured at run time (see Task Watchdog Timer API Reference)

Default value:

- No (disabled)

CONFIG_ESP_TASK_WDT_TIMEOUT_S

Task Watchdog timeout period (seconds)

Found in: Component config > ESP System Settings > CONFIG_ESP_TASK_WDT_EN > CONFIG_ESP_TASK_WDT_INIT

Timeout period configuration for the Task Watchdog Timer in seconds. This is also configurable at run time (see Task Watchdog Timer API Reference)

Range:

- from 1 to 60

Default value:

- 5

CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU0

Watch CPU0 Idle Task

Found in: Component config > ESP System Settings > CONFIG_ESP_TASK_WDT_EN > CONFIG_ESP_TASK_WDT_INIT

If this option is enabled, the Task Watchdog Timer will watch the CPU0 Idle Task. Having the Task Watchdog watch the Idle Task allows for detection of CPU starvation as the Idle Task not being called is usually a symptom of CPU starvation. Starvation of the Idle Task is detrimental as FreeRTOS household tasks depend on the Idle Task getting some runtime every now and then.

Default value:

- Yes (enabled)

CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU1

Watch CPU1 Idle Task

Found in: Component config > ESP System Settings > CONFIG_ESP_TASK_WDT_EN > CONFIG_ESP_TASK_WDT_INIT

If this option is enabled, the Task Watchdog Timer will wach the CPU1 Idle Task.

CONFIG_ESP_XT_WDT

Initialize XTAL32K watchdog timer on startup

Found in: Component config > ESP System Settings

This watchdog timer can detect oscillation failure of the XTAL32K_CLK. When such a failure is detected the hardware can be set up to automatically switch to BACKUP32K_CLK and generate an interrupt.

CONFIG_ESP_XT_WDT_TIMEOUT

XTAL32K watchdog timeout period

Found in: Component config > ESP System Settings > CONFIG_ESP_XT_WDT

Timeout period configuration for the XTAL32K watchdog timer based on RTC_CLK.

Range:

- from 1 to 255 if *CONFIG_ESP_XT_WDT*

Default value:

- 200 if *CONFIG_ESP_XT_WDT*

CONFIG_ESP_XT_WDT_BACKUP_CLK_ENABLE

Automatically switch to BACKUP32K_CLK when timer expires

Found in: Component config > ESP System Settings > CONFIG_ESP_XT_WDT

Enable this to automatically switch to BACKUP32K_CLK as the source of RTC_SLOW_CLK when the watchdog timer expires.

Default value:

- Yes (enabled) if *CONFIG_ESP_XT_WDT*

CONFIG_ESP_PANIC_HANDLER_IRAM

Place panic handler code in IRAM

Found in: Component config > ESP System Settings

If this option is disabled (default), the panic handler code is placed in flash not IRAM. This means that if ESP-IDF crashes while flash cache is disabled, the panic handler will automatically re-enable flash cache before running GDB Stub or Core Dump. This adds some minor risk, if the flash cache status is also corrupted during the crash.

If this option is enabled, the panic handler code (including required UART functions) is placed in IRAM. This may be necessary to debug some complex issues with crashes while flash cache is disabled (for example, when writing to SPI flash) or when flash cache is corrupted when an exception is triggered.

Default value:

- No (disabled)

CONFIG_ESP_DEBUG_STUBS_ENABLE

OpenOCD debug stubs

Found in: Component config > ESP System Settings

Debug stubs are used by OpenOCD to execute pre-compiled onboard code which does some useful debugging stuff, e.g. GCOV data dump.

CONFIG_ESP_DEBUG_OCDAWARE

Make exception and panic handlers JTAG/OCD aware

Found in: Component config > ESP System Settings

The FreeRTOS panic and unhandled exception handers can detect a JTAG OCD debugger and instead of panicking, have the debugger stop on the offending instruction.

Default value:

- Yes (enabled)

CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL

Interrupt level to use for Interrupt Watchdog and other system checks

Found in: Component config > ESP System Settings

Interrupt level to use for Interrupt Watchdog, IPC_ISR and other system checks.

Available options:

- Level 5 interrupt (CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL_5)
  Using level 5 interrupt for Interrupt Watchdog, IPC_ISR and other system checks.
- Level 4 interrupt (CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL_4)
  Using level 4 interrupt for Interrupt Watchdog, IPC_ISR and other system checks.

Brownout Detector Contains:

- *CONFIG_ESP_BROWNOUT_DET*

CONFIG_ESP_BROWNOUT_DET

Hardware brownout detect & reset

Found in: Component config > ESP System Settings > Brownout Detector

The ESP32-S3 has a built-in brownout detector which can detect if the voltage is lower than a specific value. If this happens, it will reset the chip in order to prevent unintended behaviour.

Default value:

- Yes (enabled)

CONFIG_ESP_BROWNOUT_DET_LVL_SEL

Brownout voltage level

Found in: Component config > ESP System Settings > Brownout Detector > CONFIG_ESP_BROWNOUT_DET

The brownout detector will reset the chip when the supply voltage is approximately below this level. Note that there may be some variation of brownout voltage level between each ESP3-S3 chip.

#The voltage levels here are estimates, more work needs to be done to figure out the exact voltages #of the brownout threshold levels.

Available options:

- 2.44V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_7)
- 2.56V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_6)
- 2.67V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_5)
- 2.84V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_4)
- 2.98V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_3)
- 3.19V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_2)
- 3.30V (CONFIG_ESP_BROWNOUT_DET_LVL_SEL_1)

CONFIG_ESP_SYSTEM_HW_STACK_GUARD

Hardware stack guard

Found in: Component config > ESP System Settings

This config allows to trigger a panic interrupt when Stack Pointer register goes out of allocated stack memory bounds.

Default value:

- Yes (enabled) if SOC_ASSIST_DEBUG_SUPPORTED

CONFIG_ESP_SYSTEM_BBPLL_RECALIB

Re-calibration BBPLL at startup

Found in: Component config > ESP System Settings

This configuration helps to address an BBPLL inaccurate issue when boot from certain bootloader version, which may increase about the boot-up time by about 200 us. Disable this when your bootloader is built with ESP-IDF version v5.2 and above.

Default value:

- Yes (enabled)

CONFIG_ESP_SYSTEM_HW_PC_RECORD

Hardware PC recording

Found in: Component config > ESP System Settings

This option will enable the PC recording function of assist_debug module. The PC value of the CPU will be recorded to PC record register in assist_debug module in real time. When an exception occurs and the CPU is reset, this register will be kept, then we can use the recorded PC to debug the causes of the reset.

Default value:

- Yes (enabled) if SOC_ASSIST_DEBUG_SUPPORTED

IPC (Inter-Processor Call)

Contains:

- *CONFIG_ESP_IPC_TASK_STACK_SIZE*
- *CONFIG_ESP_IPC_USES_CALLERS_PRIORITY*

CONFIG_ESP_IPC_TASK_STACK_SIZE

Inter-Processor Call (IPC) task stack size

Found in: Component config > IPC (Inter-Processor Call)

Configure the IPC tasks stack size. An IPC task runs on each core (in dual core mode), and allows for cross-core function calls. See IPC documentation for more details. The default IPC stack size should be enough for most common simple use cases. However, users can increase/decrease the stack size to their needs.

Range:

- from 512 to 65536

Default value:

- 1280
- 1024

CONFIG_ESP_IPC_USES_CALLERS_PRIORITY

IPC runs at caller's priority

Found in: Component config > IPC (Inter-Processor Call)

If this option is not enabled then the IPC task will keep behavior same as prior to that of ESP-IDF v4.0, hence IPC task will run at (configMAX_PRIORITIES - 1) priority.

ESP Timer (High Resolution Timer)

Contains:

- *CONFIG_ESP_TIMER_PROFILING*
- *CONFIG_ESP_TIMER_TASK_AFFINITY*
- *CONFIG_ESP_TIMER_TASK_STACK_SIZE*
- *CONFIG_ESP_TIMER_INTERRUPT_LEVEL*
- *CONFIG_ESP_TIMER_SHOW_EXPERIMENTAL*
- *CONFIG_ESP_TIMER_SUPPORTS_ISR_DISPATCH_METHOD*
- *CONFIG_ESP_TIMER_ISR_AFFINITY*

CONFIG_ESP_TIMER_PROFILING

Enable esp_timer profiling features

Found in: Component config > ESP Timer (High Resolution Timer)

If enabled, esp_timer_dump will dump information such as number of times the timer was started, number of times the timer has triggered, and the total time it took for the callback to run. This option

has some effect on timer performance and the amount of memory used for timer storage, and should only be used for debugging/testing purposes.

Default value:

- No (disabled)

CONFIG_ESP_TIMER_TASK_STACK_SIZE

High-resolution timer task stack size

Found in: Component config > ESP Timer (High Resolution Timer)

Configure the stack size of "timer_task" task. This task is used to dispatch callbacks of timers created using ets_timer and esp_timer APIs. If you are seing stack overflow errors in timer task, increase this value.

Note that this is not the same as FreeRTOS timer task. To configure FreeRTOS timer task size, see "FreeRTOS timer task stack size" option in "FreeRTOS".

Range:

- from 2048 to 65536

Default value:

- 3584

CONFIG_ESP_TIMER_INTERRUPT_LEVEL

Interrupt level

Found in: Component config > ESP Timer (High Resolution Timer)

This sets the interrupt priority level for esp_timer ISR. A higher value reduces interrupt latency by minimizing the timer processing delay.

Range:

- from 1 to 1

Default value:

- 1

CONFIG_ESP_TIMER_SHOW_EXPERIMENTAL

show esp_timer's experimental features

Found in: Component config > ESP Timer (High Resolution Timer)

This shows some hidden features of esp_timer. Note that they may break other features, use them with care.

CONFIG_ESP_TIMER_TASK_AFFINITY

esp_timer task core affinity

Found in: Component config > ESP Timer (High Resolution Timer)

The default settings: timer TASK on CPU0 and timer ISR on CPU0. Other settings may help in certain cases, but note that they may break other features, use them with care. - "CPU0": (default) esp_timer task is processed by CPU0. - "CPU1": esp_timer task is processed by CPU1. - "No affinity": esp_timer task can be processed by any CPU.

Available options:

- CPU0 (CONFIG_ESP_TIMER_TASK_AFFINITY_CPU0)
- CPU1 (CONFIG_ESP_TIMER_TASK_AFFINITY_CPU1)
- No affinity (CONFIG_ESP_TIMER_TASK_AFFINITY_NO_AFFINITY)

CONFIG_ESP_TIMER_ISR_AFFINITY

timer interrupt core affinity

Found in: Component config > ESP Timer (High Resolution Timer)

The default settings: timer TASK on CPU0 and timer ISR on CPU0. Other settings may help in certain cases, but note that they may break other features, use them with care. - "CPU0": (default) timer interrupt is processed by CPU0. - "CPU1": timer interrupt is processed by CPU1. - "No affinity": timer interrupt can be processed by any CPU. It helps to reduce latency but there is a disadvantage it leads to the timer ISR running on every core. It increases the CPU time usage for timer ISRs by N on an N-core system.

Available options:

- CPU0 (CONFIG_ESP_TIMER_ISR_AFFINITY_CPU0)
- CPU1 (CONFIG_ESP_TIMER_ISR_AFFINITY_CPU1)
- No affinity (CONFIG_ESP_TIMER_ISR_AFFINITY_NO_AFFINITY)

CONFIG_ESP_TIMER_SUPPORTS_ISR_DISPATCH_METHOD

Support ISR dispatch method

Found in: Component config > ESP Timer (High Resolution Timer)

Allows using ESP_TIMER_ISR dispatch method (ESP_TIMER_TASK dispatch method is also available). - ESP_TIMER_TASK - Timer callbacks are dispatched from a high-priority esp_timer task. - ESP_TIMER_ISR - Timer callbacks are dispatched directly from the timer interrupt handler. The ISR dispatch can be used, in some cases, when a callback is very simple or need a lower-latency.

Default value:

- No (disabled)

Wi-Fi

Contains:

- *CONFIG_ESP_WIFI_TESTING_OPTIONS*
- *CONFIG_ESP_WIFI_WPS_SOFTAP_REGISTRAR*
- *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*
- *CONFIG_ESP_WIFI_11KV_SUPPORT*
- *CONFIG_ESP_WIFI_11R_SUPPORT*
- *CONFIG_ESP_WIFI_DPP_SUPPORT*
- *CONFIG_ESP_WIFI_ENTERPRISE_SUPPORT*
- *CONFIG_ESP_WIFI_MBO_SUPPORT*
- *CONFIG_ESP_WIFI_SUITE_B_192*
- *CONFIG_ESP_WIFI_ENABLE_WPA3_OWE_STA*
- *CONFIG_ESP_WIFI_WAPI_PSK*
- *CONFIG_ESP_WIFI_ENABLE_DUMP_CTRL_BFRP*
- *CONFIG_ESP_WIFI_ENABLE_DUMP_HESIGB*
- *CONFIG_ESP_WIFI_ENABLE_DUMP_MU_CFO*
- *CONFIG_ESP_WIFI_ENABLE_DUMP_CTRL_NDPA*
- *CONFIG_ESP_WIFI_ENABLE_WIFI_RX_STATS*
- *CONFIG_ESP_WIFI_ENABLE_WIFI_TX_STATS*
- *CONFIG_ESP_WIFI_ENABLE_WPA3_SAE*
- *CONFIG_ESP_HOST_WIFI_ENABLED*
- *CONFIG_ESP_WIFI_SOFTAP_BEACON_MAX_LEN*
- *CONFIG_ESP_WIFI_CACHE_TX_BUFFER_NUM*
- *CONFIG_ESP_WIFI_DYNAMIC_RX_BUFFER_NUM*
- *CONFIG_ESP_WIFI_DYNAMIC_TX_BUFFER_NUM*
- *CONFIG_ESP_WIFI_RX_MGMT_BUF_NUM_DEF*
- *CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM*

- *CONFIG_ESP_WIFI_STATIC_TX_BUFFER_NUM*
- *CONFIG_ESP_WIFI_ESPNOW_MAX_ENCRYPT_NUM*
- *CONFIG_ESP_WIFI_SLP_DEFAULT_MAX_ACTIVE_TIME*
- *CONFIG_ESP_WIFI_SLP_DEFAULT_MIN_ACTIVE_TIME*
- *CONFIG_ESP_WIFI_SLP_DEFAULT_WAIT_BROADCAST_DATA_TIME*
- *CONFIG_ESP_WIFI_STA_DISCONNECTED_PM_ENABLE*
- *CONFIG_ESP_WIFI_DEBUG_PRINT*
- *CONFIG_ESP_WIFI_MGMT_RX_BUFFER*
- *CONFIG_ESP_WIFI_TX_BUFFER*
- *CONFIG_ESP_WIFI_MBEDTLS_CRYPTO*
- *CONFIG_ESP_WIFI_AMPDU_RX_ENABLED*
- *CONFIG_ESP_WIFI_AMPDU_TX_ENABLED*
- *CONFIG_ESP_WIFI_AMSDU_TX_ENABLED*
- *CONFIG_ESP_WIFI_NAN_ENABLE*
- *CONFIG_ESP_WIFI_CSI_ENABLED*
- *CONFIG_ESP_WIFI_EXTRA_IRAM_OPT*
- *CONFIG_ESP_WIFI_FTM_ENABLE*
- *CONFIG_ESP_WIFI_GCMP_SUPPORT*
- *CONFIG_ESP_WIFI_GMAC_SUPPORT*
- *CONFIG_ESP_WIFI_IRAM_OPT*
- *CONFIG_ESP_WIFI_MGMT_SBUF_NUM*
- *CONFIG_ESP_WIFI_ENHANCED_LIGHT_SLEEP*
- *CONFIG_ESP_WIFI_NVS_ENABLED*
- *CONFIG_ESP_WIFI_RX_IRAM_OPT*
- *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*
- *CONFIG_ESP_WIFI_SLP_IRAM_OPT*
- *CONFIG_ESP_WIFI_SOFTAP_SUPPORT*
- *CONFIG_ESP_WIFI_TASK_CORE_ID*
- *CONFIG_ESP_WIFI_TX_HETB_QUEUE_NUM*
- *WPS Configuration Options*

CONFIG_ESP_HOST_WIFI_ENABLED

Host WiFi Enable

Found in: Component config > Wi-Fi

Default value:

- No (disabled) if SOC_WIRELESS_HOST_SUPPORTED

CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM

Max number of WiFi static RX buffers

Found in: Component config > Wi-Fi

Set the number of WiFi static RX buffers. Each buffer takes approximately 1.6KB of RAM. The static rx buffers are allocated when esp_wifi_init is called, they are not freed until esp_wifi_deinit is called.

WiFi hardware use these buffers to receive all 802.11 frames. A higher number may allow higher throughput but increases memory use. If ESP_WIFI_AMPDU_RX_ENABLED is enabled, this value is recommended to set equal or bigger than ESP_WIFI_RX_BA_WIN in order to achieve better throughput and compatibility with both stations and APs.

Range:

- from 2 to 128 if SOC_WIFI_HE_SUPPORT

Default value:

- 16 if *CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP*

CONFIG_ESP_WIFI_DYNAMIC_RX_BUFFER_NUM

Max number of WiFi dynamic RX buffers

Found in: Component config > Wi-Fi

Set the number of WiFi dynamic RX buffers, 0 means unlimited RX buffers will be allocated (provided sufficient free RAM). The size of each dynamic RX buffer depends on the size of the received data frame.

For each received data frame, the WiFi driver makes a copy to an RX buffer and then delivers it to the high layer TCP/IP stack. The dynamic RX buffer is freed after the higher layer has successfully received the data frame.

For some applications, WiFi data frames may be received faster than the application can process them. In these cases we may run out of memory if RX buffer number is unlimited (0).

If a dynamic RX buffer limit is set, it should be at least the number of static RX buffers.

Range:

- from 0 to 1024 if *CONFIG_LWIP_WND_SCALE*

Default value:

- 32

CONFIG_ESP_WIFI_TX_BUFFER

Type of WiFi TX buffers

Found in: Component config > Wi-Fi

Select type of WiFi TX buffers:

If "Static" is selected, WiFi TX buffers are allocated when WiFi is initialized and released when WiFi is de-initialized. The size of each static TX buffer is fixed to about 1.6KB.

If "Dynamic" is selected, each WiFi TX buffer is allocated as needed when a data frame is delivered to the Wifi driver from the TCP/IP stack. The buffer is freed after the data frame has been sent by the WiFi driver. The size of each dynamic TX buffer depends on the length of each data frame sent by the TCP/IP layer.

If PSRAM is enabled, "Static" should be selected to guarantee enough WiFi TX buffers. If PSRAM is disabled, "Dynamic" should be selected to improve the utilization of RAM.

Available options:

- Static (CONFIG_ESP_WIFI_STATIC_TX_BUFFER)
- Dynamic (CONFIG_ESP_WIFI_DYNAMIC_TX_BUFFER)

CONFIG_ESP_WIFI_STATIC_TX_BUFFER_NUM

Max number of WiFi static TX buffers

Found in: Component config > Wi-Fi

Set the number of WiFi static TX buffers. Each buffer takes approximately 1.6KB of RAM. The static RX buffers are allocated when esp_wifi_init() is called, they are not released until esp_wifi_deinit() is called.

For each transmitted data frame from the higher layer TCP/IP stack, the WiFi driver makes a copy of it in a TX buffer. For some applications especially UDP applications, the upper layer can deliver frames faster than WiFi layer can transmit. In these cases, we may run out of TX buffers.

Range:

- from 1 to 64 if *CONFIG_ESP_WIFI_STATIC_TX_BUFFER*

Default value:

- 16 if *CONFIG_ESP_WIFI_STATIC_TX_BUFFER*

CONFIG_ESP_WIFI_CACHE_TX_BUFFER_NUM

Max number of WiFi cache TX buffers

Found in: Component config > Wi-Fi

Set the number of WiFi cache TX buffer number.

For each TX packet from uplayer, such as LWIP etc, WiFi driver needs to allocate a static TX buffer and makes a copy of uplayer packet. If WiFi driver fails to allocate the static TX buffer, it caches the uplayer packets to a dedicated buffer queue, this option is used to configure the size of the cached TX queue.

Range:
- from 16 to 128 if *CONFIG_SPIRAM*

Default value:
- 32 if *CONFIG_SPIRAM*

CONFIG_ESP_WIFI_DYNAMIC_TX_BUFFER_NUM

Max number of WiFi dynamic TX buffers

Found in: Component config > Wi-Fi

Set the number of WiFi dynamic TX buffers. The size of each dynamic TX buffer is not fixed, it depends on the size of each transmitted data frame.

For each transmitted frame from the higher layer TCP/IP stack, the WiFi driver makes a copy of it in a TX buffer. For some applications, especially UDP applications, the upper layer can deliver frames faster than WiFi layer can transmit. In these cases, we may run out of TX buffers.

Range:
- from 1 to 128

Default value:
- 32

CONFIG_ESP_WIFI_MGMT_RX_BUFFER

Type of WiFi RX MGMT buffers

Found in: Component config > Wi-Fi

Select type of WiFi RX MGMT buffers:

If "Static" is selected, WiFi RX MGMT buffers are allocated when WiFi is initialized and released when WiFi is de-initialized. The size of each static RX MGMT buffer is fixed to about 500 Bytes.

If "Dynamic" is selected, each WiFi RX MGMT buffer is allocated as needed when a MGMT data frame is received. The MGMT buffer is freed after the MGMT data frame has been processed by the WiFi driver.

Available options:

- Static (CONFIG_ESP_WIFI_STATIC_RX_MGMT_BUFFER)
- Dynamic (CONFIG_ESP_WIFI_DYNAMIC_RX_MGMT_BUFFER)

CONFIG_ESP_WIFI_RX_MGMT_BUF_NUM_DEF

Max number of WiFi RX MGMT buffers

Found in: Component config > Wi-Fi

Set the number of WiFi RX_MGMT buffers.

For Management buffers, the number of dynamic and static management buffers is the same. In order to prevent memory fragmentation, the management buffer type should be set to static first.

Range:
- from 1 to 10

Default value:
- 5

CONFIG_ESP_WIFI_CSI_ENABLED

WiFi CSI(Channel State Information)

Found in: Component config > Wi-Fi

Select this option to enable CSI(Channel State Information) feature. CSI takes about CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM KB of RAM. If CSI is not used, it is better to disable this feature in order to save memory.

Default value:
- No (disabled)

CONFIG_ESP_WIFI_AMPDU_TX_ENABLED

WiFi AMPDU TX

Found in: Component config > Wi-Fi

Select this option to enable AMPDU TX feature

Default value:
- Yes (enabled)

CONFIG_ESP_WIFI_TX_BA_WIN

WiFi AMPDU TX BA window size

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_AMPDU_TX_ENABLED

Set the size of WiFi Block Ack TX window. Generally a bigger value means higher throughput but more memory. Most of time we should NOT change the default value unless special reason, e.g. test the maximum UDP TX throughput with iperf etc. For iperf test in shieldbox, the recommended value is 9~12.

Range:
- from 2 to 64 if SOC_WIFI_HE_SUPPORT && *CONFIG_ESP_WIFI_AMPDU_TX_ENABLED*

Default value:
- 6

CONFIG_ESP_WIFI_AMPDU_RX_ENABLED

WiFi AMPDU RX

Found in: Component config > Wi-Fi

Select this option to enable AMPDU RX feature

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_RX_BA_WIN

WiFi AMPDU RX BA window size

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_AMPDU_RX_ENABLED

Set the size of WiFi Block Ack RX window. Generally a bigger value means higher throughput and better compatibility but more memory. Most of time we should NOT change the default value unless special reason, e.g. test the maximum UDP RX throughput with iperf etc. For iperf test in shieldbox, the recommended value is 9~12. If PSRAM is used and WiFi memory is preferred to allocate in PSRAM first, the default and minimum value should be 16 to achieve better throughput and compatibility with both stations and APs.

Range:

- from 2 to 64 if SOC_WIFI_HE_SUPPORT && *CONFIG_ESP_WIFI_AMPDU_RX_ENABLED*

Default value:

- 16 if *CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP* && *CONFIG_ESP_WIFI_AMPDU_RX_ENABLED*

CONFIG_ESP_WIFI_AMSDU_TX_ENABLED

WiFi AMSDU TX

Found in: Component config > Wi-Fi

Select this option to enable AMSDU TX feature

Default value:

- No (disabled) if *CONFIG_SPIRAM*

CONFIG_ESP_WIFI_NVS_ENABLED

WiFi NVS flash

Found in: Component config > Wi-Fi

Select this option to enable WiFi NVS flash

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_TASK_CORE_ID

WiFi Task Core ID

Found in: Component config > Wi-Fi

Pinned WiFi task to core 0 or core 1.

Available options:

- Core 0 (CONFIG_ESP_WIFI_TASK_PINNED_TO_CORE_0)
- Core 1 (CONFIG_ESP_WIFI_TASK_PINNED_TO_CORE_1)

CONFIG_ESP_WIFI_SOFTAP_BEACON_MAX_LEN

Max length of WiFi SoftAP Beacon

Found in: Component config > Wi-Fi

ESP-MESH utilizes beacon frames to detect and resolve root node conflicts (see documentation). However the default length of a beacon frame can simultaneously hold only five root node identifier structures, meaning that a root node conflict of up to five nodes can be detected at one time. In the occurrence of more root nodes conflict involving more than five root nodes, the conflict resolution process will detect five of the root nodes, resolve the conflict, and re-detect more root nodes. This process will repeat until all root node conflicts are resolved. However this process can generally take a very long time.

To counter this situation, the beacon frame length can be increased such that more root nodes can be detected simultaneously. Each additional root node will require 36 bytes and should be added on top of the default beacon frame length of 752 bytes. For example, if you want to detect 10 root nodes simultaneously, you need to set the beacon frame length as 932 (752+36*5).

Setting a longer beacon length also assists with debugging as the conflicting root nodes can be identified more quickly.

Range:
- from 752 to 1256

Default value:
- 752

CONFIG_ESP_WIFI_MGMT_SBUF_NUM

WiFi mgmt short buffer number

Found in: Component config > Wi-Fi

Set the number of WiFi management short buffer.

Range:
- from 6 to 32

Default value:
- 32

CONFIG_ESP_WIFI_IRAM_OPT

WiFi IRAM speed optimization

Found in: Component config > Wi-Fi

Select this option to place frequently called Wi-Fi library functions in IRAM. When this option is disabled, more than 10Kbytes of IRAM memory will be saved but Wi-Fi throughput will be reduced.

Default value:
- Yes (enabled)

CONFIG_ESP_WIFI_EXTRA_IRAM_OPT

WiFi EXTRA IRAM speed optimization

Found in: Component config > Wi-Fi

Select this option to place additional frequently called Wi-Fi library functions in IRAM. When this option is disabled, more than 5Kbytes of IRAM memory will be saved but Wi-Fi throughput will be reduced.

Default value:
- Yes (enabled) if SOC_WIFI_HE_SUPPORT
- No (disabled)

CONFIG_ESP_WIFI_RX_IRAM_OPT

WiFi RX IRAM speed optimization

Found in: Component config > Wi-Fi

Select this option to place frequently called Wi-Fi library RX functions in IRAM. When this option is disabled, more than 17Kbytes of IRAM memory will be saved but Wi-Fi performance will be reduced.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_ENABLE_WPA3_SAE

Enable WPA3-Personal

Found in: Component config > Wi-Fi

Select this option to allow the device to establish a WPA3-Personal connection with eligible AP's. PMF (Protected Management Frames) is a prerequisite feature for a WPA3 connection, it needs to be explicitly configured before attempting connection. Please refer to the Wi-Fi Driver API Guide for details.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_ENABLE_SAE_PK

Enable SAE-PK

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_WPA3_SAE

Select this option to enable SAE-PK

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_SOFTAP_SAE_SUPPORT

Enable WPA3 Personal(SAE) SoftAP

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_WPA3_SAE

Select this option to enable SAE support in softAP mode.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_ENABLE_WPA3_OWE_STA

Enable OWE STA

Found in: Component config > Wi-Fi

Select this option to allow the device to establish OWE connection with eligible AP's. PMF (Protected Management Frames) is a prerequisite feature for a WPA3 connection, it needs to be explicitly configured before attempting connection. Please refer to the Wi-Fi Driver API Guide for details.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_SLP_IRAM_OPT

WiFi SLP IRAM speed optimization

Found in: Component config > Wi-Fi

Select this option to place called Wi-Fi library TBTT process and receive beacon functions in IRAM. Some functions can be put in IRAM either by ESP_WIFI_IRAM_OPT and ESP_WIFI_RX_IRAM_OPT, or this one. If already enabled ESP_WIFI_IRAM_OPT, the other 7.3KB IRAM memory would be taken by this option. If already enabled ESP_WIFI_RX_IRAM_OPT, the other 1.3KB IRAM memory would be taken by this option. If neither of them are enabled, the other 7.4KB IRAM memory would be taken by this option. Wi-Fi power-save mode average current would be reduced if this option is enabled.

Default value:

- Yes (enabled) if SOC_WIFI_HE_SUPPORT

CONFIG_ESP_WIFI_SLP_DEFAULT_MIN_ACTIVE_TIME

Minimum active time

Found in: Component config > Wi-Fi

Only for station in WIFI_PS_MIN_MODEM or WIFI_PS_MAX_MODEM. When the station enters the active state, it will work for at least ESP_WIFI_SLP_DEFAULT_MIN_ACTIVE_TIME. If a data packet is received or sent during this period, the time will be refreshed. If the time is up, but the station still has packets to receive or send, the time will also be refreshed. unit: milliseconds.

Range:

- from 8 to 60

Default value:

- 50

CONFIG_ESP_WIFI_SLP_DEFAULT_MAX_ACTIVE_TIME

Maximum keep alive time

Found in: Component config > Wi-Fi

Only for station in WIFI_PS_MIN_MODEM or WIFI_PS_MAX_MODEM. If no packet has been sent within ESP_WIFI_SLP_DEFAULT_MAX_ACTIVE_TIME, a null data packet will be sent to maintain the connection with the AP. unit: seconds.

Range:

- from 10 to 60

Default value:

- 10

CONFIG_ESP_WIFI_SLP_DEFAULT_WAIT_BROADCAST_DATA_TIME

Minimum wait broadcast data time

Found in: Component config > Wi-Fi

Only for station in WIFI_PS_MIN_MODEM or WIFI_PS_MAX_MODEM. When the station knows through the beacon that AP will send broadcast packet, it will wait for ESP_WIFI_SLP_DEFAULT_WAIT_BROADCAST_DATA_TIME before entering the sleep process. If a broadcast packet is received with more data bits, the time will refreshed. unit: milliseconds.

Range:

- from 10 to 30

Default value:

- 15

CONFIG_ESP_WIFI_FTM_ENABLE

WiFi FTM

Found in: Component config > Wi-Fi

Enable feature Fine Timing Measurement for calculating WiFi Round-Trip-Time (RTT).

Default value:

- No (disabled)

CONFIG_ESP_WIFI_FTM_INITIATOR_SUPPORT

FTM Initiator support

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_FTM_ENABLE

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_FTM_ENABLE*

CONFIG_ESP_WIFI_FTM_RESPONDER_SUPPORT

FTM Responder support

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_FTM_ENABLE

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_FTM_ENABLE*

CONFIG_ESP_WIFI_STA_DISCONNECTED_PM_ENABLE

Power Management for station at disconnected

Found in: Component config > Wi-Fi

Select this option to enable power_management for station when disconnected. Chip will do modem-sleep when rf module is not in use any more.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_GCMP_SUPPORT

WiFi GCMP Support(GCMP128 and GCMP256)

Found in: Component config > Wi-Fi

Select this option to enable GCMP support. GCMP support is compulsory for WiFi Suite-B support.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_GMAC_SUPPORT

WiFi GMAC Support(GMAC128 and GMAC256)

Found in: Component config > Wi-Fi

Select this option to enable GMAC support. GMAC support is compulsory for WiFi 192 bit certification.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_SOFTAP_SUPPORT

WiFi SoftAP Support

Found in: Component config > Wi-Fi

WiFi module can be compiled without SoftAP to save code size.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_ENHANCED_LIGHT_SLEEP

WiFi modem automatically receives the beacon

Found in: Component config > Wi-Fi

The wifi modem automatically receives the beacon frame during light sleep.

Default value:

- No (disabled) if *CONFIG_ESP_PHY_MAC_BB_PD* && SOC_PM_SUPPORT_BEACON_WAKEUP

CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT

Wifi sleep optimize when beacon lost

Found in: Component config > Wi-Fi

Enable wifi sleep optimization when beacon loss occurs and immediately enter sleep mode when the WiFi module detects beacon loss.

CONFIG_ESP_WIFI_SLP_BEACON_LOST_TIMEOUT

Beacon loss timeout

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT

Timeout time for close rf phy when beacon loss occurs, Unit: 1024 microsecond.

Range:

- from 5 to 100 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

Default value:

- 10 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

CONFIG_ESP_WIFI_SLP_BEACON_LOST_THRESHOLD

Maximum number of consecutive lost beacons allowed

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT

Maximum number of consecutive lost beacons allowed, WiFi keeps Rx state when the number of consecutive beacons lost is greater than the given threshold.

Range:

- from 0 to 8 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

Default value:

- 3 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

CONFIG_ESP_WIFI_SLP_PHY_ON_DELTA_EARLY_TIME

Delta early time for RF PHY on

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT

Delta early time for rf phy on, When the beacon is lost, the next rf phy on will be earlier the time specified by the configuration item, Unit: 32 microsecond.

Range:
- from 0 to 100 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

Default value:
- 2 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

CONFIG_ESP_WIFI_SLP_PHY_OFF_DELTA_TIMEOUT_TIME

Delta timeout time for RF PHY off

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT

Delta timeout time for rf phy off, When the beacon is lost, the next rf phy off will be delayed for the time specified by the configuration item. Unit: 1024 microsecond.

Range:
- from 0 to 8 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

Default value:
- 2 if *CONFIG_ESP_WIFI_SLP_BEACON_LOST_OPT*

CONFIG_ESP_WIFI_ESPNOW_MAX_ENCRYPT_NUM

Maximum espnow encrypt peers number

Found in: Component config > Wi-Fi

Maximum number of encrypted peers supported by espnow. The number of hardware keys for encryption is fixed. And the espnow and SoftAP share the same hardware keys. So this configuration will affect the maximum connection number of SoftAP. Maximum espnow encrypted peers number + maximum number of connections of SoftAP = Max hardware keys number. When using ESP mesh, this value should be set to a maximum of 6.

Range:
- from 0 to 17

Default value:
- 7

CONFIG_ESP_WIFI_NAN_ENABLE

WiFi Aware

Found in: Component config > Wi-Fi

Enable WiFi Aware (NAN) feature.

Default value:
- No (disabled) if SOC_WIFI_NAN_SUPPORT

CONFIG_ESP_WIFI_MBEDTLS_CRYPTO

Use MbedTLS crypto APIs

Found in: Component config > Wi-Fi

Select this option to enable the use of MbedTLS crypto APIs. The internal crypto support within the supplicant is limited and may not suffice for all new security features, including WPA3.

It is recommended to always keep this option enabled. Additionally, note that MbedTLS can leverage hardware acceleration if available, resulting in significantly faster cryptographic operations.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_MBEDTLS_TLS_CLIENT

Use MbedTLS TLS client for WiFi Enterprise connection

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_MBEDTLS_CRYPTO

Select this option to use MbedTLS TLS client for WPA2 enterprise connection. Please note that from MbedTLS-3.0 onwards, MbedTLS does not support SSL-3.0 TLS-v1.0, TLS-v1.1 versions. In case your server is using one of these version, it is advisable to update your server. Please disable this option for compatibility with older TLS versions.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_EAP_TLS1_3

Enable EAP-TLS v1.3 Support for WiFi Enterprise connection

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_MBEDTLS_CRYPTO > CONFIG_ESP_WIFI_MBEDTLS_TLS_CLIENT

Select this option to support EAP with TLS v1.3. This configuration still supports compatibility with EAP-TLS v1.2. Please note that enabling this configuration will cause every application which uses TLS go for TLS1.3 if server supports that. TLS1.3 is still in development in mbedtls and there may be interoperability issues with this. Please modify your application to set max version as TLS1.2 if you want to enable TLS1.3 only for WiFi connection.

Default value:

- No (disabled) if *CONFIG_ESP_WIFI_MBEDTLS_TLS_CLIENT* && *CONFIG_IDF_EXPERIMENTAL_FEATURES* && *CONFIG_ESP_WIFI_MBEDTLS_CRYPTO*

CONFIG_ESP_WIFI_WAPI_PSK

Enable WAPI PSK support

Found in: Component config > Wi-Fi

Select this option to enable WAPI-PSK which is a Chinese National Standard Encryption for Wireless LANs (GB 15629.11-2003).

Default value:

- No (disabled)

CONFIG_ESP_WIFI_SUITE_B_192

Enable NSA suite B support with 192 bit key

Found in: Component config > Wi-Fi

Select this option to enable 192 bit NSA suite-B. This is necessary to support WPA3 192 bit security.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_11KV_SUPPORT

Enable 802.11k, 802.11v APIs Support

Found in: Component config > Wi-Fi

Select this option to enable 802.11k 802.11v APIs(RRM and BTM support). Only APIs which are helpful for network assisted roaming are supported for now. Enable this option with BTM and RRM enabled in sta config to make device ready for network assisted roaming. BTM: BSS transition management enables an AP to request a station to transition to a specific AP, or to indicate to a station a set of preferred APs. RRM: Radio measurements enable STAs to understand the radio environment, it enables STAs to observe and gather data on radio link performance and on the radio environment. Current implementation adds beacon report, link measurement, neighbor report.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_SCAN_CACHE

Keep scan results in cache

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_11KV_SUPPORT

Keep scan results in cache, if not enabled, those will be flushed immediately.

Default value:

- No (disabled) if *CONFIG_ESP_WIFI_11KV_SUPPORT*

CONFIG_ESP_WIFI_MBO_SUPPORT

Enable Multi Band Operation Certification Support

Found in: Component config > Wi-Fi

Select this option to enable WiFi Multiband operation certification support.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_ENABLE_ROAMING_APP

Advanced support for Wi-Fi Roaming (Experimental)

Found in: Component config > Wi-Fi

Enable Espressif's roaming app to allow for efficient Wi-Fi roaming. This includes configurable periodic environment scans, maintaining a cache of the best APs, handling low rssi events etc.

Risk Warning Please note that this feature is still experimental and enabling this potentially can lead to unpredictable scanning, connection and roaming attempts. We are still working on tuning and optimising this feature to ensure reliable and stable use.

Default value:

- No (disabled) if *CONFIG_IDF_EXPERIMENTAL_FEATURES*

Configure roaming App Contains:

- *CONFIG_ESP_WIFI_ROAMING_BACKOFF_TIME*
- *Roaming Methods*
- *Roaming triggers*
- *Scan Configuration*
- *CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING*

Roaming triggers Contains:

- *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR*
- *CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING*

CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING

Use Low RSSI to trigger roaming.

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers

Enable to use a RSSI threshold to trigger roaming.

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_THRESHOLD

WiFi RSSI threshold to trigger roaming

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers > CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING

WiFi RSSI threshold to trigger roaming value in dBm (-99 to -1). Values under -30 dbm might lead to a flood of low rssi events. This interferes with normal functioning and TX/Rx performance.

Range:

- from -99 to -30 if *CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- "-60" if *CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_OFFSET

Offset by which to reset the RSSI Threshold after attempt to roam.

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers > CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING

Decide the offset by which to decrease the Low RSSI threshold set by ESP_WIFI_ROAMING_LOW_RSSI_THRESHOLD after each failed attempt to roam. This allows for the station to keep scanning for better AP's after the Low RSSI threshold is reached in a stepped manner, rather than only attempting to roam the first time the current AP's RSSI breaches the set RSSI threshold. Setting 0 here may cause station to be flooded with low rssi events, therefore that's not recommended to be kept.

Range:

- from 0 to 99 if *CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- 5 if *CONFIG_ESP_WIFI_ROAMING_LOW_RSSI_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR

Conduct periodic scans to check if a better AP is available

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers

Conduct periodic scans periodically to check if a better AP is available.

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_THRESHOLD

Threshold at which to begin periodic scanning for a better AP.

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers > CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR

Threshold at which the station will begin scanning to find an AP with better RSSI.

Range:

- from -99 to -1 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- "-50" if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_SCAN_MONITOR_INTERVAL

Time intervals (in seconds) at which station will initiate a scan

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers > CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR

Intervals at which station will periodically scan to check if better AP is available

Range:

- from 1 to 1500 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- 30 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_SCAN_ROAM_RSSI_DIFF

RSSI difference b/w current AP and candidate AP to initiate connection

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming triggers > CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR

Minimum RSSI difference b/w current AP and a potential roaming candidate AP to trigger a roaming attempt.

Range:

- from 0 to 99 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- 15 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_SCAN_MONITOR* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Roaming Methods Contains:

- *CONFIG_ESP_WIFI_ROAMING_LEGACY_ROAMING*
- *CONFIG_ESP_WIFI_ROAMING_NETWORK_ASSISTED_ROAM*

CONFIG_ESP_WIFI_ROAMING_LEGACY_ROAMING

Support Legacy roaming approach

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming Methods

Roaming between APs that do not support 802.11kv. This will allow station to roam even when connection is not BTM supported, by forcefully disconnecting from current AP and connecting to better AP.

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_NETWORK_ASSISTED_ROAM

Support Network Assisted roaming using 802.11kv

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming Methods

Roaming between APs using network assisted Roaming. This involves BSS Transition Management mechanisms outlined in 802.11v. Note that this moves the responsibility to the AP's network, and hence isn't guaranteed to cause the station to attempt to roam each time.

Default value:

- Yes (enabled) if *CONFIG_ESP_WIFI_11KV_SUPPORT* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_NETWORK_ASSISTED_ROAMING_RETRY_COUNT

Retry count after which to switch to legacy roaming

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Roaming Methods > CONFIG_ESP_WIFI_ROAMING_NETWORK_ASSISTED_ROAM

Retry threshold after which the station should stop using Network Assisted roaming methods and start using legacy roaming instead.

Range:

- from 1 to 5 if *CONFIG_ESP_WIFI_ROAMING_NETWORK_ASSISTED_ROAM* && *CONFIG_ESP_WIFI_ROAMING_LEGACY_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- 2 if *CONFIG_ESP_WIFI_ROAMING_NETWORK_ASSISTED_ROAM* && *CONFIG_ESP_WIFI_ROAMING_LEGACY_ROAMING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Scan Configuration Contains:

- *CONFIG_ESP_WIFI_ROAMING_HOME_CHANNEL_DWELL_TIME*
- *CONFIG_ESP_WIFI_ROAMING_MAX_CANDIDATES*
- *CONFIG_ESP_WIFI_ROAMING_SCAN_MAX_SCAN_TIME*
- *CONFIG_ESP_WIFI_ROAMING_SCAN_MIN_SCAN_TIME*
- *CONFIG_ESP_WIFI_ROAMING_SCAN_CHAN_LIST*
- *CONFIG_ESP_WIFI_ROAMING_SCAN_EXPIRY_WINDOW*

CONFIG_ESP_WIFI_ROAMING_SCAN_MIN_SCAN_TIME

Minimum duration (in milliseconds) of station's per channel active scan

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

Minimum duration of active scanning per channel in milliseconds.

Range:
- from 0 to 120 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 10 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_SCAN_MAX_SCAN_TIME

Maximum duration (in milliseconds) of station's per channel active scan time

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

Maximum duration of active scanning per channel in milliseconds.

Range:
- from 30 to 120 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 70 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_HOME_CHANNEL_DWELL_TIME

Home channel dwell time scanning between consecutive channels

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

If connected, duration for which the station will return to it's home channel for Tx/Rx of frames stored in buffers between scanning on consecutive channels.

Range:
- from 30 to 150 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 30 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_SCAN_CHAN_LIST

Preferred channel list for scanning

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

Channels your wireless network operates on to allow for faster scanning. Specify the channels(between 1-14) in a comma separated manner.

Default value:
- "None" if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_SCAN_EXPIRY_WINDOW

Scan results expiry window (in seconds)

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

Duration for which the results from the most recent scans can be used by the roaming app for determining the roaming candidates.

Range:
- from 5 to 20 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 10 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_MAX_CANDIDATES

Max Candidates in the network

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > Scan Configuration

Max candidates that can be considered while scanning as a part of the network at one time.

Range:
- from 3 to 20 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 3 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_BACKOFF_TIME

Default time to wait between subsequent roaming attempts.

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App

Time to wait (in seconds) by station before registering for the RSSI event again or start continuous montoring to find better AP.

Range:
- from 0 to 120 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 15 if *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING

Send periodic neighbor report request to AP for internal list updation

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App

This option will enable station to keep sending RRM neighbor list request to AP and update its internal list.

Default value:
- Yes (enabled) if *CONFIG_ESP_WIFI_11KV_SUPPORT* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_RRM_MONITOR_TIME

Time interval (in seconds) between neighbor report requests to an AP

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING

Enable this to send periodic neighbor report requests to the AP. These neighbor report requests provide information about other APs in the same managed network. This information is used for more intelligent roaming.

Range:
- from 0 to 1500 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:
- 60 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_ROAMING_RRM_MONITOR_THRESHOLD

Threshold for sending periodic neighbor report requests

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_ROAMING_APP > Configure roaming App > CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING

The RSSI threshold beyond which we start sending periodic neighbor report requests.

Range:

- from -99 to 0 if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

Default value:

- "-20" if *CONFIG_ESP_WIFI_ROAMING_PERIODIC_RRM_MONITORING* && *CONFIG_ESP_WIFI_ENABLE_ROAMING_APP*

CONFIG_ESP_WIFI_DPP_SUPPORT

Enable DPP support

Found in: Component config > Wi-Fi

Select this option to enable WiFi Easy Connect Support.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_11R_SUPPORT

Enable 802.11R (Fast Transition) Support

Found in: Component config > Wi-Fi

Select this option to enable WiFi Fast Transition Support.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_WPS_SOFTAP_REGISTRAR

Add WPS Registrar support in SoftAP mode

Found in: Component config > Wi-Fi

Select this option to enable WPS registrar support in softAP mode.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_ENABLE_WIFI_TX_STATS

Enable Wi-Fi transmission statistics

Found in: Component config > Wi-Fi

Enable Wi-Fi transmission statistics. Total support 4 access category. Each access category will use 346 bytes memory.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT

CONFIG_ESP_WIFI_ENABLE_WIFI_RX_STATS

Enable Wi-Fi reception statistics

Found in: Component config > Wi-Fi

Enable Wi-Fi reception statistics. Total support 2 access category. Each access category will use 190 bytes memory.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT

CONFIG_ESP_WIFI_ENABLE_WIFI_RX_MU_STATS

Enable Wi-Fi DL MU-MIMO and DL OFDMA reception statistics

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENABLE_WIFI_RX_STATS

Enable Wi-Fi DL MU-MIMO and DL OFDMA reception statistics. Will use 10932 bytes memory.

Default value:

- No (disabled) if *CONFIG_ESP_WIFI_ENABLE_WIFI_RX_STATS*

CONFIG_ESP_WIFI_TX_HETB_QUEUE_NUM

WiFi TX HE TB QUEUE number for STA HE TB PPDU transmission

Found in: Component config > Wi-Fi

Set the maximum number of queue that can be aggregated by the STA in the A-MPDU carried in the HE TB PPDU.

Range:

- from 1 to 4 if SOC_WIFI_HE_SUPPORT

Default value:

- 3 if SOC_WIFI_HE_SUPPORT

CONFIG_ESP_WIFI_ENABLE_DUMP_HESIGB

Enable Wi-Fi dump HE-SIGB which is contained in DL HE MU PPDUs

Found in: Component config > Wi-Fi

Enable Wi-Fi dump HE-SIGB which is contained in DL HE MU PPDUs.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT_5G

CONFIG_ESP_WIFI_ENABLE_DUMP_MU_CFO

Enable Wi-Fi dump MU CFO

Found in: Component config > Wi-Fi

Enable Wi-Fi dump MU CFO.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT_5G

CONFIG_ESP_WIFI_ENABLE_DUMP_CTRL_NDPA

Enable Wi-Fi dump NDPA frames

Found in: Component config > Wi-Fi

Enable Wi-Fi dump NDPA frames.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT_5G

CONFIG_ESP_WIFI_ENABLE_DUMP_CTRL_BFRP

Enable Wi-Fi dump BFRP frames

Found in: Component config > Wi-Fi

Enable Wi-Fi dump BFRP frames.

Default value:

- No (disabled) if SOC_WIFI_HE_SUPPORT_5G

WPS Configuration Options

Contains:

- *CONFIG_ESP_WIFI_WPS_PASSPHRASE*
- *CONFIG_ESP_WIFI_WPS_STRICT*

CONFIG_ESP_WIFI_WPS_STRICT

Strictly validate all WPS attributes

Found in: Component config > Wi-Fi > WPS Configuration Options

Select this option to enable validate each WPS attribute rigorously. Disabling this add the workarounds with various APs. Enabling this may cause inter operability issues with some APs.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_WPS_PASSPHRASE

Get WPA2 passphrase in WPS config

Found in: Component config > Wi-Fi > WPS Configuration Options

Select this option to get passphrase during WPS configuration. This option fakes the virtual display capabilities to get the configuration in passphrase mode. Not recommended to be used since WPS credentials should not be shared to other devices, making it in readable format increases that risk, also passphrase requires pbkdf2 to convert in psk.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_DEBUG_PRINT

Print debug messages from WPA Supplicant

Found in: Component config > Wi-Fi

Select this option to print logging information from WPA supplicant, this includes handshake information and key hex dumps depending on the project logging level.

Enabling this could increase the build size ~60kb depending on the project logging level.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_TESTING_OPTIONS

Add DPP testing code

Found in: Component config > Wi-Fi

Select this to enable unity test for DPP.

Default value:

- No (disabled)

CONFIG_ESP_WIFI_ENTERPRISE_SUPPORT

Enable enterprise option

Found in: Component config > Wi-Fi

Select this to enable/disable enterprise connection support.

disabling this will reduce binary size. disabling this will disable the use of any esp_wifi_sta_wpa2_ent_* (as APIs will be meaningless)

Note that when using bigger certificates on low-power chips without crypto hardware acceleration, it is recommended to adjust the task watchdog timer (TWDT) if it is enabled. For precise information on timing requirements, you can check performance numbers at https://github.com/espressif/mbedtls/wiki/Performance-Numbers.

Default value:

- Yes (enabled)

CONFIG_ESP_WIFI_ENT_FREE_DYNAMIC_BUFFER

Free dynamic buffers during WiFi enterprise connection

Found in: Component config > Wi-Fi > CONFIG_ESP_WIFI_ENTERPRISE_SUPPORT

Select this configuration to free dynamic buffers during WiFi enterprise connection. This will enable chip to reduce heap consumption during WiFi enterprise connection.

Default value:

- No (disabled)

Core dump

Contains:

- *CONFIG_ESP_COREDUMP_CHECK_BOOT*
- *CONFIG_ESP_COREDUMP_DATA_FORMAT*
- *CONFIG_ESP_COREDUMP_CHECKSUM*
- *CONFIG_ESP_COREDUMP_TO_FLASH_OR_UART*
- *CONFIG_ESP_COREDUMP_UART_DELAY*
- *CONFIG_ESP_COREDUMP_FLASH_NO_OVERWRITE*
- *CONFIG_ESP_COREDUMP_LOGS*
- *CONFIG_ESP_COREDUMP_DECODE*
- *CONFIG_ESP_COREDUMP_CAPTURE_DRAM*
- *CONFIG_ESP_COREDUMP_MAX_TASKS_NUM*
- *CONFIG_ESP_COREDUMP_STACK_SIZE*

CONFIG_ESP_COREDUMP_TO_FLASH_OR_UART

Data destination

Found in: Component config > Core dump

Select place to store core dump: flash, uart or none (to disable core dumps generation).

Core dumps to Flash are not available if PSRAM is used for task stacks.

If core dump is configured to be stored in flash and custom partition table is used add corresponding entry to your CSV. For examples, please see predefined partition table CSV descriptions in the components/partition_table directory.

Available options:

- Flash (CONFIG_ESP_COREDUMP_ENABLE_TO_FLASH)
- UART (CONFIG_ESP_COREDUMP_ENABLE_TO_UART)
- None (CONFIG_ESP_COREDUMP_ENABLE_TO_NONE)

CONFIG_ESP_COREDUMP_DATA_FORMAT

Core dump data format

Found in: Component config > Core dump

Select the data format for core dump.

Available options:

- Binary format (CONFIG_ESP_COREDUMP_DATA_FORMAT_BIN)
- ELF format (CONFIG_ESP_COREDUMP_DATA_FORMAT_ELF)

CONFIG_ESP_COREDUMP_CHECKSUM

Core dump data integrity check

Found in: Component config > Core dump

Select the integrity check for the core dump.

Available options:

- Use CRC32 for integrity verification (CONFIG_ESP_COREDUMP_CHECKSUM_CRC32)
- Use SHA256 for integrity verification (CONFIG_ESP_COREDUMP_CHECKSUM_SHA256)

CONFIG_ESP_COREDUMP_CAPTURE_DRAM

Include whole .bss and .data sections and heap data into core dump file

Found in: Component config > Core dump

Storing these sections can help with easier debugging and troubleshooting. However, additional storage space will be required in the core dump partition. At least 128KB should be reserved, but the actual amount required may vary based on the application's DRAM usage. Note that sections located in external RAM will not be stored.

Default value:

- No (disabled) if *CONFIG_ESP_COREDUMP_DATA_FORMAT_ELF*

CONFIG_ESP_COREDUMP_CHECK_BOOT

Check core dump data integrity on boot

Found in: Component config > Core dump

When enabled, if any data are found on the flash core dump partition, they will be checked by calculating their checksum.

Default value:

- Yes (enabled) if *CONFIG_ESP_COREDUMP_ENABLE_TO_FLASH*

CONFIG_ESP_COREDUMP_LOGS

Enable coredump logs for debugging

Found in: Component config > Core dump

Enable/disable coredump logs. Logs strings from espcoredump component are placed in DRAM. Disabling these helps to save ~5KB of internal memory.

CONFIG_ESP_COREDUMP_MAX_TASKS_NUM

Maximum number of tasks

Found in: Component config > Core dump

Maximum number of tasks snapshots in core dump.

CONFIG_ESP_COREDUMP_UART_DELAY

Delay before print to UART

Found in: Component config > Core dump

Config delay (in ms) before printing core dump to UART. Delay can be interrupted by pressing Enter key.

Default value:

- 0 if *CONFIG_ESP_COREDUMP_ENABLE_TO_UART*

CONFIG_ESP_COREDUMP_FLASH_NO_OVERWRITE

Don't overwrite existing core dump

Found in: Component config > Core dump

Don't overwrite an existing core dump already present in flash. Enable this option to only keep the first of multiple core dumps.

If enabled, the core dump partition must be erased before the first core dump can be written.

Default value:

- No (disabled) if *CONFIG_ESP_COREDUMP_ENABLE_TO_FLASH*

CONFIG_ESP_COREDUMP_STACK_SIZE

Reserved stack size

Found in: Component config > Core dump

Size of the memory to be reserved for core dump stack. If 0 core dump process will run on the stack of crashed task/ISR, otherwise special stack will be allocated. To ensure that core dump itself will not overflow task/ISR stack set this to the value around 1300-1800 depending on the chosen checksum calculation method. SHA256 method needs more stack space than CRC32. NOTE: It eats DRAM.

CONFIG_ESP_COREDUMP_DECODE

Handling of UART core dumps in IDF Monitor

Found in: Component config > Core dump

Available options:

- Decode and show summary (info_corefile) (CONFIG_ESP_COREDUMP_DECODE_INFO)

- Don't decode (CONFIG_ESP_COREDUMP_DECODE_DISABLE)

FAT Filesystem support Contains:

- *CONFIG_FATFS_API_ENCODING*
- *CONFIG_FATFS_VFS_FSTAT_BLKSIZE*
- *CONFIG_FATFS_IMMEDIATE_FSYNC*
- *CONFIG_FATFS_USE_FASTSEEK*
- *CONFIG_FATFS_LONG_FILENAMES*
- *CONFIG_FATFS_MAX_LFN*
- *CONFIG_FATFS_FS_LOCK*
- *CONFIG_FATFS_VOLUME_COUNT*
- *CONFIG_FATFS_CHOOSE_CODEPAGE*
- *CONFIG_FATFS_LINK_LOCK*
- *CONFIG_FATFS_ALLOC_PREFER_EXTRAM*
- *CONFIG_FATFS_SECTOR_SIZE*
- *CONFIG_FATFS_TIMEOUT_MS*
- *CONFIG_FATFS_USE_DYN_BUFFERS*
- *CONFIG_FATFS_USE_LABEL*
- *CONFIG_FATFS_PER_FILE_CACHE*

CONFIG_FATFS_VOLUME_COUNT

Number of volumes

Found in: Component config > FAT Filesystem support

Number of volumes (logical drives) to use.

Range:
- from 1 to 10

Default value:
- 2

CONFIG_FATFS_LONG_FILENAMES

Long filename support

Found in: Component config > FAT Filesystem support

Support long filenames in FAT. Long filename data increases memory usage. FATFS can be configured to store the buffer for long filename data in stack or heap.

Available options:

- No long filenames (CONFIG_FATFS_LFN_NONE)
- Long filename buffer in heap (CONFIG_FATFS_LFN_HEAP)
- Long filename buffer on stack (CONFIG_FATFS_LFN_STACK)

CONFIG_FATFS_SECTOR_SIZE

Sector size

Found in: Component config > FAT Filesystem support

Specify the size of the sector in bytes for FATFS partition generator.

Available options:

- 512 (CONFIG_FATFS_SECTOR_512)

- 4096 (CONFIG_FATFS_SECTOR_4096)

CONFIG_FATFS_CHOOSE_CODEPAGE

OEM Code Page

Found in: Component config > FAT Filesystem support

OEM code page used for file name encodings.

If "Dynamic" is selected, code page can be chosen at runtime using f_setcp function. Note that choosing this option will increase application size by ~480kB.

Available options:

- Dynamic (all code pages supported) (CONFIG_FATFS_CODEPAGE_DYNAMIC)
- US (CP437) (CONFIG_FATFS_CODEPAGE_437)
- Arabic (CP720) (CONFIG_FATFS_CODEPAGE_720)
- Greek (CP737) (CONFIG_FATFS_CODEPAGE_737)
- KBL (CP771) (CONFIG_FATFS_CODEPAGE_771)
- Baltic (CP775) (CONFIG_FATFS_CODEPAGE_775)
- Latin 1 (CP850) (CONFIG_FATFS_CODEPAGE_850)
- Latin 2 (CP852) (CONFIG_FATFS_CODEPAGE_852)
- Cyrillic (CP855) (CONFIG_FATFS_CODEPAGE_855)
- Turkish (CP857) (CONFIG_FATFS_CODEPAGE_857)
- Portuguese (CP860) (CONFIG_FATFS_CODEPAGE_860)
- Icelandic (CP861) (CONFIG_FATFS_CODEPAGE_861)
- Hebrew (CP862) (CONFIG_FATFS_CODEPAGE_862)
- Canadian French (CP863) (CONFIG_FATFS_CODEPAGE_863)
- Arabic (CP864) (CONFIG_FATFS_CODEPAGE_864)
- Nordic (CP865) (CONFIG_FATFS_CODEPAGE_865)
- Russian (CP866) (CONFIG_FATFS_CODEPAGE_866)
- Greek 2 (CP869) (CONFIG_FATFS_CODEPAGE_869)
- Japanese (DBCS) (CP932) (CONFIG_FATFS_CODEPAGE_932)
- Simplified Chinese (DBCS) (CP936) (CONFIG_FATFS_CODEPAGE_936)
- Korean (DBCS) (CP949) (CONFIG_FATFS_CODEPAGE_949)
- Traditional Chinese (DBCS) (CP950) (CONFIG_FATFS_CODEPAGE_950)

CONFIG_FATFS_MAX_LFN

Max long filename length

Found in: Component config > FAT Filesystem support

Maximum long filename length. Can be reduced to save RAM.

CONFIG_FATFS_API_ENCODING

API character encoding

Found in: Component config > FAT Filesystem support

Choose encoding for character and string arguments/returns when using FATFS APIs. The encoding of arguments will usually depend on text editor settings.

Available options:

- API uses ANSI/OEM encoding (CONFIG_FATFS_API_ENCODING_ANSI_OEM)
- API uses UTF-8 encoding (CONFIG_FATFS_API_ENCODING_UTF_8)

CONFIG_FATFS_FS_LOCK

Number of simultaneously open files protected by lock function

Found in: Component config > FAT Filesystem support

This option sets the FATFS configuration value _FS_LOCK. The option _FS_LOCK switches file lock function to control duplicated file open and illegal operation to open objects.

* 0: Disable file lock function. To avoid volume corruption, application should avoid illegal open, remove and rename to the open objects.

* >0: Enable file lock function. The value defines how many files/sub-directories can be opened simultaneously under file lock control.

Note that the file lock control is independent of re-entrancy.

Range:
- from 0 to 65535

Default value:
- 0

CONFIG_FATFS_TIMEOUT_MS

Timeout for acquiring a file lock, ms

Found in: Component config > FAT Filesystem support

This option sets FATFS configuration value _FS_TIMEOUT, scaled to milliseconds. Sets the number of milliseconds FATFS will wait to acquire a mutex when operating on an open file. For example, if one task is performing a lengthy operation, another task will wait for the first task to release the lock, and time out after amount of time set by this option.

Default value:
- 10000

CONFIG_FATFS_PER_FILE_CACHE

Use separate cache for each file

Found in: Component config > FAT Filesystem support

This option affects FATFS configuration value _FS_TINY.

If this option is set, _FS_TINY is 0, and each open file has its own cache, size of the cache is equal to the _MAX_SS variable (512 or 4096 bytes). This option uses more RAM if more than 1 file is open, but needs less reads and writes to the storage for some operations.

If this option is not set, _FS_TINY is 1, and single cache is used for all open files, size is also equal to _MAX_SS variable. This reduces the amount of heap used when multiple files are open, but increases the number of read and write operations which FATFS needs to make.

Default value:
- Yes (enabled)

CONFIG_FATFS_ALLOC_PREFER_EXTRAM

Prefer external RAM when allocating FATFS buffers

Found in: Component config > FAT Filesystem support

When the option is enabled, internal buffers used by FATFS will be allocated from external RAM. If the allocation from external RAM fails, the buffer will be allocated from the internal RAM. Disable this option if optimizing for performance. Enable this option if optimizing for internal memory size.

Default value:

- Yes (enabled) if *CONFIG_SPIRAM_USE_CAPS_ALLOC* || *CONFIG_SPIRAM_USE_MALLOC*

CONFIG_FATFS_USE_FASTSEEK

Enable fast seek algorithm when using lseek function through VFS FAT

Found in: Component config > FAT Filesystem support

The fast seek feature enables fast backward/long seek operations without FAT access by using an in-memory CLMT (cluster link map table). Please note, fast-seek is only allowed for read-mode files, if a file is opened in write-mode, the seek mechanism will automatically fallback to the default implementation.

Default value:

- No (disabled)

CONFIG_FATFS_FAST_SEEK_BUFFER_SIZE

Fast seek CLMT buffer size

Found in: Component config > FAT Filesystem support > CONFIG_FATFS_USE_FASTSEEK

If fast seek algorithm is enabled, this defines the size of CLMT buffer used by this algorithm in 32-bit word units. This value should be chosen based on prior knowledge of maximum elements of each file entry would store.

Default value:

- 64 if *CONFIG_FATFS_USE_FASTSEEK*

CONFIG_FATFS_VFS_FSTAT_BLKSIZE

Default block size

Found in: Component config > FAT Filesystem support

If set to 0, the 'newlib' library's default size (BLKSIZ) is used (128 B). If set to a non-zero value, the value is used as the block size. Default file buffer size is set to this value and the buffer is allocated when first attempt of reading/writing to a file is made. Increasing this value improves fread() speed, however the heap usage is increased as well.

NOTE: The block size value is shared by all the filesystem functions accessing target media for given file descriptor! See 'Improving I/O performance' section of 'Maximizing Execution Speed' documentation page for more details.

Default value:

- 0

CONFIG_FATFS_IMMEDIATE_FSYNC

Enable automatic f_sync

Found in: Component config > FAT Filesystem support

Enables automatic calling of f_sync() to flush recent file changes after each call of vfs_fat_write(), vfs_fat_pwrite(), vfs_fat_link(), vfs_fat_truncate() and vfs_fat_ftruncate() functions. This feature improves file-consistency and size reporting accuracy for the FatFS, at a price on decreased performance due to frequent disk operations

Default value:

- No (disabled)

CONFIG_FATFS_USE_LABEL

Use FATFS volume label

Found in: Component config > FAT Filesystem support

Allows FATFS volume label to be specified using f_setlabel

Default value:

- No (disabled)

CONFIG_FATFS_LINK_LOCK

Perform the whole link operation under lock

Found in: Component config > FAT Filesystem support

If enabled, the whole link operation (including file copying) is performed under lock. This ensures that the link operation is atomic, but may cause performance for large files. It may create less fragmented file copy.

Default value:

- Yes (enabled)

CONFIG_FATFS_USE_DYN_BUFFERS

Use dynamic buffers

Found in: Component config > FAT Filesystem support

If enabled, the buffers used by FATFS will be allocated separately from the rest of the structure. This option is useful when using multiple FATFS instances with different sector sizes, as the buffers will be allocated according to the sector size. If disabled, the greatest sector size will be used for all FATFS instances. (In most cases, this would be the sector size of Wear Levelling library) This might cause more memory to be used than necessary.

Default value:

- Yes (enabled) if CONFIG_WL_SECTOR_SIZE_4096

FreeRTOS Contains:

- *Kernel*
- *Port*

Kernel Contains:

- *CONFIG_FREERTOS_CHECK_STACKOVERFLOW*
- *CONFIG_FREERTOS_ENABLE_BACKWARD_COMPATIBILITY*
- *CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS*
- *CONFIG_FREERTOS_MAX_TASK_NAME_LEN*
- *CONFIG_FREERTOS_IDLE_TASK_STACKSIZE*
- *CONFIG_FREERTOS_THREAD_LOCAL_STORAGE_POINTERS*
- *CONFIG_FREERTOS_QUEUE_REGISTRY_SIZE*
- *CONFIG_FREERTOS_TASK_NOTIFICATION_ARRAY_ENTRIES*
- *CONFIG_FREERTOS_HZ*
- *CONFIG_FREERTOS_TIMER_QUEUE_LENGTH*
- *CONFIG_FREERTOS_TIMER_SERVICE_TASK_CORE_AFFINITY*
- *CONFIG_FREERTOS_TIMER_SERVICE_TASK_NAME*
- *CONFIG_FREERTOS_TIMER_TASK_PRIORITY*
- *CONFIG_FREERTOS_TIMER_TASK_STACK_DEPTH*
- *CONFIG_FREERTOS_USE_APPLICATION_TASK_TAG*
- *CONFIG_FREERTOS_USE_IDLE_HOOK*

- *CONFIG_FREERTOS_USE_LIST_DATA_INTEGRITY_CHECK_BYTES*
- *CONFIG_FREERTOS_OPTIMIZED_SCHEDULER*
- *CONFIG_FREERTOS_USE_TICK_HOOK*
- *CONFIG_FREERTOS_USE_TICKLESS_IDLE*
- *CONFIG_FREERTOS_USE_TRACE_FACILITY*
- *CONFIG_FREERTOS_VTASKLIST_INCLUDE_COREID*
- *CONFIG_FREERTOS_UNICORE*
- *CONFIG_FREERTOS_SMP*
- *CONFIG_FREERTOS_USE_PASSIVE_IDLE_HOOK*

CONFIG_FREERTOS_SMP

Run the Amazon SMP FreeRTOS kernel instead (FEATURE UNDER DEVELOPMENT)

Found in: Component config > FreeRTOS > Kernel

Amazon has released an SMP version of the FreeRTOS Kernel which can be found via the following link: https://github.com/FreeRTOS/FreeRTOS-Kernel/tree/smp

IDF has added an experimental port of this SMP kernel located in components/freertos/FreeRTOS-Kernel-SMP. Enabling this option will cause IDF to use the Amazon SMP kernel. Note that THIS FEATURE IS UNDER ACTIVE DEVELOPMENT, users use this at their own risk.

Leaving this option disabled will mean the IDF FreeRTOS kernel is used instead, which is located in: components/freertos/FreeRTOS-Kernel. Both kernel versions are SMP capable, but differ in their implementation and features.

Default value:

- No (disabled)

CONFIG_FREERTOS_UNICORE

Run FreeRTOS only on first core

Found in: Component config > FreeRTOS > Kernel

This version of FreeRTOS normally takes control of all cores of the CPU. Select this if you only want to start it on the first core. This is needed when e.g. another process needs complete control over the second core.

CONFIG_FREERTOS_HZ

configTICK_RATE_HZ

Found in: Component config > FreeRTOS > Kernel

Sets the FreeRTOS tick interrupt frequency in Hz (see configTICK_RATE_HZ documentation for more details).

Range:

- from 1 to 1000

Default value:

- 100

CONFIG_FREERTOS_OPTIMIZED_SCHEDULER

configUSE_PORT_OPTIMISED_TASK_SELECTION

Found in: Component config > FreeRTOS > Kernel

Enables port specific task selection method. This option can speed up the search of ready tasks when scheduling (see configUSE_PORT_OPTIMISED_TASK_SELECTION documentation for more details).

CONFIG_FREERTOS_CHECK_STACKOVERFLOW

configCHECK_FOR_STACK_OVERFLOW

Found in: Component config > FreeRTOS > Kernel

Enables FreeRTOS to check for stack overflows (see configCHECK_FOR_STACK_OVERFLOW documentation for more details).

Note: If users do not provide their own `vApplicationStackOverflowHook()` function, a default function will be provided by ESP-IDF.

Available options:

- No checking (CONFIG_FREERTOS_CHECK_STACKOVERFLOW_NONE)
  Do not check for stack overflows (configCHECK_FOR_STACK_OVERFLOW = 0)
- Check by stack pointer value (Method 1) (CONFIG_FREERTOS_CHECK_STACKOVERFLOW_PTRVAL)
  Check for stack overflows on each context switch by checking if the stack pointer is in a valid range. Quick but does not detect stack overflows that happened between context switches (configCHECK_FOR_STACK_OVERFLOW = 1)
- Check using canary bytes (Method 2) (CONFIG_FREERTOS_CHECK_STACKOVERFLOW_CANARY)
  Places some magic bytes at the end of the stack area and on each context switch, check if these bytes are still intact. More thorough than just checking the pointer, but also slightly slower. (configCHECK_FOR_STACK_OVERFLOW = 2)

CONFIG_FREERTOS_THREAD_LOCAL_STORAGE_POINTERS

configNUM_THREAD_LOCAL_STORAGE_POINTERS

Found in: Component config > FreeRTOS > Kernel

Set the number of thread local storage pointers in each task (see configNUM_THREAD_LOCAL_STORAGE_POINTERS documentation for more details).

Note: In ESP-IDF, this value must be at least 1. Index 0 is reserved for use by the pthreads API thread-local-storage. Other indexes can be used for any desired purpose.

Range:
- from 1 to 256

Default value:
- 1

CONFIG_FREERTOS_IDLE_TASK_STACKSIZE

configMINIMAL_STACK_SIZE (Idle task stack size)

Found in: Component config > FreeRTOS > Kernel

Sets the idle task stack size in bytes (see configMINIMAL_STACK_SIZE documentation for more details).

Note:

- ESP-IDF specifies stack sizes in bytes instead of words.
- The default size is enough for most use cases.
- The stack size may need to be increased above the default if the app installs idle or thread local storage cleanup hooks that use a lot of stack memory.
- Conversely, the stack size can be reduced to the minimum if non of the idle features are used.

Range:
- from 768 to 32768

Default value:

- 1536

CONFIG_FREERTOS_USE_IDLE_HOOK

configUSE_IDLE_HOOK

Found in: Component config > FreeRTOS > Kernel

Enables the idle task application hook (see configUSE_IDLE_HOOK documentation for more details).

Note:

- The application must provide the hook function `void vApplicationIdleHook( void );`
- `vApplicationIdleHook()` is called from FreeRTOS idle task(s)
- The FreeRTOS idle hook is NOT the same as the ESP-IDF Idle Hook, but both can be enabled simultaneously.

Default value:

- No (disabled)

CONFIG_FREERTOS_USE_PASSIVE_IDLE_HOOK

Use FreeRTOS minimal idle hook

Found in: Component config > FreeRTOS > Kernel

Enables the minimal idle task application hook (see configUSE_IDLE_HOOK documentation for more details).

Note:

- The application must provide the hook function `void vApplicationPassiveIdleHook( void );`
- `vApplicationPassiveIdleHook()` is called from FreeRTOS minimal idle task(s)

Default value:

- No (disabled) if *CONFIG_FREERTOS_SMP*

CONFIG_FREERTOS_USE_TICK_HOOK

configUSE_TICK_HOOK

Found in: Component config > FreeRTOS > Kernel

Enables the tick hook (see configUSE_TICK_HOOK documentation for more details).

Note:

- The application must provide the hook function `void vApplicationTickHook( void );`
- `vApplicationTickHook()` is called from FreeRTOS's tick handling function `xTaskIncrementTick()`
- The FreeRTOS tick hook is NOT the same as the ESP-IDF Tick Interrupt Hook, but both can be enabled simultaneously.

Default value:

- No (disabled)

CONFIG_FREERTOS_MAX_TASK_NAME_LEN

configMAX_TASK_NAME_LEN

Found in: Component config > FreeRTOS > Kernel

Sets the maximum number of characters for task names (see configMAX_TASK_NAME_LEN documentation for more details).

Note: For most uses, the default of 16 characters is sufficient.

Range:
- from 1 to 256

Default value:
- 16

CONFIG_FREERTOS_ENABLE_BACKWARD_COMPATIBILITY

configENABLE_BACKWARD_COMPATIBILITY

Found in: Component config > FreeRTOS > Kernel

Enable backward compatibility with APIs prior to FreeRTOS v8.0.0. (see configENABLE_BACKWARD_COMPATIBILITY documentation for more details).

Default value:
- No (disabled)

CONFIG_FREERTOS_TIMER_SERVICE_TASK_NAME

configTIMER_SERVICE_TASK_NAME

Found in: Component config > FreeRTOS > Kernel

Sets the timer task's name (see configTIMER_SERVICE_TASK_NAME documentation for more details).

Default value:
- "Tmr Svc"

CONFIG_FREERTOS_TIMER_SERVICE_TASK_CORE_AFFINITY

configTIMER_SERVICE_TASK_CORE_AFFINITY

Found in: Component config > FreeRTOS > Kernel

Sets the timer task's core affinity (see configTIMER_SERVICE_TASK_CORE_AFFINITY documentation for more details).

Available options:

- CPU0 (CONFIG_FREERTOS_TIMER_TASK_AFFINITY_CPU0)
- CPU1 (CONFIG_FREERTOS_TIMER_TASK_AFFINITY_CPU1)
- No affinity (CONFIG_FREERTOS_TIMER_TASK_NO_AFFINITY)

CONFIG_FREERTOS_TIMER_TASK_PRIORITY

configTIMER_TASK_PRIORITY

Found in: Component config > FreeRTOS > Kernel

Sets the timer task's priority (see configTIMER_TASK_PRIORITY documentation for more details).

Range:
- from 1 to 25

Default value:
- 1

CONFIG_FREERTOS_TIMER_TASK_STACK_DEPTH

configTIMER_TASK_STACK_DEPTH

Found in: Component config > FreeRTOS > Kernel

Set the timer task's stack size (see configTIMER_TASK_STACK_DEPTH documentation for more details).

Range:
- from 1536 to 32768

Default value:
- 2048

CONFIG_FREERTOS_TIMER_QUEUE_LENGTH

configTIMER_QUEUE_LENGTH

Found in: Component config > FreeRTOS > Kernel

Set the timer task's command queue length (see configTIMER_QUEUE_LENGTH documentation for more details).

Range:
- from 5 to 20

Default value:
- 10

CONFIG_FREERTOS_QUEUE_REGISTRY_SIZE

configQUEUE_REGISTRY_SIZE

Found in: Component config > FreeRTOS > Kernel

Set the size of the queue registry (see configQUEUE_REGISTRY_SIZE documentation for more details).

Note: A value of 0 will disable queue registry functionality

Range:
- from 0 to 20

Default value:
- 0

CONFIG_FREERTOS_TASK_NOTIFICATION_ARRAY_ENTRIES

configTASK_NOTIFICATION_ARRAY_ENTRIES

Found in: Component config > FreeRTOS > Kernel

Set the size of the task notification array of each task. When increasing this value, keep in mind that this means additional memory for each and every task on the system. However, task notifications in general are more light weight compared to alternatives such as semaphores.

Range:
- from 1 to 32

Default value:
- 1

CONFIG_FREERTOS_USE_TRACE_FACILITY

configUSE_TRACE_FACILITY

Found in: Component config > FreeRTOS > Kernel

Enables additional structure members and functions to assist with execution visualization and tracing (see configUSE_TRACE_FACILITY documentation for more details).

Default value:

- No (disabled)

CONFIG_FREERTOS_USE_STATS_FORMATTING_FUNCTIONS

configUSE_STATS_FORMATTING_FUNCTIONS

Found in: Component config > FreeRTOS > Kernel > CONFIG_FREERTOS_USE_TRACE_FACILITY

Set configUSE_TRACE_FACILITY and configUSE_STATS_FORMATTING_FUNCTIONS to 1 to include the `vTaskList()` and `vTaskGetRunTimeStats()` functions in the build (see configUSE_STATS_FORMATTING_FUNCTIONS documentation for more details).

Default value:

- No (disabled) if *CONFIG_FREERTOS_USE_TRACE_FACILITY*

CONFIG_FREERTOS_USE_LIST_DATA_INTEGRITY_CHECK_BYTES

configUSE_LIST_DATA_INTEGRITY_CHECK_BYTES

Found in: Component config > FreeRTOS > Kernel

Enable list integrity checker (see configUSE_LIST_DATA_INTEGRITY_CHECK_BYTES documentation for more details).

Default value:

- No (disabled)

CONFIG_FREERTOS_VTASKLIST_INCLUDE_COREID

Enable display of xCoreID in vTaskList

Found in: Component config > FreeRTOS > Kernel

If enabled, this will include an extra column when vTaskList is called to display the CoreID the task is pinned to (0,1) or -1 if not pinned.

CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS

configGENERATE_RUN_TIME_STATS

Found in: Component config > FreeRTOS > Kernel

Enables collection of run time statistics for each task (see configGENERATE_RUN_TIME_STATS documentation for more details).

Note: The clock used for run time statistics can be configured in FREERTOS_RUN_TIME_STATS_CLK.

Default value:

- No (disabled)

CONFIG_FREERTOS_RUN_TIME_COUNTER_TYPE

configRUN_TIME_COUNTER_TYPE

Found in: Component config > FreeRTOS > Kernel > CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS

Sets the data type used for the FreeRTOS run time stats. A larger data type can be used to reduce the frequency of the counter overflowing.

Available options:

- uint32_t (CONFIG_FREERTOS_RUN_TIME_COUNTER_TYPE_U32) configRUN_TIME_COUNTER_TYPE is set to uint32_t
- uint64_t (CONFIG_FREERTOS_RUN_TIME_COUNTER_TYPE_U64) configRUN_TIME_COUNTER_TYPE is set to uint64_t

CONFIG_FREERTOS_USE_TICKLESS_IDLE

configUSE_TICKLESS_IDLE

Found in: Component config > FreeRTOS > Kernel

If power management support is enabled, FreeRTOS will be able to put the system into light sleep mode when no tasks need to run for a number of ticks. This number can be set using FREERTOS_IDLE_TIME_BEFORE_SLEEP option. This feature is also known as "automatic light sleep".

Note that timers created using esp_timer APIs may prevent the system from entering sleep mode, even when no tasks need to run. To skip unnecessary wake-up initialize a timer with the "skip_unhandled_events" option as true.

If disabled, automatic light sleep support will be disabled.

Default value:

- No (disabled) if *CONFIG_PM_ENABLE*

CONFIG_FREERTOS_IDLE_TIME_BEFORE_SLEEP

configEXPECTED_IDLE_TIME_BEFORE_SLEEP

Found in: Component config > FreeRTOS > Kernel > CONFIG_FREERTOS_USE_TICKLESS_IDLE

FreeRTOS will enter light sleep mode if no tasks need to run for this number of ticks. You can enable PM_PROFILING feature in esp_pm components and dump the sleep status with esp_pm_dump_locks, if the proportion of rejected sleeps is too high, please increase this value to improve scheduling efficiency

Range:

- from 2 to 4294967295 if *CONFIG_FREERTOS_USE_TICKLESS_IDLE*

Default value:

- 3 if *CONFIG_FREERTOS_USE_TICKLESS_IDLE*

CONFIG_FREERTOS_USE_APPLICATION_TASK_TAG

configUSE_APPLICATION_TASK_TAG

Found in: Component config > FreeRTOS > Kernel

Enables task tagging functionality and its associated API (see configUSE_APPLICATION_TASK_TAG documentation for more details).

Default value:

- No (disabled)

Port Contains:

- *CONFIG_FREERTOS_CHECK_MUTEX_GIVEN_BY_OWNER*
- *CONFIG_FREERTOS_RUN_TIME_STATS_CLK*
- *CONFIG_FREERTOS_INTERRUPT_BACKTRACE*
- *CONFIG_FREERTOS_WATCHPOINT_END_OF_STACK*
- *CONFIG_FREERTOS_ENABLE_STATIC_TASK_CLEAN_UP*
- *CONFIG_FREERTOS_TASK_PRE_DELETION_HOOK*
- *CONFIG_FREERTOS_TLSP_DELETION_CALLBACKS*

- *CONFIG_FREERTOS_ISR_STACKSIZE*
- *CONFIG_FREERTOS_PLACE_FUNCTIONS_INTO_FLASH*
- *CONFIG_FREERTOS_CHECK_PORT_CRITICAL_COMPLIANCE*
- *CONFIG_FREERTOS_CORETIMER*
- *CONFIG_FREERTOS_TASK_FUNCTION_WRAPPER*

CONFIG_FREERTOS_TASK_FUNCTION_WRAPPER

Wrap task functions

Found in: Component config > FreeRTOS > Port

If enabled, all FreeRTOS task functions will be enclosed in a wrapper function. If a task function mistakenly returns (i.e. does not delete), the call flow will return to the wrapper function. The wrapper function will then log an error and abort the application. This option is also required for GDB backtraces and C++ exceptions to work correctly inside top-level task functions.

Default value:

- Yes (enabled)

CONFIG_FREERTOS_WATCHPOINT_END_OF_STACK

Enable stack overflow debug watchpoint

Found in: Component config > FreeRTOS > Port

FreeRTOS can check if a stack has overflown its bounds by checking either the value of the stack pointer or by checking the integrity of canary bytes. (See FREERTOS_CHECK_STACKOVERFLOW for more information.) These checks only happen on a context switch, and the situation that caused the stack overflow may already be long gone by then. This option will use the last debug memory watchpoint to allow breaking into the debugger (or panic'ing) as soon as any of the last 32 bytes on the stack of a task are overwritten. The side effect is that using gdb, you effectively have one hardware watchpoint less because the last one is overwritten as soon as a task switch happens.

Another consequence is that due to alignment requirements of the watchpoint, the usable stack size decreases by up to 60 bytes. This is because the watchpoint region has to be aligned to its size and the size for the stack watchpoint in IDF is 32 bytes.

This check only triggers if the stack overflow writes within 32 bytes near the end of the stack, rather than overshooting further, so it is worth combining this approach with one of the other stack overflow check methods.

When this watchpoint is hit, gdb will stop with a SIGTRAP message. When no JTAG OCD is attached, esp-idf will panic on an unhandled debug exception.

Default value:

- No (disabled)

CONFIG_FREERTOS_TLSP_DELETION_CALLBACKS

Enable thread local storage pointers deletion callbacks

Found in: Component config > FreeRTOS > Port

ESP-IDF provides users with the ability to free TLSP memory by registering TLSP deletion callbacks. These callbacks are automatically called by FreeRTOS when a task is deleted. When this option is turned on, the memory reserved for TLSPs in the TCB is doubled to make space for storing the deletion callbacks. If the user does not wish to use TLSP deletion callbacks then this option could be turned off to save space in the TCB memory.

Default value:

- Yes (enabled)

CONFIG_FREERTOS_TASK_PRE_DELETION_HOOK

Enable task pre-deletion hook

Found in: Component config > FreeRTOS > Port

Enable this option to make FreeRTOS call a user provided hook function right before it deletes a task (i.e., frees/releases a dynamically/statically allocated task's memory). This is useful if users want to know when a task is actually deleted (in case the task's deletion is delegated to the IDLE task).

If this config option is enabled, users must define a `void vTaskPreDeletionHook( void \* pxTCB )` hook function in their application.

CONFIG_FREERTOS_ENABLE_STATIC_TASK_CLEAN_UP

Enable static task clean up hook (DEPRECATED)

Found in: Component config > FreeRTOS > Port

THIS OPTION IS DEPRECATED. Use FREERTOS_TASK_PRE_DELETION_HOOK instead.

Enable this option to make FreeRTOS call the static task clean up hook when a task is deleted.

Note: Users will need to provide a `void vPortCleanUpTCB ( void \*pxTCB )` callback

Default value:

- No (disabled)

CONFIG_FREERTOS_CHECK_MUTEX_GIVEN_BY_OWNER

Check that mutex semaphore is given by owner task

Found in: Component config > FreeRTOS > Port

If enabled, assert that when a mutex semaphore is given, the task giving the semaphore is the task which is currently holding the mutex.

CONFIG_FREERTOS_ISR_STACKSIZE

ISR stack size

Found in: Component config > FreeRTOS > Port

The interrupt handlers have their own stack. The size of the stack can be defined here. Each processor has its own stack, so the total size occupied will be twice this.

Range:

- from 2096 to 32768 if *CONFIG_ESP_COREDUMP_DATA_FORMAT_ELF*
- from 1536 to 32768

Default value:

- 2096 if *CONFIG_ESP_COREDUMP_DATA_FORMAT_ELF*
- 1536

CONFIG_FREERTOS_INTERRUPT_BACKTRACE

Enable backtrace from interrupt to task context

Found in: Component config > FreeRTOS > Port

If this option is enabled, interrupt stack frame will be modified to point to the code of the interrupted task as its return address. This helps the debugger (or the panic handler) show a backtrace from the interrupt to the task which was interrupted. This also works for nested interrupts: higher level interrupt stack can be traced back to the lower level interrupt. This option adds 4 instructions to the interrupt dispatching code.

Default value:

- Yes (enabled)

CONFIG_FREERTOS_CORETIMER

Tick timer source (Xtensa Only)

Found in: Component config > FreeRTOS > Port

FreeRTOS needs a timer with an associated interrupt to use as the main tick source to increase counters, run timers and do pre-emptive multitasking with. There are multiple timers available to do this, with different interrupt priorities.

Available options:

- Timer 0 (int 6, level 1) (CONFIG_FREERTOS_CORETIMER_0)
  Select this to use timer 0
- Timer 1 (int 15, level 3) (CONFIG_FREERTOS_CORETIMER_1)
  Select this to use timer 1
- SYSTIMER 0 (level 1) (CONFIG_FREERTOS_CORETIMER_SYSTIMER_LVL1)
  Select this to use systimer with the 1 interrupt priority.
- SYSTIMER 0 (level 3) (CONFIG_FREERTOS_CORETIMER_SYSTIMER_LVL3)
  Select this to use systimer with the 3 interrupt priority.

CONFIG_FREERTOS_RUN_TIME_STATS_CLK

Choose the clock source for run time stats

Found in: Component config > FreeRTOS > Port

Choose the clock source for FreeRTOS run time stats. Options are CPU0's CPU Clock or the ESP Timer. Both clock sources are 32 bits. The CPU Clock can run at a higher frequency hence provide a finer resolution but will overflow much quicker. Note that run time stats are only valid until the clock source overflows.

Available options:

- Use ESP TIMER for run time stats (CONFIG_FREERTOS_RUN_TIME_STATS_USING_ESP_TIMER)
  ESP Timer will be used as the clock source for FreeRTOS run time stats. The ESP Timer runs at a frequency of 1MHz regardless of Dynamic Frequency Scaling. Therefore the ESP Timer will overflow in approximately 4290 seconds.
- Use CPU Clock for run time stats (CONFIG_FREERTOS_RUN_TIME_STATS_USING_CPU_CLK)
  CPU Clock will be used as the clock source for the generation of run time stats. The CPU Clock has a frequency dependent on ESP_DEFAULT_CPU_FREQ_MHZ and Dynamic Frequency Scaling (DFS). Therefore the CPU Clock frequency can fluctuate between 80 to 240MHz. Run time stats generated using the CPU Clock represents the number of CPU cycles each task is allocated and DOES NOT reflect the amount of time each task runs for (as CPU clock frequency can change). If the CPU clock consistently runs at the maximum frequency of 240MHz, it will overflow in approximately 17 seconds.

CONFIG_FREERTOS_PLACE_FUNCTIONS_INTO_FLASH

Place FreeRTOS functions into Flash

Found in: Component config > FreeRTOS > Port

When enabled the selected Non-ISR FreeRTOS functions will be placed into Flash memory instead of IRAM. This saves up to 8KB of IRAM depending on which functions are used.

Default value:

- No (disabled)

CONFIG_FREERTOS_CHECK_PORT_CRITICAL_COMPLIANCE

Tests compliance with Vanilla FreeRTOS port*_CRITICAL calls

Found in: Component config > FreeRTOS > Port

If enabled, context of port*_CRITICAL calls (ISR or Non-ISR) would be checked to be in compliance with Vanilla FreeRTOS. e.g Calling port*_CRITICAL from ISR context would cause assert failure

Default value:

- No (disabled)

Hardware Abstraction Layer (HAL) and Low Level (LL)

Contains:

- *CONFIG_HAL_DEFAULT_ASSERTION_LEVEL*
- *CONFIG_HAL_LOG_LEVEL*
- *CONFIG_HAL_SYSTIMER_USE_ROM_IMPL*
- *CONFIG_HAL_WDT_USE_ROM_IMPL*

CONFIG_HAL_DEFAULT_ASSERTION_LEVEL

Default HAL assertion level

Found in: Component config > Hardware Abstraction Layer (HAL) and Low Level (LL)

Set the assert behavior / level for HAL component. HAL component assert level can be set separately, but the level can't exceed the system assertion level. e.g. If the system assertion is disabled, then the HAL assertion can't be enabled either. If the system assertion is enable, then the HAL assertion can still be disabled by this Kconfig option.

Available options:

- Same as system assertion level (CONFIG_HAL_ASSERTION_EQUALS_SYSTEM)
- Disabled (CONFIG_HAL_ASSERTION_DISABLE)
- Silent (CONFIG_HAL_ASSERTION_SILENT)
- Enabled (CONFIG_HAL_ASSERTION_ENABLE)

CONFIG_HAL_LOG_LEVEL

HAL layer log verbosity

Found in: Component config > Hardware Abstraction Layer (HAL) and Low Level (LL)

Specify how much output to see in HAL logs.

Available options:

- No output (CONFIG_HAL_LOG_LEVEL_NONE)
- Error (CONFIG_HAL_LOG_LEVEL_ERROR)
- Warning (CONFIG_HAL_LOG_LEVEL_WARN)
- Info (CONFIG_HAL_LOG_LEVEL_INFO)
- Debug (CONFIG_HAL_LOG_LEVEL_DEBUG)
- Verbose (CONFIG_HAL_LOG_LEVEL_VERBOSE)

CONFIG_HAL_SYSTIMER_USE_ROM_IMPL

Use ROM implementation of SysTimer HAL driver

Found in: Component config > Hardware Abstraction Layer (HAL) and Low Level (LL)

Enable this flag to use HAL functions from ROM instead of ESP-IDF.

If keeping this as "n" in your project, you will have less free IRAM. If making this as "y" in your project, you will increase free IRAM, but you will lose the possibility to debug this module, and some new features will be added and bugs will be fixed in the IDF source but cannot be synced to ROM.

Default value:

- Yes (enabled) if ESP_ROM_HAS_HAL_SYSTIMER

CONFIG_HAL_WDT_USE_ROM_IMPL

Use ROM implementation of WDT HAL driver

Found in: Component config > Hardware Abstraction Layer (HAL) and Low Level (LL)

Enable this flag to use HAL functions from ROM instead of ESP-IDF.

If keeping this as "n" in your project, you will have less free IRAM. If making this as "y" in your project, you will increase free IRAM, but you will lose the possibility to debug this module, and some new features will be added and bugs will be fixed in the IDF source but cannot be synced to ROM.

Default value:

- Yes (enabled)

Heap memory debugging

Contains:

- *CONFIG_HEAP_ABORT_WHEN_ALLOCATION_FAILS*
- *CONFIG_HEAP_TASK_TRACKING*
- *CONFIG_HEAP_PLACE_FUNCTION_INTO_FLASH*
- *CONFIG_HEAP_CORRUPTION_DETECTION*
- *CONFIG_HEAP_TRACING_DEST*
- *CONFIG_HEAP_TRACING_STACK_DEPTH*
- *CONFIG_HEAP_USE_HOOKS*
- *CONFIG_HEAP_TRACE_HASH_MAP*
- *CONFIG_HEAP_TLSF_USE_ROM_IMPL*

CONFIG_HEAP_CORRUPTION_DETECTION

Heap corruption detection

Found in: Component config > Heap memory debugging

Enable heap poisoning features to detect heap corruption caused by out-of-bounds access to heap memory.

See the "Heap Memory Debugging" page of the IDF documentation for a description of each level of heap corruption detection.

Available options:

- Basic (no poisoning) (CONFIG_HEAP_POISONING_DISABLED)
- Light impact (CONFIG_HEAP_POISONING_LIGHT)
- Comprehensive (CONFIG_HEAP_POISONING_COMPREHENSIVE)

CONFIG_HEAP_TRACING_DEST

Heap tracing

Found in: Component config > Heap memory debugging

Enables the heap tracing API defined in esp_heap_trace.h.

This function causes a moderate increase in IRAM code side and a minor increase in heap function (malloc/free/realloc) CPU overhead, even when the tracing feature is not used. So it's best to keep it disabled unless tracing is being used.

Available options:

- Disabled (CONFIG_HEAP_TRACING_OFF)
- Standalone (CONFIG_HEAP_TRACING_STANDALONE)
- Host-based (CONFIG_HEAP_TRACING_TOHOST)

CONFIG_HEAP_TRACING_STACK_DEPTH

Heap tracing stack depth

Found in: Component config > Heap memory debugging

Number of stack frames to save when tracing heap operation callers.

More stack frames uses more memory in the heap trace buffer (and slows down allocation), but can provide useful information.

CONFIG_HEAP_USE_HOOKS

Use allocation and free hooks

Found in: Component config > Heap memory debugging

Enable the user to implement function hooks triggered for each successful allocation and free.

CONFIG_HEAP_TASK_TRACKING

Enable heap task tracking

Found in: Component config > Heap memory debugging

Enables tracking the task responsible for each heap allocation.

This function depends on heap poisoning being enabled and adds four more bytes of overhead for each block allocated.

CONFIG_HEAP_TRACE_HASH_MAP

Use hash map mechanism to access heap trace records

Found in: Component config > Heap memory debugging

Enable this flag to use a hash map to increase performance in handling heap trace records.

Heap trace standalone supports storing records as a list, or a list + hash map.

Using only a list takes less memory, but calls to 'free' will get slower as the list grows. This is particularly affected when using HEAP_TRACE_ALL mode.

By using a list + hash map, calls to 'free' remain fast, at the cost of additional memory to store the hash map.

Default value:

- No (disabled) if *CONFIG_HEAP_TRACING_STANDALONE*

CONFIG_HEAP_TRACE_HASH_MAP_IN_EXT_RAM

Place hash map in external RAM

Found in: Component config > Heap memory debugging > CONFIG_HEAP_TRACE_HASH_MAP

When enabled this configuration forces the hash map to be placed in external RAM.

Default value:

- No (disabled) if *CONFIG_HEAP_TRACE_HASH_MAP*

CONFIG_HEAP_TRACE_HASH_MAP_SIZE

The number of entries in the hash map

Found in: Component config > Heap memory debugging > CONFIG_HEAP_TRACE_HASH_MAP

Defines the number of entries in the heap trace hashmap. Each entry takes 8 bytes. The bigger this number is, the better the performance. Recommended range: 200 - 2000.

Default value:

- 512 if *CONFIG_HEAP_TRACE_HASH_MAP*

CONFIG_HEAP_ABORT_WHEN_ALLOCATION_FAILS

Abort if memory allocation fails

Found in: Component config > Heap memory debugging

When enabled, if a memory allocation operation fails it will cause a system abort.

Default value:

- No (disabled)

CONFIG_HEAP_TLSF_USE_ROM_IMPL

Use ROM implementation of heap tlsf library

Found in: Component config > Heap memory debugging

Enable this flag to use heap functions from ROM instead of ESP-IDF.

If keeping this as "n" in your project, you will have less free IRAM. If making this as "y" in your project, you will increase free IRAM, but you will lose the possibility to debug this module, and some new features will be added and bugs will be fixed in the IDF source but cannot be synced to ROM.

Default value:

- Yes (enabled) if ESP_ROM_HAS_HEAP_TLSF

CONFIG_HEAP_PLACE_FUNCTION_INTO_FLASH

Force the entire heap component to be placed in flash memory

Found in: Component config > Heap memory debugging

Enable this flag to save up RAM space by placing the heap component in the flash memory

Note that it is only safe to enable this configuration if no functions from esp_heap_caps.h or esp_heap_trace.h are called from ISR.

IEEE 802.15.4 Contains:

- *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_ENABLED

IEEE802154 Enable

Found in: Component config > IEEE 802.15.4

Default value:

- Yes (enabled) if SOC_IEEE802154_SUPPORTED

CONFIG_IEEE802154_RX_BUFFER_SIZE

The number of 802.15.4 receive buffers

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

The number of 802.15.4 receive buffers

Range:

- from 2 to 100 if *CONFIG_IEEE802154_ENABLED*

Default value:

- 20 if *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_CCA_MODE

Clear Channel Assessment (CCA) mode

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

configure the CCA mode

Available options:

- Carrier sense only (CONFIG_IEEE802154_CCA_CARRIER)
  configure the CCA mode to Energy above threshold
- Energy above threshold (CONFIG_IEEE802154_CCA_ED)
  configure the CCA mode to Energy above threshold
- Carrier sense OR energy above threshold (CONFIG_IEEE802154_CCA_CARRIER_OR_ED)
  configure the CCA mode to Carrier sense OR energy above threshold
- Carrier sense AND energy above threshold (CONFIG_IEEE802154_CCA_CARRIER_AND_ED)
  configure the CCA mode to Carrier sense AND energy above threshold

CONFIG_IEEE802154_CCA_THRESHOLD

CCA detection threshold

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

set the CCA threshold, in dB

Range:

- from -120 to 0 if *CONFIG_IEEE802154_ENABLED*

Default value:

- "-60" if *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_PENDING_TABLE_SIZE

Pending table size

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

set the pending table size

Range:
- from 1 to 100 if *CONFIG_IEEE802154_ENABLED*

Default value:
- 20 if *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_MULTI_PAN_ENABLE

Enable multi-pan feature for frame filter

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

Enable IEEE802154 multi-pan

Default value:
- No (disabled) if *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_TIMING_OPTIMIZATION

Enable throughput optimization

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

Enabling this option increases throughput by ~5% at the expense of ~2.1k IRAM code size increase.

Default value:
- No (disabled) if *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_SLEEP_ENABLE

Enable IEEE802154 light sleep

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

Enabling this option allows the IEEE802.15.4 module to be powered down during automatic light sleep, which reduces current consumption.

Default value:
- No (disabled) if *CONFIG_PM_ENABLE* && *CONFIG_IEEE802154_ENABLED*

CONFIG_IEEE802154_DEBUG

Enable IEEE802154 Debug

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED

Enabling this option allows different kinds of IEEE802154 debug output. All IEEE802154 debug features increase the size of the final binary.

Default value:
- No (disabled) if *CONFIG_IEEE802154_ENABLED*

Contains:

- *CONFIG_IEEE802154_RECORD_ABORT*
- *CONFIG_IEEE802154_RECORD_CMD*
- *CONFIG_IEEE802154_RECORD_EVENT*
- *CONFIG_IEEE802154_RECORD_STATE*
- *CONFIG_IEEE802154_TXRX_STATISTIC*
- *CONFIG_IEEE802154_ASSERT*

CONFIG_IEEE802154_ASSERT

Enrich the assert information with IEEE802154 state and event

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED > CONFIG_IEEE802154_DEBUG

Enabling this option to add some probe codes in the driver, and these informations will be printed when assert.

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

CONFIG_IEEE802154_RECORD_EVENT

Enable record event information for debugging

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED > CONFIG_IEEE802154_DEBUG

Enabling this option to record event, when assert, the recorded event will be printed.

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

CONFIG_IEEE802154_RECORD_EVENT_SIZE

Record event table size

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED > CONFIG_IEEE802154_DEBUG > CONFIG_IEEE802154_RECORD_EVENT

set the record event table size

Range:

- from 1 to 50 if *CONFIG_IEEE802154_RECORD_EVENT*

Default value:

- 30 if *CONFIG_IEEE802154_RECORD_EVENT*

CONFIG_IEEE802154_RECORD_STATE

Enable record state information for debugging

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED > CONFIG_IEEE802154_DEBUG

Enabling this option to record state, when assert, the recorded state will be printed.

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

CONFIG_IEEE802154_RECORD_STATE_SIZE

Record state table size

Found in: Component config > IEEE 802.15.4 > CONFIG_IEEE802154_ENABLED > CONFIG_IEEE802154_DEBUG > CONFIG_IEEE802154_RECORD_STATE

set the record state table size

Range:

- from 1 to 50 if *CONFIG_IEEE802154_RECORD_STATE*

Default value:

- 10 if *CONFIG_IEEE802154_RECORD_STATE*

CONFIG_IEEE802154_RECORD_CMD

Enable record command information for debugging

Found in: *Component config* > *IEEE 802.15.4* > *CONFIG_IEEE802154_ENABLED* > *CONFIG_IEEE802154_DEBUG*

Enabling this option to record the command, when assert, the recorded command will be printed.

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

CONFIG_IEEE802154_RECORD_CMD_SIZE

Record command table size

Found in: *Component config* > *IEEE 802.15.4* > *CONFIG_IEEE802154_ENABLED* > *CONFIG_IEEE802154_DEBUG* > *CONFIG_IEEE802154_RECORD_CMD*

set the record command table size

Range:

- from 1 to 50 if *CONFIG_IEEE802154_RECORD_CMD*

Default value:

- 10 if *CONFIG_IEEE802154_RECORD_CMD*

CONFIG_IEEE802154_RECORD_ABORT

Enable record abort information for debugging

Found in: *Component config* > *IEEE 802.15.4* > *CONFIG_IEEE802154_ENABLED* > *CONFIG_IEEE802154_DEBUG*

Enabling this option to record the abort, when assert, the recorded abort will be printed.

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

CONFIG_IEEE802154_RECORD_ABORT_SIZE

Record abort table size

Found in: *Component config* > *IEEE 802.15.4* > *CONFIG_IEEE802154_ENABLED* > *CONFIG_IEEE802154_DEBUG* > *CONFIG_IEEE802154_RECORD_ABORT*

set the record abort table size

Range:

- from 1 to 50 if *CONFIG_IEEE802154_RECORD_ABORT*

Default value:

- 10 if *CONFIG_IEEE802154_RECORD_ABORT*

CONFIG_IEEE802154_TXRX_STATISTIC

Enable record tx/rx packets information for debugging

Found in: *Component config* > *IEEE 802.15.4* > *CONFIG_IEEE802154_ENABLED* > *CONFIG_IEEE802154_DEBUG*

Enabling this option to record the tx and rx

Default value:

- No (disabled) if *CONFIG_IEEE802154_DEBUG*

Log output Contains:

- *CONFIG_LOG_DEFAULT_LEVEL*
- *CONFIG_LOG_MASTER_LEVEL*
- *CONFIG_LOG_TIMESTAMP_SOURCE*
- *CONFIG_LOG_MAXIMUM_LEVEL*
- *CONFIG_LOG_COLORS*

CONFIG_LOG_DEFAULT_LEVEL

Default log verbosity

Found in: Component config > Log output

Specify how much output to see in logs by default. You can set lower verbosity level at runtime using esp_log_level_set function.

By default, this setting limits which log statements are compiled into the program. For example, selecting "Warning" would mean that changing log level to "Debug" at runtime will not be possible. To allow increasing log level above the default at runtime, see the next option.

Available options:

- No output (CONFIG_LOG_DEFAULT_LEVEL_NONE)
- Error (CONFIG_LOG_DEFAULT_LEVEL_ERROR)
- Warning (CONFIG_LOG_DEFAULT_LEVEL_WARN)
- Info (CONFIG_LOG_DEFAULT_LEVEL_INFO)
- Debug (CONFIG_LOG_DEFAULT_LEVEL_DEBUG)
- Verbose (CONFIG_LOG_DEFAULT_LEVEL_VERBOSE)

CONFIG_LOG_MAXIMUM_LEVEL

Maximum log verbosity

Found in: Component config > Log output

This config option sets the highest log verbosity that it's possible to select at runtime by calling esp_log_level_set(). This level may be higher than the default verbosity level which is set when the app starts up.

This can be used enable debugging output only at a critical point, for a particular tag, or to minimize startup time but then enable more logs once the firmware has loaded.

Note that increasing the maximum available log level will increase the firmware binary size.

This option only applies to logging from the app, the bootloader log level is fixed at compile time to the separate "Bootloader log verbosity" setting.

Available options:

- Same as default (CONFIG_LOG_MAXIMUM_EQUALS_DEFAULT)
- Error (CONFIG_LOG_MAXIMUM_LEVEL_ERROR)
- Warning (CONFIG_LOG_MAXIMUM_LEVEL_WARN)
- Info (CONFIG_LOG_MAXIMUM_LEVEL_INFO)
- Debug (CONFIG_LOG_MAXIMUM_LEVEL_DEBUG)
- Verbose (CONFIG_LOG_MAXIMUM_LEVEL_VERBOSE)

CONFIG_LOG_MASTER_LEVEL

Enable global master log level

Found in: Component config > Log output

Enables an additional global "master" log level check that occurs before a log tag cache lookup. This is useful if you want to compile in a lot of logs that are selectable at runtime, but avoid the performance hit during periods where you don't want log output. Examples include remote log forwarding, or disabling logs during a time-critical or CPU-intensive section and re-enabling them later. Results in larger program size depending on number of logs compiled in.

If enabled, defaults to LOG_DEFAULT_LEVEL and can be set using esp_log_set_level_master(). This check takes precedence over ESP_LOG_LEVEL_LOCAL.

Default value:

- No (disabled)

CONFIG_LOG_COLORS

Use ANSI terminal colors in log output

Found in: Component config > Log output

Enable ANSI terminal color codes in bootloader output.

In order to view these, your terminal program must support ANSI color codes.

Default value:

- Yes (enabled)

CONFIG_LOG_TIMESTAMP_SOURCE

Log Timestamps

Found in: Component config > Log output

Choose what sort of timestamp is displayed in the log output:

- Milliseconds since boot is calulated from the RTOS tick count multiplied by the tick period. This time will reset after a software reboot. e.g. (90000)
- System time is taken from POSIX time functions which use the chip's RTC and high resoultion timers to maintain an accurate time. The system time is initialized to 0 on startup, it can be set with an SNTP sync, or with POSIX time functions. This time will not reset after a software reboot. e.g. (00:01:30.000)
- NOTE: Currently this will not get used in logging from binary blobs (i.e WiFi & Bluetooth libraries), these will always print milliseconds since boot.

Available options:

- Milliseconds Since Boot (CONFIG_LOG_TIMESTAMP_SOURCE_RTOS)
- System Time (CONFIG_LOG_TIMESTAMP_SOURCE_SYSTEM)

LWIP

Contains:

- *CONFIG_LWIP_CHECK_THREAD_SAFETY*
- *Checksums*
- *CONFIG_LWIP_DHCP_COARSE_TIMER_SECS*
- *DHCP server*
- *CONFIG_LWIP_DHCP_OPTIONS_LEN*
- *CONFIG_LWIP_DHCP_DISABLE_CLIENT_ID*
- *CONFIG_LWIP_DHCP_DISABLE_VENDOR_CLASS_ID*
- *CONFIG_LWIP_DHCP_DOES_ARP_CHECK*

- *CONFIG_LWIP_DHCP_RESTORE_LAST_IP*
- *DNS*
- *CONFIG_LWIP_PPP_CHAP_SUPPORT*
- *CONFIG_LWIP_L2_TO_L3_COPY*
- *CONFIG_LWIP_IPV6_DHCP6*
- *CONFIG_LWIP_IP4_FRAG*
- *CONFIG_LWIP_IP6_FRAG*
- *CONFIG_LWIP_IP_FORWARD*
- *CONFIG_LWIP_NETBUF_RECVINFO*
- *CONFIG_LWIP_IPV4*
- *CONFIG_LWIP_AUTOIP*
- *CONFIG_LWIP_IPV6*
- *CONFIG_LWIP_ENABLE_LCP_ECHO*
- *CONFIG_LWIP_ESP_LWIP_ASSERT*
- *CONFIG_LWIP_DEBUG*
- *CONFIG_LWIP_IRAM_OPTIMIZATION*
- *CONFIG_LWIP_EXTRA_IRAM_OPTIMIZATION*
- *CONFIG_LWIP_ENABLE*
- *CONFIG_LWIP_STATS*
- *CONFIG_LWIP_TIMERS_ONDEMAND*
- *CONFIG_LWIP_DNS_SUPPORT_MDNS_QUERIES*
- *CONFIG_LWIP_PPP_MPPE_SUPPORT*
- *CONFIG_LWIP_PPP_MSCHAP_SUPPORT*
- *CONFIG_LWIP_PPP_NOTIFY_PHASE_SUPPORT*
- *CONFIG_LWIP_PPP_PAP_SUPPORT*
- *CONFIG_LWIP_PPP_DEBUG_ON*
- *CONFIG_LWIP_PPP_SERVER_SUPPORT*
- *CONFIG_LWIP_PPP_SUPPORT*
- *CONFIG_LWIP_IP4_REASSEMBLY*
- *CONFIG_LWIP_IP6_REASSEMBLY*
- *CONFIG_LWIP_SLIP_SUPPORT*
- *CONFIG_LWIP_SO_LINGER*
- *CONFIG_LWIP_SO_RCVBUF*
- *CONFIG_LWIP_SO_REUSE*
- *CONFIG_LWIP_NETIF_STATUS_CALLBACK*
- *CONFIG_LWIP_TCPIP_CORE_LOCKING*
- *CONFIG_LWIP_NETIF_API*
- *CONFIG_LWIP_PPP_VJ_HEADER_COMPRESSION*
- *Hooks*
- *ICMP*
- *CONFIG_LWIP_LOCAL_HOSTNAME*
- *CONFIG_LWIP_ND6*
- *LWIP RAW API*
- *CONFIG_LWIP_TCPIP_TASK_PRIO*
- *CONFIG_LWIP_IPV6_ND6_NUM_NEIGHBORS*
- *CONFIG_LWIP_IPV6_MEMP_NUM_ND6_QUEUE*
- *CONFIG_LWIP_MAX_SOCKETS*
- *CONFIG_LWIP_BRIDGEIF_MAX_PORTS*
- *CONFIG_LWIP_NUM_NETIF_CLIENT_DATA*
- *CONFIG_LWIP_ESP_GRATUITOUS_ARP*
- *CONFIG_LWIP_ESP_MLDV6_REPORT*
- *SNTP*
- *CONFIG_LWIP_USE_ONLY_LWIP_SELECT*
- *CONFIG_LWIP_NETIF_LOOPBACK*
- *TCP*
- *CONFIG_LWIP_TCPIP_TASK_AFFINITY*
- *CONFIG_LWIP_TCPIP_TASK_STACK_SIZE*
- *CONFIG_LWIP_TCPIP_RECVMBOX_SIZE*

- *CONFIG_LWIP_IP_REASS_MAX_PBUFS*
- *CONFIG_LWIP_IP_DEFAULT_TTL*
- *UDP*
- *CONFIG_LWIP_IPV6_RDNSS_MAX_DNS_SERVERS*

CONFIG_LWIP_ENABLE

Enable LwIP stack

Found in: Component config > LWIP

Builds normally if selected. Excludes LwIP from build if unselected, even if it is a dependency of a component or application. Some applications can switch their IP stacks, e.g., when switching between chip and Linux targets (LwIP stack vs. Linux IP stack). Since the LwIP dependency cannot easily be excluded based on a Kconfig option, it has to be a dependency in all cases. This switch allows the LwIP stack to be built selectively, even if it is a dependency.

Default value:

- Yes (enabled)

CONFIG_LWIP_LOCAL_HOSTNAME

Local netif hostname

Found in: Component config > LWIP

The default name this device will report to other devices on the network. Could be updated at runtime with esp_netif_set_hostname()

Default value:

- "espressif"

CONFIG_LWIP_NETIF_API

Enable usage of standard POSIX APIs in LWIP

Found in: Component config > LWIP

If this feature is enabled, standard POSIX APIs: if_indextoname(), if_nametoindex() could be used to convert network interface index to name instead of IDF specific esp-netif APIs (such as esp_netif_get_netif_impl_name())

Default value:

- No (disabled)

CONFIG_LWIP_TCPIP_TASK_PRIO

LWIP TCP/IP Task Priority

Found in: Component config > LWIP

LWIP tcpip task priority. In case of high throughput, this parameter could be changed up to (config-MAX_PRIORITIES-1).

Range:

- from 1 to 24

Default value:

- 18

CONFIG_LWIP_TCPIP_CORE_LOCKING

Enable tcpip core locking

Found in: Component config > LWIP

If Enable tcpip core locking,Creates a global mutex that is held during TCPIP thread operations.Can be locked by client code to perform lwIP operations without changing into TCPIP thread using callbacks. See LOCK_TCPIP_CORE() and UNLOCK_TCPIP_CORE().

If disable tcpip core locking,TCP IP will perform tasks through context switching

Default value:

- No (disabled)

CONFIG_LWIP_TCPIP_CORE_LOCKING_INPUT

Enable tcpip core locking input

Found in: Component config > LWIP > CONFIG_LWIP_TCPIP_CORE_LOCKING

when LWIP_TCPIP_CORE_LOCKING is enabled, this lets tcpip_input() grab the mutex for input packets as well, instead of allocating a message and passing it to tcpip_thread.

Default value:

- No (disabled) if *CONFIG_LWIP_TCPIP_CORE_LOCKING*

CONFIG_LWIP_CHECK_THREAD_SAFETY

Checks that lwip API runs in expected context

Found in: Component config > LWIP

Enable to check that the project does not violate lwip thread safety. If enabled, all lwip functions that require thread awareness run an assertion to verify that the TCP/IP core functionality is either locked or accessed from the correct thread.

Default value:

- No (disabled)

CONFIG_LWIP_DNS_SUPPORT_MDNS_QUERIES

Enable mDNS queries in resolving host name

Found in: Component config > LWIP

If this feature is enabled, standard API such as gethostbyname support .local addresses by sending one shot multicast mDNS query

Default value:

- Yes (enabled)

CONFIG_LWIP_L2_TO_L3_COPY

Enable copy between Layer2 and Layer3 packets

Found in: Component config > LWIP

If this feature is enabled, all traffic from layer2(WIFI Driver) will be copied to a new buffer before sending it to layer3(LWIP stack), freeing the layer2 buffer. Please be notified that the total layer2 receiving buffer is fixed and ESP32 currently supports 25 layer2 receiving buffer, when layer2 buffer runs out of memory, then the incoming packets will be dropped in hardware. The layer3 buffer is allocated from the heap, so the total layer3 receiving buffer depends on the available heap size, when heap runs out of memory, no copy will be sent to layer3 and packet will be dropped in layer2. Please make sure you fully understand the impact of this feature before enabling it.

Default value:

- No (disabled)

CONFIG_LWIP_IRAM_OPTIMIZATION

Enable LWIP IRAM optimization

Found in: Component config > LWIP

If this feature is enabled, some functions relating to RX/TX in LWIP will be put into IRAM, it can improve UDP/TCP throughput by >10% for single core mode, it doesn't help too much for dual core mode. On the other hand, it needs about 10KB IRAM for these optimizations.

If this feature is disabled, all lwip functions will be put into FLASH.

Default value:

- No (disabled)

CONFIG_LWIP_EXTRA_IRAM_OPTIMIZATION

Enable LWIP IRAM optimization for TCP part

Found in: Component config > LWIP

If this feature is enabled, some tcp part functions relating to RX/TX in LWIP will be put into IRAM, it can improve TCP throughput. On the other hand, it needs about 17KB IRAM for these optimizations.

Default value:

- No (disabled)

CONFIG_LWIP_TIMERS_ONDEMAND

Enable LWIP Timers on demand

Found in: Component config > LWIP

If this feature is enabled, IGMP and MLD6 timers will be activated only when joining groups or receiving QUERY packets.

This feature will reduce the power consumption for applications which do not use IGMP and MLD6.

Default value:

- Yes (enabled)

CONFIG_LWIP_ND6

LWIP NDP6 Enable/Disable

Found in: Component config > LWIP

This option is used to disable the Network Discovery Protocol (NDP) if it is not required. Please use this option with caution, as the NDP is essential for IPv6 functionality within a local network.

Default value:

- Yes (enabled)

CONFIG_LWIP_FORCE_ROUTER_FORWARDING

LWIP Force Router Forwarding Enable/Disable

Found in: Component config > LWIP > CONFIG_LWIP_ND6

This option is used to set the the router flag for the NA packets. When enabled, the router flag in NA packet will always set to 1, otherwise, never set router flag for NA packets.

Default value:

- No (disabled)

CONFIG_LWIP_MAX_SOCKETS

Max number of open sockets

Found in: Component config > LWIP

Sockets take up a certain amount of memory, and allowing fewer sockets to be open at the same time conserves memory. Specify the maximum amount of sockets here. The valid value is from 1 to 16.

Range:
- from 1 to 16

Default value:
- 10

CONFIG_LWIP_USE_ONLY_LWIP_SELECT

Support LWIP socket select() only (DEPRECATED)

Found in: Component config > LWIP

This option is deprecated. Do not use this option, use VFS_SUPPORT_SELECT instead.

Default value:
- No (disabled)

CONFIG_LWIP_SO_LINGER

Enable SO_LINGER processing

Found in: Component config > LWIP

Enabling this option allows SO_LINGER processing. l_onoff = 1,l_linger can set the timeout.

If l_linger=0, When a connection is closed, TCP will terminate the connection. This means that TCP will discard any data packets stored in the socket send buffer and send an RST to the peer.

If l_linger!=0,Then closesocket() calls to block the process until the remaining data packets has been sent or timed out.

Default value:
- No (disabled)

CONFIG_LWIP_SO_REUSE

Enable SO_REUSEADDR option

Found in: Component config > LWIP

Enabling this option allows binding to a port which remains in TIME_WAIT.

Default value:
- Yes (enabled)

CONFIG_LWIP_SO_REUSE_RXTOALL

SO_REUSEADDR copies broadcast/multicast to all matches

Found in: Component config > LWIP > CONFIG_LWIP_SO_REUSE

Enabling this option means that any incoming broadcast or multicast packet will be copied to all of the local sockets that it matches (may be more than one if SO_REUSEADDR is set on the socket.)

This increases memory overhead as the packets need to be copied, however they are only copied per matching socket. You can safely disable it if you don't plan to receive broadcast or multicast traffic on more than one socket at a time.

Default value:

- Yes (enabled)

CONFIG_LWIP_SO_RCVBUF

Enable SO_RCVBUF option

Found in: Component config > LWIP

Enabling this option allows checking for available data on a netconn.

Default value:

- No (disabled)

CONFIG_LWIP_NETBUF_RECVINFO

Enable IP_PKTINFO option

Found in: Component config > LWIP

Enabling this option allows checking for the destination address of a received IPv4 Packet.

Default value:

- No (disabled)

CONFIG_LWIP_IP_DEFAULT_TTL

The value for Time-To-Live used by transport layers

Found in: Component config > LWIP

Set value for Time-To-Live used by transport layers.

Range:

- from 1 to 255

Default value:

- 64

CONFIG_LWIP_IP4_FRAG

Enable fragment outgoing IP4 packets

Found in: Component config > LWIP

Enabling this option allows fragmenting outgoing IP4 packets if their size exceeds MTU.

Default value:

- Yes (enabled)

CONFIG_LWIP_IP6_FRAG

Enable fragment outgoing IP6 packets

Found in: Component config > LWIP

Enabling this option allows fragmenting outgoing IP6 packets if their size exceeds MTU.

Default value:

- Yes (enabled)

CONFIG_LWIP_IP4_REASSEMBLY

Enable reassembly incoming fragmented IP4 packets

Found in: Component config > LWIP

Enabling this option allows reassemblying incoming fragmented IP4 packets.

Default value:

- No (disabled)

CONFIG_LWIP_IP6_REASSEMBLY

Enable reassembly incoming fragmented IP6 packets

Found in: Component config > LWIP

Enabling this option allows reassemblying incoming fragmented IP6 packets.

Default value:

- No (disabled)

CONFIG_LWIP_IP_REASS_MAX_PBUFS

The maximum amount of pbufs waiting to be reassembled

Found in: Component config > LWIP

Set the maximum amount of pbufs waiting to be reassembled.

Range:

- from 10 to 100

Default value:

- 10

CONFIG_LWIP_IP_FORWARD

Enable IP forwarding

Found in: Component config > LWIP

Enabling this option allows packets forwarding across multiple interfaces.

Default value:

- No (disabled)

CONFIG_LWIP_IPV4_NAPT

Enable NAT

Found in: Component config > LWIP > CONFIG_LWIP_IP_FORWARD

Enabling this option allows Network Address and Port Translation.

Default value:

- No (disabled) if *CONFIG_LWIP_IP_FORWARD*

CONFIG_LWIP_IPV4_NAPT_PORTMAP

Enable NAT Port Mapping

Found in: Component config > LWIP > CONFIG_LWIP_IP_FORWARD > CONFIG_LWIP_IPV4_NAPT

Enabling this option allows Port Forwarding or Port mapping.

Default value:

- Yes (enabled) if *CONFIG_LWIP_IPV4_NAPT*

CONFIG_LWIP_STATS

Enable LWIP statistics

Found in: Component config > LWIP

Enabling this option allows LWIP statistics

Default value:

- No (disabled)

CONFIG_LWIP_ESP_GRATUITOUS_ARP

Send gratuitous ARP periodically

Found in: Component config > LWIP

Enable this option allows to send gratuitous ARP periodically.

This option solve the compatibility issues.If the ARP table of the AP is old, and the AP doesn't send ARP request to update it's ARP table, this will lead to the STA sending IP packet fail. Thus we send gratuitous ARP periodically to let AP update it's ARP table.

Default value:

- Yes (enabled)

CONFIG_LWIP_GARP_TMR_INTERVAL

GARP timer interval(seconds)

Found in: Component config > LWIP > CONFIG_LWIP_ESP_GRATUITOUS_ARP

Set the timer interval for gratuitous ARP. The default value is 60s

Default value:

- 60

CONFIG_LWIP_ESP_MLDV6_REPORT

Send mldv6 report periodically

Found in: Component config > LWIP

Enable this option allows to send mldv6 report periodically.

This option solve the issue that failed to receive multicast data. Some routers fail to forward multicast packets. To solve this problem, send multicast mdlv6 report to routers regularly.

Default value:

- Yes (enabled)

CONFIG_LWIP_MLDV6_TMR_INTERVAL

mldv6 report timer interval(seconds)

Found in: Component config > LWIP > CONFIG_LWIP_ESP_MLDV6_REPORT

Set the timer interval for mldv6 report. The default value is 30s

Default value:

- 40

CONFIG_LWIP_TCPIP_RECVMBOX_SIZE

TCPIP task receive mail box size

Found in: Component config > LWIP

Set TCPIP task receive mail box size. Generally bigger value means higher throughput but more memory. The value should be bigger than UDP/TCP mail box size.

Range:
- from 6 to 1024 if *CONFIG_LWIP_WND_SCALE*

Default value:
- 32

CONFIG_LWIP_DHCP_DOES_ARP_CHECK

DHCP: Perform ARP check on any offered address

Found in: Component config > LWIP

Enabling this option performs a check (via ARP request) if the offered IP address is not already in use by another host on the network.

Default value:
- Yes (enabled)

CONFIG_LWIP_DHCP_DISABLE_CLIENT_ID

DHCP: Disable Use of HW address as client identification

Found in: Component config > LWIP

This option could be used to disable DHCP client identification with its MAC address. (Client id is used by DHCP servers to uniquely identify clients and are included in the DHCP packets as an option 61) Set this option to "y" in order to exclude option 61 from DHCP packets.

Default value:
- No (disabled)

CONFIG_LWIP_DHCP_DISABLE_VENDOR_CLASS_ID

DHCP: Disable Use of vendor class identification

Found in: Component config > LWIP

This option could be used to disable DHCP client vendor class identification. Set this option to "y" in order to exclude option 60 from DHCP packets.

Default value:
- Yes (enabled)

CONFIG_LWIP_DHCP_RESTORE_LAST_IP

DHCP: Restore last IP obtained from DHCP server

Found in: Component config > LWIP

When this option is enabled, DHCP client tries to re-obtain last valid IP address obtained from DHCP server. Last valid DHCP configuration is stored in nvs and restored after reset/power-up. If IP is still available, there is no need for sending discovery message to DHCP server and save some time.

Default value:
- No (disabled)

CONFIG_LWIP_DHCP_OPTIONS_LEN

DHCP total option length

Found in: Component config > LWIP

Set total length of outgoing DHCP option msg. Generally bigger value means it can carry more options and values. If your code meets LWIP_ASSERT due to option value is too long. Please increase the LWIP_DHCP_OPTIONS_LEN value.

Range:
- from 68 to 255

Default value:
- 68

CONFIG_LWIP_NUM_NETIF_CLIENT_DATA

Number of clients store data in netif

Found in: Component config > LWIP

Number of clients that may store data in client_data member array of struct netif.

Range:
- from 0 to 256

Default value:
- 0

CONFIG_LWIP_DHCP_COARSE_TIMER_SECS

DHCP coarse timer interval(s)

Found in: Component config > LWIP

Set DHCP coarse interval in seconds. A higher value will be less precise but cost less power consumption.

Range:
- from 1 to 10

Default value:
- 1

DHCP server Contains:

- *CONFIG_LWIP_DHCPS*

CONFIG_LWIP_DHCPS

DHCPS: Enable IPv4 Dynamic Host Configuration Protocol Server (DHCPS)

Found in: Component config > LWIP > DHCP server

Enabling this option allows the device to run the DHCP server (to dynamically assign IPv4 addresses to clients).

Default value:
- Yes (enabled)

CONFIG_LWIP_DHCPS_LEASE_UNIT

Multiplier for lease time, in seconds

Found in: Component config > LWIP > DHCP server > CONFIG_LWIP_DHCPS

The DHCP server is calculating lease time multiplying the sent and received times by this number of seconds per unit. The default is 60, that equals one minute.

Range:
- from 1 to 3600

Default value:
- 60

CONFIG_LWIP_DHCPS_MAX_STATION_NUM

Maximum number of stations

Found in: Component config > LWIP > DHCP server > CONFIG_LWIP_DHCPS

The maximum number of DHCP clients that are connected to the server. After this number is exceeded, DHCP server removes of the oldest device from it's address pool, without notification.

Range:
- from 1 to 64

Default value:
- 8

CONFIG_LWIP_DHCPS_STATIC_ENTRIES

Enable ARP static entries

Found in: Component config > LWIP > DHCP server > CONFIG_LWIP_DHCPS

Enabling this option allows DHCP server to support temporary static ARP entries for DHCP Client. This will help the DHCP server to send the DHCP OFFER and DHCP ACK using IP unicast.

Default value:
- Yes (enabled)

CONFIG_LWIP_AUTOIP

Enable IPV4 Link-Local Addressing (AUTOIP)

Found in: Component config > LWIP

Enabling this option allows the device to self-assign an address in the 169.256/16 range if none is assigned statically or via DHCP.

See RFC 3927.

Default value:
- No (disabled)

Contains:

- *CONFIG_LWIP_AUTOIP_TRIES*
- *CONFIG_LWIP_AUTOIP_MAX_CONFLICTS*
- *CONFIG_LWIP_AUTOIP_RATE_LIMIT_INTERVAL*

CONFIG_LWIP_AUTOIP_TRIES

DHCP Probes before self-assigning IPv4 LL address

Found in: Component config > LWIP > CONFIG_LWIP_AUTOIP

DHCP client will send this many probes before self-assigning a link local address.

From LWIP help: "This can be set as low as 1 to get an AutoIP address very quickly, but you should be prepared to handle a changing IP address when DHCP overrides AutoIP." (In the case of ESP-IDF, this means multiple SYSTEM_EVENT_STA_GOT_IP events.)

Range:
- from 1 to 100 if *CONFIG_LWIP_AUTOIP*

Default value:

- 2 if *CONFIG_LWIP_AUTOIP*

CONFIG_LWIP_AUTOIP_MAX_CONFLICTS

Max IP conflicts before rate limiting

Found in: Component config > LWIP > CONFIG_LWIP_AUTOIP

If the AUTOIP functionality detects this many IP conflicts while self-assigning an address, it will go into a rate limited mode.

Range:

- from 1 to 100 if *CONFIG_LWIP_AUTOIP*

Default value:

- 9 if *CONFIG_LWIP_AUTOIP*

CONFIG_LWIP_AUTOIP_RATE_LIMIT_INTERVAL

Rate limited interval (seconds)

Found in: Component config > LWIP > CONFIG_LWIP_AUTOIP

If rate limiting self-assignment requests, wait this long between each request.

Range:

- from 5 to 120 if *CONFIG_LWIP_AUTOIP*

Default value:

- 20 if *CONFIG_LWIP_AUTOIP*

CONFIG_LWIP_IPV4

Enable IPv4

Found in: Component config > LWIP

Enable IPv4 stack. If you want to use IPv6 only TCP/IP stack, disable this.

Default value:

- Yes (enabled)

CONFIG_LWIP_IPV6

Enable IPv6

Found in: Component config > LWIP

Enable IPv6 function. If not use IPv6 function, set this option to n. If disabling LWIP_IPV6 then some other components (asio) will no longer be available.

Default value:

- Yes (enabled)

CONFIG_LWIP_IPV6_AUTOCONFIG

Enable IPV6 stateless address autoconfiguration (SLAAC)

Found in: Component config > LWIP > CONFIG_LWIP_IPV6

Enabling this option allows the devices to IPV6 stateless address autoconfiguration (SLAAC).

See RFC 4862.

Default value:

- No (disabled)

CONFIG_LWIP_IPV6_NUM_ADDRESSES

Number of IPv6 addresses on each network interface

Found in: Component config > LWIP > CONFIG_LWIP_IPV6

The maximum number of IPv6 addresses on each interface. Any additional addresses will be discarded.

Default value:

- 3

CONFIG_LWIP_IPV6_FORWARD

Enable IPv6 forwarding between interfaces

Found in: Component config > LWIP > CONFIG_LWIP_IPV6

Forwarding IPv6 packets between interfaces is only required when acting as a router.

Default value:

- No (disabled)

CONFIG_LWIP_IPV6_RDNSS_MAX_DNS_SERVERS

Use IPv6 Router Advertisement Recursive DNS Server Option

Found in: Component config > LWIP

Use IPv6 Router Advertisement Recursive DNS Server Option (as per RFC 6106) to copy a defined maximum number of DNS servers to the DNS module. Set this option to a number of desired DNS servers advertised in the RA protocol. This feature is disabled when set to 0.

Default value:

- 0 if *CONFIG_LWIP_IPV6_AUTOCONFIG*

CONFIG_LWIP_IPV6_DHCP6

Enable DHCPv6 stateless address autoconfiguration

Found in: Component config > LWIP

Enable DHCPv6 for IPv6 stateless address autoconfiguration. Note that the dhcpv6 client has to be started using dhcp6_enable_stateless(netif); Note that the stateful address autoconfiguration is not supported.

Default value:

- No (disabled) if *CONFIG_LWIP_IPV6_AUTOCONFIG*

CONFIG_LWIP_NETIF_STATUS_CALLBACK

Enable status callback for network interfaces

Found in: Component config > LWIP

Enable callbacks when the network interface is up/down and addresses are changed.

Default value:

- No (disabled)

CONFIG_LWIP_NETIF_LOOPBACK

Support per-interface loopback

Found in: Component config > LWIP

Enabling this option means that if a packet is sent with a destination address equal to the interface's own IP address, it will "loop back" and be received by this interface. Disabling this option disables support of loopback interface in lwIP

Default value:

- Yes (enabled)

Contains:

- *CONFIG_LWIP_LOOPBACK_MAX_PBUFS*

CONFIG_LWIP_LOOPBACK_MAX_PBUFS

Max queued loopback packets per interface

Found in: Component config > LWIP > CONFIG_LWIP_NETIF_LOOPBACK

Configure the maximum number of packets which can be queued for loopback on a given interface. Reducing this number may cause packets to be dropped, but will avoid filling memory with queued packet data.

Range:

- from 0 to 16

Default value:

- 8

TCP

Contains:

- *CONFIG_LWIP_TCP_WND_DEFAULT*
- *CONFIG_LWIP_TCP_SND_BUF_DEFAULT*
- *CONFIG_LWIP_TCP_ACCEPTMBOX_SIZE*
- *CONFIG_LWIP_TCP_RECVMBOX_SIZE*
- *CONFIG_LWIP_TCP_RTO_TIME*
- *CONFIG_LWIP_MAX_ACTIVE_TCP*
- *CONFIG_LWIP_TCP_FIN_WAIT_TIMEOUT*
- *CONFIG_LWIP_MAX_LISTENING_TCP*
- *CONFIG_LWIP_TCP_MAXRTX*
- *CONFIG_LWIP_TCP_SYNMAXRTX*
- *CONFIG_LWIP_TCP_MSL*
- *CONFIG_LWIP_TCP_MSS*
- *CONFIG_LWIP_TCP_OVERSIZE*
- *CONFIG_LWIP_TCP_QUEUE_OOSEQ*
- *CONFIG_LWIP_WND_SCALE*
- *CONFIG_LWIP_TCP_HIGH_SPEED_RETRANSMISSION*
- *CONFIG_LWIP_TCP_TMR_INTERVAL*

CONFIG_LWIP_MAX_ACTIVE_TCP

Maximum active TCP Connections

Found in: Component config > LWIP > TCP

The maximum number of simultaneously active TCP connections. The practical maximum limit is determined by available heap memory at runtime.

Changing this value by itself does not substantially change the memory usage of LWIP, except for preventing new TCP connections after the limit is reached.

Range:
- from 1 to 1024

Default value:
- 16

CONFIG_LWIP_MAX_LISTENING_TCP

Maximum listening TCP Connections

Found in: Component config > LWIP > TCP

The maximum number of simultaneously listening TCP connections. The practical maximum limit is determined by available heap memory at runtime.

Changing this value by itself does not substantially change the memory usage of LWIP, except for preventing new listening TCP connections after the limit is reached.

Range:
- from 1 to 1024

Default value:
- 16

CONFIG_LWIP_TCP_HIGH_SPEED_RETRANSMISSION

TCP high speed retransmissions

Found in: Component config > LWIP > TCP

Speed up the TCP retransmission interval. If disabled, it is recommended to change the number of SYN retransmissions to 6, and TCP initial rto time to 3000.

Default value:
- Yes (enabled)

CONFIG_LWIP_TCP_MAXRTX

Maximum number of retransmissions of data segments

Found in: Component config > LWIP > TCP

Set maximum number of retransmissions of data segments.

Range:
- from 3 to 12

Default value:
- 12

CONFIG_LWIP_TCP_SYNMAXRTX

Maximum number of retransmissions of SYN segments

Found in: Component config > LWIP > TCP

Set maximum number of retransmissions of SYN segments.

Range:
- from 3 to 12

Default value:
- 12

CONFIG_LWIP_TCP_MSS

Maximum Segment Size (MSS)

Found in: Component config > LWIP > TCP

Set maximum segment size for TCP transmission.

Can be set lower to save RAM, the default value 1460(ipv4)/1440(ipv6) will give best throughput. IPv4 TCP_MSS Range: 576 <= TCP_MSS <= 1460 IPv6 TCP_MSS Range: 1220<= TCP_MSS <= 1440

Range:
- from 536 to 1460

Default value:
- 1440

CONFIG_LWIP_TCP_TMR_INTERVAL

TCP timer interval(ms)

Found in: Component config > LWIP > TCP

Set TCP timer interval in milliseconds.

Can be used to speed connections on bad networks. A lower value will redeliver unacked packets faster.

Default value:
- 250

CONFIG_LWIP_TCP_MSL

Maximum segment lifetime (MSL)

Found in: Component config > LWIP > TCP

Set maximum segment lifetime in milliseconds.

Default value:
- 60000

CONFIG_LWIP_TCP_FIN_WAIT_TIMEOUT

Maximum FIN segment lifetime

Found in: Component config > LWIP > TCP

Set maximum segment lifetime in milliseconds.

Default value:
- 20000

CONFIG_LWIP_TCP_SND_BUF_DEFAULT

Default send buffer size

Found in: Component config > LWIP > TCP

Set default send buffer size for new TCP sockets.

Per-socket send buffer size can be changed at runtime with lwip_setsockopt(s, TCP_SNDBUF, ...).

This value must be at least 2x the MSS size, and the default is 4x the default MSS size.

Setting a smaller default SNDBUF size can save some RAM, but will decrease performance.

Range:
- from 2440 to 1024000 if *CONFIG_LWIP_WND_SCALE*

Default value:

- 5760

CONFIG_LWIP_TCP_WND_DEFAULT

Default receive window size

Found in: Component config > LWIP > TCP

Set default TCP receive window size for new TCP sockets.

Per-socket receive window size can be changed at runtime with lwip_setsockopt(s, TCP_WINDOW, ...).

Setting a smaller default receive window size can save some RAM, but will significantly decrease performance.

Range:
- from 2440 to 1024000 if *CONFIG_LWIP_WND_SCALE*

Default value:
- 5760

CONFIG_LWIP_TCP_RECVMBOX_SIZE

Default TCP receive mail box size

Found in: Component config > LWIP > TCP

Set TCP receive mail box size. Generally bigger value means higher throughput but more memory. The recommended value is: LWIP_TCP_WND_DEFAULT/TCP_MSS + 2, e.g. if LWIP_TCP_WND_DEFAULT=14360, TCP_MSS=1436, then the recommended receive mail box size is (14360/1436 + 2) = 12.

TCP receive mail box is a per socket mail box, when the application receives packets from TCP socket, LWIP core firstly posts the packets to TCP receive mail box and the application then fetches the packets from mail box. It means LWIP can caches maximum LWIP_TCP_RECCVMBOX_SIZE packets for each TCP socket, so the maximum possible cached TCP packets for all TCP sockets is LWIP_TCP_RECCVMBOX_SIZE multiples the maximum TCP socket number. In other words, the bigger LWIP_TCP_RECVMBOX_SIZE means more memory. On the other hand, if the receiv mail box is too small, the mail box may be full. If the mail box is full, the LWIP drops the packets. So generally we need to make sure the TCP receive mail box is big enough to avoid packet drop between LWIP core and application.

Range:
- from 6 to 1024 if *CONFIG_LWIP_WND_SCALE*

Default value:
- 6

CONFIG_LWIP_TCP_ACCEPTMBOX_SIZE

Default TCP accept mail box size

Found in: Component config > LWIP > TCP

Set TCP accept mail box size. Generally bigger value means supporting larger backlogs but more memory. The recommended value is 6, but applications can set it to a lower value if listening servers are meant to have a smaller backlog.

TCP accept mail box is a per socket mail box, when the application listens for connections with a given listening TCP socket. If the mailbox is full, LWIP will send a RST packet and the client will fail to connect.

Range:
- from 1 to 255 if *CONFIG_LWIP_WND_SCALE*

Default value:

- 6

CONFIG_LWIP_TCP_QUEUE_OOSEQ

Queue incoming out-of-order segments

Found in: Component config > LWIP > TCP

Queue incoming out-of-order segments for later use.

Disable this option to save some RAM during TCP sessions, at the expense of increased retransmissions if segments arrive out of order.

Default value:

- Yes (enabled)

CONFIG_LWIP_TCP_OOSEQ_TIMEOUT

Timeout for each pbuf queued in TCP OOSEQ, in RTOs.

Found in: Component config > LWIP > TCP > CONFIG_LWIP_TCP_QUEUE_OOSEQ

The timeout value is TCP_OOSEQ_TIMEOUT * RTO.

Range:

- from 1 to 30

Default value:

- 6

CONFIG_LWIP_TCP_OOSEQ_MAX_PBUFS

The maximum number of pbufs queued on OOSEQ per pcb

Found in: Component config > LWIP > TCP > CONFIG_LWIP_TCP_QUEUE_OOSEQ

If LWIP_TCP_OOSEQ_MAX_PBUFS = 0, TCP will not control the number of OOSEQ pbufs.

In a poor network environment, many out-of-order tcp pbufs will be received. These out-of-order pbufs will be cached in the TCP out-of-order queue which will cause Wi-Fi/Ethernet fail to release RX buffer in time. It is possible that all RX buffers for MAC layer are used by OOSEQ.

Control the number of out-of-order pbufs to ensure that the MAC layer has enough RX buffer to receive packets.

In the Wi-Fi scenario, recommended OOSEQ PBUFS Range: 0 <= TCP_OOSEQ_MAX_PBUFS <= CONFIG_ESP_WIFI_DYNAMIC_RX_BUFFER_NUM/(MAX_TCP_NUMBER + 1)

In the Ethernet scenario,recommended Ethernet OOSEQ PBUFS Range: 0 <= TCP_OOSEQ_MAX_PBUFS <= CONFIG_ETH_DMA_RX_BUFFER_NUM/(MAX_TCP_NUMBER + 1)

Within the recommended value range, the larger the value, the better the performance.

MAX_TCP_NUMBER represent Maximum number of TCP connections in Wi-Fi(STA+SoftAP) and Ethernet scenario.

Range:

- from 0 to 12

Default value:

- 0 if *CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP* && *CONFIG_LWIP_TCP_QUEUE_OOSEQ*

CONFIG_LWIP_TCP_SACK_OUT

Support sending selective acknowledgements

Found in: Component config > LWIP > TCP > CONFIG_LWIP_TCP_QUEUE_OOSEQ

TCP will support sending selective acknowledgements (SACKs).

Default value:

- No (disabled)

CONFIG_LWIP_TCP_OVERSIZE

Pre-allocate transmit PBUF size

Found in: Component config > LWIP > TCP

Allows enabling "oversize" allocation of TCP transmission pbufs ahead of time, which can reduce the length of pbuf chains used for transmission.

This will not make a difference to sockets where Nagle's algorithm is disabled.

Default value of MSS is fine for most applications, 25% MSS may save some RAM when only transmitting small amounts of data. Disabled will have worst performance and fragmentation characteristics, but uses least RAM overall.

Available options:

- MSS (CONFIG_LWIP_TCP_OVERSIZE_MSS)
- 25% MSS (CONFIG_LWIP_TCP_OVERSIZE_QUARTER_MSS)
- Disabled (CONFIG_LWIP_TCP_OVERSIZE_DISABLE)

CONFIG_LWIP_WND_SCALE

Support TCP window scale

Found in: Component config > LWIP > TCP

Enable this feature to support TCP window scaling.

Default value:

- No (disabled) if *CONFIG_SPIRAM_TRY_ALLOCATE_WIFI_LWIP*

CONFIG_LWIP_TCP_RCV_SCALE

Set TCP receiving window scaling factor

Found in: Component config > LWIP > TCP > CONFIG_LWIP_WND_SCALE

Enable this feature to support TCP window scaling.

Range:

- from 0 to 14 if *CONFIG_LWIP_WND_SCALE*

Default value:

- 0 if *CONFIG_LWIP_WND_SCALE*

CONFIG_LWIP_TCP_RTO_TIME

Default TCP rto time

Found in: Component config > LWIP > TCP

Set default TCP rto time for a reasonable initial rto. In bad network environment, recommend set value of rto time to 1500.

Default value:
- 1500

UDP Contains:

- *CONFIG_LWIP_UDP_RECVMBOX_SIZE*
- *CONFIG_LWIP_MAX_UDP_PCBS*

CONFIG_LWIP_MAX_UDP_PCBS

Maximum active UDP control blocks

Found in: Component config > LWIP > UDP

The maximum number of active UDP "connections" (ie UDP sockets sending/receiving data). The practical maximum limit is determined by available heap memory at runtime.

Range:
- from 1 to 1024

Default value:
- 16

CONFIG_LWIP_UDP_RECVMBOX_SIZE

Default UDP receive mail box size

Found in: Component config > LWIP > UDP

Set UDP receive mail box size. The recommended value is 6.

UDP receive mail box is a per socket mail box, when the application receives packets from UDP socket, LWIP core firstly posts the packets to UDP receive mail box and the application then fetches the packets from mail box. It means LWIP can caches maximum UDP_RECCVMBOX_SIZE packets for each UDP socket, so the maximum possible cached UDP packets for all UDP sockets is UDP_RECCVMBOX_SIZE multiples the maximum UDP socket number. In other words, the bigger UDP_RECVMBOX_SIZE means more memory. On the other hand, if the receiv mail box is too small, the mail box may be full. If the mail box is full, the LWIP drops the packets. So generally we need to make sure the UDP receive mail box is big enough to avoid packet drop between LWIP core and application.

Range:
- from 6 to 64

Default value:
- 6

Checksums Contains:

- *CONFIG_LWIP_CHECKSUM_CHECK_ICMP*
- *CONFIG_LWIP_CHECKSUM_CHECK_IP*
- *CONFIG_LWIP_CHECKSUM_CHECK_UDP*

CONFIG_LWIP_CHECKSUM_CHECK_IP

Enable LWIP IP checksums

Found in: Component config > LWIP > Checksums

Enable checksum checking for received IP messages

Default value:
- No (disabled)

CONFIG_LWIP_CHECKSUM_CHECK_UDP

Enable LWIP UDP checksums

Found in: Component config > LWIP > Checksums

Enable checksum checking for received UDP messages

Default value:

- No (disabled)

CONFIG_LWIP_CHECKSUM_CHECK_ICMP

Enable LWIP ICMP checksums

Found in: Component config > LWIP > Checksums

Enable checksum checking for received ICMP messages

Default value:

- Yes (enabled)

CONFIG_LWIP_TCPIP_TASK_STACK_SIZE

TCP/IP Task Stack Size

Found in: Component config > LWIP

Configure TCP/IP task stack size, used by LWIP to process multi-threaded TCP/IP operations. Setting this stack too small will result in stack overflow crashes.

Range:

- from 2048 to 65536

Default value:

- 3072

CONFIG_LWIP_TCPIP_TASK_AFFINITY

TCP/IP task affinity

Found in: Component config > LWIP

Allows setting LwIP tasks affinity, i.e. whether the task is pinned to CPU0, pinned to CPU1, or allowed to run on any CPU. Currently this applies to "TCP/IP" task and "Ping" task.

Available options:

- No affinity (CONFIG_LWIP_TCPIP_TASK_AFFINITY_NO_AFFINITY)
- CPU0 (CONFIG_LWIP_TCPIP_TASK_AFFINITY_CPU0)
- CPU1 (CONFIG_LWIP_TCPIP_TASK_AFFINITY_CPU1)

CONFIG_LWIP_PPP_SUPPORT

Enable PPP support

Found in: Component config > LWIP

Enable PPP stack. Now only PPP over serial is possible.

Default value:

- No (disabled)

Contains:

- *CONFIG_LWIP_PPP_ENABLE_IPV6*

CONFIG_LWIP_PPP_ENABLE_IPV6

Enable IPV6 support for PPP connections (IPV6CP)

Found in: Component config > LWIP > CONFIG_LWIP_PPP_SUPPORT

Enable IPV6 support in PPP for the local link between the DTE (processor) and DCE (modem). There are some modems which do not support the IPV6 addressing in the local link. If they are requested for IPV6CP negotiation, they may time out. This would in turn fail the configuration for the whole link. If your modem is not responding correctly to PPP Phase Network, try to disable IPV6 support.

Default value:

- Yes (enabled) if *CONFIG_LWIP_PPP_SUPPORT* && *CONFIG_LWIP_IPV6*

CONFIG_LWIP_IPV6_MEMP_NUM_ND6_QUEUE

Max number of IPv6 packets to queue during MAC resolution

Found in: Component config > LWIP

Config max number of IPv6 packets to queue during MAC resolution.

Range:

- from 3 to 20

Default value:

- 3

CONFIG_LWIP_IPV6_ND6_NUM_NEIGHBORS

Max number of entries in IPv6 neighbor cache

Found in: Component config > LWIP

Config max number of entries in IPv6 neighbor cache

Range:

- from 3 to 10

Default value:

- 5

CONFIG_LWIP_PPP_NOTIFY_PHASE_SUPPORT

Enable Notify Phase Callback

Found in: Component config > LWIP

Enable to set a callback which is called on change of the internal PPP state machine.

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_PAP_SUPPORT

Enable PAP support

Found in: Component config > LWIP

Enable Password Authentication Protocol (PAP) support

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_CHAP_SUPPORT

Enable CHAP support

Found in: Component config > LWIP

Enable Challenge Handshake Authentication Protocol (CHAP) support

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_MSCHAP_SUPPORT

Enable MSCHAP support

Found in: Component config > LWIP

Enable Microsoft version of the Challenge-Handshake Authentication Protocol (MSCHAP) support

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_MPPE_SUPPORT

Enable MPPE support

Found in: Component config > LWIP

Enable Microsoft Point-to-Point Encryption (MPPE) support

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_SERVER_SUPPORT

Enable PPP server support

Found in: Component config > LWIP

Enable to use PPP server

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_PPP_VJ_HEADER_COMPRESSION

Enable VJ IP Header compression

Found in: Component config > LWIP

Enable support for VJ header compression. Please disable this if you're using NAPT on PPP interface, since the compressed IP header might not be correctly interpreted in NAT causing the compressed packet to be dropped.

Default value:

- Yes (enabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_ENABLE_LCP_ECHO

Enable LCP ECHO

Found in: Component config > LWIP

Enable LCP echo keepalive requests

Default value:

- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_LCP_ECHOINTERVAL

Echo interval (s)

Found in: Component config > LWIP > CONFIG_LWIP_ENABLE_LCP_ECHO

Interval in seconds between keepalive LCP echo requests, 0 to disable.

Range:
- from 0 to 1000000 if *CONFIG_LWIP_ENABLE_LCP_ECHO*

Default value:
- 3 if *CONFIG_LWIP_ENABLE_LCP_ECHO*

CONFIG_LWIP_LCP_MAXECHOFAILS

Maximum echo failures

Found in: Component config > LWIP > CONFIG_LWIP_ENABLE_LCP_ECHO

Number of consecutive unanswered echo requests before failure is indicated.

Range:
- from 0 to 100000 if *CONFIG_LWIP_ENABLE_LCP_ECHO*

Default value:
- 3 if *CONFIG_LWIP_ENABLE_LCP_ECHO*

CONFIG_LWIP_PPP_DEBUG_ON

Enable PPP debug log output

Found in: Component config > LWIP

Enable PPP debug log output

Default value:
- No (disabled) if *CONFIG_LWIP_PPP_SUPPORT*

CONFIG_LWIP_SLIP_SUPPORT

Enable SLIP support (new/experimental)

Found in: Component config > LWIP

Enable SLIP stack. Now only SLIP over serial is possible.

SLIP over serial support is experimental and unsupported.

Default value:
- No (disabled)

Contains:

- *CONFIG_LWIP_SLIP_DEBUG_ON*

CONFIG_LWIP_SLIP_DEBUG_ON

Enable SLIP debug log output

Found in: Component config > LWIP > CONFIG_LWIP_SLIP_SUPPORT

Enable SLIP debug log output

Default value:
- No (disabled) if *CONFIG_LWIP_SLIP_SUPPORT*

ICMP Contains:

- *CONFIG_LWIP_ICMP*
- *CONFIG_LWIP_BROADCAST_PING*
- *CONFIG_LWIP_MULTICAST_PING*

CONFIG_LWIP_ICMP

ICMP: Enable ICMP

Found in: Component config > LWIP > ICMP

Enable ICMP module for check network stability

Default value:

- Yes (enabled)

CONFIG_LWIP_MULTICAST_PING

Respond to multicast pings

Found in: Component config > LWIP > ICMP

Default value:

- No (disabled)

CONFIG_LWIP_BROADCAST_PING

Respond to broadcast pings

Found in: Component config > LWIP > ICMP

Default value:

- No (disabled)

LWIP RAW API Contains:

- *CONFIG_LWIP_MAX_RAW_PCBS*

CONFIG_LWIP_MAX_RAW_PCBS

Maximum LWIP RAW PCBs

Found in: Component config > LWIP > LWIP RAW API

The maximum number of simultaneously active LWIP RAW protocol control blocks. The practical maximum limit is determined by available heap memory at runtime.

Range:

- from 1 to 1024

Default value:

- 16

SNTP Contains:

- *CONFIG_LWIP_SNTP_STARTUP_DELAY*
- *CONFIG_LWIP_SNTP_MAX_SERVERS*
- *CONFIG_LWIP_SNTP_UPDATE_DELAY*
- *CONFIG_LWIP_DHCP_GET_NTP_SRV*

CONFIG_LWIP_SNTP_MAX_SERVERS

Maximum number of NTP servers

Found in: Component config > LWIP > SNTP

Set maximum number of NTP servers used by LwIP SNTP module. First argument of sntp_setserver/sntp_setservername functions is limited to this value.

Range:
- from 1 to 16

Default value:
- 1

CONFIG_LWIP_DHCP_GET_NTP_SRV

Request NTP servers from DHCP

Found in: Component config > LWIP > SNTP

If enabled, LWIP will add 'NTP' to Parameter-Request Option sent via DHCP-request. DHCP server might reply with an NTP server address in option 42. SNTP callback for such replies should be set accordingly (see sntp_servermode_dhcp() func.)

Default value:
- No (disabled)

CONFIG_LWIP_DHCP_MAX_NTP_SERVERS

Maximum number of NTP servers aquired via DHCP

Found in: Component config > LWIP > SNTP > CONFIG_LWIP_DHCP_GET_NTP_SRV

Set maximum number of NTP servers aquired via DHCP-offer. Should be less or equal to "Maximum number of NTP servers", any extra servers would be just ignored.

Range:
- from 1 to 16 if *CONFIG_LWIP_DHCP_GET_NTP_SRV*

Default value:
- 1 if *CONFIG_LWIP_DHCP_GET_NTP_SRV*

CONFIG_LWIP_SNTP_UPDATE_DELAY

Request interval to update time (ms)

Found in: Component config > LWIP > SNTP

This option allows you to set the time update period via SNTP. Default is 1 hour. Must not be below 15 seconds by specification. (SNTPv4 RFC 4330 enforces a minimum update time of 15 seconds).

Range:
- from 15000 to 4294967295

Default value:
- 3600000

CONFIG_LWIP_SNTP_STARTUP_DELAY

Enable SNTP startup delay

Found in: Component config > LWIP > SNTP

It is recommended (RFC 4330) to delay the initial request after by a random timeout from 1 to 5 minutes to reduce potential load of NTP servers after simultaneous power-up of many devices. This option disables this initial delay. Please use this option with care, it could improve a single device responsiveness

but might cause peaks on the network after reset. Another option to address responsiveness of devices while using the initial random delay is to adjust LWIP_SNTP_MAXIMUM_STARTUP_DELAY.

Default value:

- Yes (enabled)

CONFIG_LWIP_SNTP_MAXIMUM_STARTUP_DELAY

Maximum startup delay (ms)

Found in: Component config > LWIP > SNTP > CONFIG_LWIP_SNTP_STARTUP_DELAY

RFC 4330 recommends a startup delay before sending the initial request. LWIP calculates this delay to a random number of milliseconds between 0 and this value.

Range:

- from 100 to 300000

Default value:

- 5000

DNS Contains:

- *CONFIG_LWIP_FALLBACK_DNS_SERVER_SUPPORT*
- *CONFIG_LWIP_DNS_MAX_SERVERS*

CONFIG_LWIP_DNS_MAX_SERVERS

Maximum number of DNS servers

Found in: Component config > LWIP > DNS

Set maximum number of DNS servers. If fallback DNS servers are supported, the number of DNS servers needs to be greater than or equal to 3.

Range:

- from 1 to 4

Default value:

- 3

CONFIG_LWIP_FALLBACK_DNS_SERVER_SUPPORT

Enable DNS fallback server support

Found in: Component config > LWIP > DNS

Enable this feature to support DNS fallback server.

Default value:

- No (disabled)

CONFIG_LWIP_FALLBACK_DNS_SERVER_ADDRESS

DNS fallback server address

Found in: Component config > LWIP > DNS > CONFIG_LWIP_FALLBACK_DNS_SERVER_SUPPORT

This option allows you to config dns fallback server address.

Default value:

- "114.114.114.114" if *CONFIG_LWIP_FALLBACK_DNS_SERVER_SUPPORT*

CONFIG_LWIP_BRIDGEIF_MAX_PORTS

Maximum number of bridge ports

Found in: Component config > LWIP

Set maximum number of ports a bridge can consists of.

Range:
- from 1 to 63

Default value:
- 7

CONFIG_LWIP_ESP_LWIP_ASSERT

Enable LWIP ASSERT checks

Found in: Component config > LWIP

Enable this option keeps LWIP assertion checks enabled. It is recommended to keep this option enabled.

If asserts are disabled for the entire project, they are also disabled for LWIP and this option is ignored.

Hooks Contains:

- *CONFIG_LWIP_HOOK_ND6_GET_GW*
- *CONFIG_LWIP_HOOK_IP6_INPUT*
- *CONFIG_LWIP_HOOK_IP6_ROUTE*
- *CONFIG_LWIP_HOOK_IP6_SELECT_SRC_ADDR*
- *CONFIG_LWIP_HOOK_NETCONN_EXTERNAL_RESOLVE*
- *CONFIG_LWIP_HOOK_TCP_ISN*

CONFIG_LWIP_HOOK_TCP_ISN

TCP ISN Hook

Found in: Component config > LWIP > Hooks

Enables to define a TCP ISN hook to randomize initial sequence number in TCP connection. The default TCP ISN algorithm used in IDF (standardized in RFC 6528) produces ISN by combining an MD5 of the new TCP id and a stable secret with the current time. This is because the lwIP implementation (*tcp_next_iss*) is not very strong, as it does not take into consideration any platform specific entropy source.

Set to LWIP_HOOK_TCP_ISN_CUSTOM to provide custom implementation. Set to LWIP_HOOK_TCP_ISN_NONE to use lwIP implementation.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_TCP_ISN_NONE)
- Default implementation (CONFIG_LWIP_HOOK_TCP_ISN_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_TCP_ISN_CUSTOM)

CONFIG_LWIP_HOOK_IP6_ROUTE

IPv6 route Hook

Found in: Component config > LWIP > Hooks

Enables custom IPv6 route hook. Setting this to "default" provides weak implementation stub that could be overwritten in application code. Setting this to "custom" provides hook's declaration only and expects the application to implement it.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_IP6_ROUTE_NONE)
- Default (weak) implementation (CONFIG_LWIP_HOOK_IP6_ROUTE_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_IP6_ROUTE_CUSTOM)

CONFIG_LWIP_HOOK_ND6_GET_GW

IPv6 get gateway Hook

Found in: Component config > LWIP > Hooks

Enables custom IPv6 route hook. Setting this to "default" provides weak implementation stub that could be overwritten in application code. Setting this to "custom" provides hook's declaration only and expects the application to implement it.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_ND6_GET_GW_NONE)
- Default (weak) implementation (CONFIG_LWIP_HOOK_ND6_GET_GW_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_ND6_GET_GW_CUSTOM)

CONFIG_LWIP_HOOK_IP6_SELECT_SRC_ADDR

IPv6 source address selection Hook

Found in: Component config > LWIP > Hooks

Enables custom IPv6 source address selection. Setting this to "default" provides weak implementation stub that could be overwritten in application code. Setting this to "custom" provides hook's declaration only and expects the application to implement it.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_IP6_SELECT_SRC_ADDR_NONE)
- Default (weak) implementation (CONFIG_LWIP_HOOK_IP6_SELECT_SRC_ADDR_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_IP6_SELECT_SRC_ADDR_CUSTOM)

CONFIG_LWIP_HOOK_NETCONN_EXTERNAL_RESOLVE

Netconn external resolve Hook

Found in: Component config > LWIP > Hooks

Enables custom DNS resolve hook. Setting this to "default" provides weak implementation stub that could be overwritten in application code. Setting this to "custom" provides hook's declaration only and expects the application to implement it.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_NETCONN_EXT_RESOLVE_NONE)
- Default (weak) implementation (CONFIG_LWIP_HOOK_NETCONN_EXT_RESOLVE_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_NETCONN_EXT_RESOLVE_CUSTOM)

CONFIG_LWIP_HOOK_IP6_INPUT

IPv6 packet input

Found in: Component config > LWIP > Hooks

Enables custom IPv6 packet input. Setting this to "default" provides weak implementation stub that could be overwritten in application code. Setting this to "custom" provides hook's declaration only and expects the application to implement it.

Available options:

- No hook declared (CONFIG_LWIP_HOOK_IP6_INPUT_NONE)
- Default (weak) implementation (CONFIG_LWIP_HOOK_IP6_INPUT_DEFAULT)
- Custom implementation (CONFIG_LWIP_HOOK_IP6_INPUT_CUSTOM)

CONFIG_LWIP_DEBUG

Enable LWIP Debug

Found in: Component config > LWIP

Enabling this option allows different kinds of lwIP debug output.

All lwIP debug features increase the size of the final binary.

Default value:

- No (disabled)

Contains:

- *CONFIG_LWIP_API_LIB_DEBUG*
- *CONFIG_LWIP_BRIDGEIF_FDB_DEBUG*
- *CONFIG_LWIP_BRIDGEIF_FW_DEBUG*
- *CONFIG_LWIP_BRIDGEIF_DEBUG*
- *CONFIG_LWIP_DHCP_DEBUG*
- *CONFIG_LWIP_DHCP_STATE_DEBUG*
- *CONFIG_LWIP_DNS_DEBUG*
- *CONFIG_LWIP_ETHARP_DEBUG*
- *CONFIG_LWIP_ICMP_DEBUG*
- *CONFIG_LWIP_ICMP6_DEBUG*
- *CONFIG_LWIP_IP_DEBUG*
- *CONFIG_LWIP_IP6_DEBUG*
- *CONFIG_LWIP_NAPT_DEBUG*
- *CONFIG_LWIP_NETIF_DEBUG*
- *CONFIG_LWIP_PBUF_DEBUG*
- *CONFIG_LWIP_SNTP_DEBUG*
- *CONFIG_LWIP_SOCKETS_DEBUG*
- *CONFIG_LWIP_TCP_DEBUG*
- *CONFIG_LWIP_UDP_DEBUG*
- *CONFIG_LWIP_DEBUG_ESP_LOG*

CONFIG_LWIP_DEBUG_ESP_LOG

Route LWIP debugs through ESP_LOG interface

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Enabling this option routes all enabled LWIP debugs through ESP_LOGD.

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_NETIF_DEBUG

Enable netif debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_PBUF_DEBUG

Enable pbuf debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_ETHARP_DEBUG

Enable etharp debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_API_LIB_DEBUG

Enable api lib debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_SOCKETS_DEBUG

Enable socket debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_IP_DEBUG

Enable IP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_ICMP_DEBUG

Enable ICMP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG* && *CONFIG_LWIP_ICMP*

CONFIG_LWIP_DHCP_STATE_DEBUG

Enable DHCP state tracking

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_DHCP_DEBUG

Enable DHCP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_IP6_DEBUG

Enable IP6 debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_ICMP6_DEBUG

Enable ICMP6 debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_TCP_DEBUG

Enable TCP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_UDP_DEBUG

Enable UDP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_SNTP_DEBUG

Enable SNTP debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_DNS_DEBUG

Enable DNS debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_NAPT_DEBUG

Enable NAPT debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG* && *CONFIG_LWIP_IPV4_NAPT*

CONFIG_LWIP_BRIDGEIF_DEBUG

Enable bridge generic debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_BRIDGEIF_FDB_DEBUG

Enable bridge FDB debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

CONFIG_LWIP_BRIDGEIF_FW_DEBUG

Enable bridge forwarding debug messages

Found in: Component config > LWIP > CONFIG_LWIP_DEBUG

Default value:

- No (disabled) if *CONFIG_LWIP_DEBUG*

mbedTLS Contains:

- *CONFIG_MBEDTLS_ASYMMETRIC_CONTENT_LEN*
- *Certificate Bundle*
- *Certificates*
- *CONFIG_MBEDTLS_CHACHA20_C*
- *CONFIG_MBEDTLS_DHM_C*
- *CONFIG_MBEDTLS_ECP_C*
- *CONFIG_MBEDTLS_ECDH_C*
- *CONFIG_MBEDTLS_ECJPAKE_C*
- *CONFIG_MBEDTLS_ECP_DP_BP256R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_BP384R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_BP512R1_ENABLED*
- *CONFIG_MBEDTLS_CMAC_C*
- *CONFIG_MBEDTLS_ECP_DP_CURVE25519_ENABLED*
- *CONFIG_MBEDTLS_ECDSA_DETERMINISTIC*
- *CONFIG_MBEDTLS_HARDWARE_ECDSA_VERIFY*
- *CONFIG_MBEDTLS_HARDWARE_ECDSA_SIGN*

- *CONFIG_MBEDTLS_ERROR_STRINGS*
- *CONFIG_MBEDTLS_ECP_FIXED_POINT_OPTIM*
- *CONFIG_MBEDTLS_HARDWARE_AES*
- *CONFIG_MBEDTLS_HARDWARE_ECC*
- *CONFIG_MBEDTLS_ATCA_HW_ECDSA_SIGN*
- *CONFIG_MBEDTLS_ATCA_HW_ECDSA_VERIFY*
- *CONFIG_MBEDTLS_HARDWARE_MPI*
- *CONFIG_MBEDTLS_HARDWARE_SHA*
- *CONFIG_MBEDTLS_DEBUG*
- *CONFIG_MBEDTLS_ECP_RESTARTABLE*
- *CONFIG_MBEDTLS_HAVE_TIME*
- *CONFIG_MBEDTLS_RIPEMD160_C*
- *CONFIG_MBEDTLS_ECP_DP_SECP192K1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP192R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP224K1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP224R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP256K1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP256R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP384R1_ENABLED*
- *CONFIG_MBEDTLS_ECP_DP_SECP521R1_ENABLED*
- *CONFIG_MBEDTLS_SHA512_C*
- *CONFIG_MBEDTLS_THREADING_C*
- *CONFIG_MBEDTLS_HKDF_C*
- *mbedTLS v3.x related*
- *CONFIG_MBEDTLS_MEM_ALLOC_MODE*
- *CONFIG_MBEDTLS_ECP_NIST_OPTIM*
- *CONFIG_MBEDTLS_POLY1305_C*
- *CONFIG_MBEDTLS_SSL_ALPN*
- *CONFIG_MBEDTLS_SSL_PROTO_DTLS*
- *CONFIG_MBEDTLS_SSL_PROTO_GMTSSL1_1*
- *CONFIG_MBEDTLS_SSL_PROTO_TLS1_2*
- *CONFIG_MBEDTLS_SSL_RENEGOTIATION*
- *Symmetric Ciphers*
- *TLS Key Exchange Methods*
- *CONFIG_MBEDTLS_SSL_MAX_CONTENT_LEN*
- *CONFIG_MBEDTLS_TLS_MODE*
- *CONFIG_MBEDTLS_CLIENT_SSL_SESSION_TICKETS*
- *CONFIG_MBEDTLS_SERVER_SSL_SESSION_TICKETS*
- *CONFIG_MBEDTLS_ROM_MD5*
- *CONFIG_MBEDTLS_USE_CRYPTO_ROM_IMPL*
- *CONFIG_MBEDTLS_DYNAMIC_BUFFER*

CONFIG_MBEDTLS_MEM_ALLOC_MODE

Memory allocation strategy

Found in: Component config > mbedTLS

Allocation strategy for mbedTLS, essentially provides ability to allocate all required dynamic allocations from,

- Internal DRAM memory only
- External SPIRAM memory only
- Either internal or external memory based on default malloc() behavior in ESP-IDF
- Custom allocation mode, by overwriting calloc()/free() using mbedtls_platform_set_calloc_free() function
- Internal IRAM memory wherever applicable else internal DRAM

Recommended mode here is always internal (*), since that is most preferred from security perspective. But if application requirement does not allow sufficient free internal memory then alternate mode can

be selected.

(*) In case of ESP32-S2/ESP32-S3, hardware allows encryption of external SPIRAM contents provided hardware flash encryption feature is enabled. In that case, using external SPIRAM allocation strategy is also safe choice from security perspective.

Available options:

- Internal memory (CONFIG_MBEDTLS_INTERNAL_MEM_ALLOC)
- External SPIRAM (CONFIG_MBEDTLS_EXTERNAL_MEM_ALLOC)
- Default alloc mode (CONFIG_MBEDTLS_DEFAULT_MEM_ALLOC)
- Custom alloc mode (CONFIG_MBEDTLS_CUSTOM_MEM_ALLOC)
- Internal IRAM (CONFIG_MBEDTLS_IRAM_8BIT_MEM_ALLOC)
  Allows to use IRAM memory region as 8bit accessible region.
  TLS input and output buffers will be allocated in IRAM section which is 32bit aligned memory. Every unaligned (8bit or 16bit) access will result in an exception and incur penalty of certain clock cycles per unaligned read/write.

CONFIG_MBEDTLS_SSL_MAX_CONTENT_LEN

TLS maximum message content length

Found in: Component config > mbedTLS

Maximum TLS message length (in bytes) supported by mbedTLS.

16384 is the default and this value is required to comply fully with TLS standards.

However you can set a lower value in order to save RAM. This is safe if the other end of the connection supports Maximum Fragment Length Negotiation Extension (max_fragment_length, see RFC6066) or you know for certain that it will never send a message longer than a certain number of bytes.

If the value is set too low, symptoms are a failed TLS handshake or a return value of MBEDTLS_ERR_SSL_INVALID_RECORD (-0x7200).

CONFIG_MBEDTLS_ASYMMETRIC_CONTENT_LEN

Asymmetric in/out fragment length

Found in: Component config > mbedTLS

If enabled, this option allows customizing TLS in/out fragment length in asymmetric way. Please note that enabling this with default values saves 12KB of dynamic memory per TLS connection.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SSL_IN_CONTENT_LEN

TLS maximum incoming fragment length

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_ASYMMETRIC_CONTENT_LEN

This defines maximum incoming fragment length, overriding default maximum content length (MBEDTLS_SSL_MAX_CONTENT_LEN).

Range:

- from 512 to 16384

Default value:

- 16384

CONFIG_MBEDTLS_SSL_OUT_CONTENT_LEN

TLS maximum outgoing fragment length

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_ASYMMETRIC_CONTENT_LEN

This defines maximum outgoing fragment length, overriding default maximum content length (MBEDTLS_SSL_MAX_CONTENT_LEN).

Range:
- from 512 to 16384

Default value:
- 4096

CONFIG_MBEDTLS_DYNAMIC_BUFFER

Using dynamic TX/RX buffer

Found in: Component config > mbedTLS

Using dynamic TX/RX buffer. After enabling this option, mbedTLS will allocate TX buffer when need to send data and then free it if all data is sent, allocate RX buffer when need to receive data and then free it when all data is used or read by upper layer.

By default, when SSL is initialized, mbedTLS also allocate TX and RX buffer with the default value of "MBEDTLS_SSL_OUT_CONTENT_LEN" or "MBEDTLS_SSL_IN_CONTENT_LEN", so to save more heap, users can set the options to be an appropriate value.

CONFIG_MBEDTLS_DYNAMIC_FREE_CONFIG_DATA

Free private key and DHM data after its usage

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_DYNAMIC_BUFFER

Free private key and DHM data after its usage in handshake process.

The option will decrease heap cost when handshake, but also lead to problem:

Because all certificate, private key and DHM data are freed so users should register certificate and private key to ssl config object again.

Default value:
- No (disabled) if *CONFIG_MBEDTLS_DYNAMIC_BUFFER*

CONFIG_MBEDTLS_DYNAMIC_FREE_CA_CERT

Free SSL CA certificate after its usage

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_DYNAMIC_BUFFER > CONFIG_MBEDTLS_DYNAMIC_FREE_CONFIG_DATA

Free CA certificate after its usage in the handshake process. This option will decrease the heap footprint for the TLS handshake, but may lead to a problem: If the respective ssl object needs to perform the TLS handshake again, the CA certificate should once again be registered to the ssl object.

Default value:
- Yes (enabled) if *CONFIG_MBEDTLS_DYNAMIC_FREE_CONFIG_DATA*

CONFIG_MBEDTLS_DEBUG

Enable mbedTLS debugging

Found in: Component config > mbedTLS

Enable mbedTLS debugging functions at compile time.

If this option is enabled, you can include "mbedtls/esp_debug.h" and call mbedtls_esp_enable_debug_log() at runtime in order to enable mbedTLS debug output via the ESP log mechanism.

Default value:

- No (disabled)

CONFIG_MBEDTLS_DEBUG_LEVEL

Set mbedTLS debugging level

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_DEBUG

Set mbedTLS debugging level

Available options:

- Warning (CONFIG_MBEDTLS_DEBUG_LEVEL_WARN)
- Info (CONFIG_MBEDTLS_DEBUG_LEVEL_INFO)
- Debug (CONFIG_MBEDTLS_DEBUG_LEVEL_DEBUG)
- Verbose (CONFIG_MBEDTLS_DEBUG_LEVEL_VERBOSE)

mbedTLS v3.x related

Contains:

- *DTLS-based configurations*
- *CONFIG_MBEDTLS_PKCS7_C*
- *CONFIG_MBEDTLS_SSL_CONTEXT_SERIALIZATION*
- *CONFIG_MBEDTLS_X509_TRUSTED_CERT_CALLBACK*
- *CONFIG_MBEDTLS_SSL_KEEP_PEER_CERTIFICATE*
- *CONFIG_MBEDTLS_SSL_CID_PADDING_GRANULARITY*
- *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*
- *CONFIG_MBEDTLS_ECDH_LEGACY_CONTEXT*
- *CONFIG_MBEDTLS_SSL_VARIABLE_BUFFER_LENGTH*

CONFIG_MBEDTLS_SSL_PROTO_TLS1_3

Support TLS 1.3 protocol

Found in: Component config > mbedTLS > mbedTLS v3.x related

TLS 1.3 related configurations

Contains:

- *CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_EPHEMERAL*
- *CONFIG_MBEDTLS_SSL_TLS1_3_COMPATIBILITY_MODE*
- *CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_PSK_EPHEMERAL*
- *CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_PSK*

CONFIG_MBEDTLS_SSL_TLS1_3_COMPATIBILITY_MODE

TLS 1.3 middlebox compatibility mode

Found in: Component config > mbedTLS > mbedTLS v3.x related > CONFIG_MBEDTLS_SSL_PROTO_TLS1_3 > TLS 1.3 related configurations

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*

CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_PSK

TLS 1.3 PSK key exchange mode

Found in: Component config > mbedTLS > mbedTLS v3.x related > CONFIG_MBEDTLS_SSL_PROTO_TLS1_3 > TLS 1.3 related configurations

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*

CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_EPHEMERAL

TLS 1.3 ephemeral key exchange mode

Found in: Component config > mbedTLS > mbedTLS v3.x related > CONFIG_MBEDTLS_SSL_PROTO_TLS1_3 > TLS 1.3 related configurations

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*

CONFIG_MBEDTLS_SSL_TLS1_3_KEXM_PSK_EPHEMERAL

TLS 1.3 PSK ephemeral key exchange mode

Found in: Component config > mbedTLS > mbedTLS v3.x related > CONFIG_MBEDTLS_SSL_PROTO_TLS1_3 > TLS 1.3 related configurations

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3*

CONFIG_MBEDTLS_SSL_VARIABLE_BUFFER_LENGTH

Variable SSL buffer length

Found in: Component config > mbedTLS > mbedTLS v3.x related

This enables the SSL buffer to be resized automatically based on the negotiated maximum fragment length in each direction.

Default value:

- No (disabled)

CONFIG_MBEDTLS_ECDH_LEGACY_CONTEXT

Use a backward compatible ECDH context (Experimental)

Found in: Component config > mbedTLS > mbedTLS v3.x related

Use the legacy ECDH context format. Define this option only if you enable MBEDTLS_ECP_RESTARTABLE or if you want to access ECDH context fields directly.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_ECDH_C* && *CONFIG_MBEDTLS_ECP_RESTARTABLE*

CONFIG_MBEDTLS_X509_TRUSTED_CERT_CALLBACK

Enable trusted certificate callbacks

Found in: Component config > mbedTLS > mbedTLS v3.x related

Enables users to configure the set of trusted certificates through a callback instead of a linked list.

See mbedTLS documentation for required API and more details.

Default value:

- No (disabled)

CONFIG_MBEDTLS_SSL_CONTEXT_SERIALIZATION

Enable serialization of the TLS context structures

Found in: Component config > mbedTLS > mbedTLS v3.x related

Enable serialization of the TLS context structures This is a local optimization in handling a single, potentially long-lived connection.

See mbedTLS documentation for required API and more details. Disabling this option will save some code size.

Default value:

- No (disabled)

CONFIG_MBEDTLS_SSL_KEEP_PEER_CERTIFICATE

Keep peer certificate after handshake completion

Found in: Component config > mbedTLS > mbedTLS v3.x related

Keep the peer's certificate after completion of the handshake. Disabling this option will save about 4kB of heap and some code size.

See mbedTLS documentation for required API and more details.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_PKCS7_C

Enable PKCS #7

Found in: Component config > mbedTLS > mbedTLS v3.x related

Enable PKCS #7 core for using PKCS #7-formatted signatures.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SSL_CID_PADDING_GRANULARITY

Record plaintext padding

Found in: Component config > mbedTLS > mbedTLS v3.x related

Controls the use of record plaintext padding in TLS 1.3 and when using the Connection ID extension in DTLS 1.2.

The padding will always be chosen so that the length of the padded plaintext is a multiple of the value of this option.

Notes: A value of 1 means that no padding will be used for outgoing records. On systems lacking division instructions, a power of two should be preferred.

Range:

- from 0 to 32 if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3* || *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID*

Default value:

- 16 if *CONFIG_MBEDTLS_SSL_PROTO_TLS1_3* || *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID*

DTLS-based configurations Contains:

- *CONFIG_MBEDTLS_SSL_DTLS_SRTP*
- *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID*

CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID

Support for the DTLS Connection ID extension

Found in: Component config > mbedTLS > mbedTLS v3.x related > DTLS-based configurations

Enable support for the DTLS Connection ID extension which allows to identify DTLS connections across changes in the underlying transport.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

CONFIG_MBEDTLS_SSL_CID_IN_LEN_MAX

Maximum length of CIDs used for incoming DTLS messages

Found in: Component config > mbedTLS > mbedTLS v3.x related > DTLS-based configurations > CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID

Maximum length of CIDs used for incoming DTLS messages

Range:

- from 0 to 32 if *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID* && *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

Default value:

- 32 if *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID* && *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

CONFIG_MBEDTLS_SSL_CID_OUT_LEN_MAX

Maximum length of CIDs used for outgoing DTLS messages

Found in: Component config > mbedTLS > mbedTLS v3.x related > DTLS-based configurations > CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID

Maximum length of CIDs used for outgoing DTLS messages

Range:

- from 0 to 32 if *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID* && *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

Default value:

- 32 if *CONFIG_MBEDTLS_SSL_DTLS_CONNECTION_ID* && *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

CONFIG_MBEDTLS_SSL_DTLS_SRTP

Enable support for negotiation of DTLS-SRTP (RFC 5764)

Found in: Component config > mbedTLS > mbedTLS v3.x related > DTLS-based configurations

Enable support for negotiation of DTLS-SRTP (RFC 5764) through the use_srtp extension.

See mbedTLS documentation for required API and more details. Disabling this option will save some code size.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_SSL_PROTO_DTLS*

Certificate Bundle Contains:

- *CONFIG_MBEDTLS_CERTIFICATE_BUNDLE*

CONFIG_MBEDTLS_CERTIFICATE_BUNDLE

Enable trusted root certificate bundle

Found in: Component config > mbedTLS > Certificate Bundle

Enable support for large number of default root certificates

When enabled this option allows user to store default as well as customer specific root certificates in compressed format rather than storing full certificate. For the root certificates the public key and the subject name will be stored.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_DEFAULT_CERTIFICATE_BUNDLE

Default certificate bundle options

Found in: Component config > mbedTLS > Certificate Bundle > CONFIG_MBEDTLS_CERTIFICATE_BUNDLE

Available options:

- Use the full default certificate bundle (CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEFAULT_FULL)
- Use only the most common certificates from the default bundles (CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEFAULT_CMN)
  Use only the most common certificates from the default bundles, reducing the size with 50%, while still having around 99% coverage.
- Do not use the default certificate bundle (CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEFAULT_NONE)

CONFIG_MBEDTLS_CUSTOM_CERTIFICATE_BUNDLE

Add custom certificates to the default bundle

Found in: Component config > mbedTLS > Certificate Bundle > CONFIG_MBEDTLS_CERTIFICATE_BUNDLE

Default value:

- No (disabled)

CONFIG_MBEDTLS_CUSTOM_CERTIFICATE_BUNDLE_PATH

Custom certificate bundle path

Found in: Component config > mbedTLS > Certificate Bundle > CONFIG_MBEDTLS_CERTIFICATE_BUNDLE > CONFIG_MBEDTLS_CUSTOM_CERTIFICATE_BUNDLE

Name of the custom certificate directory or file. This path is evaluated relative to the project root directory.

CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_DEPRECATED_LIST

Add deprecated root certificates

Found in: Component config > mbedTLS > Certificate Bundle > CONFIG_MBEDTLS_CERTIFICATE_BUNDLE

Include the deprecated list of root certificates in the bundle. This list gets updated when a certificate is removed from the Mozilla's NSS root certificate store. This config can be enabled if you would like to ensure that none of the certificates that were deployed in the product are affected because of the update to bundle. In turn, enabling this config keeps expired, retracted certificates in the bundle and it may pose a security risk.

- Deprecated cert list may grow based based on sync with upstream bundle
- Deprecated certs would be be removed in ESP-IDF (next) major release

CONFIG_MBEDTLS_CERTIFICATE_BUNDLE_MAX_CERTS

Maximum no of certificates allowed in certificate bundle

Found in: Component config > mbedTLS > Certificate Bundle > CONFIG_MBEDTLS_CERTIFICATE_BUNDLE

Default value:

- 200

CONFIG_MBEDTLS_ECP_RESTARTABLE

Enable mbedTLS ecp restartable

Found in: Component config > mbedTLS

Enable "non-blocking" ECC operations that can return early and be resumed.

Default value:

- No (disabled)

CONFIG_MBEDTLS_CMAC_C

Enable CMAC mode for block ciphers

Found in: Component config > mbedTLS

Enable the CMAC (Cipher-based Message Authentication Code) mode for block ciphers.

Default value:

- No (disabled)

CONFIG_MBEDTLS_HARDWARE_AES

Enable hardware AES acceleration

Found in: Component config > mbedTLS

Enable hardware accelerated AES encryption & decryption.

Note that if the ESP32 CPU is running at 240MHz, hardware AES does not offer any speed boost over software AES.

CONFIG_MBEDTLS_AES_USE_INTERRUPT

Use interrupt for long AES operations

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_AES

Use an interrupt to coordinate long AES operations.

This allows other code to run on the CPU while an AES operation is pending. Otherwise the CPU busy-waits.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_AES_INTERRUPT_LEVEL

AES hardware interrupt level

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_AES > CONFIG_MBEDTLS_AES_USE_INTERRUPT

This config helps to set the interrupt priority level for the AES peripheral. Value 0 (default) means that there is no preference regarding the interrupt priority level and any level from 1 to 3 can be selected (based on the availability). Note: Higher value indicates high interrupt priority.

Range:

- from 0 to 3

Default value:

- 0

CONFIG_MBEDTLS_HARDWARE_GCM

Enable partially hardware accelerated GCM

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_AES

Enable partially hardware accelerated GCM. GHASH calculation is still done in software.

If MBEDTLS_HARDWARE_GCM is disabled and MBEDTLS_HARDWARE_AES is enabled then mbedTLS will still use the hardware accelerated AES block operation, but on a single block at a time.

Default value:

- Yes (enabled) if SOC_AES_SUPPORT_GCM && *CONFIG_MBEDTLS_HARDWARE_AES*

CONFIG_MBEDTLS_GCM_SUPPORT_NON_AES_CIPHER

Enable support for non-AES ciphers in GCM operation

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_AES

Enable this config to support fallback to software definitions for a non-AES cipher GCM operation as we support hardware acceleration only for AES cipher. Some of the non-AES ciphers used in a GCM operation are DES, ARIA, CAMELLIA, CHACHA20, BLOWFISH.

If this config is disabled, performing a non-AES cipher GCM operation with the config MBEDTLS_HARDWARE_AES enabled will result in calculation of an AES-GCM operation instead for the given input values and thus could lead to failure in certificate validation which would ultimately lead to a SSL handshake failure.

This config being by-default enabled leads to an increase in binary size footprint of ~2.5KB. In case you are sure that your use case (for example, client and server configurations in case of a TLS handshake) would not involve any GCM operations using a non-AES cipher, you can safely disable this config, leading to reduction in binary size footprint.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_HARDWARE_MPI

Enable hardware MPI (bignum) acceleration

Found in: Component config > mbedTLS

Enable hardware accelerated multiple precision integer operations.

Hardware accelerated multiplication, modulo multiplication, and modular exponentiation for up to SOC_RSA_MAX_BIT_LEN bit results.

These operations are used by RSA.

CONFIG_MBEDTLS_LARGE_KEY_SOFTWARE_MPI

Fallback to software implementation for larger MPI values

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_MPI

Fallback to software implementation for RSA key lengths larger than SOC_RSA_MAX_BIT_LEN. If this is not active then the ESP will be unable to process keys greater than SOC_RSA_MAX_BIT_LEN.

Default value:

- No (disabled)

CONFIG_MBEDTLS_MPI_USE_INTERRUPT

Use interrupt for MPI exp-mod operations

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_MPI

Use an interrupt to coordinate long MPI operations.

This allows other code to run on the CPU while an MPI operation is pending. Otherwise the CPU busy-waits.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_MPI_INTERRUPT_LEVEL

MPI hardware interrupt level

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_MPI > CONFIG_MBEDTLS_MPI_USE_INTERRUPT

This config helps to set the interrupt priority level for the MPI peripheral. Value 0 (default) means that there is no preference regarding the interrupt priority level and any level from 1 to 3 can be selected (based on the availability). Note: Higher value indicates high interrupt priority.

Range:

- from 0 to 3

Default value:

- 0

CONFIG_MBEDTLS_HARDWARE_SHA

Enable hardware SHA acceleration

Found in: Component config > mbedTLS

Enable hardware accelerated SHA1, SHA256, SHA384 & SHA512 in mbedTLS.

Due to a hardware limitation, on the ESP32 hardware acceleration is only guaranteed if SHA digests are calculated one at a time. If more than one SHA digest is calculated at the same time, one will be

calculated fully in hardware and the rest will be calculated (at least partially calculated) in software. This happens automatically.

SHA hardware acceleration is faster than software in some situations but slower in others. You should benchmark to find the best setting for you.

CONFIG_MBEDTLS_HARDWARE_ECC

Enable hardware ECC acceleration

Found in: Component config > mbedTLS

Enable hardware accelerated ECC point multiplication and point verification for points on curve SECP192R1 and SECP256R1 in mbedTLS

Default value:

- Yes (enabled) if SOC_ECC_SUPPORTED

CONFIG_MBEDTLS_ECC_OTHER_CURVES_SOFT_FALLBACK

Fallback to software implementation for curves not supported in hardware

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HARDWARE_ECC

Fallback to software implementation of ECC point multiplication and point verification for curves not supported in hardware.

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_HARDWARE_ECC*

CONFIG_MBEDTLS_ROM_MD5

Use MD5 implementation in ROM

Found in: Component config > mbedTLS

Use ROM MD5 in mbedTLS.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_HARDWARE_ECDSA_SIGN

Enable ECDSA signing using on-chip ECDSA peripheral

Found in: Component config > mbedTLS

Enable hardware accelerated ECDSA peripheral to sign data on curve SECP192R1 and SECP256R1 in mbedTLS.

Note that for signing, the private key has to be burnt in an efuse key block with key purpose set to ECDSA_KEY. If no key is burnt, it will report an error

The key should be burnt in little endian format. espefuse.py utility handles it internally but care needs to be taken while burning using esp_efuse APIs

Default value:

- No (disabled) if SOC_ECDSA_SUPPORTED

CONFIG_MBEDTLS_HARDWARE_ECDSA_VERIFY

Enable ECDSA signature verification using on-chip ECDSA peripheral

Found in: Component config > mbedTLS

Enable hardware accelerated ECDSA peripheral to verify signature on curve SECP192R1 and SECP256R1 in mbedTLS.

Default value:

- Yes (enabled) if SOC_ECDSA_SUPPORTED

CONFIG_MBEDTLS_ATCA_HW_ECDSA_SIGN

Enable hardware ECDSA sign acceleration when using ATECC608A

Found in: Component config > mbedTLS

This option enables hardware acceleration for ECDSA sign function, only when using ATECC608A cryptoauth chip.

Default value:

- No (disabled)

CONFIG_MBEDTLS_ATCA_HW_ECDSA_VERIFY

Enable hardware ECDSA verify acceleration when using ATECC608A

Found in: Component config > mbedTLS

This option enables hardware acceleration for ECDSA sign function, only when using ATECC608A cryptoauth chip.

Default value:

- No (disabled)

CONFIG_MBEDTLS_HAVE_TIME

Enable mbedtls time support

Found in: Component config > mbedTLS

Enable use of time.h functions (time() and gmtime()) by mbedTLS.

This option doesn't require the system time to be correct, but enables functionality that requires relative timekeeping - for example periodic expiry of TLS session tickets or session cache entries.

Disabling this option will save some firmware size, particularly if the rest of the firmware doesn't call any standard timekeeeping functions.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_PLATFORM_TIME_ALT

Enable mbedtls time support: platform-specific

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HAVE_TIME

Enabling this config will provide users with a function "mbedtls_platform_set_time()" that allows to set an alternative time function pointer.

Default value:

- No (disabled)

CONFIG_MBEDTLS_HAVE_TIME_DATE

Enable mbedtls certificate expiry check

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_HAVE_TIME

Enables X.509 certificate expiry checks in mbedTLS.

If this option is disabled (default) then X.509 certificate "valid from" and "valid to" timestamp fields are ignored.

If this option is enabled, these fields are compared with the current system date and time. The time is retrieved using the standard time() and gmtime() functions. If the certificate is not valid for the current system time then verification will fail with code MBEDTLS_X509_BADCERT_FUTURE or MBEDTLS_X509_BADCERT_EXPIRED.

Enabling this option requires adding functionality in the firmware to set the system clock to a valid timestamp before using TLS. The recommended way to do this is via ESP-IDF's SNTP functionality, but any method can be used.

In the case where only a small number of certificates are trusted by the device, please carefully consider the tradeoffs of enabling this option. There may be undesired consequences, for example if all trusted certificates expire while the device is offline and a TLS connection is required to update. Or if an issue with the SNTP server means that the system time is invalid for an extended period after a reset.

Default value:

- No (disabled)

CONFIG_MBEDTLS_ECDSA_DETERMINISTIC

Enable deterministic ECDSA

Found in: Component config > mbedTLS

Standard ECDSA is "fragile" in the sense that lack of entropy when signing may result in a compromise of the long-term signing key.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SHA512_C

Enable the SHA-384 and SHA-512 cryptographic hash algorithms

Found in: Component config > mbedTLS

Enable MBEDTLS_SHA512_C adds support for SHA-384 and SHA-512.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_TLS_MODE

TLS Protocol Role

Found in: Component config > mbedTLS

mbedTLS can be compiled with protocol support for the TLS server, TLS client, or both server and client.

Reducing the number of TLS roles supported saves code size.

Available options:

- Server & Client (CONFIG_MBEDTLS_TLS_SERVER_AND_CLIENT)

- Server (CONFIG_MBEDTLS_TLS_SERVER_ONLY)
- Client (CONFIG_MBEDTLS_TLS_CLIENT_ONLY)
- None (CONFIG_MBEDTLS_TLS_DISABLED)

TLS Key Exchange Methods Contains:

- *CONFIG_MBEDTLS_KEY_EXCHANGE_DHE_RSA*
- *CONFIG_MBEDTLS_KEY_EXCHANGE_ECJPAKE*
- *CONFIG_MBEDTLS_PSK_MODES*
- *CONFIG_MBEDTLS_KEY_EXCHANGE_RSA*
- *CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE*

CONFIG_MBEDTLS_PSK_MODES

Enable pre-shared-key ciphersuites

Found in: Component config > mbedTLS > TLS Key Exchange Methods

Enable to show configuration for different types of pre-shared-key TLS authentatication methods.

Leaving this options disabled will save code size if they are not used.

Default value:

- No (disabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_PSK

Enable PSK based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_PSK_MODES

Enable to support symmetric key PSK (pre-shared-key) TLS key exchange modes.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_PSK_MODES*

CONFIG_MBEDTLS_KEY_EXCHANGE_DHE_PSK

Enable DHE-PSK based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_PSK_MODES

Enable to support Diffie-Hellman PSK (pre-shared-key) TLS authentication modes.

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_PSK_MODES* && *CONFIG_MBEDTLS_DHM_C*

CONFIG_MBEDTLS_KEY_EXCHANGE_ECDHE_PSK

Enable ECDHE-PSK based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_PSK_MODES

Enable to support Elliptic-Curve-Diffie-Hellman PSK (pre-shared-key) TLS authentication modes.

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_PSK_MODES* && *CONFIG_MBEDTLS_ECDH_C*

CONFIG_MBEDTLS_KEY_EXCHANGE_RSA_PSK

Enable RSA-PSK based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_PSK_MODES

Enable to support RSA PSK (pre-shared-key) TLS authentication modes.

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_PSK_MODES*

CONFIG_MBEDTLS_KEY_EXCHANGE_RSA

Enable RSA-only based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods

Enable to support ciphersuites with prefix TLS-RSA-WITH-

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_DHE_RSA

Enable DHE-RSA based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods

Enable to support ciphersuites with prefix TLS-DHE-RSA-WITH-

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_DHM_C*

CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE

Support Elliptic Curve based ciphersuites

Found in: Component config > mbedTLS > TLS Key Exchange Methods

Enable to show Elliptic Curve based ciphersuite mode options.

Disabling all Elliptic Curve ciphersuites saves code size and can give slightly faster TLS handshakes, provided the server supports RSA-only ciphersuite modes.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_ECDHE_RSA

Enable ECDHE-RSA based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE

Enable to support ciphersuites with prefix TLS-ECDHE-RSA-WITH-

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_ECDHE_ECDSA

Enable ECDHE-ECDSA based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE

Enable to support ciphersuites with prefix TLS-ECDHE-RSA-WITH-

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_ECDH_ECDSA

Enable ECDH-ECDSA based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE

Enable to support ciphersuites with prefix TLS-ECDHE-RSA-WITH-

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_ECDH_RSA

Enable ECDH-RSA based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods > CONFIG_MBEDTLS_KEY_EXCHANGE_ELLIPTIC_CURVE

Enable to support ciphersuites with prefix TLS-ECDHE-RSA-WITH-

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_KEY_EXCHANGE_ECJPAKE

Enable ECJPAKE based ciphersuite modes

Found in: Component config > mbedTLS > TLS Key Exchange Methods

Enable to support ciphersuites with prefix TLS-ECJPAKE-WITH-

Default value:

- No (disabled) if *CONFIG_MBEDTLS_ECJPAKE_C* && *CONFIG_MBEDTLS_ECP_DP_SECP256R1_ENABLED*

CONFIG_MBEDTLS_SSL_RENEGOTIATION

Support TLS renegotiation

Found in: Component config > mbedTLS

The two main uses of renegotiation are (1) refresh keys on long-lived connections and (2) client authentication after the initial handshake. If you don't need renegotiation, disabling it will save code size and reduce the possibility of abuse/vulnerability.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SSL_PROTO_TLS1_2

Support TLS 1.2 protocol

Found in: Component config > mbedTLS

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SSL_PROTO_GMTSSL1_1

Support GM/T SSL 1.1 protocol

Found in: Component config > mbedTLS

Provisions for GM/T SSL 1.1 support

Default value:

- No (disabled)

CONFIG_MBEDTLS_SSL_PROTO_DTLS

Support DTLS protocol (all versions)

Found in: Component config > mbedTLS

Requires TLS 1.2 to be enabled for DTLS 1.2

Default value:

- No (disabled)

CONFIG_MBEDTLS_SSL_ALPN

Support ALPN (Application Layer Protocol Negotiation)

Found in: Component config > mbedTLS

Disabling this option will save some code size if it is not needed.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_CLIENT_SSL_SESSION_TICKETS

TLS: Client Support for RFC 5077 SSL session tickets

Found in: Component config > mbedTLS

Client support for RFC 5077 session tickets. See mbedTLS documentation for more details. Disabling this option will save some code size.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_SERVER_SSL_SESSION_TICKETS

TLS: Server Support for RFC 5077 SSL session tickets

Found in: Component config > mbedTLS

Server support for RFC 5077 session tickets. See mbedTLS documentation for more details. Disabling this option will save some code size.

Default value:

- Yes (enabled)

Symmetric Ciphers Contains:

- *CONFIG_MBEDTLS_AES_C*
- *CONFIG_MBEDTLS_BLOWFISH_C*
- *CONFIG_MBEDTLS_CAMELLIA_C*
- *CONFIG_MBEDTLS_CCM_C*
- *CONFIG_MBEDTLS_DES_C*
- *CONFIG_MBEDTLS_GCM_C*
- *CONFIG_MBEDTLS_NIST_KW_C*
- *CONFIG_MBEDTLS_XTEA_C*

CONFIG_MBEDTLS_AES_C

AES block cipher

Found in: Component config > mbedTLS > Symmetric Ciphers

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_CAMELLIA_C

Camellia block cipher

Found in: Component config > mbedTLS > Symmetric Ciphers

Default value:

- No (disabled)

CONFIG_MBEDTLS_DES_C

DES block cipher (legacy, insecure)

Found in: Component config > mbedTLS > Symmetric Ciphers

Enables the DES block cipher to support 3DES-based TLS ciphersuites.

3DES is vulnerable to the Sweet32 attack and should only be enabled if absolutely necessary.

Default value:

- No (disabled)

CONFIG_MBEDTLS_BLOWFISH_C

Blowfish block cipher (read help)

Found in: Component config > mbedTLS > Symmetric Ciphers

Enables the Blowfish block cipher (not used for TLS sessions.)

The Blowfish cipher is not used for mbedTLS TLS sessions but can be used for other purposes. Read up on the limitations of Blowfish (including Sweet32) before enabling.

Default value:

- No (disabled)

CONFIG_MBEDTLS_XTEA_C

XTEA block cipher

Found in: Component config > mbedTLS > Symmetric Ciphers

Enables the XTEA block cipher.

Default value:

- No (disabled)

CONFIG_MBEDTLS_CCM_C

CCM (Counter with CBC-MAC) block cipher modes

Found in: Component config > mbedTLS > Symmetric Ciphers

Enable Counter with CBC-MAC (CCM) modes for AES and/or Camellia ciphers.

Disabling this option saves some code size.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_GCM_C

GCM (Galois/Counter) block cipher modes

Found in: Component config > mbedTLS > Symmetric Ciphers

Enable Galois/Counter Mode for AES and/or Camellia ciphers.

This option is generally faster than CCM.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_NIST_KW_C

NIST key wrapping (KW) and KW padding (KWP)

Found in: Component config > mbedTLS > Symmetric Ciphers

Enable NIST key wrapping and key wrapping padding.

Default value:

- No (disabled)

CONFIG_MBEDTLS_RIPEMD160_C

Enable RIPEMD-160 hash algorithm

Found in: Component config > mbedTLS

Enable the RIPEMD-160 hash algorithm.

Default value:

- No (disabled)

Certificates Contains:

- *CONFIG_MBEDTLS_PEM_PARSE_C*
- *CONFIG_MBEDTLS_PEM_WRITE_C*
- *CONFIG_MBEDTLS_X509_CRL_PARSE_C*
- *CONFIG_MBEDTLS_X509_CSR_PARSE_C*

CONFIG_MBEDTLS_PEM_PARSE_C

Read & Parse PEM formatted certificates

Found in: Component config > mbedTLS > Certificates

Enable decoding/parsing of PEM formatted certificates.

If your certificates are all in the simpler DER format, disabling this option will save some code size.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_PEM_WRITE_C

Write PEM formatted certificates

Found in: Component config > mbedTLS > Certificates

Enable writing of PEM formatted certificates.

If writing certificate data only in DER format, disabling this option will save some code size.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_X509_CRL_PARSE_C

X.509 CRL parsing

Found in: Component config > mbedTLS > Certificates

Support for parsing X.509 Certificate Revocation Lists.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_X509_CSR_PARSE_C

X.509 CSR parsing

Found in: Component config > mbedTLS > Certificates

Support for parsing X.509 Certificate Signing Requests

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_ECP_C

Elliptic Curve Ciphers

Found in: Component config > mbedTLS

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_DHM_C

Diffie-Hellman-Merkle key exchange (DHM)

Found in: Component config > mbedTLS

Enable DHM. Needed to use DHE-xxx TLS ciphersuites.

Note that the security of Diffie-Hellman key exchanges depends on a suitable prime being used for the exchange. Please see detailed warning text about this in file *mbedtls/dhm.h* file.

Default value:

- No (disabled)

CONFIG_MBEDTLS_ECDH_C

Elliptic Curve Diffie-Hellman (ECDH)

Found in: Component config > mbedTLS

Enable ECDH. Needed to use ECDHE-xxx TLS ciphersuites.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_ECDSA_C

Elliptic Curve DSA

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_ECDH_C

Enable ECDSA. Needed to use ECDSA-xxx TLS ciphersuites.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_ECJPAKE_C

Elliptic curve J-PAKE

Found in: Component config > mbedTLS

Enable ECJPAKE. Needed to use ECJPAKE-xxx TLS ciphersuites.

Default value:

- No (disabled)

CONFIG_MBEDTLS_ECP_DP_SECP192R1_ENABLED

Enable SECP192R1 curve

Found in: Component config > mbedTLS

Enable support for SECP192R1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP224R1_ENABLED

Enable SECP224R1 curve

Found in: Component config > mbedTLS

Enable support for SECP224R1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP256R1_ENABLED

Enable SECP256R1 curve

Found in: Component config > mbedTLS

Enable support for SECP256R1 Elliptic Curve.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_ECP_DP_SECP384R1_ENABLED

Enable SECP384R1 curve

Found in: Component config > mbedTLS

Enable support for SECP384R1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP521R1_ENABLED

Enable SECP521R1 curve

Found in: Component config > mbedTLS

Enable support for SECP521R1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP192K1_ENABLED

Enable SECP192K1 curve

Found in: Component config > mbedTLS

Enable support for SECP192K1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP224K1_ENABLED

Enable SECP224K1 curve

Found in: Component config > mbedTLS

Enable support for SECP224K1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_SECP256K1_ENABLED

Enable SECP256K1 curve

Found in: Component config > mbedTLS

Enable support for SECP256K1 Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_BP256R1_ENABLED

Enable BP256R1 curve

Found in: Component config > mbedTLS

support for DP Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_BP384R1_ENABLED

Enable BP384R1 curve

Found in: Component config > mbedTLS

support for DP Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_BP512R1_ENABLED

Enable BP512R1 curve

Found in: Component config > mbedTLS

support for DP Elliptic Curve.

CONFIG_MBEDTLS_ECP_DP_CURVE25519_ENABLED

Enable CURVE25519 curve

Found in: Component config > mbedTLS

Enable support for CURVE25519 Elliptic Curve.

CONFIG_MBEDTLS_ECP_NIST_OPTIM

NIST 'modulo p' optimisations

Found in: Component config > mbedTLS

NIST 'modulo p' optimisations increase Elliptic Curve operation performance.

Disabling this option saves some code size.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_ECP_FIXED_POINT_OPTIM

Enable fixed-point multiplication optimisations

Found in: Component config > mbedTLS

This configuration option enables optimizations to speedup (about 3 ~ 4 times) the ECP fixed point multiplication using pre-computed tables in the flash memory. Disabling this configuration option saves flash footprint (about 29KB if all Elliptic Curve selected) in the application binary.

end of Elliptic Curve options

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_POLY1305_C

Poly1305 MAC algorithm

Found in: Component config > mbedTLS

Enable support for Poly1305 MAC algorithm.

Default value:

- No (disabled)

CONFIG_MBEDTLS_CHACHA20_C

Chacha20 stream cipher

Found in: Component config > mbedTLS

Enable support for Chacha20 stream cipher.

Default value:

- No (disabled)

CONFIG_MBEDTLS_CHACHAPOLY_C

ChaCha20-Poly1305 AEAD algorithm

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_CHACHA20_C

Enable support for ChaCha20-Poly1305 AEAD algorithm.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_CHACHA20_C* && *CONFIG_MBEDTLS_POLY1305_C*

CONFIG_MBEDTLS_HKDF_C

HKDF algorithm (RFC 5869)

Found in: Component config > mbedTLS

Enable support for the Hashed Message Authentication Code (HMAC)-based key derivation function (HKDF).

Default value:

- No (disabled)

CONFIG_MBEDTLS_THREADING_C

Enable the threading abstraction layer

Found in: Component config > mbedTLS

If you do intend to use contexts between threads, you will need to enable this layer to prevent race conditions.

Default value:

- No (disabled)

CONFIG_MBEDTLS_THREADING_ALT

Enable threading alternate implementation

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_THREADING_C

Enable threading alt to allow your own alternate threading implementation.

Default value:

- Yes (enabled) if *CONFIG_MBEDTLS_THREADING_C*

CONFIG_MBEDTLS_THREADING_PTHREAD

Enable threading pthread implementation

Found in: Component config > mbedTLS > CONFIG_MBEDTLS_THREADING_C

Enable the pthread wrapper layer for the threading layer.

Default value:

- No (disabled) if *CONFIG_MBEDTLS_THREADING_C*

CONFIG_MBEDTLS_ERROR_STRINGS

Enable error code to error string conversion

Found in: Component config > mbedTLS

Enables mbedtls_strerror() for converting error codes to error strings. Disabling this config can save some code/rodata size as the error string conversion implementation is replaced with an empty stub.

Default value:

- Yes (enabled)

CONFIG_MBEDTLS_USE_CRYPTO_ROM_IMPL

Use ROM implementation of the crypto algorithm

Found in: Component config > mbedTLS

Enable this flag to use mbedtls crypto algorithm from ROM instead of ESP-IDF.

This configuration option saves flash footprint in the application binary. Note that the version of mbedtls crypto algorithm library in ROM is v2.16.12. We have done the security analysis of the mbedtls revision in ROM (v2.16.12) and ensured that affected symbols have been patched (removed). If in the future mbedtls revisions there are security issues that also affects the version in ROM (v2.16.12) then we shall patch the relevant symbols. This would increase the flash footprint and hence care must be taken to keep some reserved space for the application binary in flash layout.

Default value:

- No (disabled) if ESP_ROM_HAS_MBEDTLS_CRYPTO_LIB && *CONFIG_IDF_EXPERIMENTAL_FEATURES*

ESP-MQTT Configurations Contains:

- *CONFIG_MQTT_CUSTOM_OUTBOX*
- *CONFIG_MQTT_TRANSPORT_SSL*
- *CONFIG_MQTT_TRANSPORT_WEBSOCKET*
- *CONFIG_MQTT_PROTOCOL_311*
- *CONFIG_MQTT_PROTOCOL_5*
- *CONFIG_MQTT_TASK_CORE_SELECTION_ENABLED*
- *CONFIG_MQTT_USE_CUSTOM_CONFIG*
- *CONFIG_MQTT_OUTBOX_EXPIRED_TIMEOUT_MS*
- *CONFIG_MQTT_REPORT_DELETED_MESSAGES*
- *CONFIG_MQTT_SKIP_PUBLISH_IF_DISCONNECTED*
- *CONFIG_MQTT_OUTBOX_DATA_ON_EXTERNAL_MEMORY*
- *CONFIG_MQTT_MSG_ID_INCREMENTAL*

CONFIG_MQTT_PROTOCOL_311

Enable MQTT protocol 3.1.1

Found in: Component config > ESP-MQTT Configurations

If not, this library will use MQTT protocol 3.1

Default value:

- Yes (enabled)

CONFIG_MQTT_PROTOCOL_5

Enable MQTT protocol 5.0

Found in: Component config > ESP-MQTT Configurations

If not, this library will not support MQTT 5.0

Default value:

- No (disabled)

CONFIG_MQTT_TRANSPORT_SSL

Enable MQTT over SSL

Found in: Component config > ESP-MQTT Configurations

Enable MQTT transport over SSL with mbedtls

Default value:

- Yes (enabled)

CONFIG_MQTT_TRANSPORT_WEBSOCKET

Enable MQTT over Websocket

Found in: Component config > ESP-MQTT Configurations

Enable MQTT transport over Websocket.

Default value:

- Yes (enabled)

CONFIG_MQTT_TRANSPORT_WEBSOCKET_SECURE

Enable MQTT over Websocket Secure

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_TRANSPORT_WEBSOCKET

Enable MQTT transport over Websocket Secure.

Default value:

- Yes (enabled)

CONFIG_MQTT_MSG_ID_INCREMENTAL

Use Incremental Message Id

Found in: Component config > ESP-MQTT Configurations

Set this to true for the message id (2.3.1 Packet Identifier) to be generated as an incremental number rather then a random value (used by default)

Default value:

- No (disabled)

CONFIG_MQTT_SKIP_PUBLISH_IF_DISCONNECTED

Skip publish if disconnected

Found in: Component config > ESP-MQTT Configurations

Set this to true to avoid publishing (enqueueing messages) if the client is disconnected. The MQTT client tries to publish all messages by default, even in the disconnected state (where the qos1 and qos2 packets are stored in the internal outbox to be published later) The MQTT_SKIP_PUBLISH_IF_DISCONNECTED option allows applications to override this behaviour and not enqueue publish packets in the disconnected state.

Default value:

- No (disabled)

CONFIG_MQTT_REPORT_DELETED_MESSAGES

Report deleted messages

Found in: Component config > ESP-MQTT Configurations

Set this to true to post events for all messages which were deleted from the outbox before being correctly sent and confirmed.

Default value:

- No (disabled)

CONFIG_MQTT_USE_CUSTOM_CONFIG

MQTT Using custom configurations

Found in: Component config > ESP-MQTT Configurations

Custom MQTT configurations.

Default value:

- No (disabled)

CONFIG_MQTT_TCP_DEFAULT_PORT

Default MQTT over TCP port

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Default MQTT over TCP port

Default value:

- 1883 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_SSL_DEFAULT_PORT

Default MQTT over SSL port

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Default MQTT over SSL port

Default value:

- 8883 if *CONFIG_MQTT_USE_CUSTOM_CONFIG* && *CONFIG_MQTT_TRANSPORT_SSL*

CONFIG_MQTT_WS_DEFAULT_PORT

Default MQTT over Websocket port

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Default MQTT over Websocket port

Default value:

- 80 if *CONFIG_MQTT_USE_CUSTOM_CONFIG* && *CONFIG_MQTT_TRANSPORT_WEBSOCKET*

CONFIG_MQTT_WSS_DEFAULT_PORT

Default MQTT over Websocket Secure port

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Default MQTT over Websocket Secure port

Default value:

- 443 if *CONFIG_MQTT_USE_CUSTOM_CONFIG* && *CONFIG_MQTT_TRANSPORT_WEBSOCKET* && *CONFIG_MQTT_TRANSPORT_WEBSOCKET_SECURE*

CONFIG_MQTT_BUFFER_SIZE

Default MQTT Buffer Size

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

This buffer size using for both transmit and receive

Default value:

- 1024 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_TASK_STACK_SIZE

MQTT task stack size

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

MQTT task stack size

Default value:

- 6144 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_DISABLE_API_LOCKS

Disable API locks

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Default config employs API locks to protect internal structures. It is possible to disable these locks if the user code doesn't access MQTT API from multiple concurrent tasks

Default value:

- No (disabled) if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_TASK_PRIORITY

MQTT task priority

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

MQTT task priority. Higher number denotes higher priority.

Default value:

- 5 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_POLL_READ_TIMEOUT_MS

MQTT transport poll read timeut

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

Timeout when polling underlying transport for read.

Default value:

- 1000 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_EVENT_QUEUE_SIZE

Number of queued events.

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_USE_CUSTOM_CONFIG

A value higher than 1 enables multiple queued events.

Default value:

- 1 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_TASK_CORE_SELECTION_ENABLED

Enable MQTT task core selection

Found in: Component config > ESP-MQTT Configurations

This will enable core selection

CONFIG_MQTT_TASK_CORE_SELECTION

Core to use ?

Found in: Component config > ESP-MQTT Configurations > CONFIG_MQTT_TASK_CORE_SELECTION_ENABLED

Available options:

- Core 0 (CONFIG_MQTT_USE_CORE_0)
- Core 1 (CONFIG_MQTT_USE_CORE_1)

CONFIG_MQTT_OUTBOX_DATA_ON_EXTERNAL_MEMORY

Use external memory for outbox data

Found in: Component config > ESP-MQTT Configurations

Set to true to use external memory for outbox data.

Default value:

- No (disabled) if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

CONFIG_MQTT_CUSTOM_OUTBOX

Enable custom outbox implementation

Found in: Component config > ESP-MQTT Configurations

Set to true if a specific implementation of message outbox is needed (e.g. persistent outbox in NVM or similar). Note: Implementation of the custom outbox must be added to the mqtt component. These CMake commands could be used to append the custom implementation to lib-mqtt sources: idf_component_get_property(mqtt mqtt COMPONENT_LIB) set_property(TARGET ${mqtt} PROPERTY SOURCES ${PROJECT_DIR}/custom_outbox.c APPEND)

Default value:

- No (disabled)

CONFIG_MQTT_OUTBOX_EXPIRED_TIMEOUT_MS

Outbox message expired timeout[ms]

Found in: Component config > ESP-MQTT Configurations

Messages which stays in the outbox longer than this value before being published will be discarded.

Default value:

- 30000 if *CONFIG_MQTT_USE_CUSTOM_CONFIG*

Newlib

Contains:

- *CONFIG_NEWLIB_NANO_FORMAT*
- *CONFIG_NEWLIB_STDIN_LINE_ENDING*
- *CONFIG_NEWLIB_STDOUT_LINE_ENDING*
- *CONFIG_NEWLIB_TIME_SYSCALL*

CONFIG_NEWLIB_STDOUT_LINE_ENDING

Line ending for UART output

Found in: Component config > Newlib

This option allows configuring the desired line endings sent to UART when a newline ('n', LF) appears on stdout. Three options are possible:

CRLF: whenever LF is encountered, prepend it with CR

LF: no modification is applied, stdout is sent as is

CR: each occurrence of LF is replaced with CR

This option doesn't affect behavior of the UART driver (drivers/uart.h).

Available options:

- CRLF (CONFIG_NEWLIB_STDOUT_LINE_ENDING_CRLF)
- LF (CONFIG_NEWLIB_STDOUT_LINE_ENDING_LF)
- CR (CONFIG_NEWLIB_STDOUT_LINE_ENDING_CR)

CONFIG_NEWLIB_STDIN_LINE_ENDING

Line ending for UART input

Found in: Component config > Newlib

This option allows configuring which input sequence on UART produces a newline ('n', LF) on stdin. Three options are possible:

CRLF: CRLF is converted to LF

LF: no modification is applied, input is sent to stdin as is

CR: each occurrence of CR is replaced with LF

This option doesn't affect behavior of the UART driver (drivers/uart.h).

Available options:

- CRLF (CONFIG_NEWLIB_STDIN_LINE_ENDING_CRLF)
- LF (CONFIG_NEWLIB_STDIN_LINE_ENDING_LF)
- CR (CONFIG_NEWLIB_STDIN_LINE_ENDING_CR)

CONFIG_NEWLIB_NANO_FORMAT

Enable 'nano' formatting options for printf/scanf family

Found in: Component config > Newlib

In most chips the ROM contains parts of newlib C library, including printf/scanf family of functions. These functions have been compiled with so-called "nano" formatting option. This option doesn't support 64-bit integer formats and C99 features, such as positional arguments.

For more details about "nano" formatting option, please see newlib readme file, search for '--enable-newlib-nano-formatted-io': https://sourceware.org/git/?p=newlib-cygwin.git;a=blob_plain;f=newlib/README;hb=HEAD

If this option is enabled and the ROM contains functions from newlib-nano, the build system will use functions available in ROM, reducing the application binary size. Functions available in ROM run faster than functions which run from flash. Functions available in ROM can also run when flash instruction cache is disabled.

Some chips (e.g. ESP32-C6) has the full formatting versions of printf/scanf in ROM instead of the nano versions and in this building with newlib nano might actually increase the size of the binary. Which functions are present in ROM can be seen from ROM caps: ESP_ROM_HAS_NEWLIB_NANO_FORMAT and ESP_ROM_HAS_NEWLIB_NORMAL_FORMAT.

If you need 64-bit integer formatting support or C99 features, keep this option disabled.

CONFIG_NEWLIB_TIME_SYSCALL

Timers used for gettimeofday function

Found in: Component config > Newlib

This setting defines which hardware timers are used to implement 'gettimeofday' and 'time' functions in C library.

- **If both high-resolution (systimer for all targets except ESP32)** and RTC timers are used, timekeeping will continue in deep sleep. Time will be reported at 1 microsecond resolution. This is the default, and the recommended option.
- **If only high-resolution timer (systimer) is used, gettimeofday will** provide time at microsecond resolution. Time will not be preserved when going into deep sleep mode.
- **If only RTC timer is used, timekeeping will continue in** deep sleep, but time will be measured at 6.(6) microsecond resolution. Also the gettimeofday function itself may take longer to run.
- **If no timers are used, gettimeofday and time functions** return -1 and set errno to ENOSYS; they are defined as weak, so they could be overridden. If you want to customize gettimeofday() and other time functions, please choose this option and refer to the 'time.c' source file for the exact prototypes of these functions.
- **When RTC is used for timekeeping, two RTC_STORE registers are** used to keep time in deep sleep mode.

Available options:

- RTC and high-resolution timer (CONFIG_NEWLIB_TIME_SYSCALL_USE_RTC_HRT)
- RTC (CONFIG_NEWLIB_TIME_SYSCALL_USE_RTC)
- High-resolution timer (CONFIG_NEWLIB_TIME_SYSCALL_USE_HRT)
- None (CONFIG_NEWLIB_TIME_SYSCALL_USE_NONE)

NVS Contains:

- *CONFIG_NVS_LEGACY_DUP_KEYS_COMPATIBILITY*
- *CONFIG_NVS_ENCRYPTION*
- *CONFIG_NVS_COMPATIBLE_PRE_V4_3_ENCRYPTION_FLAG*
- *CONFIG_NVS_ASSERT_ERROR_CHECK*

CONFIG_NVS_ENCRYPTION

Enable NVS encryption

Found in: Component config > NVS

This option enables encryption for NVS. When enabled, XTS-AES is used to encrypt the complete NVS data, except the page headers. It requires XTS encryption keys to be stored in an encrypted partition (enabling flash encryption is mandatory here) or to be derived from an HMAC key burnt in eFuse.

Default value:

- Yes (enabled) if *CONFIG_SECURE_FLASH_ENC_ENABLED*

CONFIG_NVS_COMPATIBLE_PRE_V4_3_ENCRYPTION_FLAG

NVS partition encrypted flag compatible with ESP-IDF before v4.3

Found in: Component config > NVS

Enabling this will ignore "encrypted" flag for NVS partitions. NVS encryption scheme is different than hardware flash encryption and hence it is not recommended to have "encrypted" flag for NVS partitions.

This was not being checked in pre v4.3 IDF. Hence, if you have any devices where this flag is kept enabled in partition table then enabling this config will allow to have same behavior as pre v4.3 IDF.

CONFIG_NVS_ASSERT_ERROR_CHECK

Use assertions for error checking

Found in: Component config > NVS

This option switches error checking type between assertions (y) or return codes (n).

Default value:

- No (disabled)

CONFIG_NVS_LEGACY_DUP_KEYS_COMPATIBILITY

Enable legacy nvs_set function behavior when same key is reused with different data types

Found in: Component config > NVS

Enabling this option will switch the nvs_set() family of functions to the legacy mode: when called repeatedly with the same key but different data type, the existing value in the NVS remains active and the new value is just stored, actually not accessible through corresponding nvs_get() call for the key given. Use this option only when your application relies on such NVS API behaviour.

Default value:

- No (disabled)

NVS Security Provider

Contains:

- *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID*
- *CONFIG_NVS_SEC_KEY_PROTECTION_SCHEME*

CONFIG_NVS_SEC_KEY_PROTECTION_SCHEME

NVS Encryption: Key Protection Scheme

Found in: Component config > NVS Security Provider

This choice defines the default NVS encryption keys protection scheme; which will be used for the default NVS partition. Users can use the corresponding scheme registration APIs to register other schemes for the default as well as other NVS partitions.

Available options:

- Using Flash Encryption (CONFIG_NVS_SEC_KEY_PROTECT_USING_FLASH_ENC) Protect the NVS Encryption Keys using Flash Encryption Requires a separate 'nvs_keys' partition (which will be encrypted by flash encryption) for storing the NVS encryption keys
- Using HMAC peripheral (CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC) Derive and protect the NVS Encryption Keys using the HMAC peripheral Requires the specified eFuse block (NVS_SEC_HMAC_EFUSE_KEY_ID or the v2 API argument) to be empty or pre-written with a key with the purpose ESP_EFUSE_KEY_PURPOSE_HMAC_UP

CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID

eFuse key ID storing the HMAC key

Found in: Component config > NVS Security Provider

eFuse block key ID storing the HMAC key for deriving the NVS encryption keys

Note: The eFuse block key ID required by the HMAC scheme (CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC) is set using this config when the default NVS partition is initialized with nvs_flash_init(). The eFuse block key ID can also be set at runtime by passing the appropriate value to the NVS security scheme registration APIs.

Range:

- from 0 to 6 if *CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC*

Default value:

- 6 if *CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC*

OpenThread Contains:

- *CONFIG_OPENTHREAD_PLATFORM_MSGPOOL_MANAGEMENT*
- *CONFIG_OPENTHREAD_DEVICE_TYPE*
- *CONFIG_OPENTHREAD_RADIO_TYPE*
- *CONFIG_OPENTHREAD_BORDER_ROUTER*
- *CONFIG_OPENTHREAD_COMMISSIONER*
- *CONFIG_OPENTHREAD_CSL_DEBUG_ENABLE*
- *CONFIG_OPENTHREAD_CSL_ENABLE*
- *CONFIG_OPENTHREAD_DIAG*
- *CONFIG_OPENTHREAD_DNS_CLIENT*
- *CONFIG_OPENTHREAD_DUA_ENABLE*
- *CONFIG_OPENTHREAD_JOINER*
- *CONFIG_OPENTHREAD_LINK_METRICS*
- *CONFIG_OPENTHREAD_MACFILTER_ENABLE*
- *CONFIG_OPENTHREAD_CLI*
- *CONFIG_OPENTHREAD_SPINEL_ONLY*
- *CONFIG_OPENTHREAD_RX_ON_WHEN_IDLE*
- *CONFIG_OPENTHREAD_RADIO_STATS_ENABLE*
- *CONFIG_OPENTHREAD_SRP_CLIENT*
- *CONFIG_OPENTHREAD_TIME_SYNC*
- *CONFIG_OPENTHREAD_NCP_VENDOR_HOOK*
- *CONFIG_OPENTHREAD_MAC_MAX_CSMA_BACKOFFS_DIRECT*
- *CONFIG_OPENTHREAD_ENABLED*
- *OpenThread version message*
- *CONFIG_OPENTHREAD_XTAL_ACCURACY*
- *CONFIG_OPENTHREAD_CSL_UNCERTAIN*
- *CONFIG_OPENTHREAD_CSL_ACCURACY*
- *CONFIG_OPENTHREAD_NUM_MESSAGE_BUFFERS*
- *CONFIG_OPENTHREAD_RCP_TRANSPORT*
- *CONFIG_OPENTHREAD_MLE_MAX_CHILDREN*
- *CONFIG_OPENTHREAD_TMF_ADDR_CACHE_ENTRIES*
- *CONFIG_OPENTHREAD_SPINEL_RX_FRAME_BUFFER_SIZE*
- *CONFIG_OPENTHREAD_UART_BUFFER_SIZE*
- *Thread Address Query Config*
- *Thread Operational Dataset*
- *CONFIG_OPENTHREAD_DNS64_CLIENT*

CONFIG_OPENTHREAD_ENABLED

OpenThread

Found in: Component config > OpenThread

Select this option to enable OpenThread and show the submenu with OpenThread configuration choices.

Default value:

- No (disabled)

CONFIG_OPENTHREAD_LOG_LEVEL_DYNAMIC

Enable dynamic log level control

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_ENABLED

Select this option to enable dynamic log level control for OpenThread

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_CONSOLE_TYPE

OpenThread console type

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_ENABLED

Select OpenThread console type

Available options:

- OpenThread console type UART (CONFIG_OPENTHREAD_CONSOLE_TYPE_UART)
- OpenThread console type USB Serial/JTAG Controller (CONFIG_OPENTHREAD_CONSOLE_TYPE_USB_SERIAL_JTAG)

CONFIG_OPENTHREAD_LOG_LEVEL

OpenThread log verbosity

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_ENABLED

Select OpenThread log level.

Available options:

- No logs (CONFIG_OPENTHREAD_LOG_LEVEL_NONE)
- Error logs (CONFIG_OPENTHREAD_LOG_LEVEL_CRIT)
- Warning logs (CONFIG_OPENTHREAD_LOG_LEVEL_WARN)
- Notice logs (CONFIG_OPENTHREAD_LOG_LEVEL_NOTE)
- Info logs (CONFIG_OPENTHREAD_LOG_LEVEL_INFO)
- Debug logs (CONFIG_OPENTHREAD_LOG_LEVEL_DEBG)

Thread Operational Dataset Contains:

- *CONFIG_OPENTHREAD_NETWORK_EXTPANID*
- *CONFIG_OPENTHREAD_MESH_LOCAL_PREFIX*
- *CONFIG_OPENTHREAD_NETWORK_CHANNEL*
- *CONFIG_OPENTHREAD_NETWORK_MASTERKEY*
- *CONFIG_OPENTHREAD_NETWORK_NAME*
- *CONFIG_OPENTHREAD_NETWORK_PANID*
- *CONFIG_OPENTHREAD_NETWORK_PSKC*

CONFIG_OPENTHREAD_NETWORK_NAME

OpenThread network name

Found in: Component config > OpenThread > Thread Operational Dataset

Default value:

- "OpenThread-ESP"

CONFIG_OPENTHREAD_MESH_LOCAL_PREFIX

OpenThread mesh local prefix, format <address>/<plen>

Found in: Component config > OpenThread > Thread Operational Dataset

A string in the format "<address>/<plen>", where *<address>* is an IPv6 address and *<plen>* is a prefix length. For example "fd00:db8:a0:0::/64"

Default value:

- "fd00:db8:a0:0::/64"

CONFIG_OPENTHREAD_NETWORK_CHANNEL

OpenThread network channel

Found in: Component config > OpenThread > Thread Operational Dataset

Range:

- from 11 to 26

Default value:

- 15

CONFIG_OPENTHREAD_NETWORK_PANID

OpenThread network pan id

Found in: Component config > OpenThread > Thread Operational Dataset

Range:

- from 0 to 0xFFFE

Default value:

- "0x1234"

CONFIG_OPENTHREAD_NETWORK_EXTPANID

OpenThread extended pan id

Found in: Component config > OpenThread > Thread Operational Dataset

The OpenThread network extended pan id in hex string format

Default value:

- dead00beef00cafe

CONFIG_OPENTHREAD_NETWORK_MASTERKEY

OpenThread network key

Found in: Component config > OpenThread > Thread Operational Dataset

The OpenThread network network key in hex string format

Default value:

- 00112233445566778899aabbccddeeff

CONFIG_OPENTHREAD_NETWORK_PSKC

OpenThread pre-shared commissioner key

Found in: Component config > OpenThread > Thread Operational Dataset

The OpenThread pre-shared commissioner key in hex string format

Default value:

- 104810e2315100afd6bc9215a6bfac53

CONFIG_OPENTHREAD_RADIO_TYPE

Config the Thread radio type

Found in: Component config > OpenThread

Configure how OpenThread connects to the 15.4 radio

Available options:

- Native 15.4 radio (CONFIG_OPENTHREAD_RADIO_NATIVE)
  Select this to use the native 15.4 radio.
- Connect via UART (CONFIG_OPENTHREAD_RADIO_SPINEL_UART)
  Select this to connect to a Radio Co-Processor via UART.
- Connect via SPI (CONFIG_OPENTHREAD_RADIO_SPINEL_SPI)
  Select this to connect to a Radio Co-Processor via SPI.

CONFIG_OPENTHREAD_DEVICE_TYPE

Config the Thread device type

Found in: Component config > OpenThread

OpenThread can be configured to different device types (FTD, MTD, Radio)

Available options:

- Full Thread Device (CONFIG_OPENTHREAD_FTD)
  Select this to enable Full Thread Device which can act as router and leader in a Thread network.
- Minimal Thread Device (CONFIG_OPENTHREAD_MTD)
  Select this to enable Minimal Thread Device which can only act as end device in a Thread network. This will reduce the code size of the OpenThread stack.
- Radio Only Device (CONFIG_OPENTHREAD_RADIO)
  Select this to enable Radio Only Device which can only forward 15.4 packets to the host. The OpenThread stack will be run on the host and OpenThread will have minimal footprint on the radio only device.

CONFIG_OPENTHREAD_RCP_TRANSPORT

The RCP transport type

Found in: Component config > OpenThread

Available options:

- UART RCP (CONFIG_OPENTHREAD_RCP_UART)
  Select this to enable UART connection to host.
- SPI RCP (CONFIG_OPENTHREAD_RCP_SPI)
  Select this to enable SPI connection to host.

OpenThread version message Contains:

- *CONFIG_OPENTHREAD_PACKAGE_NAME*
- *CONFIG_OPENTHREAD_PLATFORM_INFO*

CONFIG_OPENTHREAD_PACKAGE_NAME

OpenThread package name

Found in: Component config > OpenThread > OpenThread version message

The OpenThread package name.

Default value:

- "openthread-esp32" if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_PLATFORM_INFO

platform information

Found in: Component config > OpenThread > OpenThread version message

The OpenThread platform information.

Default value:

- "esp32s3" if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_NCP_VENDOR_HOOK

Enable vendor command for RCP

Found in: Component config > OpenThread

Select this to enable OpenThread NCP vendor commands.

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_RADIO*

CONFIG_OPENTHREAD_CLI

Enable Openthread Command-Line Interface

Found in: Component config > OpenThread

Select this option to enable Command-Line Interface in OpenThread.

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_DIAG

Enable diag

Found in: Component config > OpenThread

Select this option to enable Diag in OpenThread. This will enable diag mode and a series of diag commands in the OpenThread command line. These commands allow users to manipulate low-level features of the storage and 15.4 radio.

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_COMMISSIONER

Enable Commissioner

Found in: Component config > OpenThread

Select this option to enable commissioner in OpenThread. This will enable the device to act as a commissioner in the Thread network. A commissioner checks the pre-shared key from a joining device

with the Thread commissioning protocol and shares the network parameter with the joining device upon success.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_COMM_MAX_JOINER_ENTRIES

The size of max commissioning joiner entries

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_COMMISSIONER

Default value:

- 2 if *CONFIG_OPENTHREAD_COMMISSIONER*

CONFIG_OPENTHREAD_JOINER

Enable Joiner

Found in: Component config > OpenThread

Select this option to enable Joiner in OpenThread. This allows a device to join the Thread network with a pre-shared key using the Thread commissioning protocol.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_SRP_CLIENT

Enable SRP Client

Found in: Component config > OpenThread

Select this option to enable SRP Client in OpenThread. This allows a device to register SRP services to SRP Server.

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_SRP_CLIENT_MAX_SERVICES

Specifies number of service entries in the SRP client service pool

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_SRP_CLIENT

Set the max buffer size of service entries in the SRP client service pool.

Default value:

- 5 if *CONFIG_OPENTHREAD_SRP_CLIENT*

CONFIG_OPENTHREAD_DNS_CLIENT

Enable DNS Client

Found in: Component config > OpenThread

Select this option to enable DNS Client in OpenThread.

Default value:

- Yes (enabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_BORDER_ROUTER

Enable Border Router

Found in: Component config > OpenThread

Select this option to enable border router features in OpenThread.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_PLATFORM_MSGPOOL_MANAGEMENT

Allocate message pool buffer from PSRAM

Found in: Component config > OpenThread

If enabled, the message pool is managed by platform defined logic.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED* && (*CONFIG_SPIRAM_USE_CAPS_ALLOC* || *CONFIG_SPIRAM_USE_MALLOC*)

CONFIG_OPENTHREAD_NUM_MESSAGE_BUFFERS

The number of openthread message buffers

Found in: Component config > OpenThread

Default value:

- 65 if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_SPINEL_RX_FRAME_BUFFER_SIZE

The size of openthread spinel rx frame buffer

Found in: Component config > OpenThread

Default value:

- 1024 if (*CONFIG_OPENTHREAD_MTD* || *CONFIG_OPENTHREAD_RADIO*) && (*CONFIG_OPENTHREAD_ENABLED* || *CONFIG_OPENTHREAD_SPINEL_ONLY*)
- 2048 if (*CONFIG_OPENTHREAD_FTD* || *CONFIG_OPENTHREAD_SPINEL_ONLY*) && (*CONFIG_OPENTHREAD_ENABLED* || *CONFIG_OPENTHREAD_SPINEL_ONLY*)

CONFIG_OPENTHREAD_MAC_MAX_CSMA_BACKOFFS_DIRECT

Maximum backoffs times before declaring a channel access failure.

Found in: Component config > OpenThread

The maximum number of backoffs the CSMA-CA algorithm will attempt before declaring a channel access failure.

Default value:

- 4 if *CONFIG_OPENTHREAD_ENABLED* || *CONFIG_OPENTHREAD_SPINEL_ONLY*

CONFIG_OPENTHREAD_MLE_MAX_CHILDREN

The size of max MLE children entries

Found in: Component config > OpenThread

Default value:

- 10 if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_TMF_ADDR_CACHE_ENTRIES

The size of max TMF address cache entries

Found in: Component config > OpenThread

Default value:

- 20 if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_DNS64_CLIENT

Use dns64 client

Found in: Component config > OpenThread

Select this option to acquire NAT64 address from dns servers.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED* && *CONFIG_LWIP_IPV4*

CONFIG_OPENTHREAD_DNS_SERVER_ADDR

DNS server address (IPv4)

Found in: Component config > OpenThread > CONFIG_OPENTHREAD_DNS64_CLIENT

Set the DNS server IPv4 address.

Default value:

- "8.8.8.8" if *CONFIG_OPENTHREAD_DNS64_CLIENT*

CONFIG_OPENTHREAD_UART_BUFFER_SIZE

The uart received buffer size of openthread

Found in: Component config > OpenThread

Set the OpenThread UART buffer size.

Default value:

- 2048 if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_LINK_METRICS

Enable link metrics feature

Found in: Component config > OpenThread

Select this option to enable link metrics feature

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_MACFILTER_ENABLE

Enable mac filter feature

Found in: Component config > OpenThread

Select this option to enable mac filter feature

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_CSL_ENABLE

Enable CSL feature

Found in: Component config > OpenThread

Select this option to enable CSL feature

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_XTAL_ACCURACY

The accuracy of the XTAL

Found in: Component config > OpenThread

The device's XTAL accuracy, in ppm.

Default value:

- 130

CONFIG_OPENTHREAD_CSL_ACCURACY

The current CSL rx/tx scheduling drift, in units of ± ppm

Found in: Component config > OpenThread

The current accuracy of the clock used for scheduling CSL operations

Default value:

- 1 if *CONFIG_OPENTHREAD_CSL_ENABLE*

CONFIG_OPENTHREAD_CSL_UNCERTAIN

The CSL Uncertainty in units of 10 us.

Found in: Component config > OpenThread

The fixed uncertainty of the Device for scheduling CSL Transmissions in units of 10 microseconds.

Default value:

- 1 if *CONFIG_OPENTHREAD_CSL_ENABLE*

CONFIG_OPENTHREAD_CSL_DEBUG_ENABLE

Enable CSL debug

Found in: Component config > OpenThread

Select this option to set rx on when sleep in CSL feature, only for debug

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_CSL_ENABLE*

CONFIG_OPENTHREAD_DUA_ENABLE

Enable Domain Unicast Address feature

Found in: Component config > OpenThread

Only used for Thread1.2 certification

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_TIME_SYNC

Enable the time synchronization service feature

Found in: Component config > OpenThread

Select this option to enable time synchronization feature, the devices in the same Thread network could sync to the same network time.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_ENABLED*

CONFIG_OPENTHREAD_RADIO_STATS_ENABLE

Enable Radio Statistics feature

Found in: Component config > OpenThread

Select this option to enable the radio statistics feature, you can use radio command to print some radio Statistics information.

Default value:

- No (disabled) if *CONFIG_OPENTHREAD_FTD* || *CONFIG_OPENTHREAD_MTD*

CONFIG_OPENTHREAD_SPINEL_ONLY

Enable OpenThread External Radio Spinel feature

Found in: Component config > OpenThread

Select this option to enable the OpenThread Radio Spinel for external protocol stack, such as Zigbee.

Default value:

- No (disabled)

CONFIG_OPENTHREAD_RX_ON_WHEN_IDLE

Enable OpenThread radio capability rx on when idle

Found in: Component config > OpenThread

Select this option to enable OpenThread radio capability rx on when idle. Do not support this feature when SW coexistence is enabled.

Default value:

- No (disabled) if *CONFIG_ESP_COEX_SW_COEXIST_ENABLE*

Thread Address Query Config Contains:

- *CONFIG_OPENTHREAD_ADDRESS_QUERY_RETRY_DELAY*
- *CONFIG_OPENTHREAD_ADDRESS_QUERY_MAX_RETRY_DELAY*
- *CONFIG_OPENTHREAD_ADDRESS_QUERY_TIMEOUT*

CONFIG_OPENTHREAD_ADDRESS_QUERY_TIMEOUT

Timeout value (in seconds) for a address notification response after sending an address query.

Found in: Component config > OpenThread > Thread Address Query Config

Default value:

- 3 if *CONFIG_OPENTHREAD_FTD* || *CONFIG_OPENTHREAD_MTD*

CONFIG_OPENTHREAD_ADDRESS_QUERY_RETRY_DELAY

Initial retry delay for address query (in seconds).

Found in: Component config > OpenThread > Thread Address Query Config

Default value:

- 15 if *CONFIG_OPENTHREAD_FTD* || *CONFIG_OPENTHREAD_MTD*

CONFIG_OPENTHREAD_ADDRESS_QUERY_MAX_RETRY_DELAY

Maximum retry delay for address query (in seconds).

Found in: Component config > OpenThread > Thread Address Query Config

Default value:

- 120 if *CONFIG_OPENTHREAD_FTD* || *CONFIG_OPENTHREAD_MTD*

Protocomm Contains:

- *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_0*
- *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_1*
- *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_2*

CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_0

Support protocomm security version 0 (no security)

Found in: Component config > Protocomm

Enable support of security version 0. Disabling this option saves some code size. Consult the Enabling protocomm security version section of the Protocomm documentation in ESP-IDF Programming guide for more details.

Default value:

- Yes (enabled)

CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_1

Support protocomm security version 1 (Curve25519 key exchange + AES-CTR encryption/decryption)

Found in: Component config > Protocomm

Enable support of security version 1. Disabling this option saves some code size. Consult the Enabling protocomm security version section of the Protocomm documentation in ESP-IDF Programming guide for more details.

Default value:

- Yes (enabled)

CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_2

Support protocomm security version 2 (SRP6a-based key exchange + AES-GCM encryption/decryption)

Found in: Component config > Protocomm

Enable support of security version 2. Disabling this option saves some code size. Consult the Enabling protocomm security version section of the Protocomm documentation in ESP-IDF Programming guide for more details.

Default value:

- Yes (enabled)

PThreads Contains:

- *CONFIG_PTHREAD_TASK_NAME_DEFAULT*
- *CONFIG_PTHREAD_TASK_CORE_DEFAULT*
- *CONFIG_PTHREAD_TASK_PRIO_DEFAULT*
- *CONFIG_PTHREAD_TASK_STACK_SIZE_DEFAULT*
- *CONFIG_PTHREAD_STACK_MIN*

CONFIG_PTHREAD_TASK_PRIO_DEFAULT

Default task priority

Found in: Component config > PThreads

Priority used to create new tasks with default pthread parameters.

Range:
- from 0 to 255

Default value:
- 5

CONFIG_PTHREAD_TASK_STACK_SIZE_DEFAULT

Default task stack size

Found in: Component config > PThreads

Stack size used to create new tasks with default pthread parameters.

Default value:
- 3072

CONFIG_PTHREAD_STACK_MIN

Minimum allowed pthread stack size

Found in: Component config > PThreads

Minimum allowed pthread stack size set in attributes passed to pthread_create

Default value:
- 768

CONFIG_PTHREAD_TASK_CORE_DEFAULT

Default pthread core affinity

Found in: Component config > PThreads

The default core to which pthreads are pinned.

Available options:

- No affinity (CONFIG_PTHREAD_DEFAULT_CORE_NO_AFFINITY)
- Core 0 (CONFIG_PTHREAD_DEFAULT_CORE_0)
- Core 1 (CONFIG_PTHREAD_DEFAULT_CORE_1)

CONFIG_PTHREAD_TASK_NAME_DEFAULT

Default name of pthreads

Found in: Component config > PThreads

The default name of pthreads.

Default value:

- "pthread"

SoC Settings Contains:

- *MMU Config*

MMU Config

Main Flash configuration Contains:

- *Optional and Experimental Features (READ DOCS FIRST)*
- *SPI Flash behavior when brownout*

SPI Flash behavior when brownout Contains:

- *CONFIG_SPI_FLASH_BROWNOUT_RESET_XMC*

CONFIG_SPI_FLASH_BROWNOUT_RESET_XMC

Enable sending reset when brownout for XMC flash chips

Found in: Component config > Main Flash configuration > SPI Flash behavior when brownout

When this option is selected, the patch will be enabled for XMC. Follow the recommended flow by XMC for better stability.

DO NOT DISABLE UNLESS YOU KNOW WHAT YOU ARE DOING.

Optional and Experimental Features (READ DOCS FIRST) Contains:

- *CONFIG_SPI_FLASH_AUTO_SUSPEND*
- *CONFIG_SPI_FLASH_HPM*
- *CONFIG_SPI_FLASH_SUSPEND_TSUS_VAL_US*
- *CONFIG_SPI_FLASH_HPM_DC*

CONFIG_SPI_FLASH_HPM

High Performance Mode (READ DOCS FIRST, > 80MHz)

Found in: Component config > Main Flash configuration > Optional and Experimental Features (READ DOCS FIRST)

Whether the High Performance Mode of Flash is enabled. As an optional feature, user needs to manually enable this option as a confirmation. To be back-compatible with earlier IDF versionn, this option is automatically enabled with warning when Flash running > 80Mhz.

Available options:

- Enable (CONFIG_SPI_FLASH_HPM_ENA)
- Auto (Not recommended) (CONFIG_SPI_FLASH_HPM_AUTO)
- Disabled (CONFIG_SPI_FLASH_HPM_DIS)

CONFIG_SPI_FLASH_HPM_DC

Support HPM using DC (READ DOCS FIRST)

Found in: Component config > Main Flash configuration > Optional and Experimental Features (READ DOCS FIRST)

This feature needs your bootloader to be compiled DC-aware (BOOTLOADER_FLASH_DC_AWARE=y). Otherwise the chip will not be able to boot after a reset.

Available options:

- Auto (Enable when bootloader support enabled (BOOTLOADER_FLASH_DC_AWARE)) (CONFIG_SPI_FLASH_HPM_DC_AUTO)
- Disable (READ DOCS FIRST) (CONFIG_SPI_FLASH_HPM_DC_DISABLE)

CONFIG_SPI_FLASH_AUTO_SUSPEND

Auto suspend long erase/write operations (READ DOCS FIRST)

Found in: Component config > Main Flash configuration > Optional and Experimental Features (READ DOCS FIRST)

This option is disabled by default because it is supported only for specific flash chips and for specific Espressif chips. To evaluate if you can use this feature refer to *Optional Features for Flash > Auto Suspend & Resume* of the *ESP-IDF Programming Guide*.

CAUTION: If you want to OTA to an app with this feature turned on, please make sure the bootloader has the support for it. (later than IDF v4.3)

If you are using an official Espressif module, please contact Espressif Business support to check if the module has the flash that support this feature installed. Also refer to *Concurrency Constraints for Flash on SPI1 > Flash Auto Suspend Feature* before enabling this option.

CONFIG_SPI_FLASH_SUSPEND_TSUS_VAL_US

SPI flash tSUS value (refer to chapter AC CHARACTERISTICS)

Found in: Component config > Main Flash configuration > Optional and Experimental Features (READ DOCS FIRST)

This config is used for setting Tsus parameter. Tsus means CS# high to next command after suspend. You can refer to the chapter of AC CHARACTERISTICS of flash datasheet.

SPI Flash driver Contains:

- *Auto-detect flash chips*
- *CONFIG_SPI_FLASH_BYPASS_BLOCK_ERASE*
- *CONFIG_SPI_FLASH_ENABLE_ENCRYPTED_READ_WRITE*
- *CONFIG_SPI_FLASH_ENABLE_COUNTERS*
- *CONFIG_SPI_FLASH_ROM_DRIVER_PATCH*
- *CONFIG_SPI_FLASH_YIELD_DURING_ERASE*
- *CONFIG_SPI_FLASH_CHECK_ERASE_TIMEOUT_DISABLED*
- *CONFIG_SPI_FLASH_WRITE_CHUNK_SIZE*
- *CONFIG_SPI_FLASH_OVERRIDE_CHIP_DRIVER_LIST*
- *CONFIG_SPI_FLASH_SIZE_OVERRIDE*
- *CONFIG_SPI_FLASH_ROM_IMPL*
- *CONFIG_SPI_FLASH_VERIFY_WRITE*
- *CONFIG_SPI_FLASH_DANGEROUS_WRITE*

CONFIG_SPI_FLASH_VERIFY_WRITE

Verify SPI flash writes

Found in: Component config > SPI Flash driver

If this option is enabled, any time SPI flash is written then the data will be read back and verified. This can catch hardware problems with SPI flash, or flash which was not erased before verification.

CONFIG_SPI_FLASH_LOG_FAILED_WRITE

Log errors if verification fails

Found in: Component config > SPI Flash driver > CONFIG_SPI_FLASH_VERIFY_WRITE

If this option is enabled, if SPI flash write verification fails then a log error line will be written with the address, expected & actual values. This can be useful when debugging hardware SPI flash problems.

CONFIG_SPI_FLASH_WARN_SETTING_ZERO_TO_ONE

Log warning if writing zero bits to ones

Found in: Component config > SPI Flash driver > CONFIG_SPI_FLASH_VERIFY_WRITE

If this option is enabled, any SPI flash write which tries to set zero bits in the flash to ones will log a warning. Such writes will not result in the requested data appearing identically in flash once written, as SPI NOR flash can only set bits to one when an entire sector is erased. After erasing, individual bits can only be written from one to zero.

Note that some software (such as SPIFFS) which is aware of SPI NOR flash may write one bits as an optimisation, relying on the data in flash becoming a bitwise AND of the new data and any existing data. Such software will log spurious warnings if this option is enabled.

CONFIG_SPI_FLASH_ENABLE_COUNTERS

Enable operation counters

Found in: Component config > SPI Flash driver

This option enables the following APIs:

- esp_flash_reset_counters
- esp_flash_dump_counters
- esp_flash_get_counters

These APIs may be used to collect performance data for spi_flash APIs and to help understand behaviour of libraries which use SPI flash.

CONFIG_SPI_FLASH_ROM_DRIVER_PATCH

Enable SPI flash ROM driver patched functions

Found in: Component config > SPI Flash driver

Enable this flag to use patched versions of SPI flash ROM driver functions. This option should be enabled, if any one of the following is true: (1) need to write to flash on ESP32-D2WD; (2) main SPI flash is connected to non-default pins; (3) main SPI flash chip is manufactured by ISSI.

CONFIG_SPI_FLASH_ROM_IMPL

Use esp_flash implementation in ROM

Found in: Component config > SPI Flash driver

Enable this flag to use new SPI flash driver functions from ROM instead of ESP-IDF.

If keeping this as "n" in your project, you will have less free IRAM. But you can use all of our flash features.

If making this as "y" in your project, you will increase free IRAM. But you may miss out on some flash features and support for new flash chips.

Currently the ROM cannot support the following features:

- SPI_FLASH_AUTO_SUSPEND (C3, S3)

CONFIG_SPI_FLASH_DANGEROUS_WRITE

Writing to dangerous flash regions

Found in: Component config > SPI Flash driver

SPI flash APIs can optionally abort or return a failure code if erasing or writing addresses that fall at the beginning of flash (covering the bootloader and partition table) or that overlap the app partition that contains the running app.

It is not recommended to ever write to these regions from an IDF app, and this check prevents logic errors or corrupted firmware memory from damaging these regions.

Note that this feature *does not* check calls to the esp_rom_xxx SPI flash ROM functions. These functions should not be called directly from IDF applications.

Available options:

- Aborts (CONFIG_SPI_FLASH_DANGEROUS_WRITE_ABORTS)
- Fails (CONFIG_SPI_FLASH_DANGEROUS_WRITE_FAILS)
- Allowed (CONFIG_SPI_FLASH_DANGEROUS_WRITE_ALLOWED)

CONFIG_SPI_FLASH_BYPASS_BLOCK_ERASE

Bypass a block erase and always do sector erase

Found in: Component config > SPI Flash driver

Some flash chips can have very high "max" erase times, especially for block erase (32KB or 64KB). This option allows to bypass "block erase" and always do sector erase commands. This will be much slower overall in most cases, but improves latency for other code to run.

CONFIG_SPI_FLASH_YIELD_DURING_ERASE

Enables yield operation during flash erase

Found in: Component config > SPI Flash driver

This allows to yield the CPUs between erase commands. Prevents starvation of other tasks. Please use this configuration together with `SPI\_FLASH\_ERASE\_YIELD\_DURATION\_MS` and `SPI\_FLASH\_ERASE\_YIELD\_TICKS` after carefully checking flash datasheet to avoid a watchdog timeout. For more information, please check *SPI Flash API* reference documenation under section *OS Function*.

CONFIG_SPI_FLASH_ERASE_YIELD_DURATION_MS

Duration of erasing to yield CPUs (ms)

Found in: Component config > SPI Flash driver > CONFIG_SPI_FLASH_YIELD_DURING_ERASE

If a duration of one erase command is large then it will yield CPUs after finishing a current command.

CONFIG_SPI_FLASH_ERASE_YIELD_TICKS

CPU release time (tick) for an erase operation

Found in: Component config > SPI Flash driver > CONFIG_SPI_FLASH_YIELD_DURING_ERASE

Defines how many ticks will be before returning to continue a erasing.

CONFIG_SPI_FLASH_WRITE_CHUNK_SIZE

Flash write chunk size

Found in: Component config > SPI Flash driver

Flash write is broken down in terms of multiple (smaller) write operations. This configuration options helps to set individual write chunk size, smaller value here ensures that cache (and non-IRAM resident interrupts) remains disabled for shorter duration.

CONFIG_SPI_FLASH_SIZE_OVERRIDE

Override flash size in bootloader header by ESPTOOLPY_FLASHSIZE

Found in: Component config > SPI Flash driver

SPI Flash driver uses the flash size configured in bootloader header by default. Enable this option to override flash size with latest ESPTOOLPY_FLASHSIZE value from the app header if the size in the bootloader header is incorrect.

CONFIG_SPI_FLASH_CHECK_ERASE_TIMEOUT_DISABLED

Flash timeout checkout disabled

Found in: Component config > SPI Flash driver

This option is helpful if you are using a flash chip whose timeout is quite large or unpredictable.

CONFIG_SPI_FLASH_OVERRIDE_CHIP_DRIVER_LIST

Override default chip driver list

Found in: Component config > SPI Flash driver

This option allows the chip driver list to be customized, instead of using the default list provided by ESP-IDF.

When this option is enabled, the default list is no longer compiled or linked. Instead, the *default_registered_chips* structure must be provided by the user.

See example: custom_chip_driver under examples/storage for more details.

Auto-detect flash chips Contains:

- *CONFIG_SPI_FLASH_SUPPORT_BOYA_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_GD_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_ISSI_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_MXIC_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_MXIC_OPI_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_TH_CHIP*
- *CONFIG_SPI_FLASH_SUPPORT_WINBOND_CHIP*

CONFIG_SPI_FLASH_SUPPORT_ISSI_CHIP

ISSI

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of ISSI chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_SUPPORT_MXIC_CHIP

MXIC

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of MXIC chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_SUPPORT_GD_CHIP

GigaDevice

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of GD (GigaDevice) chips if chip vendor not directly given by `chip\_drv` member of the chip struct. If you are using Wrover modules, please don't disable this, otherwise your flash may not work in 4-bit mode.

This adds support for variant chips, however will extend detecting time and image size. Note that the default chip driver supports the GD chips with product ID 60H.

CONFIG_SPI_FLASH_SUPPORT_WINBOND_CHIP

Winbond

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of Winbond chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_SUPPORT_BOYA_CHIP

BOYA

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of BOYA chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_SUPPORT_TH_CHIP

TH

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of TH chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_SUPPORT_MXIC_OPI_CHIP

mxic (opi)

Found in: Component config > SPI Flash driver > Auto-detect flash chips

Enable this to support auto detection of Octal MXIC chips if chip vendor not directly given by `chip\_drv` member of the chip struct. This adds support for variant chips, however will extend detecting time.

CONFIG_SPI_FLASH_ENABLE_ENCRYPTED_READ_WRITE

Enable encrypted partition read/write operations

Found in: Component config > SPI Flash driver

This option enables flash read/write operations to encrypted partition/s. This option is kept enabled irrespective of state of flash encryption feature. However, in case application is not using flash encryption feature and is in need of some additional memory from IRAM region (~1KB) then this config can be disabled.

SPIFFS Configuration

Contains:

- *Debug Configuration*
- *CONFIG_SPIFFS_USE_MAGIC*
- *CONFIG_SPIFFS_GC_STATS*
- *CONFIG_SPIFFS_PAGE_CHECK*
- *CONFIG_SPIFFS_FOLLOW_SYMLINKS*
- *CONFIG_SPIFFS_MAX_PARTITIONS*
- *CONFIG_SPIFFS_USE_MTIME*
- *CONFIG_SPIFFS_GC_MAX_RUNS*
- *CONFIG_SPIFFS_OBJ_NAME_LEN*
- *CONFIG_SPIFFS_META_LENGTH*
- *SPIFFS Cache Configuration*
- *CONFIG_SPIFFS_PAGE_SIZE*
- *CONFIG_SPIFFS_MTIME_WIDE_64_BITS*

CONFIG_SPIFFS_MAX_PARTITIONS

Maximum Number of Partitions

Found in: Component config > SPIFFS Configuration

Define maximum number of partitions that can be mounted.

Range:

- from 1 to 10

Default value:

- 3

SPIFFS Cache Configuration

Contains:

- *CONFIG_SPIFFS_CACHE*

CONFIG_SPIFFS_CACHE

Enable SPIFFS Cache

Found in: Component config > SPIFFS Configuration > SPIFFS Cache Configuration

Enables/disable memory read caching of nucleus file system operations.

Default value:

- Yes (enabled)

CONFIG_SPIFFS_CACHE_WR

Enable SPIFFS Write Caching

Found in: Component config > SPIFFS Configuration > SPIFFS Cache Configuration > CONFIG_SPIFFS_CACHE

Enables memory write caching for file descriptors in hydrogen.

Default value:

- Yes (enabled)

CONFIG_SPIFFS_CACHE_STATS

Enable SPIFFS Cache Statistics

Found in: Component config > SPIFFS Configuration > SPIFFS Cache Configuration > CONFIG_SPIFFS_CACHE

Enable/disable statistics on caching. Debug/test purpose only.

Default value:

- No (disabled)

CONFIG_SPIFFS_PAGE_CHECK

Enable SPIFFS Page Check

Found in: Component config > SPIFFS Configuration

Always check header of each accessed page to ensure consistent state. If enabled it will increase number of reads from flash, especially if cache is disabled.

Default value:

- Yes (enabled)

CONFIG_SPIFFS_GC_MAX_RUNS

Set Maximum GC Runs

Found in: Component config > SPIFFS Configuration

Define maximum number of GC runs to perform to reach desired free pages.

Range:

- from 1 to 10000

Default value:

- 10

CONFIG_SPIFFS_GC_STATS

Enable SPIFFS GC Statistics

Found in: Component config > SPIFFS Configuration

Enable/disable statistics on gc. Debug/test purpose only.

Default value:

- No (disabled)

CONFIG_SPIFFS_PAGE_SIZE

SPIFFS logical page size

Found in: Component config > SPIFFS Configuration

Logical page size of SPIFFS partition, in bytes. Must be multiple of flash page size (which is usually 256 bytes). Larger page sizes reduce overhead when storing large files, and improve filesystem performance when reading large files. Smaller page sizes reduce overhead when storing small (< page size) files.

Range:
- from 256 to 1024

Default value:
- 256

CONFIG_SPIFFS_OBJ_NAME_LEN

Set SPIFFS Maximum Name Length

Found in: Component config > SPIFFS Configuration

Object name maximum length. Note that this length include the zero-termination character, meaning maximum string of characters can at most be SPIFFS_OBJ_NAME_LEN - 1.

SPIFFS_OBJ_NAME_LEN + SPIFFS_META_LENGTH should not exceed SPIFFS_PAGE_SIZE - 64.

Range:
- from 1 to 256

Default value:
- 32

CONFIG_SPIFFS_FOLLOW_SYMLINKS

Enable symbolic links for image creation

Found in: Component config > SPIFFS Configuration

If this option is enabled, symbolic links are taken into account during partition image creation.

Default value:
- No (disabled)

CONFIG_SPIFFS_USE_MAGIC

Enable SPIFFS Filesystem Magic

Found in: Component config > SPIFFS Configuration

Enable this to have an identifiable spiffs filesystem. This will look for a magic in all sectors to determine if this is a valid spiffs system or not at mount time.

Default value:
- Yes (enabled)

CONFIG_SPIFFS_USE_MAGIC_LENGTH

Enable SPIFFS Filesystem Length Magic

Found in: Component config > SPIFFS Configuration > CONFIG_SPIFFS_USE_MAGIC

If this option is enabled, the magic will also be dependent on the length of the filesystem. For example, a filesystem configured and formatted for 4 megabytes will not be accepted for mounting with a configuration defining the filesystem as 2 megabytes.

Default value:

- Yes (enabled)

CONFIG_SPIFFS_META_LENGTH

Size of per-file metadata field

Found in: Component config > SPIFFS Configuration

This option sets the number of extra bytes stored in the file header. These bytes can be used in an application-specific manner. Set this to at least 4 bytes to enable support for saving file modification time.

SPIFFS_OBJ_NAME_LEN + SPIFFS_META_LENGTH should not exceed SPIFFS_PAGE_SIZE - 64.

Default value:

- 4

CONFIG_SPIFFS_USE_MTIME

Save file modification time

Found in: Component config > SPIFFS Configuration

If enabled, then the first 4 bytes of per-file metadata will be used to store file modification time (mtime), accessible through stat/fstat functions. Modification time is updated when the file is opened.

Default value:

- Yes (enabled)

CONFIG_SPIFFS_MTIME_WIDE_64_BITS

The time field occupies 64 bits in the image instead of 32 bits

Found in: Component config > SPIFFS Configuration

If this option is not set, the time field is 32 bits (up to 2106 year), otherwise it is 64 bits and make sure it matches SPIFFS_META_LENGTH. If the chip already has the spiffs image with the time field = 32 bits then this option cannot be applied in this case. Erase it first before using this option. To resolve the Y2K38 problem for the spiffs, use a toolchain with 64-bit time_t support.

Default value:

- No (disabled) if *CONFIG_SPIFFS_META_LENGTH* >= 8

Debug Configuration

Contains:

- *CONFIG_SPIFFS_DBG*
- *CONFIG_SPIFFS_API_DBG*
- *CONFIG_SPIFFS_CACHE_DBG*
- *CONFIG_SPIFFS_CHECK_DBG*
- *CONFIG_SPIFFS_TEST_VISUALISATION*
- *CONFIG_SPIFFS_GC_DBG*

CONFIG_SPIFFS_DBG

Enable general SPIFFS debug

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enabling this option will print general debug messages to the console.

Default value:

- No (disabled)

CONFIG_SPIFFS_API_DBG

Enable SPIFFS API debug

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enabling this option will print API debug messages to the console.

Default value:

- No (disabled)

CONFIG_SPIFFS_GC_DBG

Enable SPIFFS Garbage Cleaner debug

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enabling this option will print GC debug messages to the console.

Default value:

- No (disabled)

CONFIG_SPIFFS_CACHE_DBG

Enable SPIFFS Cache debug

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enabling this option will print cache debug messages to the console.

Default value:

- No (disabled)

CONFIG_SPIFFS_CHECK_DBG

Enable SPIFFS Filesystem Check debug

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enabling this option will print Filesystem Check debug messages to the console.

Default value:

- No (disabled)

CONFIG_SPIFFS_TEST_VISUALISATION

Enable SPIFFS Filesystem Visualization

Found in: Component config > SPIFFS Configuration > Debug Configuration

Enable this option to enable SPIFFS_vis function in the API.

Default value:

- No (disabled)

TCP Transport Contains:

- *Websocket*

Websocket Contains:

- *CONFIG_WS_TRANSPORT*

CONFIG_WS_TRANSPORT

Enable Websocket Transport

Found in: Component config > TCP Transport > Websocket

Enable support for creating websocket transport.

Default value:

- Yes (enabled)

CONFIG_WS_BUFFER_SIZE

Websocket transport buffer size

Found in: Component config > TCP Transport > Websocket > CONFIG_WS_TRANSPORT

Size of the buffer used for constructing the HTTP Upgrade request during connect

Default value:

- 1024

CONFIG_WS_DYNAMIC_BUFFER

Using dynamic websocket transport buffer

Found in: Component config > TCP Transport > Websocket > CONFIG_WS_TRANSPORT

If enable this option, websocket transport buffer will be freed after connection succeed to save more heap.

Default value:

- No (disabled)

Ultra Low Power (ULP) Co-processor

Contains:

- *CONFIG_ULP_ROM_PRINT_ENABLE*
- *CONFIG_ULP_COPROC_ENABLED*
- *ULP Debugging Options*
- *ULP RISC-V Settings*

CONFIG_ULP_COPROC_ENABLED

Enable Ultra Low Power (ULP) Co-processor

Found in: Component config > Ultra Low Power (ULP) Co-processor

Enable this feature if you plan to use the ULP Co-processor. Once this option is enabled, further ULP co-processor configuration will appear in the menu.

Default value:

- No (disabled)

CONFIG_ULP_COPROC_TYPE

ULP Co-processor type

Found in: Component config > Ultra Low Power (ULP) Co-processor > CONFIG_ULP_COPROC_ENABLED

Choose the ULP Coprocessor type: ULP FSM (Finite State Machine) or ULP RISC-V.

Available options:

- ULP FSM (Finite State Machine) (CONFIG_ULP_COPROC_TYPE_FSM)
- ULP RISC-V (CONFIG_ULP_COPROC_TYPE_RISCV)
- LP core RISC-V (CONFIG_ULP_COPROC_TYPE_LP_CORE)

CONFIG_ULP_COPROC_RESERVE_MEM

RTC slow memory reserved for coprocessor

Found in: Component config > Ultra Low Power (ULP) Co-processor > CONFIG_ULP_COPROC_ENABLED

Bytes of memory to reserve for ULP Co-processor firmware & data. Data is reserved at the beginning of RTC slow memory.

Range:
- from 32 to 8176 if *CONFIG_ULP_COPROC_ENABLED*

Default value:
- 4096 if *CONFIG_ULP_COPROC_ENABLED*

ULP RISC-V Settings Contains:

- *CONFIG_ULP_RISCV_UART_BAUDRATE*
- *CONFIG_ULP_RISCV_INTERRUPT_ENABLE*
- *CONFIG_ULP_RISCV_I2C_RW_TIMEOUT*

CONFIG_ULP_RISCV_INTERRUPT_ENABLE

Enable ULP RISC-V interrupts

Found in: Component config > Ultra Low Power (ULP) Co-processor > ULP RISC-V Settings

Turn on this setting to enabled interrupts on the ULP RISC-V core.

Default value:
- No (disabled) if *CONFIG_ULP_COPROC_TYPE_RISCV*

CONFIG_ULP_RISCV_UART_BAUDRATE

Baudrate used by the bitbanged ULP RISC-V UART driver

Found in: Component config > Ultra Low Power (ULP) Co-processor > ULP RISC-V Settings

The accuracy of the bitbanged UART driver is limited, it is not recommend to increase the value above 19200.

Default value:
- 9600 if *CONFIG_ULP_COPROC_TYPE_RISCV*

CONFIG_ULP_RISCV_I2C_RW_TIMEOUT

Set timeout for ULP RISC-V I2C transaction timeout in ticks.

Found in: Component config > Ultra Low Power (ULP) Co-processor > ULP RISC-V Settings

Set the ULP RISC-V I2C read/write timeout. Set this value to -1 if the ULP RISC-V I2C read and write APIs should wait forever. Please note that the tick rate of the ULP co-processor would be different than the OS tick rate of the main core and therefore can have different timeout value depending on which core the API is invoked on.

Range:
- from -1 to 4294967295 if *CONFIG_ULP_COPROC_TYPE_RISCV*

Default value:
- 500 if *CONFIG_ULP_COPROC_TYPE_RISCV*

CONFIG_ULP_ROM_PRINT_ENABLE

Enable print utilities from LP ROM

Found in: Component config > Ultra Low Power (ULP) Co-processor

Set this option to enable printf functionality from LP ROM. This option can help reduce the LP core binary size by not linking printf functionality from RAM code. Note: For LP ROM prints to work properly, make sure that the LP core boots from the LP ROM.

Default value:

- Yes (enabled) if *CONFIG_ULP_COPROC_TYPE_LP_CORE* && ESP_ROM_HAS_LP_ROM

ULP Debugging Options

Contains:

- *CONFIG_ULP_PANIC_OUTPUT_ENABLE*
- *CONFIG_ULP_HP_UART_CONSOLE_PRINT*

CONFIG_ULP_PANIC_OUTPUT_ENABLE

Enable panic handler which outputs over LP UART

Found in: Component config > Ultra Low Power (ULP) Co-processor > ULP Debugging Options

Set this option to enable panic handler functionality. If this option is enabled then the LP Core will output a panic dump over LP UART, similar to what the main core does. Output depends on LP UART already being initialized and configured. Disabling this option will reduce the LP core binary size by not linking in panic handler functionality.

CONFIG_ULP_HP_UART_CONSOLE_PRINT

Route lp_core_printf to the console HP-UART

Found in: Component config > Ultra Low Power (ULP) Co-processor > ULP Debugging Options

Set this option to route lp_core_printf to the console HP-UART. This allows you to easily view print outputs from the LP core, without having to connect to the LP-UART. This option comes with the following limitations:

1. There is no mutual exclusion between the HP-Core and the LP-Core accessing the HP-UART, which means that if both cores are logging heavily the output strings might get mangled together. 2. The HP-UART can only work while the HP-Core is running, which means that if the HP-Core is in deep sleep, the LP-Core will not be able to print to the console HP-UART.

Due to these limitations it is only recommended to use this option for easy debugging. For more serious use-cases you should use the LP-UART.

Unity unit testing library

Contains:

- *CONFIG_UNITY_ENABLE_COLOR*
- *CONFIG_UNITY_ENABLE_IDF_TEST_RUNNER*
- *CONFIG_UNITY_ENABLE_FIXTURE*
- *CONFIG_UNITY_ENABLE_BACKTRACE_ON_FAIL*
- *CONFIG_UNITY_ENABLE_64BIT*
- *CONFIG_UNITY_ENABLE_DOUBLE*
- *CONFIG_UNITY_ENABLE_FLOAT*

CONFIG_UNITY_ENABLE_FLOAT

Support for float type

Found in: Component config > Unity unit testing library

If not set, assertions on float arguments will not be available.

Default value:

- Yes (enabled)

CONFIG_UNITY_ENABLE_DOUBLE

Support for double type

Found in: Component config > Unity unit testing library

If not set, assertions on double arguments will not be available.

Default value:

- Yes (enabled)

CONFIG_UNITY_ENABLE_64BIT

Support for 64-bit integer types

Found in: Component config > Unity unit testing library

If not set, assertions on 64-bit integer types will always fail. If this feature is enabled, take care not to pass pointers (which are 32 bit) to UNITY_ASSERT_EQUAL, as that will cause pointer-to-int-cast warnings.

Default value:

- No (disabled)

CONFIG_UNITY_ENABLE_COLOR

Colorize test output

Found in: Component config > Unity unit testing library

If set, Unity will colorize test results using console escape sequences.

Default value:

- No (disabled)

CONFIG_UNITY_ENABLE_IDF_TEST_RUNNER

Include ESP-IDF test registration/running helpers

Found in: Component config > Unity unit testing library

If set, then the following features will be available:

- TEST_CASE macro which performs automatic registration of test functions
- Functions to run registered test functions: unity_run_all_tests, unity_run_tests_with_filter, unity_run_single_test_by_name.
- Interactive menu which lists test cases and allows choosing the tests to be run, available via unity_run_menu function.

Disable if a different test registration mechanism is used.

Default value:

- Yes (enabled)

CONFIG_UNITY_ENABLE_FIXTURE

Include Unity test fixture

Found in: Component config > Unity unit testing library

If set, unity_fixture.h header file and associated source files are part of the build. These provide an optional set of macros and functions to implement test groups.

Default value:

- No (disabled)

CONFIG_UNITY_ENABLE_BACKTRACE_ON_FAIL

Print a backtrace when a unit test fails

Found in: Component config > Unity unit testing library

If set, the unity framework will print the backtrace information before jumping back to the test menu. The jumping is usually occurs in assert functions such as TEST_ASSERT, TEST_FAIL etc.

Default value:

- No (disabled)

USB-OTG

Contains:

- *CONFIG_USB_HOST_ENABLE_ENUM_FILTER_CALLBACK*
- *CONFIG_USB_HOST_HW_BUFFER_BIAS*
- *CONFIG_USB_HOST_CONTROL_TRANSFER_MAX_SIZE*
- *Root Hub configuration*
- *CONFIG_USB_HOST_EXT_HUB_SUPPORT*

CONFIG_USB_HOST_CONTROL_TRANSFER_MAX_SIZE

Largest size (in bytes) of transfers to/from default endpoints

Found in: Component config > USB-OTG

Each USB device attached is allocated a dedicated buffer for its OUT/IN transfers to/from the device's control endpoint. The maximum size of that buffer is determined by this option. The limited size of the transfer buffer have the following implications: - The maximum length of control transfers is limited - Device's with configuration descriptors larger than this limit cannot be supported

Default value:

- 256

CONFIG_USB_HOST_HW_BUFFER_BIAS

Hardware FIFO size biasing

Found in: Component config > USB-OTG

The underlying hardware has size adjustable FIFOs to cache USB packets on reception (IN) or for transmission (OUT). The size of these FIFOs will affect the largest MPS (maximum packet size) and the maximum number of packets that can be cached at any one time. The hardware contains the following FIFOS: RX (for all IN packets), Non-periodic TX (for Bulk and Control OUT packets), and Periodic TX (for Interrupt and Isochronous OUT packets). This configuration option allows biasing the FIFO sizes towards a particular use case, which may be necessary for devices that have endpoints with large MPS. The MPS limits for each biasing are listed below:

Balanced: - IN (all transfer types), 408 bytes - OUT non-periodic (Bulk/Control), 192 bytes (i.e., 3 x 64 byte packets) - OUT periodic (Interrupt/Isochronous), 192 bytes

Bias IN: - IN (all transfer types), 600 bytes - OUT non-periodic (Bulk/Control), 64 bytes (i.e., 1 x 64 byte packets) - OUT periodic (Interrupt/Isochronous), 128 bytes

Bias Periodic OUT: - IN (all transfer types), 128 bytes - OUT non-periodic (Bulk/Control), 64 bytes (i.e., 1 x 64 byte packets) - OUT periodic (Interrupt/Isochronous), 600 bytes

Available options:

- Balanced (CONFIG_USB_HOST_HW_BUFFER_BIAS_BALANCED)
- Bias IN (CONFIG_USB_HOST_HW_BUFFER_BIAS_IN)
- Periodic OUT (CONFIG_USB_HOST_HW_BUFFER_BIAS_PERIODIC_OUT)

Root Hub configuration Contains:

- *CONFIG_USB_HOST_DEBOUNCE_DELAY_MS*
- *CONFIG_USB_HOST_RESET_HOLD_MS*
- *CONFIG_USB_HOST_RESET_RECOVERY_MS*
- *CONFIG_USB_HOST_SET_ADDR_RECOVERY_MS*

CONFIG_USB_HOST_DEBOUNCE_DELAY_MS

Debounce delay in ms

Found in: Component config > USB-OTG > Root Hub configuration

On connection of a USB device, the USB 2.0 specification requires a "debounce interval with a minimum duration of 100ms" to allow the connection to stabilize (see USB 2.0 chapter 7.1.7.3 for more details). During the debounce interval, no new connection/disconnection events are registered.

The default value is set to 250 ms to be safe.

Default value:

- 250

CONFIG_USB_HOST_RESET_HOLD_MS

Reset hold in ms

Found in: Component config > USB-OTG > Root Hub configuration

The reset signaling can be generated on any Hub or Host Controller port by request from the USB System Software. The USB 2.0 specification requires that "the reset signaling must be driven for a minimum of 10ms" (see USB 2.0 chapter 7.1.7.5 for more details). After the reset, the hub port will transition to the Enabled state (refer to Section 11.5).

The default value is set to 30 ms to be safe.

Default value:

- 30

CONFIG_USB_HOST_RESET_RECOVERY_MS

Reset recovery delay in ms

Found in: Component config > USB-OTG > Root Hub configuration

After a port stops driving the reset signal, the USB 2.0 specification requires that the "USB System Software guarantees a minimum of 10 ms for reset recovery" before the attached device is expected to respond to data transfers (see USB 2.0 chapter 7.1.7.3 for more details). The device may ignore any data transfers during the recovery interval.

The default value is set to 30 ms to be safe.

Default value:

- 30

CONFIG_USB_HOST_SET_ADDR_RECOVERY_MS

SetAddress() recovery time in ms

Found in: Component config > USB-OTG > Root Hub configuration

"After successful completion of the Status stage, the device is allowed a SetAddress() recovery interval of 2 ms. At the end of this interval, the device must be able to accept Setup packets addressed to the new address. Also, at the end of the recovery interval, the device must not respond to tokens sent to the old address (unless, of course, the old and new address is the same)." See USB 2.0 chapter 9.2.6.3 for more details.

The default value is set to 10 ms to be safe.

Default value:

- 10

CONFIG_USB_HOST_ENABLE_ENUM_FILTER_CALLBACK

Enable enumeration filter callback

Found in: Component config > USB-OTG

The enumeration filter callback is called before enumeration of each newly attached device. This callback allows users to control whether a device should be enumerated, and what configuration number to use when enumerating a device.

If enabled, the enumeration filter callback can be set via 'usb_host_config_t' when calling 'usb_host_install()'.

Default value:

- No (disabled)

CONFIG_USB_HOST_EXT_HUB_SUPPORT

Support USB HUB (Experimental)

Found in: Component config > USB-OTG

Feature is under development.

Default value:

- No (disabled) if *CONFIG_IDF_EXPERIMENTAL_FEATURES*

Virtual file system

Contains:

- *CONFIG_VFS_SUPPORT_IO*

CONFIG_VFS_SUPPORT_IO

Provide basic I/O functions

Found in: Component config > Virtual file system

If enabled, the following functions are provided by the VFS component.

open, close, read, write, pread, pwrite, lseek, fstat, fsync, ioctl, fcntl

Filesystem drivers can then be registered to handle these functions for specific paths.

Disabling this option can save memory when the support for these functions is not required.

Note that the following functions can still be used with socket file descriptors when this option is disabled:

close, read, write, ioctl, fcntl.

Default value:

- Yes (enabled)

CONFIG_VFS_SUPPORT_DIR

Provide directory related functions

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO

If enabled, the following functions are provided by the VFS component.

stat, link, unlink, rename, utime, access, truncate, rmdir, mkdir, opendir, closedir, readdir, readdir_r, seekdir, telldir, rewinddir

Filesystem drivers can then be registered to handle these functions for specific paths.

Disabling this option can save memory when the support for these functions is not required.

Default value:

- Yes (enabled)

CONFIG_VFS_SUPPORT_SELECT

Provide select function

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO

If enabled, select function is provided by the VFS component, and can be used on peripheral file descriptors (such as UART) and sockets at the same time.

If disabled, the default select implementation will be provided by LWIP for sockets only.

Disabling this option can reduce code size if support for "select" on UART file descriptors is not required.

CONFIG_VFS_SUPPRESS_SELECT_DEBUG_OUTPUT

Suppress select() related debug outputs

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO > CONFIG_VFS_SUPPORT_SELECT

Select() related functions might produce an inconveniently lot of debug outputs when one sets the default log level to DEBUG or higher. It is possible to suppress these debug outputs by enabling this option.

Default value:

- Yes (enabled)

CONFIG_VFS_SELECT_IN_RAM

Make VFS driver select() callbacks IRAM-safe

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO > CONFIG_VFS_SUPPORT_SELECT

If enabled, VFS driver select() callback function will be placed in IRAM.

Default value:

- No (disabled)

CONFIG_VFS_SUPPORT_TERMIOS

Provide termios.h functions

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO

Disabling this option can save memory when the support for termios.h is not required.

Default value:
- Yes (enabled)

CONFIG_VFS_MAX_COUNT

Maximum Number of Virtual Filesystems

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO

Define maximum number of virtual filesystems that can be registered.

Range:
- from 1 to 20

Default value:
- 8

Host File System I/O (Semihosting) Contains:

- *CONFIG_VFS_SEMIHOSTFS_MAX_MOUNT_POINTS*

CONFIG_VFS_SEMIHOSTFS_MAX_MOUNT_POINTS

Host FS: Maximum number of the host filesystem mount points

Found in: Component config > Virtual file system > CONFIG_VFS_SUPPORT_IO > Host File System I/O (Semihosting)

Define maximum number of host filesystem mount points.

Default value:
- 1

Wear Levelling Contains:

- *CONFIG_WL_SECTOR_MODE*
- *CONFIG_WL_SECTOR_SIZE*

CONFIG_WL_SECTOR_SIZE

Wear Levelling library sector size

Found in: Component config > Wear Levelling

Sector size used by wear levelling library. You can set default sector size or size that will fit to the flash device sector size.

With sector size set to 4096 bytes, wear levelling library is more efficient. However if FAT filesystem is used on top of wear levelling library, it will need more temporary storage: 4096 bytes for each mounted filesystem and 4096 bytes for each opened file.

With sector size set to 512 bytes, wear levelling library will perform more operations with flash memory, but less RAM will be used by FAT filesystem library (512 bytes for the filesystem and 512 bytes for each file opened).

Available options:

- 512 (CONFIG_WL_SECTOR_SIZE_512)
- 4096 (CONFIG_WL_SECTOR_SIZE_4096)

CONFIG_WL_SECTOR_MODE

Sector store mode

Found in: Component config > Wear Levelling

Specify the mode to store data into flash:

- In Performance mode a data will be stored to the RAM and then stored back to the flash. Compared to the Safety mode, this operation is faster, but if power will be lost when erase sector operation is in progress, then the data from complete flash device sector will be lost.
- In Safety mode data from complete flash device sector will be read from flash, modified, and then stored back to flash. Compared to the Performance mode, this operation is slower, but if power is lost during erase sector operation, then the data from full flash device sector will not be lost.

Available options:

- Performance (CONFIG_WL_SECTOR_MODE_PERF)
- Safety (CONFIG_WL_SECTOR_MODE_SAFE)

Wi-Fi Provisioning Manager

Contains:

- *CONFIG_WIFI_PROV_BLE_BONDING*
- *CONFIG_WIFI_PROV_BLE_SEC_CONN*
- *CONFIG_WIFI_PROV_BLE_FORCE_ENCRYPTION*
- *CONFIG_WIFI_PROV_KEEP_BLE_ON_AFTER_PROV*
- *CONFIG_WIFI_PROV_SCAN_MAX_ENTRIES*
- *CONFIG_WIFI_PROV_AUTOSTOP_TIMEOUT*
- *CONFIG_WIFI_PROV_STA_SCAN_METHOD*

CONFIG_WIFI_PROV_SCAN_MAX_ENTRIES

Max Wi-Fi Scan Result Entries

Found in: Component config > Wi-Fi Provisioning Manager

This sets the maximum number of entries of Wi-Fi scan results that will be kept by the provisioning manager

Range:

- from 1 to 255

Default value:

- 16

CONFIG_WIFI_PROV_AUTOSTOP_TIMEOUT

Provisioning auto-stop timeout

Found in: Component config > Wi-Fi Provisioning Manager

Time (in seconds) after which the Wi-Fi provisioning manager will auto-stop after connecting to a Wi-Fi network successfully.

Range:

- from 5 to 600

Default value:

- 30

CONFIG_WIFI_PROV_BLE_BONDING

Enable BLE bonding

Found in: Component config > Wi-Fi Provisioning Manager

This option is applicable only when provisioning transport is BLE.

CONFIG_WIFI_PROV_BLE_SEC_CONN

Enable BLE Secure connection flag

Found in: Component config > Wi-Fi Provisioning Manager

Used to enable Secure connection support when provisioning transport is BLE.

Default value:

- Yes (enabled) if *CONFIG_BT_NIMBLE_ENABLED*

CONFIG_WIFI_PROV_BLE_FORCE_ENCRYPTION

Force Link Encryption during characteristic Read / Write

Found in: Component config > Wi-Fi Provisioning Manager

Used to enforce link encryption when attempting to read / write characteristic

CONFIG_WIFI_PROV_KEEP_BLE_ON_AFTER_PROV

Keep BT on after provisioning is done

Found in: Component config > Wi-Fi Provisioning Manager

CONFIG_WIFI_PROV_DISCONNECT_AFTER_PROV

Terminate connection after provisioning is done

Found in: Component config > Wi-Fi Provisioning Manager > CONFIG_WIFI_PROV_KEEP_BLE_ON_AFTER_PROV

Default value:

- Yes (enabled) if *CONFIG_WIFI_PROV_KEEP_BLE_ON_AFTER_PROV*

CONFIG_WIFI_PROV_STA_SCAN_METHOD

Wifi Provisioning Scan Method

Found in: Component config > Wi-Fi Provisioning Manager

Available options:

- All Channel Scan (CONFIG_WIFI_PROV_STA_ALL_CHANNEL_SCAN)
  Scan will end after scanning the entire channel. This option is useful in Mesh WiFi Systems.
- Fast Scan (CONFIG_WIFI_PROV_STA_FAST_SCAN)
  Scan will end after an AP matching with the SSID has been detected.

CONFIG_IDF_EXPERIMENTAL_FEATURES

Make experimental features visible

Found in:

By enabling this option, ESP-IDF experimental feature options will be visible.

Note you should still enable a certain experimental feature option to use it, and you should read the corresponding risk warning and known issue list carefully.

Current experimental feature list:

- CONFIG_ESPTOOLPY_FLASHFREQ_120M && CONFIG_ESPTOOLPY_FLASH_SAMPLE_MODE_DTR
- CONFIG_SPIRAM_SPEED_120M && CONFIG_SPIRAM_MODE_OCT
- CONFIG_BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH
- CONFIG_MBEDTLS_USE_CRYPTO_ROM_IMPL
- CONFIG_ESP_WIFI_EAP_TLS1_3
- CONFIG_ESP_WIFI_ENABLE_ROAMING_APP

Default value:

- No (disabled)

Deprecated options and their replacements

- CONFIG_A2DP_ENABLE (*CONFIG_BT_A2DP_ENABLE*)
- **CONFIG_A2D_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_A2D_TRACE_LEVEL*)**
  - CONFIG_A2D_TRACE_LEVEL_NONE
  - CONFIG_A2D_TRACE_LEVEL_ERROR
  - CONFIG_A2D_TRACE_LEVEL_WARNING
  - CONFIG_A2D_TRACE_LEVEL_API
  - CONFIG_A2D_TRACE_LEVEL_EVENT
  - CONFIG_A2D_TRACE_LEVEL_DEBUG
  - CONFIG_A2D_TRACE_LEVEL_VERBOSE
- CONFIG_ADC2_DISABLE_DAC (*CONFIG_ADC_DISABLE_DAC*)
- **CONFIG_APPL_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_APPL_TRACE_LEVEL*)**
  - CONFIG_APPL_TRACE_LEVEL_NONE
  - CONFIG_APPL_TRACE_LEVEL_ERROR
  - CONFIG_APPL_TRACE_LEVEL_WARNING
  - CONFIG_APPL_TRACE_LEVEL_API
  - CONFIG_APPL_TRACE_LEVEL_EVENT
  - CONFIG_APPL_TRACE_LEVEL_DEBUG
  - CONFIG_APPL_TRACE_LEVEL_VERBOSE
- CONFIG_APP_ANTI_ROLLBACK (*CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK*)
- CONFIG_APP_ROLLBACK_ENABLE (*CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE*)
- CONFIG_APP_SECURE_VERSION (*CONFIG_BOOTLOADER_APP_SECURE_VERSION*)
- CONFIG_APP_SECURE_VERSION_SIZE_EFUSE_FIELD (*CONFIG_BOOTLOADER_APP_SEC_VER_SIZE_EFUSE_FIEL*
- **CONFIG_AVCT_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_AVCT_TRACE_LEVEL*)**
  - CONFIG_AVCT_TRACE_LEVEL_NONE
  - CONFIG_AVCT_TRACE_LEVEL_ERROR
  - CONFIG_AVCT_TRACE_LEVEL_WARNING
  - CONFIG_AVCT_TRACE_LEVEL_API
  - CONFIG_AVCT_TRACE_LEVEL_EVENT
  - CONFIG_AVCT_TRACE_LEVEL_DEBUG
  - CONFIG_AVCT_TRACE_LEVEL_VERBOSE
- **CONFIG_AVDT_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_AVDT_TRACE_LEVEL*)**
  - CONFIG_AVDT_TRACE_LEVEL_NONE
  - CONFIG_AVDT_TRACE_LEVEL_ERROR
  - CONFIG_AVDT_TRACE_LEVEL_WARNING
  - CONFIG_AVDT_TRACE_LEVEL_API

  - CONFIG_AVDT_TRACE_LEVEL_EVENT
  - CONFIG_AVDT_TRACE_LEVEL_DEBUG
  - CONFIG_AVDT_TRACE_LEVEL_VERBOSE
- **CONFIG_AVRC_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_AVRC_TRACE_LEVEL*)
  - CONFIG_AVRC_TRACE_LEVEL_NONE
  - CONFIG_AVRC_TRACE_LEVEL_ERROR
  - CONFIG_AVRC_TRACE_LEVEL_WARNING
  - CONFIG_AVRC_TRACE_LEVEL_API
  - CONFIG_AVRC_TRACE_LEVEL_EVENT
  - CONFIG_AVRC_TRACE_LEVEL_DEBUG
  - CONFIG_AVRC_TRACE_LEVEL_VERBOSE
- CONFIG_BLE_ACTIVE_SCAN_REPORT_ADV_SCAN_RSP_INDIVIDUALLY (*CONFIG_BT_BLE_ACT_SCAN_REP_ADV_SCAN*)
- CONFIG_BLE_ESTABLISH_LINK_CONNECTION_TIMEOUT (*CONFIG_BT_BLE_ESTAB_LINK_CONN_TOUT*)
- CONFIG_BLE_HOST_QUEUE_CONGESTION_CHECK (*CONFIG_BT_BLE_HOST_QUEUE_CONG_CHECK*)
- CONFIG_BLE_MESH_GATT_PROXY (*CONFIG_BLE_MESH_GATT_PROXY_SERVER*)
- CONFIG_BLE_SMP_ENABLE (*CONFIG_BT_BLE_SMP_ENABLE*)
- CONFIG_BLUEDROID_MEM_DEBUG (*CONFIG_BT_BLUEDROID_MEM_DEBUG*)
- **CONFIG_BLUEDROID_PINNED_TO_CORE_CHOICE** (*CONFIG_BT_BLUEDROID_PINNED_TO_CORE_CHOICE*)
  - CONFIG_BLUEDROID_PINNED_TO_CORE_0
  - CONFIG_BLUEDROID_PINNED_TO_CORE_1
- **CONFIG_BLUFI_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_BLUFI_TRACE_LEVEL*)
  - CONFIG_BLUFI_TRACE_LEVEL_NONE
  - CONFIG_BLUFI_TRACE_LEVEL_ERROR
  - CONFIG_BLUFI_TRACE_LEVEL_WARNING
  - CONFIG_BLUFI_TRACE_LEVEL_API
  - CONFIG_BLUFI_TRACE_LEVEL_EVENT
  - CONFIG_BLUFI_TRACE_LEVEL_DEBUG
  - CONFIG_BLUFI_TRACE_LEVEL_VERBOSE
- CONFIG_BNEP_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_BNEP_TRACE_LEVEL*)
- CONFIG_BROWNOUT_DET (*CONFIG_ESP_BROWNOUT_DET*)
- **CONFIG_BROWNOUT_DET_LVL_SEL** (*CONFIG_ESP_BROWNOUT_DET_LVL_SEL*)
  - CONFIG_BROWNOUT_DET_LVL_SEL_7
  - CONFIG_BROWNOUT_DET_LVL_SEL_6
  - CONFIG_BROWNOUT_DET_LVL_SEL_5
  - CONFIG_BROWNOUT_DET_LVL_SEL_4
  - CONFIG_BROWNOUT_DET_LVL_SEL_3
  - CONFIG_BROWNOUT_DET_LVL_SEL_2
  - CONFIG_BROWNOUT_DET_LVL_SEL_1
- **CONFIG_BTC_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_BTC_TRACE_LEVEL*)
  - CONFIG_BTC_TRACE_LEVEL_NONE
  - CONFIG_BTC_TRACE_LEVEL_ERROR
  - CONFIG_BTC_TRACE_LEVEL_WARNING
  - CONFIG_BTC_TRACE_LEVEL_API
  - CONFIG_BTC_TRACE_LEVEL_EVENT
  - CONFIG_BTC_TRACE_LEVEL_DEBUG
  - CONFIG_BTC_TRACE_LEVEL_VERBOSE
- CONFIG_BTC_TASK_STACK_SIZE (*CONFIG_BT_BTC_TASK_STACK_SIZE*)
- **CONFIG_BTH_LOG_SDP_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_SDP_TRACE_LEVEL*)
  - CONFIG_SDP_TRACE_LEVEL_NONE
  - CONFIG_SDP_TRACE_LEVEL_ERROR
  - CONFIG_SDP_TRACE_LEVEL_WARNING
  - CONFIG_SDP_TRACE_LEVEL_API
  - CONFIG_SDP_TRACE_LEVEL_EVENT
  - CONFIG_SDP_TRACE_LEVEL_DEBUG
  - CONFIG_SDP_TRACE_LEVEL_VERBOSE

- **CONFIG_BTIF_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_BTIF_TRACE_LEVEL*)
  - CONFIG_BTIF_TRACE_LEVEL_NONE
  - CONFIG_BTIF_TRACE_LEVEL_ERROR
  - CONFIG_BTIF_TRACE_LEVEL_WARNING
  - CONFIG_BTIF_TRACE_LEVEL_API
  - CONFIG_BTIF_TRACE_LEVEL_EVENT
  - CONFIG_BTIF_TRACE_LEVEL_DEBUG
  - CONFIG_BTIF_TRACE_LEVEL_VERBOSE
- **CONFIG_BTM_INITIAL_TRACE_LEVEL** (*CONFIG_BT_LOG_BTM_TRACE_LEVEL*)
  - CONFIG_BTM_TRACE_LEVEL_NONE
  - CONFIG_BTM_TRACE_LEVEL_ERROR
  - CONFIG_BTM_TRACE_LEVEL_WARNING
  - CONFIG_BTM_TRACE_LEVEL_API
  - CONFIG_BTM_TRACE_LEVEL_EVENT
  - CONFIG_BTM_TRACE_LEVEL_DEBUG
  - CONFIG_BTM_TRACE_LEVEL_VERBOSE
- CONFIG_BTU_TASK_STACK_SIZE (*CONFIG_BT_BTU_TASK_STACK_SIZE*)
- CONFIG_BT_NIMBLE_ACL_BUF_COUNT (*CONFIG_BT_NIMBLE_TRANSPORT_ACL_FROM_LL_COUNT*)
- CONFIG_BT_NIMBLE_ACL_BUF_SIZE (*CONFIG_BT_NIMBLE_TRANSPORT_ACL_SIZE*)
- CONFIG_BT_NIMBLE_HCI_EVT_BUF_SIZE (*CONFIG_BT_NIMBLE_TRANSPORT_EVT_SIZE*)
- CONFIG_BT_NIMBLE_HCI_EVT_HI_BUF_COUNT (*CONFIG_BT_NIMBLE_TRANSPORT_EVT_COUNT*)
- CONFIG_BT_NIMBLE_HCI_EVT_LO_BUF_COUNT (*CONFIG_BT_NIMBLE_TRANSPORT_EVT_DISCARD_COUNT*)
- CONFIG_BT_NIMBLE_MSYS1_BLOCK_COUNT (*CONFIG_BT_NIMBLE_MSYS_1_BLOCK_COUNT*)
- CONFIG_BT_NIMBLE_SM_SC_LVL (*CONFIG_BT_NIMBLE_SM_LVL*)
- CONFIG_BT_NIMBLE_TASK_STACK_SIZE (*CONFIG_BT_NIMBLE_HOST_TASK_STACK_SIZE*)
- CONFIG_CLASSIC_BT_ENABLED (*CONFIG_BT_CLASSIC_ENABLED*)
- **CONFIG_CONSOLE_UART** (*CONFIG_ESP_CONSOLE_UART*)
  - CONFIG_CONSOLE_UART_DEFAULT
  - CONFIG_CONSOLE_UART_CUSTOM
  - CONFIG_CONSOLE_UART_NONE, CONFIG_ESP_CONSOLE_UART_NONE
- CONFIG_CONSOLE_UART_BAUDRATE (*CONFIG_ESP_CONSOLE_UART_BAUDRATE*)
- **CONFIG_CONSOLE_UART_NUM** (*CONFIG_ESP_CONSOLE_UART_NUM*)
  - CONFIG_CONSOLE_UART_CUSTOM_NUM_0
  - CONFIG_CONSOLE_UART_CUSTOM_NUM_1
- CONFIG_CONSOLE_UART_RX_GPIO (*CONFIG_ESP_CONSOLE_UART_RX_GPIO*)
- CONFIG_CONSOLE_UART_TX_GPIO (*CONFIG_ESP_CONSOLE_UART_TX_GPIO*)
- CONFIG_CXX_EXCEPTIONS (*CONFIG_COMPILER_CXX_EXCEPTIONS*)
- CONFIG_CXX_EXCEPTIONS_EMG_POOL_SIZE (*CONFIG_COMPILER_CXX_EXCEPTIONS_EMG_POOL_SIZE*)
- CONFIG_EFUSE_SECURE_VERSION_EMULATE (*CONFIG_BOOTLOADER_EFUSE_SECURE_VERSION_EMULATE*)
- CONFIG_ENABLE_STATIC_TASK_CLEAN_UP_HOOK (*CONFIG_FREERTOS_ENABLE_STATIC_TASK_CLEAN_UP*)
- CONFIG_ESP32_APPTRACE_ONPANIC_HOST_FLUSH_TMO (*CONFIG_APPTRACE_ONPANIC_HOST_FLUSH_TMO*)
- CONFIG_ESP32_APPTRACE_PENDING_DATA_SIZE_MAX (*CONFIG_APPTRACE_PENDING_DATA_SIZE_MAX*)
- CONFIG_ESP32_APPTRACE_POSTMORTEM_FLUSH_TRAX_THRESH (*CONFIG_APPTRACE_POSTMORTEM_FLUSH_THRESH*)
- **CONFIG_ESP32_CORE_DUMP_DECODE** (*CONFIG_ESP_COREDUMP_DECODE*)
  - CONFIG_ESP32_CORE_DUMP_DECODE_INFO
  - CONFIG_ESP32_CORE_DUMP_DECODE_DISABLE
- CONFIG_ESP32_CORE_DUMP_MAX_TASKS_NUM (*CONFIG_ESP_COREDUMP_MAX_TASKS_NUM*)
- CONFIG_ESP32_CORE_DUMP_STACK_SIZE (*CONFIG_ESP_COREDUMP_STACK_SIZE*)
- CONFIG_ESP32_CORE_DUMP_UART_DELAY (*CONFIG_ESP_COREDUMP_UART_DELAY*)
- CONFIG_ESP32_DEBUG_STUBS_ENABLE (*CONFIG_ESP_DEBUG_STUBS_ENABLE*)
- CONFIG_ESP32_GCOV_ENABLE (*CONFIG_APPTRACE_GCOV_ENABLE*)
- CONFIG_ESP32_PHY_CALIBRATION_AND_DATA_STORAGE (*CONFIG_ESP_PHY_CALIBRATION_AND_DATA_STORAGE*)
- CONFIG_ESP32_PHY_DEFAULT_INIT_IF_INVALID (*CONFIG_ESP_PHY_DEFAULT_INIT_IF_INVALID*)
- CONFIG_ESP32_PHY_INIT_DATA_ERROR (*CONFIG_ESP_PHY_INIT_DATA_ERROR*)
- CONFIG_ESP32_PHY_INIT_DATA_IN_PARTITION (*CONFIG_ESP_PHY_INIT_DATA_IN_PARTITION*)

- CONFIG_ESP32_PHY_MAC_BB_PD (*CONFIG_ESP_PHY_MAC_BB_PD*)
- CONFIG_ESP32_PHY_MAX_WIFI_TX_POWER (*CONFIG_ESP_PHY_MAX_WIFI_TX_POWER*)
- CONFIG_ESP32_PTHREAD_STACK_MIN (*CONFIG_PTHREAD_STACK_MIN*)
- **CONFIG_ESP32_PTHREAD_TASK_CORE_DEFAULT (*CONFIG_PTHREAD_TASK_CORE_DEFAULT*)**
  - CONFIG_ESP32_DEFAULT_PTHREAD_CORE_NO_AFFINITY
  - CONFIG_ESP32_DEFAULT_PTHREAD_CORE_0
  - CONFIG_ESP32_DEFAULT_PTHREAD_CORE_1
- CONFIG_ESP32_PTHREAD_TASK_NAME_DEFAULT (*CONFIG_PTHREAD_TASK_NAME_DEFAULT*)
- CONFIG_ESP32_PTHREAD_TASK_PRIO_DEFAULT (*CONFIG_PTHREAD_TASK_PRIO_DEFAULT*)
- CONFIG_ESP32_PTHREAD_TASK_STACK_SIZE_DEFAULT (*CONFIG_PTHREAD_TASK_STACK_SIZE_DEFAULT*)
- CONFIG_ESP32_REDUCE_PHY_TX_POWER (*CONFIG_ESP_PHY_REDUCE_TX_POWER*)
- CONFIG_ESP32_RTC_XTAL_BOOTSTRAP_CYCLES (*CONFIG_ESP_SYSTEM_RTC_EXT_XTAL_BOOTSTRAP_CYCLES*)
- CONFIG_ESP32_SUPPORT_MULTIPLE_PHY_INIT_DATA_BIN (*CONFIG_ESP_PHY_MULTIPLE_INIT_DATA_BIN*)
- CONFIG_ESP32_WIFI_AMPDU_RX_ENABLED (*CONFIG_ESP_WIFI_AMPDU_RX_ENABLED*)
- CONFIG_ESP32_WIFI_AMPDU_TX_ENABLED (*CONFIG_ESP_WIFI_AMPDU_TX_ENABLED*)
- CONFIG_ESP32_WIFI_AMSDU_TX_ENABLED (*CONFIG_ESP_WIFI_AMSDU_TX_ENABLED*)
- CONFIG_ESP32_WIFI_CACHE_TX_BUFFER_NUM (*CONFIG_ESP_WIFI_CACHE_TX_BUFFER_NUM*)
- CONFIG_ESP32_WIFI_CSI_ENABLED (*CONFIG_ESP_WIFI_CSI_ENABLED*)
- CONFIG_ESP32_WIFI_DYNAMIC_RX_BUFFER_NUM (*CONFIG_ESP_WIFI_DYNAMIC_RX_BUFFER_NUM*)
- CONFIG_ESP32_WIFI_DYNAMIC_TX_BUFFER_NUM (*CONFIG_ESP_WIFI_DYNAMIC_TX_BUFFER_NUM*)
- CONFIG_ESP32_WIFI_ENABLE_WPA3_OWE_STA (*CONFIG_ESP_WIFI_ENABLE_WPA3_OWE_STA*)
- CONFIG_ESP32_WIFI_ENABLE_WPA3_SAE (*CONFIG_ESP_WIFI_ENABLE_WPA3_SAE*)
- CONFIG_ESP32_WIFI_IRAM_OPT (*CONFIG_ESP_WIFI_IRAM_OPT*)
- CONFIG_ESP32_WIFI_MGMT_SBUF_NUM (*CONFIG_ESP_WIFI_MGMT_SBUF_NUM*)
- CONFIG_ESP32_WIFI_NVS_ENABLED (*CONFIG_ESP_WIFI_NVS_ENABLED*)
- CONFIG_ESP32_WIFI_RX_BA_WIN (*CONFIG_ESP_WIFI_RX_BA_WIN*)
- CONFIG_ESP32_WIFI_RX_IRAM_OPT (*CONFIG_ESP_WIFI_RX_IRAM_OPT*)
- CONFIG_ESP32_WIFI_SOFTAP_BEACON_MAX_LEN (*CONFIG_ESP_WIFI_SOFTAP_BEACON_MAX_LEN*)
- CONFIG_ESP32_WIFI_STATIC_RX_BUFFER_NUM (*CONFIG_ESP_WIFI_STATIC_RX_BUFFER_NUM*)
- CONFIG_ESP32_WIFI_STATIC_TX_BUFFER_NUM (*CONFIG_ESP_WIFI_STATIC_TX_BUFFER_NUM*)
- CONFIG_ESP32_WIFI_SW_COEXIST_ENABLE (*CONFIG_ESP_COEX_SW_COEXIST_ENABLE*)
- **CONFIG_ESP32_WIFI_TASK_CORE_ID (*CONFIG_ESP_WIFI_TASK_CORE_ID*)**
  - CONFIG_ESP32_WIFI_TASK_PINNED_TO_CORE_0
  - CONFIG_ESP32_WIFI_TASK_PINNED_TO_CORE_1
- CONFIG_ESP32_WIFI_TX_BA_WIN (*CONFIG_ESP_WIFI_TX_BA_WIN*)
- **CONFIG_ESP32_WIFI_TX_BUFFER (*CONFIG_ESP_WIFI_TX_BUFFER*)**
  - CONFIG_ESP32_WIFI_STATIC_TX_BUFFER
  - CONFIG_ESP32_WIFI_DYNAMIC_TX_BUFFER
- CONFIG_ESP_GRATUITOUS_ARP (*CONFIG_LWIP_ESP_GRATUITOUS_ARP*)
- CONFIG_ESP_SYSTEM_PD_FLASH (*CONFIG_ESP_SLEEP_POWER_DOWN_FLASH*)
- CONFIG_ESP_SYSTEM_PM_POWER_DOWN_CPU (*CONFIG_PM_POWER_DOWN_CPU_IN_LIGHT_SLEEP*)
- CONFIG_ESP_TASK_WDT (*CONFIG_ESP_TASK_WDT_INIT*)
- CONFIG_ESP_WIFI_EXTERNAL_COEXIST_ENABLE (*CONFIG_ESP_COEX_EXTERNAL_COEXIST_ENABLE*)
- CONFIG_ESP_WIFI_SW_COEXIST_ENABLE (*CONFIG_ESP_COEX_SW_COEXIST_ENABLE*)
- CONFIG_EVENT_LOOP_PROFILING (*CONFIG_ESP_EVENT_LOOP_PROFILING*)
- CONFIG_EXTERNAL_COEX_ENABLE (*CONFIG_ESP_COEX_EXTERNAL_COEXIST_ENABLE*)
- CONFIG_FLASH_ENCRYPTION_ENABLED (*CONFIG_SECURE_FLASH_ENC_ENABLED*)
- CONFIG_FLASH_ENCRYPTION_UART_BOOTLOADER_ALLOW_CACHE (*CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_CACHE*)
- CONFIG_FLASH_ENCRYPTION_UART_BOOTLOADER_ALLOW_ENCRYPT (*CONFIG_SECURE_FLASH_UART_BOOTLOADER_ALLOW_ENC*)
- **CONFIG_GAP_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_GAP_TRACE_LEVEL*)**
  - CONFIG_GAP_TRACE_LEVEL_NONE
  - CONFIG_GAP_TRACE_LEVEL_ERROR
  - CONFIG_GAP_TRACE_LEVEL_WARNING
  - CONFIG_GAP_TRACE_LEVEL_API

    - CONFIG_GAP_TRACE_LEVEL_EVENT
    - CONFIG_GAP_TRACE_LEVEL_DEBUG
    - CONFIG_GAP_TRACE_LEVEL_VERBOSE
- CONFIG_GARP_TMR_INTERVAL (*CONFIG_LWIP_GARP_TMR_INTERVAL*)
- CONFIG_GATTC_CACHE_NVS_FLASH (*CONFIG_BT_GATTC_CACHE_NVS_FLASH*)
- CONFIG_GATTC_ENABLE (*CONFIG_BT_GATTC_ENABLE*)
- CONFIG_GATTS_ENABLE (*CONFIG_BT_GATTS_ENABLE*)
- **CONFIG_GATTS_SEND_SERVICE_CHANGE_MODE (*CONFIG_BT_GATTS_SEND_SERVICE_CHANGE_MODE*)**
    - CONFIG_GATTS_SEND_SERVICE_CHANGE_MANUAL
    - CONFIG_GATTS_SEND_SERVICE_CHANGE_AUTO
- **CONFIG_GATT_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_GATT_TRACE_LEVEL*)**
    - CONFIG_GATT_TRACE_LEVEL_NONE
    - CONFIG_GATT_TRACE_LEVEL_ERROR
    - CONFIG_GATT_TRACE_LEVEL_WARNING
    - CONFIG_GATT_TRACE_LEVEL_API
    - CONFIG_GATT_TRACE_LEVEL_EVENT
    - CONFIG_GATT_TRACE_LEVEL_DEBUG
    - CONFIG_GATT_TRACE_LEVEL_VERBOSE
- CONFIG_GDBSTUB_MAX_TASKS (*CONFIG_ESP_GDBSTUB_MAX_TASKS*)
- CONFIG_GDBSTUB_SUPPORT_TASKS (*CONFIG_ESP_GDBSTUB_SUPPORT_TASKS*)
- **CONFIG_HCI_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_HCI_TRACE_LEVEL*)**
    - CONFIG_HCI_TRACE_LEVEL_NONE
    - CONFIG_HCI_TRACE_LEVEL_ERROR
    - CONFIG_HCI_TRACE_LEVEL_WARNING
    - CONFIG_HCI_TRACE_LEVEL_API
    - CONFIG_HCI_TRACE_LEVEL_EVENT
    - CONFIG_HCI_TRACE_LEVEL_DEBUG
    - CONFIG_HCI_TRACE_LEVEL_VERBOSE
- CONFIG_HFP_AG_ENABLE (*CONFIG_BT_HFP_AG_ENABLE*)
- **CONFIG_HFP_AUDIO_DATA_PATH (*CONFIG_BT_HFP_AUDIO_DATA_PATH*)**
    - CONFIG_HFP_AUDIO_DATA_PATH_PCM
    - CONFIG_HFP_AUDIO_DATA_PATH_HCI
- CONFIG_HFP_CLIENT_ENABLE (*CONFIG_BT_HFP_CLIENT_ENABLE*)
- CONFIG_HFP_ENABLE (*CONFIG_BT_HFP_ENABLE*)
- **CONFIG_HID_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_HID_TRACE_LEVEL*)**
    - CONFIG_HID_TRACE_LEVEL_NONE
    - CONFIG_HID_TRACE_LEVEL_ERROR
    - CONFIG_HID_TRACE_LEVEL_WARNING
    - CONFIG_HID_TRACE_LEVEL_API
    - CONFIG_HID_TRACE_LEVEL_EVENT
    - CONFIG_HID_TRACE_LEVEL_DEBUG
    - CONFIG_HID_TRACE_LEVEL_VERBOSE
- CONFIG_INT_WDT (*CONFIG_ESP_INT_WDT*)
- CONFIG_INT_WDT_CHECK_CPU1 (*CONFIG_ESP_INT_WDT_CHECK_CPU1*)
- CONFIG_INT_WDT_TIMEOUT_MS (*CONFIG_ESP_INT_WDT_TIMEOUT_MS*)
- CONFIG_IPC_TASK_STACK_SIZE (*CONFIG_ESP_IPC_TASK_STACK_SIZE*)
- **CONFIG_L2CAP_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_L2CAP_TRACE_LEVEL*)**
    - CONFIG_L2CAP_TRACE_LEVEL_NONE
    - CONFIG_L2CAP_TRACE_LEVEL_ERROR
    - CONFIG_L2CAP_TRACE_LEVEL_WARNING
    - CONFIG_L2CAP_TRACE_LEVEL_API
    - CONFIG_L2CAP_TRACE_LEVEL_EVENT
    - CONFIG_L2CAP_TRACE_LEVEL_DEBUG
    - CONFIG_L2CAP_TRACE_LEVEL_VERBOSE
- CONFIG_L2_TO_L3_COPY (*CONFIG_LWIP_L2_TO_L3_COPY*)
- **CONFIG_LOG_BOOTLOADER_LEVEL (*CONFIG_BOOTLOADER_LOG_LEVEL*)**
    - CONFIG_LOG_BOOTLOADER_LEVEL_NONE

    - CONFIG_LOG_BOOTLOADER_LEVEL_ERROR
    - CONFIG_LOG_BOOTLOADER_LEVEL_WARN
    - CONFIG_LOG_BOOTLOADER_LEVEL_INFO
    - CONFIG_LOG_BOOTLOADER_LEVEL_DEBUG
    - CONFIG_LOG_BOOTLOADER_LEVEL_VERBOSE
- CONFIG_MAC_BB_PD (*CONFIG_ESP_PHY_MAC_BB_PD*)
- CONFIG_MAIN_TASK_STACK_SIZE (*CONFIG_ESP_MAIN_TASK_STACK_SIZE*)
- **CONFIG_MCA_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_MCA_TRACE_LEVEL*)**
    - CONFIG_MCA_TRACE_LEVEL_NONE
    - CONFIG_MCA_TRACE_LEVEL_ERROR
    - CONFIG_MCA_TRACE_LEVEL_WARNING
    - CONFIG_MCA_TRACE_LEVEL_API
    - CONFIG_MCA_TRACE_LEVEL_EVENT
    - CONFIG_MCA_TRACE_LEVEL_DEBUG
    - CONFIG_MCA_TRACE_LEVEL_VERBOSE
- CONFIG_MCPWM_ISR_IN_IRAM (*CONFIG_MCPWM_ISR_IRAM_SAFE*)
- CONFIG_NIMBLE_ATT_PREFERRED_MTU (*CONFIG_BT_NIMBLE_ATT_PREFERRED_MTU*)
- CONFIG_NIMBLE_CRYPTO_STACK_MBEDTLS (*CONFIG_BT_NIMBLE_CRYPTO_STACK_MBEDTLS*)
- CONFIG_NIMBLE_DEBUG (*CONFIG_BT_NIMBLE_DEBUG*)
- CONFIG_NIMBLE_GAP_DEVICE_NAME_MAX_LEN (*CONFIG_BT_NIMBLE_GAP_DEVICE_NAME_MAX_LEN*)
- CONFIG_NIMBLE_HS_FLOW_CTRL (*CONFIG_BT_NIMBLE_HS_FLOW_CTRL*)
- CONFIG_NIMBLE_HS_FLOW_CTRL_ITVL (*CONFIG_BT_NIMBLE_HS_FLOW_CTRL_ITVL*)
- CONFIG_NIMBLE_HS_FLOW_CTRL_THRESH (*CONFIG_BT_NIMBLE_HS_FLOW_CTRL_THRESH*)
- CONFIG_NIMBLE_HS_FLOW_CTRL_TX_ON_DISCONNECT (*CONFIG_BT_NIMBLE_HS_FLOW_CTRL_TX_ON_DISCONNECT*)
- CONFIG_NIMBLE_L2CAP_COC_MAX_NUM (*CONFIG_BT_NIMBLE_L2CAP_COC_MAX_NUM*)
- CONFIG_NIMBLE_MAX_BONDS (*CONFIG_BT_NIMBLE_MAX_BONDS*)
- CONFIG_NIMBLE_MAX_CCCDS (*CONFIG_BT_NIMBLE_MAX_CCCDS*)
- CONFIG_NIMBLE_MAX_CONNECTIONS (*CONFIG_BT_NIMBLE_MAX_CONNECTIONS*)
- **CONFIG_NIMBLE_MEM_ALLOC_MODE (*CONFIG_BT_NIMBLE_MEM_ALLOC_MODE*)**
    - CONFIG_NIMBLE_MEM_ALLOC_MODE_INTERNAL
    - CONFIG_NIMBLE_MEM_ALLOC_MODE_EXTERNAL
    - CONFIG_NIMBLE_MEM_ALLOC_MODE_DEFAULT
- CONFIG_NIMBLE_MESH (*CONFIG_BT_NIMBLE_MESH*)
- CONFIG_NIMBLE_MESH_DEVICE_NAME (*CONFIG_BT_NIMBLE_MESH_DEVICE_NAME*)
- CONFIG_NIMBLE_MESH_FRIEND (*CONFIG_BT_NIMBLE_MESH_FRIEND*)
- CONFIG_NIMBLE_MESH_GATT_PROXY (*CONFIG_BT_NIMBLE_MESH_GATT_PROXY*)
- CONFIG_NIMBLE_MESH_LOW_POWER (*CONFIG_BT_NIMBLE_MESH_LOW_POWER*)
- CONFIG_NIMBLE_MESH_PB_ADV (*CONFIG_BT_NIMBLE_MESH_PB_ADV*)
- CONFIG_NIMBLE_MESH_PB_GATT (*CONFIG_BT_NIMBLE_MESH_PB_GATT*)
- CONFIG_NIMBLE_MESH_PROV (*CONFIG_BT_NIMBLE_MESH_PROV*)
- CONFIG_NIMBLE_MESH_PROXY (*CONFIG_BT_NIMBLE_MESH_PROXY*)
- CONFIG_NIMBLE_MESH_RELAY (*CONFIG_BT_NIMBLE_MESH_RELAY*)
- CONFIG_NIMBLE_NVS_PERSIST (*CONFIG_BT_NIMBLE_NVS_PERSIST*)
- **CONFIG_NIMBLE_PINNED_TO_CORE_CHOICE (*CONFIG_BT_NIMBLE_PINNED_TO_CORE_CHOICE*)**
    - CONFIG_NIMBLE_PINNED_TO_CORE_0
    - CONFIG_NIMBLE_PINNED_TO_CORE_1
- CONFIG_NIMBLE_ROLE_BROADCASTER (*CONFIG_BT_NIMBLE_ROLE_BROADCASTER*)
- CONFIG_NIMBLE_ROLE_CENTRAL (*CONFIG_BT_NIMBLE_ROLE_CENTRAL*)
- CONFIG_NIMBLE_ROLE_OBSERVER (*CONFIG_BT_NIMBLE_ROLE_OBSERVER*)
- CONFIG_NIMBLE_ROLE_PERIPHERAL (*CONFIG_BT_NIMBLE_ROLE_PERIPHERAL*)
- CONFIG_NIMBLE_RPA_TIMEOUT (*CONFIG_BT_NIMBLE_RPA_TIMEOUT*)
- CONFIG_NIMBLE_SM_LEGACY (*CONFIG_BT_NIMBLE_SM_LEGACY*)
- CONFIG_NIMBLE_SM_SC (*CONFIG_BT_NIMBLE_SM_SC*)
- CONFIG_NIMBLE_SM_SC_DEBUG_KEYS (*CONFIG_BT_NIMBLE_SM_SC_DEBUG_KEYS*)
- CONFIG_NIMBLE_SVC_GAP_APPEARANCE (*CONFIG_BT_NIMBLE_SVC_GAP_APPEARANCE*)
- CONFIG_NIMBLE_SVC_GAP_DEVICE_NAME (*CONFIG_BT_NIMBLE_SVC_GAP_DEVICE_NAME*)

- CONFIG_NIMBLE_TASK_STACK_SIZE (*CONFIG_BT_NIMBLE_HOST_TASK_STACK_SIZE*)
- CONFIG_NO_BLOBS (*CONFIG_APP_NO_BLOBS*)
- **CONFIG_OPTIMIZATION_ASSERTION_LEVEL (*CONFIG_COMPILER_OPTIMIZATION_ASSERTION_LEVEL*)**
  - CONFIG_OPTIMIZATION_ASSERTIONS_ENABLED
  - CONFIG_OPTIMIZATION_ASSERTIONS_SILENT
  - CONFIG_OPTIMIZATION_ASSERTIONS_DISABLED
- **CONFIG_OPTIMIZATION_COMPILER (*CONFIG_COMPILER_OPTIMIZATION*)**
  - CONFIG_OPTIMIZATION_LEVEL_DEBUG, CONFIG_COMPILER_OPTIMIZATION_LEVEL_DEBUG, CONFIG_COMPILER_OPTIMIZATION_DEFAULT
  - CONFIG_OPTIMIZATION_LEVEL_RELEASE, CONFIG_COMPILER_OPTIMIZATION_LEVEL_RELEASE
- **CONFIG_OSI_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_OSI_TRACE_LEVEL*)**
  - CONFIG_OSI_TRACE_LEVEL_NONE
  - CONFIG_OSI_TRACE_LEVEL_ERROR
  - CONFIG_OSI_TRACE_LEVEL_WARNING
  - CONFIG_OSI_TRACE_LEVEL_API
  - CONFIG_OSI_TRACE_LEVEL_EVENT
  - CONFIG_OSI_TRACE_LEVEL_DEBUG
  - CONFIG_OSI_TRACE_LEVEL_VERBOSE
- CONFIG_OTA_ALLOW_HTTP (*CONFIG_ESP_HTTPS_OTA_ALLOW_HTTP*)
- **CONFIG_PAN_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_PAN_TRACE_LEVEL*)**
  - CONFIG_PAN_TRACE_LEVEL_NONE
  - CONFIG_PAN_TRACE_LEVEL_ERROR
  - CONFIG_PAN_TRACE_LEVEL_WARNING
  - CONFIG_PAN_TRACE_LEVEL_API
  - CONFIG_PAN_TRACE_LEVEL_EVENT
  - CONFIG_PAN_TRACE_LEVEL_DEBUG
  - CONFIG_PAN_TRACE_LEVEL_VERBOSE
- CONFIG_PM_POWER_DOWN_TAGMEM_IN_LIGHT_SLEEP (*CONFIG_PM_RESTORE_CACHE_TAGMEM_AFTER_LIGHT_SLEEP*)
- CONFIG_POST_EVENTS_FROM_IRAM_ISR (*CONFIG_ESP_EVENT_POST_FROM_IRAM_ISR*)
- CONFIG_POST_EVENTS_FROM_ISR (*CONFIG_ESP_EVENT_POST_FROM_ISR*)
- CONFIG_PPP_CHAP_SUPPORT (*CONFIG_LWIP_PPP_CHAP_SUPPORT*)
- CONFIG_PPP_DEBUG_ON (*CONFIG_LWIP_PPP_DEBUG_ON*)
- CONFIG_PPP_MPPE_SUPPORT (*CONFIG_LWIP_PPP_MPPE_SUPPORT*)
- CONFIG_PPP_MSCHAP_SUPPORT (*CONFIG_LWIP_PPP_MSCHAP_SUPPORT*)
- CONFIG_PPP_NOTIFY_PHASE_SUPPORT (*CONFIG_LWIP_PPP_NOTIFY_PHASE_SUPPORT*)
- CONFIG_PPP_PAP_SUPPORT (*CONFIG_LWIP_PPP_PAP_SUPPORT*)
- CONFIG_PPP_SUPPORT (*CONFIG_LWIP_PPP_SUPPORT*)
- CONFIG_REDUCE_PHY_TX_POWER (*CONFIG_ESP_PHY_REDUCE_TX_POWER*)
- **CONFIG_RFCOMM_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_RFCOMM_TRACE_LEVEL*)**
  - CONFIG_RFCOMM_TRACE_LEVEL_NONE
  - CONFIG_RFCOMM_TRACE_LEVEL_ERROR
  - CONFIG_RFCOMM_TRACE_LEVEL_WARNING
  - CONFIG_RFCOMM_TRACE_LEVEL_API
  - CONFIG_RFCOMM_TRACE_LEVEL_EVENT
  - CONFIG_RFCOMM_TRACE_LEVEL_DEBUG
  - CONFIG_RFCOMM_TRACE_LEVEL_VERBOSE
- CONFIG_SEMIHOSTFS_MAX_MOUNT_POINTS (*CONFIG_VFS_SEMIHOSTFS_MAX_MOUNT_POINTS*)
- **CONFIG_SMP_INITIAL_TRACE_LEVEL (*CONFIG_BT_LOG_SMP_TRACE_LEVEL*)**
  - CONFIG_SMP_TRACE_LEVEL_NONE
  - CONFIG_SMP_TRACE_LEVEL_ERROR
  - CONFIG_SMP_TRACE_LEVEL_WARNING
  - CONFIG_SMP_TRACE_LEVEL_API
  - CONFIG_SMP_TRACE_LEVEL_EVENT
  - CONFIG_SMP_TRACE_LEVEL_DEBUG
  - CONFIG_SMP_TRACE_LEVEL_VERBOSE
- CONFIG_SMP_SLAVE_CON_PARAMS_UPD_ENABLE (*CONFIG_BT_SMP_SLAVE_CON_PARAMS_UPD_ENABLE*)

- CONFIG_SPI_FLASH_32BIT_ADDR_ENABLE (*CONFIG_BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH*)
- CONFIG_SPI_FLASH_QUAD_32BIT_ADDR_ENABLE (*CONFIG_BOOTLOADER_CACHE_32BIT_ADDR_QUAD_FLASH*)
- **CONFIG_SPI_FLASH_WRITING_DANGEROUS_REGIONS (*CONFIG_SPI_FLASH_DANGEROUS_WRITE*)**
  - CONFIG_SPI_FLASH_WRITING_DANGEROUS_REGIONS_ABORTS
  - CONFIG_SPI_FLASH_WRITING_DANGEROUS_REGIONS_FAILS
  - CONFIG_SPI_FLASH_WRITING_DANGEROUS_REGIONS_ALLOWED
- **CONFIG_STACK_CHECK_MODE (*CONFIG_COMPILER_STACK_CHECK_MODE*)**
  - CONFIG_STACK_CHECK_NONE
  - CONFIG_STACK_CHECK_NORM
  - CONFIG_STACK_CHECK_STRONG
  - CONFIG_STACK_CHECK_ALL
- CONFIG_SUPPORT_TERMIOS (*CONFIG_VFS_SUPPORT_TERMIOS*)
- CONFIG_SUPPRESS_SELECT_DEBUG_OUTPUT (*CONFIG_VFS_SUPPRESS_SELECT_DEBUG_OUTPUT*)
- CONFIG_SW_COEXIST_ENABLE (*CONFIG_ESP_COEX_SW_COEXIST_ENABLE*)
- CONFIG_SYSTEM_EVENT_QUEUE_SIZE (*CONFIG_ESP_SYSTEM_EVENT_QUEUE_SIZE*)
- CONFIG_SYSTEM_EVENT_TASK_STACK_SIZE (*CONFIG_ESP_SYSTEM_EVENT_TASK_STACK_SIZE*)
- CONFIG_SYSVIEW_BUF_WAIT_TMO (*CONFIG_APPTRACE_SV_BUF_WAIT_TMO*)
- CONFIG_SYSVIEW_ENABLE (*CONFIG_APPTRACE_SV_ENABLE*)
- CONFIG_SYSVIEW_EVT_IDLE_ENABLE (*CONFIG_APPTRACE_SV_EVT_IDLE_ENABLE*)
- CONFIG_SYSVIEW_EVT_ISR_ENTER_ENABLE (*CONFIG_APPTRACE_SV_EVT_ISR_ENTER_ENABLE*)
- CONFIG_SYSVIEW_EVT_ISR_EXIT_ENABLE (*CONFIG_APPTRACE_SV_EVT_ISR_EXIT_ENABLE*)
- CONFIG_SYSVIEW_EVT_ISR_TO_SCHEDULER_ENABLE (*CONFIG_APPTRACE_SV_EVT_ISR_TO_SCHED_ENABLE*)
- CONFIG_SYSVIEW_EVT_OVERFLOW_ENABLE (*CONFIG_APPTRACE_SV_EVT_OVERFLOW_ENABLE*)
- CONFIG_SYSVIEW_EVT_TASK_CREATE_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_CREATE_ENABLE*)
- CONFIG_SYSVIEW_EVT_TASK_START_EXEC_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_START_EXEC_ENABL*
- CONFIG_SYSVIEW_EVT_TASK_START_READY_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_START_READY_EN*
- CONFIG_SYSVIEW_EVT_TASK_STOP_EXEC_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_STOP_EXEC_ENABLE*)
- CONFIG_SYSVIEW_EVT_TASK_STOP_READY_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_STOP_READY_ENAB*
- CONFIG_SYSVIEW_EVT_TASK_TERMINATE_ENABLE (*CONFIG_APPTRACE_SV_EVT_TASK_TERMINATE_ENABLE*)
- CONFIG_SYSVIEW_EVT_TIMER_ENTER_ENABLE (*CONFIG_APPTRACE_SV_EVT_TIMER_ENTER_ENABLE*)
- CONFIG_SYSVIEW_EVT_TIMER_EXIT_ENABLE (*CONFIG_APPTRACE_SV_EVT_TIMER_EXIT_ENABLE*)
- CONFIG_SYSVIEW_MAX_TASKS (*CONFIG_APPTRACE_SV_MAX_TASKS*)
- **CONFIG_SYSVIEW_TS_SOURCE (*CONFIG_APPTRACE_SV_TS_SOURCE*)**
  - CONFIG_SYSVIEW_TS_SOURCE_CCOUNT
  - CONFIG_SYSVIEW_TS_SOURCE_ESP_TIMER
- CONFIG_TASK_WDT (*CONFIG_ESP_TASK_WDT_INIT*)
- CONFIG_TASK_WDT_CHECK_IDLE_TASK_CPU0 (*CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU0*)
- CONFIG_TASK_WDT_CHECK_IDLE_TASK_CPU1 (*CONFIG_ESP_TASK_WDT_CHECK_IDLE_TASK_CPU1*)
- CONFIG_TASK_WDT_PANIC (*CONFIG_ESP_TASK_WDT_PANIC*)
- CONFIG_TASK_WDT_TIMEOUT_S (*CONFIG_ESP_TASK_WDT_TIMEOUT_S*)
- CONFIG_TCPIP_RECVMBOX_SIZE (*CONFIG_LWIP_TCPIP_RECVMBOX_SIZE*)
- **CONFIG_TCPIP_TASK_AFFINITY (*CONFIG_LWIP_TCPIP_TASK_AFFINITY*)**
  - CONFIG_TCPIP_TASK_AFFINITY_NO_AFFINITY
  - CONFIG_TCPIP_TASK_AFFINITY_CPU0
  - CONFIG_TCPIP_TASK_AFFINITY_CPU1
- CONFIG_TCPIP_TASK_STACK_SIZE (*CONFIG_LWIP_TCPIP_TASK_STACK_SIZE*)
- CONFIG_TCP_MAXRTX (*CONFIG_LWIP_TCP_MAXRTX*)
- CONFIG_TCP_MSL (*CONFIG_LWIP_TCP_MSL*)
- CONFIG_TCP_MSS (*CONFIG_LWIP_TCP_MSS*)
- **CONFIG_TCP_OVERSIZE (*CONFIG_LWIP_TCP_OVERSIZE*)**
  - CONFIG_TCP_OVERSIZE_MSS
  - CONFIG_TCP_OVERSIZE_QUARTER_MSS
  - CONFIG_TCP_OVERSIZE_DISABLE
- CONFIG_TCP_QUEUE_OOSEQ (*CONFIG_LWIP_TCP_QUEUE_OOSEQ*)
- CONFIG_TCP_RECVMBOX_SIZE (*CONFIG_LWIP_TCP_RECVMBOX_SIZE*)
- CONFIG_TCP_SND_BUF_DEFAULT (*CONFIG_LWIP_TCP_SND_BUF_DEFAULT*)
- CONFIG_TCP_SYNMAXRTX (*CONFIG_LWIP_TCP_SYNMAXRTX*)

- CONFIG_TCP_WND_DEFAULT (*CONFIG_LWIP_TCP_WND_DEFAULT*)
- CONFIG_TIMER_QUEUE_LENGTH (*CONFIG_FREERTOS_TIMER_QUEUE_LENGTH*)
- CONFIG_TIMER_TASK_PRIORITY (*CONFIG_FREERTOS_TIMER_TASK_PRIORITY*)
- CONFIG_TIMER_TASK_STACK_DEPTH (*CONFIG_FREERTOS_TIMER_TASK_STACK_DEPTH*)
- CONFIG_TIMER_TASK_STACK_SIZE (*CONFIG_ESP_TIMER_TASK_STACK_SIZE*)
- CONFIG_UDP_RECVMBOX_SIZE (*CONFIG_LWIP_UDP_RECVMBOX_SIZE*)
- CONFIG_WARN_WRITE_STRINGS (*CONFIG_COMPILER_WARN_WRITE_STRINGS*)
- CONFIG_WPA_11KV_SUPPORT (*CONFIG_ESP_WIFI_11KV_SUPPORT*)
- CONFIG_WPA_11R_SUPPORT (*CONFIG_ESP_WIFI_11R_SUPPORT*)
- CONFIG_WPA_DEBUG_PRINT (*CONFIG_ESP_WIFI_DEBUG_PRINT*)
- CONFIG_WPA_DPP_SUPPORT (*CONFIG_ESP_WIFI_DPP_SUPPORT*)
- CONFIG_WPA_MBEDTLS_CRYPTO (*CONFIG_ESP_WIFI_MBEDTLS_CRYPTO*)
- CONFIG_WPA_MBEDTLS_TLS_CLIENT (*CONFIG_ESP_WIFI_MBEDTLS_TLS_CLIENT*)
- CONFIG_WPA_MBO_SUPPORT (*CONFIG_ESP_WIFI_MBO_SUPPORT*)
- CONFIG_WPA_SCAN_CACHE (*CONFIG_ESP_WIFI_SCAN_CACHE*)
- CONFIG_WPA_SUITE_B_192 (*CONFIG_ESP_WIFI_SUITE_B_192*)
- CONFIG_WPA_TESTING_OPTIONS (*CONFIG_ESP_WIFI_TESTING_OPTIONS*)
- CONFIG_WPA_WAPI_PSK (*CONFIG_ESP_WIFI_WAPI_PSK*)
- CONFIG_WPA_WPS_SOFTAP_REGISTRAR (*CONFIG_ESP_WIFI_WPS_SOFTAP_REGISTRAR*)
- CONFIG_WPA_WPS_STRICT (*CONFIG_ESP_WIFI_WPS_STRICT*)

2.8 Provisioning API

2.8.1 Protocol Communication

Overview

The Protocol Communication (protocomm) component manages secure sessions and provides the framework for multiple transports. The application can also use the protocomm layer directly to have application-specific extensions for the provisioning or non-provisioning use cases.

Following features are available for provisioning:

- Communication security at the application level
  - `protocomm_security0` (no security)
  - `protocomm_security1` (Curve25519 key exchange + AES-CTR encryption/decryption)
  - `protocomm_security2` (SRP6a-based key exchange + AES-GCM encryption/decryption)
- Proof-of-possession (support with protocomm_security1 only)
- Salt and Verifier (support with protocomm_security2 only)

Protocomm internally uses protobuf (protocol buffers) for secure session establishment. Users can choose to implement their own security (even without using protobuf). Protocomm can also be used without any security layer.

Protocomm provides the framework for various transports:

- Bluetooth LE
- Wi-Fi (SoftAP + HTTPD)
- Console, in which case the handler invocation is automatically taken care of on the device side. See Transport Examples below for code snippets.

Note that for protocomm_security1 and protocomm_security2, the client still needs to establish sessions by performing the two-way handshake.

See *Unified Provisioning* for more details about the secure handshake logic.

Enabling Protocomm Security Version

The protocomm component provides a project configuration menu to enable/disable support of respective security versions. The respective configuration options are as follows:

- Support `protocomm_security0`, with no security: *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_0*, this option is enabled by default.
- Support `protocomm_security1` with Curve25519 key exchange + AES-CTR encryption/decryption: *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_1*, this option is enabled by default.
- Support `protocomm_security2` with SRP6a-based key exchange + AES-GCM encryption/decryption: *CONFIG_ESP_PROTOCOMM_SUPPORT_SECURITY_VERSION_2*.

Note: Enabling multiple security versions at once offers the ability to control them dynamically but also increases the firmware size.

SoftAP + HTTP Transport Example with Security 2

For sample usage, see wifi_provisioning/src/scheme_softap.c.

```
/* The endpoint handler to be registered with protocomm. This simply echoes back␣
↪the received data. */
esp_err_t echo_req_handler (uint32_t session_id,
                            const uint8_t *inbuf, ssize_t inlen,
                            uint8_t **outbuf, ssize_t *outlen,
                            void *priv_data)
{
    /* Session ID may be used for persistence. */
    printf("Session ID : %d", session_id);

    /* Echo back the received data. */
    *outlen = inlen;            /* Output the data length updated. */
    *outbuf = malloc(inlen);    /* This is to be deallocated outside. */
    memcpy(*outbuf, inbuf, inlen);

    /* Private data that was passed at the time of endpoint creation. */
    uint32_t *priv = (uint32_t *) priv_data;
    if (priv) {
        printf("Private data : %d", *priv);
    }

    return ESP_OK;
}

static const char sec2_salt[] = {0xf7, 0x5f, 0xe2, 0xbe, 0xba, 0x7c, 0x81, 0xcd};
static const char sec2_verifier[] = {0xbf, 0x86, 0xce, 0x63, 0x8a, 0xbb, 0x7e,␣
↪0x2f, 0x38, 0xa8, 0x19, 0x1b, 0x35,
    0xc9, 0xe3, 0xbe, 0xc3, 0x2b, 0x45, 0xee, 0x10, 0x74, 0x22, 0x1a, 0x95, 0xbe,␣
↪0x62, 0xf7, 0x0c, 0x65, 0x83, 0x50,
    0x08, 0xef, 0xaf, 0xa5, 0x94, 0x4b, 0xcb, 0xe1, 0xce, 0x59, 0x2a, 0xe8, 0x7b,␣
↪0x27, 0xc8, 0x72, 0x26, 0x71, 0xde,
    0xb2, 0xf2, 0x80, 0x02, 0xdd, 0x11, 0xf0, 0x38, 0x0e, 0x95, 0x25, 0x00, 0xcf,␣
↪0xb3, 0x3f, 0xf0, 0x73, 0x2a, 0x25,
    0x03, 0xe8, 0x51, 0x72, 0xef, 0x6d, 0x3e, 0x14, 0xb9, 0x2e, 0x9f, 0x2a, 0x90,␣
↪0x9e, 0x26, 0xb6, 0x3e, 0xc7, 0xe4,
    0x9f, 0xe3, 0x20, 0xce, 0x28, 0x7c, 0xbf, 0x89, 0x50, 0xc9, 0xb6, 0xec, 0xdd,␣
↪0x81, 0x18, 0xf1, 0x1a, 0xd9, 0x7a,
```

(continues on next page)

(continued from previous page)

```
    0x21, 0x99, 0xf1, 0xee, 0x71, 0x2f, 0xcc, 0x93, 0x16, 0x34, 0x0c, 0x79, 0x46,␣
↪0x23, 0xe4, 0x32, 0xec, 0x2d, 0x9e,
    0x18, 0xa6, 0xb9, 0xbb, 0x0a, 0xcf, 0xc4, 0xa8, 0x32, 0xc0, 0x1c, 0x32, 0xa3,␣
↪0x97, 0x66, 0xf8, 0x30, 0xb2, 0xda,
    0xf9, 0x8d, 0xc3, 0x72, 0x72, 0x5f, 0xe5, 0xee, 0xc3, 0x5c, 0x24, 0xc8, 0xdd,␣
↪0x54, 0x49, 0xfc, 0x12, 0x91, 0x81,
    0x9c, 0xc3, 0xac, 0x64, 0x5e, 0xd6, 0x41, 0x88, 0x2f, 0x23, 0x66, 0xc8, 0xac,␣
↪0xb0, 0x35, 0x0b, 0xf6, 0x9c, 0x88,
    0x6f, 0xac, 0xe1, 0xf4, 0xca, 0xc9, 0x07, 0x04, 0x11, 0xda, 0x90, 0x42, 0xa9,␣
↪0xf1, 0x97, 0x3d, 0x94, 0x65, 0xe4,
    0xfb, 0x52, 0x22, 0x3b, 0x7a, 0x7b, 0x9e, 0xe9, 0xee, 0x1c, 0x44, 0xd0, 0x73,␣
↪0x72, 0x2a, 0xca, 0x85, 0x19, 0x4a,
    0x60, 0xce, 0x0a, 0xc8, 0x7d, 0x57, 0xa4, 0xf8, 0x77, 0x22, 0xc1, 0xa5, 0xfa,␣
↪0xfb, 0x7b, 0x91, 0x3b, 0xfe, 0x87,
    0x5f, 0xfe, 0x05, 0xd2, 0xd6, 0xd3, 0x74, 0xe5, 0x2e, 0x68, 0x79, 0x34, 0x70,␣
↪0x40, 0x12, 0xa8, 0xe1, 0xb4, 0x6c,
    0xaa, 0x46, 0x73, 0xcd, 0x8d, 0x17, 0x72, 0x67, 0x32, 0x42, 0xdc, 0x10, 0xd3,␣
↪0x71, 0x7e, 0x8b, 0x00, 0x46, 0x9b,
    0x0a, 0xe9, 0xb4, 0x0f, 0xeb, 0x70, 0x52, 0xdd, 0x0a, 0x1c, 0x7e, 0x2e, 0xb0,␣
↪0x61, 0xa6, 0xe1, 0xa3, 0x34, 0x4b,
    0x2a, 0x3c, 0xc4, 0x5d, 0x42, 0x05, 0x58, 0x25, 0xd3, 0xca, 0x96, 0x5c, 0xb9,␣
↪0x52, 0xf9, 0xe9, 0x80, 0x75, 0x3d,
    0xc8, 0x9f, 0xc7, 0xb2, 0xaa, 0x95, 0x2e, 0x76, 0xb3, 0xe1, 0x48, 0xc1, 0x0a,␣
↪0xa1, 0x0a, 0xe8, 0xaf, 0x41, 0x28,
    0xd2, 0x16, 0xe1, 0xa6, 0xd0, 0x73, 0x51, 0x73, 0x79, 0x98, 0xd9, 0xb9, 0x00,␣
↪0x50, 0xa2, 0x4d, 0x99, 0x18, 0x90,
    0x70, 0x27, 0xe7, 0x8d, 0x56, 0x45, 0x34, 0x1f, 0xb9, 0x30, 0xda, 0xec, 0x4a,␣
↪0x08, 0x27, 0x9f, 0xfa, 0x59, 0x2e,
    0x36, 0x77, 0x00, 0xe2, 0xb6, 0xeb, 0xd1, 0x56, 0x50, 0x8e};

/* The example function for launching a protocomm instance over HTTP. */
protocomm_t *start_pc()
{
    protocomm_t *pc = protocomm_new();


    /* Config for protocomm_httpd_start(). */
    protocomm_httpd_config_t pc_config = {
        .data = {
        .config = PROTOCOMM_HTTPD_DEFAULT_CONFIG()
        }
    };

    /* Start the protocomm server on top of HTTP. */
    protocomm_httpd_start(pc, &pc_config);

    /* Create Security2 params object from salt and verifier. It must be valid␣
↪throughout the scope of protocomm endpoint. This does not need to be static, i.e.
↪, could be dynamically allocated and freed at the time of endpoint removal. */
    const static protocomm_security2_params_t sec2_params = {
        .salt = (const uint8_t *) salt,
        .salt_len = sizeof(salt),
        .verifier = (const uint8_t *) verifier,
        .verifier_len = sizeof(verifier),
    };

    /* Set security for communication at the application level. Just like for␣
↪request handlers, setting security creates an endpoint and registers the handler␣
↪provided by protocomm_security1. One can similarly use protocomm_security0. Only␣
↪one type of security can be set for a protocomm instance at a time. */
    protocomm_set_security(pc, "security_endpoint", &protocomm_security2, &sec2_
↪params);
```

(continues on next page)

(continued from previous page)

```
    /* Private data passed to the endpoint must be valid throughout the scope of␣
↪protocomm endpoint. This need not be static, i.e., could be dynamically␣
↪allocated and freed at the time of endpoint removal. */
    static uint32_t priv_data = 1234;

    /* Add a new endpoint for the protocomm instance, identified by a unique name,␣
↪and register a handler function along with the private data to be passed at the␣
↪time of handler execution. Multiple endpoints can be added as long as they are␣
↪identified by unique names. */
    protocomm_add_endpoint(pc, "echo_req_endpoint",
                           echo_req_handler, (void *) &priv_data);
    return pc;
}

/* The example function for stopping a protocomm instance. */
void stop_pc(protocomm_t *pc)
{
    /* Remove the endpoint identified by its unique name. */
    protocomm_remove_endpoint(pc, "echo_req_endpoint");

    /* Remove the security endpoint identified by its name. */
    protocomm_unset_security(pc, "security_endpoint");

    /* Stop the HTTP server. */
    protocomm_httpd_stop(pc);

    /* Delete, namely deallocate the protocomm instance. */
    protocomm_delete(pc);
}
```

SoftAP + HTTP Transport Example with Security 1

For sample usage, see wifi_provisioning/src/scheme_softap.c.

```
/* The endpoint handler to be registered with protocomm. This simply echoes back␣
↪the received data. */
esp_err_t echo_req_handler (uint32_t session_id,
                            const uint8_t *inbuf, ssize_t inlen,
                            uint8_t **outbuf, ssize_t *outlen,
                            void *priv_data)
{
    /* Session ID may be used for persistence. */
    printf("Session ID : %d", session_id);

    /* Echo back the received data. */
    *outlen = inlen;            /* Output the data length updated. */
    *outbuf = malloc(inlen);    /* This is to be deallocated outside. */
    memcpy(*outbuf, inbuf, inlen);

    /* Private data that was passed at the time of endpoint creation. */
    uint32_t *priv = (uint32_t *) priv_data;
    if (priv) {
        printf("Private data : %d", *priv);
    }

    return ESP_OK;
}

/* The example function for launching a protocomm instance over HTTP. */
```

(continues on next page)

(continued from previous page)

```
protocomm_t *start_pc(const char *pop_string)
{
    protocomm_t *pc = protocomm_new();


    /* Config for protocomm_httpd_start(). */
    protocomm_httpd_config_t pc_config = {
        .data = {
        .config = PROTOCOMM_HTTPD_DEFAULT_CONFIG()
        }
    };

    /* Start the protocomm server on top of HTTP. */
    protocomm_httpd_start(pc, &pc_config);

    /* Create security1 params object from pop_string. It must be valid throughout␣
↪the scope of protocomm endpoint. This need not be static, i.e., could be␣
↪dynamically allocated and freed at the time of endpoint removal. */
    const static protocomm_security1_params_t sec1_params = {
        .data = (const uint8_t *) strdup(pop_string),
        .len = strlen(pop_string)
    };

    /* Set security for communication at the application level. Just like for␣
↪request handlers, setting security creates an endpoint and registers the handler␣
↪provided by protocomm_security1. One can similarly use protocomm_security0. Only␣
↪one type of security can be set for a protocomm instance at a time. */
    protocomm_set_security(pc, "security_endpoint", &protocomm_security1, &sec1_
↪params);

    /* Private data passed to the endpoint must be valid throughout the scope of␣
↪protocomm endpoint. This need not be static, i.e., could be dynamically␣
↪allocated and freed at the time of endpoint removal. */
    static uint32_t priv_data = 1234;

    /* Add a new endpoint for the protocomm instance identified by a unique name,␣
↪and register a handler function along with the private data to be passed at the␣
↪time of handler execution. Multiple endpoints can be added as long as they are␣
↪identified by unique names. */
    protocomm_add_endpoint(pc, "echo_req_endpoint",
                           echo_req_handler, (void *) &priv_data);
    return pc;
}

/* The example function for stopping a protocomm instance. */
void stop_pc(protocomm_t *pc)
{
    /* Remove the endpoint identified by its unique name. */
    protocomm_remove_endpoint(pc, "echo_req_endpoint");

    /* Remove the security endpoint identified by its name. */
    protocomm_unset_security(pc, "security_endpoint");

    /* Stop the HTTP server. */
    protocomm_httpd_stop(pc);

    /* Delete, namely deallocate the protocomm instance. */
    protocomm_delete(pc);
}
```

Bluetooth LE Transport Example with Security 0

For sample usage, see wifi_provisioning/src/scheme_ble.c.

```
/* The example function for launching a secure protocomm instance over Bluetooth␣
↪LE. */
protocomm_t *start_pc()
{
    protocomm_t *pc = protocomm_new();

    /* Endpoint UUIDs */
    protocomm_ble_name_uuid_t nu_lookup_table[] = {
        {"security_endpoint", 0xFF51},
        {"echo_req_endpoint", 0xFF52}
    };

    /* Config for protocomm_ble_start(). */
    protocomm_ble_config_t config = {
        .service_uuid = {
            /* LSB <---------------------------------------
            * ---------------------------------------> MSB */
            0xfb, 0x34, 0x9b, 0x5f, 0x80, 0x00, 0x00, 0x80,
            0x00, 0x10, 0x00, 0x00, 0xFF, 0xFF, 0x00, 0x00,
        },
        .nu_lookup_count = sizeof(nu_lookup_table)/sizeof(nu_lookup_table[0]),
        .nu_lookup = nu_lookup_table
    };

    /* Start protocomm layer on top of Bluetooth LE. */
    protocomm_ble_start(pc, &config);

    /* For protocomm_security0, Proof of Possession is not used, and can be kept␣
↪NULL. */
    protocomm_set_security(pc, "security_endpoint", &protocomm_security0, NULL);
    protocomm_add_endpoint(pc, "echo_req_endpoint", echo_req_handler, NULL);
    return pc;
}

/* The example function for stopping a protocomm instance. */
void stop_pc(protocomm_t *pc)
{
    protocomm_remove_endpoint(pc, "echo_req_endpoint");
    protocomm_unset_security(pc, "security_endpoint");

    /* Stop the Bluetooth LE protocomm service. */
    protocomm_ble_stop(pc);

    protocomm_delete(pc);
}
```

API Reference

Header File

- components/protocomm/include/common/protocomm.h
- This header file can be included with:

```
#include "protocomm.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Functions

protocomm_t ***protocomm_new*** (void)

Create a new protocomm instance.

This API will return a new dynamically allocated protocomm instance with all elements of the protocomm_t structure initialized to NULL.

Returns

- protocomm_t* : On success
- NULL : No memory for allocating new instance

void ***protocomm_delete*** (*protocomm_t* *pc)

Delete a protocomm instance.

This API will deallocate a protocomm instance that was created using `protocomm_new()`.

Parameters **pc** -- **[in]** Pointer to the protocomm instance to be deleted

esp_err_t ***protocomm_add_endpoint*** (*protocomm_t* *pc, const char *ep_name, *protocomm_req_handler_t* h, void *priv_data)

Add endpoint request handler for a protocomm instance.

This API will bind an endpoint handler function to the specified endpoint name, along with any private data that needs to be pass to the handler at the time of call.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.
- This function internally calls the registered `add_endpoint()` function of the selected transport which is a member of the protocomm_t instance structure.

Parameters

- **pc** -- **[in]** Pointer to the protocomm instance
- **ep_name** -- **[in]** Endpoint identifier(name) string
- **h** -- **[in]** Endpoint handler function
- **priv_data** -- **[in]** Pointer to private data to be passed as a parameter to the handler function on call. Pass NULL if not needed.

Returns

- ESP_OK : Success
- ESP_FAIL : Error adding endpoint / Endpoint with this name already exists
- ESP_ERR_NO_MEM : Error allocating endpoint resource
- ESP_ERR_INVALID_ARG : Null instance/name/handler arguments

esp_err_t ***protocomm_remove_endpoint*** (*protocomm_t* *pc, const char *ep_name)

Remove endpoint request handler for a protocomm instance.

This API will remove a registered endpoint handler identified by an endpoint name.

Note:

- This function internally calls the registered `remove_endpoint()` function which is a member of the protocomm_t instance structure.

Parameters
- **pc** -- **[in]** Pointer to the protocomm instance
- **ep_name** -- **[in]** Endpoint identifier(name) string

Returns
- ESP_OK : Success
- ESP_ERR_NOT_FOUND : Endpoint with specified name doesn't exist
- ESP_ERR_INVALID_ARG : Null instance/name arguments

esp_err_t **protocomm_open_session** (*protocomm_t* *pc, uint32_t session_id)

Allocates internal resources for new transport session.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.

Parameters
- **pc** -- **[in]** Pointer to the protocomm instance
- **session_id** -- **[in]** Unique ID for a communication session

Returns
- ESP_OK : Request handled successfully
- ESP_ERR_NO_MEM : Error allocating internal resource
- ESP_ERR_INVALID_ARG : Null instance/name arguments

esp_err_t **protocomm_close_session** (*protocomm_t* *pc, uint32_t session_id)

Frees internal resources used by a transport session.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.

Parameters
- **pc** -- **[in]** Pointer to the protocomm instance
- **session_id** -- **[in]** Unique ID for a communication session

Returns
- ESP_OK : Request handled successfully
- ESP_ERR_INVALID_ARG : Null instance/name arguments

esp_err_t **protocomm_req_handle** (*protocomm_t* *pc, const char *ep_name, uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen)

Calls the registered handler of an endpoint session for processing incoming data and generating the response.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.
- Resulting output buffer must be deallocated by the caller.

Parameters
- **pc** -- **[in]** Pointer to the protocomm instance
- **ep_name** -- **[in]** Endpoint identifier(name) string
- **session_id** -- **[in]** Unique ID for a communication session
- **inbuf** -- **[in]** Input buffer contains input request data which is to be processed by the registered handler
- **inlen** -- **[in]** Length of the input buffer

- **outbuf** -- **[out]** Pointer to internally allocated output buffer, where the resulting response data output from the registered handler is to be stored
- **outlen** -- **[out]** Buffer length of the allocated output buffer

Returns

- ESP_OK : Request handled successfully
- ESP_FAIL : Internal error in execution of registered handler
- ESP_ERR_NO_MEM : Error allocating internal resource
- ESP_ERR_NOT_FOUND : Endpoint with specified name doesn't exist
- ESP_ERR_INVALID_ARG : Null instance/name arguments

esp_err_t **protocomm_set_security** (*protocomm_t* *pc, const char *ep_name, const *protocomm_security_t* *sec, const void *sec_params)

Add endpoint security for a protocomm instance.

This API will bind a security session establisher to the specified endpoint name, along with any proof of possession that may be required for authenticating a session client.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.
- The choice of security can be any `protocomm_security_t` instance. Choices `protocomm_security0` and `protocomm_security1` and `protocomm_security2` are readily available.

Parameters

- **pc** -- **[in]** Pointer to the protocomm instance
- **ep_name** -- **[in]** Endpoint identifier(name) string
- **sec** -- **[in]** Pointer to endpoint security instance
- **sec_params** -- **[in]** Pointer to security params (NULL if not needed) The pointer should contain the security params struct of appropriate security version. For protocomm security version 1 and 2 sec_params should contain pointer to struct of type protocomm_security1_params_t and protocmm_security2_params_t respectively. The contents of this pointer must be valid till the security session has been running and is not closed.

Returns

- ESP_OK : Success
- ESP_FAIL : Error adding endpoint / Endpoint with this name already exists
- ESP_ERR_INVALID_STATE : Security endpoint already set
- ESP_ERR_NO_MEM : Error allocating endpoint resource
- ESP_ERR_INVALID_ARG : Null instance/name/handler arguments

esp_err_t **protocomm_unset_security** (*protocomm_t* *pc, const char *ep_name)

Remove endpoint security for a protocomm instance.

This API will remove a registered security endpoint identified by an endpoint name.

Parameters

- **pc** -- **[in]** Pointer to the protocomm instance
- **ep_name** -- **[in]** Endpoint identifier(name) string

Returns

- ESP_OK : Success
- ESP_ERR_NOT_FOUND : Endpoint with specified name doesn't exist
- ESP_ERR_INVALID_ARG : Null instance/name arguments

esp_err_t **protocomm_set_version** (*protocomm_t* *pc, const char *ep_name, const char *version)

Set endpoint for version verification.

This API can be used for setting an application specific protocol version which can be verified by clients through the endpoint.

Note:

- An endpoint must be bound to a valid protocomm instance, created using `protocomm_new()`.

Parameters

- **`pc`** -- **[in]** Pointer to the protocomm instance
- **`ep_name`** -- **[in]** Endpoint identifier(name) string
- **`version`** -- **[in]** Version identifier(name) string

Returns

- ESP_OK : Success
- ESP_FAIL : Error adding endpoint / Endpoint with this name already exists
- ESP_ERR_INVALID_STATE : Version endpoint already set
- ESP_ERR_NO_MEM : Error allocating endpoint resource
- ESP_ERR_INVALID_ARG : Null instance/name/handler arguments

esp_err_t **`protocomm_unset_version`** (*protocomm_t* *pc, const char *ep_name)

Remove version verification endpoint from a protocomm instance.

This API will remove a registered version endpoint identified by an endpoint name.

Parameters

- **`pc`** -- **[in]** Pointer to the protocomm instance
- **`ep_name`** -- **[in]** Endpoint identifier(name) string

Returns

- ESP_OK : Success
- ESP_ERR_NOT_FOUND : Endpoint with specified name doesn't exist
- ESP_ERR_INVALID_ARG : Null instance/name arguments

Type Definitions

typedef *esp_err_t* (***`protocomm_req_handler_t`**)(uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen, void *priv_data)

Function prototype for protocomm endpoint handler.

typedef struct protocomm **`protocomm_t`**

This structure corresponds to a unique instance of protocomm returned when the API `protocomm_new()` is called. The remaining Protocomm APIs require this object as the first parameter.

Note: Structure of the protocomm object is kept private

Header File

- components/protocomm/include/security/protocomm_security.h
- This header file can be included with:

```
#include "protocomm_security.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Structures

struct **protocomm_security1_params**

Protocomm Security 1 parameters: Proof Of Possession.

Public Members

const uint8_t ***data**

Pointer to buffer containing the proof of possession data

uint16_t **len**

Length (in bytes) of the proof of possession data

struct **protocomm_security2_params**

Protocomm Security 2 parameters: Salt and Verifier.

Public Members

const char ***salt**

Pointer to the buffer containing the salt

uint16_t **salt_len**

Length (in bytes) of the salt

const char ***verifier**

Pointer to the buffer containing the verifier

uint16_t **verifier_len**

Length (in bytes) of the verifier

struct **protocomm_security**

Protocomm security object structure.

The member functions are used for implementing secure protocomm sessions.

Note: This structure should not have any dynamic members to allow re-entrancy

Public Members

int **ver**

Unique version number of security implementation

esp_err_t (***init**)(*protocomm_security_handle_t* *handle)

Function for initializing/allocating security infrastructure

esp_err_t (***cleanup**)(*protocomm_security_handle_t* handle)

Function for deallocating security infrastructure

esp_err_t (***new_transport_session***)(*protocomm_security_handle_t* handle, uint32_t session_id)

Starts new secure transport session with specified ID

esp_err_t (***close_transport_session***)(*protocomm_security_handle_t* handle, uint32_t session_id)

Closes a secure transport session with specified ID

esp_err_t (***security_req_handler***)(*protocomm_security_handle_t* handle, const void *sec_params, uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen, void *priv_data)

Handler function for authenticating connection request and establishing secure session

esp_err_t (***encrypt***)(*protocomm_security_handle_t* handle, uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen)

Function which implements the encryption algorithm

esp_err_t (***decrypt***)(*protocomm_security_handle_t* handle, uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen)

Function which implements the decryption algorithm

Type Definitions

typedef struct *protocomm_security1_params* **protocomm_security1_params_t**

Protocomm Security 1 parameters: Proof Of Possession.

typedef *protocomm_security1_params_t* **protocomm_security_pop_t**

typedef struct *protocomm_security2_params* **protocomm_security2_params_t**

Protocomm Security 2 parameters: Salt and Verifier.

typedef void ***protocomm_security_handle_t**

typedef struct *protocomm_security* **protocomm_security_t**

Protocomm security object structure.

The member functions are used for implementing secure protocomm sessions.

Note: This structure should not have any dynamic members to allow re-entrancy

Enumerations

enum **protocomm_security_session_event_t**

Events generated by the protocomm security layer.

These events are generated while establishing secured session.

Values:

enumerator **PROTOCOMM_SECURITY_SESSION_SETUP_OK**

Secured session established successfully

enumerator **PROTOCOMM_SECURITY_SESSION_INVALID_SECURITY_PARAMS**

Received invalid (NULL) security parameters (username / client public-key)

enumerator **PROTOCOMM_SECURITY_SESSION_CREDENTIALS_MISMATCH**

Received incorrect credentials (username / PoP)

Header File

- components/protocomm/include/security/protocomm_security0.h
- This header file can be included with:

```
#include "protocomm_security0.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Header File

- components/protocomm/include/security/protocomm_security1.h
- This header file can be included with:

```
#include "protocomm_security1.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Header File

- components/protocomm/include/security/protocomm_security2.h
- This header file can be included with:

```
#include "protocomm_security2.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Header File

- components/protocomm/include/crypto/srp6a/esp_srp.h
- This header file can be included with:

```
#include "esp_srp.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Functions

esp_srp_handle_t ***esp_srp_init*** (*esp_ng_type_t* ng)

Initialize srp context for given NG type.

Note: the handle gets freed with `esp_srp_free`

Parameters **ng** -- NG type given by `esp_ng_type_t`
Returns esp_srp_handle_t* srp handle

void ***esp_srp_free*** (*esp_srp_handle_t* *hd)

free esp_srp_context

Parameters **hd** -- handle to be free

esp_err_t ***esp_srp_srv_pubkey*** (*esp_srp_handle_t* *hd, const char *username, int username_len, const char *pass, int pass_len, int salt_len, char **bytes_B, int *len_B, char **bytes_salt)

Returns B (pub key) and salt. [Step2.b].

Note: *bytes_B MUST NOT BE FREED BY THE CALLER

Note: *bytes_salt MUST NOT BE FREE BY THE CALLER

Parameters

- **hd** -- esp_srp handle
- **username** -- Username not expected NULL terminated
- **username_len** -- Username length
- **pass** -- Password not expected to be NULL terminated
- **pass_len** -- Pasword length
- **salt_len** -- Salt length
- **bytes_B** -- Public Key returned
- **len_B** -- Length of the public key
- **bytes_salt** -- Salt bytes generated

Returns esp_err_t ESP_OK on success, appropriate error otherwise

esp_err_t ***esp_srp_gen_salt_verifier*** (const char *username, int username_len, const char *pass, int pass_len, char **bytes_salt, int salt_len, char **verifier, int *verifier_len)

Generate salt-verifier pair, given username, password and salt length.

Note: if API has returned ESP_OK, salt and verifier generated need to be freed by caller

---

Note: Usually, username and password are not saved on the device. Rather salt and verifier are generated outside the device and are embedded. this covenience API can be used to generate salt and verifier on the fly

for development use case. OR for devices which intentionally want to generate different password each time and can send it to the client securely. e.g., a device has a display and it shows the pin

---

Parameters

- **username** -- **[in]** username
- **username_len** -- **[in]** length of the username
- **pass** -- **[in]** password
- **pass_len** -- **[in]** length of the password
- **bytes_salt** -- **[out]** generated salt on successful generation, or NULL
- **salt_len** -- **[in]** salt length
- **verifier** -- **[out]** generated verifier on successful generation, or NULL
- **verifier_len** -- **[out]** length of the generated verifier

Returns esp_err_t ESP_OK on success, appropriate error otherwise

esp_err_t **esp_srp_set_salt_verifier**(*esp_srp_handle_t* *hd, const char *salt, int salt_len, const char *verifier, int verifier_len)

Set the Salt and Verifier pre-generated for a given password. This should be used only if the actual password is not available. The public key can then be generated using *esp_srp_srv_pubkey_from_salt_verifier()* and not *esp_srp_srv_pubkey()*

Parameters

- **hd** -- esp_srp_handle
- **salt** -- pre-generated salt bytes
- **salt_len** -- length of the salt bytes
- **verifier** -- pre-generated verifier
- **verifier_len** -- length of the verifier bytes

Returns esp_err_t ESP_OK on success, appropriate error otherwise

esp_err_t **esp_srp_srv_pubkey_from_salt_verifier**(*esp_srp_handle_t* *hd, char **bytes_B, int *len_B)

Returns B (pub key)[Step2.b] when the salt and verifier are set using *esp_srp_set_salt_verifier()*

Note: *bytes_B MUST NOT BE FREED BY THE CALLER

Parameters

- **hd** -- esp_srp handle
- **bytes_B** -- Key returned to the called
- **len_B** -- Length of the key returned

Returns esp_err_t ESP_OK on success, appropriate error otherwise

esp_err_t **esp_srp_get_session_key**(*esp_srp_handle_t* *hd, char *bytes_A, int len_A, char **bytes_key, uint16_t *len_key)

Get session key in `*bytes_key` given by len in `*len_key`. [Step2.c].

This calculated session key is used for further communication given the proofs are exchanged/authenticated with `esp_srp_exchange_proofs`

Note: *bytes_key MUST NOT BE FREED BY THE CALLER

Parameters

- **hd** -- esp_srp handle
- **bytes_A** -- Private Key
- **len_A** -- Private Key length
- **bytes_key** -- Key returned to the caller
- **len_key** -- length of the key in *bytes_key

Returns esp_err_t ESP_OK on success, appropriate error otherwise

esp_err_t **esp_srp_exchange_proofs** (*esp_srp_handle_t* *hd, char *username, uint16_t username_len, char *bytes_user_proof, char *bytes_host_proof)

Complete the authentication. If this step fails, the session_key exchanged should not be used.

This is the final authentication step in SRP algorithm [Step4.1, Step4.b, Step4.c]

Parameters

- **hd** -- esp_srp handle
- **username** -- Username not expected NULL terminated
- **username_len** -- Username length
- **bytes_user_proof** -- param in
- **bytes_host_proof** -- parameter out (should be SHA512_DIGEST_LENGTH) bytes in size

Returns esp_err_t ESP_OK if user's proof is ok and subsequently bytes_host_proof is populated with our own proof.

Type Definitions

typedef struct esp_srp_handle **esp_srp_handle_t**

esp_srp handle as the result of `esp_srp_init`

The handle is returned by `esp_srp_init` on successful init. It is then passed for subsequent API calls as an argument. `esp_srp_free` can be used to clean up the handle. After `esp_srp_free` the handle becomes invalid.

Enumerations

enum **esp_ng_type_t**

Large prime+generator to be used for the algorithm.

Values:

enumerator **ESP_NG_3072**

Header File

- components/protocomm/include/transports/protocomm_httpd.h
- This header file can be included with:

```
#include "protocomm_httpd.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Functions

esp_err_t **protocomm_httpd_start** (*protocomm_t* *pc, const *protocomm_httpd_config_t* *config)

Start HTTPD protocomm transport.

This API internally creates a framework to allow endpoint registration and security configuration for the protocomm.

Note: This is a singleton. ie. Protocomm can have multiple instances, but only one instance can be bound to an HTTP transport layer.

Parameters
- **pc** -- **[in]** Protocomm instance pointer obtained from protocomm_new()
- **config** -- **[in]** Pointer to config structure for initializing HTTP server

Returns
- ESP_OK : Success
- ESP_ERR_INVALID_ARG : Null arguments
- ESP_ERR_NOT_SUPPORTED : Transport layer bound to another protocomm instance
- ESP_ERR_INVALID_STATE : Transport layer already bound to this protocomm instance
- ESP_ERR_NO_MEM : Memory allocation for server resource failed
- ESP_ERR_HTTPD_* : HTTP server error on start

esp_err_t **protocomm_httpd_stop**(*protocomm_t* *pc)

Stop HTTPD protocomm transport.

This API cleans up the HTTPD transport protocomm and frees all the handlers registered with the protocomm.

Parameters **pc** -- **[in]** Same protocomm instance that was passed to protocomm_httpd_start()

Returns
- ESP_OK : Success
- ESP_ERR_INVALID_ARG : Null / incorrect protocomm instance pointer

Unions

union **protocomm_httpd_config_data_t**

#include <protocomm_httpd.h> Protocomm HTTPD Configuration Data

Public Members

void ***handle**

HTTP Server Handle, if ext_handle_provided is set to true

protocomm_http_server_config_t **config**

HTTP Server Configuration, if a server is not already active

Structures

struct **protocomm_http_server_config_t**

Config parameters for protocomm HTTP server.

Public Members

uint16_t **port**

Port on which the HTTP server will listen

size_t **stack_size**

Stack size of server task, adjusted depending upon stack usage of endpoint handler

unsigned **task_priority**

Priority of server task

struct **protocomm_httpd_config_t**

Config parameters for protocomm HTTP server.

Public Members

bool **ext_handle_provided**

Flag to indicate of an external HTTP Server Handle has been provided. In such as case, protocomm will use the same HTTP Server and not start a new one internally.

protocomm_httpd_config_data_t **data**

Protocomm HTTPD Configuration Data

Macros

PROTOCOMM_HTTPD_DEFAULT_CONFIG()

Header File

- components/protocomm/include/transports/protocomm_ble.h
- This header file can be included with:

```
#include "protocomm_ble.h"
```

- This header file is a part of the API provided by the `protocomm` component. To declare that your component depends on `protocomm`, add the following to your CMakeLists.txt:

```
REQUIRES protocomm
```

or

```
PRIV_REQUIRES protocomm
```

Functions

esp_err_t **protocomm_ble_start**(*protocomm_t* *pc, const *protocomm_ble_config_t* *config)

Start Bluetooth Low Energy based transport layer for provisioning.

Initialize and start required BLE service for provisioning. This includes the initialization for characteristics/service for BLE.

Parameters

- **pc** -- **[in]** Protocomm instance pointer obtained from protocomm_new()
- **config** -- **[in]** Pointer to config structure for initializing BLE

Returns

- ESP_OK : Success
- ESP_FAIL : Simple BLE start error
- ESP_ERR_NO_MEM : Error allocating memory for internal resources
- ESP_ERR_INVALID_STATE : Error in ble config
- ESP_ERR_INVALID_ARG : Null arguments

esp_err_t **protocomm_ble_stop**(*protocomm_t* *pc)

Stop Bluetooth Low Energy based transport layer for provisioning.

Stops service/task responsible for BLE based interactions for provisioning

Note: You might want to optionally reclaim memory from Bluetooth. Refer to the documentation of `esp_bt_mem_release` in that case.

Parameters **pc** -- **[in]** Same protocomm instance that was passed to protocomm_ble_start()

Returns

- ESP_OK : Success
- ESP_FAIL : Simple BLE stop error
- ESP_ERR_INVALID_ARG : Null / incorrect protocomm instance

Structures

struct **name_uuid**

This structure maps handler required by protocomm layer to UUIDs which are used to uniquely identify BLE characteristics from a smartphone or a similar client device.

Public Members

const char ***name**

Name of the handler, which is passed to protocomm layer

uint16_t **uuid**

UUID to be assigned to the BLE characteristic which is mapped to the handler

struct **protocomm_ble_event_t**

Structure for BLE events in Protocomm.

Public Members

uint16_t **evt_type**

This field indicates the type of BLE event that occurred.

uint16_t **conn_handle**

The handle of the relevant connection.

uint16_t **conn_status**

The status of the connection attempt; o 0: the connection was successfully established. o BLE host error code: the connection attempt failed for the specified reason.

uint16_t **disconnect_reason**

Return code indicating the reason for the disconnect.

struct **protocomm_ble_config**

Config parameters for protocomm BLE service.

Public Members

char **device_name**[MAX_BLE_DEVNAME_LEN + 1]

BLE device name being broadcast at the time of provisioning

uint8_t **service_uuid**[BLE_UUID128_VAL_LENGTH]

128 bit UUID of the provisioning service

uint8_t ***manufacturer_data**

BLE device manufacturer data pointer in advertisement

ssize_t **manufacturer_data_len**

BLE device manufacturer data length in advertisement

ssize_t **nu_lookup_count**

Number of entries in the Name-UUID lookup table

protocomm_ble_name_uuid_t ***nu_lookup**

Pointer to the Name-UUID lookup table

unsigned **ble_bonding**

BLE bonding

unsigned **ble_sm_sc**

BLE security flag

unsigned **ble_link_encryption**

BLE security flag

uint8_t ***ble_addr**

BLE address

Macros

MAX_BLE_DEVNAME_LEN

BLE device name cannot be larger than this value 31 bytes (max scan response size) - 1 byte (length) - 1 byte (type) = 29 bytes

BLE_UUID128_VAL_LENGTH

MAX_BLE_MANUFACTURER_DATA_LEN

Theoretically, the limit for max manufacturer length remains same as BLE device name i.e. 31 bytes (max scan response size) - 1 byte (length) - 1 byte (type) = 29 bytes However, manufacturer data goes along with BLE device name in scan response. So, it is important to understand the actual length should be smaller than (29 - (BLE device name length) - 2).

BLE_ADDR_LEN

Type Definitions

typedef struct *name_uuid* **protocomm_ble_name_uuid_t**

This structure maps handler required by protocomm layer to UUIDs which are used to uniquely identify BLE characteristics from a smartphone or a similar client device.

typedef struct *protocomm_ble_config* **protocomm_ble_config_t**

Config parameters for protocomm BLE service.

Enumerations

enum **protocomm_transport_ble_event_t**

Events generated by BLE transport.

These events are generated when the BLE transport is paired and disconnected.

Values:

enumerator **PROTOCOMM_TRANSPORT_BLE_CONNECTED**

enumerator **PROTOCOMM_TRANSPORT_BLE_DISCONNECTED**

2.8.2 Unified Provisioning

Overview

The unified provisioning support in the ESP-IDF provides an extensible mechanism to the developers to configure the device with the Wi-Fi credentials and/or other custom configuration using various transports and different security schemes. Depending on the use case, it provides a complete and ready solution for Wi-Fi network provisioning along with example iOS and Android applications. The developers can choose to extend the device-side and phone-app side implementations to accommodate their requirements for sending additional configuration data. The following are the important features of this implementation:

1. **Extensible Protocol**

The protocol is completely flexible and it offers the ability for the developers to send custom configuration in the provisioning process. The data representation is also left to the application to decide.

2. **Transport Flexibility**

The protocol can work on Wi-Fi (SoftAP + HTTP server) or on Bluetooth LE as a transport protocol. The framework provides an ability to add support for any other transport easily as long as command-response behavior can be supported on the transport.

3. **Security Scheme Flexibility**

It is understood that each use case may require different security scheme to secure the data that is exchanged in the provisioning process. Some applications may work with SoftAP that is WPA2 protected or Bluetooth LE with the "just-works" security. Or the applications may consider the transport to be insecure and may want application-level security. The unified provisioning framework allows the application to choose the security as deemed suitable.

4. **Compact Data Representation**

The protocol uses Google Protobufs as a data representation for session setup and Wi-Fi provisioning. They provide a compact data representation and ability to parse the data in multiple programming languages in native format. Please note that this data representation is not forced on application-specific data and the developers may choose the representation of their choice.

Typical Provisioning Process

Deciding on Transport

The unified provisioning subsystem supports Wi-Fi (SoftAP+HTTP server) and Bluetooth LE (GATT based) transport schemes. The following points need to be considered while selecting the best possible transport for provisioning:

1. The Bluetooth LE-based transport has the advantage of maintaining an intact communication channel between the device and the client during the provisioning, which ensures reliable provisioning feedback.
2. The Bluetooth LE-based provisioning implementation makes the user experience better from the phone apps as on Android and iOS both, the phone app can discover and connect to the device without requiring the user to go out of the phone app.
3. However, the Bluetooth LE transport consumes about 110 KB memory at runtime. If the product does not use the Bluetooth LE or Bluetooth functionality after provisioning is done, almost all the memory can be reclaimed and added into the heap.
4. The SoftAP-based transport is highly interoperable. However, there are a few considerations:
    - The device uses the same radio to host the SoftAP and also to connect to the configured AP. Since these could potentially be on different channels, it may cause connection status updates not to be reliably received by the phone
    - The phone (client) has to disconnect from its current AP in order to connect to the SoftAP. The original network will get restored only when the provisioning process is complete, and the softAP is taken down.
5. The SoftAP transport does not require much additional memory for the Wi-Fi use cases.
6. The SoftAP-based provisioning requires the phone-app user to go to `System Settings` to connect to the Wi-Fi network hosted by the device in the iOS system. The discovery (scanning) as well as connection APIs are not available for the iOS applications.

Deciding on Security

Depending on the transport and other constraints, the security scheme needs to be selected by the application developers. The following considerations need to be given from the provisioning-security perspective:

1. The configuration data sent from the client to the device and the response have to be secured.
2. The client should authenticate the device that it is connected to.
3. The device manufacturer may choose proof-of-possession (PoP), a unique per-device secret to be entered on the provisioning client as a security measure to make sure that only the user can provision the device in their possession.

There are two levels of security schemes, of which the developer may select one or a combination, depending on requirements.

1. **Transport Security**

For SoftAP provisioning, developers may choose WPA2-protected security with unique per-device passphrase. Unique per-device passphrase can also act as a proof-of-possession. For Bluetooth LE, the "just-works" security can be used as a transport-level security after assessing its provided level of security.

2. **Application Security**

The unified provisioning subsystem provides the application-level security (*Security 1 Scheme*) that provides data protection and authentication through PoP, if the application does not use the transport-level security, or if the transport-level security is not sufficient for the use case.

Device Discovery

The advertisement and device discovery is left to the application and depending on the protocol chosen, the phone apps and device-firmware application can choose appropriate method for advertisement and discovery.

For the SoftAP+HTTP transport, typically the SSID (network name) of the AP hosted by the device can be used for discovery.

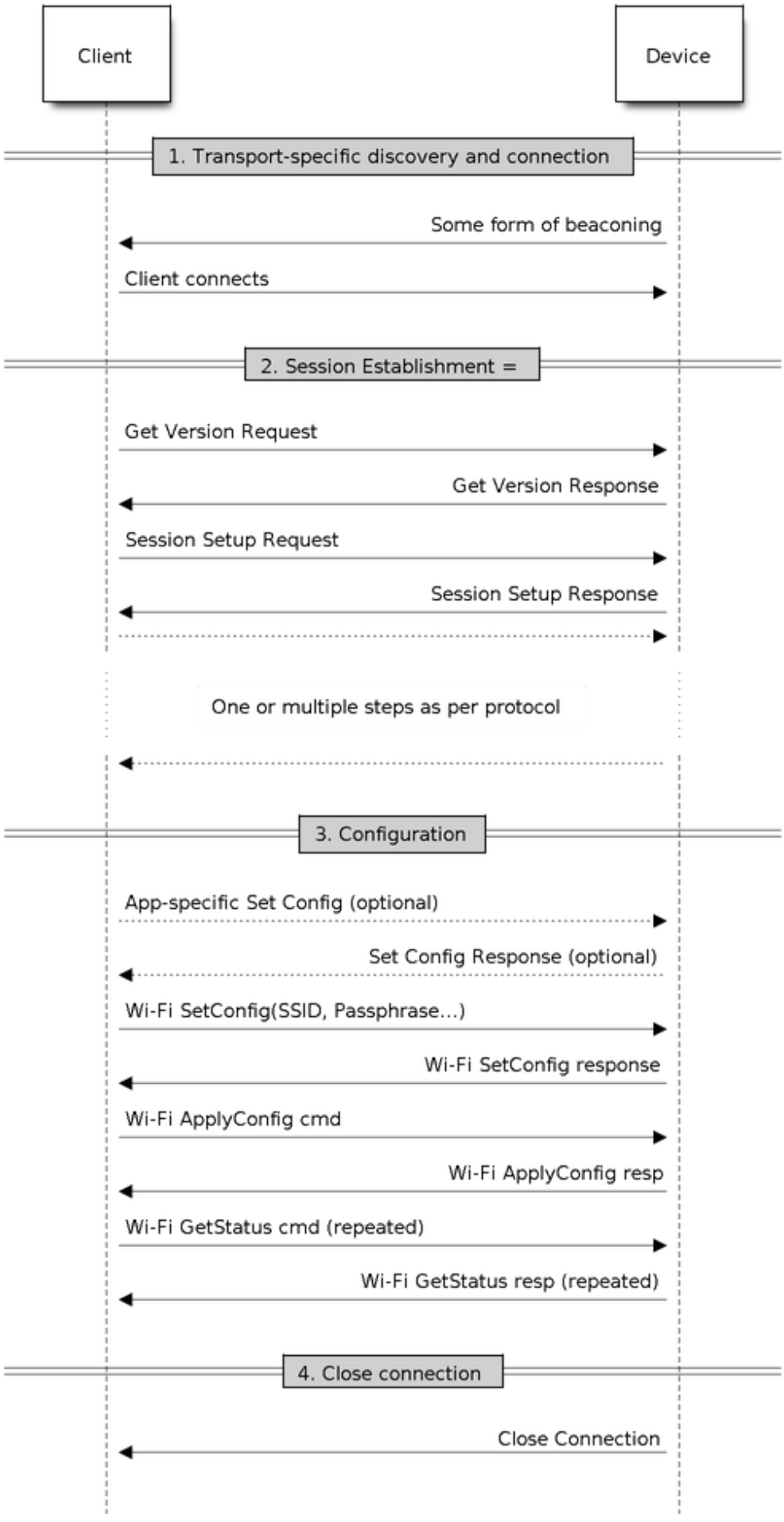


Fig. 36: Typical Provisioning Process

For the Bluetooth LE transport, device name or primary service included in the advertisement or a combination of both can be used for discovery.

Architecture

The below diagram shows the architecture of unified provisioning:

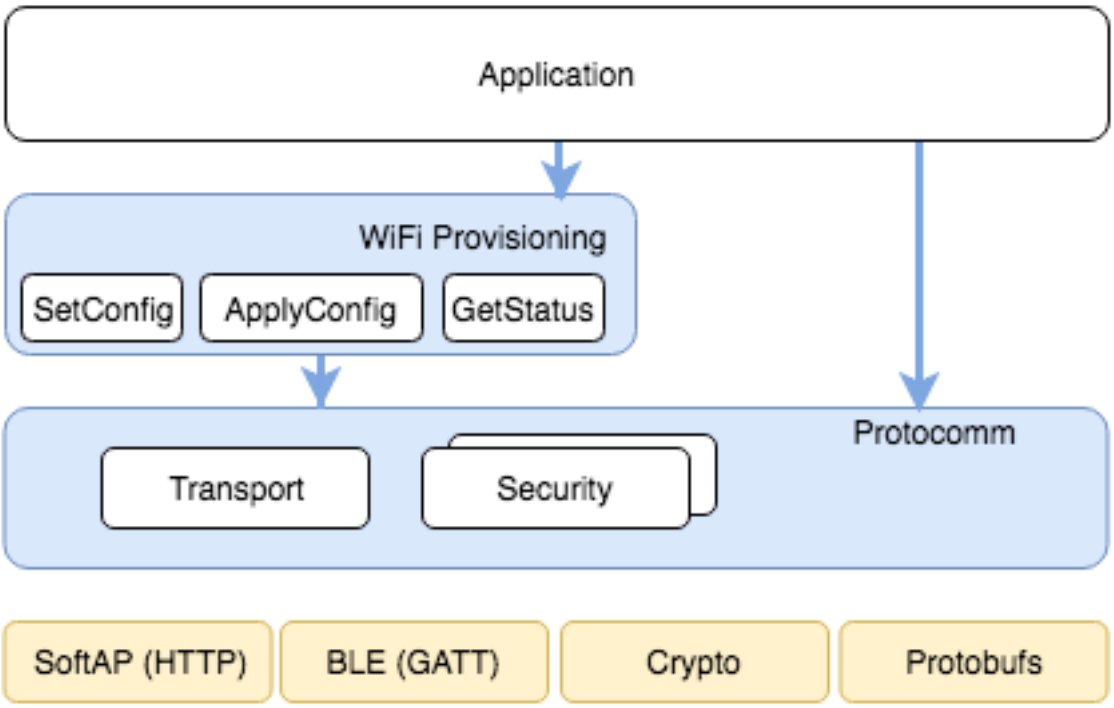


Fig. 37: Unified Provisioning Architecture

It relies on the base layer called *Protocol Communication* (protocomm) which provides a framework for security schemes and transport mechanisms. The Wi-Fi Provisioning layer uses protocomm to provide simple callbacks to the application for setting the configuration and getting the Wi-Fi status. The application has control over implementation of these callbacks. In addition, the application can directly use protocomm to register custom handlers.

The application creates a protocomm instance which is mapped to a specific transport and specific security scheme. Each transport in the protocomm has a concept of an "end-point" which corresponds to the logical channel for communication for specific type of information. For example, security handshake happens on a different endpoint from the Wi-Fi configuration endpoint. Each end-point is identified using a string and depending on the transport internal representation of the end-point changes. In case of the SoftAP+HTTP transport, the end-point corresponds to URI, whereas in case of Bluetooth LE, the end-point corresponds to the GATT characteristic with specific UUID. Developers can create custom end-points and implement handler for the data that is received or sent over the same end-point.

Security Schemes

At present, the unified provisioning supports the following security schemes:

1. Security 0

No security (No encryption).

2. Security 1

Curve25519-based key exchange, shared key derivation and AES256-CTR mode encryption of the data. It supports two modes :

a. Authorized - Proof of Possession (PoP) string used to authorize session and derive shared key.
b. No Auth (Null PoP) - Shared key derived through key exchange only.

3. Security 2

SRP6a-based shared key derivation and AES256-GCM mode encryption of the data.

Note: The respective security schemes need to be enabled through the project configuration menu. Please refer to *Enabling Protocomm Security Version* for more details.

Security 1 Scheme

The Security 1 scheme details are shown in the below sequence diagram:

Security 2 Scheme

The Security 2 scheme is based on the Secure Remote Password (SRP6a) protocol, see RFC 5054.

The protocol requires the Salt and Verifier to be generated beforehand with the help of the identifying username `I` and the plaintext password `p`. The Salt and Verifier are then stored on ESP32-S3.

- The password `p` and the username `I` are to be provided to the Phone App (Provisioning entity) by suitable means, e.g., QR code sticker.

Details about the Security 2 scheme are shown in the below sequence diagram:

Sample Code

Please refer to *Protocol Communication* and *Wi-Fi Provisioning* for API guides and code snippets on example usage.

Application implementation can be found as an example under provisioning.

Provisioning Tools

Provisioning applications are available for various platforms, along with source code:

- **Android:**
  - Bluetooth LE Provisioning app on Play Store.
  - SoftAP Provisioning app on Play Store.
  - Source code on GitHub: esp-idf-provisioning-android.
- **iOS:**
  - Bluetooth LE Provisioning app on App Store.
  - SoftAP Provisioning app on App Store.
  - Source code on GitHub: esp-idf-provisioning-ios.
- Linux/macOS/Windows: tools/esp_prov, a Python-based command line tool for provisioning.

The phone applications offer simple UI and are thus more user centric, while the command-line application is useful as a debugging tool for developers.

2.8.3 Wi-Fi Provisioning

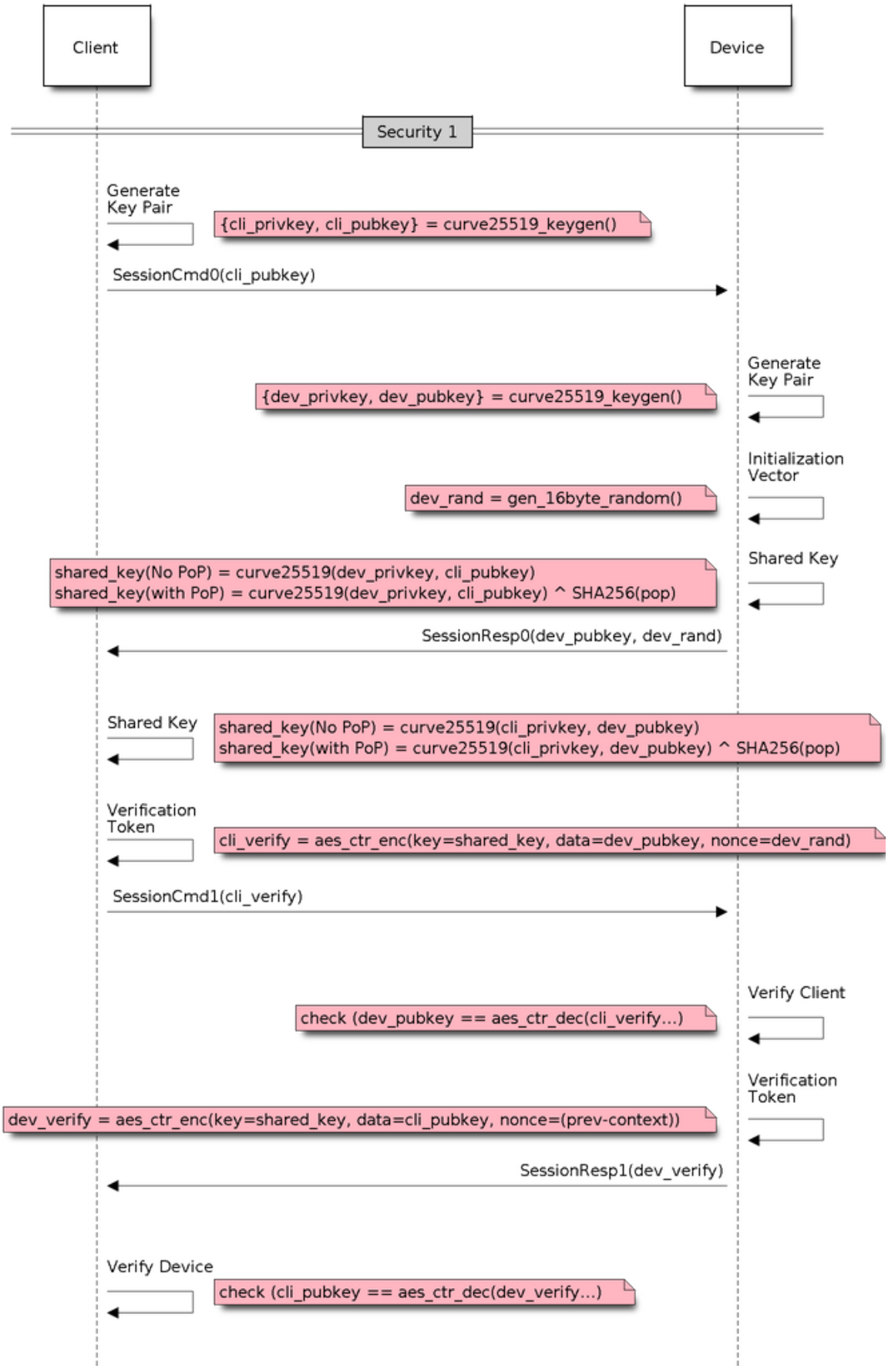


Fig. 38: Security 1

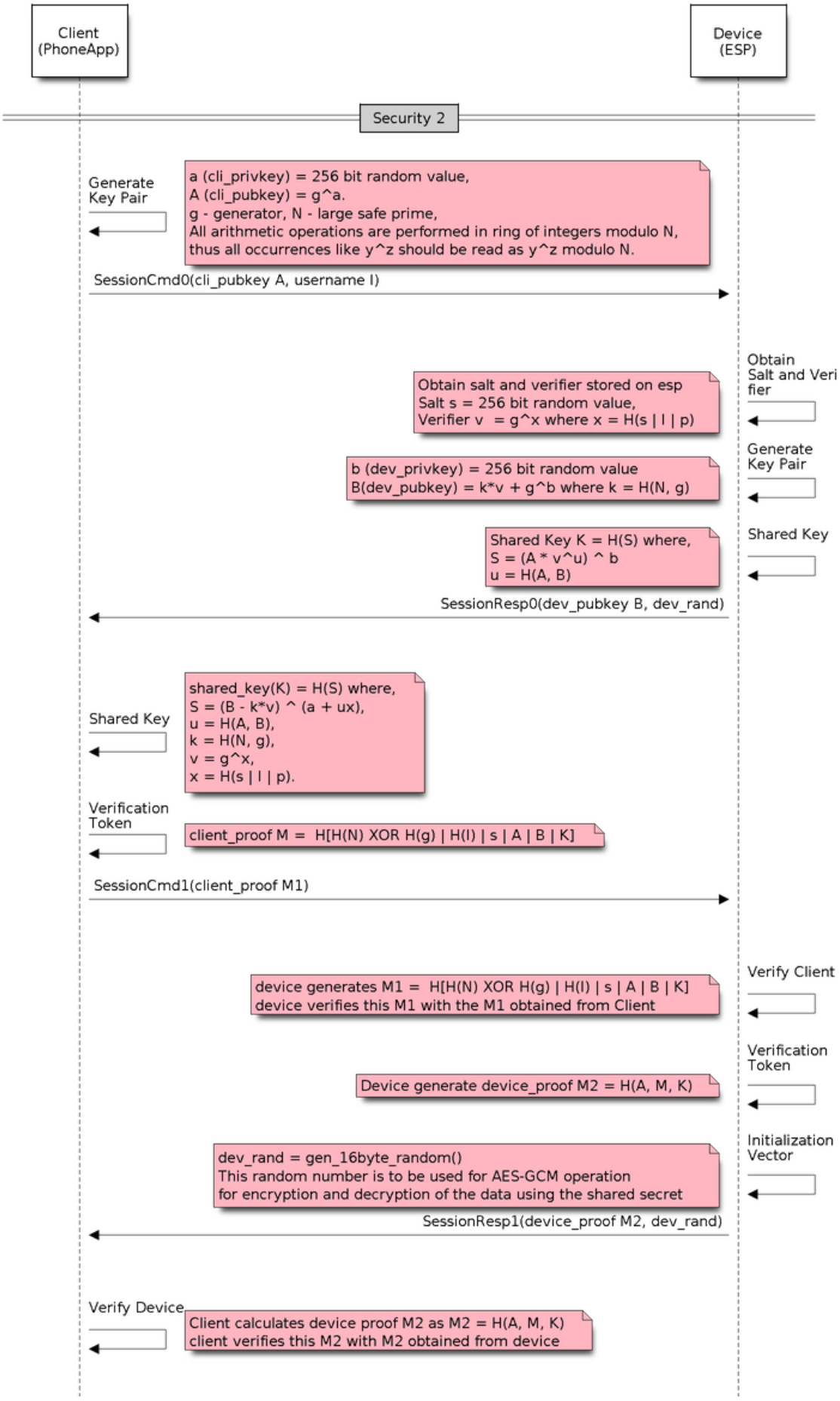


Fig. 39: Security 2

Overview

This component provides APIs that control the Wi-Fi provisioning service for receiving and configuring Wi-Fi credentials over SoftAP or Bluetooth LE transport via secure *Protocol Communication* sessions. The set of `wifi_prov_mgr_` APIs help quickly implement a provisioning service that has necessary features with minimal amount of code and sufficient flexibility.

Initialization *`wifi_prov_mgr_init()`* is called to configure and initialize the provisioning manager, and thus must be called prior to invoking any other `wifi_prov_mgr_` APIs. Note that the manager relies on other components of ESP-IDF, namely NVS, TCP/IP, Event Loop and Wi-Fi, and optionally mDNS, hence these components must be initialized beforehand. The manager can be de-initialized at any moment by making a call to *`wifi_prov_mgr_deinit()`*.

```
wifi_prov_mgr_config_t config = {
    .scheme = wifi_prov_scheme_ble,
    .scheme_event_handler = WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM
};

ESP_ERROR_CHECK( wifi_prov_mgr_init(config) );
```

The configuration structure *`wifi_prov_mgr_config_t`* has a few fields to specify the desired behavior of the manager:

- *`wifi_prov_mgr_config_t::scheme`* - This is used to specify the provisioning scheme. Each scheme corresponds to one of the modes of transport supported by protocomm. Hence, support the following options:
  - `wifi_prov_scheme_ble` - Bluetooth LE transport and GATT Server for handling the provisioning commands.
  - `wifi_prov_scheme_softap` - Wi-Fi SoftAP transport and HTTP Server for handling the provisioning commands.
  - `wifi_prov_scheme_console` - Serial transport and console for handling the provisioning commands.
- *`wifi_prov_mgr_config_t::scheme_event_handler`*: An event handler defined along with the scheme. Choosing the appropriate scheme-specific event handler allows the manager to take care of certain matters automatically. Presently, this option is not used for either the SoftAP or Console-based provisioning, but is very convenient for Bluetooth LE. To understand how, we must recall that Bluetooth requires a substantial amount of memory to function, and once the provisioning is finished, the main application may want to reclaim back this memory (or part of it) if it needs to use either Bluetooth LE or classic Bluetooth. Also, upon every future reboot of a provisioned device, this reclamation of memory needs to be performed again. To reduce this complication in using `wifi_prov_scheme_ble`, the scheme-specific handlers have been defined, and depending upon the chosen handler, the Bluetooth LE/classic Bluetooth/BTDM memory is freed automatically when the provisioning manager is de-initialized. The available options are:
  - `WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM` - Free both classic Bluetooth and Bluetooth LE/BTDM memory. Used when the main application does not require Bluetooth at all.
  - `WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BLE` - Free only Bluetooth LE memory. Used when main application requires classic Bluetooth.
  - `WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BT` - Free only classic Bluetooth. Used when main application requires Bluetooth LE. In this case freeing happens right when the manager is initialized.
  - `WIFI_PROV_EVENT_HANDLER_NONE` - Do not use any scheme specific handler. Used when the provisioning scheme is not Bluetooth LE, i.e., using SoftAP or Console, or when main application wants to handle the memory reclaiming on its own, or needs both Bluetooth LE and classic Bluetooth to function.
- *`wifi_prov_mgr_config_t::app_event_handler`* (Deprecated) - It is now recommended to catch `WIFI_PROV_EVENT` that is emitted to the default event loop handler. See definition of `wifi_prov_cb_event_t` for the list of events that are generated by the provisioning service. Here is an excerpt showing some of the provisioning events:

```
static void event_handler(void* arg, esp_event_base_t event_base,
                          int event_id, void* event_data)
{
    if (event_base == WIFI_PROV_EVENT) {
        switch (event_id) {
            case WIFI_PROV_START:
                ESP_LOGI(TAG, "Provisioning started");
                break;
            case WIFI_PROV_CRED_RECV: {
                wifi_sta_config_t *wifi_sta_cfg = (wifi_sta_config_t␣
↪*)event_data;
                ESP_LOGI(TAG, "Received Wi-Fi credentials"
                         "\n\tSSID     : %s\n\tPassword : %s",
                         (const char *) wifi_sta_cfg->ssid,
                         (const char *) wifi_sta_cfg->password);
                break;
            }
            case WIFI_PROV_CRED_FAIL: {
                wifi_prov_sta_fail_reason_t *reason = (wifi_prov_sta_fail_
↪reason_t *)event_data;
                ESP_LOGE(TAG, "Provisioning failed!\n\tReason : %s"
                         "\n\tPlease reset to factory and retry␣
↪provisioning",
                         (*reason == WIFI_PROV_STA_AUTH_ERROR) ?
                         "Wi-Fi station authentication failed" : "Wi-Fi␣
↪access-point not found");
                break;
            }
            case WIFI_PROV_CRED_SUCCESS:
                ESP_LOGI(TAG, "Provisioning successful");
                break;
            case WIFI_PROV_END:
                /* De-initialize manager once provisioning is finished */
                wifi_prov_mgr_deinit();
                break;
            default:
                break;
        }
    }
}
```

The manager can be de-initialized at any moment by making a call to *wifi_prov_mgr_deinit()*.

Check the Provisioning State Whether the device is provisioned or not can be checked at runtime by calling *wifi_prov_mgr_is_provisioned()*. This internally checks if the Wi-Fi credentials are stored in NVS.

Note that presently the manager does not have its own NVS namespace for storage of Wi-Fi credentials, instead it relies on the `esp_wifi_` APIs to set and get the credentials stored in NVS from the default location.

If the provisioning state needs to be reset, any of the following approaches may be taken:

- The associated part of NVS partition has to be erased manually
- The main application must implement some logic to call `esp_wifi_` APIs for erasing the credentials at runtime
- The main application must implement some logic to force start the provisioning irrespective of the provisioning state

```
bool provisioned = false;
ESP_ERROR_CHECK( wifi_prov_mgr_is_provisioned(&provisioned) );
```

Start the Provisioning Service At the time of starting provisioning we need to specify a service name and the corresponding key, that is to say:

- A Wi-Fi SoftAP SSID and a passphrase, respectively, when the scheme is `wifi_prov_scheme_softap`.
- Bluetooth LE device name with the service key ignored when the scheme is `wifi_prov_scheme_ble`.

Also, since internally the manager uses `protocomm`, we have the option of choosing one of the security features provided by it:

- Security 1 is secure communication which consists of a prior handshake involving X25519 key exchange along with authentication using a proof of possession `pop`, followed by AES-CTR for encryption or decryption of subsequent messages.
- Security 0 is simply plain text communication. In this case the `pop` is simply ignored.

See *Unified Provisioning* for details about the security features.

```
const char *service_name = "my_device";
const char *service_key  = "password";

wifi_prov_security_t security = WIFI_PROV_SECURITY_1;
const char *pop = "abcd1234";

ESP_ERROR_CHECK( wifi_prov_mgr_start_provisioning(security, pop, service_
↪name, service_key) );
```

The provisioning service automatically finishes only if it receives valid Wi-Fi AP credentials followed by successful connection of device to the AP with IP obtained. Regardless of that, the provisioning service can be stopped at any moment by making a call to *`wifi_prov_mgr_stop_provisioning()`*.

Note: If the device fails to connect with the provided credentials, it does not accept new credentials anymore, but the provisioning service keeps on running, only to convey failure to the client, until the device is restarted. Upon restart, the provisioning state turns out to be true this time, as credentials are found in NVS, but the device does fail again to connect with those same credentials, unless an AP with the matching credentials somehow does become available. This situation can be fixed by resetting the credentials in NVS or force starting the provisioning service. This has been explained above in *Check the Provisioning State*.

Waiting for Completion Typically, the main application waits for the provisioning to finish, then de-initializes the manager to free up resources, and finally starts executing its own logic.

There are two ways for making this possible. The simpler way is to use a blocking call to *`wifi_prov_mgr_wait()`*.

```
// Start provisioning service
ESP_ERROR_CHECK( wifi_prov_mgr_start_provisioning(security, pop, service_
↪name, service_key) );

// Wait for service to complete
wifi_prov_mgr_wait();

// Finally de-initialize the manager
wifi_prov_mgr_deinit();
```

The other way is to use the default event loop handler to catch `WIFI_PROV_EVENT` and call *`wifi_prov_mgr_deinit()`* when event ID is `WIFI_PROV_END`:

```
static void event_handler(void* arg, esp_event_base_t event_base,
                          int event_id, void* event_data)
{
    if (event_base == WIFI_PROV_EVENT && event_id == WIFI_PROV_END) {
        /* De-initialize the manager once the provisioning is finished */
```

(continues on next page)

(continued from previous page)

```
        wifi_prov_mgr_deinit();
    }
}
```

User Side Implementation When the service is started, the device to be provisioned is identified by the advertised service name, which, depending upon the selected transport, is either the Bluetooth LE device name or the SoftAP SSID.

When using SoftAP transport, for allowing service discovery, mDNS must be initialized before starting provisioning. In this case, the host name set by the main application is used, and the service type is internally set to `_esp_wifi_prov`.

When using Bluetooth LE transport, a custom 128-bit UUID should be set using *`wifi_prov_scheme_ble_set_service_uuid()`*. This UUID is to be included in the Bluetooth LE advertisement and corresponds to the primary GATT service that provides provisioning endpoints as GATT characteristics. Each GATT characteristic is formed using the primary service UUID as the base, with different auto-assigned 12th and 13th bytes, presumably counting from the 0th byte. Since an endpoint characteristic UUID is auto-assigned, it should not be used to identify the endpoint. Instead, client-side applications should identify the endpoints by reading the User Characteristic Description (`0x2901`) descriptor for each characteristic, which contains the endpoint name of the characteristic. For example, if the service UUID is set to `55cc035e-fb27-4f80-be02-3c60828b7451`, each endpoint characteristic is assigned a UUID like `55cc____-fb27-4f80-be02-3c60828b7451`, with unique values at the 12th and 13th bytes.

Once connected to the device, the provisioning-related protocomm endpoints can be identified as follows:

Table 9: Endpoints Provided by the Provisioning Service

Endpoint Name i.e., Bluetooth LE + GATT Server	URI, i.e., SoftAP + HTTP Server + mDNS	Description
prov-session	http://<mdns-hostname>.local/prov-session	Security endpoint used for session establishment
prov-scan	http://wifi-prov.local/prov-scan	the endpoint used for starting Wi-Fi scan and receiving scan results
prov-ctrl	http://wifi-prov.local/prov-ctrl	the endpoint used for controlling Wi-Fi provisioning state
prov-config	http://<mdns-hostname>.local/prov-config	the endpoint used for configuring Wi-Fi credentials on device
proto-ver	http://<mdns-hostname>.local/proto-ver	the endpoint for retrieving version info

Immediately after connecting, the client application may fetch the version/capabilities information from the `proto-ver` endpoint. All communications to this endpoint are unencrypted, hence necessary information, which may be relevant for deciding compatibility, can be retrieved before establishing a secure session. The response is in JSON format and looks like : `prov: { ver: v1.1, cap: [no_pop] }, my_app: { ver: 1.345, cap: [cloud, local_ctrl] },....` Here label `prov` provides provisioning service version `ver` and capabilities `cap`. For now, only the `no_pop` capability is supported, which indicates that the service does not require proof of possession for authentication. Any application-related version or capabilities are given by other labels, e.g., `my_app` in this example. These additional fields are set using *`wifi_prov_mgr_set_app_info()`*.

User side applications need to implement the signature handshaking required for establishing and authenticating secure protocomm sessions as per the security scheme configured for use, which is not needed when the manager is configured to use protocomm security 0.

See *Unified Provisioning* for more details about the secure handshake and encryption used. Applications must use the `.proto` files found under protocomm/proto, which define the Protobuf message structures supported by `prov-session` endpoint.

Once a session is established, Wi-Fi credentials are configured using the following set of `wifi_config` commands, serialized as Protobuf messages with the corresponding `.proto` files that can be found under wifi_provisioning/proto:

- `get_status` - For querying the Wi-Fi connection status. The device responds with a status which is one of connecting, connected or disconnected. If the status is disconnected, a disconnection reason is also to be included in the status response.
- `set_config` - For setting the Wi-Fi connection credentials.
- `apply_config` - For applying the credentials saved during `set_config` and starting the Wi-Fi station.

After session establishment, the client can also request Wi-Fi scan results from the device. The results returned is a list of AP SSIDs, sorted in descending order of signal strength. This allows client applications to display APs nearby to the device at the time of provisioning, and users can select one of the SSIDs and provide the password which is then sent using the `wifi_config` commands described above. The `wifi_scan` endpoint supports the following protobuf commands :

- `scan_start` - For starting Wi-Fi scan with various options:
  - `blocking` (input) - If true, the command returns only when the scanning is finished.
  - `passive` (input) - If true, the scan is started in passive mode, which may be slower, instead of active mode.
  - `group_channels` (input) - This specifies whether to scan all channels in one go when zero, or perform scanning of channels in groups, with 120 ms delay between scanning of consecutive groups, and the value of this parameter sets the number of channels in each group. This is useful when transport mode is SoftAP, where scanning all channels in one go may not give the Wi-Fi driver enough time to send out beacons, and hence may cause disconnection with any connected stations. When scanning in groups, the manager waits for at least 120 ms after completing the scan on a group of channels, and thus allows the driver to send out the beacons. For example, given that the total number of Wi-Fi channels is 14, then setting `group_channels` to 3 creates 5 groups, with each group having 3 channels, except the last one which has 14 % 3 = 2 channels. So, when the scan is started, the first 3 channels will be scanned, followed by a 120 ms delay, and then the next 3 channels, and so on, until all the 14 channels have been scanned.One may need to adjust this parameter as having only a few channels in a group may increase the overall scan time, while having too many may again cause disconnection. Usually, a value of 4 should work for most cases. Note that for any other mode of transport, e.g., Bluetooth LE, this can be safely set to 0, and hence achieve the shortest overall scanning time.
  - `period_ms` (input) - The scan parameter specifying how long to wait on each channel.
- `scan_status` - It gives the status of scanning process:
  - `scan_finished` (output) - When the scan has finished, this returns true.
  - `result_count` (output) - This gives the total number of results obtained till now. If the scan is yet happening, this number keeps on updating.
- `scan_result` - For fetching the scan results. This can be called even if the scan is still on going.
  - `start_index` (input) - Where the index starts from to fetch the entries from the results list.
  - `count` (input) - The number of entries to fetch from the starting index.
  - `entries` (output) - The list of entries returned. Each entry consists of `ssid`, `channel` and `rssi` information.

The client can also control the provisioning state of the device using `wifi_ctrl` endpoint. The `wifi_ctrl` endpoint supports the following protobuf commands:

- `ctrl_reset` - Resets internal state machine of the device and clears provisioned credentials only in case of provisioning failures.
- `ctrl_reprov` - Resets internal state machine of the device and clears provisioned credentials only in case the device is to be provisioned again for new credentials after a previous successful provisioning.

Additional Endpoints In case users want to have some additional protocomm endpoints customized to their requirements, this is done in two steps. First is creation of an endpoint with a specific name, and the second step is the registration of a handler for this endpoint. See *Protocol Communication* for the function signature of an endpoint handler. A custom endpoint must be created after initialization and before starting the provisioning service. Whereas, the protocomm handler is registered for this endpoint only after starting the provisioning service. Note that in the custom endpoint handler function, memory for the response of such protocomm endpoints should be allocated using heap as it gets freed by the protocomm layer once it has been sent by the transport layer.

```
wifi_prov_mgr_init(config);
wifi_prov_mgr_endpoint_create("custom-endpoint");
wifi_prov_mgr_start_provisioning(security, pop, service_name, service_
↪key);
wifi_prov_mgr_endpoint_register("custom-endpoint", custom_ep_handler,␣
↪custom_ep_data);
```

When the provisioning service stops, the endpoint is unregistered automatically.

One can also choose to call *wifi_prov_mgr_endpoint_unregister()* to manually deactivate an endpoint at runtime. This can also be used to deactivate the internal endpoints used by the provisioning service.

When/How to Stop the Provisioning Service? The default behavior is that once the device successfully connects using the Wi-Fi credentials set by the `apply_config` command, the provisioning service stops, and Bluetooth LE or SoftAP turns off, automatically after responding to the next `get_status` command. If `get_status` command is not received by the device, the service stops after a 30 s timeout.

On the other hand, if device is not able to connect using the provided Wi-Fi credentials, due to incorrect SSID or passphrase, the service keeps running, and `get_status` keeps responding with disconnected status and reason for disconnection. Any further attempts to provide another set of Wi-Fi credentials, are to be rejected. These credentials are preserved, unless the provisioning service is force started, or NVS erased.

If this default behavior is not desired, it can be disabled by calling *wifi_prov_mgr_disable_auto_stop()*. Now the provisioning service stops only after an explicit call to *wifi_prov_mgr_stop_provisioning()*, which returns immediately after scheduling a task for stopping the service. The service stops after a certain delay and `WIFI_PROV_END` event gets emitted. This delay is specified by the argument to *wifi_prov_mgr_disable_auto_stop()*.

The customized behavior is useful for applications which want the provisioning service to be stopped some time after the Wi-Fi connection is successfully established. For example, if the application requires the device to connect to some cloud service and obtain another set of credentials, and exchange these credentials over a custom protocomm endpoint, then after successfully doing so, stop the provisioning service by calling *wifi_prov_mgr_stop_provisioning()* inside the protocomm handler itself. The right amount of delay ensures that the transport resources are freed only after the response from the protocomm handler reaches the client side application.

Application Examples

For complete example implementation see provisioning/wifi_prov_mgr.

Provisioning Tools

Provisioning applications are available for various platforms, along with source code:

- **Android:**
  - Bluetooth LE Provisioning app on Play Store.
  - SoftAP Provisioning app on Play Store.
  - Source code on GitHub: esp-idf-provisioning-android.
- **iOS:**
  - Bluetooth LE Provisioning app on App Store.
  - SoftAP Provisioning app on App Store.
  - Source code on GitHub: esp-idf-provisioning-ios.
- Linux/MacOS/Windows: tools/esp_prov, a Python-based command-line tool for provisioning.

The phone applications offer simple UI and are thus more user centric, while the command-line application is useful as a debugging tool for developers.

API Reference

Header File

- components/wifi_provisioning/include/wifi_provisioning/manager.h
- This header file can be included with:

```
#include "wifi_provisioning/manager.h"
```

- This header file is a part of the API provided by the `wifi_provisioning` component. To declare that your component depends on `wifi_provisioning`, add the following to your CMakeLists.txt:

```
REQUIRES wifi_provisioning
```

or

```
PRIV_REQUIRES wifi_provisioning
```

Functions

esp_err_t **wifi_prov_mgr_init** (*wifi_prov_mgr_config_t* config)

Initialize provisioning manager instance.

Configures the manager and allocates internal resources

Configuration specifies the provisioning scheme (transport) and event handlers

Event WIFI_PROV_INIT is emitted right after initialization is complete

Parameters **config** -- **[in]** Configuration structure

Returns

- ESP_OK : Success
- ESP_FAIL : Fail

void **wifi_prov_mgr_deinit** (void)

Stop provisioning (if running) and release resource used by the manager.

Event WIFI_PROV_DEINIT is emitted right after de-initialization is finished

If provisioning service is still active when this API is called, it first stops the service, hence emitting WIFI_PROV_END, and then performs the de-initialization

esp_err_t **wifi_prov_mgr_is_provisioned** (bool *provisioned)

Checks if device is provisioned.

This checks if Wi-Fi credentials are present on the NVS

The Wi-Fi credentials are assumed to be kept in the same NVS namespace as used by esp_wifi component

If one were to call esp_wifi_set_config() directly instead of going through the provisioning process, this function will still yield true (i.e. device will be found to be provisioned)

Note: Calling wifi_prov_mgr_start_provisioning() automatically resets the provision state, irrespective of what the state was prior to making the call.

Parameters **provisioned** -- **[out]** True if provisioned, else false

Returns

- ESP_OK : Retrieved provision state successfully
- ESP_FAIL : Wi-Fi not initialized
- ESP_ERR_INVALID_ARG : Null argument supplied

bool **wifi_prov_mgr_is_sm_idle**(void)

Checks whether the provisioning state machine is idle.

Returns True if state machine is idle, else false

esp_err_t **wifi_prov_mgr_start_provisioning**(*wifi_prov_security_t* security, const void *wifi_prov_sec_params, const char *service_name, const char *service_key)

Start provisioning service.

This starts the provisioning service according to the scheme configured at the time of initialization. For scheme :

- wifi_prov_scheme_ble : This starts protocomm_ble, which internally initializes BLE transport and starts GATT server for handling provisioning requests
- wifi_prov_scheme_softap : This activates SoftAP mode of Wi-Fi and starts protocomm_httpd, which internally starts an HTTP server for handling provisioning requests (If mDNS is active it also starts advertising service with type _esp_wifi_prov._tcp)

Event WIFI_PROV_START is emitted right after provisioning starts without failure

---

Note: This API will start provisioning service even if device is found to be already provisioned, i.e. wifi_prov_mgr_is_provisioned() yields true

---

Parameters

- **security** -- **[in]** Specify which protocomm security scheme to use :
  - WIFI_PROV_SECURITY_0 : For no security
  - WIFI_PROV_SECURITY_1 : x25519 secure handshake for session establishment followed by AES-CTR encryption of provisioning messages
  - WIFI_PROV_SECURITY_2: SRP6a based authentication and key exchange followed by AES-GCM encryption/decryption of provisioning messages
- **wifi_prov_sec_params** -- **[in]** Pointer to security params (NULL if not needed). This is not needed for protocomm security 0 This pointer should hold the struct of type wifi_prov_security1_params_t for protocomm security 1 and wifi_prov_security2_params_t for protocomm security 2 respectively. This pointer and its contents should be valid till the provisioning service is running and has not been stopped or de-inited.
- **service_name** -- **[in]** Unique name of the service. This translates to:
  - Wi-Fi SSID when provisioning mode is softAP
  - Device name when provisioning mode is BLE
- **service_key** -- **[in]** Key required by client to access the service (NULL if not needed). This translates to:
  - Wi-Fi password when provisioning mode is softAP
  - ignored when provisioning mode is BLE

Returns

- ESP_OK : Provisioning started successfully
- ESP_FAIL : Failed to start provisioning service
- ESP_ERR_INVALID_STATE : Provisioning manager not initialized or already started

void **wifi_prov_mgr_stop_provisioning**(void)

Stop provisioning service.

If provisioning service is active, this API will initiate a process to stop the service and return. Once the service actually stops, the event WIFI_PROV_END will be emitted.

If wifi_prov_mgr_deinit() is called without calling this API first, it will automatically stop the provisioning service and emit the WIFI_PROV_END, followed by WIFI_PROV_DEINIT, before returning.

This API will generally be used along with wifi_prov_mgr_disable_auto_stop() in the scenario when the main application has registered its own endpoints, and wishes that the provisioning service is stopped only when

some protocomm command from the client side application is received.

Calling this API inside an endpoint handler, with sufficient cleanup_delay, will allow the response / acknowledgment to be sent successfully before the underlying protocomm service is stopped.

Cleaup_delay is set when calling wifi_prov_mgr_disable_auto_stop(). If not specified, it defaults to 1000ms.

For straightforward cases, using this API is usually not necessary as provisioning is stopped automatically once WIFI_PROV_CRED_SUCCESS is emitted. Stopping is delayed (maximum 30 seconds) thus allowing the client side application to query for Wi-Fi state, i.e. after receiving the first query and sending `Wi-Fi state connected` response the service is stopped immediately.

void **wifi_prov_mgr_wait**(void)

Wait for provisioning service to finish.

Calling this API will block until provisioning service is stopped i.e. till event WIFI_PROV_END is emitted.

This will not block if provisioning is not started or not initialized.

esp_err_t **wifi_prov_mgr_disable_auto_stop**(uint32_t cleanup_delay)

Disable auto stopping of provisioning service upon completion.

By default, once provisioning is complete, the provisioning service is automatically stopped, and all endpoints (along with those registered by main application) are deactivated.

This API is useful in the case when main application wishes to close provisioning service only after it receives some protocomm command from the client side app. For example, after connecting to Wi-Fi, the device may want to connect to the cloud, and only once that is successfully, the device is said to be fully configured. But, then it is upto the main application to explicitly call wifi_prov_mgr_stop_provisioning() later when the device is fully configured and the provisioning service is no longer required.

Note: This must be called before executing wifi_prov_mgr_start_provisioning()

Parameters **cleanup_delay** -- **[in]** Sets the delay after which the actual cleanup of transport related resources is done after a call to wifi_prov_mgr_stop_provisioning() returns. Minimum allowed value is 100ms. If not specified, this will default to 1000ms.

Returns

- ESP_OK : Success
- ESP_ERR_INVALID_STATE : Manager not initialized or provisioning service already started

esp_err_t **wifi_prov_mgr_set_app_info**(const char *label, const char *version, const char **capabilities, size_t total_capabilities)

Set application version and capabilities in the JSON data returned by proto-ver endpoint.

This function can be called multiple times, to specify information about the various application specific services running on the device, identified by unique labels.

The provisioning service itself registers an entry in the JSON data, by the label "prov", containing only provisioning service version and capabilities. Application services should use a label other than "prov" so as not to overwrite this.

Note: This must be called before executing wifi_prov_mgr_start_provisioning()

Parameters

- **label** -- **[in]** String indicating the application name.
- **version** -- **[in]** String indicating the application version. There is no constraint on format.
- **capabilities** -- **[in]** Array of strings with capabilities. These could be used by the client side app to know the application registered endpoint capabilities

- **total_capabilities** -- **[in]** Size of capabilities array

Returns

- ESP_OK : Success
- ESP_ERR_INVALID_STATE : Manager not initialized or provisioning service already started
- ESP_ERR_NO_MEM : Failed to allocate memory for version string
- ESP_ERR_INVALID_ARG : Null argument

esp_err_t **wifi_prov_mgr_endpoint_create**(const char *ep_name)

Create an additional endpoint and allocate internal resources for it.

This API is to be called by the application if it wants to create an additional endpoint. All additional endpoints will be assigned UUIDs starting from 0xFF54 and so on in the order of execution.

protocomm handler for the created endpoint is to be registered later using wifi_prov_mgr_endpoint_register() after provisioning has started.

Note: This API can only be called BEFORE provisioning is started

Note: Additional endpoints can be used for configuring client provided parameters other than Wi-Fi credentials, that are necessary for the main application and hence must be set prior to starting the application

Note: After session establishment, the additional endpoints must be targeted first by the client side application before sending Wi-Fi configuration, because once Wi-Fi configuration finishes the provisioning service is stopped and hence all endpoints are unregistered

Parameters **ep_name** -- **[in]** unique name of the endpoint

Returns

- ESP_OK : Success
- ESP_FAIL : Failure

esp_err_t **wifi_prov_mgr_endpoint_register**(const char *ep_name, *protocomm_req_handler_t* handler, void *user_ctx)

Register a handler for the previously created endpoint.

This API can be called by the application to register a protocomm handler to any endpoint that was created using wifi_prov_mgr_endpoint_create().

Note: This API can only be called AFTER provisioning has started

Note: Additional endpoints can be used for configuring client provided parameters other than Wi-Fi credentials, that are necessary for the main application and hence must be set prior to starting the application

Note: After session establishment, the additional endpoints must be targeted first by the client side application before sending Wi-Fi configuration, because once Wi-Fi configuration finishes the provisioning service is stopped and hence all endpoints are unregistered

Parameters

- **ep_name** -- **[in]** Name of the endpoint
- **handler** -- **[in]** Endpoint handler function

- **user_ctx** -- **[in]** User data

Returns

- ESP_OK : Success
- ESP_FAIL : Failure

void **wifi_prov_mgr_endpoint_unregister**(const char *ep_name)

Unregister the handler for an endpoint.

This API can be called if the application wants to selectively unregister the handler of an endpoint while the provisioning is still in progress.

All the endpoint handlers are unregistered automatically when the provisioning stops.

Parameters **ep_name** -- **[in]** Name of the endpoint

esp_err_t **wifi_prov_mgr_get_wifi_state**(*wifi_prov_sta_state_t* *state)

Get state of Wi-Fi Station during provisioning.

Parameters **state** -- **[out]** Pointer to wifi_prov_sta_state_t variable to be filled

Returns

- ESP_OK : Successfully retrieved Wi-Fi state
- ESP_FAIL : Provisioning app not running

esp_err_t **wifi_prov_mgr_get_wifi_disconnect_reason**(*wifi_prov_sta_fail_reason_t* *reason)

Get reason code in case of Wi-Fi station disconnection during provisioning.

Parameters **reason** -- **[out]** Pointer to wifi_prov_sta_fail_reason_t variable to be filled

Returns

- ESP_OK : Successfully retrieved Wi-Fi disconnect reason
- ESP_FAIL : Provisioning app not running

esp_err_t **wifi_prov_mgr_configure_sta**(wifi_config_t *wifi_cfg)

Runs Wi-Fi as Station with the supplied configuration.

Configures the Wi-Fi station mode to connect to the AP with SSID and password specified in config structure and sets Wi-Fi to run as station.

This is automatically called by provisioning service upon receiving new credentials.

If credentials are to be supplied to the manager via a different mode other than through protocomm, then this API needs to be called.

Event WIFI_PROV_CRED_RECV is emitted after credentials have been applied and Wi-Fi station started

Parameters **wifi_cfg** -- **[in]** Pointer to Wi-Fi configuration structure

Returns

- ESP_OK : Wi-Fi configured and started successfully
- ESP_FAIL : Failed to set configuration

esp_err_t **wifi_prov_mgr_reset_provisioning**(void)

Reset Wi-Fi provisioning config.

Calling this API will restore WiFi stack persistent settings to default values.

Returns

- ESP_OK : Reset provisioning config successfully
- ESP_FAIL : Failed to reset provisioning config

esp_err_t **wifi_prov_mgr_reset_sm_state_on_failure**(void)

Reset internal state machine and clear provisioned credentials.

This API should be used to restart provisioning ONLY in the case of provisioning failures without rebooting the device.

Returns

- ESP_OK : Reset provisioning state machine successfully

- ESP_FAIL : Failed to reset provisioning state machine
- ESP_ERR_INVALID_STATE : Manager not initialized

esp_err_t **wifi_prov_mgr_reset_sm_state_for_reprovision** (void)

Reset internal state machine and clear provisioned credentials.

This API can be used to restart provisioning ONLY in case the device is to be provisioned again for new credentials after a previous successful provisioning without rebooting the device.

Note: This API can be used only if provisioning auto-stop has been disabled using wifi_prov_mgr_disable_auto_stop()

Returns

- ESP_OK : Reset provisioning state machine successfully
- ESP_FAIL : Failed to reset provisioning state machine
- ESP_ERR_INVALID_STATE : Manager not initialized

Structures

struct **wifi_prov_event_handler_t**

Event handler that is used by the manager while provisioning service is active.

Public Members

wifi_prov_cb_func_t **event_cb**

Callback function to be executed on provisioning events

void ***user_data**

User context data to pass as parameter to callback function

struct **wifi_prov_scheme**

Structure for specifying the provisioning scheme to be followed by the manager.

Note: Ready to use schemes are available:

- wifi_prov_scheme_ble : for provisioning over BLE transport + GATT server
- wifi_prov_scheme_softap : for provisioning over SoftAP transport + HTTP server
- wifi_prov_scheme_console : for provisioning over Serial UART transport + Console (for debugging)

Public Members

esp_err_t (***prov_start**)(*protocomm_t* *pc, void *config)

Function which is to be called by the manager when it is to start the provisioning service associated with a protocomm instance and a scheme specific configuration

esp_err_t (***prov_stop**)(*protocomm_t* *pc)

Function which is to be called by the manager to stop the provisioning service previously associated with a protocomm instance

void *(***new_config**)(void)

Function which is to be called by the manager to generate a new configuration for the provisioning service, that is to be passed to *prov_start()*

void (***delete_config**)(void *config)

Function which is to be called by the manager to delete a configuration generated using *new_config()*

esp_err_t (***set_config_service**)(void *config, const char *service_name, const char *service_key)

Function which is to be called by the manager to set the service name and key values in the configuration structure

esp_err_t (***set_config_endpoint**)(void *config, const char *endpoint_name, uint16_t uuid)

Function which is to be called by the manager to set a protocomm endpoint with an identifying name and UUID in the configuration structure

wifi_mode_t **wifi_mode**

Sets mode of operation of Wi-Fi during provisioning This is set to :

- WIFI_MODE_APSTA for SoftAP transport
- WIFI_MODE_STA for BLE transport

struct **wifi_prov_mgr_config_t**

Structure for specifying the manager configuration.

Public Members

wifi_prov_scheme_t **scheme**

Provisioning scheme to use. Following schemes are already available:

- wifi_prov_scheme_ble : for provisioning over BLE transport + GATT server
- wifi_prov_scheme_softap : for provisioning over SoftAP transport + HTTP server + mDNS (optional)
- wifi_prov_scheme_console : for provisioning over Serial UART transport + Console (for debugging)

wifi_prov_event_handler_t **scheme_event_handler**

Event handler required by the scheme for incorporating scheme specific behavior while provisioning manager is running. Various options may be provided by the scheme for setting this field. Use WIFI_PROV_EVENT_HANDLER_NONE when not used. When using scheme wifi_prov_scheme_ble, the following options are available:

- WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM
- WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BLE
- WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BT

wifi_prov_event_handler_t **app_event_handler**

Event handler that can be set for the purpose of incorporating application specific behavior. Use WIFI_PROV_EVENT_HANDLER_NONE when not used.

Macros

WIFI_PROV_EVENT_HANDLER_NONE

Event handler can be set to none if not used.

Type Definitions

typedef void (*`wifi_prov_cb_func_t`)(void *user_data, *wifi_prov_cb_event_t* event, void *event_data)

typedef struct *wifi_prov_scheme* `wifi_prov_scheme_t`

Structure for specifying the provisioning scheme to be followed by the manager.

Note: Ready to use schemes are available:

- wifi_prov_scheme_ble : for provisioning over BLE transport + GATT server
- wifi_prov_scheme_softap : for provisioning over SoftAP transport + HTTP server
- wifi_prov_scheme_console : for provisioning over Serial UART transport + Console (for debugging)

---

typedef enum *wifi_prov_security* `wifi_prov_security_t`

Security modes supported by the Provisioning Manager.

These are same as the security modes provided by protocomm

typedef *protocomm_security2_params_t* `wifi_prov_security2_params_t`

Security 2 params structure This needs to be passed when using WIFI_PROV_SECURITY_2.

Enumerations

enum `wifi_prov_cb_event_t`

Events generated by manager.

These events are generated in order of declaration and, for the stretch of time between initialization and de-initialization of the manager, each event is signaled only once

Values:

enumerator `WIFI_PROV_INIT`

Emitted when the manager is initialized

enumerator `WIFI_PROV_START`

Indicates that provisioning has started

enumerator `WIFI_PROV_CRED_RECV`

Emitted when Wi-Fi AP credentials are received via `protocomm` endpoint `wifi_config`. The event data in this case is a pointer to the corresponding `wifi_sta_config_t` structure

enumerator `WIFI_PROV_CRED_FAIL`

Emitted when device fails to connect to the AP of which the credentials were received earlier on event `WIFI_PROV_CRED_RECV`. The event data in this case is a pointer to the disconnection reason code with type `wifi_prov_sta_fail_reason_t`

enumerator `WIFI_PROV_CRED_SUCCESS`

Emitted when device successfully connects to the AP of which the credentials were received earlier on event `WIFI_PROV_CRED_RECV`

enumerator `WIFI_PROV_END`

Signals that provisioning service has stopped

enumerator **WIFI_PROV_DEINIT**

Signals that manager has been de-initialized

enum **wifi_prov_security**

Security modes supported by the Provisioning Manager.

These are same as the security modes provided by protocomm

Values:

enumerator **WIFI_PROV_SECURITY_0**

No security (plain-text communication)

enumerator **WIFI_PROV_SECURITY_1**

This secure communication mode consists of X25519 key exchange

- proof of possession (pop) based authentication
- AES-CTR encryption

enumerator **WIFI_PROV_SECURITY_2**

This secure communication mode consists of SRP6a based authentication and key exchange

- AES-GCM encryption/decryption

Header File

- components/wifi_provisioning/include/wifi_provisioning/scheme_ble.h
- This header file can be included with:

```
#include "wifi_provisioning/scheme_ble.h"
```

- This header file is a part of the API provided by the `wifi_provisioning` component. To declare that your component depends on `wifi_provisioning`, add the following to your CMakeLists.txt:

```
REQUIRES wifi_provisioning
```

or

```
PRIV_REQUIRES wifi_provisioning
```

Functions

void **wifi_prov_scheme_ble_event_cb_free_btdm**(void *user_data, *wifi_prov_cb_event_t* event, void *event_data)

void **wifi_prov_scheme_ble_event_cb_free_ble**(void *user_data, *wifi_prov_cb_event_t* event, void *event_data)

void **wifi_prov_scheme_ble_event_cb_free_bt**(void *user_data, *wifi_prov_cb_event_t* event, void *event_data)

esp_err_t **wifi_prov_scheme_ble_set_service_uuid**(uint8_t *uuid128)

Set the 128 bit GATT service UUID used for provisioning.

This API is used to override the default 128 bit provisioning service UUID, which is 0000ffff-0000-1000-8000-00805f9b34fb.

This must be called before starting provisioning, i.e. before making a call to wifi_prov_mgr_start_provisioning(), otherwise the default UUID will be used.

Note: The data being pointed to by the argument must be valid at least till provisioning is started. Upon start, the manager will store an internal copy of this UUID, and this data can be freed or invalidated afterwards.

Parameters **uuid128** -- **[in]** A custom 128 bit UUID

Returns

- ESP_OK : Success
- ESP_ERR_INVALID_ARG : Null argument

esp_err_t **wifi_prov_scheme_ble_set_mfg_data**(uint8_t *mfg_data, ssize_t mfg_data_len)

Set manufacturer specific data in scan response.

This must be called before starting provisioning, i.e. before making a call to wifi_prov_mgr_start_provisioning().

Note: It is important to understand that length of custom manufacturer data should be within limits. The manufacturer data goes into scan response along with BLE device name. By default, BLE device name length is of 11 Bytes, however it can vary as per application use case. So, one has to honour the scan response data size limits i.e. (mfg_data_len + 2) < 31 - (device_name_length + 2). If the mfg_data length exceeds this limit, the length will be truncated.

Parameters

- **mfg_data** -- **[in]** Custom manufacturer data
- **mfg_data_len** -- **[in]** Manufacturer data length

Returns

- ESP_OK : Success
- ESP_ERR_INVALID_ARG : Null argument

esp_err_t **wifi_prov_scheme_ble_set_random_addr**(const uint8_t *rand_addr)

Set Bluetooth Random address.

This must be called before starting provisioning, i.e. before making a call to wifi_prov_mgr_start_provisioning().

This API can be used in cases where a new identity address is to be used during provisioning. This will result in this device being treated as a new device by remote devices.

This API is only to be called to set random address. Re-invoking this API after provisioning is started will have no effect.

Note: This API will change the existing BD address for the device. The address once set will remain unchanged until BLE stack tear down happens when wifi_prov_mgr_deinit is invoked.

Parameters **rand_addr** -- **[in]** The static random address to be set of length 6 bytes.

Returns

- ESP_OK : Success
- ESP_ERR_INVALID_ARG : Null argument

Macros

WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM

WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BLE

WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BT

Header File

- components/wifi_provisioning/include/wifi_provisioning/scheme_softap.h
- This header file can be included with:

```
#include "wifi_provisioning/scheme_softap.h"
```

- This header file is a part of the API provided by the `wifi_provisioning` component. To declare that your component depends on `wifi_provisioning`, add the following to your CMakeLists.txt:

```
REQUIRES wifi_provisioning
```

or

```
PRIV_REQUIRES wifi_provisioning
```

Functions

void **wifi_prov_scheme_softap_set_httpd_handle**(void *handle)

Provide HTTPD Server handle externally.

Useful in cases wherein applications need the webserver for some different operations, and do not want the wifi provisioning component to start/stop a new instance.

Note: This API should be called before wifi_prov_mgr_start_provisioning()

Parameters **handle** -- **[in]** Handle to HTTPD server instance

Header File

- components/wifi_provisioning/include/wifi_provisioning/scheme_console.h
- This header file can be included with:

```
#include "wifi_provisioning/scheme_console.h"
```

- This header file is a part of the API provided by the `wifi_provisioning` component. To declare that your component depends on `wifi_provisioning`, add the following to your CMakeLists.txt:

```
REQUIRES wifi_provisioning
```

or

```
PRIV_REQUIRES wifi_provisioning
```

Header File

- components/wifi_provisioning/include/wifi_provisioning/wifi_config.h
- This header file can be included with:

```
#include "wifi_provisioning/wifi_config.h"
```

- This header file is a part of the API provided by the `wifi_provisioning` component. To declare that your component depends on `wifi_provisioning`, add the following to your CMakeLists.txt:

```
REQUIRES wifi_provisioning
```

or

```
PRIV_REQUIRES wifi_provisioning
```

Functions

esp_err_t **wifi_prov_config_data_handler**(uint32_t session_id, const uint8_t *inbuf, ssize_t inlen, uint8_t **outbuf, ssize_t *outlen, void *priv_data)

Handler for receiving and responding to requests from master.

This is to be registered as the `wifi_config` endpoint handler (protocomm `protocomm_req_handler_t`) using `protocomm_add_endpoint()`

Structures

struct **wifi_prov_sta_conn_info_t**

WiFi STA connected status information.

Public Members

char **ip_addr**[IP4ADDR_STRLEN_MAX]

IP Address received by station

char **bssid**[6]

BSSID of the AP to which connection was estalished

char **ssid**[33]

SSID of the to which connection was estalished

uint8_t **channel**

Channel of the AP

uint8_t **auth_mode**

Authorization mode of the AP

struct **wifi_prov_config_get_data_t**

WiFi status data to be sent in response to `get_status` request from master.

Public Members

wifi_prov_sta_state_t **wifi_state**

WiFi state of the station

wifi_prov_sta_fail_reason_t **fail_reason**

Reason for disconnection (valid only when `wifi_state` is `WIFI_STATION_DISCONNECTED`)

wifi_prov_sta_conn_info_t **conn_info**

Connection information (valid only when `wifi_state` is `WIFI_STATION_CONNECTED`)

struct **wifi_prov_config_set_data_t**

WiFi config data received by slave during `set_config` request from master.

Public Members

char **ssid**[33]

SSID of the AP to which the slave is to be connected

char **password**[64]

Password of the AP

char **bssid**[6]

BSSID of the AP

uint8_t **channel**

Channel of the AP

struct **wifi_prov_config_handlers**

Internal handlers for receiving and responding to protocomm requests from master.

This is to be passed as priv_data for protocomm request handler (refer to `wifi_prov_config_data_handler()`) when calling `protocomm_add_endpoint()`.

Public Members

esp_err_t (***get_status_handler**)(*wifi_prov_config_get_data_t* *resp_data, *wifi_prov_ctx_t* **ctx)

Handler function called when connection status of the slave (in WiFi station mode) is requested

esp_err_t (***set_config_handler**)(const *wifi_prov_config_set_data_t* *req_data, *wifi_prov_ctx_t* **ctx)

Handler function called when WiFi connection configuration (eg. AP SSID, password, etc.) of the slave (in WiFi station mode) is to be set to user provided values

esp_err_t (***apply_config_handler**)(*wifi_prov_ctx_t* **ctx)

Handler function for applying the configuration that was set in `set_config_handler`. After applying the station may get connected to the AP or may fail to connect. The slave must be ready to convey the updated connection status information when `get_status_handler` is invoked again by the master.

wifi_prov_ctx_t ***ctx**

Context pointer to be passed to above handler functions upon invocation

Type Definitions

typedef struct wifi_prov_ctx **wifi_prov_ctx_t**

Type of context data passed to each get/set/apply handler function set in *`wifi_prov_config_handlers`* structure.

This is passed as an opaque pointer, thereby allowing it be defined later in application code as per requirements.

typedef struct *wifi_prov_config_handlers* **wifi_prov_config_handlers_t**

Internal handlers for receiving and responding to protocomm requests from master.

This is to be passed as priv_data for protocomm request handler (refer to `wifi_prov_config_data_handler()`) when calling `protocomm_add_endpoint()`.

Enumerations

enum **wifi_prov_sta_state_t**

WiFi STA status for conveying back to the provisioning master.

Values:

enumerator **WIFI_PROV_STA_CONNECTING**

enumerator **WIFI_PROV_STA_CONNECTED**

enumerator **WIFI_PROV_STA_DISCONNECTED**

enum **wifi_prov_sta_fail_reason_t**

WiFi STA connection fail reason.

Values:

enumerator **WIFI_PROV_STA_AUTH_ERROR**

enumerator **WIFI_PROV_STA_AP_NOT_FOUND**

Code examples for above API are provided in the provisioning directory of ESP-IDF examples.

Code example for above API is provided in wifi/smart_config.

Code example for above API is provided in wifi/wifi_easy_connect/dpp-enrollee.

2.9 Storage API

This section contains reference of the high-level storage APIs. They are based on low-level drivers such as SPI flash, SD/MMC.

- *Partitions API* allow block based access to SPI flash according to the *Partition Tables*.
- *Non-Volatile Storage library (NVS)* implements a fault-tolerant wear-levelled key-value storage in SPI NOR flash.
- *Virtual File System (VFS)* library provides an interface for registration of file system drivers. SPIFFS, FAT and various other file system libraries are based on the VFS.
- *SPIFFS* is a wear-levelled file system optimized for SPI NOR flash, well suited for small partition sizes and low throughput
- *FAT* is a standard file system which can be used in SPI flash or on SD/MMC cards
- *Wear Levelling* library implements a flash translation layer (FTL) suitable for SPI NOR flash. It is used as a container for FAT partitions in flash.

Note: It is suggested to use high-level APIs (`esp_partition` or file system) instead of low-level driver APIs to access the SPI NOR flash.

Due to the restriction of NOR flash and ESP hardware, accessing the main flash will affect the performance of the whole system. See *SPI Flash API* to learn more about the limitations.

2.9.1 FAT Filesystem Support

ESP-IDF uses the FatFs library to work with FAT filesystems. FatFs resides in the `fatfs` component. Although the library can be used directly, many of its features can be accessed via VFS using the C standard library and POSIX API functions.

Additionally, FatFs has been modified to support the runtime pluggable disk I/O layer. This allows mapping of FatFs drives to physical disks at runtime.

Using FatFs with VFS

The header file fatfs/vfs/esp_vfs_fat.h defines the functions for connecting FatFs and VFS.

The function `esp_vfs_fat_register()` allocates a `FATFS` structure and registers a given path prefix in VFS. Subsequent operations on files starting with this prefix are forwarded to FatFs APIs.

The function *`esp_vfs_fat_unregister_path()`* deletes the registration with VFS, and frees the `FATFS` structure.

Most applications use the following workflow when working with `esp_vfs_fat_` functions:

1. **Call `esp_vfs_fat_register()` to specify:**
   - Path prefix where to mount the filesystem (e.g., `"/sdcard"`, `"/spiflash"`)
   - FatFs drive number
   - A variable which receives the pointer to the `FATFS` structure
2. Call *`ff_diskio_register()`* to register the disk I/O driver for the drive number used in Step 1.
3. To mount the filesystem using the same drive number which was passed to `esp_vfs_fat_register()`, call the FatFs function `f_mount()`. If the filesystem is not present on the target logical drive, `f_mount()` will fail with the `FR_NO_FILESYSTEM` error. In such case, call `f_mkfs()` to create a fresh FatFS structure on the drive first, and then call `f_mount()` again. Note that SD cards need to be partitioned with `f_fdisk()` prior to previously described steps. For more information, see FatFs documentation.
4. Call the C standard library and POSIX API functions to perform such actions on files as open, read, write, erase, copy, etc. Use paths starting with the path prefix passed to *`esp_vfs_register()`* (for example, `"/sdcard/hello.txt"`). The filesystem uses 8.3 filenames format (SFN) by default. If you need to use long filenames (LFN), enable the *CONFIG_FATFS_LONG_FILENAMES* option. Please refer to FatFs filenames for more details.
5. Optionally, call the FatFs library functions directly. In this case, use paths without a VFS prefix, for example, `"/hello.txt"`.
6. Close all open files.
7. Call the FatFs function `f_mount()` for the same drive number with NULL `FATFS*` argument to unmount the filesystem.
8. Call the FatFs function *`ff_diskio_register()`* with NULL `ff_diskio_impl_t*` argument and the same drive number to unregister the disk I/O driver.
9. Call *`esp_vfs_fat_unregister_path()`* with the path where the file system is mounted to remove FatFs from VFS, and free the `FATFS` structure allocated in Step 1.

The convenience functions *`esp_vfs_fat_sdmmc_mount()`*, *`esp_vfs_fat_sdspi_mount()`*, and *`esp_vfs_fat_sdcard_unmount()`* wrap the steps described above and also handle SD card initialization. These functions are described in the next section.

Note: Because FAT filesystem does not support hardlinks, `link()` copies contents of the file instead. (This only applies to files on FatFs volumes.)

Using FatFs with VFS and SD Cards

The header file fatfs/vfs/esp_vfs_fat.h defines convenience functions *`esp_vfs_fat_sdmmc_mount()`*, *`esp_vfs_fat_sdspi_mount()`*, and *`esp_vfs_fat_sdcard_unmount()`*. These functions perform Steps 1–3 and 7–9 respectively and handle SD card initialization, but provide only limited error handling. Developers are encouraged to check its source code and incorporate more advanced features into production applications.

The convenience function *esp_vfs_fat_sdmmc_unmount()* unmounts the filesystem and releases the resources acquired by *esp_vfs_fat_sdmmc_mount()*.

Using FatFs with VFS in Read-Only Mode

The header file fatfs/vfs/esp_vfs_fat.h also defines the convenience functions *esp_vfs_fat_spiflash_mount_ro()* and *esp_vfs_fat_spiflash_unmount_ro()*. These functions perform Steps 1-3 and 7-9 respectively for read-only FAT partitions. These are particularly helpful for data partitions written only once during factory provisioning, which will not be changed by production application throughout the lifetime of the hardware.

Configuration options

The following configuration options are available for the FatFs component:

- *CONFIG_FATFS_USE_FASTSEEK* - If enabled, the POSIX `lseek()` function will be performed faster. The fast seek does not work for files in write mode, so to take advantage of fast seek, you should open (or close and then reopen) the file in read-only mode.
- *CONFIG_FATFS_IMMEDIATE_FSYNC* - If enabled, the FatFs will automatically call `f_sync()` to flush recent file changes after each call of `write()`, `pwrite()`, `link()`, `truncate()` and `ftruncate()` functions. This feature improves file-consistency and size reporting accuracy for the FatFs, at a price on decreased performance due to frequent disk operations.
- *CONFIG_FATFS_LINK_LOCK* - If enabled, this option guarantees the API thread safety, while disabling this option might be necessary for applications that require fast frequent small file operations (e.g., logging to a file). Note that if this option is disabled, the copying performed by `link()` will be non-atomic. In such case, using `link()` on a large file on the same volume in a different task is not guaranteed to be thread safe.

FatFS Disk IO Layer

FatFs has been extended with API functions that register the disk I/O driver at runtime.

These APIs provide implementation of disk I/O functions for SD/MMC cards and can be registered for the given FatFs drive number using the function *ff_diskio_register_sdmmc()*.

void **ff_diskio_register**(BYTE pdrv, const *ff_diskio_impl_t* *discio_impl)

Register or unregister diskio driver for given drive number.

When FATFS library calls one of disk_xxx functions for driver number pdrv, corresponding function in discio_impl for given pdrv will be called.

Parameters

- **pdrv** -- drive number
- **discio_impl** -- pointer to *ff_diskio_impl_t* structure with diskio functions or NULL to unregister and free previously registered drive

struct **ff_diskio_impl_t**

Structure of pointers to disk IO driver functions.

See FatFs documentation for details about these functions

Public Members

DSTATUS (***init**)(unsigned char pdrv)

disk initialization function

DSTATUS (***status**)(unsigned char pdrv)

disk status check function

DRESULT (***read**)(unsigned char pdrv, unsigned char *buff, uint32_t sector, unsigned count)

sector read function

DRESULT (***write**)(unsigned char pdrv, const unsigned char *buff, uint32_t sector, unsigned count)

sector write function

DRESULT (***ioctl**)(unsigned char pdrv, unsigned char cmd, void *buff)

function to get info about disk and do some misc operations

void **ff_diskio_register_sdmmc**(unsigned char pdrv, sdmmc_card_t *card)

Register SD/MMC diskio driver

Parameters

- **pdrv** -- drive number
- **card** -- pointer to sdmmc_card_t structure describing a card; card should be initialized before calling f_mount.

esp_err_t **ff_diskio_register_wl_partition**(unsigned char pdrv, *wl_handle_t* flash_handle)

Register spi flash partition

Parameters

- **pdrv** -- drive number
- **flash_handle** -- handle of the wear levelling partition.

esp_err_t **ff_diskio_register_raw_partition**(unsigned char pdrv, const *esp_partition_t* *part_handle)

Register spi flash partition

Parameters

- **pdrv** -- drive number
- **part_handle** -- pointer to raw flash partition.

FatFs Partition Generator

We provide a partition generator for FatFs (wl_fatfsgen.py) which is integrated into the build system and could be easily used in the user project.

The tool is used to create filesystem images on a host and populate it with content of the specified host folder.

The script is based on the partition generator (fatfsgen.py). Apart from generating partition, it can also initialize wear levelling.

The latest version supports both short and long file names, FAT12 and FAT16. The long file names are limited to 255 characters and can contain multiple periods (`.`) characters within the filename and additional characters `+`, `,`, `;`, `=`, `[` and `]`.

An in-depth description of the FatFs partition generator and analyzer can be found at Generating and parsing FAT partition on host.

Build System Integration with FatFs Partition Generator It is possible to invoke FatFs generator directly from the CMake build system by calling `fatfs_create_spiflash_image`:

```
fatfs_create_spiflash_image(<partition> <base_dir> [FLASH_IN_PROJECT])
```

If you prefer generating partition without wear levelling support, you can use `fatfs_create_rawflash_image`:

```
fatfs_create_rawflash_image(<partition> <base_dir> [FLASH_IN_PROJECT])
```

`fatfs_create_spiflash_image` respectively `fatfs_create_rawflash_image` must be called from project's CMakeLists.txt.

If you decide for any reason to use `fatfs_create_rawflash_image` (without wear levelling support), beware that it supports mounting only in read-only mode in the device.

The arguments of the function are as follows:

1. partition - the name of the partition as defined in the partition table (e.g., storage/fatfsgen/partitions_example.csv).
2. base_dir - the directory that will be encoded to FatFs partition and optionally flashed into the device. Beware that you have to specify the suitable size of the partition in the partition table.
3. flag `FLASH_IN_PROJECT` - optionally, users can have the image automatically flashed together with the app binaries, partition tables, etc. on `idf.py flash -p <PORT>` by specifying `FLASH_IN_PROJECT`.
4. flag `PRESERVE_TIME` - optionally, users can force preserving the timestamps from the source folder to the target image. Without preserving the time, every timestamp will be set to the FATFS default initial time (1st January 1980).
5. flag `ONE_FAT` - optionally, users can still choose to generate a FATFS volume with a single FAT (file allocation table) instead of two. This makes the free space in the FATFS volume a bit larger (by `number of sectors used by FAT * sector size`) but also more prone to corruption.

For example:

```
fatfs_create_spiflash_image(my_fatfs_partition my_folder FLASH_IN_PROJECT)
```

If FLASH_IN_PROJECT is not specified, the image will still be generated, but you will have to flash it manually using `esptool.py` or a custom build system target.

For an example, see storage/fatfsgen.

FatFs Partition Analyzer

(fatfsparse.py) is a partition analyzing tool for FatFs.

It is a reverse tool of (fatfsgen.py), i.e., it can generate the folder structure on the host based on the FatFs image.

Usage:

```
./fatfsparse.py [-h] [--wl-layer {detect,enabled,disabled}] [--verbose] fatfs_
↪image.img
```

Parameter --verbose prints detailed information from boot sector of the FatFs image to the terminal before folder structure is generated.

High-level API Reference

Header File

- components/fatfs/vfs/esp_vfs_fat.h
- This header file can be included with:

  ```
  #include "esp_vfs_fat.h"
  ```

- This header file is a part of the API provided by the `fatfs` component. To declare that your component depends on `fatfs`, add the following to your CMakeLists.txt:

  ```
  REQUIRES fatfs
  ```

  or

```
PRIV_REQUIRES fatfs
```

Functions

esp_err_t **esp_vfs_fat_register_cfg**(const *esp_vfs_fat_conf_t* *conf, FATFS **out_fs)

Register FATFS with VFS component.

This function registers given FAT drive in VFS, at the specified base path. Input arguments are held in *esp_vfs_fat_conf_t* structure. If only one drive is used, fat_drive argument can be an empty string. Refer to FATFS library documentation on how to specify FAT drive. This function also allocates FATFS structure which should be used for f_mount call.

Note: This function doesn't mount the drive into FATFS, it just connects POSIX and C standard library IO function with FATFS. You need to mount desired drive into FATFS separately.

Parameters

- **conf** -- pointer to *esp_vfs_fat_conf_t* configuration structure
- **out_fs** -- **[out]** pointer to FATFS structure which can be used for FATFS f_mount call is returned via this argument.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_register was already called
- ESP_ERR_NO_MEM if not enough memory or too many VFSes already registered

esp_err_t **esp_vfs_fat_unregister_path**(const char *base_path)

Un-register FATFS from VFS.

Note: FATFS structure returned by esp_vfs_fat_register is destroyed after this call. Make sure to call f_mount function to unmount it before calling esp_vfs_fat_unregister_ctx. Difference between this function and the one above is that this one will release the correct drive, while the one above will release the last registered one

---

Parameters **base_path** -- path prefix where FATFS is registered. This is the same used when esp_vfs_fat_register was called

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if FATFS is not registered in VFS

esp_err_t **esp_vfs_fat_sdmmc_mount**(const char *base_path, const sdmmc_host_t *host_config, const void *slot_config, const *esp_vfs_fat_mount_config_t* *mount_config, sdmmc_card_t **out_card)

Convenience function to get FAT filesystem on SD card registered in VFS.

This is an all-in-one function which does the following:

- initializes SDMMC driver or SPI driver with configuration in host_config
- initializes SD card with configuration in slot_config
- mounts FAT partition on SD card using FATFS library, with configuration in mount_config
- registers FATFS library with VFS, with prefix given by base_prefix variable

This function is intended to make example code more compact. For real world applications, developers should implement the logic of probing SD card, locating and mounting partition, and registering FATFS in VFS, with proper error checking and handling of exceptional conditions.

Note: Use this API to mount a card through SDSPI is deprecated. Please call `esp_vfs_fat_sdspi_mount()` instead for that case.

Parameters

- **base_path** -- path where partition should be registered (e.g. "/sdcard")
- **host_config** -- Pointer to structure describing SDMMC host. When using SDMMC peripheral, this structure can be initialized using SDMMC_HOST_DEFAULT() macro. When using SPI peripheral, this structure can be initialized using SDSPI_HOST_DEFAULT() macro.
- **slot_config** -- Pointer to structure with slot configuration. For SDMMC peripheral, pass a pointer to *sdmmc_slot_config_t* structure initialized using SDMMC_SLOT_CONFIG_DEFAULT.
- **mount_config** -- pointer to structure with extra parameters for mounting FATFS
- **out_card** -- **[out]** if not NULL, pointer to the card information structure will be returned via this argument

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_sdmmc_mount was already called
- ESP_ERR_NO_MEM if memory can not be allocated
- ESP_FAIL if partition can not be mounted
- other error codes from SDMMC or SPI drivers, SDMMC protocol, or FATFS drivers

esp_err_t **esp_vfs_fat_sdspi_mount**(const char *base_path, const sdmmc_host_t *host_config_input, const *sdspi_device_config_t* *slot_config, const *esp_vfs_fat_mount_config_t* *mount_config, sdmmc_card_t **out_card)

Convenience function to get FAT filesystem on SD card registered in VFS.

This is an all-in-one function which does the following:

- initializes an SPI Master device based on the SPI Master driver with configuration in slot_config, and attach it to an initialized SPI bus.
- initializes SD card with configuration in host_config_input
- mounts FAT partition on SD card using FATFS library, with configuration in mount_config
- registers FATFS library with VFS, with prefix given by base_prefix variable

This function is intended to make example code more compact. For real world applications, developers should implement the logic of probing SD card, locating and mounting partition, and registering FATFS in VFS, with proper error checking and handling of exceptional conditions.

Note: This function try to attach the new SD SPI device to the bus specified in host_config. Make sure the SPI bus specified in `host_config->slot` have been initialized by `spi_bus_initialize()` before.

Parameters

- **base_path** -- path where partition should be registered (e.g. "/sdcard")
- **host_config_input** -- Pointer to structure describing SDMMC host. This structure can be initialized using SDSPI_HOST_DEFAULT() macro.
- **slot_config** -- Pointer to structure with slot configuration. For SPI peripheral, pass a pointer to *sdspi_device_config_t* structure initialized using SDSPI_DEVICE_CONFIG_DEFAULT().
- **mount_config** -- pointer to structure with extra parameters for mounting FATFS
- **out_card** -- **[out]** If not NULL, pointer to the card information structure will be returned via this argument. It is suggested to hold this handle and use it to unmount the card later if needed. Otherwise it's not suggested to use more than one card at the same time and unmount one of them in your application.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_sdmmc_mount was already called
- ESP_ERR_NO_MEM if memory can not be allocated
- ESP_FAIL if partition can not be mounted
- other error codes from SDMMC or SPI drivers, SDMMC protocol, or FATFS drivers

esp_err_t **esp_vfs_fat_sdmmc_unmount**(void)

Unmount FAT filesystem and release resources acquired using esp_vfs_fat_sdmmc_mount.

Deprecated:

Use `esp_vfs_fat_sdcard_unmount()` instead.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_sdmmc_mount hasn't been called

esp_err_t **esp_vfs_fat_sdcard_unmount**(const char *base_path, sdmmc_card_t *card)

Unmount an SD card from the FAT filesystem and release resources acquired using `esp_vfs_fat_sdmmc_mount()` or `esp_vfs_fat_sdspi_mount()`

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the card argument is unregistered
- ESP_ERR_INVALID_STATE if esp_vfs_fat_sdmmc_mount hasn't been called

esp_err_t **esp_vfs_fat_sdcard_format_cfg**(const char *base_path, sdmmc_card_t *card, *esp_vfs_fat_mount_config_t* *cfg)

Format FAT filesystem with given configuration.

Note: This API should be only called when the FAT is already mounted.

Parameters

- **base_path** -- Path where partition should be registered (e.g. "/sdcard")
- **card** -- Pointer to the card handle, which should be initialised by calling `esp_vfs_fat_sdspi_mount` first
- **cfg** -- Pointer to structure with extra parameters for formatting FATFS (only relevant fields are used). If NULL, the previous configuration will be used.

Returns

- ESP_OK
- ESP_ERR_INVALID_STATE: FAT partition isn't mounted, call esp_vfs_fat_sdmmc_mount or esp_vfs_fat_sdspi_mount first
- ESP_ERR_NO_MEM: if memory can not be allocated
- ESP_FAIL: fail to format it, or fail to mount back

esp_err_t **esp_vfs_fat_sdcard_format**(const char *base_path, sdmmc_card_t *card)

Format FAT filesystem.

Note: This API should be only called when the FAT is already mounted.

Parameters

- **base_path** -- Path where partition should be registered (e.g. "/sdcard")
- **card** -- Pointer to the card handle, which should be initialised by calling `esp_vfs_fat_sdspi_mount` first

Returns

- ESP_OK
- ESP_ERR_INVALID_STATE: FAT partition isn't mounted, call esp_vfs_fat_sdmmc_mount or esp_vfs_fat_sdspi_mount first
- ESP_ERR_NO_MEM: if memory can not be allocated
- ESP_FAIL: fail to format it, or fail to mount back

esp_err_t **esp_vfs_fat_spiflash_mount_rw_wl**(const char *base_path, const char *partition_label, const *esp_vfs_fat_mount_config_t* *mount_config, *wl_handle_t* *wl_handle)

Convenience function to initialize FAT filesystem in SPI flash and register it in VFS.

This is an all-in-one function which does the following:

- finds the partition with defined partition_label. Partition label should be configured in the partition table.
- initializes flash wear levelling library on top of the given partition
- mounts FAT partition using FATFS library on top of flash wear levelling library
- registers FATFS library with VFS, with prefix given by base_prefix variable

This function is intended to make example code more compact.

Parameters

- **base_path** -- path where FATFS partition should be mounted (e.g. "/spiflash")
- **partition_label** -- label of the partition which should be used
- **mount_config** -- pointer to structure with extra parameters for mounting FATFS
- **wl_handle** -- **[out]** wear levelling driver handle

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if the partition table does not contain FATFS partition with given label
- ESP_ERR_INVALID_STATE if esp_vfs_fat_spiflash_mount_rw_wl was already called
- ESP_ERR_NO_MEM if memory can not be allocated
- ESP_FAIL if partition can not be mounted
- other error codes from wear levelling library, SPI flash driver, or FATFS drivers

esp_err_t **esp_vfs_fat_spiflash_unmount_rw_wl**(const char *base_path, *wl_handle_t* wl_handle)

Unmount FAT filesystem and release resources acquired using esp_vfs_fat_spiflash_mount_rw_wl.

Parameters

- **base_path** -- path where partition should be registered (e.g. "/spiflash")
- **wl_handle** -- wear levelling driver handle returned by esp_vfs_fat_spiflash_mount_rw_wl

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_spiflash_mount_rw_wl hasn't been called

esp_err_t **esp_vfs_fat_spiflash_format_cfg_rw_wl**(const char *base_path, const char *partition_label, *esp_vfs_fat_mount_config_t* *cfg)

Format FAT filesystem with given configuration.

Note: This API can be called when the FAT is mounted / not mounted. If this API is called when the FAT isn't mounted (by calling esp_vfs_fat_spiflash_mount_rw_wl), this API will first mount the FAT then format it, then restore back to the original state.

Parameters

- **base_path** -- Path where partition should be registered (e.g. "/spiflash")
- **partition_label** -- Label of the partition which should be used
- **cfg** -- Pointer to structure with extra parameters for formatting FATFS (only relevant fields are used). If NULL and mounted the previous configuration will be used. If NULL and unmounted the default configuration will be used.

Returns

- ESP_OK
- ESP_ERR_NO_MEM: if memory can not be allocated
- Other errors from esp_vfs_fat_spiflash_mount_rw_wl

esp_err_t **esp_vfs_fat_spiflash_format_rw_wl**(const char *base_path, const char *partition_label)

Format FAT filesystem.

Note: This API can be called when the FAT is mounted / not mounted. If this API is called when the FAT isn't mounted (by calling esp_vfs_fat_spiflash_mount_rw_wl), this API will first mount the FAT then format it, then restore back to the original state.

Parameters

- **base_path** -- Path where partition should be registered (e.g. "/spiflash")
- **partition_label** -- Label of the partition which should be used

Returns

- ESP_OK
- ESP_ERR_NO_MEM: if memory can not be allocated
- Other errors from esp_vfs_fat_spiflash_mount_rw_wl

esp_err_t **esp_vfs_fat_spiflash_mount_ro**(const char *base_path, const char *partition_label, const *esp_vfs_fat_mount_config_t* *mount_config)

Convenience function to initialize read-only FAT filesystem and register it in VFS.

This is an all-in-one function which does the following:

- finds the partition with defined partition_label. Partition label should be configured in the partition table.
- mounts FAT partition using FATFS library
- registers FATFS library with VFS, with prefix given by base_prefix variable

---

Note: Wear levelling is not used when FAT is mounted in read-only mode using this function.

Parameters

- **base_path** -- path where FATFS partition should be mounted (e.g. "/spiflash")
- **partition_label** -- label of the partition which should be used
- **mount_config** -- pointer to structure with extra parameters for mounting FATFS

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if the partition table does not contain FATFS partition with given label
- ESP_ERR_INVALID_STATE if esp_vfs_fat_spiflash_mount_ro was already called for the same partition
- ESP_ERR_NO_MEM if memory can not be allocated
- ESP_FAIL if partition can not be mounted
- other error codes from SPI flash driver, or FATFS drivers

esp_err_t **esp_vfs_fat_spiflash_unmount_ro**(const char *base_path, const char *partition_label)

Unmount FAT filesystem and release resources acquired using esp_vfs_fat_spiflash_mount_ro.

Parameters

- **base_path** -- path where partition should be registered (e.g. "/spiflash")
- **partition_label** -- label of partition to be unmounted

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if esp_vfs_fat_spiflash_mount_ro hasn't been called

esp_err_t **esp_vfs_fat_info**(const char *base_path, uint64_t *out_total_bytes, uint64_t *out_free_bytes)

Get information for FATFS partition.

Parameters

- **base_path** -- Base path of the partition examined (e.g. "/spiflash")
- **out_total_bytes** -- **[out]** Size of the file system
- **out_free_bytes** -- **[out]** Free bytes available in the file system

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if partition not found
- ESP_FAIL if another FRESULT error (saved in errno)

esp_err_t **esp_vfs_fat_create_contiguous_file**(const char *base_path, const char *full_path, uint64_t size, bool alloc_now)

Create a file with contiguous space at given path.

Note: The file cannot exist before calling this function (or the file size has to be 0) For more information see documentation for `f_expand` from FATFS library

---

Parameters

- **base_path** -- Base path of the partition examined (e.g. "/spiflash")
- **full_path** -- Full path of the file (e.g. "/spiflash/ABC.TXT")
- **size** -- File size expanded to, number of bytes in size to prepare or allocate for the file
- **alloc_now** -- True == allocate space now, false == prepare to allocate –see `f_expand` from FATFS

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if invalid arguments (e.g. any of arguments are NULL or size lower or equal to 0)
- ESP_ERR_INVALID_STATE if partition not found
- ESP_FAIL if another FRESULT error (saved in errno)

esp_err_t **esp_vfs_fat_test_contiguous_file**(const char *base_path, const char *full_path, bool *is_contiguous)

Test if a file is contiguous in the FAT filesystem.

Parameters

- **base_path** -- Base path of the partition examined (e.g. "/spiflash")
- **full_path** -- Full path of the file (e.g. "/spiflash/ABC.TXT")
- **is_contiguous** -- **[out]** True == allocate space now, false == prepare to allocate –see `f_expand` from FATFS

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if invalid arguments (e.g. any of arguments are NULL)
- ESP_ERR_INVALID_STATE if partition not found
- ESP_FAIL if another FRESULT error (saved in errno)

Structures

struct **esp_vfs_fat_conf_t**

Configuration structure for esp_vfs_fat_register.

Public Members

const char ***base_path**

Path prefix where FATFS should be registered,

const char *__fat_drive__

FATFS drive specification; if only one drive is used, can be an empty string.

size_t __max_files__

Maximum number of files which can be open at the same time.

struct __esp_vfs_fat_mount_config_t__

Configuration arguments for esp_vfs_fat_sdmmc_mount and esp_vfs_fat_spiflash_mount_rw_wl functions.

Public Members

bool __format_if_mount_failed__

If FAT partition can not be mounted, and this parameter is true, create partition table and format the filesystem.

int __max_files__

Max number of open files.

size_t __allocation_unit_size__

If format_if_mount_failed is set, and mount fails, format the card with given allocation unit size. Must be a power of 2, between sector size and 128 * sector size. For SD cards, sector size is always 512 bytes. For wear_levelling, sector size is determined by CONFIG_WL_SECTOR_SIZE option.

Using larger allocation unit size will result in higher read/write performance and higher overhead when storing small files.

Setting this field to 0 will result in allocation unit set to the sector size.

bool __disk_status_check_enable__

Enables real ff_disk_status function implementation for SD cards (ff_sdmmc_status). Possibly slows down IO performance.

Try to enable if you need to handle situations when SD cards are not unmounted properly before physical removal or you are experiencing issues with SD cards.

Doesn't do anything for other memory storage media.

bool __use_one_fat__

Use 1 FAT (File Allocation Tables) instead of 2. This decreases reliability, but makes more space available (usually only one sector). Note that this option has effect only when the filesystem is formatted. When mounting an already-formatted partition, the actual number of FATs may be different.

Macros

__VFS_FAT_MOUNT_DEFAULT_CONFIG__()

Type Definitions

typedef *esp_vfs_fat_mount_config_t* __esp_vfs_fat_sdmmc_mount_config_t__

2.9.2 Manufacturing Utility

Introduction

This utility is designed to create instances of factory NVS partition images on a per-device basis for mass manufacturing purposes. The NVS partition images are created from CSV files containing user-provided configurations and values.

Please note that this utility only creates manufacturing binary images which then need to be flashed onto your devices using:

- esptool.py
- **Flash Download tool (available on Windows only)**
  - Download and unzip it, and follow the instructions inside the *doc* folder.
- Direct flash programming using custom production tools.

Prerequisites

This utility is dependent on ESP-IDF's NVS Partition Generator Utility.

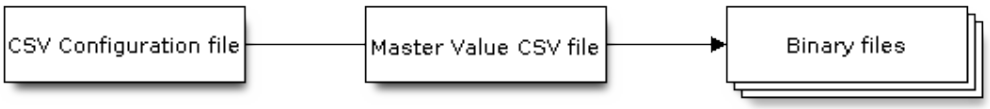
- **Operating System requirements:**
  - Linux / MacOS / Windows (standard distributions)
- **The following packages are needed to use this utility:**
  - Python

Note:

Before using this utility, please make sure that:

- The path to Python is added to the PATH environment variable.
- You have installed the packages from *requirement.txt*, the file in the root of the ESP-IDF directory.

Workflow



CSV Configuration File

This file contains the configuration of the device to be flashed.

The data in the configuration file has the following format (the *REPEAT* tag is optional):

```
name1,namespace,    <-- First entry should be of type "namespace"
key1,type1,encoding1
key2,type2,encoding2,REPEAT
name2,namespace,
key3,type3,encoding3
key4,type4,encoding4
```

Note: The first line in this file should always be the `namespace` entry.

Each line should have three parameters: `key,type,encoding`, separated by a comma. If the `REPEAT` tag is present, the value corresponding to this key in the master value CSV file will be the same for all devices.

Please refer to README of the NVS Partition Generator Utility for detailed description of each parameter.

Below is a sample example of such a configuration file:

```
app,namespace,
firmware_key,data,hex2bin
serial_no,data,string,REPEAT
device_no,data,i32
```

Note:

Make sure there are no spaces:

- before and after ','
- at the end of each line in a CSV file

Master Value CSV File

This file contains details of the devices to be flashed. Each line in this file corresponds to a device instance.

The data in the master value CSV file has the following format:

```
key1,key2,key3,.....
value1,value2,value3,....
```

Note: The first line in the file should always contain the `key` names. All the keys from the configuration file should be present here in the **same order**. This file can have additional columns (keys). The additional keys will be treated as metadata and would not be part of the final binary files.

Each line should contain the `value` of the corresponding keys, separated by a comma. If the key has the `REPEAT` tag, its corresponding value **must** be entered in the second line only. Keep the entry empty for this value in the following lines.

The description of this parameter is as follows:

`value` Data value

Data value is the value of data corresponding to the key.

Below is a sample example of a master value CSV file:

```
id,firmware_key,serial_no,device_no
1,1a2b3c4d5e6faabb,A1,101
2,1a2b3c4d5e6fccdd,,102
3,1a2b3c4d5e6feeff,,103
```

Note: *If the 'REPEAT' tag is present, a new master value CSV file will be created in the same folder as the input Master CSV File with the values inserted at each line for the key with the 'REPEAT' tag.*

This utility creates intermediate CSV files which are used as input for the NVS partition utility to generate the binary files.

The format of this intermediate CSV file is as follows:

```
key,type,encoding,value
key,namespace, ,
key1,type1,encoding1,value1
key2,type2,encoding2,value2
```

An instance of an intermediate CSV file will be created for each device on an individual basis.

Running the utility

Usage:

```
python mfg_gen.py [-h] {generate,generate-key} ...
```

Optional Arguments:

No.	Parameter	Description
1	`-h / --help`	Show the help message and exit

Commands:

Run mfg_gen.py {command} -h for additional help

No.	Parameter	Description
1	`generate`	Generate NVS partition
2	`generate-key`	Generate keys for encryption

To generate factory images for each device (Default):

Usage:

```
python mfg_gen.py generate [-h] [--fileid FILEID] [--version {1,2}] [--keygen]
                                [--inputkey INPUTKEY] [--outdir OUTDIR]
                                [--key_protect_hmac] [--kp_hmac_keygen]
                                [--kp_hmac_keyfile KP_HMAC_KEYFILE] [--kp_hmac_
↪inputkey KP_HMAC_INPUTKEY]
                                conf values prefix size
```

Positional Arguments:

Parameter	Description
`conf`	Path to configuration csv file to parse
`values`	Path to values csv file to parse
`prefix`	Unique name for each output filename prefix
`size`	Size of NVS partition in bytes (must be multiple of 4096)

Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--fileid FILEID`	Unique file identifier (any key in values file) for each filename suffix (Default: numeric value(1,2,3...))
`--version {1,2}`	Set multipage blob version. (Default: Version 2) Version 1 - Multipage blob support disabled. Version 2 - Multipage blob support enabled.
`--keygen`	Generates key for encrypting NVS partition
`--inputkey INPUTKEY`	File having key for encrypting NVS partition
`--outdir OUTDIR`	Output directory to store files created (Default: current directory)
`--key_protect_hmac`	If set, the NVS encryption key protection scheme based on HMAC peripheral is used; else the default scheme based on Flash Encryption is used
`--kp_hmac_keygen`	Generate the HMAC key for HMAC-based encryption scheme
`--kp_hmac_keyfile KP_HMAC_KEYFILE`	Path to output HMAC key file
`--kp_hmac_inputkey KP_HMAC_INPUTKEY`	File having the HMAC key for generating the NVS encryption keys

You can run the utility to generate factory images for each device using the command below. A sample CSV file is provided with the utility:

```
python mfg_gen.py generate samples/sample_config.csv samples/sample_values_
↪singlepage_blob.csv Sample 0x3000
```

The master value CSV file should have the path in the `file` type relative to the directory from which you are running the utility.

To generate encrypted factory images for each device:

You can run the utility to encrypt factory images for each device using the command below. A sample CSV file is provided with the utility:

- Encrypt by allowing the utility to generate encryption keys:

```
python mfg_gen.py generate samples/sample_config.csv samples/sample_values_
↪singlepage_blob.csv Sample 0x3000 --keygen
```

Note: Encryption key of the following format `<outdir>/keys/keys-<prefix>-<fileid>.bin` is created. This newly created file having encryption keys in `keys/` directory is compatible with NVS key-partition structure. Refer to *NVS Key Partition* for more details.

- To generate an encrypted image using the HMAC-based scheme, the above command can be used alongwith some additional parameters.
  - Encrypt by allowing the utility to generate encryption keys and the HMAC-key:

```
python mfg_gen.py generate samples/sample_config.csv samples/
↪sample_values_singlepage_blob.csv Sample 0x3000 --keygen --key_
↪protect_hmac --kp_hmac_keygen
```

Note: Encryption key of the format `<outdir>/keys/keys-<timestamp>.bin` and HMAC key of the format `<outdir>/keys/hmac-keys-<timestamp>.bin` are created.

- Encrypt by allowing the utility to generate encryption keys with user-provided HMAC-key:

```
python mfg_gen.py generate samples/sample_config.csv samples/sample_values_
↪singlepage_blob.csv Sample 0x3000 --keygen --key_protect_hmac --kp_hmac_
↪inputkey testdata/sample_hmac_key.bin
```

(continues on next page)

(continued from previous page)

Note: You can provide the custom filename for the HMAC key as well as the encryption key as a parameter.

- Encrypt by providing the encryption keys as input binary file:

```
python mfg_gen.py generate samples/sample_config.csv samples/sample_values_
↪singlepage_blob.csv Sample 0x3000 --inputkey keys/sample_keys.bin
```

To generate only encryption keys:

Usage:: python mfg_gen.py generate-key [-h] [--keyfile KEYFILE] [--outdir OUTDIR]
Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--keyfile KEYFILE`	Path to output encryption keys file
`--outdir OUTDIR`	Output directory to store files created. (Default: current directory)
`--key_protect_hmac`	If set, the NVS encryption key protection scheme based on HMAC peripheral is used; else the default scheme based on Flash Encryption is used
`--kp_hmac_keygen`	Generate the HMAC key for HMAC-based encryption scheme
`--kp_hmac_keyfile KP_HMAC_KEYFILE`	Path to output HMAC key file
`--kp_hmac_inputkey KP_HMAC_INPUTKEY`	File having the HMAC key for generating the NVS encryption keys

You can run the utility to generate only encryption keys using the command below:

```
python mfg_gen.py generate-key
```

Note: Encryption key of the following format `<outdir>/keys/keys-<timestamp>.bin` is created. Timestamp format is: `%m-%d_%H-%M`. To provide custom target filename use the --keyfile argument.

For generating encryption key for the HMAC-based scheme, the following commands can be used:

- Generate the HMAC key and the NVS encryption keys:

```
python mfg_gen.py generate-key --key_protect_hmac --kp_hmac_keygen
```

Note: Encryption key of the format `<outdir>/keys/keys-<timestamp>.bin` and HMAC key of the format `<outdir>/keys/hmac-keys-<timestamp>.bin` are created.

- Generate the NVS encryption keys, given the HMAC-key:

```
python mfg_gen.py generate-key --key_protect_hmac --kp_hmac_inputkey testdata/
↪sample_hmac_key.bin
```

Note: You can provide the custom filename for the HMAC key as well as the encryption key as a parameter.

Generated encryption key binary file can further be used to encrypt factory images created on the per device basis.

The default numeric value: 1,2,3... of the `fileid` argument corresponds to each line bearing device instance values in the master value CSV file.

While running the manufacturing utility, the following folders will be created in the specified `outdir` directory:

- `bin/` for storing the generated binary files
- `csv/` for storing the generated intermediate CSV files
- `keys/` for storing encryption keys (when generating encrypted factory images)

2.9.3 Non-Volatile Storage Library

Introduction

Non-volatile storage (NVS) library is designed to store key-value pairs in flash. This section introduces some concepts used by NVS.

Underlying Storage Currently, NVS uses a portion of main flash memory through the *esp_partition* API. The library uses all the partitions with `data` type and `nvs` subtype. The application can choose to use the partition with the label `nvs` through the *`nvs_open()`* API function or any other partition by specifying its name using the *`nvs_open_from_partition()`* API function.

Future versions of this library may have other storage backends to keep data in another flash chip (SPI or I2C), RTC, FRAM, etc.

Note: if an NVS partition is truncated (for example, when the partition table layout is changed), its contents should be erased. ESP-IDF build system provides a `idf.py erase-flash` target to erase all contents of the flash chip.

Note: NVS works best for storing many small values, rather than a few large values of the type 'string' and 'blob'. If you need to store large blobs or strings, consider using the facilities provided by the FAT filesystem on top of the wear levelling library.

Keys and Values NVS operates on key-value pairs. Keys are ASCII strings; the maximum key length is currently 15 characters. Values can have one of the following types:

- integer types: `uint8_t`, `int8_t`, `uint16_t`, `int16_t`, `uint32_t`, `int32_t`, `uint64_t`, `int64_t`
- zero-terminated string
- variable length binary data (blob)

Note: String values are currently limited to 4000 bytes. This includes the null terminator. Blob values are limited to 508,000 bytes or 97.6% of the partition size - 4000 bytes, whichever is lower.

Additional types, such as `float` and `double` might be added later.

Keys are required to be unique. Assigning a new value to an existing key replaces the old value and data type with the value and data type specified by a write operation.

A data type check is performed when reading a value. An error is returned if the data type expected by read operation does not match the data type of entry found for the key provided.

Namespaces To mitigate potential conflicts in key names between different components, NVS assigns each key-value pair to one of namespaces. Namespace names follow the same rules as key names, i.e., the maximum length is 15 characters. Furthermore, there can be no more than 254 different namespaces in one NVS partition. Namespace name is specified in the *`nvs_open()`* or *`nvs_open_from_partition`* call. This call returns an opaque handle, which is used in subsequent calls to the `nvs_get_*`, `nvs_set_*`, and *`nvs_commit()`* functions. This way, a

handle is associated with a namespace, and key names will not collide with same names in other namespaces. Please note that the namespaces with the same name in different NVS partitions are considered as separate namespaces.

NVS Iterators Iterators allow to list key-value pairs stored in NVS, based on specified partition name, namespace, and data type.

There are the following functions available:

- *nvs_entry_find()* creates an opaque handle, which is used in subsequent calls to the *nvs_entry_next()* and *nvs_entry_info()* functions.
- *nvs_entry_next()* advances an iterator to the next key-value pair.
- *nvs_entry_info()* returns information about each key-value pair

In general, all iterators obtained via *nvs_entry_find()* have to be released using *nvs_release_iterator()*, which also tolerates NULL iterators.

nvs_entry_find() and *nvs_entry_next()* set the given iterator to NULL or a valid iterator in all cases except a parameter error occurred (i.e., return ESP_ERR_NVS_NOT_FOUND). In case of a parameter error, the given iterator will not be modified. Hence, it is best practice to initialize the iterator to NULL before calling *nvs_entry_find()* to avoid complicated error checking before releasing the iterator.

Security, Tampering, and Robustness NVS is not directly compatible with the ESP32-S3 flash encryption system. However, data can still be stored in encrypted form if NVS encryption is used together with ESP32-S3 flash encryption or with the help of the HMAC peripheral. Please refer to *NVS Encryption* for more details.

If NVS encryption is not used, it is possible for anyone with physical access to the flash chip to alter, erase, or add key-value pairs. With NVS encryption enabled, it is not possible to alter or add a key-value pair and get recognized as a valid pair without knowing corresponding NVS encryption keys. However, there is no tamper-resistance against the erase operation.

The library does try to recover from conditions when flash memory is in an inconsistent state. In particular, one should be able to power off the device at any point and time and then power it back on. This should not result in loss of data, except for the new key-value pair if it was being written at the moment of powering off. The library should also be able to initialize properly with any random data present in flash memory.

NVS Encryption

Please refer to the *NVS Encryption* guide for more details.

NVS Partition Generator Utility

This utility helps generate NVS partition binary files which can be flashed separately on a dedicated partition via a flashing utility. Key-value pairs to be flashed onto the partition can be provided via a CSV file. For more details, please refer to *NVS Partition Generator Utility*.

Instead of calling the nvs_partition_gen.py tool manually, the creation of the partition binary files can also be done directly from CMake using the function nvs_create_partition_image:

```
nvs_create_partition_image(<partition> <csv> [FLASH_IN_PROJECT] [DEPENDS  dep dep␣
↪dep ...])
```

Positional Arguments:

Parameter	Description
partition	Name of the NVS partition
csv	Path to CSV file to parse

Optional Arguments:

Parameter	Description
FLASH_IN_PROJECT	Name of the NVS partition
DEPENDS	Specify files on which the command depends

If FLASH_IN_PROJECT is not specified, the image will still be generated, but you will have to flash it manually using idf.py <partition>-flash (e.g., if your partition name is nvs, then use idf.py nvs-flash).

nvs_create_partition_image must be called from one of the component CMakeLists.txt files. Currently, only non-encrypted partitions are supported.

Application Example

You can find code examples in the storage directory of ESP-IDF examples:

storage/nvs_rw_value

Demonstrates how to read a single integer value from, and write it to NVS.

The value checked in this example holds the number of the ESP32-S3 module restarts. The value's function as a counter is only possible due to its storing in NVS.

The example also shows how to check if a read/write operation was successful, or if a certain value has not been initialized in NVS. The diagnostic procedure is provided in plain text to help you track the program flow and capture any issues on the way.

storage/nvs_rw_blob

Demonstrates how to read a single integer value and a blob (binary large object), and write them to NVS to preserve this value between ESP32-S3 module restarts.

- value - tracks the number of the ESP32-S3 module soft and hard restarts.
- blob - contains a table with module run times. The table is read from NVS to dynamically allocated RAM. A new run time is added to the table on each manually triggered soft restart, and then the added run time is written to NVS. Triggering is done by pulling down GPIO0.

The example also shows how to implement the diagnostic procedure to check if the read/write operation was successful.

storage/nvs_rw_value_cxx

This example does exactly the same as storage/nvs_rw_value, except that it uses the C++ NVS handle class.

Internals

Log of Key-Value Pairs NVS stores key-value pairs sequentially, with new key-value pairs being added at the end. When a value of any given key has to be updated, a new key-value pair is added at the end of the log and the old key-value pair is marked as erased.

Pages and Entries NVS library uses two main entities in its operation: pages and entries. Page is a logical structure which stores a portion of the overall log. Logical page corresponds to one physical sector of flash memory. Pages which are in use have a *sequence number* associated with them. Sequence numbers impose an ordering on pages. Higher sequence numbers correspond to pages which were created later. Each page can be in one of the following states:

Empty/uninitialized Flash storage for the page is empty (all bytes are 0xff). Page is not used to store any data at this point and does not have a sequence number.

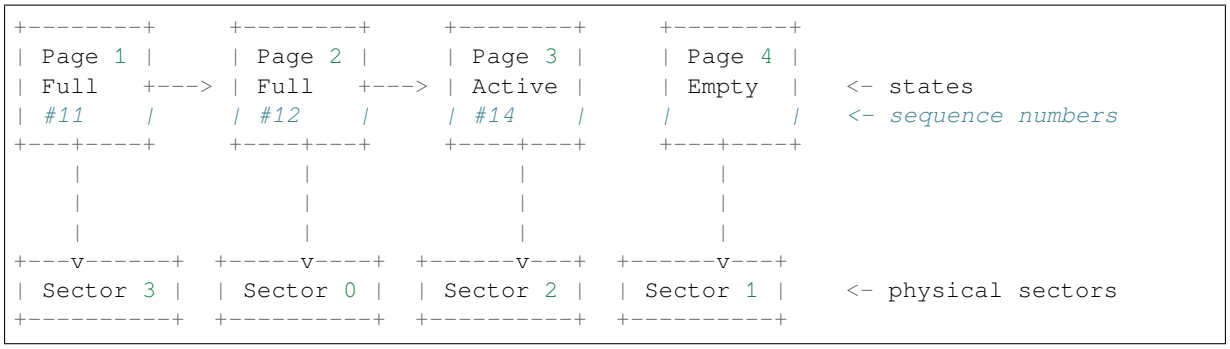
Active Flash storage is initialized, page header has been written to flash, page has a valid sequence number. Page has some empty entries and data can be written there. No more than one page can be in this state at any given moment.

Full Flash storage is in a consistent state and is filled with key-value pairs. Writing new key-value pairs into this page is not possible. It is still possible to mark some key-value pairs as erased.

Erasing Non-erased key-value pairs are being moved into another page so that the current page can be erased. This is a transient state, i.e., page should never stay in this state at the time when any API call returns. In case of a sudden power off, the move-and-erase process will be completed upon the next power-on.

Corrupted Page header contains invalid data, and further parsing of page data was canceled. Any items previously written into this page will not be accessible. The corresponding flash sector will not be erased immediately and will be kept along with sectors in **uninitialized** state for later use. This may be useful for debugging.

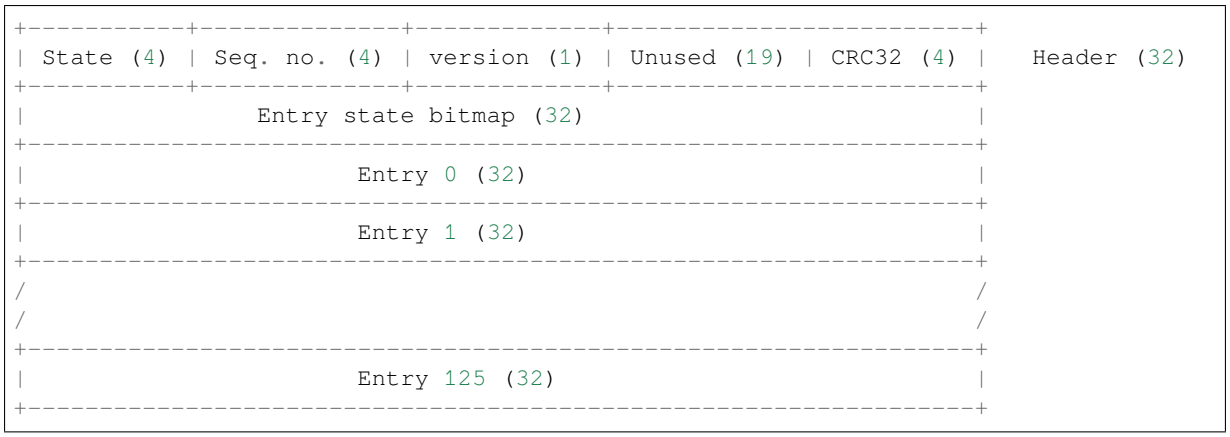
Mapping from flash sectors to logical pages does not have any particular order. The library will inspect sequence numbers of pages found in each flash sector and organize pages in a list based on these numbers.



Structure of a Page For now, we assume that flash sector size is 4096 bytes and that ESP32-S3 flash encryption hardware operates on 32-byte blocks. It is possible to introduce some settings configurable at compile-time (e.g., via menuconfig) to accommodate flash chips with different sector sizes (although it is not clear if other components in the system, e.g., SPI flash driver and SPI flash cache can support these other sizes).

Page consists of three parts: header, entry state bitmap, and entries themselves. To be compatible with ESP32-S3 flash encryption, the entry size is 32 bytes. For integer types, an entry holds one key-value pair. For strings and blobs, an entry holds part of key-value pair (more on that in the entry structure description).

The following diagram illustrates the page structure. Numbers in parentheses indicate the size of each part in bytes.



Page header and entry state bitmap are always written to flash unencrypted. Entries are encrypted if flash encryption feature of ESP32-S3 is used.

Page state values are defined in such a way that changing state is possible by writing 0 into some of the bits. Therefore it is not necessary to erase the page to change its state unless that is a change to the *erased* state.

The version field in the header reflects the NVS format version used. For backward compatibility reasons, it is decremented for every version upgrade starting at 0xff (i.e., 0xff for version-1, 0xfe for version-2 and so on).

CRC32 value in the header is calculated over the part which does not include a state value (bytes 4 to 28). The unused part is currently filled with `0xff` bytes.

The following sections describe the structure of entry state bitmap and entry itself.

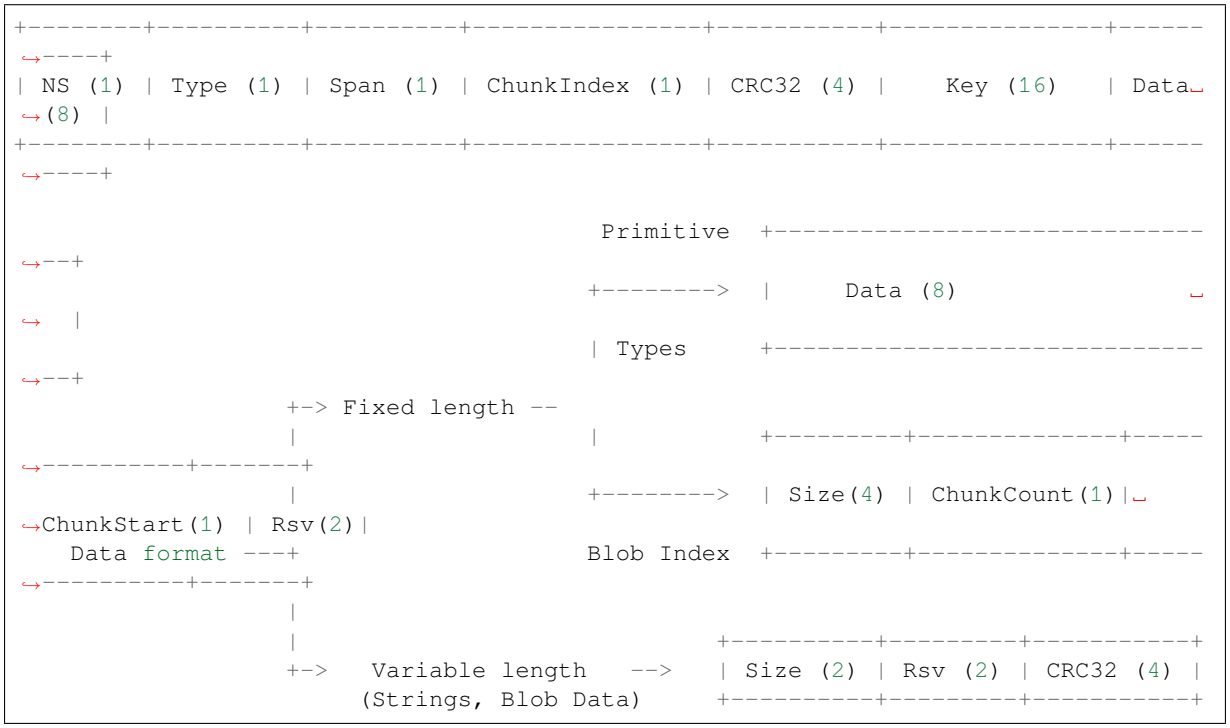
Entry and Entry State Bitmap Each entry can be in one of the following three states represented with two bits in the entry state bitmap. The final four bits in the bitmap (256 - 2 * 126) are not used.

Empty (2'b11) Nothing is written into the specific entry yet. It is in an uninitialized state (all bytes are `0xff`).

Written (2'b10) A key-value pair (or part of key-value pair which spans multiple entries) has been written into the entry.

Erased (2'b00) A key-value pair in this entry has been discarded. Contents of this entry will not be parsed anymore.

Structure of Entry For values of primitive types (currently integers from 1 to 8 bytes long), entry holds one key-value pair. For string and blob types, entry holds part of the whole key-value pair. For strings, in case when a key-value pair spans multiple entries, all entries are stored in the same page. Blobs are allowed to span over multiple pages by dividing them into smaller chunks. For tracking these chunks, an additional fixed length metadata entry is stored called "blob index". Earlier formats of blobs are still supported (can be read and modified). However, once the blobs are modified, they are stored using the new format.



Individual fields in entry structure have the following meanings:

NS Namespace index for this entry. For more information on this value, see the section on namespaces implementation.

Type One byte indicating the value data type. See the `ItemType` enumeration in nvs_flash/include/nvs_handle.hpp for possible values.

Span Number of entries used by this key-value pair. For integer types, this is equal to 1. For strings and blobs, this depends on value length.

ChunkIndex Used to store the index of a blob-data chunk for blob types. For other types, this should be `0xff`.

CRC32 Checksum calculated over all the bytes in this entry, except for the CRC32 field itself.

Key Zero-terminated ASCII string containing a key name. Maximum string length is 15 bytes, excluding a zero terminator.

Data For integer types, this field contains the value itself. If the value itself is shorter than 8 bytes, it is padded to the right, with unused bytes filled with `0xff`.

For "blob index" entry, these 8 bytes hold the following information about data-chunks:

- **Size** (Only for blob index.) Size, in bytes, of complete blob data.
- **ChunkCount** (Only for blob index.) Total number of blob-data chunks into which the blob was divided during storage.
- **ChunkStart** (Only for blob index.) ChunkIndex of the first blob-data chunk of this blob. Subsequent chunks have chunkIndex incrementally allocated (step of 1).

For string and blob data chunks, these 8 bytes hold additional data about the value, which are described below:

- **Size** (Only for strings and blobs.) Size, in bytes, of actual data. For strings, this includes zero terminators.
- **CRC32** (Only for strings and blobs.) Checksum calculated over all bytes of data.

Variable length values (strings and blobs) are written into subsequent entries, 32 bytes per entry. The `Span` field of the first entry indicates how many entries are used.

Namespaces As mentioned above, each key-value pair belongs to one of the namespaces. Namespace identifiers (strings) are stored as keys of key-value pairs in namespace with index 0. Values corresponding to these keys are indexes of these namespaces.

```
+----------------------------------------+
| NS=0 Type=uint8_t Key="wifi" Value=1   |   Entry describing namespace "wifi"
+----------------------------------------+
| NS=1 Type=uint32_t Key="channel" Value=6  |   Key "channel" in namespace "wifi"
+----------------------------------------+
| NS=0 Type=uint8_t Key="pwm" Value=2    |   Entry describing namespace "pwm"
+----------------------------------------+
| NS=2 Type=uint16_t Key="channel" Value=20 |   Key "channel" in namespace "pwm"
+----------------------------------------+
```

Item Hash List To reduce the number of reads from flash memory, each member of the Page class maintains a list of pairs: item index; item hash. This list makes searches much quicker. Instead of iterating over all entries, reading them from flash one at a time, *Page::findItem* first performs a search for the item hash in the hash list. This gives the item index within the page if such an item exists. Due to a hash collision, it is possible that a different item is found. This is handled by falling back to iteration over items in flash.

Each node in the hash list contains a 24-bit hash and 8-bit item index. Hash is calculated based on item namespace, key name, and ChunkIndex. CRC32 is used for calculation; the result is truncated to 24 bits. To reduce the overhead for storing 32-bit entries in a linked list, the list is implemented as a double-linked list of arrays. Each array holds 29 entries, for the total size of 128 bytes, together with linked list pointers and a 32-bit count field. The minimum amount of extra RAM usage per page is therefore 128 bytes; maximum is 640 bytes.

API Reference

Header File

- components/nvs_flash/include/nvs_flash.h
- This header file can be included with:

```
#include "nvs_flash.h"
```

- This header file is a part of the API provided by the `nvs_flash` component. To declare that your component depends on `nvs_flash`, add the following to your CMakeLists.txt:

```
REQUIRES nvs_flash
```

or

```
PRIV_REQUIRES nvs_flash
```

Functions

esp_err_t **nvs_flash_init** (void)

Initialize the default NVS partition.

This API initialises the default NVS partition. The default NVS partition is the one that is labeled "nvs" in the partition table.

When "NVS_ENCRYPTION" is enabled in the menuconfig, this API enables the NVS encryption for the default NVS partition as follows

a. Read security configurations from the first NVS key partition listed in the partition table. (NVS key partition is any "data" type partition which has the subtype value set to "nvs_keys")
b. If the NVS key partition obtained in the previous step is empty, generate and store new keys in that NVS key partition.
c. Internally call "nvs_flash_secure_init()" with the security configurations obtained/generated in the previous steps.

Post initialization NVS read/write APIs remain the same irrespective of NVS encryption.

Returns
- ESP_OK if storage was successfully initialized.
- ESP_ERR_NVS_NO_FREE_PAGES if the NVS storage contains no empty pages (which may happen if NVS partition was truncated)
- ESP_ERR_NOT_FOUND if no partition with label "nvs" is found in the partition table
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- one of the error codes from the underlying flash storage driver
- error codes from nvs_flash_read_security_cfg API (when "NVS_ENCRYPTION" is enabled).
- error codes from nvs_flash_generate_keys API (when "NVS_ENCRYPTION" is enabled).
- error codes from nvs_flash_secure_init_partition API (when "NVS_ENCRYPTION" is enabled) .

esp_err_t **nvs_flash_init_partition**(const char *partition_label)

Initialize NVS flash storage for the specified partition.

Parameters **partition_label** -- **[in]** Label of the partition. Must be no longer than 16 characters.

Returns
- ESP_OK if storage was successfully initialized.
- ESP_ERR_NVS_NO_FREE_PAGES if the NVS storage contains no empty pages (which may happen if NVS partition was truncated)
- ESP_ERR_NOT_FOUND if specified partition is not found in the partition table
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- one of the error codes from the underlying flash storage driver

esp_err_t **nvs_flash_init_partition_ptr**(const *esp_partition_t* *partition)

Initialize NVS flash storage for the partition specified by partition pointer.

Parameters **partition** -- **[in]** pointer to a partition obtained by the ESP partition API.

Returns
- ESP_OK if storage was successfully initialized
- ESP_ERR_NVS_NO_FREE_PAGES if the NVS storage contains no empty pages (which may happen if NVS partition was truncated)
- ESP_ERR_INVALID_ARG in case partition is NULL
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- one of the error codes from the underlying flash storage driver

esp_err_t **nvs_flash_deinit**(void)

Deinitialize NVS storage for the default NVS partition.

Default NVS partition is the partition with "nvs" label in the partition table.

Returns
- ESP_OK on success (storage was deinitialized)
- ESP_ERR_NVS_NOT_INITIALIZED if the storage was not initialized prior to this call

esp_err_t **nvs_flash_deinit_partition**(const char *partition_label)

Deinitialize NVS storage for the given NVS partition.

Parameters **partition_label** -- **[in]** Label of the partition

Returns
- ESP_OK on success

- ESP_ERR_NVS_NOT_INITIALIZED if the storage for given partition was not initialized prior to this call

esp_err_t **nvs_flash_erase**(void)

Erase the default NVS partition.

Erases all contents of the default NVS partition (one with label "nvs").

Note: If the partition is initialized, this function first de-initializes it. Afterwards, the partition has to be initialized again to be used.

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if there is no NVS partition labeled "nvs" in the partition table
- different error in case de-initialization fails (shouldn't happen)

esp_err_t **nvs_flash_erase_partition**(const char *part_name)

Erase specified NVS partition.

Erase all content of a specified NVS partition

Note: If the partition is initialized, this function first de-initializes it. Afterwards, the partition has to be initialized again to be used.

Parameters **part_name** -- **[in]** Name (label) of the partition which should be erased

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if there is no NVS partition with the specified name in the partition table
- different error in case de-initialization fails (shouldn't happen)

esp_err_t **nvs_flash_erase_partition_ptr**(const *esp_partition_t* *partition)

Erase custom partition.

Erase all content of specified custom partition.

Note: If the partition is initialized, this function first de-initializes it. Afterwards, the partition has to be initialized again to be used.

Parameters **partition** -- **[in]** pointer to a partition obtained by the ESP partition API.

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if there is no partition with the specified parameters in the partition table
- ESP_ERR_INVALID_ARG in case partition is NULL
- one of the error codes from the underlying flash storage driver

esp_err_t **nvs_flash_secure_init**(*nvs_sec_cfg_t* *cfg)

Initialize the default NVS partition.

This API initialises the default NVS partition. The default NVS partition is the one that is labeled "nvs" in the partition table.

Parameters **cfg** -- **[in]** Security configuration (keys) to be used for NVS encryption/decryption. If cfg is NULL, no encryption is used.

Returns

- ESP_OK if storage has been initialized successfully.
- ESP_ERR_NVS_NO_FREE_PAGES if the NVS storage contains no empty pages (which may happen if NVS partition was truncated)
- ESP_ERR_NOT_FOUND if no partition with label "nvs" is found in the partition table
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- one of the error codes from the underlying flash storage driver

esp_err_t **nvs_flash_secure_init_partition** (const char *partition_label, *nvs_sec_cfg_t* *cfg)

Initialize NVS flash storage for the specified partition.

Parameters

- **partition_label** -- **[in]** Label of the partition. Note that internally, a reference to passed value is kept and it should be accessible for future operations
- **cfg** -- **[in]** Security configuration (keys) to be used for NVS encryption/decryption. If cfg is null, no encryption/decryption is used.

Returns

- ESP_OK if storage has been initialized successfully.
- ESP_ERR_NVS_NO_FREE_PAGES if the NVS storage contains no empty pages (which may happen if NVS partition was truncated)
- ESP_ERR_NOT_FOUND if specified partition is not found in the partition table
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- one of the error codes from the underlying flash storage driver

esp_err_t **nvs_flash_generate_keys** (const *esp_partition_t* *partition, *nvs_sec_cfg_t* *cfg)

Generate and store NVS keys in the provided esp partition.

Parameters

- **partition** -- **[in]** Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **cfg** -- **[out]** Pointer to nvs security configuration structure. Pointer must be non-NULL. Generated keys will be populated in this structure.

Returns

- ESP_OK, if cfg was read successfully;
- ESP_ERR_INVALID_ARG, if partition or cfg is NULL;
- or error codes from esp_partition_write/erase APIs.

esp_err_t **nvs_flash_read_security_cfg** (const *esp_partition_t* *partition, *nvs_sec_cfg_t* *cfg)

Read NVS security configuration from a partition.

Note: Provided partition is assumed to be marked 'encrypted'.

Parameters

- **partition** -- **[in]** Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **cfg** -- **[out]** Pointer to nvs security configuration structure. Pointer must be non-NULL.

Returns

- ESP_OK, if cfg was read successfully;
- ESP_ERR_INVALID_ARG, if partition or cfg is NULL
- ESP_ERR_NVS_KEYS_NOT_INITIALIZED, if the partition is not yet written with keys.
- ESP_ERR_NVS_CORRUPT_KEY_PART, if the partition containing keys is found to be corrupt
- or error codes from esp_partition_read API.

esp_err_t **nvs_flash_register_security_scheme** (*nvs_sec_scheme_t* *scheme_cfg)

Registers the given security scheme for NVS encryption The scheme registered with sec_scheme_id by this API be used as the default security scheme for the "nvs" partition. Users will have to call this API explicitly in their application.

Parameters **scheme_cfg** -- **[in]** Pointer to the security scheme configuration structure that the user (or the nvs_key_provider) wants to register.

Returns

- ESP_OK, if security scheme registration succeeds;
- ESP_ERR_INVALID_ARG, if scheme_cfg is NULL;
- ESP_FAIL, if security scheme registration fails

nvs_sec_scheme_t ***nvs_flash_get_default_security_scheme**(void)

Fetch the configuration structure for the default active security scheme for NVS encryption.

Returns Pointer to the default active security scheme configuration (NULL if no scheme is registered yet i.e. active)

esp_err_t **nvs_flash_generate_keys_v2**(*nvs_sec_scheme_t* *scheme_cfg, *nvs_sec_cfg_t* *cfg)

Generate (and store) the NVS keys using the specified key-protection scheme.

Parameters

- **scheme_cfg** -- **[in]** Security scheme specific configuration
- **cfg** -- **[out]** Security configuration (encryption keys)

Returns

- ESP_OK, if cfg was populated successfully with generated encryption keys;
- ESP_ERR_INVALID_ARG, if scheme_cfg or cfg is NULL;
- ESP_FAIL, if the key generation process fails

esp_err_t **nvs_flash_read_security_cfg_v2**(*nvs_sec_scheme_t* *scheme_cfg, *nvs_sec_cfg_t* *cfg)

Read NVS security configuration set by the specified security scheme.

Parameters

- **scheme_cfg** -- **[in]** Security scheme specific configuration
- **cfg** -- **[out]** Security configuration (encryption keys)

Returns

- ESP_OK, if cfg was read successfully;
- ESP_ERR_INVALID_ARG, if scheme_cfg or cfg is NULL;
- ESP_FAIL, if the key reading process fails

Structures

struct **nvs_sec_cfg_t**

Key for encryption and decryption.

Public Members

uint8_t **eky**[NVS_KEY_SIZE]

XTS encryption and decryption key

uint8_t **tky**[NVS_KEY_SIZE]

XTS tweak key

struct **nvs_sec_scheme_t**

NVS encryption: Security scheme configuration structure.

Public Members

int **scheme_id**

Security Scheme ID (E.g. HMAC)

void ***scheme_data***

Scheme-specific data (E.g. eFuse block for HMAC-based key generation)

nvs_flash_generate_keys_t **nvs_flash_key_gen**

Callback for the nvs_flash_key_gen implementation

nvs_flash_read_cfg_t **nvs_flash_read_cfg**

Callback for the nvs_flash_read_keys implementation

Macros

NVS_KEY_SIZE

Type Definitions

typedef *esp_err_t* (***nvs_flash_generate_keys_t***)(const void *scheme_data, *nvs_sec_cfg_t* *cfg)

Callback function prototype for generating the NVS encryption keys.

typedef *esp_err_t* (***nvs_flash_read_cfg_t***)(const void *scheme_data, *nvs_sec_cfg_t* *cfg)

Callback function prototype for reading the NVS encryption keys.

Header File

- components/nvs_flash/include/nvs.h
- This header file can be included with:

```
#include "nvs.h"
```

- This header file is a part of the API provided by the `nvs_flash` component. To declare that your component depends on `nvs_flash`, add the following to your CMakeLists.txt:

```
REQUIRES nvs_flash
```

or

```
PRIV_REQUIRES nvs_flash
```

Functions

esp_err_t **nvs_set_i8** (*nvs_handle_t* handle, const char *key, int8_t value)

set int8_t value for given key

Set value for the key, given its name. Note that the actual storage will not be updated until `nvs_commit` is called. Regardless whether key-value pair is created or updated, function always requires at least one nvs available entry. See `nvs_get_stats` . After create type of operation, the number of available entries is decreased by one. After update type of operation, the number of available entries remains the same.

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function. Handles that were opened read only cannot be used.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **value** -- **[in]** The value to set.

Returns

- ESP_OK if value was set successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)

- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_READ_ONLY if storage handle was opened as read only
- ESP_ERR_NVS_INVALID_NAME if key name doesn't satisfy constraints
- ESP_ERR_NVS_NOT_ENOUGH_SPACE if there is not enough space in the underlying storage to save the value
- ESP_ERR_NVS_REMOVE_FAILED if the value wasn't updated because flash write operation has failed. The value was written however, and update will be finished after re-initialization of nvs, provided that flash operation doesn't fail again.

esp_err_t **nvs_set_u8** (*nvs_handle_t* handle, const char *key, uint8_t value)

set uint8_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_i16** (*nvs_handle_t* handle, const char *key, int16_t value)

set int16_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_u16** (*nvs_handle_t* handle, const char *key, uint16_t value)

set uint16_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_i32** (*nvs_handle_t* handle, const char *key, int32_t value)

set int32_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_u32** (*nvs_handle_t* handle, const char *key, uint32_t value)

set uint32_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_i64** (*nvs_handle_t* handle, const char *key, int64_t value)

set int64_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_u64** (*nvs_handle_t* handle, const char *key, uint64_t value)

set uint64_t value for given key

This function is the same as `nvs_set_i8` except for the data type.

esp_err_t **nvs_set_str** (*nvs_handle_t* handle, const char *key, const char *value)

set string for given key

Sets string value for the key. Function requires whole space for new data to be available as contiguous entries in same nvs page. Operation consumes 1 overhead entry and 1 entry per each 32 characters of new string including zero character to be set. In case of value update for existing key, entries occupied by the previous value and overhead entry are returned to the pool of available entries. Note that storage of long string values can fail due to fragmentation of nvs pages even if `available_entries` returned by `nvs_get_stats` suggests enough overall space available. Note that the underlying storage will not be updated until `nvs_commit` is called.

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function. Handles that were opened read only cannot be used.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **value** -- **[in]** The value to set. For strings, the maximum length (including null character) is 4000 bytes, if there is one complete page free for writing. This decreases, however, if the free space is fragmented.

Returns

- ESP_OK if value was set successfully

- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_READ_ONLY if storage handle was opened as read only
- ESP_ERR_NVS_INVALID_NAME if key name doesn't satisfy constraints
- ESP_ERR_NVS_NOT_ENOUGH_SPACE if there is not enough space in the underlying storage to save the value
- ESP_ERR_NVS_REMOVE_FAILED if the value wasn't updated because flash write operation has failed. The value was written however, and update will be finished after re-initialization of nvs, provided that flash operation doesn't fail again.
- ESP_ERR_NVS_VALUE_TOO_LONG if the string value is too long

esp_err_t **nvs_get_i8** (*nvs_handle_t* handle, const char *key, int8_t *out_value)

get int8_t value for given key

These functions retrieve value for the key, given its name. If `key` does not exist, or the requested variable type doesn't match the type which was used when setting a value, an error is returned.

In case of any error, out_value is not modified.

`out_value` has to be a pointer to an already allocated variable of the given type.

```
// Example of using nvs_get_i32:
int32_t max_buffer_size = 4096; // default value
esp_err_t err = nvs_get_i32(my_handle, "max_buffer_size", &max_buffer_size);
assert(err == ESP_OK || err == ESP_ERR_NVS_NOT_FOUND);
// if ESP_ERR_NVS_NOT_FOUND was returned, max_buffer_size will still
// have its default value.
```

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **out_value** -- Pointer to the output value. May be NULL for nvs_get_str and nvs_get_blob, in this case required length will be returned in length argument.

Returns

- ESP_OK if the value was retrieved successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_NOT_FOUND if the requested key doesn't exist
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_INVALID_NAME if key name doesn't satisfy constraints
- ESP_ERR_NVS_INVALID_LENGTH if length is not sufficient to store data

esp_err_t **nvs_get_u8** (*nvs_handle_t* handle, const char *key, uint8_t *out_value)

get uint8_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_i16** (*nvs_handle_t* handle, const char *key, int16_t *out_value)

get int16_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_u16** (*nvs_handle_t* handle, const char *key, uint16_t *out_value)

get uint16_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_i32** (*nvs_handle_t* handle, const char *key, int32_t *out_value)

get int32_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_u32** (*nvs_handle_t* handle, const char *key, uint32_t *out_value)

get uint32_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_i64** (*nvs_handle_t* handle, const char *key, int64_t *out_value)

get int64_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_u64** (*nvs_handle_t* handle, const char *key, uint64_t *out_value)

get uint64_t value for given key

This function is the same as `nvs_get_i8` except for the data type.

esp_err_t **nvs_get_str** (*nvs_handle_t* handle, const char *key, char *out_value, size_t *length)

get string value for given key

These functions retrieve the data of an entry, given its key. If key does not exist, or the requested variable type doesn't match the type which was used when setting a value, an error is returned.

In case of any error, out_value is not modified.

All functions expect out_value to be a pointer to an already allocated variable of the given type.

nvs_get_str and nvs_get_blob functions support WinAPI-style length queries. To get the size necessary to store the value, call nvs_get_str or nvs_get_blob with zero out_value and non-zero pointer to length. Variable pointed to by length argument will be set to the required length. For nvs_get_str, this length includes the zero terminator. When calling nvs_get_str and nvs_get_blob with non-zero out_value, length has to be non-zero and has to point to the length available in out_value. It is suggested that nvs_get/set_str is used for zero-terminated C strings, and nvs_get/set_blob used for arbitrary data structures.

```
// Example (without error checking) of using nvs_get_str to get a string into␣
↪dynamic array:
size_t required_size;
nvs_get_str(my_handle, "server_name", NULL, &required_size);
char* server_name = malloc(required_size);
nvs_get_str(my_handle, "server_name", server_name, &required_size);

// Example (without error checking) of using nvs_get_blob to get a binary data
into a static array:
uint8_t mac_addr[6];
size_t size = sizeof(mac_addr);
nvs_get_blob(my_handle, "dst_mac_addr", mac_addr, &size);
```

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **out_value** -- **[out]** Pointer to the output value. May be NULL for nvs_get_str and nvs_get_blob, in this case required length will be returned in length argument.
- **length** -- **[inout]** A non-zero pointer to the variable holding the length of out_value. In case out_value a zero, will be set to the length required to hold the value. In case out_value is not zero, will be set to the actual length of the value written. For nvs_get_str this includes zero terminator.

Returns

- ESP_OK if the value was retrieved successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_NOT_FOUND if the requested key doesn't exist
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_INVALID_NAME if key name doesn't satisfy constraints
- ESP_ERR_NVS_INVALID_LENGTH if `length` is not sufficient to store data

esp_err_t **nvs_get_blob**(*nvs_handle_t* handle, const char *key, void *out_value, size_t *length)

get blob value for given key

This function behaves the same as `nvs_get_str`, except for the data type.

esp_err_t **nvs_open**(const char *namespace_name, *nvs_open_mode_t* open_mode, *nvs_handle_t* *out_handle)

Open non-volatile storage with a given namespace from the default NVS partition.

Multiple internal ESP-IDF and third party application modules can store their key-value pairs in the NVS module. In order to reduce possible conflicts on key names, each module can use its own namespace. The default NVS partition is the one that is labelled "nvs" in the partition table.

Parameters

- **namespace_name** -- **[in]** Namespace name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **open_mode** -- **[in]** NVS_READWRITE or NVS_READONLY. If NVS_READONLY, will open a handle for reading only. All write requests will be rejected for this handle.
- **out_handle** -- **[out]** If successful (return code is zero), handle will be returned in this argument.

Returns

- ESP_OK if storage handle was opened successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_NOT_INITIALIZED if the storage driver is not initialized
- ESP_ERR_NVS_PART_NOT_FOUND if the partition with label "nvs" is not found
- ESP_ERR_NVS_NOT_FOUND id namespace doesn't exist yet and mode is NVS_READONLY
- ESP_ERR_NVS_INVALID_NAME if namespace name doesn't satisfy constraints
- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- ESP_ERR_NVS_NOT_ENOUGH_SPACE if there is no space for a new entry or there are too many different namespaces (maximum allowed different namespaces: 254)
- ESP_ERR_NOT_ALLOWED if the NVS partition is read-only and mode is NVS_READWRITE
- ESP_ERR_INVALID_ARG if out_handle is equal to NULL
- other error codes from the underlying storage driver

esp_err_t **nvs_open_from_partition**(const char *part_name, const char *namespace_name, *nvs_open_mode_t* open_mode, *nvs_handle_t* *out_handle)

Open non-volatile storage with a given namespace from specified partition.

The behaviour is same as nvs_open() API. However this API can operate on a specified NVS partition instead of default NVS partition. Note that the specified partition must be registered with NVS using nvs_flash_init_partition() API.

Parameters

- **part_name** -- **[in]** Label (name) of the partition of interest for object read/write/erase
- **namespace_name** -- **[in]** Namespace name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **open_mode** -- **[in]** NVS_READWRITE or NVS_READONLY. If NVS_READONLY, will open a handle for reading only. All write requests will be rejected for this handle.
- **out_handle** -- **[out]** If successful (return code is zero), handle will be returned in this argument.

Returns

- ESP_OK if storage handle was opened successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_NOT_INITIALIZED if the storage driver is not initialized
- ESP_ERR_NVS_PART_NOT_FOUND if the partition with specified name is not found
- ESP_ERR_NVS_NOT_FOUND id namespace doesn't exist yet and mode is NVS_READONLY
- ESP_ERR_NVS_INVALID_NAME if namespace name doesn't satisfy constraints

- ESP_ERR_NO_MEM in case memory could not be allocated for the internal structures
- ESP_ERR_NVS_NOT_ENOUGH_SPACE if there is no space for a new entry or there are too many different namespaces (maximum allowed different namespaces: 254)
- ESP_ERR_NOT_ALLOWED if the NVS partition is read-only and mode is NVS_READWRITE
- ESP_ERR_INVALID_ARG if out_handle is equal to NULL
- other error codes from the underlying storage driver

esp_err_t **nvs_set_blob** (*nvs_handle_t* handle, const char *key, const void *value, size_t length)

set variable length binary value for given key

Sets variable length binary value for the key. Function uses 2 overhead and 1 entry per each 32 bytes of new data from the pool of available entries. See `nvs_get_stats` . In case of value update for existing key, space occupied by the existing value and 2 overhead entries are returned to the pool of available entries. Note that the underlying storage will not be updated until `nvs_commit` is called.

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function. Handles that were opened read only cannot be used.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **value** -- **[in]** The value to set.
- **length** -- **[in]** length of binary value to set, in bytes; Maximum length is 508000 bytes or (97.6% of the partition size - 4000) bytes whichever is lower.

Returns

- ESP_OK if value was set successfully
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_READ_ONLY if storage handle was opened as read only
- ESP_ERR_NVS_INVALID_NAME if key name doesn't satisfy constraints
- ESP_ERR_NVS_NOT_ENOUGH_SPACE if there is not enough space in the underlying storage to save the value
- ESP_ERR_NVS_REMOVE_FAILED if the value wasn't updated because flash write operation has failed. The value was written however, and update will be finished after re-initialization of nvs, provided that flash operation doesn't fail again.
- ESP_ERR_NVS_VALUE_TOO_LONG if the value is too long

esp_err_t **nvs_find_key** (*nvs_handle_t* handle, const char *key, *nvs_type_t* *out_type)

Lookup key-value pair with given key name.

Note that function may indicate both existence of the key as well as the data type of NVS entry if it is found.

Parameters

- **handle** -- **[in]** Storage handle obtained with nvs_open.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.
- **out_type** -- **[out]** Pointer to the output variable populated with data type of NVS entry in case key was found. May be NULL, respective data type is then not provided.

Returns

- ESP_OK if NVS entry for key provided was found
- ESP_ERR_NVS_NOT_FOUND if the requested key doesn't exist
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- other error codes from the underlying storage driver

esp_err_t **nvs_erase_key** (*nvs_handle_t* handle, const char *key)

Erase key-value pair with given key name.

Note that actual storage may not be updated until nvs_commit function is called.

Parameters

- **handle** -- **[in]** Storage handle obtained with nvs_open. Handles that were opened read only cannot be used.
- **key** -- **[in]** Key name. Maximum length is (NVS_KEY_NAME_MAX_SIZE-1) characters. Shouldn't be empty.

Returns

- ESP_OK if erase operation was successful
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_READ_ONLY if handle was opened as read only
- ESP_ERR_NVS_NOT_FOUND if the requested key doesn't exist
- other error codes from the underlying storage driver

esp_err_t **nvs_erase_all**(*nvs_handle_t* handle)

Erase all key-value pairs in a namespace.

Note that actual storage may not be updated until nvs_commit function is called.

Parameters **handle** -- **[in]** Storage handle obtained with nvs_open. Handles that were opened read only cannot be used.

Returns

- ESP_OK if erase operation was successful
- ESP_FAIL if there is an internal error; most likely due to corrupted NVS partition (only if NVS assertion checks are disabled)
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- ESP_ERR_NVS_READ_ONLY if handle was opened as read only
- other error codes from the underlying storage driver

esp_err_t **nvs_commit**(*nvs_handle_t* handle)

Write any pending changes to non-volatile storage.

After setting any values, nvs_commit() must be called to ensure changes are written to non-volatile storage. Individual implementations may write to storage at other times, but this is not guaranteed.

Parameters **handle** -- **[in]** Storage handle obtained with nvs_open. Handles that were opened read only cannot be used.

Returns

- ESP_OK if the changes have been written successfully
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL
- other error codes from the underlying storage driver

void **nvs_close**(*nvs_handle_t* handle)

Close the storage handle and free any allocated resources.

This function should be called for each handle opened with nvs_open once the handle is not in use any more. Closing the handle may not automatically write the changes to nonvolatile storage. This has to be done explicitly using nvs_commit function. Once this function is called on a handle, the handle should no longer be used.

Parameters **handle** -- **[in]** Storage handle to close

esp_err_t **nvs_get_stats**(const char *part_name, *nvs_stats_t* *nvs_stats)

Fill structure *nvs_stats_t*. It provides info about memory used by NVS.

This function calculates the number of used entries, free entries, available entries, total entries and number of namespaces in partition.

```
// Example of nvs_get_stats() to get overview of actual statistics of data␣
↪entries :
nvs_stats_t nvs_stats;
nvs_get_stats(NULL, &nvs_stats);
```

(continues on next page)

(continued from previous page)

```
printf("Count: UsedEntries = (%lu), FreeEntries = (%lu), AvailableEntries = (
↪%lu), AllEntries = (%lu)\n",
       nvs_stats.used_entries, nvs_stats.free_entries, nvs_stats.available_
↪entries, nvs_stats.total_entries);
```

Parameters

- **part_name** -- **[in]** Partition name NVS in the partition table. If pass a NULL than will use NVS_DEFAULT_PART_NAME ("nvs").
- **nvs_stats** -- **[out]** Returns filled structure nvs_states_t. It provides info about used memory the partition.

Returns

- ESP_OK if the changes have been written successfully. Return param nvs_stats will be filled.
- ESP_ERR_NVS_PART_NOT_FOUND if the partition with label "name" is not found. Return param nvs_stats will be filled 0.
- ESP_ERR_NVS_NOT_INITIALIZED if the storage driver is not initialized. Return param nvs_stats will be filled 0.
- ESP_ERR_INVALID_ARG if nvs_stats is equal to NULL.
- ESP_ERR_INVALID_STATE if there is page with the status of INVALID. Return param nvs_stats will be filled not with correct values because not all pages will be counted. Counting will be interrupted at the first INVALID page.

esp_err_t **nvs_get_used_entry_count** (*nvs_handle_t* handle, size_t *used_entries)

Calculate all entries in a namespace.

An entry represents the smallest storage unit in NVS. Strings and blobs may occupy more than one entry. Note that to find out the total number of entries occupied by the namespace, add one to the returned value used_entries (if err is equal to ESP_OK). Because the name space entry takes one entry.

```
// Example of nvs_get_used_entry_count() to get amount of all key-value pairs␣
↪in one namespace:
nvs_handle_t handle;
nvs_open("namespace1", NVS_READWRITE, &handle);
...
size_t used_entries;
size_t total_entries_namespace;
if(nvs_get_used_entry_count(handle, &used_entries) == ESP_OK){
// the total number of entries occupied by the namespace
    total_entries_namespace = used_entries + 1;
}
```

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function.
- **used_entries** -- **[out]** Returns amount of used entries from a namespace.

Returns

- ESP_OK if the changes have been written successfully. Return param used_entries will be filled valid value.
- ESP_ERR_NVS_NOT_INITIALIZED if the storage driver is not initialized. Return param used_entries will be filled 0.
- ESP_ERR_NVS_INVALID_HANDLE if handle has been closed or is NULL. Return param used_entries will be filled 0.
- ESP_ERR_INVALID_ARG if used_entries is equal to NULL.
- Other error codes from the underlying storage driver. Return param used_entries will be filled 0.

esp_err_t **nvs_entry_find** (const char *part_name, const char *namespace_name, *nvs_type_t* type, *nvs_iterator_t* *output_iterator)

Create an iterator to enumerate NVS entries based on one or more parameters.

```
// Example of listing all the key-value pairs of any type under specified␣
↪partition and namespace
 nvs_iterator_t it = NULL;
 esp_err_t res = nvs_entry_find(<nvs_partition_name>, <namespace>, NVS_TYPE_
↪ANY, &it);
 while(res == ESP_OK) {
nvs_entry_info_t info;
     nvs_entry_info(it, &info); // Can omit error check if parameters are␣
↪guaranteed to be non-NULL
     printf("key '%s', type '%d' \n", info.key, info.type);
     res = nvs_entry_next(&it);
 }
 nvs_release_iterator(it);
```

Parameters

- **part_name** -- **[in]** Partition name
- **namespace_name** -- **[in]** Set this value if looking for entries with a specific namespace. Pass NULL otherwise.
- **type** -- **[in]** One of nvs_type_t values.
- **output_iterator** -- **[out]** Set to a valid iterator to enumerate all the entries found. Set to NULL if no entry for specified criteria was found. If any other error except ESP_ERR_INVALID_ARG occurs, `output_iterator` is NULL, too. If ESP_ERR_INVALID_ARG occurs, `output_iterator` is not changed. If a valid iterator is obtained through this function, it has to be released using `nvs_release_iterator` when not used any more, unless ESP_ERR_INVALID_ARG is returned.

Returns

- ESP_OK if no internal error or programming error occurred.
- ESP_ERR_NVS_NOT_FOUND if no element of specified criteria has been found.
- ESP_ERR_NO_MEM if memory has been exhausted during allocation of internal structures.
- ESP_ERR_INVALID_ARG if any of the parameters is NULL. Note: don't release `output_iterator` in case ESP_ERR_INVALID_ARG has been returned

esp_err_t **nvs_entry_find_in_handle**(*nvs_handle_t* handle, *nvs_type_t* type, *nvs_iterator_t* *output_iterator)

Create an iterator to enumerate NVS entries based on a handle and type.

```
// Example of listing all the key-value pairs of any type under specified␣
↪handle (which defines a partition and namespace)
 nvs_iterator_t it = NULL;
 esp_err_t res = nvs_entry_find_in_handle(<nvs_handle>, NVS_TYPE_ANY, &it);
 while(res == ESP_OK) {
nvs_entry_info_t info;
     nvs_entry_info(it, &info); // Can omit error check if parameters are␣
↪guaranteed to be non-NULL
     printf("key '%s', type '%d' \n", info.key, info.type);
     res = nvs_entry_next(&it);
 }
 nvs_release_iterator(it);
```

Parameters

- **handle** -- **[in]** Handle obtained from nvs_open function.
- **type** -- **[in]** One of nvs_type_t values.
- **output_iterator** -- **[out]** Set to a valid iterator to enumerate all the entries found. Set to NULL if no entry for specified criteria was found. If

any other error except ESP_ERR_INVALID_ARG occurs, `output_iterator` is NULL, too. If ESP_ERR_INVALID_ARG occurs, `output_iterator` is not changed. If a valid iterator is obtained through this function, it has to be released using `nvs_release_iterator` when not used any more, unless ESP_ERR_INVALID_ARG is returned.

Returns

- ESP_OK if no internal error or programming error occurred.
- ESP_ERR_NVS_NOT_FOUND if no element of specified criteria has been found.
- ESP_ERR_NO_MEM if memory has been exhausted during allocation of internal structures.
- ESP_ERR_NVS_INVALID_HANDLE if unknown handle was specified.
- ESP_ERR_INVALID_ARG if output_iterator parameter is NULL. Note: don't release `output_iterator` in case ESP_ERR_INVALID_ARG has been returned

esp_err_t **nvs_entry_next** (*nvs_iterator_t* *iterator)

Advances the iterator to next item matching the iterator criteria.

Note that any copies of the iterator will be invalid after this call.

Parameters **iterator** -- **[inout]** Iterator obtained from nvs_entry_find or nvs_entry_find_in_handle function. Must be non-NULL. If any error except ESP_ERR_INVALID_ARG occurs, `iterator` is set to NULL. If ESP_ERR_INVALID_ARG occurs, `iterator` is not changed.

Returns

- ESP_OK if no internal error or programming error occurred.
- ESP_ERR_NVS_NOT_FOUND if no next element matching the iterator criteria.
- ESP_ERR_INVALID_ARG if `iterator` is NULL.
- Possibly other errors in the future for internal programming or flash errors.

esp_err_t **nvs_entry_info** (const *nvs_iterator_t* iterator, *nvs_entry_info_t* *out_info)

Fills *nvs_entry_info_t* structure with information about entry pointed to by the iterator.

Parameters

- **iterator** -- **[in]** Iterator obtained from nvs_entry_find or nvs_entry_find_in_handle function. Must be non-NULL.
- **out_info** -- **[out]** Structure to which entry information is copied.

Returns

- ESP_OK if all parameters are valid; current iterator data has been written to out_info
- ESP_ERR_INVALID_ARG if one of the parameters is NULL.

void **nvs_release_iterator** (*nvs_iterator_t* iterator)

Release iterator.

Parameters **iterator** -- **[in]** Release iterator obtained from nvs_entry_find or nvs_entry_find_in_handle or nvs_entry_next function. NULL argument is allowed.

Structures

struct **nvs_entry_info_t**

information about entry obtained from nvs_entry_info function

Public Members

char **namespace_name**[NVS_NS_NAME_MAX_SIZE]

Namespace to which key-value belong

char **key**[NVS_KEY_NAME_MAX_SIZE]

Key of stored key-value pair

nvs_type_t **type**

Type of stored key-value pair

struct **nvs_stats_t**

Note: Info about storage space NVS.

Public Members

size_t **used_entries**

Number of used entries.

size_t **free_entries**

Number of free entries. It includes also reserved entries.

size_t **available_entries**

Number of entries available for data storage.

size_t **total_entries**

Number of all entries.

size_t **namespace_count**

Number of namespaces.

Macros

ESP_ERR_NVS_BASE

Starting number of error codes

ESP_ERR_NVS_NOT_INITIALIZED

The storage driver is not initialized

ESP_ERR_NVS_NOT_FOUND

A requested entry couldn't be found or namespace doesn’ t exist yet and mode is NVS_READONLY

ESP_ERR_NVS_TYPE_MISMATCH

The type of set or get operation doesn't match the type of value stored in NVS

ESP_ERR_NVS_READ_ONLY

Storage handle was opened as read only

ESP_ERR_NVS_NOT_ENOUGH_SPACE

There is not enough space in the underlying storage to save the value

ESP_ERR_NVS_INVALID_NAME

Namespace name doesn’t satisfy constraints

ESP_ERR_NVS_INVALID_HANDLE

Handle has been closed or is NULL

ESP_ERR_NVS_REMOVE_FAILED

The value wasn’t updated because flash write operation has failed. The value was written however, and update will be finished after re-initialization of nvs, provided that flash operation doesn’t fail again.

ESP_ERR_NVS_KEY_TOO_LONG

Key name is too long

ESP_ERR_NVS_PAGE_FULL

Internal error; never returned by nvs API functions

ESP_ERR_NVS_INVALID_STATE

NVS is in an inconsistent state due to a previous error. Call nvs_flash_init and nvs_open again, then retry.

ESP_ERR_NVS_INVALID_LENGTH

String or blob length is not sufficient to store data

ESP_ERR_NVS_NO_FREE_PAGES

NVS partition doesn't contain any empty pages. This may happen if NVS partition was truncated. Erase the whole partition and call nvs_flash_init again.

ESP_ERR_NVS_VALUE_TOO_LONG

Value doesn't fit into the entry or string or blob length is longer than supported by the implementation

ESP_ERR_NVS_PART_NOT_FOUND

Partition with specified name is not found in the partition table

ESP_ERR_NVS_NEW_VERSION_FOUND

NVS partition contains data in new format and cannot be recognized by this version of code

ESP_ERR_NVS_XTS_ENCR_FAILED

XTS encryption failed while writing NVS entry

ESP_ERR_NVS_XTS_DECR_FAILED

XTS decryption failed while reading NVS entry

ESP_ERR_NVS_XTS_CFG_FAILED

XTS configuration setting failed

ESP_ERR_NVS_XTS_CFG_NOT_FOUND

XTS configuration not found

ESP_ERR_NVS_ENCR_NOT_SUPPORTED

NVS encryption is not supported in this version

ESP_ERR_NVS_KEYS_NOT_INITIALIZED

NVS key partition is uninitialized

ESP_ERR_NVS_CORRUPT_KEY_PART

NVS key partition is corrupt

ESP_ERR_NVS_WRONG_ENCRYPTION

NVS partition is marked as encrypted with generic flash encryption. This is forbidden since the NVS encryption works differently.

ESP_ERR_NVS_CONTENT_DIFFERS

Internal error; never returned by nvs API functions. NVS key is different in comparison

NVS_DEFAULT_PART_NAME

Default partition name of the NVS partition in the partition table

NVS_PART_NAME_MAX_SIZE

maximum length of partition name (excluding null terminator)

NVS_KEY_NAME_MAX_SIZE

Maximum length of NVS key name (including null terminator)

NVS_NS_NAME_MAX_SIZE

Maximum length of NVS namespace name (including null terminator)

NVS_GUARD_SYSVIEW_MACRO_EXPANSION_PUSH()

NVS_GUARD_SYSVIEW_MACRO_EXPANSION_POP()

Type Definitions

typedef uint32_t **nvs_handle_t**

Opaque pointer type representing non-volatile storage handle

typedef *nvs_handle_t* **nvs_handle**

typedef *nvs_open_mode_t* **nvs_open_mode**

typedef struct nvs_opaque_iterator_t ***nvs_iterator_t**

Opaque pointer type representing iterator to nvs entries

Enumerations

enum **nvs_open_mode_t**

Mode of opening the non-volatile storage.

Values:

enumerator **NVS_READONLY**

Read only

enumerator **NVS_READWRITE**

Read and write

enum **nvs_type_t**

Types of variables.

Values:

enumerator **NVS_TYPE_U8**

Type uint8_t

enumerator **NVS_TYPE_I8**

Type int8_t

enumerator **NVS_TYPE_U16**

Type uint16_t

enumerator **NVS_TYPE_I16**

Type int16_t

enumerator **NVS_TYPE_U32**

Type uint32_t

enumerator **NVS_TYPE_I32**

Type int32_t

enumerator **NVS_TYPE_U64**

Type uint64_t

enumerator **NVS_TYPE_I64**

Type int64_t

enumerator **NVS_TYPE_STR**

Type string

enumerator **NVS_TYPE_BLOB**

Type blob

enumerator **NVS_TYPE_ANY**

Must be last

2.9.4 NVS Encryption

Overview

This guide provides an overview of the NVS encryption feature. NVS encryption helps to achieve secure storage on the device flash memory.

Data stored in NVS partitions can be encrypted using XTS-AES in the manner similar to the one mentioned in disk encryption standard IEEE P1619. For the purpose of encryption, each entry is treated as one `sector` and relative address of the entry (w.r.t., partition-start) is fed to the encryption algorithm as `sector-number`.

NVS encryption can be facilitated by enabling *CONFIG_NVS_ENCRYPTION* and *CONFIG_NVS_SEC_KEY_PROTECTION_SCHEME* > `CONFIG_NVS_SEC_KEY_PROTECT_USING_FLASH_ENC` or `CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC` depending on the scheme to be used.

NVS Encryption: Flash Encryption-Based Scheme

In this scheme, the keys required for NVS encryption are stored in yet another partition, which is protected using *Flash Encryption*. Therefore, enabling *Flash Encryption* becomes a prerequisite for NVS encryption here.

NVS encryption should be enabled when *Flash Encryption* is enabled because the Wi-Fi driver stores credentials (like SSID and passphrase) in the default NVS partition. It is important to encrypt them if platform-level encryption is already enabled.

For using NVS encryption using this scheme, the partition table must contain the *NVS Key Partition*. Two partition tables containing the *NVS Key Partition* are provided for NVS encryption under the partition table option (`menuconfig > Partition Table`). They can be selected with the project configuration menu (`idf.py menuconfig`). Please refer to the example security/flash_encryption for how to configure and use the NVS encryption feature.

NVS Key Partition An application requiring NVS encryption support (using the Flash Encryption-based scheme) needs to be compiled with a key-partition of the type `data` and subtype `key`. This partition should be marked as `encrypted` and its size should be the minimum partition size (4 KB). Refer to *Partition Tables* for more details. Two additional partition tables which contain the *NVS Key Partition* are provided under the partition table option (`menuconfig > Partition Table`). They can be directly used for NVS encryption. The structure of these partitions is depicted below:

```
+-----------+--------------+-------------+----+
|              XTS encryption key (32)         |
+----------------------------------------------+
|              XTS tweak key (32)              |
+----------------------------------------------+
|                  CRC32 (4)                   |
+----------------------------------------------+
```

The XTS encryption keys in the *NVS Key Partition* can be generated in one of the following two ways.

Generate the keys on ESP32-S3 chip itself

- When NVS encryption is enabled, the *`nvs_flash_init()`* API function can be used to initialize the encrypted default NVS partition. The API function internally generates the XTS encryption keys on the ESP chip. The API function finds the first *NVS Key Partition*.
- Then the API function automatically generates and stores the NVS keys in that partition by making use of the *`nvs_flash_generate_keys()`* API function provided by nvs_flash/include/nvs_flash.h. New keys are generated and stored only when the respective key partition is empty. The same key partition can then be used to read the security configurations for initializing a custom encrypted NVS partition with help of *`nvs_flash_secure_init_partition()`*.
- The API functions *`nvs_flash_secure_init()`* and *`nvs_flash_secure_init_partition()`* do not generate the keys internally. When these API functions are used for initializing encrypted NVS partitions, the keys can be generated after startup using the *`nvs_flash_generate_keys()`* API function provided by `nvs_flash.h`. The API function then writes those keys onto the key-partition in encrypted form.

Note: Please note that `nvs_keys` partition must be completely erased before you start the application in this approach. Otherwise the application may generate the *`ESP_ERR_NVS_CORRUPT_KEY_PART`*

error code assuming that `nvs_keys` partition is not empty and contains malformatted data. You can use the following command for this:

```
parttool.py --port PORT --partition-table-file=PARTITION_TABLE_FILE --
↪partition-table-offset PARTITION_TABLE_OFFSET erase_partition --
↪partition-type=data --partition-subtype=nvs_keys
```

Use a pre-generated NVS key partition

This option will be required by the user when keys in the *NVS Key Partition* are not generated by the application. The *NVS Key Partition* containing the XTS encryption keys can be generated with the help of *NVS Partition Generator Utility*. Then the user can store the pre-generated key partition on the flash with help of the following two commands:

1. Build and flash the partition table

```
idf.py partition-table partition-table-flash
```

2. Store the keys in the *NVS Key Partition* (on the flash) with the help of parttool.py (see Partition Tool section in *partition-tables* for more details)

```
parttool.py --port PORT --partition-table-offset PARTITION_TABLE_OFFSET␣
↪write_partition --partition-name="name of nvs_key partition" --input␣
↪NVS_KEY_PARTITION_FILE
```

Note: If the device is encrypted in flash encryption development mode and you want to renew the NVS key partition, you need to tell parttool.py to encrypt the NVS key partition and you also need to give it a pointer to the unencrypted partition table in your build directory (build/partition_table) since the partition table on the device is encrypted, too. You can use the following command:

```
parttool.py --esptool-write-args encrypt --port PORT --partition-table-
↪file=PARTITION_TABLE_FILE --partition-table-offset PARTITION_TABLE_
↪OFFSET write_partition --partition-name="name of nvs_key partition" --
↪input NVS_KEY_PARTITION_FILE
```

Since the key partition is marked as `encrypted` and *Flash Encryption* is enabled, the bootloader will encrypt this partition using flash encryption key on the first boot.

It is possible for an application to use different keys for different NVS partitions and thereby have multiple key-partitions. However, it is a responsibility of the application to provide the correct key-partition and keys for encryption or decryption.

NVS Encryption: HMAC Peripheral-Based Scheme

In this scheme, the XTS keys required for NVS encryption are derived from an HMAC key programmed in eFuse with the purpose *`esp_efuse_purpose_t::ESP_EFUSE_KEY_PURPOSE_HMAC_UP`*. Since the encryption keys are derived at runtime, they are not stored anywhere in the flash. Thus, this feature does not require a separate *NVS Key Partition*.

Note: This scheme enables us to achieve secure storage on ESP32-S3 **without enabling flash encryption**.

Important: Please take note that this scheme uses one eFuse block for storing the HMAC key required for deriving the encryption keys.

- When NVS encryption is enabled, the *nvs_flash_init()* API function can be used to initialize the encrypted default NVS partition. The API function first checks whether an HMAC key is present at *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID*.

Note: The valid range for the config *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID* is from 0 (*hmac_key_id_t::HMAC_KEY0*) to 5 (*hmac_key_id_t::HMAC_KEY5*). By default, the config is set to 6 (*hmac_key_id_t::HMAC_KEY_MAX*), which have to be configured before building the user application.

- If no key is found, a key is generated internally and stored at the eFuse block specified at *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID*.
- If a key is found with the purpose *esp_efuse_purpose_t::ESP_EFUSE_KEY_PURPOSE_HMAC_UP*, the same is used for the derivation of the XTS encryption keys.
- If the specified eFuse block is found to be occupied with a key with a purpose other than *esp_efuse_purpose_t::ESP_EFUSE_KEY_PURPOSE_HMAC_UP*, an error is thrown.
- The API *nvs_flash_init()* then automatically generates the NVS keys on demand by using the *nvs_flash_generate_keys_v2()* API function provided by the nvs_flash/include/nvs_flash.h. The same keys can also be used to read the security configurations (see *nvs_flash_read_security_cfg_v2()*) for initializing a custom encrypted NVS partition with help of *nvs_flash_secure_init_partition()*.
- The API functions *nvs_flash_secure_init()* and *nvs_flash_secure_init_partition()* do not generate the keys internally. When these API functions are used for initializing encrypted NVS partitions, the keys can be generated after startup using the *nvs_flash_generate_keys_v2()* API function or take and populate the NVS security configuration structure *nvs_sec_cfg_t* with *nvs_flash_read_security_cfg_v2()* and feed them into the above APIs.

Note: Users can program their own HMAC key in eFuse block beforehand by using the following command:

```
espefuse.py -p PORT burn_key <BLOCK_KEYN> <hmac_key_file.bin> HMAC_UP
```

Encrypted Read/Write

The same NVS API functions `nvs_get_*` or `nvs_set_*` can be used for reading of, and writing to an encrypted NVS partition as well.

Encrypt the default NVS partition

- To enable encryption for the default NVS partition, no additional step is necessary. When *CONFIG_NVS_ENCRYPTION* is enabled, the *nvs_flash_init()* API function internally performs some additional steps to enable encryption for the default NVS partition depending on the scheme being used (set by *CONFIG_NVS_SEC_KEY_PROTECTION_SCHEME*).
- For the flash encryption-based scheme, the first *NVS Key Partition* found is used to generate the encryption keys while for the HMAC one, keys are generated using the HMAC key burnt in eFuse at *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID* (refer to the API documentation for more details).

Alternatively, *nvs_flash_secure_init()* API function can also be used to enable encryption for the default NVS partition.

Encrypt a custom NVS partition

- To enable encryption for a custom NVS partition, *nvs_flash_secure_init_partition()* API function is used instead of *nvs_flash_init_partition()*.
- When *nvs_flash_secure_init()* and *nvs_flash_secure_init_partition()* API functions are used, the applications are expected to follow the steps below in order to perform NVS read/write operations with encryption enabled:
  1. Populate the NVS security configuration structure *nvs_sec_cfg_t*
     - For the Flash Encryption-based scheme

    * Find key partition and NVS data partition using `esp_partition_find*` API functions.
    * Populate the *`nvs_sec_cfg_t`* struct using the *`nvs_flash_read_security_cfg()`* or *`nvs_flash_generate_keys()`* API functions.
  - For the HMAC-based scheme
    * Set the scheme-specific config data with *`nvs_sec_config_hmac_t`* and register the HMAC-based scheme with the API *`nvs_sec_provider_register_hmac()`* which will also populate the scheme-specific handle (see *`nvs_sec_scheme_t`*).
    * Populate the *`nvs_sec_cfg_t`* struct using the *`nvs_flash_read_security_cfg_v2()`* or *`nvs_flash_generate_keys_v2()`* API functions.

```
nvs_sec_cfg_t cfg = {};
nvs_sec_scheme_t *sec_scheme_handle = NULL;

nvs_sec_config_hmac_t sec_scheme_cfg = {};
hmac_key_id_t hmac_key = HMAC_KEY0;
sec_scheme_cfg.hmac_key_id = hmac_key;

ret = nvs_sec_provider_register_hmac(&sec_scheme_cfg, &sec_scheme_
↪handle);
if (ret != ESP_OK) {
    return ret;
}

ret = nvs_flash_read_security_cfg_v2(sec_scheme_handle, &cfg);
if (ret != ESP_OK) {
    if (ret == ESP_ERR_NVS_SEC_HMAC_KEY_NOT_FOUND) {
        ret = nvs_flash_generate_keys_v2(&sec_scheme_handle, &cfg);
        if (ret != ESP_OK) {
            ESP_LOGE(TAG, "Failed to generate NVS encr-keys!");
            return ret;
        }
    }
    ESP_LOGE(TAG, "Failed to read NVS security cfg!");
    return ret;
}
```

2. Initialise NVS flash partition using the *`nvs_flash_secure_init()`* or *`nvs_flash_secure_init_partition()`* API functions.
3. Open a namespace using the *`nvs_open()`* or *`nvs_open_from_partition()`* API functions.
4. Perform NVS read/write operations using `nvs_get_*` or `nvs_set_*`.
5. Deinitialise an NVS partition using *`nvs_flash_deinit()`*.

Note: While using the HMAC-based scheme, the above workflow can be used without enabling any of the config options for NVS encryption - *CONFIG_NVS_ENCRYPTION*, *CONFIG_NVS_SEC_KEY_PROTECTION_SCHEME* -> `CONFIG_NVS_SEC_KEY_PROTECT_USING_HMAC` and *CONFIG_NVS_SEC_HMAC_EFUSE_KEY_ID* to encrypt the default as well as custom NVS partitions with *`nvs_flash_secure_init()`* API.

NVS Security Provider

The component nvs_sec_provider stores all the implementation-specific code for the NVS encryption schemes and would also accommodate any future schemes. This component acts as an interface to the nvs_flash component for the handling of encryption keys. nvs_sec_provider has a configuration menu of its own, based on which the selected security scheme and the corresponding settings are registered for the nvs_flash component.

This component offers factory functions with which a particular security scheme can be registered without having to worry about the APIs to generate and read the encryption keys (e.g.,

nvs_sec_provider_register_hmac()). Refer to the security/nvs_encryption_hmac example for API usage.

API Reference

Header File

- components/nvs_sec_provider/include/nvs_sec_provider.h
- This header file can be included with:

```
#include "nvs_sec_provider.h"
```

- This header file is a part of the API provided by the `nvs_sec_provider` component. To declare that your component depends on `nvs_sec_provider`, add the following to your CMakeLists.txt:

```
REQUIRES nvs_sec_provider
```

or

```
PRIV_REQUIRES nvs_sec_provider
```

Functions

esp_err_t **nvs_sec_provider_register_flash_enc**(const *nvs_sec_config_flash_enc_t* *sec_scheme_cfg, *nvs_sec_scheme_t* **sec_scheme_handle_out)

Register the Flash-Encryption based scheme for NVS Encryption.

Parameters

- **sec_scheme_cfg** -- **[in]** Security scheme specific configuration data
- **sec_scheme_handle_out** -- **[out]** Security scheme specific configuration handle

Returns

- ESP_OK, if `sec_scheme_handle_out` was populated successfully with the scheme configuration;
- ESP_ERR_INVALID_ARG, if `scheme_cfg_hmac` is NULL;
- ESP_ERR_NO_MEM, No memory for the scheme-specific handle `sec_scheme_handle_out`
- ESP_ERR_NOT_FOUND, if no `nvs_keys` partition is found

esp_err_t **nvs_sec_provider_register_hmac**(const *nvs_sec_config_hmac_t* *sec_scheme_cfg, *nvs_sec_scheme_t* **sec_scheme_handle_out)

Register the HMAC-based scheme for NVS Encryption.

Parameters

- **sec_scheme_cfg** -- **[in]** Security scheme specific configuration data
- **sec_scheme_handle_out** -- **[out]** Security scheme specific configuration handle

Returns

- ESP_OK, if `sec_scheme_handle_out` was populated successfully with the scheme configuration;
- ESP_ERR_INVALID_ARG, if `scheme_cfg_hmac` is NULL;
- ESP_ERR_NO_MEM, No memory for the scheme-specific handle `sec_scheme_handle_out`

esp_err_t **nvs_sec_provider_deregister**(*nvs_sec_scheme_t* *sec_scheme_handle)

Deregister the NVS encryption scheme registered with the given handle.

Parameters **sec_scheme_handle** -- **[in]** Security scheme specific configuration handle

Returns

- ESP_OK, if the scheme registered with `sec_scheme_handle` was deregistered successfully
- ESP_ERR_INVALID_ARG, if `sec_scheme_handle` is NULL;

Structures

struct **nvs_sec_config_flash_enc_t**

Flash encryption-based scheme specific configuration data.

Public Members

const *esp_partition_t* ***nvs_keys_part**

Partition of subtype `nvs_keys` holding the NVS encryption keys

struct **nvs_sec_config_hmac_t**

HMAC-based scheme specific configuration data.

Public Members

hmac_key_id_t **hmac_key_id**

HMAC Key ID used for generating the NVS encryption keys

Macros

ESP_ERR_NVS_SEC_BASE

Starting number of error codes

ESP_ERR_NVS_SEC_HMAC_KEY_NOT_FOUND

HMAC Key required to generate the NVS encryption keys not found

ESP_ERR_NVS_SEC_HMAC_KEY_BLK_ALREADY_USED

Provided eFuse block for HMAC key generation is already in use

ESP_ERR_NVS_SEC_HMAC_KEY_GENERATION_FAILED

Failed to generate/write the HMAC key to eFuse

ESP_ERR_NVS_SEC_HMAC_XTS_KEYS_DERIV_FAILED

Failed to derive the NVS encryption keys based on the HMAC-based scheme

NVS_SEC_PROVIDER_CFG_FLASH_ENC_DEFAULT()

Helper for populating the Flash encryption-based scheme specific configuration data.

NVS_SEC_PROVIDER_CFG_HMAC_DEFAULT()

Helper for populating the HMAC-based scheme specific configuration data.

Enumerations

enum **nvs_sec_scheme_id_t**

NVS Encryption Keys Protection Scheme.

Values:

enumerator **NVS_SEC_SCHEME_FLASH_ENC**

Protect NVS encryption keys using Flash Encryption

enumerator **NVS_SEC_SCHEME_HMAC**

Protect NVS encryption keys using HMAC peripheral

enumerator **NVS_SEC_SCHEME_MAX**

2.9.5 NVS Partition Generator Utility

Introduction

The utility nvs_flash/nvs_partition_generator/nvs_partition_gen.py creates a binary file, compatible with the NVS architecture defined in *Non-Volatile Storage Library*, based on the key-value pairs provided in a CSV file.

This utility is ideally suited for generating a binary blob, containing data specific to ODM/OEM, which can be flashed externally at the time of device manufacturing. This allows manufacturers to generate many instances of the same application firmware with customized parameters for each device, such as a serial number.

Prerequisites

To use this utility in encryption mode, install the following packages:

- `cryptography`

All the required packages are included in *requirements.txt* in the root of the ESP-IDF directory.

CSV File Format Each line of a CSV file should contain 4 parameters, separated by a comma. The table below describes each of these parameters.

No.	Parameter	Description	Notes
1	Key	Key of the data. The data can be accessed later from an application using this key.	
2	Type	Supported values are `file`, `data`, and `namespace`.	
3	Encoding	Supported values are: `u8`, `i8`, `u16`, `i16`, `u32`, `i32`, `u64`, `i64`, `string`, `hex2bin`, `base64`, and `binary`. This specifies how actual data values are encoded in the resulting binary file. The difference between the `string` and `binary` encoding is that `string` data is terminated with a NULL character, whereas `binary` data is not.	As of now, for the `file` type, only `hex2bin`, `base64`, `string`, and `binary` encoding is supported.
4	Value	Data value	`Encoding` and `Value` cells for the `namespace` field type should be empty. `Encoding` and `Value` of `namespace` are fixed and are not configurable. Any values in these cells are ignored.

Note: The first line of the CSV file should always be the column header and it is not configurable.

Below is an example dump of such a CSV file:

```
key,type,encoding,value      <-- column header
namespace_name,namespace,,   <-- First entry should be of type "namespace"
key1,data,u8,1
key2,file,string,/path/to/file
```

Note:

Make sure there are no spaces:

- before and after ','
- at the end of each line in a CSV file

NVS Entry and Namespace Association

When a namespace entry is encountered in a CSV file, each following entry will be treated as part of that namespace until the next namespace entry is found. At this point, all the following entries will be treated as part of the new namespace.

Note: First entry in a CSV file should always be a `namespace` entry.

Multipage Blob Support

By default, binary blobs are allowed to span over multiple pages and are written in the format mentioned in Section *Structure of Entry*. If you intend to use the older format, the utility provides an option to disable this feature.

Encryption-Decryption Support

The NVS Partition Generator utility also allows you to create an encrypted binary file and decrypt an encrypted one. The utility uses the XTS-AES encryption. Please refer to *NVS Encryption* for more details.

Running the Utility

Usage:

```
python nvs_partition_gen.py [-h] {generate,generate-key,encrypt,decrypt} ...
```

Optional Arguments:

No.	Parameter	Description
1	`-h` / `--help`	Show the help message and exit

Commands:

Run `nvs_partition_gen.py {command} -h` for additional help

No.	Parameter	Description
1	`generate`	Generate NVS partition
2	`generate-key`	Generate keys for encryption
3	`encrypt`	Generate NVS encrypted partition
4	`decrypt`	Decrypt NVS encrypted partition

Generate NVS Partition (Default) **Usage**:

```
python nvs_partition_gen.py generate [-h] [--version {1,2}] [--outdir OUTDIR]␣
↪input output size
```

Positional Arguments:

Parameter	Description
`input`	Path to CSV file to parse
`output`	Path to output NVS binary file
`size`	Size of NVS partition in bytes (must be multiple of 4096)

Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--version {1,2}`	Set multipage blob version (Default: Version 2) Version 1 - Multipage blob support disabled Version 2 - Multipage blob support enabled
`--outdir OUTDIR`	Output directory to store file created (Default: current directory)

You can run the utility to generate NVS partition using the command below. A sample CSV file is provided with the utility:

```
python nvs_partition_gen.py generate sample_singlepage_blob.csv sample.bin 0x3000
```

Generate Encryption Keys Partition **Usage**:

```
python nvs_partition_gen.py generate-key [-h] [--key_protect_hmac] [--kp_hmac_
↪keygen]
                                         [--kp_hmac_keyfile KP_HMAC_KEYFILE] [--
↪kp_hmac_inputkey KP_HMAC_INPUTKEY]
                                         [--keyfile KEYFILE] [--outdir OUTDIR]
```

Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--keyfile KEYFILE`	Path to output encryption keys file
`--outdir OUTDIR`	Output directory to store files created. (Default: current directory)

Optional Arguments (HMAC scheme-specific):

Parameter	Description
`--key_protect_hmac`	If set, the NVS encryption key protection scheme based on HMAC peripheral is used; else the default scheme based on flash encryption is used
`--kp_hmac_keygen`	Generate the HMAC key for HMAC-based encryption scheme
`--kp_hmac_keyfile KP_HMAC_KEYFILE`	Path to output the HMAC key file
`--kp_hmac_inputkey KP_HMAC_INPUTKEY`	File having the HMAC key for generating the NVS encryption keys

You can run the utility to generate only the encryption key partition using the command below:

```
python nvs_partition_gen.py generate-key
```

For generating encryption key for the HMAC-based scheme, the following commands can be used:

- Generate the HMAC key and the NVS encryption keys:

```
python nvs_partition_gen.py generate-key --key_protect_hmac --kp_hmac_keygen
```

Note: Encryption key of the format `<outdir>/keys/keys-<timestamp>.bin` and HMAC key of the format `<outdir>/keys/hmac-keys-<timestamp>.bin` are created.

- Generate the NVS encryption keys, given the HMAC key:

```
python nvs_partition_gen.py generate-key --key_protect_hmac --kp_hmac_inputkey␣
↪testdata/sample_hmac_key.bin
```

Note: You can provide the custom filename for the HMAC key as well as the encryption key as a parameter.

Generate Encrypted NVS Partition **Usage**:

```
python nvs_partition_gen.py encrypt [-h] [--version {1,2}] [--keygen]
                                    [--keyfile KEYFILE] [--inputkey INPUTKEY] [--
↪outdir OUTDIR]
                                    [--key_protect_hmac] [--kp_hmac_keygen]
                                    [--kp_hmac_keyfile KP_HMAC_KEYFILE] [--kp_hmac_
↪inputkey KP_HMAC_INPUTKEY]
                                    input output size
```

Positional Arguments:

Parameter	Description
`input`	Path to CSV file to parse
`output`	Path to output NVS binary file
`size`	Size of NVS partition in bytes (must be multiple of 4096)

Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--version {1,2}`	Set multipage blob version (Default: Version 2) Version 1 - Multipage blob support disabled Version 2 - Multipage blob support enabled
`--keygen`	Generates key for encrypting NVS partition
`--keyfile KEYFILE`	Path to output encryption keys file
`--inputkey INPUTKEY`	File having key for encrypting NVS partition
`--outdir OUTDIR`	Output directory to store file created (Default: current directory)

Optional Arguments (HMAC scheme-specific):

Parameter	Description
`--key_protect_hmac`	If set, the NVS encryption key protection scheme based on HMAC peripheral is used; else the default scheme based on flash encryption is used
`--kp_hmac_keygen`	Generate the HMAC key for HMAC-based encryption scheme
`--kp_hmac_keyfile KP_HMAC_KEYFILE`	Path to output HMAC key file
`--kp_hmac_inputkey KP_HMAC_INPUTKEY`	File having the HMAC key for generating the NVS encryption keys

You can run the utility to encrypt NVS partition using the command below. A sample CSV file is provided with the utility:

- Encrypt by allowing the utility to generate encryption keys:

```
python nvs_partition_gen.py encrypt sample_singlepage_blob.csv sample_encr.bin␣
↪0x3000 --keygen
```

Note: Encryption key of the format `<outdir>/keys/keys-<timestamp>.bin` is created.

- To generate an encrypted partition using the HMAC-based scheme, the above command can be used along with some additional parameters.
  - Encrypt by allowing the utility to generate encryption keys and the HMAC-key:

```
python nvs_partition_gen.py encrypt sample_singlepage_blob.csv sample_encr.
↪bin 0x3000 --keygen --key_protect_hmac --kp_hmac_keygen
```

Note: Encryption key of the format `<outdir>/keys/keys-<timestamp>.bin` and HMAC key of the format `<outdir>/keys/hmac-keys-<timestamp>.bin` are created.

  - Encrypt by allowing the utility to generate encryption keys with user-provided HMAC-key:

```
python nvs_partition_gen.py encrypt sample_singlepage_blob.csv sample_encr.
↪bin 0x3000 --keygen --key_protect_hmac --kp_hmac_inputkey testdata/
↪sample_hmac_key.bin
```

Note: You can provide the custom filename for the HMAC key as well as the encryption key as a parameter.

- Encrypt by allowing the utility to generate encryption keys and store it in provided custom filename:

```
python nvs_partition_gen.py encrypt sample_singlepage_blob.csv sample_encr.bin␣
↪0x3000 --keygen --keyfile sample_keys.bin
```

Note:

- Encryption key of the format `<outdir>/keys/sample_keys.bin` is created.
- This newly created file having encryption keys in `keys/` directory is compatible with NVS key-partition structure. Refer to *NVS Key Partition* for more details.

- Encrypt by providing the encryption keys as input binary file:

```
python nvs_partition_gen.py encrypt sample_singlepage_blob.csv sample_encr.bin␣
↪0x3000 --inputkey sample_keys.bin
```

Decrypt Encrypted NVS Partition **Usage**:

```
python nvs_partition_gen.py decrypt [-h] [--outdir OUTDIR] input key output
```

Positional Arguments:

Parameter	Description
`input`	Path to encrypted NVS partition file to parse
`key`	Path to file having keys for decryption
`output`	Path to output decrypted binary file

Optional Arguments:

Parameter	Description
`-h / --help`	Show the help message and exit
`--outdir OUTDIR`	Output directory to store files created. (Default: current directory)

You can run the utility to decrypt encrypted NVS partition using the command below:

```
python nvs_partition_gen.py decrypt sample_encr.bin sample_keys.bin sample_decr.bin
```

You can also provide the format version number:

- Multipage blob support disabled (Version 1)
- Multipage blob support enabled (Version 2)

Multipage Blob Support Disabled (Version 1) You can run the utility in this format by setting the version parameter to 1, as shown below. A sample CSV file for the same is provided with the utility:

```
python nvs_partition_gen.py generate sample_singlepage_blob.csv sample.bin 0x3000 -
↪-version 1
```

Multipage Blob Support Enabled (Version 2) You can run the utility in this format by setting the version parameter to 2, as shown below. A sample CSV file for the same is provided with the utility:

```
python nvs_partition_gen.py generate sample_multipage_blob.csv sample.bin 0x4000 --
↪version 2
```

Note:

- Minimum NVS Partition Size needed is 0x3000 bytes.
- When flashing the binary onto the device, make sure it is consistent with the application's sdkconfig.

Caveats

- Utility does not check for duplicate keys and will write data pertaining to both keys. You need to make sure that the keys are distinct.
- Once a new page is created, no data will be written in the space left on the previous page. Fields in the CSV file need to be ordered in such a way as to optimize memory.
- 64-bit datatype is not yet supported.

2.9.6 NVS Partition Parser Utility

Introduction

The utility nvs_flash/nvs_partition_tool/nvs_tool.py loads and parses an NVS storage partition for easier debugging and data extraction. The utility also features integrity check which scans the partition for potential errors. Data blobs are encoded in *base64* format.

Encrypted Partitions

This utility does not support decryption. To decrypt the NVS partition, please use the *NVS Partition Generator Utility* which does support NVS partition encryption and decryption.

Usage

There are two output format styles available with the *-f* or *--format* option:

- *json* - All of the output is printed as a JSON.
- *text* - The output is printed as a human-readable text with different selectable output styles mentioned below.

For the *text* output format, the utility provides six different output styles with the *-d* or *--dump* option:

- *all* (default) - Prints all entries with metadata.
- *written* - Prints only written entries with metadata.
- *minimal* - Prints written *namespace:key = value* pairs.
- *namespaces* - Prints all written namespaces
- *blobs* - Prints all blobs and strings (reconstructs them if they are chunked).
- *storage_info* - Prints entry states count for every page.

Note: There is also a *none* option which will not print anything. This can be used with the integrity check option if the NVS partition contents are irrelevant.

The utility also provides an integrity check feature via the *-i* or *--integrity-check* option (available only with the *text* format as it would invalidate the *json* output). This feature scans through the entire partition and prints potential errors. It can be used with the *-d none* option which will print only the potential errors.

2.9.7 SD/SDIO/MMC Driver

Overview

The SD/SDIO/MMC driver currently supports SD memory, SDIO cards, and eMMC chips. This is a protocol layer driver (sdmmc/include/sdmmc_cmd.h) which can be implemented by:

- SDMMC host driver (esp_driver_sdmmc/include/driver/sdmmc_host.h), see *SDMMC Host API* for more details.
- SDSPI host driver (esp_driver_sdspi/include/driver/sdspi_host.h), see *SD SPI Host API* for more details.

Protocol Layer vs Host Layer The SDMMC protocol layer described in this document handles the specifics of the SD protocol, such as the card initialization flow and variours data transfer command flows. The protocol layer works with the host via the `sdmmc_host_t` structure. This structure contains pointers to various functions of the host.

Host layer driver(s) implement the protocol layer driver by supporting these functions:

- Sending commands to slave devices

- Sending and receiving data
- Handling error conditions within the bus

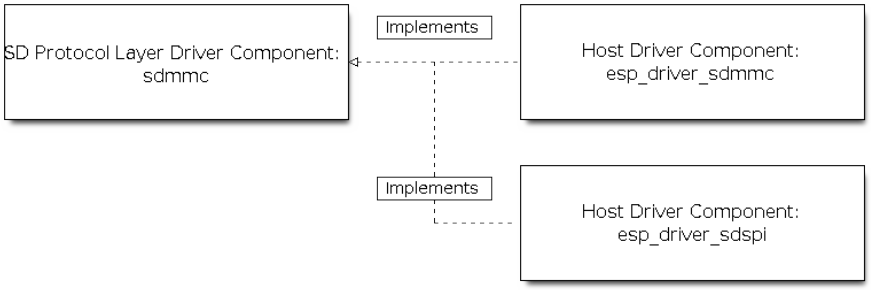


Fig. 40: SD Host Side Component Architecture

Application Example

An example which combines the SDMMC driver with the FATFS library is provided in the storage/sd_card directory of ESP-IDF examples. This example initializes the card, then writes and reads data from it using POSIX and C library APIs. See README.md file in the example directory for more information.

Combo (Memory + IO) Cards The driver does not support SD combo cards. Combo cards are treated as IO cards.

Thread Safety Most applications need to use the protocol layer only in one task. For this reason, the protocol layer does not implement any kind of locking on the `sdmmc_card_t` structure, or when accessing SDMMC or SD SPI host drivers. Such locking is usually implemented on a higher layer, e.g., in the filesystem driver.

Protocol Layer API

The protocol layer is given the `sdmmc_host_t` structure. This structure describes the SD/MMC host driver, lists its capabilities, and provides pointers to functions for the implementation driver. The protocol layer stores card-specific information in the `sdmmc_card_t` structure. When sending commands to the SD/MMC host driver, the protocol layer uses the `sdmmc_command_t` structure to describe the command, arguments, expected return values, and data to transfer if there is any.

Using API with SD Memory Cards

1. To initialize the host, call the host driver functions, e.g., *`sdmmc_host_init()`*, *`sdmmc_host_init_slot()`*.
2. To initialize the card, call *`sdmmc_card_init()`* and pass to it the parameters `host` - the host driver information, and `card` - a pointer to the structure `sdmmc_card_t` which will be filled with information about the card when the function completes.
3. To read and write sectors of the card, use *`sdmmc_read_sectors()`* and *`sdmmc_write_sectors()`* respectively and pass to it the parameter `card` - a pointer to the card information structure.
4. If the card is not used anymore, call the host driver function - e.g., *`sdmmc_host_deinit()`* - to disable the host peripheral and free the resources allocated by the driver.

Using API with eMMC Chips From the protocol layer's perspective, eMMC memory chips behave exactly like SD memory cards. Even though eMMCs are chips and do not have a card form factor, the terminology for SD cards can still be applied to eMMC due to the similarity of the protocol (*sdmmc_card_t*, *sdmmc_card_init*). Note that eMMC chips cannot be used over SPI, which makes them incompatible with the SD SPI host driver.

To initialize eMMC memory and perform read/write operations, follow the steps listed for SD cards in the previous section.

Using API with SDIO Cards Initialization and the probing process are the same as with SD memory cards. The only difference is in data transfer commands in SDIO mode.

During the card initialization and probing, performed with `sdmmc_card_init()`, the driver only configures the following registers of the IO card:

1. The IO portion of the card is reset by setting RES bit in the I/O Abort (0x06) register.
2. If 4-line mode is enabled in host and slot configuration, the driver attempts to set the Bus width field in the Bus Interface Control (0x07) register. If setting the filed is successful, which means that the slave supports 4-line mode, the host is also switched to 4-line mode.
3. If high-speed mode is enabled in the host configuration, the SHS bit is set in the High Speed (0x13) register.

In particular, the driver does not set any bits in (1) I/O Enable and Int Enable registers, (2) I/O block sizes, etc. Applications can set them by calling `sdmmc_io_write_byte()`.

For card configuration and data transfer, choose the pair of functions relevant to your case from the table below.

Action	Read Function	Write Function
Read and write a single byte using IO_RW_DIRECT (CMD52)	`sdmmc_io_read_byte()`	`sdmmc_io_write_byte()`
Read and write multiple bytes using IO_RW_EXTENDED (CMD53) in byte mode	`sdmmc_io_read_bytes()`	`sdmmc_io_write_bytes()`
Read and write blocks of data using IO_RW_EXTENDED (CMD53) in block mode	`sdmmc_io_read_blocks()`	`sdmmc_io_write_blocks()`

SDIO interrupts can be enabled by the application using the function `sdmmc_io_enable_int()`. When using SDIO in 1-line mode, the D1 line also needs to be connected to use SDIO interrupts.

If you want the application to wait until the SDIO interrupt occurs, use `sdmmc_io_wait_int()`.

API Reference

Header File

- components/sdmmc/include/sdmmc_cmd.h
- This header file can be included with:

```
#include "sdmmc_cmd.h"
```

- This header file is a part of the API provided by the `sdmmc` component. To declare that your component depends on `sdmmc`, add the following to your CMakeLists.txt:

```
REQUIRES sdmmc
```

or

```
PRIV_REQUIRES sdmmc
```

Functions

esp_err_t **sdmmc_card_init**(const sdmmc_host_t *host, sdmmc_card_t *out_card)

Probe and initialize SD/MMC card using given host

Note: Only SD cards (SDSC and SDHC/SDXC) are supported now. Support for MMC/eMMC cards will be added later.

Parameters
- **host** -- pointer to structure defining host controller
- **out_card** -- pointer to structure which will receive information about the card when the function completes

Returns
- ESP_OK on success
- One of the error codes from SDMMC host controller

void **sdmmc_card_print_info**(FILE *stream, const sdmmc_card_t *card)

Print information about the card to a stream.

Parameters
- **stream** -- stream obtained using fopen or fdopen
- **card** -- card information structure initialized using sdmmc_card_init

esp_err_t **sdmmc_get_status**(sdmmc_card_t *card)

Get status of SD/MMC card

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns
- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_write_sectors**(sdmmc_card_t *card, const void *src, size_t start_sector, size_t sector_count)

Write given number of sectors to SD/MMC card

Parameters
- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **src** -- pointer to data buffer to read data from; data size must be equal to sector_count * card->csd.sector_size
- **start_sector** -- sector where to start writing
- **sector_count** -- number of sectors to write

Returns
- ESP_OK on success or sector_count equal to 0
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_read_sectors**(sdmmc_card_t *card, void *dst, size_t start_sector, size_t sector_count)

Read given number of sectors from the SD/MMC card

Parameters
- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **dst** -- pointer to data buffer to write into; buffer size must be at least sector_count * card->csd.sector_size
- **start_sector** -- sector where to start reading
- **sector_count** -- number of sectors to read

Returns
- ESP_OK on success or sector_count equal to 0
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_erase_sectors**(sdmmc_card_t *card, size_t start_sector, size_t sector_count, sdmmc_erase_arg_t arg)

Erase given number of sectors from the SD/MMC card

Note: When sdmmc_erase_sectors used with cards in SDSPI mode, it was observed that card requires re-init after erase operation.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **start_sector** -- sector where to start erase
- **sector_count** -- number of sectors to erase
- **arg** -- erase command (CMD38) argument

Returns

- ESP_OK on success or sector_count equal to 0
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_can_discard**(sdmmc_card_t *card)

Check if SD/MMC card supports discard

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

esp_err_t **sdmmc_can_trim**(sdmmc_card_t *card)

Check if SD/MMC card supports trim

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

esp_err_t **sdmmc_mmc_can_sanitize**(sdmmc_card_t *card)

Check if SD/MMC card supports sanitize

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns

- ESP_OK if supported by the card/device
- ESP_FAIL if not supported by the card/device

esp_err_t **sdmmc_mmc_sanitize**(sdmmc_card_t *card, uint32_t timeout_ms)

Sanitize the data that was unmapped by a Discard command

Note: Discard command has to precede sanitize operation. To discard, use MMC_DICARD_ARG with sdmmc_erase_sectors argument

---

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **timeout_ms** -- timeout value in milliseconds required to sanitize the selected range of sectors.

Returns

- ESP_OK on success

- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_full_erase**(sdmmc_card_t *card)

Erase complete SD/MMC card

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_read_byte**(sdmmc_card_t *card, uint32_t function, uint32_t reg, uint8_t *out_byte)

Read one byte from an SDIO card using IO_RW_DIRECT (CMD52)

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **reg** -- byte address within IO function
- **out_byte** -- **[out]** output, receives the value read from the card

Returns

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_write_byte**(sdmmc_card_t *card, uint32_t function, uint32_t reg, uint8_t in_byte, uint8_t *out_byte)

Write one byte to an SDIO card using IO_RW_DIRECT (CMD52)

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **reg** -- byte address within IO function
- **in_byte** -- value to be written
- **out_byte** -- **[out]** if not NULL, receives new byte value read from the card (read-after-write).

Returns

- ESP_OK on success
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_read_bytes**(sdmmc_card_t *card, uint32_t function, uint32_t addr, void *dst, size_t size)

Read multiple bytes from an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs read operation using CMD53 in byte mode. For block mode, see sdmmc_io_read_blocks.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **addr** -- byte address within IO function where reading starts
- **dst** -- buffer which receives the data read from card. Aligned to 4 byte boundary unless `SDMMC_HOST_FLAG_ALLOC_ALIGNED_BUF` flag is set when calling `sdmmc_card_init`. The flag is mandatory when the buffer is behind the cache.
- **size** -- number of bytes to read, 1 to 512.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_SIZE if size exceeds 512 bytes

- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_write_bytes**(sdmmc_card_t *card, uint32_t function, uint32_t addr, const void *src, size_t size)

Write multiple bytes to an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs write operation using CMD53 in byte mode. For block mode, see sdmmc_io_write_blocks.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **addr** -- byte address within IO function where writing starts
- **src** -- data to be written. Aligned to 4 byte boundary unless `SDMMC_HOST_FLAG_ALLOC_ALIGNED_BUF` flag is set when calling `sdmmc_card_init`. The flag is mandatory when the buffer is behind the cache.
- **size** -- number of bytes to write, 1 to 512.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_SIZE if size exceeds 512 bytes
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_read_blocks**(sdmmc_card_t *card, uint32_t function, uint32_t addr, void *dst, size_t size)

Read blocks of data from an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs read operation using CMD53 in block mode. For byte mode, see sdmmc_io_read_bytes.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number
- **addr** -- byte address within IO function where writing starts
- **dst** -- buffer which receives the data read from card. Aligned to 4 byte boundary, and also cache line size if the buffer is behind the cache.
- **size** -- number of bytes to read, must be divisible by the card block size.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_SIZE if size is not divisible by 512 bytes
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_write_blocks**(sdmmc_card_t *card, uint32_t function, uint32_t addr, const void *src, size_t size)

Write blocks of data to an SDIO card using IO_RW_EXTENDED (CMD53)

This function performs write operation using CMD53 in block mode. For byte mode, see sdmmc_io_write_bytes.

By default OP Code is set (incrementing address). To send CMD53 without this bit, OR the argument `addr` with `SDMMC_IO_FIXED_ADDR`.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **function** -- IO function number

- **addr** -- byte address within IO function where writing starts
- **src** -- data to be written. Aligned to 4 byte boundary, and also cache line size if the buffer is behind the cache.
- **size** -- number of bytes to read, must be divisible by the card block size.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_SIZE if size is not divisible by 512 bytes
- One of the error codes from SDMMC host controller

esp_err_t **sdmmc_io_enable_int**(sdmmc_card_t *card)

Enable SDIO interrupt in the SDMMC host

Parameters **card** -- pointer to card information structure previously initialized using sdmmc_card_init

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if the host controller does not support IO interrupts

esp_err_t **sdmmc_io_wait_int**(sdmmc_card_t *card, TickType_t timeout_ticks)

Block until an SDIO interrupt is received

Slave uses D1 line to signal interrupt condition to the host. This function can be used to wait for the interrupt.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **timeout_ticks** -- time to wait for the interrupt, in RTOS ticks

Returns

- ESP_OK if the interrupt is received
- ESP_ERR_NOT_SUPPORTED if the host controller does not support IO interrupts
- ESP_ERR_TIMEOUT if the interrupt does not happen in timeout_ticks

esp_err_t **sdmmc_io_get_cis_data**(sdmmc_card_t *card, uint8_t *out_buffer, size_t buffer_size, size_t *inout_cis_size)

Get the data of CIS region of an SDIO card.

You may provide a buffer not sufficient to store all the CIS data. In this case, this function stores as much data into your buffer as possible. Also, this function will try to get and return the size required for you.

Parameters

- **card** -- pointer to card information structure previously initialized using sdmmc_card_init
- **out_buffer** -- Output buffer of the CIS data
- **buffer_size** -- Size of the buffer.
- **inout_cis_size** -- Mandatory, pointer to a size, input and output.
  - input: Limitation of maximum searching range, should be 0 or larger than buffer_size. The function searches for CIS_CODE_END until this range. Set to 0 to search infinitely.
  - output: The size required to store all the CIS data, if CIS_CODE_END is found.

Returns

- ESP_OK: on success
- ESP_ERR_INVALID_RESPONSE: if the card does not (correctly) support CIS.
- ESP_ERR_INVALID_SIZE: CIS_CODE_END found, but buffer_size is less than required size, which is stored in the inout_cis_size then.
- ESP_ERR_NOT_FOUND: if the CIS_CODE_END not found. Increase input value of inout_cis_size or set it to 0, if you still want to search for the end; output value of inout_cis_size is invalid in this case.
- and other error code return from sdmmc_io_read_bytes

esp_err_t **sdmmc_io_print_cis_info**(uint8_t *buffer, size_t buffer_size, FILE *fp)

Parse and print the CIS information of an SDIO card.

Note: Not all the CIS codes and all kinds of tuples are supported. If you see some unresolved code, you can add the parsing of these code in sdmmc_io.c and contribute to the IDF through the Github repository.

```
        using sdmmc_card_init
```

Parameters

- **buffer** -- Buffer to parse
- **buffer_size** -- Size of the buffer.
- **fp** -- File pointer to print to, set to NULL to print to stdout.

Returns

- ESP_OK: on success
- ESP_ERR_NOT_SUPPORTED: if the value from the card is not supported to be parsed.
- ESP_ERR_INVALID_SIZE: if the CIS size fields are not correct.

Macros

SDMMC_IO_FIXED_ADDR

Call `sdmmc_io_read_bytes`, `sdmmc_io_write_bytes`, `sdmmc_io_read_blocks` or `sdmmc_io_write_bocks` APIs with address ORed by this flag to send CMD53 with OP Code clear (fixed address)

Header File

- components/esp_driver_sdmmc/include/driver/sdmmc_types.h
- This header file can be included with:

```
#include "driver/sdmmc_types.h"
```

- This header file is a part of the API provided by the `esp_driver_sdmmc` component. To declare that your component depends on `esp_driver_sdmmc`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_sdmmc
```

or

```
PRIV_REQUIRES esp_driver_sdmmc
```

2.9.8 Partitions API

Overview

The `esp_partition` component has higher-level API functions which work with partitions defined in the *Partition Tables*. These APIs are based on lower level API provided by *SPI Flash API*.

Partition Table API

ESP-IDF projects use a partition table to maintain information about various regions of SPI flash memory (bootloader, various application binaries, data, filesystems). More information can be found in *Partition Tables*.

This component provides API functions to enumerate partitions found in the partition table and perform operations on them. These functions are declared in `esp_partition.h`:

- *`esp_partition_find()`* checks a partition table for entries with specific type, returns an opaque iterator.

- *esp_partition_get()* returns a structure describing the partition for a given iterator.
- *esp_partition_next()* shifts the iterator to the next found partition.
- *esp_partition_iterator_release()* releases iterator returned by *esp_partition_find()*.
- *esp_partition_find_first()* is a convenience function which returns the structure describing the first partition found by *esp_partition_find()*.
- *esp_partition_read()*, *esp_partition_write()*, *esp_partition_erase_range()* are equivalent to *esp_flash_read()*, *esp_flash_write()*, *esp_flash_erase_region()*, but operate within partition boundaries.

See Also

- *Partition Tables*
- *Over The Air Updates (OTA)* provides high-level API for updating applications stored in flash.
- *Non-Volatile Storage Library* provides a structured API for storing small pieces of data in SPI flash.

API Reference - Partition Table

Header File

- components/esp_partition/include/esp_partition.h
- This header file can be included with:

```
#include "esp_partition.h"
```

- This header file is a part of the API provided by the `esp_partition` component. To declare that your component depends on `esp_partition`, add the following to your CMakeLists.txt:

```
REQUIRES esp_partition
```

or

```
PRIV_REQUIRES esp_partition
```

Functions

esp_partition_iterator_t **esp_partition_find**(*esp_partition_type_t* type, *esp_partition_subtype_t* subtype, const char *label)

Find partition based on one or more parameters.

Parameters

- **type** -- Partition type, one of esp_partition_type_t values or an 8-bit unsigned integer. To find all partitions, no matter the type, use ESP_PARTITION_TYPE_ANY, and set subtype argument to ESP_PARTITION_SUBTYPE_ANY.
- **subtype** -- Partition subtype, one of esp_partition_subtype_t values or an 8-bit unsigned integer. To find all partitions of given type, use ESP_PARTITION_SUBTYPE_ANY.
- **label** -- (optional) Partition label. Set this value if looking for partition with a specific name. Pass NULL otherwise.

Returns iterator which can be used to enumerate all the partitions found, or NULL if no partitions were found. Iterator obtained through this function has to be released using esp_partition_iterator_release when not used any more.

const *esp_partition_t* ***esp_partition_find_first**(*esp_partition_type_t* type, *esp_partition_subtype_t* subtype, const char *label)

Find first partition based on one or more parameters.

Parameters

- **type** -- Partition type, one of esp_partition_type_t values or an 8-bit unsigned integer. To find all partitions, no matter the type, use ESP_PARTITION_TYPE_ANY, and set subtype argument to ESP_PARTITION_SUBTYPE_ANY.

- **subtype** -- Partition subtype, one of esp_partition_subtype_t values or an 8-bit unsigned integer To find all partitions of given type, use ESP_PARTITION_SUBTYPE_ANY.
- **label** -- (optional) Partition label. Set this value if looking for partition with a specific name. Pass NULL otherwise.

Returns pointer to *esp_partition_t* structure, or NULL if no partition is found. This pointer is valid for the lifetime of the application.

const *esp_partition_t* ***esp_partition_get**(*esp_partition_iterator_t* iterator)

Get *esp_partition_t* structure for given partition.

Parameters **iterator** -- Iterator obtained using esp_partition_find. Must be non-NULL.

Returns pointer to *esp_partition_t* structure. This pointer is valid for the lifetime of the application.

esp_partition_iterator_t **esp_partition_next**(*esp_partition_iterator_t* iterator)

Move partition iterator to the next partition found.

Any copies of the iterator will be invalid after this call.

Parameters **iterator** -- Iterator obtained using esp_partition_find. Must be non-NULL.

Returns NULL if no partition was found, valid esp_partition_iterator_t otherwise.

void **esp_partition_iterator_release**(*esp_partition_iterator_t* iterator)

Release partition iterator.

Parameters **iterator** -- Iterator obtained using esp_partition_find. The iterator is allowed to be NULL, so it is not necessary to check its value before calling this function.

const *esp_partition_t* ***esp_partition_verify**(const *esp_partition_t* *partition)

Verify partition data.

Given a pointer to partition data, verify this partition exists in the partition table (all fields match.)

This function is also useful to take partition data which may be in a RAM buffer and convert it to a pointer to the permanent partition data stored in flash.

Pointers returned from this function can be compared directly to the address of any pointer returned from *esp_partition_get()*, as a test for equality.

Parameters **partition** -- Pointer to partition data to verify. Must be non-NULL. All fields of this structure must match the partition table entry in flash for this function to return a successful match.

Returns

- If partition not found, returns NULL.
- If found, returns a pointer to the *esp_partition_t* structure in flash. This pointer is always valid for the lifetime of the application.

esp_err_t **esp_partition_read**(const *esp_partition_t* *partition, size_t src_offset, void *dst, size_t size)

Read data from the partition.

Partitions marked with an encryption flag will automatically be be read and decrypted via a cache mapping.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **dst** -- Pointer to the buffer where data should be stored. Pointer must be non-NULL and buffer must be at least 'size' bytes long.
- **src_offset** -- Address of the data to be read, relative to the beginning of the partition.
- **size** -- Size of data to be read, in bytes.

Returns ESP_OK, if data was read successfully; ESP_ERR_INVALID_ARG, if src_offset exceeds partition size; ESP_ERR_INVALID_SIZE, if read would go out of bounds of the partition; or one of error codes from lower-level flash driver.

esp_err_t **esp_partition_write**(const *esp_partition_t* *partition, size_t dst_offset, const void *src, size_t size)

Write data to the partition.

Before writing data to flash, corresponding region of flash needs to be erased. This can be done using esp_partition_erase_range function.

Partitions marked with an encryption flag will automatically be written via the esp_flash_write_encrypted() function. If writing to an encrypted partition, all write offsets and lengths must be multiples of 16 bytes. See the esp_flash_write_encrypted() function for more details. Unencrypted partitions do not have this restriction.

Note: Prior to writing to flash memory, make sure it has been erased with esp_partition_erase_range call.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **dst_offset** -- Address where the data should be written, relative to the beginning of the partition.
- **src** -- Pointer to the source buffer. Pointer must be non-NULL and buffer must be at least 'size' bytes long.
- **size** -- Size of data to be written, in bytes.

Returns ESP_OK, if data was written successfully; ESP_ERR_INVALID_ARG, if dst_offset exceeds partition size; ESP_ERR_INVALID_SIZE, if write would go out of bounds of the partition; ESP_ERR_NOT_ALLOWED, if partition is read-only; or one of error codes from lower-level flash driver.

esp_err_t **esp_partition_read_raw**(const *esp_partition_t* *partition, size_t src_offset, void *dst, size_t size)

Read data from the partition without any transformation/decryption.

Note: This function is essentially the same as *esp_partition_read()* above. It just never decrypts data but returns it as is.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **dst** -- Pointer to the buffer where data should be stored. Pointer must be non-NULL and buffer must be at least 'size' bytes long.
- **src_offset** -- Address of the data to be read, relative to the beginning of the partition.
- **size** -- Size of data to be read, in bytes.

Returns ESP_OK, if data was read successfully; ESP_ERR_INVALID_ARG, if src_offset exceeds partition size; ESP_ERR_INVALID_SIZE, if read would go out of bounds of the partition; or one of error codes from lower-level flash driver.

esp_err_t **esp_partition_write_raw**(const *esp_partition_t* *partition, size_t dst_offset, const void *src, size_t size)

Write data to the partition without any transformation/encryption.

Before writing data to flash, corresponding region of flash needs to be erased. This can be done using esp_partition_erase_range function.

Note: This function is essentially the same as *esp_partition_write()* above. It just never encrypts data but writes it as is.

Note: Prior to writing to flash memory, make sure it has been erased with esp_partition_erase_range call.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **dst_offset** -- Address where the data should be written, relative to the beginning of the partition.
- **src** -- Pointer to the source buffer. Pointer must be non-NULL and buffer must be at least 'size' bytes long.
- **size** -- Size of data to be written, in bytes.

Returns ESP_OK, if data was written successfully; ESP_ERR_INVALID_ARG, if dst_offset exceeds partition size; ESP_ERR_INVALID_SIZE, if write would go out of bounds of the partition; ESP_ERR_NOT_ALLOWED, if partition is read-only; or one of the error codes from lower-level flash driver.

esp_err_t **esp_partition_erase_range**(const *esp_partition_t* *partition, size_t offset, size_t size)

Erase part of the partition.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **offset** -- Offset from the beginning of partition where erase operation should start. Must be aligned to partition->erase_size.
- **size** -- Size of the range which should be erased, in bytes. Must be divisible by partition->erase_size.

Returns ESP_OK, if the range was erased successfully; ESP_ERR_INVALID_ARG, if iterator or dst are NULL; ESP_ERR_INVALID_SIZE, if erase would go out of bounds of the partition; ESP_ERR_NOT_ALLOWED, if partition is read-only; or one of error codes from lower-level flash driver.

esp_err_t **esp_partition_mmap**(const *esp_partition_t* *partition, size_t offset, size_t size, *esp_partition_mmap_memory_t* memory, const void **out_ptr, *esp_partition_mmap_handle_t* *out_handle)

Configure MMU to map partition into data memory.

Unlike spi_flash_mmap function, which requires a 64kB aligned base address, this function doesn't impose such a requirement. If offset results in a flash address which is not aligned to 64kB boundary, address will be rounded to the lower 64kB boundary, so that mapped region includes requested range. Pointer returned via out_ptr argument will be adjusted to point to the requested offset (not necessarily to the beginning of mmap-ed region).

To release mapped memory, pass handle returned via out_handle argument to esp_partition_munmap function.

Parameters

- **partition** -- Pointer to partition structure obtained using esp_partition_find_first or esp_partition_get. Must be non-NULL.
- **offset** -- Offset from the beginning of partition where mapping should start.
- **size** -- Size of the area to be mapped.
- **memory** -- Memory space where the region should be mapped
- **out_ptr** -- Output, pointer to the mapped memory region
- **out_handle** -- Output, handle which should be used for esp_partition_munmap call

Returns ESP_OK, if successful

void **esp_partition_munmap**(*esp_partition_mmap_handle_t* handle)

Release region previously obtained using esp_partition_mmap.

Note: Calling this function will not necessarily unmap memory region. Region will only be unmapped when there are no other handles which reference this region. In case of partially overlapping regions it is possible

that memory will be unmapped partially.

Parameters **handle** -- Handle obtained from spi_flash_mmap

esp_err_t **esp_partition_get_sha256**(const *esp_partition_t* *partition, uint8_t *sha_256)

Get SHA-256 digest for required partition.

For apps with SHA-256 appended to the app image, the result is the appended SHA-256 value for the app image content. The hash is verified before returning, if app content is invalid then the function returns ESP_ERR_IMAGE_INVALID. For apps without SHA-256 appended to the image, the result is the SHA-256 of all bytes in the app image. For other partition types, the result is the SHA-256 of the entire partition.

Parameters

- **partition** -- **[in]** Pointer to info for partition containing app or data. (fields: address, size and type, are required to be filled).
- **sha_256** -- **[out]** Returned SHA-256 digest for a given partition.

Returns

- ESP_OK: In case of successful operation.
- ESP_ERR_INVALID_ARG: The size was 0 or the sha_256 was NULL.
- ESP_ERR_NO_MEM: Cannot allocate memory for sha256 operation.
- ESP_ERR_IMAGE_INVALID: App partition doesn't contain a valid app image.
- ESP_FAIL: An allocation error occurred.

bool **esp_partition_check_identity**(const *esp_partition_t* *partition_1, const *esp_partition_t* *partition_2)

Check for the identity of two partitions by SHA-256 digest.

Parameters

- **partition_1** -- **[in]** Pointer to info for partition 1 containing app or data. (fields: address, size and type, are required to be filled).
- **partition_2** -- **[in]** Pointer to info for partition 2 containing app or data. (fields: address, size and type, are required to be filled).

Returns

- True: In case of the two firmware is equal.
- False: Otherwise

esp_err_t **esp_partition_register_external**(*esp_flash_t* *flash_chip, size_t offset, size_t size, const char *label, *esp_partition_type_t* type, *esp_partition_subtype_t* subtype, const *esp_partition_t* **out_partition)

Register a partition on an external flash chip.

This API allows designating certain areas of external flash chips (identified by the *esp_flash_t* structure) as partitions. This allows using them with components which access SPI flash through the esp_partition API.

Parameters

- **flash_chip** -- Pointer to the structure identifying the flash chip
- **offset** -- Address in bytes, where the partition starts
- **size** -- Size of the partition in bytes
- **label** -- Partition name
- **type** -- One of the partition types (ESP_PARTITION_TYPE_*), or an integer. Note that applications can not be booted from external flash chips, so using ESP_PARTITION_TYPE_APP is not supported.
- **subtype** -- One of the partition subtypes (ESP_PARTITION_SUBTYPE_*), or an integer.
- **out_partition** -- **[out]** Output, if non-NULL, receives the pointer to the resulting *esp_partition_t* structure

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if memory allocation has failed

- ESP_ERR_INVALID_ARG if the new partition overlaps another partition on the same flash chip
- ESP_ERR_INVALID_SIZE if the partition doesn't fit into the flash chip size

esp_err_t **esp_partition_deregister_external**(const *esp_partition_t* *partition)

Deregister the partition previously registered using esp_partition_register_external.

Parameters **partition** -- pointer to the partition structure obtained from esp_partition_register_external,

Returns

- ESP_OK on success
- ESP_ERR_NOT_FOUND if the partition pointer is not found
- ESP_ERR_INVALID_ARG if the partition comes from the partition table
- ESP_ERR_INVALID_ARG if the partition was not registered using esp_partition_register_external function.

void **esp_partition_unload_all**(void)

Unload partitions and free space allocated by them.

Structures

struct **esp_partition_t**

partition information structure

This is not the format in flash, that format is esp_partition_info_t.

However, this is the format used by this API.

Public Members

esp_flash_t ***flash_chip**

SPI flash chip on which the partition resides

esp_partition_type_t **type**

partition type (app/data)

esp_partition_subtype_t **subtype**

partition subtype

uint32_t **address**

starting address of the partition in flash

uint32_t **size**

size of the partition, in bytes

uint32_t **erase_size**

size the erase operation should be aligned to

char **label**[17]

partition label, zero-terminated ASCII string

bool **encrypted**

flag is set to true if partition is encrypted

bool **readonly**

flag is set to true if partition is read-only

Macros

ESP_PARTITION_SUBTYPE_OTA(i)

Convenience macro to get esp_partition_subtype_t value for the i-th OTA partition.

Type Definitions

typedef uint32_t **esp_partition_mmap_handle_t**

Opaque handle for memory region obtained from esp_partition_mmap.

typedef struct esp_partition_iterator_opaque_ ***esp_partition_iterator_t**

Opaque partition iterator type.

Enumerations

enum **esp_partition_mmap_memory_t**

Enumeration which specifies memory space requested in an mmap call.

Values:

enumerator **ESP_PARTITION_MMAP_DATA**

map to data memory (Vaddr0), allows byte-aligned access, 4 MB total

enumerator **ESP_PARTITION_MMAP_INST**

map to instruction memory (Vaddr1-3), allows only 4-byte-aligned access, 11 MB total

enum **esp_partition_type_t**

Partition type.

Note: Partition types with integer value 0x00-0x3F are reserved for partition types defined by ESP-IDF. Any other integer value 0x40-0xFE can be used by individual applications, without restriction.

Values:

enumerator **ESP_PARTITION_TYPE_APP**

Application partition type.

enumerator **ESP_PARTITION_TYPE_DATA**

Data partition type.

enumerator **ESP_PARTITION_TYPE_ANY**

Used to search for partitions with any type.

enum **esp_partition_subtype_t**

Partition subtype.

Application-defined partition types (0x40-0xFE) can set any numeric subtype value.

Note: These ESP-IDF-defined partition subtypes apply to partitions of type ESP_PARTITION_TYPE_APP and ESP_PARTITION_TYPE_DATA.

Values:

enumerator **ESP_PARTITION_SUBTYPE_APP_FACTORY**

Factory application partition.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_MIN**

Base for OTA partition subtypes.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_0**

OTA partition 0.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_1**

OTA partition 1.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_2**

OTA partition 2.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_3**

OTA partition 3.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_4**

OTA partition 4.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_5**

OTA partition 5.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_6**

OTA partition 6.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_7**

OTA partition 7.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_8**

OTA partition 8.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_9**

OTA partition 9.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_10**

OTA partition 10.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_11**

OTA partition 11.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_12**

OTA partition 12.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_13**
OTA partition 13.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_14**
OTA partition 14.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_15**
OTA partition 15.

enumerator **ESP_PARTITION_SUBTYPE_APP_OTA_MAX**
Max subtype of OTA partition.

enumerator **ESP_PARTITION_SUBTYPE_APP_TEST**
Test application partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_OTA**
OTA selection partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_PHY**
PHY init data partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_NVS**
NVS partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_COREDUMP**
COREDUMP partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_NVS_KEYS**
Partition for NVS keys.

enumerator **ESP_PARTITION_SUBTYPE_DATA_EFUSE_EM**
Partition for emulate eFuse bits.

enumerator **ESP_PARTITION_SUBTYPE_DATA_UNDEFINED**
Undefined (or unspecified) data partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_ESPHTTPD**
ESPHTTPD partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_FAT**
FAT partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_SPIFFS**
SPIFFS partition.

enumerator **ESP_PARTITION_SUBTYPE_DATA_LITTLEFS**
LITTLEFS partition.

enumerator **ESP_PARTITION_SUBTYPE_ANY**
Used to search for partitions with any subtype.

2.9.9 SPIFFS Filesystem

Overview

SPIFFS is a file system intended for SPI NOR flash devices on embedded targets. It supports wear levelling, file system consistency checks, and more.

Notes

- Currently, SPIFFS does not support directories, it produces a flat structure. If SPIFFS is mounted under `/spiffs`, then creating a file with the path `/spiffs/tmp/myfile.txt` will create a file called `/tmp/myfile.txt` in SPIFFS, instead of `myfile.txt` in the directory `/spiffs/tmp`.
- It is not a real-time stack. One write operation might take much longer than another.
- For now, it does not detect or handle bad blocks.
- SPIFFS is able to reliably utilize only around 75% of assigned partition space.
- When the filesystem is running out of space, the garbage collector is trying to find free space by scanning the filesystem multiple times, which can take up to several seconds per write function call, depending on required space. This is caused by the SPIFFS design and the issue has been reported multiple times (e.g., here) and in the official SPIFFS github repository. The issue can be partially mitigated by the SPIFFS configuration.
- When the garbage collector attempts to reclaim space by scanning the entire filesystem multiple times (usually 10 times by default), during each scan, the garbage collector frees up one block if available. Therefore, if the maximum number of runs set for the garbage collector is 'n' (configured by the SPIFFS_GC_MAX_RUNS option located in SPIFFS configuration), then n times the block size will become available for data writing. If you attempt to write data exceeding n times the block size, the write operation may fail and return an error.
- When the chip experiences a power loss during a file system operation it could result in SPIFFS corruption. However the file system still might be recovered via `esp_spiffs_check` function. More details in the official SPIFFS FAQ.

Tools

`spiffsgen.py` spiffsgen.py is a write-only Python SPIFFS implementation used to create filesystem images from the contents of a host folder. To use `spiffsgen.py`, open Terminal and run:

```
python spiffsgen.py <image_size> <base_dir> <output_file>
```

The required arguments are as follows:

- **image_size**: size of the partition onto which the created SPIFFS image will be flashed.
- **base_dir**: directory for which the SPIFFS image needs to be created.
- **output_file**: SPIFFS image output file.

There are also other arguments that control image generation. Documentation on these arguments can be found in the tool's help:

```
python spiffsgen.py --help
```

These optional arguments correspond to a possible SPIFFS build configuration. To generate the right image, please make sure that you use the same arguments/configuration as were used to build SPIFFS. As a guide, the help output indicates the SPIFFS build configuration to which the argument corresponds. In cases when these arguments are not specified, the default values shown in the help output will be used.

When the image is created, it can be flashed using `esptool.py` or `parttool.py`.

Aside from invoking the `spiffsgen.py` standalone by manually running it from the command line or a script, it is also possible to invoke `spiffsgen.py` directly from the build system by calling `spiffs_create_partition_image`:

```
spiffs_create_partition_image(<partition> <base_dir> [FLASH_IN_PROJECT] [DEPENDS␣
↪dep dep dep...])
```

This is more convenient as the build configuration is automatically passed to the tool, ensuring that the generated image is valid for that build. An example of this is while the **image_size** is required for the standalone invocation, only the **partition** name is required when using `spiffs_create_partition_image` -- the image size is automatically obtained from the project's partition table.

`spiffs_create_partition_image` must be called from one of the component `CMakeLists.txt` files.

Optionally, users can opt to have the image automatically flashed together with the app binaries, partition tables, etc. on `idf.py flash` by specifying `FLASH_IN_PROJECT`. For example:

```
spiffs_create_partition_image(my_spiffs_partition my_folder FLASH_IN_PROJECT)
```

If FLASH_IN_PROJECT/SPIFFS_IMAGE_FLASH_IN_PROJECT is not specified, the image will still be generated, but you will have to flash it manually using `esptool.py`, `parttool.py`, or a custom build system target.

There are cases where the contents of the base directory itself is generated at build time. Users can use DEPENDS/SPIFFS_IMAGE_DEPENDS to specify targets that should be executed before generating the image:

```
add_custom_target(dep COMMAND ...)

spiffs_create_partition_image(my_spiffs_partition my_folder DEPENDS dep)
```

For an example, see storage/spiffsgen.

mkspiffs Another tool for creating SPIFFS partition images is mkspiffs. Similar to `spiffsgen.py`, it can be used to create an image from a given folder and then flash that image using `esptool.py`

For that, you need to obtain the following parameters:

- **Block Size**: 4096 (standard for SPI Flash)
- **Page Size**: 256 (standard for SPI Flash)
- **Image Size**: Size of the partition in bytes (can be obtained from a partition table)
- **Partition Offset**: Starting address of the partition (can be obtained from a partition table)

To pack a folder into a 1-Megabyte image, run:

```
mkspiffs -c [src_folder] -b 4096 -p 256 -s 0x100000 spiffs.bin
```

To flash the image onto ESP32-S3 at offset 0x110000, run:

```
python esptool.py --chip esp32s3 --port [port] --baud [baud] write_flash -z␣
↪0x110000 spiffs.bin
```

Note: You can configure the `write_flash` command of `esptool.py` to write the spiffs data to an external SPI flash chip using the `--spi-connection <CLK>,<Q>,<D>,<HD>,<CS>` option. Just specify the GPIO pins assigned to the external flash, e.g. `python esptool.py write_flash --spi-connection 6,7,8,9,11 -z 0x110000 spiffs.bin`.

Notes on Which SPIFFS Tool to Use The two tools presented above offer very similar functionality. However, there are reasons to prefer one over the other, depending on the use case.

Use `spiffsgen.py` in the following cases:

1. If you want to simply generate a SPIFFS image during the build. `spiffsgen.py` makes it very convenient by providing functions/commands from the build system itself.
2. If the host has no C/C++ compiler available, because `spiffsgen.py` does not require compilation.

Use `mkspiffs` in the following cases:

1. If you need to unpack SPIFFS images in addition to image generation. For now, it is not possible with `spiffsgen.py`.
2. If you have an environment where a Python interpreter is not available, but a host compiler is available. Otherwise, a pre-compiled `mkspiffs` binary can do the job. However, there is no build system integration for `mkspiffs` and the user has to do the corresponding work: compiling `mkspiffs` during build (if a pre-compiled binary is not used), creating build rules/targets for the output files, passing proper parameters to the tool, etc.

See Also

- *Partition Table documentation*

Application Example

An example of using SPIFFS is provided in the storage/spiffs directory. This example initializes and mounts a SPIFFS partition, then writes and reads data from it using POSIX and C library APIs. See the README.md file in the example directory for more information.

High-level API Reference

Header File

- components/spiffs/include/esp_spiffs.h
- This header file can be included with:

```
#include "esp_spiffs.h"
```

- This header file is a part of the API provided by the `spiffs` component. To declare that your component depends on `spiffs`, add the following to your CMakeLists.txt:

```
REQUIRES spiffs
```

or

```
PRIV_REQUIRES spiffs
```

Functions

esp_err_t **esp_vfs_spiffs_register**(const *esp_vfs_spiffs_conf_t* *conf)

Register and mount SPIFFS to VFS with given path prefix.

Parameters **conf** -- Pointer to *esp_vfs_spiffs_conf_t* configuration structure

Returns

- ESP_OK if success
- ESP_ERR_NO_MEM if objects could not be allocated
- ESP_ERR_INVALID_STATE if already mounted or partition is encrypted
- ESP_ERR_NOT_FOUND if partition for SPIFFS was not found
- ESP_FAIL if mount or format fails

esp_err_t **esp_vfs_spiffs_unregister**(const char *partition_label)

Unregister and unmount SPIFFS from VFS

Parameters **partition_label** -- Same label as passed to esp_vfs_spiffs_register.

Returns

- ESP_OK if successful
- ESP_ERR_INVALID_STATE already unregistered

bool **esp_spiffs_mounted**(const char *partition_label)

Check if SPIFFS is mounted

Parameters **partition_label** -- Optional, label of the partition to check. If not specified, first partition with subtype=spiffs is used.

Returns

- true if mounted
- false if not mounted

esp_err_t **esp_spiffs_format**(const char *partition_label)

Format the SPIFFS partition

Parameters **partition_label** -- Same label as passed to esp_vfs_spiffs_register.

Returns

- ESP_OK if successful
- ESP_FAIL on error

esp_err_t **esp_spiffs_info**(const char *partition_label, size_t *total_bytes, size_t *used_bytes)

Get information for SPIFFS

Parameters

- **partition_label** -- Same label as passed to esp_vfs_spiffs_register
- **total_bytes** -- **[out]** Size of the file system
- **used_bytes** -- **[out]** Current used bytes in the file system

Returns

- ESP_OK if success
- ESP_ERR_INVALID_STATE if not mounted

esp_err_t **esp_spiffs_check**(const char *partition_label)

Check integrity of SPIFFS

Parameters **partition_label** -- Same label as passed to esp_vfs_spiffs_register

Returns

- ESP_OK if successful
- ESP_ERR_INVALID_STATE if not mounted
- ESP_FAIL on error

esp_err_t **esp_spiffs_gc**(const char *partition_label, size_t size_to_gc)

Perform garbage collection in SPIFFS partition.

Call this function to run GC and ensure that at least the given amount of space is available in the partition. This function will fail with ESP_ERR_NOT_FINISHED if it is not possible to reclaim the requested space (that is, not enough free or deleted pages in the filesystem). This function will also fail if it fails to reclaim the requested space after CONFIG_SPIFFS_GC_MAX_RUNS number of GC iterations. On one GC iteration, SPIFFS will erase one logical block (4kB). Therefore the value of CONFIG_SPIFFS_GC_MAX_RUNS should be set at least to the maximum expected size_to_gc, divided by 4096. For example, if the application expects to make room for a 1MB file and calls esp_spiffs_gc(label, 1024 * 1024), CONFIG_SPIFFS_GC_MAX_RUNS should be set to at least 256. On the other hand, increasing CONFIG_SPIFFS_GC_MAX_RUNS value increases the maximum amount of time for which any SPIFFS GC or write operation may potentially block.

Parameters

- **partition_label** -- Label of the partition to be garbage-collected. The partition must be already mounted.
- **size_to_gc** -- The number of bytes that the GC process should attempt to make available.

Returns

- ESP_OK on success
- ESP_ERR_NOT_FINISHED if GC fails to reclaim the size given by size_to_gc
- ESP_ERR_INVALID_STATE if the partition is not mounted
- ESP_FAIL on all other errors

Structures

struct **esp_vfs_spiffs_conf_t**

Configuration structure for esp_vfs_spiffs_register.

Public Members

const char ***base_path**

File path prefix associated with the filesystem.

const char ***partition_label**

Optional, label of SPIFFS partition to use. If set to NULL, first partition with subtype=spiffs will be used.

size_t **max_files**

Maximum files that could be open at the same time.

bool **format_if_mount_failed**

If true, it will format the file system if it fails to mount.

2.9.10 Virtual Filesystem Component

Overview

Virtual filesystem (VFS) component provides a unified interface for drivers which can perform operations on file-like objects. These can be real filesystems (FAT, SPIFFS, etc.) or device drivers which provide a file-like interface.

This component allows C library functions, such as fopen and fprintf, to work with FS drivers. At a high level, each FS driver is associated with some path prefix. When one of C library functions needs to open a file, the VFS component searches for the FS driver associated with the file path and forwards the call to that driver. VFS also forwards read, write, and other calls for the given file to the same FS driver.

For example, one can register a FAT filesystem driver with the `/fat` prefix and call `fopen("/fat/file.txt", "w")`. Then the VFS component calls the function `open` of the FAT driver and pass the argument `/file.txt` to it together with appropriate mode flags. All subsequent calls to C library functions for the returned `FILE*` stream will also be forwarded to the FAT driver.

FS Registration

To register an FS driver, an application needs to define an instance of the *`esp_vfs_t`* structure and populate it with function pointers to FS APIs:

```
esp_vfs_t myfs = {
    .flags = ESP_VFS_FLAG_DEFAULT,
    .write = &myfs_write,
    .open = &myfs_open,
    .fstat = &myfs_fstat,
    .close = &myfs_close,
    .read = &myfs_read,
};

ESP_ERROR_CHECK(esp_vfs_register("/data", &myfs, NULL));
```

Depending on the way how the FS driver declares its API functions, either `read`, `write`, etc., or `read_p`, `write_p`, etc., should be used.

Case 1: API functions are declared without an extra context pointer (the FS driver is a singleton):

```
ssize_t myfs_write(int fd, const void * data, size_t size);

// In definition of esp_vfs_t:
    .flags = ESP_VFS_FLAG_DEFAULT,
    .write = &myfs_write,
// ... other members initialized

// When registering FS, context pointer (third argument) is NULL:
ESP_ERROR_CHECK(esp_vfs_register("/data", &myfs, NULL));
```

Case 2: API functions are declared with an extra context pointer (the FS driver supports multiple instances):

```
ssize_t myfs_write(myfs_t* fs, int fd, const void * data, size_t size);

// In definition of esp_vfs_t:
    .flags = ESP_VFS_FLAG_CONTEXT_PTR,
    .write_p = &myfs_write,
// ... other members initialized

// When registering FS, pass the FS context pointer into the third argument
// (hypothetical myfs_mount function is used for illustrative purposes)
myfs_t* myfs_inst1 = myfs_mount(partition1->offset, partition1->size);
ESP_ERROR_CHECK(esp_vfs_register("/data1", &myfs, myfs_inst1));

// Can register another instance:
myfs_t* myfs_inst2 = myfs_mount(partition2->offset, partition2->size);
ESP_ERROR_CHECK(esp_vfs_register("/data2", &myfs, myfs_inst2));
```

Synchronous Input/Output Multiplexing Synchronous input/output multiplexing by `select()` is supported in the VFS component. The implementation works in the following way.

1. `select()` is called with file descriptors which could belong to various VFS drivers.
2. The file descriptors are divided into groups each belonging to one VFS driver.
3. The file descriptors belonging to non-socket VFS drivers are handed over to the given VFS drivers by `start_select()`, described later on this page. This function represents the driver-specific implementation of `select()` for the given driver. This should be a non-blocking call which means the function should immediately return after setting up the environment for checking events related to the given file descriptors.
4. The file descriptors belonging to the socket VFS driver are handed over to the socket driver by `socket_select()` described later on this page. This is a blocking call which means that it will return only if there is an event related to socket file descriptors or a non-socket driver signals `socket_select()` to exit.
5. Results are collected from each VFS driver and all drivers are stopped by de-initialization of the environment for checking events.
6. The `select()` call ends and returns the appropriate results.

Non-Socket VFS Drivers If you want to use `select()` with a file descriptor belonging to a non-socket VFS driver, then you need to register the driver with functions `start_select()` and `end_select()` similarly to the following example:

```
// In definition of esp_vfs_t:
    .start_select = &uart_start_select,
    .end_select = &uart_end_select,
// ... other members initialized
```

`start_select()` is called for setting up the environment for detection of read/write/error conditions on file descriptors belonging to the given VFS driver.

`end_select()` is called to stop/deinitialize/free the environment which was setup by `start_select()`.

Note: `end_select()` might be called without a previous `start_select()` call in some rare circumstances. `end_select()` should fail gracefully if this is the case (i.e., should not crash but return an error instead).

Please refer to the reference implementation for the UART peripheral in esp_driver_uart/src/uart_vfs.c and most particularly to the functions *`uart_vfs_dev_register()`*, `uart_start_select()`, and `uart_end_select()` for more information.

Please check the following examples that demonstrate the use of `select()` with VFS file descriptors:

- peripherals/uart/uart_select
- system/select

Socket VFS Drivers A socket VFS driver is using its own internal implementation of `select()` and non-socket VFS drivers notify it upon read/write/error conditions.

A socket VFS driver needs to be registered with the following functions defined:

```
// In definition of esp_vfs_t:
    .socket_select = &lwip_select,
    .get_socket_select_semaphore = &lwip_get_socket_select_semaphore,
    .stop_socket_select = &lwip_stop_socket_select,
    .stop_socket_select_isr = &lwip_stop_socket_select_isr,
// ... other members initialized
```

`socket_select()` is the internal implementation of `select()` for the socket driver. It works only with file descriptors belonging to the socket VFS.

`get_socket_select_semaphore()` returns the signalization object (semaphore) which is used in non-socket drivers to stop the waiting in `socket_select()`.

`stop_socket_select()` call is used to stop the waiting in `socket_select()` by passing the object returned by `get_socket_select_semaphore()`.

`stop_socket_select_isr()` has the same functionality as `stop_socket_select()` but it can be used from ISR.

Please see lwip/port/esp32xx/vfs_lwip.c for a reference socket driver implementation using LWIP.

Note: If you use `select()` for socket file descriptors only then you can disable the *CONFIG_VFS_SUPPORT_SELECT* option to reduce the code size and improve performance. You should not change the socket driver during an active `select()` call or you might experience some undefined behavior.

Paths

Each registered FS has a path prefix associated with it. This prefix can be considered as a "mount point" of this partition.

In case when mount points are nested, the mount point with the longest matching path prefix is used when opening the file. For instance, suppose that the following filesystems are registered in VFS:

- FS 1 on /data
- FS 2 on /data/static

Then:

- FS 1 will be used when opening a file called `/data/log.txt`

- FS 2 will be used when opening a file called `/data/static/index.html`
- Even if `/index.html"` does not exist in FS 2, FS 1 will **not** be searched for `/static/index.html`.

As a general rule, mount point names must start with the path separator (`/`) and must contain at least one character after path separator. However, an empty mount point name is also supported and might be used in cases when an application needs to provide a "fallback" filesystem or to override VFS functionality altogether. Such filesystem will be used if no prefix matches the path given.

VFS does not handle dots (`.`) in path names in any special way. VFS does not treat `..` as a reference to the parent directory. In the above example, using a path `/data/static/../log.txt` will not result in a call to FS 1 to open `/log.txt`. Specific FS drivers (such as FATFS) might handle dots in file names differently.

When opening files, the FS driver receives only relative paths to files. For example:

1. The `myfs` driver is registered with `/data` as a path prefix.
2. The application calls `fopen("/data/config.json", ...)`.
3. The VFS component calls `myfs_open("/config.json", ...)`.
4. The `myfs` driver opens the `/config.json` file.

VFS does not impose any limit on total file path length, but it does limit the FS path prefix to `ESP_VFS_PATH_MAX` characters. Individual FS drivers may have their own filename length limitations.

File Descriptors

File descriptors are small positive integers from `0` to `FD_SETSIZE - 1`, where `FD_SETSIZE` is defined in `sys/select.h`. The largest file descriptors (configured by `CONFIG_LWIP_MAX_SOCKETS`) are reserved for sockets. The VFS component contains a lookup-table called `s_fd_table` for mapping global file descriptors to VFS driver indexes registered in the `s_vfs` array.

Standard IO Streams (`stdin`, `stdout`, `stderr`)

If the menuconfig option `UART for console output` is not set to `None`, then `stdin`, `stdout`, and `stderr` are configured to read from, and write to, a UART. It is possible to use UART0 or UART1 for standard IO. By default, UART0 is used with 115200 baud rate; TX pin is GPIO1; RX pin is GPIO3. These parameters can be changed in menuconfig.

Writing to `stdout` or `stderr` sends characters to the UART transmit FIFO. Reading from `stdin` retrieves characters from the UART receive FIFO.

By default, VFS uses simple functions for reading from and writing to UART. Writes busy-wait until all data is put into UART FIFO, and reads are non-blocking, returning only the data present in the FIFO. Due to this non-blocking read behavior, higher level C library calls, such as `fscanf("%d\n", &var);`, might not have desired results.

Applications which use the UART driver can instruct VFS to use the driver's interrupt driven, blocking read and write functions instead. This can be done using a call to the *`uart_vfs_dev_use_driver()`* function. It is also possible to revert to the basic non-blocking functions using a call to *`uart_vfs_dev_use_nonblocking()`*.

VFS also provides an optional newline conversion feature for input and output. Internally, most applications send and receive lines terminated by the LF ("n") character. Different terminal programs may require different line termination, such as CR or CRLF. Applications can configure this separately for input and output either via menuconfig, or by calls to the functions *`uart_vfs_dev_port_set_rx_line_endings()`* and *`uart_vfs_dev_port_set_tx_line_endings()`*.

Standard Streams and FreeRTOS Tasks `FILE` objects for `stdin`, `stdout`, and `stderr` are shared between all FreeRTOS tasks, but the pointers to these objects are stored in per-task `struct _reent`.

The following code is transferred to `fprintf(__getreent()->_stderr, "42\n");` by the preprocessor:

```
fprintf(stderr, "42\n");
```

The `__getreent()` function returns a per-task pointer to `struct _reent` in newlib libc. This structure is allocated on the TCB of each task. When a task is initialized, `_stdin`, `_stdout`, and `_stderr` members of `struct _reent` are set to the values of `_stdin`, `_stdout`, and `_stderr` of `_GLOBAL_REENT` (i.e., the structure which is used before FreeRTOS is started).

Such a design has the following consequences:

- It is possible to set `stdin`, `stdout`, and `stderr` for any given task without affecting other tasks, e.g., by doing `stdin = fopen("/dev/uart/1", "r")`.
- Closing default `stdin`, `stdout`, or `stderr` using `fclose` closes the `FILE` stream object, which will affect all other tasks.
- To change the default `stdin`, `stdout`, `stderr` streams for new tasks, modify `_GLOBAL_REENT->_stdin` (`_stdout`, `_stderr`) before creating the task.

eventfd()

`eventfd()` call is a powerful tool to notify a `select()` based loop of custom events. The `eventfd()` implementation in ESP-IDF is generally the same as described in man(2) eventfd except for:

- `esp_vfs_eventfd_register()` has to be called before calling `eventfd()`
- Options `EFD_CLOEXEC`, `EFD_NONBLOCK` and `EFD_SEMAPHORE` are not supported in flags.
- Option `EFD_SUPPORT_ISR` has been added in flags. This flag is required to read and write the eventfd in an interrupt handler.

Note that creating an eventfd with `EFD_SUPPORT_ISR` will cause interrupts to be temporarily disabled when reading, writing the file and during the beginning and the ending of the `select()` when this file is set.

API Reference

Header File

- components/vfs/include/esp_vfs.h
- This header file can be included with:

```
#include "esp_vfs.h"
```

- This header file is a part of the API provided by the `vfs` component. To declare that your component depends on `vfs`, add the following to your CMakeLists.txt:

```
REQUIRES vfs
```

or

```
PRIV_REQUIRES vfs
```

Functions

ssize_t **esp_vfs_write**(struct _reent *r, int fd, const void *data, size_t size)

These functions are to be used in newlib syscall table. They will be called by newlib when it needs to use any of the syscalls.

off_t **esp_vfs_lseek**(struct _reent *r, int fd, off_t size, int mode)

ssize_t **esp_vfs_read**(struct _reent *r, int fd, void *dst, size_t size)

int **esp_vfs_open**(struct _reent *r, const char *path, int flags, int mode)

int **esp_vfs_close**(struct _reent *r, int fd)

int **esp_vfs_fstat**(struct _reent *r, int fd, struct stat *st)

int **esp_vfs_stat**(struct _reent *r, const char *path, struct stat *st)

int **esp_vfs_link**(struct _reent *r, const char *n1, const char *n2)

int **esp_vfs_unlink**(struct _reent *r, const char *path)

int **esp_vfs_rename**(struct _reent *r, const char *src, const char *dst)

int **esp_vfs_utime**(const char *path, const struct utimbuf *times)

esp_err_t **esp_vfs_register**(const char *base_path, const *esp_vfs_t* *vfs, void *ctx)

Register a virtual filesystem for given path prefix.

Parameters
- **base_path** -- file path prefix associated with the filesystem. Must be a zero-terminated C string, may be empty. If not empty, must be up to ESP_VFS_PATH_MAX characters long, and at least 2 characters long. Name must start with a "/" and must not end with "/". For example, "/data" or "/dev/spi" are valid. These VFSes would then be called to handle file paths such as "/data/myfile.txt" or "/dev/spi/0". In the special case of an empty base_path, a "fallback" VFS is registered. Such VFS will handle paths which are not matched by any other registered VFS.
- **vfs** -- Pointer to *esp_vfs_t*, a structure which maps syscalls to the filesystem driver functions. VFS component doesn't assume ownership of this pointer.
- **ctx** -- If vfs->flags has ESP_VFS_FLAG_CONTEXT_PTR set, a pointer which should be passed to VFS functions. Otherwise, NULL.

Returns ESP_OK if successful, ESP_ERR_NO_MEM if too many VFSes are registered.

esp_err_t **esp_vfs_register_fd_range**(const *esp_vfs_t* *vfs, void *ctx, int min_fd, int max_fd)

Special case function for registering a VFS that uses a method other than open() to open new file descriptors from the interval <min_fd; max_fd).

This is a special-purpose function intended for registering LWIP sockets to VFS.

Parameters
- **vfs** -- Pointer to *esp_vfs_t*. Meaning is the same as for esp_vfs_register().
- **ctx** -- Pointer to context structure. Meaning is the same as for esp_vfs_register().
- **min_fd** -- The smallest file descriptor this VFS will use.
- **max_fd** -- Upper boundary for file descriptors this VFS will use (the biggest file descriptor plus one).

Returns ESP_OK if successful, ESP_ERR_NO_MEM if too many VFSes are registered, ESP_ERR_INVALID_ARG if the file descriptor boundaries are incorrect.

esp_err_t **esp_vfs_register_with_id**(const *esp_vfs_t* *vfs, void *ctx, *esp_vfs_id_t* *vfs_id)

Special case function for registering a VFS that uses a method other than open() to open new file descriptors. In comparison with esp_vfs_register_fd_range, this function doesn't pre-registers an interval of file descriptors. File descriptors can be registered later, by using esp_vfs_register_fd.

Parameters
- **vfs** -- Pointer to *esp_vfs_t*. Meaning is the same as for esp_vfs_register().
- **ctx** -- Pointer to context structure. Meaning is the same as for esp_vfs_register().
- **vfs_id** -- Here will be written the VFS ID which can be passed to esp_vfs_register_fd for registering file descriptors.

Returns ESP_OK if successful, ESP_ERR_NO_MEM if too many VFSes are registered, ESP_ERR_INVALID_ARG if the file descriptor boundaries are incorrect.

esp_err_t **esp_vfs_unregister**(const char *base_path)

Unregister a virtual filesystem for given path prefix

Parameters **base_path** -- file prefix previously used in esp_vfs_register call

Returns ESP_OK if successful, ESP_ERR_INVALID_STATE if VFS for given prefix hasn't been registered

esp_err_t **esp_vfs_unregister_with_id**(*esp_vfs_id_t* vfs_id)

Unregister a virtual filesystem with the given index

Parameters **vfs_id** -- The VFS ID returned by esp_vfs_register_with_id

Returns ESP_OK if successful, ESP_ERR_INVALID_STATE if VFS for the given index hasn't been registered

esp_err_t **esp_vfs_register_fd**(*esp_vfs_id_t* vfs_id, int *fd)

Special function for registering another file descriptor for a VFS registered by esp_vfs_register_with_id. This function should only be used to register permanent file descriptors (socket fd) that are not removed after being closed.

Parameters

- **vfs_id** -- VFS identificator returned by esp_vfs_register_with_id.
- **fd** -- The registered file descriptor will be written to this address.

Returns ESP_OK if the registration is successful, ESP_ERR_NO_MEM if too many file descriptors are registered, ESP_ERR_INVALID_ARG if the arguments are incorrect.

esp_err_t **esp_vfs_register_fd_with_local_fd**(*esp_vfs_id_t* vfs_id, int local_fd, bool permanent, int *fd)

Special function for registering another file descriptor with given local_fd for a VFS registered by esp_vfs_register_with_id.

Parameters

- **vfs_id** -- VFS identificator returned by esp_vfs_register_with_id.
- **local_fd** -- The fd in the local vfs. Passing -1 will set the local fd as the (*fd) value.
- **permanent** -- Whether the fd should be treated as permannet (not removed after close())
- **fd** -- The registered file descriptor will be written to this address.

Returns ESP_OK if the registration is successful, ESP_ERR_NO_MEM if too many file descriptors are registered, ESP_ERR_INVALID_ARG if the arguments are incorrect.

esp_err_t **esp_vfs_unregister_fd**(*esp_vfs_id_t* vfs_id, int fd)

Special function for unregistering a file descriptor belonging to a VFS registered by esp_vfs_register_with_id.

Parameters

- **vfs_id** -- VFS identificator returned by esp_vfs_register_with_id.
- **fd** -- File descriptor which should be unregistered.

Returns ESP_OK if the registration is successful, ESP_ERR_INVALID_ARG if the arguments are incorrect.

int **esp_vfs_select**(int nfds, fd_set *readfds, fd_set *writefds, fd_set *errorfds, struct timeval *timeout)

Synchronous I/O multiplexing which implements the functionality of POSIX select() for VFS.

Parameters

- **nfds** -- Specifies the range of descriptors which should be checked. The first nfds descriptors will be checked in each set.
- **readfds** -- If not NULL, then points to a descriptor set that on input specifies which descriptors should be checked for being ready to read, and on output indicates which descriptors are ready to read.
- **writefds** -- If not NULL, then points to a descriptor set that on input specifies which descriptors should be checked for being ready to write, and on output indicates which descriptors are ready to write.
- **errorfds** -- If not NULL, then points to a descriptor set that on input specifies which descriptors should be checked for error conditions, and on output indicates which descriptors have error conditions.
- **timeout** -- If not NULL, then points to timeval structure which specifies the time period after which the functions should time-out and return. If it is NULL, then the function will not time-out. Note that the timeout period is rounded up to the system tick and incremented by one.

Returns The number of descriptors set in the descriptor sets, or -1 when an error (specified by errno) have occurred.

void **esp_vfs_select_triggered**(*esp_vfs_select_sem_t* sem)

Notification from a VFS driver about a read/write/error condition.

This function is called when the VFS driver detects a read/write/error condition as it was requested by the previous call to start_select.

Parameters **sem** -- semaphore structure which was passed to the driver by the start_select call

void **esp_vfs_select_triggered_isr**(*esp_vfs_select_sem_t* sem, BaseType_t *woken)

Notification from a VFS driver about a read/write/error condition (ISR version)

This function is called when the VFS driver detects a read/write/error condition as it was requested by the previous call to start_select.

Parameters

- **sem** -- semaphore structure which was passed to the driver by the start_select call
- **woken** -- is set to pdTRUE if the function wakes up a task with higher priority

ssize_t **esp_vfs_pread**(int fd, void *dst, size_t size, off_t offset)

Implements the VFS layer of POSIX pread()

Parameters

- **fd** -- File descriptor used for read
- **dst** -- Pointer to the buffer where the output will be written
- **size** -- Number of bytes to be read
- **offset** -- Starting offset of the read

Returns A positive return value indicates the number of bytes read. -1 is return on failure and errno is set accordingly.

ssize_t **esp_vfs_pwrite**(int fd, const void *src, size_t size, off_t offset)

Implements the VFS layer of POSIX pwrite()

Parameters

- **fd** -- File descriptor used for write
- **src** -- Pointer to the buffer from where the output will be read
- **size** -- Number of bytes to write
- **offset** -- Starting offset of the write

Returns A positive return value indicates the number of bytes written. -1 is return on failure and errno is set accordingly.

void **esp_vfs_dump_fds**(FILE *fp)

Dump the existing VFS FDs data to FILE* fp.

Dump the FDs in the format:

```
        <VFS Path Prefix>-<FD seen by App>-<FD seen by driver>

   where:
    VFS Path Prefix   : file prefix used in the esp_vfs_register call
    FD seen by App    : file descriptor returned by the vfs to the application␣
↪for the path prefix
    FD seen by driver : file descriptor used by the driver for the same file␣
↪prefix.
```

Parameters **fp** -- File descriptor where data will be dumped

Structures

struct **esp_vfs_select_sem_t**

VFS semaphore type for select()

Public Members

bool **is_sem_local**

type of "sem" is SemaphoreHandle_t when true, defined by socket driver otherwise

void ***sem**

semaphore instance

struct **esp_vfs_t**

VFS definition structure.

This structure should be filled with pointers to corresponding FS driver functions.

VFS component will translate all FDs so that the filesystem implementation sees them starting at zero. The caller sees a global FD which is prefixed with an pre-filesystem-implementation.

Some FS implementations expect some state (e.g. pointer to some structure) to be passed in as a first argument. For these implementations, populate the members of this structure which have _p suffix, set flags member to ESP_VFS_FLAG_CONTEXT_PTR and provide the context pointer to esp_vfs_register function. If the implementation doesn't use this extra argument, populate the members without _p suffix and set flags member to ESP_VFS_FLAG_DEFAULT.

If the FS driver doesn't provide some of the functions, set corresponding members to NULL.

Public Members

int **flags**

ESP_VFS_FLAG_CONTEXT_PTR and/or ESP_VFS_FLAG_READONLY_FS or ESP_VFS_FLAG_DEFAULT

ssize_t (***write_p**)(void *p, int fd, const void *data, size_t size)

Write with context pointer

ssize_t (***write**)(int fd, const void *data, size_t size)

Write without context pointer

off_t (***lseek_p**)(void *p, int fd, off_t size, int mode)

Seek with context pointer

off_t (***lseek**)(int fd, off_t size, int mode)

Seek without context pointer

ssize_t (***read_p**)(void *ctx, int fd, void *dst, size_t size)

Read with context pointer

ssize_t (***read**)(int fd, void *dst, size_t size)

Read without context pointer

ssize_t (***pread_p**)(void *ctx, int fd, void *dst, size_t size, off_t offset)

pread with context pointer

ssize_t (***pread**)(int fd, void *dst, size_t size, off_t offset)

pread without context pointer

ssize_t (***pwrite_p**)(void *ctx, int fd, const void *src, size_t size, off_t offset)

pwrite with context pointer

ssize_t (***pwrite**)(int fd, const void *src, size_t size, off_t offset)

pwrite without context pointer

int (***open_p**)(void *ctx, const char *path, int flags, int mode)

open with context pointer

int (***open**)(const char *path, int flags, int mode)

open without context pointer

int (***close_p**)(void *ctx, int fd)

close with context pointer

int (***close**)(int fd)

close without context pointer

int (***fstat_p**)(void *ctx, int fd, struct *stat* *st)

fstat with context pointer

int (***fstat**)(int fd, struct *stat* *st)

fstat without context pointer

int (***stat_p**)(void *ctx, const char *path, struct *stat* *st)

stat with context pointer

int (***stat**)(const char *path, struct *stat* *st)

stat without context pointer

int (***link_p**)(void *ctx, const char *n1, const char *n2)

link with context pointer

int (***link**)(const char *n1, const char *n2)

link without context pointer

int (***unlink_p**)(void *ctx, const char *path)

unlink with context pointer

int (***unlink**)(const char *path)

unlink without context pointer

int (***rename_p**)(void *ctx, const char *src, const char *dst)

rename with context pointer

int (***rename**)(const char *src, const char *dst)

rename without context pointer

DIR *(***opendir_p**)(void *ctx, const char *name)
: opendir with context pointer

DIR *(***opendir**)(const char *name)
: opendir without context pointer

struct dirent *(***readdir_p**)(void *ctx, DIR *pdir)
: readdir with context pointer

struct dirent *(***readdir**)(DIR *pdir)
: readdir without context pointer

int (***readdir_r_p**)(void *ctx, DIR *pdir, struct dirent *entry, struct dirent **out_dirent)
: readdir_r with context pointer

int (***readdir_r**)(DIR *pdir, struct dirent *entry, struct dirent **out_dirent)
: readdir_r without context pointer

long (***telldir_p**)(void *ctx, DIR *pdir)
: telldir with context pointer

long (***telldir**)(DIR *pdir)
: telldir without context pointer

void (***seekdir_p**)(void *ctx, DIR *pdir, long offset)
: seekdir with context pointer

void (***seekdir**)(DIR *pdir, long offset)
: seekdir without context pointer

int (***closedir_p**)(void *ctx, DIR *pdir)
: closedir with context pointer

int (***closedir**)(DIR *pdir)
: closedir without context pointer

int (***mkdir_p**)(void *ctx, const char *name, mode_t mode)
: mkdir with context pointer

int (***mkdir**)(const char *name, mode_t mode)
: mkdir without context pointer

int (***rmdir_p**)(void *ctx, const char *name)
: rmdir with context pointer

int (***rmdir**)(const char *name)
: rmdir without context pointer

int (*`fcntl_p`)(void *ctx, int fd, int cmd, int arg)
: fcntl with context pointer

int (*`fcntl`)(int fd, int cmd, int arg)
: fcntl without context pointer

int (*`ioctl_p`)(void *ctx, int fd, int cmd, va_list args)
: ioctl with context pointer

int (*`ioctl`)(int fd, int cmd, va_list args)
: ioctl without context pointer

int (*`fsync_p`)(void *ctx, int fd)
: fsync with context pointer

int (*`fsync`)(int fd)
: fsync without context pointer

int (*`access_p`)(void *ctx, const char *path, int amode)
: access with context pointer

int (*`access`)(const char *path, int amode)
: access without context pointer

int (*`truncate_p`)(void *ctx, const char *path, off_t length)
: truncate with context pointer

int (*`truncate`)(const char *path, off_t length)
: truncate without context pointer

int (*`ftruncate_p`)(void *ctx, int fd, off_t length)
: ftruncate with context pointer

int (*`ftruncate`)(int fd, off_t length)
: ftruncate without context pointer

int (*`utime_p`)(void *ctx, const char *path, const struct utimbuf *times)
: utime with context pointer

int (*`utime`)(const char *path, const struct utimbuf *times)
: utime without context pointer

int (*`tcsetattr_p`)(void *ctx, int fd, int optional_actions, const struct termios *p)
: tcsetattr with context pointer

int (*`tcsetattr`)(int fd, int optional_actions, const struct termios *p)
: tcsetattr without context pointer

int (*`tcgetattr_p`)(void *ctx, int fd, struct termios *p)

tcgetattr with context pointer

int (*`tcgetattr`)(int fd, struct termios *p)

tcgetattr without context pointer

int (*`tcdrain_p`)(void *ctx, int fd)

tcdrain with context pointer

int (*`tcdrain`)(int fd)

tcdrain without context pointer

int (*`tcflush_p`)(void *ctx, int fd, int select)

tcflush with context pointer

int (*`tcflush`)(int fd, int select)

tcflush without context pointer

int (*`tcflow_p`)(void *ctx, int fd, int action)

tcflow with context pointer

int (*`tcflow`)(int fd, int action)

tcflow without context pointer

pid_t (*`tcgetsid_p`)(void *ctx, int fd)

tcgetsid with context pointer

pid_t (*`tcgetsid`)(int fd)

tcgetsid without context pointer

int (*`tcsendbreak_p`)(void *ctx, int fd, int duration)

tcsendbreak with context pointer

int (*`tcsendbreak`)(int fd, int duration)

tcsendbreak without context pointer

esp_err_t (*`start_select`)(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, *esp_vfs_select_sem_t* sem, void **end_select_args)

start_select is called for setting up synchronous I/O multiplexing of the desired file descriptors in the given VFS

int (*`socket_select`)(int nfds, fd_set *readfds, fd_set *writefds, fd_set *errorfds, struct timeval *timeout)

socket select function for socket FDs with the functionality of POSIX select(); this should be set only for the socket VFS

void (*`stop_socket_select`)(void *sem)

called by VFS to interrupt the socket_select call when select is activated from a non-socket VFS driver; set only for the socket driver

void (***stop_socket_select_isr**)(void *sem, BaseType_t *woken)

stop_socket_select which can be called from ISR; set only for the socket driver

void *(***get_socket_select_semaphore**)(void)

end_select is called to stop the I/O multiplexing and deinitialize the environment created by start_select for the given VFS

esp_err_t (***end_select**)(void *end_select_args)

get_socket_select_semaphore returns semaphore allocated in the socket driver; set only for the socket driver

Macros

MAX_FDS

Maximum number of (global) file descriptors.

ESP_VFS_PATH_MAX

Maximum length of path prefix (not including zero terminator)

ESP_VFS_FLAG_DEFAULT

Default value of flags member in *esp_vfs_t* structure.

ESP_VFS_FLAG_CONTEXT_PTR

Flag which indicates that FS needs extra context pointer in syscalls.

ESP_VFS_FLAG_READONLY_FS

Flag which indicates that FS is located on read-only partition.

Type Definitions

typedef int **esp_vfs_id_t**

Header File

- components/vfs/include/esp_vfs_dev.h
- This header file can be included with:

```
#include "esp_vfs_dev.h"
```

- This header file is a part of the API provided by the `vfs` component. To declare that your component depends on `vfs`, add the following to your CMakeLists.txt:

```
REQUIRES vfs
```

or

```
PRIV_REQUIRES vfs
```

Functions

void **esp_vfs_dev_uart_register**(void)

void **esp_vfs_dev_uart_use_nonblocking**(int uart_num)

void **esp_vfs_dev_uart_use_driver**(int uart_num)

int **esp_vfs_dev_uart_port_set_rx_line_endings**(int uart_num, esp_line_endings_t mode)

int **esp_vfs_dev_uart_port_set_tx_line_endings**(int uart_num, esp_line_endings_t mode)

void **esp_vfs_dev_uart_set_rx_line_endings**(esp_line_endings_t mode)

Set the line endings expected to be received on UART.

This specifies the conversion between line endings received on UART and newlines ('

', LF) passed into stdin:

- ESP_LINE_ENDINGS_CRLF: convert CRLF to LF
- ESP_LINE_ENDINGS_CR: convert CR to LF
- ESP_LINE_ENDINGS_LF: no modification

Note: this function is not thread safe w.r.t. reading from UART

Parameters **mode** -- line endings expected on UART

void **esp_vfs_dev_uart_set_tx_line_endings**(esp_line_endings_t mode)

Set the line endings to sent to UART.

This specifies the conversion between newlines ('

', LF) on stdout and line endings sent over UART:

- ESP_LINE_ENDINGS_CRLF: convert LF to CRLF
- ESP_LINE_ENDINGS_CR: convert LF to CR
- ESP_LINE_ENDINGS_LF: no modification

Note: this function is not thread safe w.r.t. writing to UART

Parameters **mode** -- line endings to send to UART

void **esp_vfs_usb_serial_jtag_use_driver**(void)

set VFS to use USB-SERIAL-JTAG driver for reading and writing

Note: application must configure USB-SERIAL-JTAG driver before calling these functions With these functions, read and write are blocking and interrupt-driven.

void **esp_vfs_usb_serial_jtag_use_nonblocking**(void)

set VFS to use simple functions for reading and writing UART Read is non-blocking, write is busy waiting until TX FIFO has enough space. These functions are used by default.

Header File

- components/esp_driver_uart/include/driver/uart_vfs.h
- This header file can be included with:

```
#include "driver/uart_vfs.h"
```

- This header file is a part of the API provided by the `esp_driver_uart` component. To declare that your component depends on `esp_driver_uart`, add the following to your CMakeLists.txt:

```
REQUIRES esp_driver_uart
```

or

```
PRIV_REQUIRES esp_driver_uart
```

Functions

void **uart_vfs_dev_register**(void)

Add /dev/uart virtual filesystem driver.

This function is called from startup code to enable serial output

int **uart_vfs_dev_port_set_rx_line_endings**(int uart_num, esp_line_endings_t mode)

Set the line endings expected to be received on specified UART.

This specifies the conversion between line endings received on UART and newlines ('

', LF) passed into stdin:

- ESP_LINE_ENDINGS_CRLF: convert CRLF to LF
- ESP_LINE_ENDINGS_CR: convert CR to LF
- ESP_LINE_ENDINGS_LF: no modification

Note: this function is not thread safe w.r.t. reading from UART

Parameters

- **uart_num** -- the UART number
- **mode** -- line endings to send to UART

Returns 0 if succeeded, or -1 when an error (specified by errno) have occurred.

int **uart_vfs_dev_port_set_tx_line_endings**(int uart_num, esp_line_endings_t mode)

Set the line endings to sent to specified UART.

This specifies the conversion between newlines ('

', LF) on stdout and line endings sent over UART:

- ESP_LINE_ENDINGS_CRLF: convert LF to CRLF
- ESP_LINE_ENDINGS_CR: convert LF to CR
- ESP_LINE_ENDINGS_LF: no modification

Note: this function is not thread safe w.r.t. writing to UART

Parameters

- **uart_num** -- the UART number
- **mode** -- line endings to send to UART

Returns 0 if succeeded, or -1 when an error (specified by errno) have occurred.

void **uart_vfs_dev_use_nonblocking**(int uart_num)

set VFS to use simple functions for reading and writing UART

Read is non-blocking, write is busy waiting until TX FIFO has enough space. These functions are used by default.

Parameters **uart_num** -- UART peripheral number

void **uart_vfs_dev_use_driver**(int uart_num)

set VFS to use UART driver for reading and writing

Note: Application must configure UART driver before calling these functions With these functions, read and write are blocking and interrupt-driven.

Parameters **uart_num** -- UART peripheral number

Header File

- components/vfs/include/esp_vfs_eventfd.h
- This header file can be included with:

```
#include "esp_vfs_eventfd.h"
```

- This header file is a part of the API provided by the `vfs` component. To declare that your component depends on `vfs`, add the following to your CMakeLists.txt:

```
REQUIRES vfs
```

or

```
PRIV_REQUIRES vfs
```

Functions

esp_err_t **esp_vfs_eventfd_register**(const *esp_vfs_eventfd_config_t* *config)

Registers the event vfs.

Returns ESP_OK if successful, ESP_ERR_NO_MEM if too many VFSes are registered.

esp_err_t **esp_vfs_eventfd_unregister**(void)

Unregisters the event vfs.

Returns ESP_OK if successful, ESP_ERR_INVALID_STATE if VFS for given prefix hasn't been registered

int **eventfd**(unsigned int initval, int flags)

Structures

struct **esp_vfs_eventfd_config_t**

Eventfd vfs initialization settings.

Public Members

size_t **max_fds**

The maximum number of eventfds supported

Macros

EFD_SUPPORT_ISR

ESP_VFS_EVENTD_CONFIG_DEFAULT()

2.9.11 Wear Levelling API

Overview

Most of flash memory and especially SPI flash that is used in ESP32-S3 has a sector-based organization and also has a limited number of erase/modification cycles per memory sector. The wear levelling component helps to distribute wear and tear among sectors more evenly without requiring any attention from the user.

The wear levelling component provides API functions related to reading, writing, erasing, and memory mapping of data in external SPI flash through the partition component. The component also has higher-level API functions which work with the FAT filesystem defined in *FAT filesystem*.

The wear levelling component, together with the FAT FS component, uses FAT FS sectors of 4096 bytes, which is a standard size for flash memory. With this size, the component shows the best performance but needs additional memory in RAM.

To save internal memory, the component has two additional modes which both use sectors of 512 bytes:

- **Performance mode.** Erase sector operation data is stored in RAM, the sector is erased, and then data is copied back to flash memory. However, if a device is powered off for any reason, all 4096 bytes of data is lost.
- **Safety mode.** The data is first saved to flash memory, and after the sector is erased, the data is saved back. If a device is powered off, the data can be recovered as soon as the device boots up.

The default settings are as follows:

- Sector size is 512 bytes
- Performance mode

You can change the settings through the configuration menu.

The wear levelling component does not cache data in RAM. The write and erase functions modify flash directly, and flash contents are consistent when the function returns.

Wear Levelling access API functions

This is the set of API functions for working with data in flash:

- `wl_mount` - initializes the wear levelling module and mounts the specified partition
- `wl_unmount` - unmounts the partition and deinitializes the wear levelling module
- `wl_erase_range` - erases a range of addresses in flash
- `wl_write` - writes data to a partition
- `wl_read` - reads data from a partition
- `wl_size` - returns the size of available memory in bytes
- `wl_sector_size` - returns the size of one sector

As a rule, try to avoid using raw wear levelling functions and use filesystem-specific functions instead.

Memory Size

The memory size is calculated in the wear levelling module based on partition parameters. The module uses some sectors of flash for internal data.

See Also

- *FAT Filesystem Support*
- *Partition Tables*

Application Example

An example that combines the wear levelling driver with the FATFS library is provided in the storage/wear_levelling directory. This example initializes the wear levelling driver, mounts FatFs partition, as well as writes and reads data from it using POSIX and C library APIs. See storage/wear_levelling/README.md for more information.

High-level API Reference

Header Files

- fatfs/vfs/esp_vfs_fat.h

High-level wear levelling functions `esp_vfs_fat_spiflash_mount_rw_wl()`, `esp_vfs_fat_spiflash_unmount_rw_wl()` and struct `esp_vfs_fat_mount_config_t` are described in *FAT Filesystem Support*.

Mid-level API Reference

Header File

- components/wear_levelling/include/wear_levelling.h
- This header file can be included with:

```
#include "wear_levelling.h"
```

- This header file is a part of the API provided by the `wear_levelling` component. To declare that your component depends on `wear_levelling`, add the following to your CMakeLists.txt:

```
REQUIRES wear_levelling
```

or

```
PRIV_REQUIRES wear_levelling
```

Functions

esp_err_t **wl_mount**(const *esp_partition_t* *partition, *wl_handle_t* *out_handle)

Mount WL for defined partition.

Parameters
- **partition** -- that will be used for access
- **out_handle** -- handle of the WL instance

Returns
- ESP_OK, if the WL allocation is successful;
- ESP_ERR_INVALID_ARG, if the arguments for WL configuration are not valid;
- ESP_ERR_NO_MEM, if the WL allocation fails because of insufficient memory;

esp_err_t **wl_unmount**(*wl_handle_t* handle)

Unmount WL for defined partition.

Parameters **handle** -- WL partition handle

Returns
- ESP_OK, if the operation is successful;
- or one of error codes from lower-level flash driver.

esp_err_t **wl_erase_range**(*wl_handle_t* handle, size_t start_addr, size_t size)

Erase part of the WL storage.

Parameters
- **handle** -- WL handle that are related to the partition
- **start_addr** -- Address from where erase operation should start. Must be aligned to the result of function wl_sector_size(...).
- **size** -- Size of the range which should be erased, in bytes. Must be divisible by the result of function wl_sector_size(...)..

Returns
- ESP_OK, if the given range was erased successfully;
- ESP_ERR_INVALID_ARG, if iterator or dst are NULL;
- ESP_ERR_INVALID_SIZE, if erase would go out of bounds of the partition;
- or one of error codes from lower-level flash driver.

esp_err_t **wl_write**(*wl_handle_t* handle, size_t dest_addr, const void *src, size_t size)

Write data to the WL storage.

Before writing data to flash, corresponding region of flash needs to be erased. This can be done using wl_erase_range function.

Note: Prior to writing to WL storage, make sure it has been erased with wl_erase_range call.

Parameters
- **handle** -- WL handle corresponding to the WL partition
- **dest_addr** -- Address where the data should be written, relative to the beginning of the partition.
- **src** -- Pointer to the source buffer. Pointer must be non-NULL and buffer must be at least 'size' bytes long.
- **size** -- Size of data to be written, in bytes.

Returns
- ESP_OK, if data was written successfully;
- ESP_ERR_INVALID_ARG, if dst_offset exceeds partition size;
- ESP_ERR_INVALID_SIZE, if write would go out of bounds of the partition;
- or one of error codes from lower-level flash driver.

esp_err_t **wl_read**(*wl_handle_t* handle, size_t src_addr, void *dest, size_t size)

Read data from the WL storage.

Parameters
- **handle** -- WL module instance that was initialized before
- **dest** -- Pointer to the buffer where data should be stored. The Pointer must be non-NULL and the buffer must be at least 'size' bytes long.
- **src_addr** -- Address of the data to be read, relative to the beginning of the partition.
- **size** -- Size of data to be read, in bytes.

Returns
- ESP_OK, if data was read successfully;
- ESP_ERR_INVALID_ARG, if src_offset exceeds partition size;
- ESP_ERR_INVALID_SIZE, if read would go out of bounds of the partition;
- or one of error codes from lower-level flash driver.

size_t **wl_size**(*wl_handle_t* handle)

Get the actual flash size in use for the WL storage partition.

Parameters **handle** -- WL module handle that was initialized before
Returns usable size, in bytes

size_t **wl_sector_size**(*wl_handle_t* handle)

Get sector size of the WL instance.

Parameters **handle** -- WL module handle that was initialized before
Returns sector size, in bytes

Macros

WL_INVALID_HANDLE

Type Definitions

typedef int32_t **wl_handle_t**

wear levelling handle

Code examples for this API section are provided in the storage directory of ESP-IDF examples.

2.10 System API

2.10.1 App Image Format

Application Image Structures

An application image consists of the following:

1. The *esp_image_header_t* structure describes the mode of SPI flash and the count of memory segments.
2. The *esp_image_segment_header_t* structure describes each segment, its length, and its location in ESP32-S3's memory, followed by the data with a length of `data_len`. The data offset for each segment in the image is calculated in the following way:
   - offset for 0 Segment = sizeof(*esp_image_header_t*) + sizeof(*esp_image_segment_header_t*)
   - offset for 1 Segment = offset for 0 Segment + length of 0 Segment + sizeof(*esp_image_segment_header_t*)
   - offset for 2 Segment = offset for 1 Segment + length of 1 Segment + sizeof(*esp_image_segment_header_t*)
   - ...

The count of each segment is defined in the `segment_count` field that is stored in *esp_image_header_t*. The count cannot be more than *ESP_IMAGE_MAX_SEGMENTS*.

To get the list of your image segments, please run the following command:

```
esptool.py --chip esp32s3 image_info build/app.bin
```

```
esptool.py v2.3.1
Image version: 1
Entry point: 40080ea4
13 segments

Segment 1: len 0x13ce0 load 0x3f400020 file_offs 0x00000018 SOC_DROM
Segment 2: len 0x00000 load 0x3ff80000 file_offs 0x00013d00 SOC_RTC_DRAM
Segment 3: len 0x00000 load 0x3ff80000 file_offs 0x00013d08 SOC_RTC_DRAM
Segment 4: len 0x028e0 load 0x3ffb0000 file_offs 0x00013d10 DRAM
Segment 5: len 0x00000 load 0x3ffb28e0 file_offs 0x000165f8 DRAM
Segment 6: len 0x00400 load 0x40080000 file_offs 0x00016600 SOC_IRAM
Segment 7: len 0x09600 load 0x40080400 file_offs 0x00016a08 SOC_IRAM
```

(continues on next page)

(continued from previous page)

```
Segment 8: len 0x62e4c load 0x400d0018 file_offs 0x00020010 SOC_IROM
Segment 9: len 0x06cec load 0x40089a00 file_offs 0x00082e64 SOC_IROM
Segment 10: len 0x00000 load 0x400c0000 file_offs 0x00089b58 SOC_RTC_IRAM
Segment 11: len 0x00004 load 0x50000000 file_offs 0x00089b60 SOC_RTC_DATA
Segment 12: len 0x00000 load 0x50000004 file_offs 0x00089b6c SOC_RTC_DATA
Segment 13: len 0x00000 load 0x50000004 file_offs 0x00089b74 SOC_RTC_DATA
Checksum: e8 (valid)
Validation Hash: 407089ca0eae2bbf83b4120979d3354b1c938a49cb7a0c997f240474ef2ec76b␣
↪(valid)
```

You can also see the information on segments in the ESP-IDF logs while your application is booting:

```
I (443) esp_image: segment 0: paddr=0x00020020 vaddr=0x3f400020 size=0x13ce0 (␣
↪81120) map
I (489) esp_image: segment 1: paddr=0x00033d08 vaddr=0x3ff80000 size=0x00000 ( 0)␣
↪load
I (530) esp_image: segment 2: paddr=0x00033d10 vaddr=0x3ff80000 size=0x00000 ( 0)␣
↪load
I (571) esp_image: segment 3: paddr=0x00033d18 vaddr=0x3ffb0000 size=0x028e0 (␣
↪10464) load
I (612) esp_image: segment 4: paddr=0x00036600 vaddr=0x3ffb28e0 size=0x00000 ( 0)␣
↪load
I (654) esp_image: segment 5: paddr=0x00036608 vaddr=0x40080000 size=0x00400 (␣
↪1024) load
I (695) esp_image: segment 6: paddr=0x00036a10 vaddr=0x40080400 size=0x09600 (␣
↪38400) load
I (737) esp_image: segment 7: paddr=0x00040018 vaddr=0x400d0018 size=0x62e4c␣
↪(405068) map
I (847) esp_image: segment 8: paddr=0x000a2e6c vaddr=0x40089a00 size=0x06cec (␣
↪27884) load
I (888) esp_image: segment 9: paddr=0x000a9b60 vaddr=0x400c0000 size=0x00000 ( 0)␣
↪load
I (929) esp_image: segment 10: paddr=0x000a9b68 vaddr=0x50000000 size=0x00004 ( 4)␣
↪load
I (971) esp_image: segment 11: paddr=0x000a9b74 vaddr=0x50000004 size=0x00000 ( 0)␣
↪load
I (1012) esp_image: segment 12: paddr=0x000a9b7c vaddr=0x50000004 size=0x00000 (␣
↪0) load
```

For more details on the type of memory segments and their address ranges, see **ESP32-S3 Technical Reference Manual > System and Memory > Internal Memory** [PDF].

3. The image has a single checksum byte after the last segment. This byte is written on a sixteen byte padded boundary, so the application image might need padding.
4. If the `hash_appended` field from *`esp_image_header_t`* is set then a SHA256 checksum will be appended. The value of the SHA256 hash is calculated on the range from the first byte and up to this field. The length of this field is 32 bytes.
5. If the option *CONFIG_SECURE_SIGNED_APPS_SCHEME* is set to ECDSA then the application image will have an additional 68 bytes for an ECDSA signature, which includes:
   - version word (4 bytes)
   - signature data (64 bytes)
6. If the option *CONFIG_SECURE_SIGNED_APPS_SCHEME* is set to RSA or ECDSA (V2) then the application image will have an additional signature sector of 4 KB in size. For more details on the format of this signature sector, please refer to *Signature Block Format*.

Application Description

The `DROM` segment of the application binary starts with the *`esp_app_desc_t`* structure which carries specific fields describing the application:

- `magic_word`: the magic word for the *`esp_app_desc_t`* structure
- `secure_version`: see *Anti-rollback*
- `version`: see *App version*[1]
- `project_name`: filled from `PROJECT_NAME`[1]
- `time` and `date`: compile time and date
- `idf_ver`: version of ESP-IDF[1]
- `app_elf_sha256`: contains SHA256 hash for the application ELF file

This structure is useful for identification of images uploaded via Over-the-Air (OTA) updates because it has a fixed offset = sizeof(*`esp_image_header_t`*) + sizeof(*`esp_image_segment_header_t`*). As soon as a device receives the first fragment containing this structure, it has all the information to determine whether the update should be continued with or not.

To obtain the *`esp_app_desc_t`* structure for the currently running application, use *`esp_app_get_description()`*.

To obtain the *`esp_app_desc_t`* structure for another OTA partition, use *`esp_ota_get_partition_description()`*.

Adding a Custom Structure to an Application

Users also have the opportunity to have similar structure with a fixed offset relative to the beginning of the image.

The following pattern can be used to add a custom structure to your image:

```
const __attribute__((section(".rodata_custom_desc"))) esp_custom_app_desc_t custom_
↪app_desc = { ... }
```

Offset for custom structure is sizeof(*`esp_image_header_t`*) + sizeof(*`esp_image_segment_header_t`*) + sizeof(*`esp_app_desc_t`*).

To guarantee that the custom structure is located in the image even if it is not used, you need to add `target_link_libraries(${COMPONENT_TARGET} "-u custom_app_desc")` into `CMakeLists.txt`.

API Reference

Header File

- components/bootloader_support/include/esp_app_format.h
- This header file can be included with:

  ```
  #include "esp_app_format.h"
  ```

- This header file is a part of the API provided by the `bootloader_support` component. To declare that your component depends on `bootloader_support`, add the following to your CMakeLists.txt:

  ```
  REQUIRES bootloader_support
  ```

  or

  ```
  PRIV_REQUIRES bootloader_support
  ```

Structures

struct **`esp_image_header_t`**

Main header of binary image.

[1] The maximum length is 32 characters, including null-termination character. For example, if the length of `PROJECT_NAME` exceeds 31 characters, the excess characters will be disregarded.

Public Members

uint8_t **magic**

Magic word ESP_IMAGE_HEADER_MAGIC

uint8_t **segment_count**

Count of memory segments

uint8_t **spi_mode**

flash read mode (esp_image_spi_mode_t as uint8_t)

uint8_t **spi_speed**

flash frequency (esp_image_spi_freq_t as uint8_t)

uint8_t **spi_size**

flash chip size (esp_image_flash_size_t as uint8_t)

uint32_t **entry_addr**

Entry address

uint8_t **wp_pin**

WP pin when SPI pins set via efuse (read by ROM bootloader, the IDF bootloader uses software to configure the WP pin and sets this field to 0xEE=disabled)

uint8_t **spi_pin_drv**[3]

Drive settings for the SPI flash pins (read by ROM bootloader)

esp_chip_id_t **chip_id**

Chip identification number

uint8_t **min_chip_rev**

Minimal chip revision supported by image After the Major and Minor revision eFuses were introduced into the chips, this field is no longer used. But for compatibility reasons, we keep this field and the data in it. Use min_chip_rev_full instead. The software interprets this as a Major version for most of the chips and as a Minor version for the ESP32-C3.

uint16_t **min_chip_rev_full**

Minimal chip revision supported by image, in format: major * 100 + minor

uint16_t **max_chip_rev_full**

Maximal chip revision supported by image, in format: major * 100 + minor

uint8_t **reserved**[4]

Reserved bytes in additional header space, currently unused

uint8_t **hash_appended**

If 1, a SHA256 digest "simple hash" (of the entire image) is appended after the checksum. Included in image length. This digest is separate to secure boot and only used for detecting corruption. For secure boot signed images, the signature is appended after this (and the simple hash is included in the signed data).

struct **esp_image_segment_header_t**

Header of binary image segment.

Public Members

uint32_t **load_addr**

Address of segment

uint32_t **data_len**

Length of data

Macros

ESP_IMAGE_HEADER_MAGIC

The magic word for the *esp_image_header_t* structure.

ESP_IMAGE_MAX_SEGMENTS

Max count of segments in the image.

Enumerations

enum **esp_chip_id_t**

ESP chip ID.

Values:

enumerator **ESP_CHIP_ID_ESP32**

chip ID: ESP32

enumerator **ESP_CHIP_ID_ESP32S2**

chip ID: ESP32-S2

enumerator **ESP_CHIP_ID_ESP32C3**

chip ID: ESP32-C3

enumerator **ESP_CHIP_ID_ESP32S3**

chip ID: ESP32-S3

enumerator **ESP_CHIP_ID_ESP32C2**

chip ID: ESP32-C2

enumerator **ESP_CHIP_ID_ESP32C6**

chip ID: ESP32-C6

enumerator **ESP_CHIP_ID_ESP32H2**

chip ID: ESP32-H2

enumerator **ESP_CHIP_ID_ESP32P4**

chip ID: ESP32-P4

enumerator **ESP_CHIP_ID_INVALID**

Invalid chip ID (we defined it to make sure the esp_chip_id_t is 2 bytes size)

enum **esp_image_spi_mode_t**

SPI flash mode, used in *esp_image_header_t*.

Values:

enumerator **ESP_IMAGE_SPI_MODE_QIO**

SPI mode QIO

enumerator **ESP_IMAGE_SPI_MODE_QOUT**

SPI mode QOUT

enumerator **ESP_IMAGE_SPI_MODE_DIO**

SPI mode DIO

enumerator **ESP_IMAGE_SPI_MODE_DOUT**

SPI mode DOUT

enumerator **ESP_IMAGE_SPI_MODE_FAST_READ**

SPI mode FAST_READ

enumerator **ESP_IMAGE_SPI_MODE_SLOW_READ**

SPI mode SLOW_READ

enum **esp_image_spi_freq_t**

SPI flash clock division factor.

Values:

enumerator **ESP_IMAGE_SPI_SPEED_DIV_2**

The SPI flash clock frequency is divided by 2 of the clock source

enumerator **ESP_IMAGE_SPI_SPEED_DIV_3**

The SPI flash clock frequency is divided by 3 of the clock source

enumerator **ESP_IMAGE_SPI_SPEED_DIV_4**

The SPI flash clock frequency is divided by 4 of the clock source

enumerator **ESP_IMAGE_SPI_SPEED_DIV_1**

The SPI flash clock frequency equals to the clock source

enum **esp_image_flash_size_t**

Supported SPI flash sizes.

Values:

enumerator **ESP_IMAGE_FLASH_SIZE_1MB**

SPI flash size 1 MB

enumerator **ESP_IMAGE_FLASH_SIZE_2MB**
SPI flash size 2 MB

enumerator **ESP_IMAGE_FLASH_SIZE_4MB**
SPI flash size 4 MB

enumerator **ESP_IMAGE_FLASH_SIZE_8MB**
SPI flash size 8 MB

enumerator **ESP_IMAGE_FLASH_SIZE_16MB**
SPI flash size 16 MB

enumerator **ESP_IMAGE_FLASH_SIZE_32MB**
SPI flash size 32 MB

enumerator **ESP_IMAGE_FLASH_SIZE_64MB**
SPI flash size 64 MB

enumerator **ESP_IMAGE_FLASH_SIZE_128MB**
SPI flash size 128 MB

enumerator **ESP_IMAGE_FLASH_SIZE_MAX**
SPI flash size MAX

2.10.2 Bootloader Image Format

The bootloader image consists of the same structures as the application image, see *Application Image Structures*. The only difference is in the *Bootloader Description* structure.

To get information about the bootloader image, please run the following command:

```
esptool.py --chip esp32s3 image_info build/bootloader/bootloader.bin --version 2
```

The resultant output will resemble the following:

```
File size: 26576 (bytes)

ESP32 image header
==================
Image version: 1
Entry point: 0x40080658
Segments: 4
Flash size: 2MB
Flash freq: 40m
Flash mode: DIO

ESP32 extended image header
===========================
WP pin: 0xee
Flash pins drive settings: clk_drv: 0x0, q_drv: 0x0, d_drv: 0x0, cs0_drv: 0x0, hd_
↪drv: 0x0, wp_drv: 0x0
Chip ID: 0
Minimal chip revision: v0.0, (legacy min_rev = 0)
```

(continues on next page)

(continued from previous page)

```
Maximal chip revision: v3.99

Segments information
====================
Segment   Length   Load addr   File offs  Memory types
-------  -------  ----------  ----------  ------------
    1  0x01bb0  0x3fff0030  0x00000018  BYTE_ACCESSIBLE, DRAM, DIRAM_DRAM
    2  0x03c90  0x40078000  0x00001bd0  CACHE_APP
    3  0x00004  0x40080400  0x00005868  IRAM
    4  0x00f2c  0x40080404  0x00005874  IRAM

ESP32 image footer
==================
Checksum: 0x65 (valid)
Validation hash: 6f31a7f8512f26f6bce7c3b270f93bf6cf1ee4602c322998ca8ce27433527e92␣
↪(valid)

Bootloader information
======================
Bootloader version: 1
ESP-IDF: v5.1-dev-4304-gcb51a3b-dirty
Compile time: Mar 30 2023 19:14:17
```

Bootloader Description

The `DRAM0` segment of the bootloader binary starts with the *`esp_bootloader_desc_t`* structure which carries specific fields describing the bootloader. This structure is located at a fixed offset = sizeof(*`esp_image_header_t`*) + sizeof(*`esp_image_segment_header_t`*).

- `magic_byte`: the magic byte for the esp_bootloader_desc structure
- `reserved`: reserved for the future IDF use
- `version`: bootloader version, see *CONFIG_BOOTLOADER_PROJECT_VER*
- `idf_ver`: ESP-IDF version.[1]
- `date` and `time`: compile date and time
- `reserved2`: reserved for the future IDF use

To get the *`esp_bootloader_desc_t`* structure from the running bootloader, use *`esp_bootloader_get_description()`*.

To get the *`esp_bootloader_desc_t`* structure from a running application, use *`esp_ota_get_bootloader_description()`*.

API Reference

Header File

- components/esp_bootloader_format/include/esp_bootloader_desc.h
- This header file can be included with:

```
#include "esp_bootloader_desc.h"
```

- This header file is a part of the API provided by the `esp_bootloader_format` component. To declare that your component depends on `esp_bootloader_format`, add the following to your CMakeLists.txt:

```
REQUIRES esp_bootloader_format
```

or

[1] The maximum length is 32 characters, including null-termination character.

```
PRIV_REQUIRES esp_bootloader_format
```

Functions

const *esp_bootloader_desc_t* ***esp_bootloader_get_description**(void)

Return esp_bootloader_desc structure.

Intended for use by the bootloader.

Returns Pointer to esp_bootloader_desc structure.

Structures

struct **esp_bootloader_desc_t**

Bootloader description structure.

Public Members

uint8_t **magic_byte**

Magic byte ESP_BOOTLOADER_DESC_MAGIC_BYTE

uint8_t **reserved**[3]

reserved for IDF

uint32_t **version**

Bootloader version

char **idf_ver**[32]

Version IDF

char **date_time**[24]

Compile date and time

uint8_t **reserved2**[16]

reserved for IDF

Macros

ESP_BOOTLOADER_DESC_MAGIC_BYTE

The magic byte for the esp_bootloader_desc structure that is in DRAM.

2.10.3 Application Level Tracing

Overview

ESP-IDF provides a useful feature for application behavior analysis called **Application Level Tracing**. The feature can be enabled in menuconfig and allows transfer of arbitrary data between the host and ESP32-S3 via JTAG interface with minimal overhead on program execution.

Developers can use this library to send application specific state of execution to the host, and receive commands or other types of information in the opposite direction at runtime. The main use cases of this library are:

1. Collecting application specific data, see *Application Specific Tracing*.
2. Lightweight logging to the host, see *Logging to Host*.
3. System behaviour analysis, see *System Behavior Analysis with SEGGER SystemView*.

API Reference

Header File

- components/app_trace/include/esp_app_trace.h
- This header file can be included with:

```
#include "esp_app_trace.h"
```

- This header file is a part of the API provided by the `app_trace` component. To declare that your component depends on `app_trace`, add the following to your CMakeLists.txt:

```
REQUIRES app_trace
```

or

```
PRIV_REQUIRES app_trace
```

Functions

esp_err_t **esp_apptrace_init**(void)

Initializes application tracing module.

Note: Should be called before any esp_apptrace_xxx call.

Returns ESP_OK on success, otherwise see esp_err_t

void **esp_apptrace_down_buffer_config**(uint8_t *buf, uint32_t size)

Configures down buffer.

Note: Needs to be called before attempting to receive any data using esp_apptrace_down_buffer_get and esp_apptrace_read. This function does not protect internal data by lock.

Parameters

- **buf** -- Address of buffer to use for down channel (host to target) data.
- **size** -- Size of the buffer.

uint8_t *__esp_apptrace_buffer_get__(*esp_apptrace_dest_t* dest, uint32_t size, uint32_t tmo)

Allocates buffer for trace data. Once the data in the buffer is ready to be sent, esp_apptrace_buffer_put must be called to indicate it.

Parameters

- **dest** -- Indicates HW interface to send data.
- **size** -- Size of data to write to trace buffer.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns non-NULL on success, otherwise NULL.

esp_err_t **esp_apptrace_buffer_put** (*esp_apptrace_dest_t* dest, uint8_t *ptr, uint32_t tmo)

Indicates that the data in the buffer is ready to be sent. This function is a counterpart of and must be preceded by esp_apptrace_buffer_get.

Parameters
- **dest** -- Indicates HW interface to send data. Should be identical to the same parameter in call to esp_apptrace_buffer_get.
- **ptr** -- Address of trace buffer to release. Should be the value returned by call to esp_apptrace_buffer_get.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

esp_err_t **esp_apptrace_write** (*esp_apptrace_dest_t* dest, const void *data, uint32_t size, uint32_t tmo)

Writes data to trace buffer.

Parameters
- **dest** -- Indicates HW interface to send data.
- **data** -- Address of data to write to trace buffer.
- **size** -- Size of data to write to trace buffer.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

int **esp_apptrace_vprintf_to** (*esp_apptrace_dest_t* dest, uint32_t tmo, const char *fmt, va_list ap)

vprintf-like function to send log messages to host via specified HW interface.

Parameters
- **dest** -- Indicates HW interface to send data.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.
- **fmt** -- Address of format string.
- **ap** -- List of arguments.

Returns Number of bytes written.

int **esp_apptrace_vprintf** (const char *fmt, va_list ap)

vprintf-like function to send log messages to host.

Parameters
- **fmt** -- Address of format string.
- **ap** -- List of arguments.

Returns Number of bytes written.

esp_err_t **esp_apptrace_flush** (*esp_apptrace_dest_t* dest, uint32_t tmo)

Flushes remaining data in trace buffer to host.

Parameters
- **dest** -- Indicates HW interface to flush data on.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

esp_err_t **esp_apptrace_flush_nolock** (*esp_apptrace_dest_t* dest, uint32_t min_sz, uint32_t tmo)

Flushes remaining data in trace buffer to host without locking internal data. This is a special version of esp_apptrace_flush which should be called from panic handler.

Parameters
- **dest** -- Indicates HW interface to flush data on.
- **min_sz** -- Threshold for flushing data. If current filling level is above this value, data will be flushed. TRAX destinations only.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

esp_err_t **esp_apptrace_read**(*esp_apptrace_dest_t* dest, void *data, uint32_t *size, uint32_t tmo)

Reads host data from trace buffer.

Parameters
- **dest** -- Indicates HW interface to read the data on.
- **data** -- Address of buffer to put data from trace buffer.
- **size** -- Pointer to store size of read data. Before call to this function pointed memory must hold requested size of data
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

uint8_t ***esp_apptrace_down_buffer_get**(*esp_apptrace_dest_t* dest, uint32_t *size, uint32_t tmo)

Retrieves incoming data buffer if any. Once data in the buffer is processed, esp_apptrace_down_buffer_put must be called to indicate it.

Parameters
- **dest** -- Indicates HW interface to receive data.
- **size** -- Address to store size of available data in down buffer. Must be initialized with requested value.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns non-NULL on success, otherwise NULL.

esp_err_t **esp_apptrace_down_buffer_put**(*esp_apptrace_dest_t* dest, uint8_t *ptr, uint32_t tmo)

Indicates that the data in the down buffer is processed. This function is a counterpart of and must be preceded by esp_apptrace_down_buffer_get.

Parameters
- **dest** -- Indicates HW interface to receive data. Should be identical to the same parameter in call to esp_apptrace_down_buffer_get.
- **ptr** -- Address of trace buffer to release. Should be the value returned by call to esp_apptrace_down_buffer_get.
- **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinitely.

Returns ESP_OK on success, otherwise see esp_err_t

bool **esp_apptrace_host_is_connected**(*esp_apptrace_dest_t* dest)

Checks whether host is connected.

Parameters **dest** -- Indicates HW interface to use.
Returns true if host is connected, otherwise false

void ***esp_apptrace_fopen**(*esp_apptrace_dest_t* dest, const char *path, const char *mode)

Opens file on host. This function has the same semantic as 'fopen' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **path** -- Path to file.
- **mode** -- Mode string. See fopen for details.

Returns non zero file handle on success, otherwise 0

int **esp_apptrace_fclose**(*esp_apptrace_dest_t* dest, void *stream)

Closes file on host. This function has the same semantic as 'fclose' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **stream** -- File handle returned by esp_apptrace_fopen.

Returns Zero on success, otherwise non-zero. See fclose for details.

size_t **esp_apptrace_fwrite**(*esp_apptrace_dest_t* dest, const void *ptr, size_t size, size_t nmemb, void *stream)

Writes to file on host. This function has the same semantic as 'fwrite' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **ptr** -- Address of data to write.
- **size** -- Size of an item.
- **nmemb** -- Number of items to write.
- **stream** -- File handle returned by esp_apptrace_fopen.

Returns Number of written items. See fwrite for details.

size_t **esp_apptrace_fread**(*esp_apptrace_dest_t* dest, void *ptr, size_t size, size_t nmemb, void *stream)

Read file on host. This function has the same semantic as 'fread' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **ptr** -- Address to store read data.
- **size** -- Size of an item.
- **nmemb** -- Number of items to read.
- **stream** -- File handle returned by esp_apptrace_fopen.

Returns Number of read items. See fread for details.

int **esp_apptrace_fseek**(*esp_apptrace_dest_t* dest, void *stream, long offset, int whence)

Set position indicator in file on host. This function has the same semantic as 'fseek' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **stream** -- File handle returned by esp_apptrace_fopen.
- **offset** -- Offset. See fseek for details.
- **whence** -- Position in file. See fseek for details.

Returns Zero on success, otherwise non-zero. See fseek for details.

int **esp_apptrace_ftell**(*esp_apptrace_dest_t* dest, void *stream)

Get current position indicator for file on host. This function has the same semantic as 'ftell' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **stream** -- File handle returned by esp_apptrace_fopen.

Returns Current position in file. See ftell for details.

int **esp_apptrace_fstop**(*esp_apptrace_dest_t* dest)

Indicates to the host that all file operations are complete. This function should be called after all file operations are finished and indicate to the host that it can perform cleanup operations (close open files etc.).

Parameters **dest** -- Indicates HW interface to use.
Returns ESP_OK on success, otherwise see esp_err_t

int **esp_apptrace_feof**(*esp_apptrace_dest_t* dest, void *stream)

Test end-of-file indicator on a stream. This function has the same semantic as 'feof' except for the first argument.

Parameters
- **dest** -- Indicates HW interface to use.
- **stream** -- File handle returned by esp_apptrace_fopen.

Returns Non-Zero if end-of-file indicator is set for stream. See feof for details.

void **esp_gcov_dump**(void)

Triggers gcov info dump. This function waits for the host to connect to target before dumping data.

Enumerations

enum **esp_apptrace_dest_t**

Application trace data destinations bits.

Values:

enumerator **ESP_APPTRACE_DEST_JTAG**

JTAG destination.

enumerator **ESP_APPTRACE_DEST_TRAX**

xxx_TRAX name is obsolete, use more common xxx_JTAG

enumerator **ESP_APPTRACE_DEST_UART**

UART destination.

enumerator **ESP_APPTRACE_DEST_MAX**

enumerator **ESP_APPTRACE_DEST_NUM**

Header File

- components/app_trace/include/esp_sysview_trace.h
- This header file can be included with:

```
#include "esp_sysview_trace.h"
```

- This header file is a part of the API provided by the `app_trace` component. To declare that your component depends on `app_trace`, add the following to your CMakeLists.txt:

```
REQUIRES app_trace
```

or

```
PRIV_REQUIRES app_trace
```

Functions

static inline *esp_err_t* **esp_sysview_flush**(uint32_t tmo)

Flushes remaining data in SystemView trace buffer to host.

Parameters **tmo** -- Timeout for operation (in us). Use ESP_APPTRACE_TMO_INFINITE to wait indefinetly.

Returns ESP_OK.

int **esp_sysview_vprintf**(const char *format, va_list args)

vprintf-like function to sent log messages to the host.

Parameters

- **format** -- Address of format string.
- **args** -- List of arguments.

Returns Number of bytes written.

esp_err_t **esp_sysview_heap_trace_start**(uint32_t tmo)

Starts SystemView heap tracing.

Parameters **tmo** -- Timeout (in us) to wait for the host to be connected. Use -1 to wait forever.

Returns ESP_OK on success, ESP_ERR_TIMEOUT if operation has been timed out.

esp_err_t **esp_sysview_heap_trace_stop**(void)

Stops SystemView heap tracing.

Returns ESP_OK.

void **esp_sysview_heap_trace_alloc**(void *addr, uint32_t size, const void *callers)

Sends heap allocation event to the host.

Parameters

- **addr** -- Address of allocated block.
- **size** -- Size of allocated block.
- **callers** -- Pointer to array with callstack addresses. Array size must be CONFIG_HEAP_TRACING_STACK_DEPTH.

void **esp_sysview_heap_trace_free**(void *addr, const void *callers)

Sends heap de-allocation event to the host.

Parameters

- **addr** -- Address of de-allocated block.
- **callers** -- Pointer to array with callstack addresses. Array size must be CONFIG_HEAP_TRACING_STACK_DEPTH.

2.10.4 Call Function with External Stack

Overview

A given function can be executed with a user-allocated stack space which is independent of current task stack. This mechanism can be used to save stack space wasted by tasks which call a common function with intensive stack usage such as `printf`. The given function can be called inside the shared stack space, which is a callback function deferred by calling *esp_execute_shared_stack_function()*, passing that function as a parameter.

Warning: *esp_execute_shared_stack_function()* does only minimal preparation of the provided shared stack memory. The function passed to it for execution on the shared stack space or any of that function's callees should not do any of the following:

- Use thread-local storage
- Call vTaskDelete(NULL) to delete the currently running task

Furthermore, backtraces will be wrong when called from the function running on the shared stack or any of its callees. The limitations are quite severe, so that we might deprecate *esp_execute_shared_stack_function()* in the future. If you have any use case which can only be implemented using *esp_execute_shared_stack_function()*, please open a GitHub Issue.

Usage

esp_execute_shared_stack_function() takes four arguments:

- a mutex object allocated by the caller, which is used to protect if the same function shares its allocated stack
- a pointer to the top of stack used for that function
- the size of stack in bytes
- a pointer to the shared stack function

The user-defined function is deferred as a callback and can be called using the user-allocated space without taking space from current task stack.

The usage may look like the code below:

```
void external_stack_function(void)
{
    printf("Executing this printf from external stack! \n");
}

//Let us suppose we want to call printf using a separated stack space
//allowing the app to reduce its stack size.
void app_main()
{
    //Allocate a stack buffer, from heap or as a static form:
    StackType_t *shared_stack = malloc(8192 * sizeof(StackType_t));
    assert(shared_stack != NULL);

    //Allocate a mutex to protect its usage:
    SemaphoreHandle_t printf_lock = xSemaphoreCreateMutex();
    assert(printf_lock != NULL);

    //Call the desired function using the macro helper:
    esp_execute_shared_stack_function(printf_lock,
                                      shared_stack,
                                      8192,
                                      external_stack_function);

    vSemaphoreDelete(printf_lock);
    free(shared_stack);
}
```

API Reference

Header File

- components/esp_system/include/esp_expression_with_stack.h
- This header file can be included with:

```
#include "esp_expression_with_stack.h"
```

Functions

void **esp_execute_shared_stack_function**(*SemaphoreHandle_t* lock, void *stack, size_t stack_size, *shared_stack_function* function)

Calls function on user defined shared stack space.

After returning, the original stack is used again.

Note: if either lock, stack or stack size is invalid, the expression will be called using the current stack.

Warning: This function does minimal preparation of the provided piece of memory (`stack`). DO NOT do any of the following in `function` or any of its callees:

- Use Thread-local storage
- Use the Floating-point unit on ESP32-P4
- Use the AI co-processor on ESP32-P4
- Call vTaskDelete(NULL) (deleting the currently running task) Furthermore, backtraces will be wrong when called from `function` or any of its callees. The limitations are quite sever, so that we might deprecate this function in the future. If you have any use case which can only be implemented using this function, please open an issue on github.

Parameters

- **lock** -- Mutex object to protect in case of shared stack
- **stack** -- Pointer to user allocated stack
- **stack_size** -- Size of current stack in bytes
- **function** -- pointer to the shared stack function to be executed

Macros

ESP_EXECUTE_EXPRESSION_WITH_STACK(lock, stack, stack_size, expression)

Type Definitions

typedef void (***shared_stack_function**)(void)

2.10.5 Chip Revision

Overview

ESP32-S3 may have different revisions. These revisions mainly fix some issues, and sometimes also bring new features to the chip. *Versioning Scheme* describes the versioning of these chip revisions, and the APIs to read the versions at runtime.

There are some considerations of compatibility among application, ESP-IDF version, and chip revisions:

- Applications may depend on some fixes/features provided by a chip revision.
- When using updated version of hardware, the hardware may be incompatible with earlier versions of ESP-IDF.

Compatibility Checks of ESP-IDF describes how the application can specify its chip revision requirements, and the way ESP-IDF checks the compatibility. After that, there is troubleshooting information for this mechanism.

Versioning Scheme

A chip's revision number is typically expressed as `vX.Y`, where:

- `X` means a **Major** wafer version. If it is changed, it means that the current software version is not compatible with this released chip and the software must be updated to use this chip.
- `Y` means a **Minor** wafer version. If it is changed that means the current software version is compatible with the released chip, and there is no need to update the software.

If a newly released chip does not contain breaking changes, the chip can run the same software as the previous chip. As such, the new chip's revision number will only increment the minor version while keeping the major version the same (e.g., `v1.1` to `v1.2`).

Conversely, if a newly released chip contains breaking changes, the chip **cannot** run the same software as the previous chip. As such, the new chip's revision number will increment the major version and set the minor version to 0 (e.g., `v1.1` to `v2.0`).

This versioning scheme was selected to indicate the derivation relationship of chip revisions, and clearly distinguish changes in chips between breaking changes and non-breaking changes.

ESP-IDF is designed to execute seamlessly on future chip minor revisions with the same logic as the chip's nearest previous minor revision. Thus, users can directly port their compiled binaries to newer MINOR chip revisions without upgrading their ESP-IDF version and re-compile the whole project.

When a binary is executed on a chip revision of unexpected MAJOR revision, the software is also able to report issues according to the MAJOR revision. The major and minor versioning scheme also allows hardware changes to be branchable.

Note: The current chip revision scheme using major and minor versions was introduced from ESP-IDF v5.0 onwards. Thus bootloaders built using earlier versions of ESP-IDF will still use the legacy chip revision scheme of wafer versions.

EFuse Bits for Chip Revisions Chips have several eFuse version fields:

- Major wafer version (`WAFER_VERSION_MAJOR` eFuse)
- Minor wafer version (`WAFER_VERSION_MINOR` eFuse)
- Ignore maximum revision (`DISABLE_WAFER_VERSION_MAJOR` eFuse). See *Compatibility Checks of ESP-IDF* on how this is used.

Note: The previous versioning logic was based on a single eFuse version field (`WAFER_VERSION`). This approach makes it impossible to mark chips as breaking or non-breaking changes, and the versioning logic becomes linear.

Chip Revision APIs These APIs helps to get chip revision from eFuses:

- *`efuse_hal_chip_revision()`*. It returns revision in the `major * 100 + minor` format.
- *`efuse_hal_get_major_chip_version()`*. It returns Major revision.
- *`efuse_hal_get_minor_chip_version()`*. It returns Minor revision.

The following Kconfig definitions (in `major * 100 + minor` format) that can help add the chip revision dependency to the code:

- `CONFIG_ESP32S3_REV_MIN_FULL`
- `CONFIG_ESP_REV_MIN_FULL`
- `CONFIG_ESP32S3_REV_MAX_FULL`
- `CONFIG_ESP_REV_MAX_FULL`

Compatibility Checks of ESP-IDF

When building an application that needs to support multiple revisions of a particular chip, the minimum and maximum chip revision numbers supported by the build are specified via Kconfig.

The minimum chip revision can be configured via the *CONFIG_ESP32S3_REV_MIN* option. Specifying the minimum chip revision will limit the software to only run on a chip revisions that are high enough to support some features or bugfixes.

The maximum chip revision cannot be configured and is automatically determined by the current ESP-IDF version being used. ESP-IDF will refuse to boot any chip revision exceeding the maximum chip revision. Given that it is impossible for a particular ESP-IDF version to foresee all future chip revisions, the maximum chip revision is usually set to `maximum supported MAJOR version + 99`. The "Ignore Maximum Revision" eFuse can be set to bypass the maximum revision limitation. However, the software is not guaranteed to work if the maximum revision is ignored.

Below is the information about troubleshooting when the chip revision fails the compatibility check. Then there are technical details of the checking and software behavior on earlier version of ESP-IDF.

Troubleshooting

1. If the 2nd stage bootloader is run on a chip revision smaller than minimum revision specified in the image (i.e., the application), a reboot occurs. The following message will be printed:

```
Image requires chip rev >= v3.0, but chip is v1.0
```

To resolve this issue,

- Use a chip with the required minimum revision or higher.
- Lower the *CONFIG_ESP32S3_REV_MIN* value and rebuild the image so that it is compatible with the chip revision being used.

2. If application does not match minimum and maximum chip revisions, a reboot occurs. The following message will be printed:

```
Image requires chip rev <= v2.99, but chip is v3.0
```

To resolve this issue, update ESP-IDF to a newer version that supports the chip's revision (`CONFIG_ESP32S3_REV_MAX_FULL`). Alternatively, set the `Ignore maximal revision` bit in eFuse or use a chip revision that is compatible with the current version of ESP-IDF.

Representing Revision Requirements of a Binary Image The 2nd stage bootloader and the application binary images contain the *esp_image_header_t* header, which stores information specifying the chip revisions that the image is permitted to run on. This header has 3 fields related to revisions:

- `min_chip_rev` - Minimum chip MAJOR revision required by image (but for ESP32-C3 it is MINOR revision). Its value is determined by *CONFIG_ESP32S3_REV_MIN*.
- `min_chip_rev_full` - Minimum chip MINOR revision required by image in format: `major * 100 + minor`. Its value is determined by *CONFIG_ESP32S3_REV_MIN*.
- `max_chip_rev_full` - Maximum chip revision required by image in format: `major * 100 + minor`. Its value is determined by `CONFIG_ESP32S3_REV_MAX_FULL`. It can not be changed by user. Only Espressif can change it when a new version will be supported in ESP-IDF.

Maximum And Minimum Revision Restrictions The order for checking the minimum and maximum revisions during application boot up is as follows:

1. The 1st stage bootloader (ROM bootloader) does not check minimum and maximum revision fields from *esp_image_header_t* before running the 2nd stage bootloader.
2. The initialization phase of the 2nd stage bootloader checks that the 2nd stage bootloader itself can be launched on the chip of this revision. It extracts the minimum revision from the header of the bootloader image and checks against the chip revision from eFuses. If the chip revision is less than the minimum revision, the bootloader refuses to boot up and aborts. The maximum revision is not checked at this phase.
3. Then the 2nd stage bootloader checks the revision requirements of the application. It extracts the minimum and maximum revisions from the header of the application image and checks against the chip revision from eFuses. If the chip revision is less than the minimum revision or higher than the maximum revision, the bootloader refuses to boot up and aborts. However, if the Ignore maximum revision bit is set, the maximum revision constraint can be ignored. The ignore bit is set by the customer themselves when there is confirmation that the software is able to work with this chip revision.
4. Furthermore, at the OTA update stage, the running application checks if the new software matches the chip revision. It extracts the minimum and maximum revisions from the header of the new application image and checks against the chip revision from eFuses. It checks for revision matching in the same way that the bootloader does, so that the chip revision is between the min and max revisions (logic of ignoring max revision also applies).

Backward Compatibility with Bootloaders Built by Older ESP-IDF Versions The old bootloaders (ESP-IDF < v5.0) do not know about Major and Minor wafer version eFuses. They use one single eFuse for this - wafer version.

ESP32-S3 chip support was added in ESP-IDF v4.2. ESP32-S3 chips have `rev_min` in *esp_image_header_t* header set to `0` because `Minimum Supported ESP32-S2 Revision` Kconfig option was not introduced, which means that the old bootloader does not check the chip revision. Any app can be loaded by such bootloader in range `v0.0` to `v3.15`.

Please check the chip version using `esptool chip_id` command.

References

- Compatibility Advisory for Chip Revision Numbering Scheme
- Compatibility Between ESP-IDF Releases and Revisions of Espressif SoCs
- SoC Errata
- *ESP-IDF Versions*

API Reference

Header File

- components/hal/include/hal/efuse_hal.h
- This header file can be included with:

```
#include "hal/efuse_hal.h"
```

Functions

void **efuse_hal_get_mac**(uint8_t *mac)

get factory mac address

uint32_t **efuse_hal_chip_revision**(void)

Returns chip version.

Returns Chip version in format: Major * 100 + Minor

uint32_t **efuse_hal_blk_version**(void)

Return block version.

Returns Block version in format: Major * 100 + Minor

bool **efuse_hal_flash_encryption_enabled**(void)

Is flash encryption currently enabled in hardware?

Flash encryption is enabled if the FLASH_CRYPT_CNT efuse has an odd number of bits set.

Returns true if flash encryption is enabled.

bool **efuse_hal_get_disable_wafer_version_major**(void)

Returns the status of whether the bootloader (and OTA) will check the maximum chip version or not.

Returns true - Skip the maximum chip version check.

uint32_t **efuse_hal_get_major_chip_version**(void)

Returns major chip version.

uint32_t **efuse_hal_get_minor_chip_version**(void)

Returns minor chip version.

2.10.6 Console

ESP-IDF provides `console` component, which includes building blocks needed to develop an interactive console over serial port. This component includes the following features:

- Line editing, provided by linenoise library. This includes handling of backspace and arrow keys, scrolling through command history, command auto-completion, and argument hints.
- Splitting of command line into arguments.
- Argument parsing, provided by argtable3 library. This library includes APIs used for parsing GNU style command line arguments.
- Functions for registration and dispatching of commands.

- Functions to establish a basic REPL (Read-Evaluate-Print-Loop) environment.

Note: These features can be used together or independently. For example, it is possible to use line editing and command registration features, but use `getopt` or custom code for argument parsing, instead of argtable3. Likewise, it is possible to use simpler means of command input (such as `fgets`) together with the rest of the means for command splitting and argument parsing.

Note: When using a console application on a chip that supports a hardware USB serial interface, we suggest to disable the secondary serial console output. The secondary output will be output-only and consequently does not make sense in an interactive application.

Line Editing

Line editing feature lets users compose commands by typing them, erasing symbols using the `backspace` key, navigating within the command using the left/right keys, navigating to previously typed commands using the up/down keys, and performing autocompletion using the `tab` key.

Note: This feature relies on ANSI escape sequence support in the terminal application. As such, serial monitors which display raw UART data can not be used together with the line editing library. If you see `[6n` or similar escape sequence when running system/console example instead of a command prompt (e.g., `esp>`), it means that the serial monitor does not support escape sequences. Programs which are known to work are GNU screen, minicom, and esp-idf-monitor (which can be invoked using `idf.py monitor` from project directory).

Here is an overview of functions provided by linenoise library.

Configuration Linenoise library does not need explicit initialization. However, some configuration defaults may need to be changed before invoking the main line editing function.

- `linenoiseClearScreen()`
  Clear terminal screen using an escape sequence and position the cursor at the top left corner.
- `linenoiseSetMultiLine()`
  Switch between single line and multi line editing modes. In single line mode, if the length of the command exceeds the width of the terminal, the command text is scrolled within the line to show the end of the text. In this case the beginning of the text is hidden. Single line mode needs less data to be sent to refresh screen on each key press, so exhibits less glitching compared to the multi line mode. On the flip side, editing commands and copying command text from terminal in single line mode is harder. Default is single line mode.
- `linenoiseAllowEmpty()`
  Set whether linenoise library returns a zero-length string (if `true`) or `NULL` (if `false`) for empty lines. By default, zero-length strings are returned.
- `linenoiseSetMaxLineLen()`
  Set maximum length of the line for linenoise library. Default length is 4096 bytes. The default value can be updated to optimize RAM memory usage.

Main Loop

- `linenoise()`
  In most cases, console applications have some form of read/eval loop. `linenoise()` is the single function which handles user's key presses and returns the completed line once the `enter` key is pressed. As such, it handles the `read` part of the loop.
- `linenoiseFree()`
  This function must be called to release the command line buffer obtained from `linenoise()` function.

Hints and Completions

- `linenoiseSetCompletionCallback()`
  When the user presses the `tab` key, linenoise library invokes the completion callback. The callback should inspect the contents of the command typed so far and provide a list of possible completions using calls to `linenoiseAddCompletion()` function. `linenoiseSetCompletionCallback()` function should be called to register this completion callback, if completion feature is desired.
  `console` component provides a ready made function to provide completions for registered commands, *`esp_console_get_completion()`* (see below).
- `linenoiseAddCompletion()`
  Function to be called by completion callback to inform the library about possible completions of the currently typed command.
- `linenoiseSetHintsCallback()`
  Whenever user input changes, linenoise invokes the hints callback. This callback can inspect the command line typed so far, and provide a string with hints (which can include list of command arguments, for example). The library then displays the hint text on the same line where editing happens, possibly with a different color.
- `linenoiseSetFreeHintsCallback()`
  If the hint string returned by hints callback is dynamically allocated or needs to be otherwise recycled, the function which performs such cleanup should be registered via `linenoiseSetFreeHintsCallback()`.

History

- `linenoiseHistorySetMaxLen()`
  This function sets the number of most recently typed commands to be kept in memory. Users can navigate the history using the up/down arrows keys.
- `linenoiseHistoryAdd()`
  Linenoise does not automatically add commands to history. Instead, applications need to call this function to add command strings to the history.
- `linenoiseHistorySave()`
  Function saves command history from RAM to a text file, for example on an SD card or on a filesystem in flash memory.
- `linenoiseHistoryLoad()`
  Counterpart to `linenoiseHistorySave()`, loads history from a file.
- `linenoiseHistoryFree()`
  Releases memory used to store command history. Call this function when done working with linenoise library.

Splitting of Command Line into Arguments

`console` component provides *`esp_console_split_argv()`* function to split command line string into arguments. The function returns the number of arguments found (`argc`) and fills an array of pointers which can be passed as `argv` argument to any function which accepts arguments in `argc, argv` format.

The command line is split into arguments according to the following rules:

- Arguments are separated by spaces
- If spaces within arguments are required, they can be escaped using `\` (backslash) character.
- Other escape sequences which are recognized are `\\` (which produces literal backslash) and `\"`, which produces a double quote.
- Arguments can be quoted using double quotes. Quotes may appear only in the beginning and at the end of the argument. Quotes within the argument must be escaped as mentioned above. Quotes surrounding the argument are stripped by `esp_console_split_argv` function.

Examples:

- `abc def 1 20 .3` > [`abc`, `def`, `1`, `20`, `.3`]
- `abc "123 456" def` > [`abc`, `123 456`, `def`]
- `` `a\ b\\c\" `` > [`a b\c"`]

Argument Parsing

For argument parsing, `console` component includes argtable3 library. Please see tutorial for an introduction to argtable3. Github repository also includes examples.

Command Registration and Dispatching

`console` component includes utility functions which handle registration of commands, matching commands typed by the user to registered ones, and calling these commands with the arguments given on the command line.

Application first initializes command registration module using a call to *`esp_console_init()`*, and calls *`esp_console_cmd_register()`* function to register command handlers.

For each command, application provides the following information (in the form of `esp_console_cmd_t` structure):

- Command name (string without spaces)
- Help text explaining what the command does
- Optional hint text listing the arguments of the command. If application uses Argtable3 for argument parsing, hint text can be generated automatically by providing a pointer to argtable argument definitions structure instead.
- Command handler function (without context), or
- Command handler function (with context). If this function is given, an additional call to `esp_console_cmd_set_context()` must follow *before* the command may be called to initialize the context.

Note: You can either use a command handler function which takes a context or a command handler function which does not take a context, not both. If you use the command handler function which takes a context, you MUST call `esp_console_cmd_set_context()` to initialize its context, otherwise the function may access the uninitialized context.

A few other functions are provided by the command registration module:

- *`esp_console_run()`*
  This function takes the command line string, splits it into argc/argv argument list using *`esp_console_split_argv()`*, looks up the command in the list of registered components, and if it is found, executes its handler.
- *`esp_console_register_help_command()`*
  Adds `help` command to the list of registered commands. This command prints the list of all the registered commands, along with their arguments and help texts.
- *`esp_console_get_completion()`*
  Callback function to be used with `linenoiseSetCompletionCallback()` from linenoise library. Provides completions to linenoise based on the list of registered commands.
- *`esp_console_get_hint()`*
  Callback function to be used with `linenoiseSetHintsCallback()` from linenoise library. Provides argument hints for registered commands to linenoise.

Initialize Console REPL Environment

To establish a basic REPL environment, `console` component provides several useful APIs, combining those functions described above.

In a typical application, you only need to call *`esp_console_new_repl_uart()`* to initialize the REPL environment based on UART device, including driver install, basic console configuration, spawning a thread to do REPL task and register several useful commands (e.g., *help*).

After that, you can register your own commands with *`esp_console_cmd_register()`*. The REPL environment keeps in init state until you call *`esp_console_start_repl()`*.

Likewise, if your REPL environment is based on USB_SERIAL_JTAG device, you only need to call *esp_console_new_repl_usb_serial_jtag()* at first step. Then call other functions as usual.

Application Example

Example application illustrating usage of the `console` component is available in system/console directory. This example shows how to initialize UART and VFS functions, set up linenoise library, read and handle commands from UART, and store command history in Flash. See README.md in the example directory for more details.

Besides that, ESP-IDF contains several useful examples which are based on the `console` component and can be treated as "tools" when developing applications. For example, peripherals/i2c/i2c_tools, wifi/iperf.

API Reference

Header File

- components/console/esp_console.h
- This header file can be included with:

```
#include "esp_console.h"
```

- This header file is a part of the API provided by the `console` component. To declare that your component depends on `console`, add the following to your CMakeLists.txt:

```
REQUIRES console
```

or

```
PRIV_REQUIRES console
```

Functions

esp_err_t **esp_console_init** (const *esp_console_config_t* *config)

initialize console module

Note: Call this once before using other console module features

Parameters **config** -- console configuration

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if out of memory
- ESP_ERR_INVALID_STATE if already initialized
- ESP_ERR_INVALID_ARG if the configuration is invalid

esp_err_t **esp_console_deinit** (void)

de-initialize console module

Note: Call this once when done using console module functions

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if not initialized yet

esp_err_t **esp_console_cmd_register** (const *esp_console_cmd_t* *cmd)

Register console command.

Note: If the member `func_w_context` of cmd is set instead of func, then the member `context` is passed to the function pointed to by `func_w_context`.

Parameters **cmd** -- pointer to the command description; can point to a temporary value

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if out of memory
- ESP_ERR_INVALID_ARG if command description includes invalid arguments
- ESP_ERR_INVALID_ARG if both func and func_w_context members of cmd are non-NULL
- ESP_ERR_INVALID_ARG if both func and func_w_context members of cmd are NULL

esp_err_t **esp_console_run** (const char *cmdline, int *cmd_ret)

Run command line.

Parameters

- **cmdline** -- command line (command name followed by a number of arguments)
- **cmd_ret** -- **[out]** return code from the command (set if command was run)

Returns

- ESP_OK, if command was run
- ESP_ERR_INVALID_ARG, if the command line is empty, or only contained whitespace
- ESP_ERR_NOT_FOUND, if command with given name wasn't registered
- ESP_ERR_INVALID_STATE, if esp_console_init wasn't called

size_t **esp_console_split_argv** (char *line, char **argv, size_t argv_size)

Split command line into arguments in place.

```
* - This function finds whitespace-separated arguments in the given input line.
*
*     'abc def 1 20 .3' -> [ 'abc', 'def', '1', '20', '.3' ]
*
* - Argument which include spaces may be surrounded with quotes. In this case
*   spaces are preserved and quotes are stripped.
*
*     'abc "123 456" def' -> [ 'abc', '123 456', 'def' ]
*
* - Escape sequences may be used to produce backslash, double quote, and space:
*
*     'a\ b\\c\"' -> [ 'a b\c"' ]
*
```

Note: Pointers to at most argv_size - 1 arguments are returned in argv array. The pointer after the last one (i.e. argv[argc]) is set to NULL.

Parameters

- **line** -- pointer to buffer to parse; it is modified in place
- **argv** -- array where the pointers to arguments are written
- **argv_size** -- number of elements in argv_array (max. number of arguments)

Returns number of arguments found (argc)

void **esp_console_get_completion**(const char *buf, *linenoiseCompletions* *lc)

Callback which provides command completion for linenoise library.

When using linenoise for line editing, command completion support can be enabled like this:

linenoiseSetCompletionCallback(&esp_console_get_completion);

Parameters
- **buf** -- the string typed by the user
- **lc** -- linenoiseCompletions to be filled in

const char ***esp_console_get_hint**(const char *buf, int *color, int *bold)

Callback which provides command hints for linenoise library.

When using linenoise for line editing, hints support can be enabled as follows:

linenoiseSetHintsCallback((linenoiseHintsCallback*) &esp_console_get_hint);

The extra cast is needed because linenoiseHintsCallback is defined as returning a char* instead of const char*.

Parameters
- **buf** -- line typed by the user
- **color** -- **[out]** ANSI color code to be used when displaying the hint
- **bold** -- **[out]** set to 1 if hint has to be displayed in bold

Returns string containing the hint text. This string is persistent and should not be freed (i.e. linenoiseSetFreeHintsCallback should not be used).

esp_err_t **esp_console_register_help_command**(void)

Register a 'help' command.

Default 'help' command prints the list of registered commands along with hints and help strings if no additional argument is given. If an additional argument is given, the help command will look for a command with the same name and only print the hints and help strings of that command.

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE, if esp_console_init wasn't called

esp_err_t **esp_console_new_repl_uart**(const *esp_console_dev_uart_config_t* *dev_config, const *esp_console_repl_config_t* *repl_config, *esp_console_repl_t* **ret_repl)

Establish a console REPL environment over UART driver.

Attention This function is meant to be used in the examples to make the code more compact. Applications which use console functionality should be based on the underlying linenoise and esp_console functions.

Note: This is an all-in-one function to establish the environment needed for REPL, includes:

- Install the UART driver on the console UART (8n1, 115200, REF_TICK clock source)
- Configures the stdin/stdout to go through the UART driver
- Initializes linenoise
- Spawn new thread to run REPL in the background

Parameters
- **dev_config** -- **[in]** UART device configuration
- **repl_config** -- **[in]** REPL configuration
- **ret_repl** -- **[out]** return REPL handle after initialization succeed, return NULL otherwise

Returns
- ESP_OK on success
- ESP_FAIL Parameter error

esp_err_t **esp_console_new_repl_usb_cdc**(const *esp_console_dev_usb_cdc_config_t* *dev_config, const *esp_console_repl_config_t* *repl_config, *esp_console_repl_t* **ret_repl)

Establish a console REPL environment over USB CDC.

Attention This function is meant to be used in the examples to make the code more compact. Applications which use console functionality should be based on the underlying linenoise and esp_console functions.

Note: This is a all-in-one function to establish the environment needed for REPL, includes:

- Initializes linenoise
- Spawn new thread to run REPL in the background

Parameters

- **dev_config** -- **[in]** USB CDC configuration
- **repl_config** -- **[in]** REPL configuration
- **ret_repl** -- **[out]** return REPL handle after initialization succeed, return NULL otherwise

Returns

- ESP_OK on success
- ESP_FAIL Parameter error

esp_err_t **esp_console_new_repl_usb_serial_jtag**(const *esp_console_dev_usb_serial_jtag_config_t* *dev_config, const *esp_console_repl_config_t* *repl_config, *esp_console_repl_t* **ret_repl)

Establish a console REPL (Read-eval-print loop) environment over USB-SERIAL-JTAG.

Attention This function is meant to be used in the examples to make the code more compact. Applications which use console functionality should be based on the underlying linenoise and esp_console functions.

Note: This is an all-in-one function to establish the environment needed for REPL, includes:

- Initializes linenoise
- Spawn new thread to run REPL in the background

Parameters

- **dev_config** -- **[in]** USB-SERIAL-JTAG configuration
- **repl_config** -- **[in]** REPL configuration
- **ret_repl** -- **[out]** return REPL handle after initialization succeed, return NULL otherwise

Returns

- ESP_OK on success
- ESP_FAIL Parameter error

esp_err_t **esp_console_start_repl**(*esp_console_repl_t* *repl)

Start REPL environment.

Note: Once the REPL gets started, it won't be stopped until the user calls repl->del(repl) to destroy the REPL environment.

Parameters **repl** -- **[in]** REPL handle returned from esp_console_new_repl_xxx

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE, if repl has started already

Structures

struct **esp_console_config_t**

Parameters for console initialization.

Public Members

size_t **max_cmdline_length**

length of command line buffer, in bytes

size_t **max_cmdline_args**

maximum number of command line arguments to parse

uint32_t **heap_alloc_caps**

where to (e.g. MALLOC_CAP_SPIRAM) allocate heap objects such as cmds used by esp_console

int **hint_color**

ASCII color code of hint text.

int **hint_bold**

Set to 1 to print hint text in bold.

struct **esp_console_repl_config_t**

Parameters for console REPL (Read Eval Print Loop)

Public Members

uint32_t **max_history_len**

maximum length for the history

const char ***history_save_path**

file path used to save history commands, set to NULL won't save to file system

uint32_t **task_stack_size**

repl task stack size

uint32_t **task_priority**

repl task priority

BaseType_t **task_core_id**

repl task affinity, i.e. which core the task is pinned to

const char ***prompt**

prompt (NULL represents default: "esp> ")

size_t **max_cmdline_length**

maximum length of a command line. If 0, default value will be used

struct **esp_console_dev_uart_config_t**

Parameters for console device: UART.

Public Members

int **channel**

UART channel number (count from zero)

int **baud_rate**

Comunication baud rate.

int **tx_gpio_num**

GPIO number for TX path, -1 means using default one.

int **rx_gpio_num**

GPIO number for RX path, -1 means using default one.

struct **esp_console_dev_usb_cdc_config_t**

Parameters for console device: USB CDC.

Note: It's an empty structure for now, reserved for future

struct **esp_console_dev_usb_serial_jtag_config_t**

Parameters for console device: USB-SERIAL-JTAG.

Note: It's an empty structure for now, reserved for future

struct **esp_console_cmd_t**

Console command description.

Public Members

const char ***command**

Command name. Must not be NULL, must not contain spaces. The pointer must be valid until the call to esp_console_deinit.

const char ***help**

Help text for the command, shown by help command. If set, the pointer must be valid until the call to esp_console_deinit. If not set, the command will not be listed in 'help' output.

const char ***hint**

Hint text, usually lists possible arguments. If set to NULL, and 'argtable' field is non-NULL, hint will be generated automatically

esp_console_cmd_func_t **func**

Pointer to a function which implements the command.

Note: : Setting both `func` and `func_w_context` is not allowed.

void ***argtable**

Array or structure of pointers to arg_xxx structures, may be NULL. Used to generate hint text if 'hint' is set to NULL. Array/structure which this field points to must end with an arg_end. Only used for the duration of esp_console_cmd_register call.

esp_console_cmd_func_with_context_t **func_w_context**

Pointer to a context aware function which implements the command.

Note: : Setting both `func` and `func_w_context` is not allowed.

void ***context**

Context pointer to user-defined per-command context data. This is used if context aware function `func_w_context` is set.

struct **esp_console_repl_s**

Console REPL base structure.

Public Members

esp_err_t (***del**)(*esp_console_repl_t* *repl)

Delete console REPL environment.

Param repl **[in]** REPL handle returned from esp_console_new_repl_xxx

Return

- ESP_OK on success
- ESP_FAIL on errors

Macros

ESP_CONSOLE_CONFIG_DEFAULT()

Default console configuration value.

ESP_CONSOLE_REPL_CONFIG_DEFAULT()

Default console repl configuration value.

ESP_CONSOLE_DEV_UART_CONFIG_DEFAULT()

ESP_CONSOLE_DEV_CDC_CONFIG_DEFAULT()

ESP_CONSOLE_DEV_USB_SERIAL_JTAG_CONFIG_DEFAULT()

Type Definitions

typedef struct *linenoiseCompletions* **linenoiseCompletions**

typedef int (*`esp_console_cmd_func_t`)(int argc, char **argv)

Console command main function.

Param argc number of arguments
Param argv array with argc entries, each pointing to a zero-terminated string argument
Return console command return code, 0 indicates "success"

typedef int (*`esp_console_cmd_func_with_context_t`)(void *context, int argc, char **argv)

Console command main function, with context.

Param context a user context given at invocation
Param argc number of arguments
Param argv array with argc entries, each pointing to a zero-terminated string argument
Return console command return code, 0 indicates "success"

typedef struct *esp_console_repl_s* `esp_console_repl_t`

Type defined for console REPL.

2.10.7 eFuse Manager

Introduction

The eFuse Manager library is designed to structure access to eFuse bits and make using these easy. This library operates eFuse bits by a structure name which is assigned in eFuse table. This sections introduces some concepts used by eFuse Manager.

Hardware Description

The ESP32-S3 has a number of eFuses which can store system and user parameters. Each eFuse is a one-bit field which can be programmed to 1 after which it cannot be reverted back to 0. Some of system parameters are using these eFuse bits directly by hardware modules and have special place (for example EFUSE_BLK0).

For more details, see **ESP32-S3 Technical Reference Manual > eFuse Controller (eFuse)** [PDF]. Some eFuse bits are available for user applications.

ESP32-S3 has 11 eFuse blocks each of the size of 256 bits (not all bits are available):

- EFUSE_BLK0 is used entirely for system purposes;
- EFUSE_BLK1 is used entirely for system purposes;
- EFUSE_BLK2 is used entirely for system purposes;
- EFUSE_BLK3 (also named EFUSE_BLK_USER_DATA) can be used for user purposes;
- EFUSE_BLK4 (also named EFUSE_BLK_KEY0) can be used as key (for secure_boot or flash_encryption) or for user purposes;
- EFUSE_BLK5 (also named EFUSE_BLK_KEY1) can be used as key (for secure_boot or flash_encryption) or for user purposes;
- EFUSE_BLK6 (also named EFUSE_BLK_KEY2) can be used as key (for secure_boot or flash_encryption) or for user purposes;
- EFUSE_BLK7 (also named EFUSE_BLK_KEY3) can be used as key (for secure_boot or flash_encryption) or for user purposes;
- EFUSE_BLK8 (also named EFUSE_BLK_KEY4) can be used as key (for secure_boot or flash_encryption) or for user purposes;
- EFUSE_BLK9 (also named EFUSE_BLK_KEY5) can be used for any purpose except for flash encryption (due to a HW bug);
- EFUSE_BLK10 (also named EFUSE_BLK_SYS_DATA_PART2) is reserved for system purposes.

Each block is divided into 8 32-bits registers.

eFuse Manager Component

The component has API functions for reading and writing fields. Access to the fields is carried out through the structures that describe the location of the eFuse bits in the blocks. The component provides the ability to form fields of any length and from any number of individual bits. The description of the fields is made in a CSV file in a table form. To generate from a tabular form (CSV file) in the C-source uses the tool `efuse_table_gen.py`. The tool checks the CSV file for uniqueness of field names and bit intersection, in case of using a *custom* file from the user's project directory, the utility checks with the *common* CSV file.

CSV files:

- common (*esp_efuse_table.csv*) - contains eFuse fields which are used inside the ESP-IDF. C-source generation should be done manually when changing this file (run command `idf.py efuse-common-table`). Note that changes in this file can lead to incorrect operation.
- custom - (optional and can be enabled by *CONFIG_EFUSE_CUSTOM_TABLE*) contains eFuse fields that are used by the user in their application. C-source generation should be done manually when changing this file and running `idf.py efuse-custom-table`.

Description CSV File

The CSV file contains a description of the eFuse fields. In the simple case, one field has one line of description. Table header:

```
# field_name,  efuse_block(EFUSE_BLK0..EFUSE_BLK10), bit_start(0..255),    bit_
↪count(1..256),         comment
```

Individual params in CSV file the following meanings:

field_name

> Name of field. The prefix *ESP_EFUSE_* is added to the name, and this field name is available in the code. This name is used to access the fields. The name must be unique for all fields. If the line has an empty name, then this line is combined with the previous field. This allows you to set an arbitrary order of bits in the field, and expand the field as well (see `MAC_FACTORY` field in the common table). The field_name supports structured format using . to show that the field belongs to another field (see `WR_DIS` and `RD_DIS` in the common table).

efuse_block

> Block number. It determines where the eFuse bits are placed for this field. Available EFUSE_BLK0..EFUSE_BLK10.

bit_start

> Start bit number (0..255). The bit_start field can be omitted. In this case, it is set to bit_start + bit_count from the previous record, if it has the same efuse_block. Otherwise (if efuse_block is different, or this is the first entry), an error will be generated.

bit_count

> The number of bits to use in this field (1..-). This parameter cannot be omitted. This field also may be `MAX_BLK_LEN` in this case, the field length has the maximum block length.

comment

> This param is using for comment field, it also move to C-header file. The comment field can be omitted.

If a non-sequential bit order is required to describe a field, then the field description in the following lines should be continued without specifying a name, indicating that it belongs to one field. For example two fields `MAC_FACTORY` and `MAC_FACTORY_CRC`:

```
# Factory MAC address #
#######################
MAC_FACTORY,            EFUSE_BLK0,     72,     8,     Factory MAC addr [0]
```

(continues on next page)

(continued from previous page)

```
,                       EFUSE_BLK0,     64,     8,      Factory MAC addr [1]
,                       EFUSE_BLK0,     56,     8,      Factory MAC addr [2]
,                       EFUSE_BLK0,     48,     8,      Factory MAC addr [3]
,                       EFUSE_BLK0,     40,     8,      Factory MAC addr [4]
,                       EFUSE_BLK0,     32,     8,      Factory MAC addr [5]
MAC_FACTORY_CRC,        EFUSE_BLK0,     80,     8,      CRC8 for factory MAC address
```

This field is available in code as `ESP_EFUSE_MAC_FACTORY` and `ESP_EFUSE_MAC_FACTORY_CRC`.

Structured eFuse Fields

```
WR_DIS,                             EFUSE_BLK0,    0,     32,      Write protection
WR_DIS.RD_DIS,                      EFUSE_BLK0,    0,     1,       Write protection for␣
↪RD_DIS
WR_DIS.FIELD_1,                     EFUSE_BLK0,    1,     1,       Write protection for␣
↪FIELD_1
WR_DIS.FIELD_2,                     EFUSE_BLK0,    2,     4,       Write protection for␣
↪FIELD_2 (includes B1 and B2)
WR_DIS.FIELD_2.B1,                  EFUSE_BLK0,    2,     2,       Write protection for␣
↪FIELD_2.B1
WR_DIS.FIELD_2.B2,                  EFUSE_BLK0,    4,     2,       Write protection for␣
↪FIELD_2.B2
WR_DIS.FIELD_3,                     EFUSE_BLK0,    5,     1,       Write protection for␣
↪FIELD_3
WR_DIS.FIELD_3.ALIAS,               EFUSE_BLK0,    5,     1,       Write protection for␣
↪FIELD_3 (just a alias for WR_DIS.FIELD_3)
WR_DIS.FIELD_4,                     EFUSE_BLK0,    7,     1,       Write protection for␣
↪FIELD_4
```

The structured eFuse field looks like `WR_DIS.RD_DIS` where the dot points that this field belongs to the parent field - `WR_DIS` and cannot be out of the parent's range.

It is possible to use some levels of structured fields as WR_DIS.FIELD_2.B1 and B2. These fields should not be crossed each other and should be in the range of two fields: `WR_DIS` and `WR_DIS.FIELD_2`.

It is possible to create aliases for fields with the same range, see `WR_DIS.FIELD_3` and `WR_DIS.FIELD_3.ALIAS`.

The ESP-IDF names for structured eFuse fields should be unique. The `efuse_table_gen` tool generates the final names where the dot is replaced by _. The names for using in ESP-IDF are ESP_EFUSE_WR_DIS, ESP_EFUSE_WR_DIS_RD_DIS, ESP_EFUSE_WR_DIS_FIELD_2_B1, etc.

The `efuse_table_gen` tool checks that the fields do not overlap each other and must be within the range of a field if there is a violation, then throws the following error:

```
Field at USER_DATA, EFUSE_BLK3, 0, 256  intersected with  SERIAL_NUMBER, EFUSE_
↪BLK3, 0, 32
```

Solution: Describe `SERIAL_NUMBER` to be included in `USER_DATA`. (`USER_DATA.SERIAL_NUMBER`).

```
Field at FIELD, EFUSE_BLK3, 0, 50  out of range  FIELD.MAJOR_NUMBER, EFUSE_BLK3,␣
↪60, 32
```

Solution: Change `bit_start` for `FIELD.MAJOR_NUMBER` from 60 to 0, so `MAJOR_NUMBER` is in the `FIELD` range.

`efuse_table_gen.py` Tool

The tool is designed to generate C-source files from CSV file and validate fields. First of all, the check is carried out on the uniqueness of the names and overlaps of the field bits. If an additional *custom* file is used, it will be checked

with the existing *common* file (esp_efuse_table.csv). In case of errors, a message will be displayed and the string that caused the error. C-source files contain structures of type *esp_efuse_desc_t*.

To generate a *common* files, use the following command `idf.py efuse-common-table` or:

```
cd $IDF_PATH/components/efuse/
./efuse_table_gen.py --idf_target esp32s3 esp32s3/esp_efuse_table.csv
```

After generation in the folder $IDF_PATH/components/efuse/*esp32s3* create:

- *esp_efuse_table.c* file.
- In *include* folder *esp_efuse_table.c* file.

To generate a *custom* files, use the following command `idf.py efuse-custom-table` or:

```
cd $IDF_PATH/components/efuse/
./efuse_table_gen.py --idf_target esp32s3 esp32s3/esp_efuse_table.csv PROJECT_PATH/
↪main/esp_efuse_custom_table.csv
```

After generation in the folder PROJECT_PATH/main create:

- *esp_efuse_custom_table.c* file.
- In *include* folder *esp_efuse_custom_table.c* file.

To use the generated fields, you need to include two files:

```
#include "esp_efuse.h"
#include "esp_efuse_table.h" // or "esp_efuse_custom_table.h"
```

Supported Coding Scheme

Coding schemes are used to protect against data corruption. ESP32-S3 supports two coding schemes:

- `None`. EFUSE_BLK0 is stored with four backups, meaning each bit is stored four times. This backup scheme is automatically applied by the hardware and is not visible to software. EFUSE_BLK0 can be written many times.
- `RS`. EFUSE_BLK1 - EFUSE_BLK10 use Reed-Solomon coding scheme that supports up to 5 bytes of automatic error correction. Software encodes the 32-byte EFUSE_BLKx using RS (44, 32) to generate a 12-byte check code, and then burn the EFUSE_BLKx and the check code into eFuse at the same time. The eFuse Controller automatically decodes the RS encoding and applies error correction when reading back the eFuse block. Because the RS check codes are generated across the entire 256-bit eFuse block, each block can only be written to one time.

To write some fields into one block, or different blocks in one time, you need to use `the batch writing mode`. Firstly set this mode through *`esp_efuse_batch_write_begin()`* function then write some fields as usual using the `esp_efuse_write_...` functions. At the end to burn them, call the *`esp_efuse_batch_write_commit()`* function. It burns prepared data to the eFuse blocks and disables the `batch recording mode`.

Note: If there is already pre-written data in the eFuse block using the `Reed-Solomon` encoding scheme, then it is not possible to write anything extra (even if the required bits are empty) without breaking the previous encoding data. This encoding data will be overwritten with new encoding data and completely destroyed (however, the payload eFuses are not damaged). It can be related to: CUSTOM_MAC, SPI_PAD_CONFIG_HD, SPI_PAD_CONFIG_CS, etc. Please contact Espressif to order the required pre-burnt eFuses.

FOR TESTING ONLY (NOT RECOMMENDED): You can ignore or suppress errors that violate encoding scheme data in order to burn the necessary bits in the eFuse block.

---

eFuse API

Access to the fields is via a pointer to the description structure. API functions have some basic operation:

- *esp_efuse_read_field_blob()* - returns an array of read eFuse bits.
- *esp_efuse_read_field_cnt()* - returns the number of bits programmed as "1".
- *esp_efuse_write_field_blob()* - writes an array.
- *esp_efuse_write_field_cnt()* - writes a required count of bits as "1".
- *esp_efuse_get_field_size()* - returns the number of bits by the field name.
- *esp_efuse_read_reg()* - returns value of eFuse register.
- *esp_efuse_write_reg()* - writes value to eFuse register.
- *esp_efuse_get_coding_scheme()* - returns eFuse coding scheme for blocks.
- *esp_efuse_read_block()* - reads key to eFuse block starting at the offset and the required size.
- *esp_efuse_write_block()* - writes key to eFuse block starting at the offset and the required size.
- *esp_efuse_batch_write_begin()* - set the batch mode of writing fields.
- *esp_efuse_batch_write_commit()* - writes all prepared data for batch writing mode and reset the batch writing mode.
- *esp_efuse_batch_write_cancel()* - reset the batch writing mode and prepared data.
- *esp_efuse_get_key_dis_read()* - Returns a read protection for the key block.
- *esp_efuse_set_key_dis_read()* - Sets a read protection for the key block.
- *esp_efuse_get_key_dis_write()* - Returns a write protection for the key block.
- *esp_efuse_set_key_dis_write()* - Sets a write protection for the key block.
- *esp_efuse_get_key_purpose()* - Returns the current purpose set for an eFuse key block.
- *esp_efuse_write_key()* - Programs a block of key data to an eFuse block
- *esp_efuse_write_keys()* - Programs keys to unused eFuse blocks
- *esp_efuse_find_purpose()* - Finds a key block with the particular purpose set.
- *esp_efuse_get_keypurpose_dis_write()* - Returns a write protection of the key purpose field for an eFuse key block (for esp32 always true).
- *esp_efuse_key_block_unused()* - Returns true if the key block is unused, false otherwise.
- *esp_efuse_destroy_block()* - Destroys the data in this eFuse block. There are two things to do (1) if write protection is not set, then the remaining unset bits are burned, (2) set read protection for this block if it is not locked.

For frequently used fields, special functions are made, like this *esp_efuse_get_pkg_ver()*.

eFuse API for Keys

EFUSE_BLK_KEY0 - EFUSE_BLK_KEY5 are intended to keep up to 6 keys with a length of 256-bits. Each key has an `ESP_EFUSE_KEY_PURPOSE_x` field which defines the purpose of these keys. The purpose field is described in *esp_efuse_purpose_t*.

The purposes like `ESP_EFUSE_KEY_PURPOSE_XTS_AES_...` are used for flash encryption.

The purposes like `ESP_EFUSE_KEY_PURPOSE_SECURE_BOOT_DIGEST...` are used for secure boot.

There are some eFuse APIs useful to work with states of keys.

- *esp_efuse_get_purpose_field()* - Returns a pointer to a key purpose for an eFuse key block.
- *esp_efuse_get_key()* - Returns a pointer to a key block.
- *esp_efuse_set_key_purpose()* - Sets a key purpose for an eFuse key block.
- *esp_efuse_set_keypurpose_dis_write()* - Sets a write protection of the key purpose field for an eFuse key block.
- *esp_efuse_find_unused_key_block()* - Search for an unused key block and return the first one found.
- *esp_efuse_count_unused_key_blocks()* - Returns the number of unused eFuse key blocks in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
- *esp_efuse_get_digest_revoke()* - Returns the status of the Secure Boot public key digest revocation bit.
- *esp_efuse_set_digest_revoke()* - Sets the Secure Boot public key digest revocation bit.

- *esp_efuse_get_write_protect_of_digest_revoke()* - Returns a write protection of the Secure Boot public key digest revocation bit.
- *esp_efuse_set_write_protect_of_digest_revoke()* - Sets a write protection of the Secure Boot public key digest revocation bit.

How to Add a New Field

1. Find a free bits for field. Show *esp_efuse_table.csv* file or run `idf.py show-efuse-table` or the next command:

```
$ ./efuse_table_gen.py esp32s3/esp_efuse_table.csv --info

Max number of bits in BLK 256
Sorted efuse table:
#       field_name                                efuse_block     bit_start       ␣
↪bit_count
1       WR_DIS                                    EFUSE_BLK0          0           ␣
↪32
2       WR_DIS.RD_DIS                             EFUSE_BLK0          0           ␣
↪1
3       WR_DIS.DIS_ICACHE                         EFUSE_BLK0          2           ␣
↪1
4       WR_DIS.DIS_DCACHE                         EFUSE_BLK0          2           ␣
↪1
5       WR_DIS.DIS_DOWNLOAD_ICACHE                EFUSE_BLK0          2           ␣
↪1
6       WR_DIS.DIS_DOWNLOAD_DCACHE                EFUSE_BLK0          2           ␣
↪1
7       WR_DIS.DIS_FORCE_DOWNLOAD                 EFUSE_BLK0          2           ␣
↪1
8       WR_DIS.DIS_USB_OTG                        EFUSE_BLK0          2           ␣
↪1
9       WR_DIS.DIS_TWAI                           EFUSE_BLK0          2           ␣
↪1
10      WR_DIS.DIS_APP_CPU                        EFUSE_BLK0          2           ␣
↪1
11      WR_DIS.DIS_PAD_JTAG                       EFUSE_BLK0          2           ␣
↪1
12      WR_DIS.DIS_DOWNLOAD_MANUAL_ENCRYPT        EFUSE_BLK0          2           ␣
↪1
13      WR_DIS.DIS_USB_JTAG                       EFUSE_BLK0          2           ␣
↪1
14      WR_DIS.DIS_USB_SERIAL_JTAG                EFUSE_BLK0          2           ␣
↪1
15      WR_DIS.STRAP_JTAG_SEL                     EFUSE_BLK0          2           ␣
↪1
16      WR_DIS.USB_PHY_SEL                        EFUSE_BLK0          2           ␣
↪1
17      WR_DIS.VDD_SPI_XPD                        EFUSE_BLK0          3           ␣
↪1
18      WR_DIS.VDD_SPI_TIEH                       EFUSE_BLK0          3           ␣
↪1
19      WR_DIS.VDD_SPI_FORCE                      EFUSE_BLK0          3           ␣
↪1
20      WR_DIS.WDT_DELAY_SEL                      EFUSE_BLK0          3           ␣
↪1
21      WR_DIS.SPI_BOOT_CRYPT_CNT                 EFUSE_BLK0          4           ␣
↪1
22      WR_DIS.SECURE_BOOT_KEY_REVOKE0            EFUSE_BLK0          5           ␣
↪1
23      WR_DIS.SECURE_BOOT_KEY_REVOKE1            EFUSE_BLK0          6           ␣
↪1
```

(continues on next page)

(continued from previous page)

```
24      WR_DIS.SECURE_BOOT_KEY_REVOKE2          EFUSE_BLK0          7                ␣
↪1
25      WR_DIS.KEY_PURPOSE_0                    EFUSE_BLK0          8                ␣
↪1
26      WR_DIS.KEY_PURPOSE_1                    EFUSE_BLK0          9                ␣
↪1
27      WR_DIS.KEY_PURPOSE_2                    EFUSE_BLK0          10               ␣
↪1
28      WR_DIS.KEY_PURPOSE_3                    EFUSE_BLK0          11               ␣
↪1
29      WR_DIS.KEY_PURPOSE_4                    EFUSE_BLK0          12               ␣
↪1
30      WR_DIS.KEY_PURPOSE_5                    EFUSE_BLK0          13               ␣
↪1
31      WR_DIS.SECURE_BOOT_EN                   EFUSE_BLK0          15               ␣
↪1
32      WR_DIS.SECURE_BOOT_AGGRESSIVE_REVOKE    EFUSE_BLK0          16               ␣
↪1
33      WR_DIS.FLASH_TPUW                       EFUSE_BLK0          18               ␣
↪1
34      WR_DIS.DIS_DOWNLOAD_MODE                EFUSE_BLK0          18               ␣
↪1
35      WR_DIS.DIS_DIRECT_BOOT                  EFUSE_BLK0          18               ␣
↪1
36      WR_DIS.DIS_USB_SERIAL_JTAG_ROM_PRINT    EFUSE_BLK0          18               ␣
↪1
37      WR_DIS.FLASH_ECC_MODE                   EFUSE_BLK0          18               ␣
↪1
38      WR_DIS.DIS_USB_SERIAL_JTAG_DOWNLOAD_MODE EFUSE_BLK0         18               ␣
↪1
39      WR_DIS.ENABLE_SECURITY_DOWNLOAD         EFUSE_BLK0          18               ␣
↪1
40      WR_DIS.UART_PRINT_CONTROL               EFUSE_BLK0          18               ␣
↪1
41      WR_DIS.PIN_POWER_SELECTION              EFUSE_BLK0          18               ␣
↪1
42      WR_DIS.FLASH_TYPE                       EFUSE_BLK0          18               ␣
↪1
43      WR_DIS.FLASH_PAGE_SIZE                  EFUSE_BLK0          18               ␣
↪1
44      WR_DIS.FLASH_ECC_EN                     EFUSE_BLK0          18               ␣
↪1
45      WR_DIS.FORCE_SEND_RESUME                EFUSE_BLK0          18               ␣
↪1
46      WR_DIS.SECURE_VERSION                   EFUSE_BLK0          18               ␣
↪1
47      WR_DIS.DIS_USB_OTG_DOWNLOAD_MODE        EFUSE_BLK0          19               ␣
↪1
48      WR_DIS.DISABLE_WAFER_VERSION_MAJOR      EFUSE_BLK0          19               ␣
↪1
49      WR_DIS.DISABLE_BLK_VERSION_MAJOR        EFUSE_BLK0          19               ␣
↪1
50      WR_DIS.BLK1                             EFUSE_BLK0          20               ␣
↪1
51      WR_DIS.MAC                              EFUSE_BLK0          20               ␣
↪1
52      WR_DIS.SPI_PAD_CONFIG_CLK               EFUSE_BLK0          20               ␣
↪1
53      WR_DIS.SPI_PAD_CONFIG_Q                 EFUSE_BLK0          20               ␣
↪1
54      WR_DIS.SPI_PAD_CONFIG_D                 EFUSE_BLK0          20               ␣
↪1
```

(continues on next page)

(continued from previous page)

```
55      WR_DIS.SPI_PAD_CONFIG_CS                EFUSE_BLK0          20                ␣
↪1
56      WR_DIS.SPI_PAD_CONFIG_HD                EFUSE_BLK0          20                ␣
↪1
57      WR_DIS.SPI_PAD_CONFIG_WP                EFUSE_BLK0          20                ␣
↪1
58      WR_DIS.SPI_PAD_CONFIG_DQS               EFUSE_BLK0          20                ␣
↪1
59      WR_DIS.SPI_PAD_CONFIG_D4                EFUSE_BLK0          20                ␣
↪1
60      WR_DIS.SPI_PAD_CONFIG_D5                EFUSE_BLK0          20                ␣
↪1
61      WR_DIS.SPI_PAD_CONFIG_D6                EFUSE_BLK0          20                ␣
↪1
62      WR_DIS.SPI_PAD_CONFIG_D7                EFUSE_BLK0          20                ␣
↪1
63      WR_DIS.WAFER_VERSION_MINOR_LO           EFUSE_BLK0          20                ␣
↪1
64      WR_DIS.PKG_VERSION                      EFUSE_BLK0          20                ␣
↪1
65      WR_DIS.BLK_VERSION_MINOR                EFUSE_BLK0          20                ␣
↪1
66      WR_DIS.K_RTC_LDO                        EFUSE_BLK0          20                ␣
↪1
67      WR_DIS.K_DIG_LDO                        EFUSE_BLK0          20                ␣
↪1
68      WR_DIS.V_RTC_DBIAS20                    EFUSE_BLK0          20                ␣
↪1
69      WR_DIS.V_DIG_DBIAS20                    EFUSE_BLK0          20                ␣
↪1
70      WR_DIS.DIG_DBIAS_HVT                    EFUSE_BLK0          20                ␣
↪1
71      WR_DIS.WAFER_VERSION_MINOR_HI           EFUSE_BLK0          20                ␣
↪1
72      WR_DIS.WAFER_VERSION_MAJOR              EFUSE_BLK0          20                ␣
↪1
73      WR_DIS.ADC2_CAL_VOL_ATTEN3              EFUSE_BLK0          20                ␣
↪1
74      WR_DIS.SYS_DATA_PART1                   EFUSE_BLK0          21                ␣
↪1
75      WR_DIS.OPTIONAL_UNIQUE_ID               EFUSE_BLK0          21                ␣
↪1
76      WR_DIS.BLK_VERSION_MAJOR                EFUSE_BLK0          21                ␣
↪1
77      WR_DIS.TEMP_CALIB                       EFUSE_BLK0          21                ␣
↪1
78      WR_DIS.OCODE                            EFUSE_BLK0          21                ␣
↪1
79      WR_DIS.ADC1_INIT_CODE_ATTEN0            EFUSE_BLK0          21                ␣
↪1
80      WR_DIS.ADC1_INIT_CODE_ATTEN1            EFUSE_BLK0          21                ␣
↪1
81      WR_DIS.ADC1_INIT_CODE_ATTEN2            EFUSE_BLK0          21                ␣
↪1
82      WR_DIS.ADC1_INIT_CODE_ATTEN3            EFUSE_BLK0          21                ␣
↪1
83      WR_DIS.ADC2_INIT_CODE_ATTEN0            EFUSE_BLK0          21                ␣
↪1
84      WR_DIS.ADC2_INIT_CODE_ATTEN1            EFUSE_BLK0          21                ␣
↪1
85      WR_DIS.ADC2_INIT_CODE_ATTEN2            EFUSE_BLK0          21                ␣
↪1
```

(continues on next page)

(continued from previous page)

```
86      WR_DIS.ADC2_INIT_CODE_ATTEN3            EFUSE_BLK0          21          ␣
↪1
87      WR_DIS.ADC1_CAL_VOL_ATTEN0              EFUSE_BLK0          21          ␣
↪1
88      WR_DIS.ADC1_CAL_VOL_ATTEN1              EFUSE_BLK0          21          ␣
↪1
89      WR_DIS.ADC1_CAL_VOL_ATTEN2              EFUSE_BLK0          21          ␣
↪1
90      WR_DIS.ADC1_CAL_VOL_ATTEN3              EFUSE_BLK0          21          ␣
↪1
91      WR_DIS.ADC2_CAL_VOL_ATTEN0              EFUSE_BLK0          21          ␣
↪1
92      WR_DIS.ADC2_CAL_VOL_ATTEN1              EFUSE_BLK0          21          ␣
↪1
93      WR_DIS.ADC2_CAL_VOL_ATTEN2              EFUSE_BLK0          21          ␣
↪1
94      WR_DIS.BLOCK_USR_DATA                   EFUSE_BLK0          22          ␣
↪1
95      WR_DIS.CUSTOM_MAC                       EFUSE_BLK0          22          ␣
↪1
96      WR_DIS.BLOCK_KEY0                       EFUSE_BLK0          23          ␣
↪1
97      WR_DIS.BLOCK_KEY1                       EFUSE_BLK0          24          ␣
↪1
98      WR_DIS.BLOCK_KEY2                       EFUSE_BLK0          25          ␣
↪1
99      WR_DIS.BLOCK_KEY3                       EFUSE_BLK0          26          ␣
↪1
100     WR_DIS.BLOCK_KEY4                       EFUSE_BLK0          27          ␣
↪1
101     WR_DIS.BLOCK_KEY5                       EFUSE_BLK0          28          ␣
↪1
102     WR_DIS.BLOCK_SYS_DATA2                  EFUSE_BLK0          29          ␣
↪1
103     WR_DIS.USB_EXCHG_PINS                   EFUSE_BLK0          30          ␣
↪1
104     WR_DIS.USB_EXT_PHY_ENABLE               EFUSE_BLK0          30          ␣
↪1
105     WR_DIS.SOFT_DIS_JTAG                    EFUSE_BLK0          31          ␣
↪1
106     RD_DIS                                  EFUSE_BLK0          32          ␣
↪7
107     RD_DIS.BLOCK_KEY0                       EFUSE_BLK0          32          ␣
↪1
108     RD_DIS.BLOCK_KEY1                       EFUSE_BLK0          33          ␣
↪1
109     RD_DIS.BLOCK_KEY2                       EFUSE_BLK0          34          ␣
↪1
110     RD_DIS.BLOCK_KEY3                       EFUSE_BLK0          35          ␣
↪1
111     RD_DIS.BLOCK_KEY4                       EFUSE_BLK0          36          ␣
↪1
112     RD_DIS.BLOCK_KEY5                       EFUSE_BLK0          37          ␣
↪1
113     RD_DIS.BLOCK_SYS_DATA2                  EFUSE_BLK0          38          ␣
↪1
114     DIS_ICACHE                              EFUSE_BLK0          40          ␣
↪1
115     DIS_DCACHE                              EFUSE_BLK0          41          ␣
↪1
116     DIS_DOWNLOAD_ICACHE                     EFUSE_BLK0          42          ␣
↪1
```

(continues on next page)

(continued from previous page)

```
117     DIS_DOWNLOAD_DCACHE                     EFUSE_BLK0          43  ␣
↪1
118     DIS_FORCE_DOWNLOAD                      EFUSE_BLK0          44  ␣
↪1
119     DIS_USB_OTG                             EFUSE_BLK0          45  ␣
↪1
120     DIS_TWAI                                EFUSE_BLK0          46  ␣
↪1
121     DIS_APP_CPU                             EFUSE_BLK0          47  ␣
↪1
122     SOFT_DIS_JTAG                           EFUSE_BLK0          48  ␣
↪3
123     DIS_PAD_JTAG                            EFUSE_BLK0          51  ␣
↪1
124     DIS_DOWNLOAD_MANUAL_ENCRYPT             EFUSE_BLK0          52  ␣
↪1
125     USB_EXCHG_PINS                          EFUSE_BLK0          57  ␣
↪1
126     USB_EXT_PHY_ENABLE                      EFUSE_BLK0          58  ␣
↪1
127     VDD_SPI_XPD                             EFUSE_BLK0          68  ␣
↪1
128     VDD_SPI_TIEH                            EFUSE_BLK0          69  ␣
↪1
129     VDD_SPI_FORCE                           EFUSE_BLK0          70  ␣
↪1
130     WDT_DELAY_SEL                           EFUSE_BLK0          80  ␣
↪2
131     SPI_BOOT_CRYPT_CNT                      EFUSE_BLK0          82  ␣
↪3
132     SECURE_BOOT_KEY_REVOKE0                 EFUSE_BLK0          85  ␣
↪1
133     SECURE_BOOT_KEY_REVOKE1                 EFUSE_BLK0          86  ␣
↪1
134     SECURE_BOOT_KEY_REVOKE2                 EFUSE_BLK0          87  ␣
↪1
135     KEY_PURPOSE_0                           EFUSE_BLK0          88  ␣
↪4
136     KEY_PURPOSE_1                           EFUSE_BLK0          92  ␣
↪4
137     KEY_PURPOSE_2                           EFUSE_BLK0          96  ␣
↪4
138     KEY_PURPOSE_3                           EFUSE_BLK0         100  ␣
↪4
139     KEY_PURPOSE_4                           EFUSE_BLK0         104  ␣
↪4
140     KEY_PURPOSE_5                           EFUSE_BLK0         108  ␣
↪4
141     SECURE_BOOT_EN                          EFUSE_BLK0         116  ␣
↪1
142     SECURE_BOOT_AGGRESSIVE_REVOKE           EFUSE_BLK0         117  ␣
↪1
143     DIS_USB_JTAG                            EFUSE_BLK0         118  ␣
↪1
144     DIS_USB_SERIAL_JTAG                     EFUSE_BLK0         119  ␣
↪1
145     STRAP_JTAG_SEL                          EFUSE_BLK0         120  ␣
↪1
146     USB_PHY_SEL                             EFUSE_BLK0         121  ␣
↪1
147     FLASH_TPUW                              EFUSE_BLK0         124  ␣
↪4
```

(continues on next page)

(continued from previous page)

```
148     DIS_DOWNLOAD_MODE                       EFUSE_BLK0          128          ␣
↪1
149     DIS_DIRECT_BOOT                         EFUSE_BLK0          129          ␣
↪1
150     DIS_USB_SERIAL_JTAG_ROM_PRINT           EFUSE_BLK0          130          ␣
↪1
151     FLASH_ECC_MODE                          EFUSE_BLK0          131          ␣
↪1
152     DIS_USB_SERIAL_JTAG_DOWNLOAD_MODE       EFUSE_BLK0          132          ␣
↪1
153     ENABLE_SECURITY_DOWNLOAD                EFUSE_BLK0          133          ␣
↪1
154     UART_PRINT_CONTROL                      EFUSE_BLK0          134          ␣
↪2
155     PIN_POWER_SELECTION                     EFUSE_BLK0          136          ␣
↪1
156     FLASH_TYPE                              EFUSE_BLK0          137          ␣
↪1
157     FLASH_PAGE_SIZE                         EFUSE_BLK0          138          ␣
↪2
158     FLASH_ECC_EN                            EFUSE_BLK0          140          ␣
↪1
159     FORCE_SEND_RESUME                       EFUSE_BLK0          141          ␣
↪1
160     SECURE_VERSION                          EFUSE_BLK0          142          ␣
↪16
161     DIS_USB_OTG_DOWNLOAD_MODE               EFUSE_BLK0          159          ␣
↪1
162     DISABLE_WAFER_VERSION_MAJOR             EFUSE_BLK0          160          ␣
↪1
163     DISABLE_BLK_VERSION_MAJOR               EFUSE_BLK0          161          ␣
↪1
164     MAC                                     EFUSE_BLK1           0           ␣
↪8
165     MAC                                     EFUSE_BLK1           8           ␣
↪8
166     MAC                                     EFUSE_BLK1           16          ␣
↪8
167     MAC                                     EFUSE_BLK1           24          ␣
↪8
168     MAC                                     EFUSE_BLK1           32          ␣
↪8
169     MAC                                     EFUSE_BLK1           40          ␣
↪8
170     SPI_PAD_CONFIG_CLK                      EFUSE_BLK1           48          ␣
↪6
171     SPI_PAD_CONFIG_Q                        EFUSE_BLK1           54          ␣
↪6
172     SPI_PAD_CONFIG_D                        EFUSE_BLK1           60          ␣
↪6
173     SPI_PAD_CONFIG_CS                       EFUSE_BLK1           66          ␣
↪6
174     SPI_PAD_CONFIG_HD                       EFUSE_BLK1           72          ␣
↪6
175     SPI_PAD_CONFIG_WP                       EFUSE_BLK1           78          ␣
↪6
176     SPI_PAD_CONFIG_DQS                      EFUSE_BLK1           84          ␣
↪6
177     SPI_PAD_CONFIG_D4                       EFUSE_BLK1           90          ␣
↪6
178     SPI_PAD_CONFIG_D5                       EFUSE_BLK1           96          ␣
↪6
```

(continues on next page)

(continued from previous page)

```
179     SPI_PAD_CONFIG_D6                       EFUSE_BLK1          102                 ␣
↪6
180     SPI_PAD_CONFIG_D7                       EFUSE_BLK1          108                 ␣
↪6
181     WAFER_VERSION_MINOR_LO                  EFUSE_BLK1          114                 ␣
↪3
182     PKG_VERSION                             EFUSE_BLK1          117                 ␣
↪3
183     BLK_VERSION_MINOR                       EFUSE_BLK1          120                 ␣
↪3
184     K_RTC_LDO                               EFUSE_BLK1          141                 ␣
↪7
185     K_DIG_LDO                               EFUSE_BLK1          148                 ␣
↪7
186     V_RTC_DBIAS20                           EFUSE_BLK1          155                 ␣
↪8
187     V_DIG_DBIAS20                           EFUSE_BLK1          163                 ␣
↪8
188     DIG_DBIAS_HVT                           EFUSE_BLK1          171                 ␣
↪5
189     WAFER_VERSION_MINOR_HI                  EFUSE_BLK1          183                 ␣
↪1
190     WAFER_VERSION_MAJOR                     EFUSE_BLK1          184                 ␣
↪2
191     ADC2_CAL_VOL_ATTEN3                     EFUSE_BLK1          186                 ␣
↪6
192     SYS_DATA_PART2                          EFUSE_BLK10          0                  ␣
↪256
193     OPTIONAL_UNIQUE_ID                      EFUSE_BLK2           0                  ␣
↪128
194     BLK_VERSION_MAJOR                       EFUSE_BLK2          128                 ␣
↪2
195     TEMP_CALIB                              EFUSE_BLK2          132                 ␣
↪9
196     OCODE                                   EFUSE_BLK2          141                 ␣
↪8
197     ADC1_INIT_CODE_ATTEN0                   EFUSE_BLK2          149                 ␣
↪8
198     ADC1_INIT_CODE_ATTEN1                   EFUSE_BLK2          157                 ␣
↪6
199     ADC1_INIT_CODE_ATTEN2                   EFUSE_BLK2          163                 ␣
↪6
200     ADC1_INIT_CODE_ATTEN3                   EFUSE_BLK2          169                 ␣
↪6
201     ADC2_INIT_CODE_ATTEN0                   EFUSE_BLK2          175                 ␣
↪8
202     ADC2_INIT_CODE_ATTEN1                   EFUSE_BLK2          183                 ␣
↪6
203     ADC2_INIT_CODE_ATTEN2                   EFUSE_BLK2          189                 ␣
↪6
204     ADC2_INIT_CODE_ATTEN3                   EFUSE_BLK2          195                 ␣
↪6
205     ADC1_CAL_VOL_ATTEN0                     EFUSE_BLK2          201                 ␣
↪8
206     ADC1_CAL_VOL_ATTEN1                     EFUSE_BLK2          209                 ␣
↪8
207     ADC1_CAL_VOL_ATTEN2                     EFUSE_BLK2          217                 ␣
↪8
208     ADC1_CAL_VOL_ATTEN3                     EFUSE_BLK2          225                 ␣
↪8
209     ADC2_CAL_VOL_ATTEN0                     EFUSE_BLK2          233                 ␣
↪8
```

(continues on next page)

(continued from previous page)

```
210     ADC2_CAL_VOL_ATTEN1                     EFUSE_BLK2          241                 ␣
↪7
211     ADC2_CAL_VOL_ATTEN2                     EFUSE_BLK2          248                 ␣
↪7
212     USER_DATA                               EFUSE_BLK3          0                   ␣
↪256
213     USER_DATA.MAC_CUSTOM                    EFUSE_BLK3          200                 ␣
↪48
214     KEY0                                    EFUSE_BLK4          0                   ␣
↪256
215     KEY1                                    EFUSE_BLK5          0                   ␣
↪256
216     KEY2                                    EFUSE_BLK6          0                   ␣
↪256
217     KEY3                                    EFUSE_BLK7          0                   ␣
↪256
218     KEY4                                    EFUSE_BLK8          0                   ␣
↪256
219     KEY5                                    EFUSE_BLK9          0                   ␣
↪256

Used bits in efuse table:
EFUSE_BLK0
[0 31] [0 0] [2 2] ... [22 30] [30 38] [32 38] [40 52] [57 58] [68 70] [80 111]␣
↪[116 121] [124 157] [159 161]
EFUSE_BLK1
[0 122] [141 175] [183 191]
EFUSE_BLK10
[0 255]
EFUSE_BLK2
[0 129] [132 254]
EFUSE_BLK3
[0 255] [200 247]
EFUSE_BLK4
[0 255]
EFUSE_BLK5
[0 255]
EFUSE_BLK6
[0 255]
EFUSE_BLK7
[0 255]
EFUSE_BLK8
[0 255]
EFUSE_BLK9
[0 255]
Note: Not printed ranges are free for using. (bits in EFUSE_BLK0 are reserved for␣
↪Espressif)
```

The number of bits not included in square brackets is free (some bits are reserved for Espressif). All fields are checked for overlapping.

To add fields to an existing field, use the *Structured efuse fields* technique. For example, adding the fields: SERIAL_NUMBER, MODEL_NUMBER and HARDWARE REV to an existing `USER_DATA` field. Use `.` (dot) to show an attachment in a field.

```
USER_DATA.SERIAL_NUMBER,                EFUSE_BLK3,     0,  32,
USER_DATA.MODEL_NUMBER,                 EFUSE_BLK3,     32, 10,
USER_DATA.HARDWARE_REV,                 EFUSE_BLK3,     42, 10,
```

2. Fill a line for field: field_name, efuse_block, bit_start, bit_count, comment.
3. Run a `show_efuse_table` command to check eFuse table. To generate source files run `efuse_common_table` or `efuse_custom_table` command.

You may get errors such as `intersects with` or `out of range`. Please see how to solve them in the *Structured efuse fields* article.

Bit Order

The eFuses bit order is little endian (see the example below), it means that eFuse bits are read and written from LSB to MSB:

```
$ espefuse.py dump

USER_DATA       (BLOCK3          ) [3 ] read_regs: 03020100 07060504 0B0A0908␣
↪0F0E0D0C 13121111 17161514 1B1A1918 1F1E1D1C
BLOCK4          (BLOCK4          ) [4 ] read_regs: 03020100 07060504 0B0A0908␣
↪0F0E0D0C 13121111 17161514 1B1A1918 1F1E1D1C

where is the register representation:

EFUSE_RD_USR_DATA0_REG = 0x03020100
EFUSE_RD_USR_DATA1_REG = 0x07060504
EFUSE_RD_USR_DATA2_REG = 0x0B0A0908
EFUSE_RD_USR_DATA3_REG = 0x0F0E0D0C
EFUSE_RD_USR_DATA4_REG = 0x13121111
EFUSE_RD_USR_DATA5_REG = 0x17161514
EFUSE_RD_USR_DATA6_REG = 0x1B1A1918
EFUSE_RD_USR_DATA7_REG = 0x1F1E1D1C

where is the byte representation:

byte[0] = 0x00, byte[1] = 0x01, ... byte[3] = 0x03, byte[4] = 0x04, ..., byte[31]␣
↪= 0x1F
```

For example, csv file describes the `USER_DATA` field, which occupies all 256 bits (a whole block).

```
USER_DATA,          EFUSE_BLK3,     0,  256,     User data
USER_DATA.FIELD1,   EFUSE_BLK3,     16,  16,     Field1

ID,                 EFUSE_BLK4,     8,  3,      ID bit[0..2]
,                   EFUSE_BLK4,     16, 2,      ID bit[3..4]
,                   EFUSE_BLK4,     32, 3,      ID bit[5..7]
```

Thus, reading the eFuse `USER_DATA` block written as above gives the following results:

```
uint8_t buf[32] = { 0 };
esp_efuse_read_field_blob(ESP_EFUSE_USER_DATA, &buf, sizeof(buf) * 8);
// buf[0] = 0x00, buf[1] = 0x01, ... buf[31] = 0x1F

uint32_t field1 = 0;
size_t field1_size = ESP_EFUSE_USER_DATA[0]->bit_count; // can be used for this␣
↪case because it only consists of one entry
esp_efuse_read_field_blob(ESP_EFUSE_USER_DATA, &field1, field1_size);
// field1 = 0x0302

uint32_t field1_1 = 0;
esp_efuse_read_field_blob(ESP_EFUSE_USER_DATA, &field1_1, 2); // reads only first␣
↪2 bits
// field1 = 0x0002

uint8_t id = 0;
size_t id_size = esp_efuse_get_field_size(ESP_EFUSE_ID); // returns 6
// size_t id_size = ESP_EFUSE_USER_DATA[0]->bit_count; // cannot be used because␣
↪it consists of 3 entries. It returns 3 not 6.
```

(continues on next page)

(continued from previous page)

```
esp_efuse_read_field_blob(ESP_EFUSE_ID, &id, id_size);
// id = 0x91
// b'100 10  001
//   [3] [2] [3]

uint8_t id_1 = 0;
esp_efuse_read_field_blob(ESP_EFUSE_ID, &id_1, 3);
// id = 0x01
// b'001
```

Get eFuses During Build

There is a way to get the state of eFuses at the build stage of the project. There are two cmake functions for this:

- `espefuse_get_json_summary()` - It calls the `espefuse.py summary --format json` command and returns a json string (it is not stored in a file).
- `espefuse_get_efuse()` - It finds a given eFuse name in the json string and returns its property.

The json string has the following properties:

```
{
    "MAC": {
        "bit_len": 48,
        "block": 0,
        "category": "identity",
        "description": "Factory MAC Address",
        "efuse_type": "bytes:6",
        "name": "MAC",
        "pos": 0,
        "readable": true,
        "value": "94:b9:7e:5a:6e:58 (CRC 0xe2 OK)",
        "word": 1,
        "writeable": true
    },
}
```

These functions can be used from a top-level project `CMakeLists.txt` (get-started/hello_world/CMakeLists.txt):

```
# ...
project(hello_world)

espefuse_get_json_summary(efuse_json)
espefuse_get_efuse(ret_data ${efuse_json} "MAC" "value")
message("MAC:" ${ret_data})
```

The format of the `value` property is the same as shown in `espefuse.py summary`.

```
MAC:94:b9:7e:5a:6e:58 (CRC 0xe2 OK)
```

There is an example test system/efuse/CMakeLists.txt which adds a custom target `efuse-summary`. This allows you to run the `idf.py efuse-summary` command to read the required eFuses (specified in the `efuse_names` list) at any time, not just at project build time.

Debug eFuse & Unit Tests

Virtual eFuses The Kconfig option *CONFIG_EFUSE_VIRTUAL* virtualizes eFuse values inside the eFuse Manager, so writes are emulated and no eFuse values are permanently changed. This can be useful for debugging app and unit tests. During startup, the eFuses are copied to RAM. All eFuse operations (read and write) are performed with RAM instead of the real eFuse registers.

In addition to the *CONFIG_EFUSE_VIRTUAL* option there is *CONFIG_EFUSE_VIRTUAL_KEEP_IN_FLASH* option that adds a feature to keep eFuses in flash memory. To use this mode the partition_table should have the *efuse* partition. partition.csv: "`efuse_em, data, efuse, , 0x2000,`". During startup, the eFuses are copied from flash or, in case if flash is empty, from real eFuse to RAM and then update flash. This option allows keeping eFuses after reboots (possible to test secure_boot and flash_encryption features with this option).

Flash Encryption Testing Flash Encryption (FE) is a hardware feature that requires the physical burning of eFuses: key and FLASH_CRYPT_CNT. If FE is not actually enabled then enabling the *CONFIG_EFUSE_VIRTUAL_KEEP_IN_FLASH* option just gives testing possibilities and does not encrypt anything in the flash, even though the logs say encryption happens. The `bootloader_flash_write()` is adapted for this purpose. But if FE is already enabled on the chip and you run an application or bootloader created with the *CONFIG_EFUSE_VIRTUAL_KEEP_IN_FLASH* option then the flash encryption/decryption operations will work properly (data are encrypted as it is written into an encrypted flash partition and decrypted when they are read from an encrypted partition).

`espefuse.py` esptool includes a useful tool for reading/writing ESP32-S3 eFuse bits - espefuse.py.

```
espefuse.py -p PORT summary

espefuse.py v4.6-dev
Connecting....
Detecting chip type... ESP32-S3

=== Run "summary" command ===
EFUSE_NAME (Block) Description  = [Meaningful Value] [Readable/Writeable] (Hex␣
↪Value)
----------------------------------------------------------------------------␣
↪-----
Calibration fuses:
K_RTC_LDO (BLOCK1)                                    BLOCK1 K_RTC_LDO          ␣
↪                    = 12 R/W (0b0000011)
K_DIG_LDO (BLOCK1)                                    BLOCK1 K_DIG_LDO          ␣
↪                    = -28 R/W (0b1000111)
V_RTC_DBIAS20 (BLOCK1)                                BLOCK1 voltage of rtc dbias20 ␣
↪                    = 20 R/W (0x05)
V_DIG_DBIAS20 (BLOCK1)                                BLOCK1 voltage of digital␣
↪dbias20                    = -44 R/W (0x8b)
DIG_DBIAS_HVT (BLOCK1)                                BLOCK1 digital dbias when hvt ␣
↪                    = -36 R/W (0b11001)
ADC2_CAL_VOL_ATTEN3 (BLOCK1)                          ADC2 calibration voltage at␣
↪atten3                 = -24 R/W (0b100110)
TEMP_CALIB (BLOCK2)                                   Temperature calibration data ␣
↪                    = -10.9 R/W (0b1101101101)
OCODE (BLOCK2)                                        ADC OCode                 ␣
↪                    = 88 R/W (0x58)
ADC1_INIT_CODE_ATTEN0 (BLOCK2)                        ADC1 init code at atten0  ␣
↪                    = 432 R/W (0x6c)
ADC1_INIT_CODE_ATTEN1 (BLOCK2)                        ADC1 init code at atten1  ␣
↪                    = -16 R/W (0b100100)
ADC1_INIT_CODE_ATTEN2 (BLOCK2)                        ADC1 init code at atten2  ␣
↪                    = 88 R/W (0b010110)
ADC1_INIT_CODE_ATTEN3 (BLOCK2)                        ADC1 init code at atten3  ␣
↪                    = 0 R/W (0b100000)
ADC2_INIT_CODE_ATTEN0 (BLOCK2)                        ADC2 init code at atten0  ␣
↪                    = -72 R/W (0x92)
ADC2_INIT_CODE_ATTEN1 (BLOCK2)                        ADC2 init code at atten1  ␣
↪                    = -16 R/W (0b100100)
ADC2_INIT_CODE_ATTEN2 (BLOCK2)                        ADC2 init code at atten2  ␣
↪                    = 48 R/W (0b001100)
ADC2_INIT_CODE_ATTEN3 (BLOCK2)                        ADC2 init code at atten3  ␣
↪                    = 112 R/W (0b011100)
```

(continues on next page)

(continued from previous page)

```
ADC1_CAL_VOL_ATTEN0 (BLOCK2)                            ADC1 calibration voltage at␣
↪atten0                 = 412 R/W (0x67)
ADC1_CAL_VOL_ATTEN1 (BLOCK2)                            ADC1 calibration voltage at␣
↪atten1                 = 392 R/W (0x62)
ADC1_CAL_VOL_ATTEN2 (BLOCK2)                            ADC1 calibration voltage at␣
↪atten2                 = 356 R/W (0x59)
ADC1_CAL_VOL_ATTEN3 (BLOCK2)                            ADC1 calibration voltage at␣
↪atten3                 = 412 R/W (0x67)
ADC2_CAL_VOL_ATTEN0 (BLOCK2)                            ADC2 calibration voltage at␣
↪atten0                 = -116 R/W (0x9d)
ADC2_CAL_VOL_ATTEN1 (BLOCK2)                            ADC2 calibration voltage at␣
↪atten1                 = -72 R/W (0b1010010)
ADC2_CAL_VOL_ATTEN2 (BLOCK2)                            ADC2 calibration voltage at␣
↪atten2                 = -64 R/W (0b1010000)

Config fuses:
WR_DIS (BLOCK0)                                         Disable programming of␣
↪individual eFuses           = 0 R/W (0x00000000)
RD_DIS (BLOCK0)                                         Disable reading from BlOCK4-10 ␣
↪                   = 0 R/W (0b0000000)
DIS_ICACHE (BLOCK0)                                     Set this bit to disable Icache ␣
↪                   = False R/W (0b0)
DIS_DCACHE (BLOCK0)                                     Set this bit to disable Dcache ␣
↪                   = False R/W (0b0)
DIS_TWAI (BLOCK0)                                       Set this bit to disable CAN␣
↪function               = False R/W (0b0)
DIS_APP_CPU (BLOCK0)                                    Disable app cpu                ␣
↪                   = False R/W (0b0)
DIS_DIRECT_BOOT (BLOCK0)                                Disable direct boot mode       ␣
↪                   = False R/W (0b0)
UART_PRINT_CONTROL (BLOCK0)                             Set the default UART boot␣
↪message output mode      = Enable R/W (0b00)
PIN_POWER_SELECTION (BLOCK0)                            Set default power supply for␣
↪GPIO33-GPIO37; set wh = VDD3P3_CPU R/W (0b0)
                                                        en SPI flash is initialized
BLOCK_USR_DATA (BLOCK3)                                 User data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
BLOCK_SYS_DATA2 (BLOCK10)                               System data part 2 (reserved)
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W

Flash fuses:
FLASH_TPUW (BLOCK0)                                     Configures flash waiting time␣
↪after power-up; in u = 0 R/W (0x0)
                                                        nit of ms. If the value is less␣
↪than 15; the waiti
                                                        ng time is the configurable␣
↪value.  Otherwise; the
                                                         waiting time is twice the␣
↪configurable value
FLASH_ECC_MODE (BLOCK0)                                 Flash ECC mode in ROM          ␣
↪                   = 16to18 byte R/W (0b0)
FLASH_TYPE (BLOCK0)                                     SPI flash type                 ␣
↪                   = 4 data lines R/W (0b0)
FLASH_PAGE_SIZE (BLOCK0)                                Set Flash page size            ␣
↪                   = 0 R/W (0b00)
FLASH_ECC_EN (BLOCK0)                                   Set 1 to enable ECC for flash␣
↪boot                 = False R/W (0b0)
FORCE_SEND_RESUME (BLOCK0)                              Set this bit to force ROM code␣
↪to send a resume co = False R/W (0b0)
```

(continues on next page)

(continued from previous page)

```
                                                    mmand during SPI boot

Identity fuses:
DISABLE_WAFER_VERSION_MAJOR (BLOCK0)                Disables check of wafer version␣
↪major                = False R/W (0b0)
DISABLE_BLK_VERSION_MAJOR (BLOCK0)                  Disables check of blk version␣
↪major                  = False R/W (0b0)
WAFER_VERSION_MINOR_LO (BLOCK1)                     WAFER_VERSION_MINOR least␣
↪significant bits          = 3 R/W (0b011)
PKG_VERSION (BLOCK1)                                Package version               ␣
↪                      = 0 R/W (0b000)
BLK_VERSION_MINOR (BLOCK1)                          BLK_VERSION_MINOR             ␣
↪                      = 3 R/W (0b011)
WAFER_VERSION_MINOR_HI (BLOCK1)                     WAFER_VERSION_MINOR most␣
↪significant bit            = False R/W (0b0)
WAFER_VERSION_MAJOR (BLOCK1)                        WAFER_VERSION_MAJOR           ␣
↪                      = 0 R/W (0b00)
OPTIONAL_UNIQUE_ID (BLOCK2)                         Optional unique 128-bit ID
   = cb 3a c9 b8 88 2b c3 bc 5e f4 00 60 ac 25 be 4b R/W
BLK_VERSION_MAJOR (BLOCK2)                          BLK_VERSION_MAJOR of BLOCK2   ␣
↪                      = ADC calib V1 R/W (0b01)
WAFER_VERSION_MINOR (BLOCK0)                        calc WAFER VERSION MINOR =␣
↪WAFER_VERSION_MINOR_HI  = 3 R/W (0x3)
                                                    << 3 + WAFER_VERSION_MINOR_LO␣
↪(read only)

Jtag fuses:
SOFT_DIS_JTAG (BLOCK0)                              Set these bits to disable JTAG␣
↪in the soft way (od = 0 R/W (0b000)
                                                    d number 1 means disable ).␣
↪JTAG can be enabled in
                                                     HMAC module
DIS_PAD_JTAG (BLOCK0)                               Set this bit to disable JTAG in␣
↪the hard way. JTAG = False R/W (0b0)
                                                     is disabled permanently
STRAP_JTAG_SEL (BLOCK0)                             Set this bit to enable␣
↪selection between usb_to_jt = False R/W (0b0)
                                                    ag and pad_to_jtag through␣
↪strapping gpio10 when b
                                                    oth reg_dis_usb_jtag and reg_
↪dis_pad_jtag are equa
                                                    l to 0

Mac fuses:
MAC (BLOCK1)                                        MAC address
   = ec:da:3b:41:f2:70 (OK) R/W
CUSTOM_MAC (BLOCK3)                                 Custom MAC
   = 00:00:00:00:00:00 (OK) R/W

Security fuses:
DIS_DOWNLOAD_ICACHE (BLOCK0)                        Set this bit to disable Icache␣
↪in download mode (b = False R/W (0b0)
                                                    oot_mode[3:0] is 0; 1; 2; 3; 6;␣
↪7)
DIS_DOWNLOAD_DCACHE (BLOCK0)                        Set this bit to disable Dcache␣
↪in download mode (  = False R/W (0b0)
                                                    boot_mode[3:0] is 0; 1; 2; 3; 6;
↪ 7)
DIS_FORCE_DOWNLOAD (BLOCK0)                         Set this bit to disable the␣
↪function that forces c = False R/W (0b0)
                                                    hip into download mode
```

(continues on next page)

(continued from previous page)

```
DIS_DOWNLOAD_MANUAL_ENCRYPT (BLOCK0)                    Set this bit to disable flash␣
↪encryption when in d = False R/W (0b0)
                                                        ownload boot modes
SPI_BOOT_CRYPT_CNT (BLOCK0)                             Enables flash encryption when 1␣
↪or 3 bits are set  = Disable R/W (0b000)
                                                        and disabled otherwise
SECURE_BOOT_KEY_REVOKE0 (BLOCK0)                        Revoke 1st secure boot key     ␣
↪                   = False R/W (0b0)
SECURE_BOOT_KEY_REVOKE1 (BLOCK0)                        Revoke 2nd secure boot key     ␣
↪                   = False R/W (0b0)
SECURE_BOOT_KEY_REVOKE2 (BLOCK0)                        Revoke 3rd secure boot key     ␣
↪                   = False R/W (0b0)
KEY_PURPOSE_0 (BLOCK0)                                  Purpose of Key0                ␣
↪                   = USER R/W (0x0)
KEY_PURPOSE_1 (BLOCK0)                                  Purpose of Key1                ␣
↪                   = USER R/W (0x0)
KEY_PURPOSE_2 (BLOCK0)                                  Purpose of Key2                ␣
↪                   = USER R/W (0x0)
KEY_PURPOSE_3 (BLOCK0)                                  Purpose of Key3                ␣
↪                   = USER R/W (0x0)
KEY_PURPOSE_4 (BLOCK0)                                  Purpose of Key4                ␣
↪                   = USER R/W (0x0)
KEY_PURPOSE_5 (BLOCK0)                                  Purpose of Key5                ␣
↪                   = USER R/W (0x0)
SECURE_BOOT_EN (BLOCK0)                                 Set this bit to enable secure␣
↪boot                   = False R/W (0b0)
SECURE_BOOT_AGGRESSIVE_REVOKE (BLOCK0)                  Set this bit to enable revoking␣
↪aggressive secure  = False R/W (0b0)
                                                        boot
DIS_DOWNLOAD_MODE (BLOCK0)                              Set this bit to disable␣
↪download mode (boot_mode[3 = False R/W (0b0)
                                                        :0] = 0; 1; 2; 3; 6; 7)
ENABLE_SECURITY_DOWNLOAD (BLOCK0)                       Set this bit to enable secure␣
↪UART download mode   = False R/W (0b0)
SECURE_VERSION (BLOCK0)                                 Secure version (used by ESP-IDF␣
↪anti-rollback feat = 0 R/W (0x0000)
                                                        ure)
BLOCK_KEY0 (BLOCK4)
  Purpose: USER
                Key0 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
BLOCK_KEY1 (BLOCK5)
  Purpose: USER
                Key1 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
BLOCK_KEY2 (BLOCK6)
  Purpose: USER
                Key2 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
BLOCK_KEY3 (BLOCK7)
  Purpose: USER
                Key3 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
BLOCK_KEY4 (BLOCK8)
  Purpose: USER
                Key4 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W
```

(continues on next page)

(continued from previous page)

```
BLOCK_KEY5 (BLOCK9)
  Purpose: USER
               Key5 or user data
   = 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00␣
↪00 00 00 00 00 00 R/W

Spi Pad fuses:
SPI_PAD_CONFIG_CLK (BLOCK1)                            SPI_PAD_configure CLK          ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_Q (BLOCK1)                              SPI_PAD_configure Q(D1)        ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_D (BLOCK1)                              SPI_PAD_configure D(D0)        ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_CS (BLOCK1)                             SPI_PAD_configure CS           ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_HD (BLOCK1)                             SPI_PAD_configure HD(D3)       ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_WP (BLOCK1)                             SPI_PAD_configure WP(D2)       ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_DQS (BLOCK1)                            SPI_PAD_configure DQS          ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_D4 (BLOCK1)                             SPI_PAD_configure D4           ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_D5 (BLOCK1)                             SPI_PAD_configure D5           ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_D6 (BLOCK1)                             SPI_PAD_configure D6           ␣
↪                    = 0 R/W (0b000000)
SPI_PAD_CONFIG_D7 (BLOCK1)                             SPI_PAD_configure D7           ␣
↪                    = 0 R/W (0b000000)

Usb fuses:
DIS_USB_OTG (BLOCK0)                                   Set this bit to disable USB␣
↪function               = False R/W (0b0)
USB_EXCHG_PINS (BLOCK0)                                Set this bit to exchange USB D+␣
↪and D- pins        = False R/W (0b0)
USB_EXT_PHY_ENABLE (BLOCK0)                            Set this bit to enable external␣
↪PHY                = False R/W (0b0)
DIS_USB_JTAG (BLOCK0)                                  Set this bit to disable␣
↪function of usb switch to  = False R/W (0b0)
                                                       jtag in module of usb device
DIS_USB_SERIAL_JTAG (BLOCK0)                           Set this bit to disable usb␣
↪device                 = False R/W (0b0)
USB_PHY_SEL (BLOCK0)                                   This bit is used to switch␣
↪internal PHY and extern
   = internal PHY is assigned to USB Device while external PHY is assigned to USB␣
↪OTG R/W (0b0)
                                                       al PHY for USB OTG and USB␣
↪Device
DIS_USB_SERIAL_JTAG_ROM_PRINT (BLOCK0)                 USB printing                   ␣
↪                    = Enable R/W (0b0)
DIS_USB_SERIAL_JTAG_DOWNLOAD_MODE (BLOCK0)             Set this bit to disable UART␣
↪download mode through = False R/W (0b0)
                                                        USB
DIS_USB_OTG_DOWNLOAD_MODE (BLOCK0)                     Set this bit to disable␣
↪download through USB-OTG   = False R/W (0b0)

Vdd fuses:
VDD_SPI_XPD (BLOCK0)                                   SPI regulator power up signal  ␣
↪                    = False R/W (0b0)
VDD_SPI_TIEH (BLOCK0)                                  If VDD_SPI_FORCE is 1;␣
↪determines VDD_SPI voltage
```

(continues on next page)

(continued from previous page)

```
   = VDD_SPI connects to 1.8 V LDO R/W (0b0)
VDD_SPI_FORCE (BLOCK0)                                  Set this bit and force to use␣
↪the configuration of = False R/W (0b0)
                                                         eFuse to configure VDD_SPI

Wdt fuses:
WDT_DELAY_SEL (BLOCK0)                                  RTC watchdog timeout threshold;␣
↪in unit of slow cl = 40000 R/W (0b00)
                                                        ock cycle

Flash voltage (VDD_SPI) determined by GPIO45 on reset (GPIO45=High: VDD_SPI pin is␣
↪powered from internal 1.8V LDO
GPIO45=Low or NC: VDD_SPI pin is powered directly from VDD3P3_RTC_IO via resistor␣
↪Rspi. Typically this voltage is 3.3 V).
```

To get a dump for all eFuse registers.

```
espefuse.py -p PORT dump

espefuse.py v4.6-dev
Connecting....
Detecting chip type... ESP32-S3
BLOCK0          (                ) [0 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000
MAC_SPI_8M_0    (BLOCK1          ) [1 ] read_regs: 3b41f270 0000ecda 00000000␣
↪030c0000 2c707800 9800cc58
BLOCK_SYS_DATA  (BLOCK2          ) [2 ] read_regs: b8c93acb bcc32b88 6000f45e␣
↪4bbe25ac 8d8b16d1 924940b4 b2c4cee1 50a53ace
BLOCK_USR_DATA  (BLOCK3          ) [3 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY0      (BLOCK4          ) [4 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY1      (BLOCK5          ) [5 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY2      (BLOCK6          ) [6 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY3      (BLOCK7          ) [7 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY4      (BLOCK8          ) [8 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_KEY5      (BLOCK9          ) [9 ] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000
BLOCK_SYS_DATA2 (BLOCK10         ) [10] read_regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000 00000000 00000000

BLOCK0          (                ) [0 ] err__regs: 00000000 00000000 00000000␣
↪00000000 00000000 00000000
EFUSE_RD_RS_ERR0_REG        0x00000000
EFUSE_RD_RS_ERR1_REG        0x00000000

=== Run "dump" command ===
```

Header File

- components/efuse/esp32s3/include/esp_efuse_chip.h
- This header file can be included with:

```
#include "esp_efuse_chip.h"
```

- This header file is a part of the API provided by the `efuse` component. To declare that your component depends on `efuse`, add the following to your CMakeLists.txt:

```
REQUIRES efuse
```

or

```
PRIV_REQUIRES efuse
```

Enumerations

enum **esp_efuse_block_t**

Type of eFuse blocks ESP32S3.

Values:

enumerator **EFUSE_BLK0**

Number of eFuse BLOCK0. REPEAT_DATA

enumerator **EFUSE_BLK1**

Number of eFuse BLOCK1. MAC_SPI_8M_SYS

enumerator **EFUSE_BLK2**

Number of eFuse BLOCK2. SYS_DATA_PART1

enumerator **EFUSE_BLK_SYS_DATA_PART1**

Number of eFuse BLOCK2. SYS_DATA_PART1

enumerator **EFUSE_BLK3**

Number of eFuse BLOCK3. USER_DATA

enumerator **EFUSE_BLK_USER_DATA**

Number of eFuse BLOCK3. USER_DATA

enumerator **EFUSE_BLK4**

Number of eFuse BLOCK4. KEY0

enumerator **EFUSE_BLK_KEY0**

Number of eFuse BLOCK4. KEY0

enumerator **EFUSE_BLK5**

Number of eFuse BLOCK5. KEY1

enumerator **EFUSE_BLK_KEY1**

Number of eFuse BLOCK5. KEY1

enumerator **EFUSE_BLK6**

Number of eFuse BLOCK6. KEY2

enumerator **EFUSE_BLK_KEY2**

Number of eFuse BLOCK6. KEY2

enumerator **EFUSE_BLK7**

Number of eFuse BLOCK7. KEY3

enumerator **EFUSE_BLK_KEY3**

Number of eFuse BLOCK7. KEY3

enumerator **EFUSE_BLK8**

Number of eFuse BLOCK8. KEY4

enumerator **EFUSE_BLK_KEY4**

Number of eFuse BLOCK8. KEY4

enumerator **EFUSE_BLK9**

Number of eFuse BLOCK9. KEY5

enumerator **EFUSE_BLK_KEY5**

Number of eFuse BLOCK9. KEY5

enumerator **EFUSE_BLK_KEY_MAX**

enumerator **EFUSE_BLK10**

Number of eFuse BLOCK10. SYS_DATA_PART2

enumerator **EFUSE_BLK_SYS_DATA_PART2**

Number of eFuse BLOCK10. SYS_DATA_PART2

enumerator **EFUSE_BLK_MAX**

enum **esp_efuse_coding_scheme_t**

Type of coding scheme.

Values:

enumerator **EFUSE_CODING_SCHEME_NONE**

None

enumerator **EFUSE_CODING_SCHEME_RS**

Reed-Solomon coding

enum **esp_efuse_purpose_t**

Type of key purpose.

Values:

enumerator **ESP_EFUSE_KEY_PURPOSE_USER**

User purposes (software-only use)

enumerator **ESP_EFUSE_KEY_PURPOSE_RESERVED**

Reserved

enumerator **ESP_EFUSE_KEY_PURPOSE_XTS_AES_256_KEY_1**

XTS_AES_256_KEY_1 (flash/PSRAM encryption)

enumerator **ESP_EFUSE_KEY_PURPOSE_XTS_AES_256_KEY_2**

XTS_AES_256_KEY_2 (flash/PSRAM encryption)

enumerator **ESP_EFUSE_KEY_PURPOSE_XTS_AES_128_KEY**

XTS_AES_128_KEY (flash/PSRAM encryption)

enumerator **ESP_EFUSE_KEY_PURPOSE_HMAC_DOWN_ALL**

HMAC Downstream mode

enumerator **ESP_EFUSE_KEY_PURPOSE_HMAC_DOWN_JTAG**

JTAG soft enable key (uses HMAC Downstream mode)

enumerator **ESP_EFUSE_KEY_PURPOSE_HMAC_DOWN_DIGITAL_SIGNATURE**

Digital Signature peripheral key (uses HMAC Downstream mode)

enumerator **ESP_EFUSE_KEY_PURPOSE_HMAC_UP**

HMAC Upstream mode

enumerator **ESP_EFUSE_KEY_PURPOSE_SECURE_BOOT_DIGEST0**

SECURE_BOOT_DIGEST0 (Secure Boot key digest)

enumerator **ESP_EFUSE_KEY_PURPOSE_SECURE_BOOT_DIGEST1**

SECURE_BOOT_DIGEST1 (Secure Boot key digest)

enumerator **ESP_EFUSE_KEY_PURPOSE_SECURE_BOOT_DIGEST2**

SECURE_BOOT_DIGEST2 (Secure Boot key digest)

enumerator **ESP_EFUSE_KEY_PURPOSE_MAX**

MAX PURPOSE

Header File

- components/efuse/include/esp_efuse.h
- This header file can be included with:

```
#include "esp_efuse.h"
```

- This header file is a part of the API provided by the `efuse` component. To declare that your component depends on `efuse`, add the following to your CMakeLists.txt:

```
REQUIRES efuse
```

or

```
PRIV_REQUIRES efuse
```

Functions

esp_err_t **esp_efuse_read_field_blob** (const *esp_efuse_desc_t* *field[], void *dst, size_t dst_size_bits)

Reads bits from EFUSE field and writes it into an array.

The number of read bits will be limited to the minimum value from the description of the bits in "field" structure or "dst_size_bits" required size. Use "esp_efuse_get_field_size()" function to determine the length of the field.

Note: Please note that reading in the batch mode does not show uncommitted changes.

Parameters
- **field** -- **[in]** A pointer to the structure describing the fields of efuse.
- **dst** -- **[out]** A pointer to array that will contain the result of reading.
- **dst_size_bits** -- **[in]** The number of bits required to read. If the requested number of bits is greater than the field, the number will be limited to the field size.

Returns
- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.

bool **esp_efuse_read_field_bit**(const *esp_efuse_desc_t* *field[])

Read a single bit eFuse field as a boolean value.

Note: The value must exist and must be a single bit wide. If there is any possibility of an error in the provided arguments, call esp_efuse_read_field_blob() and check the returned value instead.

Note: If assertions are enabled and the parameter is invalid, execution will abort

Note: Please note that reading in the batch mode does not show uncommitted changes.

Parameters **field** -- **[in]** A pointer to the structure describing the fields of efuse.

Returns
- true: The field parameter is valid and the bit is set.
- false: The bit is not set, or the parameter is invalid and assertions are disabled.

esp_err_t **esp_efuse_read_field_cnt**(const *esp_efuse_desc_t* *field[], size_t *out_cnt)

Reads bits from EFUSE field and returns number of bits programmed as "1".

If the bits are set not sequentially, they will still be counted.

Note: Please note that reading in the batch mode does not show uncommitted changes.

Parameters
- **field** -- **[in]** A pointer to the structure describing the fields of efuse.
- **out_cnt** -- **[out]** A pointer that will contain the number of programmed as "1" bits.

Returns
- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.

esp_err_t **esp_efuse_write_field_blob**(const *esp_efuse_desc_t* *field[], const void *src, size_t src_size_bits)

Writes array to EFUSE field.

The number of write bits will be limited to the minimum value from the description of the bits in "field" structure or "src_size_bits" required size. Use "esp_efuse_get_field_size()" function to determine the length of the field. After the function is completed, the writing registers are cleared.

Parameters
- **field** -- **[in]** A pointer to the structure describing the fields of efuse.
- **src** -- **[in]** A pointer to array that contains the data for writing.

- **src_size_bits** -- **[in]** The number of bits required to write.

Returns

- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_efuse_write_field_cnt** (const *esp_efuse_desc_t* *field[], size_t cnt)

Writes a required count of bits as "1" to EFUSE field.

If there are no free bits in the field to set the required number of bits to "1", ESP_ERR_EFUSE_CNT_IS_FULL error is returned, the field will not be partially recorded. After the function is completed, the writing registers are cleared.

Parameters

- **field** -- **[in]** A pointer to the structure describing the fields of efuse.
- **cnt** -- **[in]** Required number of programmed as "1" bits.

Returns

- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_CNT_IS_FULL: Not all requested cnt bits is set.

esp_err_t **esp_efuse_write_field_bit** (const *esp_efuse_desc_t* *field[])

Write a single bit eFuse field to 1.

For use with eFuse fields that are a single bit. This function will write the bit to value 1 if it is not already set, or does nothing if the bit is already set.

This is equivalent to calling esp_efuse_write_field_cnt() with the cnt parameter equal to 1, except that it will return ESP_OK if the field is already set to 1.

Parameters **field** -- **[in]** Pointer to the structure describing the efuse field.

Returns

- ESP_OK: The operation was successfully completed, or the bit was already set to value 1.
- ESP_ERR_INVALID_ARG: Error in the passed arugments, including if the efuse field is not 1 bit wide.

esp_err_t **esp_efuse_set_write_protect** (*esp_efuse_block_t* blk)

Sets a write protection for the whole block.

After that, it is impossible to write to this block. The write protection does not apply to block 0.

Parameters **blk** -- **[in]** Block number of eFuse. (EFUSE_BLK1, EFUSE_BLK2 and EFUSE_BLK3)

Returns

- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_CNT_IS_FULL: Not all requested cnt bits is set.
- ESP_ERR_NOT_SUPPORTED: The block does not support this command.

esp_err_t **esp_efuse_set_read_protect** (*esp_efuse_block_t* blk)

Sets a read protection for the whole block.

After that, it is impossible to read from this block. The read protection does not apply to block 0.

Parameters **blk** -- **[in]** Block number of eFuse. (EFUSE_BLK1, EFUSE_BLK2 and EFUSE_BLK3)

Returns

- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_CNT_IS_FULL: Not all requested cnt bits is set.
- ESP_ERR_NOT_SUPPORTED: The block does not support this command.

int **esp_efuse_get_field_size**(const *esp_efuse_desc_t* *field[])

Returns the number of bits used by field.

Parameters **field** -- **[in]** A pointer to the structure describing the fields of efuse.
Returns Returns the number of bits used by field.

uint32_t **esp_efuse_read_reg**(*esp_efuse_block_t* blk, unsigned int num_reg)

Returns value of efuse register.

This is a thread-safe implementation. Example: EFUSE_BLK2_RDATA3_REG where (blk=2, num_reg=3)

Note: Please note that reading in the batch mode does not show uncommitted changes.

Parameters
- **blk** -- **[in]** Block number of eFuse.
- **num_reg** -- **[in]** The register number in the block.

Returns Value of register

esp_err_t **esp_efuse_write_reg**(*esp_efuse_block_t* blk, unsigned int num_reg, uint32_t val)

Write value to efuse register.

Apply a coding scheme if necessary. This is a thread-safe implementation. Example: EFUSE_BLK3_WDATA0_REG where (blk=3, num_reg=0)

Parameters
- **blk** -- **[in]** Block number of eFuse.
- **num_reg** -- **[in]** The register number in the block.
- **val** -- **[in]** Value to write.

Returns
- ESP_OK: The operation was successfully completed.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.

esp_efuse_coding_scheme_t **esp_efuse_get_coding_scheme**(*esp_efuse_block_t* blk)

Return efuse coding scheme for blocks.

Note: The coding scheme is applicable only to 1, 2 and 3 blocks. For 0 block, the coding scheme is always NONE.

Parameters **blk** -- **[in]** Block number of eFuse.
Returns Return efuse coding scheme for blocks

esp_err_t **esp_efuse_read_block**(*esp_efuse_block_t* blk, void *dst_key, size_t offset_in_bits, size_t size_bits)

Read key to efuse block starting at the offset and the required size.

Note: Please note that reading in the batch mode does not show uncommitted changes.

Parameters
- **blk** -- **[in]** Block number of eFuse.
- **dst_key** -- **[in]** A pointer to array that will contain the result of reading.
- **offset_in_bits** -- **[in]** Start bit in block.
- **size_bits** -- **[in]** The number of bits required to read.

Returns
- ESP_OK: The operation was successfully completed.

- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_efuse_write_block**(*esp_efuse_block_t* blk, const void *src_key, size_t offset_in_bits, size_t size_bits)

Write key to efuse block starting at the offset and the required size.

Parameters
- **blk** -- **[in]** Block number of eFuse.
- **src_key** -- **[in]** A pointer to array that contains the key for writing.
- **offset_in_bits** -- **[in]** Start bit in block.
- **size_bits** -- **[in]** The number of bits required to write.

Returns
- ESP_OK: The operation was successfully completed.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits

uint32_t **esp_efuse_get_pkg_ver**(void)

Returns chip package from efuse.

Returns chip package

void **esp_efuse_reset**(void)

Reset efuse write registers.

Efuse write registers are written to zero, to negate any changes that have been staged here.

Note: This function is not threadsafe, if calling code updates efuse values from multiple tasks then this is caller's responsibility to serialise.

---

esp_err_t **esp_efuse_disable_rom_download_mode**(void)

Disable ROM Download Mode via eFuse.

Permanently disables the ROM Download Mode feature. Once disabled, if the SoC is booted with strapping pins set for ROM Download Mode then an error is printed instead.

Note: Not all SoCs support this option. An error will be returned if called on an ESP32 with a silicon revision lower than 3, as these revisions do not support this option.

Note: If ROM Download Mode is already disabled, this function does nothing and returns success.

Returns
- ESP_OK If the eFuse was successfully burned, or had already been burned.
- ESP_ERR_NOT_SUPPORTED (ESP32 only) This SoC is not capable of disabling UART download mode
- ESP_ERR_INVALID_STATE (ESP32 only) This eFuse is write protected and cannot be written

esp_err_t **esp_efuse_set_rom_log_scheme**(*esp_efuse_rom_log_scheme_t* log_scheme)

Set boot ROM log scheme via eFuse.

Note: By default, the boot ROM will always print to console. This API can be called to set the log scheme only once per chip, once the value is changed from the default it can't be changed again.

Parameters **log_scheme** -- Supported ROM log scheme
Returns
- ESP_OK If the eFuse was successfully burned, or had already been burned.
- ESP_ERR_NOT_SUPPORTED (ESP32 only) This SoC is not capable of setting ROM log scheme
- ESP_ERR_INVALID_STATE This eFuse is write protected or has been burned already

esp_err_t **esp_efuse_enable_rom_secure_download_mode**(void)

Switch ROM Download Mode to Secure Download mode via eFuse.

Permanently enables Secure Download mode. This mode limits the use of ROM Download Mode functions to simple flash read, write and erase operations, plus a command to return a summary of currently enabled security features.

Note: If Secure Download mode is already enabled, this function does nothing and returns success.

Note: Disabling the ROM Download Mode also disables Secure Download Mode.

Returns
- ESP_OK If the eFuse was successfully burned, or had already been burned.
- ESP_ERR_INVALID_STATE ROM Download Mode has been disabled via eFuse, so Secure Download mode is unavailable.

uint32_t **esp_efuse_read_secure_version**(void)

Return secure_version from efuse field.

Returns Secure version from efuse field

bool **esp_efuse_check_secure_version**(uint32_t secure_version)

Check secure_version from app and secure_version and from efuse field.

Parameters **secure_version** -- Secure version from app.
Returns
- True: If version of app is equal or more then secure_version from efuse.

esp_err_t **esp_efuse_update_secure_version**(uint32_t secure_version)

Write efuse field by secure_version value.

Update the secure_version value is available if the coding scheme is None. Note: Do not use this function in your applications. This function is called as part of the other API.

Parameters **secure_version** -- **[in]** Secure version from app.
Returns
- ESP_OK: Successful.
- ESP_FAIL: secure version of app cannot be set to efuse field.
- ESP_ERR_NOT_SUPPORTED: Anti rollback is not supported with the 3/4 and Repeat coding scheme.

esp_err_t **esp_efuse_batch_write_begin**(void)

Set the batch mode of writing fields.

This mode allows you to write the fields in the batch mode when need to burn several efuses at one time. To enable batch mode call begin() then perform as usually the necessary operations read and write and at the end call commit() to actually burn all written efuses. The batch mode can be used nested. The commit will be done by the last commit() function. The number of begin() functions should be equal to the number of commit() functions.

Note: If batch mode is enabled by the first task, at this time the second task cannot write/read efuses. The second task will wait for the first task to complete the batch operation.

```
// Example of using the batch writing mode.

// set the batch writing mode
esp_efuse_batch_write_begin();

// use any writing functions as usual
esp_efuse_write_field_blob(ESP_EFUSE_...);
esp_efuse_write_field_cnt(ESP_EFUSE_...);
esp_efuse_set_write_protect(EFUSE_BLKx);
esp_efuse_write_reg(EFUSE_BLKx, ...);
esp_efuse_write_block(EFUSE_BLKx, ...);
esp_efuse_write(ESP_EFUSE_1, 3);  // ESP_EFUSE_1 == 1, here we write a new␣
↪value = 3. The changes will be burn by the commit() function.
esp_efuse_read_...(ESP_EFUSE_1);  // this function returns ESP_EFUSE_1 == 1␣
↪because uncommitted changes are not readable, it will be available only␣
↪after commit.
...

// esp_efuse_batch_write APIs can be called recursively.
esp_efuse_batch_write_begin();
esp_efuse_set_write_protect(EFUSE_BLKx);
esp_efuse_batch_write_commit(); // the burn will be skipped here, it will be␣
↪done in the last commit().

...

// Write all of these fields to the efuse registers
esp_efuse_batch_write_commit();
esp_efuse_read_...(ESP_EFUSE_1);  // this function returns ESP_EFUSE_1 == 3.
```

Note: Please note that reading in the batch mode does not show uncommitted changes.

Returns

- ESP_OK: Successful.

esp_err_t **esp_efuse_batch_write_cancel** (void)

Reset the batch mode of writing fields.

It will reset the batch writing mode and any written changes.

Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_STATE: Tha batch mode was not set.

esp_err_t **esp_efuse_batch_write_commit** (void)

Writes all prepared data for the batch mode.

Must be called to ensure changes are written to the efuse registers. After this the batch writing mode will be reset.

Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_STATE: The deferred writing mode was not set.

bool **esp_efuse_block_is_empty** (*esp_efuse_block_t* block)

Checks that the given block is empty.

Returns

- True: The block is empty.

- False: The block is not empty or was an error.

bool **esp_efuse_get_key_dis_read**(*esp_efuse_block_t* block)

Returns a read protection for the key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns True: The key block is read protected False: The key block is readable.

esp_err_t **esp_efuse_set_key_dis_read**(*esp_efuse_block_t* block)

Sets a read protection for the key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

bool **esp_efuse_get_key_dis_write**(*esp_efuse_block_t* block)

Returns a write protection for the key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns True: The key block is write protected False: The key block is writeable.

esp_err_t **esp_efuse_set_key_dis_write**(*esp_efuse_block_t* block)

Sets a write protection for the key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

bool **esp_efuse_key_block_unused**(*esp_efuse_block_t* block)

Returns true if the key block is unused, false otherwise.

An unused key block is all zero content, not read or write protected, and has purpose 0 (ESP_EFUSE_KEY_PURPOSE_USER)

Parameters **block** -- key block to check.
Returns

- True if key block is unused,
- False if key block is used or the specified block index is not a key block.

bool **esp_efuse_find_purpose**(*esp_efuse_purpose_t* purpose, *esp_efuse_block_t* *block)

Find a key block with the particular purpose set.

Parameters

- **purpose** -- **[in]** Purpose to search for.
- **block** -- **[out]** Pointer in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX which will be set to the key block if found. Can be NULL, if only need to test the key block exists.

Returns

- True: If found,
- False: If not found (value at block pointer is unchanged).

bool **esp_efuse_get_keypurpose_dis_write**(*esp_efuse_block_t* block)

Returns a write protection of the key purpose field for an efuse key block.

Note: For ESP32: no keypurpose, it returns always True.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns True: The key purpose is write protected. False: The key purpose is writeable.

esp_efuse_purpose_t **esp_efuse_get_key_purpose** (*esp_efuse_block_t* block)

Returns the current purpose set for an efuse key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns
- Value: If Successful, it returns the value of the purpose related to the given key block.
- ESP_EFUSE_KEY_PURPOSE_MAX: Otherwise.

const *esp_efuse_desc_t* ****esp_efuse_get_purpose_field** (*esp_efuse_block_t* block)

Returns a pointer to a key purpose for an efuse key block.

To get the value of this field use esp_efuse_read_field_blob() or esp_efuse_get_key_purpose().

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns Pointer: If Successful returns a pointer to the corresponding efuse field otherwise NULL.

const *esp_efuse_desc_t* ****esp_efuse_get_key** (*esp_efuse_block_t* block)

Returns a pointer to a key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns Pointer: If Successful returns a pointer to the corresponding efuse field otherwise NULL.

esp_err_t **esp_efuse_set_key_purpose** (*esp_efuse_block_t* block, *esp_efuse_purpose_t* purpose)

Sets a key purpose for an efuse key block.

Parameters
- **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
- **purpose** -- **[in]** Key purpose.

Returns
- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_efuse_set_keypurpose_dis_write** (*esp_efuse_block_t* block)

Sets a write protection of the key purpose field for an efuse key block.

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns
- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_efuse_block_t **esp_efuse_find_unused_key_block** (void)

Search for an unused key block and return the first one found.

See esp_efuse_key_block_unused for a description of an unused key block.

Returns First unused key block, or EFUSE_BLK_KEY_MAX if no unused key block is found.

unsigned **esp_efuse_count_unused_key_blocks**(void)

Return the number of unused efuse key blocks in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX.

bool **esp_efuse_get_digest_revoke**(unsigned num_digest)

Returns the status of the Secure Boot public key digest revocation bit.

Parameters **num_digest** -- **[in]** The number of digest in range 0..2
Returns
- True: If key digest is revoked,
- False; If key digest is not revoked.

esp_err_t **esp_efuse_set_digest_revoke**(unsigned num_digest)

Sets the Secure Boot public key digest revocation bit.

Parameters **num_digest** -- **[in]** The number of digest in range 0..2
Returns
- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

bool **esp_efuse_get_write_protect_of_digest_revoke**(unsigned num_digest)

Returns a write protection of the Secure Boot public key digest revocation bit.

Parameters **num_digest** -- **[in]** The number of digest in range 0..2
Returns True: The revocation bit is write protected. False: The revocation bit is writeable.

esp_err_t **esp_efuse_set_write_protect_of_digest_revoke**(unsigned num_digest)

Sets a write protection of the Secure Boot public key digest revocation bit.

Parameters **num_digest** -- **[in]** The number of digest in range 0..2
Returns
- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_efuse_write_key**(*esp_efuse_block_t* block, *esp_efuse_purpose_t* purpose, const void *key, size_t key_size_bytes)

Program a block of key data to an efuse block.

The burn of a key, protection bits, and a purpose happens in batch mode.

Note: This API also enables the read protection efuse bit for certain key blocks like XTS-AES, HMAC, ECDSA etc. This ensures that the key is only accessible to hardware peripheral.

Note: For SoC's with capability `SOC_EFUSE_ECDSA_USE_HARDWARE_K` (e.g., ESP32-H2), this API writes an additional efuse bit for ECDSA key purpose to enforce hardware TRNG generated k mode in the peripheral.

Parameters
- **block** -- **[in]** Block to read purpose for. Must be in range EFUSE_BLK_KEY0 to EFUSE_BLK_KEY_MAX. Key block must be unused (esp_efuse_key_block_unused).
- **purpose** -- **[in]** Purpose to set for this key. Purpose must be already unset.
- **key** -- **[in]** Pointer to data to write.
- **key_size_bytes** -- **[in]** Bytes length of data to write.

Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_INVALID_STATE: Error in efuses state, unused block not found.
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_efuse_write_keys**(const *esp_efuse_purpose_t* purposes[], uint8_t keys[][32], unsigned number_of_keys)

Program keys to unused efuse blocks.

The burn of keys, protection bits, and purposes happens in batch mode.

Note: This API also enables the read protection efuse bit for certain key blocks like XTS-AES, HMAC, ECDSA etc. This ensures that the key is only accessible to hardware peripheral.

Note: For SoC's with capability `SOC_EFUSE_ECDSA_USE_HARDWARE_K` (e.g., ESP32-H2), this API writes an additional efuse bit for ECDSA key purpose to enforce hardware TRNG generated k mode in the peripheral.

Parameters

- **purposes** -- **[in]** Array of purposes (purpose[number_of_keys]).
- **keys** -- **[in]** Array of keys (uint8_t keys[number_of_keys][32]). Each key is 32 bytes long.
- **number_of_keys** -- **[in]** The number of keys to write (up to 6 keys).

Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: Error in the passed arguments.
- ESP_ERR_INVALID_STATE: Error in efuses state, unused block not found.
- ESP_ERR_NOT_ENOUGH_UNUSED_KEY_BLOCKS: Error not enough unused key blocks available
- ESP_ERR_EFUSE_REPEATED_PROG: Error repeated programming of programmed bits is strictly forbidden.
- ESP_ERR_CODING: Error range of data does not match the coding scheme.

esp_err_t **esp_secure_boot_read_key_digests**(*esp_secure_boot_key_digests_t* *trusted_key_digests)

Read key digests from efuse. Any revoked/missing digests will be marked as NULL.

Parameters **trusted_key_digests** -- **[out]** Trusted keys digests, stored in this parameter after successfully completing this function. The number of digests depends on the SOC's capabilities.

Returns

- ESP_OK: Successful.
- ESP_FAIL: If trusted_keys is NULL or there is no valid digest.

esp_err_t **esp_efuse_check_errors**(void)

Checks eFuse errors in BLOCK0.

It does a BLOCK0 check if eFuse EFUSE_ERR_RST_ENABLE is set. If BLOCK0 has an error, it prints the error and returns ESP_FAIL, which should be treated as esp_restart.

Note: Refers to ESP32-C3 only.

Returns
- ESP_OK: No errors in BLOCK0.
- ESP_FAIL: Error in BLOCK0 requiring reboot.

esp_err_t **esp_efuse_destroy_block**(*esp_efuse_block_t* block)

Destroys the data in the given efuse block, if possible.

Data destruction occurs through the following steps: 1) Destroy data in the block:

- If write protection is inactive for the block, then unset bits are burned.
- If write protection is active, the block remains unaltered. 2) Set read protection for the block if possible (check write-protection for RD_DIS). In this case, data becomes inaccessible, and the software reads it as all zeros. If write protection is enabled and read protection can not be set, data in the block remains readable (returns an error).

Do not use the batch mode with this function as it does the burning itself!

Parameters **block** -- **[in]** A key block in the range EFUSE_BLK_KEY0..EFUSE_BLK_KEY_MAX
Returns
- ESP_OK: Successful.
- ESP_FAIL: Data remained readable because the block is write-protected and read protection can not be set.

Structures

struct **esp_efuse_desc_t**

Type definition for an eFuse field.

Public Members

esp_efuse_block_t **efuse_block**

Block of eFuse

uint8_t **bit_start**

Start bit [0..255]

uint16_t **bit_count**

Length of bit field [1..-]

struct **esp_secure_boot_key_digests_t**

Pointers to the trusted key digests.

The number of digests depends on the SOC's capabilities.

Public Members

const void ***key_digests**[3]

Pointers to the key digests

Macros

ESP_ERR_EFUSE

Base error code for efuse api.

ESP_OK_EFUSE_CNT

OK the required number of bits is set.

ESP_ERR_EFUSE_CNT_IS_FULL

Error field is full.

ESP_ERR_EFUSE_REPEATED_PROG

Error repeated programming of programmed bits is strictly forbidden.

ESP_ERR_CODING

Error while a encoding operation.

ESP_ERR_NOT_ENOUGH_UNUSED_KEY_BLOCKS

Error not enough unused key blocks available

ESP_ERR_DAMAGED_READING

Error. Burn or reset was done during a reading operation leads to damage read data. This error is internal to the efuse component and not returned by any public API.

Enumerations

enum **esp_efuse_rom_log_scheme_t**

Type definition for ROM log scheme.

Values:

enumerator **ESP_EFUSE_ROM_LOG_ALWAYS_ON**

Always enable ROM logging

enumerator **ESP_EFUSE_ROM_LOG_ON_GPIO_LOW**

ROM logging is enabled when specific GPIO level is low during start up

enumerator **ESP_EFUSE_ROM_LOG_ON_GPIO_HIGH**

ROM logging is enabled when specific GPIO level is high during start up

enumerator **ESP_EFUSE_ROM_LOG_ALWAYS_OFF**

Disable ROM logging permanently

2.10.8 Error Code and Helper Functions

This section lists definitions of common ESP-IDF error codes and several helper functions related to error handling.

For general information about error codes in ESP-IDF, see *Error Handling*.

For the full list of error codes defined in ESP-IDF, see *Error Codes Reference*.

API Reference

Header File

- components/esp_common/include/esp_check.h

- This header file can be included with:

```
#include "esp_check.h"
```

Macros

ESP_RETURN_ON_ERROR (x, log_tag, format, ...)

Macro which can be used to check the error code. If the code is not ESP_OK, it prints the message and returns. In the future, we want to switch to C++20. We also want to become compatible with clang. Hence, we provide two versions of the following macros. The first one is using the GNU extension ##__VA_ARGS__. The second one is using the C++20 feature **VA_OPT**(,). This allows users to compile their code with standard C++20 enabled instead of the GNU extension. Below C++20, we haven't found any good alternative to using ##__VA_ARGS__. Macro which can be used to check the error code. If the code is not ESP_OK, it prints the message and returns.

ESP_RETURN_ON_ERROR_ISR (x, log_tag, format, ...)

A version of ESP_RETURN_ON_ERROR() macro that can be called from ISR.

ESP_RETURN_VOID_ON_ERROR (x, log_tag, format, ...)

Macro which can be used to check the error code. If the code is not ESP_OK, it prints the message and returns. This macro is used when the function returns void.

ESP_RETURN_VOID_ON_ERROR_ISR (x, log_tag, format, ...)

A version of ESP_RETURN_VOID_ON_ERROR() macro that can be called from ISR.

ESP_GOTO_ON_ERROR (x, goto_tag, log_tag, format, ...)

Macro which can be used to check the error code. If the code is not ESP_OK, it prints the message, sets the local variable 'ret' to the code, and then exits by jumping to 'goto_tag'.

ESP_GOTO_ON_ERROR_ISR (x, goto_tag, log_tag, format, ...)

A version of ESP_GOTO_ON_ERROR() macro that can be called from ISR.

ESP_RETURN_ON_FALSE (a, err_code, log_tag, format, ...)

Macro which can be used to check the condition. If the condition is not 'true', it prints the message and returns with the supplied 'err_code'.

ESP_RETURN_ON_FALSE_ISR (a, err_code, log_tag, format, ...)

A version of ESP_RETURN_ON_FALSE() macro that can be called from ISR.

ESP_RETURN_VOID_ON_FALSE (a, log_tag, format, ...)

Macro which can be used to check the condition. If the condition is not 'true', it prints the message and returns without a value.

ESP_RETURN_VOID_ON_FALSE_ISR (a, log_tag, format, ...)

A version of ESP_RETURN_VOID_ON_FALSE() macro that can be called from ISR.

ESP_GOTO_ON_FALSE (a, err_code, goto_tag, log_tag, format, ...)

Macro which can be used to check the condition. If the condition is not 'true', it prints the message, sets the local variable 'ret' to the supplied 'err_code', and then exits by jumping to 'goto_tag'.

ESP_GOTO_ON_FALSE_ISR (a, err_code, goto_tag, log_tag, format, ...)

A version of ESP_GOTO_ON_FALSE() macro that can be called from ISR.

Header File

- components/esp_common/include/esp_err.h
- This header file can be included with:

```
#include "esp_err.h"
```

Functions

const char ***esp_err_to_name**(*esp_err_t* code)

Returns string for esp_err_t error codes.

This function finds the error code in a pre-generated lookup-table and returns its string representation.

The function is generated by the Python script tools/gen_esp_err_to_name.py which should be run each time an esp_err_t error is modified, created or removed from the IDF project.

Parameters **code** -- esp_err_t error code
Returns string error message

const char ***esp_err_to_name_r**(*esp_err_t* code, char *buf, size_t buflen)

Returns string for esp_err_t and system error codes.

This function finds the error code in a pre-generated lookup-table of esp_err_t errors and returns its string representation. If the error code is not found then it is attempted to be found among system errors.

The function is generated by the Python script tools/gen_esp_err_to_name.py which should be run each time an esp_err_t error is modified, created or removed from the IDF project.

Parameters

- **code** -- esp_err_t error code
- **buf** -- **[out]** buffer where the error message should be written
- **buflen** -- Size of buffer buf. At most buflen bytes are written into the buf buffer (including the terminating null byte).

Returns buf containing the string error message

Macros

ESP_OK

esp_err_t value indicating success (no error)

ESP_FAIL

Generic esp_err_t code indicating failure

ESP_ERR_NO_MEM

Out of memory

ESP_ERR_INVALID_ARG

Invalid argument

ESP_ERR_INVALID_STATE

Invalid state

ESP_ERR_INVALID_SIZE

Invalid size

ESP_ERR_NOT_FOUND

Requested resource not found

ESP_ERR_NOT_SUPPORTED

Operation or feature not supported

ESP_ERR_TIMEOUT

Operation timed out

ESP_ERR_INVALID_RESPONSE

Received response was invalid

ESP_ERR_INVALID_CRC

CRC or checksum was invalid

ESP_ERR_INVALID_VERSION

Version was invalid

ESP_ERR_INVALID_MAC

MAC address was invalid

ESP_ERR_NOT_FINISHED

Operation has not fully completed

ESP_ERR_NOT_ALLOWED

Operation is not allowed

ESP_ERR_WIFI_BASE

Starting number of WiFi error codes

ESP_ERR_MESH_BASE

Starting number of MESH error codes

ESP_ERR_FLASH_BASE

Starting number of flash error codes

ESP_ERR_HW_CRYPTO_BASE

Starting number of HW cryptography module error codes

ESP_ERR_MEMPROT_BASE

Starting number of Memory Protection API error codes

ESP_ERROR_CHECK(x)

Macro which can be used to check the error code, and terminate the program in case the code is not ESP_OK. Prints the error code, error location, and the failed statement to serial output.

Disabled if assertions are disabled.

ESP_ERROR_CHECK_WITHOUT_ABORT(x)

Macro which can be used to check the error code. Prints the error code, error location, and the failed statement to serial output. In comparison with ESP_ERROR_CHECK(), this prints the same error message but isn't terminating the program.

Type Definitions

typedef int **esp_err_t**

2.10.9 ESP HTTPS OTA

Overview

esp_https_ota provides simplified APIs to perform firmware upgrades over HTTPS. It is an abstraction layer over the existing OTA APIs.

Application Example

```
esp_err_t do_firmware_upgrade()
{
    esp_http_client_config_t config = {
        .url = CONFIG_FIRMWARE_UPGRADE_URL,
        .cert_pem = (char *)server_cert_pem_start,
    };
    esp_https_ota_config_t ota_config = {
        .http_config = &config,
    };
    esp_err_t ret = esp_https_ota(&ota_config);
    if (ret == ESP_OK) {
        esp_restart();
    } else {
        return ESP_FAIL;
    }
    return ESP_OK;
}
```

Server Verification

Please refer to *ESP-TLS: TLS Server Verification* for more information on server verification. The root certificate in PEM format needs to be provided to the *esp_http_client_config_t::cert_pem* member.

Note: The server-endpoint **root** certificate should be used for verification instead of any intermediate ones from the certificate chain. The reason is that the root certificate has the maximum validity and usually remains the same for a long period of time. Users can also use the *esp_http_client_config_t::crt_bundle_attach* member for verification by the ESP x509 Certificate Bundle feature, which covers most of the trusted root certificates.

Partial Image Download over HTTPS

To use the partial image download feature, enable partial_http_download configuration in esp_https_ota_config_t. When this configuration is enabled, firmware image will be downloaded in multiple HTTP requests of specified sizes. Maximum content length of each request can be specified by setting max_http_request_size to the required value.

This option is useful while fetching image from a service like AWS S3, where mbedTLS Rx buffer size (*CONFIG_MBEDTLS_SSL_IN_CONTENT_LEN*) can be set to a lower value which is not possible without enabling this configuration.

Default value of mbedTLS Rx buffer size is set to 16 KB. By using partial_http_download with max_http_request_size of 4 KB, size of mbedTLS Rx buffer can be reduced to 4 KB. With this configuration, memory saving of around 12 KB is expected.

Signature Verification

For additional security, signature of OTA firmware images can be verified. For more information, please refer to *Secure OTA Updates Without Secure Boot*.

Advanced APIs

`esp_https_ota` also provides advanced APIs which can be used if more information and control is needed during the OTA process.

Example that uses advanced ESP_HTTPS_OTA APIs: system/ota/advanced_https_ota.

OTA Upgrades with Pre-Encrypted Firmware

To perform OTA upgrades with pre-encrypted firmware, please enable *CONFIG_ESP_HTTPS_OTA_DECRYPT_CB* in component menuconfig.

Example that performs OTA upgrade with pre-encrypted firmware: system/ota/pre_encrypted_ota.

OTA System Events

ESP HTTPS OTA has various events for which a handler can be triggered by the *Event Loop Library* when the particular event occurs. The handler has to be registered using *esp_event_handler_register()*. This helps the event handling for ESP HTTPS OTA.

esp_https_ota_event_t has all the events which can happen when performing OTA upgrade using ESP HTTPS OTA.

Event Handler Example

```
/* Event handler for catching system events */
static void event_handler(void* arg, esp_event_base_t event_base,
                          int32_t event_id, void* event_data)
{
    if (event_base == ESP_HTTPS_OTA_EVENT) {
        switch (event_id) {
            case ESP_HTTPS_OTA_START:
                ESP_LOGI(TAG, "OTA started");
                break;
            case ESP_HTTPS_OTA_CONNECTED:
                ESP_LOGI(TAG, "Connected to server");
                break;
            case ESP_HTTPS_OTA_GET_IMG_DESC:
                ESP_LOGI(TAG, "Reading Image Description");
                break;
            case ESP_HTTPS_OTA_VERIFY_CHIP_ID:
                ESP_LOGI(TAG, "Verifying chip id of new image: %d", *(esp_
↪chip_id_t *)event_data);
                break;
            case ESP_HTTPS_OTA_DECRYPT_CB:
                ESP_LOGI(TAG, "Callback to decrypt function");
                break;
            case ESP_HTTPS_OTA_WRITE_FLASH:
                ESP_LOGD(TAG, "Writing to flash: %d written", *(int␣
↪*)event_data);
                break;
            case ESP_HTTPS_OTA_UPDATE_BOOT_PARTITION:
                ESP_LOGI(TAG, "Boot partition updated. Next Partition: %d
↪", *(esp_partition_subtype_t *)event_data);
                break;
            case ESP_HTTPS_OTA_FINISH:
                ESP_LOGI(TAG, "OTA finish");
                break;
            case ESP_HTTPS_OTA_ABORT:
                ESP_LOGI(TAG, "OTA abort");
```

(continues on next page)

(continued from previous page)

```
                    break;
                }
            }
        }
```

Expected data type for different ESP HTTPS OTA events in the system event loop:

- ESP_HTTPS_OTA_START : `NULL`
- ESP_HTTPS_OTA_CONNECTED : `NULL`
- ESP_HTTPS_OTA_GET_IMG_DESC : `NULL`
- ESP_HTTPS_OTA_VERIFY_CHIP_ID : `esp_chip_id_t`
- ESP_HTTPS_OTA_DECRYPT_CB : `NULL`
- ESP_HTTPS_OTA_WRITE_FLASH : `int`
- ESP_HTTPS_OTA_UPDATE_BOOT_PARTITION : `esp_partition_subtype_t`
- ESP_HTTPS_OTA_FINISH : `NULL`
- ESP_HTTPS_OTA_ABORT : `NULL`

API Reference

Header File

- components/esp_https_ota/include/esp_https_ota.h
- This header file can be included with:

```
#include "esp_https_ota.h"
```

- This header file is a part of the API provided by the `esp_https_ota` component. To declare that your component depends on `esp_https_ota`, add the following to your CMakeLists.txt:

```
REQUIRES esp_https_ota
```

or

```
PRIV_REQUIRES esp_https_ota
```

Functions

esp_err_t **esp_https_ota**(const *esp_https_ota_config_t* *ota_config)

HTTPS OTA Firmware upgrade.

This function allocates HTTPS OTA Firmware upgrade context, establishes HTTPS connection, reads image data from HTTP stream and writes it to OTA partition and finishes HTTPS OTA Firmware upgrade operation. This API supports URL redirection, but if CA cert of URLs differ then it should be appended to `cert_pem` member of `ota_config->http_config`.

Note: This API handles the entire OTA operation, so if this API is being used then no other APIs from `esp_https_ota` component should be called. If more information and control is needed during the HTTPS OTA process, then one can use `esp_https_ota_begin` and subsequent APIs. If this API returns successfully, esp_restart() must be called to boot from the new firmware image.

Parameters **ota_config** -- **[in]** pointer to *esp_https_ota_config_t* structure.

Returns

- ESP_OK: OTA data updated, next reboot will use specified partition.
- ESP_FAIL: For generic failure.
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_OTA_VALIDATE_FAILED: Invalid app image
- ESP_ERR_NO_MEM: Cannot allocate memory for OTA operation.

- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash write failed.
- For other return codes, refer OTA documentation in esp-idf's app_update component.

esp_err_t **esp_https_ota_begin**(const *esp_https_ota_config_t* *ota_config, *esp_https_ota_handle_t* *handle)

Start HTTPS OTA Firmware upgrade.

This function initializes ESP HTTPS OTA context and establishes HTTPS connection. This function must be invoked first. If this function returns successfully, then `esp_https_ota_perform` should be called to continue with the OTA process and there should be a call to `esp_https_ota_finish` on completion of OTA operation or on failure in subsequent operations. This API supports URL redirection, but if CA cert of URLs differ then it should be appended to `cert_pem` member of `http_config`, which is a part of `ota_config`. In case of error, this API explicitly sets `handle` to NULL.

Note: This API is blocking, so setting `is_async` member of `http_config` structure will result in an error.

Parameters

- **ota_config** -- **[in]** pointer to *esp_https_ota_config_t* structure
- **handle** -- **[out]** pointer to an allocated data of type `esp_https_ota_handle_t` which will be initialised in this function

Returns

- ESP_OK: HTTPS OTA Firmware upgrade context initialised and HTTPS connection established
- ESP_FAIL: For generic failure.
- ESP_ERR_INVALID_ARG: Invalid argument (missing/incorrect config, certificate, etc.)
- For other return codes, refer documentation in app_update component and esp_http_client component in esp-idf.

esp_err_t **esp_https_ota_perform**(*esp_https_ota_handle_t* https_ota_handle)

Read image data from HTTP stream and write it to OTA partition.

This function reads image data from HTTP stream and writes it to OTA partition. This function must be called only if esp_https_ota_begin() returns successfully. This function must be called in a loop since it returns after every HTTP read operation thus giving you the flexibility to stop OTA operation midway.

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure

Returns

- ESP_ERR_HTTPS_OTA_IN_PROGRESS: OTA update is in progress, call this API again to continue.
- ESP_OK: OTA update was successful
- ESP_FAIL: OTA update failed
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_INVALID_VERSION: Invalid chip revision in image header
- ESP_ERR_OTA_VALIDATE_FAILED: Invalid app image
- ESP_ERR_NO_MEM: Cannot allocate memory for OTA operation.
- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash write failed.
- For other return codes, refer OTA documentation in esp-idf's app_update component.

bool **esp_https_ota_is_complete_data_received**(*esp_https_ota_handle_t* https_ota_handle)

Checks if complete data was received or not.

Note: This API can be called just before esp_https_ota_finish() to validate if the complete image was indeed received.

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure

Returns
- false
- true

esp_err_t **esp_https_ota_finish**(*esp_https_ota_handle_t* https_ota_handle)

Clean-up HTTPS OTA Firmware upgrade and close HTTPS connection.

This function closes the HTTP connection and frees the ESP HTTPS OTA context. This function switches the boot partition to the OTA partition containing the new firmware image.

Note: If this API returns successfully, esp_restart() must be called to boot from the new firmware image esp_https_ota_finish should not be called after calling esp_https_ota_abort

---

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure
Returns
- ESP_OK: Clean-up successful
- ESP_ERR_INVALID_STATE
- ESP_ERR_INVALID_ARG: Invalid argument
- ESP_ERR_OTA_VALIDATE_FAILED: Invalid app image

esp_err_t **esp_https_ota_abort**(*esp_https_ota_handle_t* https_ota_handle)

Clean-up HTTPS OTA Firmware upgrade and close HTTPS connection.

This function closes the HTTP connection and frees the ESP HTTPS OTA context.

Note: esp_https_ota_abort should not be called after calling esp_https_ota_finish

---

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure
Returns
- ESP_OK: Clean-up successful
- ESP_ERR_INVALID_STATE: Invalid ESP HTTPS OTA state
- ESP_FAIL: OTA not started
- ESP_ERR_NOT_FOUND: OTA handle not found
- ESP_ERR_INVALID_ARG: Invalid argument

esp_err_t **esp_https_ota_get_img_desc**(*esp_https_ota_handle_t* https_ota_handle, *esp_app_desc_t* *new_app_info)

Reads app description from image header. The app description provides information like the "Firmware version" of the image.

Note: This API can be called only after esp_https_ota_begin() and before esp_https_ota_perform(). Calling this API is not mandatory.

Parameters
- **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure
- **new_app_info** -- **[out]** pointer to an allocated *esp_app_desc_t* structure

Returns
- ESP_ERR_INVALID_ARG: Invalid arguments
- ESP_ERR_INVALID_STATE: Invalid state to call this API. esp_https_ota_begin() not called yet.
- ESP_FAIL: Failed to read image descriptor
- ESP_OK: Successfully read image descriptor

int **esp_https_ota_get_image_len_read**(*esp_https_ota_handle_t* https_ota_handle)

This function returns OTA image data read so far.

Note: This API should be called only if `esp_https_ota_perform()` has been called at least once or if `esp_https_ota_get_img_desc` has been called before.

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure

Returns

- -1 On failure
- total bytes read so far

int **esp_https_ota_get_image_size**(*esp_https_ota_handle_t* https_ota_handle)

This function returns OTA image total size.

Note: This API should be called after esp_https_ota_begin() has been already called. This can be used to create some sort of progress indication (in combination with esp_https_ota_get_image_len_read())

Parameters **https_ota_handle** -- **[in]** pointer to esp_https_ota_handle_t structure

Returns

- -1 On failure or chunked encoding
- total bytes of image

Structures

struct **esp_https_ota_config_t**

ESP HTTPS OTA configuration.

Public Members

const *esp_http_client_config_t* ***http_config**

ESP HTTP client configuration

http_client_init_cb_t **http_client_init_cb**

Callback after ESP HTTP client is initialised

bool **bulk_flash_erase**

Erase entire flash partition during initialization. By default flash partition is erased during write operation and in chunk of 4K sector size

bool **partial_http_download**

Enable Firmware image to be downloaded over multiple HTTP requests

int **max_http_request_size**

Maximum request size for partial HTTP download

uint32_t **buffer_caps**

The memory capability to use when allocating the buffer for OTA update. Default capability is MALLOC_CAP_DEFAULT

Macros

ESP_ERR_HTTPS_OTA_BASE

ESP_ERR_HTTPS_OTA_IN_PROGRESS

Type Definitions

typedef void ***esp_https_ota_handle_t**

typedef *esp_err_t* (***http_client_init_cb_t**)(*esp_http_client_handle_t*)

Enumerations

enum **esp_https_ota_event_t**

Events generated by OTA process.

Values:

enumerator **ESP_HTTPS_OTA_START**

OTA started

enumerator **ESP_HTTPS_OTA_CONNECTED**

Connected to server

enumerator **ESP_HTTPS_OTA_GET_IMG_DESC**

Read app description from image header

enumerator **ESP_HTTPS_OTA_VERIFY_CHIP_ID**

Verify chip id of new image

enumerator **ESP_HTTPS_OTA_DECRYPT_CB**

Callback to decrypt function

enumerator **ESP_HTTPS_OTA_WRITE_FLASH**

Flash write operation

enumerator **ESP_HTTPS_OTA_UPDATE_BOOT_PARTITION**

Boot partition update after successful ota update

enumerator **ESP_HTTPS_OTA_FINISH**

OTA finished

enumerator **ESP_HTTPS_OTA_ABORT**

OTA aborted

2.10.10 Event Loop Library

Overview

The event loop library allows components to declare events so that other components can register handlers -- codes that executes when those events occur. This allows loosely-coupled components to attach desired behavior to state changes of other components without application involvement. This also simplifies event processing by serializing and deferring code execution to another context.

One common case is, if a high-level library is using the Wi-Fi library: it may subscribe to *ESP32 Wi-Fi Programming Model* directly and act on those events.

Note: Various modules of the Bluetooth stack deliver events to applications via dedicated callback functions instead of via the Event Loop Library.

Using `esp_event` APIs

There are two objects of concern for users of this library: events and event loops.

An event indicates an important occurrence, such as a successful Wi-Fi connection to an access point. A two-part identifier should be used when referencing events, see *declaring and defining events* for details. The event loop is the bridge between events and event handlers. The event source publishes events to the event loop using the APIs provided by the event loop library, and event handlers registered to the event loop respond to specific types of events.

Using this library roughly entails the following flow:

1. The user defines a function that should run when an event is posted to a loop. This function is referred to as the event handler, and should have the same signature as `esp_event_handler_t`.
2. An event loop is created using `esp_event_loop_create()`, which outputs a handle to the loop of type `esp_event_loop_handle_t`. Event loops created using this API are referred to as user event loops. There is, however, a special type of event loop called the default event loop which is discussed in *default event loop*.
3. Components register event handlers to the loop using `esp_event_handler_register_with()`. Handlers can be registered with multiple loops, see *notes on handler registration*.
4. Event sources post an event to the loop using `esp_event_post_to()`.
5. Components wanting to remove their handlers from being called can do so by unregistering from the loop using `esp_event_handler_unregister_with()`.
6. Event loops that are no longer needed can be deleted using `esp_event_loop_delete()`.

In code, the flow above may look like as follows:

```
// 1. Define the event handler
void run_on_event(void* handler_arg, esp_event_base_t base, int32_t id, void*␣
↪event_data)
{
    // Event handler logic
}

void app_main()
{
    // 2. A configuration structure of type esp_event_loop_args_t is needed to␣
↪specify the properties of the loop to be created. A handle of type esp_event_␣
↪loop_handle_t is obtained, which is needed by the other APIs to reference the␣
↪loop to perform their operations.
    esp_event_loop_args_t loop_args = {
        .queue_size = ...,
        .task_name = ...
        .task_priority = ...,
        .task_stack_size = ...,
        .task_core_id = ...
    };
```

(continues on next page)

(continued from previous page)

```
    esp_event_loop_handle_t loop_handle;

    esp_event_loop_create(&loop_args, &loop_handle);

    // 3. Register event handler defined in (1). MY_EVENT_BASE and MY_EVENT_ID␣
↪specify a hypothetical event that handler run_on_event should execute when it␣
↪gets posted to the loop.
    esp_event_handler_register_with(loop_handle, MY_EVENT_BASE, MY_EVENT_ID, run_
↪on_event, ...);

    ...

    // 4. Post events to the loop. This queues the event on the event loop. At␣
↪some point, the event loop executes the event handler registered to the posted␣
↪event, in this case, run_on_event. To simplify the process, this example calls␣
↪esp_event_post_to from app_main, but posting can be done from any other task␣
↪(which is the more interesting use case).
    esp_event_post_to(loop_handle, MY_EVENT_BASE, MY_EVENT_ID, ...);

    ...

    // 5. Unregistering an unneeded handler
    esp_event_handler_unregister_with(loop_handle, MY_EVENT_BASE, MY_EVENT_ID, run_
↪on_event);

    ...

    // 6. Deleting an unneeded event loop
    esp_event_loop_delete(loop_handle);
}
```

Declaring and Defining Events

As mentioned previously, events consist of two-part identifiers: the event base and the event ID. The event base identifies an independent group of events; the event ID identifies the event within that group. Think of the event base and event ID as a person's last name and first name, respectively. A last name identifies a family, and the first name identifies a person within that family.

The event loop library provides macros to declare and define the event base easily.

Event base declaration:

```
ESP_EVENT_DECLARE_BASE(EVENT_BASE);
```

Event base definition:

```
ESP_EVENT_DEFINE_BASE(EVENT_BASE);
```

Note: In ESP-IDF, the base identifiers for system events are uppercase and are postfixed with `_EVENT`. For example, the base for Wi-Fi events is declared and defined as `WIFI_EVENT`, the Ethernet event base `ETHERNET_EVENT`, and so on. The purpose is to have event bases look like constants (although they are global variables considering the definitions of macros `ESP_EVENT_DECLARE_BASE` and `ESP_EVENT_DEFINE_BASE`).

For event IDs, declaring them as enumerations is recommended. Once again, for visibility, these are typically placed in public header files.

Event ID:

```
enum {
    EVENT_ID_1,
    EVENT_ID_2,
    EVENT_ID_3,
    ...
}
```

Default Event Loop

The default event loop is a special type of loop used for system events (Wi-Fi events, for example). The handle for this loop is hidden from the user, and the creation, deletion, handler registration/deregistration, and posting of events are done through a variant of the APIs for user event loops. The table below enumerates those variants, and the user event loops equivalent.

User Event Loops	Default Event Loops
esp_event_loop_create()	*esp_event_loop_create_default()*
esp_event_loop_delete()	*esp_event_loop_delete_default()*
esp_event_handler_register_with()	*esp_event_handler_register()*
esp_event_handler_unregister_with()	*esp_event_handler_unregister()*
esp_event_post_to()	*esp_event_post()*

If you compare the signatures for both, they are mostly similar except for the lack of loop handle specification for the default event loop APIs.

Other than the API difference and the special designation to which system events are posted, there is no difference in how default event loops and user event loops behave. It is even possible for users to post their own events to the default event loop, should the user opt to not create their own loops to save memory.

Notes on Handler Registration

It is possible to register a single handler to multiple events individually by using multiple calls to *esp_event_handler_register_with()*. For those multiple calls, the specific event base and event ID can be specified with which the handler should execute.

However, in some cases, it is desirable for a handler to execute on the following situations:

(1) all events that get posted to a loop
(2) all events of a particular base identifier

This is possible using the special event base identifier `ESP_EVENT_ANY_BASE` and special event ID `ESP_EVENT_ANY_ID`. These special identifiers may be passed as the event base and event ID arguments for *esp_event_handler_register_with()*.

Therefore, the valid arguments to *esp_event_handler_register_with()* are:

1. <event base>, <event ID> - handler executes when the event with base <event base> and event ID <event ID> gets posted to the loop
2. <event base>, ESP_EVENT_ANY_ID - handler executes when any event with base <event base> gets posted to the loop
3. ESP_EVENT_ANY_BASE, ESP_EVENT_ANY_ID - handler executes when any event gets posted to the loop

As an example, suppose the following handler registrations were performed:

```
esp_event_handler_register_with(loop_handle, MY_EVENT_BASE, MY_EVENT_ID, run_on_
↪event_1, ...);
esp_event_handler_register_with(loop_handle, MY_EVENT_BASE, ESP_EVENT_ANY_ID, run_
↪on_event_2, ...);
esp_event_handler_register_with(loop_handle, ESP_EVENT_ANY_BASE, ESP_EVENT_ANY_ID,␣
↪run_on_event_3, ...);
```

(continues on next page)

(continued from previous page)

If the hypothetical event `MY_EVENT_BASE`, `MY_EVENT_ID` is posted, all three handlers `run_on_event_1`, `run_on_event_2`, and `run_on_event_3` would execute.

If the hypothetical event `MY_EVENT_BASE`, `MY_OTHER_EVENT_ID` is posted, only `run_on_event_2` and `run_on_event_3` would execute.

If the hypothetical event `MY_OTHER_EVENT_BASE`, `MY_OTHER_EVENT_ID` is posted, only `run_on_event_3` would execute.

Handler Un-Registering Itself In general, an event handler run by an event loop is **not allowed to do any registering/unregistering activity on that event loop**. There is one exception, though: un-registering itself is allowed for the handler. E.g., it is possible to do the following:

```
void run_on_event(void* handler_arg, esp_event_base_t base, int32_t id, void*␣
↪event_data)
{
    esp_event_loop_handle_t *loop_handle = (esp_event_loop_handle_t*) handler_arg;
    esp_event_handler_unregister_with(*loop_handle, MY_EVENT_BASE, MY_EVENT_ID,␣
↪run_on_event);
}

void app_main(void)
{
    esp_event_loop_handle_t loop_handle;
    esp_event_loop_create(&loop_args, &loop_handle);
    esp_event_handler_register_with(loop_handle, MY_EVENT_BASE, MY_EVENT_ID, run_
↪on_event, &loop_handle);
    // ... post-event MY_EVENT_BASE, MY_EVENT_ID and run loop at some point
}
```

Handler Registration and Handler Dispatch Order The general rule is that, for handlers that match a certain posted event during dispatch, those which are registered first also get executed first. The user can then control which handlers get executed first by registering them before other handlers, provided that all registrations are performed using a single task. If the user plans to take advantage of this behavior, caution must be exercised if there are multiple tasks registering handlers. While the 'first registered, first executed' behavior still holds true, the task which gets executed first also gets its handlers registered first. Handlers registered one after the other by a single task are still dispatched in the order relative to each other, but if that task gets pre-empted in between registration by another task that also registers handlers; then during dispatch those handlers also get executed in between.

Event Loop Profiling

A configuration option *CONFIG_ESP_EVENT_LOOP_PROFILING* can be enabled in order to activate statistics collection for all event loops created. The function *esp_event_dump()* can be used to output the collected statistics to a file stream. More details on the information included in the dump can be found in the *esp_event_dump()* API Reference.

Application Example

Examples of using the `esp_event` library can be found in system/esp_event. The examples cover event declaration, loop creation, handler registration and deregistration, and event posting.

Other examples which also adopt `esp_event` library:

- NMEA Parser , which decodes the statements received from GPS.

API Reference

Header File

- components/esp_event/include/esp_event.h
- This header file can be included with:

```
#include "esp_event.h"
```

- This header file is a part of the API provided by the `esp_event` component. To declare that your component depends on `esp_event`, add the following to your CMakeLists.txt:

```
REQUIRES esp_event
```

or

```
PRIV_REQUIRES esp_event
```

Functions

esp_err_t **esp_event_loop_create**(const *esp_event_loop_args_t* *event_loop_args, *esp_event_loop_handle_t* *event_loop)

Create a new event loop.

Parameters
- **event_loop_args** -- **[in]** configuration structure for the event loop to create
- **event_loop** -- **[out]** handle to the created event loop

Returns
- ESP_OK: Success
- ESP_ERR_INVALID_ARG: event_loop_args or event_loop was NULL
- ESP_ERR_NO_MEM: Cannot allocate memory for event loops list
- ESP_FAIL: Failed to create task loop
- Others: Fail

esp_err_t **esp_event_loop_delete**(*esp_event_loop_handle_t* event_loop)

Delete an existing event loop.

Parameters **event_loop** -- **[in]** event loop to delete, must not be NULL

Returns
- ESP_OK: Success
- Others: Fail

esp_err_t **esp_event_loop_create_default**(void)

Create default event loop.

Returns
- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for event loops list
- ESP_ERR_INVALID_STATE: Default event loop has already been created
- ESP_FAIL: Failed to create task loop
- Others: Fail

esp_err_t **esp_event_loop_delete_default**(void)

Delete the default event loop.

Returns
- ESP_OK: Success
- Others: Fail

esp_err_t **esp_event_loop_run**(*esp_event_loop_handle_t* event_loop, TickType_t ticks_to_run)

Dispatch events posted to an event loop.

This function is used to dispatch events posted to a loop with no dedicated task, i.e. task name was set to NULL in event_loop_args argument during loop creation. This function includes an argument to limit the

amount of time it runs, returning control to the caller when that time expires (or some time afterwards). There is no guarantee that a call to this function will exit at exactly the time of expiry. There is also no guarantee that events have been dispatched during the call, as the function might have spent all the allotted time waiting on the event queue. Once an event has been dequeued, however, it is guaranteed to be dispatched. This guarantee contributes to not being able to exit exactly at time of expiry as (1) blocking on internal mutexes is necessary for dispatching the dequeued event, and (2) during dispatch of the dequeued event there is no way to control the time occupied by handler code execution. The guaranteed time of exit is therefore the allotted time + amount of time required to dispatch the last dequeued event.

In cases where waiting on the queue times out, ESP_OK is returned and not ESP_ERR_TIMEOUT, since it is normal behavior.

Note: encountering an unknown event that has been posted to the loop will only generate a warning, not an error.

Parameters

- **`event_loop`** -- **[in]** event loop to dispatch posted events from, must not be NULL
- **`ticks_to_run`** -- **[in]** number of ticks to run the loop

Returns

- ESP_OK: Success
- Others: Fail

esp_err_t **`esp_event_handler_register`**(esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler, void *event_handler_arg)

Register an event handler to the system event loop (legacy).

This function can be used to register a handler for either: (1) specific events, (2) all events of a certain event base, or (3) all events known by the system event loop.

- specific events: specify exact event_base and event_id
- all events of a certain base: specify exact event_base and use ESP_EVENT_ANY_ID as the event_id
- all events known by the loop: use ESP_EVENT_ANY_BASE for event_base and ESP_EVENT_ANY_ID as the event_id

Registering multiple handlers to events is possible. Registering a single handler to multiple events is also possible. However, registering the same handler to the same event multiple times would cause the previous registrations to be overwritten.

Note: the event loop library does not maintain a copy of event_handler_arg, therefore the user should ensure that event_handler_arg still points to a valid location by the time the handler gets called

---

Parameters

- **`event_base`** -- **[in]** the base ID of the event to register the handler for
- **`event_id`** -- **[in]** the ID of the event to register the handler for
- **`event_handler`** -- **[in]** the handler function which gets called when the event is dispatched
- **`event_handler_arg`** -- **[in]** data, aside from event data, that is passed to the handler when it is called

Returns

- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for the handler
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_handler_register_with**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler, void *event_handler_arg)

Register an event handler to a specific loop (legacy).

This function behaves in the same manner as esp_event_handler_register, except the additional specification of the event loop to register the handler to.

Note: the event loop library does not maintain a copy of event_handler_arg, therefore the user should ensure that event_handler_arg still points to a valid location by the time the handler gets called

---

Parameters

- **event_loop** -- **[in]** the event loop to register this handler function to, must not be NULL
- **event_base** -- **[in]** the base ID of the event to register the handler for
- **event_id** -- **[in]** the ID of the event to register the handler for
- **event_handler** -- **[in]** the handler function which gets called when the event is dispatched
- **event_handler_arg** -- **[in]** data, aside from event data, that is passed to the handler when it is called

Returns

- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for the handler
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_handler_instance_register_with**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler, void *event_handler_arg, *esp_event_handler_instance_t* *instance)

Register an instance of event handler to a specific loop.

This function can be used to register a handler for either: (1) specific events, (2) all events of a certain event base, or (3) all events known by the system event loop.

- specific events: specify exact event_base and event_id
- all events of a certain base: specify exact event_base and use ESP_EVENT_ANY_ID as the event_id
- all events known by the loop: use ESP_EVENT_ANY_BASE for event_base and ESP_EVENT_ANY_ID as the event_id

Besides the error, the function returns an instance object as output parameter to identify each registration. This is necessary to remove (unregister) the registration before the event loop is deleted.

Registering multiple handlers to events, registering a single handler to multiple events as well as registering the same handler to the same event multiple times is possible. Each registration yields a distinct instance object which identifies it over the registration lifetime.

Note: the event loop library does not maintain a copy of event_handler_arg, therefore the user should ensure that event_handler_arg still points to a valid location by the time the handler gets called

---

Note: Calling this function with instance set to NULL is equivalent to calling esp_event_handler_register_with.

Parameters

- **event_loop** -- **[in]** the event loop to register this handler function to, must not be NULL
- **event_base** -- **[in]** the base ID of the event to register the handler for
- **event_id** -- **[in]** the ID of the event to register the handler for
- **event_handler** -- **[in]** the handler function which gets called when the event is dispatched
- **event_handler_arg** -- **[in]** data, aside from event data, that is passed to the handler when it is called
- **instance** -- **[out]** An event handler instance object related to the registered event handler and data, can be NULL. This needs to be kept if the specific callback instance should be unregistered before deleting the whole event loop. Registering the same event handler multiple times is possible and yields distinct instance objects. The data can be the same for all registrations. If no unregistration is needed, but the handler should be deleted when the event loop is deleted, instance can be NULL.

Returns

- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for the handler
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID or instance is NULL
- Others: Fail

esp_err_t **esp_event_handler_instance_register**(esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler, void *event_handler_arg, *esp_event_handler_instance_t* *instance)

Register an instance of event handler to the default loop.

This function does the same as esp_event_handler_instance_register_with, except that it registers the handler to the default event loop.

Note: the event loop library does not maintain a copy of event_handler_arg, therefore the user should ensure that event_handler_arg still points to a valid location by the time the handler gets called

---

Note: Calling this function with instance set to NULL is equivalent to calling esp_event_handler_register.

Parameters

- **event_base** -- **[in]** the base ID of the event to register the handler for
- **event_id** -- **[in]** the ID of the event to register the handler for
- **event_handler** -- **[in]** the handler function which gets called when the event is dispatched
- **event_handler_arg** -- **[in]** data, aside from event data, that is passed to the handler when it is called
- **instance** -- **[out]** An event handler instance object related to the registered event handler and data, can be NULL. This needs to be kept if the specific callback instance should be unregistered before deleting the whole event loop. Registering the same event handler multiple times is possible and yields distinct instance objects. The data can be the same for all registrations. If no unregistration is needed, but the handler should be deleted when the event loop is deleted, instance can be NULL.

Returns

- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for the handler
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID or instance is NULL
- Others: Fail

esp_err_t **esp_event_handler_unregister**(esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler)

Unregister a handler with the system event loop (legacy).

Unregisters a handler, so it will no longer be called during dispatch. Handlers can be unregistered for any combination of event_base and event_id which were previously registered. To unregister a handler, the event_base and event_id arguments must match exactly the arguments passed to esp_event_handler_register() when that handler was registered. Passing ESP_EVENT_ANY_BASE and/or ESP_EVENT_ANY_ID will only unregister handlers that were registered with the same wildcard arguments.

Note: When using ESP_EVENT_ANY_ID, handlers registered to specific event IDs using the same base will not be unregistered. When using ESP_EVENT_ANY_BASE, events registered to specific bases will also not be unregistered. This avoids accidental unregistration of handlers registered by other users or components.

Parameters

- **`event_base`** -- **[in]** the base of the event with which to unregister the handler
- **`event_id`** -- **[in]** the ID of the event with which to unregister the handler
- **`event_handler`** -- **[in]** the handler to unregister

Returns ESP_OK success

Returns ESP_ERR_INVALID_ARG invalid combination of event base and event ID

Returns others fail

esp_err_t **`esp_event_handler_unregister_with`**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, *esp_event_handler_t* event_handler)

Unregister a handler from a specific event loop (legacy).

This function behaves in the same manner as esp_event_handler_unregister, except the additional specification of the event loop to unregister the handler with.

Parameters

- **`event_loop`** -- **[in]** the event loop with which to unregister this handler function, must not be NULL
- **`event_base`** -- **[in]** the base of the event with which to unregister the handler
- **`event_id`** -- **[in]** the ID of the event with which to unregister the handler
- **`event_handler`** -- **[in]** the handler to unregister

Returns

- ESP_OK: Success
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **`esp_event_handler_instance_unregister_with`**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, *esp_event_handler_instance_t* instance)

Unregister a handler instance from a specific event loop.

Unregisters a handler instance, so it will no longer be called during dispatch. Handler instances can be unregistered for any combination of event_base and event_id which were previously registered. To unregister a handler instance, the event_base and event_id arguments must match exactly the arguments passed to esp_event_handler_instance_register() when that handler instance was registered. Passing ESP_EVENT_ANY_BASE and/or ESP_EVENT_ANY_ID will only unregister handler instances that were registered with the same wildcard arguments.

Note: When using ESP_EVENT_ANY_ID, handlers registered to specific event IDs using the same base will not be unregistered. When using ESP_EVENT_ANY_BASE, events registered to specific bases will also not be unregistered. This avoids accidental unregistration of handlers registered by other users or components.

Parameters

- **event_loop** -- **[in]** the event loop with which to unregister this handler function, must not be NULL
- **event_base** -- **[in]** the base of the event with which to unregister the handler
- **event_id** -- **[in]** the ID of the event with which to unregister the handler
- **instance** -- **[in]** the instance object of the registration to be unregistered

Returns

- ESP_OK: Success
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_handler_instance_unregister**(esp_event_base_t event_base, int32_t event_id, *esp_event_handler_instance_t* instance)

Unregister a handler from the system event loop.

This function does the same as esp_event_handler_instance_unregister_with, except that it unregisters the handler instance from the default event loop.

Parameters

- **event_base** -- **[in]** the base of the event with which to unregister the handler
- **event_id** -- **[in]** the ID of the event with which to unregister the handler
- **instance** -- **[in]** the instance object of the registration to be unregistered

Returns

- ESP_OK: Success
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_post**(esp_event_base_t event_base, int32_t event_id, const void *event_data, size_t event_data_size, TickType_t ticks_to_wait)

Posts an event to the system default event loop. The event loop library keeps a copy of event_data and manages the copy's lifetime automatically (allocation + deletion); this ensures that the data the handler receives is always valid.

Parameters

- **event_base** -- **[in]** the event base that identifies the event
- **event_id** -- **[in]** the event ID that identifies the event
- **event_data** -- **[in]** the data, specific to the event occurrence, that gets passed to the handler
- **event_data_size** -- **[in]** the size of the event data
- **ticks_to_wait** -- **[in]** number of ticks to block on a full event queue

Returns

- ESP_OK: Success
- ESP_ERR_TIMEOUT: Time to wait for event queue to unblock expired, queue full when posting from ISR
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_post_to**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, const void *event_data, size_t event_data_size, TickType_t ticks_to_wait)

Posts an event to the specified event loop. The event loop library keeps a copy of event_data and manages the copy's lifetime automatically (allocation + deletion); this ensures that the data the handler receives is always valid.

This function behaves in the same manner as esp_event_post, except the additional specification of the event loop to post the event to.

Parameters

- **event_loop** -- **[in]** the event loop to post to, must not be NULL
- **event_base** -- **[in]** the event base that identifies the event
- **event_id** -- **[in]** the event ID that identifies the event

- **event_data** -- **[in]** the data, specific to the event occurrence, that gets passed to the handler
- **event_data_size** -- **[in]** the size of the event data
- **ticks_to_wait** -- **[in]** number of ticks to block on a full event queue

Returns

- ESP_OK: Success
- ESP_ERR_TIMEOUT: Time to wait for event queue to unblock expired, queue full when posting from ISR
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID
- Others: Fail

esp_err_t **esp_event_isr_post**(esp_event_base_t event_base, int32_t event_id, const void *event_data, size_t event_data_size, BaseType_t *task_unblocked)

Special variant of esp_event_post for posting events from interrupt handlers.

Note: this function is only available when CONFIG_ESP_EVENT_POST_FROM_ISR is enabled

---

Note: when this function is called from an interrupt handler placed in IRAM, this function should be placed in IRAM as well by enabling CONFIG_ESP_EVENT_POST_FROM_IRAM_ISR

Parameters

- **event_base** -- **[in]** the event base that identifies the event
- **event_id** -- **[in]** the event ID that identifies the event
- **event_data** -- **[in]** the data, specific to the event occurrence, that gets passed to the handler
- **event_data_size** -- **[in]** the size of the event data; max is 4 bytes
- **task_unblocked** -- **[out]** an optional parameter (can be NULL) which indicates that an event task with higher priority than currently running task has been unblocked by the posted event; a context switch should be requested before the interrupt is existed.

Returns

- ESP_OK: Success
- ESP_FAIL: Event queue for the default event loop full
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID, data size of more than 4 bytes
- Others: Fail

esp_err_t **esp_event_isr_post_to**(*esp_event_loop_handle_t* event_loop, esp_event_base_t event_base, int32_t event_id, const void *event_data, size_t event_data_size, BaseType_t *task_unblocked)

Special variant of esp_event_post_to for posting events from interrupt handlers.

Note: this function is only available when CONFIG_ESP_EVENT_POST_FROM_ISR is enabled

---

Note: when this function is called from an interrupt handler placed in IRAM, this function should be placed in IRAM as well by enabling CONFIG_ESP_EVENT_POST_FROM_IRAM_ISR

Parameters

- **event_loop** -- **[in]** the event loop to post to, must not be NULL
- **event_base** -- **[in]** the event base that identifies the event
- **event_id** -- **[in]** the event ID that identifies the event
- **event_data** -- **[in]** the data, specific to the event occurrence, that gets passed to the handler

- **event_data_size** -- **[in]** the size of the event data
- **task_unblocked** -- **[out]** an optional parameter (can be NULL) which indicates that an event task with higher priority than currently running task has been unblocked by the posted event; a context switch should be requested before the interrupt is existed.

Returns

- ESP_OK: Success
- ESP_FAIL: Event queue for the loop full
- ESP_ERR_INVALID_ARG: Invalid combination of event base and event ID, data size of more than 4 bytes
- Others: Fail

esp_err_t **esp_event_dump**(FILE *file)

Dumps statistics of all event loops.

Dumps event loop info in the format:

```
      event loop
          handler
          handler
          ...
      event loop
          handler
          handler
          ...

 where:

  event loop
      format: address,name rx:total_received dr:total_dropped
      where:
          address - memory address of the event loop
          name - name of the event loop, 'none' if no dedicated task
          total_received - number of successfully posted events
          total_dropped - number of events unsuccessfully posted due to queue␣
↪being full

  handler
      format: address ev:base,id inv:total_invoked run:total_runtime
      where:
          address - address of the handler function
          base,id - the event specified by event base and ID this handler␣
↪executes
          total_invoked - number of times this handler has been invoked
          total_runtime - total amount of time used for invoking this handler
```

Note: this function is a noop when CONFIG_ESP_EVENT_LOOP_PROFILING is disabled

Parameters **file** -- **[in]** the file stream to output to

Returns

- ESP_OK: Success
- ESP_ERR_NO_MEM: Cannot allocate memory for event loops list
- Others: Fail

Structures

struct **esp_event_loop_args_t**

Configuration for creating event loops.

Public Members

int32_t **queue_size**

size of the event loop queue

const char ***task_name**

name of the event loop task; if NULL, a dedicated task is not created for event loop

UBaseType_t **task_priority**

priority of the event loop task, ignored if task name is NULL

uint32_t **task_stack_size**

stack size of the event loop task, ignored if task name is NULL

BaseType_t **task_core_id**

core to which the event loop task is pinned to, ignored if task name is NULL

Header File

- components/esp_event/include/esp_event_base.h
- This header file can be included with:

```
#include "esp_event_base.h"
```

- This header file is a part of the API provided by the `esp_event` component. To declare that your component depends on `esp_event`, add the following to your CMakeLists.txt:

```
REQUIRES esp_event
```

or

```
PRIV_REQUIRES esp_event
```

Macros

ESP_EVENT_DECLARE_BASE(id)

ESP_EVENT_DEFINE_BASE(id)

ESP_EVENT_ANY_BASE

register handler for any event base

ESP_EVENT_ANY_ID

register handler for any event id

Type Definitions

typedef void ***esp_event_loop_handle_t**

a number that identifies an event with respect to a base

typedef void (***esp_event_handler_t**)(void *event_handler_arg, esp_event_base_t event_base, int32_t event_id, void *event_data)

function called when an event is posted to the queue

typedef void ***esp_event_handler_instance_t**

context identifying an instance of a registered event handler

Related Documents

2.10.11 FreeRTOS Overview

Overview

FreeRTOS is an open source RTOS (real-time operating system) kernel that is integrated into ESP-IDF as a component. Thus, all ESP-IDF applications and many ESP-IDF components are written based on FreeRTOS. The FreeRTOS kernel is ported to all architectures (i.e., Xtensa and RISC-V) available of ESP chips.

Furthermore, ESP-IDF provides different implementations of FreeRTOS in order to support SMP (Symmetric Multiprocessing) on multi-core ESP chips. This document provides an overview of the FreeRTOS component, the different FreeRTOS implementations offered by ESP-IDF, and the common aspects across all implementations.

Implementations

The official FreeRTOS (henceforth referred to as Vanilla FreeRTOS) is a single-core RTOS. In order to support the various multi-core ESP targets, ESP-IDF supports different FreeRTOS implementations as listed below:

ESP-IDF FreeRTOS ESP-IDF FreeRTOS is a FreeRTOS implementation based on Vanilla FreeRTOS v10.5.1, but contains significant modifications to support SMP. ESP-IDF FreeRTOS only supports two cores at most (i.e., dual core SMP), but is more optimized for this scenario by design. For more details regarding ESP-IDF FreeRTOS and its modifications, please refer to the *FreeRTOS (IDF)* document.

Note: ESP-IDF FreeRTOS is currently the default FreeRTOS implementation for ESP-IDF.

Amazon SMP FreeRTOS Amazon SMP FreeRTOS is an SMP implementation of FreeRTOS that is officially supported by Amazon. Amazon SMP FreeRTOS is able to support N-cores (i.e., more than two cores). Amazon SMP FreeRTOS can be enabled via the *CONFIG_FREERTOS_SMP* option. For more details regarding Amazon SMP FreeRTOS, please refer to the official Amazon SMP FreeRTOS documentation.

Warning: The Amazon SMP FreeRTOS implementation (and its port in ESP-IDF) are currently in experimental/beta state. Therefore, significant behavioral changes and breaking API changes can occur.

Configuration

Kernel Configuration Vanilla FreeRTOS requires that ports and applications configure the kernel by adding various `#define config...` macro definitions to the `FreeRTOSConfig.h` header file. Vanilla FreeRTOS supports a list of kernel configuration options which allow various kernel behaviors and features to be enabled or disabled.

However, for all FreeRTOS ports in ESP-IDF, the FreeRTOSConfig.h header file is considered private and must not be modified by users. A large number of kernel configuration options in `FreeRTOSConfig.h` are hard-coded as they are either required/not supported by ESP-IDF. All kernel configuration options that are configurable by the user are exposed via menuconfig under `Component Config/FreeRTOS/Kernel`.

For the full list of user configurable kernel options, see *Project Configuration*. The list below highlights some commonly used kernel configuration options:

- *CONFIG_FREERTOS_UNICORE* runs FreeRTOS only on Core 0. Note that this is **not equivalent to running Vanilla FreeRTOS**. Furthermore, this option may affect behavior of components other than freertos. For more details regarding the effects of running FreeRTOS on a single core, refer to *Single-Core Mode* (if using ESP-IDF FreeRTOS) or the official Amazon SMP FreeRTOS documentation. Alternatively, users can also search for occurrences of `CONFIG_FREERTOS_UNICORE` in the ESP-IDF components.
- *CONFIG_FREERTOS_ENABLE_BACKWARD_COMPATIBILITY* enables backward compatibility with some FreeRTOS macros/types/functions that were deprecated from v8.0 onwards.

Port Configuration All other FreeRTOS related configuration options that are not part of the kernel configuration are exposed via menuconfig under `Component Config/FreeRTOS/Port`. These options configure aspects such as:

- The FreeRTOS ports themselves (e.g., tick timer selection, ISR stack size)
- Additional features added to the FreeRTOS implementation or ports

Using FreeRTOS

Application Entry Point Unlike Vanilla FreeRTOS, users of FreeRTOS in ESP-IDF **must never call** `vTaskStartScheduler()` and `vTaskEndScheduler()`. Instead, ESP-IDF starts FreeRTOS automatically. Users must define a `void app_main(void)` function which acts as the entry point for user's application and is automatically invoked on ESP-IDF startup.

- Typically, users would spawn the rest of their application's task from `app_main`.
- The `app_main` function is allowed to return at any point (i.e., before the application terminates).
- The `app_main` function is called from the `main` task.

Background Tasks During startup, ESP-IDF and the FreeRTOS kernel automatically create multiple tasks that run in the background (listed in the the table below).

Table 10: List of Tasks Created During Startup

Task Name	Description	Stack Size	Affinity	Priority
Idle Tasks (`IDLEx`)	An idle task (`IDLEx`) is created for (and pinned to) each core, where `x` is the core's number. `x` is dropped when single-core configuration is enabled.	*CONFIG_FREERTOS_IDLE_TASK_ST*	Core *x*	0
FreeRTOS Timer Task (`Tmr Svc`)	FreeRTOS will create the Timer Service/Daemon Task if any FreeRTOS Timer APIs are called by the application	*CONFIG_FREERTOS_TIMER_TASK_S*	Core *x*	*CONFIG_FREERTOS_TIMER_TASK_PRIO*
Main Task (`main`)	Task that simply calls `app_main`. This task will self delete when `app_main` returns	*CONFIG_ESP_MAIN_TASK_STACK_SI*	*CONFIG_ESP_MAIN_TASK_AFFINITY*	1
IPC Tasks (`ipcx`)	When *CONFIG_FREERTOS_UNICORE* is false, an IPC task (`ipcx`) is created for (and pinned to) each core. IPC tasks are used to implement the Inter-processor Call (IPC) feature.	*CONFIG_ESP_IPC_TASK_STACK_SIZ*	Core *x*	24
ESP Timer Task (`esp_timer`)	ESP-IDF creates the ESP Timer Task used to process ESP Timer callbacks	*CONFIG_ESP_TIMER_TASK_STACK_*	Core 0	22

Note: Note that if an application uses other ESP-IDF features (e.g., Wi-Fi or Bluetooth), those features may create

their own background tasks in addition to the tasks listed in the table above.

FreeRTOS Additions

ESP-IDF provides some supplemental features to FreeRTOS such as Ring Buffers, ESP-IDF style Tick and Idle Hooks, and TLSP deletion callbacks. See *FreeRTOS (Supplemental Features)* for more details.

FreeRTOS Heap

Vanilla FreeRTOS provides its own selection of heap implementations. However, ESP-IDF already implements its own heap (see *Heap Memory Allocation*), thus ESP-IDF does not make use of the heap implementations provided by Vanilla FreeRTOS. All FreeRTOS ports in ESP-IDF map FreeRTOS memory allocation or free calls (e.g., `pvPortMalloc()` and `pvPortFree()`) to ESP-IDF heap API (i.e., `heap_caps_malloc()` and `heap_caps_free()`). However, the FreeRTOS ports ensure that all dynamic memory allocated by FreeRTOS is placed in internal memory.

Note: If users wish to place FreeRTOS tasks/objects in external memory, users can use the following methods:

- Allocate the task or object using one of the `...CreateWithCaps()` API, such as `xTaskCreateWithCaps()` and `xQueueCreateWithCaps()` (see *IDF Additional API* for more details).
- Manually allocate external memory for those objects using `heap_caps_malloc()`, then create the objects from the allocated memory using on of the `...CreateStatic()` FreeRTOS functions.

2.10.12 FreeRTOS (IDF)

This document provides information regarding the dual-core SMP implementation of FreeRTOS inside ESP-IDF. This document is split into the following sections:

Sections

- *FreeRTOS (IDF)*
  - *Overview*
  - *Symmetric Multiprocessing*
  - *Tasks*
  - *SMP Scheduler*
  - *Critical Sections*
  - *Misc*
  - *Single-Core Mode*
  - *API Reference*

Overview

The original FreeRTOS (hereinafter referred to as **Vanilla FreeRTOS**) is a compact and efficient real-time operating system supported on numerous single-core MCUs and SoCs. However, to support dual-core ESP targets, such as ESP32, ESP32-S3, and ESP32-P4, ESP-IDF provides a unique implementation of FreeRTOS with dual-core symmetric multiprocessing (SMP) capabilities (hereinafter referred to as **IDF FreeRTOS**).

IDF FreeRTOS source code is based on Vanilla FreeRTOS v10.5.1 but contains significant modifications to both kernel behavior and API in order to support dual-core SMP. However, IDF FreeRTOS can also be configured for single-core by enabling the *CONFIG_FREERTOS_UNICORE* option (see *Single-Core Mode* for more details).

Note: This document assumes that the reader has a requisite understanding of Vanilla FreeRTOS, i.e., its features, behavior, and API usage. Refer to the Vanilla FreeRTOS documentation for more details.

Symmetric Multiprocessing

Basic Concepts Symmetric multiprocessing is a computing architecture where two or more identical CPU cores are connected to a single shared main memory and controlled by a single operating system. In general, an SMP system:

- has multiple cores running independently. Each core has its own register file, interrupts, and interrupt handling.
- presents an identical view of memory to each core. Thus, a piece of code that accesses a particular memory address has the same effect regardless of which core it runs on.

The main advantages of an SMP system compared to single-core or asymmetric multiprocessing systems are that:

- the presence of multiple cores allows for multiple hardware threads, thus increasing overall processing throughput.
- having symmetric memory means that threads can switch cores during execution. This, in general, can lead to better CPU utilization.

Although an SMP system allows threads to switch cores, there are scenarios where a thread must/should only run on a particular core. Therefore, threads in an SMP system also have a core affinity that specifies which particular core the thread is allowed to run on.

- A thread that is pinned to a particular core is only able to run on that core.
- A thread that is unpinned will be allowed to switch between cores during execution instead of being pinned to a particular core.

SMP on an ESP Target ESP targets such as ESP32, ESP32-S3, and ESP32-P4 are dual-core SMP SoCs. These targets have the following hardware features that make them SMP-capable:

- Two identical cores are known as Core 0 and Core 1. This means that the execution of a piece of code is identical regardless of which core it runs on.
- Symmetric memory (with some small exceptions).
  - If multiple cores access the same memory address simultaneously, their access will be serialized by the memory bus.
  - True atomic access to the same memory address is achieved via an atomic compare-and-swap instruction provided by the ISA.
- Cross-core interrupts that allow one core to trigger an interrupt on the other core. This allows cores to signal events to each other (such as requesting a context switch on the other core).

Note: Within ESP-IDF, Core 0 and Core 1 are sometimes referred to as `PRO_CPU` and `APP_CPU` respectively. The aliases exist in ESP-IDF as they reflect how typical ESP-IDF applications utilize the two cores. Typically, the tasks responsible for handling protocol related processing such as Wi-Fi or Bluetooth are pinned to Core 0 (thus the name `PRO_CPU`), where as the tasks handling the remainder of the application are pinned to Core 1, (thus the name `APP_CPU`).

Tasks

Creation Vanilla FreeRTOS provides the following functions to create a task:

- *`xTaskCreate()`* creates a task. The task's memory is dynamically allocated.
- *`xTaskCreateStatic()`* creates a task. The task's memory is statically allocated, i.e., provided by the user.

However, in an SMP system, tasks need to be assigned a particular affinity. Therefore, ESP-IDF provides a `...PinnedToCore()` version of Vanilla FreeRTOS's task creation functions:

- *`xTaskCreatePinnedToCore()`* creates a task with a particular core affinity. The task's memory is dynamically allocated.
- *`xTaskCreateStaticPinnedToCore()`* creates a task with a particular core affinity. The task's memory is statically allocated, i.e., provided by the user.

The `...PinnedToCore()` versions of the task creation function API differ from their vanilla counterparts by having an extra `xCoreID` parameter that is used to specify the created task's core affinity. The valid values for core affinity are:

- `0`, which pins the created task to Core 0
- `1`, which pins the created task to Core 1
- `tskNO_AFFINITY`, which allows the task to be run on both cores

Note that IDF FreeRTOS still supports the vanilla versions of the task creation functions. However, these standard functions have been modified to essentially invoke their respective `...PinnedToCore()` counterparts while setting the core affinity to `tskNO_AFFINITY`.

Note: IDF FreeRTOS also changes the units of `ulStackDepth` in the task creation functions. Task stack sizes in Vanilla FreeRTOS are specified in a number of words, whereas in IDF FreeRTOS, the task stack sizes are specified in bytes.

Execution The anatomy of a task in IDF FreeRTOS is the same as in Vanilla FreeRTOS. More specifically, IDF FreeRTOS tasks:

- Can only be in one of the following states: Running, Ready, Blocked, or Suspended.
- Task functions are typically implemented as an infinite loop.
- Task functions should never return.

Deletion Task deletion in Vanilla FreeRTOS is called via *`vTaskDelete()`*. The function allows deletion of another task or the currently running task if the provided task handle is `NULL`. The actual freeing of the task's memory is sometimes delegated to the idle task if the task being deleted is the currently running task.

IDF FreeRTOS provides the same *`vTaskDelete()`* function. However, due to the dual-core nature, there are some behavioral differences when calling *`vTaskDelete()`* in IDF FreeRTOS:

- When deleting a task that is currently running on the other core, a yield is triggered on the other core, and the task's memory is freed by one of the idle tasks.
- A deleted task's memory is freed immediately if it is not running on either core.

Please avoid deleting a task that is running on another core as it is difficult to determine what the task is performing, which may lead to unpredictable behavior such as:

- Deleting a task that is holding a mutex.
- Deleting a task that has yet to free memory it previously allocated.

Where possible, please design your own application so that when calling *`vTaskDelete()`*, the deleted task is in a known state. For example:

- Tasks self-deleting via `vTaskDelete(NULL)` when their execution is complete and have also cleaned up all resources used within the task.
- Tasks placing themselves in the suspend state via *`vTaskSuspend()`* before being deleted by another task.

SMP Scheduler

The Vanilla FreeRTOS scheduler is best described as a **fixed priority preemptive scheduler with time slicing** meaning that:

- Each task is given a constant priority upon creation. The scheduler executes the highest priority ready-state task.
- The scheduler can switch execution to another task without the cooperation of the currently running task.
- The scheduler periodically switches execution between ready-state tasks of the same priority in a round-robin fashion. Time slicing is governed by a tick interrupt.

The IDF FreeRTOS scheduler supports the same scheduling features, i.e., Fixed Priority, Preemption, and Time Slicing, albeit with some small behavioral differences.

Fixed Priority In Vanilla FreeRTOS, when the scheduler selects a new task to run, it always selects the current highest priority ready-state task. In IDF FreeRTOS, each core independently schedules tasks to run. When a particular core selects a task, the core will select the highest priority ready-state task that can be run by the core. A task can be run by the core if:

- The task has a compatible affinity, i.e., is either pinned to that core or is unpinned.
- The task is not currently being run by another core.

However, please do not assume that the two highest priority ready-state tasks are always run by the scheduler, as a task's core affinity must also be accounted for. For example, given the following tasks:

- Task A of priority 10 pinned to Core 0
- Task B of priority 9 pinned to Core 0
- Task C of priority 8 pinned to Core 1

The resulting schedule will have Task A running on Core 0 and Task C running on Core 1. Task B is not run even though it is the second-highest priority task.

Preemption In Vanilla FreeRTOS, the scheduler can preempt the currently running task if a higher priority task becomes ready to execute. Likewise in IDF FreeRTOS, each core can be individually preempted by the scheduler if the scheduler determines that a higher-priority task can run on that core.

However, there are some instances where a higher-priority task that becomes ready can be run on multiple cores. In this case, the scheduler only preempts one core. The scheduler always gives preference to the current core when multiple cores can be preempted. In other words, if the higher priority ready task is unpinned and has a higher priority than the current priority of both cores, the scheduler will always choose to preempt the current core. For example, given the following tasks:

- Task A of priority 8 currently running on Core 0
- Task B of priority 9 currently running on Core 1
- Task C of priority 10 that is unpinned and was unblocked by Task B

The resulting schedule will have Task A running on Core 0 and Task C preempting Task B given that the scheduler always gives preference to the current core.

Time Slicing The Vanilla FreeRTOS scheduler implements time slicing, which means that if the current highest ready priority contains multiple ready tasks, the scheduler will switch between those tasks periodically in a round-robin fashion.

However, in IDF FreeRTOS, it is not possible to implement perfect Round Robin time slicing due to the fact that a particular task may not be able to run on a particular core due to the following reasons:

- The task is pinned to another core.
- For unpinned tasks, the task is already being run by another core.

Therefore, when a core searches the ready-state task list for a task to run, the core may need to skip over a few tasks in the same priority list or drop to a lower priority in order to find a ready-state task that the core can run.

The IDF FreeRTOS scheduler implements a Best Effort Round Robin time slicing for ready-state tasks of the same priority by ensuring that tasks that have been selected to run are placed at the back of the list, thus giving unselected tasks a higher priority on the next scheduling iteration (i.e., the next tick interrupt or yield).

The following example demonstrates the Best Effort Round Robin time slicing in action. Assume that:

- There are four ready-state tasks of the same priority `AX`, `B0`, `C1`, and `D1` where:
  - The priority is the current highest priority with ready-state .
  - The first character represents the task's name, i.e., `A`, `B`, `C`, `D`.
  - The second character represents the task's core pinning, and `X` means unpinned.
- The task list is always searched from the head.

1. Starting state. None of the ready-state tasks have been selected to run.

```
Head [ AX , B0 , C1 , D0 ] Tail
```

2. Core 0 has a tick interrupt and searches for a task to run. Task A is selected and moved to the back of the list.

```
Core 0 ─┐
        ▼
Head [ AX , B0 , C1 , D0 ] Tail

                          [0]
Head [ B0 , C1 , D0 , AX ] Tail
```

3. Core 1 has a tick interrupt and searches for a task to run. Task B cannot be run due to incompatible affinity, so Core 1 skips to Task C. Task C is selected and moved to the back of the list.

```
Core 1 ──────┐
             ▼          [0]
Head [ B0 , C1 , D0 , AX ] Tail

                   [0]   [1]
Head [ B0 , D0 , AX , C1 ] Tail
```

4. Core 0 has another tick interrupt and searches for a task to run. Task B is selected and moved to the back of the list.

```
Core 0 ─┐
        ▼               [1]
Head [ B0 , D0 , AX , C1 ] Tail

                   [1]   [0]
Head [ D0 , AX , C1 , B0 ] Tail
```

5. Core 1 has another tick and searches for a task to run. Task D cannot be run due to incompatible affinity, so Core 1 skips to Task A. Task A is selected and moved to the back of the list.

```
Core 1 ──────┐
             ▼          [0]
Head [ D0 , AX , C1 , B0 ] Tail

                   [0]   [1]
Head [ D0 , C1 , B0 , AX ] Tail
```

The implications to users regarding the Best Effort Round Robin time slicing:

- Users cannot expect multiple ready-state tasks of the same priority to run sequentially as is the case in Vanilla FreeRTOS. As demonstrated in the example above, a core may need to skip over tasks.
- However, given enough ticks, a task will eventually be given some processing time.
- If a core cannot find a task runnable task at the highest ready-state priority, it will drop to a lower priority to search for tasks.
- To achieve ideal round-robin time slicing, users should ensure that all tasks of a particular priority are pinned to the same core.

Tick Interrupts Vanilla FreeRTOS requires that a periodic tick interrupt occurs. The tick interrupt is responsible for:

- Incrementing the scheduler's tick count
- Unblocking any blocked tasks that have timed out

- Checking if time slicing is required, i.e., triggering a context switch
- Executing the application tick hook

In IDF FreeRTOS, each core receives a periodic interrupt and independently runs the tick interrupt. The tick interrupts on each core are of the same period but can be out of phase. However, the tick responsibilities listed above are not run by all cores:

- Core 0 executes all of the tick interrupt responsibilities listed above
- Core 1 only checks for time slicing and executes the application tick hook

---

Note: Core 0 is solely responsible for keeping time in IDF FreeRTOS. Therefore, anything that prevents Core 0 from incrementing the tick count, such as suspending the scheduler on Core 0, will cause the entire scheduler's timekeeping to lag behind.

Idle Tasks Vanilla FreeRTOS will implicitly create an idle task of priority 0 when the scheduler is started. The idle task runs when no other task is ready to run, and it has the following responsibilities:

- Freeing the memory of deleted tasks
- Executing the application idle hook

In IDF FreeRTOS, a separate pinned idle task is created for each core. The idle tasks on each core have the same responsibilities as their vanilla counterparts.

Scheduler Suspension Vanilla FreeRTOS allows the scheduler to be suspended/resumed by calling `vTaskSuspendAll()` and `xTaskResumeAll()` respectively. While the scheduler is suspended:

- Task switching is disabled but interrupts are left enabled.
- Calling any blocking/yielding function is forbidden, and time slicing is disabled.
- The tick count is frozen, but the tick interrupt still occurs to execute the application tick hook.

On scheduler resumption, `xTaskResumeAll()` catches up all of the lost ticks and unblock any timed-out tasks.

In IDF FreeRTOS, suspending the scheduler across multiple cores is not possible. Therefore when `vTaskSuspendAll()` is called on a particular core (e.g., core A):

- Task switching is disabled only on core A but interrupts for core A are left enabled.
- Calling any blocking/yielding function on core A is forbidden. Time slicing is disabled on core A.
- If an interrupt on core A unblocks any tasks, tasks with affinity to core A will go into core A's own pending ready task list. Unpinned tasks or tasks with affinity to other cores can be scheduled on cores with the scheduler running.
- If the scheduler is suspended on all cores, tasks unblocked by an interrupt will be directed to the pending ready task lists of their pinned cores. For unpinned tasks, they will be placed in the pending ready list of the core where the interrupt occurred.
- If core A is on Core 0, the tick count is frozen, and a pended tick count is incremented instead. However, the tick interrupt will still occur in order to execute the application tick hook.

When `xTaskResumeAll()` is called on a particular core (e.g., core A):

- Any tasks added to core A's pending ready task list will be resumed.
- If core A is Core 0, the pended tick count is unwound to catch up with the lost ticks.

Warning: Given that scheduler suspension on IDF FreeRTOS only suspends scheduling on a particular core, scheduler suspension is **NOT** a valid method of ensuring mutual exclusion between tasks when accessing shared data. Users should use proper locking primitives such as mutexes or spinlocks if they require mutual exclusion.

Critical Sections

Disabling Interrupts Vanilla FreeRTOS allows interrupts to be disabled and enabled by calling *taskDISABLE_INTERRUPTS* and *taskENABLE_INTERRUPTS* respectively. IDF FreeRTOS provides the same API. However, interrupts are only disabled or enabled on the current core.

Disabling interrupts is a valid method of achieving mutual exclusion in Vanilla FreeRTOS (and single-core systems in general). **However, in an SMP system, disabling interrupts is not a valid method of ensuring mutual exclusion**. Critical sections that utilize a spinlock should be used instead.

API Changes Vanilla FreeRTOS implements critical sections by disabling interrupts, which prevents preemptive context switches and the servicing of ISRs during a critical section. Thus a task/ISR that enters a critical section is guaranteed to be the sole entity to access a shared resource. Critical sections in Vanilla FreeRTOS have the following API:

- `taskENTER_CRITICAL()` enters a critical section by disabling interrupts
- `taskEXIT_CRITICAL()` exits a critical section by reenabling interrupts
- `taskENTER_CRITICAL_FROM_ISR()` enters a critical section from an ISR by disabling interrupt nesting
- `taskEXIT_CRITICAL_FROM_ISR()` exits a critical section from an ISR by reenabling interrupt nesting

However, in an SMP system, merely disabling interrupts does not constitute a critical section as the presence of other cores means that a shared resource can still be concurrently accessed. Therefore, critical sections in IDF FreeRTOS are implemented using spinlocks. To accommodate the spinlocks, the IDF FreeRTOS critical section APIs contain an additional spinlock parameter as shown below:

- Spinlocks are of `portMUX_TYPE` (**not to be confused to FreeRTOS mutexes**)
- `taskENTER_CRITICAL(&spinlock)` enters a critical from a task context
- `taskEXIT_CRITICAL(&spinlock)` exits a critical section from a task context
- `taskENTER_CRITICAL_ISR(&spinlock)` enters a critical section from an interrupt context
- `taskEXIT_CRITICAL_ISR(&spinlock)` exits a critical section from an interrupt context

Note: The critical section API can be called recursively, i.e., nested critical sections. Entering a critical section multiple times recursively is valid so long as the critical section is exited the same number of times it was entered. However, given that critical sections can target different spinlocks, users should take care to avoid deadlocking when entering critical sections recursively.

Spinlocks can be allocated statically or dynamically. As such, macros are provided for both static and dynamic initialization of spinlocks, as demonstrated by the following code snippets.

- Allocating a static spinlock and initializing it using `portMUX_INITIALIZER_UNLOCKED`:

```
// Statically allocate and initialize the spinlock
static portMUX_TYPE my_spinlock = portMUX_INITIALIZER_UNLOCKED;

void some_function(void)
{
    taskENTER_CRITICAL(&my_spinlock);
    // We are now in a critical section
    taskEXIT_CRITICAL(&my_spinlock);
}
```

- Allocating a dynamic spinlock and initializing it using `portMUX_INITIALIZE()`:

```
// Allocate the spinlock dynamically
portMUX_TYPE *my_spinlock = malloc(sizeof(portMUX_TYPE));
// Initialize the spinlock dynamically
portMUX_INITIALIZE(my_spinlock);

...
```

(continues on next page)

(continued from previous page)

```
taskENTER_CRITICAL(my_spinlock);
// Access the resource
taskEXIT_CRITICAL(my_spinlock);
```

Implementation In IDF FreeRTOS, the process of a particular core entering and exiting a critical section is as follows:

- For `taskENTER_CRITICAL(&spinlock)` or `taskENTER_CRITICAL_ISR(&spinlock)`
  1. The core disables its interrupts or interrupt nesting up to `configMAX_SYSCALL_INTERRUPT_PRIORITY`.
  2. The core then spins on the spinlock using an atomic compare-and-set instruction until it acquires the lock. A lock is acquired when the core is able to set the lock's owner value to the core's ID.
  3. Once the spinlock is acquired, the function returns. The remainder of the critical section runs with interrupts or interrupt nesting disabled.
- For `taskEXIT_CRITICAL(&spinlock)` or `taskEXIT_CRITICAL_ISR(&spinlock)`
  1. The core releases the spinlock by clearing the spinlock's owner value.
  2. The core re-enables interrupts or interrupt nesting.

Restrictions and Considerations Given that interrupts (or interrupt nesting) are disabled during a critical section, there are multiple restrictions regarding what can be done within critical sections. During a critical section, users should keep the following restrictions and considerations in mind:

- Critical sections should be kept as short as possible
  - The longer the critical section lasts, the longer a pending interrupt can be delayed.
  - A typical critical section should only access a few data structures and/or hardware registers.
  - If possible, defer as much processing and/or event handling to the outside of critical sections.
- FreeRTOS API should not be called from within a critical section
- Users should never call any blocking or yielding functions within a critical section

Misc

Floating Point Usage Usually, when a context switch occurs:

- the current state of a core's registers are saved to the stack of the task being switched out
- the previously saved state of the core's registers is loaded from the stack of the task being switched in

However, IDF FreeRTOS implements Lazy Context Switching for the Floating Point Unit (FPU) registers of a core. In other words, when a context switch occurs on a particular core (e.g., Core 0), the state of the core's FPU registers is not immediately saved to the stack of the task getting switched out (e.g., Task A). The FPU registers are left untouched until:

- A different task (e.g., Task B) runs on the same core and uses FPU. This will trigger an exception that saves the FPU registers to Task A's stack.
- Task A gets scheduled to the same core and continues execution. Saving and restoring the FPU registers is not necessary in this case.

However, given that tasks can be unpinned and thus can be scheduled on different cores (e.g., Task A switches to Core 1), it is unfeasible to copy and restore the FPU registers across cores. Therefore, when a task utilizes FPU by using a `float` type in its call flow, IDF FreeRTOS will automatically pin the task to the current core it is running on. This ensures that all tasks that use FPU are always pinned to a particular core.

Furthermore, IDF FreeRTOS by default does not support the usage of FPU within an interrupt context given that the FPU register state is tied to a particular task.

Note: ESP targets that contain an FPU do not support hardware acceleration for double precision floating point arithmetic (`double`). Instead, `double` is implemented via software, hence the behavioral restrictions regarding

the `float` type do not apply to `double`. Note that due to the lack of hardware acceleration, `double` operations may consume significantly more CPU time in comparison to `float`.

Single-Core Mode

Although IDF FreeRTOS is modified for dual-core SMP, IDF FreeRTOS can also be built for single-core by enabling the *CONFIG_FREERTOS_UNICORE* option.

For single-core targets (such as ESP32-S2 and ESP32-C3), the *CONFIG_FREERTOS_UNICORE* option is always enabled. For multi-core targets (such as ESP32 and ESP32-S3), *CONFIG_FREERTOS_UNICORE* can also be set, but will result in the application only running Core 0.

When building in single-core mode, IDF FreeRTOS is designed to be identical to Vanilla FreeRTOS, thus all aforementioned SMP changes to kernel behavior are removed. As a result, building IDF FreeRTOS in single-core mode has the following characteristics:

- All operations performed by the kernel inside critical sections are now deterministic (i.e., no walking of linked lists inside critical sections).
- Vanilla FreeRTOS scheduling algorithm is restored (including perfect Round Robin time slicing).
- All SMP specific data is removed from single-core builds.

SMP APIs can still be called in single-core mode. These APIs remain exposed to allow source code to be built for single-core and multi-core, without needing to call a different set of APIs. However, SMP APIs will not exhibit any SMP behavior in single-core mode, thus becoming equivalent to their single-core counterparts. For example:

- any `...ForCore(..., BaseType_t xCoreID)` SMP API will only accept `0` as a valid value for `xCoreID`.
- `...PinnedToCore()` task creation APIs will simply ignore the `xCoreID` core affinity argument.
- Critical section APIs will still require a spinlock argument, but no spinlock will be taken and critical sections revert to simply disabling/enabling interrupts.

API Reference

This section introduces FreeRTOS types, functions, and macros. It is automatically generated from FreeRTOS header files.

Task API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/task.h
- This header file can be included with:

```
#include "freertos/task.h"
```

Functions

static inline BaseType_t **xTaskCreate**(TaskFunction_t pxTaskCode, const char *const pcName, const configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t uxPriority, *TaskHandle_t* *const pxCreatedTask)

Create a new task and add it to the list of tasks that are ready to run.

Internally, within the FreeRTOS implementation, tasks use two blocks of memory. The first block is used to hold the task's data structures. The second block is used by the task as its stack. If a task is created using xTaskCreate() then both blocks of memory are automatically dynamically allocated inside the xTaskCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a task is created using xTaskCreateStatic() then

the application writer must provide the required memory. xTaskCreateStatic() therefore allows a task to be created without using any dynamic memory allocation.

See xTaskCreateStatic() for a version that does not use any dynamic memory allocation.

xTaskCreate() can only be used to create a task that has unrestricted access to the entire microcontroller memory map. Systems that include MPU support can alternatively create an MPU constrained task using xTaskCreateRestricted().

Example usage:

```
// Task to be created.
void vTaskCode( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.
  }
}

// Function that creates a task.
void vOtherFunction( void )
{
static uint8_t ucParameterToPass;
TaskHandle_t xHandle = NULL;

// Create the task, storing the handle.  Note that the passed parameter␣
↪ucParameterToPass
// must exist for the lifetime of the task, so in this case is declared static.
↪  If it was just an
// an automatic stack variable it might no longer exist, or at least have been␣
↪corrupted, by the time
// the new task attempts to access it.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, &ucParameterToPass, tskIDLE_
↪PRIORITY, &xHandle );
  configASSERT( xHandle );

// Use the handle to delete the task.
if( xHandle != NULL )
  {
     vTaskDelete( xHandle );
  }
}
```

Note: If configNUMBER_OF_CORES > 1, this function will create an unpinned task (see tskNO_AFFINITY for more details).

Note: If program uses thread local variables (ones specified with "__thread" keyword) then storage for them will be allocated on the task's stack.

Parameters

- **pxTaskCode** -- Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop).
- **pcName** -- A descriptive name for the task. This is mainly used to facilitate debugging. Max length defined by configMAX_TASK_NAME_LEN - default is 16.
- **usStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.

- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run. Systems that include MPU support can optionally create tasks in a privileged (system) mode by setting bit portPRIVILEGE_BIT of the priority parameter. For example, to create a privileged task at priority 2 the uxPriority parameter should be set to (2 | portPRIVILEGE_BIT).
- **pxCreatedTask** -- Used to pass back a handle by which the created task can be referenced.

Returns pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

static inline *TaskHandle_t* **xTaskCreateStatic**(TaskFunction_t pxTaskCode, const char *const pcName, const uint32_t ulStackDepth, void *const pvParameters, UBaseType_t uxPriority, StackType_t *const puxStackBuffer, StaticTask_t *const pxTaskBuffer)

Create a new task and add it to the list of tasks that are ready to run.

Internally, within the FreeRTOS implementation, tasks use two blocks of memory. The first block is used to hold the task's data structures. The second block is used by the task as its stack. If a task is created using xTaskCreate() then both blocks of memory are automatically dynamically allocated inside the xTaskCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a task is created using xTaskCreateStatic() then the application writer must provide the required memory. xTaskCreateStatic() therefore allows a task to be created without using any dynamic memory allocation.

Example usage:

```
// Dimensions of the buffer that the task being created will use as its stack.
// NOTE:  This is the number of words the stack will hold, not the number of
// bytes.  For example, if each stack item is 32-bits, and this is set to 100,
// then 400 bytes (100 * 32-bits) will be allocated.
#define STACK_SIZE 200

// Structure that will hold the TCB of the task being created.
   StaticTask_t xTaskBuffer;

// Buffer that the task being created will use as its stack.  Note this is
// an array of StackType_t variables.  The size of StackType_t is dependent on
// the RTOS port.
   StackType_t xStack[ STACK_SIZE ];

// Function that implements the task being created.
void vTaskCode( void * pvParameters )
   {
// The parameter value is expected to be 1 as 1 is passed in the
// pvParameters value in the call to xTaskCreateStatic().
       configASSERT( ( uint32_t ) pvParameters == 1UL );

for( ;; )
       {
// Task code goes here.
       }
   }

// Function that creates a task.
void vOtherFunction( void )
   {
       TaskHandle_t xHandle = NULL;

// Create the task without using any dynamic memory allocation.
       xHandle = xTaskCreateStatic(
                     vTaskCode,       // Function that implements the task.
```

(continues on next page)

(continued from previous page)

```
"NAME",           // Text name for the task.
                      STACK_SIZE,      // Stack size in words, not bytes.
                      ( void * ) 1,    // Parameter passed into the task.
                      tskIDLE_PRIORITY,// Priority at which the task is created.
                      xStack,          // Array to use as the task's stack.
                      &xTaskBuffer );  // Variable to hold the task's data␣
↪structure.

// puxStackBuffer and pxTaskBuffer were not NULL, so the task will have
// been created, and xHandle will be the task's handle.  Use the handle
// to suspend the task.
      vTaskSuspend( xHandle );
   }
```

Note: If configNUMBER_OF_CORES > 1, this function will create an unpinned task (see tskNO_AFFINITY for more details).

Note: If program uses thread local variables (ones specified with "__thread" keyword) then storage for them will be allocated on the task's stack.

Parameters

- **pxTaskCode** -- Pointer to the task entry function. Tasks must be implemented to never return (i.e. continuous loop).
- **pcName** -- A descriptive name for the task. This is mainly used to facilitate debugging. The maximum length of the string is defined by configMAX_TASK_NAME_LEN in FreeRTOSConfig.h.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task will run.
- **puxStackBuffer** -- Must point to a StackType_t array that has at least ulStackDepth indexes - the array will then be used as the task's stack, removing the need for the stack to be allocated dynamically.
- **pxTaskBuffer** -- Must point to a variable of type StaticTask_t, which will then be used to hold the task's data structures, removing the need for the memory to be allocated dynamically.

Returns If neither puxStackBuffer nor pxTaskBuffer are NULL, then the task will be created and a handle to the created task is returned. If either puxStackBuffer or pxTaskBuffer are NULL then the task will not be created and NULL is returned.

void **vTaskAllocateMPURegions**(*TaskHandle_t* xTask, const MemoryRegion_t *const pxRegions)

Memory regions are assigned to a restricted task when the task is created by a call to xTaskCreateRestricted(). These regions can be redefined using vTaskAllocateMPURegions().

Example usage:

```
// Define an array of MemoryRegion_t structures that configures an MPU region
// allowing read/write access for 1024 bytes starting at the beginning of the
// ucOneKByte array.  The other two of the maximum 3 definable regions are
// unused so set to zero.
static const MemoryRegion_t xAltRegions[ portNUM_CONFIGURABLE_REGIONS ] =
{
 // Base address     Length      Parameters
```

(continues on next page)

(continued from previous page)

```
 { ucOneKByte,       1024,       portMPU_REGION_READ_WRITE },
 { 0,                0,          0 },
 { 0,                0,          0 }
};

void vATask( void *pvParameters )
{
 // This task was created such that it has access to certain regions of
 // memory as defined by the MPU configuration.  At some point it is
 // desired that these MPU regions are replaced with that defined in the
 // xAltRegions const struct above.  Use a call to vTaskAllocateMPURegions()
 // for this purpose.  NULL is used as the task handle to indicate that this
 // function should modify the MPU regions of the calling task.
 vTaskAllocateMPURegions( NULL, xAltRegions );

 // Now the task can continue its function, but from this point on can only
 // access its stack and the ucOneKByte array (unless any other statically
 // defined or shared regions have been declared elsewhere).
}
```

Parameters

- **xTask** -- The handle of the task being updated.
- **pxRegions** -- A pointer to a MemoryRegion_t structure that contains the new memory region definitions.

void **vTaskDelete**(*TaskHandle_t* xTaskToDelete)

INCLUDE_vTaskDelete must be defined as 1 for this function to be available. See the configuration section for more information.

Remove a task from the RTOS real time kernel's management. The task being deleted will be removed from all ready, blocked, suspended and event lists.

NOTE: The idle task is responsible for freeing the kernel allocated memory from tasks that have been deleted. It is therefore important that the idle task is not starved of microcontroller processing time if your application makes any calls to vTaskDelete (). Memory allocated by the task code is not automatically freed, and should be freed before the task is deleted.

See the demo application file death.c for sample code that utilises vTaskDelete ().

Example usage:

```
void vOtherFunction( void )
{
TaskHandle_t xHandle;

// Create the task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle␣
↪);

// Use the handle to delete the task.
  vTaskDelete( xHandle );
}
```

Parameters **xTaskToDelete** -- The handle of the task to be deleted. Passing NULL will cause the calling task to be deleted.

void **vTaskDelay**(const TickType_t xTicksToDelay)

Delay a task for a given number of ticks. The actual time that the task remains blocked depends on the tick

rate. The constant portTICK_PERIOD_MS can be used to calculate real time from the tick rate - with the resolution of one tick period.

INCLUDE_vTaskDelay must be defined as 1 for this function to be available. See the configuration section for more information.

vTaskDelay() specifies a time at which the task wishes to unblock relative to the time at which vTaskDelay() is called. For example, specifying a block period of 100 ticks will cause the task to unblock 100 ticks after vTaskDelay() is called. vTaskDelay() does not therefore provide a good method of controlling the frequency of a periodic task as the path taken through the code, as well as other task and interrupt activity, will affect the frequency at which vTaskDelay() gets called and therefore the time at which the task next executes. See xTaskDelayUntil() for an alternative API function designed to facilitate fixed frequency execution. It does this by specifying an absolute time (rather than a relative time) at which the calling task should unblock.

Example usage:

```
void vTaskFunction( void * pvParameters )
{
// Block for 500ms.
const TickType_t xDelay = 500 / portTICK_PERIOD_MS;

for( ;; )
  {
// Simply toggle the LED every 500ms, blocking between each toggle.
      vToggleLED();
      vTaskDelay( xDelay );
  }
}
```

Parameters **xTicksToDelay** -- The amount of time, in tick periods, that the calling task should block.

BaseType_t **xTaskDelayUntil**(TickType_t *const pxPreviousWakeTime, const TickType_t xTimeIncrement)

INCLUDE_xTaskDelayUntil must be defined as 1 for this function to be available. See the configuration section for more information.

Delay a task until a specified time. This function can be used by periodic tasks to ensure a constant execution frequency.

This function differs from vTaskDelay () in one important aspect: vTaskDelay () will cause a task to block for the specified number of ticks from the time vTaskDelay () is called. It is therefore difficult to use vTaskDelay () by itself to generate a fixed execution frequency as the time between a task starting to execute and that task calling vTaskDelay () may not be fixed [the task may take a different path though the code between calls, or may get interrupted or preempted a different number of times each time it executes].

Whereas vTaskDelay () specifies a wake time relative to the time at which the function is called, xTaskDelayUntil () specifies the absolute (exact) time at which it wishes to unblock.

The macro pdMS_TO_TICKS() can be used to calculate the number of ticks from a time specified in milliseconds with a resolution of one tick period.

Example usage:

```
// Perform an action every 10 ticks.
void vTaskFunction( void * pvParameters )
{
TickType_t xLastWakeTime;
const TickType_t xFrequency = 10;
```

(continues on next page)

(continued from previous page)

```
BaseType_t xWasDelayed;

// Initialise the xLastWakeTime variable with the current time.
    xLastWakeTime = xTaskGetTickCount ();
for( ;; )
    {
// Wait for the next cycle.
        xWasDelayed = xTaskDelayUntil( &xLastWakeTime, xFrequency );

// Perform action here. xWasDelayed value can be used to determine
// whether a deadline was missed if the code here took too long.
    }
}
```

Parameters

- **pxPreviousWakeTime** -- Pointer to a variable that holds the time at which the task was last unblocked. The variable must be initialised with the current time prior to its first use (see the example below). Following this the variable is automatically updated within xTaskDelayUntil ().
- **xTimeIncrement** -- The cycle time period. The task will be unblocked at time *pxPreviousWakeTime + xTimeIncrement. Calling xTaskDelayUntil with the same xTimeIncrement parameter value will cause the task to execute with a fixed interface period.

Returns Value which can be used to check whether the task was actually delayed. Will be pdTRUE if the task way delayed and pdFALSE otherwise. A task will not be delayed if the next expected wake time is in the past.

BaseType_t **xTaskAbortDelay**(*TaskHandle_t* xTask)

INCLUDE_xTaskAbortDelay must be defined as 1 in FreeRTOSConfig.h for this function to be available.

A task will enter the Blocked state when it is waiting for an event. The event it is waiting for can be a temporal event (waiting for a time), such as when vTaskDelay() is called, or an event on an object, such as when xQueueReceive() or ulTaskNotifyTake() is called. If the handle of a task that is in the Blocked state is used in a call to xTaskAbortDelay() then the task will leave the Blocked state, and return from whichever function call placed the task into the Blocked state.

There is no 'FromISR' version of this function as an interrupt would need to know which object a task was blocked on in order to know which actions to take. For example, if the task was blocked on a queue the interrupt handler would then need to know if the queue was locked.

Parameters **xTask** -- The handle of the task to remove from the Blocked state.

Returns If the task referenced by xTask was not in the Blocked state then pdFAIL is returned. Otherwise pdPASS is returned.

UBaseType_t **uxTaskPriorityGet**(const *TaskHandle_t* xTask)

INCLUDE_uxTaskPriorityGet must be defined as 1 for this function to be available. See the configuration section for more information.

Obtain the priority of any task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle␣
↪);
```

(continues on next page)

(continued from previous page)

```
// ...

// Use the handle to obtain the priority of the created task.
// It was created with tskIDLE_PRIORITY, but may have changed
// it itself.
if( uxTaskPriorityGet( xHandle ) != tskIDLE_PRIORITY )
  {
// The task has changed it's priority.
  }

// ...

// Is our priority higher than the created task?
if( uxTaskPriorityGet( xHandle ) < uxTaskPriorityGet( NULL ) )
  {
// Our priority (obtained using NULL handle) is higher.
  }
}
```

Parameters **xTask** -- Handle of the task to be queried. Passing a NULL handle results in the priority of the calling task being returned.

Returns The priority of xTask.

UBaseType_t **uxTaskPriorityGetFromISR**(const *TaskHandle_t* xTask)

A version of uxTaskPriorityGet() that can be used from an ISR.

eTaskState **eTaskGetState**(*TaskHandle_t* xTask)

INCLUDE_eTaskGetState must be defined as 1 for this function to be available. See the configuration section for more information.

Obtain the state of any task. States are encoded by the eTaskState enumerated type.

Parameters **xTask** -- Handle of the task to be queried.

Returns The state of xTask at the time the function was called. Note the state of the task might change between the function being called, and the functions return value being tested by the calling task.

void **vTaskGetInfo**(*TaskHandle_t* xTask, *TaskStatus_t* *pxTaskStatus, BaseType_t xGetFreeStackSpace, *eTaskState* eState)

configUSE_TRACE_FACILITY must be defined as 1 for this function to be available. See the configuration section for more information.

Populates a TaskStatus_t structure with information about a task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;
TaskStatus_t xTaskDetails;

 // Obtain the handle of a task from its name.
 xHandle = xTaskGetHandle( "Task_Name" );

 // Check the handle is not NULL.
 configASSERT( xHandle );

 // Use the handle to obtain further information about the task.
 vTaskGetInfo( xHandle,
```

(continues on next page)

(continued from previous page)

```
                &xTaskDetails,
                pdTRUE, // Include the high water mark in xTaskDetails.
                eInvalid ); // Include the task state in xTaskDetails.
}
```

Parameters

- **xTask** -- Handle of the task being queried. If xTask is NULL then information will be returned about the calling task.
- **pxTaskStatus** -- A pointer to the TaskStatus_t structure that will be filled with information about the task referenced by the handle passed using the xTask parameter.
- **xGetFreeStackSpace** -- The TaskStatus_t structure contains a member to report the stack high water mark of the task being queried. Calculating the stack high water mark takes a relatively long time, and can make the system temporarily unresponsive - so the xGetFreeStackSpace parameter is provided to allow the high water mark checking to be skipped. The high watermark value will only be written to the TaskStatus_t structure if xGetFreeStackSpace is not set to pdFALSE;
- **eState** -- The TaskStatus_t structure contains a member to report the state of the task being queried. Obtaining the task state is not as fast as a simple assignment - so the eState parameter is provided to allow the state information to be omitted from the TaskStatus_t structure. To obtain state information then set eState to eInvalid - otherwise the value passed in eState will be reported as the task state in the TaskStatus_t structure.

void **vTaskPrioritySet**(*TaskHandle_t* xTask, UBaseType_t uxNewPriority)

INCLUDE_vTaskPrioritySet must be defined as 1 for this function to be available. See the configuration section for more information.

Set the priority of any task.

A context switch will occur before the function returns if the priority being set is higher than the currently executing task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle␣
↪);

// ...

// Use the handle to raise the priority of the created task.
  vTaskPrioritySet( xHandle, tskIDLE_PRIORITY + 1 );

// ...

// Use a NULL handle to raise our priority to the same value.
  vTaskPrioritySet( NULL, tskIDLE_PRIORITY + 1 );
}
```

Parameters

- **xTask** -- Handle to the task for which the priority is being set. Passing a NULL handle results in the priority of the calling task being set.
- **uxNewPriority** -- The priority to which the task will be set.

void **vTaskSuspend**(*TaskHandle_t* xTaskToSuspend)

INCLUDE_vTaskSuspend must be defined as 1 for this function to be available. See the configuration section for more information.

Suspend any task. When suspended a task will never get any microcontroller processing time, no matter what its priority.

Calls to vTaskSuspend are not accumulative - i.e. calling vTaskSuspend () twice on the same task still only requires one call to vTaskResume () to ready the suspended task.

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle␣
↪);

// ...

// Use the handle to suspend the created task.
  vTaskSuspend( xHandle );

// ...

// The created task will not run during this period, unless
// another task calls vTaskResume( xHandle ).

//...


// Suspend ourselves.
  vTaskSuspend( NULL );

// We cannot get here unless another task calls vTaskResume
// with our handle as the parameter.
}
```

Parameters **xTaskToSuspend** -- Handle to the task being suspended. Passing a NULL handle will cause the calling task to be suspended.

void **vTaskResume**(*TaskHandle_t* xTaskToResume)

INCLUDE_vTaskSuspend must be defined as 1 for this function to be available. See the configuration section for more information.

Resumes a suspended task.

A task that has been suspended by one or more calls to vTaskSuspend () will be made available for running again by a single call to vTaskResume ().

Example usage:

```
void vAFunction( void )
{
TaskHandle_t xHandle;

// Create a task, storing the handle.
```

(continues on next page)

(continued from previous page)

```
  xTaskCreate( vTaskCode, "NAME", STACK_SIZE, NULL, tskIDLE_PRIORITY, &xHandle␣
↪);

// ...

// Use the handle to suspend the created task.
  vTaskSuspend( xHandle );

// ...

// The created task will not run during this period, unless
// another task calls vTaskResume( xHandle ).

//...


// Resume the suspended task ourselves.
  vTaskResume( xHandle );

// The created task will once again get microcontroller processing
// time in accordance with its priority within the system.
}
```

Parameters **xTaskToResume** -- Handle to the task being readied.

BaseType_t **xTaskResumeFromISR** (*TaskHandle_t* xTaskToResume)

INCLUDE_xTaskResumeFromISR must be defined as 1 for this function to be available. See the configuration section for more information.

An implementation of vTaskResume() that can be called from within an ISR.

A task that has been suspended by one or more calls to vTaskSuspend () will be made available for running again by a single call to xTaskResumeFromISR ().

xTaskResumeFromISR() should not be used to synchronise a task with an interrupt if there is a chance that the interrupt could arrive prior to the task being suspended - as this can lead to interrupts being missed. Use of a semaphore as a synchronisation mechanism would avoid this eventuality.

Parameters **xTaskToResume** -- Handle to the task being readied.

Returns pdTRUE if resuming the task should result in a context switch, otherwise pdFALSE. This is used by the ISR to determine if a context switch may be required following the ISR.

void **vTaskSuspendAll** (void)

Suspends the scheduler without disabling interrupts. Context switches will not occur while the scheduler is suspended.

After calling vTaskSuspendAll () the calling task will continue to execute without risk of being swapped out until a call to xTaskResumeAll () has been made.

API functions that have the potential to cause a context switch (for example, xTaskDelayUntil(), xQueueSend(), etc.) must not be called while the scheduler is suspended.

Example usage:

```
void vTask1( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.

// ...
```

(continues on next page)

(continued from previous page)

```
// At some point the task wants to perform a long operation during
// which it does not want to get swapped out.  It cannot use
// taskENTER_CRITICAL ()/taskEXIT_CRITICAL () as the length of the
// operation may cause interrupts to be missed - including the
// ticks.

// Prevent the real time kernel swapping out the task.
      vTaskSuspendAll ();

// Perform the operation here.  There is no need to use critical
// sections as we have all the microcontroller processing time.
// During this time interrupts will still operate and the kernel
// tick count will be maintained.

// ...

// The operation is complete.  Restart the kernel.
      xTaskResumeAll ();
  }
}
```

BaseType_t **xTaskResumeAll** (void)

Resumes scheduler activity after it was suspended by a call to vTaskSuspendAll().

xTaskResumeAll() only resumes the scheduler. It does not unsuspend tasks that were previously suspended by a call to vTaskSuspend().

Example usage:

```
void vTask1( void * pvParameters )
{
for( ;; )
  {
// Task code goes here.

// ...

// At some point the task wants to perform a long operation during
// which it does not want to get swapped out.  It cannot use
// taskENTER_CRITICAL ()/taskEXIT_CRITICAL () as the length of the
// operation may cause interrupts to be missed - including the
// ticks.

// Prevent the real time kernel swapping out the task.
      vTaskSuspendAll ();

// Perform the operation here.  There is no need to use critical
// sections as we have all the microcontroller processing time.
// During this time interrupts will still operate and the real
// time kernel tick count will be maintained.

// ...

// The operation is complete.  Restart the kernel.  We want to force
// a context switch - but there is no point if resuming the scheduler
// caused a context switch already.
if( !xTaskResumeAll () )
      {
           taskYIELD ();
      }
```

(continues on next page)

(continued from previous page)

```
    }
}
```

Returns If resuming the scheduler caused a context switch then pdTRUE is returned, otherwise pdFALSE is returned.

TickType_t **xTaskGetTickCount**(void)

Returns The count of ticks since vTaskStartScheduler was called.

TickType_t **xTaskGetTickCountFromISR**(void)

This is a version of xTaskGetTickCount() that is safe to be called from an ISR - provided that TickType_t is the natural word size of the microcontroller being used or interrupt nesting is either not supported or not being used.

Returns The count of ticks since vTaskStartScheduler was called.

UBaseType_t **uxTaskGetNumberOfTasks**(void)

Returns The number of tasks that the real time kernel is currently managing. This includes all ready, blocked and suspended tasks. A task that has been deleted but not yet freed by the idle task will also be included in the count.

char ***pcTaskGetName**(*TaskHandle_t* xTaskToQuery)

Returns The text (human readable) name of the task referenced by the handle xTaskToQuery. A task can query its own name by either passing in its own handle, or by setting xTaskToQuery to NULL.

TaskHandle_t **xTaskGetHandle**(const char *pcNameToQuery)

NOTE: This function takes a relatively long time to complete and should be used sparingly.

Returns The handle of the task that has the human readable name pcNameToQuery. NULL is returned if no matching name is found. INCLUDE_xTaskGetHandle must be set to 1 in FreeRTOSConfig.h for pcTaskGetHandle() to be available.

BaseType_t **xTaskGetStaticBuffers**(*TaskHandle_t* xTask, StackType_t **ppuxStackBuffer, StaticTask_t **ppxTaskBuffer)

Retrieve pointers to a statically created task's data structure buffer and stack buffer. These are the same buffers that are supplied at the time of creation.

Parameters

- **xTask** -- The task for which to retrieve the buffers.
- **ppuxStackBuffer** -- Used to return a pointer to the task's stack buffer.
- **ppxTaskBuffer** -- Used to return a pointer to the task's data structure buffer.

Returns pdTRUE if buffers were retrieved, pdFALSE otherwise.

UBaseType_t **uxTaskGetStackHighWaterMark**(*TaskHandle_t* xTask)

INCLUDE_uxTaskGetStackHighWaterMark must be set to 1 in FreeRTOSConfig.h for this function to be available.

Returns the high water mark of the stack associated with xTask. That is, the minimum free stack space there has been (in words, so on a 32 bit machine a value of 1 means 4 bytes) since the task started. The smaller the returned number the closer the task has come to overflowing its stack.

uxTaskGetStackHighWaterMark() and uxTaskGetStackHighWaterMark2() are the same except for their return type. Using configSTACK_DEPTH_TYPE allows the user to determine the return type. It gets around the problem of the value overflowing on 8-bit types without breaking backward compatibility for applications that expect an 8-bit return type.

Parameters **xTask** -- Handle of the task associated with the stack to be checked. Set xTask to NULL to check the stack of the calling task.

Returns The smallest amount of free stack space there has been (in words, so actual spaces on the stack rather than bytes) since the task referenced by xTask was created.

configSTACK_DEPTH_TYPE **uxTaskGetStackHighWaterMark2** (*TaskHandle_t* xTask)

INCLUDE_uxTaskGetStackHighWaterMark2 must be set to 1 in FreeRTOSConfig.h for this function to be available.

Returns the high water mark of the stack associated with xTask. That is, the minimum free stack space there has been (in words, so on a 32 bit machine a value of 1 means 4 bytes) since the task started. The smaller the returned number the closer the task has come to overflowing its stack.

uxTaskGetStackHighWaterMark() and uxTaskGetStackHighWaterMark2() are the same except for their return type. Using configSTACK_DEPTH_TYPE allows the user to determine the return type. It gets around the problem of the value overflowing on 8-bit types without breaking backward compatibility for applications that expect an 8-bit return type.

Parameters **xTask** -- Handle of the task associated with the stack to be checked. Set xTask to NULL to check the stack of the calling task.

Returns The smallest amount of free stack space there has been (in words, so actual spaces on the stack rather than bytes) since the task referenced by xTask was created.

void **vTaskSetApplicationTaskTag** (*TaskHandle_t* xTask, *TaskHookFunction_t* pxHookFunction)

Sets pxHookFunction to be the task hook function used by the task xTask. Passing xTask as NULL has the effect of setting the calling tasks hook function.

TaskHookFunction_t **xTaskGetApplicationTaskTag** (*TaskHandle_t* xTask)

Returns the pxHookFunction value assigned to the task xTask. Do not call from an interrupt service routine - call xTaskGetApplicationTaskTagFromISR() instead.

TaskHookFunction_t **xTaskGetApplicationTaskTagFromISR** (*TaskHandle_t* xTask)

Returns the pxHookFunction value assigned to the task xTask. Can be called from an interrupt service routine.

void **vTaskSetThreadLocalStoragePointer** (*TaskHandle_t* xTaskToSet, BaseType_t xIndex, void *pvValue)

Each task contains an array of pointers that is dimensioned by the configNUM_THREAD_LOCAL_STORAGE_POINTERS setting in FreeRTOSConfig.h. The kernel does not use the pointers itself, so the application writer can use the pointers for any purpose they wish. The following two functions are used to set and query a pointer respectively.

void ***pvTaskGetThreadLocalStoragePointer** (*TaskHandle_t* xTaskToQuery, BaseType_t xIndex)

void **vApplicationGetIdleTaskMemory** (StaticTask_t **ppxIdleTaskTCBBuffer, StackType_t **ppxIdleTaskStackBuffer, uint32_t *pulIdleTaskStackSize)

This function is used to provide a statically allocated block of memory to FreeRTOS to hold the Idle Task TCB. This function is required when configSUPPORT_STATIC_ALLOCATION is set. For more information see this URI: https://www.FreeRTOS.org/a00110.html#configSUPPORT_STATIC_ALLOCATION

Parameters

- **ppxIdleTaskTCBBuffer** -- A handle to a statically allocated TCB buffer
- **ppxIdleTaskStackBuffer** -- A handle to a statically allocated Stack buffer for the idle task
- **pulIdleTaskStackSize** -- A pointer to the number of elements that will fit in the allocated stack buffer

BaseType_t **xTaskCallApplicationTaskHook** (*TaskHandle_t* xTask, void *pvParameter)

Calls the hook function associated with xTask. Passing xTask as NULL has the effect of calling the Running tasks (the calling task) hook function.

pvParameter is passed to the hook function for the task to interpret as it wants. The return value is the value returned by the task hook function registered by the user.

TaskHandle_t **xTaskGetIdleTaskHandle** (void)

xTaskGetIdleTaskHandle() is only available if INCLUDE_xTaskGetIdleTaskHandle is set to 1 in FreeRTOSConfig.h.

Simply returns the handle of the idle task of the current core. It is not valid to call xTaskGetIdleTaskHandle() before the scheduler has been started.

UBaseType_t **uxTaskGetSystemState** (*TaskStatus_t* *const pxTaskStatusArray, const UBaseType_t uxArraySize, configRUN_TIME_COUNTER_TYPE *const pulTotalRunTime)

configUSE_TRACE_FACILITY must be defined as 1 in FreeRTOSConfig.h for uxTaskGetSystemState() to be available.

uxTaskGetSystemState() populates an TaskStatus_t structure for each task in the system. TaskStatus_t structures contain, among other things, members for the task handle, task name, task priority, task state, and total amount of run time consumed by the task. See the TaskStatus_t structure definition in this file for the full member list.

NOTE: This function is intended for debugging use only as its use results in the scheduler remaining suspended for an extended period.

Example usage:

```
 // This example demonstrates how a human readable table of run time stats
 // information is generated from raw data provided by uxTaskGetSystemState().
 // The human readable table is written to pcWriteBuffer
 void vTaskGetRunTimeStats( char *pcWriteBuffer )
 {
 TaskStatus_t *pxTaskStatusArray;
 volatile UBaseType_t uxArraySize, x;
 configRUN_TIME_COUNTER_TYPE ulTotalRunTime, ulStatsAsPercentage;

// Make sure the write buffer does not contain a string.
pcWriteBuffer = 0x00;

// Take a snapshot of the number of tasks in case it changes while this
// function is executing.
     uxArraySize = uxTaskGetNumberOfTasks();

// Allocate a TaskStatus_t structure for each task.  An array could be
// allocated statically at compile time.
     pxTaskStatusArray = pvPortMalloc( uxArraySize * sizeof( TaskStatus_t ) );

if( pxTaskStatusArray != NULL )
     {
// Generate raw status information about each task.
         uxArraySize = uxTaskGetSystemState( pxTaskStatusArray, uxArraySize, &
↪ulTotalRunTime );

// For percentage calculations.
         ulTotalRunTime /= 100UL;

// Avoid divide by zero errors.
if( ulTotalRunTime > 0 )
         {
// For each populated position in the pxTaskStatusArray array,
// format the raw data as human readable ASCII data
for( x = 0; x < uxArraySize; x++ )
             {
// What percentage of the total run time has the task used?
// This will always be rounded down to the nearest integer.
```

(continues on next page)

(continued from previous page)

```
// ulTotalRunTimeDiv100 has already been divided by 100.
                ulStatsAsPercentage = pxTaskStatusArray[ x ].ulRunTimeCounter␣
↪/ ulTotalRunTime;

if( ulStatsAsPercentage > 0UL )
                {
                    sprintf( pcWriteBuffer, "%s\t\t%lu\t\t%lu%%\r\n",␣
↪pxTaskStatusArray[ x ].pcTaskName, pxTaskStatusArray[ x ].ulRunTimeCounter,␣
↪ulStatsAsPercentage );
                }
else
                {
// If the percentage is zero here then the task has
// consumed less than 1% of the total run time.
                    sprintf( pcWriteBuffer, "%s\t\t%lu\t\t<1%%\r\n",␣
↪pxTaskStatusArray[ x ].pcTaskName, pxTaskStatusArray[ x ].ulRunTimeCounter );
                }

                pcWriteBuffer += strlen( ( char * ) pcWriteBuffer );
            }
        }

// The array is no longer needed, free the memory it consumes.
        vPortFree( pxTaskStatusArray );
    }
}
```

Parameters

- **pxTaskStatusArray** -- A pointer to an array of TaskStatus_t structures. The array must contain at least one TaskStatus_t structure for each task that is under the control of the RTOS. The number of tasks under the control of the RTOS can be determined using the uxTaskGetNumberOfTasks() API function.
- **uxArraySize** -- The size of the array pointed to by the pxTaskStatusArray parameter. The size is specified as the number of indexes in the array, or the number of TaskStatus_t structures contained in the array, not by the number of bytes in the array.
- **pulTotalRunTime** -- If configGENERATE_RUN_TIME_STATS is set to 1 in FreeRTOSConfig.h then *pulTotalRunTime is set by uxTaskGetSystemState() to the total run time (as defined by the run time stats clock, see https://www.FreeRTOS.org/rtos-run-time-stats.html) since the target booted. pulTotalRunTime can be set to NULL to omit the total run time information.

Returns The number of TaskStatus_t structures that were populated by uxTaskGetSystemState(). This should equal the number returned by the uxTaskGetNumberOfTasks() API function, but will be zero if the value passed in the uxArraySize parameter was too small.

void **vTaskList**(char *pcWriteBuffer)

configUSE_TRACE_FACILITY and configUSE_STATS_FORMATTING_FUNCTIONS must both be defined as 1 for this function to be available. See the configuration section of the FreeRTOS.org website for more information.

NOTE 1: This function will disable interrupts for its duration. It is not intended for normal application runtime use but as a debug aid.

Lists all the current tasks, along with their current state and stack usage high water mark.

Tasks are reported as blocked ('B'), ready ('R'), deleted ('D') or suspended ('S').

PLEASE NOTE:

This function is provided for convenience only, and is used by many of the demo applications. Do not consider it to be part of the scheduler.

vTaskList() calls uxTaskGetSystemState(), then formats part of the uxTaskGetSystemState() output into a human readable table that displays task: names, states, priority, stack usage and task number. Stack usage specified as the number of unused StackType_t words stack can hold on top of stack - not the number of bytes.

vTaskList() has a dependency on the sprintf() C library function that might bloat the code size, use a lot of stack, and provide different results on different platforms. An alternative, tiny, third party, and limited functionality implementation of sprintf() is provided in many of the FreeRTOS/Demo sub-directories in a file called printf-stdarg.c (note printf-stdarg.c does not provide a full snprintf() implementation!).

It is recommended that production systems call uxTaskGetSystemState() directly to get access to raw stats data, rather than indirectly through a call to vTaskList().

Parameters **`pcWriteBuffer`** -- A buffer into which the above mentioned details will be written, in ASCII form. This buffer is assumed to be large enough to contain the generated report. Approximately 40 bytes per task should be sufficient.

void **`vTaskGetRunTimeStats`**(char *pcWriteBuffer)

configGENERATE_RUN_TIME_STATS and configUSE_STATS_FORMATTING_FUNCTIONS must both be defined as 1 for this function to be available. The application must also then provide definitions for portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() to configure a peripheral timer/counter and return the timers current count value respectively. The counter should be at least 10 times the frequency of the tick count.

NOTE 1: This function will disable interrupts for its duration. It is not intended for normal application runtime use but as a debug aid.

Setting configGENERATE_RUN_TIME_STATS to 1 will result in a total accumulated execution time being stored for each task. The resolution of the accumulated time value depends on the frequency of the timer configured by the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() macro. Calling vTaskGetRunTimeStats() writes the total execution time of each task into a buffer, both as an absolute count value and as a percentage of the total system execution time.

NOTE 2:

This function is provided for convenience only, and is used by many of the demo applications. Do not consider it to be part of the scheduler.

vTaskGetRunTimeStats() calls uxTaskGetSystemState(), then formats part of the uxTaskGetSystemState() output into a human readable table that displays the amount of time each task has spent in the Running state in both absolute and percentage terms.

vTaskGetRunTimeStats() has a dependency on the sprintf() C library function that might bloat the code size, use a lot of stack, and provide different results on different platforms. An alternative, tiny, third party, and limited functionality implementation of sprintf() is provided in many of the FreeRTOS/Demo sub-directories in a file called printf-stdarg.c (note printf-stdarg.c does not provide a full snprintf() implementation!).

It is recommended that production systems call uxTaskGetSystemState() directly to get access to raw stats data, rather than indirectly through a call to vTaskGetRunTimeStats().

Parameters **`pcWriteBuffer`** -- A buffer into which the execution times will be written, in ASCII form. This buffer is assumed to be large enough to contain the generated report. Approximately 40 bytes per task should be sufficient.

configRUN_TIME_COUNTER_TYPE **`ulTaskGetIdleRunTimeCounter`**(void)

configGENERATE_RUN_TIME_STATS, configUSE_STATS_FORMATTING_FUNCTIONS and INCLUDE_xTaskGetIdleTaskHandle must all be defined as 1 for these functions to be available. The application must also then provide definitions for portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() to configure a peripheral timer/counter and return the timers current count value respectively. The counter should be at least 10 times the frequency of the tick count.

Setting configGENERATE_RUN_TIME_STATS to 1 will result in a total accumulated execution time being stored for each task. The resolution of the accumulated time value depends on the frequency of the timer configured by the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() macro. While uxTaskGetSystemState() and vTaskGetRunTimeStats() writes the total execution time of each task into a buffer, ulTaskGetI-

dleRunTimeCounter() returns the total execution time of just the idle task and ulTaskGetIdleRunTimePercent() returns the percentage of the CPU time used by just the idle task.

Note the amount of idle time is only a good measure of the slack time in a system if there are no other tasks executing at the idle priority, tickless idle is not used, and configIDLE_SHOULD_YIELD is set to 0.

Note: If configNUMBER_OF_CORES > 1, calling this function will query the idle task of the current core.

Returns The total run time of the idle task or the percentage of the total run time consumed by the idle task. This is the amount of time the idle task has actually been executing. The unit of time is dependent on the frequency configured using the portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() and portGET_RUN_TIME_COUNTER_VALUE() macros.

configRUN_TIME_COUNTER_TYPE **ulTaskGetIdleRunTimePercent**(void)

BaseType_t **xTaskGenericNotifyWait**(UBaseType_t uxIndexToWaitOn, uint32_t ulBitsToClearOnEntry, uint32_t ulBitsToClearOnExit, uint32_t *pulNotificationValue, TickType_t xTicksToWait)

Waits for a direct to task notification to be pending at a given index within an array of direct to task notifications.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this function to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

A task can use xTaskNotifyWaitIndexed() to [optionally] block to wait for a notification to be pending, or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification value to have a non-zero value. The task does not consume any CPU time while it is in the Blocked state.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyWait() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotifyWait() is equivalent to calling xTaskNotifyWaitIndexed() with the uxIndexToWaitOn parameter set to 0.

Parameters

- **uxIndexToWaitOn** -- The index within the calling task's array of notification values on which the calling task will wait for a notification to be received. uxIndexToWaitOn must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyWait() does not have this parameter and always waits for notifications on index 0.

- **ulBitsToClearOnEntry** -- Bits that are set in ulBitsToClearOnEntry value will be cleared in the calling task's notification value before the task checks to see if any notifications are pending, and optionally blocks if no notifications are pending. Setting ulBitsToClearOnEntry to ULONG_MAX (if limits.h is included) or 0xffffffffUL (if limits.h is not included) will have the effect of resetting the task's notification value to 0. Setting ulBitsToClearOnEntry to 0 will leave the task's notification value unchanged.
- **ulBitsToClearOnExit** -- If a notification is pending or received before the calling task exits the xTaskNotifyWait() function then the task's notification value (see the xTaskNotify() API function) is passed out using the pulNotificationValue parameter. Then any bits that are set in ulBitsToClearOnExit will be cleared in the task's notification value (note *pulNotificationValue is set before any bits are cleared). Setting ulBitsToClearOnExit to ULONG_MAX (if limits.h is included) or 0xffffffffUL (if limits.h is not included) will have the effect of resetting the task's notification value to 0 before the function exits. Setting ulBitsToClearOnExit to 0 will leave the task's notification value unchanged when the function exits (in which case the value passed out in pulNotificationValue will match the task's notification value).
- **pulNotificationValue** -- Used to pass the task's notification value out of the function. Note the value passed out will not be effected by the clearing of any bits caused by ulBitsToClearOnExit being non-zero.
- **xTicksToWait** -- The maximum amount of time that the task should wait in the Blocked state for a notification to be received, should a notification not already be pending when xTaskNotifyWait() was called. The task will not consume any processing time while it is in the Blocked state. This is specified in kernel ticks, the macro pdMS_TO_TICKS(value_in_ms) can be used to convert a time specified in milliseconds to a time specified in ticks.

Returns If a notification was received (including notifications that were already pending when xTaskNotifyWait was called) then pdPASS is returned. Otherwise pdFAIL is returned.

void **vTaskGenericNotifyGiveFromISR**(*TaskHandle_t* xTaskToNotify, UBaseType_t uxIndexToNotify, BaseType_t *pxHigherPriorityTaskWoken)

A version of xTaskNotifyGiveIndexed() that can be called from an interrupt service routine (ISR).

See https://www.FreeRTOS.org/RTOS-task-notifications.html for more details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this macro to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

vTaskNotifyGiveIndexedFromISR() is intended for use when task notifications are used as light weight and faster binary or counting semaphore equivalents. Actual FreeRTOS semaphores are given from an ISR using the xSemaphoreGiveFromISR() API function, the equivalent action that instead uses a task notification is vTaskNotifyGiveIndexedFromISR().

When task notifications are being used as a binary or counting semaphore equivalent then the task being notified should wait for the notification using the ulTaskNotifyTakeIndexed() API function rather than the xTaskNotifyWaitIndexed() API function.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an

array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyFromISR() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling xTaskNotifyGiveFromISR() is equivalent to calling xTaskNotifyGiveIndexedFromISR() with the uxIndexToNotify parameter set to 0.

Parameters

- **`xTaskToNotify`** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **`uxIndexToNotify`** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyGiveFromISR() does not have this parameter and always sends notifications to index 0.
- **`pxHigherPriorityTaskWoken`** -- vTaskNotifyGiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending the notification caused the task to which the notification was sent to leave the Blocked state, and the unblocked task has a priority higher than the currently running task. If vTaskNotifyGiveFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited. How a context switch is requested from an ISR is dependent on the port - see the documentation page for the port in use.

BaseType_t **`xTaskGenericNotifyStateClear`**(*TaskHandle_t* xTask, UBaseType_t uxIndexToClear)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

If a notification is sent to an index within the array of notifications then the notification at that index is said to be 'pending' until it is read or explicitly cleared by the receiving task. xTaskNotifyStateClearIndexed() is the function that clears a pending notification without reading the notification value. The notification value at the same array index is not altered. Set xTask to NULL to clear the notification state of the calling task.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyStateClear() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling xTaskNotifyStateClear() is equivalent to calling xTaskNotifyStateClearIndexed() with the uxIndexToNotify parameter set to 0.

Parameters

- **`xTask`** -- The handle of the RTOS task that will have a notification state cleared. Set xTask to NULL to clear a notification state in the calling task. To obtain a task's handle create the task using xTaskCreate() and make use of the pxCreatedTask parameter, or create the task using xTaskCreateStatic() and store the returned value, or use the task's name in a call to xTaskGetHandle().
- **`uxIndexToClear`** -- The index within the target task's array of notification values to act upon. For example, setting uxIndexToClear to 1 will clear the state of the notification at index 1 within the array. uxIndexToClear must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. ulTaskNotifyStateClear() does not have this parameter and always acts on the notification at index 0.

Returns pdTRUE if the task's notification state was set to eNotWaitingNotification, otherwise pdFALSE.

uint32_t **`ulTaskGenericNotifyValueClear`**(*TaskHandle_t* xTask, UBaseType_t uxIndexToClear, uint32_t ulBitsToClear)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

ulTaskNotifyValueClearIndexed() clears the bits specified by the ulBitsToClear bit mask in the notification value at array index uxIndexToClear of the task referenced by xTask.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. ulTaskNotifyValueClear() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling ulTaskNotifyValueClear() is equivalent to calling ulTaskNotifyValueClearIndexed() with the uxIndexToClear parameter set to 0.

Parameters

- **xTask** -- The handle of the RTOS task that will have bits in one of its notification values cleared. Set xTask to NULL to clear bits in a notification value of the calling task. To obtain a task's handle create the task using xTaskCreate() and make use of the pxCreatedTask parameter, or create the task using xTaskCreateStatic() and store the returned value, or use the task's name in a call to xTaskGetHandle().
- **uxIndexToClear** -- The index within the target task's array of notification values in which to clear the bits. uxIndexToClear must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. ulTaskNotifyValueClear() does not have this parameter and always clears bits in the notification value at index 0.
- **ulBitsToClear** -- Bit mask of the bits to clear in the notification value of xTask. Set a bit to 1 to clear the corresponding bits in the task's notification value. Set ulBitsToClear to 0xffffffff (UINT_MAX on 32-bit architectures) to clear the notification value to 0. Set ulBitsToClear to 0 to query the task's notification value without clearing any bits.

Returns The value of the target task's notification value before the bits specified by ulBitsToClear were cleared.

void **vTaskSetTimeOutState**(TimeOut_t *const pxTimeOut)

Capture the current time for future use with xTaskCheckForTimeOut().

Parameters **pxTimeOut** -- Pointer to a timeout object into which the current time is to be captured. The captured time includes the tick count and the number of times the tick count has overflowed since the system first booted.

BaseType_t **xTaskCheckForTimeOut**(TimeOut_t *const pxTimeOut, TickType_t *const pxTicksToWait)

Determines if pxTicksToWait ticks has passed since a time was captured using a call to vTaskSetTimeOutState(). The captured time includes the tick count and the number of times the tick count has overflowed.

Example Usage:

```
// Driver library function used to receive uxWantedBytes from an Rx buffer
// that is filled by a UART interrupt. If there are not enough bytes in the
// Rx buffer then the task enters the Blocked state until it is notified that
// more data has been placed into the buffer. If there is still not enough
// data then the task re-enters the Blocked state, and xTaskCheckForTimeOut()
// is used to re-calculate the Block time to ensure the total amount of time
// spent in the Blocked state does not exceed MAX_TIME_TO_WAIT. This
// continues until either the buffer contains at least uxWantedBytes bytes,
// or the total amount of time spent in the Blocked state reaches
// MAX_TIME_TO_WAIT - at which point the task reads however many bytes are
// available up to a maximum of uxWantedBytes.

size_t xUART_Receive( uint8_t *pucBuffer, size_t uxWantedBytes )
{
size_t uxReceived = 0;
```

(continues on next page)

(continued from previous page)

```
TickType_t xTicksToWait = MAX_TIME_TO_WAIT;
TimeOut_t xTimeOut;

// Initialize xTimeOut.  This records the time at which this function
// was entered.
    vTaskSetTimeOutState( &xTimeOut );

// Loop until the buffer contains the wanted number of bytes, or a
// timeout occurs.
while( UART_bytes_in_rx_buffer( pxUARTInstance ) < uxWantedBytes )
    {
// The buffer didn't contain enough data so this task is going to
// enter the Blocked state. Adjusting xTicksToWait to account for
// any time that has been spent in the Blocked state within this
// function so far to ensure the total amount of time spent in the
// Blocked state does not exceed MAX_TIME_TO_WAIT.
if( xTaskCheckForTimeOut( &xTimeOut, &xTicksToWait ) != pdFALSE )
        {
//Timed out before the wanted number of bytes were available,
// exit the loop.
break;
        }

// Wait for a maximum of xTicksToWait ticks to be notified that the
// receive interrupt has placed more data into the buffer.
        ulTaskNotifyTake( pdTRUE, xTicksToWait );
    }

// Attempt to read uxWantedBytes from the receive buffer into pucBuffer.
// The actual number of bytes read (which might be less than
// uxWantedBytes) is returned.
    uxReceived = UART_read_from_receive_buffer( pxUARTInstance,
                                                pucBuffer,
                                                uxWantedBytes );

return uxReceived;
}
```

See also:

https://www.FreeRTOS.org/xTaskCheckForTimeOut.html

Parameters

- **`pxTimeOut`** -- The time status as captured previously using vTaskSetTimeOutState. If the timeout has not yet occurred, it is updated to reflect the current time status.
- **`pxTicksToWait`** -- The number of ticks to check for timeout i.e. if pxTicksToWait ticks have passed since pxTimeOut was last updated (either by vTaskSetTimeOutState() or xTaskCheckForTimeOut()), the timeout has occurred. If the timeout has not occurred, pxTicksToWait is updated to reflect the number of remaining ticks.

Returns If timeout has occurred, pdTRUE is returned. Otherwise pdFALSE is returned and pxTicksToWait is updated to reflect the number of remaining ticks.

BaseType_t **`xTaskCatchUpTicks`** (TickType_t xTicksToCatchUp)

This function corrects the tick count value after the application code has held interrupts disabled for an extended period resulting in tick interrupts having been missed.

This function is similar to vTaskStepTick(), however, unlike vTaskStepTick(), xTaskCatchUpTicks() may move the tick count forward past a time at which a task should be removed from the blocked state. That means tasks may have to be removed from the blocked state as the tick count is moved.

Parameters **`xTicksToCatchUp`** -- The number of tick interrupts that have been missed due

to interrupts being disabled. Its value is not computed automatically, so must be computed by the application writer.

Returns pdTRUE if moving the tick count forward resulted in a task leaving the blocked state and a context switch being performed. Otherwise pdFALSE.

Structures

struct **xTASK_STATUS**

Used with the uxTaskGetSystemState() function to return the state of each task in the system.

Public Members

TaskHandle_t **xHandle**

The handle of the task to which the rest of the information in the structure relates.

const char ***pcTaskName**

A pointer to the task's name. This value will be invalid if the task was deleted since the structure was populated!

UBaseType_t **xTaskNumber**

A number unique to the task.

eTaskState **eCurrentState**

The state in which the task existed when the structure was populated.

UBaseType_t **uxCurrentPriority**

The priority at which the task was running (may be inherited) when the structure was populated.

UBaseType_t **uxBasePriority**

The priority to which the task will return if the task's current priority has been inherited to avoid unbounded priority inversion when obtaining a mutex. Only valid if configUSE_MUTEXES is defined as 1 in FreeRTOSConfig.h.

configRUN_TIME_COUNTER_TYPE **ulRunTimeCounter**

The total run time allocated to the task so far, as defined by the run time stats clock. See https://www.FreeRTOS.org/rtos-run-time-stats.html. Only valid when configGENERATE_RUN_TIME_STATS is defined as 1 in FreeRTOSConfig.h.

StackType_t ***pxStackBase**

Points to the lowest address of the task's stack area.

configSTACK_DEPTH_TYPE **usStackHighWaterMark**

The minimum amount of stack space that has remained for the task since the task was created. The closer this value is to zero the closer the task has come to overflowing its stack.

BaseType_t **xCoreID**

Core this task is pinned to (0, 1, or tskNO_AFFINITY). If configNUMBER_OF_CORES == 1, this will always be 0.

Macros

tskIDLE_PRIORITY

Defines the priority used by the idle task. This must not be modified.

tskNO_AFFINITY

Macro representing and unpinned (i.e., "no affinity") task in xCoreID parameters

taskVALID_CORE_ID(xCoreID)

Macro to check if an xCoreID value is valid

Returns pdTRUE if valid, pdFALSE otherwise.

taskYIELD()

Macro for forcing a context switch.

taskENTER_CRITICAL(x)

Macro to mark the start of a critical code region. Preemptive context switches cannot occur when in a critical region.

NOTE: This may alter the stack (depending on the portable implementation) so must be used with care!

taskENTER_CRITICAL_FROM_ISR()

taskENTER_CRITICAL_ISR(x)

taskEXIT_CRITICAL(x)

Macro to mark the end of a critical code region. Preemptive context switches cannot occur when in a critical region.

NOTE: This may alter the stack (depending on the portable implementation) so must be used with care!

taskEXIT_CRITICAL_FROM_ISR(x)

taskEXIT_CRITICAL_ISR(x)

taskDISABLE_INTERRUPTS()

Macro to disable all maskable interrupts.

taskENABLE_INTERRUPTS()

Macro to enable microcontroller interrupts.

taskSCHEDULER_SUSPENDED

Definitions returned by xTaskGetSchedulerState(). taskSCHEDULER_SUSPENDED is 0 to generate more optimal code when configASSERT() is defined as the constant is used in assert() statements.

taskSCHEDULER_NOT_STARTED

taskSCHEDULER_RUNNING

xTaskNotifyIndexed(xTaskToNotify, uxIndexToNotify, ulValue, eAction)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

Sends a direct to task notification to a task, with an optional value and action.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A task can use xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification to be pending. The task does not consume any CPU time while it is in the Blocked state.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotify() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotify() is equivalent to calling xTaskNotifyIndexed() with the uxIndexToNotify parameter set to 0.

eSetBits - The target notification value is bitwise ORed with ulValue. xTaskNotifyIndexed() always returns pdPASS in this case.

eIncrement - The target notification value is incremented. ulValue is not used and xTaskNotifyIndexed() always returns pdPASS in this case.

eSetValueWithOverwrite - The target notification value is set to the value of ulValue, even if the task being notified had not yet processed the previous notification at the same array index (the task already had a notification pending at that index). xTaskNotifyIndexed() always returns pdPASS in this case.

eSetValueWithoutOverwrite - If the task being notified did not already have a notification pending at the same array index then the target notification value is set to ulValue and xTaskNotifyIndexed() will return pdPASS. If the task being notified already had a notification pending at the same array index then no action is performed and pdFAIL is returned.

eNoAction - The task receives a notification at the specified array index without the notification value at that index being updated. ulValue is not used and xTaskNotifyIndexed() always returns pdPASS in this case.

pulPreviousNotificationValue - Can be used to pass out the subject task's notification value before any bits are modified by the notify function.

Parameters

- **`xTaskToNotify`** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **`uxIndexToNotify`** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotify() does not have this parameter and always sends notifications to index 0.
- **`ulValue`** -- Data that can be sent with the notification. How the data is used depends on the value of the eAction parameter.
- **`eAction`** -- Specifies how the notification updates the task's notification value, if at all. Valid values for eAction are as follows:

Returns Dependent on the value of eAction. See the description of the eAction parameter.

`xTaskNotifyAndQueryIndexed`(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pulPreviousNotifyValue)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

xTaskNotifyAndQueryIndexed() performs the same operation as xTaskNotifyIndexed() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than when the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyAndQuery() performs the same operation as xTaskNotify() with the addition that it also returns the subject task's prior notification value (the notification value as it was at the time the function is called, rather than when the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyIndexedFromISR(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pxHigherPriorityTaskWoken)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these functions to be available.

A version of xTaskNotifyIndexed() that can be used from an interrupt service routine (ISR).

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

A task can use xTaskNotifyWaitIndexed() to [optionally] block to wait for a notification to be pending, or ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification value to have a non-zero value. The task does not consume any CPU time while it is in the Blocked state.

A notification sent to a task will remain pending until it is cleared by the task calling xTaskNotifyWaitIndexed() or ulTaskNotifyTakeIndexed() (or their un-indexed equivalents). If the task was already in the Blocked state to wait for a notification when the notification arrives then the task will automatically be removed from the Blocked state (unblocked) and the notification cleared.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyFromISR() is the original API function, and remains backward compatible by always operating on the notification value at index 0 within the array. Calling xTaskNotifyFromISR() is equivalent to calling xTaskNotifyIndexedFromISR() with the uxIndexToNotify parameter set to 0.

eSetBits - The task's notification value is bitwise ORed with ulValue. xTaskNotify() always returns pdPASS in this case.

eIncrement - The task's notification value is incremented. ulValue is not used and xTaskNotify() always returns pdPASS in this case.

eSetValueWithOverwrite - The task's notification value is set to the value of ulValue, even if the task being notified had not yet processed the previous notification (the task already had a notification pending). xTaskNotify() always returns pdPASS in this case.

eSetValueWithoutOverwrite - If the task being notified did not already have a notification pending then the task's notification value is set to ulValue and xTaskNotify() will return pdPASS. If the task being notified already had a notification pending then no action is performed and pdFAIL is returned.

eNoAction - The task receives a notification without its notification value being updated. ulValue is not used and xTaskNotify() always returns pdPASS in this case.

Parameters

- **uxIndexToNotify** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyFromISR() does not have this parameter and always sends notifications to index 0.
- **xTaskToNotify** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **ulValue** -- Data that can be sent with the notification. How the data is used depends on the value of the eAction parameter.
- **eAction** -- Specifies how the notification updates the task's notification value, if at all. Valid values for eAction are as follows:
- **pxHigherPriorityTaskWoken** -- xTaskNotifyFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending the notification caused the task to which the notification was sent to leave the Blocked state, and the unblocked task has a priority higher than the currently running task. If xTaskNotifyFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited. How a context switch is requested from an ISR is dependent on the port - see the documentation page for the port in use.

Returns Dependent on the value of eAction. See the description of the eAction parameter.

xTaskNotifyAndQueryIndexedFromISR(xTaskToNotify, uxIndexToNotify, ulValue, eAction, pulPreviousNotificationValue, pxHigherPriorityTaskWoken)

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

xTaskNotifyAndQueryIndexedFromISR() performs the same operation as xTaskNotifyIndexedFromISR() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than at the time the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyAndQueryFromISR() performs the same operation as xTaskNotifyFromISR() with the addition that it also returns the subject task's prior notification value (the notification value at the time the function is called rather than at the time the function returns) in the additional pulPreviousNotifyValue parameter.

xTaskNotifyWait(ulBitsToClearOnEntry, ulBitsToClearOnExit, pulNotificationValue, xTicksToWait)

xTaskNotifyWaitIndexed(uxIndexToWaitOn, ulBitsToClearOnEntry, ulBitsToClearOnExit, pulNotificationValue, xTicksToWait)

xTaskNotifyGiveIndexed(xTaskToNotify, uxIndexToNotify)

Sends a direct to task notification to a particular index in the target task's notification array in a manner similar to giving a counting semaphore.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for more details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for these macros to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

xTaskNotifyGiveIndexed() is a helper macro intended for use when task notifications are used as light weight and faster binary or counting semaphore equivalents. Actual FreeRTOS semaphores are given using the xSemaphoreGive() API function, the equivalent action that instead uses a task notification is xTaskNotifyGiveIndexed().

When task notifications are being used as a binary or counting semaphore equivalent then the task being notified should wait for the notification using the ulTaskNotifyTakeIndexed() API function rather than the xTaskNotifyWaitIndexed() API function.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. xTaskNotifyGive() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling xTaskNotifyGive() is equivalent to calling xTaskNotifyGiveIndexed() with the uxIndexToNotify parameter set to 0.

Parameters

- **`xTaskToNotify`** -- The handle of the task being notified. The handle to a task can be returned from the xTaskCreate() API function used to create the task, and the handle of the currently running task can be obtained by calling xTaskGetCurrentTaskHandle().
- **`uxIndexToNotify`** -- The index within the target task's array of notification values to which the notification is to be sent. uxIndexToNotify must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyGive() does not have this parameter and always sends notifications to index 0.

Returns xTaskNotifyGive() is a macro that calls xTaskNotify() with the eAction parameter set to eIncrement - so pdPASS is always returned.

`vTaskNotifyGiveFromISR` (xTaskToNotify, pxHigherPriorityTaskWoken)

`vTaskNotifyGiveIndexedFromISR` (xTaskToNotify, uxIndexToNotify, pxHigherPriorityTaskWoken)

`ulTaskNotifyTakeIndexed` (uxIndexToWaitOn, xClearCountOnExit, xTicksToWait)

Waits for a direct to task notification on a particular index in the calling task's notification array in a manner similar to taking a counting semaphore.

See https://www.FreeRTOS.org/RTOS-task-notifications.html for details.

configUSE_TASK_NOTIFICATIONS must be undefined or defined as 1 for this function to be available.

Each task has a private array of "notification values" (or 'notifications'), each of which is a 32-bit unsigned integer (uint32_t). The constant configTASK_NOTIFICATION_ARRAY_ENTRIES sets the number of indexes in the array, and (for backward compatibility) defaults to 1 if left undefined. Prior to FreeRTOS V10.4.0 there was only one notification value per task.

Events can be sent to a task using an intermediary object. Examples of such objects are queues, semaphores, mutexes and event groups. Task notifications are a method of sending an event directly to a task without the need for such an intermediary object.

A notification sent to a task can optionally perform an action, such as update, overwrite or increment one of the task's notification values. In that way task notifications can be used to send data to a task, or be used as light weight and fast binary or counting semaphores.

ulTaskNotifyTakeIndexed() is intended for use when a task notification is used as a faster and lighter weight binary or counting semaphore alternative. Actual FreeRTOS semaphores are taken using the xSemaphoreTake() API function, the equivalent action that instead uses a task notification is ulTaskNotifyTakeIndexed().

When a task is using its notification value as a binary or counting semaphore other tasks should send notifications to it using the xTaskNotifyGiveIndexed() macro, or xTaskNotifyIndex() function with the eAction parameter set to eIncrement.

ulTaskNotifyTakeIndexed() can either clear the task's notification value at the array index specified by the uxIndexToWaitOn parameter to zero on exit, in which case the notification value acts like a binary semaphore, or decrement the notification value on exit, in which case the notification value acts like a counting semaphore.

A task can use ulTaskNotifyTakeIndexed() to [optionally] block to wait for a notification. The task does not consume any CPU time while it is in the Blocked state.

Where as xTaskNotifyWaitIndexed() will return when a notification is pending, ulTaskNotifyTakeIndexed() will return when the task's notification value is not zero.

NOTE Each notification within the array operates independently - a task can only block on one notification within the array at a time and will not be unblocked by a notification sent to any other array index.

Backward compatibility information: Prior to FreeRTOS V10.4.0 each task had a single "notification value", and all task notification API functions operated on that value. Replacing the single notification value with an array of notification values necessitated a new set of API functions that could address specific notifications within the array. ulTaskNotifyTake() is the original API function, and remains backward compatible by always operating on the notification value at index 0 in the array. Calling ulTaskNotifyTake() is equivalent to calling ulTaskNotifyTakeIndexed() with the uxIndexToWaitOn parameter set to 0.

Parameters

- **`uxIndexToWaitOn`** -- The index within the calling task's array of notification values on which the calling task will wait for a notification to be non-zero. uxIndexToWaitOn must be less than configTASK_NOTIFICATION_ARRAY_ENTRIES. xTaskNotifyTake() does not have this parameter and always waits for notifications on index 0.
- **`xClearCountOnExit`** -- if xClearCountOnExit is pdFALSE then the task's notification value is decremented when the function exits. In this way the notification value acts like a counting semaphore. If xClearCountOnExit is not pdFALSE then the task's notification value is cleared to zero when the function exits. In this way the notification value acts like a binary semaphore.
- **`xTicksToWait`** -- The maximum amount of time that the task should wait in the Blocked state for the task's notification value to be greater than zero, should the count not already be greater than zero when ulTaskNotifyTake() was called. The task will not consume any processing time while it is in the Blocked state. This is specified in kernel ticks, the macro pdMS_TO_TICKS(value_in_ms) can be used to convert a time specified in milliseconds to a time specified in ticks.

Returns The task's notification count before it is either cleared to zero or decremented (see the xClearCountOnExit parameter).

`xTaskNotifyStateClear`(xTask)

`xTaskNotifyStateClearIndexed`(xTask, uxIndexToClear)

`ulTaskNotifyValueClear`(xTask, ulBitsToClear)

`ulTaskNotifyValueClearIndexed`(xTask, uxIndexToClear, ulBitsToClear)

Type Definitions

typedef struct tskTaskControlBlock ***`TaskHandle_t`**

typedef BaseType_t (***`TaskHookFunction_t`**)(void*)

Defines the prototype to which the application task hook function must conform.

typedef struct *xTASK_STATUS* **`TaskStatus_t`**

Used with the uxTaskGetSystemState() function to return the state of each task in the system.

Enumerations

enum **eTaskState**

Task states returned by eTaskGetState.

Values:

enumerator **eRunning**

A task is querying the state of itself, so must be running.

enumerator **eReady**

The task being queried is in a ready or pending ready list.

enumerator **eBlocked**

The task being queried is in the Blocked state.

enumerator **eSuspended**

The task being queried is in the Suspended state, or is in the Blocked state with an infinite time out.

enumerator **eDeleted**

The task being queried has been deleted, but its TCB has not yet been freed.

enumerator **eInvalid**

Used as an 'invalid state' value.

enum **eNotifyAction**

Actions that can be performed when vTaskNotify() is called.

Values:

enumerator **eNoAction**

Notify the task without updating its notify value.

enumerator **eSetBits**

Set bits in the task's notification value.

enumerator **eIncrement**

Increment the task's notification value.

enumerator **eSetValueWithOverwrite**

Set the task's notification value to a specific value even if the previous value has not yet been read by the task.

enumerator **eSetValueWithoutOverwrite**

Set the task's notification value if the previous value has been read by the task.

enum **eSleepModeStatus**

Possible return values for eTaskConfirmSleepModeStatus().

Values:

enumerator **eAbortSleep**

A task has been made ready or a context switch pended since portSUPPRESS_TICKS_AND_SLEEP() was called - abort entering a sleep mode.

enumerator **eStandardSleep**

Enter a sleep mode that will not last any longer than the expected idle time.

Queue API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/queue.h
- This header file can be included with:

```
#include "freertos/queue.h"
```

Functions

BaseType_t **xQueueGenericSend**(*QueueHandle_t* xQueue, const void *const pvItemToQueue, TickType_t xTicksToWait, const BaseType_t xCopyPosition)

It is preferred that the macros xQueueSend(), xQueueSendToFront() and xQueueSendToBack() are used in place of calling this function directly.

Post an item on a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueGenericSend( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10,␣
↪queueSEND_TO_BACK ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
```

(continues on next page)

(continued from previous page)

```
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueGenericSend( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0,␣
↪queueSEND_TO_BACK );
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.
- **xCopyPosition** -- Can take the value queueSEND_TO_BACK to place the item at the back of the queue, or queueSEND_TO_FRONT to place the item at the front of the queue (for high priority messages).

Returns pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

BaseType_t **xQueuePeek**(*QueueHandle_t* xQueue, void *const pvBuffer, TickType_t xTicksToWait)

Receive an item from a queue without removing the item from the queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items remain on the queue so will be returned again by the next call, or a call to xQueueReceive().

This macro must not be used in an interrupt service routine. See xQueuePeekFromISR() for an alternative that can be called from an interrupt service routine.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

QueueHandle_t xQueue;

// Task to create a queue and post a value.
void vATask( void *pvParameters )
{
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
```

(continues on next page)

(continued from previous page)

```
 }

 // ...

 // Send a pointer to a struct AMessage object.  Don't block if the
 // queue is already full.
 pxMessage = & xMessage;
 xQueueSend( xQueue, ( void * ) &pxMessage, ( TickType_t ) 0 );

 // ... Rest of task code.
}

// Task to peek the data from the queue.
void vADifferentTask( void *pvParameters )
{
struct AMessage *pxRxedMessage;

 if( xQueue != 0 )
 {
// Peek a message on the created queue.  Block for 10 ticks if a
// message is not immediately available.
if( xQueuePeek( xQueue, &( pxRxedMessage ), ( TickType_t ) 10 ) )
     {
// pcRxedMessage now points to the struct AMessage variable posted
// by vATask, but the item still remains on the queue.
     }
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for an item to receive should the queue be empty at the time of the call. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required. xQueuePeek() will return immediately if xTicksToWait is 0 and the queue is empty.

Returns pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueuePeekFromISR**(*QueueHandle_t* xQueue, void *const pvBuffer)

A version of xQueuePeek() that can be called from an interrupt service routine (ISR).

Receive an item from a queue without removing the item from the queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items remain on the queue so will be returned again by the next call, or a call to xQueueReceive().

Parameters

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.

Returns pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueueReceive**(*QueueHandle_t* xQueue, void *const pvBuffer, TickType_t xTicksToWait)

Receive an item from a queue. The item is received by copy so a buffer of adequate size must be provided. The number of bytes copied into the buffer was defined when the queue was created.

Successfully received items are removed from the queue.

This function must not be used in an interrupt service routine. See xQueueReceiveFromISR for an alternative that can.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

QueueHandle_t xQueue;

// Task to create a queue and post a value.
void vATask( void *pvParameters )
{
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
 }

 // ...

 // Send a pointer to a struct AMessage object.  Don't block if the
 // queue is already full.
 pxMessage = & xMessage;
 xQueueSend( xQueue, ( void * ) &pxMessage, ( TickType_t ) 0 );

 // ... Rest of task code.
}

// Task to receive from the queue.
void vADifferentTask( void *pvParameters )
{
struct AMessage *pxRxedMessage;

 if( xQueue != 0 )
 {
// Receive a message on the created queue.  Block for 10 ticks if a
// message is not immediately available.
if( xQueueReceive( xQueue, &( pxRxedMessage ), ( TickType_t ) 10 ) )
     {
// pcRxedMessage now points to the struct AMessage variable posted
// by vATask.
     }
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for an item to receive should the queue be empty at the time of the call. xQueueReceive() will return immediately if xTicksToWait is zero and the queue is empty. The time is defined

in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

UBaseType_t **uxQueueMessagesWaiting**(const *QueueHandle_t* xQueue)

Return the number of messages stored in a queue.

Parameters **xQueue** -- A handle to the queue being queried.
Returns The number of messages available in the queue.

UBaseType_t **uxQueueSpacesAvailable**(const *QueueHandle_t* xQueue)

Return the number of free spaces available in a queue. This is equal to the number of items that can be sent to the queue before the queue becomes full if no items are removed.

Parameters **xQueue** -- A handle to the queue being queried.
Returns The number of spaces available in the queue.

void **vQueueDelete**(*QueueHandle_t* xQueue)

Delete a queue - freeing all the memory allocated for storing of items placed on the queue.

Parameters **xQueue** -- A handle to the queue to be deleted.

BaseType_t **xQueueGenericSendFromISR**(*QueueHandle_t* xQueue, const void *const pvItemToQueue, BaseType_t *const pxHigherPriorityTaskWoken, const BaseType_t xCopyPosition)

It is preferred that the macros xQueueSendFromISR(), xQueueSendToFrontFromISR() and xQueueSendToBackFromISR() be used in place of calling this function directly. xQueueGiveFromISR() is an equivalent for use by semaphores that don't actually copy any data.

Post an item on a queue. It is safe to use this function from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWokenByPost;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWokenByPost = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post each byte.
     xQueueGenericSendFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWokenByPost,
↪ queueSEND_TO_BACK );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.  Note that the
 // name of the yield function required is port specific.
 if( xHigherPriorityTaskWokenByPost )
 {
     portYIELD_FROM_ISR();
```

(continues on next page)

(continued from previous page)

```
 }
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueGenericSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueGenericSendFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.
- **xCopyPosition** -- Can take the value queueSEND_TO_BACK to place the item at the back of the queue, or queueSEND_TO_FRONT to place the item at the front of the queue (for high priority messages).

Returns pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

BaseType_t **xQueueGiveFromISR**(*QueueHandle_t* xQueue, BaseType_t *const pxHigherPriorityTaskWoken)

BaseType_t **xQueueReceiveFromISR**(*QueueHandle_t* xQueue, void *const pvBuffer, BaseType_t *const pxHigherPriorityTaskWoken)

Receive an item from a queue. It is safe to use this function from within an interrupt service routine.

Example usage:

```
QueueHandle_t xQueue;

// Function to create a queue and post some values.
void vAFunction( void *pvParameters )
{
char cValueToPost;
const TickType_t xTicksToWait = ( TickType_t )0xff;

 // Create a queue capable of containing 10 characters.
 xQueue = xQueueCreate( 10, sizeof( char ) );
 if( xQueue == 0 )
 {
// Failed to create the queue.
 }

 // ...

 // Post some characters that will be used within an ISR.  If the queue
 // is full then this task will block for xTicksToWait ticks.
 cValueToPost = 'a';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );
 cValueToPost = 'b';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );

 // ... keep posting characters ... this task may block when the queue
 // becomes full.

 cValueToPost = 'c';
 xQueueSend( xQueue, ( void * ) &cValueToPost, xTicksToWait );
}

// ISR that outputs all the characters received on the queue.
```

(continues on next page)

(continued from previous page)

```
void vISR_Routine( void )
{
BaseType_t xTaskWokenByReceive = pdFALSE;
char cRxedChar;

 while( xQueueReceiveFromISR( xQueue, ( void * ) &cRxedChar, &
↪xTaskWokenByReceive) )
 {
// A character was received.  Output the character now.
     vOutputCharacter( cRxedChar );

// If removing the character from the queue woke the task that was
// posting onto the queue xTaskWokenByReceive will have been set to
// pdTRUE.  No matter how many times this loop iterates only one
// task will be woken.
 }

 if( xTaskWokenByReceive != ( char ) pdFALSE;
 {
     taskYIELD ();
 }
}
```

Parameters
- **xQueue** -- The handle to the queue from which the item is to be received.
- **pvBuffer** -- Pointer to the buffer into which the received item will be copied.
- **pxHigherPriorityTaskWoken** -- A task may be blocked waiting for space to become available on the queue. If xQueueReceiveFromISR causes such a task to unblock *pxTaskWoken will get set to pdTRUE, otherwise *pxTaskWoken will remain unchanged.

Returns pdTRUE if an item was successfully received from the queue, otherwise pdFALSE.

BaseType_t **xQueueIsQueueEmptyFromISR** (const *QueueHandle_t* xQueue)

Queries a queue to determine if the queue is empty. This function should only be used in an ISR.

Parameters **xQueue** -- The handle of the queue being queried
Returns pdFALSE if the queue is not empty, or pdTRUE if the queue is empty.

BaseType_t **xQueueIsQueueFullFromISR** (const *QueueHandle_t* xQueue)

Queries a queue to determine if the queue is full. This function should only be used in an ISR.

Parameters **xQueue** -- The handle of the queue being queried
Returns pdFALSE if the queue is not full, or pdTRUE if the queue is full.

UBaseType_t **uxQueueMessagesWaitingFromISR** (const *QueueHandle_t* xQueue)

A version of uxQueueMessagesWaiting() that can be called from an ISR. Return the number of messages stored in a queue.

Parameters **xQueue** -- A handle to the queue being queried.
Returns The number of messages available in the queue.

void **vQueueAddToRegistry** (*QueueHandle_t* xQueue, const char *pcQueueName)

The registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call vQueueAddToRegistry() add a queue, semaphore or mutex handle to the registry if you want the handle to be available to a kernel aware debugger. If you are not using a kernel aware debugger then this function can be ignored.

configQUEUE_REGISTRY_SIZE defines the maximum number of handles the registry can hold. configQUEUE_REGISTRY_SIZE must be greater than 0 within FreeRTOSConfig.h for the registry to be available. Its value does not affect the number of queues, semaphores and mutexes that can be created - just the number that the registry can hold.

If vQueueAddToRegistry is called more than once with the same xQueue parameter, the registry will store the pcQueueName parameter from the most recent call to vQueueAddToRegistry.

Parameters

- **xQueue** -- The handle of the queue being added to the registry. This is the handle returned by a call to xQueueCreate(). Semaphore and mutex handles can also be passed in here.
- **pcQueueName** -- The name to be associated with the handle. This is the name that the kernel aware debugger will display. The queue registry only stores a pointer to the string - so the string must be persistent (global or preferably in ROM/Flash), not on the stack.

void **vQueueUnregisterQueue**(*QueueHandle_t* xQueue)

The registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call vQueueAddToRegistry() add a queue, semaphore or mutex handle to the registry if you want the handle to be available to a kernel aware debugger, and vQueueUnregisterQueue() to remove the queue, semaphore or mutex from the register. If you are not using a kernel aware debugger then this function can be ignored.

Parameters **xQueue** -- The handle of the queue being removed from the registry.

const char ***pcQueueGetName**(*QueueHandle_t* xQueue)

The queue registry is provided as a means for kernel aware debuggers to locate queues, semaphores and mutexes. Call pcQueueGetName() to look up and return the name of a queue in the queue registry from the queue's handle.

Parameters **xQueue** -- The handle of the queue the name of which will be returned.

Returns If the queue is in the registry then a pointer to the name of the queue is returned. If the queue is not in the registry then NULL is returned.

QueueSetHandle_t **xQueueCreateSet**(const UBaseType_t uxEventQueueLength)

Queue sets provide a mechanism to allow a task to block (pend) on a read operation from multiple queues or semaphores simultaneously.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

A queue set must be explicitly created using a call to xQueueCreateSet() before it can be used. Once created, standard FreeRTOS queues and semaphores can be added to the set using calls to xQueueAddToSet(). xQueueSelectFromSet() is then used to determine which, if any, of the queues or semaphores contained in the set is in a state where a queue read or semaphore take operation would be successful.

Note 1: See the documentation on https://www.FreeRTOS.org/RTOS-queue-sets.html for reasons why queue sets are very rarely needed in practice as there are simpler methods of blocking on multiple objects.

Note 2: Blocking on a queue set that contains a mutex will not cause the mutex holder to inherit the priority of the blocked task.

Note 3: An additional 4 bytes of RAM is required for each space in a every queue added to a queue set. Therefore counting semaphores that have a high maximum count value should not be added to a queue set.

Note 4: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters **uxEventQueueLength** -- Queue sets store events that occur on the queues and semaphores contained in the set. uxEventQueueLength specifies the maximum number of events that can be queued at once. To be absolutely certain that events are not lost uxEventQueueLength should be set to the total sum of the length of the queues added to the set, where binary semaphores and mutexes have a length of 1, and counting semaphores have a length set by their maximum count value. Examples:

- If a queue set is to hold a queue of length 5, another queue of length 12, and a binary semaphore, then uxEventQueueLength should be set to (5 + 12 + 1), or 18.
- If a queue set is to hold three binary semaphores then uxEventQueueLength should be set to (1 + 1 + 1), or 3.
- If a queue set is to hold a counting semaphore that has a maximum count of 5, and a counting semaphore that has a maximum count of 3, then uxEventQueueLength should be set to (5 + 3), or 8.

Returns If the queue set is created successfully then a handle to the created queue set is returned. Otherwise NULL is returned.

BaseType_t **xQueueAddToSet**(*QueueSetMemberHandle_t* xQueueOrSemaphore, *QueueSetHandle_t* xQueueSet)

Adds a queue or semaphore to a queue set that was previously created by a call to xQueueCreateSet().

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Note 1: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters

- **xQueueOrSemaphore** -- The handle of the queue or semaphore being added to the queue set (cast to an QueueSetMemberHandle_t type).
- **xQueueSet** -- The handle of the queue set to which the queue or semaphore is being added.

Returns If the queue or semaphore was successfully added to the queue set then pdPASS is returned. If the queue could not be successfully added to the queue set because it is already a member of a different queue set then pdFAIL is returned.

BaseType_t **xQueueRemoveFromSet**(*QueueSetMemberHandle_t* xQueueOrSemaphore, *QueueSetHandle_t* xQueueSet)

Removes a queue or semaphore from a queue set. A queue or semaphore can only be removed from a set if the queue or semaphore is empty.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Parameters

- **xQueueOrSemaphore** -- The handle of the queue or semaphore being removed from the queue set (cast to an QueueSetMemberHandle_t type).
- **xQueueSet** -- The handle of the queue set in which the queue or semaphore is included.

Returns If the queue or semaphore was successfully removed from the queue set then pdPASS is returned. If the queue was not in the queue set, or the queue (or semaphore) was not empty, then pdFAIL is returned.

QueueSetMemberHandle_t **xQueueSelectFromSet**(*QueueSetHandle_t* xQueueSet, const TickType_t xTicksToWait)

xQueueSelectFromSet() selects from the members of a queue set a queue or semaphore that either contains data (in the case of a queue) or is available to take (in the case of a semaphore). xQueueSelectFromSet() effectively allows a task to block (pend) on a read operation on all the queues and semaphores in a queue set simultaneously.

See FreeRTOS/Source/Demo/Common/Minimal/QueueSet.c for an example using this function.

Note 1: See the documentation on https://www.FreeRTOS.org/RTOS-queue-sets.html for reasons why queue sets are very rarely needed in practice as there are simpler methods of blocking on multiple objects.

Note 2: Blocking on a queue set that contains a mutex will not cause the mutex holder to inherit the priority of the blocked task.

Note 3: A receive (in the case of a queue) or take (in the case of a semaphore) operation must not be performed on a member of a queue set unless a call to xQueueSelectFromSet() has first returned a handle to that set member.

Parameters

- **xQueueSet** -- The queue set on which the task will (potentially) block.
- **xTicksToWait** -- The maximum time, in ticks, that the calling task will remain in the Blocked state (with other tasks executing) to wait for a member of the queue set to be ready for a successful queue read or semaphore take operation.

Returns xQueueSelectFromSet() will return the handle of a queue (cast to a QueueSetMemberHandle_t type) contained in the queue set that contains data, or the handle of a semaphore (cast

to a QueueSetMemberHandle_t type) contained in the queue set that is available, or NULL if no such queue or semaphore exists before before the specified block time expires.

QueueSetMemberHandle_t **xQueueSelectFromSetFromISR**(*QueueSetHandle_t* xQueueSet)

A version of xQueueSelectFromSet() that can be used from an ISR.

Macros

xQueueCreate(uxQueueLength, uxItemSize)

Creates a new queue instance, and returns a handle by which the new queue can be referenced.

Internally, within the FreeRTOS implementation, queues use two blocks of memory. The first block is used to hold the queue's data structures. The second block is used to hold items placed into the queue. If a queue is created using xQueueCreate() then both blocks of memory are automatically dynamically allocated inside the xQueueCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a queue is created using xQueueCreateStatic() then the application writer must provide the memory that will get used by the queue. xQueueCreateStatic() therefore allows a queue to be created without using any dynamic memory allocation.

https://www.FreeRTOS.org/Embedded-RTOS-Queues.html

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
};

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );
 if( xQueue1 == 0 )
 {
// Queue was not created and must not be used.
 }

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );
 if( xQueue2 == 0 )
 {
// Queue was not created and must not be used.
 }

 // ... Rest of task code.
}
```

Parameters

- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require. Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each posted item. Each item on the queue must be the same size.

Returns If the queue is successfully create then a handle to the newly created queue is returned. If the queue cannot be created then 0 is returned.

xQueueCreateStatic(uxQueueLength, uxItemSize, pucQueueStorage, pxQueueBuffer)

Creates a new queue instance, and returns a handle by which the new queue can be referenced.

Internally, within the FreeRTOS implementation, queues use two blocks of memory. The first block is used to hold the queue's data structures. The second block is used to hold items placed into the queue. If a queue is created using xQueueCreate() then both blocks of memory are automatically dynamically allocated inside the xQueueCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a queue is created using xQueueCreateStatic() then the application writer must provide the memory that will get used by the queue. xQueueCreateStatic() therefore allows a queue to be created without using any dynamic memory allocation.

https://www.FreeRTOS.org/Embedded-RTOS-Queues.html

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
};

#define QUEUE_LENGTH 10
#define ITEM_SIZE sizeof( uint32_t )

// xQueueBuffer will hold the queue structure.
StaticQueue_t xQueueBuffer;

// ucQueueStorage will hold the items posted to the queue.  Must be at least
// [(queue length) * ( queue item size)] bytes long.
uint8_t ucQueueStorage[ QUEUE_LENGTH * ITEM_SIZE ];

void vATask( void *pvParameters )
{
 QueueHandle_t xQueue1;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( QUEUE_LENGTH, // The number of items the queue can␣
↪hold.
                         ITEM_SIZE     // The size of each item in the queue
                         &( ucQueueStorage[ 0 ] ), // The buffer that will␣
↪hold the items in the queue.
                         &xQueueBuffer ); // The buffer that will hold the␣
↪queue structure.

 // The queue is guaranteed to be created successfully as no dynamic memory
 // allocation is used.  Therefore xQueue1 is now a handle to a valid queue.

 // ... Rest of task code.
}
```

Parameters

- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require. Items are queued by copy, not by reference, so this is the number of bytes that will be copied for each posted item. Each item on the queue must be the same size.
- **pucQueueStorage** -- If uxItemSize is not zero then pucQueueStorage must point to a uint8_t array that is at least large enough to hold the maximum number of items that can be in the queue at any one time - which is (uxQueueLength * uxItemsSize) bytes. If uxItemSize is zero then pucQueueStorage can be NULL.
- **pxQueueBuffer** -- Must point to a variable of type StaticQueue_t, which will be used to hold the queue's data structure.

Returns If the queue is created then a handle to the created queue is returned. If pxQueueBuffer is NULL then NULL is returned.

xQueueGetStaticBuffers (xQueue, ppucQueueStorage, ppxStaticQueue)

Retrieve pointers to a statically created queue's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters

- **xQueue** -- The queue for which to retrieve the buffers.
- **ppucQueueStorage** -- Used to return a pointer to the queue's storage area buffer.
- **ppxStaticQueue** -- Used to return a pointer to the queue's data structure buffer.

Returns pdTRUE if buffers were retrieved, pdFALSE otherwise.

xQueueSendToFront (xQueue, pvItemToQueue, xTicksToWait)

Post an item to the front of a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSendToFront( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) !=␣
↪pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSendToFront( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

xQueueSendToBack (xQueue, pvItemToQueue, xTicksToWait)

This is a macro that calls xQueueGenericSend().

Post an item to the back of a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSendToBack( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) !=␣
↪pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }

 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSendToBack( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
```

(continues on next page)

(continued from previous page)

```
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

xQueueSend(xQueue, pvItemToQueue, xTicksToWait)

This is a macro that calls xQueueGenericSend(). It is included for backward compatibility with versions of FreeRTOS.org that did not include the xQueueSendToFront() and xQueueSendToBack() macros. It is equivalent to xQueueSendToBack().

Post an item on a queue. The item is queued by copy, not by reference. This function must not be called from an interrupt service routine. See xQueueSendFromISR () for an alternative which may be used in an ISR.

Example usage:

```
struct AMessage
{
 char ucMessageID;
 char ucData[ 20 ];
} xMessage;

uint32_t ulVar = 10UL;

void vATask( void *pvParameters )
{
QueueHandle_t xQueue1, xQueue2;
struct AMessage *pxMessage;

 // Create a queue capable of containing 10 uint32_t values.
 xQueue1 = xQueueCreate( 10, sizeof( uint32_t ) );

 // Create a queue capable of containing 10 pointers to AMessage structures.
 // These should be passed by pointer as they contain a lot of data.
 xQueue2 = xQueueCreate( 10, sizeof( struct AMessage * ) );

 // ...

 if( xQueue1 != 0 )
 {
// Send an uint32_t.  Wait for 10 ticks for space to become
// available if necessary.
if( xQueueSend( xQueue1, ( void * ) &ulVar, ( TickType_t ) 10 ) != pdPASS )
     {
// Failed to post the message, even after 10 ticks.
     }
 }
```

(continues on next page)

(continued from previous page)

```
 if( xQueue2 != 0 )
 {
// Send a pointer to a struct AMessage object.  Don't block if the
// queue is already full.
     pxMessage = & xMessage;
     xQueueSend( xQueue2, ( void * ) &pxMessage, ( TickType_t ) 0 );
 }

 // ... Rest of task code.
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **xTicksToWait** -- The maximum amount of time the task should block waiting for space to become available on the queue, should it already be full. The call will return immediately if this is set to 0 and the queue is full. The time is defined in tick periods so the constant portTICK_PERIOD_MS should be used to convert to real time if this is required.

Returns pdTRUE if the item was successfully posted, otherwise errQUEUE_FULL.

xQueueOverwrite(xQueue, pvItemToQueue)

Only for use with queues that have a length of one - so the queue is either empty or full.

Post an item on a queue. If the queue is already full then overwrite the value held in the queue. The item is queued by copy, not by reference.

This function must not be called from an interrupt service routine. See xQueueOverwriteFromISR () for an alternative which may be used in an ISR.

Example usage:

```
void vFunction( void *pvParameters )
{
QueueHandle_t xQueue;
uint32_t ulVarToSend, ulValReceived;

 // Create a queue to hold one uint32_t value.  It is strongly
 // recommended *not* to use xQueueOverwrite() on queues that can
 // contain more than one value, and doing so will trigger an assertion
 // if configASSERT() is defined.
 xQueue = xQueueCreate( 1, sizeof( uint32_t ) );

 // Write the value 10 to the queue using xQueueOverwrite().
 ulVarToSend = 10;
 xQueueOverwrite( xQueue, &ulVarToSend );

 // Peeking the queue should now return 10, but leave the value 10 in
 // the queue.  A block time of zero is used as it is known that the
 // queue holds a value.
 ulValReceived = 0;
 xQueuePeek( xQueue, &ulValReceived, 0 );

 if( ulValReceived != 10 )
 {
```

(continues on next page)

(continued from previous page)

```
// Error unless the item was removed by a different task.
 }

 // The queue is still full.  Use xQueueOverwrite() to overwrite the
 // value held in the queue with 100.
 ulVarToSend = 100;
 xQueueOverwrite( xQueue, &ulVarToSend );

 // This time read from the queue, leaving the queue empty once more.
 // A block time of 0 is used again.
 xQueueReceive( xQueue, &ulValReceived, 0 );

 // The value read should be the last value written, even though the
 // queue was already full when the value was written.
 if( ulValReceived != 100 )
 {
// Error!
 }

 // ...
}
```

Parameters

- **xQueue** -- The handle of the queue to which the data is being sent.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.

Returns xQueueOverwrite() is a macro that calls xQueueGenericSend(), and therefore has the same return values as xQueueSendToFront(). However, pdPASS is the only value that can be returned because xQueueOverwrite() will write to the queue even when the queue is already full.

xQueueSendToFrontFromISR (xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR().

Post an item to the front of a queue. It is safe to use this macro from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendToFrontFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );
```

(continues on next page)

(continued from previous page)

```
 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
     taskYIELD ();
 }
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueSendToFrontFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendToFromFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

xQueueSendToBackFromISR(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR().

Post an item to the back of a queue. It is safe to use this macro from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendToBackFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
     taskYIELD ();
 }
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.

- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueSendToBackFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendToBackFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

xQueueOverwriteFromISR(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

A version of xQueueOverwrite() that can be used in an interrupt service routine (ISR).

Only for use with queues that can hold a single item - so the queue is either empty or full.

Post an item on a queue. If the queue is already full then overwrite the value held in the queue. The item is queued by copy, not by reference.

Example usage:

```
QueueHandle_t xQueue;

void vFunction( void *pvParameters )
{
 // Create a queue to hold one uint32_t value.  It is strongly
 // recommended *not* to use xQueueOverwriteFromISR() on queues that can
 // contain more than one value, and doing so will trigger an assertion
 // if configASSERT() is defined.
 xQueue = xQueueCreate( 1, sizeof( uint32_t ) );
}

void vAnInterruptHandler( void )
{
// xHigherPriorityTaskWoken must be set to pdFALSE before it is used.
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
uint32_t ulVarToSend, ulValReceived;

 // Write the value 10 to the queue using xQueueOverwriteFromISR().
 ulVarToSend = 10;
 xQueueOverwriteFromISR( xQueue, &ulVarToSend, &xHigherPriorityTaskWoken );

 // The queue is full, but calling xQueueOverwriteFromISR() again will still
 // pass because the value held in the queue will be overwritten with the
 // new value.
 ulVarToSend = 100;
 xQueueOverwriteFromISR( xQueue, &ulVarToSend, &xHigherPriorityTaskWoken );

 // Reading from the queue will now return 100.

 // ...

 if( xHigherPrioritytaskWoken == pdTRUE )
 {
// Writing to the queue caused a task to unblock and the unblocked task
// has a priority higher than or equal to the priority of the currently
// executing task (the task this interrupt interrupted).  Perform a context
// switch so this interrupt returns directly to the unblocked task.
     portYIELD_FROM_ISR(); // or portEND_SWITCHING_ISR() depending on the port.
 }
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.
- **pxHigherPriorityTaskWoken** -- xQueueOverwriteFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueOverwriteFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns xQueueOverwriteFromISR() is a macro that calls xQueueGenericSendFromISR(), and therefore has the same return values as xQueueSendToFrontFromISR(). However, pdPASS is the only value that can be returned because xQueueOverwriteFromISR() will write to the queue even when the queue is already full.

xQueueSendFromISR(xQueue, pvItemToQueue, pxHigherPriorityTaskWoken)

This is a macro that calls xQueueGenericSendFromISR(). It is included for backward compatibility with versions of FreeRTOS.org that did not include the xQueueSendToBackFromISR() and xQueueSendToFrontFromISR() macros.

Post an item to the back of a queue. It is safe to use this function from within an interrupt service routine.

Items are queued by copy not reference so it is preferable to only queue small items, especially when called from an ISR. In most cases it would be preferable to store a pointer to the item being queued.

Example usage for buffered IO (where the ISR can obtain more than one value per call):

```
void vBufferISR( void )
{
char cIn;
BaseType_t xHigherPriorityTaskWoken;

 // We have not woken a task at the start of the ISR.
 xHigherPriorityTaskWoken = pdFALSE;

 // Loop until the buffer is empty.
 do
 {
// Obtain a byte from the buffer.
     cIn = portINPUT_BYTE( RX_REGISTER_ADDRESS );

// Post the byte.
     xQueueSendFromISR( xRxQueue, &cIn, &xHigherPriorityTaskWoken );

 } while( portINPUT_BYTE( BUFFER_COUNT ) );

 // Now the buffer is empty we can switch context if necessary.
 if( xHigherPriorityTaskWoken )
 {
// Actual macro used here is port specific.
     portYIELD_FROM_ISR ();
 }
}
```

Parameters

- **xQueue** -- The handle to the queue on which the item is to be posted.
- **pvItemToQueue** -- A pointer to the item that is to be placed on the queue. The size of the items the queue will hold was defined when the queue was created, so this many bytes will be copied from pvItemToQueue into the queue storage area.

- **pxHigherPriorityTaskWoken** -- xQueueSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if sending to the queue caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xQueueSendFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns pdTRUE if the data was successfully sent to the queue, otherwise errQUEUE_FULL.

xQueueReset(xQueue)

Reset a queue back to its original empty state. The return value is now obsolete and is always set to pdPASS.

Type Definitions

typedef struct QueueDefinition ***QueueHandle_t**

typedef struct QueueDefinition ***QueueSetHandle_t**

Type by which queue sets are referenced. For example, a call to xQueueCreateSet() returns an xQueueSet variable that can then be used as a parameter to xQueueSelectFromSet(), xQueueAddToSet(), etc.

typedef struct QueueDefinition ***QueueSetMemberHandle_t**

Queue sets can contain both queues and semaphores, so the QueueSetMemberHandle_t is defined as a type to be used where a parameter or return value can be either an QueueHandle_t or an SemaphoreHandle_t.

Semaphore API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/semphr.h
- This header file can be included with:

```
#include "freertos/semphr.h"
```

Macros

semBINARY_SEMAPHORE_QUEUE_LENGTH

semSEMAPHORE_QUEUE_ITEM_LENGTH

semGIVE_BLOCK_TIME

vSemaphoreCreateBinary(xSemaphore)

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

This old vSemaphoreCreateBinary() macro is now deprecated in favour of the xSemaphoreCreateBinary() function. Note that binary semaphores created using the vSemaphoreCreateBinary() macro are created in a state such that the first call to 'take' the semaphore would pass, whereas binary semaphores created using xSemaphoreCreateBinary() are created in a state such that the the semaphore must first be 'given' before it can be 'taken'.

Macro that implements a semaphore by using the existing queue mechanism. The queue length is 1 as this is a binary semaphore. The data size is 0 as we don't want to actually store any data - we just want to know if the queue is empty or full.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not

use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to vSemaphoreCreateBinary ().
 // This is a macro so pass the variable in directly.
 vSemaphoreCreateBinary( xSemaphore );

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Parameters

- **xSemaphore** -- Handle to the created semaphore. Should be of type SemaphoreHandle_t.

xSemaphoreCreateBinary()

Creates a new binary semaphore instance, and returns a handle by which the new semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, binary semaphores use a block of memory, in which the semaphore structure is stored. If a binary semaphore is created using xSemaphoreCreateBinary() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateBinary() function. (see https://www.FreeRTOS.org/a00111.html). If a binary semaphore is created using xSemaphoreCreateBinaryStatic() then the application writer must provide the memory. xSemaphoreCreateBinaryStatic() therefore allows a binary semaphore to be created without using any dynamic memory allocation.

The old vSemaphoreCreateBinary() macro is now deprecated in favour of this xSemaphoreCreateBinary() function. Note that binary semaphores created using the vSemaphoreCreateBinary() macro are created in a state such that the first call to 'take' the semaphore would pass, whereas binary semaphores created using xSemaphoreCreateBinary() are created in a state such that the the semaphore must first be 'given' before it can be 'taken'.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateBinary().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateBinary();
```

(continues on next page)

(continued from previous page)

```
 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns Handle to the created semaphore, or NULL if the memory required to hold the semaphore's data structures could not be allocated.

xSemaphoreCreateBinaryStatic(pxStaticSemaphore)

Creates a new binary semaphore instance, and returns a handle by which the new semaphore can be referenced.

NOTE: In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a binary semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, binary semaphores use a block of memory, in which the semaphore structure is stored. If a binary semaphore is created using xSemaphoreCreateBinary() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateBinary() function. (see https://www.FreeRTOS.org/a00111.html). If a binary semaphore is created using xSemaphoreCreateBinaryStatic() then the application writer must provide the memory. xSemaphoreCreateBinaryStatic() therefore allows a binary semaphore to be created without using any dynamic memory allocation.

This type of semaphore can be used for pure synchronisation between tasks or between an interrupt and a task. The semaphore need not be given back once obtained, so one task/interrupt can continuously 'give' the semaphore while another continuously 'takes' the semaphore. For this reason this type of semaphore does not use a priority inheritance mechanism. For an alternative that does use priority inheritance see xSemaphoreCreateMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;
StaticSemaphore_t xSemaphoreBuffer;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateBinary().
 // The semaphore's data structures will be placed in the xSemaphoreBuffer
 // variable, the address of which is passed into the function.  The
 // function's parameter is not NULL, so the function will not attempt any
 // dynamic memory allocation, and therefore the function will not return
 // return NULL.
 xSemaphore = xSemaphoreCreateBinary( &xSemaphoreBuffer );

 // Rest of task code goes here.
}
```

Parameters

- **pxStaticSemaphore** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the semaphore's data structure, removing the need for the memory to be allocated dynamically.

Returns If the semaphore is created then a handle to the created semaphore is returned. If pxSemaphoreBuffer is NULL then NULL is returned.

xSemaphoreTake(xSemaphore, xBlockTime)

Macro to obtain a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary(), xSemaphoreCreateMutex() or xSemaphoreCreateCounting().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

// A task that creates a semaphore.
void vATask( void * pvParameters )
{
 // Create the semaphore to guard a shared resource.
 xSemaphore = xSemaphoreCreateBinary();
}

// A task that uses the semaphore.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xSemaphore != NULL )
 {
// See if we can obtain the semaphore.  If the semaphore is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTake( xSemaphore, ( TickType_t ) 10 ) == pdTRUE )
      {
// We were able to obtain the semaphore and can now access the
// shared resource.

// ...

// We have finished accessing the shared resource.  Release the
// semaphore.
         xSemaphoreGive( xSemaphore );
      }
else
      {
// We could not obtain the semaphore and can therefore not access
// the shared resource safely.
      }
 }
}
```

Parameters

- **xSemaphore** -- A handle to the semaphore being taken - obtained when the semaphore was created.
- **xBlockTime** -- The time in ticks to wait for the semaphore to become available. The macro portTICK_PERIOD_MS can be used to convert this to a real time. A block time of zero can be used to poll the semaphore. A block time of portMAX_DELAY can be used to block indefinitely (provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h).

Returns pdTRUE if the semaphore was obtained. pdFALSE if xBlockTime expired without the semaphore becoming available.

xSemaphoreTakeRecursive(xMutex, xBlockTime)

Macro to recursively obtain, or 'take', a mutex type semaphore. The mutex must have previously been created using a call to xSemaphoreCreateRecursiveMutex();

configUSE_RECURSIVE_MUTEXES must be set to 1 in FreeRTOSConfig.h for this macro to be available.

This macro must not be used on mutexes created using xSemaphoreCreateMutex().

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

Example usage:

```
SemaphoreHandle_t xMutex = NULL;

// A task that creates a mutex.
void vATask( void * pvParameters )
{
 // Create the mutex to guard a shared resource.
 xMutex = xSemaphoreCreateRecursiveMutex();
}

// A task that uses the mutex.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xMutex != NULL )
 {
// See if we can obtain the mutex.  If the mutex is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTakeRecursive( xSemaphore, ( TickType_t ) 10 ) == pdTRUE )
      {
// We were able to obtain the mutex and can now access the
// shared resource.

// ...
// For some reason due to the nature of the code further calls to
// xSemaphoreTakeRecursive() are made on the same mutex.  In real
// code these would not be just sequential calls as this would make
// no sense.  Instead the calls are likely to be buried inside
// a more complex call structure.
          xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
          xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );

// The mutex has now been 'taken' three times, so will not be
// available to another task until it has also been given back
// three times.  Again it is unlikely that real code would have
// these calls sequentially, but instead buried in a more complex
// call structure.  This is just for illustrative purposes.
          xSemaphoreGiveRecursive( xMutex );
          xSemaphoreGiveRecursive( xMutex );
          xSemaphoreGiveRecursive( xMutex );

// Now the mutex can be taken by other tasks.
      }
else
      {
// We could not obtain the mutex and can therefore not access
// the shared resource safely.
      }
 }
}
```

Parameters

- **xMutex** -- A handle to the mutex being obtained. This is the handle returned by xSemaphoreCreateRecursiveMutex();
- **xBlockTime** -- The time in ticks to wait for the semaphore to become available. The macro portTICK_PERIOD_MS can be used to convert this to a real time. A block time of zero can be used to poll the semaphore. If the task already owns the semaphore then xSemaphoreTakeRecursive() will return immediately no matter what the value of xBlock-

Time.

Returns pdTRUE if the semaphore was obtained. pdFALSE if xBlockTime expired without the semaphore becoming available.

xSemaphoreGive(xSemaphore)

Macro to release a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary(), xSemaphoreCreateMutex() or xSemaphoreCreateCounting(). and obtained using sSemaphoreTake().

This macro must not be used from an ISR. See xSemaphoreGiveFromISR () for an alternative which can be used from an ISR.

This macro must also not be used on semaphores created using xSemaphoreCreateRecursiveMutex().

Example usage:

```
SemaphoreHandle_t xSemaphore = NULL;

void vATask( void * pvParameters )
{
 // Create the semaphore to guard a shared resource.
 xSemaphore = vSemaphoreCreateBinary();

 if( xSemaphore != NULL )
 {
if( xSemaphoreGive( xSemaphore ) != pdTRUE )
      {
// We would expect this call to fail because we cannot give
// a semaphore without first "taking" it!
      }

// Obtain the semaphore - don't block if the semaphore is not
// immediately available.
if( xSemaphoreTake( xSemaphore, ( TickType_t ) 0 ) )
      {
// We now have the semaphore and can access the shared resource.

// ...

// We have finished accessing the shared resource so can free the
// semaphore.
if( xSemaphoreGive( xSemaphore ) != pdTRUE )
          {
// We would not expect this call to fail because we must have
// obtained the semaphore to get here.
          }
      }
 }
}
```

Parameters

- **xSemaphore** -- A handle to the semaphore being released. This is the handle returned when the semaphore was created.

Returns pdTRUE if the semaphore was released. pdFALSE if an error occurred. Semaphores are implemented using queues. An error can occur if there is no space on the queue to post a message - indicating that the semaphore was not first obtained correctly.

xSemaphoreGiveRecursive(xMutex)

Macro to recursively release, or 'give', a mutex type semaphore. The mutex must have previously been created using a call to xSemaphoreCreateRecursiveMutex();

configUSE_RECURSIVE_MUTEXES must be set to 1 in FreeRTOSConfig.h for this macro to be available.

This macro must not be used on mutexes created using xSemaphoreCreateMutex().

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

Example usage:

```
SemaphoreHandle_t xMutex = NULL;

// A task that creates a mutex.
void vATask( void * pvParameters )
{
 // Create the mutex to guard a shared resource.
 xMutex = xSemaphoreCreateRecursiveMutex();
}

// A task that uses the mutex.
void vAnotherTask( void * pvParameters )
{
 // ... Do other things.

 if( xMutex != NULL )
 {
// See if we can obtain the mutex.  If the mutex is not available
// wait 10 ticks to see if it becomes free.
if( xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 ) == pdTRUE )
     {
// We were able to obtain the mutex and can now access the
// shared resource.

// ...
// For some reason due to the nature of the code further calls to
// xSemaphoreTakeRecursive() are made on the same mutex.  In real
// code these would not be just sequential calls as this would make
// no sense.  Instead the calls are likely to be buried inside
// a more complex call structure.
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );
         xSemaphoreTakeRecursive( xMutex, ( TickType_t ) 10 );

// The mutex has now been 'taken' three times, so will not be
// available to another task until it has also been given back
// three times.  Again it is unlikely that real code would have
// these calls sequentially, it would be more likely that the calls
// to xSemaphoreGiveRecursive() would be called as a call stack
// unwound.  This is just for demonstrative purposes.
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );
         xSemaphoreGiveRecursive( xMutex );

// Now the mutex can be taken by other tasks.
     }
else
     {
// We could not obtain the mutex and can therefore not access
// the shared resource safely.
     }
 }
}
```

Parameters

- **xMutex** -- A handle to the mutex being released, or 'given'. This is the handle returned by xSemaphoreCreateMutex();

Returns pdTRUE if the semaphore was given.

xSemaphoreGiveFromISR (xSemaphore, pxHigherPriorityTaskWoken)

Macro to release a semaphore. The semaphore must have previously been created with a call to xSemaphoreCreateBinary() or xSemaphoreCreateCounting().

Mutex type semaphores (those created using a call to xSemaphoreCreateMutex()) must not be used with this macro.

This macro can be used from an ISR.

Example usage:

```
#define LONG_TIME 0xffff
#define TICKS_TO_WAIT 10
SemaphoreHandle_t xSemaphore = NULL;

// Repetitive task.
void vATask( void * pvParameters )
{
 for( ;; )
 {
// We want this task to run every 10 ticks of a timer.  The semaphore
// was created before this task was started.

// Block waiting for the semaphore to become available.
if( xSemaphoreTake( xSemaphore, LONG_TIME ) == pdTRUE )
     {
// It is time to execute.

// ...

// We have finished our task.  Return to the top of the loop where
// we will block on the semaphore until it is time to execute
// again.  Note when using the semaphore for synchronisation with an
// ISR in this manner there is no need to 'give' the semaphore back.
     }
 }
}

// Timer ISR
void vTimerISR( void * pvParameters )
{
static uint8_t ucLocalTickCount = 0;
static BaseType_t xHigherPriorityTaskWoken;

 // A timer tick has occurred.

 // ... Do other time functions.

 // Is it time for vATask () to run?
 xHigherPriorityTaskWoken = pdFALSE;
 ucLocalTickCount++;
 if( ucLocalTickCount >= TICKS_TO_WAIT )
 {
// Unblock the task by releasing the semaphore.
     xSemaphoreGiveFromISR( xSemaphore, &xHigherPriorityTaskWoken );
```

(continues on next page)

(continued from previous page)

```
// Reset the count so we release the semaphore again in 10 ticks time.
     ucLocalTickCount = 0;
 }

 if( xHigherPriorityTaskWoken != pdFALSE )
 {
// We can force a context switch here.  Context switching from an
// ISR uses port specific syntax.  Check the demo task for your port
// to find the syntax required.
 }
}
```

Parameters

- **xSemaphore** -- A handle to the semaphore being released. This is the handle returned when the semaphore was created.
- **pxHigherPriorityTaskWoken** -- xSemaphoreGiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if giving the semaphore caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xSemaphoreGiveFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns pdTRUE if the semaphore was successfully given, otherwise errQUEUE_FULL.

xSemaphoreTakeFromISR(xSemaphore, pxHigherPriorityTaskWoken)

Macro to take a semaphore from an ISR. The semaphore must have previously been created with a call to xSemaphoreCreateBinary() or xSemaphoreCreateCounting().

Mutex type semaphores (those created using a call to xSemaphoreCreateMutex()) must not be used with this macro.

This macro can be used from an ISR, however taking a semaphore from an ISR is not a common operation. It is likely to only be useful when taking a counting semaphore when an interrupt is obtaining an object from a resource pool (when the semaphore count indicates the number of resources available).

Parameters

- **xSemaphore** -- A handle to the semaphore being taken. This is the handle returned when the semaphore was created.
- **pxHigherPriorityTaskWoken** -- xSemaphoreTakeFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE if taking the semaphore caused a task to unblock, and the unblocked task has a priority higher than the currently running task. If xSemaphoreTakeFromISR() sets this value to pdTRUE then a context switch should be requested before the interrupt is exited.

Returns pdTRUE if the semaphore was successfully taken, otherwise pdFALSE

xSemaphoreCreateMutex()

Creates a new mutex type semaphore instance, and returns a handle by which the new mutex can be referenced.

Internally, within the FreeRTOS implementation, mutex semaphores use a block of memory, in which the mutex structure is stored. If a mutex is created using xSemaphoreCreateMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a mutex is created using xSemaphoreCreateMutexStatic() then the application writer must provided the memory. xSemaphoreCreateMutexStatic() therefore allows a mutex to be created without using any dynamic memory allocation.

Mutexes created using this function can be accessed using the xSemaphoreTake() and xSemaphoreGive() macros. The xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros must not be used.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateMutex().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateMutex();

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns If the mutex was successfully created then a handle to the created semaphore is returned. If there was not enough heap to allocate the mutex data structures then NULL is returned.

xSemaphoreCreateMutexStatic(pxMutexBuffer)

Creates a new mutex type semaphore instance, and returns a handle by which the new mutex can be referenced.

Internally, within the FreeRTOS implementation, mutex semaphores use a block of memory, in which the mutex structure is stored. If a mutex is created using xSemaphoreCreateMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a mutex is created using xSemaphoreCreateMutexStatic() then the application writer must provided the memory. xSemaphoreCreateMutexStatic() therefore allows a mutex to be created without using any dynamic memory allocation.

Mutexes created using this function can be accessed using the xSemaphoreTake() and xSemaphoreGive() macros. The xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros must not be used.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xMutexBuffer;

void vATask( void * pvParameters )
{
 // A mutex cannot be used before it has been created.  xMutexBuffer is
 // into xSemaphoreCreateMutexStatic() so no dynamic memory allocation is
 // attempted.
 xSemaphore = xSemaphoreCreateMutexStatic( &xMutexBuffer );

 // As no dynamic memory allocation was performed, xSemaphore cannot be NULL,
```

(continues on next page)

(continued from previous page)

```
 // so there is no need to check it.
}
```

Parameters

- **pxMutexBuffer** -- Must point to a variable of type StaticSemaphore_t, which will be used to hold the mutex's data structure, removing the need for the memory to be allocated dynamically.

Returns If the mutex was successfully created then a handle to the created mutex is returned. If pxMutexBuffer was NULL then NULL is returned.

xSemaphoreCreateRecursiveMutex()

Creates a new recursive mutex type semaphore instance, and returns a handle by which the new recursive mutex can be referenced.

Internally, within the FreeRTOS implementation, recursive mutexes use a block of memory, in which the mutex structure is stored. If a recursive mutex is created using xSemaphoreCreateRecursiveMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateRecursiveMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic() then the application writer must provide the memory that will get used by the mutex. xSemaphoreCreateRecursiveMutexStatic() therefore allows a recursive mutex to be created without using any dynamic memory allocation.

Mutexes created using this macro can be accessed using the xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros. The xSemaphoreTake() and xSemaphoreGive() macros must not be used.

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
 // Semaphore cannot be used before a call to xSemaphoreCreateMutex().
 // This is a macro so pass the variable in directly.
 xSemaphore = xSemaphoreCreateRecursiveMutex();

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Returns xSemaphore Handle to the created mutex semaphore. Should be of type SemaphoreHandle_t.

xSemaphoreCreateRecursiveMutexStatic (pxStaticSemaphore)

Creates a new recursive mutex type semaphore instance, and returns a handle by which the new recursive mutex can be referenced.

Internally, within the FreeRTOS implementation, recursive mutexes use a block of memory, in which the mutex structure is stored. If a recursive mutex is created using xSemaphoreCreateRecursiveMutex() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateRecursiveMutex() function. (see https://www.FreeRTOS.org/a00111.html). If a recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic() then the application writer must provide the memory that will get used by the mutex. xSemaphoreCreateRecursiveMutexStatic() therefore allows a recursive mutex to be created without using any dynamic memory allocation.

Mutexes created using this macro can be accessed using the xSemaphoreTakeRecursive() and xSemaphoreGiveRecursive() macros. The xSemaphoreTake() and xSemaphoreGive() macros must not be used.

A mutex used recursively can be 'taken' repeatedly by the owner. The mutex doesn't become available again until the owner has called xSemaphoreGiveRecursive() for each successful 'take' request. For example, if a task successfully 'takes' the same mutex 5 times then the mutex will not be available to any other task until it has also 'given' the mutex back exactly five times.

This type of semaphore uses a priority inheritance mechanism so a task 'taking' a semaphore MUST ALWAYS 'give' the semaphore back once the semaphore it is no longer required.

Mutex type semaphores cannot be used from within interrupt service routines.

See xSemaphoreCreateBinary() for an alternative implementation that can be used for pure synchronisation (where one task or interrupt always 'gives' the semaphore and another always 'takes' the semaphore) and from within interrupt service routines.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xMutexBuffer;

void vATask( void * pvParameters )
{
 // A recursive semaphore cannot be used before it is created.  Here a
 // recursive mutex is created using xSemaphoreCreateRecursiveMutexStatic().
 // The address of xMutexBuffer is passed into the function, and will hold
 // the mutexes data structures - so no dynamic memory allocation will be
 // attempted.
 xSemaphore = xSemaphoreCreateRecursiveMutexStatic( &xMutexBuffer );

 // As no dynamic memory allocation was performed, xSemaphore cannot be NULL,
 // so there is no need to check it.
}
```

Parameters

- **pxStaticSemaphore** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the recursive mutex's data structure, removing the need for the memory to be allocated dynamically.

Returns If the recursive mutex was successfully created then a handle to the created recursive mutex is returned. If pxStaticSemaphore was NULL then NULL is returned.

xSemaphoreCreateCounting (uxMaxCount, uxInitialCount)

Creates a new counting semaphore instance, and returns a handle by which the new counting semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a counting semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, counting semaphores use a block of memory, in which the counting semaphore structure is stored. If a counting semaphore is created using xSemaphoreCreateCounting() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateCounting() function. (see https://www.FreeRTOS.org/a00111.html). If a counting semaphore is created using xSemaphoreCreateCountingStatic() then the application writer can instead optionally provide the memory that will get used by the counting semaphore. xSemaphoreCreateCountingStatic() therefore allows a counting semaphore to be created without using any dynamic memory allocation.

Counting semaphores are typically used for two things:

1) Counting events.

In this usage scenario an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is therefore the difference between the number of events that have occurred and the number that have been processed. In this case it is desirable for the initial count value to be zero.

2) Resource management.

In this usage scenario the count value indicates the number of resources available. To obtain control of a resource a task must first obtain a semaphore - decrementing the semaphore count value. When the count value reaches zero there are no free resources. When a task finishes with the resource it 'gives' the semaphore back - incrementing the semaphore count value. In this case it is desirable for the initial count value to be equal to the maximum count value, indicating that all resources are free.

Example usage:

```
SemaphoreHandle_t xSemaphore;

void vATask( void * pvParameters )
{
SemaphoreHandle_t xSemaphore = NULL;

 // Semaphore cannot be used before a call to xSemaphoreCreateCounting().
 // The max value to which the semaphore can count should be 10, and the
 // initial value assigned to the count should be 0.
 xSemaphore = xSemaphoreCreateCounting( 10, 0 );

 if( xSemaphore != NULL )
 {
// The semaphore was created successfully.
// The semaphore can now be used.
 }
}
```

Parameters

- **uxMaxCount** -- The maximum count value that can be reached. When the semaphore reaches this value it can no longer be 'given'.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.

Returns Handle to the created semaphore. Null if the semaphore could not be created.

xSemaphoreCreateCountingStatic(uxMaxCount, uxInitialCount, pxSemaphoreBuffer)

Creates a new counting semaphore instance, and returns a handle by which the new counting semaphore can be referenced.

In many usage scenarios it is faster and more memory efficient to use a direct to task notification in place of a counting semaphore! https://www.FreeRTOS.org/RTOS-task-notifications.html

Internally, within the FreeRTOS implementation, counting semaphores use a block of memory, in which the counting semaphore structure is stored. If a counting semaphore is created using xSemaphoreCreateCount-

ing() then the required memory is automatically dynamically allocated inside the xSemaphoreCreateCounting() function. (see https://www.FreeRTOS.org/a00111.html). If a counting semaphore is created using xSemaphoreCreateCountingStatic() then the application writer must provide the memory. xSemaphoreCreateCountingStatic() therefore allows a counting semaphore to be created without using any dynamic memory allocation.

Counting semaphores are typically used for two things:

1) Counting events.

In this usage scenario an event handler will 'give' a semaphore each time an event occurs (incrementing the semaphore count value), and a handler task will 'take' a semaphore each time it processes an event (decrementing the semaphore count value). The count value is therefore the difference between the number of events that have occurred and the number that have been processed. In this case it is desirable for the initial count value to be zero.

2) Resource management.

In this usage scenario the count value indicates the number of resources available. To obtain control of a resource a task must first obtain a semaphore - decrementing the semaphore count value. When the count value reaches zero there are no free resources. When a task finishes with the resource it 'gives' the semaphore back - incrementing the semaphore count value. In this case it is desirable for the initial count value to be equal to the maximum count value, indicating that all resources are free.

Example usage:

```
SemaphoreHandle_t xSemaphore;
StaticSemaphore_t xSemaphoreBuffer;

void vATask( void * pvParameters )
{
SemaphoreHandle_t xSemaphore = NULL;

 // Counting semaphore cannot be used before they have been created.  Create
 // a counting semaphore using xSemaphoreCreateCountingStatic().  The max
 // value to which the semaphore can count is 10, and the initial value
 // assigned to the count will be 0.  The address of xSemaphoreBuffer is
 // passed in and will be used to hold the semaphore structure, so no dynamic
 // memory allocation will be used.
 xSemaphore = xSemaphoreCreateCounting( 10, 0, &xSemaphoreBuffer );

 // No memory allocation was attempted so xSemaphore cannot be NULL, so there
 // is no need to check its value.
}
```

Parameters

- **uxMaxCount** -- The maximum count value that can be reached. When the semaphore reaches this value it can no longer be 'given'.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.
- **pxSemaphoreBuffer** -- Must point to a variable of type StaticSemaphore_t, which will then be used to hold the semaphore's data structure, removing the need for the memory to be allocated dynamically.

Returns If the counting semaphore was successfully created then a handle to the created counting semaphore is returned. If pxSemaphoreBuffer was NULL then NULL is returned.

vSemaphoreDelete(xSemaphore)

Delete a semaphore. This function must be used with care. For example, do not delete a mutex type semaphore if the mutex is held by a task.

Parameters

- **xSemaphore** -- A handle to the semaphore to be deleted.

xSemaphoreGetMutexHolder(xSemaphore)

If xMutex is indeed a mutex type semaphore, return the current mutex holder. If xMutex is not a mutex type semaphore, or the mutex is available (not held by a task), return NULL.

Note: This is a good way of determining if the calling task is the mutex holder, but not a good way of determining the identity of the mutex holder as the holder may change between the function exiting and the returned value being tested.

xSemaphoreGetMutexHolderFromISR(xSemaphore)

If xMutex is indeed a mutex type semaphore, return the current mutex holder. If xMutex is not a mutex type semaphore, or the mutex is available (not held by a task), return NULL.

uxSemaphoreGetCount(xSemaphore)

If the semaphore is a counting semaphore then uxSemaphoreGetCount() returns its current count value. If the semaphore is a binary semaphore then uxSemaphoreGetCount() returns 1 if the semaphore is available, and 0 if the semaphore is not available.

uxSemaphoreGetCountFromISR(xSemaphore)

semphr.h

```
UBaseType_t uxSemaphoreGetCountFromISR( SemaphoreHandle_t xSemaphore );
```

If the semaphore is a counting semaphore then uxSemaphoreGetCountFromISR() returns its current count value. If the semaphore is a binary semaphore then uxSemaphoreGetCountFromISR() returns 1 if the semaphore is available, and 0 if the semaphore is not available.

xSemaphoreGetStaticBuffer(xSemaphore, ppxSemaphoreBuffer)

Retrieve pointer to a statically created binary semaphore, counting semaphore, or mutex semaphore's data structure buffer. This is the same buffer that is supplied at the time of creation.

Parameters

- **xSemaphore** -- The semaphore for which to retrieve the buffer.
- **ppxSemaphoreBuffer** -- Used to return a pointer to the semaphore's data structure buffer.

Returns pdTRUE if buffer was retrieved, pdFALSE otherwise.

Type Definitions

typedef *QueueHandle_t* **SemaphoreHandle_t**

Timer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/timers.h
- This header file can be included with:

```
#include "freertos/timers.h"
```

Functions

TimerHandle_t **xTimerCreate**(const char *const pcTimerName, const TickType_t xTimerPeriodInTicks, const BaseType_t xAutoReload, void *const pvTimerID, *TimerCallbackFunction_t* pxCallbackFunction)

Creates a new software timer instance, and returns a handle by which the created software timer can be referenced.

Internally, within the FreeRTOS implementation, software timers use a block of memory, in which the timer data structure is stored. If a software timer is created using xTimerCreate() then the required memory is automatically dynamically allocated inside the xTimerCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a software timer is created using xTimerCreateStatic() then the application writer must provide the memory that will get used by the software timer. xTimerCreateStatic() therefore allows a software timer to be created without using any dynamic memory allocation.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
* #define NUM_TIMERS 5
*
* // An array to hold handles to the created timers.
* TimerHandle_t xTimers[ NUM_TIMERS ];
*
* // An array to hold a count of the number of times each timer expires.
* int32_t lExpireCounters[ NUM_TIMERS ] = { 0 };
*
* // Define a callback function that will be used by multiple timer instances.
* // The callback function does nothing but count the number of times the
* // associated timer expires, and stop the timer once the timer has expired
* // 10 times.
* void vTimerCallback( TimerHandle_t pxTimer )
* {
* int32_t lArrayIndex;
* const int32_t xMaxExpiryCountBeforeStopping = 10;
*
*     // Optionally do something if the pxTimer parameter is NULL.
*     configASSERT( pxTimer );
*
*     // Which timer expired?
*     lArrayIndex = ( int32_t ) pvTimerGetTimerID( pxTimer );
*
*     // Increment the number of times that pxTimer has expired.
*     lExpireCounters[ lArrayIndex ] += 1;
*
*     // If the timer has expired 10 times then stop it from running.
*     if( lExpireCounters[ lArrayIndex ] == xMaxExpiryCountBeforeStopping )
*     {
*         // Do not use a block time if calling a timer API function from a
*         // timer callback function, as doing so could cause a deadlock!
*         xTimerStop( pxTimer, 0 );
*     }
* }
*
* void main( void )
* {
* int32_t x;
*
*     // Create then start some timers.  Starting the timers before the␣
↪scheduler
*     // has been started means the timers will start running immediately that
*     // the scheduler starts.
*     for( x = 0; x < NUM_TIMERS; x++ )
*     {
*         xTimers[ x ] = xTimerCreate(    "Timer",             // Just a text␣
↪name, not used by the kernel.
*                                         ( 100 * ( x + 1 ) ), // The timer␣
↪period in ticks.
```

(continues on next page)

(continued from previous page)

```
*                                           pdTRUE,                 // The timers␣
↪will auto-reload themselves when they expire.
*                                           ( void * ) x,           // Assign each␣
↪timer a unique id equal to its array index.
*                                           vTimerCallback          // Each timer␣
↪calls the same callback when it expires.
*                                       );
*
*         if( xTimers[ x ] == NULL )
*         {
*             // The timer was not created.
*         }
*         else
*         {
*             // Start the timer.  No block time is specified, and even if one␣
↪was
*             // it would be ignored because the scheduler has not yet been
*             // started.
*             if( xTimerStart( xTimers[ x ], 0 ) != pdPASS )
*             {
*                 // The timer could not be set into the Active state.
*             }
*         }
*     }
*
*     // ...
*     // Create tasks here.
*     // ...
*
*     // Starting the scheduler will start the timers running as they have␣
↪already
*     // been set into the active state.
*     vTaskStartScheduler();
*
*     // Should not reach here.
*     for( ;; );
* }
*
```

Parameters

- **pcTimerName** -- A text name that is assigned to the timer. This is done purely to assist debugging. The kernel itself only ever references a timer by its handle, and never by its name.
- **xTimerPeriodInTicks** -- The timer period. The time is defined in tick periods so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xTimerPeriodInTicks should be set to 100. Alternatively, if the timer must expire after 500ms, then xPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000. Time timer period must be greater than 0.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the xTimerPeriodInTicks parameter. If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.
- **pvTimerID** -- An identifier that is assigned to the timer being created. Typically this would be used in the timer callback function to identify which timer expired when the same callback function is assigned to more than one timer.
- **pxCallbackFunction** -- The function to call when the timer expires. Callback functions must have the prototype defined by TimerCallbackFunction_t, which is "void vCallbackFunction(TimerHandle_t xTimer);".

Returns If the timer is successfully created then a handle to the newly created timer is returned. If the timer cannot be created because there is insufficient FreeRTOS heap remaining to allocate the timer structures then NULL is returned.

TimerHandle_t **xTimerCreateStatic**(const char *const pcTimerName, const TickType_t xTimerPeriodInTicks, const BaseType_t xAutoReload, void *const pvTimerID, *TimerCallbackFunction_t* pxCallbackFunction, StaticTimer_t *pxTimerBuffer)

Creates a new software timer instance, and returns a handle by which the created software timer can be referenced.

Internally, within the FreeRTOS implementation, software timers use a block of memory, in which the timer data structure is stored. If a software timer is created using xTimerCreate() then the required memory is automatically dynamically allocated inside the xTimerCreate() function. (see https://www.FreeRTOS.org/a00111.html). If a software timer is created using xTimerCreateStatic() then the application writer must provide the memory that will get used by the software timer. xTimerCreateStatic() therefore allows a software timer to be created without using any dynamic memory allocation.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
*
* // The buffer used to hold the software timer's data structure.
* static StaticTimer_t xTimerBuffer;
*
* // A variable that will be incremented by the software timer's callback
* // function.
* UBaseType_t uxVariableToIncrement = 0;
*
* // A software timer callback function that increments a variable passed to
* // it when the software timer was created.  After the 5th increment the
* // callback function stops the software timer.
* static void prvTimerCallback( TimerHandle_t xExpiredTimer )
* {
* UBaseType_t *puxVariableToIncrement;
* BaseType_t xReturned;
*
*     // Obtain the address of the variable to increment from the timer ID.
*     puxVariableToIncrement = ( UBaseType_t * ) pvTimerGetTimerID(␣
↪xExpiredTimer );
*
*     // Increment the variable to show the timer callback has executed.
*     ( *puxVariableToIncrement )++;
*
*     // If this callback has executed the required number of times, stop the
*     // timer.
*     if( *puxVariableToIncrement == 5 )
*     {
*         // This is called from a timer callback so must not block.
*         xTimerStop( xExpiredTimer, staticDONT_BLOCK );
*     }
* }
*
*
* void main( void )
* {
*     // Create the software time.  xTimerCreateStatic() has an extra parameter
*     // than the normal xTimerCreate() API function.  The parameter is a␣
↪pointer
```

(continues on next page)

(continued from previous page)

```
*     // to the StaticTimer_t structure that will hold the software timer
*     // structure.  If the parameter is passed as NULL then the structure will be
*     // allocated dynamically, just as if xTimerCreate() had been called.
*     xTimer = xTimerCreateStatic( "T1",             // Text name for the task.  Helps debugging only.  Not used by FreeRTOS.
*                                  xTimerPeriod,     // The period of the timer in ticks.
*                                  pdTRUE,           // This is an auto-reload timer.
*                                  ( void * ) &uxVariableToIncrement,    // A variable incremented by the software timer's callback function
*                                  prvTimerCallback, // The function to execute when the timer expires.
*                                  &xTimerBuffer );  // The buffer that will hold the software timer structure.
*
*     // The scheduler has not started yet so a block time is not used.
*     xReturned = xTimerStart( xTimer, 0 );
*
*     // ...
*     // Create tasks here.
*     // ...
*
*     // Starting the scheduler will start the timers running as they have already
*     // been set into the active state.
*     vTaskStartScheduler();
*
*     // Should not reach here.
*     for( ;; );
* }
*
```

Parameters

- **pcTimerName** -- A text name that is assigned to the timer. This is done purely to assist debugging. The kernel itself only ever references a timer by its handle, and never by its name.
- **xTimerPeriodInTicks** -- The timer period. The time is defined in tick periods so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xTimerPeriodInTicks should be set to 100. Alternatively, if the timer must expire after 500ms, then xPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000. The timer period must be greater than 0.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the xTimerPeriodInTicks parameter. If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.
- **pvTimerID** -- An identifier that is assigned to the timer being created. Typically this would be used in the timer callback function to identify which timer expired when the same callback function is assigned to more than one timer.
- **pxCallbackFunction** -- The function to call when the timer expires. Callback functions must have the prototype defined by TimerCallbackFunction_t, which is "void vCallbackFunction(TimerHandle_t xTimer);".
- **pxTimerBuffer** -- Must point to a variable of type StaticTimer_t, which will be then be used to hold the software timer's data structures, removing the need for the memory to be allocated dynamically.

Returns If the timer is created then a handle to the created timer is returned. If pxTimerBuffer was NULL then NULL is returned.

void *__pvTimerGetTimerID__ (const *TimerHandle_t* xTimer)

Returns the ID assigned to the timer.

IDs are assigned to timers using the pvTimerID parameter of the call to xTimerCreated() that was used to create the timer, and by calling the vTimerSetTimerID() API function.

If the same callback function is assigned to multiple timers then the timer ID can be used as time specific (timer local) storage.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters **xTimer** -- The timer being queried.
Returns The ID assigned to the timer being queried.

void **vTimerSetTimerID** (*TimerHandle_t* xTimer, void *pvNewID)

Sets the ID assigned to the timer.

IDs are assigned to timers using the pvTimerID parameter of the call to xTimerCreated() that was used to create the timer.

If the same callback function is assigned to multiple timers then the timer ID can be used as time specific (timer local) storage.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters
- **xTimer** -- The timer being updated.
- **pvNewID** -- The ID to assign to the timer.

BaseType_t **xTimerIsTimerActive** (*TimerHandle_t* xTimer)

Queries a timer to see if it is active or dormant.

A timer will be dormant if: 1) It has been created but not started, or 2) It is an expired one-shot timer that has not been restarted.

Timers are created in the dormant state. The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() and xTimerChangePeriodFromISR() API functions can all be used to transition a timer into the active state.

Example usage:

```
* // This function assumes xTimer has already been created.
* void vAFunction( TimerHandle_t xTimer )
* {
*     if( xTimerIsTimerActive( xTimer ) != pdFALSE ) // or more simply and␣
↪equivalently "if( xTimerIsTimerActive( xTimer ) )"
*     {
*         // xTimer is active, do something.
*     }
*     else
*     {
*         // xTimer is not active, do something else.
*     }
* }
*
```

Parameters **xTimer** -- The timer being queried.

Returns pdFALSE will be returned if the timer is dormant. A value other than pdFALSE will be returned if the timer is active.

TaskHandle_t **xTimerGetTimerDaemonTaskHandle** (void)

Simply returns the handle of the timer service/daemon task. It it not valid to call xTimerGetTimerDaemonTaskHandle() before the scheduler has been started.

BaseType_t **xTimerPendFunctionCallFromISR** (*PendedFunction_t* xFunctionToPend, void *pvParameter1, uint32_t ulParameter2, BaseType_t *pxHigherPriorityTaskWoken)

Used from application interrupt service routines to defer the execution of a function to the RTOS daemon task (the timer service task, hence this function is implemented in timers.c and is prefixed with 'Timer').

Ideally an interrupt service routine (ISR) is kept as short as possible, but sometimes an ISR either has a lot of processing to do, or needs to perform processing that is not deterministic. In these cases xTimerPendFunctionCallFromISR() can be used to defer processing of a function to the RTOS daemon task.

A mechanism is provided that allows the interrupt to return directly to the task that will subsequently execute the pended callback function. This allows the callback function to execute contiguously in time with the interrupt - just as if the callback had executed in the interrupt itself.

Example usage:

```
*
*  // The callback function that will execute in the context of the daemon␣
↪task.
*  // Note callback functions must all use this same prototype.
*  void vProcessInterface( void *pvParameter1, uint32_t ulParameter2 )
*  {
*      BaseType_t xInterfaceToService;
*
*      // The interface that requires servicing is passed in the second
*      // parameter.  The first parameter is not used in this case.
*      xInterfaceToService = ( BaseType_t ) ulParameter2;
*
*      // ...Perform the processing here...
*  }
*
*  // An ISR that receives data packets from multiple interfaces
*  void vAnISR( void )
*  {
*      BaseType_t xInterfaceToService, xHigherPriorityTaskWoken;
*
*      // Query the hardware to determine which interface needs processing.
*      xInterfaceToService = prvCheckInterfaces();
*
*      // The actual processing is to be deferred to a task.  Request the
*      // vProcessInterface() callback function is executed, passing in the
*      // number of the interface that needs processing.  The interface to
*      // service is passed in the second parameter.  The first parameter is
*      // not used in this case.
*      xHigherPriorityTaskWoken = pdFALSE;
*      xTimerPendFunctionCallFromISR( vProcessInterface, NULL, ( uint32_t )␣
↪xInterfaceToService, &xHigherPriorityTaskWoken );
*
*      // If xHigherPriorityTaskWoken is now set to pdTRUE then a context
*      // switch should be requested.  The macro used is port specific and will
*      // be either portYIELD_FROM_ISR() or portEND_SWITCHING_ISR() - refer to
*      // the documentation page for the port being used.
*      portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
*
```

(continues on next page)

(continued from previous page)

```
*   }
*
```

Parameters

- **xFunctionToPend** -- The function to execute from the timer service/ daemon task. The function must conform to the PendedFunction_t prototype.
- **pvParameter1** -- The value of the callback function's first parameter. The parameter has a void * type to allow it to be used to pass any type. For example, unsigned longs can be cast to a void *, or the void * can be used to point to a structure.
- **ulParameter2** -- The value of the callback function's second parameter.
- **pxHigherPriorityTaskWoken** -- As mentioned above, calling this function will result in a message being sent to the timer daemon task. If the priority of the timer daemon task (which is set using configTIMER_TASK_PRIORITY in FreeRTOSConfig.h) is higher than the priority of the currently running task (the task the interrupt interrupted) then *pxHigherPriorityTaskWoken will be set to pdTRUE within xTimerPendFunctionCallFromISR(), indicating that a context switch should be requested before the interrupt exits. For that reason *pxHigherPriorityTaskWoken must be initialised to pdFALSE. See the example code below.

Returns pdPASS is returned if the message was successfully sent to the timer daemon task, otherwise pdFALSE is returned.

BaseType_t **xTimerPendFunctionCall**(*PendedFunction_t* xFunctionToPend, void *pvParameter1, uint32_t ulParameter2, TickType_t xTicksToWait)

Used to defer the execution of a function to the RTOS daemon task (the timer service task, hence this function is implemented in timers.c and is prefixed with 'Timer').

Parameters

- **xFunctionToPend** -- The function to execute from the timer service/ daemon task. The function must conform to the PendedFunction_t prototype.
- **pvParameter1** -- The value of the callback function's first parameter. The parameter has a void * type to allow it to be used to pass any type. For example, unsigned longs can be cast to a void *, or the void * can be used to point to a structure.
- **ulParameter2** -- The value of the callback function's second parameter.
- **xTicksToWait** -- Calling this function will result in a message being sent to the timer daemon task on a queue. xTicksToWait is the amount of time the calling task should remain in the Blocked state (so not using any processing time) for space to become available on the timer queue if the queue is found to be full.

Returns pdPASS is returned if the message was successfully sent to the timer daemon task, otherwise pdFALSE is returned.

const char *__pcTimerGetName__(*TimerHandle_t* xTimer)

Returns the name that was assigned to a timer when the timer was created.

Parameters **xTimer** -- The handle of the timer being queried.

Returns The name assigned to the timer specified by the xTimer parameter.

void **vTimerSetReloadMode**(*TimerHandle_t* xTimer, const BaseType_t xAutoReload)

Updates a timer to be either an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters

- **xTimer** -- The handle of the timer being updated.
- **xAutoReload** -- If xAutoReload is set to pdTRUE then the timer will expire repeatedly with a frequency set by the timer's period (see the xTimerPeriodInTicks parameter of the xTimerCreate() API function). If xAutoReload is set to pdFALSE then the timer will be a one-shot timer and enter the dormant state after it expires.

BaseType_t **xTimerGetReloadMode** (*TimerHandle_t* xTimer)

Queries a timer to determine if it is an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters **xTimer** -- The handle of the timer being queried.
Returns If the timer is an auto-reload timer then pdTRUE is returned, otherwise pdFALSE is returned.

UBaseType_t **uxTimerGetReloadMode** (*TimerHandle_t* xTimer)

Queries a timer to determine if it is an auto-reload timer, in which case the timer automatically resets itself each time it expires, or a one-shot timer, in which case the timer will only expire once unless it is manually restarted.

Parameters **xTimer** -- The handle of the timer being queried.
Returns If the timer is an auto-reload timer then pdTRUE is returned, otherwise pdFALSE is returned.

TickType_t **xTimerGetPeriod** (*TimerHandle_t* xTimer)

Returns the period of a timer.

Parameters **xTimer** -- The handle of the timer being queried.
Returns The period of the timer in ticks.

TickType_t **xTimerGetExpiryTime** (*TimerHandle_t* xTimer)

Returns the time in ticks at which the timer will expire. If this is less than the current tick count then the expiry time has overflowed from the current time.

Parameters **xTimer** -- The handle of the timer being queried.
Returns If the timer is running then the time in ticks at which the timer will next expire is returned. If the timer is not running then the return value is undefined.

BaseType_t **xTimerGetStaticBuffer** (*TimerHandle_t* xTimer, StaticTimer_t **ppxTimerBuffer)

Retrieve pointer to a statically created timer's data structure buffer. This is the same buffer that is supplied at the time of creation.

Parameters
- **xTimer** -- The timer for which to retrieve the buffer.
- **ppxTimerBuffer** -- Used to return a pointer to the timers's data structure buffer.

Returns pdTRUE if the buffer was retrieved, pdFALSE otherwise.

void **vApplicationGetTimerTaskMemory** (StaticTask_t **ppxTimerTaskTCBBuffer, StackType_t **ppxTimerTaskStackBuffer, uint32_t *pulTimerTaskStackSize)

This function is used to provide a statically allocated block of memory to FreeRTOS to hold the Timer Task TCB. This function is required when configSUPPORT_STATIC_ALLOCATION is set. For more information see this URI: https://www.FreeRTOS.org/a00110.html#configSUPPORT_STATIC_ALLOCATION

Parameters
- **ppxTimerTaskTCBBuffer** -- A handle to a statically allocated TCB buffer
- **ppxTimerTaskStackBuffer** -- A handle to a statically allocated Stack buffer for the idle task
- **pulTimerTaskStackSize** -- A pointer to the number of elements that will fit in the allocated stack buffer

Macros

xTimerStart (xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerStart() starts a timer that was previously created using the xTimerCreate() API function. If the timer had already been started and was already in the active state, then xTimerStart() has equivalent functionality to the xTimerReset() API function.

Starting a timer ensures the timer is in the active state. If the timer is not stopped, deleted, or reset in the mean time, the callback function associated with the timer will get called 'n' ticks after xTimerStart() was called, where 'n' is the timers defined period.

It is valid to call xTimerStart() before the scheduler has been started, but when this is done the timer will not actually start until the scheduler is started, and the timers expiry time will be relative to when the scheduler is started, not relative to when xTimerStart() was called.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerStart() to be available.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters

- **`xTimer`** -- The handle of the timer being started/restarted.
- **`xTicksToWait`** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the start command to be successfully sent to the timer command queue, should the queue already be full when xTimerStart() was called. xTicksToWait is ignored if xTimerStart() is called before the scheduler is started.

Returns pdFAIL will be returned if the start command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStart() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerStop`(xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerStop() stops a timer that was previously started using either of the The xTimerStart(), xTimerReset(), xTimerStartFromISR(), xTimerResetFromISR(), xTimerChangePeriod() or xTimerChangePeriodFromISR() API functions.

Stopping a timer ensures the timer is not in the active state.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerStop() to be available.

Example usage:

See the xTimerCreate() API function example usage scenario.

Parameters

- **`xTimer`** -- The handle of the timer being stopped.
- **`xTicksToWait`** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the stop command to be successfully sent to the timer command queue, should the queue already be full when xTimerStop() was called. xTicksToWait is ignored if xTimerStop() is called before the scheduler is started.

Returns pdFAIL will be returned if the stop command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerChangePeriod (xTimer, xNewPeriod, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerChangePeriod() changes the period of a timer that was previously created using the xTimerCreate() API function.

xTimerChangePeriod() can be called to change the period of an active or dormant state timer.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerChangePeriod() to be available.

Example usage:

```
* // This function assumes xTimer has already been created.  If the timer
* // referenced by xTimer is already active when it is called, then the timer
* // is deleted.  If the timer referenced by xTimer is not active when it is
* // called, then the period of the timer is set to 500ms and the timer is
* // started.
* void vAFunction( TimerHandle_t xTimer )
* {
*     if( xTimerIsTimerActive( xTimer ) != pdFALSE ) // or more simply and␣
↪equivalently "if( xTimerIsTimerActive( xTimer ) )"
*     {
*         // xTimer is already active - delete it.
*         xTimerDelete( xTimer );
*     }
*     else
*     {
*         // xTimer is not active, change its period to 500ms.  This will also
*         // cause the timer to start.  Block for a maximum of 100 ticks if the
*         // change period command cannot immediately be sent to the timer
*         // command queue.
*         if( xTimerChangePeriod( xTimer, 500 / portTICK_PERIOD_MS, 100 ) ==␣
↪pdPASS )
*         {
*             // The command was successfully sent.
*         }
*         else
*         {
*             // The command could not be sent, even after waiting for 100␣
↪ticks
*             // to pass.  Take appropriate action here.
*         }
*     }
* }
*
```

Parameters

- **xTimer** -- The handle of the timer that is having its period changed.
- **xNewPeriod** -- The new period for xTimer. Timer periods are specified in tick periods, so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xNewPeriod should be set to 100. Alternatively, if the timer must expire after 500ms, then xNewPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the change period command to be successfully sent to the timer command queue, should the queue already be full when xTimerChangePeriod() was

called. xTicksToWait is ignored if xTimerChangePeriod() is called before the scheduler is started.

Returns pdFAIL will be returned if the change period command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerDelete (xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerDelete() deletes a timer that was previously created using the xTimerCreate() API function.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerDelete() to be available.

Example usage:

See the xTimerChangePeriod() API function example usage scenario.

Parameters

- **xTimer** -- The handle of the timer being deleted.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the delete command to be successfully sent to the timer command queue, should the queue already be full when xTimerDelete() was called. xTicksToWait is ignored if xTimerDelete() is called before the scheduler is started.

Returns pdFAIL will be returned if the delete command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerReset (xTimer, xTicksToWait)

Timer functionality is provided by a timer service/daemon task. Many of the public FreeRTOS timer API functions send commands to the timer service task through a queue called the timer command queue. The timer command queue is private to the kernel itself and is not directly accessible to application code. The length of the timer command queue is set by the configTIMER_QUEUE_LENGTH configuration constant.

xTimerReset() re-starts a timer that was previously created using the xTimerCreate() API function. If the timer had already been started and was already in the active state, then xTimerReset() will cause the timer to re-evaluate its expiry time so that it is relative to when xTimerReset() was called. If the timer was in the dormant state then xTimerReset() has equivalent functionality to the xTimerStart() API function.

Resetting a timer ensures the timer is in the active state. If the timer is not stopped, deleted, or reset in the mean time, the callback function associated with the timer will get called 'n' ticks after xTimerReset() was called, where 'n' is the timers defined period.

It is valid to call xTimerReset() before the scheduler has been started, but when this is done the timer will not actually start until the scheduler is started, and the timers expiry time will be relative to when the scheduler is started, not relative to when xTimerReset() was called.

The configUSE_TIMERS configuration constant must be set to 1 for xTimerReset() to be available.

Example usage:

```
* // When a key is pressed, an LCD back-light is switched on.  If 5 seconds␣
↪pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer.
*
* TimerHandle_t xBacklightTimer = NULL;
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press event handler.
* void vKeyPressEventHandler( char cKey )
* {
*     // Ensure the LCD back-light is on, then reset the timer that is
*     // responsible for turning the back-light off after 5 seconds of
*     // key inactivity.  Wait 10 ticks for the command to be successfully sent
*     // if it cannot be sent immediately.
*     vSetBacklightState( BACKLIGHT_ON );
*     if( xTimerReset( xBacklightTimer, 100 ) != pdPASS )
*     {
*         // The reset command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // Perform the rest of the key processing here.
* }
*
* void main( void )
* {
* int32_t x;
*
*     // Create then start the one-shot timer that is responsible for turning
*     // the back-light off if no keys are pressed within a 5 second period.
*     xBacklightTimer = xTimerCreate( "BacklightTimer",           // Just a␣
↪text name, not used by the kernel.
*                                     ( 5000 / portTICK_PERIOD_MS), // The␣
↪timer period in ticks.
*                                     pdFALSE,                    // The timer␣
↪is a one-shot timer.
*                                     0,                          // The id is␣
↪not used by the callback so can take any value.
*                                     vBacklightTimerCallback     // The␣
↪callback function that switches the LCD back-light off.
*                                   );
*
*     if( xBacklightTimer == NULL )
*     {
*         // The timer was not created.
*     }
*     else
*     {
*         // Start the timer.  No block time is specified, and even if one was
*         // it would be ignored because the scheduler has not yet been
*         // started.
*         if( xTimerStart( xBacklightTimer, 0 ) != pdPASS )
*         {
```

(continues on next page)

(continued from previous page)

```
*              // The timer could not be set into the Active state.
*          }
*      }
*
*      // ...
*      // Create tasks here.
*      // ...
*
*      // Starting the scheduler will start the timer running as it has already
*      // been set into the active state.
*      vTaskStartScheduler();
*
*      // Should not reach here.
*      for( ;; );
* }
*
```

Parameters

- **xTimer** -- The handle of the timer being reset/started/restarted.
- **xTicksToWait** -- Specifies the time, in ticks, that the calling task should be held in the Blocked state to wait for the reset command to be successfully sent to the timer command queue, should the queue already be full when xTimerReset() was called. xTicksToWait is ignored if xTimerReset() is called before the scheduler is started.

Returns pdFAIL will be returned if the reset command could not be sent to the timer command queue even after xTicksToWait ticks had passed. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStart() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerStartFromISR(xTimer, pxHigherPriorityTaskWoken)

A version of xTimerStart() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xBacklightTimer has already been created.  When a
* // key is pressed, an LCD back-light is switched on.  If 5 seconds pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer, and unlike the example given for
* // the xTimerReset() function, the key press event handler is an interrupt
* // service routine.
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press interrupt service routine.
* void vKeyPressEventInterruptHandler( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // Ensure the LCD back-light is on, then restart the timer that is
```

(continues on next page)

(continued from previous page)

```
*      // responsible for turning the back-light off after 5 seconds of
*      // key inactivity.  This is an interrupt service routine so can only
*      // call FreeRTOS API functions that end in "FromISR".
*      vSetBacklightState( BACKLIGHT_ON );
*
*      // xTimerStartFromISR() or xTimerResetFromISR() could be called here
*      // as both cause the timer to re-calculate its expiry time.
*      // xHigherPriorityTaskWoken was initialised to pdFALSE when it was
*      // declared (in this function).
*      if( xTimerStartFromISR( xBacklightTimer, &xHigherPriorityTaskWoken ) !=␣
↪pdPASS )
*      {
*          // The start command was not executed successfully.  Take appropriate
*          // action here.
*      }
*
*      // Perform the rest of the key processing here.
*
*      // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*      // should be performed.  The syntax required to perform a context switch
*      // from inside an ISR varies from port to port, and from compiler to
*      // compiler.  Inspect the demos for the port you are using to find the
*      // actual syntax required.
*      if( xHigherPriorityTaskWoken != pdFALSE )
*      {
*          // Call the interrupt safe yield function here (actual function
*          // depends on the FreeRTOS port being used).
*      }
* }
*
```

Parameters

- **xTimer** -- The handle of the timer being started/restarted.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerStartFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/daemon task out of the Blocked state. If calling xTimerStartFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerStartFromISR() function. If xTimerStartFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns pdFAIL will be returned if the start command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerStartFromISR() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerStopFromISR(xTimer, pxHigherPriorityTaskWoken)

A version of xTimerStop() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xTimer has already been created and started.  When
* // an interrupt occurs, the timer should be simply stopped.
```

(continues on next page)

(continued from previous page)

```
*
* // The interrupt service routine that stops the timer.
* void vAnExampleInterruptServiceRoutine( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // The interrupt has occurred - simply stop the timer.
*     // xHigherPriorityTaskWoken was set to pdFALSE where it was defined
*     // (within this function).  As this is an interrupt service routine, only
*     // FreeRTOS API functions that end in "FromISR" can be used.
*     if( xTimerStopFromISR( xTimer, &xHigherPriorityTaskWoken ) != pdPASS )
*     {
*         // The stop command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters

- **xTimer** -- The handle of the timer being stopped.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerStopFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/daemon task out of the Blocked state. If calling xTimerStopFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerStopFromISR() function. If xTimerStopFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns pdFAIL will be returned if the stop command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

xTimerChangePeriodFromISR(xTimer, xNewPeriod, pxHigherPriorityTaskWoken)

A version of xTimerChangePeriod() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xTimer has already been created and started.  When
* // an interrupt occurs, the period of xTimer should be changed to 500ms.
*
* // The interrupt service routine that changes the period of xTimer.
* void vAnExampleInterruptServiceRoutine( void )
```

(continues on next page)

(continued from previous page)

```
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // The interrupt has occurred - change the period of xTimer to 500ms.
*     // xHigherPriorityTaskWoken was set to pdFALSE where it was defined
*     // (within this function).  As this is an interrupt service routine, only
*     // FreeRTOS API functions that end in "FromISR" can be used.
*     if( xTimerChangePeriodFromISR( xTimer, &xHigherPriorityTaskWoken ) !=␣
↪pdPASS )
*     {
*         // The command to change the timers period was not executed
*         // successfully.  Take appropriate action here.
*     }
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters

- **`xTimer`** -- The handle of the timer that is having its period changed.
- **`xNewPeriod`** -- The new period for xTimer. Timer periods are specified in tick periods, so the constant portTICK_PERIOD_MS can be used to convert a time that has been specified in milliseconds. For example, if the timer must expire after 100 ticks, then xNewPeriod should be set to 100. Alternatively, if the timer must expire after 500ms, then xNewPeriod can be set to (500 / portTICK_PERIOD_MS) provided configTICK_RATE_HZ is less than or equal to 1000.
- **`pxHigherPriorityTaskWoken`** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerChangePeriodFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/ daemon task out of the Blocked state. If calling xTimerChangePeriodFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/daemon task has a priority equal to or greater than the currently executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerChangePeriodFromISR() function. If xTimerChangePeriodFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns pdFAIL will be returned if the command to change the timers period could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

`xTimerResetFromISR`(xTimer, pxHigherPriorityTaskWoken)

A version of xTimerReset() that can be called from an interrupt service routine.

Example usage:

```
* // This scenario assumes xBacklightTimer has already been created.  When a
* // key is pressed, an LCD back-light is switched on.  If 5 seconds pass
* // without a key being pressed, then the LCD back-light is switched off.  In
* // this case, the timer is a one-shot timer, and unlike the example given for
* // the xTimerReset() function, the key press event handler is an interrupt
* // service routine.
*
* // The callback function assigned to the one-shot timer.  In this case the
* // parameter is not used.
* void vBacklightTimerCallback( TimerHandle_t pxTimer )
* {
*     // The timer expired, therefore 5 seconds must have passed since a key
*     // was pressed.  Switch off the LCD back-light.
*     vSetBacklightState( BACKLIGHT_OFF );
* }
*
* // The key press interrupt service routine.
* void vKeyPressEventInterruptHandler( void )
* {
* BaseType_t xHigherPriorityTaskWoken = pdFALSE;
*
*     // Ensure the LCD back-light is on, then reset the timer that is
*     // responsible for turning the back-light off after 5 seconds of
*     // key inactivity.  This is an interrupt service routine so can only
*     // call FreeRTOS API functions that end in "FromISR".
*     vSetBacklightState( BACKLIGHT_ON );
*
*     // xTimerStartFromISR() or xTimerResetFromISR() could be called here
*     // as both cause the timer to re-calculate its expiry time.
*     // xHigherPriorityTaskWoken was initialised to pdFALSE when it was
*     // declared (in this function).
*     if( xTimerResetFromISR( xBacklightTimer, &xHigherPriorityTaskWoken ) !=␣
↪pdPASS )
*     {
*         // The reset command was not executed successfully.  Take appropriate
*         // action here.
*     }
*
*     // Perform the rest of the key processing here.
*
*     // If xHigherPriorityTaskWoken equals pdTRUE, then a context switch
*     // should be performed.  The syntax required to perform a context switch
*     // from inside an ISR varies from port to port, and from compiler to
*     // compiler.  Inspect the demos for the port you are using to find the
*     // actual syntax required.
*     if( xHigherPriorityTaskWoken != pdFALSE )
*     {
*         // Call the interrupt safe yield function here (actual function
*         // depends on the FreeRTOS port being used).
*     }
* }
*
```

Parameters

- **xTimer** -- The handle of the timer that is to be started, reset, or restarted.
- **pxHigherPriorityTaskWoken** -- The timer service/daemon task spends most of its time in the Blocked state, waiting for messages to arrive on the timer command queue. Calling xTimerResetFromISR() writes a message to the timer command queue, so has the potential to transition the timer service/daemon task out of the Blocked state. If calling xTimerResetFromISR() causes the timer service/daemon task to leave the Blocked state, and the timer service/ daemon task has a priority equal to or greater than the currently

executing task (the task that was interrupted), then *pxHigherPriorityTaskWoken will get set to pdTRUE internally within the xTimerResetFromISR() function. If xTimerResetFromISR() sets this value to pdTRUE then a context switch should be performed before the interrupt exits.

Returns pdFAIL will be returned if the reset command could not be sent to the timer command queue. pdPASS will be returned if the command was successfully sent to the timer command queue. When the command is actually processed will depend on the priority of the timer service/daemon task relative to other tasks in the system, although the timers expiry time is relative to when xTimerResetFromISR() is actually called. The timer service/daemon task priority is set by the configTIMER_TASK_PRIORITY configuration constant.

Type Definitions

typedef struct tmrTimerControl ***TimerHandle_t**

typedef void (***TimerCallbackFunction_t**)(*TimerHandle_t* xTimer)

Defines the prototype to which timer callback functions must conform.

typedef void (***PendedFunction_t**)(void*, uint32_t)

Defines the prototype to which functions used with the xTimerPendFunctionCallFromISR() function must conform.

Event Group API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/event_groups.h
- This header file can be included with:

```
#include "freertos/event_groups.h"
```

Functions

EventGroupHandle_t **xEventGroupCreate**(void)

Create a new event group.

Internally, within the FreeRTOS implementation, event groups use a [small] block of memory, in which the event group's structure is stored. If an event groups is created using xEventGroupCreate() then the required memory is automatically dynamically allocated inside the xEventGroupCreate() function. (see https://www.FreeRTOS.org/a00111.html). If an event group is created using xEventGroupCreateStatic() then the application writer must instead provide the memory that will get used by the event group. xEventGroupCreateStatic() therefore allows an event group to be created without using any dynamic memory allocation.

Although event groups are not related to ticks, for internal implementation reasons the number of bits available for use in an event group is dependent on the configUSE_16_BIT_TICKS setting in FreeRTOSConfig.h. If configUSE_16_BIT_TICKS is 1 then each event group contains 8 usable bits (bit 0 to bit 7). If configUSE_16_BIT_TICKS is set to 0 then each event group has 24 usable bits (bit 0 to bit 23). The EventBits_t type is used to store event bits within an event group.

Example usage:

```
// Declare a variable to hold the created event group.
EventGroupHandle_t xCreatedEventGroup;

// Attempt to create the event group.
xCreatedEventGroup = xEventGroupCreate();
```

(continues on next page)

(continued from previous page)

```
// Was the event group created successfully?
if( xCreatedEventGroup == NULL )
{
// The event group was not created because there was insufficient
// FreeRTOS heap available.
}
else
{
// The event group was created.
}
```

Returns If the event group was created then a handle to the event group is returned. If there was insufficient FreeRTOS heap available to create the event group then NULL is returned. See https://www.FreeRTOS.org/a00111.html

EventGroupHandle_t **xEventGroupCreateStatic**(StaticEventGroup_t *pxEventGroupBuffer)

Create a new event group.

Internally, within the FreeRTOS implementation, event groups use a [small] block of memory, in which the event group's structure is stored. If an event groups is created using xEventGroupCreate() then the required memory is automatically dynamically allocated inside the xEventGroupCreate() function. (see https://www.FreeRTOS.org/a00111.html). If an event group is created using xEventGroupCreateStatic() then the application writer must instead provide the memory that will get used by the event group. xEventGroupCreateStatic() therefore allows an event group to be created without using any dynamic memory allocation.

Although event groups are not related to ticks, for internal implementation reasons the number of bits available for use in an event group is dependent on the configUSE_16_BIT_TICKS setting in FreeRTOSConfig.h. If configUSE_16_BIT_TICKS is 1 then each event group contains 8 usable bits (bit 0 to bit 7). If configUSE_16_BIT_TICKS is set to 0 then each event group has 24 usable bits (bit 0 to bit 23). The EventBits_t type is used to store event bits within an event group.

Example usage:

```
// StaticEventGroup_t is a publicly accessible structure that has the same
// size and alignment requirements as the real event group structure.  It is
// provided as a mechanism for applications to know the size of the event
// group (which is dependent on the architecture and configuration file
// settings) without breaking the strict data hiding policy by exposing the
// real event group internals.  This StaticEventGroup_t variable is passed
// into the xSemaphoreCreateEventGroupStatic() function and is used to store
// the event group's data structures
StaticEventGroup_t xEventGroupBuffer;

// Create the event group without dynamically allocating any memory.
xEventGroup = xEventGroupCreateStatic( &xEventGroupBuffer );
```

Parameters **pxEventGroupBuffer** -- pxEventGroupBuffer must point to a variable of type StaticEventGroup_t, which will be then be used to hold the event group's data structures, removing the need for the memory to be allocated dynamically.

Returns If the event group was created then a handle to the event group is returned. If pxEventGroupBuffer was NULL then NULL is returned.

EventBits_t **xEventGroupWaitBits**(*EventGroupHandle_t* xEventGroup, const *EventBits_t* uxBitsToWaitFor, const BaseType_t xClearOnExit, const BaseType_t xWaitForAllBits, TickType_t xTicksToWait)

[Potentially] block to wait for one or more bits to be set within a previously created event group.

This function cannot be called from an interrupt.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;
const TickType_t xTicksToWait = 100 / portTICK_PERIOD_MS;

// Wait a maximum of 100ms for either bit 0 or bit 4 to be set within
// the event group.  Clear the bits before exiting.
     uxBits = xEventGroupWaitBits(
                 xEventGroup,    // The event group being tested.
                 BIT_0 | BIT_4,  // The bits within the event group to wait␣
↪for.
                 pdTRUE,         // BIT_0 and BIT_4 should be cleared before␣
↪returning.
                 pdFALSE,        // Don't wait for both bits, either bit will␣
↪do.
                 xTicksToWait ); // Wait a maximum of 100ms for either bit to␣
↪be set.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// xEventGroupWaitBits() returned because both bits were set.
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// xEventGroupWaitBits() returned because just BIT_0 was set.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// xEventGroupWaitBits() returned because just BIT_4 was set.
     }
else
     {
// xEventGroupWaitBits() returned because xTicksToWait ticks passed
// without either BIT_0 or BIT_4 becoming set.
     }
}
```

Parameters

- **xEventGroup** -- The event group in which the bits are being tested. The event group must have previously been created using a call to xEventGroupCreate().
- **uxBitsToWaitFor** -- A bitwise value that indicates the bit or bits to test inside the event group. For example, to wait for bit 0 and/or bit 2 set uxBitsToWaitFor to 0x05. To wait for bits 0 and/or bit 1 and/or bit 2 set uxBitsToWaitFor to 0x07. Etc.
- **xClearOnExit** -- If xClearOnExit is set to pdTRUE then any bits within uxBitsToWaitFor that are set within the event group will be cleared before xEventGroupWaitBits() returns if the wait condition was met (if the function returns for a reason other than a timeout). If xClearOnExit is set to pdFALSE then the bits set in the event group are not altered when the call to xEventGroupWaitBits() returns.
- **xWaitForAllBits** -- If xWaitForAllBits is set to pdTRUE then xEventGroupWaitBits() will return when either all the bits in uxBitsToWaitFor are set or the specified block time expires. If xWaitForAllBits is set to pdFALSE then xEventGroupWaitBits() will return when any one of the bits set in uxBitsToWaitFor is set or the specified block time expires. The block time is specified by the xTicksToWait parameter.

- **xTicksToWait** -- The maximum amount of time (specified in 'ticks') to wait for one/all (depending on the xWaitForAllBits value) of the bits specified by uxBitsToWaitFor to become set. A value of portMAX_DELAY can be used to block indefinitely (provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h).

Returns The value of the event group at the time either the bits being waited for became set, or the block time expired. Test the return value to know which bits were set. If xEventGroupWaitBits() returned because its timeout expired then not all the bits being waited for will be set. If xEventGroupWaitBits() returned because the bits it was waiting for were set then the returned value is the event group value before any bits were automatically cleared in the case that xClearOnExit parameter was set to pdTRUE.

EventBits_t **xEventGroupClearBits** (*EventGroupHandle_t* xEventGroup, const *EventBits_t* uxBitsToClear)

Clear bits within an event group. This function cannot be called from an interrupt.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;

// Clear bit 0 and bit 4 in xEventGroup.
     uxBits = xEventGroupClearBits(
                               xEventGroup,    // The event group being updated.
                               BIT_0 | BIT_4 );// The bits being cleared.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// Both bit 0 and bit 4 were set before xEventGroupClearBits() was
// called.  Both will now be clear (not set).
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// Bit 0 was set before xEventGroupClearBits() was called.  It will
// now be clear.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// Bit 4 was set before xEventGroupClearBits() was called.  It will
// now be clear.
     }
else
     {
// Neither bit 0 nor bit 4 were set in the first place.
     }
}
```

Parameters

- **xEventGroup** -- The event group in which the bits are to be cleared.
- **uxBitsToClear** -- A bitwise value that indicates the bit or bits to clear in the event group. For example, to clear bit 3 only, set uxBitsToClear to 0x08. To clear bit 3 and bit 0 set uxBitsToClear to 0x09.

Returns The value of the event group before the specified bits were cleared.

EventBits_t **xEventGroupSetBits** (*EventGroupHandle_t* xEventGroup, const *EventBits_t* uxBitsToSet)

Set bits within an event group. This function cannot be called from an interrupt. xEventGroupSetBitsFromISR() is a version that can be called from an interrupt.

Setting bits in an event group will automatically unblock tasks that are blocked waiting for the bits.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
EventBits_t uxBits;

// Set bit 0 and bit 4 in xEventGroup.
     uxBits = xEventGroupSetBits(
                          xEventGroup,    // The event group being updated.
                          BIT_0 | BIT_4 );// The bits being set.

if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
     {
// Both bit 0 and bit 4 remained set when the function returned.
     }
else if( ( uxBits & BIT_0 ) != 0 )
     {
// Bit 0 remained set when the function returned, but bit 4 was
// cleared.  It might be that bit 4 was cleared automatically as a
// task that was waiting for bit 4 was removed from the Blocked
// state.
     }
else if( ( uxBits & BIT_4 ) != 0 )
     {
// Bit 4 remained set when the function returned, but bit 0 was
// cleared.  It might be that bit 0 was cleared automatically as a
// task that was waiting for bit 0 was removed from the Blocked
// state.
     }
else
     {
// Neither bit 0 nor bit 4 remained set.  It might be that a task
// was waiting for both of the bits to be set, and the bits were
// cleared as the task left the Blocked state.
     }
}
```

Parameters

- **xEventGroup** -- The event group in which the bits are to be set.
- **uxBitsToSet** -- A bitwise value that indicates the bit or bits to set. For example, to set bit 3 only, set uxBitsToSet to 0x08. To set bit 3 and bit 0 set uxBitsToSet to 0x09.

Returns The value of the event group at the time the call to xEventGroupSetBits() returns. There are two reasons why the returned value might have the bits specified by the uxBitsToSet parameter cleared. First, if setting a bit results in a task that was waiting for the bit leaving the blocked state then it is possible the bit will be cleared automatically (see the xClearBitOnExit parameter of xEventGroupWaitBits()). Second, any unblocked (or otherwise Ready state) task that has a priority above that of the task that called xEventGroupSetBits() will execute and may change the event group value before the call to xEventGroupSetBits() returns.

EventBits_t **xEventGroupSync**(*EventGroupHandle_t* xEventGroup, const *EventBits_t* uxBitsToSet, const *EventBits_t* uxBitsToWaitFor, TickType_t xTicksToWait)

Atomically set bits within an event group, then wait for a combination of bits to be set within the same event group. This functionality is typically used to synchronise multiple tasks, where each task has to wait for the other tasks to reach a synchronisation point before proceeding.

This function cannot be used from an interrupt.

The function will return before its block time expires if the bits specified by the uxBitsToWait parameter are set, or become set within that time. In this case all the bits specified by uxBitsToWait will be automatically cleared before the function returns.

Example usage:

```
// Bits used by the three tasks.
#define TASK_0_BIT     ( 1 << 0 )
#define TASK_1_BIT     ( 1 << 1 )
#define TASK_2_BIT     ( 1 << 2 )

#define ALL_SYNC_BITS ( TASK_0_BIT | TASK_1_BIT | TASK_2_BIT )

// Use an event group to synchronise three tasks.  It is assumed this event
// group has already been created elsewhere.
EventGroupHandle_t xEventBits;

void vTask0( void *pvParameters )
{
EventBits_t uxReturn;
TickType_t xTicksToWait = 100 / portTICK_PERIOD_MS;

for( ;; )
  {
// Perform task functionality here.

// Set bit 0 in the event flag to note this task has reached the
// sync point.  The other two tasks will set the other two bits defined
// by ALL_SYNC_BITS.  All three tasks have reached the synchronisation
// point when all the ALL_SYNC_BITS are set.  Wait a maximum of 100ms
// for this to happen.
     uxReturn = xEventGroupSync( xEventBits, TASK_0_BIT, ALL_SYNC_BITS,␣
↪xTicksToWait );

if( ( uxReturn & ALL_SYNC_BITS ) == ALL_SYNC_BITS )
     {
// All three tasks reached the synchronisation point before the call
// to xEventGroupSync() timed out.
     }
 }
}

void vTask1( void *pvParameters )
{
for( ;; )
  {
// Perform task functionality here.

// Set bit 1 in the event flag to note this task has reached the
// synchronisation point.  The other two tasks will set the other two
// bits defined by ALL_SYNC_BITS.  All three tasks have reached the
// synchronisation point when all the ALL_SYNC_BITS are set.  Wait
// indefinitely for this to happen.
     xEventGroupSync( xEventBits, TASK_1_BIT, ALL_SYNC_BITS, portMAX_DELAY );

// xEventGroupSync() was called with an indefinite block time, so
// this task will only reach here if the synchronisation was made by all
// three tasks, so there is no need to test the return value.
  }
```

(continues on next page)

(continued from previous page)

```
}

void vTask2( void *pvParameters )
{
for( ;; )
  {
// Perform task functionality here.

// Set bit 2 in the event flag to note this task has reached the
// synchronisation point.  The other two tasks will set the other two
// bits defined by ALL_SYNC_BITS.  All three tasks have reached the
// synchronisation point when all the ALL_SYNC_BITS are set.  Wait
// indefinitely for this to happen.
     xEventGroupSync( xEventBits, TASK_2_BIT, ALL_SYNC_BITS, portMAX_DELAY );

// xEventGroupSync() was called with an indefinite block time, so
// this task will only reach here if the synchronisation was made by all
// three tasks, so there is no need to test the return value.
 }
}
```

Parameters

- **xEventGroup** -- The event group in which the bits are being tested. The event group must have previously been created using a call to xEventGroupCreate().
- **uxBitsToSet** -- The bits to set in the event group before determining if, and possibly waiting for, all the bits specified by the uxBitsToWait parameter are set.
- **uxBitsToWaitFor** -- A bitwise value that indicates the bit or bits to test inside the event group. For example, to wait for bit 0 and bit 2 set uxBitsToWaitFor to 0x05. To wait for bits 0 and bit 1 and bit 2 set uxBitsToWaitFor to 0x07. Etc.
- **xTicksToWait** -- The maximum amount of time (specified in 'ticks') to wait for all of the bits specified by uxBitsToWaitFor to become set.

Returns The value of the event group at the time either the bits being waited for became set, or the block time expired. Test the return value to know which bits were set. If xEventGroupSync() returned because its timeout expired then not all the bits being waited for will be set. If xEventGroupSync() returned because all the bits it was waiting for were set then the returned value is the event group value before any bits were automatically cleared.

EventBits_t **xEventGroupGetBitsFromISR**(*EventGroupHandle_t* xEventGroup)

A version of xEventGroupGetBits() that can be called from an ISR.

Parameters **xEventGroup** -- The event group being queried.

Returns The event group bits at the time xEventGroupGetBitsFromISR() was called.

void **vEventGroupDelete**(*EventGroupHandle_t* xEventGroup)

Delete an event group that was previously created by a call to xEventGroupCreate(). Tasks that are blocked on the event group will be unblocked and obtain 0 as the event group's value.

Parameters **xEventGroup** -- The event group being deleted.

BaseType_t **xEventGroupGetStaticBuffer**(*EventGroupHandle_t* xEventGroup, StaticEventGroup_t **ppxEventGroupBuffer)

Retrieve a pointer to a statically created event groups's data structure buffer. It is the same buffer that is supplied at the time of creation.

Parameters

- **xEventGroup** -- The event group for which to retrieve the buffer.
- **ppxEventGroupBuffer** -- Used to return a pointer to the event groups's data structure buffer.

Returns pdTRUE if the buffer was retrieved, pdFALSE otherwise.

Macros

xEventGroupClearBitsFromISR(xEventGroup, uxBitsToClear)

A version of xEventGroupClearBits() that can be called from an interrupt.

Setting bits in an event group is not a deterministic operation because there are an unknown number of tasks that may be waiting for the bit or bits being set. FreeRTOS does not allow nondeterministic operations to be performed while interrupts are disabled, so protects event groups that are accessed from tasks by suspending the scheduler rather than disabling interrupts. As a result event groups cannot be accessed directly from an interrupt service routine. Therefore xEventGroupClearBitsFromISR() sends a message to the timer task to have the clear operation performed in the context of the timer task.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

// An event group which it is assumed has already been created by a call to
// xEventGroupCreate().
EventGroupHandle_t xEventGroup;

void anInterruptHandler( void )
{
// Clear bit 0 and bit 4 in xEventGroup.
     xResult = xEventGroupClearBitsFromISR(
                          xEventGroup,     // The event group being updated.
                          BIT_0 | BIT_4 ); // The bits being set.

if( xResult == pdPASS )
     {
// The message was posted successfully.
          portYIELD_FROM_ISR(pdTRUE);
     }
}
```

Note: If this function returns pdPASS then the timer task is ready to run and a portYIELD_FROM_ISR(pdTRUE) should be executed to perform the needed clear on the event group. This behavior is different from xEventGroupSetBitsFromISR because the parameter xHigherPriorityTaskWoken is not present.

Parameters

- **xEventGroup** -- The event group in which the bits are to be cleared.
- **uxBitsToClear** -- A bitwise value that indicates the bit or bits to clear. For example, to clear bit 3 only, set uxBitsToClear to 0x08. To clear bit 3 and bit 0 set uxBitsToClear to 0x09.

Returns If the request to execute the function was posted successfully then pdPASS is returned, otherwise pdFALSE is returned. pdFALSE will be returned if the timer service queue was full.

xEventGroupSetBitsFromISR(xEventGroup, uxBitsToSet, pxHigherPriorityTaskWoken)

A version of xEventGroupSetBits() that can be called from an interrupt.

Setting bits in an event group is not a deterministic operation because there are an unknown number of tasks that may be waiting for the bit or bits being set. FreeRTOS does not allow nondeterministic operations to be performed in interrupts or from critical sections. Therefore xEventGroupSetBitsFromISR() sends a message to the timer task to have the set operation performed in the context of the timer task - where a scheduler lock is used in place of a critical section.

Example usage:

```
#define BIT_0 ( 1 << 0 )
#define BIT_4 ( 1 << 4 )

// An event group which it is assumed has already been created by a call to
// xEventGroupCreate().
EventGroupHandle_t xEventGroup;

void anInterruptHandler( void )
{
BaseType_t xHigherPriorityTaskWoken, xResult;

// xHigherPriorityTaskWoken must be initialised to pdFALSE.
     xHigherPriorityTaskWoken = pdFALSE;

// Set bit 0 and bit 4 in xEventGroup.
     xResult = xEventGroupSetBitsFromISR(
                          xEventGroup,    // The event group being updated.
                          BIT_0 | BIT_4   // The bits being set.
                          &xHigherPriorityTaskWoken );

// Was the message posted successfully?
if( xResult == pdPASS )
     {
// If xHigherPriorityTaskWoken is now set to pdTRUE then a context
// switch should be requested.  The macro used is port specific and
// will be either portYIELD_FROM_ISR() or portEND_SWITCHING_ISR() -
// refer to the documentation page for the port being used.
          portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
     }
}
```

Parameters

- **xEventGroup** -- The event group in which the bits are to be set.
- **uxBitsToSet** -- A bitwise value that indicates the bit or bits to set. For example, to set bit 3 only, set uxBitsToSet to 0x08. To set bit 3 and bit 0 set uxBitsToSet to 0x09.
- **pxHigherPriorityTaskWoken** -- As mentioned above, calling this function will result in a message being sent to the timer daemon task. If the priority of the timer daemon task is higher than the priority of the currently running task (the task the interrupt interrupted) then *pxHigherPriorityTaskWoken will be set to pdTRUE by xEventGroupSetBitsFromISR(), indicating that a context switch should be requested before the interrupt exits. For that reason *pxHigherPriorityTaskWoken must be initialised to pdFALSE. See the example code below.

Returns If the request to execute the function was posted successfully then pdPASS is returned, otherwise pdFALSE is returned. pdFALSE will be returned if the timer service queue was full.

xEventGroupGetBits (xEventGroup)

Returns the current value of the bits in an event group. This function cannot be used from an interrupt.

Parameters

- **xEventGroup** -- The event group being queried.

Returns The event group bits at the time xEventGroupGetBits() was called.

Type Definitions

typedef struct EventGroupDef_t ***EventGroupHandle_t**

typedef TickType_t **EventBits_t**

Stream Buffer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/stream_buffer.h
- This header file can be included with:

```
#include "freertos/stream_buffer.h"
```

Functions

BaseType_t **xStreamBufferGetStaticBuffers**(*StreamBufferHandle_t* xStreamBuffer, uint8_t **ppucStreamBufferStorageArea, StaticStreamBuffer_t **ppxStaticStreamBuffer)

Retrieve pointers to a statically created stream buffer's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters

- **xStreamBuffer** -- The stream buffer for which to retrieve the buffers.
- **ppucStreamBufferStorageArea** -- Used to return a pointer to the stream buffer's storage area buffer.
- **ppxStaticStreamBuffer** -- Used to return a pointer to the stream buffer's data structure buffer.

Returns pdTRUE if buffers were retrieved, pdFALSE otherwise.

size_t **xStreamBufferSend**(*StreamBufferHandle_t* xStreamBuffer, const void *pvTxData, size_t xDataLengthBytes, TickType_t xTicksToWait)

Sends bytes to a stream buffer. The bytes are copied into the stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferSend() to write to a stream buffer from a task. Use xStreamBufferSendFromISR() to write to a stream buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( StreamBufferHandle_t xStreamBuffer )
{
size_t xBytesSent;
uint8_t ucArrayToSend[] = { 0, 1, 2, 3 };
char *pcStringToSend = "String to send";
const TickType_t x100ms = pdMS_TO_TICKS( 100 );

 // Send an array to the stream buffer, blocking for a maximum of 100ms to
 // wait for enough space to be available in the stream buffer.
 xBytesSent = xStreamBufferSend( xStreamBuffer, ( void * ) ucArrayToSend,␣
↪sizeof( ucArrayToSend ), x100ms );

 if( xBytesSent != sizeof( ucArrayToSend ) )
 {
// The call to xStreamBufferSend() times out before there was enough
// space in the buffer for the data to be written, but it did
```

(continues on next page)

(continued from previous page)

```
// successfully write xBytesSent bytes.
 }

 // Send the string to the stream buffer.  Return immediately if there is not
 // enough space in the buffer.
 xBytesSent = xStreamBufferSend( xStreamBuffer, ( void * ) pcStringToSend,␣
↪strlen( pcStringToSend ), 0 );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The entire string could not be added to the stream buffer because
// there was not enough free space in the buffer, but xBytesSent bytes
// were sent.  Could try again to send the remaining bytes.
 }
}
```

Parameters

- **`xStreamBuffer`** -- The handle of the stream buffer to which a stream is being sent.
- **`pvTxData`** -- A pointer to the buffer that holds the bytes to be copied into the stream buffer.
- **`xDataLengthBytes`** -- The maximum number of bytes to copy from pvTxData into the stream buffer.
- **`xTicksToWait`** -- The maximum amount of time the task should remain in the Blocked state to wait for enough space to become available in the stream buffer, should the stream buffer contain too little space to hold the another xDataLengthBytes bytes. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. If a task times out before it can write all xDataLengthBytes into the buffer it will still write as many bytes as possible. A task does not use any CPU time when it is in the blocked state.

Returns The number of bytes written to the stream buffer. If a task times out before it can write all xDataLengthBytes into the buffer it will still write as many bytes as possible.

size_t **`xStreamBufferSendFromISR`**(*StreamBufferHandle_t* xStreamBuffer, const void *pvTxData, size_t xDataLengthBytes, BaseType_t *const pxHigherPriorityTaskWoken)

Interrupt safe version of the API function that sends a stream of bytes to the stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferSend() to write to a stream buffer from a task. Use xStreamBufferSendFromISR() to write to a stream buffer from an interrupt service routine (ISR).

Example use:

```
// A stream buffer that has already been created.
StreamBufferHandle_t xStreamBuffer;
```

(continues on next page)

(continued from previous page)

```
void vAnInterruptServiceRoutine( void )
{
size_t xBytesSent;
char *pcStringToSend = "String to send";
BaseType_t xHigherPriorityTaskWoken = pdFALSE; // Initialised to pdFALSE.

 // Attempt to send the string to the stream buffer.
 xBytesSent = xStreamBufferSendFromISR( xStreamBuffer,
                                        ( void * ) pcStringToSend,
                                        strlen( pcStringToSend ),
                                        &xHigherPriorityTaskWoken );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// There was not enough free space in the stream buffer for the entire
// string to be written, ut xBytesSent bytes were written.
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xStreamBufferSendFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters

- **xStreamBuffer** -- The handle of the stream buffer to which a stream is being sent.
- **pvTxData** -- A pointer to the data that is to be copied into the stream buffer.
- **xDataLengthBytes** -- The maximum number of bytes to copy from pvTxData into the stream buffer.
- **pxHigherPriorityTaskWoken** -- It is possible that a stream buffer will have a task blocked on it waiting for data. Calling xStreamBufferSendFromISR() can make data available, and so cause a task that was waiting for data to leave the Blocked state. If calling xStreamBufferSendFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xStreamBufferSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xStreamBufferSendFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. This will ensure that the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the example code below for an example.

Returns The number of bytes actually written to the stream buffer, which will be less than xDataLengthBytes if the stream buffer didn't have enough free space for all the bytes to be written.

size_t **xStreamBufferReceive**(*StreamBufferHandle_t* xStreamBuffer, void *pvRxData, size_t xBufferLengthBytes, TickType_t xTicksToWait)

Receives bytes from a stream buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xStreamBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers

then the application writer must place each call to a reading API function (such as xStreamBufferReceive()) inside a critical section and set the receive block time to 0.

Use xStreamBufferReceive() to read from a stream buffer from a task. Use xStreamBufferReceiveFromISR() to read from a stream buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( StreamBuffer_t xStreamBuffer )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
const TickType_t xBlockTime = pdMS_TO_TICKS( 20 );

 // Receive up to another sizeof( ucRxData ) bytes from the stream buffer.
 // Wait in the Blocked state (so not using any CPU processing time) for a
 // maximum of 100ms for the full sizeof( ucRxData ) number of bytes to be
 // available.
 xReceivedBytes = xStreamBufferReceive( xStreamBuffer,
                                        ( void * ) ucRxData,
sizeof( ucRxData ),
                                        xBlockTime );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains another xReceivedBytes bytes of data, which can
// be processed here....
 }
}
```

Parameters

- **xStreamBuffer** -- The handle of the stream buffer from which bytes are to be received.
- **pvRxData** -- A pointer to the buffer into which the received bytes will be copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum number of bytes to receive in one call. xStreamBufferReceive will return as many bytes as possible up to a maximum set by xBufferLengthBytes.
- **xTicksToWait** -- The maximum amount of time the task should remain in the Blocked state to wait for data to become available if the stream buffer is empty. xStreamBufferReceive() will return immediately if xTicksToWait is zero. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. A task does not use any CPU time when it is in the Blocked state.

Returns The number of bytes actually read from the stream buffer, which will be less than xBufferLengthBytes if the call to xStreamBufferReceive() timed out before xBufferLengthBytes were available.

size_t **xStreamBufferReceiveFromISR**(*StreamBufferHandle_t* xStreamBuffer, void *pvRxData, size_t xBufferLengthBytes, BaseType_t *const pxHigherPriorityTaskWoken)

An interrupt safe version of the API function that receives bytes from a stream buffer.

Use xStreamBufferReceive() to read bytes from a stream buffer from a task. Use xStreamBufferReceiveFromISR() to read bytes from a stream buffer from an interrupt service routine (ISR).

Example use:

```
// A stream buffer that has already been created.
StreamBuffer_t xStreamBuffer;

void vAnInterruptServiceRoutine( void )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
BaseType_t xHigherPriorityTaskWoken = pdFALSE;  // Initialised to pdFALSE.

 // Receive the next stream from the stream buffer.
 xReceivedBytes = xStreamBufferReceiveFromISR( xStreamBuffer,
                                               ( void * ) ucRxData,
sizeof( ucRxData ),
                                               &xHigherPriorityTaskWoken );

 if( xReceivedBytes > 0 )
 {
// ucRxData contains xReceivedBytes read from the stream buffer.
// Process the stream here....
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xStreamBufferReceiveFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters

- **xStreamBuffer** -- The handle of the stream buffer from which a stream is being received.
- **pvRxData** -- A pointer to the buffer into which the received bytes are copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum number of bytes to receive in one call. xStreamBufferReceive will return as many bytes as possible up to a maximum set by xBufferLengthBytes.
- **pxHigherPriorityTaskWoken** -- It is possible that a stream buffer will have a task blocked on it waiting for space to become available. Calling xStreamBufferReceiveFromISR() can make space available, and so cause a task that is waiting for space to leave the Blocked state. If calling xStreamBufferReceiveFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xStreamBufferReceiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xStreamBufferReceiveFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. That will ensure the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns The number of bytes read from the stream buffer, if any.

void **vStreamBufferDelete**(*StreamBufferHandle_t* xStreamBuffer)

Deletes a stream buffer that was previously created using a call to xStreamBufferCreate() or xStreamBufferCreateStatic(). If the stream buffer was created using dynamic memory (that is, by xStreamBufferCreate()), then the allocated memory is freed.

A stream buffer handle must not be used after the stream buffer has been deleted.

Parameters **xStreamBuffer** -- The handle of the stream buffer to be deleted.

BaseType_t **xStreamBufferIsFull** (*StreamBufferHandle_t* xStreamBuffer)

Queries a stream buffer to see if it is full. A stream buffer is full if it does not have any free space, and therefore cannot accept any more data.

Parameters **xStreamBuffer** -- The handle of the stream buffer being queried.
Returns If the stream buffer is full then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferIsEmpty** (*StreamBufferHandle_t* xStreamBuffer)

Queries a stream buffer to see if it is empty. A stream buffer is empty if it does not contain any data.

Parameters **xStreamBuffer** -- The handle of the stream buffer being queried.
Returns If the stream buffer is empty then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferReset** (*StreamBufferHandle_t* xStreamBuffer)

Resets a stream buffer to its initial, empty, state. Any data that was in the stream buffer is discarded. A stream buffer can only be reset if there are no tasks blocked waiting to either send to or receive from the stream buffer.

Parameters **xStreamBuffer** -- The handle of the stream buffer being reset.
Returns If the stream buffer is reset then pdPASS is returned. If there was a task blocked waiting to send to or read from the stream buffer then the stream buffer is not reset and pdFAIL is returned.

size_t **xStreamBufferSpacesAvailable** (*StreamBufferHandle_t* xStreamBuffer)

Queries a stream buffer to see how much free space it contains, which is equal to the amount of data that can be sent to the stream buffer before it is full.

Parameters **xStreamBuffer** -- The handle of the stream buffer being queried.
Returns The number of bytes that can be written to the stream buffer before the stream buffer would be full.

size_t **xStreamBufferBytesAvailable** (*StreamBufferHandle_t* xStreamBuffer)

Queries a stream buffer to see how much data it contains, which is equal to the number of bytes that can be read from the stream buffer before the stream buffer would be empty.

Parameters **xStreamBuffer** -- The handle of the stream buffer being queried.
Returns The number of bytes that can be read from the stream buffer before the stream buffer would be empty.

BaseType_t **xStreamBufferSetTriggerLevel** (*StreamBufferHandle_t* xStreamBuffer, size_t xTriggerLevel)

A stream buffer's trigger level is the number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.

A trigger level is set when the stream buffer is created, and can be modified using xStreamBufferSetTrigger-Level().

Parameters
- **xStreamBuffer** -- The handle of the stream buffer being updated.
- **xTriggerLevel** -- The new trigger level for the stream buffer.

Returns If xTriggerLevel was less than or equal to the stream buffer's length then the trigger level will be updated and pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferSendCompletedFromISR** (*StreamBufferHandle_t* xStreamBuffer, BaseType_t *pxHigherPriorityTaskWoken)

For advanced users only.

The sbSEND_COMPLETED() macro is called from within the FreeRTOS APIs when data is sent to a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbSEND_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xStreamBufferSendCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbSEND_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters

- **xStreamBuffer** -- The handle of the stream buffer to which data was written.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xStreamBufferSendCompletedFromISR(). If calling xStreamBufferSendCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

BaseType_t **xStreamBufferReceiveCompletedFromISR**(*StreamBufferHandle_t* xStreamBuffer, BaseType_t *pxHigherPriorityTaskWoken)

For advanced users only.

The sbRECEIVE_COMPLETED() macro is called from within the FreeRTOS APIs when data is read out of a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbRECEIVE_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xStreamBufferReceiveCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbRECEIVE_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters

- **xStreamBuffer** -- The handle of the stream buffer from which data was read.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xStreamBufferReceiveCompletedFromISR(). If calling xStreamBufferReceiveCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

Macros

xStreamBufferCreateWithCallback(xBufferSizeBytes, xTriggerLevelBytes, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new stream buffer using dynamically allocated memory. See xStreamBufferCreateStatic() for a version that uses statically allocated memory (memory that is allocated at compile time).

configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 or left undefined in FreeRTOSConfig.h for xStreamBufferCreate() to be available.

Example use:

```
void vAFunction( void )
{
StreamBufferHandle_t xStreamBuffer;
const size_t xStreamBufferSizeBytes = 100, xTriggerLevel = 10;
```

(continues on next page)

(continued from previous page)

```
 // Create a stream buffer that can hold 100 bytes.  The memory used to hold
 // both the stream buffer structure and the data in the stream buffer is
 // allocated dynamically.
 xStreamBuffer = xStreamBufferCreate( xStreamBufferSizeBytes, xTriggerLevel );

 if( xStreamBuffer == NULL )
 {
// There was not enough heap memory space available to create the
// stream buffer.
 }
 else
 {
// The stream buffer was created successfully and can now be used.
 }
}
```

Parameters

- **xBufferSizeBytes** -- The total number of bytes the stream buffer will be able to hold at any one time.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.
- **pxSendCompletedCallback** -- Callback invoked when number of bytes at least equal to trigger level is sent to the stream buffer. If the parameter is NULL, it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when more than zero bytes are read from a stream buffer. If the parameter is NULL, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns If NULL is returned, then the stream buffer cannot be created because there is insufficient heap memory available for FreeRTOS to allocate the stream buffer data structures and storage area. A non-NULL value being returned indicates that the stream buffer has been created successfully - the returned value should be stored as the handle to the created stream buffer.

xStreamBufferCreateStaticWithCallback(xBufferSizeBytes, xTriggerLevelBytes, pucStreamBufferStorageArea, pxStaticStreamBuffer, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new stream buffer using statically allocated memory. See xStreamBufferCreate() for a version that uses dynamically allocated memory.

configSUPPORT_STATIC_ALLOCATION must be set to 1 in FreeRTOSConfig.h for xStreamBufferCreateStatic() to be available.

Example use:

```
// Used to dimension the array used to hold the streams.  The available space
// will actually be one less than this, so 999.
#define STORAGE_SIZE_BYTES 1000

// Defines the memory that will actually hold the streams within the stream
// buffer.
static uint8_t ucStorageBuffer[ STORAGE_SIZE_BYTES ];

// The variable used to hold the stream buffer structure.
  StaticStreamBuffer_t xStreamBufferStruct;

void MyFunction( void )
  {
  StreamBufferHandle_t xStreamBuffer;
const size_t xTriggerLevel = 1;

   xStreamBuffer = xStreamBufferCreateStatic( sizeof( ucStorageBuffer ),
                                              xTriggerLevel,
                                              ucStorageBuffer,
                                              &xStreamBufferStruct );

// As neither the pucStreamBufferStorageArea or pxStaticStreamBuffer
// parameters were NULL, xStreamBuffer will not be NULL, and can be used to
// reference the created stream buffer in other stream buffer API calls.

// Other code that uses the stream buffer can go here.
  }
```

Parameters

- **xBufferSizeBytes** -- The size, in bytes, of the buffer pointed to by the pucStreamBufferStorageArea parameter.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before a task that is blocked on the stream buffer to wait for data is moved out of the blocked state. For example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 1 then the task will be unblocked when a single byte is written to the buffer or the task's block time expires. As another example, if a task is blocked on a read of an empty stream buffer that has a trigger level of 10 then the task will not be unblocked until the stream buffer contains at least 10 bytes or the task's block time expires. If a reading task's block time expires before the trigger level is reached then the task will still receive however many bytes are actually available. Setting a trigger level of 0 will result in a trigger level of 1 being used. It is not valid to specify a trigger level that is greater than the buffer size.
- **pucStreamBufferStorageArea** -- Must point to a uint8_t array that is at least xBufferSizeBytes big. This is the array to which streams are copied when they are written to the stream buffer.
- **pxStaticStreamBuffer** -- Must point to a variable of type StaticStreamBuffer_t, which will be used to hold the stream buffer's data structure.
- **pxSendCompletedCallback** -- Callback invoked when number of bytes at least equal to trigger level is sent to the stream buffer. If the parameter is NULL, it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when more than zero bytes are read from a stream buffer. If the parameter is NULL, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns If the stream buffer is created successfully then a handle to the created stream buffer is returned. If either pucStreamBufferStorageArea or pxStaticstreamBuffer are NULL then NULL is returned.

Type Definitions

typedef struct StreamBufferDef_t ***StreamBufferHandle_t**

typedef void (***StreamBufferCallbackFunction_t**)(*StreamBufferHandle_t* xStreamBuffer, BaseType_t xIsInsideISR, BaseType_t *const pxHigherPriorityTaskWoken)

Type used as a stream buffer's optional callback.

Message Buffer API

Header File

- components/freertos/FreeRTOS-Kernel/include/freertos/message_buffer.h
- This header file can be included with:

```
#include "freertos/message_buffer.h"
```

Macros

xMessageBufferCreateWithCallback(xBufferSizeBytes, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new message buffer using dynamically allocated memory. See xMessageBufferCreateStatic() for a version that uses statically allocated memory (memory that is allocated at compile time).

configSUPPORT_DYNAMIC_ALLOCATION must be set to 1 or left undefined in FreeRTOSConfig.h for xMessageBufferCreate() to be available.

Example use:

```
void vAFunction( void )
{
MessageBufferHandle_t xMessageBuffer;
const size_t xMessageBufferSizeBytes = 100;

 // Create a message buffer that can hold 100 bytes.  The memory used to hold
 // both the message buffer structure and the messages themselves is allocated
 // dynamically.  Each message added to the buffer consumes an additional 4
 // bytes which are used to hold the length of the message.
 xMessageBuffer = xMessageBufferCreate( xMessageBufferSizeBytes );

 if( xMessageBuffer == NULL )
 {
// There was not enough heap memory space available to create the
// message buffer.
 }
 else
 {
// The message buffer was created successfully and can now be used.
 }
```

Parameters

- **xBufferSizeBytes** -- The total number of bytes (not messages) the message buffer will be able to hold at any one time. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architectures a 10 byte message will take up 14 bytes of message buffer space.

- **pxSendCompletedCallback** -- Callback invoked when a send operation to the message buffer is complete. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, then it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when a receive operation from the message buffer is complete. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns If NULL is returned, then the message buffer cannot be created because there is insufficient heap memory available for FreeRTOS to allocate the message buffer data structures and storage area. A non-NULL value being returned indicates that the message buffer has been created successfully - the returned value should be stored as the handle to the created message buffer.

xMessageBufferCreateStaticWithCallback(xBufferSizeBytes, pucMessageBufferStorageArea, pxStaticMessageBuffer, pxSendCompletedCallback, pxReceiveCompletedCallback)

Creates a new message buffer using statically allocated memory. See xMessageBufferCreate() for a version that uses dynamically allocated memory.

Example use:

```
// Used to dimension the array used to hold the messages.  The available space
// will actually be one less than this, so 999.
#define STORAGE_SIZE_BYTES 1000

// Defines the memory that will actually hold the messages within the message
// buffer.
static uint8_t ucStorageBuffer[ STORAGE_SIZE_BYTES ];

// The variable used to hold the message buffer structure.
StaticMessageBuffer_t xMessageBufferStruct;

void MyFunction( void )
{
MessageBufferHandle_t xMessageBuffer;

 xMessageBuffer = xMessageBufferCreateStatic( sizeof( ucStorageBuffer ),
                                              ucStorageBuffer,
                                              &xMessageBufferStruct );

 // As neither the pucMessageBufferStorageArea or pxStaticMessageBuffer
 // parameters were NULL, xMessageBuffer will not be NULL, and can be used to
 // reference the created message buffer in other message buffer API calls.

 // Other code that uses the message buffer can go here.
}
```

Parameters

- **xBufferSizeBytes** -- The size, in bytes, of the buffer pointed to by the pucMessageBufferStorageArea parameter. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architecture a 10 byte message will take up 14 bytes of message buffer space. The maximum number of bytes that can be stored in the message buffer is actually (xBufferSizeBytes - 1).
- **pucMessageBufferStorageArea** -- Must point to a uint8_t array that is at least xBufferSizeBytes big. This is the array to which messages are copied when they are written

to the message buffer.

- **pxStaticMessageBuffer** -- Must point to a variable of type StaticMessageBuffer_t, which will be used to hold the message buffer's data structure.
- **pxSendCompletedCallback** -- Callback invoked when a new message is sent to the message buffer. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, then it will use the default implementation provided by sbSEND_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.
- **pxReceiveCompletedCallback** -- Callback invoked when a message is read from a message buffer. If the parameter is NULL or xMessageBufferCreate() is called without the parameter, it will use the default implementation provided by sbRECEIVE_COMPLETED macro. To enable the callback, configUSE_SB_COMPLETED_CALLBACK must be set to 1 in FreeRTOSConfig.h.

Returns If the message buffer is created successfully then a handle to the created message buffer is returned. If either pucMessageBufferStorageArea or pxStaticmessageBuffer are NULL then NULL is returned.

xMessageBufferGetStaticBuffers(xMessageBuffer, ppucMessageBufferStorageArea, ppxStaticMessageBuffer)

Retrieve pointers to a statically created message buffer's data structure buffer and storage area buffer. These are the same buffers that are supplied at the time of creation.

Parameters

- **xMessageBuffer** -- The message buffer for which to retrieve the buffers.
- **ppucMessageBufferStorageArea** -- Used to return a pointer to the message buffer's storage area buffer.
- **ppxStaticMessageBuffer** -- Used to return a pointer to the message buffer's data structure buffer.

Returns pdTRUE if buffers were retrieved, pdFALSE otherwise.

xMessageBufferSend(xMessageBuffer, pvTxData, xDataLengthBytes, xTicksToWait)

Sends a discrete message to the message buffer. The message can be any length that fits within the buffer's free space, and is copied into the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferSend() to write to a message buffer from a task. Use xMessageBufferSendFromISR() to write to a message buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( MessageBufferHandle_t xMessageBuffer )
{
size_t xBytesSent;
uint8_t ucArrayToSend[] = { 0, 1, 2, 3 };
char *pcStringToSend = "String to send";
const TickType_t x100ms = pdMS_TO_TICKS( 100 );

 // Send an array to the message buffer, blocking for a maximum of 100ms to
 // wait for enough space to be available in the message buffer.
 xBytesSent = xMessageBufferSend( xMessageBuffer, ( void * ) ucArrayToSend,␣
↪sizeof( ucArrayToSend ), x100ms );
```

(continues on next page)

(continued from previous page)

```
 if( xBytesSent != sizeof( ucArrayToSend ) )
 {
// The call to xMessageBufferSend() times out before there was enough
// space in the buffer for the data to be written.
 }

 // Send the string to the message buffer.  Return immediately if there is
 // not enough space in the buffer.
 xBytesSent = xMessageBufferSend( xMessageBuffer, ( void * ) pcStringToSend,␣
↪strlen( pcStringToSend ), 0 );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The string could not be added to the message buffer because there was
// not enough free space in the buffer.
 }
}
```

Parameters

- **xMessageBuffer** -- The handle of the message buffer to which a message is being sent.
- **pvTxData** -- A pointer to the message that is to be copied into the message buffer.
- **xDataLengthBytes** -- The length of the message. That is, the number of bytes to copy from pvTxData into the message buffer. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architecture setting xDataLengthBytes to 20 will reduce the free space in the message buffer by 24 bytes (20 bytes of message data and 4 bytes to hold the message length).
- **xTicksToWait** -- The maximum amount of time the calling task should remain in the Blocked state to wait for enough space to become available in the message buffer, should the message buffer have insufficient space when xMessageBufferSend() is called. The calling task will never block if xTicksToWait is zero. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. Tasks do not use any CPU time when they are in the Blocked state.

Returns The number of bytes written to the message buffer. If the call to xMessageBufferSend() times out before there was enough space to write the message into the message buffer then zero is returned. If the call did not time out then xDataLengthBytes is returned.

xMessageBufferSendFromISR(xMessageBuffer, pvTxData, xDataLengthBytes, pxHigherPriorityTaskWoken)

Interrupt safe version of the API function that sends a discrete message to the message buffer. The message can be any length that fits within the buffer's free space, and is copied into the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferSend() to write to a message buffer from a task. Use xMessageBufferSendFromISR() to write to a message buffer from an interrupt service routine (ISR).

Example use:

```
// A message buffer that has already been created.
MessageBufferHandle_t xMessageBuffer;

void vAnInterruptServiceRoutine( void )
{
size_t xBytesSent;
char *pcStringToSend = "String to send";
BaseType_t xHigherPriorityTaskWoken = pdFALSE; // Initialised to pdFALSE.

 // Attempt to send the string to the message buffer.
 xBytesSent = xMessageBufferSendFromISR( xMessageBuffer,
                                         ( void * ) pcStringToSend,
                                         strlen( pcStringToSend ),
                                         &xHigherPriorityTaskWoken );

 if( xBytesSent != strlen( pcStringToSend ) )
 {
// The string could not be added to the message buffer because there was
// not enough free space in the buffer.
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xMessageBufferSendFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters

- **`xMessageBuffer`** -- The handle of the message buffer to which a message is being sent.
- **`pvTxData`** -- A pointer to the message that is to be copied into the message buffer.
- **`xDataLengthBytes`** -- The length of the message. That is, the number of bytes to copy from pvTxData into the message buffer. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so on most 32-bit architecture setting xDataLengthBytes to 20 will reduce the free space in the message buffer by 24 bytes (20 bytes of message data and 4 bytes to hold the message length).
- **`pxHigherPriorityTaskWoken`** -- It is possible that a message buffer will have a task blocked on it waiting for data. Calling xMessageBufferSendFromISR() can make data available, and so cause a task that was waiting for data to leave the Blocked state. If calling xMessageBufferSendFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xMessageBufferSendFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xMessageBufferSendFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. This will ensure that the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns The number of bytes actually written to the message buffer. If the message buffer didn't have enough free space for the message to be stored then 0 is returned, otherwise xDataLengthBytes is returned.

xMessageBufferReceive(xMessageBuffer, pvRxData, xBufferLengthBytes, xTicksToWait)

Receives a discrete message from a message buffer. Messages can be of variable length and are copied out of the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferReceive() to read from a message buffer from a task. Use xMessageBufferReceiveFromISR() to read from a message buffer from an interrupt service routine (ISR).

Example use:

```
void vAFunction( MessageBuffer_t xMessageBuffer )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
const TickType_t xBlockTime = pdMS_TO_TICKS( 20 );

 // Receive the next message from the message buffer.  Wait in the Blocked
 // state (so not using any CPU processing time) for a maximum of 100ms for
 // a message to become available.
 xReceivedBytes = xMessageBufferReceive( xMessageBuffer,
                                         ( void * ) ucRxData,
sizeof( ucRxData ),
                                         xBlockTime );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains a message that is xReceivedBytes long.  Process
// the message here....
 }
}
```

Parameters

- **xMessageBuffer** -- The handle of the message buffer from which a message is being received.
- **pvRxData** -- A pointer to the buffer into which the received message is to be copied.
- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum length of the message that can be received. If xBufferLengthBytes is too small to hold the next message then the message will be left in the message buffer and 0 will be returned.
- **xTicksToWait** -- The maximum amount of time the task should remain in the Blocked state to wait for a message, should the message buffer be empty. xMessageBufferReceive() will return immediately if xTicksToWait is zero and the message buffer is empty. The block time is specified in tick periods, so the absolute time it represents is dependent on the tick frequency. The macro pdMS_TO_TICKS() can be used to convert a time specified in milliseconds into a time specified in ticks. Setting xTicksToWait to portMAX_DELAY will cause the task to wait indefinitely (without timing out), provided INCLUDE_vTaskSuspend is set to 1 in FreeRTOSConfig.h. Tasks do not use any CPU time when they are in the Blocked state.

Returns The length, in bytes, of the message read from the message buffer, if any. If xMessageBufferReceive() times out before a message became available then zero is returned. If the

length of the message is greater than xBufferLengthBytes then the message will be left in the message buffer and zero is returned.

xMessageBufferReceiveFromISR(xMessageBuffer, pvRxData, xBufferLengthBytes, pxHigherPriorityTaskWoken)

An interrupt safe version of the API function that receives a discrete message from a message buffer. Messages can be of variable length and are copied out of the buffer.

NOTE: Uniquely among FreeRTOS objects, the stream buffer implementation (so also the message buffer implementation, as message buffers are built on top of stream buffers) assumes there is only one task or interrupt that will write to the buffer (the writer), and only one task or interrupt that will read from the buffer (the reader). It is safe for the writer and reader to be different tasks or interrupts, but, unlike other FreeRTOS objects, it is not safe to have multiple different writers or multiple different readers. If there are to be multiple different writers then the application writer must place each call to a writing API function (such as xMessageBufferSend()) inside a critical section and set the send block time to 0. Likewise, if there are to be multiple different readers then the application writer must place each call to a reading API function (such as xMessageBufferRead()) inside a critical section and set the receive block time to 0.

Use xMessageBufferReceive() to read from a message buffer from a task. Use xMessageBufferReceiveFromISR() to read from a message buffer from an interrupt service routine (ISR).

Example use:

```
// A message buffer that has already been created.
MessageBuffer_t xMessageBuffer;

void vAnInterruptServiceRoutine( void )
{
uint8_t ucRxData[ 20 ];
size_t xReceivedBytes;
BaseType_t xHigherPriorityTaskWoken = pdFALSE;  // Initialised to pdFALSE.

 // Receive the next message from the message buffer.
 xReceivedBytes = xMessageBufferReceiveFromISR( xMessageBuffer,
                                                ( void * ) ucRxData,
sizeof( ucRxData ),
                                                &xHigherPriorityTaskWoken );

 if( xReceivedBytes > 0 )
 {
// A ucRxData contains a message that is xReceivedBytes long.  Process
// the message here....
 }

 // If xHigherPriorityTaskWoken was set to pdTRUE inside
 // xMessageBufferReceiveFromISR() then a task that has a priority above the
 // priority of the currently executing task was unblocked and a context
 // switch should be performed to ensure the ISR returns to the unblocked
 // task.  In most FreeRTOS ports this is done by simply passing
 // xHigherPriorityTaskWoken into portYIELD_FROM_ISR(), which will test the
 // variables value, and perform the context switch if necessary.  Check the
 // documentation for the port in use for port specific instructions.
 portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

Parameters

- **xMessageBuffer** -- The handle of the message buffer from which a message is being received.
- **pvRxData** -- A pointer to the buffer into which the received message is to be copied.

- **xBufferLengthBytes** -- The length of the buffer pointed to by the pvRxData parameter. This sets the maximum length of the message that can be received. If xBufferLengthBytes is too small to hold the next message then the message will be left in the message buffer and 0 will be returned.
- **pxHigherPriorityTaskWoken** -- It is possible that a message buffer will have a task blocked on it waiting for space to become available. Calling xMessageBufferReceiveFromISR() can make space available, and so cause a task that is waiting for space to leave the Blocked state. If calling xMessageBufferReceiveFromISR() causes a task to leave the Blocked state, and the unblocked task has a priority higher than the currently executing task (the task that was interrupted), then, internally, xMessageBufferReceiveFromISR() will set *pxHigherPriorityTaskWoken to pdTRUE. If xMessageBufferReceiveFromISR() sets this value to pdTRUE, then normally a context switch should be performed before the interrupt is exited. That will ensure the interrupt returns directly to the highest priority Ready state task. *pxHigherPriorityTaskWoken should be set to pdFALSE before it is passed into the function. See the code example below for an example.

Returns The length, in bytes, of the message read from the message buffer, if any.

vMessageBufferDelete(xMessageBuffer)

Deletes a message buffer that was previously created using a call to xMessageBufferCreate() or xMessageBufferCreateStatic(). If the message buffer was created using dynamic memory (that is, by xMessageBufferCreate()), then the allocated memory is freed.

A message buffer handle must not be used after the message buffer has been deleted.

Parameters

- **xMessageBuffer** -- The handle of the message buffer to be deleted.

xMessageBufferIsFull(xMessageBuffer)

Tests to see if a message buffer is full. A message buffer is full if it cannot accept any more messages, of any size, until space is made available by a message being removed from the message buffer.

Parameters

- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns If the message buffer referenced by xMessageBuffer is full then pdTRUE is returned. Otherwise pdFALSE is returned.

xMessageBufferIsEmpty(xMessageBuffer)

Tests to see if a message buffer is empty (does not contain any messages).

Parameters

- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns If the message buffer referenced by xMessageBuffer is empty then pdTRUE is returned. Otherwise pdFALSE is returned.

xMessageBufferReset(xMessageBuffer)

Resets a message buffer to its initial empty state, discarding any message it contained.

A message buffer can only be reset if there are no tasks blocked on it.

Parameters

- **xMessageBuffer** -- The handle of the message buffer being reset.

Returns If the message buffer was reset then pdPASS is returned. If the message buffer could not be reset because either there was a task blocked on the message queue to wait for space to become available, or to wait for a a message to be available, then pdFAIL is returned.

xMessageBufferSpaceAvailable(xMessageBuffer)

message_buffer.h

```
size_t xMessageBufferSpaceAvailable( MessageBufferHandle_t xMessageBuffer );
```

Returns the number of bytes of free space in the message buffer.

Parameters

- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns The number of bytes that can be written to the message buffer before the message buffer would be full. When a message is written to the message buffer an additional sizeof(size_t) bytes are also written to store the message's length. sizeof(size_t) is typically 4 bytes on a 32-bit architecture, so if xMessageBufferSpacesAvailable() returns 10, then the size of the largest message that can be written to the message buffer is 6 bytes.

xMessageBufferSpacesAvailable(xMessageBuffer)

xMessageBufferNextLengthBytes(xMessageBuffer)

Returns the length (in bytes) of the next message in a message buffer. Useful if xMessageBufferReceive() returned 0 because the size of the buffer passed into xMessageBufferReceive() was too small to hold the next message.

Parameters

- **xMessageBuffer** -- The handle of the message buffer being queried.

Returns The length (in bytes) of the next message in the message buffer, or 0 if the message buffer is empty.

xMessageBufferSendCompletedFromISR(xMessageBuffer, pxHigherPriorityTaskWoken)

For advanced users only.

The sbSEND_COMPLETED() macro is called from within the FreeRTOS APIs when data is sent to a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbSEND_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xMessageBufferSendCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbSEND_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters

- **xMessageBuffer** -- The handle of the stream buffer to which data was written.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xMessageBufferSendCompletedFromISR(). If calling xMessageBufferSendCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

xMessageBufferReceiveCompletedFromISR(xMessageBuffer, pxHigherPriorityTaskWoken)

For advanced users only.

The sbRECEIVE_COMPLETED() macro is called from within the FreeRTOS APIs when data is read out of a message buffer or stream buffer. If there was a task that was blocked on the message or stream buffer waiting for data to arrive then the sbRECEIVE_COMPLETED() macro sends a notification to the task to remove it from the Blocked state. xMessageBufferReceiveCompletedFromISR() does the same thing. It is provided to enable application writers to implement their own version of sbRECEIVE_COMPLETED(), and MUST NOT BE USED AT ANY OTHER TIME.

See the example implemented in FreeRTOS/Demo/Minimal/MessageBufferAMP.c for additional information.

Parameters

- **xMessageBuffer** -- The handle of the stream buffer from which data was read.
- **pxHigherPriorityTaskWoken** -- *pxHigherPriorityTaskWoken should be initialised to pdFALSE before it is passed into xMessageBufferReceiveCompletedFromISR(). If calling xMessageBufferReceiveCompletedFromISR() removes a task from the Blocked state, and the task has a priority above the priority of the currently running task, then *pxHigherPriorityTaskWoken will get set to pdTRUE indicating that a context switch should be performed before exiting the ISR.

Returns If a task was removed from the Blocked state then pdTRUE is returned. Otherwise pdFALSE is returned.

Type Definitions

typedef *StreamBufferHandle_t* **`MessageBufferHandle_t`**

Type by which message buffers are referenced. For example, a call to xMessageBufferCreate() returns an MessageBufferHandle_t variable that can then be used as a parameter to xMessageBufferSend(), xMessageBufferReceive(), etc. Message buffer is essentially built as a stream buffer hence its handle is also set to same type as a stream buffer handle.

2.10.13 FreeRTOS (Supplemental Features)

ESP-IDF provides multiple features to supplement the features offered by FreeRTOS. These supplemental features are available on all FreeRTOS implementations supported by ESP-IDF (i.e., ESP-IDF FreeRTOS and Amazon SMP FreeRTOS). This document describes these supplemental features and is split into the following sections:

Contents

- *FreeRTOS (Supplemental Features)*
  - *Overview*
  - *Ring Buffers*
  - *ESP-IDF Tick and Idle Hooks*
  - *TLSP Deletion Callbacks*
  - *IDF Additional API*
  - *Component Specific Properties*
  - *API Reference*

Overview

ESP-IDF adds various new features to supplement the capabilities of FreeRTOS as follows:

- **Ring buffers**: Ring buffers provide a FIFO buffer that can accept entries of arbitrary lengths.
- **ESP-IDF Tick and Idle Hooks**: ESP-IDF provides multiple custom tick interrupt hooks and idle task hooks that are more numerous and more flexible when compared to FreeRTOS tick and idle hooks.
- **Thread Local Storage Pointer (TLSP) Deletion Callbacks**: TLSP Deletion callbacks are run automatically when a task is deleted, thus allowing users to clean up their TLSPs automatically.
- **IDF Additional API**: ESP-IDF specific functions added to augment the features of FreeRTOS.
- **Component Specific Properties**: Currently added only one component specific property `ORIG_INCLUDE_PATH`.

Ring Buffers

FreeRTOS provides stream buffers and message buffers as the primary mechanisms to send arbitrarily sized data between tasks and ISRs. However, FreeRTOS stream buffers and message buffers have the following limitations:

- Strictly single sender and single receiver
- Data is passed by copy
- Unable to reserve buffer space for a deferred send (i.e., send acquire)

Therefore, ESP-IDF provides a separate ring buffer implementation to address the issues above.

ESP-IDF ring buffers are strictly FIFO buffers that supports arbitrarily sized items. Ring buffers are a more memory efficient alternative to FreeRTOS queues in situations where the size of items is variable. The capacity of a ring buffer is not measured by the number of items it can store, but rather by the amount of memory used for storing items.

The ring buffer provides APIs to send an item, or to allocate space for an item in the ring buffer to be filled manually by the user. For efficiency reasons, **items are always retrieved from the ring buffer by reference**. As a result, all retrieved items **must also be returned** to the ring buffer by using *vRingbufferReturnItem()* or *vRingbufferReturnItemFromISR()*, in order for them to be removed from the ring buffer completely.

The ring buffers are split into the three following types:

No-Split buffers guarantee that an item is stored in contiguous memory and does not attempt to split an item under any circumstances. Use No-Split buffers when items must occupy contiguous memory. **Only this buffer type allows reserving buffer space for deferred sending.** Refer to the documentation of the functions *xRingbufferSendAcquire()* and *xRingbufferSendComplete()* for more details.

Allow-Split buffers allow an item to be split in two parts when wrapping around the end of the buffer if there is enough space at the tail and the head of the buffer combined to store the item. Allow-Split buffers are more memory efficient than No-Split buffers but can return an item in two parts when retrieving.

Byte buffers do not store data as separate items. All data is stored as a sequence of bytes, and any number of bytes can be sent or retrieved each time. Use byte buffers when separate items do not need to be maintained, e.g., a byte stream.

Note: No-Split buffers and Allow-Split buffers always store items at 32-bit aligned addresses. Therefore, when retrieving an item, the item pointer is guaranteed to be 32-bit aligned. This is useful especially when you need to send some data to the DMA.

Note: Each item stored in No-Split or Allow-Split buffers **requires an additional 8 bytes for a header**. Item sizes are also rounded up to a 32-bit aligned size, i.e., multiple of 4 bytes. However the true item size is recorded within the header. The sizes of No-Split and Allow-Split buffers will also be rounded up when created.

Usage The following example demonstrates the usage of *xRingbufferCreate()* and *xRingbufferSend()* to create a ring buffer and then send an item to it:

```
#include "freertos/ringbuf.h"
static char tx_item[] = "test_item";

...

    //Create ring buffer
    RingbufHandle_t buf_handle;
    buf_handle = xRingbufferCreate(1028, RINGBUF_TYPE_NOSPLIT);
    if (buf_handle == NULL) {
        printf("Failed to create ring buffer\n");
    }

    //Send an item
    UBaseType_t res =  xRingbufferSend(buf_handle, tx_item, sizeof(tx_item), pdMS_
↪TO_TICKS(1000));
    if (res != pdTRUE) {
        printf("Failed to send item\n");
    }
```

The following example demonstrates the usage of *xRingbufferSendAcquire()* and *xRingbufferSendComplete()* instead of *xRingbufferSend()* to acquire memory on the ring buffer (of type *RINGBUF_TYPE_NOSPLIT*) and then send an item to it. This adds one more step, but allows getting the address of the memory to write to, and writing to the memory yourself.

```
#include "freertos/ringbuf.h"
#include "soc/lldesc.h"

typedef struct {
    lldesc_t dma_desc;
    uint8_t buf[1];
} dma_item_t;

#define DMA_ITEM_SIZE(N) (sizeof(lldesc_t)+(((N)+3)&(~3)))

...

    //Retrieve space for DMA descriptor and corresponding data buffer
    //This has to be done with SendAcquire, or the address may be different when we copy
    dma_item_t *item;
    UBaseType_t res =  xRingbufferSendAcquire(buf_handle,
                       (void**) &item, DMA_ITEM_SIZE(buffer_size), pdMS_TO_TICKS(1000));
    if (res != pdTRUE) {
        printf("Failed to acquire memory for item\n");
    }
    item->dma_desc = (lldesc_t) {
        .size = buffer_size,
        .length = buffer_size,
        .eof = 0,
        .owner = 1,
        .buf = item->buf,
    };
    //Actually send to the ring buffer for consumer to use
    res = xRingbufferSendComplete(buf_handle, &item);
    if (res != pdTRUE) {
        printf("Failed to send item\n");
    }
```

The following example demonstrates retrieving and returning an item from a **No-Split ring buffer** using *xRingbufferReceive()* and *vRingbufferReturnItem()*

```
...

    //Receive an item from no-split ring buffer
    size_t item_size;
    char *item = (char *)xRingbufferReceive(buf_handle, &item_size, pdMS_TO_TICKS(1000));

    //Check received item
    if (item != NULL) {
        //Print item
        for (int i = 0; i < item_size; i++) {
            printf("%c", item[i]);
        }
        printf("\n");
        //Return Item
        vRingbufferReturnItem(buf_handle, (void *)item);
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

The following example demonstrates retrieving and returning an item from an **Allow-Split ring buffer** using *xRingbufferReceiveSplit()* and *vRingbufferReturnItem()*

```
...

    //Receive an item from allow-split ring buffer
    size_t item_size1, item_size2;
    char *item1, *item2;
    BaseType_t ret = xRingbufferReceiveSplit(buf_handle, (void **)&item1, (void
↪**)&item2, &item_size1, &item_size2, pdMS_TO_TICKS(1000));

    //Check received item
    if (ret == pdTRUE && item1 != NULL) {
        for (int i = 0; i < item_size1; i++) {
            printf("%c", item1[i]);
        }
        vRingbufferReturnItem(buf_handle, (void *)item1);
        //Check if item was split
        if (item2 != NULL) {
            for (int i = 0; i < item_size2; i++) {
                printf("%c", item2[i]);
            }
            vRingbufferReturnItem(buf_handle, (void *)item2);
        }
        printf("\n");
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

The following example demonstrates retrieving and returning an item from a **byte buffer** using *xRingbufferReceiveUpTo()* and *vRingbufferReturnItem()*

```
...

    //Receive data from byte buffer
    size_t item_size;
    char *item = (char *)xRingbufferReceiveUpTo(buf_handle, &item_size, pdMS_TO_
↪TICKS(1000), sizeof(tx_item));

    //Check received data
    if (item != NULL) {
        //Print item
        for (int i = 0; i < item_size; i++) {
            printf("%c", item[i]);
        }
        printf("\n");
        //Return Item
        vRingbufferReturnItem(buf_handle, (void *)item);
    } else {
        //Failed to receive item
        printf("Failed to receive item\n");
    }
```

For ISR safe versions of the functions used above, call *xRingbufferSendFromISR()*, *xRingbufferReceiveFromISR()*, *xRingbufferReceiveSplitFromISR()*, *xRingbufferReceiveUpToFromISR()*, and *vRingbufferReturnItemFromISR()*.

Note: Two calls to `RingbufferReceive[UpTo][FromISR]()` are required if the bytes wraps around the end of the ring buffer.

Sending to Ring Buffer The following diagrams illustrate the differences between No-Split and Allow-Split buffers as compared to byte buffers with regard to sending items or data. The diagrams assume that three items of sizes **18, 3, and 27 bytes** are sent respectively to a **buffer of 128 bytes**:

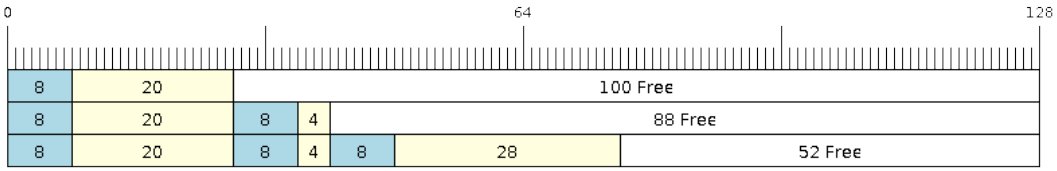


Fig. 41: Sending items to No-Split or Allow-Split ring buffers

For No-Split and Allow-Split buffers, a header of 8 bytes precedes every data item. Furthermore, the space occupied by each item is **rounded up to the nearest 32-bit aligned size** in order to maintain overall 32-bit alignment. However, the true size of the item is recorded inside the header which will be returned when the item is retrieved.

Referring to the diagram above, the 18, 3, and 27 byte items are **rounded up to 20, 4, and 28 bytes** respectively. An 8 byte header is then added in front of each item.

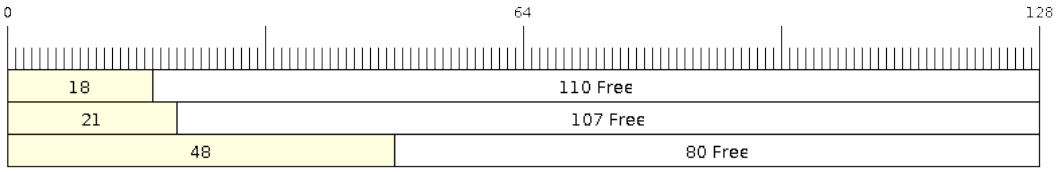


Fig. 42: Sending items to byte buffers

Byte buffers treat data as a sequence of bytes and does not incur any overhead (no headers). As a result, all data sent to a byte buffer is merged into a single item.

Referring to the diagram above, the 18, 3, and 27 byte items are sequentially written to the byte buffer and **merged into a single item of 48 bytes**.

Using SendAcquire and SendComplete Items in No-Split buffers are acquired (by `SendAcquire`) in strict FIFO order and must be sent to the buffer by `SendComplete` for the data to be accessible by the consumer. Multiple items can be sent or acquired without calling `SendComplete`, and the items do not necessarily need to be completed in the order they were acquired. However, the receiving of data items must occur in FIFO order, therefore not calling `SendComplete` for the earliest acquired item prevents the subsequent items from being received.

The following diagrams illustrate what will happen when `SendAcquire` and `SendComplete` do not happen in the same order. At the beginning, there is already a data item of 16 bytes sent to the ring buffer. Then `SendAcquire` is called to acquire space of 20, 8, 24 bytes on the ring buffer.

After that, we fill (use) the buffers, and send them to the ring buffer by `SendComplete` in the order of 8, 24, 20. When 8 bytes and 24 bytes data are sent, the consumer still can only get the 16 bytes data item. Hence, if `SendComplete` is not called for the 20 bytes, it will not be available, nor will the data items following the 20 bytes item.

When the 20 bytes item is finally completed, all the 3 data items can be received now, in the order of 20, 8, 24 bytes, right after the 16 bytes item existing in the buffer at the beginning.

Allow-Split buffers and byte buffers do not allow using `SendAcquire` or `SendComplete` since acquired buffers are required to be complete (not wrapped).

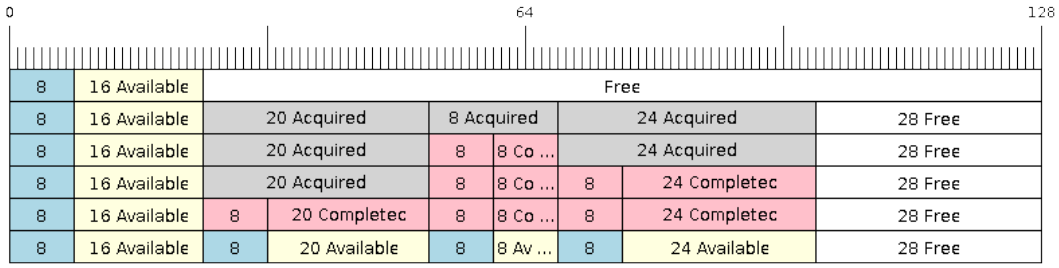


Fig. 43: SendAcquire/SendComplete items in No-Split ring buffers

Wrap Around The following diagrams illustrate the differences between No-Split, Allow-Split, and byte buffers when a sent item requires a wrap around. The diagrams assume a buffer of **128 bytes** with **56 bytes of free space that wraps around** and a sent item of **28 bytes**.

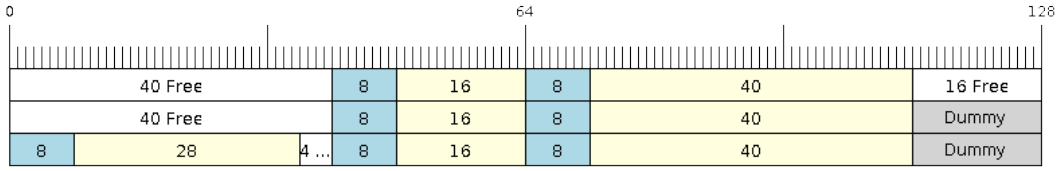


Fig. 44: Wrap around in No-Split buffers

No-Split buffers **only store an item in continuous free space and do not split an item under any circumstances**. When the free space at the tail of the buffer is insufficient to completely store the item and its header, the free space at the tail will be **marked as dummy data**. The buffer will then wrap around and store the item in the free space at the head of the buffer.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to store the 28 byte item. Therefore, the 16 bytes is marked as dummy data and the item is written to the free space at the head of the buffer instead.

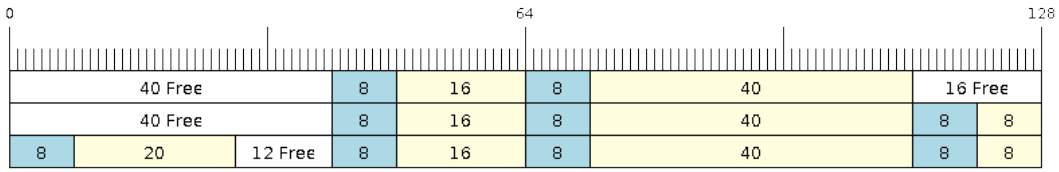


Fig. 45: Wrap around in Allow-Split buffers

Allow-Split buffers will attempt to **split the item into two parts** when the free space at the tail of the buffer is insufficient to store the item data and its header. Both parts of the split item will have their own headers, therefore incurring an extra 8 bytes of overhead.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to store the 28 byte item. Therefore, the item is split into two parts (8 and 20 bytes) and written as two parts to the buffer.

Note: Allow-Split buffers treat both parts of the split item as two separate items, therefore call `xRingbufferReceiveSplit()` instead of `xRingbufferReceive()` to receive both parts of a split item in a thread safe

manner.

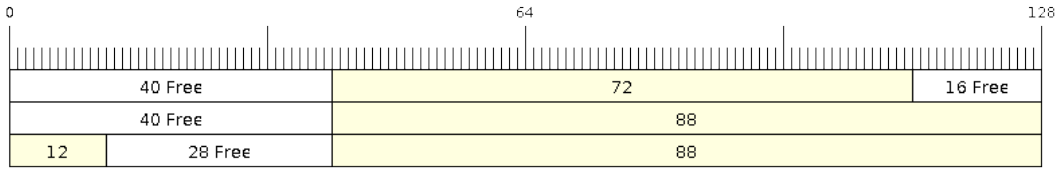


Fig. 46: Wrap around in byte buffers

Byte buffers **store as much data as possible into the free space at the tail of buffer**. The remaining data will then be stored in the free space at the head of the buffer. No overhead is incurred when wrapping around in byte buffers.

Referring to the diagram above, the 16 bytes of free space at the tail of the buffer is insufficient to completely store the 28 bytes of data. Therefore, the 16 bytes of free space is filled with data, and the remaining 12 bytes are written to the free space at the head of the buffer. The buffer now contains data in two separate continuous parts, and each continuous part is treated as a separate item by the byte buffer.

Retrieving/Returning The following diagrams illustrate the differences between No-Split and Allow-Split buffers as compared to byte buffers in retrieving and returning data:

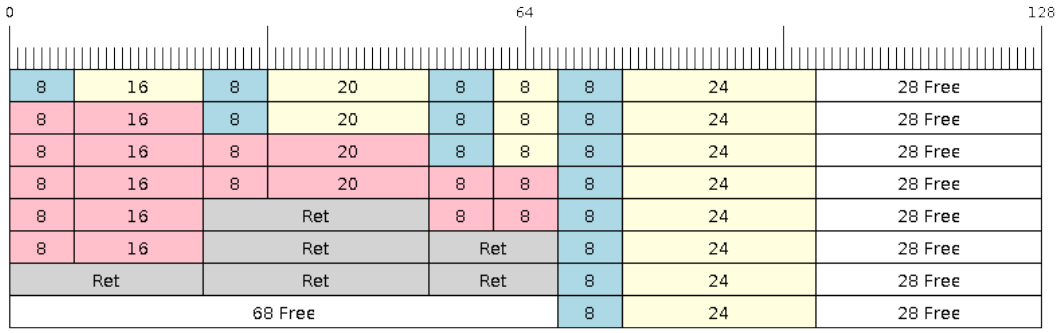


Fig. 47: Retrieving/Returning items in No-Split and Allow-Split ring buffers

Items in No-Split buffers and Allow-Split buffers are **retrieved in strict FIFO order** and **must be returned** for the occupied space to be freed. Multiple items can be retrieved before returning, and the items do not necessarily need to be returned in the order they were retrieved. However, the freeing of space must occur in FIFO order, therefore not returning the earliest retrieved item prevents the space of subsequent items from being freed.

Referring to the diagram above, the **16, 20, and 8 byte items are retrieved in FIFO order**. However, the items are not returned in the order they were retrieved. First, the 20 byte item is returned followed by the 8 byte and the 16 byte items. The space is not freed until the first item, i.e., the 16 byte item is returned.

Byte buffers **do not allow multiple retrievals before returning** (every retrieval must be followed by a return before another retrieval is permitted). When using `xRingbufferReceive()` or `xRingbufferReceiveFromISR()`, all continuous stored data will be retrieved. `xRingbufferReceiveUpTo()` or `xRingbufferReceiveUpToFromISR()` can be used to restrict the maximum number of bytes retrieved. Since every retrieval must be followed by a return, the space is freed as soon as the data is returned.

Referring to the diagram above, the 38 bytes of continuous stored data at the tail of the buffer is retrieved, returned, and freed. The next call to `xRingbufferReceive()` or `xRingbufferReceiveFromISR()` then wraps around and does the same to the 30 bytes of continuous stored data at the head of the buffer.

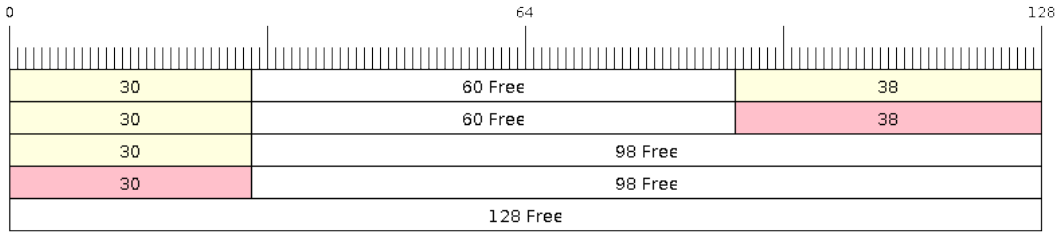


Fig. 48: Retrieving/Returning data in byte buffers

Ring Buffers with Queue Sets Ring buffers can be added to FreeRTOS queue sets using *xRingbufferAddToQueueSetRead()* such that every time a ring buffer receives an item or data, the queue set is notified. Once added to a queue set, every attempt to retrieve an item from a ring buffer should be preceded by a call to *xQueueSelectFromSet()*. To check whether the selected queue set member is the ring buffer, call *xRingbufferCanRead()*.

The following example demonstrates queue set usage with ring buffers:

```
#include "freertos/queue.h"
#include "freertos/ringbuf.h"

...

    //Create ring buffer and queue set
    RingbufHandle_t buf_handle = xRingbufferCreate(1028, RINGBUF_TYPE_NOSPLIT);
    QueueSetHandle_t queue_set = xQueueCreateSet(3);

    //Add ring buffer to queue set
    if (xRingbufferAddToQueueSetRead(buf_handle, queue_set) != pdTRUE) {
        printf("Failed to add to queue set\n");
    }

...

    //Block on queue set
    QueueSetMemberHandle_t member = xQueueSelectFromSet(queue_set, pdMS_TO_
↪TICKS(1000));

    //Check if member is ring buffer
    if (member != NULL && xRingbufferCanRead(buf_handle, member) == pdTRUE) {
        //Member is ring buffer, receive item from ring buffer
        size_t item_size;
        char *item = (char *)xRingbufferReceive(buf_handle, &item_size, 0);

        //Handle item
        ...

    } else {
        ...
    }
```

Ring Buffers with Static Allocation The *xRingbufferCreateStatic()* can be used to create ring buffers with specific memory requirements (such as a ring buffer being allocated in external RAM). All blocks of memory used by a ring buffer must be manually allocated beforehand, then passed to the *xRingbufferCreateStatic()* to be initialized as a ring buffer. These blocks include the following:

- The ring buffer's data structure of type *StaticRingbuffer_t*.

- The ring buffer's storage area of size `xBufferSize`. Note that `xBufferSize` must be 32-bit aligned for No-Split and Allow-Split buffers.

The manner in which these blocks are allocated depends on the users requirements (e.g., all blocks being statically declared, or dynamically allocated with specific capabilities such as external RAM).

Note: When deleting a ring buffer created via *`xRingbufferCreateStatic()`*, the function *`vRingbufferDelete()`* will not free any of the memory blocks. This must be done manually by the user after *`vRingbufferDelete()`* is called.

The code snippet below demonstrates a ring buffer being allocated entirely in external RAM.

```
#include "freertos/ringbuf.h"
#include "freertos/semphr.h"
#include "esp_heap_caps.h"

#define BUFFER_SIZE     400      //32-bit aligned size
#define BUFFER_TYPE     RINGBUF_TYPE_NOSPLIT
...

//Allocate ring buffer data structure and storage area into external RAM
StaticRingbuffer_t *buffer_struct = (StaticRingbuffer_t *)heap_caps_
↪malloc(sizeof(StaticRingbuffer_t), MALLOC_CAP_SPIRAM);
uint8_t *buffer_storage = (uint8_t *)heap_caps_malloc(sizeof(uint8_t)*BUFFER_SIZE,␣
↪MALLOC_CAP_SPIRAM);

//Create a ring buffer with manually allocated memory
RingbufHandle_t handle = xRingbufferCreateStatic(BUFFER_SIZE, BUFFER_TYPE, buffer_
↪storage, buffer_struct);

...

//Delete the ring buffer after used
vRingbufferDelete(handle);

//Manually free all blocks of memory
free(buffer_struct);
free(buffer_storage);
```

ESP-IDF Tick and Idle Hooks

FreeRTOS allows applications to provide a tick hook and an idle hook at compile time:

- FreeRTOS tick hook can be enabled via the *CONFIG_FREERTOS_USE_TICK_HOOK* option. The application must provide the `void vApplicationTickHook( void )` callback.
- FreeRTOS idle hook can be enabled via the *CONFIG_FREERTOS_USE_IDLE_HOOK* option. The application must provide the `void vApplicationIdleHook( void )` callback.

However, the FreeRTOS tick hook and idle hook have the following draw backs:

- The FreeRTOS hooks are registered at compile time
- Only one of each hook can be registered
- On multi-core targets, the FreeRTOS hooks are symmetric, meaning each core's tick interrupt and idle tasks ends up calling the same hook

Therefore, ESP-IDF tick and idle hooks are provided to supplement the features of FreeRTOS tick and idle hooks. The ESP-IDF hooks have the following features:

- The hooks can be registered and deregistered at run-time
- Multiple hooks can be registered (with a maximum of 8 hooks of each type per core)
- On multi-core targets, the hooks can be asymmetric, meaning different hooks can be registered to each core

ESP-IDF hooks can be registered and deregistered using the following APIs:

- For tick hooks:
  - Register using `esp_register_freertos_tick_hook()` or `esp_register_freertos_tick_hook_for_cpu()`
  - Deregister using `esp_deregister_freertos_tick_hook()` or `esp_deregister_freertos_tick_hook_for_cpu()`
- For idle hooks:
  - Register using `esp_register_freertos_idle_hook()` or `esp_register_freertos_idle_hook_for_cpu()`
  - Deregister using `esp_deregister_freertos_idle_hook()` or `esp_deregister_freertos_idle_hook_for_cpu()`

Note: The tick interrupt stays active while the cache is disabled, therefore any tick hook (FreeRTOS or ESP-IDF) functions must be placed in internal RAM. Please refer to the *SPI flash API documentation* for more details.

TLSP Deletion Callbacks

Vanilla FreeRTOS provides a Thread Local Storage Pointers (TLSP) feature. These are pointers stored directly in the Task Control Block (TCB) of a particular task. TLSPs allow each task to have its own unique set of pointers to data structures. Vanilla FreeRTOS expects users to:

- set a task's TLSPs by calling `vTaskSetThreadLocalStoragePointer()` after the task has been created.
- get a task's TLSPs by calling `pvTaskGetThreadLocalStoragePointer()` during the task's lifetime.
- free the memory pointed to by the TLSPs before the task is deleted.

However, there can be instances where users may want the freeing of TLSP memory to be automatic. Therefore, ESP-IDF provides the additional feature of TLSP deletion callbacks. These user-provided deletion callbacks are called automatically when a task is deleted, thus allowing the TLSP memory to be cleaned up without needing to add the cleanup logic explicitly to the code of every task.

The TLSP deletion callbacks are set in a similar fashion to the TLSPs themselves.

- `vTaskSetThreadLocalStoragePointerAndDelCallback()` sets both a particular TLSP and its associated callback.
- Calling the Vanilla FreeRTOS function `vTaskSetThreadLocalStoragePointer()` simply sets the TLSP's associated Deletion Callback to *NULL*, meaning that no callback is called for that TLSP during task deletion.

When implementing TLSP callbacks, users should note the following:

- The callback **must never attempt to block or yield** and critical sections should be kept as short as possible.
- The callback is called shortly before a deleted task's memory is freed. Thus, the callback can either be called from `vTaskDelete()` itself, or from the idle task.

IDF Additional API

The freertos/esp_additions/include/freertos/idf_additions.h header contains FreeRTOS-related helper functions added by ESP-IDF. Users can include this header via `#include "freertos/idf_additions.h"`.

Component Specific Properties

Besides standard component variables that are available with basic cmake build properties, FreeRTOS component also provides arguments (only one so far) for simpler integration with other modules:

- *ORIG_INCLUDE_PATH* - contains an absolute path to freertos root include folder. Thus instead of *#include "freertos/FreeRTOS.h"* you can refer to headers directly: *#include "FreeRTOS.h"*.

API Reference

Ring Buffer API

Header File

- components/esp_ringbuf/include/freertos/ringbuf.h
- This header file can be included with:

```
#include "freertos/ringbuf.h"
```

- This header file is a part of the API provided by the `esp_ringbuf` component. To declare that your component depends on `esp_ringbuf`, add the following to your CMakeLists.txt:

```
REQUIRES esp_ringbuf
```

or

```
PRIV_REQUIRES esp_ringbuf
```

Functions

RingbufHandle_t **xRingbufferCreate**(size_t xBufferSize, *RingbufferType_t* xBufferType)

Create a ring buffer.

Note: xBufferSize of no-split/allow-split buffers will be rounded up to the nearest 32-bit aligned size.

Parameters

- **xBufferSize** -- **[in]** Size of the buffer in bytes. Note that items require space for a header in no-split/allow-split buffers
- **xBufferType** -- **[in]** Type of ring buffer, see documentation.

Returns A handle to the created ring buffer, or NULL in case of error.

RingbufHandle_t **xRingbufferCreateNoSplit**(size_t xItemSize, size_t xItemNum)

Create a ring buffer of type RINGBUF_TYPE_NOSPLIT for a fixed item_size.

This API is similar to xRingbufferCreate(), but it will internally allocate additional space for the headers.

Parameters

- **xItemSize** -- **[in]** Size of each item to be put into the ring buffer
- **xItemNum** -- **[in]** Maximum number of items the buffer needs to hold simultaneously

Returns A RingbufHandle_t handle to the created ring buffer, or NULL in case of error.

RingbufHandle_t **xRingbufferCreateStatic**(size_t xBufferSize, *RingbufferType_t* xBufferType, uint8_t *pucRingbufferStorage, *StaticRingbuffer_t* *pxStaticRingbuffer)

Create a ring buffer but manually provide the required memory.

Note: xBufferSize of no-split/allow-split buffers MUST be 32-bit aligned.

Parameters

- **xBufferSize** -- **[in]** Size of the buffer in bytes.
- **xBufferType** -- **[in]** Type of ring buffer, see documentation
- **pucRingbufferStorage** -- **[in]** Pointer to the ring buffer's storage area. Storage area must have the same size as specified by xBufferSize
- **pxStaticRingbuffer** -- **[in]** Pointed to a struct of type StaticRingbuffer_t which will be used to hold the ring buffer's data structure

Returns A handle to the created ring buffer

BaseType_t **xRingbufferSend**(*RingbufHandle_t* xRingbuffer, const void *pvItem, size_t xItemSize, TickType_t xTicksToWait)

Insert an item into the ring buffer.

Attempt to insert an item into the ring buffer. This function will block until enough free space is available or until it times out.

Note: For no-split/allow-split ring buffers, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note: For no-split/allow-split buffers, an xItemSize of 0 will result in an item with no data being set (i.e., item only contains the header). For byte buffers, an xItemSize of 0 will simply return pdTRUE without copying any data.

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to insert the item into
- **pvItem** -- **[in]** Pointer to data to insert. NULL is allowed if xItemSize is 0.
- **xItemSize** -- **[in]** Size of data to insert.
- **xTicksToWait** -- **[in]** Ticks to wait for room in the ring buffer.

Returns

- pdTRUE if succeeded
- pdFALSE on time-out or when the data is larger than the maximum permissible size of the buffer

BaseType_t **xRingbufferSendFromISR**(*RingbufHandle_t* xRingbuffer, const void *pvItem, size_t xItemSize, BaseType_t *pxHigherPriorityTaskWoken)

Insert an item into the ring buffer in an ISR.

Attempt to insert an item into the ring buffer from an ISR. This function will return immediately if there is insufficient free space in the buffer.

Note: For no-split/allow-split ring buffers, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note: For no-split/allow-split buffers, an xItemSize of 0 will result in an item with no data being set (i.e., item only contains the header). For byte buffers, an xItemSize of 0 will simply return pdTRUE without copying any data.

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to insert the item into
- **pvItem** -- **[in]** Pointer to data to insert. NULL is allowed if xItemSize is 0.
- **xItemSize** -- **[in]** Size of data to insert.
- **pxHigherPriorityTaskWoken** -- **[out]** Value pointed to will be set to pdTRUE if the function woke up a higher priority task.

Returns

- pdTRUE if succeeded
- pdFALSE when the ring buffer does not have space.

BaseType_t **xRingbufferSendAcquire**(*RingbufHandle_t* xRingbuffer, void **ppvItem, size_t xItemSize, TickType_t xTicksToWait)

Acquire memory from the ring buffer to be written to by an external source and to be sent later.

Attempt to allocate buffer for an item to be sent into the ring buffer. This function will block until enough free space is available or until it times out.

The item, as well as the following items `SendAcquire` or `Send` after it, will not be able to be read from the ring buffer until this item is actually sent into the ring buffer.

Note: Only applicable for no-split ring buffers now, the actual size of memory that the item will occupy will be rounded up to the nearest 32-bit aligned size. This is done to ensure all items are always stored in 32-bit aligned fashion.

Note: An xItemSize of 0 will result in a buffer being acquired, but the buffer will have a size of 0.

Parameters

- **`xRingbuffer`** -- **[in]** Ring buffer to allocate the memory
- **`ppvItem`** -- **[out]** Double pointer to memory acquired (set to NULL if no memory were retrieved)
- **`xItemSize`** -- **[in]** Size of item to acquire.
- **`xTicksToWait`** -- **[in]** Ticks to wait for room in the ring buffer.

Returns

- pdTRUE if succeeded
- pdFALSE on time-out or when the data is larger than the maximum permissible size of the buffer

BaseType_t **`xRingbufferSendComplete`**(*RingbufHandle_t* xRingbuffer, void *pvItem)

Actually send an item into the ring buffer allocated before by `xRingbufferSendAcquire`.

Note: Only applicable for no-split ring buffers. Only call for items allocated by `xRingbufferSendAcquire`.

Parameters

- **`xRingbuffer`** -- **[in]** Ring buffer to insert the item into
- **`pvItem`** -- **[in]** Pointer to item in allocated memory to insert.

Returns

- pdTRUE if succeeded
- pdFALSE if fail for some reason.

void ***`xRingbufferReceive`**(*RingbufHandle_t* xRingbuffer, size_t *pxItemSize, TickType_t xTicksToWait)

Retrieve an item from the ring buffer.

Attempt to retrieve an item from the ring buffer. This function will block until an item is available or until it times out.

Note: A call to vRingbufferReturnItem() is required after this to free the item retrieved.

Note: It is possible to receive items with a pxItemSize of 0 on no-split/allow split buffers.

Parameters

- **`xRingbuffer`** -- **[in]** Ring buffer to retrieve the item from
- **`pxItemSize`** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **`xTicksToWait`** -- **[in]** Ticks to wait for items in the ring buffer.

Returns

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL on timeout, *pxItemSize is untouched in that case.

void ***xRingbufferReceiveFromISR**(*RingbufHandle_t* xRingbuffer, size_t *pxItemSize)

Retrieve an item from the ring buffer in an ISR.

Attempt to retrieve an item from the ring buffer. This function returns immediately if there are no items available for retrieval

---

Note: A call to vRingbufferReturnItemFromISR() is required after this to free the item retrieved.

Note: Byte buffers do not allow multiple retrievals before returning an item

---

Note: Two calls to RingbufferReceiveFromISR() are required if the bytes wrap around the end of the ring buffer.

Note: It is possible to receive items with a pxItemSize of 0 on no-split/allow split buffers.

Parameters

- **`xRingbuffer`** -- **[in]** Ring buffer to retrieve the item from
- **`pxItemSize`** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.

Returns

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL when the ring buffer is empty, *pxItemSize is untouched in that case.

BaseType_t **xRingbufferReceiveSplit**(*RingbufHandle_t* xRingbuffer, void **ppvHeadItem, void **ppvTailItem, size_t *pxHeadItemSize, size_t *pxTailItemSize, TickType_t xTicksToWait)

Retrieve a split item from an allow-split ring buffer.

Attempt to retrieve a split item from an allow-split ring buffer. If the item is not split, only a single item is retried. If the item is split, both parts will be retrieved. This function will block until an item is available or until it times out.

Note: Call(s) to vRingbufferReturnItem() is required after this to free up the item(s) retrieved.

Note: This function should only be called on allow-split buffers

---

Note: It is possible to receive items with a pxItemSize of 0 on allow split buffers.

Parameters

- **`xRingbuffer`** -- **[in]** Ring buffer to retrieve the item from
- **`ppvHeadItem`** -- **[out]** Double pointer to first part (set to NULL if no items were retrieved)
- **`ppvTailItem`** -- **[out]** Double pointer to second part (set to NULL if item is not split)

- **pxHeadItemSize** -- **[out]** Pointer to size of first part (unmodified if no items were retrieved)
- **pxTailItemSize** -- **[out]** Pointer to size of second part (unmodified if item is not split)
- **xTicksToWait** -- **[in]** Ticks to wait for items in the ring buffer.

Returns

- pdTRUE if an item (split or unsplit) was retrieved
- pdFALSE when no item was retrieved

BaseType_t **xRingbufferReceiveSplitFromISR**(*RingbufHandle_t* xRingbuffer, void **ppvHeadItem, void **ppvTailItem, size_t *pxHeadItemSize, size_t *pxTailItemSize)

Retrieve a split item from an allow-split ring buffer in an ISR.

Attempt to retrieve a split item from an allow-split ring buffer. If the item is not split, only a single item is retried. If the item is split, both parts will be retrieved. This function returns immediately if there are no items available for retrieval

Note: Calls to vRingbufferReturnItemFromISR() is required after this to free up the item(s) retrieved.

Note: This function should only be called on allow-split buffers

Note: It is possible to receive items with a pxItemSize of 0 on allow split buffers.

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **ppvHeadItem** -- **[out]** Double pointer to first part (set to NULL if no items were retrieved)
- **ppvTailItem** -- **[out]** Double pointer to second part (set to NULL if item is not split)
- **pxHeadItemSize** -- **[out]** Pointer to size of first part (unmodified if no items were retrieved)
- **pxTailItemSize** -- **[out]** Pointer to size of second part (unmodified if item is not split)

Returns

- pdTRUE if an item (split or unsplit) was retrieved
- pdFALSE when no item was retrieved

void ***xRingbufferReceiveUpTo**(*RingbufHandle_t* xRingbuffer, size_t *pxItemSize, TickType_t xTicksToWait, size_t xMaxSize)

Retrieve bytes from a byte buffer, specifying the maximum amount of bytes to retrieve.

Attempt to retrieve data from a byte buffer whilst specifying a maximum number of bytes to retrieve. This function will block until there is data available for retrieval or until it times out.

Note: A call to vRingbufferReturnItem() is required after this to free up the data retrieved.

Note: This function should only be called on byte buffers

Note: Byte buffers do not allow multiple retrievals before returning an item

Note: Two calls to RingbufferReceiveUpTo() are required if the bytes wrap around the end of the ring buffer.

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **xTicksToWait** -- **[in]** Ticks to wait for items in the ring buffer.
- **xMaxSize** -- **[in]** Maximum number of bytes to return.

Returns

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL on timeout, *pxItemSize is untouched in that case.

void ***xRingbufferReceiveUpToFromISR**(*RingbufHandle_t* xRingbuffer, size_t *pxItemSize, size_t xMaxSize)

Retrieve bytes from a byte buffer, specifying the maximum amount of bytes to retrieve. Call this from an ISR.

Attempt to retrieve bytes from a byte buffer whilst specifying a maximum number of bytes to retrieve. This function will return immediately if there is no data available for retrieval.

Note: A call to vRingbufferReturnItemFromISR() is required after this to free up the data received.

Note: This function should only be called on byte buffers

Note: Byte buffers do not allow multiple retrievals before returning an item

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to retrieve the item from
- **pxItemSize** -- **[out]** Pointer to a variable to which the size of the retrieved item will be written.
- **xMaxSize** -- **[in]** Maximum number of bytes to return. Size of 0 simply returns NULL.

Returns

- Pointer to the retrieved item on success; *pxItemSize filled with the length of the item.
- NULL when the ring buffer is empty, *pxItemSize is untouched in that case.

void **vRingbufferReturnItem**(*RingbufHandle_t* xRingbuffer, void *pvItem)

Return a previously-retrieved item to the ring buffer.

Note: If a split item is retrieved, both parts should be returned by calling this function twice

Parameters

- **xRingbuffer** -- **[in]** Ring buffer the item was retrieved from
- **pvItem** -- **[in]** Item that was received earlier

void **vRingbufferReturnItemFromISR**(*RingbufHandle_t* xRingbuffer, void *pvItem, BaseType_t *pxHigherPriorityTaskWoken)

Return a previously-retrieved item to the ring buffer from an ISR.

Note: If a split item is retrieved, both parts should be returned by calling this function twice

Parameters

- **xRingbuffer** -- **[in]** Ring buffer the item was retrieved from
- **pvItem** -- **[in]** Item that was received earlier
- **pxHigherPriorityTaskWoken** -- **[out]** Value pointed to will be set to pdTRUE if the function woke up a higher priority task.

void **vRingbufferDelete**(*RingbufHandle_t* xRingbuffer)

Delete a ring buffer.

Note: This function will not deallocate any memory if the ring buffer was created using xRingbufferCreateStatic(). Deallocation must be done manually be the user.

Parameters **xRingbuffer** -- **[in]** Ring buffer to delete

size_t **xRingbufferGetMaxItemSize**(*RingbufHandle_t* xRingbuffer)

Get maximum size of an item that can be placed in the ring buffer.

This function returns the maximum size an item can have if it was placed in an empty ring buffer.

Note: The max item size for a no-split buffer is limited to ((buffer_size/2)-header_size). This limit is imposed so that an item of max item size can always be sent to an empty no-split buffer regardless of the internal positions of the buffer's read/write/free pointers.

Parameters **xRingbuffer** -- **[in]** Ring buffer to query
Returns Maximum size, in bytes, of an item that can be placed in a ring buffer.

size_t **xRingbufferGetCurFreeSize**(*RingbufHandle_t* xRingbuffer)

Get current free size available for an item/data in the buffer.

This gives the real time free space available for an item/data in the ring buffer. This represents the maximum size an item/data can have if it was currently sent to the ring buffer.

Note: An empty no-split buffer has a max current free size for an item that is limited to ((buffer_size/2)-header_size). See API reference for xRingbufferGetMaxItemSize().

Warning: This API is not thread safe. So, if multiple threads are accessing the same ring buffer, it is the application's responsibility to ensure atomic access to this API and the subsequent Send

Parameters **xRingbuffer** -- **[in]** Ring buffer to query
Returns Current free size, in bytes, available for an entry

BaseType_t **xRingbufferAddToQueueSetRead**(*RingbufHandle_t* xRingbuffer, *QueueSetHandle_t* xQueueSet)

Add the ring buffer to a queue set. Notified when data has been written to the ring buffer.

This function adds the ring buffer to a queue set, thus allowing a task to block on multiple queues/ring buffers. The queue set is notified when the new data becomes available to read on the ring buffer.

Parameters

- **xRingbuffer** -- **[in]** Ring buffer to add to the queue set
- **xQueueSet** -- **[in]** Queue set to add the ring buffer to

Returns

- pdTRUE on success, pdFALSE otherwise

static inline BaseType_t **xRingbufferCanRead**(*RingbufHandle_t* xRingbuffer, *QueueSetMemberHandle_t* xMember)

Check if the selected queue set member is a particular ring buffer.

This API checks if queue set member returned from xQueueSelectFromSet() is a particular ring buffer. If so, this indicates the ring buffer has items waiting to be retrieved.

Parameters
- **xRingbuffer** -- **[in]** Ring buffer to check
- **xMember** -- **[in]** Member returned from xQueueSelectFromSet

Returns
- pdTRUE when selected queue set member is the ring buffer
- pdFALSE otherwise.

BaseType_t **xRingbufferRemoveFromQueueSetRead**(*RingbufHandle_t* xRingbuffer, *QueueSetHandle_t* xQueueSet)

Remove the ring buffer from a queue set.

This function removes a ring buffer from a queue set. The ring buffer must have been previously added to the queue set using xRingbufferAddToQueueSetRead().

Parameters
- **xRingbuffer** -- **[in]** Ring buffer to remove from the queue set
- **xQueueSet** -- **[in]** Queue set to remove the ring buffer from

Returns
- pdTRUE on success
- pdFALSE otherwise

void **vRingbufferGetInfo**(*RingbufHandle_t* xRingbuffer, UBaseType_t *uxFree, UBaseType_t *uxRead, UBaseType_t *uxWrite, UBaseType_t *uxAcquire, UBaseType_t *uxItemsWaiting)

Get information about ring buffer status.

Get information of a ring buffer's current status such as free/read/write/acquire pointer positions, and number of items waiting to be retrieved. Arguments can be set to NULL if they are not required.

Parameters
- **xRingbuffer** -- **[in]** Ring buffer handle
- **uxFree** -- **[out]** Pointer use to store free pointer position
- **uxRead** -- **[out]** Pointer use to store read pointer position
- **uxWrite** -- **[out]** Pointer use to store write pointer position
- **uxAcquire** -- **[out]** Pointer use to store acquire pointer position
- **uxItemsWaiting** -- **[out]** Pointer use to store number of items (bytes for byte buffer) waiting to be retrieved

void **xRingbufferPrintInfo**(*RingbufHandle_t* xRingbuffer)

Debugging function to print the internal pointers in the ring buffer.

Parameters **xRingbuffer** -- Ring buffer to show

BaseType_t **xRingbufferGetStaticBuffer**(*RingbufHandle_t* xRingbuffer, uint8_t **ppucRingbufferStorage, *StaticRingbuffer_t* **ppxStaticRingbuffer)

Retrieve the pointers to a statically created ring buffer.

Parameters
- **xRingbuffer** -- **[in]** Ring buffer
- **ppucRingbufferStorage** -- **[out]** Used to return a pointer to the queue's storage area buffer
- **ppxStaticRingbuffer** -- **[out]** Used to return a pointer to the queue's data structure buffer

Returns pdTRUE if buffers were retrieved, pdFALSE otherwise.

RingbufHandle_t **xRingbufferCreateWithCaps**(size_t xBufferSize, *RingbufferType_t* xBufferType, UBaseType_t uxMemoryCaps)

Creates a ring buffer with specific memory capabilities.

This function is similar to xRingbufferCreate(), except that it allows the memory allocated for the ring buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A queue created using this function must only be deleted using vRingbufferDeleteWithCaps()

Parameters

- **xBufferSize** -- **[in]** Size of the buffer in bytes
- **xBufferType** -- **[in]** Type of ring buffer, see documentation.
- **uxMemoryCaps** -- **[in]** Memory capabilities of the queue's memory (see esp_heap_caps.h)

Returns Handle to the created ring buffer or NULL on failure.

void **vRingbufferDeleteWithCaps**(*RingbufHandle_t* xRingbuffer)

Deletes a ring buffer previously created using xRingbufferCreateWithCaps()

Parameters **xRingbuffer** -- Ring buffer

Structures

struct **xSTATIC_RINGBUFFER**

Struct that is equivalent in size to the ring buffer's data structure.

The contents of this struct are not meant to be used directly. This structure is meant to be used when creating a statically allocated ring buffer where this struct is of the exact size required to store a ring buffer's control data structure.

Type Definitions

typedef void ***RingbufHandle_t**

Type by which ring buffers are referenced. For example, a call to xRingbufferCreate() returns a RingbufHandle_t variable that can then be used as a parameter to xRingbufferSend(), xRingbufferReceive(), etc.

typedef struct *xSTATIC_RINGBUFFER* **StaticRingbuffer_t**

Struct that is equivalent in size to the ring buffer's data structure.

The contents of this struct are not meant to be used directly. This structure is meant to be used when creating a statically allocated ring buffer where this struct is of the exact size required to store a ring buffer's control data structure.

Enumerations

enum **RingbufferType_t**

Values:

enumerator **RINGBUF_TYPE_NOSPLIT**

No-split buffers will only store an item in contiguous memory and will never split an item. Each item requires an 8 byte overhead for a header and will always internally occupy a 32-bit aligned size of space.

enumerator **RINGBUF_TYPE_ALLOWSPLIT**

Allow-split buffers will split an item into two parts if necessary in order to store it. Each item requires an 8 byte overhead for a header, splitting incurs an extra header. Each item will always internally occupy a 32-bit aligned size of space.

enumerator **RINGBUF_TYPE_BYTEBUF**

Byte buffers store data as a sequence of bytes and do not maintain separate items, therefore byte buffers have no overhead. All data is stored as a sequence of byte and any number of bytes can be sent or retrieved each time.

enumerator **RINGBUF_TYPE_MAX**

Hooks API

Header File

- components/esp_system/include/esp_freertos_hooks.h
- This header file can be included with:

```
#include "esp_freertos_hooks.h"
```

Functions

esp_err_t **esp_register_freertos_idle_hook_for_cpu**(*esp_freertos_idle_cb_t* new_idle_cb, UBaseType_t cpuid)

Register a callback to be called from the specified core's idle hook. The callback should return true if it should be called by the idle hook once per interrupt (or FreeRTOS tick), and return false if it should be called repeatedly as fast as possible by the idle hook.

Warning: Idle callbacks MUST NOT, UNDER ANY CIRCUMSTANCES, CALL A FUNCTION THAT MIGHT BLOCK.

Parameters
- **new_idle_cb** -- **[in]** Callback to be called
- **cpuid** -- **[in]** id of the core

Returns
- ESP_OK: Callback registered to the specified core's idle hook
- ESP_ERR_NO_MEM: No more space on the specified core's idle hook to register callback
- ESP_ERR_INVALID_ARG: cpuid is invalid

esp_err_t **esp_register_freertos_idle_hook**(*esp_freertos_idle_cb_t* new_idle_cb)

Register a callback to the idle hook of the core that calls this function. The callback should return true if it should be called by the idle hook once per interrupt (or FreeRTOS tick), and return false if it should be called repeatedly as fast as possible by the idle hook.

Warning: Idle callbacks MUST NOT, UNDER ANY CIRCUMSTANCES, CALL A FUNCTION THAT MIGHT BLOCK.

Parameters **new_idle_cb** -- **[in]** Callback to be called

Returns
- ESP_OK: Callback registered to the calling core's idle hook
- ESP_ERR_NO_MEM: No more space on the calling core's idle hook to register callback

esp_err_t **esp_register_freertos_tick_hook_for_cpu**(*esp_freertos_tick_cb_t* new_tick_cb, UBaseType_t cpuid)

Register a callback to be called from the specified core's tick hook.

Parameters
- **new_tick_cb** -- **[in]** Callback to be called
- **cpuid** -- **[in]** id of the core

Returns
- ESP_OK: Callback registered to specified core's tick hook
- ESP_ERR_NO_MEM: No more space on the specified core's tick hook to register the callback
- ESP_ERR_INVALID_ARG: cpuid is invalid

esp_err_t **esp_register_freertos_tick_hook**(*esp_freertos_tick_cb_t* new_tick_cb)

Register a callback to be called from the calling core's tick hook.

Parameters **new_tick_cb** -- **[in]** Callback to be called

Returns
- ESP_OK: Callback registered to the calling core's tick hook
- ESP_ERR_NO_MEM: No more space on the calling core's tick hook to register the callback

void **esp_deregister_freertos_idle_hook_for_cpu**(*esp_freertos_idle_cb_t* old_idle_cb, UBaseType_t cpuid)

Unregister an idle callback from the idle hook of the specified core.

Parameters
- **old_idle_cb** -- **[in]** Callback to be unregistered
- **cpuid** -- **[in]** id of the core

void **esp_deregister_freertos_idle_hook**(*esp_freertos_idle_cb_t* old_idle_cb)

Unregister an idle callback. If the idle callback is registered to the idle hooks of both cores, the idle hook will be unregistered from both cores.

Parameters **old_idle_cb** -- **[in]** Callback to be unregistered

void **esp_deregister_freertos_tick_hook_for_cpu**(*esp_freertos_tick_cb_t* old_tick_cb, UBaseType_t cpuid)

Unregister a tick callback from the tick hook of the specified core.

Parameters
- **old_tick_cb** -- **[in]** Callback to be unregistered
- **cpuid** -- **[in]** id of the core

void **esp_deregister_freertos_tick_hook**(*esp_freertos_tick_cb_t* old_tick_cb)

Unregister a tick callback. If the tick callback is registered to the tick hooks of both cores, the tick hook will be unregistered from both cores.

Parameters **old_tick_cb** -- **[in]** Callback to be unregistered

Type Definitions

typedef bool (***esp_freertos_idle_cb_t**)(void)

typedef void (***esp_freertos_tick_cb_t**)(void)

Additional API

Header File

- components/freertos/esp_additions/include/freertos/idf_additions.h
- This header file can be included with:

```
#include "freertos/idf_additions.h"
```

Functions

BaseType_t **xTaskCreatePinnedToCore**(TaskFunction_t pxTaskCode, const char *const pcName, const uint32_t ulStackDepth, void *const pvParameters, UBaseType_t uxPriority, *TaskHandle_t* *const pxCreatedTask, const BaseType_t xCoreID)

Create a new task that is pinned to a particular core.

This function is similar to xTaskCreate(), but allows the creation of a pinned task. The task's pinned core is specified by the xCoreID argument. If xCoreID is set to tskNO_AFFINITY, then the task is unpinned and can run on any core.

Note: If (configNUMBER_OF_CORES == 1), setting xCoreID to tskNO_AFFINITY will be be treated as 0.

Parameters

- **pxTaskCode** -- Pointer to the task entry function.
- **pcName** -- A descriptive name for the task.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pxCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **xCoreID** -- The core to which the task is pinned to, or tskNO_AFFINITY if the task has no core affinity.

Returns pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

TaskHandle_t **xTaskCreateStaticPinnedToCore**(TaskFunction_t pxTaskCode, const char *const pcName, const uint32_t ulStackDepth, void *const pvParameters, UBaseType_t uxPriority, StackType_t *const puxStackBuffer, StaticTask_t *const pxTaskBuffer, const BaseType_t xCoreID)

Create a new static task that is pinned to a particular core.

This function is similar to xTaskCreateStatic(), but allows the creation of a pinned task. The task's pinned core is specified by the xCoreID argument. If xCoreID is set to tskNO_AFFINITY, then the task is unpinned and can run on any core.

Note: If (configNUMBER_OF_CORES == 1), setting xCoreID to tskNO_AFFINITY will be be treated as 0.

Parameters

- **pxTaskCode** -- Pointer to the task entry function.
- **pcName** -- A descriptive name for the task.
- **ulStackDepth** -- The size of the task stack specified as the NUMBER OF BYTES. Note that this differs from vanilla FreeRTOS.
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.

- **puxStackBuffer** -- Must point to a StackType_t array that has at least ulStackDepth indexes
- **pxTaskBuffer** -- Must point to a variable of type StaticTask_t, which will then be used to hold the task's data structures,
- **xCoreID** -- The core to which the task is pinned to, or tskNO_AFFINITY if the task has no core affinity.

Returns The task handle if the task was created, NULL otherwise.

BaseType_t **xTaskGetCoreID**(*TaskHandle_t* xTask)

Get the current core ID of a particular task.

Helper function to get the core ID of a particular task. If the task is pinned to a particular core, the core ID is returned. If the task is not pinned to a particular core, tskNO_AFFINITY is returned.

If CONFIG_FREERTOS_UNICORE is enabled, this function simply returns 0.

[refactor-todo] See if this needs to be deprecated (IDF-8145)(IDF-8164)

Note: If CONFIG_FREERTOS_SMP is enabled, please call vTaskCoreAffinityGet() instead.

Note: In IDF FreerTOS when configNUMBER_OF_CORES == 1, this function will always return 0,

---

Parameters **xTask** -- The task to query
Returns The task's core ID or tskNO_AFFINITY

TaskHandle_t **xTaskGetCurrentTaskHandleForCore**(BaseType_t xCoreID)

Get the handle of the task currently running on a certain core.

Because of the nature of SMP processing, there is no guarantee that this value will still be valid on return and should only be used for debugging purposes.

[refactor-todo] See if this needs to be deprecated (IDF-8145)

Parameters **xCoreID** -- The core to query
Returns Handle of the current task running on the queried core

uint8_t ***pxTaskGetStackStart**(*TaskHandle_t* xTask)

Returns the start of the stack associated with xTask.

Returns the lowest stack memory address, regardless of whether the stack grows up or down.

[refactor-todo] Change return type to StackType_t (IDF-8158)

Parameters **xTask** -- Handle of the task associated with the stack returned. Set xTask to NULL to return the stack of the calling task.
Returns A pointer to the start of the stack.

void **vTaskSetThreadLocalStoragePointerAndDelCallback**(*TaskHandle_t* xTaskToSet, BaseType_t xIndex, void *pvValue, *TlsDeleteCallbackFunction_t* pvDelCallback)

Set local storage pointer and deletion callback.

Each task contains an array of pointers that is dimensioned by the configNUM_THREAD_LOCAL_STORAGE_POINTERS setting in FreeRTOSConfig.h. The kernel does not use the pointers itself, so the application writer can use the pointers for any purpose they wish.

Local storage pointers set for a task can reference dynamically allocated resources. This function is similar to vTaskSetThreadLocalStoragePointer, but provides a way to release these resources when the task gets deleted. For each pointer, a callback function can be set. This function will be called when task is deleted, with the local storage pointer index and value as arguments.

Parameters

- **xTaskToSet** -- Task to set thread local storage pointer for
- **xIndex** -- The index of the pointer to set, from 0 to configNUM_THREAD_LOCAL_STORAGE_POINTERS - 1.
- **pvValue** -- Pointer value to set.
- **pvDelCallback** -- Function to call to dispose of the local storage pointer when the task is deleted.

BaseType_t **xTaskCreatePinnedToCoreWithCaps**(TaskFunction_t pvTaskCode, const char *const pcName, const configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t uxPriority, *TaskHandle_t* *const pvCreatedTask, const BaseType_t xCoreID, UBaseType_t uxMemoryCaps)

Creates a pinned task where its stack has specific memory capabilities.

This function is similar to xTaskCreatePinnedToCore(), except that it allows the memory allocated for the task's stack to have specific capabilities (e.g., MALLOC_CAP_SPIRAM).

However, the specified capabilities will NOT apply to the task's TCB as a TCB must always be in internal RAM.

Parameters

- **pvTaskCode** -- Pointer to the task entry function
- **pcName** -- A descriptive name for the task
- **usStackDepth** -- The size of the task stack specified as the number of bytes
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pvCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **xCoreID** -- Core to which the task is pinned to, or tskNO_AFFINITY if unpinned.
- **uxMemoryCaps** -- Memory capabilities of the task stack's memory (see esp_heap_caps.h)

Returns pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

static inline BaseType_t **xTaskCreateWithCaps**(TaskFunction_t pvTaskCode, const char *const pcName, configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t uxPriority, *TaskHandle_t* *pvCreatedTask, UBaseType_t uxMemoryCaps)

Creates a task where its stack has specific memory capabilities.

This function is similar to xTaskCreate(), except that it allows the memory allocated for the task's stack to have specific capabilities (e.g., MALLOC_CAP_SPIRAM).

However, the specified capabilities will NOT apply to the task's TCB as a TCB must always be in internal RAM.

Note: A task created using this function must only be deleted using vTaskDeleteWithCaps()

Parameters

- **pvTaskCode** -- Pointer to the task entry function
- **pcName** -- A descriptive name for the task
- **usStackDepth** -- The size of the task stack specified as the number of bytes
- **pvParameters** -- Pointer that will be used as the parameter for the task being created.
- **uxPriority** -- The priority at which the task should run.
- **pvCreatedTask** -- Used to pass back a handle by which the created task can be referenced.
- **uxMemoryCaps** -- Memory capabilities of the task stack's memory (see esp_heap_caps.h)

Returns pdPASS if the task was successfully created and added to a ready list, otherwise an error code defined in the file projdefs.h

void **vTaskDeleteWithCaps**(*TaskHandle_t* xTaskToDelete)

Deletes a task previously created using xTaskCreateWithCaps() or xTaskCreatePinnedToCoreWithCaps()

Note: It is recommended to use this API to delete tasks from another task's context, rather than self-deletion. When the task is being deleted, it is vital to ensure that it is not running on another core. This API must not be called from an interrupt context.

Parameters **xTaskToDelete** -- A handle to the task to be deleted

QueueHandle_t **xQueueCreateWithCaps**(UBaseType_t uxQueueLength, UBaseType_t uxItemSize, UBaseType_t uxMemoryCaps)

Creates a queue with specific memory capabilities.

This function is similar to xQueueCreate(), except that it allows the memory allocated for the queue to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A queue created using this function must only be deleted using vQueueDeleteWithCaps()

Parameters

- **uxQueueLength** -- The maximum number of items that the queue can contain.
- **uxItemSize** -- The number of bytes each item in the queue will require.
- **uxMemoryCaps** -- Memory capabilities of the queue's memory (see esp_heap_caps.h)

Returns Handle to the created queue or NULL on failure.

void **vQueueDeleteWithCaps**(*QueueHandle_t* xQueue)

Deletes a queue previously created using xQueueCreateWithCaps()

Parameters **xQueue** -- A handle to the queue to be deleted.

static inline *SemaphoreHandle_t* **xSemaphoreCreateBinaryWithCaps**(UBaseType_t uxMemoryCaps)

Creates a binary semaphore with specific memory capabilities.

This function is similar to vSemaphoreCreateBinary(), except that it allows the memory allocated for the binary semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A binary semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters **uxMemoryCaps** -- Memory capabilities of the binary semaphore's memory (see esp_heap_caps.h)

Returns Handle to the created binary semaphore or NULL on failure.

static inline *SemaphoreHandle_t* **xSemaphoreCreateCountingWithCaps**(UBaseType_t uxMaxCount, UBaseType_t uxInitialCount, UBaseType_t uxMemoryCaps)

Creates a counting semaphore with specific memory capabilities.

This function is similar to xSemaphoreCreateCounting(), except that it allows the memory allocated for the counting semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A counting semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters
- **uxMaxCount** -- The maximum count value that can be reached.
- **uxInitialCount** -- The count value assigned to the semaphore when it is created.
- **uxMemoryCaps** -- Memory capabilities of the counting semaphore's memory (see esp_heap_caps.h)

Returns Handle to the created counting semaphore or NULL on failure.

static inline *SemaphoreHandle_t* **xSemaphoreCreateMutexWithCaps**(UBaseType_t uxMemoryCaps)

Creates a mutex semaphore with specific memory capabilities.

This function is similar to xSemaphoreCreateMutex(), except that it allows the memory allocated for the mutex semaphore to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A mutex semaphore created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters **uxMemoryCaps** -- Memory capabilities of the mutex semaphore's memory (see esp_heap_caps.h)

Returns Handle to the created mutex semaphore or NULL on failure.

static inline *SemaphoreHandle_t* **xSemaphoreCreateRecursiveMutexWithCaps**(UBaseType_t uxMemoryCaps)

Creates a recursive mutex with specific memory capabilities.

This function is similar to xSemaphoreCreateRecursiveMutex(), except that it allows the memory allocated for the recursive mutex to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A recursive mutex created using this function must only be deleted using vSemaphoreDeleteWithCaps()

Parameters **uxMemoryCaps** -- Memory capabilities of the recursive mutex's memory (see esp_heap_caps.h)

Returns Handle to the created recursive mutex or NULL on failure.

void **vSemaphoreDeleteWithCaps**(*SemaphoreHandle_t* xSemaphore)

Deletes a semaphore previously created using one of the xSemaphoreCreate...WithCaps() functions.

Parameters **xSemaphore** -- A handle to the semaphore to be deleted.

static inline *StreamBufferHandle_t* **xStreamBufferCreateWithCaps**(size_t xBufferSizeBytes, size_t xTriggerLevelBytes, UBaseType_t uxMemoryCaps)

Creates a stream buffer with specific memory capabilities.

This function is similar to xStreamBufferCreate(), except that it allows the memory allocated for the stream buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A stream buffer created using this function must only be deleted using vStreamBufferDeleteWithCaps()

Parameters

- **xBufferSizeBytes** -- The total number of bytes the stream buffer will be able to hold at any one time.
- **xTriggerLevelBytes** -- The number of bytes that must be in the stream buffer before unblocking
- **uxMemoryCaps** -- Memory capabilities of the stream buffer's memory (see esp_heap_caps.h)

Returns Handle to the created stream buffer or NULL on failure.

static inline void **vStreamBufferDeleteWithCaps**(*StreamBufferHandle_t* xStreamBuffer)

Deletes a stream buffer previously created using xStreamBufferCreateWithCaps()

Parameters **xStreamBuffer** -- A handle to the stream buffer to be deleted.

static inline *MessageBufferHandle_t* **xMessageBufferCreateWithCaps**(size_t xBufferSizeBytes, UBaseType_t uxMemoryCaps)

Creates a message buffer with specific memory capabilities.

This function is similar to xMessageBufferCreate(), except that it allows the memory allocated for the message buffer to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: A message buffer created using this function must only be deleted using vMessageBufferDeleteWithCaps()

Parameters

- **xBufferSizeBytes** -- The total number of bytes (not messages) the message buffer will be able to hold at any one time.
- **uxMemoryCaps** -- Memory capabilities of the message buffer's memory (see esp_heap_caps.h)

Returns Handle to the created message buffer or NULL on failure.

static inline void **vMessageBufferDeleteWithCaps**(*MessageBufferHandle_t* xMessageBuffer)

Deletes a stream buffer previously created using xMessageBufferCreateWithCaps()

Parameters **xMessageBuffer** -- A handle to the message buffer to be deleted.

EventGroupHandle_t **xEventGroupCreateWithCaps**(UBaseType_t uxMemoryCaps)

Creates an event group with specific memory capabilities.

This function is similar to xEventGroupCreate(), except that it allows the memory allocated for the event group to have specific capabilities (e.g., MALLOC_CAP_INTERNAL).

Note: An event group created using this function must only be deleted using vEventGroupDeleteWithCaps()

Parameters **uxMemoryCaps** -- Memory capabilities of the event group's memory (see esp_heap_caps.h)

Returns Handle to the created event group or NULL on failure.

void **vEventGroupDeleteWithCaps**(*EventGroupHandle_t* xEventGroup)

Deletes an event group previously created using xEventGroupCreateWithCaps()

Parameters **xEventGroup** -- A handle to the event group to be deleted.

Type Definitions

typedef void (***TlsDeleteCallbackFunction_t**)(int, void*)

Prototype of local storage pointer deletion callback.

2.10.14 Heap Memory Allocation

Stack and Heap

ESP-IDF applications use the common computer architecture patterns of **stack** (dynamic memory allocated by program control flow), **heap** (dynamic memory allocated by function calls), and **static memory** (memory allocated at compile time).

Because ESP-IDF is a multi-threaded RTOS environment, each RTOS task has its own stack. By default, each of these stacks is allocated from the heap when the task is created. See `xTaskCreateStatic()` for the alternative where stacks are statically allocated.

Because ESP32-S3 uses multiple types of RAM, it also contains multiple heaps with different capabilities. A capabilities-based memory allocator allows apps to make heap allocations for different purposes.

For most purposes, the C Standard Library's `malloc()` and `free()` functions can be used for heap allocation without any special consideration. However, in order to fully make use of all of the memory types and their characteristics, ESP-IDF also has a capabilities-based heap memory allocator. If you want to have a memory with certain properties (e.g., *DMA-Capable Memory* or executable-memory), you can create an OR-mask of the required capabilities and pass that to `heap_caps_malloc()`.

Memory Capabilities

The ESP32-S3 contains multiple types of RAM:

- DRAM (Data RAM) is memory that is connected to CPU's data bus and is used to hold data. This is the most common kind of memory accessed as a heap.
- IRAM (Instruction RAM) is memory that is connected to the CPU's instruction bus and usually holds executable data only (i.e., instructions). If accessed as generic memory, all accesses must be aligned to *32-Bit Accessible Memory*.
- D/IRAM is RAM that is connected to CPU's data bus and instruction bus, thus can be used either Instruction or Data RAM.

For more details on these internal memory types, see *Memory Types*.

It is also possible to connect external SPI RAM to the ESP32-S3. The *external RAM* is integrated into the ESP32-S3's memory map via the cache, and accessed similarly to DRAM.

All DRAM memory is single-byte accessible, thus all DRAM heaps possess the `MALLOC_CAP_8BIT` capability. Users can call `heap_caps_get_free_size(MALLOC_CAP_8BIT)` to get the free size of all DRAM heaps.

When calling `malloc()`, the ESP-IDF `malloc()` internally calls `heap_caps_malloc_default(size)`. This will allocate memory with the capability `MALLOC_CAP_DEFAULT`, which is byte-addressable.

Because `malloc()` uses the capabilities-based allocation system, memory allocated using `heap_caps_malloc()` can be freed by calling the standard `free()` function.

Available Heap

DRAM At startup, the DRAM heap contains all data memory that is not statically allocated by the app. Reducing statically-allocated buffers increases the amount of available free heap.

To find the amount of statically allocated memory, use the *idf.py size* command.

Note: At runtime, the available heap DRAM may be less than calculated at compile time, because, at startup, some memory is allocated from the heap before the FreeRTOS scheduler is started (including memory for the stacks of initial FreeRTOS tasks).

IRAM At startup, the IRAM heap contains all instruction memory that is not used by the app executable code.

The *idf.py size* command can be used to find the amount of IRAM used by the app.

D/IRAM Some memory in the ESP32-S3 is available as either DRAM or IRAM. If memory is allocated from a D/IRAM region, the free heap size for both types of memory will decrease.

Heap Sizes At startup, all ESP-IDF apps log a summary of all heap addresses (and sizes) at level Info:

```
I (252) heap_init: Initializing. RAM available for dynamic allocation:
I (259) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (265) heap_init: At 3FFB2EC8 len 0002D138 (180 KiB): DRAM
I (272) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM
I (278) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM
I (284) heap_init: At 4008944C len 00016BB4 (90 KiB): IRAM
```

Finding Available Heap See *Heap Information*.

Special Capabilities

DMA-Capable Memory Use the `MALLOC_CAP_DMA` flag to allocate memory which is suitable for use with hardware DMA engines (for example SPI and I2S). This capability flag excludes any external PSRAM.

32-Bit Accessible Memory If a certain memory structure is only addressed in 32-bit units, for example, an array of ints or pointers, it can be useful to allocate it with the `MALLOC_CAP_32BIT` flag. This also allows the allocator to give out IRAM memory, which is sometimes unavailable for a normal `malloc()` call. This can help to use all the available memory in the ESP32-S3.

Please note that on ESP32-S3 series chips, `MALLOC_CAP_32BIT` cannot be used for storing floating-point variables. This is because `MALLOC_CAP_32BIT` may return instruction RAM and the floating-point assembly instructions on ESP32-S3 cannot access instruction RAM.

Memory allocated with `MALLOC_CAP_32BIT` can **only** be accessed via 32-bit reads and writes, any other type of access will generate a fatal LoadStoreError exception.

External SPI Memory When *external RAM* is enabled, external SPI RAM can be allocated using standard `malloc` calls, or via `heap_caps_malloc(MALLOC_CAP_SPIRAM)`, depending on the configuration. See *Configuring External RAM* for more details.

Thread Safety

Heap functions are thread-safe, meaning they can be called from different tasks simultaneously without any limitations.

It is technically possible to call `malloc`, `free`, and related functions from interrupt handler (ISR) context (see *Calling Heap-Related Functions from ISR*). However, this is not recommended, as heap function calls may delay other interrupts. It is strongly recommended to refactor applications so that any buffers used by an ISR are pre-allocated outside of the ISR. Support for calling heap functions from ISRs may be removed in a future update.

Calling Heap-Related Functions from ISR

The following functions from the heap component can be called from the interrupt handler (ISR):

- *heap_caps_malloc()*
- `heap_caps_malloc_default()`

- heap_caps_realloc_default()
- *heap_caps_malloc_prefer()*
- *heap_caps_realloc_prefer()*
- *heap_caps_calloc_prefer()*
- *heap_caps_free()*
- *heap_caps_realloc()*
- *heap_caps_calloc()*
- *heap_caps_aligned_alloc()*
- *heap_caps_aligned_free()*

Note: However, this practice is strongly discouraged.

Heap Tracing & Debugging

The following features are documented on the *Heap Memory Debugging* page:

- *Heap Information* (free space, etc.)
- *Heap Allocation and Free Function Hooks*
- *Heap Corruption Detection*
- *Heap Tracing* (memory leak detection, monitoring, etc.)

Implementation Notes

Knowledge about the regions of memory in the chip comes from the "SoC" component, which contains memory layout information for the chip, and the different capabilities of each region. Each region's capabilities are prioritized, so that (for example) dedicated DRAM and IRAM regions are used for allocations ahead of the more versatile D/IRAM regions.

Each contiguous region of memory contains its own memory heap. The heaps are created using the *multi_heap* functionality. multi_heap allows any contiguous region of memory to be used as a heap.

The heap capabilities allocator uses knowledge of the memory regions to initialize each individual heap. Allocation functions in the heap capabilities API will find the most appropriate heap for the allocation based on desired capabilities, available space, and preferences for each region's use, and then calling *multi_heap_malloc()* for the heap situated in that particular region.

Calling free() involves finding the particular heap corresponding to the freed address, and then call *multi_heap_free()* on that particular multi_heap instance.

API Reference - Heap Allocation

Header File

- components/heap/include/esp_heap_caps.h
- This header file can be included with:

```
#include "esp_heap_caps.h"
```

Functions

esp_err_t **heap_caps_register_failed_alloc_callback**(*esp_alloc_failed_hook_t* callback)

registers a callback function to be invoked if a memory allocation operation fails

Parameters **callback** -- caller defined callback to be invoked
Returns ESP_OK if callback was registered.

void ***heap_caps_malloc**(size_t size, uint32_t caps)

Allocate a chunk of memory which has the given capabilities.

Equivalent semantics to libc malloc(), for capability-aware memory.

Parameters
- **size** -- Size, in bytes, of the amount of memory to allocate
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory to be returned

Returns A pointer to the memory allocated on success, NULL on failure

void **heap_caps_free**(void *ptr)

Free memory previously allocated via heap_caps_malloc() or heap_caps_realloc().

Equivalent semantics to libc free(), for capability-aware memory.

In IDF, `free(p)` is equivalent to `heap_caps_free(p)`.

Parameters **ptr** -- Pointer to memory previously returned from heap_caps_malloc() or heap_caps_realloc(). Can be NULL.

void ***heap_caps_realloc**(void *ptr, size_t size, uint32_t caps)

Reallocate memory previously allocated via heap_caps_malloc() or heap_caps_realloc().

Equivalent semantics to libc realloc(), for capability-aware memory.

In IDF, `realloc(p, s)` is equivalent to `heap_caps_realloc(p, s, MALLOC_CAP_8BIT)`.

'caps' parameter can be different to the capabilities that any original 'ptr' was allocated with. In this way, realloc can be used to "move" a buffer if necessary to ensure it meets a new set of capabilities.

Parameters
- **ptr** -- Pointer to previously allocated memory, or NULL for a new allocation.
- **size** -- Size of the new buffer requested, or 0 to free the buffer.
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory desired for the new allocation.

Returns Pointer to a new buffer of size 'size' with capabilities 'caps', or NULL if allocation failed.

void ***heap_caps_aligned_alloc**(size_t alignment, size_t size, uint32_t caps)

Allocate an aligned chunk of memory which has the given capabilities.

Equivalent semantics to libc aligned_alloc(), for capability-aware memory.

Parameters
- **alignment** -- How the pointer received needs to be aligned must be a power of two
- **size** -- Size, in bytes, of the amount of memory to allocate
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory to be returned

Returns A pointer to the memory allocated on success, NULL on failure

void **heap_caps_aligned_free**(void *ptr)

Used to deallocate memory previously allocated with heap_caps_aligned_alloc.

Note: This function is deprecated, please consider using heap_caps_free() instead

---

Parameters **ptr** -- Pointer to the memory allocated

void ***heap_caps_aligned_calloc**(size_t alignment, size_t n, size_t size, uint32_t caps)

Allocate an aligned chunk of memory which has the given capabilities. The initialized value in the memory is set to zero.

Parameters
- **alignment** -- How the pointer received needs to be aligned must be a power of two

- **n** -- Number of continuing chunks of memory to allocate
- **size** -- Size, in bytes, of a chunk of memory to allocate
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory to be returned

Returns A pointer to the memory allocated on success, NULL on failure

void ***heap_caps_calloc**(size_t n, size_t size, uint32_t caps)

Allocate a chunk of memory which has the given capabilities. The initialized value in the memory is set to zero.

Equivalent semantics to libc calloc(), for capability-aware memory.

In IDF, `calloc(p)` is equivalent to `heap_caps_calloc(p, MALLOC_CAP_8BIT)`.

Parameters

- **n** -- Number of continuing chunks of memory to allocate
- **size** -- Size, in bytes, of a chunk of memory to allocate
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory to be returned

Returns A pointer to the memory allocated on success, NULL on failure

size_t **heap_caps_get_total_size**(uint32_t caps)

Get the total size of all the regions that have the given capabilities.

This function takes all regions capable of having the given capabilities allocated in them and adds up the total space they have.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory
Returns total size in bytes

size_t **heap_caps_get_free_size**(uint32_t caps)

Get the total free size of all the regions that have the given capabilities.

This function takes all regions capable of having the given capabilities allocated in them and adds up the free space they have.

Note: Note that because of heap fragmentation it is probably not possible to allocate a single block of memory of this size. Use heap_caps_get_largest_free_block() for this purpose.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory
Returns Amount of free bytes in the regions

size_t **heap_caps_get_minimum_free_size**(uint32_t caps)

Get the total minimum free memory of all regions with the given capabilities.

This adds all the low watermarks of the regions capable of delivering the memory with the given capabilities.

Note: Note the result may be less than the global all-time minimum available heap of this kind, as "low watermarks" are tracked per-region. Individual regions' heaps may have reached their "low watermarks" at different points in time. However, this result still gives a "worst case" indication for all-time minimum free heap.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory
Returns Amount of free bytes in the regions

size_t **heap_caps_get_largest_free_block**(uint32_t caps)

Get the largest free block of memory able to be allocated with the given capabilities.

Returns the largest value of `s` for which `heap_caps_malloc(s, caps)` will succeed.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory
Returns Size of the largest free block in bytes.

esp_err_t **heap_caps_monitor_local_minimum_free_size_start**(void)

Start monitoring the value of minimum_free_bytes from the moment this function is called instead of from startup.

Note: This allows to detect local lows of the minimum_free_bytes value that wouldn't be detected otherwise.

Returns esp_err_t ESP_OK if the function executed properly ESP_FAIL if called when monitoring already active

esp_err_t **heap_caps_monitor_local_minimum_free_size_stop**(void)

Stop monitoring the value of minimum_free_bytes. After this call the minimum_free_bytes value calculated from startup will be returned in heap_caps_get_info and heap_caps_get_minimum_free_size.

Returns esp_err_t ESP_OK if the function executed properly ESP_FAIL if called when monitoring not active

void **heap_caps_get_info**(*multi_heap_info_t* *info, uint32_t caps)

Get heap info for all regions with the given capabilities.

Calls multi_heap_info() on all heaps which share the given capabilities. The information returned is an aggregate across all matching heaps. The meanings of fields are the same as defined for *multi_heap_info_t*, except that `minimum_free_bytes` has the same caveats described in heap_caps_get_minimum_free_size().

Parameters
- **info** -- Pointer to a structure which will be filled with relevant heap metadata.
- **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory

void **heap_caps_print_heap_info**(uint32_t caps)

Print a summary of all memory with the given capabilities.

Calls multi_heap_info on all heaps which share the given capabilities, and prints a two-line summary for each, then a total summary.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory

bool **heap_caps_check_integrity_all**(bool print_errors)

Check integrity of all heap memory in the system.

Calls multi_heap_check on all heaps. Optionally print errors if heaps are corrupt.

Calling this function is equivalent to calling heap_caps_check_integrity with the caps argument set to MALLOC_CAP_INVALID.

Note: Please increase the value of `CONFIG_ESP_INT_WDT_TIMEOUT_MS` when using this API with PSRAM enabled.

Parameters **print_errors** -- Print specific errors if heap corruption is found.
Returns True if all heaps are valid, False if at least one heap is corrupt.

bool **heap_caps_check_integrity**(uint32_t caps, bool print_errors)

Check integrity of all heaps with the given capabilities.

Calls multi_heap_check on all heaps which share the given capabilities. Optionally print errors if the heaps are corrupt.

See also heap_caps_check_integrity_all to check all heap memory in the system and heap_caps_check_integrity_addr to check memory around a single address.

Note: Please increase the value of `CONFIG_ESP_INT_WDT_TIMEOUT_MS` when using this API with PSRAM capability flag.

Parameters
- **`caps`** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory
- **`print_errors`** -- Print specific errors if heap corruption is found.

Returns True if all heaps are valid, False if at least one heap is corrupt.

bool **`heap_caps_check_integrity_addr`**(intptr_t addr, bool print_errors)

Check integrity of heap memory around a given address.

This function can be used to check the integrity of a single region of heap memory, which contains the given address.

This can be useful if debugging heap integrity for corruption at a known address, as it has a lower overhead than checking all heap regions. Note that if the corrupt address moves around between runs (due to timing or other factors) then this approach won't work, and you should call heap_caps_check_integrity or heap_caps_check_integrity_all instead.

Note: The entire heap region around the address is checked, not only the adjacent heap blocks.

Parameters
- **`addr`** -- Address in memory. Check for corruption in region containing this address.
- **`print_errors`** -- Print specific errors if heap corruption is found.

Returns True if the heap containing the specified address is valid, False if at least one heap is corrupt or the address doesn't belong to a heap region.

void **`heap_caps_malloc_extmem_enable`**(size_t limit)

Enable malloc() in external memory and set limit below which malloc() attempts are placed in internal memory.

When external memory is in use, the allocation strategy is to initially try to satisfy smaller allocation requests with internal memory and larger requests with external memory. This sets the limit between the two, as well as generally enabling allocation in external memory.

Parameters **`limit`** -- Limit, in bytes.

void ***`heap_caps_malloc_prefer`**(size_t size, size_t num, ...)

Allocate a chunk of memory as preference in decreasing order.

Attention The variable parameters are bitwise OR of MALLOC_CAP_* flags indicating the type of memory. This API prefers to allocate memory with the first parameter. If failed, allocate memory with the next parameter. It will try in this order until allocating a chunk of memory successfully or fail to allocate memories with any of the parameters.

Parameters
- **`size`** -- Size, in bytes, of the amount of memory to allocate
- **`num`** -- Number of variable parameters

Returns A pointer to the memory allocated on success, NULL on failure

void ***`heap_caps_realloc_prefer`**(void *ptr, size_t size, size_t num, ...)

Reallocate a chunk of memory as preference in decreasing order.

Parameters
- **`ptr`** -- Pointer to previously allocated memory, or NULL for a new allocation.
- **`size`** -- Size of the new buffer requested, or 0 to free the buffer.
- **`num`** -- Number of variable parameters

Returns Pointer to a new buffer of size 'size', or NULL if allocation failed.

void ***heap_caps_calloc_prefer**(size_t n, size_t size, size_t num, ...)

Allocate a chunk of memory as preference in decreasing order.

Parameters
- **n** -- Number of continuing chunks of memory to allocate
- **size** -- Size, in bytes, of a chunk of memory to allocate
- **num** -- Number of variable parameters

Returns A pointer to the memory allocated on success, NULL on failure

void **heap_caps_dump**(uint32_t caps)

Dump the full structure of all heaps with matching capabilities.

Prints a large amount of output to serial (because of locking limitations, the output bypasses stdout/stderr). For each (variable sized) block in each matching heap, the following output is printed on a single line:

- Block address (the data buffer returned by malloc is 4 bytes after this if heap debugging is set to Basic, or 8 bytes otherwise).
- Data size (the data size may be larger than the size requested by malloc, either due to heap fragmentation or because of heap debugging level).
- Address of next block in the heap.
- If the block is free, the address of the next free block is also printed.

Parameters **caps** -- Bitwise OR of MALLOC_CAP_* flags indicating the type of memory

void **heap_caps_dump_all**(void)

Dump the full structure of all heaps.

Covers all registered heaps. Prints a large amount of output to serial.

Output is the same as for heap_caps_dump.

size_t **heap_caps_get_allocated_size**(void *ptr)

Return the size that a particular pointer was allocated with.

Note: The app will crash with an assertion failure if the pointer is not valid.

Parameters **ptr** -- Pointer to currently allocated heap memory. Must be a pointer value previously returned by heap_caps_malloc, malloc, calloc, etc. and not yet freed.

Returns Size of the memory allocated at this block.

void **heap_caps_walk**(uint32_t caps, *heap_caps_walker_cb_t* walker_func, void *user_data)

Function called to walk through the heaps with the given set of capabilities.

Parameters
- **caps** -- The set of capabilities assigned to the heaps to walk through
- **walker_func** -- Callback called for each block of the heaps being traversed
- **user_data** -- Opaque pointer to user defined data

void **heap_caps_walk_all**(*heap_caps_walker_cb_t* walker_func, void *user_data)

Function called to walk through all heaps defined by the heap component.

Parameters
- **walker_func** -- Callback called for each block of the heaps being traversed
- **user_data** -- Opaque pointer to user defined data

Structures

struct **walker_heap_info**

Structure used to store heap related data passed to the walker callback function.

Public Members

intptr_t **start**

Start address of the heap in which the block is located.

intptr_t **end**

End address of the heap in which the block is located.

struct **walker_block_info**

Structure used to store block related data passed to the walker callback function.

Public Members

void ***ptr**

Pointer to the block data.

size_t **size**

The size of the block.

bool **used**

Block status. True: used, False: free.

Macros

HEAP_IRAM_ATTR

MALLOC_CAP_EXEC

Flags to indicate the capabilities of the various memory systems.

Memory must be able to run executable code

MALLOC_CAP_32BIT

Memory must allow for aligned 32-bit data accesses.

MALLOC_CAP_8BIT

Memory must allow for 8/16/...-bit data accesses.

MALLOC_CAP_DMA

Memory must be able to accessed by DMA.

MALLOC_CAP_PID2

Memory must be mapped to PID2 memory space (PIDs are not currently used)

MALLOC_CAP_PID3

Memory must be mapped to PID3 memory space (PIDs are not currently used)

MALLOC_CAP_PID4

Memory must be mapped to PID4 memory space (PIDs are not currently used)

MALLOC_CAP_PID5

Memory must be mapped to PID5 memory space (PIDs are not currently used)

MALLOC_CAP_PID6

Memory must be mapped to PID6 memory space (PIDs are not currently used)

MALLOC_CAP_PID7

Memory must be mapped to PID7 memory space (PIDs are not currently used)

MALLOC_CAP_SPIRAM

Memory must be in SPI RAM.

MALLOC_CAP_INTERNAL

Memory must be internal; specifically it should not disappear when flash/spiram cache is switched off.

MALLOC_CAP_DEFAULT

Memory can be returned in a non-capability-specific memory allocation (e.g. malloc(), calloc()) call.

MALLOC_CAP_IRAM_8BIT

Memory must be in IRAM and allow unaligned access.

MALLOC_CAP_RETENTION

Memory must be able to accessed by retention DMA.

MALLOC_CAP_RTCRAM

Memory must be in RTC fast memory.

MALLOC_CAP_TCM

Memory must be in TCM memory.

MALLOC_CAP_DMA_DESC_AHB

Memory must be capable of containing AHB DMA descriptors.

MALLOC_CAP_DMA_DESC_AXI

Memory must be capable of containing AXI DMA descriptors.

MALLOC_CAP_CACHE_ALIGNED

Memory must be aligned to the cache line size of any intermediate caches.

MALLOC_CAP_INVALID

Memory can't be used / list end marker.

Type Definitions

typedef void (*`esp_alloc_failed_hook_t`)(size_t size, uint32_t caps, const char *function_name)

callback called when an allocation operation fails, if registered

Param size in bytes of failed allocation
Param caps capabilities requested of failed allocation
Param function_name function which generated the failure

typedef struct *walker_heap_info* `walker_heap_into_t`

Structure used to store heap related data passed to the walker callback function.

typedef struct *walker_block_info* `walker_block_info_t`

Structure used to store block related data passed to the walker callback function.

typedef bool (*`heap_caps_walker_cb_t`)(*walker_heap_into_t* heap_info, *walker_block_info_t* block_info, void *user_data)

Function callback used to get information of memory block during calls to heap_caps_walk or heap_caps_walk_all.

Param heap_info See walker_heap_into_t
Param block_info See walker_block_info_t
Param user_data Opaque pointer to user defined data
Return True to proceed with the heap traversal False to stop the traversal of the current heap and continue with the traversal of the next heap (if any)

API Reference - Initialisation

Header File

- components/heap/include/esp_heap_caps_init.h
- This header file can be included with:

```
#include "esp_heap_caps_init.h"
```

Functions

void `heap_caps_init`(void)

Initialize the capability-aware heap allocator.

This is called once in the IDF startup code. Do not call it at other times.

void `heap_caps_enable_nonos_stack_heaps`(void)

Enable heap(s) in memory regions where the startup stacks are located.

On startup, the pro/app CPUs have a certain memory region they use as stack, so we cannot do allocations in the regions these stack frames are. When FreeRTOS is completely started, they do not use that memory anymore and heap(s) there can be enabled.

esp_err_t `heap_caps_add_region`(intptr_t start, intptr_t end)

Add a region of memory to the collection of heaps at runtime.

Most memory regions are defined in soc_memory_layout.c for the SoC, and are registered via heap_caps_init(). Some regions can't be used immediately and are later enabled via heap_caps_enable_nonos_stack_heaps().

Call this function to add a region of memory to the heap at some later time.

This function does not consider any of the "reserved" regions or other data in soc_memory_layout, caller needs to consider this themselves.

All memory within the region specified by start & end parameters must be otherwise unused.

The capabilities of the newly registered memory will be determined by the start address, as looked up in the regions specified in soc_memory_layout.c.

Use heap_caps_add_region_with_caps() to register a region with custom capabilities.

Note: Please refer to following example for memory regions allowed for addition to heap based on an existing region (address range for demonstration purpose only):

```
Existing region: 0x1000 <-> 0x3000
New region:      0x1000 <-> 0x3000 (Allowed)
New region:      0x1000 <-> 0x2000 (Allowed)
New region:      0x0000 <-> 0x1000 (Allowed)
New region:      0x3000 <-> 0x4000 (Allowed)
New region:      0x0000 <-> 0x2000 (NOT Allowed)
New region:      0x0000 <-> 0x4000 (NOT Allowed)
New region:      0x1000 <-> 0x4000 (NOT Allowed)
New region:      0x2000 <-> 0x4000 (NOT Allowed)
```

Parameters

- **start** -- Start address of new region.
- **end** -- End address of new region.

Returns ESP_OK on success, ESP_ERR_INVALID_ARG if a parameter is invalid, ESP_ERR_NOT_FOUND if the specified start address doesn't reside in a known region, or any error returned by heap_caps_add_region_with_caps().

esp_err_t **heap_caps_add_region_with_caps** (const uint32_t caps[], intptr_t start, intptr_t end)

Add a region of memory to the collection of heaps at runtime, with custom capabilities.

Similar to heap_caps_add_region(), only custom memory capabilities are specified by the caller.

Note: Please refer to following example for memory regions allowed for addition to heap based on an existing region (address range for demonstration purpose only):

```
Existing region: 0x1000 <-> 0x3000
New region:      0x1000 <-> 0x3000 (Allowed)
New region:      0x1000 <-> 0x2000 (Allowed)
New region:      0x0000 <-> 0x1000 (Allowed)
New region:      0x3000 <-> 0x4000 (Allowed)
New region:      0x0000 <-> 0x2000 (NOT Allowed)
New region:      0x0000 <-> 0x4000 (NOT Allowed)
New region:      0x1000 <-> 0x4000 (NOT Allowed)
New region:      0x2000 <-> 0x4000 (NOT Allowed)
```

Parameters

- **caps** -- Ordered array of capability masks for the new region, in order of priority. Must have length SOC_MEMORY_TYPE_NO_PRIOS. Does not need to remain valid after the call returns.
- **start** -- Start address of new region.
- **end** -- End address of new region.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if a parameter is invalid
- ESP_ERR_NO_MEM if no memory to register new heap.
- ESP_ERR_INVALID_SIZE if the memory region is too small to fit a heap
- ESP_FAIL if region overlaps the start and/or end of an existing region

API Reference - Multi-Heap API

(Note: The multi-heap API is used internally by the heap capabilities allocator. Most ESP-IDF programs never need to call this API directly.)

Header File

- components/heap/include/multi_heap.h
- This header file can be included with:

```
#include "multi_heap.h"
```

Functions

void ***multi_heap_aligned_alloc**(*multi_heap_handle_t* heap, size_t size, size_t alignment)

allocate a chunk of memory with specific alignment

Parameters
- **heap** -- Handle to a registered heap.
- **size** -- size in bytes of memory chunk
- **alignment** -- how the memory must be aligned

Returns pointer to the memory allocated, NULL on failure

void ***multi_heap_malloc**(*multi_heap_handle_t* heap, size_t size)

malloc() a buffer in a given heap

Semantics are the same as standard malloc(), only the returned buffer will be allocated in the specified heap.

Parameters
- **heap** -- Handle to a registered heap.
- **size** -- Size of desired buffer.

Returns Pointer to new memory, or NULL if allocation fails.

void **multi_heap_aligned_free**(*multi_heap_handle_t* heap, void *p)

free() a buffer aligned in a given heap.

Note: This function is deprecated, consider using multi_heap_free() instead

---

Parameters
- **heap** -- Handle to a registered heap.
- **p** -- NULL, or a pointer previously returned from multi_heap_aligned_alloc() for the same heap.

void **multi_heap_free**(*multi_heap_handle_t* heap, void *p)

free() a buffer in a given heap.

Semantics are the same as standard free(), only the argument 'p' must be NULL or have been allocated in the specified heap.

Parameters
- **heap** -- Handle to a registered heap.
- **p** -- NULL, or a pointer previously returned from multi_heap_malloc() or multi_heap_realloc() for the same heap.

void ***multi_heap_realloc**(*multi_heap_handle_t* heap, void *p, size_t size)

realloc() a buffer in a given heap.

Semantics are the same as standard realloc(), only the argument 'p' must be NULL or have been allocated in the specified heap.

Parameters

- **heap** -- Handle to a registered heap.
- **p** -- NULL, or a pointer previously returned from multi_heap_malloc() or multi_heap_realloc() for the same heap.
- **size** -- Desired new size for buffer.

Returns New buffer of 'size' containing contents of 'p', or NULL if reallocation failed.

size_t **multi_heap_get_allocated_size**(*multi_heap_handle_t* heap, void *p)

Return the size that a particular pointer was allocated with.

Parameters

- **heap** -- Handle to a registered heap.
- **p** -- Pointer, must have been previously returned from multi_heap_malloc() or multi_heap_realloc() for the same heap.

Returns Size of the memory allocated at this block. May be more than the original size argument, due to padding and minimum block sizes.

multi_heap_handle_t **multi_heap_register**(void *start, size_t size)

Register a new heap for use.

This function initialises a heap at the specified address, and returns a handle for future heap operations.

There is no equivalent function for deregistering a heap - if all blocks in the heap are free, you can immediately start using the memory for other purposes.

Parameters

- **start** -- Start address of the memory to use for a new heap.
- **size** -- Size (in bytes) of the new heap.

Returns Handle of a new heap ready for use, or NULL if the heap region was too small to be initialised.

void **multi_heap_set_lock**(*multi_heap_handle_t* heap, void *lock)

Associate a private lock pointer with a heap.

The lock argument is supplied to the MULTI_HEAP_LOCK() and MULTI_HEAP_UNLOCK() macros, defined in multi_heap_platform.h.

The lock in question must be recursive.

When the heap is first registered, the associated lock is NULL.

Parameters

- **heap** -- Handle to a registered heap.
- **lock** -- Optional pointer to a locking structure to associate with this heap.

void **multi_heap_dump**(*multi_heap_handle_t* heap)

Dump heap information to stdout.

For debugging purposes, this function dumps information about every block in the heap to stdout.

Parameters **heap** -- Handle to a registered heap.

bool **multi_heap_check**(*multi_heap_handle_t* heap, bool print_errors)

Check heap integrity.

Walks the heap and checks all heap data structures are valid. If any errors are detected, an error-specific message can be optionally printed to stderr. Print behaviour can be overridden at compile time by defining MULTI_CHECK_FAIL_PRINTF in multi_heap_platform.h.

Note: This function is not thread-safe as it sets a global variable with the value of print_errors.

Parameters

- **heap** -- Handle to a registered heap.
- **print_errors** -- If true, errors will be printed to stderr.

Returns true if heap is valid, false otherwise.

size_t **multi_heap_free_size**(*multi_heap_handle_t* heap)

Return free heap size.

Returns the number of bytes available in the heap.

Equivalent to the total_free_bytes member returned by multi_heap_get_heap_info().

Note that the heap may be fragmented, so the actual maximum size for a single malloc() may be lower. To know this size, see the largest_free_block member returned by multi_heap_get_heap_info().

Parameters **heap** -- Handle to a registered heap.
Returns Number of free bytes.

size_t **multi_heap_minimum_free_size**(*multi_heap_handle_t* heap)

Return the lifetime minimum free heap size.

Equivalent to the minimum_free_bytes member returned by multi_heap_get_info().

Returns the lifetime "low watermark" of possible values returned from multi_free_heap_size(), for the specified heap.

Parameters **heap** -- Handle to a registered heap.
Returns Number of free bytes.

void **multi_heap_get_info**(*multi_heap_handle_t* heap, *multi_heap_info_t* *info)

Return metadata about a given heap.

Fills a *multi_heap_info_t* structure with information about the specified heap.

Parameters

- **heap** -- Handle to a registered heap.
- **info** -- Pointer to a structure to fill with heap metadata.

void ***multi_heap_aligned_alloc_offs**(*multi_heap_handle_t* heap, size_t size, size_t alignment, size_t offset)

Perform an aligned allocation from the provided offset.

Parameters

- **heap** -- The heap in which to perform the allocation
- **size** -- The size of the allocation
- **alignment** -- How the memory must be aligned
- **offset** -- The offset at which the alignment should start

Returns void* The ptr to the allocated memory

size_t **multi_heap_reset_minimum_free_bytes**(*multi_heap_handle_t* heap)

Reset the minimum_free_bytes value (setting it to free_bytes) and return the former value.

Parameters **heap** -- The heap in which the reset is taking place
Returns size_t the value of minimum_free_bytes before it is reset

void **multi_heap_restore_minimum_free_bytes**(*multi_heap_handle_t* heap, const size_t new_minimum_free_bytes_value)

Set the value of minimum_free_bytes to new_minimum_free_bytes_value or keep the current value of minimum_free_bytes if it is smaller than new_minimum_free_bytes_value.

Parameters

- **heap** -- The heap in which the restore is taking place
- **new_minimum_free_bytes_value** -- The value to restore the minimum_free_bytes to

void **multi_heap_walk**(*multi_heap_handle_t* heap, *multi_heap_walker_cb_t* walker_func, void *user_data)

Call the tlsf_walk_pool function of the heap given as parameter with the walker function passed as parameter.

Parameters

- **heap** -- The heap to traverse
- **walker_func** -- The walker to trigger on each block of the heap
- **user_data** -- Opaque pointer to user defined data

Structures

struct **multi_heap_info_t**

Structure to access heap metadata via multi_heap_get_info.

Public Members

size_t **total_free_bytes**

Total free bytes in the heap. Equivalent to multi_free_heap_size().

size_t **total_allocated_bytes**

Total bytes allocated to data in the heap.

size_t **largest_free_block**

Size of the largest free block in the heap. This is the largest malloc-able size.

size_t **minimum_free_bytes**

Lifetime minimum free heap size. Equivalent to multi_minimum_free_heap_size().

size_t **allocated_blocks**

Number of (variable size) blocks allocated in the heap.

size_t **free_blocks**

Number of (variable size) free blocks in the heap.

size_t **total_blocks**

Total number of (variable size) blocks in the heap.

Type Definitions

typedef struct multi_heap_info *__multi_heap_handle_t__

Opaque handle to a registered heap.

typedef bool (*__multi_heap_walker_cb_t__)(void *block_ptr, size_t block_size, int block_used, void *user_data)

Callback called when walking the given heap blocks of memory.

Param block_ptr Pointer to the block data
Param block_size The size of the block
Param block_used Block status. 0: free, 1: allocated
Param user_data Opaque pointer to user defined data
Return True if the walker is expected to continue the heap traversal False if the walker is expected to stop the traversal of the heap

2.10.15 Memory Management for MMU Supported Memory

Introduction

ESP32-S3 Memory Management Unit (MMU) is relatively simple. It can do memory address translation between physical memory addresses and virtual memory addresses. So CPU can access physical memories via virtual addresses. There are multiple types of virtual memory addresses, which have different capabilities.

ESP-IDF provides a memory mapping driver that manages the relation between these physical memory addresses and virtual memory addresses, so as to achieve some features such as reading from SPI flash via a pointer.

Memory mapping driver is actually a capabilities-based virtual memory address allocator that allows applications to make virtual memory address allocations for different purposes. In the following chapters, we call this driver `esp_mmap` driver.

ESP-IDF also provides a memory synchronization driver which can be used for potential memory desynchronization scenarios.

Physical Memory Types

Memory mapping driver currently supports mapping to following physical memory type(s):

- SPI flash
- PSRAM

Virtual Memory Capabilities

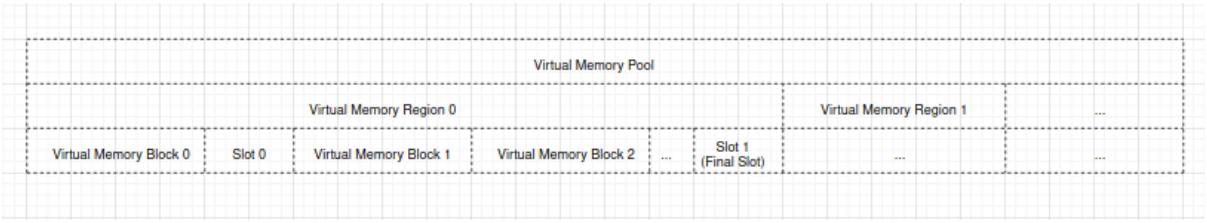
- `MMU_MEM_CAP_EXEC`: This capability indicates that the virtual memory address has the execute permission. Note this permission scope is within the MMU hardware.
- `MMU_MEM_CAP_READ`: This capability indicates that the virtual memory address has the read permission. Note this permission scope is within the MMU hardware.
- `MMU_MEM_CAP_WRITE`: This capability indicates that the virtual memory address has the write permission. Note this permission scope is within the MMU hardware.
- `MMU_MEM_CAP_32BIT`: This capability indicates that the virtual memory address allows for 32 bits or multiples of 32 bits access.
- `MMU_MEM_CAP_8BIT`: This capability indicates that the virtual memory address allows for 8 bits or multiples of 8 bits access.

You can call *`esp_mmu_map_get_max_consecutive_free_block_size()`* to know the largest consecutive mappable block size with certain capabilities.

Memory Management Drivers

Driver Concept

Terminology The virtual memory pool is made up with one or multiple virtual memory regions, see below figure:

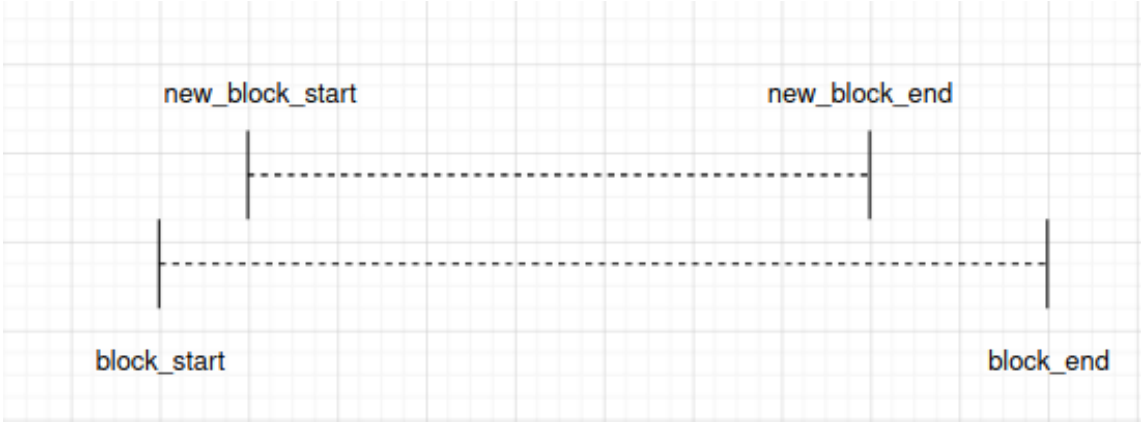


- A virtual memory pool stands for the whole virtual address range that can be mapped to physical memory.
- A virtual memory region is a range of virtual address with same attributes.

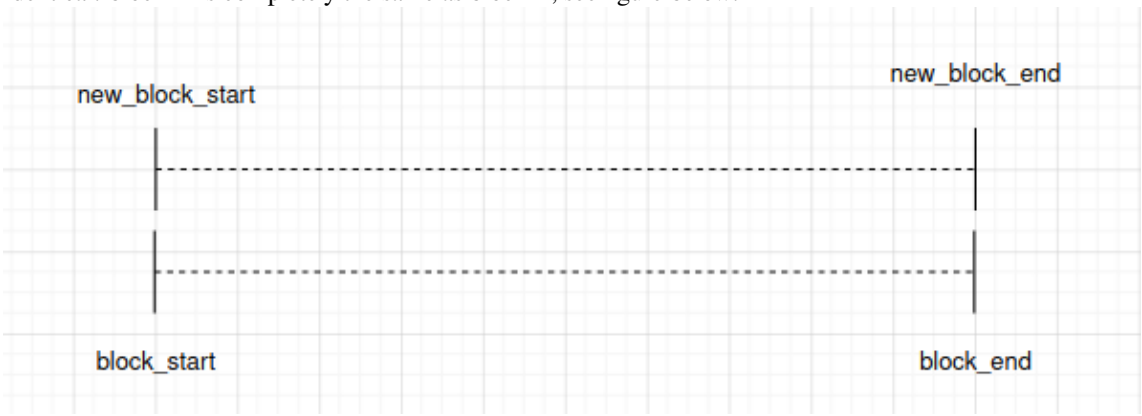
- A virtual memory block is a piece of virtual address range that is dynamically mapped.
- A slot is the virtual address range between two virtual memory blocks.
- A physical memory block is a piece of physical address range that is to-be-mapped or already mapped to a virtual memory block.
- Dynamical mapping is done by calling `esp_mmap` driver API *`esp_mmu_map()`*. This API maps the given physical memory block to a virtual memory block which is allocated by the `esp_mmap` driver.

Relation Between Memory Blocks When mapping a physical memory block A, block A can have one of the following relations with another previously mapped physical memory block B:

- Enclosed: block A is completely enclosed within block B, see figure below:

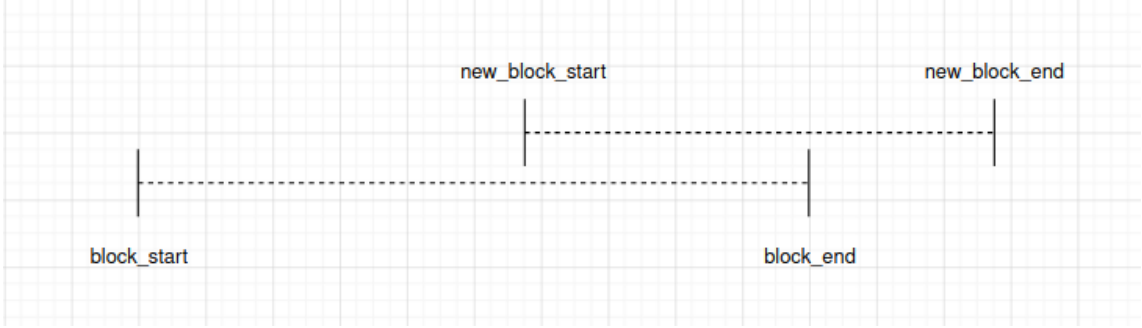


- Identical: block A is completely the same as block B, see figure below:



Note that `esp_mmap` driver considers the identical scenario **the same as the enclosed scenario**.

- Overlapped: block A is overlapped with block B, see figure below:



There is a special condition, when block A entirely encloses block B, see figure below:



Note that `esp_mmap` driver considers this scenario **the same as the overlapped scenario**.

Driver Behaviour

Memory Map You can call *`esp_mmu_map()`* to do a dynamical mapping. This API can allocate a certain size of virtual memory block according to the virtual memory capabilities you selected, then map this virtual memory block to the physical memory block as you requested. The `esp_mmap` driver supports mapping to one or more types of physical memory, so you should specify the physical memory target when mapping.

By default, physical memory blocks and virtual memory blocks are one-to-one mapped. This means, when calling *`esp_mmu_map()`*:

- If it is the enclosed scenario, this API will return an *`ESP_ERR_INVALID_STATE`*. The `out_ptr` will be assigned to the start virtual memory address of the previously mapped one which encloses the to-be-mapped one.
- If it is the identical scenario, this API will behaves exactly the same as the enclosed scenario.
- If it is the overlapped scenario, this API will by default return an *`ESP_ERR_INVALID_ARG`*. This means, `esp_mmap` driver by default does not allow mapping a physical memory address to multiple virtual memory addresses.

Specially, you can use *`ESP_MMU_MMAP_FLAG_PADDR_SHARED`*. This flag stands for one-to-multiple mapping between a physical address and multiple virtual addresses:

- If it is the overlapped scenario, this API will allocate a new virtual memory block as requested, then map to the given physical memory block.

Memory Unmap You can call *`esp_mmu_unmap()`* to unmap a previously mapped memory block. This API returns an *`ESP_ERR_NOT_FOUND`* if you are trying to unmap a virtual memory block that is not mapped to any physical memory block yet.

Memory Address Conversion The `esp_mmap` driver provides two helper APIs to do the conversion between virtual memory address and physical memory address:

- *`esp_mmu_vaddr_to_paddr()`* converts virtual address to physical address.
- *`esp_mmu_paddr_to_vaddr()`* converts physical address to virtual address.

Memory Synchronization MMU supported physical memories can be accessed by one or multiple methods.

SPI flash can be accessed by SPI1 (ESP-IDF `esp_flash` driver APIs), or by pointers. ESP-IDF `esp_flash` driver APIs have already considered the memory synchronization, so users do not need to worry about this.

PSRAM can be accessed by pointers, hardware guarantees the data consistency when PSRAM is only accessed via pointers.

PSRAM can also be accessed by EDMA. Data desynchronization may happen because hardware does not guarantee the data consistency under such condition. You should call *`esp_cache_msync()`* to synchronize the Cache and the PSRAM.

See *Memory Synchronization* for more details.

Thread Safety

APIs in `esp_mmu_map.h` are not guaranteed to be thread-safe.

APIs in `esp_cache.h` are guaranteed to be thread-safe.

API Reference

API Reference - ESP MMAP Driver

Header File

- components/esp_mm/include/esp_mmu_map.h
- This header file can be included with:

```
#include "esp_mmu_map.h"
```

- This header file is a part of the API provided by the `esp_mm` component. To declare that your component depends on `esp_mm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_mm
```

or

```
PRIV_REQUIRES esp_mm
```

Functions

esp_err_t **esp_mmu_map**(*esp_paddr_t* paddr_start, size_t size, mmu_target_t target, mmu_mem_caps_t caps, int flags, void **out_ptr)

Map a physical memory block to external virtual address block, with given capabilities.

Note: This API does not guarantee thread safety

Parameters

- **paddr_start** -- **[in]** Start address of the physical memory block
- **size** -- **[in]** Size to be mapped. Size will be rounded up by to the nearest multiple of MMU page size
- **target** -- **[in]** Physical memory target you're going to map to, see `mmu_target_t`
- **caps** -- **[in]** Memory capabilities, see `mmu_mem_caps_t`
- **flags** -- **[in]** Mmap flags
- **out_ptr** -- **[out]** Start address of the mapped virtual memory

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG: Invalid argument, see printed logs
- ESP_ERR_NOT_SUPPORTED: Only on ESP32, PSRAM is not a supported physical memory target
- ESP_ERR_NOT_FOUND: No enough size free block to use
- ESP_ERR_NO_MEM: Out of memory, this API will allocate some heap memory for internal usage
- ESP_ERR_INVALID_STATE: Paddr is mapped already, this API will return corresponding `vaddr_start + new_block_offset` as per the previously mapped block. Only to-be-mapped paddr block is totally enclosed by a previously mapped block will lead to this error. (Identical scenario will behave similarly) new_block_start new_block_end |------—New Block ------—| |------------—Block ------------—| block_start block_end

esp_err_t **esp_mmu_unmap**(void *ptr)

Unmap a previously mapped virtual memory block.

Note: This API does not guarantee thread safety

Parameters **ptr** -- **[in]** Start address of the virtual memory

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG: Null pointer
- ESP_ERR_NOT_FOUND: Vaddr is not in external memory, or it's not mapped yet

esp_err_t **esp_mmu_map_get_max_consecutive_free_block_size**(mmu_mem_caps_t caps, mmu_target_t target, size_t *out_len)

Get largest consecutive free external virtual memory block size, with given capabilities and given physical target.

Parameters

- **caps** -- **[in]** Bitwise OR of MMU_MEM_CAP_* flags indicating the memory block
- **target** -- **[in]** Physical memory target you're going to map to, see `mmu_target_t`.
- **out_len** -- **[out]** Largest free block length, in bytes.

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG: Invalid arguments, could be null pointer

esp_err_t **esp_mmu_map_dump_mapped_blocks**(FILE *stream)

Dump all the previously mapped blocks

Note: This API shall not be called from an ISR.

Note: This API does not guarantee thread safety

Parameters **stream** -- stream to print information to; use stdout or stderr to print to the console; use fmemopen/open_memstream to print to a string buffer.

Returns

- ESP_OK

esp_err_t **esp_mmu_vaddr_to_paddr**(void *vaddr, *esp_paddr_t* *out_paddr, mmu_target_t *out_target)

Convert virtual address to physical address.

Parameters

- **vaddr** -- **[in]** Virtual address
- **out_paddr** -- **[out]** Physical address
- **out_target** -- **[out]** Physical memory target, see `mmu_target_t`

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG: Null pointer, or vaddr is not within external memory
- ESP_ERR_NOT_FOUND: Vaddr is not mapped yet

esp_err_t **esp_mmu_paddr_to_vaddr**(*esp_paddr_t* paddr, mmu_target_t target, mmu_vaddr_t type, void **out_vaddr)

Convert physical address to virtual address.

Parameters

- **paddr** -- **[in]** Physical address

- **target** -- **[in]** Physical memory target, see `mmu_target_t`
- **type** -- **[in]** Virtual address type, could be either instruction or data
- **out_vaddr** -- **[out]** Virtual address

Returns

- ESP_OK
- ESP_ERR_INVALID_ARG: Null pointer
- ESP_ERR_NOT_FOUND: Paddr is not mapped yet

esp_err_t **esp_mmu_paddr_find_caps**(const *esp_paddr_t* paddr, mmu_mem_caps_t *out_caps)

If the physical address is mapped, this API will provide the capabilities of the virtual address where the physical address is mapped to.

Note: : Only return value is ESP_OK(which means physically address is successfully mapped), then caps you get make sense.

Note: This API only check one page (see CONFIG_MMU_PAGE_SIZE), starting from the `paddr`

Parameters

- **paddr** -- **[in]** Physical address
- **out_caps** -- **[out]** Bitwise OR of MMU_MEM_CAP_* flags indicating the capabilities of a virtual address where the physical address is mapped to.

Returns

- ESP_OK: Physical address successfully mapped.
- ESP_ERR_INVALID_ARG: Null pointer
- ESP_ERR_NOT_FOUND: Physical address is not mapped successfully.

Macros

ESP_MMU_MMAP_FLAG_PADDR_SHARED

Share this mapping.

MMU Memory Mapping Driver APIs for MMU supported memory

Driver Backgrounds:

Type Definitions

typedef uint32_t **esp_paddr_t**

Physical memory type.

2.10.16 Memory Synchronization

Introduction

ESP32-S3 can access its connected PSRAM via these ways:

- CPU
- DMA

By default, CPU accesses the above mentioned memory via cache. Whereas DMA accesses the memory directly, without going through cache.

This leads to potential cache data coherence issue:

- When a DMA transaction changes the content of a piece of memory, and the content has been cached already. Under this condition:

  - CPU may read stale data.
  - the stale data in the cache may be written back to the memory. The new data updated by the previous DMA transaction will be overwritten.
- CPU changes the content of an address. The content is in the cache, but not in the memory yet (cache will write back the content to the memory according to its own strategy). Under this condition:
  - The next DMA transactions to read this content from the memory will get stale data.

There are three common methods to address such cache data coherence issue:

1. Hardware based cache Coherent Interconnect, ESP32-S3 does not have such ability.
2. Use the DMA buffer from non-cacheable memory. Memory that CPU access it without going through cache is called non-cacheable memory.
3. Explicitly call a memory synchronization API to writeback the content in the cache back to the memory, or invalidate the content in the cache.

Memory Synchronisation Driver

The suggested way to deal with such cache data coherence issue is by using the memory synchronization API `esp_cache_msync()` provided by ESP-IDF *esp_mm* component.

Driver Concept Direction of the cache memory synchronization:

- `ESP_CACHE_MSYNC_FLAG_DIR_C2M`, for synchronization from cache to memory.
- `ESP_CACHE_MSYNC_FLAG_DIR_M2C`, for synchronization from memory to cache.

Type of the cache memory synchronization:

- `ESP_CACHE_MSYNC_FLAG_TYPE_DATA`, for synchronization to a data address region.
- `ESP_CACHE_MSYNC_FLAG_TYPE_INST`, for synchronization to an instruction address region.

Driver Behaviour Calling `esp_cache_msync()` will do a synchronization between cache and memory. The first parameter *addr* and the second parameter *size* together describe the memory region that is to be synchronized. About the third parameter *flags*:

- `ESP_CACHE_MSYNC_FLAG_DIR_C2M`. With this flag, content in the specified address region is written back to the memory. This direction is usually used **after** the content of an address is updated by the CPU, e.g. a memset to the address. Operation in this direction should happen **before** a DMA operation to the same address.
- `ESP_CACHE_MSYNC_FLAG_DIR_M2C`. With this flag, content in the specified address region is invalidated from the cache. This direction is usually used **after** the content of an address is updated by the DMA. Operation in this direction should happen **before** a CPU read operation to the same address.

The above two flags help select the synchronization direction. Specially, if neither of these two flags are used, `esp_cache_msync()` will by default select the `ESP_CACHE_MSYNC_FLAG_DIR_C2M` direction. Users are not allowed to set both of the two flags at the same time.

- `ESP_CACHE_MSYNC_FLAG_TYPE_DATA`.
- `ESP_CACHE_MSYNC_FLAG_TYPE_INST`.

The above two flags help select the type of the synchronization address. Specially, if neither of these two flags are used, `esp_cache_msync()` will by default select the `ESP_CACHE_MSYNC_FLAG_TYPE_DATA` direction. Users are not allowed to set both of the two flags at the same time.

- *ESP_CACHE_MSYNC_FLAG_INVALIDATE*. This flag is used to trigger a cache invalidation to the specified address region, after the region is written back to the memory. This flag is mainly used for *ESP_CACHE_MSYNC_FLAG_DIR_C2M* direction. For *ESP_CACHE_MSYNC_FLAG_DIR_M2C* direction, behaviour is the same as if the *ESP_CACHE_MSYNC_FLAG_INVALIDATE* flag is not set.
- *ESP_CACHE_MSYNC_FLAG_UNALIGNED*. This flag force the *esp_cache_msync()* API to do synchronization without checking the address and size alignment. For more details, see chapter *Address Alignment Requirement* following.

Address Alignment Requirement

There is address and size alignment requirement (in bytes) for using *esp_cache_msync()*. The alignment requirement comes from cache.

- An address region whose start address and size both meet the cache memory synchronization alignment requirement is defined as an **aligned address region**.
- An address region whose start address or size does not meet the cache memory synchronization alignment requirement is defined as an **unaligned address region**.

By default, if you specify an unaligned address region, *esp_cache_msync()* will return an *ESP_ERR_INVALID_ARG* error, together with the required alignment.

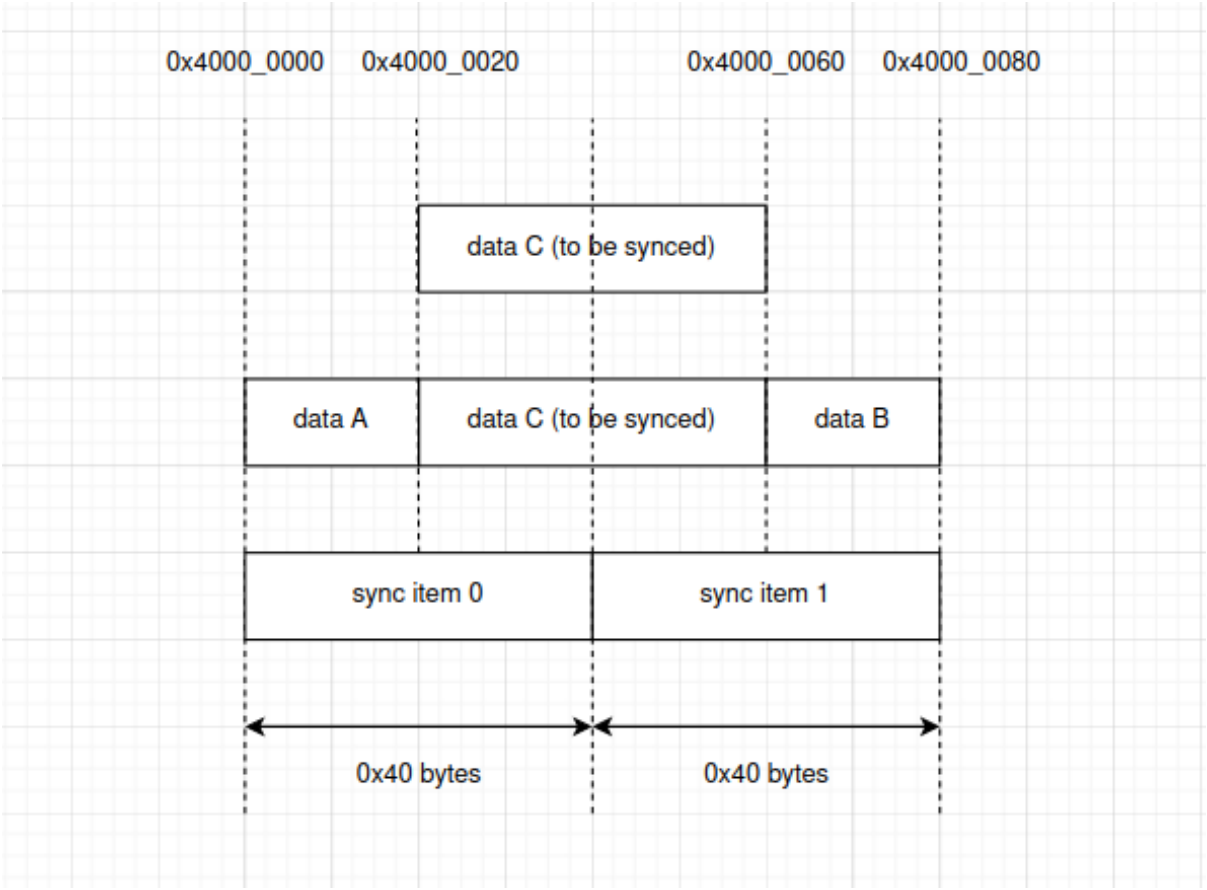
Warning for Address Alignment Requirement You can set the *ESP_CACHE_MSYNC_FLAG_UNALIGNED* flag to bypass such check. Note you should be very careful about using this flag. cache memory synchronization to an unaligned address region may silently corrupt the memory.

For example, assume:

- alignment requirement is 0x40 bytes.
- a call to *esp_cache_msync()*, with *ESP_CACHE_MSYNC_FLAG_DIR_M2C | ESP_CACHE_MSYNC_FLAG_UNALIGNED* flags, the specified address region is 0x4000_0020 ~ 0x4000_0060 (see **data C** in below graph).

Above settings will trigger a cache invalidation to the address region 0x4000_0000 ~ 0x4000_0080, see **sync item0** and **sync item1** in the below graph.

If the content in 0x4000_0000 ~ 0x4000_0020 (**data A** in the below graph) or 0x4000_0060 ~ 0x4000_0080 (**data B** in the below graph) are not written back to the memory yet, then these **data A** and **data B** will be discarded.



API Reference

API Reference - ESP Msync Driver

Header File

- components/esp_mm/include/esp_cache.h
- This header file can be included with:

```
#include "esp_cache.h"
```

- This header file is a part of the API provided by the `esp_mm` component. To declare that your component depends on `esp_mm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_mm
```

or

```
PRIV_REQUIRES esp_mm
```

Functions

esp_err_t **esp_cache_msync** (void *addr, size_t size, int flags)

Memory sync between Cache and storage memory.

For cache-to-memory (C2M) direction:

- For cache writeback supported chips (you can refer to SOC_CACHE_WRITEBACK_SUPPORTED in soc_caps.h)
  - This API will do a writeback to synchronise between cache and storage memory

  - With ESP_CACHE_MSYNC_FLAG_INVALIDATE, this API will also invalidate the values that just written
  - Note: although ESP32 is with PSRAM, but cache writeback isn't supported, so this API will do nothing on ESP32
- For other chips, this API will do nothing. The out-of-sync should be already dealt by the SDK

For memory-to-cache (M2C) direction:

- This API will by default do an invalidation

This API is cache-safe and thread-safe

Note: If you don't set direction (ESP_CACHE_MSYNC_FLAG_DIR_x flags), this API is by default C2M direction

---

Note: If you don't set type (ESP_CACHE_MSYNC_FLAG_TYPE_x flags), this API is by default doing msync for data

---

Note: You should not call this during any Flash operations (e.g. esp_flash APIs, nvs and some other APIs that are based on esp_flash APIs)

Note: If XIP_From_PSRAM is enabled (by enabling both CONFIG_SPIRAM_FETCH_INSTRUCTIONS and CONFIG_SPIRAM_RODATA), you can call this API during Flash operations

---

Parameters

- **`addr`** -- **[in]** Starting address to do the msync
- **`size`** -- **[in]** Size to do the msync
- **`flags`** -- **[in]** Flags, see `ESP_CACHE_MSYNC_FLAG_x`

Returns

- ESP_OK:
  - Successful msync
  - For C2M direction, if this chip doesn't support cache writeback, if the input addr is a cache supported one, this API will return ESP_OK
- ESP_ERR_INVALID_ARG: Invalid argument, not cache supported addr, see printed logs

Macros

`ESP_CACHE_MSYNC_FLAG_INVALIDATE`

Do an invalidation.

Cache msync flags

- For cache-to-memory (C2M) direction: setting this flag will start an invalidation after the cache writeback operation
- For memory-to-cache (M2C) direction: setting / unsetting this flag will behave similarly, trigger an invalidation

`ESP_CACHE_MSYNC_FLAG_UNALIGNED`

Allow msync to a address block that are not aligned to the data cache line size.

`ESP_CACHE_MSYNC_FLAG_DIR_C2M`

Cache msync direction: from Cache to memory.

Note: If you don't set direction (ESP_CACHE_MSYNC_FLAG_DIR_x flags), it is by default cache-to-memory (C2M) direction

ESP_CACHE_MSYNC_FLAG_DIR_M2C

Cache msync direction: from memory to Cache.

ESP_CACHE_MSYNC_FLAG_TYPE_DATA

Cache msync type: data.

Note: If you don't set type (ESP_CACHE_MSYNC_FLAG_TYPE_x flags), it is by default data type

ESP_CACHE_MSYNC_FLAG_TYPE_INST

Cache msync type: instruction.

2.10.17 Heap Memory Debugging

Overview

ESP-IDF integrates tools for requesting *heap information*, *heap corruption detection*, and *heap tracing*. These can help track down memory-related bugs.

For general information about the heap memory allocator, see *Heap Memory Allocation*.

Heap Information

To obtain information about the state of the heap, call the following functions:

- `heap_caps_get_free_size()` can be used to return the current free memory for different memory capabilities.
- `heap_caps_get_largest_free_block()` can be used to return the largest free block in the heap, which is also the largest single allocation currently possible. Tracking this value and comparing it to the total free heap allows you to detect heap fragmentation.
- `heap_caps_get_minimum_free_size()` can be used to track the heap "low watermark" since boot.
- `heap_caps_get_info()` returns a `multi_heap_info_t` structure, which contains the information from the above functions, plus some additional heap-specific data (number of allocations, etc.).
- `heap_caps_print_heap_info()` prints a summary of the information returned by `heap_caps_get_info()` to stdout.
- `heap_caps_dump()` and `heap_caps_dump_all()` output detailed information about the structure of each block in the heap. Note that this can be a large amount of output.

Heap Allocation and Free Function Hooks

Heap allocation and free detection hooks allow you to be notified of every successful allocation and free operation:

- Providing a definition of `esp_heap_trace_alloc_hook()` allows you to be notified of every successful memory allocation operation
- Providing a definition of `esp_heap_trace_free_hook()` allows you to be notified of every successful memory-free operations

This feature can be enabled by setting the *CONFIG_HEAP_USE_HOOKS* option. `esp_heap_trace_alloc_hook()` and `esp_heap_trace_free_hook()` have weak declarations (e.g., `__attribute__((weak))`), thus it is not necessary to provide declarations for both hooks. Given that it is technically possible to allocate and free memory from an ISR (**though strongly discouraged from doing so**), the `esp_heap_trace_alloc_hook()` and `esp_heap_trace_free_hook()` can potentially be called from an ISR.

It is not recommended to perform (or call API functions to perform) blocking operations or memory allocation/free operations in the hook functions. In general, the best practice is to keep the implementation concise and leave the heavy computation outside of the hook functions.

The example below shows how to define the allocation and free function hooks:

```
#include "esp_heap_caps.h"

void esp_heap_trace_alloc_hook(void* ptr, size_t size, uint32_t caps)
{
  ...
}
void esp_heap_trace_free_hook(void* ptr)
{
  ...
}

void app_main()
{
    ...
}
```

Heap Corruption Detection

Heap corruption detection allows you to detect various types of heap memory errors:

- Out-of-bound writes & buffer overflows
- Writes to freed memory
- Reads from freed or uninitialized memory

Assertions The heap implementation (heap/multi_heap.c, etc.) includes numerous assertions that will fail if the heap memory is corrupted. To detect heap corruption most effectively, ensure that assertions are enabled in the project configuration via the *CONFIG_COMPILER_OPTIMIZATION_ASSERTION_LEVEL* option.

If a heap integrity assertion fails, a line will be printed like `CORRUPT HEAP: multi_heap.c:225 detected at 0x3ffbb71c`. The memory address printed is the address of the heap structure that has corrupt content.

It is also possible to manually check heap integrity by calling *`heap_caps_check_integrity_all()`* or related functions. This function checks all of the requested heap memory for integrity and can be used even if assertions are disabled. If the integrity checks detects an error, it will print the error along with the address(es) of corrupt heap structures.

Memory Allocation Failed Hook Users can use *`heap_caps_register_failed_alloc_callback()`* to register a callback that is invoked every time an allocation operation fails.

Additionally, users can enable the *CONFIG_HEAP_ABORT_WHEN_ALLOCATION_FAILS*, which will automatically trigger a system abort if any allocation operation fails.

The example below shows how to register an allocation failure callback:

```
#include "esp_heap_caps.h"

void heap_caps_alloc_failed_hook(size_t requested_size, uint32_t caps, const char␣
↪*function_name)
```

(continues on next page)

(continued from previous page)

```
{
  printf("%s was called but failed to allocate %d bytes with 0x%X capabilities. \n
↪",function_name, requested_size, caps);
}

void app_main()
{
    ...
    esp_err_t error = heap_caps_register_failed_alloc_callback(heap_caps_alloc_
↪failed_hook);
    ...
    void *ptr = heap_caps_malloc(allocation_size, MALLOC_CAP_DEFAULT);
    ...
}
```

Finding Heap Corruption Memory corruption can be one of the hardest classes of bugs to find and fix, as the source of the corruption could be completely unrelated to the symptoms of the corruption. Here are some tips:

- A crash with a `CORRUPT HEAP:` message usually includes a stack trace, but this stack trace is rarely useful. The crash is the symptom of memory corruption when the system realizes the heap is corrupt. But usually, the corruption happens elsewhere and earlier in time.
- Increasing the heap memory debugging *Configuration* level to "Light impact" or "Comprehensive" gives you a more accurate message with the first corrupt memory address.
- Adding regular calls to *`heap_caps_check_integrity_all()`* or *`heap_caps_check_integrity_addr()`* in your code helps you pin down the exact time that the corruption happened. You can move these checks around to "close in on" the section of code that corrupted the heap.
- Based on the memory address that has been corrupted, you can use *JTAG debugging* to set a watchpoint on this address and have the CPU halt when it is written to.
- If you do not have JTAG, but you do know roughly when the corruption happens, set a watchpoint in software just beforehand via *`esp_cpu_set_watchpoint()`*. A fatal exception will occur when the watchpoint triggers. The following is an example of how to use the function - `esp_cpu_set_watchpoint(0, (void *)addr, 4, ESP_WATCHPOINT_STORE)`. Note that watchpoints are per-CPU and are set on the current running CPU only. So if you do not know which CPU is corrupting memory, call this function on both CPUs.
- For buffer overflows, *heap tracing* in `HEAP_TRACE_ALL` mode tells which callers are allocating which addresses from the heap. See *Heap Tracing To Find Heap Corruption* for more details. You can try to find the function that allocates memory with an address immediately before the corrupted address, since it is probably the function that overflows the buffer.
- Calling *`heap_caps_dump()`* or *`heap_caps_dump_all()`* can give an indication of what heap blocks are surrounding the corrupted region and may have overflowed or underflowed, etc.

Configuration Temporarily increasing the heap corruption detection level can give more detailed information about heap corruption errors.

In the project configuration menu, under `Component config`, there is a menu `Heap memory debugging`. The option *CONFIG_HEAP_CORRUPTION_DETECTION* can be set to one of the following three levels:

Basic (No Poisoning) This is the default level. By default, no special heap corruption features are enabled, but the provided assertions are enabled. A heap corruption error will be printed if any of the heap's internal data structures appear overwritten or corrupted. This usually indicates a buffer overrun or out-of-bounds write.

If assertions are enabled, an assertion will also trigger if a double-free occurs (the same memory is freed twice).

Calling *`heap_caps_check_integrity()`* in Basic mode checks the integrity of all heap structures, and print errors if any appear to be corrupted.

Light Impact At this level, heap memory is additionally "poisoned" with head and tail "canary bytes" before and after each block that is allocated. If an application writes outside the bounds of allocated buffers, the canary bytes will be corrupted, and the integrity check will fail.

The head canary word is `0xABBA1234` (`3412BAAB` in byte order), and the tail canary word is `0xBAAD5678` (`7856ADBA` in byte order).

With basic heap corruption checks, most out-of-bound writes can be detected and the number of overrun bytes before a failure is detected depends on the properties of the heap. However, the Light Impact mode is more precise as even a single-byte overrun can be detected.

Enabling light-impact checking increases the memory usage. Each individual allocation uses 9 to 12 additional bytes of memory depending on alignment.

Each time *`heap_caps_free()`* is called in Light Impact mode, the head and tail canary bytes of the buffer being freed are checked against the expected values.

When *`heap_caps_check_integrity()`* is called, all allocated blocks of heap memory have their canary bytes checked against the expected values.

In both cases, the functions involve checking that the first 4 bytes of an allocated block (before the buffer is returned to the user) should be the word `0xABBA1234`, and the last 4 bytes of the allocated block (after the buffer is returned to the user) should be the word `0xBAAD5678`.

Different values usually indicate buffer underrun or overrun. Overrun indicates that when writing to memory, the data written exceeds the size of the allocated memory, resulting in writing to an unallocated memory area; underrun indicates that when reading memory, the data read exceeds the allocated memory and reads data from an unallocated memory area.

Comprehensive This level incorporates the "light impact" detection features plus additional checks for uninitialized-access and use-after-free bugs. In this mode, all freshly allocated memory is filled with the pattern `0xCE`, and all freed memory is filled with the pattern `0xFE`.

Enabling Comprehensive mode has a substantial impact on runtime performance, as all memory needs to be set to the allocation patterns each time a *`heap_caps_malloc()`* or *`heap_caps_free()`* completes, and the memory also needs to be checked each time. However, this mode allows easier detection of memory corruption bugs which are much more subtle to find otherwise. It is recommended to only enable this mode when debugging, not in production.

Crashes in Comprehensive Mode If an application crashes when reading or writing an address related to `0xCECECECE` in Comprehensive mode, it indicates that it has read uninitialized memory. The application should be changed to either use *`heap_caps_calloc()`* (which zeroes memory), or initialize the memory before using it. The value `0xCECECECE` may also be seen in stack-allocated automatic variables, because, in ESP-IDF, most task stacks are originally allocated from the heap, and in C, stack memory is uninitialized by default.

If an application crashes, and the exception register dump indicates that some addresses or values were `0xFEFEFEFE`, this indicates that it is reading heap memory after it has been freed, i.e., a "use-after-free bug". The application should be changed to not access heap memory after it has been freed.

If a call to *`heap_caps_malloc()`* or *`heap_caps_realloc()`* causes a crash because it was expected to find the pattern `0xFEFEFEFE` in free memory and a different pattern was found, it indicates that the app has a use-after-free bug where it is writing to memory that has already been freed.

Manual Heap Checks in Comprehensive Mode Calls to *`heap_caps_check_integrity()`* may print errors relating to `0xFEFEFEFE`, `0xABBA1234`, or `0xBAAD5678`. In each case the checker is expected to find a given pattern, and will error out if not found:

- For free heap blocks, the checker expects to find all bytes set to `0xFE`. Any other values indicate a use-after-free bug where free memory has been incorrectly overwritten.
- For allocated heap blocks, the behavior is the same as for the Light Impact mode. The canary bytes `0xABBA1234` and `0xBAAD5678` are checked at the head and tail of each allocated buffer, and any variation indicates a buffer overrun or underrun.

Heap Task Tracking

Heap Task Tracking can be used to get per-task info for heap memory allocation. The application has to specify the heap capabilities for which the heap allocation is to be tracked.

Example code is provided in system/heap_task_tracking.

Heap Tracing

Heap Tracing allows the tracing of code which allocates or frees memory. Two tracing modes are supported:

- Standalone. In this mode, traced data are kept on-board, so the size of the gathered information is limited by the buffer assigned for that purpose, and the analysis is done by the on-board code. There are a couple of APIs available for accessing and dumping collected info.
- Host-based. This mode does not have the limitation of the standalone mode, because traced data are sent to the host over JTAG connection using app_trace library. Later on, they can be analyzed using special tools.

Heap tracing can perform two functions:

- Leak checking: find memory that is allocated and never freed.
- Heap use analysis: show all functions that are allocating or freeing memory while the trace is running.

How to Diagnose Memory Leaks If you suspect a memory leak, the first step is to figure out which part of the program is leaking memory. Use the *heap_caps_get_free_size()* or related functions in *heap information* to track memory use over the life of the application. Try to narrow the leak down to a single function or sequence of functions where free memory always decreases and never recovers.

Standalone Mode Once you have identified the code which you think is leaking:

- Enable the *CONFIG_HEAP_TRACING_DEST* option.
- Call the function *heap_trace_init_standalone()* early in the program, to register a buffer that can be used to record the memory trace.
- Call the function *heap_trace_start()* to begin recording all mallocs or frees in the system. Call this immediately before the piece of code which you suspect is leaking memory.
- Call the function *heap_trace_stop()* to stop the trace once the suspect piece of code has finished executing.
- Call the function *heap_trace_dump()* to dump the results of the heap trace.

The following code snippet demonstrates how application code would typically initialize, start, and stop heap tracing:

```
#include "esp_heap_trace.h"

#define NUM_RECORDS 100
static heap_trace_record_t trace_record[NUM_RECORDS]; // This buffer must be in␣
↪internal RAM

...

void app_main()
{
    ...
    ESP_ERROR_CHECK( heap_trace_init_standalone(trace_record, NUM_RECORDS) );
    ...
}

void some_function()
{
    ESP_ERROR_CHECK( heap_trace_start(HEAP_TRACE_LEAKS) );

    do_something_you_suspect_is_leaking();
```

(continues on next page)

(continued from previous page)

```
    ESP_ERROR_CHECK( heap_trace_stop() );
    heap_trace_dump();
    ...
}
```

The output from the heap trace has a similar format to the following example:

```
2 allocations trace (100 entry buffer)
32 bytes (@ 0x3ffaf214) allocated CPU 0 ccount 0x2e9b7384 caller␣
↪0x400d276d:0x400d27c1
0x400d276d: leak_some_memory at /path/to/idf/examples/get-started/blink/main/./
↪blink.c:27

0x400d27c1: blink_task at /path/to/idf/examples/get-started/blink/main/./blink.c:52

8 bytes (@ 0x3ffaf804) allocated CPU 0 ccount 0x2e9b79c0 caller␣
↪0x400d2776:0x400d27c1
0x400d2776: leak_some_memory at /path/to/idf/examples/get-started/blink/main/./
↪blink.c:29

0x400d27c1: blink_task at /path/to/idf/examples/get-started/blink/main/./blink.c:52

40 bytes 'leaked' in trace (2 allocations)
total allocations 2 total frees 0
```

Note: The above example output uses *IDF Monitor* to automatically decode PC addresses to their source files and line numbers.

The first line indicates how many allocation entries are in the buffer, compared to its total size.

In `HEAP_TRACE_LEAKS` mode, for each traced memory allocation that has not already been freed, a line is printed with:

- `XX bytes` is the number of bytes allocated.
- `@ 0x...` is the heap address returned from *`heap_caps_malloc()`* or *`heap_caps_calloc()`* .
- `Internal` or `PSRAM` is the general location of the allocated memory.
- `CPU x` is the CPU (0 or 1) running when the allocation was made.
- `ccount 0x...` is the CCOUNT (CPU cycle count) register value the allocation was made. The value is different for CPU 0 vs CPU 1.
- `caller 0x...` gives the call stack of the call to *`heap_caps_malloc()`* or *`heap_caps_free()`* , as a list of PC addresses. These can be decoded to source files and line numbers, as shown above.

The depth of the call stack recorded for each trace entry can be configured in the project configuration menu, under `Heap Memory Debugging` > `Enable heap tracing` > *CONFIG_HEAP_TRACING_STACK_DEPTH*. Up to 32 stack frames can be recorded for each allocation (the default is 2). Each additional stack frame increases the memory usage of each `heap_trace_record_t` record by eight bytes.

Finally, the total number of the 'leaked' bytes (bytes allocated but not freed while the trace is running) is printed together with the total number of allocations it represents.

A warning will be printed if the trace buffer was not large enough to hold all the allocations happened. If you see this warning, consider either shortening the tracing period or increasing the number of records in the trace buffer.

Host-Based Mode Once you have identified the code which you think is leaking:

- In the project configuration menu, navigate to `Component settings` > `Heap Memory Debugging` > *CONFIG_HEAP_TRACING_DEST* and select `Host-Based`.

- In the project configuration menu, navigate to `Component settings` > `Application Level Tracing` > *CONFIG_APPTRACE_DESTINATION1* and select `Trace memory`.
- In the project configuration menu, navigate to `Component settings` > `Application Level Tracing` > `FreeRTOS SystemView Tracing` and enable *CONFIG_APPTRACE_SV_ENABLE*.
- Call the function *heap_trace_init_tohost()* early in the program, to initialize the JTAG heap tracing module.
- Call the function *heap_trace_start()* to begin recording all memory allocation and free calls in the system. Call this immediately before the piece of code which you suspect is leaking memory.
  In host-based mode, the argument to this function is ignored, and the heap tracing module behaves like `HEAP_TRACE_ALL` is passed, i.e., all allocations and deallocations are sent to the host.
- Call the function *heap_trace_stop()* to stop the trace once the suspect piece of code has finished executing.

The following code snippet demonstrates how application code would typically initialize, start, and stop host-based mode heap tracing:

```
#include "esp_heap_trace.h"

...

void app_main()
{
    ...
    ESP_ERROR_CHECK( heap_trace_init_tohost() );
    ...
}

void some_function()
{
    ESP_ERROR_CHECK( heap_trace_start(HEAP_TRACE_LEAKS) );

    do_something_you_suspect_is_leaking();

    ESP_ERROR_CHECK( heap_trace_stop() );
    ...
}
```

To gather and analyze heap trace, do the following on the host:

1. Build the program and download it to the target as described in *Step 5. First Steps on ESP-IDF*.
2. Run OpenOCD (see *JTAG Debugging*).

Note: In order to use this feature, you need OpenOCD version `v0.10.0-esp32-20181105` or later.

3. You can use GDB to start and/or stop tracing automatically. To do this you need to prepare a special `gdbinit` file:

```
target remote :3333

mon reset halt
maintenance flush register-cache

tb heap_trace_start
commands
mon esp sysview start file:///tmp/heap.svdat
c
end

tb heap_trace_stop
commands
mon esp sysview stop
```

(continues on next page)

(continued from previous page)

```
end

c
```

Using this file GDB can connect to the target, reset it, and start tracing when the program hits breakpoint at *heap_trace_start()*. Tracing will be stopped when the program hits breakpoint at *heap_trace_stop()*. Traced data will be saved to `/tmp/heap_log.svdat`.

4. Run GDB using `xtensa-esp32s3-elf-gdb -x gdbinit </path/to/program/elf>`.
5. Quit GDB when the program stops at *heap_trace_stop()*. Traced data are saved in `/tmp/heap.svdat`.
6. Run processing script `$IDF_PATH/tools/esp_app_trace/sysviewtrace_proc.py -p -b </path/to/program/elf> /tmp/heap_log.svdat`.

The output from the heap trace has a similar format to the following example:

```
Parse trace from '/tmp/heap.svdat'...
Stop parsing trace. (Timeout 0.000000 sec while reading 1 bytes!)
Process events from '['/tmp/heap.svdat']'...
[0.002244575] HEAP: Allocated 1 bytes @ 0x3ffaffd8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.002258425] HEAP: Allocated 2 bytes @ 0x3ffaffe0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:48
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.002563725] HEAP: Freed bytes @ 0x3ffaffe0 from task "free" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:31 (discriminator 9)
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.002782950] HEAP: Freed bytes @ 0x3ffb40b8 from task "main" on core 0 by:
/home/user/projects/esp/esp-idf/components/freertos/tasks.c:4590
/home/user/projects/esp/esp-idf/components/freertos/tasks.c:4590

[0.002798700] HEAP: Freed bytes @ 0x3ffb50bc from task "main" on core 0 by:
/home/user/projects/esp/esp-idf/components/freertos/tasks.c:4590
/home/user/projects/esp/esp-idf/components/freertos/tasks.c:4590

[0.102436025] HEAP: Allocated 2 bytes @ 0x3ffaffe0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.102449800] HEAP: Allocated 4 bytes @ 0x3ffaffe8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:48
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.102666150] HEAP: Freed bytes @ 0x3ffaffe8 from task "free" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:31 (discriminator 9)
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.202436200] HEAP: Allocated 3 bytes @ 0x3ffaffe8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)
```

(continues on next page)

(continued from previous page)

```
[0.202451725] HEAP: Allocated 6 bytes @ 0x3ffafff0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:48
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.202667075] HEAP: Freed bytes @ 0x3ffafff0 from task "free" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:31 (discriminator 9)
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.302436000] HEAP: Allocated 4 bytes @ 0x3ffafff0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.302451475] HEAP: Allocated 8 bytes @ 0x3ffb40b8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:48
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.302667500] HEAP: Freed bytes @ 0x3ffb40b8 from task "free" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:31 (discriminator 9)
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

Processing completed.

Processed 1019 events

=============== HEAP TRACE REPORT ===============

Processed 14 heap events.

[0.002244575] HEAP: Allocated 1 bytes @ 0x3ffaffd8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.102436025] HEAP: Allocated 2 bytes @ 0x3ffaffe0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.202436200] HEAP: Allocated 3 bytes @ 0x3ffaffe8 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

[0.302436000] HEAP: Allocated 4 bytes @ 0x3ffafff0 from task "alloc" on core 0 by:
/home/user/projects/esp/esp-idf/examples/system/sysview_tracing_heap_log/main/
↪sysview_heap_log.c:47
/home/user/projects/esp/esp-idf/components/freertos/port.c:355 (discriminator 1)

Found 10 leaked bytes in 4 blocks.
```

Heap Tracing To Find Heap Corruption Heap tracing can also be used to help track down heap corruption. When a region in the heap is corrupted, it may be from some other part of the program that allocated memory at a nearby address.

If you have an approximate idea of when the corruption occurred, enabling heap tracing in `HEAP_TRACE_ALL`

mode allows you to record all the memory allocation functions used and the corresponding allocation addresses.

Using heap tracing in this way is very similar to memory leak detection as described above. For memories that are allocated and not freed, the output is the same. However, records will also be shown for memory that has been freed.

Performance Impact Enabling heap tracing in menuconfig increases the code size of your program, and has a very small negative impact on the performance of heap allocation or free operations even when heap tracing is not running.

When heap tracing is running, heap allocation or free operations are substantially slower than when heap tracing is stopped. Increasing the depth of stack frames recorded for each allocation (see above) also increases this performance impact.

To mitigate the performance loss when the heap tracing is enabled and active, enable *CONFIG_HEAP_TRACE_HASH_MAP*. With this configuration enabled, a hash map mechanism will be used to handle the heap trace records, thus considerably decreasing the heap allocation or free execution time. The size of the hash map can be modified by setting the value of *CONFIG_HEAP_TRACE_HASH_MAP_SIZE*.

By default, the hash map is placed into internal RAM. It can also be placed into external RAM if *CONFIG_HEAP_TRACE_HASH_MAP_IN_EXT_RAM* is enabled. In order to enable this configuration, make sure to enable *CONFIG_SPIRAM* and *CONFIG_SPIRAM_ALLOW_BSS_SEG_EXTERNAL_MEMORY*.

False-Positive Memory Leaks Not everything printed by `heap_trace_dump()` is necessarily a memory leak. The following cases may also be printed:

- Any memory that is allocated after `heap_trace_start()` but freed after `heap_trace_stop()` appears in the leaked dump.
- Allocations may be made by other tasks in the system. Depending on the timing of these tasks, it is quite possible that this memory is freed after `heap_trace_stop()` is called.
- The first time a task uses stdio - e.g., when it calls `heap_caps_printf()` - a lock, i.e., RTOS mutex semaphore, is allocated by the libc. This allocation lasts until the task is deleted.
- Certain uses of `heap_caps_printf()`, such as printing floating point numbers and allocating some memory from the heap on demand. These allocations last until the task is deleted.
- The Bluetooth, Wi-Fi, and TCP/IP libraries allocate heap memory buffers to handle incoming or outgoing data. These memory buffers are usually short-lived, but some may be shown in the heap leak trace if the data has been received or transmitted by the lower levels of the network during the heap tracing.
- TCP connections retain some memory even after they are closed due to the `TIME_WAIT` state. Once the `TIME_WAIT` period is completed, this memory will be freed.

One way to differentiate between "real" and "false positive" memory leaks is to call the suspect code multiple times while tracing is running, and look for patterns (multiple matching allocations) in the heap trace output.

API Reference - Heap Tracing

Header File

- components/heap/include/esp_heap_trace.h
- This header file can be included with:

```
#include "esp_heap_trace.h"
```

Functions

esp_err_t **heap_trace_init_standalone** (*heap_trace_record_t* *record_buffer, size_t num_records)

Initialise heap tracing in standalone mode.

This function must be called before any other heap tracing functions.

To disable heap tracing and allow the buffer to be freed, stop tracing and then call heap_trace_init_standalone(NULL, 0);

Parameters

- **record_buffer** -- Provide a buffer to use for heap trace data. Note: External RAM is allowed, but it prevents recording allocations made from ISR's.
- **num_records** -- Size of the heap trace buffer, as number of record structures.

Returns

- ESP_ERR_NOT_SUPPORTED Project was compiled without heap tracing enabled in menuconfig.
- ESP_ERR_INVALID_STATE Heap tracing is currently in progress.
- ESP_OK Heap tracing initialised successfully.

esp_err_t **heap_trace_init_tohost** (void)

Initialise heap tracing in host-based mode.

This function must be called before any other heap tracing functions.

Returns

- ESP_ERR_INVALID_STATE Heap tracing is currently in progress.
- ESP_OK Heap tracing initialised successfully.

esp_err_t **heap_trace_start** (*heap_trace_mode_t* mode)

Start heap tracing. All heap allocations & frees will be traced, until heap_trace_stop() is called.

Note: heap_trace_init_standalone() must be called to provide a valid buffer, before this function is called.

Note: Calling this function while heap tracing is running will reset the heap trace state and continue tracing.

Parameters **mode** -- Mode for tracing.

- HEAP_TRACE_ALL means all heap allocations and frees are traced.
- HEAP_TRACE_LEAKS means only suspected memory leaks are traced. (When memory is freed, the record is removed from the trace buffer.)

Returns

- ESP_ERR_NOT_SUPPORTED Project was compiled without heap tracing enabled in menuconfig.
- ESP_ERR_INVALID_STATE A non-zero-length buffer has not been set via heap_trace_init_standalone().
- ESP_OK Tracing is started.

esp_err_t **heap_trace_stop** (void)

Stop heap tracing.

Returns

- ESP_ERR_NOT_SUPPORTED Project was compiled without heap tracing enabled in menuconfig.
- ESP_ERR_INVALID_STATE Heap tracing was not in progress.
- ESP_OK Heap tracing stopped..

esp_err_t **heap_trace_resume** (void)

Resume heap tracing which was previously stopped.

Unlike heap_trace_start(), this function does not clear the buffer of any pre-existing trace records.

The heap trace mode is the same as when heap_trace_start() was last called (or HEAP_TRACE_ALL if heap_trace_start() was never called).

Returns

- ESP_ERR_NOT_SUPPORTED Project was compiled without heap tracing enabled in menuconfig.
- ESP_ERR_INVALID_STATE Heap tracing was already started.
- ESP_OK Heap tracing resumed.

size_t **heap_trace_get_count**(void)

Return number of records in the heap trace buffer.

It is safe to call this function while heap tracing is running.

esp_err_t **heap_trace_get**(size_t index, *heap_trace_record_t* *record)

Return a raw record from the heap trace buffer.

Note: It is safe to call this function while heap tracing is running, however in HEAP_TRACE_LEAK mode record indexing may skip entries unless heap tracing is stopped first.

Parameters

- **index** -- Index (zero-based) of the record to return.
- **record** -- **[out]** Record where the heap trace record will be copied.

Returns

- ESP_ERR_NOT_SUPPORTED Project was compiled without heap tracing enabled in menuconfig.
- ESP_ERR_INVALID_STATE Heap tracing was not initialised.
- ESP_ERR_INVALID_ARG Index is out of bounds for current heap trace record count.
- ESP_OK Record returned successfully.

void **heap_trace_dump**(void)

Dump heap trace record data to stdout.

Note: It is safe to call this function while heap tracing is running, however in HEAP_TRACE_LEAK mode the dump may skip entries unless heap tracing is stopped first.

---

void **heap_trace_dump_caps**(const uint32_t caps)

Dump heap trace from the memory of the capabilities passed as parameter.

Parameters **caps** -- Capability(ies) of the memory from which to dump the trace. Set MALLOC_CAP_INTERNAL to dump heap trace data from internal memory. Set MALLOC_CAP_SPIRAM to dump heap trace data from PSRAM. Set both to dump both heap trace data.

esp_err_t **heap_trace_summary**(*heap_trace_summary_t* *summary)

Get summary information about the result of a heap trace.

Note: It is safe to call this function while heap tracing is running.

Structures

struct **heap_trace_record_t**

Trace record data type. Stores information about an allocated region of memory.

Public Members

uint32_t **ccount**

CCOUNT of the CPU when the allocation was made. LSB (bit value 1) is the CPU number (0 or 1).

void ***address**

Address which was allocated. If NULL, then this record is empty.

size_t **size**

Size of the allocation.

void ***alloced_by**[CONFIG_HEAP_TRACING_STACK_DEPTH]

Call stack of the caller which allocated the memory.

void ***freed_by**[CONFIG_HEAP_TRACING_STACK_DEPTH]

Call stack of the caller which freed the memory (all zero if not freed.)

struct **heap_trace_summary_t**

Stores information about the result of a heap trace.

Public Members

heap_trace_mode_t **mode**

The heap trace mode we just completed / are running.

size_t **total_allocations**

The total number of allocations made during tracing.

size_t **total_frees**

The total number of frees made during tracing.

size_t **count**

The number of records in the internal buffer.

size_t **capacity**

The capacity of the internal buffer.

size_t **high_water_mark**

The maximum value that 'count' got to.

size_t **has_overflowed**

True if the internal buffer overflowed at some point.

Macros

CONFIG_HEAP_TRACING_STACK_DEPTH

Type Definitions

typedef struct *heap_trace_record_t* **heap_trace_record_t**

Trace record data type. Stores information about an allocated region of memory.

Enumerations

enum **heap_trace_mode_t**

Values:

enumerator **HEAP_TRACE_ALL**

enumerator **HEAP_TRACE_LEAKS**

2.10.18 ESP Timer (High Resolution Timer)

Overview

The ESP Timer feature allows for creating software timers and invoking their callback functions (dispatching callbacks) on timeout. ESP Timer is useful when user software needs to perform delayed or periodic actions, such as delayed device start/stop or periodic sampling of sensor data.

ESP Timer hides the complexity associated with managing multiple timers, dispatching callbacks, accounting for clock frequency changes (if dynamic frequency scaling is enabled), and maintaining correct time after light sleep.

For application scenarios that require better real-time performance (such as generating waveforms) or configurable timer resolution, it is recommended that *GPTimer* be used instead. Also, GPTimer has features not available in ESP Timer, such as event capture.

Finally, FreeRTOS has its own software timers. As explained in *FreeRTOS Timers*, they have much lower resolution compared to ESP Timer, but FreeRTOS timers are portable (non-dependent on ESP-IDF) which might be an advantage in some cases.

Features and Concepts

The ESP Timer API provides:

- One-shot and periodic timers
- Multiple callback dispatch methods
- Handling overdue callbacks
- Bit range: 52 bits
- Time resolution: 1 microsecond

One-Shot and Periodic Timers A one-shot timer invokes its callback function only once upon expiration and then stops operation. One-shot timers are useful for single delayed actions, such as turning off a device or reading a sensor after a specified time interval.

A periodic timer invokes its callback function upon expiration and restarts itself automatically, resulting in the callback function being invoked at a defined interval until the periodic timer is manually stopped. Periodic timers are useful for repeated actions, such as sampling sensor data, updating display information, or generating a waveform.

Callback Dispatch Methods Timer callbacks can be dispatched using the following methods:

- Task Dispatch method (default):
  - Dispatches timer callbacks from a single high-priority ESP Timer task (esp_timer task (notified by ISR) > callback)
  - Suitable for handling timer callbacks that are not time-critical
- Interrupt Dispatch method (*ESP_TIMER_ISR*):
  - Dispatches timer callbacks directly from an interrupt handler (ISR > callback)
  - Suitable for simple, low-latency timer callbacks which take a few microseconds to run
  - Ensures shorter delay between the event and the callback execution
  - Not affected by other active tasks

Task Dispatch Specifics The execution of callbacks in the ESP Timer task is serialized. Thus, when multiple timeouts occur simultaneously, the execution time of one callback will delay the execution of subsequent callbacks. For this reason, it is recommended to keep the callbacks short. If the callback needs to perform more work, the work should be deferred to a lower-priority task using FreeRTOS primitives, such as queues and semaphores.

If other FreeRTOS tasks with higher priority are running, such as an SPI Flash operation, callback dispatching will be delayed until the ESP Timer task has a chance to run.

To maintain predictable and timely execution of tasks, callbacks should never attempt block (waiting for resources) or yield (give up control) operations, because such operations disrupt the serialized execution of callbacks.

Interrupt Dispatch Specifics Timers using the Interrupt Dispatch method have their callbacks executed from an interrupt handler. As interrupts can preempt all tasks, the Interrupt Dispatch method offers lower latency. Interrupt dispatched timer callbacks should never attempt to block and should not attempt to trigger a context switch via `portYIELD_FROM_ISR()`. Instead, the function *`esp_timer_isr_dispatch_need_yield()`* should be used. The context switch will happen after all ISR dispatch timers are processed.

While using interrupt dispatched timers, the standard logging or debugging methods, such as `printf` should be avoided. To debug an application or display certain information in the console, the ESP-IDF logging macros should be used, such as *`ESP_DRAM_LOGI`*, *`ESP_EARLY_LOGI`*, etc. These macros are specifically designed to work in various contexts, including interrupt service routines.

Obtaining Current Time The time passed since the initialization of ESP Timer can be obtained using the convenience function *`esp_timer_get_time()`*. The initialization happens shortly before the `app_main` function is called. This function is fast and has no locking mechanisms that could potentially introduce delays or conflicts. As a result, it can be useful for fine-grained timing, with the accuracy of 1 us, in tasks as well as in ISR routines.

Unlike the `gettimeofday()` function, *`esp_timer_get_time()`* has the following specifics:

- Upon wakeup from deep sleep, the initialization timer restarts from zero
- The returned value has no timezone settings or daylight saving time adjustments

System Integration

This section mainly covers some aspects of how to optimize the operation of ESP Timer and integrate it with other ESP-IDF features.

Timeout Value Limits As callback dispatching can never be instantaneous, the one-shot and periodic timers created with ESP Timer also have timeout value limits. These limits cannot be estimated precisely, because they depend on multiple factors.

For reference, the ESP32 running at 240 MHz and using the Task Dispatch method has the approximate minimum timeout values as follows:

- One-shot timers: ~20 us
  - If *`esp_timer_start_once()`* is called, this is the earliest time after which the system will be able to dispatch a callback.
- Periodic timers: ~50 us
  - Periodic software timers with a smaller timeout value would simply consume most of the CPU time, which is impractical.

The lower the CPU frequency, the higher the minimum timeout values will be. The general guideline is if the required timeout values are in the order of tens of microseconds, the user application needs to undergo thorough testing to ensure stable operation.

If the minimum timeout values slightly exceed the requirements, the Interrupt Dispatch method might offer an improvement.

For even smaller timeout values, for example, to generate or receive waveforms or do bit banging, the resolution of ESP Timer may be insufficient. In this case, it is recommended to use dedicated peripherals, such as *GPTimer* or *RMT*, and their DMA features if available.

Sleep Mode Considerations If a timer is started, and there are no other tasks being executed during the wait time, the chip can be put into sleep to optimize power consumption.

Sleep can be induced in the following ways:

- **Automatic sleep** provided by *Power Management APIs*: If no tasks are being executed, the chip can automatically enter light sleep and automatically wake up at the appropriate time for ESP Timer to dispatch a pending callback.
- **Manual sleep** provided by *Sleep Mode APIs*: The chip can be put into sleep regardless of whether other tasks are being executed.

For manually induced sleep, the following sleep modes exist:

- Deep-sleep mode: ESP Timer is deactivated
  The user application restarts from scratch upon wakeup from deep sleep. This makes deep sleep unsuitable for continuous ESP Timer operation. However, deep sleep can be used if the running timers are not expected to persist across wakeups.
- Light-sleep mode: ESP Timer is suspended
  While in light sleep, ESP Timer counter and callbacks are suspended. Timekeeping is done by the RTC timer. Once the chip is woken up, the counter of ESP Timer is automatically advanced by the amount of time spent in sleep, then timekeeping and callback execution is resumed.
  At this point, ESP Timer will attempt to dispatch all unhandled callbacks if there are any. It can potentially lead to the overflow of ESP Timer callback execution queue. This behavior may be undesirable for certain applications, and the ways to avoid it are covered in *Handling Callbacks in Light-sleep Mode*.

FreeRTOS Timers Although FreeRTOS provides software timers, they have limitations:

- FreeRTOS timer resolution is bound by the tick frequency, which is typically in the range of 100 to 1000 Hz.
- Timer callbacks are dispatched from a low-priority timer task that can be preempted by other tasks, leading to decreased timer precision and accuracy.

However, FreeRTOS timers are portable (non-dependent on ESP-IDF) and are written to be deterministic as they do not dispatch callbacks from ISRs.

Usage

While setting up your ESP-IDF project, make sure to:

- Add required component dependencies to your `CMakeLists.txt`.
- Include required header files in your `.c` files.
- (Optional) Set Kconfig options. For this, see *Kconfig Options* > *ESP Timer (High Resolution Timer)*

General Procedure The general procedure to create, start, stop, and delete a timer is as follows:

1. Create a timer
   - Define a timer handle using the type `esp_timer_handle_t`
   - Set the timer configuration parameters by defining the structure `esp_timer_create_args_t` which also includes the callback function

     Note: It is recommended to keep callbacks as short as possible to avoid delaying other callbacks.

   - To create a timer, call the function `esp_timer_create()`
2. Start the timer in one-shot mode or periodic mode depending on your requirements
   - To start the timer in one-shot mode, call `esp_timer_start_once()`

- To start the timer in periodic mode, call `esp_timer_start_periodic()`; the timer will continue running until you explicitly stop it using `esp_timer_stop()`

Note: When executing a start function, ensure that the timer is not running. If a timer is running, either call `esp_timer_restart()` or stop it first using `esp_timer_stop()` and then call one of the start functions.

---

3. Stop the timer
   - To stop the running timer, call the function `esp_timer_stop()`
4. Delete the timer
   - When the timer is no longer needed, delete it to free up memory using the function `esp_timer_delete()`

Using the Interrupt Dispatch Method Out of the available *callback methods*, if you choose the Interrupt Dispatch method, follow these steps:

1. Set Kconfig options
   - Enable *CONFIG_ESP_TIMER_SUPPORTS_ISR_DISPATCH_METHOD*
2. Create a timer
   - Set the timer configuration parameters by defining the structure `esp_timer_create_args_t`

```
const esp_timer_create_args_t timer = {
  ... ,
  .dispatch_method = ESP_TIMER_ISR,
  ...
};
```

   - To create a timer, call the function `esp_timer_create()`

For further steps, refer to *General Procedure*.

Handling Callbacks in Light-sleep Mode Light sleep allows you to save power while maintaining the ability to quickly wake up for specific actions. To use ESP Timer in conjunction with Light-sleep mode, see *Sleep Mode APIs*.

During light sleep, to keep unhandled callbacks under control and avoid potential overflow of ESP Timer callback execution queue on wakeup, do one of the following:

- Prevent the invocation of callbacks in the first place: stop the timer before entering light sleep by using `esp_timer_stop()`.
- If calling the stop function is not desirable for any reason, use the option `esp_timer_create_args_t::skip_unhandled_events`. In this case, if a periodic timer expires one or more times during light sleep, then only one callback is executed on wakeup.

Debugging Timers The function `esp_timer_dump()` allows dumping information about either all or only running timers: the parameters for timers, the number of times the timers were started, triggered, skipped, and time taken by timer callbacks to execute. This information can be helpful in debugging.

To debug timers, use the following procedure:

1. Set Kconfig options for more detailed output:
   - Enable *CONFIG_ESP_TIMER_PROFILING*

   Note that enabling this option increases code size and heap memory usage.
2. Wherever required in your code, call the function `esp_timer_dump()` to print the information and use it to debug your timers.
3. Once debugging is complete, consider disabling *CONFIG_ESP_TIMER_PROFILING*.

Troubleshooting

Unstable Callback Dispatch Time While dispatching the same callback function repeatedly, if the response time varies considerably, try to stabilize it by doing the following:

- Use the *ESP_TIMER_ISR callback method*
- Use the Kconfig option *CONFIG_ESP_TIMER_TASK_AFFINITY* to run the ESP Timer task on both cores

Significant Delays Dispatching Callbacks If dispatching a callback function takes a considerable amount of time, the problem can lie in the callback function itself. More precisely, as all callback functions are processed one by one in a single esp_timer task, the delays might be caused by other callback functions earlier in the queue.

For this reason, make sure that all callback functions in your application can execute on their own quickly and without any blocking operations.

Repeated Callback Dispatches After Sleep If the callback functions are executed repeatedly upon wakeup from sleep, see *Handling Callbacks in Light-sleep Mode*.

Stack Overflow While Dispatching Callbacks If you see a stack overflow error when executing a callback function, consider reducing the stack usage within your callback function. Alternatively, try increasing the size of the ESP Timer task stack by adjusting *CONFIG_ESP_TIMER_TASK_STACK_SIZE*.

Application Examples

- system/esp_timer creates and starts one-shot and periodic software timers, shows how they work with Light-sleep mode, and then stops and deletes the timers

API Reference

Header File

- components/esp_timer/include/esp_timer.h
- This header file can be included with:

```
#include "esp_timer.h"
```

- This header file is a part of the API provided by the `esp_timer` component. To declare that your component depends on `esp_timer`, add the following to your CMakeLists.txt:

```
REQUIRES esp_timer
```

or

```
PRIV_REQUIRES esp_timer
```

Functions

esp_err_t **esp_timer_early_init**(void)

Minimal initialization of esp_timer.

This function can be called very early in startup process, after this call only *esp_timer_get_time()* function can be used.

Note: This function is called from startup code. Applications do not need to call this function before using other esp_timer APIs.

Returns

- ESP_OK on success

esp_err_t **esp_timer_init**(void)

Initialize esp_timer library.

This function will be called from startup code on every core. If Kconfig option `CONFIG_ESP_TIMER_ISR_AFFINITY` is set to `NO_AFFINITY`, it allocates the timer ISR on MULTIPLE cores and creates the timer task which can be run on any core.

Note: This function is called from startup code. Applications do not need to call this function before using other esp_timer APIs. Before calling this function, *esp_timer_early_init()* must be called by the startup code.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if allocation has failed
- ESP_ERR_INVALID_STATE if already initialized
- other errors from interrupt allocator

esp_err_t **esp_timer_deinit**(void)

De-initialize esp_timer library.

Note: Normally this function should not be called from applications

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if not yet initialized

esp_err_t **esp_timer_create**(const *esp_timer_create_args_t* *create_args, *esp_timer_handle_t* *out_handle)

Create an esp_timer instance.

Note: When timer no longer needed, delete it using *esp_timer_delete()*.

Parameters

- **create_args** -- Pointer to a structure with timer creation arguments. Not saved by the library, can be allocated on the stack.
- **out_handle** -- **[out]** Output, pointer to esp_timer_handle_t variable that holds the created timer handle.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if some of the create_args are not valid
- ESP_ERR_INVALID_STATE if esp_timer library is not initialized yet
- ESP_ERR_NO_MEM if memory allocation fails

esp_err_t **esp_timer_start_once**(*esp_timer_handle_t* timer, uint64_t timeout_us)

Start a one-shot timer.

Timer represented by `timer` should not be running when this function is called.

Parameters

- **timer** -- timer handle created using *esp_timer_create()*
- **timeout_us** -- timer timeout, in microseconds relative to the current moment

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle is invalid
- ESP_ERR_INVALID_STATE if the timer is already running

esp_err_t **esp_timer_start_periodic**(*esp_timer_handle_t* timer, uint64_t period)

Start a periodic timer.

Timer represented by `timer` should not be running when this function is called. This function starts the timer which will trigger every `period` microseconds.

Parameters
- **timer** -- timer handle created using *esp_timer_create()*
- **period** -- timer period, in microseconds

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle is invalid
- ESP_ERR_INVALID_STATE if the timer is already running

esp_err_t **esp_timer_restart**(*esp_timer_handle_t* timer, uint64_t timeout_us)

Restart a currently running timer.

Type of `timer`	Action
One-shot timer	Restarted immediately and times out once in `timeout_us` microseconds
Periodic timer	Restarted immediately with a new period of `timeout_us` microseconds

Parameters
- **timer** -- timer handle created using *esp_timer_create()*
- **timeout_us** -- Timeout in microseconds relative to the current time. In case of a periodic timer, also represents the new period.

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle is invalid
- ESP_ERR_INVALID_STATE if the timer is not running

esp_err_t **esp_timer_stop**(*esp_timer_handle_t* timer)

Stop a running timer.

This function stops the timer previously started using *esp_timer_start_once()* or *esp_timer_start_periodic()*.

Parameters **timer** -- timer handle created using *esp_timer_create()*

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE if the timer is not running

esp_err_t **esp_timer_delete**(*esp_timer_handle_t* timer)

Delete an esp_timer instance.

The timer must be stopped before deleting. A one-shot timer which has expired does not need to be stopped.

Parameters **timer** -- timer handle created using *esp_timer_create()*

Returns
- ESP_OK on success
- ESP_ERR_INVALID_STATE if the timer is running

int64_t **esp_timer_get_time**(void)

Get time in microseconds since boot.

Returns Number of microseconds since the initialization of ESP Timer

int64_t **esp_timer_get_next_alarm**(void)

Get the timestamp of the next expected timeout.

Returns Timestamp of the nearest timer event, in microseconds. The timebase is the same as for the values returned by *esp_timer_get_time()*.

int64_t **esp_timer_get_next_alarm_for_wake_up**(void)

Get the timestamp of the next expected timeout excluding those timers that should not interrupt light sleep (such timers have *esp_timer_create_args_t::skip_unhandled_events* enabled)

Returns Timestamp of the nearest timer event, in microseconds. The timebase is the same as for the values returned by *esp_timer_get_time()*.

esp_err_t **esp_timer_get_period**(*esp_timer_handle_t* timer, uint64_t *period)

Get the period of a timer.

This function fetches the timeout period of a timer. For a one-shot timer, the timeout period will be 0.

Parameters
- **timer** -- timer handle created using *esp_timer_create()*
- **period** -- memory to store the timer period value in microseconds

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if the arguments are invalid

esp_err_t **esp_timer_get_expiry_time**(*esp_timer_handle_t* timer, uint64_t *expiry)

Get the expiry time of a one-shot timer.

This function fetches the expiry time of a one-shot timer.

Note: Passing the timer handle of a periodic timer will result in an error.

Parameters
- **timer** -- timer handle created using *esp_timer_create()*
- **expiry** -- memory to store the timeout value in microseconds

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if the arguments are invalid
- ESP_ERR_NOT_SUPPORTED if the timer type is periodic

esp_err_t **esp_timer_dump**(FILE *stream)

Dump the list of timers to a stream.

By default, this function prints the list of active (running) timers. The output format is:

| Name | Period | Alarm |

- Name —timer pointer
- Period —period of timer in microseconds, or 0 for one-shot timer
- Alarm - time of the next alarm in microseconds since boot, or 0 if the timer is not started

To print the list of all created timers, enable Kconfig option `CONFIG_ESP_TIMER_PROFILING`. In this case, the output format is:

| Name | Period | Alarm | Times_armed | Times_trigg | Times_skip | Cb_exec_time |

- Name —timer name
- Period —same as above
- Alarm —same as above
- Times_armed —number of times the timer was armed via esp_timer_start_X
- Times_triggered - number of times the callback was triggered
- Times_skipped - number of times the callback was skipped
- Callback_exec_time - total time taken by callback to execute, across all calls

Parameters **stream** -- stream (such as stdout) to which to dump the information

Returns
- ESP_OK on success
- ESP_ERR_NO_MEM if can not allocate temporary buffer for the output

void **esp_timer_isr_dispatch_need_yield**(void)

Requests a context switch from a timer callback function.

This only works for a timer that has an ISR dispatch method. The context switch will be called after all ISR dispatch timers have been processed.

bool **esp_timer_is_active**(*esp_timer_handle_t* timer)

Returns status of a timer, active or not.

This function is used to identify if the timer is still active (running) or not.

Parameters **timer** -- timer handle created using *esp_timer_create()*
Returns
- 1 if timer is still active (running)
- 0 if timer is not active

esp_err_t **esp_timer_new_etm_alarm_event**(esp_etm_event_handle_t *out_event)

Get the ETM event handle of esp_timer underlying alarm event.

Note: The created ETM event object can be deleted later using esp_etm_del_event()

Note: The ETM event is generated by the underlying hardware - systimer; therefore, if the esp_timer is not clocked by systimer, then no ETM event will be generated.

Parameters **out_event** -- **[out]** Returned ETM event handle
Returns
- ESP_OK Success
- ESP_ERR_INVALID_ARG Parameter error

Structures

struct **esp_timer_create_args_t**

Timer configuration passed to *esp_timer_create()*

Public Members

esp_timer_cb_t **callback**

Callback function to execute when timer expires.

void ***arg**

Argument to pass to callback.

esp_timer_dispatch_t **dispatch_method**

Dispatch callback from task or ISR; if not specified, esp_timer task.

const char ***name**

Timer name, used in *esp_timer_dump()* function.

bool **skip_unhandled_events**

Setting to skip unhandled events in light sleep for periodic timers.

Type Definitions

typedef struct esp_timer ***esp_timer_handle_t**

Opaque type representing a single timer handle.

typedef void (***esp_timer_cb_t**)(void *arg)

Timer callback function type.

Param arg pointer to opaque user-specific data

Enumerations

enum **esp_timer_dispatch_t**

Method to dispatch timer callback.

Values:

enumerator **ESP_TIMER_TASK**

Callback is dispatched from esp_timer task.

enumerator **ESP_TIMER_ISR**

Callback is dispatched from interrupt handler.

enumerator **ESP_TIMER_MAX**

Sentinel value for the number of callback dispatch methods.

2.10.19 Internal and Unstable APIs

This section is listing some APIs that are internal or likely to be changed or removed in the next releases of ESP-IDF.

API Reference

Header File

- components/esp_rom/include/esp_rom_sys.h
- This header file can be included with:

```
#include "esp_rom_sys.h"
```

Functions

void **esp_rom_software_reset_system**(void)

Software Reset digital core include RTC.

It is not recommended to use this function in esp-idf, use esp_restart() instead.

void **esp_rom_software_reset_cpu**(int cpu_no)

Software Reset cpu core.

It is not recommended to use this function in esp-idf, use esp_restart() instead.

Parameters **cpu_no** -- : The CPU to reset, 0 for PRO CPU, 1 for APP CPU.

int **esp_rom_printf**(const char *fmt, ...)

Print formated string to console device.

Note: float and long long data are not supported!

Parameters

- **fmt** -- Format string
- **...** -- Additional arguments, depending on the format string

Returns int: Total number of characters written on success; A negative number on failure.

void **esp_rom_delay_us**(uint32_t us)

Pauses execution for us microseconds.

Parameters **us** -- Number of microseconds to pause

void **esp_rom_install_channel_putc**(int channel, void (*putc)(char c))

esp_rom_printf can print message to different channels simultaneously. This function can help install the low level putc function for esp_rom_printf.

Parameters

- **channel** -- Channel number (startting from 1)
- **putc** -- Function pointer to the putc implementation. Set NULL can disconnect esp_rom_printf with putc.

void **esp_rom_install_uart_printf**(void)

Install UART1 as the default console channel, equivalent to `esp_rom_install_channel_putc(1, esp_rom_output_putc)`

soc_reset_reason_t **esp_rom_get_reset_reason**(int cpu_no)

Get reset reason of CPU.

Parameters **cpu_no** -- CPU number
Returns Reset reason code (see in soc/reset_reasons.h)

void **esp_rom_route_intr_matrix**(int cpu_core, uint32_t periph_intr_id, uint32_t cpu_intr_num)

Route peripheral interrupt sources to CPU's interrupt port by matrix.

Usually there're 4 steps to use an interrupt:

a. Route peripheral interrupt source to CPU. e.g. esp_rom_route_intr_matrix(0, ETS_WIFI_MAC_INTR_SOURCE, ETS_WMAC_INUM)
b. Set interrupt handler for CPU
c. Enable CPU interrupt
d. Enable peripheral interrupt

Parameters

- **cpu_core** -- The CPU number, which the peripheral interrupt will inform to
- **periph_intr_id** -- The peripheral interrupt source number
- **cpu_intr_num** -- The CPU (external) interrupt number. On targets that use CLIC as their interrupt controller, this number represents the external interrupt number. For example, passing `cpu_intr_num = i` to this function would in fact bind peripheral source to CPU interrupt `CLIC_EXT_INTR_NUM_OFFSET + i`.

uint32_t **esp_rom_get_cpu_ticks_per_us**(void)

Get the real CPU ticks per us.

Returns CPU ticks per us

void **esp_rom_set_cpu_ticks_per_us**(uint32_t ticks_per_us)

Set the real CPU tick rate.

Note: Call this function when CPU frequency is changed, otherwise the `esp_rom_delay_us` can be inaccurate.

Parameters **ticks_per_us** -- CPU ticks per us

2.10.20 Inter-Processor Call (IPC)

Note: IPC stands for an **"Inter-Processor Call"** and **NOT** "Inter-Process Communication" as found on other operating systems.

Overview

Due to the dual core nature of the ESP32-S3, there are some scenarios where a certain callback must be executed from a particular core such as:

- When allocating an ISR to an interrupt source of a particular core (applies to freeing a particular core's interrupt source as well)
- On particular chips (such as the ESP32), accessing memory that is exclusive to a particular core (such as RTC Fast Memory)
- Reading the registers/state of another core

The IPC (Inter-Processor Call) feature allows a particular core (the calling core) to trigger the execution of a callback function on another core (the target core). The IPC feature allows execution of a callback function on the target core in either a task context, or an interrupt context. Depending on the context that the callback function is executed in, different restrictions apply to the implementation of the callback function.

IPC in Task Context

The IPC feature implements callback execution in a task context by creating an IPC task for each core during application startup. When the calling core needs to execute a callback on the target core, the callback will execute in the context of the target core's IPC task.

When using IPCs in a task context, users need to consider the following:

- IPC callbacks should ideally be simple and short. An IPC callback **must never block or yield**.
- The IPC tasks are created at the highest possible priority (i.e., `configMAX_PRIORITIES - 1`).
  - If *CONFIG_ESP_IPC_USES_CALLERS_PRIORITY* is enabled, the target core's IPC task will be lowered to the current priority of the target core before executing the callback.
  - If *CONFIG_ESP_IPC_USES_CALLERS_PRIORITY* is disabled, the target core will always execute the callback at the highest possible priority.
- Depending on the complexity of the callback, users may need to configure the stack size of the IPC task via *CONFIG_ESP_IPC_TASK_STACK_SIZE*.
- The IPC feature is internally protected by a mutex. Therefore, simultaneous IPC calls from two or more calling core's are serialized on a first come first serve basis.

API Usage Task Context IPC callbacks have the following restrictions:

- The callback must be of the *esp_ipc_func_t* type.
- The callback **must never block or yield** as this will result in the target core's IPC task blocking or yielding.

- The callback must avoid changing any aspect of the IPC task's state, e.g., by calling `vTaskPrioritySet(NULL, x)`.

The IPC feature offers the API listed below to execute a callback in a task context on a target core. The API allows the calling core to block until the callback's execution has completed, or return immediately once the callback's execution has started.

- *`esp_ipc_call()`* triggers an IPC call on the target core. This function will block until the target core's IPC task **begins** execution of the callback.
- *`esp_ipc_call_blocking()`* triggers an IPC on the target core. This function will block until the target core's IPC task **completes** execution of the callback.

IPC in Interrupt Context

In some cases, we need to quickly obtain the state of another core such as in a core dump, GDB stub, various unit tests, and hardware errata workarounds. The IPC ISR feature implements callback execution from a High Priority Interrupt context by reserving a High Priority Interrupt on each core for IPC usage. When a calling core needs to execute a callback on the target core, the callback will execute in the context of the High Priority Interrupt of the target core.

For such scenarios, the IPC ISR feature supports execution of callbacks in a *High Priority Interrupt* context.

When using IPCs in High Priority Interrupt context, users need to consider the following:

- Since the callback is executed in a High Priority Interrupt context, the callback must be written entirely in assembly. See the API Usage below for more details regarding writing assembly callbacks.
- The priority of the reserved High Priority Interrupt is dependent on the *CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL* option.

When the callback executes, users need to consider the following:

- The calling core will disable interrupts of priority level 3 and lower.
- Although the priority of the reserved interrupt depends on *CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL*, during the execution of IPC ISR callback, the target core will disable interrupts of priority level 5 and lower regardless of what *CONFIG_ESP_SYSTEM_CHECK_INT_LEVEL* is set to.

API Usage High Priority Interrupt IPC callbacks have the following restrictions:

- The callback must be of type *`esp_ipc_isr_func_t`* but implemented entirely in assembly.
- **The callback is invoked via the `CALLX0` instruction with register windowing disabled, thus the callback:**
  - must not call any register window related instructions, e.g., `entry` and `retw`.
  - must not call other C functions as register windowing is disabled.
- The callback should be placed in IRAM at a 4-byte aligned address.
- **On invocation of, or after returning from the callback, the registers `a2, a3, a4` are saved/restored automatically, th**
  - `a2` contains the `void *arg` of the callback.
  - `a3/a4` are free to use as scratch registers.

The IPC feature offers the API listed below to execute a callback in a High Priority Interrupt context:

- *`esp_ipc_isr_call()`* triggers an IPC call on the target core. This function will busy-wait until the target core **begins** execution of the callback.
- *`esp_ipc_isr_call_blocking()`* triggers an IPC call on the target core. This function will busy-wait until the target core **completes** execution of the callback.

The following code-blocks demonstrates a High Priority Interrupt IPC callback written in assembly that simply reads the target core's cycle count:

```
/* esp_test_ipc_isr_get_cycle_count_other_cpu(void *arg) */
// this function reads CCOUNT of the target core and stores it in arg.
// use only a2, a3 and a4 regs here.
.section    .iram1, "ax"
.align      4
.global     esp_test_ipc_isr_get_cycle_count_other_cpu
.type       esp_test_ipc_isr_get_cycle_count_other_cpu, @function
// Args:
// a2 - void* arg
esp_test_ipc_isr_get_cycle_count_other_cpu:
rsr.ccount a3
s32i    a3, a2, 0
ret
```

```
unit32_t cycle_count;
esp_ipc_isr_call_blocking(esp_test_ipc_isr_get_cycle_count_other_cpu, (void␣
↪*)cycle_count);
```

Note: The number of scratch registers available for use is sufficient for most simple use cases. But if your callback requires more scratch registers, `void *arg` can point to a buffer that is used as a register save area. The callback can then save and restore more registers. See the system/ipc/ipc_isr.

Note: For more examples of High Priority Interrupt IPC callbacks, you can refer to components/esp_system/port/arch/xtensa/esp_ipc_isr_routines.S and components/esp_system/test_apps/esp_system_unity_tests/main/port/arch/xtensa/test_ipc_isr.S.

See examples/system/ipc/ipc_isr/xtensa/main/main.c for an example of its use.

The High Priority Interrupt IPC API also provides the following convenience functions that can stall/resume the target core. These APIs utilize the High Priority Interrupt IPC, but supply their own internal callbacks:

- *esp_ipc_isr_stall_other_cpu()* stalls the target core. The calling core disables interrupts of level 3 and lower while the target core will busy-wait with interrupts of level 5 and lower disabled. The target core will busy-wait until *esp_ipc_isr_release_other_cpu()* is called.
- *esp_ipc_isr_release_other_cpu()* resumes the target core.

API Reference

Header File

- components/esp_system/include/esp_ipc.h
- This header file can be included with:

```
#include "esp_ipc.h"
```

Functions

esp_err_t **esp_ipc_call** (uint32_t cpu_id, *esp_ipc_func_t* func, void *arg)

Execute a callback on a given CPU.

Execute a given callback on a particular CPU. The callback must be of type "esp_ipc_func_t" and will be invoked in the context of the target CPU's IPC task.

- This function will block the target CPU's IPC task has begun execution of the callback

- If another IPC call is ongoing, this function will block until the ongoing IPC call completes
- The stack size of the IPC task can be configured via the CONFIG_ESP_IPC_TASK_STACK_SIZE option

---

Note: In single-core mode, returns ESP_ERR_INVALID_ARG for cpu_id 1.

Parameters
- **`cpu_id`** -- **[in]** CPU where the given function should be executed (0 or 1)
- **`func`** -- **[in]** Pointer to a function of type void func(void* arg) to be executed
- **`arg`** -- **[in]** Arbitrary argument of type void* to be passed into the function

Returns
- ESP_ERR_INVALID_ARG if cpu_id is invalid
- ESP_ERR_INVALID_STATE if the FreeRTOS scheduler is not running
- ESP_OK otherwise

esp_err_t **`esp_ipc_call_blocking`**(uint32_t cpu_id, *esp_ipc_func_t* func, void *arg)

Execute a callback on a given CPU until and block until it completes.

This function is identical to esp_ipc_call() except that this function will block until the execution of the callback completes.

Note: In single-core mode, returns ESP_ERR_INVALID_ARG for cpu_id 1.

Parameters
- **`cpu_id`** -- **[in]** CPU where the given function should be executed (0 or 1)
- **`func`** -- **[in]** Pointer to a function of type void func(void* arg) to be executed
- **`arg`** -- **[in]** Arbitrary argument of type void* to be passed into the function

Returns
- ESP_ERR_INVALID_ARG if cpu_id is invalid
- ESP_ERR_INVALID_STATE if the FreeRTOS scheduler is not running
- ESP_OK otherwise

Type Definitions

typedef void (***`esp_ipc_func_t`**)(void *arg)

IPC Callback.

A callback of this type should be provided as an argument when calling esp_ipc_call() or esp_ipc_call_blocking().

Header File

- components/esp_system/include/esp_ipc_isr.h
- This header file can be included with:

```
#include "esp_ipc_isr.h"
```

Functions

void **`esp_ipc_isr_call`**(*esp_ipc_isr_func_t* func, void *arg)

Execute an ISR callback on the other CPU.

Execute a given callback on the other CPU in the context of a High Priority Interrupt.

- This function will busy-wait in a critical section until the other CPU has started execution of the callback

- The callback must be written:
  - in assembly for XTENSA chips (such as ESP32, ESP32S3). The function is invoked using a CALLX0 instruction and can use only a2, a3, a4 registers. See :doc:`IPC in Interrupt Context </api-reference/system/ipc>` doc for more details.
  - in C or assembly for RISCV chips (such as ESP32P4).

Note: This function is not available in single-core mode.

Parameters
- **`func`** -- **[in]** Pointer to a function of type void func(void* arg) to be executed
- **`arg`** -- **[in]** Arbitrary argument of type void* to be passed into the function

void **`esp_ipc_isr_call_blocking`**(*esp_ipc_isr_func_t* func, void *arg)

Execute an ISR callback on the other CPU and busy-wait until it completes.

This function is identical to esp_ipc_isr_call() except that this function will busy-wait until the execution of the callback completes.

Note: This function is not available in single-core mode.

Parameters
- **`func`** -- **[in]** Pointer to a function of type void func(void* arg) to be executed
- **`arg`** -- **[in]** Arbitrary argument of type void* to be passed into the function

void **`esp_ipc_isr_stall_other_cpu`**(void)

Stall the other CPU.

This function will stall the other CPU. The other CPU is stalled by busy-waiting in the context of a High Priority Interrupt. The other CPU will not be resumed until esp_ipc_isr_release_other_cpu() is called.

- This function is internally implemented using IPC ISR
- This function is used for DPORT workaround.
- If the stall feature is paused using esp_ipc_isr_stall_pause(), this function will have no effect

Note: This function is not available in single-core mode.

Note: It is the caller's responsibility to avoid deadlocking on spinlocks

void **`esp_ipc_isr_release_other_cpu`**(void)

Release the other CPU.

This function will release the other CPU that was previously stalled from calling esp_ipc_isr_stall_other_cpu()

- This function is used for DPORT workaround.
- If the stall feature is paused using esp_ipc_isr_stall_pause(), this function will have no effect

Note: This function is not available in single-core mode.

void **esp_ipc_isr_stall_pause**(void)

Puase the CPU stall feature.

This function will pause the CPU stall feature. Once paused, calls to esp_ipc_isr_stall_other_cpu() and esp_ipc_isr_release_other_cpu() will have no effect. If a IPC ISR call is already in progress, this function will busy-wait until the call completes before pausing the CPU stall feature.

void **esp_ipc_isr_stall_abort**(void)

Abort a CPU stall.

This function will abort any stalling routine of the other CPU due to a pervious call to esp_ipc_isr_stall_other_cpu(). This function aborts the stall in a non-recoverable manner, thus should only be called in case of a panic().

- This function is used in panic handling code

void **esp_ipc_isr_stall_resume**(void)

Resume the CPU stall feature.

This function will resume the CPU stall feature that was previously paused by calling esp_ipc_isr_stall_pause(). Once resumed, calls to esp_ipc_isr_stall_other_cpu() and esp_ipc_isr_release_other_cpu() will have effect again.

Macros

esp_ipc_isr_asm_call(func, arg)

Execute an ISR callback on the other CPU See esp_ipc_isr_call().

esp_ipc_isr_asm_call_blocking(func, arg)

Execute an ISR callback on the other CPU and busy-wait until it completes See esp_ipc_isr_call_blocking().

Type Definitions

typedef void (***esp_ipc_isr_func_t**)(void *arg)

IPC ISR Callback.

The callback must be written:

- in assembly for XTENSA chips (such as ESP32, ESP32S3).
- in C or assembly for RISCV chips (such as ESP32P4).

A callback of this type should be provided as an argument when calling esp_ipc_isr_call() or esp_ipc_isr_call_blocking().

2.10.21 Interrupt Allocation

Overview

The ESP32-S3 has two cores, with 32 interrupts each. Each interrupt has a fixed priority, most (but not all) interrupts are connected to the interrupt matrix.

Because there are more interrupt sources than interrupts, sometimes it makes sense to share an interrupt in multiple drivers. The *esp_intr_alloc()* abstraction exists to hide all these implementation details.

A driver can allocate an interrupt for a certain peripheral by calling *esp_intr_alloc()* (or *esp_intr_alloc_intrstatus()*). It can use the flags passed to this function to specify the type, priority, and trigger method of the interrupt to allocate. The interrupt allocation code will then find an applicable interrupt, use the interrupt matrix to hook it up to the peripheral, and install the given interrupt handler and ISR to it.

The interrupt allocator presents two different types of interrupts, namely shared interrupts and non-shared interrupts, both of which require different handling. Non-shared interrupts will allocate a separate interrupt for every *esp_intr_alloc()* call, and this interrupt is use solely for the peripheral attached to it, with only one ISR that will get called. Shared interrupts can have multiple peripherals triggering them, with multiple ISRs being called when one of the peripherals attached signals an interrupt. Thus, ISRs that are intended for shared interrupts should check the interrupt status of the peripheral they service in order to check if any action is required.

Non-shared interrupts can be either level- or edge-triggered. Shared interrupts can only be level interrupts due to the chance of missed interrupts when edge interrupts are used.

To illustrate why shard interrupts can only be level-triggered, take the scenario where peripheral A and peripheral B share the same edge-triggered interrupt. Peripheral B triggers an interrupt and sets its interrupt signal high, causing a low-to-high edge, which in turn latches the CPU's interrupt bit and triggers the ISR. The ISR executes, checks that peripheral A did not trigger an interrupt, and proceeds to handle and clear peripheral B's interrupt signal. Before the ISR returns, the CPU clears its interrupt bit latch. Thus, during the entire interrupt handling process, if peripheral A triggers an interrupt, it will be missed due the CPU clearing the interrupt bit latch.

- Allocating an external interrupt will always allocate it on the core that does the allocation.
- Freeing an external interrupt must always happen on the same core it was allocated on.
- Disabling and enabling external interrupts from another core is allowed.
- Multiple external interrupt sources can share an interrupt slot by passing `ESP_INTR_FLAG_SHARED` as a flag to *esp_intr_alloc()*.

Care should be taken when calling *esp_intr_alloc()* from a task which is not pinned to a core. During task switching, these tasks can migrate between cores. Therefore it is impossible to tell which CPU the interrupt is allocated on, which makes it difficult to free the interrupt handle and may also cause debugging difficulties. It is advised to use *xTaskCreatePinnedToCore()* with a specific CoreID argument to create tasks that allocate interrupts. In the case of internal interrupt sources, this is required.

Multicore Issues

Peripherals that can generate interrupts can be divided in two types:

- External peripherals, within the ESP32-S3 but outside the Xtensa cores themselves. Most ESP32-S3 peripherals are of this type.
- Internal peripherals, part of the Xtensa CPU cores themselves.

Interrupt handling differs slightly between these two types of peripherals.

Internal Peripheral Interrupts Each Xtensa CPU core has its own set of six internal peripherals:

- Three timer comparators
- A performance monitor
- Two software interrupts

Internal interrupt sources are defined in `esp_intr_alloc.h` as `ETS_INTERNAL_*_INTR_SOURCE`.

These peripherals can only be configured from the core they are associated with. When generating an interrupt, the interrupt they generate is hard-wired to their associated core; it is not possible to have, for example, an internal timer comparator of one core generate an interrupt on another core. That is why these sources can only be managed using a task running on that specific core. Internal interrupt sources are still allocatable using *esp_intr_alloc()* as normal, but they cannot be shared and will always have a fixed interrupt level (namely, the one associated in hardware with the peripheral).

External Peripheral Interrupts The remaining interrupt sources are from external peripherals.

IRAM-Safe Interrupt Handlers

The `ESP_INTR_FLAG_IRAM` flag registers an interrupt handler that always runs from IRAM (and reads all its data from DRAM), and therefore does not need to be disabled during flash erase and write operations.

This is useful for interrupts which need a guaranteed minimum execution latency, as flash write and erase operations can be slow (erases can take tens or hundreds of milliseconds to complete).

It can also be useful to keep an interrupt handler in IRAM if it is called very frequently, to avoid flash cache misses.

Refer to the *SPI flash API documentation* for more details.

Multiple Handlers Sharing A Source

Several handlers can be assigned to a same source, given that all handlers are allocated using the `ESP_INTR_FLAG_SHARED` flag. They will all be allocated to the interrupt, which the source is attached to, and called sequentially when the source is active. The handlers can be disabled and freed individually. The source is attached to the interrupt (enabled), if one or more handlers are enabled, otherwise detached. A handler will never be called when disabled, while **its source may still be triggered** if any one of its handler enabled.

Sources attached to non-shared interrupt do not support this feature.

By default, when `ESP_INTR_FLAG_SHARED` flag is specified, the interrupt allocator will allocate only priority level 1 interrupts. Use `ESP_INTR_FLAG_SHARED | ESP_INTR_FLAG_LOWMED` to also allow allocating shared interrupts at priority levels 2 and 3.

Though the framework supports this feature, you have to use it **very carefully**. There usually exist two ways to stop an interrupt from being triggered: **disable the source** or **mask peripheral interrupt status**. ESP-IDF only handles enabling and disabling of the source itself, leaving status and mask bits to be handled by users.

Status bits shall either be masked before the handler responsible for it is disabled, or be masked and then properly handled in another enabled interrupt.

Note: Leaving some status bits unhandled without masking them, while disabling the handlers for them, will cause the interrupt(s) to be triggered indefinitely, resulting therefore in a system crash.

Troubleshooting Interrupt Allocation

On most Espressif SoCs, CPU interrupts are a limited resource. Therefore it is possible for a program to run out of CPU interrupts, for example by initializing several peripheral drivers. Typically, this will result in the driver initialization function returning `ESP_ERR_NOT_FOUND` error code.

If this happens, you can use *`esp_intr_dump()`* function to print the list of interrupts along with their status. The output of this function typically looks like this:

```
CPU 0 interrupt status:
Int  Level  Type   Status
0     1    Level  Reserved
1     1    Level  Reserved
2     1    Level  Used: RTC_CORE
3     1    Level  Used: TG0_LACT_LEVEL
...
```

The columns of the output have the following meaning:

- `Int`: CPU interrupt input number. This is typically not used in software directly, and is provided for reference only.
- `Level`: Interrupt priority (1-7) of the CPU interrupt. This priority is fixed in hardware, and cannot be changed.
- `Type`: Interrupt type (Level or Edge) of the CPU interrupt. This type is fixed in hardware, and cannot be changed.
- **`Status`: One of the possible statuses of the interrupt:**
  - `Reserved`: The interrupt is reserved either at hardware level, or by one of the parts of ESP-IDF. It can not be allocated using *`esp_intr_alloc()`*.

  - `Used: <source>`: The interrupt is allocated and connected to a single peripheral.
  - `Shared: <source1> <source2> ...`: The interrupt is allocated and connected to multiple peripherals. See *Multiple Handlers Sharing A Source* above.
  - `Free`: The interrupt is not allocated and can be used by `esp_intr_alloc()`.
- `Free (not general-use)`: The interrupt is not allocated, but is either a high-priority interrupt (priority 4-7) or an edge-triggered interrupt. High-priority interrupts can be allocated using `esp_intr_alloc()` but requires the handlers to be written in Assembly, see *High Priority Interrupts*. Edge-triggered low- and medium-priority interrupts can also be allocated using `esp_intr_alloc()`, but are not used often since most peripheral interrupts are level-triggered.

If you have confirmed that the application is indeed running out of interrupts, a combination of the following suggestions can help resolve the issue:

- On multi-core targets, try initializing some of the peripheral drivers from a task pinned to the second core. Interrupts are typically allocated on the same core where the peripheral driver initialization function runs. Therefore by running the initialization function on the second core, more interrupt inputs can be used.
- Determine the interrupts which can tolerate higher latency, and allocate them using `ESP_INTR_FLAG_SHARED` flag (optionally ORed with `ESP_INTR_FLAG_LOWMED`). Using this flag for two or more peripherals will let them use a single interrupt input, and therefore save interrupt inputs for other peripherals. See *Multiple Handlers Sharing A Source* above.
- Some peripheral driver may default to allocating interrupts with `ESP_INTR_FLAG_LEVEL1` flag, so priority 2 and 3 interrupts do not get used by default. If `esp_intr_dump()` shows that some priority 2 or 3 interrupts are available, try changing the interrupt allocation flags when initializing the driver to `ESP_INTR_FLAG_LEVEL2` or `ESP_INTR_FLAG_LEVEL3`.
- Check if some of the peripheral drivers do not need to be used all the time, and initialize or deinitialize them on demand. This can reduce the number of simultaneously allocated interrupts.

API Reference

Header File

- components/esp_hw_support/include/esp_intr_types.h
- This header file can be included with:

```
#include "esp_intr_types.h"
```

Macros

ESP_INTR_CPU_AFFINITY_TO_CORE_ID(cpu_affinity)

Convert esp_intr_cpu_affinity_t to CPU core ID.

Type Definitions

typedef void (***intr_handler_t**)(void *arg)

Function prototype for interrupt handler function

typedef struct intr_handle_data_t ***intr_handle_t**

Handle to an interrupt handler

Enumerations

enum **esp_intr_cpu_affinity_t**

Interrupt CPU core affinity.

This type specify the CPU core that the peripheral interrupt is connected to.

Values:

enumerator **ESP_INTR_CPU_AFFINITY_AUTO**

Install the peripheral interrupt to ANY CPU core, decided by on which CPU the interrupt allocator is running.

enumerator **ESP_INTR_CPU_AFFINITY_0**

Install the peripheral interrupt to CPU core 0.

enumerator **ESP_INTR_CPU_AFFINITY_1**

Install the peripheral interrupt to CPU core 1.

Header File

- components/esp_hw_support/include/esp_intr_alloc.h
- This header file can be included with:

```
#include "esp_intr_alloc.h"
```

Functions

esp_err_t **esp_intr_mark_shared**(int intno, int cpu, bool is_in_iram)

Mark an interrupt as a shared interrupt.

This will mark a certain interrupt on the specified CPU as an interrupt that can be used to hook shared interrupt handlers to.

Parameters
- **intno** -- The number of the interrupt (0-31)
- **cpu** -- CPU on which the interrupt should be marked as shared (0 or 1)
- **is_in_iram** -- Shared interrupt is for handlers that reside in IRAM and the int can be left enabled while the flash cache is disabled.

Returns ESP_ERR_INVALID_ARG if cpu or intno is invalid ESP_OK otherwise

esp_err_t **esp_intr_reserve**(int intno, int cpu)

Reserve an interrupt to be used outside of this framework.

This will mark a certain interrupt on the specified CPU as reserved, not to be allocated for any reason.

Parameters
- **intno** -- The number of the interrupt (0-31)
- **cpu** -- CPU on which the interrupt should be marked as shared (0 or 1)

Returns ESP_ERR_INVALID_ARG if cpu or intno is invalid ESP_OK otherwise

esp_err_t **esp_intr_alloc**(int source, int flags, *intr_handler_t* handler, void *arg, *intr_handle_t* *ret_handle)

Allocate an interrupt with the given parameters.

This finds an interrupt that matches the restrictions as given in the flags parameter, maps the given interrupt source to it and hooks up the given interrupt handler (with optional argument) as well. If needed, it can return a handle for the interrupt as well.

The interrupt will always be allocated on the core that runs this function.

If ESP_INTR_FLAG_IRAM flag is used, and handler address is not in IRAM or RTC_FAST_MEM, then ESP_ERR_INVALID_ARG is returned.

Parameters
- **source** -- The interrupt source. One of the ETS_*_INTR_SOURCE interrupt mux sources, as defined in soc/soc.h, or one of the internal ETS_INTERNAL_*_INTR_SOURCE sources as defined in this header.
- **flags** -- An ORred mask of the ESP_INTR_FLAG_* defines. These restrict the choice of interrupts that this routine can choose from. If this value is 0, it will default to allocating a non-shared interrupt of level 1, 2 or 3. If this is ESP_INTR_FLAG_SHARED, it will

allocate a shared interrupt of level 1. Setting ESP_INTR_FLAG_INTRDISABLED will return from this function with the interrupt disabled.
- **handler** -- The interrupt handler. Must be NULL when an interrupt of level >3 is requested, because these types of interrupts aren't C-callable.
- **arg** -- Optional argument for passed to the interrupt handler
- **ret_handle** -- Pointer to an intr_handle_t to store a handle that can later be used to request details or free the interrupt. Can be NULL if no handle is required.

Returns ESP_ERR_INVALID_ARG if the combination of arguments is invalid. ESP_ERR_NOT_FOUND No free interrupt found with the specified flags ESP_OK otherwise

esp_err_t **esp_intr_alloc_intrstatus**(int source, int flags, uint32_t intrstatusreg, uint32_t intrstatusmask, *intr_handler_t* handler, void *arg, *intr_handle_t* *ret_handle)

Allocate an interrupt with the given parameters.

This essentially does the same as esp_intr_alloc, but allows specifying a register and mask combo. For shared interrupts, the handler is only called if a read from the specified register, ANDed with the mask, returns non-zero. By passing an interrupt status register address and a fitting mask, this can be used to accelerate interrupt handling in the case a shared interrupt is triggered; by checking the interrupt statuses first, the code can decide which ISRs can be skipped

Parameters
- **source** -- The interrupt source. One of the ETS_*_INTR_SOURCE interrupt mux sources, as defined in soc/soc.h, or one of the internal ETS_INTERNAL_*_INTR_SOURCE sources as defined in this header.
- **flags** -- An ORred mask of the ESP_INTR_FLAG_* defines. These restrict the choice of interrupts that this routine can choose from. If this value is 0, it will default to allocating a non-shared interrupt of level 1, 2 or 3. If this is ESP_INTR_FLAG_SHARED, it will allocate a shared interrupt of level 1. Setting ESP_INTR_FLAG_INTRDISABLED will return from this function with the interrupt disabled.
- **intrstatusreg** -- The address of an interrupt status register
- **intrstatusmask** -- A mask. If a read of address intrstatusreg has any of the bits that are 1 in the mask set, the ISR will be called. If not, it will be skipped.
- **handler** -- The interrupt handler. Must be NULL when an interrupt of level >3 is requested, because these types of interrupts aren't C-callable.
- **arg** -- Optional argument for passed to the interrupt handler
- **ret_handle** -- Pointer to an intr_handle_t to store a handle that can later be used to request details or free the interrupt. Can be NULL if no handle is required.

Returns ESP_ERR_INVALID_ARG if the combination of arguments is invalid. ESP_ERR_NOT_FOUND No free interrupt found with the specified flags ESP_OK otherwise

esp_err_t **esp_intr_free**(*intr_handle_t* handle)

Disable and free an interrupt.

Use an interrupt handle to disable the interrupt and release the resources associated with it. If the current core is not the core that registered this interrupt, this routine will be assigned to the core that allocated this interrupt, blocking and waiting until the resource is successfully released.

Note: When the handler shares its source with other handlers, the interrupt status bits it's responsible for should be managed properly before freeing it. see `esp_intr_disable` for more details. Please do not call this function in `esp_ipc_call_blocking`.

Parameters **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus

Returns ESP_ERR_INVALID_ARG the handle is NULL ESP_FAIL failed to release this handle ESP_OK otherwise

int **esp_intr_get_cpu**(*intr_handle_t* handle)

Get CPU number an interrupt is tied to.

Parameters **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus
Returns The core number where the interrupt is allocated

int **esp_intr_get_intno**(*intr_handle_t* handle)

Get the allocated interrupt for a certain handle.

Parameters **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus
Returns The interrupt number

esp_err_t **esp_intr_disable**(*intr_handle_t* handle)

Disable the interrupt associated with the handle.

Note:

a. For local interrupts (ESP_INTERNAL_* sources), this function has to be called on the CPU the interrupt is allocated on. Other interrupts have no such restriction.
b. When several handlers sharing a same interrupt source, interrupt status bits, which are handled in the handler to be disabled, should be masked before the disabling, or handled in other enabled interrupts properly. Miss of interrupt status handling will cause infinite interrupt calls and finally system crash.

Parameters **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus
Returns ESP_ERR_INVALID_ARG if the combination of arguments is invalid. ESP_OK otherwise

esp_err_t **esp_intr_enable**(*intr_handle_t* handle)

Enable the interrupt associated with the handle.

Note: For local interrupts (ESP_INTERNAL_* sources), this function has to be called on the CPU the interrupt is allocated on. Other interrupts have no such restriction.

Parameters **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus
Returns ESP_ERR_INVALID_ARG if the combination of arguments is invalid. ESP_OK otherwise

esp_err_t **esp_intr_set_in_iram**(*intr_handle_t* handle, bool is_in_iram)

Set the "in IRAM" status of the handler.

Note: Does not work on shared interrupts.

Parameters
- **handle** -- The handle, as obtained by esp_intr_alloc or esp_intr_alloc_intrstatus
- **is_in_iram** -- Whether the handler associated with this handle resides in IRAM. Handlers residing in IRAM can be called when cache is disabled.

Returns ESP_ERR_INVALID_ARG if the combination of arguments is invalid. ESP_OK otherwise

void **esp_intr_noniram_disable**(void)

Disable interrupts that aren't specifically marked as running from IRAM.

void **esp_intr_noniram_enable**(void)

Re-enable interrupts disabled by esp_intr_noniram_disable.

void **esp_intr_enable_source**(int inum)

enable the interrupt source based on its number

Parameters **inum** -- interrupt number from 0 to 31

void **esp_intr_disable_source**(int inum)

disable the interrupt source based on its number

Parameters **inum** -- interrupt number from 0 to 31

static inline int **esp_intr_flags_to_level**(int flags)

Get the lowest interrupt level from the flags.

Parameters **flags** -- The same flags that pass to `esp_intr_alloc_intrstatus` API

static inline int **esp_intr_level_to_flags**(int level)

Get the interrupt flags from the supplied level (priority)

Parameters **level** -- The interrupt priority level

esp_err_t **esp_intr_dump**(FILE *stream)

Dump the status of allocated interrupts.

Parameters **stream** -- The stream to dump to, if NULL then stdout is used
Returns ESP_OK on success

Macros

ESP_INTR_FLAG_LEVEL1

Interrupt allocation flags.

These flags can be used to specify which interrupt qualities the code calling esp_intr_alloc* needs. Accept a Level 1 interrupt vector (lowest priority)

ESP_INTR_FLAG_LEVEL2

Accept a Level 2 interrupt vector.

ESP_INTR_FLAG_LEVEL3

Accept a Level 3 interrupt vector.

ESP_INTR_FLAG_LEVEL4

Accept a Level 4 interrupt vector.

ESP_INTR_FLAG_LEVEL5

Accept a Level 5 interrupt vector.

ESP_INTR_FLAG_LEVEL6

Accept a Level 6 interrupt vector.

ESP_INTR_FLAG_NMI

Accept a Level 7 interrupt vector (highest priority)

ESP_INTR_FLAG_SHARED

Interrupt can be shared between ISRs.

ESP_INTR_FLAG_EDGE

Edge-triggered interrupt.

ESP_INTR_FLAG_IRAM

ISR can be called if cache is disabled.

ESP_INTR_FLAG_INTRDISABLED

Return with this interrupt disabled.

ESP_INTR_FLAG_LOWMED

Low and medium prio interrupts. These can be handled in C.

ESP_INTR_FLAG_HIGH

High level interrupts. Need to be handled in assembly.

ESP_INTR_FLAG_LEVELMASK

Mask for all level flags

ETS_INTERNAL_TIMER0_INTR_SOURCE

Platform timer 0 interrupt source.

The esp_intr_alloc* functions can allocate an int for all ETS_*_INTR_SOURCE interrupt sources that are routed through the interrupt mux. Apart from these sources, each core also has some internal sources that do not pass through the interrupt mux. To allocate an interrupt for these sources, pass these pseudo-sources to the functions.

ETS_INTERNAL_TIMER1_INTR_SOURCE

Platform timer 1 interrupt source.

ETS_INTERNAL_TIMER2_INTR_SOURCE

Platform timer 2 interrupt source.

ETS_INTERNAL_SW0_INTR_SOURCE

Software int source 1.

ETS_INTERNAL_SW1_INTR_SOURCE

Software int source 2.

ETS_INTERNAL_PROFILING_INTR_SOURCE

Int source for profiling.

ETS_INTERNAL_UNUSED_INTR_SOURCE

Interrupt is not assigned to any source.

ETS_INTERNAL_INTR_SOURCE_OFF

Provides SystemView with positive IRQ IDs, otherwise scheduler events are not shown properly

ESP_INTR_ENABLE(inum)

Enable interrupt by interrupt number

ESP_INTR_DISABLE(inum)

Disable interrupt by interrupt number

2.10.22 Logging library

Overview

The logging library provides three ways for setting log verbosity:

- **At compile time**: in menuconfig, set the verbosity level using the option *CONFIG_LOG_DEFAULT_LEVEL*.
- Optionally, also in menuconfig, set the maximum verbosity level using the option *CONFIG_LOG_MAXIMUM_LEVEL*. By default, this is the same as the default level, but it can be set higher in order to compile more optional logs into the firmware.
- **At runtime**: all logs for verbosity levels lower than *CONFIG_LOG_DEFAULT_LEVEL* are enabled by default. The function `esp_log_level_set()` can be used to set a logging level on a per-module basis. Modules are identified by their tags, which are human-readable ASCII zero-terminated strings.
- **At runtime**: if *CONFIG_LOG_MASTER_LEVEL* is enabled then a `Master logging level` can be set using `esp_log_set_level_master()`. This option adds an additional logging level check for all compiled logs. Note that this will increase application size. This feature is useful if you want to compile a lot of logs that are selectable at runtime, but also want to avoid the performance hit from looking up the tags and their log level when you don't want log output.

There are the following verbosity levels:

- Error (lowest)
- Warning
- Info
- Debug
- Verbose (highest)

Note: The function `esp_log_level_set()` cannot set logging levels higher than specified by *CONFIG_LOG_MAXIMUM_LEVEL*. To increase log level for a specific file above this maximum at compile time, use the macro *LOG_LOCAL_LEVEL* (see the details below).

How to Use Logging Library

In each C file that uses logging functionality, define the TAG variable as shown below:

```
static const char* TAG = "MyModule";
```

Then use one of logging macros to produce output, e.g:

```
ESP_LOGW(TAG, "Baud rate error %.1f%%. Requested: %d baud, actual: %d baud", error␣
↪* 100, baud_req, baud_real);
```

Several macros are available for different verbosity levels:

- `ESP_LOGE` - Error (lowest)
- `ESP_LOGW` - Warning
- `ESP_LOGI` - Info
- `ESP_LOGD` - Debug
- `ESP_LOGV` - Verbose (highest)

Additionally, there are `ESP_EARLY_LOGx` versions for each of these macros, e.g. `ESP_EARLY_LOGE`. These versions have to be used explicitly in the early startup code only, before heap allocator and syscalls have been initialized. Normal `ESP_LOGx` macros can also be used while compiling the bootloader, but they will fall back to the same implementation as `ESP_EARLY_LOGx` macros.

There are also ESP_DRAM_LOGx versions for each of these macros, e.g. *ESP_DRAM_LOGE*. These versions are used in some places where logging may occur with interrupts disabled or with flash cache inaccessible. Use of this macros should be as sparse as possible, as logging in these types of code should be avoided for performance reasons.

Note: Inside critical sections interrupts are disabled so it's only possible to use ESP_DRAM_LOGx (preferred) or ESP_EARLY_LOGx. Even though it's possible to log in these situations, it's better if your program can be structured not to require it.

To override default verbosity level at file or component scope, define the LOG_LOCAL_LEVEL macro.

At file scope, define it before including esp_log.h, e.g.:

```
#define LOG_LOCAL_LEVEL ESP_LOG_VERBOSE
#include "esp_log.h"
```

At component scope, define it in the component CMakeLists:

```
target_compile_definitions(${COMPONENT_LIB} PUBLIC "-DLOG_LOCAL_LEVEL=ESP_LOG_
↪VERBOSE")
```

To configure logging output per module at runtime, add calls to the function *esp_log_level_set()* as follows:

```
esp_log_level_set("*", ESP_LOG_ERROR);        // set all components to ERROR level
esp_log_level_set("wifi", ESP_LOG_WARN);      // enable WARN logs from WiFi stack
esp_log_level_set("dhcpc", ESP_LOG_INFO);     // enable INFO logs from DHCP client
```

Note: The "DRAM" and "EARLY" log macro variants documented above do not support per module setting of log verbosity. These macros will always log at the "default" verbosity level, which can only be changed at runtime by calling esp_log_level("*", level).

Even when logs are disabled by using a tag name, they will still require a processing time of around 10.9 microseconds per entry.

Master Logging Level To enable the Master logging level feature, the *CONFIG_LOG_MASTER_LEVEL* option must be enabled. It adds an additional level check for ESP_LOGx macros before calling *esp_log_write()*. This allows to set a higher *CONFIG_LOG_MAXIMUM_LEVEL*, but not inflict a performance hit during normal operation (only when directed). An application may set the master logging level (*esp_log_set_level_master()*) globally to enforce a maximum log level. ESP_LOGx macros above this level will be skipped immediately, rather than calling *esp_log_write()* and doing a tag lookup. It is recommended to only use this in an top-level application and not in shared components as this would override the global log level for any user using the component. By default, at startup, the Master logging level is *CONFIG_LOG_DEFAULT_LEVEL*.

Note that this feature increases application size because the additional check is added into all ESP_LOGx macros.

The snippet below shows how it works. Setting the Master logging level to ESP_LOG_NONE disables all logging globally. *esp_log_level_set()* does not currently affect logging. But after the Master logging level is released, the logs will be printed as set by *esp_log_level_set()*.

```
// Master logging level is CONFIG_LOG_DEFAULT_LEVEL at start up and = ESP_LOG_INFO
ESP_LOGI("lib_name", "Message for print");          // prints a INFO message
esp_log_level_set("lib_name", ESP_LOG_WARN);        // enables WARN logs from lib_
↪name

esp_log_set_level_master(ESP_LOG_NONE);             // disables all logs globally.␣
↪esp_log_level_set has no effect at the moment

ESP_LOGW("lib_name", "Message for print");          // no print, Master logging␣
↪level blocks it
```

(continues on next page)

(continued from previous page)

```
esp_log_level_set("lib_name", ESP_LOG_INFO);        // enable INFO logs from lib_
↪name
ESP_LOGI("lib_name", "Message for print");          // no print, Master logging␣
↪level blocks it

esp_log_set_level_master(ESP_LOG_INFO);             // enables all INFO logs␣
↪globally

ESP_LOGI("lib_name", "Message for print");          // prints a INFO message
```

Logging to Host via JTAG By default, the logging library uses the vprintf-like function to write formatted output to the dedicated UART. By calling a simple API, all log output may be routed to JTAG instead, making logging several times faster. For details, please refer to Section *Logging to Host*.

Thread Safety The log string is first written into a memory buffer and then sent to the UART for printing. Log calls are thread-safe, i.e., logs of different threads do not conflict with each other.

Application Example

The logging library is commonly used by most ESP-IDF components and examples. For demonstration of log functionality, check ESP-IDF's examples directory. The most relevant examples that deal with logging are the following:

- system/ota
- storage/sd_card
- protocols/https_request

API Reference

Header File

- components/log/include/esp_log.h
- This header file can be included with:

```
#include "esp_log.h"
```

Functions

void **esp_log_set_level_master**(*esp_log_level_t* level)

Master log level.

Optional master log level to check against for ESP_LOGx macros before calling esp_log_write. Allows one to set a higher CONFIG_LOG_MAXIMUM_LEVEL but not impose a performance hit during normal operation (only when instructed). An application may set esp_log_set_level_master(level) to globally enforce a maximum log level. ESP_LOGx macros above this level will be skipped immediately, rather than calling esp_log_write and doing a cache hit.

The tradeoff is increased application size.

Parameters **level** -- Master log level

esp_log_level_t **esp_log_get_level_master**(void)

Returns master log level.

Returns Master log level

void **esp_log_level_set**(const char *tag, *esp_log_level_t* level)

Set log level for given tag.

If logging for given component has already been enabled, changes previous setting.

Note: Note that this function can not raise log level above the level set using CONFIG_LOG_MAXIMUM_LEVEL setting in menuconfig. To raise log level above the default one for a given file, define LOG_LOCAL_LEVEL to one of the ESP_LOG_* values, before including esp_log.h in this file.

Parameters

- **tag** -- Tag of the log entries to enable. Must be a non-NULL zero terminated string. Value "*" resets log level for all tags to the given value.
- **level** -- Selects log level to enable. Only logs at this and lower verbosity levels will be shown.

esp_log_level_t **esp_log_level_get**(const char *tag)

Get log level for a given tag, can be used to avoid expensive log statements.

Parameters **tag** -- Tag of the log to query current level. Must be a non-NULL zero terminated string.

Returns The current log level for the given tag

vprintf_like_t **esp_log_set_vprintf**(*vprintf_like_t* func)

Set function used to output log entries.

By default, log output goes to UART0. This function can be used to redirect log output to some other destination, such as file or network. Returns the original log handler, which may be necessary to return output to the previous destination.

Note: Please note that function callback here must be re-entrant as it can be invoked in parallel from multiple thread context.

Parameters **func** -- new Function used for output. Must have same signature as vprintf.

Returns func old Function used for output.

uint32_t **esp_log_timestamp**(void)

Function which returns timestamp to be used in log output.

This function is used in expansion of ESP_LOGx macros. In the 2nd stage bootloader, and at early application startup stage this function uses CPU cycle counter as time source. Later when FreeRTOS scheduler start running, it switches to FreeRTOS tick count.

For now, we ignore millisecond counter overflow.

Returns timestamp, in milliseconds

char ***esp_log_system_timestamp**(void)

Function which returns system timestamp to be used in log output.

This function is used in expansion of ESP_LOGx macros to print the system time as "HH:MM:SS.sss". The system time is initialized to 0 on startup, this can be set to the correct time with an SNTP sync, or manually with standard POSIX time functions.

Currently, this will not get used in logging from binary blobs (i.e. Wi-Fi & Bluetooth libraries), these will still print the RTOS tick time.

Returns timestamp, in "HH:MM:SS.sss"

uint32_t **esp_log_early_timestamp**(void)

Function which returns timestamp to be used in log output.

This function uses HW cycle counter and does not depend on OS, so it can be safely used after application crash.

Returns timestamp, in milliseconds

void **esp_log_write**(*esp_log_level_t* level, const char *tag, const char *format, ...)

Write message into the log.

This function is not intended to be used directly. Instead, use one of ESP_LOGE, ESP_LOGW, ESP_LOGI, ESP_LOGD, ESP_LOGV macros.

This function or these macros should not be used from an interrupt.

void **esp_log_writev**(*esp_log_level_t* level, const char *tag, const char *format, va_list args)

Write message into the log, va_list variant.

This function is provided to ease integration toward other logging framework, so that esp_log can be used as a log sink.

See also:

esp_log_write()

Macros

ESP_LOG_BUFFER_HEX_LEVEL(tag, buffer, buff_len, level)

Log a buffer of hex bytes at specified level, separated into 16 bytes each line.

Parameters

- **tag** -- description tag
- **buffer** -- Pointer to the buffer array
- **buff_len** -- length of buffer in bytes
- **level** -- level of the log

ESP_LOG_BUFFER_CHAR_LEVEL(tag, buffer, buff_len, level)

Log a buffer of characters at specified level, separated into 16 bytes each line. Buffer should contain only printable characters.

Parameters

- **tag** -- description tag
- **buffer** -- Pointer to the buffer array
- **buff_len** -- length of buffer in bytes
- **level** -- level of the log

ESP_LOG_BUFFER_HEXDUMP(tag, buffer, buff_len, level)

Dump a buffer to the log at specified level.

The dump log shows just like the one below:

```
 W (195) log_example: 0x3ffb4280   45 53 50 33 32 20 69 73  20 67 72 65 61 74␣
↪2c 20  |ESP32 is great, |
 W (195) log_example: 0x3ffb4290   77 6f 72 6b 69 6e 67 20  61 6c 6f 6e 67 20␣
↪77 69  |working along wi|
 W (205) log_example: 0x3ffb42a0   74 68 20 74 68 65 20 49  44 46 2e 00       ␣
↪       |th the IDF..|
```

It is highly recommended to use terminals with over 102 text width.

Parameters

- **tag** -- description tag
- **buffer** -- Pointer to the buffer array

- **buff_len** -- length of buffer in bytes
- **level** -- level of the log

ESP_LOG_BUFFER_HEX(tag, buffer, buff_len)

Log a buffer of hex bytes at Info level.

See also:

`esp_log_buffer_hex_level`

Parameters

- **tag** -- description tag
- **buffer** -- Pointer to the buffer array
- **buff_len** -- length of buffer in bytes

ESP_LOG_BUFFER_CHAR(tag, buffer, buff_len)

Log a buffer of characters at Info level. Buffer should contain only printable characters.

See also:

`esp_log_buffer_char_level`

Parameters

- **tag** -- description tag
- **buffer** -- Pointer to the buffer array
- **buff_len** -- length of buffer in bytes

ESP_EARLY_LOGE(tag, format, ...)

macro to output logs in startup code, before heap allocator and syscalls have been initialized. Log at `ESP_LOG_ERROR` level.

See also:

`printf`,`ESP_LOGE`,`ESP_DRAM_LOGE` In the future, we want to become compatible with clang. Hence, we provide two versions of the following macros which are using variadic arguments. The first one is using the GNU extension ##__VA_ARGS__. The second one is using the C++20 feature **VA_OPT**(,). This allows users to compile their code with standard C++20 enabled instead of the GNU extension. Below C++20, we haven't found any good alternative to using ##__VA_ARGS__.

ESP_EARLY_LOGW(tag, format, ...)

macro to output logs in startup code at `ESP_LOG_WARN` level.

See also:

`ESP_EARLY_LOGE`,`ESP_LOGE`, `printf`

ESP_EARLY_LOGI(tag, format, ...)

macro to output logs in startup code at `ESP_LOG_INFO` level.

See also:

`ESP_EARLY_LOGE`,`ESP_LOGE`, `printf`

ESP_EARLY_LOGD(tag, format, ...)

macro to output logs in startup code at `ESP_LOG_DEBUG` level.

See also:

ESP_EARLY_LOGE,ESP_LOGE, printf

ESP_EARLY_LOGV (tag, format, ...)

macro to output logs in startup code at ESP_LOG_VERBOSE level.

See also:

ESP_EARLY_LOGE,ESP_LOGE, printf

_ESP_LOG_EARLY_ENABLED (log_level)

ESP_LOG_EARLY_IMPL (tag, format, log_level, log_tag_letter, ...)

ESP_LOGE (tag, format, ...)

ESP_LOGW (tag, format, ...)

ESP_LOGI (tag, format, ...)

ESP_LOGD (tag, format, ...)

ESP_LOGV (tag, format, ...)

ESP_LOG_LEVEL (level, tag, format, ...)

runtime macro to output logs at a specified level.

See also:

printf

Parameters

- **tag** -- tag of the log, which can be used to change the log level by esp_log_level_set at runtime.
- **level** -- level of the output log.
- **format** -- format of the output log. See printf
- **...** -- variables to be replaced into the log. See printf

ESP_LOG_LEVEL_LOCAL (level, tag, format, ...)

runtime macro to output logs at a specified level. Also check the level with LOG_LOCAL_LEVEL. If CONFIG_LOG_MASTER_LEVEL set, also check first against esp_log_get_level_master().

See also:

printf, ESP_LOG_LEVEL

ESP_DRAM_LOGE (tag, format, ...)

Macro to output logs when the cache is disabled. Log at ESP_LOG_ERROR level.

Similar to

Usage: ESP_DRAM_LOGE(DRAM_STR("my_tag"), "format", orESP_DRAM_LOGE(TAG, "format", ...)`, where TAG is a char* that points to a str in the DRAM.

See also:

ESP_EARLY_LOGE, the log level cannot be changed per-tag, however esp_log_level_set("*", level) will set the default level which controls these log lines also.

See also:

`esp_rom_printf`,`ESP_LOGE`

Note: Unlike normal logging macros, it's possible to use this macro when interrupts are disabled or inside an ISR.

Note: Placing log strings in DRAM reduces available DRAM, so only use when absolutely essential.

ESP_DRAM_LOGW (tag, format, ...)

macro to output logs when the cache is disabled at `ESP_LOG_WARN` level.

See also:

`ESP_DRAM_LOGW`,`ESP_LOGW`, `esp_rom_printf`

ESP_DRAM_LOGI (tag, format, ...)

macro to output logs when the cache is disabled at `ESP_LOG_INFO` level.

See also:

`ESP_DRAM_LOGI`,`ESP_LOGI`, `esp_rom_printf`

ESP_DRAM_LOGD (tag, format, ...)

macro to output logs when the cache is disabled at `ESP_LOG_DEBUG` level.

See also:

`ESP_DRAM_LOGD`,`ESP_LOGD`, `esp_rom_printf`

ESP_DRAM_LOGV (tag, format, ...)

macro to output logs when the cache is disabled at `ESP_LOG_VERBOSE` level.

See also:

`ESP_DRAM_LOGV`,`ESP_LOGV`, `esp_rom_printf`

Type Definitions

typedef int (***vprintf_like_t**)(const char*, va_list)

Enumerations

enum **esp_log_level_t**

Log level.

Values:

enumerator **ESP_LOG_NONE**

No log output

enumerator **ESP_LOG_ERROR**

Critical errors, software module can not recover on its own

enumerator **ESP_LOG_WARN**

Error conditions from which recovery measures have been taken

enumerator **ESP_LOG_INFO**

Information messages which describe normal flow of events

enumerator **ESP_LOG_DEBUG**

Extra information which is not necessary for normal use (values, pointers, sizes, etc).

enumerator **ESP_LOG_VERBOSE**

Bigger chunks of debugging information, or frequent messages which can potentially flood the output.

2.10.23 Miscellaneous System APIs

Software Reset

To perform software reset of the chip, the *esp_restart()* function is provided. When the function is called, execution of the program stops, both CPUs are reset, the application is loaded by the bootloader and starts execution again.

Additionally, the *esp_register_shutdown_handler()* function can register a routine that will be automatically called before a restart (that is triggered by *esp_restart()*) occurs. This is similar to the functionality of `atexit` POSIX function.

Reset Reason

ESP-IDF applications can be started or restarted due to a variety of reasons. To get the last reset reason, call *esp_reset_reason()* function. See description of *esp_reset_reason_t* for the list of possible reset reasons.

Heap Memory

Two heap-memory-related functions are provided:

- *esp_get_free_heap_size()* returns the current size of free heap memory.
- *esp_get_minimum_free_heap_size()* returns the minimum size of free heap memory that has ever been available (i.e., the smallest size of free heap memory in the application's lifetime).

Note that ESP-IDF supports multiple heaps with different capabilities. The functions mentioned in this section return the size of heap memory that can be allocated using the `malloc` family of functions. For further information about heap memory, see *Heap Memory Allocation*.

MAC Address

These APIs allow querying and customizing MAC addresses for different supported network interfaces (e.g., Wi-Fi, Bluetooth, Ethernet).

To fetch the MAC address for a specific network interface (e.g., Wi-Fi, Bluetooth, Ethernet), call the function `esp_read_mac()`.

In ESP-IDF, the MAC addresses for the various network interfaces are calculated from a single **base MAC address**. By default, the Espressif base MAC address is used. This base MAC address is pre-programmed into the ESP32-S3 eFuse in the factory during production.

Interface	MAC Address (4 universally administered, default)	MAC Address (2 universally administered)
Wi-Fi Station	base_mac	base_mac
Wi-Fi SoftAP	base_mac, +1 to the last octet	*Local MAC* (derived from Wi-Fi Station MAC)
Bluetooth	base_mac, +2 to the last octet	base_mac, +1 to the last octet
Ethernet	base_mac, +3 to the last octet	*Local MAC* (derived from Bluetooth MAC)

Note: The *configuration* configures the number of universally administered MAC addresses that are provided by Espressif.

Note: Although ESP32-S3 has no integrated Ethernet MAC, it is still possible to calculate an Ethernet MAC address. However, this MAC address can only be used with an external ethernet interface such as an SPI-Ethernet device. See *Ethernet*.

Custom Interface MAC Sometimes you may need to define custom MAC addresses that are not generated from the base MAC address. To set a custom interface MAC address, use the `esp_iface_mac_addr_set()` function. This function allows you to overwrite the MAC addresses of interfaces set (or not yet set) by the base MAC address. Once a MAC address has been set for a particular interface, it will not be affected when the base MAC address is changed.

Custom Base MAC The default base MAC is pre-programmed by Espressif in eFuse BLK1. To set a custom base MAC instead, call the function `esp_iface_mac_addr_set()` with the `ESP_MAC_BASE` argument (or `esp_base_mac_addr_set()`) before initializing any network interfaces or calling the `esp_read_mac()` function. The custom MAC address can be stored in any supported storage device (e.g., flash, NVS).

The custom base MAC addresses should be allocated such that derived MAC addresses will not overlap. Based on the table above, users can configure the option *CONFIG_ESP32S3_UNIVERSAL_MAC_ADDRESSES* to set the number of valid universal MAC addresses that can be derived from the custom base MAC.

Note: It is also possible to call the function `esp_netif_set_mac()` to set the specific MAC used by a network interface after network initialization. But it is recommended to use the base MAC approach documented here to avoid the possibility of the original MAC address briefly appearing on the network before being changed.

Custom MAC Address in eFuse When reading custom MAC addresses from eFuse, ESP-IDF provides a helper function `esp_efuse_mac_get_custom()`. Users can also use `esp_read_mac()` with the `ESP_MAC_EFUSE_CUSTOM` argument. This loads the MAC address from eFuse BLK3. The `esp_efuse_mac_get_custom()` function assumes that the custom base MAC address is stored in the following format:

Field	# of bits	Range of bits
MAC address	48	200:248

Note: The eFuse BLK3 uses RS-coding during burning, which means that all eFuse fields in this block must be burnt at the same time.

Once custom eFuse MAC address has been obtained (using *esp_efuse_mac_get_custom()* or *esp_read_mac()*), you need to set it as the base MAC address. There are two ways to do it:

1. Use an old API: call *esp_base_mac_addr_set()*.
2. Use a new API: call *esp_iface_mac_addr_set()* with the ESP_MAC_BASE argument.

Local Versus Universal MAC Addresses ESP32-S3 comes pre-programmed with enough valid Espressif universally administered MAC addresses for all internal interfaces. The table above shows how to calculate and derive the MAC address for a specific interface according to the base MAC address.

When using a custom MAC address scheme, it is possible that not all interfaces can be assigned with a universally administered MAC address. In these cases, a locally administered MAC address is assigned. Note that these addresses are intended for use on a single local network only.

See this article for the definition of locally and universally administered MAC addresses.

Function *esp_derive_local_mac()* is called internally to derive a local MAC address from a universal MAC address. The process is as follows:

1. The U/L bit (bit value 0x2) is set in the first octet of the universal MAC address, creating a local MAC address.
2. If this bit is already set in the supplied universal MAC address (i.e., the supplied "universal" MAC address was in fact already a local MAC address), then the first octet of the local MAC address is XORed with 0x4.

Chip Version

esp_chip_info() function fills *esp_chip_info_t* structure with information about the chip. This includes the chip revision, number of CPU cores, and a bit mask of features enabled in the chip.

SDK Version

esp_get_idf_version() returns a string describing the ESP-IDF version which is used to compile the application. This is the same value as the one available through IDF_VER variable of the build system. The version string generally has the format of `git describe` output.

To get the version at build time, additional version macros are provided. They can be used to enable or disable parts of the program depending on the ESP-IDF version.

- *ESP_IDF_VERSION_MAJOR*, *ESP_IDF_VERSION_MINOR*, *ESP_IDF_VERSION_PATCH* are defined to integers representing major, minor, and patch version.
- *ESP_IDF_VERSION_VAL* and *ESP_IDF_VERSION* can be used when implementing version checks:

```
#include "esp_idf_version.h"

#if ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(4, 0, 0)
    // enable functionality present in ESP-IDF v4.0
#endif
```

App Version

The application version is stored in *esp_app_desc_t* structure. It is located in DROM sector and has a fixed offset from the beginning of the binary file. The structure is located after *esp_image_header_t* and *esp_image_segment_header_t* structures. The type of the field version is string and it has a maximum length of 32 chars.

To set the version in your project manually, you need to set the `PROJECT_VER` variable in the `CMakeLists.txt` of your project. In application `CMakeLists.txt`, put `set(PROJECT_VER "0.1.0.1")` before including `project.cmake`.

If the *CONFIG_APP_PROJECT_VER_FROM_CONFIG* option is set, the value of *CONFIG_APP_PROJECT_VER* will be used. Otherwise, if the `PROJECT_VER` variable is not set in the project, it will be retrieved either from the `$(PROJECT_PATH)/version.txt` file (if present) or using git command `git describe`. If neither is available, `PROJECT_VER` will be set to "1". Application can make use of this by calling *esp_app_get_description()* or *esp_ota_get_partition_description()* functions.

API Reference

Header File

- components/esp_system/include/esp_system.h
- This header file can be included with:

```
#include "esp_system.h"
```

Functions

esp_err_t **esp_register_shutdown_handler**(*shutdown_handler_t* handle)

Register shutdown handler.

This function allows you to register a handler that gets invoked before the application is restarted using esp_restart function.

Parameters **handle** -- function to execute on restart

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if the handler has already been registered
- ESP_ERR_NO_MEM if no more shutdown handler slots are available

esp_err_t **esp_unregister_shutdown_handler**(*shutdown_handler_t* handle)

Unregister shutdown handler.

This function allows you to unregister a handler which was previously registered using esp_register_shutdown_handler function.

- ESP_OK on success
- ESP_ERR_INVALID_STATE if the given handler hasn't been registered before

void **esp_restart**(void)

Restart PRO and APP CPUs.

This function can be called both from PRO and APP CPUs. After successful restart, CPU reset reason will be SW_CPU_RESET. Peripherals (except for Wi-Fi, BT, UART0, SPI1, and legacy timers) are not reset. This function does not return.

esp_reset_reason_t **esp_reset_reason**(void)

Get reason of last reset.

Returns See description of esp_reset_reason_t for explanation of each value.

uint32_t **esp_get_free_heap_size**(void)

Get the size of available heap.

Note: Note that the returned value may be larger than the maximum contiguous block which can be allocated.

Returns Available heap size, in bytes.

uint32_t **esp_get_free_internal_heap_size**(void)

Get the size of available internal heap.

Note: Note that the returned value may be larger than the maximum contiguous block which can be allocated.

Returns Available internal heap size, in bytes.

uint32_t **esp_get_minimum_free_heap_size**(void)

Get the minimum heap that has ever been available.

Returns Minimum free heap ever available

void **esp_system_abort**(const char *details)

Trigger a software abort.

Parameters **details** -- Details that will be displayed during panic handling.

Type Definitions

typedef void (***shutdown_handler_t**)(void)

Shutdown handler type

Enumerations

enum **esp_reset_reason_t**

Reset reasons.

Values:

enumerator **ESP_RST_UNKNOWN**

Reset reason can not be determined.

enumerator **ESP_RST_POWERON**

Reset due to power-on event.

enumerator **ESP_RST_EXT**

Reset by external pin (not applicable for ESP32)

enumerator **ESP_RST_SW**

Software reset via esp_restart.

enumerator **ESP_RST_PANIC**

Software reset due to exception/panic.

enumerator **ESP_RST_INT_WDT**

Reset (software or hardware) due to interrupt watchdog.

enumerator **ESP_RST_TASK_WDT**

Reset due to task watchdog.

enumerator **ESP_RST_WDT**

Reset due to other watchdogs.

enumerator **ESP_RST_DEEPSLEEP**

Reset after exiting deep sleep mode.

enumerator **ESP_RST_BROWNOUT**

Brownout reset (software or hardware)

enumerator **ESP_RST_SDIO**

Reset over SDIO.

enumerator **ESP_RST_USB**

Reset by USB peripheral.

enumerator **ESP_RST_JTAG**

Reset by JTAG.

enumerator **ESP_RST_EFUSE**

Reset due to efuse error.

enumerator **ESP_RST_PWR_GLITCH**

Reset due to power glitch detected.

enumerator **ESP_RST_CPU_LOCKUP**

Reset due to CPU lock up.

Header File

- components/esp_common/include/esp_idf_version.h
- This header file can be included with:

```
#include "esp_idf_version.h"
```

Functions

const char ***esp_get_idf_version**(void)

Return full IDF version string, same as 'git describe' output.

Note: If you are printing the ESP-IDF version in a log file or other information, this function provides more information than using the numerical version macros. For example, numerical version macros don't differentiate between development, pre-release and release versions, but the output of this function does.

Returns constant string from IDF_VER

Macros

ESP_IDF_VERSION_MAJOR

Major version number (X.x.x)

ESP_IDF_VERSION_MINOR

Minor version number (x.X.x)

ESP_IDF_VERSION_PATCH

Patch version number (x.x.X)

ESP_IDF_VERSION_VAL(major, minor, patch)

Macro to convert IDF version number into an integer

To be used in comparisons, such as ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(4, 0, 0)

ESP_IDF_VERSION

Current IDF version, as an integer

To be used in comparisons, such as ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(4, 0, 0)

Header File

- components/esp_hw_support/include/esp_mac.h
- This header file can be included with:

```
#include "esp_mac.h"
```

Functions

esp_err_t **esp_base_mac_addr_set**(const uint8_t *mac)

Set base MAC address with the MAC address which is stored in BLK3 of EFUSE or external storage e.g. flash and EEPROM.

Base MAC address is used to generate the MAC addresses used by network interfaces.

If using a custom base MAC address, call this API before initializing any network interfaces. Refer to the ESP-IDF Programming Guide for details about how the Base MAC is used.

Note: Base MAC must be a unicast MAC (least significant bit of first byte must be zero).

Note: If not using a valid OUI, set the "locally administered" bit (bit value 0x02 in the first byte) to avoid collisions.

Parameters **mac** -- base MAC address, length: 6 bytes. length: 6 bytes for MAC-48
Returns ESP_OK on success ESP_ERR_INVALID_ARG If mac is NULL or is not a unicast MAC

esp_err_t **esp_base_mac_addr_get**(uint8_t *mac)

Return base MAC address which is set using esp_base_mac_addr_set.

Note: If no custom Base MAC has been set, this returns the pre-programmed Espressif base MAC address.

Parameters **mac** -- base MAC address, length: 6 bytes. length: 6 bytes for MAC-48
Returns ESP_OK on success ESP_ERR_INVALID_ARG mac is NULL ESP_ERR_INVALID_MAC base MAC address has not been set

esp_err_t **esp_efuse_mac_get_custom**(uint8_t *mac)

Return base MAC address which was previously written to BLK3 of EFUSE.

Base MAC address is used to generate the MAC addresses used by the networking interfaces. This API returns the custom base MAC address which was previously written to EFUSE BLK3 in a specified format.

Writing this EFUSE allows setting of a different (non-Espressif) base MAC address. It is also possible to store a custom base MAC address elsewhere, see esp_base_mac_addr_set() for details.

Note: This function is currently only supported on ESP32.

Parameters **mac** -- base MAC address, length: 6 bytes/8 bytes. length: 6 bytes for MAC-48 8 bytes for EUI-64(used for IEEE 802.15.4, if CONFIG_SOC_IEEE802154_SUPPORTED=y)

Returns ESP_OK on success ESP_ERR_INVALID_ARG mac is NULL ESP_ERR_INVALID_MAC CUSTOM_MAC address has not been set, all zeros (for esp32-xx) ESP_ERR_INVALID_VERSION An invalid MAC version field was read from BLK3 of EFUSE (for esp32) ESP_ERR_INVALID_CRC An invalid MAC CRC was read from BLK3 of EFUSE (for esp32)

esp_err_t **esp_efuse_mac_get_default**(uint8_t *mac)

Return base MAC address which is factory-programmed by Espressif in EFUSE.

Parameters **mac** -- base MAC address, length: 6 bytes/8 bytes. length: 6 bytes for MAC-48 8 bytes for EUI-64(used for IEEE 802.15.4, if CONFIG_SOC_IEEE802154_SUPPORTED=y)

Returns ESP_OK on success ESP_ERR_INVALID_ARG mac is NULL

esp_err_t **esp_read_mac**(uint8_t *mac, *esp_mac_type_t* type)

Read base MAC address and set MAC address of the interface.

This function first get base MAC address using esp_base_mac_addr_get(). Then calculates the MAC address of the specific interface requested, refer to ESP-IDF Programming Guide for the algorithm.

The MAC address set by the esp_iface_mac_addr_set() function will not depend on the base MAC address.

Parameters

- **mac** -- base MAC address, length: 6 bytes/8 bytes. length: 6 bytes for MAC-48 8 bytes for EUI-64(used for IEEE 802.15.4, if CONFIG_SOC_IEEE802154_SUPPORTED=y)
- **type** -- Type of MAC address to return

Returns ESP_OK on success

esp_err_t **esp_derive_local_mac**(uint8_t *local_mac, const uint8_t *universal_mac)

Derive local MAC address from universal MAC address.

This function copies a universal MAC address and then sets the "locally

administered" bit (bit 0x2) in the first octet, creating a locally administered MAC address.

If the universal MAC address argument is already a locally administered MAC address, then the first octet is XORed with 0x4 in order to create a different locally administered MAC address.

Parameters

- **local_mac** -- base MAC address, length: 6 bytes. length: 6 bytes for MAC-48
- **universal_mac** -- Source universal MAC address, length: 6 bytes.

Returns ESP_OK on success

esp_err_t **esp_iface_mac_addr_set**(const uint8_t *mac, *esp_mac_type_t* type)

Set custom MAC address of the interface. This function allows you to overwrite the MAC addresses of the interfaces set by the base MAC address.

Parameters

- **mac** -- MAC address, length: 6 bytes/8 bytes. length: 6 bytes for MAC-48 8 bytes for EUI-64(used for ESP_MAC_IEEE802154 type, if CONFIG_SOC_IEEE802154_SUPPORTED=y)
- **type** -- Type of MAC address

Returns ESP_OK on success

size_t **esp_mac_addr_len_get**(*esp_mac_type_t* type)

Return the size of the MAC type in bytes.

If CONFIG_SOC_IEEE802154_SUPPORTED is set then for these types:

- ESP_MAC_IEEE802154 is 8 bytes.
- ESP_MAC_BASE, ESP_MAC_EFUSE_FACTORY and ESP_MAC_EFUSE_CUSTOM the MAC size is 6 bytes.
- ESP_MAC_EFUSE_EXT is 2 bytes. If CONFIG_SOC_IEEE802154_SUPPORTED is not set then for all types it returns 6 bytes.

Parameters **type** -- Type of MAC address
Returns 0 MAC type not found (not supported) 6 bytes for MAC-48. 8 bytes for EUI-64.

Macros

MAC2STR(a)

MACSTR

Enumerations

enum **esp_mac_type_t**

Values:

enumerator **ESP_MAC_WIFI_STA**

MAC for WiFi Station (6 bytes)

enumerator **ESP_MAC_WIFI_SOFTAP**

MAC for WiFi Soft-AP (6 bytes)

enumerator **ESP_MAC_BT**

MAC for Bluetooth (6 bytes)

enumerator **ESP_MAC_ETH**

MAC for Ethernet (6 bytes)

enumerator **ESP_MAC_IEEE802154**

if CONFIG_SOC_IEEE802154_SUPPORTED=y, MAC for IEEE802154 (8 bytes)

enumerator **ESP_MAC_BASE**

Base MAC for that used for other MAC types (6 bytes)

enumerator **ESP_MAC_EFUSE_FACTORY**

MAC_FACTORY eFuse which was burned by Espressif in production (6 bytes)

enumerator **ESP_MAC_EFUSE_CUSTOM**

MAC_CUSTOM eFuse which was can be burned by customer (6 bytes)

enumerator **ESP_MAC_EFUSE_EXT**

if CONFIG_SOC_IEEE802154_SUPPORTED=y, MAC_EXT eFuse which is used as an extender for IEEE802154 MAC (2 bytes)

Header File

- components/esp_hw_support/include/esp_chip_info.h
- This header file can be included with:

```
#include "esp_chip_info.h"
```

Functions

void **esp_chip_info**(*esp_chip_info_t* *out_info)

Fill an *esp_chip_info_t* structure with information about the chip.

Parameters **out_info** -- **[out]** structure to be filled

Structures

struct **esp_chip_info_t**

The structure represents information about the chip.

Public Members

esp_chip_model_t **model**

chip model, one of esp_chip_model_t

uint32_t **features**

bit mask of CHIP_FEATURE_x feature flags

uint16_t **revision**

chip revision number (in format MXX; where M - wafer major version, XX - wafer minor version)

uint8_t **cores**

number of CPU cores

Macros

CHIP_FEATURE_EMB_FLASH

Chip has embedded flash memory.

CHIP_FEATURE_WIFI_BGN

Chip has 2.4GHz WiFi.

CHIP_FEATURE_BLE

Chip has Bluetooth LE.

CHIP_FEATURE_BT

Chip has Bluetooth Classic.

CHIP_FEATURE_IEEE802154

Chip has IEEE 802.15.4.

CHIP_FEATURE_EMB_PSRAM

Chip has embedded psram.

Enumerations

enum **esp_chip_model_t**

Chip models.

Values:

enumerator **CHIP_ESP32**

ESP32.

enumerator **CHIP_ESP32S2**

ESP32-S2.

enumerator **CHIP_ESP32S3**

ESP32-S3.

enumerator **CHIP_ESP32C3**

ESP32-C3.

enumerator **CHIP_ESP32C2**

ESP32-C2.

enumerator **CHIP_ESP32C6**

ESP32-C6.

enumerator **CHIP_ESP32H2**

ESP32-H2.

enumerator **CHIP_ESP32P4**

ESP32-P4.

enumerator **CHIP_ESP32C61**

ESP32-C61.

enumerator **CHIP_POSIX_LINUX**

The code is running on POSIX/Linux simulator.

Header File

- components/esp_hw_support/include/esp_cpu.h
- This header file can be included with:

```
#include "esp_cpu.h"
```

Functions

void **esp_cpu_stall**(int core_id)

Stall a CPU core.

Parameters **core_id** -- The core's ID

void **esp_cpu_unstall**(int core_id)

Resume a previously stalled CPU core.

Parameters **core_id** -- The core's ID

void **esp_cpu_reset**(int core_id)

Reset a CPU core.

Parameters **core_id** -- The core's ID

void **esp_cpu_wait_for_intr**(void)

Wait for Interrupt.

This function causes the current CPU core to execute its Wait For Interrupt (WFI or equivalent) instruction. After executing this function, the CPU core will stop execution until an interrupt occurs.

int **esp_cpu_get_core_id**(void)

Get the current core's ID.

This function will return the ID of the current CPU (i.e., the CPU that calls this function).

Returns The current core's ID [0..SOC_CPU_CORES_NUM - 1]

void ***esp_cpu_get_sp**(void)

Read the current stack pointer address.

Returns Stack pointer address

esp_cpu_cycle_count_t **esp_cpu_get_cycle_count**(void)

Get the current CPU core's cycle count.

Each CPU core maintains an internal counter (i.e., cycle count) that increments every CPU clock cycle.

Returns Current CPU's cycle count, 0 if not supported.

void **esp_cpu_set_cycle_count**(*esp_cpu_cycle_count_t* cycle_count)

Set the current CPU core's cycle count.

Set the given value into the internal counter that increments every CPU clock cycle.

Parameters **cycle_count** -- CPU cycle count

void ***esp_cpu_pc_to_addr**(uint32_t pc)

Convert a program counter (PC) value to address.

If the architecture does not store the true virtual address in the CPU's PC or return addresses, this function will convert the PC value to a virtual address. Otherwise, the PC is just returned

Parameters **pc** -- PC value
Returns Virtual address

void **esp_cpu_intr_get_desc**(int core_id, int intr_num, *esp_cpu_intr_desc_t* *intr_desc_ret)

Get a CPU interrupt's descriptor.

Each CPU interrupt has a descriptor describing the interrupt's capabilities and restrictions. This function gets the descriptor of a particular interrupt on a particular CPU.

Parameters

- **core_id** -- **[in]** The core's ID
- **intr_num** -- **[in]** Interrupt number
- **intr_desc_ret** -- **[out]** The interrupt's descriptor

void **esp_cpu_intr_set_ivt_addr**(const void *ivt_addr)

Set the base address of the current CPU's Interrupt Vector Table (IVT)

Parameters **ivt_addr** -- Interrupt Vector Table's base address

bool **esp_cpu_intr_has_handler**(int intr_num)

Check if a particular interrupt already has a handler function.

Check if a particular interrupt on the current CPU already has a handler function assigned.

Note: This function simply checks if the IVT of the current CPU already has a handler assigned.

Parameters **intr_num** -- Interrupt number (from 0 to 31)
Returns True if the interrupt has a handler function, false otherwise.

void **esp_cpu_intr_set_handler**(int intr_num, *esp_cpu_intr_handler_t* handler, void *handler_arg)

Set the handler function of a particular interrupt.

Assign a handler function (i.e., ISR) to a particular interrupt on the current CPU.

Note: This function simply sets the handler function (in the IVT) and does not actually enable the interrupt.

Parameters
- **intr_num** -- Interrupt number (from 0 to 31)
- **handler** -- Handler function
- **handler_arg** -- Argument passed to the handler function

void ***esp_cpu_intr_get_handler_arg**(int intr_num)

Get a handler function's argument of.

Get the argument of a previously assigned handler function on the current CPU.

Parameters **intr_num** -- Interrupt number (from 0 to 31)
Returns The the argument passed to the handler function

void **esp_cpu_intr_enable**(uint32_t intr_mask)

Enable particular interrupts on the current CPU.

Parameters **intr_mask** -- Bit mask of the interrupts to enable

void **esp_cpu_intr_disable**(uint32_t intr_mask)

Disable particular interrupts on the current CPU.

Parameters **intr_mask** -- Bit mask of the interrupts to disable

uint32_t **esp_cpu_intr_get_enabled_mask**(void)

Get the enabled interrupts on the current CPU.

Returns Bit mask of the enabled interrupts

void **esp_cpu_intr_edge_ack**(int intr_num)

Acknowledge an edge interrupt.

Parameters **intr_num** -- Interrupt number (from 0 to 31)

void **esp_cpu_configure_region_protection**(void)

Configure the CPU to disable access to invalid memory regions.

esp_err_t **esp_cpu_set_breakpoint**(int bp_num, const void *bp_addr)

Set and enable a hardware breakpoint on the current CPU.

Note: This function is meant to be called by the panic handler to set a breakpoint for an attached debugger during a panic.

Note: Overwrites previously set breakpoint with same breakpoint number.

Parameters
- **bp_num** -- Hardware breakpoint number [0..SOC_CPU_BREAKPOINTS_NUM - 1]
- **bp_addr** -- Address to set a breakpoint on

Returns ESP_OK if breakpoint is set. Failure otherwise

esp_err_t **esp_cpu_clear_breakpoint**(int bp_num)

Clear a hardware breakpoint on the current CPU.

Note: Clears a breakpoint regardless of whether it was previously set

---

Parameters **bp_num** -- Hardware breakpoint number [0..SOC_CPU_BREAKPOINTS_NUM - 1]

Returns ESP_OK if breakpoint is cleared. Failure otherwise

esp_err_t **esp_cpu_set_watchpoint**(int wp_num, const void *wp_addr, size_t size, *esp_cpu_watchpoint_trigger_t* trigger)

Set and enable a hardware watchpoint on the current CPU.

Set and enable a hardware watchpoint on the current CPU, specifying the memory range and trigger operation. Watchpoints will break/panic the CPU when the CPU accesses (according to the trigger type) on a certain memory range.

Note: Overwrites previously set watchpoint with same watchpoint number. On RISC-V chips, this API uses method0(Exact matching) and method1(NAPOT matching) according to the riscv-debug-spec-0.13 specification for address matching. If the watch region size is 1byte, it uses exact matching (method 0). If the watch region size is larger than 1byte, it uses NAPOT matching (method 1). This mode requires the watching region start address to be aligned to the watching region size.

Parameters
- **wp_num** -- Hardware watchpoint number [0..SOC_CPU_WATCHPOINTS_NUM - 1]
- **wp_addr** -- Watchpoint's base address, must be naturally aligned to the size of the region
- **size** -- Size of the region to watch. Must be one of 2^n and in the range of [1 ... SOC_CPU_WATCHPOINT_MAX_REGION_SIZE]
- **trigger** -- Trigger type

Returns ESP_ERR_INVALID_ARG on invalid arg, ESP_OK otherwise

esp_err_t **esp_cpu_clear_watchpoint**(int wp_num)

Clear a hardware watchpoint on the current CPU.

Note: Clears a watchpoint regardless of whether it was previously set

---

Parameters **wp_num** -- Hardware watchpoint number [0..SOC_CPU_WATCHPOINTS_NUM - 1]

Returns ESP_OK if watchpoint was cleared. Failure otherwise.

bool **esp_cpu_dbgr_is_attached**(void)

Check if the current CPU has a debugger attached.

Returns True if debugger is attached, false otherwise

void **esp_cpu_dbgr_break**(void)

Trigger a call to the current CPU's attached debugger.

intptr_t **esp_cpu_get_call_addr**(intptr_t return_address)

Given the return address, calculate the address of the preceding call instruction This is typically used to answer the question "where was the function called from?".

Parameters **return_address** -- The value of the return address register. Typically set to the value of __builtin_return_address(0).

Returns Address of the call instruction preceding the return address.

bool **esp_cpu_compare_and_set**(volatile uint32_t *addr, uint32_t compare_value, uint32_t new_value)

Atomic compare-and-set operation.

Parameters

- **addr** -- Address of atomic variable
- **compare_value** -- Value to compare the atomic variable to
- **new_value** -- New value to set the atomic variable to

Returns Whether the atomic variable was set or not

Structures

struct **esp_cpu_intr_desc_t**

CPU interrupt descriptor.

Each particular CPU interrupt has an associated descriptor describing that particular interrupt's characteristics. Call esp_cpu_intr_get_desc() to get the descriptors of a particular interrupt.

Public Members

int **priority**

Priority of the interrupt if it has a fixed priority, (-1) if the priority is configurable.

esp_cpu_intr_type_t **type**

Whether the interrupt is an edge or level type interrupt, ESP_CPU_INTR_TYPE_NA if the type is configurable.

uint32_t **flags**

Flags indicating extra details.

Macros

ESP_CPU_INTR_DESC_FLAG_SPECIAL

Interrupt descriptor flags of *esp_cpu_intr_desc_t*.

The interrupt is a special interrupt (e.g., a CPU timer interrupt)

ESP_CPU_INTR_DESC_FLAG_RESVD

The interrupt is reserved for internal use

Type Definitions

typedef uint32_t **esp_cpu_cycle_count_t**

CPU cycle count type.

This data type represents the CPU's clock cycle count

typedef void (***esp_cpu_intr_handler_t**)(void *arg)

CPU interrupt handler type.

Enumerations

enum **esp_cpu_intr_type_t**

CPU interrupt type.

Values:

enumerator **ESP_CPU_INTR_TYPE_LEVEL**

enumerator **ESP_CPU_INTR_TYPE_EDGE**

enumerator **ESP_CPU_INTR_TYPE_NA**

enum **esp_cpu_watchpoint_trigger_t**

CPU watchpoint trigger type.

Values:

enumerator **ESP_CPU_WATCHPOINT_LOAD**

enumerator **ESP_CPU_WATCHPOINT_STORE**

enumerator **ESP_CPU_WATCHPOINT_ACCESS**

Header File

- components/esp_app_format/include/esp_app_desc.h
- This header file can be included with:

```
#include "esp_app_desc.h"
```

- This header file is a part of the API provided by the `esp_app_format` component. To declare that your component depends on `esp_app_format`, add the following to your CMakeLists.txt:

```
REQUIRES esp_app_format
```

or

```
PRIV_REQUIRES esp_app_format
```

Functions

const *esp_app_desc_t* ***esp_app_get_description** (void)

Return esp_app_desc structure. This structure includes app version.

Return description for running app.

Returns Pointer to esp_app_desc structure.

int **esp_app_get_elf_sha256** (char *dst, size_t size)

Fill the provided buffer with SHA256 of the ELF file, formatted as hexadecimal, null-terminated. If the buffer size is not sufficient to fit the entire SHA256 in hex plus a null terminator, the largest possible number of bytes will be written followed by a null.

Parameters

- **dst** -- Destination buffer
- **size** -- Size of the buffer

Returns Number of bytes written to dst (including null terminator)

char ***esp_app_get_elf_sha256_str**(void)

Return SHA256 of the ELF file which is already formatted as hexadecimal, null-terminated included. Can be used in panic handler or core dump during when cache is disabled. The length is defined by CONFIG_APP_RETRIEVE_LEN_ELF_SHA option.

Returns Hexadecimal SHA256 string

Structures

struct **esp_app_desc_t**

Description about application.

Public Members

uint32_t **magic_word**

Magic word ESP_APP_DESC_MAGIC_WORD

uint32_t **secure_version**

Secure version

uint32_t **reserv1**[2]

reserv1

char **version**[32]

Application version

char **project_name**[32]

Project name

char **time**[16]

Compile time

char **date**[16]

Compile date

char **idf_ver**[32]

Version IDF

uint8_t **app_elf_sha256**[32]

sha256 of elf file

uint32_t **reserv2**[20]

reserv2

Macros

ESP_APP_DESC_MAGIC_WORD

The magic word for the esp_app_desc structure that is in DROM.

2.10.24 Over The Air Updates (OTA)

OTA Process Overview

The OTA update mechanism allows a device to update itself based on data received while the normal firmware is running (for example, over Wi-Fi, Bluetooth or Ethernet).

OTA requires configuring the *Partition Tables* of the device with at least two OTA app slot partitions (i.e., `ota_0` and `ota_1`) and an OTA Data Partition.

The OTA operation functions write a new app firmware image to whichever OTA app slot that is currently not selected for booting. Once the image is verified, the OTA Data partition is updated to specify that this image should be used for the next boot.

OTA Data Partition

An OTA data partition (type `data`, subtype `ota`) must be included in the *Partition Tables* of any project which uses the OTA functions.

For factory boot settings, the OTA data partition should contain no data (all bytes erased to 0xFF). In this case, the ESP-IDF software bootloader will boot the factory app if it is present in the partition table. If no factory app is included in the partition table, the first available OTA slot (usually `ota_0`) is booted.

After the first OTA update, the OTA data partition is updated to specify which OTA app slot partition should be booted next.

The OTA data partition is two flash sectors (0x2000 bytes) in size, to prevent problems if there is a power failure while it is being written. Sectors are independently erased and written with matching data, and if they disagree a counter field is used to determine which sector was written more recently.

App Rollback

The main purpose of the application rollback is to keep the device working after the update. This feature allows you to roll back to the previous working application in case a new application has critical errors. When the rollback process is enabled and an OTA update provides a new version of the app, one of three things can happen:

- The application works fine, *`esp_ota_mark_app_valid_cancel_rollback()`* marks the running application with the state `ESP_OTA_IMG_VALID`. There are no restrictions on booting this application.
- The application has critical errors and further work is not possible, a rollback to the previous application is required, *`esp_ota_mark_app_invalid_rollback_and_reboot()`* marks the running application with the state `ESP_OTA_IMG_INVALID` and reset. This application will not be selected by the bootloader for boot and will boot the previously working application.
- If the *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is set, and a reset occurs without calling either function then the application is rolled back.

The following code serves detect the initial boot for an application after the OTA update. Upon the first boot, the application checks its state and performs diagnostics. If the diagnostics are successful, the application should call *`esp_ota_mark_app_valid_cancel_rollback()`* to confirm the operability of the application. If the diagnostics fail, the application should call *`esp_ota_mark_app_invalid_rollback_and_reboot()`* to roll back to the previous working application.

If the application is not able to boot or execute this code due to an abort/reboot/power loss error, the bootloader marks this application as `ESP_OTA_IMG_INVALID` in the next booting attempt and rolls back to the previous working application.

```
const esp_partition_t *running = esp_ota_get_running_partition();
esp_ota_img_states_t ota_state;
if (esp_ota_get_state_partition(running, &ota_state) == ESP_OK) {
    if (ota_state == ESP_OTA_IMG_PENDING_VERIFY) {
        // run diagnostic function ...
        bool diagnostic_is_ok = diagnostic();
        if (diagnostic_is_ok) {
            ESP_LOGI(TAG, "Diagnostics completed successfully! Continuing␣
↪execution ...");
            esp_ota_mark_app_valid_cancel_rollback();
        } else {
            ESP_LOGE(TAG, "Diagnostics failed! Start rollback to the previous␣
↪version ...");
            esp_ota_mark_app_invalid_rollback_and_reboot();
        }
    }
}
```

For the example incorporating the above code snippet, see the system/ota/native_ota_example example.

Note: The state is not written to the binary image of the application but rather to the `otadata` partition. The partition contains a `ota_seq` counter, which is a pointer to the slot (`ota_0`, `ota_1`, ...) from which the application will be selected for boot.

App OTA State States control the process of selecting a boot app:

States	Restriction of selecting a boot app in bootloader
ESP_OTA_IMG_VALID	None restriction. Will be selected.
ESP_OTA_IMG_UNDEFINED	None restriction. Will be selected.
ESP_OTA_IMG_INVALID	Will not be selected.
ESP_OTA_IMG_ABORTED	Will not be selected.
ESP_OTA_IMG_NEW	If *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is set it will be selected only once. In bootloader the state immediately changes to `ESP_OTA_IMG_PENDING_VERIFY`.
ESP_OTA_IMG_PENDING_VERIFY	If *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is set it will not be selected, and the state will change to `ESP_OTA_IMG_ABORTED`.

If *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is not enabled (by default), then the use of the following functions *esp_ota_mark_app_valid_cancel_rollback()* and *esp_ota_mark_app_invalid_rollback_and_reboot()* are optional, and `ESP_OTA_IMG_NEW` and `ESP_OTA_IMG_PENDING_VERIFY` states are not used.

An option in Kconfig *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* allows you to track the first boot of a new application. In this case, the application must confirm its operability by calling *esp_ota_mark_app_valid_cancel_rollback()* function, otherwise the application will be rolled back upon reboot. It allows you to control the operability of the application during the boot phase. Thus, a new application has only one attempt to boot successfully.

Rollback Process The description of the rollback process when *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is enabled:

- The new application is successfully downloaded and *esp_ota_set_boot_partition()* function makes this partition bootable and sets the state `ESP_OTA_IMG_NEW`. This state means that the application is new and should be monitored for its first boot.
- Reboot *esp_restart()*.

- The bootloader checks for the ESP_OTA_IMG_PENDING_VERIFY state if it is set, then it will be written to ESP_OTA_IMG_ABORTED.
- The bootloader selects a new application to boot so that the state is not set as ESP_OTA_IMG_INVALID or ESP_OTA_IMG_ABORTED.
- The bootloader checks the selected application for ESP_OTA_IMG_NEW state if it is set, then it will be written to ESP_OTA_IMG_PENDING_VERIFY. This state means that the application requires confirmation of its operability, if this does not happen and a reboot occurs, this state will be overwritten to ESP_OTA_IMG_ABORTED (see above) and this application will no longer be able to start, i.e., there will be a rollback to the previous working application.
- A new application has started and should make a self-test.
- If the self-test has completed successfully, then you must call the function *esp_ota_mark_app_valid_cancel_rollback()* because the application is awaiting confirmation of operability (ESP_OTA_IMG_PENDING_VERIFY state).
- If the self-test fails, then call *esp_ota_mark_app_invalid_rollback_and_reboot()* function to roll back to the previous working application, while the invalid application is set ESP_OTA_IMG_INVALID state.
- If the application has not been confirmed, the state remains ESP_OTA_IMG_PENDING_VERIFY, and the next boot it will be changed to ESP_OTA_IMG_ABORTED, which prevents re-boot of this application. There will be a rollback to the previous working application.

Unexpected Reset If a power loss or an unexpected crash occurs at the time of the first boot of a new application, it will roll back the application.

Recommendation: Perform the self-test procedure as quickly as possible, to prevent rollback due to power loss.

Only OTA partitions can be rolled back. Factory partition is not rolled back.

Booting Invalid/aborted Apps Booting an application which was previously set to ESP_OTA_IMG_INVALID or ESP_OTA_IMG_ABORTED is possible:

- Get the last invalid application partition *esp_ota_get_last_invalid_partition()*.
- Pass the received partition to *esp_ota_set_boot_partition()*, this will update the otadata.
- Restart *esp_restart()*. The bootloader will boot the specified application.

To determine if self-tests should be run during startup of an application, call the *esp_ota_get_state_partition()* function. If result is ESP_OTA_IMG_PENDING_VERIFY then self-testing and subsequent confirmation of operability is required.

Where the States Are Set A brief description of where the states are set:

- ESP_OTA_IMG_VALID state is set by *esp_ota_mark_app_valid_cancel_rollback()* function.
- ESP_OTA_IMG_UNDEFINED state is set by *esp_ota_set_boot_partition()* function if *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is not enabled.
- ESP_OTA_IMG_NEW state is set by *esp_ota_set_boot_partition()* function if *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is enabled.
- ESP_OTA_IMG_INVALID state is set by *esp_ota_mark_app_invalid_rollback_and_reboot()* function.
- ESP_OTA_IMG_ABORTED state is set if there was no confirmation of the application operability and occurs reboots (if *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is enabled).
- ESP_OTA_IMG_PENDING_VERIFY state is set in a bootloader if *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* option is enabled and selected app has ESP_OTA_IMG_NEW state.

Anti-rollback

Anti-rollback prevents rollback to application with security version lower than one programmed in eFuse of chip.

This function works if set *CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK* option. In the bootloader, when selecting a bootable application, an additional security version check is added which is on the chip and in the application image. The version in the bootable firmware must be greater than or equal to the version in the chip.

CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK and *CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE* options are used together. In this case, rollback is possible only on the security version which is equal or higher than the version in the chip.

A Typical Anti-rollback Scheme Is

- New firmware released with the elimination of vulnerabilities with the previous version of security.
- After the developer makes sure that this firmware is working. He can increase the security version and release a new firmware.
- Download new application.
- To make it bootable, run the function *esp_ota_set_boot_partition()*. If the security version of the new application is smaller than the version in the chip, the new application will be erased. Update to new firmware is not possible.
- Reboot.
- In the bootloader, an application with a security version greater than or equal to the version in the chip will be selected. If otadata is in the initial state, and one firmware was loaded via a serial channel, whose secure version is higher than the chip, then the secure version of efuse will be immediately updated in the bootloader.
- New application booted. Then the application should perform diagnostics of the operation and if it is completed successfully, you should call *esp_ota_mark_app_valid_cancel_rollback()* function to mark the running application with the `ESP_OTA_IMG_VALID` state and update the secure version on chip. Note that if was called *esp_ota_mark_app_invalid_rollback_and_reboot()* function a rollback may not happen as the device may not have any bootable apps. It will then return `ESP_ERR_OTA_ROLLBACK_FAILED` error and stay in the `ESP_OTA_IMG_PENDING_VERIFY` state.
- The next update of app is possible if a running app is in the `ESP_OTA_IMG_VALID` state.

Recommendation:

If you want to avoid the download/erase overhead in case of the app from the server has security version lower than the running app, you have to get `new_app_info.secure_version` from the first package of an image and compare it with the secure version of efuse. Use `esp_efuse_check_secure_version(new_app_info.secure_version)` function if it is true then continue downloading otherwise abort.

```
....
bool image_header_was_checked = false;
while (1) {
    int data_read = esp_http_client_read(client, ota_write_data, BUFFSIZE);
    ...
    if (data_read > 0) {
        if (image_header_was_checked == false) {
            esp_app_desc_t new_app_info;
            if (data_read > sizeof(esp_image_header_t) + sizeof(esp_image_segment_
↪header_t) + sizeof(esp_app_desc_t)) {
                // check current version with downloading
                if (esp_efuse_check_secure_version(new_app_info.secure_version) ==␣
↪false) {
                  ESP_LOGE(TAG, "This a new app can not be downloaded due to a␣
↪secure version is lower than stored in efuse.");
                  http_cleanup(client);
                  task_fatal_error();
                }

                image_header_was_checked = true;

                esp_ota_begin(update_partition, OTA_SIZE_UNKNOWN, &update_handle);
            }
        }
        esp_ota_write( update_handle, (const void *)ota_write_data, data_read);
```

(continues on next page)

(continued from previous page)

```
    }
}
...
```

Restrictions:

- The number of bits in the `secure_version` field is limited to 16 bits. This means that only 16 times you can do an anti-rollback. You can reduce the length of this efuse field using *CONFIG_BOOTLOADER_APP_SEC_VER_SIZE_EFUSE_FIELD* option.
- Factory and Test partitions are not supported in anti rollback scheme and hence partition table should not have partition with SubType set to `factory` or `test`.

`security_version`:

- In application image it is stored in `esp_app_desc` structure. The number is set *CONFIG_BOOTLOADER_APP_SECURE_VERSION*.

Secure OTA Updates Without Secure Boot

The verification of signed OTA updates can be performed even without enabling hardware secure boot. This can be achieved by setting *CONFIG_SECURE_SIGNED_APPS_NO_SECURE_BOOT* and *CONFIG_SECURE_SIGNED_ON_UPDATE_NO_SECURE_BOOT*

Tuning OTA Performance

- Erasing the update partition at once instead of sequential erasing (default mechanism) while write operation might help in reducing the overall time taken for firmware upgrade. To enable this, set *esp_https_ota_config_t::bulk_flash_erase* to true in *esp_https_ota_config_t* structure. If the partition to be erased is too large, task watchdog could be triggered. It is advised to increase the watchdog timeout in such cases.

```
esp_https_ota_config_t ota_config = {
    .bulk_flash_erase = true,
}
```

- Tuning the `esp_https_ota_config_t::http_config::buffer_size` can also help in improving the OTA performance.
- *esp_https_ota_config_t* has a member *esp_https_ota_config_t::buffer_caps* which can be used to specify the memory type to use when allocating memory to the OTA buffer. Configuring this value to MALLOC_CAP_INTERNAL might help in improving the OTA performance when SPIRAM is enabled.
- For optimizing network performance, please refer to **Improving Network Speed** section in the *Speed Optimization* for more details.

OTA Tool `otatool.py`

The component `app_update` provides a tool app_update/otatool.py for performing OTA partition-related operations on a target device. The following operations can be performed using the tool:

- read contents of otadata partition (read_otadata)
- erase otadata partition, effectively resetting device to factory app (erase_otadata)
- switch OTA partitions (switch_ota_partition)
- erasing OTA partition (erase_ota_partition)
- write to OTA partition (write_ota_partition)
- read contents of OTA partition (read_ota_partition)

The tool can either be imported and used from another Python script or invoked from shell script for users wanting to perform operation programmatically. This is facilitated by the tool's Python API and command-line interface, respectively.

Python API Before anything else, make sure that the `otatool` module is imported.

```
import sys
import os

idf_path = os.environ["IDF_PATH"]  # get value of IDF_PATH from environment
otatool_dir = os.path.join(idf_path, "components", "app_update")  # otatool.py␣
↪lives in $IDF_PATH/components/app_update

sys.path.append(otatool_dir)  # this enables Python to find otatool module
from otatool import *  # import all names inside otatool module
```

The starting point for using the tool's Python API to do is create a `OtatoolTarget` object:

```
# Create a parttool.py target device connected on serial port /dev/ttyUSB1
target = OtatoolTarget("/dev/ttyUSB1")
```

The created object can now be used to perform operations on the target device:

```
# Erase otadata, resetting the device to factory app
target.erase_otadata()

# Erase contents of OTA app slot 0
target.erase_ota_partition(0)

# Switch boot partition to that of app slot 1
target.switch_ota_partition(1)

# Read OTA partition 'ota_3' and save contents to a file named 'ota_3.bin'
target.read_ota_partition("ota_3", "ota_3.bin")
```

The OTA partition to operate on is specified using either the app slot number or the partition name.

More information on the Python API is available in the docstrings for the tool.

Command-line Interface The command-line interface of `otatool.py` has the following structure:

```
otatool.py [command-args] [subcommand] [subcommand-args]

- command-args - these are arguments that are needed for executing the main␣
↪command (parttool.py), mostly pertaining to the target device
- subcommand - this is the operation to be performed
- subcommand-args - these are arguments that are specific to the chosen operation
```

```
# Erase otadata, resetting the device to factory app
otatool.py --port "/dev/ttyUSB1" erase_otadata

# Erase contents of OTA app slot 0
otatool.py --port "/dev/ttyUSB1" erase_ota_partition --slot 0

# Switch boot partition to that of app slot 1
otatool.py --port "/dev/ttyUSB1" switch_ota_partition --slot 1

# Read OTA partition 'ota_3' and save contents to a file named 'ota_3.bin'
otatool.py --port "/dev/ttyUSB1" read_ota_partition --name=ota_3 --output=ota_3.bin
```

More information can be obtained by specifying `--help` as argument:

```
# Display possible subcommands and show main command argument descriptions
otatool.py --help

# Show descriptions for specific subcommand arguments
otatool.py [subcommand] --help
```

See Also

- *Partition Tables*
- *Partitions API*
- *SPI Flash API*
- *ESP HTTPS OTA*

Application Example

End-to-end example of OTA firmware update workflow: system/ota.

API Reference

Header File

- components/app_update/include/esp_ota_ops.h
- This header file can be included with:

  ```
  #include "esp_ota_ops.h"
  ```

- This header file is a part of the API provided by the `app_update` component. To declare that your component depends on `app_update`, add the following to your CMakeLists.txt:

  ```
  REQUIRES app_update
  ```

  or

  ```
  PRIV_REQUIRES app_update
  ```

Functions

const *esp_app_desc_t* ***esp_ota_get_app_description**(void)

Return esp_app_desc structure. This structure includes app version.

Return description for running app.

Note: This API is present for backward compatibility reasons. Alternative function with the same functionality is `esp_app_get_description`

Returns Pointer to esp_app_desc structure.

int **esp_ota_get_app_elf_sha256**(char *dst, size_t size)

Fill the provided buffer with SHA256 of the ELF file, formatted as hexadecimal, null-terminated. If the buffer size is not sufficient to fit the entire SHA256 in hex plus a null terminator, the largest possible number of bytes will be written followed by a null.

Note: This API is present for backward compatibility reasons. Alternative function with the same functionality is `esp_app_get_elf_sha256`

Parameters

- **dst** -- Destination buffer
- **size** -- Size of the buffer

Returns Number of bytes written to dst (including null terminator)

esp_err_t **esp_ota_begin**(const *esp_partition_t* *partition, size_t image_size, *esp_ota_handle_t* *out_handle)

Commence an OTA update writing to the specified partition.

The specified partition is erased to the specified image size.

If image size is not yet known, pass OTA_SIZE_UNKNOWN which will cause the entire partition to be erased.

On success, this function allocates memory that remains in use until esp_ota_end() is called with the returned handle.

Note: If the rollback option is enabled and the running application has the ESP_OTA_IMG_PENDING_VERIFY state then it will lead to the ESP_ERR_OTA_ROLLBACK_INVALID_STATE error. Confirm the running app before to run download a new app, use esp_ota_mark_app_valid_cancel_rollback() function for it (this should be done as early as possible when you first download a new application).

Parameters

- **partition** -- Pointer to info for partition which will receive the OTA update. Required.
- **image_size** -- Size of new OTA app image. Partition will be erased in order to receive this size of image. If 0 or OTA_SIZE_UNKNOWN, the entire partition is erased.
- **out_handle** -- On success, returns a handle which should be used for subsequent esp_ota_write() and esp_ota_end() calls.

Returns

- ESP_OK: OTA operation commenced successfully.
- ESP_ERR_INVALID_ARG: partition or out_handle arguments were NULL, or partition doesn't point to an OTA app partition.
- ESP_ERR_NO_MEM: Cannot allocate memory for OTA operation.
- ESP_ERR_OTA_PARTITION_CONFLICT: Partition holds the currently running firmware, cannot update in place.
- ESP_ERR_NOT_FOUND: Partition argument not found in partition table.
- ESP_ERR_OTA_SELECT_INFO_INVALID: The OTA data partition contains invalid data.
- ESP_ERR_INVALID_SIZE: Partition doesn't fit in configured flash size.
- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash write failed.
- ESP_ERR_OTA_ROLLBACK_INVALID_STATE: If the running app has not confirmed state. Before performing an update, the application must be valid.

esp_err_t **esp_ota_write**(*esp_ota_handle_t* handle, const void *data, size_t size)

Write OTA update data to partition.

This function can be called multiple times as data is received during the OTA operation. Data is written sequentially to the partition.

Parameters

- **handle** -- Handle obtained from esp_ota_begin
- **data** -- Data buffer to write
- **size** -- Size of data buffer in bytes.

Returns

- ESP_OK: Data was written to flash successfully, or size = 0
- ESP_ERR_INVALID_ARG: handle is invalid.
- ESP_ERR_OTA_VALIDATE_FAILED: First byte of image contains invalid app image magic byte.
- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash write failed.
- ESP_ERR_OTA_SELECT_INFO_INVALID: OTA data partition has invalid contents

esp_err_t **esp_ota_write_with_offset**(*esp_ota_handle_t* handle, const void *data, size_t size, uint32_t offset)

Write OTA update data to partition at an offset.

This function can write data in non-contiguous manner. If flash encryption is enabled, data should be 16 bytes aligned.

Note: While performing OTA, if the packets arrive out of order, esp_ota_write_with_offset() can be used to write data in non-contiguous manner. Use of esp_ota_write_with_offset() in combination with esp_ota_write() is not recommended.

Parameters
- **handle** -- Handle obtained from esp_ota_begin
- **data** -- Data buffer to write
- **size** -- Size of data buffer in bytes
- **offset** -- Offset in flash partition

Returns
- ESP_OK: Data was written to flash successfully.
- ESP_ERR_INVALID_ARG: handle is invalid.
- ESP_ERR_OTA_VALIDATE_FAILED: First byte of image contains invalid app image magic byte.
- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash write failed.
- ESP_ERR_OTA_SELECT_INFO_INVALID: OTA data partition has invalid contents

esp_err_t **esp_ota_end**(*esp_ota_handle_t* handle)

Finish OTA update and validate newly written app image.

Note: After calling esp_ota_end(), the handle is no longer valid and any memory associated with it is freed (regardless of result).

Parameters **handle** -- Handle obtained from esp_ota_begin().

Returns
- ESP_OK: Newly written OTA app image is valid.
- ESP_ERR_NOT_FOUND: OTA handle was not found.
- ESP_ERR_INVALID_ARG: Handle was never written to.
- ESP_ERR_OTA_VALIDATE_FAILED: OTA image is invalid (either not a valid app image, or - if secure boot is enabled - signature failed to verify.)
- ESP_ERR_INVALID_STATE: If flash encryption is enabled, this result indicates an internal error writing the final encrypted bytes to flash.

esp_err_t **esp_ota_abort**(*esp_ota_handle_t* handle)

Abort OTA update, free the handle and memory associated with it.

Parameters **handle** -- obtained from esp_ota_begin().

Returns
- ESP_OK: Handle and its associated memory is freed successfully.
- ESP_ERR_NOT_FOUND: OTA handle was not found.

esp_err_t **esp_ota_set_boot_partition**(const *esp_partition_t* *partition)

Configure OTA data for a new boot partition.

Note: If this function returns ESP_OK, calling esp_restart() will boot the newly configured app partition.

Parameters **partition** -- Pointer to info for partition containing app image to boot.

Returns

- ESP_OK: OTA data updated, next reboot will use specified partition.
- ESP_ERR_INVALID_ARG: partition argument was NULL or didn't point to a valid OTA partition of type "app".
- ESP_ERR_OTA_VALIDATE_FAILED: Partition contained invalid app image. Also returned if secure boot is enabled and signature validation failed.
- ESP_ERR_NOT_FOUND: OTA data partition not found.
- ESP_ERR_FLASH_OP_TIMEOUT or ESP_ERR_FLASH_OP_FAIL: Flash erase or write failed.

const *esp_partition_t* ***esp_ota_get_boot_partition**(void)

Get partition info of currently configured boot app.

If esp_ota_set_boot_partition() has been called, the partition which was set by that function will be returned.

If esp_ota_set_boot_partition() has not been called, the result is usually the same as esp_ota_get_running_partition(). The two results are not equal if the configured boot partition does not contain a valid app (meaning that the running partition will be an app that the bootloader chose via fallback).

If the OTA data partition is not present or not valid then the result is the first app partition found in the partition table. In priority order, this means: the factory app, the first OTA app slot, or the test app partition.

Note that there is no guarantee the returned partition is a valid app. Use esp_image_verify(ESP_IMAGE_VERIFY, ...) to verify if the returned partition contains a bootable image.

Returns Pointer to info for partition structure, or NULL if partition table is invalid or a flash read operation failed. Any returned pointer is valid for the lifetime of the application.

const *esp_partition_t* ***esp_ota_get_running_partition**(void)

Get partition info of currently running app.

This function is different to esp_ota_get_boot_partition() in that it ignores any change of selected boot partition caused by esp_ota_set_boot_partition(). Only the app whose code is currently running will have its partition information returned.

The partition returned by this function may also differ from esp_ota_get_boot_partition() if the configured boot partition is somehow invalid, and the bootloader fell back to a different app partition at boot.

Returns Pointer to info for partition structure, or NULL if no partition is found or flash read operation failed. Returned pointer is valid for the lifetime of the application.

const *esp_partition_t* ***esp_ota_get_next_update_partition**(const *esp_partition_t* *start_from)

Return the next OTA app partition which should be written with a new firmware.

Call this function to find an OTA app partition which can be passed to esp_ota_begin().

Finds next partition round-robin, starting from the current running partition.

Parameters **start_from** -- If set, treat this partition info as describing the current running partition. Can be NULL, in which case esp_ota_get_running_partition() is used to find the currently running partition. The result of this function is never the same as this argument.

Returns Pointer to info for partition which should be updated next. NULL result indicates invalid OTA data partition, or that no eligible OTA app slot partition was found.

esp_err_t **esp_ota_get_partition_description**(const *esp_partition_t* *partition, *esp_app_desc_t* *app_desc)

Returns esp_app_desc structure for app partition. This structure includes app version.

Returns a description for the requested app partition.

Parameters

- **partition** -- **[in]** Pointer to app partition. (only app partition)

- **app_desc** -- **[out]** Structure of info about app.

Returns

- ESP_OK Successful.
- ESP_ERR_NOT_FOUND app_desc structure is not found. Magic word is incorrect.
- ESP_ERR_NOT_SUPPORTED Partition is not application.
- ESP_ERR_INVALID_ARG Arguments is NULL or if partition's offset exceeds partition size.
- ESP_ERR_INVALID_SIZE Read would go out of bounds of the partition.
- or one of error codes from lower-level flash driver.

esp_err_t **esp_ota_get_bootloader_description**(const *esp_partition_t* *bootloader_partition, *esp_bootloader_desc_t* *desc)

Returns the description structure of the bootloader.

Parameters

- **bootloader_partition** -- **[in]** Pointer to bootloader partition. If NULL, then the current bootloader is used (the default location). offset = CONFIG_BOOTLOADER_OFFSET_IN_FLASH, size = CONFIG_PARTITION_TABLE_OFFSET - CONFIG_BOOTLOADER_OFFSET_IN_FLASH,
- **desc** -- **[out]** Structure of info about bootloader.

Returns

- ESP_OK Successful.
- ESP_ERR_NOT_FOUND Description structure is not found in the bootloader image. Magic byte is incorrect.
- ESP_ERR_INVALID_ARG Arguments is NULL.
- ESP_ERR_INVALID_SIZE Read would go out of bounds of the partition.
- or one of error codes from lower-level flash driver.

uint8_t **esp_ota_get_app_partition_count**(void)

Returns number of ota partitions provided in partition table.

Returns

- Number of OTA partitions

esp_err_t **esp_ota_mark_app_valid_cancel_rollback**(void)

This function is called to indicate that the running app is working well.

Returns

- ESP_OK: if successful.

esp_err_t **esp_ota_mark_app_invalid_rollback_and_reboot**(void)

This function is called to roll back to the previously workable app with reboot.

If rollback is successful then device will reset else API will return with error code. Checks applications on a flash drive that can be booted in case of rollback. If the flash does not have at least one app (except the running app) then rollback is not possible.

Returns

- ESP_FAIL: if not successful.
- ESP_ERR_OTA_ROLLBACK_FAILED: The rollback is not possible due to flash does not have any apps.

const *esp_partition_t* ***esp_ota_get_last_invalid_partition**(void)

Returns last partition with invalid state (ESP_OTA_IMG_INVALID or ESP_OTA_IMG_ABORTED).

Returns partition.

esp_err_t **esp_ota_get_state_partition**(const *esp_partition_t* *partition, esp_ota_img_states_t *ota_state)

Returns state for given partition.

Parameters

- **partition** -- **[in]** Pointer to partition.
- **ota_state** -- **[out]** state of partition (if this partition has a record in otadata).

Returns

- ESP_OK: Successful.
- ESP_ERR_INVALID_ARG: partition or ota_state arguments were NULL.
- ESP_ERR_NOT_SUPPORTED: partition is not ota.
- ESP_ERR_NOT_FOUND: Partition table does not have otadata or state was not found for given partition.

esp_err_t **esp_ota_erase_last_boot_app_partition**(void)

Erase previous boot app partition and corresponding otadata select for this partition.

When current app is marked to as valid then you can erase previous app partition.

Returns

- ESP_OK: Successful, otherwise ESP_ERR.

bool **esp_ota_check_rollback_is_possible**(void)

Checks applications on the slots which can be booted in case of rollback.

These applications should be valid (marked in otadata as not UNDEFINED, INVALID or ABORTED and crc is good) and be able booted, and secure_version of app >= secure_version of efuse (if anti-rollback is enabled).

Returns

- True: Returns true if the slots have at least one app (except the running app).
- False: The rollback is not possible.

esp_err_t **esp_ota_revoke_secure_boot_public_key**(*esp_ota_secure_boot_public_key_index_t* index)

Revokes the signature digest denoted by the given index. This should be called in the application only after the rollback logic otherwise the device may end up in unrecoverable state.

Relevant for Secure boot v2 on ESP32-S2, ESP32-S3, ESP32-C3, ESP32-C6, ESP32-H2 where up to 3 key digests can be stored (Key #N-1, Key #N, Key #N+1). When a key used to sign an app is invalidated, an OTA update is to be sent with an app signed with at least one of the other two keys which has not been revoked already. After successfully booting the OTA app should call this function to revoke Key #N-1.

Parameters **index** -- - The index of the signature block to be revoked

Returns

- ESP_OK: If revocation is successful.
- ESP_ERR_INVALID_ARG: If the index of the public key to be revoked is incorrect.
- ESP_FAIL: If secure boot v2 has not been enabled.

Macros

OTA_SIZE_UNKNOWN

Used for esp_ota_begin() if new image size is unknown

OTA_WITH_SEQUENTIAL_WRITES

Used for esp_ota_begin() if new image size is unknown and erase can be done in incremental manner (assuming write operation is in continuous sequence)

ESP_ERR_OTA_BASE

Base error code for ota_ops api

ESP_ERR_OTA_PARTITION_CONFLICT

Error if request was to write or erase the current running partition

ESP_ERR_OTA_SELECT_INFO_INVALID

Error if OTA data partition contains invalid content

ESP_ERR_OTA_VALIDATE_FAILED

Error if OTA app image is invalid

ESP_ERR_OTA_SMALL_SEC_VER

Error if the firmware has a secure version less than the running firmware.

ESP_ERR_OTA_ROLLBACK_FAILED

Error if flash does not have valid firmware in passive partition and hence rollback is not possible

ESP_ERR_OTA_ROLLBACK_INVALID_STATE

Error if current active firmware is still marked in pending validation state (ESP_OTA_IMG_PENDING_VERIFY), essentially first boot of firmware image post upgrade and hence firmware upgrade is not possible

Type Definitions

typedef uint32_t **esp_ota_handle_t**

Opaque handle for an application OTA update.

esp_ota_begin() returns a handle which is then used for subsequent calls to esp_ota_write() and esp_ota_end().

Enumerations

enum **esp_ota_secure_boot_public_key_index_t**

Secure Boot V2 public key indexes.

Values:

enumerator **SECURE_BOOT_PUBLIC_KEY_INDEX_0**

Points to the 0th index of the Secure Boot v2 public key

enumerator **SECURE_BOOT_PUBLIC_KEY_INDEX_1**

Points to the 1st index of the Secure Boot v2 public key

enumerator **SECURE_BOOT_PUBLIC_KEY_INDEX_2**

Points to the 2nd index of the Secure Boot v2 public key

Debugging OTA Failure

2.10.25 Performance Monitor

The Performance Monitor component provides APIs to use ESP32-S3 internal performance counters to profile functions and applications.

Application Example

An example which combines performance monitor is provided in `examples/system/perfmon` directory. This example initializes the performance monitor structure and execute them with printing the statistics.

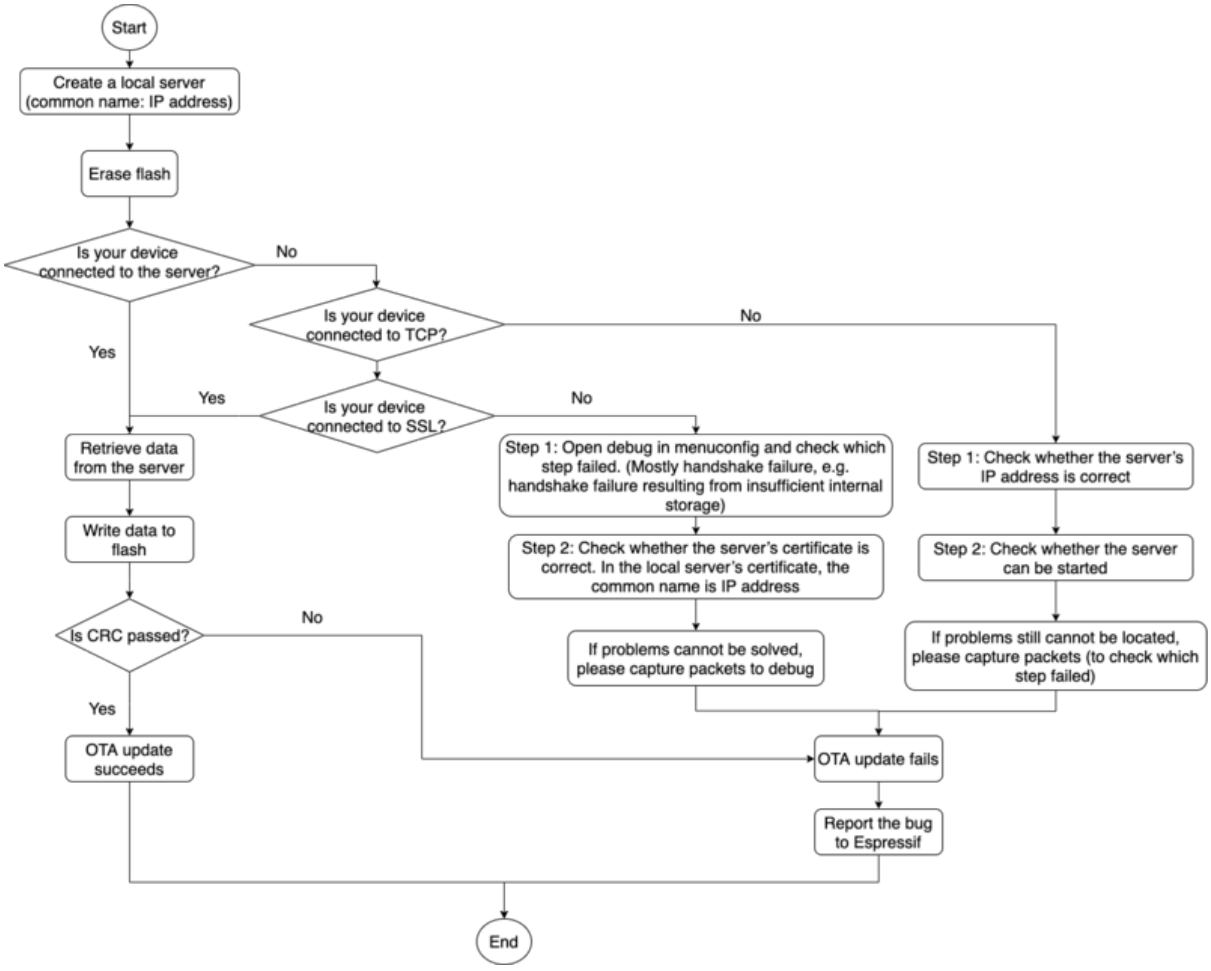


Fig. 49: How to Debug When OTA Fails (click to enlarge)

High-Level API Reference

Header Files

- perfmon/include/perfmon.h

API Reference

Header File

- components/perfmon/include/xtensa_perfmon_access.h
- This header file can be included with:

```
#include "xtensa_perfmon_access.h"
```

- This header file is a part of the API provided by the `perfmon` component. To declare that your component depends on `perfmon`, add the following to your CMakeLists.txt:

```
REQUIRES perfmon
```

or

```
PRIV_REQUIRES perfmon
```

Functions

esp_err_t **xtensa_perfmon_init**(int id, uint16_t select, uint16_t mask, int kernelcnt, int tracelevel)

Init Performance Monitoor.

Initialize performance monitor register with define values

Parameters
- **id** -- **[in]** performance counter number
- **select** -- **[in]** select value from PMCTRLx register
- **mask** -- **[in]** mask value from PMCTRLx register
- **kernelcnt** -- **[in]** kernelcnt value from PMCTRLx register
- **tracelevel** -- **[in]** tracelevel value from PMCTRLx register

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if one of the arguments is not correct

esp_err_t **xtensa_perfmon_reset**(int id)

Reset PM counter.

Reset PM counter. Writes 0 to the PMx register.

Parameters **id** -- **[in]** performance counter number

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if id out of range

void **xtensa_perfmon_start**(void)

Start PM counters.

Start all PM counters synchronously. Write 1 to the PGM register

void **xtensa_perfmon_stop**(void)

Stop PM counters.

Stop all PM counters synchronously. Write 0 to the PGM register

uint32_t **xtensa_perfmon_value**(int id)

Read PM counter.

Read value of defined PM counter.

Parameters **id** -- **[in]** performance counter number
Returns
- Performance counter value

esp_err_t **xtensa_perfmon_overflow**(int id)

Read PM overflow state.

Read overflow value of defined PM counter.

Parameters **id** -- **[in]** performance counter number
Returns
- ESP_OK if there is no overflow (overflow = 0)
- ESP_FAIL if overflow occure (overflow = 1)

void **xtensa_perfmon_dump**(void)

Dump PM values.

Dump all PM register to the console.

Header File

- components/perfmon/include/xtensa_perfmon_apis.h
- This header file can be included with:

```
#include "xtensa_perfmon_apis.h"
```

- This header file is a part of the API provided by the `perfmon` component. To declare that your component depends on `perfmon`, add the following to your CMakeLists.txt:

```
REQUIRES perfmon
```

or

```
PRIV_REQUIRES perfmon
```

Functions

esp_err_t **xtensa_perfmon_exec**(const *xtensa_perfmon_config_t* *config)

Execute PM.

Execute performance counter for dedicated function with defined parameters

Parameters **config** -- **[in]** pointer to the configuration structure
Returns
- ESP_OK if no errors
- ESP_ERR_INVALID_ARG if one of the required parameters not defined
- ESP_FAIL - counter overflow

void **xtensa_perfmon_view_cb**(void *params, uint32_t select, uint32_t mask, uint32_t value)

Dump PM results.

Callback to dump perfmon result to a FILE* stream specified in perfmon_config_t::callback_params. If callback_params is set to NULL, will print to stdout

Parameters
- **params** -- **[in]** used parameters passed from configuration (callback_params). This parameter expected as FILE* hanle, where data will be stored. If this parameter NULL, then data will be stored to the stdout.
- **select** -- **[in]** select value for current counter
- **mask** -- **[in]** mask value for current counter
- **value** -- **[in]** counter value for current counter

Structures

struct **xtensa_perfmon_config**

Performance monitor configuration structure.

Structure to configure performance counter to measure dedicated function

Public Members

int **repeat_count**

how much times function will be called before the calback will be repeated

float **max_deviation**

Difference between min and max counter number 0..1, 0 - no difference, 1 - not used

void ***call_params**

This pointer will be passed to the call_function as a parameter

void (***call_function**)(void *params)

pointer to the function that have to be called

void (***callback**)(void *params, uint32_t select, uint32_t mask, uint32_t value)

pointer to the function that will be called with result parameters

void ***callback_params**

parameter that will be passed to the callback

int **tracelevel**

trace level for all counters. In case of negative value, the filter will be ignored. If it's >=0, then the perfmon will count only when interrupt level > tracelevel. It's useful to monitor interrupts.

uint32_t **counters_size**

amount of counter in the list

const uint32_t ***select_mask**

list of the select/mask parameters

Type Definitions

typedef struct *xtensa_perfmon_config* **xtensa_perfmon_config_t**

Performance monitor configuration structure.

Structure to configure performance counter to measure dedicated function

2.10.26 Power Management

Overview

Power management algorithm included in ESP-IDF can adjust the advanced peripheral bus (APB) frequency, CPU frequency, and put the chip into Light-sleep mode to run an application at smallest possible power consumption, given the requirements of application components.

Application components can express their requirements by creating and acquiring power management locks.

For example:

- Driver for a peripheral clocked from APB can request the APB frequency to be set to 80 MHz while the peripheral is used.
- RTOS can request the CPU to run at the highest configured frequency while there are tasks ready to run.
- A peripheral driver may need interrupts to be enabled, which means it has to request disabling Light-sleep.

Since requesting higher APB or CPU frequencies or disabling Light-sleep causes higher current consumption, please keep the usage of power management locks by components to a minimum.

Configuration

Power management can be enabled at compile time, using the option *CONFIG_PM_ENABLE*.

Enabling power management features comes at the cost of increased interrupt latency. Extra latency depends on a number of factors, such as the CPU frequency, single/dual core mode, whether or not frequency switch needs to be done. Minimum extra latency is 0.2 us (when the CPU frequency is 240 MHz and frequency scaling is not enabled). Maximum extra latency is 40 us (when frequency scaling is enabled, and a switch from 40 MHz to 80 MHz is performed on interrupt entry).

Dynamic frequency scaling (DFS) and automatic Light-sleep can be enabled in an application by calling the function `esp_pm_configure()`. Its argument is a structure defining the frequency scaling settings, `esp_pm_config_t`. In this structure, three fields need to be initialized:

- `max_freq_mhz`: Maximum CPU frequency in MHz, i.e., the frequency used when the `ESP_PM_CPU_FREQ_MAX` lock is acquired. This field is usually set to the default CPU frequency.
- `min_freq_mhz`: Minimum CPU frequency in MHz, indicating the frequency used when not holding the power management lock.
  - `light_sleep_enable`: Whether the system should automatically enter Light-sleep when no locks are acquired (`true`/`false`).

Alternatively, if you enable the option *CONFIG_PM_DFS_INIT_AUTO* in menuconfig, the maximum CPU frequency will be determined by the *CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ* setting, and the minimum CPU frequency will be locked to the XTAL frequency.

Note: Automatic Light-sleep is based on FreeRTOS Tickless Idle functionality. If automatic Light-sleep is requested while the option *CONFIG_FREERTOS_USE_TICKLESS_IDLE* is not enabled in menuconfig, `esp_pm_configure()` will return the error *ESP_ERR_NOT_SUPPORTED*.

Note: In Light-sleep, peripherals are clock gated, and interrupts (from GPIOs and internal peripherals) will not be generated. A wakeup source described in the *Sleep Modes* documentation can be used to trigger wakeup from the Light-sleep state.

For example, the EXT0 and EXT1 wakeup sources can be used to wake up the chip via a GPIO.

Power Management Locks

Applications have the ability to acquire/release locks in order to control the power management algorithm. When an application acquires a lock, the power management algorithm operation is restricted in a way described below. When the lock is released, such restrictions are removed.

Power management locks have acquire/release counters. If the lock has been acquired a number of times, it needs to be released the same number of times to remove associated restrictions.

ESP32-S3 supports three types of locks described in the table below.

Lock	Description
`ESP_PM_CPU_FREQ_MAX`	Requests CPU frequency to be at the maximum value set with *`esp_pm_configure()`*. For ESP32-S3, this value can be set to 80 MHz, 160 MHz, or 240 MHz.
`ESP_PM_APB_FREQ_MAX`	Requests the APB frequency to be at the maximum supported value. For ESP32-S3, this is 80 MHz.
`ESP_PM_NO_LIGHT_SLEEP`	Disables automatic switching to Light-sleep.

ESP32-S3 Power Management Algorithm

The table below shows how CPU and APB frequencies will be switched if dynamic frequency scaling is enabled. You can specify the maximum CPU frequency with either *`esp_pm_configure()`* or *CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ*.

Max CPU Frequency Set	Lock Acquisition	CPU and APB Frequencies
240	`ESP_PM_CPU_FREQ_MAX` acquired	CPU: 240 MHz APB: 80 MHz
	`ESP_PM_APB_FREQ_MAX` acquired, `ESP_PM_CPU_FREQ_MAX` not acquired	CPU: 80 MHz APB: 80 MHz
	None	Min values for both frequencies set with *`esp_pm_configure()`*
160	`ESP_PM_CPU_FREQ_MAX` acquired	CPU: 160 MHz APB: 80 MHz
	`ESP_PM_APB_FREQ_MAX` acquired, `ESP_PM_CPU_FREQ_MAX` not acquired	CPU: 80 MHz APB: 80 MHz
	None	Min values for both frequencies set with *`esp_pm_configure()`*
80	Any of `ESP_PM_CPU_FREQ_MAX` or `ESP_PM_APB_FREQ_MAX` acquired	CPU: 80 MHz APB: 80 MHz
	None	Min values for both frequencies set with *`esp_pm_configure()`*

If none of the locks are acquired, and Light-sleep is enabled in a call to *esp_pm_configure()*, the system will go into Light-sleep mode. The duration of Light-sleep will be determined by:

- FreeRTOS tasks blocked with finite timeouts
- Timers registered with *High resolution timer* APIs

Light-sleep duration is chosen to wake up the chip before the nearest event (task being unblocked, or timer elapses).

To skip unnecessary wake-up, you can consider initializing an `esp_timer` with the `skip_unhandled_events` option as `true`. Timers with this flag will not wake up the system and it helps to reduce consumption.

Dynamic Frequency Scaling and Peripheral Drivers

When DFS is enabled, the APB frequency can be changed multiple times within a single RTOS tick. The APB frequency change does not affect the operation of some peripherals, while other peripherals may have issues. For example, Timer Group peripheral timers keeps counting, however, the speed at which they count changes proportionally to the APB frequency.

Peripheral clock sources such as `REF_TICK`, `XTAL`, `RC_FAST` (i.e., `RTC_8M`), their frequencies will not be influenced by APB frequency. And therefore, to ensure the peripheral behaves consistently during DFS, it is recommended to select one of these clocks as the peripheral clock source. For more specific guidelines, please refer to the "Power Management" section of each peripheral's "API Reference > Peripherals API" page.

Currently, the following peripheral drivers are aware of DFS and use the `ESP_PM_APB_FREQ_MAX` lock for the duration of the transaction:

- SPI master
- I2C
- I2S (If the APLL clock is used, then it will use the `ESP_PM_NO_LIGHT_SLEEP` lock)
- SDMMC

The following drivers hold the `ESP_PM_APB_FREQ_MAX` lock while the driver is enabled:

- **SPI slave**: between calls to *spi_slave_initialize()* and *spi_slave_free()*.
- **GPTimer**: between calls to *gptimer_enable()* and *gptimer_disable()*.
- **Ethernet**: between calls to *esp_eth_driver_install()* and *esp_eth_driver_uninstall()*.
- **WiFi**: between calls to *esp_wifi_start()* and *esp_wifi_stop()*. If modem sleep is enabled, the lock will be released for the periods of time when radio is disabled.
- **TWAI**: between calls to *twai_driver_install()* and *twai_driver_uninstall()* (only when the clock source is set to *TWAI_CLK_SRC_APB*).
- **Bluetooth**: between calls to *esp_bt_controller_enable()* and *esp_bt_controller_disable()*. If Bluetooth Modem-sleep is enabled, the `ESP_PM_APB_FREQ_MAX` lock will be released for the periods of time when radio is disabled. However the `ESP_PM_NO_LIGHT_SLEEP` lock will still be held.

The following peripheral drivers are not aware of DFS yet. Applications need to acquire/release locks themselves, when necessary:

- PCNT
- Sigma-delta
- The legacy timer group driver
- MCPWM

Light-sleep Peripheral Power Down

API Reference

Header File

- components/esp_pm/include/esp_pm.h
- This header file can be included with:

```
#include "esp_pm.h"
```

- This header file is a part of the API provided by the `esp_pm` component. To declare that your component depends on `esp_pm`, add the following to your CMakeLists.txt:

```
REQUIRES esp_pm
```

or

```
PRIV_REQUIRES esp_pm
```

Functions

esp_err_t **esp_pm_configure**(const void *config)

Set implementation-specific power management configuration.

Parameters **config** -- pointer to implementation-specific configuration structure (e.g. esp_pm_config_esp32)

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the configuration values are not correct
- ESP_ERR_NOT_SUPPORTED if certain combination of values is not supported, or if CONFIG_PM_ENABLE is not enabled in sdkconfig

esp_err_t **esp_pm_get_configuration**(void *config)

Get implementation-specific power management configuration.

Parameters **config** -- pointer to implementation-specific configuration structure (e.g. esp_pm_config_esp32)

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the pointer is null

esp_err_t **esp_pm_lock_create**(*esp_pm_lock_type_t* lock_type, int arg, const char *name, *esp_pm_lock_handle_t* *out_handle)

Initialize a lock handle for certain power management parameter.

When lock is created, initially it is not taken. Call esp_pm_lock_acquire to take the lock.

This function must not be called from an ISR.

Note: If the lock_type argument is not valid, it will cause an abort.

Parameters

- **lock_type** -- Power management constraint which the lock should control
- **arg** -- argument, value depends on lock_type, see esp_pm_lock_type_t
- **name** -- arbitrary string identifying the lock (e.g. "wifi" or "spi"). Used by the esp_pm_dump_locks function to list existing locks. May be set to NULL. If not set to NULL, must point to a string which is valid for the lifetime of the lock.
- **out_handle** -- **[out]** handle returned from this function. Use this handle when calling esp_pm_lock_delete, esp_pm_lock_acquire, esp_pm_lock_release. Must not be NULL.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if the lock structure can not be allocated
- ESP_ERR_INVALID_ARG if out_handle is NULL
- ESP_ERR_NOT_SUPPORTED if CONFIG_PM_ENABLE is not enabled in sdkconfig

esp_err_t **esp_pm_lock_acquire**(*esp_pm_lock_handle_t* handle)

Take a power management lock.

Once the lock is taken, power management algorithm will not switch to the mode specified in a call to esp_pm_lock_create, or any of the lower power modes (higher numeric values of 'mode').

The lock is recursive, in the sense that if esp_pm_lock_acquire is called a number of times, esp_pm_lock_release has to be called the same number of times in order to release the lock.

This function may be called from an ISR.

This function is not thread-safe w.r.t. calls to other esp_pm_lock_* functions for the same handle.

Parameters **handle** -- handle obtained from esp_pm_lock_create function

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle is invalid
- ESP_ERR_NOT_SUPPORTED if CONFIG_PM_ENABLE is not enabled in sdkconfig

esp_err_t **esp_pm_lock_release**(*esp_pm_lock_handle_t* handle)

Release the lock taken using esp_pm_lock_acquire.

Call to this functions removes power management restrictions placed when taking the lock.

Locks are recursive, so if esp_pm_lock_acquire is called a number of times, esp_pm_lock_release has to be called the same number of times in order to actually release the lock.

This function may be called from an ISR.

This function is not thread-safe w.r.t. calls to other esp_pm_lock_* functions for the same handle.

Parameters **handle** -- handle obtained from esp_pm_lock_create function

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle is invalid
- ESP_ERR_INVALID_STATE if lock is not acquired
- ESP_ERR_NOT_SUPPORTED if CONFIG_PM_ENABLE is not enabled in sdkconfig

esp_err_t **esp_pm_lock_delete**(*esp_pm_lock_handle_t* handle)

Delete a lock created using esp_pm_lock.

The lock must be released before calling this function.

This function must not be called from an ISR.

Parameters **handle** -- handle obtained from esp_pm_lock_create function

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the handle argument is NULL
- ESP_ERR_INVALID_STATE if the lock is still acquired
- ESP_ERR_NOT_SUPPORTED if CONFIG_PM_ENABLE is not enabled in sdkconfig

esp_err_t **esp_pm_dump_locks**(FILE *stream)

Dump the list of all locks to stderr

This function dumps debugging information about locks created using esp_pm_lock_create to an output stream.

This function must not be called from an ISR. If esp_pm_lock_acquire/release are called while this function is running, inconsistent results may be reported.

Parameters **stream** -- stream to print information to; use stdout or stderr to print to the console; use fmemopen/open_memstream to print to a string buffer.

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if CONFIG_PM_ENABLE is not enabled in sdkconfig

Structures

struct **esp_pm_config_t**

Power management config.

Pass a pointer to this structure as an argument to esp_pm_configure function.

Public Members

int **max_freq_mhz**

Maximum CPU frequency, in MHz

int **min_freq_mhz**

Minimum CPU frequency to use when no locks are taken, in MHz

bool **light_sleep_enable**

Enter light sleep when no locks are taken

Type Definitions

typedef *esp_pm_config_t* **esp_pm_config_esp32_t**

backward compatibility newer chips no longer require this typedef

typedef *esp_pm_config_t* **esp_pm_config_esp32s2_t**

typedef *esp_pm_config_t* **esp_pm_config_esp32s3_t**

typedef *esp_pm_config_t* **esp_pm_config_esp32c3_t**

typedef *esp_pm_config_t* **esp_pm_config_esp32c2_t**

typedef *esp_pm_config_t* **esp_pm_config_esp32c6_t**

typedef struct esp_pm_lock ***esp_pm_lock_handle_t**

Opaque handle to the power management lock.

Enumerations

enum **esp_pm_lock_type_t**

Power management constraints.

Values:

enumerator **ESP_PM_CPU_FREQ_MAX**

Require CPU frequency to be at the maximum value set via esp_pm_configure. Argument is unused and should be set to 0.

enumerator **ESP_PM_APB_FREQ_MAX**

Require APB frequency to be at the maximum value supported by the chip. Argument is unused and should be set to 0.

enumerator **ESP_PM_NO_LIGHT_SLEEP**

Prevent the system from going into light sleep. Argument is unused and should be set to 0.

enumerator **ESP_PM_LOCK_MAX**

2.10.27 POSIX Threads Support

Overview

ESP-IDF is based on FreeRTOS but offers a range of POSIX-compatible APIs that allow easy porting of third-party code. This includes support for common parts of the POSIX Threads `pthread` API.

POSIX Threads are implemented in ESP-IDF as wrappers around equivalent FreeRTOS features. The runtime memory or performance overhead of using the pthreads API is quite low, but not every feature available in either pthreads or FreeRTOS is available via the ESP-IDF pthreads support.

Pthreads can be used in ESP-IDF by including standard `pthread.h` header, which is included in the toolchain libc. An additional ESP-IDF specific header, `esp_pthread.h`, provides additional non-POSIX APIs for using some ESP-IDF features with pthreads.

C++ Standard Library implementations for `std::thread`, `std::mutex`, `std::condition_variable`, etc., are realized using pthreads (via GCC libstdc++). Therefore, restrictions mentioned here also apply to the equivalent C++ standard library functionality.

RTOS Integration

Unlike many operating systems using POSIX Threads, ESP-IDF is a real-time operating system with a real-time scheduler. This means that a thread will only stop running if a higher priority task is ready to run, the thread blocks on an OS synchronization structure like a mutex, or the thread calls any of the functions `sleep`, *`vTaskDelay()`*, or `usleep`.

Note: When calling a standard libc or C++ sleep function, such as `usleep` defined in `unistd.h`, the task will only block and yield the core if the sleep time is longer than *one FreeRTOS tick period*. If the time is shorter, the thread will busy-wait instead of yielding to another RTOS task.

By default, all POSIX Threads have the same RTOS priority, but it is possible to change this by calling a *custom API*.

Standard Features

The following standard APIs are implemented in ESP-IDF.

Refer to standard POSIX Threads documentation, or `pthread.h`, for details about the standard arguments and behaviour of each function. Differences or limitations compared to the standard APIs are noted below.

Thread APIs

- **`pthread_create()`**
  - The `attr` argument is supported for setting stack size and detach state only. Other attribute fields are ignored.
  - Unlike FreeRTOS task functions, the `start_routine` function is allowed to return. A detached type thread is automatically deleted if the function returns. The default joinable type thread will be suspended until `pthread_join()` is called on it.

- `pthread_join()`
- `pthread_detach()`
- `pthread_exit()`
- `sched_yield()`
- **`pthread_self()`**
  - An assert will fail if this function is called from a FreeRTOS task which is not a pthread.
- `pthread_equal()`

Thread Attributes

- `pthread_attr_init()`
- **`pthread_attr_destroy()`**
  - This function does not need to free any resources and instead resets the `attr` structure to defaults. The implementation is the same as `pthread_attr_init()`.
- `pthread_attr_getstacksize()` / `pthread_attr_setstacksize()`
- `pthread_attr_getdetachstate()` / `pthread_attr_setdetachstate()`

Once

- `pthread_once()`

Static initializer constant `PTHREAD_ONCE_INIT` is supported.

Note: This function can be called from tasks created using either pthread or FreeRTOS APIs.

Mutexes POSIX Mutexes are implemented as FreeRTOS Mutex Semaphores (normal type for "fast" or "error check" mutexes, and Recursive type for "recursive" mutexes). This means that they have the same priority inheritance behavior as mutexes created with `xSemaphoreCreateMutex()`.

- `pthread_mutex_init()`
- `pthread_mutex_destroy()`
- `pthread_mutex_lock()`
- `pthread_mutex_timedlock()`
- `pthread_mutex_trylock()`
- `pthread_mutex_unlock()`
- `pthread_mutexattr_init()`
- `pthread_mutexattr_destroy()`
- `pthread_mutexattr_gettype()` / `pthread_mutexattr_settype()`

Static initializer constant `PTHREAD_MUTEX_INITIALIZER` is supported, but the non-standard static initializer constants for other mutex types are not supported.

Note: These functions can be called from tasks created using either pthread or FreeRTOS APIs.

Condition Variables

- **`pthread_cond_init()`**
  - The `attr` argument is not implemented and is ignored.
- `pthread_cond_destroy()`
- `pthread_cond_signal()`
- `pthread_cond_broadcast()`
- `pthread_cond_wait()`
- `pthread_cond_timedwait()`

Static initializer constant `PTHREAD_COND_INITIALIZER` is supported.

- The resolution of `pthread_cond_timedwait()` timeouts is the RTOS tick period (see *CONFIG_FREERTOS_HZ*). Timeouts may be delayed up to one tick period after the requested timeout.

Note: These functions can be called from tasks created using either pthread or FreeRTOS APIs.

Semaphores In ESP-IDF, POSIX **unnamed** semaphores are implemented. The accessible API is described below. It implements semaphores as specified in the POSIX standard, unless specified otherwise.

- sem_init()
- sem_destroy()
  - `pshared` is ignored. Semaphores can always be shared between FreeRTOS tasks.
- sem_post()
  - If the semaphore has a value of `SEM_VALUE_MAX` already, `-1` is returned and `errno` is set to `EAGAIN`.
- sem_wait()
- sem_trywait()
- sem_timedwait()
  - The time value passed by abstime will be rounded up to the next FreeRTOS tick.
  - The actual timeout happens after the tick that the time was rounded to and before the following tick.
  - It is possible, though unlikely, that the task is preempted directly after the timeout calculation, delaying the timeout of the following blocking operating system call by the duration of the preemption.
- sem_getvalue()

Read/Write Locks The following API functions of the POSIX reader-writer locks specification are implemented:

- pthread_rwlock_init()
  - The `attr` argument is not implemented and is ignored.
- pthread_rwlock_destroy()
- pthread_rwlock_rdlock()
- pthread_rwlock_tryrdlock()
- pthread_rwlock_wrlock()
- pthread_rwlock_trywrlock()
- pthread_rwlock_unlock()

The static initializer constant `PTHREAD_RWLOCK_INITIALIZER` is supported.

Note: These functions can be called from tasks created using either pthread or FreeRTOS APIs.

Thread-Specific Data

- **`pthread_key_create()`**
  - The `destr_function` argument is supported and will be called if a thread function exits normally, calls `pthread_exit()`, or if the underlying task is deleted directly using the FreeRTOS function *`vTaskDelete()`*.
- `pthread_key_delete()`
- `pthread_setspecific()` / `pthread_getspecific()`

Note: These functions can be called from tasks created using either pthread or FreeRTOS APIs. When calling these functions from tasks created using FreeRTOS APIs, *CONFIG_FREERTOS_TLSP_DELETION_CALLBACKS* config option must be enabled to ensure the thread-specific data is cleaned up before the task is deleted.

Note: There are other options for thread local storage in ESP-IDF, including options with higher performance. See *Thread Local Storage*.

Not Implemented

The `pthread.h` header is a standard header and includes additional APIs and features which are not implemented in ESP-IDF. These include:

- `pthread_cancel()` returns `ENOSYS` if called.
- `pthread_condattr_init()` returns `ENOSYS` if called.

Other POSIX Threads functions (not listed here) are not implemented and will produce either a compiler or a linker error if referenced from an ESP-IDF application. If you identify a useful API that you would like to see implemented in ESP-IDF, please open a feature request on GitHub with the details.

ESP-IDF Extensions

The API *`esp_pthread_set_cfg()`* defined in the `esp_pthreads.h` header offers custom extensions to control how subsequent calls to `pthread_create()` behaves. Currently, the following configuration can be set:

- Default stack size of new threads, if not specified when calling `pthread_create()` (overrides *CONFIG_PTHREAD_TASK_STACK_SIZE_DEFAULT*).
- Stack memory capabilities determine which kind of memory is used for allocating pthread stacks. The field takes ESP-IDF heap capability flags, as defined in heap/include/esp_heap_caps.h. The memory must be 8-bit accessible (MALLOC_CAP_8BIT), besides other custom flags the user can choose from. The user is responsible for ensuring the correctness of the stack memory capabilities. For more information about memory locations, refer to the documentation of *Memory Capabilities*.
- RTOS priority of new threads (overrides *CONFIG_PTHREAD_TASK_PRIO_DEFAULT*).
- Core affinity / core pinning of new threads (overrides *CONFIG_PTHREAD_TASK_CORE_DEFAULT*).
- FreeRTOS task name for new threads (overrides *CONFIG_PTHREAD_TASK_NAME_DEFAULT*)

This configuration is scoped to the calling thread (or FreeRTOS task), meaning that *`esp_pthread_set_cfg()`* can be called independently in different threads or tasks. If the `inherit_cfg` flag is set in the current configuration then any new thread created will inherit the creator's configuration (if that thread calls `pthread_create()` recursively), otherwise the new thread will have the default configuration.

Examples

- system/pthread demonstrates using the pthreads API to create threads.
- cxx/pthread demonstrates using C++ Standard Library functions with threads.

API Reference

Header File

- components/pthread/include/esp_pthread.h
- This header file can be included with:

```
#include "esp_pthread.h"
```

- This header file is a part of the API provided by the `pthread` component. To declare that your component depends on `pthread`, add the following to your CMakeLists.txt:

```
REQUIRES pthread
```

or

```
PRIV_REQUIRES pthread
```

Functions

esp_pthread_cfg_t **esp_pthread_get_default_config**(void)

Creates a default pthread configuration based on the values set via menuconfig.

Returns A default configuration structure.

esp_err_t **esp_pthread_set_cfg**(const *esp_pthread_cfg_t* *cfg)

Configure parameters for creating pthread.

This API allows you to configure how the subsequent pthread_create() call will behave. This call can be used to setup configuration parameters like stack size, priority, configuration inheritance etc.

If the 'inherit' flag in the configuration structure is enabled, then the same configuration is also inherited in the thread subtree.

Note: If cfg->stack_alloc_caps is 0, it is automatically set to valid default stack memory capabilities. If cfg->stack_alloc_caps is non-zero, the developer is responsible for its correctenss. This function only checks that the capabilities are MALLOC_CAP_8BIT, the rest is unchecked.

Note: Passing non-NULL attributes to pthread_create() will override the stack_size parameter set using this API

Parameters **cfg** -- The pthread config parameters

Returns

- ESP_OK if configuration was successfully set
- ESP_ERR_NO_MEM if out of memory
- ESP_ERR_INVALID_ARG if stack_size is less than PTHREAD_STACK_MIN
- ESP_ERR_INVALID_ARG if stack_alloc_caps does not include MALLOC_CAP_8BIT

esp_err_t **esp_pthread_get_cfg**(*esp_pthread_cfg_t* *p)

Get current pthread creation configuration.

This will retrieve the current configuration that will be used for creating threads.

Parameters **p** -- Pointer to the pthread config structure that will be updated with the currently configured parameters

Returns

- ESP_OK if the configuration was available
- ESP_ERR_NOT_FOUND if a configuration wasn't previously set

esp_err_t **esp_pthread_init**(void)

Initialize pthread library.

Structures

struct **esp_pthread_cfg_t**

pthread configuration structure that influences pthread creation

Public Members

size_t **stack_size**

The stack size of the pthread

size_t **prio**

The thread's priority

bool **inherit_cfg**

Inherit this configuration further

const char ***thread_name**

The thread name.

int **pin_to_core**

The core id to pin the thread to. Has the same value range as xCoreId argument of xTaskCreatePinnedToCore.

uint32_t **stack_alloc_caps**

A bit mask of memory capabilities (MALLOC_CAPS*) to use when allocating the stack. The memory must be 8 bit accessible (MALLOC_CAP_8BIT). The developer is responsible for the correctenss of `stack_alloc_caps`.

Macros

PTHREAD_STACK_MIN

2.10.28 Random Number Generation

ESP32-S3 contains a hardware random number generator (RNG). You can use the APIs *`esp_random()`* and *`esp_fill_random()`* to obtained random values from it.

The hardware RNG produces true random numbers so long as one or more of the following conditions are met:

- RF subsystem is enabled. i.e., Wi-Fi or Bluetooth are enabled.
- The internal entropy source (SAR ADC) has been enabled by calling *`bootloader_random_enable()`* and not yet disabled by calling *`bootloader_random_disable()`*.
- While the ESP-IDF *Second Stage Bootloader* is running. This is because the default ESP-IDF bootloader implementation calls *`bootloader_random_enable()`* when the bootloader starts, and *`bootloader_random_disable()`* before executing the application.

When any of these conditions are true, samples of physical noise are continuously mixed into the internal hardware RNG state to provide entropy. Consult the **ESP32-S3 Technical Reference Manual > Random Number Generator (RNG)** [PDF] chapter for more details.

If none of the above conditions are true, the output of the RNG should be considered as pseudo-random only.

Startup

During startup, the ESP-IDF bootloader temporarily enables the non-RF internal entropy source (SAR ADC using internal reference voltage noise) that provides entropy for any first boot key generation.

However, after the application starts executing, then normally only pseudo-random numbers are available until Wi-Fi or Bluetooth are initialized or until the internal entropy source has been enabled again.

To re-enable the entropy source temporarily during application startup, or for an application that does not use Wi-Fi or Bluetooth, call the function *`bootloader_random_enable()`* to re-enable the internal entropy source. The function *`bootloader_random_disable()`* must be called to disable the entropy source again before using any of the following features:

- ADC
- Wi-Fi or Bluetooth

Note: The entropy source enabled during the boot process by the ESP-IDF Second Stage Bootloader seeds the internal RNG state with some entropy. However, the internal hardware RNG state is not large enough to provide a continuous stream of true random numbers. This is why a continuous entropy source must be enabled whenever true random numbers are required.

Note: If an application requires a source of true random numbers but cannot permanently enable a hardware entropy source, consider using a strong software DRBG implementation such as the mbedTLS CTR-DRBG or HMAC-DRBG, with an initial seed of entropy from hardware RNG true random numbers.

Secondary Entropy

ESP32-S3 RNG contains a secondary entropy source, based on sampling an asynchronous 8 MHz internal oscillator (see the Technical Reference Manual for details). This entropy source is always enabled in ESP-IDF and is continuously mixed into the RNG state by hardware. In testing, this secondary entropy source was sufficient to pass the Dieharder random number test suite without the main entropy source enabled (test input was created by concatenating short samples from continuously resetting ESP32-S3). However, it is currently only guaranteed that true random numbers are produced when the main entropy source is also enabled as described above.

API Reference

Header File

- components/esp_hw_support/include/esp_random.h
- This header file can be included with:

```
#include "esp_random.h"
```

Functions

uint32_t **esp_random**(void)

Get one random 32-bit word from hardware RNG.

If Wi-Fi or Bluetooth are enabled, this function returns true random numbers. In other situations, if true random numbers are required then consult the ESP-IDF Programming Guide "Random Number Generation" section for necessary prerequisites.

This function automatically busy-waits to ensure enough external entropy has been introduced into the hardware RNG state, before returning a new random number. This delay is very short (always less than 100 CPU cycles).

Returns Random value between 0 and UINT32_MAX

void **esp_fill_random**(void *buf, size_t len)

Fill a buffer with random bytes from hardware RNG.

Note: This function is implemented via calls to esp_random(), so the same constraints apply.

Parameters

- **buf** -- Pointer to buffer to fill with random numbers.
- **len** -- Length of buffer in bytes

Header File

- components/bootloader_support/include/bootloader_random.h
- This header file can be included with:

```
#include "bootloader_random.h"
```

- This header file is a part of the API provided by the `bootloader_support` component. To declare that your component depends on `bootloader_support`, add the following to your CMakeLists.txt:

```
REQUIRES bootloader_support
```

or

```
PRIV_REQUIRES bootloader_support
```

Functions

void **bootloader_random_enable**(void)

Enable an entropy source for RNG if RF subsystem is disabled.

The exact internal entropy source mechanism depends on the chip in use but all SoCs use the SAR ADC to continuously mix random bits (an internal noise reading) into the HWRNG. Consult the SoC Technical Reference Manual for more information.

Can also be called from app code, if true random numbers are required without initialized RF subsystem. This might be the case in early startup code of the application when the RF subsystem has not started yet or if the RF subsystem should not be enabled for power saving.

Consult ESP-IDF Programming Guide "Random Number Generation" section for details.

Warning: This function is not safe to use if any other subsystem is accessing the RF subsystem or the ADC at the same time!

void **bootloader_random_disable**(void)

Disable entropy source for RNG.

Disables internal entropy source. Must be called after bootloader_random_enable() and before RF subsystem features, ADC, or I2S (ESP32 only) are initialized.

Consult the ESP-IDF Programming Guide "Random Number Generation" section for details.

void **bootloader_fill_random**(void *buffer, size_t length)

Fill buffer with 'length' random bytes.

Note: If this function is being called from app code only, and never from the bootloader, then it's better to call esp_fill_random().

Parameters

- **buffer** -- Pointer to buffer
- **length** -- This many bytes of random data will be copied to buffer

getrandom()

A compatible version of the Linux `getrandom()` function is also provided for ease of porting:

```
#include <sys/random.h>

ssize_t getrandom(void *buf, size_t buflen, unsigned int flags);
```

This function is implemented by calling *esp_fill_random()* internally.

The `flags` argument is ignored. This function is always non-blocking but the strength of any random numbers is dependent on the same conditions described above.

Return value is -1 (with `errno` set to `EFAULT`) if the `buf` argument is NULL, and equal to `buflen` otherwise.

getentropy()

A compatible version of the Linux `getentropy()` function is also provided for easy porting:

```
#include <unistd.h>

int getentropy(void *buffer, size_t length);
```

This function is implemented by calling `getrandom()` internally.

The strength of any random numbers is dependent on the same conditions described above.

Return value is 0 on success and -1 otherwise with `errno` set to:

- `EFAULT` if the `buffer` argument is NULL.
- `EIO` if the `length` is more then 256.

2.10.29 Sleep Modes

Overview

ESP32-S3 supports two major power saving modes: Light-sleep and Deep-sleep. According to the features used by an application, there are some sub sleep modes. See *Sleep Modes* for these sleep modes and sub sleep modes. Additionally, there are some power-down options that can be configured to further reduce the power consumption. See *Power-down Options* for more details.

There are several wakeup sources in the sleep modes. These sources can also be combined so that the chip will wake up when any of the sources are triggered. *Wakeup Sources* describes these wakeup sources and configuration APIs in detail.

The configuration of power-down options and wakeup sources are optional. They can be configured at any moment before entering the sleep modes.

Then the application can call sleep start APIs to enter one of the sleep modes. See *Entering Sleep* for more details. When the wakeup condition is met, the application is awoken from sleep. See *Checking Sleep Wakeup Cause* on how to get the wakeup cause, and *Disable Sleep Wakeup Source* on how to handle the wakeup sources after wakeup.

Sleep Modes

In Light-sleep mode, the digital peripherals, most of the RAM, and CPUs are clock-gated and their supply voltage is reduced. Upon exit from Light-sleep, the digital peripherals, RAM, and CPUs resume operation and their internal states are preserved.

In Deep-sleep mode, the CPUs, most of the RAM, and all digital peripherals that are clocked from APB_CLK are powered off. The only parts of the chip that remain powered on are:

- RTC controller
- ULP coprocessor
- RTC FAST memory
- RTC SLOW memory

Wi-Fi/Bluetooth and Sleep Modes In Deep-sleep and Light-sleep modes, the wireless peripherals are powered down. Before entering Deep-sleep or Light-sleep modes, the application must disable Wi-Fi and Bluetooth using the appropriate calls (i.e., `nimble_port_stop()`, `nimble_port_deinit()`, *`esp_bluedroid_disable()`*, *`esp_bluedroid_deinit()`*, *`esp_bt_controller_disable()`*, *`esp_bt_controller_deinit()`*, *`esp_wifi_stop()`*). Wi-Fi and Bluetooth connections are not maintained in Deep-sleep or Light-sleep mode, even if these functions are not called.

If Wi-Fi/Bluetooth connections need to be maintained, enable Wi-Fi/Bluetooth Modem-sleep mode and automatic Light-sleep feature (see *Power Management APIs*). This allows the system to wake up from sleep automatically when required by the Wi-Fi/Bluetooth driver, thereby maintaining the connection.

Sub Sleep Modes Tables below list the sub sleep modes in the first row and the features they support in the first column. Modes that support more features may consume more power during sleep mode. The sleep system automatically selects the mode that satisfies all the features required by the user while consuming least power.

Deep-sleep:

	DSLP_ULTRA_LOW	DSLP_DEFAULT	DSLP_8MD256/ DSLP_ADC_TSENS
ULP/Touch sensor (ESP32-S2 and ESP32-S3 only)	Y	Y	Y
RTC IO input/RTC memory at high temperature		Y	Y
ADC_TSEN_MONITOR			Y
8MD256 as the clock source for RTC_SLOW_CLK			Y

Features:

1. RTC IO input/RTC memory at high temperature (experimental): Use RTC IO as input pins, or use RTC memory at high temperature. The chip can go into ultra low power mode when these features are disabled. Controlled by API `rtc_sleep_enable_ultra_low()`.
2. ADC_TSEN_MONITOR: Use ADC/Temperature Sensor in monitor mode (controlled by ULP). Enabled by API `ulp_adc_init()` or its higher level APIs. Only available for ESP32-S2 and ESP32-S3 chips with monitor mode.
3. 8MD256 as the clock source for RTC_SLOW_CLK: When 8MD256 is selected as the clock source for RTC_SLOW_CLK using the Kconfig option `CONFIG_RTC_CLK_SRC_INT_8MD256`, the chip will automatically enter this sub sleep mode during Deep-sleep mode.

Light-sleep:

	LSLP_DEFAULT	LSLP_ADC_TSENS	LSLP_8MD256	LSLP_LEDC8M/ LSLP_XTAL_FPU
ULP/Touch sensor (ESP32-S2 and ESP32-S3 only)	Y	Y	Y	Y
RTC IO input/RTC memory at high temperature	Y	Y	Y	Y
ADC_TSEN_MONITOR		Y	Y	Y
8MD256 as the clock source for RTC_SLOW_CLK			Y	Y
8 MHz RC clock source used by digital peripherals				Y
Keep the XTAL clock on				Y

Features: (Also see 8MD256 and ADC_TSEN_MONITOR features for Deep-sleep mode above)

1. 8 MHz RC clock source used by digital peripherals: Currently, only LEDC uses this clock source during Light-sleep mode. When LEDC selects this clock source, this feature is automatically enabled.
2. Keep the XTAL clock on: Keep the XTAL clock on during Light-sleep mode. Controlled by `ESP_PD_DOMAIN_XTAL` power domain.

Default mode of ESP32-S3 already supports ADC_TSEN_MONITOR feature.

Wakeup Sources

Wakeup sources can be enabled using `esp_sleep_enable_X_wakeup` APIs. Wakeup sources are not disabled after wakeup, you can disable them using *`esp_sleep_disable_wakeup_source()`* API if you do not need them any more. See *Disable Sleep Wakeup Source*.

Following are the wakeup sources supported on ESP32-S3.

Timer The RTC controller has a built-in timer which can be used to wake up the chip after a predefined amount of time. Time is specified at microsecond precision, but the actual resolution depends on the clock source selected for RTC_SLOW_CLK.

For details on RTC clock options, see **ESP32-S3 Technical Reference Manual > ULP Coprocessor** [PDF].

RTC peripherals or RTC memories do not need to be powered on during sleep in this wakeup mode.

`esp_sleep_enable_timer_wakeup()` function can be used to enable sleep wakeup using a timer.

Touchpad The RTC IO module contains the logic to trigger wakeup when a touch sensor interrupt occurs. To wakeup from a touch sensor interrupt, users need to configure the touch pad interrupt before the chip enters Deep-sleep or Light-sleep modes.

`esp_sleep_enable_touchpad_wakeup()` function can be used to enable this wakeup source.

External Wakeup (`ext0`) The RTC IO module contains the logic to trigger wakeup when one of RTC GPIOs is set to a predefined logic level. RTC IO is part of the RTC peripherals power domain, so RTC peripherals will be kept powered on during Deep-sleep if this wakeup source is requested.

The RTC IO module is enabled in this mode, so internal pullup or pulldown resistors can also be used. They need to be configured by the application using *`rtc_gpio_pullup_en()`* and *`rtc_gpio_pulldown_en()`* functions before calling *`esp_deep_sleep_start()`*.

`esp_sleep_enable_ext0_wakeup()` function can be used to enable this wakeup source.

> **Warning:** After waking up from sleep, the IO pad used for wakeup will be configured as RTC IO. Therefore, before using this pad as digital GPIO, users need to reconfigure it using *`rtc_gpio_deinit()`* function.

External Wakeup (`ext1`) The RTC controller contains the logic to trigger wakeup using multiple RTC GPIOs. One of the following two logic functions can be used to trigger ext1 wakeup:

- wake up if any of the selected pins is high (`ESP_EXT1_WAKEUP_ANY_HIGH`)
- wake up if any of the selected pins is low (`ESP_EXT1_WAKEUP_ANY_LOW`)

This wakeup source is controlled by the RTC controller. Unlike `ext0`, this wakeup source supports wakeup even when the RTC peripheral is powered down. Although the power domain of the RTC peripheral, where RTC IOs are located, is powered down during sleep modes, ESP-IDF will automatically lock the state of the wakeup pin before the system enters sleep modes and unlock upon exiting sleep modes. Therefore, the internal pull-up or pull-down resistors can still be configured for the wakeup pin:

```
esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_ON);
rtc_gpio_pullup_dis(gpio_num);
rtc_gpio_pulldown_en(gpio_num);
```

If we turn off the `RTC_PERIPH` domain, we will use the HOLD feature to maintain the pull-up and pull-down on the pins during sleep. HOLD feature will be acted on the pin internally before the system enters sleep modes, and this can further reduce power consumption:

```
rtc_gpio_pullup_dis(gpio_num);
rtc_gpio_pulldown_en(gpio_num);
```

If certain chips lack the `RTC_PERIPH` domain, we can only use the HOLD feature to maintain the pull-up and pull-down on the pins during sleep modes:

```
gpio_pullup_dis(gpio_num);
gpio_pulldown_en(gpio_num);
```

esp_sleep_enable_ext1_wakeup_io() function can be used to append ext1 wakeup IO and set corresponding wakeup level.

esp_sleep_disable_ext1_wakeup_io() function can be used to remove ext1 wakeup IO.

Note: Due to hardware limitation, when we use more than one IO for EXT1 wakeup, it is not allowed to configure different wakeup levels for the IOs, and there is corresponding inspection mechanism in *esp_sleep_enable_ext1_wakeup_io()*.

Warning:

- To use the EXT1 wakeup, the IO pad(s) are configured as RTC IO. Therefore, before using these pads as digital GPIOs, users need to reconfigure them by calling the *rtc_gpio_deinit()* function.
- If the RTC peripherals are configured to be powered down (which is by default), the wakeup IOs will be set to the holding state before entering sleep. Therefore, after the chip wakes up from Light-sleep, please call `rtc_gpio_hold_dis` to disable the hold function to perform any pin re-configuration. For Deep-sleep wakeup, this is already being handled at the application startup stage.

ULP Coprocessor Wakeup ULP coprocessor can run while the chip is in sleep mode, and may be used to poll sensors, monitor ADC or touch sensor values, and wake up the chip when a specific event is detected. ULP coprocessor is part of the RTC peripherals power domain, and it runs the program stored in RTC SLOW memory. RTC SLOW memory will be powered on during sleep if this wakeup mode is requested. RTC peripherals will be automatically powered on before ULP coprocessor starts running the program; once the program stops running, RTC peripherals are automatically powered down again.

esp_sleep_enable_ulp_wakeup() function can be used to enable this wakeup source.

GPIO Wakeup (Light-sleep Only) In addition to EXT0 and EXT1 wakeup sources described above, one more method of wakeup from external inputs is available in Light-sleep mode. With this wakeup source, each pin can be individually configured to trigger wakeup on high or low level using *gpio_wakeup_enable()* function. Unlike EXT0 and EXT1 wakeup sources, which can only be used with RTC IOs, this wakeup source can be used with any IO (RTC or digital).

esp_sleep_enable_gpio_wakeup() function can be used to enable this wakeup source.

Warning: Before entering Light-sleep mode, check if any GPIO pin to be driven is part of the VDD_SPI power domain. If so, this power domain must be configured to remain ON during sleep.

For example, on ESP32-WROOM-32 board, GPIO16 and GPIO17 are linked to VDD_SPI power domain. If they are configured to remain high during Light-sleep, the power domain should be configured to remain powered ON. This can be done with *esp_sleep_pd_config()*:

```
esp_sleep_pd_config(ESP_PD_DOMAIN_VDDSDIO, ESP_PD_OPTION_ON);
```

UART Wakeup (Light-sleep Only) When ESP32-S3 receives UART input from external devices, it is often necessary to wake up the chip when input data is available. The UART peripheral contains a feature which allows waking up the chip from Light-sleep when a certain number of positive edges on RX pin are seen. This number of positive edges can be set using *uart_set_wakeup_threshold()* function. Note that the character which triggers wakeup (and any characters before it) will not be received by the UART after wakeup. This means that the external device typically needs to send an extra character to the ESP32-S3 to trigger wakeup before sending the data.

esp_sleep_enable_uart_wakeup() function can be used to enable this wakeup source.

After waking-up from UART, you should send some extra data through the UART port in Active mode, so that the internal wakeup indication signal can be cleared. Otherwises, the next UART wake-up would trigger with two less rising edges than the configured threshold value.

Disable Sleep Wakeup Source Previously configured wakeup sources can be disabled later using *esp_sleep_disable_wakeup_source()* API. This function deactivates trigger for the given wakeup source. Additionally, it can disable all triggers if the argument is `ESP_SLEEP_WAKEUP_ALL`.

Power-down Options

The application can force specific powerdown modes for RTC peripherals and RTC memories. In Deep-sleep mode, we can also isolate some IOs to further reduce current consumption.

Power-down of RTC Peripherals and Memories By default, *esp_deep_sleep_start()* and *esp_light_sleep_start()* functions power down all RTC power domains which are not needed by the enabled wakeup sources. To override this behaviour, *esp_sleep_pd_config()* function is provided.

If some variables in the program are placed into RTC SLOW memory (for example, using `RTC_DATA_ATTR` attribute), RTC SLOW memory will be kept powered on by default. This can be overridden using *esp_sleep_pd_config()* function, if desired.

Power-down of Flash By default, to avoid potential issues, *esp_light_sleep_start()* function does **not** power down flash. To be more specific, it takes time to power down the flash and during this period the system may be woken up, which then actually powers up the flash before this flash could be powered down completely. As a result, there is a chance that the flash may not work properly.

So, in theory, it is ok if you only wake up the system after the flash is completely powered down. However, in reality, the flash power-down period can be hard to predict (for example, this period can be much longer when you add filter capacitors to the flash's power supply circuit) and uncontrollable (for example, the asynchronous wake-up signals make the actual sleep time uncontrollable).

Warning: If a filter capacitor is added to your flash power supply circuit, please do everything possible to avoid powering down flash.

Therefore, it is recommended not to power down flash when using ESP-IDF. For power-sensitive applications, it is recommended to use Kconfig option *CONFIG_ESP_SLEEP_FLASH_LEAKAGE_WORKAROUND* to reduce the power consumption of the flash during Light-sleep, instead of powering down the flash.

It is worth mentioning that PSRAM has a similar Kconfig option *CONFIG_ESP_SLEEP_PSRAM_LEAKAGE_WORKAROUND*.

However, for those who have fully understood the risk and are still willing to power down the flash to further reduce the power consumption, please check the following mechanisms:

- Setting Kconfig option *CONFIG_ESP_SLEEP_POWER_DOWN_FLASH* only powers down the flash when the RTC timer is the only wake-up source **and** the sleep time is longer than the flash power-down period.
- Calling `esp_sleep_pd_config(ESP_PD_DOMAIN_VDDSDIO, ESP_PD_OPTION_OFF)` powers down flash when the RTC timer is not enabled as a wakeup source **or** the sleep time is longer than the flash power-down period.

Note:

- ESP-IDF does not provide any mechanism that can power down the flash in all conditions when Light-sleep.
- *esp_deep_sleep_start()* function forces power down flash regardless of user configuration.

Configuring IOs (Deep-sleep Only) Some ESP32-S3 IOs have internal pullups or pulldowns, which are enabled by default. If an external circuit drives this pin in Deep-sleep mode, current consumption may increase due to current flowing through these pullups and pulldowns.

To isolate a pin to prevent extra current draw, call `rtc_gpio_isolate()` function.

For example, on ESP32-WROVER module, GPIO12 is pulled up externally, and it also has an internal pulldown in the ESP32 chip. This means that in Deep-sleep, some current flows through these external and internal resistors, increasing Deep-sleep current above the minimal possible value.

Add the following code before *esp_deep_sleep_start()* to remove such extra current:

```
rtc_gpio_isolate(GPIO_NUM_12);
```

Entering Sleep

esp_light_sleep_start() or *esp_deep_sleep_start()* functions can be used to enter Light-sleep or Deep-sleep modes correspondingly. After that, the system configures the parameters of RTC controller according to the requested wakeup sources and power-down options.

It is also possible to enter sleep modes with no wakeup sources configured. In this case, the chip will be in sleep modes indefinitely until external reset is applied.

UART Output Handling Before entering sleep mode, *esp_deep_sleep_start()* will flush the contents of UART FIFOs.

When entering Light-sleep mode using *esp_light_sleep_start()*, UART FIFOs will not be flushed. Instead, UART output will be suspended, and remaining characters in the FIFO will be sent out after wakeup from Light-sleep.

Checking Sleep Wakeup Cause

esp_sleep_get_wakeup_cause() function can be used to check which wakeup source has triggered wakeup from sleep mode.

For touchpad, it is possible to identify which touch pin has caused wakeup using *esp_sleep_get_touchpad_wakeup_status()* functions.

For ext1 wakeup sources, it is possible to identify which touch pin has caused wakeup using *esp_sleep_get_ext1_wakeup_status()* functions.

Application Example

- protocols/sntp: the implementation of basic functionality of Deep-sleep, where ESP module is periodically waken up to retrieve time from NTP server.
- wifi/power_save: the usage of Wi-Fi Modem-sleep mode and automatic Light-sleep feature to maintain Wi-Fi connections.
- bluetooth/nimble/power_save: the usage of Bluetooth Modem-sleep mode and automatic Light-sleep feature to maintain Bluetooth connections.
- system/deep_sleep: the usage of various Deep-sleep wakeup triggers and ULP coprocessor programming.
- system/light_sleep: the usage of Light-sleep wakeup triggered by various sources supported by the chip (Timer, GPIO, Touch Sensor, etc.).

API Reference

Header File

- components/esp_hw_support/include/esp_sleep.h
- This header file can be included with:

```
#include "esp_sleep.h"
```

Functions

esp_err_t **esp_sleep_disable_wakeup_source**(*esp_sleep_source_t* source)

Disable wakeup source.

This function is used to deactivate wake up trigger for source defined as parameter of the function.

See docs/sleep-modes.rst for details.

Note: This function does not modify wake up configuration in RTC. It will be performed in esp_deep_sleep_start/esp_light_sleep_start function.

Parameters **source** -- - number of source to disable of type esp_sleep_source_t

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if trigger was not active

esp_err_t **esp_sleep_enable_ulp_wakeup**(void)

Enable wakeup by ULP coprocessor.

Note: On ESP32, ULP wakeup source cannot be used when RTC_PERIPH power domain is forced, to be powered on (ESP_PD_OPTION_ON) or when ext0 wakeup source is used.

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if additional current by touch (CONFIG_RTC_EXT_CRYST_ADDIT_CURRENT) is enabled.
- ESP_ERR_INVALID_STATE if ULP co-processor is not enabled or if wakeup triggers conflict

esp_err_t **esp_sleep_enable_timer_wakeup**(uint64_t time_in_us)

Enable wakeup by timer.

Parameters **time_in_us** -- time before wakeup, in microseconds
Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if value is out of range (TBD)

esp_err_t **esp_sleep_enable_touchpad_wakeup**(void)

Enable wakeup by touch sensor.

Note: On ESP32, touch wakeup source can not be used when RTC_PERIPH power domain is forced to be powered on (ESP_PD_OPTION_ON) or when ext0 wakeup source is used.

Note: The FSM mode of the touch button should be configured as the timer trigger mode.

Returns
- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if additional current by touch (CONFIG_RTC_EXT_CRYST_ADDIT_CURRENT) is enabled.
- ESP_ERR_INVALID_STATE if wakeup triggers conflict

touch_pad_t **esp_sleep_get_touchpad_wakeup_status**(void)

Get the touch pad which caused wakeup.

If wakeup was caused by another source, this function will return TOUCH_PAD_MAX;

Returns touch pad which caused wakeup

bool **esp_sleep_is_valid_wakeup_gpio**(gpio_num_t gpio_num)

Returns true if a GPIO number is valid for use as wakeup source.

Note: For SoCs with RTC IO capability, this can be any valid RTC IO input pin.

Parameters **gpio_num** -- Number of the GPIO to test for wakeup source capability
Returns True if this GPIO number will be accepted as a sleep wakeup source.

esp_err_t **esp_sleep_enable_ext0_wakeup**(gpio_num_t gpio_num, int level)

Enable wakeup using a pin.

This function uses external wakeup feature of RTC_IO peripheral. It will work only if RTC peripherals are kept on during sleep.

This feature can monitor any pin which is an RTC IO. Once the pin transitions into the state given by level argument, the chip will be woken up.

Note: This function does not modify pin configuration. The pin is configured in esp_deep_sleep_start/esp_light_sleep_start, immediately before entering sleep mode.

Note: ESP32: ext0 wakeup source can not be used together with touch or ULP wakeup sources.

Parameters

- **gpio_num** -- GPIO number used as wakeup source. Only GPIOs with the RTC functionality can be used. For different SoCs, the related GPIOs are:
  - ESP32: 0, 2, 4, 12-15, 25-27, 32-39;
  - ESP32-S2: 0-21;
  - ESP32-S3: 0-21.
- **level** -- input level which will trigger wakeup (0=low, 1=high)

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if the selected GPIO is not an RTC GPIO, or the mode is invalid
- ESP_ERR_INVALID_STATE if wakeup triggers conflict

esp_err_t **esp_sleep_enable_ext1_wakeup**(uint64_t io_mask, *esp_sleep_ext1_wakeup_mode_t* level_mode)

Enable wakeup using multiple pins.

This function uses external wakeup feature of RTC controller. It will work even if RTC peripherals are shut down during sleep.

This feature can monitor any number of pins which are in RTC IOs. Once selected pins go into the state given by level_mode argument, the chip will be woken up.

Note: This function does not modify pin configuration. The pins are configured in esp_deep_sleep_start/esp_light_sleep_start, immediately before entering sleep mode.

Note: Internal pullups and pulldowns don't work when RTC peripherals are shut down. In this case, external resistors need to be added. Alternatively, RTC peripherals (and pullups/pulldowns) may be kept enabled using esp_sleep_pd_config function. If we turn off the `RTC_PERIPH` domain or certain chips lack the `RTC_PERIPH` domain, we will use the HOLD feature to maintain the pull-up and pull-down on the pins during sleep. HOLD feature will be acted on the pin internally before the system entering sleep, and this can further reduce power consumption.

Note: Call this func will reset the previous ext1 configuration.

Note: This function will be deprecated in release/v6.0. Please switch to use `esp_sleep_enable_ext1_wakeup_io` and `esp_sleep_disable_ext1_wakeup_io`

Parameters

- **io_mask** -- Bit mask of GPIO numbers which will cause wakeup. Only GPIOs which have RTC functionality can be used in this bit map. For different SoCs, the related GPIOs are:
  - ESP32: 0, 2, 4, 12-15, 25-27, 32-39
  - ESP32-S2: 0-21
  - ESP32-S3: 0-21
  - ESP32-C6: 0-7
  - ESP32-H2: 7-14
- **level_mode** -- Select logic function used to determine wakeup condition: When target chip is ESP32:
  - ESP_EXT1_WAKEUP_ALL_LOW: wake up when all selected GPIOs are low
  - ESP_EXT1_WAKEUP_ANY_HIGH: wake up when any of the selected GPIOs is high When target chip is ESP32-S2, ESP32-S3, ESP32-C6 or ESP32-H2:
  - ESP_EXT1_WAKEUP_ANY_LOW: wake up when any of the selected GPIOs is low
  - ESP_EXT1_WAKEUP_ANY_HIGH: wake up when any of the selected GPIOs is high

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if io_mask is zero,, or mode is invalid

esp_err_t **esp_sleep_enable_ext1_wakeup_io**(uint64_t io_mask, *esp_sleep_ext1_wakeup_mode_t* level_mode)

Enable ext1 wakeup pins with IO masks.

This will append selected IOs to the wakeup IOs, it will not reset previously enabled IOs. To reset specific previously enabled IOs, call esp_sleep_disable_ext1_wakeup_io with the io_mask. To reset all the enabled IOs, call esp_sleep_disable_ext1_wakeup_io(0).

This function uses external wakeup feature of RTC controller. It will work even if RTC peripherals are shut down during sleep.

This feature can monitor any number of pins which are in RTC IOs. Once selected pins go into the state given by level_mode argument, the chip will be woken up.

Note: This function does not modify pin configuration. The pins are configured in esp_deep_sleep_start/esp_light_sleep_start, immediately before entering sleep mode.

Note: Internal pullups and pulldowns don't work when RTC peripherals are shut down. In this case, external resistors need to be added. Alternatively, RTC peripherals (and pullups/pulldowns) may be kept enabled using esp_sleep_pd_config function. If we turn off the `RTC_PERIPH` domain or certain chips lack the `RTC_PERIPH` domain, we will use the HOLD feature to maintain the pull-up and pull-down on the pins during sleep. HOLD feature will be acted on the pin internally before the system entering sleep, and this can further reduce power consumption.

Parameters

- **io_mask** -- Bit mask of GPIO numbers which will cause wakeup. Only GPIOs which have RTC functionality can be used in this bit map. For different SoCs, the related GPIOs are:
  - ESP32: 0, 2, 4, 12-15, 25-27, 32-39
  - ESP32-S2: 0-21
  - ESP32-S3: 0-21
  - ESP32-C6: 0-7
  - ESP32-H2: 7-14
- **level_mode** -- Select logic function used to determine wakeup condition: When target chip is ESP32:
  - ESP_EXT1_WAKEUP_ALL_LOW: wake up when all selected GPIOs are low
  - ESP_EXT1_WAKEUP_ANY_HIGH: wake up when any of the selected GPIOs is high When target chip is ESP32-S2, ESP32-S3, ESP32-C6 or ESP32-H2:
  - ESP_EXT1_WAKEUP_ANY_LOW: wake up when any of the selected GPIOs is low
  - ESP_EXT1_WAKEUP_ANY_HIGH: wake up when any of the selected GPIOs is high

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if any of the selected GPIOs is not an RTC GPIO, or mode is invalid
- ESP_ERR_NOT_ALLOWED when wakeup level will become different between ext1 IOs if !SOC_PM_SUPPORT_EXT1_WAKEUP_MODE_PER_PIN

esp_err_t **esp_sleep_disable_ext1_wakeup_io**(uint64_t io_mask)

Disable ext1 wakeup pins with IO masks. This will remove selected IOs from the wakeup IOs.

Parameters **io_mask** -- Bit mask of GPIO numbers which will cause wakeup. Only GPIOs which have RTC functionality can be used in this bit map. If value is zero, this func will remove all previous ext1 configuration. For different SoCs, the related GPIOs are:

- ESP32: 0, 2, 4, 12-15, 25-27, 32-39
- ESP32-S2: 0-21
- ESP32-S3: 0-21
- ESP32-C6: 0-7
- ESP32-H2: 7-14

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if any of the selected GPIOs is not an RTC GPIO.

esp_err_t **esp_sleep_enable_gpio_wakeup**(void)

Enable wakeup from light sleep using GPIOs.

Each GPIO supports wakeup function, which can be triggered on either low level or high level. Unlike EXT0 and EXT1 wakeup sources, this method can be used both for all IOs: RTC IOs and digital IOs. It can only be used to wakeup from light sleep though.

To enable wakeup, first call gpio_wakeup_enable, specifying gpio number and wakeup level, for each GPIO which is used for wakeup. Then call this function to enable wakeup feature.

Note: On ESP32, GPIO wakeup source can not be used together with touch or ULP wakeup sources.

Returns

- ESP_OK on success
- ESP_ERR_INVALID_STATE if wakeup triggers conflict

esp_err_t **esp_sleep_enable_uart_wakeup**(int uart_num)

Enable wakeup from light sleep using UART.

Use uart_set_wakeup_threshold function to configure UART wakeup threshold.

Wakeup from light sleep takes some time, so not every character sent to the UART can be received by the application.

Note: ESP32 does not support wakeup from UART2.

Parameters **uart_num** -- UART port to wake up from

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if wakeup from given UART is not supported

esp_err_t **esp_sleep_enable_bt_wakeup**(void)

Enable wakeup by bluetooth.

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if wakeup from bluetooth is not supported

esp_err_t **esp_sleep_disable_bt_wakeup**(void)

Disable wakeup by bluetooth.

Returns

- ESP_OK on success
- ESP_ERR_NOT_SUPPORTED if wakeup from bluetooth is not supported

esp_err_t **esp_sleep_enable_wifi_wakeup**(void)

Enable wakeup by WiFi MAC.

Returns

- ESP_OK on success

esp_err_t **esp_sleep_disable_wifi_wakeup**(void)

Disable wakeup by WiFi MAC.

Returns
- ESP_OK on success

esp_err_t **esp_sleep_enable_wifi_beacon_wakeup**(void)

Enable beacon wakeup by WiFi MAC, it will wake up the system into modem state.

Returns
- ESP_OK on success

esp_err_t **esp_sleep_disable_wifi_beacon_wakeup**(void)

Disable beacon wakeup by WiFi MAC.

Returns
- ESP_OK on success

uint64_t **esp_sleep_get_ext1_wakeup_status**(void)

Get the bit mask of GPIOs which caused wakeup (ext1)

If wakeup was caused by another source, this function will return 0.

Returns bit mask, if GPIOn caused wakeup, BIT(n) will be set

esp_err_t **esp_sleep_pd_config**(*esp_sleep_pd_domain_t* domain, *esp_sleep_pd_option_t* option)

Set power down mode for an RTC power domain in sleep mode.

If not set set using this API, all power domains default to ESP_PD_OPTION_AUTO.

Parameters
- **domain** -- power domain to configure
- **option** -- power down option (ESP_PD_OPTION_OFF, ESP_PD_OPTION_ON, or ESP_PD_OPTION_AUTO)

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if either of the arguments is out of range

esp_err_t **esp_deep_sleep_try_to_start**(void)

Enter deep sleep with the configured wakeup options.

The reason for the rejection can be such as a short sleep time.

Note: In general, the function does not return, but if the sleep is rejected, then it returns from it.

Returns
- No return - If the sleep is not rejected.
- ESP_ERR_SLEEP_REJECT sleep request is rejected(wakeup source set before the sleep request)

void **esp_deep_sleep_start**(void)

Enter deep sleep with the configured wakeup options.

Note: The function does not do a return (no rejection). Even if wakeup source set before the sleep request it goes to deep sleep anyway.

esp_err_t **esp_light_sleep_start**(void)

Enter light sleep with the configured wakeup options.

Returns

- ESP_OK on success (returned after wakeup)
- ESP_ERR_SLEEP_REJECT sleep request is rejected(wakeup source set before the sleep request)
- ESP_ERR_SLEEP_TOO_SHORT_SLEEP_DURATION after deducting the sleep flow overhead, the final sleep duration is too short to cover the minimum sleep duration of the chip, when rtc timer wakeup source enabled

esp_err_t **esp_deep_sleep_try** (uint64_t time_in_us)

Enter deep-sleep mode.

The device will automatically wake up after the deep-sleep time Upon waking up, the device calls deep sleep wake stub, and then proceeds to load application.

Call to this function is equivalent to a call to esp_deep_sleep_enable_timer_wakeup followed by a call to esp_deep_sleep_start.

The reason for the rejection can be such as a short sleep time.

Note: In general, the function does not return, but if the sleep is rejected, then it returns from it.

Parameters **time_in_us** -- deep-sleep time, unit: microsecond

Returns

- No return - If the sleep is not rejected.
- ESP_ERR_SLEEP_REJECT sleep request is rejected(wakeup source set before the sleep request)

void **esp_deep_sleep** (uint64_t time_in_us)

Enter deep-sleep mode.

The device will automatically wake up after the deep-sleep time Upon waking up, the device calls deep sleep wake stub, and then proceeds to load application.

Call to this function is equivalent to a call to esp_deep_sleep_enable_timer_wakeup followed by a call to esp_deep_sleep_start.

Note: The function does not do a return (no rejection).. Even if wakeup source set before the sleep request it goes to deep sleep anyway.

Parameters **time_in_us** -- deep-sleep time, unit: microsecond

esp_err_t **esp_deep_sleep_register_hook** (*esp_deep_sleep_cb_t* new_dslp_cb)

Register a callback to be called from the deep sleep prepare.

Warning: deepsleep callbacks should without parameters, and MUST NOT, UNDER ANY CIRCUMSTANCES, CALL A FUNCTION THAT MIGHT BLOCK.

Parameters **new_dslp_cb** -- Callback to be called

Returns

- ESP_OK: Callback registered to the deepsleep misc_modules_sleep_prepare
- ESP_ERR_NO_MEM: No more hook space for register the callback

void **esp_deep_sleep_deregister_hook**(*esp_deep_sleep_cb_t* old_dslp_cb)

Unregister an deepsleep callback.

Parameters **old_dslp_cb** -- Callback to be unregistered

esp_sleep_wakeup_cause_t **esp_sleep_get_wakeup_cause**(void)

Get the wakeup source which caused wakeup from sleep.

Returns cause of wake up from last sleep (deep sleep or light sleep)

void **esp_wake_deep_sleep**(void)

Default stub to run on wake from deep sleep.

Allows for executing code immediately on wake from sleep, before the software bootloader or ESP-IDF app has started up.

This function is weak-linked, so you can implement your own version to run code immediately when the chip wakes from sleep.

See docs/deep-sleep-stub.rst for details.

void **esp_set_deep_sleep_wake_stub**(*esp_deep_sleep_wake_stub_fn_t* new_stub)

Install a new stub at runtime to run on wake from deep sleep.

If implementing esp_wake_deep_sleep() then it is not necessary to call this function.

However, it is possible to call this function to substitute a different deep sleep stub. Any function used as a deep sleep stub must be marked RTC_IRAM_ATTR, and must obey the same rules given for esp_wake_deep_sleep().

void **esp_set_deep_sleep_wake_stub_default_entry**(void)

Set wake stub entry to default `esp_wake_stub_entry`

esp_deep_sleep_wake_stub_fn_t **esp_get_deep_sleep_wake_stub**(void)

Get current wake from deep sleep stub.

Returns Return current wake from deep sleep stub, or NULL if no stub is installed.

void **esp_default_wake_deep_sleep**(void)

The default esp-idf-provided esp_wake_deep_sleep() stub.

See docs/deep-sleep-stub.rst for details.

void **esp_deep_sleep_disable_rom_logging**(void)

Disable logging from the ROM code after deep sleep.

Using LSB of RTC_STORE4.

esp_err_t **esp_sleep_cpu_pd_low_init**(void)

CPU Power down low-level initialize, enable CPU power down during light sleep.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM not enough retention memory

esp_err_t **esp_sleep_cpu_pd_low_deinit**(void)

CPU Power down low-level deinitialize, disable CPU power down during light sleep.

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM not enough retention memory

esp_err_t **esp_sleep_cpu_retention_init**(void)

CPU Power down initialize.

Returns

- ESP_OK on success

- ESP_ERR_NO_MEM not enough retention memory

esp_err_t **esp_sleep_cpu_retention_deinit**(void)

CPU Power down de-initialize.

Release system retention memory.

Returns

- ESP_OK on success

void **esp_sleep_config_gpio_isolate**(void)

Configure to isolate all GPIO pins in sleep state.

void **esp_sleep_enable_gpio_switch**(bool enable)

Enable or disable GPIO pins status switching between slept status and waked status.

Parameters **enable** -- decide whether to switch status or not

Macros

ESP_PD_DOMAIN_RTC8M

ESP_SLEEP_POWER_DOWN_CPU

Type Definitions

typedef void (***esp_deep_sleep_cb_t**)(void)

typedef *esp_sleep_source_t* **esp_sleep_wakeup_cause_t**

typedef void (***esp_deep_sleep_wake_stub_fn_t**)(void)

Function type for stub to run on wake from sleep.

Enumerations

enum **esp_sleep_ext1_wakeup_mode_t**

Logic function used for EXT1 wakeup mode.

Values:

enumerator **ESP_EXT1_WAKEUP_ANY_LOW**

Wake the chip when any of the selected GPIOs go low.

enumerator **ESP_EXT1_WAKEUP_ANY_HIGH**

Wake the chip when any of the selected GPIOs go high.

enumerator **ESP_EXT1_WAKEUP_ALL_LOW**

enum **esp_sleep_pd_domain_t**

Power domains which can be powered down in sleep mode.

Values:

enumerator **ESP_PD_DOMAIN_RTC_PERIPH**

RTC IO, sensors and ULP co-processor.

enumerator **ESP_PD_DOMAIN_XTAL**

XTAL oscillator.

enumerator **ESP_PD_DOMAIN_RC_FAST**

Internal Fast oscillator.

enumerator **ESP_PD_DOMAIN_CPU**

CPU core.

enumerator **ESP_PD_DOMAIN_VDDSDIO**

VDD_SDIO.

enumerator **ESP_PD_DOMAIN_MODEM**

MODEM, includes WiFi, Bluetooth and IEEE802.15.4.

enumerator **ESP_PD_DOMAIN_MAX**

Number of domains.

enum **esp_sleep_pd_option_t**

Power down options.

Values:

enumerator **ESP_PD_OPTION_OFF**

Power down the power domain in sleep mode.

enumerator **ESP_PD_OPTION_ON**

Keep power domain enabled during sleep mode.

enumerator **ESP_PD_OPTION_AUTO**

Keep power domain enabled in sleep mode, if it is needed by one of the wakeup options. Otherwise power it down.

enum **esp_sleep_source_t**

Sleep wakeup cause.

Values:

enumerator **ESP_SLEEP_WAKEUP_UNDEFINED**

In case of deep sleep, reset was not caused by exit from deep sleep.

enumerator **ESP_SLEEP_WAKEUP_ALL**

Not a wakeup cause, used to disable all wakeup sources with esp_sleep_disable_wakeup_source.

enumerator **ESP_SLEEP_WAKEUP_EXT0**

Wakeup caused by external signal using RTC_IO.

enumerator **ESP_SLEEP_WAKEUP_EXT1**

Wakeup caused by external signal using RTC_CNTL.

enumerator **ESP_SLEEP_WAKEUP_TIMER**

Wakeup caused by timer.

enumerator **ESP_SLEEP_WAKEUP_TOUCHPAD**

Wakeup caused by touchpad.

enumerator **ESP_SLEEP_WAKEUP_ULP**

Wakeup caused by ULP program.

enumerator **ESP_SLEEP_WAKEUP_GPIO**

Wakeup caused by GPIO (light sleep only on ESP32, S2 and S3)

enumerator **ESP_SLEEP_WAKEUP_UART**

Wakeup caused by UART (light sleep only)

enumerator **ESP_SLEEP_WAKEUP_WIFI**

Wakeup caused by WIFI (light sleep only)

enumerator **ESP_SLEEP_WAKEUP_COCPU**

Wakeup caused by COCPU int.

enumerator **ESP_SLEEP_WAKEUP_COCPU_TRAP_TRIG**

Wakeup caused by COCPU crash.

enumerator **ESP_SLEEP_WAKEUP_BT**

Wakeup caused by BT (light sleep only)

enum **esp_sleep_mode_t**

Sleep mode.

Values:

enumerator **ESP_SLEEP_MODE_LIGHT_SLEEP**

light sleep mode

enumerator **ESP_SLEEP_MODE_DEEP_SLEEP**

deep sleep mode

enum **[anonymous]**

Values:

enumerator **ESP_ERR_SLEEP_REJECT**

enumerator **ESP_ERR_SLEEP_TOO_SHORT_SLEEP_DURATION**

2.10.30 SoC Capabilities

This section lists the macro definitions of the ESP32-S3's SoC hardware capabilities. These macros are commonly used by conditional-compilation directives (e.g., `#if`) in ESP-IDF to determine which hardware-dependent features are supported, thus control what portions of code are compiled.

Warning: These macro definitions are currently not considered to be part of the public API, and may be changed in a breaking manner (see *ESP-IDF Versions* for more details).

API Reference

Header File

- components/soc/esp32s3/include/soc/soc_caps.h
- This header file can be included with:

```
#include "soc/soc_caps.h"
```

Macros

SOC_ADC_SUPPORTED

SOC_UART_SUPPORTED

SOC_PCNT_SUPPORTED

SOC_PHY_SUPPORTED

SOC_WIFI_SUPPORTED

SOC_TWAI_SUPPORTED

SOC_GDMA_SUPPORTED

SOC_AHB_GDMA_SUPPORTED

SOC_GPTIMER_SUPPORTED

SOC_LCDCAM_SUPPORTED

SOC_MCPWM_SUPPORTED

SOC_DEDICATED_GPIO_SUPPORTED

SOC_CACHE_SUPPORT_WRAP

SOC_ULP_SUPPORTED

SOC_ULP_FSM_SUPPORTED

SOC_RISCV_COPROC_SUPPORTED

SOC_BT_SUPPORTED

SOC_USB_OTG_SUPPORTED

SOC_USB_SERIAL_JTAG_SUPPORTED

SOC_CCOMP_TIMER_SUPPORTED

SOC_ASYNC_MEMCPY_SUPPORTED

SOC_SUPPORTS_SECURE_DL_MODE

SOC_EFUSE_KEY_PURPOSE_FIELD

SOC_EFUSE_SUPPORTED

SOC_SDMMC_HOST_SUPPORTED

SOC_RTC_FAST_MEM_SUPPORTED

SOC_RTC_SLOW_MEM_SUPPORTED

SOC_RTC_MEM_SUPPORTED

SOC_PSRAM_DMA_CAPABLE

SOC_XT_WDT_SUPPORTED

SOC_I2S_SUPPORTED

SOC_RMT_SUPPORTED

SOC_SDM_SUPPORTED

SOC_GPSPI_SUPPORTED

SOC_LEDC_SUPPORTED

SOC_I2C_SUPPORTED

SOC_SYSTIMER_SUPPORTED

SOC_SUPPORT_COEXISTENCE

SOC_TEMP_SENSOR_SUPPORTED

SOC_AES_SUPPORTED

SOC_MPI_SUPPORTED

SOC_SHA_SUPPORTED

SOC_HMAC_SUPPORTED

SOC_DIG_SIGN_SUPPORTED

SOC_FLASH_ENC_SUPPORTED

SOC_SECURE_BOOT_SUPPORTED

SOC_MEMPROT_SUPPORTED

SOC_TOUCH_SENSOR_SUPPORTED

SOC_BOD_SUPPORTED

SOC_CLK_TREE_SUPPORTED

SOC_MPU_SUPPORTED

SOC_WDT_SUPPORTED

SOC_SPI_FLASH_SUPPORTED

SOC_RNG_SUPPORTED

SOC_LIGHT_SLEEP_SUPPORTED

SOC_DEEP_SLEEP_SUPPORTED

SOC_LP_PERIPH_SHARE_INTERRUPT

SOC_PM_SUPPORTED

SOC_XTAL_SUPPORT_40M

SOC_APPCPU_HAS_CLOCK_GATING_BUG

SOC_ADC_RTC_CTRL_SUPPORTED

< SAR ADC Module

SOC_ADC_DIG_CTRL_SUPPORTED

SOC_ADC_ARBITER_SUPPORTED

SOC_ADC_DIG_IIR_FILTER_SUPPORTED

SOC_ADC_MONITOR_SUPPORTED

SOC_ADC_DMA_SUPPORTED

SOC_ADC_DIG_SUPPORTED_UNIT(UNIT)

SOC_ADC_PERIPH_NUM

SOC_ADC_CHANNEL_NUM(PERIPH_NUM)

SOC_ADC_MAX_CHANNEL_NUM

SOC_ADC_ATTEN_NUM

Digital

SOC_ADC_DIGI_CONTROLLER_NUM

SOC_ADC_PATT_LEN_MAX

SOC_ADC_DIGI_MIN_BITWIDTH

SOC_ADC_DIGI_MAX_BITWIDTH

SOC_ADC_DIGI_RESULT_BYTES

SOC_ADC_DIGI_DATA_BYTES_PER_CONV

SOC_ADC_DIGI_IIR_FILTER_NUM

SOC_ADC_DIGI_MONITOR_NUM

F_sample = F_digi_con / 2 / interval. F_digi_con = 5M for now. 30 <= interval<= 4095

SOC_ADC_SAMPLE_FREQ_THRES_HIGH

SOC_ADC_SAMPLE_FREQ_THRES_LOW

RTC

SOC_ADC_RTC_MIN_BITWIDTH

SOC_ADC_RTC_MAX_BITWIDTH

Calibration

SOC_ADC_CALIBRATION_V1_SUPPORTED

support HW offset calibration version 1

SOC_ADC_SELF_HW_CALI_SUPPORTED

support HW offset self calibration ADC power control is shared by PWDET, TempSensor

SOC_ADC_SHARED_POWER

SOC_APB_BACKUP_DMA

SOC_BROWNOUT_RESET_SUPPORTED

SOC_CACHE_WRITEBACK_SUPPORTED

SOC_CACHE_FREEZE_SUPPORTED

SOC_CPU_CORES_NUM

SOC_CPU_INTR_NUM

SOC_CPU_HAS_FPU

SOC_HP_CPU_HAS_MULTIPLE_CORES

SOC_CPU_BREAKPOINTS_NUM

SOC_CPU_WATCHPOINTS_NUM

SOC_CPU_WATCHPOINT_MAX_REGION_SIZE

SOC_DS_SIGNATURE_MAX_BIT_LEN

The maximum length of a Digital Signature in bits.

SOC_DS_KEY_PARAM_MD_IV_LENGTH

Initialization vector (IV) length for the RSA key parameter message digest (MD) in bytes.

SOC_DS_KEY_CHECK_MAX_WAIT_US

Maximum wait time for DS parameter decryption key. If overdue, then key error. See TRM DS chapter for more details

SOC_AHB_GDMA_VERSION

SOC_GDMA_NUM_GROUPS_MAX

SOC_GDMA_PAIRS_PER_GROUP

SOC_GDMA_PAIRS_PER_GROUP_MAX

SOC_AHB_GDMA_SUPPORT_PSRAM

SOC_GPIO_PORT

SOC_GPIO_PIN_COUNT

SOC_GPIO_SUPPORT_PIN_GLITCH_FILTER

SOC_GPIO_FILTER_CLK_SUPPORT_APB

SOC_GPIO_SUPPORT_RTC_INDEPENDENT

SOC_GPIO_SUPPORT_FORCE_HOLD

SOC_GPIO_VALID_GPIO_MASK

SOC_GPIO_VALID_OUTPUT_GPIO_MASK

SOC_GPIO_IN_RANGE_MAX

SOC_GPIO_OUT_RANGE_MAX

SOC_GPIO_VALID_DIGITAL_IO_PAD_MASK

SOC_GPIO_CLOCKOUT_BY_IO_MUX

SOC_GPIO_CLOCKOUT_CHANNEL_NUM

SOC_DEDIC_GPIO_OUT_CHANNELS_NUM
8 outward channels on each CPU core

SOC_DEDIC_GPIO_IN_CHANNELS_NUM
8 inward channels on each CPU core

SOC_DEDIC_GPIO_OUT_AUTO_ENABLE
Dedicated GPIO output attribution is enabled automatically

SOC_I2C_NUM

SOC_HP_I2C_NUM

SOC_I2C_FIFO_LEN
I2C hardware FIFO depth

SOC_I2C_CMD_REG_NUM

Number of I2C command registers

SOC_I2C_SUPPORT_SLAVE

SOC_I2C_SUPPORT_HW_CLR_BUS

SOC_I2C_SUPPORT_XTAL

SOC_I2C_SUPPORT_RTC

SOC_I2C_SUPPORT_10BIT_ADDR

SOC_I2C_SLAVE_SUPPORT_BROADCAST

SOC_I2C_SLAVE_SUPPORT_I2CRAM_ACCESS

SOC_I2S_NUM

SOC_I2S_HW_VERSION_2

SOC_I2S_SUPPORTS_XTAL

SOC_I2S_SUPPORTS_PLL_F160M

SOC_I2S_SUPPORTS_PCM

SOC_I2S_SUPPORTS_PDM

SOC_I2S_SUPPORTS_PDM_TX

SOC_I2S_PDM_MAX_TX_LINES

SOC_I2S_SUPPORTS_PDM_RX

SOC_I2S_PDM_MAX_RX_LINES

SOC_I2S_SUPPORTS_TDM

SOC_LEDC_SUPPORT_APB_CLOCK

SOC_LEDC_SUPPORT_XTAL_CLOCK

SOC_LEDC_CHANNEL_NUM

SOC_LEDC_TIMER_BIT_WIDTH

SOC_LEDC_SUPPORT_FADE_STOP

SOC_MCPWM_GROUPS

2 MCPWM groups on the chip (i.e., the number of independent MCPWM peripherals)

SOC_MCPWM_TIMERS_PER_GROUP

The number of timers that each group has.

SOC_MCPWM_OPERATORS_PER_GROUP

The number of operators that each group has.

SOC_MCPWM_COMPARATORS_PER_OPERATOR

The number of comparators that each operator has.

SOC_MCPWM_GENERATORS_PER_OPERATOR

The number of generators that each operator has.

SOC_MCPWM_TRIGGERS_PER_OPERATOR

The number of triggers that each operator has.

SOC_MCPWM_GPIO_FAULTS_PER_GROUP

The number of fault signal detectors that each group has.

SOC_MCPWM_CAPTURE_TIMERS_PER_GROUP

The number of capture timers that each group has.

SOC_MCPWM_CAPTURE_CHANNELS_PER_TIMER

The number of capture channels that each capture timer has.

SOC_MCPWM_GPIO_SYNCHROS_PER_GROUP

The number of GPIO synchros that each group has.

SOC_MCPWM_SWSYNC_CAN_PROPAGATE

Software sync event can be routed to its output.

SOC_MMU_LINEAR_ADDRESS_REGION_NUM

SOC_MMU_PERIPH_NUM

SOC_PCNT_GROUPS

SOC_PCNT_UNITS_PER_GROUP

SOC_PCNT_CHANNELS_PER_UNIT

SOC_PCNT_THRES_POINT_PER_UNIT

SOC_RMT_GROUPS

One RMT group

SOC_RMT_TX_CANDIDATES_PER_GROUP

Number of channels that capable of Transmit in each group

SOC_RMT_RX_CANDIDATES_PER_GROUP

Number of channels that capable of Receive in each group

SOC_RMT_CHANNELS_PER_GROUP

Total 8 channels

SOC_RMT_MEM_WORDS_PER_CHANNEL

Each channel owns 48 words memory (1 word = 4 Bytes)

SOC_RMT_SUPPORT_RX_PINGPONG

Support Ping-Pong mode on RX path

SOC_RMT_SUPPORT_RX_DEMODULATION

Support signal demodulation on RX path (i.e. remove carrier)

SOC_RMT_SUPPORT_TX_ASYNC_STOP

Support stop transmission asynchronously

SOC_RMT_SUPPORT_TX_LOOP_COUNT

Support transmit specified number of cycles in loop mode

SOC_RMT_SUPPORT_TX_LOOP_AUTO_STOP

Hardware support of auto-stop in loop mode

SOC_RMT_SUPPORT_TX_SYNCHRO

Support coordinate a group of TX channels to start simultaneously

SOC_RMT_SUPPORT_TX_CARRIER_DATA_ONLY

TX carrier can be modulated to data phase only

SOC_RMT_SUPPORT_XTAL

Support set XTAL clock as the RMT clock source

SOC_RMT_SUPPORT_RC_FAST

Support set RC_FAST clock as the RMT clock source

SOC_RMT_SUPPORT_APB

Support set APB as the RMT clock source

SOC_RMT_SUPPORT_DMA

RMT peripheral can connect to DMA channel

SOC_LCD_I80_SUPPORTED

Intel 8080 LCD is supported

SOC_LCD_RGB_SUPPORTED

RGB LCD is supported

SOC_LCD_I80_BUSES

Has one LCD Intel 8080 bus

SOC_LCD_RGB_PANELS

Support one RGB LCD panel

SOC_LCD_I80_BUS_WIDTH

Intel 8080 bus width

SOC_LCD_RGB_DATA_WIDTH

Number of LCD data lines

SOC_LCD_SUPPORT_RGB_YUV_CONV

Support color format conversion between RGB and YUV

SOC_RTC_CNTL_CPU_PD_DMA_BUS_WIDTH

SOC_RTC_CNTL_CPU_PD_REG_FILE_NUM

SOC_RTC_CNTL_CPU_PD_DMA_ADDR_ALIGN

SOC_RTC_CNTL_CPU_PD_DMA_BLOCK_SIZE

SOC_RTC_CNTL_CPU_PD_RETENTION_MEM_SIZE

SOC_RTC_CNTL_TAGMEM_PD_DMA_BUS_WIDTH

SOC_RTC_CNTL_TAGMEM_PD_DMA_ADDR_ALIGN

SOC_RTCIO_PIN_COUNT

SOC_RTCIO_INPUT_OUTPUT_SUPPORTED

SOC_RTCIO_HOLD_SUPPORTED

SOC_RTCIO_WAKE_SUPPORTED

SOC_SDM_GROUPS

SOC_SDM_CHANNELS_PER_GROUP

SOC_SDM_CLK_SUPPORT_APB

SOC_SPI_PERIPH_NUM

SOC_SPI_PERIPH_CS_NUM(i)

SOC_SPI_MAX_CS_NUM

SOC_SPI_MAXIMUM_BUFFER_SIZE

SOC_SPI_SUPPORT_DDRCLK

SOC_SPI_SLAVE_SUPPORT_SEG_TRANS

SOC_SPI_SUPPORT_CD_SIG

SOC_SPI_SUPPORT_CONTINUOUS_TRANS

SOC_SPI_SUPPORT_SLAVE_HD_VER2

SOC_SPI_SUPPORT_CLK_APB

SOC_SPI_SUPPORT_CLK_XTAL

SOC_SPI_PERIPH_SUPPORT_MULTILINE_MODE(host_id)

SOC_SPI_PERIPH_SUPPORT_CONTROL_DUMMY_OUT

SOC_MEMSPI_IS_INDEPENDENT

SOC_SPI_MAX_PRE_DIVIDER

SOC_SPI_SUPPORT_OCT

SOC_SPI_SCT_SUPPORTED

SOC_SPI_SCT_SUPPORTED_PERIPH(PERIPH_NUM)

SOC_SPI_SCT_REG_NUM

SOC_SPI_SCT_BUFFER_NUM_MAX

SOC_SPI_SCT_CONF_BITLEN_MAX

SOC_MEMSPI_SRC_FREQ_120M

SOC_MEMSPI_SRC_FREQ_80M_SUPPORTED

SOC_MEMSPI_SRC_FREQ_40M_SUPPORTED

SOC_MEMSPI_SRC_FREQ_20M_SUPPORTED

SOC_SPIRAM_SUPPORTED

SOC_SPIRAM_XIP_SUPPORTED

SOC_SYSTIMER_COUNTER_NUM

SOC_SYSTIMER_ALARM_NUM

SOC_SYSTIMER_BIT_WIDTH_LO

SOC_SYSTIMER_BIT_WIDTH_HI

SOC_SYSTIMER_FIXED_DIVIDER

SOC_SYSTIMER_INT_LEVEL

SOC_SYSTIMER_ALARM_MISS_COMPENSATE

SOC_TIMER_GROUPS

SOC_TIMER_GROUP_TIMERS_PER_GROUP

SOC_TIMER_GROUP_COUNTER_BIT_WIDTH

SOC_TIMER_GROUP_SUPPORT_XTAL

SOC_TIMER_GROUP_SUPPORT_APB

SOC_TIMER_GROUP_TOTAL_TIMERS

SOC_TOUCH_SENSOR_VERSION
: Hardware version of touch sensor

SOC_TOUCH_SENSOR_NUM
: 15 Touch channels

SOC_TOUCH_SUPPORT_SLEEP_WAKEUP
: Touch sensor supports sleep awake

SOC_TOUCH_SUPPORT_WATERPROOF
: Touch sensor supports waterproof

SOC_TOUCH_SUPPORT_PROX_SENSING

Touch sensor supports proximity sensing

SOC_TOUCH_PROXIMITY_CHANNEL_NUM

Support touch proximity sensing channel number.

SOC_TOUCH_PROXIMITY_MEAS_DONE_SUPPORTED

Support touch proximity sensing measure done interrupt type.

SOC_TOUCH_SAMPLE_CFG_NUM

The sample configuration number in total, each sampler can be used to sample on one frequency

SOC_TWAI_CONTROLLER_NUM

SOC_TWAI_CLK_SUPPORT_APB

SOC_TWAI_BRP_MIN

SOC_TWAI_BRP_MAX

SOC_TWAI_SUPPORTS_RX_STATUS

SOC_UART_NUM

SOC_UART_HP_NUM

SOC_UART_FIFO_LEN

The UART hardware FIFO length

SOC_UART_BITRATE_MAX

Max bit rate supported by UART

SOC_UART_SUPPORT_FSM_TX_WAIT_SEND

SOC_UART_SUPPORT_WAKEUP_INT

Support UART wakeup interrupt

SOC_UART_SUPPORT_APB_CLK

Support APB as the clock source

SOC_UART_SUPPORT_RTC_CLK

Support RTC clock as the clock source

SOC_UART_SUPPORT_XTAL_CLK

Support XTAL clock as the clock source

SOC_USB_OTG_PERIPH_NUM

SOC_SHA_DMA_MAX_BUFFER_SIZE

SOC_SHA_SUPPORT_DMA

SOC_SHA_SUPPORT_RESUME

SOC_SHA_GDMA

SOC_SHA_SUPPORT_SHA1

SOC_SHA_SUPPORT_SHA224

SOC_SHA_SUPPORT_SHA256

SOC_SHA_SUPPORT_SHA384

SOC_SHA_SUPPORT_SHA512

SOC_SHA_SUPPORT_SHA512_224

SOC_SHA_SUPPORT_SHA512_256

SOC_SHA_SUPPORT_SHA512_T

SOC_MPI_MEM_BLOCKS_NUM

SOC_MPI_OPERATIONS_NUM

SOC_RSA_MAX_BIT_LEN

SOC_AES_SUPPORT_DMA

SOC_AES_GDMA

SOC_AES_SUPPORT_AES_128

SOC_AES_SUPPORT_AES_256

SOC_PM_SUPPORT_EXT0_WAKEUP

SOC_PM_SUPPORT_EXT1_WAKEUP

SOC_PM_SUPPORT_EXT_WAKEUP

Compatible to the old version of IDF

SOC_PM_SUPPORT_WIFI_WAKEUP

SOC_PM_SUPPORT_BT_WAKEUP

SOC_PM_SUPPORT_TOUCH_SENSOR_WAKEUP
Supports waking up from touch pad trigger

SOC_PM_SUPPORT_CPU_PD

SOC_PM_SUPPORT_TAGMEM_PD

SOC_PM_SUPPORT_RTC_PERIPH_PD

SOC_PM_SUPPORT_RC_FAST_PD

SOC_PM_SUPPORT_VDDSDIO_PD

SOC_PM_SUPPORT_MAC_BB_PD

SOC_PM_SUPPORT_MODEM_PD
Modem here includes wifi and ble

SOC_CONFIGURABLE_VDDSDIO_SUPPORTED

SOC_PM_SUPPORT_DEEPSLEEP_CHECK_STUB_ONLY
Supports CRC only the stub code in RTC memory

SOC_PM_CPU_RETENTION_BY_RTCCNTL

SOC_PM_MODEM_RETENTION_BY_BACKUPDMA

SOC_CLK_RC_FAST_D256_SUPPORTED

SOC_RTC_SLOW_CLK_SUPPORT_RC_FAST_D256

SOC_CLK_RC_FAST_SUPPORT_CALIBRATION

SOC_CLK_XTAL32K_SUPPORTED
Support to connect an external low frequency crystal

SOC_EFUSE_DIS_DOWNLOAD_ICACHE

SOC_EFUSE_DIS_DOWNLOAD_DCACHE

SOC_EFUSE_HARD_DIS_JTAG

SOC_EFUSE_DIS_USB_JTAG

SOC_EFUSE_SOFT_DIS_JTAG

SOC_EFUSE_DIS_DIRECT_BOOT

SOC_EFUSE_DIS_ICACHE

SOC_EFUSE_BLOCK9_KEY_PURPOSE_QUIRK

SOC_SECURE_BOOT_V2_RSA

SOC_EFUSE_SECURE_BOOT_KEY_DIGESTS

SOC_EFUSE_REVOKE_BOOT_KEY_DIGESTS

SOC_SUPPORT_SECURE_BOOT_REVOKE_KEY

SOC_FLASH_ENCRYPTED_XTS_AES_BLOCK_MAX

SOC_FLASH_ENCRYPTION_XTS_AES

SOC_FLASH_ENCRYPTION_XTS_AES_OPTIONS

SOC_FLASH_ENCRYPTION_XTS_AES_128

SOC_FLASH_ENCRYPTION_XTS_AES_256

SOC_MEMPROT_CPU_PREFETCH_PAD_SIZE

SOC_MEMPROT_MEM_ALIGN_SIZE

SOC_PHY_DIG_REGS_MEM_SIZE

SOC_MAC_BB_PD_MEM_SIZE

SOC_WIFI_LIGHT_SLEEP_CLK_WIDTH

SOC_SPI_MEM_SUPPORT_AUTO_WAIT_IDLE

SOC_SPI_MEM_SUPPORT_AUTO_SUSPEND

SOC_SPI_MEM_SUPPORT_AUTO_RESUME

SOC_SPI_MEM_SUPPORT_SW_SUSPEND

SOC_SPI_MEM_SUPPORT_OPI_MODE

SOC_SPI_MEM_SUPPORT_TIMING_TUNING

SOC_SPI_MEM_SUPPORT_CONFIG_GPIO_BY_EFUSE

SOC_SPI_MEM_SUPPORT_WRAP

SOC_MEMSPI_TIMING_TUNING_BY_MSPI_DELAY

SOC_MEMSPI_CORE_CLK_SHARED_WITH_PSRAM

SOC_COEX_HW_PTI

SOC_EXTERNAL_COEX_ADVANCE
HARDWARE ADVANCED EXTERNAL COEXISTENCE CAPS

SOC_EXTERNAL_COEX_LEADER_TX_LINE
EXTERNAL COEXISTENCE TX LINE CAPS

SOC_SDMMC_USE_GPIO_MATRIX

SOC_SDMMC_NUM_SLOTS

SOC_SDMMC_SUPPORT_XTAL_CLOCK

SOC_SDMMC_DELAY_PHASE_NUM

SOC_TEMPERATURE_SENSOR_SUPPORT_FAST_RC

SOC_WIFI_HW_TSF
Support hardware TSF

SOC_WIFI_FTM_SUPPORT
Support FTM

SOC_WIFI_GCMP_SUPPORT
Support GCMP(GCMP128 and GCMP256)

SOC_WIFI_WAPI_SUPPORT
Support WAPI

SOC_WIFI_CSI_SUPPORT
Support CSI

SOC_WIFI_MESH_SUPPORT
Support WIFI MESH

SOC_WIFI_SUPPORT_VARIABLE_BEACON_WINDOW

Support delta early time for rf phy on/off

SOC_WIFI_PHY_NEEDS_USB_WORKAROUND

SoC has WiFi and USB PHYs interference, needs a workaround

SOC_BLE_SUPPORTED

Support Bluetooth Low Energy hardware

SOC_BLE_MESH_SUPPORTED

Support BLE MESH

SOC_BLE_50_SUPPORTED

Support Bluetooth 5.0

SOC_BLE_DEVICE_PRIVACY_SUPPORTED

Support BLE device privacy mode

SOC_BLUFI_SUPPORTED

Support BLUFI

SOC_ULP_HAS_ADC

SOC_PHY_COMBO_MODULE

Support Wi-Fi and BLE

2.10.31 System Time

Overview

ESP32-S3 uses two hardware timers for the purpose of keeping system time. System time can be kept by using either one or both of the hardware timers depending on the application's purpose and accuracy requirements for system time. The two hardware timers are:

- **RTC timer**: This timer allows time keeping in various sleep modes, and can also persist time keeping across any resets (with the exception of power-on resets which reset the RTC timer). The frequency deviation depends on the *RTC Timer Clock Sources* and affects the accuracy only in sleep modes, in which case the time will be measured at 6.6667 µs resolution.
- **High-resolution timer**: This timer is not available in sleep modes and will not persist over a reset, but has greater accuracy. The timer uses the APB_CLK clock source (typically 80 MHz), which has a frequency deviation of less than ±10 ppm. Time will be measured at 1 µs resolution.

The possible combinations of hardware timers used to keep system time are listed below:

- RTC and high-resolution timer (default)
- RTC
- High-resolution timer
- None

It is recommended that users stick to the default option as it provides the highest accuracy. However, users can also select a different setting via the *CONFIG_NEWLIB_TIME_SYSCALL* configuration option.

RTC Timer Clock Sources

The RTC timer has the following clock sources:

- `Internal 136 kHz RC oscillator` (default): Features the lowest Deep-sleep current consumption and no dependence on any external components. However, the frequency stability of this clock source is affected by temperature fluctuations, so time may drift in both Deep-sleep and Light-sleep modes.
- `External 32 kHz crystal`: Requires a 32 kHz crystal to be connected to the XTAL_32K_P and XTAL_32K_N pins. This source provides a better frequency stability at the expense of a slightly higher (by 1 µA) Deep-sleep current consumption.
- `External 32 kHz oscillator at XTAL_32K_P pin`: Allows using 32 kHz clock generated by an external circuit. The external clock signal must be connected to the XTAL_32K_P pin. The amplitude should be less than 1.2 V for sine wave signal and less than 1 V for square wave signal. Common mode voltage should be in the range of 0.1 < Vcm < 0.5xVamp, where Vamp stands for signal amplitude. In this case, the XTAL_32K_P pin cannot be used as a GPIO pin.
- `Internal 17.5 MHz oscillator, divided by 256 (about 68 kHz)`: Provides better frequency stability than the `Internal 136 kHz RC oscillator` at the expense of a higher (by 5 µA) Deep-sleep current consumption. It also does not require external components.

The choice depends on your requirements for system time accuracy and power consumption in sleep modes. To modify the RTC clock source, set *CONFIG_RTC_CLK_SRC* in project configuration.

More details about the wiring requirements for the external crystal or external oscillator, please refer to the Hardware Design Guidelines.

Get Current Time

To get the current time, use the POSIX function `gettimeofday()`. Additionally, you can use the following standard C library functions to obtain time and manipulate it:

```
gettimeofday
time
asctime
clock
ctime
difftime
gmtime
localtime
mktime
strftime
adjtime*
```

To stop smooth time adjustment and update the current time immediately, use the POSIX function `settimeofday()`.

If you need to obtain time with one second resolution, use the following code snippet:

```
time_t now;
char strftime_buf[64];
struct tm timeinfo;

time(&now);
// Set timezone to China Standard Time
setenv("TZ", "CST-8", 1);
tzset();

localtime_r(&now, &timeinfo);
strftime(strftime_buf, sizeof(strftime_buf), "%c", &timeinfo);
ESP_LOGI(TAG, "The current date/time in Shanghai is: %s", strftime_buf);
```

If you need to obtain time with one microsecond resolution, use the code snippet below:

```
struct timeval tv_now;
gettimeofday(&tv_now, NULL);
int64_t time_us = (int64_t)tv_now.tv_sec * 1000000L + (int64_t)tv_now.tv_usec;
```

SNTP Time Synchronization

To set the current time, you can use the POSIX functions `settimeofday()` and `adjtime()`. They are used internally in the lwIP SNTP library to set current time when a response from the NTP server is received. These functions can also be used separately from the lwIP SNTP library.

Some lwIP APIs, including SNTP functions, are not thread safe, so it is recommended to use *esp_netif component* when interacting with SNTP module.

To initialize a particular SNTP server and also start the SNTP service, simply create a default SNTP server configuration with a particular server name, then call *esp_netif_sntp_init()* to register that server and start the SNTP service.

```
esp_sntp_config_t config = ESP_NETIF_SNTP_DEFAULT_CONFIG("pool.ntp.org");
esp_netif_sntp_init(&config);
```

This code automatically performs time synchronization once a reply from the SNTP server is received. Sometimes it is useful to wait until the time gets synchronized, *esp_netif_sntp_sync_wait()* can be used for this purpose:

```
if (esp_netif_sntp_sync_wait(pdMS_TO_TICKS(10000)) != ESP_OK) {
    printf("Failed to update system time within 10s timeout");
}
```

To configure multiple NTP servers (or use more advanced settings, such as DHCP provided NTP servers), please refer to the detailed description of *SNTP API* in *esp_netif* documentation.

The lwIP SNTP library could work in one of the following sync modes:

- *SNTP_SYNC_MODE_IMMED* (default): Updates system time immediately upon receiving a response from the SNTP server after using `settimeofday()`.
- *SNTP_SYNC_MODE_SMOOTH*: Updates time smoothly by gradually reducing time error using the function `adjtime()`. If the difference between the SNTP response time and system time is more than 35 minutes, update system time immediately by using `settimeofday()`.

If you want to choose the *SNTP_SYNC_MODE_SMOOTH* mode, please set the `esp_sntp_config::smooth` to `true` in the SNTP configuration struct. Otherwise (and by default) the *SNTP_SYNC_MODE_IMMED* mode will be used.

For setting a callback function that is called when time gets synchronized, use the *esp_sntp_config::sync_cb* field in the configuration struct.

An application with this initialization code periodically synchronizes the time. The time synchronization period is determined by *CONFIG_LWIP_SNTP_UPDATE_DELAY* (the default value is one hour). To modify the variable, set *CONFIG_LWIP_SNTP_UPDATE_DELAY* in project configuration.

A code example that demonstrates the implementation of time synchronization based on the lwIP SNTP library is provided in the protocols/sntp directory.

Note that it is also possible to use lwIP API directly, but care must be taken to thread safety. Here we list the thread-safe APIs:

- *sntp_set_time_sync_notification_cb()* can be used to set a callback function that notifies of the time synchronization process.
- *sntp_get_sync_status()* and *sntp_set_sync_status()* can be used to get/set time synchronization status.
- *sntp_set_sync_mode()* can be used to set the synchronization mode.

- *esp_sntp_setoperatingmode()* sets the preferred operating mode.:cpp:enumerator:*ESP_SNTP_OPMODE_POLL* and *esp_sntp_init()* initializes SNTP module.
- *esp_sntp_setservername()* configures one SNTP server.

Timezones

To set the local timezone, use the following POSIX functions:

1. Call `setenv()` to set the `TZ` environment variable to the correct value based on the device location. The format of the time string is the same as described in the GNU libc documentation (although the implementation is different).
2. Call `tzset()` to update C library runtime data for the new timezone.

Once these steps are completed, call the standard C library function `localtime()`, and it returns the correct local time taking into account the timezone offset and daylight saving time.

Year 2036 and 2038 Overflow Issues

SNTP/NTP 2036 Overflow SNTP/NTP timestamps are represented as 64-bit unsigned fixed point numbers, where the first 32 bits represent the integer part, and the last 32 bits represent the fractional part. The 64-bit unsigned fixed point number represents the number of seconds since 00:00 on 1st of January 1900, thus SNTP/NTP times will overflow in the year 2036.

To address this issue, lifetime of the SNTP/NTP timestamps has been extended by convention by using the MSB (bit 0 by convention) of the integer part to indicate time ranges between years 1968 to 2104 (see RFC2030 for more details). This convention is implemented in lwIP library SNTP module. Therefore SNTP-related functions in ESP-IDF are future-proof until year 2104.

Unix Time 2038 Overflow Unix time (type `time_t`) was previously represented as a 32-bit signed integer, leading to an overflow in year 2038 (i.e., Y2K38 issue). To address the Y2K38 issue, ESP-IDF uses a 64-bit signed integer to represent `time_t` starting from release v5.0, thus deferring `time_t` overflow for another 292 billion years.

API Reference

Header File

- components/lwip/include/apps/esp_sntp.h
- This header file can be included with:

```
#include "esp_sntp.h"
```

- This header file is a part of the API provided by the `lwip` component. To declare that your component depends on `lwip`, add the following to your CMakeLists.txt:

```
REQUIRES lwip
```

or

```
PRIV_REQUIRES lwip
```

Functions

void **sntp_sync_time** (struct timeval *tv)

This function updates the system time.

This is a weak-linked function. It is possible to replace all SNTP update functionality by placing a sntp_sync_time() function in the app firmware source. If the default implementation is used, calling sntp_set_sync_mode() allows the time synchronization mode to be changed to instant or smooth. If a callback

function is registered via sntp_set_time_sync_notification_cb(), it will be called following time synchronization.

Parameters **tv** -- Time received from SNTP server.

void **sntp_set_sync_mode**(*sntp_sync_mode_t* sync_mode)

Set the sync mode.

Modes allowed: SNTP_SYNC_MODE_IMMED and SNTP_SYNC_MODE_SMOOTH.

Parameters **sync_mode** -- Sync mode.

sntp_sync_mode_t **sntp_get_sync_mode**(void)

Get set sync mode.

Returns SNTP_SYNC_MODE_IMMED: Update time immediately. SNTP_SYNC_MODE_SMOOTH: Smooth time updating.

sntp_sync_status_t **sntp_get_sync_status**(void)

Get status of time sync.

After the update is completed, the status will be returned as SNTP_SYNC_STATUS_COMPLETED. After that, the status will be reset to SNTP_SYNC_STATUS_RESET. If the update operation is not completed yet, the status will be SNTP_SYNC_STATUS_RESET. If a smooth mode was chosen and the synchronization is still continuing (adjtime works), then it will be SNTP_SYNC_STATUS_IN_PROGRESS.

Returns SNTP_SYNC_STATUS_RESET: Reset status. SNTP_SYNC_STATUS_COMPLETED: Time is synchronized. SNTP_SYNC_STATUS_IN_PROGRESS: Smooth time sync in progress.

void **sntp_set_sync_status**(*sntp_sync_status_t* sync_status)

Set status of time sync.

Parameters **sync_status** -- status of time sync (see sntp_sync_status_t)

void **sntp_set_time_sync_notification_cb**(*sntp_sync_time_cb_t* callback)

Set a callback function for time synchronization notification.

Parameters **callback** -- a callback function

void **sntp_set_sync_interval**(uint32_t interval_ms)

Set the sync interval of SNTP operation.

Note: SNTPv4 RFC 4330 enforces a minimum sync interval of 15 seconds. This sync interval will be used in the next attempt update time throught SNTP. To apply the new sync interval call the sntp_restart() function, otherwise, it will be applied after the last interval expired.

Parameters **interval_ms** -- The sync interval in ms. It cannot be lower than 15 seconds, otherwise 15 seconds will be set.

uint32_t **sntp_get_sync_interval**(void)

Get the sync interval of SNTP operation.

Returns the sync interval

bool **sntp_restart**(void)

Restart SNTP.

Returns True - Restart False - SNTP was not initialized yet

void **esp_sntp_setoperatingmode**(*esp_sntp_operatingmode_t* operating_mode)

Sets SNTP operating mode. The mode has to be set before init.

Parameters **operating_mode** -- Desired operating mode

void **esp_sntp_init**(void)

Init and start SNTP service.

void **esp_sntp_stop**(void)

Stops SNTP service.

void **esp_sntp_setserver**(u8_t idx, const ip_addr_t *addr)

Sets SNTP server address.

Parameters
- **idx** -- Index of the server
- **addr** -- IP address of the server

void **esp_sntp_setservername**(u8_t idx, const char *server)

Sets SNTP hostname.

Parameters
- **idx** -- Index of the server
- **server** -- Name of the server

const char ***esp_sntp_getservername**(u8_t idx)

Gets SNTP server name.

Parameters **idx** -- Index of the server
Returns Name of the server

const ip_addr_t ***esp_sntp_getserver**(u8_t idx)

Get SNTP server IP.

Parameters **idx** -- Index of the server
Returns IP address of the server

bool **esp_sntp_enabled**(void)

Checks if sntp is enabled.

Returns true if sntp module is enabled

uint8_t **esp_sntp_getreachability**(uint8_t idx)

Gets the server reachability shift register as described in RFC 5905.

Parameters **idx** -- Index of the SNTP server
Returns reachability shift register

esp_sntp_operatingmode_t **esp_sntp_getoperatingmode**(void)

Get the configured operating mode.

Returns operating mode enum

static inline void **sntp_setoperatingmode**(u8_t operating_mode)

if not build within lwip, provide translating inlines, that will warn about thread safety

static inline void **sntp_servermode_dhcp**(int set_servers_from_dhcp)

static inline void **sntp_setservername**(u8_t idx, const char *server)

static inline void **sntp_init**(void)

static inline const char ***sntp_getservername**(u8_t idx)

static inline const ip_addr_t ***sntp_getserver**(u8_t idx)

static inline uint8_t **sntp_getreachability**(uint8_t idx)

static inline *esp_sntp_operatingmode_t* **sntp_getoperatingmode**(void)

Macros

esp_sntp_sync_time

Aliases for esp_sntp prefixed API (inherently thread safe)

esp_sntp_set_sync_mode

esp_sntp_get_sync_mode

esp_sntp_get_sync_status

esp_sntp_set_sync_status

esp_sntp_set_time_sync_notification_cb

esp_sntp_set_sync_interval

esp_sntp_get_sync_interval

esp_sntp_restart

SNTP_OPMODE_POLL

Type Definitions

typedef void (***sntp_sync_time_cb_t**)(struct timeval *tv)

SNTP callback function for notifying about time sync event.

Param tv Time received from SNTP server.

Enumerations

enum **sntp_sync_mode_t**

SNTP time update mode.

Values:

enumerator **SNTP_SYNC_MODE_IMMED**

Update system time immediately when receiving a response from the SNTP server.

enumerator **SNTP_SYNC_MODE_SMOOTH**

Smooth time updating. Time error is gradually reduced using adjtime function. If the difference between SNTP response time and system time is large (more than 35 minutes) then update immediately.

enum **sntp_sync_status_t**

SNTP sync status.

Values:

enumerator **SNTP_SYNC_STATUS_RESET**

enumerator **SNTP_SYNC_STATUS_COMPLETED**

enumerator **SNTP_SYNC_STATUS_IN_PROGRESS**

enum **esp_sntp_operatingmode_t**

SNTP operating modes per lwip SNTP module.

Values:

enumerator **ESP_SNTP_OPMODE_POLL**

enumerator **ESP_SNTP_OPMODE_LISTENONLY**

2.10.32 Asynchronous Memory Copy

Overview

ESP32-S3 has a DMA engine which can help to offload internal memory copy operations from the CPU in an asynchronous way.

The async memcpy API wraps all DMA configurations and operations. The signature of *esp_async_memcpy()* is almost the same as the standard libc `memcpy` function.

The DMA allows multiple memory copy requests to be queued up before the first one is completed, which allows overlap of computation and memory copy. Moreover, it is still possible to know the exact time when a memory copy request is completed by registering an event callback.

If the async memcpy is constructed upon the AHB GDMA, it is also possible to copy data from/to PSRAM with a proper alignment.

Configure and Install Driver

There are several ways to install the async memcpy driver, depending on the underlying DMA engine:

- *esp_async_memcpy_install_gdma_ahb()* is used to install the async memcpy driver based on the AHB GDMA engine.
- *esp_async_memcpy_install()* is a generic API to install the async memcpy driver with a default DMA engine. If the SoC has the CP DMA engine, the default DMA engine is CP DMA. Otherwise, the default DMA engine is AHB GDMA.

Driver configuration is described in *async_memcpy_config_t*:

- `backlog`: This is used to configure the maximum number of memory copy transactions that can be queued up before the first one is completed. If this field is set to zero, then the default value 4 will be applied.
- `dma_burst_size`: Set the burst size in a DMA burst transfer.
- `flags`: This is used to enable some special driver features.

```
async_memcpy_config_t config = ASYNC_MEMCPY_DEFAULT_CONFIG();
// update the maximum data stream supported by underlying DMA engine
config.backlog = 8;
async_memcpy_handle_t driver = NULL;
ESP_ERROR_CHECK(esp_async_memcpy_install(&config, &driver)); // install driver␣
↪with default DMA engine
```

Send Memory Copy Request

esp_async_memcpy() is the API to send memory copy request to DMA engine. It must be called after driver is installed successfully. This API is thread safe, so it can be called from different tasks.

Different from the libc version of `memcpy`, you can optionally pass a callback to *esp_async_memcpy()*, so that you can be notified when the memory copy is finished. Note that the callback is executed in the ISR context, please make sure you will not call any blocking functions in the callback.

The prototype of the callback function is *async_memcpy_isr_cb_t*. The callback function should only return true if it wakes up a high priority task by RTOS APIs like `xSemaphoreGiveFromISR()`.

```
// Callback implementation, running in ISR context
static bool my_async_memcpy_cb(async_memcpy_handle_t mcp_hdl, async_memcpy_event_t␣
↪*event, void *cb_args)
{
    SemaphoreHandle_t sem = (SemaphoreHandle_t)cb_args;
    BaseType_t high_task_wakeup = pdFALSE;
    xSemaphoreGiveFromISR(semphr, &high_task_wakeup); // high_task_wakeup set to␣
↪pdTRUE if some high priority task unblocked
    return high_task_wakeup == pdTRUE;
}

// Create a semaphore used to report the completion of async memcpy
SemaphoreHandle_t semphr = xSemaphoreCreateBinary();

// Called from user's context
ESP_ERROR_CHECK(esp_async_memcpy(driver_handle, to, from, copy_len, my_async_
↪memcpy_cb, my_semaphore));
// Do something else here
xSemaphoreTake(my_semaphore, portMAX_DELAY); // Wait until the buffer copy is done
```

Uninstall Driver

esp_async_memcpy_uninstall() is used to uninstall asynchronous memcpy driver. It is not necessary to uninstall the driver after each memcpy operation. If you know your application will not use this driver anymore, then this API can recycle the memory and other hardware resources for you.

API Reference

Header File

- components/esp_hw_support/include/esp_async_memcpy.h
- This header file can be included with:

```
#include "esp_async_memcpy.h"
```

Functions

esp_err_t **esp_async_memcpy_install_gdma_ahb**(const *async_memcpy_config_t* *config, *async_memcpy_handle_t* *mcp)

Install async memcpy driver, with AHB-GDMA as the backend.

Parameters

- **config** -- **[in]** Configuration of async memcpy
- **mcp** -- **[out]** Returned driver handle

Returns

- ESP_OK: Install async memcpy driver successfully
- ESP_ERR_INVALID_ARG: Install async memcpy driver failed because of invalid argument

- ESP_ERR_NO_MEM: Install async memcpy driver failed because out of memory
- ESP_FAIL: Install async memcpy driver failed because of other error

esp_err_t **esp_async_memcpy_install** (const *async_memcpy_config_t* *config, *async_memcpy_handle_t* *mcp)

Install async memcpy driver with the default DMA backend.

Note: On chip with CPDMA support, CPDMA is the default choice. On chip with AHB-GDMA support, AHB-GDMA is the default choice.

Parameters

- **config** -- **[in]** Configuration of async memcpy
- **mcp** -- **[out]** Returned driver handle

Returns

- ESP_OK: Install async memcpy driver successfully
- ESP_ERR_INVALID_ARG: Install async memcpy driver failed because of invalid argument
- ESP_ERR_NO_MEM: Install async memcpy driver failed because out of memory
- ESP_FAIL: Install async memcpy driver failed because of other error

esp_err_t **esp_async_memcpy_uninstall** (*async_memcpy_handle_t* mcp)

Uninstall async memcpy driver.

Parameters **mcp** -- **[in]** Handle of async memcpy driver that returned from `esp_async_memcpy_install`

Returns

- ESP_OK: Uninstall async memcpy driver successfully
- ESP_ERR_INVALID_ARG: Uninstall async memcpy driver failed because of invalid argument
- ESP_FAIL: Uninstall async memcpy driver failed because of other error

esp_err_t **esp_async_memcpy** (*async_memcpy_handle_t* mcp, void *dst, void *src, size_t n, *async_memcpy_isr_cb_t* cb_isr, void *cb_args)

Send an asynchronous memory copy request.

Note: The callback function is invoked in interrupt context, never do blocking jobs in the callback.

Parameters

- **mcp** -- **[in]** Handle of async memcpy driver that returned from `esp_async_memcpy_install`
- **dst** -- **[in]** Destination address (copy to)
- **src** -- **[in]** Source address (copy from)
- **n** -- **[in]** Number of bytes to copy
- **cb_isr** -- **[in]** Callback function, which got invoked in interrupt context. Set to NULL can bypass the callback.
- **cb_args** -- **[in]** User defined argument to be passed to the callback function

Returns

- ESP_OK: Send memory copy request successfully
- ESP_ERR_INVALID_ARG: Send memory copy request failed because of invalid argument
- ESP_FAIL: Send memory copy request failed because of other error

Structures

struct **async_memcpy_event_t**

Async memory copy event data.

Public Members

void ***data**

Event data

struct **async_memcpy_config_t**

Type of async memcpy configuration.

Public Members

uint32_t **backlog**

Maximum number of transactions that can be prepared in the background

size_t **sram_trans_align**

DMA transfer alignment (both in size and address) for SRAM memory

size_t **psram_trans_align**

DMA transfer alignment (both in size and address) for PSRAM memory

size_t **dma_burst_size**

DMA transfer burst size, in bytes

uint32_t **flags**

Extra flags to control async memcpy feature

Macros

ASYNC_MEMCPY_DEFAULT_CONFIG()

Default configuration for async memcpy.

Type Definitions

typedef struct async_memcpy_context_t ***async_memcpy_handle_t**

Async memory copy driver handle.

typedef bool (***async_memcpy_isr_cb_t**)(*async_memcpy_handle_t* mcp_hdl, *async_memcpy_event_t* *event, void *cb_args)

Type of async memcpy interrupt callback function.

Note: User can call OS primitives (semaphore, mutex, etc) in the callback function. Keep in mind, if any OS primitive wakes high priority task up, the callback should return true.

Param mcp_hdl Handle of async memcpy
Param event Event object, which contains related data, reserved for future
Param cb_args User defined arguments, passed from esp_async_memcpy function
Return Whether a high priority task is woken up by the callback function

2.10.33 ULP Coprocessor Programming

The Ultra Low Power (ULP) coprocessor is a simple finite state machine (FSM) which is designed to perform measurements using the ADC, temperature sensor, and external I2C sensors, while the main processors are in Deep-sleep mode. The ULP coprocessor can access the `RTC_SLOW_MEM` memory region, and registers in the `RTC_CNTL`, `RTC_IO`, and `SARADC` peripherals. The ULP coprocessor uses fixed-width 32-bit instructions, 32-bit memory addressing, and has 4 general-purpose 16-bit registers. This coprocessor is referred to as `ULP FSM` in ESP-IDF.

ESP32-S3 provides a second type of ULP coprocessor which is based on a RISC-V instruction set architecture. For details regarding *ULP RISC-V* refer *ULP-RISC-V Coprocessor*.

Installing the Toolchain

The ULP FSM coprocessor code is written in assembly and compiled using the binutils-esp32ulp toolchain.

If you have already set up ESP-IDF with CMake build system according to the *Getting Started Guide*, then the ULP FSM toolchain will already be installed.

Programming ULP FSM

The ULP FSM can be programmed using the supported instruction set. Alternatively, the ULP FSM coprocessor can also be programmed using C Macros on the main CPU. Theses two methods are described in the following section:

ESP32-S3 ULP Coprocessor Instruction Set

This document provides details about the instructions used by ESP32-S3 ULP FSM coprocessor assembler.

ULP FSM coprocessor has 4 16-bit general purpose registers, labeled R0, R1, R2, R3. It also has an 8-bit counter register (`stage_cnt`) which can be used to implement loops. Stage count register is accessed using special instructions.

ULP coprocessor can access 8 K bytes of RTC_SLOW_MEM memory region. Memory is addressed in 32-bit word units. It can also access peripheral registers in `RTC_CNTL`, `RTC_IO`, and `SENS` peripherals.

All instructions are 32-bit. Jump instructions, ALU instructions, peripheral register and memory access instructions are executed in 1 cycle. Instructions which work with peripherals (TSENS, ADC, I2C) take variable number of cycles, depending on peripheral operation.

The instruction syntax is case insensitive. Upper and lower case letters can be used and intermixed arbitrarily. This is true both for register names and instruction names.

Note About Addressing ESP32-S3 ULP FSM coprocessor's `JUMP`, `ST`, `LD` family of instructions expect the address argument to be expressed in the following way depending on the type of address argument used:

- When the address argument is presented as a label then the instruction expects the address to be expressed as 32-bit words.
  Consider the following example program:

```
entry:
        NOP
        NOP
        NOP
        NOP
loop:
        MOVE R1, loop
        JUMP R1
```

When this program is assembled and linked, address of label `loop` will be equal to 16 (expressed in bytes). However `JUMP` instruction expects the address stored in register `R1` to be expressed in 32-bit words. To account for this common use case, the assembler will convert the address of label `loop` from bytes to words when generating the `MOVE` instruction. Hence, the code generated code will be equivalent to:

```
0000    NOP
0004    NOP
0008    NOP
000c    NOP
0010    MOVE R1, 4
0014    JUMP R1
```

- The other case is when the argument of `MOVE` instruction is not a label but a constant. In this case, assembler will **use the value as is**, without any conversion:

```
.set        val, 0x10
MOVE        R1, val
```

In this case, value loaded into `R1` will be `0x10`.
However, when an immediate value is used as an offset in `LD` and `ST` instructions, the assembler considers the address argument in bytes and converts it to 32-bit words before executing the instruction:

```
ST R1, R2, 4        // offset = 4 bytes; Mem[R2 + 4 / 4] = R1
```

In this case, the value in R1 is stored at the memory location pointed by [R2 + offset / 4].
Consider the following code:

```
        .global array
array:  .long 0
        .long 0
        .long 0
        .long 0

        MOVE R1, array
        MOVE R2, 0x1234
        ST R2, R1, 0        // write value of R2 into the first array element,
                            // i.e., array[0]

        ST R2, R1, 4        // write value of R2 into the second array element,
                            // (4 byte offset), i.e., array[1]

        ADD R1, R1, 2       // this increments address by 2 words (8 bytes)
        ST R2, R1, 0        // write value of R2 into the third array element,
                            // i.e., array[2]
```

Note About Instruction Execution Time ULP coprocessor is clocked from `RTC_FAST_CLK`, which is normally derived from the internal 8 MHz oscillator. Applications which need to know exact ULP clock frequency can calibrate it against the main XTAL clock:

```
#include "soc/rtc.h"

// calibrate 8M/256 clock against XTAL, get 8M/256 clock period
uint32_t rtc_8md256_period = rtc_clk_cal(RTC_CAL_8MD256, 100);
uint32_t rtc_fast_freq_hz = 1000000ULL * (1 << RTC_CLK_CAL_FRACT) * 256 / rtc_
↪8md256_period;
```

ULP coprocessor needs certain number of clock cycles to fetch each instruction, plus certain number of cycles to execute it, depending on the instruction. See description of each instruction below for details on the execution time.

Instruction fetch time is:

- 2 clock cycles —for instructions following ALU and branch instructions
- 4 clock cycles —in other cases

Note that when accessing RTC memories and RTC registers, ULP coprocessor has lower priority than the main CPUs. This means that ULP coprocessor execution may be suspended while the main CPUs access same memory region as the ULP.

The detailed description of all instructions is presented below:

Difference Between ESP32 ULP and ESP32-S3 ULP Instruction Sets Compared to the ESP32 ULP FSM coprocessor, the ESP32-S3 ULP FSM coprocessor has an extended instruction set. The ESP32-S3 ULP FSM is not binary compatible with ESP32 ULP FSM, but a ESP32 ULP FSM assembled program is expected to work on the ESP32-S3 ULP FSM after rebuilding.

The list of the new instructions that was added to the ESP32-S3 ULP FSM is: `LDL`, `LDH`, `STL`, `STH`, `ST32`, `STO`, `STI`, `STI32`.

NOP - No Operation Syntax

NOP

Operands

None

Cycles

2 cycle to execute, 4 cycles to fetch next instruction

Description

No operation is performed. Only the PC is incremented.

Example:

```
1:    NOP
```

ADD - Add to Register Syntax

ADD Rdst, Rsrc1, Rsrc2

ADD Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction adds source register to another source register or to a 16-bit signed value and stores the result in the destination register.

Examples:

```
1:    ADD R1, R2, R3         // R1 = R2 + R3

2:    Add R1, R2, 0x1234     // R1 = R2 + 0x1234

3:    .set value1, 0x03      // constant value1=0x03
      Add R1, R2, value1     // R1 = R2 + value1
```

(continues on next page)

(continued from previous page)

```
4:     .global label          // declaration of variable label
       add R1, R2, label      // R1 = R2 + label
         ...
       label: nop             // definition of variable label
```

SUB - Subtract from Register

Syntax

SUB Rdst, Rsrc1, Rsrc2

SUB Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction subtracts the source register from another source register or subtracts a 16-bit signed value from a source register, and stores the result to the destination register.

Examples:

```
1:          SUB R1, R2, R3              // R1 = R2 - R3

2:          sub R1, R2, 0x1234          // R1 = R2 - 0x1234

3:          .set value1, 0x03           // constant value1=0x03
            SUB R1, R2, value1          // R1 = R2 - value1
4:          .global label               // declaration of variable label
            SUB R1, R2, label           // R1 = R2 - label
              ....
  label:    nop                         // definition of variable label
```

AND - Bitwise Logical AND of Two Operands

Syntax

AND Rdst, Rsrc1, Rsrc2

AND Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction does a bitwise logical AND of a source register and another source register or a 16-bit signed value and stores the result to the destination register.

Examples:

```
1:        AND R1, R2, R3          // R1 = R2 & R3

2:        AND R1, R2, 0x1234      // R1 = R2 & 0x1234

3:        .set value1, 0x03       // constant value1=0x03
          AND R1, R2, value1      // R1 = R2 & value1

4:        .global label           // declaration of variable label
          AND R1, R2, label       // R1 = R2 & label
              ...
  label:  nop                     // definition of variable label
```

OR - Bitwise Logical OR of Two Operands Syntax

OR Rdst, Rsrc1, Rsrc2

OR Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction does a bitwise logical OR of a source register and another source register or a 16-bit signed value and stores the result to the destination register.

Examples:

```
1:        OR R1, R2, R3           // R1 = R2 || R3

2:        OR R1, R2, 0x1234       // R1 = R2 || 0x1234

3:        .set value1, 0x03       // constant value1=0x03
          OR R1, R2, value1       // R1 = R2 || value1

4:        .global label           // declaration of variable label
          OR R1, R2, label        // R1 = R2 || label
          ...
  label: nop                      // definition of variable label
```

LSH - Logical Shift Left Syntax

LSH Rdst, Rsrc1, Rsrc2

LSH Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction does a logical shift to left of the source register by the number of bits from another source register or a 16-bit signed value and stores the result to the destination register.

Note: Shift operations which are greater than 15 bits have undefined results.

Examples:

```
1:        LSH R1, R2, R3              // R1 = R2 << R3

2:        LSH R1, R2, 0x03            // R1 = R2 << 0x03

3:        .set value1, 0x03           // constant value1=0x03
          LSH R1, R2, value1          // R1 = R2 << value1

4:        .global label               // declaration of variable label
          LSH R1, R2, label           // R1 = R2 << label
          ...
  label:  nop                         // definition of variable label
```

RSH - Logical Shift Right Syntax

RSH Rdst, Rsrc1, Rsrc2

RSH Rdst, Rsrc1, imm

Operands

- **Rdst** - Register R[0..3]
- **Rsrc1** - Register R[0..3]
- **Rsrc2** - Register R[0..3]
- **Imm** - 16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction does a logical shift to right of a source register by the number of bits from another source register or a 16-bit signed value and stores the result to the destination register.

Note: Shift operations which are greater than 15 bits have undefined results.

Examples:

```
1:        RSH R1, R2, R3              // R1 = R2 >> R3

2:        RSH R1, R2, 0x03            // R1 = R2 >> 0x03

3:        .set value1, 0x03           // constant value1=0x03
          RSH R1, R2, value1          // R1 = R2 >> value1

4:        .global label               // declaration of variable label
          RSH R1, R2, label           // R1 = R2 >> label
  label:  nop                         // definition of variable label
```

MOVE –Move to Register Syntax

MOVE Rdst, Rsrc

MOVE Rdst, imm

Operands

- **Rdst** –Register R[0..3]
- **Rsrc** –Register R[0..3]
- **Imm** –16-bit signed value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction moves the value from the source register or a 16-bit signed value to the destination register.

Note: Note that when a label is used as an immediate, the address of the label will be converted from bytes to words. This is because `LD`, `ST`, and `JUMP` instructions expect the address register value to be expressed in words rather than bytes. See the section *Note About Addressing* for more details.

Examples:

```
1:        MOVE       R1, R2             // R1 = R2

2:        MOVE       R1, 0x03           // R1 = 0x03

3:        .set       value1, 0x03       // constant value1=0x03
          MOVE       R1, value1         // R1 = value1

4:        .global     label             // declaration of label
          MOVE        R1, label         // R1 = address_of(label) / 4
          ...
  label:  nop                           // definition of label
```

ST –Store Data to the Memory Syntax

ST Rsrc, Rdst, offset

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Offset** –13-bit signed value, offset in bytes

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores the 16-bit value of Rsrc to the lower half-word of memory with address Rdst + offset. The upper half-word is written with the current program counter (PC) (expressed in words, shifted left by 5 bits) OR'd with Rdst (0..3):

```
Mem[Rdst + offset / 4]{31:0} = {PC[10:0], 3'b0, Rdst, Rsrc[15:0]}
```

The application can use the higher 16 bits to determine which instruction in the ULP program has written any particular word into memory.

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:        ST  R1, R2, 0x12        // MEM[R2 + 0x12 / 4] = R1

2:        .data                   // Data section definition
  Addr1:  .word     123           // Define label Addr1 16 bit
          .set      offs, 0x00    // Define constant offs
          .text                   // Text section definition
          MOVE      R1, 1         // R1 = 1
          MOVE      R2, Addr1     // R2 = Addr1
          ST        R1, R2, offs  // MEM[R2 +  0 / 4] = R1
                                  // MEM[Addr1 + 0] will be 32'h600001
```

STL –Store Data to the Lower 16 Bits of 32-bit Memory Syntax

STL Rsrc, Rdst, offset, Label

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Offset** –11-bit signed value, offset in bytes
- **Label** –2-bit user defined unsigned value

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores the 16-bit value of Rsrc to the lower half-word of the memory with address [Rdst + offset / 4]:

```
Mem[Rdst + offset / 4]{15:0} = {Rsrc[15:0]}
Mem[Rdst + offset / 4]{15:0} = {Label[1:0],Rsrc[13:0]}
```

The `ST` and the `STL` commands can be used interchangeably and have been provided to maintain backward compatibility with previous versions of the ULP core.

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:        STL  R1, R2, 0x12           // MEM[R2 + 0x12 / 4] = R1

2:        .data                       // Data section definition
  Addr1:  .word     123               // Define label Addr1 16 bit
          .set      offs, 0x00        // Define constant offs
          .text                       // Text section definition
          MOVE      R1, 1             // R1 = 1
          MOVE      R2, Addr1         // R2 = Addr1
          STL       R1, R2, offs      // MEM[R2 +  0 / 4] = R1
                                      // MEM[Addr1 + 0] will be 32'hxxxx0001
3:
          MOVE      R1, 1             // R1 = 1
          STL       R1, R2, 0x12, 1   // MEM[R2 + 0x12 / 4] = 0xxxxx4001
```

STH –Store data to the higher 16 bits of 32-bit memory Syntax

STH Rsrc, Rdst, offset, Label

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Offset** –11-bit signed value, offset in bytes
- **Label** –2-bit user defined unsigned value

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores the 16-bit value of Rsrc to the upper half-word of memory with address [Rdst + offset / 4]:

```
Mem[Rdst + offset / 4]{31:16} = {Rsrc[15:0]}
Mem[Rdst + offset / 4]{31:16} = {Label[1:0],Rsrc[13:0]}
```

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:          STH   R1, R2, 0x12              // MEM[R2 + 0x12 / 4][31:16] = R1

2:          .data                           // Data section definition
  Addr1:    .word      123                  // Define label Addr1 16 bit
            .set       offs, 0x00           // Define constant offs
            .text                           // Text section definition
            MOVE       R1, 1                // R1 = 1
            MOVE       R2, Addr1            // R2 = Addr1
            STH        R1, R2, offs         // MEM[R2 +  0 / 4] = R1
                                            // MEM[Addr1 + 0] will be 32'h0001xxxx
3:
            MOVE       R1, 1                // R1 = 1
            STH        R1, R2, 0x12, 1      // MEM[R2 + 0x12 / 4] 0x4001xxxx
```

ST32 –Store 32-bits data to the 32-bits memory Syntax

ST32 Rsrc, Rdst, offset, Label

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Offset** –11-bit signed value, offset in bytes
- **Label** –2-bit user defined unsigned value

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores 11 bits of the PC value, label value and the 16-bit value of Rsrc to the 32-bit memory with address [Rdst + offset / 4]:

```
Mem[Rdst + offset / 4]{31:0} = {PC[10:0],0[2:0],Label[1:0],Rsrc[15:0]}
```

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:        ST32  R1, R2, 0x12, 0       // MEM[R2 + 0x12 / 4][31:0] = {PC[10:0],
↪0[2:0],Label[1:0],Rsrc[15:0]}

2:        .data                       // Data section definition
  Addr1:  .word     123               // Define label Addr1 16 bit
          .set      offs, 0x00        // Define constant offs
          .text                       // Text section definition
          MOVE      R1, 1             // R1 = 1
          MOVE      R2, Addr1         // R2 = Addr1
          ST32      R1, R2, offs, 1   // MEM[R2 +  0] = {PC[10:0],0[2:0],
↪Label[1:0],Rsrc[15:0]}
                                      // MEM[Addr1 + 0] will be 32'h00010001
```

STO –Set offset value for auto increment operation Syntax

STO offset

Operands

- **Offset** –11-bit signed value, offset in bytes

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction sets the 16-bit value to the offset register:

```
offset = value / 4
```

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:        STO  0x12                 // Offset = 0x12 / 4

2:        .data                     // Data section definition
  Addr1:  .word     123             // Define label Addr1 16 bit
          .set      offs, 0x00      // Define constant offs
          .text                     // Text section definition
          STO       offs            // Offset = 0x00
```

STI –Store data to the 32-bits memory with auto increment of predefined offset address Syntax

STI Rsrc, Rdst, Label

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Label** –2-bit user defined unsigned value

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores the 16-bit value of Rsrc to the lower and upper half-word of memory with address [Rdst + offset / 4].

The offset value is auto incremented when the STI instruction is called twice. Make sure to execute the `STO` instruction to set the offset value before executing the STI instruction:

```
Mem[Rdst + offset / 4]{15:0/31:16} = {Rsrc[15:0]}
Mem[Rdst + offset / 4]{15:0/31:16} = {Label[1:0],Rsrc[13:0]}
```

Examples:

```
1:          STO  4                  // Set offset to 4
            STI  R1, R2             // MEM[R2 + 4 / 4][15:0] = R1
            STI  R1, R2             // MEM[R2 + 4 / 4][31:16] = R1
                                    // offset += (1 * 4) //offset is incremented by␣
↪1 word
            STI  R1, R2             // MEM[R2 + 8 / 4][15:0] = R1
            STI  R1, R2             // MEM[R2 + 8 / 4][31:16] = R1
```

STI32 –Store 32-bits data to the 32-bits memory with auto increment of address offset Syntax

STI32 Rsrc, Rdst, Label

Operands

- **Rsrc** –Register R[0..3], holds the 16-bit value to store
- **Rdst** –Register R[0..3], address of the destination, in 32-bit words
- **Label** –2-bit user defined unsigned value

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction stores 11 bits of the PC value, label value and the 16-bit value of Rsrc to the 32-bit memory with address [Rdst + offset / 4].

The offset value is auto incremented each time the STI32 instruction is called. Make sure to execute the `STO` instruction to set the offset value before executing the STI32 instruction:

```
Mem[Rdst + offset / 4]{31:0} = {PC[10:0],0[2:0],Label[1:0],Rsrc[15:0]}
```

Examples:

```
1:          STO    0x12
            STI32  R1, R2, 0     // MEM[R2 + 0x12 / 4][31:0] = {PC[10:0],0[2:0],
↪Label[1:0],Rsrc[15:0]}
                                 // offset += (1 * 4) //offset is incremented by 1␣
↪word
            STI32  R1, R2, 0     // MEM[R2 + 0x16 / 4][31:0] = {PC[10:0],0[2:0],
↪Label[1:0],Rsrc[15:0]}
```

LD –Load Data from the Memory Syntax

LD Rdst, Rsrc, offset

Operands

- **Rdst** –Register R[0..3], destination
- **Rsrc** –Register R[0..3], holds address of destination, in 32-bit words

- **Offset** –13-bit signed value, offset in bytes

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction loads the lower 16-bit half-word from memory with address [Rsrc + offset / 4] into the destination register Rdst:

```
Rdst[15:0] = Mem[Rsrc + offset / 4][15:0]
```

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:          LD   R1, R2, 0x12              // R1 = MEM[R2 + 0x12 / 4]

2:          .data                          // Data section definition
  Addr1:    .word      123                 // Define label Addr1 16 bit
            .set       offs, 0x00          // Define constant offs
            .text                          // Text section definition
            MOVE       R1, 1               // R1 = 1
            MOVE       R2, Addr1           // R2 = Addr1 / 4 (address of label is␣
↪converted into words)
            LD         R1, R2, offs        // R1 = MEM[R2 +  0]
                                           // R1 will be 123
```

LDL –Load Data from the Lower Half-Word of the 32-bit Memory

Syntax

LDL Rdst, Rsrc, offset

Operands

- **Rdst** –Register R[0..3], destination
- **Rsrc** –Register R[0..3], holds address of destination, in 32-bit words
- **Offset** –13-bit signed value, offset in bytes

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction loads the lower 16-bit half-word from memory with address [Rsrc + offset / 4] into the destination register Rdst:

```
Rdst[15:0] = Mem[Rsrc + offset / 4][15:0]
```

The `LD` and the `LDL` commands can be used interchangeably and have been provided to maintain backward compatibility with previous versions of the ULP core.

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:          LDL   R1, R2, 0x12            // R1 = MEM[R2 + 0x12 / 4]

2:          .data                         // Data section definition
  Addr1:    .word      123                // Define label Addr1 16 bit
            .set       offs, 0x00         // Define constant offs
            .text                         // Text section definition
            MOVE       R1, 1              // R1 = 1
            MOVE       R2, Addr1          // R2 = Addr1 / 4 (address of label is␣
↪converted into words)
            LDL        R1, R2, offs       // R1 = MEM[R2 +  0]
                                          // R1 will be 123
```

LDH –Load data from upper half-word of the 32-bit memory Syntax

LDH Rdst, Rsrc, offset

Operands

- **Rdst** –Register R[0..3], destination
- **Rsrc** –Register R[0..3], holds address of destination, in 32-bit words
- **Offset** –13-bit signed value, offset in bytes

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction loads the upper 16-bit half-word from memory with address [Rsrc + offset / 4] into the destination register Rdst:

```
Rdst[15:0] = Mem[Rsrc + offset / 4][15:0]
```

Note: Note that the offset specified in bytes is converted to a 32-bit word offset before execution. See the section *Note About Addressing* for more details.

Examples:

```
1:          LDH   R1, R2, 0x12            // R1 = MEM[R2 + 0x12 / 4]

2:          .data                         // Data section definition
  Addr1:    .word      0x12345678         // Define label Addr1 16 bit
            .set       offs, 0x00         // Define constant offs
            .text                         // Text section definition
            MOVE       R1, 1              // R1 = 1
            MOVE       R2, Addr1          // R2 = Addr1 / 4 (address of label is␣
↪converted into words)
            LDH        R1, R2, offs       // R1 = MEM[R2 +  0]
                                          // R1 will be 0x1234
```

JUMP –Jump to an Absolute Address Syntax

JUMP Rdst

JUMP ImmAddr

JUMP Rdst, Condition

JUMP ImmAddr, Condition

Operands

- **Rdst** –Register R[0..3] containing address to jump to (expressed in 32-bit words)
- **ImmAddr** –13 bits address (expressed in bytes), aligned to 4 bytes
- **Condition**: - EQ –jump if the last ALU operation result was zero - OV –jump if the last ALU has set overflow flag

Cycles

2 cycles to execute, 2 cycles to fetch next instruction.

Description

The instruction makes jump to the specified address. Jump can be either unconditional or based on an ALU flag.

Examples:

```
1:        JUMP        R1             // Jump to address in R1 (address in R1 is in␣
↪32-bit words)

2:        JUMP        0x120, EQ      // Jump to address 0x120 (in bytes) if ALU␣
↪result is zero

3:        JUMP        label          // Jump to label
          ...
  label:  nop                        // Definition of label

4:        .global     label          // Declaration of global label

          MOVE        R1, label      // R1 = label (value loaded into R1 is in words)
          JUMP        R1             // Jump to label
          ...
  label:  nop                        // Definition of label
```

JUMPR –Jump to a Relative Offset (Condition Based on R0) Syntax

JUMPR Step, Threshold, Condition

Operands

- **Step** –relative shift from current position, in bytes
- **Threshold** –threshold value for branch condition
- **Condition:**
  - **EQ** (equal) –jump if value in R0 == threshold
  - **LT** (less than) –jump if value in R0 < threshold
  - **LE** (less or equal) –jump if value in R0 <= threshold
  - **GT** (greater than) –jump if value in R0 > threshold
  - **GE** (greater or equal) –jump if value in R0 >= threshold

Cycles

Conditions **EQ**, **GT** and **LT**: 2 cycles to execute, 2 cycles to fetch next instruction.

Conditions **LE** and **GE** are implemented in the assembler using two **JUMPR** instructions:

```
// JUMPR target, threshold, LE is implemented as:

          JUMPR target, threshold, EQ
          JUMPR target, threshold, LT

// JUMPR target, threshold, GE is implemented as:

          JUMPR target, threshold, EQ
          JUMPR target, threshold, GT
```

Therefore the execution time depends on the branches taken: either 2 cycles to execute + 2 cycles to fetch, or 4 cycles to execute + 4 cycles to fetch.

Description

The instruction makes a jump to a relative address if condition is true. Condition is the result of comparison of R0 register value and the threshold value.

Examples:

```
1:pos:    JUMPR       16, 20, GE   // Jump to address (position + 16 bytes) if␣
↪value in R0 >= 20

2:        // Down counting loop using R0 register
          MOVE        R0, 16       // load 16 into R0
  label:  SUB         R0, R0, 1    // R0--
          NOP                      // do something
          JUMPR       label, 1, GE // jump to label if R0 >= 1
```

JUMPS –Jump to a Relative Address (Condition Based on Stage Count) Syntax

JUMPS Step, Threshold, Condition

Operands

- **Step** –relative shift from current position, in bytes
- **Threshold** –threshold value for branch condition
- **Condition:**
  - **EQ** (equal) –jump if value in stage_cnt == threshold
  - **LT** (less than) –jump if value in stage_cnt < threshold
  - **LE** (less or equal) - jump if value in stage_cnt <= threshold
  - **GT** (greater than) –jump if value in stage_cnt > threshold
  - **GE** (greater or equal) —jump if value in stage_cnt >= threshold

Cycles

2 cycles to execute, 2 cycles to fetch next instruction.

Description

The instruction makes a jump to a relative address if condition is true. Condition is the result of comparison of count register value and threshold value.

Examples:

```
1:pos:    JUMPS     16, 20, EQ     // Jump to (position + 16 bytes) if stage_cnt␣
↪== 20

2:        // Up counting loop using stage count register
          STAGE_RST                // set stage_cnt to 0
  label:  STAGE_INC  1             // stage_cnt++
          NOP                      // do something
          JUMPS       label, 16, LT  // jump to label if stage_cnt < 16
```

STAGE_RST –Reset Stage Count Register Syntax

STAGE_RST

Operands

No operands

Description

The instruction sets the stage count register to 0

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Examples:

```
1:        STAGE_RST       // Reset stage count register
```

STAGE_INC –Increment Stage Count Register Syntax

STAGE_INC Value

Operands

- **Value** –8 bits value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction increments the stage count register by the given value

Examples:

```
1:        STAGE_INC       10          // stage_cnt += 10

2:        // Up counting loop example:
          STAGE_RST                   // set stage_cnt to 0
  label:  STAGE_INC  1                // stage_cnt++
          NOP                         // do something
          JUMPS       label, 16, LT   // jump to label if stage_cnt < 16
```

STAGE_DEC –Decrement Stage Count Register Syntax

STAGE_DEC Value

Operands

- **Value** –8 bits value

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction decrements the stage count register by the given value

Examples:

```
1:        STAGE_DEC       10        // stage_cnt -= 10;

2:        // Down counting loop example
          STAGE_RST                 // set stage_cnt to 0
          STAGE_INC  16             // increment stage_cnt to 16
  label:  STAGE_DEC  1              // stage_cnt--;
          NOP                       // do something
          JUMPS       label, 0, GT  // jump to label if stage_cnt > 0
```

HALT –End the Program Syntax

HALT

Operands

No operands

Cycles

2 cycles to execute

Description

The instruction halts the ULP coprocessor and restarts the ULP wakeup timer, if it is enabled

Examples:

```
1:       HALT      // Halt the coprocessor
```

WAKE –Wake up the Chip Syntax

WAKE

Operands

No operands

Cycles

2 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction sends an interrupt from the ULP coprocessor to the RTC controller.

- If the SoC is in deep sleep mode, and ULP wakeup is enabled, this causes the SoC to wake up.
- If the SoC is not in deep sleep mode, and ULP interrupt bit (RTC_CNTL_ULP_CP_INT_ENA) is set in RTC_CNTL_INT_ENA_REG register, RTC interrupt will be triggered.

Note: Note that before using WAKE instruction, ULP program may need to wait until RTC controller is ready to wake up the main CPU. This is indicated using RTC_CNTL_RDY_FOR_WAKEUP bit of RTC_CNTL_LOW_POWER_ST_REG register. If WAKE instruction is executed while RTC_CNTL_RDY_FOR_WAKEUP is zero, it has no effect (wake up does not occur). If the WAKE instruction is intended to be used while the main CPU is not in sleep mode then the RTC_CNTL_MAIN_STATE_IN_IDLE (bit 27) of RTC_CNTL_LOW_POWER_ST_REG can be used to check whether main CPU is in normal mode or sleep mode.

Examples:

```
1: is_rdy_for_wakeup:                    // Read RTC_CNTL_RDY_FOR_WAKEUP bit
          READ_RTC_FIELD(RTC_CNTL_LOW_POWER_ST_REG, RTC_CNTL_RDY_FOR_WAKEUP)
          AND r0, r0, 1
          JUMP is_rdy_for_wakeup, eq     // Retry until the bit is set
          WAKE                           // Trigger wake up
          REG_WR 0x006, 24, 24, 0        // Stop ULP timer (clear RTC_CNTL_ULP_CP_
↪SLP_TIMER_EN)
          HALT                           // Stop the ULP program
          // After these instructions, SoC will wake up,
          // and ULP will not run again until started by the main program.

1: check_wakeup:                         // Read RTC_CNTL_RDY_FOR_WAKEUP and RTC_
↪CNTL_MAIN_STATE_IN_IDLE bit
          READ_RTC_REG(RTC_CNTL_LOW_POWER_ST_REG, 27, 1)
          MOVE r1, r0                    // Copy result in to r1
          READ_RTC_FIELD(RTC_CNTL_LOW_POWER_ST_REG, RTC_CNTL_RDY_FOR_WAKEUP)
          OR r0, r0, r1
          JUMP check_wakeup, eq          // Retry until either of the bit are set
          WAKE                           // Trigger wake up
          HALT                           // Stop the ULP program
```

WAIT –Wait Some Number of Cycles Syntax

WAIT Cycles

Operands

- **Cycles** –number of cycles for wait

Cycles

(2 + **Cycles**) cycles to execute, 4 cycles to fetch next instruction

Description

The instruction delays for a given number of cycles.

Examples:

```
1:        WAIT     10          // Do nothing for 10 cycles

2:        .set  wait_cnt, 10   // Set a constant
          WAIT  wait_cnt       // wait for 10 cycles
```

TSENS –Do Measurement with Temperature Sensor Syntax

- **TSENS Rdst, Wait_Delay**

Operands

- **Rdst** –Destination Register R[0..3], result will be stored to this register
- **Wait_Delay** –number of cycles used to perform the measurement

Cycles

(2 + **Wait_Delay** + 3 * TSENS_CLK) to execute, 4 cycles to fetch next instruction

Description

The instruction performs measurement using TSENS and stores the result into a general purpose register

Examples:

```
1:        TSENS     R1, 1000     // Measure temperature sensor for 1000 cycles,
                                 // and store result to R1
```

ADC –Do Measurement with ADC Syntax

- **ADC Rdst, Sar_sel, Mux**
- **ADC Rdst, Sar_sel, Mux, 0** —deprecated form

Operands

- **Rdst** –Destination Register R[0..3], result will be stored to this register
- **Sar_sel** –Select ADC: 0 = SARADC1, 1 = SARADC2
- **Mux** - selected PAD, SARADC Pad[Mux-1] is enabled. If the user passes Mux value 1, then ADC pad 0 gets used.

Cycles

`23 + max(1, SAR_AMP_WAIT1) + max(1, SAR_AMP_WAIT2) + max(1, SAR_AMP_WAIT3) + SARx_SAMPLE_CYCLE + SARx_SAMPLE_BIT` cycles to execute, 4 cycles to fetch next instruction

Description

The instruction makes measurements from ADC

Examples:

```
1:        ADC      R1, 0, 1      // Measure value using ADC1 pad 2 and store
↪result into R1
```

REG_RD –Read from Peripheral Register Syntax

REG_RD Addr, High, Low

Operands

- **Addr** –Register address, in 32-bit words
- **High** –Register end bit number
- **Low** –Register start bit number

Cycles

4 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction reads up to 16 bits from a peripheral register into a general purpose register: `R0 = REG[Addr][High:Low]`.

This instruction can access registers in RTC_CNTL, RTC_IO, SENS, and RTC_I2C peripherals. Address of the register, as seen from the ULP, can be calculated from the address of the same register on the PeriBUS1 as follows:

```
addr_ulp = (addr_peribus1 - DR_REG_RTCCNTL_BASE) / 4
```

Examples:

```
1:        REG_RD      0x120, 7, 4     // load 4 bits: R0 = {12'b0, REG[0x120][7:4]}
```

REG_WR –Write to Peripheral Register Syntax

REG_WR Addr, High, Low, Data

Operands

- **Addr** –Register address, in 32-bit words
- **High** –Register end bit number
- **Low** –Register start bit number
- **Data** –Value to write, 8 bits

Cycles

8 cycles to execute, 4 cycles to fetch next instruction

Description

The instruction writes up to 8 bits from an immediate data value into a peripheral register: `REG[Addr][High:Low] = data`.

This instruction can access registers in RTC_CNTL, RTC_IO, SENS, and RTC_I2C peripherals. Address of the register, as seen from the ULP, can be calculated from the address of the same register on the PeriBUS1 as follows:

```
addr_ulp = (addr_peribus1 - DR_REG_RTCCNTL_BASE) / 4
```

Examples:

```
1:        REG_WR      0x120, 7, 0, 0x10   // set 8 bits: REG[0x120][7:0] = 0x10
```

Convenience Macros for Peripheral Registers Access ULP source files are passed through C preprocessor before the assembler. This allows certain macros to be used to facilitate access to peripheral registers.

Some existing macros are defined in `soc/soc_ulp.h` header file. These macros allow access to the fields of peripheral registers by their names. Peripheral registers' names which can be used with these macros are the ones defined in `soc/rtc_cntl_reg.h`, `soc/rtc_io_reg.h`, `soc/sens_reg.h`, and `soc/rtc_i2c_reg.h`.

READ_RTC_REG(rtc_reg, low_bit, bit_width) Read up to 16 bits from rtc_reg[low_bit + bit_width - 1 : low_bit] into R0. For example:

```
#include "soc/soc_ulp.h"
#include "soc/rtc_cntl_reg.h"

/* Read 16 lower bits of RTC_CNTL_TIME0_REG into R0 */
READ_RTC_REG(RTC_CNTL_TIME0_REG, 0, 16)
```

READ_RTC_FIELD(rtc_reg, field) Read from a field in rtc_reg into R0, up to 16 bits. For example:

```
#include "soc/soc_ulp.h"
#include "soc/sens_reg.h"

/* Read 8-bit SENS_TSENS_OUT field of SENS_SAR_SLAVE_ADDR3_REG into R0 */
READ_RTC_FIELD(SENS_SAR_SLAVE_ADDR3_REG, SENS_TSENS_OUT)
```

WRITE_RTC_REG(rtc_reg, low_bit, bit_width, value) Write immediate value into rtc_reg[low_bit + bit_width - 1 : low_bit], bit_width <= 8. For example:

```
#include "soc/soc_ulp.h"
#include "soc/rtc_io_reg.h"

/* Set BIT(2) of RTC_GPIO_OUT_DATA_W1TS field in RTC_GPIO_OUT_W1TS_REG */
WRITE_RTC_REG(RTC_GPIO_OUT_W1TS_REG, RTC_GPIO_OUT_DATA_W1TS_S + 2, 1, 1)
```

WRITE_RTC_FIELD(rtc_reg, field, value) Write immediate value into a field in rtc_reg, up to 8 bits. For example:

```
#include "soc/soc_ulp.h"
#include "soc/rtc_cntl_reg.h"

/* Set RTC_CNTL_ULP_CP_SLP_TIMER_EN field of RTC_CNTL_STATE0_REG to 0 */
WRITE_RTC_FIELD(RTC_CNTL_STATE0_REG, RTC_CNTL_ULP_CP_SLP_TIMER_EN, 0)
```

Programming ULP FSM Coprocessor Using C Macros (Legacy) In addition to the existing binutils port for the ESP32-S3 ULP coprocessor, it is possible to generate programs for the ULP FSM coprocessor by embedding assembly-like macros into an ESP32-S3 application. Here is an example how this can be done:

```
const ulp_insn_t program[] = {
    I_MOVI(R3, 16),          // R3 <- 16
    I_LD(R0, R3, 0),         // R0 <- RTC_SLOW_MEM[R3 + 0]
    I_LD(R1, R3, 1),         // R1 <- RTC_SLOW_MEM[R3 + 1]
    I_ADDR(R2, R0, R1),      // R2 <- R0 + R1
    I_ST(R2, R3, 2),         // R2 -> RTC_SLOW_MEM[R2 + 2]
    I_HALT()
};
size_t load_addr = 0;
size_t size = sizeof(program)/sizeof(ulp_insn_t);
ulp_process_macros_and_load(load_addr, program, &size);
ulp_run(load_addr);
```

The `program` array is an array of `ulp_insn_t`, i.e., ULP coprocessor instructions. Each `I_XXX` preprocessor define translates into a single 32-bit instruction. Arguments of these preprocessor defines can be register numbers (`R0`

—R3) and literal constants. See the API reference section at the end of this guide for descriptions of instructions and arguments they take.

Note: Because some of the instruction macros expand to inline function calls, defining such array in global scope will cause the compiler to produce an "initializer element is not constant" error. To fix this error, move the definition of instructions array into local scope.

Note: Load, store and move instructions use **addresses expressed in 32-bit words**. Address 0 corresponds to the first word of RTC_SLOW_MEM. This is different to how address arguments are handled in assembly code of the same instructions. See the section *Note About Addressing* for more details for reference.

To generate branch instructions, special M_ preprocessor defines are used. M_LABEL define can be used to define a branch target. Label identifier is a 16-bit integer. M_Bxxx defines can be used to generate branch instructions with target set to a particular label.

Implementation note: these M_ preprocessor defines will be translated into two ulp_insn_t values: one is a token value which contains label number, and the other is the actual instruction. ulp_process_macros_and_load function resolves the label number to the address, modifies the branch instruction to use the correct address, and removes the extra ulp_insn_t token which contains the label number.

Here is an example of using labels and branches:

```
const ulp_insn_t program[] = {
    I_MOVI(R0, 34),          // R0 <- 34
    M_LABEL(1),              // label_1
    I_MOVI(R1, 32),          // R1 <- 32
    I_LD(R1, R1, 0),         // R1 <- RTC_SLOW_MEM[R1]
    I_MOVI(R2, 33),          // R2 <- 33
    I_LD(R2, R2, 0),         // R2 <- RTC_SLOW_MEM[R2]
    I_SUBR(R3, R1, R2),      // R3 <- R1 - R2
    I_ST(R3, R0, 0),         // R3 -> RTC_SLOW_MEM[R0 + 0]
    I_ADDI(R0, R0, 1),       // R0++
    M_BL(1, 64),             // if (R0 < 64) goto label_1
    I_HALT(),
};
RTC_SLOW_MEM[32] = 42;
RTC_SLOW_MEM[33] = 18;
size_t load_addr = 0;
size_t size = sizeof(program)/sizeof(ulp_insn_t);
ulp_process_macros_and_load(load_addr, program, &size);
ulp_run(load_addr);
```

API Reference

Header File

- components/ulp/ulp_fsm/include/esp32s3/ulp.h
- This header file can be included with:

```
#include "ulp.h"
```

- This header file is a part of the API provided by the ulp component. To declare that your component depends on ulp, add the following to your CMakeLists.txt:

```
REQUIRES ulp
```

or

```
PRIV_REQUIRES ulp
```

Functions

static inline uint32_t **SOC_REG_TO_ULP_PERIPH_SEL**(uint32_t reg)

Map SoC peripheral register to periph_sel field of RD_REG and WR_REG instructions.

Parameters **reg** -- peripheral register in RTC_CNTL_, RTC_IO_, SENS_, RTC_I2C peripherals.

Returns periph_sel value for the peripheral to which this register belongs.

Unions

union **ulp_insn**

#include <ulp.h> Instruction format structure.

All ULP instructions are 32 bit long. This union contains field layouts used by all of the supported instructions. This union also includes a special "macro" instruction layout. This is not a real instruction which can be executed by the CPU. It acts as a token which is removed from the program by the ulp_process_macros_and_load function.

These structures are not intended to be used directly. Preprocessor definitions provided below fill the fields of these structure with the right arguments.

Public Members

uint32_t **cycles**

Number of cycles to sleep

TBD, cycles used for measurement

uint32_t **unused**

Unused

uint32_t **opcode**

Opcode (OPCODE_DELAY)

Opcode (OPCODE_ST)

Opcode (OPCODE_LD)

Opcode (OPCODE_HALT)

Opcode (OPCODE_BRANCH)

Opcode (OPCODE_ALU)

Opcode (OPCODE_WR_REG)

Opcode (OPCODE_RD_REG)

Opcode (OPCODE_ADC)

Opcode (OPCODE_TSENS)

Opcode (OPCODE_I2C)

Opcode (OPCODE_END)

Opcode (OPCODE_MACRO)

struct *ulp_insn*::[anonymous] **delay**

Format of DELAY instruction

uint32_t **dreg**

Register which contains data to store

Register where the data should be loaded to

Register which contains target PC, expressed in words (used if .reg == 1)

Destination register

Register where to store ADC result

Register where to store temperature measurement result

uint32_t **sreg**

Register which contains address in RTC memory (expressed in words)

Register with operand A

uint32_t **label**

Data label, 2-bit user defined unsigned value

Label number

uint32_t **upper**

0: write the low half-word; 1: write the high half-word

uint32_t **wr_way**

0: write the full-word; 1: with the label; 3: without the label

uint32_t **unused1**

Unused

uint32_t **offset**

Offset to add to sreg

Absolute value of target PC offset w.r.t. current PC, expressed in words

uint32_t **unused2**

Unused

uint32_t **sub_opcode**

Sub opcode (SUB_OPCODE_ST)

Sub opcode (SUB_OPCODE_BX)

Sub opcode (SUB_OPCODE_B)

Sub opcode (SUB_OPCODE_ALU_REG)

Sub opcode (SUB_OPCODE_ALU_IMM)

Sub opcode (SUB_OPCODE_ALU_CNT)

Sub opcode (SUB_OPCODE_WAKEUP)

SUB_OPCODE_MACRO_LABEL or SUB_OPCODE_MACRO_BRANCH

struct *ulp_insn*::[anonymous] **st**

Format of ST instruction

uint32_t **rd_upper**

0: read the high half-word; 1: read the low half-word

struct *ulp_insn*::[anonymous] **ld**

Format of LD instruction

struct *ulp_insn*::[anonymous] **halt**

Format of HALT instruction

uint32_t **addr**

Target PC, expressed in words (used if .reg == 0)

Address within either RTC_CNTL, RTC_IO, or SARADC

uint32_t **reg**

Target PC in register (1) or immediate (0)

uint32_t **type**

Jump condition (BX_JUMP_TYPE_xxx)

struct *ulp_insn*::[anonymous] **bx**

Format of BRANCH instruction (absolute address)

uint32_t **imm**

Immediate value to compare against

Immediate value of operand B

Immediate value

uint32_t **cmp**

Comparison to perform: B_CMP_L or B_CMP_GE

uint32_t **sign**

Sign of target PC offset: 0: positive, 1: negative

struct *ulp_insn*::[anonymous] **b**

Format of BRANCH instruction (relative address)

uint32_t **treg**

Register with operand B

uint32_t **sel**

Operation to perform, one of ALU_SEL_xxx

struct *ulp_insn*::[anonymous] **alu_reg**

Format of ALU instruction (both sources are registers)

struct *ulp_insn*::[anonymous] **alu_imm**

Format of ALU instruction (one source is an immediate)

uint32_t **unused3**

Unused

struct *ulp_insn*::[anonymous] **alu_cnt**

Format of ALU instruction with stage count register and an immediate

uint32_t **periph_sel**

Select peripheral: RTC_CNTL (0), RTC_IO(1), SARADC(2)

uint32_t **data**

8 bits of data to write

Data to read or write

uint32_t **low**

Low bit

uint32_t **high**

High bit

struct *ulp_insn*::[anonymous] **wr_reg**

Format of WR_REG instruction

struct *ulp_insn*::[anonymous] **rd_reg**

Format of RD_REG instruction

uint32_t **mux**

Select SARADC pad (mux + 1)

uint32_t **sar_sel**

Select SARADC0 (0) or SARADC1 (1)

struct *ulp_insn*::[anonymous] **adc**

Format of ADC instruction

uint32_t **wait_delay**

Cycles to wait after measurement is done

uint32_t **reserved**

Reserved, set to 0

struct *ulp_insn*::[anonymous] **tsens**

Format of TSENS instruction

uint32_t **i2c_addr**

I2C slave address

uint32_t **low_bits**

TBD

uint32_t **high_bits**

TBD

uint32_t **i2c_sel**

TBD, select reg_i2c_slave_address[7:0]

uint32_t **rw**

Write (1) or read (0)

struct *ulp_insn*::[anonymous] **i2c**

Format of I2C instruction

uint32_t **wakeup**

Set to 1 to wake up chip

struct *ulp_insn*::[anonymous] **end**

Format of END instruction with wakeup

struct *ulp_insn*::[anonymous] **macro**

Format of tokens used by LABEL and BRANCH macros

Macros

R0

general purpose register 0

R1

general purpose register 1

R2

general purpose register 2

R3

general purpose register 3

OPCODE_WR_REG

Instruction: write peripheral register (RTC_CNTL/RTC_IO/SARADC) (not implemented yet)

OPCODE_RD_REG

Instruction: read peripheral register (RTC_CNTL/RTC_IO/SARADC) (not implemented yet)

RD_REG_PERIPH_RTC_CNTL

Identifier of RTC_CNTL peripheral for RD_REG and WR_REG instructions

RD_REG_PERIPH_RTC_IO

Identifier of RTC_IO peripheral for RD_REG and WR_REG instructions

RD_REG_PERIPH_SENS

Identifier of SARADC peripheral for RD_REG and WR_REG instructions

RD_REG_PERIPH_RTC_I2C

Identifier of RTC_I2C peripheral for RD_REG and WR_REG instructions

OPCODE_I2C

Instruction: read/write I2C (not implemented yet)

OPCODE_DELAY

Instruction: delay (nop) for a given number of cycles

OPCODE_ADC

Instruction: SAR ADC measurement (not implemented yet)

OPCODE_ST

Instruction: store indirect to RTC memory

SUB_OPCODE_ST_AUTO

Automatic Storage Mode - Access continuous addresses. Use SUB_OPCODE_ST_OFFSET to configure the initial address before using this instruction.

SUB_OPCODE_ST_OFFSET

Automatic Storage Mode - Configure the initial address.

SUB_OPCODE_ST

Manual Storage Mode. Store 32 bits, 16 MSBs contain PC, 16 LSBs contain value from source register

OPCODE_ALU

Arithmetic instructions

SUB_OPCODE_ALU_REG

Arithmetic instruction, both source values are in register

SUB_OPCODE_ALU_IMM

Arithmetic instruction, one source value is an immediate

SUB_OPCODE_ALU_CNT

Arithmetic instruction between counter register and an immediate (not implemented yet)

ALU_SEL_ADD

Addition

ALU_SEL_SUB

Subtraction

ALU_SEL_AND

Logical AND

ALU_SEL_OR

Logical OR

ALU_SEL_MOV

Copy value (immediate to destination register or source register to destination register

ALU_SEL_LSH

Shift left by given number of bits

ALU_SEL_RSH

Shift right by given number of bits

ALU_SEL_STAGE_INC

Increment stage count register

ALU_SEL_STAGE_DEC

Decrement stage count register

ALU_SEL_STAGE_RST

Reset stage count register

OPCODE_BRANCH

Branch instructions

SUB_OPCODE_B

Branch to a relative offset

SUB_OPCODE_BX

Branch to absolute PC (immediate or in register)

SUB_OPCODE_BS

Branch to a relative offset by comparing the stage_cnt register

BX_JUMP_TYPE_DIRECT

Unconditional jump

BX_JUMP_TYPE_ZERO

Branch if last ALU result is zero

BX_JUMP_TYPE_OVF

Branch if last ALU operation caused and overflow

B_CMP_L

Branch if R0 is less than an immediate

B_CMP_G

Branch if R0 is greater than an immediate

B_CMP_E

Branch if R0 is equal to an immediate

BS_CMP_L

Branch if stage_cnt is less than an immediate

BS_CMP_GE

Branch if stage_cnt is greater than or equal to an immediate

BS_CMP_LE

Branch if stage_cnt is less than or equal to an immediate

OPCODE_END

Stop executing the program

SUB_OPCODE_END

Stop executing the program and optionally wake up the chip

SUB_OPCODE_SLEEP

Stop executing the program and run it again after selected interval

OPCODE_TSENS

Instruction: temperature sensor measurement (not implemented yet)

OPCODE_HALT

Halt the coprocessor

OPCODE_LD

Indirect load lower 16 bits from RTC memory

OPCODE_MACRO

Not a real opcode. Used to identify labels and branches in the program

SUB_OPCODE_MACRO_LABEL

Label macro

SUB_OPCODE_MACRO_BRANCH

Branch macro

SUB_OPCODE_MACRO_LABELPC

Label pointer macro

I_DELAY (cycles_)

Delay (nop) for a given number of cycles

I_HALT ()

Halt the coprocessor.

This instruction halts the coprocessor, but keeps ULP timer active. As such, ULP program will be restarted again by timer. To stop the program and prevent the timer from restarting the program, use I_END(0) instruction.

I_WR_REG (reg, low_bit, high_bit, val)

Write literal value to a peripheral register

reg[high_bit : low_bit] = val This instruction can access RTC_CNTL_, RTC_IO_, SENS_, and RTC_I2C peripheral registers.

I_RD_REG (reg, low_bit, high_bit)

Read from peripheral register into R0

R0 = reg[high_bit : low_bit] This instruction can access RTC_CNTL_, RTC_IO_, SENS_, and RTC_I2C peripheral registers.

I_WR_REG_BIT (reg, shift, val)

Set or clear a bit in the peripheral register.

Sets bit (1 << shift) of register reg to value val. This instruction can access RTC_CNTL_, RTC_IO_, SENS_, and RTC_I2C peripheral registers.

I_WAKE ()

Wake the SoC from deep sleep.

This instruction initiates wake up from deep sleep. Use esp_deep_sleep_enable_ulp_wakeup to enable deep sleep wakeup triggered by the ULP before going into deep sleep. Note that ULP program will still keep running until the I_HALT instruction, and it will still be restarted by timer at regular intervals, even when the SoC is woken up.

To stop the ULP program, use I_HALT instruction.

To disable the timer which start ULP program, use I_END() instruction. I_END instruction clears the RTC_CNTL_ULP_CP_SLP_TIMER_EN_S bit of RTC_CNTL_ULP_CP_TIMER_REG register, which controls the ULP timer.

I_END ()

Stop ULP program timer.

This is a convenience macro which disables the ULP program timer. Once this instruction is used, ULP program will not be restarted anymore until ulp_run function is called.

ULP program will continue running after this instruction. To stop the currently running program, use I_HALT().

I_TSENS (reg_dest, delay)

Perform temperature sensor measurement and store it into reg_dest.

Delay can be set between 1 and ((1 << 14) - 1). Higher values give higher measurement resolution.

I_ADC (reg_dest, adc_idx, pad_idx)

Perform ADC measurement and store result in reg_dest.

adc_idx selects ADC (0 or 1). pad_idx selects ADC pad (0 - 7).

I_ST_MANUAL (reg_val, reg_addr, offset_, label_, upper_, wr_way_)

Store lower half-word, upper half-word or full-word data from register reg_val into RTC memory address.

This instruction can be used to write data to discontinuous addresses in the RTC_SLOW_MEM. The value is written to an offset calculated by adding the value of reg_addr register and offset_ field (this offset is expressed in 32-bit words). The storage method is dictated by the wr_way and upper field settings as summarized in the following table:

```
* |--------|-------|-------------------------------------------------------------
 ↪-----------------------------|-----------------------------|
* | wr_way | upper |                                          data               ␣
 ↪                              |          operation          |
* |--------|-------|-------------------------------------------------------------
 ↪-----------------------------|-----------------------------|
```

(continues on next page)

(continued from previous page)

```
* |        |       |                                                                 ␣
↪                              | Write full-word, including |
* |   0    |   X   | RTC_SLOW_MEM[addr + offset_]{31:0} = {insn_PC[10:0], 3'
b0, label_[1:0], reg_val[15:0]} | the PC and the data        |
* |--------|-------|-----------------------------------------------------------
↪-----------------------------|-----------------------------|
* |        |       |                                                                 ␣
↪                              | Store the data with label  |
* |   1    |   0   | RTC_SLOW_MEM[addr + offset_]{15:0} = {label_[1:0], reg_
↪val[13:0]}                      | in the low half-word       |
* |--------|-------|-----------------------------------------------------------
↪-----------------------------|-----------------------------|
* |        |       |                                                                 ␣
↪                              | Store the data with label  |
* |   1    |   1   | RTC_SLOW_MEM[addr + offset_]{31:16} = {label_[1:0], reg_
↪val[13:0]}                      | in the high half-word      |
* |--------|-------|-----------------------------------------------------------
↪-----------------------------|-----------------------------|
* |        |       |                                                                 ␣
↪                              | Store the data without     |
* |   3    |   0   | RTC_SLOW_MEM[addr + offset_]{15:0} = reg_val[15:0]       ␣
↪                              | label in the low half-word |
* |--------|-------|-----------------------------------------------------------
↪-----------------------------|-----------------------------|
* |        |       |                                                                 ␣
↪                              | Store the data without     |
* |   3    |   1   | RTC_SLOW_MEM[addr + offset_]{31:16} = reg_val[15:0]      ␣
↪                              | label in the high half-word|
* |--------|-------|-----------------------------------------------------------
↪-----------------------------|-----------------------------|
*
```

SUB_OPCODE_ST = manual_en:1, offset_set:0, wr_auto:0

I_ST (reg_val, reg_addr, offset_)

Store value from register reg_val into RTC memory.

I_ST() instruction provides backward compatibility for code written for esp32 to be run on esp32s2. This instruction is equivalent to calling I_ST_MANUAL() instruction with label = 0, upper = 0 and wr_way = 3.

I_STL (reg_val, reg_addr, offset_)

Store value from register reg_val to lower 16 bits of the RTC memory address.

This instruction is equivalent to calling I_ST_MANUAL() instruction with label = 0, upper = 0 and wr_way = 3.

I_STH (reg_val, reg_addr, offset_)

Store value from register reg_val to upper 16 bits of the RTC memory address.

This instruction is equivalent to calling I_ST_MANUAL() instruction with label = 0, upper = 1 and wr_way = 3.

I_ST32 (reg_val, reg_addr, offset_, label_)

Store value from register reg_val to full 32 bit word of the RTC memory address.

This instruction is equivalent to calling I_ST_MANUAL() instruction with wr_way = 0.

I_STL_LABEL (reg_val, reg_addr, offset_, label_)

Store value from register reg_val with label to lower 16 bits of RTC memory address.

This instruction is equivalent to calling I_ST_MANUAL() instruction with label = label_, upper = 0 and wr_way = 1.

I_STH_LABEL (reg_val, reg_addr, offset_, label_)

Store value from register reg_val with label to upper 16 bits of RTC memory address.

This instruction is equivalent to calling I_ST_MANUAL() instruction with label = label_, upper = 1 and wr_way = 1.

I_ST_AUTO (reg_val, reg_addr, label_, wr_way_)

Store lower half-word, upper half-word or full-word data from register reg_val into RTC memory address with auto-increment of the offset value.

This instruction can be used to write data to continuous addresses in the RTC_SLOW_MEM. The initial address must be set using the SUB_OPCODE_ST_OFFSET instruction before the auto store instruction is called. The data written to the RTC memory address could be written to the full 32 bit word or to the lower half-word or the upper half-word. The storage method is dictated by the wr_way field and the number of times the SUB_OPCODE_ST_AUTO instruction is called. write_cnt indicates the later. The following table summarizes the storage method:

```
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* | wr_way | write_cnt |                                            data          ␣
↪                                 |          operation         |
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* |        |           |                                                          ␣
↪                                 | Write full-word, including |
* |   0    |     X     | RTC_SLOW_MEM[addr + offset_]{31:0} = {insn_PC[10:0],␣
↪3'b0, label_[1:0], reg_val[15:0]} | the PC and the data        |
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* |        |           |                                                          ␣
↪                                 | Store the data with label  |
* |   1    |    odd    | RTC_SLOW_MEM[addr + offset_]{15:0} = {label_[1:0],␣
↪reg_val[13:0]}                     | in the low half-word       |
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* |        |           |                                                          ␣
↪                                 | Store the data with label  |
* |   1    |    even   | RTC_SLOW_MEM[addr + offset_]{31:16} = {label_[1:0],␣
↪reg_val[13:0]}                     | in the high half-word      |
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* |        |           |                                                          ␣
↪                                 | Store the data without     |
* |   3    |    odd    | RTC_SLOW_MEM[addr + offset_]{15:0} = reg_val[15:0]   ␣
↪                                 | label in the low half-word |
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
* |        |           |                                                          ␣
↪                                 | Store the data without     |
* |   3    |    even   | RTC_SLOW_MEM[addr + offset_]{31:16} = reg_val[15:0]  ␣
↪                                 | label in the high half-word|
* |--------|-----------|-------------------------------------------------------
↪--------------------------------|----------------------------|
*
```

The initial address offset is incremented after each store operation as follows:

- When a full-word is written, the offset is automatically incremented by 1 after each SUB_OPCODE_ST_AUTO operation.
- When a half-word is written (lower half-word first), the offset is automatically incremented by 1 after two SUB_OPCODE_ST_AUTO operations.
  SUB_OPCODE_ST_AUTO = manual_en:0, offset_set:0, wr_auto:1

I_STO (offset_)

Set the initial address offset for auto-store operation

This instruction sets the initial address of the RTC_SLOW_MEM to be used by the auto-store operation. The offset is incremented automatically. Refer I_ST_AUTO() for detailed explaination.

SUB_OPCODE_ST_OFFSET = manual_en:0, offset_set:1, wr_auto:1

I_STI (reg_val, reg_addr)

Store value from register reg_val to 32 bit word of the RTC memory address.

This instruction is equivalent to calling I_ST_AUTO() instruction with label = 0 and wr_way = 3. The data in reg_val will be either written to the lower half-word or the upper half-word of the RTC memory address depending on the count of the number of times the I_STI() instruction is called. The initial offset is automatically incremented with I_STI() is called twice. Refer I_ST_AUTO() for detailed explaination.

I_STI_LABEL (reg_val, reg_addr, label_)

Store value from register reg_val with label to 32 bit word of the RTC memory address.

This instruction is equivalent to calling I_ST_AUTO() instruction with label = label_ and wr_way = 1. The data in reg_val will be either written to the lower half-word or the upper half-word of the RTC memory address depending on the count of the number of times the I_STI_LABEL() instruction is called. The initial offset is automatically incremented with I_STI_LABEL() is called twice. Refer I_ST_AUTO() for detailed explaination.

I_STI32 (reg_val, reg_addr, label_)

Store value from register reg_val to full 32 bit word of the RTC memory address.

This instruction is equivalent to calling I_ST_AUTO() instruction with label = label_ and wr_way = 0. The data in reg_val will be written to the RTC memory address along with the label and the PC. The initial offset is automatically incremented each time the I_STI32() instruction is called. Refer I_ST_AUTO() for detailed explaination.

I_LD_MANUAL (reg_dest, reg_addr, offset_, rd_upper_)

Load lower half-word, upper half-word or full-word data from RTC memory address into the register reg_dest.

This instruction reads the lower half-word or upper half-word of the RTC memory address depending on the value of rd_upper_. The following table summarizes the loading method:

```
* |----------|-------------------------------------------------------|----------
↪---------------|
* | rd_upper |                          data                         |       ␣
↪operation        |
* |----------|-------------------------------------------------------|----------
↪---------------|
* |          |                                                       | Read␣
↪lower half-word of |
* |    0     | reg_dest{15:0} = RTC_SLOW_MEM[addr + offset_]{31:16}  | the␣
↪memory               |
* |----------|-------------------------------------------------------|----------
↪---------------|
* |          |                                                       | Read␣
↪upper half-word of |
* |    1     | reg_dest{15:0} = RTC_SLOW_MEM[addr + offset_]{15:0}   | the␣
↪memory               |
* |----------|-------------------------------------------------------|----------
↪---------------|
*
```

I_LD (reg_dest, reg_addr, offset_)

Load lower 16 bits value from RTC memory into reg_dest register.

Loads 16 LSBs (rd_upper = 1) from RTC memory word given by the sum of value in reg_addr and value of offset_. I_LD() instruction provides backward compatibility for code written for esp32 to be run on esp32s2.

I_LDL (reg_dest, reg_addr, offset_)

Load lower 16 bits value from RTC memory into reg_dest register.

I_LDL() instruction and I_LD() instruction can be used interchangably.

I_LDH (reg_dest, reg_addr, offset_)

Load upper 16 bits value from RTC memory into reg_dest register.

Loads 16 MSBs (rd_upper = 0) from RTC memory word given by the sum of value in reg_addr and value of offset_.

I_BL (pc_offset, imm_value)

Branch relative if R0 register less than the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_BG (pc_offset, imm_value)

Branch relative if R0 register greater than the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_BE (pc_offset, imm_value)

Branch relative if R0 register is equal to the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_BXR (reg_pc)

Unconditional branch to absolute PC, address in register.

reg_pc is the register which contains address to jump to. Address is expressed in 32-bit words.

I_BXI (imm_pc)

Unconditional branch to absolute PC, immediate address.

Address imm_pc is expressed in 32-bit words.

I_BXZR (reg_pc)

Branch to absolute PC if ALU result is zero, address in register.

reg_pc is the register which contains address to jump to. Address is expressed in 32-bit words.

I_BXZI (imm_pc)

Branch to absolute PC if ALU result is zero, immediate address.

Address imm_pc is expressed in 32-bit words.

I_BXFR (reg_pc)

Branch to absolute PC if ALU overflow, address in register

reg_pc is the register which contains address to jump to. Address is expressed in 32-bit words.

I_BXFI (imm_pc)

Branch to absolute PC if ALU overflow, immediate address

Address imm_pc is expressed in 32-bit words.

I_BSLE (pc_offset, imm_value)

Branch relative if stage_cnt is less than or equal to the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_BSGE (pc_offset, imm_value)

Branch relative if stage_cnt register is greater than or equal to the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_BSL (pc_offset, imm_value)

Branch relative if stage_cnt register is less than the immediate value.

pc_offset is expressed in words, and can be from -127 to 127 imm_value is a 16-bit value to compare R0 against

I_ADDR (reg_dest, reg_src1, reg_src2)

Addition: dest = src1 + src2

I_SUBR (reg_dest, reg_src1, reg_src2)

Subtraction: dest = src1 - src2

I_ANDR (reg_dest, reg_src1, reg_src2)

Logical AND: dest = src1 & src2

I_ORR (reg_dest, reg_src1, reg_src2)

Logical OR: dest = src1 | src2

I_MOVR (reg_dest, reg_src)

Copy: dest = src

I_LSHR (reg_dest, reg_src, reg_shift)

Logical shift left: dest = src << shift

I_RSHR (reg_dest, reg_src, reg_shift)

Logical shift right: dest = src >> shift

I_ADDI (reg_dest, reg_src, imm_)

Add register and an immediate value: dest = src1 + imm

I_SUBI (reg_dest, reg_src, imm_)

Subtract register and an immediate value: dest = src - imm

I_ANDI (reg_dest, reg_src, imm_)

Logical AND register and an immediate value: dest = src & imm

I_ORI (reg_dest, reg_src, imm_)

Logical OR register and an immediate value: dest = src | imm

I_MOVI (reg_dest, imm_)

Copy an immediate value into register: dest = imm

I_LSHI (reg_dest, reg_src, imm_)

Logical shift left register value by an immediate: dest = src << imm

I_RSHI (reg_dest, reg_src, imm_)

Logical shift right register value by an immediate: dest = val >> imm

I_STAGE_INC (reg_dest, reg_src, imm_)

Increment stage_cnt register by an immediate: stage_cnt = stage_cnt + imm

I_STAGE_DEC (reg_dest, reg_src, imm_)

Decrement stage_cnt register by an immediate: stage_cnt = stage_cnt - imm

I_STAGE_RST (reg_dest, reg_src, imm_)

Reset stage_cnt register by an immediate: stage_cnt = 0

M_LABEL (label_num)

Define a label with number label_num.

This is a macro which doesn't generate a real instruction. The token generated by this macro is removed by ulp_process_macros_and_load function. Label defined using this macro can be used in branch macros defined below.

M_BRANCH (label_num)

Token macro used by M_B and M_BX macros. Not to be used directly.

M_BL (label_num, imm_value)

Macro: branch to label label_num if R0 is less than immediate value.

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

M_BG (label_num, imm_value)

Macro: branch to label label_num if R0 is greater than immediate value

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

M_BE (label_num, imm_value)

Macro: branch to label label_num if R0 equal to the immediate value

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

M_BX (label_num)

Macro: unconditional branch to label

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

M_BXZ (label_num)

Macro: branch to label if ALU result is zero

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

M_BXF (label_num)

Macro: branch to label if ALU overflow

This macro generates two ulp_insn_t values separated by a comma, and should be used when defining contents of ulp_insn_t arrays. First value is not a real instruction; it is a token which is removed by ulp_process_macros_and_load function.

Compiling the ULP Code

To compile the ULP FSM code as part of the component, the following steps must be taken:

1. The ULP FSM code, written in assembly, must be added to one or more files with `.S` extension. These files must be placed into a separate directory inside the component directory, for instance, `ulp/`.

Note: When registering the component (via `idf_component_register`), this directory should not be added to the `SRC_DIRS` argument. The logic behind this is that the ESP-IDF build system will compile files found in `SRC_DIRS` based on their extensions. For `.S` files, `xtensa-esp32s3-elf-as` assembler is used. This is not

desirable for ULP FSM assembly files, so the easiest way to achieve the distinction is by placing ULP FSM assembly files into a separate directory. The ULP FSM assembly source files should also **not** be added to `SRCS` for the same reason. See the steps below for how to properly add ULP FSM assembly source files.

2. Call `ulp_embed_binary` from the component CMakeLists.txt after registration. For example:

```
...
idf_component_register()

set(ulp_app_name ulp_${COMPONENT_NAME})
set(ulp_s_sources ulp/ulp_assembly_source_file.S)
set(ulp_exp_dep_srcs "ulp_c_source_file.c")

ulp_embed_binary(${ulp_app_name} "${ulp_s_sources}" "${ulp_exp_dep_srcs}")
```

The first argument to `ulp_embed_binary` specifies the ULP FSM binary name. The name specified here will also be used by other generated artifacts such as the ELF file, map file, header file and linker export file. The second argument specifies the ULP FSM assembly source files. Finally, the third argument specifies the list of component source files which include the header file to be generated. This list is needed to build the dependencies correctly and ensure that the generated header file will be created before any of these files are compiled. See the section below for the concept of generated header files for ULP applications.

3. Build the application as usual (e.g., `idf.py app`).
   Inside, the build system will take the following steps to build ULP FSM program:
   1. **Run each assembly file (foo.S) through the C preprocessor.** This step generates the preprocessed assembly files (foo.ulp.S) in the component build directory. This step also generates dependency files (foo.ulp.d).
   2. **Run preprocessed assembly sources through the assembler.** This produces object (foo.ulp.o) and listing (foo.ulp.lst) files. Listing files are generated for debugging purposes and are not used at later stages of the build process.
   3. **Run the linker script template through the C preprocessor.** The template is located in `components/ulp/ld` directory.
   4. **Link the object files into an output ELF file** (`ulp_app_name.elf`). The Map file (`ulp_app_name.map`) generated at this stage may be useful for debugging purposes.
   5. **Dump the contents of the ELF file into a binary** (`ulp_app_name.bin`) which can then be embedded into the application.
   6. **Generate a list of global symbols** (`ulp_app_name.sym`) in the ELF file using `esp32ulp-elf-nm`.
   7. **Create an LD export script and a header file** (`ulp_app_name.ld` and `ulp_app_name.h`) containing the symbols from `ulp_app_name.sym`. This is done using the `esp32ulp_mapgen.py` utility.
   8. **Add the generated binary to the list of binary files** to be embedded into the application.

Accessing the ULP FSM Program Variables

Global symbols defined in the ULP FSM program may be used inside the main program.

For example, the ULP FSM program may define a variable `measurement_count` which will define the number of ADC measurements the program needs to make before waking up the chip from Deep-sleep:

```
                        .global measurement_count
measurement_count:      .long 0

                        // later, use measurement_count
                        move r3, measurement_count
                        ld r3, r3, 0
```

The main program needs to initialize this variable before the ULP program is started. The build system makes this possible by generating the `${ULP_APP_NAME}.h` and `${ULP_APP_NAME}.ld` files which define the global

symbols present in the ULP program. Each global symbol defined in the ULP program is included in these files and are prefixed with `ulp_`.

The header file contains the declaration of the symbol:

```
extern uint32_t ulp_measurement_count;
```

Note that all symbols (variables, arrays, functions) are declared as `uint32_t`. For functions and arrays, take the address of the symbol and cast it to the appropriate type.

The generated linker script file defines the locations of symbols in RTC_SLOW_MEM:

```
PROVIDE ( ulp_measurement_count = 0x50000060 );
```

To access the ULP program variables from the main program, the generated header file should be included using an `include` statement. This will allow the ULP program variables to be accessed as regular variables:

```
#include "ulp_app_name.h"

// later
void init_ulp_vars() {
    ulp_measurement_count = 64;
}
```

Starting the ULP FSM Program

To run a ULP FSM program, the main application needs to load the ULP program into RTC memory using the *`ulp_load_binary()`* function, and then start it using the *`ulp_run()`* function.

Note that the `Enable Ultra Low Power (ULP) Coprocessor` option must be enabled in menuconfig to work with ULP. To select the type of ULP to be used, the `ULP Co-processor type` option must be set. To reserve memory for the ULP, the `RTC slow memory reserved for coprocessor` option must be set to a value big enough to store ULP code and data. If the application components contain multiple ULP programs, then the size of the RTC memory must be sufficient to hold the largest one.

Each ULP program is embedded into the ESP-IDF application as a binary blob. The application can reference this blob and load it in the following way (suppose ULP_APP_NAME was defined to `ulp_app_name`):

```
extern const uint8_t bin_start[] asm("_binary_ulp_app_name_bin_start");
extern const uint8_t bin_end[]   asm("_binary_ulp_app_name_bin_end");

void start_ulp_program() {
    ESP_ERROR_CHECK( ulp_load_binary(
        0 // load address, set to 0 when using default linker scripts
        bin_start,
        (bin_end - bin_start) / sizeof(uint32_t)) );
}
```

Once the program is loaded into RTC memory, the application can start it by passing the address of the entry point to the `ulp_run` function:

```
ESP_ERROR_CHECK( ulp_run(&ulp_entry - RTC_SLOW_MEM) );
```

Declaration of the entry point symbol comes from the generated header file mentioned above, `${ULP_APP_NAME}.h`. In the assembly source of the ULP FSM application, this symbol must be marked as `.global`:

```
        .global entry
entry:
        // code starts here
```

ESP32-S3 ULP Program Flow

ESP32-S3 ULP coprocessor is started by a timer. The timer is started once *ulp_run()* is called. The timer counts a number of RTC_SLOW_CLK ticks (by default, produced by an internal 90 kHz RC oscillator). The number of ticks is set using RTC_CNTL_ULP_CP_TIMER_1_REG register.

The application can set ULP timer period values by *ulp_set_wakeup_period()* function.

Once the timer counts the number of ticks set in the selected RTC_CNTL_ULP_CP_TIMER_1_REG register, ULP coprocessor powers up and starts running the program from the entry point set in the call to *ulp_run()*.

The program runs until it encounters a halt instruction or an illegal instruction. Once the program halts, ULP coprocessor powers down, and the timer is started again.

To disable the timer (effectively preventing the ULP program from running again), clear the RTC_CNTL_ULP_CP_SLP_TIMER_EN bit in the RTC_CNTL_ULP_CP_TIMER_REG register. This can be done both from ULP code and from the main program.

Application Examples

- ULP FSM Coprocessor counts pulses on an IO while main CPU is in Deep-sleep: system/ulp/ulp_fsm/ulp.
- ULP FSM Coprocessor polls ADC in while main CPU is in Deep-sleep: system/ulp/ulp_fsm/ulp_adc.

API Reference

Header File

- components/ulp/ulp_fsm/include/ulp_fsm_common.h
- This header file can be included with:

```
#include "ulp_fsm_common.h"
```

- This header file is a part of the API provided by the ulp component. To declare that your component depends on ulp, add the following to your CMakeLists.txt:

```
REQUIRES ulp
```

or

```
PRIV_REQUIRES ulp
```

Functions

esp_err_t **ulp_isr_register**(*intr_handler_t* fn, void *arg)

Register ULP wakeup signal ISR.

Note: The ISR routine will only be active if the main CPU is not in deepsleep

---

Parameters

- **fn** -- ISR callback function
- **arg** -- ISR callback function arguments

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if callback function is NULL
- ESP_ERR_NO_MEM if heap memory cannot be allocated for the interrupt

esp_err_t **ulp_isr_deregister**(*intr_handler_t* fn, void *arg)

Deregister ULP wakeup signal ISR.

Parameters

- **fn** -- ISR callback function
- **arg** -- ISR callback function arguments

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if callback function is NULL
- ESP_ERR_INVALID_STATE if a handler matching both callback function and its arguments isn't registered

esp_err_t **ulp_process_macros_and_load**(uint32_t load_addr, const *ulp_insn_t* *program, size_t *psize)

Resolve all macro references in a program and load it into RTC memory.

Parameters

- **load_addr** -- address where the program should be loaded, expressed in 32-bit words
- **program** -- ulp_insn_t array with the program
- **psize** -- size of the program, expressed in 32-bit words

Returns

- ESP_OK on success
- ESP_ERR_NO_MEM if auxiliary temporary structure can not be allocated
- one of ESP_ERR_ULP_xxx if program is not valid or can not be loaded

esp_err_t **ulp_load_binary**(uint32_t load_addr, const uint8_t *program_binary, size_t program_size)

Load ULP program binary into RTC memory.

ULP program binary should have the following format (all values little-endian):

a. MAGIC, (value 0x00706c75, 4 bytes)
b. TEXT_OFFSET, offset of .text section from binary start (2 bytes)
c. TEXT_SIZE, size of .text section (2 bytes)
d. DATA_SIZE, size of .data section (2 bytes)
e. BSS_SIZE, size of .bss section (2 bytes)
f. (TEXT_OFFSET - 12) bytes of arbitrary data (will not be loaded into RTC memory)
g. .text section
h. .data section

Linker script in components/ulp/ld/esp32.ulp.ld produces ELF files which correspond to this format. This linker script produces binaries with load_addr == 0.

Parameters

- **load_addr** -- address where the program should be loaded, expressed in 32-bit words
- **program_binary** -- pointer to program binary
- **program_size** -- size of the program binary

Returns

- ESP_OK on success
- ESP_ERR_INVALID_ARG if load_addr is out of range
- ESP_ERR_INVALID_SIZE if program_size doesn't match (TEXT_OFFSET + TEXT_SIZE + DATA_SIZE)
- ESP_ERR_NOT_SUPPORTED if the magic number is incorrect

esp_err_t **ulp_run**(uint32_t entry_point)

Run the program loaded into RTC memory.

Parameters **entry_point** -- entry point, expressed in 32-bit words
Returns ESP_OK on success

Macros

ESP_ERR_ULP_BASE

Offset for ULP-related error codes

ESP_ERR_ULP_SIZE_TOO_BIG

Program doesn't fit into RTC memory reserved for the ULP

ESP_ERR_ULP_INVALID_LOAD_ADDR

Load address is outside of RTC memory reserved for the ULP

ESP_ERR_ULP_DUPLICATE_LABEL

More than one label with the same number was defined

ESP_ERR_ULP_UNDEFINED_LABEL

Branch instructions references an undefined label

ESP_ERR_ULP_BRANCH_OUT_OF_RANGE

Branch target is out of range of B instruction (try replacing with BX)

Type Definitions

typedef union *ulp_insn* **ulp_insn_t**

Header File

- components/ulp/ulp_common/include/ulp_common.h
- This header file can be included with:

```
#include "ulp_common.h"
```

- This header file is a part of the API provided by the `ulp` component. To declare that your component depends on `ulp`, add the following to your CMakeLists.txt:

```
REQUIRES ulp
```

or

```
PRIV_REQUIRES ulp
```

Functions

esp_err_t **ulp_set_wakeup_period**(size_t period_index, uint32_t period_us)

Set one of ULP wakeup period values.

ULP coprocessor starts running the program when the wakeup timer counts up to a given value (called period). There are 5 period values which can be programmed into SENS_ULP_CP_SLEEP_CYCx_REG registers, x = 0..4 for ESP32, and one period value which can be programmed into RTC_CNTL_ULP_CP_TIMER_1_REG register for ESP32-S2/S3. By default, for ESP32, wakeup timer will use the period set into SENS_ULP_CP_SLEEP_CYC0_REG, i.e. period number 0. ULP program code can use SLEEP instruction to select which of the SENS_ULP_CP_SLEEP_CYCx_REG should be used for subsequent wakeups.

However, please note that SLEEP instruction issued (from ULP program) while the system is in deep sleep mode does not have effect, and sleep cycle count 0 is used.

For ESP32-S2/S3 the SLEEP instruction not exist. Instead a WAKE instruction will be used.

Note: The ULP FSM requires two clock cycles to wakeup before being able to run the program. Then additional 16 cycles are reserved after wakeup waiting until the 8M clock is stable. The FSM also requires two more clock cycles to go to sleep after the program execution is halted. The minimum wakeup period that may

be set up for the ULP is equal to the total number of cycles spent on the above internal tasks. For a default configuration of the ULP running at 150kHz it makes about 133us.

Parameters
- **period_index** -- wakeup period setting number (0 - 4)
- **period_us** -- wakeup period, us

Returns
- ESP_OK on success
- ESP_ERR_INVALID_ARG if period_index is out of range

void **ulp_timer_stop**(void)

Stop the ULP timer.

Note: This will stop the ULP from waking up if halted, but will not abort any program currently executing on the ULP.

void **ulp_timer_resume**(void)

Resume the ULP timer.

Note: This will resume an already configured timer, but does no other configuration

2.10.34 ULP RISC-V Coprocessor Programming

The ULP RISC-V coprocessor is a variant of the ULP present in ESP32-S3. Similar to ULP FSM, the ULP RISC-V coprocessor can perform tasks such as sensor readings while the main CPU stays in low power modes. The main difference between ULP FSM and ULP RISC-V is that the latter can be programmed in C using standard GNU tools. The ULP RISC-V coprocessor can access the RTC_SLOW_MEM memory region, and registers in `RTC_CNTL`, `RTC_IO`, and `SARADC` peripherals. The RISC-V processor is a 32-bit fixed point machine. Its instruction set is based on RV32IMC which includes hardware multiplication and division, and compressed code.

Installing the ULP RISC-V Toolchain

The ULP RISC-V coprocessor code is written in C (assembly is also possible) and compiled using the RISC-V toolchain based on GCC.

If you have already set up ESP-IDF with CMake build system according to the *Getting Started Guide*, then the toolchain should already be installed.

Note: In earlier versions of ESP-IDF, RISC-V toolchain had a different prefix: `riscv-none-embed-gcc`.

Compiling the ULP RISC-V Code

To compile the ULP RISC-V code as part of the component, the following steps must be taken:

1. The ULP RISC-V code, written in C or assembly (must use the `.S` extension), must be placed in a separate directory inside the component directory, for instance, `ulp/`.